C és modern C++ rendszerszemléletben DevOps szakembereknek

Gellai István

2026

Tartalomjegyzék

Impresszum **9**

I. rész: C alapok és rendszerprogramozás **10**

1 A C és C++ mentális modellje és a labor környezete **11**
1.1 Absztrakt gép, forrásprogram és natív folyamat 11
1.2 Szabványok, dialektusok és hordozhatósági szerződés 18
1.3 Reprodukálható fordítási és diagnosztikai labor 25
1.4 A szükséges C szintaxis tömör áttekintése 36
1.5 Első megfigyelési ciklus a forrástól a futásig 42

2 A fordítás és a preprocessor C-ben **49**
2.1 Fordítási fázisok és fordítási egységek 49
2.2 Headerek, deklarációk és include-higiénia 56
2.3 Makrók, feltételes fordítás és veszélyes helyettesítés 63
2.4 Preprocessor-labor és feature detection 72
2.5 Warningok, optimalizálási profilok és link-driver 78

3 Linkelés, ELF, programindulás és leállás **85**
3.1 Szimbólumok, linkage és relocation 85
3.2 Statikus és megosztott könyvtárak 92
3.3 ELF-betöltés, PLT/GOT és dinamikus szimbólumok 98
3.4 main, startup, argumentumok és környezet 105
3.5 Leállás, cleanup és nem lokális vezérlés 111

4 C adattípusok, konverziók és kifejezések **118**
4.1 Egész típusok és adatmodellek 118
4.2 Promóciók, signedness és túlcsordulás 124
4.3 Lebegőpontos reprezentáció és numerikus hibák 131
4.4 Deklarációk, typedef és típusminősítők 138
4.5 Kifejezések kiértékelése, sequencing és bitműveletek 145

5 C objektumok, tárolási idő és memóriamodell **152**
5.1 Scope, linkage, storage duration és lifetime 152
5.2 C objektumok a folyamat virtuális címterében 159
5.3 Inicializálás, indeterminate érték és reprezentáció 166
5.4 Effective type, aliasing és objektumreprezentáció 174
5.5 Élettartam- és memóriahibák vizsgálati laborja 182

6 Pointerek és összetett deklarációk C-ben **191**
6.1 Címképzés, dereferencia és null pointer 191
6.2 Pointeraritmetika, objektumhatár és provenance 199
6.3 const pointerek, több szintű indirection és void* 206
6.4 Deklarátorok: tömb-, pointer- és függvénykapcsolatok 214
6.5 Függvénypointerek, callbackek és context pointer 223

7 Tömbök, sztringek és nem tulajdonló adatszeletek C-ben 232
7.1 Tömbobjektum, decay és függvényparaméter . . . 232
7.2 Többdimenziós tömbök, VLA és layout . . . 239
7.3 Nullterminált sztring és a hossz szerződése . . . 246
7.4 Memóriamásolás, összehasonlítás és byte view . . . 252
7.5 Szövegkódolás, locale és UTF-8 C-ben . . . 259

8 Dinamikus memória és explicit tulajdonlás C-ben 267
8.1 malloc, calloc, free és allokációs szerződés . . . 267
8.2 realloc és dinamikusan növekvő buffer . . . 275
8.3 Alignment és speciális allokációk . . . 283
8.4 Tulajdonlási szerződés és cleanup ágak . . . 291
8.5 Allokációs hiba-injektálás és leak diagnosztika . . . 300

9 struct, union, alignment és bináris layout 310
9.1 Struktúrák, tagok és értékmásolás . . . 310
9.2 Padding, alignment és offsetof . . . 318
9.3 Union, tagged union, enum és bit-field . . . 326
9.4 Byte order és biztonságos szerializálás . . . 334
9.5 Layout- és protokolllabor két architektúrán . . . 344

10 Függvények, modulok és C API-tervezés 354
10.1 Függvényhívás, prototípus és paraméterátadás . . . 354
10.2 Variadikus függvények és típusbiztonság . . . 361
10.3 static inline, `_Generic` és headeres eszköztár . . . 367
10.4 Opaque handle és stabil C könyvtári interfész . . . 374
10.5 C API-szerződések ellenőrzése . . . 382

11 A C standard könyvtár és hibakezelés 390
11.1 stdio, FILE és pufferelés . . . 390
11.2 Formázás és megbízható szövegfeldolgozás . . . 396
11.3 Hibakód, errno és hibaadatok élettartama . . . 404
11.4 Idő, véletlen és környezeti állapot . . . 410
11.5 Numerikus és ritkább C könyvtári eszközök térképe . . . 416

12 C mint Linux/POSIX rendszerprogramozási nyelv 424
12.1 File descriptor és robusztus I/O ciklus . . . 424
12.2 Fájlrendszer-műveletek, mmap és tartósság . . . 431
12.3 fork, exec, pipe és gyermekfolyamat . . . 437
12.4 Signal, megszakítás és graceful shutdown . . . 443
12.5 Socket API, framing és bufferkezelés . . . 450
12.6 poll/epoll és explicit eseményállapotgép . . . 459

13 Adatszerkezetek és parser C-ben 467
13.1 Dinamikus tömb és string builder . . . 467
13.2 Láncolt és intrusive listák . . . 476
13.3 Hash table, comparator és ownership . . . 482
13.4 Arena, pool és region lifetime . . . 489
13.5 Bounds-ellenőrzött parser és állapotgép . . . 496

14 Assembly, hívási ABI és hardverkapcsolatok 504
14.1 C forrásból assembly és gépi kód . . . 504
14.2 Hívási konvenció, stack frame és visszatérés . . . 512

14.3 Optimalizáló, as-if és UB következményei . . . 520
14.4 Cache, locality, alignment és false sharing . . . 527
14.5 volatile, compiler barrier és hardver barrier . . . 534
14.6 ABI- és optimalizálásdiagnosztikai labor . . . 541

15 Szálak és atomikus műveletek C-ben 550
15.1 Szál-életciklus, pthreads és C11 threads . . . 550
15.2 Data race, happens-before és publikalás . . . 558
15.3 Mutex, condition variable és producer–consumer . . . 566
15.4 C atomics és memory order . . . 575
15.5 Konkurens élettartam és nehéz hibák . . . 582

16 C hibakeresés, tesztelés és biztonságos kód 591
16.1 Debugger, core dump és optimalizált program . . . 591
16.2 ASan, UBSan, MSan, TSan és Valgrind . . . 597
16.3 Unit test, property test és fuzzing . . . 603
16.4 C biztonsági hibaminták és védekezés . . . 611
16.5 C minőségi kapuk és hibajavítási jegyzőkönyv . . . 617

17 C záróprojekt: stream-alapú diagnosztikai szolgáltatás 624
17.1 Projektkövetelmény, protokoll és ownership térkép . . . 624
17.2 C könyvtári mag és CLI megvalósítási labor . . . 634
17.3 Nonblocking TCP adapter és leállítás . . . 654
17.4 Záróreview: helyesség, diagnosztika és mérés . . . 681

18 Modern C, C23 és a C/C++ határ 690
18.1 C23 nyelvi újdonságok és inkompatibilis változások . . . 690
18.2 C23 könyvtár és preprocessor . . . 695
18.3 C és C++: azonos szintaxis, eltérő szemantika . . . 701
18.4 C hordozhatósági audit és freestanding kitekintés . . . 707

II. rész: Modern C++ 714

19 Átmenet C-ről modern C++-ra 715
19.1 C++ programmodell és zero-overhead elv . . . 715
19.2 Namespace, header és névfeloldás . . . 721
19.3 Reference, pointer és const C++-ban . . . 727
19.4 auto, decltype és típuslevezetés . . . 733
19.5 Overload, konverzió és explicit interfész . . . 741

20 C++ objektumok, inicializálás és élettartam 749
20.1 Az inicializálás különböző alakjai . . . 749
20.2 Osztály, konstruktor és invariáns . . . 758
20.3 Destruktor és részleges konstrukció . . . 765
20.4 Temporary, value category és dangling . . . 773
20.5 Statikus és thread-local objektumok életciklusa . . . 780
20.6 Tároló újrafelhasználása és C++ objektumszabályok . . . 791

21 RAII, erőforráskezelés és smart pointerek 801
21.1 new/delete és az allokáció-konstrukció kapcsolat . . . 801
21.2 RAII wrapper file descriptorhoz és lockhoz . . . 807
21.3 unique_ptr és egyedi tulajdonlás . . . 816
21.4 shared_ptr, control block és weak_ptr . . . 825

21.5 Owner, view és erőforrásszerződés C++ API-ban . . . 832

22 Értékszemantika, másolás és mozgatás 839
22.1 Rule of zero, three és five . . . 839
22.2 Mély másolás, értéktípus és exception guarantee . . . 848
22.3 Move constructor, moved-from állapot és std::move . . . 855
22.4 Copy elision, RVO/NRVO és értékvisszaadás . . . 862
22.5 noexcept, swap és container-reallocation . . . 872

23 Osztálytervezés, láthatóság és operátorok 881
23.1 public/private/protected és invariánsvédelem . . . 881
23.2 Értéktípus, kompozíció és immutable interfész . . . 887
23.3 Operátortúlterhelés és konverziós operátor . . . 896
23.4 Egyenlőség, spaceship és rendezési kategóriák . . . 906
23.5 Osztály- és operátortervezési labor . . . 914

24 Öröklés, virtuális függvények és OOP 923
24.1 Öröklés, base subobject és hozzáférés . . . 923
24.2 Virtual, override, final és dispatch . . . 931
24.3 Virtuális destruktor, slicing és ownership . . . 939
24.4 RTTI, dynamic_cast és típusazonosság . . . 947
24.5 Többszörös öröklés és virtuális bázis . . . 956

25 Kivételek és C++ hibakezelési stratégiák 964
25.1 throw, catch és stack unwinding . . . 964
25.2 No-throw, strong és basic exception guarantee . . . 972
25.3 noexcept határok és hibapolicy . . . 980
25.4 error_code, system_error és expected . . . 986
25.5 Hibakezelési és exception safety labor . . . 993

26 Template-ek és generikus programozás alapjai 1001
26.1 Függvénysablon, instantiation és deduction . . . 1001
26.2 Osztálysablon, nem típusparaméter és CTAD . . . 1006
26.3 Forwarding reference és perfect forwarding . . . 1013
26.4 Specialization, overload és explicit instantiation . . . 1022
26.5 Dependent name, kétfázisú lookup és ADL . . . 1030
26.6 Generikus interfész és sablondiagnosztikai labor . . . 1039

27 Concepts, type traits és compile-time generikusság 1050
27.1 Type traits, tag dispatch és if constexpr . . . 1050
27.2 Concepts, requires és constrained overload . . . 1059
27.3 SFINAE és örökölt template minták . . . 1067
27.4 Variadic template, pack és fold expression . . . 1072
27.5 Customization point, CRTP és policy design . . . 1081

28 STL tárolók és érvényességi szerződések 1090
28.1 std::array, std::vector és folytonos tárolás . . . 1090
28.2 std::string, std::string_view és szövegownership . . . 1096
28.3 deque, list és forward_list . . . 1103
28.4 map, set és rendezett asszociatív tárolók . . . 1111
28.5 unordered tárolók és container adapterek . . . 1120
28.6 Tárolóválasztási és invalidációs labor . . . 1127

29 Iteratorok és standard algoritmusok **1136**
29.1 Iterator-kategóriák és fél-nyílt tartomány . . . 1136
29.2 Transform, copy, remove és módosító algoritmusok . . . 1143
29.3 Rendezés, keresés és comparator szerződés . . . 1152
29.4 Numerikus algoritmusok és kiértékelési sorrend . . . 1158
29.5 Execution policy és párhuzamos algoritmusok . . . 1167

30 Lambdák, callable-ek és ranges **1176**
30.1 Lambda, capture és closure lifetime . . . 1176
30.2 invoke, function wrapper és type erasure . . . 1183
30.3 Ranges algoritmusok, projection és sentinel . . . 1189
30.4 Lazy view pipeline és mellékhatások . . . 1196
30.5 Borrowed range, dangling és iterator lifetime . . . 1204
30.6 Ranges- és callback-élettartam labor . . . 1214

31 Standard utility típusok és memória-view-k **1223**
31.1 optional, variant és visit . . . 1223
31.2 tuple, pair, any és type erasure . . . 1230
31.3 span, byte és bounds-szerződések . . . 1235
31.4 Bitműveletek és objektumreprezentáció C++-ban . . . 1243
31.5 Multidimenziós view és numeric utility térkép . . . 1250

32 C++ szöveg, fájlok, idő és könyvtári I/O **1258**
32.1 iostream, stream state és pufferelés . . . 1258
32.2 charconv, format és C++23 print . . . 1264
32.3 filesystem, path és OS-határ . . . 1273
32.4 chrono, duration és időpont . . . 1281
32.5 Szövegkódolás, locale és random . . . 1289

33 Compile-time C++, ODR és modulok **1296**
33.1 constexpr, consteval és constant evaluation . . . 1296
33.2 Compile-time adatok és típusos konfiguráció . . . 1302
33.3 ODR, inline és több fordítási egység . . . 1310
33.4 C++20 modules és header unit . . . 1317
33.5 Feature-test, standard library és build capability . . . 1323

34 Modern C++ konkurencia és szinkronizáció **1331**
34.1 thread, jthread és kooperatív cancellation . . . 1331
34.2 RAII lockok, condition variable és wait predicate . . . 1336
34.3 C++ atomics, atomic_ref és publication . . . 1343
34.4 promise, future, async és task eredmény . . . 1354
34.5 semaphore, latch, barrier és szinkron stream . . . 1362
34.6 shared ownership, reclamation és konkurens review . . . 1371

35 Coroutine-ok és aszinkron erőforrás-életciklus **1382**
35.1 Coroutine mint felfüggeszthető állapotgép . . . 1382
35.2 promise_type, awaiter és coroutine_handle . . . 1388
35.3 Dangling capture, resumption és cancellation . . . 1396
35.4 Generator és streaming feldolgozás . . . 1404
35.5 Aszinkron I/O adapter és coroutine-labor . . . 1412

36 C++ ABI, C interfész és FFI **1428**
36.1 Névtorzítás, layout és C++ runtime ABI . . . 1428

36.2 extern C és két nyelvű header . . . 1434
36.3 Stabil shared library határ és verziózás . . . 1444
36.4 Python és más nyelvek FFI-kapcsolata . . . 1452
36.5 ABI- és FFI-labor, kompatibilitási teszt . . . 1459

37 Teljesítmény, allocatorok és memóriaelrendezés C++-ban 1474
37.1 Allocator model és allocator_traits . . . 1474
37.2 pmr, memory_resource és region lifetime . . . 1481
37.3 Object layout, EBO és adatközpontú tervezés . . . 1488
37.4 Profilozás, benchmark és cost model . . . 1496
37.5 Template, virtual és std::function költsége . . . 1503
37.6 Optimalizálási labor correctness kapukkal . . . 1510

38 Natív build, tesztek, CI és disztribúció 1521
38.1 Build graph, Make/Ninja és incremental build . . . 1521
38.2 Modern CMake target-alapú projekt . . . 1526
38.3 Függőségek, package discovery és linkhatárok . . . 1533
38.4 CTest, negatív tesztek és sanitizer CI . . . 1540
38.5 Cross-compilation és multiarch build . . . 1548
38.6 Release, debug symbol és reprodukálhatóság . . . 1555

39 C++ záróprojekt: a C rendszer átalakítása 1563
39.1 Portterv és azonos viselkedési szerződés . . . 1563
39.2 C++ core: parser, erőforrások és értéktípusok . . . 1577
39.3 Task, queue és szolgáltatás-életciklus . . . 1586
39.4 CMake, tesztek és mérési csomag . . . 1604
39.5 Záróreview és C/C++ összehasonlítás . . . 1614

III. rész: C++23 elmélyítés és C++26 kitekintés 1625

40 C++23 nyelvi újdonságok: tagfüggvények, konstans kiértékelés és kifejezések 1626
40.1 C++23 laborprofil, szabványstátusz és feature-test . . . 1626
40.2 Explicit object parameter: deducing this . . . 1632
40.3 static operator(), többdimenziós indexelés és lambdaváltozások . . . 1641
40.4 Konstans kiértékelés C++23-ban: if consteval és constexpr bővítések . . . 1648
40.5 Érték, élettartam és kifejezések: auto(x), implicit move és range-for . . . 1657
40.6 Lexikális és preprocessor bővítések, kiterjesztett lebegőpontos típusok . . . 1666

41 Hibakezelés, diagnosztika és kimenet C++23-ban 1673
41.1 std::expected mélyebben: monadikus hibaláncok . . . 1673
41.2 optional monadikus műveletei és hibamodellek kompozíciója . . . 1684
41.3 std::print, std::println és tartományok formázása . . . 1691
41.4 std::stacktrace és futásidejű diagnosztika . . . 1698

42 Ranges C++23-ban: konverzió, új view-k és algoritmusok 1705
42.1 ranges::to és tárolókonstrukció range-ből . . . 1705
42.2 Kombináló view-k: zip, adjacent, enumerate, cartesian_product . . . 1712
42.3 Szeletelő és átalakító view-k: chunk, slide, chunk_by, stride, join_with . . . 1721
42.4 C++23 range-algoritmusok és ranges-labor . . . 1728

43 Tárolók, memória és alacsony szintű eszközök C++23-ban 1738
43.1 flat_map és flat_set: rendezett tárolás folytonos memóriában . . . 1738
43.2 mdspan mélyebben: layoutok, accessorok és C-interop . . . 1745

43.3 Callable és forwarding eszközök: move_only_function, forward_like, bind_back . 1754
43.4 Alacsony szintű és C-interop eszközök C++23-ban 1762

44 Modulok, generátorok és migráció C++23-ra 1775
44.1 import std és import std.compat 1775
44.2 std::generator mélyebben: rekurzió, allokáció és ranges 1782
44.3 Elavult, eltávolított és viselkedést váltó elemek C++23-ban 1790
44.4 Migrációs labor: a C++ záróprojekt átállítása C++23-ra 1796

45 Rövid kitekintés: C++26 1804
45.1 C++26 nyelvi kitekintés: reflexió, contracts és kisebb újdonságok 1804
45.2 C++26 könyvtári és konkurencia-kitekintés 1811

IV. rész: Kitekintés és referencia 1820

46 Haladó irányok, új szabványok és referencia-függelék 1821
46.1 C++23 összefoglaló és migrációs táblázat 1821
46.2 C++26 és későbbi irányok ellenőrzött státusszal 1827
46.3 Freestanding, embedded, kernel és GPU irányok 1833
46.4 Standard könyvtári lefedettség és ökoszisztéma-térkép 1837
46.5 Hibaminták, döntési táblák és önellenőrző térkép 1841

Függelékek 1852
Fogalomtár (magyar–angol) 1852
Képlet- és jelölésgyűjtemény 1854
Kompetenciatérkép és tanulási útvonal 1856
Bibliográfia és forráskulcsok 1859

Impresszum

- **Szerző:** Gellai István
- **Szerkesztő:** Gellai István
- **Kiadó:** Tananyag Simple
- **Kiadás:** 1.0.0 – 2026
- **Licenc:** MIT
- **Errata:** review/reports/STATUS.md
- **Forráskód:** labs/

A teljes licencszöveg a tároló gyökerének `LICENSE` fájljában található (a könyvkönyvtárból: `../../LICENSE`).

I. rész: C alapok és rendszerprogramozás

1 A C és C++ mentális modellje és a labor környezete

1.1 Absztrakt gép, forrásprogram és natív folyamat

Egy magas szintű nyelvben gyakran elegendő úgy gondolkodni, hogy a változó értéket nevez meg, a futtatókörnyezet pedig gondoskodik annak tárolásáról és kezeléséről. C-ben ez a kép nem elég: külön kell választanunk az értéket, az azt tároló objektumot, az objektum típusát és a gépi megvalósítást. Ugyanakkor a másik véglet is félrevezető: a C-forrás nem processzorutasítások hordozható jelölésrendszere.

A helyes kiindulópont a **nyelvi absztrakt gép** (*abstract machine*): a program jelentését meghatározó szabályrendszer. Ezt kapcsoljuk össze a már ismert natív folyamatmodellel. A nyelvi állításokat itt az **ISO C17**, a konkrét rendszerpéldákat a **Linux-labor** szintjén kezeljük. A szabványos követelmény, az implementáció választása és egy futás megfigyelése különböző bizonyíték.

1.1.1 Hogyan használd ezt a könyvet

Célcsoport. A könyv olyan DevOps-, SRE- és rendszerüzemeltetési szakembereknek szól, akik a Linuxot és legalább egy szkript- vagy magas szintű nyelvet napi szinten használják, és natív (C, C++) komponenseket kell építeniük, hibát keresniük bennük, vagy felelősséggel üzemeltetniük őket. Nem kezdő programozási tankönyv: a vezérlési szerkezeteket és az alapvető algoritmusokat ismertnek tekinti.

Előismeretek. A következőket feltételezzük:

- **Shell:** fájlok és könyvtárak kezelése, átirányítás és pipe, kilépési kód (`$?`), környezeti változók, egyszerű `sh`-szkript `set -eu`-val.
- **Linux:** folyamat, fájlleíró, jogosultság, csomagkezelő; a `man` oldalak olvasása; alapvető diagnosztikai eszközök (`ps`, `strace` vagy `ltrace` legalább névről).
- **Algoritmika:** tömb, láncolt lista, hash tábla, verem és sor; az $O(n)$ és $O(n \log n)$ költség jelentése; rekurzió.
- **Programozás:** legalább egy nyelven függvény, ciklus, feltétel, rekord vagy objektum, hibakód vagy kivétel; a Git alapműveletei.

Rövid szintfelmérő. Ha az alábbiak közül bármelyikre nem tudsz biztosan válaszolni, előbb az adott előismeretet érdemes pótolni, mert a későbbi elakadás nagy eséllyel ebből fakad, nem a C-ből vagy a C++-ból.

1. Futtasd: `printf 'a\n' > t.txt; grep x t.txt; echo $?; grep x nincs.txt; echo $?`. Mit ír ki a két `echo $?`, és miért különbözik? (A „nincs találat” és a „végrehajtási hiba, például hiányzó fájl” két külön esetet ad.)
2. Mi a különbség egy folyamat és egy fájlleíró között? Mi történik egy fájlleíróval `fork` után?
3. Mi a költsége egy elem beszúrásának egy tömb elejére, illetve egy egyszeresen láncolt lista elejére, ha a lista fejmutatója már rendelkezésre áll?
4. Mit jelent, hogy egy függvény „hibakóddal” jelez hibát, és ki felel a hibakód ellenőrzéséért?

5. Hogyan futtatsz egy parancsot úgy, hogy a standard kimenete egy fájlba, a hibakimenete egy másikba kerüljön?

Javítókulcs. (1) A találat nélküli `grep` kilépési kódja `1`, a hiányzó fájlé `2`; a `0` a találatot jelenti. (2) A folyamat futó program, a fájlleíró egy folyamat által nyitott erőforrás-hivatkozás; `fork` után a gyerek öröklött másolatot kap ugyanarra a nyitott fájlleírásra (közös offset). (3) Tömb elejére: O(n) eltolás; láncolt lista elejére (fejmutatóval): O(1). (4) A hívott a hibát a visszatérési értékben jelzi, a hívó felel az ellenőrzésért. (5) `parancs > ki.txt 2> hiba.txt`. Értékelés: 5/5 → folytasd; 3–4 → pótold a hiányzó témát (a shell átirányítása és a kilépési kódok a Linux/POSIX bevezetőben, a tömb- és láncolt lista költségmodellje a 28. fejezet konténer-részében); 2 → előbb a shell és a programozás alapjai.

Tanulási útvonalak. A könyv négy részből áll. Az alapútvonal kötelező; a haladó útvonal választható, és az alapútvonalra épül.

Útvonal	Fejezetek	Cél
C alapútvonal	1–17	C17 és Linux/POSIX rendszerprogramozás, a 17. fejezet záróprojektjével
C/C++ híd	18–19	a C23 és a C/C++ határ, átmenet a modern C++-ra
C++ alapútvonal	20–39	C++20 erőforráskezelés, generikus programozás, könyvtár, konkurencia, build; a 39. fejezet portjával
Haladó útvonal	40–46	C++23 elmélyítés, C++26 kitekintés, referencia-függelék

Az alapútvonal kötelező laborjai, ebben a sorrendben: 1.3 (reprodukálható fordítási labor, ez a **következő kötelező labor** e fejezet után), 2.4, 5.5, 9.5, 14.6, 17.2–17.3 (C záróprojekt), 23.5, 25.5, 26.6, 28.6, 30.6, 35.5, 36.5, 37.6 és 39 (a C rendszer C++20 portja). A haladó útvonal laborjai: 40.1, 42.4 és 44.4. A laborok profilját és tesztelt forráscsomagjait az 1.3 alfejezet írja le.

Kötelező laborok ellenőrző lapja. Minden laborhoz: belépőfeltétel, elvárt artefaktum, sikerkritérium, elakadás esetén visszamutatás.

Labor	Belépőfeltétel	Elvárt artefaktum	Sikerkritérium (kész, ha…)	Elakadás esetén
1.3	1. fejezet	működő `lab-gcc` környezet	a `lab-check.sh` `LAB-CHECK: OK`-t ír	1.3 felállítás, verzióellenőrzés
2.4 – 14.6	az előző labor + a fejezet	a fejezet programja és a naplója	a fejezet „Ellenőrzési feltétel” sora teljesül, sanitizerrel hibátlan	az adott fejezet „Tipikus hibák” része

Labor	Belépőfeltétel	Elvárt artefaktum	Sikerkritérium (kész, ha…)	Elakadás esetén
17.2–17.3	1–16. fejezet	`ch17-diag` csomag (CLI + TCP szerver)	a csomag `check.sh`-ja és a nyolc szerver-teszt sikeres	a 9–16. fejezetek I/O részei
23.5 – 35.5	C++ alapok (20–22.)	a fejezet C++ programja	a fejezet teszt/ellenőrző parancsa 0-val tér vissza	a C++ előismeret-pótlás (18–19.)
36.5, 37.6	ABI/build	`ch36-ffi` csomag	a csomag `check.sh` sikeres	14–15. fejezet
38.2 → 39	17. fejezet projekt + C++20	`ch38-cmake`, `ch39-port` csomag	a portolt szolgáltatás tesztje és a `diag.h` viselkedése változatlan	17.2, 38. fejezet
40.1, 42.4, 44.4 (haladó)	az alapútvonal kész	`lab-next` környezet (1.3)	a feature-teszt nem utasít el, a program lefut	az 1.3 `lab-next` mátrixa

Mi számít befejezettnek: a laborhoz tartozó „Ellenőrzési feltétel” teljesül, és a futtatott parancsok kimenete a laborjegyzőkönyvedben szerepel.

Az önellenőrzéshez és a visszatéréshez a Hibaminták, döntési táblák és önellenőrző térkép (46.5) ad tünetből kiinduló térképet: ha egy későbbi fejezetben elakadsz, ott keresd meg a tünetet, és az a releváns fejezetre és előismeretre mutat.

1.1.2 A forrástól a folyamatig

A **forrásprogram** ember által szerkesztett szövegekből áll. A **fordítási egység** (*translation unit*) nem egyszerűen egy `.c` fájl: a preprocessor által feldolgozott forrás, a beillesztett headerek tartalmával, a feltételes részek kiválasztása és a makróhelyettesítések után.

A **végrehajtható állomány** a fordítási és linkelési folyamat egyik lehetséges eredménye. A **folyamat** ezzel szemben futásidejű rendszerpéldány: saját virtuális címtérrel, végrehajtási állapottal és OS-erőforrásokkal. Ugyanabból az állományból több folyamat indulhat; az állomány nem tartalmazza előre valamennyi futás konkrét állapotát.

A Linux-labor szokásos eszközlánca:

Szereplő	Feladata	Fontos megkülönböztetés
Preprocessor	Include-ok, makrók és feltételes fordítás feldolgozása	Szövegszintű előkészítés, nem programfuttatás
Fordító (*compiler*)	Nyelvi elemzés, ellenőrzés, optimalizálás, célkód előállítása	A nyelvi jelentésből készít implementációt

Szereplő	Feladata	Fontos megkülönböztetés
Assembler	Assemblyből gépi kódot tartalmazó objektumfájl készítése	Integrálva is lehet a fordítóba
Linker	Objektumfájlok és könyvtárak összekapcsolása, szimbólumhivatkozások feloldása	Egy objektumfájl rendszerint még nem önálló program
Betöltő (*loader*)	A program futásának előkészítése	Linuxon a kernel és dinamikus programnál a dinamikus linker is részt vesz benne
Runtime	Indulási, leállási és egyéb futásidejű támogatás	Nem szükségképpen értelmező vagy virtuális gép
libc	A C könyvtári szolgáltatásainak és gyakran rendszerinterfészeknek a megvalósítása	Nem azonos sem a kernellel, sem a teljes runtime-mal

Ezek logikai szerepek, nem feltétlenül külön programok. A `gcc` vagy `clang` parancs rendszerint **fordítódriverként** több lépést vezérel. Az objektumfájl nevében szereplő „objektum” pedig nem azonos a nyelvi memóriamodell objektumával.

A fordítási fázisok részleteit a Fordítási fázisok és fordítási egységek tárgyalja. Itt az a fontos, hogy az ISO C nem követel ELF-formátumot, Linux-folyamatot vagy külön assemblerprogramot.

1.1.3 Objektum, érték, típus, cím és bájt

ISO C17: az **objektum** (*object*) a végrehajtási környezet olyan adattárolási tartománya, amely értékeket tartalmazhat. Nem feltétlenül objektumorientált osztálypéldány: egy `int` változó által megnevezett tárolási tartomány is objektum.

Az **érték** az adat jelentése, például a negyvenkettő egész szám. A **típus** meghatározza többek között az értékkészletet, a megengedett műveleteket és az értékek tárolási értelmezését. Egy kifejezésnek is lehet értéke anélkül, hogy külön, címezhető objektum tartozna hozzá.

Az objektum **címe** pointerértékkel jelölhető, amikor a nyelv megengedi a címképzést. A pointer nem általánosan „egy egész számba írt fizikai cím”. A Linux felhasználói folyamatában megfigyelt szokásos cím virtuális cím; a fizikai leképezést az OS és a hardver kezeli.

Nyelvi fogalom	Lehetséges gépi megvalósítás	Ami nem következik belőle
Lokális egész objektum	Memóriahely, regiszter vagy optimalizálással elhagyott tárolás	Nem szükségképpen kap stackhelyet
Objektum értéke	Bitekkel kódolt reprezentáció vagy előre kiszámított konstans	Nem szükségképpen történik memóriából olvasás
Objektum címe	A laborban rendszerint virtuális címként megvalósított pointer	Nem garantált, hogy futások között változatlan
Objektumméret	`sizeof` által megadott C bájtszám	Nem azonos a hasznos értékbitek számával

A könyvben **C_byte** jelöli a `sizeof` egységét. Egy C bájt `CHAR_BIT` bitből áll; ezt a `<limits.h>` adja meg. ISO C17 szerint:

$$\text{sizeof}(\texttt{char}) = 1, \qquad \texttt{CHAR_BIT} \geq 8.$$

A **bájt** ezért nem minden C-implementációban azonos a nyolcbites **oktettel**. A kijelölt x86-64 laborban nyolcbites bájtokkal dolgozunk, de ez a laborprofil tulajdonsága.

Ha `x` olyan objektum, amelyre a `sizeof x` megengedett, reprezentációjának mérete bitekben:

$$B_x = (\text{sizeof}\, x) \cdot \texttt{CHAR_BIT}.$$

Ez a teljes reprezentáció mérete. Nem állítja, hogy minden bit önálló értékinformációt hordoz: a típusnak lehetnek kitöltőbitjei is.

1.1.4 Az absztrakt gép és az as-if elv

Az absztrakt gép nem egy háttérben futó szoftver. Azt írja le, hogyan értelmezzük a program műveleteit és azok megfigyelhető következményeit.

Az **as-if elv** előképe így foglalható össze: a megvalósítás eltérhet a forrás lépésenkénti, naiv végrehajtásától, amennyiben megőrzi a definiált programvégrehajtás szabvány által megkövetelt megfigyelhető viselkedését. Az I/O és a `volatile` hozzáférések például külön követelményekhez kötöttek; ezek részletei később következnek.

Ebből nem következik, hogy minden forrásbeli változónak külön memóriahelye lesz, vagy minden aritmetikai kifejezéshez futásidejű utasítás tartozik. A debugger által mutatható lokális változók megőrzése sem általános nyelvi követelmény.

A **nem definiált viselkedés** (*undefined behavior*, UB) esetén a szabvány nem ír elő viselkedési követelményeket. Ez nem ugyanaz, mint egy dokumentálandó implementációs választás. Egy sikeres próbafutás önmagában egyik nyelvi előfeltétel teljesülését sem bizonyítja.

1.1.5 Hosted, freestanding, C és C++

ISO C17: a **hosted** környezet a szabvány hosted végrehajtásra előírt indulási és könyvtári szolgáltatásait biztosítja. A szokásos Linux-parancssori C-program ilyen környezetben fut; a program indulásakor meghívott függvény neve `main`.

Freestanding környezetben az indulás módja és az induló függvény neve implementáció által meghatározott, a kötelező könyvtári készlet pedig szűkebb. Kernel, bootloader vagy mikrokontrolleres program gyakran ilyen környezethez tartozik. A „freestanding” azonban nem egyszerűen az „OS nélküli” szinonimája: a nyelvi és könyvtári szerződésről szól.

A **C és a C++ önálló nyelvek**. Hasonló szintaxisuk nem jelent azonos konverziós, inicializálási vagy élettartam-szabályokat. A könyv C-alapja C17, C++-alapja C++20. Egy C-program C++-fordítóval való elfogadtatása nem általános helyességi ellenőrzés; a különbségeket a C és C++: azonos szintaxis, eltérő szemantika rendszerezi.

1.1.6 Kidolgozott példa: a változó nem kötelező memóriahely

Az alábbi teljes program minimumkövetelménye a könyvben C17 hosted környezet. A GCC-buildparancsok profilja a központi profiltábla `lab-gcc` profilja, a `clang` sor a `lab-clang` profilhoz tartozik. **Buildjegyzőkönyv:** a két GCC-parancs (`-O0`, `-O2`) a `lab-gcc` profilban figyelmeztetés nélkül fordított, a program mindkét esetben `value=42; size=4 C_byte; CHAR_BIT=8` kimenettel futott; a Clang-sort nem ellenőriztük.

```c
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int count = 21;
    int total = count * 2;

    if (printf("value=%d; size=%zu C_byte; CHAR_BIT=%d\n",
               total, sizeof total, CHAR_BIT) < 0) {
        return EXIT_FAILURE;
    }

    if (fflush(stdout) == EOF) {
        return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}
```

Mentés `model.c` néven, majd külön laborlépésként fordítás:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g model.c -o model-o0
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g model.c -o model-o2
clang -std=c17 -Wall -Wextra -Wpedantic -O2 -g model.c -o model-clang
```

A parancsokhoz a kijelölt verziójú eszközöknek már telepítve kell lenniük. A környezet rögzítését a Reprodukálható fordítási és diagnosztikai labor részletezi.

A `count` és `total` nyelvi objektumok. A `sizeof total` a `total` típusának tárolási méretét adja, nem a 42 leírásához szükséges minimális bitszámot. A `%zu` a `sizeof` eredménytípusának megfelelő formátum.

A `stdout` standard streamet a futtatási környezet biztosítja; a program használja, de nem zárja le. A lezárás érvénytelenítené ezt a használati lehetőséget. A példa a kiíráskor és a puffer ürítésekor jelzett hibát sikertelen kilépési állapottal kezeli.

Bizonyításvázlat az előre kiszámítás megengedhetőségére:

1. A `count` inicializálás után 21.
2. A szorzás eredménye $21 \cdot 2 = 42$; mindkét érték minden C17-es `int` típusban ábrázolható.
3. A két objektum címét nem kérjük le, és nincs közbülső módosítás.
4. A kiírásnak a 42 értéket kell megkapnia, de ehhez nem szükséges két külön memóriahely vagy futásidejű szorzás.

Sikeres I/O mellett tehát a `value` mező elvárt értéke 42. A méretmezők az implementációt jellemzik. A tényleges utasítássor és a debuggerben látható változók köre csak vizsgálattal állapítható meg; az `-O0` sem garantál szó szerinti forrás–utasítás megfelelést.

1.1.7 Gyakori hibák

A „lokális változó a stacken van” hasznos kezdeti gépi kép lehet, de nem nyelvi garancia. Javítása: előbb az objektum típusát és élettartamát azonosítsuk, utána vizsgáljuk a megvalósítást.

A „lefordult, tehát helyes” következtetés szintén hibás. A fordító elfogadása nem bizonyítja minden lehetséges végrehajtás szabályosságát.

A „nálam négy bájt, tehát mindig négy bájt” általánosítás összekeveri a target tulajdonságát a nyelvi követelménnyel. A megfigyelés mellé mindig kell környezeti profil.

1.1.8 Feladatok

1.1.8.1 Magyarázó feladat

Miért nem azonos a `model.c`, a fordítási egység, a `model-o2` és az abból indított folyamat? Melyikből lehet egyszerre több futó példány?

1.1.8.2 Kódolási feladat

Egészítsd ki a programot a `sizeof(char)` és a `sizeof total * CHAR_BIT` matematikai szorzatának kiírásával. Melyik eredmény garantált, és melyik implementációfüggő? Tartsd meg az I/O-hibakezelést.

1.1.8.3 Diagnosztikai feladat

Az optimalizált program debuggerében a `count` értékére „optimized out” jelzés jelenik meg, miközben sikeres I/O mellett a program 42-t ír ki. Ez önmagában fordítóhibát bizonyít? Mit ellenőriznél?

1.1.9 Megoldások

1.1.9.1 A program különböző alakjai

A `model.c` forrásfájl; a fordítási egység a preprocesszálás utáni nyelvi bemenet; a `model-o2` linkelt végrehajtható állomány; a folyamat annak futásidejű példánya. Ugyanabból az állományból több folyamat indulhat.

Ellenőrzési feltétel: a magyarázat különítse el a szerkesztett szöveget, a fordítás bemenetét, a buildterméket és a futási állapotot.

1.1.9.2 A méretek kiírása

A meglévő `printf` helyére illeszthető részlet, az eredeti hibakezelő blokkal:

```
if (printf("char=%zu C_byte; total=%zu bits\n",
           sizeof(char),
           sizeof total * (size_t)CHAR_BIT) < 0) {
    return EXIT_FAILURE;
}
```

A szorzás eredménytípusa így `size_t`, tehát a `%zu` megfelelő. Előfeltétel, hogy a matematikai szorzat beleférjen ebbe a típusba; a kijelölt laborprofilban ez teljesül.

A `char` mérete garantáltan 1 C_byte. A második eredmény a teljes objektumreprezentáció bitmérete, nem az értékbitek száma.

Ellenőrzési feltétel: a `char` mező 1, a második mező pedig a külön megfigyelt méret és `CHAR_BIT` szorzata legyen.

1.1.9.3 Az optimalizált változó értelmezése

Nem bizonyít fordítóhibát. A fordító elhagyhatta a külön tárolást, a debuginformáció pedig nem feltétlenül teszi rekonstruálhatóvá a változót.

Ellenőrizni kell a forrás és a bináris összetartozását, a fordítási kapcsolókat, a fordítóverziót és szükség esetén a gépi kódot.

Ellenőrzési feltétel: a következtetés a megfigyelhető programviselkedésre támaszkodjon, ne egy feltételezett stackhely hiányára.

1.1.10 Összefoglaló

A C-program jelentését az absztrakt gép szabályai adják; a natív folyamat ezek egyik megvalósítása. Az objektum nem fizikai memóriahely, a C bájt nem szükségképpen oktett, a változó pedig nem garantált stackrekesz. A helyes rendszerszemlélet minden állításnál megkülönbözteti a nyelvi követelményt, az implementáció szerződését és a konkrét labor megfigyelését.

1.2 Szabványok, dialektusok és hordozhatósági szerződés

A „C-ben írt program” önmagában még nem pontos technikai meghatározás. Tudnunk kell, melyik nyelvváltozat szabályait követi, milyen könyvtárakra támaszkodik, és milyen platformfeltételeket vár el. Ugyanaz a forrás lefordulhat két gépen úgy, hogy eltérő szolgáltatások állnak rendelkezésére, vagy egyes típusok más reprezentációt kapnak.

Az Absztrakt gép, forrásprogram és natív folyamat különválasztotta a nyelvi jelentést és a tényleges végrehajtást. Most ezt **hordozhatósági szerződéssé** alakítjuk: rögzítjük, mit követelhet a program a nyelvtől, a könyvtártól és a környezettől.

1.2.1 A könyv nyelvi alapprofilja

A C példák alapja **ISO C17**, a C++ példáké **ISO C++20**. A C23 és C++23 képességeket külön jelöljük. C és C++ két külön nyelv: a C++ nem egyszerűen egy másik kapcsolóval fordított C.

GCC/Clang eszközprofilban a nyelvi módot mindig explicit választjuk:

```
gcc -std=c17 -Wall -Wextra -pedantic-errors profile.c -o profile-gcc
clang -std=c17 -Wall -Wextra -pedantic-errors profile.c -o profile-clang

g++ -std=c++20 -Wall -Wextra -pedantic-errors capability.cpp -o capability-gcc
clang++ -std=c++20 -Wall -Wextra -pedantic-errors capability.cpp -o
 ↪  capability-clang
```

Ezek fordítási parancsok a későbbi teljes példákhoz. A figyelmeztetések hasznos diagnosztikát adnak; a `-pedantic-errors` a választott szabványhoz kapcsolódó kötelező diagnosztikák szigorítását szolgálja. **A sikeres fordítás egyik kapcsolókészlettel sem bizonyítja a program helyességét vagy teljes hordozhatóságát.**

A `gnu17` és `gnu++20` GNU dialektusok: szabványos alaphoz fordítóbővítéseket adnak. Használatuk lehet indokolt, de külön szerződésként kell megjelenniük. Az ISO mód sem jelenti automatikusan minden bővítés elutasítását.

A könyv profiljait névvel azonosítjuk, és egyetlen helyen, a központi profiltáblaban rögzítjük: a `gcc`/`g++` parancsok a `lab-gcc`, a `clang`/`clang++` parancsok a `lab-clang` profilhoz tartoznak; a C23, C++23 és C++26 kitekintés a `lab-next` profilt használja. A Clang által kiválasztott C++ könyvtárat a buildnaplóban külön rögzíteni kell. **Buildjegyzőkönyv:** a lenti `profile.c` és `capability.cpp` a fenti GCC-parancsokkal a `lab-gcc` profilban hiba és figyelmeztetés nélkül fordult; a `profile-gcc` kimenete `__STDC_VERSION__=201710`, a `capability-gcc` kimenet nélkül, `0` kóddal futott. A Clang-sorokat nem ellenőriztük.

Példaleltár és kiadási build. A könyv minden C és C++ kódblokkja egy gépi leltárban szerepel (`labs/example_inventory.py`, a jegyzőkönyv a `labs/release/` mappában), amely a blokkokat öt fajtába sorolja: *teljes program* (`main`-nel, buildelve és futtatva), *fordítási egység* (fájlnévvel jelölt header vagy main nélküli forrás, a fejezet többi forrásával együtt buildelve), *részlet* (kontextust igénylő, önállóan nem fordítható kivonat), *negatív példa* (szándékosan hibás; csak fordítási próbát kap, és a jegyzőkönyv rögzíti, hogy fordítási időben bukik-e, vagy futásidejű, illetve UB-hibát szemléltet) és *kitekintés* (C++26, modul-, platform- vagy fordítófüggő kód, amelyet a laborprofil nem épít). A teljes programokat és fordítási egységeket a `lab-gcc` profil buildeli; ha egy C++23-as könyvtári elem a GCC 13 libstdc++-ból hiányzik, a próba a Clang 22 + libc++ 22 párossal ismétlődik. Ahol mindkét könyvtárból hiányzik a képesség (`std::generator`, `std::start_lifetime_as`), vagy a fájlt a fejezet CMake-receptje generálja, a jegyzőkönyv a kudarcot indokolt kivételként, okkal együtt rögzíti. A jegyzőkönyv fejezetenként a forrásfájl SHA-256 kivonatát is tartalmazza, így pontosan az adott kéziratváltozathoz kötött; a kézirat módosítása után a leltárt újra kell futtatni.

1.2.2 Történeti térkép

A szabványév nyelvi és könyvtári követelményrendszert jelöl, nem egy fordító kiadási évét. Az alábbi térkép tájékozódási segítség, nem teljes változáslista.

Nyelvváltozat	Jellegzetes változás	Szabványos verziójelző
C89 / ISO C90	Az ANSI C, majd ISO C alapjainak rögzítése	`__STDC_VERSION__` még nincs
C99	Például `//`, `long long`, `<stdint.h>`	`199901L`
C11	Atomikus műveletek, szálkezelési könyvtár, `_Generic`	`201112L`
C17	Elsősorban hibajavítások és pontosítások	`201710L`
C23	Például `nullptr`, `typeof`, új könyvtári elemek	`202311L`
C++11	Mozgatás, lambdák, konkurenciamodell	`201103L`
C++14	Többek között generikus lambdák, bővebb `constexpr`	`201402L`
C++17	Például strukturált kötés, `if constexpr`, `filesystem`	`201703L`
C++20	Például concepts, ranges, coroutine-ok	`202002L`
C++23	Például `std::expected`, `std::print`	`202302L`

A C sorainál a jelző `__STDC_VERSION__`, a C++ sorainál `__cplusplus`. A C verziómakrója a C95 módosítással jelent meg, `199409L` értékkel.

A végleges szabványhoz rendelt érték nem jelenti azt, hogy minden részleges implementáció ezt jelenti: fejlesztés alatt álló módok korábbi, átmeneti értéket is használhatnak. C11-ben egyes új szolgáltatások támogatása opcionális volt; a szabványév ezért önmagában nem megfelelő képességvizsgálat.

1.2.3 Az állítások rétegei

Egy állítás mellé oda kell tudnunk írni, **melyik szerződésből következik**.

Réteg	Mit határoz meg?	Példa
ISO nyelv	Típusok, kifejezések, objektumok, élettartam szabályai	Előjeles egész túlcsordulásának megítélése
ISO standard könyvtár	A nyelvi szabvány könyvtári felülete	C-ben `printf`, C++-ban `std::vector`
POSIX	Operációsrendszer-szolgáltatások hordozható felülete	`fork`, `read`, `pthread_create`
Linux	Linuxhoz kötődő felületek és viselkedés	`epoll`
ABI	Bináris illesztési szerződés	Paraméterátadás, adattípus-reprezentáció, objektumformátumhoz kapcsolódó szabályok

Réteg	Mit határoz meg?	Példa
Fordítóbővítés	Az adott implementáció szabványon túli képessége	GNU statement expression

Az ISO standard könyvtár is a nyelvi szabvány része; azért választjuk külön, mert implementációja és kiadási ciklusa eltérhet a fordító nyelvi elemzőjétől.

Az ABI, vagyis *Application Binary Interface* nem azonos az API-val. Az API a forrásból használható felületet írja le; az ABI azt is meghatározza, hogyan találkoznak a lefordított részek. Egy méretállításnál ezért kevés annyi, hogy „64 bites Linux”: az ABI-t és az adatmodellt is meg kell nevezni.

Forráskezeléskor az ISO szabályt szabványszöveggel, a POSIX követelményt POSIX dokumentációval, a Linux API-t Linux dokumentációval, a bővítést fordítódokumentációval kell ellenőrizni. A verzió és a konkrét szakasz lényeges. A helyi mérés csak a rögzített profil megfigyelése, nem általános nyelvi bizonyíték; egy keresési kiindulóoldal pedig önmagában nem bizonyíték.

1.2.3.1 Forrásjegyzék

Ez a könyv központi forrásjegyzéke (a szögletes rövidítések minden fejezetben ide mutatnak): az azonosító, a revízió/dátum, a státusz (kiadott vagy draft) és az URL egy táblában van. A hivatkozások a rövidítésre kattintva ezt a szakaszt nyitják meg.

A könyv a szabály- és státuszállításoknál az alábbi, azonosítóval és változattal megadott dokumentumokra hivatkozik. A szögletes zárójeles rövidítést használjuk a szövegben, a szakaszt pedig a szabványtervezetek stabil szakaszcímkéjével adjuk meg, például *[C++20] [dcl.link]*. A nyilvános munkatervezet és a kiadott ISO-szöveg között szerkesztési eltérés lehet; a könyv a tervezetet hivatkozási alapként használja, a kiadott szabványt nem idézi szó szerint.

Rövidítés	Dokumentum	Változat, dátum	Státusz	URL
[C17]	ISO/IEC 9899:2018, *Programming languages — C*	kiadott szabvány, 2018	kiadott (fizetős; a nyilvános megfelelő munkatervezet: WG14 N2176)	https://www.open-std.org/jtc1/sc22/wg14/www/d
[C23]	ISO/IEC 9899:2024; nyilvános munkatervezet: WG14 N3096	kiadott szabvány, 2024; N3096: 2023-04-01	kiadott; a tervezet nem azonos a kiadott szöveggel	https://www.open-std.org/jtc1/sc22/wg14/www/d
[C++20]	ISO/IEC 14882:2020; nyilvános munkatervezet: WG21 N4861	kiadott szabvány, 2020; N4861: 2020-04-01	kiadott; a tervezet az utolsó nyilvános C++20 munkaszöveg	https://timsong-cpp.git-hub.io/cppwp/n4861/
[C++23]	ISO/IEC 14882:2024; nyilvános munkatervezet: WG21 N4950	kiadott szabvány, 2024; N4950: 2023-05-10	kiadott; a tervezet nem azonos a kiadott szöveggel	https://www.open-std.org/jtc1/sc22/wg21/docs/pa

Rövidítés	Dokumentum	Változat, dátum	Státusz	URL
[C++26]	a következő ISO/IEC 14882 kiadás	lásd a státuszt alább	**draft** (nem kiadott szabvány)	https://eel.is/c++draft/ (élő munkaszöveg)
[POSIX]	IEEE Std 1003.1-2024 (POSIX.1-2024)	kiadott szabvány, 2024	kiadott	https://pubs.opengroup.org/onlinepubs/9799919799/
[psABI]	*System V Application Binary Interface, AMD64 Architecture Processor Supplement*, `gitlab.com/x86-psABIs/x86-64-ABI`	a `master` ág `e1ce0983` commitja, 2025-03-12	élő munkaág, rögzített commit	https://gitlab.com/x86-psABIs/x86-64-ABI
[GCC]	*Using the GNU Compiler Collection*, GCC 13.3.0	a `lab-gcc` profil fordítója	kiadott	https://gcc.gnu.org/onlinedocs/gcc-13.3.0/gcc/
[glibc]	*The GNU C Library Reference Manual*, glibc 2.39	a `lab-gcc` profil C könyvtára	kiadott	https://www.gnu.org/software/libc/manual/2.39/
[man]	Linux man-pages	6.7 (Ubuntu 24.04 `manpages` 6.7-2 csomag)	kiadott	https://man7.org/linux/man-pages/

A [C++26] státusza 2026-10-25-i ellenőrzés szerint. A WG21 a 2026. márciusi londoni (Croydon) ülésen, 2026. március 28-án lezárta a C++26 technikai munkáját; a beérkezett nemzeti észrevételeket feldolgozta, képességet nem vett ki és nem adott hozzá. A WG21 2026-07-28-i convenori jelentése (N5056) szerint a szabvány DIS-szavazáson van, és a kiadást 2026-ra ütemezik. Az ISO általi kiadást az ellenőrzés napján nem találtuk igazolva, ezért a könyv a C++26-ot **technikailag lezárt, még nem kiadott** szabványként kezeli. Az implementációs támogatás ettől független; a könyv ezt csak a saját profiljaira méri (lásd a C++26 nyelvi kitekintést).

1.2.4 Nyelvi mód és tényleges képesség

ISO C17 példa, teljes `profile.c` fájl; fordítása a fenti GCC/Clang parancsokkal:

```
#include <stdio.h>

#if !defined(__STDC_VERSION__) || __STDC_VERSION__ < 201710L
#error "C17 or newer is required"
#endif

int main(void)
{
    if (printf("__STDC_VERSION__=%ld\n",
```

```
                __STDC_VERSION__) < 0) {
        return 1;
    }
    return 0;
}
```

C17 módban a kiírandó makróérték `201710`; ez elvárt érték, nem közölt mérés. Az előfeldolgozó ellenőrzése a jelentett nyelvi szintet vizsgálja, nem az implementáció minden részletét. A kiírás jelzett hibáját a program nemnulla visszatérési értékkel továbbítja.

A **képességvizsgáló makrók** (*feature-test macros*) ennél szűkebb kérdésre válaszolnak. C++-ban a `__cpp_...` jelzők nyelvi, a `__cpp_lib_...` jelzők könyvtári képességekhez tartoznak. A könyvtári jelzőket a megfelelő fejléc vagy a C++20-ban rendelkezésre álló `<version>` teszi elérhetővé.

Az alábbi teljes `capability.cpp` **C++20 példa** ugyanazon eszközprofilhoz:

```
#include <version>

#if __cplusplus < 202002L
#error "C++20 or newer is required"
#endif

#if defined(__cpp_lib_int_pow2) && __cpp_lib_int_pow2 >= 202002L
#include <bit>
#endif

constexpr bool is_power_of_two(unsigned int x)
{
#if defined(__cpp_lib_int_pow2) && __cpp_lib_int_pow2 >= 202002L
    return std::has_single_bit(x);
#else
    return x != 0u && (x & (x - 1u)) == 0u;
#endif
}

static_assert(!is_power_of_two(0u));
static_assert(is_power_of_two(16u));
static_assert(!is_power_of_two(18u));

int main()
{
    return 0;
}
```

A fordító először a nyelvi szintet ellenőrzi. Ezután a kiválasztott standard könyvtár jelzője dönti el, melyik megvalósítás forduljon le. A `static_assert` a három megadott esetet fordításkor ellenőrzi.

Bizonyításvázlat a tartalék megoldáshoz: pozitív, előjel nélküli x esetén az `x - 1u` a legalsó egyes bitet nullára, az alatta levő nullákat egyesre változtatja. Az `x & (x - 1u)` tehát eltávolítja a legalsó egyes bitet. Az eredmény pontosan akkor nulla, ha eredetileg egyetlen egyes bit volt. A külön nullavizsgálat kizárja a nullát.

A jelző mellett érdemes fordítási és linkelési képességpróbát is végezni. Egy fejléc megléte nem bizonyítja az összes szükséges könyvtári szimbólum elérhetőségét.

1.2.5 Három különböző viselkedési kategória

1.2.5.1 Nem definiált viselkedés

ISO fogalom: nem definiált viselkedés (*undefined behavior*, UB) esetén a szabvány nem támaszt követelményt a viselkedésre.

Hibás C17 részlet: `int n = INT_MAX; ++n;`, ahol `INT_MAX` a `<limits.h>` szerinti maximum. A növelés eredménye nem ábrázolható `int`-ként.

Nem következik belőle garantált körbefordulás, kivétel vagy összeomlás. A látszólag helyes futás sem igazolja a kódot. Javításként a művelet előtt ellenőrizni kell a tartományt.

1.2.5.2 Nem specifikált választás

ISO fogalom: nem specifikált (*unspecified*) viselkedésnél a szabvány több lehetőséget enged meg, és nem követeli meg a választás dokumentálását.

Például az `f() + g()` kifejezés nem írja elő, melyik függvényhívás történjen előbb, ha a kifejezés egyébként szabályos. Ha a programnak meghatározott sorrend kell, a hívásokat külön, egymást követő utasításokba kell tenni. Ez nem UB: a választás megengedett lehetőségekre korlátozott.

1.2.5.3 Implementáció által meghatározott viselkedés

ISO fogalom: az *implementation-defined* viselkedés olyan megengedett választás, amelyet az implementációnak dokumentálnia kell.

Példa a sima `char` előjelessége. Nem szabad abból kiindulni, hogy minden targeten ugyanaz. Ha az előjelesség lényeges, használjunk explicit `signed char` vagy `unsigned char` típust.

Gyakori hiba mindhárom kategóriát „platformfüggőnek” nevezni. Ezzel elvész a döntő különbség: dokumentálandó választásról, megengedett szabadságról vagy garancia nélküli végrehajtásról beszélünk-e.

1.2.6 Feladatok

1. **Magyarázó:** sorold réteghez a `printf`, `epoll`, GNU statement expression és függvényparaméterek regiszterekben történő átadása állításait. Miért kevés hozzájuk a „C17 program” megjelölés?
2. **Kódolási:** egészítsd ki a C++ példát a `1u` és `3u` bemenetek fordítási idejű ellenőrzésével. Indokold az elvárt eredményeket.
3. **Diagnosztikai:** egy projekt `-std=c++20` mellett lefordul, de nem használható benne egy kívánt C++20 könyvtári szolgáltatás. Egy másik gépen működik. Milyen adatokat és próbákat kérnél a hiba besorolásához?

1.2.7 Megoldások

1. A `printf` ISO C standard könyvtári felület; az `epoll` Linux API; a statement expression fordítóbővítés; a paraméterátadás ABI-kérdés. **Ellenőrzési feltétel:** minden állításhoz megfelelő rétegű dokumentáció legyen rendelhető. A nyelvi mód nem választ Linux API-t vagy ABI-t.

2. A meglévő teljes példába illesztendő C++20 részlet:

   ```
   static_assert(is_power_of_two(1u));
   static_assert(!is_power_of_two(3u));
   ```

   Az $1 = 2^0$ egyetlen egyes bitet tartalmaz; a 3 bináris alakjában kettő van. **Ellenőrzési feltétel:** a kiegészített program mindkét fordítóval forduljon le. Ez a példákat ellenőrzi, nem helyettesíti az általános indoklást.

3. Kérném a teljes fordítási parancsot, a compiler, a standard könyvtár és a target verzióadatait, a releváns feature-test makrót, valamint minimális fordítási és linkelési próbát. Külön vizsgálnám a hiányzó deklarációt és a hiányzó linkelhető megvalósítást. **Ellenőrzési feltétel:** a két gép közti konkrét eltérés legyen azonosítva; a „másik compiler” önmagában nem diagnózis.

1.2.8 Összefoglaló

A hordozhatóság kimondott függőségi szerződés. Rögzítsük az explicit nyelvi módot, külön vizsgáljuk a nyelvi és könyvtári támogatást, és minden állítást rendeljünk a megfelelő ISO, POSIX, Linux, ABI vagy fordítóbővítési réteghez. A sikeres fordítás, a dokumentált garancia és a helyi megfigyelés három különböző bizonyíték.

1.3 Reprodukálható fordítási és diagnosztikai labor

A reprodukálható labor célja nem pusztán az, hogy „ugyanaz a program nálam is lefusson”. Azt kell rögzítenünk, **melyik forrásból, milyen nyelvi módban, mely eszközökkel és milyen célrendszerre** készült a vizsgált program. Egy fordítási figyelmeztetés, sanitizer-jelentés vagy eltérő eredmény csak ezzel a háttérrel értelmezhető.

A Szabványok, dialektusok és hordozhatósági szerződés alapján külön kezeljük az ISO nyelvi szabályokat és a Linuxon választott implementációt. Ebben a részben parancssori laboralapot készítünk; buildrendszerre vagy Linux-specifikus programozási API-ra még nincs szükség.

1.3.1 A laborprofil mint reprodukciós szerződés

Laborprofilnak nevezzük a forrás, a fordítási és futási környezet, valamint a vizsgálati beállítások rögzített együttesét. Legalább a következőket tartalmazza:

- a forrás Git-azonosítóját és az esetleges helyi módosításokat;
- a fordítók, könyvtárak, linker és diagnosztikai eszközök pontos kiadását;
- a célarchitektúrát, ABI-t és nyelvi módot;
- a teljes parancssorokat, releváns környezeti változókat és kimeneteket.

A **megismételhető kísérlet** és a **bájtra azonos build** nem ugyanaz. Az előbbihez az eredmények és körülmények összehasonlíthatósága kell; az utóbbihoz többek között az időbélyegek, beégetett útvonalak és függőségek további szabályozása is szükséges. Itt az első célt alapozzuk meg.

1.3.1.1 A könyv központi profiljai

A könyv minden buildpéldája pontosan egy, névvel azonosított profilhoz tartozik. A profilokat egyetlen helyen, az alábbi táblában rögzítjük; a fejezetek csak a nevükre hivatkoznak. Ha egy példa nem nevez meg profilt, a `lab-gcc` profil érvényes; a C23, C++23 és C++26 kitekintő példák alapértelmezése a `lab-next`.

Profil	Státusz	Mire szolgál
`lab-gcc`	**ellenőrzött**: a könyvben közölt buildjegyzőkönyvek ebben készültek	a C17 és C++20 alapprofil, minden teljes program és labor
`lab-clang`	választott, a könyvben nem ellenőrzött	második fordító ugyanarra a célra; diagnosztikai összevetés
`lab-aarch64`	választott, a könyvben nem ellenőrzött	összehasonlító cél az ABI- és memóriamodell-fejezetekhez
`lab-next`	választott, a könyvben nem ellenőrzött	C23, C++23 és C++26 kitekintés újabb fordítóval

A `lab-gcc` profil pontos összetétele:

Összetevő	Rögzített választás
Disztribúció	Ubuntu 24.04.3 LTS
Cél	x86-64 Linux, System V ABI, LP64, hosted user-space
C fordító	GCC 13.3.0 (`gcc-13` 13.3.0-6ubuntu2~24.04.1)
C++ fordító	G++ 13.3.0, ugyanabból a csomagból
C könyvtár	glibc 2.39 (`libc6` 2.39-0ubuntu8.9)
C++ könyvtár	libstdc++: a GCC 13 headerjei (`_GLIBCXX_RELEASE` = 13, `__GLIBCXX__` = 20240904); futásidejű `libstdc++.so.6` a `libstdc++6` 14.2.0-4ubuntu2~24.04.1 csomagból
Assembler és linker	GNU binutils 2.42, GNU `ld.bfd`
Nyelvi mód	`-std=c17`, illetve `-std=c++20`; GNU dialektus nélkül
Diagnosztika	GDB 15.0.50 (Ubuntu csomag), strace 6.8
Build és teszt	GNU Make 4.3; Python 3.12.3; CMake 4.4.4 és Ninja 1.13 (`pip` csomagból, nem disztribúciós csomag)
Kernel a jegyzőkönyvek idején	Linux 7.0.0 (x86-64)

A C++ könyvtár sora szándékosan két adatot tartalmaz. A fordítás a GCC 13 libstdc++ headerjeivel történik, a programok viszont a disztribúció újabb, visszafelé kompatibilis `libstdc++.so.6` futásidejű könyvtárával futnak. A kettő nem ugyanaz, és egy hibaelemzésnél mindkettő számít.

Profilleltár és gépi ellenőrzés. A fejezetek buildblokkjai és profilmondatai csak a fenti négy profilnévre hivatkoznak (a profil nélküli példa `lab-gcc`); konkrét fordító- vagy glibc-verziót profilnév nélkül nem neveznek meg. Ezt a `labs/profile_check.py` gépileg ellenőrzi (minden, a `lab-gcc` alapértelmezésén kívüli verziómegjelölés mellett definiált profilnévnek kell szerepelnie), így az azonos nevű profilnak egyetlen feloldása van.

A többi profil a következő kiadásokat jelöli ki. Ezek választott profilspecifikációk, nem itt ellenőrzött telepítések; a könyv nem állít róluk fordítási vagy futási eredményt.

Profil	Fordító	Könyvtárak	Cél
`lab-clang`	Clang 18.1.8	glibc 2.39; libstdc++ a `lab-gcc`-ből, vagy libc++ 18 – a választást a buildnaplóban rögzíteni kell	mint `lab-gcc`
`lab-aarch64`	GCC 13.3.0 natív AArch64 környezetben	glibc 2.39, libstdc++ 13	AArch64 Linux, AAPCS64, LP64
`lab-next`	GCC 14.2.0 vagy újabb; Clang 19.1.7 vagy újabb	glibc 2.40 vagy újabb, binutils 2.43.1 vagy újabb, a fordítóhoz tartozó standard könyvtár	x86-64 Linux

A `lab-next` konkrét, ellenőrzött kombinációja (haladó C++23 laborokhoz). A `lab-next` fentebbi „vagy újabb” megadása nem telepíthető recept; a kötelező haladó laborokhoz (a 40.1, 42.4 és 44.4 szerinti kijelölt laborok) az alábbi, a példaleltár kiadási buildjében (`labs/release/REPORT.md`) ténylegesen használt kombináció az **ellenőrzött lab-next környezet**:

Összetevő	Rögzített választás	Telepítés
Fordító + standard könyvtár	Clang 22 és libc++ 22 (a `ziglang` 0.17.0 Python-csomagból: `python3 -m ziglang c++`)	`python3 -m venv ~/lab-next && ~/lab-next/bin/pip install 'ziglang==0.17.0'`
Build	`python3 -m ziglang c++ -std=c++23 -stdlib=libc++ …`; CMake 4.4.4 és Ninja 1.13 (pip)	lásd a `lab-gcc` felállítását
`<mdspan>`	a Kokkos referencia-implementáció (single-header, commit `86eefa3d6653`) egy `-I` shim könyvtáron át	a shim `mdspan` fájlja a Kokkos headert includeolja
TSan	`setarch -R` a futtatáshoz (ASLR-megkötés ezen a kernelen)	—

Képességmátrix (= ellenőrzötten fordul és fut; = egyik elérhető standard könyvtárban sincs meg, a példa **csak olvasásra** szánt, és a feature-teszt szándékosan elutasít):

Képesség	`lab-gcc` (GCC 13.3)	`lab-next` (Clang 22 + libc++ 22)
`std::expected`, a C++23	`expected` , a	
`<ranges>` kiegészítések	ranges-kiegészítések részben	
`<mdspan>`		(Kokkos shim)
`std::generator`		(csak olvasásra)
`std::start_lifetime_as`		(csak olvasásra)
`import std` / modulok		külön CMake 3.28+/Ninja dyndep recept szükséges

A kijelölt haladó laborok teljes, dokumentáltan működő környezete tehát a `lab-next` fenti kombinációja; a jelű képességekre épülő példák nem futtathatók semelyik elérhető környezetben, ezeket a szöveg „csak olvasásra szánt” jelöléssel látja el.

Más kiadások is használhatók, de azok külön profilt alkotnak, saját névvel. A helyi jegyzékben a disztribúciós csomagverziót és eredetet is rögzíteni kell: azonos upstream verzió mögött eltérő javításkészlet állhat. A buildjegyzőkönyvekben a „GCC 13.3.0” megjelölés a `lab-gcc` profilt jelenti.

1.3.1.2 A `lab-gcc` profil felállítása

A `lab-gcc` profil egy Ubuntu 24.04 LTS (x86-64) rendszer disztribúciós csomagjaiból áll össze; külön fordítóépítés nem kell. Friss telepítésen vagy konténerben (például az `ubuntu:24.04` image-ből indulva):

```
# Előfeltétel: root jogosultság. Hosztgépen `sudo`; az `ubuntu:24.04`
 ↪  konténerben általában root a felhasználó,
# és a `sudo` csomag nincs telepítve (ilyenkor a `sudo` előtagot hagyd el, vagy
 ↪  előbb telepítsd).
sudo apt-get update
sudo apt-get install -y build-essential gdb strace python3 python3-venv
# opcionális, a CMake-es fejezetekhez (38-39): rögzített verziók, nem a
 ↪  mindenkori legújabb
python3 -m venv ~/lab-venv && ~/lab-venv/bin/pip install 'cmake==4.4.4'
 ↪  'ninja==1.13.0'
# a venv-et a PATH elé kell tenni (vagy: source ~/lab-venv/bin/activate),
 ↪  különben a rendszer cmake-je fut
export PATH="$HOME/lab-venv/bin:$PATH"
command -v cmake ninja && cmake --version | head -1 && ninja --version
```

Ez egy friss telepítési recept, nem a profil pontos reprodukciója. Az `apt-get` és az `ubuntu:24.04` címke a telepítés napján aktuális kiadásokat hozza, így eltérhet a táblázat pontos verzióitól; ezért *eltérhető indulási profilnak* tekintendő. A pontos profil felállításához: (1) rögzítsd az image digestjét (`docker image inspect --format '{{index .RepoDigests 0}}' ubuntu:24.04`) és használd azt a címke helyett; (2) kérd a táblázat pontos csomagverzióit, például `sudo apt-get install -y gcc-13=13.3.0-6ubuntu2~24.04.1 g++-13=13.3.0-6ubuntu2~24.04.1 libc6-dev=2.39-0ubuntu8.9` (ha az archívum már nem tartalmazza, snapshot.ubuntu.com használható); (3) futtasd az alábbi verzióellenőrző lépést, és őrizd meg a kimenetet a buildnaplóval együtt:

```
gcc --version | head -1        # várt: gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1)
 ↪ 13.3.0
g++ --version | head -1        # várt: g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1)
 ↪ 13.3.0
ldd --version | head -1        # várt: glibc 2.39
ld --version | head -1         # várt: GNU ld (GNU Binutils for Ubuntu) 2.42
make --version | head -1       # várt: GNU Make 4.3
python3 --version              # várt: Python 3.12.3
```

Eltérés esetén a jegyzőkönyvben fel kell tüntetni a tényleges verziót és azt, hogy az eltérés (például újabb minor kiadás) hogyan kezelendő; a könyv kimenetei csak a táblázat verzióira érvényesek. A buildnapló és a verzióellenőrzés kimenete a `labs/release/` jegyzőkönyvben a lab-gcc futtatásokhoz tartozik.

A `build-essential` a GCC/G++ 13-at, a GNU binutils-t, a `make`-et és a glibc fejlesztői fájljait hozza; az ASan és az UBSan futásidejű könyvtára (`libasan8`, `libubsan1`) a `gcc-13` csomag függősége. A rögzített csomagverziók a központi profiltáblában szerepelnek; egy későbbi frissítés után a `dpkg -l gcc-13 g++-13 libc6-dev binutils gdb` kimenetét kell a laborjegyzőkönyvbe menteni, és eltérés esetén a profilt újra rögzíteni. Más disztribúció vagy macOS külön, nem ellenőrzött profil; ott a megfelelő csomagneveket és a sanitizer-támogatást külön kell igazolni.

A felállítás után az alábbi szkript egy lépésben ellenőrzi a C és C++ fordítást és linkelést, az ASan és az UBSan működését szimbolizált forráshellyel, valamint a `gdb`-t:

```
#!/usr/bin/env bash
# lab-check.sh - a lab-gcc profil működőképességének ellenőrzése
set -euo pipefail
work=$(mktemp -d)
trap 'rm -rf "$work"' EXIT
cd "$work"

cat > hello.c <<'C'
#include <stdio.h>
int main(void) { puts("c17 ok"); return 0; }
C
cat > hello.cpp <<'CPP'
#include <iostream>
#include <vector>
int main() { std::vector<int> v{1, 2, 3}; std::cout << "c++20 ok " << v.size()
 ↪ << '\n'; }
CPP
cat > heap_bug.c <<'C'
#include <stdlib.h>
int main(void) {
    int *p = malloc(4 * sizeof *p);
    volatile int *q = p;   /* volatile: az optimalizáló nem ejtheti ki */
    q[4] = 1;              /* szándékos heap-buffer-overflow */
    free(p);
    return 0;
}
C
cat > int_bug.c <<'C'
#include <limits.h>
```

```
#include <stdio.h>
int main(int argc, char **argv) {
    (void)argv;
    int x = INT_MAX - 1 + argc;   /* argc == 1: INT_MAX */
    printf("%d\n", x + 1);       /* szándékos signed overflow */
    return 0;
}
C

gcc -std=c17 -g -Wall -Wextra -Wpedantic -O2 hello.c -o hello_c && ./hello_c
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 hello.cpp -o hello_cpp &&
 ↪  ./hello_cpp

# A várt hibát úgy kérdezzük le, hogy a set -e ne lépjen ki:
expect_failure() {      # minta fájl parancs...
    local pattern=$1 out=$2 status
    shift 2
    if "$@" 2> "$out"; then status=0; else status=$?; fi
    test "$status" -ne 0 || { echo "váratlan siker: $*" >&2; return 1; }
    grep -q "$pattern" "$out" || { echo "nincs '$pattern': $*" >&2; return 1; }
}

gcc -std=c17 -g -O1 -fsanitize=address -fno-omit-frame-pointer \
    heap_bug.c -o heap_bug
expect_failure 'heap-buffer-overflow' asan.txt ./heap_bug
grep -q 'heap_bug.c:5' asan.txt            # szimbolizált forráshely
echo "asan ok"

gcc -std=c17 -g -O1 -fsanitize=undefined -fno-sanitize-recover=undefined \
    int_bug.c -o int_bug
expect_failure 'signed integer overflow' ubsan.txt ./int_bug
echo "ubsan ok"

gdb --batch -ex 'break main' -ex run ./hello_c > gdb.txt 2>&1
grep -q 'Breakpoint 1, main' gdb.txt        # szimbólum és forráshely
echo "gdb ok"
echo "LAB-CHECK: OK"
```

Két részlet tanulságos. Az ASan csak a ténylegesen végrehajtott memória-hozzáférést látja. `-O1` mellett a GCC egy `free` előtti, soha ki nem olvasott írást kiejthet, és akkor nincs mit jelezni; a próba ezért `volatile` hozzáférést használ. A `volatile` itt csak **szimptomatikus, implementációfüggő megoldás**: azt kéri a fordítótól, hogy az adott hozzáférést ne távolítsa el, de általában nem szabványos bizonyíték arra, hogy egy nem definiált viselkedésű művelet változatlanul megmarad (UB-nál a fordítónak nincs kötelessége). Ezért a próba csak az ASan működésének ellenőrzésére szolgál, nem általános módszer UB megőrzésére. A kombinált `-fsanitize=address,undefined` buildben pedig az UBSan alapértelmezésben jelez, de továbbfut, és a program 0-val is kiléphet; a próbák ezért külön buildet és `-fno-sanitize-recover=undefined`-et használnak.

Buildjegyzőkönyv (`lab-gcc` profil). A szkript a profil gépén `c17 ok`, `c++20 ok 3`, `asan ok`, `ubsan ok`, `gdb ok`, `LAB-CHECK: OK` kimenettel, 0 kóddal futott. A fenti `apt-get` felállítást tiszta konténerben nem próbáltuk ki, mert a jegyzőkönyv gépén nem volt konténer-futtatókörnyezet; a csomaglista a jegyzőkönyv gépének `dpkg -l` kimenetéből származik.

1.3.1.3 Tesztelt forráscsomagok

Letöltés (verzió 1.0, a kiadás `labs/` mappájában): `ch17-diag-1.0.tar.gz`, `ch36-ffi-1.0.tar.gz`, `ch38-cmake-1.0.tar.gz`, `ch39-port-1.0.tar.gz`; ellenőrzőösszeg: `SHA256SUMS` (`sha256sum -c SHA256SUMS`). Kicsomagolás után a csomag `README.md`-je és `check.sh`-ja írja le a futtatást. A csomagok a `labs/make_release.py` kimenetei, a fejezetekből generáltak.

A nagyobb laborokhoz a kiadás mellékleteként fejezetenként elkülönített forráscsomag tartozik: `ch17-diag` (17. fejezet: C mag, CLI, TCP-adapter, referencia-szerver), `ch36-ffi` (36.5: ABI- és FFI-labor), `ch38-cmake` (38.2: target-alapú CMake projekt) és `ch39-port` (39. fejezet: C++20 port, szolgáltatás, provenance). A csomagok fájljai bájtra azonosak a könyvben közölt kódblokkokkal, mert ugyanabból a forrásból generáljuk őket; a hosszú kódokat tehát nem kell a PDF-ből másolni. Minden csomag `README.md` fájlja rögzíti a profilt és a forrásfejezetek SHA-256 lenyomatát, a `check.sh` pedig a fejezetekben közölt build- és tesztparancsokat futtatja a `lab-gcc` profilban.

A konténer reprodukciós eszköz, nem önmagában bizonyíték. Az image tartalomazonosítója hasznosabb, mint egy változó címke; ugyanakkor a konténer továbbra is a gazdagép kernelét használja. A kernelverziót, CPU-környezetet és esetleges emulációt külön dokumentáljuk.

1.3.2 Az eszközök szerepe és azonosítása

A `gcc` és `clang` itt **fordítómeghajtó** (*compiler driver*): nemcsak fordítást, hanem linkelést is vezérelhet.

Eszköz	Szerep	Fontos korlát
`gcc`, `clang`	C fordítása, linkelés vezérlése	Az alapértelmezett nyelvi módra nem támaszkodunk
`g++`, `clang++`	C++ fordítása, C++ könyvtárak linkelése	C++ program végső linkeléséhez ezeket használjuk
`as`, `ld`, `readelf`, `nm`, `objdump`	Binutils-eszközök: assembly, linkelés, binárisvizsgálat	Clang használhat beépített assemblert
`gdb`	Folyamatállapot és forrás–gépi kód kapcsolat vizsgálata	Optimalizálás mellett változók eltűnhetnek
`strace`	Linux rendszerhívások megfigyelése	Nem lát minden könyvtári vagy nyelvi műveletet

A PATH-ban található `ld` verziója még nem bizonyítja, hogy a fordító valóban azt használta. A driver `-###` kapcsolójával megvizsgálható a tervezett alparancssor; ez nem hajtja végre a fordítást.

1.3.2.1 Könyvtárstruktúra és naplózás

A következő parancsok **Bash alatt, a tanuló külön laborjában** használhatók. Feltételük, hogy a PATH már a választott eszközökre mutasson.

```
set -euo pipefail

export LC_ALL=C
export TZ=UTC

mkdir -p src negative/compile-fail negative/ub logs
```

```
mkdir -p build/{gcc,clang}/{debug,sanitize}

record() {
    printf '$' | tee -a logs/session.log
    printf ' %q' "$@" | tee -a logs/session.log
    printf '\n' | tee -a logs/session.log
    "$@" 2>&1 | tee -a logs/session.log
}

for tool in gcc g++ as ld gdb strace; do       # a lab-gcc profil kötelező
 ↪  eszközei
    record command -v "$tool"
    record "$tool" --version
done
for tool in clang clang++; do                   # lab-clang: csak ha telepítve
 ↪  van
    if command -v "$tool" >/dev/null; then
        record "$tool" --version
    else
        printf '# %s nem érhető el (lab-clang profil nélkül)\n' "$tool" \
            | tee -a logs/session.log
    fi
done

record uname -m
record uname -r
record gcc -dumpmachine
record getconf GNU_LIBC_VERSION
record g++ -print-file-name=libstdc++.so
record locale
```

A `pipefail` azért szükséges, hogy a `tee` sikeres naplózása ne fedje el a megelőző parancs hibáját. A `set -e` miatt a váratlanul sikertelen parancs megszakítja ezt a laborfolyamatot. A szándékosan hibás vagy opcionális lépéseket ezért `if` feltételébe tesszük: egy elvárt hiba státuszát `if parancs; then status=0; else status=$?; fi` alakban kérdezzük le, és utána ellenőrizzük, hogy valóban nem nulla. Ezt a mintát használja a könyv minden negatív futási próbája.

A `getconf` itt glibc-alapú, natív környezet azonosítására szolgál, nem keresztfordítási célkönyvtár vizsgálatára. A `libstdc++.so` útvonala pedig önmagában nem pontos verzióazonosító: a csomagjegyzéket és a tényleges fájl ellenőrző összegét is érdemes menteni.

A forrásokat UTF-8-ként tároljuk. Az `LC_ALL=C` a diagnosztika és rendezés kiszámíthatóságát szolgálja; **nem állít be UTF-8 locale-t**. Ha egy későbbi kísérlet Unicode-szövegkezelést vizsgál, telepített UTF-8 locale-t kell választani és külön rögzíteni. Az alábbi példák csak ASCII-karaktereket használnak.

1.3.3 Warning-, debug- és sanitizerprofil

GCC/Clang-beállítás: a C alapmód `-std=c17`, a C++ alapmód `-std=c++20`. Ezekhez rendeljük:

```
warnings=(-Wall -Wextra -Wpedantic)
debug=(-g -O1 -fno-omit-frame-pointer)
```

A `-Wall` neve ellenére nem kapcsol be minden figyelmeztetést. A `-Wextra` további csoportokat engedélyez, a `-Wpedantic` pedig a választott ISO módhoz kapcsolódó diagnosztikát kér. Egyik sem teljes szabványossági vagy helyességi bizonyítás.

A `-Wconversion` hasznos lehet például értéktartományt szűkítő implicit konverziók feltárására. Külön auditprofilként adjuk hozzá, és minden találatot értelmezzünk; a puszta cast nem bizonyítja a konverzió helyességét.

A debugprofilban:

- a `-g` hibakeresési információt kér;
- az `-O1` mérsékelt optimalizálást engedélyez;
- a `-fno-omit-frame-pointer` segítheti a veremvisszakövetést.

Ettől még nem garantált minden forrásváltozó megfigyelhetősége vagy minden függvényben külön veremkeret. Optimalizálatlan összehasonlításhoz külön `-O0` build készíthető.

1.3.4 Kidolgozott példa két nyelven, két fordítóval

A programok a típusméreteket **C bájtban** írják ki. ISO C/C++ szerint a `sizeof` egysége a karaktertárolás egysége; bitméretét a `CHAR_BIT` adja meg.

A `src/probe.c` teljes program, minimum C17 követelménnyel:

```c
#include <limits.h>
#include <stdio.h>

int main(void)
{
    if (printf("CHAR_BIT=%d int=%zu long=%zu pointer=%zu\n",
               CHAR_BIT, sizeof(int), sizeof(long),
               sizeof(void *)) < 0) {
        return 1;
    }

    return fflush(stdout) == EOF;
}
```

A `src/probe.cpp` teljes program, a laborban C++20 móddal:

```cpp
#include <climits>
#include <iostream>

int main()
{
    std::cout << "CHAR_BIT=" << CHAR_BIT
              << " int=" << sizeof(int)
              << " long=" << sizeof(long)
              << " pointer=" << sizeof(void*)
              << '\n' << std::flush;

    return std::cout ? 0 : 1;
}
```

A példák nem hoznak létre alkalmazásszintű borrowed pointert: a `sizeof(void*)` csak egy típus méretét kérdezi le.

A korábbi Bash-munkamenetben:

```
record gcc -std=c17 "${warnings[@]}" "${debug[@]}" \
    src/probe.c -o build/gcc/debug/probe-c

record g++ -std=c++20 "${warnings[@]}" "${debug[@]}" \
    src/probe.cpp -o build/gcc/debug/probe-cpp

record clang -std=c17 "${warnings[@]}" "${debug[@]}" \
    src/probe.c -o build/clang/debug/probe-c

record clang++ -std=c++20 -stdlib=libstdc++ \
    "${warnings[@]}" "${debug[@]}" \
    src/probe.cpp -o build/clang/debug/probe-cpp

record clang++ -std=c++20 -stdlib=libstdc++ -### \
    src/probe.cpp -o build/clang/debug/probe-cpp
```

A `-stdlib=libstdc++` a könyvtárcsaládot választja ki, nem annak pontos kiadását. Több GCC-telepítés esetén a Clang toolchain-keresését is rögzíteni kell; erre szolgálhat a `--gcc-toolchain` kapcsoló. A futáskor betöltött könyvtárat a loader környezete is befolyásolja.

ABI-függő várt adatok, nem mért eredmények:

Mennyiség	x86-64 System V LP64	AArch64 Linux LP64
`CHAR_BIT`	8	8
`sizeof(int)`	4	4
`sizeof(long)`	8	8
`sizeof(void*)`	8	8

Az egyező méretek nem jelentenek bináris kompatibilitást. Az AArch64-kísérlethez külön natív profil a legegyszerűbb. Keresztfordításnál a célkönyvtárakat és a futtatási környezetet is külön kell azonosítani; a `uname -m` önmagában csak a futó környezet egy adatát adja meg.

1.3.5 Elkülönített hibapélda és diagnosztika

A helyes források a `src`, a szándékosan hibás példák a `negative` könyvtárba kerülnek. A fordításkor elutasítandó esetek és a futás közbeni UB külön csoportot alkotnak.

Szándékosan hibás, C17-es tanítási példa, `negative/ub/signed_overflow.c`:

```c
#include <limits.h>

int increment(int value)
{
    return value + 1;
}

int main(void)
```

```
{
    return increment(INT_MAX);
}
```

ISO C17-szabály: a signed összeadás eredményének reprezentálhatónak kell lennie a megfelelő típusban. Itt

$$\text{INT_MAX} + 1 > \text{INT_MAX},$$

ezért az összeadás nem definiált viselkedést okoz. Nincs garantált programkimenet.

Külön sanitizerbuild:

```
record gcc -std=c17 "${warnings[@]}" "${debug[@]}" \
    -fsanitize=address,undefined -fno-sanitize-recover=all \
    negative/ub/signed_overflow.c \
    -o build/gcc/sanitize/signed-overflow

record clang -std=c17 "${warnings[@]}" "${debug[@]}" \
    -fsanitize=address,undefined -fno-sanitize-recover=all \
    negative/ub/signed_overflow.c \
    -o build/clang/sanitize/signed-overflow
```

Ehhez a választott fordítóhoz tartozó sanitizer-runtime telepítése is szükséges. A kapcsolók ebben az egyutas fordításban a fordítási és linkelési lépéshez is eljutnak.

A későbbi kézi futtatáskor UBSan-diagnosztika várható, de annak pontos szövege és a kilépési státusz rögzítendő megfigyelés. Az ASan jelenléte nem jelenti, hogy ezt az aritmetikai hibát az ASan keresi. A sanitizer hibamentes futása pedig csak a ténylegesen bejárt végrehajtásokra ad korlátozott bizonyítékot.

1.3.6 Feladatok

1. **Magyarázó:** Miért elégtelen laborazonosító a „Linux, GCC 13, konténerben” leírás? Nevezz meg öt hiányzó adatot.
2. **Kódolási:** Bővítsd a C próbaprogramot a `sizeof(short)` és `sizeof(long long)` kiírásával. Fordítsd mindkét fordítóval, majd külön `-Wconversion` mellett.
3. **Diagnosztikai:** Egy fordítási parancs `tee` használata mellett sikeresnek látszik, pedig a forrás szintaktikailag hibás. Hogyan különíted el a naplózási hibát a várt fordítási kudarctól? Mi legyen a negatív teszt sikerfeltétele?

1.3.7 Megoldások

1.3.7.1 Magyarázó feladat

Hiányozhat a teljes fordító- és csomagverzió, libc, linker, cél-ABI, nyelvi mód, kapcsolók, image-azonosító vagy forrásverzió. Az ellenőrzési feltétel: másik mérnöknek ne kelljen találgatnia ezeket a környezet összeállításakor.

1.3.7.2 Kódolási feladat

A `printf` formátumába két új `%zu`, argumentumlistájába `sizeof(short)` és `sizeof(long long)` kerül. Ezek eredménye `size_t`, ezért megfelelő a `%zu`. A választott két ABI-profilban rendre 2 és 8 C bájt várható. Ellenőrzés: mindkét fordítás sikeres, a diagnosztika áttekintett, az esetleges eltérés dokumentált.

1.3.7.3 Diagnosztikai feladat

Alapértelmezésben a pipeline státusza a végső parancsé lehet, így a sikeres `tee` elfedheti a fordító hibáját. Bash alatt a `pipefail`, illetve a `PIPESTATUS` közvetlen vizsgálata segít.

A negatív fordítási teszt sikere az **elvárt okból bekövetkező elutasítás**. Hiányzó header vagy elérhetetlen fordító nem megfelelő bizonyíték. Előbb ellenőrizni kell, hogy ugyanazzal a profillal a helyes alapminta lefordul.

1.3.8 Összefoglaló

A megbízható labor külön kezeli a forrást, a toolchaint, a cél-ABI-t és a futási környezetet. Explicit nyelvi módot, két fordítót, elkülönített buildkönyvtárakat és naplózott parancsokat használ. A warningok és sanitizerek értékes megfigyelési eszközök, de nem helyettesítik a program szerződésének ellenőrzését.

1.4 A szükséges C szintaxis tömör áttekintése

C-ben a rövid forráskód mögött is pontos típus- és élettartam-szabályok állnak. Most nem a teljes nyelvtant tanuljuk meg: azt a szintaktikai készletet rögzítjük, amellyel a következő memória- és típuslaborok programjai olvashatók.

Az előfeltétel a Reprodukálható fordítási és diagnosztikai labor: különösen az explicit nyelvi mód és a fordítási diagnosztika használata. Az alábbi nyelvi magyarázatok **ISO C17** szabályokra épülnek, nem C++-szabályokra.

1.4.1 Deklaráció, definíció, inicializálás és értékadás

A **deklaráció** (*declaration*) egy azonosítóról közöl információt a fordítóval: például megadja egy változó típusát vagy egy függvény paramétereit és visszatérési típusát.

A **definíció** (*definition*) olyan deklaráció, amely az objektumot vagy függvényt meghatározza: objektumnál tárolót hoz létre, függvénynél megadja a törzset. Nem minden deklaráció definíció.

Az alábbi **C17-részletben** az első két sor fájlszinten, az utolsó három egy függvény törzsében értendő:

```c
extern int status_code;        /* Deklaráció, nem definíció. */
int twice(int value);         /* Függvénydeklaráció, prototípus. */

/* Függvénytörzsben: */
int pending = 3;               /* Definíció és inicializálás. */
pending = 5;                   /* Értékadó kifejezésutasítás. */
int completed;                /* Definíció, inicializáló nélkül. */
```

Az **inicializálás** (*initialization*) az objektum kezdeti értékének meghatározása. Az **értékadás** (*assignment*) egy már létező objektumban tárolt értéket módosít. Bár mindkettőben szerepelhet `=`, nyelvtanilag és szemantikailag külön műveletek.

A `completed` automatikus lokális változó nem kap automatikusan nullát. Értéke kezdetben meghatározatlan (*indeterminate*); ne olvasd, amíg megfelelő értéket nem tároltál benne. Ez nem általános állítás minden inicializáló nélküli objektumról: más tárolási idejű objektumokra később más szabályokat ismerünk meg.

Az **utasítás** (*statement*) végrehajtási lépést ír le. A `pending = 5;` kifejezésutasítás, a `return 0;` visszatérési utasítás, a `{ ... }` pedig összetett utasítás, más néven blokk. A deklaráció nem kifejezésutasítás, még ha gyakran ugyanúgy pontosvessző zárja is.

1.4.2 Operátorok, precedencia és zárójelezés

Az operátorok operandusokon végeznek műveletet. Az operandusok típusa számít: két `int` érték osztásakor például `7 / 2` eredménye `3`, nem `3.5`.

A következő táblázat részleges precedenciatérkép, a szorosabban kötő műveletektől lefelé:

Operátorok	Jelentés
`!x`, `-x`, `++x`	Logikai negálás, előjelváltás, prefix növelés
`*`, `/`, `%`	Szorzás, osztás, egész maradék
`+`, `-`	Összeadás, kivonás
`<`, `<=`, `>`, `>=`	Rendezési összehasonlítás
`==`, `!=`	Egyenlőség, különbözőség
`&&`	Logikai ÉS
`||`	Logikai VAGY
`=`, `+=`, `-=`	Értékadás, összetett értékadás

Ezért:

$$2 + 3 \cdot 4 = 2 + 12 = 14,$$

míg a C-beli `(2 + 3) * 4` eredménye `20`.

A zárójel csoportosítást fejez ki, de **nem általános kiértékelésisorrend-vezérlő eszköz**. A `left() + right()` esetén a `+` nem írja elő, melyik függvényhívás történjen előbb.

A `&&` és `||` viszont rövidzáras (*short-circuit*) operátor. A jobb operandus csak akkor értékelődik ki, ha az eredmény eldöntéséhez szükséges, és ilyenkor a bal operandus kiértékelése megelőzi azt. Például:

```
/* C17-részlet; divisor inicializált int. */
if (divisor != 0 && 12 / divisor > 2) {
    /* Az osztás csak nem nulla osztóval történik meg. */
}
```

C-ben a nulla hamis, a nem nulla igaz feltételnek számít. Az összehasonlító és logikai operátorok `int` típusú `0` vagy `1` eredményt adnak.

Két gyakori tévedés:

- Az `=` értékadás, a `==` összehasonlítás. Az `if (ready = 1)` szabályos lehet, de többnyire logikai hiba.
- A `0 <= x <= limit` nem matematikai láncolt összehasonlítás. Helyesen: `0 <= x && x <= limit`.

1.4.3 Blokkok, lokális változók és vezérlés

A kapcsos zárójelek blokkot határolnak. Egy szokásos lokális változó neve a deklarációjától a blokk végéig használható; ezt a név **hatókörének** (*scope*) nevezzük. A hatókör nem azonos az objektum élettartamával, még ha egyszerű példákban határaik szorosan kapcsolódnak is.

C17-ben a deklarációk és utasítások keverhetők. Célszerű a változót az első szükséges használata közelében, lehetőleg inicializálva definiálni.

A `/* ... */` komment több soros lehet, de nem ágyazható önmagába. A `//` komment a sor végéig tart.

1.4.3.1 A vezérlési szerkezetek rövid referenciája

Az alábbi **C17-utasításrészletekben** a használt változók megfelelően deklarált, inicializált egész változók.

Szerkezet	Példa és működés
`if–else`	`if (ready != 0) { count = 1; } else { count = 0; }` – a feltétel alapján választ ágat
`for`	`for (int i = 0; i < 3; ++i) { total += i; }` – inicializálás, feltétel, törzs, léptetés
`while`	`while (remaining > 0) { --remaining; }` – elöl tesztel, akár egyszer sem fut
`do–while`	`do { ++attempts; } while (attempts < 3);` – hátul tesztel, legalább egyszer fut

A `for` inicializáló része egyszer hajtódik végre. A feltételt minden iteráció előtt ellenőrzi; a léptetés a törzs után következik. A példabeli `i` a cikluson kívül már nem használható.

A `switch` egész típusú kifejezés alapján választ. A `case` címkék egész konstanskifejezéseket tartalmaznak; a `default` a fel nem sorolt értékek ága. A kiválasztott címkétől a végrehajtás továbbhalad, ezért a következő `case` **nem zárja le** automatikusan az előző ágat.

A `break` a legbelső körülvevő ciklusból vagy `switch`-ből lép ki. A `continue` a legbelső ciklus következő iterációjához vezet; `for` esetén előbb a léptetés történik meg. A `return` az egész függvényből tér vissza.

Használj kapcsos zárójeleket egyutasításos ágaknál is. Az `if (ready);` utáni pontosvessző üres utasítás: az utána írt blokk már nem feltételes.

1.4.4 Teljes program és közös kilépési ág

A következő önálló program minimumkövetelménye **ISO C17, hosted környezet**. A GCC-parancs profilja a központi profiltábla `lab-gcc` profilja, a Clang-parancsé a `lab-clang`.

A `syntax.c` tartalma:

```
#include <stdio.h>
#include <stdlib.h>

int contribution(int state)
{
    switch (state) {
    case 0:
        return 0;
    case 1:
        return 2;
    default:
        return 1;
    }
}

int main(void)
{
    int result = EXIT_FAILURE;
    int total = 0;

    for (int state = 0; state < 3; ++state) {
        total += contribution(state);
    }

    if (printf("total=%d\n", total) < 0) {
        goto cleanup;
    }

    if (fflush(stdout) == EOF) {
        goto cleanup;
    }

    result = EXIT_SUCCESS;

cleanup:
    return result;
}
```

A `main(void)` paraméter nélküli, `int` visszatérési típusú belépési függvény. A `<stdio.h>` deklarálja a használt I/O-függvényeket, és elérhetővé teszi többek között a `stdout` és `EOF` neveket. A `<stdlib.h>` adja az `EXIT_SUCCESS` és `EXIT_FAILURE` makrókat.

A `contribution` definíciója megelőzi a hívást, ezért itt külön előzetes prototípusra nincs szükség. A `switch` ágai `return` utasítással fejeződnek be, így nem kell utánuk `break`.

A ciklus közbülső eredményei:

`state`	Visszakapott érték	`total` a hozzáadás után
0	0	0
1	2	2
2	1	3

Sikeres kimeneti műveletek mellett az elvárt sor `total=3`. A `%d` egy `int` érték formázására szolgál, a `\n` újsorjel.

A `printf` negatív eredménye hibát jelez. Mivel a standard kimenet pufferelt lehet, a program a kikényszerített kiírás, az `fflush` eredményét is ellenőrzi. Ezek a visszatérési értékek, nem a képernyőn látott szöveg döntik el a hibakimenetet.

1.4.4.1 A `goto` szerepe

A `goto cleanup;` ugyanazon függvény `cleanup` címkéjére ugrik. Itt még nincs saját erőforrás, amelyet fel kellene szabadítani: ez a későbbi közös takarítási ág előképe. A `stdout` a standard könyvtár által kezelt stream; a program nem vesz át külön tulajdonjogot felette.

Mindkét hibaág megőrzi az előre beállított `EXIT_FAILURE` értéket. A sikerértéket csak az összes ellenőrzés után állítjuk be. A címke előtt álló `result` minden odavezető úton inicializált.

1.4.4.2 Fordítás

A profilnak megfelelő fordítóval:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g syntax.c -o syntax-gcc
clang -std=c17 -Wall -Wextra -Wpedantic -O0 -g syntax.c -o syntax-clang
```

Ezek GCC/Clang-parancsok, nem ISO C-szintaxis. **Buildjegyzőkönyv:** a GCC-parancs a `lab-gcc` profilban figyelmeztetés nélkül fordított, a program `total=3` kimenettel, `0` kóddal futott; a Clang-sort nem ellenőriztük. A figyelmeztetésmentes fordítás önmagában egyébként sem bizonyítaná a helyességet.

1.4.5 Feladatok

1. **Fogalmi:** Osztályozd a következő, függvénytörzsbe szánt sorokat. Melyik deklaráció, definíció, inicializálás vagy értékadás?

   ```
   int quota;
   quota = 4;
   int used = 0;
   int check(int value);
   ```

2. **Kódolási:** Írj `int sum_even(int limit)` függvényt, amely a `[0, limit]` intervallum páros egészeit összeadja. Csak `0 <= limit && limit <= 100` fogadható el; más bemenetre térjen vissza `-1` értékkel.

3. **Diagnosztikai:** Az alábbi C17-részletben a kívánt eredmény kizárólag az `idle` sor. Miért nem ezt kapjuk? Keress két külön hibát. Feltételezd a `<stdio.h>` beillesztését és a sikeres I/O-t.

```c
int ready = 0;
int mode = 1;

if (ready = 1) {
    mode = 2;
}

switch (mode) {
case 1:
    puts("idle");
case 2:
    puts("active");
    break;
default:
    puts("unknown");
    break;
}
```

1.4.6 Megoldások

1.4.6.1 Fogalmi megoldás

Az `int quota;` deklaráció és definíció, explicit inicializálás nélkül. A `quota = 4;` értékadás, kifejezésutasítás. Az `int used = 0;` deklaráció, definíció és inicializálás. Az `int check(int value);` függvénydeklaráció, de nem függvénydefiníció.

Ellenőrzési feltétel: a `quota` első olvasása csak az értékadás után történhet.

1.4.6.2 Kódolási megoldás

Az alábbi C17-függvénydefiníció a teljes program `main` függvénye elé illeszthető:

```c
int sum_even(int limit)
{
    if (limit < 0 || limit > 100) {
        return -1;
    }

    int total = 0;
    for (int value = 0; value <= limit; ++value) {
        if (value % 2 == 0) {
            total += value;
        }
    }
    return total;
}
```

Bizonyításvázlat: minden iteráció elején `total` a már megvizsgált páros értékek összege. A feltételes hozzáadás ezt megőrzi. Kilépéskor minden megengedett értéket megvizsgáltunk. A legnagyobb összeg:

$$2 + 4 + \cdots + 100 = 2(1 + 2 + \cdots + 50) = 2 \cdot \frac{50 \cdot 51}{2} = 2550.$$

Ez a C17 által minimálisan megkövetelt `int`-tartományba is belefér.

Ellenőrzési feltétel: a `-1`, `0`, `5`, `100`, `101` bemenetekre rendre `-1`, `0`, `6`, `2550`, `-1` az elvárt eredmény.

1.4.6.3 Diagnosztikai megoldás

Az `if (ready = 1)` egyet tárol `ready` változóba, és igaz feltételt eredményez. Emiatt `mode` kettő lesz, tehát az eredeti részlet `active` sort ír.

Az első javítás `if (ready == 1)`. Ezután viszont az `idle` után végrehajtódik az `active` kiírása is, mert hiányzik a `break` az első ágból.

Ellenőrzési feltétel: mindkét javítás után `ready` nulla, `mode` egy marad, és sikeres I/O mellett csak az `idle` sor jelenik meg.

1.4.7 Összefoglaló

C-kód olvasásakor különítsd el a nevek deklarálását, az objektumok definícióját, az inicializálást és a későbbi értékadást. A precedencia csoportosítást, nem általános végrehajtási sorrendet határoz meg. A blokkok a lokális nevek hatókörét rendezik; a vezérlési szerkezetek pedig azt, mely utasítások hajtódnak végre. A közös kilépési ág már most segít láthatóvá tenni a siker és a hiba szerződését.

1.5 Első megfigyelési ciklus a forrástól a futásig

A natív program megértéséhez nem elég látni, hogy „a fordító lefutott”. Külön kell megfigyelni a forrás feldolgozását, az objektumfájl létrejöttét, a linkelést és a folyamat futását. Ugyanaz a tünet — például hogy nincs kiírt üzenet — több különböző szakasz hibájából is származhat.

A szükséges C szintaxis tömör áttekintése alapján már olvasható egy rövid `main` függvény. Most ennek útját követjük végig, minden lépéshez megfigyelhető terméket vagy hibajelzést rendelve.

1.5.1 A megfigyelés feltételei

A példák minimumkövetelménye **ISO C17**, hosted környezetben. A választott laborprofil Linux, x86-64 System V ABI, LP64 adatmodell, a `lab-gcc` profil (profiltábla). Ebben a profilban egy C bájt nyolc bit. A shellparancsokat Bash alatt értelmezzük.

Ez választott, nem itt megmért konfiguráció. Disztribúciós javítások és fordítóbeállítások is módosíthatják a részleteket. A saját laborodhoz rögzítsd a teljes verziókimeneteket:

```
uname -srmo
gcc --version
gcc -dumpmachine
clang --version
getconf GNU_LIBC_VERSION
readelf --version
file --version
strace --version
```

A fejezetben az **elvárt eredmény** a program szerződéséből vagy a megadott eszközműveletből következik. **Ténylegesen mért eredménynek** csak a saját futtatásod rögzített kimenetét nevezd; az alábbi példák nem dokumentált tesztfuttatások.

1.5.2 A teljes program és első buildje

A `hello.c` teljes tartalma:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    if (puts("hello") == EOF || fflush(stdout) == EOF) {
        (void)fputs("hello: stdout error\n", stderr);
        return EXIT_FAILURE;
    }

    if (fputs("hello: done\n", stderr) == EOF ||
        fflush(stderr) == EOF) {
        return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}
```

ISO C17: a `puts` a szöveg után újsort ír a standard kimenetre. A `fputs` nem egészíti ki újsorral a szöveget. Mindkettő hibát jelezhet; a példában az `EOF` értékkel való összehasonlítás ezt vizsgálja.

A `stdout` és `stderr` könyvtári streamek, nem közvetlenül Linux file descriptorok. Életciklusukat itt a C futtatókörnyezet kezeli; a program nem zárja le őket kézzel. A kilépés utáni használatuk nem értelmezhető ebben a folyamatban.

Miért szerepel külön `fflush`? A sikeres könyvtári írás jelentheti azt is, hogy az adat csak egy felhasználói térben lévő pufferbe került. A kiürítés később hibázhat. A program még a sikeres visszatérés előtt ellenőrzi ezt. Ha maga a hibaüzenet kiírása is sikertelen, a hibastátusz marad az egyetlen jelzés.

Fordítás és linkelés egyetlen driverhívással:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g hello.c -o hello
```

A `gcc` itt fordítási lépéseket és linkelést is vezérel. A `-O0` és `-g` megfigyeléshez választott beállítás; nem helyességi garancia. A warningkapcsolók hasznos ellenőrzéseket kérnek, de nem bizonyítják a hibamentességet.

1.5.3 A köztes termékek előállítása és vizsgálata

Ugyanezt az utat állítsuk meg három ponton:

```
gcc -std=c17 -E hello.c -o hello.i
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -S hello.c -o hello.s
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g -c hello.c -o hello.o

head -n 20 hello.i
head -n 60 hello.s
```

A három parancs itt külön-külön az eredeti forrásból indul; nem az előző parancs kimenetét fogyasztja.

Megállási pont	Termék	Mit keress benne?
`-E`	`hello.i`, előfeldolgozott szöveg	Beillesztett headerdeklarációk, kibontott makrók, forráspozíció-jelölések
`-S`	`hello.s`, assemblyszöveg	Címkék, direktívák és gépi utasítások szöveges alakja
`-c`	`hello.o`, objektumfájl	Gépi kód, szimbólumok, relokációs és debugadatok

A `.i` fájl elején valószínűleg nem a `main` látszik, hanem a headerekből származó tartalom. Az assembly első sorai sem feltétlenül utasítások: az assemblernek szóló direktívák is szerepelnek benne.

Linux/ELF és GNU Binutils: az objektumfájlt most a platform eszközeivel vizsgáljuk:

```
file hello.o
readelf -h hello.o
nm hello.o
nm -u hello.o
objdump -dr hello.o
```

A `file` gyors típusazonosítást ad. A `readelf -h` az ELF fejlécét mutatja; ennél a profilnál relokálható objektumot, vagyis `REL` típust várunk.

Az `nm` szimbólumokat sorol fel. A `main` az objektumban definiált függvény; a szokásos megjelenítésben `T` jelölést kap. Az `U` olyan szimbólumot jelöl, amelyre az objektum hivatkozik, de amelyet nem itt definiáltunk. Ez önmagában **nem hiba**: a külső függvények feloldása a linkelési út része.

Az `objdump -dr` gépi kódot fordít vissza assemblyalakra, és megmutatja a relokációkat is. A relokáció leegyszerűsítve olyan információ, amellyel a linker egy címhez vagy szimbólumhoz kötött hivatkozást rendezhet. Az utasítások részletes értelmezése most nem cél.

A forrásbeli függvénynevek és az objektum külső hivatkozásai nem feltétlenül egyeznek egy az egyben: a fordító könyvtári műveleteket is átalakíthat. Például egy `fputs` hívás helyén más könyvtári hivatkozás jelenhet meg.

Végül linkeljünk az objektumból:

```
gcc hello.o -o hello
file hello
readelf -h hello
```

A végrehajtható ELF típusa lehet `EXEC`, illetve pozíciófüggetlen végrehajtható állomány, PIE esetén `DYN`. A `DYN` tehát nem jelenti automatikusan azt, hogy kizárólag megosztott könyvtárat vizsgálunk.

1.5.4 Standard kimenetek és kilépési státusz

A program két külön csatornát használ: a `stdout` a normál eredményé, a `stderr` a diagnosztikáé. Egy `stderr`-üzenet önmagában nem jelent sikertelen futást.

```
if ./hello >stdout.txt 2>stderr.txt; then status=0; else status=$?; fi
printf 'exit=%d\n' "$status"
cat stdout.txt
cat stderr.txt
```

A státuszt `if ...; then status=0; else status=$?; fi` alakban kérdezzük le. Az 1.3 laborban ajánlott `set -euo pipefail` beállítás mellett egy sima `./hello ...` sor sikertelen futáskor az egész szkriptet leállítaná, és a `status=$?` sor sosem futna le; az `if` feltételében álló parancs hibája viszont nem váltja ki az `errexit`-et.

Elvárt eredmény sikeres I/O esetén:

Megfigyelés	Elvárt érték
`stdout.txt`	`hello`, majd újsor
`stderr.txt`	`hello: done`, majd újsor
Bash kilépési státusz	`0`

ISO C17: az `EXIT_SUCCESS` sikeres, az `EXIT_FAILURE` sikertelen befejezést jelez a környezetnek. **A választott Linux/glibc profilban** ezek értéke rendre 0 és 1; a hordozható C programnak nem kell ezt a számpárt beégetnie.

A `$?` értékét közvetlenül a vizsgált parancs után kell elmenteni. Egy közbeiktatott `cat` után már annak státuszát olvasnád. Pipeline esetén pedig Bash alapbeállításban az utolsó parancs státuszát adja vissza, nem feltétlenül a vizsgált programét.

1.5.5 Könyvtári függvény vagy rendszerhívás?

A rendszerhívás (*system call*) a kernel szolgáltatásának belépési határa. A C könyvtári függvény ezzel szemben nyelvi könyvtári interfész: végezhet munkát kizárólag felhasználói térben, vagy kezdeményezhet rendszerhívásokat.

Linux, strace: az írásokat és a folyamatlezárást így figyelheted:

```
strace -o trace.txt -e trace=write,writev,exit_group \
    ./hello >stdout.txt 2>stderr.txt
cat trace.txt
```

Az `-o` külön fájlba teszi a trace-t, így az nem keveredik a program diagnosztikájával. A szűrés miatt ez nem a teljes rendszerhívás-történet.

A választott profilon tipikusan az 1-es és 2-es descriptorra irányuló írásokat, végül sikeres lezárást várunk. A pontos sorokat azonban a tényleges futtatásból kell leolvasni.

Nem `puts` rendszerhívást keresünk: ilyen Linux-rendszerhívás nincs. Több könyvtári írás összevonható egy kernelhívásba, egy nagyobb írás pedig több kernelhívást is igényelhet. A megfigyelt `write` sikere ráadásul nem bizonyítja, hogy az adat tartós háttértárra került.

1.5.6 A hiba helyének meghatározása

A diagnózis első kérdése: **melyik szakasz nem teljesítette a szerződését?**

Szakasz	Példa	Következmény
Fordítás	Hiányzó pontosvessző	Nem keletkezik használható új objektum
Linkelés	Deklarált, de fel nem oldható függvény	Nem keletkezik használható új végrehajtható állomány
Futás	A kimenet írása hibázik	A folyamat elindul, de hibastátusszal zárulhat

Egy külön forrásmásolat `main` törzsébe tett `int unused = 0;` a megadott warningprofilban várhatóan figyelmeztetést vált ki. Ez nem ugyanaz, mint egy fordítási hiba: `-Werror` nélkül a build ettől még sikerülhet.

Gyakori csapda a régi bináris futtatása sikertelen fordítás után. Egymáshoz kötött kézi próbánál használd az `&&` operátort:

```
gcc -std=c17 -Wall -Wextra -Wpedantic hello.c -o hello && ./hello
```

Így a futtatás csak sikeres build után következik.

1.5.7 Feladatok

1. **Magyarázó:** Miért nem bizonyít linkelési hibát az `nm hello.o` kimenetében egy `U` jelölés? Miért nem futtatható közvetlenül a `hello.o`?
2. **Kódolási:** Bővítsd a programot úgy, hogy a `stdout` második sora `ready` legyen. Mindkét írást és a kiürítést ellenőrizd. Linuxon vizsgáld meg a futást `/dev/full` kimenettel is.
3. **Diagnosztikai:** Az alábbi teljes, ISO C17 forrást mentsd `missing.c` néven. Jósolj eredményt külön az objektumképzésre és a linkelésre.

```
void missing(void);

int main(void)
{
    missing();
    return 0;
}
```

1.5.8 Megoldások

1.5.8.1 Külső hivatkozás az objektumban

Az `U` feloldandó külső hivatkozás. Akkor okoz linkelési hibát, ha az adott linkelés nem talál hozzá megfelelő definíciót. Az objektum még nem kész folyamatkép: hiányzik belőle a végrehajtható programhoz szükséges összerendezés és indítási infrastruktúra.

Ellenőrzési feltétel: a `hello.o` tartalmazhat `U` szimbólumokat, miközben a `gcc hello.o -o hello` sikeresen befejeződik.

1.5.8.2 Két sor és írási hiba

Az eredeti program első `if` blokkjának helyére kerülő C17-részlet:

```
if (puts("hello") == EOF ||
    puts("ready") == EOF ||
    fflush(stdout) == EOF) {
    (void)fputs("hello: stdout error\n", stderr);
    return EXIT_FAILURE;
}
```

A rövidzáras kiértékelés miatt az első észlelt hiba után hibakezelés következik. Fordítsd újra, majd:

```
if ./hello >/dev/full 2>error.txt; then status=0; else status=$?; fi
printf 'exit=%d\n' "$status"
cat error.txt
test "$status" -ne 0                  # váratlan siker: a negatív teszt bukjon
grep -q 'stdout error' error.txt      # a diagnosztika megjelent
```

Linux-specifikus előfeltétel: a `/dev/full` elérhető, és az átirányítás sikerül. Az eszköz az írást hibával utasítja el. Elvárt ellenőrzés: normál kimeneten két sor és nulla státusz; `/dev/full` esetén nem nulla státusz, írható `stderr` mellett hibaüzenet. A részlet `set -euo pipefail` mellett is végigfut; ha a program váratlanul sikeresen lépne ki, a `test` sor megbuktatja a szkriptet.

1.5.8.3 A deklaráció nem definíció

```
gcc -std=c17 -Wall -Wextra -Wpedantic -c missing.c -o missing.o
nm -u missing.o
gcc missing.o -o missing
```

Az objektumképzés elvártan sikerül: a deklaráció alapján a hívás ellenőrizhető. A linkelés viszont sikertelen, mert nincs `missing`-definíció.

Javításként ugyanahhoz a forráshoz hozzáadható ez a C17-részlet:

```
void missing(void)
{
}
```

Újrafordítás után a linkelésnek sikerülnie kell. A pontos diagnosztikai szöveg eszköz- és verziófüggő.

1.5.9 Rövid összefoglaló

A megfigyelési ciklus lánca: forrás, köztes termékek, objektum, linkelt állomány, folyamat, kimenetek és státusz. Az objektum szimbólumai, a kernel trace-e és a shell státusza különböző kérdésekre válaszolnak. Megbízható következtetéshez mindig nevezd meg a szakaszt, a vizsgált szerződést és azt, hogy elvárt vagy ténylegesen mért eredményről beszélsz.

2 A fordítás és a preprocessor C-ben

2.1 Fordítási fázisok és fordítási egységek

A natív build hibáinak értelmezéséhez két külön modellt kell használnunk. Az **ISO C fordítási fázisai** azt határozzák meg, hogyan kell a forrásprogramot értelmezni. A **toolchain lépései** ezzel szemben konkrét eszközöket, folyamatokat és fájlokat jelentenek: preprocessort, fordítót, assemblert és linkert. A két felosztás összefügg, de nem azonos.

Az Első megfigyelési ciklus a forrástól a futásig részben megismert megállítási pontokra építünk. Most azt vizsgáljuk meg, pontosan mi jut el az egyes pontokig, és milyen információ hiányzik még ott. Ez különösen fontos, amikor egy CI-buildben a „compilation failed” üzenet valójában headerkeresési, típusellenőrzési vagy linkelési problémát takar.

2.1.1 A szabvány szerinti fordítási fázisok

ISO C17: a fordítás nyolc fogalmi fázisból áll. Az implementációnak úgy kell viselkednie, mintha ezek a megfelelő sorrendben történnének; nem köteles nyolc külön programot vagy nyolc köztes fájlt létrehozni.

Fázis	Fogalmi művelet	Következmény
1.	A fizikai forrás karaktereinek leképezése a forráskarakter-készletre; a C17 szerinti trigráfok helyettesítése	A forrásfájl külső kódolása és a nyelvi karakterek nem azonos fogalmak
2.	A közvetlenül egymást követő backslash és újsor törlése	Fizikai sorok logikai sorrá kapcsolódhatnak
3.	Felbontás preprocessing tokenekre és whitespace-re; a kommentek helyettesítése egy-egy szóközzel	A komment nem egyszerűen „eltűnik”
4.	Előfeldolgozási direktívák végrehajtása, makrók helyettesítése, include-ok feldolgozása	Kialakul az előfeldolgozás utáni fordítási egység
5.	A karakter- és sztringliterálok karaktereinek, illetve escape-jeinek feldolgozása a végrehajtási karakterkészlet szerint	Például a `\n` escape nem két kiírandó karaktert jelent
6.	Szomszédos sztringliterálok összefűzése	A `"net" "work"` egyetlen sztringliterállá válik

Fázis	Fogalmi művelet	Következmény
7.	A preprocessing tokenek nyelvi tokenekké alakítása; szintaktikai és szemantikai elemzés, fordítás	Itt érvényesülnek a C nyelvtani és típusozási szabályai
8.	Külső objektum- és függvényhivatkozások feloldása, szükséges könyvtári összetevők hozzárendelése	Létrejön a végrehajtáshoz szükséges programkép

Az include-dal bevont állományokon az első négy fázis rekurzívan végbemegy. A táblázat nem ír elő ELF-formátumot, assemblyfájlt vagy külön linkerfolyamatot: ezek implementációs megoldások.

2.1.1.1 Preprocessing token és nyelvi token

A **preprocessing token**, vagyis előfeldolgozási token az előfeldolgozó által kezelt lexikai egység. Ilyen például az azonosító, az előfeldolgozási szám, a sztringliterál és az írásjel; meghatározott környezetben a headernév is külön kategória.

Az előfeldolgozási szám megengedőbb kategória, mint egy érvényes C számkonstans. Például az `1e+` egyetlen preprocessing token lehet, de nem érvényes lebegőpontos konstans. Ezért az előfeldolgozás sikere még nem bizonyítja, hogy a program lexikailag és szemantikailag helyes C-program.

A tokenizálás általában a következő tokenként felvehető leghosszabb karaktersorozatot választja. Emiatt a whitespace helye számíthat: az `a ++ + b` és az `a + ++b` nem ugyanazt a tokensorozatot jelenti.

2.1.1.2 A sorrend látható következményei

Az alábbi **C17-részlet** szemléltetésre szolgál, nem önálló program:

```
// A kovetkezo fizikai sor is komment lesz: \
int timeout = 30;

unsigned/**/int retries;
const char label[] = "net" "\n" "ok";
```

A feldolgozás lépései:

1. A második fázis törli az első sor végén álló backslash–újsor párt.
2. Az `int timeout = 30;` így ugyanannak a logikai sornak a része lesz, mint a `//` komment.
3. A harmadik fázis ezt az egész kommentet szóközre cseréli. A `timeout` deklarációja tehát nem marad meg.
4. Az `unsigned/**/int` alakból `unsigned int` lesz, nem `unsignedint`.
5. A `"\n"` escape-je újsorkaraktert jelöl; ez nem azonos a forrásfájl sorösszekapcsolásával.
6. A három szomszédos sztringliterál összefűződik. Az eredmény hat tartalmi karakter és egy lezáró nullkarakter, ezért `sizeof label` értéke 7 C bájt.

A line splicing feltétele **közvetlen szomszédság**: ISO C17-ben a backslash és az újsor közé tett szóköz már nem része ennek a szabálynak. Egy fordítóbővítés esetleges engedékenységére nem építünk.

2.1.2 Hogyan jön létre a fordítási egység?

ISO C17: egy forrásfájl és az általa `#include` direktívákkal bevont headerek, illetve forrásfájlok együtt alkotják az **előfeldolgozási fordítási egységet** (*preprocessing translation unit*). Az előfeldolgozás után ezt **fordítási egységnek** (*translation unit*, TU) nevezzük.

A fordítási egység tehát nem egyszerűen „egy `.c` fájl”. Tartalmát a bevont headerek, a makródefiníciók és a feltételes fordítás döntései is alakítják. Ugyanaz a `.c` fájl eltérő előfeldolgozási konfigurációval eltérő TU-t eredményezhet.

2.1.2.1 Kidolgozott, két fordítási egységes példa

A következő három fájl teljes, minimális C17-programot alkot. A függvény csak felső korlátot alkalmaz; a negatív értékeket változatlanul hagyja.

`limit.h`:

```
#define LIMIT_MAX 10

int limit_to_max(int value);
```

`limit.c`:

```
#include "limit.h"

int limit_to_max(int value)
{
    return value > LIMIT_MAX ? LIMIT_MAX : value;
}
```

`main.c`:

```
#include "limit.h"

int main(void)
{
    return limit_to_max(12) == LIMIT_MAX ? 0 : 1;
}
```

Ebben a minimális példában minden TU egyszer tartalmazza a headert. Az ismételt bevonás elleni védelem és az include-higiénia a Headerek, deklarációk és include-higiénia témája.

A `main.c` előfeldolgozásának lényegi eredménye:

```
int limit_to_max(int value);

int main(void)
{
    return limit_to_max(12) == 10 ? 0 : 1;
}
```

Ez szemléltető, előfeldolgozott **részlet**: a tényleges eszközkimenet sorjelöléseket is tartalmazhat. A makródefiníció már nincs jelen, a használata helyén `10` áll.

Fontos, hogy a `limit_to_max` **definíciója nincs ebben a TU-ban**. A `main.c` fordításához a deklaráció biztosítja a hívás típusinformációját. A függvénytörzs másik fordítási egységben van.

2.1.3 A gyakorlati toolchain lépései

GCC/Clang laborprofil: ISO C17, Linux hosted környezet, x86-64 System V ABI, LP64. Kijelölt eszközkiadások: a `lab-next` profil (profiltábla) és GNU binutils 2.43. A parancsok az ennek megfelelően kiválasztott eszközöket feltételezik; itt elvárt artefaktumokat mutatunk be, nem dokumentált fordítási vagy futtatási eredményeket.

A fájlok közös könyvtárából:

```
CC=gcc

"$CC" -std=c17 -Wall -Wextra -Wpedantic -E main.c -o main.i
"$CC" -std=c17 -Wall -Wextra -Wpedantic -O0 -S main.i -o main.s
"$CC" -c main.s -o main.o

"$CC" -std=c17 -Wall -Wextra -Wpedantic -O0 -c limit.c -o limit.o
"$CC" main.o limit.o -o demo
```

A Clang-változathoz a `CC=clang` beállítással külön munkakönyvtárban ismételhető meg a build. Az azonos nyelvi jelentés nem követeli meg azonos assembly vagy azonos diagnosztikai szöveg előállítását.

Lépés	Artefaktum	Mit jelent?
`-E`	`main.i`	Előfeldolgozott forrás szöveges megjelenítése
`-S`	`main.s`	Célarchitektúrához tartozó assembly
Assembly fordítása	`main.o`	Relokálható objektumfájl
`-c limit.c`	`limit.o`	A másik TU fordításának és assemblálásának eredménye
Linkelés	`demo`	A rendszer betöltője számára előkészített végrehajtható állomány

A kapcsolók a **compiler driver** működését vezérlik. A driver nem maga az összes feldolgozó komponens: szükség szerint további eszközöket indít vagy integrált komponenseket használ.

A `-c` jelentése nem „csak a parserig menj”. C forrásból objektumfájlt készít, de nem linkel teljes programot. Hasonlóképpen, a `-E` kimenete sem a szabvány valamely belső állapotának kötelező, karakterre pontos reprezentációja.

2.1.3.1 Parser, szemantikai elemzés és IR

A **parser**, vagyis szintaktikai elemző azt ellenőrzi, hogyan alkotnak a tokenek deklarációkat, kifejezéseket és utasításokat. Ebből rendszerint absztrakt szintaxisfa, *abstract syntax tree* (AST) készül.

A **szemantikai elemzés** a nyelvi jelentést ellenőrzi: például létezik-e a használt deklaráció, megfelelőek-e az operandustípusok, és megengedett-e egy konverzió. A két feladat implementációban összefonódhat; nem szükséges két külön menetnek lennie.

Az **IR**, *intermediate representation*, köztes reprezentáció a fordító belső programleírása. Egy fordító többféle IR-t használhat, eltérő absztrakciós szinteken. Ezeken történhetnek optimalizálások, majd

a kódgenerálás gépi utasításokat választ és regisztereket rendel az értékekhez. Az IR nem ISO C által előírt formátum.

Az assembler az assemblyből gépi kódot és kapcsolódó objektumfájl-adatokat készít. A `main.o` tartalmazhat még feloldatlan hivatkozást a `limit_to_max` függvényre: ettől az objektumfájl nem hibás. A feloldás a linkelés feladata.

2.1.4 Külön fordítás és hiányzó információ

A **külön fordítás** (*separate compilation*) során a fordítási egységeket önállóan dolgozzuk fel. Hagyományos, LTO nélküli fordításban a `main.c` elemzésekor a fordító nem látja a `limit.c` függvénytörzsét.

A közös header helyességi szerepe ezért alapvető: ugyanazt a deklarációt adja a hívónak és a megvalósításnak. Ha a megvalósítás is bevonja, a fordító helyben összevetheti a deklarációt a definícióval.

Bizonyításvázlat a helyi ellenőrzés korlátjáról: a `main.c` TU-jában csak a deklarált függvénytípus érhető el. Ebből a fordító ellenőrizheti a hívás argumentumait, de nem következtethet egy nem látott definíció tényleges típusára. Ehhez kereszt-egység információ kellene.

Elkülönített **hibás C17-részletként** képzeljük el, hogy a `limit.c` nem vonja be a headert, és ezt definiálja:

```
long limit_to_max(long value)
{
    return value > 10L ? 10L : value;
}
```

A hívó továbbra is `int limit_to_max(int)` típust ismer. A két típus inkompatibilis; a teljes program viselkedése nem definiált. A hagyományos linker ettől még összekapcsolhatja az azonos nevű szimbólumokat. A sikeres link tehát nem bizonyítja a kereszt-egység típushelyességet.

A linkidőbeli optimalizálás, *link-time optimization* (LTO), több információt tehet elérhetővé, és egyes eltéréseket diagnosztizálhat. Nem helyettesíti a helyes deklarációs szerződést.

2.1.5 Hibahely-meghatározás és gyakori tévedések

Jelenség	Elsődleges vizsgálati hely
Nem található a bevont header	Előfeldolgozás, include-keresési konfiguráció
Érvénytelen számkonstans	Lexikai feldolgozás, preprocessing token átalakítása
Hiányzó pontosvessző	Szintaktikai elemzés
Nem deklarált azonosító	Szemantikai elemzés
Módosított vagy rossz célarchitektúrájú assembly elutasítása	Assembler
A buildből kimaradt függvénydefiníció	Linkelés

Ezek logikai kategóriák, nem kötelezően külön folyamatok. A hiba tényleges helyét a diagnosztika és az előállított utolsó sikeres artefaktum együtt segít azonosítani.

Gyakori tévedés, hogy minden headerben jelzett hiba előfeldolgozási hiba. Egy header hibás C-deklarációját az előfeldolgozó gond nélkül továbbíthatja; a parser vagy a szemantikai elemző később utasítja el.

Szintén hibás következtetés, hogy a sikeres `-c` teljes programhelyességet bizonyít. Csak az adott fordítási egység objektumfájlig történő feldolgozása sikerült. A hiányzó definíciók, az inkompatibilis kereszt-egység deklarációk és a futásidejű hibák ettől még fennmaradhatnak.

2.1.6 Feladatok

2.1.6.1 Tokenek és sorrend

Magyarázd meg, miért nem deklarálódik a korábbi példában a `timeout`, miközben az `unsigned/**/int retries;` érvényes deklaráció marad. Miért 7 C bájt a `label` mérete?

2.1.6.2 Két fordítási egység használata

Módosítsd kizárólag a `main.c` fájlt úgy, hogy a `12 → 10` és a `7 → 7` esetet is ellenőrizze. Siker esetén térjen vissza nullával. Fordítsd újra csak ezt a TU-t, majd linkeld a változatlan `limit.o` mellé.

2.1.6.3 Diagnosztikai szétválasztás

Három külön módosítást vizsgálj:

- A `limit.h` nincs az include-keresés számára elérhető helyen.
- A header elérhető, de a linkparancsból kimarad a `limit.o`.
- Mindkét objektumfájl jelen van, de a megvalósítás a fent bemutatott inkompatibilis `long` változat.

Melyik esetben hol várható hiba? Melyikben nem kötelező diagnosztikára számítani?

2.1.7 Megoldások

2.1.7.1 Tokenek és sorrend

A sorösszekapcsolás megelőzi a kommentfelismerést, ezért a `timeout` deklarációja a `//` komment részévé válik. A blokkkommentet viszont egy szóköz helyettesíti, így két külön kulcsszó marad: `unsigned` és `int`.

A `label` tartalma három betű, egy újsorkarakter és két további betű. Ehhez egy lezáró nullkarakter tartozik:

$$3 + 1 + 2 + 1 = 7.$$

Az ellenőrzés feltétele a műveletek helyes sorrendje; nem az, hogy egy eszköz pontosan milyen whitespace-szel jeleníti meg az előfeldolgozott szöveget.

2.1.7.2 Két fordítási egység használata

A teljes új `main.c`, az előző C17-laborprofilhoz:

```
#include "limit.h"

int main(void)
{
    if (limit_to_max(12) != LIMIT_MAX) {
        return 1;
    }

    if (limit_to_max(7) != 7) {
        return 1;
    }

    return 0;
}
```

```
"$CC" -std=c17 -Wall -Wextra -Wpedantic -O0 -c main.c -o main.o
"$CC" main.o limit.o -o demo
```

A `limit.o` újrafelhasználásának feltétele, hogy sem a megvalósítás, sem annak függőségei vagy fordítási profilja nem változott. A program elvárt visszatérési értéke nulla; ez ellenőrzési feltétel, nem közölt futtatási eredmény.

2.1.7.3 Diagnosztikai szétválasztás

Az elérhetetlen header az előfeldolgozásnál okoz hibát. A kihagyott `limit.o` esetén a fordítás sikerülhet, de a linkelés nem találja a függvény definícióját.

Az inkompatibilis típusú megvalósítás külön-külön lefordulhat, és hagyományos linkeléssel össze is kapcsolható. A kereszt-egység eltérésre nem kötelező diagnosztikát kapni; a program ettől még hibás, viselkedése nem definiált. Javítása a közös deklaráció következetes használata, majd az érintett TU-k újrafordítása.

2.1.8 Összefoglaló

A szabvány fordítási fázisai a forrás értelmezését írják le; a toolchain lépései ennek gyakorlati megvalósítását. A fordítási egység az include-ok és az előfeldolgozási döntések eredménye, nem pusztán egy fájlnév. A parser szerkezetet, a szemantikai elemzés nyelvi helyességet vizsgál; a kódgenerálás és az assembler objektumfájlt állít elő, a linker pedig összekapcsolja az egységeket.

Hibakereséskor mindig két kérdést tegyünk fel: **meddig jutott el a feldolgozás, és milyen információ állt ott rendelkezésre?**

2.2 Headerek, deklarációk és include-higiénia

A header nem külön lefordítandó modul, hanem több fordítási egységbe beilleszthető interfészleírás. Ahogy a Fordítási fázisok és fordítási egységek részben láttuk, a fordító az előfeldolgozás után már a beillesztett deklarációkat látja. Ebből fontos következmény adódik: ugyanaz a header különböző forrásfájlokban, különböző előfeldolgozási környezetben is megjelenhet.

A jó nyilvános header önállóan használható, többszöri beillesztése nem okoz hibát, és csak az implementáció használatához szükséges információt teszi közzé. Ebben az egységben egy két forrásfájlból álló, egyszerű erőforráskeret-könyvtáron építjük fel ezt a szerződést. A nyelvi szabályok alapja **ISO C17**; a konkrét keresési utak és buildparancsok külön jelölt **GCC-laborhoz** tartoznak.

2.2.1 Deklaráció, definíció és az interfész határa

A **deklaráció** (*declaration*) információt közöl egy azonosítóról: például annak típusáról és nyelvi kapcsolásáról. A **definíció** (*definition*) olyan deklaráció, amely függvénynél megadja a függvénytörzset, objektumnál pedig az objektumot is meghatározza, tárolót biztosítva számára.

C17-ben, fájlhatókörben:

Alak	Jelentés
`int budget_ready(void);`	Függvénydeklaráció, törzs nélkül
`int budget_ready(void) { return 1; }`	Függvénydefiníció
`extern unsigned budget_count;`	Objektumdeklaráció, nem definíció
`unsigned budget_count = 0;`	Objektumdefiníció
`unsigned budget_count;`	Feltételes definíció (*tentative definition*)
`extern unsigned budget_count = 0;`	Az inicializáló miatt definíció

A feltételes definíció nem egyszerű „előzetes említés”: ha a fordítási egységben nincs megfelelő másik definíció, a fordítási egység végén meghatározott szabályok szerint definícióvá válik. Ezért a nyilvános headerbe tett `unsigned budget_count;` több fordítási egység esetén hibás programszerkezethez vezethet. Egyes eszközbeállítások elfedhetik ezt, de erre hordozható programot nem építünk.

Az `extern` nem linkerparancs. Az inicializáló nélküli, szokásos publikus objektumdeklarációban lehetővé teszi egy máshol definiált objektum használatát. A könyvtárunk egyetlen `.c` fájlban definiálja a közzétett verzióobjektumot.

Fontos C/C++ különbség: **C-ben a fájlhatókörű `const` önmagában nem teszi belső kapcsolásúvá az objektum nevét**. A C++-ból ismert szabályt itt nem alkalmazhatjuk.

2.2.2 Include-keresés és ismételt beillesztés

2.2.2.1 Idézőjel és szögletes zárójel

Az alábbi két forma keresési módot választ, nem hozzáférési jogosultságot:

```
/* ISO C17 szintaxisrészlet */
#include "budget.h"
#include <stdlib.h>
```

ISO C17: a keresési helyek és eljárások implementáció által meghatározottak. Az idézőjeles forma sikertelen keresése után a beillesztést a szögletes zárójeles formának megfelelően is meg kell kísérelni.

GCC-labor: különleges keresési kapcsolók nélkül az idézőjeles forma először a beillesztést tartalmazó fájl könyvtárában keres, majd az idézőjeles kereséshez megadott helyeken, végül a szögletes zárójeles keresés útvonalain. A `-Iinclude` az `include` könyvtárat hozzáadja a kereséshez; ez mindkét formánál szerepet kaphat. A szögletes zárójel tehát nem jelenti azt, hogy „csak rendszerheader”.

Üzemeltetési szempontból a keresési út a build bemenete. Egy projektbeli `stdlib.h` nem kívánt módon elfedheti a rendszerheader nevét, ha rossz könyvtár kerül a keresés elejére. A keresési utakat és sorrendjüket ezért ugyanúgy kontrollálni kell, mint a függőségek verzióit.

2.2.2.2 Include guard és `#pragma once`

Az **include guard** egy egyedi makrónévvel védi a header tartalmát. Az első beillesztés definiálja a nevet; a következő beillesztés ugyanabban a fordítási egységben kihagyja a védett részt.

A guard hatóköre nem a teljes build. Két külön `.c` fájl mindegyike megkapja a header tartalmát. Következésképpen a guard **nem javítja meg a headerbe tett külső függvény- vagy objektumdefiníciót**.

A `#pragma once` GCC és Clang alatt elterjedt alternatíva, de a „fájlt csak egyszer illeszd be” jelentés nem ISO C17-szolgáltatás. Használata implementációs függőség; a következő példák hordozható guardot alkalmaznak. A guard neve projektazonosítót tartalmaz, és nem kezdődik fenntartott aláhúzásos mintával.

2.2.3 Két forrásfájlos könyvtár nyilvános headerrel

A könyvtár egy véges keretet kezel. A kliens létrehoz egy keretet, majd egységeket foglal belőle. A sikertelen foglalás nem változtatja meg az állapotot.

A fájlok:

- `include/budget.h`: nyilvános interfész;
- `src/budget_private.h`: belső reprezentáció;
- `src/budget.c`: létrehozás, megszüntetés és verzióobjektum;
- `src/budget_take.c`: foglalási művelet;
- `app.c`: külön kliensprogram.

2.2.3.1 A nyilvános header

ISO C17; célarchitektúrától független forráskód. Az `include/budget.h` teljes tartalma:

```
#ifndef DEVOPS_BUDGET_H_INCLUDED
#define DEVOPS_BUDGET_H_INCLUDED

struct budget;

enum budget_status {
    BUDGET_OK,
    BUDGET_INVALID,
```

```
    BUDGET_EXHAUSTED
};

extern const unsigned budget_api_version;

/* Tulajdont ad at; allokacios hiban NULL az eredmeny. */
struct budget *budget_create(unsigned capacity);

/* NULL eseten nincs hatasa; kulonben ervenyes ownert igenyel. */
void budget_destroy(struct budget *budget);

/* Nem tarolja el a kolcsonzott pointert.
   Hiba eseten az allapot valtozatlan. */
enum budget_status budget_take(struct budget *budget,
                               unsigned amount);

#endif
```

A `struct budget;` **elődeklaráció** (*forward declaration*): bevezeti a struktúratípus nevét, de nem adja meg a tagjait. A típus itt **nem teljes típus** (*incomplete type*). Pointer deklarálható rá, de a kliens nem kérdezheti le a méretét, nem érheti el a tagjait, és nem definiálhat belőle érték szerint objektumot.

Ezt nevezzük **opaque**, vagyis átlátszatlan típusnak. A reprezentáció nem része a nyilvános headernek.

A header **önmagában elegendő** (*self-contained*): használata nem igényel előtte másik `#include` sort. Jelenleg egyetlen rendszerheaderre sincs szüksége. Ha később `size_t` jelenne meg az interfészben, közvetlenül be kellene illesztenie a `<stddef.h>` headert, nem bízhatna a kliens include-sorrendjében.

2.2.3.2 A privát reprezentáció és az implementáció

A `src/budget_private.h` csak a könyvtár implementációjához tartozik:

```
#ifndef DEVOPS_BUDGET_PRIVATE_H_INCLUDED
#define DEVOPS_BUDGET_PRIVATE_H_INCLUDED

#include "budget.h"

struct budget {
    unsigned remaining;
};

#endif
```

A `src/budget.c`:

```
#include "budget.h"
#include "budget_private.h"

#include <stdlib.h>
```

```
const unsigned budget_api_version = 1u;

struct budget *budget_create(unsigned capacity)
{
    struct budget *budget = malloc(sizeof *budget);
    if (budget == NULL) {
        return NULL;
    }

    budget->remaining = capacity;
    return budget;
}

void budget_destroy(struct budget *budget)
{
    free(budget);
}
```

A `malloc` és `free` itt a dinamikus tároló megszerzését és felszabadítását végzi; részletes memóriaszerződésük későbbi téma. Lényeges, hogy a `sizeof *budget` használatakor a struktúratípus már teljes.

A `src/budget_take.c`:

```
#include "budget.h"
#include "budget_private.h"

#include <stddef.h>

enum budget_status budget_take(struct budget *budget,
                               unsigned amount)
{
    if (budget == NULL) {
        return BUDGET_INVALID;
    }

    if (amount > budget->remaining) {
        return BUDGET_EXHAUSTED;
    }

    budget->remaining -= amount;
    return BUDGET_OK;
}
```

Mindkét forrás először a saját nyilvános headert illeszti be. Ez hasznos ellenőrzési gyakorlat: kisebb eséllyel takarja el egy előző include a publikus header hiányzó függőségét.

2.2.3.3 Az interfész pontos szerződése

A `capacity` és az `amount` az `unsigned` típus értéktartományába tartozó egységszám; nem feltételezünk konkrét bitszélességet.

- A létrehozás siker esetén új, kizárólagosan birtokolható objektumot ad, allokációs hibánál null pointert.
- A foglalás null pointerre `BUDGET_INVALID` eredményt ad. Minden nem null pointernek élő, e könyvtár által létrehozott objektumra kell mutatnia.
- Elégtelen keretnél `BUDGET_EXHAUSTED` az eredmény, állapotváltozás nélkül.
- Nulla egység foglalása érvényes objektumon sikeres.
- Ugyanazon objektum párhuzamos használatához külső szinkronizáció szükséges.

Szerep	Tulajdonos és élettartam	Érvénytelenítés, hibakimenet
Tulajdonosi pointer	A kliens felel a pontosan egyszeri megszüntetésért	A pointer másolása nem hoz létre új objektumot
Dinamikus tároló	Sikeres létrehozástól a `budget_destroy` hívásáig él	A megszüntetés felszabadítja
A `budget_take` paramétere	A hívás idejére kölcsönzött hozzáférés; nem tárolódik el	Null pointerre hibakód; dangling pointerre nincs biztonságos hibafelismerés
További pointermásolatok	Nem hosszabbítják meg az objektum élettartamát	Megszüntetés után érvénytelenek; ismételt használatuk UB-hez vezethet

Helyességi bizonyításvázlat. Legyen az aktuális keret r, a kérés a. Ha $a > r$, a függvény módosítás nélkül visszatér. Egyébként $0 \leq a \leq r$, ezért $r - a$ ábrázolható az `unsigned` típusban, és az új keret nem nagyobb a réginél. A kivonást tehát olyan ellenőrzés előzi meg, amely kizárja az alulcsordulást.

2.2.3.4 Kliens és build

Az `app.c` teljes, ISO C17 kliens:

```c
#include "budget.h"

#include <stdlib.h>

int main(void)
{
    struct budget *budget = budget_create(3u);
    if (budget == NULL) {
        return EXIT_FAILURE;
    }

    int ok =
        budget_take(budget, 2u) == BUDGET_OK &&
        budget_take(budget, 2u) == BUDGET_EXHAUSTED &&
        budget_take(budget, 1u) == BUDGET_OK;

    budget_destroy(budget);
    return ok ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

A választott parancsprofil: **Linux x86-64, System V ABI, LP64; a `lab-next` profil (profiltábla)**, explicit C17 móddal. Ez laborprofil, nem itt elvégzett fordítás vagy futtatás.

A projekt gyökeréből:

```
gcc -std=c17 -Wall -Wextra -pedantic-errors -Iinclude \
    -c src/budget.c -o budget.o
gcc -std=c17 -Wall -Wextra -pedantic-errors -Iinclude \
    -c src/budget_take.c -o budget_take.o
ar rcs libbudget.a budget.o budget_take.o

gcc -std=c17 -Wall -Wextra -pedantic-errors -Iinclude \
    -c app.c -o app.o
gcc app.o libbudget.a -o budget_demo
```

Az archívum a két implementációs objektumfájlt tartalmazza; a kliens csak a nyilvános headert használja. **Várt eredmény:** sikeres allokáció esetén a kliens sikeres kilépést jelez, és közvetve ellenőrzi, hogy az elutasított foglalás nem fogyasztott keretet.

2.2.4 Minimális függőség és konfigurációs egyezés

A minimális include nem az include-ok mechanikus törlését jelenti. Minden fájl közvetlenül illessze be azokat a headereket, amelyek deklarációit használja. Ne építsen arra, hogy egy másik header véletlenül továbbadja őket.

Ciklikus függőségnél az include guard legfeljebb a végtelen beillesztést akadályozza meg; a hiányzó típusinformációt nem pótolja. Ha két struktúra csak pointerrel hivatkozik egymásra, elődeklarációval megszakítható a függőség. Érték szerinti taghoz viszont teljes típus kell; a kölcsönös érték szerinti beágyazás nem oldható meg include-sorrenddel.

Különösen veszélyes a makrókkal módosított reprezentáció. Az alábbi **hibát szemléltető C17-részlet** a privát struktúra alternatív definíciója:

```
struct budget {
#ifdef BUDGET_DIAGNOSTICS
    unsigned failed_requests;
#endif
    unsigned remaining;
};
```

Ha a létrehozást és a foglalást végző forrás eltérő makróbeállítással fordul, ugyanahhoz az objektumhoz eltérő típusleírást használhatnak. A linkelés ettől még sikerülhet; a futás nem válik helyessé. Konkrét layout vagy hibakimenet nem garantálható, a program UB-be kerülhet.

Az include guard itt sem segít. Sőt, egy fordítási egységen belül az első beillesztéskor érvényes konfigurációt rögzíti: későbbi makróváltoztatás nem dolgoztatja fel újra a headert.

Javításként a reprezentációt befolyásoló konfiguráció legyen központi, minden érintett fordítási egységre egységes buildbemenet. Változtatásakor minden függő objektumfájlt újra kell fordítani. A nyilvános API alakját lehetőleg ne változtassuk kliensenként beállítható makrókkal.

2.2.5 Feladatok

1. **Magyarázó:** miért helyes a headerben az `extern const unsigned budget_api_version;`, és miért nem megfelelő helyette a `const unsigned budget_api_version = 1u;`?
2. **Kódolási:** készíts `tests/header_check.c` fájlt, amely elsőként, majd ismételten is beilleszti a nyilvános headert. Ellenőrizd fordítással, linkelés nélkül.
3. **Diagnosztikai:** a `budget.c` makró nélkül, a `budget_take.c` pedig `BUDGET_DIAGNOSTICS` mellett fordult a fenti alternatív struktúrával. A linker nem jelez hibát. Mit bizonyít ez, és hogyan javítanád a buildet?

2.2.6 Megoldások

2.2.6.1 Deklaráció és egyetlen definíció

Az `extern` deklaráció megadja az objektum típusát, de nem definiál új objektumot minden fordítási egységben. Az inicializált alak viszont definíció lenne minden olyan egységben, amely a headert beilleszti. A helyes szerkezet: publikus deklaráció a headerben, egyetlen definíció az implementációban.

Ellenőrzési feltétel: a könyvtár és több külön kliensfordítási egység együtt is csak egy verzióobjektum-definíciót tartalmazzon.

2.2.6.2 Önállóság és ismételhetőség ellenőrzése

A teljes tesztfájl:

```
#include "budget.h"
#include "budget.h"

int header_check(void)
{
    return budget_api_version == 1u;
}
```

Azonos GCC-laborprofilban:

```
gcc -std=c17 -Wall -Wextra -pedantic-errors -Iinclude \
    -fsyntax-only tests/header_check.c
```

Várt ellenőrzési feltétel: nincs diagnosztika. Az első include az előzetes függőség nélküli használatot, a második a guard működését vizsgálja. Linkelés hiányában ez még nem ellenőrzi a verzióobjektum definíciójának meglétét.

2.2.6.3 Konfigurációs eltérés felismerése

A sikeres linkelés csak azt mutatja, hogy az adott eszköz elfogadta az összekapcsolást; nem bizonyítja a fordítási egységek típusleírásainak egyezését.

A javítás egységes konfiguráció, az érintett objektumfájlok tiszta újrafordítása, majd az előfeldolgozott struktúradefiníciók összevetése.

Ellenőrzési feltétel: mindkét implementáció kompatibilis teljes típust lásson. Egyetlen sikeres próbahívás ehhez nem elegendő bizonyíték.

2.2.7 Rövid összefoglaló

A nyilvános header deklarációkat és használati szerződést közöl; az implementáció biztosítja a definíciókat. Az önállóságot közvetlen függőségekkel, az ismételhetőséget include guarddal, a reprezentáció elrejtését opaque típussal érjük el. A keresési út és a makrókonfiguráció a build lényegi bemenete: eltérésük olyan hibát is okozhat, amelyet a linker nem vesz észre.

2.3 Makrók, feltételes fordítás és veszélyes helyettesítés

A makró látszólag lehet konstans, függvény vagy rövidített utasítás, de a C nyelvben ezek egyikével sem azonos: az előfeldolgozó, vagyis a *preprocessor* helyettesítési szabálya. Ez a különbség akkor válik veszélyessé, amikor egy ártalmatlannak tűnő makróhívás megváltoztatja a műveleti sorrendet, vagy többször hajt végre egy mellékhatást.

A Headerek, deklarációk és include-higiénia részben látott headerek makródefiníciókat is közvetíthetnek. Ezek azonban nem követik a függvények és változók blokkszintű hatóköreit. A példákhoz A szükséges C szintaxis tömör áttekintése kifejezés- és függvényfogalmait használjuk.

A nyelvi alap az **ISO C17**; az eltéréseket külön jelöljük. A teljes példa kijelölt laborprofilja a `lab-next` profil (profiltábla), Linux x86-64 System V ABI, LP64 és a `lab-next` profil (profiltábla). A bemutatott eredmények levezetett, várt eredmények, nem dokumentált futtatási megfigyelések.

2.3.1 A makró tokeneket helyettesít

Az *object-like macro*, azaz objektumszerű makró egy azonosítóhoz helyettesítési listát rendel. A névben szereplő „objektum” itt nem nyelvi memóriaobjektumot jelent: a makrónak nincs címe, típusa vagy tárolója.

Az alábbi C17-részlet két definíciót mutat:

```
#define RETRY_LIMIT 3
#define WINDOW_SIZE 2 + 3
```

A `RETRY_LIMIT` megfelelő előfordulásának helyére a `3` előfeldolgozási token kerül. A `WINDOW_SIZE` három tokent helyettesít be: `2`, `+`, `3`.

Ezért:

```
/* C17-részlet. */
int capacity = 10 * WINDOW_SIZE;
```

a kifejtés után lényegében:

```
int capacity = 10 * 2 + 3;
```

A fordító ezt $10 \cdot 2 + 3 = 23$ értékű kifejezésként értelmezi, nem pedig $10 \cdot (2 + 3) = 50$ alakban. A javítás:

```
#define WINDOW_SIZE (2 + 3)
```

A helyettesítés **tokenalapú**, nem tetszőleges részszövegek cseréje. A `RETRY_LIMIT_EXTRA` azonosító része nem cserélődik ki, és a `"RETRY_LIMIT"` sztringliterál belseje sem.

A *function-like macro*, vagyis függvényszerű makró paramétereket fogad:

```
#define SQUARE(x) ((x) * (x))
```

Definíciókor a makrónév és a nyitó zárójel között nem lehet szóköz. A `#define SQUARE (x)` már objektumszerű makrót definiálna. Használatkor viszont a `SQUARE (3)` is kiváltja a függvényszerű makró kifejtését.

A makró argumentuma tokensorozat, nem előre kiszámított érték. Az argumentumokat vessző választja el, kivéve a belső, párosított kerek zárójeleken belüli vesszőket. A kapcsos zárójel önmagában nem védi meg a vesszőt az argumentumelválasztó szereptől.

2.3.2 Zárójelezés és többszörös kiértékelés

Egy kifejezésmakróban rendszerint külön kell zárójelezni minden paraméterelőfordulást és az egész eredményt:

```
#define BAD_SQUARE(x) x * x
#define SQUARE(x) ((x) * (x))
```

A `BAD_SQUARE(2 + 3)` kifejtése:

```
2 + 3 * 2 + 3
```

Először a szorzás történik meg, ezért az eredmény $2 + 6 + 3 = 11$. A javított változat:

```
((2 + 3) * (2 + 3))
```

eredménye 25.

A zárójelezés azonban **nem szünteti meg a többszörös kiértékelést**. A `SQUARE(n++)` hibás, UB-s tanítási példa: kifejtése `((n++) * (n++))`, amelyben ugyanazon skalár objektum két módosítása között nincs megfelelő sorrendi kapcsolat. Ehhez nincs garantált eredmény; nem helyes azt mondani, hogy „legfeljebb kétszer növeli”.

Nem minden többszörös kiértékelés UB. A következő önálló program definiált, de meglepő viselkedést szemléltet.

Fájl: `macro_demo.c`; minimum C17, a fent megadott laborprofil.

```
#include <stdio.h>

#define BAD_SQUARE(x) x * x
#define SQUARE(x) ((x) * (x))
#define MIN_EXPR(a, b) ((a) < (b) ? (a) : (b))

static inline int min_int(int a, int b)
```

```
{
    return a < b ? a : b;
}

int main(void)
{
    int n = 3;
    int macro_result = MIN_EXPR(n++, 10);
    int after_macro = n;

    n = 3;
    int function_result = min_int(n++, 10);

    if (printf("%d %d\n",
               BAD_SQUARE(2 + 3), SQUARE(2 + 3)) < 0) {
        return 1;
    }

    if (printf("%d %d; %d %d\n",
               macro_result, after_macro,
               function_result, n) < 0) {
        return 1;
    }

    return 0;
}
```

Külön laborban használható fordítási parancsok:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 macro_demo.c -o macro_demo-gcc
clang -std=c17 -Wall -Wextra -Wpedantic -O0 macro_demo.c -o macro_demo-clang
```

Bizonyításvázlat a makrós és függvényes változat eltérésére. A makró helyettesítése:

```
((n++) < (10) ? (n++) : (10))
```

1. Az első `n++` értéke `3`, mellékhatása után `n` értéke `4`.
2. A feltétel igaz. A feltételes operátor első operandusának kiértékelése megelőzi a kiválasztott ágét.
3. A második `n++` értéke `4`, majd `n` értéke `5`.

Így a makrós eredmény `4`, a végső `n` pedig `5`. A függvényhívás egyetlen `n++` argumentumot értékel ki: a függvény `3` és `10` értéket kap, ezért az eredmény `3`, a végső `n` pedig `4`.

Sikeres kimeneti írás mellett a várt sorok:

```
11 25
4 5; 3 4
```

A `static inline` függvény valódi, típusellenőrzött függvény. Az `inline` nem garantálja a gépi kód beillesztését, és nem ez biztosítja az egyszeri argumentumkiértékelést: az a függvényhívás tulajdonsága. Részletes kapcsolási szabályait a static inline, _Generic és headeres eszköztár tárgyalja.

A függvény sem tesz minden hívást biztonságossá: a `min_int(n++, n++)` C17-ben továbbra is UB. A külön argumentumok kiértékeléséhez a nyelv nem biztosít itt szükséges sorrendet; ezt a sequencing szabályai részletezik.

2.3.3 Argumentumkifejtés, sztringesítés és tokenösszeillesztés

Közönséges helyettesítéskor az argumentumokban található makrók előbb kifejtődnek. A helyettesítés eredményét az előfeldolgozó újra megvizsgálja további makrók szempontjából. Két fontos kivétel a `#` és a `##` közvetlen operandusaként szereplő paraméter.

A `#` *stringification*, azaz sztringesítés: az argumentum tokenjeiből sztringliterált készít. A `##` *token pasting*, azaz tokenösszeillesztés: két tokenből egyetlen új előfeldolgozási tokent alkot.

```
/* C17-részlet. */
#define SLOT 7

#define STR_RAW(x) #x
#define STR(x) STR_RAW(x)

#define CAT_RAW(a, b) a ## b
#define CAT(a, b) CAT_RAW(a, b)
```

A különbségek:

Használat	Kifejtés	Indok
`STR_RAW(SLOT)`	`"SLOT"`	A `#` mellett nincs előzetes argumentumkifejtés.
`STR(SLOT)`	`"7"`	A külső szint előbb kifejti a `SLOT` makrót.
`CAT_RAW(metric_, SLOT)`	`metric_SLOT`	A `##` mellett nincs előzetes kifejtés.
`CAT(metric_, SLOT)`	`metric_7`	A külső szint a második argumentumot 7-re fejti ki.

A sztringesítés nem a forrás karakterhű másolata: például a belső whitespace-sorozatokat egyetlen szóközzé alakítja. Az összeillesztésnek érvényes előfeldolgozási tokent kell eredményeznie; nem alkalmas tetszőleges szövegek összeragasztására.

A kétlépcsős minta hasznos generált azonosítókhoz és diagnosztikai sztringekhez. Közönséges számításhoz azonban többnyire szükségtelen bonyolítás.

2.3.4 X-macro: egyetlen taglista több helyen

Az **X-macro** technika egy adatleírást egyetlen helyen, egy listamakróban rögzít, és ebből több kódrészletet generál: a struct deklarációját, a tagok számát, a név szerinti kiírást vagy egy szerializálót. A listamakró paramétere maga is egy makró (szokásosan `X`), amelyet minden felhasználási hely másképp definiál. Így egy új tag felvétele egyetlen sor, és a struct, a számláló és a kiírás nem csúszhat el egymástól.

```c
/* xmacro.c - C17: egyetlen taglistából struct, tagszám és kiírás */
#include <stddef.h>
#include <stdio.h>

/* A taglista egyetlen helyen: X(típus, név, printf-formátum). */
#define CONFIG_FIELDS(X)              \
    X(int,          port,    "%d")    \
    X(unsigned,     workers, "%u")    \
    X(const char *, bind,    "%s")

/* 1. A struct a listából épül. */
#define DECLARE_FIELD(type, name, fmt) type name;
struct config {
    CONFIG_FIELDS(DECLARE_FIELD)
};
#undef DECLARE_FIELD

/* 2. A tagszám a listából számolódik: minden tag egy "+ 1". */
#define COUNT_FIELD(type, name, fmt) + 1
enum { CONFIG_FIELD_COUNT = 0 CONFIG_FIELDS(COUNT_FIELD) };
#undef COUNT_FIELD

/* 3. A kiírás is a listából generálódik: a nevet a # operátor adja. */
#define PRINT_FIELD(type, name, fmt) \
    if (printf("%s=" fmt "\n", #name, c->name) < 0) return -1;
static int print_config(const struct config *c)
{
    CONFIG_FIELDS(PRINT_FIELD)
    return 0;
}
#undef PRINT_FIELD

_Static_assert(CONFIG_FIELD_COUNT == 3, "a taglista három elemű");

int main(void)
{
    const struct config c = { 8080, 4u, "127.0.0.1" };
    printf("fields=%d\n", CONFIG_FIELD_COUNT);
    return print_config(&c) == 0 ? 0 : 1;
}
```

A `CONFIG_FIELDS(DECLARE_FIELD)` előfeldolgozás után `int port; unsigned workers; const char * bind;` lesz, a tagszám pedig a `0 + 1 + 1 + 1` konstans kifejezés. A kiírásban a `#name` sztringesítés (lásd fent) adja a tag nevét. Minden felhasználás után `#undef`-eljük a segédmakrót, hogy a következő felhasználás saját definíciót adhasson neki.

A technika korlátai: a lista és a típus szinkronja csak addig garantált, amíg minden tag ténylegesen a listából jön létre; a hibaüzenetek a makrókifejtésre mutatnak, nem a listára; a debugger és sok eszköz a kifejtett kódot látja. A C++26 statikus reflexiója (C++26 nyelvi kitekintés) éppen ezt a kézi listát váltja ki: ott a tagok listája magából a típusból kérdezhető le.

Buildjegyzőkönyv (`lab-gcc` profil, `gcc -std=c17 -Wall -Wextra -Wpedantic -O2`). A program figyelmeztetés nélkül fordult, kimenete: `fields=3`, `port=8080`, `workers=4`, `bind=127.0.0.1`.

A `gcc -E -P` kimenete a fenti struct-deklarációt és `CONFIG_FIELD_COUNT = 0 + 1 + 1 + 1`-et mutatta.

2.3.5 Variadikus makrók és szabványverziók

ISO C99-től, így C17-ben is használható variadikus makró:

```
/* C17-részlet; <stdio.h> szükséges. */
#define LOG(...) fprintf(stderr, __VA_ARGS__)
```

Például a `LOG("ready\n")` és a `LOG("attempt=%d\n", attempt)` egyaránt használható. A `__VA_ARGS__` a változó argumentumrész tokenjeit helyettesíti be.

Rögzített paraméter után más a helyzet:

```
#define TRACE(fmt, ...) fprintf(stderr, fmt, __VA_ARGS__)
```

C17-ben a `TRACE("ready\n")` elhagyott variadikus argumentummal nem szabványos használat. A `TRACE("ready\n", )` pedig a helyettesítés után fölösleges vesszőt hagy a függvényhívásban.

ISO C23-ban a `__VA_OPT__` erre szabványos megoldást ad:

```
/* Kizárólag C23-részlet. */
#define TRACE(fmt, ...) \
    fprintf(stderr, fmt __VA_OPT__(,) __VA_ARGS__)
```

A vessző csak nem üres variadikus tokenlista esetén jelenik meg. A GNU-féle `, ##__VA_ARGS__` vesszőelnyelő megoldás fordítóbővítés, nem C17-es helyettesítője ennek.

A C17-es fallback a teljes argumentumlistát fogadó `LOG(...)`, vagy két külön nevű makró. A C23-as alak használata előtt a tényleges fordítótámogatást külön ellenőrizni kell; ezt a Preprocessor-labor és feature detection folytatja.

2.3.6 Több utasítás egyetlen szintaktikai egységként

Két csupasz utasítást helyettesítő makró megtörheti az `if`–`else` szerkezetet. A szokásos burkolás:

```
/* C17-részlet. */
#define ADD_AND_COUNT(total, count, value) \
    do {                                   \
        (total) += (value);                 \
        ++(count);                         \
    } while (0)
```

Használata:

```
/* C17-részlet. */
if (accepted)
    ADD_AND_COUNT(total, count, value);
else
    ++rejected;
```

A `do` törzse egyszer végrehajtódik, majd a nulla feltétel lezárja a ciklust. Az egész szerkezet egyetlen utasítás; a pontosvesszőt a hívó írja.

A példa szerződése szerint `total` és `count` módosítható egész változók, és egyik aritmetikai művelet sem lép túl a típus megengedett tartományán. A burkolás csak szintaktikai védelmet ad: nem biztosít típusbiztonságot, nem javítja a paraméterek ismételt használatát, és a benne szereplő `break` vagy `continue` célpontját is befolyásolja.

A makróba bevezetett lokális név ráadásul ütközhet a hívó nevével. A projektprefix csökkenti ezt a kockázatot, de nem hoz létre valódi névhigiéniát.

2.3.7 Feltételes fordítás és konfigurációs szerződés

A `#if` fordítás előtt választ tokenszakaszokat; nem futásidejű `if`. A `#ifdef NAME` csak a makró létezését vizsgálja. Ezért egy nullára definiált kapcsolóra is igaz.

Egy C17-es konfigurációs részlet:

```c
/* app_trace.c -- C17: APP_TRACE csak a 0 vagy 1 token lehet */
#include <stdio.h>

#ifndef APP_TRACE
#define APP_TRACE 0
#endif

/* Tokenellenőrzés: csak APP_TRACE_OK_0 és APP_TRACE_OK_1 létezik, minden más
   token (2, banana) definiálatlan nevet ad, ami a #if-ben 0 -> #error. */
#define APP_TRACE_OK_0 1
#define APP_TRACE_OK_1 1
#define APP_CAT_(a, b) a##b
#define APP_CAT(a, b) APP_CAT_(a, b)
#if !APP_CAT(APP_TRACE_OK_, APP_TRACE)
#error "APP_TRACE must be exactly the token 0 or 1"
#endif

#if APP_TRACE
#define APP_LOG(...) fprintf(stderr, __VA_ARGS__)
#else
#define APP_LOG(...) ((void)0)
#endif

int main(void) { APP_LOG("trace on\n"); return 0; }
```

A konfiguráció szerződése: az `APP_TRACE` pontosan a `0` vagy az `1` token. Egy egyszerűbb `#if (APP_TRACE != 0) && (APP_TRACE != 1)` ellenőrzés ezt **nem** biztosítja: a `#if` a makrókifejtés után megmaradó azonosítót nullának veszi, így `-DAPP_TRACE=banana` mellett a feltétel hamis, nincs `#error`, és a naplózás csendben kikapcsol. A fenti tokenellenőrzés ezért a kapott tokent egy névhez fűzi: csak az `APP_TRACE_OK_0` és az `APP_TRACE_OK_1` létezik, minden más token definiálatlan nevet ad, ami a `#if`-ben nulla, és `#error`-t vált ki. A laborban emellett a `-Wundef -Werror=undef` kapcsoló az összes `#if`-ben elgépelt azonosítót hibává teszi.

Buildjegyzőkönyv (`lab-gcc` profil). `gcc -std=c17 -Wall -Wextra -Wpedantic` mellett az alapérték, a `-DAPP_TRACE=0` és a `-DAPP_TRACE=1` lefordult; a `-DAPP_TRACE=2` és a `-DAPP_TRACE=banana` `#error "APP_TRACE must be exactly the token 0 or 1"` hibával

elutasult. A régi, `!= 0 && != 1` alakú ellenőrzés `banana`-ra hiba nélkül lefordult; `-Wundef -Werror=undef` mellett viszont `"banana" is not defined, evaluates to 0 [-Werror=undef]` hibát adott. A GCC/Clang `-DAPP_TRACE=1` kapcsolója fordítóeszköz-funkció, nem C nyelvi szintaxis.

A `defined(APP_TRACE)` az előfeldolgozási feltételben a definíció létezését vizsgálja; a `#ifdef APP_TRACE` ennek egyszerűbb alakja. A `#error` diagnosztikát követel meg, ezért alkalmas tiltott konfiguráció jelzésére; a buildben ezt hibának kell kezelni.

A `#if` makrókifejtése után megmaradó közönséges azonosítókat az előfeldolgozó nullának tekinti. Egy elgépelés így észrevétlenül kikapcsolhat egy ágat. A GCC/Clang `-Wundef` figyelmeztetése ebben segíthet. C-változó, `enum`-konstans vagy `sizeof` nem használható úgy, mintha a `#if` a C fordító teljes kifejezésértelmezője volna.

Fontos hibaminta:

```
/* C17-részlet: konfigurációfüggő mellékhatás! */
APP_LOG("processed=%d\n", ++processed);
```

Kikapcsolt naplózásnál a növelés is eltűnik. A program szükséges állapotváltozását külön utasításba kell tenni. Mindkét konfigurációt fordítani és ellenőrizni kell: az inaktív ág sikeres kihagyása nem bizonyítja annak helyességét.

2.3.8 Feladatok

2.3.8.1 Kifejtés és precedencia

Fejtsd ki a `#define DOUBLE(x) x + x` makróval a `3 * DOUBLE(2 + 1)` kifejezést! Mi az értéke? Javítsd a zárójelezést, majd indokold meg, miért veszélyes továbbra is a `DOUBLE(n++)`.

2.3.8.2 Függvényes alternatíva

Írj `static inline int twice_int(int x)` függvényt a $[-1000, 1000]$ bemeneti tartományra. Milyen eredményt és végső `n` értéket vársz, ha kezdetben `n = 4`, és a hívás `twice_int(n++)`?

2.3.8.3 Konfigurációs diagnózis

Miért jelez hibát az alábbi, szándékosan hibás C17-részlet? Javítsd úgy, hogy csak az `APP_TRACE` egyes értéke legyen tiltott!

```
#define APP_TRACE 0

#ifdef APP_TRACE
#error "Trace mode is forbidden in this build"
#endif
```

2.3.9 Megoldások

2.3.9.1 Kifejtés és precedencia

A kifejtés `3 * 2 + 1 + 2 + 1`, értéke 10. A javítás:

```
#define DOUBLE(x) ((x) + (x))
```

Ezzel az eredmény $3 \cdot (3 + 3) = 18$. Az ellenőrzési feltétel a tokenkifejtésből kapott műveleti sorrend. A `DOUBLE(n++)` továbbra is két, egymáshoz képest nem megfelelően rendezett módosítást eredményez, tehát UB.

2.3.9.2 Függvényes alternatíva

```
/* C17-részlet; a macro_demo.c main függvénye elé illeszthető. */
static inline int twice_int(int x)
{
    return x + x;
}
```

A megadott tartományban az eredmény $[-2000, 2000]$, amelyet minden szabványos C17-es `int` képes ábrázolni. Az argumentum egyszer értékelődik ki: az eredmény `8`, a végső `n` értéke `5`. Ez a két érték az ellenőrzési feltétel.

2.3.9.3 Konfigurációs diagnózis

Az `APP_TRACE` definiálva van, ezért a `#ifdef` ága aktív, függetlenül a nulla értéktől. A javítás:

```
#if APP_TRACE
#error "Trace mode is forbidden in this build"
#endif
```

Ellenőrzési feltétel: `0` mellett ez a direktíva ne okozzon diagnosztikát, `1` mellett okozzon. Külső buildkonfigurációhoz a fix definíció helyett a korábbi `#ifndef`-es alapértékminta szükséges.

2.3.10 Összefoglaló

A makró tokenhelyettesítés, nem függvényhívás. A zárójelezés védi a precedenciát, de nem akadályozza meg a mellékhatások ismétlését. Számításokhoz általában típusellenőrzött függvényt válassz; makrót ott használj, ahol valóban előfeldolgozás szükséges. A `#`, `##`, variadikus argumentumok és konfigurációs kapcsolók esetén a kifejtett tokensorozatot vizsgáld, ne csupán a hívás látszólagos jelentését.

2.4 Preprocessor-labor és feature detection

Az előfeldolgozó hibáit gyakran nem az eredeti forrás, hanem a fordítónak ténylegesen átadott tokensorozat teszi érthetővé. Egy makró neve helyes műveletet sugallhat, miközben a helyettesítés mást eredményez; egy konfigurációs makró pedig már szerepelhet a végső makrólistában, noha a rá támaszkodó header feldolgozásakor még nem létezett.

A Makrók, feltételes fordítás és veszélyes helyettesítés szabályaira építve most bizonyítékokat gyűjtünk: előfeldolgozott forrást, makrólistát és include-nyomvonalat. Ezekből különítjük el a helyettesítési hibát, az include-sorrend hibáját és a platformképességekre vonatkozó téves feltételezést.

2.4.1 Laborprofil és megfigyelési eszközök

A példák alapja **ISO C17**. A választott konkrét laborprofil Linux, x86-64 System V ABI, LP64 adatmodell, a `lab-next` profil (profiltábla); a két fordító a `lab-next` profil (profiltábla). Más kiadás is használható, de a tényleges verziót és targetet rögzíteni kell. Az alábbi tokenrészletek levezetett várakozások, nem elvégzett fordítási mérések.

Az **előfeldolgozott forrás** (*preprocessed source*) az include-ok, feltételes fordítási ágak és makróhelyettesítések feldolgozása után kapott szöveges reprezentáció. Nem őrzi meg a forrás minden szerkezeti információját, viszont megmutatja, milyen deklarációk és kifejezések jutnak tovább.

A következő kapcsolók **GCC/Clang-eszközfunkciók**, nem ISO C szintaktikai elemek:

Kapcsoló	Mit mutat?	Mire nem elegendő?
`-E`	Az előfeldolgozás eredményét	Nem végez teljes típusellenőrzést vagy linkelést
`-E -dM`	Az előfeldolgozás végén fennálló makródefiníciókat	Nem rekonstruálja a makróállapot történetét
`-H`	A ténylegesen beolvasott headerek beágyazását	Nem bizonyítja az interfészek helyességét

A `-H` nyomvonala a standard hibakimenetre kerül. Formátuma és a rendszerheaderek útvonala verzió- és környezetfüggő.

2.4.2 Kidolgozott labor: két egymástól független hiba

Készítsd el az alábbi három fájlt ugyanabban a könyvtárban. Ez **szándékosan hibás, de a megadott kifejezésekkel nem UB-t bemutató C17 tanítási példa**.

Az `agent_config.h`:

```
#ifndef AGENT_CONFIG_H
#define AGENT_CONFIG_H

#ifndef AGENT_WIDE
#define AGENT_WIDE 1
#endif

#define AGENT_SCALE(x) x * 2
```

```
#endif
```

Az `agent_api.h`:

```
#ifndef AGENT_API_H
#define AGENT_API_H

#if defined(AGENT_WIDE) && AGENT_WIDE
enum { AGENT_CAPACITY = 32 };
#else
enum { AGENT_CAPACITY = 16 };
#endif

struct agent_record {
    unsigned char bytes[AGENT_CAPACITY];
};

#endif
```

A `main.c`:

```
#include "agent_api.h"
#include "agent_config.h"
#include <stdio.h>

#define DIAG_INT(value) \
    ((void)fprintf(stderr, "%s:%d:%s: %s=%d\n", \
                   __FILE__, __LINE__, __func__, #value, (value)))

int main(void)
{
    int scaled = AGENT_SCALE(1 + 3);

    DIAG_INT(scaled);
    DIAG_INT(AGENT_CAPACITY);
    return 0;
}
```

A fejlesztő szándéka szerint az alapértelmezett kapacitás 32, a skálázott érték pedig $(1 + 3) \cdot 2 = 8$. A program azonban egyik szándékot sem valósítja meg.

2.4.2.1 Bizonyítékgyűjtés

Az alábbi Bash-parancsok külön laborlépések. A `CC` értékét a rögzített kiadást kiválasztó parancsra állítsd; a teljes sorozatot Clanggal is ismételd meg, külön munkakönyvtárban.

```
CC=gcc
FLAGS=(-std=c17 -Wall -Wextra -Wpedantic)

"$CC" --version > compiler.txt
"$CC" -dumpmachine >> compiler.txt
```

```
"$CC" "${FLAGS[@]}" main.c -o agent
"$CC" "${FLAGS[@]}" -E main.c -o main.i
"$CC" "${FLAGS[@]}" -E -dM main.c > macros.txt
"$CC" "${FLAGS[@]}" -E -H main.c \
    -o /dev/null 2> includes.txt

grep -n -E 'AGENT_CAPACITY|int scaled' main.i
```

A `main.i` rendszerheaderekből származó részei hosszúak lehetnek. Elsőként a saját azonosítóidra keress. Az alapértelmezett kimenet forráshelyjelöléseit érdemes megtartani: ezek kapcsolják vissza a tokensorozatot az eredeti fájlokhoz. A GCC/Clang `-P` kapcsolója olvashatóbb kimenetet adhat, de eltávolítja ezeket a jelöléseket.

2.4.2.2 A hiba okának bizonyítása

A lényeges, kézzel levezetett tokenrészletek:

```
/* Előfeldolgozott részletek, nem önálló program. */
enum { AGENT_CAPACITY = 16 };

int scaled = 1 + 3 * 2;
```

Bizonyításvázlat a makróhibához. Az `AGENT_SCALE` helyettesítési listája `x * 2`. Az argumentum tokenjei `1 + 3`; a helyettesítés nem tesz köréjük automatikusan zárójelet. Ezért:

$$1 + 3 * 2 = 1 + (3 * 2) = 7.$$

A hiba már az előfeldolgozott kifejezésben jelen van, tehát nem az optimalizáló változtatta meg a helyes számítást.

Bizonyításvázlat az include-sorrend hibájához. Az `agent_api.h` feldolgozásakor az `AGENT_WIDE` még nincs definiálva. A feltétel hamis, ezért a 16-os ág marad meg. A későbbi `agent_config.h` már nem értékeli újra a korábbi direktívát. Emiatt egyszerre lehet igaz, hogy:

- a `macros.txt` végső állapotában `AGENT_WIDE` értéke 1;
- a `main.i` tényleges deklarációjában `AGENT_CAPACITY` értéke 16.

A végső makrólista tehát önmagában félrevezető bizonyíték.

2.4.2.3 Javítás és ellenőrzési feltétel

A makró javított definíciója az `agent_config.h` fájlban:

```
#define AGENT_SCALE(x) ((x) * 2)
```

Ez a precedenciaproblémát javítja. Az aritmetikai művelet helyességéhez továbbra is szükséges, hogy az eredmény ábrázolható legyen a művelet típusában.

Az include-hiba tartós javítása nem pusztán a két include felcserélése. Az `agent_api.h` saját include guardján belül, a feltételes ág előtt maga olvassa be a függőségét:

```
#include "agent_config.h"
```

Így a header önmagában is rendelkezik a saját deklarációihoz szükséges konfigurációval. Az ellenőrzési feltétel: alapértelmezett konfigurációnál az előfeldolgozott deklaráció kapacitása 32, a kifejezés pedig `((1 + 3) * 2)` legyen.

2.4.3 Parancssori konfiguráció: `-D` és `-U`

A GCC/Clang `-DNAME=value` kapcsolója a forrás feldolgozása előtt definiálja a makrót. A `-DNAME` rövid alak értéke 1. A `-UNAME` megszünteti a definíciót; a `-D` és `-U` kapcsolók egymáshoz viszonyított sorrendje számít.

A **javított** headerrel:

```
"$CC" "${FLAGS[@]}" -UAGENT_WIDE -DAGENT_WIDE=0 \
    -E main.c -o narrow.i

"$CC" "${FLAGS[@]}" -DAGENT_WIDE=0 -UAGENT_WIDE \
    -E main.c -o default.i
```

Az első esetben a konfigurációs header már definiált, nulla értékű makrót talál, ezért a kapacitás 16. A másodikban nincs kezdeti definíció, így a header beállítja az alapértelmezett 1-et, és a kapacitás 32.

Két gyakori tévedés következik ebből. A `-U` nem tiltja meg, hogy a forrás később újradefiniálja a nevet. Továbbá a `#ifdef AGENT_WIDE` nulla értéknél is igaz: a definíció létezését, nem az értékét vizsgálja.

2.4.4 Forráshely és diagnosztikai helper

ISO C17: a `__FILE__` előre definiált makró az aktuális feltételezett forrásfájl nevét adja sztringliterálként, a `__LINE__` pedig a forrássor számát egész konstansként. Ezeket a `#line` direktíva is befolyásolhatja, ezért nem feltétlenül a fizikai fájl változatlan helyazonosítói.

A `__func__` ezzel szemben **nem makró**. Függvényen belüli, előre definiált azonosító; szemantikailag statikus tárolási idejű, konstans karaktertömböt jelöl a függvény nevével. Emiatt nem várható el, hogy a `-dM` listájában szerepeljen.

A `DIAG_INT` helper szerződése szűk:

- csak a `%d` formátumnak megfelelő, alapértelmezett argumentumpromóció után `int` típusú értéket fogad;
- argumentumát egyszer értékeli ki; a `#value` csak a tokenek sztringesítését végzi;
- kizárólag függvényen belül használható a `__func__` miatt;
- az I/O-hibát szándékosan figyelmen kívül hagyja, tehát nem megbízható naplózási alrendszer.

A diagnosztikai sztringek statikus tárolási idejűek; nincs tulajdonátadás. A `stderr` könyvtár által biztosított stream, amelyet ez a példa nem zár le. Egy korábban lezárt stream használata megsértené a helper előfeltételét.

2.4.5 Platformteszt helyett képességvizsgálat

A **platformteszt** környezeti kategóriát azonosít. A **képességvizsgálat** (*feature detection*) azt ellenőrzi, hogy a programhoz szükséges konkrét nyelvi vagy könyvtári művelet rendelkezésre áll-e.

Vizsgálat	Mit jelent?	Mit nem bizonyít?
`__linux__`	A fordító Linux targetet jelez	Egy adott libc API elérhetőségét
`__clang__`	Clang fordítócsaládot jelez	Egy adott bővítés támogatását
`__GNUC__`	GNU-kompatibilitást jelző makró	Önmagában nem bizonyítja, hogy GCC fordít
`__STDC_VERSION__`	Az ISO C nyelvi változat jelzése	POSIX API vagy futásidejű szolgáltatás meglétét
Fordítási/linkelési próba	A kipróbált használat elfogadhatóságát	A futásidejű sikerességet

A libc **feature-test makrója** más fogalom: gyakran nem képességet jelent be, hanem azt kéri, hogy a headerek mely interfészeket tegyék láthatóvá.

2.4.5.1 POSIX-interfész fordítási próbája

A következő önálló `probe_clock.c` **C17 + POSIX** példa, nem tisztán ISO C:

```c
#include <time.h>

int main(void)
{
    struct timespec ts;
    return clock_gettime(CLOCK_MONOTONIC, &ts) == 0 ? 0 : 1;
}
```

Fordítás és linkelés a választott Linux/glibc profilhoz:

```
"$CC" "${FLAGS[@]}" -D_POSIX_C_SOURCE=200809L \
    probe_clock.c -o probe_clock
```

A `ts` a `main` automatikus tárolási idejű objektuma. A hívás csak a hívás időtartamára kap kölcsönzött hozzáférést; nincs tulajdonátadás. Élettartama a blokk végén megszűnik. Hibánál a program nem olvassa a tartalmát, hanem 1-et ad vissza.

A `_POSIX_C_SOURCE` definíciójának **minden rendszerheader előtt** kell érvényesülnie. Nem elég közvetlenül a `<time.h>` elé tenni, ha például a `<stdio.h>` már korábban beolvasta a libc közös konfigurációs headereit. A parancssori definíció ezt az időzítési hibát elkerüli.

A sikeres fordítás és linkelés azt igazolja, hogy ez a használat az adott header-, kapcsoló- és könyvtárkészlettel elfogadható. Nem igazolja, hogy a függvény futás közben sikeres lesz. Régebbi vagy más libc-profilnál a szükséges linkkönyvtárak is eltérhetnek. A csak `-fsyntax-only` kapcsolóval végzett próba pedig még a linkelhetőséget sem ellenőrzi.

2.4.6 Makrókonfiguráció mint ABI-bemenet

A konfiguráció nem mindig csupán algoritmust választ: nyilvános típusokat is módosíthat.

Az alábbi táblázat a javított laborheaderre vonatkozik, a megadott x86-64 System V/LP64 profilban. A méret a **tömbtagé**, nem mért teljes struktúraméret.

`AGENT_WIDE`	A `bytes` tag típusa	A tag mérete
0	`unsigned char[16]`	16 C bájt
1	`unsigned char[32]`	32 C bájt

Ha egy könyvtár és kliense eltérő értékkel dolgozza fel ezt az interfészt, eltérően értelmezheti ugyanazt a határon átadott objektumot. A linker rendszerint nem végez teljes C-típusellenőrzést; a sikeres linkelés nem zárja ki az inkompatibilis deklarációkat vagy a későbbi hibás memóriahozzáférést.

Hasonló, implementációspecifikus példa a glibc `_FILE_OFFSET_BITS` beállítása: egyes 32 bites profilokon típusokat és interfészleképezéseket változtat, míg a szokásos 64 bites LP64 környezetben nem ugyanilyen változást okoz.

A reprodukcióhoz ezért a forrásverzió mellett őrizd meg a teljes fordítóparancsot, a kapcsolók sorrendjét, a konfigurációs headereket, az include-útvonalakat, a sysrootot, a fordító- és libc-verziót. A makrólista hasznos pillanatfelvétel, de nem helyettesíti ezeket.

2.4.7 Feladatok

2.4.7.1 Magyarázó feladat

Miért nem cáfolja az include-sorrend hibáját, ha a `macros.txt` szerint `AGENT_WIDE` értéke 1? Nevezz meg két további bizonyítékot.

2.4.7.2 Kódolási feladat

Javítsd mindkét hibát. Ezután készíts külön előfeldolgozott fájlt `AGENT_WIDE=0` és `AGENT_WIDE=1` beállítással. A header működése ne függjön a kliens include-sorrendjétől.

2.4.7.3 Diagnosztikai feladat

Egy projekt Linuxon fordul, ezért automatikusan engedélyezi a `clock_gettime` használatát. A `_POSIX_C_SOURCE` definícióját a `<stdio.h>` után helyezi el. Magyarázd meg a két külön hibát, és tervezz ellenőrzést futtatás nélkül.

2.4.8 Megoldások

2.4.8.1 A végső makróállapot értelmezése

A makrólista az előfeldolgozás végi állapotot mutatja. A korábbi feltételes ág kiválasztását nem módosítja egy későbbi definíció.

Az include-nyomvonal bizonyítja a fájlok feldolgozási sorrendjét, a `main.i` pedig megmutatja a ténylegesen megmaradt 16-os deklarációt. Az ellenőrzés akkor teljes, ha mindhárom adat ugyanahhoz a fordítóparancshoz tartozik.

2.4.8.2 A javított konfiguráció ellenőrzése

Az `agent_api.h` maga include-olja az `agent_config.h` fájlt; a skálázó makró `((x) * 2)` alakú.

A két előfeldolgozásban a kapacitás rendre 16 és 32, a skálázott kifejezés mindkettőben `((1 + 3) * 2)`. További ellenőrzésként egy kizárólag az `agent_api.h` fájlt include-oló C17 fordítási egységnek is elfogadhatónak kell lennie.

2.4.8.3 A képességpróba javítása

A Linux-target azonosítása nem bizonyítja a szükséges libc-interfész láthatóságát vagy linkelhetőségét. Ettől függetlenül a késői feature-test definíció hatástalan lehet.

A javítás: `_POSIX_C_SOURCE=200809L` beállítása minden rendszerheader előtt, majd a bemutatott próba fordítása és linkelése a valódi target környezetében. Ellenőrzési feltétel a sikeres build; futásidejű működést ebből nem szabad állítani.

2.4.9 Összefoglaló

A `-E` a tényleges tokensorozatot, a `-dM` a végső makróállapotot, a `-H` az include-feldolgozás nyomvonalát mutatja. Együtt alkalmasak az előfeldolgozási hibák okának bizonyítására.

Az OS azonosítása, a fordító felismerése, az interfészek láthatóságának kérése és a képesség tényleges vizsgálata külön feladat. A konfigurációs makrókat pedig buildbemenetként kell kezelni: némelyikük az ABI-szerződés része.

2.5 Warningok, optimalizálási profilok és link-driver

Egy fordítóparancs nem pusztán kapcsolók gyűjteménye: egy buildlépés szerződését írja le. Meghatározza az elfogadott nyelvet, a keresett hibamintákat, az optimalizálási célt és azt, hogy milyen további eszközök kapják meg a feldolgozás eredményét. Ugyanaz a kapcsolólista nem megfelelő minden projekthez, fordítóverzióhoz és buildfázishoz.

A Fordítási fázisok és fordítási egységek rész alapján különítsük el a forrás feldolgozását, az objektumfájl előállítását és a végső linkelést. A Reprodukálható fordítási és diagnosztikai labor elvét követve a profilhoz mindig fordító-, target- és eszközverzió is tartozzon.

2.5.1 Diagnosztika: mit jelent egy warning?

ISO C17-szabály: a *constraint* a nyelvi leírásban kifejezetten előírt korlátozás. Például ha látható az `int f(int);` prototípus, akkor az `f(1, 2)` hívás megsérti az argumentumok számára vonatkozó követelményt.

Ha egy fordítási egység szintaktikai szabályt vagy constraintet sért, a megfelelő C-implementációnak legalább egy diagnosztikai üzenetet kell adnia. Ez nem jelent szükségképpen üzenetet minden hibáról, és nem írja elő általánosan a fordítás leállítását.

Ezzel szemben a **warning** a fordító diagnosztikai kategóriája. Jelölhet kötelezően diagnosztizálandó nyelvsértést, de figyelmeztethet teljesen szabályos, mégis gyanús kódra is. Egy használatlan lokális változó például önmagában nem teszi hibássá a C-programot.

GCC/Clang-kapcsolók: a következő nevek nem az ISO C részei. Pontos tartalmuk és alapértelmezésük verziófüggő.

Kapcsoló	Rendeltetés	Fontos korlát
`-Wall`	Sok gyakori hibaminta diagnosztikáját bekapcsolja	Neve ellenére nem minden warningot kapcsol be
`-Wextra`	További, általában hasznos diagnosztikákat engedélyez	Nem jelenti az összes fennmaradó warningot
`-Wpedantic`	A kiválasztott ISO nyelvi módhoz kapcsolódó, többek között kiterjesztéseket érintő diagnosztikákat kér	Nem teljes szabványossági bizonyítás
`-Wconversion`	Értéket esetleg megváltoztató implicit konverziókra figyelmeztet	Tudatos konverziók mellett is sok jelzést adhat
`-Wshadow`	Más deklarációkat elfedő neveket jelez	Pontos lefedettsége fordítónként eltérhet

A `-std=c17` és a `-Wpedantic` együtt értelmezendő: az előbbi választja ki a nyelvi módot. A warningmentes build azonban sem a hordozhatóságot, sem a nem definiált viselkedés hiányát nem bizonyítja.

A `-Wconversion` és a `-Wshadow` különösen hasznos saját kódon, de örökölt projektben célszerű fokozatosan bevezetni őket. Egy explicit cast elnémíthat egy konverziós warningot, de **nem teszi automatikusan helyessé a konverziót**.

A `-Werror` a warningokat buildhibává emeli. Rögzített CI-eszközláncon ez erős minőségi kapu lehet; változó fordítóverzió mellett viszont egy új diagnosztika korábban elfogadott buildet is megállíthat. A GCC és Clang `-pedantic-errors` kapcsolója bizonyos, a szabvány által megkövetelt diagnosztikákat hibává emel; nem egyszerű szinonimája a `-Werror` kapcsolónak.

2.5.2 Optimalizálási szintek és debug információ

Az optimalizálási szint azt szabályozza, milyen átalakításokat mérlegeljen a fordító. Nem a program helyességi fokozata, és nem univerzális teljesítményskála.

Szint	Jellemző cél	Mire számítsunk?
`-O0`	Az optimalizálások nagy részének mellőzése	Egyszerűbb forrásszintű követés; nem garantál szó szerinti forrás–utasítás megfelelést
`-Og`	Hibakeresést támogató optimalizálás	Hasznos kompromisszum lehet; GCC és Clang megvalósítása eltérhet
`-O1`	Mérsékelt optimalizálás	Korlátozottabb fordítási költséggel végez átalakításokat
`-O2`	Általános teljesítményorientált profil	Gyakori release-kiindulópont
`-O3`	További, agresszívebb optimalizálás	Növelheti a kódméretet, és nem garantáltan gyorsabb
`-Os`	Kódméretre hangolt optimalizálás	Nem garantálja sem a legkisebb fájlt, sem a legjobb futásidőt

A pontos optimalizálási lépések fordító-, verzió- és targetfüggők. Különösen a `-Og` jelentését nem szabad két fordító között azonosnak tekinteni.

A **debug információ** olyan metaadat, amely a gépi program elemeit forrásfájlokhoz, sorokhoz, típusokhoz és változókhoz kapcsolja. A `-g` ennek előállítását kéri; nem kapcsolja ki az optimalizálást.

Indokolt kiinduló profilok:

- **Fejlesztői profil:** `-std=c17 -Og -g`, megfelelő warningokkal.
- **Forráskövetési kontrollprofil:** `-std=c17 -O0 -g`, ha az optimalizált hibakeresés félrevezetőnek tűnik.
- **Release-profil:** `-std=c17 -O2 -g`, ugyanazzal a diagnosztikai alappal.

Release-ben a `-g` incidenselemzéskor értékes. Az optimalizált változó ettől még eltűnhet, több változó osztozhat ugyanazon tárolón, és a debugger léptetése eltérhet a forrássorrendtől. A debug információ jelenléte nem állítja vissza az optimalizálás előtti programot.

A release-profilból nem következik automatikusan a `-DNDEBUG`: az külön döntés, mert megváltoztatja az `assert` használatának hatását. A `-O3` választását pedig reprezentatív mérésnek kell indokolnia, nem a nagyobb számnak.

2.5.3 Kapcsolók buildfázis szerint

A **compiler driver**, például a `gcc`, vezérlőprogram: a bemenet és a kapcsolók alapján meghívja a szükséges fordítási eszközöket.

Kapcsoló	Elsődleges szerep
`-std=c17`, `-W...`	Nyelvi feldolgozás és diagnosztika
`-Iinclude`, `-DNAME=value`	Headerkeresés és preprocessor-konfiguráció
`-O2`, `-g`	Kódgenerálás és debug információ
`-c`	Objektumfájl előállítása, végső linkelés nélkül
`-Ldir`	Linkeléskor használt könyvtárkeresési útvonal

Kapcsoló	Elsődleges szerep
`-lname`	Könyvtár kiválasztása a linker számára
`-Wl,...`	Argumentumok továbbítása a linkernek

A `-I` és a `-L` tehát nem felcserélhető. Az előbbi deklarációkat tartalmazó headerek kereséséhez kell; az utóbbi a linkelhető könyvtárakhoz. A header megtalálása nem biztosítja a benne deklarált függvény implementációját.

Linux/GNU eszközláncon a `-lwave` szokásosan például `libwave.so` vagy `libwave.a` nevű fájl kiválasztásához vezet. A keresés részletei és a statikus vagy megosztott változat választása a linkelési beállításoktól függ.

A

```
-Wl,-Map,build/app.map
```

driverargumentum a linkernek két argumentumot továbbít: `-Map` és `build/app.map`. Itt a vesszők a driver által feldolgozott elválasztók.

A `-L` nem általános futásidejű könyvtárkeresési beállítás. A dinamikus betöltés külön kérdés; ezt a Statikus és megosztott könyvtárak rész bontja ki.

Fontos következmény: hagyományos, linkidőbeli optimalizálás nélküli buildben a már elkészült `.o` fájlok mellé tett `-O2` nem optimalizálja újra a függvényeket. A release-profilhoz a forrásokat is újra kell fordítani.

Egyes kapcsolók több fázist érintenek. Például a támogatott sanitizerprofiloknál a megfelelő beállítás rendszerint fordításkor és a végső driveres linkeléskor is szükséges. Ezért a kapcsolókat jelentésük, nem pusztán nevük alapján kell szétválasztani.

2.5.4 Kidolgozott példa: külön fordítás és driveres linkelés

Laborprofil: ISO C17; Linux x86-64, System V ABI, LP64; a `lab-next` profil (profiltábla). Összehasonlító fordítóként a `lab-next` profil (profiltábla) használható ugyanazzal a rendszerkörnyezettel. Ezek kijelölt laborverziók, nem itt végzett fordítás eredményei. A parancsok feltételezik, hogy a `gcc` és az `ar` ezekre a rögzített eszközökre mutat.

A példa három fájlból áll. Az `include/wave.h`:

```
#ifndef BOOK_WAVE_H
#define BOOK_WAVE_H

double wave_sample(double radians);

#endif
```

Az `src/wave.c`:

```
#include "wave.h"
#include <math.h>

double wave_sample(double radians)
{
    return sin(radians);
}
```

Az `src/main.c`:

```c
#include "wave.h"

int main(void)
{
    double sample = wave_sample(0.5);
    return sample > 0.0 ? 0 : 1;
}
```

A program értékeket ad át; nem kezel dinamikus erőforrást vagy kölcsönzött mutatót. A hívás véges, érvényes argumentumot ad a matematikai függvénynek. A várt sikerjelzés nulla visszatérési érték, nem itt megfigyelt futási eredmény.

A fájlok létrehozása után, tiszta buildkönyvtárból:

```
mkdir -p build/debug

gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
    -Og -g -Iinclude -c src/wave.c -o build/debug/wave.o

gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
    -Og -g -Iinclude -c src/main.c -o build/debug/main.o

ar rcs build/debug/libwave.a build/debug/wave.o

gcc -fuse-ld=bfd build/debug/main.o \
    -Lbuild/debug -lwave -lm \
    -Wl,-Map,build/debug/app.map -o build/debug/app
```

Az `ar` archívumot készít; ez nem a végső program linkelése. Az utolsó parancsban a `gcc` driver hívja meg a GNU BFD linkert.

Az argumentumok szerepe lépésenként:

1. A `main.o` hivatkozik a `wave_sample` függvényre.
2. A `-Lbuild/debug -lwave` elérhetővé teszi annak implementációját.
3. A `-lm` a profil matematikai könyvtárát adja hozzá a `sin` feloldásához.

A `-lm` buildkövetelmény nem ISO C-szabály, hanem ennek a rendszerkörnyezetnek a sajátossága. A könyvtárakat a rájuk hivatkozó bemenetek után helyezzük; különösen statikus archívumoknál számít a sorrend.

Release-változathoz mindkét forrást újra kell fordítani `-O2 -g` beállítással, elkülönített `build/release` útvonalakra, majd az archívumot és a programot is újra előállítani. A debug- és release-objektumok keverése ne legyen véletlen buildállapot.

2.5.5 Miért ne közvetlenül az `ld` linkeljen?

A linker nem ugyanazokat az alapértelmezéseket ismeri, mint a nyelvi driver. A driver a telepített eszközlánc konfigurációja alapján választja ki többek között:

- az induló és lezáró objektumfájlokat;
- a rendszerkönyvtárak keresési útvonalait;
- a fordító segéd-runtime-ját;

- a nyelvhez szükséges runtime-könyvtárakat.

Linux/GNU környezetben a C-folyamat rendszerint nem közvetlenül a `main` függvénynél indul. Ezért a puszta `ld main.o ...` parancs akkor is hiányos lehet, ha minden saját függvény implementációja rendelkezésre áll.

A `gcc` és a `g++` közötti különbség sem egyszerűen fájlkiterjesztés:

- A `gcc` telepített C++ frontenddel képes `.cpp` fájlt fordítani, de a végső linkeléskor nem adja automatikusan hozzá ugyanazokat a C++ könyvtárakat, mint a `g++`.
- GNU környezetben a `g++` automatikusan hozzáadja a C++ standard könyvtárat, tipikusan a `libstdc++`-t.
- A `g++` a `.c` bemenetet is alapértelmezetten C++-ként kezeli; ezért nem helyettesíti következmények nélkül a C-fordítás `gcc` parancsát.

Vegyes programnál a C-forrást C-ként, a C++-forrást megfelelő C++ módban fordítsuk, a végső linkelést pedig rendszerint a C++ driver végezze. Clang esetén ennek megfelelő pár a `clang` és a `clang++`; a tényleges C++ könyvtár konfigurációfüggő.

2.5.6 Feladatok

2.5.6.1 Magyarázó feladat

Egy fejlesztő szerint a `-O2 -g` értelmetlen, mert az optimalizált program nem hibakereshető. Javítsd az állítást, és javasolj fejlesztői, illetve release-profilt.

2.5.6.2 Kódolási feladat

Az alábbi, C17-es fordítási egységet csak diagnosztikai kiindulópontként kezeld:

```c
int accepted_count(double value)
{
    return value;
}
```

Módosítsd úgy, hogy csak a $[0, 100]$ tartományt fogadja el, törtrészt elhagyó konverzióval. Minden más bemenetre, NaN esetén is, adjon `-1` értéket. Ellenőrző fordítás:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -c count.c -o count.o
```

2.5.6.3 Diagnosztikai feladat

A laborprogramot valaki közvetlenül így linkeli:

```
ld build/debug/main.o -Lbuild/debug -lwave -lm -o app
```

Miért hibás buildstratégia ez? Hogyan javítanád? Elegendő-e a javított linkparancshoz `-O2` kapcsolót adni release készítéséhez?

2.5.7 Megoldások

2.5.7.1 A debug információ független döntés

Az optimalizált program hibakereshető, de a forrásszintű megfigyelések korlátozottabbak lehetnek. Indokolt fejlesztői kiindulópont a `-Og -g`, release-ben pedig a `-O2 -g`, mindkettőnél explicit `-std=c17` és warningprofil.

Ellenőrzési feltétel: a debugger forrássorokat tudjon azonosítani; ne követeljük meg minden változó folyamatos megfigyelhetőségét.

2.5.7.2 Ellenőrzött konverzió

C17-es megoldás:

```
int accepted_count(double value)
{
    if (!(value >= 0.0 && value <= 100.0)) {
        return -1;
    }

    return (int)value;
}
```

A feltétel NaN esetén is elutasít, mert a tartományba tartozást igazoló összehasonlítások nem teljesülnek. Az elfogadott tartomány minden egész eredménye ábrázolható `int` típusban; a konverzió a törtrészt elhagyja.

Ellenőrzési feltétel: például `0.0 → 0`, `12.75 → 12`, `100.0 → 100`; negatív, száznál nagyobb és NaN bemenetnél `-1`. A cast helyességét a tartományellenőrzés indokolja, nem a warning eltűnése.

2.5.7.3 A driver visszaállítása

A közvetlen `ld`-hívásból hiányozhatnak a startup- és runtime-összetevők, valamint szükséges keresési beállítások. A javítás a kidolgozott példa driveres linkparancsa.

Ellenőrzési feltétel: a linklépést a megfelelő nyelvi driver végezze. Release-hez minden érintett forrást újra kell fordítani a release-profillal; a meglévő debug-objektumok mellé írt `-O2` önmagában nem elegendő.

2.5.8 Összefoglaló

A warningprofil hibakeresési eszköz, nem helyességi bizonyíték. Az optimalizálást és a debug információt külön döntésként kezeljük. A kapcsolókat buildfázis szerint rendezzük: `-I` a headerekhez, `-L` és `-l` a linkeléshez, `-Wl,` pedig a linkerargumentumok továbbításához tartozik. A végső linkelést általában a megfelelő compiler driver végezze, rögzített és dokumentált eszközlánccal.

3 Linkelés, ELF, programindulás és leállás

3.1 Szimbólumok, linkage és relocation

A linker nem C-forráskódot olvas: objektumfájlokban tárolt szimbólumokat és hivatkozásokat kapcsol össze. Emiatt a sikeres fordítás még nem jelenti azt, hogy a teljes program összeállítható. Egy függvény deklarációja elegendő lehet a hívás lefordításához, miközben a megvalósítása hiányzik; egy hibás header pedig több objektumfájlba is eljuttathatja ugyanannak az objektumnak a definícióját.

A Headerek, deklarációk és include-higiénia részben megismert deklaráció–definíció különbséget most bináris szinten vizsgáljuk. A parancsok a Warningok, optimalizálási profilok és link-driver rész gyakorlatát követik: a végső linkelést is a fordító driverével végezzük.

3.1.1 Nyelvi kapcsolás és definíció

ISO C17. A nyelvi kapcsolás (*linkage*) azt határozza meg, hogy különböző deklarációk azonosítója ugyanazt az objektumot vagy függvényt jelöli-e.

- **Külső kapcsolás, external linkage:** különböző fordítási egységek deklarációi is jelölhetik ugyanazt az entitást.
- **Belső kapcsolás, internal linkage:** az azonos entitásra vonatkozó kapcsolat egy fordítási egységen belül érvényesül.
- **Nincs kapcsolás, no linkage:** nincs ilyen, kapcsolás alapján létrejövő azonosság. Ide tartozik például egy közönséges lokális változó vagy függvényparaméter.

A kapcsolás nem azonos a hatókörrel vagy az élettartammal. Egy blokkon belüli `static int count;` azonosítónak nincs kapcsolása, noha objektuma nem szűnik meg a blokk elhagyásakor. Ezeket a különbségeket a Scope, linkage, storage duration és lifetime rész tárgyalja részletesen.

Az alábbi deklarációk egymástól független példák, **fájlhatókörben**, korábbi deklaráció nélkül:

Deklaráció	Szerep	Kapcsolás
`extern int limit;`	Nem definiáló deklaráció	Külső
`int limit = 3;`	Objektumdefiníció	Külső
`extern int limit = 3;`	Objektumdefiníció az inicializáló miatt	Külső
`int limit;`	Tentative definition	Külső
`static int limit;`	Tentative definition	Belső
`int budget(void);`	Függvénydeklaráció	Külső
`static int budget(void);`	Függvénydeklaráció	Belső

Az `extern` tehát nem egyszerűen azt jelenti, hogy „máshol van definiálva”. Inicializálóval definíciót is adhat. Korábbi, látható deklaráció esetén pedig annak kapcsolását is örökölheti: egy fájlhatókörű `static int limit;` után álló `extern int limit;` ugyanazt a belső kapcsolású objektumot deklarálja.

3.1.1.1 Mit jelent a tentative definition?

ISO C17. A *tentative definition* olyan fájlhatókörű objektumdeklaráció, amelynek nincs inicializálója, és tárolásiosztály-meghatározója nincs, vagy `static`.

Az alábbi teljes fordítási egység C17-ben érvényes:

```
int requests;
int requests;
```

Ha a fordítási egység végéig nem jelenik meg hozzá inicializálóval ellátott definíció, a nyelvi hatás olyan, mintha egy nullaértékkel inicializált definíció szerepelne benne. Ha később `int requests = 5;` is következik, az adja meg az objektum kezdeti értékét.

Ez **fordítási egységen belüli** szabály. Nem engedély arra, hogy minden `.c` fájl külön definíciót kapjon ugyanahhoz a külső kapcsolású objektumhoz. A példáinkban használt külső objektumokhoz és függvényekhez a teljes programban egyetlen definíció szükséges.

3.1.2 Több fordítási egység vizsgálata

A következő példa minimumszabványa **ISO C17**. A megfigyelésekhez választott laborprofil: Linux, x86-64 System V ABI, LP64, `CHAR_BIT == 8`; a `lab-next` profil (profiltábla), GNU binutils 2.43 és a `lab-next` profil (profiltábla). Ez reprodukciós célprofil, nem dokumentált futtatási eredmény. A szimbólumokra vonatkozó táblázat alább **várt értelmezés**, nem bemásolt eszközkimenet.

Az `retry.h` tartalma:

```
#ifndef RETRY_H
#define RETRY_H

extern int retry_limit;
extern int calls;

int retry_budget(void);

#endif
```

A `retry.c` tartalma:

```
#include "retry.h"

int retry_limit = 3;
int calls;

static int reserve = 1;

int retry_budget(void)
{
    calls = 1;
    return retry_limit - reserve;
}
```

A `main.c` tartalma:

```c
#include "retry.h"

int main(void)
{
    int budget = retry_budget();
    return budget == 2 && calls == 1 ? 0 : 1;
}
```

A program nem módosítja a `retry_limit` és `reserve` értékét; így a kivonás eredménye biztosan ábrázolható `int` típusban. A `calls` név itt csak a függvény meghívását jelzi, nem általános számláló.

A tanulói laborban használható fordítási és vizsgálati parancsok:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 \
    -fno-lto -fno-common -fno-pie -c retry.c main.c

nm -A retry.o main.o
readelf -Ws retry.o
readelf -Ws main.o
readelf -rW retry.o
readelf -rW main.o

gcc -no-pie retry.o main.o -o retry-demo
```

Clang használatakor a fordítási és linkelési sorban a `gcc` helyére `clang` kerül. Az explicit opciók kizárják a vizsgálatból az LTO-t, a common-kezelést és a PIE alapértelmezésének eltéréseit. Éles programok hardeningprofilját ebből nem szabad levezetni.

GNU binutils / ELF. A lényeges szimbólumok várható értelmezése:

Fájl, név	`nm` jelölés	ELF-kötés, szekció	Jelentés
`retry.o: retry_limit`	`D`	`GLOBAL, .data`	Inicializált objektumdefiníció
`retry.o: calls`	`B`	`GLOBAL, .bss`	Nullaértékkel inicializált definíció
`retry.o: reserve`	`d`	`LOCAL, .data`	Belső kapcsolású objektum
`retry.o: retry_budget`	`T`	`GLOBAL, .text`	Függvénydefiníció
`main.o: retry_budget`	`U`	`GLOBAL, UND`	Feloldásra váró hivatkozás
`main.o: calls`	`U`	`GLOBAL, UND`	Feloldásra váró hivatkozás

Az `nm` betűi eszközkonvenciók: a nagybetű gyakran globális, a kisbetű lokális szimbólumot jelöl, de vannak kivételek. A `readelf -Ws` külön oszlopokban mutatja a típust, kötést (*binding*), láthatóságot (*visibility*) és szekcióindexet.

Egy puszta `extern` deklaráció nem feltétlenül eredményez szimbólumtábla-bejegyzést. A `main.c` például nem használja a `retry_limit` objektumot, ezért annak deklarációja önmagában nem igényel feloldandó gépi hivatkozást.

3.1.2.1 A linkage nem ELF-láthatóság

ELF / fordítóbővítés. A C kapcsolása és az ELF láthatósága külön szerződés.

Tulajdonság	Milyen kérdésre válaszol?
C linkage	Mely deklarációk jelölik ugyanazt az entitást?
ELF binding, például `LOCAL`, `GLOBAL`, `WEAK`	Hogyan vesz részt a szimbólum a linker feloldásában?
ELF visibility, például `DEFAULT`, `HIDDEN`	Milyen elérhetőségi és helyettesíthetőségi szabályok vonatkoznak rá az ELF-modellben?

Egy külső kapcsolású C függvény ELF-szimbóluma lehet `GLOBAL HIDDEN`. Ettől nem válik C szerinti belső kapcsolású függvénnyé: más objektumfájlból továbbra is hivatkozható ugyanazon végső linkelt komponensen belül. A `-fvisibility=hidden` GCC/Clang opció ezért nem helyettesíti a fájlhatókörű `static` használatát.

Egy lokális automatikus változó pedig rendszerint nem linker által feloldandó szimbólum. A debuggerben látható neve nem bizonyítja az ellenkezőjét: a hibakeresési információ külön adatszerkezet.

3.1.3 Relocation: a hiányzó cím kiszámítása

A `main.c` fordításakor a fordító ismeri a `retry_budget` típusát, de végleges címét még nem. A gépi kód megfelelő mezőjéhez ezért **relokációs rekordot** (*relocation record*) állít elő.

ELF / x86-64 ABI. A rekord megadja:

- melyik szekció melyik helyét kell javítani;
- mely szimbólumhoz kapcsolódik a javítás;
- milyen számítást kell alkalmazni;
- milyen hozzáadandó érték, *addend* tartozik hozzá.

Az x86-64 ELF-ben gyakori RELA-formátum külön mezőben tárolja az addendet. A `readelf -rW` ezeket a rekordokat mutatja.

A linker először összegyűjti a bemeneti szekciókat, kiosztja azok helyét, majd feloldja a szimbólumokat és elvégzi az előírt javításokat. Az undefined szimbólum tehát egy `.o` fájlban rendszerint normális köztes állapot.

3.1.3.1 Egy PC-relatív javítás levezetése

Az x86-64 `R_X86_64_PC32` relokáció számítása:

$$D = S + A - P,$$

ahol S a szimbólum címe, A az addend, P pedig a javítandó mező címe. Az eredménynek előjeles 32 bites értékként ábrázolhatónak kell lennie.

A következő címek **illusztrációk** a megadott x86-64 laborprofilhoz, nem a példaprogram mért címei:

Mennyiség	Illusztratív érték
S	`0x402000`
P	`0x401004`
A	`-4`

Ekkor:

$$D = \texttt{0x402000} - 4 - \texttt{0x401004} = \texttt{0xFF8}.$$

Bizonyításvázlat. Olyan utasításnál, amelynek négy oktettes eltolásmezője után közvetlenül a következő utasítás kezdődik, annak címe $P + 4$. A processzor innen számol:

$$(P + 4) + D = P + 4 + S - 4 - P = S.$$

A `-4` tehát összehangolja a relokáció mezőkezdethez kötött számítását a processzor következő utasításhoz viszonyított címzésével.

Nem minden hivatkozás `PC32`: függvényhívásnál például `R_X86_64_PLT32` is megjelenhet. A konkrét típust vizsgálni kell, nem kikövetkeztetni pusztán a C-kifejezésből. A dinamikus feloldást az ELF-betöltés, PLT/GOT és dinamikus szimbólumok rész folytatja.

3.1.4 Erős, gyenge és common szimbólumok

ELF / GCC–Clang-bővítés. Az *erős* és *gyenge* szimbólum nem ISO C nyelvi kategória. Szokásos terminológiával egy közönséges globális definíció erős, míg a `STB_WEAK` kötésű szimbólum gyenge.

Szokásos ELF-linkelésben egy erős definíció elsőbbséget élvez egy azonos nevű gyenge definícióval szemben. Két erős definíció általában multiple-definition hibát okoz. Több gyenge definíció közötti választásra nem szabad hordozható alkalmazáslogikát építeni.

GCC/Clang-bővítéssel például az alábbi **különálló fordítási egység** adhat gyenge alapértelmezést:

```
/* ISO C17 szintaxisra épülő GCC/Clang ELF-bővítés. */
__attribute__((weak))
int site_limit(void)
{
    return 3;
}
```

```
gcc -std=c17 -Wall -Wextra -c fallback.c -o fallback.o
nm fallback.o
readelf -Ws fallback.o
```

A várt besorolás függvénynél `W`, illetve `WEAK`. Ez platformhoz kötött felülírási mechanizmus, nem általános C-modularizálási eszköz.

3.1.4.1 Miért fontos a `-fno-common`?

Régebbi toolchainprofilokban a fájlhatókörű `int total;` gyakran **common szimbólumként** jelent meg. A common-bejegyzés még nem közönséges, szekcióhoz rendelt definíció: helyigényt és igazítási követelményt közöl. Az `nm` jellemző jelölése `C`, a `readelf` szekcióindexe `COM`.

A linker több ilyen bejegyzést hagyományosan összevonhatott. Ez elfedte a headerbe írt tentative definition hibáját.

Modern GCC- és Clang-profilokban gyakori alapértelmezés a `-fno-common`; a reprodukálható parancs mégis írja ki explicit módon. Ekkor a tentative definition tipikusan rendes `.bss`-definícióvá válik, és a több fordítási egységben elhelyezett példányok ütköznek.

A `-fcommon` visszakapcsolása nem helyettesíti a javítást:

- headerbe `extern int total;`;
- pontosan egy `.c` fájlba `int total;`.

3.1.5 Tipikus diagnosztikai tévedések

Undefined reference esetén újabb deklaráció hozzáadása. A deklaráció már elegendő volt a fordításhoz. A hiányzó definíciót tartalmazó objektumfájlt kell bevonni, vagy a tévesen belső kapcsolású megvalósítást kell javítani.

Multiple definition esetén include guard hozzáadása. A guard egy fordítási egységen belül működik; külön `.c` fájlok között nem akadályozza meg a definíció megismétlődését.

Sikeres linkelésből típushelyességre következtetés. A szokásos ELF-linker nem teljes C-típusellenőrző. Össze nem illő deklarációk is azonos szimbólumnévre mutathatnak. A közös header és annak megvalósításoldali beillesztése ezért helyességi követelmény.

3.1.6 Feladatok

1. **Magyarázat.** Miért `d` a `reserve`, de `D` a `retry_limit`? Külső kapcsolású marad-e egy `HIDDEN` láthatóságú függvény?
2. **Kódolás.** Egészítsd ki a példát `extern int last_budget;` deklarációval. Egyetlen definíció legyen, a `retry_budget` tárolja bele az eredményt, a `main` pedig ellenőrizze azt. Milyen szimbólumokat várs?
3. **Diagnosztika.** Két külön fájl, `a.c` és `b.c`, egyaránt kizárólag az alábbi teljes tartalmat kapja:

   ```
   int total;
   ```

   Vizsgáld meg őket `-fno-common`, majd `-fcommon` mellett. Magyarázd meg a részleges linkelés eltérését. Mi történne az eredeti példában, ha csak a `main.o` kerülne a végső linkparancsba?

3.1.7 Megoldások

3.1.7.1 Kapcsolás és láthatóság

A `reserve` fájlhatókörű `static` objektum: belső kapcsolású, a laborprofilban lokális ELF-szimbólummal. A `retry_limit` külső kapcsolású, globális szimbólum. Mindkettő inicializált adat, innen a `d` és `D`.

A `HIDDEN` nem változtatja meg a C kapcsolását. Ellenőrzési feltétel: a `readelf -Ws` kimenetében a `Bind` és `Vis` oszlopot külön értelmezzük.

3.1.7.2 Egyetlen új definíció

A headerbe kerüljön `extern int last_budget;`, a `retry.c` fájlba pedig egyszer `int last_budget;`.

A függvény módosított részlete:

```
int retry_budget(void)
{
    calls = 1;
    last_budget = retry_limit - reserve;
    return last_budget;
}
```

A `main` visszatérési kifejezése legyen:

```
return budget == 2 && calls == 1 && last_budget == budget ? 0 : 1;
```

Az eredeti buildparancsok változatlanok. Ellenőrzési feltétel: a `retry.o` egy `B` definíciót, a `main.o` egy `U` hivatkozást tartalmazzon `last_budget` néven; a teljes program sikeres ellenőrzésének várt kilépési állapota nulla.

3.1.7.3 Common és hiányzó objektumfájl

```
gcc -std=c17 -fno-common -c a.c b.c
nm -A a.o b.o
gcc -r a.o b.o -o combined.o
```

Mindkét objektumban rendes definíció várható; a részleges linkelés várhatóan multiple-definition hibával leáll. A `-r` újabb relokálható objektumot készítene, ezért itt nem szükséges `main`.

```
gcc -std=c17 -fcommon -c a.c b.c
nm -A a.o b.o
gcc -r a.o b.o -o combined.o
```

Itt `C` bejegyzések és sikeres összevonás várható. Ez a toolchain viselkedése, nem az ISO C többdefiníciós engedménye.

Az eredeti példából a `retry.o` kihagyása feloldatlan `retry_budget` és `calls` hivatkozást hagy. Ellenőrzési feltétel: a hiányzó neveket össze kell vetni a `main.o` `U` bejegyzéseivel.

3.1.8 Összefoglaló

A C kapcsolása deklarációk azonosságát, az ELF szimbólumtáblája a bináris összekapcsolás adatait írja le. A relokáció a még ismeretlen címek javítási receptje. Az `nm` és a `readelf` segítségével elkülöníthető a hiányzó definíció, a többszörös definíció és a common-kezelés. A megbízható alapminta változatlan: közös deklaráció a headerben, egyetlen megvalósítás, explicit fordítási profil.

3.2 Statikus és megosztott könyvtárak

A könyvtár nem pusztán újrahasznosítható forráskód: fordítási, linkelési és telepítési szerződés is. Statikus könyvtár használatakor a szükséges gépi kód rendszerint bekerül a fogyasztó binárisába. Megosztott könyvtárnál a bináris függőséget rögzít, amelyet futáskor a betöltési környezetnek kell teljesítenie. DevOps-szempontból ez meghatározza, mit kell újraépíteni egy biztonsági frissítés után, és miért indulhat el egy program a buildkonténerben, miközben a futtatóképben már nem.

A Szimbólumok, linkage és relocation részben megismert feloldatlan szimbólumhivatkozásokra építünk. Most ugyanahhoz az interfészhez készítünk `libcount.a` és `libcount.so` könyvtárat, majd különválasztjuk a linkeléskori és futásidejű keresést.

3.2.1 Két könyvtárforma, két függőségi modell

GNU/ELF-eszközlánc: a statikus könyvtár általában `ar` formátumú archívum, amely relokálható objektumfájlokat és szimbólumindexet tartalmaz. A linker alapesetben nem az egész archívumot másolja be: azokat az objektumtagokat emeli ki, amelyek szükségesek az addig felismert feloldatlan hivatkozásokhoz. A kiemelés egysége objektumfájl, nem egyetlen függvény; a későbbi optimalizálás és szekcióeldobás ettől külön mechanizmus.

A **megosztott objektum** (*shared object*) önálló ELF-bináris. Kódja külön fájlban marad, és a fogyasztó rendszerint futásidejű függőséget rögzít rá.

Állomány	Szerep	Kell a fogyasztó futásához?
`count.h`	Fordításkori API-deklaráció	Nem
`libcount.a`	Objektumtagok linkeléskori forrása	Maga az archívum nem
`libcount.so`	A linker által keresett fejlesztői név	Nem feltétlenül ez a név
`libcount.so.1`	A példában rögzített futásidejű könyvtárnév	Igen, feloldható állományként

Fontos különbség: egyetlen könyvtár statikus felhasználása **nem jelenti a teljes program statikus linkelését**. A program más függőségei, például a libc, továbbra is lehetnek megosztott könyvtárak.

3.2.2 A libcount felépítése

A példák minimuma ISO C17. A választott, reprodukáláshoz rögzítendő mintaprofil Linux x86-64, System V ABI, LP64 adatmodell, a `lab-next` profil (profiltábla), GNU binutils 2.43 és a `lab-next` profil (profiltábla). Második fordítóként a `lab-next` profil (profiltábla) használható, ugyanazzal a célplatformmal és külön rögzített linkerrel. Ezek választott verziók, nem itt ellenőrzött futtatási környezetek; az alábbi eredmények elvárások, nem mért kimenetek.

A három forrásfájl ugyanabba a friss munkakönyvtárba kerüljön.

count.h:

```
#ifndef COUNT_H
#define COUNT_H

unsigned count_add(unsigned total, unsigned delta);

#endif
```

count.c:

```
#include "count.h"

unsigned count_add(unsigned total, unsigned delta)
{
    return total + delta;
}
```

main.c:

```
#include <stdio.h>
#include "count.h"

int main(void)
{
    if (printf("%u\n", count_add(2u, 3u)) < 0) {
        return 1;
    }
    return 0;
}
```

ISO C17: a függvény értékeket fogad és ad vissza; nincs erőforrás-tulajdonlás vagy kölcsönzött hozzáférés. Az `unsigned` összeadás az adott típus értéktartományának megfelelő moduláris összeadás. A függvény nem jelzi a matematikai összeg tartománytúllépését; ez az API szerződésének része, nem linkelési kérdés.

Először készítsünk objektumfájlokat és archívumot:

```
mkdir -p build/lib build/bin

gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -fno-lto \
    -fPIC -c count.c -o build/count.pic.o

gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -fno-lto \
    -fPIE -c main.c -o build/main.o
```

```
ar rcs build/lib/libcount.a build/count.pic.o

gcc -pie build/main.o build/lib/libcount.a \
    -o build/bin/count-static
```

Az `ar rcs` parancsban az `r` tagot illeszt be vagy cserél, a `c` jelzi az archívum létrehozásának szándékát, az `s` pedig szimbólumindexet készít.

A statikus archívumba itt szándékosan PIC objektum kerül, hogy ugyanazt az objektumot használhassuk a megosztott változathoz is. **A PIC kód nem teszi dinamikussá a linkelést.**

A megosztott változat:

```
gcc -shared build/count.pic.o \
    -Wl,-soname,libcount.so.1 \
    -o build/lib/libcount.so.1.0

ln -sfn libcount.so.1.0 build/lib/libcount.so.1
ln -sfn libcount.so.1 build/lib/libcount.so

gcc -pie build/main.o -Lbuild/lib -lcount \
    -Wl,--enable-new-dtags,-rpath,'$ORIGIN/../lib' \
    -o build/bin/count-shared
```

A `-Lbuild/lib` linkeléskori keresési könyvtárat ad meg. A `-lcount` a megadott GNU/ELF-profilban normál, dinamikus linkelési módban a megosztott változatot részesíti előnyben, ha az adott keresési helyen az archívum is jelen van. A statikus példában ezért írtuk ki közvetlenül a `.a` fájl nevét.

Sikeres betöltés és kimeneti írás esetén mindkét program várt kimenete `5`, majd újsor.

3.2.3 Statikus linksorrend: mikor válik szükségessé egy archívumtag?

A hagyományos GNU linkerfeldolgozásban az archívum helye jelentéssel bír. Az alábbi sorrend szándékosan hibás:

```
# Szándékosan hibás linksorrend.
gcc -pie build/lib/libcount.a build/main.o \
    -o build/bin/count-wrong
```

Indoklás, lépésenként: tekintsük csak a `count_add` hivatkozását.

1. Az archívum feldolgozásakor még nincs rá feloldatlan hivatkozás.
2. Emiatt a linker nem emeli ki a `count.pic.o` tagot.
3. A később feldolgozott `main.o` létrehozza a hivatkozást.
4. A linker normál esetben nem tér vissza automatikusan a korábbi archívumhoz.

A várt következmény feloldatlan `count_add` szimbólum miatti linkelési hiba. A javítás: előbb a fogyasztó objektum, utána a függőség.

Ugyanez vonatkozik archívumok közötti függőségre. Ha `libfront.a` igényli `libback.a` függvényeit, tipikusan `-lfront -lback` a megfelelő sorrend. Ciklikus archívumfüggőségnél a GNU linker `--start-group` és `--end-group` kapcsolói ismételt keresést kérhetnek, de a ciklus gyakran modulhatár-problémát jelez.

Megosztott könyvtáraknál se feltételezzünk korlátlan sorrendfüggetlenséget: például a `--as-needed` beállítás befolyásolja, mely könyvtárakhoz marad meg futásidejű függőségi bejegyzés.

3.2.4 PIC, shared object és PIE

GCC/ELF-eszközlánc: a három kapcsolócsoport különböző feladatot végez.

Kapcsoló	Fázis	Rendeltetés
`-fPIC`	Fordítás	Megosztott objektumhoz alkalmas helyfüggetlen kód
`-shared`	Linkelés	Megosztott objektum előállítása
`-fPIE`, majd `-pie`	Fordítás, majd linkelés	Helyfüggetlen végrehajtható program előállítása

A *position-independent code* nem azt jelenti, hogy semmilyen relokáció nem szükséges. Azt jelenti, hogy a generált kód a célplatform megfelelő címzési mechanizmusaival támogatja az eltérő betöltési címet.

A PIE (*position-independent executable*) továbbra is program, nem általánosan használható könyvtár. A példában a `count-static` is PIE: a saját `count` kódját statikusan tartalmazza, miközben helyfüggetlen végrehajtható állomány.

ELF-ben a shared object és a PIE is lehet `ET_DYN` típusú. Ez az egy fejlécmező tehát önmagában nem elegendő a szerep eldöntéséhez. A betöltési mechanizmusokat az ELF-betöltés, PLT/GOT és dinamikus szimbólumok rész bontja ki.

3.2.4.1 SONAME és DT_NEEDED

A `SONAME` a megosztott könyvtárba írt logikai függőségi név. A példában:

- a tényleges állomány `libcount.so.1.0`;
- a könyvtár `DT_SONAME` értéke `libcount.so.1`;
- a fogyasztó `DT_NEEDED` bejegyzése ezért `libcount.so.1`.

A `libcount.so` fejlesztői szimbolikus link segíti a linkelést, de a futásidejű keresés nem erre a névre támaszkodik. A `SONAME` nem automatikus kompatibilitásvizsgálat: az azonos nevet viselő, hibásan inkompatibilis könyvtár is megtalálható és betölthető.

3.2.5 Futásidejű keresés és ellenőrzés

Linux/glibc: ha a függőség neve nem tartalmaz perjelet, normál, nem biztonsági végrehajtási módban a keresés fő sorrendje leegyszerűsítve:

1. Az alkalmazható `DT_RPATH` bejegyzések, a megfelelő `DT_RUNPATH` hiányában.
2. Az `LD_LIBRARY_PATH`.
3. A függőséget igénylő objektum `DT_RUNPATH` bejegyzése.
4. A loader gyorsítótára, rendszerint `/etc/ld.so.cache`.
5. Az alapértelmezett, architektúrafüggő könyvtárak.

A részletes algoritmust befolyásolja a függőségi lánc, a glibc verziója, a hardverképesség-alapú könyvtárválasztás és egyes linkkapcsolók. Biztonsági végrehajtási módban, például bizonyos privilegizált programoknál, környezeti változók figyelmen kívül maradhatnak. Más libc betöltőjére ezt a sorrendet nem szabad automatikusan átvinni.

A régi `RPATH` és az újabb `RUNPATH` fontos eltérése, hogy a `RUNPATH` csak az adott objektum közvetlen függőségeinek keresésére vonatkozik. Ha a `libcount` egy további privát könyvtárat igényel, annak megtalálását nem garantálja a főprogram `RUNPATH` értéke. Az `RPATH` a függőségi láncon tovább is érvényesülhet, és az `LD_LIBRARY_PATH`-hoz viszonyított elsőbbsége is eltér.

A példában a `--enable-new-dtags` kifejezetten az új bejegyzést kéri. A `$ORIGIN` a bejegyzést tartalmazó bináris könyvtárát jelöli, ezért a `build/bin` alatti program a szomszédos `build/lib` könyvtárban kereshet. Az egyszeres idézőjel megakadályozza, hogy a shell idő előtt helyettesítse a dollárjeles kifejezést.

Ez áthelyezhető csomagelrendezést támogat, de csak megfelelő jogosultságokkal biztonságos: nem megbízható felhasználó által írható keresési könyvtár kódbetöltési kockázat.

3.2.5.1 readelf az ldd előtt

A külön laborban, a programok elindítása nélkül vizsgálhatók a metaadatok:

```
ar t build/lib/libcount.a
readelf -hW build/bin/count-shared
readelf -dW build/bin/count-shared
readelf -dW build/lib/libcount.so.1.0
readelf -dW build/bin/count-static
```

Az ellenőrzési feltételek:

- az archívum tartalmazza a `count.pic.o` tagot;
- a megosztott könyvtár `SONAME` értéke `libcount.so.1`;
- a dinamikus fogyasztóban van ilyen `NEEDED` bejegyzés és megfelelő `RUNPATH`;
- a statikus fogyasztóban nincs `libcount`-függőség, de a libc dinamikus függősége megmarad.

Az `ldd` az aktuális környezetben feloldott függőségekhez hasznos, de **ismeretlen, nem megbízható binárison ne használd**: megvalósítástól és a bináris interpreterétől függően kódvégrehajtást is előidézhet. A `readelf` adatként értelmezi az állományt; ez előnyösebb, bár sérült bemenetekhez naprakész eszköz és elkülönített vizsgálókörnyezet indokolt.

A `readelf -d` sem teljes futásidejű bizonyíték: közvetlen metaadatokat mutat, nem a teljes feloldott függőségi fát, és nem látja előre az alkalmazás által később dinamikusan betöltött modulokat.

3.2.6 API, ABI és frissítési szerződés

Az **API** a forrásból látható interfész és viselkedési szerződés. Az **ABI** a már lefordított részek együttműködésének bináris szerződése: többek között szimbólumnevek, paraméterátadás, visszatérés és adatreprezentáció.

Változtatás	Kompatibilitási következmény
Belső algoritmus cseréje változatlan szerződéssel	Lehet API- és ABI-kompatibilis

Változtatás	Kompatibilitási következmény
Új függvény hozzáadása	A régi fogyasztóknak általában kompatibilis
Meglévő szimbólum eltávolítása	Régi fogyasztók működését megszakíthatja
`unsigned` helyett `unsigned long` paraméter	A mintaprofilban eltérő típusszélesség; ABI-vizsgálat és rendszerint új ABI-verzió szükséges

Inkompatibilis ABI-váltáskor szokás új `SONAME`, például `libcount.so.2` használata. Egy `.so.1` nevű szimbolikus link átirányítása egy inkompatibilis `.so.2` állományra nem javítás.

Statikus felhasználásnál a könyvtár frissítése után a fogyasztót újra kell linkelni. Kompatibilis megosztott könyvtárfrissítésnél ez elkerülhető lehet, de a már futó folyamat nem cseréli le automatikusan a betöltött kódját: a telepítési tervnek újraindítással is számolnia kell.

3.2.7 Gyakori hibák

A `-L` nem futásidejű keresési útvonal; a sikeres linkelés ezért nem bizonyítja a telepítés helyességét. A `.a` használata nem helyettesíti a teljes statikus linkelés külön döntését. A `.so` fájlnév pedig önmagában nem jelent megfelelő `SONAME`-ot vagy kompatibilis ABI-t.

A javítás közös mintája: a buildparancs mellett mindig vizsgáld a tényleges ELF-metaadatokat, majd külön ellenőrizd a futtatási környezetet.

3.2.8 Feladatok

1. **Magyarázó:** hogyan lehet a `count-static` egyszerre PIE és részben dinamikusan linkelt program? Melyik állomány nem kell a telepítéséhez?
2. **Kódolási:** adj hozzá `unsigned count_zero(void)` függvényt, amely nullát ad vissza. Készíts új megosztott könyvtárat változatlan `SONAME`-mal. Indokold a kompatibilitás irányát.
3. **Diagnosztikai:** a linkelés sikeres, de a telepített program nem találja a `libcount.so.1` fájlt. A csomag csak `libcount.so` és `libcount.so.1.0` állományokat tartalmaz. Mit ellenőrzöl és javítasz?

3.2.9 Megoldások

3.2.9.1 Magyarázó feladat

A PIE a program címzési és betöltési tulajdonsága. A statikus `libcount`-felhasználás azt jelenti, hogy annak szükséges objektumtagja bekerült a programba. Ettől a libc még dinamikus függőség maradhat. A `libcount.a` nem kell futáskor.

Ellenőrzés: a `count-static` dinamikus szakaszában nincs `libcount`-hoz tartozó `NEEDED`.

3.2.9.2 Kódolási feladat

A header védelmén belüli deklaráció:

```
unsigned count_zero(void);
```

A `count.c` kiegészítése, az eredeti C17-profilban:

```
unsigned count_zero(void)
{
    return 0u;
}
```

Az előző fordítási és megosztott linkelési parancsok ismétlésével elkészíthető az új változat. Ellenőrizd a dinamikus szimbólumtáblát:

```
readelf --dyn-syms -W build/lib/libcount.so.1.0
```

A régi fogyasztó továbbra is megtalálja a változatlan `count_add` függvényt. Az új `count_zero` függvényt használó fogyasztó viszont nem működik a régi könyvtárral. A kompatibilitás tehát nem szimmetrikus; a csomagfüggőségnek szükség esetén minimumverziót is ki kell fejeznie.

3.2.9.3 Diagnosztikai feladat

Először a fogyasztó `NEEDED` és `RUNPATH` bejegyzéseit vizsgáljuk. Ezután ellenőrizzük, hogy a tényleges keresési könyvtárban létezik-e a kért `libcount.so.1` név.

A mintacsomagból hiányzik a futásidejű szimbolikus link:

```
ln -sfn libcount.so.1.0 build/lib/libcount.so.1
```

Ez csak akkor megfelelő javítás, ha a célállomány valóban az elvárt ABI-t valósítja meg. Az ellenőrzési feltétel a teljes, elérhető névlánc és a helyes keresési útvonal, nem pusztán a `.so` kiterjesztésű fájl jelenléte.

3.2.10 Összefoglaló

A statikus archívum linkeléskori objektumforrás; a megosztott könyvtár futásidejű függőség. A linksorrend, a `SONAME`, a `DT_NEEDED` és a loader keresési útvonalai külön szerződéselemek. Megbízható kiadásnál ezeket együtt kell ellenőrizni az API/ABI-kompatibilitással és a telepített állományokkal.

3.3 ELF-betöltés, PLT/GOT és dinamikus szimbólumok

Egy megosztott könyvtár függvényének meghívásakor a gépi kód nem feltétlenül tartalmazza a függvény végleges címét. A könyvtár betöltési címe futásonként változhat, és a névhez tartozó definíció kiválasztásában a dinamikus linker is részt vehet. A hívás működéséhez ezért össze kell kapcsolnunk az ELF-fájl szerkezetét, a virtuális címtér kialakítását és a futásidejű szimbólumfeloldást.

A Statikus és megosztott könyvtárak részben látott linkelési modellre építünk. Az alábbi ELF-, PLT/GOT- és védelmi mechanizmusok nem az ISO C részei: konkrét modellünk a Linux, az x86-64 System V ABI és a glibc dinamikus linkere.

3.3.1 Section és segment: két nézet ugyanarról a fájlról

Az ELF **sectionje**, vagyis szekciója a fájl linkeléshez és elemzéshez használható logikai egysége. A section header table leírja többek között a szekciók típusát, méretét, fájlon belüli helyét és tulajdonságait. Ilyen egység a gépi kódot tartalmazó `.text` vagy a szimbólumtábla.

A **program header** egy programfejléc-bejegyzés: a betöltéshez vagy futáshoz szükséges tartományt, illetve metaadatot ír le. A bejegyzések táblája a program header table. Különösen fontos típusai:

- `PT_LOAD`: a virtuális címtérbe betöltendő szegmens;
- `PT_INTERP`: a programhoz használandó interpreter, Linuxon rendszerint a dinamikus linker elérési útja;
- `PT_DYNAMIC`: a dinamikus linkelés metaadatainak helye.

A szekció és a szegmens nem egy-egy megfeleltetésben áll. Egy betölthető szegmens több, összeillő hozzáférési igényű szekciót is tartalmazhat. A betöltő számára a programfejlécek az irányadók; egy megfelelően elkészített futtatható ELF-nek a végrehajtáshoz nincs szüksége section header table-re.

3.3.1.1 Tipikus szekciók a virtuális címtérben

Az alábbi táblázat az x86-64 Linux/GNU eszközlánc szokásos elrendezését szemlélteti, nem univerzális layoutot ír elő. A méretek ELF-beli bájtokban értendők; ebben a laborprofilban egy C bájt nyolc bit.

Szekció	Tipikus tartalom	Tartalmi bájtok a fájlban	Tipikus futásidejű hozzáférés
`.text`	Függvények gépi kódja	Igen	Olvasás, végrehajtás
`.rodata`	Sztringliterálok, csak olvasható adatok	Igen	Olvasás
`.data`	Módosítható, fájlból inicializált adatok	Igen	Olvasás, írás
`.bss`	Nullázással inicializált tároló	Nincs megfelelő tartalmi terület	Olvasás, írás

A `.bss` szokásosan `SHT_NOBITS` típusú: mérete van, de tartalma nem foglal ugyanannyi helyet a fájlban. Egy `PT_LOAD` szegmensnél `p_filesz` jelöli a fájlból származó tartomány méretét, `p_memsz` pedig a memóriabeli méretet. Feltétel:

$$0 \le p_{\text{filesz}} \le p_{\text{memsz}}.$$

A memóriabeli tartomány fájlból már nem fedezett végét nullákkal kell inicializálni. Ez magyarázza, hogyan lehet egy nagy nullázott adattartomány futásidejű memóriaigénye jelentős, miközben a bináris alig növekszik.

A hozzáférési jogokat nem a `.text` név varázsolja elő: a betölthető szegmens jelzői és az operációs rendszer leképezései határozzák meg őket. A lapméret és az igazítás miatt a szekcióhatár nem feltétlenül lapvédelmi határ.

3.3.2 A betöltéstől a függvényhívásig

Linuxon az `exec` során a kernel értelmezi a végrehajtható ELF programfejléceit. Dinamikusan linkelt programnál betölti az interpretert is, majd annak adja át a vezérlést.

A dinamikus linker feldolgozza a függőségeket, betölti a szükséges megosztott objektumokat, végrehajtja az előírt relokációkat és megszervezi a szükséges inicializálásokat. Ezután a program belépési pontjára kerül a vezérlés. Ez nem közvetlenül a C `main`; a folytatást a main, startup, argumentumok és környezet rész tárgyalja.

Pozíciófüggetlen ELF-objektumnál sok cím számítása szemléletesen:

$$A_{\text{futás}} = B + V,$$

ahol B a betöltési eltolás (*load bias*), V az objektumhoz tartozó ELF virtuális cím, $A_{\text{futás}}$ pedig a tényleges futásidejű cím. Ez nem minden relokáció képlete: külső szimbólum esetén előbb a megfelelő definíciót is meg kell találni.

3.3.2.1 PLT és GOT

A **Procedure Linkage Table**, röviden PLT, függvényhívásokat közvetítő gépi kódú bejegyzéseket tartalmazhat. A **Global Offset Table**, vagyis GOT, futásidőben használt címeket tároló táblázat. A GOT nem kizárólag függvénycímekre szolgál.

Egy hagyományos, x86-64 ELF-es, késleltetett feloldású hívás útja:

1. A hívó kód a függvény PLT-bejegyzését hívja.
2. A PLT a hozzá tartozó GOT-bejegyzésen keresztül ugrik.
3. Az első híváskor ez még a dinamikus linker feloldó útvonalához vezet.
4. A linker megkeresi a megfelelő szimbólumdefiníciót, és rendszerint frissíti a GOT-bejegyzést.
5. A következő hívások már a feloldott címre jutnak.

Ez a **lazy binding**, a késleltetett feloldás. Csökkentheti az induláskori munkát, de az első hívás többletköltséget és későn jelentkező szimbólumhibát hozhat.

Az **eager binding**, vagy azonnali feloldás esetén a függvényhívási relokációk is betöltéskor rendeződnek. A GNU linker `-z now` kapcsolója ilyen követelményt rögzíthet az objektumban.

A PLT/GOT nem kötelező, változatlan utasítássorozat. A fordító `-fno-plt` beállítása, a linker optimalizálásai és a célarchitektúra módosíthatják a hívás alakját. A lazy binding elsősorban az erre alkalmas függvényhívási relokációkra vonatkozik: nem azt jelenti, hogy minden címfeloldás elhalasztható.

3.3.3 ASLR, PIE és RELRO

Az **ASLR** az operációs rendszer címelrendezés-véletlenítési mechanizmusa. A **PIE** pozíciófüggetlen végrehajtható program: lehetővé teszi, hogy a főprogram is változó címen helyezkedjen el. GNU eszközláncnál ehhez tipikusan `-fPIE` fordítás és `-pie` linkelés tartozik.

A PIE és az ASLR nem ugyanaz. Az előbbi a bináris tulajdonsága, az utóbbi futtatási környezeti mechanizmus. Az ELF `ET_DYN` típusa önmagában sem bizonyítja, hogy az állomány PIE: megosztott könyvtár is lehet ilyen.

A **RELRO**, vagyis *relocation read-only*, olyan területet jelöl ki, amelyet a loader a szükséges relokációk után csak olvashatóvá tehet.

GNU linkelési beállítás	Szokásos következmény
`-z relro`	A `PT_GNU_RELRO` tartomány lezárható; a lazy bindinghoz szükséges helyek írhatók maradhatnak
`-z now`	Az érintett objektum függvényhívási relokációinak azonnali feloldása
`-z relro -z now`	Szokásos GNU elrendezésben „full RELRO”, a hívási GOT-terület védelmével is

A `-z now` önmagában nem tesz memóriát csak olvashatóvá. A főprogram beállításaiból pedig nem szabad a folyamat összes megosztott könyvtárának védelmét levezetni.

3.3.4 Dinamikus szimbólumok, verziózás és interposition

A teljes, gyakran eltávolítható `.symtab` és a dinamikus linkeléshez használt `.dynsym` különböző szerepű. A dinamikus linker nem minden forrásbeli függvénynevet lát: az exportálás, a szimbólum láthatósága és a linkelési döntések korlátozzák a halmazt.

A GNU ELF **szimbólumverziózás** egy névhez verzióazonosítót is társíthat. A vizsgálóeszközök például `probe_value@PROBE_1.0` jelöléssel mutathatnak verziózott definíciót; a `@@` az alapértelmezett verziót jelölheti. A fogyasztó verziókövetelménye így különböző ABI-generációk megkülönböztetésére használható. Ez nem C-típusellenőrzés, és nem helyettesíti a helyes függvénydeklarációt.

A **symbol interposition** során egy névhez a keresési szabályok alapján másik objektum definíciója is kiválasztható. Linux/glibc alatt az `LD_PRELOAD` tipikus eszköz erre, például instrumentációhoz.

Nem minden hívás téríthető el így. A rejtett láthatóság, a helyi kötés, egyes linkerbeállítások és a közvetlenre optimalizált hívások korlátozhatják az interpositiont. Biztonságos végrehajtási módban a loader a környezeti befolyásolás egy részét is korlátozza.

3.3.5 Kidolgozott labor: közvetlen hívás és explicit betöltés

A választott reprodukciós célprofil: Linux x86-64, System V ABI, LP64, a `lab-next` profil (profiltábla). A példák C17-et használnak; a dinamikus betöltési példa külön jelölt platformkiterjesztést is igényel. A parancsok és az elvárt megfigyelések nem dokumentált futtatási eredmények.

A `probe.h` fájl:

```
#ifndef PROBE_H
#define PROBE_H

int probe_value(void);

#endif
```

A `probe.c` könyvtári implementáció:

```c
#include "probe.h"

int probe_value(void)
{
    return 42;
}
```

A `direct.c` közvetlen fogyasztó:

```c
#include "probe.h"

int main(void)
{
    return probe_value() == 42 ? 0 : 1;
}
```

Fordítás és linkelés:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g \
    -fPIC -shared probe.c \
    -Wl,-soname,libprobe.so -Wl,-z,relro,-z,now \
    -o libprobe.so

gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g \
    -fPIE -fplt -pie direct.c -L. -lprobe \
    -Wl,-rpath,'$ORIGIN' -Wl,-z,relro,-z,now \
    -o direct
```

Az idézőjelezett `$ORIGIN` a loader számára marad meg: ebben a beállításban a futtatható állomány könyvtárához viszonyított keresést tesz lehetővé.

A vizsgálat lépései:

```
readelf -SW direct
readelf -lW direct
readelf -dW direct
readelf -rW direct
readelf --dyn-syms -W libprobe.so
readelf -V direct
objdump -d direct
```

A section–segment hozzárendelést, a `NEEDED` függőséget, a relokációkat és a `probe_value` hívási útját kell összekapcsolni. Konkrét címérték vagy PLT-utasítássorozat nincs előírva.

3.3.5.1 `dlopen` és `dlsym`

A POSIX `dlopen` futás közben betöltött objektumhoz ad handle-t; a `dlsym` a handle keresési tartományában keres szimbólumot. A `RTLD_NOW` azonnali feloldást kér, a `RTLD_LOCAL` pedig nem teszi általánosan globálissá az új objektum szimbólumait.

A `dynamic.c` teljes platformpélda:

```c
#include <dlfcn.h>
#include <stdio.h>

typedef int (*probe_fn)(void);

int main(void)
{
    void *handle = dlopen("./libprobe.so", RTLD_NOW | RTLD_LOCAL);
    if (handle == NULL) {
        fputs("dlopen failed\n", stderr);
        return 1;
    }

    (void)dlerror(); /* Korabbi hiba torlese. */
    void *address = dlsym(handle, "probe_value");
    const char *error = dlerror();

    if (error != NULL || address == NULL) {
        fputs(error != NULL ? error : "NULL symbol address", stderr);
        fputc('\n', stderr);
        (void)dlclose(handle);
        return 1;
    }

    /* Linux/POSIX szerzodes; nem hordozhato ISO C konverzio. */
    probe_fn function = (probe_fn)address;
    int value = function();

    if (dlclose(handle) != 0) {
        return 1;
    }
    return value == 42 ? 0 : 1;
}
```

```
gcc -std=c17 -Wall -Wextra -O0 -g \
    -fPIE -pie dynamic.c -ldl \
    -Wl,-z,relro,-z,now -o dynamic
```

A `dlsym` eredményének függvénypointerré alakítását az ISO C17 önmagában nem garantálja; itt a Linux/POSIX platform szerződésére támaszkodunk. A `-Wpedantic` ezért szándékosan hiányzik ebből a parancsból. A konverzió nem ellenőrzi a függvény típusát: az `int (void)` szerződésnek ténylegesen egyeznie kell.

Érték	Tulajdon és élettartam	Érvénytelenítés, hibakimenet
`handle`	A program felel a sikeres megnyitás lezárásáért	`NULL` jelzi a megnyitási hibát; lezárás után nem használjuk
`address`, `function`	Kölcsönzött hozzáférés a betöltött kódhoz	`dlclose` után nem támaszkodunk az érvényességükre
`error`	A runtime által kezelt, kölcsönzött szöveg	A következő `dlerror` felülírhatja; nem szabad `free`-vel felszabadítani

A `dlsym` null eredménye önmagában nem általános hibateszt; ezért használjuk a `dlerror` protokollját. A példa ettől függetlenül elutasítja a null címet, mert meghívható függvényt vár. A `dlclose` nem feltétlenül távolítja el azonnal a könyvtárat, de erre nem építhető élettartam-garancia.

3.3.6 Gyakori félreértések

A section nem önálló memória-leképezés: a betöltési nézetet a programfejlécekből kell kiolvasni. A `.bss` kis fájlköltsége nem jelent nulla futásidejű memóriaigényt.

A sikeres linkelés nem garantálja, hogy futáskor minden könyvtár és megfelelő szimbólumverzió rendelkezésre áll. Lazy binding esetén egy hiányzó függvénydefiníció hibája akár az első érintett hívásig is késhet.

Végül a sikeres `dlsym` csak névfeloldást igazol. Nem bizonyítja a függvénytípus, az ABI vagy az élettartam helyességét.

3.3.7 Feladatok

1. **Magyarázat:** Miért nem szükséges, hogy minden `.text` szekcióhoz külön `PT_LOAD` tartozzon? Miért lehet a `.bss` memóriabeli mérete nagyobb a fájlbeli tartalmánál?
2. **Kódolás:** Készíts a `direct` programból lazy bindingot kérő változatot. Melyik linkeropciót változtatod meg, és mit vizsgálsz meg?
3. **Diagnosztika:** Egy pluginból kapott függvénypointert eltárolnak, lezárják a handle-t, majd később meghívják. Miért nem bizonyít helyességet, ha ez néhány futásban működik?

3.3.8 Megoldások

1. A szekció linkelési, a szegmens betöltési egység; egy szegmens több szekciót foghat össze. A `.bss` tartalmát nullázás állítja elő, nem azonos méretű fájlbeli bájtsorozat. **Ellenőrzés:** a `readelf -lW` hozzárendelése és a megfelelő `p_memsz`, `p_filesz` értékek összevetése.
2. A fogyasztó linkelésében a `-z now` helyére `-z lazy` kerül, például `-Wl,-z,relro,-z,lazy` alakban. **Ellenőrzés:** a dinamikus jelzők és a hívási relokációk vizsgálata. Az azonnali feloldási jelző hiánya még nem bizonyít tényleges lazy működést: például a glibc `LD_BIND_NOW` környezeti beállítása felülírhatja a várakozást.
3. Más hivatkozás vagy loaderbeállítás életben tarthatta a könyvtárat. Ez esetleges körülmény, nem a tárolt pointer szerződése. A javítás a handle életben tartása minden lehetséges hívás végéig. **Ellenőrzési feltétel:** lezárás után sem közvetlen, sem eltárolt callbacken keresztüli hívás nem maradhat lehetséges.

3.3.9 Összefoglaló

Az ELF szekciói a linkelési, programfejlécei a betöltési nézetet adják. A dinamikus linker a leképezett objektumokhoz címeket és szimbólumdefiníciókat rendel; a PLT/GOT ennek egyik tipikus hívási mechanizmusa. A PIE, ASLR és RELRO különböző feladatokat lát el. Explicit betöltéskor pedig a névfeloldáson túl a függvénytípus és a betöltött kód élettartama is az alkalmazás szerződésének része.

3.4 main, startup, argumentumok és környezet

A natív program indulása két külön szerződés találkozása. Az ISO C azt határozza meg, milyen feltételekkel kapja meg a vezérlést a C program, és milyen adatokat használhat. A Linux, az ELF-formátum, a hívási ABI és a C futtatókörnyezete azt valósítja meg, hogyan jut el idáig a folyamat.

Az ELF-betöltésről szóló rész alapján már megkülönböztetjük a betöltőt és a dinamikus linkert. Most ezt kapcsoljuk össze a `main` függvénnyel. A deklarációk olvasásához elegendő a C szintaxis tömör áttekintése; a pointerek részletes modelljét később bontjuk ki.

3.4.1 Az ISO C hosted belépési szerződése

ISO C17 — hosted környezet. Hosted, vagyis teljes C futtatási környezetben a program indulásakor meghívott függvény neve `main`, visszatérési típusa `int`. Két szabványos definíciós alakja:

- `int main(void)`: a program nem kér hozzáférést a parancssori argumentumokhoz;
- `int main(int argc, char *argv[])`: a program megkapja az argumentumszámot és az argumentumokat.

A második alakban a paraméterként szereplő `char *argv[]` típusa `char **argv` típusra igazodik. Ezért az `int main(int argc, char **argv)` is megfelelő alak. A paraméternevek szabadon választhatók.

Az implementáció további, dokumentált alakokat is támogathat. Ettől azonban például a `void main(void)` nem válik hordozható hosted C belépési ponttá. A `main` visszatérési értéke a program befejezési állapotának része, nem elhagyható díszítés.

3.4.1.1 Az argumentumok pontos feltételei

Az ISO C17 a következőket garantálja:

- `argc` nemnegatív.
- `argv[argc]` null pointer.
- Ha `argc > 0`, akkor `argv[0]` a program nevét reprezentáló sztringre mutat. Ha a név nem érhető el, ez üres sztring lehet.
- Ha `argc > 1`, akkor az `argv[1]`–`argv[argc - 1]` elemek a program argumentumait reprezentáló sztringekre mutatnak.

A sztring nullkarakterrel, `'\0'`-val lezárt karaktersorozat. Ez nem azonos a null pointerrel: az üres sztringnek van érvényes tárolója, amelynek első karaktere `'\0'`; az `argv[argc]` viszont nem sztringre mutat.

Az argumentumsztringek tartománya tehát:

$$0 \le i < argc.$$

Az `argv[argc]` csak lezáró jelzőérték, nem nyomtatható ki `%s` formátummal. Az `argv[0]` sem garantált abszolút útvonal, és nem hiteles bizonyíték a futó végrehajtható fájl azonosságára.

Linux/shell kapcsolódás. A shell még a program indítása előtt feldolgozza az idézőjeleket és az argumentumképzést. Az `alpha "two words"` szövegből szokásos shellhíváskor két argumentum lesz, nem három; az idézőjelek általában nem kerülnek bele az átadott sztringbe.

3.4.2 Tároló, módosíthatóság és élettartam

ISO C17. Az `argc` és `argv` paraméterek, valamint az argumentumtömb által mutatott sztringek módosíthatók. Az argumentumsztringek nem csupán a belépés pillanatában használhatók: a program befejezéséig rendelkezésre állnak.

Ez nem jelent korlátlanul növelhető buffert. Egy argumentum meglévő karakterének átírása és egy hosszabb sztring bemásolása különböző művelet; az utóbbi túllépheti a rendelkezésre álló tárolót.

Hozzáférés	Tulajdonos és élettartam	Változás, érvénytelenítés és hibakimenet
Az eredeti `argv` argumentumtömb és sztringjei	A tárolót a futtatókörnyezet biztosítja; a program kölcsönzött hozzáférést kap	A saját módosítás megváltoztathatja az adatot; a tárolóhoz való hozzáférés a program befejezéséig érvényes. `free` nem hívható rá
`argv[argc]`	Lezáró null pointer, nincs mögötte argumentumsztring	Dereferálása nem definiált viselkedés
A `getenv` eredménye	A környezetkezelés által biztosított, kölcsönzött hozzáférés	Későbbi környezetlekérdezés vagy platformspecifikus módosítás megváltoztathatja, illetve érvénytelenítheti; hiány esetén null pointer

A táblázat az eredeti argumentumtárolóról szól. Ha az `argv` paramétert más pointerértékre állítjuk, attól az újonnan hivatkozott tároló nem örökli az eredeti élettartam-garanciát.

3.4.3 Környezeti adatok lekérdezése

ISO C17. A környezet szabványos lekérdezési felülete a `<stdlib.h>` fejlécben deklarált `getenv`. Például a `getenv("APP_MODE")`:

- null pointert ad, ha a keresett környezeti adat nem található;
- egy sztringre mutató pointert ad, ha megtalálható;
- üres sztringet is adhat, ami nem azonos a hiánnyal.

A visszaadott sztringet a program nem módosíthatja, és nem szabad `free`-vel felszabadítania. Bár a függvény visszatérési típusa `char *`, a helyi változóhoz célszerű `const char *` típust használni. Ez kifejezi a hozzáférés szerződését.

Egy későbbi `getenv` hívás felülírhatja a korábban kapott sztringet. POSIX környezetben a környezetmódosító műveletekhez további érvényességi korlátok tartoznak. Tartósan megőrzendő értékről ezért saját másolat szükséges.

Linux-kiterjesztés. Egyes futtatókörnyezetek támogatják az `int main(int argc, char *argv[], char *envp[])` alakot. Az `envp` ilyenkor tipikusan null pointerrel lezárt környezeti

sztringtömb. Ez nem a két hordozható ISO C alak egyike; szabványos C kódhoz használjuk a `getenv` felületet.

Linuxon az induló argumentumokat és környezetet az indító folyamat adja át. A gyermek környezetének későbbi módosítása nem írja át a szülő shell környezetét. Mindkét adatforrás külső bemenet: validálást igényelhet, és titkokat is tartalmazhat.

3.4.4 A Linux CRT útja a `main` függvényig

A CRT, azaz *C runtime startup* a C futtatókörnyezet indulását biztosító kód. Nem azonos magával a teljes C standard könyvtárral, bár szorosan együttműködik vele.

Linux/ELF, szokásos glibc-alapú program. Az indulás fő útvonala:

1. Egy sikeres programindításkor a kernel kialakítja az új folyamatképet, és előkészíti az induló adatokat.
2. Dinamikusan linkelt programnál az ELF-ben megadott interpreter, vagyis a dinamikus linker kap először vezérlést. Elvégzi a szükséges betöltési és relokációs feladatokat.
3. A vezérlés eljut a végrehajtható fájl ELF-belépési címére. A szokásos toolchain ezt a CRT `_start` belépési kódjára állítja.
4. A startupkód és a libc együtt előkészíti a C környezetet, majd meghívja a `main` függvényt.
5. A `main` visszatérésekor a futtatókörnyezet normál befejezést kezdeményez.

Réteg	Belépési szerződés
ISO C17 hosted program	A programindítás C-szintű függvénye a `main`
Linux ELF	A belépési cím az ELF fejléc adata; nem szükségképpen a `main` címe
x86-64 System V ABI labor	Az induló gépi állapot az ABI folyamatinicializálási szerződését követi
glibc CRT	Tipikus belépési szimbólum a `_start`; gyakori közreműködő a `__libc_start_main`

A `_start` nem szokásos C függvényhívás címzettje: nincs mögötte olyan rendes hívó, amelyhez egyszerű C `return` művelettel visszatérhetne. A `main` viszont már a létrehozott C környezetben fut.

A pontos feladatmegosztás libc-, toolchain- és linkelésimód-függő. Statikus programnál nincs szükség ugyanarra a dinamikusinterpreter-útvonalra. A fordítódriver azért fontos, mert rendszerint automatikusan hozzáadja a megfelelő startup objektumokat és könyvtárakat.

3.4.5 Kidolgozott argumentum- és környezetvizsgáló példa

Az alábbi teljes program minimuma ISO C17 hosted környezet; maga a C kód nem igényel Linux-kiterjesztést.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    if (printf("argc=%d\n", argc) < 0) {
```

```
        return EXIT_FAILURE;
    }

    for (int i = 0; i < argc; ++i) {
        if (printf("argv[%d]=%s\n", i, argv[i]) < 0) {
            return EXIT_FAILURE;
        }
    }

    const char *mode = getenv("APP_MODE");

    if (printf("APP_MODE=%s\n",
               mode != NULL ? mode : "(unset)") < 0) {
        return EXIT_FAILURE;
    }

    if (fflush(stdout) == EOF) {
        return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}
```

A választott külön laborprofil: Linux/x86-64, System V AMD64 ABI, LP64, a `lab-next` profil (profiltábla). A parancsokban a fordítónevek ezeket a kiadásokat jelöljék. A példát itt nem kíséri fordítási vagy futtatási mérés.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g \
    startup_demo.c -o startup_demo

clang -std=c17 -Wall -Wextra -Wpedantic -O0 -g \
    startup_demo.c -o startup_demo-clang
```

A szemléltető shellhívás: `APP_MODE=verbose ./startup_demo alpha "two words"`. Ennek várt argumentumképe:

Elem	Várt érték ebben a shellhívásban
`argc`	`3`
`argv[0]`	`./startup_demo`
`argv[1]`	`alpha`
`argv[2]`	`two words`
`argv[3]`	null pointer
`getenv("APP_MODE")` tartalma	`verbose`

Helyességi indoklás — bizonyításvázlat. A ciklus csak akkor használja az `argv[i]` elemet, ha $0 \leq i < argc$. Ezért nem adja át az `argv[argc]` null pointert a `%s` konverziónak. A környezeti értéket külön ellenőrzi; nincs újabb `getenv` vagy környezetmódosítás a lekérdezés és felhasználás között. A program nem szabadít fel kölcsönzött tárolót.

A végső `fflush` azért szerepel, mert egy korábbi sikeres `printf` után a pufferelt kiírás még meghiúsulhat. A program ezt hibastátusszal jelzi. Éles szolgáltatásban az argumentumok és környezeti értékek ilyen naplózása titokszivárgást okozhat.

3.4.6 Visszatérés és a shell által látott státusz

ISO C17. A `main` kezdeti hívásából való visszatérés normál programbefejezést jelent, a visszaadott értékkel végzett `exit` hatásának megfelelően. A részletes cleanup-szabályokat a leállásról szóló következő rész tárgyalja.

A `return 0;` és a `return EXIT_SUCCESS;` sikeres befejezést jelez. Az `EXIT_FAILURE` sikertelenséget jelez; numerikus értékét nem kell minden implementációban `1`-nek feltételezni. C17-ben a `main` záró kapcsos zárójelének elérése is `0` visszaadásának felel meg. Ez nem általánosítható minden `int` visszatérési típusú függvényre.

Linux/POSIX. Normál befejezéskor a szülő számára hozzáférhető kilépési kód az átadott érték alsó nyolc bitje. Nemnegatív r esetén:

$$s = r \bmod 256.$$

Így `return 256;` esetén $s = 0$, amit a shell sikernek értelmezhet. Ezért nem helyes tetszőleges belső hibaszámot változtatás nélkül folyamatstátuszként használni.

Egy közvetlenül futtatott egyszerű parancs után a shell `$?` értéke mutatja annak státuszát. Jel miatti megszűnés más kategória; a Bash például rendszerint $128 + N$ értékkel jelzi az N számú jel miatti befejezést. Ez shellkonvenció, nem a `main` visszatérési szabálya.

3.4.7 Hosted és freestanding környezet

ISO C17. Freestanding környezetben az induláskor meghívott függvény nevét és típusát az implementáció határozza meg. Nem követelmény a `main`, és nem feltételezhető a teljes hosted standard könyvtár rendelkezésre állása.

A freestanding program nem egyszerűen „statikusan linkelt program”. Egy statikus Linux CLI továbbra is lehet hosted C program. Hasonlóképpen a GCC vagy Clang `-ffreestanding` kapcsolója önmagában nem készít megfelelő linkerelrendezést, induló gépi állapotot vagy eszközinicializálást.

A döntő különbség a vállalt futtatási szerződés, nem a fájlkiterjesztés vagy az, hogy van-e dinamikus linker.

3.4.8 Gyakori hibák és javításuk

Az `argv[1]` feltétel nélküli használata helyett előbb `argc > 1` szükséges. A `getenv` eredményének közvetlen dereferálása helyett nullpointer-ellenőrzés kell. Az argumentumok és környezeti értékek felszabadítása helyett a kölcsönzött hozzáférés szabályát kell követni.

A `main` és az ELF-belépési cím azonosságának feltételezése téves rétegkeverés. Ugyanilyen hiba a shell nyolcbites státuszát az ISO C teljes, `int` típusú visszatérési értékével azonosítani.

3.4.9 Feladatok

1. **Magyarázó:** Mit szabad feltételezni `argv[0]`-ról, ha `argc == 0`, illetve ha `argc == 1`? Mit jelent az üres sztring?
2. **Kódolási:** Módosítsd a példát úgy, hogy legfeljebb egy felhasználói argumentumot fogadjon el. Hiányzó vagy üres `APP_MODE` esetén használja a `"standard"` értéket.
3. **Diagnosztikai:** Egy Linux-program ELF-belépési címe nem a `main` címe. A program `return 256;` utasítással fejeződik be, a shell mégis sikert mutat. Magyarázd meg mindkét jelenséget.

3.4.10 Megoldások

3.4.10.1 Argumentumhatárok

Ha `argc == 0`, akkor `argv[0]` az `argv[argc]` lezáró null pointer, nem sztring. Ha `argc == 1`, akkor `argv[0]` érvényes sztringre mutat, de lehet üres. Az üres sztring első karaktere `'\0'`.

Az ellenőrzési feltétel: `%s` csak az `i < argc` feltétellel elért argumentumelemhez használható.

3.4.10.2 Bemeneti korlát és alapérték

A következő C17-részlet első ellenőrzése a `main` elejére kerüljön; a környezetlekérdezés pedig váltsa fel az eredeti `mode` deklarációt:

```
if (argc > 2) {
    return EXIT_FAILURE;
}

const char *mode = getenv("APP_MODE");
if (mode == NULL || mode[0] == '\0') {
    mode = "standard";
}
```

A `||` rövidzáras kiértékelése miatt null pointer esetén nem történik `mode[0]` hozzáférés. A sztringliterál statikus tárolási idejű; nem módosítjuk és nem szabadítjuk fel.

Ellenőrzéskor külön eset legyen a hiányzó, üres és nem üres környezeti érték, valamint két felhasználói argumentum elutasítása.

3.4.10.3 A két külön belépési és befejezési réteg

Az ELF-belépési cím rendszerint a CRT `_start` kódjára mutat; a `main` később kap vezérlést. Az eltérés ezért önmagában nem hiba.

A normál Linux-kilépési kód pedig:

$$256 \bmod 256 = 0.$$

A shell így sikert láthat. Ellenőrzéskor külön kell vizsgálni az ELF-belépési pontot, a `main` elérését és a szülő által megfigyelt státuszt.

3.4.11 Összefoglaló

A `main` a hosted C program belépési szerződése, nem feltétlenül a folyamat első végrehajtott kódja. Az argumentumok és környezeti értékek kölcsönzött adatok, eltérő módosíthatósági és érvényességi szabályokkal. A visszatérési érték pedig több rétegen halad át: az ISO C befejezési szerződésén, az operációs rendszer státuszábrázolásán és végül a shell értelmezésén.

3.5 Leállás, cleanup és nem lokális vezérlés

Egy folyamat megszűnése és egy alkalmazási művelet szabályos befejezése nem ugyanaz. A kernel visszaveheti a folyamat memóriáját és lezárhatja file descriptorait, miközben az utolsó naplóüzenet még egy felhasználói térben élő pufferben van, a konfigurációs fájl félkész, a távoli tranzakció pedig rendezetlen.

A main, startup, argumentumok és környezet részben megismert programindulás párja ezért nem egyetlen „kilépés” művelet. Meg kell különböztetnünk a C runtime által végzett lezárást, a kernel erőforrás-visszavételét és az alkalmazás saját helyreállítási feladatait.

Az alábbi nyelvi szabályok **ISO C17** szerint értendők; a **Linux/POSIX** sajátosságokat külön jelöljük. A példák választott laborprofilja Linux, x86-64 System V ABI, LP64, a `lab-next` profil (profiltábla), explicit C17 módban. A parancsok fordítási útmutatók, nem dokumentált teszteredmények.

3.5.1 A leállítási utak szerződése

A **normál programleállás** a C szabvány szerinti kategória: nem feltétlenül jelent sikeres üzleti műveletet. Az `exit(EXIT_FAILURE)` például normál leállás, sikertelenséget jelző állapottal.

Művelet	Hatás	Automatikus alkalmazásszintű cleanup
`return` egy közönséges függvényből	Visszatérés a hívóhoz	Csak a ténylegesen végrehajtott kód
`return status` a `main` kezdeti hívásából	Normál leállás, az `exit(status)` leállítási viselkedésével	`atexit` callbackek és standard streamlezárás
`exit(status)`	Normál programleállás	`atexit` callbackek, streamürítés és -zárás
`_Exit(status)`	Normál leállás kilépési callbackek nélkül	Az ISO C szerint a streamkezelés implementáció által meghatározott
POSIX `_exit(status)`	Folyamatleállítás a szokásos C cleanup nélkül	Nincs `atexit`, nincs stdio-pufferürítés
`quick_exit(status)`	Korlátozott normál leállás	`at_quick_exit` callbackek, majd `_Exit`
`abort()`	Rendellenes leállás	Nincs `atexit` vagy `at_quick_exit` callback

ISO C17: az `exit`, `_Exit`, `quick_exit`, `abort` és a callback-regisztráló függvények deklarációja a `<stdlib.h>` fejlécben található. A `main` végének elérése `return 0` hatású. Hordozható sikerjelzés

a `0` vagy az `EXIT_SUCCESS`, hordozható sikertelenségjelzés az `EXIT_FAILURE`; más állapotértékek értelmezése implementációfüggő.

Az `exit` nem tér vissza, és nem hajtja végre az elhagyott függvények hátralévő utasításait. C-ben nincs olyan általános mechanizmus, amely minden lokális erőforráshoz automatikus lezárást kapcsolna.

Linux/POSIX: a `<unistd.h>` fejlécben deklarált `_exit` és az `_Exit` hatása egyenértékű. Nem ürítik a C `FILE` streamek felhasználói térbeli puffereit, de a folyamat file descriptorai lezáródnak. Az „azonnali” ezért nem valós idejű késleltetési garancia: a kerneloldali lezárás is igénybe vehet időt.

Az `abort()` rendellenes leállást kezdeményez, a `SIGABRT` jelzéshez kapcsolódva. Core dump keletkezése nem garantált: rendszerbeállításoktól is függ. **ISO C17 szerint** a streamek ürítése és lezárása ennél is implementáció által meghatározott; naplómegőrzési stratégiát nem szabad erre építeni.

3.5.2 Kilépési callbackek: hasznos tartalék, nem tranzakciókezelés

Az

```
int atexit(void (*func)(void));
```

deklaráció egy paraméter és visszatérési érték nélküli függvényt regisztrál normál leállásra. A regisztráció sikere nulla, hibája nem nulla visszatérési értékkel jelezhető.

ISO C17: az előre regisztrált callbackek a regisztráció fordított sorrendjében futnak. Ha előbb az alapinfrastruktúra, majd az arra épülő komponens cleanupját regisztráljuk, utóbbi záródik le előbb. Ez azonban csak akkor helyes, ha a valódi függőségek megfelelnek ennek a sorrendnek.

Az `at_quick_exit` külön callbacklistát kezel. A `quick_exit` nem futtatja az `atexit` listáját; az `exit` pedig nem futtatja az `at_quick_exit` listáját.

Egy kilépési callbacknél négy szerződést kell tisztázni:

- Ki birtokolja a lezárandó erőforrást?
- Érvényes-e még a callback által elért állapot?
- Mi akadályozza meg a kétszeres lezárást?
- Hogyan válik láthatóvá a lezárás hibája?

Az utolsó különösen fontos: a callback `void` visszatérésű, tehát egy `fclose` hibáját nem tudja egyszerű visszatérési értékkel átadni az `exit` hívójának. A kritikus lezárást ezért még a végső kilépési állapot kiválasztása előtt végezzük el. Kilépési callbackből ne próbáljunk `longjmp` segítségével „visszamenekülni”; ez nem megengedett helyreállítási út.

3.5.3 Pufferelés és ellenőrizhető lezárás

Egy sikeres `fputs` jelentheti csupán azt, hogy az adat bekerült a stdio-pufferbe. A tényleges továbbítás hibája később, az `fflush` vagy az `fclose` során is jelentkezhet.

ISO C17: az `exit` a callbackek után üríti és lezárja a nyitott streameket, és eltávolítja a `tmpfile` által létrehozott fájlokat. Ebből nem következik, hogy a folyamat kilépési állapota automatikusan sikertelenségre változik egy ürítési hiba miatt.

Az alábbi teljes program, `shutdown.c`, ezért explicit ellenőrzi a lezárást. Külön laborkönyvtárban használjuk: a `shutdown-demo.txt` meglévő tartalmát felülírja.

```c
#include <stdio.h>
#include <stdlib.h>

static FILE *owned;

static int close_owned(void)
{
    FILE *file = owned;
    owned = NULL;
    return file == NULL ? 0 : fclose(file);
}

static void fallback_cleanup(void)
{
    (void)close_owned();
}

int main(void)
{
    if (atexit(fallback_cleanup) != 0) {
        return EXIT_FAILURE;
    }

    owned = fopen("shutdown-demo.txt", "w");
    if (owned == NULL) {
        return EXIT_FAILURE;
    }

    int failed = fputs("completed\n", owned) == EOF;

    if (fflush(owned) == EOF) {
        failed = 1;
    }
    if (close_owned() == EOF) {
        failed = 1;
    }

    return failed ? EXIT_FAILURE : EXIT_SUCCESS;
}
```

Fordítás a megadott profil valamelyik fordítójával:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 shutdown.c -o shutdown-gcc
clang -std=c17 -Wall -Wextra -Wpedantic -O2 shutdown.c -o shutdown-clang
```

Elem	Tulajdonlás és élettartam	Érvénytelenítés, hibakimenet
`owned`	Statikus tárolási idejű pointerobjektum; a program streamtulajdonlását jelöli	`NULL` jelentése: nincs birtokolt stream
Megnyitott stream	Sikeres `fopen` után a program birtokolja	Az `fclose` után nem használható, hiba esetén sem

Elem	Tulajdonlás és élettartam	Érvénytelenítés, hibakimenet
Lokális `file`	A lezáráshoz átvett handle	Nem őrizhető meg a lezárás után
`fallback_cleanup`	A runtime normál leálláskor hívja; csak statikus állapotot ér el	Hibát nem továbbít a kilépési állapotba

Az `owned` nullázása még az `fclose` előtt kizárja, hogy a későbbi fallback ugyanazt a streamet ismét lezárja. Az explicit út minden hibát sikertelenségre fordít; a callback csak tartalék.

Az explicit `fflush` itt ellenőrzési pont: az `fclose` egyébként maga is megkísérli a pufferelt adatok továbbítását. Ürítési hiba után sem szabad kihagyni a lezárást.

Linux/POSIX: a sikeres `fflush` nem jelent tartós tárolóra írást. A kernelhez továbbított adat még elveszhet rendszerösszeomláskor. A tartóssági szerződés további műveleteket igényel; ezt a fájlrendszer-műveletek és tartósság rész tárgyalja.

3.5.4 Mit vesz vissza a kernel?

Linux/POSIX: folyamatmegszűnéskor a kernel felszabadítja a folyamat címterét, és lezárja a folyamat file descriptorait. Ez nem egyenértékű a következőkkel:

- a felhasználói térben pufferelt napló továbbítása;
- egy részlegesen felülírt fájl korábbi állapotának visszaállítása;
- egy névvel létrehozott ideiglenes fájl eltávolítása;
- egy távoli művelet visszagörgetése vagy véglegesítése;
- a gyermekfolyamatok alkalmazási szabály szerinti leállítása.

Az alapvető különbség: **az erőforrás visszavétele nem alkalmazási tranzakció**. A kernel azt tudja, mely descriptorok tartoznak a folyamathoz; azt nem, hogy egy konfigurációcsere mikor tekinthető befejezettnek.

Ezért a helyes terv két rétegű: normál működésben ellenőrzött cleanup, megszakadás esetére pedig helyreállítható állapot és újraindítási protokoll.

3.5.5 Nem lokális vezérlés: `setjmp` és `longjmp`

ISO C17: a `<setjmp.h>` fejléc `setjmp` makrója egy `jmp_buf` objektumba ment visszatérési környezetet. Közvetlen meghívásakor nullát ad; egy megfelelő `longjmp` után ugyanaz a `setjmp` nem nulla értékkel tér vissza.

A `longjmp(env, value)` által előidézett visszatérési érték `value`, kivéve, hogy a nulla argumentum helyett egy lesz. Ez vezérlésátadás, nem állapot-visszagörgetés: fájlírást, memóriamódosítást és erőforrásfoglalást nem von vissza.

A teljes `jump.c` példa:

```
#include <setjmp.h>
#include <stdlib.h>

static jmp_buf checkpoint;
```

```c
static void deep_call(void)
{
    longjmp(checkpoint, 7);
}

int main(void)
{
    volatile int phase = 0;

    if (setjmp(checkpoint) == 0) {
        phase = 1;
        deep_call();
    }

    return phase == 1 ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 jump.c -o jump-gcc
clang -std=c17 -Wall -Wextra -Wpedantic -O2 jump.c -o jump-clang
```

A `checkpoint` statikus tárolója a programé, de a bele mentett környezet csak addig használható, amíg a hozzá tartozó `main`-hívás aktív. A tároló hosszú élettartama nem hosszabbítja meg a mentett hívási környezet érvényességét. Másik szálból ugrani, vagy már visszatért függvény környezetébe visszaugrani nem definiált viselkedés.

3.5.5.1 Miért szükséges itt a `volatile`?

A `longjmp` után indeterminate értékűek azok a lokális változók, amelyek egyszerre:

1. automatikus tárolási idejűek;
2. a `setjmp` hívását tartalmazó függvényhez tartoznak;
3. nem `volatile` minősítésűek;
4. a `setjmp` és a `longjmp` között megváltoztak.

A `phase` az első, második és negyedik feltételt teljesíti. A `volatile` nélkül a visszaugrás utáni összehasonlítására nem építhetnénk helyes programot. A `volatile` itt nem szálbiztonságot és nem erőforrás-biztonságot jelent.

Helyességi indoklás: a közvetlen visszatéréskor belépünk az `if` törzsébe, majd a `phase` egy lesz. A `longjmp` után a feltételben szereplő `setjmp` értéke hét, így a törzs kimarad. A `volatile` változó megtartott értéke alapján a példa sikeres vezérlés-visszatérést jelez.

A `setjmp` használati helye is korlátozott: a bemutatott `if`-feltétel megengedett, de az `int result = setjmp(checkpoint);` inicializálás nem hordozható, szabályos C17 használat.

Ha a `deep_call` erőforrást foglalna, majd annak lezárása előtt ugrana, az elhagyott cleanup utasítás nem futna le. Biztonságosabb az erőforrást a helyreállítási pont tulajdonában tartani, vagy az ugrás előtt explicit felszabadítani. Általános C hibakezeléshez rendszerint egyszerűbbek a visszatérési értékek és a közös cleanup ágak.

3.5.6 Signal miatti leállás és a cleanup határa

Linux/POSIX: egy alapértelmezett leállító signal nem az `exit` hívásán keresztül szünteti meg a folyamatot. Az `atexit` callbackekre és a stdio ürítésére ilyenkor nem támaszkodhatunk. A `SIGKILL` nem fogható el és nem blokkolható.

Egy `SIGTERM` kezelőjéből sem helyes tetszőleges cleanupkódot meghívni. A handler megszakíthatott egy könyvtári műveletet; a stdio-függvények és az `exit` nem általánosan async-signal-safe műveletek.

A szokásos terv: a handler megengedett módon jelzi a leállítási igényt, a normál vezérlés pedig elvégzi a cleanupot. Ennek pontos feltételeit a signal és graceful shutdown rész adja meg. Még ilyen terv mellett is kell helyreállítási stratégia a kikényszerített leállásra.

3.5.7 Feladatok

3.5.7.1 Leállítási garanciák

Linux/POSIX alatt egy teljesen pufferelt streamben ki nem írt adat van. Mi várható `exit`, `_exit` és `abort` esetén? Melyik garantálja a tartósságot?

3.5.7.2 Ellenőrzött lezárófüggvény

Készíts `finish_output(int prior_error)` függvényt a `shutdown.c` programhoz. Előfeltétele, hogy `owned` érvényes stream. Ürítési hiba után is zárjon, és minden hibát őrizzen meg a kilépési állapotban.

3.5.7.3 Ugrás és erőforrásvesztés

A `jump.c` programban elhagyják a `volatile` minősítőt. Emellett a `deep_call` fájlt nyit, de a `longjmp` miatt nem éri el az `fclose` hívását. Mi a két különálló probléma?

3.5.8 Megoldások

3.5.8.1 Leállítási garanciák

Az `exit` megkísérli az ürítést és a lezárást; az I/O sikere nem garantált. A POSIX `_exit` nem üríti a stdio-puffert. Az `abort` esetén nem szabad hordozható ürítési garanciát feltételezni. Tartósságot egyik sem garantál.

Ellenőrzési feltétel: különítsük el az ürítési kísérletet, annak sikerét és a tartós tárolást.

3.5.8.2 Ellenőrzött lezárófüggvény

A következő részlet a `close_owned` után illeszthető; az eredeti fordítási parancsok használhatók:

```
static int finish_output(int prior_error)
{
    int failed = prior_error != 0;

    if (fflush(owned) == EOF) {
        failed = 1;
    }
    if (close_owned() == EOF) {
        failed = 1;
    }

    return failed ? EXIT_FAILURE : EXIT_SUCCESS;
}
```

A `main` sikeres megnyitást követő része így helyettesíthető:

```
return finish_output(fputs("completed\n", owned) == EOF);
```

Ellenőrzési feltétel: minden hibaág eléri a lezárást, a korábbi hiba nem vész el, és a fallback már `NULL` értéket talál.

3.5.8.3 Ugrás és erőforrásvesztés

A módosított, nem `volatile phase` értéke a visszaugrás után indeterminate; nincs garantált eredmény. Ettől függetlenül kimarad a megnyitott stream explicit lezárása.

A `volatile` visszaállítása csak az első problémát javítja. A stream tulajdonlását olyan szintre kell helyezni, amely a visszaugrás után is elvégzi és ellenőrzi a lezárást.

Ellenőrzési feltétel: mind a változóérték érvényességére, mind az erőforrás minden vezérlési úton történő lezárására külön indoklás szükséges.

3.5.9 Összefoglaló

A leállítási utak eltérő runtime-garanciákat adnak. A kritikus I/O-t a végső kilépési állapot kiválasztása előtt kell ellenőrizni; az `atexit` legfeljebb kiegészítő védelem. A kernel erőforrás-visszavétele nem helyettesíti a tranzakciólezárást. A `longjmp` pedig csak a vezérlést helyezi vissza: sem az erőforrásokat, sem az alkalmazási állapotot nem állítja automatikusan helyre.

4 C adattípusok, konverziók és kifejezések

4.1 Egész típusok és adatmodellek

Egy számláló, memóriaméret vagy protokollmező típusának kiválasztása reprezentációs szerződés: milyen értékeket kell tárolni, és milyen feltételezések maradhatnak igazak másik platformon is? C-ben az `int` nem jelent automatikusan 32 bitet, a `long` nem feltétlenül pointerméretű, és még a „bájt” sem szükségképpen nyolc bit.

A Szabványok, dialektusok és hordozhatósági szerződés megkülönböztetését követve külön kezeljük az **ISO C17 garanciáit**, az **ABI által rögzített tulajdonságokat** és a saját programmal **megfigyelt értékeket**. A labor a main, startup, argumentumok és környezet részben megismert hosted programmodellre épül.

4.1.1 Egész típusok, C bájtok és értéktartományok

Az egész típus (*integer type*) diszkrét egész értékek tárolására szolgál. Egy típus **értéktartománya** nem azonos a számára fenntartott tároló bitmintáinak halmazával: a reprezentáció kitöltőbiteket (*padding bits*) is tartalmazhat.

ISO C17. A `sizeof` eredményének egysége egy **C bájt**, amelyet a könyvben C_byte néven különböztetünk meg a nyolcbites oktettől. Egy C bájt bitszámát a `<limits.h>` fejléc `CHAR_BIT` makrója adja meg:

$$\text{CHAR_BIT} \geq 8.$$

Mindig teljesül:

$$\text{sizeof}(\texttt{char}) = \text{sizeof}(\texttt{signed char}) = \text{sizeof}(\texttt{unsigned char}) = 1.$$

Egy `T` típusú objektum tárolója tehát

$$\text{sizeof}(T) \cdot \text{CHAR_BIT}$$

bitből áll. Ez **tárolási bitszám**, nem feltétlenül az értéket hordozó bitek száma.

A `char`, `signed char` és `unsigned char` három különböző típus. A sima `char` a másik kettő valamelyikével azonos tartományú és reprezentációjú; a választás implementáció által meghatározott. Ellenőrzésére a `CHAR_MIN` alkalmas: ha nulla, a sima `char` előjel nélküliként viselkedik.

A `short`, `int`, `long` és `long long` önmagában előjeles típust jelöl. Például a `short`, `short int` és `signed short int` ugyanaz a típus. Az előjel nélküli változatokat az `unsigned` jelöli; az önálló `unsigned` jelentése `unsigned int`.

ISO C17 minimumkövetelmények: az alábbi tartományokat a típusoknak legalább le kell fedniük.

Előjeles típus	Garantáltan lefedett tartomány	Előjel nélküli pár garantált tartománya
`signed char`	$[-127, 127]$	$[0, 255]$
`short`	$[-32767, 32767]$	$[0, 65535]$
`int`	$[-32767, 32767]$	$[0, 65535]$
`long`	$[-2147483647, 2147483647]$	$[0, 4294967295]$
`long long`	$[-(2^{63}-1), 2^{63}-1]$	$[0, 2^{64}-1]$

A tényleges határokat a `<limits.h>` adja: például `INT_MIN`, `INT_MAX`, `UINT_MAX`, illetve `LONG_MIN`, `LONG_MAX`, `ULONG_MAX`. A `long long` megfelelői `LLONG_MIN`, `LLONG_MAX` és `ULLONG_MAX`.

A méretekre:

$$1 = \text{sizeof}(\texttt{char}) \leq \text{sizeof}(\texttt{short}) \leq \text{sizeof}(\texttt{int}) \leq \text{sizeof}(\texttt{long}) \leq \text{sizeof}(\texttt{long long}).$$

Az egyenlőség megengedett. Az `int` és a `long` attól még külön típus, hogy ugyanakkora.

Bizonyításvázlat az unsigned tartományhoz. Ha egy előjel nélküli típusnak v értékbitje van, a legnagyobb érték minden értékbitet egyre állít:

$$1 + 2 + 4 + \cdots + 2^{v-1} = 2^v - 1.$$

Tartománya ezért $[0, 2^v - 1]$. A lényeges feltétel az **értékbitek** száma: kitöltőbitek jelenlétében v nem helyettesíthető automatikusan a teljes tárolási bitszámmal.

4.1.2 Adatmodellek és ABI-határok

Az **adatmodell** (*data model*) néhány alapvető típus szélességének együttese. Az ILP32 az `int`, `long` és pointer 32 bites szélességére utal. LP64 esetén a `long` és a pointer 64 bites, az `int` 32 bites. LLP64 esetén a `long long` és a pointer 64 bites, de az `int` és a `long` 32 bites.

Az alábbi **ABI-profilokban** `CHAR_BIT == 8`; a méretek C bájtban szerepelnek. Ezek profilhoz kötött értékek, nem ISO-garanciák és nem közölt mérési eredmények.

Típus	ILP32: Linux i386, System V	LP64: Linux x86-64, System V	LLP64: Windows x64
`char`	1	1	1
`short`	2	2	2
`int`	4	4	4
`long`	4	8	4
`long long`	8	8	8
`void *`	4	8	8
Fordítási profil	GCC 14.2.0, `-std=c17 -m32`	GCC 14.2.0, `-std=c17 -m64`	MSVC 19.40, x64, `/std:c17`

A modellnevek önmagukban nem határozzák meg például a `short` méretét vagy a típusok igazítását; a táblázat ezért konkrét ABI-példákat is megnevez.

Gyakorlati következmény: a „64 bites rendszer” nem elegendő egy típusméret meghatározásához. Egy Linuxon pointerméretűnek bizonyuló `unsigned long` Windows x64 alatt csak 32 bites lehet. A gépi szóhossz, az egész típusok mérete és a pointerméret külön tulajdonság.

A táblázat szerializálási formátumot sem definiál. Egy `long` objektum tárolójának közvetlen kiírása nem eredményez hordozható protokollmezőt; ehhez külön reprezentációs és bájtsorrend-szerződés szükséges.

4.1.3 Szélesség és feladat szerint választott típusok

4.1.3.1 Exact-width, least és fast típusok

ISO C17. A `<stdint.h>` többféle típuscsaládot kínál.

Az **exact-width**, vagyis pontos szélességű típusok neve például `int32_t` és `uint32_t`. Ezek pontosan a megadott bitszámúak, kitöltőbit nélkül. Az `int32_t` tartománya $[-2^{31}, 2^{31}-1]$, a `uint32_t` tartománya $[0, 2^{32}-1]$.

Ezek a típusok **opcionálisak**: csak megfelelő alaptípus rendelkezésre állásakor léteznek. Elérhetőségük a hozzájuk tartozó határmakróval ellenőrizhető, például `#ifdef UINT32_MAX` segítségével.

A pontos bitszám nem jelent rögzített C bájtszámot. Ha `uint32_t` létezik:

$$\text{sizeof}(\texttt{uint32_t}) \cdot \text{CHAR_BIT} = 32.$$

Ez nyolcbites C bájttal négy, tizenhat bitessel két C bájt. Utóbbi környezetben `uint8_t` nem létezhet.

A **least-width** család, például `uint_least32_t`, legalább a megadott szélességhez szükséges tartományt biztosítja, a követelményt kielégítő legkisebb méretű megfelelő egész típussal. A **fast-width** család, például `uint_fast32_t`, az implementáció által rendszerint leggyorsabbnak kijelölt, legalább ilyen tartományú típust adja. A `fast` nem egy konkrét alkalmazáson végzett teljesítménymérés eredménye; egy `uint_fast32_t` lehet 64 bites is.

A 8, 16, 32 és 64 szélességű `least` és `fast` családok kötelezők. A megfelelő határmakrók például `UINT_LEAST32_MAX` és `INT_FAST32_MIN`.

A `typedef`-nevek nem hoznak létre új, elkülönült egész típusokat. A `uint32_t` valamely létező egész típus alternatív neve; nem szabad feltételezni, hogy ez mindenhol `unsigned int`.

4.1.3.2 Méretek, különbségek és pointerek egész reprezentációja

Típus	Rendeltetés	Fontos korlát
`size_t`	A `sizeof` eredményének előjel nélküli típusa	Nem szükségképpen `unsigned long`; felső határa `SIZE_MAX`
`ptrdiff_t`	Pointerkivonás előjeles eredménytípusa	A különbségnek ábrázolhatónak kell lennie
`uintptr_t`	Opcionális előjel nélküli típus `void *` oda-vissza konverziójához	Nem hordozható hálózati címformátum

A `size_t` és `ptrdiff_t` elérhető a `<stddef.h>` fejlécből. Pointerkivonásnál az azonos tömbhöz tartozás feltételeit is teljesíteni kell; ezt a Pointeraritmetika, objektumhatár és provenance tárgyalja. A különbség elemekben, nem általában C bájtokban értendő.

A `uintptr_t` a `<stdint.h>` opcionális típusa. Szerződése szerint egy érvényes `void *` érték ilyen egésszé, majd vissza `void *` típussá alakítva az eredetivel egyenlő pointert ad. Ez nem jogosít fel tetszőleges egész érték pointerként történő használatára, és nem konzerválja a hivatkozott objektum élettartamát.

4.1.4 Előjeles reprezentáció: C17 és C23

ISO C17. Az általános előjeles egész típusokhoz három reprezentáció megengedett: előjel–abszolútérték (*sign-and-magnitude*), egyes komplemens (*ones' complement*) és kettes komplemens (*two's complement*).

Nyolc tárolási bit, kitöltőbit nélküli ábrázolás és minden bitminta értékként kezelése mellett az első kettő $[-127, 127]$, a kettes komplemens $[-128, 127]$ tartományt ad. Az első kettő negatív zérót is kódol. Ez magyarázza a C17 minimumtáblázatának szimmetrikus negatív határait.

Fontos kivétel: az opcionális `intN_t` típusok **már C17-ben is kettes komplemensűek**.

C23-eltérés. Az előjeles egész típusok reprezentációja kettes komplemensre szűkül. Ettől azonban az `int` nem lesz kötelezően 32 bites, a C bájt nem lesz kötelezően oktett, és a típusméretek sem válnak platformfüggetlenné.

A reprezentációt a műveleti szabályoktól is el kell választani: az előjeles túlcsordulás C23-ban sem válik általánosan körbeforduló aritmetikává. A konverziókat és túlcsordulást a Promóciók, signedness és túlcsordulás rész tárgyalja.

4.1.5 Megfigyelési labor

A következő önálló program minimuma **ISO C17, hosted környezet**. Választott laborprofil: Linux x86-64 System V, LP64, a `lab-next` profil (profiltábla); fordítók a `lab-next` profil (profiltábla). A helyi eszközverziókat külön ellenőrizni kell. A példához itt nem tartozik tényleges fordítási vagy futtatási eredmény.

```c
/* integer_probe.c */
#include <inttypes.h>
#include <limits.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>

int main(void)
{
    printf("CHAR_BIT=%d\n", CHAR_BIT);
    printf("sizeof: char=%zu short=%zu int=%zu "
           "long=%zu long long=%zu void*=%zu\n",
           sizeof(char), sizeof(short), sizeof(int),
           sizeof(long), sizeof(long long), sizeof(void *));

    printf("char: %jd..%ju\n",
           (intmax_t)CHAR_MIN, (uintmax_t)CHAR_MAX);
```

```
    printf("int: %d..%d; unsigned int: 0..%u\n",
           INT_MIN, INT_MAX, UINT_MAX);
    printf("long: %ld..%ld; unsigned long: 0..%lu\n",
           LONG_MIN, LONG_MAX, ULONG_MAX);
    printf("long long: %lld..%lld\n", LLONG_MIN, LLONG_MAX);

    printf("size_t: %zu C_byte; max=%zu\n",
           sizeof(size_t), (size_t)SIZE_MAX);
    printf("ptrdiff_t: %zu C_byte; %td..%td\n",
           sizeof(ptrdiff_t),
           (ptrdiff_t)PTRDIFF_MIN, (ptrdiff_t)PTRDIFF_MAX);

#ifdef UINT32_MAX
    uint32_t field = 42;
    printf("uint32_t: %zu C_byte; value=%" PRIu32 "\n",
           sizeof(field), field);
#else
    puts("uint32_t nem erheto el");
#endif

#ifdef UINTPTR_MAX
    printf("uintptr_t: %zu C_byte\n", sizeof(uintptr_t));
#else
    puts("uintptr_t nem erheto el");
#endif

    return 0;
}
```

Fordítás a megadott kiadásokra mutató `gcc` és `clang` parancsokkal:

```
gcc -std=c17 -m64 -Wall -Wextra -Wpedantic -Wconversion -O0 -g integer_probe.c
 ↪  -o integer_probe-gcc
clang -std=c17 -m64 -Wall -Wextra -Wpedantic -Wconversion -O0 -g integer_probe.c
 ↪  -o integer_probe-clang
```

A formátumok a típusok szerződését követik: `size_t` esetén `%zu`, `ptrdiff_t` esetén `%td`, `uint32_t` esetén a `<inttypes.h>` által adott `PRIu32` használható. A `char` határainak kiírásánál az `intmax_t` és `uintmax_t` minden megfelelő előjeles, illetve előjel nélküli egész érték befogadására alkalmas típust biztosít.

A labor megfigyeléseit két lépésben értelmezzük. Először ellenőrizzük az ISO-minimumokat és méretrelációkat. Ezután összevetjük az adatokat a kiválasztott ABI-profillal. Egy LP64-nek megfelelő eredmény nem bizonyítja, hogy ugyanaz a program minden platformon ugyanilyen méreteket kap.

4.1.6 Gyakori hibák és javításuk

A `sizeof(T) * 8` kifejezés rejtett oktettfeltételezés. Tárolási bitszámhoz `CHAR_BIT` szükséges, értéktartományhoz pedig a megfelelő határmakró.

A `uint32_t` feltétel nélküli használata elrejti az opcionális típus követelményét. Ha pontos szélesség kell, ellenőrizzük az elérhetőséget; ha csak legalább ekkora tartomány kell, válasszunk `uint_least32_t` típust.

A méretérték `unsigned long` változóba másolása LP64-en működőképesnek tűnhet, LLP64-en azonban értékvesztést okozhat. Méretekhez alapértelmezésben `size_t` való.

A formátumspecifikátort sem a feltételezett mérethez, hanem a tényleges típushoz kell választani. Az azonos bitszám önmagában nem tesz helyessé egy variadikus `printf` hívást.

4.1.7 Feladatok

1. **Magyarázó feladat.** Mely állítások ISO C17-garanciák: `sizeof(char) == 1`, `CHAR_BIT == 8`, `sizeof(int) == 4`, `INT_MAX >= 32767`? Következik-e nyolcbites C bájtból a négybájtos `int`?
2. **Kódolási feladat.** Egészítsd ki a laborprogramot a `uint_least32_t` és `uint_fast32_t` méretének és felső határának kiírásával. Használj típushelyes formátumokat.
3. **Diagnosztikai feladat.** Az alábbi, már deklarált `payload` objektumra vonatkozó C17-részlet miért hordozhatósági hiba LLP64-en? Javítsd ki. Feltételezz olyan implementációt, amelyen az objektum mérete meghaladhatja az `ULONG_MAX` értékét.

   ```c
   /* Hibás hordozhatósági feltételezés; szemléltető részlet. */
   unsigned long bytes = sizeof payload;
   printf("%lu\n", bytes);
   ```

4.1.8 Megoldások

4.1.8.1 Az ISO-garanciák azonosítása

Az első és a negyedik állítás garantált. A másik kettő platformfüggő.

Nyolcbites C bájttal az `int` legalább két C bájtot igényel, mert legalább 65535 különböző értéket kell tudnia tárolni, miközben egyetlen ilyen bájtnak csak 256 bitmintája van. Ebből nem következik a négybájtos méret.

Ellenőrzési feltétel: az indoklás tartományból és szabványos garanciából induljon ki, ne egyetlen gép méréséből.

4.1.8.2 A típuscsaládok megfigyelése

A következő C17-részlet a laborprogram `main` függvényébe illeszthető; annak fejléceit és buildparancsait használja:

```c
printf("uint_least32_t: %zu C_byte; max=%" PRIuLEAST32 "\n",
       sizeof(uint_least32_t), UINT_LEAST32_MAX);
printf("uint_fast32_t: %zu C_byte; max=%" PRIuFAST32 "\n",
       sizeof(uint_fast32_t), UINT_FAST32_MAX);
```

Ellenőrzési feltétel: mindkét felső határ legalább $2^{32}-1$. A két méret egyezése nem követelmény, és a nagyobb méret önmagában nem bizonyít nagyobb végrehajtási sebességet.

4.1.8.3 A mérettárolás javítása

Az LLP64-profilban a `size_t` 64 bites, az `unsigned long` 32 bites. A kiírás formátuma a hibás változatban helyes a változóhoz, de a méret már az inicializáláskor elveszhet.

Javított C17-részlet, `<stddef.h>` és `<stdio.h>` mellett:

```
size_t bytes = sizeof payload;
printf("%zu\n", bytes);
```

Ellenőrzési feltétel: a `sizeof` eredménye ne kerüljön szűkebb típusba a tárolás vagy kiírás miatt. A nagy objektum tényleges létrehozása nem szükséges a típushiba felismeréséhez.

4.1.9 Rövid összefoglaló

A típusnév, a tárolási méret és az értéktartomány három külön információ. A `sizeof` C bájtban mér, a `CHAR_BIT` a bájt bitszámát, a határmakrók az értéktartományt adják. Az ABI konkrét méreteket rögzít, de ezek nem ISO-garanciák. Pontos szélességhez ellenőrzött exact-width típus, minimális tartományhoz `least` típus, objektummérethez `size_t` használható.

4.2 Promóciók, signedness és túlcsordulás

Egy hálózati hosszmező, egy előjeles hibajelzés és egy memóriaméret gyakran ugyanabban a kifejezésben találkozik. Matematikailag mindegyik egész szám, C-ben azonban a típusuk dönti el, hogyan történik az összehasonlítás, a kivonás vagy a szorzás. A hibás konverzió következménye lehet elutasított érvényes index, óriásira változó negatív hossz vagy túl kicsire számított allokáció.

Az Egész típusok és adatmodellek alapján már különválasztjuk a típus nevét, tartományát és reprezentációját. Most a **kifejezés típusát** követjük: ezt általában nem az eredményt fogadó változó, hanem az operátor és az operandusok típusa határozza meg.

Az alábbi nyelvi szabályok az ISO C17-re vonatkoznak. A konkrét számpéldák laborprofilja Linux, x86-64 System V ABI, LP64, `CHAR_BIT == 8`; itt az `int` 32 bites, a `long` 64 bites, a `size_t` pedig `unsigned long`. A használt exact-width típusok rendelkezésre állását külön feltételezzük. A példák eredményei levezetett elvárások, nem dokumentált futtatási eredmények.

4.2.1 Az implicit konverziók két lépcsője

4.2.1.1 Egész promóciók

Az **egész promóciók** (*integer promotions*) a kis rangú egész típusokat emelik `int` vagy `unsigned int` típusra. A rang (*integer conversion rank*) a nyelv által meghatározott típusviszony; nem egyszerűen a tárolási méret másik neve.

A szokásos kis egész típusokra, például `_Bool`, `char`, `signed char`, `unsigned char`, `short` és `unsigned short` esetén:

- ha az `int` az eredeti típus **minden értékét** képes ábrázolni, a cél `int`;
- különben a cél `unsigned int`.

Nem az aktuális érték számít. Egy `unsigned short` akkor is ugyanarra a típusra promótálódik, ha éppen nulla van benne. Az enumerációk és bitmezők is érintettek lehetnek, saját típusszabályaik szerint.

C17-részlet a megadott laborprofilra:

```
uint16_t received = 4;
uint16_t header = 8;
int payload = received - header;
```

A levezetés:

1. A `uint16_t` értéktartománya $[0, 65535]$.
2. A profil 32 bites `int` típusa ezt teljes egészében ábrázolja.
3. Mindkét operandus `int` típusra promótálódik.
4. A kivonás ezért `int` aritmetikával történik: az eredmény `-4`.

Az unsigned változók tehát **nem garantálják**, hogy az aritmetikai művelet is unsigned lesz.

4.2.1.2 Szokásos aritmetikai konverziók

Az olyan bináris operátoroknál, mint a `+`, `-`, `*`, `/` és az összehasonlítások, az egész promóciókat a **szokásos aritmetikai konverziók** (*usual arithmetic conversions*) követik.

Két egész operandusra, a promóciók után:

1. Azonos típus esetén nincs további konverzió.
2. Azonos előjelesség esetén a kisebb rangú operandus a nagyobb rangú típusra konvertálódik.
3. Vegyes signed/unsigned esetben legyen az előjeles típus S, az előjel nélküli U:
   - ha U rangja legalább akkora, mint S rangja, a közös típus U;
   - különben, ha S minden U-értéket ábrázol, a közös típus S;
   - egyébként a közös típus az S-nek megfelelő unsigned típus.

A laborprofilon:

Operandustípusok	Közös típus	Indok
`int`, `unsigned int`	`unsigned int`	Az unsigned rang nem kisebb
`long`, `unsigned int`	`long`	A 64 bites `long` minden 32 bites unsigned értéket ábrázol
`int`, `size_t`	`unsigned long`	A `size_t` unsigned, rangja nagyobb

Összehasonlításnál ez a **vizsgált operandusok** közös típusa; az összehasonlítás eredménye C-ben `int`, értéke nulla vagy egy.

Ezek a szabályok nem minden operátorra alkalmazhatók változtatás nélkül. Az értékadás például a kiszámított értéket a bal oldal típusára konvertálja; a léptetőoperátorok külön szabályait a Kifejezések kiértékelése, sequencing és bitműveletek tárgyalja.

4.2.2 Negatív index unsigned hossz mellett

C17-részlet a laborprofilra:

```
int index = -1;
size_t length = 100;

if (index < length) {
    /* Nem ez az ág következik a típuskonverziókból. */
}
```

A közös típus `size_t`, vagyis `unsigned long`. Az egész érték unsigned típusra konvertálásakor az eredmény a cél értéktartományához tartozó moduló szerinti érték.

Ha a cél maximuma M, akkor:

$$-1 \equiv M \pmod{M+1}.$$

Az összehasonlítás így a profilon `SIZE_MAX < 100`, tehát hamis. A matematikailag kisebb negatív index a konverzió miatt nagy pozitív értékként szerepel.

A helyes ellenőrzés előbb a jelentést, utána a reprezentációt kezeli. Hordozható C17-részlet, `<stdint.h>` használatával:

```
if (index >= 0 &&
    (uintmax_t)index < (uintmax_t)length) {
    /* Az index a [0, length) tartományban van. */
}
```

A `&&` rövidzáras: negatív indexnél a jobb oldal nem értékelődik ki. A `uintmax_t` minden unsigned egész típus értékeit képes ábrázolni; a nemnegatív `int` érték is belefér.

A megszokott `(size_t)index < length` változat ugyancsak megfelelő, ha előtte ellenőrizzük a nemnegativitást, és az alkalmazott profilban minden nemnegatív `int` belefér a `size_t` típusba. Ez a választott LP64 profilban teljesül, de nem önálló, minden C-implementációra érvényes axióma.

4.2.3 Moduláris aritmetika, UB és szűkítés

Ha a művelet eredménytípusa N értékbites unsigned egész típus, az összeadás, kivonás és szorzás eredménye modulo 2^N értendő:

$$r = (a \circ b) \bmod 2^N, \qquad \circ \in \{+, -, \times\}.$$

Ez definiált **körbefordulás** (*wraparound*), de méretszámításban rendszerint programhiba. Egy körbefordult kéréshez túl kevés memória foglalható.

Ezzel szemben, ha egy előjeles aritmetikai eredmény nem ábrázolható az eredménytípusban, az **nem definiált viselkedés**, UB. Az `INT_MAX + 1` nem hordozható módja az `INT_MIN` előállításának. Az optimalizáló nem köteles a processzor esetleges körbefordulását megőrizni.

Fontos különbség az aritmetika és az utólagos konverzió között:

```c
/* C17-részlet; uint8_t létezik. */
uint8_t a = 250;
uint8_t b = 10;
uint8_t result = a + b;
```

Itt az operandusok `int` típusra promótálódnak. A `260` eredmény ábrázolható `int`-ben, tehát nincs aritmetikai túlcsordulás. Ezután az értékadás `uint8_t` típusra konvertál: $260 \bmod 256 = 4$.

Szűkítésnek (*narrowing conversion*) itt az olyan konverziót nevezzük, amelynél a cél nem képes a forrástípus minden értékét ábrázolni. C17-ben:

- unsigned cél esetén a tartományon kívüli érték modulárisan konvertálódik;
- signed cél esetén a nem ábrázolható érték eredménye implementáció által meghatározott, vagy implementáció által meghatározott jelzés keletkezik.

A signed célra történő konverzió tehát nem ugyanaz a szabálykategória, mint a signed aritmetikai túlcsordulás.

Az explicit cast **konverziós kérés, nem tartományellenőrzés**. C17-részlet:

```c
if (wire_length <= UINT16_MAX) {
    uint16_t short_length = (uint16_t)wire_length;
    /* short_length itt veszteségmentesen előállított érték. */
}
```

Feltétel: `wire_length` típusa `uint32_t`, és mindkét exact-width típus elérhető. Signed forrásnál az alsó határt is ellenőrizni kell.

Különösen veszélyes minta:

```c
/* Hibás lehet: a cast túl későn történik. */
size_t bytes = (size_t)(count * stride);
```

Ha `count` és `stride` típusa `int`, előbb `int` szorzás történik. A későbbi cast nem javítja meg a korábbi UB-t.

4.2.4 Méretszámítás allokáció előtt

Legyen a, b két `size_t` érték, $M =$ `SIZE_MAX`.

Az összeadás pontosan akkor fér el, ha:

$$a + b \le M \iff a \le M - b.$$

A szorzás pedig akkor fér el, ha:

$$b = 0 \quad \text{vagy} \quad a \le \left\lfloor \frac{M}{b} \right\rfloor.$$

Bizonyításvázlat. Mivel $0 \le b \le M$, az $M - b$ kivonás biztonságos. Pozitív b mellett az $ab \le M$ egyenlőtlenség osztható b-vel; egész a esetén a lefelé kerekített hányados adja a pontos korlátot. A nulla szorzót külön kezeljük, így nincs nullával osztás.

A következő teljes C17-program egy már dekódolt hálózati elemszámból számít méretet. Nem végez allokációt: annak külön hibaszerződése a malloc, calloc, free és allokációs szerződés témája. A 24 és 16 itt alkalmazásszerződésből származó, C bájtban megadott méret, nem struktúralayout.

```c
#include <stdint.h>
#include <inttypes.h>
#include <stdio.h>

#ifndef UINT32_MAX
#error "This example requires uint32_t."
#endif

int main(void)
{
    uint32_t wire_count = 0;

    /* Kizárólag ismert, ábrázolható tesztérték beolvasása. */
    if (sscanf("42", "%" SCNu32, &wire_count) != 1) {
        fputs("conversion failed\n", stderr);
        return 1;
    }

    if ((uintmax_t)wire_count > (uintmax_t)SIZE_MAX) {
        fputs("count does not fit size_t\n", stderr);
        return 1;
    }

    const size_t count = (size_t)wire_count;
    const size_t stride = 24;
    const size_t header = 16;

    if (stride != 0 && count > SIZE_MAX / stride) {
        fputs("payload size overflow\n", stderr);
        return 1;
    }
    const size_t payload = count * stride;

    if (payload > SIZE_MAX - header) {
        fputs("total size overflow\n", stderr);
        return 1;
    }
    const size_t total = header + payload;

    printf("count=%" PRIu32 " total=%zu\n", wire_count, total);
    return 0;
}
```

Az `sscanf` nem tulajdonló hozzáférést kap a `wire_count` helyi objektumhoz, és a címet nem tárolja el. Az objektum tárolója és élettartama a `main` blokkjához kötött; a hívás alatt nem érvénytelenedik. A bemeneti sztringliterál statikus tárolási idejű. A program saját felismert hibakimenete üzenet és nem nulla kilépési státusz.

Választott, itt nem ellenőrzött labor-eszközlánc: a `lab-next` profil (profiltábla), a fent megadott Linux-targettel. Fordítás külön laborlépésként:

```
gcc -std=c17 -O2 -Wall -Wextra -Wpedantic -Wconversion -Wsign-conversion sizes.c
 ↪  -o sizes-gcc
```

```
clang -std=c17 -O2 -Wall -Wextra -Wpedantic -Wconversion -Wsign-conversion
 ↪  sizes.c -o sizes-clang
```

A számtani levezetés szerint $42 \cdot 24 + 16 = 1024$. Ez várt eredmény, nem mérés. Az aritmetikai ellenőrzés mellett valódi szolgáltatásban külön erőforráskorlát is szükséges: a reprezentálható méret nem feltétlenül elfogadható méret.

4.2.5 Mérethelyes formátumok

A variadikus `printf` nem következteti ki az argumentum típusát a formátumszövegből. A formátum és az átadott típus össze nem illése UB-t okozhat.

Értéktípus	Kiírás	Beolvasás
`size_t`	`%zu`	`%zu`, `size_t *` argumentummal
`ptrdiff_t`	`%td`	`%td`, `ptrdiff_t *` argumentummal
`uint32_t`	`"%" PRIu32`	`"%" SCNu32`, `uint32_t *` argumentummal
`int64_t`	`"%" PRId64`	`"%" SCNd64`, `int64_t *` argumentummal

A `PRI` és `SCN` makrókat az `<inttypes.h>` biztosítja; az exact-width változatok az adott típus elérhetőségéhez kötődnek. A kiírási és beolvasási makrók nem felcserélhetők.

A `SCN` makró csak a **típushelyes formátumot** adja meg. C17-ben a célobjektumban nem ábrázolható numerikus `scanf`-eredmény UB lehet; a visszatérési érték ellenőrzése önmagában nem teszi biztonságossá a tetszőleges bemenetet. Ezért használ a példa ismert literált. Külső numerikus szöveghez célszerű például `strtoumax`-alapú feldolgozás, külön előjel-, tartomány-, végpozíció- és hibavizsgálattal; ennek részleteit a Formázás és megbízható szövegfeldolgozás tárgyalja.

4.2.6 Gyakori hibák és javításuk

- **„Unsigned változókkal nem lehet UB a szorzás.”** A promóciók miatt például két `uint16_t` szorzása a profilon `int` szorzás; nagy operandusokkal túlcsordulhat.
- **„A nagyobb célváltozó megvédi a műveletet.”** Nem: a jobb oldal típusát előbb kell levezetni.
- **„A cast megszüntette a warningot, tehát javítottam.”** A tartomány bizonyítása nélkül csak a diagnosztika tűnhetett el.
- **„Előbb összeadok, majd megnézem, belefért-e.”** Méretszámításnál inkább a művelet előtti, kivonásra vagy osztásra épülő korlátot használjuk.

4.2.7 Feladatok

4.2.7.1 Magyarázat: kivonás és értékadás

A laborprofilon `a` és `b` típusa `uint16_t`, értékük 3 és 5. Mi a különbség az `int x = a - b;` és a `size_t y = a - b;` között? Nevezd meg a kivonás és az értékadás konverzióit.

4.2.7.2 Kódolás: alkalmazási méretkorlát

Írj `frame_fits(count, stride, header, limit)` függvényt `size_t` paraméterekkel, amely túlcsordulás nélkül eldönti:

$$header + count \cdot stride \leq limit.$$

A nulla `stride` is legyen megengedett.

4.2.7.3 Diagnosztika: késői cast

A laborprofilon elemezd ezt az elkülönített, hibás C17-részletet:

```
int count = 100000000;
int stride = 24;
size_t bytes = (size_t)(count * stride);
printf("%u\n", bytes);
```

Keress két egymástól független hibát. UB esetére ne jósolj kimenetet.

4.2.8 Megoldások

4.2.8.1 A kivonás eredménye

Mindkét operandus `int` típusra promótálódik, ezért a kivonás eredménye `int` típusú `-2`. Az `x` értéke `-2`; a `y` inicializálásakor külön unsigned konverzió történik:

$$-2 \bmod (\texttt{SIZE_MAX} + 1) = \texttt{SIZE_MAX} - 1.$$

Ellenőrzési feltétel: a levezetés külön kezelje az operátor és a célváltozó típusát.

4.2.8.2 Korlátvizsgáló függvény

C17-függvényrészlet; önálló programhoz hívó és `main` szükséges:

```
#include <stdbool.h>
#include <stddef.h>

bool frame_fits(size_t count, size_t stride,
                size_t header, size_t limit)
{
    if (header > limit) {
        return false;
    }

    return stride == 0 ||
           count <= (limit - header) / stride;
}
```

A kivonást az első ellenőrzés védi, az osztást a rövidzáras `||`. Szorzást egyáltalán nem végzünk.

Ellenőrzési esetek: `(42,24,16,1024)` igaz, `(43,24,16,1024)` hamis; nulla `stride` mellett kizárólag a fejléc mérete számít.

4.2.8.3 A két független hiba

A szorzás `int` típusban történik. A matematikai eredmény 2 400 000 000, amely a profil `INT_MAX` értékénél nagyobb: ez UB, még a cast előtt.

Emellett `%u` helyett `%zu` szükséges a `size_t` kiírásához. A javítás sorrendje: előjel- és tartományvizsgálat, konverzió `size_t` típusra, szorzási korlátvizsgálat, majd szorzás és típushelyes kiírás. A warning hasznos bizonyíték lehet egy hibára, de hiánya nem bizonyítja a helyességet.

4.2.9 Összefoglaló

Egész kifejezéseknél először a promóciókat, majd a közös típust, végül az értékadás konverzióját vizsgáld. Az unsigned körbefordulás definiált, de méretszámításban veszélyes; a signed aritmetikai túlcsordulás UB. A cast nem ellenőrzés. Negatív értékeket unsigned konverzió előtt, összeadást és szorzást pedig a művelet végrehajtása előtt kell tartományvizsgálattal védeni.

4.3 Lebegőpontos reprezentáció és numerikus hibák

Egy monitorozó rendszerben a késleltetés, a CPU-terhelés és az időbélyeg gyakran lebegőpontos érték. Ezek azonban nem valós számok korlátlan pontosságú tárolói: véges számú reprezentálható érték közül választanak. Emiatt két különböző időpont azonos tárolt értékké válhat, egy összegzés eredménye függhet a műveleti sorrendtől, és egy optimalizálási kapcsoló megváltoztathatja a numerikus szerződést.

A Promóciók, signedness és túlcsordulás részhez hasonlóan itt is külön kell választanunk a típus, a konverzió és a művelet szabályait. Az alap ISO C17; az IEEE 754 szerinti, kettes alapú működést külön implementációs feltételként kezeljük.

4.3.1 Típusok és ellenőrizhető numerikus profil

Az ISO C17 három alapvető valós lebegőpontos típusa a `float`, a `double` és a `long double`. Értékkészletük egymásba ágyazott: a `float` értékei a `double`, a `double` értékei a `long double` értékkészletének részét képezik. A `long double` ettől még nem feltétlenül pontosabb a `double`-nél.

A `sizeof` önmagában nem mondja meg a pontosságot: tárolási méretet mér C bájtban, amely kitöltést is tartalmazhat. A numerikus tulajdonságokat a `<float.h>` írja le.

Makró	Jelentés
`FLT_RADIX`	A lebegőpontos reprezentáció számrendszerének alapja
`FLT_MANT_DIG`, `DBL_MANT_DIG`, `LDBL_MANT_DIG`	A szignifikand pontossága `FLT_RADIX` alapú számjegyekben
`DBL_MIN`, `DBL_MAX`	A legkisebb pozitív normál, illetve legnagyobb véges `double`

Makró	Jelentés
`DBL_TRUE_MIN`	A legkisebb pozitív reprezentálható `double`, a subnormál értékeket is beleértve
`DBL_EPSILON`	Az 1 és az azt követő reprezentálható `double` különbsége
`DBL_DIG`	Biztonságosan megőrizhető decimális pontosság a decimális–bináris–decimális irányban
`DBL_DECIMAL_DIG`	A `double`–decimális–`double` visszaalakításhoz elegendő decimális számjegyszám
`FLT_EVAL_METHOD`	A kifejezések esetleges szélesebb köztes kiértékelésének jelzése

A `DBL_` makróknak megfelelő `FLT_` és `LDBL_` változataik is vannak. A subnormál támogatásról a `*_HAS_SUBNORM` makrók adnak információt; értékük akár meghatározhatatlan támogatást is jelezhet.

Konkrét laborprofil, nem ISO-követelmény: Linux, x86-64 System V ABI, LP64, `CHAR_BIT == 8`, alapértelmezett ABI-szerinti `long double`.

Típus	Feltételezett formátum	Szignifikand pontossága	Tárolási méret
`float`	IEEE 754 binary32	24 bit	4 C bájt
`double`	IEEE 754 binary64	53 bit	8 C bájt
`long double`	x87 kiterjesztett formátum	64 bit	16 C bájt, ebből 80 bit a numerikus formátum

Ez nem univerzális „64 bites gépes” táblázat. Más ABI vagy fordítókapcsoló más `long double`-t eredményezhet.

A literál típusa is számít: `0.1` típusa `double`, `0.1f` típusa `float`, `0.1L` típusa `long double`. A korábban `float`-ra kerekített érték későbbi `double`-re alakítása nem állítja vissza az elveszett információt.

4.3.2 Reprezentáció és időadatok felbontása

Kettes alapú formátumban egy nem nulla **normál szám** alakja:

$$x = (-1)^s \left(1 + \frac{k}{2^{p-1}}\right) 2^e,$$

ahol s az előjelbit, p a szignifikand pontossága, k egész, $0 \le k < 2^{p-1}$, e pedig a megengedett tartományba eső kitevő.

Azonos kitevő mellett a szomszédos értékek távolsága:

$$\Delta = 2^{e-(p-1)}.$$

Bizonyításvázlat. Két szomszédos reprezentációban k eggyel változik. A szignifikand különbsége $2^{-(p-1)}$; ezt 2e-vel szorozva kapjuk a távolságot.

Binary64 esetén $p = 53$. Egy körülbelül $1{,}7 \cdot 10^9$ másodperces Unix-időbélyegre:

$$2^{30} \leq t < 2^{31},$$

ezért

$$\Delta = 2^{30-52} = 2^{-22} \text{ s} \approx 238{,}4 \text{ ns}.$$

A nagy abszolút időbélyeg tehát már nem képes minden nanoszekundumos eltérést megkülönböztetni. Ha előbb két időbélyeget `double`-re alakítunk, majd kivonjuk őket, az elveszett részletek nem nyerhetők vissza.

Gyakorlati megoldás: az időpontokat megfelelő egész reprezentációban tartjuk, tartományellenőrzött kivonással képezzük a különbséget, és csak az időtartamot alakítjuk másodpercben megadott `double`-re. A kisebb nagyságrendhez finomabb abszolút felbontás tartozik.

4.3.2.1 Subnormál értékek és különleges állapotok

A **subnormál számok** a legkisebb normál szám és a nulla közötti tartomány egy részét fedik le, csökkenő relatív pontossággal. Binary64 esetén:

- a legkisebb pozitív normál érték 2^{-1022};
- a legkisebb pozitív subnormál érték 2^{-1074};
- a subnormál értékek lépésköze állandó.

Ez a fokozatos alulcsordulás (*gradual underflow*). Nem minden végrehajtási profil őrzi meg: egyes hardveres üzemmódok a subnormál eredményeket nullára cserélik.

Az IEEE 754 szerinti megvalósítás további értékeket is biztosít:

- **Infinity:** pozitív vagy negatív végtelen; nem azonos a legnagyobb véges számmal.
- **NaN, Not a Number:** például érvénytelen numerikus művelet eredményének jelölése. NaN esetén az `x == x` hamis, az `x != x` igaz; vizsgálatára mégis az `isnan` való.
- **Signed zero:** a pozitív és negatív nulla numerikusan egyenlő, de a `signbit` megkülönbözteti őket.

Az ISO C17 önmagában nem ígéri minden ilyen érték és minden IEEE-kivételszabály támogatását. Különösen nem szabad abból kiindulni, hogy a nullával osztás minden C implementáción egyszerűen végtelent ad.

Mérési pipeline-ban általában külön adatminőségi szabály szükséges: elfogadunk-e végtelent, vagy minden nem véges értéket hibának tekintünk? A NaN nem helyettesíti automatikusan a hiányzó adat explicit jelölését.

4.3.3 Kerekítés, hiba és összehasonlítás

A 0,1 = 1/10 kettes számrendszerben végtelen törtként írható fel. Véges bináris reprezentáció ezért csak közelíti. Ugyanez igaz sok művelet pontos matematikai eredményére.

Legyen x a pontos, $\hat{x}$ a tárolt érték. Az abszolút és relatív hiba:

$$E_{\text{abs}} = |\hat{x} - x|, \qquad E_{\text{rel}} = \frac{|\hat{x} - x|}{|x|}, \quad x \neq 0.$$

Legközelebbi értékre kerekítésnél, normál tartományban, túlcsordulás és alulcsordulás nélkül egy binary64 művelet szokásos hibamodellje:

$$\text{fl}(a \circ b) = (a \circ b)(1 + \delta), \qquad |\delta| \leq 2^{-53}.$$

Ez egyetlen megfelelően kerekített művelet hibakorlátja, nem egy teljes algoritmusé. A `DBL_EPSILON` ebben a profilban 2^{-52}: sem a legkisebb pozitív érték, sem univerzális összehasonlítási küszöb.

Közel azonos mennyiségek kivonásakor a bemenetek hibája az eredményhez képest nagy lehet. Maga a kivonás akár pontos is lehet, miközben a különbség relatív hibája jelentős: a probléma a már pontatlan bemenetekből indul.

Numerikus összehasonlításra gyakori szerződés:

$$|a - b| \leq \max\left(\tau_{\text{abs}}, \tau_{\text{rel}} \max(|a|, |b|)\right).$$

Az abszolút tolerancia a mennyiség mértékegységében értendő; a relatív tolerancia dimenzió nélküli. Nulla közelében az abszolút tag szükséges. A toleranciát a szenzor felbontása, az algoritmus hibája és az alkalmazás követelménye alapján választjuk, nem egy véletlenszerű „kis epsilonból”.

4.3.3.1 Kidolgozott C17-példa

A következő függvény **nemnegatív, véges késleltetésekre** készült. Hibás érték vagy tolerancia esetén hamisat ad. A $0 \leq \tau_{\text{rel}} \leq 1$ feltétel és a nemnegatív bemenetek megakadályozzák a különbség, illetve a relatív küszöb túlcsordulását. Nem ad át tulajdonjogot és nem tart fenn kölcsönzött hozzáférést.

```c
#include <float.h>
#include <math.h>
#include <stdbool.h>
#include <stdio.h>

static bool latency_close(double a, double b,
                          double atol, double rtol)
{
    if (!isfinite(a) || !isfinite(b) ||
        !isfinite(atol) || !isfinite(rtol) ||
        a < 0.0 || b < 0.0 || atol < 0.0 ||
        rtol < 0.0 || rtol > 1.0) {
        return false;
    }
```

```
    return fabs(a - b) <= fmax(atol, rtol * fmax(a, b));
}

int main(void)
{
    double sum = 0.1 + 0.2;
    double reference = 0.3;

    printf("radix=%d, double precision=%d, eval=%d\n",
           FLT_RADIX, DBL_MANT_DIG, FLT_EVAL_METHOD);
    printf("sizes: %zu %zu %zu C_byte\n",
           sizeof(float), sizeof(double), sizeof(long double));
    printf("sum=%.*g, hex=%a\n",
           DBL_DECIMAL_DIG, sum, sum);
    printf("exact=%d, tolerant=%d\n",
           sum == reference,
           latency_close(sum, reference, 1e-12, 1e-9));
    return 0;
}
```

Javasolt, rögzített kiinduló fordítópárok: a `lab-next` profil (profiltábla) mellett, a fenti x86-64 célprofilon. A parancsok külön laborhoz tartoznak; nem jelentenek elvégzett fordítást vagy tesztet.

```
gcc -std=c17 -O2 -Wall -Wextra -Wpedantic \
    -march=x86-64 -fno-fast-math -ffp-contract=off \
    numeric.c -lm -o numeric-gcc

clang -std=c17 -O2 -Wall -Wextra -Wpedantic \
    -march=x86-64 -fno-fast-math -ffp-contract=off \
    numeric.c -lm -o numeric-clang
```

Binary64 és legközelebbi értékre kerekítés mellett a várt decimális összeg `0.30000000000000004`; a pontos összehasonlítás hamis, a választott toleranciás összehasonlítás igaz. Ezek profilhoz kötött várt eredmények.

4.3.4 Könyvtár, környezet és optimalizálás

A `<math.h>` műveletei között szerepel a `sqrt`, `log`, `fabs`, `nextafter` és `fma`. A megfelelő `f` és `l` végződésű változatok általában `float`, illetve `long double` értékekkel dolgoznak. A `fpclassify`, `isfinite`, `isnan` és `signbit` az értékek osztályozását segítik.

Hibakezelésnél a `math_errhandling` jelzi az `errno`-alapú és/vagy lebegőpontos kivételjelzést. Nem helyes minden függvényhívás után kizárólag az `errno`-t figyelni.

A `<fenv.h>` a lebegőpontos környezetet teszi elérhetővé:

- `fegetround`, `fesetround`: kerekítési mód lekérdezése és módosítása;
- `feclearexcept`, `fetestexcept`: támogatott kivételflagek törlése és vizsgálata.

A lebegőpontos kivételflag nem feltétlenül jelent processzmegszakítást vagy signalt. A támogatott módokat, makrókat és a függvények visszatérési értékét ellenőrizni kell.

ISO C17 mechanizmus: a `#pragma STDC FENV_ACCESS ON` jelzi, hogy a program megfigyeli vagy módosítja a környezetet. **Fordítókorlát:** támogatása és optimalizálással való együttműködése verziófüggő; a `-frounding-math` önmagában nem bizonyít teljes támogatást.

A műveletösszevonás (*contraction*) szintén fontos. A külön szorzás és összeadás két kerekítést végezhet:

$$\mathrm{round}(\mathrm{round}(ab) + c),$$

míg az összevont művelet egyet:

$$\mathrm{round}(ab + c).$$

Az eredmények eltérhetnek. A `#pragma STDC FP_CONTRACT OFF` ennek szabványos szabályozási eszköze; tényleges támogatását ellenőrizni kell. A laborparancs ezért külön `-ffp-contract=off` kapcsolót használ. Az explicit `fma(a,b,c)` továbbra is egyszeri kerekítésű műveletet kér.

A GCC/Clang `-ffast-math` kapcsolója nem pusztán gyorsabb végrehajtást választ. Verzió- és célfüggő engedményeket kapcsolhat be: átrendezheti a műveleteket, feltételezheti a NaN és infinity hiányát, figyelmen kívül hagyhatja az előjeles nullát vagy a dinamikus kerekítést. Egyes profilok a subnormál kezelést is módosítják. Emiatt akár egy `isfinite`-alapú adatellenőrzés szerződése is sérülhet.

A reprodukálhatósági jegyzőkönyv rögzítse a fordító és libc verzióját, cél-ABI-t, kapcsolókat, kerekítési módot, contraction-szabályt, subnormál kezelést és releváns `float.h` értékeket. Ez szükséges, de a matematikai könyvtárak eltérő közelítései miatt nem mindig elegendő a platformok közötti bitazonossághoz.

4.3.5 Hexadecimális literálok és kiírás

A hexadecimális lebegőpontos literálban a `p` utáni kitevő kettes alapú:

$$\texttt{0x1.8p+1} = \left(1 + \frac{8}{16}\right) 2^1 = 3.$$

A `0x1.999999999999ap-4` a binary64-ben szokásos, 0,1-hez legközelebbi értéket írja le pontosan. Ez alkalmas regressziós tesztek bemeneteinek rögzítésére.

A `%a` hexadecimális kiírás kettes alapú profilban alapértelmezett pontossággal pontosan megkülönböztethető alakot ad. Decimális visszaolvasáshoz használjuk például:

```
/* Részlet; C17, <float.h> és <stdio.h> szükséges. */
printf("%.*g\n", DBL_DECIMAL_DIG, value);
```

A `value` itt `double`. `long double` esetén `LDBL_DECIMAL_DIG` és `%.*Lg`, illetve `%La` szükséges. A `printf` variadikus hívásában a `float` automatikusan `double`-re promótálódik.

A reprodukálható szöveghez a locale-t, a visszaolvasási eljárást és a kerekítési környezetet is rögzíteni kell. A fenti teljes program nem módosít locale-t, így a kezdeti `C` locale-t használja.

4.3.6 Gyakori hibák

- **„A `double` pontosan tárolja a decimális adatot.”** Csak bizonyos értékeket; pontos elszámoláshoz gyakran skálázott egész reprezentáció kell.
- **„Minden == hibás.”** Pontosan reprezentálható vagy diszkrét állapotértékeknél helyes lehet. A toleranciás közelség viszont nem tranzitív, ezért nem általános egyenlőségi reláció.
- **„Az epsilon a mérési tolerancia.”** Az epsilon reprezentációs tulajdonság; a tolerancia alkalmazási szerződés.
- **„A több tizedesjegyes kiírás pontosabbá teszi az adatot.”** Csak a már tárolt értéket mutatja részletesebben.
- **„A fast-math eredmény ugyanaz, csak gyorsabb.”** A megengedett numerikus viselkedés is változik.

4.3.7 Feladatok

1. **Magyarázó:** mekkora a binary64 értékek távolsága közvetlenül 2^{32} másodperc fölött? Megőrizhető minden 100 ns-os eltérés?
2. **Kódolási:** egészítsd ki a példát tesztekkel. Ellenőrizd a kétféle nullát, egy negatív késleltetés elutasítását, az abszolút tolerancia szerepét és egy túl nagy relatív tolerancia elutasítását.
3. **Diagnosztikai:** legyen `a = 1e16`, `b = -1e16`, `c = 1.0`. Binary64, legközelebbi kerekítés és külön műveletek mellett miért különbözhet `(a + b) + c` és `a + (b + c)`? Mit következtethetsz egy fast-math fordításból?

4.3.8 Megoldások

4.3.8.1 Időbélyeg felbontása

Itt $e = 32$, ezért:

$$\Delta = 2^{32-52} = 2^{-20}\ \text{s} \approx 953{,}674\ \text{ns}.$$

Nem őrizhető meg minden 100 ns-os eltérés. Ellenőrzésként a `nextafter(t, DBL_MAX) - t` különbség vizsgálható `t = 0x1p32` mellett, a rögzített profilban.

4.3.8.2 A tolerancia tesztelése

Az eredeti C17-programhoz adható részlet:

```c
#include <assert.h>

static void test_latency_close(void)
{
    assert(latency_close(0.0, -0.0, 0.0, 0.0));
    assert(!latency_close(-0.1, 0.1, 1.0, 0.0));
    assert(latency_close(0.0, 5e-10, 1e-9, 0.0));
    assert(!latency_close(0.0, 5e-10, 0.0, 1e-9));
    assert(!latency_close(1.0, 1.0, 0.0, 2.0));
}
```

Hívd meg a függvényt a `main` elején, és ne definiáld az `NDEBUG` makrót. Az ellenőrzési feltétel: minden állítás teljesül. A harmadik és negyedik teszt különíti el az abszolút és relatív küszöb szerepét.

4.3.8.3 Műveleti sorrend

Az `a + b` pontosan nulla, így az első kifejezés eredménye 1. A 10^{16} körüli binary64 lépésköz 2; a `b + c` félúton lévő pontos eredménye pároshoz kerekítéskor visszakerekedik `b`-re. A második kifejezés így 0.

Fast-math mellett az átrendezés megengedett lehet, ezért a zárójelezésből önmagában nem következik ez az eredménypár. Azonos kimenet sem bizonyítja a kapcsoló ártalmatlanságát: csak az adott fordítás megfigyelése.

4.3.9 Összefoglaló

A lebegőpontos pontosság a nagyságrendtől, reprezentációtól és műveleti sorrendtől függ. Megbízható mérési kódhoz tartomány- és toleranciaszerződés, különlegesérték-kezelés, valamint rögzített fordítási profil szükséges. A pontos kiírás láthatóvá teszi a tárolt értéket; az elveszett információt azonban sem több számjegy, sem későbbi típusbővítés nem állítja vissza.

4.4 Deklarációk, typedef és típusminősítők

C-ben egy deklaráció nemcsak nevet ad valaminek: meghatározza, milyen típusú értékeket és műveleteket enged meg a program. A típus pontos olvasása ezért helyességi kérdés. Nem mindegy, hogy egy pointeren keresztül nem szabad módosítani egy objektumot, maga a pointer nem változtatható meg, vagy az objektumot eleve módosíthatatlanként definiáltuk.

Az Egész típusok és adatmodellek alapján a típusnevet már megkülönböztetjük a konkrét mérettől. A Headerek, deklarációk és include-higiénia részben pedig láttuk, hogyan közölnek deklarációkat a fordítási egységek. Most a deklaráció belső szerkezetét és a hozzá kapcsolódó vállalásokat vizsgáljuk.

4.4.1 A deklaráció olvasása

ISO C17: a deklaráció egy vagy több azonosító tulajdonságait közli. A definíció ennél többet tesz: például egy objektum számára tárolót is biztosít, vagy megadja egy függvény törzsét.

Az alábbiak deklarációs részletek:

```
extern int pending;       /* Deklaráció, nem objektumdefiníció. */
int completed = 0;        /* Objektumdefiníció. */
int classify(int value);  /* Függvénydeklaráció. */
```

A deklarációban elkülöníthetjük a közös deklarációmegadókat — például az `int` típusmegadót és a `const` típusminősítőt — az egyes nevekhez tartozó **deklarátoroktól** (*declarator*). A deklarátor fejezi ki többek között a pointer-, tömb- és függvénykapcsolatot.

```
int *first, second;
```

Itt csak `first` pointer; `second` típusa `int`. A `*` nem a teljes deklaráció minden nevéhez kapcsolódik. Olvashatósági okból az eltérő szerkezetű deklarációkat célszerű külön sorba írni.

A típust a névtől kifelé olvassuk. A zárójelezés elsődleges, a `[]` és a függvényt jelölő `()` pedig erősebben kapcsolódik, mint a pointert jelölő `*`.

Deklaráció	A deklarált név típusa
`int *p;`	Pointer `int` objektumra
`int *items[4];`	Négy `int` pointerből álló tömb
`int (*row)[4];`	Pointer négy `int` elemből álló tömbre
`int convert(int);`	Egy `int` paraméterű, `int` eredményű függvény
`int (*operation)(int);`	Pointer ilyen függvényre

Az `int (*row)[4]` levezetése: `row` neve mellett a zárójelben `*` áll, tehát pointer; a zárójeles egységet `[4]` követi, tehát tömbre mutat; az elemtípus `int`. Az `int *items[4]` esetében viszont először a `[4]` kapcsolódik a névhez, ezért tömböt kapunk.

Ez egyelőre típustérkép, nem használati útmutató: a deklaráció önmagában nem biztosít érvényes pointerértéket. A részletes olvasási módszert a Deklarátorok: tömb-, pointer- és függvénykapcsolatok rész folytatja.

4.4.2 A typedef típusálnevet hoz létre

ISO C17: a `typedef` meglévő típushoz rendel másik nevet; nem hoz létre új, attól elkülönülő típust.

```
typedef unsigned long RequestCount;
typedef int *IntPtr;
```

A `RequestCount` és az `unsigned long` ugyanazt a típust jelöli. Az álnév dokumentálhat mértékegységet vagy szerepet, de önmagában nem akadályozza meg például egy kérésszám és egy időtartam összekeverését.

A `typedef` nem szöveges helyettesítés. Ez különösen minősítőkkel fontos:

```
typedef int *IntPtr;
const IntPtr cursor = 0;
```

A levezetés:

1. Az `IntPtr` teljes típusa „pointer `int` objektumra”.
2. A `const` ezt a teljes típust minősíti.
3. `cursor` ezért konstans pointer `int` objektumra, vagyis típusa `int *const`.

Ez nem azonos a `const int *` típussal. A példabeli `cursor` null pointer, nem biztosít objektumhoz hozzáférést; dereferálása UB lenne.

4.4.2.1 Tagnevek és közönséges azonosítók

C-ben a `struct`, `union` és `enum` **tagnevei** (*tag*) külön névtérbe tartoznak a közönséges azonosítóktól. A `typedef`-nevek viszont ugyanabba a névtérbe kerülnek, mint a változó- és függvénynevek, valamint az enumerátornevek.

```
typedef struct Task Task;

struct Task {
    int state;
};
```

Az első sorban a `struct Task` még nem teljes típus: a tagjai nincsenek megadva. A `typedef` ettől függetlenül létrehozhatja hozzá a `Task` álnevet. A második deklaráció teljessé teszi a struktúratípust.

A két `Task` név azért fér meg egymással, mert külön névtérben van. Egy ugyanabban a hatókörben megjelenő `int Task;` azonban már ütközne a `typedef`-névvel.

4.4.3 A const hozzáférési korlátozás és objektumtulajdonság

ISO C17: `const`-minősítésű lvalue-n keresztül nem megengedett az érték módosítása. Ettől külön szabály, hogy egy eredetileg `const`-minősítésű típussal definiált objektum módosításának kísérlete nem definiált viselkedés.

Deklaráció	Mi nem módosítható ezen a hozzáférésen keresztül?
`const int *p;`	A mutatott `int`
`int *const p = 0;`	A pointerobjektum
`const int *const p = 0;`	Sem a pointer, sem rajta keresztül a mutatott érték

Az első eset nem bizonyítja, hogy a mutatott objektum minden más hozzáférés számára is módosíthatatlan. A következő, függvénytörzsbe illeszthető részlet szabályos:

```
int value = 4;
const int *view = &value;
int *writer = &value;

*writer = 9;
/* *view értéke most 9. */
```

A tárolót a helyi `value` objektum adja; `view` és `writer` kölcsönzött hozzáférések. A módosítás nem érvényteleníti őket, de `value` élettartamának vége igen. Az azon túli dereferálás UB.

Ezzel szemben az alábbi **hibás, UB-t szemléltető részlet**, nem futtatandó teszt:

```
const int frozen = 4;
int *writer = (int *)&frozen;
*writer = 9;  /* UB: az eredetileg const objektum módosítása. */
```

A cast csak a kifejezés típusát változtatja meg, az objektum definícióját nem. Ha az eredeti objektum nem `const`, a minősítés eltávolítása után végzett módosítás önmagában nem feltétlenül UB; az API szerződése azonban ezt akkor is tilthatja.

4.4.3.1 A const nem konstans kifejezés

C17-ben ez a blokkhatókörű részlet változó hosszúságú tömböt, VLA-t deklarál:

```
const int count = 4;
int values[count];
```

A `count` nem **egész konstans kifejezés** (*integer constant expression*), hiába kapott konstans inicializálót. A VLA-támogatás C17-ben opcionális. Rögzített tömbmérethez itt megfelelő lehet:

```
enum { ITEM_COUNT = 4 };
int values[ITEM_COUNT];
```

A `const` a szálbiztonságot sem garantálja. Egy `const` pointer mögötti, valójában módosítható objektumot más szál írhat; megfelelő szinkronizáció hiányában ebből adatverseny lehet.

4.4.4 A volatile és a restrict eltérő vállalásai

4.4.4.1 Volatile: megfigyelhető hozzáférés, nem szinkronizáció

ISO C17: a `volatile` olyan objektumokhoz való hozzáférést támogat, amelyek értéke a szokásos programfolyamon kívül változhat, illetve amelyek hozzáférése külső mellékhatással járhat. A volatile-hozzáféréseknek az absztrakt gép szabályait kell követniük; azt, hogy pontosan mi számít ilyen hozzáférésnek, az implementáció határozza meg.

Platformfeltétel: memóriába térképezett eszközregisztereknél a `volatile` csak egy része a szerződésnek. A leképezés, a hozzáférési szélesség és az esetleges hardveres barrier külön platformkövetelmény.

A `volatile` nem garantál:

- atomikus olvasást vagy írást;
- szálak közötti szinkronizációt;
- megfelelő sorrendet a nem volatile adatok publikalásához.

Ezért egy `volatile` jelzőváltozó nem helyettesíti a mutexet vagy az atomikus műveletet. A részletek a volatile, compiler barrier és hardver barrier, illetve a C atomics és memory order részekhez tartoznak.

4.4.4.2 Restrict: vállalás a hozzáférési utakról

ISO C17: a `restrict` pointerekhez kapcsolható típusminősítő. Nem futásidejű ellenőrzés, hanem az optimalizáló által felhasználható aliasing-szerződés.

Első munkamodellként: ha egy objektumot egy restricted pointerből származtatott hozzáférésen keresztül érünk el, és az objektum módosul, akkor az érintett blokk végrehajtása során az objektum többi releváns hozzáférésének is ugyanahhoz a restricted hozzáférési úthoz kell igazodnia. A teljes szabály ennél részletesebb.

Ez nem azt jelenti, hogy más pointerérték nem létezhet ugyanarra az objektumra. A tényleges hozzáférések számítanak. Két, kizárólag olvasásra használt pointer átfedése sem általánosan tiltott.

A következő másolófüggvényhez egyszerű, elegendő szerződést választunk: a forrás- és céltartomány nem fedheti egymást.

4.4.5 Kidolgozott példa: olvasható nézet és külön céltartomány

A példa minimuma ISO C17. A választott laborprofil Linux x86-64 System V, a `lab-next` profil (profiltábla), bővítmények nélkül. Az alábbi ellenőrzések és kimenet levezetett elvárások, nem közölt futtatási eredmények.

```
#include <assert.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdio.h>

enum { ITEM_COUNT = 3 };
enum Status { STATUS_OK = 0, STATUS_BAD = 1 };

static void copy_values(size_t n,
                        int *restrict dst,
                        const int *restrict src)
{
    for (size_t i = 0; i < n; ++i) {
        dst[i] = src[i];
    }
}

int main(void)
{
    int source[ITEM_COUNT] = {3, 5, 8};
    int target[ITEM_COUNT] = {0};
    const int *view = source;

    source[0] = 13;
    copy_values(ITEM_COUNT, target, view);

    bool ready = -4;
    enum Status status = ready ? STATUS_OK : STATUS_BAD;

    assert(view[0] == 13);
    assert(target[0] == 13);
    assert(target[1] == 5);
    assert(target[2] == 8);
    assert(ready == true);

    printf("%d %d %d\n", target[0], (int)ready, (int)status);
    return 0;
}
```

Fordítás `types.c` fájlból, a megfelelő kiadásra beállított driverrel:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -Wcast-qual -O2 types.c -o types-gcc

clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -Wcast-qual -O2 types.c -o types-clang
```

Pozitív `n` esetén mindkét pointer mögött legalább `n` megfelelően inicializált, illetve írható `int` elemnek kell rendelkezésre állnia. A két tartomány legyen diszjunkt. Nulla `n` esetén a függvény nem végez pointerműveletet vagy memóriahozzáférést.

Tulajdonosi szerep és tároló	Kölcsönzött hozzáférés	Élettartamhatár
A `main` helyi `source` tömbje	`view`, majd a hívás `src` paramétere	A `main` blokkjának elhagyása
A `main` helyi `target` tömbje	A hívás `dst` paramétere	A `main` blokkjának elhagyása

A függvény nem tárolja el a pointereket, nem allokál, és nem érvénytelenít hozzáférést. Nem jelez szerződéssértést: az érvénytelen tartomány vagy a restricted hozzáférések tiltott átfedése UB-t okozhat.

A `source[0]` módosítása szabályos, mert a tömb nem `const`; `view` csak olvasási hozzáférést biztosít. A várt kimenet `13 1 0`.

4.4.6 Enum, bool és literálok C17-ben

Az `enum Status` felsorolt típus. **C17-ben az enumerátornevek**, például `STATUS_OK`, `int` típusú konstansok; értéküknek ábrázolhatónak kell lennie `int`-ként. Maga az enumerált típus nem egyszerűen az `int` másik neve: kompatibilis egész típusának kiválasztása implementáció által meghatározott.

A `_Bool` beépített C17 típus. A `<stdbool.h>` a `bool`, `true` és `false` makrókat adja: rendre `_Bool`, `1` és `0` jelentéssel. Skalár érték `_Bool`-ra konvertálásakor a nullával egyenlő értékből `0`, minden másból `1` lesz. Ezért lesz a példában `-4` értékéből igaz.

Néhány fontos literáltípus:

Kifejezés	Típus C17-ben
`12`, `12U`, `12L`	Rendre `int`, `unsigned int`, `long`
`1.0`, `1.0f`, `1.0L`	Rendre `double`, `float`, `long double`
`'A'`	`int`, nem `char`
`true` a megfelelő header után	`int` típusú `1` makrókifejtés

Nagyobb egész literálok típusát a számrendszer, az utótag és az ábrázolhatóság együtt határozza meg; ezt az egész típusok rész tárgyalja. A közönséges sztringliterál C17-ben `char` elemtípusú tömb, módosításának kísérlete mégis UB. A „nem `const` típusú” tehát nem minden esetben jelent módosíthatóságot.

4.4.7 Feladatok

1. **Típusmagyarázat.** Mit jelentenek az alábbi, egymástól független deklarációk? Miben különbözik `a` és `b`?

```c
typedef int *Handle;
const Handle a = 0;
const int *b = 0;
int (*c)[3];
```

2. **Kódolás.** Írj `status_is_ok` függvényt a teljes példához, amely `enum Status` paraméterből `bool` eredményt készít. Miért hibás erre egyszerűen a státusz közvetlen logikai konverziója?

3. **Diagnosztika.** Értékeld külön-külön:

   - a `const` objektum módosítását castolt pointeren át;
   - a `copy_values(2, source + 1, source)` hívást;
   - azt az állítást, hogy egy közös jelző `volatile` minősítése szálbiztossá teszi az adatátadást.

4.4.8 Megoldások

4.4.8.1 Típusmagyarázat

`a` konstans pointer `int` objektumra; `b` módosítható pointer `const int` hozzáféréssel; `c` pointer háromelemű `int` tömbre. Az ellenőrzési feltétel: `a` újraértékadása fordítási diagnosztikát igényel, `b` újraértékadása önmagában megengedett. A null pointerek dereferálása egyik esetben sem megengedett.

4.4.8.2 Kódolás

A teljes példa elé illeszthető függvényrészlet:

```
static bool status_is_ok(enum Status status)
{
    return status == STATUS_OK;
}
```

A siker kódja nulla, amely közvetlen `_Bool`-konverzióval hamis lenne. A függvény a státusz **jelentését**, nem puszta nullaságát képezi le. Ellenőrzésként a `main` törzsében legyen igaz a `status_is_ok(STATUS_OK)` és a `!status_is_ok(STATUS_BAD)` kifejezés.

4.4.8.3 Diagnosztika

Az első eset UB: a cast nem változtatja meg a ténylegesen `const` objektumot.

A második esetben a céltartomány `source[1]` és `source[2]`, a forrás pedig `source[0]` és `source[1]`. A `source[1]` objektumot külön restricted hozzáférési utakon olvassuk és írjuk, ezért a hívás sérti a szerződést. Külön céltömb használata megfelelő javítás.

A harmadik állítás hamis: a `volatile` nem hoz létre szálak közötti szinkronizációt. A helyességhez megfelelő atomikus protokoll vagy zárolás szükséges; pusztán sikeres tesztfutás nem ellenőrzési bizonyíték.

4.4.9 Rövid összefoglaló

A deklarátort a névtől kifelé olvassuk, a `typedef` pedig teljes típushoz rendel álnevet. A `const` módosítási korlátozás, nem általános konstanskifejezés-jelölő vagy szálbiztonsági garancia. A `volatile` a hozzáférések megfigyelhetőségéhez, a `restrict` a hozzáférési utak aliasing-szerződéséhez kapcsolódik. Helyes használatukhoz mindig a tényleges objektumot, annak élettartamát és az összes releváns hozzáférést kell vizsgálni.

4.5 Kifejezések kiértékelése, sequencing és bitműveletek

Egy C-kifejezés helyességét nem dönti el, hogy a megfelelő operátorokat használtuk-e. Tudnunk kell azt is, melyik operandus milyen típusúvá alakul, mikor történnek meg a módosítások, és milyen sorrendet garantál a nyelv. Ezek a kérdések egy állapotjelző bitmező vagy egy számláló kezelésekor ugyanúgy előkerülnek, mint egy összetett függvényhívásban.

A Promóciók, signedness és túlcsordulás részben megismert konverziós szabályokra, valamint a Deklarációk, typedef és típusminősítők fogalmaira építünk. A nyelvi szabályok itt **ISO C17-re** vonatkoznak; a hasonló C++-szintaxisra nem szabad automatikusan átvinni őket.

4.5.1 Értékszámítás, mellékhatás és sequencing

Egy kifejezés kiértékelése kétféle tevékenységet tartalmazhat:

- Az **értékszámítás** (*value computation*) meghatározza az eredményt, illetve adott esetben azt az objektumot, amelyre egy kifejezés hivatkozik.
- A **mellékhatás** (*side effect*) megváltoztatja a végrehajtási környezet állapotát. Ilyen egy objektum módosítása, a `volatile` objektum elérése vagy egy fájl módosítása.

Az `i++` értéke például `i` régi értéke; külön mellékhatása `i` növelése. A régi érték kiszámítása megelőzi ezt a módosítást, de ebből még nem következik sorrend a környező operandusokhoz képest.

A **sequenced-before** az egy szálon belüli kiértékelések aszimmetrikus, tranzitív sorrendi relációja. Ha A sequenced-before B, akkor A kiértékelése megelőzi B kiértékelését. Két további esetet különböztetünk meg:

Viszony	Jelentés
Unsequenced	Sem A nem előzi meg garantáltan B-t, sem fordítva.
Indeterminately sequenced	Az egyik teljes kiértékelés megelőzi a másikat, de a nyelv nem rögzíti, melyik.

Az utóbbi tehát nem ugyanaz, mint a sorrendi kapcsolat hiánya.

ISO C17 – alapvető UB-szabály: ha egy skalár objektum módosítása unsequenced viszonyban áll ugyanannak az objektumnak egy másik módosításával vagy az értékét felhasználó értékszámítással, a viselkedés nem definiált.

Az operátorprecedencia csak a kifejezés csoportosítását határozza meg. Az `a() + b() * c()` alakból nem következik, hogy először `b()` fut le.

4.5.1.1 Függvényargumentumok és teljes kifejezések

Az alábbiak **elkülönített tanítási részletek**, nem futtatandó tesztek. Tegyük fel, hogy `i` és `j` különálló `int` objektumok, és nincs túlcsordulás.

Kifejezés	C17 szerinti minősítés
`i++ + j++`	Definiált; az operandusok között nincs garantált sorrend.

Kifejezés	C17 szerinti minősítés
`i++ + i`	UB: a módosítás és az olvasás unsequenced.
`consume(i++, i++)`	UB: ugyanaz az objektum két unsequenced módosítást kap.
`consume(i++, i)`	UB: unsequenced módosítás és olvasás.
`i = i++ + 1`	UB C17-ben: a két módosítás között nincs szükséges sorrend.

A függvényargumentumok kiértékelése befejeződik a meghívott függvény törzsének megkezdése előtt, de ettől az argumentumok egymás közötti sorrendje még nincs rögzítve.

Biztonságos javítás, C17-részletként, egy megfelelően deklarált `consume` mellett:

```
int i = 3;
int first = i++;
int second = i;
consume(first, second);
```

A külön inicializálók teljes kifejezések (*full expressions*). Az előző teljes kifejezés értékszámításai és mellékhatásai megelőzik a következőét. Így a hívás argumentumértékei `3` és `4`.

Fontos finomság: a `consume(next(), next())` nem szükségképpen UB akkor sem, ha a két `next()` ugyanazt a számlálót módosítja. A két függvénytörzs végrehajtása egymáshoz képest indeterminately sequenced: nem fonódnak össze. Az argumentumokhoz rendelt értékek sorrendje ettől még nem specifikált. Feltétel természetesen, hogy a függvények önmagukban helyesek legyenek.

4.5.2 A rövidzár és a vesszőoperátor garanciái

ISO C17: a következő operátorok valódi sorrendi garanciát adnak.

Operátor	Garancia
`left && right`	`left` előbb értékelődik ki; `right` csak igaz bal operandus esetén.
`left || right`	`left` előbb értékelődik ki; `right` csak hamis bal operandus esetén.
`cond ? yes : no`	A feltétel megelőzi a kiválasztott ágat; csak az egyik ág értékelődik ki.
`left, right`	A bal operandus kiértékelése és mellékhatásai megelőzik a jobbét.

A `&&` és `||` eredménye `int` típusú, értéke `0` vagy `1`.

Például `unsigned int` típusú `num` és `den` mellett:

```
/* C17-részlet; num és den már inicializált unsigned int objektumok. */
int large_ratio = den != 0u && num / den > 3u;
```

A rövidzár kizárja a nullával osztást. A `&` nem helyettesítheti a `&&` operátort: nincs rövidzáras garanciája.

A vessző két szerepét külön kell választani:

```c
/* C17-részlet; consume megfelelően deklarált függvény. */
int i = 3;
int result = (i++, i * 2);  /* Vesszőoperátor: result értéke 8. */
consume(i, result);         /* Az argumentumelválasztó vessző nem operátor. */
```

A sorrendet itt a vesszőoperátor biztosítja, nem önmagában a zárójel.

4.5.3 Bitműveletek: előbb a típust, aztán a biteket vizsgáld

A `&`, `|`, `^` és `~` bitenkénti műveleteket végeznek. Az egész promóciók ezekre is vonatkoznak: például egy `unsigned char` operandus `~` művelete gyakran `int` típusban történik, nem az eredeti keskeny típusban.

A shiftoperátoroknál mindkét operandus egész promóción megy át, de nem a szokásos aritmetikai konverziók határozzák meg a közös típust. Az eredmény típusa a **promótált bal operandus típusa**.

Legyen W ennek bitbeli szélessége, előjeles típusnál az előjelbitet is beleszámítva, padding nélkül. Mindkét shift előfeltétele:

$$0 \le k < W,$$

ahol k a jobb operandus értéke. Negatív vagy túl nagy eltolás UB.

ISO C17:

- Előjel nélküli balra tolásnál az eredmény $x2^k$ modulo 2^W.
- Előjel nélküli jobbra tolásnál az eredmény $\lfloor x/2^k \rfloor$.
- Előjeles balra tolás csak akkor definiált, ha $x \ge 0$, és $x2^k$ reprezentálható az eredmény előjeles típusában.
- Nemnegatív előjeles érték jobbra tolása egész osztásnak felel meg.
- Negatív előjeles érték jobbra tolásának eredménye implementáció által meghatározott.

Egy 32 bites `int`/`unsigned int` profilban ezért a `1 << 31` és a `1u << 32` egyaránt UB, de más okból. Az első eredménye nem reprezentálható `int`-ként, a második eltolásszáma túl nagy.

A művelet **utáni** cast nem javítja meg a hibás shiftet:

```c
/* HIBÁS C17-részlet, 32 bites unsigned int profilban: UB. */
uintmax_t bit = (uintmax_t)(1u << 40);
```

A megfelelően széles típust a shift **előtt** kell biztosítani.

4.5.3.1 Biztonságos maszk eltérő szélességekre

Az alsó n bitet kijelölő maszk matematikai értéke $2^n - 1$. A közvetlen `(1 << n) - 1` azonban a teljes típusszélességnél hibás.

Használjuk inkább, $n > 0$ esetén:

$$t = 2^{n-1}, \qquad M_n = t \mid (t - 1).$$

Bizonyításvázlat: t egyetlen beállított bitet tartalmaz az $n-1$ pozíción; $t-1$ az összes alatta levő bitet tartalmazza. Bitenkénti VAGY kapcsolatuk pontosan n darab egyes.

A következő teljes program minimum C17-et igényel. A `width` logikai szélesség **bitben**, nem `sizeof`-érték. A támogatott tartomány 1–64 bit; a `uintmax_t` legalább 64 értékbitje elegendő. Nincs szükség opcionális `uint64_t` típusra.

```
/* masks.c */
#include <stdbool.h>
#include <stdint.h>
#include <inttypes.h>
#include <stdio.h>

#define MAX_FIELD_BITS 64u

_Static_assert(
    UINTMAX_MAX >= UINTMAX_C(0xffffffffffffffff),
    "at least 64 value bits are required"
);

struct mask_result {
    bool ok;
    uintmax_t value;
};

static struct mask_result low_mask(unsigned width, unsigned count)
{
    if (width == 0u || width > MAX_FIELD_BITS || count > width) {
        return (struct mask_result){false, UINTMAX_C(0)};
    }

    if (count == 0u) {
        return (struct mask_result){true, UINTMAX_C(0)};
    }

    uintmax_t top = UINTMAX_C(1) << (count - 1u);
    return (struct mask_result){true, top | (top - UINTMAX_C(1))};
}

int main(void)
{
    struct mask_result small = low_mask(8u, 4u);
    struct mask_result full = low_mask(64u, 64u);

    if (!small.ok || !full.ok) {
        return 1;
    }

    if (printf("%" PRIxMAX "\n%" PRIxMAX "\n",
               small.value, full.value) < 0) {
        return 2;
    }

    return 0;
}
```

A hibás kérés `ok == false`, `value == 0` eredményt ad; a nulla hosszúságú maszk viszont sikeres kérés. A legnagyobb shift 63, ezért nincs szélességnyi eltolás.

A fordítási parancsokhoz választott laborprofil: Linux, x86-64 System V ABI, LP64, a `lab-next` profil (profiltábla). A program nem támaszkodik e profil konkrét típusszélességeire. A parancsok feltételezik, hogy a megadott kiadások érhetők el az adott neveken; itt nem történt fordítás vagy futtatás.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O2 masks.c -o masks-gcc
clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O2 masks.c -o masks-clang
```

A szabályokból levezetett, nem mért kimenet: `f`, majd `ffffffffffffffff`.

Maszkellenőrzésnél mindig ügyelj a precedenciára: a helyes alak `(flags & mask) == 0`. A `flags & mask == 0` mást jelent, mert az egyenlőségvizsgálat köt erősebben.

4.5.4 `sizeof`, VLA és konstans kifejezések

ISO C17: a `sizeof` eredménye `size_t`, mértékegysége C bájt. Nem VLA típusú operandusa nem értékelődik ki: a `sizeof(i++)` ezért nem növeli `i` értékét, ha `i` például `int`.

A változó hosszúságú tömb, azaz *variable length array* (VLA) kivétel. Az alábbi C17-részlet VLA-támogatást igényel; C17-ben ez opcionális képesség:

```
#include <stddef.h>

void example(void)
{
    size_t n = 3;
    size_t bytes = sizeof(int[n++]);
    /* n értéke 4; bytes értéke 3 * sizeof(int). */
    (void)bytes;
}
```

Itt a méret meghatározásához ki kell értékelni a tömbhatárt. Ezzel szemben egy korábban létrehozott `int a[n++]` objektum `sizeof a` művelete nem értékeli újra a deklarációbeli `n++` kifejezést: a tömb mérete a létrehozáskor rögzült. Egyes összetett, változó méretű típusokat tartalmazó esetekben a méretkifejezés kiértékelése el is maradhat, ha nem befolyásolná a `sizeof` eredményét. **Méretkifejezésbe ezért ne rejts mellékhatást.**

Az **egész konstans kifejezés** (*integer constant expression*, ICE) egész típusú, szigorúan korlátozott összetételű konstans kifejezés. Alkotóelemei lehetnek például egész-, karakter- és felsorolási konstansok, megfelelő `sizeof` és `_Alignof` kifejezések; közvetlen egészre castolt lebegőpontos konstans is megengedett. Kiértékelt függvényhívás vagy objektummódosítás nem lehet benne.

C17-ben egy `const int` objektum neve önmagában nem ICE, még akkor sem, ha az optimalizáló ismeri az értékét.

```
/* C17-részlet, fájlszinten is elhelyezhető. */
#define FIELD_BITS 5u
#define FIELD_SHIFT 9u

_Static_assert(FIELD_SHIFT < 64u, "invalid shift");
_Static_assert(FIELD_BITS > 0u &&
```

```
        FIELD_BITS <= 64u - FIELD_SHIFT,
        "field does not fit");
```

A `_Static_assert` egész konstans kifejezést követel; nulla értéknél fordítási diagnosztikát ír elő. Futásidejű bemenetet nem ellenőriz. A makró pedig szöveges helyettesítés: nem attól lesz konstans kifejezés, hogy nagybetűs a neve.

4.5.5 Feladatok

4.5.5.1 Sorrend és mellékhatás

Mindegyik esetet külön, `int i = 3, j = 8;` kezdőállapotból vizsgáld:

- `(i++, i)`
- `i++ + j++`
- `consume(i++, i)`

Add meg a definiált esetek eredményét és végállapotát, illetve az UB okát.

4.5.5.2 Diagnosztikai feladat

Miért hibás C17-ben az alábbi részlet? A második sor vizsgálatához tegyél fel 32 bites `unsigned int` típust.

```
/* HIBÁS tanítási részlet. */
const int bits = 32;
_Static_assert(bits == 32, "unexpected width");
uintmax_t mask = (uintmax_t)(1u << bits) - 1u;
```

Javítsd külön a fordítási idejű ellenőrzést és a maszkképzést.

4.5.5.3 Ellenőrzött mezőkinyerés

Egészítsd ki a teljes programot egy `extract_field(x, shift, count)` függvénnyel. A 64 bites logikai szó alsó bitjeitől számozva nyerjen ki mezőt. Utasítsa el a `shift >= 64` és `count > 64 - shift` eseteket, és használja a meglévő hibaszerződést.

4.5.6 Megoldások

4.5.6.1 Sorrend és mellékhatás

A `(i++, i)` eredménye `4`, végül `i == 4`: a vesszőoperátor sorrendet biztosít.

Az `i++ + j++` eredménye `11`, végül `i == 4`, `j == 9`. Külön objektumok módosulnak; az operandusok sorrendje nem befolyásolja ezt az eredményt.

A `consume(i++, i)` UB: `i` módosítása és értékének felhasználása unsequenced. Nincs garantált kimenete. Ellenőrzéskor a sorrendi relációt kell igazolni, nem egy megfigyelt futásra támaszkodni.

4.5.6.2 Diagnosztikai feladat

A `bits` objektum neve nem egész konstans kifejezés. Használható helyette például:

```
/* C17-részlet. */
enum { BITS = 32 };
_Static_assert(BITS == 32, "unexpected width");
```

A shift ettől még hibás maradna: 32 bites operandust 32-vel tolna. A szélesítés túl későn történik. A javítás a korábbi programban:

```
struct mask_result mask = low_mask(32u, 32u);
```

Ellenőrzési feltétel: `mask.ok` igaz, értéke matematikailag $2^{32} - 1$.

4.5.6.3 Ellenőrzött mezőkinyerés

Az alábbi C17-részlet a programban, a `low_mask` után helyezhető el:

```
static struct mask_result extract_field(
    uintmax_t x, unsigned shift, unsigned count)
{
    if (shift >= MAX_FIELD_BITS) {
        return (struct mask_result){false, UINTMAX_C(0)};
    }

    struct mask_result mask =
        low_mask(MAX_FIELD_BITS - shift, count);

    if (!mask.ok) {
        return mask;
    }

    return (struct mask_result){true, (x >> shift) & mask.value};
}
```

Előbb ellenőrizzük a shiftet, és csak utána számítjuk ki a fennmaradó szélességet. A jobbra tolás így mindig megengedett; a maszk eltávolítja a mező fölötti biteket.

Ellenőrzési esetek: `extract_field(0xb0, 4, 4)` értéke `0xb`; a `(shift=63, count=1)` kérés megengedett, a `(63, 2)` és `(64, 0)` elutasítandó. A nulla hosszúságú, egyébként érvényes mező eredménye nulla.

4.5.7 Összefoglaló

A precedencia a csoportosítást, a sequencing a kiértékelések sorrendjét szabályozza. A külön teljes kifejezések és a rövidzáras operátorok világos sorrendi garanciákat adnak; a függvényargumentumok elválasztó vesszői nem.

Bitműveletnél mindig a promótált típust, az előjelet és az eltolásszám tartományát vizsgáld. A `sizeof` többnyire nem értékeli ki operandusát, de a VLA kivételt jelent. A `_Static_assert` fordítási idejű szerződést ellenőriz; a futásidejű bemenetekhez külön ellenőrzés szükséges.

5 C objektumok, tárolási idő és memóriamodell

5.1 Scope, linkage, storage duration és lifetime

Egy változó neve eltűnhet a látókörünkből úgy, hogy a hozzá tartozó objektum tovább él. Fordítva is előfordulhat: egy pointerváltozó még elérhető, miközben az objektum, amelyre korábban mutatott, már nem létezik. Az ilyen helyzeteket nem lehet pontosan leírni pusztán azzal, hogy valami „lokális”, „globális”, „stacken van” vagy „heapen van”.

Négy külön kérdést kell feltennünk:

- **Hatókör (*scope*):** a program mely részében használható egy adott deklarációhoz tartozó név?
- **Nyelvi kapcsolás (*linkage*):** különböző deklarációk ugyanazt az entitást nevezik-e meg?
- **Tárolási idő (*storage duration*):** milyen szabály határozza meg az objektum számára fenntartott tároló időtartamát?
- **Élettartam (*lifetime*):** a végrehajtás mely időszakában létezik az adott objektum?

A Szimbólumok, linkage és relocation rész linkeroldali nézőpontját most ISO C17 nyelvi szabályokkal egészítjük ki. A deklarációk olvasásához a Deklarációk, typedef és típusminősítők rész jelöléseit használjuk.

5.1.1 Hatókör: hol jelent valamit a név?

A hatókör a forrásprogramhoz tartozó fogalom, nem futás közbeni időintervallum. Ha egy belső deklaráció elfed egy külsőt, attól a külső objektum nem szűnik meg.

Blokkhatókör (*block scope*) tartozik például a függvénytörzsben vagy egy belső blokkban deklarált közönséges lokális azonosítóhoz. A függvénydefiníció paraméterének neve szintén blokkhatókörű, és a függvénytörzs végéig használható. Egy közönséges objektumazonosító hatóköre a deklarátora befejezése után kezdődik, tehát nem visszamenőleg, a blokk elején.

Fájlhatókör (*file scope*) tartozik a blokkokon és paraméterlistákon kívüli deklarációk azonosítóihoz. Ez a deklaráció helyétől a fordítási egység végéig tart. Nem jelenti azt, hogy a név automatikusan használható egy másik fordítási egységben: ott is megfelelő deklaráció kell.

Függvényhatókör (*function scope*) C-ben kizárólag a címkékhez tartozik. Egy `goto` célcímkéje az egész függvényben megnevezhető, akár a címke szöveges előfordulása előtt is. A függvény neve ettől még nem függvényhatókörű!

Függvényprototípus-hatókör (*function prototype scope*) vonatkozik a nem definícióként szereplő függvénydeklaráció paraméterneveire. Például az `int parse(int count);` deklarációban a `count` neve az adott függvénydeklarátor végéig rendelkezik hatókörrel. Ez a deklaráció nem hoz létre futás közbeni paraméterobjektumot.

Az alábbi ISO C17-részletben két külön objektum szerepel:

```c
/* Függvénytörzs-részlet. */
int limit = 10;
{
    int limit = 3;  /* Elfedi a külső nevet. */
    limit += 1;
}
/* Itt ismét a külső limit neve használható; értéke 10. */
```

A külső objektum a belső blokk végrehajtása alatt is él. Csak a nevével történő közvetlen elérését akadályozza az elfedés.

5.1.2 Kapcsolás: mely deklarációk tartoznak össze?

A kapcsolás az azonosítókhoz kapcsolódik; nem azt mondja meg, hogy egy objektum meddig él.

Külső kapcsolás (*external linkage*) esetén a megfelelő deklarációk különböző fordítási egységekben is ugyanazt az objektumot vagy függvényt jelölhetik. Ehhez a deklarációknak, típusoknak és definícióknak a C szabályai szerint összhangban kell lenniük.

Belső kapcsolás (*internal linkage*) esetén az összetartozó deklarációk ugyanazon fordítási egységen belül jelölnek közös entitást. Tipikus eset a fájlhatókörben `static` kulcsszóval deklarált objektum vagy függvény.

Kapcsolás nélkül (*no linkage*) szerepelnek például a közönséges automatikus lokális változók, a paraméterek és a blokkban deklarált `static` objektumok nevei. Azonos írásmódjuk nem kapcsolja össze őket más blokkok hasonló nevű deklarációival.

Az `extern` nem egyszerűen azt jelenti, hogy „külső kapcsolás”. Ha látható egy korábbi, belső vagy külső kapcsolású deklaráció, az `extern` ezt a kapcsolást örökölheti:

```c
/* ISO C17, fordításiegység-részlet. */
static int state;

void reset_state(void)
{
    extern int state;  /* A korábbi, belső kapcsolású objektum. */
    state = 0;
}
```

Itt nincs második `state` objektum. A blokkhatókörű deklaráció ugyanazt nevezi meg, mint a fájlhatókörű deklaráció. Korábbi látható deklaráció nélkül egy blokkhatókörű `extern int state;` külső kapcsolást adna.

5.1.3 A négy fogalom összehasonlítása

A következő táblázat ISO C17 szerinti, egymástól független deklarációs helyzeteket foglal össze. A tárolási idő és az élettartam **az objektum tulajdonsága**, nem a névé. A `malloc(n)` sorban pozitív méretű, sikeres allokációt feltételezünk.

Deklaráció vagy entitás	A név hatóköre	Kapcsolás	Az objektum tárolási ideje	Az objektum élettartama
Fájlhatókörű `int total;`	Fájl	Külső	Statikus	A program teljes végrehajtása

Deklaráció vagy entitás	A név hatóköre	Kapcsolás	Az objektum tárolási ideje	Az objektum élettartama
Fájlhatókörű `static int hidden;`	Fájl	Belső	Statikus	A program teljes végrehajtása
Blokkban `int local;`	Blokk	Nincs	Automatikus	A blokk adott végrehajtásába való belépéstől annak befejezéséig
Függvénydefiníció `int arg` paramétere	Blokk	Nincs	Automatikus	Az adott függvényhíváshoz kötött
Blokkban `static int count;`	Blokk	Nincs	Statikus	A program teljes végrehajtása
Blokkban `extern int total;`, a fenti `total` látható deklarációjával	Blokk	Külső	A megnevezett objektumé: statikus	A program teljes végrehajtása
Fájlhatókörű `_Thread_local int tls;`	Fájl	Külső	Szálhoz kötött	Szálanként külön objektum, az adott szál teljes végrehajtása
Blokkban `static _Thread_local int cache;`	Blokk	Nincs	Szálhoz kötött	Szálanként külön objektum, az adott szál teljes végrehajtása
Blokkban `int *p = malloc(n);`: maga a `p`	Blokk	Nincs	Automatikus	A blokk adott végrehajtásához kötött
Az előző `p` által megcímzett allokált objektum	Nincs deklarált neve	Név hiányában nem alkalmazható	Allokált	Az allokációtól a deallokációig
`int parse(int count);` prototípusban `count`	Prototípus	Nincs	Nem keletkezik objektum	Nem alkalmazható
Függvényen belüli `done:` címke	Függvény	Nincs	Nem objektum	Nem alkalmazható

A táblázat két fontos ellenpéldát tartalmaz. A blokkhatókörű `static` neve csak helyben használható, de objektuma az egész végrehajtás alatt él. A `malloc` eredményét tároló automatikus pointerváltozó megszűnhet anélkül, hogy az allokált objektum megszűnne.

5.1.3.1 A tárolási idők pontos határai

Automatikus tárolási időnél a nem változó hosszúságú objektum élettartama a hozzá tartozó blokkba való belépéstől a blokk végrehajtásának bármilyen módon történő befejezéséig tart. Másik

függvény meghívása csak felfüggeszti ezt a végrehajtást: nem fejezi be az élettartamot. Rekurzív belépéskor új objektumpéldány keletkezik.

A változó hosszúságú tömb, vagyis VLA kivétel: élettartama a deklarációjától a hatókörének elhagyásáig tart. Részleteit a Többdimenziós tömbök, VLA és layout rész tárgyalja.

Statikus tárolási időnél az objektum a program teljes végrehajtása alatt él. Inicializálása a programindulás előtt megtörténik. C-ben a blokkhatókörű `static` inicializálása nem az első függvényhíváskor végrehajtott, tetszőleges kód.

Szálhoz kötött tárolási időnél minden szálhoz külön objektum tartozik. Blokkhatókörben a `_Thread_local` mellé `static` vagy `extern` szükséges. Másik szálból az objektum közvetett elérésének eredménye ISO C17 szerint implementáció által meghatározott; a szálak közötti pointerátadásra ezért itt nem építünk hordozható szerződést.

Allokált tárolási időnél a létrehozás és megszüntetés nem a blokkhatárt követi. A részletes könyvtári szerződés a malloc, calloc, free és allokációs szerződés témája. A sikeres `realloc` a régi objektum élettartamát lezárja akkor is, ha az új tároló címe ugyanolyannak látszik; ezt a realloc és dinamikusan növekvő buffer rész bontja ki.

Ezek nyelvi kategóriák, nem kötelező stack-, heap- vagy ELF-szekció-hozzárendelések. A gyakori megvalósítást a C objektumok a folyamat virtuális címterében rész külön kezeli.

5.1.4 Mikor használható még a pointer?

Legyen egy objektum élettartama a végrehajtás eseményeivel jelölt

$$L_x = [b_x, e_x),$$

ahol b_x a kezdet, e_x a megszűnés. Egy t időpontban végzett objektumhozzáférés szükséges feltétele:

$$b_x \le t < e_x.$$

Ez nem elégséges feltétel: a megfelelő objektumhatár, típus, igazítás és művelettől függően az inicializált érték is szükséges. Ezeket a következő részek pontosítják.

Bizonyításvázlat. A pointer másolása csak pointerértéket másol; nem módosítja a célobjektum tárolási idejét. Ha a célobjektum automatikus élettartama a függvényből visszatéréskor véget ér, egy kimásolt cím nem hosszabbíthatja meg azt.

Az élettartamon kívüli objektumhozzáférés nem definiált viselkedés, azaz UB. ISO C17-ben az objektumra vagy közvetlenül mögé mutató pointer értéke az objektum élettartamának végén meghatározatlanná (*indeterminate*) válik. A régi címet ezért ne hasonlítgassuk vagy naplózzuk érvényességvizsgálatként.

5.1.4.1 Hibapélda: lokális objektum címének visszaadása

```
/* ISO C17, szándékosan hibás részlet. */
static int *bad_value(void)
{
    int value = 42;
    return &value;
}

/* A *bad_value() kiértékelése UB: a célobjektum már nem él. */
```

A `value` tárolójának életciklusát a függvényhívás kezeli; a visszaadott pointer csak kölcsönzött hozzáférés lenne. Érvénytelenítő eseménye a visszatérés. Nincs meghatározott hibakimenet: sem a `42`, sem összeomlás nem garantált.

A tárolóban esetleg megmaradó bitek nem jelentenek tovább élő objektumot. A `NULL`-ellenőrzés sem igazolja az élettartamot. Skalár eredménynél a természetes javítás az érték szerinti visszaadás.

5.1.5 Blokkon belüli `static`: tartós, de közös állapot

Az alábbi önálló példa minimum ISO C17-et igényel. Választott laborprofil: a `lab-next` profil (profiltábla), Linux x86-64 System V ABI, a `lab-next` profil (profiltábla). A program nem támaszkodik konkrét címméretre vagy objektumelrendezésre.

```
/* lifetime.c */
#include <assert.h>
#include <stdio.h>

static unsigned fresh_value(void)
{
    unsigned value = 0;
    return ++value;
}

static unsigned *shared_value(void)
{
    static unsigned value = 0;
    value = (value + 1u) % 10u;
    return &value;
}

int main(void)
{
    unsigned first = fresh_value();
    unsigned second = fresh_value();

    unsigned *borrowed = shared_value();
    unsigned snapshot = *borrowed;
    unsigned *again = shared_value();

    assert(first == 1u && second == 1u);
    assert(borrowed == again);
```

```
    assert(snapshot == 1u && *borrowed == 2u);

    printf("%u %u %u %u\n",
           first, second, snapshot, *borrowed);
    return 0;
}
```

Fordítás a külön tanulói laborban:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 -g lifetime.c -o lifetime
```

A szabályokból levezetett várt kimenet `1 1 1 2`; ez nem mért futtatási eredmény.

A `fresh_value` minden hívása új automatikus objektumot kap. A `shared_value` összes hívása ugyanazt a statikus objektumot módosítja. A `snapshot` önálló értékmásolat, míg a két pointer ugyanahhoz az objektumhoz ad hozzáférést.

A statikus tárolót a C futtatási környezet kezeli; logikai tulajdonosa a függvényt biztosító modul. A pointerek nem tulajdonosok, ezért nem szabad rajtuk `free`-t hívni. Célobjektumuk a program végéig él. Újabb hívás nem érvényteleníti a pointert, de módosítja a megfigyelt értéket. A példa szerződése soros használat; nincs allokációs hibája, az assertionök szerződéssértést jelezhetnek.

A **reentrancia** (***reentrancy***) azt jelenti, hogy egy művelet korábbi meghívásának befejezése előtt újra meghívható a szerződése megsértése nélkül. Egy statikus munkapuffer vagy részszámítási állapot ezt könnyen megakadályozza: a belső hívás felülírhatja a külső hívás adatait. A hosszú élettartam tehát nem jelent hívásonként külön állapotot.

A módosítható statikus objektum párhuzamos elérése szinkronizáció nélkül adatversenyt is okozhat. A `static` önmagában nem szinkronizáció, a `_Thread_local` pedig nem oldja meg az ugyanazon szálon történő rekurzív vagy callbackes újrabelépést.

5.1.6 Gyakori hibák

A „lokális tehát rövid életű” következtetést a blokkhatókörű `static` cáfolja. A „`static` tehát belső kapcsolású” következtetés csak megfelelő deklarációs környezetben igaz: blokkban a névnek nincs kapcsolása.

Az automatikus pointerváltozó megszűnése nem szabadítja fel az általa megcímzett allokációt. Fordítva, a célobjektum megszűnése nem szünteti meg az összes rá mutató pointerváltozót. Ezek megmaradhatnak érvénytelen értékkel.

Végül a lokális cím visszaadásának `static`-kal történő „javítása” szerződésváltozás: a hívásonként külön eredményből közös, később felülírható állapot lesz.

5.1.7 Feladatok

5.1.7.1 Fogalmi besorolás

Fájlhatókörben szerepel a `static int state;`, egy függvény blokkjában pedig az `extern int state;`. Hány objektum van? Milyen a második név hatóköre, kapcsolása, illetve a megnevezett objektum tárolási ideje?

5.1.7.2 Érték szerinti javítás

Írd át a `bad_value` függvényt úgy, hogy statikus objektum és allokáció nélkül adjon vissza `42`-t. Indokold, miért nem baj a lokális objektum megszűnése!

5.1.7.3 Élettartam-diagnózis

Vizsgáld meg ezt az ISO C17 szerinti, szándékosan hibás függvénytörzs-részletet:

```
int *saved = 0;
{
    int object = 9;
    saved = &object;
}
int result = *saved;
```

Ki kezeli az `object` tárolóját? Mikor érvénytelenedik a kölcsönzött hozzáférés? Javítana-e a helyzeten, ha csak a `saved` deklarációjához adnánk `static` kulcsszót?

5.1.8 Megoldások

5.1.8.1 Fogalmi besorolás megoldása

Egy objektum van. A második név blokkhatókörű, de a látható korábbi deklarációból belső kapcsolást örököl. Az objektum statikus tárolási idejű, élettartama a program teljes végrehajtása.

Ellenőrzési feltétel: a válasz külön kezelje a második deklaráció szűkebb hatókörét és a változatlan objektumélettartamot.

5.1.8.2 Érték szerinti javítás megoldása

```
/* ISO C17, függvénydefiníció-részlet. */
static int good_value(void)
{
    int value = 42;
    return value;
}
```

A visszatérési kifejezés kiértékelésekor a `value` még él. A hívó az egész értéket kapja meg, nem az objektum címét. Ellenőrzési feltétel: az eredmény további használata ne függjön a függvény lokális tárolójától.

5.1.8.3 Élettartam-diagnózis megoldása

Az `object` automatikus tárolójának életciklusát a belső blokk végrehajtása kezeli. A `saved` csak kölcsönzött hozzáférést tárol. A belső blokk elhagyása lezárja a célobjektum élettartamát; az ezt követő dereferencia UB, garantált eredmény nélkül.

A `saved` statikussá tétele csak a pointerobjektum életét hosszabbítaná meg. A célobjektumét nem. Helyes javítás például a szükséges érték kimásolása még a belső blokkban. Ellenőrzési feltétel: minden célobjektum-hozzáférés annak élettartamán belül történjen.

5.1.9 Összefoglaló

A hatókör a név használhatóságát, a kapcsolás a deklarációk összetartozását írja le. A tárolási idő az objektum tárolásának időbeli kategóriája, az élettartam pedig az objektum létezésének konkrét intervalluma. Egy pointer fennmaradása nem hosszabbítja meg a célobjektum életét; egy hosszú életű objektum pedig még nem biztosít reentráns vagy szálbiztos interfészt.

5.2 C objektumok a folyamat virtuális címterében

A C-objektum tárolási ideje nem a virtuális címterület egyik tartományának neve. Az „automatikus objektum” nyelvi fogalom; a „vermen elhelyezett lokális változó” egy gyakori megvalósítás leírása. Ez a különbség hibakereséskor döntő: attól, hogy egy cím továbbra is olvasható memórialapra mutat, az ott korábban létező objektum élettartama már véget érhetett.

A Scope, linkage, storage duration és lifetime részben bevezetett különbségekre építünk: a hatókör a név használhatóságáról, a nyelvi kapcsolás a deklarációk közötti azonosságról, a tárolási idő és az élettartam pedig az objektum létezéséről szól. Most ezeket kapcsoljuk össze egy Linux-folyamat megfigyelhető memóriatérképével, anélkül, hogy a térképet a C nyelv definíciójának tekintenénk.

5.2.1 A virtuális cím nem fizikai hely

ISO C17 — nyelvi modell. Az objektum olyan adattárolási terület, amelynek tartalma értékeket reprezentálhat. A C nem ír elő Linux-folyamatot, lapozást, vermet vagy egy „heap” nevű memóriatartományt. Egy pointer sem általánosan hordozható fizikai memóriacím.

Linux, x86-64 — megvalósítási modell. A fejezet laborprofiljában az objektumpointerek a folyamat virtuális címteréhez kapcsolódnak. A virtuális cím egy folyamat címfordítási környezetében értelmezett cím; a fizikai cím a hardver fizikai címterének egy helyét azonosítja. Ugyanaz a virtuális cím két különböző folyamatban rendszerint nem ugyanahhoz a fizikai tárolóhoz tartozik.

A címfordítást a hardver és a kernel lapokra bontva kezeli. Egyszerű, azonos méretű lapokat feltételező modellben, v virtuális cím és P lapméret mellett:

$$q = \left\lfloor \frac{v}{P} \right\rfloor, \qquad r = v \bmod P, \qquad v = qP + r.$$

Itt q a virtuális lapszám, r a lapon belüli eltolás. Ha a lap jelen van, és a fordítás f fizikai lapkeretre mutat, akkor a fizikai cím $fP + r$. A folyamat memóriatérképe azonban nem árulja el önmagában f értékét.

Egy létező leképezéshez nem szükségképpen tartozik azonnal saját, rezidens fizikai lap. A demand paging, a megosztott lapok és a másolás íráskor (*copy-on-write*) tovább bontják a „lefoglalt memória” hétköznapi fogalmát.

A leképezés olvasási, írási és végrehajtási jogosultságokat is hordoz. Ezek **nem helyettesítik a C szabályait**: egy írható lapra történő írás is lehet nem definiált viselkedés, például ha az érintett objektum már nem él.

5.2.2 Gyakori memóriaterületek és a nyelvi objektumok

Az alábbi megfeleltetések Linux hosted, x86-64 System V ABI, LP64 profilban gyakoriak, de nem ISO C-elhelyezési garanciák.

Nyelvi objektum vagy tároló	Gyakori megvalósítás	Fontos korlát
Automatikus lokális objektum	Verem, regiszter, esetleg külön tároló nélkül	Az optimalizáló megszüntetheti vagy átalakíthatja
Módosítható statikus objektum, nem nulla kezdőértékkel	Írható adatleképezés, gyakran `.data`	A szekciónév nem nyelvi tulajdonság
Nullakezdőértékű statikus objektum	Gyakran `.bss`	A fájlban rendszerint nem szerepel minden kezdeti nulla bájt
Statikus, nem módosítható adat	Gyakran `.rodata` vagy más csak olvasható tartomány	A `const` önmagában nem lapvédelmi utasítás
Szálankénti tárolási idejű objektum	Thread-local storage, TLS	Nem azonos az adott szál automatikus objektumaival
Allokált tároló	`[heap]` vagy más, például anonim leképezés	A `malloc` nem köteles a `[heap]` tartományból dolgozni

Az ELF-betöltés, PLT/GOT és dinamikus szimbólumok részhez kapcsolódó fontos különbség: az ELF **szekciói** és a folyamat **virtuálismemória-leképezései** nem egy-egy arányban felelnek meg egymásnak. A betöltés szegmensekhez kapcsolódik; több szekció kerülhet azonos leképezésbe. A `/proc` térképében ezért nem feltétlenül találunk `.rodata` vagy `.bss` feliratú sort.

A „mapping” sem kizárólag a heapen kívüli területeket jelenti. A programkód, a verem, a megosztott könyvtárak és az allocator által használt tartományok egyaránt leképezésekből állnak.

5.2.2.1 Miért nem következik az automatikus tárolási időből veremelhelyezés?

Indoklásvázlat. Az ISO C17 az objektum nyelvi tulajdonságait és a program megfigyelhető viselkedését írja elő, nem minden objektum gépi tárolóhelyét. Ha egy lokális objektum értéke mindig egy konstans, a fordító helyettesítheti a felhasználásokat ezzel a konstanssal. Ha az érték csak számításokhoz kell, regiszterben tarthatja. Ha szükségtelen, elhagyhatja.

A cím megfigyelhetővé tétele korlátozza ezeket az átalakításokat, de abból sem következik általános nyelvi tételként, hogy az objektum „a stacken van”. A fordítónak a megfigyelhető viselkedést kell megőriznie.

5.2.3 Címmérési labor a saját folyamatban

Kijelölt, itt nem futtatott laborprofil: ISO C17, a `lab-next` profil (profiltábla), GDB 15.2, Linux x86-64, System V ABI, LP64, `CHAR_BIT == 8`. A tényleges reprodukcióban a kernel és az eszközök pontos verzióját is rögzíteni kell. Az alábbiak laborutasítások és értelmezési szempontok, nem dokumentált mérési eredmények.

A program többféle objektum címét és saját memóriatérképét írja ki. A `4096` itt az allokáció kért mérete C bájtban; **nem a lapméretre vonatkozó állítás**.

```
/* address_map.c - ISO C17; a /proc elerese Linux-specifikus. */
#include <stdio.h>
#include <stdlib.h>

static int initialized = 7;
static int zeroed;
static const char label[] = "address probe";

static void frames(unsigned depth)
{
    unsigned marker = depth;

    printf("depth=%u marker=%p\n", depth, (void *)&marker);

    if (depth != 0) {
        frames(depth - 1);
    }

    printf("return marker=%p value=%u\n",
           (void *)&marker, marker);
}

int main(void)
{
    int automatic = 11;
    void *owner = malloc(4096);

    if (owner == NULL) {
        fputs("allocation failed\n", stderr);
        return EXIT_FAILURE;
    }

    FILE *maps = fopen("/proc/self/maps", "r");
    if (maps == NULL) {
        perror("fopen /proc/self/maps");
        free(owner);
        return EXIT_FAILURE;
    }

    printf("initialized=%p\nzeroed=%p\nlabel=%p\n",
           (void *)&initialized, (void *)&zeroed,
           (void *)label);
    printf("automatic=%p\nowner-slot=%p\nallocation=%p\n",
           (void *)&automatic, (void *)&owner, owner);

    frames(2);

    char line[512];
    while (fgets(line, sizeof line, maps) != NULL) {
        fputs(line, stdout);
    }
```

```
    int failed = ferror(maps) != 0;

    if (fclose(maps) != 0) {
        failed = 1;
    }

    free(owner);

    if (fflush(stdout) == EOF || ferror(stdout)) {
        failed = 1;
    }

    return failed ? EXIT_FAILURE : EXIT_SUCCESS;
}
```

Fordítás a külön laborban:

```
gcc -std=c17 -Wall -Wextra -Wpedantic \
    -O0 -g3 -fno-omit-frame-pointer -fPIE -pie \
    address_map.c -o address-map
```

A `%p` formátumhoz `void *` argumentumot adunk. A kiírás alakja implementáció által meghatározott; ebben a Linux-profilban hexadecimális címalak várható.

Különösen fontos az `owner-slot` és az `allocation` különbsége. Az előbbi magának az automatikus pointerobjektumnak a címe, az utóbbi a benne tárolt pointerérték által jelölt allokált terület kezdete. A pointer tárolási ideje nem azonos a célobjektum tárolási idejével.

5.2.3.1 Tulajdonlás és élettartam a példában

A tulajdonos itt programtervezési szerep: a C típusrendszere nem kényszeríti ki.

Objektum vagy erőforrás	Tulajdonos, felelős	Kölcsönzött hozzáférés	Élettartamhatár, hibakimenet
Allokált 4096 C bájt	A `main`, az `owner` pointeren keresztül	A cím kiírása; nincs tartós alias	`free(owner)`; sikertelen kéréskor null pointer
`owner` pointerobjektum, `automatic`	A `main` blokkhoz kötött automatikus objektumok	Címük átadása a `printf` hívásnak	A blokk elhagyása
Rekurzív `marker` objektumok	Az egyes függvényaktivációk	Címük átadása a `printf` hívásnak	Az adott hívás visszatérése
`initialized`, `zeroed`, `label`	Statikus tárolási idejű objektumok	Csak címmegfigyelés	A program teljes végrehajtása alatt élnek
Megnyitott `FILE` adatfolyam	A `main`, a `maps` handle révén	Könyvtári hívások a lezárásig	`fclose`; megnyitási, olvasási vagy lezárási hiba lehetséges

A kiírt cím nem hosszabbít meg élettartamot. A program a felszabadítás után nem használja újra az `owner` régi pointerértékét.

5.2.4 A térkép és a debugger összekapcsolása

Linux — diagnosztikai interfész. A `/proc/self/maps` sorai többek között a következő adatokat tartalmazzák:

- kezdő- és végcím;
- jogosultságok;
- fájlbeli eltolás;
- eszköz- és inode-adat;
- elérési út vagy különleges megnevezés, ha van.

A tartomány fél-nyílt: $[a, b)$. A `b` végcím már nem része a tartománynak. A `rw-p` jelölés olvasható, írható, privát leképezést jelez; a „privát” nem jelenti azt, hogy minden lap már saját fizikai példányban létezik.

Illusztráció, nem mérés, a megadott x86-64 laborprofilhoz:

Adat	Érték
Leképezés kezdete, a	`0x0000555500004000`
Leképezés vége, b	`0x0000555500006000`
Vizsgált virtuális cím, v	`0x0000555500004120`

Először ellenőrizzük:

$$a \leq v < b.$$

Ez teljesül. A cím tartományon belüli eltolása:

$$v - a = \texttt{0x120} = 288 \text{ C bájt.}$$

A tartomány mérete:

$$b - a = \texttt{0x2000} = 8192 \text{ C bájt.}$$

Ezzel csak a **leképezéshez tartozást** állapítottuk meg. Nem bizonyítottuk az objektum méretét, élettartamát vagy a rajta végzett művelet nyelvi helyességét. Az intervallumvizsgálat itt Linux-címadatok számszerű elemzése; nem felhívás különálló C-objektumokra mutató pointerek relációs összehasonlítására.

A debuggerben az `address-map` betöltése után használható a `break frames`, majd a `run`. Megálláskor a `print &marker`, `print &initialized` és `info proc mappings` kapcsolja össze az objektumcímeket a leképezésekkel. Két további `continue` után a `bt` a legbelső rekurzív hívásból mutathatja a hívási láncot.

A `/proc/self/maps` mindig az azt megnyitó folyamatra vonatkozik. Ha egy shellben a `cat /proc/self/maps` parancsot használjuk, a `cat` térképét látjuk, nem egy másik vizsgált programét. Ezért olvassa a példaprogram saját maga a fájlt.

5.2.5 Veremkeretek, rekurzió és allocatorok

A **veremkeret** (*stack frame*) egy függvényaktivációhoz társított, megvalósításfüggő állapot. Tartalmazhat mentett regisztereket, lokális tárolót és a visszatéréshez szükséges adatokat; az ABI és a fordító döntései befolyásolják.

A `frames(2)` végrehajtása közben három függvényaktiváció létezik egyszerre, saját `marker` objektummal. A hívás utáni kiírás azt is megmutatja, hogy a külső aktiváció objektuma a belső hívás alatt tovább él. Az egyszerű farokhívásos átalakítást ez megnehezíti, de általánosan nem tiltja meg az optimalizálást.

Az x86-64 gyakori hívási megvalósításában a verem kisebb címek felé növekszik. Ebből nem következik, hogy tetszőleges lokális változók deklarációs sorrendje kiolvasható a címekből, vagy hogy két rekurziós szint címtávolsága minden buildben állandó. A részletes hívási szerződést a Hívási konvenció, stack frame és visszatérés tárgyalja.

Az allokált tárolónál más határt kell felismerni.

ISO C17 — könyvtári szerződés. A sikeres `malloc` megfelelő tárolót biztosít; a `free` megszünteti annak használati jogosultságát a C-program számára.

Linux és libc — megvalósítás. Az allocator használhatja a program break változtatását, amelyhez Linuxon a `brk` mechanizmus kapcsolódik, illetve kérhet külön `mmap` leképezéseket. A választás allocator-, verzió-, konfiguráció- és futástörténet-függő.

Ezért nincs általános megfeleltetés:

$$\text{egy malloc-hívás} \not\equiv \text{egy rendszerhívás} \not\equiv \text{egy új leképezés.}$$

Egy `free` után a tároló az allocatornál maradhat későbbi felhasználásra. A térkép változatlansága nem őrzi meg a felszabadított objektum élettartamát. A részletes allokációs szerződés a malloc, calloc, free és allokációs szerződés témája.

5.2.6 ASLR, optimalizálás és megfigyelési hibák

Az **ASLR** (*address space layout randomization*) a megfelelően konfigurált Linux-rendszerben több címtartomány elhelyezését változtathatja futásonként. A példát PIE-ként fordítjuk, hogy a végrehajtható program betöltési alapja is részt vehessen ebben. Nem minden cím változik szükségképpen minden indításkor.

A debugger befolyásolhatja a környezetet. GDB-ben a `show disable-randomization` mutatja az erre vonatkozó beállítást; a `set disable-randomization off` a debugger által kezdeményezett kikapcsolást tiltja le. Ezt indítás előtt kell beállítani, és a rendszerkorlátozások továbbra is számítanak.

Összehasonlításhoz készíthető optimalizált változat:

```
gcc -std=c17 -Wall -Wextra -Wpedantic \
    -O2 -g3 -fPIE -pie \
    address_map.c -o address-map-O2
```

Az `-O2` mellett egyes függvényhívások beágyazhatók, a tárolóhelyek megváltozhatnak, és a debugger bizonyos értékekre `optimized out` jelzést adhat. Ez nem bizonyítja, hogy a forrásnyelvi objektumnak nem volt élettartama.

A mérés maga is alakítja a folyamatot: a `stdio` és a fájlmegnyitás további allokációkat okozhat. Ezért ne rendeljünk minden új térképsort automatikusan a saját `malloc` hívásunkhoz.

A legfontosabb hibajavítási szabály: **a leképezés érvényességét és az objektum érvényességét külön vizsgáljuk**. A segmentation fault hiánya nem helyességi bizonyíték.

5.2.7 Feladatok

5.2.7.1 Magyarázó feladat

Hasonlítsd össze az `owner` pointerobjektumot és az általa jelölt allokált tárolót. Mi történik, ha a `main` elhagyja a blokkot anélkül, hogy meghívná a `free` függvényt? Milyen következtetést nem szabad levonni az `owner-slot` címéből?

5.2.7.2 Kódolási feladat

Egészítsd ki a `frames` függvényt egy `static unsigned shared;` objektummal, és minden belépéskor írasd ki a címét. Hasonlítsd össze a rekurzív hívások `shared` és `marker` címeit.

5.2.7.3 Diagnosztikai feladat

Egy mérnök három megállapítást tesz:

- „A `malloc` eredménye nincs a `[heap]` sorban, ezért hibás.”
- „A GDB alatt ismétlődő címek bizonyítják, hogy a rendszer nem használ ASLR-t.”
- „A `free` után a leképezés megmaradt, ezért a régi pointer még használható.”

Magyarázd meg mindhárom következtetés hibáját, és adj ellenőrzési lépést.

5.2.8 Megoldások

5.2.8.1 A magyarázó feladat megoldása

Az `owner` automatikus tárolási idejű pointerobjektum; a célterület allokált tárolási idejű. Az automatikus pointer megszűnése nem hívja meg a `free` függvényt. Ha az utolsó hozzáférési lehetőség is elveszik, memóriafelszabadítási hiba keletkezik. A folyamat végén történő OS-takarítás ettől külön mechanizmus.

Az `owner-slot` csak a pointerobjektum helyét mutatja, nem az allokált területét. Ellenőrzési feltétel: külön azonosítsuk az `&owner` és az `owner` kifejezés jelentését, majd kövessük a felszabadítás minden vezérlési útját.

5.2.8.2 A kódolási feladat megoldása

A korábbi C17-függvény törzsének elejéhez illeszthető részlet:

```
static unsigned shared;
printf("shared=%p\n", (void *)&shared);
```

Minden aktiváció ugyanarra a statikus objektumra hivatkozik, miközben saját automatikus `marker` objektuma van. A megadott laborprofilban a `shared` címének egy futáson belül azonosnak kell mutatkoznia. A `marker` címek között nem követelünk meghatározott távolságot vagy sorrendet.

A címátadás kölcsönzött hozzáférés, nem tulajdonátadás; a részlet nem őriz meg pointert az automatikus objektum élettartamán túl.

5.2.8.3 A diagnosztikai feladat megoldása

A `[heap]` felirat nem foglalja magában az allocator minden lehetséges tárolóját. Először az allokáció címét tartalmazó leképezést kell megkeresni; az allocator döntési mechanizmusa külön vizsgálatot igényel.

A debugger ASLR-beállítását ellenőrizni kell, majd külön, debugger nélkül is megismételni a megfigyelést. Azonos címek önmagukban nem írják le a teljes rendszer házirendjét.

A harmadik állítás nyelvi és OS-szintet kever. A `free` lezárja az allokált objektum élettartamát akkor is, ha a lapok megmaradnak. Az ellenőrzés alapja a felszabadítási út követése, nem egy próbaolvasás: a hibás hozzáférésnek nincs garantált kimenete.

5.2.9 Összefoglaló

A C az objektumok élettartamát és használati szabályait határozza meg; Linux a virtuális leképezéseket és lapjogosultságokat kezeli; a fordító és az allocator köti össze e két nézőpontot. A stack, heap, `.data`, `.bss` és `.rodata` hasznos megvalósítási fogalmak, de nem helyettesítik a nyelvi szerződést. A memóriatérkép helyet mutat, nem élettartamot; a debugger megfigyelést ad, nem általános nyelvi garanciát.

5.3 Inicializálás, indeterminate érték és reprezentáció

Az „inicializáljuk nullára” kifejezés C-ben legalább három különböző műveletet takarhat: egy számnak nulla értéket adunk, egy pointert null pointer értékre állítunk, vagy egy tárolási terület minden bitjét nullázzuk. Ezek gyakori platformokon hasonló gépi műveletekre vezethetnek, de a nyelvi szerződésük nem azonos.

A tárolási időről és élettartamról tanultak alapján az objektum létezése és értékének használhatósága külön kérdés. Egy objektum élettartama már elkezdődhetett úgy, hogy az értékét még nem szabad kiolvasni. Az egész típusok és adatmodellek pedig arra emlékeztetnek, hogy a típus neve, értékkészlete és reprezentációja nem ugyanaz.

Ebben az egységben az **ISO C17** szabályait használjuk. A C bájt a `sizeof` egysége; bitszáma `CHAR_BIT`, nem általánosan nyolc.

5.3.1 Inicializálás a tárolási idő függvényében

Az **inicializálás** (*initialization*) az objektum kezdeti értékének megadása. Nem azonos a későbbi értékadással, még akkor sem, ha mindkettőben szerepelhet az `=` jel.

Az alábbi C17-kódrészletben:

```
void example(void)
{
    int a;       /* Nincs explicit inicializáló. */
    int b = 0;   /* Inicializálás. */
    a = 7;       /* Értékadás a már létező objektumnak. */
}
```

Az `a` deklarációja önmagában nem hiba. Akkor keletkezik probléma, ha a program megfelelő értékadás előtt használja az értékét.

5.3.1.1 Automatikus, statikus és szálhoz kötött objektumok

ISO C17: az explicit inicializáló nélküli automatikus tárolási idejű objektum értéke *indeterminate*, azaz meghatározatlan. Ezzel szemben a statikus és szálhoz kötött tárolási idejű objektumok inicializáló nélkül is a típusuknak megfelelő kezdeti értéket kapják.

Objektum	Inicializáló nélküli kezdeti állapot
Automatikus tárolási idejű objektum	Meghatározatlan érték
Statikus tárolási idejű aritmetikai objektum	Pozitív vagy előjel nélküli nulla
Statikus tárolási idejű pointer	Null pointer érték
Statikus tárolási idejű aggregátum	Tagjai, elemei rekurzívan inicializálódnak
Szálhoz kötött objektum	Ugyanezek a típus szerinti szabályok, szálanként külön objektumra

A statikus tárolási idő nem jelent szükségképpen fájlszintű nevet. A függvényblokkban deklarált `static int count;` is nulla értékkel indul, és nem inicializálódik újra minden függvényhíváskor. A `_Thread_local` objektumból minden szálnak saját példánya van.

C17-részlet:

```
static int process_count;                  /* Kezdetben 0. */
static _Thread_local int thread_count;     /* Szálanként kezdetben 0. */

void work(void)
{
    static int calls;    /* Kezdetben 0, nem hívásonként új objektum. */
    int temporary;       /* Meghatározatlan érték. */

    temporary = 12;      /* Ettől kezdve az értéke használható. */
}
```

A hatókörből tehát nem következik az inicializálási szabály: a `calls` és a `temporary` neve egyaránt blokkhatókörű, tárolási idejük mégis eltér.

5.3.1.2 Részleges aggregátuminicializálás

Tömbnél és struktúránál a meg nem adott elemek, illetve tagok úgy inicializálódnak, mint a statikus tárolási idejű objektumok.

C17-részlet:

```
struct Options {
    unsigned retries;
    double delay;
    void *context;
};

void configure(void)
{
    struct Options options = {0};
    unsigned limits[4] = {3};
}
```

Az `options.retries` értéke nulla, az `options.delay` értéke lebegőpontos pozitív nulla, az `options.context` értéke null pointer. A `limits` elemei rendre `3`, `0`, `0`, `0`.

A `{0}` nem azt mondja, hogy „írj nulla bájtokat az egész objektumba”. Az első tag explicit inicializálót kap, a többi tagra pedig a hiányzó inicializálók nyelvi szabálya érvényesül. Az üres `{}` inicializáló nem szabványos C17-megoldás.

5.3.2 Meghatározatlan érték és trap reprezentáció

ISO C17: az **indeterminate value** két lehetőség valamelyike:

- a típus egy érvényes, de nem specifikált értéke (*unspecified value*);
- **trap reprezentáció** (*trap representation*).

A trap reprezentáció olyan objektumreprezentáció, amely nem képvisel érvényes értéket az adott típusban. A „trap” elnevezés nem garantál processzorkivételt vagy azonnali programleállást. Ha egy ilyen reprezentációjú objektum tárolt értékét nem karaktertípusú lvalue-val olvassuk ki, a viselkedés nem definiált.

A C17 fogalomhasználatában az *indeterminate* elsősorban az érték minősítése. Nem helyes minden inicializálatlan objektumot úgy leírni, hogy „véletlen biteket tartalmaz, amelyek egy véletlen, de használható értéket jelentenek”. Más szabványverziók vagy a C++ eltérő terminológiáját sem szabad automatikusan rávetíteni.

5.3.2.1 Egy egyértelmű UB-példa

Az alábbi **hibás C17-függvényrészlet** kizárólag elemzésre szolgál:

```
int retry_limit(void)
{
    int limit;
    return limit;   /* Nem definiált viselkedés. */
}
```

A `limit` automatikus tárolási idejű, nincs inicializálva, a címét nem vettük, és az értékét olvassuk. Erre az esetre a C17 szabálya nem definiált viselkedést mond ki; nem kell hozzá trap reprezentáció létezését feltételezni.

Nem ígérhető, hogy a függvény nullát, „stacken maradt adatot” vagy minden futáskor más számot ad vissza. A fordító optimalizáláskor abból indulhat ki, hogy a helyes program nem hajt végre ilyen műveletet.

A javítás:

```
/* Helyes C17-függvényrészlet. */
int retry_limit(void)
{
    int limit = 0;
    return limit;
}
```

Fontos különbség, hogy a nem VLA típusú `limit` objektumra alkalmazott `sizeof limit` nem olvassa annak értékét. Az inicializálatlan objektum puszta létezése tehát nem UB.

A karaktertípusú reprezentáció-hozzáférésnek külön szabályai vannak. Ebből azonban nem következik, hogy az inicializálatlan adat általánosan biztonságosan felhasználható. A helyes program szerződése egyszerűbb: minden értékolvasást előzzön meg megfelelő inicializálás vagy értékadás.

5.3.3 Nulla érték és csupa nulla bit

ISO C17: a null pointer olyan pointerérték, amely nem mutat objektumra vagy függvényre. Egy nulla értékű egész konstanskifejezés pointertípusra konvertálva null pointert eredményez. C-ben az ilyen kifejezés `void *`-ra konvertált alakja is null pointer konstans.

C17-részlet:

```
int *a = 0;
int *b = (void *)0;
```

Mindkét pointer null pointer értékű. A fordító feladata az adott implementáció megfelelő reprezentációjának előállítása. Ebből nem következik, hogy ez a reprezentáció csupa nulla bit.

Művelet	Hordozható jelentés C17-ben
`int n = 0;`	Egész nulla érték
`double x = 0;`	Lebegőpontos pozitív nulla
`int *p = 0;`	Null pointer érték
`struct Options o = {0};`	Típus szerinti taginicializálás
`memset(&o, 0, sizeof o)`	Minden C bájt nulla értékűre írása

Az egész típusoknál a csupa nulla bit reprezentáció a nulla értéket képviseli. Ugyanez **nem általános garancia** pointerekre és lebegőpontos típusokra.

5.3.3.1 Mit végez a `memset`?

A `memset` a megadott értéket `unsigned char` típusra konvertálja, majd ennek megfelelő bájtokkal tölti fel a kért tartományt.

A következő C17-részlet helyes bájtpuffer-nullázás; a `<string.h>` fejléc szükséges hozzá:

```
unsigned char buffer[64];
memset(buffer, 0, sizeof buffer);
```

A tároló a `buffer` tömb része, élettartamát a befoglaló blokk szabályozza. A függvény csak a hívás idejére kap kölcsönzött hozzáférést; a megadott tartomány teljes egészében írható. Hibakimenet nincs a szerződés teljesülésekor. Érvénytelen cím vagy túl nagy hossz esetén nem definiált viselkedés következhet.

Ezzel szemben a pointertagot tartalmazó struktúra `memset`-es nullázása önmagában nem hordozható pointerinicializálás. Az így létrehozott pointerreprezentáció akár érvénytelen is lehet; a későbbi tagolvasás ezért nem feltétlenül megengedett.

5.3.3.2 Mit garantál a `calloc`?

A `calloc` siker esetén olyan allokált tárolót ad, amelynek minden bitje nulla. Nem garantálja, hogy az abban később értelmezett pointerek null pointerek, vagy a lebegőpontos objektumok nulla értékűek.

Egy kívánt típus szerinti kezdeti állapotot explicit értékadással adhatunk meg:

```
/* C17-részlet: p már ellenőrzött, megfelelő méretű
   és igazítású allokációra mutat. */
*p = (struct Options){0};
```

Itt a compound literal tagjai nyelvi szabályok szerint inicializálódnak, majd a struktúraértéket a célobjektumba másoljuk. Ha úgyis minden tagot beállítunk, a megelőző bájtnullázás gyakran szükségtelen; az allokáció részletes szerződését a dinamikus memóriáról szóló egység tárgyalja.

5.3.4 Padding és a reprezentáció összehasonlítása

Az **objektumreprezentáció** az objektum tárolását alkotó bájtok összessége. Struktúrában a tagok között vagy az utolsó tag után kitöltés, azaz **padding** lehet. Ennek mérete és elhelyezkedése implementációfüggő; itt nem feltételezünk konkrét ABI-layoutot.

Két objektum értékazonossága nem jelenti feltétlenül bájtazonosságukat. A tagok értékei egyezhetnek, miközben a padding bájtok eltérnek. Egyes típusoknál az érték és a reprezentáció kapcsolata önmagában sem egy-egyértelmű.

A `memcmp` a megadott bájtsorozatokat hasonlítja össze, `unsigned char` értékekként. Ezért általában nem alkalmas struktúrák szemantikai egyenlőségének eldöntésére.

Bizonyításvázlat: legyen két struktúrában minden tag értéke azonos. Ha van olyan padding bájt, amelynek értéke eltérhet a két objektumban, akkor a `memcmp` eltérést találhat, noha a program számára jelentéssel bíró tagok megegyeznek. Az értékegyenlőségből tehát nem következik reprezentációegyenlőség.

ISO C17: az explicit inicializáló nélküli statikus vagy szálhoz kötött aggregátum kezdeti paddingje nulla bitekre inicializálódik. Ez nem általános garancia minden inicializálási módra. Ráadásul

struktúrába vagy annak tagjába történő későbbi tároláskor a padding bájtok nem specifikált értékeket vehetnek fel. A „kezdetben mindent nulláztam” ezért nem tartós reprezentációs invariáns.

Üzemeltetési következményként nyers struktúrabájtokat naplózni, hálózatra küldeni vagy hash-elni nemcsak hordozhatósági gond: paddingen keresztül nem kívánt tárolótartalom is megjelenhet. A protokollmezőket külön kell kódolni.

5.3.5 Kidolgozott példa: értékazonosság és bájtazonosság

Az alábbi `init.c` teljes, hosted C17-program. A választott, helyben ellenőrizendő laborprofil: Linux, x86-64 System V ABI, LP64, a `lab-next` profil (profiltábla). A program nem támaszkodik konkrét struktúraméretre. A példa fordítása és futtatása itt nem történt meg.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct State {
    unsigned char ready;
    unsigned count;
    void *context;
};

static struct State process_state;
static _Thread_local struct State thread_state;

static int state_equal(struct State a, struct State b)
{
    return a.ready == b.ready
        && a.count == b.count
        && a.context == b.context;
}

int main(void)
{
    struct State local = {0};
    struct State *heap = calloc(1, sizeof *heap);

    if (heap == NULL) {
        fputs("allocation failed\n", stderr);
        return EXIT_FAILURE;
    }

    /* A calloc bájtnullázása helyett erre alapozzuk
       a tagok típus szerinti kezdeti értékét. */
    *heap = (struct State){0};

    int values_equal =
        state_equal(local, process_state)
        && state_equal(local, thread_state)
        && state_equal(local, *heap);
```

```
    int bytes_equal =
        memcmp(&local, &process_state, sizeof local) == 0;

    printf("values_equal=%d\n", values_equal);
    printf("bytes_equal=%d\n", bytes_equal);

    free(heap);
    return EXIT_SUCCESS;
}
```

Fordítás a választott kiadásoknak megfelelő parancsokkal:

```
gcc -std=c17 -O2 -Wall -Wextra -Wpedantic init.c -o init-gcc
clang -std=c17 -O2 -Wall -Wextra -Wpedantic init.c -o init-clang
```

Objektum vagy hozzáférés	Élettartam és tulajdonlás
`local`	Automatikus objektum; a `main` blokkjából kilépéskor élettartama véget ér
`process_state`	Statikus tárolási idejű objektum
`thread_state`	A végrehajtó szál saját, szálhoz kötött objektuma
`heap` pointerobjektum	Automatikus objektum; az allokáció felszabadításának felelősségét hordozza
Allokált tároló	A sikeres `calloc` és a `free` között él
`memcmp` argumentumai	Csak a hívás idejére kölcsönzött hozzáférések; nincs tulajdonátadás

Sikeres allokáció esetén a `values_equal` számított értéke `1`: minden összehasonlított tag azonos értéket kapott. A `bytes_equal` értékére nincs ugyanez a hordozható ígéret. Ez nem UB-demonstráció, hanem a kétféle egyenlőség szétválasztása.

A `free` érvényteleníti az allokált objektumra mutató hozzáféréseket. A program utána nem olvassa a `heap` értékét és nem dereferálja azt. Az allokációs hibautat ellenőrzi; a szabványos kimenet írási hibájának részletes kezelését a példa nem tárgyalja.

5.3.6 Gyakori hibák és javításuk

- **„A lokális változó nálam mindig nulla.”** Ez megfigyelés lehet, nem inicializálási garancia. Adj explicit inicializálót.
- **„A null pointer a nulla memóriacím.”** A nyelv null pointer értéket definiál, nem univerzális bitmintát.
- **„A `{0}` és a `memset` felcserélhető.”** Az előbbi típus szerinti értékeket ad, az utóbbi bájtokat ír.
- **„A `memcmp` eltérése eltérő tagértékeket bizonyít.”** Padding miatt ez nem következik. Hasonlítsd össze a jelentéssel bíró tagokat.
- **„Nem kaptam warningot, tehát helyes.”** A diagnosztika hiánya nem bizonyítja az inicializálás helyességét. A vizsgálati módszereket az élettartam- és memóriahibák laborja bővíti.

5.3.7 Feladatok

5.3.7.1 Magyarázó feladat: kezdeti állapotok

Egy függvényben szerepel az `int a;`, a `static int b;` és az `int *p = 0;` deklaráció. Mely értékek olvashatók azonnal? Mit tudunk `p` bitmintájáról?

5.3.7.2 Kódolási feladat: hordozható alapállapot

Írj C17-függvényt, amely egy `struct Options` értéket ad vissza nulla próbálkozásszámmal, nulla késleltetéssel és null pointer környezettel. Ne használj bájtnullázást.

5.3.7.3 Diagnosztikai feladat: hibás teszt

Egy teszt két `struct State` objektumot `{0}` inicializálóval hoz létre, majd megköveteli, hogy a `memcmp` eredménye nulla legyen. Érvényes ez a követelmény? Mi legyen a javított ellenőrzés?

5.3.7.4 Haladó feladat: hibás javítás

Valaki az inicializálatlan `int limit;` olvasását úgy „javítja”, hogy a tesztet csak optimalizálás nélkül fordítja. Miért elégtelen ez? Adj valódi javítást és ellenőrzési feltételt.

5.3.8 Megoldások

5.3.8.1 Kezdeti állapotok

A `b` értéke nulla, a `p` értéke null pointer; mindkettő olvasható. Az `a` megfelelő értékadás nélküli kiolvasása ebben a helyzetben nem megengedett. A `p` reprezentációjáról nem következik, hogy csupa nulla bit.

Ellenőrzési feltétel: a magyarázat külön kezelje a tárolási időt, az értéket és a reprezentációt.

5.3.8.2 Hordozható alapállapot

C17-részlet a korábban megadott `struct Options` definíciójával:

```
struct Options default_options(void)
{
    struct Options result = {0};
    return result;
}
```

A függvény értéket ad vissza, nem a lokális objektum címét.

Ellenőrzési feltétel: a visszaadott tagokra `retries == 0`, `delay == 0.0` és `context == 0` teljesül. A padding tartalma nem része a feltételnek.

5.3.8.3 A hibás teszt javítása

A bájtazonosság követelménye nem hordozható. A padding és a reprezentáció lehetséges eltérései miatt két értékazonos objektum bájtjai különbözhetnek.

A javított ellenőrzés a példabeli `state_equal(a, b)` eredményét követeli igaznak.

Ellenőrzési feltétel: a teszt kizárólag a szerződésben jelentéssel bíró tagokra épüljön.

5.3.8.4 Valódi javítás optimalizálási kapcsoló helyett

Az optimalizálás kikapcsolása nem változtatja meg a C17 szabályait. A javítás megfelelő inicializálás vagy minden olvasáshoz vezető vezérlési úton végrehajtott értékadás.

Ellenőrzési feltétel: minden értékolvasást megelőz egy megfelelő értékadás. Ezt vezérlésifolyam-elemzéssel kell indokolni; néhány sikeres futás önmagában nem bizonyíték.

5.3.9 Összefoglaló

Az objektum élettartama, kezdeti értéke és reprezentációja külön fogalom. Az automatikus objektum nem kap általános implicit nullainicializálást; a statikus és szálhoz kötött objektum igen, mégpedig típus szerinti értékekkel. A `memset` és a `calloc` bájtokat nulláz, nem általánosan szemantikai értékeket. A padding miatt az értékazonosság nem bájtazonosság, az inicializálatlan érték hibás olvasásához pedig nem rendelhető stabil, megjósolható kimenet.

5.4 Effective type, aliasing és objektumreprezentáció

A memória olvashatósága nem pusztán cím- és lapvédelmi kérdés. Egy tartomány lehet a folyamat számára írható, megfelelő méretű és megfelelően igazított, miközben egy adott C típuson keresztüli hozzáférés mégis nem definiált viselkedést okoz. A fordító ugyanis nem csupán bájtokat lát: az objektumok típusára és a hozzáférések kapcsolatára is támaszkodik.

Az Inicializálás, indeterminate érték és reprezentáció rész alapján már különbséget teszünk érték és objektumreprezentáció között. A Deklarációk, typedef és típusminősítők pedig megadta a típusdeklarációk alapját. Most azt vizsgáljuk, milyen típusú kifejezéssel szabad egy objektum tárolt értékéhez hozzáférni.

A nyelvi szabályok alapja ebben a részben **ISO C17**. A konkrét gépi reprezentáció, illetve a fordítói optimalizálás ettől külön kezelendő.

5.4.1 Deklarált típus és effective type

Az **effective type**, azaz a hozzáférés megítéléséhez használt effektív típus nem azonos a pointer változó típusával. Egy cast megváltoztathatja egy pointerkifejezés típusát, de ettől a célobjektum típusa nem változik meg.

5.4.1.1 Deklarációval létrehozott objektumok

Ha az objektumnak van deklarált típusa, az effektív típusa a deklarált típusa.

```
/* ISO C17, szemléltető részlet. */
unsigned int counter = 12u;
unsigned char buffer[64];
```

A `counter` típusa `unsigned int`; a `buffer` karakterelemekből álló tömb. A tömb nem „típus nélküli nyers memória”, még akkor sem, ha kizárólag átmeneti tárolóként használjuk.

Ezért egy `unsigned int` reprezentációjának bemásolása a `buffer` elejére **nem hoz létre ott deklarált `unsigned int` objektumot**. A visszaolvasás helyes módszere egy megfelelő típusú célobjektumba végzett másolás, nem pedig a karaktertömb címének átalakítása és dereferálása. A megfelelő igazítás önmagában ezt nem javítaná ki.

5.4.1.2 Allokált tároló deklarált típus nélkül

A `malloc` által biztosított objektumnak nincs deklarált típusa. A C17 ezért a hozzáférés módjából határozza meg az effektív típust:

- Ha nem karaktertípusú lvalue-on keresztül értéket tárolunk benne, az lvalue típusa lesz az effektív típus ehhez a hozzáféréshez és a későbbi, értéket nem módosító hozzáférésekhez.
- Ha `memcpy` vagy `memmove` másol bele értéket, az effektív típus a forrásobjektum effektív típusa lesz — ha a forrásnak van ilyen típusa — a másoláshoz és az azt követő, értéket nem módosító hozzáférésekhez.
- Az egyéb hozzáférésekre a szabály az adott hozzáféréshez használt lvalue típusából indul ki. Ez azonban nem tesz érvényessé inicializálatlan vagy hibás reprezentációt.

Az effektív típus tehát nem feltétlenül az allokáció teljes élettartamára szóló, megváltoztathatatlan címke. Másfelől a pointer átírása vagy castolása nem típusváltó művelet.

Tároló	Van deklarált típusa?	Mi határozza meg az effektív típust?
`unsigned int x;`	Igen	A deklaráció
`unsigned char bytes[32];`	Igen	A deklaráció; másolástól sem lesz egész objektum
`malloc` eredménye	Nincs	A C17 hozzáférési és másolási szabályai
Deklarált objektumba végzett `memcpy`	Igen	A cél deklarált típusa változatlan

5.4.2 Lvalue-hozzáférés és strict aliasing

Az **lvalue** itt objektumot kijelölő kifejezés: például az `x` vagy a `*p`. Nem csak íráskor használunk lvalue-ot; a `result = *p;` kifejezésben a `*p` olvasás tárgya.

Aliasingról akkor beszélünk, amikor különböző hozzáférési utak ugyanazt az objektumot vagy egymást átfedő tárolót érik el. Ez önmagában nem hiba. Két `int *` teljesen szabályosan mutathat ugyanarra az `int` objektumra.

Az **ISO C17 típuson alapuló hozzáférési szabálya**, amelyet gyakran *strict aliasing* szabálynak neveznek, megengedi a tárolt érték elérését olyan lvalue-val, amelynek típusa:

- az effektív típussal kompatibilis típus;
- ennek megfelelően minősített változata, például `const` változat;
- a megfelelő előjeles vagy előjel nélküli típus, a minősített változatokat is beleértve;
- olyan struktúra- vagy uniontípus, amely a felsorolt típusok valamelyikét tagként tartalmazza, akár beágyazottan;
- valamely karaktertípus.

Az összetett típusokra vonatkozó kivétel például lehetővé teszi, hogy a fordító számoljon egy struktúrára és annak egész típusú tagjára mutató pointer átfedésével. Nem jogosít fel arra, hogy egy önálló egész objektum köré képzeletbeli struktúrát olvassunk: a méret- és objektumhatárok továbbra is érvényesek.

Egy `int` objektum olvasása `const int` lvalue-val rendben van. A megfelelő `unsigned int` típus szintén külön megengedett eset. A `float` viszont nem válik kompatibilissé az `int` típussal attól, hogy egy platformon azonos méretűek.

A hozzáférés helyessége több feltétel együttes teljesülését követeli meg. Ha egy hozzáférés helyes, akkor mind az öt teljesül:

1. a hozzáférés élő objektumot ér el;
2. a hozzáférés az objektum határain belül marad;
3. a cím megfelelően igazított;
4. a hozzáférés lvalue-jának típusa megengedett (lásd a fenti listát);
5. az olvasott érték érvényes reprezentáció.

Ez szükséges feltételek összefoglalása, nem teljes helyességi bizonyítás: például az adatverseny tilalmát külön is teljesíteni kell. Maga a pointerkonverzió is problémás lehet, ha az eredmény nem megfelelően igazított.

5.4.3 Karakteres hozzáférés és reprezentációmásolás

ISO C17: egy objektum reprezentációja karaktertípusú lvalue-on keresztül hozzáférhető. C-ben ide tartozik a `char`, a `signed char` és az `unsigned char`; reprezentáció vizsgálatához általában az utóbbi a legegyértelműbb.

Az objektumreprezentáció mérete `sizeof object` **C bájt**. Egy C bájt `CHAR_BIT` bitből áll; a nyelvi szabály nem feltételez nyolcbites oktetteket.

A kivétel iránya lényeges:

> Egy egész objektum reprezentációját szabad karakterként vizsgálni; egy karaktertömböt ettől még nem szabad tetszőleges egész objektumként olvasni.

A karakteres írás sem veszélytelen. Egy reprezentáció bájtjainak önkényes módosítása létrehozhat olyan bitmintát, amely a későbbi típusos olvasáskor nem érvényes.

5.4.3.1 Teljes példa: pillanatkép és típusos másolat

A következő program két eltérő esetet mutat:

1. egy ismert, érvényes érték reprezentációját karaktertömbön át visszamásolja azonos típusú objektumba;
2. deklarált típus nélküli allokált tárolóba másol egy típusos forrásobjektumból.

A programban szereplő `add_u` függvény `restrict` szerződését a következő rész bontja ki.

Fordítási profil: ISO C17, GNU bővítések nélkül. A helyi laborhoz választott profil Linux/x86-64 System V ABI, a `lab-next` profil (profiltábla). A program nem használ konkrét típusméretet vagy bájtsorrendet. Az alábbi parancsok helyi ellenőrzésre szolgálnak; nem dokumentálnak itt elvégzett fordítást vagy futtatást.

```
/* representation.c */
#include <assert.h>
#include <stddef.h>
#include <stdlib.h>
#include <string.h>

/*
 * Szerzodes:
 * - n > 0 eseten dst es src legalabb n elo elemre mutat;
 * - dst elemei irhatok, src elemei olvashatok;
 * - a ket feldolgozott tartomany nem fed at;
 * - a fuggveny nem tartja meg a kolcsonzott pointereket.
 */
static void add_u(size_t n,
                  unsigned int *restrict dst,
                  const unsigned int *restrict src)
{
    for (size_t i = 0; i < n; ++i) {
        dst[i] += src[i];
    }
}

int main(void)
{
    unsigned int original = 0x1234u;
    unsigned char image[sizeof original];
    unsigned int restored;

    memcpy(image, &original, sizeof original);
    memcpy(&restored, image, sizeof restored);
    assert(restored == original);

    unsigned int *copy = malloc(sizeof *copy);
    if (copy == NULL) {
        return EXIT_FAILURE;
    }

    memcpy(copy, &original, sizeof *copy);
    assert(*copy == original);

    unsigned int destination[3] = {1u, 2u, 3u};
    unsigned int source[3] = {10u, 20u, 30u};

    add_u(3, destination, source);
    assert(destination[0] == 11u);
    assert(destination[1] == 22u);
```

```
    assert(destination[2] == 33u);

    free(copy);
    copy = NULL;
    return EXIT_SUCCESS;
}
```

A parancsokban a fordítónevek a fenti verziókat jelöljék:

```
gcc -std=c17 -O2 -Wall -Wextra -Wpedantic representation.c -o representation-gcc
clang -std=c17 -O2 -Wall -Wextra -Wpedantic representation.c -o
 ↪  representation-clang
```

Objektum vagy hozzáférés	Tulajdonos és élettartam	Invalidálás, hibakimenet
`original`, `image`, `restored`	A `main` automatikus objektumai	A blokk elhagyásakor megszűnnek
`copy` által elért tároló	A felszabadításért a `main` felel; sikeres `malloc`-tól `free`-ig él	`free` érvényteleníti; allokációs hiba esetén `EXIT_FAILURE`
`source`, `destination`	A `main` automatikus tömbjei	A blokk elhagyásakor megszűnnek
`src`, `dst`	Az `add_u` kölcsönzött hozzáférései	Csak a hívás alatt használatosak; szerződéssértésre nincs hibakód, UB következhet

Helyességi bizonyításvázlat. Az első másolás pontosan `sizeof original` C bájtot ment el. Az `image` közben nem módosul. A második másolás ugyanannyi bájtot helyez egy azonos típusú, élő célobjektumba, ezért az eredeti, érvényes érték reprezentációját állítja vissza. A padding esetleges jelenléte ezt nem akadályozza.

A sikeres `malloc` elegendő méretű és az `unsigned int` számára megfelelően igazított tárolót ad. A `memcpy` típusos forrása miatt az allokált cél effektív típusa a következő olvasásra `unsigned int`. A `*copy` még a felszabadítás előtt történik.

5.4.3.2 Amit a `memcpy` nem garantál

A `memcpy` reprezentációt másol, nem numerikus értéket konvertál. Eltérő típusok közötti másolásnál külön kell igazolni, hogy:

- a forrásból a megadott bájtszám olvasható, a célba írható;
- a tartományok nem fednek át;
- a cél későbbi típusos olvasásához a másolt reprezentáció érvényes.

Átfedéshez `memmove` szükséges, de az sem old meg típus- vagy reprezentációs hibát.

Az egyenlő méret önmagában kevés. Egy egész bitmintája nem feltétlenül érvényes lebegőpontos vagy pointerreprezentáció. Külső adatból származó bájtok közvetlen bemásolása ezért nem helyettesíti a protokoll szerinti dekódolást.

Azonos értékű objektumok paddingje eltérhet, ezért a `memcmp` általában nem értékegyenlőség-vizsgálat. A memóriabeli pillanatkép pedig nem automatikusan hordozható fájlformátum.

5.4.4 Union type punning C-ben

A **type punning** ugyanazon reprezentáció más típus szerinti értelmezése. C-ben a union erre külön nyelvi lehetőséget ad.

```c
/* ISO C17, feltetelesen hasznalhato szemlelteto reszlet. */
union numeric_view {
    float f;
    unsigned int u;
};

union numeric_view value = {.f = 1.0f};
unsigned int bits = value.u;
```

ISO C17: ha nem azt a uniontagot olvassuk, amelyen keresztül utoljára tároltunk, az érték reprezentációjának megfelelő részét az olvasott tag típusának reprezentációjaként értelmezzük újra. Ez csapdareprezentációhoz is vezethet. Ha az olvasott tag nagyobb, a korábban írt tagon túli bájtok értéke nem specifikált.

A részlet használatához ezért dokumentált reprezentációs feltételek szükségesek. Nem állítjuk, hogy az `1.0f` valamely univerzális egész bitmintát eredményez. Az `unsigned int` szélessége és a `float` formátuma sincs itt rögzítve.

A unionos forma nem azonos egy önálló `float` objektum `unsigned int *`-en keresztüli olvasásával. A union puszta deklarálása máshol sem tesz szabályossá tetszőleges pointeres type punningot.

C/C++ határ: ezt a C-s lehetőséget nem szabad automatikusan C++-ra átvinni; ott az aktív uniontag és az objektumélettartam szabályait külön kell alkalmazni.

Recept	C17 szerinti minősítés
Objektum olvasása `unsigned char`-ként	Megengedett reprezentációs hozzáférés
Egész objektum olvasása nem kompatibilis lebegőpontos lvalue-val	Nem megengedett
Ismert érvényes reprezentáció visszamásolása azonos típusba	Megfelelő méret és élettartam mellett helyes
Karaktertömb castolása típusos pointerré, majd olvasás	A cast nem létesít megfelelő típusú objektumot
Másik uniontag olvasása	C-s reprezentáció-újraértelmezés, reprezentációs korlátokkal

5.4.5 `restrict`: külön szerződés az aliasingról

A `restrict` nem azt jelenti, hogy két pointer címe soha nem lehet azonos. Hozzáférési szerződés, amelynek jelentése a blokk végrehajtásához és a ténylegesen elért objektumokhoz kötődik.

Az itt vizsgált egyszerű esetben: ha egy objektumot egy `restrict` pointerből származó hozzáféréssel elérünk, és azt a blokk végrehajtása során módosítjuk is, akkor az objektum összes érintett hozzáférésének ugyanahhoz a korlátozott hozzáférési útvonalhoz kell tartoznia. A pointer másolatai és belőle képzett elemcímek lehetnek ilyen származtatott utak. A részletes szabály a mutatott típus `const` minősítését is korlátozza, ha a hozzáfért objektum módosul.

Az `add_u` függvényben a `dst[i]` olvasódik és módosul. Ha ugyanaz az elem a `src` útvonalon is olvasódik, sérül a szerződés. Az API ezért egymást nem átfedő feldolgozott tartományokat követel.

- Két külön tömb megfelel.
- Egy nagyobb tömb két nem átfedő szelete is megfelelhet.
- Az `add_u(n, a, a)` hívás pozitív `n` esetén sérti a szerződést.
- Nulla elemszámnál nincs objektumhozzáférés, ezért az átfedés önmagában nem okoz ilyen sértést.

A csak olvasott, sehol nem módosított objektumok aliasolását a `restrict` önmagában nem tiltja.

Optimalizálási következmény: a fordító a típus- és `restrict` szerződésekre támaszkodva tarthat értékeket regiszterben, átrendezhet műveleteket vagy vektorizálhat ciklusokat. A megsértett szerződés nem „lassabb kódot”, hanem UB-t jelent.

A GCC/Clang `-fno-strict-aliasing` kapcsolója implementációspecifikus optimalizálási beállítás, nem C17-es helyességi bizonyítás. Nem javítja ki a dangling pointert, a hibás igazítást vagy a `restrict` megsértését. A warningok és sanitizerek hiánya sem bizonyítja e szerződések teljesülését.

5.4.6 Feladatok

5.4.6.1 Fogalmi osztályozás

Egy élő, inicializált `int x` objektumot `const int`, `unsigned int`, `float`, illetve `unsigned char` lvalue-on keresztül szeretnénk olvasni. Melyik hozzáférést engedi a típusszabály? A pointerképzés egyéb feltételeit tekintsd teljesítettnek.

5.4.6.2 Biztonságos pillanatkép készítése

Írj C17-részletet, amely egy `double` objektum érvényes értékét karaktertömbbe menti, majd egy másik `double` objektumba visszaállítja. Ne feltételezz konkrét méretet vagy reprezentációt.

5.4.6.3 Optimalizálásfüggő hiba diagnózisa

Egy fejlesztő az `add_u(3, values, values)` hívást használja egy legalább háromelemű, inicializált `unsigned int` tömb elemeinek megduplázására. Optimalizálás nélkül a várt értékeket látja. Miért hibás a szerződés? Adj olyan javítást, amely megőrzi az eredeti `add_u` nem átfedő tartományokra szóló interfészét.

5.4.7 Megoldások

5.4.7.1 A hozzáférések minősítése

A `const int` kompatibilis típus minősített változata, ezért megengedett. Az `unsigned int` a megfelelő előjel nélküli típus, ezért szintén megengedett. A `float` nem tartozik a kivételekhez. Az `unsigned char` reprezentációs hozzáférésként megengedett, de egyetlen ilyen olvasás nem az egész `int` numerikus értékét adja.

Ellenőrzési feltétel: a típusszabály teljesülése mellett az objektum él, inicializált, és a hozzáférés a határain belül marad.

5.4.7.2 A pillanatkép megvalósítása

```c
/* ISO C17-reszlet; a forditasi egyseghez: #include <string.h> */
double source = 1.25;
unsigned char snapshot[sizeof source];
double destination;

memcpy(snapshot, &source, sizeof source);
memcpy(&destination, snapshot, sizeof destination);
```

A három objektum ugyanazon blokk automatikus objektuma; a `memcpy` csak a hívások idejére kap kölcsönzött hozzáférést. Nincs dinamikus allokáció vagy hibakód. A blokk elhagyása lezárja az élettartamukat.

Ellenőrzési feltétel: a pillanatkép nem módosul a két másolás között, minden objektum él, és a másolások tartományai nem fednek át. Ebben a példában `destination == source` teljesül. Ez nem általános lebegőpontos egyenlőségteszt minden lehetséges értékre, például NaN-ra.

5.4.7.3 A `restrict`-hiba javítása

A hívás ugyanazokat az objektumokat módosítja a `dst`, és olvassa a független `src` útvonalon. Ez pozitív elemszámnál szerződéssértés. Az optimalizálás nélküli megfigyelés nem teszi helyessé.

A megduplázás külön, egyetlen hozzáférési utat használó művelet lehet:

```c
/* ISO C17-reszlet: values elo, irhato, legalabb haromelemu tomb. */
for (size_t i = 0; i < 3; ++i) {
    values[i] += values[i];
}
```

A tömb tulajdonosa a hívó; a ciklus nem hosszabbítja meg az élettartamát és nem tart meg pointert. Az `add_u` változatlanul megmarad nem átfedő tartományokhoz.

Ellenőrzési feltétel: legalább három inicializált elem legyen írható. Az előjel nélküli összeadás túlcsordulása definiált moduloaritmetika; ha ez alkalmazási szinten nem elfogadható, külön tartományellenőrzés szükséges.

5.4.8 Rövid összefoglaló

A tároló címe és mérete nem elegendő a típusos hozzáférés igazolásához. A deklarált típus, az allokált objektum effektív típusa és az lvalue típusa együtt határozza meg, mely hozzáférések megengedettek.

A karakteres hozzáférés reprezentációt vizsgál; a `memcpy` reprezentációt másol; egyik sem általános értékkonverzió. A unionos type punning C-specifikus feltételekhez kötött. A `restrict` pedig külön hozzáférési ígéret: megsértése ugyanolyan helyességi hiba, mint egy tiltott típuson keresztüli olvasás.

5.5 Élettartam- és memóriahibák vizsgálati laborja

A memóriahiba vizsgálatának első kérdése nem az, hogy „melyik memórialapon található ez a cím?”, hanem az, hogy **melyik élő objektumhoz, milyen típusú és milyen előfeltételekkel megengedett hozzáférés történik**. Egy olvasható virtuális memórialap nem tesz érvényessé egy lejárt élettartamú objektumra mutató pointert. Ugyanígy a sanitizer csendje sem bizonyítja a program helyességét.

A labor a virtuális címtér és a C objektumok különbségére, az effective type és az objektumreprezentáció szabályaira, valamint a reprodukálható diagnosztikai környezetre épít. Minden esetben három bizonyítékot kapcsolunk össze: a forráskódból levezetett szabályt, a fordítás feltételeit és a tényleges eszközmegfigyelést.

5.5.1 Laborprofil és bizonyítási módszer

A példák minimumszabványa **ISO C17**, explicit `-std=c17` kapcsolóval. A választott konkrét laborprofil Linux hosted környezet, x86-64 System V ABI, LP64 adatmodell és nyolcbites C bájt. A reprodukcióhoz kijelölt eszközverziók a `lab-next` profil (profiltábla), illetve Clang és compiler-rt 19.1.7, a `lab-next` profil (profiltábla) mellett. Ez előírt vizsgálati profil, nem elvégzett teszt: a helyi telepítés pontos verzióit és a kernel kiadását külön rögzíteni kell.

Az alábbi parancsokban a `gcc` és `clang` ezekre a kiadásokra utal. Más verzió használata megengedett, de eltérésként kerüljön a hibajegybe. A kapcsolók és a sanitizer-runtime elérhetősége implementációfüggő.

ISO C17 — alapfogalmak. Az objektum élettartama (*lifetime*) az az időszak, amelyben az objektum létezik, és a rá vonatkozó hozzáféréseknek van élettartam szempontjából érvényes célpontjuk. Az élettartamon kívüli objektumhivatkozás nem definiált viselkedés, azaz UB. Az inicializálatlan automatikus objektum értéke meghatározatlan (*indeterminate*); ez nem egyszerűen „valamilyen véletlen, de használható szám”.

A vizsgálat menete:

1. Azonosítsd az objektumot, a tárolóját és a kölcsönzött hozzáféréseket.
2. Határozd meg az élettartam végét, illetve az inicializálás és a típushelyes hozzáférés feltételeit.
3. Keresd meg az első szabálysértő kiértékelést.
4. Csak ezután értelmezd a warningot, sanitizer-leletet vagy futási tünetet.

5.5.2 Négy elkülönített hibás eset

Az alábbi `faults.c` teljes fordítási egység. **Szándékosan hibás tanítási példa:** minden kiválasztható eset UB-t tartalmaz, ezért egyikhez sem tartozik garantált kimenet. A `CASE` fordítási makró egyetlen esetet választ ki.

```
#include <stdio.h>

#ifndef CASE
#define CASE 1
#endif

#if CASE == 1

static int *expired(void)
```

```
{
    int local = 42;
    return &local;
}

int main(void)
{
    int *p = expired();
    printf("%d\n", *p);
    return 0;
}

#elif CASE == 2

int main(void)
{
    int *p;
    {
        int local = 42;
        p = &local;
    }
    printf("%d\n", *p);
    return 0;
}

#elif CASE == 3

int main(void)
{
    int retries;
    printf("%d\n", retries);
    return 0;
}

#elif CASE == 4

int main(void)
{
    _Static_assert(sizeof(int) == sizeof(float),
                   "This case requires equal sizes.");
    _Static_assert(_Alignof(int) >= _Alignof(float),
                   "This case requires sufficient alignment.");

    _Alignas(int) float value = 1.0f;
    printf("%d\n", *(int *)&value);
    return 0;
}

#else
#error Unsupported CASE
#endif
```

Az első három eset nem támaszkodik konkrét típusszélességre. A negyedik feltételeit a kijelölt target teljesíti; az ellenőrzések és az igazítás azért szerepelnek, hogy a példában ne a méret vagy az igazítás legyen a hozzáférési hiba magyarázata.

5.5.2.1 Lokális objektumra mutató pointer visszaadása

Az első esetben a visszaadott pointer **nem viszi magával az objektumot**. A cím előállításakor a `local` még él; a függvényből való visszatéréskor az élettartama véget ér. A hívó ezért nem kap érvényes hozzáférést egy `int` objektumhoz.

ISO C17: amikor a mutatott objektum élettartama véget ér, a rá mutató pointer értéke indeterminate lesz. A hibát ezért nem helyes úgy modellezni, hogy „a pointer biztosan megmarad, csak a mögötte lévő adat változhat”. A későbbi dereferencia mindenképpen érvénytelen objektumhozzáférést kísérel meg.

Nem az okozza a hibát, hogy egy másik függvény „felülírta a stacket”. Az ilyen felülírás legfeljebb látványossá teszi a már korábban bekövetkezett érvénytelenséget.

5.5.2.2 Use-after-scope

A második esetben a függvény még nem tért vissza, de a belső blokk objektumának élettartama már lejárt. Ez a *use-after-scope*: hozzáférési kísérlet a blokkhoz kötött objektum élettartamának vége után.

Eset	Objektum és tároló	Élettartam kezelése	Kölcsönzött hozzáférés	Érvénytelenítő esemény	Következmény
`CASE=1`	`local`, automatikus tároló	A meghívott blokkhoz kötött; nincs kézi felszabadítás	A visszaadott pointer	Visszatérés az `expired` függvényből	Későbbi hozzáférés: UB
`CASE=2`	`local`, automatikus tároló	A belső blokkhoz kötött	A külső `p`	Kilépés a belső blokkból	Későbbi hozzáférés: UB
`CASE=4`	`value`, automatikus tároló	A `main` blokkjához kötött	A konvertált `int *`	A vizsgált ponton nincs élettartamvége	Inkompatibilis típusú olvasás: UB

A második esetben a `p` pointerobjektum maga továbbra is él. Ebből nem következik a mutatott objektum fennmaradása.

Bizonyításvázlat. Legyen a belső blokkba lépés időpontja t_0, a kilépésé t_1, a későbbi olvasásé t_2. A nem VLA `local` élettartama itt $[t_0, t_1)$, miközben $t_2 > t_1$. Az olvasás tehát kívül esik az élettartamon. A levezetésben sem memóriacím, sem stackelrendezés nem szerepel.

5.5.2.3 Inicializálatlan és inkompatibilis típusú olvasás

A harmadik eset `retries` változója nem kap értéket a használata előtt. Ebben a konkrét esetben egy inicializálatlan, automatikus, `register` tárolási osztállyal is deklarálható objektum értékét olvassuk, amelynek a címét sem képeztük: az olvasás UB. A javításnak a program jelentéséből kell következnie. Ha a kezdeti próbálkozásszám nulla, legyen `int retries = 0;`; ha bemenetből származik, az olvasást csak sikeres bemenetfeldolgozás után szabad elvégezni.

A negyedik esetben a `value` él és inicializált, mégis hibás a hozzáférés.

ISO C17 — típushozzáférés: a deklarált `float` objektum tárolt értékét nem olvashatjuk tetszőlegesen `int` típusú lvalue-n keresztül. A pointerkonverzió nem alakítja át az objektumot. Az egyenlő méret és a megfelelő igazítás szükséges lehet bizonyos műveletekhez, de nem helyettesíti az aliasing szabályait.

5.5.3 Helyes ellenpélda: a hívó tárolója és karakteres reprezentáció

A következő `fixed.c` teljes C17 program. Az eredmény tárolóját a hívó biztosítja; a függvény csak a hívás idejére kölcsönzi.

```
#include <stdio.h>
#include <string.h>

static int produce(int *out)
{
    if (out == NULL) {
        return -1;
    }

    *out = 42;
    return 0;
}

int main(void)
{
    int result = 0;

    if (produce(&result) != 0) {
        return 1;
    }

    float value = 1.0f;
    unsigned char bytes[sizeof value];
    memcpy(bytes, &value, sizeof value);

    printf("result=%d, representation_size=%zu\n",
           result, sizeof bytes);
    return 0;
}
```

```
gcc -std=c17 -O2 -g -Wall -Wextra -Wpedantic fixed.c -o fixed
```

A `produce` szerződése szerint a nem null `out` egy élő, írható, megfelelően igazított `int` objektumra mutat. A null pointerhez `-1` hibakód tartozik; az egyébként érvénytelen pointert a függvény nem képes általánosan felismerni.

A `result` tárolóját és élettartamát a `main` blokkja biztosítja; a kölcsönzött `out` nem kerül eltárolásra. A hozzáférési jogosultságot a `result` élettartamának vége szüntetné meg, de ez a hívás után történik.

A `memcpy` alatt a forrás `value` és a cél `bytes` is élő, nem átfedő helyi objektum. A könyvtári művelet nem tartja meg a pointereket, és az előfeltételek teljesülése esetén nincs külön hibakódja. A másolt mennyiség `sizeof value` **C bájt**. A másolás objektumreprezentációt őriz meg, nem numerikus `float`–`int` konverziót végez.

5.5.4 Padding és strukturált értékek összehasonlítása

A struktúra értéke és teljes tárolási reprezentációja nem azonos fogalom. A tagok között vagy a struktúra végén kitöltés, azaz *padding* lehet; ezek a bájtok nem külön adatmezők.

A következő `compare.c` C17 fordítási egység két összehasonlítást mutat:

```
#include <stdbool.h>
#include <string.h>

struct state {
    unsigned char tag;
    unsigned int count;
};

bool state_equal(struct state a, struct state b)
{
    return a.tag == b.tag && a.count == b.count;
}

/* Hibás, ha a cél a tagértékek egyenlősége. */
bool state_equal_wrong(struct state a, struct state b)
{
    return memcmp(&a, &b, sizeof a) == 0;
}
```

```
gcc -std=c17 -O2 -Wall -Wextra -Wpedantic -c compare.c -o compare.o
```

Előfeltétel, hogy mindkét bemeneti struktúra tagjai érvényes, inicializált értéket tartalmazzanak. A `memcmp` a függvény élő paraméterobjektumait kölcsönzi a hívás idejére; nincs eltárolt pointer.

ISO C17: a padding bájtjainak értékére nem építhető hordozható értékegyenlőségi szerződés. Két struktúra tagjai lehetnek egyenlők eltérő reprezentáció mellett. A `memcmp` ezért reprezentáció-egyenlőséget vizsgál, nem a kívánt strukturált értékegyenlőséget.

Ez nem automatikusan ugyanolyan típusú UB, mint az inicializálatlan `int` közvetlen olvasása. A karakteres reprezentáció-hozzáférést és a típusos értékolvasást külön kell kezelni. Az inicializálatlanságot követő eszköz ugyanakkor jelezheti, ha az összehasonlítás eredménye inicializálatlannak követett paddingtől függ.

A teljes tároló előzetes nullázása nem általános helyettesítője a tagonkénti összehasonlításnak: későbbi struktúratárolások után sem alapozhatunk hordozható szerződést a padding rögzített tartalmára.

5.5.5 Optimalizálás és sanitizer-diagnosztika

5.5.5.1 O0 és O2: kísérlet, nem UB-garancia

Azonos esetet külön binárisokba fordíts:

```
gcc -std=c17 -O0 -g -Wall -Wextra -Wpedantic -DCASE=2 faults.c -o scope-O0
gcc -std=c17 -O2 -g -Wall -Wextra -Wpedantic -DCASE=2 faults.c -o scope-O2
```

A külön laborfuttatásoknál jegyezd fel a kimenetet, a kilépési állapotot és a diagnosztikát. Lehetséges, hogy a két változat azonosan látszik működni; az is, hogy eltérően. Egyik sem változtatja meg a nyelvi hibát.

A `-O0` nem kapcsolja ki az UB-t, nem jelent minden részletében közvetlen forráskód-végrehajtást, és nem garantálja egy hibás olvasás megőrzését. Az optimalizáló különbség csak megfigyelés, nem önmagában fordítóhiba.

5.5.5.2 ASan és UBSan

Az AddressSanitizer, röviden ASan, instrumentált memóriahozzáférésekkel többek között objektumhatár- és élettartamhibákat keres. Az UndefinedBehaviorSanitizer, röviden UBSan, kiválasztott nem definiált műveletekhez ad futásidejű ellenőrzéseket.

```
clang -std=c17 -O1 -g -Wall -Wextra -Wpedantic \
  -fsanitize=address,undefined \
  -fsanitize-address-use-after-scope \
  -fno-omit-frame-pointer \
  -DCASE=2 faults.c -o scope-san
```

A visszatérés utáni hozzáféréshez a kijelölt Clang-profilban külön mód is kérhető:

```
clang -std=c17 -O1 -g -Wall -Wextra -Wpedantic \
  -fsanitize=address \
  -fsanitize-address-use-after-return=always \
  -fno-omit-frame-pointer \
  -DCASE=1 faults.c -o return-san
```

Eszközfüggő korlát: a kapcsoló, az instrumentált útvonal és az optimalizálás is befolyásolja a leletet. A fordító már a visszaadott lokális cím miatt is figyelmeztethet, illetve úgy alakíthatja át a hibás programot, hogy nem a várt sanitizer-kategória jelenik meg.

A UBSan nem „minden UB detektora”. Különösen nem általános effective-type-ellenőrző, és nem teljes inicializálatlanság-detektor.

5.5.5.3 MSan és az instrumentálás határa

A MemorySanitizer, röviden MSan, az inicializáltsági állapotot követi. A harmadik esethez külön build szükséges:

```
clang -std=c17 -O1 -g -Wall -Wextra -Wpedantic \
  -fsanitize=memory -fsanitize-memory-track-origins=2 \
  -fno-omit-frame-pointer -fPIE -pie \
  -DCASE=3 faults.c -o uninit-msan
```

A megbízható követéshez a releváns programrészeknek és függőségeknek MSan-kompatibilis instrumentálásra vagy megfelelő runtime-interceptorra van szükségük. A rendszer-libc egyes hívásait interceptorok kezelhetik; ebből nem következik, hogy tetszőleges előre fordított külső könyvtár biztonságosan bevonható. Az MSan külön konfiguráció, nem egyszerűen az ASan mellé kapcsolható további ellenőrzés.

Eszköz	Hasznos szerep	Amit a csendje nem bizonyít
Fordítói warning	Statikusan felismerhető hibák jelzése	Minden útvonal helyessége
ASan	Instrumentált határ- és élettartamhibák keresése	Inicializáltság és típushelyesség
UBSan	Kiválasztott UB-k ellenőrzése	Az összes C szabály teljesülése
MSan	Inicializálatlan adatok felhasználásának követése	Élettartam-, bounds- és aliasing-helyesség

5.5.6 Hibajegy: tünetből indokolt javítás

A második esethez tartozó hibajegyben különüljön el:

- **Feltétel:** konkrét forrásverzió, `CASE=2`, compiler, runtime, target és kapcsolók.
- **Reprodukció:** teljes fordítási parancs, a vizsgált bináris és a külön futtatás tényleges eredménye.
- **Megfigyelés:** például a sanitizer pontos lelete, vagy az, hogy nem keletkezett lelet. Nem várt eredményt írunk tényként.
- **Nyelvi ok:** a belső blokk végén megszűnik a `local` élettartama; a későbbi hozzáférés érvénytelen.
- **Javítás:** az érték felhasználása még a blokkban, értékmásolás kifelé, vagy megfelelően hosszú életű hívói tároló.
- **Ellenőrzés:** forrásalapú élettartam-indoklás, regressziós teszt és új diagnosztikai futtatás.

Gyakori látszatjavítás a `volatile`, az optimalizálás kikapcsolása vagy a sanitizer elnémítása. Ezek nem hosszabbítják meg az objektum életét. A lokális változó `static`-ká alakítása ugyan módosítja az élettartamot, de közös állapotot és új újrahívhatósági, illetve konkurenciafeltételeket vezet be; ezért nem automatikusan megfelelő API-javítás.

5.5.7 Feladatok

1. **Magyarázat:** a `CASE=2` mindkét optimalizálási szinten `42`-t ír ki. Bizonyítja-e ez a pointer érvényességét? Különítsd el a pointerobjektum és a mutatott objektum élettartamát.

2. **Kódolás:** javítsd a `CASE=2` esetet úgy, hogy az eredményt a belső blokk után használod, de sem dinamikus, sem statikus tárolót nem vezetsz be.
3. **Diagnosztika:** egy inicializálatlan számláló használatánál az ASan nem jelez, az MSan igen. Egy másik tesztben azonos tagértékű struktúrákra a `memcmp` eltérést mutat. Adj mindkét jelenséghez nyelvi magyarázatot, megfelelő javítást és ellenőrzési feltételt.

5.5.8 Megoldások

5.5.8.1 A változatlan kimenet értékelése

Nem bizonyít helyességet. A `p` pointerobjektum a `main` blokkjában tovább él, de a `local` élettartama véget ért. A szabálysértést a blokkhatár és a későbbi hozzáférés bizonyítja; a kiírt szám csak egy konkrét hibás futás megfigyelése.

Ellenőrzési feltétel: az indoklás akkor is érvényes maradjon, ha megváltozik a kimenet vagy a memóriatérkép.

5.5.8.2 Értékmásolás a blokkhatáron át

Az alábbi C17 részlet a `CASE=2` ág teljes `main` függvényét helyettesíti; ugyanazokkal a parancsokkal fordítható.

```
int main(void)
{
    int result = 0;
    {
        int local = 42;
        result = local;
    }

    printf("%d\n", result);
    return 0;
}
```

A másoláskor mindkét objektum él, és a forrás inicializált. A későbbi olvasás már a különálló `result` objektumot használja.

Ellenőrzési feltétel: minden olvasás élő, inicializált objektumra vonatkozzon; ne maradjon kölcsönzött pointer a lejárt objektumhoz.

5.5.8.3 A különböző diagnosztikai eredmények magyarázata

Az ASan nem általános inicializálatlanság-követő. Az MSan-leletet a forrás megfelelő inicializálást nélkülöző végrehajtási útvonalával kell összekapcsolni. A javítás az alkalmazási jelentésnek megfelelő inicializálás vagy az olvasás előfeltételének biztosítása.

A struktúráknál az eltérő padding lehetséges magyarázat, de konkrét futási bizonyíték nélkül nem kizárólagos diagnózis: a tagokat és a méretet is ellenőrizni kell. A javítás tagonkénti érték-összehasonlítás.

Ellenőrzési feltétel: azonos tagértékek mindig egyenlőséget adjanak, bármely vizsgált tag eltérése pedig egyenlőtlenséget, a padding tartalmától függetlenül.

5.5.9 Összefoglaló

A memóriahiba okát a C objektum-, élettartam-, inicializálási és típushozzáférési szabályai adják; a memóriatérkép csak implementációs környezet. Az optimalizálási különbség és a sanitizer-lelet hasznos bizonyíték, de egyik sem helyettesíti a forrásból levezetett indoklást. A jó javítás a hibás szerződést szünteti meg, nem csupán a megfigyelt tünetet.

6 Pointerek és összetett deklarációk C-ben

6.1 Címképzés, dereferencia és null pointer

A pointer nem maga a referált objektum, hanem olyan érték, amellyel egy objektumhoz közvetett hozzáférést fejezhetünk ki. A pointert tároló változó eközben szintén önálló objektum: saját típusa, tárolója, címe és élettartama van. A helyes programhoz ezt a két objektumot következetesen külön kell választani.

A Scope, linkage, storage duration és lifetime rész alapján egy név láthatósága és egy objektum élettartama külön fogalom. Az Inicializálás, indeterminate érték és reprezentáció rész pedig arra figyelmeztetett, hogy a lefoglalt tároló önmagában nem biztosít használható értéket. Pointereknél mindkét különbség döntő: a pointerváltozó még létezhet akkor is, amikor az egykor referált objektum már nem él.

Ebben a részben objektumokra mutató pointerekkel dolgozunk, ISO C17 szerint. A függvénypointereket később tárgyaljuk.

6.1.1 Név, objektum, pointerérték és saját cím

A következő C17-részlet egy függvény blokkjában értendő:

```
int count = 3;
int *p = &count;
```

A deklarációban az `int *p` jelentése: `p` egy `int` típusú objektumra mutató pointer. A kifejezésben szereplő egyoperandusú `&` operátor címképzést végez: `&count` típusa `int *`, értéke a `count` objektumra mutató pointer.

A kapcsolatot címek kitalálása nélkül is lerajzolhatjuk:

Név → saját objektum	Az objektumban tárolt érték	További kapcsolat
`count` → `int` objektum	`3`	Nincs pointerkapcsolat
`p` → `int *` objektum	`&count`	Tárolt pointerérték → `count` objektuma

A három fontos kifejezés:

Kifejezés	Típus	Jelentés
`p`	`int *`	A pointerobjektumból kiolvasott pointerérték
`&p`	`int **`	A pointerobjektum saját címe
`*p`	`int` típusú lvalue	A referált `count` objektumot jelölő kifejezés

Az *lvalue* itt objektumot jelölő kifejezés: például egy értékadás bal oldalán megmondja, melyik objektumot módosítjuk.

```
/* C17-részlet, az előző deklarációk után. */
int snapshot = *p;  /* count értékének olvasása */
*p = 8;             /* count módosítása */
```

A `*` ebben a környezetben az indirekció, közkeletűen dereferencia operátora. Önmagában nem feltétlenül jelent értékkiolvasást: a környező kifejezés dönti el, hogy olvasunk vagy írunk.

Mindkét hozzáférés érvényes, mert `count` élő, inicializált, megfelelően igazított `int` objektum; `p` pontosan erre mutat, és az írás célja módosítható. A példában a környező függvény kezeli mindkét automatikus objektum élettartamát. `p` csak kölcsönzött hozzáférést ad: sem a pointer másolása, sem létezése nem hosszabbítja meg `count` élettartamát.

A címképzés itt közönséges, címezhető objektumra történik. C17-ben például bitmezőnek és `register` tárolásiosztály-jelölővel deklarált objektumnak nem képezhető így címe.

6.1.2 Null pointer constant és `NULL`

ISO C17: null pointer constant az olyan egész konstanskifejezés, amelynek értéke nulla, vagy ilyen kifejezés `void *` típusra alakítva. Például a `0`, a `0L` és a `(void *)0` megfelel ennek.

Ha null pointer constant pointertípusra konvertálódik, az eredmény az adott típus null pointere. Ez nem mutat objektumra vagy függvényre.

```
/* C17-részlet. */
#include <stddef.h>

int *a = 0;
int *b = NULL;
int *c = (void *)0;
```

Mindhárom változó null pointerrel inicializálódik. A `NULL` könyvtári makró, amelyet például a `<stddef.h>` definiál; kifejtése az implementáció által meghatározott null pointer constant. Nem szabad arra építeni, hogy minden implementációban ugyanaz a szöveges alakja vagy típusa.

Fontos különbség a nulla **érték** és a megfelelő **konstanskifejezés** között:

```
/* C17: hibás tanítási részlet, diagnosztikát igénylő
   nyelvi megszorítássértés. */
int zero = 0;
int *bad = zero;
```

A `zero` változó kiolvasása nem egész konstanskifejezés, ezért itt nincs megengedett implicit egész–pointer konverzió. Egy cast hozzáadása sem megfelelő javítás: az egészből pointerré alakítás más szabályok alá tartozik.

A null pointer szemantikai érték, nem előírt bitminta. A nyelv nem követeli meg, hogy reprezentációja csupa nulla bit legyen. Ezzel szemben a szabványos nulla-inicializálás pointer esetén valóban null pointert eredményez; például egy inicializáló nélküli statikus tárolási idejű pointer ilyen értéket kap.

Használható pointerértéknél az alábbi tesztek egyenértékűek:

- `p == NULL`: null-e?

- `p == 0`: null-e?
- `!p`: null-e?
- `if (p)`: nem null-e?

Egyik sem általános érvényességvizsgálat. A nem null pointer is lehet dereferálásra alkalmatlan.

6.1.3 A dereferencia érvényességi szerződése

Egy pointerrel végzett objektumhozzáférést nem a numerikus cím „hihetősége”, hanem több feltétel együttese igazol.

Ellenőrzendő feltétel	Mit kell tudnunk?
Pointerérték	A pointer inicializált, használható értékű, nem null
Élettartam	A referált objektum élettartama még tart
Típus	A hozzáférés típusa megfelel az objektumra vonatkozó típus- és aliasing-szabályoknak
Igazítás (*alignment*)	A cím teljesíti a hozzáférési típus igazítási követelményét
Tartomány	A pointer ténylegesen elérhető objektumot vagy tömbelemet jelöl
Művelet	Olvasáskor az érték használható; íráskor az objektum módosítható

Többszálú programban ehhez a hozzáférések megfelelő szinkronizálása is hozzátartozik; itt egyszálú példákat vizsgálunk.

Az `int *` típus nem bizonyítja automatikusan ezeket a feltételeket. Egy cast nem hoz létre `int` objektumot, nem javítja meg az igazítást, és nem éleszt újjá egy megszűnt objektumot. A pointer típusa a hozzáférés szándékát és nyelvi szabályait írja le, nem futásidejű tanúsítvány.

Tartományhatárra példa a tömb végét közvetlenül követő, *one-past* pointer. Bizonyos műveletekhez szabályos érték, de azon a helyen már nincs dereferálható tömbelem. Ennek részletes szabályait a Pointeraritmetika, objektumhatár és provenance rész tárgyalja.

Bizonyításvázlat az alapmintára. Az `int count = 3` élő, inicializált `int` objektumot hoz létre. A `&count` közvetlenül ennek címét adja, ezért az igazítás és az objektum kijelölése megfelelő. Az érték `int *` változóba kerül, típuseltérés nélkül. Amíg `count` él és `p` ezt az értéket őrzi, a `*p` útján történő olvasás, illetve írás teljesíti a fenti feltételeket.

A lényeg nem egy cím ellenőrzése, hanem a pointer létrejöttének és későbbi használatának követése.

6.1.4 Null, inicializálatlan és dangling pointer

A három állapotot nem szabad összemosni.

Állapot	Hogyan keletkezik?	Mire használható?
Null	Például `int *p = NULL;`	Összehasonlítható, továbbadható megfelelő szerződés szerint; nem dereferálható
Inicializálatlan automatikus pointer	Például blokkbeli `int *p;`	Értéke indeterminate; használat előtt értéket kell adni neki
Dangling	A korábban referált objektum élettartama megszűnik	A korábbi célhoz már nem biztosít érvényes hozzáférést

Az inicializálatlan pointer nem „véletlenszerűen null vagy érvényes”. Indeterminate értékét nem szabad érvényes pointerértékként kiolvasni, ellenőrizni vagy kiírni.

A dangling állapotnál ugyancsak fontos C17-részlet, hogy az objektum élettartamának végén a rá mutató pointer értéke indeterminate lesz:

```c
/* C17-részlet egy függvényben; nincs hibás dereferencia. */
#include <stddef.h>

int *p = NULL;

{
    int local = 9;
    p = &local;
}                   /* local élettartama véget ér */

/* Itt p régi értékét nem olvassuk ki. */
p = NULL;           /* Az új értékadás nem olvassa a régi értéket. */
```

A belső blokk kezeli `local` élettartamát; `p` kölcsönzött hozzáférése a blokk végén érvénytelenné válik. A pointerobjektum tárolója ettől még létezik. A `p = NULL` biztonságos felülírás, de nem módosítja a korábban esetleg készített pointermásolatokat.

Gyakori hibás javítás az `if (p != NULL)` beszúrása egy élettartamhibás hozzáférés elé. Ez nem állítja helyre az objektumot; ebben a példában már a pointerérték kiolvasása sem megalapozott.

6.1.5 Kidolgozott példa: a pointer átállítása és a cél módosítása

A következő teljes program minimum ISO C17-et igényel. Választott, helyben ellenőrizendő laborprofil: Linux x86-64, System V ABI, LP64, 8 bites C bájt, a `lab-next` profil (profiltábla). A program nem használ platformbővítést; az alábbi eredmények nyelvi levezetésből származnak, nem itt rögzített futtatásból.

```c
/* pointer_basics.c */
#include <stddef.h>
#include <stdio.h>

int main(void)
{
    int first = 3;
    int second = 7;
```

```
    int *p = &first;

    printf("first cime: %p\n", (void *)&first);
    printf("p erteke:   %p\n", (void *)p);
    printf("p sajat cime: %p\n", (void *)&p);

    int observed = *p;
    *p = observed + 1;

    p = &second;
    *p = 8;

    printf("first=%d, second=%d\n", first, second);
    printf("pointermeret=%zu, int-meret=%zu\n",
           sizeof p, sizeof *p);

    p = NULL;
    if (p == NULL) {
        puts("p most nem jelol objektumot");
    }

    return 0;
}
```

Fordítás a megadott verziókra beállított fordítóparancsokkal:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O0 -g \
    pointer_basics.c -o pointer_basics-gcc

clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O0 -g \
    pointer_basics.c -o pointer_basics-clang
```

A `main` blokk kezeli az összes helyi objektum tárolóját és élettartamát. `p` nem tulajdonos, csak az aktuálisan kijelölt objektumhoz ad kölcsönzött hozzáférést.

Művelet	Referált objektum	Érvényesség indoka
`observed = *p`	`first`	Élő, inicializált `int`; címét `&first` szolgáltatta
`*p = observed + 1`	`first`	Ugyanez, továbbá módosítható; a kiszámolt érték `4`
`p = &second`	Nincs dereferencia	Csak `p` tárolt pointerértékét cseréljük le
`*p = 8`	`second`	Élő, módosítható `int`; helyes típus, igazítás és objektumhatár

A levezetett értékeredmény `first=4, second=8`. A `p = &second` nem másolja át `first` értékét, és nem semmisíti meg `first` objektumát. A `p = NULL` ugyancsak csak a pointerobjektumot módosítja.

A dereferenciákhoz nem tartozik futásidejű hibakód: helyességüket a program szerkezete biztosítja. Az előfeltételek megsértését nem szabad garantált folyamathibának tekinteni; nem definiált viselkedés, vagyis UB lehet az eredmény.

6.1.6 Pointerméret és címkiírás

A `sizeof p` a pointerobjektum mérete, míg a `sizeof *p` az `int` objektumtípus mérete. A `sizeof` egysége C bájt; ennek bitszáma `CHAR_BIT`.

Méretkifejezés	Mit mér?	A megadott x86-64 System V LP64 profilban
`sizeof p`	Egy `int *` objektumot	8 C bájt
`sizeof *p`	Egy `int` objektumot	4 C bájt
`sizeof(char)`	Egy `char` objektumot	1 C bájt

Ezek profilhoz kötött méretadatok, nem univerzális C-layout. A pointer mérete nem a referált objektum méretének függvénye, és a pointer nem tárol automatikusan objektumhosszt.

A példában a `sizeof *p` operandusa nem változó hosszúságú tömbtípusú, ezért nem értékelődik ki: nincs tényleges dereferencia. Így ez a méretkifejezés akkor is használható lenne, ha `p` null lenne. A helyes méret lekérdezése azonban nem igazolja egy későbbi `*p` hozzáférés helyességét.

ISO C17: a `printf %p` konverziója `void *` argumentumot vár. Ezért írjuk:

```
/* C17-részlet; p itt használható int * értéket tartalmaz. */
printf("%p\n", (void *)p);
```

A változó argumentumszámú részben az `int *` nem konvertálódik automatikusan `void *` típussá. A cast itt szükséges típusillesztés, nem érvényességjavítás.

A `%p` szöveges alakja implementáció által meghatározott; nem kötelező egy adott hexadecimális formátum. A kiírás a pointerértéket figyeli meg, nem olvassa a referált objektumot. Az eredmény nem bizonyít hozzáférési jogot, élettartamot vagy hordozható memórialayoutot, és külön futások között sem tekintendő stabil azonosítónak.

6.1.7 Feladatok

6.1.7.1 Magyarázó feladat: két pointer, két objektum

A következő C17-részlet ugyanazon élő blokkban fut:

```
int x = 5;
int y = 9;
int *p = &x;
int *q = p;

*q = 6;
p = &y;
```

Rajzold le a végső név–objektum–pointer kapcsolatokat. Mennyi `x`, `y` és `*q`? Miért nem változott meg `q` a `p` átállításakor?

6.1.7.2 Kódolási feladat: opcionális kimeneti pointer

Írj `int store_if_present(int *out, int value)` függvényt. Null `out` esetén ne írjon, és adjon vissza `0`-t; egyébként tárolja `value` értékét, és adjon vissza `1`-et.

Fogalmazd meg a nem null argumentum teljes szerződését is.

6.1.7.3 Diagnosztikai feladat: elégtelen nullvizsgálat

Az alábbi hibás C17-részletben a szükséges headerek rendelkezésre állnak:

```
int *p = NULL;

{
    int temporary = 12;
    p = &temporary;
}

if (p != NULL) {
    printf("%p: %d\n", p, *p);
}
```

Azonosítsd az élettartamhibát és a formátumszerződés megsértését. Javítsd a példát úgy, hogy a kiírás a `temporary` élő értékét használja.

6.1.8 Megoldások

6.1.8.1 Magyarázó feladat megoldása

A végső kapcsolatok:

Pointerobjektum	Tárolt érték → cél
`p`	`&y` → `y`, értéke `9`
`q`	`&x` → `x`, értéke `6`

Tehát `x == 6`, `y == 9`, és `*q == 6`. A `q = p` inicializálás pointerértéket másolt; nem hozott létre állandó összeköttetést a két pointerváltozó között.

A `*q` hozzáférések érvényesek, mert `x` élő, inicializált, módosítható `int`, és `q` a címét tartalmazza. A kölcsönzött hozzáférés az `x` élettartamának végén érvénytelenedik. Ellenőrzéskor a pointerek célját és az objektumok értékét külön kell követni.

6.1.8.2 Kódolási feladat megoldása

```c
/* C17-függvényrészlet. */
#include <stddef.h>

int store_if_present(int *out, int value)
{
    if (out == NULL) {
        return 0;
    }

    *out = value;
    return 1;
}
```

Nem null `out` esetén a hívó köteles élő, megfelelően igazított, módosítható `int` objektumra mutató pointert átadni. Az objektumnak a hívás végéig élnie kell. Korábbi értékének inicializáltsága itt nem szükséges, mert a függvény csak írja.

A hívó kezeli az objektum élettartamát; a függvény kölcsönzi a hozzáférést, nem őrzi meg a pointert. Csak a null esethez tartozik meghatározott hibajelzés. A dangling argumentum nem válik biztonságossá a nullvizsgálattól.

Ellenőrzési feltétel: `store_if_present(NULL, 7)` eredménye `0`; élő `int` címével és `7` értékkel hívva az eredmény `1`, a cél értéke pedig `7`.

6.1.8.3 Diagnosztikai feladat megoldása

A blokk végén `temporary` élettartama megszűnik, ezért `p` korábbi értékére már a feltételben sem támaszkodhatunk. Ettől független szerződéshiba, hogy `%p` mellett nincs `void *` argumentum. Nincs garantált hibás kimenet.

A javított C17-részlet:

```c
int *p = NULL;

{
    int temporary = 12;
    p = &temporary;

    printf("%p: %d\n", (void *)p, *p);
    p = NULL;
}
```

A dereferencia idején `temporary` élő, inicializált `int`, a címképzés biztosítja a megfelelő típust, igazítást és objektumhatárt. A pointert még az élettartam vége előtt nullra állítjuk. Ellenőrzési feltétel: a kiírt egész érték `12`, a blokk után pedig `p` null; konkrét címszöveget nem követelünk meg.

6.1.9 Összefoglaló

A pointerváltozó és a referált objektum két külön objektum. `p` pointerérték, `&p` a pointerobjektum saját címe, `*p` pedig közvetett objektumelérés.

A biztonságos dereferencia indoklásához élettartam, megfelelő típus, igazítás, tartomány és műveleti jogosultság szükséges. A nullteszt ezekből csak a null értéket zárja ki. A pointermásolás nem hosszabbít élettartamot, a cast nem javít érvényességet, a címkiírás pedig megfigyelés, nem helyességi bizonyíték.

6.2 Pointeraritmetika, objektumhatár és provenance

A pointeraritmetika nem tetszőleges memóriacímekkel végzett egészaritmetika. C-ben a pointer referált típusa meghatározza a lépés egységét, az objektumhatár pedig azt, hogy a lépés egyáltalán megengedett-e. Egy numerikusan hihető cím ezért még nem jelent jogszerű hozzáférést: számít, melyik tömbből indultunk, meddig tart annak élettartama, és milyen műveletet akarunk végezni.

A Címképzés, dereferencia és null pointer részben megkülönböztettük a pointer értékét a rajta keresztül elért objektumtól. Most ehhez tartományt rendelünk. A Promóciók, signedness és túlcsordulás szabályai továbbra is érvényesek: az index számítása már azelőtt hibás lehet, hogy a pointeraritmetika elkezdődne.

6.2.1 A pointerlépés egysége

ISO C17: pointerhez egész értéket adhatunk, illetve pointerből egész értéket vonhatunk ki, ha a pointer teljes objektumtípusra mutat, és a művelet az alább meghatározott tömbhatárokon belül marad. A „teljes” itt azt jelenti, hogy a referált típus mérete ismert; a `void` és a függvénytípus nem ilyen.

Legyen `a` egy N elemű, `T` elemtípusú tömb, ahol $N > 0$. Ha

$$p = a + i,$$

akkor a `p + n` eredménye az $i + n$ pozíciót jelöli. A megengedettség feltétele:

$$0 \le i \le N, \qquad 0 \le i + n \le N.$$

Ezek matematikai indexekre vonatkozó feltételek; nem azt javasolják, hogy az ellenőrzéshez előbb esetleg túlcsorduló C-kifejezést számítsunk ki.

6.2.1.1 Tömbkép és címeltolódás

A következő számpélda profilja **Linux, x86-64 System V ABI, LP64,** `CHAR_BIT == 8`, `sizeof(int) == 4`. Az offsetek C bájtban értendők. A `0x1000` kezdőcím kizárólag illusztráció, nem mérési eredmény és nem garantált elhelyezés.

Legyen a tömb tartalma `{12, 7, -3, 9}`:

Pozíció	Jelentés	Offset a tömb kezdetétől	Illusztratív cím	Elemként dereferálható?
`a + 0`	`a[0]`, értéke `12`	0	`0x1000`	Igen
`a + 1`	`a[1]`, értéke `7`	4	`0x1004`	Igen
`a + 2`	`a[2]`, értéke `-3`	8	`0x1008`	Igen
`a + 3`	`a[3]`, értéke `9`	12	`0x100c`	Igen
`a + 4`	Tömbvégi határjelző	16	`0x1010`	Nem

Ha `p = a + 1`, akkor `p + 2` nem két C bájttal, hanem két `int` elemmel lép tovább:

$$\Delta = 2 \cdot \text{sizeof}(\texttt{int}) = 2 \cdot 4 = 8 \text{ C bájt.}$$

Az eredmény `a + 3`, tehát az illusztratív cím `0x100c`.

Bizonyításvázlat. A tömbelemek folytonosan, egymást követően helyezkednek el. Egy elem `sizeof(T)` C bájtot foglal, beleértve az elem típusának esetleges belső kitöltését is. Ezért két, n indexkülönbségű elem kezdete között $n \cdot \text{sizeof}(T)$ C bájt az eltolódás. Ez a tárolási kapcsolat magyarázza a lépésméretet, de nem alakítja a nyelvi pointerműveletet korlátlan egészaritmetikává.

Az `unsigned char *` egyes lépése egy C bájt, nem feltétlenül nyolc bit. A `void *` aritmetikáját az ISO C17 nem engedi meg; ha egy fordító mégis elfogadja, az bővítés, nem hordozható C.

6.2.2 Azonos tömb, objektumhatár és one-past

ISO C17: pointeraritmetikával a tömb elemeire és a közvetlenül az utolsó elem utáni, *one-past* pozícióra juthatunk. A one-past érték képzése megengedett, elemként történő dereferálása nem.

A négy elemű tömbnél:

- `a + 4` képezhető;
- `(a + 4) - 1` az utolsó elemre mutat;
- `*(a + 4)` nem használható tömbelem elérésére;
- `a + 5` már önmagában sem megengedett.

Az utolsó pont fontos: nem elég elkerülni a hibás pointer dereferálását. A tömbön túlvezető aritmetikai eredmény képzése is nem definiált viselkedés, azaz UB lehet. Az utólagos visszalépés nem javítja meg.

A szokásos feldolgozási tartomány ezért fél-nyílt:

$$[\texttt{first}, \texttt{last}).$$

A `first` az első feldolgozható elemre, a `last` a kizárt végpontra mutat. A `last` lehet a tömb belsejében vagy annak one-past pozícióján. Az üres tartományt azonos kezdő- és végpointer jelölheti, feltéve, hogy ezek a szerződés szerint érvényes pointerek.

A null pointer más eset: **a null pointerhez adott nulla sem megengedett pointeraritmetika**. Egy üres tartományt null pointerpárral megengedő API-nak ezért külön kell kezelnie ezt az esetet; a `last - first` ott nem válik automatikusan szabályossá.

6.2.2.1 Az objektum nem azonos a memórialappal

Az aritmetikai határt nem az OS által leképezett lap, hanem a C-objektum határa adja. Két egymás mellé kerülő önálló változó nem válik kételemű tömbbé.

E műveletek szabályai szerint egy nem tömbelemként létező objektum úgy kezelhető, mint egy egyelemű tömb. Egy élő `int x` esetén tehát `&x + 1` képezhető, de azon keresztül nem érhetünk el egy feltételezett „következő változót”.

A határ betartása szükséges, de nem elégséges: dereferáláskor az élettartamnak, a referált típusnak és a hozzáférés egyéb feltételeinek is megfelelőnek kell lennie. A megmaradó numerikus cím nem tartja életben az objektumot.

6.2.3 Pointerkülönbség és reprezentálhatóság

ISO C17: két megfelelő, kompatibilis teljes objektumtípusra mutató pointer kivonása akkor megengedett, ha ugyanannak a tömbnek az elemeit vagy one-past pozícióját jelölik, és az eredmény reprezentálható `ptrdiff_t` típussal.

A `ptrdiff_t` a `<stddef.h>` fejlécben deklarált előjeles egész típus. Az eredmény **elemszám**, nem C bájtban mért távolság.

Ha

$$p = a + i, \qquad q = a + j,$$

akkor

$$q - p = j - i.$$

A korábbi példában:

$$(a + 4) - (a + 1) = 4 - 1 = 3.$$

A fordított kivonás eredménye -3. A két pozíció közötti tárolási eltolódás a megadott profilban $3 \cdot 4 = 12$ C bájt, de a pointerkülönbség továbbra is `3`.

A reprezentálhatósági feltétel nem formaság. Egy implementáció elvileg megengedhet olyan nagy tömböt, amelynek két távoli pozíciója közötti különbség nem fér el `ptrdiff_t` típusban. Ilyenkor a kivonás UB, még azonos tömb esetén is. A `<stdint.h>` fejléc `PTRDIFF_MIN` és `PTRDIFF_MAX` makrói írják le e típus értéktartományát.

A későbbi konverzió nem javítja meg a problémát. A `(size_t)(q - p)` előbb elvégzi a pointerkivonást, és csak utána konvertál. Ha pedig a különbség szabályosan negatív, az előjel nélküli konverzió akkor is váratlanul nagy értéket eredményezhet.

6.2.4 Relációs összehasonlítás: nem általános címrendezés

ISO C17: ugyanazon tömbben a `<`, `<=`, `>` és `>=` operátorok az elemek sorrendjét követik. A one-past pozíció minden tényleges elem után következik.

Két független tömb pointereire viszont nem alapozhatunk hordozható rendezést. Attól, hogy a debuggerben látható címek számként rendezhetők, a `p < q` még nem lesz definiált C-művelet.

Vannak külön aggregátumszabályok: például ugyanazon struktúra később deklarált tagjának pointere nagyobbnak hasonlít, a union tagjainak pointerei pedig egyenlőnek. Ezek nem adnak általános felhatalmazást független objektumok összehasonlítására, és pointerkivonásra sem.

Az `==` és `!=` nem azonos kategória a relációs rendezéssel. Megfelelő típusú, meghatározott pointerértékek egyenlősége független objektumok esetén is vizsgálható. Az egyenlőség azonban önmagában nem bizonyít közös tömbhöz tartozást: egy tömbvégi pointer és egy közvetlenül utána elhelyezett objektum kezdőpointere egyenlőnek hasonlíthat. Ettől az első tömb határa nem tolódik ki.

6.2.5 Pointer, egész reprezentáció és provenance

ISO C17: a pointer–egész konverziók általános eredménye implementáció által meghatározott. Ha a pointer értéke a választott egész típusban nem reprezentálható, a pointerből egészbe konverzió UB. Nem szabad automatikusan feltételezni, hogy az `unsigned long` vagy a `size_t` megfelelő pointertároló típus.

A `<stdint.h>` opcionálisan biztosítja a `uintptr_t` előjel nélküli egész típust. Ha létezik, bármely érvényes `void *` pointer átalakítható erre, majd változatlan értékből vissza `void *` típusra úgy, hogy a visszakapott pointer az eredetivel egyenlőnek hasonlítson. A `UINTPTR_MAX` makró jelenléte használható az elérhetőség ellenőrzésére.

Objektumpointer esetén a szemléltetett út:

$$\texttt{int *} \longrightarrow \texttt{void *} \longrightarrow \texttt{uintptr_t} \longrightarrow \texttt{void *}.$$

Ez nem garantálja, hogy az egész reprezentációhoz hozzáadott `sizeof(int)` ugyanazt eredményezi, mint a típusos pointerlépés. Nem általános garancia függvénypointerekre sem. A visszaalakítás nem hosszabbítja meg az eredeti objektum élettartamát.

6.2.5.1 Mit jelent itt a provenance?

A *provenance* a pointer eredetének és objektumhoz kapcsolódásának leírására használt fogalom. Hasznos mentális modell: a pointer nem pusztán egy számsor, hanem hozzáférési lehetőség egy meghatározott objektumhoz, meghatározott feltételekkel.

Ez nem feltétlenül külön, futásidőben tárolt mezőt jelent. Nem azonos a lapvédelemmel vagy egy OS-jogosultsággal sem.

A provenance szabványosítása és pontos modellezése szabványfejlesztési, implementációs és kutatási kérdés. Az ISO C17 konkrét objektum-, élettartam- és pointeraritmetikai szabályait nem szabad összemosni későbbi javaslatokkal, és nem indokolt egységes, minden fordítóra érvényes jövőbeli modellt feltételezni.

A gyakorlati következtetés stabil: ha rendelkezésre áll az eredeti, típusos pointer, azon végezz határellenőrzött aritmetikát. Ne próbálj egész címekből hozzáférési jogosultságot rekonstruálni.

6.2.6 Kidolgozott példa: negatív telemetriaérték keresése

Az alábbi teljes program minimum **ISO C17**-et igényel. Kijelölt laborprofilja: Linux x86-64 System V LP64, a `lab-next` profil (profiltábla). A program nem igényel GNU-bővítést. A parancsok fordítási javaslatok; itt nem történt fordítás vagy futtatás.

A kereső előfeltétele, hogy `first` és `last` ugyanazon élő `int` tömb rendezett határpontjai legyenek. Nem fogad null pointerpárt. Sikertelen kereséskor `last` értékével tér vissza; ez nem dereferálható találat.

Szerep	Objektum vagy hozzáférés	Élettartam és érvénytelenítés
Tulajdonosi felelősség	A `main` blokkja kezeli a `samples` tömböt	A blokk elhagyásáig
Tároló	A `samples` automatikus tárolási idejű tömb	Nem történik áthelyezés vagy újrafoglalás
Kölcsönzött hozzáférések	`first`, `last`, `p`, `hit`, `before`, `after`	Nem élhetik túl érvényes hozzáférésként a tömböt
Sikertelen keresés	`hit == last`	Ekkor nincs elemolvasás
Szerződéssértés	Hibás határ vagy lejárt élettartam	Nincs általános futásidejű ellenőrzés; UB következhet

```c
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>

static int *find_first_negative(int *first, int *last)
{
    for (int *p = first; p != last; ++p) {
        if (*p < 0) {
            return p;
        }
    }
    return last;
}

int main(void)
{
    int samples[] = {12, 7, -3, 9};
    size_t n = sizeof samples / sizeof samples[0];

    int *first = samples;
    int *last = first + n;
    int *hit = find_first_negative(first, last);

    /* A negyelemu tombnel a kulonbseg reprezentalhato. */
    ptrdiff_t count = last - first;
    printf("count=%td\n", count);

    if (hit != last) {
        printf("index=%td value=%d\n", hit - first, *hit);
    } else {
        puts("not found");
```

```
    }

#ifdef UINTPTR_MAX
    void *before = first;
    uintptr_t encoded = (uintptr_t)before;
    void *after = (void *)encoded;

    printf("roundtrip_equal=%d\n", after == before);
#endif

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O2 \
    pointer_bounds.c -o pointer_bounds.gcc

clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O2 \
    pointer_bounds.c -o pointer_bounds.clang
```

A várt első két sor `count=4` és `index=2 value=-3`. Ha a `uintptr_t` rendelkezésre áll, a várt harmadik sor `roundtrip_equal=1`. Ezek levezetett elvárások, nem rögzített futtatási eredmények.

Bizonyításvázlat. A ciklus elején `p` a `first` és `last` közötti zárt pointertartományban van. A törzs csak `p != last` esetén fut, így a dereferencia a fél-nyílt feldolgozási tartományban marad. Az utolsó elem utáni növelés még szabályos végpointert képez; ezt követően a ciklus kilép. A függvény nem tárolja el a kölcsönzött pointereket.

6.2.7 Gyakori hibák és javításuk

A `sizeof(T)` értékével történő kézi felszorzás gyakran kettős skálázást okoz: a `p + n` már elemekben lép. A kívánt harmadik következő elemhez `p + 3` kell, nem `p + 3 * sizeof *p`.

A „majd ellenőrzöm, hogy túlment-e” megközelítés is hibás lehet. Előbb az indexet kell ellenőrizni, és csak utána képezni a pointert. Egy már hibásan létrehozott pointer utólagos összehasonlítása nem helyreállítás.

A figyelmeztetés- vagy sanitizermentes futás nem bizonyítja a szabályosságot. Különösen a puszta, később nem dereferált határátlépő pointerképzést nem feltétlenül jelzi minden diagnosztikai eszköz.

6.2.8 Feladatok

6.2.8.1 Lépés és különbség

Egy élő `int a[6]` tömbben `p = a + 2`. A fenti laborprofilban osztályozd a `p + 3`, `p + 4`, `p + 5` kifejezéseket. Mennyi `(a + 6) - p`, és hány C bájt a megfelelő tárolási eltolódás?

6.2.8.2 Ellenőrzött pointerképzés

Írj `position` nevű függvényt `int *base`, `size_t length`, `size_t index` paraméterekkel. Null bázis vagy `index > length` esetén adjon vissza null pointert, egyébként `base + index` értékét.

Előfeltétel: nem null `base` esetén a hívó biztosítja, hogy az élő tömbben legalább `length` elem következzen a bázistól. Ki a tulajdonos, és dereferálható-e minden nem null eredmény?

6.2.8.3 Diagnosztikai vizsgálat

Az alábbi **hibás ISO C17-részlet** három műveletét egymástól függetlenül vizsgáld. Mindkét tömb helyi, élő objektum; egyik pointer sem tulajdonos.

```
int a[2] = {1, 2};
int b[2] = {3, 4};

int *past = a + 3;
ptrdiff_t distance = b - a;
int ordered = a < b;
```

Melyik művelet hibás, és változtat-e ezen, ha a debugger szomszédos címeket mutat?

6.2.9 Megoldások

6.2.9.1 Lépés és különbség

A `p + 3` az `a[5]` elemre mutat, dereferálható. A `p + 4` az `a + 6` one-past érték, képezhető, de nem dereferálható. A `p + 5` képzése UB.

A különbség $6-2=4$ elem; a megadott profilban az eltolódás $4\cdot 4=16$ C bájt. Ellenőrzési feltétel: a képzett index legyen a $[0,6]$ tartományban, elemhozzáféréshez pedig a $[0,6)$ tartományban.

6.2.9.2 Ellenőrzött pointerképzés

ISO C17-megoldásrészlet, a `<stddef.h>` fejléc szükséges hozzá:

```
int *position(int *base, size_t length, size_t index)
{
    if (base == NULL || index > length) {
        return NULL;
    }
    return base + index;
}
```

A hívó marad a tömb tulajdonosi felelőse; az eredmény kölcsönzött pointer. A tömb élettartamának vége érvényteleníti a hozzáférést. Az `index == length` eredmény a kijelölt tartomány végpontja, ezért nem kezelhető feltétlenül dereferálható elemként.

Ellenőrzési esetek: null bázis, nulla index, `index == length`, valamint `index > length`. A függvény a hívó által megadott `length` valódiságát nem tudja megállapítani.

6.2.9.3 Diagnosztikai vizsgálat

Mindhárom művelet hibás: az `a + 3` túllép a megengedett one-past határon, a kivonás különböző tömböket érint, a relációs összehasonlítás pedig független tömbök pointereit rendezi. Ezekhez nincs garantált kimenet.

A szomszédos gépi címek nem változtatják meg az objektumhatárokat. A helyességi ellenőrzés alapja a tömbazonosság és az indexhatár, nem a debugger címképe vagy a diagnosztika hiánya.

6.2.10 Összefoglaló

A pointeraritmetika egysége a referált típus egy eleme. A megengedett tartomány a tömb első elemétől a one-past pozícióig tart, de a végpointer nem dereferálható. Kivonáskor közös tömb és `ptrdiff_t`-reprezentálhatóság szükséges; relációs összehasonlítással nem rendezhetők tetszőleges objektumcímek. A `uintptr_t` opcionális round-trip lehetőség, nem általános címmanipulációs felhatalmazás. A numerikus reprezentáció soha nem helyettesíti az objektum-, élettartam- és tartományszerződést.

6.3 const pointerek, több szintű indirection és void*

Egy pointerparaméter típusa nem pusztán azt közli, hogy hol található egy objektum. Azt is meghatározza, milyen műveleteket enged meg az adott hozzáférési út. Az `int *`, a `const int *` és az `int *const` ezért eltérő mutálhatósági szerződést fejez ki, miközben egyik sem mondja meg önmagában, ki felel a referált objektum élettartamáért.

A címképzésről és dereferenciáról tanultak alapján a pointerobjektumot meg kell különböztetnünk attól az objektumtól, amelyre mutat. A típusminősítők szabályait most erre a két — később több — szintre alkalmazzuk. A tárgyalás alapja ISO C17; a GNU-bővítéseket külön jelöljük.

6.3.1 A referált objektum és a pointer külön minősíthető

Legyen `T` egy minősítés nélküli objektumtípus, például `int`. Az alábbi táblázat inicializált, érvényes pointerekre vonatkozik. A referált objektum olvasásához és írásához az élettartamra, típusra és hozzáférhetőségre vonatkozó további feltételeknek is teljesülniük kell.

Deklaráció	A `p` pointer átállítható?	A referált objektum módosítható `*p` útján?
`T *p`	Igen	Igen
`const T *p`	Igen	Nem
`T *const p`	Nem	Igen
`const T *const p`	Nem	Nem

A `const T *` és a `T const *` ugyanazt a típust jelöli: pointer konstansként elért `T` objektumra. A csillag utáni `const` viszont magát a pointert minősíti.

C17-részlet:

```c
int retries = 3;
int fallback = 5;

const int *view = &retries;
view = &fallback;               /* Megengedett. */
/* *view = 8; */                /* Constraint-sértés. */

int *const fixed_pointer = &retries;
*fixed_pointer = 8;             /* Megengedett. */
/* fixed_pointer = &fallback; */ /* Constraint-sértés. */

const int *const fixed_view = &retries;
```

A *constraint-sértés* olyan nyelvi megszorítás megsértése, amelyhez a megfelelő C-implementációnak diagnosztikát kell adnia. Nem helyes ezeket egyszerűen „futásidejű hibának” nevezni.

A referált `const` hozzáférési korlátozás, nem általános változatlansági garancia. A fenti `retries` továbbra is módosítható a saját nevén vagy más, megfelelő típusú pointeren keresztül. Ha azonban maga az objektum `const` minősítéssel jött létre, módosításának kísérlete nem definiált viselkedés, akkor is, ha egy cast eltávolította a hozzáférési út minősítését.

6.3.1.1 Mit ígér a const a függvényhatáron?

Az alábbiak **C17 API-deklarációrészletek**:

```c
int read_setting(const int *setting, int *result);

void update_setting(int *setting);
void update_setting(int *const setting);
```

A `read_setting` közvetlenül a `setting` paraméteren keresztül nem írhatja a referált egész objektumot. A `result` viszont kimeneti hely lehet. A típusokból még nem derül ki, megengedett-e a null pointer, vagy mi történik hibakor: ezt dokumentálni kell.

Az `update_setting` két deklarációja kompatibilis, ugyanazt a függvényt is deklarálhatják. **ISO C17 szerint** a paraméter legkülső minősítése nem különbözteti meg a függvénytípusokat. A definícióban szereplő `int *const setting` a helyi paraméterobjektum átállítását tiltja:

- a `setting = másik_pointer` nem megengedett;
- a `*setting = 10` megfelelő előfeltételekkel megengedett.

A pointert a függvény érték szerint kapja. A helyi `setting` átállítása egyébként sem állítaná át a hívó pointerváltozóját.

A `const` nem jelent tulajdonjogot, szálbiztonságot vagy automatikusan hosszabb élettartamot. Nem ígéri azt sem, hogy egy összetett objektum minden további pointeren át elérhető része változatlan marad.

6.3.2 Két indirekciós szint és kimeneti paraméter

Ha a függvénynek a hívó pointerváltozóját kell módosítania, annak címére van szüksége. Innen ered a `T **` kimeneti paraméter.

Nézzük ezt az **API-részletet**:

```
int select_larger(const int *a, const int *b,
                  const int **out);
```

A hívó ilyen változót adhat át:

```
const int *selected = NULL;
/* select_larger(&first, &second, &selected); */
```

A három szint jelentése:

Kifejezés a függvényben	Mit jelöl?	Módosíthatóság
`out`	Helyi pointerparaméter	Átállítható, de ez nem változtatja meg a hívó argumentumát
`*out`	A hívó `selected` pointerobjektuma	Átállítható
`**out`	A kiválasztott egész objektum	Ezen az úton nem módosítható

Az `out` tehát nem „írhatóvá teszi a konstans adatot”. Egy olyan pointerobjektumhoz ad írható hozzáférést, amelynek értéke olvasási nézetet biztosít.

Egy megfelelő szerződés például:

> Siker esetén a visszatérési érték `0`, és `*out` a nagyobb értékű bemeneti objektumra mutat; egyenlőségnél az elsőre. Null `out` esetén a függvény `-1` értékkel tér vissza, és nincs kimeneti írás. Egyébként null bemeneti pointer esetén `*out` null lesz, a visszatérési érték pedig `-1`. A nem null bemeneteknek élő, olvasható `int` objektumokra, a nem null `out` értéknek élő, írható `const int *` objektumra kell mutatnia.

A nullvizsgálat nem képes felismerni a dangling vagy más módon érvénytelen pointert. Ezek kizárása a hívó előfeltétele.

A visszaadott pointer **kölcsönzött hozzáférés**: nincs új objektum, nincs allokáció, nincs tulajdonátadás.

6.3.3 Miért nem konvertálható általánosan a T** const T**-rá?

Az `int *` → `const int *` konverzió megengedett. Az új hozzáférési út kevesebb műveletet enged: nem lehet rajta keresztül írni.

Ebből azonban nem következik az `int **` → `const int **` konverzió megengedettsége.

Hibás, diagnosztikát igénylő C17-részlet; nem futtatandó:

```c
const int frozen = 42;
int *p = NULL;

const int **slot = &p;  /* Nem megengedett implicit konverzió. */
*slot = &frozen;
*p = 7;
```

Bizonyításvázlat — a tiltás típusbiztonsági indoka:

1. A `p` típusa `int *`, ezért a `*p` kifejezés típusa módosítható `int`.
2. Ha a megjelölt konverzió megengedett lenne, a `slot` a `p` pointerobjektumot `const int *` tárolására alkalmas helyként kezelhetné.
3. A `*slot = &frozen` így egy valóban konstans objektum címét juttathatná a `p` változóba.
4. A `*p = 7` már nem hordozna `const` minősítést, mégis konstans objektum módosítását kísérelné meg.

Az utolsó lépések a hipotetikusan megengedett konverzió következményét szemléltetik, nem a hibás program garantált futását. Egy cast nem teszi helyessé ezt az objektumhozzáférési szerződést.

Fontos ellenpélda:

```c
int value = 1;
int *p = &value;
int *const *slot = &p;  /* Megengedett C17-ben. */
```

Itt a `const` a közvetlenül referált **pointerobjektumot** védi. A `*slot = másik_pointer` tiltott, a `**slot = 2` megengedett. Nem lehet tehát ezen az úton másfajta pointerértéket becsempészni a `p` változóba.

A szabály nem az, hogy „több csillagnál tilos a `const` hozzáadása”, hanem az, hogy pontosan melyik szinten történik a minősítés.

6.3.4 A void* típuseltörlés, nem általános hozzáférési engedély

ISO C17 szerint objektumtípusra mutató pointer konvertálható `void *` típusra és vissza; a visszakonvertált pointer az eredetivel egyenlőnek hasonlítandó. A megfelelő konverziók C-ben implicit módon is használhatók:

```c
int counter = 7;
void *opaque = &counter;
int *restored = opaque;
```

Ez C-kód: C++-ban a `void *` → `int *` implicit konverzió nem megengedett.

Az `opaque` nem őrzi meg ellenőrizhető futásidejű típusinformációként, hogy `int` objektumot jelöl. A visszaalakítás helyességét az API szerződésének kell biztosítania. Más típusra alakítva a dereferencia igazítási vagy objektumhozzáférési szabályt sérthet.

A minősítéseket itt is meg kell tartani:

```c
const int limit = 100;
const void *opaque_readonly = &limit;
const int *limit_view = opaque_readonly;
```

A `const int *` → `void *` implicit konverzió eldobná a referált `const` minősítést, ezért nem megfelelő. A `void *` szabály továbbá objektumpointerekre vonatkozik, nem általános ISO C garancia függvénypointerek tárolására.

6.3.4.1 Miért nem általános kimeneti paraméter a void**?

Konverzió	ISO C17
`int *` → `void *`	Megengedett
`int **` → `void *`	Megengedett: egy pointerobjektum címének típusát töröljük
`int **` → `void **`	Nem megengedett implicit módon

A `void **` egy `void *` típusú objektumra mutat. Nem tetszőleges pointerobjektum címét jelenti.

Ha egy függvény `void **out` paraméteren keresztül ír, a hívónak valódi `void *` kimeneti objektumot kell biztosítania. Egy `int *` változó címének `(void **)` castolása nem alakítja át magát a változót `void *` objektummá. A rajta keresztüli írás nem kompatibilis lvalue-típussal érné el a pointerobjektumot; ezt az azonos pointerméret sem teszi helyessé.

A helyes minta: külön `void *` változó fogadja az eredményt, majd annak **értékét** konvertáljuk a dokumentált céltípusra.

6.3.4.2 A void* aritmetika nem ISO C

A `void` nem teljes objektumtípus, ezért ISO C17-ben nem végezhető `void *` pointeraritmetika, és nem olvasható vagy írható rajta keresztül referált objektum.

GNU-bővítésként egyes fordítómódok megengedik a `void *` aritmetikát, egy C bájtos lépéssel. Ez nem hordozható ISO C-kód. Objektumreprezentáció bejárásához megfelelő előfeltételekkel `unsigned char *`, illetve `const unsigned char *` használható. A konverzió nem szünteti meg az objektumhatárokra vonatkozó szabályokat, és egy C bájt nem feltétlenül nyolc bit.

6.3.5 Kidolgozott példa és ownership-szerződés

Az alábbi teljes program két különböző kimeneti paramétert használ. A `select_larger` a hívó objektumaira ad nézetet; a `borrowed_counter` modulhoz tartozó, statikus tárolási idejű objektumot tesz elérhetővé típuseltörléssel.

```
#include <assert.h>
#include <stddef.h>

static int select_larger(const int *a, const int *b,
                         const int **out)
{
    if (out == NULL) {
        return -1;
    }

    *out = NULL;
    if (a == NULL || b == NULL) {
```

```
        return -1;
    }

    *out = (*a >= *b) ? a : b;
    return 0;
}

/*
 * Siker: *out egy modultulajdonú, írható int objektumra mutat.
 * Az objektum a program teljes futása alatt él.
 * Nincs tulajdonátadás; a visszaadott pointer nem adható free()-nek.
 * Null out: -1, kimeneti írás nélkül.
 */
static int borrowed_counter(void **out)
{
    static int counter = 7;

    if (out == NULL) {
        return -1;
    }

    *out = &counter;
    return 0;
}

int main(void)
{
    int a = 12;
    int b = 19;
    const int *selected = NULL;

    if (select_larger(&a, &b, &selected) != 0) {
        return 1;
    }
    assert(selected == &b);
    assert(*selected == 19);

    b = 23;
    assert(*selected == 23);  /* A const nézet nem másolat. */

    void *raw = NULL;
    if (borrowed_counter(&raw) != 0) {
        return 1;
    }

    int *counter = raw;       /* A dokumentált típusra alakítjuk. */
    assert(*counter == 7);
    *counter = 8;

    const void *erased_view = selected;
    const int *restored_view = erased_view;
    assert(restored_view == selected);
```

```
    int status = select_larger(NULL, &b, &selected);
    assert(status == -1);
    assert(selected == NULL);

    return 0;
}
```

A választott fordítási profil: **C17, Linux x86-64 System V, LP64**, a `lab-next` profil (profiltábla). A következő parancsokban a fordítónevek ezekre a kiválasztott kiadásokra értendők. **Buildjegyzőkönyv:** a kiadási példaleltár a fenti programot a `lab-gcc` profilban (GCC 13.3.0, `-std=c17 -Wall -Wextra`) figyelmeztetés nélkül lefordította, és a program `0` kóddal futott, vagyis az `assert` feltételek teljesültek. A a `lab-next` profil (profiltábla) sorokat nem ellenőriztük.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -pedantic-errors -O0 -g pointer_contract.c -o pointer_contract_gcc

clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
      -pedantic-errors -O0 -g pointer_contract.c -o pointer_contract_clang
```

Az ownership- és élettartam-térkép:

Objektum vagy hozzáférés	Tulajdonos, tároló	Érvényesség és érvénytelenítés
`a`, `b`	A hívó `main` automatikus tárolású objektumai	Blokkjuk elhagyásáig élnek
`selected` referált objektuma	Kölcsönzött; a tároló továbbra is a hívóé	A célozott objektum élettartamának végén a hozzáférés érvénytelenné válik
`out` a kiválasztás alatt	Kölcsönzött hozzáférés a hívó pointerobjektumához	Csak a hívás alatt használjuk, nem tároljuk el
Statikus `counter`	A modulhoz tartozó statikus tároló	A program futása alatt él
`raw`, `counter`, `restored_view`	Helyi, nem tulajdonló pointerobjektumok	Megszűnésük nem szünteti meg a referált objektumot

A `b` értékének módosítása nem érvényteleníti a rá mutató nézetet, csak megváltoztatja a megfigyelhető értéket. A `selected` átállítása sem szünteti meg a korábbi célt. A program egyszálú; a módosítható statikus számláló konkurens használatához külön szinkronizációs szerződés kellene.

6.3.6 Gyakori hibák és javításuk

„A `const` pointeren át látott adat nem változhat.” Más hozzáférési úton változhat. Ha pillanatfelvétel kell, másolat szükséges.

„A kimeneti paraméternek adott pointert a függvény majd validálja.” Általános pointerérvényesség-ellenőrzés nincs. A nullvizsgálat csak egyetlen hibás állapotot fed le.

„A cast megoldja a dupla pointer inkompatibilitását.” A cast az objektum deklarált típusát nem változtatja meg. A javítás megfelelő típusú köztes pointerobjektum, nem erősebb cast.

„A visszaadott pointert nekem kell felszabadítanom.” Ezt csak ownership-szerződés döntheti el. A példában minden eredmény kölcsönzött, ezért nincs felszabadítási kötelezettség vagy jogosultság.

6.3.7 Feladatok

1. **Magyarázó feladat.** Mit enged meg az `int *const *slot` az `out`, `*out`, `**out` szinteknek megfelelően? Átadható-e neki egy `int *p` változó címe?
2. **Kódolási feladat.** Írd meg az `int increment_selected(int *const *slot)` függvényt. Null `slot`, null `*slot` vagy `INT_MAX` érték esetén adjon `-1` eredményt, módosítás nélkül. Egyébként növelje eggyel a referált egész értéket, és térjen vissza nullával. A bemenet kölcsönzött; a függvény nem tárolhatja el.
3. **Diagnosztikai feladat.** Miért hibás a következő két sor két különböző módon? Javítsd őket a kidolgozott példa API-jával.

```
int *typed = NULL;
borrowed_counter((void **)&typed);
const int **read_slot = &typed;
```

6.3.8 Megoldások

6.3.8.1 A mutálhatósági szintek felismerése

A helyi `slot` paraméter átállítható. A `*slot` pointerobjektum ezen az úton nem módosítható, a `**slot` egész objektum viszont igen. Az `int **` → `int *const *` konverzió megengedett, ezért `&p` átadható.

Ellenőrzési feltétel: a függvénytörzsben a `*slot = NULL` diagnosztikát igényelne, a `**slot = 4` pedig megfelelő élettartam- és hozzáférési feltételekkel megengedett.

6.3.8.2 Ellenőrzött módosítás dupla pointeren keresztül

A következő **C17-részlet** az előző programba illeszthető, azonos fordítási profillal:

```
#include <limits.h>

int increment_selected(int *const *slot)
{
    if (slot == NULL || *slot == NULL) {
        return -1;
    }
    if (**slot == INT_MAX) {
        return -1;
    }

    ++**slot;
    return 0;
}
```

Előfeltétel, hogy a nem null `slot` élő, olvasható pointerobjektumra, annak nem null értéke pedig élő, írható `int` objektumra mutasson. A kölcsönzés csak a hívásra szól; a függvény nem érvénytelenít semmit.

Ellenőrzési feltételek: `4` értékből `5` lesz; `INT_MAX` változatlan marad; null bemenetnél nincs dereferencia; a hívó pointerének értéke egyik esetben sem változik.

6.3.8.3 A kimeneti pointer típusának helyreállítása

Az első hibás sor castja elfedheti a diagnosztikát, de a függvény ezután egy `int *` objektumba írna `void *` típusú lvalue-n keresztül. A második sor önmagában tiltott implicit minősítéskonverzió, amely diagnosztikát igényel.

Javított C17-részlet:

```
void *raw = NULL;

if (borrowed_counter(&raw) == 0) {
    int *typed = raw;
    const int *read_view = typed;
    const int **read_slot = &read_view;

    assert(**read_slot == *typed);
}
```

Most minden kimeneti hely tényleges típusa megfelel az író pointer referált típusának. A `read_view` külön pointerobjektum, nem a `typed` tárolójának átértelmezése. A célt továbbra is a modul birtokolja.

6.3.9 Összefoglaló

A `const` mindig egy meghatározott típusszintet minősít. A `const T *` a referált objektum ezen az úton történő módosítását, a `T *const` a pointer átállítását tiltja. A `T **` pointerobjektum módosítására használható, de nem általánosan konvertálható `const T **` vagy `void **` típusra.

A jó pointer-API a típus mellett rögzíti a nullkezelést, a hibakimenetet, a tulajdonost, a kölcsönzés időtartamát és az érvénytelenítés feltételeit. A minősítés a szerződés fontos része — de nem helyettesíti a teljes szerződést.

6.4 Deklarátorok: tömb-, pointer- és függvénykapcsolatok

Az összetett C-deklarációk olvasása nem különleges memóriafeladat, hanem néhány nyelvtani szabály következetes alkalmazása. Ez rendszerprogramozásban közvetlenül hasznos: egy interfészből meg kell tudnod állapítani, hogy pointerek tábláját, összefüggő tömb címét vagy egy később meghívható függvény címét kapod-e.

A szintaktikai áttekintésből ismert deklarációkat és a többszintű pointerek, illetve a `const` szabályait használjuk fel. A típus azonban csak a szerződés egyik része: nem mondja meg automatikusan az elemszámot, a tulajdonlást vagy a referált objektum hátralévő élettartamát.

Az alábbi nyelvi szabályok az **ISO C17** profilra vonatkoznak. A rövid kódblokkok jelölt deklaráció- vagy függvénytörzs-részletek; a mini labor teljes fordítási egységet tartalmaz.

6.4.1 A deklarátor olvasási algoritmusa

Egy deklarációban a **deklarációs specifikátorok** (*declaration specifiers*) adják például az `int` típust és a típusminősítőket. A **deklarátor** (*declarator*) ehhez kapcsolja az azonosítót, valamint a pointer-, tömb- és függvényszerkezetet.

Az alábbi ISO C17 deklarációrészletben az `int` közös, de a két deklarátor különbözik:

```
int *p, count;
```

Csak `p` pointer; `count` típusa `int`. Ezért összetett típusoknál célszerű deklarációnként egy azonosítót használni.

A deklarátornyelvtanban a `[]` tömbképző és a paraméterlistát tartalmazó `()` függvényképző szorosabban kapcsolódik, mint a pointert jelölő `*`. A csoportosító zárójel ezt módosítja. Ez nem kifejezéskiértékelési sorrend: a fordító itt típust épít fel.

Használható olvasási algoritmus:

1. Keresd meg a deklarált azonosítót, és különítsd el a deklarációs specifikátorokat.
2. Az aktuális zárójelezési szinten először az azonosítóhoz kapcsolódó jobb oldali `[]` és függvényparaméter-lista jelentését olvasd el.
3. Ezután dolgozd fel a bal oldali `*` jeleket és a hozzájuk tartozó minősítőket.
4. A csoportosító zárójelből kilépve folytasd a külső szinten.
5. Végül illeszd hozzá a specifikátorok által megadott típust.

A paraméterlistában szereplő deklarációkat külön kell értelmezni. A módszer ezért pontosabb a feltétel nélküli „mindig jobbra, majd balra” szabálynál.

Például:

```
/* ISO C17 deklarációrészlet. */
int *a[3];
int (*b)[3];
```

Az első sorban `a` után közvetlenül `[3]` áll:

> `a` háromelemű tömb, amelynek elemei pointerek `int` objektumokra.

A második sorban a csoportosítás miatt előbb a `*b` kapcsolatot kell feldolgozni:

> `b` pointer egy háromelemű, `int` elemeket tartalmazó tömbre.

A zárójel tehát nem díszítés: más típust és más műveleti szerződést hoz létre.

6.4.2 Pointertömb és tömbre mutató pointer

Legyen `N` pozitív egész konstanskifejezés. A két alaptípus:

Deklaráció	A deklarált objektum típusa	Referált objektum
`int *a[N]`	`N` darab `int *` elemből álló tömb	Minden elem külön `int` objektumra mutathat
`int (*p)[N]`	Egyetlen pointer	Egy teljes, `N` elemű `int` tömb

A pointertömb **pointerértékeket tárol**. A referált egész objektumok lehetnek különböző helyeken, ugyanazon tömbben, vagy több pointer mutathat ugyanarra az objektumra. A típus nem garantál összefüggő `int` tárolót.

A tömbre mutató pointer ezzel szemben egy tömbobjektumot referál. A méret a referált típus része: az `int (*)[3]` és az `int (*)[4]` nem kompatibilis pointertípus.

Az alábbi ISO C17 függvénytörzs-részletben minden tároló automatikus tárolási idejű:

```
enum { N = 3 };

int values[N] = {10, 20, 30};
int *addresses[N] = {&values[0], &values[1], &values[2]};
int (*row)[N] = &values;

*addresses[1] = 25;
(*row)[2] = 35;
```

Az első írás a második pointertömb-elemen keresztül módosítja `values[1]` értékét. A másodikban:

1. `row` tömbre mutató pointer;
2. `*row` a referált tömböt jelöli;
3. `(*row)[2]` annak harmadik elemét jelöli.

A `row[2]` mást jelentene: pointerindexeléssel a harmadik referált tömböt keresné. Ebben a példában nincs ilyen objektum, ezért az ilyen hozzáférés nem definiált viselkedést okozna.

Szereplő	Tároló és tulajdonosi felelősség	Élettartam és érvénytelenítés
`values`	A blokk által kezelt egészérték-tároló	A blokk elhagyásáig él
`addresses`	Saját pointerelemeket tárol; az egész értékeket csak kölcsönzi	A `values` élettartamának vége érvényteleníti a referált hozzáférést
`row`	Egy pointerértéket tárol; a tömböt csak kölcsönzi	Ugyanaz az élettartamkorlát érvényes

Itt nincs dinamikus allokáció és nincs visszaadott hibakód. A tartomány- vagy élettartam-szerződés megsértése nem automatikusan ellenőrzött hiba.

A méretkülönbség számszerű ABI-feltételezés nélkül is levezethető:

$$\text{sizeof}(addresses) = N \cdot \text{sizeof}(int*)$$

mert a tömb `N` pointerelemet tartalmaz, míg

$$\text{sizeof}(*row) = N \cdot \text{sizeof}(int).$$

Ezzel szemben `sizeof row` egy pointerobjektum mérete. Minden méret C bájtban értendő; itt sem az `int`, sem a pointer konkrét szélességét nem feltételezzük.

Gyakori tévedés, hogy a tömbre mutató pointert `int **` típussal helyettesítik. Az `int **` egy `int *` objektumra mutat, nem `int` elemek tömbjére. A tömbkifejezések pointerré alakulását részletesen a tömbobjektum és decay tárgyalja; ez az átalakulás sem teszi azonossá ezeket a típusokat.

6.4.3 Függvénypointerek és függvénypointert visszaadó függvények

A függvényképző ugyanúgy részt vesz az olvasási algoritmusban:

```
/* ISO C17 deklarációrészlet. */
int *produce(void);
int (*transform)(int);
```

A `produce` paraméter nélküli függvény, amely `int *` értékkel tér vissza. A `transform` pointer egy olyan függvényre, amely egy `int` paramétert fogad és `int` eredményt ad.

C17-ben a `(void)` azt jelenti, hogy a függvénynek nincs paramétere. Egy `int f();` deklaráció üres zárójele nem ugyanez: nem ad információt a paraméterek számáról és típusáról.

Most tekintsük ezt:

```
/* ISO C17 deklarációrészlet. */
int (*select_transform(int kind))(int);
```

Az olvasás közbülső lépései:

1. `select_transform(int kind)`: az azonosító egy `int` paraméterű függvényt jelöl.
2. `*select_transform(...)`: a függvény visszatérési értéke pointer.
3. A külső `(int)`: ez a pointer egy `int` paraméterű függvényre mutat.
4. A kezdeti `int`: a referált függvény visszatérési típusa.

Vagyis egy stratégia-választó függvényt deklaráltunk, amely egy másik függvény címét adja vissza. Ilyen szerkezet használható például feldolgozási mód vagy ellenőrzési szabály kiválasztására.

C-függvény nem térhet vissza közvetlenül tömb- vagy függvénytípussal, de visszaadhat ezekre mutató pointert.

Egy függvénypointer meghívható `transform(value)` vagy `(*transform)(value)` alakban. Előfeltétel, hogy nem null, megfelelő függvényre mutató értéket tartalmazzon, és a referált függvény típusa kompatibilis legyen a híváshoz használt függvénytípussal. A cast nem javítja ki az inkompatibilis hívási szerződést.

Függvénypointerekből tömb is képezhető:

```
/* ISO C17 deklarációrészlet. */
int (*operations[2])(int);
```

Ez két pointerből álló tömb, nem függvényeket tartalmazó tömb. A callbackek és a hozzájuk társított állapot szerződését a következő egység részletezi.

6.4.4 `typedef`: olvashatóság és a rejtett `const`

A `typedef` meglévő típushoz rendel nevet; nem hoz létre új, elkülönült típust. Jó felhasználása az összetett típus rétegenkénti megnevezése:

```
/* ISO C17 deklarációrészlet. */
typedef int transform_fn(int);
typedef transform_fn *transform_ptr;
typedef transform_ptr selector_fn(int);

transform_ptr select_transform(int kind);
```

A `transform_fn` függvénytípus, nem pointertípus. A `transform_ptr` már pointer erre a függvénytípusra. A `selector_fn` olyan függvénytípus, amely `transform_ptr` értéket ad vissza.

Ez ugyanazt a szerkezetet fejezi ki, mint a korábbi összetett deklaráció, de a fogalmi határok nevet kapnak. Hasznos szabály: az aliasnevek jelezzék, ha pointert rejtenek.

A legfontosabb minősítőcsapda:

```
/* ISO C17 függvénytörzs-részlet. */
typedef int *int_ptr;

int value = 7;
int_ptr const fixed = &value;
const int *readonly = &value;
```

A `fixed` típusa `int * const`: **maga a pointer nem írható át**, a referált `int` viszont módosítható. A `readonly` típusa `const int *`: ezen az elérési úton nem módosítható az egész objektum, a pointerérték viszont lecserélhető.

A `typedef` nem szöveghelyettesítés. A `const` az alias által megnevezett teljes pointertípust minősíti.

Mindkét pointer `value` nem tulajdonló elérése; a blokk elhagyása megszünteti a célobjektum élettartamát. Egy tiltott értékadás fordítási diagnosztikát igényel, míg az élettartam megsértését a típus önmagában nem feltétlenül jelzi.

6.4.5 Incomplete típusokra mutató pointerek

Nem teljes típus (*incomplete type*) esetén még nem áll rendelkezésre az objektumméret meghatározásához szükséges összes információ. A pointertípus ettől még teljes lehet:

```
/* ISO C17 deklarációrészlet. */
struct session;
typedef struct session Session;

Session *current;
```

A fordító ismeri `current` pointertípusát, de nem ismeri a `struct session` tagjait és méretét.

Ezért `sizeof current` megengedett, `sizeof *current` viszont ezen a ponton nem. Tagelérés és `current + 1` sem használható: az utóbbihoz teljes referált objektumtípus kell.

A deklaráció nem hoz létre `Session` objektumot, és nem határozza meg a tulajdonlást. Egy későbbi interfésznek kell előírnia, ki hozza létre, meddig tartja életben és ki szünteti meg a referált objektumot. Erre épül az opaque handle, de itt csak a típusszintű alapot rögzítjük.

Ismeretlen méretű tömb is lehet incomplete:

```
/* ISO C17 deklarációrészlet. */
extern int samples[];
int (*unknown_row)[];
```

A `samples` deklarációja nem adja meg a tömb elemszámát. Az `unknown_row` pedig ilyen tömbre mutató pointer. A pointer tárolható, de `sizeof *unknown_row` és `unknown_row + 1` nem megengedett, amíg a referált típus nem teljes.

6.4.6 Fordítóval ellenőrizhető típus-egyezési mini labor

A következő `declarators.c` teljes ISO C17 program. A `_Generic` típuskompatibilitás szerint választ ágat, a `_Static_assert` pedig fordításkor ellenőrzi az így kapott konstans állítást. Ez **nem** bizonyítja a futásidejű élettartam- vagy tartományhelyességet.

```
#include <stddef.h>

enum { N = 3 };

typedef int row_type[N];
typedef int transform_fn(int);
typedef transform_fn *transform_ptr;
typedef transform_ptr selector_fn(int);

struct session;
typedef struct session Session;

/* Nyers deklarátor; a definíció ugyanazt aliasokkal írja le. */
int (*select_transform(int kind))(int);

static int identity(int value)
{
    return value;
}

static int nonnegative(int value)
{
    return value < 0 ? 0 : value;
}

transform_ptr select_transform(int kind)
{
    return kind == 0 ? identity : nonnegative;
}

_Static_assert(
    _Generic(&select_transform, selector_fn *: 1, default: 0),
    "selector type mismatch"
);

int main(void)
{
    int values[N] = {-2, 5, 7};
    int *addresses[N] = {
        &values[0], &values[1], &values[2]
    };
    row_type *row = &values;
    transform_ptr operations[2] = {
        select_transform(0), select_transform(1)
    };
    Session *session = NULL;
```

```
    _Static_assert(
        _Generic(row, int (*)[N]: 1, default: 0),
        "row pointer type mismatch"
    );
    _Static_assert(
        _Generic(&addresses[0], int **: 1, default: 0),
        "pointer element address mismatch"
    );
    _Static_assert(
        sizeof addresses == N * sizeof(int *),
        "pointer array size mismatch"
    );

#ifdef NEGATIVE_TYPES
    row = addresses; /* Szándékos constraint-sértés. */
#endif

    (*row)[1] = operations[1]((*row)[0]);
    (void)session;

    return values[1] == 0 && operations[0](7) == 7 ? 0 : 1;
}
```

A lokális egészérték-tároló kezelője a `main` blokkja; a `row` és az `addresses` csak kölcsönöz. Érvényességük nem lépi túl a `values` élettartamát. A függvénypointerek a programban definiált függvényeket jelölik; nincs tulajdonátruházás vagy dinamikus kódbetöltés. A `session` null érték, ezért nem jelöl elérhető `Session` objektumot, és a program nem is dereferálja.

Választott laborprofil: Linux x86-64, System V ABI, LP64; a `lab-next` profil (profiltábla) fejlécekkel. A példában nincs ABI-specifikus méretfeltételezés. A parancsok a megfelelő verziójú `gcc` és `clang` elérhetőségét feltételezik; más kiadások használatát külön rögzítsd.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror \
    declarators.c -o declarators-gcc

clang -std=c17 -Wall -Wextra -Wpedantic -Werror \
    declarators.c -o declarators-clang
```

A negatív próba szándékosan hibás fordítási egységet képez:

```
gcc -std=c17 -Wall -Wextra -pedantic-errors \
    -DNEGATIVE_TYPES -fsyntax-only declarators.c

clang -std=c17 -Wall -Wextra -pedantic-errors \
    -DNEGATIVE_TYPES -fsyntax-only declarators.c
```

Az értékadás jobb oldalán az `addresses` kifejezés az első pointerelemre mutató `int **` értékké alakul. Ez nem kompatibilis a bal oldal `int (*)[N]` típusával.

Az elvárt ellenőrzési eredmény: a pozitív változat lefordul, a negatív változatot a megadott kapcsolókkal a fordító elutasítja. A pozitív program elvárt kilépési állapota `0`. Ezek elvárások, nem dokumentált fordítási vagy futtatási eredmények; a diagnosztika pontos szövege verziófüggő.

6.4.7 Feladatok

6.4.7.1 Magyarázó feladat

Értelmezd az alábbi ISO C17 deklarációrészlet minden sorát. Melyik deklarál függvényt, és melyik objektumot?

```
int *slots[4];
int (*block)[4];
int (*actions[4])(int);
int (*choose(void))(int);
```

6.4.7.2 Kódolási feladat

Alakítsd át az `actions` és a `choose` deklarációját `typedef` használatával. Külön nevezd meg a függvénytípust és a pointertípust. Adj fordítási idejű ellenőrzést arra, hogy `choose` valóban paraméter nélküli, megfelelő függvénypointert visszaadó függvény.

6.4.7.3 Diagnosztikai feladat

Az alábbi ISO C17 függvénytörzs-részlet két szándékos hibát tartalmaz. Azonosítsd és javítsd őket cast nélkül.

```
typedef int *int_ptr;

int x = 1;
int y = 2;
int_ptr const p = &x;
p = &y;

struct record;
struct record *r = 0;
(void)sizeof *r;
```

6.4.8 Megoldások

6.4.8.1 A deklarációk értelmezése

A `slots` négy `int *` elemet tartalmazó tömbobjektum. A `block` pointerobjektum, amely négy `int` elemből álló tömbre mutathat.

Az `actions` négy függvénypointert tartalmazó tömbobjektum. Minden referált függvény egy `int` paramétert fogad és `int` eredményt ad.

A `choose` függvény: nincs paramétere, és egy `int` paraméterű, `int` eredményű függvényre mutató pointert ad vissza.

Az ellenőrzés kulcsa az első kapcsolódó típusművelet: `actions` esetén tömbképzés, `choose` esetén függvényképzés történik.

6.4.8.2 Az aliasos változat

Az alábbi ISO C17 részlet fájlszinten helyezhető el:

```
typedef int action_fn(int);
typedef action_fn *action_ptr;
typedef action_ptr chooser_fn(void);

action_ptr actions[4];
action_ptr choose(void);

_Static_assert(
    _Generic(&choose, chooser_fn *: 1, default: 0),
    "choose type mismatch"
);
```

Az ellenőrzés azért az `&choose` kifejezést használja, mert annak típusa pointer a deklarált függvénytípusra. A részlet önmagában szintaktikai ellenőrzéssel vizsgálható; egy ténylegesen `choose`-t hívó teljes programhoz függvénydefiníció is kell.

6.4.8.3 A két hiba javítása

A `p` konstans pointer, ezért nem rendelhető hozzá új cím. Ha az átirányítás szükséges, a deklaráció legyen `int_ptr p = &x;`. Ha csak a referált értéket kell módosítani, az eredeti deklaráció mellett `*p = 2;` megengedett. A két javítás eltérő célt valósít meg.

A `struct record` nem teljes típus, ezért mérete nem kérdezhető le. Ha a pointer mérete kell, `sizeof r` a javítás. Ha a struktúra mérete szükséges, előbb teljes struktúradefiníciót kell adni.

A második hiba nem a null pointer dereferálásából ered: ebben az esetben a `sizeof` nem értékeli ki az operandusát. Az operandus nem teljes típusa miatt hibás. Az ellenőrzési feltétel mindkét javításnál az, hogy a választott szándékhoz illő változat C17 módban diagnosztika nélkül fordítható legyen.

6.4.9 Rövid összefoglaló

Az összetett deklarátorokat az azonosítótól kifelé, a zárójelezési szinteket követve olvasd. A `[]` és a függvényparaméter-lista szorosabban kapcsolódik, mint a `*`.

A pointertömb, a tömbre mutató pointer és a pointerre mutató pointer különböző típus és különböző tárolási szerződés. A `typedef` olvashatóvá teheti ezeket, de nem szöveghelyettesítés: a pointeraliasra alkalmazott `const` magát a pointert minősíti.

A fordítóval ellenőrizhető típuskompatibilitás fontos első kapu. A tulajdonlást, az élettartamot és az érvényes hozzáférési tartományt azonban továbbra is külön kell megadni és betartani.

6.5 Függvénypointerek, callbackek és context pointer

A C függvénypointere azt teszi lehetővé, hogy egy algoritmus vagy eseményforrás működésének egy részét a hívó határozza meg. Rendezésnél ez az összehasonlítás, bejárásnál az elemen végzett művelet, eseménykezelésnél az esemény feldolgozása. A visszahívás (*callback*) azonban rendszerint állapotot is igényel: számlálót, konfigurációt vagy egy feldolgozó objektumot.

C-ben ezt jellemzően két összetartozó érték képviseli: egy függvénypointer és egy `void *` context pointer. A helyes interfész nemcsak ezek típusát, hanem élettartamukat, tulajdonlásukat és hibaszerződésüket is meghatározza.

A deklarátorokról szóló rész alapján már olvasható a függvénypointer deklarációja. A `void *` tárgyalásából pedig azt kell felidézni, hogy az általános objektumpointer nem őrzi meg a referált objektum típusinformációját.

6.5.1 Függvénypointer és hívási kompatibilitás

ISO C17: a függvénypointer függvényre mutató érték. Nem objektum tárolóját jelöli, és nem alkalmazható rá pointeraritmetika.

Az alábbi deklarációrészlet egy összehasonlító függvény pointertípusát nevezi el:

```
/* ISO C17; deklarációrészlet. */
typedef int (*compare_fn)(const void *, const void *);
```

A `compare_fn` olyan pointertípus, amelyen keresztül két `const void *` argumentumot fogadó, `int` értéket visszaadó függvény hívható. A függvény neve pointert igénylő kifejezésben rendszerint automatikusan függvénypointerré alakul; az explicit `&` is használható. Ha `fn` megfelelő függvénypointer, akkor az `fn(a, b)` és a `(*fn)(a, b)` ugyanazt a hívást fejezi ki.

A hívás feltételei:

- a pointer nem null, és hívható függvényt jelöl;
- a pointer által meghatározott függvénytípus kompatibilis a tényleges függvény típusával;
- az argumentumok teljesítik a függvény további szerződését.

A kompatibilitás nem pusztán az argumentumok számát jelenti. A visszatérési típus, a paramétertípusok és a variadikus jelleg is számít. Célszerű a callbacket eleve az API-ban megadott prototípussal definiálni.

ISO C17: különböző függvénypointer-típusok között lehet explicit konverziót végezni, és az eredeti típusra visszakonvertált pointer egyenlő az eredetivel. Ez azonban **nem teszi megengedetté az inkompatibilis típuson keresztüli hívást**: annak viselkedése nem definiált. Egy cast nem alakítja át a függvény paraméterkezelését.

Paraméter nélküli C17 függvénynél a `(void)` prototípust használd. Az üres `()` deklaráció C17-ben nem ugyanezt mondja: nem ad paramétertípus-információt.

6.5.2 A callback és a context közös szerződése

A context pointer (*context pointer*, felhasználói állapotpointer) az algoritmus számára átlátszatlan adat. Az algoritmus tárolhatja vagy továbbadhatja, de csak az a callback értelmezheti, amely ismeri a mögötte álló típust.

A pointerpár fogalmilag egyetlen egység:

$$\text{visszahívás} = (\text{függvény},\ \text{hozzá tartozó állapot}).$$

Ha a függvény az egyik regisztrációból, az állapot egy másikból származik, a típusellenőrzés ezt többnyire nem észleli. A callback hibás objektumtípust feltételezhet, vagy érvénytelen tárolóhoz férhet hozzá.

Két élettartam-modellt kell megkülönböztetni.

Szinkron visszahívásnál a feldolgozó függvény nem őrzi meg a pointerpárt. Minden callback befejeződik, mielőtt a feldolgozó visszatér. A hívó megfelelően élő lokális objektuma ezért használható contextként.

Tárolt regisztrációnál az eseményforrás megőrzi a pointerpárt. A contextnek minden lehetséges későbbi hívás alatt élnie kell. Nem elég addig megőrizni, amíg a regisztrációs függvény visszatér.

Az alábbi példában minden hozzáférés kölcsönzött:

Erőforrás	Tulajdonos, illetve kezelő	Elvárt élettartam	Érvénytelenítő esemény
Bejárt tömb tárolója	A `main` lokális tömbobjektuma	A teljes bejárás alatt	A tömb élettartamának vége
Bejárási context	A `main` lokális állapotobjektuma	A `walk_ints` visszatéréséig	Az állapot élettartamának vége
Regisztrált context	A `main` lokális állapotobjektuma	Minden hívás befejezéséig és a leiratkozásig	Korai élettartamvég
Eseményregisztráció	A hívó által kezelt `event_slot`	A regisztráció használata alatt	A slot élettartamának vége
Callback függvény	A program kódja	Minden lehetséges hívás alatt	Platformfüggő modulkezelésnél például túl korai modulkitöltés helyett modul-*kirakás*

Az utolsó eset nem az ISO C dinamikusbetöltési szolgáltatása: ha megosztott modulok kirakása is lehetséges, annak külön platformfüggő szerződés kell. Az alábbi önálló program függvényei a program futása alatt rendelkezésre állnak.

6.5.3 Kidolgozott példa: rendezés, visitor és eseménykezelő

A következő interfész három eredményt különböztet meg:

- `CB_OK`: a feldolgozás folytatható;
- `CB_STOP`: szándékos korai megállás;

- `CB_ERROR`: feldolgozási vagy szerződésellenőrzéssel felismert hiba.

A látogató (*visitor*) értékként kapja az aktuális egész számot. Nem kap az elemre mutató, eltárolható pointert. A bejáró nem tartja meg sem a callbacket, sem a contextet.

Az eseménypélda egyetlen `int` eseményértéket továbbít. A slot már megőrzi a pointerpárt, de nem veszi át a context tulajdonlását.

Példaprofil: ISO C17, hosted Linux, x86-64 System V ABI, LP64. Választott reprodukciós eszközprofil: a `lab-next` profil (profiltábla). A kód nem igényel GNU nyelvi bővítést. Az alábbiakhoz nem tartozik itt tényleges fordítási vagy futási eredmény.

```
/* callbacks.c */
#include <limits.h>
#include <stddef.h>
#include <stdlib.h>

enum cb_result {
    CB_OK,
    CB_STOP,
    CB_ERROR
};

typedef enum cb_result (*callback_fn)(int value, void *ctx);

struct counter {
    int threshold;
    size_t limit;
    size_t accepted;
};

static enum cb_result accept_limited(int value, void *ctx)
{
    if (ctx == NULL)
        return CB_ERROR;

    struct counter *state = ctx;

    if (value < state->threshold)
        return CB_OK;

    if (state->accepted >= state->limit)
        return CB_STOP;

    ++state->accepted;
    return CB_OK;
}

static enum cb_result walk_ints(
    const int *items, size_t count, callback_fn fn, void *ctx)
{
    if (fn == NULL || (count != 0 && items == NULL))
        return CB_ERROR;
```

```
    for (size_t i = 0; i < count; ++i) {
        enum cb_result result = fn(items[i], ctx);

        if (result == CB_STOP)
            return CB_STOP;
        if (result != CB_OK)
            return CB_ERROR;
    }

    return CB_OK;
}

static int compare_ints(const void *left, const void *right)
{
    const int *a = left;
    const int *b = right;

    return (*a > *b) - (*a < *b);
}

struct event_slot {
    callback_fn fn;
    void *ctx;
    int active;
};

static enum cb_result event_slot_set(
    struct event_slot *slot, callback_fn fn, void *ctx)
{
    if (slot == NULL || slot->active)
        return CB_ERROR;

    slot->fn = fn;
    slot->ctx = fn != NULL ? ctx : NULL;
    return CB_OK;
}

static enum cb_result event_slot_emit(struct event_slot *slot, int value)
{
    if (slot == NULL || slot->active || slot->fn == NULL)
        return CB_ERROR;

    slot->active = 1;
    enum cb_result result = slot->fn(value, slot->ctx);
    slot->active = 0;

    if (result != CB_OK && result != CB_STOP)
        return CB_ERROR;
    return result;
}
```

```c
int main(void)
{
    int values[] = { INT_MAX, -3, 0, INT_MIN, 4 };
    size_t count = sizeof values / sizeof values[0];

    qsort(values, count, sizeof values[0], compare_ints);

    struct counter visitor_state = { 0, 2, 0 };
    enum cb_result result =
        walk_ints(values, count, accept_limited, &visitor_state);

    if (result != CB_STOP || visitor_state.accepted != 2)
        return EXIT_FAILURE;

    struct counter event_state = { 10, 1, 0 };
    struct event_slot slot = { 0 };

    if (event_slot_set(&slot, accept_limited, &event_state) != CB_OK)
        return EXIT_FAILURE;

    result = event_slot_emit(&slot, 12);

    if (event_slot_set(&slot, NULL, NULL) != CB_OK)
        return EXIT_FAILURE;

    if (result != CB_OK || event_state.accepted != 1)
        return EXIT_FAILURE;

    return EXIT_SUCCESS;
}
```

A kiválasztott profilra beállított fordítókkal a külön tanulói labor buildparancsai:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -Wstrict-prototypes \
    -O0 -g callbacks.c -o callbacks-gcc

clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion -Wstrict-prototypes \
    -O0 -g callbacks.c -o callbacks-clang
```

A rendezésből következő várt sorrend `INT_MIN, -3, 0, 4, INT_MAX`. A visitor az első két értéket átugorja, majd a `0` és `4` elfogadásával eléri a kettes korlátot. Az `INT_MAX` feldolgozásakor megállást kér.

Fontos különbség: a korlát **elérése** még nem állítja le azonnal ezt a callbacket. A következő elfogadható érték váltja ki a `CB_STOP` eredményt. Emiatt pontosan két elfogadható elemet tartalmazó bemenetnél a bejárás `CB_OK` eredménnyel is befejeződhet.

A számláló nem csordul túl: növelés csak akkor történik, ha

$$\texttt{accepted} < \texttt{limit}.$$

Így növelés után $\texttt{accepted} \leq \texttt{limit}$, amely eleve ábrázolható `size_t` érték.

6.5.3.1 A nem ellenőrizhető előfeltételek

A nullvizsgálatok nem általános pointerérvényesség-vizsgálatok. A hívó további kötelezettségei:

- `items` legalább `count` olvasható `int` elemet jelöl;
- a callback nem érvényteleníti és nem módosítja a bejárt tömböt;
- az `accept_limited` contextje élő, megfelelően igazított `struct counter`;
- a slot használat előtt inicializált;
- a callback normál visszatéréssel fejeződik be.

A hibás típusú vagy dangling contextből nem garantáltan lesz `CB_ERROR`: az ilyen hozzáférés UB lehet. A callback korábbi mellékhatásait sem `CB_STOP`, sem `CB_ERROR` nem vonja vissza. Ez az API nem tranzakció.

6.5.4 A `qsort` comparatorának követelményei

ISO C17 könyvtári szerződés: a `qsort` comparatorának negatív, nulla vagy pozitív eredménye rendre kisebb, egyenértékű vagy nagyobb elemet jelent. Nem szükséges pontosan `-1`, `0`, `1` értékeket visszaadni.

Az alábbi gyakori megoldás hibás:

```
/* Hibás C17 kódrészlet: signed túlcsordulás miatt UB lehet. */
/* return *a - *b; */
```

Például az `INT_MAX - (-3)` matematikai eredménye nem ábrázolható `int` típussal.

A helyes változat csak relációs eredményeket von ki:

$$c(a, b) = (a > b) - (a < b).$$

Bizonyításvázlat: a relációs operátorok `int` típusú `0` vagy `1` eredményt adnak. Ha $a < b$, az eredmény $0 - 1 = -1$; egyenlőségnél $0 - 0 = 0$; ha $a > b$, akkor $1 - 0 = 1$. Mindhárom ábrázolható, az eredeti számok kivonása elmarad.

A comparatornak következetes rendezési viszonyt kell megadnia. Ne módosítsa az összehasonlított elemeket, és ne változtassa a sorrendet számláló, véletlen vagy idő alapján. A rendező mozgatja az elemeket, ezért az aktuális tárolási cím nem megfelelő kulcs egy logikai rekord azonosítására. Egyenértékű elemek eredeti sorrendjének megőrzését a `qsort` nem garantálja.

A szabványos C17 `qsort` nem kap contextet, és nincs külön comparator-hibacsatornája. A nulla nem hiba, hanem egyenértékűség. Contextes platformváltozatok léteznek, de nevüket, paramétersorrendjüket és támogatásukat külön kell ellenőrizni.

6.5.5 Reentrancia és callback alatti invalidáció

A reentrancia (*reentrancy*) azt jelenti, hogy egy művelet újra meghívható, miközben egy korábbi végrehajtása még nem fejeződött be. Ez egyszálú programban is előfordulhat: a callback közvetve újabb eseménykibocsátást indít.

A `walk_ints` nem használ közös globális állapotot. Független adatokkal és contexttel egymásba ágyazható. Ugyanannak a módosuló contextnek az újrahasználatához viszont külön alkalmazásszintű szabály szükséges. A globális változóval pótolt `qsort`-context ezért törékeny: egy belső rendezés felülírhatja a külső konfigurációját.

Az eseményslot más döntést hoz: az `active` jelzővel megtiltja ugyanazon slot visszatérő kibocsátását és callback alatti átregisztrálását. Ilyenkor `CB_ERROR` keletkezik.

Ez csak az API-n keresztüli módosításokat védi. A callback nem írhatja közvetlenül át a slot mezőit, nem szüntetheti meg a slot vagy context élettartamát, és nem kerülheti meg a normál visszatérést. Az `active` közönséges `int`, tehát ez **egyszálú szerződés**, nem szálbiztonsági mechanizmus.

Általános eseménykezelőben a leiratkozás önmagában nem feltétlenül jelenti, hogy nincs már folyamatban hívás. Contextet csak akkor szabad megszüntetni, amikor új hívás már nem indulhat, és minden korábbi befejeződött. A példabeli szinkron, egyszálú modellben az inaktív állapotban sikeres leiratkozás ezt biztosítja.

6.5.5.1 Függvénypointer nem objektumpointer

ISO C17 hordozhatósági határ: az objektumpointer `void *`-rá, majd eredeti objektumpointer-típusára történő konverziójának garanciája nem általánosítható függvénypointerre.

Ezért a függvénycímet ne rejtsd a contextbe. A példában külön `callback_fn` és `void *` mező szerepel. Egyenlő gépi méretük egy adott ABI-n nem bizonyítja a konverzió hordozhatóságát. Ha egy platform API-ja külön garanciát ad, az platformfüggő szerződés, nem általános ISO C szabály.

6.5.6 Gyakori hibák és javításuk

Hibaminta	Következmény	Javítás
Inkompatibilis callback castolása	A hívás UB lehet	Megfelelő prototípusú adapterfüggvény
Lokális context megőrzése a visszatérés után	Dangling hozzáférés	Hosszabb élettartam vagy időben történő leiratkozás
Nem null pointer automatikusan érvényesnek tekintése	Típus- vagy élettartamhiba	Explicit hívói előfeltételek
Context felszabadítása callback közben	Aktív vagy későbbi hozzáférés sérülhet	Dokumentált leállítási és felszabadítási sorrend
Comparatorban kivonás	Signed túlcsordulás	Relációs eredmények különbsége
Hibajelzés után teljes visszagörgetés feltételezése	Részlegesen módosított állapot	Mellékhatások és rollback külön szerződése

6.5.7 Feladatok

6.5.7.1 Élettartam értelmezése

Egy függvény lokális `struct counter` objektum címét regisztrálja egy nála hosszabb életű eseményslotba, majd visszatér. Miért hibás ez akkor is, ha a regisztráció sikeres?

6.5.7.2 Callback készítése

Írj a példához illeszkedő callbacket, amely csak számlálja a negatív értékeket. Ha a számláló már `SIZE_MAX`, adjon `CB_ERROR` eredményt változtatás nélkül.

6.5.7.3 Reentrancia diagnosztikája

A slot callbackje ugyanazon slotra ismét meghívja az `event_slot_emit` függvényt. Mi történik? Mi változna, ha a külső callback `longjmp` segítségével hagyná el a hívási láncot?

6.5.8 Megoldások

6.5.8.1 Élettartam értelmezése

A regisztráció pointerértéket másol, nem contextobjektumot. A lokális objektum élettartama a függvényből visszatérve véget ér; a későbbi callback nem férhet hozzá.

Javításként a context lehet a hívó hosszabb életű objektuma, vagy a regisztrációt még az élettartam vége előtt meg kell szüntetni. Az ellenőrzési feltétel: minden lehetséges callback-hívás teljes egészében a context élettartamába essen.

6.5.8.2 Callback készítése

A következő ISO C17 részlet a példabeli típusdefiníciók után illeszthető be:

```
#include <stdint.h>

struct negative_count {
    size_t value;
};

static enum cb_result count_negative(int value, void *ctx)
{
    if (ctx == NULL)
        return CB_ERROR;

    struct negative_count *state = ctx;

    if (value >= 0)
        return CB_OK;

    if (state->value == SIZE_MAX)
        return CB_ERROR;

    ++state->value;
    return CB_OK;
}
```

A hívó tulajdonában álló contextnek élő `struct negative_count` objektumot kell jelölnie. Ellenőrzési esetek: nemnegatív bemenetnél nincs változás; negatív bemenetnél `SIZE_MAX - 1` értékről még megengedett a növelés; `SIZE_MAX` esetén hiba történik, módosítás nélkül.

6.5.8.3 Reentrancia diagnosztikája

A belső kibocsátás `active != 0` miatt `CB_ERROR` eredménnyel visszatér, újabb callback nélkül. Ha a külső callback normálisan visszatér, az eredeti kibocsátás nullázza az `active` mezőt.

A `longjmp` átugorhatná ezt a nullázást, ezért sérti a callback normál visszatérését előíró szerződést. Ellenőrzési feltétel: minden engedélyezett callback-kimenet után végrehajtódjon az aktív állapot megszüntetése.

6.5.9 Összefoglaló

A C callback interfésze függvénytípusból, contextből és viselkedési szerződésből áll. A helyességhez kompatibilis hívást, megfelelő contexttípust, elegendő élettartamot és egyértelmű hibajelzést kell biztosítani. Tárolt regisztrációnál a leiratkozás és a folyamatban lévő hívások befejezése különösen fontos. A cast nem javít típushibát, a nullvizsgálat pedig nem bizonyít érvényességet.

7 Tömbök, sztringek és nem tulajdonló adatszeletek C-ben

7.1 Tömbobjektum, decay és függvényparaméter

C-ben a tömbobjektum ismeri a méretét a típusán keresztül, egy rá mutató pointer azonban önmagában nem hordoz elemszámot. Ez a különbség különösen függvényhatáron fontos: az `int a[]` paraméter nem „mérettel együtt átadott tömb”, hanem pointerparaméter. A biztonságos interfésznek ezért külön kell közölnie, hány elemhez szabad hozzáférnie.

Az alábbi nyelvi szabályok az **ISO C17** alapra vonatkoznak. A Pointeraritmetika, objektumhatár és provenance részben megismert fél-nyílt tartományokat, valamint a Deklarátorok: tömb-, pointer- és függvénykapcsolatok típusolvasási módszerét használjuk: először mindig azt döntjük el, hogy az adott objektum tömb vagy pointer, és csak utána vizsgáljuk a kifejezésbeli viselkedését.

7.1.1 A tömbobjektum és inicializálása

A tömb (*array*) azonos elemtípusú objektumok folytonosan tárolt sorozata. Egy rögzített méretű `T a[N]` deklarációban `N` pozitív egész konstanskifejezés, `T` pedig megfelelő, teljes objektumtípus. Az érvényes indexek tartománya $[0, N)$.

A következő C17-részlet három valódi tömbobjektumot definiál:

```
int a[4] = {3, 7};        /* értékek: 3, 7, 0, 0 */
int b[] = {10, 20, 30};   /* a típus int[3] */
int zeros[5] = {0};       /* mind az öt elem értéke nulla */
```

Az inicializálóból kimaradó elemek itt nulla értéket kapnak. Ez nem azonos az inicializáló teljes elhagyásával: egy blokkban definiált, automatikus tárolási idejű `int values[5];` elemei indeterminate állapotúak; nem feltételezhető róluk, hogy nullák.

A `b[]` üres szögletes zárójele nem dinamikus méretet jelent. A fordító az inicializálóból határozza meg a háromelemes tömbtípust. A nulla hosszúságú tömb nem szabványos C17 tömbobjektum.

Az elemek között nincs külön, tömb által hozzáadott kitöltés. Az elemtípus saját mérete azonban tartalmazhat kitöltést. Az alábbi táblázat ABI-független összefüggéseket mutat; minden méret és eltolás **C bájtban**, vagyis a `sizeof` egységében értendő.

Objektum	Eltolás a tömb elejétől	Méret
`a[0]`	`0`	`sizeof(int)`
`a[1]`	`sizeof(int)`	`sizeof(int)`
`a[2]`	`2 * sizeof(int)`	`sizeof(int)`
`a[3]`	`3 * sizeof(int)`	`sizeof(int)`
Teljes `a` tömb	`0`	`4 * sizeof(int)`

A tömb nem értékadható úgy, mint egy egész változó: két már létező tömb között az `a = other;` nem megengedett tömbmásolás, hanem fordításkor diagnosztizálandó szabálysértés.

7.1.2 Array-to-pointer conversion és kivételei

A tömbkifejezés a legtöbb kifejezéskörnyezetben az első elemére mutató pointerértékké konvertálódik. Ennek neve **array-to-pointer conversion**, gyakori informális neve *decay*.

```
/* C17-részlet, az előző a tömb deklarációja után. */
int *p = a;
```

Itt két külön objektum létezik:

- `a` négy `int` elemet tartalmazó tömbobjektum;
- `p` pointerobjektum, amely `a[0]` címét tárolja.

Nem maga a tömb változik pointerré. Az `a` kifejezésből keletkezik pointerérték; sem elemmásolás, sem tulajdonátadás nem történik. A `p` kölcsönzött hozzáférés, amely csak `a` élettartama alatt használható az elemek elérésére.

7.1.2.1 A konverzió kivételei

C17-ben különösen fontosak a következő esetek:

- A `sizeof` közvetlen tömboperandusa nem konvertálódik pointerré.
- Az egyoperandusú `&` operandusaként a tömb megőrzi tömbtípusát.
- A tömb inicializálására használt sztringliterál nem pointerként inicializál: például a `char text[] = "ok";` háromelemes karaktertömböt hoz létre, a lezáró nullkarakterrel együtt.

A C17 konverziós szabályának kivétellistájában az `_Alignof` is szerepel. Ennek szabványos C17-szintaxisa azonban típusnevet fogad: `_Alignof(int[4])` megengedett, `_Alignof a` nem szabványos C17 kifejezésalak.

Az előző deklarációk mellett:

Kifejezés	Jelentés
`a` a `p = a` értékadásban	Az első elemre mutató `int *` érték
`sizeof a`	A teljes tömb mérete
`sizeof p`	A pointerobjektum mérete
`&a`	Az egész tömbre mutató `int (*)[4]` érték
`sizeof(a + 0)`	Pointerméret, mert az összeadásban már megtörténik a konverzió

Az `a + 1` egy `int` elemmel lép előre. Az `&a + 1` egy teljes `int[4]` tömbbel lép tovább, és az egész tömb utáni one-past pointert adja. A típus tehát meghatározza a lépés egységét.

7.1.3 Elemszám, indexelés és határok

Valódi, teljes tömbtípusú `a` objektumra:

$$\text{sizeof}(a) = N \cdot \text{sizeof}(a[0]).$$

Mivel egy objektumtípus mérete pozitív:

$$N = \frac{\text{sizeof}(a)}{\text{sizeof}(a[0])}.$$

Bizonyításvázlat: a tömb pontosan N folytonosan elhelyezett elemet tartalmaz; minden elem mérete `sizeof a[0]`. A teljes méret ezért ezek összege, az osztás pedig visszaadja az elemszámot.

```
/* C17-részlet; size_t miatt szükséges: #include <stddef.h> */
size_t count = sizeof a / sizeof a[0];
```

Ez a módszer **csak valódi tömbön** határozza meg az elemszámot. A `sizeof p / sizeof p[0]` lefordulhat, de pointerméretet oszt elemmérettel. Az eredménynek nincs általános kapcsolata az elérhető elemek számával.

Rögzített méretű tömbnél a fenti `sizeof` operandusai nem értékelődnek ki; a `sizeof a[0]` nem olvassa ki az első elem értékét. A változó hosszúságú tömbök külön szabályait a Többdimenziós tömbök, VLA és layout rész tárgyalja.

Az indexelés definíció szerint pointeraritmetikára épül:

$$p[i] \equiv *(p + i).$$

Ha `p` az első elemre mutat, és `n` elem használható, akkor dereferáláshoz $0 \le i < n$ szükséges. A `p + n` lehet érvényes one-past érték, de nem dereferálható. A folytonos fizikai tárolás nem teszi megengedetté a tömbhatár átlépését.

7.1.4 Tömbparaméter: deklarációs adjustment

A függvényparaméter tömbalakja külön szabály alapján módosul pointerparaméterré. Ez a **parameter adjustment**, nem egy függvényen belüli tömbobjektum későbbi konverziója.

A következő C17-prototípusok ugyanazt a függvénytípust deklarálják:

```
void process(int a[]);
void process(int a[10]);
void process(int *a);
```

A függvény tehát pointerértéket kap. Nem jön létre tízelemű helyi tömb, és nem érkezik rejtett elemszám. A függvénytörzsben a `sizeof a` pointerméretet ad.

A deklarációban szereplő `[10]` önmagában még azt sem követeli meg, hogy legalább tíz elem legyen elérhető. Erre külön paraméteralak szolgál.

7.1.4.1 `const` és `static` a paraméterben

Paraméteralak	Módosított paramétertípus	Jelentés
`int a[]`	`int *a`	Az elemek írhatók
`const int a[]`	`const int *a`	Az elemek ezen a hozzáférésen át nem írhatók
`int a[const]`	`int *const a`	A definíció törzsében a helyi pointer nem értékadható újra
`int a[static 8]`	`int *a`	Minden híváskor legalább nyolc elemhez kell hozzáférést biztosítani

A `const int a[]` nem teszi az eredeti tömböt minden hozzáférésen át változtathatatlanná: más, megfelelő pointeren keresztül továbbra is módosítható lehet.

A szögletes zárójelen belüli `const` viszont a pointerparamétert minősíti. A függvénytípus kompatibilitásánál ez a felső szintű minősítés nem különbözteti meg az interfészt.

A `[static 8]` itt nem tárolási időt jelent. A hívó minimumkapacitási vállalása, amelyre a fordító is támaszkodhat. Nem futásidejű ellenőrzés: null pointer vagy túl rövid tartomány átadása megsérti a szerződést, és nem definiált viselkedéshez vezet. A `const int a[static 8]` a minimumkapacitást csak olvasható hozzáféréssel kombinálja.

Ha a típussal valóban meg akarjuk őrizni a négyes kiterjedést, használható az `int (*a)[4]` paraméter. Ez pointer egy négyes tömbre; a `sizeof *a` tömbméretet ad. Ettől azonban a pointer érvényességét továbbra is külön biztosítani kell.

7.1.5 Explicit hosszparaméteres API

Az általános tömbinterfész két adatot kap: kezdőpointert és elemszámot. A hossz nem bájtméret, hanem az adott elemtípusban értelmezett darabszám.

A következő teljes program minimum **C17**-et igényel. A laborhoz előírt profil: Linux, x86-64 System V ABI, LP64, a `lab-next` profil (profiltábla). A program nem támaszkodik ezek konkrét pointer- vagy `int`-méretére.

```c
/* array_api.c */
#include <stdbool.h>
#include <stddef.h>
#include <stdio.h>

static bool int_fill(int *a, size_t n, int value)
{
    if (a == NULL && n != 0) {
        return false;
    }

    for (size_t i = 0; i < n; ++i) {
        a[i] = value;
    }
    return true;
}

int main(void)
```

```
{
    int values[4] = {3, 7};
    int *p = values;
    size_t n = sizeof values / sizeof values[0];

    if (!int_fill(p, n, 9)) {
        return 1;
    }

    for (size_t i = 0; i < n; ++i) {
        if (values[i] != 9) {
            return 1;
        }
    }

    if (!int_fill(NULL, 0, 9)) {
        return 1;
    }

    printf("count=%zu array_bytes=%zu pointer_bytes=%zu\n",
           n, sizeof values, sizeof p);
    return 0;
}
```

Fordítás a külön tanulói laborban:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O0 -g \
    array_api.c -o array_api-gcc

clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O0 -g \
    array_api.c -o array_api-clang
```

Ezek elvégzendő ellenőrzések, nem dokumentált fordítási vagy futtatási eredmények. A nyelvi szabályokból következő elvárt elemszám négy; a kiírt bájtméretek a célplatformtól függenek.

Az API szerződése:

Szerep vagy feltétel	Követelmény
Tulajdonos és tároló	A hívó kezeli a tömb élettartamát; itt `main` helyi `values` objektuma biztosítja a tárolót
Kölcsönzött hozzáférés	`p` és a függvény `a` paramétere nem tulajdonos
Nem üres tartomány	Legalább `n` élő, írható `int` elem legyen elérhető
Üres tartomány	`n == 0` esetén a null pointer is elfogadott
Érvénytelenítés	A tároló élettartamának vége megszünteti az elemekhez való hozzáférés érvényességét
Hibakimenet	Null pointer és pozitív hossz esetén `false`, módosítás nélkül

A függvény nem tárolja el a pointert. A kitöltés módosítja az értékeket, de nem érvényteleníti az elemekre mutató pointereket.

Korlát: a függvény nem tudja ellenőrizni, hogy a nem null pointer mögött ténylegesen van-e `n` elem. A túlbecsült hossz továbbra is szerződéssértés és határon túli íráshoz vezethet.

A ciklus minden belépéskor `i < n` feltételt ellenőriz, ezért igaz hossz esetén csak érvényes indexet használ. Üres tartománynál sem dereferencia, sem pointeraritmetika nem történik. A legrosszabb műveletszám pozitív `n` mellett $O(n)$, üres tartománynál $O(1)$; nincs allokáció vagy amortizált költség. A hozzáférés szekvenciális, ezért jó térbeli lokalitású.

7.1.6 Gyakori hibák

A „tömb egy pointer” rövidítés eltakarja a lényegi különbséget: a tömb elemeket tartalmaz, a pointer egy hozzáférési értéket tárol. Emiatt a függvényparaméteren végzett `sizeof` nem alkalmas elemszámlálásra.

A másik tipikus hiba a `<= n` ciklusfeltétel. Az utolsó elem indexe `n - 1`, nem `n`; üres tartománynál pedig még a `n - 1` kiszámítása is kerülendő. A természetes bejárási alak `i < n`.

Végül a `static` paraméteralak nem helyettesíti a bemenetellenőrzést. Erősebb előfeltételt ad, nem biztonsági hálót.

7.1.7 Feladatok

7.1.7.1 Típus és méret

Adott `int a[6];` és `int *p = a;`. Mit jelent a `sizeof a`, `sizeof p`, `sizeof(a + 0)` és `&a`? Melyikből határozható meg az elemszám?

7.1.7.2 Ellenőrzött elemírás

Írj `bool int_set(int *a, size_t n, size_t index, int value)` függvényt! Hibás index vagy null pointer esetén adjon `false` értéket, módosítás nélkül. Egyébként pontosan egy elemet írjon át.

7.1.7.3 Hibás tömbparaméter diagnózisa

Az alábbi C17-részlet hibás API-megvalósítás; a `size_t` deklarációjához a `<stddef.h>` szükséges:

```
void clear(int a[8])
{
    size_t n = sizeof a / sizeof a[0];

    for (size_t i = 0; i <= n; ++i) {
        a[i] = 0;
    }
}
```

Azonosítsd a két független méretkezelési hibát! Megoldaná-e őket a `[static 8]` használata?

7.1.8 Megoldások

7.1.8.1 Típus és méret

A `sizeof a` értéke `6 * sizeof(int)`. A `sizeof p` pointerméret; a `sizeof(a + 0)` szintén az `int *` típus mérete. Az `&a` típusa `int (*)[6]`.

Az elemszámot a `sizeof a / sizeof a[0]` adja. Ellenőrzési feltétel: ez minden megfelelő C17-implementáción hat, függetlenül a pointermérettől.

7.1.8.2 Ellenőrzött elemírás

A teljes program fejléceivel használható C17-részlet:

```
static bool int_set(int *a, size_t n,
                    size_t index, int value)
{
    if (index >= n || a == NULL) {
        return false;
    }

    a[index] = value;
    return true;
}
```

A hívó a tulajdonos; a pointer csak a hívás idejére kölcsönzött. A sikeres ág előfeltétele a megadott hosszúságú, élő és írható tartomány. A tároló élettartamának vége érvényteleníti a hozzáférést.

Ellenőrizendő: `index == n` esetén nincs írás; `n == 0` mindig hibát ad; érvényes indexnél kizárólag a kijelölt elem változik.

7.1.8.3 Hibás tömbparaméter diagnózisa

Az első hiba, hogy `a` pointerparaméter: a méretarány nem nyolcat, hanem pointerméret és elemméret hányadosát számolja. A második hiba a `<=`: még helyes elemszám esetén is eggyel túl sok elemhez férne hozzá.

A `[static 8]` egyik hibát sem javítja. Az előfeltételt erősítené, de a paraméter továbbra is pointer maradna.

A javítás külön `size_t n` paraméter és `i < n` feltétel; például használható az `int_fill(a, n, 0)`. Ellenőrzési feltétel: igaz hossz mellett pontosan `n` elem módosuljon. Határon túli hozzáférés esetére nincs garantált kimenet vagy garantált összeomlás.

7.1.9 Összefoglaló

A tömbobjektum elemeket és típusbeli kiterjedést jelent; a pointer önmagában nem közöl hosszt. A decay, a paraméter-adjustment és a `sizeof` külön szabályok. Megbízható C API-ban a pointer mellé explicit elemszám, élettartam- és írhatósági előfeltétel, valamint világos hibaszerződés tartozik.

7.2 Többdimenziós tömbök, VLA és layout

Egy mérési táblázat vagy hálózati feldolgozó munkaterület esetén a „kétdimenziós adat” még nem határozza meg a tárolást. Lehet egyetlen folytonos tömbobjektum, de lehet különálló sorokra mutató pointerek táblája is. A két megoldáshoz eltérő pointertípus, indexelési szabály és élettartam-szerződés tartozik.

A Tömbobjektum, decay és függvényparaméter legfontosabb megállapítása itt is érvényes: a tömb nem pointer, bár számos kifejezésben az első elemére mutató pointerré konvertálódik. Többdimenziós tömbnél ez az első elem maga is tömb. A méretszámításokhoz pedig felidézzük a Promóciók, signedness és túlcsordulás szabályát: a `size_t` előjel nélküli aritmetikája körbefordulhat, ezért a szorzást megelőzően kell ellenőrizni a reprezentálhatóságot.

7.2.1 Tömbök tömbje és row-major elrendezés

ISO C17: az `int a[2][3]` deklaráció egy kételemű tömböt hoz létre, amelynek minden eleme háromelemű `int` tömb. A sorok és az egyes sorokon belüli elemek folytonosan helyezkednek el. Ez a sorfolytonos, angolul **row-major** elrendezés: a jobb szélső index változtatásával jutunk a következő skalár elemhez.

C17 deklarációrészlet:

```
int a[2][3] = {
    {10, 11, 12},
    {20, 21, 22}
};
```

Az `a[r][c]` jelentése:

```
*(*(a + r) + c)
```

Az első pointerlépés teljes sort, a második egyetlen `int` elemet lép át. Ha a soronkénti elemszám N, az elemtípus T, akkor a tömb kezdetéhez viszonyított eltolás:

$$\operatorname{offset}(r, c) = r \cdot \operatorname{sizeof}(T[N]) + c \cdot \operatorname{sizeof}(T) = (rN + c)\operatorname{sizeof}(T).$$

Az eltolás egysége **C_byte**, vagyis a `sizeof` egysége. Az indexelés előfeltétele $0 \le r < R$ és $0 \le c < N$, ahol R a sorok száma.

Bizonyításvázlat. Egy sor pontosan N darab $Ttípusú elemből áll, ezért mérete $N\operatorname{sizeof}(T)$. Az r korábbi sor és a kiválasztott sor c korábbi eleme adja a képletet. A tömb nem illeszt külön kitöltést az elemei közé; ha maga T tartalmaz kitöltést, az már része asizeof(T)‘ értékének.

7.2.1.1 Fizikai sorhossz és logikai szélesség

A **stride**, azaz lépésköz azt mondja meg, milyen távol vannak egymástól az egymást követő sorok kezdetei. Mindig közölni kell az egységét. Ebben a fejezetben a `stride` elemszámot jelent.

Egy logikailag háromoszlopos mátrix fizikailag öt elemet is fenntarthat soronként. Ekkor:

$$\text{offset}(r, c) = (rS + c)\,\text{sizeof}(T),$$

ahol S a fizikai sorhossz, C a logikai oszlopszám, és $S \geq C$.

A következő számok **Linux x86-64, System V ABI, LP64, C17** profilra vonatkoznak, ahol `sizeof(int) == 4` és `CHAR_BIT == 8`. Ezek levezetett értékek, nem mérési eredmények.

Tároló	Logikai méret	Stride, elem	Második sor eltolása, C_byte	`[1][2]` eltolása, C_byte	Teljes méret, C_byte
`int a[2][3]`	2×3	3	12	20	24
`int b[2][5]`	2×3	5	20	28	40

A második esetben a sorok utolsó két eleme alkalmazásszinten kihagyott hely. Ez nem a fordító által beillesztett padding: ezek valódi, indexelhető `int` objektumok.

7.2.2 Sorra mutató pointer vagy pointerekre mutató pointer?

Az `a` tömb kifejezésbeli konverziójának eredménye `int (*)[3]`: pointer egy háromelemű `int` tömbre. **Nem `int **`.**

C17 deklarációrészlet, `<stddef.h>` mellett:

```
void clear_rows(size_t rows, int (*a)[3]);

/* Ugyanaz a függvényparaméter-típus: */
void clear_rows(size_t rows, int a[][3]);
```

A háromoszlopos sor típusa meghatározza az `a + 1` lépésméretét. A sorok száma külön szerződés: a hívónak legalább `rows` darab élő, írható sort kell biztosítania.

Más reprezentációt ír le ez a C17-részlet:

```
int first[3] = {1, 2, 3};
int second[3] = {4, 5, 6};
int *rows[2] = {first, second};
```

Itt a `rows` konverziója valóban `int **`. A `rows[r][c]` először kiolvassa az `r`-edik pointerobjektumot, majd azon keresztül éri el az elemet.

Tulajdonság	Folytonos mátrix	Sorpointer-tömb
Tipikus paraméter	`T (*a)[N]`	`T **a`
Sor kezdete	Típusból számított lépés	Tárolt pointerérték

Tulajdonság	Folytonos mátrix	Sorpointer-tömb
Sorhossz	Azonos fizikai sorhossz	Soronként eltérhet
Tároló tulajdonosa	Egy mátrixtárolóért felelős hívó	Pointertábla és sorok tulajdonosai külön rögzítendők
Kölcsönzés érvényessége	A teljes tároló élettartamáig	A pointertábla és minden használt sor élettartamáig

A fenti lokális példában a befoglaló blokk kezeli mindhárom tömb élettartamát; a sorpointerek nem tulajdonolják a sorokat. A blokk elhagyása érvényteleníti a hozzájuk kötött kölcsönzött hozzáféréseket.

Külön dinamikusan allokált soroknál több allokáció, részleges sikertelenség és soronkénti felszabadítás kezelendő; ennek eszközeit a malloc, calloc, free és allokációs szerződés tárgyalja. A pointertábla ugyanakkor folytonos tároló soraira is mutathat: a `T **` önmagában nem bizonyít szétszórt elhelyezést.

Mindkét reprezentáció indexelése állandó műveletszámú, de a pointertáblás változat egy további pointerérték betöltését igényli. A folytonos, soronként bejárt mátrix rendszerint kedvezőbb lokalitást ad; ez teljesítményre vonatkozó várakozás, nem nyelvi garancia.

Gyakori hiba: a mátrixot `(int **)a` kaszttal átadni. A kaszt nem hoz létre sorpointer-objektumokat. Az így végzett `a[r][c]` hozzáférésnek nincs érvényes tárolási szerződése, és UB-hez vezethet. A javítás a megfelelő signature, nem egy újabb kaszt.

7.2.3 VLA és variably modified típusok

A **változó hosszúságú tömb** (*variable length array*, VLA) tömbméretét nem egész konstanskifejezés határozza meg. Mérete például függvényparaméterből származhat.

C17 deklarációrészlet, VLA-támogatással és `<stddef.h>` mellett:

```
void fill(size_t rows, size_t cols, int (*a)[cols]);
```

Az `a` pointerváltozó, nem VLA-objektum, de a típusa VLA-tól függ. Az ilyen típus a **változóan módosított típusok** (*variably modified types*, VM) körébe tartozik. Az `int a[rows][cols]` paraméteralak itt szintén `int (*a)[cols]` alakra igazodik; nem foglal mátrixot és nem ellenőrzi automatikusan a sorok számát.

Szabványprofil	VLA és VM támogatása
ISO C17	Opcionális szolgáltatáscsomag; az `__STDC_NO_VLA__` makró 1 értéke a támogatás hiányát jelzi.
ISO C23	A VM-típusok és az allokált tárolási idejű VLA-k támogatása kötelező; az automatikus tárolási idejű VLA-objektumok támogatása maradhat opcionális.
ISO C23 makrójelentés	Az `__STDC_NO_VLA__ == 1` már az automatikus tárolási idejű VLA-objektumok támogatásának hiányára vonatkozik.

A C23 nyelvi módot és a konkrét képességet külön fordítási próbával kell ellenőrizni. A C17-es tiltó makróvizsgálatot ezért nem szabad változatlan jelentéssel C23-ra átvinni.

A kiértékelt VLA-méretnek pozitívnak kell lennie. Ez a függvénydefiníció paramétertípusában szereplő sorhosszra is lényeges: a nulla méretet **a hívás előtt** kell elutasítani, nem csak a függvénytörzs elején.

7.2.3.1 Miért veszélyes a bemenetről vett automatikus VLA?

Az `int temporary[rows][cols];` deklaráció automatikus tárolási idejű objektumot hoz létre. Az ISO C nem ír elő hozzá vermet, de a szokásos Linux-fordítók jellemzően stackterületet használnak.

Egy reprezentálható méret még nem feltétlenül fér el a rendelkezésre álló stackben. A VLA-létrehozásnak nincs hordozható, `NULL`-lal jelzett sikertelen kimenete. Nem megbízható bemenet esetén ezért külön szükséges:

- a méretek pozitivitásának és reprezentálhatóságának ellenőrzése;
- alkalmazásszintű felső korlát;
- megfelelő tárolási stratégia.

A méretek ellenőrzése után képzett VLA-ra mutató pointer nem azonos egy nagy automatikus VLA létrehozásával.

7.2.4 Ellenőrzött méretszámítás és kidolgozott példa

Teljes fizikai sorokat fenntartó tárolónál legyen:

$$E = RS, \qquad B = E\,\mathrm{sizeof}(T).$$

Itt E az elemszám, B a C_byte-ban mért méret. Pozitív R, S mellett a két biztonságos szorzás feltétele:

$$R \le \left\lfloor \frac{\mathrm{SIZE_MAX}}{S} \right\rfloor, \qquad E \le \left\lfloor \frac{\mathrm{SIZE_MAX}}{\mathrm{sizeof}(T)} \right\rfloor.$$

Az osztásos ellenőrzés azért előzi meg a szorzást, mert a már körbefordult eredményből az eredeti méret nem állítható helyre megbízhatóan.

Az alábbi teljes C17-program VLA/VM-támogatást igényel, de nem hoz létre futásidejű méretű automatikus tömböt. Választott laborprofil: Linux x86-64 System V, LP64, a `lab-next` profil (profiltábla). A fordítási parancsok laborutasítások; a példához itt nem tartozik végrehajtott mérés.

```
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>

#if defined(__STDC_NO_VLA__) && __STDC_NO_VLA__
#error "This C17 example requires VLA/VM support"
#endif

static int checked_bytes(size_t rows, size_t cols,
                         size_t stride, size_t *bytes)
{
```

```
    if (bytes == NULL || rows == 0 || cols == 0 || stride < cols)
        return 0;

    if (rows > SIZE_MAX / stride)
        return 0;

    size_t elements = rows * stride;

    if (elements > SIZE_MAX / sizeof(int))
        return 0;

    *bytes = elements * sizeof(int);
    return 1;
}

static void fill(size_t rows, size_t cols, size_t stride,
                 int (*a)[stride])
{
    for (size_t r = 0; r < rows; ++r)
        for (size_t c = 0; c < cols; ++c)
            a[r][c] = 7;
}

int main(void)
{
    int matrix[2][5] = {{0}};
    size_t bytes;

    if (!checked_bytes(2, 3, 5, &bytes))
        return 1;

    if (bytes != sizeof matrix)
        return 1;

    fill(2, 3, 5, matrix);

    if (printf("%zu %d %d\n",
               bytes, matrix[1][2], matrix[1][4]) < 0)
        return 1;

    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O2 matrix.c -o matrix-gcc
clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O2 matrix.c -o
 ↪ matrix-clang
```

A választott ABI mellett, sikeres kimeneti művelet esetén a **várt**, nem mért kimenet `40 7 0`. Csak a logikai oszlopok módosulnak.

A tulajdonos a `main` blokkja; a tároló a `matrix`, a kölcsönzött hozzáférés a `fill` paramétere. A tároló élettartama lefedi a teljes hívást, és a blokk elhagyása érvénytelenítené a megőrzött pointert. A `fill` sem pointert, sem tulajdonjogot nem őriz meg.

A `checked_bytes` hibára nullát ad, és nem írja a kimeneti objektumot. Nem null `bytes` esetén élő, írható `size_t` objektumra mutató pointer szükséges. A `fill` nem ellenőrző függvény: a hívó garantálja a pozitív méreteket, a helyes fizikai sortípust és a megfelelő írható tárolót. A méretellenőrzés önmagában nem bizonyítja, hogy egy tetszőleges pointer mögött valóban van ekkora objektum.

7.2.4.1 Lapos nézetnél a valódi objektumhatár számít

Valódi egydimenziós tömbben az index $rS + c$. Ha $r < R$, $c < C \leq S$, akkor:

$$rS + c \leq (R-1)S + (S-1) = RS - 1.$$

Ez a határellenőrzés bizonyításvázlata; feltétele, hogy RS reprezentálható, és a tároló legalább ennyi elemet tartalmaz.

Ne következtess ebből arra, hogy egy valódi `int matrix[R][S]` objektumot korlátlanul bejárhatsz a `&matrix[0][0]` pointerrel. Az így kapott `int *` az első sor elemeire mutat; az azon túli pointeraritmetikát nem teszi szabályossá a fizikailag szomszédos következő sor. Tömbök tömbjét sorpointerrel, lapos tárolót lapos pointerrel kezelj.

7.2.5 Feladatok

1. **Levezetés.** A megadott x86-64 laborprofilban `int a[3][4]` esetén mekkora a sorstride C_byte-ban, az `a[2][1]` eltolása és a teljes méret? Miért hibás hozzá az `int **` paraméter?
2. **Kódolás.** A példa `checked_bytes` függvényére építve írj ellenőrzött `set_cell` függvényt valódi egydimenziós `int` tömb logikai mátrixnézetéhez! A kapacitás elemszám legyen. Hibánál ne változzon a tároló.
3. **Diagnosztika.** Egy függvény közvetlenül bemenetről veszi a `rows` és `cols` értékét, deklarál egy `int a[rows][cols]` tömböt, majd `(int **)a` értéket ad tovább. Nevezz meg legalább három egymástól független hibaforrást, és javasolj javítást!

7.2.6 Megoldások

7.2.6.1 Elrendezés és típus

A sorstride $4 \cdot 4 = 16$ C_byte. Az eltolás:

$$(2 \cdot 4 + 1) \cdot 4 = 36 \text{ C_byte}.$$

A teljes méret $3 \cdot 4 \cdot 4 = 48$ C_byte. A megfelelő paraméter `int (*a)[4]`; az `int **` pointerobjektumokat feltételezne a sorok helyén.

Ellenőrzési feltétel: `sizeof a[0] == 16` és `sizeof a == 48` a megjelölt profilon.

7.2.6.2 Ellenőrzött lapos hozzáférés

C17-részlet az előző program `checked_bytes` függvénye után:

```
static int set_cell(int *a, size_t capacity,
                    size_t rows, size_t cols, size_t stride,
                    size_t r, size_t c, int value)
{
    size_t bytes;

    if (a == NULL || r >= rows || c >= cols)
        return 0;

    if (!checked_bytes(rows, cols, stride, &bytes))
        return 0;

    if (bytes / sizeof(int) > capacity)
        return 0;

    a[r * stride + c] = value;
    return 1;
}
```

A hívó tulajdonolja a legalább `capacity` elemű egydimenziós tömböt; a függvény csak a hívás idejére kölcsönzi. A tulajdonos tárolójának megszűnése érvényteleníti a pointert. Sikerre egyet, ellenőrzött hibára nullát ad. Hamis kapacitásadat vagy dangling pointer ellen nem véd.

Ellenőrzési feltétel: `int data[10] = {0};` mellett a $(R, C, S) = (2, 3, 5)$ nézet `(1,2)` cellája a 7-es indexet módosítja; a `(1,3)` kérés hibát ad, változtatás nélkül.

7.2.6.3 Bemeneti VLA diagnózisa

A nulla méret érvénytelen VLA-határ. A túl nagy méret túlterhelheti a tipikus stacktárolást akkor is, ha a méretszámítás reprezentálható. A `size_t`-szorzás külön ellenőrzés nélkül körbefordulhat. Végül az `int **` kaszt hibás reprezentációt állít.

A javítás: ellenőrzött méretszámítás, alkalmazási méretkorlát, megfelelő tárolási stratégia és a tényleges reprezentációhoz illő signature.

Ellenőrzési feltétel: hibás méretnél se VLA-létrehozás, se memóriaművelet ne történjen; érvényes bemenetnél a paramétertípus egyezzen a tároló szerkezetével.

7.2.7 Rövid összefoglaló

A folytonos kétdimenziós C-tömb tömbök tömbje; természetes paramétertípusa sorra mutató pointer, nem `T **`. A helyes hozzáféréshez együtt szükséges a logikai méret, a fizikai stride, a reprezentálható méretszámítás és az élő tároló. A VLA támogatása szabványverzió-függő, használata pedig nem helyettesíti sem a kapacitásellenőrzést, sem az erőforráskorlátokat.

7.3 Nullterminált sztring és a hossz szerződése

Egy hálózati üzenet bájtsorozat, egy C-sztring viszont olyan adat, amelynek végét egy meghatározott érték jelzi. A kettő összekeverése nem pusztán szövegkezelési hiba: határon túli olvasást, csonkolást vagy protokollértelmezési eltérést okozhat.

A Tömbobjektum, decay és függvényparaméter rész alapján a függvénynek átadott pointerből nem állapítható meg a mögöttes tömb mérete. A const pointerek, több szintű indirection és void* rész pedig megmutatta, hogy a `const char *` korlátozza az adott hozzáférésen keresztüli módosítást, de önmagában nem igazolja sem a tároló élettartamát, sem a terminátor jelenlétét. Most ezekből építünk explicit sztringszerződést.

7.3.1 Sztring, hossz és kapacitás

ISO C17: a sztring (*string*) karakterek összefüggő sorozata, amelyet az első nullkarakter lezár; a sztring ezt a lezáró karaktert is magában foglalja. A nullkarakter értéke nulla, szokásos jelölése `'\0'`. A NUL terminátor nem azonos a `'0'` számjeggyel, és nem azonos a null pointer jelölésére használt `NULL` makróval.

Három mennyiséget kell külön követni:

- **Kapacitás, C:** a kezdőcímtől rendelkezésre álló tároló mérete C bájtban.
- **Hasznos hossz, L:** az első NUL előtti C bájtok száma.
- **A sztring által elfoglalt terület:** $L + 1$ C bájt, a terminátorral együtt.

Mivel `sizeof(char) == 1`, egy `char` tömb elemszáma egyben a C bájtokban mért mérete. Ez nem jelenti általánosan azt, hogy egy C bájt nyolc bit: ezt a `CHAR_BIT` értéke határozza meg.

Egy C kapacitású tárolóban elhelyezett sztring alapinvariánsa:

$$0 \le L < C, \qquad \text{data}[L] = 0.$$

Itt a `data[L]` bájt a nullterminátor, azaz a `'\0'` karakter.

A terminátor előtti elemek egyike sem lehet nulla, különben a sztring korábban véget érne. Az üres sztring hossza nulla, de tárolásához akkor is legalább egy C bájt szükséges.

A *karakter* szó itt különösen veszélyes. A bájtok száma nem feltétlenül egyezik meg sem a Unicode-kódpontok, sem a felhasználó által egyetlen jelnek érzékelt egységek számával. A `strlen` nem megjelenített karaktereket számol; ezt a különbséget a Szövegkódolás, locale és UTF-8 C-ben rész bontja ki.

7.3.2 `char` tömb és string literal

Az alábbi, függvénytörzsbe illeszthető részlet ISO C17-kód, külön platformfeltétel nélkül:

```
char name[] = "api";
const char *label = "api";
char field[3] = "api";
```

A három deklaráció eltérő szerződést jelent.

Deklaráció	Tároló és kezelés	Élettartam	Módosíthatóság és sztringállapot
`name`	Önálló, automatikus tömb; a blokk kezeli	A blokk végrehajtásának elhagyásáig	Elemei írhatók; kezdetben NUL-lal lezárt sztring
`label`	Nem tulajdonló pointer egy literál statikus tárolójára	A literál a program teljes futása alatt él	A literál módosítása UB
`field`	Önálló, automatikus tömb; a blokk kezeli	A blokk végrehajtásának elhagyásáig	Elemei írhatók, de nincs bennük terminátor

A `name` inicializálásakor a tömb megkapja az `'a'`, `'p'`, `'i'`, `'\0'` értékeket. A `name[0] = 'A'` megengedett; a `name[3] = '!'` szintén érvényes elemírás, de megszünteti a sztring lezárását.

ISO C17: a `char field[3] = "api";` megengedett inicializálás. Ha nincs hely a terminátornak, az kimarad. A tömb ezért érvényes adat, de nem sztring.

C-ben a `char *p = "api";` deklaráció is megengedett, a literál módosítása azonban nem definiált viselkedés. A `const char *` jobb interfész, mert az ilyen írási kísérleteket fordításkor észlelhetővé teszi.

A tömbre mutató kölcsönzött pointert a tömb élettartamának vége érvényteleníti. A terminátor felülírása másfajta hiba: a tároló és a pointer még érvényes lehet, miközben a sztringszerződés már nem teljesül.

7.3.3 Mit tud, és mit nem tud a `strlen`?

A `<string.h>` fejlécben deklarált `strlen` egy érvényes sztring hasznos hosszát adja vissza `size_t` típussal. A terminátort nem számolja bele.

A függvény **előfeltételezi**, hogy a pointertől indulva elérhető egy olvasható, NUL-lal lezárt sztring. Nem kap kapacitást, ezért nem alkalmas annak ellenőrzésére, hogy egy tetszőleges buffer sztring-e. A fenti `strlen(field)` hívás nem definiált viselkedéshez vezet: a függvény a tömbön belül nem talál terminátort.

A `strlen(name)` kezdetben három, a `sizeof name` négy. Ezzel szemben a `sizeof label` a pointer mérete, nem a literál hossza vagy kapacitása.

A terminátor megkeresése általánosan $O(L)$ idejű bejárás; nem igényel allokációt, és nem módosítja a tárolót. A pontos gépi megvalósítás könyvtár- és targetfüggő. Ha minden hozzáfűzés előtt újra megkeressük az egyre hosszabb sztring végét, sok kis művelet összköltsége négyzetessé válhat. Gyakran ezért érdemes a hasznos hosszt külön állapotként nyilvántartani.

7.3.4 Másolás és hozzáfűzés: a korlát nem automatikus

ISO C17: a `strcpy(dst, src)` átmásolja a forrássztringet a terminátorral együtt. Legyen a forrás hossza S, a cél kapacitása C. A szükséges méretfeltétel:

$$S < C.$$

Emellett a forrásnak érvényes sztringnek, a célnak megfelelően nagy, írható tárolónak kell lennie, és a másolt tartományok nem fedhetik át egymást. A függvény nem kap kapacitást, nem végez szerződés szerinti bounds-ellenőrzést, és nem jelez „kevés hely” hibát.

A `strcat(dst, src)` a cél meglévő terminátorának helyétől másolja be a forrást. Ha a cél jelenlegi hossza D, akkor:

$$D + S + 1 \leq C.$$

Ezt ellenőrző kódban célszerű kivonással megfogalmazni:

$$D < C \quad \text{és} \quad S < C - D.$$

Bizonyításvázlat. A $D < C$ feltétel mellett a $C - D$ kivonás nem csordul alul. Ez a cél jelenlegi végétől rendelkezésre álló hely, amelybe a forrás S bájtjának és egy terminátornak kell befférnie. Így elkerülhető a `D + S + 1` esetleges `size_t` túlcsordulása.

Ez csak akkor valódi védelem, ha D és S már megbízhatóan ismert. Egy lezáratlan bufferre előbb meghívott `strlen` hibáját az utólagos méretellenőrzés nem javítja meg.

7.3.4.1 Miért nem általánosan biztonságos a `strncpy`?

A `strncpy(dst, src, n)` nem egyszerűen „kapacitásbiztos `strcpy`”.

- Ha a forrás terminátora az első `n` karakter között található, a függvény nullákkal tölti fel a célt összesen `n` elemig.
- Ha ott nincs terminátor, pontosan `n` karaktert másol, és **nem tesz hozzá NUL-t**.

A célban tehát legalább `n` írható elem szükséges. A forrásban vagy elérhető terminátornak, vagy legalább `n` olvasható karakternek kell lennie; az átfedés itt sem megengedett.

Ez például nullákkal kitöltött, rögzített szélességű mezők előállítására használható. Általános sztringmásolásnál viszont könnyen lezáratlan eredményt ad, és rövid forrás esetén is `n` célbájtot ír. A végső NUL kézi beírása lezárhatja az eredményt, de önmagában nem teszi észlelhetővé az adatvesztést.

7.3.5 `snprintf`: a visszatérési érték is része a szerződésnek

ISO C17: a `snprintf(dst, cap, format, ...)` legfeljebb `cap` C bájtot ír, a lezáró NUL-t is beleértve. Pozitív kapacitás és nemnegatív visszatérési érték esetén a tárolt eredmény lezárt.

A visszatérési érték `int`, és kétféle jelentése van:

- Negatív érték: kódolási hiba.
- Nemnegatív R: ennyi karaktert írt volna elegendő hely esetén, a lezáró NUL nélkül.

Pozitív kapacitásnál tehát:

$$\begin{aligned} R < 0 &\Rightarrow \text{hiba}, \\ 0 \leq R < C &\Rightarrow \text{teljes eredmény}, \\ R \geq C &\Rightarrow \text{csonkolás}. \end{aligned}$$

A negatív értéket **a `size_t` konverzió előtt** kell kezelni. A `cap == 0` speciális eset: nincs írás, és a célpointer lehet null pointer; a hívás méretigényt számíthat, de nem hoz létre tárolt sztringet.

A korlát kizárólag a célra vonatkozik. A `%s` argumentumának továbbra is érvényes sztringnek kell lennie. A forrás és a cél átfedését sem teszi biztonságossá; a `snprintf(buf, cap, "%s!", buf)` hibás megoldás. Külső adatot pedig `%s` argumentumként adjunk át, ne formátumsztringként.

7.3.5.1 Kidolgozott példa: teljes eredmény vagy üres cél

A következő függvény hiba esetén nem ad tovább részleges konfigurációs értéket: pozitív kapacitásnál üres sztringre állítja a célt.

A hívó birtokolja a forrás és a cél tárolóját; a függvény csak a hívás idejére kölcsönzi őket, pointert nem őriz meg. A `name` érvényes sztring, a `dst` pozitív `cap` esetén legalább ennyi írható elemet jelöl, és a két tartomány nem fed át.

Profil: ISO C17, GNU bővítések nélkül. Választott, itt nem futtatással ellenőrzött laborprofil: Linux x86-64 System V, a `lab-next` profil (profiltábla). A kód nem igényel nyolcbites C bájtot.

```
#include <assert.h>
#include <stddef.h>
#include <stdio.h>
#include <string.h>

enum text_status {
    TEXT_OK,
    TEXT_NO_SPACE,
    TEXT_ENCODING_ERROR
};

static enum text_status
format_endpoint(char *dst, size_t cap,
                const char *name, unsigned port)
{
    if (cap == 0) {
        return TEXT_NO_SPACE;
    }

    int result = snprintf(dst, cap, "node=%s;port=%u",
                          name, port);

    if (result < 0) {
        dst[0] = '\0';
        return TEXT_ENCODING_ERROR;
    }

    if ((size_t)result >= cap) {
        dst[0] = '\0';
        return TEXT_NO_SPACE;
    }

    return TEXT_OK;
}
```

```c
int main(void)
{
    const char name[] = "api";
    char output[32];
    char small[8];

    enum text_status status =
        format_endpoint(output, sizeof output, name, 443u);

    assert(status == TEXT_OK);
    assert(strcmp(output, "node=api;port=443") == 0);
    assert(strlen(output) == 17);

    status = format_endpoint(small, sizeof small, name, 443u);
    assert(status == TEXT_NO_SPACE);
    assert(small[0] == '\0');

    return 0;
}
```

Fordítás a külön laborban, a megfelelő verziójú eszközökkel:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -O0 -g endpoint.c -o endpoint-gcc

clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -O0 -g endpoint.c -o endpoint-clang
```

A várt teljes eredmény 17 hasznos C bájt, ezért legalább 18 elem kell hozzá. Nyolcelemű célban a `snprintf` legfeljebb hét hasznos bájtot tárol, de továbbra is 17-et ad vissza. A wrapper ezt csonkolásként kezeli, majd kiüríti a célt. Ezek levezetett elvárások, nem közölt futtatási eredmények.

A művelet felülírja a cél korábbi tartalmát; annak korábbi hosszadata többé nem használható. A célra mutató pointerek ettől még nem válnak dangling pointerré.

7.3.6 Beágyazott NUL és bináris protokolladat

Tekintsük az alábbi ISO C17-adatdeklarációt:

```c
char packet[] = {'O', 'K', '\0', 'X'};
```

A tömb négy C bájtot tartalmaz, de a `strlen(packet)` eredménye kettő. A kezdőcímen az `"OK"` sztring található; az utána következő `'X'` már nem része annak.

Egy protokoll szempontjából azonban mind a négy bájt lehet jelentős. Ha az egyik komponens explicit hosszal, a másik sztringként kezeli ugyanazt az adatot, eltérő üzenetet értelmezhetnek. Bináris adathoz ezért pointer és hossz együtt szükséges; a NUL ott közönséges adatérték lehet.

Hasonlóképpen a `snprintf` visszatérési értéke sem mindig egyezik meg a cél `strlen`-értékével: például a `%c` formázás nulla értékkel belső NUL-t írhat. A fenti endpoint-formátum ilyen karaktert nem állít elő. Bináris másoláshoz a Memóriamásolás, összehasonlítás és byte view rész ad megfelelő eszközöket.

7.3.7 Feladatok

7.3.7.1 Magyarázó feladat: melyik mennyiség mit jelent?

A `char data[] = {'a', '\0', 'b', '\0'};` deklarációnál add meg a kapacitást, a `strlen(data)` eredményét és a kezdőcímen található sztring tárolóigényét. Megőrzi-e a teljes tömb tartalmát a `strcpy`?

7.3.7.2 Kódolási feladat: elutasító másolás

Írj `bool copy_string(char *dst, size_t cap, const char *src)` függvényt. Kevés hely esetén adjon `false` értéket, és ne módosítsa a célt. A forrás érvényes sztring; pozitív kapacitásnál a cél megfelelően nagy, írható tároló. Átfedés nincs.

7.3.7.3 Diagnosztikai feladat: határeset

Az alábbi ISO C17-részlet nem UB-példa, hanem hibás sikerellenőrzés:

```
char buf[8];
int n = snprintf(buf, sizeof buf, "%s", "12345678");

if (n <= (int)sizeof buf) {
    puts(buf);  /* Tévesen teljes eredménynek minősíti. */
}
```

Mi a visszatérési érték és a tárolt sztring? Javítsd az ellenőrzést úgy, hogy a negatív eredményt is kezelje.

7.3.8 Megoldások

7.3.8.1 A hossz értelmezése

A kapacitás négy, a `strlen` eredménye egy, a kezdőcímen található sztring tárolóigénye kettő C bájt. A `strcpy` csak az `'a'`, `'\0'` értékeket másolja át; a teljes bináris tartalmat nem.

Ellenőrzési feltétel: a tömb méretét, az első sztring hosszát és annak terminátoros méretét külön mennyiségként kell megadni.

7.3.8.2 Elutasító másolás megvalósítása

ISO C17-függvényrészlet, a szükséges headerekkel; külön tesztprogramba illeszthető:

```
#include <stdbool.h>
#include <stddef.h>
#include <string.h>

bool copy_string(char *dst, size_t cap, const char *src)
{
    if (cap == 0) {
        return false;
    }
```

```
    size_t len = strlen(src);
    if (len >= cap) {
        return false;
    }

    for (size_t i = 0; i < len; ++i) {
        dst[i] = src[i];
    }
    dst[len] = '\0';
    return true;
}
```

A `len < cap` feltétel biztosítja a terminátor helyét anélkül, hogy `len + 1` összeadást kellene ellenőrizni. A függvény nem allokál, lineáris költségű, és nem őriz meg kölcsönzött pointert.

Ellenőrizendő az üres forrás egy- és nullaelemű céllal, a pontosan elegendő kapacitás, valamint az egy bájttal kisebb cél változatlansága.

7.3.8.3 A csonkolás felismerése

A visszatérési érték nyolc, a tárolt sztring `"1234567"`. Az egyenlőség már csonkolást jelent.

Helyes ISO C17-ellenőrzési részlet:

```
if (n < 0) {
    /* Kódolási hiba. */
} else if ((size_t)n >= sizeof buf) {
    /* Csonkolás. */
} else {
    puts(buf);
}
```

A teljes eredmény ágába csak nemnegatív, a kapacitásnál szigorúan kisebb érték kerülhet.

7.3.9 Összefoglaló

A C-sztring szerződés: élő, olvasható tárolóban elérhető NUL terminátort feltételez. A kapacitás, a hasznos bájthossz és a szöveges karakterek száma külön fogalom. A `strlen` nem validátor, a `strcpy` és `strcat` nem kapacitásellenőrző, a `strncpy` pedig nem garantál általánosan lezárt eredményt. A `snprintf` korlátozza az írást, de sikerét és csonkolását külön ellenőrizni kell. Bináris adatnál mindig explicit hosszra építs, ne terminátorra.

7.4 Memóriamásolás, összehasonlítás és byte view

A hálózatról vagy fájlból kapott adat nem feltétlenül sztring: tartalmazhat nulla értékű bájtokat, és nincs szükség lezáró karakterre. A nullterminált sztringeknél a tartalom jelöli a véget; egy bináris adatszeletnél ezt külön hossz tárolja. A biztonságos feldolgozás alapja ezért nem egy magányos pointer, hanem **pointer, hossz és élettartam-szerződés együtt**.

Ebben az egységben minden hossz C bájtban, azaz a `sizeof` egységében értendő. Egy C bájt `CHAR_BIT` bit; az itt bemutatott algoritmusok nem igénylik, hogy ez pontosan nyolc legyen. A nyelvi és könyvtári szabályok alapja ISO C17.

7.4.1 A nem tulajdonló byte view szerződése

A *byte view* egy összefüggő bájttartomány nem tulajdonló nézete. Minimális reprezentációja:

```
/* ISO C17; deklarációrészlet, platformfüggetlen. */
#include <stddef.h>

typedef struct {
    const unsigned char *data;
    size_t size;
} byte_view;
```

A struktúra itt csupán két összetartozó adat csomagolása. A `data` a tartomány elejét, a `size` az olvasható C bájtok számát adja meg. Nem tárol kapacitást, és nem felel az adatok felszabadításáért.

A fejezet API-jaiban a következő invariánst használjuk:

- Az üres nézet kanonikus alakja `{NULL, 0}`.
- Nem üres nézetnél `data` érvényes, és tőle kezdve legalább `size` C bájt jogszerűen olvasható.
- A teljes tartomány ugyanazon megfelelő tárolási tartományon belül marad; két szomszédos objektumot nem tekintünk automatikusan egyetlen tömbnek.
- A tároló élettartama lefedi a nézet minden használatát.

A `const` csak az ezen hozzáférési úton történő módosítást tiltja. A tulajdonos más pointeren keresztül továbbra is megváltoztathatja a tartalmat. A nézet ezért nem pillanatfelvétel.

Az objektumreprezentációról tanultak szerint karaktertípuson, így `unsigned char` hozzáférésen keresztül megvizsgálható egy objektum reprezentációja. Ez aliasing szempontból megengedett hozzáférési út, de nem szünteti meg az élettartam- és határkövetelményeket. Egy helyes típusú pointer is lehet dangling vagy tartományon kívüli.

7.4.2 A négy memóriaművelet pontos szerepe

Az ISO C17 `<string.h>` függvényei bájttartományokat kezelnek, nem nullterminált sztringeket. A nulla értékű bájt egyiknél sem lezárójel.

Függvény	Művelet	Átfedési és hozzáférési szerződés
`memcpy(dst, src, n)`	`n` C bájt másolása	A forrás olvasható, a cél írható; a másolt tartományok nem fedhetik egymást
`memmove(dst, src, n)`	`n` C bájt másolása	A forrás olvasható, a cél írható; az átfedés megengedett
`memset(dst, c, n)`	`n` C bájt kitöltése	A cél írható; `c` előbb `unsigned char` értékké alakul
`memcmp(a, b, n)`	Két `n` hosszú tartomány összehasonlítása	Mindkettő olvasható; egymással átfedhetnek

Mindegyiknél a hívó biztosítja a megfelelő méretű, élő tárolót. A függvények nem vesznek át tulajdonjogot, és nem őrzik meg a pointereket a hívás után. A másoló- és kitöltőfüggvény a célpointert adja vissza; ez nem hibajelzés. A megsértett előfeltételhez általában nem definiált viselkedés, nem pedig visszaadott hibakód tartozik.

7.4.2.1 Átfedés: nem optimalizálási részlet

A `memcpy` tiltása a ténylegesen érintett tartományokra vonatkozik. Egyetlen nagy tömb két diszjunkt része másolható vele. Azonos kezdőcím esetén viszont pozitív hossznál teljes az átfedés, ezért még a „saját magára másolás” sem megfelelő `memcpy`-hívás.

A `memmove` eredménye olyan, mintha először egy külön ideiglenes bájttömbbe másolná a forrást, majd onnan a célba. Ez szemantikai modell: nem követel tényleges allokációt. A megvalósítás például választhat megfelelő másolási irányt.

A `memmove` tehát az átfedést kezeli, de **nem javítja ki a hibás hosszt, a dangling pointert vagy az írhatatlan célt**. A karakteres hozzáférés aliasing-engedménye sem írja felül a `memcpy` külön átfedési tilalmát.

7.4.2.2 Nulla hossz és null pointer

ISO C17 könyvtári előfeltétel: a pointerargumentumoknak nulla hossz esetén is érvényes értékeknek kell lenniük. Az alábbiak ezért nem hordozható, szabályos „üres műveletek”:

```
/* HIBÁS ISO C17-részletek: ne használd őket. */
memcpy(NULL, NULL, 0);
memmove(NULL, NULL, 0);
memset(NULL, 0, 0);
memcmp(NULL, NULL, 0);
```

A nulla hossz azt jelenti, hogy nincs másolt, kitöltött vagy összehasonlított bájt; nem törli a pointerargumentumokra vonatkozó követelményeket. Saját API-nk elfogadhat `{NULL, 0}` nézetet, de ekkor **a könyvtári hívás előtt külön ágon kell kezelnie az üres esetet**. Null pointeren még a `+ 0` pointeraritmetika sem megengedett.

7.4.2.3 Összehasonlítás és kitöltés

A `memcmp` az első eltérő bájtpár alapján rendez, a bájtokat `unsigned char` értékként értelmezve. A visszatérési értékből csak az előjelre támaszkodhatunk:

$$r < 0, \qquad r = 0, \qquad r > 0.$$

Nem garantált, hogy az eredmény pontosan `-1` vagy `1`. Különböző hosszúságú szeletek egyenlőségéhez előbb a hosszakat kell összehasonlítani. Egy közös prefix egyezése önmagában nem egyenlőség.

A `memset` bájtokat tölt ki, nem típusos értékeket állít be. A `memset(ints, 1, sizeof ints)` nem egyes értékű `int` elemeket hoz létre. Hasonlóképpen az általános null pointer-reprezentációról nem feltételezhető, hogy minden bitje nulla. Az itt használt `unsigned char` puffer nullázása viszont egyértelmű.

Szokásos lineáris költségmodellben a másolás és kitöltés $O(n)$, a `memcmp` legrosszabb esetben $O(n)$ munka. Az utóbbi korán megállhat; titkok összehasonlítására nem ad állandó idejű működési garanciát. Ezek folytonos hozzáférések, és nem adnak át új tulajdonolt tárolót a hívónak.

7.4.3 Bounds-ellenőrzött részszelet

Legyen a teljes hossz N, a kezdőeltolás o, a kért hossz pedig k. A kívánt matematikai feltétel:

$$0 \le o \le N, \qquad 0 \le k \le N - o.$$

A kézenfekvő `o + k <= N` ellenőrzés veszélyes: a `size_t` előjel nélküli egész típus, ezért az összeadás körbefordulhat.

Bizonyításvázlat. Először ellenőrizzük, hogy $o \le N$. Ekkor az $N - o$ kivonás nem csordul alul. Ha ezután $k \le N - o$, akkor matematikailag $o + k \le N$. Mivel N ábrázolható `size_t` értékként, a megfelelő összeg is ábrázolható lenne — de az ellenőrzéshez ki sem kell számítani.

A pointerképzés csak az ellenőrzések után következhet. Nulla hosszú eredménynél a kanonikus üres nézetet állítjuk elő, így null pointerhez sem adunk eltolást. A háttérszabályokat a pointeraritmetika és objektumhatárok rész rögzíti.

7.4.3.1 Élettartam és invalidáció

Szereplő	Felelősség, élettartam	Érvénytelenítő esemény
Tulajdonos	Gondoskodik a tároló fennmaradásáról	Automatikus tároló blokkjának elhagyása; dinamikus tároló felszabadítása
Tároló	Tartalmazza az elérhető bájtokat	Élettartamának vége; sikeres újraallokáció a régi nézetek számára
`byte_view` objektum	Csak pointert és hosszt tárol	Saját megszűnése a leírót szünteti meg, nem a háttéradatot
Részszelet	Ugyanazt a tárolót kölcsönzi	Ugyanazok a háttértároló-események, mint az eredeti nézetnél

A nézet lemásolása nem hosszabbít élettartamot. Egy lokális tömbre mutató nézet visszaadása ezért hibás. A tartalom átírása önmagában nem érvényteleníti a pointert, de megváltoztatja, amit a nézeten keresztül olvasunk.

7.4.4 Kidolgozott példa: átfedő puffermozgatás

A következő program két API-t definiál:

- A `byte_sub` nem írja a forrásadatot. Az eredményparaméter legyen érvényes, írható `byte_view` objektum, a forrás bájtjaitól különálló tárolóban. Hiba esetén változatlan marad.

- A `byte_copy` írható célpuffert kap, annak kapacitásával. A forrás és cél átfedhet; ezért `memmove`-ot használ. Nulla kapacitásnál a cél lehet null pointer. Hiba esetén nem másol.

Mindkét függvény csak a hívás idejére kölcsönzi a tárolót. A null pointert és bizonyos mérethibákat felismerik, de a hazug hosszt vagy dangling pointert nem: az érvényes háttértároló továbbra is hívói előfeltétel.

```
/* bytes.c - ISO C17, hosted környezet. */
#include <stdbool.h>
#include <stddef.h>
#include <string.h>

typedef struct {
    const unsigned char *data;
    size_t size;
} byte_view;

static bool byte_sub(byte_view src, size_t offset, size_t count,
                     byte_view *out)
{
    if (out == NULL)
        return false;
    if (src.size != 0 && src.data == NULL)
        return false;

    if (offset > src.size)
        return false;
    if (count > src.size - offset)
        return false;

    byte_view result = {NULL, 0};
    if (count != 0) {
        result.data = src.data + offset;
        result.size = count;
    }

    *out = result;
    return true;
}

static bool byte_copy(unsigned char *dst, size_t capacity,
                      byte_view src)
{
    if (src.size > capacity)
        return false;
    if (src.size == 0)
        return true;
    if (dst == NULL || src.data == NULL)
        return false;

    memmove(dst, src.data, src.size);
    return true;
```

```
}

int main(void)
{
    unsigned char buffer[8];
    const unsigned char initial[] = {0x10, 0x00, 0x20, 0x30};
    const unsigned char expected[] = {
        0x10, 0x00, 0x00, 0x20, 0x30
    };

    memset(buffer, 0, sizeof buffer);

    /* Különálló tömbök: nincs átfedés. */
    memcpy(buffer, initial, sizeof initial);

    byte_view all = {buffer, sizeof buffer};
    byte_view part = {NULL, 0};

    if (!byte_sub(all, 1, 3, &part))
        return 1;

    /* Forrás: [1, 4), cél: [2, 5), tehát átfednek. */
    if (!byte_copy(buffer + 2, sizeof buffer - 2, part))
        return 1;

    if (memcmp(buffer, expected, sizeof expected) != 0)
        return 1;

    return 0;
}
```

A tároló tulajdonosi szerepét a `main` tölti be: a `buffer` automatikus tömb a függvény végéig él. Az `all` és `part` csak kölcsönzött hozzáférések. A másolás nem invalidálja őket, de a `part` által megfigyelt bájtokat is megváltoztatja.

Kijelölt példalabor: Linux x86-64, System V ABI, LP64, `CHAR_BIT == 8`; a `lab-next` profil (profiltábla). A kód nem támaszkodik erre a konkrét ABI-ra. Az alábbi fordításokhoz a parancsnevek a megadott kiadásokat jelöljék; a példa itt nincs lefordítva vagy futtatva.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O2 \
    bytes.c -o bytes-gcc
clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O2 \
    bytes.c -o bytes-clang
```

A szerződésekből levezetett várt eredmény nulla kilépési kód, nem mért futtatási eredmény.

7.4.5 Objektumreprezentáció nem egyenlő szerializációval

Egy objektum `sizeof` méretű reprezentációja másolható megfelelő bájtpufferbe, de ettől még nem válik hordozható üzenetté.

A struktúra kitöltőbájtokat tartalmazhat; egész típusok szélessége és bájtsorrendje eltérhet; pointerértékek más folyamatban rendszerint értelmetlenek. A kitöltés ráadásul nem kívánt memóriaadatot

is kiszivárogtathat. Két mezőnként egyenlő struktúra reprezentációja sem szükségképpen azonos, ezért a `memcmp` nem általános struktúraegyenlőség.

A szerializáció külön adatformátumot határoz meg: mezőszélességet, bájtsorrendet, megengedett értékeket és hosszszabályokat. A byte view ezt a már definiált formátumot szállíthatja, de nem definiálja helyettünk. A visszamásolás sem tesz egy tetszőleges bájtsorozatot automatikusan érvényes típusos értékké.

7.4.6 Feladatok

1. **Szerződésmagyarázat.** Egy tízbájtos tömb első nyolc bájtját két pozícióval jobbra mozgatjuk. Melyik másolófüggvény használható? Miért nem válik szabályossá a `memcmp(NULL, NULL, 0)` a nulla hossztól?

2. **Kódolás.** Írj `byte_equal` függvényt két érvényes byte view összehasonlítására. Két üres nézet legyen egyenlő; eltérő hosszúságú nézetek ne legyenek azok.

3. **Diagnosztika.** Mi hibás ebben az ISO C17-részletben? Tegyük fel, hogy `s` egyébként érvényes nézet.

   ```
   /* HIBÁS részlet; a byte_view deklarációját feltételezi. */
   if (offset + count <= s.size) {
       out->data = s.data + offset;
       out->size = count;
   }
   ```

   Vizsgáld a túlcsordulást, az üres nézetet és az eredményparamétert is.

7.4.7 Megoldások

7.4.7.1 Szerződésmagyarázat

A forrás `[0,8)`, a cél `[2,10)`: átfednek, de mindkettő a tömbön belül van. `memmove` használható; `memcpy` nem.

A nulla hossz nem engedélyez null pointerargumentumot a C17 `memcmp` számára. Saját összehasonlítónk az üres esetet könyvtári hívás nélkül kezeli. Ellenőrzési feltétel: a mozgatás után a cél nyolc bájtja egyezzen a forrás **mozgatás előtti** nyolc bájtjával.

7.4.7.2 Egyenlőségvizsgálat

Az előző C17-fájlba illeszthető részlet:

```
static bool byte_equal(byte_view a, byte_view b)
{
    if (a.size != b.size)
        return false;
    if (a.size == 0)
        return true;

    return memcmp(a.data, b.data, a.size) == 0;
}
```

Mindkét nézet háttértárolója a hívás végéig éljen és legyen olvasható; egymással átfedhetnek. A függvény nem módosít adatot, nem tart meg pointert. Hibás nézetre nem ígér hibajelzést. Ellenőrzési esetek: két üres nézet; azonos tartalom; eltérő hossz; azonos hossz mellett egy eltérő bájt; beágyazott nulla bájt.

7.4.7.3 A hibás részszelet javítása

Az `offset + count` körbefordulhat. Például `offset == SIZE_MAX`, `count == 2` esetén az összeg `1`, így átmehet egy nyolcbájtos nézet hibás ellenőrzésén. A későbbi pointeraritmetika már önmagában szabálytalan.

Üres `{NULL, 0}` nézetnél a `NULL + 0` sem megengedett. Végül `out` érvényességét is biztosítani kell: a null pointer felismerhető, de egy nem null, mégis érvénytelen pointer általánosan nem ellenőrizhető.

A kidolgozott `byte_sub` mindhárom problémát kezeli a saját előfeltételei mellett. Ellenőrzési határesetek: `offset == size, count == 0` siker; ugyanott `count == 1` hiba; `offset > size` hiba; üres bemenetből üres eredmény siker.

7.4.8 Összefoglaló

A byte view hozzáférést ír le, nem tulajdonjogot. Helyességéhez együtt kell a valós hossz, az élő tároló és a megengedett hozzáférés. Átfedésnél `memmove` szükséges; üres nézetnél külön ág védi a könyvtári előfeltételeket. A részszelet határát kivonással ellenőrizzük, az objektumreprezentációt pedig nem tévesztjük össze hordozható adatformátummal.

7.5 Szövegkódolás, locale és UTF-8 C-ben

Egy hálózatról érkező adatfolyam oktettekből áll; attól válik szöveggé, hogy egy kódolási szerződés szerint értelmezzük. A C `char` típusa önmagában nem fejez ki ilyen szerződést: tárolhat szövegrészletet, terminátort vagy bináris adatot is.

A Nullterminált sztring és a hossz szerződése rész alapján már tudjuk, hogy a nullterminált sztring és a pointer–hossz pár eltérő interfész. Most ehhez hozzáadjuk a kódolást: egy tartomány lehet memóriahatár szempontjából érvényes, miközben hibás UTF-8-at tartalmaz.

7.5.1 Bájt, kódpont és megjelenített karakter

Az Egész típusok és adatmodellek jelölését követve egy **C bájt** `CHAR_BIT` bitből áll. Az **oktett** pontosan nyolcbites egység. Az UTF-8 oktettsorozatokkal definiált kódolás; a fejezet konkrét példáiban ezért `CHAR_BIT == 8` az előfeltétel.

A **Unicode-kódpont** egy azonosító a `U+0000`–`U+10FFFF` tartományban. A **Unicode skalárérték** olyan kódpont, amely nem tartozik a `U+D800`–`U+DFFF` surrogate tartományba. Érvényes UTF-8 csak skalárértékeket kódol.

A felhasználó által egyetlen karakternek érzékelt egységet gyakran **grafémaklaszterként** (*grapheme cluster*) közelítjük. Ez több kódpontból is állhat, és nem azonos sem a kirajzolt glyph-ek számával, sem a termináloszlopszélességgel.

Szöveg	UTF-8 oktettek hexadecimálisan	Kódpontok száma	Grafémaklaszterek száma
`A`	`41`	1	1
`á`	`C3 A1`	1	1
`a` és utána kombináló ékezet	`61 CC 81`	2	1

Az utolsó két sor megjelenése lehet azonos, a bájtsorozat mégsem az. A bájtonkénti összehasonlítás ezért nem általános nyelvi szöveg-összehasonlítás. A normalizálás és a grafémák teljes kezelése külön Unicode-feldolgozást igényel.

7.5.1.1 Forráskódolás és végrehajtási kódolás

ISO C17: a forrásfájl fizikai kódolása és a közönséges sztringliterálok végrehajtási kódolása (*execution encoding*) külön kérdés. Egy UTF-8-ban mentett forrásfájl önmagában nem garantálja, hogy a benne szereplő `"á"` literál UTF-8 oktetteket eredményez.

A `"\u00E1"` univerzális karakterneve egyértelműsíti a szándékolt karaktert, de a közönséges literál továbbra is a végrehajtási karakterkészlet szabályait követi. Ezzel szemben a C17 `u8"\u00E1"` UTF-8 sztringliterál; nyolcbites C bájt mellett tartalma `C3 A1 00`.

C17-ben az ilyen literál is `char` elemtípusú tömb. A típus tehát nem különbözteti meg az UTF-8 szöveget más `char` adatoktól. A kódolás az API dokumentált szerződésének része marad.

7.5.2 `char`, karakterosztályozás és locale

7.5.2.1 A `<ctype.h>` előfeltétele

ISO C17: a sima `char` előjelessége implementációfüggő. Ha előjeles, egy UTF-8 oktett `char` értékként negatív lehet.

Az `isspace`, `isalpha`, `tolower` és a többi `<ctype.h>` függvény argumentuma csak:

- `unsigned char` típussal ábrázolható érték; vagy
- az `EOF` értéke

lehet. Más argumentum nem definiált viselkedést okoz.

Egy érvényes `text[i]` tömbelem osztályozásának megfelelő alakja ezért `isspace((unsigned char)text[i])`. A konverzió helyreállítja a bájt nemnegatív értékét az egészpromóció előtt.

Ez **nem Unicode-dekódolás**. Egy többoktettes UTF-8 kódolás minden oktettjére külön meghívott `isalpha` nem dönti el, hogy a kódpont betű-e; a bájtonkénti `tolower` pedig nem általános Unicode kisbetűsítés.

Az `EOF` külön figyelmet igényel: például az `fgetc` eredményét először `int` változóban kell megvizsgálni. Az `unsigned char` konverzió után már nem különböztethető meg megbízhatóan a fájlvég egy adatbájttól.

7.5.2.2 Az `LC_CTYPE` globális szerződése

ISO C17: a `<locale.h>` `setlocale` függvénye módosítja a program locale-beállítását. Az `LC_CTYPE` kategória többek között a karakterosztályozásra és a multibájtos konverziókra hat.

Induláskor a szabványos környezet a `"C"` locale-nak felel meg. A `setlocale(LC_CTYPE, "")` a környezet által meghatározott beállítást kéri; sikertelenségkor null pointert ad vissza.

A siker **nem bizonyítja**, hogy a kiválasztott kódolás UTF-8. Az olyan nevek, mint a `"C.UTF-8"`, nem ISO C által garantált locale-nevek; elérhetőségük az adott libc-től és telepítéstől függ.

A globális állapot problémája szolgáltatásokban különösen fontos. Egy könyvtár locale-váltása más komponensek értelmezését is megváltoztathatja. Jó alapelv, hogy az alkalmazás inicializáláskor állítja be a szükséges locale-t, és feldolgozás közben nem változtatja. Egy rögzítetten UTF-8 hálózati protokollhoz gyakran jobb locale-független dekódert választani.

7.5.3 Széles karakterek és állapotot hordozó konverziók

7.5.3.1 A `wchar_t` nem univerzális Unicode-típus

ISO C17: a `wchar_t` széles karakterek tárolására szolgáló egész típus, de mérete és értelmezése implementációfüggő. Sem a neve, sem a `sizeof(wchar_t)` értéke nem bizonyítja, hogy minden eleme egy Unicode-kódpont.

Linux/glibc környezetben gyakori a 32 bites, Unicode-hoz illeszkedő megvalósítás, de ez implementációs tulajdonság, nem hordozható nyelvi garancia.

Header	Konverziós függvények	Kimeneti/bemeneti egység
`<wchar.h>`	`mbrtowc`, `wcrtomb`	`wchar_t`
`<uchar.h>`	`mbrtoc16`, `c16rtomb`	`char16_t`
`<uchar.h>`	`mbrtoc32`, `c32rtomb`	`char32_t`

A multibájtos oldal ezeknél is az aktuális `LC_CTYPE` szerinti kódolás. A `mbrtoc32` tehát nem automatikusan „UTF-8-dekóder”.

C17-ben a `char16_t` és `char32_t` Unicode-értelmezéséhez is ellenőrizni kell az implementáció garanciáit; erre szolgálnak például a `__STDC_UTF_16__` és `__STDC_UTF_32__` makrók. UTF-16 esetén egy nem BMP-beli skalárérték két kódegységet igényel.

7.5.3.2 Az `mbstate_t` a feldolgozás része

Az újraindítható konverziók **konverziós állapotot** tárolnak egy `mbstate_t` objektumban. A `mbstate_t state = {0};` kezdeti konverziós állapotot hoz létre.

Az állapotnak túl kell élnie az összetartozó bemeneti darabok feldolgozását. Külön adatfolyamokhoz külön állapot tartozik. Null állapotpointer helyett célszerű saját objektumot átadni, így nem támaszkodunk a függvény belső, megosztott állapotára.

A bemeneti konverziók fontos eredményei:

Eredmény	Jelentés és teendő
Pozitív hossz	Siker; ennyi C bájtot fogyasztott az aktuális bemenetből
`0`	Null karakter; UTF-8 esetén egy oktettet fogyasztott
`(size_t)-2`	Hiányos szekvencia; az állapot már rögzíti a feldolgozott részt
`(size_t)-1`	Kódolási hiba; az állapot nem használható tovább megalapozatlanul
`(size_t)-3`	Az ezt támogató konverziónál további kimeneti egység, új bemenet fogyasztása nélkül

Az utolsó eset például `mbrtoc16` esetén fontos: egy UTF-16 surrogate pár második egysége későbbi hívásban jelenhet meg.

Kidolgozott lépés: UTF-8 locale mellett a `C3 A1` szekvenciát két darabban kapjuk. Az első, egyoktettes `mbrtoc32` hívás eredménye `(size_t)-2`. Ugyanazzal az állapottal ezután csak az új `A1` oktettet adjuk át. A `C3` ismételt átadása megkettőzné a már feldolgozott bemenetet.

Hibánál az alkalmazásnak kell döntenie elutasításról vagy helyreállításról. Az állapot nullázása önmagában nem határozza meg, melyik bemeneti pozíción biztonságos folytatni.

7.5.4 Locale-független UTF-8 határellenőrzés

Az UTF-8 önszinkronizáló, változó hosszúságú kódolás: egy skalárérték egy–négy oktett. Nem elég azonban a kezdőbájt alapján megszámolni a folytatóbájtokat. El kell utasítani:

- a hibás kezdő- és folytatóbájtokat;
- a szükségesnél hosszabb, úgynevezett *overlong* alakokat;
- a surrogate kódpontokat;
- a `U+10FFFF` fölötti értékeket.

Az alábbi program egyetlen skalárérték kódolását ellenőrzi, majd végigjár egy explicit hosszúságú puffert.

Fordítási profil: ISO C17, GNU bővítések nélkül; választott laborfordító GCC 14.2, Linux x86-64 System V, LP64, a `lab-next` profil (profiltábla). A kód nem függ a locale-tól; a nyolcbites C bájtot fordításkor ellenőrzi. Az alábbi eredmények elvárások, nem dokumentált futtatási eredmények.

```c
#include <limits.h>
#include <stddef.h>
#include <stdio.h>

_Static_assert(CHAR_BIT == 8, "Eight-bit bytes required");

/* 1: valid unit; 0: incomplete; -1: invalid.
   *used changes only on success. */
static int utf8_unit(const unsigned char *p, size_t n,
                     size_t *used)
{
    if (n == 0)
        return 0;
```

```
    unsigned char b = p[0];
    size_t k;

    if (b <= 0x7F) {
        *used = 1;
        return 1;
    }

    if (b >= 0xC2 && b <= 0xDF)
        k = 2;
    else if (b >= 0xE0 && b <= 0xEF)
        k = 3;
    else if (b >= 0xF0 && b <= 0xF4)
        k = 4;
    else
        return -1;

    for (size_t i = 1; i < k; ++i) {
        if (i == n)
            return 0;

        unsigned char c = p[i];
        if (c < 0x80 || c > 0xBF)
            return -1;

        if (i == 1 &&
            ((b == 0xE0 && c < 0xA0) ||
             (b == 0xED && c > 0x9F) ||
             (b == 0xF0 && c < 0x90) ||
             (b == 0xF4 && c > 0x8F)))
            return -1;
    }

    *used = k;
    return 1;
}

int main(void)
{
    const unsigned char data[] = {
        0x41, 0xC3, 0xA1, 0x00, 0x42
    };
    size_t offset = 0;
    size_t count = 0;

    while (offset < sizeof data) {
        size_t used = 0;
        int status = utf8_unit(data + offset,
                               sizeof data - offset, &used);
        if (status != 1) {
            fprintf(stderr, "UTF-8 error at %zu: %d\n",
```

```
                    offset, status);
            return 1;
        }
        offset += used;
        ++count;
    }

    printf("octets=%zu, scalar_values=%zu\n",
           sizeof data, count);
    return 0;
}
```

Fordítás a külön tanulói laborban:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -O2 utf8_units.c -o utf8_units
```

7.5.4.1 Memória- és hibaszerződés

Szerep	Szerződés
Tulajdonos és tároló	A `main` automatikus `data` tömbje
Kölcsönzött nézet	A `p`, `n` pár; a függvény nem tárolja el
Előfeltétel	`n > 0` esetén `p` legalább `n` olvasható C bájtot jelöl
Eredménytároló	`used` élő, írható `size_t` objektumra mutat, a bemeneten kívül
Érvénytelenítés	A tulajdonos blokkjának elhagyása; közben nincs tárolócsere
Hibakimenet	Hiányos vagy hibás kódolásnál nincs előrelépés és nincs új `used` érték

A várt számlálás öt oktett és négy skalárérték. A beágyazott `00` érvényes UTF-8, az `U+0000` kódolása. Nullterminált sztringként ugyanez a tartalom az első három C bájt után véget érne.

Bizonyításvázlat: az ASCII-ág egyetlen oktettet fogad el. A további ágak kizárják az illegális kezdőértékeket, ellenőrzik a folytatóoktetteket, majd a második oktett speciális korlátaival kizárják az overlong alakokat, surrogate értékeket és a felső határ átlépését. A `p[i]` olvasását mindig megelőzi a hosszellenőrzés.

A teljes bejárás $O(n)$ idejű, $O(1)$ kiegészítő tárolót használ, nem allokál, és sorfolytonosan olvas. A segédfüggvény hívásonként legfeljebb négy oktettet vizsgál.

7.5.4.2 Vágás és hálózati darabolás

A `C3 A1` két oktettje között elvágott szöveg első része hiányos UTF-8. Hálózati olvasás végén ez még nem feltétlenül hiba: a következő olvasás hozhatja a folytatást. A logikai üzenet végén viszont már elutasítandó.

A segédfüggvény nem őriz állapotot. Hiányos eredménynél ezért a hívónak kell megtartania a teljes befejezetlen szekvenciát. Ez szándékosan eltér az `mbstate_t`-alapú konverzió szerződésétől.

A skalárérték-határon végzett vágás érvényes UTF-8-at hagyhat, mégis szétszakíthat egy grafémaklasztert. A kódolási érvényesség és a megjelenítési helyesség két külön követelmény.

7.5.5 Feladatok

1. **Értelmezés:** a `61 CC 81 00 42` sorozat hány oktett, skalárérték és első null előtti C bájt? Hány grafémaklaszterből áll a null előtti szöveg?
2. **Kódolás:** a fenti segédfüggvénnyel készíts `utf8_prefix` függvényt, amely előzetesen teljes egészében validált bemenetből megadja a legfeljebb `limit` oktettes, skalárérték-határon végződő prefix hosszát. Ne másoljon és ne allokáljon.
3. **Diagnosztika:** UTF-8 locale mellett egy program a `C3` oktettre `(size_t)-2` eredményt kap. Ugyanazzal az állapottal ezután a teljes `C3 A1` sorozatot adja át újra. Emellett minden bemeneti `char` értékre közvetlenül `isalpha`-t hív. Mi a két külön hiba?

7.5.6 Megoldások

7.5.6.1 Értelmezés

Öt oktett és négy skalárérték található a sorozatban: `U+0061`, `U+0301`, `U+0000`, `U+0042`. Az első null előtti hossz három C bájt. Az `a` és a kombináló ékezet együtt egy grafémaklaszter.

Ellenőrzési feltétel: a bájthossz nem számolható a megjelenített karakterek megszámlálásával.

7.5.6.2 Skalárérték-határon végződő prefix

C17-részlet, a fenti `utf8_unit` definíciója után illeszthető:

```
static size_t utf8_prefix(const unsigned char *p, size_t n,
                          size_t limit)
{
    size_t offset = 0;

    if (limit > n)
        limit = n;

    while (offset < limit) {
        size_t used = 0;
        if (utf8_unit(p + offset, n - offset, &used) != 1)
            break;
        if (used > limit - offset)
            break;
        offset += used;
    }
    return offset;
}
```

A bemeneti tároló tulajdonosa a hívó; a függvény nem tart meg pointert. Az előzetes validálás a szerződés része: az eredmény önmagában nem jelzi, hogy a kihagyott utótag hibás-e.

A `41 C3 A1 42` bemeneten a 2 oktettes korlát eredménye 1, a 3 oktettesé 3. A `limit - offset` kivonás biztonságos, mert a ciklusban `offset < limit`.

7.5.6.3 Állapot és osztályozás

Az első hiba a bemenet megkettőzése: az állapot már feldolgozta a `C3` oktettet, ezért csak az új `A1` oktettet kell átadni.

A második az `isalpha` argumentumtartománya: negatív, `EOF`-tól különböző `char` érték átadása UB. Az `unsigned char` konverzió ezt javítja, de Unicode-betűosztályozást nem valósít meg.

Ellenőrzési feltétel: próbáljuk a szekvenciát minden oktetthatáron darabolni, és külön vizsgáljuk a karakterosztályozás típushelyességét.

7.5.7 Összefoglaló

A szövegkezelési szerződésnek legalább a tároló élettartamát, a bájthosszt, a kódolást és a hibapolitikát rögzítenie kell. A `char` nem graféma, a `wchar_t` nem hordozható Unicode-kódponttípus, a locale pedig rejtett globális bemenet lehet. Hálózati UTF-8 feldolgozásakor a memóriahatár, a skalárértékhatár és a felhasználó által érzékelt karakterhatár külön ellenőrzési szint.

8 Dinamikus memória és explicit tulajdonlás C-ben

8.1 malloc, calloc, free és allokációs szerződés

A dinamikus memóriafoglalás nem egyszerűen „hely kérése a heapen”. A C program szempontjából új tárolási terület keletkezik, amelynek méretét, használatát és felszabadítását a programnak kell helyesen kezelnie. A sikeres foglalás önmagában nem jelenti azt, hogy a tároló már érvényes alkalmazásszintű adatokat tartalmaz.

Ebben az egységben négy külön kérdést követünk: **mekkora tároló szükséges, sikerült-e lefoglalni, mely részei inicializáltak, és ki köteles felszabadítani**. Az egészaritmetikai előfeltételeket a Promóciók, signedness és túlcsordulás, a típusos hozzáférés szabályait az Effective type, aliasing és objektumreprezentáció tárgyalta.

A példák ISO C17-et használnak. A konkrét laborprofil Linux, x86-64 System V ABI, LP64, 8 bites C bájt, a `lab-next` profil (profiltábla). A fordítási parancs reprodukciós útmutató; a példához itt nem tartozik végrehajtott fordítás vagy mérés.

8.1.1 Az allokáció alapvető szerződése

ISO C17. A `<stdlib.h>` fejlécben deklarált `malloc(size)` egy `size` C bájt méretű területet foglal. A C bájt a `sizeof` egysége; bitszáma `CHAR_BIT`, nem általánosan nyolc.

Pozitív méret esetén a visszatérés:

- sikerkor a lefoglalt terület kezdetére mutató pointer;
- sikertelenségkor null pointer, amelyet rendszerint `NULL`-lal hasonlítunk össze.

A `malloc` nem inicializálja az adatokat. A tárolt értékek indeterminate állapotúak: nem tekinthetők használható kezdőértéknek, és típusos olvasásuk nem definiált viselkedéshez is vezethet.

A `calloc(count, element_size)` `count` darab, egyenként `element_size` C bájt méretű elemhez foglal tárolót, majd annak minden bitjét nullára állítja. Pozitív paraméterek mellett itt is ellenőrizni kell a null pointer visszatérést.

A `malloc` és a `calloc` eredménye `void *`, amely C-ben megfelelő objektumpointerré implicit módon konvertálható. Ezért a helyes idiomatikus alak:

```
/* ISO C17; részlet, <stdlib.h> szükséges.
   Előfeltétel: count pozitív, a szorzás nem csordul túl. */
int *p = malloc(count * sizeof *p);
```

Nem szükséges `(int *)` cast. A fejléc viszont kötelező: a cast nem pótolja a függvény megfelelő deklarációját.

A `sizeof *p` a célobjektum méretét fejezi ki. Ha később `p` típusa változik, a méretkifejezés követi. Ebben a példában az operandus nem változó hosszúságú tömbtípus, ezért a `sizeof` nem értékeli ki a `*p` kifejezést: nem történik dereferencia.

A szokásos foglalófüggvények megfelelő igazítást biztosítanak a fundamentális igazítási követelményű típusokhoz, például az itt használt `int` típushoz. Ez nem korlátlan ígéret minden kiterjesztett igazításra; azt az Alignment és speciális allokációk rész tárgyalja.

Operációs rendszer szintje. A sikeres foglalásból nem következik, hogy ugyanennyi fizikai memória azonnal rezidenssé vált. Hasonlóan, a `free` sem ígéri a folyamat RSS-ének azonnali csökkenését. Ezeket ne keverjük össze a C nyelvi élettartam-szerződésével.

8.1.2 Méretszámítás: túlcsordulás és nulla elemszám

Legyen n az elemszám, $s = \mathrm{sizeof}(*p)$ az elemméret C bájtban, $M =$ `SIZE_MAX` pedig a `size_t` legnagyobb értéke. A szükséges méret:

$$B = n \cdot s.$$

Mivel $s > 0$, a szorzat pontosan akkor reprezentálható `size_t` értékként, ha

$$n \le \left\lfloor \frac{M}{s} \right\rfloor.$$

Bizonyításvázlat. A reprezentálhatóság feltétele $ns \le M$. Pozitív s mellett oszthatunk vele; az egész n felső korlátját a hányados egészrésze adja. Emiatt az ellenőrzést szorzás nélkül is elvégezhetjük:

```
/* ISO C17; részlet, <stdint.h> szükséges a SIZE_MAX makróhoz. */
if (count > SIZE_MAX / sizeof *p) {
    /* A kért méret nem reprezentálható. */
}
```

A `malloc(count * sizeof *p)` hívás önmagában nem védi ki a hibát. A szorzás már a függvény meghívása előtt megtörténik. A `size_t` előjel nélküli aritmetikájának körbefordulása definiált, de az így kapott túl kicsi foglalás későbbi túlindexeléshez vezethet.

A megadott x86-64 laborprofilban a `size_t` 64 bites, az `int` mérete négy C bájt. Ekkor:

$$\left\lfloor \frac{2^{64} - 1}{4} \right\rfloor = 2^{62} - 1.$$

A 2^{62} elemszámhoz tartozó szorzat már 2^{64} lenne, amely ebben az előjel nélküli típusban nullára fordul körbe. A számítás helyessége természetesen nem garantálja, hogy egy kisebb, reprezentálható kérés teljesíthető.

Ha az elemszám előjeles külső adatból származik, a negatív értéket **a `size_t`-ra konvertálás előtt** kell elutasítani.

8.1.2.1 Az üres tömb legyen API-döntés

ISO C17. Nulla méretű foglalásnál implementáció által meghatározott, hogy null pointert vagy más eredményt kapunk-e. A visszakapott nem null pointert sem használhatjuk objektum elérésére; megfelelően felszabadítani viszont lehet.

Egyszerűbb saját szerződést választani:

> Nulla elemszám esetén a művelet sikeres, az eredmény `NULL`, és nem történik allokáció.

Ez megkülönbözteti az üres eredményt az allokációs hibától, ezért a függvény külön státuszkódot ad vissza. A reprezentációhoz ne képezzünk automatikusan `NULL + 0` kifejezést sem; üres tartománynál nincs szükség pointeraritmetikára.

A `calloc(count, sizeof *p)` előnye, hogy nem a hívó szorozza össze előre az argumentumokat. A saját méretellenőrzés mégis hasznos: elkülöníti a reprezentálhatatlan méretet az erőforráshiánytól, és egyértelmű API-szerződést ad. A `calloc(1, count * sizeof *p)` alak ezt az előnyt elveszíti.

8.1.3 Teljes példa: foglalás és logikai inicializálás

Az alábbi függvény `initial_value` értékekkel feltöltött tömböt készít. Az `out` egy kölcsönzött hozzáférés a hívó pointerobjektumához.

Előfeltétele, hogy `out` élő, írható `int *` objektumra mutasson, amelynek kezdeti értéke `NULL`. Ez kizárja egy korábban birtokolt foglalás csendes felülírását. Sikertelenségkor az objektum változatlan marad; siker esetén a hívó kapja meg a felszabadítás felelősségét.

Az `allocation.c` teljes tartalma:

```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

enum array_status {
    ARRAY_OK,
    ARRAY_TOO_LARGE,
    ARRAY_NO_MEMORY
};

static enum array_status
int_array_create(size_t count, int initial_value, int **out)
{
    int *p = NULL;

    if (count == 0) {
        return ARRAY_OK;
    }

    if (count > SIZE_MAX / sizeof *p) {
        return ARRAY_TOO_LARGE;
    }

    p = malloc(count * sizeof *p);
```

```
    if (p == NULL) {
        return ARRAY_NO_MEMORY;
    }

    for (size_t i = 0; i < count; ++i) {
        p[i] = initial_value;
    }

    *out = p;
    return ARRAY_OK;
}

int main(void)
{
    const size_t count = 4;
    int *values = NULL;

    enum array_status status = int_array_create(count, 7, &values);

    if (status != ARRAY_OK) {
        const char *message =
            status == ARRAY_TOO_LARGE
                ? "A kert meret nem abrazolhato."
                : "A memoriafoglalas sikertelen.";

        fprintf(stderr, "%s\n", message);
        return EXIT_FAILURE;
    }

    for (size_t i = 0; i < count; ++i) {
        printf("%d\n", values[i]);
    }

    free(values);
    values = NULL;
    return EXIT_SUCCESS;
}
```

Fordítás a megadott profilban:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
    -O1 -g allocation.c -o allocation
```

Sikeres foglalás és zavartalan kimeneti I/O esetén a várt kimenet négy sor, mindegyikben `7`. Ez várt viselkedés, nem közölt futtatási eredmény.

A műveletek sorrendje lényeges. Először ellenőrizzük a méretet, majd tárolót foglalunk. A típusos értékadás minden elembe érvényes `int` értéket ír. A kimeneti pointert csak a teljes inicializálás után állítjuk be.

8.1.3.1 Tároló, effective type és logikai állapot

ISO C17. A dinamikusan foglalt tárolóhoz nem tartozik deklarált típus. Az `int *` pointer deklarálása vagy a foglalási eredmény hozzárendelése önmagában nem inicializálja a tárolót. A példában az `int` lvalue-kon keresztüli írás határozza meg az érintett objektumok effective type-ját az írás és a megfelelő későbbi hozzáférések számára.

Az effective type teljes szabályrendszere ennél részletesebb. Különösen a `memcpy` nem azonos egy tetszőleges bájtonkénti feltöltéssel; ezt az Effective type, aliasing és objektumreprezentáció, illetve a Memóriamásolás, összehasonlítás és byte view kapcsolja össze.

A tulajdonlási és élettartam-szerződés:

Résztvevő	Szerep	Érvényesség és megszűnés
Lokális `p`	Sikeres foglalás után ideiglenes tulajdonos	A függvény visszatéréséig él; a felelősséget előtte átadja
Allokált tároló	Az elemek tárolási területe	Sikeres foglalástól a `free`-ig él
`values`	Sikeres visszatérés után tulajdonos	A `main` blokkban él; egyszer felszabadítja a tárolót
`out`	Kölcsönzött hozzáférés a `values` pointerobjektumhoz	Csak a függvényhívás alatt használjuk, nem tároljuk el
Például `&values[1]`	Lehetséges kölcsönzött elempointer	Csak elegendő elemszám és élő tároló mellett érvényes; a `free` érvényteleníti

Az `out` előfeltételének megsértésére a függvény nem ad külön hibakódot: az szerződésszegés, amely nem definiált viselkedéshez vezethet. A mérethiba és az allokációs hiba viszont szabályosan jelentett hibakimenet.

8.1.4 Mit inicializál a calloc?

A minden bitjén nulla reprezentáció az egész típusoknál a nulla értéket adja. Nem általános viszont, hogy ugyanez null pointert vagy lebegőpontos nullát jelent.

A következő, ISO C17-es függvényrészlet az előző program fejléceit és `enum array_status` definícióját használja. Ugyanaz az `out`-szerződése, de kizárólag nulla értékű `int` elemeket állít elő:

```
static enum array_status
zero_int_array_create(size_t count, int **out)
{
    int *p = NULL;

    if (count == 0) {
        return ARRAY_OK;
    }

    if (count > SIZE_MAX / sizeof *p) {
        return ARRAY_TOO_LARGE;
    }

    p = calloc(count, sizeof *p);
```

```
    if (p == NULL) {
        return ARRAY_NO_MEMORY;
    }

    *out = p;
    return ARRAY_OK;
}
```

Itt a nullázott reprezentáció elegendő a kívánt egész kezdőértékekhez. Más alkalmazásszerződésben nem feltétlenül: ha egy `int` tömb érvénytelen file descriptorait `-1` jelöli, a `calloc` eredménye logikailag hibás kezdőállapot, mert a nulla érvényes descriptorérték lehet.

A nullázás továbbá nem helyettesíti az elemszám, a kapacitás vagy más invariáns nyilvántartását. Ha minden elemet rögtön más értékkel írunk felül, a `malloc` általában világosabban fejezi ki a szándékot. Teljesítménykülönbséget mérés nélkül ne állítsunk.

8.1.5 A free pontos előfeltételei és a gyakori hibák

ISO C17. A `free` argumentuma lehet null pointer; a `free(NULL)` nem végez műveletet. Egyébként olyan pointerérték szükséges, amely megfelelő memóriafoglaló függvényből származik, és az általa jelölt foglalást még nem szabadították fel. A jelen egységben ezek a `malloc` és a `calloc`.

Nem szükséges ugyanazt a pointerváltozót használni, amelybe az eredmény eredetileg került: egy szabályosan másolt kezdőpointer is megfelelhet. A felszabadítás felelősségének azonban egyértelműnek kell maradnia. A C típusrendszere ezt az **egyszeri tulajdonlást** nem kényszeríti ki.

Nem adható át:

- automatikus vagy statikus objektum címe;
- a foglalás belsejére vagy közvetlenül mögé mutató pointer;
- már felszabadított foglalásra hivatkozó pointer.

Az alábbiak **egymástól független, hibás ISO C17-részletek**. Kezdetben `p` legalább két inicializált `int` elem sikeres foglalásának tulajdonosa. Egyikhez sem tartozik garantált kimenet.

```
/* Interior pointer free: nem a foglalás kezdete. */
free(p + 1);

/* Double free: ugyanazt a foglalást másodszor szabadítaná fel. */
free(p);
free(p);

/* Use-after-free: az objektum élettartama már véget ért. */
free(p);
int value = p[0];
```

A `free` nem nullázza a hívó pointerváltozóját. A megszűnt objektumra hivatkozó pointerértéket nem szabad további hozzáféréshez használni; hordozható C17 kódban annak kiolvasására vagy összehasonlítására se építsünk.

A még élő pointerobjektumnak új érték adható:

```
/* ISO C17; részlet. p élő lokális pointerobjektum. */
free(p);
p = NULL;
```

Ez nem javítja meg a többi aliast. A tárolóra korábban kapott minden kölcsönzött hozzáférés érvénytelenné válik.

A másik gyakori hiba a kezdőpointer elvesztése: ha a tulajdonos változót felülírjuk, és másutt nem őrizzük a kezdőcímet, a tároló felszabadíthatatlanná válhat. Ez memóriaszivárgás. A megoldás nem egy utólagos cast, hanem a tulajdonosi felelősség megőrzése.

8.1.6 Feladatok

8.1.6.1 Magyarázó feladat: három külön állapot

Mi a különbség a `malloc` sikeres visszatérése, a `calloc` nullázása és az alkalmazásszintű inicializálás között? Miért kell külön státusz a nulla elemszámú tömbhöz, ha annak reprezentációja `NULL`?

8.1.6.2 Kódolási feladat: hely egy záró bájtnak

Írj `byte_buffer_create(size_t payload, unsigned char **out)` függvényt az előző státuszkódokkal. Foglaljon `payload + 1` C bájtot, és az utolsóba írjon nullát. A payload többi bájtját ne inicializálja.

Az `out` kezdeti értékére ugyanaz a szerződés vonatkozzon, mint korábban. Nulla payload esetén is kell egyetlen bájt.

8.1.6.3 Diagnosztikai feladat: tulajdonos és alias

Független, hibás ISO C17-részletben `base` két inicializált `int` elem foglalásának tulajdonosa:

```
int *cursor = base + 1;
free(base);
base = NULL;
printf("%d\n", *cursor);
free(cursor);
```

Azonosítsd a hibákat, és adj javított műveleti sorrendet. A `cursor` eredetileg kölcsönzött pointer, nem tulajdonos.

8.1.7 Megoldások

8.1.7.1 A foglalás nem kész alkalmazásállapot

A `malloc` tárolót biztosít, de használható kezdőértékeket nem. A `calloc` a reprezentáció bitjeit nullázza. A logikai inicializálás az alkalmazás által megkövetelt érvényes állapot létrehozása; ez adott esetben lehet nulla, máskor például `-1`.

A `NULL` önmagában nem különbözteti meg az üres sikeres eredményt az allokációs hibától.

Ellenőrzési feltétel: a nulla elemszámú hívás eredménye `ARRAY_OK` és `NULL`; az erőforráshiányé `ARRAY_NO_MEMORY`.

8.1.7.2 Az összeadás ellenőrzése

ISO C17-részlet, az előző program fejléceivel és státusztípusával:

```
static enum array_status
byte_buffer_create(size_t payload, unsigned char **out)
{
    if (payload == SIZE_MAX) {
        return ARRAY_TOO_LARGE;
    }

    size_t count = payload + 1;
    unsigned char *p = malloc(count * sizeof *p);

    if (p == NULL) {
        return ARRAY_NO_MEMORY;
    }

    p[payload] = 0;
    *out = p;
    return ARRAY_OK;
}
```

A `sizeof(unsigned char)` definíció szerint egy C bájt, ezért itt az összeadás jelenti a túlcsordulási kockázatot. A payload tartalmát a hívónak kell meghatároznia használat előtt. Siker esetén a hívó a tulajdonos; a `free` minden ide mutató kölcsönzött pointert érvénytelenít.

Ellenőrzési feltétel: `payload == 0` esetén egy nulla bájt keletkezik; `payload == SIZE_MAX` esetén nincs foglalás, a kimenet változatlan. Sikernél a hívó egyszer felszabadítja az eredményt.

8.1.7.3 Az alias nem követi a tulajdonos nullázását

A `free(base)` megszünteti az allokált objektumok élettartamát. A `base = NULL` csak a `base` pointerobjektumot módosítja; a `cursor` nem válik biztonságossá. Dereferálása use-after-free, felszabadítása pedig szintén hibás. Élő foglalás esetén sem lenne felszabadítható, mert belső pointer.

Javított sorrend, azonos előfeltételekkel:

```
/* ISO C17; részlet, <stdio.h> és <stdlib.h> szükséges. */
int *cursor = base + 1;
printf("%d\n", *cursor);

cursor = NULL;
free(base);
base = NULL;
```

Ellenőrzési feltétel: minden elemhozzáférés a felszabadítás előtt történik, és pontosan egy `free` kapja meg a foglalás kezdőpointerét.

8.1.8 Összefoglaló

A biztonságos dinamikus tömbkezelés sorrendje: méretszerződés, túlcsordulás-ellenőrzés, foglalás, hibaellenőrzés, inicializálás, majd egyértelmű tulajdonoshoz rendelés. A `sizeof *p` összeköti a méretszámítást a cél típusával; a `calloc` nullázása nem általános logikai inicializálás.

A `free` null pointerrel megengedett, egyébként élő foglalás kezdőpointerét igényli. A tároló felszabadítása minden hozzá tartozó kölcsönzött hozzáférést érvénytelenít, függetlenül attól, hány pointerváltozó őrzi még a korábbi címet.

8.2 realloc és dinamikusan növekvő buffer

Egy beolvasópuffer végleges mérete gyakran csak feldolgozás közben derül ki. A dinamikus növelés azonban nem pusztán „több memória kérése”: megváltozhat a tároló helye és élettartama, érvénytelenné válhatnak a belső pointerek, az allokáció pedig meghiúsulhat. A helyes megoldás ezért állapotátmeneteket és tulajdonlási szerződést követ.

A malloc, calloc, free és allokációs szerződés alapján a sikeresen lefoglalt tároló felszabadításáért kijelölt tulajdonos felel. A Pointeraritmetika, objektumhatár és provenance szabályaiból pedig azt idézzük fel, hogy egy pointer érvényességéhez nem elegendő a látszólag megfelelő cím: élő tárolóhoz és megengedett tartományhoz kell kapcsolódnia.

8.2.1 A realloc két külön állapotátmenete

ISO C17. A `<stdlib.h>` által deklarált `realloc(p, n)` egy korábbi dinamikus allokációt cserél le `n` C bájt méretű allokációra. A következőkben feltesszük, hogy $n > 0$, és `p` vagy null pointer, vagy egy még fel nem szabadított, megfelelő memóriafoglaló függvénytől származó allokáció kezdőpointere. Belső elemre mutató pointer, automatikus tömb címe vagy már felszabadított pointer nem megfelelő argumentum.

Ha `p == NULL`, a hívás a megadott méretű `malloc` hívásnak megfelelően működik. Nem null `p` esetén két lényegesen különböző kimenet van.

Kimenet, pozitív méretnél	Korábbi tároló	Tulajdonjog	Korábbi belső pointerek
Kudarc: az eredmény `NULL`	Megmarad, tartalma változatlan	A hívónál marad	Továbbra is érvényesek
Siker: az eredmény nem null	A korábbi objektum élettartama véget ér	A visszaadott pointerrel kezelt tárolóért a hívó felel	Érvénytelenné válnak

A biztonságos minta ideiglenes pointert használ. Az alábbi **ISO C17 kódrészletben** `p` az egyedüli tulajdonosi fogantyú, `new_bytes > 0`, a hibajelzés pedig a környező függvény feladata:

```
unsigned char *replacement = realloc(p, new_bytes);
if (replacement == NULL) {
    /* p és a korábbi tároló változatlanul használható. */
    return BUFFER_NOMEM;
}
p = replacement;
```

A közvetlen `p = realloc(p, new_bytes)` értékadás kudarc esetén felülírná a régi tároló elérhetőségét. Ha nincs más, felszabadításhoz használható kezdőpointer, memóriaszivárgás keletkezik.

8.2.1.1 Az azonos cím nem őrzi meg a régi pointer érvényességét

Siker után a korábbi allokációra mutató pointerek — köztük az elemekre mutató aliasok — nem használhatók tovább. Ez akkor is igaz, ha az implementáció ugyanazon a numerikus címen tudta megoldani az átméretezést. A régi pointerértéket még az újjal való összehasonlítás céljából sem szabad kiértékelni.

Ha egy pufferbeli pozíciót meg kell őrizni, **indexet vagy eltolást** tároljunk. Sikeres növelés után az új kezdőpointerből képezzük újra a hozzáférést. Ehhez az indexnek továbbra is a megfelelő tartományba kell esnie; elem olvasásakor az index kisebb legyen a logikai méretnél.

Az ideiglenes pointeres minta tehát két dolgot választ szét:

1. az új allokáció megszerzésének kísérletét;
2. a tulajdonosi állapot siker utáni véglegesítését.

Kudarc esetén a második lépés elmarad.

8.2.2 Megőrzött tartalom, inicializálás és nulla méret

ISO C17. Legyen a korábbi allokáció mérete A, az újé pedig B, C bájtban. Sikeres `realloc` esetén az első

$$\min(A, B)$$

C bájt tartalma megmarad. Növeléskor a hozzáadott rész nincs inicializálva; nem szabad például automatikusan nullterminátort vagy nulla számlálóértékeket feltételezni benne.

Ha egy puffer kapacitása 64, de csak 23 C bájt érvényes adatot tartalmaz, a 128-ra növelés nem hoz létre további 105 érvényes adatbájtot. A logikai méret továbbra is 23. A korábban nem inicializált terület megőrzése sem teszi annak tartalmát inicializálttá.

Szűkítéskor a levágott tartomány adatai elvesznek. Ha a logikai méret nagyobb lenne az új kapacitásnál, azt külön rendezni kellene; az alábbi megvalósítás ezért csak növel, kapacitást nem csökkent.

8.2.2.1 Miért kerüljük a realloc(p, 0) alakot?

ISO C17. A nulla méretű használat implementáció által meghatározott viselkedést érint, és elavuló használatnak számít. Meglévő allokációnál a `NULL` eredményből nem lehet hordozhatóan arra következtetni, hogy a korábbi tároló megmaradt.

C23-eltérés. Meglévő, nem null pointerrel végzett `realloc(p, 0)` esetén a viselkedés nem definiált.

A hordozható szerződés ezért egyszerű: felszabadításhoz `free` kell, nem nulla méretű `realloc`. Ezután a tulajdonosi állapotot külön nullázzuk. A bemutatott puffer egyetlen ágon sem hívja nulla mérettel a `realloc` függvényt.

8.2.3 Méret, kapacitás és ellenőrzött növekedés

A **logikai méret** (`size`) a már használható adatok mennyisége. A **kapacitás** (`capacity`) az allokációban rendelkezésre álló hely. Alapinvariánsunk:

$$0 \le \text{size} \le \text{capacity}.$$

A puffer három mezőjét itt egy `struct` fogja össze; a struktúrák részletes tárgyalása később következik.

Összetevő	Jelentés és élettartam
Tulajdonos: a `byte_buffer` objektum	Egy allokáció felszabadításáért felel; mezőit csak a pufferműveletek módosítják
Tároló: `data` által jelölt allokáció	Sikeres allokációtól a következő sikeres átméretezésig vagy felszabadításig él
Érvényes adatok	A `[0, size)` indextartomány
Még nem adatként használható hely	A `[size, capacity)` indextartomány
Kölcsönzött hozzáférés	Növelő átméretezésig vagy felszabadításig használható; elemolvasáshoz az index legyen `size` alatt

További invariáns, hogy `capacity == 0` pontosan akkor áll fenn, amikor `data == NULL`. A struktúra egyszerű másolása **nem** hoz létre új tulajdonost vagy külön allokációt; két másolat külön felszabadítása hibás lenne.

8.2.3.1 A méretszámítás levezetése

Ha s meglévő elemhez k újat adunk, az új méret $s + k$. `size_t` típusban az összeadás előtt kell ellenőrizni:

$$k \le \text{SIZE_MAX} - s.$$

Egy $w =$ `sizeof(T)` C bájtos elemtípus n eleméhez a szorzás feltétele:

$$n \le \left\lfloor \frac{\text{SIZE_MAX}}{w} \right\rfloor.$$

Előbb ellenőrzünk, és csak utána számoljuk ki az `n * sizeof(T)` kifejezést. A szorzás utáni ellenőrzés már elveszett információból dolgozna.

Példánk `unsigned char` elemeket tárol, ezért egy elem mérete pontosan egy C bájt, `CHAR_BIT` értékétől függetlenül.

A kapacitást 16-ról indítjuk, majd megduplázzuk. A `2 * capacity` művelethez:

$$\text{capacity} \le \left\lfloor \frac{\text{SIZE_MAX}}{2} \right\rfloor$$

szükséges. Ha ez nem teljesül, a jelölhető maximumra telítünk. Ez csak az aritmetikai helyességet biztosítja: ekkora allokáció sikerére természetesen nincs garancia. Szolgáltatásban ennél jóval kisebb, alkalmazásszintű memórialimit is indokolt.

8.2.4 Teljes példa: hibánál változatlan puffer

Az alábbi `buffer.c` önálló **ISO C17**, hosted program, fordítóbővítések nélkül. A műveletek érvényes, inicializált puffert feltételeznek; ugyanazt a puffert nem módosítják párhuzamosan.

A `buffer_reserve` legalább a kért kapacitást biztosítja, de nem változtatja meg a logikai méretet. A `buffer_push` csak sikeres helybiztosítás után írja be az új elemet és növeli a méretet.

```
#include <assert.h>
#include <stddef.h>
#include <stdint.h>
#include <stdlib.h>

struct byte_buffer {
    unsigned char *data;
    size_t size;
    size_t capacity;
};

enum buffer_result {
    BUFFER_OK,
    BUFFER_RANGE,
    BUFFER_NOMEM
};

static enum buffer_result
buffer_reserve(struct byte_buffer *b, size_t required)
{
    if (required <= b->capacity) {
        return BUFFER_OK;
    }

    size_t next = b->capacity == 0 ? 16 : b->capacity;

    while (next < required) {
        if (next > SIZE_MAX / 2) {
            next = SIZE_MAX;
            break;
        }
        next *= 2;
    }

    unsigned char *replacement = realloc(b->data, next);
    if (replacement == NULL) {
        return BUFFER_NOMEM;
    }

    b->data = replacement;
    b->capacity = next;
    return BUFFER_OK;
}

static enum buffer_result
```

```
buffer_push(struct byte_buffer *b, unsigned char value)
{
    if (b->size == SIZE_MAX) {
        return BUFFER_RANGE;
    }

    enum buffer_result result =
        buffer_reserve(b, b->size + 1);

    if (result != BUFFER_OK) {
        return result;
    }

    b->data[b->size] = value;
    ++b->size;
    return BUFFER_OK;
}

static void buffer_destroy(struct byte_buffer *b)
{
    free(b->data);
    *b = (struct byte_buffer){0};
}

int main(void)
{
    struct byte_buffer b = {0};

    for (size_t i = 0; i < 12; ++i) {
        if (buffer_push(&b, (unsigned char)i) != BUFFER_OK) {
            buffer_destroy(&b);
            return EXIT_FAILURE;
        }
    }

    const size_t saved_index = 7; /* Nem belső pointer. */

    for (size_t i = 12; i < 40; ++i) {
        if (buffer_push(&b, (unsigned char)i) != BUFFER_OK) {
            buffer_destroy(&b);
            return EXIT_FAILURE;
        }
    }

    assert(b.size == 40);
    assert(b.capacity == 64);
    assert(saved_index < b.size);

    {
        const unsigned char *view = b.data + saved_index;
        assert(*view == 7);
        /* A kölcsönzés alatt nincs átméretezés. */
```

```
    }

    buffer_destroy(&b);
    return EXIT_SUCCESS;
}
```

A `main` tulajdonosa `b`; a `view` csak kölcsönzött hozzáférés. Élettartamát szándékosan a felszabadítás előtti blokkra korlátoztuk. Allokációs hibánál a program felszabadítja a megmaradt tárolót, és hibás kilépési állapotot ad vissza.

Külön tanulói laborhoz kijelölt profil: Linux x86-64 System V, LP64, a `lab-next` profil (profiltábla). A kód maga nem igényel 8 bites C bájtot vagy LP64 adatmodellt.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -O0 -g buffer.c -o buffer-gcc

clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -O0 -g buffer.c -o buffer-clang
```

Ezek laborparancsok, nem dokumentált fordítási eredmények. Az ellenőrző állítások a sikeres út várt tulajdonságait írják le.

A függvények hibagaranciája erős: `BUFFER_NOMEM` vagy `BUFFER_RANGE` esetén a puffer mérete, kapacitása, tárolója és korábbi adatai változatlanok. A foglalást nem igénylő `reserve` a belső pointereket sem érvényteleníti.

8.2.5 Geometrikus növekedés és amortizált költség

Mi történne, ha minden új elemnél pontosan eggyel növelnénk a kapacitást? Ha az allocator minden alkalommal átmásolná a korábbi adatokat, N beszúrás összes másolási munkája:

$$0 + 1 + \cdots + (N-1) = \frac{N(N-1)}{2},$$

tehát $O(N^2)$ lenne.

Bizonyításvázlat. Duplázásnál a növelésekhez tartozó korábbi kapacitások geometrikus sorozatot alkotnak:

$$16 + 32 + \cdots + 2^k \cdot 16 = 16(2^{k+1} - 1).$$

Üres pufferből induló, egyenkénti beszúrásoknál ez $2N$ alatt marad. Az N új elem beírásával együtt az összes adatmozgatási munka $O(N)$, így egy beszúrás **amortizált költsége** $O(1)$.

Ez nem jelenti azt, hogy minden beszúrás konstans idejű. Egy növelést kiváltó művelet $O(N)$ adatmozgatást igényelhet. Az ISO C az allocator futási idejére nem ad ilyen komplexitási garanciát: a levezetés a mozgatott adatmennyiség költségmodellje. Az utolsó, `SIZE_MAX`-ra telítő növelés legfeljebb egyszer következhet be, és nem változtatja meg az összesített nagyságrendet.

Az alábbi adatok a növekedési szabályból következnek, nem mérési eredmények. A másolási oszlop azt feltételezi, hogy minden növelés a teljes régi allokáció másolásával jár.

Sikeres beszúrások száma	Kapacitás, C bájt	Allokációs hívások száma	Összes korábbi adat másolása, C bájt
1	16	1	0
16	16	1	0
17	32	2	16
32	32	2	16
40	64	3	48
65	128	4	112

A folytonos tároló kedvező térbeli lokalitást ad. Ennek ára a kihasználatlan kapacitás, az időnkénti nagyobb késleltetés és a növeléskori pointerérvénytelenítés. A puffer kapacitása nem azonos a folyamat RSS-évl vagy az allocator belső foglalási méretével.

8.2.6 Gyakori hibák és javításuk

A legsúlyosabb hiba a tulajdonosi pointer közvetlen felülírása. Javítása az ideiglenes pointer, majd a siker utáni véglegesítés.

Másik gyakori hiba a kapacitás idő előtti frissítése. Ha az allokáció később meghiúsul, a metaadat több helyet ígérne, mint amennyi ténylegesen rendelkezésre áll. A kapacitás csak siker után változhat.

A belső pointerek megtartását nem javítja az a megfigyelés, hogy „nálunk nem mozdult el a puffer”. Pozíciót indexként kell megőrizni.

Végül a `reserve` nem inicializálás: nagyobb kapacitás nem jelent több olvasható adatot. Az új logikai elemeket létrehozó műveletnek kell értéket írnia hozzájuk.

8.2.7 Feladatok

8.2.7.1 Állapotok megkülönböztetése

Egy puffer mérete 23, kapacitása 32. Legalább 50 C bájt kapacitást kérünk. Mi történik a megadott megvalósításban kudarc, illetve siker esetén? Hány C bájt tartalmának megőrzését garantálja a `realloc`, és hány C bájt marad logikailag érvényes adat?

8.2.7.2 Nullával feltöltő átméretezés

Írj `buffer_resize_zero` függvényt a példához. Növeléskor az új logikai elemeket töltse nullával; csökkentéskor csak a logikai méret változzon. Allokációs hibánál maradjon változatlan a puffer.

8.2.7.3 Belső pointer diagnosztikája

Az alábbi **hibás ISO C17 kódrészlet** előtt `b.size == b.capacity == 16`, és `b` érvényes tulajdonos:

```
const unsigned char *saved = b.data + 3;

if (buffer_reserve(&b, 32) == BUFFER_OK) {
    unsigned char value = *saved; /* Hibás. */
    (void)value;
}
```

Miért hibás akkor is, ha a tároló látszólag ugyanott marad? Javítsd ki.

8.2.8 Megoldások

8.2.8.1 Állapotok megkülönböztetése

Kudarc esetén `size == 23`, `capacity == 32`, és a régi tároló, valamint minden korábban érvényes belső pointer megmarad.

Siker esetén a duplázás miatt `capacity == 64`, de `size == 23`. A `realloc` az első 32 C bájt tartalmát őrzi meg; ettől még csak az első 23 C bájt logikailag érvényes adat. A korábbi allokációra mutató pointerek érvénytelenné válnak.

Ellenőrzési feltétel: mindkét kimeneten teljesüljön a méret–kapacitás invariáns, és az első 23 adatbájt maradjon változatlan.

8.2.8.2 Nullával feltöltő átméretezés

Az alábbi **ISO C17 kiegészítő részlet** a `buffer.c` fájlban, a `buffer_reserve` után helyezhető el; ugyanazok az include-ok és fordítási parancsok használhatók.

```
static enum buffer_result
buffer_resize_zero(struct byte_buffer *b, size_t new_size)
{
    enum buffer_result result = buffer_reserve(b, new_size);
    if (result != BUFFER_OK) {
        return result;
    }

    for (size_t i = b->size; i < new_size; ++i) {
        b->data[i] = 0;
    }

    b->size = new_size;
    return BUFFER_OK;
}
```

A tulajdonos továbbra is a hívó puffere. Kapacitásnövelés érvényteleníti a régi belső pointereket; puszta méretcsökkentés nem szünteti meg a tárolót, de a levágott indexek már kívül esnek a logikai adattartományon. Hibánál minden változatlan.

Ellenőrzési feltétel: növelés után az új tartomány minden eleme nulla; csökkentés után a kapacitás változatlan. Csökkentést követő újranöveléskor az ismét láthatóvá tett elemeket is nullázni kell.

8.2.8.3 Belső pointer diagnosztikája

Sikeres `realloc` után `saved` a megszűnt élettartamú korábbi allokációra hivatkozik. Dereferálása nem definiált viselkedés; nincs garantált eredménye.

A javítás indexet őriz meg, majd az aktuális kezdőpointert használja:

```
const size_t saved_index = 3;

if (buffer_reserve(&b, 32) == BUFFER_OK) {
    assert(saved_index < b.size);
    unsigned char value = b.data[saved_index];
    (void)value;
}
```

Ez szintén a meglévő programhoz tartozó C17-részlet. Ellenőrzési feltétele az index érvényessége és a harmadik indexű adat megőrzése, nem a régi és az új cím azonossága.

8.2.9 Összefoglaló

Pozitív méretű `realloc` kudarcakor a régi tároló megmarad; siker esetén kizárólag a visszaadott pointerből származtatott hozzáférések használhatók. A biztonságos puffer külön kezeli a méretet és a kapacitást, ellenőrzi az aritmetikát, és csak siker után módosítja tulajdonosi állapotát. A geometrikus növekedés amortizáltan olcsó beszúrást ad, de az egyedi növelések késleltetését és pointerérvénytelenítését nem szünteti meg.

8.3 Alignment és speciális allokációk

A dinamikusan lefoglalt tároló mérete önmagában nem határozza meg, hogy milyen objektum helyezhető el benne. A kezdőcímnek az objektum **igazítási követelményét** (*alignment requirement*) is teljesítenie kell. Egy megfelelően nagy, de rosszul igazított tároló nem válik alkalmassá egy típus befogadására attól, hogy a címét a megfelelő pointertípusra konvertáljuk.

A malloc, calloc, free és allokációs szerződés alapján már ismerjük a dinamikus tároló élettartamát és felszabadítását. Most ehhez hozzáadjuk az igazítás szerződését. A Deklarációk, typedef és típusminősítők fogalmaira építve külön kezeljük a típus követelményét, az egyedi objektum deklarációját és az allocator vállalását.

8.3.1 A típus és az objektum igazítása

ISO C17. Az igazítás azt korlátozza, hogy egy objektum milyen címeken helyezkedhet el. Mértékegysége a C bájt, vagyis a `sizeof` egysége; ez nem általánosan azonos a nyolcbites oktettel.

Egy teljes objektumtípus igazítását a `_Alignof(T)` operátor adja meg. Eredménye `size_t` típusú egész konstanskifejezés. Például `_Alignof(double)` a `double` típus igazítását jelenti, nem egy kiválasztott `double` objektum címéből megfigyelt tulajdonságot.

A `_Alignas` deklarációban ír elő igazítást. Argumentuma lehet támogatott igazítási érték vagy típusnév; utóbbi esetben az adott típus igazítása az előírás. Nem gyengítheti az objektum egyébként szükséges igazítását. A nulla értékű igazítási előírásnak nincs hatása.

Az alábbi C17 deklarációrészlet olyan compiler/target profilt feltételez, amely ebben a kontextusban támogatja a 64 C bájtos igazítást:

```
_Alignas(64) unsigned char workspace[1000];

struct simd_block {
    _Alignas(64) unsigned char bytes[64];
};
```

A két eset közötti különbség fontos:

- A `workspace` objektum kezdete legalább 64 C bájtos igazítású, de az `unsigned char[1000]` típus igazítása ettől nem változik meg. `_Alignof(unsigned char[1000])` továbbra is 1.
- A struktúra igazításának lehetővé kell tennie az igazított tag elhelyezését, ezért `_Alignof(struct simd_block)` legalább 64.

A tömb kezdetének igazítása nem jelenti minden elem ugyanilyen igazítását. A `workspace[1]` egy C bájttal követi a kezdetet. Ez az `unsigned char` számára megfelelő, de nem 64-es igazítású cím.

8.3.1.1 Fundamental és extended alignment

ISO C17. A **fundamental alignment**, vagy alapvető igazítás, legfeljebb `_Alignof(max_align_t)` értékű igazítás. A `max_align_t` típust a `<stddef.h>` deklarálja. Az ennél nagyobb igazítás **extended alignment**, azaz kiterjesztett igazítás; az ilyen követelményű típus túligazított (*over-aligned*).

A támogatott kiterjesztett igazítások és használati kontextusaik implementáció által meghatározottak. A megengedett igazítási értékek kettőhatványok, de ebből nem következik, hogy minden kettőhatvány minden kontextusban támogatott.

A `max_align_t` tehát nem a gépen elképzelhető legnagyobb igazítást jelenti. Különösen nem ígéri, hogy a közönséges allokáció automatikusan teljesít minden SIMD-utasítás vagy platformbővítmény követelményét.

8.3.2 Mit vállalnak az allocator API-k?

Az alábbi táblázatban pozitív méretű kéréseket feltételezünk. Könyvtári API-ban célszerű a nulla elemszámot külön, például null pointerrel reprezentált üres állapotként kezelni.

API és réteg	A sikeres kérés igazítási és méretfeltétele	Hibajelzés	Élettartamot lezáró művelet
`malloc`, ISO C17	Elegendő méret esetén alkalmas alapvető igazítású objektumhoz; tetszőleges kiterjesztett igazítást nem ígér	Null pointer	`free`

API és réteg	A sikeres kérés igazítási és méretfeltétele	Hibajelzés	Élettartamot lezáró művelet
`aligned_alloc`, ISO C17	Az igazítás érvényes és támogatott; a méret az igazítás egész számú többszöröse	Null pointer	`free`
`posix_memalign`, POSIX	Az igazítás kettőhatvány és `sizeof(void *)` többszöröse; a méretnek nem kell az igazítás többszörösének lennie	Közvetlenül visszaadott hibakód	`free`
`mmap`, POSIX/Linux	Leképezési, nem C-típushoz szabott szerződés; megfelelő használat mellett lapigazított kezdőcím	`MAP_FAILED`	`munmap`

Mindegyik sikeres allokációhoz vagy leképezéshez ki kell jelölni egy tulajdonost. A belőle képzett pointerek csak kölcsönzött hozzáférések: a tároló felszabadítása vagy leválasztása mindet érvényteleníti.

8.3.2.1 `malloc` és SIMD

ISO C17. A `malloc` elegendő méretű tárolója használható alapvető igazítású típusokhoz. Ebből nem vezethető le egy önkényesen választott 32 vagy 64 C bájtos igazítás.

Platformfüggő SIMD. Az utasításkészlet, az intrinsic és a fordítóbővítmény szerződése dönt arról, hogy szükséges-e erősebb igazítás. Egyes műveletek igazítatlan hozzáférést is támogatnak; mások igazítási előfeltételt támasztanak. A túligazítás önmagában sem gyorsulást, sem cache-line-elkülönítést nem garantál.

Az igazítás csak egy helyességi feltétel. A megfelelő méret, élettartam, inicializálás és effektív típus szabályai változatlanul érvényesek.

8.3.2.2 `aligned_alloc`

ISO C17. A `<stdlib.h>` által deklarált függvény az igazítást és a lefoglalandó méretet külön kapja meg:

```
/* Deklaráció emlékeztetőként; a tényleges deklarációt a header adja. */
void *aligned_alloc(size_t alignment, size_t size);
```

A fejezetben használt szerződés szerint a hívó előzetesen biztosítja:

1. az igazítás az implementáció által támogatott érvényes érték;
2. a méret az igazítás többszöröse;
3. a méretszámítás nem csordult túl.

Az allocator nem inicializálja az adatokat. A null pointert ellenőrizni kell; ISO C alapján nem építünk arra, hogy az `errno` meghatározott hibakódot tartalmaz.

8.3.2.3 `posix_memalign`

POSIX, nem ISO C. A függvény egy kimeneti paraméterbe írja a kapott pointert:

```
/* POSIX deklaráció emlékeztetőként. */
int posix_memalign(void **memptr, size_t alignment, size_t size);
```

Az igazítás kettőhatvány és `sizeof(void *)` többszöröse legyen. A méret lehet például 1000 akkor is, ha az igazítás 64.

Siker esetén a visszatérés 0. Hibás igazításra `EINVAL`, elégtelen memóriára `ENOMEM` hibakódot ad vissza. **A visszatérési értéket kell vizsgálni, nem az `errno` változót.** A jelenlegi POSIX-szerződés szerint hibánál a kimeneti pointer változatlan marad.

A kimeneti argumentumhoz valódi `void *` változó címét használjuk. Egy `T **` pointer `void **` típusra castolása nem általános megoldás: a többszintű pointerkonverzió nem követi a `void *` és objektumpointer közötti egyszerű konverzió szabályát.

8.3.3 Méretkerekítés túlcsordulás nélkül

Legyen $n > 0$ a hasznos méret, $a > 0$ az igazítás, mindkettő C bájtban. Az `aligned_alloc` számára szükséges legkisebb megfelelő méret:

$$r = \left\lceil \frac{n}{a} \right\rceil a.$$

A közvetlen `(n + a - 1) / a * a` kifejezésben az összeadás túlcsordulhat. Biztonságosabb először a maradékot számítani:

$$m = n \bmod a.$$

Ha $m = 0$, akkor $r = n$. Egyébként a hozzáadandó kitöltés:

$$d = a - m, \qquad r = n + d.$$

Az összeadás pontosan akkor ábrázolható `size_t` típusban, ha

$$n \le \mathrm{SIZE_MAX} - d.$$

Bizonyításvázlat. Ha $m \neq 0$, akkor $0 < m < a$, így $0 < d < a$. Az n számhoz éppen a következő a-többszörösig hiányzó mennyiséget adjuk. A kivonásos ellenőrzés még az összeadás előtt kizárja a túlcsordulást.

A következő adatok matematikai méretszámítások, nem mért allocator-layoutok:

Hasznos méret n	Igazítás a	Lefoglalt méret r	Többlet
1000	64	1024	24
1024	64	1024	0

Hasznos méret n	Igazítás a	Lefoglalt méret r	Többlet
1025	64	1088	63

8.3.3.1 Teljes példa két allokációs API-val

A választott, helyben ellenőrizendő laborprofil: Linux x86-64 System V ABI, LP64, `CHAR_BIT == 8`, a `lab-next` profil (profiltábla). A program C17-et és a POSIX `posix_memalign` függvényét használja; a profilban 64 C bájtos igazítást kérünk.

A fordítási parancsok elvégzendő ellenőrzések, nem dokumentált teszteredmények.

```c
/* alignment_demo.c */
#define _POSIX_C_SOURCE 200112L

#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

/* A 0 visszatérés érvénytelen bemenetet vagy túlcsordulást jelez. */
static size_t rounded_size(size_t n, size_t alignment)
{
    if (n == 0 || alignment == 0) {
        return 0;
    }

    size_t remainder = n % alignment;
    if (remainder == 0) {
        return n;
    }

    size_t padding = alignment - remainder;
    if (n > SIZE_MAX - padding) {
        return 0;
    }

    return n + padding;
}

int main(void)
{
    const size_t alignment = 64;
    const size_t payload = 1000;
    const size_t reserved = rounded_size(payload, alignment);

    if (reserved == 0) {
        fputs("Invalid allocation size\n", stderr);
        return EXIT_FAILURE;
    }

    void *owner = aligned_alloc(alignment, reserved);
```

```
    if (owner == NULL) {
        fputs("aligned_alloc failed\n", stderr);
        return EXIT_FAILURE;
    }

    /* Borrowed view: csak az owner felszabadításáig érvényes. */
    unsigned char *view = owner;
    for (size_t i = 0; i < payload; ++i) {
        view[i] = 0;
    }

    printf("fundamental=%zu requested=%zu reserved=%zu\n",
           _Alignof(max_align_t), alignment, reserved);

    free(owner);
    owner = NULL;
    view = NULL;

    /* POSIX: a méretet itt nem kell 64 többszörösére kerekíteni. */
    void *posix_owner = NULL;
    int rc = posix_memalign(&posix_owner, alignment, payload);
    if (rc != 0) {
        fprintf(stderr, "posix_memalign error: %d\n", rc);
        return EXIT_FAILURE;
    }

    free(posix_owner);
    posix_owner = NULL;
    return EXIT_SUCCESS;
}
```

Fordítás a profilhoz beállított fordítókkal:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 \
    alignment_demo.c -o alignment-gcc

clang -std=c17 -Wall -Wextra -Wpedantic -O2 \
    alignment_demo.c -o alignment-clang
```

Az első tároló tulajdonosa az `owner` által képviselt programállapot, a `view` nem tulajdonló hozzáférés. A `free(owner)` lezárja a tároló élettartamát. A nullázás csak a két helyi változót módosítja; esetleges további pointermásolatokat nem tenne biztonságossá.

A logikai adatméret 1000 marad, még ha az első foglalás 1024 C bájtos is. A többlet lefoglalt tároló nem automatikusan inicializált alkalmazásadat.

8.3.4 Lapigazítás és objektumigazítás

POSIX/Linux. A fájl- és anonim leképezéseknél megjelenő lapigazítás az operációs rendszer lapkezeléséhez kapcsolódik. Linuxon a null kezdőcímjavaslattal létrehozott sikeres `mmap` leképezés kezdete lapigazított. A rendszer lapméretét futás közben kell lekérdezni; nem általános állandóként 4096-nak tekinteni.

Legyen P a lapméret és A egy objektum megkövetelt igazítása. Ha

$$P \bmod A = 0,$$

akkor a lapigazított alapcím teljesíti az A igazítást is. Egy ettől k C bájtra kezdődő objektumhoz azonban még szükséges:

$$k \bmod A = 0.$$

Az alábbi feltételes Linux x86-64 laborpélda $P = 4096$ lapmérettel számol:

Objektum helye	Követelmény	Következik az igazítás?
Leképezés kezdete	64	Igen
Kezdettől 12 C bájtra	16	Nem
Kezdettől 64 C bájtra	64	Igen
Leképezés kezdete	8192	Nem feltétlenül

A lapigazítás tehát nem helyettesíti az objektumelhelyezés ellenőrzését. A lefoglalt tartományba beleférés és a típus szerinti hozzáférés feltételei külön megmaradnak.

A leképezés tulajdonosa a `munmap` végrehajtásáért felel. A leválasztott részre mutató minden kölcsönzött pointer érvénytelenné válik. Az `mmap` hibajelzése `MAP_FAILED`, nem null pointer; a leképezést pedig nem szabad `free` segítségével felszabadítani. A részletes leképezési szerződést a Fájlrendszer-műveletek, mmap és tartósság tárgyalja.

8.3.5 Gyakori hibák és javításuk

„A cast megigazítja a címet.” Nem: nem változtatja meg a tároló helyét. Már a hibásan igazított objektumpointer-konverzió is sértheti a C követelményeit; nem csak a későbbi dereferálás veszélyes.

„A `malloc` nálam mindig 64-re igazított.” Néhány megfigyelt cím nem API-garancia. Kiterjesztett követelményhez megfelelő allokációs szerződés kell.

„Igazított foglalás után közönséges `realloc` használható.” A `realloc` nem ígéri az eredeti kiterjesztett igazítás megőrzését. Biztonságos növeléshez új igazított foglalás, a megtartandó adatok másolása, majd a régi tároló felszabadítása szükséges. Az eredeti tulajdonosi állapotot csak sikeres új foglalás után szabad feladni.

„Kézzel beljebb tolom a pointert, majd azt felszabadítom.” A belső cím nem válik allokációs alapcímmé. A `free` számára az eredeti visszakapott pointer kell; ezért a kézi igazítás külön owner- és view-nyilvántartást igényel.

8.3.6 Feladatok

1. **Magyarázó feladat.** Miért nem változtatja meg a `_Alignas(64) unsigned char data[128]` deklaráció az `unsigned char[128]` típus igazítását? Garantáltan 64-re igazított-e a második elem?
2. **Kódolási feladat.** Egy API `count` darab, `element_size` C bájtos elemhez kér tárolót. Írd meg az elemszorzás és az ezt követő igazítási kerekítés ellenőrzését. A nulla elemszám legyen elutasított bemenet.
3. **Diagnosztikai feladat.** Egy függvény `posix_memalign` segítségével foglal, hibánál `perror`-t hív, siker esetén eltárolja a kezdőcímet és egy belső pointert. Később a belső pointert adja át `free`-nek. Mi a két szerződéshiba?
4. **Alkalmazási feladat.** Egy leképezés lapmérete 4096 C bájt. Garantáltan teljesül-e a 128-as objektumigazítás a kezdettől 384 C bájtra?

8.3.7 Megoldások

8.3.7.1 Deklaráció és típus

A `_Alignas` az adott objektum deklarációjára vonatkozik, nem hoz létre új tömbtípust. `_Alignof(unsigned char[128])` értéke 1. A második elem egy C bájttal követi a 64-re igazított kezdetet, ezért nem 64-re igazított. Ellenőrzési feltétel: külön vizsgáljuk az objektum kezdetét, az elemtípus igazítását és az elemeltolást.

8.3.7.2 Ellenőrzött elemszorzás

A következő C17 részlet a teljes példában szereplő `rounded_size` függvényt és headereket használja:

```
static size_t allocation_size(size_t count,
                              size_t element_size,
                              size_t alignment)
{
    if (count == 0 || element_size == 0) {
        return 0;
    }
    if (count > SIZE_MAX / element_size) {
        return 0;
    }

    return rounded_size(count * element_size, alignment);
}
```

Pozitív `element_size` mellett az osztásos feltétel biztosítja, hogy a szorzás ábrázolható. Ezután a kerekítés saját ellenőrzése védi az összeadást. Például 125 darab 8 C bájtos elem, 64-es igazítással 1024-es eredményt ad. A függvény az allocator igazításstámogatását nem ellenőrzi.

8.3.7.3 Hibakód és tulajdonos

A `posix_memalign` visszatérési kódját kell feldolgozni; a `perror` más hibához tartozó `errno` értéket jeleníthet meg. A `free` pedig kizárólag az eredeti foglalási pointert kaphatja. A belső pointer kölcsönzött hozzáférés, amelyet a felszabadítás érvénytelenít. Ellenőrzési feltétel: minden sikeres foglaláshoz pontosan egy megfelelő felszabadítás tartozzon.

8.3.7.4 Leképezésen belüli igazítás

Igen: $4096 \bmod 128 = 0$, és $384 \bmod 128 = 0$. Ez az igazítást bizonyítja, nem a teljes hozzáférés helyességét. Az objektumnak a leképezett tartományban is el kell férnie, és a leképezésnek a használat teljes ideje alatt fenn kell maradnia.

8.3.8 Rövid összefoglaló

Az igazítási döntés két kérdésből áll: **mit követel az objektum, és mit vállal az allocator?** A `_Alignof`, `_Alignas` és `max_align_t` az első kérdést tisztázza. A `malloc`, `aligned_alloc`, `posix_memalign` és `mmap` különböző választ ad a másodikra.

A helyes megoldás ellenőrzi a méretaritmetikát és az API-specifikus feltételeket, majd megőrzi az eredeti tulajdonosi pointert. Sem a cast, sem a lapigazítás, sem néhány kedvező mérési megfigyelés nem helyettesíti ezt a szerződést.

8.4 Tulajdonlási szerződés és cleanup ágak

A dinamikus memória kezelése nem ér véget a sikeres `malloc` hívásnál. Egy függvény megszerezhet több erőforrást, majd a következő műveletnél hibát észlelhet. A helyesség ilyenkor azt jelenti, hogy minden megszerzett erőforrásnak megmarad az egyértelmű felelőse, és minden kilépési út pontosan a szükséges felszabadításokat végzi el.

Az allokációs szerződés alapján a megszerzett tárolót megfelelően fel kell szabadítani. A pointerek és többszintű indirekció eszközt adnak ahhoz, hogy egy függvény pointert adjon vissza output paraméteren keresztül. A cleanup és nem lokális vezérlés tárgyalásából pedig fontos felidézni: a C nem futtat automatikus erőforrás-felszabadítást pusztán azért, mert elhagyunk egy blokkot.

8.4.1 Tulajdonos, kölcsönzött hozzáférés és átadás

A **tulajdonos** (*owner*) az a programszereplő, amely egy erőforrás felszabadításáért felel. Ez lehet hívó függvény, modul vagy egy adatstruktúrán keresztül megvalósított erőforrás-kezelő. Az ownership C-ben elsősorban API-szerződés: a pointer típusa önmagában nem írja le.

A **kölcsönzött pointer** (*borrowed pointer*) hozzáférést enged, de nem ruházza át a felszabadítás felelősségét. Érvényes használatához egyszerre kell teljesülnie, hogy:

- a hivatkozott objektum élettartama még tart;
- a hozzáférés az objektumhatárokon belül marad;
- a művelet megfelel a típus- és módosíthatósági szabályoknak;
- nem történt a pointert érvénytelenítő művelet.

A `const int *` nem tulajdonlási jelölés. Azt korlátozza, hogy ezen a hozzáférési úton módosítható-e az `int`; nem akadályozza meg, hogy a valódi tulajdonos közben felszabadítsa a tárolót.

A **tulajdonlás átadása** (*ownership transfer*) a felszabadítási felelősség szerződéses áthelyezése. A pointer másolása önmagában nem átadás. Például az `a = b` után két pointerváltozó tartalmazhatja ugyanazt a pointerértéket, de ettől nem lesz két jogosult felszabadító.

Hasznos szerződés: „siker esetén átveszi, hiba esetén nem veszi át”. Más szabály is tervezhető, de a hívónak minden kimenetnél tudnia kell, ki takarít.

Szerep	Mit tehet?	Meddig érvényes?	Felszabadítás
Tulajdonos	A szerződés szerinti műveleteket végezheti	Az erőforrás lezárásáig vagy átadásáig	Az ő felelőssége
Kölcsönző	Csak az engedett hozzáférést használhatja	Legfeljebb a kölcsönzési idő végéig, invalidáció nélkül	Nem jogosult rá
Átadó	Átadás előtt tulajdonosként jár el	Sikeres átadás után már nem tulajdonos	Siker után nem szabadít
Átvevő	Sikeres átvételtől tulajdonos	Saját lezárásáig vagy további átadásig	Siker után az ő felelőssége

8.4.2 A létrehozás és megszüntetés szerződéspárjai

A `create/destroy` és az `init/deinit` nem C nyelvi kulcsszavak, hanem elterjedt API-elnevezések. A konkrét jelentésüket dokumentálni kell. Ebben a fejezetben:

- az `init` már létező kezelőobjektumhoz szerez belső erőforrásokat;
- a `deinit` ezeket felszabadítja, de a kezelőobjektum tárolóját nem;
- a `create` a kezelőobjektumot is dinamikusan foglalja;
- a `destroy` a belső erőforrásokat és a kezelőobjektumot is felszabadítja.

Így egy automatikus tárolási idejű kezelőobjektumhoz `init/deinit`, egy `create` által visszaadott objektumhoz pedig `destroy` tartozik. E párok felcserélése szivárgáshoz vagy nem definiált viselkedéshez vezethet.

A példánk üres állapota:

$$E = (\texttt{label} = \text{NULL}, \texttt{values} = \text{NULL}, \texttt{count} = 0).$$

Az `init` csak ilyen állapotú objektumon hívható. Siker után teljesen használható objektumot ad; hiba után visszaállítja az üres állapotot. Ez egyszerűbb hívói szerződés, mint a „néhány mező talán már érvényes” állapot átadása.

Az output paraméteres `create` szerződése:

- az `out` egy írható `struct batch *` objektumra mutat;
- ez a kimeneti hely kezdetben `NULL`, nem tartalmaz elveszíthető tulajdont;
- siker esetén `*out` az új objektum, amelyért a hívó felel;
- hiba esetén `*out == NULL`.

A `struct batch **` ezért nem díszítés: a függvény a hívó pointerobjektumát módosítja.

8.4.3 Kidolgozott példa: mérési adatok létrehozása

Egy mérési köteg címkét és nemnegatív egész mérési értékeket tárol. A bemeneteket lemásolja; a hívó bemeneti tárolóira nem tart meg hivatkozást. A három dinamikus erőforrás a kezelőobjektum, a címke másolata és az értéktömb.

Nyelvi profil: ISO C17, hosted környezet, GNU nyelvi bővítések nélkül. A `malloc`, `free` és a pointerhasználat szabályai nyelvi/könyvtári szabályok; az üres állapot és az átadás időpontja a saját API-nk szerződése.

A példa előfeltételei: a nem null címkepointer érvényes, nullterminált sztringre mutat. A méretellenőrzésen átjutó, nem null bemeneti tömbből `count` darab inicializált `int` olvasható. A bemenetek a hívás alatt változatlanok, és nem fedik át a módosított kezelőobjektumot. A kód nem biztosít konkurens hozzáférési szinkronizációt.

A teljes `ownership.c` fájl:

```
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum batch_status {
    BATCH_OK,
    BATCH_ARGUMENT,
    BATCH_RANGE,
    BATCH_NOMEM,
    BATCH_VALUE
};

struct batch {
    char *label;
    int *values;
    size_t count;
};

/* b elo, ervenyes objektum; ures vagy reszlegesen inicializalt is lehet. */
static void batch_deinit(struct batch *b)
{
    free(b->values);
    free(b->label);
    *b = (struct batch){0};
}

/* Elofeltetel: b ervenyes, es ures allapotban van. */
static enum batch_status batch_init(
    struct batch *b,
    const char *label,
    const int *values,
    size_t count)
{
    enum batch_status status;
    size_t label_length;
```

```
    if (label == NULL || values == NULL ||
        count == 0 || label[0] == '\0') {
        return BATCH_ARGUMENT;
    }

    if (count > SIZE_MAX / sizeof *b->values) {
        return BATCH_RANGE;
    }

    label_length = strlen(label);
    if (label_length == SIZE_MAX) {
        return BATCH_RANGE;
    }

    b->label = malloc(label_length + 1);
    if (b->label == NULL) {
        status = BATCH_NOMEM;
        goto cleanup;
    }
    memcpy(b->label, label, label_length + 1);

    b->values = malloc(count * sizeof *b->values);
    if (b->values == NULL) {
        status = BATCH_NOMEM;
        goto cleanup;
    }

    for (size_t i = 0; i < count; ++i) {
        if (values[i] < 0) {
            status = BATCH_VALUE;
            goto cleanup;
        }
        b->values[i] = values[i];
    }

    b->count = count;
    return BATCH_OK;

cleanup:
    batch_deinit(b);
    return status;
}

/* Csak NULL vagy batch_create altal atadott, meg elo objektum. */
static void batch_destroy(struct batch *b)
{
    if (b == NULL) {
        return;
    }

    batch_deinit(b);
    free(b);
```

```
}

/* Elofeltetel: out ervenyes, irhato, es *out == NULL. */
static enum batch_status batch_create(
    const char *label,
    const int *values,
    size_t count,
    struct batch **out)
{
    struct batch *b;
    enum batch_status status;

    *out = NULL;

    b = malloc(sizeof *b);
    if (b == NULL) {
        return BATCH_NOMEM;
    }
    *b = (struct batch){0};

    status = batch_init(b, label, values, count);
    if (status != BATCH_OK) {
        goto cleanup;
    }

    *out = b;       /* Tulajdonlas atadasa a hivonak. */
    b = NULL;       /* A helyi valtozo mar nem tulajdonos. */

cleanup:
    batch_destroy(b);
    return status;
}

int main(void)
{
    const int input[] = {3, 7, 2};
    struct batch *b = NULL;

    enum batch_status status =
        batch_create("latency", input, 3, &b);

    if (status != BATCH_OK) {
        fprintf(stderr, "batch_create: status=%d\n", (int)status);
        return EXIT_FAILURE;
    }

    {
        const int *view = b->values;
        printf("%s: %d\n", b->label, view[0]);
        /* A kolcsonzott hozzaferes itt veget er. */
    }
```

```
    batch_destroy(b);
    b = NULL;
    return EXIT_SUCCESS;
}
```

A `struct batch` mezőihez a `->` operátorral férünk hozzá pointeren keresztül. A példa nem kíván konkrét struktúraelrendezést vagy rögzített `int`-szélességet.

Választott laborprofil: Linux x86-64, System V ABI, LP64; a `lab-next` profil (profiltábla), illetve a `lab-next` profil (profiltábla). Az alábbi parancsok feltételezik, hogy a `gcc` és `clang` neveken ezek a kiadások érhetők el. A példa itt nem futtatott vagy mért eredmény.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
    -O0 -g ownership.c -o ownership-gcc

clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
    -O0 -g ownership.c -o ownership-clang
```

A példában minden allokációméret C bájtban értendő. A szorzás biztonsági feltétele, ha n az elemszám és $s = \mathrm{sizeof}(\texttt{int})$:

$$ns \le \mathrm{SIZE_MAX} \iff n \le \left\lfloor \frac{\mathrm{SIZE_MAX}}{s} \right\rfloor .$$

Mivel $s > 0$, az osztásos ellenőrzést még a potenciálisan túlcsorduló szorzás előtt elvégezhetjük.

8.4.4 Minden hibakimenet végigvezetése

A részleges inicializálást nem az jelenti, hogy inicializálatlan pointermezőket próbálunk felszabadítani. A kezelőobjektum minden mezője már a foglalások előtt meghatározott értékű. A `(struct batch){0}` típusos inicializálás null pointerértékeket hoz létre; nem támaszkodik a null pointer csupa nulla bites reprezentációjára.

Kilépési ok	Addig megszerzett erőforrás	Takarítás	Hívó által látott állapot
A kezelőobjektum foglalása sikertelen	Nincs	Nincs	`*out == NULL`
Hibás argumentum vagy méret	Kezelőobjektum	Külső `destroy`	`*out == NULL`
A címke foglalása sikertelen	Kezelőobjektum	Belső `deinit`, külső `destroy`	`*out == NULL`
Az értéktömb foglalása sikertelen	Kezelőobjektum, címke	Címke, majd kezelőobjektum	`*out == NULL`
Negatív érték másolás közben	Mindhárom	Értéktömb, címke, kezelőobjektum	`*out == NULL`
Siker	Mindhárom	A függvény nem szabadítja fel az átadott objektumot	A hívó tulajdonos

A negatív érték felismerésekor az értéktömb csak részben lehet feltöltve. A `free` számára ez nem akadály: nem olvassa ki az egyes `int` értékeket. Összetettebb elemeknél viszont, amelyek saját

erőforrásokat birtokolnak, külön számlálni kellene a már sikeresen inicializált elemeket, és csak azokat lebontani.

A példában a `count` csak a teljes siker után kap végleges értéket. A cleanup nem ebből találja ki, mely foglalások sikerültek: ezt a null vagy érvényes tulajdonló pointermezők jelzik.

8.4.4.1 Miért működik a közös cleanup ág?

A `goto cleanup` itt előrefelé ugrik egyetlen takarítási pontra. Nem rejti el a program állapotát: éppen a sok külön `return` mellé másolt felszabadításokat helyettesíti.

Bizonyításvázlat. Minden sikeres foglalás eredménye azonnal egy tulajdonló mezőbe kerül. A még meg nem szerzett erőforrást `NULL` jelöli. Minden későbbi hiba eléri a `batch_deinit` hívást, amely minden ilyen mezőt felszabadít, majd üres állapotot állít elő. A külső `batch_destroy` ismételt `deinit` hívása ezért már csak null pointereket ad át a `free` függvénynek. Végül felszabadul a kezelőobjektum is.

Siker esetén a `*out = b` átadja a tulajdont, a `b = NULL` pedig megszünteti a helyi cleanup felszabadítási kötelezettségét. Így egyetlen átadott erőforrás sem szabadul fel idő előtt.

8.4.4.2 Fordított sorrend és élettartamhatárok

A megszerzési sorrend:

$$\text{kezelőobjektum} \rightarrow \text{címke} \rightarrow \text{értéktömb}.$$

A felszabadítás ennek fordítottja. A két belső foglalás itt egymástól független, de a kezelőobjektumot utoljára kell elengedni: addig abból olvassuk ki a felszabadítandó pointereket.

Általánosan, ha egy később létrehozott erőforrás egy korábbira támaszkodik, előbb a függő erőforrást kell megszüntetni. A fordított megszerzési sorrend ezért gyakori, de a valódi szabály a függőségek tiszteletben tartása.

Objektum vagy hozzáférés	Tulajdonos	Élettartam / invalidáció
`input` bemeneti tömb	A `main` automatikus objektuma	A blokk végéig él; a létrehozás csak kölcsönzi
Dinamikus kezelőobjektum	Kezdetben `batch_create`, siker után a hívó	`batch_destroy` megszünteti
Címke és értéktömb tárolója	A kezelőobjektumon keresztül kezelt tulajdon	`batch_deinit` vagy `batch_destroy` felszabadítja
`view` kölcsönzött pointer	Nem tulajdonos	A belső tároló felszabadítása érvényteleníti

A kölcsönzött pointer használata felszabadítás után nem hibakódot eredményez, hanem érvénytelen hozzáférés esetén nem definiált viselkedést.

8.4.5 Idempotens takarítás és hibákat elfedő konvenciók

Egy művelet **idempotens**, ha ismételt alkalmazása ugyanazt az állapotot eredményezi, mint egyszeri alkalmazása. A `batch_deinit` ilyen, amíg maga a kezelőobjektum él: az első hívás üres állapotot hoz létre, a következő ezt változatlanul hagyja.

A `batch_destroy` viszont **nem idempotens ugyanazzal a korábbi pointerértékkel**. Az első hívás megszünteti a kezelőobjektum élettartamát. A másodikhoz már nincs élő objektum. A `NULL` elfogadása ettől külön tulajdonság.

A hívó pointerének nullázása hasznos, de csak azt az egy pointerobjektumot módosítja. Más aliasokat és kölcsönzött nézeteket nem javít meg.

Gyakori hibák és javításuk:

- **Újrainicializálás egy már erőforrást birtokló objektumon.** Elveszítheti a régi pointereket. Előbb szabályos `deinit` szükséges, vagy külön, dokumentált cseremővelet.
- **`destroy` automatikus objektumra.** A belső erőforrások után nem dinamikusan foglalt tárolót próbálna felszabadítani. Ilyenkor `deinit` kell.
- **Nem üres output hely felülírása.** A régi tulajdon elveszhet. Az üres kimeneti hely előfeltételét nem helyettesíti a függvény eleji nullázás.
- **A nullázás összekeverése az aliasok érvényesítésével.** A másolatok továbbra is dangling pointerek.
- **Korai visszatérés egy késői hibából.** A negatív érték ágában a közvetlen `return` szivárgást okozna; a cleanup ágra kell jutni.

A közös cleanup csak a hozzá vezető vezérlési utakra érvényes. Nem véd önmagában nem lokális kilépés vagy a folyamat rendellenes megszűnése ellen.

8.4.6 Feladatok

8.4.6.1 Szerződés értelmezése

A `batch_create` sikeres visszatérése után megszűnhet-e a bemeneti `values` tömb élettartama? Ki szabadítja fel a `b->values` tárolót?

8.4.6.2 Részleges inicializálás követése

A bemenet `{4, -1, 8}`. Vezesd végig a létrehozást, feltételezve, hogy minden foglalás sikeres. Mely tömbelemek kapnak értéket, és mi lesz az output paraméter állapota?

8.4.6.3 Nullázó felszabadító készítése és diagnózis

Írj `batch_destroy_slot(struct batch **slot)` függvényt, amely felszabadítja a `*slot` objektumot, majd nullázza a hívó pointerét. Ezután magyarázd meg, miért nem javítja meg a következő, szándékosan hibás C17-kódrészletet:

```
/* Hibas reszlet: b egy sikeresen letrehozott objektum tulajdonosa. */
struct batch *alias = b;
batch_destroy_slot(&b);
batch_destroy(alias);  /* Ervenytelen felszabaditas. */
```

8.4.7 Megoldások

8.4.7.1 Szerződés értelmezése

Igen. A függvény saját tárolóba másolja az értékeket, ezért a siker után nem függ a bemeneti tömb élettartamától. A `b->values` felszabadítását a hívó a `batch_destroy(b)` művelettel kezdeményezi.

Ellenőrzési feltétel: az implementáció nem tárolja el a bemeneti `values` pointert; a mező saját foglalás eredményét tartalmazza.

8.4.7.2 Részleges inicializálás követése

Először létrejön a kezelőobjektum, a címke és az értéktömb. A nulladik elem megkapja a `4` értéket. Az első indexnél a negatív bemenet miatt a függvény még a tárolás előtt a cleanup ágra ugrik.

A `deinit` felszabadítja az értéktömböt és a címkét, majd üres állapotot állít elő. A külső cleanup felszabadítja a kezelőobjektumot. Az eredmény `BATCH_VALUE`, az output pedig `NULL`.

Ellenőrzési feltétel: egyik sikeres foglalás sem marad felszabadítatlanul, és a részlegesen feltöltött tömb inicializálatlan elemeit semmi nem olvassa.

8.4.7.3 Nullázó felszabadító

Az alábbi C17-részlet az előző teljes példába illeszthető, a `batch_destroy` definíciója után:

```
/* Elofeltetel: slot ervenyes, irhato pointerobjektumra mutat. */
static void batch_destroy_slot(struct batch **slot)
{
    batch_destroy(*slot);
    *slot = NULL;
}
```

Ugyanazzal az élő, önálló kimeneti hellyel ismét meghívható: a második hívás már `NULL` értéket ad tovább. A `slot` tárolója nem lehet a felszabadítandó objektum része.

Az `alias` azonban külön pointerobjektum. A `b` nullázása nem módosítja, ezért a második felszabadítás érvénytelen. A javítás a második `destroy` elhagyása és az alias minden további használatának megszüntetése.

Ellenőrzési feltétel: minden megszerzett erőforráshoz egyetlen tényleges felszabadítás tartozik; a felszabadítás után egyetlen alias sem kerül felhasználásra.

8.4.8 Összefoglaló

A megbízható C erőforráskezelés alapja az explicit szerződés: ki tulajdonos, ki kölcsönző, és melyik művelet adja át a felelősséget. A részleges inicializálást meghatározott mezőértékekkel és egyértelmű hiba utáni állapottal kell kezelni.

A `goto cleanup` áttekinthető eszköz a hibautak egyesítésére. A helyességet azonban nem maga az ugrás biztosítja, hanem az, hogy minden sikeres szerzés követhető, minden függőség megfelelő sorrendben szűnik meg, és egyetlen erőforrás sem szabadul fel kétszer.

8.5 Allokációs hiba-injektálás és leak diagnosztika

A sikeres allokációkat lefedő teszt még nem bizonyítja, hogy a program erőforrás-kezelése helyes. A valódi kérdés az, hogy **minden megszakított felépítési folyamat után helyreállnak-e az invariánsok**: felszabadulnak-e a már megszerzett erőforrások, érvényes marad-e a korábbi állapot, és nem marad-e használatban érvénytelen pointer.

A realloc és dinamikusan növekvő buffer rész alapján sikeres átméretezés után a régi allokációból származó pointereket nem használhatjuk tovább. A tulajdonlási szerződés és cleanup ágak adja a hibautak szerkezetét; az élettartam- és memóriahibák vizsgálati laborja pedig a diagnosztikai szemléletet. Most ezekhez determinisztikus hibákat és ellenőrizhető erőforrásmérleget kapcsolunk.

8.5.1 Determinisztikus allokációs hiba

Az **allokációs hiba-injektálás** (*allocation failure injection*) szándékosan sikertelenre változtat egy kiválasztott foglalási kísérletet. Nem szükséges hozzá tényleges memóriahiány: egy tesztadapter az N-edik hívásnál közvetlenül null pointert ad vissza.

A számlálás foglalási **kísérleteket** jelent, nem csak sikeres foglalásokat. A pozitív méretű `realloc` szintén kísérlet, akkor is, ha az implementáció egyébként helyben tudná végrehajtani.

Ha egy adott bemenet sikeres útján K foglalási kísérlet történik, célszerű külön tesztesetben megvizsgálni:

- a hibamentes futást;
- az első foglalás kudarcát;
- majd külön-külön minden további foglalás kudarcát, egészen K-ig.

Minden eset friss állapotból indul. Ez az adott út hibapontjait fedi le; más bemenetekhez tartozó elágazásokat, újrapróbálkozásokat és a hibakezelés közbeni további foglalásokat nem automatikusan.

Az alábbi adapter szerződése szűk:

- egyetlen szál használja;
- nulla méretű kérést nem fogad;
- az átméretezés bemenete élő, nem null pointer;
- csak az adapteren át kezelt allokációkat számolja;
- szimulált hibánál nem állítja az `errno` értékét.

Ez az utóbbi fontos különbség: itt ISO C17 szerinti, null pointerrel jelzett sikertelenséget tesztelünk, nem egy POSIX-specifikus hibakód-szerződést.

8.5.1.1 Az erőforrásmérleg

Legyen L az élő, általunk nyilvántartott allokációk száma. Ekkor:

$$L_{\text{után}} = L_{\text{előtt}} + A - F,$$

ahol A az új, sikeres foglalások, F pedig a nem null pointerrel végzett, érvényes felszabadítások száma.

A nem null pointeren végzett, pozitív méretű `realloc` ezt a logikai darabszámot nem változtatja meg. Sikertelenségkor megmarad a régi allokáció; siker esetén annak helyébe lép az új.

Bizonyításvázlat. Kezdetben $L = 0$. Minden sikeres `malloc` vagy `calloc` eggyel növeli, minden érvényes felszabadítás eggyel csökkenti az értéket. A fenti szerződésű átméretezés nettó változása nulla. A műveletsor hosszára végzett indukcióval az összefüggés minden prefixre fennáll.

A nulla végösszeg szükséges, de önmagában nem elégséges bizonyíték: egy puszta számláló nem azonosítja az allokációkat, és nem ellenőrzi a pointerek eredetét.

8.5.2 Kidolgozott labor: részleges felépítés visszagörgetése

A példában egy eredmény két erőforrást birtokol: egy egészérték-tömböt és egy címkét. A felépítés három foglalási kísérletből áll: `malloc`, `calloc`, majd `realloc`.

Szerződés: a `build` üres kimenetet kap. Siker esetén átadja a két allokáció tulajdonlását; hibánál a kimenet üres marad. A `destroy` üres objektumra is meghívható.

Szerep	Tulajdonos	Élettartamhatár és érvénytelenítés
Felépítés alatti tömb és címke	A `build` cleanup-logikája	Hiba esetén a közös takarító ág szabadítja fel
Sikeres eredmény két allokációja	A hívó `result` objektuma	A `destroy` felszabadítja őket
A tömbből képzett kölcsönzött pointer	Nem tulajdonos	Sikeres átméretezés vagy felszabadítás érvényteleníti
Az eredményt leíró `result` objektum	A teszteset automatikus objektuma	Hatókörének elhagyása önmagában nem szabadítja fel a dinamikus tárolót

Laborprofil: ISO C17, Linux hosted környezet, x86-64 System V ABI, LP64. Rögzíthető kiinduló verziók: a `lab-next` profil (profiltábla); Clang esetén illeszkedő compiler-rt 19.1.7. A helyi elérhetőséget és párosítást ellenőrizni kell. A következő példákhoz nem tartozik dokumentált fordítási vagy futtatási eredmény.

A teljes `fi.c`:

```
#include <assert.h>
#include <stddef.h>
#include <stdlib.h>
#include <string.h>

struct fi {
    size_t calls;
    size_t fail_at;  /* 0: nincs injektalt hiba */
    size_t live;
};

struct result {
    int *data;
    char *label;
    size_t len;
```

```
};

static int reject(struct fi *f)
{
    ++f->calls;
    return f->fail_at != 0 && f->calls == f->fail_at;
}

static void *fi_malloc(struct fi *f, size_t bytes)
{
    assert(bytes > 0);
    if (reject(f))
        return NULL;

    void *p = malloc(bytes);
    if (p != NULL)
        ++f->live;
    return p;
}

static void *fi_calloc(struct fi *f, size_t count, size_t size)
{
    assert(count > 0 && size > 0);
    if (reject(f))
        return NULL;

    void *p = calloc(count, size);
    if (p != NULL)
        ++f->live;
    return p;
}

static void *fi_resize(struct fi *f, void *p, size_t bytes)
{
    assert(p != NULL && bytes > 0);
    if (reject(f))
        return NULL;       /* p allokacioja valtozatlanul el */

    return realloc(p, bytes);
}

static void fi_free(struct fi *f, void *p)
{
    if (p != NULL) {
        assert(f->live > 0);
        free(p);
        --f->live;
    }
}

static int build(struct fi *f, struct result *out)
{
```

```
    int *data = NULL;
    char *label = NULL;
    int status = -1;

    assert(out->data == NULL);
    assert(out->label == NULL && out->len == 0);

    data = fi_malloc(f, 4 * sizeof *data);
    if (data == NULL)
        goto done;

    for (size_t i = 0; i < 4; ++i)
        data[i] = (int)i;

    label = fi_calloc(f, 8, sizeof *label);
    if (label == NULL)
        goto done;
    memcpy(label, "probe", sizeof "probe");

    int *next = fi_resize(f, data, 8 * sizeof *data);
    if (next == NULL)
        goto done;
    data = next;

    for (size_t i = 4; i < 8; ++i)
        data[i] = (int)i;

    out->data = data;
    out->label = label;
    out->len = 8;
    data = NULL;          /* tulajdonlas atadva */
    label = NULL;
    status = 0;

done:
    fi_free(f, label);
    fi_free(f, data);
    return status;
}

static void destroy(struct fi *f, struct result *r)
{
    fi_free(f, r->label);
    fi_free(f, r->data);
    r->label = NULL;
    r->data = NULL;
    r->len = 0;
}

static size_t run_case(size_t fail_at)
{
    struct fi f = {0, fail_at, 0};
```

```
    struct result r = {NULL, NULL, 0};

    int status = build(&f, &r);

    if (fail_at == 0) {
        assert(status == 0);
        assert(f.calls == 3 && f.live == 2);
        assert(r.len == 8);
        assert(strcmp(r.label, "probe") == 0);
        for (size_t i = 0; i < r.len; ++i)
            assert(r.data[i] == (int)i);
    } else {
        assert(status == -1);
        assert(f.calls == fail_at);
        assert(r.data == NULL && r.label == NULL);
        assert(r.len == 0 && f.live == 0);
    }

    destroy(&f, &r);
    assert(f.live == 0);
    assert(r.data == NULL && r.label == NULL && r.len == 0);
    return f.calls;
}

int main(void)
{
    size_t count = run_case(0);
    for (size_t n = 1; n <= count; ++n)
        (void)run_case(n);
    return EXIT_SUCCESS;
}
```

A méretek itt kis, rögzített értékek, amelyek a megadott profilban reprezentálhatók. Változó elemszámnál a korábban tanult szorzási túlcsordulás-ellenőrzés továbbra is szükséges.

A várt állapotok:

Injektált hiba	Elért kísérletek száma	Élő allokáció a `build` visszatérésekor	Élő allokáció a teszt végén
Nincs	3	2	0
Első kísérlet	1	0	0
Második kísérlet	2	0	0
Harmadik kísérlet	3	0	0

A sikeres ágon a teszt az eredeti négy elem megőrzését is ellenőrzi. A harmadik kísérlet kudarcakor pedig a régi tömb még él, ezért a cleanup jogosan felszabadítja.

A nem injektált foglalások ténylegesen is meghiúsulhatnak. Ilyenkor a teszt korábbi hibát észlelhet a tervezettnél: ez nem feltétlenül cleanup-hiba, hanem a tesztkörnyezet elégtelen erőforrása.

8.5.3 Elkülönített lifetime-hibák

A következő `bad.c` **szándékosan hibás, negatív teszt**. Mindig csak egy hibamódot válasszunk: az első hiba után a program további viselkedésére már nincs nyelvi garancia.

```c
#include <stdlib.h>

int main(int argc, char **argv)
{
    if (argc != 2)
        return 2;

    int *p = malloc(2 * sizeof *p);
    if (p == NULL)
        return EXIT_FAILURE;
    p[0] = 42;
    p[1] = 7;

    switch (argv[1][0]) {
    case 'u':
        free(p);
        return p[0];       /* UB: use-after-free */

    case 'd':
        free(p);
        free(p);           /* UB: double-free */
        return EXIT_SUCCESS;

    case 'r': {
        int *slice = p + 1;
        int *next = realloc(p, 4 * sizeof *p);
        if (next == NULL) {
            free(p);
            return EXIT_FAILURE;
        }
        p = next;
        int value = *slice; /* UB: stale slice */
        free(p);
        return value;
    }

    default:
        free(p);
        return 2;
    }
}
```

Az allokáció tulajdonosa mindhárom ágban a `main`; a `slice` csak kölcsönzött hozzáférés. A kezdeti foglalás hibája szabályos hibakilépés. Az `r` ág sikertelen átméretezéskor szabályosan felszabadítja a régi tárolót.

A hibák azonban különböznek:

- **Use-after-free:** az objektum élettartama már véget ért, mégis olvasunk belőle.
- **Double-free:** ugyanazon, már megszűnt allokációt próbáljuk ismét felszabadítani.
- **Stale slice:** az átméretezés sikerült, de a korábbi tárolóból képzett nézetet használjuk.

ISO C17 szabály: sikeres `realloc` után a régi allokáció élettartama véget ér. Ez akkor is igaz, ha a visszaadott pointer ugyanazt a numerikus címet reprezentálná. Ne a régi és az új pointer összehasonlításával próbáljuk eldönteni az érvényességet.

A helyes megoldás az index megtartása és a pointer újraképzése. Az alábbi részlet az `r` ág helyes változata, az előző inicializálással együtt értendő:

```
size_t index = 1; /* az eredeti ket inicializalt elem egyike */

int *next = realloc(p, 4 * sizeof *p);
if (next == NULL) {
    free(p);
    return EXIT_FAILURE;
}
p = next;

int *slice = p + index;
int value = *slice;
free(p);                    /* slice ettol kezdve ervenytelen */
return value;
```

Zsugorításnál az index és a nézet teljes hosszának érvényességét külön ellenőrizni kell. A régi kapacitás nem bizonyíték az új tartomány elérhetőségére.

8.5.4 Diagnosztikai eszközök: szerepek és korlátok

A fordítási parancsok külön laborhoz tartoznak. Az ellenőrzésekhez ne legyen definiálva az `NDEBUG`, mert az kikapcsolná az `assert` vizsgálatait.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -g -O0 fi.c -o fi
gcc -std=c17 -Wall -Wextra -Wpedantic -g -O0 bad.c -o bad

clang -std=c17 -Wall -Wextra -Wpedantic -g -O1 \
  -fno-omit-frame-pointer -fsanitize=address fi.c -o fi-asan

clang -std=c17 -Wall -Wextra -Wpedantic -g -O0 \
  -fno-omit-frame-pointer -fsanitize=address bad.c -o bad-asan
```

A hibás fájlnál fordítói figyelmeztetés is megjelenhet; ez nem garantált diagnózis.

Eszköz	Elsődleges szerep	Fontos korlát
AddressSanitizer, ASan	Többek között felszabadítás utáni hozzáférés és érvénytelen felszabadítás felismerése	Csak a végrehajtott utakat látja; nem teljes nyelvi érvényességvizsgáló
LeakSanitizer, LSan	Elérhetőségi vizsgálat alapján potenciális szivárgások keresése	Az elérhető, de szükségtelenül megtartott memóriát rendszerint nem tekinti leaknek

Eszköz	Elsődleges szerep	Fontos korlát
Valgrind Memcheck	Futás közbeni memóriahozzáférés- és leakdiagnosztika	Jelentős futási többletköltség; platform- és verziófüggő támogatás

A megadott Linux-profilban az ASan runtime leakellenőrzést is támogathat; ezt az `ASAN_OPTIONS` változó `detect_leaks=1` beállításával lehet kérni. Az integrációt a tényleges compiler/runtime pároson ellenőrizni kell. Az önálló `-fsanitize=leak` másik lehetséges profil, nem az ASan hozzáférés-ellenőrzésének helyettesítője.

Valgrindhoz például a 3.24.0 kiadás rögzíthető kiinduló verzióként. A szokásos összehasonlítást a sanitizer nélküli, debug információt tartalmazó binárison végezzük. A Memcheck `--leak-check=full` és `--show-leak-kinds=all` kapcsolója részletesebb jelentést kér; CI-ben külön döntés, mely leakkategóriák okozzanak hibakilépést.

Az `u`, `d` és `r` esetet külön folyamatban vizsgáljuk. Nincs garantált UB-kimenet vagy garantált szövegű jelentés.

Különösen fontos: ha a `realloc` helyben dolgozik, a stale pointer címe továbbra is címezhető memóriára mutathat. Az ASan hallgatása ilyenkor **nem igazolja a C-kód helyességét**. Mozgatásos diagnosztikai esetet úgy kényszeríthetünk ki, hogy még az élő régi tömb mellett külön új tömböt foglalunk, átmásoljuk a két inicializált elemet, majd felszabadítjuk a régit. Ez célzott tesztmodell, nem a valódi `realloc` implementációjának mérése.

8.5.5 Leak, élő cache és RSS

A **memóriaszivárgás** tulajdonlási probléma: a felszabadítási kötelezettség nem teljesül, például mert elveszítjük az utolsó használható tulajdonosi hivatkozást.

Egy élő cache más helyzet. Ha szándékosan birtokolja az objektumokat, és van ürítési vagy kilakoltatási szabálya, a memória megtartása lehet helyes. A korlátlanul növekvő cache ugyanakkor súlyos üzemeltetési hiba lehet, még ha a leakdetektor mindent elérhetőnek lát is.

Az RSS (*resident set size*) az operációs rendszer által elszámolt rezidens memória mérőszáma, nem az alkalmazás élő `malloc`-allokációinak összege. Linuxon többek között az allocator visszatartott területei, töredezettség, stackek és leképezések is befolyásolják. A `free` nem ígér azonnali RSS-csökkenést.

Megfigyelés	Amit nem bizonyít
`live == 0`, de magas RSS	Önmagában nem bizonyít leaket
„Still reachable” jelentés	Nem bizonyítja a cache méretpolitikájának helyességét
Nincs leakjelentés	Nem zárja ki az elérhető memória indokolatlan megtartását
Stabil RSS	Nem zárja ki kis mennyiségű memória folyamatos elvesztését

Ezért együtt vizsgáljuk a tulajdonosi állapotot, az allokációk és kért C bájtok mérlegét, a cache elemszámát és az OS-mutatókat.

Gyakori hibás javítás, hogy `free(p)` után csak `p = NULL` kerül a kódba. Ez hasznos állapotfrissítés, de más aliasokat nem módosít, és korábbi hibás hozzáférést nem tesz helyessé.

8.5.6 Feladatok

8.5.6.1 A hibautak levezetése

Írd le a második és harmadik foglalás kudarcakor élő erőforrásokat, majd a cleanup hatását. Miért marad változatlan a `live` értéke sikertelen átméretezéskor?

8.5.6.2 Új foglalási pont beépítése

Egészítsd ki a `build` függvényt egy 16 C bájtos ideiglenes munkaterülettel. Foglalása az átméretezés után, a tulajdonlás átadása előtt történjen. A munkaterület siker esetén sem kerülhet a kimenetbe. Bővítsd az ellenőrzéseket.

8.5.6.3 Stale slice és félrevezető memóriaadat

Egy feltételezett vizsgálatban az `r` eset nem ad ASan-jelentést, a teljes cleanup után pedig az RSS nem csökken. Bizonyítja-e bármelyik megfigyelés a helyességet vagy a leaket? Adj célzott javítást és ellenőrzési tervet.

8.5.7 Megoldások

8.5.7.1 A hibautak levezetése

A második kísérlet előtt csak a tömb él: $L = 1$. A címke foglalása sikertelen, ezért a null címke felszabadítása nem csinál semmit; a tömb felszabadítása után $L = 0$.

A harmadik kísérlet előtt a tömb és a címke is él: $L = 2$. Sikertelen, pozitív méretű `realloc` esetén a régi tömb megmarad. A két cleanup-művelet sorrendben $2 \to 1 \to 0$ változást eredményez.

Ellenőrzési feltétel: hibás visszatéréskor üres kimenet és nulla élő allokáció.

8.5.7.2 Új foglalási pont beépítése

A `build` elején, minden lehetséges ugrás előtt adjuk hozzá:

```
unsigned char *scratch = NULL;
```

A sikeres átméretezés és az elemek inicializálása után, de a kimenet kitöltése előtt:

```
scratch = fi_malloc(f, 16);
if (scratch == NULL)
    goto done;
```

A közös cleanup elején:

```
fi_free(f, scratch);
```

Ezek a korábbi C17-fájlba illesztendő részletek. A `scratch` tulajdonosa végig a `build`; minden visszatérési úton felszabadul, kölcsönzött nézet nem kerül ki belőle.

A sikeres ág elvárt hívásszáma négyre változik. A `main` a mért hívásszámból képezi a hibapontokat, ezért az új negyedik esetet is lefedi.

Ellenőrzési feltétel: sikeres visszatéréskor továbbra is csak két allokáció él; minden injektált hibánál nulla.

8.5.7.3 Stale slice és félrevezető memóriaadat

Egyik megfigyelés sem elegendő. Az ASan nem feltétlenül látja a helyben végzett átméretezéshez tartozó nyelvi élettartamsértést. A változatlan RSS pedig lehet allocator-retenció következménye.

A javítás: az elemindexet őrizzük meg, majd sikeres átméretezés után az új pointerből képezzük újra a nézetet. A vizsgálati terv külön tartalmazza a sikertelen átméretezést, a sikeres tartalommegőrzést és a mozgatást kényszerítő negatív tesztet.

Ellenőrzési feltétel: a javított út nem használ régi aliast, minden hibakimenet rendezett, és az erőforrásmérleg visszaáll. A sanitizer és a leakdetektor ezt kiegészítő bizonyíték, nem teljes helyességi bizonyítás.

8.5.8 Összefoglaló

Az allokációs hibák teszteléséhez nem tényleges memóriahiány, hanem pontosan kiválasztott hibapont kell. Az N-edik foglalás elutasítása ellenőrizhetővé teszi a részleges felépítés cleanup-ját. Az erőforrásmérleg a tulajdonlási szerződést vizsgálja; az ASan, LSan és Memcheck végrehajtott hibákat keres. Egyik sem helyettesíti az élettartam-szabályokat: sikeres `realloc` után a régi nézeteket újra kell képezni, az RSS pedig nem tekinthető leakszámlálónak.

9 struct, union, alignment és bináris layout

9.1 Struktúrák, tagok és értékmásolás

Egy C API gyakran összetartozó adatokat ad át: például egy üzenet azonosítóját, címkéjét, hosszát és tartalmát. A `struct` ezeket egyetlen aggregált értékbe foglalja. Ettől azonban még nem válik automatikusan erőforrás-kezelővé: egy pointertag másolása nem másolja a mögötte lévő tárolót, és nem ruház át tulajdonjogot.

A Tulajdonlási szerződés és cleanup ágak részben megismert owner–borrowed megkülönböztetés ezért a struktúrák tagjaira is érvényes. A pointereket tartalmazó deklarációk olvasásához a Deklarátorok: tömb-, pointer- és függvénykapcsolatok rész ad előfeltételt.

Az alábbi példák ISO C17-et használnak. A méretek C bájtban értendők: a `sizeof` egysége nem általánosan nyolcbites oktett.

9.1.1 Típusnév, adattag és aggregált érték

A struktúratípus névvel ellátott adattagokból (*member*) álló típus. Egy struktúraobjektum tartalmazza saját tagobjektumait; egy beágyazott struktúra vagy fix méretű tömb ténylegesen része a befoglaló objektumnak. Pointertag esetén viszont csak a pointerobjektum része: a referált tároló külön létezik.

Az alábbi `blob.h` egy másolt tartalmú, változó hosszúságú adatobjektum publikus interfésze:

```
#ifndef BLOB_H
#define BLOB_H

#include <stddef.h>

struct blob_meta {
    unsigned sequence;
    char label[8];
};

struct blob_view {
    struct blob_meta meta;
    const unsigned char *data;
    size_t size;
};

struct blob;
typedef struct blob Blob;

Blob *blob_create(struct blob_meta meta,
                  const void *src, size_t size);
```

```
struct blob_view blob_get_view(const Blob *blob);

void blob_destroy(Blob *blob);

#endif
```

A `blob_meta` a struktúra címkéje, angolul *struct tag*. ISO C17-ben a típus neve itt `struct blob_meta`, nem önmagában `blob_meta`. A címkék külön névtérben vannak a közönséges azonosítóktól.

A `typedef struct blob Blob;` ezzel szemben típusnevet vezet be a közönséges azonosítók névterébe. A `Blob` és a `struct blob` ugyanazt a típust jelöli: a `typedef` nem hoz létre új, elkülönült típust, és nem ad ownership-szemantikát.

A tagnevek a saját struktúrájukon belül értelmezendők. A `blob_view` `meta` tagja beágyazott struktúra, a `label` pedig tömbtag. A kétféle tagelérés:

- `value.meta.sequence`: közvetlenül egy struktúraérték tagjának elérése;
- `pointer->meta.sequence`: pointeren keresztüli tagelérés.

A `p->member` jelentése `(*p).member`. Kiértékelt hozzáféréskor `p` megfelelő típusú, élő objektumra kell mutasson; a `->` nem ellenőrzi sem a null pointert, sem az élettartamot.

9.1.2 Inicializálás és értékmásolás működés közben

Az aggregált inicializáló a tagokat inicializálja. A *designated initializer* tagnevekkel jelöli a célokat, így nem kell fejben követni a deklarációs sorrendet. A nem megadott tagok ilyen inicializáláskor a statikus inicializálásnak megfelelő értéket kapják: például az egész tag nulla, a pointertag null pointer lesz. Ez szemantikai szabály, nem „minden reprezentációs bájt nulla” ígéret.

A `(struct blob_meta){ ... }` alak összetett literál, angolul *compound literal*. ISO C17-ben ez névtelen objektumot hoz létre. Függvényblokkon belül automatikus tárolási idejű, az adott blokk végéig él; nem csak a pontosvesszőig.

A következő teljes `demo.c` a később megadott implementációval fordítható:

```
#include "blob.h"

#include <assert.h>
#include <stdlib.h>

int main(void)
{
    unsigned char input[] = {0x41u, 0x42u};

    Blob *owner = blob_create(
        (struct blob_meta){
            .sequence = 7u,
            .label = "job"
        },
        input,
        sizeof input
    );
```

```
    if (owner == NULL) {
        return EXIT_FAILURE;
    }

    struct blob_view first = blob_get_view(owner);
    struct blob_view second = first;
    struct blob_view *alias = &first;

    second.meta.label[0] = 'J';
    assert(first.meta.label[0] == 'j');
    assert(second.data == first.data);

    alias->meta.sequence = 8u;
    assert(first.meta.sequence == 8u);
    assert(second.meta.sequence == 7u);

    input[0] = 0xffu;
    assert(first.data[0] == 0x41u);

    second = (struct blob_view){
        .meta = {.sequence = 9u}
    };
    assert(second.data == NULL);
    assert(second.size == 0u);

    blob_destroy(owner);

    /* first.data innen nem használható az adat elérésére. */
    return EXIT_SUCCESS;
}
```

Az első két nézet inicializálása után `first` és `second` külön objektum. A `second.meta.label[0]` módosítása ezért nem érinti `first` címkéjét. Bár tömbök között önálló értékadás nem végezhető, a struktúra másolása a fix méretű tömbtagok tartalmát is másolja.

Az `alias` nem új struktúraérték: ugyanahhoz a `first` objektumhoz biztosít másik hozzáférési utat. Az azon keresztüli módosítás tehát `first` módosítása.

A kezdeti `second = first` jellegű inicializálás és a későbbi struktúra-értékadás hasonló értékmásolást eredményez, de külön művelet. Értékadásnál a cél már létező, módosítható objektum, a forrás típusa pedig kompatibilis struktúratípus kell legyen. Például `const` adattagot tartalmazó struktúra nem rendelhető felül ilyen értékadással.

9.1.3 Shallow copy és tulajdonlási szerződés

A struktúramásolás a pointertag értékét másolja, nem a referált objektumot. Ezt a referált erőforráshoz képest sekély másolásnak, *shallow copy*-nak nevezzük.

A példában mindkét nézet `data` tagja ugyanarra a tárolóra mutat. A `meta` viszont beágyazott érték: külön másolat, nem élő kapcsolat a `Blob` metaadataihoz.

Elem	Tulajdonosi szerep	Tároló és élettartam	Érvénytelenítő esemény
`input`	A hívó által kezelt forrásobjektum	Automatikus tömb, a blokk végéig él	Kilépés a blokkból
`owner`	A dinamikus `Blob` lezárásáért felelős pointerváltozó	Maga a pointer automatikus objektum	A változó megszűnése önmagában nem szabadít fel
`*owner` és adattartalma	Egyetlen birtokolt allokáció	Sikeres létrehozástól a `blob_destroy` hívásig	`blob_destroy(owner)`
`first`, `second`	Saját metaadatérték, kölcsönzött adatelérés	A nézetobjektumok a blokk végéig élnek	Az adattároló felszabadítása érvényteleníti a kölcsönzött hozzáférést
`alias`	Nem tulajdonló pointer `first` objektumhoz	`first` élettartamán belül használható	`first` élettartamának vége

A `const unsigned char *` csak az ezen a pointeren keresztüli írást tiltja. Nem fejez ki tulajdonjogot, és önmagában nem garantálja, hogy más hozzáférési út sem módosíthatja az adatot.

Ha ugyanilyen pointertag tulajdonosi szerződésű lenne, a sekély másolás veszélyes lehetne. Két struktúra ugyanazt a tárolót tekinthetné sajátjának, majd mindkettő felszabadítaná: ez dupla felszabadításhoz és nem definiált viselkedéshez vezetne. C-ben ezt nem oldja meg automatikus másoló konstruktor vagy destruktor. Explicit klónozási, ownership-átadási vagy tiltott másolási szerződés kell.

A másolás a tagértékekre vonatkozik, nem a teljes objektumreprezentáció azonosságára. Struktúrára nincs beépített `==`, a `memcmp` pedig nem általános értékegyenlőség-vizsgálat. A padding szerepét a Padding, alignment és offsetof rész tárgyalja.

9.1.4 Incomplete type és opaque API

A fejlécben a

```
struct blob;
```

előzetes deklaráció (*forward declaration*). Bejelenti a típust, de nem közli a tagjait, így az ott még befejezetlen típus, *incomplete type*.

A kliens deklarálhat rá pointert, átadhatja függvénynek és tárolhatja. Nem definiálhat azonban `Blob` objektumot érték szerint, nem használhatja rá a `sizeof` operátort, és nem érheti el a rejtett tagokat. A pointer mérete ismert lehet attól függetlenül, hogy a referált típus mérete nem ismert.

Az *opaque handle* itt `Blob *`: a kliens a műveleteket ismeri, a reprezentációt nem. Ez nem hardveres hozzáférés-védelem, hanem típusos API-határ. Lehetővé teszi az implementáció változtatását anélkül, hogy a kliens a belső tagelrendezésre támaszkodna; a teljes ABI-stabilitáshoz további szerződések is szükségesek.

Az API feltételei:

- A `blob_create` a metaadatot és a forrás `size` C bájtját másolja. A forrásnak a hívás idején olvashatónak kell lennie; `NULL` csak nulla hosszal megengedett.

- Siker esetén a hívó egyetlen felszabadítandó tulajdonosi handle-t kap. Hibás null/hossz pár, mérettúlcsordulás vagy allokációs hiba esetén az eredmény `NULL`.
- A `blob_get_view` élő, nem null handle-t vár. A visszaadott nézet nem tulajdonos; adatelérése a `Blob` megsemmisítéséig érvényes.
- A `blob_destroy(NULL)` megengedett. Ugyanazt az allokációt kétszer megsemmisíteni tilos.

9.1.5 Flexible array member és ellenőrzött méretszámítás

A rejtett típus utolsó tagja rugalmas tömbtag, *flexible array member*. ISO C17-ben ilyen, méret nélkül deklarált tömbtag több névvel rendelkező tagot tartalmazó struktúra utolsó tagja lehet.

A teljes `blob.c`:

```
#include "blob.h"

#include <stdint.h>
#include <stdlib.h>
#include <string.h>

struct blob {
    struct blob_meta meta;
    size_t size;
    unsigned char data[];
};

Blob *blob_create(struct blob_meta meta,
                  const void *src, size_t size)
{
    if (src == NULL && size != 0u) {
        return NULL;
    }

    if (size > SIZE_MAX - sizeof(Blob)) {
        return NULL;
    }

    Blob *blob = malloc(sizeof *blob + size);
    if (blob == NULL) {
        return NULL;
    }

    blob->meta = meta;
    blob->size = size;

    if (size != 0u) {
        memcpy(blob->data, src, size);
    }

    return blob;
}

struct blob_view blob_get_view(const Blob *blob)
```

```
{
    return (struct blob_view){
        .meta = blob->meta,
        .data = blob->size == 0u ? NULL : blob->data,
        .size = blob->size
    };
}

void blob_destroy(Blob *blob)
{
    free(blob);
}
```

A `data` nem pointertag. Elemei a struktúrával közös allokációban találhatók, ezért nincs második allokáció és második felszabadítás. Az input tartalmának másolása miatt a létrehozott objektum független a forrás későbbi módosításától vagy megszűnésétől.

Legyen $S = \text{sizeof}(\texttt{Blob})$, n az elemek száma, E az elemméret. Az általános allokációs méret:

$$N = S + nE.$$

Itt `unsigned char` elemek vannak, ezért $E = 1$ C bájt, így $N = S + n$.

Bizonyításvázlat a túlcsordulás kizárására. Mivel S ábrázolható `size_t` érték, a `SIZE_MAX`$-S$ kivonás biztonságos. Ha

$$n \leq \texttt{SIZE_MAX} - S,$$

akkor mindkét oldalhoz S-t adva

$$S + n \leq \texttt{SIZE_MAX}.$$

Az összeadás tehát nem fordul körbe. Nagyobb elemtípusnál még a szorzás előtt ezt kell ellenőrizni:

$$n \leq \left\lfloor \frac{\texttt{SIZE_MAX} - S}{E} \right\rfloor.$$

A sikeres méretellenőrzés nem garantál sikeres allokációt; ezért külön kell vizsgálni a `malloc` eredményét.

A `sizeof(Blob)` nem tartalmazza a kért dinamikus elemszámot, de tartalmazhat paddinget. Nem szabad feltételezni, hogy pontosan megegyezik a `data` kezdőeltolásával. A `sizeof(Blob) + n` alak elegendő helyet foglal, az alkalmazás pedig kizárólag a tárolt `size` szerinti `[0,size)` tartományt tekinti tartalomnak.

A rugalmas tömbtagra nem alkalmazható `sizeof`. Az ilyen struktúra ISO C17-ben másik struktúra tagja vagy tömb eleme sem lehet; pointerként természetesen tárolható. A struktúra-értékadás nem biztosítja a rugalmas tömbrész tartalmának másolását: klónozásához külön allokáció és explicit tartalommásolás szükséges.

9.1.5.1 Fordítási profil

A három fájlhoz választott laborprofil: Linux, x86-64 System V ABI, LP64; a `lab-next` profil (profiltábla). Ez profil-előírás, nem itt ellenőrzött futtatási környezet. A verziózott parancsnevek mögötti pontos kiadást külön rögzíteni kell.

```
gcc-14 -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -O0 -g blob.c demo.c -o demo-gcc

clang-19 -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -O0 -g blob.c demo.c -o demo-clang
```

A példák itt nem kerültek lefordításra vagy futtatásra. Az `assert` feltételek az elvárt viselkedést rögzítik, nem közölt mérési eredmények.

9.1.6 Gyakori hibák és javításuk

„A struktúra másolása mindent lemásol.” A pointer értékét igen, a referált tárolót nem. A javítás explicit klónozás vagy dokumentált borrowed kapcsolat.

„Az owner pointer nullázása minden nézetet érvénytelenít.” A tároló felszabadítása érvényteleníti a hozzáféréseket; egyetlen változó nullázása nem módosítja a többi pointert. A nézetek élettartamát kell korlátozni.

„A compound literal örökké él.” Blokkon belüli compound literal címét visszaadni dangling pointert eredményezhet. Érték szerinti visszaadás vagy megfelelően birtokolt tároló kell.

„A struktúra kiírható hálózatra a `sizeof` méretével.” A belső reprezentáció nem hordozható protokollszerződés. Ennek helyes alternatíváját a Byte order és biztonságos szerializálás rész vezeti le.

9.1.7 Feladatok

9.1.7.1 Másolat vagy alias?

Magyarázd meg, miért változik `first.meta.sequence` az `alias` használatakor, de miért nem változik `first.meta.label` a `second` módosításakor! Használható marad-e `first.meta.sequence` a `blob_destroy` után?

9.1.7.2 Önálló klón

Egészítsd ki az API-t `Blob *blob_clone(const Blob *source);` függvénnyel! Null forrásra adjon `NULL`-t. Siker esetén a klón az eredeti megsemmisítése után is használható legyen.

9.1.7.3 Három hibás feltételezés diagnózisa

Értékeld külön:

- A kliens `sizeof(Blob)` alapján foglalna.
- Az implementáció ellenőrzés nélkül számítaná a `sizeof(Blob) + size` összeget.
- Két megfelelő méretű allokáció között a `*destination = *source` műveletet tartalomklónozásnak tekintené.

Mindegyiknél nevezd meg a hibafajtát és a javítást!

9.1.8 Megoldások

9.1.8.1 Másolat vagy alias?

Az `alias` ugyanarra a `first` objektumra mutat. A `second` ezzel szemben külön struktúra, saját beágyazott metaadattal és címketömbbel.

A `blob_destroy` után `first.meta.sequence` továbbra is olvasható: a nézet saját tagja, amelynek élettartama nem ért véget. A `first.data` által korábban elért tartalom viszont már nem használható. Ellenőrzési feltétel, hogy külön kezeld a nézetobjektum és a referált tároló élettartamát.

9.1.8.2 Önálló klón

A deklaráció a fejlécbe, az alábbi kiegészítés a `blob.c` fájlba kerül:

```
Blob *blob_clone(const Blob *source)
{
    if (source == NULL) {
        return NULL;
    }

    return blob_create(
        source->meta,
        source->size == 0u ? NULL : source->data,
        source->size
    );
}
```

A nem null forrásnak a hívás teljes ideje alatt élnie kell. A függvény új allokációt hoz létre, ezért a két objektum külön tulajdonosi kötelezettség. Hibakor `NULL` az eredmény, a forrás változatlan.

Ellenőrzési feltétel: az eredeti megsemmisítése után a klónból kapott új nézet tartalma egyezzen az eredetileg bemásolt tartalommal; végül a klónt is pontosan egyszer kell megsemmisíteni.

9.1.8.3 Három hibás feltételezés diagnózisa

A kliensoldali `sizeof(Blob)` befejezetlen típus miatt fordításkor diagnosztizálandó korlátsértés. A foglalást az implementációra kell bízni.

Az ellenőrizetlen `size_t` összeadás körbefordulhat. Maga az előjel nélküli körbefordulás definiált, de túl kicsi allokációt eredményezhet; az ezt követő túlindexelés már UB. Az összeadás előtti korlátellenőrzés szükséges.

A struktúra-értékadás nem klónozza a rugalmas tömbrész tartalmát. A javítás új allokáció és explicit tartalommásolás, például a `blob_clone` használata.

9.1.9 Összefoglaló

A `struct` beágyazott tagjai az aggregált érték részei; a pointertagok referált erőforrásai nem. A másolat és az alias külön fogalom, az ownership pedig explicit API-szerződés. Az opaque típus elrejti a reprezentációt, a flexible array member pedig egyetlen allokációban kapcsolhat fejlécet és változó hosszúságú tartalmat. A helyességhez mindehhez élettartamkorlát, ellenőrzött méretszámítás és egyértelmű hibakezelés szükséges.

9.2 Padding, alignment és offsetof

A struktúra tagjainak logikai sorrendje nem jelenti azt, hogy a tagok hézagmentesen követik egymást a memóriában. A fordító az igazítási követelmények miatt kitöltést (*padding*) helyezhet közéjük és az objektum végére. Ez befolyásolja a memóriaigényt, a tömbök lépésközét és a bináris interfészek kompatibilitását, de nem része a struktúrával modellezett logikai értéknek.

A Struktúrák, tagok és értékmásolás rész értékszemantikájára, valamint az Alignment és speciális allokációk igazításfogalmára építve most egy konkrét elrendezést vezetünk le. Ezután olyan mérőprogramot készítünk, amely összehasonlítja a levezetést a helyi fordító eredményével. A közölt számok **levezetett várakozások, nem futtatással kapott eredmények**.

9.2.1 A nyelvi garancia és az ABI választása

ISO C17. Egy közönséges, nem bitmező tagokat tartalmazó struktúrában a tagok címei deklarációs sorrendben növekednek. Az első tag előtt nincs padding, a tagok között és az utolsó tag után azonban lehet. A struktúra `sizeof` értéke a teljes objektum mérete, a kitöltéssel együtt.

Kétféle kitöltést különböztetünk meg:

- **Belső padding:** két tag közötti, egyik taghoz sem tartozó tárolási terület.
- **Záró padding:** az utolsó tag vége és a struktúraobjektum vége közötti terület.

A méret és az igazítás mértékegysége a C bájt, azaz **C_byte**. Egy ilyen bájt bitszáma `CHAR_BIT`; nem minden C implementációban nyolc.

A `sizeof(T)` a típus objektumainak méretét, az `_Alignof(T)` az igazítási követelményét adja meg. A `<stddef.h>` fejléc `offsetof(T, member)` makrója a tag kezdetének eltolását szolgáltatja a struktúra kezdetétől, ugyancsak C bájtban. Mindhárom eredmény `size_t` típusú.

Az `offsetof` nem egy konkrét példány címét méri: a típus elrendezésének tulajdonságát kérdezi le. Bitmezőre nem használható. Nem érdemes null pointerből képzett tagcímekkel „házilag” helyettesíteni.

ABI-laborfeltevés. A következő levezetés profilja Linux, x86-64 System V ABI, LP64 adatmodell, nyolcbites C bájt, természetes tagigazítás, csomagolási kapcsolók és attribútumok nélkül. A választott fordítók a `lab-next` profil (profiltábla), a libc a `lab-next` profil (profiltábla); a helyi verziókat és a tényleges párosítást külön rögzíteni kell.

Ebben a profilban a példánkhoz szükséges típusjellemzők:

Típus	Méret, C_byte	Igazítás, C_byte
`unsigned char`	1	1
`uint16_t`	2	2
`uint32_t`	4	4

Az exact-width típusok általánosan nem kötelezően elérhetők. A mérőprogram ezért ellenőrzi jelenlétüket. A táblázat ABI-feltevéseit a program helyben ellenőrzi; itt nem áll rendelkezésre külön ellenőrzött ABI-forrásjegyzet.

9.2.2 Az elrendezés levezetése

Az alábbi C17-részlet lesz a mérés alapja; a teljes programban is szerepel:

```
struct Record {
    unsigned char kind;
    uint32_t count;
    uint16_t code;
};
```

9.2.2.1 Igazítási kerekítés

Legyen x a következő szabad eltolás, $a > 0$ pedig a következő tag igazítása. A legkisebb megfelelő eltolás:

$$\mathrm{round_up}(x, a) = a \left\lceil \frac{x}{a} \right\rceil .$$

Maradékos osztással ugyanez:

$$\mathrm{round_up}(x, a) = x + ((a - (x \bmod a)) \bmod a).$$

Bizonyításvázlat. Ha $x \bmod a = 0$, nincs szükség kitöltésre. Egyébként, ha a maradék r, akkor pontosan $a - r$ bájt hozzáadásával érjük el a következő a-val osztható eltolást. Ennél kisebb nemnegatív növekmény nem ad megfelelő igazítást.

Ez matematikai képlet. C-kódban a gyakori `(x + a - 1) / a * a` alak túlcsordulhat. A maradékból számított padding hozzáadásakor is ellenőrizni kell, hogy az összeg ábrázolható-e.

A választott ABI közönséges struktúráinál a levezetési modell:

1. A következő tag kezdetét felkerekítjük a tag igazítására.
2. Hozzáadjuk a tag méretét.
3. Az utolsó tag végét felkerekítjük a struktúra igazítására.

Itt a struktúra igazítása a tagigazítások maximuma:

$$A = \max(1, 4, 2) = 4.$$

Ez a konkrét profil szabálya, nem minden C implementációra érvényes „mindig a minimumot választja” garancia.

9.2.2.2 Belső és záró kitöltés

A `kind` az első tag, ezért eltolása nulla, vége pedig $0 + 1 = 1$.

A `count` négybájtos igazítást igényel:

$$\text{round_up}(1, 4) = 4.$$

A következő tag tehát nem az 1., hanem a 4. bájton kezdődik. Közéjük három bájt padding kerül. A `count` vége $4 + 4 = 8$.

A `code` kétbájtos igazítása már teljesül:

$$\text{round_up}(8, 2) = 8.$$

Vége $8 + 2 = 10$. A teljes méretet még négy többszörösére kerekítjük:

$$S = \text{round_up}(10, 4) = 12.$$

Tartomány, C_byte	Tartalom	Hossz
$[0, 1)$	`kind`	1
$[1, 4)$	belső padding	3
$[4, 8)$	`count`	4
$[8, 10)$	`code`	2
$[10, 12)$	záró padding	2

A tagok méretének összege $1 + 4 + 2 = 7$, de a struktúráé 12 C bájt.

9.2.2.3 Miért kell a záró padding?

ISO C17. A tömbelemek folytonosan helyezkednek el; az egymást követő elemek közötti lépésköz az elemtípus `sizeof` értéke. Nincs ezen felüli, külön tömbelemközi rés.

Ha a struktúra mérete 10 lenne, a második elem kezdete az elsőhöz képest 10 bájttal tolódna el. Ez nem osztható néggyel, így nem biztosítaná a szükséges igazítást.

A 12 bájtos mérettel:

$$\text{elemkezdés}(k) = B + 12k,$$

ahol B a megfelelően igazított tömbkezdet. Mivel $12k$ osztható néggyel, minden elem megfelelő helyre kerül.

Elem	Kezdet	`count` kezdete	Objektum vége
`records[0]`	0	4	12
`records[1]`	12	16	24
`records[2]`	24	28	36

Az eltolások a tömb kezdetéhez viszonyított értékek, nem univerzális memóriacímek.

9.2.3 A levezetés összevetése a helyi eredménnyel

A következő önálló program ISO C17-et és a megadott laborprofilt célozza. Nem olvassa ki a padding tartalmát: méreteket, igazításokat és eltolásokat vizsgál.

```
#include <limits.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>

#if CHAR_BIT != 8
#error "This lab requires 8-bit C bytes."
#endif

#if !defined(UINT16_MAX) || !defined(UINT32_MAX)
#error "This lab requires uint16_t and uint32_t."
#endif

struct Record {
    unsigned char kind;
    uint32_t count;
    uint16_t code;
};

static int check(const char *name, size_t actual, size_t expected)
{
    int same = actual == expected;

    printf("%-24s actual=%zu expected=%zu %s\n",
           name, actual, expected, same ? "OK" : "DIFF");
    return same;
}

int main(void)
{
    struct Record records[2];
    int ok = 1;

    ok &= check("sizeof(uint16_t)", sizeof(uint16_t), 2);
    ok &= check("alignof(uint16_t)", _Alignof(uint16_t), 2);
    ok &= check("sizeof(uint32_t)", sizeof(uint32_t), 4);
    ok &= check("alignof(uint32_t)", _Alignof(uint32_t), 4);
```

```
    ok &= check("Record.kind", offsetof(struct Record, kind), 0);
    ok &= check("Record.count", offsetof(struct Record, count), 4);
    ok &= check("Record.code", offsetof(struct Record, code), 8);
    ok &= check("alignof(Record)", _Alignof(struct Record), 4);
    ok &= check("sizeof(Record)", sizeof(struct Record), 12);
    ok &= check("sizeof(records)", sizeof records, 24);

    return ok ? 0 : 1;
}
```

A `check` szöveges neveket kölcsönöz statikus tárolási idejű sztringliterálokból; nem tárolja el és nem módosítja őket. Nincs felszabadítási feladata vagy futás közbeni invalidáló művelete. A `records` automatikus objektum, de inicializálatlansága itt nem probléma: a `sizeof` nem olvassa az elemek értékét.

Fordítás a külön tanulói laborban, a megfelelő verziókra beállított parancsnevekkel:

```
gcc -std=c17 -O2 -Wall -Wextra -Wconversion -pedantic-errors \
    layout.c -o layout-gcc

clang -std=c17 -O2 -Wall -Wextra -Wconversion -pedantic-errors \
    layout.c -o layout-clang
```

A program kimenetében az `actual` oszlop lesz a helyi megfigyelés, az `expected` a levezetett érték. A várt visszatérési kód nulla. Az egyes nem nulla kód **profilkülönbséget** jelez, nem önmagában hibás C implementációt.

Eltéréskor először a targetet, a fordítóverziót és a tényleges kapcsolókat vizsgáld. Különösen fontosak a csomagolási beállítások és a korábban hatályba lépett pragmák. Az egyezés sem bizonyít általános hordozhatóságot: csak ezt a típust és ezt a konfigurációt ellenőrzi.

9.2.4 Tagátrendezés: kisebb objektum, más ABI

Ugyanebben a profilban az alábbi, önállóan nem teljes C17-részlet kevesebb kitöltést igényel:

```
struct CompactRecord {
    uint32_t count;
    uint16_t code;
    unsigned char kind;
};
```

Tulajdonság	`Record`	`CompactRecord`
`count` eltolása	4	0
`code` eltolása	8	4
`kind` eltolása	0	6
Tagok utáni végpont	10	7
Igazítás	4	4
Teljes méret	12	8
Padding összesen	5	1

A második változatnál a végső kerekítés $\text{round_up}(7, 4) = 8$. Egy N elemű tömb tárolóigénye így $12N$ helyett $8N$ C bájt, vagyis egyharmaddal kisebb. Ez tárolóméret-számítás, nem gyorsulási mérés.

A csökkenő igazítás szerinti rendezés gyakran hasznos heurisztika, de nem általános optimalizálási bizonyítás.

ABI-következmény. A két struktúra nem binárisan felcserélhető. Megváltoznak a tageltolások, a méret és a tömblépésköz. Egy korábban lefordított kliens továbbra is a régi helyeken keresné az adatokat. Az azonos tagnevek és azonos logikai jelentés nem mentik meg az ABI-kompatibilitást.

Publikus struktúra átrendezését ezért interfészváltoztatásként kell kezelni. Az opaque handle-alapú határ lehetőségeit az Opaque handle és stabil C könyvtári interfész tárgyalja.

9.2.5 Logikai érték és teljes objektumreprezentáció

ISO C17. A padding nem tagérték. Tartalmára nem szabad alkalmazásszintű jelentést építeni; struktúrába vagy tagjába történő értéktárolás után a padding bájtjai nem specifikált értékeket vehetnek fel.

Ez pontosabb megfogalmazás, mint pusztán azt mondani, hogy „inicializálatlanok”: az inicializált tagokból sem következik stabil, kanonikus teljes bájtreprezentáció.

Két struktúra logikailag egyenlő lehet, miközben a paddingjük eltér. Emiatt a teljes objektumon végzett `memcmp` nem megfelelő általános értékegyenlőség-vizsgálat.

A helyes összehasonlítás ennél a típusnál, C17-részletként:

```
static int record_equal(struct Record a, struct Record b)
{
    return a.kind == b.kind
        && a.count == b.count
        && a.code == b.code;
}
```

Előfeltétel, hogy mindkét argumentum minden tagja inicializált, érvényes értéket tartalmazzon. Paddinget a függvény nem vizsgál.

Fontos különbségek:

- A struktúraérték-adás átadja a tagértékeket, de nem ígéri a padding azonos másolását.
- A `memcpy` objektumreprezentációt másol, a paddinggel együtt.
- Az előzetes `memset` nem általános garancia arra, hogy későbbi értékadások után is nullák maradnak a kitöltő bájtok.
- A `memcmp` hibás szemantikai választásából nem következik, hogy minden ilyen hívás automatikusan UB.

A teljes struktúratároló naplózása vagy külső félnek átadása korábbi memóriatartalom részleteit is felfedheti a paddingen keresztül. Ez információszivárgási kockázat, nem csupán hordozhatósági probléma. Külső adatformátumhoz explicit mezőkódolás kell; ezt a Byte order és biztonságos szerializálás fejti ki.

9.2.6 Packed elrendezés és igazítatlan hozzáférés

GCC/Clang-bővítés, nem ISO C17. Az alábbi különálló tanítási részlet csomagolt elrendezést kér:

```
struct __attribute__((packed)) PackedRecord {
    unsigned char kind;
    uint32_t count;
    uint16_t code;
};
```

A megadott x86-64 laborprofilban a várt eltolások 0, 1 és 5; a várt méret 7, az igazítás 1. Ezeket külön fordítói próbával kell ellenőrizni.

A `count` így nincs feltétlenül a saját típusának természetes igazításán. A fordító a packed tag közvetlen elérésekor ismeri ezt, és ennek megfelelő kódot állíthat elő. A tag címének közönséges `uint32_t *` pointerként való továbbadása viszont veszélyes: a fogadó kód már megfelelő igazítású objektumot feltételezhet. Ha ez nem teljesül, a dereferálás UB lehet. Az, hogy a hardver bizonyos igazítatlan utasításokat végrehajt, nem írja felül a C szabályait.

A `#pragma pack` szintén implementációspecifikus. Hatása további deklarációkra is átterjedhet; a mentési-visszaállítási fegyelem hiánya különösen veszélyes headerekben.

A packed elrendezés:

- nem oldja meg a bájtsorrendet;
- nem ellenőrzi a bemeneti hosszakat;
- nem ad automatikusan hordozható fájl- vagy hálózati formátumot;
- lassabb vagy összetettebb hozzáférést is eredményezhet.

9.2.7 Cache line és a false sharing előképe

A típusigazítás és a cache line mérete külön fogalom. A négyszeres címre igazított objektum nem feltétlenül cache line-határra igazított.

Egy **külön hardverfeltevésként** 64 bájtos cache line-t használó gépen több kis struktúra kerülhet ugyanabba a vonalba. Ha külön szálak külön objektumokat módosítanak, de azok ugyanazon cache line-on osztoznak, a koherenciaforgalom teljesítményveszteséget okozhat. Ez a *false sharing*.

A jelenség helyesen szinkronizált programban is előfordulhat; nem azonos a nyelvi adatversennyel. A padding csökkentése ezért javíthatja a lokalitást, de ronthatja a szálak közötti elkülönítést. A részletes költségmodell a Cache, locality, alignment és false sharing témája.

9.2.8 Feladatok

9.2.8.1 Magyarázó feladat: tömbméret

A laborprofilban mekkora egy `struct Record records[5]` tömb? Mennyi helyet foglalnak a tagok összesen, és mennyi a padding? Miért nincs külön rés a tömbelemek között?

9.2.8.2 Kódolási feladat: átrendezés

Egészítsd ki a mérőprogramot a `CompactRecord` vizsgálatával. Ellenőrizd minden tag eltolását, az igazítást és a méretet. Készíts hozzá tagonkénti egyenlőségvizsgálatot is.

9.2.8.3 Diagnosztikai feladat: „javítás” packed típussal

Egy szolgáltatás teljes struktúrákat ír naplófájlba, és `memcmp` segítségével dönti el, változott-e egy rekord. A paddingproblémát packed attribútummal javítanák. Mely hibák maradnak meg, és milyen új kockázat jelenik meg?

9.2.9 Megoldások

9.2.9.1 Tömbméret

A teljes méret:

$$5 \cdot 12 = 60 \text{ C_byte}.$$

A tagok méretének összege $5 \cdot 7 = 35$, a paddingé $5 \cdot 5 = 25$ C bájt. A záró padding már minden elem 12 bájtos méretében szerepel; ezen felül nincs külön tömbelemközi rés.

Ellenőrzési feltétel: a helyi profilban `sizeof records == 60`.

9.2.9.2 Átrendezés ellenőrzése

A várt eltolások: `count`: 0, `code`: 4, `kind`: 6. Az igazítás 4, a méret 8.

Az egyenlőségvizsgálat ugyanazt a három tagértéket hasonlítsa össze, mint a `record_equal`, csak `struct CompactRecord` paraméterekkel.

Ellenőrzési feltétel: mindkét fordító minden layout-ellenőrzése egyezést jelezzen. Az egyenlőségteszt azonos tagértékekre legyen igaz, bármely egyetlen tag megváltoztatására hamis. Teljes struktúrára végzett `memcmp` ne szerepeljen benne.

9.2.9.3 A packed „javítás” értékelése

A padding eltüntetése nem definiál tartós, verziózható külső formátumot, és nem kezeli a bájtsorrendet. Más tagtípusoknál az eltérő értékreprezentációk miatt a bájtszintű összehasonlítás padding nélkül sem általános értékegyenlőség.

Új kockázat az igazítatlan tagelérés és különösen a tagcím közönséges típusos pointerként való továbbadása. A javítás a tagonkénti összehasonlítás és a külön meghatározott külső kódolás.

Ellenőrzési feltétel: a naplóformátum ne függjön `sizeof(struct ...)` értékétől vagy tageltolásoktól; az összehasonlítás kizárólag logikai tagértékeket vizsgáljon.

9.2.10 Összefoglaló

A struktúra méretét a tagméretek mellett az igazítás és a padding határozza meg. A záró padding biztosítja, hogy tömbben a következő elem is megfelelően igazított legyen. Az `offsetof`, `_Alignof` és `sizeof` a helyi elrendezés ellenőrzésének eszközei.

A tagátrendezés csökkentheti a memóriaigényt, de ABI-változás. A logikai érték nem azonos a teljes objektumreprezentációval; a padding nem megbízható összehasonlítási vagy adatátviteli alap, a packed elrendezés pedig nem általános megoldás.

9.3 Union, tagged union, enum és bit-field

Egy mérési esemény tartalmazhat számlálót, terhelési arányt vagy hibainformációt. Ha egyszerre pontosan egy változat értelmes, a három külön mező helyett közös tárolót használhatunk. A megtakarítás azonban csak akkor biztonságos, ha azt is egyértelműen nyilvántartjuk, melyik változatot tároltuk.

A C-ben ezt rendszerint egy `union` és egy külön változatazonosító együttese, a **címkézett unió** (*tagged union*, más néven *discriminated union*) valósítja meg. A kapcsolatot a nyelv nem ellenőrzi automatikusan: az invariáns a program szerződésének része.

Az alábbiak **ISO C17** szabályokra épülnek. A Padding, alignment és offsetof fejezetből a méret–igazítás kapcsolatát, az Effective type, aliasing és objektumreprezentáció részből az érték és reprezentáció különbségét használjuk.

9.3.1 Union: különböző tagok közös tárolón

A `struct` tagjai külön, deklarációs sorrendjüknek megfelelő tárolási területet kapnak. A `union` tagjainak tárolója ezzel szemben átfed: minden tag ugyanott kezdődik.

C17 deklarációs részlet:

```
union measurement {
    unsigned long count;
    double load;
};
```

A `union measurement` mérete elegendő bármelyik tag tárolásához, igazítása pedig lehetővé teszi bármelyik tag megfelelő elhelyezését. Mérete legalább a legnagyobb tag mérete, de kitöltés miatt annál nagyobb is lehet:

$$\text{sizeof}(U) \geq \max_i \text{sizeof}(M_i).$$

Itt U az uniótípus, M_i annak valamely tagtípusa; a méretek C bájtban értendők. A union mérete tehát nem a tagméretek összege.

A tagválasztást az inicializálás vagy a tagba történő értékadás határozza meg:

```
/* C17, önálló programba illeszthető részlet. */
union measurement m = { .count = 120UL };
m.load = 0.75;
```

A második utasítás után a `load` tag értékét tároljuk. Nem két független változónk van: a korábbi `count` nem marad meg külön.

Az **aktív tag** kifejezést itt a legutóbb eltárolt változat rövid neveként használjuk. Ez nem jelenti a C++ objektumélettartam-szabályainak átvételét. C17-ben másik tag olvasásának saját reprezentációs szabályai vannak, amelyeket később külön tárgyalunk.

Fontos következmény: a union önmagában nem tartalmaz futásidejű információt arról, melyik tagba írtunk. A debugger sem feltétlenül tudja ezt kikövetkeztetni.

9.3.2 Enum mint változatazonosító

A felsorolási típus, az `enum`, nevesített egész konstansokat kapcsol össze:

```
enum sample_kind {
    SAMPLE_COUNT,
    SAMPLE_LOAD,
    SAMPLE_FAILURE
};
```

ISO C17: explicit értékadás nélkül az első felsorolási konstans értéke nulla, a következőké eggyel nagyobb. A felsorolási konstansok típusa `int`, értéküknek pedig `int`-tel ábrázolhatónak kell lenniük.

Ettől külön kérdés magának az `enum sample_kind` típusnak a reprezentációja. Egy `char`, előjeles vagy előjel nélküli egész típussal kompatibilis; a megfelelő típus kiválasztása implementáció által meghatározott. Nem következik tehát, hogy minden enum mérete négy C bájt.

Az enum ráadásul nem zárt, automatikusan ellenőrzött értékhalmaz. Egy enumobjektum tárolhat a nevesített konstansoktól eltérő, a kompatibilis típussal ábrázolható értéket is. Emiatt a `switch` `default` ága hasznos védelmi határ.

Ez azonban nem teszi biztonságossá inicializálatlan vagy érvénytelen reprezentációjú objektum olvasását. A `default` csak olyan értéket kezelhet, amelynek kiértékelése már önmagában is érvényes.

A C23 rögzített alaptípusú enumlehetőségeit itt nem használjuk; azok nem részei a C17 szerződésnek.

9.3.3 Címkézett értéktípus és invariánsai

A címke és a tároló közötti kapcsolatot minden változatra külön rögzítsük:

Címke	Tárolt uniontag	Invariáns
`SAMPLE_COUNT`	`count`	Bármely `unsigned long` érték elfogadható
`SAMPLE_LOAD`	`load`	$0 \le load \le 1$; NaN és végtelen nem megengedett
`SAMPLE_FAILURE`	`failure`	`code > 0`; `retryable` logikai érték

Az invariáns része az is, hogy a címke valóban az eltárolt uniontagot azonosítja. A címke átírása önmagában nem konvertálja az adatot.

9.3.3.1 Teljes C17 példa

A típus csak beágyazott értékeket tartalmaz: nincs dinamikus memória, erőforrásátadás vagy kölcsönzött sztring. Ezért a struktúra értékmásolása teljes, önálló másolatot eredményez.

A következő kód `sample.c` néven fordítható. Minimumkövetelménye hosted ISO C17. Javasolt, de itt nem fordítással ellenőrzött laborprofil: a `lab-next` profil (profiltábla), Linux x86-64 System V ABI, LP64, a `lab-next` profil (profiltábla), alapértelmezett adattípus-elrendezéssel.

```
#include <stdbool.h>
#include <stdio.h>

enum sample_kind {
    SAMPLE_COUNT,
    SAMPLE_LOAD,
    SAMPLE_FAILURE
};

struct failure_info {
    int code;
    bool retryable;
};

union sample_payload {
    unsigned long count;
    double load;
    struct failure_info failure;
};

struct sample {
    enum sample_kind kind;
    union sample_payload data;
};

static struct sample sample_count(unsigned long count)
{
    return (struct sample) {
        .kind = SAMPLE_COUNT,
        .data = { .count = count }
    };
}

static bool sample_load(double load, struct sample *out)
{
    if (out == NULL || !(load >= 0.0 && load <= 1.0)) {
        return false;
    }

    *out = (struct sample) {
        .kind = SAMPLE_LOAD,
        .data = { .load = load }
    };
    return true;
```

```
}

static bool sample_failure(int code, bool retryable,
                           struct sample *out)
{
    if (out == NULL || code <= 0) {
        return false;
    }

    *out = (struct sample) {
        .kind = SAMPLE_FAILURE,
        .data = {
            .failure = {
                .code = code,
                .retryable = retryable
            }
        }
    };
    return true;
}

static bool sample_print(struct sample s)
{
    switch (s.kind) {
    case SAMPLE_COUNT:
        return printf("count=%lu\n", s.data.count) >= 0;

    case SAMPLE_LOAD:
        return printf("load=%.2f\n", s.data.load) >= 0;

    case SAMPLE_FAILURE:
        return printf("failure=%d retryable=%d\n",
                      s.data.failure.code,
                      s.data.failure.retryable) >= 0;

    default:
        return false;
    }
}

int main(void)
{
    struct sample a = sample_count(42UL);
    struct sample b = sample_count(0UL);
    struct sample c = sample_count(0UL);

    if (!sample_load(0.75, &b) ||
        !sample_failure(5, true, &c)) {
        return 1;
    }

    if (!sample_print(a) ||
```

```
        !sample_print(b) ||
        !sample_print(c)) {
        return 1;
    }

    return 0;
}
```

Fordítási parancsok a megfelelő kiadáshoz rendelt fordítóparancsokkal:

```
gcc-14 -std=c17 -Wall -Wextra -Wpedantic -O2 -fno-fast-math sample.c -o
 ↪  sample-gcc
clang-19 -std=c17 -Wall -Wextra -Wpedantic -O2 -fno-fast-math sample.c -o
 ↪  sample-clang
```

A parancsnevek önmagukban nem garantálják a pontos javítóverziót; azt a laborban külön rögzíteni kell. A példához nem tartozik mért futási eredmény.

9.3.3.2 Élettartam és hibaszerződés

Szereplő	Tulajdonlás és élettartam	Módosítás vagy hibakimenet
`a`, `b`, `c`	A `main` helyi objektumai; saját beágyazott tárolóval	Élettartamuk a blokk elhagyásáig tart
`out`	Nem tulajdonló hozzáférés a hívó írható `struct sample` objektumához	Nem marad meg a hívás után
Sikeres konstrukció	A célobjektum teljes értékét lecseréli	A korábbi változat szerinti értelmezés többé nem használható
Sikertelen konstrukció	Nem ír a célobjektumba	`false`; a cél korábbi értéke változatlan
`sample_print`	Értékmásolatot kap	Ismeretlen címkénél vagy azonnal jelzett kimeneti hibánál `false`

Nem null `out` esetén előfeltétel, hogy ténylegesen élő, megfelelően igazított, írható `struct sample` objektumra mutasson. A nullvizsgálat dangling pointert nem ismer fel.

Invariánsmegőrzési bizonyításvázlat. A konstruktor először ellenőrzi a bemenetet. Hibánál nem történik értékadás. Sikernél az összetett literál egyszerre írja le a helyes címkét és az ahhoz tartozó tagot, majd a struktúraértékadás ezt az értéket másolja a célba. A `load` feltétel a NaN-t is elutasítja, mert az összehasonlítások nem lesznek igazak. Ez a bizonyítás szabványos lebegőpontos szemantikát feltételez; ezért nem használunk `fast-math` módot.

Az „egyszerre” itt forráskódbeli értékkonstrukciót jelent, nem szálak közötti atomikusságot.

9.3.3.3 A közös tároló számszerű következménye

A fenti x86-64 System V LP64 laborprofilhoz, rövidített enum és packing nélkül, a következő elrendezéssel számolunk. Feltételezett méret/igazítás párok: enum és `int` 4/4, `unsigned long` és `double` 8/8, `_Bool` 1/1.

Elem	Offset a közvetlen befoglalóban	Méret	Igazítás
`failure_info.code`	0	4	4
`failure_info.retryable`	4	1	1
`struct failure_info`	—	8	4
`sample_payload` bármely tagja	0	8	tagfüggő
`union sample_payload`	—	8	8
`sample.kind`	0	4	4
`sample.data`	8	8	8
`struct sample`	—	16	8

Minden adat C bájtban értendő. A táblázat profilhoz kötött levezetés, nem mérési jegyzőkönyv és nem általános C-garancia. A helyi ellenőrzés eszköze a `sizeof`, `_Alignof` és `offsetof`, az előző részben tanult módon.

9.3.4 Union type punning: reprezentáció, nem konverzió

ISO C17: ha nem a legutóbb eltárolt tagot olvassuk, a megfelelő objektumreprezentációt az olvasott tag típusának reprezentációjaként értelmezzük. Ezt nevezik unionon keresztüli **type punningnak**.

Szándékosan nem hordozható, elemzésre szolgáló C17 részlet:

```c
union bits {
    unsigned int integer;
    float real;
};

union bits b = { .integer = 1u };
float x = b.real;  /* Nem azonos ezzel: (float)b.integer */
```

Ez nem numerikus egész–lebegőpontos konverzió. Az eredmény az ábrázolásoktól, méretektől és bájtsorrendtől függ. Ha az új tag nagyobb, a korábbi tárolás által nem meghatározott bájtok is érintettek lehetnek. Az előálló reprezentáció **csapdareprezentáció** (*trap representation*) is lehet; ilyen érték olvasása nem definiált viselkedést okozhat.

Két gyakori szélsőséges állítás ezért egyaránt hibás:

- „C-ben minden inaktív uniontag olvasása UB.”
- „C-ben a union bármilyen típuskonverziót biztonságosan megold.”

Az azonos `sizeof` sem bizonyítja a reprezentációk kompatibilitását. A `memcpy` segíthet elkerülni bizonyos aliasing- és igazítási hibákat, de önmagában nem garantál érvényes célreprezentációt. Ezeket a C-szabályokat C++-ra sem szabad változtatás nélkül átvinni.

Külön C17-garancia vonatkozik a **közös kezdősorozatra**: ha egy union struktúratagjai megfelelő kezdeti tagjaikban kompatibilis típusúak — bitmezőknél azonos szélességgel —, a közös kezdet

vizsgálható a másik struktúratagon keresztül is, ahol a teljes uniontípus deklarációja látható. Ez szűk kivétel, nem általános engedély a struktúrák tetszőleges részének átértelmezésére.

Címkézett értéktípusnál egyszerűbb szabályt alkalmazunk: mindig a címkéhez tartozó tagot olvassuk.

9.3.5 Bitmezők: logikai szélesség, nem wire layout

A **bitmező** (*bit-field*) egész típusú struktúra- vagy uniontag, amelyhez bitszélességet adunk meg. Folyamaton belüli jelzőkhöz kényelmes lehet:

```c
/* C17 deklarációs részlet; a fizikai elrendezés ABI-függő. */
struct local_flags {
    unsigned int ready : 1;
    unsigned int mode : 3;
    unsigned int : 2;
    unsigned int : 0;
    unsigned int retries : 4;
    signed int delta : 4;
    _Bool enabled : 1;
};
```

ISO C17: a szélesség nemnegatív egész konstanskifejezés; nem haladhatja meg az alapul szolgáló típusban rendelkezésre álló bitszámot. A biztosan használható típusok `_Bool`, `signed int` és `unsigned int`; más típusok megengedése implementáció által meghatározott. A `_Bool` bitmező legfeljebb egy bit széles lehet.

Az egyszerű `int` bitmező előjelessége implementáció által meghatározott, ezért azt érdemes explicit megadni. Egy w bites `unsigned int` bitmező értéktartománya:

$$0 \le x \le 2^w - 1.$$

Így a `mode` tartománya 0–7. A `signed int delta : 4` pontos negatív tartományánál C17-ben a signed reprezentációt is figyelembe kell venni; a −8–7 tartomány kettes komplemens profilt feltételez.

A névtelen, nem nulla szélességű mező elérhetetlen kitöltési helyet foglal. A **nulla szélességű mező** csak névtelen lehet: megtiltja, hogy további bitmező kerüljön az előző bitmezőt tartalmazó allokációs egységbe. Nem jelent hordozható „következő négybájtos címre igazítást”.

A fizikai elrendezés több külön döntést tartalmaz:

Tulajdonság	C17 szerinti korlát
Magas vagy alacsony helyiérték felől történő kiosztás	Implementáció által meghatározott
Nem elegendő hely esetén egységhatár átlépése	Implementáció által meghatározott
Az allokációs egység igazítása	Nem specifikált
Konkrét elrendezés fordítók közötti egyezése	Az ABI és fordítási opciók szerződésétől függ

A bitmezőnek nem képezhető címe, és közvetlenül `sizeof` sem alkalmazható rá. Bitmezőre hordozható `offsetof`-alapú offsetet sem kaphatunk.

Ezért a bitmezős struktúra nem hordozható hálózati formátum. A bitkiosztás nem vezethető le pusztán a processzor bájtsorrendjéből. Wire protokollhoz explicit oktettsorrend, maszkok és eltolások szükségesek; ezt a Byte order és biztonságos szerializálás rész folytatja.

9.3.6 Gyakori hibák

A címke átírása uniontag-értékadás nélkül csak inkonzisztens objektumot hoz létre. Javítása a teljes új változat konstrukciója.

A `default` ág nem általános unionvalidátor. Felismerhet ismeretlen címkét, de nem bizonyítja, hogy egy ismert címke mellett valóban a megfelelő tagot tárolták.

A „nálam két bájt” megfigyelés nem bitmező-protokollszerződés. Más ABI, packing vagy fordítóopció megváltoztathatja az elrendezést.

Végül az értékmásolás csak a jelenlegi, beágyazott adatokat tartalmazó típusnál jelent teljesen független értéket. Pointertag hozzáadása után külön tulajdonlási szerződésre lenne szükség.

9.3.7 Feladatok

9.3.7.1 Magyarázó feladat

Egy `SAMPLE_COUNT` érték címkéjét `SAMPLE_LOAD`-ra írjuk, a unionhoz nem nyúlunk. Megőrzi-e ez az invariánst? Megjavítaná-e egy `default` ág?

9.3.7.2 Kódolási feladat

Írj `sample_is_retryable(struct sample s)` függvényt. Csak újrapróbálható hibaváltozat esetén adjon igaz értéket, és más változatnál ne olvassa a `failure` tagot.

9.3.7.3 Diagnosztikai feladat

Egy protokoll két oktettjéből az első felső három bitje típusazonosító, a fennmaradó tizenhárom bit pedig hossz, magas helyiértékű oktettel előre. A bemenet `A0 01`.

Miért nem megfelelő dekóder egy három- és tizenhárom bites `unsigned int` bitmezőt tartalmazó struktúra? Add meg a helyes mezőértékeket explicit bitműveletekkel.

9.3.8 Megoldások

9.3.8.1 Magyarázó feladat megoldása

Az invariáns sérül: a címke és a legutóbb eltárolt tag eltér. A `default` nem segít, mert a címke ismert. A `sample_load` konstruktor használata helyreállítja a kapcsolatot, és az értéktartományt is ellenőrzi.

Ellenőrzési feltétel: minden változatváltás után együtt legyen helyes a címke, a választott tag és annak tartalmi invariánsa.

9.3.8.2 Kódolási feladat megoldása

C17 részlet, a teljes példa típusdeklarációi után:

```
static bool sample_is_retryable(struct sample s)
{
    return s.kind == SAMPLE_FAILURE && s.data.failure.retryable;
}
```

A `&&` rövidzáras: a jobb oldalt csak igaz bal oldal esetén értékeli ki. Előfeltétel a típus invariánsát teljesítő bemenet.

Ellenőrzési feltétel: számlálóra és terhelésre hamis; hibára a `retryable` értékével egyező eredmény.

9.3.8.3 Diagnosztikai feladat megoldása

A bitmezők sorrendje, allokációs egysége és kitöltése nem a protokoll szabályait követi. Még a struktúra kétoktettes mérete sem garantált.

Nyolcbites oktettprofilban, helyi tömbbel:

```
/* C17, CHAR_BIT == 8 előfeltétellel; elemzési részlet. */
unsigned char bytes[2] = { 0xA0u, 0x01u };

unsigned int type = (bytes[0] >> 5) & 0x07u;
unsigned int length =
    ((unsigned int)(bytes[0] & 0x1Fu) << 8) |
    (unsigned int)bytes[1];
```

A tároló a helyi `bytes` tömb; nincs megtartott kölcsönzött hozzáférés. A felső három bit `101`, ezért `type == 5`. Az alsó öt bit nulla, így `length == 1`. Ezek levezetett ellenőrzési értékek, nem futtatási eredmények.

9.3.9 Összefoglaló

A union közös tárolót ad, nem automatikus változatkezelést. Biztonságos értéktípushoz explicit címke, változatonkénti invariáns és címkevezérelt hozzáférés szükséges. A C17 union type punning reprezentációt értelmez át, ezért trap- és hordozhatósági korlátai vannak. A bitmezők logikai szélességet fejeznek ki, de fizikai elrendezésükből nem következik hordozható wire formátum.

9.4 Byte order és biztonságos szerializálás

A hálózaton nem C-objektumok, hanem egy protokoll szerint értelmezhető oktettek haladnak. A szerializálás (*serialization*) egy programbeli érték átalakítása meghatározott külső reprezentációvá; a deszerializálás ennek ellenőrzött visszaalakítása. Egy hordozható formátum az értékeket, a mezők szélességét, sorrendjét és érvényességi feltételeit rögzíti, nem pedig egy fordító pillanatnyi memóriaelrendezését.

Az előfeltételek közül a memóriamásolás és byte view adja a reprezentáció elérésének eszközeit, a promóciók és túlcsordulás az aritmetikai biztonságot. Az union, enum és bit-field tárgyalásából

pedig azt visszük tovább, hogy egy natív típus kényelmes belső modell lehet anélkül, hogy külső adatformátumnak alkalmas volna.

9.4.1 Host order és wire order

A **host byte order** a több C bájtot elfoglaló skaláris értékek adott implementációban használt bájtsorrendje. A **wire order** a protokoll által előírt sorrend. A kettő egyezhet, de a protokoll helyessége nem függhet ettől.

ISO C17: a C bájt, azaz a `sizeof` egysége `CHAR_BIT` bitet tartalmaz; nem általánosan nyolcat. A hálózati **oktett** viszont definíció szerint nyolc bit. E fejezet codecje ezért explicit feltételként követeli meg a `CHAR_BIT == 8` tulajdonságot és a használt exact-width egész típusokat.

A `0x12345678` érték négyoktettes reprezentációja:

Sorrend	Oktettek növekvő offset szerint
Big-endian	`12 34 56 78`
Little-endian	`78 56 34 12`

Big-endian esetén a legnagyobb helyi értékű oktett kerül előre. Ha egy k oktettes, előjel nélküli mező oktettjei $b_0, \dots, b_{k-1}$, akkor:

$$x = \sum_{i=0}^{k-1} b_i \, 256^{k-1-i}, \qquad 0 \le b_i < 256.$$

Az explicit codec ezt az összefüggést valósítja meg. Nem szükséges előbb felismernie a host bájtsorrendjét.

9.4.1.1 Miért nem protokoll a natív struktúra?

Egy struktúra `sizeof` hosszúságú objektumreprezentációjának közvetlen kiírása több különböző szerződést mos össze:

Natív tulajdonság	Külső formátumbeli probléma
Tagok közötti és végi padding	Nem jelent adatmezőt; tartalma nem megbízható, és információt szivárogtathat
Több-bájtos egészek reprezentációja	A fogadó más bájtsorrendet használhat
`long`, `size_t`, enum szélessége	Eltérhet a platformok vagy fordítási profilok között
Pointertag	Címet tartalmaz, nem a hivatkozott adatot; más folyamatban nem használható
Bit-field elrendezése	Nem helyettesít explicit bitpozíciókat meghatározó protokollt

A padding kikapcsolása fordítóbővítéssel legfeljebb az egyik problémát módosítja. Nem definiál bájtsorrendet, pointerjelentést vagy verziózást. A helyes megoldás a szemantikai mezők egyenkénti kódolása.

9.4.2 Egy kis telemetriaüzenet szerződése

A példában egy verziót, egy sorszámot, egy hőmérsékletet és egy átlátszatlan payloadot továbbítunk. A teljes wire formátum:

Offset oktettben	Hossz oktettben	Mező	Szabály
0	1	Verzió	Pontosan `1`
1	2	Payloadhossz	Előjel nélküli, big-endian
3	8	Sorszám	Előjel nélküli, big-endian
11	2	Hőmérséklet	16 bites kettes komplemens kód, big-endian
13	L	Payload	Pontosan L oktett

Ez **protokollelrendezés**, nem ABI-layout; a táblázat nem függ a host struktúraoffsetjeitől.

A hőmérséklet skálázott egész szám:

$$T = \frac{q}{100}\ ^\circ\mathrm{C}, \qquad -32768 \le q \le 32767.$$

Így például $q = -125$ jelentése $-1{,}25\,^\circ$C. A protokollnak a mértékegységet és a skálát is rögzítenie kell: a „16 bites signed érték” önmagában kevés. Ha az alkalmazás lebegőpontos mérésből állítja elő q-t, a kerekítési és tartományhibapolitikát még a codec meghívása előtt kell alkalmaznia.

Az előjeles érték wire kódja:

$$u = \begin{cases} q, & q \ge 0, \\ q + 65536, & q < 0. \end{cases}$$

Visszafelé:

$$q = \begin{cases} u, & u < 32768, \\ u - 65536, & u \ge 32768. \end{cases}$$

Ez matematikai átalakítás, nem a host előjeles reprezentációjának másolása. A belső típus ezért `int32_t`: a dekódolás közbülső értékei biztonságosan elférnek benne. Nem végzünk tartományon kívüli unsigned–signed konverziót.

9.4.3 Igazítatlan buffer és POSIX konverziók

Egy fogadott puffer adott offsetje nem feltétlenül megfelelően igazított egy egész típushoz. A `uint64_t *`-ra alakított cím dereferálása nem biztonságos dekódolási módszer: sérülhet az igazítási követelmény, és külön vizsgálatot igényelnének az objektum- és aliasing-szabályok is. Az, hogy a hardver bizonyos igazítatlan betöltéseket támogat, nem ad nyelvi felmentést.

Két bevált módszer van:

- karaktertípusú hozzáférés és explicit értékösszeállítás;
- `memcpy` egy megfelelő típusú helyi objektumba, majd bájtsorrend-konverzió.

POSIX API: a `<arpa/inet.h>` fejléc `htons` és `htonl` függvénye a 16, illetve 32 bites hostértéket alakítja hálózati sorrendű reprezentációhoz. Az ellenkező irány a `ntohs`, illetve `ntohl`. Ezek nem ISO C függvények. A névben szereplő `long` nem a host `unsigned long` tetszőleges szélességét jelenti.

Az alábbi **C17 + POSIX kódrészlet** négy olvasható oktettet igényel. A tároló tulajdonosa a hívó; a kölcsönzött `src` hozzáférésnek a hívás végéig érvényesnek kell maradnia. A függvény nem ellenőrzi a hosszt, és nem ad hibakódot: az előfeltételt a hívónak kell biztosítania.

```
#include <arpa/inet.h>
#include <stdint.h>
#include <string.h>

uint32_t read_u32_be_posix(const unsigned char *src)
{
    uint32_t network_value;
    memcpy(&network_value, src, sizeof network_value);
    return ntohl(network_value);
}
```

A helyi `network_value` igazítása megfelelő. A `memcpy` viszont önmagában nem cserél bájtsorrendet; ezért szükséges utána a konverzió.

A POSIX `htonl` nem nyolcoktettes codec. A következő példában a 64 bites mezőt is explicit műveletekkel kódoljuk, így nincs szükség platformspecifikus `htonll` változatra.

9.4.4 Bounds-ellenőrzött encoder és decoder

Az encoder a hívó bufferébe ír. A decoder nem másolja a payloadot, hanem **kölcsönzött nézetet** ad rá.

Objektum vagy hozzáférés	Tulajdonos és élettartam	Érvénytelenítés
Encoder bemeneti üzenete és payloadja	A hívó; a hívás végéig érvényes	Megszüntetésük a hívás alatt szerződéssértés
Encoder célbuffere	A hívó; legalább `cap` írható C bájt	Felszabadítás, sikeres `realloc`
Decoder bemeneti buffere	A hívó; legalább `n` olvasható C bájt	Felszabadítás, sikeres `realloc`
Dekódolt payloadnézet	Nem tulajdonol tárolót	A bemenet élettartamának vége; újrafelhasználása megszünteti a nézet üzenetjelentését
Dekódolt metaadatobjektum	A hívó	Megszűnése nem szabadítja fel a payloadot

A bemenet nem módosulhat konkurensen az olvasással. Az encoder célbuffere nem fedheti át a forrásadatokat vagy a metaadatobjektumokat; a decoder kimeneti objektuma nem fedheti át a bemenetet. A pointerek által jelzett tényleges tárolóméret helyessége hívói előfeltétel: ezt egy C-függvény a számszerű hosszparaméterből nem tudja ellenőrizni.

A visszatérési hibák esetén az encoder célbuffere és `written` kimenete, illetve a decoder kimeneti objektuma változatlan marad.

9.4.4.1 Teljes C17 példa

A `codec.c` ISO C17-et, nyolcbites C bájtot, valamint `uint16_t`, `uint32_t`, `uint64_t` és `int32_t` támogatást igényel. Nem használ POSIX függvényt.

```
#include <assert.h>
#include <limits.h>
#include <stddef.h>
#include <stdint.h>
#include <string.h>

#if CHAR_BIT != 8
#error "This codec requires 8-bit C bytes"
#endif

#if !defined(UINT16_MAX) || !defined(UINT32_MAX) || \
    !defined(UINT64_MAX) || !defined(INT32_MAX)
#error "Required exact-width integer types are unavailable"
#endif

enum { HEADER_SIZE = 13 };

enum codec_status {
    CODEC_OK,
    CODEC_INVALID,
    CODEC_SHORT,
    CODEC_FORMAT
};

struct message {
    uint64_t sequence;
    int32_t temperature_centi;
    const unsigned char *payload;
    size_t payload_size;
};

/* Belső előfeltétel: 1 <= n <= 8, p legalább n bájtot jelöl. */
static void put_be(unsigned char *p, size_t n, uint64_t x)
{
    while (n != 0) {
        --n;
        p[n] = (unsigned char)(x & UINT64_C(255));
        x >>= 8;
    }
}

static uint64_t get_be(const unsigned char *p, size_t n)
{
    uint64_t x = 0;

    for (size_t i = 0; i < n; ++i) {
        x = (x << 8) | (uint64_t)p[i];
```

```
    }
    return x;
}

static enum codec_status encode_message(
    unsigned char *dst, size_t cap,
    const struct message *m, size_t *written)
{
    if (dst == NULL || m == NULL || written == NULL) {
        return CODEC_INVALID;
    }

    if ((m->payload_size != 0 && m->payload == NULL) ||
        m->payload_size > UINT16_MAX ||
        m->temperature_centi < -INT32_C(32768) ||
        m->temperature_centi > INT32_C(32767)) {
        return CODEC_INVALID;
    }

    if (m->payload_size > SIZE_MAX - HEADER_SIZE) {
        return CODEC_INVALID;
    }

    const size_t total = HEADER_SIZE + m->payload_size;
    if (cap < total) {
        return CODEC_SHORT;
    }

    const int32_t q = m->temperature_centi;
    const uint64_t raw = q < 0
        ? (uint64_t)(q + INT32_C(65536))
        : (uint64_t)q;

    dst[0] = 1;
    put_be(dst + 1, 2, (uint64_t)m->payload_size);
    put_be(dst + 3, 8, m->sequence);
    put_be(dst + 11, 2, raw);

    if (m->payload_size != 0) {
        memcpy(dst + HEADER_SIZE, m->payload, m->payload_size);
    }

    *written = total;
    return CODEC_OK;
}

static enum codec_status decode_message(
    const unsigned char *src, size_t n, struct message *out)
{
    if (src == NULL || out == NULL) {
        return CODEC_INVALID;
    }
```

```
    if (n < HEADER_SIZE) {
        return CODEC_SHORT;
    }
    if (src[0] != 1) {
        return CODEC_FORMAT;
    }

    const size_t length = (size_t)get_be(src + 1, 2);
    const size_t available = n - HEADER_SIZE;

    if (length > available) {
        return CODEC_SHORT;
    }
    if (length < available) {
        return CODEC_FORMAT;
    }

    const uint32_t raw = (uint32_t)get_be(src + 11, 2);
    const int32_t q = raw < UINT32_C(32768)
        ? (int32_t)raw
        : (int32_t)raw - INT32_C(65536);

    const struct message result = {
        .sequence = get_be(src + 3, 8),
        .temperature_centi = q,
        .payload = src + HEADER_SIZE,
        .payload_size = length
    };

    *out = result;
    return CODEC_OK;
}

int main(void)
{
    const unsigned char payload[] = { 0x41, 0x00, 0xff };
    const struct message input = {
        .sequence = UINT64_C(0x0102030405060708),
        .temperature_centi = -125,
        .payload = payload,
        .payload_size = sizeof payload
    };

    const unsigned char expected[] = {
        0x01, 0x00, 0x03,
        0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08,
        0xff, 0x83,
        0x41, 0x00, 0xff
    };

    unsigned char storage[1 + sizeof expected];
    unsigned char *wire = storage + 1;
```

```
    size_t written = 0;
    struct message output = {0};

    assert(encode_message(wire, sizeof expected,
                          &input, &written) == CODEC_OK);
    assert(written == sizeof expected);
    assert(memcmp(wire, expected, written) == 0);

    assert(decode_message(wire, written, &output) == CODEC_OK);
    assert(output.sequence == input.sequence);
    assert(output.temperature_centi == input.temperature_centi);
    assert(output.payload_size == sizeof payload);
    assert(memcmp(output.payload, payload, sizeof payload) == 0);

    assert(decode_message(wire, written - 1, &output) == CODEC_SHORT);
    return 0;
}
```

Az eltolt `wire` cím demonstrálja, hogy a codec nem igényel egész típushoz megfelelően igazított bemenetet. Az eltolás önmagában nem bizonyítja, hogy az adott futásban a cím ténylegesen igazítatlan.

A külön elvégzendő labor választott profilja: Linux x86-64 System V ABI, LP64, a `lab-next` profil (profiltábla), alapértelmezett ABI-opciókkal. A parancsok e konkrét fordítókiadásokra mutató `gcc`, illetve `clang` programot feltételeznek. A példához itt nem tartozik elvégzett fordítás vagy mérés.

```
gcc -std=c17 -O2 -Wall -Wextra -Wconversion -Wpedantic \
    -pedantic-errors codec.c -o codec-gcc

clang -std=c17 -O2 -Wall -Wextra -Wconversion -Wpedantic \
    -pedantic-errors codec.c -o codec-clang
```

9.4.5 A méretellenőrzés és az értékmegőrzés indoklása

Az encoderben nem elegendő előbb kiszámítani a teljes méretet, és utána összevetni a kapacitással. Ha H a fejlécméret és L a payloadhossz, először ezt ellenőrizzük:

$$L \le \mathrm{SIZE_MAX} - H.$$

Csak ezután képezzük $N = H + L$ értékét. Így az összeadás nem fordul körbe. A 16 bites hosszmező külön korlát: az aritmetikai reprezentálhatóság és a protokoll érvényessége két eltérő feltétel.

A decoder előbb megköveteli az $n \ge H$ feltételt. Emiatt az $n - H$ kivonás biztonságos, és a payloadhossz közvetlenül összevethető a rendelkezésre álló területtel. Ebben az API-ban egy buffer **pontosan egy teljes üzenet**: hiányzó payload esetén `CODEC_SHORT`, többletbájt esetén `CODEC_FORMAT` az eredmény. TCP-streamhez külön framingréteg szükséges; azt a socket API és framing tárgyalja.

9.4.5.1 Helyességi bizonyításvázlat

A `get_be` első j iterációja után az akkumulátor az első j oktett big-endian értékét tartalmazza. A következő lépés $x' = 256x + b_j$, pontosan a következő oktett hozzáfűzése. Legfeljebb nyolc oktett esetén az eredmény elfér `uint64_t`-ban. A `put_be` fordítva, az alsó nyolc bit leválasztásával és jobbra léptetéssel állítja elő ugyanezt a sorozatot.

Minden fejlécolvasást megelőz a teljes fejléc méretellenőrzése. Az encoder minden lehetséges hibavisszatérése az első írás előtt található; a decoder csak a teljes ellenőrzés után rendeli hozzá a helyi eredményt a kimenethez.

A mintában:

$$65536 - 125 = 65411 = \texttt{0xFF83}.$$

A payloadhossz `00 03`, a hőmérséklet `FF 83`, a teljes üzenet 16 oktett. Ezek levezetett elvárt értékek, nem mérési eredmények. A round-trip megőrzi a mezőértékeket és a payload tartalmát, de nem a payload pointerének címét.

Az encoder költsége $O(H + L)$, allokáció nélkül, folytonos írással és payloadmásolással. A decoder $O(H)$ munkát végez, nem allokál, és nem járja be a payloadot; ennek ára a kölcsönzött nézet élettartam-szerződése.

9.4.6 Gyakori hibák és javításuk

Csak `memcpy` használata. Az igazítás problémáját kezeli, a bájtsorrendet nem. Utána konverzió kell, vagy eleve explicit oktettkiolvasás.

Előjeles nyers mező közvetlen castolása. C17-ben egy nem reprezentálható unsigned érték signed típussá alakítása nem általános dekódolási recept. A példában a kivonás még a tágabb, biztonságos tartományban történik.

A payloadhossz ellenőrzése a másolás után. Ekkor a bounds-sértés már megtörtént. Minden ellenőrzést az első érintett hozzáférés elé kell helyezni.

A dekódolt pointer megtartása a fogadópuffer újrafelhasználása után. A nézet nem saját másolat. Hosszabb megőrzéshez külön tulajdonolt tároló szükséges. Nulla hosszúságnál a visszaadott pointer lehet one-past; nem dereferálható.

Csak encoder–decoder round-trip tesztelése. Két azonosan hibás függvény egymást elfedheti. Ezért kell a protokollból kézzel levezetett oktettsorozat is.

9.4.7 Feladatok

1. **Magyarázó feladat.** Mi a sorszám wire reprezentációja, ha értéke `0x0102030405060708`? Miért nem változhat ez a host architektúrájával?
2. **Kódolási feladat.** Bővítsd a tesztet a `-32768`, `32767` és `32768` hőmérsékletértékekkel. Ellenőrizd a wire oktetteket és a hibakimenet változatlanságát.
3. **Diagnosztikai feladat.** Egy fejlesztő így ellenőrzi a méretet: `total = HEADER_SIZE + length; if (total > cap) ...`. A decoderből pedig visszaadott payloadpointert eltárolja, majd új fogadáshoz felhasználja ugyanazt a buffert. Mi a két egymástól független hiba?

9.4.8 Megoldások

9.4.8.1 A bájtsorrend értelmezése

A nyolc oktett `01 02 03 04 05 06 07 08`. A wire order a protokoll tulajdonsága; a host order csak a programbeli egész objektumreprezentációját befolyásolja. Ellenőrzési feltételként a kódolt buffer megfelelő nyolc oktettjét hasonlítsd ehhez a rögzített sorozathoz.

9.4.8.2 Határértékteszt

Az alábbi C17 részlet a teljes példa `main` függvényében, a meglévő változókkal használható:

```
struct message edge = input;

edge.temperature_centi = -32768;
assert(encode_message(wire, sizeof expected,
                      &edge, &written) == CODEC_OK);
assert(wire[11] == 0x80 && wire[12] == 0x00);
assert(decode_message(wire, written, &output) == CODEC_OK);
assert(output.temperature_centi == -32768);

edge.temperature_centi = 32767;
assert(encode_message(wire, sizeof expected,
                      &edge, &written) == CODEC_OK);
assert(wire[11] == 0x7f && wire[12] == 0xff);

unsigned char saved[sizeof expected];
memcpy(saved, wire, sizeof saved);
written = 999;

edge.temperature_centi = 32768;
assert(encode_message(wire, sizeof expected,
                      &edge, &written) == CODEC_INVALID);
assert(written == 999);
assert(memcmp(wire, saved, sizeof saved) == 0);
```

A hibás érték nem csonkolható csendben 16 bitre. Az ellenőrzés egyszerre vizsgálja az érvényességi korlátot és a módosítás nélküli hibavisszatérést.

9.4.8.3 Overflow és nézetélettartam

Az összeadás a kapacitásvizsgálat előtt körbefordulhat. A javítás az előzetes `length > SIZE_MAX - HEADER_SIZE` ellenőrzés, majd a kapacitásvizsgálat.

A második hiba ettől független: a pointer címe megmaradhat, miközben már más üzenet adataira mutat. Ez tároló-újrafelhasználáskor logikai érvénytelenítés; felszabadításkor dangling hozzáférés lenne. Ellenőrzési feltétel, hogy minden nézethasználat megelőzze a buffer újrafelhasználását, vagy előtte saját másolat készüljön.

9.4.9 Összefoglaló

A hordozható protokoll értékeket és oktettsorrendet definiál, nem natív struktúrákat másol. A biztonságos codec külön ellenőrzi a formátumot, a méretaritmetikát és a hozzáférési határokat. Az explicit oktettkezelés megszünteti az igazítás- és host-bájtsorrend-függést; a másolásmentes dekódolás viszont explicit kölcsönzési és élettartam-szerződést igényel.

9.5 Layout- és protokolllabor két architektúrán

Két gépen azonos `sizeof` értéket mérni még nem bizonyítja, hogy egy struktúra bájtjai hordozható hálózati üzenetet alkotnak. A natív memóriaelrendezést a nyelvi szabályok, a target ABI-ja és a fordítási beállítások együtt határozzák meg. A hálózati reprezentációt ezzel szemben a protokoll szerződésének kell rögzítenie.

Ebben a laborban ugyanazt a C17 programot vizsgáljuk x86-64 és AArch64 környezetben. Külön dokumentáljuk a layoutot, a generált memória-hozzáféréseket és a protokolltesztek eredményét. A kiindulópont a Padding, alignment és offsetof, a Byte order és biztonságos szerializálás, valamint a Reprodukálható fordítási és diagnosztikai labor.

9.5.1 A labor profilja és bizonyítékai

A példák minimumkövetelménye ISO C17. A két összehasonlítandó referencia-profil:

Tulajdonság	x86-64 profil	AArch64 profil
Operációs környezet	Linux, hosted	Linux, hosted
ABI	x86-64 System V, LP64	AAPCS64, LP64
Bájt	`CHAR_BIT == 8`	`CHAR_BIT == 8`
Byte order	little-endian	little-endian
Fordító	GCC 14.2.0	GCC 14.2.0
C könyvtár	glibc 2.40	glibc 2.40
Nyelvi mód	`-std=c17`	`-std=c17`

Ezek választott laborfeltételek, nem egy már elvégzett mérés környezeti adatai. Más kiadás használható, de külön profilként dokumentálandó. Rögzítsd a tényleges fordítóverziót, target triple-t, libc-verziót, fordítási kapcsolókat, kernelverziót, továbbá azt, hogy valódi hardveren vagy emulátorban futott-e a program.

Háromféle bizonyítékot különíts el:

- **ISO C17 szabály:** például a `sizeof` C bájtban adja meg az objektum méretét.
- **ABI-/fordítófüggő előrejelzés:** például a vizsgált struktúra várható mérete 12 C bájt.
- **Mérési megfigyelés:** a konkrét bináris által kiírt érték, az assembly vagy a sanitizer diagnosztikája.

Az alábbi konkrét layoutértékek előrejelzések; a kódhoz nem tartozik itt közölt futtatási eredmény.

9.5.2 Az üzenet és a wire format szerződése

Az üzenet három mezője: egy műveletazonosító, egy sorszám és egy számláló. Az érvényes azonosítók `1` és `2`; mindkettőhöz a mezőtípusok teljes értéktartománya megengedett.

A **wire format**, vagyis a továbbított bájtsorozat formátuma pontosan hét oktett:

Mező	Natív x86-64: várható offset/méret	Natív AArch64: várható offset/méret	Wire: offset/méret
`tag`	0 / 1	0 / 1	0 / 1
belső padding	1 / 3	1 / 3	nincs
`sequence`	4 / 4	4 / 4	1 / 4
`count`	8 / 2	8 / 2	5 / 2
záró padding	10 / 2	10 / 2	nincs
teljes méret	12	12	7
igazítás	4	4	oktettenként feldolgozható

A natív oszlopok az előbbi profilokra, szokásos ABI-layouttal, packingot módosító kapcsolók nélkül értendők. A wire formátumban a többoktettes mezők **big-endian** sorrendűek.

A dekóder pontosan egy teljes rekordot fogad:

- hét oktettnél rövidebb input: `P_SIZE`;
- hét oktettnél hosszabb input: szintén `P_SIZE`;
- ismeretlen `tag`: `P_TAG`;
- null pointer: `P_ARGUMENT`;
- siker: `P_OK`.

Hiba esetén a kimeneti objektum változatlan marad. A függvény nem stream-framingot végez: több összefűzött rekordból a hívó választja ki a feldolgozandó hétoktettes tartományt.

9.5.3 Teljes mérő- és protokollteszt

A következő `lab.c` önálló C17 program. Az exact-width típusokat és a nyolcbites C bájtot fordításkor ellenőrzi. Ezek a labor követelményei, nem minden C implementációra igaz állítások.

```
#include <assert.h>
#include <limits.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>

#if !defined(UINT16_MAX) || !defined(UINT32_MAX)
#error "This lab requires uint16_t and uint32_t"
#endif

#ifdef NDEBUG
#error "This test program requires enabled assertions"
#endif
```

```
_Static_assert(CHAR_BIT == 8, "This protocol lab requires octets");

enum { WIRE_SIZE = 7 };

enum status {
    P_OK,
    P_ARGUMENT,
    P_SIZE,
    P_TAG
};

struct message {
    unsigned char tag;
    uint32_t sequence;
    uint16_t count;
};

static int valid_tag(unsigned char tag)
{
    return tag == 1 || tag == 2;
}

static int same(struct message a, struct message b)
{
    return a.tag == b.tag &&
           a.sequence == b.sequence &&
           a.count == b.count;
}

static enum status encode(unsigned char *dst, size_t n,
                          const struct message *src)
{
    if (dst == NULL || src == NULL)
        return P_ARGUMENT;
    if (n != WIRE_SIZE)
        return P_SIZE;
    if (!valid_tag(src->tag))
        return P_TAG;

    dst[0] = src->tag;
    dst[1] = (unsigned char)(src->sequence >> 24);
    dst[2] = (unsigned char)(src->sequence >> 16);
    dst[3] = (unsigned char)(src->sequence >> 8);
    dst[4] = (unsigned char)src->sequence;
    dst[5] = (unsigned char)(src->count >> 8);
    dst[6] = (unsigned char)src->count;
    return P_OK;
}

static enum status decode(const unsigned char *src, size_t n,
                          struct message *dst)
{
```

```
    if (src == NULL || dst == NULL)
        return P_ARGUMENT;
    if (n != WIRE_SIZE)
        return P_SIZE;
    if (!valid_tag(src[0]))
        return P_TAG;

    struct message tmp = {
        .tag = src[0],
        .sequence = ((uint32_t)src[1] << 24) |
                    ((uint32_t)src[2] << 16) |
                    ((uint32_t)src[3] << 8) |
                    (uint32_t)src[4],
        .count = (uint16_t)(((uint32_t)src[5] << 8) |
                            (uint32_t)src[6])
    };

    *dst = tmp;
    return P_OK;
}

/* Külön vizsgálható, natív reprezentációt kezelő függvények. */
uint32_t load_aligned(const uint32_t *p)
{
    return *p;
}

void store_aligned(uint32_t *p, uint32_t value)
{
    *p = value;
}

uint32_t load_native_bytes(const unsigned char *p)
{
    uint32_t value;
    memcpy(&value, p, sizeof value);
    return value;
}

void store_native_bytes(unsigned char *p, uint32_t value)
{
    memcpy(p, &value, sizeof value);
}

static void expect_decode_error(const unsigned char *src, size_t n,
                                enum status expected)
{
    const struct message before = {2, UINT32_MAX, UINT16_MAX};
    struct message after = before;

    assert(decode(src, n, &after) == expected);
    assert(same(after, before));
```

```
}

int main(void)
{
    printf("message: size=%zu align=%zu\n",
           sizeof(struct message), _Alignof(struct message));
    printf("offsets: tag=%zu sequence=%zu count=%zu\n",
           offsetof(struct message, tag),
           offsetof(struct message, sequence),
           offsetof(struct message, count));
    printf("uint32_t: size=%zu align=%zu\n",
           sizeof(uint32_t), _Alignof(uint32_t));

    const unsigned char golden[WIRE_SIZE] = {
        0x01, 0x01, 0x23, 0x45, 0x67, 0x89, 0xab
    };
    const struct message expected = {
        1, UINT32_C(0x01234567), UINT16_C(0x89ab)
    };
    unsigned char bytes[WIRE_SIZE];
    struct message decoded = {0};

    assert(encode(bytes, sizeof bytes, &expected) == P_OK);
    assert(memcmp(bytes, golden, sizeof golden) == 0);
    assert(decode(golden, sizeof golden, &decoded) == P_OK);
    assert(same(decoded, expected));

    const uint32_t sequences[] = {
        0, 1, UINT32_C(0x80000000), UINT32_MAX
    };
    const uint16_t counts[] = {0, 1, UINT16_MAX};

    for (unsigned char tag = 1; tag <= 2; ++tag) {
        for (size_t i = 0;
             i < sizeof sequences / sizeof sequences[0]; ++i) {
            for (size_t j = 0;
                 j < sizeof counts / sizeof counts[0]; ++j) {
                struct message input = {tag, sequences[i], counts[j]};
                assert(encode(bytes, sizeof bytes, &input) == P_OK);
                assert(decode(bytes, sizeof bytes, &decoded) == P_OK);
                assert(same(input, decoded));
            }
        }
    }

    for (size_t n = 0; n < WIRE_SIZE; ++n)
        expect_decode_error(golden, n, P_SIZE);

    const unsigned char too_long[WIRE_SIZE + 1] = {1};
    expect_decode_error(too_long, sizeof too_long, P_SIZE);

    unsigned char bad_tag[WIRE_SIZE];
```

```
    memcpy(bad_tag, golden, sizeof bad_tag);
    bad_tag[0] = 3;
    expect_decode_error(bad_tag, sizeof bad_tag, P_TAG);
    expect_decode_error(NULL, WIRE_SIZE, P_ARGUMENT);

    assert(decode(golden, sizeof golden, NULL) == P_ARGUMENT);

    struct message invalid = expected;
    invalid.tag = 0;
    memset(bytes, 0xa5, sizeof bytes);
    assert(encode(bytes, sizeof bytes, &invalid) == P_TAG);
    for (size_t i = 0; i < sizeof bytes; ++i)
        assert(bytes[i] == 0xa5);

    puts("All protocol assertions passed.");
    return 0;
}
```

Fordítás mindkét target saját környezetében:

```
gcc -std=c17 -O2 -g -Wall -Wextra -Wpedantic -Wconversion \
    lab.c -o lab
```

A tesztet külön laborlépésként indítsd el; a jegyzőkönyvbe a tényleges kimenetet és kilépési státuszt másold. Az utolsó üzenet a program **várt sikerjelzése**, nem itt igazolt teszteredmény.

9.5.3.1 Tulajdonlás és érvényesség

Hozzáférés	Tulajdonos és élettartam	Előfeltétel, invalidálás és hibakimenet
`encode`, `decode` pointerei	a hívó objektumai; kölcsönzött hozzáférés a hívás végéig	a nem null pointer megfelelő élő objektumra/tartományra mutat; a forrás és cél nem fed át
`load_aligned`, `store_aligned`	a hívó élő `uint32_t` objektuma	megfelelő igazítás és olvashatóság/írhatóság; megsértésük UB
`load_native_bytes`, `store_native_bytes`	a hívó bájttartománya	legalább `sizeof(uint32_t)` elérhető C bájt; külön egész típusú igazítás nem kell
tesztadatok	`main` automatikus objektumai	élettartamuk a blokk végén ér véget; a függvények nem tárolnak el pointert

Az élettartam vége vagy a tároló felszabadítása érvénytelenítené a kölcsönzött hozzáférést. A hosszparaméter nem bizonyítja a tároló tényleges méretét: a hazudott hossz bounds-hibához vezethet. Az API hibakódjai nem helyettesítik ezeket az előfeltételeket.

9.5.4 A golden adat és a round-trip értelmezése

A **golden byte vector** a specifikációból, az encoder működésétől függetlenül előállított referencia-bájtsorozat. A példában:

$$\text{sequence} = 1 \cdot 2^{24} + 35 \cdot 2^{16} + 69 \cdot 2^{8} + 103 = \texttt{0x01234567}.$$

Ezért a négy továbbított oktett `01 23 45 67`. A `count` hasonlóan `89 ab`.

A **round-trip tulajdonság**:

$$D(E(m)) = m,$$

ahol m érvényes üzenet, E az encoder, D a decoder. Az egyenlőség itt mezőértékek egyenlőségét jelenti, nem a struktúra teljes objektumreprezentációjának azonosságát.

Bizonyításvázlat. Az encoder a sorszámot négy, egymást nem átfedő nyolcbites részre bontja. A decoder ezeket eredeti helyükre tolja, majd bitenkénti VAGY művelettel egyesíti. A számlálónál ugyanez két résszel történik. A `tag` változatlanul másolódik, ezért mindhárom mező helyreáll.

A teszt 24 határérték-kombinációt jár be, nem a teljes értéktartományt. A golden teszt azért is szükséges, mert két azonosan hibás, például little-endian encoder és decoder sikeres round-tripet adhatna.

A struktúrák összehasonlítására használt `memcmp` tipikus hiba lenne: a padding nem logikai mező. A wire bájttömbök összehasonlítására viszont itt megfelelő.

9.5.5 Load/store és unaligned hozzáférés assemblyben

A már definiált négy hozzáférési függvény assemblyjét mindkét gépen állítsd elő:

```
gcc -std=c17 -O2 -S lab.c -o lab.s
```

AArch64-en készíts külön változatot:

```
gcc -std=c17 -O2 -mstrict-align -S lab.c -o lab-strict.s
```

GCC-specifikus kapcsoló: a `-mstrict-align` AArch64 targeten használható; nem ISO C tulajdonság.

A következőket keresd, ne egy előre feltételezett teljes utasítássorozatot:

Függvény	x86-64-en lehetséges alak	AArch64-en lehetséges alak
igazított betöltés/tárolás	`movl` memóriaoperandussal	`ldr` / `str` W regiszterrel
bájtforrásból `memcpy`	szintén egyetlen gépi betöltés	konfigurációtól függően `ldr` vagy kisebb betöltések
bájtcélba `memcpy`	szintén egyetlen gépi tárolás	konfigurációtól függően `str` vagy kisebb tárolások

Ezek keresési támpontok, nem mért kimenetek. A `memcpy` nem kötelezően függvényhívás: a fordító a kisméretű másolást beépítheti.

A lényegi különbség a forrásprogram szerződése. A `uint32_t *` dereferálásához megfelelően igazított `uint32_t` objektum szükséges. A bájtforrásos `memcpy` nem követel ilyen igazítást a forrástól. A fordító ettől még használhat az adott környezetben megengedett unaligned gépi betöltést.

Az AArch64 sem jelent általános „unaligned hozzáféréskor mindig összeomlik” szabályt. Az utasítás, memóriatípus és rendszerbeállítás is számít; a hardver toleranciája pedig nem teszi definiálttá a hibás C kódot.

9.5.6 Sanitizer-lelet és helyes javítás

Az alábbi `bad.c` **szándékosan hibás, UB-t tartalmazó diagnosztikai példa**. Nincs garantált normál kimenete.

```
#include <limits.h>
#include <stdint.h>

#ifndef UINT32_MAX
#error "This example requires uint32_t"
#endif

_Static_assert(CHAR_BIT == 8, "Octet-based lab");
_Static_assert(_Alignof(uint32_t) > 1,
               "This experiment requires nontrivial alignment");

int main(void)
{
    _Alignas(uint32_t)
    unsigned char storage[sizeof(uint32_t) + 1] = {0};

    const unsigned char *borrowed = storage + 1;
    return (int)*(const uint32_t *)(const void *)borrowed;
}
```

A `storage` a tároló tulajdonosa; a `borrowed` a blokk végéig élő, kölcsönzött bájthozzáférés. A tartomány elég hosszú, de az egybájtos eltolás miatt a cím nem megfelelően igazított. Ráadásul a deklarált bájttömbön keresztül így nem kapunk szabályos `uint32_t` objektumhozzáférést sem.

```
gcc -std=c17 -O1 -g -Wall -Wextra \
    -fsanitize=alignment -fno-sanitize-recover=alignment \
    bad.c -o bad
```

A sanitizer várhatóan hibás igazítású betöltést jelez; a pontos szöveg és forráshely a fordítókiadástól függ. A jegyzőkönyvben az elvárt igazítást és az érintett műveletet azonosítsd. Egyetlen lelet nem a program összes hibájának felsorolása.

A helyes natív javítás a korábbi `load_native_bytes`: megfelelően igazított helyi `uint32_t` objektumba másolunk. A labor exact-width unsigned típusánál minden bitminta érvényes értéket képvisel.

Ez azonban **nem byte-order konverzió**. Wire adatokhoz a `decode` explicit mezőösszeállítása kell. A cast eltüntetése javíthatja az objektumhozzáférést, de önmagában nem valósítja meg a protokollt.

9.5.7 Feladatok

9.5.7.1 Layout dokumentálása

Futtasd a mérőprogramot mindkét profilon. Készíts táblázatot a tényleges méretről, igazításról és offsetekről. Indokold, miért nem bizonyít hordozható wire formátumot az eredmények egyezése.

9.5.7.2 Protokollteszt bővítése

Egészítsd ki a tesztet úgy, hogy az összes lehetséges oktettértéket ellenőrizze `tag` mezőként. Érvénytelen tag esetén ellenőrizd a kimenet változatlanságát is.

9.5.7.3 Diagnosztika és javítás

Dokumentáld a hibás program sanitizer-leletét. Készíts javított változatot `memcpy`-vel, majd hasonlítsd össze a négy hozzáférési függvény assemblyjét mindkét targeten. AArch64-en a `-mstrict-align` változatot is vizsgáld.

9.5.8 Megoldások

9.5.8.1 Layout dokumentálása

A referencia-profilokon a várakozás: `sizeof == 12`, `_Alignof == 4`, az offsetek pedig `0`, `4`, `8`. Eltéréskor először a tényleges ABI-t, kapcsolókat és típusokat ellenőrizd.

Az egyezés csak két konkrét profilra vonatkozó megfigyelés. A natív reprezentáció paddingot tartalmaz, mezősorrendjének változtatása módosíthatja a layoutot, és a little-endian mezők nem a protokoll big-endian reprezentációját adják. Ellenőrzési feltétel: a mérési táblázat mellé teljes környezeti profil tartozik.

9.5.8.2 Protokollteszt bővítése

A következő C17 részlet a `main` függvénybe illeszthető, a `golden` deklarációja után:

```
for (unsigned int t = 0; t < 256; ++t) {
    unsigned char candidate[WIRE_SIZE];
    memcpy(candidate, golden, sizeof candidate);
    candidate[0] = (unsigned char)t;

    if (t == 1 || t == 2) {
        struct message result = {0};
        assert(decode(candidate, sizeof candidate, &result) == P_OK);
        assert(result.tag == t);
    } else {
        expect_decode_error(candidate, sizeof candidate, P_TAG);
    }
}
```

A ciklusváltozó szándékosan nem `unsigned char`: annak körbefordulása könnyen végtelen ciklust okozna. Ellenőrzési feltétel: két elfogadott és 254 elutasított tag, hibánál változatlan kimenet.

9.5.8.3 Diagnosztika és javítás

A javított `bad.c` tartalmazza a `<string.h>` headert, és a hibás castos olvasás helyett ezt használja:

```
uint32_t value;
memcpy(&value, borrowed, sizeof value);
return (int)value;
```

Ez a megadott, nullákkal feltöltött bemenetre vonatkozó részlet. Ellenőrzési feltétel: ezen a hozzáférésen nincs alignment-lelet, és a bounds továbbra is helyes.

Az assemblyben a `memcpy` akár ugyanarra az utasításra egyszerűsödhet, mint a hibás castos olvasás. Ettől a két C program helyessége nem lesz azonos: az utasításazonosság nem bizonyítja a nyelvi előfeltételek teljesülését.

9.5.9 Összefoglaló

A layoutmérés egy konkrét build tulajdonságait mutatja meg; a wire formátumot külön szerződés rögzíti. A golden teszt a reprezentációt, a round-trip az oda-vissza értékmegőrzést, a negatív tesztek a hibaszerződést ellenőrzik. Az assembly megmagyarázza a végrehajtást, a sanitizer pedig hibákat tárhat fel, de egyik sem helyettesíti az igazítási, objektum-, bounds- és élettartam-előfeltételek indoklását.

10 Függvények, modulok és C API-tervezés

10.1 Függvényhívás, prototípus és paraméterátadás

Egy C API megértéséhez külön kell választani a függvénynek átadott **értéket** attól az **objektumtól**, amelyhez az érték hozzáférést biztosít. Egy számláló átadása nem engedi meg a hívó számlálójának módosítását; a címének átadása viszont igen. Ez nem kétféle paraméterátadási mechanizmus: C-ben mindkettő **érték szerinti átadás** (*pass-by-value*).

A fejezet a C17 nyelvi szabályait tárgyalja, nem azt, hogy az adott ABI regiszterben vagy memóriában továbbítja az értékeket. Az előzmények közül a struktúrák értékmásolása, a promóciók és konverziók, valamint a függvénypointerek típusszerződése lesz különösen fontos.

10.1.1 Az érték szerinti átadás pontos jelentése

Függvényhíváskor az argumentumkifejezések kiértékelődnek, és a szükséges konverziók után az értékeik inicializálják a függvény paraméterobjektumait. A paraméter a hívott függvény saját objektuma: módosítása önmagában nem módosítja azt az objektumot, amelyből az argumentum értéke származott.

Ez három gyakori esetben a következőt jelenti:

- **Skalárparaméter:** egy egész szám paramétere külön egész objektum.
- **Struktúraparaméter:** a struktúra értéke másolódik; a paraméter tagjai külön tagobjektumok.
- **Pointerparaméter:** a pointerérték másolódik, nem a referált objektum.

Az utolsó esetben két külön pointerobjektum ugyanarra az objektumra mutathat. A paraméter átállítása csak a helyi pointert változtatja meg, a dereferálás utáni értékadás azonban a közösen elért objektumot.

10.1.1.1 Bizonyításvázlat a pointeres esetre

Legyen a hívó pointerobjektuma p_c, a paraméterobjektum pedig p_f. Tegyük fel, hogy p_c egy élő, írható, megfelelő típusú és igazítású x objektumra mutat. A hívás kezdetén:

$$p_f = p_c = \&x.$$

A `p_f = NULL` értékadás célobjektuma p_f, ezért p_c változatlan marad. Ezzel szemben az ezt megelőző `*p_f = 42` értékadás célobjektuma x.

A különbséget tehát az értékadás bal oldala határozza meg. **A pointerparaméter nem referencia-paraméter:** nem válik a hívó pointerváltozójának másik nevévé.

Ha magát a hívó pointerobjektumát kell módosítani, annak címét kell átadni, például `unsigned **slot` paraméterként. Ekkor is pointerérték másolódik; a `*slot = replacement` a hívó pointerobjektumát módosítja.

10.1.2 Kidolgozott példa: három másolat, eltérő hatások

A következő teljes program minimuma ISO C17. Nem igényel konkrét egészszélességet vagy ABI-specifikus elrendezést.

Tervezett laborprofil: Linux x86-64, System V ABI, LP64; a `lab-next` profil (profiltábla), illetve a `lab-next` profil (profiltábla). A példát itt nem fordítottuk és nem futtattuk; az ellenőrzések a szabályokból levezetett elvárásokat rögzítik.

Az `api_call.c` tartalma:

```
#include <assert.h>
#include <limits.h>
#include <stddef.h>

struct sample {
    unsigned count;
    unsigned *shared;
};

enum add_status {
    ADD_OK,
    ADD_BAD_ARGUMENT,
    ADD_OVERFLOW
};

enum add_status add_unsigned(unsigned a, unsigned b,
                             unsigned *out);

static void change_scalar(unsigned value)
{
    value = 99u;
    (void)value;
}

static void change_record(struct sample value)
{
    value.count = 99u;
    *value.shared = 77u;
}

static void change_through_pointer(unsigned *p)
{
    *p = 42u;
    p = NULL;
    (void)p;
}

int main(void)
{
    unsigned n = 7u;
    struct sample s = {3u, &n};
    unsigned *p = &n;
```

```
    change_scalar(n);
    assert(n == 7u);

    change_record(s);
    assert(s.count == 3u);
    assert(n == 77u);

    change_through_pointer(p);
    assert(n == 42u);
    assert(p == &n);

    unsigned result = 123u;
    enum add_status status = add_unsigned(2u, 3u, &result);
    assert(status == ADD_OK);
    assert(result == 5u);

    status = add_unsigned(UINT_MAX, 1u, &result);
    assert(status == ADD_OVERFLOW);
    assert(result == 5u);

    status = add_unsigned(2u, 3u, NULL);
    assert(status == ADD_BAD_ARGUMENT);

    return 0;
}

enum add_status add_unsigned(unsigned a, unsigned b,
                             unsigned *out)
{
    if (out == NULL) {
        return ADD_BAD_ARGUMENT;
    }
    if (b > UINT_MAX - a) {
        return ADD_OVERFLOW;
    }

    *out = a + b;
    return ADD_OK;
}
```

Fordítás a rögzített profil megfelelő fordítójával:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wstrict-prototypes \
    -Wmissing-prototypes -Wconversion -Werror=implicit-function-declaration \
    -O0 -g api_call.c -o api-call-gcc

clang -std=c17 -Wall -Wextra -Wpedantic -Wstrict-prototypes \
    -Wmissing-prototypes -Wconversion -Werror=implicit-function-declaration \
    -O0 -g api_call.c -o api-call-clang
```

A parancsok a tanuló külön laborjához tartoznak; a `gcc` és `clang` tényleges verzióját ellenőrizni kell. Az `assert` ellenőrzéseihez ne legyen definiálva az `NDEBUG`.

A végrehajtásból elvárt állapotok:

Hívás után	n	s.count	p
`change_scalar(n)`	7	3	`&n`
`change_record(s)`	77	3	`&n`
`change_through_pointer(p)`	42	3	`&n`

A struktúramásolás nem mély másolás. A `value.count` külön tagobjektum, de a `value.shared` pointerértéke ugyanaz, mint az `s.shared` értéke. A referált `n` objektumból nem készül másolat.

10.1.2.1 Tulajdonlás és élettartam

Objektum vagy hozzáférés	Tulajdonos, tároló	Érvényesség és érvénytelenítés
`n`, `s`, `p`, `result`	A hívó, a `main` automatikus tárolójában	A befoglaló blokk végéig élnek
`s.shared`, `p`	Kölcsönzött hozzáférés `n`-hez	`n` élettartamának végén elveszítik használhatóságukat
Pointerparaméterek	A hívott függvény helyi pointerobjektumai	A visszatéréskor megszűnnek; a referált objektum ettől még élhet
`out`	A hívó eredményobjektumát kölcsönzi	A függvény nem tárolja el, nem szabadítja fel

A módosító segédfüggvények érvényes, írható objektumra mutató pointert követelnek. Nincs hibastátuszuk; null vagy dangling pointer dereferálása nem definiált viselkedés lenne. Az `add_unsigned` a null értéket külön kezeli, de egy nem null pointer érvényességét nem tudja általánosan ellenőrizni.

10.1.3 Deklaráció, prototípus és definíció

A **deklaráció** közli a függvény nevét és típusát. A **prototípus** olyan függvénydeklaráció, amely a paraméterek típusinformációját is megadja. A **definíció** ezen túl a függvénytörzset is tartalmazza.

A példában:

```
/* C17 deklarációrészlet: prototípus, függvénytörzs nélkül. */
enum add_status add_unsigned(unsigned a, unsigned b,
                             unsigned *out);
```

A deklaráció lehetővé teszi, hogy a később definiált függvényt a `main` megfelelő típusinformáció birtokában hívja. A paraméternevek nem részei a függvénytípusnak, de dokumentációként hasznosak.

Könyvtárban a prototípus közös headerbe kerüljön, amelyet a hívó és a megvalósítás is beilleszt. A linker sikeres névfeloldása önmagában nem bizonyítja a típusok kompatibilitását.

10.1.3.1 `f(void)` és `f()` C17-ben

Alak	C17 jelentés
`int f(void);`	Prototípus: a függvény nem fogad argumentumot
`int f();`	Nem prototípus: nincs információ a paraméterek számáról és típusáról
`int f() { return 0; }`	Paraméter nélküli definíció, de nem prototípusos alak

Az `int f();` nem azt jelenti, hogy tetszőleges argumentumlista helyes, és nem variadikus deklaráció. Csupán hiányzik az a típusinformáció, amellyel a fordító a hívást teljesebben ellenőrizhetné.

C23-változás: az üres `()` paraméterlista nulla paramétert meghatározó prototípusos alak. C17-kompatibilis kódban továbbra is az egyértelmű `f(void)` alakot használjuk. A fordító C23-módjának támogatását külön kell ellenőrizni.

A deklaráció teljes hiánya más probléma: C17-ben az implicit függvénydeklaráció nem megengedett. Az olyan hívás, amelynél a függvény neve nincs megfelelően deklarálva, diagnosztikát igényel.

10.1.4 Argumentumkonverzió és inkompatibilis hívás

Látható prototípus mellett az argumentumok a megfelelő paramétertípushoz igazodnak. Ez nem jelent veszteségmentességet. Az `int` paraméternek átadott `3.75` például `3` értékké konvertálódik; tartományon kívüli lebegőpontos értéknél a helyzet már nem biztonságos.

Nem prototípusos C17-hívásnál az **alapértelmezett argumentumpromóciók** (*default argument promotions*) érvényesülnek: például a `float` érték `double` lesz, a kisebb egész típusok pedig egészpromóción mennek át. A hívó ilyenkor nem tud a tényleges paraméterlistához igazodni.

Három hibát érdemes elkülöníteni:

1. **Látható prototípus megsértése:** például egy `void stop(void)` függvény hívása `stop(1)` alakban. Ez fordítási diagnosztikát igényel.
2. **Fordítási egységek közötti eltérés:** a hívó és a definíció inkompatibilis függvénytípust használ. A hagyományos linkelés ezt nem feltétlenül ismeri fel.
3. **Inkompatibilis függvénypointeren keresztüli hívás:** egy cast nem teszi kompatibilissé a függvénytípusokat. Például egy `long(long)` függvény `int (*)(int)` típuson keresztüli meghívása nem definiált viselkedés.

A javítás nem a warning elnémítása, hanem az egységes prototípus és a helyes konverziós szerződés.

Az argumentumok kiértékelési sorrendjére sem szabad balról jobbra történő végrehajtást feltételezni. A C17 szerinti `f(i++, i++)` hívás nem definiált viselkedés: az érték szerinti átadás nem rendezi sorba a mellékhatásokat.

10.1.5 Visszatérési érték és output paraméter

A `return` kifejezés értéke a függvény visszatérési típusára konvertálódik. Skalár vagy struktúra érték szerinti visszaadása nem igényli a lokális eredményobjektum további életben maradását. Lokális objektum címének visszaadása viszont nem hosszabbítja meg annak élettartamát.

Tervezési szabályként minden szabályosan visszatérő, nem `void` út adjon explicit eredményt. C17-ben a nem `void` függvény végére jutás után a hívás értékének felhasználása UB; a `main` végére jutás speciálisan nulla visszaadásának felel meg.

Az `add_unsigned` két csatornát választ szét:

- a visszatérési érték a művelet státusza;
- az `out` a sikeres művelet eredményének helye.

Szerződése szerint hiba esetén az eredményobjektum változatlan. A túlcsordulás-ellenőrzés levezetése matematikai egészekkel, $M = \texttt{UINT_MAX}$ jelöléssel:

$$a + b \le M \quad \Longleftrightarrow \quad b \le M - a.$$

Mivel $0 \le a \le M$, az `M - a` kivonás ábrázolható. Ezért előbb ellenőrizhető a feltétel, és csak utána szükséges összeadni.

Gyakori hiba előbb írni az `*out` objektumba, majd hibát jelezni: ez megsértené a „hiba esetén változatlan” ígéretet. Ugyanilyen hiba a státusz figyelmen kívül hagyása és az eredmény automatikus érvényesnek tekintése.

10.1.6 Feladatok

1. **Magyarázat:** miért marad változatlan az `s.count`, miközben az `n` megváltozik a struktúraparaméteres hívásban? Miért nem lesz null a hívó `p` változója?
2. **Kódolás:** készíts `multiply_unsigned` függvényt ugyanolyan státusz- és output-szerződéssel, mint az összeadás. Null output esetén jelezzen argumentumhibát, ábrázolhatatlan szorzatnál túlcsordulást.
3. **Diagnosztika:** két külön C17 fordítási egység lényegi részletei következnek. A második szándékosan hibás hívást tartalmaz. Mi történik az argumentummal, és hogyan javítanád?

```c
/* scale.c - C17 részlet */
int scale(int value)
{
    return value;
}

/* client.c - hibás C17 részlet */
int scale();

int invoke(void)
{
    return scale(1.0f);
}
```

10.1.7 Megoldások

10.1.7.1 A másolat és az elért objektum

A `value.count` a paraméterstruktúra tagja, ezért módosítása nem érinti az `s.count` tagot. A `value.shared` viszont ugyanarra az `n` objektumra mutat, mint az eredeti pointertag. Hasonlóképpen a `p = NULL` csak a helyi pointerparamétert módosítja.

Ellenőrzési feltétel: külön meg kell nevezni az értékadás célobjektumát és a referált objektumot.

10.1.7.2 Ellenőrzött szorzás

Az előző program deklarációit és include-jait használó C17-részlet:

```
enum add_status multiply_unsigned(unsigned a, unsigned b,
                                  unsigned *out)
{
    if (out == NULL) {
        return ADD_BAD_ARGUMENT;
    }
    if (a != 0u && b > UINT_MAX / a) {
        return ADD_OVERFLOW;
    }

    *out = a * b;
    return ADD_OK;
}
```

Pozitív a mellett a feltétel $b \leq \lfloor M/a \rfloor$. A nullás eset külön kezelése kizárja a nullával osztást. A tulajdonos továbbra is a hívó; az `out` csak a hívás idejére kölcsönzött, írható hozzáférés.

Ellenőrzendő: szorzás nullával, `UINT_MAX` szorzása eggyel és kettővel, null output, valamint az eredmény változatlansága hibánál.

10.1.7.3 A hiányzó prototípus következménye

A `float` argumentum a nem prototípusos hívásban `double` értékké promótálódik, miközben a definíció `int` paramétert vár. A hívás viselkedése nem definiált; nincs garantált eredménye.

A javítás közös headerben az `int scale(int value);` prototípus, mindkét fordítási egységbe beillesztve. Ekkor az `1.0f` argumentum szabályosan `int` értékké konvertálódik. Az ellenőrzési feltétel nem pusztán a sikeres linkelés, hanem a közös, kompatibilis deklaráció használata.

10.1.8 Összefoglaló

C-ben a skalár, a struktúra és a pointer egyaránt érték szerint adódik át. A pointermásolat közös objektumhoz adhat hozzáférést, de nem lesz a hívó pointerváltozójának referenciája. A megbízható API látható prototípussal, kompatibilis definícióval, explicit élettartam-feltételekkel és egyértelmű sikeres, illetve hibás kimeneti állapottal rendelkezik.

10.2 Variadikus függvények és típusbiztonság

A variadikus függvény (*variadic function*) változó számú argumentumot fogadhat. C-ben azonban a rugalmasságért a típusellenőrzés részleges elvesztésével fizetünk: a függvény nem kap automatikusan sem argumentumdarabszámot, sem futásidejű típusleírást. A hívónak és a hívottnak ezért külön szerződésben kell megállapodnia.

A legismertebb példa a `printf`: első argumentuma egyszerre szöveg és a további argumentumokat leíró séma. Megértéséhez a Függvényhívás, prototípus és paraméterátadás, valamint a Promóciók, signedness és túlcsordulás szabályaira építünk. Az alábbi nyelvi magyarázat ISO C17-re vonatkozik; a fordítói attribútumot külön bővítésként kezeljük.

10.2.1 Az ellipszis és a rejtett szerződés

A prototípus végén álló `...`, azaz ellipszis jelzi a változó argumentumlistát:

```
/* C17 deklarációrészlet. */
int count_positive(int count, ...);
```

C17-ben az ellipszist legalább egy névvel rendelkező paraméternek kell megelőznie. A rögzített paraméterekre a prototípus szerinti ellenőrzés és konverzió érvényes. Az ellipszishez tartozó argumentumokhoz viszont nincs deklarált céltípus.

A fenti API szerződése lehet:

- `count` nemnegatív;
- utána pontosan `count` darab, promóció után `int` típusú argumentum következik;
- az eredmény a pozitív értékek száma;
- negatív `count` esetén az eredmény `-1`, argumentumolvasás nélkül.

A fordító a deklarációból nem tudja ellenőrizni a második feltételt. A `count` csak programadat: nem köti automatikusan a nyelv a tényleges argumentumlistához.

A variadikus függvény darabszám helyett használhat formátumsztringet, lezáró jelzőértéket vagy külön típusleírókat. Mindegyik **külső protokoll**, nem a `...` által szolgáltatott információ.

10.2.2 Default argument promotions és a printf

Az ellipszisnek átadott argumentumokra az **alapértelmezett argumentumpromóciók** (*default argument promotions*) vonatkoznak:

Eredeti típus	Átadott típus
`float`	`double`
`_Bool`, `signed char`, `short`	`int`
`char`, `unsigned char`, `unsigned short`	`int`, ha minden értékük ábrázolható benne; különben `unsigned int`
`int`, `unsigned int`, `long`, `double`	Változatlan

A `char` esetén tehát az implementáció által meghatározott előjelességet és az értéktartományt is figyelembe kell venni.

Fontos, hogy az `int` **nem** változik automatikusan `double`-lé attól, hogy a formátumban `%f` szerepel. A formátumsztring nem vesz részt a nyelvi argumentumkonverzióban.

```c
/* C17 részlet; környező függvény és <stdio.h> szükséges. */
float rate = 1.5f;
signed char delta = -2;

printf("rate=%.1f delta=%d\n", rate, delta);
```

A hívás lépései:

1. A `rate` értéke `float` típusból `double` típusra promoválódik.
2. A `delta` értéke `signed char` típusból `int` típusra promoválódik.
3. A `printf` a `%f` alapján `double`, a `%d` alapján `int` argumentumot vár.
4. Az átadott és elvárt típusok megfelelnek egymásnak.

Gyakori formátumszerződések:

Formátum	Elvárt argumentum
`%d`	`int`
`%u`	`unsigned int`
`%ld`	`long`
`%zu`	`size_t`
`%f`	`double`
`%Lf`	`long double`
`%s`	Karaktertömb elejére mutató pointer; ebben a fejezetben nullterminált sztringet követelünk meg
`%p`	`void *`

A `printf` esetén a `%lf` ugyanúgy `double` argumentumot kezel, mint a `%f`; ezt nem szabad más formázási API-kra automatikusan általánosítani.

A `%p` esetén objektumpointert explicit `(void *)` konverzióval adj át: az ellipszis önmagában nem alakítja például az `int *` argumentumot `void *` típusúvá. A `*`-gal megadott mezőszélesség és pontosság külön-külön egy `int` argumentumot fogyaszt.

10.2.3 A va_list életciklusa

A `<stdarg.h>` biztosítja a változó argumentumlista bejárását. A `va_list` implementációfüggő bejárási állapot, nem hordozható „pointer a veremre”.

Művelet	Jelentés és feltétel
`va_start(ap, last)`	Inicializálja a bejárást; `last` az utolsó név szerinti paraméter
`va_arg(ap, T)`	Kiolvassa a következő argumentumot `T` típussal, és továbblépteti az állapotot
`va_copy(dst, src)`	Önálló bejárási állapotot készít `src` aktuális pozíciójáról
`va_end(ap)`	Lezárja az adott inicializált állapot használatát

Minden `va_start` és `va_copy` művelethez a megfelelő `va_end` tartozik, ugyanabban a függvényben. Ez korai visszatérésnél is kötelező. A `va_end` nem szabadítja fel az argumentumként átadott erőforrásokat.

C17-ben a `va_start` utolsó név szerinti paramétere nem lehet `register` deklarációjú, tömb- vagy függvénytípusú, illetve a saját promoválódott típusával inkompatibilis típusú. Ezért például `float`, `char` vagy `short` helyett célszerű `int`-et, illetve megfelelő pointerparamétert használni.

A `va_list` másolására ne használj egyszerű értékadást vagy `memcpy`-t: egyes implementációkon tömbtípus, máshol összetettebb állapot. A hordozható művelet a `va_copy`.

Ha egy meghívott segédfüggvény `va_arg`-gal fogyasztja az átadott listát, a hívó nem feltételezheti annak változatlan pozícióját. Megőrzéshez előzetesen külön másolatot kell készíteni.

10.2.4 Kidolgozott példa: számlálás és kétcélú naplózás

Az alábbi teljes C17 fordítási egység két szerződést mutat be. A `count_positive` darabszám alapján olvas, a `mirror_log` pedig ugyanazt a formázott üzenetet két streamre írja.

A laborhoz választott profil: Linux x86-64 System V, a `lab-next` profil (profiltábla). Az attribútum nélküli mag ISO C17; az attribútum GCC/Clang-bővítés. A példa itt nem fordított vagy futtatott mérési eredmény.

```
/* variadic_demo.c */
#include <stdarg.h>
#include <stdio.h>

#if defined(__GNUC__) || defined(__clang__)
#define PRINTF_LIKE(fmt, args) \
    __attribute__((format(printf, fmt, args)))
#else
#define PRINTF_LIKE(fmt, args)
#endif

static int count_positive(int count, ...)
{
    if (count < 0) {
        return -1;
    }

    va_list ap;
    va_start(ap, count);

    int result = 0;
    for (int i = 0; i < count; ++i) {
        int value = va_arg(ap, int);
        if (value > 0) {
            ++result;
        }
    }

    va_end(ap);
    return result;
}

static int mirror_log(FILE *first, FILE *second,
                      const char *format, ...)
```

```
    PRINTF_LIKE(3, 4);

static int mirror_log(FILE *first, FILE *second,
                      const char *format, ...)
{
    va_list ap;
    va_list copy;

    va_start(ap, format);
    va_copy(copy, ap);

    int a = vfprintf(first, format, ap);
    int b = vfprintf(second, format, copy);

    va_end(copy);
    va_end(ap);

    return (a < 0 || b < 0) ? -1 : 0;
}

int main(void)
{
    float rate = 1.5f;
    signed char delta = -2;
    int positives = count_positive(4, -3, 0, 7, 2);

    int status = mirror_log(
        stdout, stderr,
        "rate=%.1f delta=%d positive=%d\n",
        rate, delta, positives
    );

    return status < 0 ? 1 : 0;
}
```

Fordítás a kiválasztott GCC-vel:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wformat=2 \
    -O0 -g variadic_demo.c -o variadic_demo
```

A számlálás várt eredménye 2. A naplózó mindkét streamre a `rate=1.5 delta=-2 positive=2` sort próbálja kiírni. A két stream megfigyelhető sorrendjére ebből nem következik garancia.

A másolatot **az első fogyasztás előtt** készítjük. A `vfprintf` után az adott lista nem használható újra következő bejárásként; a megfelelő `va_end` azonban szükséges.

A kölcsönzési szerződés:

Erőforrás vagy tároló	Tulajdonos és élettartam	Érvénytelenítés, hibakimenet
A két stream	A hívó/környezet kezeli; a naplózó csak kölcsönzi őket	A hívás alatt nyitott, bájtos kimenetre használható streamek szükségesek

Erőforrás vagy tároló	Tulajdonos és élettartam	Érvénytelenítés, hibakimenet
Formátumsztring	A hívó biztosítja; a példában statikus élettartamú sztringliterál	A hívás végéig érvényes és változatlan
Esetleges `%s` adatok	A hívó tulajdona; a naplózó nem tárolja el a pointereket	Felszabadítás vagy érvénytelen sztring szerződéssértés, nem megbízható hibakód
Két `va_list` állapot	A `mirror_log` kezeli	Saját `va_end` zárja le mindkettőt

A `mirror_log` `-1` értéke valamelyik `vfprintf` hibáját jelzi. Részleges kimenet már létrejöhetett, visszagörgetés nincs. A siker sem jelent tartós tárolást vagy későbbi pufferürítési hibák kizárását.

10.2.5 Típuseltérés és diagnosztika

A `va_arg(ap, T)` nem típuskonverziót kér: azt állítja, hogy a következő argumentum megfelelő típusú.

```
/* Szándékosan hibás C17 részletek; nem futtatandók. */
printf("%f\n", 1);              /* int helyett double kellene */
printf("%ld\n", 1);             /* int helyett long kellene */
count_positive(3, 10, 20);      /* nincs harmadik olvasható érték */
```

Ezek nem definiált viselkedéshez, **UB-hez** vezetnek. Nincs garantált kimenet, és a látszólagos működés sem bizonyít helyességet. Hasonlóan hibás a `va_arg(ap, float)`, ha a hívó `float` kifejezést adott át: az ténylegesen `double`-ként érkezett.

Az ISO C17 `va_arg`-szabályának vannak szűk kivételei: megfelelő signed/unsigned típuspárnál, ha az érték mindkettőben ábrázolható, illetve `void *` és karaktertípusra mutató pointer párosánál. Ezekből nem következik általános típusátjárhatóság, és nem írják felül a `printf` saját formátumszerződését.

A fel nem használt argumentum önmagában nem UB: a `printf("%d\n", 1, 2)` a második értéket figyelmen kívül hagyja. A túl kevés argumentum viszont veszélyes, mert a formátum nem létező adat kiolvasását kérheti.

A példabeli `format(printf, 3, 4)` attribútum jelentése: a harmadik paraméter formátumsztring, az ellenőrzendő argumentumok a negyediktől kezdődnek. Ez nem módosítja az átadást, csak diagnosztikai információt ad a fordítónak. Nem ISO C17-képesség, és nem általános futásidejű védelem.

Különösen fontos, hogy külső szöveg ne váljon formátummá. A `printf(input)` helyett `printf("%s", input)` szükséges, feltéve, hogy `input` érvényes, nullterminált sztringre mutat. A sztring tárolójának a teljes hívás alatt élnie kell.

10.2.6 Típusos alternatívák

Homogén adatokhoz rendszerint jobb a pointer–darabszám interfész:

```
/* C17 deklarációrészlet; <stddef.h> szükséges. */
size_t count_positive_array(const int *values, size_t count);
```

Itt a fordító ellenőrzi az elemtípust, bár a tömbhatárt továbbra is a szerződés rögzíti. A függvény kölcsönzi a tömböt, nem tárolja el és nem szabadítja fel. `count > 0` esetén legalább `count` olvasható `int` szükséges; `count == 0` esetén megengedhető a null pointer.

Eltérő típusú adatokhoz névvel ellátott struktúra vagy típusjelölős `union`-elemek tömbje használható. Ezek hosszabb deklarációért cserébe ellenőrizhetőbb interfészt adnak. Variadikus API-t akkor érdemes választani, ha a rugalmasság valóban indokolja a rejtett szerződés költségét.

10.2.7 Feladatok

1. **Magyarázó:** Milyen típusként érkezik a példában `rate` és `delta`? Miért nem javítaná ki a `%f` formátum egy közvetlenül átadott `1` argumentum típusát?
2. **Kódolási:** Valósítsd meg a `count_positive_array` függvényt. Üres bemenetre fogadjon el null pointert, és ne dereferálja azt.
3. **Diagnosztikai:** Miért hibás a `mirror_log`, ha a második `vfprintf` is `ap`-t kap? Mi történik, ha a `va_copy` csak az első `vfprintf` után kerülne? Melyik korábbi hibát várhatjuk formátumellenőrzéstől, és melyiket nem?

10.2.8 Megoldások

10.2.8.1 Promóció és formátum

A `rate` `double`, a `delta` `int` típusként érkezik. A formátum feldolgozása a hívott függvényben történik; az argumentumkonverzió már a hívás részeként megtörtént. Az `1` ezért `int` marad.

Ellenőrzési feltétel: a javítás például `printf("%f\n", 1.0)` legyen; ne pusztán a warning elnyomása.

10.2.8.2 Típusos számláló

Önálló C17 fordítási egység:

```
/* positive_array.c */
#include <stddef.h>

size_t count_positive_array(const int *values, size_t count)
{
    size_t result = 0;

    for (size_t i = 0; i < count; ++i) {
        if (values[i] > 0) {
            ++result;
        }
    }

    return result;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic \
    -c positive_array.c -o positive_array.o
```

Bizonyításvázlat: minden iteráció előtt `result` az első `i` elem pozitív értékeinek száma, ezért $0 \le result \le i \le count$. Az eredmény nem csordul túl. Üres bemenetnél a ciklustörzs nem fut.

Ellenőrzési feltétel: `(NULL, 0)` eredménye `0`; a `{-3, 0, 7, 2}` tömbé `2`. A művelet $O(count)$ idejű, folytonos hozzáférésű, nem allokál és nem érvénytelenít tárolót.

10.2.8.3 Fogyasztott lista és ellenőrzési korlát

Az első `vfprintf` elfogyasztja a bejárási állapotot; ugyanazzal nem indítható újra a lista. Utólagos másolása sem állítja vissza a kezdetet. A javítás az előzetes `va_copy`, majd mindkét állapot lezárása.

Ellenőrzési feltétel: minden inicializált lista minden visszatérési úton kapjon `va_end` hívást. A literális formátumhoz adott hibás típusra a fordító figyelmeztethet; a `count_positive(3, 10, 20)` saját darabszámszerződését a formátumellenőrzés nem ismeri.

10.2.9 Összefoglaló

A variadikus C API típus- és darabszámszerződését a programozó biztosítja. A promóciókat az argumentum eredeti típusa, nem a formátum vezérli. A `va_list` szabályos inicializálást, `va_copy` alapú másolást és lezárást igényel. A fordítói formátumellenőrzés hasznos védőháló, de ahol lehet, a típusos tömb- vagy struktúrainterfész erősebb alapot ad.

10.3 static inline, `_Generic` és headeres eszköztár

A headerbe írt segédfüggvény egyszerre nyelvi és fordításszervezési döntés. Nyelvi kérdés, hogy melyik fordítási egységhez tartozik a definíció, milyen kapcsolása van, és hogyan ellenőrizhetők az argumentumok. Ettől különálló optimalizálási kérdés, hogy a fordító létrehoz-e tényleges gépi függvényhívást.

A makrókról szóló rész megmutatta a szöveges helyettesítés veszélyeit; a függvényhívás tárgyalása pedig a típusos paraméterátadás előnyeit. Most ezeket kapcsoljuk össze: a `static inline` végzi a típusos műveletet, a `_Generic` pedig szükség esetén kiválasztja a megfelelő függvényt. A megoldás megértéséhez a nyelvi kapcsolás belső és külső változatának megkülönböztetésére is szükség lesz.

10.3.1 Az inline jelentése és a tényleges gépi beillesztés

ISO C17: az `inline` függvényspecifikátor. Nem új kapcsolási kategória, nem makróhelyettesítés, és nem kötelezi a fordítót arra, hogy a függvény törzsét a hívás helyére illessze.

A tényleges gépi beillesztés, azaz *inlining*, optimalizálói transzformáció. A fordító:

- egy `inline` függvényt is megvalósíthat tényleges hívással;
- egy `inline` nélkül definiált függvényt is beilleszthet;
- különböző hívási helyeken különbözően dönthet.

A döntést befolyásolhatja az optimalizálási profil, a függvénytörzs láthatósága, a kódméret és a célarchitektúra. Az `inline` tehát nem teljesítménygarancia. A hívási konvenció részleteit később, a hívási ABI tárgyalásakor vizsgáljuk.

10.3.1.1 C17 és GNU89: azonos íráskép, eltérő definíciós szabály

A következő táblázat fájlhatókörű **függvénydefiníciókra** vonatkozik. Feltételezzük, hogy ugyanabban a fordítási egységben nincs további, a helyzetet megváltoztató deklaráció.

Definíció alakja	ISO C17	GNU89 inline-szemantika
`static inline int f(int x) { ... }`	Belső kapcsolású függvény definíciója	Szintén belső kapcsolású
`inline int f(int x) { ... }`	Inline definíció; nem biztosít külső definíciót	Külső definíciót biztosít
`extern inline int f(int x) { ... }`	Külső definíciót biztosít	Nem biztosítja a külön hívható külső definíciót

ISO C17: ha egy külső kapcsolású függvény összes fájlhatókörű deklarációja `inline`, és egyik sem tartalmaz `extern` specifikátort, akkor a definíciója *inline definíció*. Ez nem helyettesíti a programban szükséges külső definíciót. Ha a függvényt hívjuk, a teljes programnak biztosítania kell azt akkor is, ha egy adott optimalizált build látszólag nélküle is linkelhető.

Ezért veszélyes a headerben a puszta `inline`: optimalizálás nélkül előjöhet a hiányzó külső definíció miatti linkerhiba. Az `extern inline` vak hozzáadása sem általános javítás: C17-ben több fordítási egységben több külső definíciót eredményezhet.

GNU-fordítóbővítés: a GNU89-szabályokat ne azonosítsuk minden GNU nyelvi móddal. Például GCC-ben a `-std=gnu17` önmagában nem jelent GNU89 inline-szemantikát; a `-fgnu89-inline` külön kapcsoló. A könyv példái explicit `-std=c17` módot használnak, ilyen felülbírálás nélkül.

Külső kapcsolású inline definíciókra további C17-korlátozások is vonatkoznak: például nem hivatkozhatnak belső kapcsolású azonosítóra, és nem definiálhatnak módosítható statikus tárolási idejű objektumot. Egyszerű headeres segédfüggvényhez ezért általában a `static inline` a könnyebben indokolható választás.

10.3.2 Típusos headerhelper static inline használatával

A `static` itt **belső kapcsolást** ad. Minden beemelő fordítási egység saját függvénydefiníciót kap. Ez nem okoz azonos nevű külső szimbólumok közötti ütközést.

Készítsünk telítődő növelést számlálókhoz. A szerződés:

- a bemenet érték szerint átadott `int` vagy `long`;
- a legnagyobb ábrázolható érték változatlan marad;
- minden más érték eggyel nő;
- nincs allokáció, megőrzött állapot vagy hibakód.

A példák ISO C17-et igényelnek. A parancsokhoz választható rögzített laborprofil: Linux x86-64 System V ABI, LP64, a `lab-next` profil (profiltábla). A kód nem használ profilspecifikus típusszélességet. Az alábbiak fordítási és ellenőrzési utasítások, nem dokumentált futtatási eredmények.

A `counter_util.h` teljes tartalma:

```c
#ifndef COUNTER_UTIL_H
#define COUNTER_UTIL_H

#include <limits.h>

static inline int counter_inc_int(int value)
{
    return value == INT_MAX ? INT_MAX : value + 1;
}

static inline long counter_inc_long(long value)
{
    return value == LONG_MAX ? LONG_MAX : value + 1L;
}

#define counter_inc(value) \
    _Generic(+(value),      \
        int: counter_inc_int, \
        long: counter_inc_long \
    )((value))

#endif
```

A `main.c`:

```c
#include <assert.h>

#include "counter_util.h"

int main(void)
{
    int attempts = 3;

    int next = counter_inc(attempts++);
    assert(attempts == 4);
    assert(next == 4);

    assert(counter_inc(INT_MAX) == INT_MAX);
    assert(counter_inc(LONG_MAX) == LONG_MAX);
    assert(counter_inc(0L) == 1L);

    return 0;
}
```

Fordítás:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 main.c -o counter-gcc
clang -std=c17 -Wall -Wextra -Wpedantic -O2 main.c -o counter-clang
```

Az ellenőrzéshez a tanuló külön futtathatja a két programot. Elvárt tulajdonság, hogy az engedélyezett `assert` ellenőrzések teljesülnek.

Bizonyításvázlat a túlcsordulás kizárására. Jelölje M az adott előjeles típus maximumát, x pedig a paramétert. Ha $x = M$, a feltételes operátor nem értékeli ki az összeadást. Egyébként az

egész értékek diszkrétsége miatt $x \leq M - 1$, tehát $x + 1 \leq M$. Az alsó korlátot a növelés nem sértheti meg. A művelet ezért minden érvényes bemeneti értékre definiált.

A műveleti költség mindkét ágon $O(1)$, dinamikus allokáció és tárolóérvénytelenítés nélkül. A tényleges utasításszám ettől még fordító- és célfüggő.

10.3.2.1 A belső kapcsolás következményei

Két fordítási egységben a header azonos nevű `static inline` függvényei külön függvényeket jelölnek. Függvénycímüket ezért ne használjuk fordítási egységeken átívelő közös azonosítóként.

Ha ilyen függvénybe módosítható lokális `static` objektum kerül, annak állapota is külön példány lesz minden fordítási egységben. Ez meglepő hibát okozhat például „globális” hívásszámláló vagy cache megvalósításakor. Közös állapothoz vagy egyetlen közös függvényidentitáshoz inkább headerbeli deklaráció és egy `.c` fájlban elhelyezett definíció való.

10.3.3 Típusválasztás _Generic segítségével

ISO C17, C11 óta: a `_Generic` generikus választás (*generic selection*). Egy vezérlő kifejezés típusa alapján választ az asszociációk között:

```
/* Szintaxisvázlat, nem önálló program. */
_Generic(vezerlo_kifejezes,
    tipus_a: kifejezes_a,
    tipus_b: kifejezes_b,
    default: alapertelmezett_kifejezes)
```

A vezérlő kifejezés **nem értékelődik ki**. A kiválasztás típuskompatibilitáson alapul, nem az értéken vagy a típus nevének szövegén.

Az asszociációk típusai teljes, nem variábilisan módosított objektumtípusok lehetnek. Két kompatibilis típust nem sorolhatunk fel külön: például egy `typedef` név és az általa jelölt típus nem két külön választási lehetőség. Legfeljebb egy `default` adható meg. Ha nincs megfelelő típus és nincs `default`, az diagnózist igénylő korlátmegsértés.

A példánk lényegében ezt végzi:

```
/* Magyarázó részlet a fenti makró szerkezetéről. */
_Generic(+(value),
    int: counter_inc_int,
    long: counter_inc_long
)((value))
```

A kiválasztott kifejezés egy függvényjelölő; ezt követi a tényleges függvényhívás.

10.3.3.1 Miért nincs többszörös kiértékelés?

A `counter_inc(attempts++)` esetén:

1. A vezérlő kifejezés `+(attempts++)`, de nem értékelődik ki.
2. Típusa alapján a választás `counter_inc_int`.
3. A tényleges argumentumként szereplő `attempts++` egyszer értékelődik ki.
4. A függvény a növelés előtti érték másolatát kapja.

Az argumentum tehát kétszer szerepel a helyettesített szövegben, de csak egyszer kerül kiértékelésre. A szöveges előfordulások száma önmagában nem bizonyít kiértékelési hibát.

Az unáris `+` tudatos interfészdöntés: elvégzi az egész típusú promóciókat. A makró olyan aritmetikai kifejezést fogad el, amelynek típusa ezek után `int` vagy `long`. Így például a `signed char` is az `int`ághoz kerül; az `unsigned int`, a `double` és a pointer nem támogatott. A `default` hiánya itt védelmi eszköz.

A `_Generic` nem általános mechanizmus tetszőleges hibás kód elrejtésére. A ki nem választott asszociációkat is szintaktikailag és szemantikailag ellenőrizni kell. Ha például egy ágban `(x)->field` szerepel, attól az még hibás lehet, hogy az adott hívásnál másik ág választódna ki. A függvényjelölők kiválasztása ezt a gyakori tervezési csapdát elkerüli.

10.3.4 Makró vagy inline függvény: hol jelentkezik a költség?

A makró nem feltétlenül gyorsabb a függvénynél. Az optimalizáló egy látható függvénytörzsből gyakran ugyanazt a gépi kódot állíthatja elő. A valódi különbség elsősorban a szemantikai szerződés.

Tulajdonság	Kifejezésmakró	`static inline` függvény
Argumentumkezelés	Szöveges helyettesítés	Típusos paraméterátadás
Többszörös kiértékelés	A makrótörzstől függ	Argumentumonként egyszer
Tényleges hívás	Nincs saját függvényhívása	Elmaradhat vagy megmaradhat
Diagnosztika	Helyettesítés után keletkezik	A függvénytörzs külön ellenőrizhető
Header módosítása	A fogyasztók újrafordítása szükséges	Ugyanúgy szükséges

A függvény argumentumonkénti egyszeri kiértékelése nem rögzíti több argumentum egymáshoz viszonyított sorrendjét. Az `f(i++, i++)` C17-ben továbbra is hibás.

Mindkét headeres megoldás növelheti a fordítási munkát, a tényleges beillesztés pedig a gépi kód méretét és az utasításcache terhelését. Teljesítményállításhoz mérés szükséges, nem elegendő az `inline` szó jelenléte.

10.3.5 Könyvtári példa: tgmath.h

ISO C17: a `<tgmath.h>` típusgenerikus matematikai makrókat biztosít. Például a `sqrt` megfelelő argumentummal a `sqrtf`, `sqrt` vagy `sqrtl` műveletének felel meg. A könyvtár nem köteles ezt éppen `_Generic` használatával megvalósítani.

Önálló C17-példa, `math_example.c`:

```
#include <tgmath.h>

int main(void)
{
    float root = sqrt(9.0f);
    double power = pow(2.0f, 3);
    long double wide_root = sqrt(9.0L);
```

```
    (void)root;
    (void)power;
    (void)wide_root;
    return 0;
}
```

A `pow(2.0f, 3)` fontos részlet: a típusválasztásban az egész argumentum `double` választását eredményezi, ezért itt nem a `powf` műveletét kapjuk. Komplex argumentumoknál a megfelelő komplex művelet is kiválasztható, amennyiben az adott makróhoz létezik ilyen változat.

Linux/glibc profilban a matematikai könyvtár explicit linkelésével:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 \
    math_example.c -o math-example -lm
```

A `-lm` a megadott rendszer linkelési szokása, nem ISO C fordítási követelmény.

10.3.6 Feladatok

10.3.6.1 Magyarázó feladat: eltűnő linkerhiba

Egy header `inline int twice(int x) { return x * 2; }` definíciót tartalmaz, külső definíció sehol nincs. A hívások kis, túlcsordulást nem okozó értékeket adnak át. Miért nem bizonyítja a program helyességét, ha `-O2` mellett linkelhető, `-O0` mellett viszont nem?

10.3.6.2 Kódolási feladat: unsigned támogatás

Bővítsd a `counter_inc` interfészt `unsigned int` támogatással. A növelés `UINT_MAX` értéknél telítődjön. Ellenőrizd a nulla, a maximum előtti érték és a maximum esetét.

10.3.6.3 Diagnosztikai feladat: megismételt mellékhatás

Az alábbi hibás tervezésű makrónál induljon `i` értéke 3-ról:

```
/* Diagnosztikai példa: mellékhatásos argumentummal kerülendő. */
#define BAD_INC(x) ((x) == INT_MAX ? INT_MAX : (x) + 1)

/* Függvénytörzs részlete, <limits.h> szükséges. */
int i = 3;
int result = BAD_INC(i++);
```

Mennyi lesz `i` és `result`? UB történik-e? Mi változik a helyes `counter_inc(i++)` használatával?

10.3.7 Megoldások

10.3.7.1 A külső definíció hiánya

A puszta inline definíció C17-ben nem biztosít külső definíciót. Az optimalizáló eltüntetheti a gépi hívást, így elfedheti a linkelési tünetet, de nem javítja meg a programszerződést.

Headerhelper esetén javítás a `static inline`. Egyetlen közös külső függvényhez headerbeli prototípus és `.c` fájlbeli definíció használható. Ellenőrzési feltétel: mindkét optimalizálási profilban szabályos definíciós szerkezet és sikeres build.

10.3.7.2 Az unsigned ág

A headerbe kerülő C17-részlet:

```
static inline unsigned int counter_inc_uint(unsigned int value)
{
    return value == UINT_MAX ? UINT_MAX : value + 1u;
}
```

A generikus listához hozzáadandó:

```
/* Új asszociáció; a szomszédos elemeket vessző választja el. */
unsigned int: counter_inc_uint
```

Az ellenőrzések a `main` törzsében:

```
assert(counter_inc(0u) == 1u);
assert(counter_inc(UINT_MAX - 1u) == UINT_MAX);
assert(counter_inc(UINT_MAX) == UINT_MAX);
```

Az unsigned körbefordulás definiált lenne, de sértené a telítődő művelet szerződését. Ezért itt is szükséges a feltétel.

10.3.7.3 A makró mellékhatása

A feltétel először `3`-at vizsgál, és `i` értékét `4`-re növeli. A hamis ág újra kiértékeli az argumentumot: az értéke `4`, majd `i` értéke `5` lesz. Az eredmény `4 + 1`, tehát `result == 5`.

Ebben a konkrét esetben nincs UB: a feltételes operátor feltételének kiértékelése megelőzi a kiválasztott ágét. A hiba a kétszeri, nem kívánt mellékhatás.

A helyes változatnál `i == 4` és `result == 4`. Ezek külön-külön `assert` állításokkal ellenőrizhetők.

10.3.8 Összefoglaló

A `static inline` típusos, belső kapcsolású headerhelper készítésére alkalmas; nem kényszeríti ki a gépi beillesztést. A külső kapcsolású C inline-szabályokat külön kell kezelni, különösen GNU89-kód átvételekor. A `_Generic` kiértékeletlen vezérlő kifejezéssel választ típust: függvényjelölők kiválasztásával egyszeri argumentumkiértékelésű interfész építhető. A `<tgmath.h>` ugyanezen típusgenerikus API-gondolat szabványos könyvtári példája.

10.4 Opaque handle és stabil C könyvtári interfész

Egy könyvtári interfész akkor is szerződés, ha a fejlécben mindössze néhány függvénydeklaráció látható. Meg kell mondania, ki birtokolja az állapotot, meddig érvényesek a kölcsönzött pointerek, mi változhat hiba esetén, és milyen párhuzamos használat megengedett. Az **opaque handle**, vagyis rejtett reprezentációjú objektumra mutató fogantyú az állapot elrendezését elrejti, de ezeket a kötelezettségeket nem helyettesíti.

A paraméterátadásnál látott értékmásolás, a tulajdonlási szerződés és a könyvtári linkelés most egy kis kvótakönyvtárban kapcsolódik össze. A könyvtár egy számlálót kezel felső korláttal, és sikeres módosításkor opcionális callbacket hív.

10.4.1 Rejtett reprezentáció, látható kötelezettségek

ISO C17: az alábbi elődeklarálás nem teljes típust (*incomplete type*) vezet be:

```
/* C17 deklarációrészlet. */
typedef struct quota quota;
```

A kliens deklarálhat `quota *` pointert, de nem képezhet `sizeof(quota)` kifejezést, nem érheti el a tagokat, és nem hozhat létre ilyen típusú objektumot érték szerint. A teljes struktúradefiníció kizárólag az implementációban szerepel.

A pointer másolható, de ettől nem keletkezik új tulajdonos. Ha két pointer ugyanarra az objektumra mutat, az egyik révén végrehajtott megsemmisítés mindkettő hozzáférését érvényteleníti.

Az interfészünk szerződése:

Elem	Tulajdonos és élettartam	Érvénytelenítés, hibakimenet
`quota *` objektum	Sikeres létrehozás után a kliens felel érte	Pontosan egyszeri `quota_destroy`; minden alias érvénytelenné válik
Belső tároló	A könyvtár foglalja és szabadítja fel	A kliens nem használhat rajta közvetlenül `free`-t
Opciószerkezet	A hívóé; csak a `create` idejére kölcsönzött	A hívás után megszűnhet
Callback és `userdata`	Nem kerül tulajdonba; a kliens biztosítja érvényességüket a megsemmisítésig	A könyvtár nem szabadítja fel a contextet
Hibaüzenet	Könyvtári, csak olvasható statikus tároló	Nem felszabadítható; a könyvtár betöltött élettartamáig használható

A `create` hibakor null pointert ír az érvényes kimeneti helyre. Ez a hely nem tartalmazhat elveszíthető, élő tulajdont. A sikertelen `add` változatlanul hagyja az állapotot, és nem hív callbacket.

Szálbiztonsági szerződés: külön handle-ök párhuzamosan használhatók, ha callbackjeik és közös contextadataik is ezt megengedik. Ugyanazon handle minden műveletét, a megsemmisítést is beleértve, a hívó sorosítja. Nincs belső zárolás.

A callback szinkron módon, a hívó szálán fut, már módosított állapot mellett. Nem léphet vissza ugyanazon handle API-jába, nem semmisítheti meg azt, és rendesen vissza kell térnie. C++ callbackből kivétel sem léphet át ezen a határon.

10.4.2 A minimális publikus fejléc

A következő `quota.h` C17-ből és C++20-ból is használható. Az állapot layoutja rejtett; az opcióstruktúra layoutja viszont szándékosan publikus, ezért kompatibilitási kötelezettség.

```
#ifndef QUOTA_H
#define QUOTA_H

#include <stddef.h>

#ifndef QUOTA_API
#define QUOTA_API
#endif

#ifdef __cplusplus
extern "C" {
#endif

#define QUOTA_API_VERSION 1u

#define QUOTA_OK        0
#define QUOTA_EINVAL    1
#define QUOTA_EVERSION  2
#define QUOTA_ENOMEM    3
#define QUOTA_ELIMIT    4

typedef struct quota quota;

typedef void (*quota_notify)(void *userdata, unsigned long total);

typedef struct quota_options {
    size_t struct_size;
    unsigned api_version;
    unsigned long limit;
    quota_notify notify;
    void *userdata;
} quota_options;

QUOTA_API int quota_create(const quota_options *options, quota **out);
QUOTA_API int quota_add(quota *q, unsigned long amount);
QUOTA_API void quota_destroy(quota *q);
QUOTA_API const char *quota_status_text(int status);

#ifdef __cplusplus
}
#endif

#endif
```

A fejléc csak a `size_t` miatt függ a `<stddef.h>` headertől. Nem szivárogtat ki allokátort, zárolási típust vagy Linux-specifikus struktúrát. A prefix csökkenti a névütközés esélyét.

A `QUOTA_API` a dinamikus szimbólumláthatóság konfigurálási pontja. Ez nem azonos a C nyelvi kapcsolással: egy külső kapcsolású függvényt az ELF-könyvtárból még el lehet rejteni. A kizárólag implementációs segédfüggvényeket továbbra is `static` deklarációval célszerű elkülöníteni.

Az `extern "C"` őr a későbbi C++ fogyasztás alapja, nem általános ABI-függetlenségi garancia. Részleteit a két nyelvű headerek tárgyalják.

10.4.3 Implementáció és allocatorhatár

A teljes `quota.c` ISO C17-et használ; az exportláthatóság fordítófüggő beállítása külön, a build-parancsban jelenik meg.

```
#include "quota.h"

#include <limits.h>
#include <stdlib.h>

struct quota {
    unsigned long limit;
    unsigned long used;
    quota_notify notify;
    void *userdata;
};

int quota_create(const quota_options *options, quota **out)
{
    if (out == NULL) {
        return QUOTA_EINVAL;
    }
    *out = NULL;

    quota_options cfg = {
        sizeof(quota_options),
        QUOTA_API_VERSION,
        ULONG_MAX,
        NULL,
        NULL
    };

    if (options != NULL) {
        if (options->struct_size < sizeof *options) {
            return QUOTA_EINVAL;
        }
        if (options->api_version != QUOTA_API_VERSION) {
            return QUOTA_EVERSION;
        }
        cfg = *options;
    }
```

```
    quota *q = malloc(sizeof *q);
    if (q == NULL) {
        return QUOTA_ENOMEM;
    }

    q->limit = cfg.limit;
    q->used = 0;
    q->notify = cfg.notify;
    q->userdata = cfg.userdata;

    *out = q;
    return QUOTA_OK;
}

int quota_add(quota *q, unsigned long amount)
{
    if (q == NULL) {
        return QUOTA_EINVAL;
    }
    if (amount > q->limit - q->used) {
        return QUOTA_ELIMIT;
    }

    q->used += amount;
    if (q->notify != NULL) {
        q->notify(q->userdata, q->used);
    }
    return QUOTA_OK;
}

void quota_destroy(quota *q)
{
    free(q);
}

const char *quota_status_text(int status)
{
    switch (status) {
    case QUOTA_OK:       return "success";
    case QUOTA_EINVAL:   return "invalid argument";
    case QUOTA_EVERSION: return "unsupported API version";
    case QUOTA_ENOMEM:   return "allocation failed";
    case QUOTA_ELIMIT:   return "quota exceeded";
    default:             return "unknown status";
    }
}
```

A null `options` alapértelmezett konfigurációt jelent: `ULONG_MAX` korlátot és callback nélküli működést. Az `unsigned long` itt nyelvi típus, nem hordozhatóan rögzített szélességű protokollmező. A null handle megsemmisítése megengedett.

A callback minden sikeres `add` után lefut, nulla növekménynél is. A könyvtári műveletrész konstans számú műveletet végez, allokáció nélkül; a callback költségére nincs ilyen vállalás.

10.4.3.1 Az állapotmegőrzés indoklása

Bizonyításvázlat. Legyen u az aktuális érték, L a korlát, a a növekmény. Az invariáns:

$$0 \le u \le L \le \text{ULONG_MAX}.$$

Ezért $L - u$ ábrázolható. Ha $a > L - u$, a függvény írás nélkül visszatér. Egyébként:

$$a \le L - u \quad \Longrightarrow \quad u + a \le L,$$

tehát az összeadás sem csordul túl, és az invariáns megmarad. A közvetlen `u + a > L` ellenőrzés hibás lenne, mert az unsigned összeadás már az összehasonlítás előtt körbefordulhatna.

10.4.3.2 Mit nem ellenőriz a könyvtár?

A nullellenőrzés nem pointervalidálás. A nem null `out` írható, megfelelő típusú objektumra mutasson, és ne fedje át az opciókat. A nem null `options` valódi, olvasható opcióobjektumot jelöljön. Az élő handle helyett átadott felszabadított pointer szerződéssértés, nem garantáltan visszaadható hibakód.

A **hívó–hívott allokátorhatár** (*caller/callee allocator boundary*) szabálya egyszerű: aki foglal, az biztosítja a hozzá tartozó felszabadító műveletet. A kliens ezért akkor sem hívhat `free(q)`-t, ha az aktuális implementáció történetesen `malloc`-ot használ. Később pool, más runtime vagy kiegészítő belső erőforrás is megjelenhet.

A hibaüzenet nem handle-ből kölcsönzött adat: a handle megsemmisítése nem érvényteleníti. A szöveg tartalmára nem építünk programlogikát; erre a numerikus státusz szolgál.

10.4.4 Méretmező, verziózás és bővíthetőség

A `struct_size` C bájtban, vagyis a `sizeof` egységében értendő. Nem biztonsági ellenőrzés: a könyvtár nem tudja megállapítani, hogy a hívó igazat állított-e a rendelkezésre álló objektum méretéről.

A jelenlegi implementáció az első verzió teljes struktúráját megköveteli, a nagyobb méretet elfogadja, de az ismeretlen folytatást nem olvassa. Az `api_version` az inkompatibilis szerződésváltást jelöli.

Egy későbbi opcionális mező hozzáadásakor három szabály szükséges:

1. A korábbi mezők típusa, sorrendje, offsetje és jelentése változatlan marad.
2. Az új könyvtár továbbra is elfogadja a régi méretet; nem követeli automatikusan az új `sizeof` értéket.
3. A hiányzó új mezőhöz dokumentált alapérték tartozik.

Ha egy mező offsetje o, mérete w, a közölt struktúraméret pedig S, akkor az olvashatóság méretfeltétele:

$$S \ge o \quad \text{és} \quad S - o \ge w.$$

A kivonásos alak elkerüli az ellenőrzésben az $o+w$ túlcsordulását. Az ABI-tervben azt is biztosítani kell, hogy új mező ne kerüljön a régi struktúra záró paddingjébe: különben a régi méret alapján jelen levőnek látszhatna. Ezt minden támogatott target ABI-ra külön kell kezelni.

Az ismeretlen mezők figyelmen kívül hagyása csak opcionális viselkedésnél elfogadható. Egy kötelező biztonsági beállítást nem szabad csendben elhagyni: ahhoz képességlekérdezés, új függvény vagy elutasítható verzióváltás kell.

A forráskompatibilitás újrafordíthatóságot, a bináris kompatibilitás meglévő gépi kód használhatóságát, a viselkedési kompatibilitás pedig a korábbi vállalások megőrzését jelenti. Az opaque handle csak a belső layoutot választja le ezekről; az exportált függvények és a publikus opciószerkezet továbbra is ABI-felület.

10.4.5 C++ fogyasztó és fordítási profil

A következő teljes `client.cpp` C++20-as program. A `probe` objektum tulajdonosa a `main`; a könyvtár csak a címét őrzi, és a handle megszűnése előtt minden callback befejeződik.

```
#include "quota.h"

#include <cstdio>

struct probe {
    unsigned count;
    unsigned long last;
};

extern "C" void record_total(void *userdata, unsigned long total)
{
    auto *p = static_cast<probe *>(userdata);
    ++p->count;
    p->last = total;
}

int main()
{
    probe events{};
    quota_options options{};
    options.struct_size = sizeof options;
    options.api_version = QUOTA_API_VERSION;
    options.limit = 10;
    options.notify = record_total;
    options.userdata = &events;

    quota *q = nullptr;
    int rc = quota_create(&options, &q);
    if (rc != QUOTA_OK) {
        std::puts(quota_status_text(rc));
        return 1;
    }

    int first = quota_add(q, 9);
```

```
    int second = quota_add(q, 2);

    quota_destroy(q);
    q = nullptr;

    std::puts(quota_status_text(second));
    return first == QUOTA_OK && second == QUOTA_ELIMIT
        && events.count == 1 && events.last == 9 ? 0 : 1;
}
```

Kijelölt, itt nem ellenőrzött laborprofil: Linux x86-64 System V, LP64; GCC/G++ 14.2.0, a `lab-next` profil (profiltábla), libstdc++ 14.2.0, GNU binutils 2.43. A parancsok ezekre beállított eszközneveket feltételeznek. A példát itt nem fordítottuk és nem futtattuk.

GCC-kiterjesztés, Linux/ELF: a rejtett alaplláthatóság mellett a `QUOTA_API` exportálja a publikus függvényeket.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -fPIC -fvisibility=hidden \
  '-DQUOTA_API=__attribute__((visibility("default")))' \
  -c quota.c -o quota.o

gcc -shared quota.o -o libquota.so

g++ -std=c++20 -Wall -Wextra -Wpedantic \
  client.cpp -L. -lquota -Wl,-rpath,'$ORIGIN' -o quota-client
```

A C implementációt C fordítóval készítjük el; a C++ fordító csak a klienst dolgozza fel. A várt működés egy sikeres növelés, egy állapotváltozás nélküli elutasítás és egyetlen callback. Ez elvárt eredmény, nem mérési jegyzőkönyv.

10.4.6 Gyakori hibák

A `quota_destroy(q)` nem nullázza a hívó változóját: a pointerparaméter is érték szerint adódik át. A későbbi `q = NULL` vagy `q = nullptr` hasznos helyi védelem, de más aliasokat nem javít meg.

A `userdata` megőrzése nem élettartam-hosszabbítás. Ha a kliens egy már visszatért függvény lokális objektumának címét adta meg, a későbbi callback érvénytelen hozzáférést végezhet.

Ugyancsak hibás az „opaque, tehát szálbiztos” következtetés. A reprezentáció elrejtése hozzáférési korlát, nem szinkronizáció.

10.4.7 Feladatok

1. **Magyarázó:** miért szűnhet meg az opcióstruktúra közvetlenül a sikeres létrehozás után, miközben a `userdata` által jelölt objektum nem? Mi történik a handle másolatával megsemmisítéskor?
2. **Kódolási:** egészítsd ki a C++ klienst úgy, hogy az elutasított növelés után még egy egységet hozzáadjon. Ellenőrizd, hogy a végső callbackérték tíz, a hívásszám kettő.
3. **Diagnosztikai:** egy új könyvtármező hozzáadása után a fejlesztő változatlanul hagyja a `struct_size < sizeof *options` ellenőrzést. Miért hibás ez régi kliensekkel? Miért nem elegendő pusztán eltávolítani az ellenőrzést?

10.4.8 Megoldások

10.4.8.1 Az élettartamok szétválasztása

A `create` lemásolja az opciók értékeit, ezért az opcióobjektumot később nem olvassa. A `userdata` pointerének másolása viszont nem másolja le a mögöttes objektumot. Annak a callbackek teljes ideje alatt élnie kell.

Megsemmisítéskor minden handle-alias érvénytelenné válik. Az ellenőrzési feltétel: ne maradjon később használható hozzáférési út a megszűnt objektumhoz.

10.4.8.2 Állapotmegőrzési próba

A kliensben az első két `add` után, a megsemmisítés és az eredeti visszatérés helyére kerüljön az alábbi **C++20-részlet**:

```
int third = quota_add(q, 1);
bool ok = first == QUOTA_OK && second == QUOTA_ELIMIT
    && third == QUOTA_OK
    && events.count == 2 && events.last == 10;

quota_destroy(q);
q = nullptr;
return ok ? 0 : 1;
```

A korábbi buildparancs használható. A vizsgált feltétel nem pusztán a hibakód: az elutasított művelet nem fogyaszthat kvótát és nem generálhat callbacket.

10.4.8.3 Visszafelé kompatibilis méretkezelés

Az új `sizeof` nagyobb lehet a régi kliens által átadott méretnél, így az ellenőrzés kompatibilis hívást utasítana el. Eltávolítása viszont a hiányzó mező olvasásához vezethet.

A javítás a régi minimális méret megőrzése, mezőnkénti méretellenőrzés és alapértékadás. Ellenőrzési feltétel, hogy az új könyvtár régi klienssel is működjön, és semmilyen hiányzó mezőt ne olvasson.

10.4.9 Összefoglaló

A stabil C interfész rejtett állapotból és látható szerződésből áll. A `create/destroy` pár rögzíti az ownership- és allokátorhatárt; a callback, a context és a hibaadat külön élettartam-szabályt kap. A méretmező támogatja a bővítést, de csak megőrzött ABI-prefixszel, alapértékekkel és tudatos verziópolitikával. Az `extern "C"` előkészíti a C++ fogyasztást; a helyességet továbbra is az explicit szerződés biztosítja.

10.5 C API-szerződések ellenőrzése

Egy C API helyessége nem merül ki abban, hogy néhány szokásos bemenetre megfelelő eredményt ad. A kliensnek azt is tudnia kell, mi történik üres adattal, elégtelen kapacitással vagy allokációs hibával; meddig érvényesek a kapott pointerek; és használható-e egy új könyvtárkiadás a korábban lefordított programmal.

Az ellenőrzés alapja ezért a **szerződés**, nem pusztán az implementáció. Az opaque handle és stabil interfész tulajdonlási szabályait tesztelhető állításokká alakítjuk. Az allokációs hiba-injektálás módszereire és a kétarchitektúrás layout-labor kompatibilitási megfigyeléseire építünk, részletes megismétlésük nélkül.

10.5.1 Mit jelent a szerződés ellenőrzése?

Az API-szerződés négy összetevője:

- **Előfeltétel:** amit a hívónak biztosítania kell, például egy nem null pointer mögötti objektum élettartamát.
- **Utófeltétel:** amit a függvény garantál, ha az előfeltételek teljesültek.
- **Hibaszerződés:** a dokumentáltan kezelt hibás bemenetek visszatérési értéke és az output állapota.
- **Érvényességi szerződés:** tulajdonlás, kölcsönzés és érvénytelenítés szabályai.

Fontos különbség van a **kezelt hibás bemenet** és az **előfeltétel megsértése** között. Egy dokumentáltan elutasított null pointer tesztelhető hibakimenet. Egy felszabadított objektumra mutató pointer nem feltétlenül az.

ISO C17: egy általános pointerértékből nem állapítható meg hordozhatóan, hogy mögötte megfelelő típusú, élő, elegendő méretű objektum található-e. A szerződésellenőrzés ezért nem jelent tetszőleges címek biztonságos kipróbálását.

Opaque handle esetén például:

Bemenet	Lehetséges dokumentált szerződés	Ellenőrzés
Élő handle	A művelet végrehajtható	Normál teszt
Null handle	Argumentumhiba; destruktornál akár megengedett üres művelet	Külön teszt mindkét műveletre
Már megsemmisített handle	Előfeltétel-sértés	Elkülönített diagnosztikai teszt, garantált hibakód nélkül
Más objektum címéből képzett handle	Előfeltétel-sértés	Nem része a normál funkcionális tesztkészletnek

A handle belsejében tárolt „magic number” nem oldja meg az utolsó két esetet: annak kiolvasásához előbb már hozzá kellene férni az objektumhoz.

10.5.2 Kidolgozott példa: átfedést engedő másolási API

Egy kis, allokációmentes másolási magon egyszerre ellenőrizhető a null-policy, a kapacitás, az aliasolás és a hibakimenet.

A szerződés:

- A méretek C bájtban értendők; egy C bájt nem szükségképpen nyolc bit.
- `src == NULL` csak `n == 0` mellett megengedett.
- `dst == NULL` csak `capacity == 0` mellett megengedett.
- A nem null `src` legalább `n` olvasható, a nem null `dst` legalább `capacity` írható C bájtot jelöl.
- A két adatterület átfedhet.
- `written` egy írható `size_t` objektumot jelöl, amely nem fedhet át egyik adatterülettel sem. A null `written` kezelt argumentumhiba.
- Siker esetén az első `n` C bájt átmásolódik, és `*written == n`.
- Hiba esetén sem a célterület, sem az érvényes `written` objektum nem változik.
- Argumentumhiba megelőzi a kapacitáshibát.

Objektum vagy hozzáférés	Tulajdonos	Érvényesség és érvénytelenítés
Forrás- és céltároló	Hívó	A hívás alatt élő, megfelelően hozzáférhető tároló szükséges
`src`, `dst`	Kölcsönzött hozzáférés	A függvény nem őrzi meg; felszabadítás vagy tárolócsere érvénytelenítheti
`written` objektuma	Hívó	A teljes hívás alatt élő, írható, megfelelően igazított objektum
Hibakimenet	Visszaadott státusz	A már létező outputokat változatlanul hagyja

A hívás közben más végrehajtási szál nem módosíthatja a forrást, és nem végezhet ütköző hozzáférést a célhoz vagy a számlálóhoz.

10.5.2.1 Header és implementáció

A következő két fájl ISO C17-et igényel, fordítóbővítés és konkrét ABI-feltételezés nélkül.

`copy_api.h`:

```
#ifndef COPY_API_H
#define COPY_API_H

#include <stddef.h>

enum copy_status {
    COPY_OK = 0,
    COPY_ARGUMENT = 1,
    COPY_CAPACITY = 2
};

enum copy_status copy_bytes(
```

```
    void *dst, size_t capacity,
    const void *src, size_t n,
    size_t *written);

#endif
```

copy_api.c:

```
#include "copy_api.h"

#include <string.h>

enum copy_status copy_bytes(
    void *dst, size_t capacity,
    const void *src, size_t n,
    size_t *written)
{
    if (written == NULL ||
        (dst == NULL && capacity != 0) ||
        (src == NULL && n != 0)) {
        return COPY_ARGUMENT;
    }

    if (n > capacity) {
        return COPY_CAPACITY;
    }

    if (n != 0) {
        memmove(dst, src, n);
    }

    *written = n;
    return COPY_OK;
}
```

ISO C17: a `memmove` megengedi az átfedést, a `memcpy` nem. Az `n != 0` ág miatt az üres esetben nem továbbítunk null pointert a könyvtári másolófüggvénynek.

Bizonyításvázlat. Minden hibával visszatérő ág megelőzi az első írást, ezért hibánál az output változatlan. Sikeres, nem üres másoláskor az ellenőrzésből $n \leq capacity$ következik; a tárolók érvényességét az előfeltételek biztosítják. Az átfedést a `memmove` kezeli. Végül a számláló külön objektumba íródik, így nem rongálja a másolt adatot.

10.5.2.2 Teszteljük az állapotot is

A `copy_test.c` önálló tesztprogram. A feladatok később kiegészítik határértékekkel.

```
#include "copy_api.h"

#include <assert.h>
#include <string.h>

int main(void)
```

```
{
    unsigned char data[] = {10, 20, 30, 40};
    const unsigned char initial[] = {10, 20, 30, 40};
    const unsigned char shifted[] = {10, 10, 20, 30};
    size_t written = 91;

    assert(copy_bytes(data, 2, data, 3, &written)
           == COPY_CAPACITY);
    assert(written == 91);
    assert(memcmp(data, initial, sizeof data) == 0);

    assert(copy_bytes(data + 1, 3, data, 3, &written)
           == COPY_OK);
    assert(written == 3);
    assert(memcmp(data, shifted, sizeof data) == 0);

    assert(copy_bytes(NULL, 0, NULL, 0, &written)
           == COPY_OK);
    assert(written == 0);

    written = 91;
    assert(copy_bytes(data, sizeof data, NULL, 1, &written)
           == COPY_ARGUMENT);
    assert(written == 91);
    assert(memcmp(data, shifted, sizeof data) == 0);

    return 0;
}
```

Az ellenőrzendő eredmény a sikeres, assertion-hiba nélküli befejezés; ez nem közölt futtatási eredmény. A tesztet ne fordítsuk `NDEBUG` definícióval, mert akkor az `assert` kifejezései sem értékelődnek ki.

10.5.3 Határértékek és allokációs hibák

Rögzített c kapacitásnál, ha $c > 0$ és $c + 1$ ábrázolható, a jellegzetes hosszértékek:

$$n \in \{0,\ 1,\ c-1,\ c,\ c+1\}.$$

A tesztforrásnak a legnagyobb vizsgált `n` értékhez is elegendő tárolót kell biztosítania. Egy apró tömb mellé megadott óriási hossz nem feltétlenül határértékteszt: sértheti a szerződést.

Az output megváltozását őrértékkel, a céltartalmat előzetes másolattal ellenőrizzük. Siker esetén a cél `n` utáni része is maradjon változatlan.

A másolási mag nem allokál. Egy ráépülő tulajdonló konstruktor viszont allokálhat handle-t, majd adattárolót. Tegyük fel, hogy nem üres bemenetre pontosan ezt a két allokációt végzi, és hibánál a kezdetben null kimeneti handle változatlan marad.

Injektált hiba	Sikeres allokációk	Visszatérésig felszabadított blokkok	Kimeneti handle
Első allokáció	0	0	Null
Második allokáció	1	1	Null
Nincs hiba	2	0	Élő, tulajdonolt

Siker után a destruktornak mindkét blokkot fel kell szabadítania. A teszt tehát ne csak a hibakódot, hanem a ki nem szabadított blokkok számát és a kimeneti handle állapotát is ellenőrizze. Az injektálás számlálóit minden teszteset előtt alaphelyzetbe kell állítani.

10.5.4 Header és build ellenőrzése

A nyilvános header ne függjön attól, hogy a kliens előtte milyen más headert töltött be. Külön fordítási egységben, első includeként és ismételten is ellenőrizzük.

`header_probe.c`:

```
#include "copy_api.h"
#include "copy_api.h"
```

Választott, itt nem futtatott laborprofil: Linux x86-64, System V ABI, a `lab-next` profil (profiltábla). Az alábbi parancsok az adott verziókra mutató fordítóneveket feltételezik:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror \
    -fsyntax-only header_probe.c

clang -std=c17 -Wall -Wextra -Wpedantic -Werror \
    -fsyntax-only header_probe.c

gcc -std=c17 -Wall -Wextra -Wpedantic -Werror \
    copy_api.c copy_test.c -o copy-test-gcc

clang -std=c17 -Wall -Wextra -Wpedantic -Werror \
    copy_api.c copy_test.c -o copy-test-clang
```

A header-próba deklarációs önállóságot vizsgál, nem implementációs helyességet vagy bináris kompatibilitást.

10.5.5 Forrás- és bináris kompatibilitás

Forráskompatibilitás esetén a korábbi kliensforrás módosítás nélkül használható az új headerrel és könyvtárral. A puszta sikeres fordítás még nem bizonyítja a viselkedési szerződés megőrzését.

Bináris kompatibilitás esetén a korábban lefordított kliens újrafordítás nélkül használható az új könyvtárral, a rögzített platform és ABI feltételei között.

Változtatás	Forrásszintű következmény	Bináris kockázat
Új, külön nevű függvény	Rendszerint kompatibilis	A régi exportok megőrzésével rendszerint kompatibilis

Változtatás	Forrásszintű következmény	Bináris kockázat
Privát opaque reprezentáció módosítása	Nem látható a kliensnek	A külső szerződés megtartásával kompatibilis lehet
Nyilvános struktúra bővítése	Egyes kliensek tovább fordulnak	Méret és tageltolás változhat
Függvény szignatúrájának módosítása	Diagnosztika nélkül is átcsúszhat	A változatlan szimbólumnév nem védelem
Hibakimenet szabályának módosítása	Többnyire tovább fordul	Layoutváltozás nélkül is viselkedési törés

Linux/ELF eszközvizsgálat: GNU binutils 2.42 `nm` eszközével a már elkészített régi és új könyvtár exportnevei összevethetők:

```
nm -D --defined-only old/libcopy.so |
    awk '{print $3}' | LC_ALL=C sort > old.symbols

nm -D --defined-only new/libcopy.so |
    awk '{print $3}' | LC_ALL=C sort > new.symbols

diff -u old.symbols new.symbols
```

Ez csak névlista-diff. Nem ellenőrzi a paramétertípusokat, a struktúralayoutot vagy az ownership-szabályokat. A `readelf --dyn-syms --wide` kimenetéből további szimbólumtulajdonságok vizsgálhatók.

A kompatibilitási tesztmátrix két külön esete:

- régi kliensforrás újrafordítása az új headerrel;
- a régi headerrel készült kliensbináris futtatása az új könyvtárral.

Az első nem helyettesíti a másodikat.

10.5.6 Fuzzolható, külső mellékhatástól mentes mag

A „mellékhatásmentes mag” itt nem azt jelenti, hogy nincs outputírás. A függvénynek nincs fájl-, hálózati, környezeti vagy rejtett globális mellékhatása; hatása a dokumentált kimenetre korlátozódik.

A fuzzer bájtokból készítsen **érvényes tárolókat és ellenőrzött hosszakat**, ne nyers pointerértékeket. Tulajdonosa a tesztharness; a mag csak a hívás idejére kölcsönzi a tárolókat. Azok felszabadítása a visszatérés után történhet.

A másolási API hasznos tulajdonságai:

- siker esetén a célprefix az eredeti forrástartalommal egyezik;
- a cél többi része változatlan;
- hiba esetén minden output változatlan;
- azonos kezdőállapotból azonos eredmény születik;
- átfedésnél az elvárt tartalmat a hívás előtti forráspillanatkép adja.

A sanitizer nem helyettesíti ezeket: memóriabiztonsági hiba nélkül is visszaadható rossz státusz vagy hibás tartalom.

10.5.7 Gyakori hibák

A leggyakoribb tesztelési hiány a hibakód ellenőrzése az output vizsgálata nélkül. Ugyanilyen veszélyes az azonos input és output cím tesztelése, miközben a részleges átfedés kimarad.

További hibaforrás, ha minden „érvénytelen pointerhez” hibakódot várunk, vagy a változatlan szimbólumlistát teljes ABI-bizonyítéknak tekintjük. A javítás mindkét esetben a vizsgált állítás és előfeltételeinek pontosítása.

10.5.8 Feladatok

10.5.8.1 Null és érvényesség

Érvényes `written` mellett mi történik, ha mindkét adatpointer null és mindkét méret nulla? Mi változik `capacity == 1` esetén? Ugyanígy kezelhető-e egy felszabadított célterület?

10.5.8.2 Határértékteszt

Egészítsd ki a tesztprogramot öt C bájtos forrással, négy C bájtos céllal és a $0 \leq n \leq 5$ hosszokkal. Ellenőrizd a státuszt, a számlálót és a teljes céltartalmat.

10.5.8.3 Allokációs diagnózis

A konstruktor második allokációjának hibája után null handle marad, de egy blokk továbbra is foglalt. Melyik szerződés sérült?

10.5.8.4 Kompatibilitási diagnózis

Egy érték szerint átadott nyilvános struktúra új mezőt kapott. Az exportlista változatlan, az újrafordított tesztek sikeresek. Mi hiányzik az ellenőrzésből?

10.5.9 Megoldások

10.5.9.1 Null és érvényesség

Az első hívás sikeres, és nullázza a számlálót. A második `COPY_ARGUMENT` eredményt ad, a számláló változatlan. A felszabadított tároló nem kezelt argumentumhiba: a szükséges élettartam hiányzik, ezért nincs garantált hibakimenet.

10.5.9.2 Határértékteszt

Az alábbi részlet a meglévő `main` függvénybe illeszthető:

```
const unsigned char src[5] = {1, 2, 3, 4, 5};

for (size_t n = 0; n <= sizeof src; ++n) {
    unsigned char dst[4] = {99, 99, 99, 99};
    size_t count = 91;

    enum copy_status status =
        copy_bytes(dst, sizeof dst, src, n, &count);

    assert(status == (n <= sizeof dst
                      ? COPY_OK : COPY_CAPACITY));
    assert(count == (n <= sizeof dst ? n : 91));

    for (size_t i = 0; i < sizeof dst; ++i) {
        unsigned char expected =
            (n <= sizeof dst && i < n) ? src[i] : 99;
        assert(dst[i] == expected);
    }
}
```

A forrás minden vizsgált hosszhoz elegendő. A teljes cél ellenőrzése a sikeres másolás utáni érintetlen részt is lefedi.

10.5.9.3 Allokációs diagnózis

A kimeneti szerződés teljesült, az erőforrás-szerződés nem: a részlegesen létrehozott handle tárolója kiszivárgott. A második hibautat úgy kell javítani, hogy a korábbi allokációt felszabadítsa. Ellenőrzési feltétel: visszatéréskor nulla élő új blokk.

10.5.9.4 Kompatibilitási diagnózis

Hiányzik a régi kliensbináris vizsgálata és a publikus layout összehasonlítása. A struktúra mérete, igazítása és ABI szerinti átadása is érintett lehet. Az újrafordítás éppen elfedi, hogy a régi kliens más reprezentációra készült.

10.5.10 Összefoglaló

A jó API-teszt a szerződés határait vizsgálja: null és üres bemenetet, kapacitást, átfedést, hibautáni állapotot és tulajdonlást. A header-próba, az allokációs hibateszt, a fuzzolható mag és a kompatibilitási mátrix különböző bizonyítékot ad. Egyik sem helyettesíti a többit, és egyik sem teszi biztonságossá az előfeltételeket megsértő hívást.

11 A C standard könyvtár és hibakezelés

11.1 stdio, FILE és pufferelés

Egy szolgáltatás naplója terminálon azonnal megjelenhet, pipe-ba irányítva viszont csak később. Egy sikeres írás után pedig még a fájl lezárása is jelezhet hibát. Mindkét jelenség megértéséhez külön kell választani a C könyvtár streamjét, az operációs rendszer I/O-rétegét és a tartós tárolást.

A fejezet az API-paraméterek, az explicit tulajdonlás és cleanup, valamint a sztringek hosszának ismeretére épít. Itt azonban nem sztringeket, hanem tetszőleges adatot is kezelünk: egy nulla értékű C bájt nem jelenti a bemenet végét.

11.1.1 A `FILE*` egy állapottal rendelkező stream hozzáférése

ISO C17: a `<stdio.h>` fejléc `FILE` típusa a stream vezérléséhez szükséges információkat hordozó objektumtípus. A stream (*stream*) a program és egy bemeneti vagy kimeneti adatforrás közötti rendezett adatkapcsolat. A `FILE` belső mezőire nem építünk; a hozzáférést könyvtári függvényeken keresztül végezzük.

POSIX/Linux: a fájlhoz, terminálhoz vagy pipe-hoz kapcsolt stdio stream jellemzően file descriptorra épül. A `FILE*` azonban nem descriptor: saját pufferelést, EOF-jelzőt és hibajelzőt tart fenn. Egy könyvtári írás ezért nem feltétlenül jelent azonnali kernelhívást.

A program indulásakor három előre megnyitott stream érhető el:

Név	Rendeltetés	Szokásos tulajdonlási szerződés
`stdin`	Standard bemenet	A segédfüggvény kölcsönzi, nem zárja le
`stdout`	Standard kimenet	A segédfüggvény kölcsönzi; a program felel a végső hibaellenőrzésért
`stderr`	Standard diagnosztikai kimenet	Diagnosztikára használjuk, sikere nem garantált

Ezek nem feltétlenül terminálhoz kapcsolódnak. Az átirányítás megváltoztatja a háttérben elérhető erőforrást, és ezzel a megfigyelhető pufferelést is.

A fejezet példái bájtorientált stdio-műveleteket használnak; ugyanazon streamen nem keverik őket széles karakteres I/O-val.

11.1.2 Megnyitás, módok és lezárás

ISO C17: az `fopen(path, mode)` siker esetén `FILE*`, hiba esetén null pointer eredményt ad. A mód nem pusztán jogosultság: a fájl tartalmára gyakorolt hatást is meghatározza.

Alapmód	Művelet	Nem létező fájl	Létező fájl
`"r"`	Olvasás	Hiba	Tartalma megmarad
`"w"`	Írás	Létrehozás	Tartalma megnyitáskor elvész
`"a"`	Hozzáfűzés	Létrehozás	Az írások a mindenkori végére kerülnek
`"r+"`	Olvasás és írás	Hiba	Tartalma megmarad
`"w+"`	Olvasás és írás	Létrehozás	Tartalma megnyitáskor elvész
`"a+"`	Olvasás és hozzáfűzés	Létrehozás	Az írások a végére kerülnek

A `b` módosító bináris streamet kér: például `"rb"`, `"wb"` vagy `"r+b"`. **ISO C17 szerint** a szöveges stream külső reprezentációja átalakulhat. **POSIX/Linux alatt** a `b` nem változtatja meg a működést, de hordozható bináris kódban így is indokolt.

A `+` módú, úgynevezett update stream irányváltásának szabályai fontosak:

- Írás után olvasás előtt `fflush` vagy fájlpozicionáló művelet szükséges.
- Olvasás után írás előtt fájlpozicionáló művelet szükséges, kivéve, ha az olvasás elérte a fájl végét.

Az `fclose` kiüríti a függő kimenetet, és lezárja a kapcsolatot. Siker esetén nullát, észlelt hiba esetén `EOF` értéket ad. A hibája ugyanúgy számít, mint az írás hibája.

Erőforrás vagy tároló	Tulajdonos	Érvényességi határ
Sikeres `fopen` eredménye	A megnyitást vállaló kódrész	Az `fclose` hívásáig
Függvénynek átadott, kölcsönzött `FILE*`	A hívó	A hívó lezárása érvényteleníti
Saját, `setvbuf`-nak átadott puffertömb	A tömböt kezelő kódrész	Legalább a stream lezárásának befejezéséig éljen

Az `fclose` hívása után a stream többé nem használható, akkor sem, ha a lezárás hibát jelzett. Ugyanazzal a pointerrel nem próbáljuk újra a lezárást.

11.1.3 Elemszám, rövid művelet és EOF

ISO C17: a `fread(ptr, size, count, stream)` és `fwrite(ptr, size, count, stream)` visszatérési értéke a sikeresen feldolgozott **teljes elemek száma**, nem általában a C bájtok száma.

Ha s egy elem mérete, n a kért elemszám, akkor a kért adatmennyiség:

$$B = s \cdot n.$$

A visszaadott r elemszámra $0 \leq r \leq n$. Ha $s = 1$, akkor r közvetlenül C bájtszámként értelmezhető. A tárolónak a teljes kért tartományt le kell fednie; a méretszámítás nem csordulhat túl.

Ezért nyers adatmásoláshoz egyszerű szerződés:

```
fread(buffer, 1, sizeof buffer, input)
```

Itt `buffer` tényleges tömb legyen, ne pointer: pointeren a `sizeof` nem a mögöttes tároló kapacitását adja.

A rövid olvasás jelenthet fájlvéget vagy olvasási hibát. Ezeket külön kell vizsgálni:

- `feof(stream)` nem nulla, ha a stream EOF-jelzője be van állítva.
- `ferror(stream)` nem nulla, ha a hibajelző be van állítva.

Ezek állapotlekérdezések, nem a következő művelet jóslatai. Az utolsó rendelkezésre álló bájt sikeres beolvasása önmagában még nem feltétlenül állítja be az EOF-jelzőt. Ezért hibás a „míg nincs EOF, olvass” logika: előbb olvasunk, utána értékeljük az eredményt.

A rövid `fwrite` írási hibát jelez. Nem alkalmazunk rá automatikusan descriptor-szintű „írd ki a maradékot” ciklust. A hibajelző törlése sem javítja meg az alapul szolgáló hibát.

11.1.3.1 Miért `int` az `fgetc` eredménye?

Az `fgetc` egy karaktert ad vissza, `unsigned char` értékként `int`-re konvertálva, vagy `EOF`-ot jelez. Az `EOF` negatív `int` konstans, de nem szabad feltételezni, hogy pontosan `-1`.

A könyv szokásos, nyolcbites C bájtú laborprofiljában a karakterértékek 0–255 közöttiek, ezért az `int` megkülönbözteti őket az EOF-tól. Ha az eredményt előbb `char` változóba szűkítjük, ez az információ elveszhet: például egy érvényes bájt összekeveredhet az EOF-fal.

Az `EOF` eredmény után itt is `ferror` különíti el az olvasási hibát a rendes végállapottól.

11.1.4 Pufferelés és láthatóság

ISO C17: három pufferelési módot különböztetünk meg:

- `_IOFBF`: teljes pufferelés;
- `_IOLBF`: sorpufferelés;
- `_IONBF`: pufferelés nélküli működés.

Teljes pufferelésnél a kimenet gyűlhet a könyvtár pufferében. Sorpufferelésnél az újsor karakter is kiváltja a továbbítást; a puffer megtelése és az explicit kiürítés szintén számít. A pufferelés nélküli működés sem jelent lemezre történő tartós mentést.

Linux/glibc tipikus viselkedés: a terminálhoz kapcsolt `stdout` sorpufferelt, a fájlhoz vagy pipe-hoz kapcsolt `stdout` teljesen pufferelt. Ez magyarázza, hogy egy soronként naplózó program pipe mögött késleltetve jelenítheti meg a sorokat.

ISO C17 garancia: a `stderr` kezdetben nem teljesen pufferelt. Ez nem azonos a minden implementációra érvényes „biztosan pufferelés nélküli” állítással.

A `setvbuf` módosítja a pufferelést. A stream megnyitása után, de minden más streamművelet előtt kell meghívni; a szabvány a korábbi sikertelen `setvbuf` hívásra kivételt enged. Nulla a siker, nem nulla a sikertelenség.

Például a `setvbuf(stdout, NULL, _IOLBF, BUFSIZ)` sorpufferelést kér saját puffertömb átadása nélkül. Ezt még az első kiírás előtt kell ellenőrizni. Saját tömb esetén annak tárolója nem szűnhet meg a stream lezárása előtt.

11.1.4.1 Mit garantál az `fflush`?

ISO C17: megfelelő kimeneti streamen az `fflush` a még ki nem írt adatokat a host környezetnek továbbítja, hogy azok a fájlba kerüljenek. Siker esetén nulla, írási hibánál `EOF` az eredmény. Update streamen akkor alkalmazható így, ha a legutóbbi művelet nem bevitel volt. Az `fflush(NULL)` az erre alkalmas kimeneti streameket üríti.

Három szintet különítsünk el:

Művelet	Lényeges határ
Sikeres `fwrite`	A stdio elfogadta a teljes adatmennyiséget
Sikeres `fflush`	A függő stdio-kimenet továbbítása megtörtént
POSIX `fsync`	Az operációs rendszer fájlszinkronizálási szerződését kéri

Az `fflush` nem `fsync`. A tartósság további feltételeit a fájlrendszer-műveletekről szóló rész tárgyalja.

Az `fflush(stdin)` nem hordozható „bemenettörlés”. **ISO C17 szerint** ilyen bemeneti streamen a viselkedése nem definiált. A POSIX bizonyos bemeneti streamekre további szabályokat ad, de ebből nem következik általános terminálbemenet-eldobási módszer.

11.1.5 Kidolgozott példa: bináris másolás ellenőrzött lezárással

A program két különböző fájl között másol. **Előfeltétel:** a két útvonal nem jelölheti ugyanazt a mögöttes fájlt, hard linken keresztül sem; különben a `"wb"` megnyitás megsemmisítheti a bemenetet. A célfájl hibánál részleges maradhat: ez nem tranzakciós másoló.

Minimum: ISO C17, hosted környezet. Választott laborprofil: Linux, x86-64 System V ABI, LP64, a `lab-next` profil (profiltábla). A példa itt nem fordított vagy futtatott mérési eredmény.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    FILE *input = NULL;
    FILE *output = NULL;
    unsigned char buffer[8192];
    int status = EXIT_FAILURE;

    if (argc != 3) {
        fputs("usage: copy source destination\n", stderr);
        return EXIT_FAILURE;
    }

    input = fopen(argv[1], "rb");
```

```
    if (input == NULL) {
        fputs("cannot open source\n", stderr);
        goto cleanup;
    }

    output = fopen(argv[2], "wb");
    if (output == NULL) {
        fputs("cannot open destination\n", stderr);
        goto cleanup;
    }

    for (;;) {
        size_t n = fread(buffer, 1, sizeof buffer, input);

        if (n != 0 && fwrite(buffer, 1, n, output) != n) {
            fputs("write failed\n", stderr);
            goto cleanup;
        }

        if (n < sizeof buffer) {
            if (ferror(input)) {
                fputs("read failed\n", stderr);
                goto cleanup;
            }
            break;
        }
    }

    status = EXIT_SUCCESS;

cleanup:
    if (output != NULL && fclose(output) == EOF) {
        fputs("destination close failed\n", stderr);
        status = EXIT_FAILURE;
    }
    if (input != NULL && fclose(input) == EOF) {
        fputs("source close failed\n", stderr);
        status = EXIT_FAILURE;
    }
    return status;
}
```

Fordítás a megfelelő verziójú driverekkel:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 copy.c -o copy-gcc
clang -std=c17 -Wall -Wextra -Wpedantic -O2 copy.c -o copy-clang
```

A `main` mindkét stream tulajdonosa. A könyvtár a `buffer` tömböt csak az adott művelet alatt használja; a tömb a teljes másolás során él. Az `fclose` minden megszerzett streamre pontosan egyszer fut le.

Helyességi bizonyításvázlat: minden teljes ciklusléps a beolvasott n C bájtot pontosan egyszer adja át az írásnak. Rövid írásnál a program hibával leáll. Rövid olvasásnál megkülönbözteti az

olvasási hibát a fájlvégtől. A kezdetben sikeresnek minősített másolást a késleltetett lezárási hiba még sikertelenné teszi.

Külön `fflush` nem szükséges: az ellenőrzött `fclose` elvégzi a kiürítést is. A diagnosztikai `fputs` hívások szándékosan best-effort jellegűek; hibájukat nem jelentjük újabb diagnosztikával. A hibás kilépési állapot ettől még megmarad.

11.1.6 Feladatok

11.1.6.1 Magyarázó feladat

Egy program minden esemény után újsorral zárt sort ír `stdout`-ra. Terminálon gyorsan látszik a sor, pipe mögött késik. Miért? Garantálna-e tartósságot egy soronkénti `fflush`?

11.1.6.2 Kódolási feladat

Írj `copy_chars(FILE *input, FILE *output)` függvényt `fgetc` és `fputc` használatával. Adjon nullát siker, egyet hiba esetén. A streameket kölcsönözze, ne zárja le és ne ürítse explicit módon.

11.1.6.3 Diagnosztikai feladat

Mi hibás az alábbi C17-részletben? A streamek egyébként érvényesek.

```
/* Hibás tanítási példa; nem futtatási minta. */
while (!feof(input)) {
    char c = fgetc(input);
    fputc(c, output);
}
fflush(stdin);
```

11.1.7 Megoldások

11.1.7.1 A pufferelés magyarázata

A tipikus Linux/glibc terminálkimenet sorpufferelt, a pipe-kimenet teljesen pufferelt. Az újsor ezért a második esetben nem feltétlenül továbbítja a felgyűlt adatot. Az ellenőrzött `fflush(stdout)` megszünteti ezt a stdio-szintű várakozást, de tartósságot nem garantál.

Ellenőrzési feltétel: a magyarázat külön kezelje a könyvtári puffert, a kernelnek történő átadást és a tartós tárolást.

11.1.7.2 Karakterenkénti másolás

C17-függvényrészlet, a szükséges fejléccel:

```c
#include <stdio.h>

int copy_chars(FILE *input, FILE *output)
{
    int c;

    while ((c = fgetc(input)) != EOF) {
        if (fputc(c, output) == EOF) {
            return 1;
        }
    }
    return ferror(input) ? 1 : 0;
}
```

Előfeltétel a két, megfelelő irányban használható, bájtorientált vagy még orientálatlan stream, kezdetben törölt EOF- és hibajelzőkkel. A hívó a tulajdonos; a függvény alatt nem zárhatja le őket. A kimenet hibánál részleges lehet. A késleltetett íráshiba ellenőrzése a hívó későbbi `fflush` vagy `fclose` műveletének feladata.

Ellenőrzési feltétel: üres bemenetet, valamint a laborprofilban `0x00` és `0xFF` bájtokat tartalmazó bemenetet is helyesen kezeljen.

11.1.7.3 A hibák diagnózisa

A `feof` túl korai vizsgálat: a ciklus sikertelen olvasás után is megpróbálhat kiírni egy karaktert. A `char` elveszítheti az EOF és az érvényes karakter közötti különbséget. Az olvasási hiba nem feltétlenül állít EOF-jelzőt, ezért a ciklus végtelenné válhat. Az írás eredményét sem ellenőrzi. Végül az `fflush(stdin)` ISO C17 alatt nem definiált viselkedésű.

Javítás és ellenőrzési feltétel: az előző megoldás szerint előbb az `int` eredményt vizsgáljuk, ellenőrizzük az írást, majd a ciklus után az olvasási hibát. EOF-ot soha nem írunk ki adatként.

11.1.8 Rövid összefoglaló

A `FILE*` állapottal és pufferelléssel rendelkező streamhez ad hozzáférést, nem puszta descriptor. A helyes sorrend: művelet, visszatérési érték ellenőrzése, szükség esetén EOF- és hibajelző vizsgálata. A teljes másolás sikeréhez a lezárást is ellenőrizni kell. Az `fflush` a stdio-kimenet továbbításáról szól, nem tartósságról, és nem hordozható bemenettörlő művelet.

11.2 Formázás és megbízható szövegfeldolgozás

A konfigurációs érték feldolgozása nem attól sikeres, hogy a bemenet elejéből előállt egy szám. A `"443xyz"` szövegnek van számmá alakítható kezdete, mégsem érvényes portszám. Megbízható feldolgozáskor külön vizsgáljuk a bemenet nyelvtanát, a konverzió sikerét, a reprezentálhatóságot, az alkalmazási tartományt és a teljes bemenet elfogyasztását.

A variadikus függvényeknél megismert típusszerződés itt közvetlen biztonsági követelmény. A stdio és pufferelés adja az I/O hátterét, a locale és szövegkódolás pedig megmagyarázza, miért nem feltétlenül környezetfüggetlen egy szöveges konverzió. Az alábbi nyelvi és könyvtári szabályok ISO C17-re vonatkoznak; a POSIX-kiegészítéseket külön jelöljük.

11.2.1 Formázás: a formátumsztring típusszerződés

A `printf` formátumsztringje meghatározza, milyen típusú argumentumot olvas ki a függvény. A variadikus argumentumokon végrehajtott alapértelmezett promóciókat figyelembe kell venni: például a `float` argumentum `double` lesz.

Kiírandó típus	`printf`-formátum
`int`	`%d`
`unsigned int`	`%u`
`long`	`%ld`
`unsigned long long`	`%llu`
`size_t`	`%zu`
`double`, illetve argumentumként átadott `float`	`%f`, `%e` vagy `%g`
`long double`	`%Lf`, `%Le` vagy `%Lg`
`int32_t`, ha rendelkezésre áll	`"%" PRId32`

Az utolsó sorhoz a `<stdint.h>` és `<inttypes.h>` fejlécek tartoznak. Az exact-width típusok opcionálisak: például az `int32_t` elérhetőségét az `INT32_MAX` makró meglétével ellenőrizhetjük. Ha `count` ilyen típusú, a helyes alak:

```
/* ISO C17-részlet; <stdint.h>, <inttypes.h>, <stdio.h> mellett.
   Előfeltétel: int32_t elérhető, count inicializált int32_t. */
printf("count=%" PRId32 "\n", count);
```

A típusnévben szereplő szélesség nem határozza meg, hogy a típus `int`, `long` vagy más megfelelő egész típus aliasa. A hibás formátum–argumentum kapcsolat nem hordozható konverzió, hanem nem definiált viselkedést okozhat.

Külső szöveg soha ne legyen közvetlenül formátumsztring: `printf("%s", text)` használható, `printf(text)` nem megfelelő. Ettől még a sortörésekkel történő loginjektálás ellen külön védekezés kell.

11.2.1.1 A `snprintf` eredményének értelmezése

A `snprintf(dst, capacity, ...)` legfeljebb `capacity` C bájtot használ, a lezáró nullkarakterrel együtt. Pozitív kapacitásnál, érvényes célterület esetén nullterminál. A visszatérési érték:

- negatív, ha kódolási hiba történt;
- egyébként az a karakterszám, amely elegendő hely esetén létrejött volna, a lezáró nullkarakter nélkül.

Ezért a helyes ellenőrzési sorrend:

```c
/* ISO C17-részlet; dst valódi, írható tömb, value típusa long. */
int n = snprintf(dst, sizeof dst, "value=%ld", value);
if (n < 0) {
    /* Kódolási hiba. */
} else if ((size_t)n >= sizeof dst) {
    /* Csonkolás: a teljes eredmény nem fért el. */
}
```

Előbb vizsgáljuk a negatív eredményt, csak utána konvertálunk `size_t`-ra. A `sizeof dst` itt tömbméret; pointerparaméteren nem a mögöttes tároló kapacitását adná.

11.2.2 Miért nem elég a `scanf` vagy az `atoi`?

A `scanf`-család visszatérési értéke a sikeresen hozzárendelt bemeneti elemek száma. A hozzárendelést elnyomó `*` és a `%n` nem növeli ezt a számot. Illesztési hibánál az eredmény lehet nulla; az első konverzió befejezése előtti bemeneti hibánál `EOF`.

A mezőszélesség kötelező védelem a sztringeknél. A `%15s` legfeljebb 15 karaktert tárol, majd nullkaraktert ír, tehát legalább 16 elemű `char` tömb kell hozzá. A `%c` ezzel szemben nem tesz lezáró nullkaraktert.

A szélességkorlát azonban **nem bizonyítja a teljes mező elfogadását**. Egy hosszabb token maradéka a bemenetben maradhat, vagy a következő konverzióhoz kerülhet. Két sikeres hozzárendelés sem bizonyítja, hogy nincs további szemét a sorban.

Numerikus bemenetnél súlyosabb korlát, hogy ha az eredmény nem reprezentálható a célobjektumban, a C17 `scanf`-szerződése szerint nem definiált viselkedés következhet be. A return count tehát nem általános túlcsordulás-ellenőrzés. Arra is figyeljünk, hogy itt `%f` `float *`, `%lf` pedig `double *` argumentumot vár.

Az `atoi` még kevesebb ellenőrzési lehetőséget ad: nincs végpointere, és a nulla eredményből nem állapítható meg a siker. A nem reprezentálható eredmény sem kezelhető megbízhatóan. Ellenőrizetlen konfigurációhoz használjunk `strto...` függvényt.

11.2.3 Ellenőrzött numerikus konverzió

A `strtol`, `strtoull` és `strtod` a bemenet elején álló megfelelő alakot dolgozza fel. Az `endptr` az első fel nem dolgozott karakterre mutat.

A teljes elfogadás feltétele:

$$\text{siker} = \text{volt konverzió} \wedge \neg\text{konverziós tartományhiba} \wedge \text{megengedett érték} \wedge \text{megengedett lezárás}.$$

A vizsgálat lépései:

1. Alkalmazzuk a saját lexikai szabályainkat: engedünk-e szóközt, előjelet vagy alternatív számábrázolást?
2. Állítsuk `errno` értékét nullára, majd végezzük el a konverziót.
3. Az `end == text` feltétel azt jelzi, hogy nem történt konverzió.
4. Az `ERANGE` konverziós tartományhibát jelez.
5. Ellenőrizzük a lezárást és az alkalmazási értéktartományt.

Az `errno` itt csak a konverzióhoz kapcsolódó jelzés: sikeres hívás nem köteles nullára állítani. A végjel vizsgálatát nem helyettesíti.

11.2.3.1 Kidolgozott példa: portkonfigurációból normalizált log

A következő program egy `port=443` alakú argumentumot fogad. A szerződés szándékosan szigorú: csak decimális számjegyek, nincs előjel vagy szóköz, a tartomány pedig $1 \ldots 65535$. Hiba esetén a kimeneti érték változatlan.

ISO C17, teljes program — `parse_config.c`:

```
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum parse_status {
    PARSE_OK,
    PARSE_SYNTAX,
    PARSE_OVERFLOW,
    PARSE_TRAILING,
    PARSE_DOMAIN
};

static enum parse_status
parse_port(const char *text, long *out)
{
    if (text[0] < '0' || text[0] > '9') {
        return PARSE_SYNTAX;
    }

    char *end;
    errno = 0;
    long value = strtol(text, &end, 10);
    int conversion_errno = errno;

    if (end == text) {
        return PARSE_SYNTAX;
    }
    if (conversion_errno == ERANGE) {
        return PARSE_OVERFLOW;
    }
    if (*end != '\0') {
        return PARSE_TRAILING;
    }
    if (value < 1 || value > 65535) {
        return PARSE_DOMAIN;
    }

    *out = value;
    return PARSE_OK;
}
```

```
int main(int argc, char **argv)
{
    if (argc != 2 || strncmp(argv[1], "port=", 5) != 0) {
        fputs("expected: port=DECIMAL\n", stderr);
        return EXIT_FAILURE;
    }

    long port = 0;
    enum parse_status status = parse_port(argv[1] + 5, &port);
    if (status != PARSE_OK) {
        fprintf(stderr, "invalid port: status=%d\n", (int)status);
        return EXIT_FAILURE;
    }

    char record[64];
    int n = snprintf(record, sizeof record,
                     "event=config port=%ld\n", port);
    if (n < 0 || (size_t)n >= sizeof record) {
        fputs("log formatting failed\n", stderr);
        return EXIT_FAILURE;
    }

    if (fputs(record, stdout) == EOF || fflush(stdout) == EOF) {
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

Reprodukáláshoz választott laborprofil: Linux, x86-64 System V ABI, LP64, a `lab-next` profil (profiltábla). A következő parancsok fordítási útmutatók, nem itt elvégzett fordítás bizonyítékai:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -Wformat=2 \
    parse_config.c -o parse-config-gcc

clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion -Wformat=2 \
    parse_config.c -o parse-config-clang
```

A várt besorolások:

Bemenet	Eredmény
`port=443`	Siker
`port=-1`	Szintaktikai hiba
`port=443xyz`	Hátramaradt bemenet
`port=70000`	Alkalmazási tartományhiba
`port=`	Szintaktikai hiba
A `LONG_MAX` decimális alakja után még egy `0`	Konverziós túlcsordulás

Ha több hiba is fennáll, a program ellenőrzési sorrendje határozza meg az elsődleges hibát: a túlcsordulás például megelőzi a hátramaradt bemenet jelentését.

Helyességi bizonyításvázlat. A kezdőkarakter-vizsgálat kizárja az előjelet és a kezdő szóközt. A tízes alapú konverzió és a nullkarakterre mutató végpointer együtt biztosítja a teljes számjegysor

feldolgozását. A konverziós és alkalmazási tartományvizsgálat után az érték érvényes portszám. Az egyetlen kimeneti értékadás ezek után történik.

Objektum vagy hozzáférés	Tulajdonos és élettartam	Érvénytelenítés
`argv[1]`, illetve `text`	A programindítási környezet tárolója; a parser kölcsönzi	A program végéig rendelkezésre áll
`end`	Nem tulajdonló pointer ugyanabba a sztringbe	Az alapul szolgáló tároló megszűnése
`out`	A hívó írható `long` objektuma; itt `port`	A hívó objektumának élettartamvége
`record`	A `main` automatikus tömbje	Újraíráskor a tartalom, kilépéskor a tároló

A parser előfeltétele érvényes nullterminált bemenet és érvényes, írható kimeneti objektum; pointereket nem őriz meg.

11.2.3.2 Előjel nélküli és lebegőpontos értékek

A `strtoull` elfogad mínuszjelet is: a `"-1"` eredménye lehet `ULLONG_MAX` tartományhiba nélkül. Ezért a „nemnegatív kvóta” szerződést nem biztosítja önmagában az előjel nélküli céltípus. Követeljünk kezdő számjegyet, használjunk tízes alapot, majd ellenőrizzük az `ERANGE`, a végpointer és az alkalmazási felső korlát feltételeit.

A `strtod` esetén az `ERANGE` túlcsordulást vagy alulcsordulást is jelezhet. A nulla eredmény önmagában nem hiba. A függvény végtelent, NaN-t és hexadecimális lebegőpontos alakot is elfogadhat; ezek engedélyezése külön döntés.

Egy véges, $0 \ldots 3600$ másodperces időköznél például:

```
/* ISO C17-részlet: <errno.h>, <math.h>, <stdlib.h>.
   text: érvényes, kölcsönzött nullterminált sztring.
   out: a hívó írható double objektuma.
   A lexikai szabályokat a hívó már ellenőrizte.
   A hibás ágakban out változatlan. */
char *end;
errno = 0;
double value = strtod(text, &end);
int conversion_errno = errno;

if (end == text) {
    /* Nincs szám. */
} else if (conversion_errno == ERANGE) {
    /* Konverziós tartományhiba. */
} else if (*end != '\0') {
    /* Részleges feldolgozás. */
} else if (!isfinite(value) || value < 0.0 || value > 3600.0) {
    /* Alkalmazási tartományhiba. */
} else {
    *out = value;
}
```

Ez a szemantikai ellenőrzés nem tiltja a hexadecimális alakot. Ha csak decimális protokollformátum engedélyezett, annak nyelvtanát külön ellenőrizni kell.

11.2.4 Teljes sor, karakterosztály és locale

ISO C17: az `fgets` legfeljebb a megadott kapacitásnál eggyel kevesebb karaktert olvas, megtartja a beolvasott sortörést és siker esetén nullterminál. A hiányzó sortörés nem bizonyít hibát: lehet hosszú sor részlete vagy az utolsó, sortörés nélküli sor. A teljes sorhatár eldöntéséhez további olvasás kellhet. Túl hosszú rekordnál a maradékot is kezeljük, különben a következő olvasás hamis új rekordot kap.

A `NULL` eredménynél `ferror` és `feof` alapján különítsük el a hibát és a bemenet végét. Az olvasási hibával érintett buffert ne dolgozzuk fel.

POSIX: a `getline` szükség szerint allokálja vagy növeli a hívó által kezelt buffert. Glibc alatt a deklarációhoz használható a fejlécek előtt definiált `_POSIX_C_SOURCE=200809L`. Visszatérési típusa `ssize_t`; siker esetén a hossz tartalmazza a beolvasott elválasztót, de nem a lezáró nullkaraktert. `-1` jelzi a bemenet végét vagy hibát.

A buffer tulajdonosa a hívó: végül `free` szükséges, hibakimenet után is, ha történt allokáció. Új hívás felülírhatja a tartalmat, áthelyezés pedig érvényteleníti a korábbi belső pointereket. A korlátlan sorhossz memóriafogyasztási kockázat; szigorú erőforráskorláthoz korlátozott sorolvasó kell.

Beágyazott nullkarakter esetén a `strlen` nem a teljes beolvasott hosszt adja. A `getline` visszaadott hosszával ez felismerhető; az `fgets`-re épülő egyszerű sztringfeldolgozásnál a nullkarakter-mentesség legyen explicit bemeneti feltétel.

A `<ctype.h>` függvényeinek argumentuma csak `EOF` vagy `unsigned char`-ként reprezentálható érték lehet:

```
/* ISO C17-részlet, <ctype.h>; ch egy char érték. */
int whitespace = isspace((unsigned char)ch);
```

Egy negatív `char` közvetlen átadása UB lehet. A `getchar` eredményét viszont először `int`-ként, az `EOF` megőrzésével kezeljük.

A `strtod` tizedesjele a numerikus locale-tól függ. Gépi konfigurációnál rögzítsük a nyelvtant és a locale-feltételeket; könyvtári segédfüggvény ne módosítsa váratlanul a folyamat globális locale-ját. Ha csak ASCII szóköz és tabulátor megengedett, ezek explicit vizsgálata pontosabb szerződés, mint minden locale-függő whitespace elfogadása.

11.2.5 Feladatok

1. **Magyarázat:** mi a hiba a `size_t length` érték `%u` formátumú kiírásában? Miért jelent csonkolást a `snprintf` esetén a kapacitással egyenlő eredmény?
2. **Kódolás:** a portparser mintájára készíts `parse_quota` függvényt `unsigned long long` kimenettel és paraméterként kapott felső korláttal. A nulla legyen érvényes, a mínuszjel ne.
3. **Diagnosztika:** egy logfeldolgozó `%7s %u` formátummal olvas `char level[8]` és `unsigned int count` objektumba. Két hozzárendelést teljes sikernek tekint. Sorolj fel három problémát, és javasolj javítást.

11.2.6 Megoldások

11.2.6.1 Formátum és kapacitás

A `%u` `unsigned int` argumentumot vár, a `size_t` nem szükségképpen ez a típus. A helyes formátum `%zu`.

Ha a visszaadott hossz n, akkor a szükséges kapacitás:

$$C_{\text{szükséges}} = n + 1,$$

mert a nullkarakter is helyet foglal. Ezért $n = C$ már csonkolást jelent. Ellenőrzési feltétel: teljes sikerhez `n >= 0` és `(size_t)n < capacity` kell.

11.2.6.2 Kvótaparser

A korábbi programhoz illeszthető ISO C17-függvényrészlet, azonos pointer- és hibaszerződéssel:

```
static enum parse_status
parse_quota(const char *text, unsigned long long limit,
            unsigned long long *out)
{
    if (text[0] < '0' || text[0] > '9')
        return PARSE_SYNTAX;

    char *end;
    errno = 0;
    unsigned long long value = strtoull(text, &end, 10);
    int conversion_errno = errno;

    if (end == text)
        return PARSE_SYNTAX;
    if (conversion_errno == ERANGE)
        return PARSE_OVERFLOW;
    if (*end != '\0')
        return PARSE_TRAILING;
    if (value > limit)
        return PARSE_DOMAIN;

    *out = value;
    return PARSE_OK;
}
```

Ellenőrzési feltétel: `"0"` és a korlát értéke sikeres; `"-1"` szintaktikai hiba; `"12x"` részleges feldolgozás; minden hibánál a korábban beállított kimenet változatlan.

11.2.6.3 Logfeldolgozó javítása

A hét karakternél hosszabb első mező maradéka a következő konverzióhoz kerülhet. A numerikus túlcsordulás nem kezelhető biztonságosan return counttal. Két sikeres hozzárendelés után további szemét is maradhat.

Először teljes, hosszkorlátozott rekordot olvassunk. Ezután különítsük el és ellenőrizzük a mezőhatárokat, a számot pedig `strtoull` és az `UINT_MAX` korlát segítségével dolgozzuk fel. Ellenőrzési eset legyen hosszú szintnév, túl nagy szám és érvényes rekord utáni extra token.

11.2.7 Összefoglaló

A formátumsztring típusszerződés, a buffer kapacitása tárolási szerződés, a parser pedig nyelvtani és értéktartomány-szerződés. Megbízható feldolgozás csak akkor történik, ha mindhárom teljesül. A „kaptam egy számot” helyett mindig azt bizonyítsuk: **pontosan a megengedett teljes bemenetből kaptam megengedett értéket**.

11.3 Hibakód, errno és hibaadatok élettartama

A hibakezelési szerződés nem pusztán azt mondja meg, hogy egy művelet sikertelen volt. A hívónak azt is tudnia kell, **melyik művelet hibázott, milyen hibakategóriában, és meddig érvényesek a kapott hibaadatok**. Különösen fontos ez cleanup közben: a felszabadítás vagy lezárás maga is hibázhat, és közben felülírhatja az eredeti hibára utaló állapotot.

A Tulajdonlási szerződés és cleanup ágak részben kialakított egyetlen lezárási felelősséget most hibamegőrzéssel egészítjük ki. A Formázás és megbízható szövegfeldolgozás tanulságát is megtartjuk: az eredmény értékét és a művelet sikerességét csak a konkrét API szerződése alapján lehet megkülönböztetni.

11.3.1 Három különböző hibajelzési modell

A **hibajelző visszatérési érték**, vagy sentinel, az eredmények közül fenntartott érték: például a `fopen` null pointerrel jelzi a sikertelenséget. Az **explicit hibakód** közvetlenül az eredmény része. Az `errno` ezzel szemben különálló, módosítható diagnosztikai állapot.

Modell	Példa	Mit kell vizsgálni?
Sentinel	ISO C17 `fopen`	A visszaadott pointer null-e
Sentinel és további hibakód	POSIX `fopen`	Null eredmény után az `errno`
Több összetevőből álló konverziós eredmény	ISO C17 `strtol`	Végpointer, visszatérési érték és az előírt módon előkészített `errno`
Explicit hibakód	POSIX `pthread_create`	A visszaadott kód: nulla siker, egyébként maga a hibaszám
Saját eredménytípus	Az alábbi példa	A visszaadott struktúra kategóriája és mezői

ISO C17: az `errno` az `<errno.h>` fejlécben meghatározott makró, amely módosítható, `int` típusú balértéket biztosít. Ne deklaráljuk saját kezűleg `extern int errno` alakban. Állapota szálanként különálló; nem egyetlen, minden szál által közösen írt egész változó.

Az `errno` **nem általános sikerességi teszt**. Korábbi hiba értéke megmaradhat benne, és egy sikeresen visszatérő könyvtári függvény is módosíthatja. Ezért az alábbi gondolatmenet hibás:

> „Meghívtam egy függvényt, az `errno` nem nulla, tehát a függvény hibázott.”

Először a dokumentált sikertelenségi feltételt kell ellenőrizni. Az `errno` csak akkor értelmezhető, ha az adott API szerződése azt ehhez a helyzethez hozzárendeli. **Az ISO C17 `fopen` null eredményére vonatkozó garanciájából önmagában nem következik POSIX-szerű `errno`-diagnosztika; ezt a POSIX-szerződés adja hozzá.**

A `strtol` külön figyelmet igényel, mert például a nulla érvényes eredmény is lehet. Előtte `errno = 0` szükséges a tartományhiba felismeréséhez; utána a végpointer jelzi, történt-e konverzió, az `ERANGE` pedig a dokumentált tartományhibát. Ez speciális szerződés, nem univerzális „nullázd, hívd, nézd” recept.

11.3.2 Hibakategória és újrapróbálhatóság

Az alkalmazási **tartományhiba** (*domain error*) azt jelenti, hogy a bemenet kívül esik az alkalmazás által elfogadott értékek vagy állapotok halmazán. Például egy időkorlát csak az $1 \leq t \leq 3600$ tartományban lehet érvényes. Ez nem azonos a matematikai könyvtár `EDOM` hibakódjával.

Az **OS-hiba** egy rendszerhez kötődő művelet hibája, például sikertelen fájlmegnyitás. A hiba eredetét nem az dönti el, hogy `errno` közvetítette-e: a `strtol` által jelzett `ERANGE` konverziós tartományhiba, nem fájlrendszer- vagy kernelhiba.

Az **újrapróbálhatóság** (*retriable/non-retriable*) ettől független döntés:

- ugyanaz a hibás időkorlát változatlan bemenettel nem javul meg;
- egy dokumentált `EAGAIN` megfelelő várakozás után kezelhető lehet;
- egy hozzáférési hiba konfigurációváltozás nélkül rendszerint nem oldódik meg;
- egy átmeneti OS-hiba sem teszi biztonságossá egy részben végrehajtott művelet megismétlését.

A retry-döntéshez tehát a hibakód mellett kell a művelet, a már bekövetkezett mellékhatás, az idempotencia és az időkeret is. A `fclose` sikertelensége például nem engedély arra, hogy ugyanazzal a `FILE *` értékkel ismét lezárást próbáljunk.

11.3.3 Kidolgozott hibaszerződés: az első hiba megőrzése

Írjunk olyan függvényt, amely időkorlátot ment egy szövegfájlba. Szerződése:

- az érvénytelen időkorlátot fájlművelet előtt elutasítja;
- az első hibát tekinti elsődlegesnek;
- a későbbi lezárási hibát külön megőrzi;
- minden hibaadatot értékként ad vissza, kölcsönzött üzenetpointer nélkül;
- OS-hibánál visszaállítja az elsődleges hibaszámot az `errno`-ba is; más eredménynél az `errno` nem része a szerződésnek.

A `path` előfeltétele: a hívás teljes ideje alatt olvasható, érvényes nullterminált sztringre mutat. A függvény nem ellenőrizheti általánosan egy tetszőleges pointer érvényességét.

Profil: ISO C17 nyelv, Linux/POSIX libc-szerződés a hibaszámokhoz. Választott fordítási profil: a `lab-next` profil (profiltábla), x86-64 Linux. Az alábbi példa nem mért vagy igazoltan tesztelt futás eredménye.

```c
/* error_contract.c */
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>

enum error_kind {
    APP_OK,
    APP_TIMEOUT_RANGE,
    APP_OS_ERROR
};

enum operation {
    OP_NONE,
    OP_VALIDATE,
    OP_OPEN,
    OP_WRITE,
    OP_CLOSE
};

struct app_error {
    enum error_kind kind;
    enum operation operation;
    int os_errno;
    int cleanup_errno;
};

static struct app_error
write_timeout(const char *path, unsigned seconds)
{
    struct app_error error = { APP_OK, OP_NONE, 0, 0 };

    if (seconds < 1U || seconds > 3600U) {
        return (struct app_error) {
            APP_TIMEOUT_RANGE, OP_VALIDATE, 0, 0
        };
    }

    FILE *file = fopen(path, "w");
    if (file == NULL) {
        const int original_errno = errno;
        return (struct app_error) {
            APP_OS_ERROR, OP_OPEN, original_errno, 0
        };
    }

    if (fprintf(file, "timeout=%u\n", seconds) < 0) {
        const int original_errno = errno;
        error = (struct app_error) {
            APP_OS_ERROR, OP_WRITE, original_errno, 0
        };
    }
```

```
    if (fclose(file) == EOF) {
        const int close_errno = errno;

        if (error.kind == APP_OK) {
            error = (struct app_error) {
                APP_OS_ERROR, OP_CLOSE, close_errno, 0
            };
        } else {
            error.cleanup_errno = close_errno;
        }
    }

    /* A stream lezárult; a file értékét többé nem használjuk. */
    if (error.kind == APP_OS_ERROR) {
        errno = error.os_errno;
    }

    return error;
}

int main(int argc, char *argv[])
{
    if (argc != 2) {
        (void)fprintf(stderr, "usage: error_contract PATH\n");
        return EXIT_FAILURE;
    }

    const struct app_error error = write_timeout(argv[1], 30U);

    if (error.kind != APP_OK) {
        (void)fprintf(stderr,
                      "kind=%d operation=%d errno=%d cleanup_errno=%d\n",
                      (int)error.kind, (int)error.operation,
                      error.os_errno, error.cleanup_errno);
        return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}
```

Fordítás a választott profilhoz tartozó `gcc` használatával:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O2 \
    error_contract.c -o error_contract
```

A `fprintf` sikeressége még nem jelenti, hogy a pufferelt adat később is hibamentesen kiírható. Ezért a `fclose` eredményét önállóan ellenőrizzük. A sikeres lezárás sem tartósági garancia; annak követelményeit a Fájlrendszer-műveletek, mmap és tartósság tárgyalja.

Objektum vagy erőforrás	Tulajdonos	Élettartam és érvényesség	Hibakimenet
`path` mögötti sztring	Hívó	A függvény csak kölcsönzi a hívás idejére; felszabadítás vagy idő előtti élettartamvég érvényteleníti	Előfeltétel-sértést a hibakód nem tesz biztonságossá
Megnyitott stream	`write_timeout`	Sikeres megnyitástól az egyszeri `fclose` hívásig	Megnyitási, írási vagy lezárási OS-hiba
Visszaadott `app_error`	Hívó	Saját érték; könyvtári hívások nem érvénytelenítik, saját felülírása vagy élettartamvége igen	Elsődleges és opcionális másodlagos hiba

Bizonyításvázlat. Megnyitási hiba esetén nincs lezárandó stream. Sikeres megnyitás után minden vezérlési út pontosan egyszer jut el a `fclose` hívásig. Írási hiba esetén az elsődleges hibakódot még a cleanup előtt lemásoljuk. A lezárási ág ezt csak akkor tölti ki, ha korábban nem volt hiba; egyébként kizárólag a másodlagos mezőt módosítja. Így az első hiba nem veszhet el.

A függvény **nem tranzakciós**: a `"w"` mód már megnyitáskor csonkolhatja a létező fájlt. Sikertelen eredmény mellett is lehet mellékhatás, ezért automatikus újrapróbálás vagy „hiba esetén minden változatlan” garancia nem következik belőle.

11.3.4 Hibaüzenetek és élettartamuk

ISO C17: a `perror` az aktuális `errno` alapján ír diagnosztikát a standard hibakimenetre. Ha a hibás művelet és a `perror` közé cleanup vagy naplózás kerül, már más hibát magyarázhat. Előbb mentsük el a hibaszámot; ha később `perror` szükséges, közvetlenül előtte állítsuk vissza.

A `strerror(code)` a megadott hibaszámhoz tartozó szöveget adja vissza. Ez nem hívó által birtokolt sztring: nem szabad felszabadítani vagy módosítani. Az ISO C17 szerződése alapján egy későbbi `strerror` hívás felülírhatja a tartalmát, és általános párhuzamos használati biztonságot sem feltételezhetünk.

A szálanként különálló `errno` tehát **nem teszi automatikusan szálbiztossá a hibaüzenetek tárolását**. Másik szálnak a hibaszám másolatát kell megfelelő szinkronizációval átadni, nem arra számítani, hogy ott ugyanaz az `errno` olvasható.

A `strerror_r` nem ISO C API, és neve mögött eltérő interfészek találhatók:

Változat	Visszatérés	Következmény
POSIX/XSI alak	`int` státusz	A hívó pufferébe ír; a sikerességet ellenőrizni kell
GNU/glibc alak	`char *`	A visszaadott pointer nem feltétlenül a megadott pufferre mutat

A glibc által láthatóvá tett alakot feature-test makrók, például `_GNU_SOURCE`, is befolyásolják. Hordozható wrapperhez a konkrét libc-verzió deklarációját és hibaszerződését ellenőrizni kell; a két alakot casttal összemosni hibás.

Tartós diagnosztikához általában jobb a **hibakategória, művelet és numerikus kód érték szerinti tárolása**, majd a szöveg előállítása a megjelenítés helyén. Ha a szöveget is meg kell őrizni, saját tárolóba kell másolni, explicit kapacitással és csonkolási szabállyal. A lokalizált üzenetszöveg nem stabil gépi hibakód.

11.3.5 Gyakori hibák és javításuk

A cleanup utáni `errno` elmentése túl késő: mindig közvetlenül a dokumentált hiba felismerése után másoljuk ki. Ugyanez vonatkozik a naplózó függvények elé kerülő mentésre.

Az első hibát egy későbbi lezárási hibával felülírni diagnosztikai információvesztés. A két esemény külön mezőt érdemel. Ha viszont korábban minden sikerült, a lezárási hiba lesz az elsődleges.

Hasonlóan hibás az összes nemnulla hibaszámot „OS-hiba” vagy „újrapróbálható hiba” kategóriába sorolni. A kategóriát a művelet szerződése, az újrapróbálhatóságot pedig a teljes végrehajtási helyzet határozza meg.

11.3.6 Feladatok

11.3.6.1 Magyarázó feladat

Egy sikeres `fopen` után az `errno` értéke `EACCES`. Bizonyít-e ez megnyitási hibát? Mit kell vizsgálni?

11.3.6.2 Kódolási feladat

Bővítsd a példát úgy, hogy a `seconds == 0` és a `seconds > 3600` külön alkalmazási hibakódot kapjon. Hibás bemenet esetén semmilyen fájlművelet ne történjen.

11.3.6.3 Diagnosztikai feladat

Tegyük fel, hogy a tesztadapter írási hibát jelez `ENOSPC` kóddal, majd a lezárás is hibázik `EIO` kóddal. Milyen mezőértékeket kell kapnia a hívónak? Mi történjen, ha csak a lezárás hibázik?

11.3.7 Megoldások

11.3.7.1 A sikeresség megállapítása

Nem bizonyít hibát. A nemnulla pointer a `fopen` sikerét jelzi; az `errno` ilyenkor nem használható a siker felülbírálására. Ellenőrzési feltétel: a hibakezelési ág kizárólag null eredmény esetén fusson.

11.3.7.2 Külön tartományhibák

Az alábbi ISO C17 részletek az előző teljes program megfelelő deklarációját és validációs ágát helyettesítik:

```
enum error_kind {
    APP_OK,
    APP_TIMEOUT_ZERO,
    APP_TIMEOUT_TOO_LARGE,
    APP_OS_ERROR
};
```

```
if (seconds == 0U) {
    return (struct app_error) {
        APP_TIMEOUT_ZERO, OP_VALIDATE, 0, 0
    };
}
if (seconds > 3600U) {
    return (struct app_error) {
        APP_TIMEOUT_TOO_LARGE, OP_VALIDATE, 0, 0
    };
}
```

Ellenőrzési értékek: `0`, `1`, `3600`, `3601`. A két szélső érvénytelen esetben a megnyitást helyettesítő tesztadapter hívásszáma nulla legyen.

11.3.7.3 Az elsődleges hiba megőrzése

Két hiba esetén az elvárt állapot: `APP_OS_ERROR`, `OP_WRITE`, `os_errno == ENOSPC`, `cleanup_errno == EIO`. Visszatéréskor az `errno` ismét `ENOSPC`.

Ha csak a lezárás hibázik: `APP_OS_ERROR`, `OP_CLOSE`, `os_errno == EIO`, `cleanup_errno == 0`. Ellenőrzési feltétel mindkét esetben az egyszeri lezárás és az elsődleges hibát megőrző eredmény.

11.3.8 Összefoglaló

A visszatérési érték mutatja meg, mikor vizsgálható a hibajelzés; az `errno` önmagában nem sikerességi teszt. Az eredeti hibát cleanup előtt kell elmenteni. A tartós hibaszerződés saját értékként tárolja a kategóriát, műveletet és hibakódot, a szöveges magyarázatot pedig külön kezeli. A tartományhiba, az OS-hiba és az újrapróbálhatóság három külön döntési szempont.

11.4 Idő, véletlen és környezeti állapot

Egy szolgáltatás naplóbejegyzéséhez naptári időpont kell, egy hálózati kérés timeoutjához eltelt idő, teljesítményelemzéshez pedig akár CPU-idő. Ezek nem ugyanannak a mennyiségnek különböző formátumai. Hasonlóképpen más szerződés szükséges egy reprodukálható teszt véletlenszámaihoz és egy hozzáférési tokenhez.

Az időzóna, a locale, a véletlengenerátor és a környezeti változók további közös tulajdonsága a rejtett, megosztott állapot. A megfelelő függvény kiválasztása ezért egyszerre jelent időmodell-,

biztonsági és élettartam-döntést. A hibák kezelésénél a Hibakód, errno és hibaadatok élettartama, a numerikus átalakításoknál a Lebegőpontos reprezentáció és numerikus hibák szabályaira építünk.

11.4.1 Naptári idő, falióra és CPU-idő

A **falióra** (*wall clock*) a naptári időhöz kapcsolódó óra. Alkalmas események időbélyegzésére, de az értéke adminisztratív beavatkozás vagy időszinkronizáció miatt változhat. A **monoton óra** nem lép visszafelé; elsősorban időtartamokhoz és határidőkhöz használjuk. A **CPU-idő** azt közelíti, mennyi processzoridőt használt fel a program, nem azt, meddig volt életben.

API és szabványhatár	Jelentés	Hibajelzés és korlát
`time`, ISO C17	Aktuális naptári idő	`(time_t)-1`, ha nem érhető el
`timespec_get`, ISO C17	A megadott időalap szerinti idő; szabványos alap: `TIME_UTC`	Siker: a megadott alap értéke; hiba: `0`
`clock`, ISO C17	A program processzoridejének közelítése	`(clock_t)-1`, ha nem áll rendelkezésre vagy nem ábrázolható
`clock_gettime`, POSIX	Kiválasztott rendszeróra	Siker: `0`; hiba: `-1`, részletek az `errno`-ban

11.4.1.1 `time_t`, `struct tm` és időpontok különbsége

ISO C17: a `time_t` naptári idő reprezentálására szolgáló aritmetikai típus. A hordozható C-program nem feltételezheti, hogy egy adott szélességű egész, vagy hogy Unix-epoch óta eltelt másodperceket tartalmaz.

Két érvényes naptári időpont különbségéhez ezért:

```c
/* ISO C17-részlet; szükséges fejléc: <time.h>.
   begin és end érvényes, sikeresen lekért naptári időpontok. */
double elapsed_seconds = difftime(end, begin);
```

A `difftime` az első argumentum mínusz a második különbségét adja másodpercben. A közvetlen `end - begin` nem helyettesíti hordozhatóan: a reprezentációra támaszkodna, és a kivonás a `time_t` típusában történne.

A `struct tm` felbontott naptári időt tárol. Például a `tm_year` az 1900 óta eltelt éveket, a `tm_mon` a januártól számított, nullával kezdődő hónapszámot tartalmazza. Nem célszerű saját mezőaritmetikával kezelni a hónapváltást vagy a nyári időszámítást.

ISO C17: a `gmtime` UTC szerinti, a `localtime` helyi idő szerinti felbontást készít. Mindkettő null pointerrel jelezheti, hogy az átalakítás nem végezhető el. A visszaadott `struct tm` nem a hívó tulajdona.

11.4.1.2 Nagyobb felbontás nem jelent monotonitást

ISO C17: a

```c
/* ISO C17-részlet; szükséges fejléc: <time.h>. */
struct timespec stamp;
int ok = timespec_get(&stamp, TIME_UTC);
```

hívás után csak `ok == TIME_UTC` esetén használható az eredmény. A `tv_sec` az implementáció által meghatározott epochhoz viszonyított másodpercrészt, a `tv_nsec` a másodpercen belüli nanoszekundumos mezőt tartalmazza.

A mező mértékegysége nem azonos az óra tényleges felbontásával vagy pontosságával. A `TIME_UTC` nem biztosít monoton órát: timeout mérésére ugyanúgy veszélyes lehet, mint a `time`.

A `clock` más problémát old meg. Két sikeres mérésből, ábrázolható és körbefordulástól mentes intervallumon:

$$T_{\mathrm{CPU}} = \frac{c_1 - c_0}{\mathrm{CLOCKS_PER_SEC}}.$$

A kivonást és konverziót úgy kell megvalósítani, hogy ne történjen előbb egész túlcsordulás. A `CLOCKS_PER_SEC` skálatényező, nem garantált mérési felbontás. Alvás vagy I/O-várakozás alatt a falióra halad, miközben a program CPU-ideje alig nőhet.

11.4.2 Timeout monoton órával

POSIX: a `clock_gettime(CLOCK_MONOTONIC, ...)` időtartamokhoz használható. Az óra kezdőpontjának nincs naptári jelentése. Két értéket csak azonos órából és összehasonlítható rendszerkörnyezetből szabad összevetni.

Linux: a `CLOCK_MONOTONIC` nem számolja bele a rendszer felfüggesztésében töltött időt. Ha a timeoutnak ezt is tartalmaznia kell, külön Linux-specifikus óraválasztás, például `CLOCK_BOOTTIME` szükséges. A monotonitás nem jelent változatlan frekvenciát vagy valós idejű ütemezési garanciát.

Timeoutot célszerű egyszer kiszámított **határidővel** (*deadline*) kezelni. Ha minden megszakítás után újraindítjuk a teljes várakozást, a teljes művelet túllépheti az eredeti keretet.

Legyen a kezdőérték (s, n), a megengedett idő pedig (a, b), normalizált nanoszekundumos mezőkkel. Ekkor:

$$u = n + b, \qquad d_s = s + a + \left\lfloor \frac{u}{10^9} \right\rfloor, \qquad d_n = u \bmod 10^9.$$

Az összeadásokat csak akkor szabad elvégezni, ha az eredmények ábrázolhatók a használt típusokban. Szemléltető példa:

$$(120,\ 900\,000\,000) + (0,\ 250\,000\,000) = (121,\ 150\,000\,000).$$

A következő teljes fordítási egység egy már létrehozott határidőt ellenőriz:

```c
/* deadline.c - ISO C17 szintaxis, POSIX API */
#define _POSIX_C_SOURCE 200809L
#include <time.h>

/*
 * Előfeltétel:
 * deadline nem NULL, élő, olvasható objektumra mutat;
 * CLOCK_MONOTONIC alapján készült;
 * 0 <= tv_nsec < 1000000000.
 *
 * Eredmény: 1 = lejárt, 0 = még nem járt le,
```

```
 *          -1 = óraleolvasási hiba, errno szerint.
 */
int deadline_expired(const struct timespec *deadline)
{
    struct timespec now;

    if (clock_gettime(CLOCK_MONOTONIC, &now) == -1) {
        return -1;
    }

    return now.tv_sec > deadline->tv_sec ||
           (now.tv_sec == deadline->tv_sec &&
            now.tv_nsec >= deadline->tv_nsec);
}
```

Bizonyításvázlat: normalizált mezők esetén egy nagyobb másodpercérték minden lehetséges nanoszekundumos mező mellett későbbi időpontot jelent. Azonos másodpercnél a nanoszekundumos mezők döntenek. Az egyenlőség már lejárt határidőt jelent.

A függvény nem várakoztat. Egy blokkoló műveletnek a még hátralévő időt kell átadni, megszakítás után pedig ugyanahhoz a határidőhöz kell visszatérni. Az ellenőrzés önmagában nem tudja megakadályozni, hogy az ütemező később engedje tovább a folyamatot.

Javasolt, itt nem fordítással ellenőrzött laborprofil: Linux x86-64, a `lab-next` profil (profiltábla), explicit C17 módban:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -c deadline.c -o deadline-gcc.o
clang -std=c17 -Wall -Wextra -Wpedantic -c deadline.c -o deadline-clang.o
```

Ez objektumfájlt készítő modul, ezért nincs `main` függvénye. Más, különösen régebbi platformon a linkelési követelményeket külön ellenőrizni kell.

11.4.3 Időzóna, locale és kölcsönzött tároló

A naptári időpont és annak megjelenítése külön réteg. Ugyanazt az időpontot a `localtime` az időzóna szerint bontja fel, a `strftime` pedig a választott formátum és részben a locale szerint írja ki.

ISO C17: a `setlocale` megosztott locale-állapotot módosít; a `strftime` többek között a `LC_TIME` kategóriától függ. **POSIX:** a `TZ` környezeti változó és a `tzset` az időzóna-kezelés része. Ezeket nem szabad kérésenként, párhuzamos kiszolgálás közben biztonságosan elszigetelt beállításnak tekinteni.

Hozzáférés	Tulajdonos és élettartam	Érvénytelenítés, hibakimenet
`gmtime`/`localtime` eredménye	Könyvtári statikus tároló; kölcsönzött pointer	Kapcsolódó időkonverziós hívások felülírhatják; hiba: null pointer
`getenv` eredménye	Implementáció által kezelt tároló; kölcsönzött sztring	Újabb `getenv` felülírhatja; környezetmódosításra sem tartható meg biztonságosan; hiány: null pointer

Hozzáférés	Tulajdonos és élettartam	Érvénytelenítés, hibakimenet
`deadline_expired` argumentuma	A hívó által fenntartott objektum	Élettartamvége után nem használható; párhuzamos, szinkronizálatlan módosítása hibás
Hívói `struct tm` másolat	A hívó saját objektuma	A könyvtár későbbi hívása nem írja felül

Ez például hibás mentális modell:

```
/* ISO C17, szemléltető részlet; <time.h>.
   Nem megfelelő: a két pointer nem jelent két független eredményt. */
struct tm *local = localtime(&now);
struct tm *utc = gmtime(&now);
```

A második hívás felülírhatja az első eredményt. Egyszálú, közbeeső hívástól mentes kódban a null pointer ellenőrzése utáni azonnali struktúramásolat megoldás. Párhuzamos hívásoknál azonban a másolás önmagában nem szinkronizáció.

POSIX: a `localtime_r` és `gmtime_r` hívói tárolóba ír. Ez megszünteti a visszatérési objektum megosztását, de nem teszi problémamentessé az időzóna vagy környezet egyidejű átállítását.

11.4.4 Véletlen: tesztadat vagy biztonsági titok?

ISO C17: a `rand` a $[0, \text{RAND_MAX}]$ tartományból ad pszeudovéletlen értéket. A `srand` inicializálja a generátor állapotát. Ugyanaz a seed ugyanazon implementációban ugyanazt a sorozatot eredményezi; különböző C könyvtárak között nem garantált azonos sorozat.

A `rand` **nem kriptográfiailag biztonságos generátor**, azaz nem CSPRNG. A

```
/* ISO C17-részlet; <stdlib.h>, <time.h>.
   Biztonsági token előállítására alkalmatlan. */
srand((unsigned)time(NULL));
```

nem változtat ezen: a kezdőérték gyakran becsülhető, és azonos kezdőértékek ismétlődhetnek. A megosztott generátorállapot miatt egy másik komponens hívásai a tesztek reprodukálhatóságát is befolyásolhatják.

A `rand() % k` még ideálisan egyenletes bemenetből is torzíthat. Ha matematikailag

$$R = \text{RAND_MAX} + 1 = qk + r, \qquad 0 \le r < k,$$

akkor r maradékhoz $q+1$, a többihez q bemeneti érték tartozik. Ez matematikai képlet: a `RAND_MAX + 1` C-kifejezés `int` típusban túlcsordulhat. A modulo-torzítás javítása sem teszi kriptográfiailag biztonságossá a generátort.

Linux-specifikus: titkokhoz a kernel véletlenforrását használó `getrandom` megfelelő kiindulópont, nem ISO C17 API. A visszaadott bájtszámot ellenőrizni kell; rövid eredmény, megszakítás és inicializálási várakozás is része lehet a szerződésnek. Hibakor nem elfogadható „tartalékmegoldásként” `rand`-ra váltani.

11.4.5 Környezeti változók és konfigurációs pillanatkép

ISO C17: a `getenv` null pointert ad, ha a változó nem található. A megtalált üres sztring ettől különböző állapot. A visszaadott sztringet nem szabad módosítani vagy `free`-vel felszabadítani.

POSIX: a `setenv` és `unsetenv` környezetet módosít. A párhuzamos olvasás és módosítás biztonságát nem szabad automatikusan feltételezni. A másolatkészítés is olvasás: ha közben a forrás változhat vagy érvénytelenné válhat, a másolás sem biztonságos.

Szolgáltatásnál jó szerződés az induláskori konfigurációs pillanatkép: még a munkaszálak előtt kiolvasni, validálni és saját tárolóba másolni az értékeket. Ezután a program ne a folyamat környezetét használja élő konfigurációs adatbázisként.

11.4.6 Feladatok

1. **Óraválasztás.** Válassz API-t napló-időbélyeghez, kétmásodperces timeout méréséhez és egy számítás CPU-költségéhez. Mi történhet, ha a faliórát közben visszaállítják?
2. **Kódolás.** Készíts `copy_service_mode` függvényt, amely a `SERVICE_MODE` értékét hívói bufferbe másolja. Eredmény: `0` siker, `1` hiányzó változó, `-1` elégtelen kapacitás. Hibánál maradjon változatlan a buffer.
3. **Diagnosztika.** Egy szolgáltatás minden kérés előtt újrahívja a `srand(time(NULL))` kifejezést, `rand`-ból készít hozzáférési tokent, globálisan módosítja a `TZ` értékét, és minden megszakítás után új teljes timeouttal vár. Nevezd meg a négy független hibát.

11.4.7 Megoldások

11.4.7.1 Óraválasztás

Naplóhoz ISO C17 `time` vagy `timespec_get(TIME_UTC)`, timeout méréséhez POSIX `CLOCK_MONOTONIC`, CPU-költséghez ISO C17 `clock` megfelelő.

A falióra visszaállítása negatív időeltérést vagy megnyúlt timeoutot okozhat. Ellenőrzési feltétel: a timeout minden újrapróbálkozása ugyanahhoz a monoton határidőhöz viszonyítson.

11.4.7.2 Konfiguráció másolása

```c
/* env_copy.c - ISO C17 */
#include <stddef.h>
#include <stdlib.h>
#include <string.h>

/*
 * dst legalább cap C bájt írható tárolóra mutat, ha cap > 0.
 * A forrás és cél nem fed át.
 * A környezet és a getenv-hozzáférés a művelet alatt stabil.
 */
int copy_service_mode(char *dst, size_t cap)
{
    const char *value = getenv("SERVICE_MODE");
    if (value == NULL) {
```

```
        return 1;
    }

    size_t length = strlen(value);
    if (length >= cap) {
        return -1;
    }

    memcpy(dst, value, length + 1);
    return 0;
}
```

A korábbi laborprofilban:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -c env_copy.c
```

A cél tulajdonosa a hívó; a függvény nem őrzi meg egyik pointert sem. A kapacitásellenőrzés a másolás előtt történik, ezért a hibautak nem módosítják a célt. Ellenőrizendő: hiányzó és üres érték, pontosan elegendő kapacitás, valamint a lezáró nullkarakter számára már elégtelen buffer.

11.4.7.3 Diagnosztikai javítás

Az ismételt seedelés ismétlődő sorozatokat okozhat; a `rand` nem alkalmas tokenhez; a globális időzóna-módosítás összekeverheti a párhuzamos kérések állapotát; a teljes timeout újraindítása túllépi a teljes időkeretet.

A javítás: megfelelő OS-véletlenforrás ellenőrzött hibakezeléssel, rögzített vagy explicit időzóna-kezelés, valamint egyetlen monoton határidő. A tesztgenerátor seedjét külön, reprodukálható tesztkonfigurációként kell kezelni.

11.4.8 Összefoglaló

Naptári időpont, eltelt idő és CPU-idő külön fogalom. Timeout méréséhez monoton óra kell; a nanoszekundumos mező önmagában nem biztosít sem monotonitást, sem nanoszekundumos pontosságot. A `rand` nem biztonsági eszköz. Az időkonverzió, locale és környezet pedig megosztott állapotot és kölcsönzött tárolót hozhat az API-ba: az ownership, az élettartam és a párhuzamos módosítás szerződését ezeknél is explicitté kell tenni.

11.5 Numerikus és ritkább C könyvtári eszközök térképe

A standard fejléc kiválasztása nem pusztán névkeresés: egyben szerződésválasztás. Egy numerikus függvénynél számít az értelmezési tartomány és a hibajelzés módja; karakterfeldolgozásnál az aktuális locale és a konverziós állapot; diagnosztikánál pedig az, hogy a vizsgálat egyáltalán bekerül-e a programba.

Ez a térkép a lebegőpontos reprezentációra, a típusfüggő headeres eszközökre és a környezeti állapotra épít. Az alap **ISO C17, hosted környezetben**; az opcionális képességeket külön jelöljük. A fejléc megléte önmagában nem bizonyít például teljes IEC 60559 lebegőpontos működést vagy meghatározott Unicode-kódolást.

11.5.1 Numerikus függvények és lebegőpontos környezet

A `<math.h>` valós matematikai műveleteket, osztályozó makrókat és hibajelzési eszközöket ad. A függvénycsaládokban rendszerint a toldalék nélküli változat `double`, az `f` végű `float`, az `l` végű `long double` típussal dolgozik:

```
/* ISO C17; <math.h>; kifejezésrészletek. */
sqrt(9.0)
sqrtf(9.0f)
sqrtl(9.0L)
```

A típust nem szabad összekeverni a pontossággal: a `long double` nem minden implementációban ad több értékes számjegyet a `double` típusnál.

A **tartományhiba** (*domain error*) azt jelenti, hogy az argumentum kívül esik a függvény értelmezési tartományán; ilyen a valós `sqrt(-1.0)`. Az **eredménytartomány-hiba** (*range error*) az eredmény reprezentálhatóságával kapcsolatos, például túlcsorduláskor. A `math_errhandling` jelzi, hogy az implementáció `errno`-alapú jelzést (`MATH_ERRNO`), lebegőpontos kivételjelzőket (`MATH_ERREXCEPT`), vagy mindkettőt használja. Az `errno == 0` ezért önmagában nem általános sikerfeltétel.

11.5.1.1 Kidolgozott numerikus példa

A következő programban a matematikai előfeltételt futásidejű ellenőrzés biztosítja. Az assertion csak egy belső invariánst dokumentál.

```
/* numeric.c - ISO C17, hosted környezet */
#include <assert.h>
#include <float.h>
#include <inttypes.h>
#include <limits.h>
#include <math.h>
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    const double squared = 9.0;

    if (!isfinite(squared) || squared < 0.0) {
        return EXIT_FAILURE;
    }

    const double root = sqrt(squared);
    assert(root >= 0.0);

    const uintmax_t samples = 42;
    const size_t object_bytes = sizeof root;

    if (printf("root=%.17g samples=%" PRIuMAX
               " sizeof=%zu CHAR_BIT=%d DBL_MANT_DIG=%d\n",
               root, samples, object_bytes,
```

```
                CHAR_BIT, DBL_MANT_DIG) < 0) {
        return EXIT_FAILURE;
    }

    return fflush(stdout) == 0 ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

A fordítási parancs választott laborprofilja: a `lab-next` profil (profiltábla), Linux x86-64, System V ABI, LP64. Ez nem futtatással ellenőrzött eredményközlés.

```
gcc -std=c17 -Wall -Wextra -Wpedantic numeric.c -lm -o numeric
```

A `-lm` e toolchain linkelési konvenciója, nem ISO C-előírás. A matematikailag várt gyökérték 3; a kiírt típusjellemzőket a tényleges implementáció szolgáltatja. A `sizeof` egysége **C bájt**, amelynek bitszámát a `CHAR_BIT` adja meg.

Indoklás. A negatív, végtelen és NaN bemeneteket kizártuk. Nemnegatív, véges `double` érték négyzetgyöke nem okoz túlcsordulást; a példában a bemenet és az eredmény is pontosan reprezentálható. Az általános numerikus pontosság kérdését ettől még nem oldja meg egy assertion.

11.5.1.2 Komplex számok és típusgenerikus matematika

A `<complex.h>` komplex típusokhoz és műveletekhez tartozik. C17-ben a komplex támogatás opcionális; hiányát a `__STDC_NO_COMPLEX__` makró jelezheti.

A `<tgmath.h>` típusgenerikus makrói az argumentum típusa alapján választanak matematikai függvényt. Ez nem C++-féle túlterhelés.

```
/* ISO C17; komplex támogatást igénylő részlet. */
#include <complex.h>
#include <tgmath.h>

/* Függvénytörzsben: */
double complex z = 1.0 + 2.0 * I;
double complex root = sqrt(z); /* A megfelelő csqrt-változat. */
double magnitude = cabs(root);
```

Itt a `sqrt` komplex műveletet választ. Ha ugyanaz a makró egész argumentumot kap, a választásnál az egész típus rendszerint `double`-nak számít. Az „azonos függvénynév” tehát nem jelent azonos eredménytípust vagy költséget.

11.5.1.3 A `<fenv.h>` szerepe

A **lebegőpontos környezet** a támogatott kerekítési módokat és a lebegőpontos kivételjelzőket foglalja magába. A jelző nem C++-kivétel, és nem szükségképpen operációsrendszer-szignál.

A jelzők tartósan beállítva maradhatnak, amíg nem töröljük őket. Ezért egy művelet vizsgálatának sorrendje:

1. a releváns jelzők törlése, a `feclearexcept` eredményének ellenőrzésével;
2. a vizsgált művelet;
3. a jelzők lekérdezése a `fetestexcept` segítségével.

Például a `fetestexcept(FE_INVALID) != 0` beállított érvénytelenművelet-jelzőt keres, **ha a `FE_INVALID` támogatott és definiált**. Törlés nélkül egy korábbi művelet hibáját is láthatjuk.

Ha az aritmetika környezetfüggő működésére támaszkodunk, a `#pragma STDC FENV_ACCESS ON` fordítói támogatását külön ellenőrizni kell. A fejléc jelenléte ezt nem bizonyítja; a környezetet figyelmen kívül hagyó optimalizálási módok, például bizonyos `-ffast-math` beállítások, alkalmatlanok ilyen vizsgálathoz.

11.5.2 Assertion és üzemi hibakezelés

A `<assert.h>` `assert` makrója belső programinvariánsok diagnosztikájára szolgál. Aktív állapotban a hamis feltétel diagnosztikát és rendellenes programleállást vált ki.

Ha az `NDEBUG` definiált a fejléc feldolgozásakor, az assertion feltétele **nem értékelődik ki**. Az `NDEBUG=0` is definiált makró, tehát szintén kikapcsolja az assertionöket.

```
/* Hibás tervezési példa; a művelet eltűnhet a release buildből. */
assert(++count < limit);
```

A javítás nem pusztán az inkrementálás áthelyezése. Előbb a művelet saját előfeltételeit — például a túlcsordulás kizárását — kell biztosítani, utána külön végrehajtani, végül mellékhatásmentesen ellenőrizni az invariánst.

Felhasználói bemenetet, allokációs hibát vagy fájlmegnyitási hibát nem kezelünk kizárólag assertionnel. Ezek a program üzemi hibaszerződésének részei, release buildben is.

11.5.3 Locale, széles és többbájtos karakterek

A „széles karakter” (*wide character*) nem azonos azzal, hogy „Unicode-kódpont”. A `wchar_t` szélessége és a hozzá kapcsolódó karakterkódolás implementációfüggő.

Fejléc	Feladat és minimális példa	Fontos feltétel
`<locale.h>`	`setlocale(LC_CTYPE, "")`	Környezetből választ locale-t; sikertelenségkor null pointert ad
`<wchar.h>`	`mbrtowc(&wc, "A", 1, &state)`	Állapottartó többbájtos dekódolás
`<wctype.h>`	`iswalpha((wint_t)wc)`	Locale-függő karakterosztályozás
`<uchar.h>`	`mbrtoc16(&unit, "A", 1, &state16)`	`char16_t` kódegységet állít elő

A táblázat kifejezésrészleteiben `wc` egy `wchar_t`, `unit` egy `char16_t`, a két állapot pedig külön, nullával inicializált `mbstate_t` objektum. A szükséges deklaráció például:

```
/* ISO C17; <wchar.h>; függvénytörzs-részlet. */
mbstate_t state = {0};
wchar_t wc;
size_t consumed = mbrtowc(&wc, "A", 1, &state);
```

A hívó tulajdonolja a kimeneti és állapotobjektumot; a függvény csak a hívás idejére kölcsönzi a címüket. Az állapotot ugyanazon konverziós folyamat folytatásához meg kell őrizni. Az objektum élettartamának vége érvényteleníti a rá mutató pointert; az állapot felülírása pedig elveszítheti a dekódolás folytatásához szükséges információt.

A `mbrtowc` eredménye nem egyszerű siker/hiba érték: `(size_t)-1` érvénytelen sorozatot, `(size_t)-2` hiányos sorozatot, nulla nullkaraktert jelez. A `mbrtoc16` ezenfelül `(size_t)-3` eredménnyel is visszatérhet, amikor korábban feldolgozott bemenetből további kódegységet ad ki.

A `__STDC_UTF_16__` és `__STDC_UTF_32__` makrók a megfelelő kódolási garanciákat jelzik. A típusnevekből önmagukban ne következtess teljes Unicode-feldolgozásra.

A `setlocale` visszatérési sztringjét az implementáció kezeli: nem szabad felszabadítani vagy módosítani, és későbbi `setlocale` hívás felülírhatja. A locale váltása megosztott környezeti állapotot érint; részletes szövegkezelési hátterét a kódolásról szóló rész tárgyalja.

11.5.4 Szignálok és nem lokális vezérlés

A `<signal.h>` az ISO C szignálmodelljét adja. Minimális jelzőobjektum például:

```
/* ISO C17; fájlszintű deklaráció. */
#include <signal.h>
static volatile sig_atomic_t stop_requested = 0;
```

Ez nem általános szálközi szinkronizációs eszköz. Egy aszinkron szignálkezelőből megengedett műveletek köre erősen korlátozott; a `printf` vagy a tetszőleges cleanup meghívását nem teszi biztonságossá ez a deklaráció. Az ISO C és a POSIX szignálszerződéseit a dedikált szignálfejezet választja szét.

A `<setjmp.h>` `setjmp`/`longjmp` párosa mentett végrehajtási környezethez ugrik vissza:

```
/* ISO C17; <setjmp.h>; függvénytörzs-részlet. */
jmp_buf env;

if (setjmp(env) == 0) {
    longjmp(env, 1);
}
```

Az első visszatérés nulla; az ugrás utáni visszatérés itt egy. A `setjmp` hívást tartalmazó függvénynek még aktívnak kell lennie. A `setjmp` után módosított, nem `volatile`, automatikus lokális változók értéke az ugrás után indeterminate lehet.

A vezérlésugrás nem futtat automatikus erőforrás-cleanupot. Ezért nem egyszerű helyettesítője a hibavisszatérésnek; lásd a nem lokális vezérlés életciklus-korlátait.

11.5.5 Típushatárok és hordozható referenciaértékek

Fejléc	Mire használjuk?	Gyakori tévedés
`<limits.h>`	`CHAR_BIT`, `INT_MAX`, `LONG_MAX`	A `long` nem mindenhol 64 bites

Fejléc	Mire használjuk?	Gyakori tévedés
`<float.h>`	`FLT_RADIX`, `DBL_MANT_DIG`, `DBL_MAX`	A `DBL_MIN` a legkisebb pozitív normál érték, nem a legnegatívabb
`<stddef.h>`	`size_t`, `ptrdiff_t`, `offsetof`, `max_align_t`	A `sizeof` nem szükségképpen oktettekben mér
`<inttypes.h>`	`PRIuMAX`, `PRId64`, `strtoimax`	A `%ld` nem általános helyettesítője a típusspecifikus formátumnak

A `DBL_EPSILON` az 1 és a következő nagyobb reprezentálható `double` érték különbségét jellemzi; nem univerzális numerikus tolerancia.

Az exact-width típusok, például az `int64_t`, opcionálisak. A hozzájuk tartozó formázó makrókra csak a megfelelő típus rendelkezésre állásakor támaszkodhatunk. A fenti program ezért használja a hordozható `uintmax_t`–`PRIuMAX` párost. Az `offsetof(T, member)` eredménye C bájtban értendő; bit-fieldre nem alkalmazható.

11.5.6 A C17 fejléc-térkép és az opcionális részek

A C17 29 standard fejlécnevének teljes térképe:

Terület	Fejlécek
Numerika és környezet	`<math.h>`, `<complex.h>`, `<tgmath.h>`, `<fenv.h>`
Típusjellemzők	`<limits.h>`, `<float.h>`, `<stddef.h>`, `<stdint.h>`, `<inttypes.h>`
Karakterek és locale	`<ctype.h>`, `<locale.h>`, `<wchar.h>`, `<wctype.h>`, `<uchar.h>`
I/O, memória, idő, hibák	`<stdio.h>`, `<stdlib.h>`, `<string.h>`, `<time.h>`, `<errno.h>`
Diagnosztika és vezérlés	`<assert.h>`, `<signal.h>`, `<setjmp.h>`
Nyelvi segédeszközök	`<stdarg.h>`, `<stdbool.h>`, `<stdalign.h>`, `<stdnoreturn.h>`, `<iso646.h>`
Konkurencia	`<stdatomic.h>`, `<threads.h>`

A táblázat leltár, nem minden környezetre érvényes támogatási garancia. A komplex támogatás mellett az atomikus és szálkönyvtári támogatás is lehet opcionális; a kapcsolódó jelzések `__STDC_NO_ATOMICS__` és `__STDC_NO_THREADS__`. Freestanding implementációban a könyvtári minimum szűkebb.

11.5.6.1 Annex K: névleges bounds-checking nem jelent automatikus biztonságot

Az Annex K opcionális, bounds-checking interfészeket ad, például `memcpy_s` és `strcpy_s` nevekkel. Nem önálló fejléc.

Támogatását a `__STDC_LIB_EXT1__` jelzi; használati igényét a releváns fejlécek előtt definiált, `1` értékű `__STDC_WANT_LIB_EXT1__` makróval kell megadni. A definíció önmagában nem teremti meg a hiányzó libc-támogatást.

Ezek az interfészek sem tudják általában ellenőrizni, hogy a hívó által közölt pufferméret megfelel-e a tényleges objektumnak. Futásidejű szerződéssértéskor constraint handler is szerepet kap, így a hibakezelés nem feltétlenül egyszerű visszatérési kód. Egy gyártói `_s` nevű függvény nem szükségképpen Annex K-kompatibilis. Hordozható API-t ezért ne építs ellenőrizetlen elérhetőségükre.

11.5.7 Feladatok

1. **Magyarázat.** Miért hibás általános állítás, hogy a `DBL_EPSILON` minden összehasonlításhoz megfelelő tolerancia, illetve hogy a `wchar_t` Unicode-kódpontot tárol?
2. **Kódolás.** A numerikus program `samples` változója legyen módosítható. Növeld eggyel túlcsordulás nélkül, majd assertionnel ellenőrizd, hogy pozitív. Az üzemi működés `-DNDEBUG` mellett se változzon.
3. **Diagnosztika.** Egy C17 build feltétel nélkül használ komplex számokat és `memcpy_s`-t. Numerikus hibát kizárólag `errno` alapján keres, miközben `-ffast-math` kapcsolóval fordít. Milyen különálló ellenőrzések hiányoznak?

11.5.8 Megoldások

11.5.8.1 Magyarázó feladat

A `DBL_EPSILON` az egység körüli reprezentációt jellemzi. A megfelelő tolerancia függ a nagyságrendtől, a számítás hibájától és az alkalmazás követelményeitől. A `wchar_t` jelentése implementáció- és locale-függő; egy kódegység ráadásul nem szükségképpen teljes kódpont.

Ellenőrzési feltétel: a válasz különítse el a reprezentációt az alkalmazási pontosságtól, illetve a típust a kódolástól.

11.5.8.2 Kódolási feladat

Az eredeti deklaráció legyen `uintmax_t samples = 42;`, majd következzen:

```
/* A numeric.c main függvényébe illesztendő C17-részlet. */
if (samples == UINTMAX_MAX) {
    return EXIT_FAILURE;
}
++samples;
assert(samples > 0);
```

Az unsigned aritmetika körbefordulása definiált lenne, de megsértené a számláló szerződését. A guard ezt előzi meg.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -DNDEBUG \
    numeric.c -lm -o numeric-release
```

Ellenőrzési feltétel: mindkét buildben a számláló várt értéke 43; `UINTMAX_MAX` kezdőértéknél mindkettő hibával térjen vissza.

11.5.8.3 Diagnosztikai feladat

Külön kell vizsgálni a komplex támogatást, az Annex K implementációs és deklarációs feltételeit, valamint a `math_errhandling` jelzését. A lebegőpontos környezetre támaszkodó kódhoz a fordító támogatását és optimalizálási beállításait is ellenőrizni kell.

Ellenőrzési feltétel: a konfiguráció vagy igazolja a szükséges képességeket, vagy világosan elutasítja az adott buildet; nem helyettesíti a hiányzó szerződést puszta függvénynév-egyezéssel.

11.5.9 Továbbtanulás és összefoglalás

Elsődleges továbbtanulási kiindulópont a WG14 dokumentumgyűjteménye és a GNU C Library kézikönyve. Ezek itt kutatási kiindulópontok, nem konkrét C17-szakaszok ellenőrzött bizonyítékai. A szabványverziót és a használt libc dokumentációjának verzióját külön kell azonosítani.

A fejlécválasztás helyes sorrendje: **feladat, nyelvi szerződés, opcionális képesség, implementációs támogatás, hibakezelés**. Az assertion nem üzemi validáció; a széles karakter nem automatikusan Unicode; a numerikus hibajelzés nem kizárólag `errno`; a standard név pedig nem jelenti minden környezetben ugyanazt a rendelkezésre állást.

12 C mint Linux/POSIX rendszerprogramozási nyelv

12.1 File descriptor és robusztus I/O ciklus

A rendszerhívás sikeres visszatérése nem feltétlenül jelenti a teljes alkalmazásszintű művelet befejezését. Egy `read()` kevesebbet olvashat a kértnél, egy `write()` csak a buffer elejét továbbíthatja, a `close()` pedig akkor is jelezhet hibát, amikor a descriptor már nem használható. A robusztus I/O ezért nem egyetlen hívás, hanem dokumentált állapotátmenetek sorozata.

Az errno kezelésénél megismert szabály változatlan: az `errno` értékét csak hibát jelző visszatérés után értelmezzük, és cleanup előtt elmentjük. A bufferekhez a byte view és memóriatartományok szerződését használjuk. A standard descriptorok kapcsolatát a programindulással a main, argumentumok és környezet tárgyalta.

12.1.1 A descriptor mint tulajdonolt erőforrás

POSIX: a file descriptor, röviden **FD**, egy folyamat descriptortáblájának nemnegatív egész indexe. Nem maga a fájl, és nem C-pointer. Az FD egy megnyitott fájlleírásra (*open file description*) hivatkozik; ehhez tartozhat például az aktuális fájlpozíció és az `O_NONBLOCK` állapot.

Az FD-t tároló `int` objektum élettartama és az OS-erőforrás élettartama különbözik. Az egész érték lemásolása nem hoz létre új descriptort.

Művelet	Tulajdonlási hatás	Érvénytelenítés, hibakimenet
`open()`	Siker esetén új FD tulajdonosa a hívó	Hiba: `-1`, nincs új erőforrás
`read()`, `write()`	Csak kölcsönzi az FD-t és a buffert	Nem zárja le az FD-t; hiba: `-1` és `errno`
`dup()`	Új, külön lezárandó FD-t ad	Hiba: `-1`; az eredeti változatlan
`dup2()`	A cél-FD kötését lecserélheti	A régi célhoz tartozó kölcsönzések szemantikailag elavulnak
`close()`	Lezárási kísérlet; a pontos hibapolicy platformfüggő	A szám később más erőforráshoz kioszthető
`fileno()`	A `FILE` által kezelt FD kölcsönzése	`fclose()` után nem használható

A `dup()` eredménye ugyanarra a megnyitott fájlleírásra hivatkozik: a fájlpozíció és a fájlállapotjelzők közösek. A két FD azonban külön lezárandó. A `dup2(oldfd, newfd)` az ellenőrzés és csere közötti versenyablak nélkül helyettesíti a célkötést; nem azonos egy kézzel írt `close(newfd); dup(oldfd);` sorozattal. Érvényes, azonos argumentumoknál nem hoz létre új FD-t. A lecserélt cél lezárásának esetleges hibáját nem jelenti külön.

POSIX, Linuxon is: az `O_CLOEXEC` az új FD-hez close-on-exec jelzőt állít be: sikeres `exec` esetén az FD bezáródik. Ez nem helyettesíti a normál cleanupot, viszont elkerüli a megnyitás és egy későbbi jelzőbeállítás közötti öröklési versenyablakot. A `dup()` és az eltérő FD-k közötti `dup2()` nem viszi tovább ezt a jelzőt az új FD-re; azonos argumentumú `dup2()` változatlanul hagyja.

12.1.2 A visszatérési értékből következő állapot

A `read()` és `write()` eredménye `ssize_t`, mert nemnegatív darabszám mellett a `-1` hibajelzést is reprezentálnia kell. Csak a hibavizsgálat után alakítsuk `size_t` típusra.

POSIX, pozitív kérési méret mellett:

Eredmény	`read()`	`write()`
Pozitív érték	Ennyi C bájt érvényes a bufferben	Ennyi C bájtnyi prefixet elfogadott
`0`	Szokásos fájlnál vagy byte-stream végpontnál EOF	A helpernek előrehaladás nélküli esetet kell kezelnie
`-1`, `EINTR`	Megszakítás; a hívás nem közölt átvitt adatot	Megszakítás; a hívás nem közölt átvitt adatot
`-1`, `EAGAIN` vagy `EWOULDBLOCK`	Most nem végezhető el blokkolás nélkül	Most nem végezhető el blokkolás nélkül
Más hiba	Hibapolicy szükséges	Hibapolicy szükséges

A **rövid művelet** (*partial I/O*) pozitív, de a kértnél kisebb eredmény. Önmagában nem hiba és nem EOF. Ha megszakítás előtt történt adatátvitel, a hívás pozitív darabszámot is visszaadhat.

Az EOF sem feltétlenül végleges tulajdonság: egy később megnövelt fájlból újra lehet adatot olvasni. Pipe esetén az üres buffer és az összes íróvég lezárása együtt vezet EOF-hoz.

A következő helper szerződése normál fájlokra, pipe-okra és byte-stream végpontokra vonatkozik, nem speciális eszközök minden üzemmódjára. Egyes terminálbeállításoknál például a nulla olvasási eredmény más jelentésű lehet.

A **blocking** mód nem garantál teljes átvitelt. A **nonblocking** mód pedig azt követeli meg az alkalmazástól, hogy `EAGAIN` után várjon egy megfelelő eseményre, ne foglalja a processzort azonnali újrapróbálkozással. Linuxon az `O_NONBLOCK` normál fájlokra nem jelent általános garanciát arra, hogy az I/O soha nem vár háttértárra.

12.1.3 Hibakezelt read/write helper

A szerződés külön kezeli az előrehaladást, a leállás okát és a hibakódot:

- `read_some()` legfeljebb egy olvasási kísérletet végez; rövid olvasás megengedett.
- `write_all()` addig ír, amíg minden adat elfogy, vagy jelentendő állapot következik be.
- `IO_INTR` esetén a hívó dönt az újrapróbálásról vagy megszakításról.
- Nulla hossz esetén nincs rendszerhívás; az eredmény `IO_OK`, nem EOF.
- A helper nem veszi át az FD tulajdonjogát, és nem őriz meg bufferpointert.

Példaprofil: ISO C17, POSIX API, Linux x86-64 System V, LP64, 8 bites C bájt. Választott laborfordítók: a `lab-next` profil (profiltábla); libc:. A kódhoz itt nem tartozik fordítási vagy futtatási jegyzőkönyv.

Az alábbi `fd_io.c` önállóan fordítható modul:

```c
#define _POSIX_C_SOURCE 200809L

#include <errno.h>
#include <limits.h>
#include <stddef.h>
#include <unistd.h>

typedef enum {
    IO_OK, IO_EOF, IO_WAIT, IO_INTR, IO_ERROR
} io_state;

typedef struct {
    size_t done;
    io_state state;
    int error;
} io_result;

static size_t request_size(size_t remaining)
{
    const size_t limit = (size_t)SSIZE_MAX;
    return remaining < limit ? remaining : limit;
}

static io_result stopped(size_t done, int error)
{
    io_state state = IO_ERROR;

    if (error == EINTR)
        state = IO_INTR;
    else if (error == EAGAIN || error == EWOULDBLOCK)
        state = IO_WAIT;

    return (io_result){done, state, error};
}

io_result read_some(int fd, void *buffer, size_t capacity)
{
    if (capacity == 0)
        return (io_result){0, IO_OK, 0};

    ssize_t n = read(fd, buffer, request_size(capacity));

    if (n > 0)
        return (io_result){(size_t)n, IO_OK, 0};
    if (n == 0)
        return (io_result){0, IO_EOF, 0};

    int saved = errno;
    return stopped(0, saved);
}

io_result write_all(int fd, const void *buffer, size_t length)
```

```
{
    const unsigned char *bytes = buffer;
    size_t done = 0;

    while (done < length) {
        ssize_t n = write(fd, bytes + done,
                          request_size(length - done));

        if (n > 0) {
            done += (size_t)n;
            continue;
        }

        if (n == 0)
            return (io_result){done, IO_ERROR, EIO};

        int saved = errno;
        return stopped(done, saved);
    }

    return (io_result){done, IO_OK, 0};
}

/* Kifejezetten Linux-policy: a tulajdonjogot elfogyasztja. */
int close_owned_linux(int *owner)
{
    int fd = *owner;
    *owner = -1;

    if (fd == -1)
        return 0;
    if (close(fd) == 0)
        return 0;

    return errno;
}
```

Fordítás, linkelés és futtatás nélkül:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O2 \
    -c fd_io.c -o fd_io.gcc.o

clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O2 \
    -c fd_io.c -o fd_io.clang.o
```

12.1.3.1 Előfeltételek és határértékek

A buffer tulajdonosa a hívó. Pozitív hossz esetén a teljes megadott tartomány legyen élő és megfelelően hozzáférhető: olvasásnál írható, írásnál olvasható. Felszabadítás, áthelyező `realloc` vagy az objektum élettartamának vége érvényteleníti a kölcsönzést. A buffer és az FD a hívás közben nem szüntethető meg más végrehajtási szálból.

A példa nem támogat külön igazítási szerződést igénylő `O_DIRECT` I/O-t. Pipe vagy socket írásánál a hívónak a `SIGPIPE` policy-t is rendeznie kell: különben a folyamat megszűnhet, mielőtt az `EPIPE` eredményt feldolgozná.

Bizonyításvázlat a méretszámítás helyességére. Legyen N a teljes hossz, d a már továbbított mennyiség. A ciklusinvariáns:

$$0 \le d \le N \le \text{SIZE_MAX}.$$

A következő kérés:

$$k = \min(N - d, \text{SSIZE_MAX}).$$

Pozitív eredménynél $0 < r \le k$, ezért:

$$d + r \le d + (N - d) = N.$$

Így sem a kivonás, sem az összeadás nem csordul túl. A pointert csak $d < N$ mellett képezzük; nulla hosszhoz null pointer is adható. Az `SSIZE_MAX` korlát elkerüli az ennél nagyobb kérés implementációfüggő kezelésére támaszkodást.

A nulla eredményű íráshoz rendelt `EIO` a helper saját hibapolicy-ja: megakadályozza az előrehaladás nélküli végtelen ciklust.

12.1.3.2 Kidolgozott folytatási példa

Tegyük fel, hogy `read_some()` egy 4096 C bájtos bufferbe 1200 C bájtot olvas. Ez siker, nem EOF. A `write_all()` először 700-at továbbít, majd `EINTR` miatt visszatér:

```
/* Szemléltető eredmény, nem futtatási mérés. */
io_result result = {700, IO_INTR, EINTR};
```

A következő próbálkozás csak a maradék 500 C bájtot kaphatja:

```
/* Részlet: buffer él, out_fd kölcsönzött, újrapróbálás engedélyezett. */
io_result next = write_all(out_fd, buffer + 700, 1200 - 700);
```

A teljes 1200 ismételt elküldése adatduplikáció lenne. A buffer csak az összes hátralévő adat továbbítása vagy tudatos eldobása után használható új olvasáshoz. `IO_WAIT` esetén ugyanezt a maradékot kell megőrizni a későbbi folytatáshoz.

12.1.4 Megnyitás és a close hibapolicy-ja

A megnyitás szokásos mintája, POSIX-függvényen belüli részletként:

```
/* Szükséges headerek: <fcntl.h>, <errno.h>. */
int fd = open(path, O_RDONLY | O_CLOEXEC);
if (fd == -1)
    return errno;

/* fd tulajdonosa ez a függvény; I/O közben csak kölcsönadja. */
```

A hibás ellenőrzés, `fd <= 0`, elutasítaná a teljesen érvényes nulla descriptort is.

Linux-policy: a fenti `close_owned_linux()` egyszer próbál lezárni, és a tulajdonosi változót `-1` értékre állítja. A függvény kizárólag érvényes, saját tulajdonú FD-t vagy `-1` értéket fogadhat. Az `owner` pointer egy írható, élő `int` objektum kölcsönzése.

Linuxon a lezárás hibája — az eleve érvénytelen FD-t jelző `EBADF` esetétől eltekintve — nem jogosít fel ugyanazon szám újabb lezárására. A szám addigra más erőforráshoz tartozhat. Ez az `EINTR` esetre is vonatkozik.

Platform- és verzióhatár: ez nem univerzális POSIX-policy. Más rendszereken az `EINTR` utáni FD-állapot eltérhet; a célzott POSIX-kiadás és a tényleges libc/OS szerződését együtt kell ellenőrizni. Egy POSIX.1-2024 követelmény megnevezése önmagában nem bizonyítja a telepített Linux/libc viselkedését. Az `EINTR` kezelését ezért műveletenként, nem általános retry-makróval tervezzük.

A lezárási hibát nem szabad elhallgatni: korábbi írás késleltetett hibáját is jelezheti. A korábbi I/O-hibát azonban előbb el kell menteni; a cleanup hibája másodlagos diagnosztika legyen. A sikeres `write()` és `close()` sem általános tartósági garancia; ezt a fájlrendszer és tartósság tárgyalja.

12.1.5 Stdio és descriptor keverése

A `FILE` saját felhasználói pufferrel dolgozik. Emiatt a `fprintf()` után közvetlenül kiadott `write(fileno(stream), ...)` nem feltétlenül a kívánt sorrendben jelenik meg.

POSIX: az ugyanazon megnyitott fájlleírást használó streamek és descriptorok között az aktív kezelőre (*active handle*) vonatkozó szinkronizálási szabályokat kell követni. Kimeneti streamről descriptorra váltás előtt tipikusan ellenőrzött `fflush()` szükséges. Seekelhető bemenetnél a read-ahead puffer miatt a logikai streampozíció és a kernelpozíció eltérhet; például egy ellenőrzött `fseek(stream, 0, SEEK_CUR)` rendezheti a pozíciót a váltás előtt.

Nem seekelhető bemenetnél az előre beolvasott adat általában nem adható vissza a pipe-nak. Ilyenkor célszerű egyetlen I/O-rétegnél maradni. Az `fflush(stdin)` nem hordozható ISO C-megoldás.

A `fileno()` nem ad tulajdonjogot. A sikeres `fdopen()` után a lezárási felelősséget a streamhez kell rendelni: a `fclose()` mellett külön `close()` dupla lezárási hiba.

12.1.6 Feladatok

1. **Magyarázó:** egy 8192 C bájtos olvasási kérés 1500-at ad vissza. Mi bizonyított, és mi nem?
2. **Kódolási:** írj blocking `write_all_blocking()` wrappert, amely kizárólag `IO_INTR` után próbál újra, megőrzi az összesített előrehaladást, és nem pörög `IO_WAIT` esetén.
3. **Diagnosztikai:** Linuxon a program addig ismétli a `close(fd)` hívást, amíg az `EINTR` eltűnik. Közben másik szál fájlokat nyit. Mi a hiba, és hogyan javítanád?

12.1.7 Megoldások

12.1.7.1 Rövid olvasás értelmezése

A buffer első 1500 C bájtja érvényes új adat. Nem bizonyított az EOF, és nem bizonyított alkalmazásszintű üzenethatár sem. Ellenőrzési feltétel: a feldolgozó pontosan ezt az 1500-at fogyassza el; a buffer többi részét ne tekintse frissen olvasottnak.

12.1.7.2 Blocking wrapper

Az előző modulhoz hozzáadható részlet:

```
io_result write_all_blocking(int fd, const void *buffer, size_t length)
{
    const unsigned char *bytes = buffer;
    size_t total = 0;

    while (total < length) {
        io_result r = write_all(fd, bytes + total, length - total);
        total += r.done;

        if (r.state == IO_OK)
            return (io_result){total, IO_OK, 0};
        if (r.state != IO_INTR)
            return (io_result){total, r.state, r.error};
    }

    return (io_result){total, IO_OK, 0};
}
```

Az előfeltételek változatlanok; a hívó engedélyezi a megszakítás utáni újrapróbálást. Ez nem megfelelő policy, ha a signal leállítási kérés. Ellenőrzési feltétel: részleges írás, majd `EINTR` után minden adat pontosan egyszer szerepeljen a továbbított sorozatban; `IO_WAIT` azonnal térjen vissza.

12.1.7.3 Hibás close-retry

Az első hívás már felszabadíthatta az FD számát. Az ismételt hívás így a másik szál új fájlját zárhatja le. Javítás: egyszeri Linux-lezárás, a tulajdonosi változó érvénytelenítése, a hibakód rögzítése, újrapróbálás nélkül. Ellenőrzési feltétel: egy tulajdonlási életciklusból legfeljebb egy `close()` kísérlet induljon.

12.1.8 Összefoglaló

A robusztus I/O három külön adatot őriz: az előrehaladást, a megállás okát és az erőforrás tulajdonosát. A rövid művelet siker; az EOF, az átmeneti várakozás és a megszakítás külön állapot. Folytatáskor csak a maradék tartomány használható, lezáráskor pedig platformhoz kötött policy szükséges — nem általános retry-szabály.

12.2 Fájlrendszer-műveletek, mmap és tartósság

Egy konfigurációs fájl lecserélése három külön kérdést vet fel: melyik fájlobjektumot módosítjuk, mit látnak közben az olvasók, és mi marad meg rendszerösszeomlás után? A sikeres `write`, a memóriában látható módosítás és a tartós állapot nem ugyanaz.

A file descriptor és robusztus I/O ciklus tulajdonlási szabályaira építünk: minden megszerzett erőforráshoz lezárási felelősség tartozik. Az igazításról tanultakat most a memórialapokhoz igazított fájlleképezésre alkalmazzuk. Az alábbi rendszerhívások nem az ISO C részei; a POSIX-felületet és a Linux-specifikus feltételeket külön jelöljük.

12.2.1 Fájlazonosság, méret és directory-relative műveletek

POSIX: a `stat(path, &st)` útvonal alapján kér metaadatokat, és követi a végső szimbolikus linket. A `fstat(fd, &st)` a már megnyitott fájlhoz tartozó adatokat adja vissza. Szabályos fájlnál a `struct stat st_size` mezője a logikai méret, típusa `off_t`.

Az `off_t` előjeles egész típus; nem szabad automatikusan `long` vagy `size_t` helyett használni, illetve ezekre ellenőrzés nélkül konvertálni. A fájl lehet nagyobb a folyamatban egyetlen tartományként kezelhető memóriánál.

Linux/glibc: 32 bites célon a fordítás elejétől érvényes `_FILE_OFFSET_BITS=64` makró választja ki a nagyfájl-interfészeket. Ez nem ISO C- vagy általános POSIX-mechanizmus. Az x86-64 LP64 laborban az `off_t` eleve 64 bites; más libc és ABI esetén a szerződést külön ellenőrizni kell. A fordítási egységek és könyvtári határok beállításai legyenek következetesek.

12.2.1.1 TOCTOU és az útvonal rögzített kiindulópontja

A *time-of-check to time-of-use* (TOCTOU) hiba akkor keletkezik, amikor az ellenőrzés és a használat között megváltozhat az ellenőrzött állapot. Például a `stat(path)` után végzett `open(path)` már más fájlt nyithat meg.

Az `openat(dirfd, name, flags, ...)` relatív `name` esetén a `dirfd` által azonosított könyvtárból indul, nem a folyamat aktuális munkakönyvtárából. Ez stabil kiindulópont, de önmagában nem teljes útvonalbiztonsági megoldás:

- az abszolút útvonal figyelmen kívül hagyja a `dirfd` értékét;
- a `..` kijuthat a könyvtárból;
- az `O_NOFOLLOW` csak a végső útvonalkomponens szimbolikuslink-követését tiltja.

A későbbi példában ezért pontosan egy, nem üres komponens engedélyezett; a `.` és `..` tiltott. Először megnyitjuk a fájlt, majd ugyanazon descriptoron végzünk `fstat` ellenőrzést. Ez a fájlazonosság problémáját kezeli, de más folyamat későbbi írását vagy csonkítását nem akadályozza meg.

12.2.2 Fájlleképezés: határok és élettartam

POSIX: az `mmap` fájlrészhez rendel virtuális címtartományt. Sikertelenségkor `MAP_FAILED` értéket ad, nem null pointert. A leképezési hossz pozitív, az offset pedig a rendszer által megkövetelt lapméret többszöröse legyen. A lapméret a `sysconf(_SC_PAGESIZE)` hívással kérdezhető le; annak hibáját is kezelni kell.

A `MAP_SHARED` leképezés módosításai a fájl állapotába kerülhetnek. A `MAP_PRIVATE` módosításai nem írják át a fájlt, és `msync` sem teszi őket fájlmódosítássá. A `MAP_PRIVATE` ugyanakkor **nem garantál változatlan pillanatképet** egy más által közben módosított fájlról.

Erőforrás	Tulajdonos	Érvényesség és megszüntetés
Könyvtár-FD	megnyitó kódrész	`close` lezárja a directory-relative kiindulópontot
Fájl-FD	megnyitó kódrész	`close` lezárja ezt a descriptort
Mapping címtartománya	`mmap` hívója	`munmap` megszünteti; a fájl-FD lezárása nem
Mappingre mutató pointer	kölcsönzött hozzáférés	csak a leképezés és a hozzáférhető fájltartomány fennállásáig használható

12.2.2.1 Tetszőleges fájlrész leképezése

Legyen a kívánt kezdőoffset $s \geq 0$, a hossz $L > 0$, a lapméret $P > 0$. A leképezés kezdete és hossza:

$$a = s - (s \bmod P), \qquad d = s - a, \qquad n = d + L.$$

Az `mmap` offsetje a, hossza n; a kívánt nézet a visszakapott címhez képest d bájtnál kezdődik. A hossz nem szükségképpen laptöbbszörös.

Bizonyításvázlat. Az a osztható P-vel, továbbá $0 \leq d < P$. A kívánt rész a mappingen belül a $[d, d + L) = [d, n)$ tartomány, tehát annak végén nem lép túl. C-ben az összeadás előtt szükséges az $L \leq \texttt{SIZE_MAX} - d$ ellenőrzés, valamint a típuskonverziók érvényességének igazolása. Ha a fájlméret S, akkor $s \leq S$ és $L \leq S - s$ ellenőrzendő; ez nem igényli a potenciálisan túlcsorduló $s + L$ kiszámítását.

12.2.2.2 Kidolgozott példa: nullák számlálása

A program egy megbízható könyvtárútvonalból indul, és egyetlen fájlnévkomponenst fogad. **Előfeltétel:** a megnyitott fájlt a vizsgálat alatt senki nem módosítja vagy csonkítja. Ezt a program nem kényszeríti ki.

A példa ISO C17 nyelvet és POSIX API-kat használ. Választott laborprofil: Linux 6.8, x86-64 System V ABI, LP64, 8 bites C bájt; fordítóként a `lab-next` profil (profiltábla). Ezek reprodukciós célverziók; a példához itt nem tartozik dokumentált fordítási vagy futtatási eredmény.

```c
#define _POSIX_C_SOURCE 200809L

#include <fcntl.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc != 3 || argv[2][0] == '\0' ||
        strchr(argv[2], '/') != NULL ||
        strcmp(argv[2], ".") == 0 ||
        strcmp(argv[2], "..") == 0) {
        fprintf(stderr, "Hasznalat: mapzeros KONYVTAR FAJLNEV\n");
        return EXIT_FAILURE;
    }

    int dirfd = -1, fd = -1;
    int result = EXIT_FAILURE;
    void *mapping = MAP_FAILED;
    size_t length = 0, zeros = 0;
    struct stat st;

    dirfd = open(argv[1], O_RDONLY | O_DIRECTORY | O_CLOEXEC);
    if (dirfd == -1) {
        perror("open directory");
        goto cleanup;
    }

    fd = openat(dirfd, argv[2], O_RDONLY | O_NOFOLLOW | O_CLOEXEC);
    if (fd == -1) {
        perror("openat");
        goto cleanup;
    }

    if (fstat(fd, &st) == -1) {
        perror("fstat");
        goto cleanup;
    }

    if (!S_ISREG(st.st_mode) || st.st_size < 0) {
        fprintf(stderr, "Nem megfelelo fajltipus vagy meret\n");
        goto cleanup;
    }

    if ((uintmax_t)st.st_size > (uintmax_t)SIZE_MAX ||
        (uintmax_t)st.st_size > (uintmax_t)PTRDIFF_MAX) {
        fprintf(stderr, "A fajlhoz ablakos feldolgozas szukseges\n");
```

```
        goto cleanup;
    }

    length = (size_t)st.st_size;
    if (length != 0) {
        mapping = mmap(NULL, length, PROT_READ, MAP_PRIVATE, fd, 0);
        if (mapping == MAP_FAILED) {
            perror("mmap");
            goto cleanup;
        }

        const unsigned char *bytes = mapping;
        for (size_t i = 0; i < length; ++i) {
            if (bytes[i] == 0)
                ++zeros;
        }
    }

    if (printf("meret=%zu, nullak=%zu\n", length, zeros) < 0)
        goto cleanup;

    result = EXIT_SUCCESS;

cleanup:
    if (mapping != MAP_FAILED && munmap(mapping, length) == -1) {
        perror("munmap");
        result = EXIT_FAILURE;
    }
    if (fd != -1 && close(fd) == -1) {
        perror("close file");
        result = EXIT_FAILURE;
    }
    if (dirfd != -1 && close(dirfd) == -1) {
        perror("close directory");
        result = EXIT_FAILURE;
    }
    return result;
}
```

Fordítás külön laborlépésként:

```
gcc-14 -std=c17 -D_FILE_OFFSET_BITS=64 -O2 \
  -Wall -Wextra -Wpedantic -Wconversion mapzeros.c -o mapzeros-gcc

clang-19 -std=c17 -D_FILE_OFFSET_BITS=64 -O2 \
  -Wall -Wextra -Wpedantic -Wconversion mapzeros.c -o mapzeros-clang
```

A verziózott parancsnevek telepítésfüggők; a tényleges fordítókiadást rögzíteni kell. A `PTRDIFF_MAX` korlát konzervatív programbeli döntés, nem általános `mmap`-méréthatár.

Az üres fájlt külön ág kezeli, mert nulla hosszúságú mapping nem készíthető. A ciklus csak a $[0, \texttt{length})$ tartományt olvassa. A borrowed `bytes` pointert a `munmap` érvényteleníti. A Linux-profilnak megfelelően a program a sikertelen `close` hívást jelenti, de nem ismétli meg vakon.

12.2.3 Sparse fájl, csonkítás és SIGBUS

A *sparse* fájl logikai méretéhez nem feltétlenül tartozik ugyanannyi fizikailag lefoglalt tároló. A lyukak olvasáskor nullákat szolgáltatnak. A példaprogram nullaszámlálása ezért nem bizonyítja a sparse tárolást: fizikailag tárolt nullák ugyanazt az eredményt adják.

POSIX/Linux: az `ftruncate` módosítja a fájl logikai méretét. Növeléskor az új tartomány nullaként olvasható, de ebből nem következik előzetes fizikai tárhelyfoglalás. A már létező mapping hossza nem növekszik automatikusan.

Csonkításkor a mapping címtartománya megmaradhat, miközben a mögöttes fájlrész eltűnik. Linuxon a fájlvégen teljesen túlra került laphoz való hozzáférés `SIGBUS` jelzést okozhat. Az utolsó részleges lap nullakitöltése sem jogosít fel a program szerződésében rögzített fájlméreten túli olvasásra.

Ez különösen fontos hibakimenet: a sikeres `mmap` nem garantálja, hogy minden későbbi memóriahozzáférés sikerül. Egy újabb `fstat` sem oldja meg a versenyhelyzetet, mert utána ismét történhet csonkítás. Megoldás lehet az együttműködő kizárás, vagy a publikált fájlok módosításának tiltása és új fájl rename-alapú publikálása.

12.2.4 Visszaírás, atomikus csere és tartósság

A kernel pufferében jelen levő adat még nem feltétlenül tartós. A fontos műveletek eltérő hatókörűek:

Művelet	Szerződésének lényegi része
`msync(..., MS_SYNC)`	a megosztott mapping módosításainak szinkron visszaírása
`fsync(fd)`	fájladatok és hozzájuk tartozó metaadatok szinkronizálása
`fdatasync(fd)`	adatok és későbbi elérésükhöz szükséges metaadatok szinkronizálása
Könyvtár-FD `fsync`	Linuxon, támogató fájlrendszeren a névtérmódosítás tartóssági lépése

A fájlméret a tartalom eléréséhez szükséges metaadat, tehát a `fdatasync` nem hagyhatja figyelmen kívül pusztán azért, mert metaadat. Az `msync` kezdőcímének lapra igazítottnak kell lennie; eltolt borrowed view helyett a megfelelő mappingtartományt kell megadni. Egyik fájlszinkronizáció sem helyettesíti automatikusan a könyvtárbejegyzés tartósítását.

12.2.4.1 Tartós konfigurációcsere

POSIX: azonos fájlrendszeren a sikeres `renameat` atomikusan cserélheti le a célnevet. A név szerinti olvasók nem félkész fájlt kapnak; a korábban megnyitott descriptorok továbbra is a régi fájlobjektumra hivatkoznak. Az atomikus névcsere azonban nem összeomlás utáni tartóssági garancia.

Linux-laborminta, azonos, megbízható könyvtárban, konkurens publikáló nélkül:

1. Ideiglenes fájl létrehozása `openat` segítségével, `O_CREAT | O_EXCL | O_WRONLY | O_CLOEXEC` zászlókkal.
2. Teljes tartalom kiírása robusztus ciklussal; a szükséges jogosultságok beállítása.

3. Az ideiglenes fájlon `fsync`; hiba esetén nincs publikálás.
4. `renameat` az ideiglenes névről a célra.
5. `fsync` a tartalmazó könyvtáron.
6. A descriptorok lezárása, a hibák jelentése.

A sorrend indoka: először az új fájl állapotát, utána az arra mutató névcsere állapotát tesszük tartóssá. Eltérő könyvtárak közötti mozgatásnál mindkét könyvtár érintett lehet.

Ha a rename sikerül, de a könyvtárszinkronizáció hibázik, az új állapot már látható: az eredmény **publikálva, tartóssága bizonytalan**, nem egyszerűen „nem történt semmi”. A garancia a fájlrendszer és a tárolási réteg helyesen teljesített szinkronizációs szerződésétől is függ.

12.2.5 Feladatok

1. **Magyarázó:** $P = 4096$, $s = 5000$, $L = 3000$ mellett számítsd ki a mapping offsetjét, hosszát és a borrowed view eltolását.
2. **Kódolási:** alakítsd át a példaprogramot úgy, hogy legfeljebb az első 4096 bájtot vizsgálja. Üres fájlra se hívjon `mmap`-et.
3. **Diagnosztikai:** egy szolgáltatás felülírja az aktív fájlt, majd rename-et végez egy másik fájllal. Egy olvasó `SIGBUS` miatt leáll, áramszünet után pedig az új név sem megbízható. Azonosíts két független szerződéshibát.

12.2.6 Megoldások

12.2.6.1 Igazított tartomány

$a = 4096$, $d = 904$, $n = 3904$. Ellenőrzés: $a + d = 5000$, és $d + L = n$. A mapping hossza nem laptöbbszörös, ami megengedett.

12.2.6.2 Korlátozott vizsgálat

A nemnegatív fájlméret ellenőrzése után, még a konverzió előtt:

```
/* C17/POSIX részlet a fenti main függvényhez.
   A teljes fájl méretkorlát-ellenőrzését és konverzióját helyettesíti. */
length = (uintmax_t)st.st_size > (uintmax_t)4096
       ? (size_t)4096
       : (size_t)st.st_size;
```

A meglévő nemnulla-hossz ág és cleanup megtartható. Ellenőrzési esetek: 0, 1, 4096 és 4097 bájtos fájl. A tartalom stabilitásának előfeltétele változatlan.

12.2.6.3 Két külön hibamodell

Az aktív fájl csonkítása érvényteleníthetí az olvasó által elérhető backing tartományt. Új fájlt kell írni, a publikált fájlt pedig változatlanul hagyni.

A rename önmagában nem tartósítás: előtte fájl-, utána könyvtárszinkronizáció szükséges. Ellenőrzéskor minden lépésnél vizsgálni kell a hibakimenetet. Egy folyamat kilövése önmagában nem modellezi az áramvesztést, mert a kernel még visszaírhatja a puffereket.

12.2.7 Rövid összefoglaló

A fájlazonosságot descriptorral, a mapping használhatóságát határ- és élettartam-szerződéssel, a tartósságot explicit szinkronizációval kezeljük. A mapping nem pillanatkép, a rename nem tartóssági művelet, a sikeres írás pedig nem bizonyítja az összeomlás utáni megmaradást.

12.3 fork, exec, pipe és gyermekfolyamat

A `fork` nem egyszerű függvényhívás egy másik végrehajtási környezetbe: siker esetén a hívás után két folyamat folytatja ugyanazt a programot. Ettől kezdve külön kell követnünk a memóriát, a file descriptorok tulajdonlását és a folyamatok lezárását. Az `execve` pedig nem új folyamatot hoz létre, hanem a hívó folyamat programképét cseréli le.

Az FD-kezelés, a leállási szabályok és a többszintű pointerek ismeretére építünk. Az itt használt folyamatkezelő függvények **POSIX API-k**, nem az ISO C standard könyvtárának részei.

12.3.1 A fork három kimenete

A `fork()` visszatérési típusa `pid_t`. A visszatérési értéket először előjel szerint kell megkülönböztetni:

Visszatérési érték	Hol folytatódik a végrehajtás?	Következmény
`-1`	Csak a szülőben	Nem jött létre gyermek; az `errno` jelzi a hibát
`0`	A gyermekben	A gyermek saját végrehajtási ágába lép
Pozitív érték	A szülőben	Az érték az új gyermek folyamatazonosítója

A két folyamat ütemezési sorrendje nem garantált. Attól, hogy a szülői ág előbb szerepel a forrásban, még végrehajtódhat előbb a gyermek.

POSIX-szemantika: a gyermek a szülő memóriájának másolatával indul, a kifejezetten megosztott leképezések kivételével. Egy örökölt pointer ugyanazt a virtuális címet jelölheti, de a szokásos privát heap- és stackterületeken már a saját folyamat másolatára mutat. A gyermek `free(p)` hívása ezért nem szabadítja fel a szülő példányát.

Linux-megvalósítás: a privát memória másolását jellemzően *copy-on-write* késlelteti. Kezdetben több leképezés hivatkozhat ugyanarra a fizikai oldalra; íráskor válik szükségessé a megfelelő elkülönítés. Ez implementációs mechanizmus, nem engedély folyamatok közötti pointerhasználatra.

12.3.1.1 A copy-on-write megfigyelése

Egy külön kísérletben foglaljunk nagy privát területet, és még a `fork` előtt érintsük meg az oldalait. A gyermek először várakozzon, majd szinkronizált lépésben módosítson oldalanként egy bájtot. Linuxon a két folyamat `/proc/<pid>/smaps` adatai közül a `Pss` és a `Private_Dirty` változása segíthet értelmezni a megosztás megszűnését.

Ez mérési terv, nem rögzített eredmény: az oldalnagyság, a huge page használata és az egyéb memóriaírások befolyásolják a megfigyelést. Az RSS önmagában nem megbízható bizonyíték, mert a megosztott rezidens oldalak is beleszámíthatnak.

12.3.2 Az örökölt FD és az exec szerződése

A descriptorok táblája is másolódik, de a megfelelő FD-k ugyanarra a kernelbeli *open file description*-ionre hivatkoznak. Emiatt például a fájlpozíció és az olyan állapotjelzők, mint az `O_NONBLOCK`, közösek lehetnek. Az egyik folyamat `close` hívása csak a saját descriptorát zárja le.

Az `execve(path, argv, envp)` siker esetén nem tér vissza. A PID megmarad, de a korábbi program memóriaképe lecserélődik. Nem futnak le a régi program `atexit` kezelői.

Az argumentumok szerződése:

- `path` nullterminált elérési út; az `execve` nem keres a `PATH` alapján;
- `argv` null pointerrel lezárt pointertömb, elemei nullterminált sztringek;
- az `argv[0]` szokásosan a program neve;
- `envp` hasonló tömb, tipikusan `NÉV=érték` sztringekkel.

A tömböknek és sztringjeiknek a hívás alatt érvényesnek kell maradniuk. Siker után nem marad rájuk kölcsönzött hivatkozás a régi címtérben. Sikertelen `execve` esetén `-1` az eredmény, és a gyermek saját `errno` értéke írja le a hibát.

A **close-on-exec**, vagyis `FD_CLOEXEC` descriptorjelző sikeres `exec` során lezáratja az adott FD-t. A `fork` nem zárja le, a sikertelen `execve` pedig nem aktiválja ezt a lezárást.

12.3.3 Pipe, dup2 és lezárási sorrend

A `pipe(p)` két descriptort ad: `p[0]` az olvasási, `p[1]` az írási vég. Ha a gyermek standard kimenetét szeretnénk olvasni, a gyermekben a

```
/* C17 + POSIX; részlet, p[1] érvényes pipe-íróvég. */
dup2(p[1], STDOUT_FILENO);
```

művelet a `STDOUT_FILENO` descriptorhoz rendeli az íróvéget. A `dup2` szükség esetén lecseréli a cél korábbi descriptorát; a külön `close(1); dup(...)` megoldás nem egyenértékű, mert közben más művelet elfoglalhatja az 1-es helyet.

Ha a két FD különböző, az új célon a `FD_CLOEXEC` törlődik. Ha azonosak, a `dup2` nem módosítja a jelzőt sem. Ezért fontos előfeltétel az alábbi példában, hogy a standard descriptorok már nyitva legyenek.

A beállítások után ezt az ownership-térképet akarjuk:

Erőforrás	Szülő	Gyermek
`p[0]`	Tulajdonolja, olvasás után lezárja	Lezárja
Eredeti `p[1]`	Rögtön lezárja	A `dup2` után lezárja
Gyermek `STDOUT_FILENO`	Nincs ilyen saját descriptor	A pipe íróvégére hivatkozik
Gyermek kilépési állapota	`waitpid` hívással begyűjti	Kilépéssel rendelkezésre bocsátja

Az EOF feltétele: a pipe kiürült, és már egyetlen folyamatnak sincs nyitott íróvége. Ha a szülő megtartja a saját `p[1]` descriptorát, akkor maga akadályozza meg az EOF-ot.

12.3.4 Kidolgozott példa: gyermek kimenetének átvétele

A példa `/bin/echo` futtatásával előállított adatot továbbítja a szülő standard kimenetére.

Profil: ISO C17, POSIX.1-2008 API-k, Linux/x86-64 System V, LP64. Választott laborverziók: a `lab-next` profil (profiltábla). Feltételezzük, hogy `/bin/echo` létezik; a program egyszálú; a 0, 1, 2 descriptor nyitott és blokkoló; nincs más gyermekbegyűjtő, és a `SIGCHLD` kezelése alapértelmezett. A standard kimenet legyen terminál vagy közönséges fájl.

A `process_pipe.c` teljes tartalma:

```c
#define _POSIX_C_SOURCE 200809L

#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

/* Borrow: buf legalább n olvasható bájt; itt n legfeljebb 4096. */
static int write_all(int fd, const char *buf, size_t n)
{
    while (n != 0) {
        ssize_t k = write(fd, buf, n);
        if (k > 0) {
            buf += (size_t)k;
            n -= (size_t)k;
        } else if (k == -1 && errno == EINTR) {
            continue;
        } else {
            if (k == 0)
                errno = EIO;
            return -1;
        }
    }
    return 0;
}

static _Noreturn void child_fail(int code)
{
    static const char msg[] = "child: setup or exec failed\n";
    /* A diagnosztika hibája nem változtatja meg a kilépési szándékot. */
    (void)write_all(STDERR_FILENO, msg, sizeof msg - 1);
    _exit(code);
}

int main(void)
{
    int p[2];

    if (fflush(NULL) == EOF) {
```

```
        perror("fflush");
        return EXIT_FAILURE;
    }
    if (pipe(p) == -1) {
        perror("pipe");
        return EXIT_FAILURE;
    }

    /* Új pipe-descriptorokon a kezdeti FD_CLOEXEC törölt. */
    if (fcntl(p[0], F_SETFD, FD_CLOEXEC) == -1 ||
        fcntl(p[1], F_SETFD, FD_CLOEXEC) == -1) {
        perror("fcntl");
        (void)close(p[0]);
        (void)close(p[1]);
        return EXIT_FAILURE;
    }

    pid_t pid = fork();
    if (pid == -1) {
        perror("fork");
        (void)close(p[0]);
        (void)close(p[1]);
        return EXIT_FAILURE;
    }

    if (pid == 0) {
        char arg0[] = "echo";
        char arg1[] = "gyermek";
        char env0[] = "LC_ALL=C";
        char *argv[] = {arg0, arg1, NULL};
        char *envp[] = {env0, NULL};

        (void)close(p[0]);
        if (dup2(p[1], STDOUT_FILENO) == -1)
            child_fail(126);
        (void)close(p[1]);

        execve("/bin/echo", argv, envp);
        child_fail(127);
    }

    (void)close(p[1]);

    char buf[4096];
    int io_failed = 0;

    for (;;) {
        ssize_t n = read(p[0], buf, sizeof buf);
        if (n > 0) {
            if (write_all(STDOUT_FILENO, buf, (size_t)n) == -1) {
                perror("write");
                io_failed = 1;
```

```
                break;
            }
        } else if (n == 0) {
            break;
        } else if (errno != EINTR) {
            perror("read");
            io_failed = 1;
            break;
        }
    }
    (void)close(p[0]);

    int status;
    pid_t result;
    do {
        result = waitpid(pid, &status, 0);
    } while (result == -1 && errno == EINTR);

    if (result == -1) {
        perror("waitpid");
        return EXIT_FAILURE;
    }

    if (WIFEXITED(status)) {
        int code = WEXITSTATUS(status);
        fprintf(stderr, "exit=%d\n", code);
        return (!io_failed && code == 0)
            ? EXIT_SUCCESS : EXIT_FAILURE;
    }

    if (WIFSIGNALED(status))
        fprintf(stderr, "signal=%d\n", WTERMSIG(status));

    return EXIT_FAILURE;
}
```

Fordítás a megjelölt fordítókiadásokkal:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 \
    process_pipe.c -o process_pipe_gcc

clang -std=c17 -Wall -Wextra -Wpedantic -O2 \
    process_pipe.c -o process_pipe_clang
```

Ezek laborparancsok, nem dokumentált fordítási eredmények.

A `buf` tulajdonosa a szülő `main` függvénye; a `write_all` csak a hívás idejére kölcsönzi, nem tárolja el a pointert. Az argumentumtömbök a gyermek lokális objektumai: sikeres `execve` megszünteti a régi memóriaképet, sikertelenségkor az `_exit` lezárja a folyamatot.

A szülő előbb olvas, és csak utána vár. Fordított sorrendben nagyobb kimenetnél holtpont keletkezhet: a gyermek a megtelt pipe-ra várna, a szülő pedig a gyermek kilépésére.

12.3.5 Leállás, pufferek és státuszok

A `fork` a felhasználói térben tárolt `stdio` puffereket is másolja. Ha mindkét folyamat `exit` útján áll le, ugyanazt a korábban pufferelt kimenetet mindketten kiírhatják.

Az egyszálú példában ezért történik ellenőrzött `fflush(NULL)` a `fork` előtt. A gyermek hibakimenetén pedig `_exit` szerepel: ez nem üríti a C streameket, és nem futtatja az örökölt `atexit` kezelőket.

A `waitpid(pid, &status, 0)`:

- a megadott gyermekre vár;
- megszakadhat `EINTR` hibával;
- siker esetén begyűjti a befejeződött gyermek állapotát.

A `status` **nem közvetlen exit code**. Csak `WIFEXITED(status)` igaz értéke után használható a `WEXITSTATUS`. Jel általi megszűnésnél a megfelelő pár a `WIFSIGNALED` és `WTERMSIG`. A megállítás és folytatás külön várakozási opciókat és további makrókat igényel.

A 126 és 127 itt választott hibajelzés. A 127 önmagában nem bizonyít sikertelen `execve` hívást: a sikeresen elindított program is visszaadhatja.

12.3.6 Többszálú fork és a posix_spawn alternatívája

Többszálú folyamat `fork` hívása után a gyermekben csak a hívó szál marad meg. A memória azonban tartalmazhat más szál által zárolt mutexet vagy könyvtári belső állapotot. A zároló szál már nincs jelen, így a gyermek könnyen végleg elakadhat.

POSIX-korlát: a gyermekben a sikeres `exec` előtt csak az *async-signal-safe* követelményeknek megfelelő műveleteket szabad végezni. A `printf`, `perror`, `malloc`, `free` és `exit` nem megfelelő általános megoldás. A szűk előkészítési út tipikusan `close`, `dup2`, `write`, `execve`, hiba esetén `_exit`. Ez nem pusztán teljesítményajánlás, hanem helyességi korlát.

A példában a `pipe` és az utólagos `fcntl` közötti időablak egyszálú környezetben elfogadható. Többszálú Linux-programban egy másik szál közben gyermeket indíthatna; az atomi close-on-exec létrehozásra használható `pipe2(..., O_CLOEXEC)` külön platform- és libc-támogatási ellenőrzést igényel.

Ha a cél csak programindítás rendezett FD-átirányítással, gyakran jobb a `posix_spawn`: fájlműveleti leírókkal adható meg a `dup2` és `close`, így kevesebb saját gyermekoldali kód szükséges. Hibajelzése közvetlen hibaszám, nem automatikusan `errno`; a gyermeket siker esetén továbbra is be kell gyűjteni. Bizonyos gyermekoldali indítási hibák 127-es kilépésként jelentkezhetnek.

12.3.7 Feladatok

1. **Magyarázat:** miért nem változik a szülő privát változója, ha a gyermek átírja, miközben egy örökölt fájl olvasása mégis módosíthat közös állapotot?
2. **Kódolás:** egészítsd ki a példát külön hibapipe-pal, amelyen a gyermek továbbítja az `execve` hibájának `errno` értékét. Melyik végnek kell close-on-exec jelző?
3. **Diagnosztika:** egy változatban a szülő nem zárja le `p[1]`-et, és olvasás előtt hívja a `waitpid`-et. Mutasd meg a két, egymástól független elakadási okot!

12.3.8 Megoldások

12.3.8.1 Privát memória és közös kernelerőforrás

A privát memóriát a folyamatok szemantikailag külön birtokolják; a copy-on-write ezt csak optimalizálja. Az örökölt FD-k viszont közös open file descriptionre hivatkozhatnak, így például a fájlpozíció közös.

Ellenőrzés: külön vizsgáld a memóriában tárolt értéket és a következő olvasás fájlpozícióját; ezek eltérő erőforrásrétegek.

12.3.8.2 Külön exec-hibacsatorna

A gyermek hibapipe-íróvége legyen `FD_CLOEXEC`. Sikeres `execve` lezárja; sikertelenségkor a gyermek azonnal mentse az `errno` értékét, továbbítsa rövid írásokat és `EINTR`-t kezelve, majd hívjon `_exit(127)`-et. A szülő a saját íróvégét zárja le, és teljes rekordot vagy EOF-ot olvasson.

Ellenőrzés: nem létező programútvonalnál teljes hibarekord várható. A rekord nélküli EOF önmagában nem abszolút sikerbizonyíték: a gyermek a jelentés előtt is megszűnhet. A várakozási státusz szintén szükséges.

12.3.8.3 Két külön elakadás

A megőrzött szülői íróvég miatt nincs EOF. Ettől függetlenül az olvasás előtti várakozás megtelt pipe-nál holtpontot okozhat.

Javítás: a szülő rögtön zárja le az íróvéget, ürítse a csatornát, végül gyűjtse be a gyermeket. Az ellenőrzéshez kis és a pipe kapacitásánál nagyobb kimenetet is használj.

12.3.9 Összefoglaló

A helyes folyamatindítás három külön életciklust rendez: a memóriaképét, az örökölt FD-két és a gyermek begyűjtését. A sikeres `exec` képcsere, az EOF descriptor-tulajdonlási következmény, a kilépési státusz pedig külön dekódolandó eredmény. Többszálú szülő esetén a gyermek előkészítési útját különösen szűkre kell szabni.

12.4 Signal, megszakítás és graceful shutdown

A signalkezelő nem egyszerűen egy másik callback: olyan ponton szakíthatja meg a programot, ahol egy könyvtári művelet belső állapota még átmeneti. Ezért a szabályos leállítás alapelve nem az, hogy „a handler mindent felszabadít”, hanem az, hogy **a handler leállítási szándékot jelez, a normál vezérlési út pedig végrehajtja a leállítás protokollját**.

A fork, exec, pipe és gyermekfolyamat részben megismert erőforrás-életciklus itt is érvényes: minden erőforrásnak legyen lezárásért felelős tulajdonosa. Az `errno` szabályai pedig különösen fontossá válnak, mert a signalkezelő a megszakított kód hibajelzését is elronthatja.

12.4.1 Signalállapot és kézbesítés

ISO C17: a `<signal.h>` korlátozott signalmodellt biztosít, többek között a `signal`, `raise` és `sig_atomic_t` eszközökkel. A POSIX szolgáltatások signalkezeléséhez ennél részletesebb interfész szükséges.

POSIX: egy signalhoz tartozó **disposition**, vagyis kezelési beállítás lehet alapértelmezett művelet, figyelmen kívül hagyás vagy saját handler. Ezt célszerű `sigaction` segítségével beállítani, amely a handler mellett maszkolási és működési szabályokat is megad.

A signal **blokkolása** nem figyelmen kívül hagyást jelent. A blokkolt signal függőben maradhat, majd a blokkolás megszüntetése után kézbesíthető. A hagyományos, nem valós idejű signalok nem megbízható eseményszámlálók: több azonos, függőben lévő jelzés összeolvadhat.

POSIX folyamat- és szálmodell:

- A disposition a folyamat szálai között közös.
- A signalmaszk szálankénti állapot.
- A folyamatnak címzett signal egy arra jogosult, nem blokkoló szálhoz kerülhet.
- A szálnak címzett signal az adott szálhoz tartozik.

Egyszálú programban a maszkot `sigprocmask` kezeli. Többszálú programban erre `pthread_sigmask` használatos; ott a `sigprocmask` viselkedésére nem építünk. A `SIGKILL` és `SIGSTOP` nem fogható el, nem ignorálható és nem blokkolható.

12.4.2 Mit szabad a handlerben?

Az **aszinkron signalbiztonság** (*async-signal-safety*) azt jelenti, hogy egy művelet akkor is használható handlerből, amikor egy signal a normál végrehajtást tetszőleges ponton szakította meg. Ez erősebb követelmény, mint a szálbiztosság: egy belső mutex megvédhet több száltól, de holtpontot okozhat, ha a handler ugyanazon a szálon újra megszerezné.

Az alábbi részlet **hibás tanítási példa**, nem végrehajtási minta:

```c
/* ISO C17 / POSIX: hibás, aszinkron handlerbe nem való műveletek. */
static void bad_handler(int signo)
{
    printf("signal: %d\n", signo);
    void *temporary = malloc(128);
    free(temporary);
}
```

POSIX: a `printf`, `malloc` és `free` nem async-signal-safe. A megszakítás történhet éppen a stdio-puffer vagy az allocator belső adatszerkezetének módosítása közben. Az eredmény lehet holtpont vagy sérült állapot; nincs garantált kimenet.

Ezzel szemben a `write` POSIX szerint async-signal-safe, de ettől még blokkolhat. A signalbiztonság tehát nem időkorlát és nem általános helyességi garancia.

12.4.2.1 A korlátozott jelzőminta

A legegyszerűbb szabályos megoldás:

```
/* ISO C17; a teljes POSIX-példa ugyanezt használja. */
static volatile sig_atomic_t stop_requested = 0;

static void request_stop(int signo)
{
    (void)signo;
    stop_requested = 1;
}
```

A `sig_atomic_t` olyan egész típus, amelynek hozzáférése aszinkron megszakítás mellett is atomi egységként kezelhető. A `volatile` biztosítja a jelző megfelelő megfigyelhetőségét ebben a mintában; **nem** teszi általánosan szálbiztossá a programot, és nem publikál más adatokat.

A szerződés szándékosan szűk:

- a handler csak állandó értéket rendel a jelzőhöz;
- nincs `++`, összetett állapotmódosítás vagy megosztott struktúra;
- a jelző nem signalok darabszámát tárolja;
- nem használjuk szálak közötti szinkronizáció helyett.

Ez a handler nem módosítja az `errno` értékét. Ha egy POSIX-handler olyan megengedett függvényt hív, amely módosíthatja, belépéskor el kell mentenie, visszatérés előtt pedig helyreállítania.

12.4.3 Megszakítás és újrapróbálás

POSIX: a `sigaction` `SA_RESTART` jelzője bizonyos blokkoló műveletek automatikus újraindítását kéri a handler visszatérése után. Nem minden rendszerhívásra érvényes; az egyes interfészek és a platform szabályait külön kell vizsgálni.

Leállításnál az automatikus újraindítás hátrány is lehet: a handler beállítja a jelzőt, de a program visszatér a blokkoló várakozásba. A `SA_RESTART` elhagyása önmagában szintén nem garantálja, hogy minden várakozás megszakad.

Az `EINTR` kezelése ezért alkalmazási policy:

1. Ha a művelet pozitív elemszámmal tért vissza, a haladást először könyveljük el.
2. Ha `-1` az eredmény és `errno == EINTR`, megvizsgáljuk a leállítási állapotot.
3. Csak akkor próbálkozunk újra, ha ezt az aktuális állapot megengedi.

A „minden `EINTR` után feltétel nélkül újra” ciklus könnyen leállíthatatlan szolgáltatást eredményez.

12.4.3.1 A jelzőellenőrzés és az elalvás közötti rés

Ez a részlet **hibás szinkronizációs minta**:

```
/* POSIX-részlet: elveszett ébresztés lehetséges. */
while (!stop_requested) {
    pause();
}
```

Ha a signal az ellenőrzés után, de a `pause` előtt érkezik, a jelző beáll, majd a program mégis elalszik.

POSIX: a `pselect` a várakozás idejére atomian cseréli a hívó szál signalmaszkját. Így a jelző ellenőrzése alatt blokkolhatjuk a leállító signalokat, és csak a várakozás megkezdésével együtt engedélyezzük őket.

12.4.4 Kidolgozott példa: minimális handler, normál cleanup

A következő program az ébresztési és tulajdonlási mintát mutatja, nem teljes szolgáltatás. A `tmpfile` egy helyettesítő alkalmazási erőforrás; a program leállításkor normál kódból ír bele és zárja le.

Profil: ISO C17 és POSIX interfészek, Linux hosted környezet. Választott fordítási célprofil: x86-64 System V ABI, LP64, a `lab-next` profil (profiltábla). A példa nem közöl ellenőrzött fordítási vagy futtatási eredményt.

```
#define _POSIX_C_SOURCE 200809L

#include <errno.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/select.h>

static volatile sig_atomic_t stop_requested = 0;

static void request_stop(int signo)
{
    (void)signo;
    stop_requested = 1;
}

int main(void)
{
    sigset_t stop_set;
    sigset_t old_mask;
    sigset_t wait_mask;

    sigemptyset(&stop_set);
    sigaddset(&stop_set, SIGINT);
    sigaddset(&stop_set, SIGTERM);

    if (sigprocmask(SIG_BLOCK, &stop_set, &old_mask) == -1) {
        perror("sigprocmask");
        return EXIT_FAILURE;
    }

    struct sigaction action = {0};
    action.sa_handler = request_stop;
    action.sa_flags = 0;
    sigemptyset(&action.sa_mask);
```

```
    sigaddset(&action.sa_mask, SIGINT);
    sigaddset(&action.sa_mask, SIGTERM);

    if (sigaction(SIGINT, &action, NULL) == -1 ||
        sigaction(SIGTERM, &action, NULL) == -1) {
        perror("sigaction");
        return EXIT_FAILURE;
    }

    wait_mask = old_mask;
    sigdelset(&wait_mask, SIGINT);
    sigdelset(&wait_mask, SIGTERM);

    FILE *journal = tmpfile();
    if (journal == NULL) {
        perror("tmpfile");
        return EXIT_FAILURE;
    }

    int failed = 0;

    while (!stop_requested) {
        int result = pselect(0, NULL, NULL, NULL, NULL, &wait_mask);

        if (result == -1 && errno != EINTR) {
            perror("pselect");
            failed = 1;
            break;
        }
    }

    /* Normál vezérlési út: itt már használható stdio. */
    if (fputs("shutdown\n", journal) == EOF) {
        perror("fputs");
        failed = 1;
    }
    if (fclose(journal) == EOF) {
        perror("fclose");
        failed = 1;
    }

    return failed ? EXIT_FAILURE : EXIT_SUCCESS;
}
```

Fordítás a kiválasztott kiadásokkal:

```
gcc -std=c17 -O2 -Wall -Wextra -Wpedantic shutdown_wait.c -o shutdown_wait_gcc
clang -std=c17 -O2 -Wall -Wextra -Wpedantic shutdown_wait.c -o
 ↪ shutdown_wait_clang
```

A parancsok a tanuló külön laborjához tartoznak. Ellenőrzési feltétel: a futó programnak küldött `SIGTERM` után a várakozás véget ér, és sikeres cleanup esetén a kilépési státusz sikeres.

Objektum vagy erőforrás	Tulajdonos és élettartam	Érvénytelenítés, hibakimenet
`stop_requested`	Statikus tárolási idejű objektum	A folyamat végéig él; a handler csak értéket rendel hozzá
`journal` által elért stream	A `main` a tulajdonos; a handler nem fér hozzá	`fclose` után nem használható, hibás visszatérés esetén sem
`&wait_mask` kölcsönzött hozzáférés	A tároló a `main` lokális objektuma	A `pselect` hívás teljes ideje alatt érvényes; nincs megtartott pointer
Signalbeállítások	A teljes program által kezelt folyamatállapot	A példa kilépésig megtartja őket; nem általános könyvtári inicializáló

Bizonyításvázlat. A ciklusfeltétel vizsgálatakor a két leállító signal blokkolt. Ha már függőben vannak, a `pselect` ideiglenes maszkja kézbesíthetővé teszi őket. Ha később érkeznek, megszakíthatják a várakozást. Visszatéréskor az eredeti, blokkoló maszk helyreáll, majd a ciklus újra ellenőrzi a jelzőt. Nincs külön, védelem nélküli rés az ellenőrzés és az elalvás között.

A példa korlátja fontos: a cleanup alatt a leállító signalok blokkoltak, és a `fclose` nem rendelkezik általános időkorláttal. Ez helyes egyszeri leállítási minta, de még nem határidős shutdown.

12.4.5 Linux-alternatíva: signalfd

Linux-only: a `signalfd` segítségével kiválasztott signalok file descriptoron olvasható eseményekké alakíthatók. Ehhez az útvonalhoz nem szükséges aszinkron handler.

A használati szerződés:

1. A kiválasztott signalokat blokkoljuk.
2. Létrehozzuk a descriptort, jellemzően `SFD_CLOEXEC | SFD_NONBLOCK` jelzőkkel.
3. A descriptort az eseményhurok figyeli.
4. Olvashatóságkor `read` segítségével `struct signalfd_siginfo` rekordokat veszünk át.
5. A feldolgozás normál vezérlési úton módosítja a shutdown állapotát.

A puffernek legalább egy teljes rekordot kell befogadnia. Pozitív olvasási eredménynél a teljes rekordokat dolgozzuk fel; `EAGAIN` esetén pillanatnyilag nincs további esemény, `EINTR` esetén a normál megszakítási policy érvényes. A hagyományos signalok összeolvadását a `signalfd` nem szünteti meg.

A descriptor tulajdonosa az eseményhurok; a regisztráció és a kiolvasás kölcsönzött hozzáférés. A tulajdonos lezárása érvényteleníti ezeket. Többszálú programban a maszkot célszerű a szálak létrehozása előtt beállítani: különben egy másik, nem blokkoló szálhoz kerülhet a signal. Másik szálnak címzett signalt a descriptor olvasója nem vehet át tetszőlegesen.

A descriptor eseményhurokba kötését a poll/epoll és explicit eseményállapotgép rész bontja ki.

12.4.6 A graceful shutdown állapotgépe

A szabályos leállítás nem egyetlen `close`, hanem állapotátmenetek sora:

Állapot	Megengedett munka	Következő átmenet
`RUNNING`	Új munka fogadása és feldolgozása	Leállítási kérés → `QUIESCING`
`QUIESCING`	Új munka kizárása, befejezési határidő rögzítése	→ `DRAINING`
`DRAINING`	Már elfogadott munka korlátozott befejezése	Kész → `CLOSING`; határidő → megszakítás
`CLOSING`	Erőforrások egyszeri lezárása	→ kilépés

A türelmi időt monoton órával kell értelmezni. Ha T a határidő, t az aktuális idő, akkor az eseményvárakozás legfeljebb

$$\Delta t = \max(0, T - t)$$

ideig tarthat. Ez csak akkor ad alkalmazási szintű korlátot, ha a köztes munkalépések sem blokkolhatnak korlátlanul.

Egy újabb, külön megfigyelt leállítási kérés gyorsíthatja a megszakítást, de az összeolvadás miatt nem szabad „pontosan két signalra” építeni. A határidő és a külső supervisor kényszerített leállítása külön védelmi réteg.

12.4.7 Gyakori hibák

A handlerből végzett naplózást normál kódba kell áttenni. A `volatile` jelzőt nem szabad általános konkurens adatcsatornává bővíteni. A vak `EINTR`-újrapróbálás helyett állapotfüggő döntés szükséges. Végül a „graceful” nem jelenthet végtelen várakozást: külön kell tervezni az új munka kizárását, a folyamatban lévő munka befejezését és a kényszerített megszakítást.

12.4.8 Feladatok

1. **Magyarázó:** miért nem biztonságos a handlerből meghívni egy egyébként szálbiztos naplózót? Miért nem helyettesíti ezt a problémát a `volatile`?
2. **Kódolási:** írj döntési részletet egy pozitív kapacitású `read` után: külön kezelje a haladást, EOF-ot, megszakítás miatti leállást, újrapróbálást és egyéb hibát.
3. **Diagnosztikai:** egy szolgáltatás jelzőt ellenőriz, majd `pause`-t hív. Néha csak a második `SIGTERM` után áll le. Adj eseménysorrendet és javítást.

12.4.9 Megoldások

12.4.9.1 Szálbiztosság és újrabelépés

A naplózó tarthat belső lockot éppen akkor, amikor a signal megszakítja. A handler ugyanazt a lockot kérve a megszakított művelet befejezésére várhat, amely viszont csak a handler visszatérése után folytatódhat. A `volatile` nem old fel lockot és nem teszi újrabeléphetővé a könyvtárat.

Ellenőrzési feltétel: a handlerben ne legyen nem async-signal-safe hívás; a naplózás normál útvonalon történjen.

12.4.9.2 Olvasási döntés

A következő C17/POSIX-részlet egy ciklusba illeszthető. `n` a `read` eredménye, `saved_errno` pedig a hiba után azonnal elmentett `errno`. A feldolgozó és hibakezelő műveletek alkalmazásfüggők.

```
if (n > 0) {
    /* Könyveld el és dolgozd fel az n bájtnyi haladást. */
} else if (n == 0) {
    /* EOF. */
    break;
} else if (saved_errno == EINTR) {
    if (stop_requested) {
        break;
    }
    continue;
} else {
    /* Egyéb hiba kezelése. */
    break;
}
```

Ellenőrzési feltétel: pozitív eredménynél nincs újraküldés vagy újrafeldolgozás ugyanarra a tartományra. Ez a részlet az eredményt osztályozza; önmagában nem oldja meg a következő blokkoló hívás előtti ébresztési rést.

12.4.9.3 Elveszett ébresztés

Az eseménysorrend: hamis jelző beolvasása → signal → handler beállítja a jelzőt → visszatérés → `pause`. A program ekkor újabb signalra vár.

A javítás blokkolt jelzőellenőrzés és atomikus maszkkal végzett `pselect`, vagy Linuxon blokkolt signalok feldolgozása `signalfd`-n keresztül.

Ellenőrzési feltétel: ne maradjon olyan végrehajtási rés, amelyben a leállítási eseményt már feldolgoztuk, de utána még korlátlanul elalhatunk.

12.4.10 Összefoglaló

A minimális handler csak jelez. A normál vezérlési út dönt az újrapróbálásról, vezeti a shutdown állapotgépét és zárja le az erőforrásokat. A helyességhez együtt szükséges az async-signal-safety, a maszkolás, az elveszett ébresztés kizárása és a határidős leállítási policy.

12.5 Socket API, framing és bufferkezelés

A socketet C-ben nem „kapcsolatobjektumként”, hanem file descriptoron keresztül kezeljük. A fordító nem akadályozza meg a kétszeres lezárást, a már megszűnt buffer használatát vagy az üzenethatárok téves feltételezését. Ezeket explicit életciklus- és protokollszerződéssel kell kizárnunk.

A file descriptor és robusztus I/O ciklus részleges műveletekre vonatkozó szabályai itt is érvényesek. A hosszmező feldolgozása a biztonságos szerializálásra, a megszakítások kezelése pedig a signalokról szóló részre épül. A cél most ezek összekapcsolása egy teljes socket-életciklussá.

12.5.1 Címek és socket-életciklus

POSIX API: a `socket()` egy kommunikációs végpont descriptorát hozza létre. Sikeres visszatérése nem jelenti, hogy a végpont már kapcsolódott is.

A szerver alapvető sorrendje:

`getaddrinfo` → `socket` → `bind` → `listen` → `accept`

A `bind()` helyi címhez rendeli a végpontot; a `listen()` hallgató socketté alakítja. Az `accept()` **új descriptort** ad vissza a létrejött kapcsolathoz. A hallgató descriptor ettől továbbra is használható új kapcsolatok fogadására.

A kliens sorrendje:

`getaddrinfo` → `socket` → `connect`

A `getaddrinfo()` címjelöltek láncolt listáját állítja elő. Minden jelöltnél együtt kell használni az `ai_family`, `ai_socktype`, `ai_protocol`, `ai_addr` és `ai_addrlen` mezőket. Egy sikertelen jelölt socketjét le kell zárni, majd új sockettel próbálható a következő cím.

A `struct sockaddr` az API általános címparaméter-típusa; önmagában nem általános méretű címtároló. IPv4-címhez `struct sockaddr_in`, IPv6-címhez `struct sockaddr_in6` tartozik. Ismeretlen családú visszatérő címhez a megfelelő méretű és igazítású `struct sockaddr_storage` használható.

A `socklen_t` a címhossz API-típusa. Az `accept()` hosszparamétere bemenetként a rendelkezésre álló tároló mérete, kimenetként a visszaadott cím hossza. Ezért minden új próbálkozás előtt inicializáljuk.

Erőforrás vagy tároló	Tulajdonos	Kölcsönzött használat	Élettartam vége
`getaddrinfo()` eredménylistája	végpontnyitó függvény	`socket`, `bind`, `connect` előkészítése	`freeaddrinfo()`
Hallgató és kapcsolati descriptor	`main()`	munkamenet-kezelő függvény	`close()`
Üzenetbuffer	munkamenet-kezelő függvény	küldő és fogadó segédfüggvények	függvényből kilépés
Fogadott partnercím tárolója	`main()` blokkja	`accept()` kimeneti paramétere	blokk vége

A descriptor száma lezárás után újra kioszthatóó. Egy elavult descriptor használata ezért nem feltétlenül `EBADF`: akár egy másik erőforrást is elérhet.

12.5.2 Byte stream helyett explicit üzenetszerződés

POSIX stream-socket: a `send()` és `recv()` nem őrzi meg az alkalmazás üzenethatárait. Egy küldés több fogadásra darabolódhat, több küldés pedig egy fogadásban jelenhet meg.

Pozitív kért fogadási hossz mellett:

- `recv() > 0`: ennyi C bájt érkezett;
- `recv() == 0`: a fogadási irány véget ért, a korábban érkezett adatok elfogytak;
- `recv() == -1`: az `errno` értelmezendő.

A pozitív `send()`-eredmény csak az elfogadott adatmennyiséget jelzi, nem a partner alkalmazásának feldolgozási visszaigazolását.

A példában egy keret:

$$\text{keret} = 4 \text{ oktettes, hálózati sorrendű hossz} \parallel \text{payload}.$$

A hossz kizárólag a payload mérete. A megengedett tartomány $0 \le L \le 4096$. A négyezres korlát alkalmazásszintű erőforrás-szabály, nem TCP-korlát.

A feldolgozás sorrendje:

1. A teljes, négyoktettes fejléc összegyűjtése.
2. A hálózati bájtsorrend dekódolása.
3. A hossz ellenőrzése a buffer kapacitásával szemben.
4. Pontosan L oktett fogadása.

Bizonyításvázlat. Legyen d az eddig átvitt, N a kívánt adatmennyiség. A ciklus invariánsa $0 \le d \le N$. Minden hívás legfeljebb $N-d$ elemet kér, pozitív r eredmény után pedig $d' = d+r \le N$. A pointer csak a még feldolgozatlan tartományra mutat. Hiba esetén nincs állítás teljes üzenetről: a részben módosított buffer tartalma nem publikálható kész payloadként.

12.5.3 Kidolgozott példa: egykeretes kérés és válasz

A következő program egyetlen kapcsolatot kezel. A kliens elküldi a `ping` payloadot, majd `shutdown(fd, SHUT_WR)` hívással lezárja a küldési irányát. Ez **half-close**: a fogadási irány tovább működik. A szerver egy keretet és az azt követő EOF-ot várja, visszaküldi a keretet, majd szintén lezárja küldési irányát.

A protokoll tehát irányonként pontosan egy keretet enged. Az üres payload érvényes; a hiányzó vagy csonka keret nem az.

Profil: ISO C17; Linux hosted user-space, x86-64 System V ABI, LP64, nyolcbites C bájt. Választott fordítási profil: a `lab-next` profil (profiltábla); második fordítóként a `lab-next` profil (profiltábla) ugyanazzal a libc-vel. Az alábbi parancsok helyi ellenőrzésre szolgálnak, nem dokumentált fordítási vagy futtatási eredmények.

```c
/* frame_demo.c */
#define _POSIX_C_SOURCE 200809L

#include <arpa/inet.h>
#include <errno.h>
#include <limits.h>
#include <netdb.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

_Static_assert(CHAR_BIT == 8 && sizeof(uint32_t) == 4,
               "This example requires 8-bit bytes and uint32_t.");

enum { CAPACITY = 4096 };
```

```c
/* Required data: any EOF before completion is a protocol error. */
static int receive_exact(int fd, void *buffer, size_t size)
{
    unsigned char *p = buffer;
    size_t done = 0;

    while (done < size) {
        ssize_t n = recv(fd, p + done, size - done, 0);
        if (n > 0) {
            done += (size_t)n;
        } else if (n == 0) {
            errno = EPROTO;
            return -1;
        } else if (errno != EINTR) {
            return -1;
        }
    }
    return 0;
}

static int send_all(int fd, const void *buffer, size_t size)
{
    const unsigned char *p = buffer;
    size_t done = 0;

    while (done < size) {
        ssize_t n = send(fd, p + done, size - done, MSG_NOSIGNAL);
        if (n > 0) {
            done += (size_t)n;
        } else if (n == 0) {
            errno = EIO;  /* No progress: do not loop forever. */
            return -1;
        } else if (errno != EINTR) {
            return -1;
        }
    }
    return 0;
}

static int receive_frame(int fd, unsigned char *data, uint32_t *length)
{
    uint32_t wire;
    if (receive_exact(fd, &wire, sizeof wire) < 0)
        return -1;

    uint32_t n = ntohl(wire);
    if (n > CAPACITY) {
        errno = EMSGSIZE;
        return -1;
    }
    if (receive_exact(fd, data, (size_t)n) < 0)
```

```
        return -1;

    *length = n;
    return 0;
}

static int send_frame(int fd, const unsigned char *data, uint32_t length)
{
    if (length > CAPACITY) {
        errno = EMSGSIZE;
        return -1;
    }

    uint32_t wire = htonl(length);
    if (send_all(fd, &wire, sizeof wire) < 0)
        return -1;
    return send_all(fd, data, (size_t)length);
}

static int expect_eof(int fd)
{
    unsigned char extra;
    ssize_t n;
    do {
        n = recv(fd, &extra, 1, 0);
    } while (n < 0 && errno == EINTR);

    if (n == 0)
        return 0;
    if (n > 0)
        errno = EPROTO;  /* Extra data violates this protocol. */
    return -1;
}

static int session(int fd, int server)
{
    unsigned char data[CAPACITY] = {'p', 'i', 'n', 'g'};
    uint32_t length = 4;

    if (!server) {
        if (send_frame(fd, data, length) < 0 ||
            shutdown(fd, SHUT_WR) < 0)
            return -1;
    }

    if (receive_frame(fd, data, &length) < 0 || expect_eof(fd) < 0)
        return -1;

    if (server) {
        if (send_frame(fd, data, length) < 0)
            return -1;
        return shutdown(fd, SHUT_WR);
```

```
    }

    if (length != 4 || memcmp(data, "ping", 4) != 0) {
        errno = EPROTO;
        return -1;
    }
    return 0;
}

static int open_endpoint(const char *host, const char *service, int server)
{
    struct addrinfo hints = {0};
    struct addrinfo *addresses = NULL;

    hints.ai_family = AF_UNSPEC;
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags = server ? AI_PASSIVE : 0;

    const char *node = strcmp(host, "-") == 0 ? NULL : host;
    int e = getaddrinfo(node, service, &hints, &addresses);
    if (e != 0) {
        if (e == EAI_SYSTEM)
            perror("getaddrinfo");
        else
            fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(e));
        return -1;
    }

    int fd = -1;
    int saved = EADDRNOTAVAIL;

    for (const struct addrinfo *a = addresses; a != NULL; a = a->ai_next) {
        fd = socket(a->ai_family, a->ai_socktype, a->ai_protocol);
        if (fd < 0) {
            saved = errno;
            continue;
        }

        int ok;
        if (server)
            ok = bind(fd, a->ai_addr, a->ai_addrlen) == 0 &&
                 listen(fd, 8) == 0;
        else
            ok = connect(fd, a->ai_addr, a->ai_addrlen) == 0;

        if (ok)
            break;

        saved = errno;
        (void)close(fd);  /* Preserve the primary failure. */
        fd = -1;
    }
```

```
    freeaddrinfo(addresses);
    if (fd < 0) {
        errno = saved;
        perror("open_endpoint");
    }
    return fd;
}

int main(int argc, char **argv)
{
    if (argc != 4 ||
        (strcmp(argv[1], "s") != 0 && strcmp(argv[1], "c") != 0)) {
        fprintf(stderr, "usage: %s s|c host port\n", argv[0]);
        return 1;
    }

    int server = strcmp(argv[1], "s") == 0;
    int endpoint = open_endpoint(argv[2], argv[3], server);
    if (endpoint < 0)
        return 1;

    int fd = endpoint;
    int result = 1;

    if (server) {
        struct sockaddr_storage peer;
        socklen_t peer_length;
        do {
            peer_length = (socklen_t)sizeof peer;
            fd = accept(endpoint, (struct sockaddr *)&peer, &peer_length);
        } while (fd < 0 && errno == EINTR);

        if (fd < 0) {
            perror("accept");
            goto cleanup;
        }
    }

    if (session(fd, server) < 0)
        perror("session");
    else
        result = 0;

cleanup:
    if (server && fd >= 0 && close(fd) < 0) {
        perror("close connection");
        result = 1;
    }
    if (close(endpoint) < 0) {
        perror("close endpoint");
        result = 1;
```

```
    }
    return result;
}
```

Fordítás, majd két külön terminálban indítás:

```
gcc -std=c17 -O2 -Wall -Wextra -Wpedantic -Wconversion \
    frame_demo.c -o frame_demo
clang -std=c17 -O2 -Wall -Wextra -Wpedantic -Wconversion \
    frame_demo.c -o frame_demo-clang

# Első terminál:
./frame_demo s 127.0.0.1 9090

# Második terminál:
./frame_demo c 127.0.0.1 9090
```

Sikeres esetben a várt eredmény mindkét folyamatnál nulla kilépési státusz. A szerver az első használható címre köt, nem minden IPv4- és IPv6-címre egyszerre. A `-` host szerveroldalon wildcard címet kér; ezt csak tudatos hálózati kitettséggel használjuk.

A példában minden descriptor a `main()` tulajdona. A segédfüggvények csak kölcsönzik. A fogadó buffer legalább `CAPACITY` elemű, írható tároló; a hosszparaméter érvényes kimeneti objektumra mutat. Küldés közben a forrásnak olvashatónak és változatlannak kell maradnia. Hiba után a munkamenet megszakad, a descriptort lezárjuk: egy részben elküldött keretet nem kezdünk újra az elejéről.

12.5.4 SIGPIPE, timeout és nonblocking állapot

Linux-labor: a `send(..., MSG_NOSIGNAL)` megakadályozza, hogy az adott küldés `SIGPIPE` jelzéssel leállítsa a folyamatot; a hibát, például az `EPIPE` értéket továbbra is kezelni kell. A flag nem ISO C-szolgáltatás. Más platformok hordozhatósági rétegében a támogatást, illetve például a `SO_NOSIGPIPE` lehetőségét külön kell ellenőrizni. A `SIGPIPE` folyamat egészére történő figyelmen kívül hagyása szélesebb hatású policy.

A példa blokkoló, egyszálú laborprogram, nincs benne időkorlát vagy `fork/exec`. Egy hibás partner ezért korlátlan ideig feltarthatja, például ha nem küldi el a teljes fejlécet vagy az elvárt EOF-ot.

Linux/POSIX API: a socket fogadási és küldési várakozása `SO_RCVTIMEO`, illetve `SO_SNDTIMEO` opcióval korlátozható. Ez azonban nem azonos a teljes üzenetre vonatkozó határidővel. Ha minden rövid művelet új várakozási keretet kap, egy lassan csöpögő partner sokáig fenntarthatja a munkamenetet.

A teljes művelethez monoton órán alapuló abszolút határidő kell:

$$T_{\text{maradék}} = \max(0, D - t),$$

ahol D az egyszer meghatározott határidő, t az aktuális idő. A névfeloldás idejét külön is számításba kell venni.

Nonblocking módban a `fcntl()` segítségével, a meglévő flag-ek megtartásával állítjuk be az `O_NONBLOCK` értéket. Az `EAGAIN` vagy `EWOULDBLOCK` nem EOF és nem indok aktív újrapróbálkozási ciklusra: a műveletet később folytatjuk.

Ekkor tartós állapotként meg kell őrizni:

- a fejlécből már összegyűjtött mennyiséget;
- az ellenőrzött payloadhosszt és a fogadási kurzort;
- a kiküldendő tárolót és a küldési kurzort.

A fenti szinkron segédfüggvények erre **nem alkalmasak változtatás nélkül**, mert visszatéréskor elveszítik belső kurzorukat. A buffer tulajdonosának az utolsó folytatásig élnie kell; felszabadítás, `realloc()` vagy felülírás érvényteleníthet a függőben lévő hozzáférést.

Nonblocking `connect()` esetén az `EINPROGRESS` folyamatban lévő kapcsolódást jelent. Az írhatósági esemény után `getsockopt(..., SO_ERROR, ...)` segítségével kell ellenőrizni a tényleges eredményt. A folytatások szervezését a poll/epoll és explicit eseményállapotgép rész tárgyalja.

12.5.5 Gyakori hibák

A legsúlyosabb hiba az egyetlen `recv()`-re épített üzenetkezelés. Ugyanilyen veszélyes az ellenőrizetlen hossz alapján történő allokáció vagy pointerléptetés.

A payload nem feltétlenül sztring: nincs garantált lezáró nullkarakter, ezért a `%s` és a `strlen()` nem általános feldolgozási eszköz.

Végül az EOF, a protokollhiba és az átmeneti várakozási állapot különböző esemény. A jelen protokollban a teljes keret utáni EOF elvárt; a fejléc vagy payload közbeni EOF csonkolás.

12.5.6 Feladatok

1. A fejléc és payload három részletben érkezik: `00 00`, majd `00 05 61`, végül `62 63 64 65`. Mekkora a payload, és mikor tekinthető késznek?
2. Egy általánosabb decoder egyetlen bufferben tárolná a fejlécet és a payloadot. Írj ellenőrzést, amely a dekódolt `uint32_t n` alapján biztonságosan számolja ki a teljes méretet. A payload felső korlátja `size_t limit`.
3. Nonblocking küldésnél a program egy lokális buffer címét elmenti, majd `EAGAIN` után visszatér. Később a teljes keretet újraküldi. Milyen két, egymástól független hibát tartalmaz?

12.5.7 Megoldások

12.5.7.1 A darabolt keret

A hossz hálózati sorrendben öt. Az első részlet csak két fejlécoktettet tartalmaz. A második befejezi a fejlécet, és egy payloadoktettet ad. A harmadik után áll rendelkezésre az öt payloadoktett.

Ellenőrzési feltétel: ugyanannak a keretnek minden lehetséges darabolása azonos payloadot eredményezzen; a befejezés előtti EOF hibát adjon.

12.5.7.2 Ellenőrzött teljes méret

Az előző Linux/C17 profilhoz tartozó részlet, `<stdint.h>` és `<errno.h>` használatával:

```
if ((uintmax_t)n > (uintmax_t)limit ||
    (uintmax_t)n > (uintmax_t)SIZE_MAX - 4) {
    errno = EMSGSIZE;
    return -1;
}
size_t total = (size_t)n + 4;
```

Előbb ellenőrzünk, és csak utána adunk össze. Tesztelendő a nulla, a megengedett maximum, az első tiltott érték és — ahol a típusméretek lehetővé teszik — a reprezentálhatatlan teljes méret.

12.5.7.3 Élettartam és részleges haladás

A visszatérés megszüntetheti a lokális buffer élettartamát: a tárolt pointer dangling lesz. Ettől függetlenül az egész keret újraküldése megismétli a korábban már elfogadott prefixet.

A javítás tartós tulajdonoshoz kötött buffer és megőrzött küldési kurzor. Ellenőrzési feltétel: tetszőleges rövid küldések és várakozások mellett a partner pontosan egyszer kapja meg a keret minden oktettjét.

12.5.8 Összefoglaló

A socket API descriptorokat és adatfolyamot ad, nem kész üzenetkezelést. A helyességet az explicit ownership, az ellenőrzött framing, a részleges haladás megőrzése és a megkülönböztetett hibakimenetek biztosítják. Blokkoló kódban ez ciklusokat jelent; nonblocking kódban ugyanez tartós állapotgéppé és hosszabb buffer-élettartammá alakul.

12.6 poll/epoll és explicit eseményállapotgép

Egy nonblocking TCP szolgáltatásban a vezérlés visszatérése nem jelenti a művelet befejezését. A kérésből még hiányozhatnak bájtok, a válasz egy része a felhasználói bufferben várakozhat, a kapcsolat pedig már leállítás alatt állhat. Az eseménykezelő feladata ezért nem pusztán az „olvasható socketek” feldolgozása, hanem a kapcsolatok **explicit állapotának és erőforrás-életciklusának** fenntartása.

A Socket API, framing és bufferkezelés rész rövid műveletekre vonatkozó szabályaira építünk. A dinamikus buffer helyett most rögzített kapacitást választunk, hogy a memóriakorlát közvetlenül bizonyítható legyen. A lezárás alapja a korábban megismert tulajdonlási és cleanup-szerződés.

12.6.1 Készenlétjelzés: poll és epoll

POSIX: a `poll` egy `struct pollfd` tömbben megadott descriptorok készenlétét vizsgálja. A program minden várakozáskor átadja a figyelt halmazt, majd végignézi a visszakapott `revents` mezőket. Kevés kapcsolatnál ez egyszerű és megfelelő megoldás.

Linux: az `epoll` külön kernelobjektumban tartja a regisztrációkat. Az `epoll_ctl` módosítja a figyelt halmazt, az `epoll_wait` pedig eseményrekordokat ad vissza. Ez más költségmodellt jelent, de nem igazolja azt az általános állítást, hogy „az epoll minden művelete konstans idejű”.

A **készenlétjelzés** (*readiness notification*) nem befejezett I/O-művelet és nem helyfoglalás. Olvashatóság után a `recv` visszaadhat adatot, EOF-ot, hibát vagy akár `EAGAIN` értéket. Minden kezelt socket legyen nonblocking; a visszatérési érték marad a tényleges művelet eredménye.

Írási készenlétet csak akkor kérjünk, ha van elküldendő adat. Egy üres kimeneti sorhoz folyamatosan figyelt `POLLOUT` vagy `EPOLLOUT` könnyen CPU-t fogyasztó üres ciklust eredményez.

12.6.2 Level-triggered és edge-triggered feldolgozás

A `poll`, valamint az `epoll` alapértelmezett, **szintvezérelt** (*level-triggered, LT*) működése ismét jelez, amíg a figyelt készenléti feltétel fennáll. Ez megengedi a korlátozott munkavégzést: egy kapcsolat egy fordulóban csak néhány műveletet kap, majd más kapcsolatok következhetnek.

A Linux `EPOLLET` kapcsolója **élvezérelt** (*edge-triggered, ET*) működést kér. Itt nem szabad arra számítani, hogy a feldolgozatlan készenlét miatt újabb értesítés érkezik.

A *drain*, vagyis kiürítés szabálya:

- olvasáskor ismételjük a `recv` hívást `EAGAIN/EWOULDBLOCK`, EOF vagy végleges hiba eléréséig;
- íráskor folytassuk, amíg a kimeneti sor kiürül, vagy a `send` `EAGAIN/EWOULDBLOCK` eredményt ad;
- ET listener esetén az elfogadást is `EAGAIN`-ig kell folytatni.

Az `EINTR` önmagában nem bizonyítja a kiürítést.

Két lényeges kivételt az alkalmazásnak kell kezelnie. Ha elfogyott a memóriakeret, az olvasást megállítjuk; ha elfogyott az egy kapcsolatra jutó munkakeret, átadjuk a vezérlést. **ET módban mindkét esetben saját futásra kész sor szükséges:** amikor ismét lehet dolgozni, aktívan folytatjuk a műveletet, nem várunk feltétlenül új kernelértesítésre. Ugyanez fontos, amikor az üres kimeneti sorba először kerül adat.

Kis szolgáltatáshoz az LT mód jó kiindulás: egyszerűbb helyességi érvelést és természetesebb méltányosságot ad.

12.6.3 Korlátozott bájtvisszhang-szolgáltatás

Példánk legfeljebb 32 kapcsolatot kezel egyetlen eseménykezelő szálon. A protokoll bájtvisszhang: minden befogadott bájtot változatlanul visszaküldünk; nincs külön üzenetkeretezés.

Kapcsolatonként 1024 C bájtos olvasási munkabuffer és 4096 C bájtos kimeneti buffer tartozik az állapotobjektumhoz. Az olvasott adat azonnal a kimeneti sorba kerül, ezért a feldolgozó visszatérésekor nincs külön feldolgozatlan bemenet.

Erőforrás	Tulajdonos	Kölcsönzött hozzáférés	Érvénytelenné válás
Kapcsolatobjektum	A dispatcher slotja	Az aktuális feldolgozó hívás	`free`
Socket FD	Kapcsolatobjektum	`recv`, `send`	`close`; a szám később újra kiosztható
Beágyazott bufferek	Kapcsolatobjektum	Szinkron I/O és másolás	Objektumfelszabadítás; a tartalom korábban is változik

epoll-regisztráció	Eseménykezelő	Slot–generáció token	Törlés; már visszaadott esemény még létezhet

A **visszanyomás** (*backpressure*) itt az olvasási érdeklődés szüneteltetése. Ha a kimenet eléri a 4096 bájtot, nem fogadunk be több adatot; 2048 bájtra csökkenéskor újra engedélyezzük az olvasást. A két küszöb közötti hiszterézis csökkenti a kapcsolgatást. Ez nem azonos az `EPOLLET` élvezérlésével.

A felhasználói bufferek felső korlátja:

$$M_{\text{buffer}} \leq 32(1024 + 4096) = 163840$$

C bájt. Ehhez hozzáadódnak az állapotmezők, az allokátor és a slotok költségei. A kernel socketbufferei és az epoll belső tárolója nem szerepelnek ebben a korlátban.

12.6.3.1 LT feldolgozómag C-ben

Célprofil: ISO C17, Linux, x86-64 System V ABI, LP64, `CHAR_BIT == 8`; választott laborfordítók a `lab-next` profil (profiltábla) környezettel. Az alábbi `echo_core.c` önállóan objektumfájllá fordítható mag, nem teljes szerver. Nem állítunk róla elvégzett fordítást vagy futtatást.

A dispatcher hozza létre a kapcsolatot: érvényes, nonblocking FD, `ACTIVE` állapot, nulla `used`, hamis `paused`. A `c` pointer kölcsönzött; a hívás alatt a tulajdonos nem zárhatja le vagy szabadíthatja fel az objektumot.

```
#include <errno.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <string.h>
#include <sys/epoll.h>
#include <sys/socket.h>
#include <sys/types.h>

enum conn_state { ACTIVE, DRAINING };
enum { RX_CAP = 1024, TX_CAP = 4096, TX_LOW = 2048 };

struct conn {
    int fd;
    enum conn_state state;
    bool paused;
    size_t used;
    unsigned char rx[RX_CAP];
    unsigned char tx[TX_CAP];
};

static bool temporary_error(int e)
{
    return e == EAGAIN || e == EWOULDBLOCK || e == EINTR;
}
```

```
uint32_t conn_interest(const struct conn *c)
{
    uint32_t mask = 0;

    if (c->state == ACTIVE && !c->paused)
        mask |= EPOLLIN | EPOLLRDHUP;
    if (c->used != 0)
        mask |= EPOLLOUT;

    return mask; /* Szándékosan nincs EPOLLET. */
}

/*
 * Eredmény: 0 = megtartás, 1 = szabályos befejezés,
 *           -1 = I/O-hiba; errno azonnal elmentendő.
 * Az EPOLLERR/EPOLLHUP kezelése a dispatcher feladata.
 */
int conn_step(struct conn *c, uint32_t events)
{
    if ((events & EPOLLOUT) && c->used != 0) {
        ssize_t n = send(c->fd, c->tx, c->used, MSG_NOSIGNAL);

        if (n > 0) {
            size_t done = (size_t)n;
            c->used -= done;
            memmove(c->tx, c->tx + done, c->used);
        } else if (n == 0) {
            errno = EIO; /* Nem fogadunk el nulla előrehaladást. */
            return -1;
        } else if (!temporary_error(errno)) {
            return -1;
        }
    }

    if (c->used <= TX_LOW)
        c->paused = false;

    if (c->state == ACTIVE && !c->paused &&
        (events & (EPOLLIN | EPOLLRDHUP))) {
        size_t want = TX_CAP - c->used;
        if (want > sizeof c->rx)
            want = sizeof c->rx;

        ssize_t n = recv(c->fd, c->rx, want, 0);

        if (n > 0) {
            size_t got = (size_t)n;
            memcpy(c->tx + c->used, c->rx, got);
            c->used += got;
        } else if (n == 0) {
            c->state = DRAINING;
```

```
        } else if (!temporary_error(errno)) {
            return -1;
        }
    }

    if (c->used == TX_CAP)
        c->paused = true;

    return c->state == DRAINING && c->used == 0 ? 1 : 0;
}
```

A két külön fordítási ellenőrzés parancsa:

```
gcc -std=c17 -O2 -Wall -Wextra -Wpedantic -Wconversion \
    -c echo_core.c -o echo_core-gcc.o
clang -std=c17 -O2 -Wall -Wextra -Wpedantic -Wconversion \
    -c echo_core.c -o echo_core-clang.o
```

A mag eseményenként legfeljebb egy küldést és egy fogadást végez. Ez **LT-re helyes stratégia**, ET-re változtatás nélkül nem az. Minden megtartott kapcsolatnál a dispatcher a `conn_interest` eredményével frissíti a regisztrációt; az `epoll_ctl` hibája is lezárási ok.

Bizonyításvázlat a bufferkorlátra. Kezdetben `used == 0`. A küldés legfeljebb `used` bájtot távolít el. Fogadáskor

$$\text{want} = \min(1024,\ 4096 - \text{used}),$$

tehát siker esetén

$$\text{used}_{\text{új}} = \text{used}_{\text{régi}} + n \le \text{used}_{\text{régi}} + \text{want} \le 4096.$$

A teljes bufferhez mindig szüneteltetett olvasás tartozik, így a fogadás nem kap nulla hosszúságú kérést.

A `memmove` elküldésenként legfeljebb 4096 bájtot mozgat. Ez nem optimális, de kis terhelésnél egyszerű, folytonos tárolású megoldás, további allokáció nélkül. Nagyobb terhelésnél körbufferrel csökkenthető a másolás.

12.6.4 FD-újrahasználat és elavult események

Az FD egész szám, nem tartós kapcsolat-azonosító. Tegyük fel, hogy egy eseménykötegfeldolgozásakor bezárjuk a 41-es descriptort, majd egy új kapcsolat ismét a 41-et kapja. A korábbi eseményt nem szabad az új kapcsolatra alkalmazni.

Az epoll `data.u64` mezőjébe ezért slotot és generációt csomagolunk, például:

```
((uint64_t)generation << 32) | slot
```

A slot-tábla pointert és generációt tárol. Feldolgozáskor először ellenőrizzük a slot tartományát, foglaltságát és a generáció egyezését; csak ezután dereferáljuk a pointert. Generációtúlcsorduláskor a példabeli slotot végleg kivonjuk, nem engedjük az azonosítók körbefordulását. `poll` esetén az eseménytömb mellé ugyanilyen azonosító-pillanatkép rendelhető.

A lezárási út egyetlen központi művelet:

1. A slotot érvénytelenítjük, generációját léptetjük.
2. `EPOLL_CTL_DEL` segítségével megszüntetjük a regisztrációt.
3. Egyszer meghívjuk a `close`-t.
4. Felszabadítjuk a kapcsolatobjektumot és vele mindkét buffert.

Linux: a `close` hibája után nem próbáljuk vakon újrazárni ugyanazt az FD-számot: közben más erőforráshoz kerülhetett. A cleanup hibáit rögzítjük, de nem tartjuk életben miattuk az objektumot.

Ez a modell kizárja az FD külső duplikálását és a párhuzamos lezárást. Nyers `data.ptr` használatakor az elavult esemény már felszabadított objektumra mutathatna; a generációs ellenőrzés ezt a dereferálást előzi meg.

12.6.5 Shutdown, timeout és cancellation

A szabályos leállítás és a megszakítás külön átmenet:

Állapot és kiváltó ok	Művelet	Következő állapot
`ACTIVE`, pozitív fogadás	Adat a kimeneti sorba	`ACTIVE`
`ACTIVE`, `recv == 0`	További olvasás tiltása	`DRAINING`
`ACTIVE`, szolgáltatásleállítás	Új bemenet tiltása, határidő indítása	`DRAINING`
`DRAINING`, üres kimenet	Központi cleanup	Lezárt
Bármely élő állapot, végleges hiba vagy cancellation	Függő adatok eldobása, cleanup	Lezárt
Bármely élő állapot, lejárt határidő	Cleanup	Lezárt

Az `EPOLLRDHUP` csak jelzés: lehet még olvasatlan adat. A mag ezért a tényleges EOF-ot a `recv == 0` alapján állapítja meg. A dispatcher `EPOLLERR` esetén a `SO_ERROR` segítségével rögzíti a socket hibáját; e szolgáltatás `EPOLLERR` és teljes `EPOLLHUP` esetén megszakítja a kapcsolatot, akár függő adat eldobásával.

A lassú kliens nemcsak memóriát, hanem kapcsolati helyet is foglal. Például 30 másodperces inaktivitási és 5 másodperces leállítási határidőt használhatunk. Az inaktivitást csak pozitív bájtszámú I/O szünteti meg, nem egy készenlétjelzés. A folyamatosan csöpögtető kliens ellen külön maximális kapcsolatélettartam vagy kimeneti várakozási korlát szükséges.

A határidőket monoton órával kezeljük. A várakozás ideje a legközelebbi határidőhöz igazodik; `EINTR` után újraszámítjuk. Folyamatos eseményáradat mellett is ellenőrizzük az időzítőket minden korlátozott feldolgozási fordulóban.

A szolgáltatásleállítás először megszünteti az új kapcsolatok fogadását. A már befogadott kimenetet megpróbáljuk elküldeni, majd a határidőnél kényszerítetten lezárunk. A sikeres `send` itt sem bizonyítja a másik alkalmazás általi feldolgozást.

12.6.6 Gyakori hibák

A memóriakorlát nélküli kimeneti sor csak elrejti a lassú kliens problémáját. Az állandó írási érdeklődés üres ciklust okozhat. Az ET módban egyszer végrehajtott `recv` kapcsolatot akaszthat meg. A regisztráció törlése pedig nem tünteti el a már visszaadott eseményrekordokat: az objektumazonosságot feldolgozáskor is ellenőrizni kell.

12.6.7 Feladatok

1. **Magyarázat:** miért nem elegendő ET módban az olvasási érdeklődés későbbi visszakapcsolása után egyszerűen új eseményre várni?
2. **Kódolás:** egészítsd ki a magot egy `conn_begin_shutdown` függvénnyel, amely megőrzi a kimenetet, és jelzi, ha már azonnal lezárható a kapcsolat.
3. **Diagnosztika:** egy esemény tokenje `(slot=7, generation=12)`. A slotban már a 13-as generáció él, ugyanazzal az FD-számmal. Mi történjen? Milyen további lezárási tesztet végeznél?

12.6.8 Megoldások

12.6.8.1 ET és az újraindítás

Az olvasatlan adat már jelen lehet, ezért új készenléti átmenetre nem támaszkodhatunk. A kapcsolatot saját futásra kész sorba tesszük, majd a megengedett kereten belül folytatjuk a fogadást.

Ellenőrzési feltétel: az olvasás akkor is folytatódik, ha a kliens a szünet után egyetlen új bájtot sem küld.

12.6.8.2 Szabályos leállítás kezdeményezése

Az előző C17 fordítási egységhez hozzáadható:

```
int conn_begin_shutdown(struct conn *c)
{
    c->state = DRAINING;
    return c->used == 0 ? 1 : 0;
}
```

A dispatcher külön rögzíti a határidőt. Nulla eredménynél frissíti az érdeklődést; egy esetén azonnal lezár.

Ellenőrzési feltétel: nincs új fogadás, a meglévő kimenet változatlan, ismételt leállításkérés nem hosszabbítja meg a határidőt.

12.6.8.3 Elavult token

Az eseményt eldobjuk, még az új kapcsolat pointerének dereferálása előtt. Az FD azonossága nem írja felül a generáció eltérését.

További tesztben ugyanarra a kapcsolatra több eseményrekordot adunk egy kötegben, és az első feldolgozása lezárást okoz.

Ellenőrzési feltétel: minden későbbi rekord kiesik az ellenőrzésen; pontosan egy `close` és egy felszabadítás történik.

12.6.9 Összefoglalás

A robusztus eseménykezelő alapja a készenlét és a tényleges I/O-eredmény megkülönböztetése. A korlátozott bufferek, a backpressure és a határidők együtt védik a szolgáltatást. Az explicit állapotgép és a generációs azonosítás pedig biztosítja, hogy a lezárás ne hagyjon életben sem erőforrást, sem régi kapcsolatra mutató feldolgozható eseményt.

13 Adatszerkezetek és parser C-ben

13.1 Dinamikus tömb és string builder

A dinamikus tömb nem egyszerűen egy pointer és egy számláló: olyan tulajdonló objektum, amelynek minden művelet után igazolható állapotban kell maradnia. Ez különösen fontos hosszú életű szolgáltatásokban, ahol egy sikertelen allokáció nem teheti használhatatlanná a korábban összegyűjtött adatot.

A realloc és dinamikusan növekvő buffer szabályaira építve most vector-szerű C tárolót készítünk, majd erre string buildert építünk. A példában az elemek `unsigned char` értékek: nincs elemenként felszabadítandó erőforrás, így a tároló növelése nem igényel külön elemmásolási vagy destrukciós protokollt.

13.1.1 Reprezentáció és invariánsok

A **reprezentációs invariáns** olyan feltételrendszer, amely minden publikus művelet belépésekor és visszatérésekor teljesül. A bájtvektor három mezője:

- `data`: az allokált tároló kezdete;
- `size`: az inicializált, logikailag tárolt elemek száma;
- `capacity`: az allokált tárolóban elférő elemek száma.

A választott reprezentáció invariánsai:

$$0 \leq \text{size} \leq \text{capacity},$$

továbbá `capacity == 0` pontosan akkor igaz, ha `data == NULL`. Pozitív kapacitásnál `data` egy, a tároló által kizárólagosan birtokolt allokációra mutat. A `[0,size)` indextartomány olvasható elemeket tartalmaz; a `[size,capacity)` tartomány pusztán rendelkezésre álló tároló.

A kapacitás nem jogosít fel a logikai tartományon kívüli olvasásra. A végpozíció, `data + size`, nem egy tárolt elem címe. Üres, null pointeres reprezentációnál még ezt a pointerkifejezést sem képezzük.

Entitás	Tulajdonos	Élettartam és hozzáférés
Vektorleíró objektum	A hívó	A hívó inicializálja és végül megsemmisíti
Allokált elemtároló	A vektorleíró	Sikeres újraallokálásig vagy felszabadításig él
Elemre mutató pointer, adatszelet	Nem tulajdonló, kölcsönzött hozzáférés	Nem élheti túl a hivatkozott tárolót; hossza külön szerződés
String builder tárolója	A builder belső bájtvektora	Ugyanazok az allokációs és invalidációs szabályok érvényesek

Az allocator ownership itt egyszerű: a tároló `realloc` segítségével szerez memóriát, és `free` segítségével adja vissza. Nem fogad be idegen, például automatikus tárolási idejű tömböt.

A struktúrák értékmásolása miatt a leíró közönséges értékadása csak a pointert másolná, az allokációt nem. Két ilyen példány nem lehet egyszerre tulajdonos: az kettős felszabadításhoz vezethet. A példában ezért nincs implicit másolási szerződés.

13.1.2 Műveleti és hibaszerződés

A `reserve(need)` legalább `need` kapacitást biztosít, de nem változtatja meg a `size` értékét. A `push(value)` egy értéket, az `append(src,n)` pedig `n` elemet fűz a végére.

Az implementáció a következő **változatlan állapotot garantáló hibaszerződést** használja:

> Túlcsordulás, hibás belső szelet vagy allokációs hiba esetén a korábbi tartalom, méret, kapacitás és tulajdonviszony változatlan.

Ez érvényes bemeneti leírókat feltételez. Érvénytelen pointert vagy nem olvasható forrástartományt a C függvény nem tud általánosan felismerni.

Az `append` külső forrása a hívás alatt érvényes és változatlan maradjon, és ne fedje át sem a leíró objektumot, sem annak allokált tárolóját. `n == 0` esetén a forrás lehet null pointer. Saját tartalom hozzáfűzésére külön, **indexalapú** műveletet adunk.

Ez a szétválasztás nem fölösleges kényelmetlenség. Egy belső pointer már a másolás előtt dangling állapotba kerülhet, ha a kapacitás növelése sikeres `realloc` hívást végez. Ezen a `memcpy` lecserélése `memmove`-ra nem segít.

13.1.3 Biztonságos növekedés és amortizált költség

Legyen az elemméret $w > 0$ C bájt. A legnagyobb reprezentálható elemszám:

$$M = \left\lfloor \frac{\mathrm{SIZE_MAX}}{w} \right\rfloor .$$

Előbb az elemszámot ellenőrizzük, és csak utána számítjuk ki a foglalási méretet:

$$c \le M \quad \Longrightarrow \quad c \cdot w \le \mathrm{SIZE_MAX}.$$

Hozzáfűzéskor az összeadás is ellenőrzendő. Ha s a jelenlegi méret és n a hozzáfűzendő elemszám, akkor:

$$n \le \mathrm{SIZE_MAX} - s$$

szükséges ahhoz, hogy az `s + n` kifejezés ne forduljon körbe. A generikus esetben az így kapott elemszámot még M-hez is hasonlítani kell.

A kapacitást geometrikusan, itt kétszerezéssel növeljük. A határ közelében nem számítjuk ki vakon a `2 * capacity` értéket: ha a duplázás már túl nagy lenne, M-re telítünk.

13.1.3.1 Bizonyításvázlat: amortizált elemmozgatás

Tegyük fel, hogy üres tárolóból indulunk, egyesével végzünk sikeres `push` műveleteket, nincs zsugorítás vagy külön felhasználói `reserve`, és az újraallokáláskor a régi elemek átmásolásával számolunk.

A geometrikus növekedésekhez tartozó másolási munka:

$$1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1.$$

Az utolsó ilyen növekedéshez már több mint 2^k elem beszúrása kellett. Ezért N beszúrásra az összes korábbi elem mozgatása kevesebb mint $2N$. Az új elemek N beírásával együtt a munka $O(N)$, tehát egy `push` **amortizált** költsége $O(1)$ elemmozgatás.

Sikeres beszúrások száma	Kapacitás	Összes korábbi elem másolása a modellben
1	1	0
2	2	1
3	4	3
5	8	7
9	16	15

Ez ellenőrizhető költségmodell, nem mérési eredmény. Egyetlen növelő `push` legrosszabb esetben $O(N)$ elemet mozgathat; az allocator futásidejére ebből nem következik korlát. Az összefüggő tárolás kedvez a szekvenciális hozzáférés lokalitásának, és a geometrikus növekedés $O(\log N)$ allokációs hívással jár.

13.1.4 C17 megvalósítás

Az alábbi teljes fájl bájtvektort és arra épített string buildert tartalmaz. Minden méret C bájtban értendő; nincs szükség `CHAR_BIT == 8` feltételezésre.

Nyelvi profil: ISO C17, hosted környezet, fordítóbővítések nélkül.
Kijelölt, itt nem validált laborprofil: Linux x86-64 System V ABI, LP64; a `lab-next` profil (profiltábla);. A kód nem használ platformfüggő API-t. Ellenőrzött külső forrásjegyzet és tényleges fordítási eredmény ehhez a példához nem áll rendelkezésre.

A builder belső vektorának `size` értéke **tartalmazza a terminátort**. Sikeres inicializálás után:

$$1 \le \text{size} \le \text{capacity}, \qquad \text{data[size-1]} = 0.$$

A logikai szöveghossz ezért `size - 1`.

```c
/* buffer.c - ISO C17 */
#include <assert.h>
#include <stddef.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>

enum buf_status {
```

```
    BUF_OK,
    BUF_OVERFLOW,
    BUF_NOMEM,
    BUF_BOUNDS
};

struct bytes {
    unsigned char *data;
    size_t size;
    size_t capacity;
};

struct strbuf {
    struct bytes bytes;
};

static enum buf_status
bytes_reserve(struct bytes *v, size_t need)
{
    const size_t max = SIZE_MAX / sizeof *v->data;

    if (need > max)
        return BUF_OVERFLOW;
    if (need <= v->capacity)
        return BUF_OK;

    size_t capacity = v->capacity ? v->capacity : 1;

    while (capacity < need) {
        capacity = capacity > max / 2 ? max : capacity * 2;
    }

    void *replacement =
        realloc(v->data, capacity * sizeof *v->data);

    if (replacement == NULL)
        return BUF_NOMEM;

    v->data = replacement;
    v->capacity = capacity;
    return BUF_OK;
}

/* src: külső, nem átfedő, a hívás alatt változatlan forrás. */
static enum buf_status
bytes_append(struct bytes *v, const void *src, size_t n)
{
    if (n == 0)
        return BUF_OK;
    if (n > SIZE_MAX - v->size)
        return BUF_OVERFLOW;
```

```
    size_t need = v->size + n;
    enum buf_status status = bytes_reserve(v, need);

    if (status != BUF_OK)
        return status;

    memcpy(v->data + v->size, src, n);
    v->size = need;
    return BUF_OK;
}

static enum buf_status
bytes_push(struct bytes *v, unsigned char value)
{
    return bytes_append(v, &value, 1);
}

/* Saját, már inicializált tartomány hozzáfűzése. */
static enum buf_status
bytes_append_slice(struct bytes *v, size_t offset, size_t n)
{
    if (offset > v->size || n > v->size - offset)
        return BUF_BOUNDS;
    if (n == 0)
        return BUF_OK;
    if (n > SIZE_MAX - v->size)
        return BUF_OVERFLOW;

    size_t old_size = v->size;
    enum buf_status status = bytes_reserve(v, old_size + n);

    if (status != BUF_OK)
        return status;

    memcpy(v->data + old_size, v->data + offset, n);
    v->size = old_size + n;
    return BUF_OK;
}

static void
bytes_destroy(struct bytes *v)
{
    free(v->data);
    *v = (struct bytes){0};
}

/* Csak új vagy már megsemmisített objektumra hívható. */
static enum buf_status
sb_init(struct strbuf *s)
{
    s->bytes = (struct bytes){0};
    return bytes_push(&s->bytes, 0);
```

```
}

/* Előfeltétel: sikeresen inicializált builder, külső forrás. */
static enum buf_status
sb_append(struct strbuf *s, const void *src, size_t n)
{
    struct bytes *v = &s->bytes;

    if (n == 0)
        return BUF_OK;
    if (n > SIZE_MAX - v->size)
        return BUF_OVERFLOW;

    size_t need = v->size + n;
    enum buf_status status = bytes_reserve(v, need);

    if (status != BUF_OK)
        return status;

    memcpy(v->data + v->size - 1, src, n);
    v->data[need - 1] = 0;
    v->size = need;
    return BUF_OK;
}

/* Kölcsönzött hozzáférés; nem ruház át tulajdonjogot. */
static const char *
sb_cstr(const struct strbuf *s)
{
    return (const char *)s->bytes.data;
}

static void
sb_destroy(struct strbuf *s)
{
    bytes_destroy(&s->bytes);
}

int main(void)
{
    struct bytes v = {0};
    struct strbuf s = {0};
    int result = EXIT_FAILURE;

    if (bytes_push(&v, (unsigned char)'A') != BUF_OK ||
        bytes_append(&v, "BC", 2) != BUF_OK ||
        bytes_append_slice(&v, 0, v.size) != BUF_OK ||
        sb_init(&s) != BUF_OK ||
        sb_append(&s, "node=", 5) != BUF_OK ||
        sb_append(&s, "ready", 5) != BUF_OK) {
        goto cleanup;
    }
```

```
    assert(v.size == 6);
    assert(memcmp(v.data, "ABCABC", 6) == 0);
    assert(s.bytes.size == 11);
    assert(strcmp(sb_cstr(&s), "node=ready") == 0);

    result = EXIT_SUCCESS;

cleanup:
    sb_destroy(&s);
    bytes_destroy(&v);
    return result;
}
```

A kijelölt kiadásoknak megfelelő fordítóparancsokkal:

```
gcc-14 -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O2 \
    buffer.c -o buffer-gcc

clang-19 -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O2 \
    buffer.c -o buffer-clang
```

A parancsnevek önmagukban nem garantálják a patchverziót; a tényleges kiadást a laborban rögzíteni kell. A program nem ír ki eredményt: sikeres műveletek esetén az állítások teljesülése és a sikeres kilépési állapot az elvárt ellenőrzés.

13.1.4.1 Miért helyes a hozzáfűzés?

A külső hozzáfűzés előbb ellenőrzi a méretet, majd biztosítja a tárolót, és csak ezután módosítja a tartalmat. Az allokációs hiba tehát megelőzi az első tartalmi változtatást.

A saját szeletnél a forrás címe csak a `reserve` után képződik újra az indexből. A forrás a régi `[0,old_size)` tartományban van, a cél pedig `old_size`-nál kezdődik, ezért a két másolt tartomány nem fed át: itt a `memcpy` megfelelő.

A builder a régi terminátor helyére másol, majd az új végére nullát ír. A terminátor helyigénye már benne volt a régi `size` értékében, ezért az új foglalási igény `size + n`, nem pedig ellenőrizetlenül `size + n + 1`.

13.1.5 Pointerstabilitás és gyakori hibák

ISO C17 szabály: sikeres, pozitív méretű `realloc` után a régi allokációra hivatkozó pointereket érvénytelennek kell tekinteni, akkor is, ha az új tároló numerikus címe azonosnak látszik.

Művelet eredménye	Korábbi elemre mutató pointer
`reserve`, növelés nélkül	Érvényes marad
Sikeres tárolónövelés	Érvénytelen; újra kell képezni
`append`, növelés nélkül	Érvényes marad, de a régi nézet hossza nem nő meg
Visszaadott műveleti hiba	Érvényes marad, az állapot változatlan
`destroy`	Érvénytelen

A `sb_cstr` eredménye sem tulajdonló. Újraallokálás vagy megsemmisítés érvényteleníti; növelés nélküli hozzáfűzés pedig ugyanazon címen megváltoztatja a látható sztringet.

Gyakori hiba a `data = realloc(data, ...)` közvetlen értékadás: sikertelenségkor elveszhet a régi allokáció címe. A példában az ideiglenes `replacement` ezt akadályozza meg.

Másik hiba a terminátor és a logikai hossz összekeverése. A nullterminált sztringeknél tanultak szerint egy beágyazott nulla korábban lezárja a C-sztringet. Builderünk megőrzi a beágyazott nullákat, ezért `strlen(sb_cstr(&s))` nem feltétlenül egyenlő `s.bytes.size - 1` értékével. Kizárólag szöveget elfogadó API esetén a beágyazott nullákat külön kellene elutasítani.

13.1.6 Feladatok

1. **Invariáns és költség.** Üres vektorba kilenc elemet szúrunk. Mennyi a méret, a kapacitás és a modell szerinti összes másolás? Invalidál-e ezután egy `reserve(7)` hívás?
2. **Kódolás.** Készíts `sb_append_slice` függvényt, amely a builder saját szövegének egy indexelt részét fűzi hozzá. A forrásszelet nem tartalmazhatja a záró terminátort. Hiba esetén maradjon változatlan az állapot.
3. **Diagnosztika.** Az alábbi C17-részlet előtt `v.size == v.capacity == 8`. Mi a hiba, és miért nem megoldás a másolófüggvény lecserélése?

```c
/* HIBÁS: sikeres növelés után érvénytelen pointert olvas. */
const unsigned char *saved = v.data;

if (bytes_push(&v, (unsigned char)'X') == BUF_OK) {
    unsigned char first = saved[0];
    (void)first;
}
```

13.1.7 Megoldások

13.1.7.1 Invariáns és költség

A méret 9, a kapacitás 16. A korábbi elemek másolásának modell szerinti összege:

$$1 + 2 + 4 + 8 = 15.$$

A `reserve(7)` nem csökkenti a kapacitást, nem allokál, és nem invalidál pointert. Ellenőrzési feltétel: a méret és a kapacitás is változatlan.

13.1.7.2 Saját szövegszelet hozzáfűzése

A következő részlet a teljes példába, a `main` elé illeszthető; ugyanaz a C17-profil és tulajdonlási szerződés érvényes rá.

```c
static enum buf_status
sb_append_slice(struct strbuf *s, size_t offset, size_t n)
{
    struct bytes *v = &s->bytes;
    size_t length = v->size - 1; /* inicializált builder */

    if (offset > length || n > length - offset)
        return BUF_BOUNDS;
    if (n == 0)
        return BUF_OK;
    if (n > SIZE_MAX - v->size)
        return BUF_OVERFLOW;

    size_t need = v->size + n;
    enum buf_status status = bytes_reserve(v, need);

    if (status != BUF_OK)
        return status;

    memcpy(v->data + length, v->data + offset, n);
    v->data[need - 1] = 0;
    v->size = need;
    return BUF_OK;
}
```

Az indexek túlélik az újraallokálást; a pointereket utána képezzük. A forrás legfeljebb `length`-ig tart, a cél ott kezdődik, tehát nincs átfedés.

Ellenőrzési esetek: `"abc"` teljes önhozzáfűzése `"abcabc"`; üres szeletnél nincs változás; tartományhiba és injektált allokációs hiba esetén a teljes korábbi állapot megmarad. Sikeres növeléskor minden korábbi borrowed pointer érvénytelen.

13.1.7.3 Érvénytelen pointer újraképzése

A kilencedik elemhez újraallokálás kell. Siker esetén `saved` már nem használható: az olvasás nem definiált viselkedés, garantált kimenete nincs.

A helyes hozzáférés siker után `v.data[0]`, vagyis az aktuális tárolóból újraképzett pointer. A `memmove` csak élő tartományok átfedő másolását kezeli, megszűnt tárolóhoz nem ad új élettartamot.

Allokációs hibánál a régi pointer továbbra is érvényes, de az elem nem került be. A két hibautat külön kell ellenőrizni.

13.1.8 Összefoglaló

A megbízható C tároló alapja az explicit invariáns, az egyértelmű tulajdonos és a változatlan állapotot megőrző hibakezelés. A geometrikus növekedés amortizált költséget javít, pointerstabilitást azonban nem biztosít. Saját tartalom másolásakor indexet őrizzünk, ne régi belső pointert. A string builder mindehhez hozzáadja a terminátor következetesen fenntartott invariánsát.

13.2 Láncolt és intrusive listák

Egy láncolt listában a pointerek helyes átállítása csak a feladat egyik fele. A másik: ki tartja életben a csomópontot, ki szabadíthatja fel, és mit jelent egy korábban eltett bejárópozíció a lista módosítása után? Egy feladatütemezőben például ugyanaz a munkaobjektum átkerülhet a várakozó listából a futó listába anélkül, hogy maga az objektum elköltözne.

A fejezet a struktúrák értékmásolására, az explicit tulajdonlási szerződésre és a pointeraritmetika objektumhatáraira épít. A példák ISO C17-et használnak; a listákhoz egy időben egy végrehajtási szál fér hozzá.

13.2.1 A kapcsolatok és a tulajdonlás külön szerződés

Az **egyszeresen láncolt lista** (*singly linked list*) csomópontja a következő csomópontra mutat. A **kétszeresen láncolt lista** (*doubly linked list*) az előzőre is tárol hivatkozást. Ezek a kapcsolatok önmagukban semmit sem mondanak a tulajdonlásról.

A **tulajdonló lista** (*owning list*) felel a csomópontjai felszabadításáért. Gyakori megvalósításában minden beszúrás külön csomópontot allokál. A csomópontban tárolt payload tulajdonlása ettől még külön döntés: egy benne tárolt pointer lehet kölcsönzött is.

Az **intrusive lista** kapcsolómezői magába a nyilvántartott objektumba vannak beágyazva. Emiatt nincs szükség külön listacsomópont-allokációra. Az intrusive jelleg azonban nem tulajdonlási szabály: itt kifejezetten nem tulajdonló intrusive listát tervezünk.

Szempont	Tulajdonló listamodell	Az itt használt intrusive modell
Csomópont tulajdonosa	A lista	A hívó
Kapcsolómezők helye	Külön csomópontban is lehetnek	A munkaobjektumban
Eltávolítás	Leválasztás és rendszerint felszabadítás	Csak leválasztás
Áthelyezés másik listába	Tulajdonosváltás is	Csak tagságváltás
Payload felszabadítása	Külön szerződés szerint	A külső tulajdonos feladata

Ugyanaz az objektum több intrusive listában is szerepelhet, de minden egyidejű tagsághoz külön kapcsolópár szükséges. Ugyanazt a `next`/`prev` párt két lista nem használhatja.

13.2.2 Sentinel és reprezentációs invariáns

A **sentinel**, vagy őrszemcsomópont, szerkezeti elem, nem felhasználói adat. Körkörös, kétszeresen láncolt listánál az őrszem `next` mezője az első, `prev` mezője az utolsó elemre mutat. Üres listában mindkettő önmagára mutat.

Minden bekapcsolt csomópontra, az őrszemet is beleértve, megköveteljük:

$$n \to \text{next} \to \text{prev} = n, \qquad n \to \text{prev} \to \text{next} = n.$$

További invariáns, hogy a `next` kapcsolatokat követve minden adatelem pontosan egyszer érinthető az őrszemhez való visszatérés előtt. A lokális egyenletek önmagukban ezt nem bizonyítják: egy különálló kör is teljesítheti őket.

Egyszeresen láncolt listában egy ismert előd után beszúrni vagy törölni konstans számú kapcsolatmódosítás. Pusztán a törlendő csomópont ismerete általában nem elég: az előd megkeresése lineáris lehet. Kétszeresen láncolt listánál az előd közvetlenül rendelkezésre áll.

Az őrszemet tartalmazó listaobjektumnak stabil címen kell maradnia. A `struct` értékmásolása nem készít új, önálló listát: az átmásolt pointerek továbbra is az eredeti objektumokra mutatnak.

13.2.3 Kidolgozott példa: munkák áthelyezése

A következő teljes programban a `main` a `job` objektumok tulajdonosa. A listák csak kölcsönzik őket. A `member_of` mező nyilvántartja a tagságot, így felismerhető a kétszeres beszúrás és a rossz listából történő eltávolítás.

Előfeltételek: minden átadott pointer élő, megfelelően inicializált objektumra mutat; a listák invariánsai teljesülnek; a bekapcsolt objektumokat és listaobjektumokat nem másolják vagy költöztetik. A `false` visszatérés felismert szerződésszértést jelent, változatlan állapottal. Érvénytelen pointert ezek az ellenőrzések nem tesznek biztonságossá.

A `list.c` tartalma:

```
#include <stdbool.h>
#include <stddef.h>
#include <stdlib.h>

struct list;

struct job {
    struct job *prev;
    struct job *next;
    struct list *member_of;
    unsigned id;
};

struct list {
    struct job root;
};

static void list_init(struct list *l)
{
    l->root = (struct job){
        .prev = &l->root,
        .next = &l->root,
        .member_of = l
    };
}

static bool list_push_back(struct list *l, struct job *n)
{
    if (n->member_of != NULL ||
        n->next != NULL || n->prev != NULL)
        return false;

    struct job *tail = l->root.prev;
```

```
    n->prev = tail;
    n->next = &l->root;
    tail->next = n;
    l->root.prev = n;
    n->member_of = l;
    return true;
}

static bool list_unlink(struct list *l, struct job *n)
{
    if (n == &l->root || n->member_of != l)
        return false;

    n->prev->next = n->next;
    n->next->prev = n->prev;
    n->prev = NULL;
    n->next = NULL;
    n->member_of = NULL;
    return true;
}

static bool list_move_back(struct list *dst,
                           struct list *src,
                           struct job *n)
{
    if (dst == src || n == &src->root ||
        n->member_of != src)
        return false;

    if (!list_unlink(src, n))
        return false;

    /* Az invariáns szerint n most leválasztott. */
    return list_push_back(dst, n);
}

int main(void)
{
    struct list pending;
    struct list running;
    struct job a = {.id = 7};
    struct job b = {.id = 8};

    list_init(&pending);
    list_init(&running);

    if (!list_push_back(&pending, &a) ||
        !list_push_back(&pending, &b))
        return EXIT_FAILURE;

    struct job *cursor = &a;
```

```
    if (!list_move_back(&running, &pending, &a))
        return EXIT_FAILURE;

    if (cursor->member_of != &running)
        return EXIT_FAILURE;

    if (!list_unlink(&pending, &b) ||
        !list_unlink(&running, cursor))
        return EXIT_FAILURE;

    return EXIT_SUCCESS;
}
```

Fordítási profil: ISO C17, hosted környezet; javasolt laborprofil Linux x86-64, System V ABI, LP64, a `lab-next` profil (profiltábla). A kód nem használ ABI-függő layoutot vagy fordítóbővítést. Az alábbi parancsokhoz nem tartozik itt igazolt fordítási vagy futási eredmény.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O1 -g list.c -o list-gcc
clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion -O1 -g list.c -o list-clang
```

Beszúráskor először az új csomópont két kapcsolatát állítjuk be, majd a két szomszédot kapcsoljuk hozzá. A művelet közben átmenetileg nem teljesül minden invariáns; ezért ez nem konkurens hozzáférésre alkalmas algoritmus.

Bizonyításvázlat: az üres lista teljesíti az invariánst. Beszúrás csak a régi utolsó elem és az őrszem közötti kapcsolatot bontja ketté. Leválasztás az eltávolított elem két szomszédját közvetlenül összeköti. Minden más kapcsolat változatlan, ezért a teljes lista invariánsa megmarad. A `list_move_back` ellenőrzései után a leválasztás sikeres, majd a beszúrás feltételei szükségképpen teljesülnek: nincs allokációs vagy más várható közbülső hibakimenet.

13.2.4 Cursorérvényesség és splice

A **cursor** itt eltett csomópont-pointer és az ahhoz társított bejárási szerződés. Külön kell választani az objektum létezését és a listabeli pozíció érvényességét.

Művelet	A csomópont-pointer állapota	Listabeli jelentése
Másik elem beszúrása	Érvényes marad	Továbbra is ugyanaz az elem
Saját elem leválasztása	Az objektum él, pointere használható	Már nem listapozíció
Saját elem áthelyezése	Címe változatlan	A célhoz tartozik
Objektum felszabadítása vagy élettartamvége	Dangling	Nem használható tovább
Lista újrainicializálása nem üresen	Az elemek még élhetnek	A tagsági szerződés megsérül

A példában `cursor` az áthelyezés után továbbra is `a`-ra mutat, de már nem használható a `pending` lista bejárójaként.

A **splice** meglévő csomópontok átkapcsolása másik listapozícióra, másolás és új csomópont-allokáció nélkül. A példában egyetlen elemet mozgatunk eltérő listák között. Tulajdonló listánál ehhez a felszabadítási szabályok kompatibilitása és a tulajdonjog átadása is szükséges.

Általános, $[first, last)$ tartományt mozgató splice esetén rögzíteni kell, hogy a tartomány összefüggő, a forráshoz tartozik, és a célpozíció nem esik a mozgatott tartomány belsejébe. Az üres tartományt és a saját listán belüli áthelyezést külön kell specifikálni.

Fontos költségkülönbség: a határkapcsolatok átállítása lehet $O(1)$, de a példabeli `member_of` mező frissítése k mozgatott elemnél $O(k)$. A diagnosztikai metaadat is része a költségmodellnek.

13.2.5 Locality és a konstans idejű művelet korlátai

Az $O(1)$ beszúrás feltétele, hogy a pozíció már ismert. Ha előbb azonosító alapján keresünk, a teljes művelet $O(n)$ lehet. Tulajdonló listánál az allokáció költsége sem következik a kapcsolatmódosítás konstans műveletszámából.

Bejáráskor a következő cím az aktuális csomópontból olvasható ki. Ez a **pointer chasing** adatfüggő hozzáférési lánc: a következő olvasás gyakran csak az előző eredménye után indítható. Szétszórt csomópontoknál gyenge lehet a cache-locality, és több lapot kell érinteni.

A folytonos tömb szintén $O(n)$ időben járható be, mégis kedvezőbb lehet az előbetöltés és a cache-kihasználás. Az intrusive lista elhagyja a külön csomópont-allokációt, de nem garantál jó localityt: azt az objektumok tényleges elhelyezése határozza meg. Itt nem mért gyorsulásról, hanem eltérő költségtényezőkről beszélünk.

13.2.6 A container_of hordozhatósági határa

Generikus intrusive megoldások gyakran külön kapcsolóstruktúrát ágyaznak a payloadba. A `container_of` jellegű művelet a kapcsolómező pointeréből próbálja visszakapni a befoglaló objektum pointerét.

A gyakori recept a mező címéből kivonja az `offsetof(T, member)` értéket. Az `offsetof` ISO C17-es eszköz, de a numerikusan helyes offset önmagában nem igazolja a teljes visszaalakítás hordozhatóságát. A karakterpointeres aritmetika objektumhatárait és a kapott pointer jogosultságát is vizsgálni kell.

Egyes makrók GNU `typeof` vagy statement expression bővítést használnak; ezek nem ISO C17-szolgáltatások. Más változat szintaktikailag szabványosnak látszhat, miközben továbbra is implementációs feltételezésekre épít.

További előfeltétel a tényleges befoglaló típus, a megfelelő tag, valamint az élő befoglaló objektum. Önálló sentinel-hookra alkalmazva a makró nem varázsol mögé payloadot.

ISO C17: a struktúra és első, nem bitmező tagja közötti megfelelő pointerkonverzió külön nyelvi garanciát kap; ez nem általános felhatalmazás tetszőleges belső tagból történő visszalépésre. A példánk teljes, típusos `job` őrszemet használ, ezért nincs szüksége `container_of` műveletre.

13.2.7 Gyakori hibák

A kétszeres beszúrás felülírhatja a már használt kapcsolatokat, elveszett elemeket vagy hibás ciklust létrehozva. A tagságjelző ezt a szabályos API-használat keretein belül felismeri.

A körkörösség önmagában nem hiba: a sentinel-lista szándékosan körkörös. Az a hiba, ha a bejárás az őrszem elérése nélkül ismétel csomópontot.

A felszabadítás előtti leválasztás elmulasztása **dangling `next`/`prev` kapcsolatot** hagy a listában. A törlési ciklus másik gyakori hibája, hogy a következő pointert csak a leválasztás vagy felszabadítás

után olvassa ki. A következő pozíciót előre kell elmenteni, és annak élettartamát is garantálni kell.

13.2.8 Feladatok

1. **Magyarázat:** egy egyszeresen láncolt listában csak a törlendő csomópont pointerét ismered. Miért nem ígérhetsz általánosan $O(1)$ törlést? Mi változik kétszeresen láncolt listánál?
2. **Kódolás:** írj függvényt, amely a példabeli listából leválasztja a páros `id` értékű munkákat. Az objektumokat nem szabad felszabadítani.
3. **Diagnosztika:** mi romlik el, ha egy bekapcsolt munkát újra beszúrnak, egy nem üres listát érték szerint lemásolnak, vagy egy heapen tárolt munkát leválasztás nélkül felszabadítanak? Adj ellenőrzési feltételt mindháromhoz.

13.2.9 Megoldások

13.2.9.1 A törlés előfeltétele

Egyszeresen láncolt listánál az elől `next` mezőjét kell módosítani, ezért az elől megkeresése $O(n)$ lehet. Kétszeresen láncolt listánál az ismert, érvényes csomópont `prev` mezője biztosítja ezt az információt. Az ellenőrzési feltétel: a törölt csomópont kihagyásával minden megmaradó elem elérhető, eredeti sorrendben.

13.2.9.2 Biztonságos leválasztó bejárás

Az alábbi C17-részlet a `list.c` függvényeire épül. A tulajdonos továbbra is a hívó; minden csomópont él a teljes bejárás alatt. A leválasztás csak az aktuális listapozíciót érvényteleníti.

```
static bool unlink_even(struct list *l)
{
    struct job *p = l->root.next;

    while (p != &l->root) {
        struct job *next = p->next;

        if (p->id % 2u == 0u && !list_unlink(l, p))
            return false;

        p = next;
    }
    return true;
}
```

Ellenőrzésként próbálj üres, csak páros, csak páratlan és váltakozó listát. A megmaradó elemek sorrendje változatlan; minden leválasztott elem három kapcsolati mezője nullpointer. A `false` belső szerződéssértést jelez, nem tranzakciós visszagörgetést.

13.2.9.3 Három eltérő sérülés

A kétszeres beszúrást a `member_of != NULL` feltétel utasítja el; ellenőrizni kell, hogy a lista változatlan maradt.

Az értékmásolat kapcsolatai az eredeti őrszemre és csomópontokra mutatnak. Javítása nem pointerfoltozás: új listát kell inicializálni, majd meghatározott másolási vagy áthelyezési szerződést alkalmazni.

Leválasztás nélküli felszabadítás után a szomszédok érvénytelen élettartamra hivatkoznak. A tulajdonosnak előbb sikeresen le kell választania az elemet, majd felszabadítania; utána egyetlen listakapcsolat és használható cursor sem hivatkozhat rá.

13.2.10 Összefoglaló

A lista kapcsolati topológiája, tulajdonlása és cursorérvényessége három külön szerződés. Az intrusive tárolás stabil című objektumokat és explicit tagságkezelést igényel. A splice megőrizheti a csomópont címét, miközben megváltoztatja a tagságát vagy tulajdonosát. A konstans számú pointermódosítás pedig nem jelent automatikusan olcsó keresést, allokációt vagy cache-barát bejárást.

13.3 Hash table, comparator és ownership

Egy konfigurációs kulcs–érték tárolóban a gyors keresés csak akkor hasznos, ha közben egyértelmű marad, ki birtokolja a kulcsokat, mikor szűnnek meg a visszaadott pointerek érvényes hozzáférést biztosítani, és milyen állapot marad allokációs hiba után. A hash table ezért nem pusztán algoritmus: tárolási, élettartam- és hibakezelési szerződés is.

A dinamikus tömbnél megismert kapacitáskezelésre, a listák csomópont-élettartamára és a callbackek context-szerződésére építünk. Az alábbi modell ISO C17-et használ, egyetlen szálból, konkurens hozzáférés nélkül.

13.3.1 Hash, egyenlőség és comparator

Legyen $H(k)$ a kulcs hashértéke, $E(a, b)$ pedig az egyenlőségi predikátum. A helyes működés szükséges feltétele:

$$E(a, b) \implies H(a) = H(b).$$

A fordított irány nem követelmény: különböző kulcsoknak lehet azonos hashértéke. Ez **ütközés**, angolul *collision*; az azonosságot végül az egyenlőségi vizsgálat dönti el.

Az egyenlőségnek ekvivalenciarelációnak kell lennie: reflexívnek, szimmetrikusnak és tranzitívnak. A tárolt kulcs hashértéke és egyenlőségi osztálya nem változhat meg a táblabeli tagság alatt.

Konfigurációs tárolónkban a kulcs és az érték nullterminált karakterlánc, beágyazott nullkarakter nélkül. Az alapváltozat kis- és nagybetűérzékeny: az egyenlőség feltétele `strcmp(a, b) == 0`. Ez **ISO C17 könyvtári viselkedésre épülő, választott API-szerződés**, nem minden konfigurációs formátum kötelező szemantikája.

A *comparator* gyakran háromirányú összehasonlító függvényt jelent: negatív, nulla vagy pozitív eredménnyel. Egy logikai equality callback viszont igaz értéket ad egyenlőségkor. A kettő közvetlen felcserélése megfordíthatja a jelentést.

Ha például az equality figyelmen kívül hagyja az ASCII-kisbetű/nagybetű különbséget, a hashnek is ugyanilyen normalizálással kell dolgoznia. Nem elegendő csak az összehasonlítást módosítani.

Callbackes API-nál a callback contextjének tulajdonosa a hívó; a contextnek minden használatkor élnie kell. A benne tárolt hashmag vagy összehasonlítási szabály megváltoztatása újraépítés nélkül megsérti az invariánst.

13.3.2 Open addressing és chaining: tárolási következmények

Nyílt címzésnél (*open addressing*) a bejegyzések közvetlenül egy slottömbben helyezkednek el. Ütközéskor egy keresési sorozat, a *probe sequence* további slotokat vizsgál. Lineáris próbálásnál a kezdőhely:

$$p_0 = H(k) \bmod m,$$

ahol $m > 0$ a slotok száma; ezután növekvő indexek következnek, a tömb végén körbefordulással.

Láncolásnál (*separate chaining*) a hash egy bucketet választ, amely például külön allokált csomópontok listáját tartja.

Tulajdonság	Nyílt címzés	Külön allokált csomópontos láncolás
Tároló	Folytonos slottömb	Buckettömb és csomópontok
Locality	Jellemzően jó a slotok között	Pointerkövetés, szétszórt csomópontok
Allokáció	Tömbnöveléskor; a payload külön is allokálható	Tipikusan beszúrásonként csomópont
Rehash	Bejegyzések más slotokba kerülnek	A csomópontok újraláncolhatók
Címstabilitás	Slotpointer nem stabil rehashkor	Csomópontcím megtartható
Törlés	Tombstone vagy külön átrendezési algoritmus	Csomópont eltávolítása

Ezek implementációs következmények, nem univerzális garanciák. A láncolás sem tesz minden iterátort stabillá: a buckethez vagy következő kapcsolathoz kötött bejárási állapot megváltozhat.

13.3.3 Kapacitás, tombstone és keresési invariáns

Nyílt címzésnél három slotállapotot használunk:

- `EMPTY`: a jelenlegi tábla felépítése óta nem foglalt;
- `LIVE`: élő bejegyzést tartalmaz;
- `DEAD`: korábban foglalt, törölt hely, azaz **tombstone**.

Legyen n az élő, d a törölt slotok száma. Két arány fontos:

$$\alpha = \frac{n}{m}, \qquad \beta = \frac{n + d}{m}.$$

Az α a szokásos **terhelési tényező** (*load factor*). A keresést azonban a tombstone-ok is hosszabbítják, ezért az effektív foglaltságot jelző β sem hagyható figyelmen kívül.

13.3.3.1 Kidolgozott ütközési példa

Legyen $m = 8$, és egy szemléltető hashfüggvény az `env`, `log`, `port` kulcsokhoz egyaránt a 2-es kezdőhelyet adja. Beszúrás után:

Index	2	3	4	5
Állapot	`LIVE`	`LIVE`	`LIVE`	`EMPTY`
Kulcs	`env`	`log`	`port`	—

A `log` törlésekor a 3-as slot `DEAD` lesz. A `port` keresése megvizsgálja a 2-es slotot, áthalad a 3-ason, majd megtalálja a 4-est. Ha törléskor `EMPTY` állapotot írnánk, a keresés idő előtt „nincs ilyen kulcs” eredménnyel állna meg.

Beszúráskor megjegyezhetjük az első tombstone helyét, de tovább kell keresnünk egy egyező kulcs vagy egy `EMPTY` slot eléréséig. Különben már létező kulcsból készíthetnénk második példányt. Minden keresési ciklus legfeljebb m slotot vizsgáljon: ez sérült kapacitáspolitika esetén is kizárja a végtelen körbeforgást.

A példabeli politika az élő elemszámot legfeljebb a kapacitás felére engedi. Sok tombstone esetén azonos kapacitású újraépítés, sok élő elem esetén növelés szükséges. Az újraépítés eltünteti a tombstone-okat.

Növelés előtt két külön határt ellenőrzünk:

$$m \le \left\lfloor \frac{\text{SIZE_MAX}}{2} \right\rfloor, \qquad m_{\text{új}} \le \left\lfloor \frac{\text{SIZE_MAX}}{\text{sizeof(slot)}} \right\rfloor.$$

Az első a duplázást, a második az allokáció méretszorzását védi. A méret egysége **C bájt**, nem feltétlenül nyolcbites oktett. Beszúrásnál az `n + 1` kiszámítása helyett például az `n < m / 2` feltétellel ellenőrizhetjük, hogy elfér-e még egy elem a választott korláton belül.

13.3.4 Ownership és invalidáció

Az **owned** kulcsot vagy értéket a tábla birtokolja; a **borrowed** hozzáférés nem jár felszabadítási felelősséggel. A kettőt kulcsra és értékre külön kell eldönteni.

Erőforrás	Tulajdonos	Kötelezettség
Slottömb	Tábla	Rehashkor és megszüntetéskor felszabadítja
Másolt kulcs	Tábla	Törlésig él, tartalma változatlan
Másolt érték	Tábla	Cseréig vagy törlésig él

Erőforrás	Tulajdonos	Kötelezettség
Kölcsönzött kulcs	Hívó	Tagság végéig él és szemantikailag változatlan
Kölcsönzött érték	Hívó	Minden táblán keresztüli használatkor él

Egy újrahasznált parserbufferre mutató borrowed kulcs tehát akkor is hibás lehet, ha a buffer még nincs felszabadítva: tartalmának módosítása miatt a bejegyzés rossz keresési sorozatban marad.

A következő kód külön allokált, owned kulcsokat és értékeket feltételez. Minden ilyen allokációnak pontosan egy tulajdonosa van; a kulcs- és értékpointerek nem ugyanazt az allokációt birtokolják.

Művelet	Slotra mutató borrowed pointer	Külön allokált kulcs-/értékszövegre mutató pointer
Sikertelen rehash	Megmarad	Megmarad
Sikeres rehash	Érvénytelen	Megmarad
Bejegyzés törlése	A bejegyzéshez tartozó hozzáférés megszűnik	A törölt bejegyzés szövegeire mutató pointerek érvénytelenek
Érték cseréje	Slot megmaradhat	Régi értékpointer érvénytelen
Tábla megszüntetése	Érvénytelen	Érvénytelen

Owned beszúrásnál előbb mindkét szövegmásolatot létrehozzuk, majd biztosítjuk a kapacitást, és csak ezután publikáljuk a bejegyzést. Hiba esetén a részlegesen elkészült másolatokat felszabadítjuk. Értékcserekor is előbb másolunk, és csak siker után szabadítjuk fel a régi értéket.

13.3.5 Tranzakciós rehash C17-ben

Az alábbi `cfg.c` önállóan objektumfájllá fordítható modulrész: a rehash műveletet tartalmazza, nem teljes hashkönyvtárat.

Előfeltétel: `t` érvényes, inicializált táblára mutat; `live` és `used` pontos; `live <= used <= cap`. Üres kezdeti állapota `struct table t = {0};`. A `hash` mező a teljes hashérték, nem a korábbi kapacitással képzett bucketindex.

```
#include <stddef.h>
#include <stdint.h>
#include <stdlib.h>

enum slot_state { EMPTY, LIVE, DEAD };

struct slot {
    enum slot_state state;
    size_t hash;
    char *key;
    char *value;
};

struct table {
```

```
    struct slot *slots;
    size_t cap;
    size_t live;
    size_t used;  /* LIVE + DEAD */
};

enum cfg_status {
    CFG_OK,
    CFG_LIMIT,
    CFG_NOMEM
};

enum cfg_status cfg_rehash(struct table *t, size_t new_cap)
{
    if (new_cap < 8 ||
        t->live > new_cap / 2 ||
        new_cap > SIZE_MAX / sizeof(struct slot)) {
        return CFG_LIMIT;
    }

    struct slot *fresh = malloc(new_cap * sizeof *fresh);
    if (fresh == NULL) {
        return CFG_NOMEM;
    }

    for (size_t i = 0; i < new_cap; ++i) {
        fresh[i] = (struct slot){0};
    }

    for (size_t i = 0; i < t->cap; ++i) {
        if (t->slots[i].state != LIVE) {
            continue;
        }

        size_t p = t->slots[i].hash % new_cap;
        while (fresh[p].state == LIVE) {
            ++p;
            if (p == new_cap) {
                p = 0;
            }
        }
        fresh[p] = t->slots[i];
    }

    free(t->slots);       /* Nem a kulcsokat és értékeket! */
    t->slots = fresh;
    t->cap = new_cap;
    t->used = t->live;
    return CFG_OK;
}
```

Külön elvégzendő laborprofil: a `lab-next` profil (profiltábla), Linux x86-64 System V, LP64;

ISO C17 mód. A példa itt nem végrehajtott fordítási vagy futási mérés.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -c cfg.c -o cfg-gcc.o

clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -c cfg.c -o cfg-clang.o
```

Helyességi bizonyításvázlat. A méretszorzást ellenőrzés előzi meg. Allokációs hiba előtt a régi táblát nem módosítjuk. Sikeres allokáció után legfeljebb `new_cap / 2` élő bejegyzést helyezünk át, ezért minden keresés talál üres slotot. Az áthelyezés nem allokál és nem hív hibázható callbacket. A régi tábla a commitig marad tulajdonos; az ideiglenes pointermásolatok nem jelentenek két önálló felszabadítási felelősséget. Végül csak a régi slottömböt szabadítjuk fel.

A rehash várható költsége megfelelő hash-eloszlási modell mellett $O(m_{\text{új}} + n)$, rossz ütközési mintával azonban $O(m_{\text{új}} + n^2)$ is lehet. Átmenetileg mindkét slottömb él.

Korlátos effektív foglaltság és megfelelő hash-eloszlás mellett a keresés várhatóan konstans számú slotot vizsgál; legrosszabb esetben $O(m)$-et. A kulcs hosszát külön számolni kell: a hashképzés egy k hosszú sztringre tipikusan $O(k)$. Geometrikus növelésnél a növelési költség beszúrásonként amortizálható, de ez nem jelent worst-case konstans késleltetést.

13.3.6 Rosszindulatú bemenet és gyakori hibák

A **hash DoS** lényege, hogy a támadó sok, azonos vagy kedvezőtlenül csoportosuló hashértékű kulcsot választ. Ettől a beszúrások összköltsége négyzetessé válhat. A nagyobb kapacitás nem javítja meg az azonos teljes hashértékeket.

Nem megbízható bemenethez támadásállóságra tervezett, titkos maggal kulcsolt hash, továbbá elemszám-, kulcshossz- és memóriahatár indokolt. A puszta „véletlen só hozzáadása” önmagában nem bizonyít támadásállóságot.

Tipikus hibák:

- hashazonosságot kulcsegyenlőségnek tekinteni;
- borrowed kulcs tartalmát tagság közben módosítani;
- törléskor tombstone helyett `EMPTY` állapotot írni;
- rehash előtt felszabadítani a régi táblát;
- a kapacitás növelését ellenőrizni, de a méretszorzást nem;
- a slotpointer stabilitását összekeverni a külön allokált payload stabilitásával.

13.3.7 Feladatok

1. **Magyarázó:** Az equality szerint `"PORT"` és `"port"` azonos, a hash viszont az eredeti karaktereket dolgozza fel. Mi hibás, és milyen feltétellel javítható?
2. **Kódolási:** Egészítsd ki a modult `cfg_erase_at` függvénnyel. Kapjon indexet, utasítsa el a nem élő slotot, szabadítsa fel a payloadot, és tartsa helyesen a számlálókat.
3. **Diagnosztikai:** Elmentettél egy slotpointert és az abban tárolt értékre mutató pointert. Mi használható sikertelen rehash, sikeres rehash, majd az adott bejegyzés törlése után?

13.3.8 Megoldások

13.3.8.1 Hash és equality javítása

Sérülhet az $E(a, b) \Rightarrow H(a) = H(b)$ feltétel. Mindkét műveletnek ugyanazt a stabil normalizálást kell alkalmaznia, például explicit ASCII-kisbetűsítést. Ellenőrzési feltétel: minden tesztelt, egyenlőnek minősített kulcspár hashértéke legyen azonos. Ez szükséges teszt, nem teljes bizonyítás.

13.3.8.2 Törlés tombstone-nal

Az alábbi C17-részlet ugyanabba a `cfg.c` fájlba illeszthető, és az előző parancsokkal fordítható. A tábla tulajdonlási előfeltételei változatlanok.

```
int cfg_erase_at(struct table *t, size_t i)
{
    if (i >= t->cap || t->slots[i].state != LIVE) {
        return 0;
    }

    free(t->slots[i].key);
    free(t->slots[i].value);
    t->slots[i] = (struct slot){ .state = DEAD };
    --t->live;
    return 1;
}
```

A `used` változatlan: egy élő slotból egy tombstone lett. Ellenőrzési feltétel: ütköző kulcsok közül a középső törlése után a később elhelyezett kulcs továbbra is megtalálható legyen. A törölt kulcs és érték borrowed pointerei többé nem használhatók.

13.3.8.3 Pointerek érvényessége

Sikertelen rehash után mindkét pointer használható marad. Sikeres rehash után a régi slotpointer már nem használható hozzáférésre, az értékpointer viszont igen, mert annak allokációját nem mozgattuk.

A bejegyzés későbbi törlése az értékpointert is érvénytelenné teszi. Ellenőrzési feltétel: sikeres rehash után a slotot új kereséssel kell megszerezni; törlés után a régi payloadpointert sem szabad dereferálni.

13.3.9 Összefoglaló

A hash table helyessége a hash/egyenlőség konzisztenciáján, a stabil kulcsszemantikán és az explicit ownershipen alapul. Nyílt címzésnél a tombstone őrzi a keresési útvonalat, a tranzakciós rehash pedig a régi állapotot védi hiba esetén. A kapacitás, az invalidáció és a támadható bemenet kezelése ugyanannak az API-szerződésnek a része.

13.4 Arena, pool és region lifetime

Egy kérés feldolgozásakor a parser gyakran sok apró, rövid életű objektumot hoz létre: tokeneket, szintaxisfa-csomópontokat és ideiglenes másolatokat. Ha ezekre ugyanaddig van szükség, az egyenkénti `malloc`–`free` kezelés helyett célszerű közös **régióhoz** (*region*) kötni őket. A feldolgozás végén az egész régió felszabadítható.

Az **aréna** (*arena*) ilyen közös életciklusú allokációkat szolgál ki. A **pool** ezzel szemben tipikusan azonos típusú vagy méretű elemek helyeit hasznosítja újra, egyenkénti visszaadással. A választás elsődleges szempontja tehát nem a sebességígéret, hanem az élettartam.

Az igazítás szabályaira, az explicit tulajdonlási szerződésre, valamint a `sizeof` és az igazítás kapcsolatára építünk. A következő megoldás ISO C17-et használ; nem támaszkodik POSIX- vagy Linux-specifikus API-ra.

13.4.1 A régió élettartama és szerződése

Az itt megvalósított aréna egymáshoz láncolt tárolóblokkokból, úgynevezett **chunkokból** áll. Egy chunkon belül az új objektum helyét egy monoton növekvő foglalási pozíció, a *bump pointer* elve határozza meg. Ezt cím helyett `size_t` eltolással tároljuk.

Az API szerződése:

- az arénát nullára inicializált állapotból használjuk;
- az aréna objektuma kizárólagos tulajdonosa a chunkoknak;
- a visszaadott pointerek kölcsönzött hozzáférések, külön nem adhatók át `free`-nek;
- új chunk létrehozása nem mozgatja a korábbi objektumokat;
- a teljes reset minden chunkot felszabadít, így minden korábban kapott pointert érvénytelenít;
- sikertelen foglaláskor az aréna és a korábbi allokációk változatlanok.

Szereplő	Tulajdonos	Érvényesség vége
Aréna vezérlőobjektuma	Példánkban a hívó függvény	A hívó hatókörének elhagyása
Chunk és adattárolója	Az aréna	Teljes reset
Tokenre mutató pointer	Nem tulajdonló hozzáférés	A tokent tartalmazó tároló felszabadítása
Parserből megtartott eredmény	Külön kijelölt tulajdonos szükséges	Saját szerződése szerint

Az aréna vezérlőobjektumának hatókörből kilépése **nem** szabadítja fel automatikusan a chunkokat. Előtte reset szükséges. Ha a parser eredménye túléli a kérést, vagy külön tárolóba kell másolni, vagy az egész aréna tulajdonlását kell szabályosan átadni.

13.4.2 Bump pointer és igazítás

Legyen egy chunk kapacitása C, foglalt előtagjának hossza u, a kért méret n, az igazítás pedig a. Minden mennyiség C bájtban értendő; egy C bájt bitszáma `CHAR_BIT`.

Az alapinvariáns:

$$0 \le u \le C.$$

Feltesszük, hogy a chunk kezdőcíme megfelel az a igazításnak. A következő megfelelő eltoláshoz szükséges padding:

$$r = u \bmod a, \qquad p = \begin{cases} 0, & r = 0, \\ a - r, & r \neq 0. \end{cases}$$

A foglalás kezdete $u + p$, új foglalási pozíciója $u + p + n$. Ezeket azonban csak azután számítjuk ki, hogy igazoltuk:

$$p \leq C - u \quad \text{és} \quad n \leq C - u - p.$$

Így az ellenőrzést nem előzi meg túlcsorduló összeadás. A tömör `(u + a - 1) / a * a` képlet ellenőrizetlenül veszélyes.

Kidolgozott számpélda. Egy feltételezett, megfelelően igazított chunkban legyen $C = 64$, $u = 13$, $a = 8$, $n = 16$. Ekkor $r = 5$, $p = 3$. A fennmaradó hely 51, ezért $3 \leq 51$ és $16 \leq 48$. Az objektum a 16-os eltolásnál kezdődik, az új pozíció 32. Ez aritmetikai illusztráció, nem mért memórialayout.

Bizonyításvázlat. A konstrukció miatt $(u+p) \bmod a = 0$. A két méretellenőrzésből $u+p+n \leq C$ következik, ezért a foglalás a chunkon belül marad. Az új foglalás a korábbi foglalt előtag után kezdődik, tehát a korábbi allokációkkal sem fed át.

13.4.3 Alignmentbiztos, növekvő aréna

ISO C17: a `malloc` által adott tároló megfelelően igazított a fundamentális igazítási követelményű típusok számára. A példánk csak ilyen típusokat támogat: a hívó `_Alignof(T)` értéket ad át, amely nem nagyobb `_Alignof(max_align_t)` értékénél. A kiterjesztett, túlzott igazítású típusok külön megoldást igényelnek.

A chunk fejlécét és adattárolóját külön allokáljuk. Ez chunkonként két allokáció, viszont az adatkezdő cím igazítását nem kell egy fejléc utáni eltolással korrigálni. A `malloc`-kal kapott adattárolónak nincs deklarált típusa; a `unsigned char *` itt a címszámítást szolgálja. A kliens a megfelelően igazított helyen típushelyes írással inicializálja az objektumokat.

Az alábbi teljes fájl neve `arena.c`. Minden arénaparaméternek élő, írható objektumra kell mutatnia; párhuzamos hozzáférés nincs megengedve.

```
#include <assert.h>
#include <stddef.h>
#include <stdint.h>
#include <stdlib.h>

struct chunk {
    struct chunk *next;
    unsigned char *data;
    size_t capacity;
    size_t used;
};

struct arena {
    struct chunk *head;
```

```
};

/* NULL: nulla meret, nem tamogatott igazitas, overflow vagy OOM.
   Hiba eseten az arena valtozatlan. */
static void *arena_alloc(struct arena *a, size_t count,
                         size_t element_size, size_t alignment)
{
    const size_t base_alignment = _Alignof(max_align_t);

    if (count == 0 || element_size == 0 ||
        alignment == 0 || alignment > base_alignment ||
        base_alignment % alignment != 0) {
        return NULL;
    }

    if (count > SIZE_MAX / element_size) {
        return NULL;
    }
    const size_t n = count * element_size;

    struct chunk *c = a->head;
    if (c != NULL) {
        const size_t remainder = c->used % alignment;
        const size_t padding =
            remainder == 0 ? 0 : alignment - remainder;
        const size_t remaining = c->capacity - c->used;

        if (padding <= remaining && n <= remaining - padding) {
            const size_t start = c->used + padding;
            c->used = start + n;
            return c->data + start;
        }
    }

    size_t capacity = 1024;
    if (c != NULL) {
        capacity = c->capacity <= SIZE_MAX / 2
                 ? c->capacity * 2
                 : SIZE_MAX;
    }
    if (capacity < n) {
        capacity = n;
    }

    struct chunk *fresh = malloc(sizeof *fresh);
    if (fresh == NULL) {
        return NULL;
    }

    unsigned char *data = malloc(capacity);
    if (data == NULL) {
        free(fresh);
```

```
        return NULL;
    }

    *fresh = (struct chunk) {
        .next = a->head,
        .data = data,
        .capacity = capacity,
        .used = n
    };
    a->head = fresh;
    return data;
}

/* Minden korabban visszaadott pointert ervenytelenit. */
static void arena_reset(struct arena *a)
{
    while (a->head != NULL) {
        struct chunk *c = a->head;
        a->head = c->next;
        free(c->data);
        free(c);
    }
}

struct token {
    size_t begin;
    size_t length;
};

int main(void)
{
    struct arena scratch = {0};

    struct token *tokens = arena_alloc(
        &scratch, 2, sizeof *tokens, _Alignof(struct token));

    if (tokens == NULL) {
        arena_reset(&scratch);
        return EXIT_FAILURE;
    }

    tokens[0] = (struct token){ .begin = 0, .length = 4 };
    tokens[1] = (struct token){ .begin = 5, .length = 3 };

    void *extra = arena_alloc(
        &scratch, 2048, sizeof(unsigned char),
        _Alignof(unsigned char));

    if (extra == NULL) {
        arena_reset(&scratch);
        return EXIT_FAILURE;
    }
```

```
    assert(tokens[0].length == 4);
    arena_reset(&scratch);
    assert(scratch.head == NULL);

    /* tokens es extra innen nem hasznalhato. */
    return EXIT_SUCCESS;
}
```

Javasolt, külön ellenőrizendő laborprofil: Linux x86-64, System V ABI, LP64, a `lab-next` profil (profiltábla). A példa nem függ e konkrét ABI méreteitől. A fordítási parancsok önálló laborhoz tartoznak; itt nincs hozzájuk közölt futtatási eredmény.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -O1 -g -fsanitize=address,undefined arena.c -o arena-gcc

clang -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -O1 -g -fsanitize=address,undefined arena.c -o arena-clang
```

A `count * element_size` szorzás előtt osztásos korlátellenőrzés történik. A növekedés rendszerint kétszerezés, nagy kérésnél közvetlenül a szükséges kapacitást választjuk. A kétszerezést külön túlcsordulás-ellenőrzés védi.

Új chunkban a foglalás a megfelelően igazított kezdőcímen indul, ezért ott nem szükséges `n + alignment - 1` méretet kérni. A régi chunkokat nem `realloc`-áljuk: ez őrzi meg a korábbi pointerek érvényességét.

A meglévő chunkból történő foglalás $O(1)$ saját műveletet végez, külső allokáció nélkül. A növekedés költségéhez viszont hozzátartozik a két `malloc`; ezek késleltetésére nem adunk konstans idejű garanciát. Az egymás utáni objektumok lokalitása jó lehet, de padding és kihasználatlan chunkvégek maradnak. A reset k chunk esetén $O(k)$ bejárást és $2k$ felszabadító hívást végez.

13.4.4 Pool és free list: egyenkénti újrafelhasználás

A **szabadlista** (*free list*) a pool éppen elérhető helyeit láncolja össze. Foglaláskor levesszük az első helyet; visszaadáskor visszafűzzük. Mindkettő $O(1)$ saját művelet, ha a pool már rendelkezik tárolóval.

A következő C17-deklarációrészlet az előző `struct token` definíciójára épül:

```
struct token_slot {
    struct token value;
    struct token_slot *next_free;
};

struct token_pool {
    struct token_slot slots[16];
    struct token_slot *free_head;
};
```

A `struct` és a tömb nyelvi szabályai gondoskodnak az elemek igazításáról és szükséges paddingjéről. A szabadlista metaadata külön tagban van: nem írunk egy tetszőleges objektum elejére pointert.

A pool inicializálásakor mind a 16 helyet a szabadlistába kell láncolni. Foglaláskor egy `struct token_slot *` kezelő adható vissza, amelynek `value` tagját a kliens inicializálja. Üres listánál a

hibakimenet `NULL`; visszaadáskor a hely csak akkor fűzhető vissza, ha ehhez a poolhoz tartozik és jelenleg ki van adva.

A típus nélküli pool ennél nehezebb: a slot méretének a payloadot és a szabadlista-pointert is be kell fogadnia, a kezdőcímet és a lépésközt pedig mindkettőhöz igazítani kell. Az effective type szabályait sem helyettesíti egy pointercast.

A pool tulajdonosa a teljes slottömb tulajdonosa. A kiadott hozzáférés a visszaadásig érvényes. Visszaadáskor a kölcsönzés **API szerint** megszűnik, noha ebben a deklarált tömbös megoldásban maga a C objektum tovább él. A későbbi hozzáférés így szerződéssértés lehet akkor is, ha memóriadiagnosztika nem jelzi.

13.4.5 Reset, erőforrások és szálbiztonság

ISO C17: a `free` nem hív objektumdestruktort. Ha egy arénában tárolt rekord fájlt, socketet vagy külön allokált memóriát birtokol, a rekord tárolójának felszabadítása nem zárja le ezeket.

A helyes sorrend:

1. megszüntetjük az új hozzáférések lehetőségét;
2. lezárjuk a rekordok által birtokolt külső erőforrásokat;
3. végrehajtjuk a resetet.

Cleanup-lista használható, de annak elemei és bejárási adatai is éljenek a takarítás végéig. Egy cleanup-hiba kezelését külön kell meghatározni; a memóriatakarítás nem jelent automatikusan sikeres erőforráslezárást.

Az aréna nem szálbiztos: a `head` és a `used` módosítása szinkronizálatlan konkurens hozzáférés esetén adatversenyhez vezethet. Praktikus megoldás a **kérésenkénti aréna** (*per-request arena*), amelynek egyszerre egy végrehajtója van. Ha a kérés objektumait több szál olvassa, reset előtt minden olvasónak be kell fejeznie a hozzáférést. Csak a foglalást mutexszel védeni nem elegendő.

Gyakori hiba az aréna érték szerinti másolása: ezzel két vezérlőobjektum ugyanazokat a chunkokat tekinti sajátjának. Ugyancsak hibás az egyedi `free`, a reset utáni pointerhasználat, illetve a poolhely kétszeri visszaadása. Ezeket a tulajdonlási szerződésnek kell kizárnia, nem a szerencsés futási eredménynek.

13.4.6 Feladatok

1. **Magyarázó:** Egy chunk kapacitása 48, foglalási pozíciója 19. Befér-e egy 24 C bájtos, 8-as igazítású objektum? Mi történik 25 C bájtos kérésnél?
2. **Kódolási:** Írj `token_array` wrappert, amely `n` tokennek kér megfelelően igazított tárolót, majd minden tokent inicializál!
3. **Diagnosztikai:** Egy worker arénában épít tokentömböt, átadja pointerét egy eredménysornak, majd resetel. Miért nem elég, hogy a sor mutexszel védett? Adj két javítást!

13.4.7 Megoldások

13.4.7.1 Igazított foglalás

$19 \bmod 8 = 3$, ezért a padding 5. A fennmaradó hely $48 - 19 = 29$, ebből igazítás után 24 használható.

A 24 C bájtos kérés pontosan befér: kezdete 24, vége 48. A 25 C bájtos nem fér be, ezért új chunk szükséges. Ellenőrzési feltétel: a végpozíció legfeljebb a kapacitás, a kezdőpozíció osztható az igazítással.

13.4.7.2 Inicializáló wrapper

C17-részlet az előző teljes példához:

```
static struct token *token_array(struct arena *a, size_t n)
{
    struct token *p = arena_alloc(
        a, n, sizeof *p, _Alignof(struct token));

    if (p == NULL) {
        return NULL;
    }

    for (size_t i = 0; i < n; ++i) {
        p[i] = (struct token){ .begin = 0, .length = 0 };
    }
    return p;
}
```

A tulajdonos továbbra is az aréna; a wrapper nem ad önálló ownershipet. Reset minden visszaadott pointert érvénytelenít. Nulla elemszám, túlcsordulás vagy allokációs hiba esetén `NULL` az eredmény.

Ellenőrizendő a nulla elemszám, a helyes mezőinicializálás, valamint — ha reprezentálható — a `SIZE_MAX / sizeof(struct token) + 1` elemszám visszautasítása.

13.4.7.3 Eredménysor és élettartam

A mutex a sor műveleteit védi, nem hosszabbítja meg a tokenek tárolójának élettartamát. A fogyasztó reset után már felszabadított tárolóra hivatkozna.

Az egyik javítás a tokenek átmásolása a sor által birtokolt tárolóba. A másik a teljes aréna kizárólagos tulajdonlásának átadása, majd reset a fogyasztó befejezése után. Utóbbi nem egyszerű másolás: a korábbi tulajdonosnak le kell mondania a chunkokról.

Ellenőrzési feltétel: a fogyasztó utolsó hozzáférése előzze meg a tároló felszabadítását.

13.4.8 Összefoglalás

Arénát közös élettartamú objektumokhoz, poolt egyenként újrafelhasznált helyekhez válassz. A biztonságos aréna ellenőrzi az igazítást és a méretszámítást, növekedéskor nem mozgatja a korábbi objektumokat, resetkor pedig minden kölcsönzést megszüntet. A memória felszabadítása, a külső erőforrások lezárása és a konkurens hozzáférések befejezése három külön felelősség.

13.5 Bounds-ellenőrzött parser és állapotgép

Egy parser helyessége nem csupán azt jelenti, hogy felismeri a megfelelő formátumot. Azt is garantálnia kell, hogy a csonka, hibás vagy szándékosan támadó bemenet feldolgozása során nem olvas objektumhatáron túl, nem számol túlcsorduló méretekkel, és nem ad vissza már érvénytelen tárolóra mutató eredményt.

A Byte order és biztonságos szerializálás, valamint a Memóriamásolás, összehasonlítás és byte view szabályaira építve egy egyszerű bináris formátumot dolgozunk fel: két, big-endian sorrendű oktett adja meg az utánuk következő payload hosszát. A payload tetszőleges bináris adat, tehát nullkaraktert is tartalmazhat. A parser alkalmazási korlátja 4096 oktett.

13.5.1 Cursor, tartomány és hibaszerződés

A **cursor** a következő feldolgozandó pozíció. Pointeres reprezentációban három értéket tartunk nyilván:

- `begin`: a teljes bemeneti tartomány kezdete;
- `cur`: a következő olvasás helye;
- `end`: a tartomány utáni, már nem dereferálható pozíció.

ISO C17-szabály: a pointerkivonás csak megfelelő közös tömbobjektumon belül, az egy-past pozíciót is megengedve értelmezett; az eredménynek ábrázolhatónak kell lennie `ptrdiff_t` típusban. A parser előfeltétele ezért:

$$0 \le \mathrm{cur} - \mathrm{begin} \le \mathrm{end} - \mathrm{begin} \le \mathrm{PTRDIFF_MAX}.$$

Ekkor a hátralévő elemszám és az aktuális offset:

$$R = \mathrm{end} - \mathrm{cur}, \qquad O = \mathrm{cur} - \mathrm{begin}.$$

Mivel `unsigned char` elemekkel dolgozunk, ezek C bájtban értendők. A példában külön előfeltétel a `CHAR_BIT == 8`, így egy C bájt egy oktett.

Az invariáns nem igazolható tetszőleges pointerek összehasonlításával: már az összehasonlítás vagy kivonás is megsértheti a nyelvi szabályokat. A helyes tartomány létrehozása a hívó kötelessége. Üres bemenethez is használunk egy élő objektumból képzett pointert; nem vonunk ki két null pointert.

A parser egy kötelező következő rekordot keres, és az alábbi szerződést vállalja:

Eredmény	Jelentés	Cursor és eredmény
`P_OK`	Egy teljes rekord elkészült	A cursor előrelép; a view érvényes
`P_NEED_MORE`	A prefix még kiegészíthető	Mindkettő változatlan
`P_TRUNCATED`	A végleges bemenet hiányos	Mindkettő változatlan
`P_LIMIT`	A deklarált hossz túllépi a korlátot	Mindkettő változatlan

Ez **tranzakciós partial-result policy**: hibánál nincs részleges payload-eredmény. A hibaadat saját enumot és offsetet tartalmaz, nem a bemenetből kölcsönzött hibaüzenetet.

13.5.2 Kidolgozott C17-parser

Az alábbi teljes fájl neve `parser.c`. A `c`, `out` és `error` érvényes, egymástól és a bemenettől elkülönülő objektumokra mutasson. A bemenet a hívás alatt nem változhat. Tulajdonosa a hívó; a parser nem allokál és nem szabadít fel.

```c
#include <assert.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <limits.h>

_Static_assert(CHAR_BIT == 8, "The wire format requires 8-bit bytes.");

enum { MAX_PAYLOAD = 4096 };

enum parse_code {
    P_OK,
    P_NEED_MORE,
    P_TRUNCATED,
    P_LIMIT
};

struct cursor {
    const unsigned char *begin;
    const unsigned char *cur;
    const unsigned char *end;
};

struct token_view {
    const unsigned char *data;
    size_t size;
};

struct parse_error {
    enum parse_code code;
    size_t offset;
};
```

```
static enum parse_code report(struct parse_error *error,
                              enum parse_code code,
                              size_t offset)
{
    *error = (struct parse_error){code, offset};
    return code;
}

enum parse_code parse_one(struct cursor *c, bool final,
                          struct token_view *out,
                          struct parse_error *error)
{
    const size_t remaining = (size_t)(c->end - c->cur);
    const size_t start = (size_t)(c->cur - c->begin);
    const enum parse_code incomplete =
        final ? P_TRUNCATED : P_NEED_MORE;

    if (remaining < 2) {
        return report(error, incomplete, start + remaining);
    }

    const size_t length =
        (size_t)c->cur[0] * 256u + (size_t)c->cur[1];

    if (length > MAX_PAYLOAD) {
        return report(error, P_LIMIT, start);
    }

    if (length > remaining - 2) {
        return report(error, incomplete, start + remaining);
    }

    const unsigned char *payload = c->cur + 2;

    *out = (struct token_view){payload, length};
    c->cur = payload + length;
    return report(error, P_OK, (size_t)(c->cur - c->begin));
}

#ifndef PARSER_FUZZ
int main(void)
{
    const unsigned char input[] = {
        0x00, 0x03, 0x41, 0x00, 0xff
    };

    struct cursor c = {input, input, input + sizeof input};
    struct token_view out = {NULL, 0};
    struct parse_error error = {P_OK, 0};

    const enum parse_code result =
        parse_one(&c, true, &out, &error);
```

```
    assert(result == P_OK);
    assert(out.size == 3);
    assert(out.data[0] == 0x41);
    assert(out.data[1] == 0x00);
    assert(out.data[2] == 0xff);
    assert(c.cur == c.end);
    assert(error.offset == sizeof input);
    return 0;
}
#endif
```

Fordítási profil: ISO C17; Linux x86-64, System V ABI, LP64, 8 bites C bájt. Konkrét laborválasztásként a `lab-next` profil (profiltábla) és használható; a telepített párosítást külön rögzíteni kell. A parancsok nem jelentenek dokumentált fordítási vagy futtatási eredményt.

```
gcc-14 -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
    -O1 -g -fsanitize=address,undefined \
    parser.c -o parser-check
```

13.5.2.1 Miért ilyen sorrendben ellenőrzünk?

A hosszmező legnagyobb értéke:

$$255 \cdot 256 + 255 = 65535.$$

Ez a választott profil `size_t` típusában ábrázolható. Hosszabb, például 64 bites wire-hossz esetén külön ellenőrizni kellene a `size_t` tartományába illeszkedést; a szűkítő konverzió nem helyettesíti ezt.

A payload ellenőrzésénél nem a `2 + length > remaining` összeget képezzük, hanem előbb igazoljuk, hogy `remaining >= 2`, majd kivonunk:

$$\text{length} \leq \text{remaining} - 2.$$

Bizonyításvázlat: az első ellenőrzés biztosítja a két fejlécbájt olvashatóságát és a kivonás alulcsordulás-mentességét. A második biztosítja, hogy a teljes payload a tartományon belül van. Csak ezek után képezzük a payload végét. A hibapozícióként használt `start + remaining` pontosan a teljes tartomány hossza, ezért az előfeltételek mellett nem csordul túl.

A golden case deklarált hossza három. Payload-offsetje kettő, következő cursor-offsetje öt. Ha az utolsó oktett hiányzik, az eredmény végleges bemenetnél `P_TRUNCATED`, az offset pedig négy: itt kellett volna folytatódnia az adatnak. A `P_LIMIT` ezzel szemben a hibás hosszmező kezdetére mutat.

13.5.3 View, saját másolat vagy arena?

A `token_view` csak pointert és hosszt másol. Nem másolja át a payloadot, és nem veszi át annak tulajdonlását.

Reprezentáció	Tulajdonos	Érvényesség és érvénytelenítés
Bemeneti payload-view	A bemeneti tároló tulajdonosa	A tároló felszabadítása vagy sikeres újraallokálása érvényteleníti
Külön allokált másolat	Az eredmény tulajdonosa	A hozzá tartozó `free`-ig él
Arenába másolt token	Az arena tulajdonosa	Reset vagy megsemmisítés érvényteleníti
Parser hibaadata	A hívó `parse_error` objektuma	Nem függ a bemenet további élettartamától

A bemenet felülírása nem feltétlenül szünteti meg a pointer nyelvi érvényességét, de megváltoztatja a view által megfigyelt adatot. Emiatt egy újrahasznosított fogadóbufferből származó view-t nem szabad változatlan eredményként tárolni.

Saját, nullterminált sztringhez előbb ellenőrizni kell, hogy az $n + 1$ tárolóméret ábrázolható, majd kezelni az allokáció hibáját. A másolás és a lezáró nullkarakter elhelyezése után is meg kell őrizni a hosszt, ha beágyazott nullkarakter megengedett. Bináris payloadhoz a `strlen` nem használható.

Sok, közös élettartamú tokenhez az Arena, pool és region lifetime modellje megfelelő. Az arena azonban csak a ténylegesen bemásolt adatot birtokolja: egy benne tárolt view továbbra is függhet egy külső inputbuffertől. Hash táblába helyezett tokenkulcsnál ugyanezt az ownership-döntést kell meghozni; lásd Hash table, comparator és ownership.

13.5.4 Streaming feldolgozás mint állapotgép

A `parse_one` teljes, összefüggő tartományt vizsgál. `P_NEED_MORE` után a hívónak meg kell őriznie a teljes feldolgozatlan prefixet, és kibővített tartománnyal újrapróbálkoznia. Újraallokálás után a cursor pointereit megőrzött **offsetekből**, az új bázisból kell újraképezni.

Valódi streaming parser ehelyett menet közben is elfogyaszthatja a chunkokat. Egy egyszerű megvalósítás a parserállapot saját, 4096 oktettes payload-tárolójába másol:

Állapot	Tárolt invariáns	Átmenet
`HEADER`	0 vagy 1 fejlécoktett összegyűjtve	Két oktett után hosszellenőrzés
`BODY`	$0 \le used < length \le 4096$	Másolás után teljes payload esetén `READY`
`READY`	Egy teljes rekord rendelkezésre áll	A fogyasztó átvétele után reset
`ERROR`	Megőrzött hibakód és abszolút offset	Csak explicit reset engedélyezett

Nulla hossz esetén a fejléc után közvetlenül `READY` következik. A parser ne írja felül a kész eredményt annak átvétele előtt.

A mintabemenet három chunkban:

1. `00`: egy fejlécoktett, további adat szükséges;
2. `03 41`: ismert a háromoktettes hossz, egy payload-oktett megvan;
3. `00 ff`: elkészül a rekord.

Másoláskor a mennyiség:

$$k = \min(\text{chunk remaining},\ \text{length} - \text{used}).$$

Az állapot saját tárolójára mutató eredmény-view legfeljebb a következő felülírásig/resetig, illetve az állapotobjektum életének végéig használható. A chunk kölcsönzött tárolója viszont a feldolgozóhívás után elengedhető, ha minden szükséges adatot átmásoltunk.

A stream API külön adja vissza az elfogyasztott chunkhosszt. Itt a `NEED_MORE` nem jelenti azt, hogy semmi sem fogyott el. Végleges bemenetnél a részleges fejléc vagy payload csonka rekord; két rekord között az üres `HEADER` állapot szabályos streamvég lehet. Az abszolút offset növelése előtt ellenőrizni kell:

$$k \le \text{SIZE_MAX} - \text{total}.$$

13.5.4.1 Mélység- és erőforráskorlátok

A példanyelv lapos, így nincs bemenetfüggő rekurzió. Egymásba ágyazható konfigurációs nyelvnél azonban minden rekurzív leszállás **előtt** ellenőrizni kell a mélységkorlátot. Ha `depth >= max_depth`, a parser adjon külön mélységhibát, ne hívja meg önmagát.

A mélységkorlát nem helyettesíti a tokenek számára vagy az összes másolt adatra vonatkozó keretet. Az állapotgép ciklusa pedig vagy fogyasszon inputot, vagy váltson előrehaladást biztosító állapotba, vagy térjen vissza. Különben UB nélkül is létrejöhet szolgáltatásmegtagadást okozó végtelen ciklus.

13.5.5 Golden case, round-trip és fuzz harness

A golden case ismert wire-bájtokhoz ismert eredményt rendel. A round-trip tulajdonság azt ellenőrzi, hogy egy érvényes rekord szerializálása, majd visszaolvasása megőrzi a hosszt és a payloadot. Önmagában ez kevés: az encoder és a decoder ugyanazt a byte-order hibát is tartalmazhatja.

Az alábbi részt a `parser.c` végéhez lehet hozzáadni. A bemenetet a fuzzoló biztosítja, a harness csak a hívás idejére kölcsönzi; sem pointert, sem view-t nem őriz meg.

```
#ifdef PARSER_FUZZ
int LLVMFuzzerTestOneInput(const unsigned char *data, size_t size)
{
    unsigned char empty = 0;

    if (size > (size_t)PTRDIFF_MAX) {
        return 0;
    }

    const unsigned char *base = size == 0 ? &empty : data;

    for (int final = 0; final < 2; ++final) {
        struct cursor c = {base, base, base + size};
        struct token_view out = {NULL, SIZE_MAX};
        struct parse_error error = {P_OK, 0};
```

```
        const enum parse_code result =
            parse_one(&c, final != 0, &out, &error);

        assert(error.code == result);
        assert(error.offset <= size);

        if (result == P_OK) {
            assert(size >= 2);
            assert(out.size <= MAX_PAYLOAD);
            assert(out.size <= size - 2);
            assert(out.data == base + 2);
            assert(c.cur == out.data + out.size);
        } else {
            assert(c.cur == base);
            assert(out.data == NULL);
            assert(out.size == SIZE_MAX);
        }
    }

    return 0;
}
#endif
```

Eszközspecifikus profil: a `lab-next` profil (profiltábla), elérhető libFuzzer- és sanitizer-runtimemal. Ezek nem ISO C szolgáltatások.

```
clang-19 -std=c17 -DPARSER_FUZZ \
    -Wall -Wextra -Wpedantic -O1 -g \
    -fsanitize=fuzzer,address,undefined \
    parser.c -o parser-fuzz
```

A hibás input itt normál visszatérési értéket eredményez; az assertion megsértése parser- vagy harnesshibát jelez. A fuzzolás és a sanitizerek hibakereső eszközök, nem teljes helyességbizonyítások.

13.5.6 Gyakori hibák

- **Bounds-ellenőrzés előtti dereferencia:** már a hosszmező olvasása is kilóghat. Előbb a fejléc méretét ellenőrizzük.
- **`cur + length` képzése az ellenőrzéshez:** az érvénytelen pointerképzés megelőzheti az összehasonlítást. Maradékhosszal számolunk.
- **Minden hiányos input végleges hibának minősítése:** streaming esetben külön kell választani a várakozást és a csonka streamvéget.
- **Részben kitöltött eredmény hibánál:** a hívó könnyen kész rekordnak tekinti. A publikálás csak az összes ellenőrzés után történjen.
- **View megtartása buffer-újrahasznosítás után:** másolás vagy dokumentált owner-élettartam szükséges.

13.5.7 Feladatok

1. **Magyarázó:** Mi az eredmény a `00 03 41` bemenetre `final == false`, illetve `final == true` mellett? Hol van a hibaoffset?
2. **Kódolási:** Tervezz `parse_exact` wrappert, amely pontosan egy rekordot fogad el, és elutasítja a trailing adatot. Hibánál maradjon változatlan a kimenet.
3. **Diagnosztikai:** A streaming parser payload-view-ját egy eseménysorba tesszük, majd ugyanabban a tárolóban fogadjuk a következő rekordot. Miért romolhat el az eredmény sanitizerjelzés nélkül? Adj javítást és ellenőrzési feltételt.

13.5.8 Megoldások

13.5.8.1 Hiányos payload

Nem végleges inputnál `P_NEED_MORE`, véglegesnél `P_TRUNCATED` az eredmény. Az offset három: ott hiányzik a következő oktett. A cursor és az `out` változatlan.

Ellenőrzés: ugyanazt a háromoktettes tartományt mindkét `final` értékkel vizsgáljuk, és az állapot megőrzését is ellenőrizzük.

13.5.8.2 Pontosan egy rekord

A wrapper helyi cursorral és helyi view-val hívja a `parse_one` függvényt. Siker után ellenőrzi, hogy a helyi cursor elérte-e az `end` értéket. Ha nem, külön trailing-data hibát ad az első fölösleges oktett offsetjével. A kimenetet csak teljes siker esetén másolja ki.

Ellenőrzés: a golden case sikeres, ugyanaz egy további oktettel hibás, és a korábbi kimenet mindkét hibafajtánál érintetlen.

13.5.8.3 Újrahasznosított tároló

A pointer még élő objektumba mutathat, csak annak tartalma változott meg. Ez logikai élettartam-/ownership-hiba, ezért nem szükségszerű a sanitizerjelzés.

Javításként a sor saját payload-másolatot birtokolhat, vagy átveheti egy külön rekordtároló tulajdonlását.

Ellenőrzés: két eltérő rekord egymás utáni fogadása után az első sorbejegyzés tartalma változatlan; felszabadítása pontosan egyszer történik.

13.5.9 Összefoglaló

A biztonságos parser ellenőrzött tartományból, bizonyítottan érvényes cursorral olvas. A hosszszámítás megelőzi a pointerléptetést, a hibaoffset jelentése rögzített, és a részleges eredmény kezelése explicit. Streaming esetben külön szerződés szükséges a fogyasztásra, az állapotra és a tároló újrahasznosítására. A bounds-helyesség és a visszaadott view élettartamának helyessége két külön, egyformán szükséges garancia.

14 Assembly, hívási ABI és hardverkapcsolatok

14.1 C forrásból assembly és gépi kód

Amikor egy natív szolgáltatás profilozása egy rövid C függvényre mutat, a forrás önmagában nem árulja el, milyen utasítások hajtják végre a munkát. Egy változó eltűnhet, egy szorzás címképző utasítássá alakulhat, egy forrássorhoz pedig több, egymástól távoli gépi utasítás tartozhat. Az assembly olvasásának célja ezért nem a C „visszafordítása”, hanem a fordító által választott megvalósítás megértése.

A fordítási fázisoknál megkülönböztettük a fordítást, az assemblálást és a linkelést. Most ezeket megfigyelési pontként használjuk. A paraméterátadás nyelvi szabályait és a méret, igazítás, offset fogalmait ismertnek tekintjük; a hívási konvenció részletes tárgyalása a következő egység feladata.

14.1.1 A nyelvi szerződéstől az utasításkódolásig

Az **assembly** egy utasításkészlet szöveges jelölése, kiegészítve az assemblernek szóló direktívákkal, címkékkel és szimbólumhivatkozásokkal. A **gépi kód** az utasítások processzor által értelmezett bináris kódolása. A **disassembly** ennek visszaalakítása olvasható utasításokká; nem állítja vissza az eredeti C programot.

Négy különböző állítási szintet kell elválasztani:

Szint	Mit határoz meg?	Példa
ISO C17	A program nyelvi jelentését	Az `unsigned` aritmetika moduláris
Hívási ABI	A külön fordított kódrészek együttműködését	Mely regiszterben érkezik egy argumentum
Utasításkészlet, ISA	A gépi utasítás jelentését és kódolását	Mit csinál az x86-64 `add`
Fordító és kapcsolói	Egy konkrét megvalósítás kiválasztását	Szorzás helyett használ-e `lea` utasítást

Az **alkalmazási bináris interfész** (*Application Binary Interface*, ABI) nem nyelvi garancia. Az, hogy egy paraméter egy adott regiszterben érkezik, nem következik a C függvénydeklarációjából önmagában.

14.1.1.1 Konkrét megfigyelési profil

Az alábbi példák választott célprofilja:

Tulajdonság	Érték
Nyelv	ISO C17, `-std=c17`
Platform	Linux, x86-64, System V AMD64 ABI, LP64
Alaptípusok	`CHAR_BIT == 8`, 32 bites `unsigned`, 64 bites `size_t`
Fordítók	GCC 14.2.0 és Clang 19.1.7
Bináriseszközök	GNU Binutils 2.43.1
Rendszerfejlécek	glibc 2.40 profil
Kódgenerálás	`-march=x86-64 -mtune=generic`, LTO nélkül
Diagnosztika	DWARF 5 debugadatok, `-O0` és `-O2`

Itt egy **C bájt**, a `sizeof` egysége, nyolc bites oktett. Más célplatformra ezt nem visszük át automatikusan.

A közölt assemblyrészletek kézzel összeállított, a profilhoz illeszkedő szemléltetések, nem dokumentált fordítási eredmények. A kapcsolók rögzítik a vizsgálat körülményeit, de a fordító tényleges kimenetét a saját laborban kell előállítani. Más fordítókiadás eltérő, mégis helyes utasítássorozatot választhat.

14.1.2 Assembly előállítása és objektumfájl vizsgálata

Az alábbi `inspect.c` önálló fordítási egység, minimum C17 követelménnyel. Objektumfájllá fordítható; nem teljes végrehajtható program. A `report` csak deklarált külső függvény, definíciója egy későbbi linkeléshez kellene.

```
#include <limits.h>
#include <stddef.h>

_Static_assert(CHAR_BIT == 8, "8-bit C bytes required");
_Static_assert(sizeof(unsigned) == 4 &&
               UINT_MAX == 0xffffffffU,
               "32-bit unsigned required");
_Static_assert(sizeof(size_t) == 8, "64-bit size_t required");

unsigned scale(unsigned x)
{
    unsigned twice = 2U * x;
    unsigned result = twice + 3U;
    return result;
}

unsigned bump(unsigned *items, size_t i, unsigned delta)
{
    unsigned old = items[i];
    unsigned next = old + delta;
    items[i] = next;
    return next;
}

extern void report(unsigned code);
```

```
unsigned report_if_nonzero(unsigned code)
{
    if (code == 0U)
        return 0U;

    report(code);
    return 1U;
}
```

A fordítási parancsok Bash alatt, a táblázat szerinti eszközökkel értendők:

```
flags=(
  -std=c17 -Wall -Wextra -Wpedantic
  -march=x86-64 -mtune=generic
  -fno-lto -fcf-protection=none
  -g -gdwarf-5
)

for cc in gcc clang; do
  for opt in 0 2; do
    "$cc" "${flags[@]}" "-O${opt}" -S -masm=intel \
      inspect.c -o "${cc}-O${opt}.s"
    "$cc" "${flags[@]}" "-O${opt}" -c \
      inspect.c -o "${cc}-O${opt}.o"
  done
done

objdump -dr -Mintel gcc-O2.o
objdump -dS -Mintel gcc-O2.o
readelf --debug-dump=decodedline gcc-O2.o
```

Fordítóeszköz: a GCC és Clang `-S` kapcsolója assemblyszövegnél állítja meg a feldolgozást. A `-c` objektumfájlt készít, linkelés nélkül. A `-masm=intel` a szöveges assembly megjelenítési módját választja ki, nem másik utasításkészletet.

Bináriseszköz: az `objdump -d` disassemblyt készít; a `-r` a relokációkat is megmutatja. Ez a külső `report` hívásánál fontos: az objektumfájl még nem tartalmazza a végleges célcímet. Az `objdump -S` ezzel szemben forrásszöveget próbál keverni a disassembly mellé. Ugyanaz a kapcsolóbetű itt mást jelent, mint a fordítónál.

A fordító assemblykimenetében látható `.loc`, `.cfi_*` vagy `.section` sorok direktívák. Nem mindegyik sor jelent végrehajtandó gépi utasítást.

14.1.3 Regiszterek, operandusok és szintaxis

A **regiszter** a processzor architekturálisan megnevezett tárolója. x86-64-en a `rax` 64 bites regiszter, az `eax` ugyanennek az alsó 32 bites része. **ISA-szabály:** az `eax` írása a `rax` felső 32 bitjét nullázza.

Az **operandus** egy utasítás bemenete vagy célja. Lehet regiszter, közvetlen konstans vagy memóriaoperandus. A memóriaoperandus nem magát a pointert jelenti, hanem az általa kijelölt tárolóhoz történő hozzáférést.

Az x86 assembly két gyakori szöveges alakja:

Művelet	Intel	AT&T
Regisztermásolás	`mov eax, edi`	`movl %edi, %eax`
Konstans hozzáadása	`add eax, 3`	`addl $3, %eax`
32 bites betöltés	`mov eax, DWORD PTR [rdi]`	`movl (%rdi), %eax`
Indexelt betöltés	`mov eax, DWORD PTR [rdi+rsi*4]`	`movl (%rdi,%rsi,4), %eax`

Intel szintaxisban általában a cél áll elöl, AT&T alakban a forrás. AT&T-ben a `%` regisztert, a `$` közvetlen értéket jelöl; az utasításnév `l` utótagja itt 32 bites operandusméretre utal, nem a C `long` típusára.

A további szemléltetések Intel szintaxisú assemblyrészletek, nem önálló assemblyprogramok.

14.1.3.1 Az utasításnak bájtjai is vannak

Az **utasításkódolás** (*instruction encoding*) az a szabályrendszer, amely az utasítást és operandusait bináris mezőkre képezi. Például:

Utasítás	Egy érvényes x86-64 kódolás, hexadecimális oktettekkel
`add eax, edx`	`01 d0`
`ret`	`c3`

A `01` műveleti kód és a `d0` operandusválasztó mező együtt adja meg az összeadást. A processzor nem az `add` karaktereit hajtja végre.

Az x86-64 utasításhossz változó: nem minden utasítás ugyanannyi oktett. Emiatt a disassemblernek helyes utasításhatárról kell indulnia. A rövidebb kód ugyanakkor nem szükségképpen gyorsabb: a függőségek, memóriahozzáférések és a konkrét mikroarchitektúra is számítanak.

14.1.4 Skaláris művelet, indexelt címzés és elágazás

14.1.4.1 Számolás memóriaelérés nélkül

A `scale` egy lehetséges optimalizált megvalósítása:

```
scale:
    lea eax, [rdi+rdi+3]    # az eredmény alsó 32 bitje: 2*x + 3
    ret
```

ABI-feltétel: ebben a profilban az első `unsigned` argumentum az `edi` regiszterben, a visszatérési érték az `eax` regiszterben jelenik meg.

A `lea`, vagyis *load effective address*, itt megtévesztő nevű: kiszámítja a címzési kifejezést, de **nem olvas memóriát**. A szögletes zárójel önmagában tehát nem bizonyít memóriahozzáférést; az utasítás jelentését is ismerni kell.

ISO C17: az előjel nélküli műveletek az adott típus értéktartományához igazodó moduláris aritmetikát követik. A választott profilban:

$$\text{twice} = (2x) \bmod 2^{32},$$

$$\text{result} = (\text{twice} + 3) \bmod 2^{32} = (2x + 3) \bmod 2^{32}.$$

Bizonyításvázlat. Ha két egész szám különbsége 2^{32} többszöröse, ugyanannak a konstansnak a hozzáadása után is az marad. Ezért a közbülső redukció elhagyható, ha a végén az alsó 32 bitet tartjuk meg. Ez az érvelés az előjel nélküli műveletre épül; signed túlcsordulásra nem alkalmazható.

14.1.4.2 Betöltés és visszaírás

A `bump` lehetséges megvalósítása:

```
bump:
    mov eax, DWORD PTR [rdi+rsi*4]  # load: old = items[i]
    add eax, edx                    # next = old + delta
    mov DWORD PTR [rdi+rsi*4], eax # store: items[i] = next
    ret
```

Itt az ABI szerint `rdi` hordozza az `items`, `rsi` az `i`, `edx` a `delta` értékét. A memóriaoperandus effektív címe:

$$A = B + i \cdot s,$$

ahol B a tömb kezdőcíme, i az index, $s = \text{sizeof}(\texttt{unsigned}) = 4$ C bájt.

Például `i == 3` esetén az eltolás $3 \cdot 4 = 12$ C bájt. A hardveres címképzés azonban nem végez C tömbhatár-ellenőrzést.

A függvény teljes hozzáférési szerződése:

Szempont	Követelmény
Tulajdonos	A hívó kezeli a tömb tárolóját és élettartamát
Kölcsönzött hozzáférés	Az `items` nem tulajdonló pointer, a függvény nem őrzi meg
Tartomány	`i < n`, ahol `n` az elérhető tömbelemek száma
Tároló	Az elem élő, inicializált, írható és megfelelően igazított
Érvénytelenítés	Felszabadítás, sikeres átfoglalás vagy élettartamvég nem történhet a hozzáférés közben
Konkurencia	Nem lehet szinkronizálatlan, ütköző hozzáférés
Hibakimenet	Nincs hibajelzés; a szerződés megsértése UB-hez vezethet

Érvényes hozzáférés mellett az előjel nélküli összeg körbefordulása nem hiba. Ezzel szemben egy hozzáférhetőnek látszó, de már felszabadított cím használatát az assembly nem teszi helyessé.

14.1.4.3 Feltételes ugrás és függvényhívás

A `report_if_nonzero` lehetséges alakja:

```
report_if_nonzero:
    test edi, edi       # állapotjelzők beállítása code alapján
    je .Lzero           # ugrás, ha code == 0
    sub rsp, 8          # hívás előtti veremigazítás ebben az alakban
    call report         # objektumfájlban relokálható hivatkozás
    mov eax, 1
    add rsp, 8
    ret
.Lzero:
    xor eax, eax        # nulla visszatérési érték
    ret
```

A `test` nem módosítja az `edi` értékét, de beállítja többek között a nullaállapot-jelzőt. A `je` ezt vizsgálja. Ez a **feltételes elágazás** (*conditional branch*) egyik alapmintája.

A `call` vezérlést ad át és visszatérési információt hoz létre; a `ret` visszatér. A veremigazítás ABI-követelmény, részleteit a hívási konvencióról szóló egység bontja ki. A `report` normál visszatérése esetén a függvény eredménye egy; ha a külső függvény nem tér vissza, ez a visszatérési út sem valósul meg.

Egy C `if` nem feltétlenül eredményez ugrást: a fordító választhat feltételes adatmozgatást, aritmetikai megoldást vagy teljes egyszerűsítést is.

14.1.5 `-O0`, `-O2` és a megszűnő változók

A `scale` optimalizálatlan alakjában megjelenhetnek külön tárolóhelyek a paraméterhez és a lokális változókhoz. Az alábbi részletből a veremkeret létrehozását és lebontását elhagytuk:

```
    mov DWORD PTR [rbp-20], edi
    mov eax, DWORD PTR [rbp-20]
    add eax, eax
    mov DWORD PTR [rbp-4], eax    # twice tárolása
    mov eax, DWORD PTR [rbp-4]
    add eax, 3
    mov DWORD PTR [rbp-8], eax    # result tárolása
    mov eax, DWORD PTR [rbp-8]
```

Ezek az offsetek szemléltető fordítói választások a megadott x86-64 profilban, nem a C lokális változóinak előírt elrendezései.

`-O2` mellett a teljes számítás elférhet az előző `lea` utasításban. A `twice` és `result` nyelvi változókhoz ekkor nem tartozik külön gépi tárolóhely.

Ez nem azt jelenti, hogy a fordító „megsértette” a forrást. A változók önálló tárolása itt nem szükséges a program megfigyelhető viselkedésének előállításához. A részletes optimalizálási indoklást az as-if szabály tárgyalása adja.

Gyakori tévedés a `-O0` kimenetet közvetlen nyelvi modellnek tekinteni. Az is fordítói megvalósítás, és nem garantál sem egy utasítást forrássoronként, sem minden változóhoz külön stackhelyet. A `volatile` hozzáadása pusztán a változó „láthatóvá tételéért” szintén félrevezető: megváltoztatja a program szerződését.

14.1.6 Forrássorok, DWARF és optimalizált hibakeresés

A **DWARF** debugadat-formátum többek között típusokat, változóleírásokat és gépikód-címekhez kapcsolt forráshelyeket tárolhat. A `-g` nem kapcsolja ki az optimalizálást; a `-O2 -g` értelmes kombináció.

A forrás és a gépi kód kapcsolata azonban nem egy-egy megfeleltetés:

- egy forrássorhoz több utasítástartomány tartozhat;
- több forrássor számítása egyetlen utasításba olvadhat;
- egy változó helye utasítástartományonként változhat;
- egy érték lehet konstansként vagy kifejezésként leírható;
- elegendő debugleírás hiányában a debugger `optimized out` állapotot jelezhet.

Az `objdump -dS` ezért tájékozódási segédlet, nem szemantikai bizonyíték. A `readelf --debug-dump=decodedline` a sortábla cím–forráshely kapcsolatait teszi vizsgálhatóvá. A forrásszöveg megjelenítéséhez a megfelelő fájlnak is elérhetőnek kell lennie.

Egy utólag módosított forrás mellé tett régi objektumfájl megtévesztő diagnózist adhat. Incidensvizsgálatnál ezért a binárist, debugadatokat, pontos forrásrevíziót és fordítási profilt együtt kell megőrizni.

14.1.7 Feladatok

14.1.7.1 Magyarázó feladat

Mit csinál a `mov eax, DWORD PTR [rdi+rsi*4]`? Melyik információ C-szintű, ABI-szintű, illetve utasításkészlet-szintű? Mitől lenne más a `lea eax, [rdi+rsi*4]`?

14.1.7.2 Kódolási feladat

Bővítsd az `inspect.c` fájlt egy `unsigned affine(unsigned x)` függvénnyel, amely `5U * x + 7U` értékét adja vissza. Fordítsd mindkét fordítóval `-O0` és `-O2` mellett. Keress szorzást, összeadást vagy címképző utasítást.

14.1.7.3 Diagnosztikai feladat

Egy kolléga szerint az optimalizált `bump` biztonságosan kezeli az `i == n` esetet, mert a disassemblyben nem látható hibára ugrás, és a debugger olvasható memóriát mutat a számított címen. Cáfold az állítást. Magyarázd meg azt is, miért nem bizonyít hibát, ha az `old` változó `optimized out`.

14.1.8 Megoldások

14.1.8.1 A magyarázó feladat megoldása

A `mov` négy oktettet olvas az `rdi + 4*rsi` címen, és az értéket az `eax` regiszterbe helyezi. A címképzés és az utasítás jelentése ISA-szabály. Az, hogy `rdi` a tömbpointer és `rsi` az index, itt ABI-következmény. A hozzáférés megengedettségét a C objektumhatár-, igazítás- és élettartamszabályai döntik el.

A `lea` csak kiszámítaná a címkifejezés alsó 32 bitjét, memóriaolvasás nélkül.

Ellenőrzési feltétel: külön tudd választani a cím kiszámíthatóságát a címzett objektumhoz való hozzáférés jogosságától.

14.1.8.2 A kódolási feladat megoldása

Az előző fájlhoz illeszthető C17-részlet:

```
unsigned affine(unsigned x)
{
    return 5U * x + 7U;
}
```

A korábbi buildparancsok változatlanul használhatók. Egy lehetséges optimalizált alak:

```
affine:
    lea eax, [rdi+rdi*4+7]
    ret
```

A címkifejezés $x + 4x + 7 = 5x + 7$. A célregiszterben megtartott alsó 32 bit megfelel a profil szerinti előjel nélküli eredménynek.

Ellenőrzési feltétel: nem pontosan ezt az utasítássort kell kapnod. Rögzítsd a tényleges kimenetet, és vezesd le, hogyan állítja elő ugyanazt a moduláris értéket.

14.1.8.3 A diagnosztikai feladat megoldása

Az `i == n` index a tömb utáni pozíciót jelöli. Az oda mutató one-past pointer bizonyos műveletekhez képezhető, de nem dereferálható. A `bump` olvasása és írása ezért megsérti a szerződést; az esethez nincs garantált kimenet.

A hardveresen olvasható lap nem bizonyít élő, megfelelő típusú C objektumot. A hiányzó ellenőrzés pedig nem bizonyít biztonságot: a függvény a hívóra bízza a tartomány helyességét.

Az `old` tárolása szükségtelenné válhatott, illetve a debugadat nem feltétlenül teszi rekonstruálhatóvá.

Ellenőrzési feltétel: a javításnak a memóriaelérés előtt kell biztosítania az `i < n` feltételt, vagy dokumentált hívói invariánssal kell garantálnia azt.

14.1.9 Összefoglaló

Az assembly egy konkrét fordítás megvalósítását mutatja, nem a C nyelv univerzális működését. A `-S` fordítói nézetet, az `objdump` bináris nézetet ad; a DWARF ezekhez kapcsol forrásinformációt. Load, store, branch és call felismerésével már értelmezhető számos kis függvény, de a helyességet továbbra is a nyelvi szerződésből kell levezetni. Az optimalizálás megszüntethet változókat és összevonhat műveleteket, miközben a program előírt viselkedése változatlan marad.

14.2 Hívási konvenció, stack frame és visszatérés

A függvényhívás C-ben értékek átadását és egy eredmény visszaadását jelenti. A nyelv azonban nem írja elő, hogy ezek az értékek melyik processzorregiszterbe vagy memóriacímre kerüljenek. Ezt a konkrét platform **hívási konvenciója** (*calling convention*) szabályozza az alkalmazásbináris interfész, azaz az **ABI** (*Application Binary Interface*) részeként.

Az ABI teszi lehetővé, hogy külön fordított függvények együttműködjenek. A hívó és a hívott félnek ugyanúgy kell értelmeznie a paramétereket, a visszatérési értéket, a regiszterek megőrzését és a verem állapotát. Egy helytelen függvénydeklaráció ezért nem pusztán típushiba: eltérő gépi szerződést is létrehozhat.

Az assembly olvasásához a C forrásból assembly és gépi kód, a variadikus hívások nyelvi szabályaihoz a Variadikus függvények és típusbiztonság részre támaszkodunk.

14.2.1 A vizsgált ABI és a megfigyelés korlátai

A konkrét példák profilja:

Tulajdonság	Feltételezés
Nyelv	ISO C17
Platform	Linux hosted user-space, ELF
Architektúra és ABI	x86-64, System V AMD64 psABI
Adatmodell	LP64; `CHAR_BIT == 8`
Utasításkészlet	Alap x86-64, AVX nélkül
Fordítás	Külön fordítási egység, LTO nélkül

A labor profilja a központi táblában rögzített `lab-gcc` (GCC 13.3.0, GNU binutils 2.42); a `clang` parancsok a `lab-clang` profilhoz tartoznak. Az alábbi parancsok nem linkelnek, ezért libc-verziótól függő működést nem vizsgálnak. A ténylegesen használt binárisok verzióját és targetjét a laborjegyzőkönyvben rögzíteni kell.

Forrás: az ABI-szabályok forrása a forrásjegyzék psABI tétele, elsősorban annak *Data Representation* (adatreprezentáció) és *Function Calling Sequence* (hívási sorrend) szakasza a 3. *Low Level System Information* fejezetben. A következő szabályok ennek összefoglalói, nem dokumentált mérési eredmények. A kódok fordításáról vagy futtatásáról nem állítunk ellenőrzött eredményt; az assemblypélda szemléltető.

Az ABI-szerződés akkor érdekes, amikor tényleges gépi híváshatár marad. Beillesztett függvénynél nincs külön paraméterátadási határ, és a fordító belső függvényváltozatokhoz saját konvenciót is használhat, ha ezzel nem sért külső szerződést.

14.2.2 Egész és lebegőpontos argumentumok

ABI-szabály, a megadott profilban: az egész típusú és pointerargumentumok első hat regiszterhelye sorrendben:

`rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`.

A skalár `float` és `double` argumentumok legfeljebb nyolc SSE-regisztert használhatnak: `xmm0`–`xmm7`. A két regisztersorozat számlálója egymástól független.

Például a következő deklarációban:

```
/* C17 deklarációrészlet, System V AMD64 LP64 profil. */
double combine(long a, double x, long b, double y);
```

`a` az `rdi`, `b` az `rsi`, `x` az `xmm0`, `y` az `xmm1` regiszterhez tartozik. A `double` eredmény helye `xmm0`.

Az „egész argumentum regiszterben” nem jelenti azt, hogy minden típus 64 bites. Egy `int` értékének értelmezéséhez a típus és az ABI további szabályai is szükségesek; a regiszter fennmaradó bitjeire nem szabad önkényes feltevést építeni.

A regiszterhelyek elfogyása után a megfelelő argumentumok a verembeli argumentumterületre kerülnek. Ez nem írja elő, hogy a fordító sorozatos `push` utasításokat használjon: gyakran egyszer foglal helyet, majd megfelelő offsetekre ír.

14.2.3 Struktúrák osztályozása és visszaadása

Összetett értékeknél nem elegendő a paraméter sorszáma. Az ABI az értéket **eightbyte**, vagyis ebben a profilban nyolc C bájtos egységek alapján osztályozza.

A teljes algoritmusban többek között `INTEGER`, `SSE`, `SSEUP`, `X87`, `X87UP`, `COMPLEX_X87`, `NO_CLASS` és `MEMORY` osztály szerepel. A következő egyszerűsítés természetesen igazított, egész típusú, `float` és `double` tagokból álló szokásos struktúrákra használható:

1. A struktúrát nyolcbájtos részekre bontjuk.
2. A tagok az általuk elfoglalt részek osztályozásához járulnak hozzá.
3. Egész típusú vagy pointertag `INTEGER`, `float` vagy `double` tag `SSE` hozzájárulást ad.
4. Ugyanazon részben az `INTEGER` és `SSE` összevonásából `INTEGER` lesz.
5. Az így körülhatárolt struktúrák közül a 16 C bájtnál nagyobbak `MEMORY` átadásúak.

Ez nem általános algoritmus tetszőleges vektortípusra, `long double`-ra vagy fordítóbővítéses elrendezésre. A nem megfelelően igazított tag szintén memóriabeli átadáshoz vezethet.

14.2.3.1 Kidolgozott C példa

Az alábbi `abi_demo.c` önállóan tárgyfájllá fordítható C17 fordítási egység. A `helper` definíciója csak egy későbbi linkeléshez szükséges.

```
#include <limits.h>
#include <stddef.h>

struct pair {
    double weight;
    unsigned long count;
};

struct triple {
    unsigned long a;
    unsigned long b;
    unsigned long c;
};

_Static_assert(CHAR_BIT == 8, "8-bit C byte required");
_Static_assert(sizeof(unsigned long) == 8, "LP64 required");
_Static_assert(sizeof(double) == 8, "8-byte double required");
```

```
_Static_assert(offsetof(struct pair, count) == 8, "layout mismatch");
_Static_assert(sizeof(struct pair) == 16, "layout mismatch");
_Static_assert(_Alignof(struct pair) == 8, "alignment mismatch");
_Static_assert(sizeof(struct triple) == 24, "layout mismatch");

struct pair adjust(struct pair p,
                   unsigned long delta,
                   double factor)
{
    p.weight *= factor;
    p.count += delta;
    return p;
}

struct triple make_triple(unsigned long x)
{
    return (struct triple){x, x + 1UL, x + 2UL};
}

extern unsigned long helper(unsigned long x);

unsigned long twice_step(unsigned long x)
{
    unsigned long first = helper(x);
    return helper(first) + first;
}
```

Az `adjust` példában feltétel, hogy a lebegőpontos szorzás eredménye ábrázolható legyen. Az előjel nélküli összeadás körbefordulása ezzel szemben definiált.

A megadott profilban:

Objektum vagy tag	Offset C bájtban	Méret C bájtban
`pair.weight`	0	8
`pair.count`	8	8
teljes `pair`	—	16
`triple.a`, `triple.b`, `triple.c`	0, 8, 16	egyenként 8
teljes `triple`	—	24

A `pair` első eightbyte-ja `SSE`, második része `INTEGER`. Ebből az `adjust` hívási térképe:

Érték	Hely
`p.weight`	`xmm0`
`p.count`	`rdi`
`delta`	`rsi`
`factor`	`xmm1`
eredmény `weight` tagja	`xmm0`
eredmény `count` tagja	`rax`

A visszaadás tehát nem feltétlenül egyetlen regisztert jelent. Két `INTEGER` rész tipikusan `rax` és `rdx`, két megfelelő `SSE` rész `xmm0` és `xmm1` használatával tér vissza; a példában a két külön regisztercsalád kombinálódik.

Fontos kivétel: ha egy struktúra átadásához nincs elegendő szabad regiszter valamelyik szükséges osztályból, a teljes argumentum a verembe kerül. Nem osztjuk szét tetszőlegesen úgy, hogy egyik fele regiszterben, másik fele a veremben legyen.

A megfigyeléshez:

```
gcc -std=c17 -O2 -Wall -Wextra -Wpedantic \
    -march=x86-64 -mtune=generic -mno-avx -fno-lto \
    -fomit-frame-pointer -S -masm=intel abi_demo.c -o abi_demo.gcc.s

clang -std=c17 -O2 -Wall -Wextra -Wpedantic \
    -march=x86-64 -mtune=generic -mno-avx -fno-lto \
    -fomit-frame-pointer -S -masm=intel abi_demo.c -o abi_demo.clang.s

gcc -std=c17 -O2 -Wall -Wextra -Wpedantic \
    -march=x86-64 -mno-avx -fno-lto \
    -c abi_demo.c -o abi_demo.o
```

14.2.3.2 Nagy visszatérési érték: sret

A 24 C bájtos `triple` visszaadása `MEMORY` eset. A hívó biztosítja az eredmény tárolóját, és ennek címét rejtett első argumentumként, `rdi`-ben adja át. Ezt gyakran **sret**, azaz *structure return* megoldásnak nevezik.

A látható `x` paraméter ezért `rsi`-be kerül. A függvény a kijelölt tárolóba ír, majd annak címét `rax`-ban is visszaadja.

Szerep	Szerződés
Tulajdonos	A hívó kezeli az eredménytároló élettartamát
Kölcsönzött hozzáférés	A hívott függvény a rejtett pointerrel ír bele
Élettartamfeltétel	A tároló a hívás teljes ideje alatt érvényes, megfelelő méretű és igazítású
Érvénytelenítés	Az eredményhez kapcsolódó tároló élettartamának későbbi vége
Hibakimenet	Az ABI nem ad hibakódot hibás tárolóra; a szerződéssértés nem biztonságosan kezelhető

Ez nem változtatja pointeres API-vá a C deklarációt: nyelvi szinten továbbra is struktúraértéket adunk vissza.

14.2.4 Ki őrzi meg a regisztereket?

A **caller-saved** regiszter értékét a hívott fél felülírhatja. Ha a hívónak az érték a hívás után is kell, neki kell gondoskodnia a megőrzéséről.

A **callee-saved** regisztert a hívott fél használhatja, de visszatéréskor helyre kell állítania a belépéskori értéket.

Regisztercsoport	Felelősség
`rax`, `rcx`, `rdx`, `rsi`, `rdi`, `r8`–`r11`	Caller-saved
`rbx`, `rbp`, `r12`–`r15`	Callee-saved
`xmm0`–`xmm15`	Caller-saved ebben a profilban
`rsp`	A visszatéréshez előírt veremállapotot helyre kell állítani

A táblázat az adatregiszterek fő szabályait foglalja össze, nem a teljes processzorállapot ABI-előírásait.

A `twice_step` függvényben a `first` értéknek túl kell élnie a második `helper` hívást. Egy lehetséges, Intel-szintaxisú megvalósítás:

```
# Szemléltető részlet, nem mért fordítókimenet.
twice_step:
    push rbx
    call helper
    mov rbx, rax
    mov rdi, rax
    call helper
    add rax, rbx
    pop rbx
    ret
```

Az `rbx` megőrzi a `first` értékét a második híváson át, mert a `helper` köteles megőrizni az `rbx`-et. Ugyanakkor a `twice_step` is köteles megőrizni saját hívója eredeti `rbx` értékét: ezért kell a `push` és `pop`.

A „callee-saved” tehát nem azt jelenti, hogy a függvény szabadon birtokolja a regisztert, hanem azt, hogy használatáért helyreállítási kötelezettséggel tartozik.

14.2.5 Visszatérési cím és veremkeret

A **veremkeret** (*stack frame*) a függvény végrehajtásához kapcsolódó verembeli területek összefoglaló neve. Tartalmazhat mentett regisztereket, lokális objektumokat, ideiglenes értékeket és további hívások argumentumterületét. Nem kötelező minden függvényhez klasszikus, `rbp`-alapú keretet kialakítani.

14.2.5.1 A 16 bájtos igazítás levezetése

A példák normál, különleges túlzott igazítást nem igénylő hívásaiban a `call` előtt:

$$P \equiv 0 \pmod{16},$$

ahol P az `rsp` értéke. A közeli x86-64 `call` nyolcbájtos visszatérési címet tesz a veremre, így a függvény belépésekor:

$$E = P - 8, \qquad E \equiv 8 \pmod{16}.$$

Ezért belépéskor nem maga az `rsp`, hanem az `rsp + 8` igazított 16 bájtra.

Bizonyításvázlat: a hívó igazított állapotából a visszatérési cím tárolása nyolc bájtot fogyaszt. A példabeli `push rbx` újabb nyolcat von le:

$$E - 8 = P - 16 \equiv 0 \pmod{16}.$$

A két további `call` ezért megfelelő igazítás mellett hajtható végre.

Egyszerű, hét `long` argumentumos hívásnál, belépéskor:

Hely	Tartalom
`[rsp, rsp+8)`	Visszatérési cím
`[rsp+8, rsp+16)`	Hetedik argumentum
Első hat egész argumentumregiszter	Első hat argumentum

A visszatérési cím és a visszatérési érték külön fogalom. Az előbbi a vezérlés folytatási helye; az utóbbi például `rax`, `xmm0` vagy a rejtett eredménytároló tartalma.

14.2.5.2 Prológus, epilógus és frame pointer

A **prológus** a szükséges belépési műveleteket, az **epilógus** a kilépés előkészítését jelenti. Klasszikus megoldás az `rbp` mentése, az aktuális verempozíció hozzárendelése, majd lokális hely foglalása.

A **frame-pointer omission** az állandó keretmutató elhagyása. Ettől még lehet veremkeret, és a fordító által készített unwind-információ alapján a hívási lánc is visszafejthető lehet. Fordítva: a frame pointer jelenléte önmagában nem garantál teljes hibakereshetőséget.

A visszatéréskor felszabaduló veremhely tartalma fizikailag megmaradhat, de ettől a benne volt lokális C objektum élettartama nem hosszabbodik meg. Lokális objektumra mutató pointer visszaadása ettől továbbra is hibás.

14.2.6 Red zone és variadikus hívások

A profilban az aktuális `rsp` alatti 128 bájtos terület a **red zone**. Az ABI ezt védi a megfelelő user-space jelzés- és megszakításkezelés felől történő felülírástól. Levélfüggvény — amely nem hív másik függvényt — ezért bizonyos ideiglenes adatokat az `rsp` módosítása nélkül tárolhat itt.

A red zone viszont **nem őrzi meg az adatokat egy normál függvényhíváson át**. A `call` és a hívott függvény veremhasználata beleírhat ebbe a területbe. Nem szabad továbbá a Linux user-space szabályát kernelkódra vagy más ABI-ra átvinni.

Variadikus hívásnál további rejtett információ jelenik meg: az `al` a vektorregiszterekben átadott argumentumokhoz használt regiszterek számának felső korlátját jelzi, 0 és 8 között. Ebbe a névvel rendelkező és a variadikus argumentumok egyaránt beleszámítanak.

A variadikus argumentumok tehát nem szükségképpen mind a veremben vannak. A megvalósítás regisztermentési területet és verembeli argumentumterületet is használhat a `va_arg` kiszolgálásához. A `va_list` ezért nem helyettesíthető hordozhatóan „a következő veremcímre mutató pointerrel”. A nyelvi default argument promotion szabályok ettől függetlenül érvényesek.

14.2.7 AArch64 és Windows x64: az összehasonlítás határa

Az AArch64 vizsgálatához az adott platformhoz tartozó hivatalos AAPCS64-szabályokat, Windows x64 esetén a Microsoft hivatalos hívásikonvenció-leírását kell alapul venni. Az architektúra neve önmagában nem teljes ABI-profil.

Mivel ezekből nincs megadott, ellenőrzött specifikációs forrásjegyzet, itt nem állítunk fel számszerű regiszter- vagy veremkiosztási összehasonlítást. A helyes összevetés külön ellenőrzi az argumentumosztályozást, a megőrzendő regisztereket, az igazítást, a rejtett eredménypointert és a variadikus hívásokat. A System V táblázatát nem szabad puszta regiszterátnevezéssel más platformra alkalmazni.

14.2.8 Gyakori hibák

- **„A hetedik paraméter verembe kerül."** Csak az argumentumok osztályozása után dönthető el; az egész és SSE-regiszterek külön sorozatot alkotnak.
- **„Egy struktúra mindig pointerrel megy át."** Kis struktúra több regiszterben is átadható; ez továbbra is érték szerinti átadás.
- **„Az `rcx` nálam megmaradt."** A caller-saved érték megmaradása egy konkrét függvényben nem szerződés.
- **„Belépéskor az `rsp` 16-ra igazított."** A vizsgált normál hívásban az `rsp + 8` igazított.
- **„Nincs `rbp`, tehát nincs stack frame."** A keretmutató és a veremhasználat külön kérdés.
- **„A helyes assembly bizonyítja a helyes C programot."** Egy megfigyelt utasítássor nem teszi definiálttá a forrásprogram UB-jét.

14.2.9 Feladatok

14.2.9.1 Magyarázó feladat

Elemezd ezt a deklarációt a megadott profilban:

```
/* C17 deklarációrészlet. */
long pick(long a, double x, long b, double y,
          long c, long d, long e, long f, long g);
```

Add meg az argumentumok és az eredmény helyét. Hol található `g` közvetlenül belépéskor?

14.2.9.2 Kódolási feladat

Bővítsd az `abi_demo.c` fájlt egy `send_pair` függvénnyel. Ez hat `unsigned long`, egy `struct pair` és egy `double` argumentummal hívjon egy külső `consume` függvényt. A hat egész érték legyen 1–6, az utolsó `double` 7.0.

Vezesd le, hogy a `pair` miért kerül teljes egészében a verembe, miközben az utolsó argumentum regiszterben maradhat.

14.2.9.3 Diagnosztikai feladat

Egy kézzel írt változat ezt csinálja közvetlenül belépés után:

```
# Szándékosan hibás ABI-részlet.
call helper
mov rcx, rax
mov rdi, rax
call helper
add rax, rcx
ret
```

Keress két egymástól független hibát, és adj javítást.

14.2.10 Megoldások

14.2.10.1 A magyarázó feladat megoldása

`a`, `b`, `c`, `d`, `e`, `f` rendre `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`. Az `x` és `y` az `xmm0`, illetve `xmm1` regiszterbe kerül. A `g` belépéskor `[rsp+8]` címen található; az eredmény `rax`.

Ellenőrzési feltétel: a két argumentumosztály számlálását külön kell elvégezni, és a veremoffsetben figyelembe kell venni a visszatérési címet.

14.2.10.2 A kódolási feladat megoldása

Az előző fordítási egységhez adható részlet:

```
extern void consume(unsigned long a, unsigned long b,
                    unsigned long c, unsigned long d,
                    unsigned long e, unsigned long f,
                    struct pair p, double tail);

void send_pair(struct pair p)
{
    consume(1UL, 2UL, 3UL, 4UL, 5UL, 6UL, p, 7.0);
}
```

A hat egész argumentum elfoglalja az összes egész argumentumregisztert. A `pair` `INTEGER` részének már nincs hely, ezért a teljes struktúra verembeli átadású. Az ehhez megkezdett SSE-regiszterkiosztás sem marad érvényben; a `tail` az `xmm0`-t használhatja.

Ellenőrzési feltétel: a generált kódban a hívási határon kell vizsgálni a kiosztást. A fordító más utasításssorrendet vagy farokhívást is választhat; az ABI-állapotnak kell egyeznie.

14.2.10.3 A diagnosztikai feladat megoldása

Az első hiba az igazítás: a belépési `rsp` mellett az azonnali `call` nem teljesíti a normál 16 bájtos hívás előtti feltételt.

A második hiba az `rcx` használata tartós tárolóként. A második `helper` hívás ezt felülírhatja.

A korábbi `twice_step` assemblyváltozat mindkettőt javítja: a `push rbx` helyreállítja a hívásokhoz szükséges igazítást és megmenti a hívó `rbx` értékét; az `rbx` pedig megőrzi a köztes eredményt. A `pop rbx` visszaállítja a regisztert és a visszatéréshez szükséges verempozíciót.

Ellenőrzési feltétel: minden `call` előtt helyes igazítás, minden visszatérési úton helyreállított callee-saved állapot szükséges.

14.2.11 Összefoglaló

A C deklarációból a paraméterek típusos osztályozásán keresztül jutunk el a gépi átadáshoz. A System V AMD64 profil külön egész és SSE-regisztersorozatot, összetettérték-osztályozást és előírt veremigazítást használ. A caller-saved értékek megőrzése a hívó, a callee-saved regiszterek helyreállítása a hívott felelőssége. A frame pointer, a red zone és a rejtett eredménypointer ABI-eszköz: egyik sem módosítja a C értékátadási és élettartam-szabályait.

14.3 Optimalizáló, as-if és UB következményei

Az optimalizáló nem a forráskód sorait próbálja gyorsabban végrehajtani, hanem olyan gépi programot állít elő, amely a C nyelvi szerződésén belül ugyanazt a megfigyelhető viselkedést valósítja meg. Ehhez műveleteket tüntethet el, függvényhatárokat bonthat le, és átrendezheti a számításokat. A fejlesztő feladata ezért nem az, hogy egy konkrét utasítássorrendet „belelásson” a C programba, hanem hogy pontosan ismerje az optimalizálás előfeltételeit.

A fejezet az effective type és aliasing, a kiértékelési sorrend, valamint a C forrás és assembly kapcsolatának szabályaira épít. Központi állítása: **egy futtatás eredménye megfigyelés, nem a nyelvi szerződés módosítása**.

14.3.1 Az as-if szabály és a megfigyelhető viselkedés

ISO C17. Az *as-if szabály* az optimalizálási szabadság szokásos összefoglalása: a megvalósítás eltérhet az absztrakt gép lépéseitől, amennyiben megőrzi a szabvány által megkövetelt megfigyelhető viselkedést (*observable behavior*).

Ide tartoznak a szabvány szerinti `volatile` hozzáférések, valamint a fájl- és interaktív I/O-ra előírt követelmények. Egy közönséges lokális változó minden köztes értéke, egy függvény tényleges meghívása vagy a gépi utasítások száma önmagában nem megfigyelhető viselkedés ebben az értelemben. A futásidő üzemi szempontból fontos, de nem olyan változatlan mennyiség, amelyet az as-if szabály megkövetel.

Három alapvető transzformáció:

- **Konstanskiértékelés (*constant folding*):** egy fordításkor ismert kifejezés helyettesítése az értékével.
- **Holt kód eliminálása (*dead code elimination*):** olyan számítás eltávolítása, amely nem befolyásol megkövetelt megfigyelhető viselkedést.

- **Behelyettesítés (*inlining*):** a hívás helyettesítése a függvénytörzs megfelelő változatával. Ez további optimalizálásokat tesz lehetővé.

Az inlininghez nem szükséges az `inline` kulcsszó, és a kulcsszó nem is garantálja a behelyettesítést.

14.3.1.1 Kidolgozott példa és laborprofil

Az alábbi `opt.c` teljes, C17-es program. A számtani minták nem igényelnek konkrét egészszélességet.

Választott laborprofil: Linux, x86-64 System V ABI, LP64, `CHAR_BIT == 8`; a `lab-next` profil (profiltábla). Ezek rögzített vizsgálati verziók, nem itt végrehajtott fordítás adatai. A parancsokhoz a megfelelő helyi toolchaint kell kiválasztani.

```
#include <limits.h>
#include <stddef.h>
#include <stdio.h>

static unsigned bias(unsigned x)
{
    return x + 3u;
}

unsigned folded(void)
{
    unsigned discarded = 9u * 9u;
    (void)discarded;
    return bias(4u * 2u);
}

int checked_inc(int x, int *out)
{
    if (x == INT_MAX)
        return 0;

    *out = x + 1;
    return 1;
}

void add(size_t n, float *restrict dst,
         const float *restrict src)
{
    for (size_t i = 0; i < n; ++i)
        dst[i] += src[i];
}

int main(void)
{
    float dst[] = {1.0f, 2.0f, 3.0f, 4.0f};
    const float src[] = {10.0f, 20.0f, 30.0f, 40.0f};
    int next = 0;
```

```
    int ok = checked_inc(41, &next);
    add(4, dst, src);

    printf("%u %d %d %.0f\n",
           folded(), ok, next, (double)dst[3]);
    return 0;
}
```

A mutatók nem tulajdonolnak tárolót. Szerződésük:

Hozzáférés	Tulajdonos és élettartam	Előfeltétel, érvénytelenítés, hibakimenet
`checked_inc`: `out`	A példában `main` helyi `next` objektuma	Sikerágon élő, írható `int` objektumra mutat. `INT_MAX` esetén nincs írás, az eredmény `0`; egyébként `1`.
`add`: `dst`, `src`	A példában `main` két külön tömbje	Legalább `n` elérhető elem; a cél írható, a forrás olvasható, a hozzáfért tartományok nem fedik egymást.
Mindkét kölcsönzés	Csak a hívás idejére szükséges	A függvények nem őrzik meg a pointereket. Az objektum élettartamának vége érvényteleníti a hozzáférést; érvénytelen memória használatára nincs hibakód, az UB.

Fordítás és assembly előállítása:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O0 opt.c -o opt-O0
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 opt.c -o opt-O2

gcc -std=c17 -O2 -march=x86-64 -mtune=generic \
    -fopt-info-vec-optimized -fopt-info-vec-missed \
    -S opt.c -o opt-gcc.s

clang -std=c17 -O2 -march=x86-64 -mtune=generic \
    -Rpass=loop-vectorize -Rpass-missed=loop-vectorize \
    -Rpass-analysis=loop-vectorize \
    -S opt.c -o opt-clang.s
```

A program nyelvi levezetés alapján várt kimenete `11 1 42 44`; ez nem közölt mérési eredmény.

A `folded` egyszerűsítése lépésenként:

1. A `4u * 2u` értéke `8u`.
2. A `bias(8u)` eredménye `11u`.
3. A `discarded` értéke sem a visszatérési értéket, sem más megfigyelhető hatást nem befolyásol.
4. A függvény ezért viselkedésében egyenértékű a `return 11u;` törzzsel.

Bizonyításvázlat. A számítások eredménye ábrázolható, nincs I/O vagy `volatile` hozzáférés, és a megszüntetett lokális objektumhoz nem marad hozzáférés. A behelyettesítés és az értékegyszerűsítés tehát nem változtatja meg a megfigyelhető eredményt.

Ez nem garantál egy konkrét assemblyalakot. A `-O0` sem jelent minden fordításkori egyszerűsítést kizáró, soronkénti gépi leképezést.

14.3.2 Aliasing, strict aliasing és restrict

Az *alias analysis* azt vizsgálja, hogy két hozzáférés érintheti-e ugyanazt a tárolót. Ha egy írás módosíthat egy korábban beolvasott értéket, a fordító nem használhatja automatikusan újra a régi értéket.

ISO C17. Az effective type hozzáférési szabályait gyakran *strict aliasing* néven említjük. A megengedett típuskombinációkat az aliasingról szóló rész rögzíti. Egy pointerkonverzió önmagában nem teszi szabályossá az új típuson keresztüli dereferálást.

Például egy helyi `float f` objektum `*(int *)&f` alakú olvasása nem válik helyessé attól, hogy az adott targeten a két típus mérete és igazítása megfelelő lenne. Az objektumreprezentáció vizsgálatára használjunk karaktertípusú hozzáférést vagy megfelelő `memcpy`-másolást; ez utóbbi után a célreprezentáció érvényességét is figyelembe kell venni.

A `restrict` másfajta információt ad. Nem típusok közötti különbséget, hanem egy blokk végrehajtása során érvényes hozzáférési ígéretet fejez ki. Leegyszerűsítve: ha egy objektumot egy korlátozott pointeren alapuló hozzáféréssel érünk el, és az objektum módosul is, az érintett hozzáféréseknek követniük kell a `restrict` előírt kapcsolatait.

Ez **nem** azt jelenti, hogy bármely két `restrict` pointerértéknek különböznie kell. A tényleges hozzáférések számítanak.

Az `add` számára könnyen ellenőrizhető, elegendő hívási szerződés:

- a cél- és forrástartomány elegendő elemet tartalmaz;
- a hozzáfért tartományok nem fedik egymást;
- az objektumok a teljes hívás alatt élnek.

Ekkor a célba írás nem módosíthat később beolvasandó forráselemet. `n == 0` esetén a törzs nem végez pointeraritmetikát vagy memória-hozzáférést.

Fordítómegvalósítás. A GCC és Clang `-fno-strict-aliasing` kapcsolója módosítja a típusokon alapuló aliasfeltételezések használatát. Ez nem általános javítás hibás C programokra: nem old meg bounds-, alignment-, lifetime- vagy `restrict`-sértést. A helyes megoldás az érvényes hozzáférési szerződés.

14.3.3 Signed overflow és objektumhatárok

ISO C17. Az előjeles egész túlcsordulása nem definiált viselkedés. Ez nem csupán azt jelenti, hogy az eredmény „furcsa” lehet: az érintett végrehajtás viselkedésére a szabvány nem ír elő követelményt.

Az alábbi C17-részlet hibás, ha minden `int` bemenetre működő túlcsordulásvizsgálatnak szánjuk:

```c
/* UB-s tanítási példa x == INT_MAX esetén. */
int grows(int x)
{
    return x + 1 > x;
}
```

Definiált végrehajtásban:

$$x \leq \text{INT_MAX} - 1.$$

Ebből következik:

$$x + 1 > x.$$

A fordító ezért a teljes törzset helyettesítheti `return 1;` utasítással. Nem köteles az `INT_MAX` esetét hardveres körbefordulásként kezelni. Ha egy konkrét fordítás mégis ilyen viselkedést mutat, az nem teszi szabályossá a programot.

A `checked_inc` ezzel szemben **a művelet előtt** ellenőrzi a veszélyes bemenetet. A feltétel hamis ágában az összeadás bizonyíthatóan ábrázolható; hibánál az eredményobjektum változatlan marad.

Az `unsigned` eset eltér: az összeadás modulo UINT_MAX + 1 történik, ahol az összeg itt matematikai jelölés. Ezért egy `unsigned x` értékre az `x + 1u > x` feltétel hamis lehet, mégpedig `x == UINT_MAX` esetén.

Az objektumhatárok ugyanezt a gondolkodást igénylik. Hibás C17-függvénytörzs-részlet:

```c
/* Hibás: az ellenőrzés túl későn történik. */
int value = a[i];
if (i >= n)
    return -1;
return value;
```

Itt `a` a hívó élő, `n` elemű `int` tömbjének kölcsönzött elérése, `i` és `n` típusa `size_t`. Az élettartamnak a hívás végéig tartania kell. Ha `i >= n`, már az első sor megsérti a hozzáférés szerződését; a későbbi hibakód nem menti meg a végrehajtást.

A javítás az ellenőrzés előrehozása. Az `i < n` feltétel azonban önmagában nem igazolja a pointert: a tároló mérete, élettartama és olvashatósága külön előfeltétel.

14.3.4 Vektorizálás és ciklustranszformációk

A *vektorizálás* több iteráció műveletét végzi el SIMD-utasításokkal. Más gyakori ciklustranszformáció a cikluskibontás, a ciklusinvariáns számítás kiemelése és a ciklusok összevonása.

Az `add` esetében a diszjunkt tartományok miatt nincs iterációk közötti memóriafüggőség: az egyik iteráció írása nem változtatja meg a másik által olvasott értéket. Ez lehetővé teheti több elem párhuzamos feldolgozását.

A lehetőség azonban nem garancia. A fordító költségmodellje figyelembe veheti a ciklushosszt, az igazítást, a maradékelemek kezelését és a rendelkezésre álló utasításkészletet. `restrict` nélkül is készülhet vektoros változat, például futásidejű átfedésvizsgálattal és skalár tartalékággal.

Lebegőpontos redukciónál külön akadály az összeadási sorrend:

$$(a + b) + c \neq a + (b + c)$$

lehet a kerekítés miatt. Egy soros összegzés tetszőleges újracsoportosítása ezért nem következik az as-if szabályból. A `-ffast-math` jellegű kapcsolók további numerikus feltételezéseket engednek; ezek külön szerződést igényelnek.

A laborparancsok *optimization remark* üzenetei az alkalmazott vagy elutasított transzformációkat magyarázzák. Nem helyességi bizonyítványok. Egy „vectorized” üzenet nem igazolja, hogy a hívó betartotta a `restrict` szerződést, és nem jelent mért gyorsulást sem.

14.3.5 Külön fordítás, LTO és PGO

Külön fordításkor a fordító gyakran csak egy külső függvény deklarációját ismeri. Nem látja a törzsét, ezért bizonyos mellékhatásokkal és adatfüggőségekkel számolnia kell. Típusinformáció, ismert könyvtári szerződés vagy más igazolt információ így is segíthet, de a láthatóság korlátozott.

A **link-time optimization**, azaz LTO fordítási egységeken át teszi elérhetővé az optimalizáláshoz szükséges reprezentációt. Lehetővé tehet egységek közötti inliningot, konstanspropagációt és holt kód eltávolítását. GCC- és Clang-környezetben tipikus kapcsolója a `-flto`; a fordítási és linkelési lépésnek, valamint a linkernek együtt kell támogatnia.

A **profile-guided optimization**, azaz PGO reprezentatív futások profilját használja. A gyakori ágak, hívási célok és kódutak alapján módosíthatja például a kódelrendezést vagy az inlining döntéseit.

A profil gyakoriságot jelez, nem nyelvi lehetetlenséget bizonyít. Egy ritka, de definiált ág helyességét továbbra is meg kell őrizni. Elavult vagy nem reprezentatív profil ronthatja a teljesítményt.

LTO és PGO gyakran tesz láthatóvá korábban rejtett UB-következményeket. Ilyenkor nem az optimalizálás „rontotta el” a szerződést: több információ alapján érvényesítette a program által vállalt feltételeket.

14.3.6 A volatile korlátai és gyakori tévedések

ISO C17. A `volatile` minősítés az érintett hozzáféréseket a megfigyelhető viselkedéshez kapcsolja; annak részlete, hogy mi számít hozzáférésnek, implementáció által meghatározott.

Ha a példabeli `discarded` objektum `volatile unsigned` lenne, a hozzáféréseit nem lehetne közönséges holt lokális változóként elhagyni. A jobb oldali `9u * 9u` konstanskiértékelése ettől még megengedett.

A `volatile` ugyanakkor nem:

- teszi atomikussá az összetett műveleteket;
- hoz létre szálak közötti szinkronizációt;
- általános compiler barrier vagy hardveres memóriakorlát;
- javítja ki a signed overflow-t vagy az érvénytelen dereferálást.

A barriermechanizmusokat a következő kapcsolódó rész, a publikalás szabályait a happens-before tárgyalása választja szét.

Gyakori diagnosztikai hiba az optimalizálás kikapcsolását javításnak tekinteni. A `-O0` és `-O2` közötti eltérés hasznos jel, de önmagában sem UB-t, sem fordítóhibát nem bizonyít. Először a nyelvi szerződést, majd a kapcsolók feltételeit, végül a gépi megvalósítást kell vizsgálni. Ugyanígy a warningok hiánya sem bizonyítja a program helyességét.

14.3.7 Feladatok

14.3.7.1 Magyarázó feladat: eltűnő számítások

Indokold meg, miért helyettesíthető a `folded` törzse `return 11u;` utasítással. Mi változik, ha a `discarded volatile unsigned` típusú?

14.3.7.2 Kódolási feladat: ellenőrzött összeadás

Írj C17-ben `checked_add(int a, int b, int *out)` függvényt. Túlcsordulás esetén adjon `0`-t, és ne módosítsa a célobjektumot; siker esetén tárolja az összeget, és adjon `1`-et. Az ellenőrzés maga sem okozhat túlcsordulást.

14.3.7.3 Diagnosztikai feladat: átfedő tartományok

Az `add` függvényt így hívnánk:

```
/* UB-s tanítási hívás; nem helyességi teszt. */
float a[5] = {1.0f, 2.0f, 3.0f, 4.0f, 5.0f};
add(4, a + 1, a);
```

Az `a` tömb a hívó tulajdona, és a teljes hívás alatt él. Mi sérül mégis? Mi lenne a soros ciklus eredménye a `restrict` nélküli változatban? Miért nem bizonyíték a vektorizálási jelentés?

14.3.8 Megoldások

14.3.8.1 Az eltűnő számítások indoklása

A konstansszorzás eredménye `8u`, a `bias` ezt `11u`-ra alakítja. A közönséges `discarded` objektum számítása és olvasása nem befolyásolja a megfigyelhető eredményt.

`volatile` esetén a hozzáféréseket meg kell őrizni a megvalósítás hozzáférési szabályai szerint, de a szorzás továbbra is fordításkor kiértékelhető. Az ellenőrzési feltétel tehát nem egy konkrét utasításszám, hanem a megkövetelt hozzáférések és a visszatérési érték megőrzése.

14.3.8.2 Az ellenőrzött összeadás megvalósítása

C17-részlet, az `opt.c` fájlba illeszthető; a szükséges `<limits.h>` már szerepel benne:

```
int checked_add(int a, int b, int *out)
{
    if (b > 0 && a > INT_MAX - b)
        return 0;

    if (b < 0 && a < INT_MIN - b)
        return 0;

    *out = a + b;
    return 1;
}
```

Pozitív `b` esetén az `INT_MAX - b`, negatív `b` esetén az `INT_MIN - b` ábrázolható. A rövidzáras kiértékelés biztosítja, hogy csak a megfelelő kivonás történjen meg. Az összeadásig eljutva az eredmény már bizonyítottan tartományon belüli.

Az `out` kölcsönzött pointer; sikerágon élő, írható `int` objektumot kell jelölnie. A függvény nem őrzi meg. Ellenőrzési esetek: `(INT_MAX, 1)` és `(INT_MIN, -1)` sikertelen; `(INT_MAX, -1)`, `(INT_MIN, 1)` és `(0, 0)` sikeres. Hibánál egy előzetesen beállított célértéknek változatlannak kell maradnia.

14.3.8.3 Az átfedés diagnózisa

A módosított céltartomány és az olvasott forrástartomány átfed: ugyanazokat az elemeket a két, egymástól független `restrict` hozzáférési útvonalon érjük el. A hívás ezért megsérti a szerződést; nincs garantált eredménye.

A `restrict` nélküli soros ciklus eredménye:

$$[1,\ 3,\ 6,\ 10,\ 15].$$

Minden iteráció az előző iteráció által már módosított forráselemet olvassa. Ez valódi ciklusfüggőség. Ellenőrzéskor ezt a definiált változatot kell a soros szemantikával összevetni; az UB-s változat „helyesnek látszó” eredménye és optimalizálási jelentése nem bizonyít semmit a hívás érvényességéről.

14.3.9 Összefoglaló

Az optimalizálás a definiált viselkedés megőrzésére épül, nem a forrássorok megőrzésére. Az aliasing-, `restrict`-, overflow- és bounds-szabályok ezért közvetlenül befolyásolják a keletkező gépi kódot. Az LTO több információt, a PGO költségbecslést, az optimization remark diagnosztikát ad. Egyik sem helyettesíti az érvényes nyelvi szerződést, és a megfigyelt futási eredmény sem írhatja azt felül.

14.4 Cache, locality, alignment és false sharing

Két, egyaránt $O(n)$ költségű bejárás futásideje nagyságrendileg eltérhet attól függően, hogyan helyezkednek el az adatok, és milyen függőségi láncot alkotnak a memóriaelérések. Egy szolgáltatás metrikatáblájának feldolgozásakor ezért nemcsak az számít, hány összeadást végzünk, hanem az is, mennyi adatot mozgatunk meg ezekért.

A Padding, alignment és offsetof részben az objektumok nyelvi igazítási követelményeit vizsgáltuk; itt ezeket megkülönböztetjük a cache szempontjából kedvező elhelyezéstől. A láncolt listák bejárási szerződését pedig hardveres költségmodellel egészítjük ki. Végig érvényes az optimalizálóról tanult korlát: a forráskód nem utasításjegyzék, és egy assemblymegfigyelés nem nyelvi garancia.

14.4.1 Cache line, TLB és laplokalitás

Hardvermodell. A cache a memória egyes tartományainak gyorsabban elérhető másolatát tartja. Az adatmozgatás és a koherenciakövetés fontos egysége a **cache line**, magyarul cache-sor. Mérete nem C-tulajdonság, és a cache-szintek, processzorok viselkedése sem egységes.

A **térbeli lokalitás** (*spatial locality*) azt jelenti, hogy egy elérés közelében fekvő adatokat is hamarosan felhasználunk. Az **időbeli lokalitás** (*temporal locality*) ugyanannak az adatnak a rövid időn belüli újrafelhasználása. Mindkettő csak az adatok elhelyezésével **és** a hozzáférési mintával együtt értelmezhető.

Legyen a cache-sor mérete B oktett. Egy $D > 0$ oktettes folytonos tartomány, amelynek kezdete az első soron belül o eltolású, összesen

$$N_{\text{line}} = \left\lceil \frac{o + D}{B} \right\rceil$$

sort érint, ahol $0 \le o < B$.

Levezetés. A tartomány vége a kezdő cache-sor elejétől $o + D$ távolságra esik. Ezt kell B méretű egységekkel lefedni; a tört utolsó egység is teljes érintett sornak számít. Ez címtartomány-számítás, nem cache-miss-előrejelzés: a sorok már jelen lehetnek valamelyik cache-ben.

A **TLB** (*translation lookaside buffer*) címfordításokat gyorsítótáraz. Nem az adatcache helyettesítője: más kérdés, hogy ismert-e a virtuális cím fordítása, és más, hogy rendelkezésre áll-e a kívánt adat.

Egyszerű modellben E használható TLB-bejegyzés és P lapméret mellett a lefedett tartomány nagyságrendje $E \cdot P$. Az asszociativitás, az eltérő lapméretek és a többi memóriaelérés ezt módosítják. Sok lap egy-egy adatának olvasása gyenge **laplokalitást** (*page locality*) eredményezhet akkor is, ha kevés hasznos bájtot dolgozunk fel.

A TLB-miss nem azonos a laphibával: szükségessé tehet lapfordítási táblák bejárását anélkül, hogy az operációs rendszer laphibakezelője futna. A nagyobb lapok javíthatják a TLB-lefedettséget, de nem javítják automatikusan az adatcache lokalitását.

14.4.2 Folytonos tömb vagy pointerkövetés?

Folytonos tömbnél a következő elem címe az alaptól és az indextől függ. A processzor és a fordító több későbbi hozzáférést is előkészíthet; a szabályos címfolyam az előbetöltésnek is kedvez.

Láncolt listánál gyakran csak az aktuális csomópont betöltése után ismerjük meg a következő címét. Ez **pointerkövetés** (*pointer chasing*): az egymástól függő memóriaelérések korlátozzák a párhuzamosítást. Véletlenszerűen szétszórt csomópontoknál a cache- és TLB-kihagyások késleltetése összeadódhat.

Szempont	Folytonos tömb	Szétszórt, külön allokált lista
Teljes bejárás	$O(n)$	$O(n)$
Következő cím	Indexből számítható	Betöltött pointertől függ
Allokáció	Tipikusan egy összefüggő tároló	Tipikusan csomópontonként
Térbeli lokalitás	Szekvenciális bejáráskor jó	Az elhelyezéstől függ

Szempont	Folytonos tömb	Szétszórt, külön allokált lista
Érvénytelenítés	Áthelyező növeléskor minden elempointer	Törléskor az érintett csomópontra mutató hozzáférések

A táblázat nem általános győztest hirdet. Poolból kiosztott, bejárási sorrendben elhelyezett lista lehet cache-barát. Egy tömb véletlenszerű indexelése pedig elveszítheti előnyének jelentős részét. Ismert csomópont gyakori törlésekor a lista algoritmikus előnye fontosabb lehet, mint a ritka teljes bejárás.

Mindkét esetben előfeltétel az élő tároló és az érvényes bejárási tartomány. A jobb lokalitás nem tesz helyessé egy dangling pointert.

14.4.3 AoS és SoA: az elrendezést a művelethez válasszuk

Az **AoS** (*array of structures*) rekordok tömbje; a **SoA** (*structure of arrays*) mezőnként külön tömböket használ.

Vegyünk egy metrikarekordot négy `unsigned long` mezővel. A következő számpélda profilja **Linux, x86-64 System V ABI, LP64, nyolcbites C bájt**, továbbá modellfeltevésként 64 oktettes cache-sor. Ez nem minden platformra érvényes layout.

Mező	Eltolás C bájtban	Méret C bájtban
`requests`	0	8
`errors`	8	8
`latency_sum`	16	8
`reserved`	24	8
Teljes rekord	—	32; igazítás: 8

Tegyük fel, hogy csak a `requests` mezőt összegezzük. Cache-sor-határra igazított kezdés és nyolccal osztható n elemszám mellett:

- AoS esetén egy sor két rekordot, de csak 16 oktett hasznos mezőadatot tartalmaz;
- a SoA megfelelő oszlopában egy sor nyolc hasznos értéket tartalmaz.

Az érintett cache-sorok száma ezért

$$N_{\mathrm{AoS}} = \frac{n}{2}, \qquad N_{\mathrm{SoA}} = \frac{n}{8}.$$

A hasznosadat-arány AoS esetén $16/64 = 25\%$, SoA esetén 100%. Hideg cache-t és teljes soronkénti adatmozgatást feltételező modellben ez $32n$, illetve $8n$ oktett.

Ez **nem négyszeres gyorsulás ígérete**. Ha minden rekord összes mezőjét használjuk, az AoS jó lokalitást adhat. Véletlenszerűen kiválasztott teljes rekordok feldolgozásakor a SoA több különálló cache-sort is igényelhet.

14.4.3.1 Fordítható összehasonlítási alap

Az alábbi **ISO C17** program helyességi alap, nem kész benchmark. A választott laborprofil a `lab-next` profil (profiltábla), a fenti x86-64 célplatformon; a példát itt nem fordítottuk és nem mértük. A statikus ellenőrzések a feltételezett layout eltérését fordítási hibává teszik.

A SoA-ból csak a művelethez szükséges oszlopot tároljuk.

```
/* locality.c */
#include <limits.h>
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>

struct sample {
    unsigned long requests;
    unsigned long errors;
    unsigned long latency_sum;
    unsigned long reserved;
};

_Static_assert(CHAR_BIT == 8, "8-bit C byte required");
_Static_assert(sizeof(unsigned long) == 8, "LP64 field size required");
_Static_assert(_Alignof(struct sample) == 8, "unexpected alignment");
_Static_assert(sizeof(struct sample) == 32, "unexpected record size");
_Static_assert(offsetof(struct sample, errors) == 8, "unexpected layout");
_Static_assert(offsetof(struct sample, latency_sum) == 16,
               "unexpected layout");
_Static_assert(offsetof(struct sample, reserved) == 24,
               "unexpected layout");

static unsigned long sum_aos(const struct sample *p, size_t n)
{
    unsigned long sum = 0;
    for (size_t i = 0; i < n; ++i)
        sum += p[i].requests;
    return sum;
}

static unsigned long sum_column(const unsigned long *p, size_t n)
{
    unsigned long sum = 0;
    for (size_t i = 0; i < n; ++i)
        sum += p[i];
    return sum;
}

int main(void)
{
    const size_t n = 1000000;
    struct sample *aos = malloc(n * sizeof *aos);
    unsigned long *requests = malloc(n * sizeof *requests);
```

```
    if (aos == NULL || requests == NULL) {
        free(requests);
        free(aos);
        return EXIT_FAILURE;
    }

    for (size_t i = 0; i < n; ++i) {
        unsigned long value = (unsigned long)(i % 1000);
        aos[i] = (struct sample){value, 0, 0, 0};
        requests[i] = value;
    }

    unsigned long a = sum_aos(aos, n);
    unsigned long b = sum_column(requests, n);
    int status = a == b ? EXIT_SUCCESS : EXIT_FAILURE;
    printf("%lu %lu\n", a, b);

    free(requests);
    free(aos);
    return status;
}
```

```
gcc -std=c17 -O3 -Wall -Wextra -Wpedantic \
    -march=x86-64 -mtune=generic locality.c -o locality-gcc

clang -std=c17 -O3 -Wall -Wextra -Wpedantic \
    -march=x86-64 -mtune=generic locality.c -o locality-clang
```

A két allokáció tulajdonosa a `main`; az összegző függvények csak a hívás idejére kölcsönöznek hozzáférést az élő `[0,n)` tartományhoz. Pointert nem őriznek meg. A `free` érvényteleníti a hozzáférést; allokációs hiba sikertelen kilépést eredményez. Hibás pointer vagy túl nagy átadott elemszám nem ellenőrzött hibakimenet, hanem UB-hez vezethet.

A rögzített méretszorzatok a laborprofilban ábrázolhatók; általános bemeneti elemszámhoz túlcsordulás-ellenőrzés kellene. A várt összeg mindkét oldalon $1000 \cdot (0 + \cdots + 999) = 499\,500\,000$.

14.4.4 Objektumigazítás és false sharing

ISO C17. Az objektum igazítása azt szabályozza, milyen címeken helyezhető el szabályosan. Ettől különböző követelmény, hogy két gyakran írt objektum ne kerüljön ugyanabba a hardveres koherenciaegységbe.

A **cache-koherencia** (*cache coherence*) a különböző magok cache-eiben levő másolatok összhangját biztosító hardvermechanizmus. Íráskor a protokoll más magok példányait érvénytelenítheti, illetve írási jogosultságot adhat át.

False sharing, azaz hamis megosztás akkor jelentkezik, amikor különböző végrehajtók logikailag különálló adatokat használnak, de azok közös koherenciaegységben vannak, és legalább az egyik hozzáférési folyam ír. A költség egységszinten jelentkezik, nem a C-mezők határain.

Az előző laborprofilban, 64 oktettes sorhatáron kezdődő tárolónál:

Elrendezés	Első számláló eltolása	Második eltolása	Elkülönülés
Két egymás utáni `unsigned long`	0	8	Ugyanaz a sor
Ugyanez, a teljes tömb kezdete 64-re igazítva	0	8	Továbbra is ugyanaz
64 C bájtos, 64-re igazított slotok	0	64	Külön sor

A teljes tömb igazítása tehát nem választja szét az elemeit: a **kezdőcím és az elemlépés együtt** számít. A padding javíthatja a helyzetet, de növeli a memória-, cache- és TLB-lábnyomot. Túlzott igazítású dinamikus tárolónál a megfelelő allokációs szerződést is biztosítani kell; a közönséges `malloc` nem általános megoldás minden kiterjesztett igazításra.

Külön skalár objektumok kizárólagos szálankénti írása lehet adatversenymentes, mégis szenvedhet false sharingtől. Fordítva: ugyanazon nem atomikus objektum versengő elérését a padding nem javítja meg. A nyelvi szinkronizációt a Data race, happens-before és publikalás rész tárgyalja.

14.4.5 Prefetch és memória-sávszélesség

Az **előbetöltés** (*prefetch*) megpróbálja az adatot a tényleges felhasználás előtt közelebb hozni a végrehajtóhoz. A hardveres előbetöltő gyakran felismer szabályos címfolyamokat. Pointerkövetésnél a következő cím későn válik ismertté, ezért ez nehezebb.

Az előbetöltés nem ingyenes: sávszélességet és cache-helyet használ. Rossz mintánál haszontalan adatokat tölthet be, vagy kiszoríthat hasznos sorokat. A szoftveres prefetch nem hordozható C-alapeszköz; bevezetését mérésnek kell indokolnia.

Ha egy vizsgált memóriaszinten D oktett adatnak kell áthaladnia, és az adott hozzáférési mintához tartozó effektív sávszélesség W, akkor egyszerű alsókorlát-modellben

$$T \geq \frac{D}{W}.$$

Ez nem teljes futásidőképlet. Függő cache-miss-eknél a késleltetés, sok független hozzáférésnél inkább a sávszélesség korlátozhat. Több szál nem feltétlenül gyorsít, ha a közös memóriaút már telített.

14.4.6 Mérés: nincs univerzális ciklusszám

Az „egy memóriaolvasás ennyi ciklus” állítás körülmények nélkül értelmetlen. Számít a cache-szint, a TLB állapota, a NUMA-elhelyezés, a konkurens terhelés, az órajel és a párhuzamosan kiszolgálható kérések száma.

A példaprogram teljes futásideje nem izolált AoS/SoA-mérés: allokál, inicializál, kiír, és rögzített sorrendben dolgozik. Egy megbízható laborban:

- az allokációt és a lapok első érintését különítsük el a vizsgált bejárástól;
- több munkahalmazméretet használjunk, ne csak egy cache-be férő esetet;
- váltogassuk a változatok sorrendjét, és ismételt mérések eloszlását közöljük;
- rögzítsük a CPU-, NUMA-, fordító- és optimalizálási profilt;
- ellenőrizzük, hogy az optimalizáló valóban meghagyta-e a mérendő munkát.

A visszaadott összeg elfogyasztása szükséges, de nem mindig elég: ismételt, változatlan bemenetű számítást a fordító összevonhat. A `volatile` rutinszerű hozzáadása viszont megváltoztatná a vizsgált műveletet.

False-sharing-mérésnél külön kontroll legyen az egy szál, a közeli és az elkülönített számláló, valamint a magok elhelyezése. A padding utáni gyorsulás önmagában még nem bizonyítja az okot.

14.4.7 Feladatok

1. **Magyarázó:** A fenti modellben $n = 8192$, a kezdőcímek 4096 oktettes laphatáron vannak. Hány cache-sort és lapot érint a kizárólagos `requests`-bejárás AoS és SoA esetén?
2. **Kódolási:** Írj AoS-függvényt, amely mind a négy mezőt összegzi! Miért nem tisztességes ezt az egyetlen SoA-oszlop összegzésével összehasonlítani?
3. **Diagnosztikai:** Két szál külön `unsigned long` tömbelemeket ír. A tömb kezdetét 64-re igazítják, de nincs gyorsulás. Adj elrendezési magyarázatot és ellenőrzési tervet!

14.4.8 Megoldások

14.4.8.1 Cache-sorok és lapok

AoS esetén $8192/2 = 4096$ cache-sort érintünk; a rekordtömb mérete 262 144 oktett, vagyis 64 lap. Minden lapról olvasunk mezőt.

SoA esetén $8192/8 = 1024$ cache-sort és $65\,536/4096 = 16$ lapot érintünk.

Ellenőrzési feltétel a megadott igazítás, sor- és lapméret. Ezek érintett egységek, nem garantált miss-számok.

14.4.8.2 Azonos művelet összehasonlítása

A következő C17-részlet az eredeti fájlba, a `main` elé illeszthető; ugyanazzal a buildparanccsal fordítható:

```
static unsigned long sum_all(const struct sample *p, size_t n)
{
    unsigned long sum = 0;
    for (size_t i = 0; i < n; ++i) {
        sum += p[i].requests;
        sum += p[i].errors;
        sum += p[i].latency_sum;
        sum += p[i].reserved;
    }
    return sum;
}
```

A kölcsönzési szerződés azonos a `sum_aos` szerződésével. Az általános eredmény `unsigned long` szerinti moduláris összeg; matematikai összeghez külön értékkorlát kell.

Érvényes összehasonlításkor a SoA-változatnak ugyanazon négy mezőt kell feldolgoznia. Ellenőrzéshez a tesztadatok mindegyik mezőjét töltsük nem nulla értékekkel, és vessük össze az eredményeket.

14.4.8.3 Az igazítás nem elemenkénti elkülönítés

A két elem eltolása továbbra is 0 és 8: a közös kezdőcím igazítása nem változtatja meg az elemlépést. Külön, megfelelően igazított és méretezett slotokkal kell kontrollváltozatot készíteni.

Ellenőrizzük a `sizeof` és `_Alignof` értékeket, a tényleges címkülönbséget, a szálak elhelyezését és a generált írásokat. Előbb igazoljuk az adatverseny-mentességet; utána több munkamennyiséggel mérjük a közeli és elkülönített változatot. Nem garantált, hogy a false sharing lesz a domináns költség.

14.4.9 Rövid összefoglaló

A memóriaelérés költségét az elrendezés, a hozzáférési minta és a hardverállapot együtt határozza meg. A folytonos tárolás és a SoA gyakran csökkenti a fölösleges adatmozgatást, de nem minden műveletnél előnyös. Az objektumigazítás nyelvi helyességi követelmény; a cache-line-elkülönítés teljesítménytervezési döntés. A modellt számolással, a programot helyességi ellenőrzéssel, a teljesítményállítást pedig dokumentált méréssel kell alátámasztani.

14.5 volatile, compiler barrier és hardver barrier

A `volatile` gyakran azért kerül egy változó deklarációjába, mert a programozó azt szeretné: „a fordító ne tüntesse el ezt a hozzáférést”. Ez bizonyos feladatoknál indokolt, de nem azonos azzal, hogy „egy másik szál biztonságosan megfigyelheti az adatot”, vagy hogy „az eszköz már végrehajtotta az írással kért műveletet”. A három állítás három külön szerződéshez tartozik.

A típusminősítőkről szóló rész a `volatile` nyelvi helyét, az optimalizálóról szóló rész az as-if szabályt, a cache és locality tárgyalása pedig a hardveres környezetet alapozta meg. Most ezeket választjuk szét úgy, hogy a következő fejezet atomikus műveleteit már ne „különleges gépi utasításként”, hanem nyelvi szinkronizációs szerződés részeként értelmezzük.

14.5.1 Három különböző sorrendiség

A forráskód sorrendje, a gépi utasítások sorrendje és a memóriahatások más végrehajtók számára látható sorrendje nem ugyanaz.

Szint	Miről rendelkezik?	Mit nem bizonyít önmagában?
ISO C nyelvi modell	Megengedett kiértékelések, objektumhozzáférések, szálközi szinkronizáció	Konkrét utasítássorozatot
Fordító	A forrásból előállított gépi program szerkezete	Más magok vagy eszközök megfigyelési sorrendjét
CPU és platform	Az utasítások, memóriatípusok és barrier utasítások hardveres hatása	A C forrás adatverseny-mentességét

Fordítói átrendezésnek (*compiler reordering*) nevezzük, amikor a fordító a megengedett viselkedés megőrzésével más sorrendű gépi műveleteket állít elő. Hozzáféréseket összevonhat vagy elhagyhat, értékeket regiszterben tarthat. Ez nem feltétlenül két szomszédos utasítás egyszerű felcserélése.

Sorrenden kívüli végrehajtáskor (*out-of-order execution*) a processzor a gépi utasítások belső végrehajtását ütemezi át. Ettől külön kérdés a memóriahatások megfigyelési sorrendje: például egy írási puffer (*store buffer*) késleltetheti egy tárolás más magok számára való láthatóságát. Memóriasorrendi problémához tehát nem feltétlenül szükséges out-of-order mag.

Az egy szálból megfigyelhető helyes végrehajtás nem követeli meg minden belső művelet forrás szerinti fizikai sorrendjét. A fordító és a processzor is kihasználhatja ezt, de más szabályrendszer alapján.

ISO C17: a nyelvi szálközi modell azt határozza meg, hogy mely hozzáférések rendezettek egymáshoz képest. A később pontosított *happens-before* reláció nem pusztán azt jelenti, hogy a falióra mérve az egyik művelet korábban történt. Szabályosan létrehozott nyelvi kapcsolat szükséges.

Következésképpen a disassemblyben látható „jó sorrend” önmagában nem bizonyítja egy többszálú C program helyességét.

14.5.2 Mit jelent a volatile, és mit nem?

ISO C17: a `volatile` típusminősítővel jelzett objektumhoz tartozó hozzáféréseket az absztrakt gép szabályai szerint kell kezelni. Annak pontos meghatározása, hogy mi számít ilyen hozzáférésnek, implementáció által meghatározott. Hardverkapcsolatnál ezért a fordító dokumentált szerződése is szükséges.

A `volatile` nem általános optimalizálástiltás. Különösen nem biztosítja, hogy:

- a hozzáférés atomikus;
- a környező, nem `volatile` hozzáférések minden kívánt sorrendben maradnak;
- két szál között szinkronizáció jön létre;
- az adat megkerüli a cache-t;
- egy eszköz befejezte az írással kezdeményezett műveletet.

14.5.2.1 Hibás publikálás volatile jelzővel

Az alábbi **hibás C17 tanítási részlet**, nem önálló program. A feltételezés szerint a `publish` és a `consume` két külön szálon, további szinkronizáció nélkül fut:

```
static unsigned payload;
static volatile unsigned ready;

void publish(void)
{
    payload = 42u;
    ready = 1u;
}

unsigned consume(void)
{
    while (ready == 0u) {
    }
    return payload;
}
```

A szándék az, hogy a fogyasztó csak az adat elkészülte után olvasson. A `volatile` miatt az implementáció szokásos hozzáférési szerződése mellett a fordító nem helyettesítheti egyszerűen egyetlen kezdeti olvasással a jelző ismételt lekérdezését. Ebből azonban még nem következik szálközi helyesség.

ISO C17: adatverseny áll fenn, ha különböző szálak egymással konfliktusban álló hozzáférései közül legalább az egyik nem atomikus, és egyik hozzáférés sem történik a másik előtt a nyelvi *happens-before* reláció szerint. Az adatverseny nem definiált viselkedéshez vezet.

Itt már a `ready` írása és olvasása adatversenyt képezhet. A `payload` publikálásához sincs megfelelő szinkronizáció.

Indoklásvázlat:

1. A `volatile unsigned` nem atomikus típus.
2. Ugyanazt a jelzőobjektumot külön szál írja és olvassa.
3. A `volatile` nem létesít szálközi rendezési kapcsolatot.
4. Ezért a program nem kap nyelvi helyességi garanciát attól, hogy a fordító tényleges betöltéseket generál.

Az objektumok statikus tárolási idejűek, élettartamuk a program végrehajtásáig tart; kezelésükért a modul felel. Nincs felszabadítás miatti érvénytelenítés. A hiba itt **nem élettartamhiba, hanem szinkronizációs hiba**, garantált kimenet nélkül.

14.5.3 Compiler barrier és hardveres fence

A **fordítói gát** (*compiler barrier*) az optimalizáló számára korlátozza bizonyos műveletek átrendezését. Hatóköre a konkrét primitív szerződésétől függ: például memóriahozzáférésekre vonatkozhat anélkül, hogy minden regisztermővelet helyét rögzítené.

Egy tisztán fordítói gát nem feltétlenül eredményez gépi utasítást. Emiatt nem várhatjuk tőle, hogy írási puffereket rendezzen, vagy egy perifériával folytatott tranzakciót befejezzen.

A **hardveres memóriagát** (*hardware memory barrier*, *fence*) a processzor architekturális szabályai szerint rendel egymáshoz memóriahatásokat. Nem szükségképpen „mindent megállító” művelet: számíthat a hozzáférés fajtája, a memóriatípus és a megfigyelők köre.

Architekturális példák: x86-64-en az `mfence`, AArch64-en a `dmb` család kapcsolódik memóriasorrendi követelményekhez. Ezek nem felcserélhető receptek. Az AArch64 barrier tartománya és fajtája, illetve mindkét architektúrán a memóriatípus is lényeges. Az utasításválasztást itt nem vezetjük le.

Eszköz	Elsődleges szerep	Önmagában kijavítja a fenti adatversenyt?
`volatile`	Volatile hozzáférések implementáció szerinti kezelése	Nem
Compiler barrier	Fordítói átrendezések korlátozása	Nem
CPU-fence	Meghatározott hardveres memóriahatások rendezése	Nem
Megfelelő atomikus protokoll vagy mutex	Nyelvi szálközi szinkronizáció	Igen, ha a teljes protokoll helyes

Fontos fordított összefüggés: egy helyes nyelvi szinkronizációs művelet nem mindig igényel külön hardveres fence utasítást. Egy adott célarchitektúra meglévő sorrendiségi garanciái elegendők lehetnek. A hiányzó fence utasítás tehát önmagában nem bizonyít hibát.

14.5.3.1 GCC/Clang builtinok: nem ISO API

Fordítóbővítés: a GCC és a Clang `__atomic_*` builtin családja nem az ISO C hordozható felülete. Például:

- a `__atomic_signal_fence(__ATOMIC_SEQ_CST)` fordítói rendezést szolgál a futó szál és az ugyanazon szálon végrehajtott signal handler viszonyában, hardveres gát nélkül;
- a `__atomic_thread_fence(__ATOMIC_SEQ_CST)` szálközi fence szemantikát kér, amelyet a fordító a célarchitektúrához képez le.

Ezek nem szöveges „itt tilos bármit megmozdítani” jelölők, és nem általános MMIO-eszközök. Az elfogadott argumentumokat és a célplatformos leképezést a konkrét fordítókiadás alapján kell ellenőrizni.

Egy ilyen fence beillesztése a hibás `ready` változó mellé nem teszi atomikussá annak hozzáféréseit.

14.5.4 MMIO és közönséges megosztott memória

A **memóriatérképezett I/O** (*memory-mapped I/O*, MMIO) olyan platformmechanizmus, amelyben egy címtartomány elérései eszközműveleteket kezdeményeznek. Az olvasás például állapotot adhat vissza, FIFO-elemet fogyaszthat el vagy törölhet egy jelzést.

Ezzel szemben a szálak által közösen használt közönséges RAM esetében általában C objektumokat olvasunk és írunk.

Tulajdonság	Közönséges megosztott RAM	MMIO
Jelentés	Programadat	Eszközprotokoll része
Cache-elhetőség	Gyakran cache-elhető, koherens memória	Platform- és leképezésfüggő
Olvasási mellékhatás	Közönséges objektumnál nincs eszközmellékhatás	Lehet
Helyes rendezés eszköze	Nyelvi szinkronizáció	Eszköz-, OS- és architektúraszerződés
`volatile` szerepe	Nem helyettesíti a szinkronizációt	Része lehet a hozzáférési megoldásnak

A `volatile` nem változtatja meg a laptábla vagy más hardveres attribútum által meghatározott cache-elhetőséget. Az MMIO sem egyszerűen „mindig cache nélküli memória”: létezhetnek például írásösszevonást engedő tartományok. Ugyanazon eszköz különböző régióihoz is eltérő attribútumok tartozhatnak.

Linux/platform: egy tetszőleges fizikai cím pointerré alakítása nem hoz létre érvényes felhasználói leképezést. Az eszközhöz hozzáférést megfelelő driver- vagy más dokumentált OS-interfésznek kell biztosítania.

A sorrendiség és a befejeződés is külön fogalom. Egy elküldött, de még továbbítás alatt álló eszközírás után akár dokumentált visszaolvasásra vagy státuszvizsgálatra is szükség lehet. Általános CPU-fence nem helyettesíti az eszköz protokollját.

14.5.4.1 Kidolgozott példa: korlátozott számú státuszlekérdezés

A következő C17 fordítási egység **feltételes platformpélda**, nem önmagában teljes MMIO-megoldás.

Előfeltételei:

- létezik `uint32_t`, és a platform 8 bites C bájtot használ;
- `status` megfelelően igazított, érvényes leképezett státuszregiszterre mutat;
- a fordító és a platform dokumentálja a kívánt 32 bites volatile olvasást;
- a regiszter ismételten, káros mellékhatás nélkül olvasható;
- a szükséges eszközrendezést a hívó környezet biztosítja;
- `mask != 0`.

```c
/* mmio_poll.c - C17, platformfüggő MMIO-hozzáférési szerződéssel */
#include <limits.h>
#include <stddef.h>
#include <stdint.h>

#ifndef UINT32_MAX
#error "This example requires uint32_t"
#endif

_Static_assert(CHAR_BIT == 8, "This profile requires 8-bit C bytes");
_Static_assert(sizeof(uint32_t) == 4, "Unexpected uint32_t size");

int mmio_poll_mask(volatile const uint32_t *status,
                   uint32_t mask,
                   size_t attempts)
{
    for (size_t i = 0; i < attempts; ++i) {
        uint32_t sample = *status;

        if ((sample & mask) == mask) {
            return 1;
        }
    }

    return 0;
}
```

A `const` ezen a hozzáférésen keresztül tiltja az írást; nem állítja, hogy az eszköz állapota változatlan. Az egy iterációban végzett egyetlen olvasás után a teszt a helyi `sample` értéken dolgozik.

Szerep	Szerződés
Tulajdonos	A leképezést kezelő hívó komponens
Tároló	Platform által elérhetővé tett eszközregiszter-tartomány
Kölcsönzött hozzáférés	`status`, a teljes hívás alatt érvényesen
Érvénytelenítés	Leképezés megszüntetése vagy eszközhozzáférés visszavonása

Eredmény és hibakimenet	`1`: feltétel teljesült; `0`: próbakeret elfogyott; hibás pointerre nincs garantált hibakód

Legyen `mask = 0x05`, a három olvasott érték pedig szemléltetésként `0x00`, `0x04`, `0x05`. A teszt rendre:

$$0x00 \,\&\, 0x05 = 0x00, \qquad 0x04 \,\&\, 0x05 = 0x04, \qquad 0x05 \,\&\, 0x05 = 0x05.$$

Csak a harmadik minta tartalmazza az összes kért bitet, ezért a függvény akkor ad vissza `1`-et. Ez feltételezett bemeneti sorozat, nem mérési eredmény.

Legfeljebb A olvasás történik, ahol $A =$ `attempts`; a költség $O(A)$, allokáció nincs. Ez **nem időkorlát**: egy eszközolvasás késleltetése nincs a függvény által korlátozva.

Választott fordítási profil: Linux x86-64 System V, LP64, GCC 14.2 vagy a `lab-next` profil (profiltábla) fejlécekkel. Az alábbi parancsok megfelelően telepített fordítókat feltételeznek; nem igazolják az eszközoldali szerződést.

```
gcc -std=c17 -O2 -Wall -Wextra -Wpedantic \
    -S mmio_poll.c -o mmio_poll.gcc.s

clang -std=c17 -O2 -Wall -Wextra -Wpedantic \
    -S mmio_poll.c -o mmio_poll.clang.s
```

A példa nincs itt fordítással vagy futtatással ellenőrizve. Az assembly vizsgálatának célja a hozzáférések leképezése, nem az MMIO-protokoll vagy a szálközi helyesség bizonyítása.

14.5.5 Átvezetés a stdatomic használatához

Az ISO C17 `<stdatomic.h>` fejléce atomikus típusokat, műveleteket és fence függvényeket biztosít. Ezeknél már a **nyelvi szerződésből** indulunk ki, és annak megvalósítása a fordító feladata.

A következő részekben a publikálás kérdése így fogalmazódik át: melyik művelet teszi közzé az adatot, melyik figyeli meg ezt, és milyen kapcsolat rendezi közéjük a közönséges hozzáféréseket? Nem elég annyi, hogy „valahol van egy atomikus változó”.

Az `atomic_signal_fence` és az `atomic_thread_fence` célja sem azonos; az utóbbi használhatóságához is teljes szinkronizációs protokoll szükséges. A részleteket az adatversenyről és publikálásról, majd a C atomikus műveleteiről szóló egység dolgozza fel.

A leggyakoribb hibák javítási iránya röviden:

- **„Volatile, tehát szálbiztos.”** Helyette atomikus protokoll vagy mutex szükséges.
- **„Van fence, tehát nincs adatverseny.”** A hozzáférések nyelvi kapcsolatát kell ellenőrizni.
- **„Volatile, tehát nincs cache.”** A leképezés memóriatípusát kell vizsgálni.
- **„Az assemblyben jó a sorrend.”** Ez megfigyelés, nem teljes helyességi bizonyíték.

14.5.6 Feladatok

14.5.6.1 Magyarázó feladat: mit bizonyít az assembly?

Egy `volatile` változó két olvasásához két betöltési utasítást látsz. Mit bizonyít ez a fordítóról, és mit nem bizonyít a CPU-ról, az eszközről vagy a szálközi helyességről?

14.5.6.2 Kódolási feladat: ellenőrizhető bemeneti hiba

Módosítsd az `mmio_poll_mask` függvényt úgy, hogy nulla maszk esetén `-1` legyen az eredmény, és ekkor ne érje el a regisztert. Mi történjen nulla próbakeretnél?

14.5.6.3 Diagnosztikai feladat: fence a hibás jelző körül

A hibás publikálási példában a fejlesztő mindkét szálon erős thread fence-et helyez el a jelző hozzáférése közelében. A változók típusát nem módosítja. Megszűnt-e az adatverseny? Adj egy helyes javítási irányt.

14.5.7 Megoldások

14.5.7.1 Az assembly megfigyelésének határa

A látott kódrészlet két betöltési utasítást tartalmaz az adott fordítási profil mellett. Ebből nem következik atomikusság, megfelelő cache-attribútum, eszközoldali befejeződés vagy nyelvi szinkronizáció.

Az ellenőrzési feltétel: minden állítást a saját rétegéhez kell kötni. A teljes program helyességét nem lehet pusztán az utasítások megszámolásával igazolni.

14.5.7.2 Nulla maszk kezelése

A függvénytörzs elejére, minden dereferencia elé kerüljön a következő C17 részlet:

```
if (mask == 0) {
    return -1;
}
```

Így a nulla maszk már kezelt bemeneti hiba, nem előfeltétel-sértés. Nem nulla maszk és nulla próbakeret esetén a ciklus nem fut le, az eredmény `0`.

Ellenőrzési feltétel: mindkét esetben nulla regiszterolvasás történjen. Ez nem teszi általában ellenőrzötté a pointer érvényességét.

14.5.7.3 A fence nem alakítja át a változó típusát

Nem szűnt meg az adatverseny. A `ready` továbbra is nem atomikus objektum, és a fence-ek puszta jelenléte nem hoz létre megfelelő kapcsolatot a hozzáférések között.

Helyes irány például a jelző és az adat minden konfliktusban álló hozzáférésének ugyanazzal a mutexszel történő védelme. Másik lehetőség egy megfelelő atomikus publikálási protokoll. Az ellenőrzési feltétel nem egy fence utasítás megjelenése, hanem az összes érintett hozzáférés nyelvi rendezése.

14.5.8 Összefoglaló

A `volatile` hozzáférési minősítő, nem szálközi szinkronizáció. A compiler barrier a fordítót, a hardveres fence meghatározott gépi memóriahatásokat korlátoz. Egyik sem javítja ki automatikusan a C adatversenyét.

MMIO esetén az eszköz és a platform hozzáférési szerződése dönt; közönséges megosztott memóriánál a nyelvi szinkronizációból kell kiindulni. A helyes sorrend: először a szerződés, utána annak fordítói és hardveres megvalósítása.

14.6 ABI- és optimalizálásdiagnosztikai labor

Egy assemblyrészlet önmagában nem magyarázat. Ugyanaz a C függvény eltérő gépi kódot kaphat más optimalizálással, fordítóval vagy célarchitektúrával, miközben a nyelvi szerződése változatlan. A labor célja ezért nem egy „helyes assemblyalak” felismerése, hanem ellenőrizhető következtetések készítése: mit ír elő a nyelv, mit az ABI, és mit választott az adott fordító?

A hívási konvencióról szóló rész argumentum- és visszatérési szabályaira, valamint a reprodukálható labor környezetrögzítésére építünk. A volatile és barrier eszközök itt nem megoldások az optimalizálás megfigyelésére: indokolatlan használatukkal éppen a vizsgált program szerződését változtatnánk meg.

A példákhoz nem tartozik tényleges fordítási vagy mérési eredmény. Az assemblyalakok illusztrációk; a jegyzőkönyvbe csak a saját eszközökkel előállított lelet kerüljön. Ellenőrzött forrásjegyzet hiányában a psABI- és fordítódokumentáció pontos kiadásának, szakaszának hozzárendelése is laborfeladat.

14.6.1 A kísérlet szerződése

Az **ABI-lelet** egy konkrét bináris interfész megfigyelése: például mely regiszterben érkezik egy argumentum. Az **optimalizálási lelet** egy konkrét fordítás eredménye: például eltűnt-e egy hívás vagy ismételt memóriabetöltés. Egyik sem válik pusztán a megfigyeléstől ISO C-garanciává.

A választott referenciaprofil:

Réteg	Laborprofil
Nyelv	ISO C17, explicit `-std=c17`
Elsődleges target	Linux, x86-64 System V, LP64
Adatreprezentáció	`CHAR_BIT == 8`, `sizeof(long) == 8`
Fordítók	GCC 14.2.0 és Clang 19.1.7
Eszközök	GNU binutils 2.43.1, GDB 15.2
Hosted környezet	glibc 2.40
Összehasonlító target	Linux AArch64, LP64, AAPCS64 alapkonvenció

Ez kiválasztott, nem ellenőrzötten telepített környezet. Más kiadások is használhatók, de az eltérést rögzíteni kell. Mentsük el a verziókat, a target triple-t, a teljes parancsokat és a forrás Git-azonosítóját. A linker tényleges verziója is számít: a compiler driver nem feltétlenül a feltételezett linkert indítja.

ISO C17-feltételek: a példában minden bemeneti egész érték legyen $[-1000, 1000]$ között. Így az összeadások és szorzások eredménye reprezentálható. A pointerparaméterek inicializált, élő

`long` objektumokra mutassanak; a módosított objektum legyen írható. Az ABI-vizsgálat nem tesz jogszerűvé hibás pointerhozzáférést.

14.6.2 A vizsgált program

A három fájl szándékosan különíti el a fordítási egységen belül látható és kívül definiált függvényeket.

A `lab.h` tartalma:

```
#ifndef LAB_H
#define LAB_H

struct pair {
    long first, second;
};

struct triple {
    long first, second, third;
};

long args(long a, long b, long c, long d,
          long e, long f, long g);

struct pair pair_make(long x);
struct triple triple_make(long x);

long reload(long *p, long *q);
long reload_disjoint(long *restrict p, long *restrict q);

long add_one(long x);
long local_use(long x);
long external_use(long x);
long across_call(long x);

#endif
```

A `lab.c` tartalma:

```
#include "lab.h"

long args(long a, long b, long c, long d,
          long e, long f, long g)
{
    return a + b + c + d + e + f + g;
}

struct pair pair_make(long x)
{
    return (struct pair){x, x + 1};
}

struct triple triple_make(long x)
```

```
{
    return (struct triple){x, x + 1, x + 2};
}

long reload(long *p, long *q)
{
    long before = *p;
    *q = 7;
    return before + *p;
}

long reload_disjoint(long *restrict p, long *restrict q)
{
    long before = *p;
    *q = 7;
    return before + *p;
}

long add_one(long x)
{
    return x + 1;
}

long local_use(long x)
{
    return 2 * add_one(x);
}

long across_call(long x)
{
    long saved = 3 * x;
    long result = external_use(x);
    return saved + result;
}
```

A `driver.c` tartalma:

```
#include <limits.h>
#include "lab.h"

_Static_assert(CHAR_BIT == 8, "This lab requires 8-bit C bytes");
_Static_assert(sizeof(long) == 8, "This lab requires 64-bit long");

long external_use(long x)
{
    return 2 * add_one(x);
}

int main(void)
{
    long x = 3;
    long y = 9;
```

```
    struct pair p = pair_make(10);
    struct triple t = triple_make(10);

    long shared = reload(&x, &x);
    x = 3;
    long separate = reload_disjoint(&x, &y);

    int ok =
        args(1, 2, 3, 4, 5, 6, 7) == 28 &&
        p.first == 10 && p.second == 11 &&
        t.first == 10 && t.second == 11 && t.third == 12 &&
        shared == 10 && separate == 6 &&
        local_use(4) == 10 &&
        external_use(4) == 10 &&
        across_call(4) == 22;

    return ok ? 0 : 1;
}
```

A pointerek szerződése külön is áttekinthető:

Elem	Tulajdonos és élettartam	Kölcsönzött hozzáférés	Érvénytelenítés, hibakimenet
`x`, `y` tárolója	A `main` automatikus objektumai	A hívások időtartamára	Az objektum élettartamának vége után a dereferencia UB
`reload` paraméterei	Nem vesznek át tulajdont	Azonos objektumot is jelölhetnek	Hibás pointerre nincs ellenőrzött hibakód
`reload_disjoint` paraméterei	Nem vesznek át tulajdont	Ebben a függvényben külön objektumokra kell mutatniuk	Az aliasing-előfeltétel megsértése UB
Struktúraeredmények	A hívó által kezelt értékek	Nem adnak vissza kölcsönzött pointert	Nincs dinamikus allokáció vagy felszabadítás

A program elvárt sikerjelzése a nulla kilépési állapot; ez nem futtatással igazolt eredmény.

14.6.2.1 Fordítási mátrix

Az alábbi Bash-parancsok a külön laborban fordítási artefaktumokat készítenek, nem futtatják a programot. Feltételük a fenti x86-64 Linux eszközlánc.

```
for cc in gcc clang; do
    for opt in 0 2; do
        name="${cc}-O${opt}"
        flags=(-std=c17 "-O${opt}" -g3
               -Wall -Wextra -Wpedantic
               -march=x86-64 -mtune=generic -fno-lto)
```

```
        "$cc" "${flags[@]}" -c lab.c -o "${name}-lab.o"
        "$cc" "${flags[@]}" -c driver.c -o "${name}-driver.o"
        "$cc" -fno-lto "${name}-lab.o" "${name}-driver.o" \
            -o "$name"

        "$cc" "${flags[@]}" -S -masm=intel lab.c \
            -o "${name}-lab.s"
    done
done
```

A linkidejű optimalizálás, azaz LTO kikapcsolása kísérleti kontroll: a `lab.c` fordításakor az `external_use` törzse nem látható. A `-g3` debuginformációt kér, de nem tiltja az optimalizálást és nem garantálja minden változó megjeleníthetőségét.

14.6.3 Argumentumok és struktúraeredmények

ABI-ellenőrzési lista: a következő várakozásokat a választott x86-64 psABI és AAPCS64 dokumentum megfelelő kiadásával kell alátámasztani. Nem variadikus, közönséges egész argumentumú függvényeket vizsgálunk; vektoros és speciális platformkonvenciókat nem.

Eset	x86-64 System V	AArch64, AAPCS64
`args` első hat argumentuma	`rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`	`x0`–`x5`
Hetedik argumentum	Veremben	`x6`
`long` visszatérés	`rax`	`x0`
`pair_make` eredménye	`rax`, `rdx`	`x0`, `x1`
`triple_make` eredménytárolójának címe	Rejtett argumentumként `rdi`	`x8`
`triple_make` explicit `x` argumentuma	`rsi`	`x0`

A két struktúra layoutja a megadott LP64 ABI-profilokban, C bájtban:

Típus	Tagoffsetek	Méret	Igazítás
`struct pair`	0, 8	16	8
`struct triple`	0, 8, 16	24	8

Ezek konkrét típusok konkrét ABI szerinti eredményei. A „kis struktúra mindig regiszterben tér vissza” nem általános szabály: a méret mellett az ABI osztályozási szabályai és a tagtípusok is számítanak.

x86-64-en jelölje S a függvénybelépéskori `rsp` értékét. A `call` által elhelyezett visszatérési cím nyolc C bájtot foglal, ezért ennél az aláírásnál a hetedik argumentum címe:

$$A_g = S + 8.$$

Ez belépéskori összefüggés. Ha a prologue később összesen k bájttal csökkenti `rsp` értékét, akkor ugyanaz a cím az aktuális veremmutatóhoz képest:

$$A_g = rsp + k + 8.$$

A struktúraeredményhez átadott rejtett pointer tárolót jelöl, nem ownership-átadást. A hívó által biztosított eredménytároló nem szükségképpen azonos egy debuggerben látható forrásváltozó végső helyével.

14.6.4 A bináris eszközök közös használata

A különböző eszközök eltérő kérdésekre válaszolnak:

```
readelf -h -W gcc-O2-lab.o
readelf -s -W gcc-O2-lab.o
readelf -r -W gcc-O2-lab.o

nm -S --size-sort gcc-O2-lab.o

objdump -drwC -Mintel gcc-O2-lab.o
objdump -d -Mintel gcc-O2
```

A `readelf` megmutatja az ELF célarchitektúráját, szimbólumait és relokációit. A `nm` segítségével feljegyezhető a függvényszimbólumok mérete. Az `objdump` a gépi utasításokat mutatja; relokálható objektumban a hívás céljának értelmezéséhez a relokációt is olvasni kell.

Az `.s` fájlban még assemblerdirektívák is szerepelnek, ezért a sorok száma nem utasításszám. A szimbólumméret pedig nem végrehajtási idő.

A debuggerben először pontos belépési pontot vizsgáljunk. A `break *args` a függvény gépi belépési címére kér töréspontot; a soralapú megállás már a prologue utánra eshet. A külön interaktív munkamenetben használható parancsok:

- `disassemble /r args`: utasítások és gépi bájtok;
- `info registers rdi rsi rdx rcx r8 r9 rsp`: argumentumregiszterek és veremmutató;
- `x/4gx $rsp`: a belépéskori verem környezetének vizsgálata;
- `si`: egyetlen gépi utasítás végrehajtása;
- `info frame`: a debugger frame-értelmezése.

A debugger memóriavizsgálata nem bizonyítja, hogy ugyanazt a címet a C program szabályosan dereferálhatná.

14.6.4.1 Stack slot, regisztermentés és spill

A **register spill** egy regiszterben tartott vagy oda szánt érték memóriába helyezése a regiszterallokáció során. Nem minden veremírás spill:

- `-O0` mellett a lokális változó kezdettől veremhelyet kaphat;
- egy callee-saved regiszter mentése a hívó állapotának megőrzése;
- a verembe helyezett argumentum az ABI argumentumátadásának része.

Az `across_call` esetében a `saved` értékének a hívás után is rendelkezésre kell állnia. A fordító megőrizheti regiszterben, veremhelyre mentheti, vagy alkalmas alakban újraszámolhatja. A tényleges spill hiánya is érvényes lelet.

Mérjük meg a szimbólumméretet, a látható veremmutató-csökkentéseket, valamint külön számoljuk a regisztermentéseket és lokális memóriaműveleteket. Ez az egyszerű kódvizsgálat nem méri a teljes dinamikus veremhasználatot: x86-64-en például red-zone használata miatt lokális tároló explicit `rsp`-csökkentés nélkül is megjelenhet.

14.6.5 Inlining és aliasing: korrekt ellenpéldák

14.6.5.1 A hívás eltűnése nem nyelvi ígéret

A `local_use` számára az `add_one` törzse látható. A fordító a hívást helyettesítheti a törzzsel, majd egyszerűsítheti:

$$2 \cdot add_one(x) = 2(x + 1) = 2x + 2.$$

Bizonyításvázlat: a megadott bemeneti tartományban minden közbülső signed eredmény reprezentálható, és nincs mellékhatás; az átalakítás megőrzi a visszatérési értéket.

Az `inline` kulcsszó hiánya tehát nem tiltja az inliningot. Megfordítva, jelenléte sem garantál gépi beillesztést. Az `external_use` másik fordítási egységből hívja az `add_one` függvényt, ezért a jelenlegi, LTO nélküli profil jó kontroll.

A hívás hiánya önmagában még nem bizonyít inliningot: más kódban dead-code elimination vagy konstanshelyettesítés is eltüntethet hívást.

14.6.5.2 Azonos típusú pointerek aliasolhatnak

A `reload(&x, &x)` szabályos. Ha kezdetben `x == 3`, akkor:

1. `before` értéke 3;
2. a `*q = 7` ugyanazt az objektumot módosítja;
3. a második `*p` értéke 7;
4. az eredmény $3 + 7 = 10$.

A következő **lehetséges, nem mért** x86-64 Intel-szintaxisú részlet ezt kezeli:

```
# ILLUSZTRACIO: nem konkret compiler-output.
reload:
    mov rax, QWORD PTR [rdi]
    mov QWORD PTR [rsi], 7
    add rax, QWORD PTR [rdi]
    ret
```

A memóriaolvasásokat ne csak `mov` utasítások alapján számoljuk: itt az `add` is olvas memóriát.

A `reload_disjoint` szerződése kizárja a két hozzáférési út érintett objektumainak átfedését. Emiatt a `*q` írás nem változtathatja meg a `*p` értékét, és a második betöltés elhagyható. Külön `x == 3` és `y == 9` objektumokkal az eredmény 6, az utóállapot `x == 3`, `y == 7`.

Azonos pointerekkel meghívni ezt a `restrict` változatot **nem korrekt ellenpélda**, hanem szerződéssértés és UB. A fordító optimalizálását csak definiált viselkedésű bemenetekkel szabad cáfolni.

14.6.6 Cross-target vizsgálat és dokumentációs indoklás

A header és a `lab.c` nem igényel libc-headereket, így megfelelő AArch64 backenddel assembly sysroot nélkül is előállítható:

```
clang --target=aarch64-linux-gnu -std=c17 -O2 -g \
    -Wall -Wextra -Wpedantic -fno-lto \
    -S lab.c -o lab-aarch64.s
```

Ez fordítási kísérlet, nem AArch64-futtatás. A teljes program linkeléséhez megfelelő target runtime, libc és linker is szükséges. A futtatás külön natív gépet vagy megfelelő emulációs környezetet igényelne; emulátoros időmérés nem a target processzor natív teljesítménye.

A dokumentációs indoklás három külön ellenőrzésből álljon:

- **ABI:** argumentumkiosztás, aggregate return, megőrzendő regiszterek és veremkövetelmények;
- **compiler:** optimalizálási kapcsolók, LTO és frame-pointer policy;
- **eszköz:** az ELF-szimbólumméret, relokáció és disassembly megjelenítésének jelentése.

Mindenhez rögzítsük a ténylegesen olvasott dokumentum kiadását és szakaszát. A kutatási kiinduló URL nem bizonyíték.

14.6.7 A jegyzőkönyv és az assembly kommentálása

Függvény	Profil	Szimbólumméret	Releváns memóriaolvasások	Veremműveletek	Magyarázat
`reload`	GCC, `-O0`	saját adat	saját adat	saját adat	lelet és indoklás
`reload`	GCC, `-O2`	saját adat	saját adat	saját adat	lelet és indoklás
`across_call`	Clang, `-O2`	saját adat	saját adat	saját adat	lelet és indoklás

A statikus utasításszám csak a megvizsgált kódalak jellemzője; elágazó kódnál nem azonos egy futás műveletszámával.

Az assembly mellé háromféle kommentet írjunk:

- **MEGFIGYELÉS:** „Ebben az artefaktumban nincs `call` a `local_use` törzsében.”
- **ABI-SZERZŐDÉS:** „Ezen a targeten a skalár `long` eredmény az ABI szerinti eredményregiszterben érkezik.”
- **ISO C17:** „A két megvalósításnak a megadott előfeltételek mellett azonos megfigyelhető viselkedést kell adnia.”

Gyakori hiba a „mindig” szó: „az optimalizált programnak nincs stack frame”, „a pointerdereferencia mindig egy load”, „a struktúra mindig rejtett pointerrel tér vissza”. Ezek helyett nevezzük meg a típust, targetet és artefaktumot.

14.6.8 Feladatok

1. **Magyarázó:** vezessük le a hetedik argumentum helyét x86-64 belépéskor és egy nyolcbájtos `push` után. Magyarázzuk meg, miért más a helyzete AArch64-en.
2. **Kódolási:** egészítsük ki a `main` ellenőrzését azzal, hogy a külön objektumokon végzett `reload_disjoint` után `x == 3` és `y == 7` is teljesüljön. Indokoljuk, miért hibás az azonos pointerekkel végzett „optimalizálási teszt”.
3. **Diagnosztikai:** hasonlítsuk össze mindkét fordító `-O0` és `-O2` artefaktumában a `local_use`, `external_use` és `across_call` függvényeket. Osztályozzuk a veremírásokat, és fogalmazzuk meg, mit nem bizonyít az AArch64 `.s` fájl előállítása.

14.6.9 Megoldások

14.6.9.1 Argumentumhely

Belépéskor a visszatérési cím az S címen, a hetedik argumentum $S+8$ címen van. Egy nyolcbájtos `push` után $rsp = S-8$, tehát az argumentum címe $rsp+16$. AArch64-en a hét egész argumentum még elfér az argumentumregiszterekben. Az ellenőrzés feltétele a pontos függvénybelépési pont és az adott prologue követése.

14.6.9.2 Utóállapot ellenőrzése

A meglévő `ok` kifejezéshez hozzáadható:

```
/* C17; reszlet a driver.c ellenorzeseben. */
x == 3 && y == 7
```

Ez a visszatérési érték mellett a módosítás helyét is ellenőrzi. Az azonos pointeres `restrict` hívás UB volna; annak tetszőleges eredménye nem optimalizálási hiba bizonyítéka.

14.6.9.3 Diagnosztikai következtetés

Nincs előre garantált utasítássor. A helyes megoldás minden állításához artefaktumot és profilt rendel, külön kezeli a lokális tárolást, callee-saved mentést és spillt, valamint elfogadja ezek hiányát is. Az AArch64 assembly bizonyítja, hogy az adott fordító előállított targetkódot; nem bizonyít sikeres linkelést, futást vagy natív teljesítményt.

14.6.10 Összefoglaló

A megbízható ABI-diagnózis három réteget választ szét: nyelvi szerződést, dokumentált bináris interfészt és konkrét fordítói megvalósítást. A reprodukálható labor kontrollált profilokat, definiált viselkedésű programot és egymást kiegészítő bináris eszközöket használ. A cél nem egy assemblyminta bemagolása, hanem olyan jegyzőkönyv, amelyből minden következtetés feltétele és bizonyítékának korlátja visszakereshető.

15 Szálak és atomikus műveletek C-ben

15.1 Szál-életciklus, pthreads és C11 threads

Egy szál indítása nem pusztán új végrehajtási út létrehozása: kölcsönadunk neki adatokat, felelőst rendelünk a befejezésének megfigyeléséhez, és meghatározzuk, mi történjen leállításkor. A legfontosabb kérdés ezért nem az, hogy „melyik függvény indít szálat?”, hanem az, hogy **ki garantálja a szál által használt objektumok élettartamát, és ki várja meg az utolsó hozzáférést**.

A Tulajdonlási szerződés és cleanup ágak elvei itt is érvényesek. A Függvénypointerek, callbackek és context pointer mintája most párhuzamos végrehajtással egészül ki. A fork, exec, pipe és gyermekfolyamat kapcsán tárgyalt folyamatoktól eltérően a szálak ugyanannak a folyamatnak az objektumait érhetik el; külön végrehajtási állapotuk nem jelent külön tulajdonlást.

15.1.1 Két API, közös életciklus

POSIX: a pthreads API deklarációit a `<pthread.h>` biztosítja. A szál belépési függvényének típusa:

```
/* Deklarációs részlet; C17 + POSIX. */
void *worker(void *context);
```

ISO C17: a C11-ben bevezetett, opcionálisan támogatott szálkönyvtár a `<threads.h>` fejlécben található. Ennek belépési függvénye eltérő típusú:

```
/* Deklarációs részlet; C17, támogatott <threads.h> mellett. */
int worker(void *context);
```

A két függvénytípus nem felcserélhető. A típuseltérést függvénypointer-casttal elfedni nem javítás.

Feladat	POSIX pthreads	ISO C17 szálkönyvtár
Szálazonosító típusa	`pthread_t`	`thrd_t`
Indítás	`pthread_create`	`thrd_create`
Befejezés megvárása	`pthread_join`	`thrd_join`
Leválasztás	`pthread_detach`	`thrd_detach`
Belépési függvény eredménye	`void *`	`int`
API sikerjelzése	Az itt tárgyalt függvényeknél `0`	`thrd_success`
Szálmegszakítási API	Van cancellation mechanizmus	Nincs megfelelő standard cancellation API

A továbbiakban **csatlakoztatható szálnak** (*joinable thread*) nevezzük azt, amelynek befejezését egy másik szál join művelettel megvárhatja. A **leválasztott szál** (*detached thread*) befejezésekor a szálkezelő rendszer a szálhoz tartozó erőforrásokat külön join nélkül visszanyerheti.

A belépési függvény visszatérése és a szál erőforrásainak visszanyerése tehát külön esemény. Egy már befejeződött joinable szálhoz is tartozhatnak visszanyerésre váró erőforrások.

15.1.2 Indítás, context és tulajdonlás

A következő pthread-hívás új szálat kér, alapértelmezett attribútumokkal:

```
/* Hívási részlet; C17 + POSIX, a változók deklarációját feltételezi. */
int rc = pthread_create(&thread, NULL, worker, &job);
```

Siker esetén a szál joinable. Az új szál **már a `pthread_create` visszatérése előtt is elkezdhet futni**. Emiatt a `job` szükséges inicializálását az indítás előtt be kell fejezni.

A `void *` paraméter csak pointerérték. Nem másolja le a megcímzett objektumot, és nem hosszabbítja meg annak élettartamát.

Egy stacken létrehozott context teljesen megfelelő, ha az azt tartalmazó blokk nem ér véget a worker utolsó hozzáférése előtt. Ennek egyszerű biztosítéka a blokk elhagyását megelőző sikeres join.

Entitás	Tulajdonos vagy felelős	Élettartami szerződés	Érvénytelenítés
`job` objektum és tárolója	Az indító függvény	A worker befejezéséig él	A létrehozó blokk vége
Worker `context` pointere	Kölcsönzött hozzáférés	Csak élő `job` objektumot érhet el	A `job` élettartamának vége
Joinable szál életciklusa	Kijelölt join-felelős	Sikeres joinnal vagy tudatos detach-csel rendezendő	Az életciklus lezárása
Worker által írt eredmény	A `job` része	Az indító csak sikeres join után olvassa	A `job` élettartamának vége

A `pthread_create` hibája esetén nem jött létre kezelendő szál; a kimeneti szálazonosítót nem szabad joinhoz használni.

15.1.2.1 Kidolgozott példa: kölcsönzött context, sikeres join után olvasott eredmény

A példa minimumkövetelménye C17 és POSIX pthreads. Konkrét reprodukciós célprofilként Linux x86-64 System V ABI, a `lab-next` profil (profiltábla) választható. Ez megadott, nem fordítással ellenőrzött profil; a saját laborban a tényleges verziókat rögzíteni kell.

A `job.count` előfeltétele ebben a példában legfeljebb 1000. Nincs külső cancellation, és csak a `main` kezeli a szálazonosítót.

```
/* thread_job.c - C17 + POSIX pthreads */
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

struct job {
    unsigned long count;
    unsigned long result;
```

```
};

static void *worker(void *context)
{
    struct job *job = context;
    unsigned long sum = 0;

    for (unsigned long i = 0; i < job->count; ++i) {
        sum += i;
    }

    job->result = sum;
    return context;
}

static void fatal_pthread(const char *operation, int code)
{
    fprintf(stderr, "%s: hibakod=%d\n", operation, code);
    exit(EXIT_FAILURE);
}

int main(void)
{
    struct job job = {
        .count = 1000,
        .result = 0
    };
    pthread_t thread;

    int rc = pthread_create(&thread, NULL, worker, &job);
    if (rc != 0) {
        fatal_pthread("pthread_create", rc);
    }

    void *returned = NULL;
    rc = pthread_join(thread, &returned);
    if (rc != 0) {
        fatal_pthread("pthread_join", rc);
    }

    if (returned != &job) {
        fputs("Ervenytelen worker-eredmeny\n", stderr);
        return EXIT_FAILURE;
    }

    printf("%lu\n", job.result);
    return EXIT_SUCCESS;
}
```

Fordítás a külön laborban:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -pthread thread_job.c -o thread_job
```

```
clang -std=c17 -Wall -Wextra -Wpedantic -O2 -pthread thread_job.c -o
 ↪  thread_job-clang
```

A `-pthread` a GCC/Clang platformfüggő driveropciója, nem ISO C nyelvi elem. A fordítási és linkelési lépésben is használni kell; a fenti parancsok mindkettőt elvégzik.

A számítás szerint várt, nem mért eredmény:

$$\sum_{i=0}^{999} i = \frac{999 \cdot 1000}{2} = 499500.$$

Helyességi bizonyításvázlat.

1. A `main` inicializálja a contextet a szál indítása előtt.
2. A worker futása alatt a `main` nem módosítja a bemenetet, és nem olvassa az eredményt.
3. A sikeres `pthread_join` megvárja a worker befejezését, és biztosítja a szükséges szinkronizációt az eredmény ezt követő olvasásához.
4. A `job` blokkja csak ezután ér véget.

A pontos memóriamodellbeli kapcsolatot a Data race, happens-before és publikalás tárgyalja.

A join hibája különösen fontos: ilyenkor a program nem állíthatja, hogy a worker már nem használja a contextet. A példa ezért folyamatmegszakító hibapolitikát alkalmaz. Egy könyvtár nem dönthet önkényesen így; annak az unresolved életciklusért viselt felelősséget is meg kell őriznie vagy át kell adnia.

15.1.3 Hibakód, eredmény és `errno`

POSIX: a `pthread_create`, `pthread_join` és `pthread_detach` hibakódját a függvény visszatérési értéke adja. Nem a szokásos „`-1`, majd `errno`” mintát követik.

Ezért hibás diagnosztikai minta:

```
/* HIBÁS részlet: a perror nem a visszaadott pthread-hibakódot írja ki. */
if (pthread_create(&thread, NULL, worker, &job) != 0) {
    perror("pthread_create");
}
```

A `perror` az aktuális `errno` alapján dolgozik, amely nem feltétlenül kapcsolódik ehhez a hibához. A helyes minta eltárolja a visszatérési értéket; a teljes példában ezért közvetlenül a numerikus hibakód szerepel.

A `pthread_join` esetében két külön eredménycsatorna van:

- a függvény `int` visszatérési értéke a **join művelet státusza**;
- a második paraméteren keresztül kapott `void *` a **worker eredménye**.

A worker nem adhat vissza használható pointert a saját megszűnt automatikus objektumára. Heapobjektum pointerét visszaadhatja, ha az átadott tulajdonlást és a felszabadítás felelősét dokumentáljuk. A példában nincs tulajdonátruházás: ugyanaz a kölcsönzött pointer tér vissza.

15.1.4 C11 threads: támogatás és fordítási próba

ISO C17: a szálkönyvtár opcionális. A `__STDC_NO_THREADS__` makró 1 értéke jelzi a támogatás hiányát. A makró vizsgálata mellett a tényleges compiler–libc–linker összeállítást is ellenőrizni kell.

A következő önálló program egyszerre mutatja az API eltérő eredményszerződését, és használható fordítási/linkelési próbának:

```c
/* c11_probe.c - C17, opcionális ISO szálkönyvtár */
#if defined(__STDC_NO_THREADS__) && __STDC_NO_THREADS__ == 1
#error "Az implementacio nem tamogatja a C szalkonyvtarat"
#endif

#include <stdlib.h>
#include <threads.h>

static int worker(void *context)
{
    (void)context;
    return 7;
}

int main(void)
{
    thrd_t thread;

    int status = thrd_create(&thread, worker, NULL);
    if (status != thrd_success) {
        return EXIT_FAILURE;
    }

    int worker_result = 0;
    status = thrd_join(thread, &worker_result);
    if (status != thrd_success) {
        exit(EXIT_FAILURE);
    }

    return worker_result == 7 ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

A Linuxos GCC/Clang célprofilhoz:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -pthread c11_probe.c -o c11_probe
```

Itt nincs kölcsönzött contextobjektum: a callback nem dereferálja a null pointert. A szál életciklusáért a `main` felel. A sikertelen indítás hibakimenet, a join hibája folyamatleállítást eredményez.

A `thrd_create` státusza például `thrd_success`, `thrd_nomem` vagy `thrd_error` lehet. Ezek nem POSIX hibakódok, és nem `errno`-értékek. A `thrd_join` által átvett `7` pedig nem könyvtári státusz, hanem a worker alkalmazásszintű eredménye.

A sikeres fordítás és linkelés csak a szükséges felület elérhetőségét igazolja. A működés vizsgálata külön futtatási lépés; itt ilyen ellenőrzést nem feltételezünk.

15.1.5 Thread-local storage: szálanként külön objektum

ISO C17: a `_Thread_local` tárolásiosztály-specifikátor szálhoz kötött tárolási idejű objektumot jelöl. Minden szálnak saját példánya van, amelynek élettartama az adott szál végrehajtásának idejéhez kötődik.

```
/* Részlet; C17, TLS-t támogató compiler/target.
   A számláló túlcsordulása unsigned moduloaritmetika. */
static _Thread_local unsigned long completed_jobs;

static void record_completion(void)
{
    ++completed_jobs;
}
```

A `static` itt fájlszinten belső nyelvi kapcsolást ad; a `_Thread_local` ettől függetlenül a tárolási időt határozza meg.

A szálankénti számláló nem közös összesítő. Attól, hogy minden worker növeli a saját példányát, a főszál példányában nem jelenik meg az összeg.

A TLS-cím továbbadása sem általános megosztási technika:

- másik szálból történő közvetett hozzáférés C17-ben implementáció által meghatározott;
- a tulajdonos szál befejeződése után az objektum élettartama véget ér;
- a `_Thread_local` önmagában nem biztosít alkalmazási erőforrásokat felszabadító destruktort.

Például egy TLS-pointerben tárolt heapobjektum felszabadításáról továbbra is gondoskodni kell. Az explicit szálkilépési destruktorokhoz a pthread kulcsalapú TLS vagy a C szálkönyvtár `tss_*` felülete külön mechanizmust nyújt.

15.1.6 Join, detach és folyamatleállítás

A detach nem leállítási kérés, és nem várakozás. Csak azt változtatja meg, hogyan rendeződik a szálkezelési erőforrás életciklusa.

Művelet	Megvárja a befejezést?	Felszabadítja az alkalmazás contextjét?
Sikeres `pthread_join`	Igen	Nem
Sikeres `pthread_detach`	Nem	Nem
Worker visszatérése	A worker befejeződik	Csak saját explicit cleanup esetén
Folyamat megszűnése	Nem szabályos join-sorozat	Az alkalmazási lezárási protokoll nem garantált

Egy szálhoz egyértelmű join-felelőst rendeljünk. Azonos szál párhuzamos joinolása, saját magunk joinolása, illetve már leválasztott vagy lezárt életciklusú szálazonosító használata nem elfogadható protokoll. A `pthread_t` nem hordozható egész szám, és nem tartós, korlátlanul újrahasználható azonosító.

A rendezett leállítás tipikus sorrendje:

1. Új munka fogadásának leállítása.

2. Szinkronizált leállítási állapot beállítása.
3. A várakozó workerek felébresztése.
4. A folyamatban lévő munka és saját erőforrások lezárása.
5. Minden joinable worker joinolása.
6. A közös context és a szinkronizációs objektumok felszabadítása.

A stopjelző nem lehet szinkronizálatlan közös `bool`, és a `volatile` sem teszi megfelelővé. A megoldást a Mutex, condition variable és producer–consumer rész építi fel.

POSIX folyamatéletciklus: a `main` visszatérése vagy az `exit` a folyamatot állítja le, nem csak a hívó szálat. A detached workerek sem maradnak életben. A `pthread_exit` ezzel szemben csak a hívó szálat fejezi be, de nem helyettesíti a join-felelősséget, és nem hosszabbítja meg a megszűnő automatikus contextobjektumok élettartamát.

15.1.7 Cancellation és cleanup: korlátozott eszközök

POSIX: a `pthread_cancel` megszakítási kérelmet küld. A sikeres visszatérés nem bizonyítja, hogy a célpont már befejeződött.

Az alapértelmezett, halasztott (*deferred*) cancellation engedélyezett állapotban meghatározott cancellation pointokon léphet érvénybe. Egy ilyen pont nélküli számítás nem feltétlenül reagál rá. Az aszinkron cancellation tetszőleges ponton szakíthatja félbe az invariánsok helyreállítását; általános alkalmazáskódban kerülendő.

A `pthread_cleanup_push` és `pthread_cleanup_pop` cleanupkezelőket rendelhet az aktuális szálhoz. Ezek:

- fordított regisztrációs sorrendben futnak cancellation vagy `pthread_exit` hatására;
- a `pop` paraméterével explicit módon is végrehajthatók;
- makróként is megvalósíthatók, ezért lexikálisan páros használatot igényelnek;
- nem tesznek automatikusan cancellation-biztossá egy erőforrás-szerzési protokollt.

A belépési függvény közönséges visszatérése nem futtatja automatikusan a még regisztrált cleanupkezelőket. A cleanup-régiót átugró `return` vagy `goto` sem megfelelő használat.

Sikeres join esetén a megszakított pthread eredménye `PTHREAD_CANCELED` lehet; ez nem joinhibakód. Ráadásul maga a `pthread_join` is cancellation point, ezért a join-felelős megszakíthatóságát is kezelni kell.

Szolgáltatásokban általában könnyebben indokolható a kooperatív shutdown: a worker ismert, helyes állapotban észleli a leállítási kérést, majd maga végez cleanupot.

15.1.8 Feladatok

15.1.8.1 Élettartam magyarázata

Egy függvény lokális `struct job` címével indít workert, sikeresen detach-eli a szálat, majd visszatér. Miért nem elegendő a detach? Adj két javítást.

15.1.8.2 Több worker indítása

Bővítsd a pthread-példát két, külön contextet használó workerre. Ha a második indítás sikertelen, az első szál életciklusát akkor is szabályosan rendezd.

15.1.8.3 Hibadiagnosztika

Egy alkalmazás a `pthread_create` hibáját `perror` segítségével írja ki, sikeres indítás után pedig join nélkül kiolvassa a worker eredményét. A worker végtelen számítási ciklusát `pthread_cancel` hívással próbálja leállítani. Azonosíts három külön hibát vagy hibás feltételezést.

15.1.9 Megoldások

15.1.9.1 Élettartam magyarázata

A detach csak a szálkezelési erőforrásokra vonatkozik. A függvény visszatérése megszünteti a lokális context élettartamát, miközben a worker még hozzáférhet.

Két javítás:

- maradjon joinable a szál, és a függvény visszatérés előtt sikeresen joinolja;
- legyen külön allokált context, amelynek tulajdonlását sikeres indításkor átadjuk a workernek.

A második változatban sikertelen indításkor az indító szabadít fel; siker esetén a worker. Ellenőrzési feltétel: minden hibautat is figyelembe véve pontosan egy felszabadítás legyen, az utolsó hozzáférés után.

15.1.9.2 Több worker indítása

A kiinduló programba illeszthető vezérlési részlet, változatlan workerrel és hibakezelővel:

```
/* Részlet; C17 + POSIX, a thread_job.c definícióival. */
struct job jobs[2] = {
    { .count = 1000, .result = 0 },
    { .count = 1000, .result = 0 }
};
pthread_t threads[2];
size_t started = 0;
int create_error = 0;

for (size_t i = 0; i < 2; ++i) {
    int rc = pthread_create(&threads[i], NULL, worker, &jobs[i]);
    if (rc != 0) {
        create_error = rc;
        break;
    }
    ++started;
}

for (size_t i = 0; i < started; ++i) {
    int rc = pthread_join(threads[i], NULL);
    if (rc != 0) {
        fatal_pthread("pthread_join", rc);
    }
}

if (create_error != 0) {
```

```
    fprintf(stderr, "pthread_create: hibakod=%d\n", create_error);
    return EXIT_FAILURE;
}
```

A `main` tulajdonolja a contexttömböt; minden worker csak a saját elemét kölcsönzi. A tömb a joinokig él. Ellenőrzési feltétel: pontosan `started` érvényes szálazonosítót joinolunk, és indítási hiba miatt nem hagyjuk el idő előtt a context blokkát.

15.1.9.3 Hibadiagnosztika

Az első hiba a téves hibacsatorna: a pthread visszaadott hibakódját kell használni, nem az `errno` aktuális értékét.

A második a szinkronizálatlan eredményolvasás: megfelelő rendezés nélkül ütközhet a worker írásával. Ebben a mintában a sikeres join utáni olvasás a javítás.

A harmadik a cancellation téves értelmezése: a kérelem nem azonnali befejezés, és cancellation point nélküli ciklus nem feltétlenül reagál. Ellenőrzési feltétel: a leállításnak legyen dokumentált észlelési pontja, majd igazolt befejezése.

15.1.10 Rövid összefoglaló

A szálkezelés alapja az élettartam és a felelősség: az indító érvényes contextet ad, a worker betartja a hozzáférési szerződést, egy kijelölt szereplő pedig rendezi a befejezést. A join megvárja a szálat; a detach nem. A pthread függvények közvetlen hibakódot, a C szálkönyvtár saját státuszokat használ. A TLS szálanként külön objektumot ad, de nem automatikus erőforrás-cleanupot. A kiszámítható shutdown elsődleges eszköze a kooperatív leállítás és az igazolt utolsó hozzáférés.

15.2 Data race, happens-before és publikalás

A szálbiztos program helyességét nem az igazolja, hogy „a tesztekben mindig a jó értéket kaptuk”, hanem az, hogy minden megosztott hozzáféréshez meg tudjuk nevezni a szükséges rendezési és élettartam-garanciát. A C memóriamodelljében ezeket a garanciákat elsősorban a *happens-before* relációval fejezzük ki.

Az előző részben megismert szálindítás és csatlakozás adja az életciklus keretét. A sequencing szabályai egy szálon belül rendezik a kiértékeléseket; most ehhez kapcsolunk szálak közötti rendezést. A `volatile` korlátait is felidézzük: megosztott adatból önmagában nem készít szinkronizált adatot.

15.2.1 Ütköző hozzáférések és adatverseny

ISO C17: két kiértékelés ütköző hozzáférést (*conflicting access*) jelent, ha ugyanazt a memóriahelyet érintik, és az egyik módosítja, a másik pedig olvassa vagy módosítja azt. Két puszta olvasás tehát nem ütköző.

A memóriahely itt nyelvi fogalom, nem cache line. Skalár objektum, illetve bizonyos szomszédos bitmezők együttese alkot memóriahelyet. Egy struktúra különálló, nem bitmező tagjai külön memóriahelyek lehetnek akkor is, ha a hardveren ugyanabba a cache line-ba kerülnek.

Egy végrehajtásban **adatverseny (*data race*)** van, ha:

- különböző szálak két ütköző műveletet hajtanak végre;
- legalább az egyik művelet nem atomikus;
- egyik művelet sem előzi meg a másikat a *happens-before* reláció szerint.

Az adatverseny C17-ben nem definiált viselkedést, UB-t okoz. Nem csupán elveszett növelésre vagy régi értékre kell számítani: a szabvány a program eredményére nem ad garanciát.

Tegyük fel, hogy egy `static unsigned counter;` objektumon két dolgozó egyaránt `++counter` műveletet végez. Ez nem atomikus növelés. A kifejezés értékének meghatározásához olvasás, a módosításához írás tartozik:

A szál eseményei	B szál eseményei
A_R: `counter` olvasása	B_R: `counter` olvasása
A_W: `counter` módosítása	B_W: `counter` módosítása

A táblázat nem gépiutasítás-lista és nem garantált kimenetű végrehajtás. A konfliktusokat mutatja: például A_W és B_R, valamint A_W és B_W között megfelelő szinkronizáció nélkül nincs szükséges rendezés.

Az objektum tárolója statikus, élettartama végig tart; itt tehát nem dangling hozzáférés, hanem adatverseny a hiba. Egyetlen processzormag, ritka végrehajtás vagy a szálak közé tett késleltetés sem változtatja ezt meg.

15.2.2 A rendezési relációk és a végrehajtási gráf

A végrehajtási gráf (*execution graph*) csúcsai műveletek, élei pedig nyelvi vagy szinkronizációs kapcsolatok.

15.2.2.1 Sequenced-before

A *sequenced-before*, röviden *sb*, egy szálon belüli rendezés. Például az alábbi külön teljes kifejezések esetén az első értékadás megelőzi a másodikat:

```
/* ISO C17-részlet; x és y ezen a ponton csak a hívó szálé. */
x = 10;
y = 20;
```

Ez nem jelenti azt, hogy minden összetett C-kifejezés részkifejezései balról jobbra értékelődnek ki.

15.2.2.2 Synchronizes-with

A *synchronizes-with*, röviden *sw*, meghatározott szinkronizációs műveletek között létesül.

POSIX: megfelelően használt mutex esetén az egyik szál feloldása és ugyanazon mutex későbbi sikeres megszerzése biztosítja a szükséges szálak közötti memória-szinkronizációt. Két különböző mutex használata önmagában nem ad ilyen kapcsolatot.

Az ISO C17 saját szálkönyvtára hasonló célt szolgáló műveleteket kínál, de a `pthread_*` függvények POSIX API-k, nem ISO C függvények.

15.2.2.3 Happens-before

A *happens-before*, röviden *hb*, a helyesség szempontjából lényeges megelőzési reláció. Az itt használt, mutexes részmodellben az *sb* és *sw* élekből tranzitív módon vezetjük le.

Legyen:

- W: megosztott adat írása;
- U: a mutex feloldása;
- L: ugyanazon mutex későbbi sikeres megszerzése;
- R: az adat olvasása.

Ekkor:

$$W \xrightarrow{sb} U \xrightarrow{sw} L \xrightarrow{sb} R$$

amiből:

$$W \xrightarrow{hb} R.$$

Bizonyításvázlat: az írás a feloldás előtt történik a kiíró szál programrendjében. A feloldás és a későbbi megszerzés szinkronizál. Az olvasás a megszerzés után történik az olvasó szálban. Ezek összekapcsolása rendezi az ütköző hozzáféréseket.

A „később” itt nem egyszerű falióra-megfigyelés. Attól, hogy egy napló szerint A korábban futott, még nem azonosítottunk *hb* élt.

A szál-életciklus is ad rendezést: a létrehozás előtti inicializálás átadható az új szálnak, a sikeres `pthread_join` pedig lehetővé teszi a befejezett szál eredményeinek megfigyelését. Két versengő dolgozó utólagos csatlakoztatása azonban nem rendezi visszamenőleg a dolgozók egymás közötti hozzáféréseit.

15.2.3 Adatverseny és logikai race condition

A magasabb szintű *race condition*, vagyis logikai versenyhelyzet azt jelenti, hogy az eredmény a műveletek kedvezőtlen összefonódásától függ, és sérül az alkalmazás szerződése. Ez adatverseny nélkül is bekövetkezhet.

Legyen a kapcsolatok számlálója kezdetben 9, a korlát pedig 10. Minden hozzáférés ugyanazon mutex alatt történik, de az ellenőrzés és a növelés külön kritikus szakasz:

Lépés	Mutex alatt végzett művelet
A ellenőriz	`active < 10` igaz
B ellenőriz	`active < 10` igaz
A lefoglal	`active` értéke 10 lesz
B lefoglal	`active` értéke 11 lesz

Nincs adatverseny, mert a számláló hozzáférései rendezettek. A kapacitáskorlát mégis sérül.

A javítás nem „még több zárolás”, hanem a megfelelő műveleti egység: az ellenőrzésnek és a foglalásnak **egyetlen kritikus szakaszban** kell lennie. A memóriahelyek védelme és az alkalmazási invariáns védelme két külön bizonyítási feladat.

15.2.4 Megosztott pointer publikálása

A publikálás (*publication*) azt jelenti, hogy egy előkészített objektumot más szál számára hozzáférhetővé teszünk, és biztosítjuk az inicializálás megfelelő megfigyelhetőségét.

Két külön kérdést kell megválaszolni:

1. Rendezett-e a pointert tároló objektum olvasása és írása?
2. Rendezett-e a megcímzett objektum inicializálása és későbbi használata, és él-e még az objektum?

Az alábbi programban a főszál publikál egy konfigurációt. A dolgozó egyszer próbálja kiolvasni; megengedett eredmény, hogy még nincs konfiguráció. Ez nem várakozási protokoll.

Példaprofil: ISO C17 és POSIX threads, Linux x86-64 System V ABI, LP64. A külön laborhoz választott eszközverziók: a `lab-next` profil (profiltábla). A példa itt nem fordított vagy futtatott mérési eredmény; a helyi verziókat rögzíteni kell.

A `publication.c` teljes tartalma:

```
#define _POSIX_C_SOURCE 200809L
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

struct config {
    unsigned port;
    unsigned workers;
};

static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
static const struct config *published = NULL;

static void check(int rc, const char *operation)
{
    if (rc != 0) {
        fprintf(stderr, "%s: pthread hibakod=%d\n", operation, rc);
        _Exit(EXIT_FAILURE);
    }
}

static void *reader(void *unused)
{
    (void)unused;

    check(pthread_mutex_lock(&mutex), "lock");
    const struct config *p = published;
    check(pthread_mutex_unlock(&mutex), "unlock");

    if (p != NULL) {
        printf("port=%u workers=%u\n", p->port, p->workers);
    } else {
        puts("Meg nincs publikalt konfiguracio.");
    }
```

```
    return NULL;
}

int main(void)
{
    struct config cfg;
    pthread_t thread;

    check(pthread_create(&thread, NULL, reader, NULL), "create");

    cfg.port = 8080;
    cfg.workers = 4;

    check(pthread_mutex_lock(&mutex), "lock");
    published = &cfg;
    check(pthread_mutex_unlock(&mutex), "unlock");

    check(pthread_join(thread, NULL), "join");

    published = NULL;  /* Mar nincs masik hozzafero szal. */
    check(pthread_mutex_destroy(&mutex), "destroy");
    return EXIT_SUCCESS;
}
```

Fordítás a megadott verziókra beállított fordítóparancsokkal:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g -pthread publication.c -o
 ↪  publication-gcc
clang -std=c17 -Wall -Wextra -Wpedantic -O2 -g -pthread publication.c -o
 ↪  publication-clang
```

A hibapolicy szándékosan fail-fast: pthread-hiba esetén az egész folyamat befejeződik. Különösen fontos, hogy sikertelen `join` után ne térjünk vissza úgy, mintha a dolgozó biztosan befejeződött volna. A pthread-függvények visszatérési hibakódját vizsgáljuk, nem az `errno` értékét.

15.2.4.1 A publikálás levezetése

Ha a dolgozó nem null pointert olvasott, a releváns gráf:

Főszál	Kapcsolat	Dolgozó
I: `cfg` inicializálása	$I \xrightarrow{sb} P$	
P: `published = &cfg`	$P \xrightarrow{sb} U$	
U: mutex feloldása	$U \xrightarrow{sw} L$	L: mutex megszerzése
		Q: pointer kiolvasása
		mutex feloldása
		R: konfiguráció olvasása

Mivel $L \xrightarrow{sb} Q \xrightarrow{sb} R$, következik:

$$I \xrightarrow{hb} R.$$

A konfiguráció olvasása lehet a kritikus szakaszon kívül, mert publikálás után **senki nem módosítja**. A `const` pointer ezt a hozzáférési szándékot jelzi, de önmagában nem akadályozná meg, hogy a főszál a saját `cfg` objektumát módosítsa.

Ha a dolgozó szerzi meg először a mutexet, null pointert olvas, és nem érinti az inicializálás alatt álló konfigurációt.

15.2.4.2 Tulajdonlás és élettartam

Elem	Tulajdonos, tároló	Kölcsönzött hozzáférés és érvényesség
`cfg`	A főszál által kezelt automatikus objektum	A `main` blokkjának végéig él
`published`	Statikus pointerobjektum	Nem birtokolja `cfg`-t; közös használatakor mutex védi
Dolgozó `p` pointere	Lokális pointerobjektum	`cfg`-re mutató kölcsönzés; a dolgozó befejezéséig használható
`mutex`	A főszál által lezárt szinkronizációs erőforrás	Csak a dolgozó csatlakoztatása után semmisül meg

A `cfg` élettartamának vége érvényteleníténé a kölcsönzött hozzáférést. A `join` biztosítja, hogy ez csak az utolsó dolgozói használat után történjen meg.

A publikálás tehát nem élettartam-kezelés. Egy megfelelő atomikus pointerátadás sem akadályozná meg önmagában az objektum túl korai felszabadítását.

15.2.5 A volatile és a diagnosztika korlátai

ISO C17: a `volatile` nem hoz létre szálak közötti szinkronizációt, és nem tesz egy hozzáférést atomikussá. A pointerobjektumot volatile-nak jelölő deklaráció:

```
/* ISO C17, deklarációrészlet.
   A volatile NEM teszi biztonsagossa a zarolas elhagyasat. */
static const struct config * volatile published = NULL;
```

nem pótolja a mutexet. A pointer párhuzamos olvasása és írása továbbra is adatversenyt okozhat. A megcímzett konfiguráció inicializálásának átadását sem oldja meg.

A **ThreadSanitizer, TSan** instrumentált végrehajtásokban keres adatversenyre utaló hozzáféréseket. A laborban külön diagnosztikai build készíthető:

```
clang -std=c17 -Wall -Wextra -Wpedantic -O1 -g \
  -fsanitize=thread -pthread publication.c -o publication-tsan
./publication-tsan
```

Ehhez az adott targeten támogatott, a fordítóhoz illeszkedő sanitizer futtatókönyvtár szükséges.

Egy lelet értelmezésekor keresd meg:

- a két ütköző hozzáférést és azok hívási vermét;
- az érintett memóriahelyet;

- a hozzáférő szálak létrehozási helyét;
- a hiányzó szinkronizációs kapcsolatot.

A lelet nem pusztán „rossz értéket” mutat: a rendezési szerződés hiányára irányítja a figyelmet. Ugyanakkor a **lelet hiánya nem bizonyít helyességet**. A futás nem jár be minden ágat; hiányos instrumentálás vagy nem támogatott szinkronizáció is korlátozhatja a megfigyelést. A kapacitásellenőrzés korábbi logikai hibája pedig megfelelő mutexhasználat mellett nem is adatverseny.

15.2.6 Gyakori hibák és javításuk

- **„A pointer befér egy gépi szóba.”** A hardveres hozzáférés feltételezett oszthatatlansága nem helyettesíti a C memóriamodelljét.
- **„Csak az írónál zárolok.”** Az olvasót ugyanabba a szinkronizációs protokollba kell bevonni.
- **„Már publikáltam, ezért felszabadíthatom.”** A publikálás után is meg kell várni minden kölcsönzés végét.
- **„Minden mezőt külön levédtem.”** Ettől a több mezőt vagy több lépést érintő invariáns még sérülhet.

15.2.7 Feladatok

1. **Magyarázó feladat.** Két szál ugyanazt a nem atomikus objektumot csak olvassa. Ezután az egyik szálban megjelenik egy írás is. Milyen további információ kell az adatverseny eldöntéséhez? Miért nem elég a műveletek időbélyege?

2. **Kódolási feladat.** A példában a főszál publikálás után is módosíthassa a konfigurációt, mindig a mutex alatt. Írd át a dolgozót úgy, hogy a kiírás továbbra is zárolás nélkül történjen, de konzisztens értékmásolatot használjon.

3. **Diagnosztikai feladat.** Valaki eltávolítja a publikálási példából a mutex megszerzését és feloldását, majd a pointerobjektumot `volatile`-nak jelöli. Egy TSan-futás nem jelez hibát. Elfogadható-e a módosítás? Megoldaná-e a problémát a program végén végrehajtott `join`?

15.2.8 Megoldások

15.2.8.1 Az ütközés és a rendezés vizsgálata

Két olvasás önmagában nem ütközik. Írás megjelenésekor az összes érintett olvasással és írással vizsgálni kell a *hb* kapcsolatot. Ha különböző szálak ütköző, legalább részben nem atomikus hozzáférései rendezetlenek, adatverseny van.

Ellenőrzési feltétel: minden ütköző párhoz megnevezhető a rendezési lánc. Az időbélyeg nem ilyen lánc, csak megfigyelt időadat.

15.2.8.2 Pillanatkép készítése a mutex alatt

Az eredeti fájlban a dolgozó függvény helyettesíthető ezzel a C17/POSIX-részlettel:

```
static void *reader(void *unused)
{
    (void)unused;
    struct config snapshot;
    int available;

    check(pthread_mutex_lock(&mutex), "lock");
    available = (published != NULL);
    if (available) {
        snapshot = *published;
    }
    check(pthread_mutex_unlock(&mutex), "unlock");

    if (available) {
        printf("port=%u workers=%u\n",
               snapshot.port, snapshot.workers);
    }
    return NULL;
}
```

A dolgozó birtokolja saját `snapshot` objektumát; a megosztott objektum kölcsönzött elérése a kritikus szakaszra korlátozódik. A főszál továbbra is életben tartja `cfg`-t a csatlakozásig.

Ellenőrzési feltétel: minden publikálás utáni módosítás ugyanazon mutex alatt történjen, és a teljes struktúramásolás egyetlen kritikus szakaszban maradjon. A másolat itt önálló, mert a struktúra nem tartalmaz további kölcsönzött pointereket.

15.2.8.3 A negatív TSan-futás értékelése

A módosítás nem fogadható el. A pointer írása és olvasása között nincs megfelelő rendezés; a `volatile` nem teremt ilyet. A sikeres `join` a dolgozó befejezését rendezi a főszál későbbi műveletei elé, nem a főszál korábbi publikálását a dolgozó korábbi olvasása elé.

Ellenőrzési feltétel: vissza kell állítani a bizonyítható szinkronizációs protokollt. Az ismételt TSan-futtatás kiegészítő diagnosztika, nem helyettesítő bizonyítás.

15.2.9 Összefoglaló

Az adatversenyhez ütköző, különböző szálakon végzett, legalább részben nem atomikus és *happens-before* szerint rendezetlen hozzáférések kellenek. Következménye UB. A logikai versenyhelyzet ennél tágabb: adatversenymentes műveletekből is összeállhat hibás protokoll.

Megosztott pointer használatakor külön bizonyítsd a pointer átadását, az objektum inicializálásának megfigyelhetőségét és az élettartamot. A mutex ezekhez rendezést adhat; a `volatile` és a hibajelzés nélkül lefutó teszt nem ad ilyen garanciát.

15.3 Mutex, condition variable és producer–consumer

Egy producer–consumer rendszerben a termelő munkát ad át, a fogyasztó pedig feldolgozza azt. A nehézség nem pusztán az, hogy egyszerre csak egy szál módosítsa a közös tárolót: azt is biztosítani kell, hogy a várakozó szálak megfelelő állapotváltozáskor felébredjenek, leálláskor pedig senki ne maradjon örökre blokkolva.

A Data race, happens-before és publikalás részben megismert szinkronizációs kapcsolatot most mutexszel hozzuk létre. A Dinamikus tömb és string builder tárolóinvariánsaihoz hasonlóan pontos állapotleírásból indulunk, de rögzített kapacitású sort használunk. Ez szándékos: a megtelt sor várakozásra kényszeríti a termelőt, így természetes **visszanyomást**, azaz *backpressure*-t ad.

15.3.1 Mutex, lock ownership és invariáns

A **mutex** kölcsönös kizárást biztosító szinkronizációs objektum. Sikeres zárolás után a hívó szál birtokolja a zárolást (*lock ownership*); a vizsgált, nem rekurzív használatban neki kell azt feloldania. Ez nem azonos az objektum tulajdonlásával: a mutex élettartamáért felelős szál nem feltétlenül az, amelyik pillanatnyilag zárolja.

A **kritikus szakasz** (*critical section*) a sikeres zárolás és feloldás közötti végrehajtás. A mutex csak akkor védi az adatot, ha minden konkurens hozzáférés ugyanazt a zárolási szerződést követi. Egy kizárólag olvasó diagnosztikai lekérdezés sem kerülheti meg ezt.

Művelet	POSIX pthreads	ISO C17, C11-ben bevezetett API
Mutex típusa	`pthread_mutex_t`	`mtx_t`
Inicializálás	`pthread_mutex_init`	`mtx_init`, például `mtx_plain`
Zárolás, feloldás	`pthread_mutex_lock`, `pthread_mutex_unlock`	`mtx_lock`, `mtx_unlock`
Feltételváltozó	`pthread_cond_t`	`cnd_t`
Várakozás	`pthread_cond_wait`	`cnd_wait`
Ébresztés	`pthread_cond_signal`, `pthread_cond_broadcast`	`cnd_signal`, `cnd_broadcast`

A két API nem keverhető. A pthreads függvények jellemzően közvetlenül adják vissza a hibaszámot, nem az `errno` változón keresztül. A C API eredményeit például a `thrd_success` és `thrd_timedout` értékekkel kell összehasonlítani. C17-ben a szálkönyvtár támogatása opcionális; a `__STDC_NO_THREADS__` jelzés mellett a tényleges header- és linkelhetőséget is ellenőrizni kell.

A sor kapacitása legyen $C > 0$, elemszáma n, következő olvasási indexe h, következő írási indexe t. Az invariáns:

$$0 \le n \le C, \qquad 0 \le h, t < C, \qquad t = (h + n) \bmod C.$$

A `closed` állapot csak hamisból igazra változhat. Lezárás után nincs új beszúrás, de a már elfogadott elemek kiolvashatók. A mutex ezeket az állításokat együtt védi, nem csupán egyes számlálók írását.

Több mutex esetén előre rögzített **zárolási sorrend** (*lock ordering*) szükséges: ha minden művelet előbb `A`-t, majd `B`-t zárolja, nem alakulhat ki olyan kör, amelyben az egyik szál `A` mellett `B`-re, a másik `B` mellett `A`-ra vár. A példában egyetlen mutex szerepel.

15.3.2 A feltételváltozó nem eseményszámláló

A **feltételváltozó** (*condition variable*) várakozási mechanizmus. Nem maga tárolja, hogy „érkezett munka”; ezt a mutexszel védett állapotból képzett logikai feltétel, a **predicate** mondja meg.

Művelet	A várakozás befejezésének predicate-je	Siker feltétele
Beszúrás	`closed || count < CAPACITY`	`!closed && count < CAPACITY`
Kivétel	`closed || count > 0`	`count > 0`

POSIX szerint a `pthread_cond_wait` a birtokolt mutexet a várakozás megkezdésével összehangoltan feloldja, visszatérés előtt pedig újra zárolja. Ez hidalja át azt a veszélyes rést, amely külön „unlock, majd alvás” műveleteknél elveszett ébresztéshez vezethetne.

A predicate-et mindig ciklusban ellenőrizzük:

```
/* C17/POSIX szemléltető részlet; belépéskor m zárolva.
   A teljes példában a visszatérési kódokat is ellenőrizzük. */
while (!predicate) {
    pthread_cond_wait(&cv, &m);
}
```

Lehetséges **álébredés** (*spurious wakeup*), de enélkül is szükséges a ciklus: mire a felébresztett szál visszaszerzi a mutexet, másik fogyasztó már elvihette az elemet.

A `signal` nem tárol később beváltható értesítést. Ha nincs várakozó, a jövőbeli hívó az állapotot ellenőrzi, nem egy korábbi jelzésre támaszkodik.

15.3.3 Teljes példa: lezárható bounded queue

A következő `queue.c` ISO C17 nyelvet és POSIX szál-API-t használ. A `pthread_condattr_setclock` elérhetősége platformfeltétel.

Kijelölt laborprofil: Linux, x86-64 System V ABI, LP64; a `lab-next` profil (profiltábla). A parancsok fordítási útmutatók; a példa itt nem fordított vagy mért eredmény.

A sor értékként másol `int` elemeket, tehát nem ad át dinamikus objektumokra mutató pointereket. A főszál termel, a munkaszál fogyaszt.

```
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <pthread.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

enum { CAPACITY = 8 };

enum q_result { Q_OK, Q_CLOSED, Q_TIMEOUT };
```

```c
struct queue {
    pthread_mutex_t mutex;
    pthread_cond_t not_empty;
    pthread_cond_t not_full;
    int data[CAPACITY];
    size_t head, tail, count;
    bool closed;
};

/* Váratlan szinkronizációs hibánál fail-fast hibapolitika. */
static void sync_check(int error)
{
    if (error != 0) {
        fprintf(stderr, "pthread-hiba: %d\n", error);
        abort();
    }
}

static int q_init(struct queue *q)
{
    pthread_condattr_t attr;
    int error = pthread_condattr_init(&attr);
    if (error != 0)
        return error;

    error = pthread_condattr_setclock(&attr, CLOCK_MONOTONIC);
    if (error != 0)
        goto fail_attr;

    error = pthread_mutex_init(&q->mutex, NULL);
    if (error != 0)
        goto fail_attr;

    error = pthread_cond_init(&q->not_empty, &attr);
    if (error != 0)
        goto fail_mutex;

    error = pthread_cond_init(&q->not_full, &attr);
    if (error != 0)
        goto fail_empty;

    q->head = 0;
    q->tail = 0;
    q->count = 0;
    q->closed = false;
    sync_check(pthread_condattr_destroy(&attr));
    return 0;

fail_empty:
    sync_check(pthread_cond_destroy(&q->not_empty));
fail_mutex:
    sync_check(pthread_mutex_destroy(&q->mutex));
```

```
fail_attr:
    sync_check(pthread_condattr_destroy(&attr));
    return error;
}

/* deadline: NULL vagy abszolút CLOCK_MONOTONIC határidő.
   Nem NULL esetén érvényes, változatlan timespec szükséges. */
static enum q_result q_push(
    struct queue *q, int value, const struct timespec *deadline)
{
    sync_check(pthread_mutex_lock(&q->mutex));

    while (!q->closed && q->count == CAPACITY) {
        int error;
        if (deadline != NULL) {
            error = pthread_cond_timedwait(
                &q->not_full, &q->mutex, deadline);
        } else {
            error = pthread_cond_wait(&q->not_full, &q->mutex);
        }

        if (error == ETIMEDOUT)
            break;
        sync_check(error);
    }

    enum q_result result;
    if (q->closed) {
        result = Q_CLOSED;
    } else if (q->count == CAPACITY) {
        result = Q_TIMEOUT;
    } else {
        q->data[q->tail] = value;
        q->tail = (q->tail + 1) % CAPACITY;
        ++q->count;
        sync_check(pthread_cond_signal(&q->not_empty));
        result = Q_OK;
    }

    sync_check(pthread_mutex_unlock(&q->mutex));
    return result;
}

/* out érvényes, kizárólagosan írható int objektumra mutat.
   Q_CLOSED esetén *out változatlan. */
static enum q_result q_pop(struct queue *q, int *out)
{
    sync_check(pthread_mutex_lock(&q->mutex));

    while (!q->closed && q->count == 0)
        sync_check(pthread_cond_wait(&q->not_empty, &q->mutex));
```

```
    enum q_result result;
    if (q->count == 0) {
        result = Q_CLOSED;
    } else {
        *out = q->data[q->head];
        q->head = (q->head + 1) % CAPACITY;
        --q->count;
        sync_check(pthread_cond_signal(&q->not_full));
        result = Q_OK;
    }

    sync_check(pthread_mutex_unlock(&q->mutex));
    return result;
}

static void q_close(struct queue *q)
{
    sync_check(pthread_mutex_lock(&q->mutex));
    q->closed = true;
    sync_check(pthread_cond_broadcast(&q->not_empty));
    sync_check(pthread_cond_broadcast(&q->not_full));
    sync_check(pthread_mutex_unlock(&q->mutex));
}

/* Csak minden használó befejeződése után hívható. */
static void q_destroy(struct queue *q)
{
    sync_check(pthread_cond_destroy(&q->not_full));
    sync_check(pthread_cond_destroy(&q->not_empty));
    sync_check(pthread_mutex_destroy(&q->mutex));
}

struct worker {
    struct queue *queue;
    long sum;
};

static void *consume(void *argument)
{
    struct worker *w = argument;
    int value;
    long sum = 0;

    while (q_pop(w->queue, &value) == Q_OK)
        sum += value;  /* Feldolgozás a sor mutexén kívül. */

    w->sum = sum;
    return NULL;
}

int main(void)
{
```

```
    struct queue q;
    int error = q_init(&q);
    if (error != 0) {
        fprintf(stderr, "q_init: %d\n", error);
        return EXIT_FAILURE;
    }

    struct worker worker = { .queue = &q, .sum = 0 };
    pthread_t thread;
    error = pthread_create(&thread, NULL, consume, &worker);
    if (error != 0) {
        fprintf(stderr, "pthread_create: %d\n", error);
        q_destroy(&q);
        return EXIT_FAILURE;
    }

    bool failed = false;
    for (int value = 1; value <= 1000; ++value) {
        if (q_push(&q, value, NULL) != Q_OK) {
            failed = true;
            break;
        }
    }

    q_close(&q);
    sync_check(pthread_join(thread, NULL));
    q_destroy(&q);

    if (printf("osszeg=%ld\n", worker.sum) < 0)
        return EXIT_FAILURE;

    return !failed && worker.sum == 500500L
        ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -pthread queue.c -o queue-gcc
clang -std=c17 -Wall -Wextra -Wpedantic -O2 -pthread queue.c -o queue-clang
```

A szerződésből következő **várt**, nem megfigyelt összeg:

$$\sum_{k=1}^{1000} k = \frac{1000 \cdot 1001}{2} = 500500.$$

15.3.3.1 Tulajdonlás és hibaszerződés

Objektum vagy hozzáférés	Tulajdonos és élettartam	Érvénytelenítés, hibakimenet
`q`, benne a tároló és a szinkronizációs objektumok	Főszál; inicializálástól az összes használó befejezéséig	`q_destroy` után nincs további sorhívás
`worker`, a callback argumentuma	Főszál; legalább a sikeres `join` végéig	Korábbi hatókörvég dangling hozzáférést okozna

Objektum vagy hozzáférés	Tulajdonos és élettartam	Érvénytelenítés, hibakimenet
`q_pop` kimeneti pointere	A hívó kölcsönadott objektuma; a hívás alatt él	`Q_CLOSED` nem módosítja
`deadline`	A hívó változatlan, kölcsönadott objektuma; a hívás végéig él	Érvénytelen időérték szerződéssértés

Az inicializált sort nem másoljuk értékadással vagy `memcpy`-val. A mutex és a feltételváltozók nem másolható sorállapotként kezelendők.

A részleges inicializálás hibája visszatérési kódot ad, és csak a már létrehozott erőforrásokat bontja le. Üzem közben a lezárás és az időtúllépés normális eredmény. Váratlan szinkronizációs hibánál a példa megszakítja a folyamatot: nem állítja, hogy ismeretlen zárolási állapotból biztonságosan folytathat.

A szerződés kizárja a szálak kényszerített cancellationjét. A POSIX condition wait cancellation point; annak támogatásához külön cleanup protokoll kellene.

15.3.4 Helyesség és leállítás

Invariánsmegőrzési bizonyításvázlat. Beszúrás csak $n < C$ esetén történik. Utána:

$$n' = n + 1, \qquad t' = (t + 1) \bmod C,$$

ezért

$$t' = (h + n + 1) \bmod C = (h + n') \bmod C.$$

Kivételnél $n > 0$, majd $h' = (h + 1) \bmod C$ és $n' = n - 1$, tehát $(h' + n') \bmod C = t$. A lezárás nem változtatja az indexeket. A mutex miatt két ilyen átmenet nem módosíthatja egyszerre az állapotot.

A `q_close` **ürítve leállást** jelent:

1. A mutex alatt beállítja a lezárást.
2. Mindkét feltételváltozón `broadcast` történik.
3. Feloldja a mutexet.
4. A vezérlőszál megvárja a használó szálakat.
5. Csak ezután történik megsemmisítés.

Mindkét várakozási csoportot ébreszteni kell: a termelők telt soron, a fogyasztók üres soron várhatnak. A lezárást követő beszúrás visszautasított; a fogyasztók a meglévő elemeket még feldolgozzák.

Leállási bizonyításvázlat. A `closed` beállítása után egyik várakozási ciklus sem kezd újabb várakozást. A `broadcast` a már várakozókat is eléri, a később belépők pedig az állapotból ismerik fel a lezárást. A `join` alatt nincs birtokolt sormutex, így a kilépéshez szükséges zárolást nem akadályozza a várakozó főszál.

Ez feltételes állítás: a szálaknak futási lehetőséget kell kapniuk, a feldolgozásnak végesnek kell lennie, és nem akadhatnak el más erőforráson. A `q_close` nem szakít meg tetszőleges hálózati I/O-t.

Általános, többtermelős shutdownnál a lezárás előzze meg a potenciálisan blokkolt termelők `join`-ját. Ellenkező esetben éppen azt a szálat várhatnánk, amely csak a lezárás után tudna visszatérni.

15.3.5 Timed wait: az óra és a predicate külön kérdés

A POSIX `pthread_cond_timedwait` abszolút határidőt fogad. Az órabázis a feltételváltozó attribútuma; alapértelmezésben `CLOCK_REALTIME`, a példában explicit `CLOCK_MONOTONIC`.

Óra	Jellemző használat és korlát
`CLOCK_REALTIME`	Faliórához kötött határidő; az óra átállítása befolyásolhatja a várakozást
`CLOCK_MONOTONIC`	Eltelt idő alapú korlát; nem követi a falióra ugrásait
C17 `cnd_timedwait`	`TIME_UTC` alapú abszolút idő; nincs POSIX-szerű óraattributuma

A határidőt ugyanabból az órából kell képezni, mint amelyet a feltételváltozó használ. Legyen az aktuális idő (s, n), a hozzáadandó idő pedig Δ nanoszekundum. Megfelelően széles, túlcsordulás-ellenőrzött számítással:

$$u = n + \Delta, \qquad d_s = s + \left\lfloor \frac{u}{10^9} \right\rfloor, \qquad d_n = u \bmod 10^9.$$

A `tv_nsec` mezőre $0 \le d_n < 10^9$ szükséges. A végső másodpercértéknek bele kell férnie a célmezőbe. Álébredés után nem számolunk új határidőt: az újraindított relatív várakozás korlátlanul meghosszabbíthatná a műveletet.

Az `ETIMEDOUT` nem bizonyítja, hogy a predicate a visszatéréskor hamis. A mutex visszaszerzéséig más szál helyet szabadíthatott fel vagy lezárhatta a sort. Ezért a példában az állapot dönt:

- lezárt sor: `Q_CLOSED`;
- nyitott, továbbra is telt sor: `Q_TIMEOUT`;
- nyitott sor szabad hellyel: sikeres beszúrás.

Ez nem szigorú teljes végrehajtási határidő. A mutex első megszerzése sem időkorlátos, és a határidő lejárta után rendelkezésre álló helyet a művelet még felhasználhatja.

15.3.6 Költség és gyakori hibák

A beszúrás és kivétel állapotmódosítása worst-case $O(1)$, külön allokáció nélkül, folytonos tömbtárolóval. A várakozási idő viszont nem korlátos; a mutex nem biztosít alkalmazási szintű méltányosságot. A másolt elemek érvényességét a sor későbbi módosítása nem érinti.

Gyakori hiba az `if` használata `while` helyett, a predicate zárolás nélküli olvasása, illetve a feldolgozás végrehajtása a sormutex alatt. Utóbbi nem feltétlen adatverseny, de megszünteti a párhuzamosságot, és további zárolásokkal holtpontot okozhat.

A `broadcast` önmagában nem állít le semmit: állapotváltozás nélkül a helyesen megírt várakozók visszaalszanak. A `close` pedig nem teszi azonnal biztonságossá a `destroy` hívást.

15.3.7 Feladatok

1. **Magyarázó:** Miért különbözik a kivétel várakozási predicate-je a siker feltételétől? Mi történjen lezárt, de nem üres soron?
2. **Kódolási:** Készíts egyszálú timeout-tesztet. Töltsd tele a sort, képezz `CLOCK_MONOTONIC` alapján normalizált, 50 ms múlva lejáró határidőt, majd próbálj új elemet beszúrni. Ezután zárd le és ürítsd ki.
3. **Diagnosztikai:** Egy leállító függvény zárolja a sormutexet, beállítja a `closed` mezőt, `broadcast`-ot hív, majd még a feloldás előtt `pthread_join`-nal várja a fogyasztót. Miért akadhat el akkor is, ha minden adat-hozzáférés zárolt?

15.3.8 Megoldások

15.3.8.1 Predicate és ürítés

A `closed || count > 0` azt jelenti, hogy nincs értelme tovább várni. Sikerhez azonban elem is szükséges. Lezárt, nem üres soron a kivétel sikeres; lezárt, üres soron `Q_CLOSED` az eredmény.

Ellenőrzési feltétel: minden korábban sikeresen beszúrt elem pontosan egyszer kivehető, majd a következő kivétel lezárást jelez.

15.3.8.2 Timeout-teszt

A teszt `q_init` után `CAPACITY` sikeres beszúrást végez `NULL` határidővel. A `clock_gettime` hibáját külön, annak `errno`-s szerződése szerint kezeli. A hozzáadandó idő 50 000 000 nanoszekundum; ha az összeg eléri az egymilliárdot, egy másodperc átvitel szükséges.

Fogyasztó hiányában az új beszúrás `Q_TIMEOUT` eredményt ad. Lezárás után a feltöltött értékeket FIFO-sorrendben kell visszakapni, végül `Q_CLOSED` következik.

Ellenőrzési feltétel: a sikertelen beszúrás értéke nincs a sorban. Pontos 50 ms-os visszatérést nem szabad elvárni az ütemezőtől.

15.3.8.3 Join a kritikus szakaszban

A fogyasztónak az ébredés után vissza kell szereznie a mutexet. A főszál ezt birtokolja, miközben a fogyasztó befejeződésére vár:

$$\text{főszál} \rightarrow \text{fogyasztó befejezése} \rightarrow \text{mutex} \rightarrow \text{főszál}.$$

A javítás a `join` áthelyezése a feloldás után.

Ellenőrzési feltétel: egyetlen szálra sem várunk úgy, hogy közben a befejeződéséhez szükséges zárolást birtokoljuk.

15.3.9 Összefoglaló

A mutex invariánst véd; a feltételváltozó az invariánshoz kapcsolódó predicate megváltozására enged várni. A predicate-et ciklusban, ugyanazon mutex alatt ellenőrizzük. Leálláskor állapotot módosítunk, minden érintett várakozót ébresztünk, feloldunk, majd csatlakozunk a szálakhoz. A timeout óraalapja és eredményszemantikája az API-szerződés része, nem pusztán időzítési részlet.

15.4 C atomics és memory order

Az atomikus művelet nem „kis mutex”, és nem általános védelem minden megosztott állapot körül. Egy meghatározott objektumon biztosít oszthatatlan hozzáférést; a más objektumokra vonatkozó rendezést a választott memóriarendezés (*memory order*) határozza meg. Az objektum élettartamát egyik sem hosszabbítja meg.

A Data race, happens-before és publikalás alapján két külön kérdést kell megválaszolnunk: versenymentes-e maga a hozzáférés, és létrejön-e a szükséges *happens-before* kapcsolat? A Mutex, condition variable és producer–consumer megoldásai továbbra is az alapértelmezett választások összetett invariánsokhoz. Itt egyszerű számlálóra és egyszeri adatátadásra szorítkozunk.

15.4.1 Atomikus objektumok és műveletek

ISO C17. Az `_Atomic` atomikus típust jelöl; például az `_Atomic(unsigned)` atomikus `unsigned` típus. A `<stdatomic.h>` többek között az ennek megfelelő `atomic_uint` típusnevet és az atomikus műveleteket biztosítja. C17-ben az atomikus támogatás opcionális: a `__STDC_NO_ATOMICS__` makró jelzi, ha az implementáció nem támogatja.

Az atomikusság (*atomicity*) azt jelenti, hogy a művelet az érintett objektumon oszthatatlan. Egy atomikus olvasás nem két külön írásból összerakott „félértéket” vesz át. Ez azonban nem tesz több egymást követő műveletet közös tranzakcióvá.

Művelet	Hatás	Eredmény
`atomic_load`	Érték olvasása	Az olvasott érték
`atomic_store`	Érték beírása	Nincs visszatérési érték
`atomic_exchange`	Érték lecserélése	A korábbi érték
`atomic_fetch_add`	Összeadás egyetlen read–modify–write művelettel	A korábbi érték
`atomic_compare_exchange_*`	Feltételes csere	A siker logikai értéke

Az **RMW**, vagyis *read–modify–write*, egyetlen oszthatatlan olvasás–módosítás–írás. Az `_explicit` végződésű változatok memóriarendezést is kapnak; az enélküliek alapértelmezése `memory_order_seq_cst`.

Az inicializálás külön lépés. Az `atomic_init` használatakor másik szál még nem férhet hozzá az objektumhoz; ez nem konkurens újrainicializálási eszköz. Statikus objektumhoz C17-ben használható az `ATOMIC_VAR_INIT`.

Az atomikus objektum megfelelő tárolót és igazítást igényel. Egy közönséges `unsigned` címének atomikus pointerré alakítása nem hoz létre megfelelő atomikus objektumot.

15.4.2 Sequential consistency és relaxed számláló

15.4.2.1 Az első alapmodell: sequential consistency

ISO C17. A *sequential consistency*, röviden SC, a legkézenfekvőbb kiindulópont. A `memory_order_seq_cst` műveletekhez a szabvány által előírt rendezésekkel összhangban álló egyetlen teljes sorrend tartozik. Egyszerű, kizárólag SC atomikus hozzáféréseket használó példákban úgy gondolkodhatunk, mintha a szálak műveletei egy közös sorrendbe lennének összefésülve.

Ez nem jelenti minden közönséges memóriahozzáférés automatikus szinkronizálását, és a különálló atomikus lépéseket sem egyesíti:

```
/* ISO C17, hibás számlálási logika; függvénytörzs-részlet.
   counter: élő, inicializált atomic_uint objektum. */
unsigned old = atomic_load(&counter);
atomic_store(&counter, old + 1U);
```

Két szál egyaránt olvashat nullát, majd mindkettő egyet írhat. Az atomikus objektumon nincs adatverseny, de egy növelés elveszik: ez **logikai race**.

15.4.2.2 Mikor elegendő a relaxed?

Ha csak egy számot összesítünk, és a szám értéke nem ad engedélyt más adatok olvasására, gyakran elegendő a `memory_order_relaxed`.

Az alábbi ISO C17 kódrészlet statikus tárolási idejű számlálót használ. Tulajdonosa a program; a függvény nem vesz át erőforrást, és nem ad vissza kölcsönzött hozzáférést.

```
#include <stdatomic.h>

static atomic_ulong completed = ATOMIC_VAR_INIT(0UL);

void record_completion(void)
{
    (void)atomic_fetch_add_explicit(
        &completed, 1UL, memory_order_relaxed);
}

unsigned long completion_snapshot(void)
{
    return atomic_load_explicit(
        &completed, memory_order_relaxed);
}
```

Minden RMW egyetlen lépésben növeli a számlálót. Minden atomikus objektum módosításainak saját teljes sorrendje van, a *modification order*; ezt a relaxed rendezés sem szünteti meg.

Ha összesen N növelés történik, a kezdőérték nulla, és $N \leq$ ULONG_MAX, akkor az összes növelő szál sikeres bevárása után olvasott érték N. A túlcsordulás unsigned típuson körbefordulást okozna, ami általában sérti a metrikaszerződést.

A relaxed művelet viszont **nem publikálja az elkészült munkához tartozó adatokat**. Attól, hogy a megfigyelt számláló elér egy küszöböt, egy másik tömb vagy eredménystruktúra még nem válik biztonságosan olvashatóvá.

15.4.3 Egyszeri publikálás release/acquire kapcsolattal

Egy szál két közönséges mezőt tölt ki, majd jelzi, hogy az üzenet elkészült. Az olvasó kizárólag a jelzés megfelelő átvétele után olvassa a mezőket.

Példaprofil: ISO C17 és POSIX pthreads; Linux x86-64 System V célrendszer. Választott laborverziók: a `lab-next` profil (profiltábla). A szinkronizálás atomikus része ISO C17, a szálkezelés POSIX. Az alábbiak reprodukálási parancsok; itt nem tartozik hozzájuk elvégzett fordítás vagy futtatási eredmény.

```
/* publication.c */
#include <pthread.h>
#include <stdatomic.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

struct message {
    int value;
    unsigned jobs;
    atomic_bool ready;
};

static void *produce(void *arg)
{
    struct message *m = arg;

    m->value = 42;
    m->jobs = 7U;
    atomic_store_explicit(&m->ready, true, memory_order_release);
    return NULL;
}

int main(void)
{
    struct message m;
    atomic_init(&m.ready, false);

    pthread_t thread;
    int rc = pthread_create(&thread, NULL, produce, &m);
    if (rc != 0) {
        fprintf(stderr, "pthread_create: %d\n", rc);
        return EXIT_FAILURE;
    }

    while (!atomic_load_explicit(&m.ready, memory_order_acquire)) {
        /* Kizárólag szemléltető aktív várakozás. */
    }

    /* Szándékosan a join előtt: itt az atomikus publikálás véd. */
    int value = m.value;
    unsigned jobs = m.jobs;
```

```
    rc = pthread_join(thread, NULL);
    if (rc != 0) {
        fprintf(stderr, "pthread_join: %d\n", rc);
        abort();
    }

    printf("%d %u\n", value, jobs);
    return EXIT_SUCCESS;
}
```

```
gcc -std=c17 -O2 -Wall -Wextra -Wpedantic \
    -pthread publication.c -o publication-gcc

clang -std=c17 -O2 -Wall -Wextra -Wpedantic \
    -pthread publication.c -o publication-clang
```

Sikeres szálműveletek és befejeződő végrehajtás esetén a várt kimenet `42 7`. A pörgő várakozás nem termelési ajánlás: CPU-időt fogyaszt, és önmagában nem ad időkorlátos előrehaladási garanciát. Hosszabb várakozáshoz használjunk condition variable-t.

15.4.3.1 Pontosan mely adatokat teszi olvashatóvá?

Bizonyításvázlat, ISO C17. Jelölje:

- W a `value` vagy `jobs` mező írását;
- S a release store műveletet;
- L azt az acquire load műveletet, amely `true` értéket olvas;
- R az adott mező olvasását.

Ebben a programban egyetlen művelet ír `true` értéket. Ezért L az S által írt értéket olvassa, és létrejön a *synchronizes-with* kapcsolat:

$$W \text{ sequenced-before } S \text{ synchronizes-with } L \text{ sequenced-before } R.$$

Ebből:

$$W \text{ happens-before } R.$$

A mezőket publikálás után senki nem módosítja, ezért olvasásuk versenymentes, és az írt értékeket szolgáltatja.

A kapcsolat tehát a release előtt rendezett írásokat teszi elérhetővé az acquire utáni olvasások számára. Nem védi a producer későbbi módosításait, és nem jelent általános kizárólagos hozzáférést. A `ready` egyszerű visszaállítása sem teszi ezt automatikusan újrahasználható üzenetsorrá.

15.4.3.2 Élettartam és hibakimenet

Elem	Tulajdonos és tároló	Kölcsönzés, érvénytelenítés
`m` és mezői	A `main` automatikus objektuma	A `produce` pointere kölcsönzött; `m` élettartamának vége érvénytelenítené
`arg`	A worker függvényparamétere	Nem tulajdonolja és nem szabadítja fel `m`-et
Szál-erőforrás	A `main` felelőssége bevárni	A sikeres `pthread_join` lezárja a kötelezettséget

Sikertelen létrehozáskor nincs elindított worker. Sikertelen beváráskor a példa megszakítja a folyamatot: nem feltételezi igazolatlanul, hogy a worker már befejeződött. A pthread függvények itt közvetlen hibakódot adnak, nem az `errno` ellenőrzése szükséges.

15.4.4 Compare-exchange: feltételes módosítás

A *compare-and-exchange*, röviden CAS, összehasonlítja az atomikus objektum értékét az `expected` által megadott várt értékkel.

- Siker esetén beírja a kívánt értéket, és `true` értékkel tér vissza.
- Sikertelenség esetén nem módosítja az atomikus objektumot, viszont az olvasott értékkel felülírja az `expected` változót.

A `weak` változat egyező értékek mellett is jelezhet sikertelenséget: ez a *spurious failure*. Újrapróbálkozó ciklusban használható. A `strong` változatnak nincs ilyen megengedett álhibája, de az sem garantálja az egész algoritmus befejeződését.

Az alábbi ISO C17 függvényrészlet legfeljebb `limit` értékig enged növelni:

```
#include <stdatomic.h>
#include <stdbool.h>

bool try_increment_below(atomic_uint *counter, unsigned limit)
{
    unsigned expected =
        atomic_load_explicit(counter, memory_order_relaxed);

    while (expected < limit) {
        if (atomic_compare_exchange_weak_explicit(
                counter, &expected, expected + 1U,
                memory_order_relaxed,
                memory_order_relaxed)) {
            return true;
        }
        /* expected már a sikertelen kísérletben olvasott érték. */
    }
    return false;
}
```

A pointer nem null, élő, inicializált atomikus objektumra mutat. Tulajdonosa a hívó; annak tárolóját csak minden párhuzamos hívás befejezése után szüntetheti meg. Az `expected` minden hívás saját lokális objektuma. A `false` kapacitáshiányt jelent, nem rendszerhibát.

Bizonyításvázlat. Tegyük fel, hogy kezdetben `counter <= limit`, minden módosító ezt a függvényt használja, és ugyanaz a korlát. A sikeres CAS pillanatában az aktuális érték valóban `expected`, továbbá:

$$\texttt{expected} < \texttt{limit} \quad \Longrightarrow \quad \texttt{expected} + 1 \leq \texttt{limit}.$$

Az összeadás sem csordul túl, mert `limit` unsigned érték. Sikertelen CAS nem változtatja meg a számlálót. A korlát így megmarad. Ez kizárólag számszerű kvóta: nem publikál hozzá tartozó erőforrást vagy payloadot.

15.4.4.1 A failure order szabályai

A CAS sikeres ága RMW; a sikertelen ág csak olvasás. **ISO C17-ben** a sikertelenségi rendezés nem lehet `memory_order_release` vagy `memory_order_acq_rel`, és nem lehet erősebb a sikeres ág rendezésénél.

A `consume` lehetőséget mellőzve:

Success order	Megengedett failure order
`relaxed`	`relaxed`
`acquire`	`relaxed`, `acquire`
`release`	`relaxed`
`acq_rel`	`relaxed`, `acquire`
`seq_cst`	`relaxed`, `acquire`, `seq_cst`

A táblázatban minden név `memory_order_` előtaggal értendő. A `release` siker és `acquire` sikertelenség például C17-ben nem megengedett páros.

15.4.5 Az atomic nem egyenlő lock-free-vel

Az `atomic_is_lock_free(&object)` azt kérdezi le, hogy az adott atomikus objektum műveletei zárolásmentesek-e az implementációban. A válasz típus- és célrendszerfüggő; egyes műveleteket az implementáció belső zárolással valósíthat meg. Bizonyos fordító/célrendszer párosításokhoz külön atomikus futásidejű könyvtár linkelése is szükséges lehet.

A lock-free művelet sem tesz automatikusan lock-free algoritmussá egy teljes rendszert. Nem garantál várakozásmentes (*wait-free*) végrehajtást, szálankénti méltányosságot vagy alacsony késleltetést. A CAS-ciklus ismétlésszámának például nincs itt bizonyított felső korlátja.

A `memory_order_consume` C17-ben adatfüggőségen alapuló rendezési mechanizmus része, nem egyszerűen „olcsó acquire”. Implementációs gyakorlata és későbbi szabványverziókhoz kapcsolódó kezelése külön ellenőrzést igényel. Tanulási és általános publikálási alapként használjunk `acquire` rendezést; ne vigyük át automatikusan a C++ verziószabályait C-re.

15.4.6 Gyakori hibák

A legsúlyosabb tévedés a rendezés és az élettartam összekeverése. Egy pointer release/acquire publikálása láthatóvá teheti az objektum inicializálását, de nem akadályozza meg annak idő előtti felszabadítását.

Hasonlóan hibás a „jelző atomikus, tehát minden biztonságos” érvelés. A release/acquire kapcsolatnak ténylegesen létre kell jönnie; két relaxed művelet ezt nem pótolja. A `volatile` sem helyettesít atomikus hozzáférést vagy szinkronizálást.

Végül az erősebb rendezés nem javít algoritmikus hibát: a különálló load és store SC mellett is elveszíthet növeléseket.

15.4.7 Feladatok

1. **Magyarázat.** Két szál SC load, majd SC store segítségével növel egy nulláról induló atomikus számlálót. Adj olyan műveletsorrendet, amelyben a végeredmény egy. Mi a javítás?
2. **Kódolás.** Bővítsd a CAS-példát `try_add_bounded(counter, amount, limit)` függvénnyé. Csak akkor adjon hozzá `amount` értéket, ha az eredmény nem haladja meg a korlátot. A nulla növekmény legyen megengedett.
3. **Diagnózis.** A publikálási példában:

   - csak az acquire loadot relaxedre cserélik;
   - mindkét atomikus művelet SC lesz;
   - a tulajdonos bevárás nélkül kilép a kölcsönadott objektum blokkjából.

   Melyik változtatás helyes, és milyen bizonyítás hiányzik a többiből?

15.4.8 Megoldások

15.4.8.1 Elvesző növelés

Megengedett sorrend: A olvas nullát; B olvas nullát; A egyet ír; B egyet ír. A megoldás egyetlen `atomic_fetch_add_explicit` művelet. Puszta számláláshoz relaxed rendezés elég.

Ellenőrzési feltétel: két befejezett, bevárt növelő után, más módosító nélkül, az eredmény kettő.

15.4.8.2 Korlátos összeadás

ISO C17 kódrészlet; ugyanazok az ownership- és élettartamfeltételek érvényesek, mint az előző CAS-függvénynél.

```
#include <stdatomic.h>
#include <stdbool.h>

bool try_add_bounded(atomic_uint *counter,
                     unsigned amount,
                     unsigned limit)
{
```

```
    unsigned old =
        atomic_load_explicit(counter, memory_order_relaxed);

    for (;;) {
        if (old > limit || amount > limit - old) {
            return false;
        }

        if (atomic_compare_exchange_weak_explicit(
                counter, &old, old + amount,
                memory_order_relaxed,
                memory_order_relaxed)) {
            return true;
        }
    }
}
```

Az első feltétel rövidzáras kiértékelése után a kivonás matematikailag is értelmes. A második feltétel biztosítja:

$$\texttt{amount} \le \texttt{limit} - \texttt{old} \quad \Longrightarrow \quad \texttt{old} + \texttt{amount} \le \texttt{limit}.$$

Ellenőrzési feltételek: nulla növekmény, pontosan a korlátig növelés, túllépés és `UINT_MAX` közeli értékek. Párhuzamos ellenőrzésnél minden módosító azonos korláttal használja ezt a műveletet.

15.4.8.3 Publikálási és élettartamhiba

A relaxed load nem hozza létre a szükséges *synchronizes-with* kapcsolatot. Mivel a payload olvasása a `join` előtt történik, annak írásával adatverseny áll elő: ez UB, nem pusztán „esetleg régi érték”.

A két SC művelet megfelelő: a store release-, a load acquire-hatása is biztosított.

A korai blokk-kilépés megszüntetheti az objektum élettartamát, miközben a worker még használja. Ezt semmilyen memory order nem javítja. Külön befejezési és élettartam-protokoll szükséges.

15.4.9 Összefoglaló

Induljunk SC műveletekkel, és csak megindokolt esetben gyengítsük a rendezést. Önálló számlálóhoz gyakran relaxed RMW elegendő; payload publikálásához release/acquire kapcsolat kell. A CAS sikertelenségi rendezése külön szabályokat követ. Mindig külön bizonyítsuk az atomikusságot, a szükséges *happens-before* kapcsolatot és az objektum élettartamának érvényességét.

15.5 Konkurens élettartam és nehéz hibák

Egy pointer atomikus beolvasása még nem biztosítja, hogy a megcímzett objektum él, mire dereferáljuk. A konkurens program helyességéhez ezért két külön kérdésre kell válaszolni: **hogyan egyeztetjük a hozzáféréseket, és mikor szüntethetjük meg biztonságosan az objektum élettartamát?**

A C atomics és memory order részben megismert publikálási kapcsolat láthatóvá teheti az inicializált adatokat, de önmagában nem akadályozza meg későbbi felszabadításukat. Most ezt az élettartamproblémát kapcsoljuk össze az előrehaladási garanciákkal és a diagnosztika korlátaival. A cél nem egy általános lock-free könyvtár megvalósítása, hanem annak felismerése, hogy egy megoldás pontosan mit garantál — és mit nem.

15.5.1 Biztonságos hozzáférés és biztonságos felszabadítás

A memória-visszanyerés, angolul *memory reclamation*, annak eldöntése, hogy egy megosztott adatszerkezetből eltávolított objektum tárolója mikor szabadítható fel vagy használható újra.

Az eltávolítás és a felszabadítás nem ugyanaz:

1. Az objektumot elérhetetlenné tesszük az **új** olvasók számára.
2. Megvárjuk vagy bizonyítjuk, hogy a **korábbi** olvasók sem használhatják többé.
3. Megszüntetjük az élettartamát, és felszabadítjuk a tárolóját.

ISO C17: ha egy szál dereferál egy már felszabadított objektumra mutató pointert, a viselkedés nem definiált. Az, hogy a címen még látszólag ugyanazok a bitek találhatók, nem élettartam-garancia.

Tegyük fel, hogy egy olvasó atomikusan beolvassa a megosztott pointert. Ezután egy másik szál atomikusan nullára cseréli azt, és felszabadítja az objektumot. Az olvasó következő dereferálása már hibás lehet. Mindez `memory_order_seq_cst` mellett is megtörténhet: az atomikus pointerműveletek rendezése nem hosszabbítja meg a megcímzett objektum élettartamát.

15.5.1.1 ABA: ugyanaz az érték, megváltozott történet

Az ABA-probléma akkor jelentkezik, amikor egy állapot értéke $A \to B \to A$ módon változik, és egy későbbi összehasonlítás az eredeti A értéket látva nem észleli a közbülső változásokat.

Nézzünk egy veremfejet. A fogalmi példában a `head` és a `next` mezők atomikusak, és egyetlen csomópontot sem szabadítunk fel. Így az ABA-hibát külön vizsgálhatjuk az adatversenytől és a dangling pointerektől.

Lépés	T1	T2	Verem
Kezdet			`A → B → C`
Olvasás	Elmenti: `head=A`, `A.next=B`		`A → B → C`
Módosítás	Szünetel	Kiveszi `A`-t, majd `B`-t	`C`
Visszahelyezés		Beállítja `A.next=C`, visszateszi `A`-t	`A → C`
CAS	`head` értékét `A`-ról a korábban olvasott `B`-re cseréli		`B → C`

A compare-and-exchange sikeres, pedig T1 elavult kapcsolat alapján dolgozik. `B` visszakerül a verembe, noha T2 már kivette, és akár saját tulajdonának tekinti.

A felszabadítás utáni cím-újrafelhasználás hasonló azonossági problémát okozhat: ugyanaz a numerikus cím később más objektumhoz tartozhat. C-ben azonban a régi pointert nem kezelhetjük automatikusan ártalmatlan címazonosítóként a megcímzett objektum élettartamának vége után.

Egy generációszámmal kiegészített fej csökkentheti az ABA veszélyét, de feltételei vannak:

- a pointert és a generációt együtt kell atomikusan összehasonlítani;
- a számláló körbefordulását kezelni vagy kizárni kell;
- az összetett atomikus művelet nem feltétlenül lock-free;
- a verziózás önmagában nem teszi biztonságossá a dereferálást.

15.5.2 Reclamation-megoldások fogalmi térképe

Megközelítés	Mi tartja életben az objektumot?	Lényeges korlát
Mutex alatti használat	Minden hozzáférés és eltávolítás ugyanahhoz a zárhoz kötött	Blokkolás; a borrowed pointer nem szökhet ki
Referenciaszámlálás	Minden használó érvényes tulajdonosi referenciát tart	Az első referencia megszerzését is védeni kell
Epoch-alapú reclamation	Az eltávolítás előtti olvasói szakaszok befejezése	Egy elakadt résztvevő sok objektum felszabadítását késleltetheti
Hazard pointer	Az olvasó közzéteszi, mely objektumot védi	Helyes publikálási, újraellenőrzési és vizsgálati protokoll szükséges

15.5.2.1 Referenciaszámlálás

A *reference count* az objektum élettartamáért felelős referenciákat számolja. Az utolsó referencia elengedése engedélyezheti a felszabadítást.

A kényes pont a referencia megszerzése. Ez a gondolatmenet hibás:

> Beolvasom a megosztott pointert, majd atomikusan növelem a benne található referenciaszámlálót.

A két művelet között az utolsó tulajdonos felszabadíthatja az objektumot. A számláló növelése ekkor már megszűnt élettartamú objektumot érint.

Biztonságos kiindulás például, ha a megosztott pointer beolvasása és a referencia megszerzése ugyanazon mutex alatt történik, mint az eltávolítás. Már birtokolt referenciából további referencia létrehozásakor viszont az eredeti referencia biztosítja az élettartamot. A ciklusok, a számláló túlcsordulása és az utolsó elengedés szinkronizációja külön kezelendő.

15.5.2.2 Epoch és hazard pointer

Epoch-alapú kezelésnél az olvasó védett szakaszba lép. Az eltávolított objektum először *retired*, vagyis visszavonásra jelölt állapotba kerül. Csak akkor szabadítható fel, amikor egyetlen olyan korábbi olvasói szakasz sem aktív, amely még elérhette.

Hazard pointernél az olvasó fogalmilag:

1. beolvassa a megosztott pointert;
2. közzéteszi saját védelmi bejegyzésében;

3. újraellenőrzi a megosztott pointert;
4. csak sikeres ellenőrzés után dereferál;
5. a használat végén megszünteti a védelmet.

Ez **nem kész implementációs recept**. A publikálás, az újraellenőrzés és a felszabadító szál vizsgálata közötti memory-order kapcsolat bizonyítandó; néhány release/acquire művelet önmagában nem bizonyítás.

A hazard pointer megakadályozhatja a védett objektum felszabadítását, de nem old meg minden ABA-helyzetet: egy még élő csomópont eltávolítása és visszahelyezése továbbra is algoritmikus problémát okozhat.

15.5.3 Kidolgozott példa: másolat a zár alatt, felszabadítás utána

Konfigurációs állapot olvasásakor gyakran nincs szükség tartós borrowed pointerre. Elég a szükséges értéket a zár alatt kimásolni.

Laborprofil: ISO C17 és POSIX pthreads; Linux x86-64 System V, LP64, `CHAR_BIT=8`; választott fordítók a `lab-next` profil (profiltábla), libc a `lab-next` profil (profiltábla). Az alábbi példa nem közölt fordítási vagy mérési eredmény.

```
/* lifetime.c */
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

struct record {
    unsigned value;
};

static struct {
    pthread_mutex_t mutex;
    struct record *current;
} state = { PTHREAD_MUTEX_INITIALIZER, NULL };

static void check(int error)
{
    if (error != 0) {
        fprintf(stderr, "pthread hibakod: %d\n", error);
        abort();
    }
}

static void *reader(void *unused)
{
    (void)unused;

    for (unsigned i = 0; i < 10000; ++i) {
        unsigned snapshot = 0;

        check(pthread_mutex_lock(&state.mutex));
        if (state.current != NULL)
            snapshot = state.current->value;
```

```
        check(pthread_mutex_unlock(&state.mutex));

        /* Itt mar csak a sajat masolatot hasznaljuk. */
        if (snapshot != 0 && snapshot != 42)
            abort();
    }

    return NULL;
}

int main(void)
{
    state.current = malloc(sizeof *state.current);
    if (state.current == NULL)
        return EXIT_FAILURE;
    state.current->value = 42;

    pthread_t thread;
    check(pthread_create(&thread, NULL, reader, NULL));

    check(pthread_mutex_lock(&state.mutex));
    struct record *retired = state.current;
    state.current = NULL;
    check(pthread_mutex_unlock(&state.mutex));

    free(retired);

    check(pthread_join(thread, NULL));
    check(pthread_mutex_destroy(&state.mutex));
    return EXIT_SUCCESS;
}
```

A tanuló külön laborjában használható fordítási parancsok:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O1 -g -pthread \
    lifetime.c -o lifetime

clang -std=c17 -Wall -Wextra -Wpedantic -O1 -g -pthread \
    -fsanitize=thread lifetime.c -o lifetime-tsan
```

A `-pthread` és a sanitizer kapcsoló fordítóeszköz-szintű beállítás, nem ISO C nyelvi elem. A ThreadSanitizer futtatási támogatását a tényleges környezetben külön ellenőrizni kell.

Elem	Tulajdonos és élettartam	Érvénytelenítés
Dinamikus `record`	A megosztott hely kezelője, majd eltávolításkor a `main` helyi `retired` változójához kötött felelősség	`free(retired)`
Olvasói hozzáférés	Kölcsönzött, kizárólag a zárolt szakaszban	A használati jogosultság a feloldáskor véget ér
`snapshot`	Az olvasó saját automatikus objektuma	Az adott iteráció vége

Elem	Tulajdonos és élettartam	Érvénytelenítés
Mutex	A program kezeli; az olvasó befejezéséig használható	Sikeres join után történő megsemmisítés

Bizonyításvázlat. Ha az olvasó előbb szerzi meg a mutexet, befejezi a másolást, mielőtt az eltávolító beléphet. Ha az eltávolító lép be előbb, nullára állítja a pointert, ezért a későbbi olvasó nem dereferálja. Mivel pointer nem szökik ki a kritikus szakaszból, az eltávolítás utáni `free` biztonságos. A join nem a `record`, hanem a még használt szinkronizációs objektum lezárását teszi biztonságossá.

Ez POSIX mutex- és szál-életciklus-szerződésre épül. Allokációs hibakor a program hibakóddal kilép; pthread-hibakor szándékosan azonnal leáll. Ez egyszerűsített hibapolitika, nem szolgáltatás-üzemeltetési helyreállítás.

15.5.4 Előrehaladás: mit jelent a „nem blokkoló”?

Az *előrehaladási garancia* azt írja le, hogy milyen feltételekkel fejeződnek be a műveletek. Nem azonos az alacsony késleltetéssel.

Garancia	Jelentés
Obstruction-free	Egy művelet véges számú saját lépésben befejeződik, ha a szála kellően hosszú ideig egyedül fut
Lock-free	Folyamatos végrehajtás mellett rendszerszinten mindig történik műveletbefejezés; egy adott szál éhezhet
Wait-free	Minden művelet véges, előre korlátozott számú saját lépésben befejeződik, más szálak előrehaladásától függetlenül

A szokásos modellben:

$$\text{wait-free} \Rightarrow \text{lock-free} \Rightarrow \text{obstruction-free}.$$

A wait-free korlát függhet például a bemenet méretétől és a résztvevők rögzített számától. Ezek lépésszámra vonatkozó állítások: egyáltalán nem ütemezett szálnak nem adnak faliórás határidőt.

ISO C17: az `atomic_is_lock_free` az adott atomikus objektum műveleteinek tulajdonságáról tájékoztat, nem a teljes algoritmusról. Lock-free atomikus műveletekből felépített spinlock továbbra is várhat egy megállt zártulajdonosra.

A teljes hívási lánc számít. Egy lock-free mutatómódosítás köré helyezett blokkoló allocator, naplózás vagy reclamation-várakozás megváltoztathatja a teljes API garanciáját. Az epoch-alapú visszanyerésnél külön vizsgálandó, hogy az elakadt olvasó csak a memória felszabadítását akadályozza-e, vagy a műveleteket is.

15.5.4.1 Deadlock, livelock, starvation és fairness

Deadlock, vagyis holtpont esetén a résztvevők egymástól függően várnak. Két mutex ellentétes sorrendű megszerzése klasszikus ellenpélda. Következetes globális zársorrend megszüntetheti az ilyen körkörös várakozást.

Livelock esetén a szálak aktívak, de nem történik hasznos előrehaladás: például mindig egyszerre hátrálnak ki, majd egyszerre próbálkoznak újra.

Starvation, vagyis éhezés esetén mások haladnak, de egy résztvevő tartósan nem jut sorra. Ez lock-free algoritmusban is megengedett.

A *fairness* a kiszolgálás méltányosságára vonatkozik. Sem a helyes mutexhasználatból, sem a lock-free tulajdonságból nem következik automatikusan FIFO-sorrend vagy korlátos várakozás. A backoff javíthatja a gyakorlatot, de önmagában nem bizonyít fairness-t.

15.5.5 False sharing atomikus számlálókon

A Cache, locality, alignment és false sharing alapján külön objektumok is osztozhatnak egy cache line-on. Két szál saját atomikus számlálóját módosíthatja adatverseny nélkül, mégis ugyanannak a koherenciaegységnek a tulajdonjogáért versenyezhetnek.

Az alábbi **szemléltető layout** a fenti x86-64 laborprofilban is külön ellenőrzendő: 64 C bájtos cache line-t, 8 C bájtos számlálót és 64-re igazított kezdőcímet feltételez.

Elrendezés	Első számláló offsetje	Második számláló offsetje	Cache line
Tömör	0	8	Közös
Szétválasztott	0	64	Külön

Ezek nem az ABI-ból automatikusan következő layoutadatok. A tényleges `sizeof`, offset, igazítás és hardverjellemző ellenőrzendő; dinamikus tárolónál az allokáció igazítása is számít.

A `memory_order_relaxed` nem szünteti meg a false sharinget: az írás továbbra is koherenciaforgalmat okozhat. Megoldás lehet a külön elhelyezés vagy a szálankénti helyi számlálás ritkább összesítéssel. Utóbbi viszont módosítja a megfigyelhető számláló frissességi szerződését. Az eredményt mérni kell; a padding nagyobb memóriaigénnyel és rosszabb összesítési localityval járhat.

15.5.6 Diagnosztika: bizonyíték, de nem bizonyítás

A ThreadSanitizer dinamikus adatverseny-detektor. Egy jelzés konkrét végrehajtás problémájára adhat bizonyítékot; egy jelzés nélküli stresszteszt nem bizonyítja minden végrehajtás helyességét.

Különösen:

- a tisztán atomikus ABA-hiba nem feltétlenül adatverseny;
- az el nem ért use-after-free rejtve marad;
- a deadlock, az éhezés és a fairness nem vezethető le a TSan csendjéből;
- az instrumentálás módosítja az időzítést.

A stresszteszt változtassa a szálszámot, a szüneteket, az eltávolítás gyakoriságát és a tároló újrafelhasználását. A hibainjektálási módszerekkel együtt vizsgálja a részleges inicializálást és a leállítást is. A helyességi érvelés alapja azonban továbbra is az invariáns, az élettartamhatár és a szinkronizációs kapcsolat.

15.5.7 Feladatok

1. **Magyarázó:** Az ABA-táblázat minden műveletét `seq_cst` műveletté tesszük. Megszűnik a hiba? Segítene önmagában egy hazard pointer?
2. **Kódolási:** Írj a példához `copy_value(unsigned *out)` függvényt. Null pointer paraméternél adjon `-1`-et, hiányzó rekordnál `0`-t, sikeres másoláskor `1`-et. Ne adjon vissza borrowed pointert.
3. **Diagnosztikai:** Egy atomikus számlálókat használó szolgáltatás TSan alatt nem jelez hibát, négy szállal mégis lassabb, és egy worker hosszú ideig nem végez műveletet. Adj külön hipotézist a teljesítményromlásra és az éhezésre, valamint ellenőrzési tervet.

15.5.8 Megoldások

15.5.8.1 Az ABA értelmezése

Nem szűnik meg. A `seq_cst` rendezés mellett is létrejöhet a táblázat szerinti sorrend. A CAS az aktuális értéket vizsgálja, nem annak történetét. A hazard pointer az élettartamot védheti, de itt minden csomópont él: az eltávolítás–visszahelyezés okozta logikai ABA megmarad.

Ellenőrzési feltétel: az érvelés külön kezelje az értékegyezést, az objektum élettartamát és az adatszerkezeti tagságot.

15.5.8.2 Másoló API

Az előző fájlba illeszthető C17/POSIX-részlet; ugyanazokkal a parancsokkal fordítható:

```
static int copy_value(unsigned *out)
{
    if (out == NULL)
        return -1;

    int result = 0;
    check(pthread_mutex_lock(&state.mutex));
    if (state.current != NULL) {
        *out = state.current->value;
        result = 1;
    }
    check(pthread_mutex_unlock(&state.mutex));
    return result;
}
```

Az `out` a hívó tulajdonában lévő, élő, írható `unsigned` objektumra mutasson, amelyhez nincs szinkronizálatlan konkurens hozzáférés. A függvény csak a hívás idejére kölcsönzi; `0` és `-1` eredménynél nem módosítja.

Ellenőrzési feltétel: minden rekorddereferálás a mutex alatt történik, és semmilyen rekordpointer nem hagyja el a függvényt.

15.5.8.3 Teljesítmény és éhezés szétválasztása

A lassulás egyik hipotézise false sharing. Ezt igazoltan elkülönített számlálókkal, azonos munkamennyiséggel és reprodukálható szálelrendezéssel kell összehasonlítani.

Az éhezés oka lehet egy CAS-ciklus tartós vesztese vagy kedvezőtlen ütemezés. Mérj worker-enként befejezett műveletszámot és újrapróbálkozást, ne csak összesített throughputot.

Ellenőrzési feltétel: a jobb összteljesítmény önmagában nem igazol fairness-t; a TSan csendje egyik hipotézist sem dönti el.

15.5.9 Összefoglaló

Az atomikus hozzáférés, az objektum élettartama és az előrehaladás három külön szerződés. Az ABA az értékazonosság korlátját mutatja meg; a reclamation azt szabályozza, mikor tűnhet el végleg az objektum. Kiindulásként a mutex alatt készített másolat gyakran egyszerűbb és jobban bizonyítható, mint egy tartós borrowed pointer. Lock-free vagy wait-free állítást csak a teljes műveletre, világos feltételekkel szabad tenni; a mérés és a stresszteszt ezt kiegészíti, nem helyettesíti.

16 C hibakeresés, tesztelés és biztonságos kód

16.1 Debugger, core dump és optimalizált program

Egy C program incidensvizsgálatakor a folyamat állapotát vissza kell kapcsolnunk a forrásprogram objektumaihoz: melyik függvény futott, milyen pointert kapott, meddig élt a hivatkozott objektum, és melyik művelet sértette meg a szerződést? A debugger ezt a kapcsolatot segít helyreállítani, de nem szolgáltat automatikus helyességbizonyítást.

Az ABI- és optimalizálásdiagnosztikai labor alapján a gépi hívási keret nem feltétlenül követi a forrás függvényhatárait. Az élettartam- és memóriahibák vizsgálati laborjából pedig fontos felidézni: az olvasható memóriacím önmagában nem bizonyítja, hogy ott még él a megfelelő C objektum.

16.1.1 Mit mutat a debugger?

GDB-eszközmodell. A debugger a gépi állapotot, a végrehajtható fájlt és a debug információt együtt értelmezi.

Fogalom	Jelentés és korlát
Breakpoint, töréspont	Megállás függvénynél, forrássornál vagy gépi utasításnál. Optimalizáláskor egy forrássorhoz több hely tartozhat, vagy nem tartozik külön utasítás.
Watchpoint, adatfigyelési pont	Egy kifejezés értékváltozását figyeli. A hardveres támogatás méret- és darabszámkorlátos; a szoftveres megoldás lassú lehet.
Backtrace, hívási visszakövetés	A debugger által rekonstruált hívási lánc. Nem teljes végrehajtási történet.
Frame, hívási keret	Egy híváshoz rendelt vizsgálati környezet: argumentumok, lokális változók, visszatérési hely. Inline függvényhez fizikai stack frame nélkül is megjelenhet logikai frame.

A `bt` megmutatja a hívási láncot, a `frame N` kiválaszt egy keretet, az `up` a hívó felé lép. Ez a kiválasztás nem futtatja a programot: azt változtatja meg, hogy például az `info locals` melyik függvény változóit értelmezi.

A watchpoint sem időgép. Egy betöltött core dumpban már nem figyelhetjük meg a korábbi írást; ehhez élő, reprodukált futás szükséges.

16.1.2 Kidolgozott példa: szerződéssértés UB nélkül

A következő programban szándékosan hibás a ciklusfeltétel. A hozzáférést ellenőrző függvény azonban még a tömbhatár átlépése előtt megszakítja a végrehajtást.

ISO C17. A példa nem támaszkodik nem definiált viselkedésre. Az `abort()` rendellenes befejezést kezdeményez; a core fájl létrejötte már platform- és konfigurációfüggő.

Választott, helyben ellenőrizendő laborprofil: Linux, x86-64 System V ABI, LP64, 8 bites C bájt, 4 C bájtos `int`; a `lab-next` profil (profiltábla), GNU binutils 2.43, a `lab-next` profil (profiltábla) és GDB 15.2. A parancsok ehhez választott laborutasítások, nem dokumentált futtatási eredmények.

A `batch.c` teljes tartalma:

```
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>

struct batch {
    size_t count;
    int values[3];
};

static int value_at(const struct batch *batch, size_t index)
{
    if (batch == NULL ||
        batch->count > sizeof batch->values / sizeof batch->values[0] ||
        index >= batch->count) {
        fputs("batch bounds contract violated\n", stderr);
        abort();
    }

    return batch->values[index];
}

int main(void)
{
    struct batch owned = {
        .count = 3,
        .values = {10, 20, 30}
    };
    int total = 0;
    size_t i = 0;

    for (; i <= owned.count; ++i) { /* Szándékos logikai hiba. */
        total += value_at(&owned, i);
    }

    printf("%d\n", total);
    return EXIT_SUCCESS;
}
```

A `value_at` előfeltétele: a nem null `batch` pointer élő, olvasható `struct batch` objektumra mutat, amelyet közben más végrehajtás nem módosít. A nullvizsgálat nem tudja ellenőrizni ezt az élettartam-szerződést.

Szerep	Objektum vagy hozzáférés	Élettartam és érvénytelenítés
Tulajdonos	A `main` lokális `owned` objektuma	Automatikus tárolású; normál esetben a blokk elhagyásáig él.
Tároló	Az `owned` objektum részeként tárolt `values` tömb	Élettartama az azt tartalmazó objektumhoz kötött.
Kölcsönzött hozzáférés	A `value_at` `batch` paramétere	A hívás alatt használható; nem kerül eltárolásra vagy visszaadásra.
Hibakimenet	Diagnosztikai üzenet és `abort()`	Nem ad vissza érvénytelen elemet, és nem olvas tömbhatáron túl.

GNU eszközlánc. Készítsünk diagnosztikai és optimalizált változatot:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Og -g3 \
    -fno-omit-frame-pointer -Wl,--build-id \
    batch.c -o batch-debug

gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -g3 \
    -fno-omit-frame-pointer -Wl,--build-id \
    batch.c -o batch-opt
```

A `-g3` debug információt kér, nem tiltja az optimalizálást. A frame pointer megtartása egyes esetekben segíti a visszakövetést, de nem garantálja a lokális változók elérhetőségét.

16.1.2.1 Élő vizsgálat

Az első `value_at` hívásnál megálló munkamenet:

```
gdb -q ./batch-debug \
    -ex 'break value_at' \
    -ex run \
    -ex bt \
    -ex 'info args' \
    -ex 'print *batch' \
    -ex 'print batch->values[0]@3' \
    -ex 'x/3wd batch->values'
```

A `print *batch` típusosan értelmezi a struktúrát. A `@3` GDB-szintaxis három egymás melletti elemből képez megjelenítendő mesterséges tömböt; nem C operátor.

Az `x/3wd` nyers memóriavizsgálat: három, egyenként négy bájtos egységet jelenít meg előjeles decimális formában. Ez a választott laborprofilban illeszkedik az `int` reprezentációméretéhez, de nem általános C-garancia. Az `x` nem ellenőriz ownershipet, objektumhatárt vagy élettartamot.

Az `up` után a `main` keretében az `info locals` megmutathatja az `owned`, `i` és `total` értékét. A `watch i`, majd `continue` segítségével követhetjük az index változását. A töréspont és a

watchpoint felváltva is megállíthatja a programot; az `info breakpoints` listájából kiválasztott pontot a `disable N` paranccsal kapcsolhatjuk ki.

Levezetés. A logikai tartomány $[0, n)$, ahol $n = 3$. A hibás `i <= n` feltétel az $i = 3$ értéket is beengedi. Ekkor az `index >= batch->count` igaz, ezért a függvény megszakítja a futást a dereferálás előtt. A forrás alapján várt hibakép tehát `index == count == 3`; ez nem állítás egy ténylegesen megfigyelt debugger-kimenetről.

16.1.3 Core dump és azonosítható build

Linux. A core dump egy folyamat állapotának utólag vizsgálható pillanatképe: tartalmazhat regisztereket, szálállapotokat és memóriatartományokat. Nem feltétlenül tartalmaz minden leképezést vagy minden szükséges fájlt, és nem őrzi meg az események történetét.

Külön laborban, a debuggeren kívüli futáshoz:

```
ulimit -c unlimited
cat /proc/sys/kernel/core_pattern
./batch-debug
```

A `ulimit` csak az adott shellből induló folyamat korlátját módosítja, a megengedett felső korlátig. A `core_pattern`, a jogosultságok, a folyamat dumpolhatósága és a szolgáltatáskezelő szabályai továbbra is meghatározók. Konténerben a fogadó mechanizmus akár a hoston működhet.

Systemd-alapú környezetben a `coredumpctl list`, majd a megfelelő bejegyzéshez tartozó `coredumpctl dump PID --output=core.incident` használható; a `PID` helyére a kiválasztott azonosító kerül. Ez systemd-eszköz, nem ISO C vagy POSIX szolgáltatás.

A megfelelő core betöltése:

```
gdb -q ./batch-debug core.incident \
    -ex 'info files' \
    -ex 'info sharedlibrary' \
    -ex 'thread apply all bt'
```

A megszakítás helye várhatóan az `abort()` megvalósításában vagy az alatta futó libc-kódban látszik. Ez nem jelenti azt, hogy a libc hibás. A backtrace-ben keressük meg az első saját kódhoz tartozó keretet, majd a tényleges sorszámával válasszuk ki.

A minimális hibakép:

- a megszakítás oka `SIGABRT`;
- a saját kódbeli ellenőrzés elutasította az indexet;
- a releváns állapot várhatóan `index == count`;
- a hívó ciklusfeltétele megengedi ezt az értéket.

Ez már javítható állítás, nem pusztán egy stack trace.

16.1.3.1 Bináris, szimbólum és megosztott könyvtár

A forráscommit egyezése kevés. Más fordító, kapcsolók vagy függőségek más gépi kódot és címkiosztást eredményezhetnek. Az eredeti végrehajtható fájlt, megosztott könyvtárakat és a hozzájuk tartozó debug fájlokat kell megőrizni.

GNU ELF-eszközlánc. A build-id egy buildhez rendelt azonosító, amely például a következő paranccsal vizsgálható:

```
readelf -n batch-opt
```

Hasznos az artefaktumok összerendeléséhez, de önmagában nem kriptográfiai hitelességigazolás.

Külön debug fájl készítésének példája:

```
objcopy --only-keep-debug batch-opt batch-opt.debug
objcopy --strip-debug batch-opt
objcopy --add-gnu-debuglink=batch-opt.debug batch-opt
```

A `.gnu_debuglink` a megfelelő debug fájl megtalálását és ellenőrzését segíti. A core mellé ne utólag újrafordított „azonos verziót”, hanem az adott kiadási artefaktumot adjuk.

Más gépről származó core esetén az eredeti gyökérfájlrendszer másolatához a `set sysroot /srv/incident/rootfs`, a debug fájlokhoz a `set debug-file-directory /srv/incident/symbols` állítható be, lehetőleg a core betöltése előtt. Az `info sharedlibrary` segít ellenőrizni a felismert könyvtárakat. A build-id-t és a szimbólumegyezést minden releváns megosztott könyvtárnál vizsgáljuk; a helyi libc nem automatikusan megfelelő.

A core érzékeny adatokat, például hitelesítő adatokat is tartalmazhat. Ugyanúgy kontrollált hozzáféréssel és megőrzési idővel kezeljük, mint más incidensartefaktumot.

16.1.4 Optimalizált változók, inline keretek és assembly

GDB és compiler debug információ. Az `<optimized out>` nem azt jelenti, hogy a változó nulla vagy inicializálatlan. Azt jelzi, hogy a debugger az adott ponton nem tudja rekonstruálni az értékét. A fordító megszüntethette a tárolását, összevonhatta más számítással, vagy a debug információ nem írja le megfelelően az aktuális helyét.

Az optimalizált példában a fordító akár a számítások jelentős részét is elhagyhatja, mert azok nem befolyásolják az elérhető megfigyelhető hatásokat. A `value_at` inline beépítése után külön gépi hívás nélkül is szerepelhet a backtrace-ben.

Hasznos parancsok:

- `disassemble /s main`: assembly forrássorokkal kevert nézetben;
- `disassemble /r main`: assembly a gépi kód bájtjaival;
- `x/8i $pc`: utasítások az aktuális programszámlálótól;
- `info registers`: regiszterállapot;
- `info line *$pc`: az aktuális címhez rendelt forráshely.

A kevert nézet nem állítja vissza a forrás szerinti végrehajtási sorrendet. A `next` és `step` ezért kihagyhat sorokat, vagy látszólag visszafelé léphet. Gépi szinten a `stepi` használható, de a regisztereket az ABI és az adott utasítássorozat alapján kell értelmezni.

Gyakori hiba a debugger által olvasható pointert érvényesnek tekinteni, a legfelső frame-et gyökéroként kezelni, vagy hiányzó értékeket feltételezésekkel kitölteni. Helyette válasszuk külön a megfigyelt tényt, a forrásból levezetett állítást és a még ellenőrizendő hipotézist.

16.1.5 Feladatok

1. **Magyarázó:** Miért nem tömbtúlolvasás bizonyítéka ebben a példában a `SIGABRT`? Miért nem elegendő egy pointer olvashatósága az érvényességéhez?
2. **Kódolási:** Javítsd a ciklust. Vizsgáld meg a `count` értékeit: 0, 1 és 3. Adj meg ellenőrzési feltételt az eredményre és a hozzáférésekre.
3. **Diagnosztikai:** Az optimalizált core vizsgálatakor az `i` értéke nem érhető el, és a libc szimbólumaihoz eltérésjelzés tartozik. Milyen sorrendben folytatod az elemzést?

16.1.6 Megoldások

16.1.6.1 Magyarázó feladat

Az ellenőrzés az elemelérés előtt hívja az `abort()` függvényt, ezért ezen az útvonalon nem történik tömbtúlolvasás. Az olvasható memória lehet már megszűnt objektum tárolója is. Az ellenőrzés alapja a vezérlési út és az élettartam-szerződés, nem egy önmagában sikeres memóriakiolvasás.

16.1.6.2 Kódolási feladat

A javított feltétel `i < owned.count`. A forrásból várt összegek rendre 0, 10 és 60.

Bizonyításvázlat: a ciklusmag csak $i < n$ mellett fut; az index nulláról indul és egyesével nő. Ezért minden meghívott hozzáférésre $0 \le i < n$. Ellenőrzési feltétel: a megfelelő összeg jelenik meg, és egyik érvényes eset sem éri el az `abort()` ágat.

16.1.6.3 Diagnosztikai feladat

Először az artefaktumegyezést kell helyreállítani: eredeti bináris, libc, további könyvtárak és debug fájlok. Ezután újra ellenőrizhető a backtrace. A hiányzó `i` helyett megvizsgálható a hívási argumentum, a struktúra állapota és az utasítássorozat, ha ezek rekonstruálhatók. Ha nem, ezt korlátként kell rögzíteni. Másik builddel történő reprodukció kiegészítő bizonyíték, nem az eredeti futás állapota.

16.1.7 Összefoglaló

A megbízható hibaképhez pontos artefaktumegyezés, megfelelő frame-választás és a C objektumszerződéseinek ismerete kell. A debugger pillanatképet értelmez; az optimalizálás és a hiányzó adatok korlátozzák, mit tudhatunk meg belőle.

16.2 ASan, UBSan, MSan, TSan és Valgrind

A memóriahiba helye és a látható meghibásodás helye gyakran eltér. Egy határon túli írás után a folyamat még sokáig működhet; egy felszabadított objektum olvasása pedig csak más allokációs mintázat mellett okozhat összeomlást. A dinamikus diagnosztikai eszközök célja, hogy a hibás művelethez közelebb adjanak értelmezhető jelzést.

Az allokációs hiba-injektálás a hibautak kezelését vizsgálta; itt elsősorban a végrehajtott memóriaműveleteket ellenőrizzük. A konkurens élettartam szabályaihoz a TSan, a debuggeres vizsgálathoz pedig a szimbolizált hívási lánc kapcsolódik. Egyetlen hibamentes futás sem igazolja a teljes program helyességét: csak adott bemenetek, végrehajtási utak és szálütemezések kerültek vizsgálatra.

16.2.1 Eszközválasztás a hibatípus alapján

A **sanitizer** fordítói és futtatókörnyezeti diagnosztikai mechanizmus. A fordító ellenőrzésekkel egészíti ki, azaz **instrumentálja** a programot; a kapcsolódó futtatókönyvtár nyilvántartásokat vezet és jelentéseket készít. A Valgrind/Memcheck ezzel szemben a gépi program végrehajtását felügyeli, ezért nem igényel sanitizeres újrafordítást.

Eszköz	Elsődleges cél	Lényeges korlát
AddressSanitizer, ASan	Határon túli hozzáférés, use-after-free, egyes élettartam- és felszabadítási hibák	Nem ismeri minden alkalmazásszintű objektumhatár jelentését
UndefinedBehaviorSanitizer, UBSan	Kiválasztott nem definiált műveletek	Nem ellenőrzi a C összes UB-szabályát
MemorySanitizer, MSan	Inicializálatlan értékek felhasználása	Különösen érzékeny a függőségek instrumentálására
ThreadSanitizer, TSan	Adatversenyek	Nem általános logikai race- vagy holtpontdetektor
LeakSanitizer, LSan	Elvesztett dinamikus allokációk	Az elérhető memória is lehet alkalmazásszintű szivárgás
Valgrind/Memcheck	Érvénytelen hozzáférés, definiálatlan értékhasználat, leak	Jelentős futási többletköltség; nem általános UB-ellenőrző

16.2.1.1 ASan és UBSan: egymást kiegészítő ellenőrzések

Az ASan a programmemóriához kiegészítő nyilvántartást, **árnyékmemóriát** (*shadow memory*) rendel. Többek között jelöli az allokációk körüli tiltott területeket és a felszabadított tárolót. Így észlelhet heap-, stack- és globális bufferhatár-sértést, kétszeres felszabadítást vagy nem megfelelő cím átadását a `free` számára.

A hatókör elhagyása utáni használathoz (*use-after-scope*) a Clang profiljában explicit megadható a `-fsanitize-address-use-after-scope`. A függvényből visszatérés utáni használat külön mechanizmus; támogatása és beállítása platform- és verziófüggő. Az ASan nem teljes lifetime-ellenőrző.

Fontos vakfolt az objektumon belüli határátlépés: egy struktúratag tömbjéből a következő tagba történő hibás hozzáférés nem feltétlenül érint tiltott területet. Ugyanígy nem feltétlenül jelez, ha egy nagy allokáción belül túllépjük az alkalmazás által érvényesnek tekintett adathosszt.

ISO C17-szabály és eszközellenőrzés különbsége: a signed egész túlcsordulás nem definiált viselkedés; a UBSan megfelelő ellenőrzése ezt a végrehajtott műveletnél jelezheti. A `-fsanitize=undefined` csoport többek között igazítási, null pointerrel kapcsolatos, eltolási, aritmetikai és bizonyos tömbhatár-ellenőrzéseket kapcsol be. Nem ad általános effective type-, aliasing- vagy élettartam-ellenőrzést.

Az unsigned aritmetika modulo viselkedése önmagában nem UB, ezért az unsigned méretszámítás körbefordulásának felismerésére ez a csoport nem elegendő. A Clang külön unsigned-overflow ellenőrzése hasznos hibakeresési kiegészítés lehet, de nem szabad nyelvi tilalomként értelmezni.

16.2.1.2 MSan, TSan és a leak-ellenőrzések

Az MSan nem csupán azt követi, hogy egy cím elérhető-e, hanem azt is, hogy az ott tárolt érték inicializált-e. Egy inicializálatlan érték másolása továbbviheti a jelölést; a jelentés például akkor születhet meg, amikor az érték vezérlési döntést befolyásol. Az origin tracking az eredet visszakeresését segíti.

MSan esetén a nem instrumentált függőség különösen veszélyes határ: hibák maradhatnak rejtve, illetve helyesen előállított adatok is definiálatlannak látszhatnak. A saját kód mellett a releváns függőségeket is instrumentálni kell. A rendszerkönyvtári hívások egy részét speciális elfogó függvények, **interceptorok** kezelik; ez nem általános felmentés minden külső könyvtárra.

A TSan a memóriahozzáféréseket és az általa ismert szinkronizációt követi. Az adatverseny meghatározásához a korábban tárgyalt happens-before viszony szükséges. A két külön zárolás között végrehajtott „ellenőrzés, majd módosítás” például lehet logikailag hibás akkor is, ha minden hozzáférés adatversenymentes. A saját szinkronizációs megoldásokat a TSan nem feltétlenül érti.

Clang-eszközszabály: az ASan, MSan és TSan egymással nem kombinálható sanitizerprofilok. A TSanhoz ezért külön bináris és külön futtatás készüljön; ne próbáljuk az összes ellenőrzést egyetlen kapcsolólistába zsúfolni.

A LSan jellemzően a leak-vizsgálat időpontjában elérhetőségi elemzést végez. Linuxon gyakran az ASan runtime részeként használható, de az integráció platformfüggő. Az önálló `-fsanitize=leak` profil elsősorban runtime-linkelést igényel, nem ASan-szerű hozzáférés-instrumentálást. Korai összeomlás vagy ASan-leállás esetén a szokásos kilépéskori leak-vizsgálat elmaradhat.

16.2.2 A teljes build instrumentálása

A következő parancsok **nem futtatott laborutasítások**. Választott mintaprofiljuk: ISO C17, Linux x86-64 System V ABI, Clang/LLVM és compiler-rt 18.1.8, glibc 2.39; a Memcheck-példához Valgrind 3.23.0. A telepített csomagokat, disztribúciós javításokat és támogatást a laborban ellenőrizni kell. A `clang` és `llvm-symbolizer` itt a rögzített LLVM-telepítés eszközeit jelenti.

A teljes instrumentálási lánc három részből áll:

1. A saját fordítási egységeket — a tesztelt könyvtárakat is — a megfelelő sanitizerkapcsolókkal fordítjuk.
2. A végső linket a **compiler driverrel**, itt a `clang` paranccsal végezzük, a szükséges sanitizerkapcsolókkal.
3. Futáskor a megfelelő runtime és ténylegesen a kívánt instrumentált megosztott könyvtárak töltődnek be.

A nyers `ld` meghívása nem helyettesíti automatikusan a driver runtime-választását és linkbeállításait. A csak tesztprogramon bekapcsolt sanitizer nem instrumentálja utólag a korábban elkészített statikus archívumot.

Profil	Fordításkor	Végső driveres linkkor
ASan + UBSan	`-fsanitize=address,undefined`	Ugyanez
MSan	`-fsanitize=memory -fsanitize-memory-track-origins=2 -fPIE`	`-fsanitize=memory -fsanitize-memory-track-origins=2 -pie`
TSan	`-fsanitize=thread -fPIE`	`-fsanitize=thread -pie`
Memcheck	`-g`, sanitizer nélkül	Sanitizer nélkül

Minden profilnak külön buildkönyvtár kell. Ez az objektumfájlok és függőségek véletlen keverését is megelőzi. Az instrumentálatlan külső kódot lefedettségi korlátként kell nyilvántartani, nem „ellenőrzött” komponensként.

16.2.3 Kidolgozott példa: olvasás az allokáció után

A következő három fájl önálló C17-példát alkot. **A `checksum.c` szándékosan hibás, UB-t tartalmazó tanítási példa; nincs garantált kimenete.**

A kívánt függvényszerződés: pozitív `n` esetén `p` legalább `n` inicializált `unsigned char` elem olvasását engedje; nulla hossz esetén null pointer is megengedett. A függvény nem tartja meg a címet.

`checksum.h`:

```
#ifndef CHECKSUM_H
#define CHECKSUM_H

#include <stddef.h>

unsigned checksum(const unsigned char *p, size_t n);

#endif
```

`checksum.c`:

```
#include "checksum.h"

unsigned checksum(const unsigned char *p, size_t n)
{
    unsigned sum = 0;
    for (size_t i = 0; i <= n; ++i) { /* HIBA: <= */
        sum += p[i];
    }
    return sum;
}
```

`main.c`:

```c
#include "checksum.h"

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    const size_t n = 4;
    unsigned char *p = malloc(n * sizeof *p);
    if (p == NULL) {
        return EXIT_FAILURE;
    }

    for (size_t i = 0; i < n; ++i) {
        p[i] = (unsigned char)i;
    }

    printf("%u\n", checksum(p, n));
    free(p);
    return EXIT_SUCCESS;
}
```

Tulajdonos	Tároló és kölcsönzés	Élettartam, érvénytelenítés	Hibakimenet
A `main` felel a felszabadításért	`malloc`-tároló; a `checksum` csak olvasásra kölcsönzi	Sikeres allokációtól `free`-ig; a hívás közben nincs felszabadítás	Allokációs hibánál `EXIT_FAILURE`; a hibás indexelésnél UB

A `sizeof *p` értéke egy C bájt. A foglalás tehát négy C bájt, az érvényes indexek:

$$0 \le i < n.$$

A hibás ciklus eléri az $i = n$ értéket is. A `p + n` one-past pointer képzése megengedett, de a `p[n]`, vagyis `*(p + n)` olvasása nem.

Fordítás és külön laborfuttatás:

```
mkdir -p build/asan

clang -std=c17 -Wall -Wextra -Wpedantic -O1 -g \
  -fno-omit-frame-pointer -fno-optimize-sibling-calls \
  -fsanitize=address,undefined -fsanitize-address-use-after-scope \
  -fno-sanitize-recover=all \
  -c checksum.c -o build/asan/checksum.o

clang -std=c17 -Wall -Wextra -Wpedantic -O1 -g \
  -fno-omit-frame-pointer -fno-optimize-sibling-calls \
  -fsanitize=address,undefined -fsanitize-address-use-after-scope \
  -fno-sanitize-recover=all \
  -c main.c -o build/asan/main.o
```

```
clang -fsanitize=address,undefined \
  build/asan/main.o build/asan/checksum.o -o build/asan/checksum

ASAN_OPTIONS=detect_leaks=1:symbolize=1 build/asan/checksum
```

A várt diagnosztikai kategória heap-buffer-overflow olvasás. Ez nem rögzített futtatási eredmény; a jelentés pontos alakja eszközfüggő. A `-fno-sanitize-recover=all` az érintett UBSan-ellenőrzéseknél megakadályozza, hogy a teszt hiba után sikeres kilépésig folytatódjon.

A javítás: `i < n`.

Helyességi bizonyításvázlat: kezdetben `i == 0`; a ciklustörzs csak `i < n` mellett fut. A szerződés ekkor garantálja az adott elem olvashatóságát. A tulajdonos a teljes hívás alatt életben tartja a tárolót. Nulla hossznál nincs dereferálás. A javított példában a várható összeg $0 + 1 + 2 + 3 = 6$.

16.2.4 Memcheck, stack trace és suppression policy

Memcheckhez külön, nem sanitizeres bináris készüljön:

```
mkdir -p build/memcheck

clang -std=c17 -Wall -Wextra -Wpedantic -Og -g \
  main.c checksum.c -o build/memcheck/checksum

valgrind --tool=memcheck --leak-check=full \
  --show-leak-kinds=all --errors-for-leak-kinds=definite,possible \
  --track-origins=yes --error-exitcode=99 \
  build/memcheck/checksum
```

A Memcheck hasznos instrumentálatlan függőségeknél, valamint definiálatlan értékek és szivárgások vizsgálatakor. A heap-allokációk határát ismeri, de például a stacken belüli egyedi tömbhatárok felismerése korlátozott. Az ASan-bináris Memcheck alatti futtatása nem jó alapértelmezett kombináció.

A **hívási lánc** (*stack trace*) értelmezhetőségéhez kell a megfelelő bináris, megosztott könyvtár, debug információ és szimbolizáló. A `-g` és a frame pointer megtartása segít, de nem szünteti meg az optimalizálás minden hatását. Az `llvm-symbolizer` legyen elérhető; az eltérő buildből származó szimbólum félrevezető lehet.

Ne csak a jelentés legfelső sorát olvassuk:

- ASan use-after-free esetén a hibás hozzáférés, az allokáció és a felszabadítás láncát is;
- MSan esetén a felhasználást és — origin tracking mellett — az eredetet;
- TSan esetén mindkét ütköző hozzáférést és a szinkronizációs környezetet.

A **suppression** ismert jelentés célzott elnyomása, nem hibajavítás. Legyen hozzá felelős, indoklás, követhető hibajegy és felülvizsgálati feltétel. Külső függőség ismert hibájához szűk minta indokolható; saját kód teljes könyvtárának elnémítása nem.

A runtime-suppression és a fordításkori instrumentálási kizárás eltérő mechanizmus, eszközönként eltérő támogatással. Utóbbi szükséges követési információt is eltüntethet. Egy MSan-instrumentálási hiányt ezért nem helyes egyszerűen elnyomni.

16.2.5 Feladatok

16.2.5.1 Eszközválasztás

Egy parser inicializálatlan lokális változó alapján választ ágat; egy kapcsolatkezelő felszabadítás után olvas; két szál szinkronizáció nélkül ír ugyanabba a nem atomikus objektumba. Mely profilokat választanád? Mit bizonyít egy tiszta futás?

16.2.5.2 Javítás és határteszt

Javítsd a `checksum` ciklusát. Ellenőrizd a nulla hosszú bemenetet, egyetlen `9` értékű elemet és a példában szereplő négy elemet. Miért nem kell nulla hossznál érvényes elemet biztosítani?

16.2.5.3 Builddiagnózis

Egy CI-feladat csak a `main.c` fájlt fordítja ASannal, egy régi `libparser.a` archívumot használ, `ld`-vel linkel, majd ugyanebbe a profilba TSan-t is bekapcsol. Mi a javítás sorrendje?

16.2.6 Megoldások

16.2.6.1 Eszközválasztás indoklása

Az inicializálatlan értékhez MSan vagy Memcheck, a use-after-free hibához ASan, az adatversenyhez TSan illik. MSanhoz a függőségi lefedettséget is ellenőrizni kell. A tiszta futás kizárólag azt mutatja, hogy az adott végrehajtásban az eszköz nem jelentett felismert hibát; nem igazolja más bemenetek vagy ütemezések helyességét.

16.2.6.2 Határteszt indoklása

A javítás `i < n`. A helyes implementáció `main` függvényébe illeszthető ellenőrzés:

```
const unsigned char one[] = {9};
const unsigned char four[] = {0, 1, 2, 3};

if (checksum(NULL, 0) != 0 ||
    checksum(one, 1) != 9 ||
    checksum(four, 4) != 6) {
    return EXIT_FAILURE;
}
```

A tömbök tárolóját a hívó biztosítja; a kölcsönzés a hívásokra korlátozódik, és legkésőbb a blokk elhagyásakor érvénytelenné válna. Nulla hossznál a ciklustörzs nem fut, tehát a null pointert nem dereferáljuk. Ellenőrzési feltétel a három elvárt érték és a diagnosztika nélküli futás, a szerződés kézi ellenőrzésével együtt.

16.2.6.3 A build javítása

Először külön ASan- és TSan-buildkönyvtárat kell létrehozni. Ezután minden saját fordítási egységet, köztük a `libparser.a` tagjait is újra kell fordítani az adott profillal. A végső linket `clang` végezze a megfelelő sanitizerkapcsolóval.

Az ellenőrzés tárgya nem pusztán a sikeres link: a buildnaplóban minden érintett fordítás és a végső link parancsát is vizsgálni kell, valamint a futáskor betöltött függőségeket. A két profil külön futtatandó.

16.2.7 Összefoglaló

A diagnosztikai profil hibatípushoz választott végrehajtási környezet. Az ASan és UBSan gyakran közös alap; az MSan és TSan külön buildet és körültekintő függőségkezelést igényel. A LSan és Memcheck további nézőpontot ad. A használható eredmény feltétele a helyes instrumentálási lánc, a driveres linkelés, az értelmezhető jelentés és a szűk suppression policy — nem pusztán a nulla kilépési kód.

16.3 Unit test, property test és fuzzing

A parser tesztelésének kiindulópontja nem a kód elágazásainak száma, hanem az API szerződése: milyen bemenet érvényes, milyen eredmény tartozik hozzá, és mi marad változatlan hiba esetén? A bounds-ellenőrzött parsernél megfogalmazott invariánsokat most végrehajtható ellenőrzésekké alakítjuk. Az API-szerződések adják az elvárt viselkedést, a sanitizerek pedig további hibajelzőként működnek.

A cél három egymást kiegészítő módszer használata: konkrét példák unit tesztje, általános tulajdonságok ellenőrzése és új bemenetek automatizált keresése.

16.3.1 A szerződéstől a tesztig

A **unit teszt** egy elkülönített programrész viselkedését ellenőrzi ismert bemenet és elvárt eredmény alapján. A **tulajdonságalapú teszt** (*property test*) bemenetek egy családjára ellenőriz állítást. A **fuzzing** automatizált bemenet-előállítással keres hibát; a **lefedettségvezérelt fuzzing** (*coverage-guided fuzzing*) a végrehajtási visszajelzés alapján részesít előnyben új kódutakat elérő bemeneteket.

Mindhárom módszerhez kell **tesztorákulum**: szabály, amely eldönti, hogy a megfigyelt viselkedés helyes-e. Az összeomlás hiánya önmagában gyenge orákulum: egy parser biztonságosan visszaadhat hibás értéket is.

16.3.1.1 Egy kisméretű codec szerződése

A példaprotokoll két oktetten, big-endian sorrendben tárolja a payload hosszát, majd pontosan ennyi payload-oktett következik. A megengedett maximum 1024 oktett. A decoder a lezárt, teljes keretet kapja; csonka keretet és további adatot egyaránt elutasít.

Laborprofil: ISO C17; Linux x86-64, System V ABI, LP64, `CHAR_BIT == 8`. Választott eszközverziók: a `lab-next` profil (profiltábla), illetve Clang és compiler-rt 19.1.7. Ezek rögzített

laborcélok, nem itt ellenőrzött futtatási eredmények; a telepített komponenseket és kapcsolókat helyben ellenőrizni kell.

A `codec.h` teljes tartalma:

```
#ifndef CODEC_H
#define CODEC_H

#include <limits.h>
#include <stddef.h>
#include <stdint.h>
#include <string.h>

_Static_assert(CHAR_BIT == 8, "This codec requires octets.");

enum { FRAME_MAX = 1024 };

struct byte_view {
    const unsigned char *data;
    size_t size;
};

/* 0: siker; 1: elutasítás. Hiba esetén nincs kimeneti módosítás. */
static int codec_decode(const unsigned char *p, size_t n,
                        struct byte_view *out)
{
    if (n < 2)
        return 1;

    size_t len = ((size_t)p[0] << 8) | (size_t)p[1];

    if (len > FRAME_MAX || len != n - 2)
        return 1;

    *out = (struct byte_view){p + 2, len};
    return 0;
}

static int codec_encode(const unsigned char *p, size_t n,
                        unsigned char *dst, size_t cap,
                        size_t *written)
{
    if (n > FRAME_MAX || n > SIZE_MAX - 2 || cap < n + 2)
        return 1;

    dst[0] = (unsigned char)(n >> 8);
    dst[1] = (unsigned char)(n & 255u);
    if (n != 0)
        memcpy(dst + 2, p, n);

    *written = n + 2;
    return 0;
}
```

```
#endif
```

ISO C17-előfeltételek: a bemeneti pointerhez tartozó hossz ténylegesen olvasható tartományt jelöl; nulla hossznál a bemeneti pointer lehet null. A kimeneti paraméterek érvényes objektumokra mutatnak. Az encoder célterülete legalább `cap` C bájt írható tárolót jelöl, és nem fed át a bemenettel vagy a `written` objektummal. A decoder `out` objektuma sem fed át a bemenettel. Ezeket a pointerérvényességi feltételeket a függvények nem tudják általánosan ellenőrizni.

Objektum vagy hozzáférés	Tulajdonos, élettartam és érvényesség
Bemeneti tároló	A hívó tulajdona; a hívás alatt olvasható és változatlan
Encoder célterülete	A hívó tulajdona; a hívás alatt írható
Decoder által visszaadott nézet	Kölcsönzött hozzáférés; nem szabad felszabadítani
Nézet érvénytelenítése	A bemeneti tároló felszabadítása, élettartamának vége vagy áthelyezése
Bemenet felülírása	A nézet címe érvényes maradhat, de már nem az eredeti payloadot mutatja
Hibakimenet	`1`; a kimeneti objektumok és az encoder célterülete változatlanok

Üres payloadnál a nézet pointere lehet one-past érték; nem dereferálható.

16.3.1.2 Mérethatárok levezetése

Legyen n a payload hossza, $h = 2$ a fejléc mérete, c a célkapacitás. Sikeres kódoláshoz:

$$n \le 1024, \qquad n \le \text{SIZE_MAX} - h, \qquad c \ge n + h.$$

Előbb az összeadás reprezentálhatóságát vizsgáljuk, és csak utána képezzük az összeget. A decoder hasonlóan előbb ellenőrzi az $n \ge 2$ feltételt, majd kivonással számolja a tényleges payloadméretet.

Bizonyításvázlat. Az encoder második feltétele biztosítja, hogy `n + 2` ne csorduljon körbe; a harmadik elegendő célterületet követel. A decoder első ellenőrzése után `n - 2` nem csordul alul. A hosszmező és a tényleges payloadhossz egyenlősége egyszerre zárja ki a csonkolást és a záró többletadatot.

Itt az 1024-es korlát már önmagában kizárja az összeadási túlcsordulást. Az explicit ellenőrzés a méretképzés szabályát teszi láthatóvá.

16.3.2 Boundary tesztek és round-trip tulajdonságok

A **határértékteszt** (*boundary test*) a szerződés váltópontjait és azok közvetlen környezetét vizsgálja.

Eset	Elvárt viselkedés
Teljes bemeneti hossz 0 vagy 1	Elutasítás: nincs teljes fejléc
Payloadhossz 0, 1, 1024	Siker megfelelő keretben

Eset	Elvárt viselkedés
Payloadhossz 1025	Encoder-elutasítás
Érvényes keret bármely valódi prefixe	Elutasítás: csonkolás
Érvényes keret egy további oktettel	Elutasítás: többletadat
Szükségesnél eggyel kisebb célterület	Elutasítás, változatlan kimenet
Reprezentálhatatlan méretösszeg	Elutasítás még a memóriaelérés előtt

A következő `test_codec.c` megvalósítja a fő határteszteket:

```
#include "codec.h"
#include <assert.h>

int main(void)
{
    unsigned char payload[FRAME_MAX + 1] = {0};
    unsigned char frame[FRAME_MAX + 3] = {0};
    const size_t lengths[] = {0, 1, FRAME_MAX};

    for (size_t k = 0; k < sizeof lengths / sizeof lengths[0]; ++k) {
        size_t n = lengths[k];
        size_t used = 0;
        struct byte_view view = {NULL, 0};

        assert(codec_encode(payload, n, frame,
                            sizeof frame, &used) == 0);
        assert(used == n + 2);
        assert(codec_decode(frame, used, &view) == 0);
        assert(view.size == n);
        if (n != 0)
            assert(memcmp(view.data, payload, n) == 0);

        for (size_t cut = 0; cut < used; ++cut)
            assert(codec_decode(frame, cut, &view) == 1);

        assert(codec_decode(frame, used + 1, &view) == 1);

        used = 99;
        assert(codec_encode(payload, n, frame, n + 1, &used) == 1);
        assert(used == 99);
    }

    size_t used = 99;
    assert(codec_encode(payload, FRAME_MAX + 1, frame,
                        sizeof frame, &used) == 1);
    assert(codec_encode(payload, 0, frame, 0, &used) == 1);

    const unsigned char oversized[] = {0xff, 0xff};
    struct byte_view view = {NULL, 0};
    assert(codec_decode(oversized, sizeof oversized, &view) == 1);

    const unsigned char golden[] = {0x00, 0x01, 0x41};
```

```
    assert(codec_encode(golden + 2, 1, frame,
                        sizeof frame, &used) == 0);
    assert(used == sizeof golden);
    assert(memcmp(frame, golden, sizeof golden) == 0);
}
```

A bufferobjektumok tulajdonosa a `main`; a nézeteket csak ezen objektumok élettartamán belül használjuk. Az encoder újabb írása után a korábbi nézetből nem ellenőrizzük az előző tartalmat.

Jelölje E az encodert, D a decodert. Érvényes payloadra:

$$D(E(x)) = x.$$

Ez értékegyezést jelent, nem pointerazonosságot. Kanonikus, elfogadott keretre ezenfelül:

$$E(D(b)) = b.$$

A második tulajdonság nem általános minden formátumra: ha több reprezentáció ugyanazt az értéket jelenti, visszakódoláskor normalizált alak keletkezhet.

Gyakori hiba: kizárólag round-trip tesztre hagyatkozni. Két, egyformán rossz byte ordert használó függvény teljesítheti az első tulajdonságot. Ezért kell a független, ismert `00 01 41` referencia is.

16.3.3 Állapotmentes fuzz harness

A **harness** a fuzzert az API-hoz kapcsoló vékony adapter. Az alábbi `fuzz_codec.c` LLVM libFuzzer-specifikus belépési pontot használ; maga a codec továbbra is C17.

```
#include "codec.h"
#include <stdlib.h>

static void require(int condition)
{
    if (!condition)
        abort();
}

int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size)
{
    if (size > FRAME_MAX + 2u)
        return 0;

    unsigned char encoded[FRAME_MAX + 2];
    size_t used = 0;
    struct byte_view a = {NULL, 0};
    struct byte_view b = {NULL, 0};

    /* A tetszőleges bemenetet payloadként is vizsgáljuk. */
    if (size <= FRAME_MAX) {
        require(codec_encode(data, size, encoded,
                             sizeof encoded, &used) == 0);
```

```
        require(codec_decode(encoded, used, &a) == 0);
        require(a.size == size);
        if (size != 0)
            require(memcmp(a.data, data, size) == 0);
    }

    /* Ugyanazt a bemenetet teljes keretként is vizsgáljuk. */
    int first = codec_decode(data, size, &a);
    int second = codec_decode(data, size, &b);
    require(first == second);

    if (first == 0) {
        require(a.size == b.size);
        if (a.size != 0)
            require(memcmp(a.data, b.data, a.size) == 0);

        require(codec_encode(a.data, a.size, encoded,
                             sizeof encoded, &used) == 0);
        require(used == size);
        require(memcmp(encoded, data, size) == 0);
    }

    return 0;
}
```

A `data` tárolójának tulajdonosa a fuzzing runtime; csak a callback alatt kölcsönözzük. Az `encoded` automatikus tárolási idejű helyi tömb. Egyetlen pointer sem kerül globális állapotba, és egyik nézetet sem használjuk tárolójának megszűnése után. Szabálytalan protokolladatnál normális visszatérés történik; tulajdonságsértésnél `abort()` állítja meg a futást.

A harness nem használ órát, hálózatot vagy véletlen állapotot. Az azonos inputra kétszer végzett decode a determinisztikusság egy ellenőrzése, nem teljes bizonyítása. Állapotos modulnál minden hívásra új kontextust kell létrehozni, majd lezárni.

16.3.3.1 Fordítás, corpus és erőforráskorlát

A fordítási és futtatási parancsok külön tanulói laborhoz tartoznak:

```
gcc -std=c17 -O2 -g -Wall -Wextra -Wpedantic -UNDEBUG \
    test_codec.c -o test_codec

clang -std=c17 -O1 -g -Wall -Wextra -Wpedantic \
    -fno-omit-frame-pointer \
    -fsanitize=fuzzer,address,undefined \
    -fno-sanitize-recover=undefined \
    fuzz_codec.c -o fuzz_codec

./test_codec

mkdir -p corpus artifacts
./fuzz_codec corpus -artifact_prefix=artifacts/ \
    -max_len=1026 -timeout=2 -rss_limit_mb=512 -max_total_time=60
```

A `-UNDEBUG` biztosítja, hogy a unit tesztek `assert` ellenőrzései ne tűnjenek el. A harness `require` függvénye ettől független.

A **corpus** a megőrzött bemenetek halmaza. Hasznos kezdőminták az üres payload kerete, egy egyoktettes payload, maximális érvényes keret és csonkolt változataik. A fuzzer ezek mutálásával keres további lefedettséget. Az ASan és az UBSan a végrehajtott útvonalakon további hibákat jelezhet; a szemantikailag hibás, de memóriabiztos eredményt továbbra is a tulajdonságellenőrzés találja meg.

A korlátok külön feladatot látnak el:

- Az **input cap** a harness által feldolgozott méretet korlátozza.
- A **timeout** egy túl lassú feldolgozás ellen véd.
- A kampányidő és a memóriakorlát a labor erőforrás-felhasználását határolja.

A fuzzer korlátai mellé CI-szintű folyamat- és erőforrásfelügyelet is kell. A harness méretellenőrzése nem akadályozza meg egy túlméretes corpusfájl korábbi betöltését, ezért a corpus fájlméreteit is ellenőrizni kell.

Ez a codec nem rekurzív. Beágyazott formátumnál explicit mélységkeret szükséges, például legfeljebb 32 egymásba ágyazott elem; a következő belépés előtt kell elutasítani. A mélységkorlát mellett teljes munkakeret is indokolt lehet, mert a kis mélység nem zár ki nagyon széles struktúrát.

16.3.4 Minimalizálás és regresszió

Két külön műveletet nevezzünk meg:

- **Corpusminimalizálás:** a releváns lefedettséget kisebb bemenethalmazzal tartja fenn.
- **Hibabemenet-minimalizálás:** egy hibát előidéző inputot egyszerűsít, miközben megőrzi a hibajelzést.

LLVM libFuzzer-parancspéldák; az `artifacts/crash-ID` helyére a tényleges hibafájl kerül:

```
mkdir -p corpus_small
./fuzz_codec -merge=1 corpus_small corpus

./fuzz_codec -minimize_crash=1 \
    -exact_artifact_path=minimized.bin artifacts/crash-ID
```

A minimalizálás nem garantál globálisan legkisebb inputot. Ellenőrizni kell azt is, hogy a kisebb bemenet ugyanazt a gyökérokot éri-e el, nem egy másik hibát.

A javítás után az input kerüljön regressziós tesztbe. Ne csak az legyen az elvárás, hogy „nem omlik össze”: rögzítsük a visszatérési értéket, a kimenet tartalmát és hiba esetén annak változatlanságát.

16.3.5 Feladatok

1. **Magyarázó:** miért nem elég a `decode(encode(x)) == x` tulajdonság? Adj olyan közös encoder–decoder hibát, amelyet nem talál meg.
2. **Kódolási:** egészítsd ki a unit tesztet úgy, hogy elégtelen célkapacitásnál a teljes célbuffer és a `written` is bizonyíthatóan változatlan maradjon.
3. **Diagnosztikai:** feltételezzük, hogy a decoder első ellenőrzése tévesen `n < 1`. Miért érdekes az egyetlen nulla oktettből álló input? Írj regressziós tesztet a helyes viselkedésre.

16.3.6 Megoldások

16.3.6.1 Magyarázó feladat

Ha mindkét függvény little-endian hosszmezőt használ, egymással konzisztensen működhetnek, miközben megsértik a protokollt. Független referencia szükséges. Ellenőrzési feltétel: az egyoktettes `0x41` payload kódolása pontosan `00 01 41`.

16.3.6.2 Kódolási feladat

A következő C17-részlet a `test_codec.c` `main` függvényébe illeszthető:

```
unsigned char dst[4];
unsigned char before[4];
const unsigned char src[] = {0x41};
size_t written = 77;

memset(dst, 0xa5, sizeof dst);
memcpy(before, dst, sizeof dst);

assert(codec_encode(src, sizeof src, dst, 2, &written) == 1);
assert(written == 77);
assert(memcmp(dst, before, sizeof dst) == 0);
```

Három oktett kellene, de csak kettőt engedélyezünk. A helyi tömbök tulajdonosa a tesztfüggvény; nincs átfedés vagy kimenekített nézet. A visszatérési érték és mindkét változatlansági feltétel együtt alkotja az orákulumot.

16.3.6.3 Diagnosztikai feladat

A hibás ellenőrzés átengedi az egyoktettes bemenetet, majd a kód kiolvassa `p[1]` értékét a tömbön kívül. Ez UB; nincs garantált kimenete. A javított változat regressziós tesztje:

```
const unsigned char one[] = {0};
struct byte_view result = {NULL, 77};

assert(codec_decode(one, sizeof one, &result) == 1);
assert(result.data == NULL && result.size == 77);
```

A tesztet a korábbi unit tesztbe illesztjük. Elvárt eredménye az elutasítás és a kimenet változatlansága; sanitizeres futtatásban emellett nem várunk memóriahibajelzést.

16.3.7 Összefoglaló

A meaningful teszt a szerződésből származik: mérethatárt, értékazonosságot, kanonikus reprezentációt vagy hibagaranciát ellenőriz. A határtesztek célzottak, a property tesztek bemenetcsaládokat fednek le, a coverage-guided fuzzing pedig új végrehajtási utakat keres. Az izolált harness korlátos erőforrással dolgozik, a minimalizált hibából pedig pontos elvárást rögzítő regressziós teszt készül. Egyik hibamentes futás sem helyességigazolás; együtt azonban lényegesen erősebb ellenőrzési rendszert alkotnak.

16.4 C biztonsági hibaminták és védekezés

A C program biztonsági hibája gyakran egy megsértett szerződés következménye: a hossz nem fér bele a tárolóba, a pointer mögötti objektum már nem él, vagy a fájlnév ellenőrzése és használata nem ugyanarra az objektumra vonatkozik. A javítás ezért nem pusztán egy újabb feltétel hozzáadása. Meg kell nevezni, **milyen invariánst állít helyre**, és milyen feltételek mellett.

A formázás és szövegfeldolgozás, a fájlrendszer-műveletek, valamint a tesztelés és fuzzing korábbi szerződéseire építünk. A nyelvi állítások alapja ISO C17; a platformfüggő védekezéseket külön jelöljük.

16.4.1 Bounds és egészaritmetika: ellenőrzés a művelet előtt

A memória-biztonság (*memory safety*) szükséges feltétele, hogy minden hozzáférés élő objektum megfelelő tartományán belül történjen. A bemenetvalidálás (*input validation*) ennél tágabb: a bemenetnek a szintaktikai, szemantikai és erőforrás-korlátokat is teljesítenie kell. Egy reprezentálható, de több terabájtos hossz például nem feltétlenül elfogadható kérés.

ISO C17. A `size_t` előjel nélküli egész típus. Túlcsordulása moduláris aritmetikát követ; ettől az eredmény még lehet hibás allokációs méret. Az előjeles egész túlcsordulása ezzel szemben nem definiált viselkedés.

Legyen C a kapacitás, o az eltolás, n pedig a feldolgozandó C bájtok száma. A kívánt feltétel matematikailag:

$$0 \le o \le C, \qquad 0 \le n \le C - o.$$

C-ben ezért az ellenőrzés:

```
/* C17-részlet; minden változó size_t típusú. */
if (offset > capacity || length > capacity - offset) {
    /* A tartomány elutasítása. */
}
```

A rövidzáras kiértékelés miatt a kivonás csak akkor történik meg, amikor nem csordul alul. Az `offset + length <= capacity` alak önmagában hibás lehet, mert már az összeadás körbefordulhat.

Hasonlóan, k darab, egyenként s C bájtos elemhez:

$$ks \le \mathrm{SIZE_MAX} \quad \Longleftrightarrow \quad k \le \left\lfloor \frac{\mathrm{SIZE_MAX}}{s} \right\rfloor \qquad (s > 0).$$

Bizonyításvázlat. Pozitív s esetén az egyenlőtlenség osztható s-sel; egész k mellett a felső korlát lefelé kerekítendő. A vizsgálatot a szorzás előtt kell elvégezni. Ez csak a reprezentálhatóságot igazolja, az allokáció sikerét és az alkalmazási kvótát nem.

16.4.1.1 Kidolgozott példa: korlátozott szövegmásolat

A következő teljes program minimum ISO C17-et igényel. A `text_copy` szerződése szerint nem nulla hossz esetén a `src` legalább `n` olvasható C bájtra mutat. Ezt egy nem null pointer ellenőrzése önmagában nem bizonyítja.

```
/* safe_text.c */
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static char *text_copy(const char *src, size_t n)
{
    if ((src == NULL && n != 0) ||
        n == SIZE_MAX || n > 4096) {
        return NULL;
    }

    char *p = malloc(n + 1);
    if (p == NULL) {
        return NULL;
    }

    if (n != 0) {
        memcpy(p, src, n);
    }
    p[n] = '\0';
    return p;
}

int main(void)
{
    const char input[] = "status=ready";
    char *message = text_copy(input, sizeof input - 1);

    if (message == NULL) {
        return EXIT_FAILURE;
    }

    int failed = printf("%s\n", message) < 0;
    free(message);
    message = NULL;

    if (fflush(stdout) == EOF) {
        failed = 1;
    }
    return failed ? EXIT_FAILURE : EXIT_SUCCESS;
}
```

Az ellenőrzés után $n + 1$ reprezentálható. Sikeres allokációkor a másolás a `[0,n)` tartományt írja, a terminátor az `n` indexre kerül; mindkettő a lefoglalt `[0,n+1)` tartományon belül van. A függvény

legfeljebb lineáris idejű, szekvenciális másolást és egyetlen allokációt végez.

Objektum vagy hozzáférés	Tulajdonos és élettartam	Érvénytelenítés, hibakimenet
`input` tömb	A `main` lokális objektuma	A blokkból kilépéskor megszűnik
`src`	Kölcsönzött hozzáférés a hívás idejére	A forrás megszüntetése vagy tiltott egyidejű módosítása szerződésszegés
Visszaadott tároló	Siker után a hívó felelőssége	`free` érvényteleníti; hiba esetén `NULL`, nincs átadott tároló
`message`	Tulajdonosi szerepű pointerváltozó	Nullázása csak ezt a változót módosítja

A tömör példa minden hibát `NULL` eredménnyel jelez; könyvtári API-ban érdemes különválasztani az érvénytelen argumentumot, a limitet és a memóriahiányt. A másolás nem validál szövegkódolást, és beágyazott nullkaraktert sem tilt: azt a `%s` megjelenítés már szövegvégként értelmezi.

16.4.2 Use-after-free és double-free: tulajdonlási hibák

ISO C17. A `free` megszünteti az allokált objektum élettartamát. Az arra támaszkodó kölcsönzött hozzáférések többé nem használhatók. A felszabadított objektum elérése (*use-after-free*) és ugyanazon allokáció ismételt felszabadítása (*double-free*) nem definiált viselkedés.

A `free(message); message = NULL;` hasznos helyi állapotváltás, de nem teljes megoldás: egy korábban elmentett alias ettől még dangling marad. A `free(NULL)` megengedett, ám ez nem teszi biztonságossá a többi példányt.

A helyes invariáns:

> Minden allokációhoz egyértelmű felszabadítási felelősség tartozik; a kölcsönzések befejeződnek a felszabadítás előtt, és minden végrehajtási út legfeljebb egyszer szabadít fel.

Hibaágaknál ezért közös cleanup út vagy egyértelmű tulajdonosátadás szükséges. Az „eddig nem omlott össze” nem élettartam-bizonyíték.

16.4.3 Format string és nem megbízható szöveg

ISO C17. A formátumsztring vezérli, milyen variadikus argumentumokat és típusokat vár a formázó függvény. A `printf(input)` ezért nem biztonságos, ha az `input` külső eredetű: az adatot vezérlőnyelvként értelmezi.

A helyes változat `printf("%s", input)`, feltéve, hogy az `input` élő, olvasható, nullterminált karaktertömbre mutat. Pointer–hossz szeletnél a `fwrite(input, 1, length, stream)` elkerüli a terminátor keresését, de a tartomány érvényességét továbbra is a hívó garantálja, és a részleges írást kezelni kell.

A formátumhiba megszüntetése nem jelent teljes kimenetbiztonságot. Újsorral meghamisítható egy naplóbejegyzés szerkezete, vezérlőkarakterek befolyásolhatják a terminált. Strukturált naplózáshoz kontextusfüggő escape-elés és hosszkorlát is szükséges. A sztring „biztonságossága” mindig a felhasználás módjához tartozik.

16.4.4 TOCTOU, útvonalak és symlinkek

A *time-of-check to time-of-use* — TOCTOU — hiba akkor keletkezik, amikor az ellenőrzés és a használat között megváltozhat az ellenőrzött állapot. Fájlrendszernél a `lstat(path)` után végzett `open(path, ...)` új névfeloldást indít: a két művelet más objektumot érhet el. Az `access` sem foglalja le a későbbi megnyitás eredményét.

POSIX/Linux alkalmazási minta. Egy meglévő szabályos fájl olvasására használható szűk szerződés:

1. A hívó birtokol egy megbízható könyvtárhoz tartozó, nyitott `dirfd` descriptort.
2. A bemenet egyetlen nem üres névkomponens; nem `"."`, nem `".."`, és nem tartalmaz `/` karaktert.
3. A program `openat(dirfd, name, O_RDONLY | O_CLOEXEC | O_NOFOLLOW | O_NONBLOCK)` hívással nyit.
4. A visszaadott descriptoron `fstat` segítségével ellenőrzi a típust és az alkalmazási metaadat-feltételeket.
5. Elutasításkor lezárja; siker esetén a hívó lesz a descriptor tulajdonosa.

Az `O_NONBLOCK` például FIFO esetén segít elkerülni a megnyitás miatti várakozást; nem általános időkorlát. A minta feltétele, hogy a könyvtár és környezete ne engedjen támadó által létrehozott, veszélyes eszközobjektumokat: a megnyitásnak már az utólagos típusellenőrzés előtt lehet hatása.

Az `O_NOFOLLOW` a végső komponens symlinkként követését tiltja. Többkomponensű útvonal köztes symlinkjeire nem ad általános védelmet. Linuxon az `openat2` megfelelő `RESOLVE_*` korlátozásai erősebb feloldási szerződést adhatnak, de kernel- és interfésztámogatásukat külön ellenőrizni kell.

A megnyitott descriptor stabil objektumhivatkozás, **nem változhatatlan tartalompillanatkép**. Más folyamat továbbra is módosíthatja a fájlt. A hard linkek, mount-határok és hozzáférési jogosultságok szintén külön szabályzatot igényelnek. A `realpath` és egy sztringprefix-vizsgálat nem helyettesíti ezt.

16.4.5 Inicializálatlan adatok, padding és titkok

ISO C17. A struktúratagok értéke és a teljes objektumreprezentáció nem azonos fogalom. A kitöltőbájtok (*padding*) nem alkalmazási mezők. A `fwrite(&record, sizeof record, 1, stream)` ezért nem megfelelő általános szerializálás: paddinget és kihagyott inicializálás esetén korábbi memóriaadatokat is kivihet.

A javítás mezőnként felépített kimenet, amelynek minden elküldött C bájtját explicit értékadás előzi meg. Például egy 8 bites oktetteket használó protokollban egy címke és egy 32 bites számláló öt oktettként kódolható, rögzített bájtsorrenddel. Ehhez a profilban `CHAR_BIT == 8` és a választott egész típus megfelelő tartománya szükséges. A struktúra kezdeti nullázása önmagában nem hordozható padding-szerződés későbbi tagírások után.

A titkok törlésének más a problémája. Egy használat utáni `memset(secret, 0, size)` eltávolítható, ha az optimalizáló szerint az írásnak nincs megfigyelhető hatása. A `free` pedig nem ígér törlést.

Platform- és verziófüggő megoldás: glibc környezetben az `explicit_bzero` erre szolgáló interfész; a deklaráció elérhetőségéhez megfelelő feature-test profil kell. C23-ban a `memset_explicit` külön lehetőség, de nem része a C17 alapnak. A `memset_s` opcionális Annex K interfész, nem feltételezhető általánosan.

A törlés előfeltétele élő, írható tároló és helyes méret; a sorrend: használat befejezése, explicit törlés, felszabadítás. Ez sem garantálja a regiszterekben, korábbi másolatokban, naplókban vagy core dumpban maradt titkok eltűnését.

16.4.6 Fordítási védelem: következménycsökkentés, nem javítás

A következő választott laborprofil: Linux x86-64 System V, LP64, a `lab-next` profil (profiltábla), GNU binutils 2.43. A parancs nem itt végzett fordítás eredménye; helyi támogatását és a létrejött bináris tulajdonságait ellenőrizni kell.

```
gcc -std=c17 -O2 -g \
  -Wall -Wextra -Wpedantic -Wformat=2 -Werror=format-security \
  -fstack-protector-strong -D_FORTIFY_SOURCE=3 \
  -fPIE -pie -Wl,-z,relro,-z,now \
  safe_text.c -o safe_text
```

Védelem	Szerepe	Korlátja
PIE	Helyfüggetlen végrehajtható állomány; ASLR mellett változhat a betöltési cím	Nem végez bounds- vagy élettartam-ellenőrzést
RELRO és azonnali kötés	A relocációk után kijelölt ELF-területek írásvédelme	Nem védi az összes módosítható adatot
Stack protector	Bizonyos stack frame-ek sérülésének észlelése ellenőrzőértékkel	Nem minden írást vagy objektumot ellenőriz
Fortify	Kiválasztott könyvtári hívások erősített ellenőrzése ismert vagy meghatározható objektummérettel	Nem általános pointervalidátor

Ezek rétegzett védelmek. Egyik sem állítja helyre a hibás ownership-szerződést vagy a TOCTOU miatt megsértett bizalmi határt.

16.4.7 Feladatok

1. **Magyarázó:** Miért hibás az `offset + length <= capacity` ellenőrzés? Vizsgáld a laborprofilban az `offset = SIZE_MAX - 3`, `length = 8`, `capacity = 16` esetet.
2. **Kódolási:** Írj `checked_bytes` függvényt, amely elemszámból és elemméretből ellenőrzött szorzatot készít. Hiba esetén a kimeneti érték maradjon változatlan.
3. **Diagnosztikai:** Egy szolgáltatás `lstat` után `open` hívással olvas felhasználói fájlnevet, majd nyers struktúrát küld válaszként. Sanitizeres tesztje hibamentes. Nevezz meg két egymástól független hibát, javításukat és ellenőrzési feltételüket.

16.4.8 Megoldások

16.4.8.1 Tartományellenőrzés

A megadott profilban az előjel nélküli összeadás eredménye `4`, ezért a hibás feltétel igaz. A helyes ellenőrzés először az `offset > capacity` feltétellel elutasít. Ellenőrzési pontok: nulla hossz, pontos végpont, egy bájttal túllépett végpont és `SIZE_MAX` közeli értékek.

16.4.8.2 Ellenőrzött szorzat

Az alábbi C17-részlethez `<stdbool.h>`, `<stddef.h>` és `<stdint.h>` szükséges:

```
bool checked_bytes(size_t count, size_t element_size, size_t *out)
{
    if (out == NULL ||
        (element_size != 0 && count > SIZE_MAX / element_size)) {
        return false;
    }

    *out = count * element_size;
    return true;
}
```

Az `out` nem null esetben élő, írható `size_t` objektumra mutat a hívás végéig; a függvény nem tárolja el. Hiba esetén nincs írás. Ellenőrizendő a nulla szorzat, a `SIZE_MAX * 1` elfogadása és a `SIZE_MAX * 2` elutasítása. A nulla eredmény későbbi allokációs jelentését külön kell meghatározni.

16.4.8.3 Fájlnévverseny és adatszivárgás

Az első hiba a külön névfeloldások közötti TOCTOU. Javítása a korlátozott descriptorrelatív megnyitás és a megnyitott objektum ellenőrzése. Tesztelendő a symlink, tiltott névkomponens és nem szabályos fájl elutasítása; a teljes feltételrendszert kódreview-val is vizsgálni kell.

A második hiba a nyers objektumreprezentáció kiküldése. Javítása explicit szerializálás. Ellenőrzési feltétel, hogy minden kiküldött bájt meghatározott mezőből származzon, és a kimenet ne függjön paddingtől. A sanitizer hibamentes futása egyik szerződést sem bizonyítja.

16.4.9 Összefoglaló

A biztonságos javítás megnevezi az ellenőrzött tartományt, az aritmetikai korlátot, a tulajdonost és a bizalmi határt. A külső szöveg adat marad, a fájlművelet megnyitott objektumhoz kötődik, a kimenet nem tartalmaz véletlen memóriareprezentációt. A fordítási védelmek és tesztek ezt támogatják, de a helyes szerződést nem helyettesítik.

16.5 C minőségi kapuk és hibajavítási jegyzőkönyv

A hibajavítás akkor válik felülvizsgálható mérnöki eredménnyé, ha más is meg tudja állapítani, mi váltotta ki a hibát, mely szerződés sérült, hogyan szünteti meg ezt a változtatás, és mit ellenőriztek ténylegesen. A „nálam már nem omlik össze” megfigyelés ehhez kevés: eltűnhetett a kiváltó körülmény anélkül, hogy a hiba megszűnt volna.

A C biztonsági hibaminták és védekezés a megsérthető szerződéseket, az ASan, UBSan, MSan, TSan és Valgrind az eszközök megfigyelési határait, az Unit test, property test és fuzzing pedig a tesztbemenetek előállítását tárgyalta. Most ezekből állítunk össze reprodukálható ellenőrzési ciklust.

16.5.1 Minőségi kapu és bizonyítékhatár

A **minőségi kapu** (*quality gate*) előre meghatározott elfogadási feltétel, amely egy azonosítható forrásállapotra, rögzített környezetben végzett ellenőrzések eredményét értékeli. Nem pusztán egy CI-job neve: tartalmazza a szükséges konfigurációt, a siker feltételeit és a megőrzendő bizonyítékokat.

Például:

> A változás akkor fogadható el, ha a kétfordítós build új diagnosztika nélkül elkészül, minden kötelező teszt sikeres, és nincs kivizsgálatlan sanitizer- vagy statikuselemző-jelzés.

A „sikeres” jelentését is rögzíteni kell. A kihagyott vagy az infrastruktúra hibája miatt el sem indult teszt nem sikeres teszt. A támogatás hiánya külön állapot, amelyhez indoklás és jóváhagyott kivétel tartozik.

Az ellenőrzési ciklus:

1. a trigger és a megfigyelt eltérés rögzítése;
2. minimális reprodukció és regressziós teszt készítése;
3. a kiváltó ok azonosítása;
4. célzott javítás;
5. az érintett minőségi kapuk ismételt ellenőrzése;
6. a fennmaradó bizonytalanság dokumentálása.

A regressziós tesztnek lehetőség szerint a hibás változatot el kell utasítania, a javítottat pedig el kell fogadnia. Enélkül nem világos, hogy valóban a javítandó tulajdonságot ellenőrzi-e.

16.5.2 Kétfordítós build és diagnosztikai szabályzat

ISO C17: a példakód szabványos C17-et használ. **Eszközprofil:** az alábbi laborhoz kijelölt környezet Linux x86-64, System V ABI, LP64, a `lab-next` profil (profiltábla), LLVM/Clang és clang-tidy 18.1.8. A sanitizer-runtime a megfelelő fordítói telepítéshez tartozzon. Ez kijelölt, nem itt kipróbált profil; a parancsok nem futtatási eredmények.

A `gcc-14` és `clang-18` programnév önmagában nem rögzíti a javítóverziót. A környezetet csomagverzióval vagy konténerképdigesttel is azonosítani kell.

A GCC és a Clang eltérő diagnosztikát és optimalizálási döntéseket adhat. A két build ezért hasznos keresztellenőrzés, de nem két független helyességbizonyítás.

Profil	Fordítók	Cél
Debug	GCC és Clang	Könnyen vizsgálható állapot, regressziós tesztek
Release	GCC és Clang	Optimalizált és `NDEBUG`-os viselkedés ellenőrzése
ASan + UBSan	GCC és Clang	Instrumentált memória- és kiválasztott UB-ellenőrzések
TSan	Külön Clang-build	Konkurens tesztek vizsgálata

Fordítóeszköz-szabály: a TSan-profilt külön kell tartani az ASan-profiltól; nem minden sanitizer kombinálható. Minden profilt saját tárgyfájlokból kell felépíteni, és a szükséges instrumentációs kapcsolókat a végső linkelésnél is meg kell adni.

A warning policy legyen szigorú, de kezelhető. Saját kódban célszerű a kijelölt warningokat hibaként kezelni. Külső függőségekre külön szabályzat vonatkozhat. Elhallgattatás csak szűk hatókörben, indoklással és felülvizsgálható kivétellel történjen. Fordítófrissítéskor az új diagnosztikákat külön migrációban érdemes rendezni, nem a `-Werror` tartós kikapcsolásával.

16.5.3 Kidolgozott példa: részleges inicializálás

Egy objektum két puffert birtokol. A hiba: a második allokáció sikertelenségekor az első puffer felszabadítása elmarad. A normál sikeres teszt ezt nem gyakorolja.

A **hiba-injektálás** (*failure injection*) itt az adott sorszámú allokáció determinisztikus meghiúsítása. A regressziós teszt a sikeres eset mellett mindkét hibapontot ellenőrzi.

16.5.3.1 Szerződés és tulajdonlás

A `pair_init` előfeltétele, hogy a kimeneti objektum üres legyen. Siker esetén két puffer tulajdonosa lesz; hiba esetén üres marad. A függvény `0` értékkel jelez sikert, `-1` értékkel allokációs hibát.

Objektum vagy hozzáférés	Tulajdonos és élettartam	Érvénytelenítés
Átmeneti pufferek	Inicializálás közben a helyi `tmp` kezeli őket	Hibás inicializáláskor felszabadítás
Sikeresen átadott pufferek	A hívó `p` objektuma birtokolja őket	`pair_destroy`
`out`, `a`, `p` paraméterpointer	Kölcsönzött hozzáférés a hívó élő objektumaihoz; nincs eltárolás	A hívó objektumának élettartamvége
Számlálóállapot	A teszt aktuális iterációjának helyi objektuma	Az iteráció vége

A tulajdonos objektum élettartamának vége C-ben nem szabadítja fel automatikusan a dinamikus tárolót: a `pair_destroy` hívás a szerződés része.

Az alábbi teljes `gate.c` programban a méretek C bájtban értendők. Nincs konkrét objektumlayoutra vonatkozó feltételezés.

```c
#include <stddef.h>
#include <stdlib.h>

struct alloc_state {
    size_t calls;
    size_t fail_at; /* 0: nincs injektált hiba */
    size_t live;
};

struct pair {
    void *first;
    void *second;
};

static void *tracked_alloc(struct alloc_state *a, size_t bytes)
{
    ++a->calls;
    if (a->fail_at != 0 && a->calls == a->fail_at)
        return NULL;

    void *p = malloc(bytes);
    if (p != NULL)
        ++a->live;
    return p;
}

static void tracked_free(struct alloc_state *a, void *p)
{
    if (p != NULL) {
        free(p);
        --a->live;
    }
}

static int pair_init(struct pair *out, struct alloc_state *a)
{
    struct pair tmp = {0};

    tmp.first = tracked_alloc(a, 8);
    if (tmp.first == NULL)
        return -1;

    tmp.second = tracked_alloc(a, 16);
    if (tmp.second == NULL) {
        tracked_free(a, tmp.first);
        return -1;
    }

    *out = tmp; /* Logikai tulajdonátadás. */
    return 0;
}
```

```
static void pair_destroy(struct pair *p, struct alloc_state *a)
{
    tracked_free(a, p->second);
    tracked_free(a, p->first);
    *p = (struct pair){0};
}

int main(void)
{
    for (size_t fail = 0; fail <= 2; ++fail) {
        struct alloc_state a = {0, fail, 0};
        struct pair p = {0};
        int rc = pair_init(&p, &a);

        if (rc != (fail == 0 ? 0 : -1))
            return EXIT_FAILURE;
        if (a.calls != (fail == 1 ? 1u : 2u))
            return EXIT_FAILURE;

        if (fail == 0) {
            if (p.first == NULL || p.second == NULL || a.live != 2)
                return EXIT_FAILURE;
        } else {
            if (p.first != NULL || p.second != NULL || a.live != 0)
                return EXIT_FAILURE;
        }

        pair_destroy(&p, &a);
        if (p.first != NULL || p.second != NULL || a.live != 0)
            return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}
```

A teszt feltételezi, hogy a nem injektált apró allokációk sikerülnek. Valós memóriahiány esetén nem nulla kilépési állapot keletkezhet; annak eredetét külön kell kivizsgálni. Az ellenőrzések szándékosan nem `assert` hívások: release profilban, `NDEBUG` mellett is megmaradnak.

16.5.3.2 Buildparancsok és statikus elemzés

A következő Bash-parancsok fordítanak, de nem indítják el a tesztprogramokat:

```
common=(-std=c17 -Wall -Wextra -Wpedantic -Wconversion
        -Wshadow -Wstrict-prototypes -Wmissing-prototypes
        -Wformat=2 -Werror)

gcc-14 "${common[@]}" -O0 -g3 gate.c -o gate-gcc-debug
clang-18 "${common[@]}" -O0 -g3 gate.c -o gate-clang-debug

gcc-14 "${common[@]}" -O2 -DNDEBUG gate.c -o gate-gcc-release
clang-18 "${common[@]}" -O2 -DNDEBUG gate.c -o gate-clang-release
```

```
gcc-14 "${common[@]}" -O1 -g -fno-omit-frame-pointer \
  -fsanitize=address,undefined -fno-sanitize-recover=all \
  gate.c -o gate-gcc-asan-ubsan

clang-18 "${common[@]}" -O1 -g -fno-omit-frame-pointer \
  -fsanitize=address,undefined -fno-sanitize-recover=all \
  gate.c -o gate-clang-asan-ubsan

clang-18 "${common[@]}" -O1 -g -fno-omit-frame-pointer \
  -fsanitize=thread gate.c -o gate-clang-tsan

gcc-14 "${common[@]}" -fanalyzer -c gate.c -o gate-analyzer.o

clang-tidy-18 \
  --checks='-*,clang-analyzer-*,bugprone-sizeof-expression' \
  gate.c -- -std=c17
```

Eszközfüggő feltételek: a GCC statikus elemzőjének elérhetősége a telepítéstől is függhet; a clang-tidy külön csomag lehet. Az ellenőrzéskészletet az adott verzióhoz kell rögzíteni. C projekthez ne kapcsoljunk be válogatás nélkül C++-specifikus modernizáló szabályokat.

Többfájlos projektben az elemző ugyanazokat az include-útvonalakat, makrókat és targetbeállításokat kapja, mint a build; ezt célszerű fordítási adatbázissal biztosítani. A clang-tidy diagnosztikája önmagában nem feltétlenül jelent sikertelen CI-jobot: az elfogadási szabályt és a kilépési állapot kezelését külön konfigurálni kell.

A példa egyszálú. TSan-buildje legfeljebb az instrumentált környezet próbája, nem konkurenciateszt. A valódi TSan-kapuhoz konkurens teszterhelés szükséges.

16.5.4 Erőforrásmérleg és hibajavítási jegyzőkönyv

Tulajdonátadás nélküli vizsgálati tartományban legyen B a még lezárandó erőforrások száma, A a sikeres szerzéseké, R az érvényes elengedéseké. Ekkor:

$$B_{\text{vég}} = B_{\text{kezdet}} + A - R.$$

A második allokáció injektált hibájánál kezdetben $B = 0$, majd egy sikeres szerzés történik: $A = 1$. A javított visszagörgetés egyszer elenged: $R = 1$, így $B_{\text{vég}} = 0$. A hibás ágban $R = 0$, ezért egy kötelezettség megmarad.

Bizonyításvázlat: minden sikeres szerzés eggyel növeli, minden hozzá tartozó érvényes elengedés eggyel csökkenti a mérleget; a műveletsor szerinti indukció adja az összefüggést. Feltétel, hogy minden műveletet számoljunk, és egy szerzéshez legfeljebb egy elengedés tartozzon.

A nulla számláló önmagában nem elég: hibás könyvelés, számlálatlan allokáció vagy egymást számszerűen elfedő hibák mellett félrevezető lehet. File descriptorokra és más erőforrásokra külön, azonosságot is követő nyilvántartás szükséges.

Egy review-zható mintajegyzőkönyv:

Mező	Tartalom
Trigger	A második allokáció sikertelen

Mező	Tartalom
Megsértett szerződés	Sikertelen inicializálás után nem maradhat birtokolt puffer
Root cause	A részleges sikerhez nem tartozott visszagörgetés
Változás	Az első puffer felszabadítása a második allokáció hibájánál
Regresszió	`fail_at = 2`; elvárt üres kimenet és nulla élő allokáció
Ellenőrzés	Mindkét fordító, debug/release, statikus elemzés, ASan/UBSan
Nyitott korlát	A teszt nem általános bizonyíték minden hívóra vagy erőforrásra

Az itt felsorolt eredmények **elvárások**, nem közölt mérések. A kitöltött jegyzőkönyvnek a tényleges kilépési állapotot és naplókat kell tartalmaznia.

16.5.4.1 Eredetkövetés: forrás, fordító és mérés

A bizonyíték **eredetkövetése** (*provenance*) legalább a következőket kapcsolja össze:

- forráscommit, munkafa-eltérések, reprodukáló bemenet és injektálási pont;
- pontos fordító-, elemző-, libc- és sanitizer-runtime-verzió;
- target, kernel, buildparancsok és releváns környezeti változók;
- binárisazonosító, tesztnapló, kilépési állapot és futási időpont;
- az értelmezéshez használt dokumentáció verziója és konkrét szakasza.

A szabványos nyelvi állítást, az eszköz dokumentált képességét és a saját mérési megfigyelést külön kell kezelni. Csak ténylegesen elolvasott forrásra hivatkozzunk; keresési kiindulópont nem bizonyíték. Ellenőrzött forrásjegyzet hiányában a dokumentációs ellenőrzés nyitott tétel marad.

Idő- vagy memóriaköltség mérésénél a terhelést és az ismétléseket is rögzíteni kell. Sanitizeres időeredmény nem azonos a release program teljesítményével.

16.5.5 Gyakori hibák

A **zöld CI összetévesztése a helyességgel** a legsúlyosabb értelmezési hiba. A zöld állapot csak a kijelölt ellenőrzések eredménye; a nem vizsgált bemenetek és ütemezések nyitva maradnak.

Szintén gyakori, hogy a **teszt csak a visszatérési kódot nézi**, miközben a javítandó hiba erőforrásszivárgás. A regresszióban ezért a hibakód mellett a kimeneti állapot és az erőforrásmérleg is ellenőrzendő.

Végül a **globális diagnosztika-elhallgattatás** látszólag helyreállítja a kaput, de megszünteti annak információértékét. Kivételhez konkrét jelzés, indoklás, felelős és felülvizsgálati feltétel tartozzon.

16.5.6 Feladatok

1. **Magyarázó:** miért nem bizonyítja a hibamentes ASan/UBSan-futás, hogy minden erőforrás megfelelően lezárul?
2. **Kódolási:** egészítsd ki a tesztet annak ellenőrzésével, hogy a `pair_destroy` kétszer is meghívható ugyanarra az objektumra.
3. **Diagnosztikai:** egy változás debug profilban sikeres, release profilban hibás. A tesztelők azt mondják: „biztosan optimalizálóhiba”. Milyen bizonyítékokat kérnél, és mi legyen az elfogadás feltétele?

16.5.7 Megoldások

1. A futás csak a végrehajtott útvonalakat vizsgálja, az eszközök pedig meghatározott hibakategóriákat keresnek. Például egy lezáratlan file descriptor nem általános ASan/UBSan-célpont. Az elfogadható válasz megnevezi mind a lefedettségi, mind az eszközérzékenységi korlátot, és külön erőforrásellenőrzést javasol.

2. Az első destrukció utáni sikeres ellenőrzést követően illeszd be ezt a C17-részletet:

   ```
   pair_destroy(&p, &a);
   if (p.first != NULL || p.second != NULL || a.live != 0)
       return EXIT_FAILURE;
   ```

   Az első hívás nullázza a pointereket, ezért a második nem enged el újabb erőforrást. Az ellenőrzési feltétel: minden injektálási esetben nulla mérleg és üres objektum maradjon, runtime-diagnosztika nélkül.

3. Először azonos forrást, bemenetet és pontos buildparancsokat kell kérni. Külön vizsgálandó az optimalizálás és a `NDEBUG` hatása; különösen az, hogy került-e szükséges mellékhatás `assert` kifejezésbe. Ezután minimális reprodukció, a C-szerződések ellenőrzése és második fordítós összehasonlítás következik. Elfogadás csak megindokolt root cause és a release profilban is aktív regressziós teszt alapján történhet.

16.5.8 Összefoglaló

A jó minőségi kapu nem hibamentességet ígér, hanem reprodukálható ellenőrzést. A kétfordítós build, a C-hez illesztett statikus elemzés, az elkülönített runtime-profilok és a hiba-injektálás egymást kiegészítő bizonyítékot adnak. A hibajavítási jegyzőkönyv ezt összekapcsolja a kiváltó okkal, a változással és a regresszióval — és ugyanilyen világosan megnevezi, mi maradt bizonyítatlan.

17 C záróprojekt: stream-alapú diagnosztikai szolgáltatás

17.1 Projektkövetelmény, protokoll és ownership térkép

A záróprojektben egyetlen, pontosan körülhatárolt feladatot valósítunk meg: `név=érték` alakú, LF-fel lezárt rekordokból álló bájtfolyamot ellenőrzünk, majd rövid, gépileg feldolgozható összesítést készítünk róla. Ugyanazt a `dg_*` parserkönyvtárat használja a fájlból vagy standard bemenetről olvasó `diag` parancs és a TCP-kapcsolatokat kiszolgáló adapter. A 39. fejezet C++20 portja is ezt a szerződést viszi tovább, változtatás nélkül.

A cél nem egy éles monitoringrendszer, hanem egy ellenőrizhető rendszerprogramozási szerződés: mely bájtok érvényesek, mennyi erőforrást használhatunk, ki birtokolja az állapotot, és mi történik részleges bemenet vagy leállítás esetén.

A bounds-ellenőrzött parser állapotátmeneteire, az eseményvezérelt I/O modelljére és az opaque könyvtári interfész elválasztására építünk. A könyvtári magot és a CLI-t a C könyvtári mag és CLI megvalósítási labor, a hálózati adaptert a Nonblocking TCP adapter és leállítás valósítja meg. Ez az alfejezet a **kanonikus projektszerződés**: a későbbi fejezetek kódja és tesztjei ehhez igazodnak, és ha eltérnek tőle, az hiba.

17.1.1 A szolgáltatás és a parancssori felület szerződése

A **rekordfolyam** nulla vagy több, közvetlenül egymás után írt rekordból áll. Nincs köztük elválasztó, és nincs kötelező zárórekord. Az üres folyam érvényes. A bemenet végét a fájlvég jelzi.

A CLI használata:

```
diag [FILE|-]
```

Argumentum nélkül vagy `-` argumentummal a program a standard bemenetet olvassa, egyébként a megadott fájlt. Kettőnél több parancssori szó használati hiba.

A CLI nem keres új rekordhatárt sérült bemenet után. Az első hibánál megáll, mert az ismeretlen bájtok átugrása könnyen érvényesnek láttatna egy sérült folyamot. Részleges sikerjelentés nincs: hibás bemenetnél a standard kimenet üres marad.

Siker esetén a standard kimenetre pontosan egy sor kerül:

```
OK records=2 sum=19 bytes=11
```

Mező	Szerződés
`records`	A teljes egészében ellenőrzött, érvényes rekordok száma
`sum`	Az érvényes rekordok értékeinek összege
`bytes`	A parser által elfogyasztott bemeneti oktettek száma

A mezősorrend, az egyetlen ASCII szóköz elválasztó és a záró LF a szerződés része. A sor nem tartalmaz bemenetből átvett szöveget vagy útvonalat, ezért hossza korlátos, és nincs szükség escape-elésre.

Hiba esetén a diagnosztika a standard hibakimenetre kerül, így a gépi kimenetet nem szennyezi. Parserhibánál a sor a hibakódot és a hiba bájtoffsetjét tartalmazza:

```
ERR code=truncated offset=10
```

Más hibához nem tartozik értelmes bájtpozíció, ezért ott csak a kód szerepel, például `ERR code=open`.

Diagnosztikai kód	Forrás	Kilépési kód
`syntax`, `truncated`, `limit`	Parser: hibás, csonka vagy túl hosszú bemenet	4
`state`	Parser: API-használati hiba, például `finish` utáni `feed`	5
`usage`	Túl sok argumentum	2
`open`, `read`, `close`	A bemenet megnyitása, olvasása vagy saját fájl lezárása	3
`alloc`, `setup`	Parserallokáció vagy a `SIGPIPE` kezelésének beállítása	5
`write`	A jelentés kiírása vagy flush-a	6

Sikeres futás kilépési kódja `0`. A kilépési kódok a program rendes vezérlési útjaira vonatkoznak, nem a folyamat külső, kényszerített megszüntetésére. A hibakimenet írása *best effort*: hibás `stderr` esetén üzenetet nem tudunk garantálni.

17.1.2 A rekord pontos formátuma

17.1.2.1 Nyelvtan és rekordtípusok

Projektszerződés: a bemenet 8 bites oktettekből áll. A laborban `CHAR_BIT == 8`, ezért egy oktett egyben egy C bájt. Ezt a könyvtár fordításkor ellenőrzi. A karakterek alább mindig a megadott ASCII-kódú oktettet jelentik.

Egy rekord nyelvtana ABNF-hez hasonló jelöléssel:

```
rekord = név "=" érték LF
név    = 1*16( %x61-7A )      ; a-z
érték  = 1*5( %x30-39 )       ; 0-9, decimális értéke <= 65535
LF     = %x0A
```

Elem	Szerződés
Név	1–16 ASCII kisbetű, kizárólag `a`–`z`
Elválasztó	Egyetlen `=` (`0x3d`)
Érték	1–5 decimális számjegy, értéke legfeljebb 65535

Elem	Szerződés
Vezető nullák	Megengedettek, de beleszámítanak az öt számjegybe
Rekordlezárás	Kötelező LF; CRLF nem elfogadott
Ismétlődő név	Megengedett; minden rekord külön számít
Teljes bemenet	Legfeljebb 1 048 576 oktett

Minden más oktett a nyelvtan által nem engedett helyen szintaktikai hiba: nagybetű, szóköz, `\r`, `0x00`, üres név vagy érték, 17. névbetű, 6. számjegy, illetve olyan számjegy, amellyel az érték 65535 fölé nőne. A parser bájtokat vizsgál, nem C-sztringeket: a beágyazott nullbájt nem rövidíti meg a bemenetet, hanem hiba.

A parser nem tárolja el a neveket: a projekt a rekordfolyam szerkezeti érvényességét és összesített számait vizsgálja, nem metrikanevekből épít adatbázist.

17.1.2.2 Kidolgozott rekordpélda

A `a=7` és a `cpu=12` rekord együtt, hexadecimálisan:

```
61 3d 37 0a
63 70 75 3d 31 32 0a
```

Az első rekord 4, a második 7 oktett, összesen:

$$B = 4 + 7 = 11.$$

Az eredmény `OK records=2 sum=19 bytes=11`, mert $7 + 12 = 19$. A bemenet tetszőleges darabolásban érkezhet: egy olvasás visszaadhat két oktettet, egy későbbi a maradékot. A parser eredménye nem függhet ettől a darabolástól.

Ha a második rekord záró LF-je hiányzik, a fájlvégkor:

- a parser 10 oktettet fogyasztott el;
- az első rekord érvényes, de részleges sikerjelentés nincs;
- a kimenet `ERR code=truncated offset=10`, a kilépési kód 4;
- az offset a bemenet vége, vagyis az első hiányzó oktett helye.

17.1.2.3 A TCP-adapter kérés–válasz szerződése

A TCP-adapter ugyanerre a nyelvtanra épül, de kérés–válasz protokollként működik. **Egy kérés pontosan egy rekord**, az első LF-ig bezárólag. A válasz az az egyetlen sor, amelyet a CLI erre az egy rekordból álló bemenetre a standard kimenetre (siker) vagy a standard hibakimenetre (parserhiba) írna. Az offset ezért a kérés első oktettjéhez viszonyított.

Kérés	Válasz
`a=7\n`	`OK records=1 sum=7 bytes=4\n`
`cpu=12\n`	`OK records=1 sum=12 bytes=7\n`
`A=1\n`	`ERR code=syntax offset=0\n`
Parserallokációs hiba	`ERR code=alloc\n`

Mivel az LF minden kérést egyértelműen lezár, egy szintaktikailag hibás kérés után a kapcsolat a következő kéréssel folytatódhat. Keretszintű hiba viszont kapcsolatbontást okoz válasz nélkül: ilyen a kapcsolat bemeneti tárolóját (4096 oktett) LF nélkül megtöltő adat, illetve a félbemaradt kérés a kliens küldési irányának lezárásakor.

17.1.3 Erőforráskorlátok és ellenőrzési sorrend

A limitek a protokoll megvalósításának részei, nem pusztán üzemeltetési ajánlások. Mindegyiket egyetlen helyen, névvel ellátott konstans rögzíti; a számításokat ezekből vezetjük le.

Erőforrás	Projektlimit	Konstans, forrás
Egy munkamenet teljes bemenete	1 048 576 oktett	`DG_MAX_INPUT`, `include/diag.h`
Egy rekord, LF-fel	legfeljebb 23 oktett	a nyelvtanból: $16 + 1 + 5 + 1$
Parser saját tárolója	egyetlen, rögzített méretű objektum	`struct dg_parser`
CLI olvasópuffere	4096 C bájt	`src/main.c`
Aktív TCP-kapcsolatok	64	a szerver kapcsolattáblája
Kapcsolatonkénti bemeneti tároló	4096 C bájt	`DIAG_IN_CAP`, `include/diag_tcp.h`
Kapcsolatonkénti kimeneti tároló	1024 C bájt	`DIAG_OUT_CAP`, `include/diag_tcp.h`

A parser inkrementálisan dolgozik, ezért egyik puffernek sem kell egy teljes munkamenetet vagy rekordot befogadnia. A TCP bemeneti tároló a leghosszabb érvényes kérésnél (23 oktett) jóval nagyobb, így pipelined kérések is elférnek benne. A válasz legfeljebb 32 oktett (`OK records=1 sum=65535 bytes=23\n`), tehát a kimeneti tároló is elegendő.

A TCP-kiszolgáló (`diag_server [PORT]`) csak a `127.0.0.1` címen figyel; `0` vagy hiányzó port esetén a kernel választ portot. Indulás után a standard kimenetre egyetlen `LISTEN 127.0.0.1:PORT` sort ír. Kilépési kódja rendes leállítás után `0`, hibás argumentumnál `2`, indítási vagy eseményciklus-hibánál `5`. TLS, hitelesítés és hálózati publikálás nem része ennek a projektnek.

17.1.3.1 Méretellenőrzés túlcsordulás nélkül

Legyen M a munkamenet méretlimitje, B az eddig elfogyasztott oktettek száma. A parser minden új oktett **előtt** ellenőrzi:

$$B < M.$$

Ha $B = M$, az új oktettet nem fogyasztja el, és `limit` hibát ad B offsettel. Nem vezetünk be $B + n$ alakú összeget, amely túlcsordulhatna: a számláló oktettenként, egyesével nő, és az invariáns szerint sosem lépi át M-et.

Az érték túlcsordulását ugyanígy a művelet előtt zárjuk ki. Ha v az eddigi érték és d az új számjegy, akkor a $v' = 10v + d$ érték akkor fér el, ha

$$v \le \left\lfloor \frac{65535 - d}{10} \right\rfloor.$$

A részletes levezetést és az összeg felső korlátját a megvalósítási labor adja.

Az ellenőrzési sorrend determinisztikus:

1. Új oktett előtt a teljes bemeneti limit.
2. Az oktett a rekord aktuális fázisában: névbetű, `=`, számjegy vagy LF.
3. Számjegynél a számjegyszám és az értékhatár.
4. Fájlvégnél (`finish`) a félbemaradt rekord.

A határ elérése önmagában nem hiba. Pontosan M oktettnyi érvényes bemenet után a fájlvég sikeres; egy további oktett már `limit` hibát jelent, elfogyasztás nélkül.

A hibát kiváltó oktettet a parser nem fogyasztja el, ezért szintaktikai hibánál az offset az első elutasított oktett helye, csonkolásnál a bemenet vége.

17.1.3.2 Idő- és memóriakeret

A TCP-adapter monoton órán mért határidői:

Határidő	Érték	Kezdőesemény
Tétlen, üres kapcsolat	30 s	Az utolsó teljesen feldolgozott kérés vagy az elfogadás
Befejezetlen kérés	5 s	A parseradapter első `DIAG_MORE` eredménye
Függő válasz	5 s	A válasz sorba állítása
Teljes szerverleállítás	3 s	A leállítás kezdete

Egy-egy új oktett vagy részleges küldés nem indítja újra a határidőt. Ezek alkalmazásszintű határidők, nem valós idejű ütemezési garanciák: a lejáratot az eseményciklus észleli.

A kapcsolatok puszta adattömbjeinek felső korlátja:

$$64\,(4096 + 1024) = 327\,680 \text{ C bájt} = 320 \text{ KiB}.$$

Ez nem RSS-garancia. Nem tartalmazza a struktúratagokat, az allocator metaadatait, a libc belső állapotát, a végrehajtási vermet vagy a kernel socketpuffereit. Azokat külön kell mérni. Az aktívkapcsolat-limit a már elfogadott kapcsolatokra vonatkozik, nem a kernel várakozó kapcsolati sorára.

17.1.4 Modulhatárok és hibaszerződések

17.1.4.1 A könyvtári mag nem végez I/O-t

A projekt fájlszintű felosztása:

Modul	Feladat	Hibakimenet
`include/diag.h`, `src/diag.c`	Opaque parserállapot: `dg_create`, `dg_feed`, `dg_finish`, `dg_get_report`, `dg_get_error`, `dg_code_name`, `dg_destroy`	Ragadós `dg_error`; létrehozási hiba: `NULL`
`src/main.c`	CLI: argumentumok, bemeneti `FILE`, report formázása	Kilépési kód és diagnosztika
`include/diag_tcp.h`, `src/tcp_io.c`	Kapcsolatonkénti nonblocking olvasás, írás, backpressure	`enum diag_step`
`src/tcp_reply.c`	Core-adapter a TCP-hez: egy kérés feldolgozása és a válasz formázása	`enum diag_core_result`
`src/diag_server.c`	Szerver-vezérlő: listener, `epoll`, határidők, leállítás	Kapcsolat lezárása; kilépési kód

A kapcsolóelemek pontos jelentése:

- **Bemenet:** a `dg_feed` kölcsönzött `dg_view` bájtszeletet kap. A parser a hívás után nem őrzi meg a pointert, és teljes rekordot sem tárol: állapota a fázis, a névhossz, a számjegyszám és a részérték.
- **Fogyasztott bájtszám:** a `dg_report.bytes` a sikeresen elfogyasztott oktettek száma; hiba esetén ez egyben a hiba offsetje. A TCP-adapter a kérés hosszát adja vissza `consumed` értékként.
- **EOF:** a fájlvéget kizárólag a `dg_finish` jelzi. A nulla hosszúságú `feed` nem fájlvég. Félbemaradt rekordnál a `finish` `truncated` hibát ad.
- **Abort:** külön abort-művelet nincs. A még nem lezárt parser `dg_destroy` hívással, `finish` nélkül megszüntethető; ilyenkor nem keletkezik report. A TCP-adapter minden kéréshez rövid életű parsert hoz létre, és a hívás végén mindig megszünteti.
- **Terminális állapot:** az első hiba ragadós; utána minden `feed` és `finish` sikertelen, az eredeti hibaadat változatlan. Sikeres `finish` után a további `finish` sikeres, a további `feed` viszont `state` hiba: ez API-használati hiba, nem a bemenet hibája.
- **Eredményformázás:** a `dg_report` és a `dg_error` értékként kimásolható, a `dg_code_name` statikus szöveget ad. A CLI és a TCP-adapter ugyanazt a sorformátumot írja.

17.1.4.2 A specifikációs példák mint futtatható teszt

Az alábbi teszt a fenti nyelvtan példáit rögzíti, és a megvalósítási labor könyvtárához linkelve fut. A táblázat a specifikációból származik, nem az implementáció kimenetéből; így a megvalósítást a szerződéshez mérjük, nem önmagához.

```
/* tests/spec_examples.c -- a 17.1 specifikációs példái a 17.2 magon */
#include "diag.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct example {
    const char *input;      /* a bemenet bájtjai */
```

```
    size_t size;            /* beágyazott 0x00 miatt külön hossz */
    dg_code code;           /* DG_OK vagy a várt hibakód */
    size_t offset;          /* hiba esetén */
    size_t records;         /* siker esetén */
    unsigned long long sum;
    size_t bytes;
};

#define EX(s) s, sizeof s - 1

static const struct example examples[] = {
    { EX(""),                     DG_OK,          0, 0,  0,  0 },
    { EX("a=7\ncpu=12\n"),        DG_OK,          0, 2, 19, 11 },
    { EX("a=65535\n"),            DG_OK,          0, 1, 65535, 8 },
    { EX("abcdefghijklmnop=1\n"), DG_OK,          0, 1,  1, 19 },
    { EX("a=7\ncpu=12"),          DG_TRUNCATED, 10, 0,  0,  0 },
    { EX("cpu="),                 DG_TRUNCATED,  4, 0,  0,  0 },
    { EX("a=65536\n"),            DG_SYNTAX,     6, 0,  0,  0 },
    { EX("a=000001\n"),           DG_SYNTAX,     7, 0,  0,  0 },
    { EX("abcdefghijklmnopq=1\n"),DG_SYNTAX,    16, 0,  0,  0 },
    { EX("a=1\r\n"),              DG_SYNTAX,     3, 0,  0,  0 },
    { EX("a=\n"),                 DG_SYNTAX,     2, 0,  0,  0 },
    { EX("=1\n"),                 DG_SYNTAX,     0, 0,  0,  0 },
    { EX("A=1\n"),                DG_SYNTAX,     0, 0,  0,  0 },
    { EX("a=1\0\n"),              DG_SYNTAX,     3, 0,  0,  0 },
};

int main(void)
{
    int failures = 0;

    for (size_t i = 0; i < sizeof examples / sizeof examples[0]; ++i) {
        const struct example *ex = &examples[i];
        dg_parser *parser = dg_create(NULL);
        if (parser == NULL)
            return EXIT_FAILURE;

        dg_view view = { (const unsigned char *)ex->input, ex->size };
        bool ok = dg_feed(parser, view) && dg_finish(parser);
        dg_error e = dg_get_error(parser);
        dg_report r = dg_get_report(parser);

        bool pass = ok ? ex->code == DG_OK && r.records == ex->records &&
                         r.sum == ex->sum && r.bytes == ex->bytes
                       : e.code == ex->code && e.offset == ex->offset;
        if (!pass) {
            (void)fprintf(stderr, "example %zu failed\n", i);
            ++failures;
        }
        dg_destroy(parser);
    }
    return failures == 0 ? EXIT_SUCCESS : EXIT_FAILURE;
```

```
}
```

A `"a=000001\n"` példa a hatodik számjegynél (offset 7) hibás, akkor is, ha az érték kicsi: a számjegyszám-korlát a nyelvtan része, nem az értékhatár következménye.

Fordítás és futtatás a 17.2 buildje után:

```
gcc -std=c17 -O2 -Wall -Wextra -Wpedantic -Wconversion -Wshadow -Werror \
    -Iinclude tests/spec_examples.c build/gcc/libdiag.a \
    -o build/gcc/spec_examples
build/gcc/spec_examples
```

Buildjegyzőkönyv (`lab-gcc` profil). A teszt, a 17.2 könyvtára és CLI-je Linux x86-64 környezetben, GCC 13.3.0, glibc 2.39 és GNU binutils 2.42 mellett hibaüzenet és figyelmeztetés nélkül fordult, és mind a 14 példa sikeres volt. Clanggal ebben a jegyzőkönyvben nem fordítottunk.

17.1.5 Tulajdonlási térkép és leállítás

17.1.5.1 Owner, tároló és kölcsönzés

Erőforrás vagy nézet	Tulajdonos	Élettartam és érvénytelenítés
CLI által megnyitott bemeneti `FILE`	CLI	`fopen` sikerétől a cleanup-ágban végzett egyszeri `fclose`-ig
`stdin`, `stdout`, `stderr`	Futási környezet; a CLI csak kölcsönzi	A CLI nem zárja le
Listener fd	Szerver-vezérlő	Sikeres létrehozástól leállításig
Elfogadott socket fd	Kapcsolatrekesz	Átadás után kizárólag a rekesz teardownja zárja le
CLI parserobjektuma	CLI	`dg_create` sikerétől `dg_destroy`-ig
TCP-kérés parserobjektuma	`diag_core_reply` hívás	A hívás végéig; visszatérés előtt megszűnik
Kapcsolat bemeneti és kimeneti tömbje	Kapcsolatrekesz	A rekesz élettartama; a bemenet tömörítéskor változik
Parsernek átadott bájtszelet	Nem tulajdonló nézet	Csak a `dg_feed` hívás alatt használható
Következő kiírandó válaszrész	Nem tulajdonló nézet	A kimeneti tömb újrafelhasználása érvényteleníti

A kapcsolat életciklusa szöveges ownership diagramként:

```
szerver-vezérlő (owner)
  -> kapcsolatrekesz (owner)
       -> socket fd
       -> bemeneti tömb -> diag_core_reply alatti borrowed szelet
            -> rövid életű parser (a hívás alatt)
       -> kimeneti tömb -> részleges írás alatti borrowed szelet
```

A parser élettartama a TCP-adapterben rövidebb a kapcsolaténál: a válasz a hívás végéig saját másolatként a kimeneti tömbbe kerül. A kimeneti tömb tartalma viszont egy részleges írás után változatlanul megmarad, amíg az utolsó bájt el nem ment.

Az elfogadás utáni inicializálási hiba külön ownership eset. Ha a kapcsolatrekesz még nem vette át a socketet, az elfogadó ág zárja le. Sikeres átadás után már nem zárhatja le másodszor.

17.1.5.2 EOF, hiba és shutdown

A CLI egy munkamenetet dolgoz fel: a fájlvégnél hívja a `dg_finish` műveletet, majd kiírja a jelentést.

A TCP-kapcsolaton a `recv` nulla visszatérési értéke jelzi a kliens küldési irányának lezárását. Ha ekkor nincs tárolt, félbemaradt kérés, a kapcsolat rendben lezárul; ha van, az keretszintű hiba, és a kapcsolat válasz nélkül bomlik. A `recv` ideiglenes „nincs adat” eredménye nem fájlvég.

A kapcsolat állapotai és az eseményérdeklődés a TCP-adapter állapottábláját követik. Reset vagy írási hiba esetén a válasz nem kézbesíthető; ilyenkor teardown következik. A helyi sikeres `send()` sem bizonyítja, hogy a távoli alkalmazás feldolgozta a választ.

Kezelt leállításkor:

1. A listener figyelése megszűnik, a listener bezárul; új kapcsolatot nem fogadunk.
2. Rögzítjük a közös, 3 másodperces leállítási határidőt.
3. Minden kapcsolaton eldobjuk a még fel nem dolgozott bemenetet; új kérést nem dolgozunk fel.
4. A már sorba állított válaszokat a határidőig kiírjuk, majd féllezárás következik.
5. A határidő után a maradó kapcsolatokat lezárjuk, és felszabadítjuk az állapotukat.

A signalkezelés mechanikáját nem itt vezetjük le. A követelmény az, hogy a felszabadítás és a reportkészítés rendes vezérlési környezetben történjen.

17.1.6 Elfogadási tesztek és tipikus hibák

Az elfogadás a minőségi kapuk elvére épül: minden követelményhez megfigyelhető ellenőrzés tartozik.

Eset	Elvárt eredmény
Üres bemenet	`OK records=0 sum=0 bytes=0`, kilépési kód 0
A bemutatott 11 oktettes folyam	`OK records=2 sum=19 bytes=11`
Ugyanez minden lehetséges kettévágással	Azonos végeredmény
Fájlvég félbemaradt rekordban	`truncated`, offset a bemenet vége, kilépési kód 4
Nyelvtanon kívüli oktett	`syntax`, az első elutasított oktett offsetje
`65535`, illetve `65536` érték	Elfogadás, illetve `syntax` az utolsó számjegynél
262 144 darab `a=0\n` rekord; majd plusz egy oktett	Siker 1 048 576 bájttal, illetve `limit` 1 048 576 offsettel
`diag FILE extra`	`ERR code=usage`, kilépési kód 2, üres stdout
Nem létező fájl	`ERR code=open`, kilépési kód 3
TCP: két érvényes kérés egy szegmensben, majd egy hibás	Három válaszsor, sorrendben
TCP: részleges válaszírás, ismételt várakozás	Nincs adatvesztés, duplikálás vagy pufferfelülírás

Eset	Elvárt eredmény
TCP: 64 aktív kapcsolat után új elfogadás	Az új kapcsolat nem kap rekeszt, nincs állapotbővítés
Létrehozási hiba és shutdown	Minden saját fd és tároló pontosan egyszer szabadul

A pontos méretlimit fixture-je a legrövidebb rekordból épül: $262\,144 \cdot 4 = 1\,048\,576$ oktett.

Gyakori hiba a `bytes` számláló növelése a teljes `read` eredményével akkor is, ha a parser hibát talált a darab közepén. A javítás: a jelentett bájtszám mindig a parser által elfogyasztott mennyiség, nem az olvasott.

Ugyancsak hibás a `finish` meghívása átmeneti olvasási szünetben, a kimeneti tömb újrafelhasználása részleges írás közben, illetve az fd másolatainak külön tulajdonosként kezelése. Az fd egész értékének másolása nem hoz létre új erőforrást.

17.1.7 Feladatok

17.1.7.1 Magyarázó feladat: hol van a hibapozíció?

A bemutatott kétrekordos folyamból a második rekord értékének utolsó számjegye és a záró LF is hiányzik: `a=7\ncpu=1`. Add meg a CLI kimenetét és kilépési kódját. Miért nem szabad a második rekordot megszámolni, és miért nincs részleges `OK` jelentés?

17.1.7.2 Kódolási feladat: specifikációs példák bővítése

Bővítsd a `tests/spec_examples.c` táblázatát a következő esetekkel: üres sor (`\n`), dupla `=`, 16 betűs név öt számjeggyel, `cpu` LF nélkül, valamint két rekord, amelyek közül a második hibás. Minden várt értéket a nyelvtanból vezess le, ne a program futtatásából.

17.1.7.3 Diagnosztikai feladat: eltűnő válasz és fd-szivárgás

Egy TCP-adapter a választ lokális tömbben formázza, eltárolja annak pointerét a kapcsolatban, majd visszatér. Másik hibája, hogy sikertelen rekeszinicializáláskor visszatér az elfogadási ágból, de nem zárja le a socketet. Készíts ownership-alapú javítási tervet.

17.1.8 Megoldások

17.1.8.1 A hiányos rekord eredménye

A bemenet `a=7\ncpu=1`, 9 oktett. A parser mind a 9 oktettet elfogyasztja, mert mindegyik a nyelvtan szerinti helyen áll. A fájlvégnél a második rekord a számfázisban van, ezért a `finish` csonkolást jelez:

```
ERR code=truncated offset=9
```

A kilépési kód 4, a standard kimenet üres. A második rekord csak a záró LF után számítható be, mert addig nem tudható, hogy érvényes-e. Részleges `OK` jelentés azért nincs, mert a szerződés a teljes munkamenetről szól: egy csonka folyam összegét a fogyasztó tévesen teljesnek tekinthetné.

Ellenőrzési feltétel: minden bemeneti darabolás ugyanezt az eredményt adja.

17.1.8.2 A bővített példák várt értékei

Bemenet	Várt eredmény
`"\n"`	`syntax`, offset 0: üres név
`"a==1\n"`	`syntax`, offset 2: a második `=` nem számjegy
`"abcdefghijklmnop=65535\n"`	siker: 1 rekord, összeg 65535, 23 bájt
`"cpu"`	`truncated`, offset 3
`"a=1\nb=x\n"`	`syntax`, offset 6

Az utolsó esetben az első rekord 4 oktettje elfogyott, a `b` és a `=` is; az `x` a 6. offseten áll. A tesztek a tényleges visszatérési értéket ellenőrizzék, ne kizárólag kikapcsolható `assert` állításokra épüljenek.

17.1.8.3 A tulajdonlás javítása

A válasz tárolója legyen a kapcsolatrekesz kimeneti tömbje; így a részleges írás után is él. A lokális tömbre mutató pointer a függvényből visszatéréskor dangling lesz.

A socketnek az elfogadás pillanatától legyen azonosítható tulajdonosa. Inicializálási hiba esetén az elfogadó ág zárja le; sikeres átadás után a rekesz teardownja felel érte.

Ellenőrzési feltétel: késleltetett írással és létrehozási hiba-injektálással se maradjon érvénytelen pointer vagy nyitva felejtett fd.

17.1.9 Összefoglaló

A projekt lényegi eredménye a közös, ellenőrizhető szerződés: egyértelmű rekordnyelvtan, rögzített CLI-kimenet és kilépési kódok, korlátozott erőforrás-használat, I/O-tól független parser és explicit ownership. A CLI és a TCP-adapter ugyanazt a `dg_*` magot használja, és ugyanazt a sorformátumot írja; csak az adatmozgatás és a lezárás mechanikája különbözik. A következő megvalósítási lépések és a 39. fejezet C++ portja ehhez a szerződéshez mérendő, nem pusztán néhány sikeres futáshoz.

17.2 C könyvtári mag és CLI megvalósítási labor

A könyvtári mag és a parancssori program közötti határ akkor hasznos, ha nem pusztán két forrásfájlt választ el, hanem két külön szerződést is. A mag bájtsorozatot értelmez, állapotot tart fenn és értékként visszaadható eredményt készít. A CLI fájlt nyit, olvasási hibát kezel, jelentést formáz, majd kilépési kóddal jelzi a művelet eredményét.

A Projektkövetelmény, protokoll és ownership térkép kanonikus projektszerződését itt valósítjuk meg: ugyanazt a rekordnyelvtant, CLI-felületet, kimeneti formátumot és kilépésikód-rendszert. Az önállóan elkészíthető laborhoz a szükséges részleteket megismételjük; eltérés esetén a 17.1

szerződése az irányadó. A TCP-adapter és a 39. fejezet C++ portja ugyanezt a könyvtári interfészt használja: a parser nem ismer sem `FILE` objektumot, sem file descriptort, sem hálózati kapcsolatot.

17.2.1 Implementációs szerződés és laborprofil

A bemenet nulla vagy több, LF-fel lezárt rekordból áll:

```
név=érték\n
```

A `\n` itt egyetlen, `0x0a` értékű oktettet jelöl, nem két nyomtatható karaktert.

Elem	Szerződés
Név	1–16 ASCII kisbetű, kizárólag `a`–`z`
Elválasztó	Egyetlen `=`
Érték	1–5 decimális számjegy, értéke legfeljebb 65535
Vezető nullák	Megengedettek, de beleszámítanak az öt számjegybe
Rekordlezárás	Kötelező LF; CRLF nem elfogadott
Üres bemenet	Érvényes, nulla rekorddal
Maximális bemenet	1 048 576 C bájt
Ismétlődő név	Megengedett; minden rekord külön számít
Eredmény	Rekordszám, értékösszeg, elfogadott bájtok száma

A parser nem tárolja el a neveket: ez a változat a rekordfolyam szerkezeti érvényességét és összesített számait vizsgálja, nem metrikanevekből épít adatbázist.

Nyelvi és platformbesorolás. A könyvtár ISO C17 nyelvi és könyvtári eszközöket használ, 8 bites C bájtot, megfelelő méretű `size_t` típust és ASCII-kompatibilis karakterkészletet feltételezve. A CLI Linux/POSIX kiegészítést használ a `SIGPIPE` kezelésére. Nem használunk exact-width egész típust, így például a `uint64_t` elérhetősége nem előfeltétel.

A labor profilja a központi profiltábla `lab-gcc` profilja: x86-64 Linux, System V ABI, LP64, GCC 13.3.0, glibc 2.39, GNU binutils 2.42, explicit `-std=c17`, GNU nyelvi bővítések nélkül. A buildszkriptek `clang` ága a `lab-clang` profilhoz tartozik. A labor buildjét és tesztjeit a `lab-gcc` profilban lefuttattuk; az eredményeket az alfejezet végi buildjegyzőkönyv rögzíti. A saját laborban a tényleges fordító-, libc- és eszközverziókat a központi tábla mintájára kell rögzíteni.

17.2.1.1 A CLI stabil külső felülete

A használat:

```
diag [FILE|-]
```

Argumentum nélkül vagy `-` argumentummal a program a standard bemenetet olvassa. Siker esetén a standard kimeneten pontosan egy sor jelenik meg:

```
OK records=2 sum=19 bytes=11
```

A mezősorrend, az ASCII szóközök és a záró LF a szerződés részei.

Kilépési kód	Jelentés	Például
0	Teljes siker	A jelentés kiírása és flush művelete is sikerült
2	Hibás CLI-használat	Túl sok argumentum
3	Bemeneti I/O-hiba	Megnyitás, olvasás vagy saját fájl lezárása
4	Hibás bemenet	Szintaxis, csonkolás vagy méretlimit
5	Erőforrás- vagy belső állapothiba	Allokáció vagy jelkezelési beállítás
6	Jelentésírási hiba	Sikertelen formázott kiírás vagy flush

Parserhiba esetén a standard hibakimenet formátuma:

```
ERR code=truncated offset=10
```

Más hibákhoz nem kapcsolunk értelmetlen bájtoffsetet, például:

```
ERR code=open
```

A hibakimenet írása *best effort*: hibás `stderr` esetén üzenetet nem tudunk garantálni. A kilépési kódok a program rendes vezérlési útjaira vonatkoznak, nem például a folyamat külső, kényszerített megszüntetésére.

17.2.2 Fájlszerkezet, tulajdonlás és tárolóválasztás

A projekt elrendezése:

```
include/
    diag.h
src/
    diag.c
    main.c
tests/
    test_diag.c
    spec_examples.c
    cli_args.sh
    golden.in
    golden.out
    truncated.in
    truncated.err
build/
```

Az opaque parser olyan típus, amelynek deklarációja publikus, objektumreprezentációja viszont csak a megvalósításban látható. A kliens ezért nem tudja közvetlenül elrontani például az aktuális számjegyszámot. A határ nem teszi automatikusan örökké ABI-kompatibilissé a teljes API-t, de elrejti a parser belső méretét és elrendezését.

Erőforrás vagy adat	Tulajdonos	Élettartam, érvénytelenítés
Parserobjektum	CLI vagy teszt	Létrehozástól `dg_destroy` hívásig
Allokátor-context	Az API hívója	A parser megsemmisítéséig élnie kell
Olvasópuffer	CLI	Automatikus tároló a `main` végéig
`dg_view` által jelölt adat	A hívó	A `dg_feed` hívás alatt olvasható; a parser nem őrzi meg
Másolt `dg_error`	A másolatot tároló hívó	Független a parser és a bemenet későbbi élettartamától
Saját megnyitott `FILE`	CLI	Pontosan egyszer lezárandó
`stdin`, `stdout`, `stderr`	Futási környezet	A CLI kölcsönzött hozzáféréssel használja

Miért nincs arena? Az arena közös élettartamú objektumcsoportok egyszerű felszabadítására alkalmas. Itt azonban nem őrzünk rekordokat vagy neveket: a parser egyetlen, állandó méretű objektum. Egy arena bevezetése csak további állapotot és hibalehetőséget teremtene.

A byte view nem tulajdonló, hosszával együtt átadott adatszelet. Nem nullterminált sztring, ezért beágyazott nullbájt esetén sem rövidül meg „magától”: a parser azt szintaktikai hibaként látja.

17.2.3 A könyvtári mag

17.2.3.1 Publikus header: `include/diag.h`

A következő fájl önállóan include-olható C17 header. Nem teszi szükségessé, hogy a kliens előtte más headert is beillesszen.

```
#ifndef DIAG_H
#define DIAG_H

#include <stdbool.h>
#include <stddef.h>

#define DG_MAX_INPUT ((size_t)1048576UL)

typedef struct dg_parser dg_parser;

typedef struct {
    const unsigned char *data;
    size_t size;
} dg_view;

typedef enum {
    DG_OK = 0,
    DG_SYNTAX,
    DG_TRUNCATED,
```

```
    DG_TOO_LARGE,
    DG_STATE
} dg_code;

typedef struct {
    dg_code code;
    size_t offset;
} dg_error;

typedef struct {
    size_t bytes;
    size_t records;
    unsigned long long sum;
} dg_report;

typedef struct {
    void *(*allocate)(size_t size, void *context);
    void (*release)(void *pointer, void *context);
    void *context;
} dg_allocator;

/* NULL allocator selects malloc/free. */
dg_parser *dg_create(const dg_allocator *allocator);
void dg_destroy(dg_parser *parser);

/* Input is borrowed only for the duration of this call. */
bool dg_feed(dg_parser *parser, dg_view input);

/* Declares end of input. Repeated successful finish is allowed. */
bool dg_finish(dg_parser *parser);

/* Returned values do not borrow from parser. */
dg_error dg_get_error(const dg_parser *parser);
dg_report dg_get_report(const dg_parser *parser);

/* Returns a static string; the caller must not release it. */
const char *dg_code_name(dg_code code);

#endif
```

A hívási előfeltételek fontosak:

- a `dg_destroy(NULL)` megengedett, más parserparaméternek élő parserre kell mutatnia;
- nem nulla hosszúságú view esetén a teljes megadott tartománynak olvashatónak kell lennie;
- nulla hosszúsághoz null pointer is tartozhat;
- az egyedi allokátor mindkét callbackje kötelező, és megfelelően igazított tárolót kell szolgáltatnia;
- ugyanazt a parserobjektumot egyszerre csak egy végrehajtási szál használhatja.

A konstruktor lemásolja az allokátorleírót, de nem másolja le a context által jelölt objektumot. A context élettartama ezért továbbra is a hívó felelőssége.

17.2.3.2 Megvalósítás: `src/diag.c`

A parser két fázist különböztet meg: névolvasást és számolvasást. A névolvasás nulla névhosszal egyben rekordhatárt is jelent.

```
#include "diag.h"

#include <limits.h>
#include <stdint.h>
#include <stdlib.h>

_Static_assert(CHAR_BIT == 8, "8-bit C bytes required");
_Static_assert(SIZE_MAX >= 1048577UL, "size_t too narrow");
_Static_assert('a' == 97 && 'z' == 122 &&
               '0' == 48 && '9' == 57 &&
               '=' == 61 && '\n' == 10,
               "ASCII-compatible execution character set required");

enum phase {
    PH_KEY = 0,
    PH_NUMBER
};

struct dg_parser {
    dg_allocator allocator;
    dg_report report;
    dg_error error;
    size_t key_length;
    unsigned digits;
    unsigned long value;
    enum phase phase;
    bool finished;
};

static void *default_allocate(size_t size, void *context)
{
    (void)context;
    return malloc(size);
}

static void default_release(void *pointer, void *context)
{
    (void)context;
    free(pointer);
}

static bool fail(dg_parser *parser, dg_code code)
{
    if (parser->error.code == DG_OK) {
        parser->error.code = code;
        parser->error.offset = parser->report.bytes;
    }
```

```
    return false;
}

dg_parser *dg_create(const dg_allocator *allocator)
{
    dg_allocator chosen = {
        default_allocate, default_release, NULL
    };

    if (allocator != NULL) {
        chosen = *allocator;
    }

    dg_parser *parser =
        chosen.allocate(sizeof *parser, chosen.context);

    if (parser == NULL) {
        return NULL;
    }

    *parser = (dg_parser){ .allocator = chosen };
    return parser;
}

void dg_destroy(dg_parser *parser)
{
    if (parser != NULL) {
        dg_allocator allocator = parser->allocator;
        allocator.release(parser, allocator.context);
    }
}

bool dg_feed(dg_parser *parser, dg_view input)
{
    if (parser->error.code != DG_OK) {
        return false;
    }
    if (parser->finished) {
        return fail(parser, DG_STATE);
    }

    for (size_t i = 0; i < input.size; ++i) {
        if (parser->report.bytes == DG_MAX_INPUT) {
            return fail(parser, DG_TOO_LARGE);
        }

        unsigned char byte = input.data[i];

        if (parser->phase == PH_KEY) {
            if (byte >= 'a' && byte <= 'z') {
                if (parser->key_length == 16) {
                    return fail(parser, DG_SYNTAX);
```

```
                }
                ++parser->key_length;
            } else if (byte == '=' && parser->key_length != 0) {
                parser->phase = PH_NUMBER;
                parser->digits = 0;
                parser->value = 0;
            } else {
                return fail(parser, DG_SYNTAX);
            }
        } else {
            if (byte >= '0' && byte <= '9') {
                unsigned long digit = (unsigned long)(byte - '0');

                if (parser->digits == 5 ||
                    parser->value > (65535UL - digit) / 10UL) {
                    return fail(parser, DG_SYNTAX);
                }

                parser->value = parser->value * 10UL + digit;
                ++parser->digits;
            } else if (byte == '\n' && parser->digits != 0) {
                ++parser->report.records;
                parser->report.sum += parser->value;
                parser->phase = PH_KEY;
                parser->key_length = 0;
            } else {
                return fail(parser, DG_SYNTAX);
            }
        }

        ++parser->report.bytes;
    }

    return true;
}

bool dg_finish(dg_parser *parser)
{
    if (parser->error.code != DG_OK) {
        return false;
    }

    if (parser->phase != PH_KEY || parser->key_length != 0) {
        return fail(parser, DG_TRUNCATED);
    }

    parser->finished = true;
    return true;
}

dg_error dg_get_error(const dg_parser *parser)
{
```

```
    return parser->error;
}

dg_report dg_get_report(const dg_parser *parser)
{
    return parser->report;
}

const char *dg_code_name(dg_code code)
{
    switch (code) {
    case DG_OK:        return "ok";
    case DG_SYNTAX:    return "syntax";
    case DG_TRUNCATED: return "truncated";
    case DG_TOO_LARGE: return "limit";
    case DG_STATE:     return "state";
    }
    return "state";
}
```

17.2.3.3 Offset, tartós hibaadat és számtani korlátok

Az offset nulla alapú. Szintaktikai hibánál az első elutasított bájt helyét, csonkolásnál a bemenet végét jelöli. A limit túllépésekor az első már nem engedélyezett bájt offsetje pontosan `DG_MAX_INPUT`.

A számláló csak sikeresen feldolgozott bájt után nő. Emiatt a hibát kiváltó bájt helye további korrekció nélkül rendelkezésre áll. A hiba *ragadós*: az első hiba után minden újabb feed és finish sikertelen, az eredeti hibaadat változatlan marad.

A `dg_error` nem tartalmaz bemenetbe mutató pointert vagy ideiglenes formázópuffert. A másolata a parser megsemmisítése után is használható. A hibakód szöveges neve statikus sztring; azt sem kell felszabadítani.

A számfeldolgozásnál legyen v az eddigi érték, d az új számjegy, $M = 65535$. A következő érték:

$$v' = 10v + d.$$

Mivel $0 \le d \le 9$, a megengedhetőség feltétele:

$$v \le \left\lfloor \frac{M - d}{10} \right\rfloor.$$

Ezt **a szorzás előtt** ellenőrizzük. Nem előbb számolunk egy esetleg túl nagy értéket, hogy utólag vizsgáljuk.

Bizonyításvázlat az összeg korlátjára. A legrövidebb rekord négy bájt, például `a=0\n`. Ha $L = 1048576$, akkor legfeljebb $L/4 = 262144$ teljes rekord fér el. Így:

$$\text{sum} \le 262144 \cdot 65535 = 17179607040.$$

Ez belefér az ISO C17 által az `unsigned long long` számára garantált minimális értéktartományba. A bizonyítás a rögzített bemeneti limitre támaszkodik: a limit későbbi emelésekor ezt az érvelést is felül kell vizsgálni.

A feldolgozás legrosszabb esetben $O(n)$ idő és $O(1)$ parsermemória. Bájtanként korlátos számú művelet történik, nincs amortizált növekedés vagy rekordfüggő allokáció. A hívás után az inputpuffer azonnal újrahasználható.

17.2.4 A hibakezelt CLI

A CLI blokkoló, kötegelt feldolgozó. A `fread` használata itt nem helyettesíti a későbbi nonblocking állapotgépet.

A ciklus legfeljebb a fennmaradó keret plusz egy bájtját kéri ki a `FILE` adatfolyamból. Az extra bájt szükséges ahhoz, hogy megkülönböztessük a pontosan limitméretű bemenetet a túl hosszútól. Ez alkalmazási szintű korlát: a standard I/O belső előreolvasása külön mechanizmus.

17.2.4.1 Teljes program: `src/main.c`

A fájl C17 és Linux/POSIX környezetet igényel. A buildparancsban szereplő feature-test makró teszi elérhetővé a használt POSIX deklarációkat.

```
#include "diag.h"

#include <signal.h>
#include <stdbool.h>
#include <stdio.h>
#include <string.h>

static void complain(const char *code)
{
    (void)fprintf(stderr, "ERR code=%s\n", code);
}

static int parser_failure(const dg_parser *parser)
{
    dg_error error = dg_get_error(parser);

    (void)fprintf(stderr, "ERR code=%s offset=%zu\n",
                  dg_code_name(error.code), error.offset);

    return error.code == DG_STATE ? 5 : 4;
}

int main(int argc, char **argv)
{
    int status = 0;
    FILE *input = stdin;
    bool own_input = false;
    dg_parser *parser = NULL;
    dg_report report = {0};
    size_t seen = 0;
```

```
unsigned char buffer[4096];

struct sigaction action = {0};
action.sa_handler = SIG_IGN;

if (sigemptyset(&action.sa_mask) == -1 ||
    sigaction(SIGPIPE, &action, NULL) == -1) {
    complain("setup");
    status = 5;
    goto cleanup;
}

if (argc > 2) {
    complain("usage");
    status = 2;
    goto cleanup;
}

if (argc == 2 && strcmp(argv[1], "-") != 0) {
    input = fopen(argv[1], "rb");
    if (input == NULL) {
        complain("open");
        status = 3;
        goto cleanup;
    }
    own_input = true;
}

parser = dg_create(NULL);
if (parser == NULL) {
    complain("alloc");
    status = 5;
    goto cleanup;
}

for (;;) {
    size_t remaining = DG_MAX_INPUT - seen + 1;
    size_t wanted =
        remaining < sizeof buffer ? remaining : sizeof buffer;

    size_t count = fread(buffer, 1, wanted, input);

    if (ferror(input)) {
        complain("read");
        status = 3;
        goto cleanup;
    }

    if (count != 0) {
        dg_view view = { buffer, count };

        if (!dg_feed(parser, view)) {
```

```
                status = parser_failure(parser);
                goto cleanup;
            }
            seen += count;
        }

        if (feof(input)) {
            break;
        }

        if (count == 0) {
            complain("read");
            status = 3;
            goto cleanup;
        }
    }

    if (!dg_finish(parser)) {
        status = parser_failure(parser);
        goto cleanup;
    }

    report = dg_get_report(parser);

cleanup:
    if (own_input) {
        if (fclose(input) == EOF && status == 0) {
            complain("close");
            status = 3;
        }
    }

    dg_destroy(parser);

    if (status == 0) {
        if (fprintf(stdout,
                    "OK records=%zu sum=%llu bytes=%zu\n",
                    report.records, report.sum, report.bytes) < 0) {
            complain("write");
            status = 6;
        }

        if (fflush(stdout) == EOF) {
            if (status == 0) {
                complain("write");
            }
            status = 6;
        }
    }

    return status;
}
```

A `seen` változó csak teljesen sikeres feed után változik. Ekkor legfeljebb a limitet érheti el, ezért a `DG_MAX_INPUT - seen` kivonás nem csordul alul.

Az olvasás után előbb az I/O-hibát ellenőrizzük. Ha ugyanaz a `fread` részadatot és hibajelzést is eredményez, az I/O-hiba elsőbbséget kap; a részadatból nem készül sikeres jelentés.

A cleanup ágban a `fclose` minden saját fájlon lefut, akkor is, ha már van korábbi hiba. A lezárási hiba csak akkor módosítja az eredményt, ha addig sikeres volt a feldolgozás. Így egy másodlagos takarítási hiba nem tünteti el az elsődleges parserhibát.

A jelentés a saját bemenet sikeres lezárása után jelenik meg. A `%zu` a `size_t`, a `%llu` az `unsigned long long` megfelelő formátuma.

Linux/POSIX viselkedés: megszűnt olvasóoldalú pipe esetén az alapértelmezett `SIGPIPE` megszakíthatná a folyamatot a saját hibakezelés előtt. Ezért a CLI figyelmen kívül hagyja ezt a jelet, és az írás hibáját vizsgálja. A sikeres `fprintf` még nem elég: pufferelt kimenetnél a hiba csak `fflush` alatt is megjelenhet.

A kimenet nem tranzakciós. Íráshiba után maradhat részleges jelentés a fogadónál; csak a nulla kilépési kód igazolja a teljes sikerességet.

17.2.5 Tesztek és fordítás

17.2.5.1 Könyvtári tesztek: `tests/test_diag.c`

A teszt C17-ben fordítható. A `CHECK` ellenőrzései `NDEBUG` mellett sem tűnnek el.

```
#include "diag.h"

#include <stdio.h>
#include <stdlib.h>

#define CHECK(condition) do {                                          \
    if (!(condition)) {                                                \
        (void)fprintf(stderr, "CHECK failed at line %d\n", __LINE__); \
        exit(EXIT_FAILURE);                                            \
    }                                                                  \
} while (0)

static void test_splits(void)
{
    static const unsigned char input[] = "a=7\ncpu=12\n";
    const size_t length = sizeof input - 1;

    for (size_t split = 0; split <= length; ++split) {
        dg_parser *parser = dg_create(NULL);
        CHECK(parser != NULL);

        CHECK(dg_feed(parser, (dg_view){ input, split }));
        CHECK(dg_feed(parser,
                      (dg_view){ input + split, length - split }));
        CHECK(dg_finish(parser));
        CHECK(dg_finish(parser));
```

```
        dg_report report = dg_get_report(parser);
        CHECK(report.records == 2);
        CHECK(report.sum == 19);
        CHECK(report.bytes == 11);

        dg_destroy(parser);
    }
}

static void test_errors(void)
{
    unsigned char bad[] = "a=7\ncpu=12x\n";
    dg_parser *parser = dg_create(NULL);
    CHECK(parser != NULL);

    CHECK(!dg_feed(parser, (dg_view){ bad, sizeof bad - 1 }));
    dg_error saved = dg_get_error(parser);
    CHECK(saved.code == DG_SYNTAX);
    CHECK(saved.offset == 10);

    bad[10] = '\n';
    CHECK(!dg_feed(parser, (dg_view){ bad, sizeof bad - 1 }));
    CHECK(dg_get_error(parser).offset == saved.offset);
    CHECK(!dg_finish(parser));
    dg_destroy(parser);

    CHECK(saved.code == DG_SYNTAX);
    CHECK(saved.offset == 10);

    static const unsigned char truncated[] = "a=7\ncpu=12";
    parser = dg_create(NULL);
    CHECK(parser != NULL);
    CHECK(dg_feed(parser,
                  (dg_view){ truncated, sizeof truncated - 1 }));
    CHECK(!dg_finish(parser));
    CHECK(dg_get_error(parser).code == DG_TRUNCATED);
    CHECK(dg_get_error(parser).offset == 10);
    dg_destroy(parser);
}

struct test_heap {
    size_t allocations;
    size_t releases;
    bool fail;
};

static void *test_allocate(size_t size, void *context)
{
    struct test_heap *heap = context;
    ++heap->allocations;
    return heap->fail ? NULL : malloc(size);
}
```

```
static void test_release(void *pointer, void *context)
{
    struct test_heap *heap = context;
    ++heap->releases;
    free(pointer);
}

static void test_allocation(void)
{
    struct test_heap heap = {0};
    dg_allocator allocator = {
        test_allocate, test_release, &heap
    };

    heap.fail = true;
    CHECK(dg_create(&allocator) == NULL);
    CHECK(heap.allocations == 1);
    CHECK(heap.releases == 0);

    heap.fail = false;
    dg_parser *parser = dg_create(&allocator);
    CHECK(parser != NULL);

    static const unsigned char bad[] = "!";
    CHECK(!dg_feed(parser, (dg_view){ bad, 1 }));
    dg_destroy(parser);
    dg_destroy(NULL);

    CHECK(heap.allocations == 2);
    CHECK(heap.releases == 1);
}

static void test_limit(void)
{
    static const unsigned char record[] = "a=0\n";

    for (int extra = 0; extra < 2; ++extra) {
        dg_parser *parser = dg_create(NULL);
        CHECK(parser != NULL);

        for (size_t i = 0; i < DG_MAX_INPUT / 4; ++i) {
            CHECK(dg_feed(parser, (dg_view){ record, 4 }));
        }

        CHECK(dg_get_report(parser).bytes == DG_MAX_INPUT);

        if (extra == 0) {
            CHECK(dg_finish(parser));
        } else {
            CHECK(!dg_feed(parser, (dg_view){ record, 1 }));
            CHECK(dg_get_error(parser).code == DG_TOO_LARGE);
```

```
            CHECK(dg_get_error(parser).offset == DG_MAX_INPUT);
        }

        dg_destroy(parser);
    }
}

int main(void)
{
    test_splits();
    test_errors();
    test_allocation();
    test_limit();
    return 0;
}
```

A darabolási teszt a golden bemenet minden két részre bontását kipróbálja, beleértve az üres első vagy második részt. A `input + length` one-past pointer csak nulla hosszúságú view-ban szerepel, és nem dereferáljuk.

Bizonyításvázlat a darabolásfüggetlenségre. A parser minden állapotváltozása egy bemeneti bájthoz kötődik; a feed-hívások határán nincs implicit rekordlezárás vagy állapotnullázás. Ugyanaz a bájtsorozat ezért ugyanazt az állapotátmenet-sorozatot eredményezi. A végleges lezárás kizárólag a `dg_finish` feladata. A teszt ennek konkrét ellenőrzése, nem minden lehetséges bemenet teljes bizonyítása.

Az allokációs teszt lefedi a könyvtár egyetlen allokációs pontjának hibáját. Nem vizsgálja a libc belső allokációit, és önmagában nem bizonyítja a teljes CLI minden erőforráshibás útját.

17.2.5.2 Fordítás két fordítóval

Az alábbi parancsokat a tanuló külön laborban használhatja. Alapértelmezésben csak a GCC-vel (`lab-gcc` profil) épít, a kötelező út így GCC-only tiszta gépen is végigfut. A Clang-mátrix (opcionális `lab-clang` profil) külön parancs: `COMPILERS="gcc clang" sh build.sh`; ha a `clang` nincs telepítve, a ciklus világos üzenettel kihagyja.

```
set -eu

for cc in ${COMPILERS:-gcc}; do
    if ! command -v "$cc" >/dev/null 2>&1; then echo "SKIP: a $cc nincs
     ↪  telepítve (opcionális fordító)"; continue; fi
    mkdir -p "build/$cc"

    "$cc" -std=c17 -O2 -g \
        -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
        -Iinclude -c src/diag.c -o "build/$cc/diag.o"

    ar rcs "build/$cc/libdiag.a" "build/$cc/diag.o"

    "$cc" -std=c17 -O2 -g \
        -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
        -D_POSIX_C_SOURCE=200809L -Iinclude \
        src/main.c "build/$cc/libdiag.a" -o "build/$cc/diag"
```

```
    "$cc" -std=c17 -O2 -g \
        -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
        -Iinclude tests/test_diag.c "build/$cc/libdiag.a" \
        -o "build/$cc/test_diag"
done
```

A statikus könyvtár a rá hivatkozó forrásból képzett objektum után kerül a linkparancsba. A publikus header önmagában nem tartalmaz definíciót a parser függvényeihez; azokat a könyvtár szolgáltatja.

17.2.5.3 Golden és csonkolt bemenet

A fixture fájlokat az alábbi parancsok pontosan, a záró LF jelenlétét is egyértelművé téve állítják elő:

```
printf 'a=7\ncpu=12\n' > tests/golden.in
printf 'OK records=2 sum=19 bytes=11\n' > tests/golden.out

printf 'a=7\ncpu=12' > tests/truncated.in
printf 'ERR code=truncated offset=10\n' > tests/truncated.err
```

A golden teszt ismert bemenethez előre rögzített, bájtpontosan összehasonlítható kimenetet rendel. A csonkolt változatból csak az utolsó LF hiányzik, de ettől az egész feldolgozás sikertelen: nincs részleges sikerjelentés.

```
set -eu

for cc in ${COMPILERS:-gcc}; do
    [ -x "build/$cc/test_diag" ] || { echo "SKIP: build/$cc hiányzik"; continue;
    ↪ }
    "build/$cc/test_diag"

    "build/$cc/diag" tests/golden.in > "build/$cc/golden.actual"
    cmp tests/golden.out "build/$cc/golden.actual"

    rc=0
    "build/$cc/diag" tests/truncated.in \
        > "build/$cc/truncated.out" \
        2> "build/$cc/truncated.err" || rc=$?

    test "$rc" -eq 4
    test ! -s "build/$cc/truncated.out"
    cmp tests/truncated.err "build/$cc/truncated.err"
done
```

A parancssori felület maga is a szerződés része, ezért az argumentumfeldolgozásnak külön regressziós tesztje van. A szkript a kilépési kódot, a teljes standard kimenetet és a teljes hibakimenetet is összeveti a projektszerződés táblázatával:

```
#!/bin/sh
# tests/cli_args.sh DIAG -- az argumentumfeldolgozás regressziós tesztje
set -u
```

```
diag=$1
fail=0
errf=$(mktemp) || exit 2

expect() {  # várt_rc várt_stdout várt_stderr parancs...
    want_rc=$1 want_out=$2 want_err=$3
    shift 3
    out=$("$@" 2>"$errf") && rc=0 || rc=$?
    err=$(cat "$errf")
    if [ "$rc" != "$want_rc" ] || [ "$out" != "$want_out" ] ||
       [ "$err" != "$want_err" ]; then
        echo "FAIL: $* -> rc=$rc out=[$out] err=[$err]" >&2
        fail=1
    fi
}

expect 0 'OK records=2 sum=19 bytes=11' '' "$diag" tests/golden.in
expect 0 'OK records=2 sum=19 bytes=11' '' sh -c "\"$diag\" - < tests/golden.in"
expect 0 'OK records=2 sum=19 bytes=11' '' sh -c "\"$diag\" < tests/golden.in"
expect 2 '' 'ERR code=usage' "$diag" tests/golden.in extra
expect 3 '' 'ERR code=open' "$diag" tests/does-not-exist.in
expect 4 '' 'ERR code=truncated offset=10' "$diag" tests/truncated.in
rm -f "$errf"
exit $fail
```

Futtatása fordítónként: `sh tests/cli_args.sh build/gcc/diag`. Sikeres esetben nem ír semmit, és `0` kóddal lép ki.

Buildjegyzőkönyv (`lab-gcc` profil). A fenti forrásokat, a fixture-öket, a golden- és a csonkolási tesztet, a `tests/cli_args.sh` szkriptet és a sanitizeres könyvtártesztet Linux x86-64 környezetben, GCC 13.3.0 és GNU binutils 2.42 mellett lefuttattuk; a fordítás figyelmeztetés nélküli volt, minden ellenőrzés sikeres. A sanitizeres buildet GCC-vel (`-fsanitize=address,undefined`) futtattuk. Clanggal és a `lab-next` profillal (profiltábla) ebben a jegyzőkönyvben nem fordítottunk; ezeket a saját laborban kell rögzíteni.

A könyvtár és tesztje külön diagnosztikai buildben is fordítható:

```
if command -v clang >/dev/null 2>&1; then
    clang -std=c17 -O1 -g -Wall -Wextra -Wpedantic \
        -fsanitize=address,undefined -fno-omit-frame-pointer \
        -Iinclude src/diag.c tests/test_diag.c \
        -o build/test_diag_san
else echo "SKIP: a clang nincs telepítve (a lab-clang profil opcionális, nem
 ↪  része az alaprecepnek)"; fi
```

Ez fordítási recept, nem sanitizermentes futásról szóló állítás.

17.2.6 Gyakori hibák és javításuk

Hibaminta	Miért hibás?	Javítás
`strlen` használata az olvasópufferen	Nem garantált a nullterminálás; a nullbájt is bemenet	A `fread` által visszaadott hossz átadása
Rekordlezárás minden feed végén	Az I/O-darabolás nem protokollhatár	Állapot megőrzése a hívások között
Offset pointerként való tárolása	A puffer újrahasználata érvényteleníti a tartalmi hivatkozást	Számszerű offset és másolható hibakód
Sikerjelentés a `finish` előtt	A végső rekord csonkolt lehet	Explicit EOF-feldolgozás
`fflush` eredményének elhagyása	A tényleges írás később is hibázhat	Flush és hibakód ellenőrzése
A cleanup hibája mindig felülírja az eredményt	Elveszhet az eredeti ok	Elsődleges hiba megőrzése

17.2.7 Feladatok

17.2.7.1 Magyarázó feladat: prefix és teljes eredmény

A parser megkapja az `a=7\ncpu=12` bemenetet. Milyen jelentést ad vissza közvetlenül a feed után? Mi történik finish esetén, és miért nem írhatja ki a CLI a prefix jelentését sikeres eredményként?

17.2.7.2 Kódolási feladat: numerikus határértékek

Bővítsd a tesztet a következő bemenetekkel:

- `k=65535\n`;
- `k=65536\n`;
- `k=000000\n`.

Ellenőrizd az elfogadást, illetve az első hibás bájt offsetjét. A teszt minden parserobjektumot szabadítson fel.

17.2.7.3 Diagnosztikai feladat: sikeres parse, hibás kimenet

Linuxon irányítsd a golden bemenet jelentését a `/dev/full` eszközre. Milyen kilépési kódot várs? Miért lehet hibás az a javítás, amely csak a `fprintf` eredményét ellenőrzi? Magyarázd meg azt is, miért nem várunk `release` hívást a sikertelen konstruktor után.

17.2.8 Megoldások

17.2.8.1 Prefix és teljes eredmény

A feed sikeres: még nem tudhatja, érkezik-e folytatás. A pillanatnyi jelentés:

- `bytes = 10`;
- `records = 1`;
- `sum = 7`.

A második rekord értéke már feldolgozható volt, de LF nélkül még nincs teljes rekord. A finish eredménye `DG_TRUNCATED`, offsetje 10. Az ellenőrzési feltétel: a CLI 4-es kóddal lép ki, és a standard kimenete üres. A prefixstatisztika diagnosztikai adat, nem végleges sikereredmény.

17.2.8.2 Numerikus határértékek

Az alábbi C17 részlet a tesztfájlba illeszthető. Hozzá kell adni a `<string.h>` include-ot, és a `main` függvényből meg kell hívni a tesztet.

```
static void test_numeric_boundaries(void)
{
    const struct {
        const char *input;
        dg_code code;
        size_t offset;
    } cases[] = {
        { "k=65535\n",  DG_OK,     0 },
        { "k=65536\n",  DG_SYNTAX, 6 },
        { "k=000000\n", DG_SYNTAX, 7 }
    };

    for (size_t i = 0; i < sizeof cases / sizeof cases[0]; ++i) {
        dg_parser *parser = dg_create(NULL);
        CHECK(parser != NULL);

        bool accepted = dg_feed(parser, (dg_view){
            (const unsigned char *)cases[i].input,
            strlen(cases[i].input)
        });

        if (cases[i].code == DG_OK) {
            CHECK(accepted);
            CHECK(dg_finish(parser));
            CHECK(dg_get_report(parser).sum == 65535);
        } else {
            CHECK(!accepted);
            dg_error error = dg_get_error(parser);
            CHECK(error.code == cases[i].code);
            CHECK(error.offset == cases[i].offset);
        }

        dg_destroy(parser);
    }
}
```

Itt a `strlen` helyes: a tesztadat garantáltan nullterminált sztringliterál, beágyazott nullbájt nélkül. A `65536` utolsó számjegye a 6-os offseten sérti az értékkorlátot; a hatodik nulla a 7-es offseten sérti a számjegyszám korlátját.

17.2.8.3 Hibás kimenet és allokációs hiba

A `/dev/full` Linux-specifikus diagnosztikai eszköz. Az ellenőrzés várt eredménye 6-os kilépési kód és `ERR code=write` hibasor, feltéve, hogy a hibakimenet írható.

```
rc=0
build/gcc/diag tests/golden.in > /dev/full 2> build/full.err || rc=$?
test "$rc" -eq 6
```

A rövid jelentés elférhet a standard I/O pufferében, ezért a `fprintf` sikeres lehet, miközben a `fflush` hibázik. Mindkét műveletet ellenőrizni kell.

Sikertelen konstruktor esetén az allokátor null pointert adott vissza: nem keletkezett tulajdonolt parserobjektum. Nincs mit felszabadítani. A teszt elvárt számlálói egy sikertelen, majd egy sikeres konstrukció és annak megsemmisítése után: két allokációs kísérlet, egy felszabadítás.

17.2.9 Rövid összefoglaló

A könyvtári mag kizárólag állapotot és másolható eredményt birtokol; a bemenetet csak a feed-hívás idejére kölcsönzi. A CLI feladata az I/O, a végleges lezárás, a stabil report és a hibakódok kezelése.

A helyességhez nem elég, hogy egy golden bemenet „jó számokat” ad. A darabolási határok, a csonkolt EOF, a pontos méretlimit, az allokációs hiba és a késleltetett kimeneti hiba ugyanannak a szerződésnek a részei. Ez a világos határ teszi lehetővé, hogy a következő lépésben a könyvtár változtatása nélkül építsünk hozzá nonblocking TCP-adaptert.

17.3 Nonblocking TCP adapter és leállítás

A TCP adapter feladata nem a parser újraírása, hanem a bájtfolyam, a kapcsolat élettartama és a korlátozott erőforrások összehangolása. Egy sikeres `recv()` nem jelent teljes kérést, egy sikeres `send()` pedig nem jelenti feltétlenül a teljes válasz elküldését. A kapcsolat ezért nem pusztán egy file descriptor: saját bemeneti és kimeneti tárolóval, feldolgozási állapottal, határidőkkel és eseményazonossággal rendelkező objektum.

A C könyvtári mag és CLI megvalósítási labor parserét szinkron adapterhatáron használjuk. A poll/epoll és explicit eseményállapotgép alapjaira építve most azt rögzítjük, mikor szabad új adatot fogadni, mikor érvénytelenednek a parser nézetei, és hogyan bizonyítható a kapcsolat lezárásának helyessége.

17.3.1 Platformprofil és adapterhatár

Platformlabor: Linux hosted környezet, x86-64 System V ABI, LP64, nyolcbites C bájt. A nyelvi alap ISO C17; az `epoll`, az `accept4()` és a `signalfd` Linux API. A profil a központi profiltábla `lab-gcc` profilja; a `clang` receptek a `lab-clang` profilhoz tartoznak. A ténylegesen elvégzett fordítások és futtatások jegyzőkönyve az egyes kódrészek után áll.

A szolgáltatás egyetlen szálon, level-triggered `epoll` ciklussal működik. Oktatási konfigurációként csak loopback címen figyel, egyszerre legfeljebb 64 kapcsolatot kezel.

A parser köré készülő `diag_reply_fn` adapter (a projektben a `diag_core_reply`, lásd lent) három eredményt adhat:

- `DIAG_MORE`: a rendelkezésre álló prefix még nem teljes kérés;
- `DIAG_REPLY`: egy teljes kérés feldolgozása és válaszának szerializálása sikerült;
- `DIAG_BAD`: a bemenet hibás, a kapcsolat megszakítható.

A függvény nem végez I/O-t, nem tart meg pointert, és nem allokál korlátlan mennyiségű tárolót. A bemenet hosszjelölt bájtszelet, nem szükségképpen nullterminált sztring. `DIAG_REPLY` esetén pozitív fogyasztási és válaszhosszt ad vissza; a válasz teljes egészében a kapcsolat kimeneti tömbjébe kerül.

Projektbeli elfogadási pont: a kérés akkor számít elfogadottnak, amikor a teljes válasz bekerült a kimeneti sorba. A kernelből már átvett, de még fel nem dolgozott bájtok önmagukban nem jelentenek elfogadott kérést. Ez a graceful shutdown szerződésének fontos része.

17.3.1.1 Tulajdonlási és érvényességi térkép

Erőforrás vagy nézet	Tulajdonos	Érvényesség vége, invalidáló művelet	Hibakezelés
Elfogadott socket FD	Átadásig az accept ág, utána a kapcsolatrekesz	Egyszeri `close()`	Sikertelen inicializáláskor is lezárandó
Bemeneti tömb	Kapcsolatrekesz	Rekesz megszűnése; tartalma tömörítéskor vagy újrafelhasználáskor változik	Túlméretes frame: kapcsolatbontás
Parser bemeneti nézete	Nem tulajdonló, csak a szinkron hívás alatt él	A hívás vége; későbbi `memmove()` megváltoztatja a hivatkozott tartalmat	Nem tehető várakozó eseménybe
Kimeneti tömb	Kapcsolatrekesz	Teljes elküldés vagy megszakítás után újrahasználható	Részleges küldéskor megőrzendő
Eseménytoken	Értékként tárolt slot- és generációazonosító	Generációváltás után elavult	El kell dobni, nem dereferálható pointer

A parser ideiglenes eredménye például mutathat a bemeneti tömbben található mezőre. Ebből a választ **még a parseradapter visszatérése előtt** szerializálni kell. A válaszsorba nem a mező pointere, hanem a szükséges bájtok másolata kerül.

17.3.2 Socketállapotok és accept loop

A kapcsolat állapotát a küldési sor foglaltságával együtt értelmezzük.

Állapot	Új kérés feldolgozható?	Eseményérdeklődés	Kilépés
`ACTIVE`, üres kimenet	Igen	`EPOLLIN`, `EPOLLRDHUP`	Válasz, EOF, hiba vagy timeout
`ACTIVE`, függő válasz	Nem	`EPOLLOUT`	Válasz elküldése után újra feldolgozás

Állapot	Új kérés feldolgozható?	Eseményérdeklődés	Kilépés
`DRAIN_SEND`	Nem	`EPOLLOUT`, ha szükséges	Kimenet kiürülése után féllezárás
`DRAIN_PEER`	Nem; a bejövő adat eldobandó	`EPOLLIN`, `EPOLLRDHUP`	EOF vagy shutdown-határidő
`DEAD`	Nem	Nincs	Rekesz később új generációval használható

Linux API. Az elfogadott socket nonblocking és close-on-exec tulajdonságát explicit kérjük:

```
/* Részlet a listener eseménykezelőjéből; Linux API. */
int fd = accept4(listener_fd, NULL, NULL,
                 SOCK_NONBLOCK | SOCK_CLOEXEC);
```

Ez részlet, nem önálló program; a listener már inicializált, nonblocking socket.

Az accept loop nem korlátlan ciklus. Egy eseménykezelés legfeljebb például 16 accept-kísérletet végez:

1. Szabad kapcsolatrekesz hiányában megszünteti a listener eseményfigyelését.
2. `EAGAIN` vagy `EWOULDBLOCK` esetén visszatér az eseményciklushoz.
3. `EINTR` esetén újrapróbálhat, de a kísérlet beleszámít a keretbe.
4. Siker után inicializálja a rekeszt, regisztrálja az FD-t, majd átadja a tulajdonlást.
5. Bármely közbülső hiba esetén lezárja a még át nem adott FD-t.

Az `EMFILE` és `ENFILE` nem indokolhat gyors, végtelen újrapróbálást. Ilyenkor a listener figyelését időlegesen fel kell függeszteni, és későbbi, határidővezérelt próbát ütemezni. A kapcsolatlimit elérésekor ugyanez akadályozza meg a folyamatosan olvashatónak jelzett listener miatti pörgést. Rekesz felszabadulásakor a figyelés újra engedélyezhető.

A kernel listen backlogja nem a felhasználói kapcsolattábla része: a 64-es limit önmagában nem korlátozza az összes kerneloldali hálózati memóriát.

17.3.3 Korlátos input/output sor

A legegyszerűbb backpressure-szabály: kapcsolatonként egyszerre legfeljebb egy teljes válasz várakozhat. Amíg az nincs elküldve, nem fogadunk új adatot, és nem dolgozunk fel további kérést.

Az alábbi komponens az aktív I/O-t és a kimenetürítést valósítja meg. A listener, az `epoll`-regisztráció, az időzítés és a végső teardown a szerveroldali vezérlő feladata.

17.3.3.1 Fordítható I/O-komponens

Az `include/diag_tcp.h` fájl:

```
#ifndef DIAG_TCP_H
#define DIAG_TCP_H

#include <stdbool.h>
#include <stddef.h>
```

```
#define DIAG_IN_CAP  4096u
#define DIAG_OUT_CAP 1024u

enum diag_core_result { DIAG_MORE, DIAG_REPLY, DIAG_BAD };
enum diag_step {
    DIAG_KEEP, DIAG_DONE, DIAG_BAD_FRAME,
    DIAG_SYS_ERROR, DIAG_CORE_ERROR
};

typedef enum diag_core_result (*diag_reply_fn)(
    const unsigned char *input, size_t input_size,
    size_t *consumed,
    unsigned char *output, size_t output_capacity,
    size_t *produced);

struct diag_io {
    int fd;
    bool draining;
    size_t used;
    size_t off, end;
    unsigned char input[DIAG_IN_CAP];
    unsigned char output[DIAG_OUT_CAP];
};

/* A core-adapter (src/tcp_reply.c): egy kérés = egy rekordsor. */
enum diag_core_result diag_core_reply(
    const unsigned char *input, size_t input_size,
    size_t *consumed,
    unsigned char *output, size_t output_capacity,
    size_t *produced);

enum diag_step diag_read(struct diag_io *, diag_reply_fn);
enum diag_step diag_write(struct diag_io *, diag_reply_fn);
enum diag_step diag_quiesce(struct diag_io *);

#endif
```

Az src/tcp_io.c fájl:

```
#include "diag_tcp.h"

#include <errno.h>
#include <string.h>
#include <sys/socket.h>

static enum diag_step prepare(struct diag_io *c,
                              diag_reply_fn reply)
{
    if (c->draining)
        return DIAG_DONE;
    if (c->used == 0)
```

```
        return DIAG_KEEP;

    size_t consumed = 0;
    size_t produced = 0;
    enum diag_core_result r = reply(
        c->input, c->used, &consumed,
        c->output, sizeof c->output, &produced);

    if (r == DIAG_BAD)
        return DIAG_BAD_FRAME;

    if (r == DIAG_MORE) {
        if (consumed != 0 || produced != 0)
            return DIAG_CORE_ERROR;
        return c->used == sizeof c->input
             ? DIAG_BAD_FRAME : DIAG_KEEP;
    }

    if (r != DIAG_REPLY ||
        consumed == 0 || consumed > c->used ||
        produced == 0 || produced > sizeof c->output)
        return DIAG_CORE_ERROR;

    /*
     * A reply() már visszatért: nincs élő borrowed
     * parsernézet, ezért a bemenet tömöríthető.
     */
    memmove(c->input, c->input + consumed,
            c->used - consumed);
    c->used -= consumed;
    c->off = 0;
    c->end = produced;
    return DIAG_KEEP;
}

enum diag_step diag_read(struct diag_io *c,
                         diag_reply_fn reply)
{
    if (c->draining || c->off != c->end)
        return DIAG_CORE_ERROR;
    if (c->used == sizeof c->input)
        return DIAG_BAD_FRAME;

    ssize_t n = recv(c->fd, c->input + c->used,
                     sizeof c->input - c->used, 0);

    if (n > 0) {
        c->used += (size_t)n;
        return prepare(c, reply);
    }
    if (n == 0)
        return c->used == 0 ? DIAG_DONE : DIAG_BAD_FRAME;
```

```
    if (errno == EINTR || errno == EAGAIN ||
        errno == EWOULDBLOCK)
        return DIAG_KEEP;
    return DIAG_SYS_ERROR;
}

enum diag_step diag_write(struct diag_io *c,
                          diag_reply_fn reply)
{
    if (c->off >= c->end)
        return DIAG_CORE_ERROR;

    ssize_t n = send(c->fd, c->output + c->off,
                     c->end - c->off, MSG_NOSIGNAL);

    if (n < 0) {
        if (errno == EINTR || errno == EAGAIN ||
            errno == EWOULDBLOCK)
            return DIAG_KEEP;
        return DIAG_SYS_ERROR;
    }
    if (n == 0) {
        errno = EIO;
        return DIAG_SYS_ERROR;
    }

    c->off += (size_t)n;
    if (c->off != c->end)
        return DIAG_KEEP;

    c->off = c->end = 0;
    return prepare(c, reply);
}

enum diag_step diag_quiesce(struct diag_io *c)
{
    c->draining = true;
    c->used = 0;  /* A még el nem fogadott kérések eldobása. */
    return c->off == c->end ? DIAG_DONE : DIAG_KEEP;
}
```

Fordítási receptek a két profilhoz (`lab-gcc`, illetve `lab-clang`):

```
gcc -std=c17 -D_GNU_SOURCE -Wall -Wextra -Wpedantic \
  -Wconversion -Wshadow -O2 -g -Iinclude \
  -c src/tcp_io.c -o tcp_io.gcc.o

if command -v clang >/dev/null 2>&1; then
  clang -std=c17 -D_GNU_SOURCE -Wall -Wextra -Wpedantic \
    -Wconversion -Wshadow -O2 -g -Iinclude \
    -c src/tcp_io.c -o tcp_io.clang.o
else echo "SKIP: a clang nincs telepítve (a lab-clang profil opcionális, nem
 ↪  része az alaprecepnek)"; fi
```

Ezek objektumfájlt készítenek, nem teljes szervert. A `diag_reply_fn` megvalósítása a core parser adaptere; ezt a következő szakasz adja meg. A komponenseket a lentebbi „Teljes referencia-szerver” szakasz köti össze futtatható programmá. Kezdetben egy még nem aktív rekesz I/O-része így inicializálható: `(struct diag_io){ .fd = fd }`. Élő kapcsolatot ezzel felülírni FD-szivárgás lenne.

17.3.3.2 A core-adapter: `src/tcp_reply.c`

A projektszerződés szerint egy TCP-kérés pontosan egy rekord az első LF-ig, a válasz pedig az a sor, amelyet a CLI erre az egy rekordból álló bemenetre írna. Az adapter ezért a C könyvtári mag változatlan `dg_*` API-ját hívja: kérésenként rövid életű parsert hoz létre, a sort betáplálja, lezárja, a jelentést vagy a hibát a CLI-vel azonos formátumban a kimeneti tömbbe írja, majd a parsert megszünteti.

```
#include "diag.h"
#include "diag_tcp.h"

#include <stdio.h>
#include <string.h>

/*
 * Egy TCP-kérés pontosan egy LF-fel lezárt rekordsor. A válasz az
 * a sor, amelyet a CLI erre az egyetlen sorból álló bemenetre a
 * stdout-ra (OK) vagy a stderr-re (ERR) írna.
 */
enum diag_core_result diag_core_reply(
    const unsigned char *input, size_t input_size,
    size_t *consumed,
    unsigned char *output, size_t output_capacity,
    size_t *produced)
{
    const unsigned char *lf = memchr(input, '\n', input_size);
    if (lf == NULL)
        return DIAG_MORE;   /* a felső korlátot a tcp_io.c tartja */

    size_t line = (size_t)(lf - input) + 1;
    dg_parser *parser = dg_create(NULL);
    int n;

    if (parser == NULL) {
        n = snprintf((char *)output, output_capacity,
                     "ERR code=alloc\n");
    } else if (dg_feed(parser, (dg_view){ input, line }) &&
               dg_finish(parser)) {
        dg_report r = dg_get_report(parser);
        n = snprintf((char *)output, output_capacity,
                     "OK records=%zu sum=%llu bytes=%zu\n",
                     r.records, r.sum, r.bytes);
    } else {
        dg_error e = dg_get_error(parser);
        n = snprintf((char *)output, output_capacity,
```

```
                    "ERR code=%s offset=%zu\n",
                    dg_code_name(e.code), e.offset);
    }
    dg_destroy(parser);   /* abort: lezárás finish nélkül is */

    if (n < 0 || (size_t)n >= output_capacity)
        return DIAG_BAD;
    *consumed = line;
    *produced = (size_t)n;
    return DIAG_REPLY;
}
```

A szerződés kapcsolóelemei itt konkrétak:

- **Fogyasztott bájtszám:** a `consumed` a kérés hossza az LF-fel együtt. A parser hibája esetén is a teljes kérés elfogy, mert az LF egyértelműen lezárta; a hiba helyét a válasz offsetje adja.
- **EOF:** a kérés végét a `dg_finish` jelzi. LF nélküli, félbemaradt kérést az adapter nem lát lezártnak (`DIAG_MORE`); a kliens féllezárásakor ezt a `diag_read` keretszintű hibának minősíti (`DIAG_BAD_FRAME`).
- **Abort:** a parser minden ágon `dg_destroy` hívással szűnik meg, akkor is, ha a `feed` hibát adott és `finish` nem futott. A `dg_destroy(NULL)` megengedett, ezért az allokációs hibaág is ugyanide fut.
- **Eredményformázás:** az `snprintf` formátumai a CLI-ével betűre azonosak. A legrövidebb kapacitás is bőven elég: a leghosszabb válasz 32 oktett, a `DIAG_OUT_CAP` 1024.

A függvény nem végez I/O-t, nem tart meg pointert a bemenetre, és egyetlen, rögzített méretű allokációt végez kérésenként. Ha ez a terhelésnél számít, a parserobjektum a kapcsolatrekeszbe is áthelyezhető, `dg_allocator` segítségével, a viselkedés megváltoztatása nélkül.

A core, az I/O-komponens és az adapter együttes linkelését egy socketpáron futó teszt ellenőrzi. A kliens két érvényes, egy hibás és egy félbemaradt kérést küld egyetlen írással, majd lezárja a küldési irányát:

```
/* Linkpróba: core + TCP I/O + core-adapter, valódi socketpáron. */
#include "diag_tcp.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#define CHECK(condition) do {                                       \
    if (!(condition)) {                                             \
        (void)fprintf(stderr, "CHECK failed at line %d\n", __LINE__); \
        exit(EXIT_FAILURE);                                         \
    }                                                               \
} while (0)

int main(void)
{
    static const char request[] = "a=7\ncpu=12\nA=1\nmem=";
    static const char expected[] =
```

```
        "OK records=1 sum=7 bytes=4\n"
        "OK records=1 sum=12 bytes=7\n"
        "ERR code=syntax offset=0\n";
    int sv[2];
    char reply[256];
    size_t got = 0;

    CHECK(socketpair(AF_UNIX, SOCK_STREAM, 0, sv) == 0);
    CHECK(write(sv[1], request, sizeof request - 1)
          == (ssize_t)(sizeof request - 1));
    CHECK(shutdown(sv[1], SHUT_WR) == 0);

    struct diag_io c = { .fd = sv[0] };
    enum diag_step step = diag_read(&c, diag_core_reply);

    while (step == DIAG_KEEP) {
        step = c.off != c.end ? diag_write(&c, diag_core_reply)
                              : diag_read(&c, diag_core_reply);
    }
    /* A félbemaradt "mem=" kérés EOF-nál keretszintű hiba. */
    CHECK(step == DIAG_BAD_FRAME);

    for (;;) {
        ssize_t n = recv(sv[1], reply + got, sizeof reply - got,
                         MSG_DONTWAIT);
        if (n <= 0)
            break;
        got += (size_t)n;
    }
    CHECK(got == sizeof expected - 1);
    CHECK(memcmp(reply, expected, got) == 0);

    close(sv[0]);
    close(sv[1]);
    return 0;
}
```

```
gcc -std=c17 -O2 -g -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
    -Werror -D_GNU_SOURCE -Iinclude \
    src/tcp_io.c src/tcp_reply.c tests/test_tcp_link.c \
    build/gcc/libdiag.a -o build/gcc/test_tcp_link
build/gcc/test_tcp_link
```

A `libdiag.a` a megvalósítási labor buildjéből származik. A teszt `AF_UNIX` socketpárt használ, mert a `recv`/`send` szemantikája ehhez elég, és nem kell portot foglalni; a valódi loopback TCP-teszt ezt kiegészíti, nem helyettesíti.

Buildjegyzőkönyv (`lab-gcc` profil). A `tcp_io.c`, a `tcp_reply.c` és a linkpróba Linux x86-64 környezetben, GCC 13.3.0 és glibc 2.39 mellett `-Werror` beállítással figyelmeztetés nélkül fordult; a teszt sikeresen lefutott, `-fsanitize=address,undefined` builddel is. A teljes, `epoll`-alapú szerver-vezérlő nem része ennek az alfejezetnek, ezért arról futtatási eredményt nem állítunk.

API-előfeltételek: a kapcsolatpointer nem null, az objektum él, az FD érvényes és nonblocking;

a callback nem null, a hívás teljes ideje alatt elérhető, és betartja a kapacitásokat. A visszaadott hossz ellenőrzése nem teszi utólag biztonságossá a callback esetleges túlindexelését.

A `DIAG_SYS_ERROR` után az `errno` értékét más rendszerhívás előtt kell elmenteni. A `DIAG_CORE_ERROR` programozási szerződéssértés, nem egyszerűen hibás klienskérés.

17.3.3.3 Részleges frame és backpressure

Tegyük fel, hogy a kérés a 7 oktettes `cpu=12\n`, de a socket először csak 4-et ad át.

1. `used = 4`, a bemenetben még nincs LF, az adapter eredménye `DIAG_MORE`.
2. A következő olvasás további 3 oktettet ad: `used = 7`.
3. Az adapter 7 oktettet fogyaszt, és a 28 oktettes `OK records=1 sum=12 bytes=7\n` választ készíti el.
4. Az első `send()` például csak 3 oktettet küld: `off = 3`, `end = 28`.
5. A következő küldés címe `output + 3`, hossza $28 - 3 = 25$.

A kérés és a válasz hossza a szerződésből következik; a darabolás (4+3, illetve 3+25) illusztratív, nem mért eredmény.

Ha ugyanabban az olvasásban a következő kérés eleje is megérkezett, azt a `memmove()` megőrzi. Különösen fontos, hogy az utolsó válaszbájt elküldése után a `prepare()` azonnal megvizsgálja a már tárolt bemenetet. Ellenkező esetben a szerver olyan `EPOLLIN` eseményre várhatna, amely nem érkezik meg: az adat már felhasználói tárolóban van.

Bizonyításvázlat – tárolókorlátok. Az invariánsok:

$$0 \le used \le I, \qquad 0 \le off \le end \le O,$$

ahol $I = 4096$, $O = 1024$, az egység C bájt.

A `recv()` legfeljebb $I - used$ bájtot kap célterületként. A parser csak üres kimeneti sor mellett fut, és legfeljebb O bájtot írhat. A `send()` legfeljebb $end - off$ bájtot kap, ezért pozitív eredménye nem lépteti `off` értékét `end` fölé. `EAGAIN` egyik határt sem módosítja. A megtelt, mégis befejezetlen bemenet elutasítása kizárja a korlátlan növekedést.

A 64 kapcsolat puszta adattömbjeinek felső korlátja:

$$64(4096 + 1024) = 327680 \text{ C bájt} = 320 \text{ KiB}$$

a választott nyolcbites profilban. Ehhez hozzáadódnak a struktúratagok, padding, időzítők, eseménytömb és a core korlátozott munkaterülete. Ez felhasználói tárolókorlát, nem teljes RSS- vagy kernelmemória-korlát.

A tömörítés egy kérés után legrosszabb esetben $O(I)$ másolás; folytonos tárolón dolgozik, allokáció nélkül, és érvényteleníti a bemenet korábbi tartalmára építő nézeteket.

17.3.4 FD-generáció és teardown

Az FD egész száma újrahasználható. Egy lezárt `fd = 12` helyére később másik kapcsolat kerülhet, miközben egy már kiolvasott eseménykötegben még tartalmaz régi bejegyzést.

Ezért az eseményadat nem puszta FD, hanem:

$$token = (generation \ll 32) \mid slot.$$

A profilban rendelkezésre álló `uint32_t` és `uint64_t` típusokkal a shiftet 64 bites értéken kell végezni. A slotok stabil helyen maradnak a szerver teljes élettartama alatt.

```
/* Dispatch-részlet; a speciális listener/signal tokenek
   feldolgozása ezt megelőzi. */
uint64_t token = events[k].data.u64;
uint32_t index = (uint32_t)token;
uint32_t generation = (uint32_t)(token >> 32);

if (index >= CONNECTION_LIMIT)
    continue;
if (!slots[index].active ||
    slots[index].generation != generation)
    continue;
```

Új kapcsolatnál a generáció nő. A körbefordulás nem megengedett: `UINT32_MAX` elérése után a rekeszt nyugdíjazni kell. Így a stale event, vagyis az elavult esemény nem válhat később ismét érvényessé.

A teardown sorrendje:

1. A rekesz logikailag inaktívvá válik.
2. Az FD-t kivesszük az `epoll` figyeléséből.
3. A tárolt FD-t `-1`-re állítjuk, a korábbi értéket egyszer lezárjuk.
4. Érvénytelenítjük a kapcsolati határidőket és ürítjük a számlálókat.

Linux-specifikus szabály: a `close()` hibájára nem szabad automatikusan újabb `close()`-zal válaszolni; az FD száma közben újra felhasználható lehet. A hibát rögzítjük.

Bizonyításvázlat – egyszeri lezárás. Minden hibakimenet ugyanabba a teardown útvonalba vezet. Az útvonal első művelete megszünteti az aktív tulajdonosi állapotot, így későbbi esemény már nem indíthat újabb lezárást. A parserhívás szinkron, ezért teardown közben nincs élő parsernézet. Az új generáció pedig kizárja, hogy régi esemény az új tulajdonost érje el.

17.3.5 Timeout és graceful shutdown

Az időzítők monotón órára épülnek. Példapolicy:

- üres, tétlen kapcsolat: 30 másodperc;
- befejezetlen frame: 5 másodperc a parser első `DIAG_MORE` eredményétől;
- függő válasz: 5 másodperc a sorba állításától;
- teljes szerverleállítás: legfeljebb 3 másodperc.

A frame-határidőt nem hosszabbítjuk meg minden beérkező bájttal. Különben a nagyon lassan csöpögtető kliens korlátlan ideig foglalhatna rekeszt. A válasz határideje sem csúszik minden részleges küldéskor.

A következő várakozás a legközelebbi aktív határidőhöz igazodik. A lejáratokat minden eseményköteg után ellenőrizni kell, nem csak akkor, amikor az `epoll_wait()` timeouttal tér vissza. A folyamatos forgalom különben kiéheztethetné az időzítőket.

Linux API. A blokkolt `SIGTERM` és `SIGINT` jeleket `signalfd` teheti a közös eseményfolyam részévé. A shutdown kezdetekor:

1. megszüntetjük a listener figyelését és lezárjuk a listenert;
2. rögzítjük a közös, abszolút shutdown-határidőt;
3. minden kapcsolatnál meghívjuk a `diag_quiesce()` függvényt;
4. csak a már sorba állított válaszokat ürítjük.

A kimenet kiürülése után `shutdown(fd, SHUT_WR)` következik, majd `DRAIN_PEER`: korlátozott mennyiségekben olvasunk és eldobunk adatot az EOF-ig vagy a közös határidőig. Ez elkerüli azt a leegyszerűsítést, hogy olvasatlan TCP-adattal azonnal bezárjuk a socketet, ami resethez vezethet. A határidő utáni kényszerlezárás már vállaltan megszakíthat adatátvitelt.

Az `EPOLLRDHUP` nem azonos a kapcsolat azonnali eldobásával: a peer küldési iránya zárult le, miközben adat még olvasható lehet. Az EOF-ot a `recv() == 0` igazolja. A `HUP` sem helyettesíti a maradék bemenet kezelését; `EPOLLERR` esetén a socket hibáját külön vizsgáljuk.

A sikeres `send()` csak a helyi kernel általi átvételt jelenti. Az alkalmazásszintű feldolgozás igazolásához külön protokoll-visszaigazolás kellene.

17.3.6 Teljes referencia-szerver

Az eddigi elemeket — az accept loopot, a generációs tokent, a korlátos I/O-komponenst, a core-adaptert, a határidőket és a `signalfd`-alapú leállítást — egyetlen futtatható program köti össze. A szerver egyszálú, level-triggered `epoll` ciklus; a kapcsolatrekeszek statikus tömbben élnek, így címük a futás teljes ideje alatt stabil.

A parancssori felület: `diag_server [PORT]`. A szerver kizárólag a `127.0.0.1` címen figyel; a `0` vagy a hiányzó port esetén a kernel választ portot. Sikeres indulás után a standard kimenetre pontosan egy sort ír, például `LISTEN 127.0.0.1:40123`, így egy tesztkliens a tényleges portot is megtudja. Kilépési kód: rendes leállítás után `0`, hibás argumentumnál `2`, indítási vagy eseményciklus-hibánál `5`.

```c
/* src/diag_server.c -- referencia-szerver: listener, epoll, határidők,
 * signalfd-alapú graceful shutdown. ISO C17 + Linux API. */
#include "diag_tcp.h"

#include <arpa/inet.h>
#include <errno.h>
#include <netinet/in.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/epoll.h>
#include <sys/signalfd.h>
```

```
#include <sys/socket.h>
#include <time.h>
#include <unistd.h>

/* A 17.1 határidői; tesztbuildben -D kapcsolóval rövidíthetők. */
#ifndef DIAG_IDLE_MS
#define DIAG_IDLE_MS 30000
#endif
#ifndef DIAG_FRAME_MS
#define DIAG_FRAME_MS 5000
#endif
#ifndef DIAG_REPLY_MS
#define DIAG_REPLY_MS 5000
#endif
#ifndef DIAG_SHUTDOWN_MS
#define DIAG_SHUTDOWN_MS 3000
#endif

#define CONNECTION_LIMIT 64
#define ACCEPT_BURST 16
#define RETRY_MS 100
#define TOKEN_LISTENER UINT64_MAX
#define TOKEN_SIGNAL (UINT64_MAX - 1)

enum phase { PH_IDLE, PH_FRAME, PH_REPLY, PH_DRAIN_PEER };

struct slot {
    struct diag_io io;
    uint32_t generation;
    bool active;
    bool retired;
    enum phase phase;
    int64_t deadline;              /* monoton óra, ms */
};

static struct slot slots[CONNECTION_LIMIT];
static size_t active_count;
static int ep = -1;
static int listener = -1;
static bool listening;             /* a listener az epollban van-e */
static int64_t listener_retry = -1;
static bool stopping;
static int64_t stop_deadline;

static int64_t now_ms(void)
{
    struct timespec ts;
    (void)clock_gettime(CLOCK_MONOTONIC, &ts);
    return (int64_t)ts.tv_sec * 1000 + ts.tv_nsec / 1000000;
}

static uint64_t token_of(size_t i)
```

```
{
    return (uint64_t)slots[i].generation << 32 | (uint64_t)i;
}

static int set_interest(size_t i, int op)
{
    struct slot *s = &slots[i];
    struct epoll_event ev = { .data.u64 = token_of(i) };

    if (s->phase != PH_DRAIN_PEER && s->io.off < s->io.end)
        ev.events = EPOLLOUT;
    else
        ev.events = EPOLLIN | EPOLLRDHUP;
    return epoll_ctl(ep, op, s->io.fd, &ev);
}

static void resume_listener(void)
{
    struct epoll_event ev = { .events = EPOLLIN,
                              .data.u64 = TOKEN_LISTENER };

    if (stopping || listening || listener < 0)
        return;
    if (epoll_ctl(ep, EPOLL_CTL_ADD, listener, &ev) == 0) {
        listening = true;
        listener_retry = -1;
    }
}

static void pause_listener(int64_t retry_at)
{
    if (listening) {
        (void)epoll_ctl(ep, EPOLL_CTL_DEL, listener, NULL);
        listening = false;
    }
    listener_retry = retry_at;    /* -1: szabad rekeszre várunk */
}

static void teardown(size_t i)
{
    struct slot *s = &slots[i];
    int fd = s->io.fd;

    s->active = false;            /* 1. logikailag inaktív */
    (void)epoll_ctl(ep, EPOLL_CTL_DEL, fd, NULL);
    s->io.fd = -1;
    if (close(fd) == -1)          /* egyszer; hiba esetén sem újra */
        perror("close");
    if (s->generation == UINT32_MAX)
        s->retired = true;        /* körbefordulás helyett nyugdíj */
    --active_count;
    resume_listener();
```

```
}

static void refresh(size_t i, bool new_reply)
{
    struct slot *s = &slots[i];
    enum phase p;

    if (s->io.off < s->io.end)
        p = PH_REPLY;
    else if (s->io.used > 0)
        p = PH_FRAME;
    else
        p = PH_IDLE;

    /* Fázisváltáskor indul új határidő; bájtonként nem hosszabbodik. */
    if (p != s->phase || (p == PH_REPLY && new_reply)) {
        int64_t span = p == PH_REPLY ? DIAG_REPLY_MS
                     : p == PH_FRAME ? DIAG_FRAME_MS : DIAG_IDLE_MS;
        s->phase = p;
        s->deadline = now_ms() + span;
    }
}

static void begin_drain_peer(size_t i)
{
    struct slot *s = &slots[i];

    if (shutdown(s->io.fd, SHUT_WR) == -1) {
        teardown(i);
        return;
    }
    s->phase = PH_DRAIN_PEER;
    s->deadline = stop_deadline;
    if (set_interest(i, EPOLL_CTL_MOD) == -1)
        teardown(i);
}

static void handle_step(size_t i, enum diag_step step, bool new_reply)
{
    switch (step) {
    case DIAG_KEEP:
        refresh(i, new_reply);
        if (set_interest(i, EPOLL_CTL_MOD) == -1)
            teardown(i);
        return;
    case DIAG_DONE:
        if (slots[i].io.draining)
            begin_drain_peer(i);  /* leállítás: a kimenet kiürült */
        else
            teardown(i);          /* rendezett EOF, nincs függő válasz */
        return;
    case DIAG_CORE_ERROR:
```

```
        (void)fprintf(stderr, "diag_server: core contract violation\n");
        teardown(i);
        return;
    case DIAG_BAD_FRAME:
    case DIAG_SYS_ERROR:
        teardown(i);
        return;
    }
}

static void drain_peer(size_t i)
{
    unsigned char sink[1024];
    ssize_t n = recv(slots[i].io.fd, sink, sizeof sink, 0);

    if (n > 0)
        return;                     /* eldobjuk; a szint-trigger újra jelez */
    if (n == -1 && (errno == EINTR || errno == EAGAIN ||
                    errno == EWOULDBLOCK))
        return;
    teardown(i);                    /* EOF vagy hiba */
}

static void accept_burst(void)
{
    for (int k = 0; k < ACCEPT_BURST; ++k) {
        size_t i = 0;
        while (i < CONNECTION_LIMIT && (slots[i].active || slots[i].retired))
            ++i;
        if (i == CONNECTION_LIMIT) {
            pause_listener(-1);
            return;
        }

        int fd = accept4(listener, NULL, NULL,
                         SOCK_NONBLOCK | SOCK_CLOEXEC);
        if (fd == -1) {
            if (errno == EINTR || errno == ECONNABORTED)
                continue;
            if (errno == EMFILE || errno == ENFILE ||
                errno == ENOBUFS || errno == ENOMEM)
                pause_listener(now_ms() + RETRY_MS);
            else if (errno != EAGAIN && errno != EWOULDBLOCK)
                perror("accept4");
            return;
        }

        struct slot *s = &slots[i];
        s->io = (struct diag_io){ .fd = fd };
        ++s->generation;
        s->phase = PH_IDLE;
        s->deadline = now_ms() + DIAG_IDLE_MS;
```

```
        if (set_interest(i, EPOLL_CTL_ADD) == -1) {
            s->io.fd = -1;
            (void)close(fd);      /* még az accept ág a tulajdonos */
            continue;
        }
        s->active = true;         /* tulajdonátadás a rekesznek */
        ++active_count;
    }
}

static void start_stop(void)
{
    if (stopping)
        return;
    stopping = true;
    stop_deadline = now_ms() + DIAG_SHUTDOWN_MS;

    pause_listener(-1);
    if (close(listener) == -1)
        perror("close");
    listener = -1;

    for (size_t i = 0; i < CONNECTION_LIMIT; ++i) {
        if (!slots[i].active || slots[i].phase == PH_DRAIN_PEER)
            continue;
        if (diag_quiesce(&slots[i].io) == DIAG_DONE)
            begin_drain_peer(i);
        else
            handle_step(i, DIAG_KEEP, false);
    }
}

static void dispatch(const struct epoll_event *ev, int sfd)
{
    uint64_t token = ev->data.u64;

    if (token == TOKEN_LISTENER) {
        accept_burst();
        return;
    }
    if (token == TOKEN_SIGNAL) {
        struct signalfd_siginfo info;
        while (read(sfd, &info, sizeof info) == (ssize_t)sizeof info)
            start_stop();
        return;
    }

    uint32_t i = (uint32_t)token;
    uint32_t generation = (uint32_t)(token >> 32);
    if (i >= CONNECTION_LIMIT || !slots[i].active ||
        slots[i].generation != generation)
        return;                   /* elavult esemény */
```

```
    struct slot *s = &slots[i];
    if (s->phase == PH_DRAIN_PEER) {
        drain_peer(i);
    } else if (ev->events & EPOLLERR) {
        teardown(i);
    } else if (s->io.off < s->io.end) {
        size_t used = s->io.used;
        enum diag_step step = diag_write(&s->io, diag_core_reply);
        handle_step(i, step, s->io.used < used);
    } else {
        handle_step(i, diag_read(&s->io, diag_core_reply), true);
    }
}

static void expire(int64_t now)
{
    if (listener_retry >= 0 && now >= listener_retry)
        resume_listener();

    for (size_t i = 0; i < CONNECTION_LIMIT; ++i) {
        if (slots[i].active &&
            (now >= slots[i].deadline || (stopping && now >= stop_deadline)))
            teardown(i);
    }
}

static int next_timeout(void)
{
    int64_t next = -1;

    if (listener_retry >= 0)
        next = listener_retry;
    if (stopping && (next < 0 || stop_deadline < next))
        next = stop_deadline;
    for (size_t i = 0; i < CONNECTION_LIMIT; ++i) {
        if (slots[i].active && (next < 0 || slots[i].deadline < next))
            next = slots[i].deadline;
    }
    if (next < 0)
        return -1;

    int64_t wait = next - now_ms();
    return wait <= 0 ? 0 : wait > 60000 ? 60000 : (int)wait;
}

static int setup(unsigned long port, int *sfd)
{
    sigset_t mask;
    struct sockaddr_in addr = {
        .sin_family = AF_INET,
        .sin_port = htons((uint16_t)port),
```

```
        .sin_addr.s_addr = htonl(INADDR_LOOPBACK),
    };
    socklen_t length = sizeof addr;
    int one = 1;

    if (sigemptyset(&mask) == -1 || sigaddset(&mask, SIGTERM) == -1 ||
        sigaddset(&mask, SIGINT) == -1 ||
        sigprocmask(SIG_BLOCK, &mask, NULL) == -1)
        return -1;
    *sfd = signalfd(-1, &mask, SFD_NONBLOCK | SFD_CLOEXEC);
    ep = epoll_create1(EPOLL_CLOEXEC);
    listener = socket(AF_INET, SOCK_STREAM | SOCK_NONBLOCK | SOCK_CLOEXEC, 0);
    if (*sfd == -1 || ep == -1 || listener == -1)
        return -1;

    struct epoll_event ev = { .events = EPOLLIN, .data.u64 = TOKEN_SIGNAL };
    if (setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &one, sizeof one) ||
        bind(listener, (struct sockaddr *)&addr, sizeof addr) == -1 ||
        listen(listener, 128) == -1 ||
        getsockname(listener, (struct sockaddr *)&addr, &length) == -1 ||
        epoll_ctl(ep, EPOLL_CTL_ADD, *sfd, &ev) == -1)
        return -1;

    resume_listener();
    if (!listening)
        return -1;
    if (printf("LISTEN 127.0.0.1:%u\n", (unsigned)ntohs(addr.sin_port)) < 0 ||
        fflush(stdout) == EOF)
        return -1;
    return 0;
}

int main(int argc, char **argv)
{
    unsigned long port = 0;
    int sfd = -1;
    int status = 0;

    if (argc > 2) {
        (void)fprintf(stderr, "ERR code=usage\n");
        return 2;
    }
    if (argc == 2) {
        char *end;
        errno = 0;
        port = strtoul(argv[1], &end, 10);
        if (errno != 0 || end == argv[1] || *end != '\0' || port > 65535) {
            (void)fprintf(stderr, "ERR code=usage\n");
            return 2;
        }
    }
```

```
    if (setup(port, &sfd) == -1) {
        perror("diag_server: setup");
        status = 5;
    } else {
        struct epoll_event events[CONNECTION_LIMIT + 2];

        while (!stopping || active_count > 0) {
            int n = epoll_wait(ep, events, CONNECTION_LIMIT + 2,
                               next_timeout());
            if (n == -1) {
                if (errno == EINTR)
                    continue;
                perror("epoll_wait");
                status = 5;
                break;
            }
            for (int k = 0; k < n; ++k)
                dispatch(&events[k], sfd);
            expire(now_ms());      /* minden köteg után, nem csak timeoutkor */
        }
    }

    for (size_t i = 0; i < CONNECTION_LIMIT; ++i) {
        if (slots[i].active)
            teardown(i);
    }
    if (listener >= 0)
        (void)close(listener);
    if (sfd >= 0)
        (void)close(sfd);
    if (ep >= 0)
        (void)close(ep);
    return status;
}
```

Néhány döntés, amely a kódból nem feltétlenül olvasható ki:

- **Fázis és határidő.** A `refresh` minden lépés után a sorállapotból számolja ki a fázist: függő válasz esetén `REPLY`, tárolt, befejezetlen kérésnél `FRAME`, egyébként `IDLE`. Új határidő csak fázisváltáskor vagy új válasz sorba állításakor indul, így a csöpögtetett bájtok és a részleges küldések nem hosszabbítják meg.
- **Új válasz felismerése.** Írás után akkor került új válasz a sorba, ha a `diag_write` közben a tárolt bemenet csökkent, vagyis az adapter egy kérést elfogyasztott.
- **Kapcsolatlimit.** Szabad rekesz hiányában a listener kikerül az `epoll` figyeléséből; egy rekesz felszabadulásakor a `teardown` visszaveszi. Így a folyamatosan olvashatónak jelzett listener nem pörgeti a ciklust. `EMFILE` és hasonló hibáknál 100 ms múlva próbálkozik újra.
- **Leállítás.** A `SIGTERM` vagy `SIGINT` után a listener bezárul, minden kapcsolat `diag_quiesce` hívást kap, a függő válaszok kiürülnek, majd `shutdown(SHUT_WR)` és `DRAIN_PEER` következik. A közös határidő után minden maradó kapcsolat lezárul, és a program `0` kóddal kilép.
- **Határidő-ellenőrzés.** Az `expire` minden eseményköteg után fut, nem csak `epoll_wait` timeout esetén, így a folyamatos forgalom nem éhezteti ki az időzítőket.

Fordítás a megvalósítási labor magjával együtt:

```
gcc -std=c17 -O2 -g -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
    -Werror -D_GNU_SOURCE -Iinclude \
    src/diag_server.c src/tcp_io.c src/tcp_reply.c src/diag.c \
    -o build/gcc/diag_server
```

A határidők fordítási makrók, ezért az elfogadási teszt rövidített határidőkkel fordított példányt használ; a szerver logikája ugyanaz:

```
gcc -std=c17 -O1 -g -D_GNU_SOURCE -Iinclude \
    -DDIAG_IDLE_MS=1500 -DDIAG_FRAME_MS=500 \
    -DDIAG_REPLY_MS=500 -DDIAG_SHUTDOWN_MS=1000 \
    -fsanitize=address,undefined -fno-omit-frame-pointer \
    src/diag_server.c src/tcp_io.c src/tcp_reply.c src/diag.c \
    -o build/gcc/diag_server_test
python3 tests/server_test.py build/gcc/diag_server_test
```

Az elfogadási teszt egy valódi loopback TCP-kliens. Pipelined és hibás kéréseket, bájtonként érkező bemenetet, a leghosszabb érvényes kérés minden kettévágását, rendezett és félbemaradt EOF-ot, a három határidőt, a 64 kapcsolatos limitet, nagy mennyiségű kérést egyidejű olvasással (backpressure), valamint graceful shutdownt ellenőriz. A leállítási tesztben egy kliens folyamatosan küld, de nem olvas; a szervernek ennek ellenére a közös határidőn belül ki kell lépnie.

```
#!/usr/bin/env python3
"""tests/server_test.py SERVER -- a referencia-szerver elfogadási tesztje.

A szervert rövidített határidőkkel fordítva kell átadni:
IDLE=1500, FRAME=500, REPLY=500, SHUTDOWN=1000 ms.
"""
import signal
import socket
import subprocess
import sys
import threading
import time

IDLE, FRAME, SHUTDOWN = 1.5, 0.5, 1.0
LIMIT = 64


def start(binary):
    proc = subprocess.Popen([binary], stdout=subprocess.PIPE, text=True)
    line = proc.stdout.readline().strip()
    assert line.startswith("LISTEN 127.0.0.1:"), line
    return proc, int(line.rsplit(":", 1)[1])


def connect(port):
    s = socket.create_connection(("127.0.0.1", port), timeout=3)
    return s


def recv_until_closed(s, timeout=3):
    s.settimeout(timeout)
```

```
    data = b""
    while True:
        chunk = s.recv(4096)
        if not chunk:
            return data
        data += chunk


def recv_lines(s, count, timeout=3):
    s.settimeout(timeout)
    data = b""
    while data.count(b"\n") < count:
        chunk = s.recv(4096)
        assert chunk, "a kapcsolat idő előtt bezárult"
        data += chunk
    return data


def closed_within(s, seconds):
    """Igaz, ha a szerver a megadott időn belül lezárja a kapcsolatot."""
    s.settimeout(seconds)
    try:
        while s.recv(4096):
            pass
        return True
    except (socket.timeout, TimeoutError):
        return False
    except ConnectionResetError:
        return True


def test_pipelined_and_error(port):
    s = connect(port)
    s.sendall(b"a=7\ncpu=12\nA=1\n")
    assert recv_lines(s, 3) == (b"OK records=1 sum=7 bytes=4\n"
                                b"OK records=1 sum=12 bytes=7\n"
                                b"ERR code=syntax offset=0\n")
    s.close()


def test_partial_input(port):
    s = connect(port)
    for b in b"cpu=12\n":                # bájtonként, szünetekkel
        s.sendall(bytes([b]))
        time.sleep(0.02)
    assert recv_lines(s, 1) == b"OK records=1 sum=12 bytes=7\n"
    s.close()


def test_max_request_splits(port):
    request = b"abcdefghijklmnop=65535\n"    # 23 bájt, a leghosszabb
    s = connect(port)
```

```
    s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
    for split in range(len(request) + 1):
        s.sendall(request[:split])
        time.sleep(0.005)
        s.sendall(request[split:])
        assert recv_lines(s, 1) == b"OK records=1 sum=65535 bytes=23\n"
    s.close()


def test_eof(port):
    s = connect(port)
    s.sendall(b"a=1\n")
    s.shutdown(socket.SHUT_WR)            # rendezett EOF teljes kérés után
    assert recv_until_closed(s) == b"OK records=1 sum=1 bytes=4\n"
    s.close()

    s = connect(port)
    s.sendall(b"mem=")
    s.shutdown(socket.SHUT_WR)            # félbemaradt kérés: nincs válasz
    assert recv_until_closed(s) == b""
    s.close()


def test_timeouts(port):
    s = connect(port)
    s.sendall(b"a=")                      # befejezetlen kérés
    t0 = time.monotonic()
    assert closed_within(s, FRAME + 1.0)
    assert time.monotonic() - t0 >= FRAME - 0.05
    s.close()

    s = connect(port)                     # tétlen kapcsolat
    t0 = time.monotonic()
    assert closed_within(s, IDLE + 1.0)
    assert time.monotonic() - t0 >= IDLE - 0.05
    s.close()

    s = connect(port)                     # csöpögtetés nem hosszabbít
    t0 = time.monotonic()
    s.sendall(b"a")
    closed = False
    for _ in range(40):
        time.sleep(0.05)
        try:
            s.sendall(b"a")
        except (BrokenPipeError, ConnectionResetError):
            closed = True
            break
    assert closed or closed_within(s, 0.5)
    assert time.monotonic() - t0 < FRAME + 1.0
    s.close()
```

```
def test_connection_limit(port):
    conns = [connect(port) for _ in range(LIMIT)]
    for c in conns:                          # mind a 64 kiszolgált
        c.sendall(b"a=1\n")
        assert recv_lines(c, 1) == b"OK records=1 sum=1 bytes=4\n"

    extra = connect(port)                    # a kernel elfogadja, mi nem
    extra.sendall(b"b=2\n")
    extra.settimeout(0.3)
    try:
        assert extra.recv(64) == b"", "a 65. kapcsolat nem kaphat választ"
    except (socket.timeout, TimeoutError):
        pass

    conns.pop().close()                      # egy rekesz felszabadul
    assert recv_lines(extra, 1) == b"OK records=1 sum=2 bytes=4\n"
    extra.close()
    for c in conns:
        c.close()


def test_backpressure(port):
    count = 50000
    s = connect(port)
    replies = []

    def reader():
        replies.append(recv_lines(s, count, timeout=10))

    t = threading.Thread(target=reader)
    t.start()
    s.sendall(b"".join(b"a=%d\n" % (i % 65536) for i in range(count)))
    t.join()
    expected = b"".join(b"OK records=1 sum=%d bytes=%d\n"
                        % (i % 65536, len(b"a=%d\n" % (i % 65536)))
                        for i in range(count))
    assert replies[0] == expected, "elveszett, duplikált vagy sérült válasz"
    s.close()


def test_graceful_shutdown(binary):
    proc, port = start(binary)
    done = connect(port)                     # kész válasz, majd tétlen
    done.sendall(b"a=1\n")
    assert recv_lines(done, 1) == b"OK records=1 sum=1 bytes=4\n"
    partial = connect(port)                  # félbemaradt kérés
    partial.sendall(b"b=")
    flood = connect(port)                    # küld, de nem olvas
    flood.setblocking(False)
    stop = threading.Event()
```

```
    def flooder():
        while not stop.is_set():
            try:
                flood.send(b"c=1\n" * 1024)
            except BlockingIOError:
                time.sleep(0.01)
            except OSError:
                return

    t = threading.Thread(target=flooder)
    t.start()
    time.sleep(0.3)

    t0 = time.monotonic()
    proc.send_signal(signal.SIGTERM)
    assert recv_until_closed(done) == b""        # féllezárás, nincs új adat
    assert recv_until_closed(partial) == b""     # el nem fogadott kérés
    rc = proc.wait(timeout=SHUTDOWN + 2)
    elapsed = time.monotonic() - t0
    stop.set()
    t.join()
    assert rc == 0, rc
    assert elapsed < SHUTDOWN + 1.0, elapsed     # a flood sem tartja fel
    for c in (done, partial, flood):
        c.close()
    try:
        connect(port).close()
        raise AssertionError("leállás után új kapcsolat")
    except ConnectionRefusedError:
        pass


def main():
    binary = sys.argv[1]
    proc, port = start(binary)
    try:
        for test in (test_pipelined_and_error, test_partial_input,
                     test_max_request_splits, test_eof,
                     test_timeouts, test_connection_limit, test_backpressure):
            test(port)
            print("ok", test.__name__)
    finally:
        proc.send_signal(signal.SIGTERM)
        rc = proc.wait(timeout=5)
    assert rc == 0, rc
    test_graceful_shutdown(binary)
    print("ok test_graceful_shutdown")


if __name__ == "__main__":
    main()
```

Buildjegyzőkönyv (`lab-gcc` profil). A `diag_server.c` GCC 13.3.0-val, glibc 2.39 mellett, Linux x86-64 (6.x kernel) környezetben `-Werror` beállítással, az alapértelmezett és a rövidített határidőkkel is figyelmeztetés nélkül fordult. A `tests/server_test.py` mind a nyolc tesztesete sikeres volt Python 3-mal, egyszer ASan/UBSan builddel (a szerver kilépési kódja ekkor is 0 volt, tehát a sanitizer sem jelzett), háromszor optimalizált builddel. A teszt nem garantálja a rövid `send()` és a spontán `EAGAIN` előfordulását; ezekhez a lent leírt syscall-helyettesítő réteg kell.

17.3.7 Tesztstratégia és gyakori hibák

Az **in-process parserteszt** determinisztikusan vizsgálhatja ugyanazon kérés minden kettévágását, a maximális frame-et, hibás prefixeket és a fogyasztási hosszakat. Fontos tulajdonság: időtúllépés nélkül, megfelelő kimenetürítés mellett a bemenet darabolása nem változtathatja meg a válaszok sorrendjét és tartalmát.

A **socket integration teszt** valódi loopback TCP kapcsolaton vizsgálja az acceptet, a half-close-t, az eseményérdeklődést, a lassú olvasót, a kapcsolatlimitet és a shutdown-határidőt; a projektben ez a `tests/server_test.py`. A port kiosztását a kernelre lehet bízni, majd `getsockname()`-mel lekérdezni; a szerver ezért írja ki a `LISTEN` sort.

A rövid `send()` és az `EAGAIN` nem garantálható pusztán kis üzenetekkel. Ezekhez determinisztikus syscall-helyettesítő tesztréteg is szükséges; a valódi TCP-teszt ezt kiegészíti, nem helyettesíti. A stale event szintén közvetlen tokeninjektálással ellenőrizhető.

Tipikus hibák:

- **Mindig figyelt `EPOLLOUT`:** üres kimenet mellett pörgő eseményciklust okozhat.
- **Parserpointer eltárolása:** a következő tömörítés után más adatra mutathat.
- **Csak FD alapján végzett dispatch:** új kapcsolatot érhet régi esemény.
- **`EAGAIN` hibaként kezelése:** normális backpressure miatt bont kapcsolatot.
- **Korlátlan eseménykezelő ciklus:** egy kliens kiszoríthatja az időzítőket és más kapcsolatokat.

17.3.8 Feladatok

17.3.8.1 Élettartam és memóriakorlát

A parser egy mezőre mutató pointert ad. Miért nem elegendő ezt a kimeneti sorban eltárolni? Mennyi a puszta input/output tárolókorlát 48 kapcsolat esetén?

17.3.8.2 Eseményérdeklődés megvalósítása

Írj függvényt, amely aktív kapcsolatnál csak függő kimenet esetén kér `EPOLLOUT` eseményt; egyébként olvasást és peer-féllezárást figyel.

17.3.8.3 Leállítási hiba diagnosztikája

A szerver shutdownkor bezárja a listenert, de a kapcsolatoknál továbbra is olvas és új válaszokat állít elő. Miért nem garantált a leállás? Milyen teszt bizonyítja a javítást?

17.3.9 Megoldások

17.3.9.1 Élettartam és memóriakorlát

A pointer nem tulajdonolja a mező bájtjait. A `memmove()` vagy egy új olvasás megváltoztathatja a hivatkozott tartalmat. A választ még a szinkron parserhívás alatt saját kimeneti tárolóba kell másolni.

$$48(4096 + 1024) = 245760 \text{ C bájt} = 240 \text{ KiB}.$$

Ellenőrzési feltétel: a már sorba állított válasz akkor sem változik, ha a bemeneti tömböt tesztben felülírjuk.

17.3.9.2 Eseményérdeklődés megvalósítása

Az alábbi részlet a szerver fordítási egységébe illeszthető:

```
#include "diag_tcp.h"
#include <stdint.h>
#include <sys/epoll.h>

static uint32_t active_interest(const struct diag_io *c)
{
    return c->off < c->end
         ? EPOLLOUT
         : EPOLLIN | EPOLLRDHUP;
}
```

Előfeltétele az aktív állapot; a shutdown állapotait külön kezeljük. Linuxon az `EPOLLERR` és `EPOLLHUP` külön kérés nélkül is jelentkezhet.

Ellenőrzési feltétel: a válasz utolsó bájtjának elküldése és a tárolt bemenet feldolgozása után az érdeklődés megfelel az új sorállapotnak.

17.3.9.3 Leállítási hiba diagnosztikája

A listener lezárása csak az új kapcsolatokat akadályozza meg. A meglévő kliensek végtelen munkát termelhetnek.

A javítás az új kérések elfogadásának leállítása, a már elfogadott válaszok ürítése, majd a féllezárás és a közös határidő. Tesztben egy kliens shutdown után is folyamatosan küld, és nem olvassa a válaszokat. A szervernek nem szabad új válaszokat előállítania; a határidő után minden kapcsolat lezárandó. A mérési ellenőrzéshez kis, dokumentált ütemezési tolerancia szükséges.

17.3.10 Összefoglaló

A helyes nonblocking adapter három határt tart fenn: tárolóhatárt, érvényességi határt és időhatárt. A korlátos válaszsor adja a backpressure-t, a szinkron parseradapter lezárja a borrowed nézetek használatát, a generációs token védi az újrahasznált kapcsolatrekeszeket. A graceful shutdown pedig nem puszta `close()`, hanem új munka tiltásából, válaszürítésből, féllezárásból és végső határidőből álló állapotátmenet. A „Teljes referencia-szerver” szakasz ezeket egyetlen futtatható programmá köti össze, és loopback TCP-teszttel ellenőrzi.

17.4 Záróreview: helyesség, diagnosztika és mérés

A záróreview nem egyszerűen annak ellenőrzése, hogy a szolgáltatás néhány helyes kérésre válaszol-e. Azt kell megmutatnia, hogy a könyvtári mag, a CLI és a TCP-adapter ugyanazt a dokumentált szerződést valósítja meg, erőforrásaikat hibautakon is rendezik, és viselkedésük reprodukálhatóan vizsgálható.

A Nonblocking TCP adapter és leállítás állapotgépét most nem tervezzük újra: az állapotátmenetekhez ellenőrizhető invariánsokat és teszteket rendelünk. A C minőségi kapuk és hibajavítási jegyzőkönyv módszeréből pedig kiadási bizonyítékcsomagot készítünk. Ez lesz a későbbi C++-port összehasonlítási alapja.

Az alábbi kód és parancsok laborreceptek; végrehajtott fordítást, sikeres tesztet vagy mért teljesítményt nem dokumentálnak.

17.4.1 Az audit tárgya és bizonyítékai

Az **auditálható szerződés** előfeltételek, megfigyelhető eredmények és megőrzendő invariánsok együttese. Például az „allokációs hibát kezelünk” önmagában nem szerződés. Pontosabb állítás:

> Ha a konstrukció bármely allokációja meghiúsul, a függvény hibát jelez, nem ad át részlegesen inicializált objektumot, a kimeneti paraméter változatlan marad, és minden addig megszerzett erőforrást felszabadít.

Az audit három eltérő bizonyítékot kapcsol össze:

- **Kódszintű indoklás:** mely invariánsokból következik a helyesség?
- **Tesztmegfigyelés:** milyen konkrét bemenet és hibainjektálás mellett mit figyeltünk meg?
- **Környezeti azonosítás:** pontosan melyik forrásból, fordítóval és futtatási környezetben keletkezett az eredmény?

Egyik sem helyettesíti a másikat. Egy sikeres sanitizer-futás nem bizonyítja az összes lehetséges végrehajtás helyességét; egy helyesnek tűnő cleanup-gráf pedig nem igazolja, hogy a kiadott bináris valóban abból a forrásból készült.

A példák választott, itt nem ellenőrzött referenciaprofilja: Linux hosted környezet, x86-64 System V ABI, LP64 adatmodell, nyolcbites C bájt; a `lab-next` profil (profiltábla). A tényleges laborban rögzítsük a kernel, a linker és az elemzőeszközök verzióját is. Más verzió használata nem önmagában hiba, hanem másik dokumentált profil.

A megadott anyag nem tartalmaz ellenőrzött külső forrásjegyzetet. A verziófüggő eszközviselkedést ezért a tényleges labor eszközdokumentációjával kell megerősíteni; az alábbi kapcsolók nem helyettesítik ezt az ellenőrzést.

17.4.2 Ownership és cleanup: minden erőforrásnak legyen története

Az **ownership**, vagyis tulajdonlás itt a lezárási felelősség egyértelmű hozzárendelése. A kölcsönzött hozzáférés nem hoz létre új tulajdonost, és nem hosszabbítja meg az erőforrás élettartamát.

Az audit első kézzelfogható eredménye egy kitöltött ownership-térkép:

Erőforrás	Tulajdonos	Átadás vagy élettartamhatár	Invalidálás	Hibautas kötelezettség
Figyelő socket	Szolgáltatásvezérlő	Sikeres létrehozástól lezárásig	Lezárás	Inicializálási hiba után is lezárni
Elfogadott socket	Kezdetben az elfogadó ág, majd a kapcsolatobjektum	Csak sikeres kapcsolat-inicializáláskor kerül át	Kapcsolat lezárása	Sikertelen átadáskor az elfogadó ág zárja
Bemeneti és kimeneti tároló	Kapcsolatobjektum	Sikeres foglalástól felszabadításig	Felszabadítás, esetleges újrafoglalás	Részleges inicializálásnál is rendezni
Parser bemeneti nézete	Nincs saját tulajdonlása	Legfeljebb az API-ban engedett időtartamig	Tömörítés, felülírás, újrafoglalás	Nem tárolható el engedély nélkül
Válaszhoz tartozó adatok	A konkrét válaszszerződés szerint	A küldés befejezéséig szükségesek	Korai felszabadítás vagy felülírás	Hiba esetén is pontosan egyszer rendezni

Különösen veszélyes a feldolgozott bemenetre mutató válasz. Ha a parser kölcsönzött szeletet ad vissza, a TCP-adapter pedig később küldi el, akkor a bemeneti buffer tömörítése már a küldés előtt invalidálhatja a szeletet. Javításként vagy saját válaszpéldányt készítünk, vagy az eredeti tároló élettartamát és változatlanságát biztosítjuk a küldés végéig.

A kapcsolatállapot egyik alapinvariánsa:

$$0 \le \text{sent} \le \text{out_len} \le \text{out_cap}.$$

Az elküldendő tartomány $[\text{sent}, \text{out_len})$. Ez nem a teljes kapacitás, és nem nullterminált sztringszerződés.

Bizonyításvázlat. Kezdetben `sent == out_len == 0`. A válasz előállítása csak elegendő kapacitás mellett növelheti az `out_len` értékét. Pozitív, k hosszúságú írás után kizárólag akkor növeljük `sent` értékét k-val, ha $k \le \text{out_len} - \text{sent}$. Átmeneti írhatatlanság nem módosítja a számlálókat. A buffer újrafelhasználása csak a küldés befejezésekor vagy a kapcsolat megszüntetésekor engedett.

A cleanup-ágakat megszerzési sorrendben járjuk végig: mi történik a socket létrehozása, a kapcsolatobjektum foglalása, majd valamelyik buffer foglalása után bekövetkező hibánál? A már lezárt descriptor tulajdonosi mezője kapjon érvénytelen őrértéket, például `-1`-et.

Linux-specifikus megjegyzés: a `close()` hibájára nem szabad automatikus újrapróbálkozó ciklust építeni. A numerikus descriptor időközben új erőforráshoz tartozhat; a tényleges célrendszer lezárási szabályait külön kell ellenőrizni.

17.4.3 Dinamikus diagnosztika és hibainjektálás

A sanitizer, a hibainjektálás és a fuzzolás különböző kérdésekre válaszol.

GCC/Clang eszközprofil: az AddressSanitizer, röviden ASan, számos tiltott memória-hozzáférést felismerhet. Az UndefinedBehaviorSanitizer, röviden UBSan, többek között bizonyos túlcsordulási és igazítási hibákat jelezhet. Nem teljes UB-detektorok. A szabályos tartományon belüli, de szemantikailag hibás válasz egyiknek sem feltétlenül feltűnő. A leak-ellenőrzés támogatását és bekapcsolását külön rögzítsük.

A **hibainjektálás** egy hibaforrás irányított helyettesítése. Allokációnál például a kijelölt sorszámú foglalás tér vissza sikertelenül. A számlálás minden teszteset elején újraindul, és csak a vizsgált komponens foglalásaira vonatkozik.

17.4.3.1 Kidolgozott cleanup-próba

Az alábbi önálló `audit_cleanup.c` nem a projekt parserét helyettesíti: a kétlépcsős konstrukció auditmintáját mutatja. Minimum ISO C17; a fenti hosted GCC/Clang-profilhoz használható, platform-API nélkül.

Az `allocator` tesztkontextusnak az utolsó felszabadításig élnie kell. A `result` tulajdonolja a `data` tárolót; a hívó tulajdonolja a sikeresen visszaadott `result` objektumot. A `create_result()` előfeltétele érvényes `a` és `out`, továbbá `*out == NULL`. A hibakimenet `-1`, változatlan kimeneti pointerrel.

```
#include <stddef.h>
#include <stdlib.h>
#include <string.h>

struct allocator {
    size_t calls;
    size_t fail_at;
    size_t live;
};

struct result {
    unsigned char *data;
    size_t size;
};

static void check(int condition)
{
    if (!condition)
        abort();
}

static void *take(struct allocator *a, size_t n)
{
    ++a->calls;
    if (a->calls == a->fail_at)
        return NULL;

    void *p = malloc(n);
```

```
    if (p != NULL)
        ++a->live;
    return p;
}

static void release(struct allocator *a, void *p)
{
    if (p != NULL) {
        check(a->live != 0);
        --a->live;
        free(p);
    }
}

static int create_result(struct allocator *a, struct result **out)
{
    struct result *r = take(a, sizeof *r);
    if (r == NULL)
        return -1;

    r->data = take(a, 32);
    if (r->data == NULL) {
        release(a, r);
        return -1;
    }

    r->size = 32;
    memset(r->data, 0, r->size);
    *out = r;
    return 0;
}

static void destroy_result(struct allocator *a, struct result **out)
{
    struct result *r = *out;
    if (r != NULL) {
        release(a, r->data);
        release(a, r);
        *out = NULL;
    }
}

int main(void)
{
    for (size_t fail = 1; fail <= 3; ++fail) {
        struct allocator a = {0, fail, 0};
        struct result *r = NULL;
        int rc = create_result(&a, &r);

        if (fail <= 2) {
            check(rc == -1 && r == NULL);
        } else {
```

```
            check(rc == 0 && r != NULL);
            check(r->size == 32);
        }

        destroy_result(&a, &r);
        check(r == NULL && a.live == 0);
    }
    return 0;
}
```

A `data` elemeire mutató bármely kölcsönzött pointer a `destroy_result()` során invalidálódik. A tesztkontextus nem általános allokátorkönyvtár: egyszálú, rövid, korlátos tesztre készült.

Fordítás külön diagnosztikai binárisokba:

```
gcc -std=c17 -O1 -g -Wall -Wextra -Wpedantic \
  -fsanitize=address,undefined -fno-omit-frame-pointer \
  -fno-sanitize-recover=undefined \
  audit_cleanup.c -o audit-cleanup-gcc

if command -v clang >/dev/null 2>&1; then
  clang -std=c17 -O1 -g -Wall -Wextra -Wpedantic \
    -fsanitize=address,undefined -fno-omit-frame-pointer \
    -fno-sanitize-recover=undefined \
    audit_cleanup.c -o audit-cleanup-clang
else echo "SKIP: a clang nincs telepítve (a lab-clang profil opcionális, nem
 ↪  része az alaprecepnek)"; fi
```

A várt eredmény az első két esetben konstrukciós hiba, a harmadikban siker; minden eset végén nulla élő foglalás. A sikeres eset feltétele, hogy a valódi `malloc()` hívások sikerüljenek. A tesztellenőrzések szándékosan nem `assert` alapúak: `NDEBUG` mellett sem tűnnek el.

A projektben ugyanígy járjuk be a konstrukció, a buffernövelés és a válaszelőállítás foglalási pontjait. A sikeres út foglalásszáma csak kiindulópont: a hibakezelés maga is tartalmazhat új foglalást.

17.4.3.2 Parser-fuzzolás értelmes orákulummal

A **fuzz-orákulum** az a szabály, amely eldönti, hogy egy végrehajtás hibás-e. A „nem omlott össze” szükséges, de gyenge orákulum.

A parserhez célszerű azonos bemenetet kétféleképpen beadni: egyetlen darabban és több részletben. Előfeltétel, hogy ugyanaz a parserkonfiguráció, ugyanaz az EOF-esemény és elegendő kimeneti kapacitás álljon rendelkezésre. Az összehasonlítás a végső üzenetsorozatra és hibakategóriára vonatkozik, nem az egyes `feed` hívások köztes státuszára.

A harness ellenőrizze továbbá, hogy:

- az elfogyasztott hossz nem nagyobb az átadott hossznál;
- a feldolgozás halad, vagy egyértelműen további bemenetet kér;
- a maximális kerethossz túllépése dokumentált hibát ad;
- csonka keret és tiszta EOF megkülönböztethető;
- minden iteráció saját állapotot hoz létre és teljesen lebontja.

Clang/LibFuzzer eszközprofilban az adapter belépési pontja `LLVMFuzzerTestOneInput`; a `-fsanitize=fuzzer,address,undefined` kapcsolóval készített célprogram nem tartalmaz saját `main()` függvényt. A fuzzer bemenete kölcsönzött: a projekt nem szabadíthatja fel és nem őrizheti meg a híváson túl.

17.4.4 Részleges I/O és leállítás integrációs tesztje

POSIX socket-szerződés: a TCP bájtfolyam, ezért a küldési hívások határai nem képeznek üzenethatárokat. Az olvashatósági vagy írhatósági jelzés sem ígéri a teljes protokollüzenet feldolgozhatóságát.

A determinisztikus tesztben ezért helyettesíthető I/O-réteg adja elő a részleges eredményeket; a valódi socketes teszt külön ellenőrzi az integrációt. A socketbufferek csökkentése önmagában nem garantál egy meghatározott rövid írást.

Teszthelyzet	Elvárt ellenőrzés
Egy keret több bemeneti részletben	Pontosan egy feldolgozott kérés
Több keret egy olvasásban	Minden teljes keret feldolgozódik, helyes sorrendben
Rövid írás, majd `EAGAIN`	A küldési pozíció megmarad, nincs duplikáció
`EINTR` adatátvitel nélkül	A számlálók nem változnak
Peer féllezárása	A teljes kérések és a csonka maradék külön kezelése
Nem olvasó kliens	Korlátos kimeneti memória és érvényes leállítási határidő
Peer eltűnése küldés közben	Kapcsolati hiba, nem kontrollálatlan folyamatleállás

Kidolgozott példa: egy hét C bájtos válasznál a helyettesített író előbb `3`, majd `EAGAIN`, végül `4` eredményt ad. A `sent` sorozata:

$$0 \to 3 \to 3 \to 7.$$

A második pozitív írásnak a negyedik bájttól kell indulnia. Az első címtől történő újraküldés tartalmilag hibás, még akkor is, ha egyetlen memóriaellenőrző sem jelez.

A leállítás tesztelése előtt rögzítsük, mit jelent a **befogadott kérés**. Egy használható auditpolicy szerint ez olyan teljes, érvényes kérés, amelyhez az adapter már biztosította a válaszhoz szükséges helyet. A stop esemény észlelése után nincs új befogadás; a már sorban álló válaszok a határidőig ürülhetnek, majd minden kapcsolat lezárul. A félkész bemenet nem válik utólag befogadott kéréssé.

A teszt egy befogadott és egy csonka kéréssel indít leállítást. Nem rögzített idejű alvásokkal találgat: megfigyelhető állapotokra és határidős várakozásokra támaszkodik. Külön eset vizsgálja a válaszokat nem olvasó klienst. A határidő számítása monoton órát használjon; a leállítás észlelési késleltetését külön korlátozzuk.

17.4.5 Bináris, függőségek és buildprofil

A review tárgya a kiadott bináris is. Az auditcsomagban szerepeljen a forrás commitazonosítója és módosítottsága, a teljes fordítási és linkelési parancs, a fordítóverzió, a célarchitektúra, valamint az artifact ellenőrző összege.

Tartsunk külön:

- warningokkal épülő GCC- és Clang-profilt;
- ASan/UBSan diagnosztikai profilt;
- optimalizált, sanitizer nélküli release-profilt.

GNU binutils eszközprofil: ha az auditált artifact neve `build/release/diag`, az alábbi parancsok futtatás nélkül vizsgálják a binárist:

```
readelf -hW build/release/diag
readelf -dW build/release/diag
readelf --dyn-syms -W build/release/diag
nm -u build/release/diag
```

Az ELF-fejlécből a formátumot és géptípust, a dinamikus szakaszból többek között a deklarált közvetlen függőségeket vizsgáljuk. A feloldatlan dinamikus szimbólum önmagában nem linkelési hiba: annak futásidejű feloldása lehet szándékos.

A `DT_NEEDED` lista nem teljes futásidejű függőségtérkép: tranzitív és dinamikusan betöltött komponensek is létezhetnek. Ellenőrizzük, hogy a release-be nem került sanitizer-runtime, és a megosztott könyvtár exportlistáján nem jelentek meg véletlenül belső segédfüggvények.

17.4.6 Memória-, allokáció- és syscall-mérés

Mérés előtt rögzítsük a kérdést. Más kísérlet kell a kapcsolatmemória felső korlátjához, és más az egy kérésre jutó rendszerhívások számához.

A jegyzőkönyv tartalmazza a CPU-t, a kernelt, a libc-t, a konténer- és cgroup-korlátokat, a terhelésgenerátor helyét, a kéréseloszlást, a párhuzamos kapcsolatok számát, a bemelegítést és az ismétlésszámot. A localhostteszt elsősorban helyi adapterköltséget mutat, nem távoli hálózati teljesítményt.

Mennyiség	Lehetséges megfigyelés	Korlát
Élő kért heapméret	Saját allokátorhatár számlálói	Nem tartalmazza az allokátor teljes overheadjét
RSS/PSS	Linux `/proc` memóriaadatok	Nem azonos a heapfoglalással
Allokációhívások	`malloc`/`calloc`/`realloc` körüli számlálás	Csak a lefedett foglalási útvonalakat látja
Rendszerhívásszám	Linux `strace` összesítés	A tracing módosítja a futást
Késleltetés és throughput	Külön, tracing nélküli mérés	Terheléstől és ütemezéstől függ

Ha a kapcsolatobjektum külön foglalt, és két külön foglalt buffert tulajdonol, akkor a közvetlenül kért tároló:

$$B_{\text{conn}} = \text{sizeof}(\texttt{struct connection}) + C_{\text{in}} + C_{\text{out}}.$$

Itt minden tag C bájtban értendő. Beágyazott tömböt nem adunk hozzá még egyszer. A képlet nem tartalmaz allokátormetadata-, stack- vagy kernel-socketbuffer-költséget, ezért nem RSS-becslés.

Egy rögzített mérési intervallumban, $N > 0$ sikeresen kiszolgált kérésre:

$$a = \frac{A}{N}, \qquad s = \frac{S}{N},$$

ahol A az intervallumban számlált allokációhívások, S a kiválasztott rendszerhívások száma. Rögzíteni kell, hogy a kapcsolatlétrehozás benne van-e az intervallumban, és mely syscallok kerültek S-be.

Ne vonjuk össze a sanitizeres, tracinges és tiszta release-mérést. Mindegyik más kérdésre válaszol. A felszabadítás után magas maradó RSS sem bizonyít automatikusan leaket: az allokátor megtarthat felszabadult tárolót. Az ownership-számlálás és az OS-szintű memóriaadat együtt értelmezendő.

17.4.7 A C++-port funkcionális alapvonala

A későbbi összehasonlítás feltétele a **megfigyelhető viselkedési szerződés** befagyasztása. Ez nem azonos belső adatszerkezeteket követel, hanem azonos külső eredményt.

Rögzítsük a protokoll pontos bájtsorozatait, keret- és kapcsolatszámkorlátait, a hibakategóriákat, a válaszok sorrendjét, az EOF és a leállítás jelentését. A CLI-nél ide tartoznak a kilépési kódok és a standard kimenetre kerülő géppel feldolgozható adatok is.

A közös tesztcsomag tartalmazzon bemenet–eredmény párokat, darabolási menetrendeket és hibainjektálási forgatókönyveket. Időbélyeges diagnosztikai sorok helyett strukturált eseményeket hasonlítsunk össze, ha a szöveges forma nem része a szerződésnek.

A C++-változatnak nem kell ugyanannyi `malloc()` hívást vagy ugyanazokat a belső szimbólumokat produkálnia. Az allokáció, memória és syscallszám összehasonlítási mutató; a funkcionális megfelelés külön kapu.

17.4.8 Feladatok

17.4.8.1 Ownership-hiány felismerése

Az elfogadó ág létrehoz egy socketet, majd kapcsolatobjektumot foglal. Sikertelen foglaláskor hibát naplóz és visszatér. Kié a socket, és mely ellenőrzés hiányzik?

17.4.8.2 Hibainjektálási próba bővítése

Egészítsd ki a `result` típust egy második, külön foglalt tárolóval. A konstrukció most három allokációt végezzen. Ellenőrizd mindhárom meghibásodási pontot és a sikeres esetet.

17.4.8.3 Részleges írás diagnosztikája

A helyettesített író `3`, `EAGAIN`, `4` eredményt ad. A kliens hét helyett tíz bájtot kap, mert az első három bájt megismétlődik. Miért lehet tiszta az ASan-futás, és milyen állapotot kell naplózni?

17.4.8.4 Mérési állítás értékelése

A C-programot sanitizerrel, a C++-változatot optimalizált release-ben mérték. A C++-változat kisebb RSS-t mutatott. Levonható-e következtetés a nyelvek memóriahatékonyságáról?

17.4.9 Megoldások

17.4.9.1 Ownership-hiány felismerése

A sikeres átadásig az elfogadó ág a tulajdonos. Sikertelen kapcsolatfoglaláskor neki kell lezárnia a socketet. Ellenőrzési feltétel: ismételt injektált hibák után a folyamat nyitott descriptorainak száma nem növekszik tartósan, és ugyanazt a descriptort nem zárják kétszer.

17.4.9.2 Hibainjektálási próba bővítése

A harmadik foglalás hibájánál fel kell szabadítani a második lépésben létrehozott tárolót és a `result` objektumot. A destruktor mindkét tárolót rendezze. A `fail_at` értékei legyenek `1`, `2`, `3`, majd `4`; az utolsó nem talál foglalási pontot. Minden eset után `live == 0`, hibánál pedig a kimeneti pointer változatlanul null marad.

17.4.9.3 Részleges írás diagnosztikája

A hiba protokollszintű duplikáció, nem feltétlenül tiltott memória-hozzáférés. Naplózandó a küldés előtti `sent`, a kért hossz és a visszatérési eredmény. A harmadik hívás kezdőpozíciója `3`, kért hossza `4` legyen. Az orákulum a teljes fogadott bájtsorozatot is ellenőrizze.

17.4.9.4 Mérési állítás értékelése

Nem. A sanitizer saját memóriaköltsége és az eltérő optimalizálási profil összekeveredik az implementáció különbségével. Azonos terheléssel, összevethető release-profilban, több ismétléssel kell újramérni. Az RSS mellé kértbájt- és allokációszámlálás is szükséges.

17.4.10 Összefoglaló

A záróreview eredménye nem egyetlen „sikeres” jelzés, hanem visszakövethető bizonyítékcsomag: ownership-térkép, invariánsok, hibautas tesztek, parser-fuzzolás, I/O- és shutdown-vizsgálat, binárisaudit és dokumentált mérés. A C++-port akkor hasonlítható tisztességesen a C-változathoz, ha előbb ugyanazt a funkcionális szerződést teljesíti, és csak ezután értékeljük az eltérő erőforrás- és teljesítményköltségeket.

18 Modern C, C23 és a C/C++ határ

18.1 C23 nyelvi újdonságok és inkompatibilis változások

A C23-ra váltás nem egyszerűen új fordítókapcsoló bekapcsolása. Egyes korábbi fordítóbővítések szabványos nyelvi alakot kaptak, új kulcsszavak jelentek meg, néhány régi konstrukció pedig megváltozott vagy megszűnt. A migráció célja ezért nem az, hogy „leforduljon”, hanem hogy a program helyességi szerződése megmaradjon.

A Szabványok, dialektusok és hordozhatósági szerződés alapján külön kezeljük az ISO C17-et, az ISO C23-at és a fordítóbővítéseket. A változtatásokat a Záróreview: helyesség, diagnosztika és mérés módszerével kell ellenőrizni: rögzített környezetben, pozitív és negatív fordítási próbákkal.

18.1.1 Támogatásvizsgálat: nyelvi mód és tényleges képesség

A `__STDC_VERSION__` C23-hoz tartozó értéke `202311L`. Ez a választott nyelvi verzió jelzése, **nem minden képesség hibátlan megvalósításának bizonyítéka**. Egy régebbi fordító `-std=c2x` módja pedig nem automatikusan azonos a végleges C23-támogatással.

Az alábbiakhoz javasolt, itt nem futtatott vizsgálati profil: Linux, x86-64 System V ABI, LP64, a `lab-next` profil (profiltábla), újabb kiadással. Ez vizsgálati célkörnyezet, nem teljes támogatási ígéret. A tényleges fordítókiadást, sysrootot és headereket külön rögzíteni kell.

A következő C23-kódrészleteket **képességenként külön** illeszd ebbe a teljes próbaprogramba:

```
/* probe.c: a behelyettesített részletek minimuma ISO C23. */
int main(void)
{
    /* Egy vizsgált részlet helye. */
    return 0;
}
```

Fordítási próbák:

```
gcc -std=c23 -Wall -Wextra -pedantic-errors -Werror=attributes \
    -fsyntax-only probe.c
clang -std=c23 -Wall -Wextra -pedantic-errors -Werror=attributes \
    -fsyntax-only probe.c
```

A C17-alternatívákat ugyanezekkel a kapcsolókkal, de `-std=c17` módban vizsgáld. A részletekhez alapvetően nem kell libc-header; ahol mégis, azt külön jelezzük. A `-fsyntax-only` nyelvi és headerpróba, nem linkelési vagy futási teszt.

18.1.2 Logikai értékek és null pointer

ISO C23: a `bool`, `true` és `false` nyelvi elemek; használatukhoz nem szükséges `<stdbool.h>`. A `true` és `false` típusa `bool`.

```
/* C23-részlet; két külön képességpróbára is bontandó. */
bool enabled = true;
int *p = nullptr;
(void)enabled;
(void)p;
```

C17-alternatíva: `<stdbool.h>` mellett ugyanaz a logikaiérték-szintaxis használható, de makrókon keresztül: a `bool` a `_Bool` típushoz kapcsolódik, a `true` és `false` pedig egész konstansokra helyettesítődik. Header nélkül `_Bool enabled = 1;` írható.

A C23 `nullptr` előre definiált nullpointer-konstans, saját típussal. A típus `nullptr_t` neve a `<stddef.h>` headerből érhető el; ehhez külön headerpróba kell: a header beillesztése után fordítsd a `nullptr_t n = nullptr;` deklarációt. **C17-alternatíva:** például `int *p = 0;`.

A `nullptr` nem jelent foglalt tárolót, és nem teszi típusbiztonságossá a variadikus hívásokat. Ha egy ilyen interfész `void *` argumentumot vár, add át a megfelelő pointertípust, például `(void *)nullptr`.

Pointerhelyzet	Tulajdonos és élettartam	Érvénytelenítés, hibakimenet
`p = nullptr`	Nincs hivatkozott objektum vagy birtokolt tároló	Dereferálása nem definiált viselkedés
Dinamikus buffer pointere	A felszabadításért felelős programrész a tulajdonos	`free` és sikeres `realloc` érvényteleníti a régi hozzáféréseket

Migrációs törés lehet egy korábbi `int nullptr;` deklaráció is: C17-ben az azonosító megengedett, C23-ban már foglalt nyelvi elem.

18.1.3 Típuslevezetés és típuslekérdezés

ISO C23: az `auto` inicializálóból történő típuslevezetést (*type inference*) tesz lehetővé. Az egyszerű hordozható használat egyetlen deklarált objektumot és inicializálót tartalmaz.

```
/* C23-részlet. */
auto retries = 3u;
(void)retries;
```

A `retries` típusa `unsigned int`; ez fordításidejű típusválasztás, nem dinamikus típusosság. **C17-alternatíva:** `unsigned int retries = 3u;`. C17-ben az `auto` csak tárolásiosztály-jelölő, például `auto int retries = 3;`.

A `typeof` egy kifejezés vagy típusnév típusát adja meg, a `typeof_unqual` pedig annak minősítők nélküli, nem atomikus változatát:

```c
/* C23-részlet; a két typeof-alak külön is próbálandó. */
const int source = 4;
typeof(source) preserved = 5;
typeof_unqual(source) writable = 6;
writable = 7;
(void)preserved;
(void)writable;
```

A `preserved` típusa `const int`, a `writable` típusa `int`. **C17-alternatíva:** ezek explicit kiírása; általános, szabványos `typeof` operátor nincs.

A GNU-dialektusban korábban elfogadott `typeof` vagy `__auto_type` nem bizonyít ISO C17-támogatást. A fenti szigorú fordítási próba éppen ezt választja el a szabványos C23-használattól. A `typeof` operandusáról változó hosszúságú tömbtípusok környezetében nem szabad általánosan feltételezni, hogy biztosan nem értékelődik ki.

18.1.4 `constexpr`: C-ben objektum, nem C++-szerű függvény

ISO C23: a `constexpr` konstans objektum definiálására használható; nem vezet be C++-szerű `constexpr` függvényeket.

```c
/* C23-részlet. */
constexpr int capacity = 8;
int slots[capacity];
_Static_assert(capacity == 8, "unexpected capacity");
(void)slots;
```

Ebben az egész típusú példában a `capacity` konstans kifejezésben használható, ezért a `slots` nem változó hosszúságú tömb. A `_Static_assert` szándékosan a C17-ben is létező alak: így a próba a `constexpr` képességet vizsgálja.

C17-alternatíva: `enum { capacity = 8 };` vagy makró. A `const int capacity = 8;` C17-ben önmagában nem teszi a nevet egész konstans kifejezésben használhatóvá.

Összehasonlításként ez önálló **ISO C++20** program, könyvtári függőség nélkül:

```cpp
// constexpr_demo.cpp
constexpr int twice(int value)
{
    return 2 * value;
}

static_assert(twice(4) == 8);

int main()
{
    return 0;
}
```

```
g++ -std=c++20 -Wall -Wextra -pedantic-errors \
    -fsyntax-only constexpr_demo.cpp
clang++ -std=c++20 -Wall -Wextra -pedantic-errors \
    -fsyntax-only constexpr_demo.cpp
```

A függvényes alak C23-ba másolva nem válik érvényes C-vé. Az azonos kulcsszó eltérő nyelvi képességet jelöl.

18.1.5 Egész típusok, enumok és literálok

ISO C23: a `_BitInt(N)` bitszélességet megadó egész típus. Előjeles alakban $N \geq 2$, előjel nélkül $N \geq 1$. Az előjeles alak szélességébe az előjelbit is beleszámít. A támogatott felső korlátot a `<limits.h>` `BITINT_MAXWIDTH` makrójával, majd konkrét fordítási próbával kell ellenőrizni.

```c
/* C23-részlet; a típust a literáloktól külön vizsgálja. */
_BitInt(10) signed_value = 300;
unsigned _BitInt(9) unsigned_value = 300;
(void)signed_value;
(void)unsigned_value;
```

Az előjeles, N bites tartomány:

$$-2^{N-1} \leq x \leq 2^{N-1} - 1.$$

Ezért 300 előjeles tárolásához 10 bit kell: $2^8 - 1 = 255 < 300$, míg $2^9 - 1 = 511 \geq 300$. Előjel nélkül 9 bit elegendő.

Bizonyításvázlat: kettes komplemensben az előjelbit súlya -2^{N-1}, az alsó bitek maximális összege $2^{N-1} - 1$. Ebből adódnak a határok.

C17-alternatíva: megfelelő tartományú hagyományos egész típus, szükség esetén több szóból felépített reprezentáció. A `uintN_t` típusok csak elérhetőségük ellenőrzése után használhatók, és nem helyettesítenek tetszőleges bitszélességet.

A kapcsolódó C23-újdonságok:

Képesség és külön fordítandó próba	C17-alternatíva
`int x = 0b1010;` – bináris literál	`int x = 0xA;`
`int x = 1'024;` – számjegyelválasztó	`int x = 1024;`
`auto x = 300wb;` – előjeles `_BitInt` literál	Megfelelő hagyományos típus és literál
`auto x = 300uwb;` – előjel nélküli változat	Megfelelő előjel nélküli típus
`enum opcode : unsigned char { OP_DATA = 1 };`	Hagyományos enum, vagy `unsigned char` typedef és külön konstansok

A literálpróbák végére kerüljön `(void)x;`. A `wb` és `uwb` a szükséges legkisebb megfelelő bitszélességet választja.

A rögzített mögöttes típusú enum (*fixed underlying enum*) értékeinek reprezentálhatónak kell lenniük a megadott típusban. Az `unsigned char` azonban nem általánosan „8 bites”: ezt `CHAR_BIT` határozza meg. A bitszélességből a `_BitInt` tárolóméretre, paddingje és ABI-beli átadása sem következik; ezek külön implementációs vizsgálatok.

18.1.6 Attribútumok és `static_assert`

ISO C23: szabványos attribútumalak például a `[[maybe_unused]]` és a `[[nodiscard]]`.

```
/* C23-részlet. */
[[maybe_unused]] int reserved = 0;
[[nodiscard]] int check_configuration(void);
static_assert(sizeof(char) == 1);
```

A `nodiscard` az eredmény figyelmen kívül hagyásának diagnosztizálását támogatja, nem futásidejű ellenőrzés. A `static_assert` kulcsszó, és az üzenet elhagyható.

C17-alternatíva: `_Static_assert(feltetel, "uzenet");`; a `static_assert` név `<assert.h>` mellett makróként használható, két argumentummal. A bemutatott attribútumoknak nincs általános ISO C17-megfelelőjük: a fordítóspecifikus helyettesítés külön bővítésszerződés.

Attribútumonként ellenőrizd a `__has_c_attribute` operátor elérhetőségét és például a `__has_c_attribute(nodiscard)` eredményét. Az elfogadott szintaxis önmagában kevés: az ismeretlen attribútum figyelmen kívül hagyható. Ezért kell a figyelmeztetést hibává emelő próba; a `static_assert` alakjait pedig az attribútumoktól külön fordítsd.

18.1.7 Inkompatibilis változások és migrációs hibák

Függvénydeklarációk. C17-ben az `int ping();` nem közli a paraméterek számát és típusát. C23-ban nulla paramétert jelent, mint az `int ping(void);`. A régi K&R-függvénydefiníciók megszűntek. Mindkét verzióhoz jó migrációs cél az explicit prototípus.

Negatív próbaként a `ping(1)` hívásnak C23-ban diagnosztikát kell kiváltania. C17-beli elfogadása nem bizonyít helyes hívást. Az implicit `int` és az előzetes deklaráció nélküli függvényhívás már C17-ben sem megengedett; ezek nem új C23-szigorítások.

Kettes komplemens. C23 előírja az előjeles egész típusok kettes komplemens reprezentációját. C17 több reprezentációt megengedett. Ettől az előjeles túlcsordulás továbbra is UB; a migráció nem jogosít fel körbeforduló előjeles aritmetikára. Ezt fordítási siker vagy próbaértékek nem bizonyítják biztonságosnak.

Nulla méretű `realloc`. A `realloc(p, 0)` C17-ben implementáció által meghatározott kezelésű; C23-ban nem definiált viselkedés. Nem helyes futtatással „felderíteni”, hogyan működik.

A mindkét verzióban megfelelő javítás: ha a kért méret nulla, `free(p)` után állítsd a tulajdonosi pointert nullára. Egyébként ideiglenes pointerrel hívd a `realloc`-ot. Nem nulla méretű sikertelen hívásnál a régi foglalás megmarad; siker esetén a korábbi kölcsönzött hozzáférések érvényüket vesztik, akkor is, ha a cím numerikusan változatlan. A pontos minta a realloc és dinamikusan növekvő buffer részben szerepel.

18.1.8 Feladatok

1. Egy `typeof`-ot használó fájl lefordul `-std=gnu17` módban. Mit bizonyít ez, és milyen további próbák szükségesek?
2. Írj C23-ban fordításidejű, 16 értékű `capacity` objektumot és vele rögzített méretű tömböt. Add meg a C17-alternatívát.
3. Auditálj egy C17-modult, amely `int flush();` deklarációt, K&R-definíciót és `realloc(buffer, requested)` hívást használ. A `requested` lehet nulla. Milyen javítások és ellenőrzések kellenek?

18.1.9 Megoldások

1. Csak az adott GNU-dialektus támogatása bizonyított. Külön próbáld `-std=c17 -pedantic-errors` és `-std=c23 -pedantic-errors` mellett, mindkét rögzített fordítóval. Az ellenőrzési feltétel a dialektusfüggő és a szabványos elfogadás elkülönítése.
2. C23-ban `constexpr int capacity = 16; int data[capacity];`. C17-ben `enum { capacity = 16 }; int data[capacity];`. Adj hozzá `_Static_assert(capacity == 16, "capacity");` ellenőrzést. Mindkét változatnak a saját nyelvi módjában kell fordulnia; C17-ben egy puszta `const int` nem egyenértékű helyettesítés.
3. Írj `int flush(void);` deklarációt, és alakítsd a K&R-definíciót típusos paraméterlistájúvá. A nulla méretet külön felszabadítási ág kezelje. Ellenőrizd a nulla, a nem nulla sikeres és az injektált allokációs hibás esetet; továbbá azt, hogy az érvénytelenített kölcsönzött pointereket senki sem használja. A hibás nulla méretű hívást ne futtasd C23-as tesztként.

18.1.10 Összefoglaló

A C23-váltás képességenkénti döntés. Minden új konstrukcióhoz tartozzon C17-alternatíva, explicit nyelvi mód és elkülönített támogatáspróba. Különösen vizsgáld az új kulcsszavakkal ütköző neveket, a prototípus nélküli deklarációkat és a nulla méretű `realloc`-ot. A modernebb szintaxis nem helyettesíti az ownership-, élettartam- és túlcsordulási szerződést.

18.2 C23 könyvtár és preprocessor

A C23 könyvtári újdonságai több, rendszerprogramozásban visszatérő művelethez adnak szabványos eszközt: ellenőrzött egész aritmetikához, bitvizsgálathoz és érzékeny adatok törléséhez. Használatuk azonban nem következik pusztán a C23 nyelvi mód bekapcsolásából. A preprocessort, a fordítót, a headereket és a linkeléskor, illetve futáskor használt könyvtárat külön kell vizsgálni.

A C23 nyelvi újdonságok és inkompatibilis változások rész verziózási alapjaira építünk; a ritkább numerikus szolgáltatások előzménye a Numerikus és ritkább C könyvtári eszközök térképe. Itt a döntési szabály a fontos: **képességet ellenőrizzünk, ne támogatást feltételezzünk egy verziószámból.**

18.2.1 Nyelvi mód, header és könyvtári képesség

Az **ISO C23** nyelvi verziót a `__STDC_VERSION__` makró `202311L` értéke jelzi. Ez nem szolgáltatásonkénti megfelelőségi teszt. Korábbi fordítókiadások részleges C23-támogatása más verzióértékkel vagy `-std=c2x` kapcsolóval is megjelenhet; ezeket külön profilként kell kezelni.

A képességvizsgálat (*feature detection*) három eltérő kérdést választ szét:

Vizsgálat	Mit igazol?	Mit nem igazol?
Nyelvi fordítási próba	A fordító elfogadja a használt konstrukciót	Egy könyvtári függvény elérhetőségét
Header és konkrét API fordítása	A deklaráció vagy makró használható a megadott típusokkal	Külső függvényszimbólum linkelhetőségét

Vizsgálat	Mit igazol?	Mit nem igazol?
Linkelési, szükség esetén célgépi próba	A kiválasztott könyvtári környezetben használható a szolgáltatás	Más telepítési környezet kompatibilitását

A headert akár a fordítócsomag is szállíthatja, nem feltétlenül a libc. Ez különösen fontos a fordítói beépített műveletekre épülő headereknél.

A példák választott **laborprofilja** Linux, x86-64 System V ABI, LP64, `CHAR_BIT == 8`, a `labnext` profil (profiltábla), újabb kiadással. Ez vizsgálati célprofil, nem közölt fordítási eredmény. A parancsokban szereplő `gcc` a rögzített kiadást jelentse; Clanggal ugyanazokat a próbákat külön kell elvégezni.

18.2.2 Ellenőrzött aritmetika és bitműveletek

Az **ISO C23** `<stdckdint.h>` headerének `ckd_add`, `ckd_sub` és `ckd_mul` makrói ellenőrzött egész aritmetikát végeznek. A matematikai eredmény ábrázolhatóságát a célobjektum típusában vizsgálják; igaz visszatérési érték jelzi az ábrázolhatatlanságot. Ez nem egyszerűen az operandusok szokásos közös típusában végzett művelet utólagos ellenőrzése.

Az alábbi példa minden operandushoz és célobjektumhoz `size_t` típust használ. Hiba esetén a belső eredményt nem teszi közzé.

Az **ISO C23** `<stdbit.h>` unsigned egész értékek bitmintázatának vizsgálatát támogatja. Például:

- `stdc_count_ones_ui(x)`: az egyes bitek száma egy `unsigned int` értékben;
- `stdc_bit_width_ui(x)`: a reprezentáláshoz szükséges értékbitek száma; nullára nulla;
- `stdc_has_single_bit_ui(x)`: pontosan egy bit van-e beállítva, vagyis az érték pozitív kettőhatvány-e.

Ezek értékműveletek, nem memóriába írt bájtsorrendek vizsgálatai. Nem helyettesítik a hálózati szerializálást.

18.2.2.1 Kidolgozott példa C17 fallbackkal

Egy keret tárolóigénye legyen

$$B = nw + h,$$

ahol n az elemszám, w az elemméret C bájtban, h pedig a fejléc mérete. Mindhárom nemnegatív, `size_t` érték.

A következő teljes program alapértelmezésben **ISO C17**; a két konfigurációs kapcsoló külön engedélyezi a C23-szolgáltatásokat.

```
/* capacity.c */
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdlib.h>

#ifndef HAVE_C23_CHECKED
#define HAVE_C23_CHECKED 0
#endif
#ifndef HAVE_C23_BITS
#define HAVE_C23_BITS 0
#endif

#if HAVE_C23_CHECKED
#include <stdckdint.h>
#endif
#if HAVE_C23_BITS
#include <stdbit.h>
#endif

static bool frame_size(size_t n, size_t w, size_t h, size_t *out)
{
    size_t product;
    size_t total;

#if HAVE_C23_CHECKED
    if (ckd_mul(&product, n, w) ||
        ckd_add(&total, product, h)) {
        return false;
    }
#else
    if (w != 0 && n > SIZE_MAX / w) {
        return false;
    }
    product = n * w;
    if (h > SIZE_MAX - product) {
        return false;
    }
    total = product + h;
#endif

    *out = total;
    return true;
}

static unsigned int ones(unsigned int value)
{
#if HAVE_C23_BITS
    return stdc_count_ones_ui(value);
#else
    unsigned int count = 0;
    while (value != 0u) {
```

```
        value &= value - 1u;
        ++count;
    }
    return count;
#endif
}

int main(void)
{
    size_t bytes = 0;

    if (!frame_size(4, 24, 16, &bytes) || bytes != 112)
        return EXIT_FAILURE;

    if (frame_size(SIZE_MAX, 2, 0, &bytes) || bytes != 112)
        return EXIT_FAILURE;

    if (ones(0xA4u) != 3u)
        return EXIT_FAILURE;

    return EXIT_SUCCESS;
}
```

Az `out` előfeltétele: nem null pointer, és élő, írható `size_t` objektumra mutat; az objektumhoz nincs szinkronizálatlan konkurens hozzáférés.

Hozzáférés	Tulajdonos és élettartam	Érvénytelenítés, hibakimenet
`out`	A hívó birtokolja; a függvény csak a hívás alatt kölcsönzi	Hiba esetén változatlan; a pointert a függvény nem őrzi meg
`&product`, `&total`	A `frame_size` lokális objektumai	Élettartamuk a függvényből kilépéskor véget ér

Bizonyításvázlat. Legyen $M = \texttt{SIZE_MAX}$. Ha $w > 0$, akkor

$$nw \le M \iff n \le \left\lfloor \frac{M}{w} \right\rfloor .$$

Ezért a szorzás előtt osztással vizsgálható az ábrázolhatóság. Ha $w = 0$, a szorzat nulla. A már ellenőrzött $p = nw$ után

$$p + h \le M \iff h \le M - p.$$

A kivonás biztonságos, mert $p \le M$. A fallback tehát nem előbb számol túlcsorduló eredményt, hanem a művelet előtt ellenőriz.

A bitfallback minden iterációban törli a legalsó beállított bitet: a `value - 1u` az addigi legalsó egyest nullává, az alatta levő nullákat egyessé változtatja; az ÉS-művelet csak azt az egyest törli.

Fordítás és a két képesség külön próbája:

```
gcc -std=c17 -Wall -Wextra -Wpedantic capacity.c -o capacity-c17

gcc -std=c23 -Wall -Wextra -Wpedantic \
    -DHAVE_C23_CHECKED=1 capacity.c -o capacity-checked

gcc -std=c23 -Wall -Wextra -Wpedantic \
    -DHAVE_C23_BITS=1 capacity.c -o capacity-bits
```

A várt sikeres futási eredmény nulla kilépési státusz; ez nem mért eredmény. A konfiguráció csak sikeres próbák után állítsa egyre a megfelelő kapcsolókat. A két képességet ne egyetlen „van C23” kapcsoló helyettesítse.

18.2.3 Érzékeny adatok törlése és sztringmásolás

Az **ISO C23** `<string.h>` headerében megjelenő `memset_explicit` rendeltetése, hogy a törlő írás ne maradjon el pusztán azért, mert az objektumot később már nem olvassák. A közönséges `memset` ilyen körülmények között eltávolítható optimalizáláskor.

A szerződés továbbra is érvényes, írható, megfelelő méretű tárolót követel. A titok tulajdonosa töröljön **felszabadítás előtt**, miután megszűnt minden párhuzamos hozzáférés. A művelet nem törli automatikusan a korábbi másolatokat, regiszterben maradt értékeket, core dumpot vagy swapot.

C17 fallback: nincs azonos garanciájú, általánosan rendelkezésre álló ISO C17-függvény. Használható dokumentált platformprimitív, külön képességpróbával. Ha az alkalmazás biztonsági szerződése megköveteli a törlést, megfelelő primitív hiányában a konfiguráció hibázzon. A sima `memset` és egy ellenőrizetlen „volatile trükk” nem tekinthető automatikusan egyenértékű helyettesítésnek.

A `strdup` és `strndup` szintén **ISO C23** könyvtári függvény; korábban **POSIX** környezetben voltak szabványosak, ISO C17-ben nem.

A `strdup` nullterminált bemenetet vár. A `strndup(s, n)` legfeljebb az első n karaktert másolja, korábbi nullkarakternél megáll, és siker esetén terminátort ad az eredményhez. A bemenetnek a vizsgált tartományban olvashatónak kell lennie; a korlát nem tesz érvényessé egy dangling pointert.

Mindkettő új allokációt ad vissza: siker esetén a hívó tulajdona, `free` érvényteleníti; allokációs hibánál null pointert kapunk. C17-ben saját nevű wrapper készíthető: korlátozott hosszkeresés, az `len + 1` méret ellenőrzése, `malloc`, másolás, terminálás. Ne deklaráljuk kézzel a hiányzó szabványos nevet.

18.2.4 Preprocessor: jelenlét helyett használhatóság

Az **ISO C23** `__has_include` azt vizsgálja, hogy egy header az adott keresési környezetben bevonható-e. Nem igazolja sem az API teljességét, sem a külső függvények linkelhetőségét. Régebbi preprocesszorokhoz először magának az operátornak az elérhetőségét is vizsgálni kell; a konfigurált fordítási próba gyakran egyszerűbb.

További C23-eszközök:

- A `#embed` fordításkor külső erőforrásból állít elő inicializálásban használható adatot. A build függőségei közé az erőforrásfájlt is fel kell venni; ez nem futásidejű fájlolvasás. A `limit(...)` korlátozhatja a bevont adatmennyiséget, a `__has_embed` előzetes vizsgálatot támogat.

- A `#elifdef` és `#elifndef` a megfelelő `#elif defined(...)`, illetve negált alak rövidítése.
- A `#warning` szabványosított diagnosztikai direktíva; a build warning-policyja határozza meg, hogy megállítja-e a fordítást.
- A `__VA_OPT__` variadikus makróban csak akkor illeszti be argumentumát, ha a variadikus rész makróhelyettesítés után nem üres.

C23-as, önállóan nem teljes programrészlet:

```
#define VALUES(...) { 0 __VA_OPT__(,) __VA_ARGS__ }

static const int first[] = VALUES();
static const int second[] = VALUES(4, 9);
```

A két inicializáló rendre `{ 0 }` és `{ 0, 4, 9 }`. C17 fallback lehet két külön makrófelület. A `#embed` helyett előállított C-inicializáló használható, reprodukálható generálási szabállyal. A C17-es ágból a C23-direktívákat célszerű külön fájlba választani; a preprocesszor új képességeinek felismerését ne ugyanarra a képességre építsük.

18.2.5 Ritkább kiterjesztések és tipikus tévedések

A C23 könyvtári térképen külön vizsgálandók a következők:

Terület	Példa	Ellenőrzési szempont
Numerika	`roundeven`, további matematikai műveletek	Deklaráció, linkelés, szükség esetén `libm`
Feltételes lebegőpontos képességek	Decimális lebegőpontos típusok és kapcsolódó támogatás	Fordítói és könyvtári képesség külön
Karakterkonverzió	`<uchar.h>`, `mbrtoc8`, `c8rtomb`	Locale- és konverziósállapot-szerződés
Méretezett felszabadítás	`free_sized`, `free_aligned_sized`	A méret és igazítás pontos előfeltétele

A „külön vizsgálandó” nem jelenti, hogy minden felsorolt szolgáltatás szabvány szerint opcionális. A feltételes szabványos képességet és a hiányos implementációt meg kell különböztetni.

Gyakori hiba a header jelenlétét teljes támogatásnak tekinteni, a túlcsordulást a művelet után ellenőrizni, vagy biztonsági garanciát gyengébb fallbackkal észrevétlenül lecserélni. Ezeket külön konfigurációs kapcsoló, előzetes aritmetikai ellenőrzés és dokumentált hibapolitika javítja.

18.2.6 Feladatok

1. Miért nem elegendő a `-std=c23` és a `<string.h>` sikeres bevonása a `memset_explicit` használatához?
2. Bővítsd a program ellenőrzéseit a `frame_size(SIZE_MAX, 0, 7, ...)` és a `frame_size(1, 1, SIZE_MAX, ...)` esetekkel.
3. Egy migrációban a hiányzó `memset_explicit` helyére `memset` kerül, a `strndup` támogatását pedig csak headerellenőrzéssel döntik el. Diagnosztizáld és javítsd a konfigurációt.

18.2.7 Megoldások

18.2.7.1 A támogatási rétegek szétválasztása

A nyelvi mód nem biztosít libc-szimbólumot, a `<string.h>` pedig C17-ben is létezik. A konkrét függvény deklarációját fordítással, szimbólumát linkeléssel kell ellenőrizni; dinamikus linkelésnél a célkörnyezet is számít. Ellenőrzési feltétel: a próba valóban hivatkozzon a függvényre, és ezt az optimalizáló ne tüntesse el.

18.2.7.2 Határértékteszt kódolása

A következő C17-kompatibilis részlet a korábbi `main` függvénybe illeszthető, ugyanazokkal a buildparancsokkal:

```
if (!frame_size(SIZE_MAX, 0, 7, &bytes) || bytes != 7)
    return EXIT_FAILURE;

if (frame_size(1, 1, SIZE_MAX, &bytes) || bytes != 7)
    return EXIT_FAILURE;
```

Az első esetben a szorzat nulla, tehát az eredmény hét. A másodikban $1 + \texttt{SIZE_MAX}$ nem ábrázolható. Az ellenőrzés a hibajelzés mellett a változatlan kimeneti objektumot is vizsgálja.

18.2.7.3 Biztonsági és konfigurációs hiba

A `memset` nem őrzi meg az explicit törlés garanciáját. Dokumentált platformfallback vagy konfigurációs hiba szükséges. A `strndup` számára konkrét fordítási és linkelési próba kell; C17/POSIX profilban az előírt feature-test makrókat a headerek előtt kell beállítani. Ellenőrzési feltétel: minden engedélyezett és fallback ág külön buildben, azonos alkalmazásszintű szerződéssel működjön.

18.2.8 Összefoglaló

A C23-eszközök választását műveleti szerződés vezesse: ellenőrzött aritmetikához `stdckdint.h`, bitvizsgálathoz `stdbit.h`, érzékeny törléshez megfelelő garanciájú primitív kell. A compiler, a headerek és a libc képessége külön bizonyítandó. A jó C17 fallback a viselkedést őrzi meg; ahol ez nem biztosítható, a hiányt látható hibává teszi.

18.3 C és C++: azonos szintaxis, eltérő szemantika

A C++ fordító nem egyszerűen több hibát kereső C fordító. A két nyelvnek külön típus-, objektum- és élettartam-szabályai vannak: ugyanaz a forrás lehet mindkettőben elfogadható, mégis eltérő jelentésű. A sikeres fordítás ezért szükséges, de nem elégséges kompatibilitási ellenőrzés.

Az összehasonlítás alapja itt **ISO C17 és ISO C++20**, nem valamely fordító bővítményekkel kiegészített alapértelmezett módja. A C23 könyvtár és preprocessor részben tárgyalt képességellenőrzés továbbra is szükséges; a C23 támogatását nem feltételezzük. Különösen fontos előzmény az effective type és aliasing, valamint a C inline-szabályrendszere.

18.3.1 Külön fordítási mód, külön ellenőrzés

A labor választott célprofilja Linux, x86-64 System V ABI, LP64; a `lab-next` profil (profiltábla), mindkét fordítóhoz libstdc++ 14.2.0. Ez reprodukciós célprofil, nem itt végrehajtott mérés vagy igazolt telepítés. A példák nem támaszkodnak konkrét objektumelrendezésre.

A következő két teljes program fordítási parancsa:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -pedantic-errors c_alloc.c -o c_alloc
g++ -std=c++20 -Wall -Wextra -Wpedantic -pedantic-errors cpp_alloc.cpp -o
 ↪  cpp_alloc
```

A második ellenőrzéshez használjuk ugyanezeket a kapcsolókat `clang`, illetve `clang++` mellett. A további kódok jelölt részletek; önálló fájlba és szükség szerint függvénytörzsbe helyezve vizsgálhatók.

18.3.2 `void*`: a konverzió nem objektumkezelés

ISO C17: a `void*` implicit módon konvertálható objektumtípusra mutató pointerré. **ISO C++20:** ez a visszairányú implicit konverzió nem megengedett. Objektumpointerből `void*` viszont mindkét nyelvben képezhető, a típusminősítők korlátozásainak megtartásával.

A teljes C17-program:

```c
/* c_alloc.c */
#include <stdlib.h>

int main(void)
{
    int *p = malloc(sizeof *p);
    if (p == NULL) {
        return 1;
    }

    *p = 42;
    int result = (*p == 42) ? 0 : 2;
    free(p);
    return result;
}
```

A megfelelő C++20-program:

```cpp
// cpp_alloc.cpp
#include <cstdlib>

int main()
{
    int *p = static_cast<int *>(std::malloc(sizeof(int)));
    if (p == nullptr) {
        return 1;
    }

    *p = 42;
    int result = (*p == 42) ? 0 : 2;
```

```
    std::free(p);
    return result;
}
```

A cast C++-ban a pointerkonverziót teszi explicitté; **nem inicializál és nem hív konstruktort**. C++20-ban a `malloc` az implicit élettartam-kezdés szabályai alapján létrehozhatja az itt szükséges `int` objektumot. Ebből nem következik, hogy tetszőleges osztálypéldány ugyanígy kezelhető.

Helyességi bizonyításvázlat. Sikertelen allokáció után nincs dereferencia. Siker esetén a méret és az igazítás elegendő egy `int` számára. Az értékadás megelőzi az olvasást; C-ben ez az allokált objektum effective type-ját is meghatározza az érintett hozzáférésekhez. A felszabadítás pontosan egyszer történik.

Szerződési elem	Mindkét programban
Tulajdonos	A `main`, amely `p` segítségével kezeli az allokációt
Tároló	A sikeres `malloc` által biztosított terület
Kölcsönzött hozzáférés	Nem kerül kiadásra
Érvénytelenítés	`free`, illetve `std::free`
Hibakimenet	Allokációs hibánál `1`; siker esetén várt visszatérés `0`

18.3.3 Sztringliterál: eltérő típus, azonos módosítási tilalom

A közönséges `"ok"` literál C17-ben `char[3]`, C++20-ban `const char[3]` típusú. A három elem a két karakterből és a lezáró nullkarakterből adódik.

Az alábbi deklarációk függvénytörzsbe illeszthető részletek:

```
/* C17: elfogadható deklaráció. */
char *s = "ok";

/* Hibás használat: végrehajtása UB. */
/* s[0] = 'O'; */

// C++20: szándékosan hibás deklaráció, diagnosztikát igényel.
char *s = "ok";
```

C-ben tehát a nem `const` elemtípus **nem jelent módosítási engedélyt**. Mindkét nyelvben megfelelő:

```
/* C17 és C++20 alatt is használható részlet. */
const char *view = "ok";
char text[] = "ok";
text[0] = 'O';
```

A `view` nem tulajdonló hozzáférés statikus tárolási idejű literálhoz; a literál tárolója a program végéig él. A `text` ezzel szemben saját, módosítható tömbobjektum. Függvényblokkon belül deklarálva élettartama a blokk elhagyásakor véget ér; az elemeire korábban képzett pointerek ezután dangling hozzáférést jelentenének.

18.3.4 `const`, `inline` és névterek

A kapcsolás (*linkage*) azt szabályozza, hogy külön deklarációk ugyanazt az entitást jelölhetik-e. Ez nem azonos a hatókörrel.

Deklaráció és környezet	C17	C++20
`const int limit = 64;`, fájl-, illetve névtérszinten, előzmény nélkül	Külső kapcsolás	Alapesetben belső kapcsolás
`static inline int f(void) { return 1; }`	Fordítási egységenként külön függvény	Fordítási egységenként külön függvény
Külső kapcsolású `inline` függvény	Sajátos inline/külső definíciós szabályok	Több fordítási egységben definiálható az ODR feltételeivel

A C++-os `const` sorban fontos az „alapesetben”: például az `extern`, az `inline`, illetve egy korábbi deklaráció módosíthatja a kapcsolási helyzetet. Ha egy header egyszerűen `const int limit = 64;` definíciót tartalmaz, több C fordítási egységben többszörös külső definíció keletkezhet; C++-ban rendszerint külön objektumok jönnek létre.

C17-ben a kizárólag `inline`, de nem `extern` fájlszintű deklarációkkal rendelkező függvény inline definíciója nem biztosít külső definíciót. Ha a függvényt használjuk, arról külön gondoskodni kell. A C++ többdefiníciós lehetőségét ezért nem szabad C-re átvinni. Az `inline` egyik nyelvben sem garantál gépi kódba történő beillesztést.

C-nek vannak külön azonosító-névterei, például a struktúratagek számára, de nincs C++-szerű, felhasználó által megnyitható névtere:

```
// C++20-részlet; C17-ben nincs megfelelő namespace szintaxis.
namespace diagnostics {
    int status = 0;
}
```

18.3.5 Union, aliasing és az objektum létrejötte

ISO C17: union másik tagján keresztül olvasva a tárolt reprezentáció újraértelmezése történik. Ez nem általános, hordozható számkonverzió: nem meghatározott reprezentációs bitek és trap reprezentáció is problémát okozhatnak.

ISO C++20: inaktív uniontag értékének olvasása általában UB. Léteznek szűk kivételek, például megfelelő standard-layout struktúrák közös kezdősorozatához kapcsolódóan.

```
/* Összehasonlító részlet; nem hordozható konverziós recept. */
union sample {
    unsigned int bits;
    float value;
};

union sample s = { .bits = 0 };
float x = s.value;
```

C-ben a unionra vonatkozó reprezentációs szabályokat kell alkalmazni; C++20-ban az utolsó sor inaktív tagot olvas. Attól, hogy mindkét fordító elfogadja a szintaxist, a művelet még nem lesz mindkét nyelvben helyes.

A C effective type szabályait sem lehet változtatás nélkül C++-ra alkalmazni. Ott az objektumtípus és az objektum-élettartam szabályait kell követni. Egy pointercast önmagában egyik nyelvben sem igazolja az aliasing helyességét.

A különbség konstruktor mellett különösen látható:

```
// C++20-részlet, függvénytörzsben.
struct Counter {
    int value;
    Counter() : value(7) {}
};

Counter *p = new Counter;
int result = p->value;
delete p;
```

A `new Counter` tárolót foglal és konstruktort hív; a `delete` lezárja az objektum élettartamát és felszabadítja a tárolót. A tulajdonos itt a befoglaló függvény, kölcsönzött pointer nem kerül kiadásra. A szokásos dobó `new` allokációs hibája kivétel, nem nullpointer.

Ha ezt `malloc` és cast helyettesítené, a konstruktor nem futna le. A C++20 implicit objektumlétrehozási szabálya erre a konstruktoros típusra nem ad ilyen helyettesítést. C-ben a memóriafoglalás és az értékek beállítása explicit programlépések; C++-ban az inicializálás konstruktorhívást is magában foglalhat.

18.3.6 Designated initializer és compound literal

A kijelölt inicializáló (*designated initializer*) mindkét választott szabványban létezik, de eltérő korlátozásokkal:

```
/* C17-részlet: a deklarációs sorrend felcserélhető. */
struct Pair { int first; int second; };
struct Pair p = { .second = 2, .first = 1 };

// C++20-részlet: aggregátum, deklarációs sorrendben.
struct Pair { int first; int second; };
Pair p{ .first = 1, .second = 2 };
```

C++20-ban a designatoroknak a tagok deklarációs sorrendjét kell követniük. A C tömbindex-designatorai és beágyazott kijelölései sem emelhetők át általánosan C++20-ba.

A C compound literal kifejezéssel névtelen objektum hozható létre:

```
/* C17-részlet; consume deklarációja előzőleg látható. */
consume((struct Pair){ .first = 1, .second = 2 });
```

Ez nem ISO C++20-szintaxis. Érték szerint fogadó függvénynél megfelelő C++-alak lehet:

```
// C++20-részlet.
consume(Pair{1, 2});
```

Pointert megőrző interfésznél azonban ez nem mechanikus helyettesítés. A blokkban létrehozott C compound literal objektum élettartama a befoglaló blokkhoz kötődik; a bemutatott C++ ideiglenes objektumé a hívást tartalmazó teljes kifejezés végén lezárul.

18.3.7 Gyakori hibák

A legveszélyesebb javítás a „tegyünk castot oda, ahol a C++ fordító panaszkodik”. Ez elrejtheti a valódi objektumélettartam-problémát. Hasonlóan hibás a bővítményként elfogadott compound literalt szabványos C++-nak tekinteni, vagy egyetlen fordítási egység alapján következtetni a header kapcsolási helyességére. A negatív fordítási teszt, a többfájlos linkteszt és a szemantikai ellenőrzés külön feladat.

18.3.8 Feladatok

1. **Magyarázat:** miért helyes deklaráció C17-ben a `char *p = "abc";`, és miért helytelen mégis a `p[0] = 'A';`? Mi változik C++20-ban?
2. **Kódolás:** írj érték szerint visszatérő `make_pair` függvényt külön C17 és C++20 nyelven. Az eredmény tagjai legyenek `1` és `2`; ne használj dinamikus memóriát.
3. **Diagnosztika:** mentsd a fordított sorrendű C designated initializer példáját `probe.c` fájlba. Ellenőrizd mindkét nyelven, majd jósolj meg egy olyan hibát, amelyet ez a vizsgálat nem talál meg.

```
gcc -x c -std=c17 -Wall -Wextra -pedantic-errors -fsyntax-only probe.c
g++ -x c++ -std=c++20 -Wall -Wextra -pedantic-errors -fsyntax-only probe.c
```

18.3.9 Megoldások

18.3.9.1 Sztring és módosíthatóság

C17-ben a literál elemtípusa `char`, így a pointer inicializálása megengedett, de a literál módosítását külön szabály tiltja: végrehajtása UB. C++20-ban már a deklarációhoz is diagnosztika szükséges a `const` elvesztése miatt. Módosítható tárolóhoz mindkét nyelvben `char p[] = "abc";` megfelelő. Az ellenőrzési feltétel a saját tömb és a literál tárolójának megkülönböztetése, nem egy UB-program megfigyelt kimenete.

18.3.9.2 Érték szerint visszatérő függvény

Önálló fordítási egységként, a fenti módokban `-c` kapcsolóval ellenőrizhető:

```
/* C17 */
struct Pair { int first; int second; };

struct Pair make_pair(void)
{
    return (struct Pair){ .first = 1, .second = 2 };
}
```

```cpp
// C++20
struct Pair { int first; int second; };

Pair make_pair()
{
    return Pair{ .first = 1, .second = 2 };
}
```

Mindkét függvény értéket ad vissza, nem megszűnő lokális objektum pointerét. A hívó ellenőrizze mindkét tag értékét.

18.3.9.3 Fordítás és szemantika

A C17-vizsgálat várt eredménye elfogadás; C++20-ban a kijelölések sorrendje diagnosztikát igényel. A konkrét üzenet fordítófüggő.

A `-fsyntax-only` nem végez linkelést, ezért például a több fordítási egységben definiált, külső kapcsolású C `const` objektum problémáját nem mutatja meg. Az elfogadott uniontag-olvasás helyességét sem bizonyítja.

18.3.10 Összefoglaló

A C és a C++ közötti átjárásnál külön kell ellenőrizni a konverziót, kapcsolást, inicializálást, aliasingot és élettartamot. A két nyelvi módban történő fordítás hasznos első szűrő; a helyességhez azonban a kiválasztott nyelv saját szemantikai szabályai szükségesek.

18.4 C hordozhatósági audit és freestanding kitekintés

Egy C program akkor hordozható, ha minden feltételezése, amelyet a nyelvről, a platformról vagy a buildről tesz, ki van mondva, címkével el van látva és ellenőrizhető. Ez az egység azt a munkafolyamatot adja, amellyel ezt a listát összeállítod, és megmutatja, hol ér véget a hosted C: a freestanding tartományban. A cél nem az, hogy „mindenhol fusson”, hanem hogy pontosan tudd, *hol* fut, *miért*, és mi történik, ha a feltétel nem teljesül.

18.4.1 A hordozhatósági audit mint szerződés

A **hordozhatósági audit** verziózott dokumentum, amely a forráskódon kívül eső tulajdonságokra épülő állításokat gyűjti össze, mindegyiket címkével, ellenőrzési móddal és fallbackkel. Nem dokumentáció a dokumentáció kedvéért: a sorok egy része gépi ellenőrzéssé alakítható (`#if`, `_Static_assert`, configure-próba), a többi pedig a labornaplóba kerül. Az audit akkor jó, ha egy új targetre való portoláskor a kérdés nem az, hogy „mi romolhat el”, hanem hogy melyik sorok nem teljesülnek.

A címkézés a legfontosabb fegyelem, mert a négy kategória jogi ereje eltérő:

- **ISO** – a nyelvi szabály minden megfelelő implementáción kötelező, tehát ez a legerősebb állítás.
- **implementation-defined** – a szabvány választási lehetőséget ad, de az implementációnak dokumentálnia kell a választását.

- **POSIX / Linux / ABI** – a nyelv fölötti réteg; nem következik az ISO szabályból, és nem is minden Linuxon azonos (libc, kernel, architektúra).
- **compiler extension / build konfiguráció** – csak a rögzített compiler- és flag-profillal igaz.
- **mérés** – amit a laborban megfigyeltünk; ez illusztráció, nem garancia.

Az audittábla kötelező mezői: feltételezés, címke, ellenőrzés, fallback. Egy részleges példa:

Feltételezés	Címke	Ellenőrzés	Fallback / teendő
`CHAR_BIT == 8`	ISO (`limits.h`)	`#if CHAR_BIT != 8` → `#error`	bájt-alapú protokoll újraértelmezése, vagy a target kizárása
`uint32_t` létezik	ISO, opcionális (`stdint.h`)	`#if !defined(UINT32_MAX)`	`uint_least32_t`, méretfüggetlen kód
`sizeof(void*) == sizeof(long)`	ABI (LP64)	`sizeof` mérés a laborban	`uintptr_t`, vagy pointer→egész konverzió elkerülése
`_Alignof(T)` ismert	ISO	`_Alignof`, `offsetof`	explicit padding, byte-wise szerializálás
little-endian tárolás	ABI/platform	`__BYTE_ORDER__` vagy futásidejű próba	bájtonkénti írás/olvasás
IEC 60559 `float`/`double`	ISO, opcionális (Annex F)	`__STDC_IEC_559__` (C17)	fixpontos vagy saját formátum
`epoll` elérhető	Linux	`#ifdef __linux__` + header	`poll` adapter
`__builtin_bswap16`	compiler extension	`__has_builtin`	shift-alapú megvalósítás

18.4.2 Nyelvi feltételezések: C bájt, egészek, pointermodell

A `sizeof` egysége a **C bájt**, amelynek bitszáma `CHAR_BIT`; a 8 bites oktett feltételezés, nem nyelvi tény. Mivel az `unsigned char` nem tartalmaz kitöltő bitet, `UCHAR_MAX == 2^CHAR_BIT - 1` mindig teljesül, így a `CHAR_BIT` ellenőrzése preprocesszorral elvégezhető. Ha `CHAR_BIT != 8`, akkor a bájt-alapú hálózati protokoll, a `uint8_t` és a POSIX `read()`/`write()` oktett-szemantikája egyszerre esik ki: a POSIX oktettben számol, a C bájtban, és a kettő csak 8 bites bájtnál esik egybe. Ezért a hosted POSIX laborokban a 8 bitet a laborprofil rögzíti, nem a kód feltételezi.

Az **exact-width** típusok (`int8_t`, `uint32_t`, …) opcionálisak: csak akkor léteznek, ha van pontosan olyan szélességű, kitöltő bit nélküli típus. Kötelező viszont a `int_leastN_t`, `int_fastN_t` és `intmax_t` család, a `intptr_t`/`uintptr_t` pedig szintén opcionális. A helyes ellenőrzés nem a header létezésére, hanem a típusnév létezésére kérdez: `#if defined(UINT8_MAX)`. A `sizeof(int) == 4` állítás se nem ISO, se nem ABI-következmény — ez tipikusan mérés, amit a labornaplóban kell rögzíteni.

A **pointeradatmodell** (LP64, ILP32, LLP64) ABI-szintű fogalom. A `sizeof(void*) == sizeof(long)` állítás Linux x86-64-en igaz, Windowson nem; a pointer egésszé alakítása csak `uintptr_t`-vel és akkor is implementation-defined. A portolható kód nem a pointer értékére, hanem a pointer *aránya*/sorrendjére épít, és ahol lehet, elkerüli a konverziót.

18.4.3 Platform- és numerikus feltételezések: igazítás, endian, lebegőpont

Az **igazítás** lekérdezhető (`_Alignof`, `offsetof`, C23-ban `alignof`), de a `struct` layout soha nem szerializációs formátum: két architektúrán ugyanaz a struktúra más offsetekkel és paddinggel rendelkezhet, amit a korábbi layout-labor Layout- és protokolllabor két architektúrán (9.5) már rögzített. A címigazítás levezetése: legyen $a > 0$ az igazítás, $n \geq 0$; a legkisebb a-val osztható $m \geq n$

$$m = \left\lfloor \frac{n + a - 1}{a} \right\rfloor a,$$

és ha $a = 2^k$, akkor $m = (n + 2^k - 1)$ & $\sim (2^k - 1)$. Az előfeltétel, hogy $n + a - 1$ ne csorduljon túl: `if (n > SIZE_MAX - (a - 1))` hibakezelés kell, különben a „round-up” csendben nullára teker.

Endianra az ISO C-nek nincs makrója. A `__BYTE_ORDER__` GCC/Clang kiterjesztés, tehát az `#if` csak a rögzített compiler-profillal érvényes; hordozható megoldás a bájtonkénti (de) szerializálás. A `union`-on keresztüli type punning C-ben implementation-defined, C++-ban nem definiált viselkedés — a két nyelv eltérését a C és C++: azonos szintaxis, eltérő szemantika (18.3) rész tárgyalta, itt csak a következmény számít: az auditban ez nem lehet „ISO” sor.

Lebegőpontnál az IEC 60559 (Annex F) csak akkor áll, ha az implementáció definiálja a megfelelő makrót. C17-ben ez jellemzően `__STDC_IEC_559__`; a C23 ezt átdolgozta (a bináris és decimális formátumok külön makrót kaptak). A pontos makrónév és jelentés verziófüggő, ezért a konkrét draft és a tényleges compiler dokumentációja ellenőrizendő — ebben az egységben erre nem hivatkozom igazolt forrásként, és a labornaplóban kell rögzíteni.

18.4.4 Feature-test és fallback policy

A policy egyetlen helyen, egy `portability.h`-ban élő `#if` létra: minden feltételezés ott dől el, és a nem támogatott kombináció **fordítási hiba**, nem hajnali 3-kor jelentkező futásidejű meglepetés. A vizsgálandó makrók: `__STDC_VERSION__`, `__STDC_HOSTED__`, `__STDC_NO_THREADS__`, `__STDC_NO_ATOMICS__`, `__STDC_NO_VLA__`, valamint `__has_include` és `__has_builtin`. Utóbbiak C23-ban szabványosak, korábbi nyelvekben compiler-kiterjesztések, ezért `defined(...)` őr mögé kerülnek.

```
/* portability.h - illusztráció, a laborban fordítandó, nem lefuttatott példa
 ↪  */
#ifndef PORTABILITY_H
#define PORTABILITY_H

#include <limits.h>
#include <stddef.h>
#include <stdint.h>

#ifdef __cplusplus
#  define PT_STATIC_ASSERT(cond, msg) static_assert(cond, msg)
#else
#  define PT_STATIC_ASSERT(cond, msg) _Static_assert(cond, msg)
#endif

#if CHAR_BIT != 8
```

```
#  error "Ez a fordítási profil 8 bites C bajtra epul (CHAR_BIT == 8)."
#endif

#if !defined(UINT8_MAX)
#  error "Nincs pontosan 8 bites unsigned egesz tipus ezen az implementacion."
#endif

PT_STATIC_ASSERT(sizeof(uint32_t) == 4, "uint32_t nem 4 C bajt");

static uint16_t pt_bswap16_portable(uint16_t v)
{
    return (uint16_t)((uint16_t)(v >> 8) | (uint16_t)(v << 8));
}

static uint16_t pt_bswap16(uint16_t v)
{
#if defined(__has_builtin)
#  if __has_builtin(__builtin_bswap16)
    return __builtin_bswap16(v);
#  else
    return pt_bswap16_portable(v);
#  endif
#else
    return pt_bswap16_portable(v);
#endif
}

#endif /* PORTABILITY_H */
```

A buildparancs a rögzített profillal (a verziókat a labornapló tartalmazza):

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -c portability.c -o portability.o
g++ -std=c++20 -Wall -Wextra -Wpedantic -x c++ -c portability.c -o
 ↪  portability_cpp.o
```

A második sor csak azt mutatja, hogy a header mindkét nyelvben fordul; a két nyelv nem közös nyelv, a `PT_STATIC_ASSERT` éppen a szintaktikai különbséget izolálja.

18.4.5 Freestanding: startup, MMIO, könyvtári korlátok

Freestanding az az implementációs mód, ahol `__STDC_HOSTED__ == 0`. A szabvány itt csak egy **kötelező minimumot** ír elő: a program indulása és leállása (a belépési pont neve, típusa, a leállás módja) implementation-defined, és a könyvtárból csak néhány header tartalma garantált. Ez alsó korlát, nem tilalom: a freestanding definíció nem mondja, hogy nincs operációs rendszer vagy további könyvtár. Freestanding módban fordul például a Linux-kernel (OS-en belül fut, `kmalloc`-kal és `printk`-val), és freestanding egy olyan embedded toolchain is, amely saját `malloc`-ot vagy minimál `printf`-et nyújt — ezek a platform szolgáltatásai, nem a szabvány garanciái, tehát az auditban platformtényként kell címkézni őket.

A kötelező freestanding headerek C17-ben: `float.h`, `iso646.h`, `limits.h`, `stdalign.h`, `stdarg.h`, `stdbool.h`, `stddef.h`, `stdint.h`, `stdnoreturn.h`. A C23 (N3096 munkatervezet, 4. szakasz) ezt a listát **bővíti**: a felsoroltak mind megmaradnak, új a `stdbit.h`, és a freestanding implementációnak a `string.h` nagy részét (a `strdup`, `strndup`, `strcoll`, `strxfrm`, `strerror` kivételével), valamint a

`stdlib.h`-ból a `memalignment` függvényt is biztosítania kell. A C23 egyes neveket kulcsszóvá tett, ezért három kompatibilitási header tartalma kiürült vagy elavult — de a header maga megmaradt, a `#include <stdbool.h>` C23-ban is érvényes:

Header	C17 tartalma	C23 tartalma	Megjegyzés
`stdbool.h`	`bool`, `true`, `false` makró, `__bool_true_false_are_defined`	csak az obsolescent `__bool_true_false_are_defined`	`bool`, `true`, `false` C23-ban kulcsszó
`stdalign.h`	`alignas`, `alignof` makró, `__alignas_is_defined`, `__alignof_is_defined`	nincs tartalma („provides no content”)	`alignas`, `alignof` C23-ban kulcsszó
`stdnoreturn.h`	`noreturn` makró (`_Noreturn`-re)	`noreturn` makró, a header és a makró obsolescent	helyette `[[noreturn]]` attribútum; a `_Noreturn` is obsolescent
`assert.h` (hosted)	`static_assert` makró	a `static_assert` kulcsszó, a makró kikerült	a `thread_local` is kulcsszó lett (C17-ben `threads.h`-makró)
`stdbit.h`	—	új, freestanding is	bitszámláló és bájtsorrend-segédek

A táblázat a C23 munkatervezet 7. fejezetével és a C header inventory (46.4.1) leltárral egyezik. A labor GCC 13.3 (`-std=c2x`) és Clang 22 (`-std=c23`) mellett `-ffreestanding` módban lefordította azt a próbafájlt, amely mind a kilenc C17-es freestanding headert és a `stdbit.h`-t beemeli, és a `bool`, `alignas`, `static_assert` nevet header nélküli kulcsszóként, a `noreturn`-t a `stdnoreturn.h` makrójaként használja; `-std=c17` alatt ugyanez a forrás a makrós úton fordult. A fejezet egyébként nem feltételezi a C23 teljes támogatását.

Egy konkrét, OS nélküli mikrokontrolleres célon (ez egy példa, nem a freestanding definíciója) a **startup** tipikusan: reset vektor → `.data` inicializált szakasz másolása ROM-ból RAM-ba → `.bss` nullázása → `main`. A `main` visszatérésének itt nincs értelme, a belépési pont neve és hívási konvenciója pedig platform- és toolchain-függő; ezt linker script és startup assembly rögzíti, nem a C forrás.

Az **MMIO** `volatile` pointerrel érhető el fix címen. A `volatile` azt biztosítja, hogy a compiler ne optimalizálja ki és ne rendezze át a hozzáféréseket egymáshoz képest; nem biztosít sorrendet más végrehajtó egységekkel szemben, ahogy azt a volatile, compiler barrier és hardver barrier (14.5) rész kifejtette.

```
/* freestanding vazlat - hosted Linuxon nem futtathato, nem tesztelt pelda */
#include <stdint.h>

#define UART_BASE 0x40000000UL
#define UART_DR   (*(volatile uint32_t *)(UART_BASE + 0x00UL))

void _start(void)   /* a belepesi pont neve es konvencioja platformfuggo */
{
    UART_DR = (uint32_t)'O';
    for (;;) { }
}
```

```
arm-none-eabi-gcc -std=c17 -ffreestanding -nostdlib -Wall -Wextra \
  -T link.ld -o app.elf start.c
```

A target triple, a linker script és a linkelési cím platformtény: ez nem hordozható, hanem az audit egy sorában dokumentált, konkrét profil. Freestanding módban a szabvány nem garantál `malloc`-ot, `printf`-et vagy szálkezelést; hogy a platform ad-e ilyet (kernel-API, a toolchain minimál libc-je, RTOS), az platformtény. Ha nincs, saját allokátort és saját formázót írunk; ha van, a forrását és a szerződését rögzítjük — mindkét eset bekerül az auditba.

18.4.6 Header-leltár és továbbtanulási térkép

A header-leltár az audit melléklete: headerenként, hogy hosted-only (`stdio.h`, `stdlib.h`, `string.h`, `time.h`, `locale.h`, `signal.h`, `threads.h`) vagy freestanding, és mi a portabilitási kockázata. A továbbtanulási térkép elsődleges dokumentumokra mutat: az ISO/IEC 9899:2018 szövege, a C23 nyilvános munkatervezete (N3096 — munkadokumentum, nem az ISO kiadott szabványa), a GCC „Standards” oldala, a Clang dokumentációja és a kiválasztott target ABI-specifikációja. Ezek konkrét verzióját a labornapló rögzíti; ebben az egységben nem állítom, hogy bármelyiküket ellenőriztem vagy lefuttattam.

18.4.7 Tipikus hibák

A leggyakoribb hiba a `CHAR_BIT == 8` és a `sizeof(int) == 4` hallgatólagos feltételezése. Gyakori a `sizeof(void*) == sizeof(long)` átvitele Linuxról Windowsra, a pointer `long`-ba töltése és a csendben csonkolódó cím. Sok kód `%d`-vel ír ki `size_t`-et, ami C++-ban és C-ben egyaránt nem definiált viselkedés; a helyes `%zu`. Gyakori a `union`-os endian-trükk, ami C-ben implementation-defined, C++-ban UB, tehát az auditban nem kaphat „ISO” címkét. Végül tipikus a `#ifdef __linux__` egyedüli ellenőrzésként: ez nem fedi a libc- és architektúra-különbségeket, és nem ad fallbacket, ha az `epoll` nem elérhető.

18.4.8 Összefoglaló

A hordozhatósági audit nem stílus, hanem szerződés: feltételezés, címke, ellenőrzés, fallback. A nyelvi mag (`CHAR_BIT`, szélességek, pointermodell, igazítás) preprocesszorral és statikus állításokkal kényszeríthető ki; a platform és az ABI csak méréssel és dokumentációval. A feature-test policy egy helyen dönt, és a nem támogatott kombinációt fordítási hibává teszi. Freestanding módban a szabvány csak a minimumot garantálja; a startup, az MMIO és minden további könyvtár a platform vagy a saját kód felelőssége, és ezt az auditban kell rögzíteni.

18.4.9 Feladatok

1. Címkézés. Sorold be az alábbi állításokat ISO / implementation-defined / POSIX / Linux / compiler extension / ABI / mérés kategóriába, és indokold: (a) `unsigned char` értékkészlete $0 \dots 2^{CHAR_BIT} - 1$; (b) `sizeof(long) == 8`; (c) `errno` beállítása hiba esetén a `read()` hívás után; (d) `__builtin_expect` működik; (e) a `uint8_t` létezik.

2. Kódolás. Egészítsd ki a `portability.h`-t úgy, hogy `CHAR_BIT != 8` esetén a build álljon meg, kivéve ha `PT_ALLOW_NON_OCTET` definiált; és adj `pt_rotl32` fallbacket `__builtin_rotateleft32` hiányában.

3. Diagnosztika. Az alábbi részlet Linux x86-64-en működik, másutt nem. Találd meg a hibákat, javítsd, és add meg az ellenőrzési feltételt:

```
void *p = get_buffer();
long addr = (long)p;
printf("buffer @ %d, meret %d\n", (int)addr, (int)sizeof(p));
```

18.4.10 Megoldások

1. (a) ISO: az `unsigned char` kitöltő bit nélküli, ezért a `limits.h` garantálja. (b) ABI + mérés: LP64 alatt 8, ILP32 alatt 4; az állítás csak profilhoz köthető. (c) POSIX: az `errno` szemantikája a POSIX `read()` szerződése, nem az ISO C-é. (d) Compiler extension: GCC/Clang-specifikus, `__has_builtin`-nel őrizendő. (e) ISO, de opcionális: csak akkor igaz, ha létezik pontosan 8 bites, kitöltő bit nélküli típus; `UINT8_MAX` definiáltságával ellenőrizhető. Ellenőrzési feltétel: minden sor mellett szerepel a konkrét header vagy dokumentum, amelyre a címke támaszkodik.

2. A védelmet a `#if CHAR_BIT != 8` ágba tett `#if !defined(PT_ALLOW_NON_OCTET)` / `#error` pár adja, a `PT_ALLOW_NON_OCTET`-et pedig a buildrendszer definiálja (`-DPT_ALLOW_NON_OCTET`), nem a forrás. A rotáció fallbackje shift-alapú: `((v << n) | (v >> (32 - n)))` `uint32_t`-n, `n % 32` normalizálással, `n == 0` külön kezelve (a `>> 32` ugyanis nem definiált). Ellenőrzési feltétel: a `__has_builtin` mindkét ága forduljon le, és a két implementáció eredménye `n = 0, 1, 31` esetén egyezzen.

3. Két hiba: a pointer `long`-ba töltése LLP64/ILP32 alatt csonkol, és a `%d` `int`-et vár, miközben `size_t`-et adunk át — ez nem definiált viselkedés. Helyes változat: a pointert csak akkor alakítjuk egésszé, ha `uintptr_t` létezik (`#if defined(UINTPTR_MAX)`), különben az audit „nem támogatott” ágra fut; a formátum `%zu`, a címhez `PRIxPTR` a `<inttypes.h>`-ból, ha az elérhető. Ellenőrzési feltétel: `-Wall -Wextra -Wpedantic -Wformat=2` mellett nincs figyelmeztetés, és a kimenet `sizeof(void*)` értékétől függetlenül helyes.

II. rész: Modern C++

19 Átmenet C-ről modern C++-ra

19.1 C++ programmodell és zero-overhead elv

A C++ nem „C osztályokkal”: a nyelv saját absztrakt géppel, objektum-élettartammal, értékszemantikával és típusokba zárt szerződésekkel dolgozik. Ez a rész azt mutatja meg, hogyan épül erre a C-ben megszerzett memória- és erőforrás-szemlélet, és miért nem lehet a C++ viselkedését pusztán a C szabályainak kiterjesztéseként megérteni. A C hordozhatósági és freestanding kérdéseit itt csak előfeltételként idézzük fel: C hordozhatósági audit és freestanding kitekintés.

19.1.1 A C++20 programmodell: objektum, érték, invariáns

A C++20 alapú program ugyanúgy fordítási egységekből épül fel, mint a C program, és a linkelés, a nyelvi kapcsolás (linkage) valamint az ODR szabályai határozzák meg, mi kerül a futtatható állományba. A futásidejű modell azonban több ponton eltér. A C++ban egy objektum élettartamának kezdete és vége kódot futtathat: a konstruktor felépíti az objektumot és az invariánsát, a destruktor felszabadítja az erőforrásait. Emellett a nyelv ismer referenciákat, túlterhelést, value categorykat, sablonokat és kivételeket. Ezek részletei külön részekben következnek; itt az érték, az invariáns és a tulajdonlás hármasa a lényeg.

Értékszemantika (value semantics). Egy típus értékszemantikájú, ha a példányai önálló értékként viselkednek: a másolás független példányt hoz létre, amely ugyanazt az absztrakt értéket képviseli; az értékadás felülírja a cél értékét; az egyenlőség az absztrakt értéken értelmezett. A beépített típusoknál ez természetes. Osztályoknál az alapértelmezett másolás tagról tagra történik, ami akkor helyes, ha a típusnak nincs közvetlenül kezelt, megosztott erőforrása. Egy nyitott file descriptorot vagy egy `new`-val szerzett puffert tartalmazó típus esetében a sekély másolás két tulajdonost hozna létre ugyanarra az erőforrásra, ami dupla felszabadításhoz vezet. Az ilyen típust vagy nem másolhatóvá kell tenni, vagy dokumentált, megosztott tulajdonlással kell ellátni.

Invariáns. Az invariáns olyan állítás a típus belső reprezentációjáról, amelynek minden nyilvános művelet előtt és után, valamint minden tagfüggvény belépésekor fenn kell állnia. A konstruktor feladata létrehozni; a destruktor csak érvényes állapotból futhat, de nem köteles az invariánst „továbbvinni”. Például egy `fd_owner` invariánsa lehet az, hogy a tárolt descriptor vagy `-1`, vagy egy nyitott, a folyamat által birtokolt file descriptor. Ha a descriptor értéke véletlenül egy másik objektum által már lezárt erőforrásra mutat, az invariáns sérül, és a további műveletek viselkedése nem definiált.

Tulajdonlás (ownership). A tulajdonos az az objektum, amelynek joga és kötelezettsége az erőforrás felszabadítása. Egy erőforrásnak egy adott időpontban egyértelmű tulajdonosa legyen, kivéve, ha a megosztott tulajdonlás szabályai dokumentáltak. A C-ben a tulajdonlás kézi: minden hibás kimeneti ágon `free`-re, `fclose`-ra vagy `close`-ra van szükség. A C++ban a tulajdonlást tipikusan egy objektumhoz kötjük, és a destruktorára bízzuk a felszabadítást.

Szempont	C	C++20
Erőforrás felszabadítása	kézi, minden kimeneti ágon	owner destruktorában, scope végén és unwind során
Másolás	jellemzően byte-másolás vagy kézi mély másolás	típus által meghatározott; owner gyakran nem másolható
Inicializálás	értékadás, `memset`, függvényhívás	konstruktor, taginicializálás, invariáns
Hibajelzés	visszatérési érték, `errno`	visszatérési érték, `errno`, kivétel, később `expected`
ABI-határ	stabil C ABI	névtorzítás, C++ ABI; `extern "C"` határ szükséges

19.1.2 A zero-overhead elv: cél, nem garancia

A C++ tervezési hagyományában gyakran így fogalmazzák meg a zero-overhead elvet: „amit nem használsz, azért ne fizess; amit használsz, azt ne tudd kézzel jobban megírni”. Ez kettős követelmény. Egyrészt a nem használt nyelvi képességek ne okozzanak kötelező futásidejű költséget. Másrészt egy használt absztrakció forduljon legalább olyan jó gépi kódra, mint egy egyenértékű, gondosan kézzel írt megoldás.

Ez **tervezési cél és minőségi elv**, nem univerzális teljesítménygarancia. A szabvány a program megfigyelhető viselkedését írja le, nem azt, hogy egy adott konstrukció hány utasításból vagy memóriahozzáférésből áll. A tényleges költség függ a fordítótól, az optimalizálási szinttől, a cél-ABI-tól, a standard library implementációjától, a beágyazástól és attól, hogy a fordító látja-e a függvény definícióját. Ugyanaz a forrás `-O0` és `-O2` mellett, illetve GCC és Clang alatt eltérő lehet.

A zero-overhead nem azt jelenti, hogy „nulla költség”. A `std::vector` növekedéskor allokálhat; a `std::function` típus-erasure miatt allokálhat; egy kivétel dobása drága lehet; a virtuális hívás indirekt. Azt jelenti, hogy ezek a költségek a használt funkcióhoz tartoznak, és nem feltétlenül nagyobbak, mint egy egyenértékű kézi megoldásé. A fordító például egy kis wrapper függvényt be tud inline-olni, egy `std::sort` hívást ugyanolyan hatékony gépi kódra tud fordítani, mint egy kézi rendezést, és a nem használt kivételkezelés nem feltétlenül lassítja a normál utat.

A C++ absztrakció akkor hasznos, ha az invariánst típusba zárja, a hibát korai fázisba viszi, a cleanupot determinisztikussá teszi, és a hot pathon a fordító ki tudja iktatni a burkolást. Akkor érdemes visszafogni, ha a mérés azt mutatja, hogy a költség nem elfogadható, ha az ABI-határ miatt nem lehet inline-olni, ha a fordítási idő vagy a kódméret kritikus, vagy ha a csapat számára a karbantartási kockázat nagyobb, mint a nyereség. A döntést mérésnek, generated assemblynek és profile-nak kell alátámasztania, nem a „C++ úgyis gyors” feltételezésnek.

19.1.3 RAII: az erőforrás az objektum élettartamához kötve

A RAII (Resource Acquisition Is Initialization) magyarul leggyakrabban „erőforrás-életciklus kötése objektum-élettartamhoz”. A név félrevezető: nem az inicializálás a lényeg, hanem az, hogy az erőforrás megszerzése a konstruktorban, felszabadítása pedig ugyanazon objektum destruktorában történik. Az automatikus tárolási idejű objektumok a blokk elhagyásakor megszűnnek, normál `return` esetén és kivétel miatti stack unwinding során is. Így a felszabadítás nem függ attól, melyik `return` ágon jutunk ki.

A RAII helyes használatának feltételei: a konstruktor vagy teljesen felépíti az invariánst, vagy dob; a destruktornak nem szabad dobnia; az owner vagy nem másolható, vagy a másolása nem duplikálja a tulajdonjogot; és a vezérlés nem kerüli meg a destruktorokat. A `std::exit` nem futtatja az automatikus objektumok destruktorait, az `abort` szintén nem, a `longjmp` pedig nem definiált viselkedés, ha olyan automatikus objektumot hagyna ki, amelynek nem triviális destruktora van. A `longjmp` és a C++ kivételek keverése különösen veszélyes.

Bizonyításvázlat (RAII determinizmusa). Tegyük fel, hogy minden erőforrás-akvizíció egy automatikus objektum konstruktorában történik, és minden felszabadítás ugyanazon objektum destruktorában. A nyelv szabályai szerint az automatikus objektumok a blokk elhagyásakor megszűnnek, a konstrukció fordított sorrendjében, akár normál kimenet, akár kivétel miatti unwinding történik. Ezért a sikeresen felépült owner objektumok destruktorai lefutnak. Ha egy konstruktor dob, a már felépült tagok és bázisok destruktorai lefutnak, a teljes objektum destruktora nem; így a részlegesen felépült állapot nem szivárog. A garancia azonban csak a fenti feltételek mellett áll: nem másolható owner, nem dobó destruktor, és nincs `longjmp`/`abort`/`std::exit` általi megkerülés.

Az alábbi C változat minden hibás ágon kézi takarítást végez. A példa szándékosan egyszerű, és nem kezeli a `fclose` hibáit; a lényeg a cleanup-ágak számának növekedése.

```
#include <stdio.h>

int copy_file(const char *src, const char *dst) {
    FILE *in = fopen(src, "rb");
    if (!in) return -1;
    FILE *out = fopen(dst, "wb");
    if (!out) { fclose(in); return -1; }

    char buf[4096];
    size_t n;
    while ((n = fread(buf, 1, sizeof buf, in)) > 0) {
        if (fwrite(buf, 1, n, out) != n) {
            fclose(in);
            fclose(out);
            return -1;
        }
    }
    int rc = ferror(in) ? -1 : 0;
    fclose(in);
    fclose(out);
    return rc;
}
```

A C++ változatban a két `FILE*` egy-egy owner objektumba kerül. Ha a második `fopen` sikertelen, az első owner destruktora lefut; ha írás közben kivétel keletkezik, mindkét owner destruktora lefut.

```
#include <cstddef>
#include <cstdio>
#include <stdexcept>

class file {
public:
    explicit file(const char* path, const char* mode)
```

```
        : f_{std::fopen(path, mode)} {
        if (!f_) throw std::runtime_error{"fopen failed"};
    }
    ~file() { if (f_) std::fclose(f_); }

    file(const file&) = delete;
    file& operator=(const file&) = delete;

    std::FILE* get() const noexcept { return f_; }

private:
    std::FILE* f_;
};

void copy_file(const char* src, const char* dst) {
    file in{src, "rb"};
    file out{dst, "wb"};

    char buf[4096];
    while (std::size_t n = std::fread(buf, 1, sizeof buf, in.get())) {
        if (std::fwrite(buf, 1, n, out.get()) != n)
            throw std::runtime_error{"write failed"};
    }
    if (std::ferror(in.get()))
        throw std::runtime_error{"read failed"};
}
```

A példa nem lett lefuttatva; a következő parancsok a tanuló laborjában ellenőrizhetik, hogy a fordítási egység a választott compilerrel és standard libraryvel elfogadható:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -c copy_file.cpp -o copy_file.o
clang++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -c copy_file.cpp -o copy_file.o
```

Itt érdemes megjegyezni egy fontos árnyalatot: a RAII determinisztikus felszabadítást ad, de nem feltétlenül hibajelzést. A `fclose` hibát jelezhet, például flush közben; a destruktor nem dobhat, ezért a zárás hibáját nem tudja a hívó felé továbbítani. Ilyenkor a helyes szerződés gyakran egy explicit `close()` tagfüggvény, amely jelzi a hibát, a destruktor pedig csak best-effort fallback. Ez a minta később a file descriptor, a lock és a socket wrapperénél is visszatér.

19.1.4 Ökoszisztéma, implementációk és a C-ből át nem vihető szabályok

A könyv alapja a C++20, explicit `-std=c++20` kapcsolóval. A C++23 használatát külön jelölni kell, és nem elég a nyelvi támogatást ellenőrizni: a compiler és a standard library támogatása két külön dolog. A feature-test makrók (`__cpp_...`, `__cpp_lib_...`) és egy kis compile probe segítenek, de a konkrét értékek és elérhetőség a laborban rögzített compiler- és library-verziótól függenek. A C++23-as kódot tehát jelölt, ellenőrzött kiterjesztésként kezeljük, nem a C++20 alap részeként.

A `g++` és a `clang++` fordítási driver, nem maga a standard library. GCC esetén jellemzően a libstdc++, Clang esetén Linuxon gyakran szintén a libstdc++, de konfigurálható libc++ használatára is; a `-stdlib=libc++` Clang-specifikus kapcsoló. A standard library verziója és a compiler verziója együtt határozza meg, milyen fejlécek, feature-ök és ABI-k érhetők el. A laborban

ezért rögzíteni kell a compilerverziót, a library implementációját és a fontos buildkapcsolókat; a `-v` és a feature-test külön ellenőrzés lehet.

A standard library a nyelvhez tartozó, hordozható alapot adja, de nem független a implementációtól. Harmadik féltől származó könyvtárak, például Boost, fmt vagy Abseil, újabb funkciókat, más ABI-t és külön dependency-kezelést hozhatnak. Használatukkor nem csak a funkcionalitást kell értékelni, hanem a licencet, a supply chain kockázatot, a bináris kompatibilitást és a build-időt is. A standard library nem feltétlenül „ingyenes” a bináris méret vagy az ABI szempontjából, a third-party pedig nem feltétlenül rosszabb; a döntés mérnöki trade-off.

A C-ből C++ba átlépve néhány szabály nem vihető át változtatás nélkül. A `void*` nem konvertálódik implicit módon `T*`-ra, explicit cast kell. A string literál C++ban `const char[N]`, ezért `char*`-ra konvertálni nem lehet. A változó hosszúságú tömb (VLA) nem standard C++; helyette dinamikus tárolót vagy standard tárolót kell használni. A `malloc`/`free` nem helyettesíti a `new`/`delete`-et olyan típusnál, amelynek nem triviális konstruktora vagy destruktora van: a tároló megszerzése és az objektum élettartamának megkezdése C++ban két külön dolog. Nem triviális típuson `memcpy` használata nem definiált viselkedés; a C++ ABI-határon pedig `extern "C"` szükséges a névtorzítás elkerüléséhez. Ezek részletei a referenciák, az inicializálás és az ABI-fejezetekben következnek.

19.1.5 Gyakori hibák

Gyakori tévedés a „C++ = C + osztályok” modell: ebből következik, hogy valaki `memcpy`-vel másol nem triviális objektumot, vagy `malloc`-kal próbál életet adni egy `std::string`-nek. Ugyanilyen hiba a zero-overhead elvet teljesítménygaranciának tekinteni, és mérés nélkül azt állítani, hogy egy absztrakció „ingyen van”. A RAII-t sokan azonosítják a smart pointerekkel, holott a lock, a file descriptor és a socket is RAII-val kezelhető. Végül: a destruktor nem univerzális hibakezelő; a zárás hibáját külön szerződésben kell jelenteni, ha a hívónak tudnia kell róla.

19.1.6 Feladatok

1. **Érvelő feladat.** Egy kolléga azt állítja, hogy a `std::vector<int>` `push_back` művelete a zero-overhead elv miatt garantáltan gyorsabb, mint egy kézzel kezelt dinamikus tömb. Fogalmazd meg, milyen feltételek mellett lehet igaz vagy hamis az állítás, és milyen mérést végeznél.
2. **Kódolási feladat.** Egészítsd ki a fejezet `file` osztályát egy `close()` tagfüggvénnyel, amely a `fclose` hibáját jelzi, és biztosítja, hogy a destruktor ne zárja le másodszor ugyanazt a streame-t. Írd le a szerződést és a lehetséges kivételt.
3. **Diagnosztikai feladat.** Az alábbi tanítási példa szándékosan hibás; ne futtasd. Magyarázd meg, miért nem definiált a viselkedése, és javítsd úgy, hogy egyértelmű tulajdonos legyen.

```
struct owner {
    int* p;
    explicit owner(int* q) : p{q} {}
    ~owner() { delete p; }
};

void f() {
    int x = 0;
    owner a{&x};
```

```
    owner b = a;
}
```

19.1.7 Megoldások

1. **Megoldás.** A zero-overhead elv nem mondja ki, hogy a `std::vector` minden körülmények között gyorsabb. Azt mondja, hogy a használt absztrakció ne legyen rosszabb egy egyenértékű kézi megoldásnál, és a nem használt funkciókért ne fizessünk. A `vector` amortizált növekedést, allokációt és invalidációt végez; a kézi tömb is ugyanezt teheti. A helyes összehasonlítás azonos műveletsor, azonos optimalizálási szint és azonos allocator mellett történik, és méri a futásidőt, az allokációk számát, a csúcsmemóriát és a generált kódot. A válasz tehát: az állítás feltételek nélkül hamis; mérés és a `vector` szerződésének ismerete nélkül nem dönthető el.

2. **Megoldás.** A `close()` végezze el az explicit zárást, állítsa `nullptr`-ra a pointert, és hiba esetén dobjon vagy jelezzen visszatérési értékkel. A destruktor csak akkor zárjon, ha a pointer még nem `nullptr`; így a sikeres `close()` után nem lesz második `fclose`. A szerződés: `close()` után az objektum „nem birtokol streamet”, a destruktor no-throw marad. Ha a `fclose` hibát jelez, a stream a gyakorlatban már le van zárva; a pointert ettől függetlenül nullázni kell a dupla zárás elkerülésére. A hiba továbbítására például `std::runtime_error` vagy egy külön `close_result` használható; ezt a későbbi hibakezelési rész részletezi.

3. **Megoldás.** A hiba kettős: a `delete p` egy stacken lévő `int` címére mutat, ami nem definiált viselkedés; ráadásul a `b = a` sekély másolással ugyanazt a pointert másolja, így a két destruktor ugyanazt az erőforrást próbálja felszabadítani. A javítás egyik változata a másolás törlése és dinamikus allocating használata:

```
class owner {
public:
    explicit owner(int* p) : p_{p} {}
    ~owner() { delete p_; }
    owner(const owner&) = delete;
    owner& operator=(const owner&) = delete;

private:
    int* p_;
};

void f() {
    owner a{new int{42}};
}
```

Az ellenőrzési feltétel: a tulajdonos egyértelmű, a másolás nem duplikálja a tulajdonjogot, és a destruktor csak egyszer szabadít fel. A hosszú távú megoldás később a `std::unique_ptr`, amely ugyanezt a szerződést fejezi ki.

19.1.8 Összefoglaló

A C++ programmodellje a C objektum- és memóriamodelljére épül, de az élettartamhoz konstruktort és destruktort rendel. Az értékszemantika önálló példányokat, az invariáns típusba zárt szerződést, a tulajdonlás pedig egyértelmű felszabadítási felelőst jelent. A RAII ezt úgy valósítja meg, hogy az erőforrást egy automatikus objektum élettartamához köti; ez determinisztikus

cleanupot ad, de nem csodaszer: a destruktor nem dobhat, és a zárás hibáját külön kell jelenteni. A zero-overhead elv hasznos tervezési irány, amely szerint a nem használt funkciókért nem fizetünk, a használt absztrakció pedig közel azonos kódot érhet el a kézi megoldással; ez azonban nem garancia, hanem feltételekkel és méréssel ellenőrizhető cél.

19.2 Namespace, header és névfeloldás

A C-ben a legtöbb könyvtár a globális névtérben él, ezért két független könyvtár azonos nevű függvénye vagy változója linkeléskor ütközhet; a hiba gyakran csak a végső program összeállításakor jelentkezik. A C++ `namespace` nyelvi eszköze ezt a problémát már a forrás szintjén kezeli: a neveket névterekbe szervezi, a minősített név pedig egyértelművé teszi, melyik entitásra hivatkozunk. Ez az alfejezet többmodulos könyvtár példáján mutatja be a névütközések elkerülését, a header- és implementációszervezést, valamint a linkage és az ODR előképét. Az előfeltétel a C++ programmodell (C++ programmodell és zero-overhead elv (19.1)), a C header-higiénia (Headerek, deklarációk és include-higiénia (2.2)) és a szimbólumok/linkage alapjai (Szimbólumok, linkage és relocation (3.1)).

19.2.1 Namespace, beágyazott namespace és alias

A namespace deklaratív régió, amely neveket fog össze. A benne deklarált nevek a namespace tagjai; kívülről minősített névvel (`acme::diag::log`) vagy a név bevitelét engedélyező `using`-deklarációval érhetők el. A minősítetlen névfeloldás a mindenkori hatókörből kifelé haladva keresi a nevet; a minősített névfeloldás csak a megnevezett névtérben és annak inline namespace-eiben keres. A függvényhívásoknál ehhez még argumentumfüggő névfeloldás (argument-dependent lookup, ADL) is társulhat, amely a függvényargumentumok típusainak névtereit is figyelembe veszi; ennek részletei később, a template-es fejezetben kerülnek elő.

A beágyazott namespace definíciója C++17 óta tömören írható:

```
namespace acme::diag {   // C++17: egyenértékű a beágyazott namespace-ekkel
int version_major = 1;
}
```

A namespace újranyitható: ugyanazt a névteret több headerben és `.cpp`-ben is folytathatjuk, ami a moduláris szervezés alapja. Az inline namespace a verziózás eszköze: tagjai úgy látszanak, mintha a befoglaló névtérben lennének deklaráva, miközben a minősített név megőrzi a verziót. A névtér-alias (`namespace d = acme::diag;`) alternatív nevet vezet be; az alias attól a ponttól érvényes, ahol deklaráljuk, és nem nyitható újra namespace-ként. Publikus headerben az alias is névimport, ezért inkább a `.cpp`-ben vagy szűk hatókörben használjuk.

Fontos megkülönböztetni a `namespace detail` konvenciót a nyelvi védelemtől: a `detail` név semmilyen hozzáférés-védelmet nem ad, csak jelzi, hogy a felhasználónak nem ajánlott rá támaszkodnia. A valódi belső implementációs részleteket névtelen namespace-be vagy külön, nem publikált headerbe érdemes tenni.

19.2.2 using deklaráció és using direktíva

A két eszköz szemantikailag különbözik, és a header-higiénia szempontjából különböző kockázatot hordoz.

A **using-deklaráció** (`using acme::diag::log;`) egy nevet visz be a jelenlegi deklarációs régióba. Függvénynév esetén a cél-névtérben látható túlterhelések is részt vesznek a további túlterhelésfeloldásban. A bevitel a deklaráció helyétől a hatókör végéig tart. Ha ugyanabban a hatókörben már van azonos nevű, összeférhetetlen entitás, a program hibás; ezért a using-deklarációt lehetőleg szűk, helyi hatókörben alkalmazzuk.

A **using-direktíva** (`using namespace acme::diag;`) nem visz be nevet a hatókörbe. A minősítetlen névfeloldás során a megnevezett névtér nevei úgy látszanak, mintha a direktívát és a névteret közrefogó legközelebbi névtérben deklarálták volna őket. Emiatt a direktíva hatása nem lokális: egy headerben elhelyezett `using namespace std;` minden olyan fordítási egység globális névterébe „beszivárog", amelyik include-olja a headert, és minden későbbi minősítetlen névfeloldást befolyásol. Ez váratlan ambiguitásokat okozhat, különösen ADL-lel kombinálva, és megváltoztathatja a felhasználó kódjának jelentését.

Gyakorlati szabály: publikus headerben namespace-scope-on ne legyen `using namespace`. A `using`-deklaráció is lehet névimport, ezért headerben csak indokolt esetben, szűk névre használjuk; a `.cpp`-ben vagy függvényhatókörben viszont elfogadható, ha javítja az olvashatóságot. A `using namespace std;` még `.cpp`-ben is kerülendő nagy fájlokban; helyette `std::` minősítés vagy szűk `using std::string;` használható.

Forma	Nyelvi hatás	Headerben
`using N::f;`	nevet visz be a jelenlegi régióba	lehetőség szerint kerülendő namespace-scope-on
`using namespace N;`	nem visz be nevet; a minősítetlen lookup kiterjesztése	ne használjuk
`namespace a = N;`	alternatív név	csak indokolt esetben, inkább `.cpp`-ben

19.2.3 Header és implementáció

A header a szerződést tartalmazza: típusdeklarációkat, függvényprototípusokat, osztálydefiníciókat, template-eket, `inline` függvényeket és `inline` változókat. A nem inline függvények és változók definíciója a `.cpp`-be kerül, ahol a nevet a namespace minősítésével definiáljuk. A header védelmét include guard vagy `#pragma once` biztosítja; a „include what you use" elv szerint minden használt típushoz a megfelelő header-t include-oljuk, és a header ne támaszkodjon arra, hogy a felhasználó „úgyis" include-ol valamit.

A névtelen namespace a fordítási egységen belüli segédfüggvények és -típusok elhelyezésére való. A benne deklarált nevek belső linkage-t kapnak, és minden fordítási egységben külön entitást jelölnek. A `.cpp`-ben ez kiválóan alkalmas arra, hogy a publikus API-t ne szennyezzük segédfüggvényekkel. Headerbe viszont nem való: minden includer külön példányt kapna, ami bloat-hoz, meglepő cím-összehasonlításokhoz és – ha inline függvény vagy template használja – ODR-problémákhoz vezethet.

19.2.4 const, inline változó és az ODR előképe

A namespace-scope változók linkage-je nem mindig external. A `const` (és így a `constexpr`) változó alapértelmezésben belső linkage-t kap, ha nincs `extern`-ként deklarálva és nem `inline`. Ezért a headerben definiált `const int max_retries = 3;` minden fordítási egységben külön objektum; értékként használva általában működik, de címe TU-nként eltérhet, és ha egy inline függvény

hivatkozik rá, az inline függvény definíciói különböző entitásokat neveznek meg – ez ODR-sértés, akár diagnosztika nélkül is.

A C++17 `inline` változója ezt oldja meg: `inline constexpr int max_retries = 3;` external linkage-ű, egyetlen entitás a programban, és több fordítási egységben is definiálható, feltéve, hogy a definíciók azonosak. Ugyanez az elv érvényes az `inline` függvényekre is.

Az egységes definíció szabálya (One Definition Rule, ODR) előképe: egy nem inline függvénynek vagy változónak az egész programban pontosan egy definíciója lehet; egy inline függvénynek vagy változónak több TU-ban is lehet definíciója, de ezeknek azonosnak kell lenniük, és ugyanazokat az entitásokat kell megnevezniük. Az osztálydefiníciók több TU-ban is szerepelhetnek, ha azonosak. A szabály megsértése gyakran „ill-formed, no diagnostic required” (NDR): a fordító nem köteles jelezni, a program viselkedése pedig nem definiált.

Deklaráció namespace scope-on	Linkage	Egyediség
`int g;`	external	egy definíció a programban
`static int g;`	internal	TU-nként külön objektum
`namespace { int g; }`	internal	TU-nként külön objektum; típusokra is jó
`const int c = 1;`	internal	TU-nként külön objektum
`inline constexpr int c = 1;`	external	egy inline entitás, több azonos definíció

19.2.5 Kidolgozott példa: többmodulos diagnosztikai könyvtár

A következő példa egy publikus headerből, egy implementációs `.cpp`-ből és egy `main.cpp`-ből áll. A publikus API a `acme::diag` névtérben él; a `.cpp`-ben névtelen namespace tartalmazza a segédfüggvényt, a fejlécben pedig `inline constexpr` biztosítja a verzióállandó egyetlenségét.

```
// include/acme/diag.hpp
#ifndef ACME_DIAG_HPP
#define ACME_DIAG_HPP

#include <string>
#include <string_view>

namespace acme::diag {

inline constexpr int version_major = 1;

enum class level { info, warning, error };

void log(level lvl, std::string_view message);
std::string format(level lvl, std::string_view message);

namespace detail {
std::string_view level_name(level lvl);
} // namespace detail

} // namespace acme::diag
```

```
#endif

// src/diag.cpp
#include "acme/diag.hpp"

#include <cstdio>
#include <string>

namespace {
// TU-local helper: belső linkage, a könyvtár felhasználója nem látja.
std::string bracketed(std::string_view s) {
    std::string out;
    out.reserve(s.size() + 2);
    out.push_back('[');
    out.append(s.data(), s.size());
    out.push_back(']');
    return out;
}
} // anonymous namespace

namespace acme::diag::detail {

std::string_view level_name(level lvl) {
    switch (lvl) {
    case level::info:    return "INFO";
    case level::warning: return "WARN";
    case level::error:   return "ERROR";
    }
    return "?";
}

} // namespace acme::diag::detail

namespace acme::diag {

std::string format(level lvl, std::string_view message) {
    std::string out;
    const auto name = detail::level_name(lvl);
    out.reserve(name.size() + message.size() + 2);
    out.append(name);
    out.append(": ");
    out.append(message.data(), message.size());
    return out;
}

void log(level lvl, std::string_view message) {
    const auto line = format(lvl, message);
    std::fputs(bracketed(line).c_str(), stderr);
    std::fputc('\n', stderr);
}
```

```
} // namespace acme::diag

// src/main.cpp
#include "acme/diag.hpp"

#include <string>

int main() {
    using acme::diag::level; // szűk using-deklaráció, csak függvényhatókörben
    acme::diag::log(level::info, "start");
    const std::string s = acme::diag::format(level::warning, "disk almost
     ↪  full");
    acme::diag::log(level::warning, s);
}
```

Fordítás és futtatás a tanuló laborjában:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -Iinclude src/diag.cpp src/main.cpp -o
 ↪  diag_demo
./diag_demo
```

A tervezett kimenet: `[INFO: start]`, majd `[WARN: disk almost full]`. A példa C++20 nyelvi szabályokra épül; a kimenet tervezett, nem mért eredmény, és a kód fordítását/futtatását itt nem állítjuk megtörténtnek.

19.2.6 Tipikus hibák és javításuk

- **`using namespace std;` publikus headerben.** A direktíva minden includer globális névterébe szivárog, és ambiguitást okozhat. Javítás: távolítsuk el; a `.cpp`-ben használjunk `std::` minősítést vagy szűk `using std::string;`-et.
- **Nem inline függvény definíciója headerben.** Több TU-ba include-olva többszörös definíció lesz. Javítás: deklaráció a headerben, definíció a `.cpp`-ben; vagy tudatos `inline`.
- **`const int` headerben, majd `inline` függvényben használva.** A konstans TU-nként külön objektum, az inline függvény definíciói eltérő entitásra hivatkozhatnak. Javítás: `inline constexpr int`.
- **Névtelen namespace headerben.** Minden TU külön példányt kap; ODR-veszély és kódduplikáció. Javítás: a segédfüggvény a `.cpp` névtelen namespace-ébe kerüljön.
- **Namespace-alias globális headerben.** Az alias nevét minden includer globális névterébe viszi. Javítás: az alias a `.cpp`-be vagy szűk hatókörbe kerüljön.

19.2.7 Összefoglaló

A namespace a C++ névütközés-kezelésének alapja: a publikus API-t érdemes saját, dokumentált névtérbe szervezni, a belső segédeszközöket pedig névtelen namespace-be tenni a `.cpp`-ben. A `using`-deklaráció nevet visz be, a `using`-direktíva csak a minősítetlen névfeloldást terjeszti ki; utóbbi headerben különösen veszélyes. A header a szerződést, a `.cpp` a definíciókat tartalmazza. A `const`/`constexpr` változók alapértelmezett belső linkage-je és az `inline` változók external linkage-je az ODR szempontjából kulcsfontosságú: headerben használt állandóhoz `inline constexpr` kell. A C `extern "C"` határ kezelése későbbi rész témája.

19.2.8 Feladatok

1. **Alap – namespace-be szervezés.** Alakítsd át a következő C-szerű API-t úgy, hogy a `acme::diag` névtérbe kerüljön, és a `main` minősített névvel hívja: `void log_msg(int level, const char* msg);`, `int log_level;`. Írd le, hogyan változik a hívás, és miért nem kell a `log_msg` nevet átnevezni.

2. **Közepes – header-javítás.** Adott egy header: `#pragma once`, `#include <string>`, `using namespace std;`, `int counter = 0;`, `void bump() { ++counter; }`. Sorolj fel legalább három problémát, és javítsd a headert úgy, hogy több TU-ból is include-olható legyen. Indokold a linkage-változtatásokat.

3. **Haladó – ODR-diagnosztika.** Egy headerben ez áll: `const int max_retries = 3;` és `inline bool retryable(int n) { return n < max_retries; }`. Két `.cpp` include-olja a headert, és mindkettő használja a `retryable` függvényt. Mi a probléma? Javítsd a kódot, és fogalmazd meg, milyen ellenőrzéssel lehetne kimutatni a hibát.

19.2.9 Megoldások

1. megoldás. A névtérbe szervezés:

```
namespace acme::diag {
void log_msg(int level, const char* msg);
extern int log_level; // deklaráció; a definíció valamelyik .cpp-ben
}
```

A hívás `acme::diag::log_msg(level, "start");` lesz. A függvény nevét nem kell megváltoztatni, mert a namespace már egyedivé teszi a teljes nevet; a linker szintjén a minősített név szerepel. A `log_level` esetében a headerben csak `extern` deklaráció állhat, a definíció a `.cpp`-be kerül. Ellenőrzés: a header önmagában is fordítható legyen, és két különböző könyvtár azonos nevű `log_msg`-je ne ütközzön, ha külön namespace-ben van.

2. megoldás. Problémák: a `using namespace std;` minden includer globális névterébe szivárog; a `counter` nem inline, ezért több TU-ból include-olva többszörös definíció lesz; a `bump` nem inline, ezért szintén többszörös definíció lesz; a header nem biztosítja, hogy a `counter` egyetlen entitás legyen. Javítás:

```
#pragma once
#include <string>

namespace acme::diag {
inline int counter = 0; // C++17: egy inline entitás
inline void bump() { ++counter; }
}
```

A `using namespace std;` eltávolítandó; ha kell, a `.cpp`-ben `using std::string;` használható. Ellenőrzés: két TU-ból include-olva és linkelve ne legyen „multiple definition” hiba; a `counter` címe a programban minden TU-ban ugyanaz legyen.

3. megoldás. A `const int max_retries = 3;` namespace-scope-on belső linkage-ű, mert nem `extern` és nem `inline`. Minden TU külön `max_retries` objektumot kap. Az `inline bool retryable(int)` external linkage-ű inline függvény, amelynek minden TU-beli definíciója a saját TU-jának `max_retries` objektumára hivatkozik. Ezért az inline függvény definíciói nem ugyanazt az entitást nevezik meg, ami ODR-sértés; a program ill-formed, diagnosztika nélkül is. Javítás:

```
inline constexpr int max_retries = 3;
inline bool retryable(int n) { return n < max_retries; }
```

Az `inline constexpr` external linkage-ű, egyetlen entitás a programban. Ellenőrzés: két TU-ból is include-olva linkeljük a programot, és hasonlítsuk össze a `&max_retries` címét a két TU-ban – egyezniük kell. Ez a cím-összehasonlítás csak ellenőrző eszköz, nem a program logikájának része.

19.3 Reference, pointer és const C++-ban

A C-ben megszokott pointeres API-k után a C++ referencia első ránézésre kényelmesebb szintaxist ad ugyanarra: egy másik objektumra hivatkozó név. A kényelem azonban nem jelent automatikus biztonságot. A referencia ugyanúgy élettartam-szerződést hordoz, mint a pointer, csak a szerződés egy részét a nyelv kényszeríti ki: a referencia létrejöttekor kötelező egy létező objektumhoz kötődnie, és később nem lehet másikhoz kötni. Azt viszont, hogy a hivatkozott objektum meddig él, a referencia nem tudja és nem is garantálja. Ez a fejezet ezt a kettősséget járja körül; a C++ programmodell és a zero-overhead elv C++ programmodell és zero-overhead elv (19.1), valamint a C pointeres const-szabályai const pointerek, több szintű indirection és void* (6.3) már ismertek.

19.3.1 A referencia szemantikája: `T&`, `const T&` és `T&&`

A referencia a nyelvi modellben nem önálló objektum, hanem egy már létező objektum alternatív neve. Nincs saját értéke, nincs „null értéke”, és a `sizeof` a referált típust adja vissza. Az, hogy a fordító a gépi szinten pointerként ábrázolja-e, implementációfüggő; a szerződés szempontjából a referencia névalias.

A három alapalak:

- `T&`: nem-const lvalue referencia. Csak olyan kifejezéshez köthető, amelynek van azonosított objektuma (lvalue), és amelynek típusa `T` vagy `T`-vé konvertálható. Temporaryhez nem köthető.
- `const T&`: const lvalue referencia. Lvalue-hez és rvalue-hoz is köthető; rvalue esetén egy temporary jön létre, amelynek élettartama a referencia élettartamáig hosszabbodhat.
- `T&&`: rvalue referencia. Nem-const rvalue-hoz köt; lvalue-hoz nem. A nevezett `T&&` változó maga is lvalue kifejezés, ezért kell a továbbadásához `std::move`; ennek részletei a mozgatás témájában Move constructor, moved-from állapot és std::move (22.3) következnek. Template-paraméterként a `T&&` forwarding reference is lehet, amit Forwarding reference és perfect forwarding (26.3) tárgyal.

A referencia deklarációjában a `&` és `&&` nem halmozható egymásra, és referencia-referencia nem létezik; a template-paramétereknél érvényes „referenciacollapse” külön szabály. Referenciatömb és referencia-pointer sem képezhető, de pointerre mutató referencia igen: `int*&`.

19.3.2 Referenciakötés és az újrakötés hiánya

A referenciakötés (reference binding) nem azonos az értékadással. A legfontosabb félreértés, hogy a referencia „átállítható”:

```
int a = 1, b = 2;
int& r = a;
r = b;          // a értéke lesz 2; r továbbra is az a objektumot nevezi

int* p = &a;
p = &b;         // p most b-re mutat; a változatlan
```

A `r = b` sor nem újrakötés, hanem `a = b`. A referencia a deklaráció után minden előfordulásában ugyanazt az objektumot jelöli. Ez a különbség adja az API-szerződés egyik fő választási szempontját: ha a függvénynek egy másik objektumra kell „átállnia”, pointer kell; ha a név jelentése rögzített, referencia.

Függvényparaméterként a `void f(int& x)` a hívó objektumát nevezi, nem másolatot készít. A `void g(int x)` másolatot kap, a `void h(const int& x)` nem másol, de nem is módosíthat. A referencia tag esetében a kötést a konstruktor inicializáló listájában kell elvégezni; az implicit copy assignment emiatt törölt lesz, mert a referenciatag nem assignolható:

```
struct Alias {
    int& r;
    explicit Alias(int& x) : r(x) {}
};
// Alias x(a), y(b); x = y;  // fordítási hiba: operator= törölt
```

Ez nem hiba, hanem szerződés: a referenciát tartalmazó típus alapból nem újraköthető. Ha mégis újraköthetőség kell, pointer tagot vagy `std::reference_wrapper`-t használunk, és dokumentáljuk az élettartamot.

19.3.3 Null és hiányzó érték: pointer, `optional`, szerződés

A `T&` szerződése: „kötelező, érvényes objektum”. A hívó nem adhat át `nullptr`-t, és a függvény nem is tesztelheti, hogy van-e érték. Ahol a hiányzó állapot megengedett, három reprezentáció jön szóba:

Szempont	`T&`	`T*`	`std::optional<T>`
Lehet „nincs érték”?	nem	igen (`nullptr`)	igen
Újraköthető?	nem	igen	értékadással igen
Tulajdonol?	nem	nem (nyers pointer)	a tárolt értéket igen
Élettartamot garantál?	nem	nem	a saját példányára igen
Tipikus API-jelentés	kötelező érvényes	opcionális, nem tulajdonló	opcionális érték

A `std::optional<T>` (C++17) akkor természetes, ha a „nincs” állapot értékszerű, és az érték másolása nem túl drága. A pointer akkor, ha a hívó tulajdonában lévő objektumra mutatunk, és a `nullptr` a hiányt jelzi. A `std::reference_wrapper<T>` opcionális referenciát ad, de élettartamot ez sem garantál. Egyik megoldás sem mentesít az alól, hogy a szerződésben rögzítsük: ki meddig él, és mely művelet invalidál.

19.3.4 Const correctness és logikai const

A `const T&` a hívó felé azt ígéri, hogy a függvény ezen a csatornán nem módosítja az objektumot. Ez nem azt jelenti, hogy az objektum örökre változatlan, és nem is azt, hogy más szálon nem módosítja valaki. A `const` csak a mi hozzáférési csatornánkat zárja le; a mögöttes objektum lehet nem-const, és más néven át módosítható. A `const_cast` a const eltávolítására csak akkor lenne „legális”, ha az eredeti objektum nem const; ha const, a módosítás nem definiált viselkedés. A gyakorlatban a `const_cast` írásra szinte mindig tervezési hiba.

A tagfüggvények const jelölése a logikai const (logical const) szerződése: a hívás nem változtatja meg a kívülről megfigyelhető állapotot. Cache, számláló vagy mutex miatt mégis módosulhat belső állapot; erre való a `mutable`:

```
class RequestCounter {
public:
    std::size_t count() const {
        ++calls_;          // mutable: a megfigyelhető állapot nem változik
        return calls_;
    }
private:
    mutable std::size_t calls_ = 0;
};
```

A `mutable` nem mentesít a szinkronizáció alól: ha több szál hívja a `count()`-ot, a `calls_` módosítása adatverseny lehet. A `const` tehát nem szálbiztonsági garancia; erről RAII lockok, condition variable és wait predicate (34.2) és C++ atomics, atomic_ref és publication (34.3) szól részletesen.

19.3.5 Temporary binding és az élettartam-hosszabbítás előképe

A `const T&` és a `T&&` rvalue-ra is köthető. Ilyenkor a kifejezésből egy temporary objektum jön létre, és a referencia élettartamához igazodva hosszabbodhat meg a temporary élettartama (lifetime extension):

```
const std::string& ok = std::string("abc");   // a temporary élettartama
 ↪  megnyúlik
const int& number = 42;                        // ugyanígy
```

A hosszabbítás azonban nem minden esetre érvényes. Nem hosszabbít, ha a referencia egy függvény visszatérési értékeként érkezik, és a függvény már visszaadta a temporary-t; nem hosszabbít, ha a temporary egy tag-referenciához kötődik a konstruktorban; és nem hosszabbít, ha a temporary egy másik kifejezés része, amelynek eredménye referencia. A klasszikus csapda:

```
struct Holder {
    const std::string& ref;
    Holder() : ref(std::string("temp")) {}   // NEM hosszabbít
};
```

A `Holder` példány `ref` tagja a konstruktor után dangling (érvénytelen): a temporary a konstruktor végén megszűnik. A javítás értéktag (`std::string ref;`), vagy olyan tervezés, ahol a referált objektum élettartama igazoltan túléli a `Holder`-t. A temporary-k és value category-k részletes tárgyalása Temporary, value category és dangling (20.4).

19.3.6 A referencia nem tulajdonlás és nem élettartam-garancia

A referencia nem tulajdonol. Nem zárja le a fájlt, nem szabadítja fel a memóriát, nem hosszabbítja meg egy tároló élettartamát, és nem akadályozza meg a tároló invalidációját. A szerződés mindössze annyi: a referált objektumnak élnie kell a referencia használatáig. Ezt a szerződést a nyelv nem ellenőrzi.

```
std::vector<std::string> v{"a"};
const std::string& r = v[0];
v.push_back("b");         // újraallokálhat; r invalidálódhat
// std::cout << r;        // nem definiált viselkedés lehet
```

Ugyanígy dangling lesz a referencia, ha lokális objektumra mutat, ha a scope-ból kilépő objektumra hivatkozik, vagy ha egy `std::move` után „moved-from” állapotú objektumot használunk (utóbbi nem feltétlenül dangling, de az értéke nem specifikált; lásd Move constructor, moved-from állapot és std::move (22.3)). Az ownership kérdését a RAII és a smart pointerek tárgyalják RAII wrapper file descriptorhoz és lockhoz (21.2), unique_ptr és egyedi tulajdonlás (21.3); a referencia ezekhez képest mindig nem tulajdonló nézet.

19.3.7 Kidolgozott példa: egy API élettartam-szerződése

Az alábbi program egy fordítási egységben mutatja a helyes és a tanítási célból hibás eseteket. A `bad_name()` hívása szándékosan ki van kommentelve: a visszaadott referencia dangling, a viselkedés nem definiált, ezért nincs garantált kimenet.

```
#include <iostream>
#include <string>
#include <vector>

// Helyes: értéket adunk vissza, a hívó tulajdonol.
std::string make_name() { return "worker-01"; }

// Helyes: a referált objektum a hívó tulajdonában van.
const std::string& name_of(const std::string& owner) { return owner; }

// Tanítási példa: lokális objektumra visszaadott referencia.
[[maybe_unused]] const std::string& bad_name() {
    std::string local = "worker-02";
    return local;   // UB: local a visszatéréskor megszűnik
}

struct Endpoint {
    const std::string& host;   // nem tulajdonol
    explicit Endpoint(const std::string& h) : host(h) {}
};

int main() {
    std::string owner = "db-01";
    const std::string& alias = owner;   // ugyanaz az objektum
    owner += ":5432";                   // az alias ezt látja
    std::cout << alias << '\n';         // db-01:5432
```

```
    const std::string& tmp = make_name();  // élettartam-hosszabbítás
    std::cout << tmp << '\n';              // worker-01

    Endpoint ep(owner);                    // helyes: owner túlél
    std::cout << ep.host << '\n';          // db-01:5432

    std::vector<std::string> v{"a"};
    const std::string& r = v[0];
    v.push_back("b");                      // r invalidálódhat
    // std::cout << r;                     // UB lehet
    return 0;
}
```

A buildparancs és a futtatás a tanuló laborjában:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O0 ref_demo.cpp -o ref_demo
./ref_demo
```

A definiált rész várható kimenete három sor: `db-01:5432`, `worker-01`, `db-01:5432`. Ez nem mért eredmény, hanem a definiált viselkedésből következő illusztráció; a kikommentelt sorok futtatása nem definiált viselkedés, ezért ott kimenetről nem beszélhetünk. A fordító a `bad_name`-re figyelmeztethet (`-Wreturn-local-addr`), de a figyelmeztetés hiánya nem bizonyíték a helyességre.

19.3.8 Tipikus hibák és javításuk

- **Referenciatag alapértelmezett konstruktorral.** A `Holder`-hez hasonló típusnál a temporary a konstruktor végén megszűnik. Javítás: értéktag, vagy a referált objektum élettartamának dokumentált garantálása.
- **Belső tagra visszaadott `const T&`, miközben a tároló invalidálható.** A hívó nem tudja, meddig érvényes a referencia. Javítás: érték visszaadása, vagy a szerződés és az invalidáló műveletek dokumentálása.
- **`const_cast` írásra.** Ha az eredeti objektum const, a módosítás nem definiált viselkedés. Javítás: a const helyes áttervezése, nem eltávolítása.
- **A `const T&` „biztonságos” feltevése szálak között.** A const csak a csatornát zárja; a mögöttes objektum más szálon módosulhat. Javítás: szinkronizáció vagy atomikus műveletek.
- **Null referencia feltételezése.** A `T&` nem lehet null; ha a hiányzó érték megengedett, pointer vagy `std::optional<T>` kell.
- **`auto` és `auto&` keverése.** Az `auto x = r;` másolatot készít, az `auto& x = r;` referenciát; a különbség a típuslevezetés témája auto, decltype és típuslevezetés (19.4).

19.3.9 Összefoglaló

A referencia névalias: létrejöttekor kötelező létező objektumhoz kötni, később nem lehet újrakötni, és nem lehet null. A `T&` a kötelező érvényes objektum, a `const T&` a nem módosító hozzáférés, a `T&&` az rvalue-kötés jelölése. A `const T&` és a `T&&` képes temporary élettartamát hosszabbítani, de csak közvetlen kötés esetén. A referencia nem tulajdonol, és nem garantál élettartamot: a dangling továbbra is a programozó szerződése. A pointer és a referencia közötti választás ezért API-szerződés kérdése: kötelező érvényes objektum esetén referencia, opcionális vagy újraköthető esetben pointer, értékszerű hiányzó állapot esetén `optional`.

19.3.10 Feladatok

1. **Magyarázó feladat.** Magyarázd el, miért nem fordul le az `int& r;` deklaráció inicializálás nélkül, és mi a különbség az `int& r = a; r = b;` valamint az `int* p = &a; p = &b;` szemantikája között. Ellenőrizd a választ egy kis fordítható programmal.

2. **Kódolási feladat.** Írj `std::string join(const std::vector<std::string>& parts, const std::string& sep)` függvényt, amely értéket ad vissza. Ezután tervezz egy `Config` osztályt, amelynek `raw()` tagfüggvénye `const std::string&`-t ad vissza. Dokumentáld, mi invalidálja a visszaadott referenciát.

3. **Diagnosztikai feladat.** Az alábbi kódban hol keletkezik dangling, és miért? Javítsd a lehető legrobusztusabb módon.

```
struct Session {
    const std::string& user;
    Session() : user("anonymous") {}
};
```

4. **Tervezési feladat.** Az `Endpoint` példában a `host` kötelező. Alakítsd át úgy, hogy a host hiányozhasson, és indokold, miért `std::optional<std::string>`-et, `const std::string*`-t vagy más reprezentációt választottál. Ne feledd az élettartam-szerződést.

19.3.11 Megoldások

19.3.11.1 1. megoldás

Az `int& r;` azért nem fordul le, mert a referencia deklarációjakor kötelező inicializálni: nincs „null referencia”, és nincs későbbi kötés. A `r = b` értékadás, nem újrakötés: az `a` értéke lesz `b` értéke, a referencia továbbra is `a`-t nevezi. A pointer ezzel szemben újraköthető: `p = &b` után `p` a `b` címét tartalmazza, az `a` változatlan. Ellenőrzés: a két programban írd ki `a`, `b`, `r`/`*p` értékét; a referencia esetén `a` megváltozik, a pointer esetén nem. A fordítási feltétel: `g++ -std=c++20 -Wall -Wextra`.

19.3.11.2 2. megoldás

```
#include <string>
#include <vector>

std::string join(const std::vector<std::string>& parts, const std::string& sep)
 ↪  {
    std::string out;
    for (std::size_t i = 0; i < parts.size(); ++i) {
        if (i != 0) out += sep;
        out += parts[i];
    }
    return out;
}
```

A `join` értéket ad vissza, a hívó tulajdonolja az eredményt. A `Config::raw() const std::string&`-je csak addig érvényes, amíg a `Config` példány él, és amíg a belső stringet nem módosítja vagy nem mozgatja el a `Config` destruktora vagy egy nem-const tagfüggvény. Ezt a dokumentációban rögzíteni kell; ha a hívó a `Config` megszűnése után is használja, dangling lesz. Ellenőrzés: a `join` eredményét egy blokk után is használva helyes; a `Config` referenciáját a `Config` megszűnése után használva nem definiált viselkedés.

19.3.11.3 3. megoldás

A `"anonymous"` string literálból a `std::string` temporary a konstruktor inicializáló listájában jön létre, és a konstruktor végén megszűnik. A `user` referencia ezért dangling lesz. A legrobusztusabb javítás értéktag:

```
struct Session {
    std::string user;
    Session() : user("anonymous") {}
};
```

Ekkor a `Session` tulajdonolja a stringet, és a `user` élettartama a `Session` élettartamához kötődik. Alternatíva egy `static const std::string default_user = "anonymous";` és `const std::string& user;` kötése: ez nem dangling, de megosztott, és a `Session` példányok ugyanarra a statikus objektumra hivatkoznak, ami más élettartam-szerződést jelent. Ellenőrzés: `Session s; std::cout << s.user;` az értéktag változatban definiált; a referencia változatban nem definiált viselkedés.

19.3.11.4 4. megoldás

Ha a hiányzó host értékszerű, és a példány tulajdonolhatja a nevet, `std::optional<std::string> host;` a legegyszerűbb: kifejezi a „van/nincs” állapotot, és a `Session` élettartamához köti a stringet. Ha a hostot nem akarjuk másolni, és a hívó garantálja, hogy a referált objektum túléli az `Endpoint`-ot, akkor `const std::string* host = nullptr;` is működik; ekkor a `nullptr` a hiány, de az élettartamot továbbra is dokumentálni kell. A `T&` azért nem alkalmas, mert nem tud „nincs” állapotot képviselni. A választás indoklása a tulajdonlási igény és a másolási költség alapján történik; mindkét esetben rögzíteni kell, mi invalidálja a hozzáférést.

19.4 auto, decltype és típuslevezetés

Ez az alfejezet a C-ből hozott deklarációs rutinra épül: ott a típust általában mi írtuk ki, itt viszont a fordító is megállapíthatja. Ez nem kényelmi „var”-szerűség, hanem két, pontosan definiált levezetési szabályrendszer: az `auto` a sablonargumentum-levezetést használja, a `decltype` pedig a deklarált típust vagy a kifejezés értékkategóriáját. A különbségtétel nem akadémikus: a rosszul megválasztott alak néma másolatokhoz, elveszett `const`-hoz vagy érvénytelen élettartamú referenciához vezet. A referenciák és a `const` C++-beli szemantikáját itt előfeltételként használjuk, részletesen lásd Reference, pointer és const C++-ban; a deklarációs szintaxis C-s alapjait Deklarációk, typedef és típusminősítők.

19.4.1 Az auto levezetése

Definíció. Az `auto` helyőrzővel írt deklarációban a fordító a deklarált típusból egy „mintát” képez, és a kezdőértékből a sablonargumentum-levezetés szabályai szerint tölti ki. A minta alakja dönti el, hogy a kezdőérték referenciája és minősítői megmaradnak-e: ha a mintában szerepel `&`, `&&` vagy `const`, akkor az adott tulajdonság megjelenik a levezetett típusban, ha nem, akkor elveszik. Ez a „template deduction for dummies” elv: `auto`-t helyettesítve egy `template<class T> void f(T)` paraméterébe ugyanazt kapjuk.

Következmény, amit a legtöbb kezdő elront: a csupasz `auto` **soha nem lesz referencia**, és a kezdőérték **top-level `const`/`volatile` minősítője elvész**. A referencia és a `const` megőrzéséhez a deklarációban kell jelezni.

deklaráció	kezdőérték	levezetett típus
`auto a = i;`	`int i`	`int`
`auto b = ci;`	`const int ci`	`int`
`auto c = ri;`	`int& ri`	`int`
`auto& d = ci;`	`const int ci`	`const int&`
`auto&& e = ci;`	`const int ci`	`const int&`
`auto&& f = 42;`	prvalue `int`	`int&&`
`const auto& g = i;`	`int i`	`const int&`
`auto* h = &i;`	`int i`	`int*`

Az `auto&&` alak külön figyelmet érdemel: lvalue kezdőértéknél lvalue referenciát, rvalue-nál rvalue referenciát ad (forwarding reference), ezért gyűjtőnévként használható ott, ahol nem akarunk dönteni a két eset között.

Kidolgozott példa. A következő program minden állítását `static_assert`-tel ellenőrzi, így a levezetett típus nem „szemre”, hanem fordítási időben ellenőrizhető. A kód C++20-at igényel, és a `<type_traits>` fejlécet használja.

```
#include <type_traits>

int main() {
    int i = 0;
    const int ci = 1;
    int& ri = i;
    const int& cri = ci;

    auto a  = ci;        // int: a const elvész
    auto b  = ri;        // int: referencia -> érték, másolat
    auto& c = ri;        // int&: a tényleges i-re hivatkozik
    auto& d = ci;        // const int&: const megmarad
    auto* p = &ci;       // const int*
    auto&& e = ci;       // const int&

    static_assert(std::is_same_v<decltype(a), int>);
    static_assert(std::is_same_v<decltype(b), int>);
    static_assert(std::is_same_v<decltype(c), int&>);
    static_assert(std::is_same_v<decltype(d), const int&>);
    static_assert(std::is_same_v<decltype(p), const int*>);
    static_assert(std::is_same_v<decltype(e), const int&>);
```

```
    static_assert(std::is_same_v<decltype(cri), const int&>);
}
```

Fordítás és ellenőrzés (a tanuló saját laborjában; itt nem futtattuk):

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o deduce deduce.cpp && ./deduce
# vagy
clang++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o deduce deduce.cpp && ./deduce
```

Ha bármelyik `static_assert` hamis, a program nem fordul le: ez erősebb ellenőrzés annál, mintha a futáskor nyomtatnánk típusneveket. Megjegyzendő korlát: a `decltype` itt deklarált típusra kérdez rá, ezért a `static_assert` a deklaráció helyességét igazolja, nem azt, hogy a mögöttes futásidejű érték mit tartalmaz.

Két további, gyakran előforduló eset. Az `auto x = {1, 2, 3};` alakban a kezdőérték `std::initializer_list<int>`, nem `int` és nem tömb; ez a `auto` kivételes szabálya. Függvény visszatérési típusaként az `auto` minden `return` utasításból ugyanazt a típust köteles levezetni, és braced-init-listből nem lehet visszatérési típust levezetni.

19.4.2 decltype: név és kifejezés

Definíció. A `decltype(e)` a zárójelben álló kifejezés típusát adja vissza, de a kifejezést nem értékeli ki (unevaluated operand). A szabály kétágú:

1. Ha `e` zárójel nélküli azonosító-kifejezés (változó, paraméter, structured binding tag) **vagy zárójel nélküli osztálytag-elérés** (`obj.member`, `p->member`), akkor az eredmény a **deklarált típus**, a referencia és a `const` megtartásával.
2. Minden más esetben az értékkategória dönt: lvalue → `T&`, xvalue → `T&&`, prvalue → `T`.

A zárójel tehát nem kozmetika. `decltype(i)` az `int`, `decltype((i))` viszont `int&`, mert a zárójelezett alak nem azonosító-kifejezés, hanem lvalue kifejezés.

```
int i = 0;
const int ci = 1;

decltype(i)      a = i;    // int
decltype((i))    b = i;    // int&   <- a plusz zárójel számít
decltype(ci)     c = ci;   // const int
decltype((ci))   d = ci;   // const int&
decltype(i + 1)  e = 0;    // int (prvalue)
// tagelérés: struct S { int member; }; S obj{1};
// decltype(obj.member)    -> int   (zárójel nélküli tagelérés: a tag deklarált
 ↪  típusa)
// decltype((obj.member))  -> int&  (zárójelezve lvalue kifejezés; const
 ↪  objektumnál const int&)
decltype(i = 1)  f = i;    // int&  (az értékadás lvalue)
```

Ez a szabály teszi lehetővé, hogy generikus kódban a visszatérési típus tökéletesen kövesse a paramétert, ugyanakkor könnyű vele meglepni magunkat: egy makróba tett `decltype((x))` hirtelen referenciát ad, ahol értéket vártunk.

19.4.3 decltype(auto) és a visszatérés csapdái

Definíció. A `decltype(auto)` helyőrző a kezdőértékre (vagy a `return` operandusára) a `decltype` szabályait alkalmazza az `auto` szabályai helyett. Célja a tökéletes visszaadás: referenciát adó kifejezésből referenciát, értékből értéket csinál.

A veszély pontosan itt van: a `decltype(auto)` **nem nyújtja meg** a lokális változó élettartamát, csak a típust követi.

```
// TANÍTÁSI PÉLDA - nem definiált viselkedés, nincs garantált kimenet
decltype(auto) bad() {
    int local = 42;
    return (local);        // (local) lvalue -> int&, a local kilépéskor
     ↪  megszűnik
}

// Helyes változat: érték visszaadása
int good() {
    int local = 42;
    return local;          // int, másolat
}

// decltype(auto)-val is helyes, mert itt név áll zárójel nélkül
decltype(auto) also_good() {
    int local = 42;
    return local;          // decltype(local) == int
}
```

A `bad()` eredményét használó hívó kód érvénytelen élettartamú objektumra hivatkozik; ez a dangling esete, és UB, nem „véletlen szemét”, amit meg lehet figyelni. A javítás mindig szerződés kérdése: ha a függvény referenciát ad vissza, annak a hívó élettartamához kell kötnie, nem a függvény lokálisához.

19.4.4 Structured binding és a rejtett backing object

Definíció. Az `auto [a, b] = expr;` deklaráció egy rejtett változót (`e`, a backing object) hoz létre `expr`-ből, majd az `a`, `b` neveket az `e` részeihez köti (tömb esetén elemekhez, `std::tuple`-szerű típusnál `get<>` hívás eredményéhez, osztálynál nyilvános tagokhoz). Az `a`, `b` **nem önálló objektumok**, hanem az `e` alterületeit jelölő nevek; típusukat az `e` típusa és a hozzáférés módja határozza meg.

Ebből következik a gyakorlati szabály, két külön kérdéssel. (1) **Mi az `e`?** Ha `auto`-t írunk, az `e` a forrás másolata; ha `auto&` vagy `const auto&` áll, az `e` referencia, és a nevek az eredeti objektumot aliasolják. (2) **Mit tartalmaz az `e`?** A másolat csak a forrás *értéktagjait* választja le. Ha a forrás referenciát vagy pointert tárol — `std::tie(a, b)` eredménye, `std::tuple<int&>`, pointertag —, akkor a másolat is ugyanarra az objektumra hivatkozik, és a neveken át végzett írás a külső objektumot módosítja. Az `auto` tehát csak akkor „választja le a forrást”, ha annak minden tagja érték, mint a lenti `std::pair<const std::string, int>` esetén.

```
// binding_alias.cpp - C++20: auto [..] másolata is aliasolhat
#include <cassert>
#include <tuple>
```

```cpp
int main() {
    int a = 1, b = 2;
    auto [x, y] = std::tie(a, b);    // e: tuple<int&, int&> másolat
    x = 10;                          // a másolt tuple referenciáján át a-t írja
    assert(a == 10);

    std::tuple<int&> t{b};
    auto [r] = t;                    // e: tuple<int&> másolat, r mégis b-re
     ↪ hivatkozik
    r = 20;
    assert(b == 20);

    struct P { int v; int* p; };
    P src{1, &a};
    auto [v, p] = src;               // e: P másolat; v független, p ugyanoda
     ↪ mutat
    v = 5; *p = 30;
    assert(src.v == 1 && a == 30);
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic binding_alias.cpp` figyelmeztetés nélkül fordul, és a futás egyik `assert`-et sem sérti: az `x`, `r`, `*p` írása a külső `a`, `b`, `a` objektumot változtatta, a `v` írása nem hatott a `src.v`-re.

```cpp
#include <map>
#include <string>

void demo(std::map<std::string, int>& m) {
    auto [k, v] = *m.begin();        // e: másolat a pair-ről
    v = 99;                          // csak a másolatot módosítja, m változatlan

    auto& [k2, v2] = *m.begin();     // e: referencia a map elemére
    v2 = 99;                         // most már a tároló eleme változik

    for (const auto& [key, value] : m) {
        (void)key; (void)value;      // olvasás, másolás nélkül
    }
}
```

A háttérben álló objektum élettartama a deklaráció hatóköre: ha `expr` ideiglenes volt, és referenciát kötünk hozzá a deklarációban, a nyelv megnyújtja az `e` élettartamát; ha viszont az `expr` egy függvényhívás eredményeként **referenciát** ad vissza egy már megszűnt objektumra, a binding ezt nem tudja megjavítani.

19.4.5 Range-for: másolat és referencia

A range-for hátterében a szabvány egy `auto&&` kötést készít a tartományra, majd iterátorokkal járja be. A ciklusváltozó deklarációja ezért ugyanazt a döntést kéri, mint az `auto`:

```cpp
#include <string>
#include <vector>
```

```
struct Metric { std::string name; long value; };

void bump(std::vector<Metric>& v) {
    for (auto m : v) { ++m.value; }        // HIBA: a másolatot növeli
}

void bump_fixed(std::vector<Metric>& v) {
    for (auto& m : v) { ++m.value; }       // helyes: a tényleges elem
}

long total(const std::vector<Metric>& v) {
    long sum = 0;
    for (const auto& m : v) { sum += m.value; }  // olvasás, másolás nélkül
    return sum;
}
```

A `for (auto m : v)` alak helyes, ha csak olvasunk és az elem kicsi, de minden iterációnál másol; nagy `std::string`-et tartalmazó struktúránál ez mérhető költség. A `for (auto& m : v)` nem másol; konstans tartományon is lefordul, mert az `auto&` megőrzi a levezetett `const`-ot (`const std::vector<int>` elemein `const int&`), csak az elemek módosítása tiltott (az értékadás fordítási hibát ad); más szóval külön kérdés a referencia levezetése és az azon keresztüli írhatóság; olvasáshoz a `const auto&` a kifejezőbb választás, mert jelzi, hogy nem módosítunk. Élettartam szempontjából a ciklus saját maga által létrehozott ideiglenes tartományát a nyelv a ciklus végéig életben tartja (C++20-ban így definiált); a C++23 ezt a szabályt tovább pontosítja a range-init kifejezésben keletkező ideiglenesekre, ennek részletei verziófüggők, és itt nem tárgyaljuk őket. Az viszont nem megengedett következtetés, hogy egy függvényből visszakapott referencia automatikusan megnyúlik: `const auto& r = makeVector()[0];` után `r` érvénytelen, mert az ideiglenes vektor a teljes kifejezés végén megszűnik.

19.4.6 Alias using és olvasható típusjelölések

A `using` kulcsszóval típusálneveket vezethetünk be; ez a `typedef` modern megfelelője, és sablonoknál az egyetlen olvasható alak. A `using` nem hoz létre új típust, csak nevet ad egy létezőnek, ezért a levezetés szempontjából átlátszó.

```
#include <cstddef>
#include <vector>

using Size = std::size_t;
using Callback = void (*)(int);                 // függvénypointer alias
template<class T> using Vec = std::vector<T>;   // sablon-alias

Size count(const Vec<int>& v);                  // olvashatóbb, mint
 ↪  std::size_t
```

A `using` és az `auto` együtt különösen hasznos hosszú visszatérési típusoknál, illetve ott, ahol a típusnév a fejlécben zajt jelent. A fejlécben továbbra is érdemes kiírni a típust vagy aliast használni, mert az `auto` a deklaráció helyén derül ki, és a fejlécet olvasó számára nem látszik.

19.4.7 Tipikus hibák és javításuk

- **Elveszett `const`/referencia.** `auto x = ci;` után `x` módosítható `int`, és nem aliasol. Javítás: `const auto& x = ci;`, ha csak olvasni akarunk, vagy `auto&`, ha módosítani.
- **Néma másolás ciklusban.** `for (auto m : v)` nagy elemeknél pazarló; `auto&` vagy `const auto&` a szándék kifejezése.
- **`decltype((x))` referencia-visszaadás.** Generikus kódban a zárójel hirtelen referenciát ad; ha értéket akarunk, `decltype(x)` vagy `std::decay_t`.
- **`decltype(auto)` lokális referenciával.** A `return (local);` dangling referenciát ad; a helyes megoldás érték visszaadása, vagy a referencia élettartamának dokumentált külső biztosítása.
- **Structured binding másolaton.** `auto [k, v] = *it;` után a `v` módosítása nem hat a tárolóra; `auto& [k, v] = *it;` kell.

19.4.8 Összefoglaló

Az `auto` a sablonargumentum-levezetés szabályait követi: csupasz alakban elveszti a referenciát és a top-level `const`-ot, ezért a szándékot a mintában (`&`, `const&`, `&&`) kell kifejezni. A `decltype` zárójel nélküli névre a deklarált típust adja, zárójelezett kifejezésre az értékkategóriából következtet, és nem értékeli ki az operandust. A `decltype(auto)` a `decltype` szabályait viszi a visszatérési típusba, ami hasznos forwardingnál, de nem nyújtja meg a lokális élettartamát. A structured binding mögött rejtett backing object áll, a range-for ciklusváltozója pedig ugyanazt a másolat-referencia döntést igényli. A pontos típus ellenőrzésének olcsó eszköze a `static_assert(std::is_same_v<...>)`.

A szabvány pontos paragrafusaira ebben az egységben nem hivatkozunk ellenőrzött forrásként: a megadott forrásjegyzet ehhez a részhez nem tartalmaz tételt, ezért a fenti állítások a bemutatott kód és a levezetési szabályok szintjén értelmezendők, nem hivatalos szöveghely-igazolással.

19.4.9 Feladatok

1. **Típusmegállapítás (fogalmi).** Adott: `int i = 0; const int ci = 1; int& ri = i; const int& cri = ci;`. Határozza meg a következő deklarációk levezetett típusát, és írjon mindegyikhez `static_assert`-et: `auto a = ci;`, `auto& b = ci;`, `auto&& c = ci;`, `auto&& d = 42;`, `decltype(ri)`, `decltype((ri))`, `decltype(ci + 1)`.
2. **Kódolási.** A `bump` és `total` függvényeket tartalmazó példát alakítsa át úgy, hogy sehol ne keletkezzen felesleges másolat, a módosító változat tényleg módosítson, és a const tartományon is forduljon le. Indokolja minden `auto`-alak választását.
3. **Diagnosztikai.** Az alábbi függvény fordító által elfogadott, de hibás. Mutassa meg, miért, adja meg a javított változatot, és mondja meg, milyen eszközzel lehetne a hibát futás előtt elkapni:

```
decltype(auto) firstPositive(const std::vector<int>& v) {
    for (const auto& x : v)
        if (x > 0) return (x);
    return 0;
}
```

Elsőként azt a kérdést válaszolja meg, **miért nem fordul** a függvény. Második lépésként írja meg a javított változatot `std::optional<int>` visszatéréssel, és tesztelje üres, csupa negatív és pozitív elemet is tartalmazó vektorral. Harmadikként mutasson külön példát olyan referenciára, amelynek élettartama valóban a vektoréhez kötött (a vektor megszűnése vagy átméretezése után használva).

4. **Tervezési (alias).** Vezessen be `using` aliasokat egy olyan fejlécben, amely `std::map<std::string, std::vector<int>>` típust használ paraméterként és visszatérési típusként, és indokolja, miért nem `auto`-t ír a fejlécbe.

19.4.10 Megoldások

1. megoldás. `a` típusa `int` (a `const` elvész, mert a mintában nincs `const`). `b` típusa `const int&` (a `auto&` megőrzi a `const`-ot). `c` típusa `const int&`: az `auto&&` lvalue kezdőértéknél lvalue referenciát ad, a `const` megmarad. `d` típusa `int&&`, mert prvalue-ból rvalue referencia lesz. `decltype(ri)` az `int&` (deklarált típus), `decltype((ri))` szintén `int&`, mert a zárójelezett név lvalue; a két alak itt egybeesik, de nem minden esetben. `decltype(ci + 1) int`, mert az összeg prvalue. Ellenőrzési feltétel: a `static_assert`-ek forduljanak le `-std=c++20` módban, és a program ne tartalmazzon futásidejű tesztet, mert a kérdés tisztán típusszintű.

2. megoldás. A módosító változat `for (auto& m : v)`, mert a ciklusváltozónak a tároló elemét kell aliasolnia. Az összesítő változat `for (const auto& m : v)`, mert csak olvasunk, így nincs másolat és a `const` tartomány is elfogadott. Ha a `Metric` nagyon kicsi és olcsón másolható (például két `int`), a `for (auto m : v)` is védhető, de ezt mérés vagy dokumentált költségmodell nélkül nem állítjuk. Ellenőrzési feltétel: a módosító függvény után a tároló elemei tényleg megváltoztak, az összesítő pedig `const std::vector<Metric>&` paraméterrel is fordul.

3. megoldás. A függvény **nem fordul le**: a `return (x);` zárójelezett lvalue kifejezés, ezért a `decltype(auto)` ezen az ágon `const int&`-et vezet le, a `return 0;` ág viszont `int`-et. A `decltype(auto)` visszatérési típusa minden `return`-ből ugyanarra kell, hogy adódjon, ezért a GCC hibát ad („inconsistent deduction for auto return type: ‘const int&’ and then ‘int’”). Ez kötelező fordítási hiba, nem futásidejű probléma. Fontos: az `x` referencia a **vektor elemére**, nem a ciklus lokális változójára; a referencia saját neve a ciklus végén megszűnik, de a hivatkozott elem addig él, amíg a vektor — tehát a hiba nem *stack-use-after-scope*, és ebből a programból ilyen diagnózis nem következik. Javított változat, amely a „nincs találat” esetet is kifejezi:

```
std::optional<int> firstPositive(const std::vector<int>& v) {
    for (int x : v)
        if (x > 0) return x;
    return std::nullopt;
}
```

Várt eredmények: üres vektor és `{-1, -2}` esetén `std::nullopt`; `{-1, 5, 7}` esetén `5`. (Ellenőrizve: GCC 13.3, `-std=c++20`, ASan+UBSan, hibátlan.) A vektor élettartamától függő referencia külön példa: `const int& r = v[1]; v.push_back(...);` — az átméretezés érvényteleníti `r`-t (ASan `heap-use-after-free`), és ez a valódi dangling eset.

4. megoldás. Például:

```
#include <map>
#include <string>
#include <vector>

using IntVec = std::vector<int>;
using Table = std::map<std::string, IntVec>;

Table load(const Table& input);
```

A fejlécben azért nem `auto`, mert a visszatérési típus a deklaráció helyén derül ki, a fejlécet olvasónak viszont a szerződést kell látnia, és a fordítónak is stabil szignatúrára van szüksége a külön fordított egységekhez. Az alias nem új típus, ezért az `IntVec` és a `std::vector<int>` felcserélhető, a levezetést nem befolyásolja. Ellenőrzési feltétel: a fejléc önállóan include-olható, és a fordítása `-std=c++20` módban figyelmeztetés nélkül sikerül.

19.5 Overload, konverzió és explicit interfész

A C-ben egy függvénynév egyetlen szignatúrához tartozik: ha kétfajta paraméterrel akarunk ugyanarra a logikai műveletre hívni, külön nevet adunk (`print_int`, `print_str`), vagy `_Generic`-et használunk, amely a vezérlő kifejezés típusa alapján *kiválaszt*, de nem rangsorol (lásd static inline, _Generic és headeres eszköztár (10.3)). A C++ ugyanezt a döntést a fordítóra bízza, de nem szabadon: az overload resolution egy szigorúan definiált, fordítás idejű rangsorolás. Ez egyszerre kényelem és kockázat. Kényelem, mert a hívó oldalon nem kell típusnevet írni; kockázat, mert minden implicit konverzió részt vesz a döntésben, és a „legjobb” jelölt nem feltétlenül az, amit a szerző szándékozott. Ennek az egységnek a célja, hogy a jelöltválasztást mechanikusan le tudd vezetni, felismerd a csapdákat (különösen a `0`/`NULL` és a pointer→`bool` esetét), és olyan interfészt tervezz, amelyben a konverzió ára látható: `explicit`, erős típusok, lista-inicializálás. A C++20 alapú tárgyalás a korábbi részekre épül: az integrál promóciók és a signedness már C-ben is átírták a típusokat (Promóciók, signedness és túlcsordulás (4.2)), a típuslevezetés pedig C++-ban tovább tágítja a jelöltek körét (auto, decltype és típuslevezetés (19.4)).

19.5.1 Overload candidate, viability és conversion ranking

Három fogalmat kell élesen szétválasztani. A **jelöltlista (candidate set)** azokból a deklarációkból áll, amelyeket a névfeloldás a hívás helyén talál; a lista a hívás előtt, a nevek alapján áll össze, tehát egy nem megfelelő paraméterezésű függvény is bekerülhet. Az **életképesség (viability)** szűrője: az argumentumszám egyeztethető (a default argumentumok beszámítanak), és minden argumentumhoz létezik implicit konverziós sorozat (implicit conversion sequence, ICS). A **legjobb életképes** jelölt kiválasztásának első és leggyakrabban döntő lépése az argumentumonkénti összehasonlítás: az a jelölt jobb, amelyik minden argumentumban nem rosszabb a másiknál, és legalább egyben szigorúan jobb. Ez azonban egyszerűsítés, nem a teljes definíció: ha a konverziós sorozatok alapján döntetlen van, a szabvány függvényszintű döntő szabályokkal folytatja (lásd lent, C++20 [over.match.best]). Ha a végén sincs egyetlen legjobb jelölt, a hívás **kétértelmű (ambiguous)**, és ez *fordítási hiba*, nem futásidejű döntés — ellentétben például a dinamikus dispatch-csel.

Az ICS rangsora kategóriákból áll. A kategória az **első szintű szűrő**: a jobb kategóriájú sorozat mindig nyer. Azonos kategórián belül viszont további, döntő szabályok következnek (lásd lent):

szint	mi tartozik ide	példa
Exact Match	identity, lvalue→rvalue, tömb→pointer, függvény→pointer, minősítés-konverzió	`int`→`int`, `int*`→`const int*`
Promotion	integrál promóció, lebegőpontos promóció	`char`→`int`, `float`→`double`

szint	mi tartozik ide	példa
Conversion	integrál konverzió, lebegő integrál, pointerkonverzió, boolean konverzió	`unsigned`→`int`, `double`→`int`, `T*`→`void*`
User-defined	pontosan egy felhasználói konverzió (konstruktor vagy konverziós operátor)	`const char*`→`std::string`
Ellipsis	`...`	—

Két fontos következmény. Egyrészt a promóció olcsóbb a konverziónál, ezért a `char` argumentum `int` paramétert választ a `long` paraméter helyett — ugyanaz a szabály, amely C-ben a `printf` varargsát is mozgatja. Másrészt az azonos kategória önmagában nem dönt. Ha `unsigned int` argumentummal hívunk egy `int`, egy `long` és egy `double` paraméterű overloadot, mindhárom *Conversion*, és ezek közül egyik döntő szabály sem emel ki egyet, tehát a hívás kétértelmű. Ezt a fordító jelzi; a javítás nem az, hogy „valamelyiket mégis válassza”, hanem hogy a hívó oldalon típusossá tesszük az értéket, vagy megszüntetjük a fölösleges overloadot. A kétértelműség itt a konkrét jelöltekből következik, nem abból, hogy azonos rangon belül soha nem lenne preferencia.

```
// overload_rank.cpp - C++20
#include <cstdio>
#include <string_view>

void pick(int)              { std::puts("pick(int)"); }
void pick(long)             { std::puts("pick(long)"); }
void pick(double)           { std::puts("pick(double)"); }
void pick(std::string_view) { std::puts("pick(string_view)"); }

int main() {
    pick('a');   // char -> int promóció
    pick(0);     // int, exact match
    pick(0L);    // long, exact match
    pick(1.0f);  // float -> double promóció
    pick("x");   // csak a string_view életképes (user-defined)
    // pick(0u); // HIBA: unsigned -> int/long/double mind Conversion
}
```

Azonos rangon belül a szabvány több döntő szabályt ad (C++20 [over.ics.rank]). A három leggyakoribb:

- ha az egyik sorozat a másik valódi részsorozata, az rövidebb nyer: `int*` argumentumnál az `int*` paraméter (identitás) jobb a `const int*` paraméternél (identitás + minősítés-konverzió), noha mindkettő *Exact Match*;
- a pointer vagy tagpointer `bool`-ra konvertálása rosszabb minden más konverziónál: `int*` argumentumnál a `void*` és a `bool` paraméter is *Conversion*, mégis a `void*` nyer;
- referenciakötésnél, ha mindkettő identitás-kötés, rvalue argumentumhoz az rvalue-referencia (`&&`) jobb a `const&`-nél; lvalue-hoz pedig a `&&` nem is köthető, így ott csak a `const&` életképes.

```
// overload_same_rank.cpp - C++20: azonos rang, mégis egyértelmű választás
#include <cstdio>
#include <string>
```

```
void q(int*)        { std::puts("q(int*)"); }
void q(const int*)  { std::puts("q(const int*)"); }

void v(void*)       { std::puts("v(void*)"); }
void v(bool)        { std::puts("v(bool)"); }

void r(const std::string&) { std::puts("r(const std::string&)"); }
void r(std::string&&)      { std::puts("r(std::string&&)"); }

int main() {
    int x = 0;
    int* p = &x;
    q(p);                  // mindkettő Exact Match; a minősítés nélküli nyer
    v(p);                  // mindkettő Conversion; a pointer->bool rosszabb
    std::string s = "a";
    r(s);                  // lvalue: csak a const& köthető
    r(std::string{"b"});   // rvalue: mindkettő identitás-kötés; a && nyer
}
```

A program kimenete a `lab-gcc` profilban sorrendben `q(int*)`, `v(void*)`, `r(const std::string&)`, `r(std::string&&)`. Ugyanitt az `overload_rank.cpp` kimenete `pick(int)`, `pick(int)`, `pick(long)`, `pick(double)`, `pick(string_view)`, a kikommentezett `pick(0u)` sor visszakapcsolva pedig `call of overloaded 'pick(unsigned int)' is ambiguous` fordítási hibát ad.

Ha az argumentumonkénti összehasonlítás döntetlen, mert a jelöltek konverziós sorozatai páronként egyformák, a szabvány (C++20 [over.match.best]) további, függvényszintű szabályokat alkalmaz. A legfontosabbak:

- a nem sablon függvény jobb, mint egy sablonból generált specializáció;
- két sablonspecializáció közül a *specializáltabb* sablon nyer (partial ordering);
- C++20-ban azonos szignatúrájú sablonok közül a *megkötöttebb* (more constrained) nyer;
- felhasználói konverzióval történő inicializálásnál a konverzió *utáni* standard konverziós sorozat dönt;
- összehasonlító operátoroknál a nem átírt (nem `<=>`/fordított) jelölt jobb az átírtnál.

```
// overload_tiebreak.cpp - C++20: azonos konverziós sorozat, függvényszintű
 ↪  döntés
#include <concepts>
#include <cstdio>

void f(int)                    { std::puts("f(int) - nem sablon"); }
template<class T> void f(T)    { std::puts("f(T) - sablon"); }

template<class T> void g(T)    { std::puts("g(T)"); }
template<class T> void g(T*)   { std::puts("g(T*) - specializáltabb"); }

template<class T> void h(T)                    { std::puts("h(T)"); }
template<std::integral T> void h(T)            { std::puts("h(integral T) -
 ↪  megkötöttebb"); }

int main() {
    int x = 0;
```

```
    f(x);    // mindkét jelölt Exact Match: a nem sablon nyer
    g(&x);   // mindkettő Exact Match: a specializáltabb sablon nyer
    h(x);    // mindkettő Exact Match: a megkötöttebb (constrained) nyer
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic overload_tiebreak.cpp` figyelmeztetés nélkül fordul; a kimenet sorrendben `f(int) - nem sablon`, `g(T*) - specializáltabb`, `h(integral T) - megkötöttebb`.

A sablonok levezetését és részleges rendezését a Függvénysablon, instantiation és deduction (26.1), a megkötések szerinti választást a Concepts, requires és constrained overload (27.2) tárgyalja részletesen.

A `pick("x")` eset érdekes: a `const char*` nem konvertálható `int`-té, mert a standard konverziós sorozat nem tartalmazhat két lépést (pointer→`bool`→`int` nem megengedett lánc), így egyetlen életképes jelölt marad, és a user-defined sorozat nyer — versenytárs nélkül a legalacsonyabb szint is elég. A fordítás és a futtatás a tanuló laborjában:

```
g++ --version
g++ -std=c++20 -Wall -Wextra -Wpedantic -Wconversion -Wold-style-cast \
    overload_rank.cpp -o overload_rank
```

Buildjegyzőkönyv (`lab-gcc` profil). Mindkét program a fenti kapcsolókkal figyelmeztetés nélkül fordult, a kimeneteket fentebb közöltük. A `-Wconversion` hasznos, de nem helyettesíti a brace-inicializálást: az assignment konverziókat figyelmeztetésként jelzi, a brace-init a narrowingot hibaként tiltja.

19.5.2 Default argumentum, nullptr és a 0/NULL csapda

A default argumentum nem része a függvény típusának, csak a hívás helyén, a statikus típus alapján egészíti ki az argumentumlistát. Overload szempontjából viszont részt vesz az életképességben, ezért két látszólag különböző szignatúra összeérhet:

```
void connect(int fd, int timeout_ms = 0);
void connect(int fd);
// connect(3);  // kétértelmű: mindkettő életképes, mindkettő exact match
```

Ez a fajta kétértelműség különösen rossz, mert a hiba a *fejléc módosításakor* keletkezik, a hívó kódban. A helyes minta: a default argumentumot ne kombináld azzal a szignatúrával, amelyből a default elhagyásával ugyanaz a hívás lenne.

A nullpointer körül három különböző dolog él együtt. A `nullptr` típusa `std::nullptr_t`, amely bármely pointer típusra konvertálható, egész típusra viszont nem; ezért a pointer paramétert választja, és soha nem esik bele az egész overloadba. A `0` literál típusa `int`, tehát az `int` paramétert választja exact match-csel, és ha nincs `int` overload, akkor a pointer felé *konverzió* minősül, ami versenyben lehet más konverziókkal. A `NULL`makró definíciója implementációfüggő: gyakran `0`, `0L` vagy GCC alatt `__null`, ezért ugyanaz a hívás különböző fordítókon mást választhat, vagy kétértelmű lehet.

```
void send(const char*);   // 1
void send(int);           // 2
send(0);        // 2: int, exact match
send(nullptr);  // 1: nullptr_t -> const char*
// send(NULL);  // hordozhatatlan: a NULL definíciójától függ
```

A tanulság nem az, hogy „a `NULL` rossz”, hanem hogy **ne terhelj túl egyszerre pointer és egész paraméterre ugyanabban aszerepben**. Ha mégis muszáj, a hívó oldalon `nullptr`-t írj, az egész oldalt pedig tedd erős típussá (`enum class`, `struct Port { explicit Port(int); }`). Ugyanide tartozik a pointer→`bool` csapda, amely a legtöbb C++ knnyvtárban előfordul:

```
void log(bool enabled);                  // 1
void log(const std::string& message);    // 2
log("indítás");  // 1-et választja! const char* -> bool standard konverzió
```

A `const char*`→`bool` standard konverzió, a `const char*`→`std::string` user-defined; a rangsor szerint az előbbi nyer. A `std::string_view` paraméter sem segít, mert az is user-defined sorozat. A javítás szerkezeti: a `bool` paramétert ne hagyd pointerrel versenyezni — használj `enum class`-t, vagy `void log(const char*) = delete;` deklarációval tedd a hívást szándékosan hibássá.

19.5.3 Explicit interfész: explicit konstruktor és konverzió

Az egyparaméteres, nem `explicit` konstruktor egyben konverziós operátor is: részt vesz az implicit konverziós sorozatokban, és ezzel a jelöltválasztásban. Az `explicit` ezt kiveszi a játékból — az explicit konstruktor csak direkt inicializálással hívható, implicit konverzióként nem. Ugyanez igaz az `explicit operator bool()` alakú konverziós operátorra, azzal a kivétellel, hogy a *kontextuális* `bool`-konverzió (`if`, `while`, `!`, `&&`) továbbra is működik, mert ott a szabvány kifejezetten engedi.

```
struct Port {
    explicit Port(int number) : number_{number} {}
    int value() const noexcept { return number_; }
private:
    int number_;
};

Port a{8080};         // OK: direkt inicializálás
// Port b = 8080;     // hiba: explicit miatt nincs implicit konverzió
```

A szabály arra késztet, hogy a döntést írásban hozd meg: az `explicit` akkor maradjon el, ha a konverzió *szemantikailag* azonos (pl. `std::string const char*`-ból), egyébként alapértelmezésben `explicit`. C++20-ban az `explicit(bool)` feltételes explicitét is használhatod, ha a típusparamétertől függ, hogy a konverzió veszélyes-e; ennek részletei az inicializálásról szóló részhez tartoznak (Az inicializálás különböző alakjai (20.1)).

19.5.4 A C++ kasztok kategóriái

A C-stílusú kaszt (`(T)x`) több kasztot próbál egymás után, ezért elrejti a szándékot. A C++ négy nevesített kasztja más-más szerződést fejez ki, más előfeltétellel és más hiba móddal:

kaszt	tipikus használat	előfeltétel	hiba módja
`static_cast`	numerikus konverzió, `void*`→`T*`, explicit konstruktor, fel/lefele kaszt	lefele kasztnál a dinamikus típus valóban a leszármazott	rossz dinamikus típusnál nem definiált viselkedés
`const_cast`	`const`/`volatile` levétele	az objektum nem *valóban* const	módosítás esetén nem definiált viselkedés

kaszt	tipikus használat	előfeltétel	hiba módja
`reinterpret_cast`	pointer egész, bájtszintű újraértelmezés	a reprezentáció értelmezése dokumentált	aliasing-szabálysértés, nem hordozható
`dynamic_cast`	polimorf típus ellenőrzött kasztja	polimorf alaposztály, RTTI engedélyezve	pointerre `nullptr`, referenciára `std::bad_cast`

```
struct Base { virtual ~Base() = default; };
struct Derived : Base { void only_derived(); };

void f(Base* b) {
    if (auto* d = dynamic_cast<Derived*>(b)) d->only_derived(); // ellenőrzött
    // static_cast<Derived*>(b)->only_derived();  // feltételez: ha nem Derived,
     ↪  UB
}

const int c = 7;
int* p = const_cast<int*>(&c);
// *p = 9;  // UB: c valóban const objektum
```

A `reinterpret_cast` pointer és `std::uintptr_t` között használható címként, de típus-punningra nem: két különböző típusú objektum ugyanazon a tárolón való átértelmezése aliasing-szabályba ütközik, helyette `std::memcpy` kell (Effective type, aliasing és objektumreprezentáció (5.4)). A `dynamic_cast` futásidejű költsége valós, ezért forró útvonalon nem ez a hibakezelés eszköze; ott `std::variant`/`visit` a természetes alternatíva.

19.5.5 Narrowing és lista-inicializálás

A brace-inicializálás (`{}`) szigorúbb szerződést kínál, mert a *narrowing* konverziókat fordítási hibává teszi. Narrowing például: lebegőpontosból egészbe; hosszabb lebegőpontosból rövidebbre; olyan egész→egész, amely nem tudja a forrás minden értékét ábrázolni; és egész→lebegőpontos — kivéve, ha a forrás konstans kifejezés, és az érték pontosan ábrázolható a célban.

```
int a = 3.9;     // a == 3, csendben csonkol (nem UB)
// int b{3.9};  // hiba: narrowing
// char c{300}; // hiba: 300 nem fér el char-ban
double d{1};     // OK: 1 pontosan ábrázolható
Port p{70000};  // ha Port int-et vár: OK; ha std::uint16_t-t: narrowing hiba
```

A `{}` tehát nem minden konverziót tilt meg (az `int`→`long` például megengedett), csak az érték-vesztőket; és nem véd a user-defined konverziókkal szemben sem. Két további következmény: az üres `{}` érték-inicializál, a `{}` pedig előnyben részesíti az `initializer_list` konstruktorokat — ezért a `std::vector<int> v(3)` és `v{3}` jelentése eltér; a tárolók tárgyalására a későbbi részben térünk vissza (std::array, std::vector és folytonos tárolás (28.1)).

19.5.6 Tipikus hibák és javításuk

- **`NULL`-lal hívott pointer/egész overload.** Tünet: fordítónként változó vagy kétértelmű hívás. Javítás: `nullptr`, illetve a két overload közül az egyik megszüntetése.

- **Pointer mellett `bool` paraméter.** Tünet: sztring literál „igaz” ágba fut. Javítás: `enum class`, `= delete` guard, vagy tag-típus.
- **Default argumentum + rövidebb overload.** Tünet: kétértelműség a hívónál. Javítás: egyetlen szignatúra, esetleg overload helyett névvel ellátott alternatíva.
- **Két azonos rangú konverzió.** Tünet: `unsigned` argumentumnál kétértelműség. Javítás: típusos hívás, nem új overload.
- **C-stílusú kaszt.** Tünet: `const` csendben lekerül, vagy pointerből egész lesz. Javítás: nevesített kaszt, `-Wold-style-cast`.
- **Azt hinni, hogy az overload visszatérési típuson is működik.** Nem: a visszatérési típus nem része a szignatúrának, csak a paraméterek (és a `const`/ref-minősítés a tagfüggvényen).

19.5.7 Összefoglaló

Az overload resolution névfeloldás → életképesség → konverziós rangsor lépésekből áll, és a döntés fordítás idejű; az azonos kategóriájú konverziók között nincs preferencia, ezért a kétértelműség valódi tervezési hiba. A default argumentum életképessé tesz olyan szignatúrákat, amelyek ütközhetnek; a `nullptr`, a `0` és a `NULL` három különböző dolog, és a pointer→`bool` konverzió a legtöbb meglepetés forrása. Az `explicit` és az erős típusok az implicit konverzió részvételét szűkítik, a brace-init a narrowingot tiltja, a nevesített kasztok pedig a kasztolás előfeltételét teszik olvashatóvá. Ebben az egységben nem állt rendelkezésre forrásjegyzet, ezért a fenti szabályok a C++20 szerinti megfogalmazást követik; a konkrét compiler-viselkedést a rögzített verziójú laborban kell ellenőrizni.

19.5.8 Feladatok

1. **Predikció.** Add meg, melyik `pick` overload fut az `overload_rank.cpp`-ben a `pick('a')`, `pick(0)`, `pick(0L)`, `pick(1.0f)`, `pick(true)`, `pick(0u)`, `pick("x")` hívásokra, vagy indokold, miért fordítási hiba.
2. **Interfész-javítás.** Adott a `void set_flag(bool)` és a `void set_flag(const std::string&)` pár. Írd le, mit választ a `set_flag("experimental")` hívás, majd tervezz olyan interfészt, amelyben a sztring literál nem konvertálódhat csendben `bool`-lá.
3. **Diagnosztika.** Az alábbi részlet nem fordul le. Azonosítsd a hibákat, add meg a javított változatot, és a fordítási parancsot, amellyel ellenőrizhető, hogy a javítás valóban lefordul.

```
void open(int fd, int flags = 0);
void open(int fd);
open(3);
int n{2.5};
```

19.5.9 Megoldások

1. megoldás. `pick('a')`: `char`→`int` promóció, a `long`/`double` felé konverzió, tehát `pick(int)`. `pick(0)`: exact match, `pick(int)`. `pick(0L)`: exact match, `pick(long)`. `pick(1.0f)`: `float`→`double` promóció, `pick(double)`. `pick(true)`: `bool`→`int` promóció, `pick(int)`. `pick(0u)`: `unsigned`→`int`, →`long`, →`double` mind Conversion, és a `string_view` felé nincs sorozat, ezért **kétértelmű, fordítási hiba**; ezért van kommentben. `pick("x")`: csak a `string_view` életképes, user-defined sorozattal. Ellenőrzés: a fenti `g++` parancs hibátlanul lefut, a `pick(0u)` sor kommentből való kivétele viszont hibát ad — a hibaüzenetnek tartalmaznia kell az „ambiguous” megfogalmazást.

2. megoldás. A `const char*→bool` standard konverzió (boolean konverzió), a `const char*→std::string` user-defined, ezért a hívás a `bool` overloadot választja, és a szöveg elveszik. A `std::string_view` bevezetése nem javít, mert az is user-defined sorozat. Helyes megoldás: a logikai kapcsolót tegyük erős típussá, és a pointer felé mutató utat tegyük szándékosan hibássá:

```
enum class Flag { on, off };
void set_flag(Flag f);
void set_flag(const std::string& s);
void set_flag(const char*) = delete;  // a literál nem eshet át csendben
```

Ellenőrzési feltétel: `set_flag("x")` fordítási hibát ad (a törölt overload életképes, ezért a hívás hibás), `set_flag(Flag::on)` és `set_flag(std::string{"x"})` viszont lefordul. A `= delete` itt nem „eltávolítás”, hanem diagnosztikai szerződés: a hívó szándéka láthatóvá válik.

3. megoldás. Két hiba van. Egyrészt az `open(3)` kétértelmű: az `open(int,int = 0)` és az `open(int)` is életképes, mindkettő exact match-csel az első argumentumban, ezért nincs győztes. Másrészt az `int n{2.5};` narrowing miatt hibás. Javítás: hagyjuk el a rövidebb overloadot (a default argumentum már kiszolgálja a hívást), és tegyük a konstanst típusossá:

```
void open(int fd, int flags = 0);
// open(int) törölve: a default argumentummal együtt ütközne
open(3);          // OK
int n{2};         // egész konstans
double d{2.5};    // a nem egész érték ott marad, ahol van
```

Ellenőrzés: `g++ -std=c++20 -Wall -Wextra -Wpedantic -Wconversion prog.cpp -o prog` hibátlanul lefut. Ha a `2.5`-öt visszaírod `int n{...}` alakba, a fordítónak narrowing hibát kell adnia; ha mégsem, akkor a fordítópipeline nem a várt szabvány-módot használja, és a laborprofil felülvizsgálandó.

20 C++ objektumok, inicializálás és élettartam

20.1 Az inicializálás különböző alakjai

A C++ inicializálás az a terület, ahol a legtöbb tapasztalt, más nyelvről érkező mérnök is meglepődik: ugyanaz a négy-öt karakter látszólag mást jelent skalár, aggregate, `std::vector` vagy saját osztály esetén. DevOps-környezetben ez nem elméleti szépséghiba: konfigurációs struktúrák, buffer-méretek, socket-portok és flag-ek inicializálása közvetlenül hibás viselkedéshez, néha nem definiált viselkedéshez (undefined behavior, UB) vezet. Ez az alfejezet a C++20-as alapot feltételezi, és a C-ből ismert objektum-/tárolási modellre épül; a korábbi Inicializálás, indeterminate érték és reprezentáció (5.3) és az Overload, konverzió és explicit interfész (19.5) ismeretét rövid felidézésként használja.

A tárgyalás határait itt az inicializálási formák jelentik. A konstruktorok és destruktorok sorrendjét, a részleges konstrukciót, valamint a temporary lifetime részletes eseteit a következő alfejezetek (Osztály, konstruktor és invariáns (20.2), Destruktor és részleges konstrukció (20.3), Temporary, value category és dangling (20.4)) tárgyalják; itt csak a megértéshez szükséges mértékben utalunk rájuk. A példák C++20-at feltételeznek; a C++23-as eltéréseket külön jelölöm, és a konkrét GCC/Clang-viselkedést a tanuló laborjában kell ellenőrizni. Ehhez a generáláshoz nem kaptam ellenőrzött forrásjegyzetet, ezért a szabványparagrafusokra itt nem teszek kattintható hivatkozást, és a „fordítottam/teszteltem” állítást sem használom.

20.1.1 Az inicializálás formáinak térképe

A C++ inicializálási kategóriái nem teljesen függetlenek: a szintaxis sokszor egy általánosabb szabályt indít el, amelynek része a zero initialization, a value initialization vagy éppen az aggregate initialization. A következő táblázat a leggyakoribb eseteket foglalja össze; a részletek utána következnek.

Forma	Tipikus szintaxis	Skalár típus	Osztálytípus	Fontos megjegyzés
default initialization	`T x;`, `new T`	nincs inicializálás; indeterminált	default konstruktor	lokális skalárnál olvasása UB
value initialization	`T()`, `T{}`, `new T()`	nulla	ha a default ctor nem user-provided: zero-init + default-init; különben default-init	`T x{}` gyakori „nullázó” alak
zero initialization	statikus tárolású `T x;`	nulla vagy null pointer	tagok nullázása	static/thread_local esetén a program indulása előtt

Forma	Tipikus szintaxis	Skalár típus	Osztálytípus	Fontos megjegyzés
direct initialization	`T x(args);`, `T x{args};`	konverzió	konstruktor, explicit is	`static_cast<T>(args)` is ide tartozik
copy initialization	`T x = expr;`, `T x = {args};`	konverzió	csak nem explicit konstruktor	függvényargumentum, return is
list initialization	`T x{...};`, `T x = {...};`	narrowing tilos	initializer_list preferencia, aggregate	`{}` és `()` eltér
aggregate initialization	aggregate `T x{...};`	tagok sorrendben	DMI vagy zero/value-init	C++20: designated initializer

A default initialization pontos definíciója: az objektum deklarációjában nincs initializer, illetve `new T` esetén nincs zárójel. Osztálytípusnál a default konstruktor hívódik; ha nincs hívható default ctor, a kód ill-formed. Tömbnél minden elem default-initializálódik. Skalárnál — például `int x;` — a nyelv nem ír elő értéket: az objektum indeterminált értékű lesz, és ha ezt kiolvassuk, UB történik. Ugyanez igaz a `new int` kifejezésre is; a `new int()` viszont value-initializál, azaz nulláz.

A value initialization akkor lép fel, amikor `T()`, `T{}`, `new T()`, `new T{}` alakot használunk, illetve amikor egy tagot a mem-initializer listában üres zárójelpárral inicializálunk. Skalárnál az eredmény nulla. Osztálynál a szabály attól függ, hogy a default konstruktor user-provided-e: ha nem az, akkor először zero initialization, majd default initialization történik; ha user-provided, akkor „csak” a konstruktor fut. Ez a megkülönböztetés magyarázza, miért lesz `std::vector<int> v(3);` elemei nulla értékűek, miközben egy saját, user-provided default konstruktorral rendelkező osztály `T{}` esetén nem feltétlenül nullázza a tagokat.

A zero initialization nem önállóan kiírt szintaxis, hanem bizonyos helyzetekben automatikusan végrehajtott előzetes lépés: statikus és thread-local tárolási idő esetén a program indulása előtt, valamint a value initialization részeként. Skalárnál nullát, pointer esetén null pointert, osztály esetén a tagok nullázását jelenti. Fontos, hogy ez nem azonos a default initializationnal: `static int x;` nullázódik, a lokális `int x;` nem.

20.1.2 Direct, copy és list initialization

A direct initialization a `T x(args);`, `T x{args};`, `T(args)` és `static_cast<T>(args)` alakokat fedi. Lényege, hogy a konstruktorok közül az összes figyelembe vehető, beleértve az `explicit` konstruktorokat is. Copy initialization esetén viszont a `T x = expr;` és a `T x = {args};` alakot használjuk, valamint ilyen történik függvényargumentum átadásakor és visszatérési értéknél; itt az `explicit` konstruktorok nem játszanak szerepet. Ezért a `T x = T(args);` alak nem tud olyan explicit konstruktort hívni, mint a `T x(args);`.

A list initialization a kapcsos zárójeles alakok gyűjtőneve: direct-list-initialization (`T x{...};`) és copy-list-initialization (`T x = {...};`). A lista-inicializálás előnye, hogy tiltja a narrowing conversiont, elkerüli a „most vexing parse” problémát, és aggregate esetén aggregate initializationt indít. Hátránya, hogy az `initializer_list` konstruktorokat előnyben részesíti, ami a megszokott `()`-es hívásnál meglepőbb eredményt adhat.

Nézzük a klasszikus `std::vector`-példát:

```cpp
#include <vector>

int main() {
    std::vector<int> a(3, 7); // három darab 7-es
    std::vector<int> b{3, 7}; // két elem: 3 és 7
    std::vector<int> c(3);    // három value-initialized int: 0, 0, 0
    std::vector<int> d{3};    // egy elem: 3
}
```

A különbség oka, hogy a `b{3, 7}` listában a `std::initializer_list<int>` konstruktor viabilis, ezért a lista-inicializálás első fázisa azt választja. A `d{3}` ugyanígy egy elemű listát ad, nem három nullát. Aki a `std::vector<int> v{10};` alakot „tíz nulla” jelentéssel írja, egy elemű, 10-es értékű vektort kap. Ugyanez a minta jelenik meg `std::string` esetén is:

```cpp
#include <string>

int main() {
    std::string a(5, 'x'); // "xxxxx"
    std::string b{5, 'x'}; // két karakter: char(5) és 'x'
}
```

A `b` esetében az `initializer_list<char>` konstruktor lesz a nyertes; az `5` int konstans kifejezésként elfér `char`-ban, ezért a narrowing nem tiltja ki. Ha viszont `std::string c{300, 'x'};` szerepelne, a 300 nem férne el `char`-ban, így a lista-inicializálás ill-formed lenne. Ez jól mutatja, hogy az `initializer_list` preferencia nem „mindig jó”, hanem a lista típusától és a konverzióktól függ.

20.1.3 Aggregate initialization és verziófüggése

Aggregate a C++20 szerint az a tömb vagy osztály, amelynek nincs felhasználó által deklarált vagy örökölt konstruktora, nincs private/protected nem statikus adattagja, nincs virtuális függvénye, és nincs virtuális, private vagy protected bázisosztálya. Aggregate inicializálásnál a kapcsos zárójelben lévő értékek sorrendben az aggregate **elemeit** inicializálják, és a kimaradó elemek default member initializerrel (DMI), ennek hiányában üres listából kapnak értéket. Hogy mi számít elemnek, az aggregate fajtájától függ:

- **tömb:** az elemek indexsorrendben;
- **bázis nélküli osztály:** a nem statikus adattagok deklarációs sorrendben;
- **bázisos osztály (C++17 óta):** előbb a közvetlen bázisok deklarációs sorrendben, *utána* a nem statikus adattagok.

A bázis tehát saját elem: a `D{{1}, 2}` listában a `{1}` a `B` bázisé, a `2` a `D::y`-é. Brace elision miatt a `D{1, 2}` is ugyanezt jelenti — az első `1` itt sem `D::y` inicializálója, hanem a `B::x`-é.

```cpp
// aggregate_base.cpp - C++20: bázis a tagok előtt
#include <cassert>

struct B { int x; };
struct D : B { int y; };

int main() {
    D d{{1}, 2};        // első elem: a B bázis ({1}), második: D::y
    assert(d.x == 1 && d.y == 2);
```

```
    D e{1, 2};          // brace elision: az 1 a B::x-be kerül, nem D::y-ba
    assert(e.x == 1 && e.y == 2);
    // D f{.y = 2};     // hiba: a bázis nem nevezhető meg, és designatorral
                        // csak közvetlen nem statikus tag inicializálható
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic aggregate_base.cpp` figyelmeztetés nélkül fordul, a futás egyik `assert`-et sem sérti. A designated initializer csak közvetlen nem statikus adattagot nevezhet meg: a `D{.x = 1, .y = 2}` GCC 13.3 és Clang 22 alatt is hibát ad (`'D' has no non-static data member named 'x'`), a `D{.y = 2}` viszont fordul, a kihagyott bázis üres listából inicializálódik (GCC `-Wextra` alatt `missing initializer` figyelmeztetéssel).

```
struct Point {
    int x;
    int y;
};

Point p1{1, 2};       // x = 1, y = 2
Point p2{1};          // x = 1, y = 0
Point p3{};           // x = 0, y = 0
Point p4{.x = 3, .y = 4}; // C++20 designated initializer
```

A verziófüggés itt valós és könnyen megbotlik rajta az ember. C++11-ben a DMI-t tartalmazó osztály még nem számított aggregate-nek; C++14 óta igen. C++17-ben a nyilvános, nem virtuális bázisosztály már megengedett aggregate esetén; C++20 viszont minden felhasználó által deklarált konstruktort kizár, nem csak a user-provided vagy explicit konstruktorokat. C++20-ban jelentek meg a designated initializerek és a zárójeles aggregate initialization is (`Point p(1, 2);`), amelyek C++17-ben még nem elérhetőek. A designated initializer C++20-as szabályai szigorúbbak a C99-esnél: a megnevezett tagok sorrendje a deklarációs sorrend kell legyen, és a C-s beágyazott, illetve tömb-designatorok nem részei a C++20 nyelvnek. C++23 esetén a fordító és a standard library támogatását külön kell ellenőrizni; a fenti példák C++20-as baseline-t feltételeznek.

A DMI és az aggregate initialization együtt különösen hasznos konfigurációs struktúráknál:

```
struct Config {
    int timeout_ms = 1000;
    bool verbose = false;
};

Config c1;          // default initialization: DMI-k érvényesülnek
Config c2{};        // value/list initialization: DMI-k érvényesülnek
Config c3{500};     // aggregate init: timeout_ms = 500, verbose a DMI-ből false
```

A `c3` esetében a `verbose` kimarad, ezért a DMI-ját kapja, nem indeterminált értéket. Ez a minta DevOps-eszközökben gyakori: a „default érték” és a „felülírás” egy helyen, olvashatóan jelenik meg. Fontos azonban, hogy ha az osztályhoz később felhasználó által deklarált konstruktort adunk, C++20-ban az aggregate státusz megszűnik, és a `Config c3{500};` jelentése megváltozhat; ilyenkor a konstruktorok és a `{}`/`()` különbség újraellenőrzendő.

20.1.4 Most vexing parse és egyértelmű deklaráció

A „most vexing parse” (szó szerint „legbosszantóbb elemzés”) az a C++ szabály, amely szerint ami függvénydeklarációként is értelmezhető, azt függvénydeklarációként kell értelmezni. A legegyszerűbb eset:

```
struct S {
    S(int);
};

S s1(1);  // objektum, direct initialization
S s2();   // függvénydeklaráció: s2 visszatérési típusa S, nincs paramétere
S s3{};   // objektum, value/list initialization
```

A `S s2();` tehát nem default konstruktorral létrehozott objektum, hanem függvénydeklaráció. A `{}` használata ezt a kétértelműséget megszünteti. Egy összetettebb, gyakran idézett példa:

```
#include <iterator>
#include <vector>

// Ez függvénydeklarációt deklarál, nem vektort!
std::vector<int> v(std::istream_iterator<int>(std::cin),
                   std::istream_iterator<int>());

// Egyértelmű objektumkonstrukció:
std::vector<int> w{std::istream_iterator<int>(std::cin),
                   std::istream_iterator<int>()};
```

A `w` esetében a lista-inicializálás először az `initializer_list<int>` konstruktort vizsgálja; mivel az iterátorok nem konvertálhatók `int`-té, ez nem viabilis, így a második fázisban a range konstruktor hívódik. Ez a példa jól mutatja, hogy a `{}` nem „varázsszer”, hanem egy másik overload-halmazt indít el; a lényeg, hogy a deklaráció egyértelmű legyen.

20.1.5 Narrowing tiltása és kivételei

A list initialization egyik legfontosabb biztonsági szabálya, hogy a narrowing conversion ill-formed. A szabvány fő esetei: lebegőpontosból egészbe; `long double`-ből `double`/`float`-ba vagy `double`-ből `float`-ba; egészből vagy unscoped enumerációból lebegőpontosba; egészből vagy unscoped enumerációból olyan egész típusba, amely nem tudja az összes forrásértéket ábrázolni; valamint pointer vagy pointer-to-member típusból `bool`-ba. A szabály alól vannak constant expression kivételek: ha a forrás constant expression, az érték belefér a cél típusba, és visszakonvertálva az eredeti értéket adja, akkor az adott átalakítás nem számít narrowingnak. A `double` → `float` esetén a tartományba esés is elég, a pontos ábrázolás nem követelmény.

```
// C++20: ezek list initializationban ill-formedak
int i{2.0};                 // lebegőpontos -> egész
char c{1000};               // 1000 nem fér el char-ban
std::vector<int> v{1, 2.0}; // 2.0 -> int

// Ezek viszont megengedettek
constexpr int n = 42;
char c2{n};         // 42 elfér char-ban, és visszakonvertálva 42
```

```cpp
float f{1.1};       // double -> float, constant expression, tartományban
unsigned u{42};     // int -> unsigned, 42 visszakonvertálva 42
```

A constant expression kivétel levezetése röviden: a narrowing tilalmának célja, hogy a lista-inicializálás ne veszítsen el információt csendben. Ha a forrás értéke fordítási időben ismert, és a cél típusban ábrázolható, valamint a visszakonvertálás ugyanazt az értéket adja, akkor a konverzió nem veszít információt; ezért a szabvány megengedi. A `float f{1.1};` esetében a `double` érték nem ábrázolható pontosan `float`-ban, de a szabály a tartományba esést nézi, nem a bitpontos egyezést. A `char c{1000};` esetében viszont a 1000 nem fér el, ezért a kivétel nem alkalmazható.

A gyakorlatban a fordítók a narrowingöt általában hibaként diagnosztizálják `-std=c++20` mellett, de a diagnosztika formája és a figyelmeztetés/hiba státusz fordító- és flagfüggő lehet. A szabvány szerint a kód ill-formed, azaz diagnosztikát kell kapnia; a „lefordult, tehát legális” következtetés nem biztonságos. A C++23 ezen a területen nem változtatta meg alapvetően a list-initialization narrowing szabályát, de a konkrét fordítót és library-t mindig ellenőrizni kell.

20.1.6 Inicializálatlan skalár és default member initializer

A C++ nem minden változót nulláz automatikusan. Lokális, automatikus tárolási idejű skalárnál az `int x;` indeterminált értéket hagy, és az érték kiolvasása UB. Ugyanez igaz a `new int` kifejezésre, míg a `new int()` nulláz. Statikus tárolási időnél viszont a nyelv garantálja a zero initializationt:

```cpp
int global;         // zero-initialized: 0

void f() {
    int local;      // indeterminált; olvasása UB
    int zero{};     // value-initialized: 0
    int* p = new int;   // indeterminált int
    int* q = new int(); // nullázott int
    delete p;
    delete q;
}
```

A default member initializer (DMI) segít elkerülni a „elfelejtett tag inicializálása” hibát. A DMI akkor érvényesül, ha a konstruktor nem inicializálja azt a tagot, illetve aggregate initializationnál, ha a tag kimarad. C++11-ben a DMI még kizárta az aggregate státuszt, C++14 óta viszont nem; ezért a `Config` példa C++14/17/20 alatt aggregate marad, feltéve, hogy a C++20-as aggregate-feltételek többi része teljesül.

```cpp
struct Endpoint {
    std::string host;
    int port = 8080;
    bool tls = false;
};
```

Itt a `port` és a `tls` DMI-t kap; a `host` viszont `std::string`, amelynek van default konstruktora, ezért `Endpoint e;` esetén üres string lesz. Ha a `host` skalár lenne, például `int fd;`, akkor a DMI hiányában default initializationnál indeterminált maradna.

20.1.7 Kidolgozott példa: init_forms.cpp

A következő példa egyetlen fordítási egységben mutatja a fő formákat. A kód C++20-at feltételez; a designated initializer miatt C++17-tel nem fordul. A buildparancsot a példa után adom meg; a kimenet „elvárt/illusztratív”, nem mért eredmény.

```
#include <iostream>
#include <string>
#include <vector>

struct Point {
    int x;
    int y;
};

struct Config {
    int timeout_ms = 1000;
    bool verbose = false;
};

int main() {
    int a{};        // value-init: 0
    int b = 42;     // copy-init
    int c(43);      // direct-init
    int d{44};      // direct-list-init

    Point p1{1, 2};           // aggregate init
    Point p2{.x = 3, .y = 4}; // C++20 designated
    Point p3{};               // minden tag nulla

    Config cfg1;              // default-init, DMI-k érvényesülnek
    Config cfg2{};            // value/list-init, DMI-k érvényesülnek
    Config cfg3{500};         // aggregate init: timeout_ms = 500

    std::vector<int> v1(3, 7); // három 7-es
    std::vector<int> v2{3, 7}; // két elem: 3, 7

    std::cout << a << ' ' << b << ' ' << c << ' ' << d << '\n';
    std::cout << p1.x << ',' << p1.y << ' '
              << p2.x << ',' << p2.y << ' '
              << p3.x << ',' << p3.y << '\n';
    std::cout << cfg1.timeout_ms << ' '
              << cfg2.timeout_ms << ' '
              << cfg3.timeout_ms << ' ' << cfg3.verbose << '\n';
    std::cout << v1.size() << ' ' << v2.size() << '\n';
}
```

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -Wconversion init_forms.cpp -o
 ↪  init_forms
# vagy
clang++ -std=c++20 -Wall -Wextra -Wpedantic -Wconversion init_forms.cpp -o
 ↪  init_forms
```

A várt kimenet alakja: `0 42 43 44`, majd `1,2 3,4 0,0`, majd `1000 1000 500 0`, végül `3 2`. Ez a példa nem bizonyítja a fordító viselkedését minden targeten; a tanuló laborjában a tényleges kimenetet és a warningokat rögzíteni kell.

20.1.8 Tipikus hibák és javításuk

Az első és leggyakoribb hiba a `std::vector<int> v{10};` „tíz nulla” szándékkal való használata. Javítás: `std::vector<int> v(10);`, ha méretre van szükség, vagy `std::vector<int> v(10, 0);`, ha explicit nullázás a cél. A második: `S s();` után azt várni, hogy objektum jön létre; ez függvénydeklaráció. Javítás: `S s;` vagy `S s{};`. A harmadik: `int x;` lokális változó olvasása; javítás `int x{};`, illetve ha a „nincs érték” állapot kell, `std::optional<int>` (a későbbi optional, variant és visit (31.1) témája). A negyedik: list initialization használata olyan helyen, ahol az `initializer_list` konstruktor átveszi az irányítást; ilyenkor tudatosan `()`-t vagy explicit típust kell választani. Az ötödik: C++20-as designated initializert C++17-es buildben használni; a buildprofil és a `-std=` kapcsoló ellenőrzése kötelező. A hatodik: azt feltételezni, hogy a DMI minden konstruktorban érvényesül; valójában ha a konstruktor inicializálja a tagot, a DMI nem lép be.

20.1.9 Összefoglaló

A C++ inicializálását négy kérdés mentén érdemes olvasni: van-e initializer, az `()` vagy `{}` alakot használjuk-e, osztály- vagy skalártípusról van-e szó, és aggregate-e a típus. A default initialization skalárnál nem ad értéket; a value initialization nulláz, illetve a nem user-provided default konstruktor előtt zero initializationt végez. A direct initialization az explicit konstruktorokat is figyelembe veszi, a copy initialization nem. A list initialization narrowingöt tilt, elkerüli a most vexing parse-ot, de az `initializer_list` konstruktorokat előnyben részesíti. Az aggregate initialization verziófüggő: C++14 óta a DMI, C++17 óta a nyilvános bázis, C++20 óta a designated initializer és a zárójeles aggregate init a fő változások. A biztonságos minta: konfigurációs aggregate-eknél DMI, skalároknál `{}`, és minden `{}`/`()` választásnál tudatos overload- és narrowing-ellenőrzés.

20.1.10 Feladatok

20.1.10.1 1. feladat: inicializálási formák osztályozása

Az alábbi kódban minden deklarációhoz írd le, melyik inicializálási forma érvényesül, és milyen értéket kapnak a tagok/az objektum. Feltételezd, hogy a kód függvényen belül van, C++20-ban fordít, és a `Point` aggregate.

```
struct Point { int x; int y; };

Point a;
Point b{};
Point c{1};
Point d(1, 2);
std::vector<int> v1(3);
std::vector<int> v2{3};
```

20.1.10.2 2. feladat: konfigurációs struktúra és DMI

Készíts egy `Endpoint` aggregate-et `std::string host;`, `int port = 8080;`, `bool tls = false;` tagokkal. Mutasd meg három inicializálási módon: default initialization, üres list initialization és egy olyan aggregate initialization, ahol csak a `host` és a `port` kap értéket. Magyarázd el, mi történik a `tls`-sel. Ezután adj hozzá egy `Endpoint(std::string, int)` konstruktort, és indokold, miért változik meg C++20-ban az aggregate státusz, valamint hogyan kell módosítani a példát.

20.1.10.3 3. feladat: diagnosztika és javítás

Az alábbi részlet több okból sem fordul C++20-ban. Azonosítsd a hibákat, írd le a diagnosztika várható okát, és javítsd a kódot a szándék szerint: `S` objektumot akarunk létrehozni, `x` egész legyen 3, a vektor pedig két elemet tartalmazzon: 1-et és 2-t.

```
struct S { S(int); };
S s();
int x{3.14};
std::vector<int> v{1, 2.5};
```

20.1.11 Megoldások

20.1.11.1 1. feladat megoldása

A `Point a;` default initialization: mivel `Point` aggregate és nincs felhasználó által deklarált konstruktora, az implicit default konstruktor fut, de a skalár tagok default initializationja nem ad értéket. Lokális objektumnál `a.x` és `a.y` indeterminált; olvasásuk UB. A `Point b{};` üres list initialization: aggregate initialization üres listával, a tagok value initializationt kapnak, így `x = 0`, `y = 0`. A `Point c{1};` aggregate initialization: `x = 1`, a kimaradó `y` value initializationnal `0` lesz. A `Point d(1, 2);` C++20-ban zárójeles aggregate initialization, `x = 1`, `y = 2`; C++17-ben ez nem fordulna, mert ott még nincs parenthesized aggregate initialization. A `std::vector<int> v1(3);` méret-konstruktor: három value-initialized `int`, azaz `0, 0, 0`. A `std::vector<int> v2{3};` list initialization: az `initializer_list<int>` konstruktor viabilis, ezért egy elemű vektor lesz, értéke `3`. Ellenőrzési feltétel: a `v1.size() == 3`, `v2.size() == 1`, és a `v2[0] == 3`; a `Point a` értékeit nem szabad kiolvasni.

20.1.11.2 2. feladat megoldása

A kiinduló struktúra:

```
#include <string>

struct Endpoint {
    std::string host;
    int port = 8080;
    bool tls = false;
};

Endpoint e1;                 // default-init: host üres, port=8080, tls=false
Endpoint e2{};               // value/list-init: ugyanaz
Endpoint e3{"example.org", 443}; // host, port; tls a DMI-ból false
```

Az `e3` esetében a `tls` kimarad, ezért a DMI érvényesül, nem indeterminált érték. Ha hozzáadjuk az `Endpoint(std::string, int)` konstruktort, akkor C++20-ban a `Endpoint` már nem aggregate, mert van felhasználó által deklarált konstruktora. Ekkor a `Endpoint e3{"example.org", 443};` a konstruktort hívja, és a `tls` csak akkor lesz `false`, ha a konstruktor nem inicializálja, de a DMI érvényesül; ha a konstruktor inicializálja, a DMI nem lép be. A példát úgy kell módosítani, hogy vagy megtartjuk az aggregate-et (nincs saját konstruktor), vagy a konstruktorban explicitté tesszük a kívánt defaultot. Ellenőrzési feltétel: `e1.port == 8080`, `e3.tls == false`, és a konstruktoros változatban a `tls` értéke a konstruktor viselkedésétől függ, nem automatikusan az aggregate initializationtól.

20.1.11.3 3. feladat megoldása

A `S s();` most vexing parse: függvénydeklarációt deklarál, nem `S` objektumot. Javítás: `S s{0};` vagy `S s(0);`, illetve ha default konstruktor is van, `S s{};`. Az `int x{3.14};` list initialization, a `double` → `int` átalakítás narrowing, ezért ill-formed. Javítás a szándék szerint: `int x = 3;` vagy `int x{3};`; ha a 3.14 értéket akarjuk megtartani, `double x{3.14};` kell. A `std::vector<int> v{1, 2.5};` szintén narrowing a `2.5` → `int` miatt; javítás: `std::vector<int> v{1, 2};`, vagy ha a 2.5 is kell, `std::vector<double> v{1.0, 2.5};`. Ellenőrzési feltétel: a javított kód C++20-ban `-std=c++20 -Wall -Wextra -Wpedantic` mellett fordul, a `v.size() == 2`, és a `v` elemei a szándékolt típusban tárolódnak; a `S s()` helyett tényleg objektum jön létre.

20.2 Osztály, konstruktor és invariáns

A C `struct` a korábbi részekben elsősorban memórialayout és értékmásolás volt (Struktúrák, tagok és értékmásolás (9.1)); a C++ ugyanezt a szerkezetet felhasználói típussá bővíti, amelynek saját érvényességi szerződése, konstrukciós folyamata és élettartama van. Ez az alfejezet azt a kérdést járja körül, mitől „érvényes” egy C++ objektum, hogyan kell a konstruktornak létrehoznia és megőriznie az invariánst, és milyen csapdák vannak a taginicializálás sorrendjében. Az inicializálási formák taxonómiáját (Az inicializálás különböző alakjai (20.1)) itt nem ismételjük meg; a hangsúly a konstruktor és az invariáns kapcsolatán van.

20.2.1 class és struct: alapértelmezett hozzáférés és szerződés

A `class` és a `struct` a C++ban ugyanaz a nyelvi konstrukció. A különbség kizárólag az alapértelmezett hozzáférésben és az alapértelmezett öröklési hozzáférésben van: `class` esetén a tagok és az öröklés alapértelmezésben `private`, `struct` esetén `public`. Mindkettő lehet konstruktoros, privát tagú, virtuális függvényes vagy sablonparaméterként használt. A gyakorlati konvenció az, hogy `struct`-ot passzív adatköteghez, `class`-t pedig olyan típushoz használunk, amelynek van fenntartandó invariánsa.

Definíció. Az osztály *invariánsa* olyan állítás a nem statikus adattagok értékéről és kapcsolatáról, amelynek a konstruktor sikeres befejezése után minden nyilvánosan megfigyelhető állapotban igaznak kell lennie, és amelyet minden nyilvános tagfüggvény köteles megőrizni. Például egy `Counter` esetében az invariáns lehet $0 \le \text{value} \le \text{limit}$; egy porttartománynál $0 \le \text{lo} \le \text{hi} \le 65535$.

Érvényes objektum alatt itt azt értjük, hogy a tároló létrejött, az objektum élettartama elkezdődött, a konstruktor sikeresen visszatért, és az invariáns teljesül. A C++ memóriamodelljében a teljes osztályobjektum élettartama az inicializálás befejezésével, azaz a konstruktor sikeres visszatérésével veszi kezdetét. Az alobjektumok élettartama korábban is elkezdődhet, amikor a

saját inicializálásuk befejeződik, de a teljes objektumra vonatkozó szerződés csak a konstruktor után érvényes.

Az érvényes objektum létrehozásának szerződése röviden:

1. a tároló megszerzése történhet automatikus, statikus vagy dinamikus úton;
2. a konstruktornak inicializálnia kell minden tagot, majd vagy létre kell hoznia az invariánst, vagy kivételt kell dobnia;
3. sikeres visszatérés után a nyilvános műveletek feltételezhetik az invariánst;
4. ha a konstruktor kivétellel kilép, a teljes objektum soha nem jön létre, ezért a teljes objektum destruktora nem fut le.

Az invariánst a kódban is dokumentálni kell, például a class fejlécében egy `// Invariáns: ...` kommenttel, és azt is meg kell mondani, mely műveletek tartják fenn. Ez nem esztétikai kérdés: a későbbi karbantartó abból tudja meg, mit nem szabad „csak óvatosan” megkerülni.

20.2.2 Tagfüggvény, this és cv-qualifier

A nem statikus tagfüggvény minden hívásban kap egy implicit `this` pointert az aktuális objektumra. Nem `const` tagfüggvényben `this` típusa `X*`, `const` tagfüggvényben `const X*`. Ezért const objektumon csak `const` tagfüggvény hívható, és egy `const` tagfüggvény nem módosíthatja a nem `mutable` adattagokat.

```
class Gauge {
public:
    int value() const noexcept { return value_; } // this: const Gauge*
    void set(int v) noexcept { value_ = v; }      // this: Gauge*
private:
    int value_{0};
};
```

A `const` itt nem pusztán formai jelölés: az interfész része. Ha egy tagfüggvény logikailag csak olvas, jelöljük `const`-nak, mert így const objektumon és const referencián keresztül is használható. A `mutable` az a szűk kivétel, amikor egy gyorsítótár vagy diagnosztikai számláló módosítása nem számít logikai állapotváltozásnak; ezt indokolni kell, nem alapértelmezésként használni.

20.2.3 Member initializer list és a deklarációs sorrend

A tagok inicializálása nem a member initializer list sorrendjében, hanem a nem statikus adattagok **deklarációs sorrendjében** történik. Az alaposztályok ezt megelőzik, de az öröklési részleteket itt nem tárgyaljuk. Ha a listában más sorrend szerepel, a fordító figyelmeztethet (`-Wreorder`), a viselkedés viszont a deklarációs sorrend marad.

```
class Window {
public:
    Window(int begin, int length)
        : begin_{begin}, length_{length}, end_{begin_ + length_} {}
private:
    int begin_;
    int length_;
    int end_;
};
```

Itt a deklarációs sorrend `begin_`, `length_`, `end_`, ezért az `end_{begin_ + length_}` kifejezésben mindkét korábbi tag már inicializált. A helyes sorrend nem stílus kérdése: ha egy inicializáló kifejezés egy később inicializált tagot olvas, a program nem definiált viselkedésű.

```
struct Overlap {          // tanítási példa: hibás
    int end;              // előbb deklarált
    int begin;
    Overlap(int b, int len)
        : begin{b}, end{begin + len} {} // UB: end initjekor begin még nem
        ↪  inicializált
};
```

A javítás vagy a deklarációs sorrend megcserélése, vagy az `end` kiszámítása a konstruktortestben. Ugyanígy hiba, ha egy skalár tagot kihagyunk a listából és nincs osztályon belüli inicializálója: az ilyen tag alapértelmezett inicializálása skalár esetén határozatlan értéket eredményez. Osztály típusú tag esetén a kihagyott tag az alapértelmezett konstruktorával jön létre.

20.2.4 Default, delegating és explicit konstruktorok

A **default konstruktor** az a konstruktor, amely nulla argumentummal hívható. Lehet implicit, `= default`-tal kért, vagy `= delete`-tel tiltott. A `= default` azt kéri a fordítótól, hogy generálja a szokásos tagonkénti viselkedést; ez akkor hasznos, ha például az osztályon belüli inicializálók már minden tagot ellátnak. A `= delete` világossá teszi, hogy bizonyos konstrukció nem megengedett.

A **delegáló konstruktor** a member initializer listjében egy másik konstruktort hív meg:

```
class TokenBucket {
public:
    explicit TokenBucket(std::uint32_t capacity)
        : TokenBucket{capacity, capacity} {} // delegálás

    explicit TokenBucket(std::uint32_t capacity, std::uint32_t initial)
        : capacity_{capacity}, tokens_{initial} {
        if (capacity_ == 0)
            throw std::invalid_argument{"capacity must be > 0"};
        if (tokens_ > capacity_)
            throw std::invalid_argument{"initial > capacity"};
    }
private:
    const std::uint32_t capacity_;
    std::uint32_t tokens_;
};
```

Delegáláskor a célkonstruktor fut le először teljesen, utána a delegáló konstruktor teste. A delegáló lista csak magát a delegálást tartalmazhatja; más tagot ott nem lehet inicializálni. Ha a célkonstruktor kivételt dob, a delegáló konstrukció is meghiúsul, és a teljes objektum nem jön létre.

Az **explicit konstruktor** megakadályozza a nem kívánt implicit konverziót. Egyargumentumú konstruktornál például az `explicit TokenBucket(std::uint32_t)` tiltja a `TokenBucket b = 16;` alakot, miközben a `TokenBucket b{16};` és a `static_cast<TokenBucket>(16)` továbbra is működik. Többargumentumú konstruktornál az `explicit` a másoló-listainicializálást (`TokenBucket b = {16, 8};`) zárja ki. Fő szabály: ha a konstruktor nem szándékolt típuskonverziót valósítana meg, tegyük `explicit`-té.

20.2.5 In-class member initializer, const és reference tagok

Az **osztályon belüli taginicializáló** (in-class member initializer, gyakran NSDMI) a tag deklarációjánál megadott alapérték. Akkor használódik, ha az adott konstruktor member initializer listje nem említi a tagot; ha említi, a lista nyer.

```
class Sampler {
public:
    explicit Sampler(Logger& log, unsigned period_ms = 1000)
        : log_{log}, period_ms_{period_ms} {}
private:
    Logger& log_;                  // nem tulajdonol; a Loggernek túl kell élnie
    const unsigned period_ms_;
    unsigned failures_{0};         // NSDMI, a listában nem szerepel
};
```

A `const` tagot inicializálni kell; a konstruktortestben már nem lehet értéket adni neki. A referencia tagot szintén a konstrukciókor kell kötni, és később nem lehet átkötni. Mindkettő következménye, hogy az implicit másoló értékadó operátor törlődik, mert a `const` tag nem írható felül, a referencia tag pedig nem köthető át. Ha az osztálynak értékadhatónak kell lennie, vagy nem `const`/referencia tagot használunk, vagy saját értékadót írunk.

A referencia tag élettartam-szerződése külön figyelmet érdemel. A `Sampler` nem tulajdonosa a `Logger`-nek: a `Logger` objektumnak túl kell élnie a `Sampler`-t. Ha a referenciát ideigleneshez kötjük, az objektum élettartama után dangling referenciánk marad. A member initializer listben egy ideigleneshez kötött referencia tag esetében a temporary élettartama nem hosszabbodik meg a teljes objektum élettartamára; ez tipikus, nehezen észrevehető hiba. Ilyen esetben vagy érték szerint tároljunk, vagy dokumentált tulajdonosi viszonyt és hosszabb élettartamú forrást használjunk.

20.2.6 A konstruktor hibája és a létre nem jött objektum

Ha a konstruktor teste kivételt dob, a teljes objektum destruktora nem fut le, mert a teljes objektum soha nem jött létre. A már teljesen felépített tagok és alaposztályok destruktorai viszont lefutnak, fordított sorrendben. Ha egy tag konstruktora dob, az addig felépített tagok destruálódnak, majd a kivétel továbbterjed. Ezért a konstruktorban szerzett nyers erőforrást azonnal RAII-tagba kell tenni, különben a kivétel utáni részleges lebontás nem fogja felszabadítani.

```
class TwoFiles { // részlet: a File egy RAII wrapper
public:
    TwoFiles(const std::string& a, const std::string& b)
        : first_{a}, second_{b} {
        if (!first_.ok() || !second_.ok())
            throw std::runtime_error{"open failed"};
    }
private:
    File first_;
    File second_;
};
```

Ha a `second_` konstruktora dob, a `first_` destruálódik. Ha a test dob, a `second_`, majd a `first_` destruálódik; a `~TwoFiles()` nem fut. Dinamikus létrehozásnál a `new` kifejezés a megfelelő

felszabadítást is elvégzi, de a konstruktorban nyitott nyers file descriptor vagy `malloc`-olt blokk továbbra is elveszhet, ha nem tag destruktora kezeli.

A konstruktor további fontos korlátja, hogy a `this` még nem teljes objektumra mutat. Virtuális hívás a konstruktorban nem a legderivedebb osztály verzióját választja, és a `this` másoknak való átadása az invariáns felállítása előtt hibás publikálás. A részletes öröklési és virtuális dispatch szabályokat a későbbi rész tárgyalja; itt az a mérvadó, hogy a konstruktor ne tegye megfigyelhetővé a félig felépített objektumot.

20.2.7 Kidolgozott példa: TokenBucket teljes kóddal

Az alábbi program önállóan fordítható. A `TokenBucket` invariánsa: `capacity_ > 0` és `tokens_ <= capacity_`; a `rejected_` számláló nem része a szűkebb invariánsnak, de mindig nemnegatív.

```
#include <cstdint>
#include <iostream>
#include <stdexcept>

class TokenBucket {
public:
    explicit TokenBucket(std::uint32_t capacity)
        : TokenBucket{capacity, capacity} {}

    explicit TokenBucket(std::uint32_t capacity, std::uint32_t initial)
        : capacity_{capacity}, tokens_{initial} {
        // Invariáns: capacity_ > 0 és tokens_ <= capacity_.
        if (capacity_ == 0)
            throw std::invalid_argument{"TokenBucket: capacity must be > 0"};
        if (tokens_ > capacity_)
            throw std::invalid_argument{"TokenBucket: initial > capacity"};
    }

    std::uint32_t capacity() const noexcept { return capacity_; }
    std::uint32_t tokens() const noexcept { return tokens_; }
    std::uint64_t rejected() const noexcept { return rejected_; }

    bool take() noexcept {
        if (tokens_ == 0) {
            ++rejected_;
            return false;
        }
        --tokens_;
        return true;
    }

private:
    const std::uint32_t capacity_;
    std::uint32_t tokens_;
    std::uint64_t rejected_{0};
};

int main() {
```

```
    TokenBucket bucket{2};
    const bool a = bucket.take();
    const bool b = bucket.take();
    const bool c = bucket.take();
    std::cout << a << ' ' << b << ' ' << c
              << " rejected=" << bucket.rejected() << '\n';
}
```

Fordítás a laborprofilban rögzített GCC vagy Clang verzióval:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -Wreorder token_bucket.cpp -o
 ↪  token_bucket
```

A tagok deklarációs sorrendje `capacity_`, `tokens_`, `rejected_`; a member initializer list csak az első kettőt említi, a `rejected_` az osztályon belüli inicializálóból kap nullát. A konstruktor először inicializálja a tagokat, majd ellenőrzi az invariánst; ha az ellenőrzés bukik, kivételt dob, és a teljes `TokenBucket` nem jön létre. A forráskódból levezetve az első két `take()` igazat, a harmadik hamisat ad, a `rejected_` értéke 1 lesz; ez nem mért futtatási eredmény, hanem a kódból következő várt viselkedés, amelyet a laborban ellenőrizni kell.

20.2.8 Gyakori hibák és javításuk

Hiba	Következmény	Javítás
A member initializer list sorrendjét tekintjük tényleges sorrendnek	Egy később inicializált tag olvasása UB; `-Wreorder` figyelmeztetés	A deklarációs sorrendhez igazítani a listát, vagy a számítást a testbe tenni
Skalár tag kihagyása inicializálásból	Határozatlan érték	NSDMI vagy explicit inicializálás
`const` vagy referencia tag mellett implicit értékadásra számítunk	Az értékadó operátor törölt	Saját értékadó, vagy pointer/érték tag
Referencia tag ideigleneshez kötése	Dangling a konstrukció után	Érték szerinti tárolás vagy hosszabb élettartamú owner
Nyers erőforrás a konstruktortestben	Kivétel esetén leak	RAII tag a member initializer listben
Hiányzó `explicit`	Nem kívánt implicit konverzió	`explicit` a konstruktoron
`this` publikálása az invariáns előtt	Félig ép objektum megfigyelése	Csak sikeres konstrukció után publikálni

20.2.9 Összefoglaló

A C++ osztály érvényességi szerződése a konstruktorban jön létre: a tagok deklarációs sorrendben inicializálódnak, a member initializer list csak ezt a sorrendet írja le, nem módosítja. Az osztályon belüli inicializáló akkor lép életbe, ha a konstruktor nem említi a tagot. A `const` és referencia tagok kötelező inicializálást és dokumentált élettartam-viszonyt jelentenek, valamint törlik az implicit értékadást. A delegáló konstruktor centralizálja az invariáns ellenőrzését; az `explicit` megakadályozza a nem kívánt konverziót. Ha a konstruktor dob, a teljes objektum soha nem jön létre, ezért a teljes objektum destruktora nem fut le, csak a már felépített tagok destruktorai.

20.2.10 Feladatok

1. feladat (diagnózis). Az `Overlap` példában miért nem definiált a viselkedés, és hogyan javítanád? Add meg a javított deklarációs sorrendet vagy a javított konstruktort, és indokold, hogy a javítás miért szünteti meg a hibát.

2. feladat (kódolás). Készíts `Counter` osztályt, amelynek invariánsa $0 \le \text{value} \le \text{limit}$. A `limit` legyen `const int` tag, a `value` induljon nulláról osztályon belüli inicializálóval. Az `explicit Counter(int limit)` dobjon `std::invalid_argument`-ot negatív limitre. Legyen `bool increment()`, amely a limit elérésekor `false`-t ad, egyébként növel és `true`-t ad. Írd ki az invariánst kommentben, és add meg a fordítási parancsot.

3. feladat (diagnosztika és javítás). Az alábbi osztály nem fordul le. Magyarázd meg, miért, majd javítsd úgy, hogy a `Job` csak érvényes `id`-vel és élő `Logger`-rel jöhessen létre. Mi történne, ha a `Logger&` helyett `const Logger&` tagot használnánk, és a konstruktornak egy ideiglenes `Logger`-t adnánk?

```
class Job {
public:
    Job() = default;
    int id() const { return id_; }
private:
    const int id_;
    Logger& log_;
};
```

20.2.11 Megoldások

1. megoldás. A deklarációban `end` megelőzi `begin`-t, ezért az inicializálás sorrendje is `end`, majd `begin`, függetlenül attól, hogy a lista `begin`-t írja előbb. Az `end{begin + len}` inicializáló kifejezés még azelőtt olvassa a `begin` tagot, hogy az inicializálódott volna; egy automatikus tárolójú `int` határozatlan értékének olvasása nem definiált viselkedés. Javítás: a `begin` legyen előbb deklarálva, vagy az `end` értékét a konstruktortestben számítsuk ki:

```
struct Overlap {
    int begin;
    int end;
    Overlap(int b, int len) : begin{b}, end{b + len} {}
};
```

Ellenőrzési feltétel: a javított változat fordítható `-Wall -Wextra -Wpedantic -Wreorder` kapcsolókkal, és nincs olyan inicializáló kifejezés, amely korábban deklarált tagnál később deklarálttól függ.

2. megoldás.

```
#include <stdexcept>

class Counter {
public:
    explicit Counter(int limit) : limit_{limit} {
        // Invariáns: 0 <= value_ <= limit_.
        if (limit_ < 0)
            throw std::invalid_argument{"Counter: negative limit"};
```

```
    }

    int value() const noexcept { return value_; }
    int limit() const noexcept { return limit_; }

    bool increment() {
        if (value_ == limit_)
            return false;
        ++value_;
        return true;
    }

private:
    const int limit_;
    int value_{0};
};
```

A `limit_` a member initializer listből, a `value_` az osztályon belüli inicializálóból kap értéket. A konstruktor teste a tagok inicializálása után ellenőriz, és hiba esetén dob, így érvénytelen `Counter` nem jön létre. Ellenőrzési feltételek: `Counter c{-1};` dobjon `std::invalid_argument`-ot; `Counter c{0};` esetén `c.increment()` adjon `false`-t; `Counter c{1};` esetén az első hívás `true`, a második `false` legyen, és `c.value()` maradjon 1. Fordítás:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -c counter.cpp -o counter.o
```

Megjegyzés: a `const` tag miatt a `Counter` implicit értékadó operátora törlődik; ha értékadás kell, saját operátort kell írni, vagy nem `const` tagot kell használni.

3. megoldás. A `Job() = default` olyan alapértelmezett konstruktort kér, amelynek inicializálnia kellene a `const int id_` és `Logger& log_` tagokat, de erre nincs inicializáló. Ezért a defaulted konstruktor deleted lesz, és a `Job` nem hozható létre. A javítás:

```
class Job {
public:
    Job(int id, Logger& log) : id_{id}, log_{log} {}
    int id() const noexcept { return id_; }
private:
    const int id_;
    Logger& log_;
};
```

Így a `Job` csak akkor jön létre, ha a hívó átad egy `id`-t és egy létező `Logger`-t; a `Logger` tulajdonosa a hívó, és a `Logger`nek túl kell élnie a `Job`-ot. Ha `const Logger&` tagot használnánk, a `Job j{1, Logger{}};` lefordulhatna, de a tag egy olyan ideigleneshez kötődne, amely a teljes kifejezés végén megszűnik, így a `Job` élettartama alatt dangling referenciát tartana. Ellenőrzési feltétel: a javított osztály fordítható, a `Job j{1, logger};` érvényes, a `Job j{};` pedig nem fordul le.

20.3 Destruktor és részleges konstrukció

A destruktor az objektum élettartamának végét jelöli, nem a tároló felszabadítását. Ez a különbség a C++ erőforrás-kezelésének egyik sarokköve: a destruktor a logikai élettartam lezárása, a tároló felszabadítása (storage release) külön művelet. A fejezet a C++20 nyelvi magjára támaszkodik, és

az Osztály, konstruktor és invariáns (20.2) részben bevezetett konstruktor/invariáns fogalomra, valamint a Leállás, cleanup és nem lokális vezérlés (3.5) rész C cleanup-ágára épít. A cél, hogy a konstrukció és a destrukció sorrendjét, a részleges felépülés eseteit és a tárolókezelés különválasztását biztosan le tudd vezetni.

20.3.1 Az automatikus objektum élettartamának vége és a scope exit

Automatikus tárolási idővel rendelkező objektum esetén a destruktor a hatókör (scope) elhagyásakor fut le. A hatókör kilépése (scope exit) többféle lehet: a blokk vége, `return`, `throw` miatti stack unwinding, `goto`-val történő kiugrás, illetve `break`/`continue` a ciklusblokkból. A lényeg, hogy a blokkban deklarált automatikus objektumok destruktorai a deklaráció sorrendjének fordítottjában futnak le. Ha egy blokkban `A a; B b;` szerepel, akkor a kilépéskor előbb `~B()`, majd `~A()` fut.

Ez a szabály akkor is érvényes, ha a kilépés nem a blokk utolsó utasítása. Egy `return` előtt a lokális objektumok destruálódnak, majd a visszatérési érték átadása következik. Egy `throw` esetén a stack unwinding során a kilépett hatókörök automatikus objektumai destruálódnak, egészen addig, amíg a kivételt el nem kapják. A `goto`-val történő kiugrás szintén lezárja az elhagyott hatókör automatikus objektumait; ezért a `goto` használata C++-ban különösen óvatosságot igényel, ha az ugrás objektumokat kerül meg.

Fontos kivétel a `std::exit`: ez nem hívja meg az automatikus tárolási idővel rendelkező objektumok destruktorát, a statikus tárolási idővel rendelkező objektumokét viszont igen. Ez a Leállás, cleanup és nem lokális vezérlés (3.5) részben tárgyalt nem lokális vezérléshez kapcsolódik: a `std::exit`, a `std::abort` és a `std::terminate` nem tekinthető úgy, mintha a normál scope exit lefutna. A destruktorokra épülő takarítás ezért csak a normál visszatérés, a kivétel és a blokkból való kilépés esetén garantált.

A destruktor hívása megkezdi az objektum élettartamának végét. A tároló ettől még létezhet: az automatikus objektum tárolója a stack frame része, és a frame lebontásával szabadul fel. A dinamikus objektum tárolója csak a `delete` deallokációs lépésével szabadul fel. A kettő nem azonos.

20.3.2 A részobjektumok destrukciójának fordított sorrendje

Egy osztály destruktora nem csak a saját törzsét futtatja: sorra hívja a nem statikus adattagok és a bázisosztályok destruktorait is. A konstrukció sorrendje és a destrukció sorrendje egymás tükörképe. A konstrukció sorrendje:

1. a virtuális bázisok,
2. a közvetlen bázisok a bázislista sorrendjében,
3. a nem statikus adattagok a deklarációs sorrendben,
4. végül a konstruktor törzse.

A destrukció sorrendje ennek fordítottja:

1. a destruktor törzse,
2. az adattagok fordított deklarációs sorrendben,
3. a közvetlen bázisok fordított bázislista-sorrendben,
4. végül a virtuális bázisok.

Sorrend	Konstrukció	Destrukció

Sorrend	Konstrukció	Destrukció
első	virtuális bázisok	destruktor törzs
második	közvetlen bázisok	adattagok fordított deklarációs sorrendben
harmadik	adattagok deklarációs sorrendben	közvetlen bázisok fordított sorrendben
utolsó	konstruktor törzs	virtuális bázisok

A leggyakoribb hiba, hogy a taginicializáló lista (mem-initializer lista) sorrendjét tekintjük mérvadónak. Nem az: a tényleges inizializálási és destrukciós sorrendet a deklarációs sorrend dönti el. Ha a taginicializáló lista eltér ettől, a fordító figyelmeztethet, de a szemantika a deklarációs sorrend.

Tekintsük a következő példát. A `D` bázisa `B`, adattagjai `A a; C c;`. A konstrukció: `B()`, `A()`, `C()`, majd a `D` törzse. A destrukció: a `D` törzse, majd `~C()`, `~A()`, végül `~B()`.

```
#include <iostream>

struct A { A() { std::cout << "A()\n"; } ~A() { std::cout << "~A()\n"; } };
struct B { B() { std::cout << "B()\n"; } ~B() { std::cout << "~B()\n"; } };
struct C { C() { std::cout << "C()\n"; } ~C() { std::cout << "~C()\n"; } };

struct D : B {
    A a;
    C c;
    D() { std::cout << "D()\n"; }
    ~D() { std::cout << "~D()\n"; }
};

int main() {
    D d;
}
```

A fordítás a tanuló laborjában például ezzel a paranccsal történhet:

```
g++ -std=c++20 -Wall -Wextra -pedantic -o order order.cpp
```

A program várható kimenete (a tényleges futtatás a labor feladata): `B()`, `A()`, `C()`, `D()`, `~D()`, `~C()`, `~A()`, `~B()`. A példa nem állítja, hogy lefutott; a kimenet a fenti sorrendi szabályokból vezethető le. Ellenőrzési feltétel: a `~C()` megelőzi a `~A()`-t, és a `~B()` az utolsó.

20.3.3 Kivétel a konstrukció alatt: részleges felépülés

Ha egy konstruktor a törzsében vagy egy adattag/bázis konstruktorában kivételt dob, az objektum élettartama nem kezdődött el. A szabvány nyelvén ez nem „részben felépült objektum”, hanem egy olyan helyzet, amikor a teljes objektum nem jött létre; a már teljesen felépült részobjektumok (subobjectek) élettartama viszont véget ér, és a destruktoruk lefut. A teljes objektum destruktora nem fut le. A „részleges konstrukció” (partial construction) itt pedagógiai név erre a helyzetre.

A szabály lényege: a konstruktor törzsének megkezdése előtt a bázisok és az adattagok már felépültek; ha a törzs dob, ezek destruálódnak. Ha egy adattag konstruktora dob, az utána következő adattagok és a törzs már nem épülnek fel; az előbb felépült adattagok és bázisok destruálódnak. A dobó adattag destruktora nem fut, mert annak a konstrukciója nem fejeződött be. A teljes objektum destruktora sem fut.

```
#include <iostream>
#include <stdexcept>

struct Base {
    Base() { std::cout << "Base()\n"; }
    ~Base() { std::cout << "~Base()\n"; }
};
struct Member {
    Member() { std::cout << "Member()\n"; }
    ~Member() { std::cout << "~Member()\n"; }
};
struct Thrower {
    Thrower() { std::cout << "Thrower()\n"; throw std::runtime_error("boom"); }
    ~Thrower() { std::cout << "~Thrower()\n"; }
};

struct Derived : Base {
    Member m;
    Thrower t;
    Derived() { std::cout << "Derived()\n"; }
    ~Derived() { std::cout << "~Derived()\n"; }
};

int main() {
    try {
        Derived d;
    } catch (const std::exception& e) {
        std::cout << "caught: " << e.what() << "\n";
    }
}
```

```
g++ -std=c++20 -Wall -Wextra -pedantic -o partial partial.cpp
```

A várható kimenet: `Base()`, `Member()`, `Thrower()`, majd `~Member()`, `~Base()`, végül `caught: boom`. A `~Thrower()` és a `~Derived()` nem fut le. A `Derived` törzse sem fut le. A `Member` és a `Base` destruktora azért fut, mert ők teljesen felépültek. A destrukció sorrendje itt is fordított: a `Member` a `Base` előtt destruálódik.

Ebből következik a gyakorlati szabály: a konstruktor törzsében szerzett, nem adattag erőforrásokért a konstruktornak kell felelnie. Ha a törzs dob, az adattagok destruktora lefut, de a törzsben nyitott file descriptor, lefoglalt memória vagy zárolt mutex nem lesz automatikusan felszabadítva. Ezért érdemes minden erőforrást adattaggá tenni, amelynek a destruktora takarít. A részleges felépülés így válik kezelhetővé.

20.3.4 A destruktor default és noexcept jellege

A destruktor kivételspecifikációja C++11 óta implicit: ha nincs kiírva, a destruktor pontosan akkor `noexcept(true)`, ha minden bázis és adattag destruktora az; ha valamelyik `noexcept(false)`, akkor a destruktor is `noexcept(false)`. Ez a szabály **a saját, felhasználó által írt destruktorra is érvényes**, ha nem ad meg `noexcept` specifikációt; a törzsében álló `throw` nem befolyásolja az implicit specifikációt, az csak a részobjektumoktól függ. Explicit `noexcept(true)` vagy `noexcept(false)` mindkét irányban felülírja. A `= default` destruktor nem azt jelenti, hogy nem történik semmi: a tagok és bázisok destruktorát hívja, csak a definíciót a fordító állítja elő.

Eset	A destruktor noexcept specifikációja	Következmény
nincs felhasználói destruktor; minden adattag és bázis destruktora `noexcept(true)`	`noexcept(true)`	a destruktor nem dob
valamely adattag vagy bázis destruktora `noexcept(false)`	`noexcept(false)`	a destruktor dobhat, a hívónak kezelnie kell
felhasználói destruktor `noexcept` megadása nélkül	mint az implicitnél: `noexcept(true)`, ha minden bázis és tag destruktora az, különben `noexcept(false)`	`noexcept(true)` esetén a törzsből kilépő kivétel `std::terminate`-hez vezet
felhasználói destruktor explicit `noexcept(true)`, `noexcept(false)` tag mellett	`noexcept(true)`	a tag destruktorából kilépő kivétel is `std::terminate`
felhasználói destruktor `noexcept(false)`	`noexcept(false)`	tudatos kivétel; ritka és veszélyes

```
// destructor_exception_spec.cpp - C++20
#include <type_traits>

struct Risky  { ~Risky() noexcept(false) {} };

struct Owner  { Risky r; ~Owner() {} };                  // saját dtor, nincs
 ↪ specifikáció
struct Forced { Risky r; ~Forced() noexcept(true) {} };  // explicit
 ↪ noexcept(true)
struct Plain  { int v; ~Plain() { /* throw itt is csak terminate-et adna */ }
 ↪ };

static_assert(!std::is_nothrow_destructible_v<Owner>);   // a tagtól örökli:
 ↪ noexcept(false)
static_assert( std::is_nothrow_destructible_v<Forced>); // explicit felülírás
static_assert( std::is_nothrow_destructible_v<Plain>);   // minden részobjektum
 ↪ noexcept

int main() {}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic destructor_exception_spec.cpp` figyelmeztetés nélkül fordul, tehát mindhárom `static_assert` teljesül: az `Owner` saját destruktora a `Risky` tagtól `noexcept(false)`-t örököl, a `Forced` explicit `noexcept(true)`-ja ezt felülírja.

A destruktornak nem szabad kivételt dobnia. Ha egy `noexcept(true)` destruktor mégis dob, a program `std::terminate`-mel leáll. Ha `noexcept(false)` destruktort használunk, és a destruktor egy másik kivétel miatti unwinding közben dob, az is `std::terminate`-hez vezet. A takarítási hibát ezért naplózni kell, vagy egy külön, explicit `close()` metódusra kell bízni, amelynek a hibáját a hívó kezelheti. A destruktor feladata a források felszabadítása, nem a hibák propagálása.

20.3.5 delete, destruktor és a tároló felszabadítása

A `delete` kifejezés két lépést végez: először meghívja a dinamikus objektum destruktorát, majd felszabadítja a tárolót az `operator delete` segítségével. A `delete[]` ugyanezt teszi a tömb minden elemére, majd az `operator delete[]`-et hívja. A destruktor önmagában nem szabadítja fel a tárolót; a tároló felszabadítása külön művelet.

Ez akkor válik igazán láthatóvá, ha a konstrukció és a tárolókezelés szétválik. A placement new csak a konstrukciót végzi el egy már meglévő tárolóban; a destruktort explicit kell meghívni, a tárolóról pedig külön kell gondoskodni.

```
#include <iostream>
#include <new>

struct Tracer {
    Tracer() { std::cout << "Tracer()\n"; }
    ~Tracer() { std::cout << "~Tracer()\n"; }
};

int main() {
    alignas(Tracer) unsigned char buf[sizeof(Tracer)];
    Tracer* p = new (buf) Tracer;
    p->~Tracer();
}
```

```
g++ -std=c++20 -Wall -Wextra -pedantic -o storage storage.cpp
```

A várható kimenet: `Tracer()`, `~Tracer()`. A `buf` tárolója tovább él; a `delete p` itt nem definiált viselkedés lenne, mert a `p` nem `new`-val jött létre. A destruktor csak az objektum élettartamát zárta le, a tárolót nem szabadította fel. Ugyanez a kettéválasztás figyelhető meg a `std::vector` esetében is: a `clear()` destruálja az elemeket, de a kapacitás megmarad; a `shrink_to_fit()` a tárolót is felszabadíthatja. Az allocator modellben a `destroy` és a `deallocate` külön művelet; erről az Allocator model és allocator_traits (37.1) rész szól.

A `delete` és a destruktor összekeverése tipikus hiba. A `delete` a destruktor és a deallokáció együttese; a destruktor nem „delete-el”. A placement new-val létrehozott objektumra a `delete` használata UB. A `delete[]` elhagyása tömb esetén szintén UB. A polimorf bázispointeren történő `delete` csak virtuális destruktorral biztonságos; erről a Virtuális destruktor, slicing és ownership (24.3) rész szól.

20.3.6 Részleges inicializálás C-ben és C++-ban

C-ben nincs destruktor, ezért a részleges inicializálás takarítását kézzel kell megoldani. A tipikus minta a `goto cleanup`: minden erőforrás megszerzése után a hibás ág a közös cleanup címkére ugrik, amely a már megszerzett erőforrásokat felszabadítja. A cleanup ágnak minden lehetséges hibapontot le kell fednie, és a sorrendet kézzel kell helyesen tartani.

```
int open_pair(const char *path, char **out_buf, size_t n) {
    int fd = open(path, O_RDONLY);
    if (fd < 0) return -1;
    char *buf = malloc(n);
    if (buf == NULL) {
        close(fd);
        return -1;
    }
    *out_buf = buf;
    return fd;
}
```

Ez részlet; a hívó felelős a `close(fd)` és a `free(buf)` hívásáért. A C++ megfelelője az, hogy a már felépült adattagok destruktora automatikusan takarít, de csak akkor, ha minden erőforrás adattag. A következő részlet ezt szemlélteti:

```
#include <fcntl.h>
#include <unistd.h>
#include <cstddef>
#include <memory>
#include <stdexcept>

class Resource {
    int fd_ = -1;
    std::unique_ptr<char[]> buf_;
public:
    Resource(const char* path, std::size_t n)
        : fd_(open(path, O_RDONLY)), buf_(new char[n]) {
        if (fd_ < 0) throw std::runtime_error("open");
    }
    ~Resource() { if (fd_ >= 0) close(fd_); }
    Resource(const Resource&) = delete;
    Resource& operator=(const Resource&) = delete;
};
```

Ez részlet; a teljes RAII-kezelést a RAII wrapper file descriptorhoz és lockhoz (21.2) rész tárgyalja. A példa lényege: a `fd_` nyers `int`, a `buf_` viszont `std::unique_ptr`. Ha a `buf_` allokációja `std::bad_alloc`-ot dob, a `fd_` már nyitva van, de nincs olyan destruktora, amely bezárná; a `Resource` destruktora nem fut le, mert a konstrukció nem fejeződött be. A `fd_` nyitva marad. A megoldás az, hogy a file descriptor is RAII-adattaggá válik (például egy `unique_fd` wrapperrel), így a részleges felépülés alatt is lezáródik. A C cleanup-ág és a C++ RAII ugyanazt a célt szolgálja: a már megszerzett erőforrások felszabadítását a hibaágakon is. A C++ eszköze az adattagok destruktora; a konstruktor törzsében szerzett nem adattag erőforrásokért továbbra is a konstruktornak kell felelnie.

20.3.7 Gyakori hibák és javításuk

A leggyakoribb hibák: kivétel dobása a destruktorban (a `noexcept(true)` miatt `std::terminate`); a `delete` és a destruktor összekeverése; a taginicializáló lista sorrendjének összetévesztése a deklarációs sorrenddel; nyers erőforrás a konstruktor törzsében, amely nem adattag, ezért részleges felépülés esetén nem szabadul fel; `delete` használata placement new-val létrehozott objektumra; a `delete[]` elhagyása tömb esetén; virtuális destruktor hiánya polimorf bázisnál. A javítás

rendre: a destruktor ne dobjon; a `delete` a destruktor és a deallokáció együttese; a tagok sorrendjét a deklaráció dönti el; minden erőforrást adattaggá kell tenni; placement new esetén explicit destruktorhívás és külön tárolókezelés; tömbnél `delete[]`; polimorf bázisnál virtuális destruktor.

20.3.8 Összefoglaló

A destruktor az objektum élettartamának végét jelöli; a tároló felszabadítása ettől független művelet. Automatikus objektumnál a scope exit hívja a destruktort, a blokkban deklarált objektumok fordított sorrendben destruálódnak. Az osztály destruktora a törzs után az adattagokat fordított deklarációs sorrendben, majd a bázisokat fordított sorrendben destruálja. Ha a konstrukció kivétellel megszakad, a már felépült részobjektumok destruálódnak, a teljes objektum destruktora nem. A destruktor alapértelmezés szerint `noexcept(true)`, ezért nem szabad dobnia. A `delete` a destruktort és a deallokációt együtt végzi; a placement new és az explicit destruktorhívás esetén a kettő szétválik.

20.3.9 Feladatok

1. Adott a vázlat: `struct Base { ~Base(); }; struct M { ~M(); }; struct D : Base { M m1; M m2; ~D(); };` Írd fel a konstrukció és a destrukció sorrendjét, és indokold, miért nem a taginicializáló lista sorrendje dönt.
2. Írj programot, amelyben a `Derived` bázisa `Base`, adattagjai `Member m; Thrower t;`, és a `Thrower` konstruktora `std::runtime_error`-t dob. Naplózd a konstruktorokat és destruktorokat. Fordítsd `g++ -std=c++20 -Wall -Wextra -pedantic` parancsal. Add meg a várható kimenetet és az indoklást.
3. Az alábbi részletekben találj hibát, és javítsd: (a) `struct Bad { ~Bad() { throw std::runtime_error("x"); } };` (b) `Tracer* p = new (buf) Tracer; delete p;`.

20.3.10 Megoldások

20.3.10.1 Az első feladat megoldása

A konstrukció sorrendje: `Base`, `m1`, `m2`, majd a `D` törzse. A destrukció sorrendje: a `D` törzse, `~m2`, `~m1`, végül `~Base`. Az adattagok a deklarációs sorrendben inicializálódnak, nem a taginicializáló lista sorrendjében; a destruktorok ennek fordítottjában futnak. Ellenőrzési feltétel: a felsorolt sorrend minden lépése szerepel, és a `m2` a `m1` előtt destruálódik.

20.3.10.2 A második feladat megoldása

A fejezet `Derived`/`Thrower` példája pontosan ezt mutatja. A várható kimenet: `Base()`, `Member()`, `Thrower()`, majd a `Thrower` konstruktorában a dobás után `~Member()`, `~Base()`, végül `caught: boom`. A `~Thrower()` és a `~Derived()` nem fut, mert a `Thrower` nem épült fel, a `Derived` élettartama pedig el sem kezdődött. Ellenőrzési feltétel: a kimenetben a `~Thrower()` és a `~Derived()` nem szerepel, a `~Member()` a `~Base()` előtt van, és a `caught` a végén.

20.3.10.3 A harmadik feladat megoldása

(a) A destruktor alapértelmezés szerint `noexcept(true)`, ezért a benne dobott kivétel `std::terminate`-hez vezet. A javítás: a destruktor ne dobjon; a hibát naplózni kell, vagy a takarítást egy explicit `close()` metódusra kell bízni. Ha mégis `noexcept(false)`-t használunk, a hívónak kell kezelnie, és ügyelni kell arra, hogy egy másik kivétel miatti unwinding közben a dobás ismét `std::terminate`-hez vezet. (b) A `delete p` nem definiált viselkedés, mert a `p` placement new-val jött létre, nem `new`-val; a helyes hívás `p->~Tracer();`. A `buf` tárolója tovább él; a `delete` a tárolót is felszabadítaná, ami itt hibás. Ellenőrzési feltétel: a javított kódban a destruktor nem dob, és a placement new-os objektumra nincs `delete`.

20.3.11 Verzió- és forráskorlátok

A fejezet a C++20 nyelvi magra támaszkodik. A C++23 és a konkrét fordítók eltéréseit a laborban, a választott fordító dokumentációjával kell ellenőrizni. A megadott kiinduló URL-ek nem kerültek feldolgozásra, ezért a fejezet nem tartalmaz ellenőrzött külső forráshivatkozást; a szabványparagrafusok és a mérési eredmények ellenőrzése a labor része.

20.4 Temporary, value category és dangling

A temporary objektumok (ideiglenesek) és az értékkategóriák (value category) adják a modern C++ egyik legtöbb buktatót rejtő területét. A témát érdemes a nyelvi memóriamodell felől közelíteni: az értékkategória az *kifejezés* tulajdonsága, nem a típusé, és az ideiglenesek élettartamát pontosan meghatározott szabályok szerint hosszabbítja meg a fordító — de csak néhány, jól körülhatárolt helyzetben. Ez az alfejezet ezeket a szabályokat rögzíti; a `string_view`/`span` konkrét dangling-tünetei a későbbi, std::string, std::string_view és szövegownership (28.2) és span, byte és bounds-szerződések (31.3) részekben kerülnek elő, itt csak előkészítjük a mechanizmust.

20.4.1 Kifejezéstípus és értékkategória

A C++-ban minden kifejezésnek van egy *statikus típusa* és egy *értékkategóriája*. A típus a deklarációból és a környezetből következik; az értékkategória azt írja le, hogy a kifejezés milyen módon használható fel (azonosítható-e egy objektum, mozgatható-e, köthető-e rá referencia). A két fogalom független: `std::string s;` esetén `s` típusa `std::string`, kategóriája *lvalue*; `std::move(s)` típusa `std::string&&`, kategóriája *xvalue*; `std::string{"x"}` típusa `std::string`, kategóriája *prvalue*. Ugyanaz a típus tehát három kategóriában is megjelenhet, attól függően, hogyan jött létre a kifejezés.

A definíciók (C++20 alapján):

- **lvalue**: olyan glvalue, amely nem xvalue. Azonosítható vele egy objektum vagy függvény, és implicit módon nem jelöl erőforrás-átadást.
- **xvalue** (expiring value): olyan glvalue, amely egy objektumot vagy bit-fieldet jelöl, és amelynek erőforrásai újrahasznosíthatók; tipikusan `std::move(...)` eredménye, tag-hozzáférés egy rvalue-n, vagy `T&&` visszatérés.
- **prvalue** (pure rvalue): olyan kifejezés, amelynek kiértékelése inicializál egy objektumot vagy bit-fieldet, vagy kiszámítja egy operátor operandusának értékét. Például literál, `T{}`, `f()` ha `T`-t ad vissza értékként, aritmetikai kifejezés.

- **glvalue** = lvalue xvalue („generalized lvalue”): olyan kifejezés, amelynek kiértékelése egy objektum, bit-field vagy függvény *azonosságát* határozza meg. Ez nem azonos a „címezhető” kifejezéssel: egy bit-field vagy egy `register`-minősítésű változó glvalue lehet, miközben nem vehető a címe (`&` nem alkalmazható rá).
- **rvalue** = prvalue xvalue: minden, ami mozgatható, illetve ami nem glvalue.

A gyakorlatban ezt a térképet érdemes fejben tartani:

Kifejezés	Kategória	Miért
`x` (változó)	lvalue	van azonosítója
`*p`, beépített `a[i]` (lvalue tömb/pointer)	lvalue	a jelölt objektumot azonosítja
`std::move(x)`	xvalue	`T&&` cast
`f()` ahol `T f()`	prvalue	értéket inicializál
`f()` ahol `T&& f()`	xvalue	`T&&` visszatérés
`x.m` ahol `x` lvalue	lvalue	tag-hozzáférés megőrzi
`T{}.m` ahol `T{}.m` tag	xvalue	tag-hozzáférés rvalue-n
`42`, `1+2`	prvalue	számított érték

A kifejezés **típusa** (amelyre a nyelv a kifejezés típusaként hivatkozik: sosem referencia), az **értékkategória** és a `decltype(kifejezés)` eredménye három külön adat. Ezt az alábbi táblázat fogja össze `std::string s;` mellett:

Kifejezés	A kifejezés típusa	Értékkategória	`decltype(kifejezés)`
`s`	`std::string`	lvalue	`std::string` (zárójel nélküli név: deklarált típus)
`(s)`	`std::string`	lvalue	`std::string&`
`std::move(s)`	`std::string`	xvalue	`std::string&&`
`std::string{}`	`std::string`	prvalue	`std::string`

A kifejezés típusa tehát `std::string`; a `std::string&&` a `decltype` eredménye (az értékkategóriát kódolja referenciaként), nem a kifejezés típusa.

Fontos: egy változó neve *mindig* lvalue, még akkor is, ha a változó típusa `T&&`. Ez az alapja annak, hogy `std::move` egyáltalán létrejöhet: azért kell, hogy a nevesített változóból xvalue-t csináljunk.

20.4.2 Temporary materialization

C++17-től a modell „temporary materialization” néven ismert: ha egy prvalue-t olyan helyen használunk, ahol glvalue-t vár a nyelv (referencia-kötés, tag-hozzáférés, `sizeof`, feltételes operátor egyik ága), a fordító *materializál* egy ideiglenes objektumot, és a továbbiakban arra hivatkozunk. Ez a modell váltotta fel a korábbi, „mikor keletkezik temporary” kérdésre adott homályosabb választ. Gyakorlati következménye: a prvalue önmagában nem rendelkezik címmel, csak a materializált objektum. Ez teszi lehetővé a garantált copy elisiont (C++17): `T x = T{};` esetén még egy mozgatás sem történik, mert a prvalue közvetlenül `x`-et inicializálja.

A materializálást a nyelv automatikusan végzi, nem kell rá kifejezést írni. A fontos pont az, hogy a materializált ideiglenes élettartama a *full expression* végéig tart, kivéve, ha a lifetime extension szabályai közbe lépnek.

20.4.3 Full expression és lifetime extension

Full expression (teljes kifejezés) az a legnagyobb kifejezés, amely nem része egy másik kifejezésnek; a `;`-ig, `if (...)` zárójeléig, `for (...)` fejéig, `return` operandusáig tart. A materializált ideiglenesek alapértelmezés szerint a full expression végén pusztulnak el, fordított sorrendben a létrehozásukhoz képest.

A **lifetime extension** (élettartam-hosszabbítás) kivétel, és nem univerzális szabály: a hosszabbítás a kötés *kontextusától* függ, nem pusztán attól, hogy a referencia közvetlenül egy prvalue-ra kötődik. Az alapeset (C++20 [class.temporary]) az, amikor a referencia:

- közvetlenül egy prvalue-ra kötődik (`const T& r = T{};`, `T&& r = T{};`),
- egy tagra kötődik, amely egy materializált temporary tagja (`const T& r = S{}.member;`), vagy
- egy tömb egy elemére kötődik, ahol a tömb temporary.

A hosszabbítás **nem** érvényes, ha a referencia egy függvény visszatérési értéke (`const T& f() { return T{}; }` — a temporary a `return` full expression végén megszűnik, a hívó egy dangling referenciát kap), ha függvényparaméterre kötődik (`void g(const T&); g(T{});` — a temporary a hívás teljes kifejezésének végén megszűnik), ha egy függvényhíváson keresztül érjük el (`S{}.getRef()` — a hívás eredménye nem „örökli” a temporary élettartamát), vagy ha `std::initializer_list`-t adunk vissza függvényből.

Kontextusonként (a `struct A { const T& r; };` aggregátummal):

Kötés helye	Példa	A temporary élettartama
lokális referencia	`const T& r = T{1};`	a referencia hatókörének végéig
referencia-paraméter	`g(T{1});`	a hívó teljes kifejezésének végéig
`return` operandusa	`const T& f() { return T{1}; }`	a `return` végéig — a hívó lógó referenciát kap
kapcsos aggregátum-init	`A a{T{1}};`	az `a` élettartamáig
zárójeles aggregátum-init (C++20)	`A b(T{1});`	**nem** hosszabbodik: az utasítás végéig
new-initializer	`new A{T{1}}`	a `new`-t tartalmazó teljes kifejezés végéig — utána lóg

```
// ref_member_lifetime.cpp - C++20: hol hosszabbodik a temporary élettartama?
#include <cstdio>

struct T {
    int v;
    explicit T(int x) : v(x) {}
    ~T() { std::printf("  ~T(%d)\n", v); }
};
struct A { const T& r; };

int main() {
    std::puts("A a{T{1}}:");
    A a{T{1}};                    // kapcsos aggregátum-init: a T{1} élettartama
 ↪  a-ig nyúlik
    std::puts("  utasitas vege");
```

```
    std::puts("A b(T{2}):");
    A b(T{2});                    // C++20 zárójeles aggregátum-init: NINCS
↪   hosszabbítás
    std::puts("  utasitas vege");
    std::puts("new A{T{3}}:");
    A* p = new A{T{3}};           // new-initializer: a temporary a teljes
↪   kifejezés végéig él
    std::puts("  utasitas vege");
    delete p;
    (void)a; (void)b;
    std::puts("main vege");
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -O0` kimenete: az `A a{T{1}}` után a `~T(1)` csak a `main vege` után fut; az `A b(T{2})` és a `new A{T{3}}` esetén a `~T(2)`, illetve a `~T(3)` még az adott utasítás végén lefut, tehát a `b.r` és a `p->r` utána lógó referencia.

A `std::initializer_list` külön eset: a `{...}` mögötti háttértömb egy temporary, amelynek élettartama pontosan úgy hosszabbodik, mintha referenciát kötnénk rá. Ezért `auto il = {1, 2, 3};` biztonságos (a háttértömb `il` élettartamáig él), de `return {1, 2, 3};` egy `std::initializer_list`-t visszaadó függvényből dangling: a háttértömb a `return` után megszűnik, a visszaadott lista egy halott tömbre mutat.

20.4.4 Visszaadott referencia, initializer_list és range-for csapdák

A leggyakoribb hibaforrások:

1. **Visszaadott referencia lokálisra vagy temporary-ra.**

   ```
   const std::string& bad() { return std::string{"x"}; }   // dangling
   const std::string& bad2() { std::string s; return s; }   // dangling
   ```

   Mindkettő UB, amint a hívó használja a visszaadott referenciát. A `-Wall -Wextra` ezt gyakran jelzi; a `-Wdangling-reference` (GCC 13+) célzottabban szűr.

2. **Tag-hozzáférés függvényhíváson keresztül.**

   ```
   struct Holder { std::string name; const std::string& ref() const { return
   ↪  name; } };
   const std::string& r = Holder{"x"}.ref();   // dangling: a Holder temporary
   ↪  a full expression végén megszűnik
   ```

   A `Holder{"x"}` temporary élettartama csak a full expression végéig tart, mert a `ref()` visszatérési értéke nem „örökli” a temporary élettartamát. Ez a `string_view`/`span` későbbi dangling-mintáinak előképe.

3. **Range-for és a range-initializer.**

   ```
   struct H { std::vector<int> v; const std::vector<int>& items() const {
   ↪  return v; } };
   for (int x : H{}.items()) { /* ... */ }   // C++20: dangling; C++23
   ↪  (__cpp_range_based_for >= 202211L): biztonságos
   ```

C++20-ban a range-for desugaringja csak a `__range` referenciára kötött közvetlen temporary-t hosszabbítja meg; a `H{}` temporary a full expression végén megszűnik, és a ciklus egy halott `vector`-on iterál. A **P2718R0** („Fix for Range-based for Loop”, P2644R1 nyomán) ezt Defect Report-ként javítja: a range-initializerben keletkező *minden* temporary élettartama a ciklus végéig tart. Ez a változás **C++23 nyelvi változás** (a CWG 900 státusza C++23); nem követeljük meg, hogy a C++20 mód visszamenőleg ugyanezt jelentse. A támogatás fordítónként eltér, ezért feature-teszttel ellenőrizendő: `__cpp_range_based_for >= 202211L`. A hosszabbítás nem általános: a **függvényparaméter-objektumokat** nem terjeszti ki, ezért például `for (auto e : f2(g()))`, ahol `f2` érték szerinti paraméterére kapott temporary-ra referenciát ad vissza, továbbra sem biztonságos. A C++20 mód szerint a fenti kód nem hordozható.

4. **Tag referencia-taggal (reference member) rendelkező osztályban.**

```
// T: nyomkövető destruktoros típus (ctor/dtor üzenettel)
struct S { const T& r; };                          // aggregátum
struct C { const T& r; explicit C(const T& t) : r(t) {} };   //
 ↪ konstruktoros referencia-tag
S s{ T{1} };       // kapcsos aggregátum-init: a T{1} s élettartamáig él
 ↪ (NEM dangling)
S p(T{2});         // C++20 zárójeles aggregátum-init: nincs hosszabbítás,
 ↪ T{2} az utasítás végén megszűnik
C c{ T{3} };       // konstruktoros tag: nincs hosszabbítás, T{3} az
 ↪ utasítás végén megszűnik
S* n = new S{ T{4} };   // new-initializer: nincs hosszabbítás
```

A szabály tehát nem „a reference tag mindig dangling”: **kapcsos aggregátum-inicializálásnál** a referencia-taghoz kötött temporary élettartama a tag objektumáéig hosszabbodik. Nem hosszabbodik zárójeles (C++20) aggregátum-inicializálásnál, a konstruktor tag-inicializálójában és a `new`-initializerben. Destruktor-nyomkövetéssel (GCC 13.3, `-std=c++20`): `S s{T{1}}` esetén a `T1` destruktora a blokk végén fut, a másik három esetben közvetlenül az inicializáló utasítás után — ezek a valóban hibás változatok (nem dereferáljuk őket).

Ezek a minták mind azt illusztrálják, hogy a lifetime extension nem „ragadós”: nem öröklődik függvényhíváson, visszatérésen, konstruktoros reference tagon, zárójeles aggregátum-inicializáláson és `new`-initializeren keresztül (a kapcsos aggregátum-inicializálás a kivétel).

20.4.5 Tipikus hibák és javításuk

- **`auto&&` mindenre**: az `auto&&` nem varázsszer. Ha egy függvény referenciát ad vissza, az `auto&&` sem hosszabbítja meg az eredeti temporary-t.
- **`std::move` túlhasználata**: a `std::move` nem hosszabbít semmit, csak kategóriakonverzió. Visszaadott referenciát „megmenteni” vele nem lehet.
- **Range-for value vs. reference**: `for (auto x : v)` másol, `for (auto& x : v)` módosít; a temporary range-re mindkettő ugyanúgy dangling, ha a range-initializer temporary-it nem hosszabbítja meg a szabály.
- **`initializer_list` visszaadása**: a `return {1,2,3};` egy `std::initializer_list`-t adó függvényből tipikusan dangling; a helyes minta `std::array` vagy `std::vector` visszaadása.
- **Diagnosztika**: a `-fsanitize=address` (ASan) a legtöbb ilyen hibát elkapja a használat pillanatában, de a heapen túl nem mindig; a `-fsanitize=undefined` a referencia-élettartamot nem feltétlenül ellenőrzi. A fordító warningjai (GCC `-Wdangling-reference`, Clang `-Wdangling`) és a kód review a fő eszköz.

Az alábbi példa önállóan fordítható, és szemlélteti a hosszabbítás működését és elmaradását. A program a `Probe` destruktorában ír a kimenetre, így a létrehozás és megszűnés sorrendje megfigyelhető.

```
// lifetime_demo.cpp
// Fordítás: g++ -std=c++20 -Wall -Wextra -O0 -g lifetime_demo.cpp -o
 ↪  lifetime_demo
#include <cstdio>
#include <string>
#include <utility>

struct Probe {
    std::string name;
    explicit Probe(std::string n) : name(std::move(n)) {
        std::printf("ctor %s\n", name.c_str());
    }
    ~Probe() { std::printf("dtor %s\n", name.c_str()); }
    const std::string& ref() const { return name; }
};

Probe makeProbe() { return Probe{"made"}; }
const std::string& dangling() { return std::string{"temp"}; }

int main() {
    // 1) lifetime extension: az ideiglenes a referencia élettartamáig él
    const std::string& a = std::string{"extended"};
    std::printf("a=%s\n", a.c_str());

    // 2) extension elmarad: a temporary a full expression végén megszűnik
    const std::string& b = makeProbe().ref();   // dangling; itt már nem lenne
     ↪  szabad használni
    (void)b;

    // 3) közvetlen temporary-kötés a return után: dangling
    const std::string& c = dangling();          // dangling; használat UB
    (void)c;

    return 0;
}
```

A kimenet sorrendje (a `Probe` példányok esetében) a következő elvet követi: `ctor` és `dtor` párokban jelenik meg, kivéve, ahol a hosszabbítás megtörténik. A `b` és `c` esetén a temporary (`made`, illetve `temp`) destruktora a full expression végén fut le, mielőtt a program a következő utasításra lépne — ezt a kimenet sorrendje mutatja. A példa futtatása nem helyettesíti a `-fsanitize=address` alatti ellenőrzést: a dangling referencia használata UB, és a megfigyelt viselkedés nem garantált.

20.4.6 Feladatok

1. feladat (magyarázó). Az alábbi kifejezések közül melyik lvalue, xvalue, prvalue? Indokolj. `x`, `std::move(x)`, `x + 1`, `*p`, `f()` ahol `int f()`, `f()` ahol `int&& f()`, `x.m` ahol `x` lvalue, `std::move(x).m`.

2. feladat (kódolási). Az alábbi kódrészlet danglinget tartalmaz. Írd át úgy, hogy ne legyen dangling, és indokold a választott javítást. Törekedj arra, hogy a szemantika (a hívó által látott érték) ne változzon.

```
const std::string& name_of(const std::string& prefix) {
    return prefix + "-suffix";
}
```

3. feladat (diagnosztikai). Tervezz olyan tesztkörnyezetet, amely képes megkülönböztetni a „C++20-as, P2718R0 nélküli” és a „C++23-as, P2718R0-val javított” range-for viselkedést. Milyen megfigyelést rögzítesz, és hogyan támasztod alá, hogy a két verzió között valóban eltér a viselkedés?

20.4.7 Megoldások

1. megoldás. `x`: lvalue (nevesített változó). `std::move(x)`: xvalue (kategóriakonverzió `T&&`-ra). `x + 1`: prvalue (számított érték). `*p`: lvalue (a pointer által jelölt objektum). `f()` `int f()` esetén: prvalue. `f()` `int&& f()` esetén: xvalue (a visszatérési típus `T&&`, ez a kategóriakonvenció szerint xvalue). `x.m` lvalue `x` esetén: lvalue (tag-hozzáférés megőrzi a kategóriát). `std::move(x).m`: xvalue (tag-hozzáférés rvalue-n). A döntő szabály: a tag-hozzáférés megőrzi az operandus kategóriáját (rvalue objektum tagja xvalue), az indexelés viszont **típusfüggő**: beépített `a[i]` (tömb/pointer) lvalue, kivéve ha a tömb maga rvalue (`std::move(arr)[i]` xvalue), míg a felhasználói `operator[]` kategóriáját a visszatérési típusa adja (`T&` → lvalue, `T&&` → xvalue, `T` → prvalue); a `std::move` `T&&`-ra castol, ami xvalue-t eredményez; a literál és az aritmetikai művelet prvalue.

2. megoldás. A `prefix + "-suffix"` egy `std::string` prvalue; a visszaadott referencia a full expression végén dangling. Helyes javítások: (a) adjuk vissza értékként: `std::string name_of(const std::string& prefix) { return prefix + "-suffix"; }`; (b) ha a hívó csak olvassa, adjunk vissza `std::string`-et, és a hívó kösse `const std::string&`-hez — ez a hívó oldalán hosszabbít; (c) ha mindenképp referencia kell, a háttértárolónak a hívó élettartamáig kell élnie (pl. `static` cache, ami viszont szálbiztonsági és újrahasználati kérdéseket vet fel). Az (a) a legrobusztusabb: értékszemantika, nincs rejtett élettartam-függés. Ellenőrzés: `-Wall -Wextra -Wdangling-reference` (GCC) vagy `-Wdangling` (Clang) és `-fsanitize=address` alatt futtatva a javított kód nem jelez.

3. megoldás. Készíts két kis programot: az egyikben `H{}.items()`-et iterálunk `for (int x : H{}.items())` formában, ahol `H::items() const std::vector<int>&`-t ad vissza. A `Probe`-hoz hasonló nyomkövetéssel (vagy a `vector` méretének és tartalmának kiírásával) rögzítjük, hogy a ciklus teste érvényes adatot lát-e, illetve az ASan jelez-e. C++20 módban (és olyan fordítóval, amelynek `__cpp_range_based_for` értéke 202211L alatt van) a `H{}` temporary a full expression végén megszűnik, a ciklus dangling, az ASan `heap-use-after-free`-t jelez. C++23 módban, olyan fordítóval, amely támogatja a P2718R0-t (`__cpp_range_based_for >= 202211L`), a temporary a ciklus végéig él, az ASan nem jelez; a GCC 13.3 `lab-gcc` profilban ez a támogatás még nincs meg, ezért ott csak a C++20-as viselkedés figyelhető meg. A megfigyelést a `-std=c++20` és `-std=c++23` két buildjén, azonos forráson kell elvégezni, és a két eredményt rögzíteni — a különbség a verzióhelyes viselkedés bizonyítéka. A megoldás korlátja: ez **nyelvi (front-end) változás**, a viselkedést a fordító és annak módja (`-std`) adja, nem a standard könyvtár (libstdc++/libc++) — a próbát a `__cpp_range_based_for` nyelvi makróval kell végezni; a konkrét fordítóverziót a laborprofilban rögzíteni kell.

20.4.8 Összefoglaló

A temporary objektumok élettartamát a full expression, a materializálás és a lifetime extension hármasa szabályozza. Az lvalue/xvalue/prvalue kategóriák a *kifejezésre* vonatkoznak, nem a típusra; a glvalue az azonosítható, az rvalue a mozgatható kifejezések halmaza. A hosszabbítás csak közvetlen referencia-kötésnél, tagra kötésnél és tömb-elemre kötésnél érvényes; függvény visszatérésén, függvényhíváson, reference tagon és `std::initializer_list` visszaadásán keresztül nem. A range-for esetében a C++20 és a C++23 (P2718R0 DR) között van eltérés, amelyet a fordító és a standard library tényleges verziója dönt el. A `string_view`/`span` dangling-minták ezekre a szabályokra épülnek; a következő részekben ezeket a konkrét típusok szerződésein keresztül vizsgáljuk tovább.

20.5 Statikus és thread-local objektumok életciklusa

A statikus élettartamú globális objektumok a C++ egyik legkényelmesebb, ugyanakkor legkönnyebben félreértett eszközei. Egy démon, egy mellékprogram, egy plugin-registry vagy egy diagnosztikai szolgáltatás tipikus indulási pontja: „legyen egy globális konfiguráció, egy logger, egy regisztrációs tábla”. A kényelem ára, hogy az objektum életciklusa összekapcsolódik a folyamat indulásával és leállásával, és éppen ezen a határon keletkeznek azok a hibák, amelyek fejlesztés közben nem, éles környezetben viszont annál látványosabban jönnek elő. Ez az alfejezet a statikus és `thread_local` objektumok életciklusát a folyamat startuptól a leállásig vezeti végig, a korábban rögzített destruktor-szabályokra építve (Destruktor és részleges konstrukció (20.3)), a header- és névfeloldási ismereteket előfeltételezve (Namespace, header és névfeloldás (19.2)), és a startup-sorrendet a folyamatindulás leírására támasztva (main, startup, argumentumok és környezet (3.4)).

A tárgyalás a C++20 profilt követi, C++23 kiegészítésekkel; a szabványhelyek a forrásjegyzék C++20 tétele szerint: tárolási idők [basic.stc], élettartam [basic.life], statikus és dinamikus inicializálás [basic.start.static] és [basic.start.dynamic], blokk-hatókörű static és `thread_local` változó [stmt.dcl]. A fejezet programjait a `lab-gcc` profilban lefordítottuk és lefuttattuk; az eredményeket a buildjegyzőkönyvek rögzítik.

20.5.1 Tárolási idő, élettartam és statikus inicializálás

Két fogalmat kell szétválasztani. A **tárolási idő** (*storage duration*) azt mondja meg, meddig áll rendelkezésre az objektum tárolója. Az **élettartam** (*lifetime*) azt, hogy mikor létezik maga az objektum: osztálytípusnál nem triviális konstruktor esetén az élettartam az inicializálás befejezésével kezdődik, és a destruktorhívás megkezdésével ér véget ([basic.life]). A tároló tehát létezhet úgy is, hogy benne még, vagy már nincs objektum.

Definíció. A C++ négy tárolási időt különböztet meg:

- **statikus**: a tároló a program teljes futása alatt rendelkezésre áll. Ide tartozik a namespace-hatókörű változó, a `static` minősítésű függvény- és namespace-hatókörű változó, valamint az osztály statikus adattagja;
- **szál** (*thread*): a `thread_local` változó tárolója szálanként külön van, és a szál futása alatt áll rendelkezésre;
- **automatikus**: a blokk-hatókörű, nem `static` és nem `thread_local` változó tárolója a blokk végéig él;
- **dinamikus**: a `new` kifejezéssel (vagy allokátorral) kért tároló a felszabadításig él.

A statikus tárolási idő nem jelenti, hogy az objektum a program indulásától létezik. Egy függvény lokális `static` változójának tárolója végig rendelkezésre áll, de az objektum élettartama csak akkor kezdődik el, amikor a vezérlés először áthalad a deklarációján, és a dinamikus inicializálás befejeződik ([stmt.dcl]). Ha a függvényt soha nem hívjuk meg, a konstruktor és a destruktor sem fut le. Ugyanez igaz a blokk-hatókörű `thread_local` változóra szálanként. A következő program ezt mutatja meg:

```
// lazy_static.cpp - C++20: a tárolási idő nem azonos az élettartammal
#include <cstdio>
#include <thread>

struct Noisy {
    const char* name;
    explicit Noisy(const char* n) : name(n) { std::printf("ctor %s\n", name); }
    ~Noisy() { std::printf("dtor %s\n", name); }
};

Noisy& used()   { static Noisy n{"used"}; return n; }
Noisy& unused() { static Noisy n{"unused"}; return n; }   // sosem hívjuk

Noisy& per_thread() { thread_local Noisy n{"per_thread"}; return n; }

int main(int argc, char**) {
    std::puts("main");
    if (argc > 100) unused();          // futás közben nem teljesül
    used();
    std::thread quiet([] {});          // nem éri el a per_thread()-et
    quiet.join();
    std::thread busy([] { per_thread(); });
    busy.join();
    std::puts("end of main");
}
```

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -pthread lazy_static.cpp -o
 ↪  lazy_static
./lazy_static
```

Buildjegyzőkönyv (`lab-gcc` profil). A kimenet: `main`, `ctor used`, `ctor per_thread`, `dtor per_thread`, `end of main`, `dtor used`. Az `unused` objektum konstruktora és destruktora nem fut, a `quiet` szálon nem jön létre `per_thread` példány, a `busy` szálé pedig a szál kilépésekor szűnik meg.

20.5.1.1 A statikus tárolási idejű változók inicializálása

A statikus vagy szál tárolási idejű változó inicializálása nem három egymást követő fázis, hanem egy döntési folyamat ([basic.start.static], [basic.start.dynamic]):

1. **Statikus inicializálás.** Ha a változó **konstans inicializálást** kaphat (az inicializáló konstans kifejezés, és a kiválasztott konstruktor is `constexpr`), akkor azt kapja. Ha nem, a változó **zero-inicializálást** kap. Ez *értékszabály*, nem bájtszabály: aritmetikai típus a `0` értéket, pointer és tagpointer a saját típusának null értékét kapja, osztálynál és tömbnél

a részobjektumok rekurzívan zero-inicializálódnak, a paddingbitek nullák. Hogy ez a memóriában csupa nulla bájt-e, az ABI kérdése: a null adattag-pointer például az Itanium C++ ABI-ban (x86-64 Linux) `-1` reprezentációjú, a csupa nulla bájt pedig a 0 eltolású tagra mutat. Ezért a `memset(&x, 0, sizeof x)` nem hordozható módja egy null pointer vagy tagpointer előállításának. A kettő közül pontosan az egyik történik, és mindkettő felhasználói kód futtatása nélkül.

2. **Dinamikus inicializálás.** Ha a változó nem konstans inicializálást kapott, de van inicializálója, akkor a zero-inicializálás után futásidőben is lefut: konstruktorhívás, nem `constexpr` függvényhívás. Namespace-hatókörű változónál ez a `main` előtt (vagy legkésőbb az első használat előtt) történik, blokk-hatókörűnél az első áthaladáskor.

```
// memberptr_zero.cpp - C++20: a null tagpointer nem feltétlenül csupa nulla
 ↪  bájt
#include <cstdio>
#include <cstring>

struct S { int a; int b; };

int S::*g_mp;          // statikus tárolás: zero-inicializálás -> null
 ↪  tagpointer

int main() {
    int S::*bytes;
    std::memset(&bytes, 0, sizeof bytes);   // csupa nulla bájt
    unsigned char rep[sizeof g_mp];
    std::memcpy(rep, &g_mp, sizeof rep);
    std::printf("g_mp == nullptr: %d\n", g_mp == nullptr);
    std::printf("bytes == nullptr: %d, bytes == &S::a: %d\n",
                bytes == nullptr, bytes == &S::a);
    std::printf("g_mp elso bajtja: 0x%02x\n", rep[0]);
}
```

Buildjegyzőkönyv (`lab-gcc` profil, x86-64). A kimenet: `g_mp == nullptr: 1`, `bytes == nullptr: 0`, `bytes == &S::a: 1`, `g_mp elso bajtja: 0xff`. Ez ABI-tény, nem nyelvi garancia; egy más ABI-jú célon más eredmény is helyes lehet.

A statikus inicializálás minden dinamikus inicializálás előtt megtörténik (*strongly happens before*). Ezért a konstans vagy zero-inicializált objektumokat bármely más statikus objektum konstruktora olvashatja, és a konstans inicializált érték már ilyenkor a végleges. A szabvány megengedi, hogy az implementáció egy dinamikus inicializálást is előre, statikusan végezzen el, ha az eredmény ugyanaz lenne; erre a programozó nem építhet. Az sem következik a kategóriából, hogy a változó hol helyezkedik el a binárisban: ez az implementáció dolga.

Három példa besorolása:

Deklaráció	Inicializálás
`constexpr int kMax = 64;` (namespace-hatókör)	konstans inicializálás
`int g_port = read_port();` ahol `read_port` nem `constexpr`	zero-inicializálás (0), majd dinamikus inicializálás a `main` előtt
`int& counter() { static int c = read_port(); return c; }`	zero-inicializálás, majd dinamikus inicializálás az első hívásnál

A dinamikus inicializálás a fordítási egységek között nincs sorrendbe állítva; ez a következő szakasz problémája.

A C++20 `constinit` kulcsszó pontosan ezt a kötelezettséget fejezi ki: a jelölt változónak statikus (konstans vagy zero) inicializálást kell kapnia, különben a fordító diagnosztikát ad. Fontos, hogy a `constinit` **nem** tesz `const`-tá; a változó futásidőben írható marad. Ez teszi alkalmassá olyan globális állapotra, amelyet `main` előtt is el kell érni, de később módosul, például egy `std::atomic<bool>` indulásjelzőre:

```
#include <atomic>
constinit std::atomic<int> g_initialized{0};  // konstans inicializált,
 ↪  futásidőben írható
inline int g_schema_version = 3;              // C++17 inline változó, egyetlen
 ↪  példány
constexpr int kMaxConnections = 64;           // fordítási idejű konstans
```

Az `inline` változó (C++17) a headerben definiálható, egyetlen program-szintű példánnyal; ez kiváltja az `extern` + `.cpp`-definíció mintát, de **nem** adja meg az inicializálási sorrendet, és nem tesz `const`-tá. A `constexpr` változó implicit módon `const`, így futásidőben nem írható.

20.5.2 Fordítási egységek közti inicializálási sorrend

A sorrendgarancia a változó fajtájától függ ([basic.start.dynamic]):

- **Rendezett (ordered)** inicializálású a nem `inline`, nem sablonból példányosított namespace-hatókörű statikus változó. Ezek egy fordítási egységen belül a definíciók sorrendjében, felülről lefelé inicializálódnak.
- **Részben rendezett** a nem sablonból példányosított `inline` változó (C++17): egy `W` változó előtt akkor inicializálódik garantáltan, ha `W` nem rendezetlen, és `W` *minden* definícióját megelőzi az `inline` változó valamelyik definíciója (például mert a header minden érintett fordítási egységben `W` előtt szerepel).
- **Rendezetlen** egy osztálysablon implicit vagy explicit példányosított statikus adattagja és a változósablon specializációja: ezekre semmilyen automatikus sorrend nem érvényes, ugyanazon a fordítási egységen belül sem.

Két különböző `.cpp` fájl között a szabvány a rendezett változókra sem állít fel sorrendet: a sorrend attól függ, milyen sorrendben kerülnek a fordítási egységek a linkelésbe, és hogy a fordító milyen inicializálási rutint generál. Ez a híres **static initialization order problem** (statikus inicializálási sorrend-probléma).

Az alábbi **negatív labor** szándékosan hibás: a viselkedése a linkelési sorrendtől függ. Fájlleltár: `config.cpp`, `bootstrap.cpp`, `main.cpp`.

```
// config.cpp
#include <string>
std::string make_name() { return "diagnostics"; }  // nem constexpr
std::string g_service_name = make_name();           // dinamikus inicializálás
```

```
// bootstrap.cpp
#include <iostream>
#include <string>
extern std::string g_service_name;

struct Banner {
```

```
    Banner() { std::cout << "starting: " << g_service_name << '\n'; }
};
Banner g_banner;  // sorrendje a g_service_name-hez képest nem meghatározott

// main.cpp
int main() {}  // a kimenetet a g_banner konstruktora adja, main előtt
```

Fordítás és futtatás mindkét objektumfájl-sorrendben:

```
g++ -std=c++20 -Wall -Wextra -O2 config.cpp bootstrap.cpp main.cpp -o demo-ab
g++ -std=c++20 -Wall -Wextra -O2 bootstrap.cpp config.cpp main.cpp -o demo-ba
./demo-ab
./demo-ba
```

Buildjegyzőkönyv (`lab-gcc` profil). A `demo-ab` kimenete `starting: diagnostics`, a `demo-ba` kimenete `starting:` (üres név), mindkettő 0 kóddal lép ki; ASan/UBSan builddel a hiba nem jelentkezik diagnosztikaként. A `make_name()` szándékos: ha a változót közvetlenül a `"diagnostics"` literállal inicializálnánk, C++20-ban a `std::string` `constexpr` konstruktora miatt az inicializálás konstans inicializálás lehet (a `lab-gcc` profilban az is lett, mindkét sorrend helyesen futott), és a példa nem mutatná a hibát. C++17 módban ugyanez a literálos változat a `demo-ba` sorrendben üres nevet írt ki.

Két külön kérdést kell szétválasztani. (1) **A sorrend.** Hogy a két fordítási egység közül melyik dinamikus inicializálása fut előbb, azt a szabvány nem határozza meg; ez önmagában nem hiba. (2) **A hozzáférés.** Ha a `g_banner` konstruktora fut előbb, akkor egy olyan `std::string` objektumot használ, amelynek az élettartama még nem kezdődött el — ez nem definiált viselkedés. A fenti „üres név” tehát nem a program „másik helyes kimenete”, hanem egy UB-s futás egyetlen megfigyelt eredménye; a nem definiált viselkedésre semmilyen kimenetlista nem adható. **Ez a hibakategória nem tesztelhető megbízhatóan**: ugyanaz a forrás a linkelési sorrendtől vagy a fordító verziójától függően másképp viselkedhet.

Ugyanabban a fordítási egységben a rendezett változók közti függőség helyes, ha a függő változó *később* van definiálva:

```
// same_tu_order.cpp - C++20: ugyanabban a fordítási egységben rendezett
 ↪ függőség
#include <iostream>
#include <string>

std::string make_name() { return "diagnostics"; }      // nem constexpr

std::string g_name = make_name();                      // (1) ordered, előbb
 ↪ definiálva
std::string g_banner = "starting: " + g_name;          // (2) később: g_name már
 ↪ kész

int main() { std::cout << g_banner << '\n'; }
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 same_tu_order.cpp` kimenete `starting: diagnostics`. Ha viszont a két definíció sorrendjét felcseréljük, a `g_banner` inicializálója egy még el nem kezdődött élettartamú `g_name`-et olvas: ez egyetlen fordítási egységen belül is ugyanaz a túl korai hozzáférés, tehát UB, nem „nem specifikált sorrend”.

A megoldás nem az, hogy „megpróbáljuk jól sorrendezni a `.cpp` fájlokat”, mert a sorrend törékeny és dokumentálatlan. A megoldás a lusta, első használatkor történő inicializálás.

20.5.3 Function-local static és szálbiztos inicializálás

Definíció. A *function-local static* (függvény-hatókörű statikus változó) statikus tárolási idejű objektum, amelynek dinamikus inicializálása akkor történik meg, amikor a vezérlés először halad át a deklarációján.

```
// config.hpp
#pragma once
#include <string>
struct Config { std::string service_name; };
const Config& config();

// config.cpp
#include "config.hpp"
const Config& config() {
    static const Config c{"diagnostics"};
    return c;
}

// bootstrap.cpp
#include "config.hpp"
#include <iostream>
struct Banner {
    Banner() { std::cout << "starting: " << config().service_name << '\n'; }
};
Banner g_banner;
```

A garanciák, amelyekre a minta épül: (1) az inicializálás pontosan egyszer **fejeződik be sikeresen**; ha az inicializáló kivétellel lép ki, az objektum nem számít inicializáltnak, és a vezérlés következő áthaladása újra megkísérli; (2) a C++11 óta a konkurens belépés is biztonságos — ha két szál egyszerre ér el a deklarációhoz, az egyik inicializál, a másik megvárja a befejezést („magic statics”, guaranteed initialization); (3) az inicializálás befejeződése után az objektum élettartama a folyamat végéig tart; (4) a destruálás az inicializálás befejezésének fordított sorrendjében történik.

Az (1) pontot érdemes kipróbálni, mert a „pontosan egyszer” könnyen „pontosan egy kísérletnek” olvasható. A `Service` első konstrukciója dob, a második sikerül; a már felépült `Buffer` tag a dobáskor felszabadul, így a részleges erőforrásszerzés visszagörgetődik:

```
// retry_static.cpp - C++20: kivétellel megszakadt function-local static
 ↪  újrapróbálása
#include <cstdio>
#include <memory>
#include <stdexcept>

int g_attempts = 0;

struct Buffer {                          // részleges erőforrás: RAII-tag
    std::unique_ptr<char[]> data{new char[64]};
    Buffer()  { std::puts("Buffer acquired"); }
    ~Buffer() { std::puts("Buffer released"); }
```

```
};

struct Service {
    Buffer buf;                         // előbb ez épül fel
    Service() {
        ++g_attempts;
        std::printf("Service() attempt %d\n", g_attempts);
        if (g_attempts == 1) throw std::runtime_error("backend not ready");
    }
    ~Service() { std::puts("~Service()"); }
};

Service& service() {
    static Service s;                   // dobás esetén nem számít inicializáltnak
    return s;
}

int main() {
    try { service(); }
    catch (const std::exception& e) { std::printf("caught: %s\n", e.what()); }
    service();                          // új kísérlet, most sikerül
    service();                          // már inicializált: nem fut újra
    std::printf("attempts = %d\n", g_attempts);
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 retry_static.cpp` kimenete: `Buffer acquired`, `Service() attempt 1`, `Buffer released`, `caught: backend not ready`, `Buffer acquired`, `Service() attempt 2`, `attempts = 2`, majd a program végén `~Service()`, `Buffer released`. Az inicializáló tehát kétszer futott, a `Service` egyszer jött létre és egyszer destruálódott. Ha a konstruktor törzse nem adattagban szerez erőforrást, a visszagörgetés a konstruktor felelőssége (lásd Destruktor és részleges konstrukció (20.3)).

A javított változat fájlleltára: `config.hpp`, `config.cpp`, `bootstrap.cpp` és a fenti `main.cpp`.

```
g++ -std=c++20 -Wall -Wextra -O2 config.cpp bootstrap.cpp main.cpp -o demo-ab
g++ -std=c++20 -Wall -Wextra -O2 bootstrap.cpp config.cpp main.cpp -o demo-ba
./demo-ab
./demo-ba
```

Itt a kimenet determinisztikusan `starting: diagnostics`, tekintet nélkül a linkelési sorrendre. **Buildjegyzőkönyv** (`lab-gcc` profil): mindkét sorrend ezt a kimenetet adta.

Két fontos korlátot kell ismerni. Egyrészt **rekurzív inicializálás**: ha a `config()` konstruktorából közvetlenül vagy közvetetten újra belépünk a `config()`-ba, miközben az inicializálás még nem fejeződött be, a viselkedés nem definiált. Másrészt a function-local static **nem oldja meg automatikusan a destruálási sorrendet** — csak az inicializálását. A költség külön kérdés. A nyelvi szabály csak azt mondja ki, hogy az inicializálás egyszer fejeződik be sikeresen; a megvalósítás ABI-függő. Az Itanium C++ ABI-ban (x86-64 Linux) a statikus objektumhoz egyetlen, **minden szál által közös** guard változó tartozik, nem szálankénti. Az első hívás a `__cxa_guard_acquire`/`__cxa_guard_release` párral, zárolva inicializál; a további hívások gyors útja a guard egy bájtjának betöltése és egy elágazás. **Mért eredmény** (`lab-gcc`, `g++ -O2 -S`, egy `static const std::string` accessor): a függvény eleje `movzbl` a guard változóra, `testb`, `je` a lassú ágra; a lassú ágban `call __cxa_guard_acquire@PLT` és `call __cxa_guard_release@PLT`;

a guard egy 8 bájtos `.local` szimbólum. Ez a gyors út forró ciklusban is jellemzően kicsi költség, de nem nulla, és fordító- meg optimalizálásfüggő; ha mérhető, a referenciát egyszer, a ciklus előtt érdemes kiolvasni.

20.5.4 Destrukciós sorrend és leálláskori függőségek

A statikus tárolási idejű objektumok destruálása a folyamat rendes leállásakor történik, az inicializálás befejezésének **fordított** sorrendjében. A `main` visszatérése után, illetve `std::exit` hívásakor. Ez a szabály a fordítottja az inicializálásának, de ugyanazt a sebezhetőséget termeli ki a másik irányban: a **static destruction order fiasco**.

Tipikus eset: egy globális `Logger` objektum és egy globális `Metrics` objektum, ahol a `Metrics` destruktora még naplózni szeretne. Ha a `Logger` destruálódik előbb, a `Metrics::~Metrics()` egy élettartama végét ért objektum tagfüggvényét hívja — nem definiált viselkedés, gyakorlatban összeomlás vagy elveszett naplóbejegyzés. A jelenség különösen zavaró, mert a hiba csak a leállás utolsó pillanataiban jelentkezik, ahol a diagnosztika már nem működik.

Két bevett javítás:

1. **Hozzáférési függvény function-local static-kal.** A `Logger`-t nem globális objektumként, hanem `Logger& logger() { static Logger l; return l; }` formában érjük el. A destruálás így az inicializálás *befejezésének* sorrendjéhez kötődik. Ha a `Metrics` konstruktora maga is hívja a `logger()`-t, akkor a `Logger` inicializálása a `Metrics` konstruktorán belül fejeződik be, tehát **előbb**, mint a `Metrics` konstrukciója; a fordított sorrend miatt a `Metrics` destruálódik előbb, és a destruktora még élő naplózót lát. A veszélyes eset az, amikor egy statikus objektum konstrukciója a naplózó első használata *előtt* fejeződik be (a konstruktora nem hívja a `logger()`-t), a destruktora viszont naplózna: ekkor a `Logger` később lett kész, ezért előbb szűnik meg.

```
// shutdown_order.cpp - C++20: ki használja a loggert a konstruktorában?
#include <cstdio>

struct Logger {
    Logger()  { std::puts("L kész"); }
    ~Logger() { std::puts("L dtor"); }
    void log(const char* m) { std::printf("  log: %s\n", m); }
};
Logger& logger() { static Logger l; return l; }

struct Metrics {                       // biztonságos: a konstruktor már használja
    Metrics()  { logger().log("Metrics indul"); std::puts("M kész"); }
    ~Metrics() { logger().log("Metrics leáll"); std::puts("M dtor"); }
};

struct Late {                          // veszélyes: a konstruktor NEM használja
    Late()  { std::puts("X kész"); }
    ~Late() { std::puts("X dtor (itt loggert hívna: már destruált lenne)"); }
};

Late    g_late;                        // X előbb kész, mint L
Metrics g_metrics;                     // L kész a Metrics konstruktorán belül

int main() { logger().log("main"); }
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 shutdown_order.cpp` kimenete: `X kész`, `L kész`, `log: Metrics indul`, `M kész`, `log: main`, `log: Metrics leáll`, `M dtor`, `L dtor`, `X dtor ...`. A biztonságos idővonal: L kész → M kész → M dtor → L dtor. A veszélyes idővonal: X kész → L kész → L dtor → X dtor; ha az `X` destruktora naplózna, egy már destruált `Logger`-t hívna. A szabály tehát: **ha egy statikus objektum destruktora használ egy másikat, a konstruktorában is érintse meg** (az accessor hívásával), így a sorrend garantált. 2. **Szándékos „soha nem törlődő” példány.** `static Logger* l = new Logger(); return *l;` — a példány a folyamat végéig él, nem destruálódik. Ez szivárgásnak tűnik, de a folyamat leállásakor a `free` már úgyis megtörténne; a szándékos elhagyás dokumentált és gyakori minta olyan eszközöknél, amelyeket a leállási sorrendben bármikor hívhatnak. Az ára, hogy a `Logger` destruálási mellékhatásai (puffer kiírása, fájl lezárása) elmaradnak — ezt külön flush-mechanizmussal kell pótolni.

Statikus destruktorból `std::exit`-et hívni nem definiált viselkedés, mert az újra belép a leállási folyamatba; kivételt dobni belőle a `noexcept` destruktor miatt `std::terminate`-hez vezet. Ezekre a destruktor-szabályoknál (Destruktor és részleges konstrukció (20.3)) már kitérünk; itt csak a leállási sorrend szempontjából relevánsak.

20.5.5 `thread_local` élettartam

A `thread_local` kulcsszó **szál tárolási időt** (`thread storage duration`) ad: minden szálnak saját tárolója van a változóhoz, amely a szál futása alatt áll rendelkezésre. Az objektum élettartama ettől külön kérdés: csak akkor kezdődik el, amikor az adott szálon az inicializálás megtörténik (lásd lent), és a szál kilépésekor ér véget. Egy szál, amely a változót soha nem használja, nem feltétlenül hoz létre belőle objektumot, ahogy a fenti `lazy_static.cpp` is mutatja.

```
#include <iostream>
#include <thread>

thread_local int t_request_count = 0;

void handle() { ++t_request_count; }

int main() {
    handle();
    handle();
    std::thread worker([] {
        handle();
        std::cout << "worker: " << t_request_count << '\n';  // 1
    });
    worker.join();
    std::cout << "main: " << t_request_count << '\n';        // 2
}
```

```
g++ -std=c++20 -Wall -Wextra -O2 -pthread tls.cpp -o tls
./tls
```

Buildjegyzőkönyv (`lab-gcc` profil): a kimenet `worker: 1`, majd `main: 2` — a szálak egymástól független példányokat látnak. Ez a tulajdonság teszi a `thread_local`-t kiváló eszközzé szálankénti puffer, konverziós buffer vagy diagnosztikai számláló tárolására.

Az élettartam-szabályok: namespace-hatókörű `thread_local` esetén a dinamikus inicializálás az adott szálon az első használat előtt történik meg; blokk-hatókörű `thread_local` esetén az első áthaladáskor. A destruálás a leállás módjától függ ([basic.start.term]):

- **Rendes szálkilépés** (a szál kezdőfüggvénye visszatér): a szál `thread_local` objektumai az inicializálás befejezésének fordított sorrendjében destruálódnak.
- **A `main` visszatérése vagy `std::exit` hívása:** a hívó (fő)szál `thread_local` objektumai destruálódnak, és ez a szabvány szerint *minden* statikus tárolású objektum destruálása előtt történik meg (strongly happens before). Ez garantált sorrend, nem implementációfüggő.
- **Rendkívüli leállás** (`std::abort`, `std::quick_exit`, `std::_Exit`, `std::terminate`, kezeletlen szignál): sem a `thread_local`, sem a statikus objektumok destruktora nem fut (a `quick_exit` csak az `at_quick_exit` kezelőket hívja).

A még futó *más* szálak `thread_local` objektumaira ez nem ad garanciát: ha egy mellékszál a `main` visszatérésekor még fut, az már önmagában hibás leállás.

```
// tls_vs_static.cpp - C++20: a főszál thread_local-ja a statikusak előtt szűnik
 ↪  meg
#include <cstdio>
#include <cstdlib>

struct Noisy {
    const char* n;
    explicit Noisy(const char* s) : n(s) { std::printf("ctor %s\n", n); }
    ~Noisy() { std::printf("dtor %s\n", n); }
};

Noisy g_static{"static"};                          // namespace-szintű statikus
Noisy& tls() { thread_local Noisy t{"thread_local"}; return t; }

int main(int argc, char**) {
    tls();                                         // a statikus UTÁN jön létre
    if (argc > 1) std::exit(0);                    // std::exit: ugyanaz a sorrend
    return 0;
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 tls_vs_static.cpp` mindkét futása (`./tls_vs_static` a `return`-ös, `./tls_vs_static x` az `std::exit`-es úton) ugyanazt adta: `ctor static`, `ctor thread_local`, `dtor thread_local`, `dtor static` — bár a `thread_local` később jött létre, a statikus destruálása előtt szűnt meg, ahogy a szabály előírja.

Két gyakorlati csapda. Egyrészt a `thread_local` nagy objektum szálankénti példányosítása memóriarobbanást okozhat sok szál esetén; érdemes a méretet korlátozni, vagy szálonkénti arena allokátorral dolgozni. Másrészt a dinamikus `thread_local` (amikor a kezdeti érték nem konstans) drágább lehet, mint a konstans inicializálású változat, és `dlopen`-elt könyvtárakkal való interakciója platformfüggő; a laborban érdemes mérni, nem feltételezni.

20.5.6 Tipikus hibák

- **Sorrend feltételezése linkelési sorrend alapján.** A `.cpp` fájlok sorrendje a linker parancssorban nem szerződés; a build rendszer átalakítása csendben elrontja. Javítás: function-local static.

- **Globális objektum, amely más globális objektum konstruktorában már olvas.** Ha a másik objektum dinamikus inicializálású, és más fordítási egységben (vagy ugyanabban, de később) van definiálva, a hozzáférés megelőzheti az élettartama kezdetét, és ez nem definiált viselkedés. Konstans vagy zero-inicializált objektum olvasása ilyenkor is biztonságos. Javítás: `constinit`/`constexpr`, vagy lusta inicializálás.
- **`constinit` és `const` összekeverése.** A `constinit` csak az inicializálás módját rögzíti; a változó írható marad. Ha `const` kell, mindkettőt ki kell írni.
- **Naplózás statikus destruktorból.** A naplózó már destruálódhatott. Javítás: soha nem törlődő naplózó példány, vagy a naplózás tiltása a leállási fázisban egy `shutting_down` flaggel.
- **`thread_local` nagy példányok minden szálon.** Memóriafogyasztás; javasolt a méretkorlát és a szál-specifikus újrafelhasználás.
- **Rekurzív function-local static inicializálás.** A „Meyers singleton” konstruktorából visszahívni a saját accessorát nem definiált viselkedés.

20.5.7 Feladatok

1. feladat (definíció-azonosítás). Az alábbi deklarációk közül melyiknél garantált, hogy a dinamikus inicializálás nem fut, és miért? `constexpr int a = 4;` — `constinit std::atomic<int> b{0};` — `std::string c = "x";` — `inline std::vector<int> d{1,2,3};` — `const char* const e = "x";`

2. feladat (kódolás). Adott egy kétfájlos program, amelyben egy `Registry` globális objektum konstruktorában egy másik globális `std::string`-et olvas. Alakítsd át a programot úgy, hogy a viselkedés determinisztikus legyen, és a `Registry` konstruktora mindenképpen a helyes értéket lássa. Indokold, miért garantált az új sorrend.

3. feladat (diagnosztika). Egy szolgáltatás leállításkor néha nem írja ki az utolsó metrikákat. A naplózó `logger()` accessoron keresztül érhető el, a metrikagyűjtő viszont egy globális objektum, amelynek destruktora hívja a naplózót. Magyarázd el, milyen sorrendprobléma léphet fel, milyen eszközzel lehetne igazolni, és milyen két javítást javasolsz, milyen trade-offal.

20.5.8 Megoldások

1. megoldás. Hordozhatóan csak az `a`, `b` és `e` esetén garantált, hogy nincs dinamikus inicializálás: `a` konstans kifejezés, `b` a `constinit` miatt kötelezően konstans inicializált, `e` pedig konstans mutató egy literálra, ami konstans kifejezés. A `c` és a `d` besorolása az implementációtól függ. C++20-ban a `std::string` és a `std::vector` konstruktora `constexpr`, de egy konstans kifejezés nem hagyhat maga után dinamikus allokációt. A `d` három eleme heapet igényel, ezért dinamikus inicializálást kap. A `c` rövid literáljánál a kisstring-optimalizáció (SSO) miatt nincs allokáció, így az inicializálás konstans lehet; ezt a szabvány nem írja elő, és nem is tiltja. Ezért a `constinit std::string` **képességpróba**, nem negatív teszt: hogy a fordító elfogadja-e, az a könyvtár SSO-méretén és `constexpr` támogatásán múlik, és az elfogadás nem hiba. A `lab-gcc` profilban a `constinit std::string g_short = "x";` lefordul (a Clang 22 + libc++ is elfogadta), a hosszú, SSO-n túli literál pedig `does not have a constant initializer` hibát ad. Biztosan elutasított negatív példához futásidejű bemenet kell: `constinit int g_port = read_port();`, ahol a `read_port()` a `PORT` környezeti változót olvassa. Ezt minden megfelelő fordító diagnosztizálja; a `lab-gcc` profilban a hiba: `'constinit' variable 'g_port' does not have a constant initializer`, illetve `call to non-'constexpr' function 'int read_port()'`.

2. megoldás. A `Registry`-t és a konfigurációs sztringet egyaránt function-local static-ká alakítjuk: `const std::string& registry_name() { static const std::string n = load();`

`return n; }`, és a `Registry` konstruktora ezt hívja. Az első hívás inicializálja a sztringet (a `load()` lefut), a második hívás már a kész objektumot adja vissza; a C++11-es garancia miatt a konkurens belépés is biztonságos, és a `Registry` mindig a már inicializált sztringet látja. Az eredeti globális `Registry` objektumot is érdemes accessorrá alakítani, hogy a destruálási sorrend is a használathoz kötődjön. Ellenőrzési feltétel: a két `main`-hívás közötti kimenet determinisztikus, és `-fsanitize=address` mellett sem jelez a futás.

3. megoldás. A hiba forrása a statikus destruálási sorrend: ha a metrikagyűjtő konstruktora nem hívja a `logger()`-t, a `Logger` inicializálása csak a metrikagyűjtő konstrukciója *után* fejeződik be, ezért a fordított sorrendben előbb destruálódik; a metrikagyűjtő destruktora így már destruált naplózót hív, ami nem definiált viselkedés. (Ha a konstruktor is használja a `logger()`-t, a sorrend helyes.) Igazolás: `-fsanitize=address` a leállási fázisban „use-after-destruction” jellegű hibát jelezhet, illetve a naplóbejegyzés elmaradása reprodukálható, ha a linkelési sorrendet megváltoztatjuk. Két javítás: (a) a logger soha nem törlődő példányként (`static Logger* l = new Logger();`) éljen, dokumentált flush-mechanizmussal; (b) a metrikagyűjtő destruktora egy `shutting_down` flag alapján ne naplózzon, hanem a metrikákat `std::atexit`-tel regisztrált, később futó függvényben írja ki. Az (a) előnye az egyszerűség, hátránya a destruálási mellékhatások elmaradása; a (b) előnye a rendezett leállás, hátránya a bonyolultabb, könnyen elfelejthető regisztráció.

20.5.9 Összefoglaló

A tárolási idő és az élettartam két külön fogalom: a statikus tárolójú objektum is csak az inicializálásakor kezd élni. Az inicializálás döntési folyamat: konstans inicializálás, ha lehetséges, egyébként zero-inicializálás, majd szükség esetén dinamikus inicializálás. A statikus inicializálás garantáltan minden dinamikus inicializálás előtt megtörténik, ezért az ilyen objektumok globálisan biztonságosan olvashatók; a `constinit` és `constexpr` ezt a garanciát teszi kifejezhetővé. A dinamikus inicializálás sorrendje fordítási egységek között nem meghatározott, amit a function-local static lusta inicializálása old meg, a C++11 óta szálbiztosan. A destruálás az inicializálás befejezésének fordított sorrendjében történik, és ez a leállási fázisban újabb függőségi csapdákat okoz; a legrobusztusabb minta a hozzáférési függvény vagy a szándékosan soha nem törlődő példány. A `thread_local` szálankénti tárolót ad; az objektum az adott szálon az inicializáláskor kezd élni, és a szál kilépésekor szűnik meg; a fő szál `thread_local` objektumai a `main` visszatérésekor vagy `std::exit`-kor garantáltan a statikus objektumok előtt destruálódnak. Mindhárom terület közös tanulsága: az életciklust nem a véletlen linkelési sorrendre, hanem explicit szerződésre kell építeni.

20.6 Tároló újrafelhasználása és C++ objektumszabályok

A nyers tároló és a benne élő objektum a C++ objektummodell két külön fogalma. A gyakorlatban mégis gyakran mosódik össze a kettő: puffert foglalunk, `reinterpret_cast`-tal „objektumot csinálunk”, majd a tárolót újra felhasználjuk. A C++20 szabályai szerint azonban a byte-ok önmagukban nem alkotnak objektumot; az objektum élettartamának el kell kezdődnie, és a tároló újrafelhasználásakor a régi pointer, referencia vagy név csak meghatározott feltételek mellett érvényes tovább. Ez a rész ezeket a feltételeket tárgyalja. Fontos határ: a C `effective type` szabályait nem emeljük át a C++-ba; a C++ saját objektum-élettartam-szabályai érvényesek.

20.6.1 A tároló és az objektum szétválasztása

Tárolón (storage) itt egy byte-okból álló, címezhető területet értünk. Objektum a C++ memóriamodell értelmében egy típussal rendelkező entitás, amelynek élettartama van. A tároló lehet egy `std::byte` tömb, egy `operator new` által adott terület, vagy egy `malloc` eredménye. Az objektum viszont akkor jön létre, amikor a megfelelő típusú konstrukció lezajlik, vagy amikor egy implicit-lifetime típus esetében a szabvány által megnevezett művelet implicit módon elindítja az élettartamát.

A két fogalom szétválasztása azért kritikus, mert a tároló mérete és igazítása még nem garantálja, hogy ott használható objektum van. Ha például egy `std::string` méretű byte-pufferre `reinterpret_cast<std::string*>`-tal mutatunk, és meghívjuk a `size()` tagfüggvényt, az nem „kicsit hibás”, hanem undefined behavior (UB): nincs ott `std::string` objektum, amelynek állapota értelmezhető lenne. Ugyanígy a `malloc(sizeof(T))` eredménye önmagában még nem `T` objektum, ha a `T` nem implicit-lifetime típus.

Az implicit-lifetime típusok köre a szabványban pontosan definiált: ide tartoznak a skalár típusok, a megfelelő aggregátumok, valamint azok az osztálytípusok, amelyek legalább egy triviális „eligible” konstruktorral és triviális, nem törölt destruktorral rendelkeznek. A `std::string` nem ilyen, a `struct Point { int x; int y; };` viszont tipikusan igen. C++23-ban a `std::is_implicit_lifetime` trait segíthet, de a library-támogatást a konkrét fordítónál ellenőrizni kell.

20.6.2 Az élettartam kezdete és vége

Egy `T` típusú objektum élettartama akkor kezdődik, amikor a megfelelő méretű és igazítású tároló rendelkezésre áll, és az inicializálás — az üres, úgynevezett vacuous initialization is — befejeződött. Triviális alapértelmezett konstruktor esetén ez gyakorlatilag azt jelenti, hogy a tároló megszerzése után az élettartam elkezdődik; nem triviális konstruktor esetén a konstruktor lefutása után.

Az élettartam vége C++20 szerint (C++20 [basic.life]/1) a típus fajtájától függ, nem a destruktor trivialitásától:

- **osztálytípusnál** az élettartam véget ér, amikor a destruktorhívás *megkezdődik* — akkor is, ha a destruktor triviális —, vagy amikor a tárolót felszabadítják vagy egy másik objektum számára újra felhasználják;
- **nem osztálytípusnál** (például `int`) az élettartam véget ér, amikor az objektumot megsemmisítik (C++20-ban a pszeudo-destruktorhívás, így a `std::destroy_at(int*)` is ilyen), illetve a tároló felszabadításakor vagy újrafelhasználásakor.

A triviális destruktor tehát *költség* szempontjából semmi (nem generál kódot), *szemantikailag* viszont ugyanúgy lezárja az élettartamot. A `std::destroy_at(p)` után két dolog marad külön: (1) a **tároló és a pointer** korlátozottan tovább használható — a `p` értéke `void*`-ként átadható, a tároló újra felhasználható egy új objektum számára, és ki lehet rá szabadítani; (2) a régi **objektum tagjai** viszont már nem: a `p->x` olvasása, nem statikus tagfüggvény hívása vagy a `p` leszármazott→bázis konverziója nem definiált viselkedés ([basic.life]/6). A tároló újrafelhasználása destruktorhívás nélkül triviális destruktornál szabályos; nem triviális destruktorú típusnál a destruktor elmaradása önmagában nem UB, de a C++20 szövege szerint az a program, amely a destruktor mellékhatásaira (felszabadítás, flush) épít, nem definiált viselkedésű.

Ha az új objektum pontosan ugyanabba a tárolóba kerül, mint a régi, felmerül a „transparently replaceable” (átlátszóan helyettesíthető) kérdése. A C++20 szabvány (C++20 [basic.life]/8) szerint a régi pointer, referencia vagy név automatikusan az új objektumra utal, ha az eredeti `o1` objektumot az új `o2` **átlátszóan helyettesíti**, azaz:

1. az `o2` tárolója pontosan lefedi az `o1` tárolóját;
2. az `o1` és az `o2` típusa a legfelső szintű cv-minősítéstől eltekintve azonos;
3. az `o1` nem teljes `const` objektum;
4. sem az `o1`, sem az `o2` nem *potenciálisan átfedő részobjektum* (bázisosztály-részobjektum vagy `[[no_unique_address]]` tag);
5. vagy mindkettő teljes objektum, vagy rendre a `p1` és `p2` objektumok közvetlen részobjektumai, és a `p1`-et a `p2` átlátszóan helyettesíti.

A `const` vagy referencia típusú nem statikus adattag **nem** zárja ki a helyettesítést. A C++17 szövege még kizárta, de ezt a C++20 előtt elfogadott javítás megszüntette, és a C++20 szövegben már nem szerepel. Ha a feltételek nem teljesülnek, a régi pointerrel való hozzáférés UB; ilyenkor jön szóba a `std::launder`.

20.6.3 Placement new, construct_at és destroy_at

A placement new egy létező tárolóban hívja meg a konstruktort:

```
T* p = ::new (static_cast<void*>(storage)) T(args...);
```

Ehhez a `<new>` fejléc kell. A placement new nem foglal memóriát; a tárolóért és annak élettartamáért a hívó felel. Nem triviális destruktor esetén a destruktort explicit módon kell meghívni, például `p->~T();` vagy C++17-től `std::destroy_at(p);` formában.

C++20-tól a `std::construct_at` és a `std::destroy_at` kényelmesebb, constexpr-képesebb alternatívát ad. A `std::construct_at` a megadott `T*` helyen konstruál, és visszaadja az új objektum pointerét; a `std::destroy_at` meghívja a destruktort. A következő példa egy `std::string` objektumot kétszer hoz létre ugyanabban a puffban:

```
// storage_reuse.cpp
#include <cstddef>
#include <iostream>
#include <memory>
#include <string>

int main() {
    alignas(std::string) std::byte storage[sizeof(std::string)];

    std::string* s = std::construct_at(
        reinterpret_cast<std::string*>(storage), "első");
    std::cout << *s << '\n';
    std::destroy_at(s);

    s = std::construct_at(
        reinterpret_cast<std::string*>(storage), "második");
    std::cout << *s << '\n';
    std::destroy_at(s);
}
```

Fordítási példa:

```
g++ -std=c++20 -Wall -Wextra -pedantic \
    -fsanitize=address,undefined storage_reuse.cpp -o storage_reuse
```

Buildjegyzőkönyv (`lab-gcc` profil). A fenti ASan/UBSan build kimenete `első`, majd `második`, diagnosztika nélkül, 0 kilépési kóddal.

A példa a második konstrukció után a `construct_at` visszatérési értékét menti el az `s`-be. Ez **egyértelmű és általánosan használható gyakorlat**, nem ebben a példában fennálló kényszer: a `construct_at` eredménye mindig az új objektumra mutat, ezért `std::launder` nélkül is biztonságos, bármilyen típust és tárolót használunk. Ebben a konkrét esetben azonban a régi `s` sem lenne hibás: az új `std::string` ugyanazt a tárolót pontosan lefedi, azonos típusú, nem `const` teljes objektum, tehát a régit **átlátszóan helyettesíti** (lásd fent, [basic.life]/8), és a régi pointer automatikusan az új objektumra utal. A visszaadott pointer elmentése azért jobb szokás, mert akkor is helyes marad, ha a feltételek valamelyike nem teljesül — például más típust építünk a tárolóba, vagy a régi objektum `const` volt.

C++17-es fallbackként a placement new és az explicit destruktorhívás használható:

```
std::string* s = ::new (static_cast<void*>(storage)) std::string("első");
// ...
s->~std::string();
```

20.6.4 Alignment és storage size előfeltételek

Egy `T` objektum elhelyezéséhez a tárolónak legalább `sizeof(T)` byte méretűnek és legalább `alignof(T)` igazításúnak kell lennie. Ha ezek bármelyike nem teljesül, a placement new és az azt követő használat UB. A gyakori hiba:

```
char buf[sizeof(T)];          // lehet, hogy nincs T-hez igazítva
T* p = ::new (buf) T{};       // UB lehet
```

A helyes alak például:

```
alignas(T) std::byte storage[sizeof(T)];
```

A `std::aligned_storage` C++23-ban deprecated; helyette az `alignas(T)` és a `std::byte` tömb a javasolt egyszerű minta. Több objektum esetén a méret `N * sizeof(T)`, az igazítás továbbra is `alignof(T)`; a tömb elemeit azonban külön kell konstruálni és destruálni.

Fontos az over-aligned típusok kezelése. A hagyományos `operator new` csak a fundamentális igazításra garantál megfelelő tárolót; ha `alignof(T)` ennél nagyobb, C++17-től az aligned `operator new`, vagy platformon `std::aligned_alloc` használható. A `malloc` eredménye szintén nem garantált over-aligned típusokhoz.

20.6.5 Trivially copyable, byte copy és objektumreprezentáció

Egy objektum objektumreprezentációja `sizeof(T)` byte-ból áll, amely tartalmazhat paddinget is. Ha `T` trivially copyable, a byte-ok másolása megőrzi az értéket, feltéve, hogy a célobjektum létezik, és a másolás a megfelelő módon történik. Erre való a `std::memcpy` vagy C++20-tól a `std::bit_cast`; utóbbi azonos méretű, trivially copyable típusok között használható.

```
#include <cstdint>
#include <cstring>
#include <type_traits>

struct Header {
    std::uint32_t magic;
```

```
    std::uint16_t version;
};

static_assert(std::is_trivially_copyable_v<Header>);

Header h{0x4d5a, 1};
std::byte bytes[sizeof(Header)];
std::memcpy(bytes, &h, sizeof h);

Header h2;
std::memcpy(&h2, bytes, sizeof h2);
```

Ez a minta akkor helyes, ha a `Header` valóban trivially copyable, a `h2` objektum már létezik, és a byte-sorrendet a protokoll külön kezeli. A `std::string` nem trivially copyable; a `memcpy` vele UB, mert a belső pointer és allokáció kétszeresen lenne tulajdonolva.

C-oldali kontrasztként a `malloc` és `memcpy` utáni `effective type` szabály C-ben más:

```
/* c_effective_type.c */
#include <stdlib.h>
#include <string.h>

int main(void) {
    void *p = malloc(sizeof(int));
    if (!p) return 1;
    int x = 42;
    memcpy(p, &x, sizeof x); /* C: itt az effective type int lesz */
    int y = *(int *)p;
    free(p);
    return y == 42 ? 0 : 1;
}
```

```
gcc -std=c17 -Wall -Wextra -pedantic c_effective_type.c -o c_effective_type
```

C++-ban ez a magyarázat önmagában nem elég. C++20-ban (P0593, a korábbi szabványokra is visszamenőleges hibajavításként) bizonyos műveletek **implicit módon létrehozhatnak** implicit-lifetime típusú objektumokat a tárolóban, ha ez a programnak definiált viselkedést ad: a `malloc`/`calloc`/`realloc`, az `operator new`/`operator new[]`, a `std::allocator<T>::allocate`, egy `unsigned char` vagy `std::byte` tömb élettartamának kezdete, valamint a `memcpy` és a `memmove` a céltárolóban (C++20 [intro.object]/10–13). A fenti C-s minta (`malloc`, majd `memcpy` egy `int`-ből) ezért C++20-ban is jól definiált, ha az `int` olvasása előtt a tároló egy `int` objektumot tartalmaz — ezt az implicit létrehozás biztosítja. A nem implicit-lifetime típusokhoz (például `std::string`) viszont továbbra is explicit konstrukció kell. A C `effective type` szabályait tehát nem szabad a C++ objektummodell helyettesítőjeként használni.

20.6.6 A std::launder szerepe

A `std::launder` egy szűk, speciális eszköz. Preconditionja nagyjából a következő: a pointer egy olyan címre mutat, ahol egy `T` típusú objektum él, az objektum élettartama tart, és az eredményen keresztül elérhető minden byte a bemeneti pointeren keresztül is elérhető. A `std::launder` nem hoz létre objektumot, nem igazít tárolót, és nem aktivál union tagot; csak egy létező objektumra ad pointert olyan esetben, amikor a régi pointer nem feltétlenül arra mutat.

Először egy ellenpélda: a `const` tagot tartalmazó típus C++20-ban átlátszóan helyettesíthető, ezért itt **nincs** szükség `std::launder`-re.

```
// transparent_const_member.cpp - C++20: a const tag nem akadálya
#include <iostream>
#include <new>

struct X {
    const int n;
};

int main() {
    alignas(X) unsigned char storage[sizeof(X)];

    X* p = ::new (static_cast<void*>(storage)) X{1};
    std::cout << p->n << '\n';

    ::new (static_cast<void*>(storage)) X{2};   // a régi X élettartama véget ér
    std::cout << p->n << '\n';                  // szabályos: p az új X-re utal
}
```

Mind az öt feltétel teljesül: az új `X` pontosan ugyanazt a tárolót foglalja el; a típusok azonosak; a régi `X` nem teljes `const` objektum (csak a tagja `const`); egyik sem részobjektum, mindkettő teljes objektum. A második `p->n` ezért szabályosan 2.

Valóban `std::launder`-t igényel viszont, ha a régi objektum **teljes `const` objektum** volt. Dinamikus tárolásnál a helyére szabad új objektumot létrehozni; statikus, szál vagy automatikus tárolású `const` objektumnál ez maga is UB lenne ([basic.life]/10).

```
// launder_demo.cpp - C++20: teljes const objektum helyére épített új objektum
#include <iostream>
#include <new>

struct X {
    int n;
};

int main() {
    const X* p = new const X{1};   // o1: teljes const objektum, dinamikus
     ↪  tárolás
    std::cout << p->n << '\n';

    ::new (const_cast<X*>(p)) X{2};   // o2 ugyanott; o1 élettartama véget ér

    // int bad = p->n;   // UB: o1 teljes const objektum volt
    std::cout << std::launder(p)->n << '\n';    // az o2-re mutat
    delete std::launder(p);
}
```

A levezetés a [basic.life]/8 feltételeivel: (1) az új `X` pontosan lefedi a régit; (2) a típusok a legfelső szintű `const`-tól eltekintve azonosak; (3) **nem teljesül**: az `o1` teljes `const` objektum volt, ezért a régi `p` nem utal automatikusan az új objektumra, és a `p->n` UB lenne; (4) és (5) teljesül. A `std::launder(p)` előfeltételei ([ptr.launder]) teljesülnek: a `p` címén egy élettartamában lévő `X`

objektum van (az `o2`), és az eredményen keresztül csak olyan bájtok érhetők el, amelyek a `p`-n keresztül is. A `launder` tehát az új objektumra ad pointert; a `delete` is ezen keresztül történik. Ugyanígy `launder` kellene akkor is, ha egy bázisosztály-részobjektum helyére hoznánk létre új objektumot (4. feltétel).

Buildjegyzőkönyv (`lab-gcc` profil). Mindkét program `g++ -std=c++20 -Wall -Wextra -Wpedantic` beállítással figyelmeztetés nélkül fordult, és `1`, `2` kimenettel futott, ASan/UBSan builddel is.

20.6.7 Union aktív tag és típuselérhetőség

A unionban egyszerre legfeljebb egy tag aktív. Az aktív tag élettartama akkor kezdődik, amikor azzá válik, és akkor ér véget, amikor egy másik tag lesz aktív, vagy a union objektum élettartama véget ér. Nem triviális tag esetén a tagot konstruálni és destruálni kell; a puszta byte-csere nem elég. Az inaktív tag olvasása UB, kivéve a standard-layout unionok közös kezdősorozatára (common initial sequence) vonatkozó szűk kivételt.

```
union U {
    int i;
    float f;
};

void f() {
    U u;
    u.i = 42;         // i aktív
    // float x = u.f; // UB: f nem aktív
    u.f = 3.14f;      // most f aktív; i élettartama véget ért
}
```

A `std::launder` nem teszi aktívvá az inaktív union tagot; erre a `std::variant` vagy egy explicit aktív-tag állapotgép a helyes megoldás.

A típuselérhetőség (type accessibility) ettől külön kérdés. C++-ban egy objektum tárolt értékét csak a szabvány által engedett típusú glvalue-n keresztül lehet elérni: az objektum dinamikus típusa, annak cv-változata, hasonló típus, a megfelelő signed/unsigned pár, az objektumot tartalmazó aggregate/union, bázisosztály, illetve `char`, `unsigned char` vagy `std::byte`. A `char`/`std::byte` nézet mindig megengedett byte-szintű vizsgálatra, de ettől még nem lesz a pufferben `T` objektum.

20.6.8 Tipikus hibák

- A `reinterpret_cast<T*>(buf)` utáni azonnali dereferálás: még nincs `T` objektum.
- `char buf[sizeof(T)]` használata `alignas(T)` nélkül: az igazítás nem garantált.
- Nem triviális destruktorú típusnál a `destroy_at` elmaradása a tároló újrafelhasználása előtt.
- A régi pointer használata placement new után, amikor a transparently replaceable feltétel nem teljesül (például teljes `const` objektum vagy bázisosztály-részobjektum helyén), `std::launder` nélkül.
- A `std::launder` univerzális javításként való alkalmazása olyan helyen, ahol nincs életben lévő objektum.
- `memcpy` használata `std::string` vagy más nem trivially copyable típus esetén.
- Az inaktív union tag olvasása, vagy a `std::launder` használata az aktív tag „kijelölésére”.
- A C `effective type` szabályainak átemelése a C++ objektum-élettartam helyére.

20.6.9 Feladatok

1. Az alábbi kódban mi a hiba? Javítsd minimálisan, és add meg a fordítási parancsot!

```
#include <cstddef>
#include <string>

int main() {
    std::byte buf[sizeof(std::string)];
    auto* p = reinterpret_cast<std::string*>(buf);
    p->assign("hello");
}
```

2. Írj egy rövid programot, amely egy `alignas(std::string) std::byte` puffert kétszer használ `std::string` tárolására! A második konstrukcióhoz a `std::construct_at` visszatérési értékét használd. Magyarázd el, miért nem kell `std::launder` a visszakapott pointeren!

3. Adott a `transparent_const_member.cpp` és a `launder_demo.cpp`. Magyarázd el a [basic.life]/8 feltételeivel, miért szabályos az elsőben a második `p->n`, és miért UB a másodikban a `p->n`, miközben a `std::launder(p)->n` definiált! Sorolj fel két olyan esetet, amikor a `std::launder` nem segít!

4. Az alábbi unionos kódban hol lehet UB? Javítsd `std::variant`-tal, vagy explicit aktív-tag követéssel!

```
union U {
    int i;
    float f;
};

int main() {
    U u;
    u.i = 42;
    float x = u.f;
}
```

20.6.9.1 Megoldások

1. A `buf` nincs `std::string`-hez igazítva, és nincs benne `std::string` objektum; a `p->assign` ezért UB. Helyes minimal fix:

```
#include <cstddef>
#include <memory>
#include <string>

int main() {
    alignas(std::string) std::byte buf[sizeof(std::string)];
    std::string* p = std::construct_at(
        reinterpret_cast<std::string*>(buf), "hello");
    std::destroy_at(p);
}
```

Fordítás: `g++ -std=c++20 -Wall -Wextra -pedantic -fsanitize=address,undefined storage_fix.cpp -o storage_fix`. Ellenőrzési feltétel: a `construct_at` visszatérési értékét használjuk, és a destruktor lefut.

2. Példa:

```
#include <cstddef>
#include <iostream>
#include <memory>
#include <string>

int main() {
    alignas(std::string) std::byte storage[sizeof(std::string)];

    std::string* first = std::construct_at(
        reinterpret_cast<std::string*>(storage), "első");
    std::cout << *first << '\n';
    std::destroy_at(first);

    std::string* second = std::construct_at(
        reinterpret_cast<std::string*>(storage), "második");
    std::cout << *second << '\n';
    std::destroy_at(second);
}
```

A `construct_at` visszatérési értéke már az új objektumra mutat, ezért nincs szükség `std::launder`-re. A `launder` akkor kellhet, ha egy korábbi pointert szeretnénk az új objektumra irányítani, és a transparently replaceable feltétel nem teljesül. Ellenőrzési feltétel: a második konstrukcióhoz nem a `first` pointert használjuk, és a destruktorok lefutnak.

3. A `transparent_const_member.cpp`-ben mind az öt feltétel teljesül: a `const` tag nem akadály, csak a teljes `const` objektum az; a régi `p` ezért az új `X`-re utal, és a `p->n` értéke 2. A `launder_demo.cpp`-ben a régi objektum teljes `const` objektum volt, így a 3. feltétel sérül; a régi `p` nem utal az új objektumra, a `p->n` UB. A `std::launder(p)` az új objektumra ad pointert, mert a címen élő `X` van, és a launder előfeltételei teljesülnek. A `std::launder` nem segít, ha nincs életben lévő `T` objektum a címen (például elmaradt a placement new), és nem segít az inaktív union tag olvasásán sem; ilyenkor nem lehet „launderrel” objektumot vagy aktív tagot varázsolni. Ellenőrzési feltétel: a válasz megkülönbözteti az élettartam létrehozását a pointer újrairányításától.

4. Az `u.i = 42` után az `i` aktív, az `f` olvasása UB. Egy lehetséges javítás `std::variant`-tal:

```
#include <variant>

int main() {
    std::variant<int, float> u;
    u = 42;
    float x = std::get<float>(u); // dob, mert int az aktív alternatíva
}
```

A `std::get<float>` itt kivételt dob, mert nem az `float` az aktív tag; ez már definiált viselkedés. Explicit aktív-tag követéssel egy `enum class Active { Int, Float };` és a megfelelő konstrukció/destrukció lenne a másik helyes irány. Ellenőrzési feltétel: a megoldás nem olvassa az inaktív union tagot, és kezeli a tagváltást.

20.6.10 Összefoglaló

A C++-ban a tároló és az objektum nem ugyanaz. Az objektum élettartama a megfelelő méretű és igazítású tároló megszerzésével és az inicializálás befejezésével kezdődik; nem triviális destruktor esetén a destruktorhívás zárja le. A placement new, a `std::construct_at` és a `std::destroy_at` teszi lehetővé a tároló újrafelhasználását, de csak a méret, az igazítás és az élettartam-szabályok betartásával. A `std::launder` szűk eszköz: létező objektumra ad pointert, nem hoz létre objektumot. A trivially copyable típusok byte-szinten másolhatók, a nem trivially copyable típusoknál konstrukció kell. A union aktív tagjának szabályai és a strict aliasing külön rétegek; a C `effective type` szabályai nem helyettesítik a C++ objektummodellt.

21 RAII, erőforráskezelés és smart pointerek

21.1 new/delete és az allokáció-konstrukció kapcsolat

A C-ben a dinamikus élettartam két külön döntés volt: a malloc, calloc, free és allokációs szerződés (8.1) szerinti allokációs szerződés (`malloc`/`free`) és a programozó fejében tartott konstrukciós szabály. A C++ ezt a kettőt egyetlen nyelvi kifejezésbe, a *new expression*be zárja, de nem olvasztja össze: az allokáció és az objektum létrehozása továbbra is két külön lépés, csak egy kifejezés hajtja végre mindkettőt a megfelelő sorrendben és a megfelelő hibakezeléssel. Ez a rész pontosan ezt a szétválasztást tárja fel. Azért érdemes vele komolyan foglalkozni, mert a C++ leggyakoribb erőforrás-hibái nem a `delete` elfelejtéséből, hanem a *nem megfelelő pár* megválasztásából és a *konstruktorhiba* félreértéséből fakadnak. A cél nem az, hogy jól írjunk kézi `new`/`delete`-et, hanem hogy pontosan értsük, mit vállal helyettünk a standard owner, amit a fejezet második felében használni fogunk.

21.1.1 A new expression anatómiája

21.1.1.1 Allokátorfüggvény, konstrukció, new expression

Három fogalmat kell élesen elkülöníteni, mert a köznyelv mindhármat „new”-nak hívja.

Az **allokátorfüggvény** (allocation function) egy közönséges függvény, amelynek deklarációja `void* operator new(std::size_t)`. A feladata kizárólag az, hogy nyers tárolót (storage) szerezzen, és visszaadjon egy pointert a tároló kezdetére. Nem hoz létre objektumot, nem futtat konstruktort, és nem tud semmit a majdani objektum típusáról – csak a kért méretről. A párja a **felszabadító függvény** (deallocation function), `void operator delete(void*) noexcept`, amely a tárolót adja vissza; az élettartam lezárása nem a dolga.

A **new expression** a szintaktikai alak: `new T`, `new T(args...)`, `new T[n]`. Ez az, ami a kettőt összeköti, és amit a fordító állít elő. A **konstrukció** pedig az az objektum-létrehozási lépés, amelyet a new expression végez el a megszerzett tárolón – pontosan úgy, ahogy a Tároló újrafelhasználása és C++ objektumszabályok (20.6) tároló-újrafelhasználási szabályai leírják.

A gyakorlati következmény: `operator new(sizeof(T))` önmagában **nem** hoz létre `T`-t. Nyers tárolót ad, benne `T` élettartama nem kezdődött el, a pointeren keresztüli `T`-ként való használat nem definiált viselkedés.

21.1.1.2 A new és a delete expression lépései

Egy `new T(args...)` kiértékelése a következő sorrendben történik:

1. A fordító kiszámítja a kért méretet (`sizeof(T)`), tömb esetén a darabszám figyelembevételével.
2. Kiválasztja és meghívja az illeszkedő allokátorfüggvényt. Over-aligned típus esetén ez az igazítást is átvevő alak lesz (lásd később).

3. Ha az allokátorfüggvény *nem dobó* alakú és `nullptr`-t adott vissza, a new expression eredménye `nullptr`, és **nem** történik konstrukció.
4. A megszerzett tárolón lefut a konstruktor.
5. Ha a konstruktor kivételt dob, a new expression meghívja az allokátorfüggvényhez tartozó felszabadító függvényt, majd továbbengedi a kivételt. A tároló tehát nem szivárog, akkor sem, ha a felhasználó `try` blokkja elfelejtkezik róla.

Egy `delete p` ennek a tükörképe: `nullptr` esetén nincs hatás; egyébként lefut a destruktor, majd a felszabadító függvény. Fontos korlát: a standard megengedi, hogy a fordító egy *replaceable globális* allokátorfüggvény hívását elhagyja (allocation elision), ha a tárolót máshonnan tudja biztosítani. Ezért „az `operator new` minden `new`-ra lefut” nem univerzális állítás; osztályhatókörű (`static`) `operator new` esetén viszont nincs ilyen elhagyás, ezért a diagnosztikai példákban azt használjuk.

21.1.2 Skalár és tömb alak, helyes párosítás

A `new T` és a `new T[n]` két külön allokátorfüggvényt hív (`operator new` és `operator new[]`), és két külön felszabadítót vár (`operator delete` és `operator delete[]`). A keverés nem definiált viselkedés – nem „csak memóriaszivárgás”. A tömb-alaknál a fordító a destruktorokat a konstrukció fordított sorrendjében futtatja, és az implementáció jogosult arra, hogy az elemek darabszámát a visszaadott pointer elé rejtett fejlécben tárolja. Az ilyen könyvelés nem specifikált, tehát sem a fejléc meglétére, sem a `new T[n]` és `new T` azonos címére nem építhetünk.

Tárolószerzés	Helyes felszabadítás	Destruktor fut?
`new T`	`delete p`	igen
`new T[n]`	`delete[] p`	igen, fordított sorrendben
`new (std::nothrow) T`	`delete p`	igen
`new (buf) T` (placement)	`p->~T()`	kézzel, a tároló nem szabadul fel
`std::malloc(n)`	`std::free(p)`	nem

Ha egy osztály saját `operator new`-t deklarál, de `operator new[]`-t nem, akkor az `new T[n]` forma fordítási hibát ad – ez szándékos védelem, nem hiányosság.

21.1.3 Allokációs hiba, nothrow és konstruktor-kivétel

A dobó `operator new` szerződése: vagy érvényes, megfelelően igazított tárolót ad vissza, vagy `std::bad_alloc`-ot dob. Nem adhat vissza `nullptr`-t; ha mégis megteszi, az szerződésszegés. Hibánál a standard által beállított *new-handler* lánc fut le, amíg valamelyik kezelő nem szabadít fel memóriát vagy nem dob.

A `new (std::nothrow) T` alak a `std::nothrow_t` paraméterű allokátorfüggvényt hívja, amely elkapja a `bad_alloc`-ot és `nullptr`-t ad. Ez a védelem **kizárólag az allokációra** vonatkozik. Ha az allokáció sikerül, de a konstruktor dob, akkor a new expression felszabadítja a tárolót, és a kivétel **továbbterjed** – a `nothrow` alak nem nyeli el. Ez a leggyakoribb félreértés ezen a téren.

21.1.3.1 Kidolgozott példa: allokáció és konstrukció számlálása

A példa osztályhatókörű allokátorral teszi láthatóvá a két lépést, és megmutatja, hogy a konstruktorhiba után a felszabadítás megtörténik.

```
// new_delete_demo.cpp - ISO C++20, GCC/Clang
#include <cstdio>
#include <cstdlib>
#include <new>
#include <stdexcept>

class Probe {
public:
    static int allokacio;
    static int deallokacio;
    static int konstrukcio;

    static void* operator new(std::size_t n) {
        ++allokacio;
        std::printf("  operator new(%zu)\n", n);
        if (void* p = std::malloc(n)) return p;
        throw std::bad_alloc();
    }

    static void operator delete(void* p) noexcept {
        ++deallokacio;
        std::printf("  operator delete\n");
        std::free(p);
    }

    explicit Probe(int id) : id_(id) {
        if (id < 0) throw std::runtime_error("negativ id");
        ++konstrukcio;
    }
    ~Probe() = default;

    int id() const noexcept { return id_; }

private:
    int id_;
};

int Probe::allokacio = 0;
int Probe::deallokacio = 0;
int Probe::konstrukcio = 0;

int main() {
    Probe* a = new Probe(1);
    std::printf("a->id() = %d\n", a->id());
    delete a;

    try {
```

```
        Probe* b = new Probe(-1);   // allokacio sikeres, konstrukcio dob
        (void)b;
    } catch (const std::runtime_error& e) {
        std::printf("kivetel: %s\n", e.what());
    }

    std::printf("allokacio=%d deallokacio=%d konstrukcio=%d\n",
                Probe::allokacio, Probe::deallokacio, Probe::konstrukcio);
}
```

Fordítás és futtatás a laborban:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 \
    new_delete_demo.cpp -o new_delete_demo
./new_delete_demo
```

A forrás logikája alapján a várt kimenet (a tanuló laborjában ellenőrizendő, mérés nem történt):

```
  operator new(4)
a->id() = 1
  operator delete
  operator new(4)
kivetel: negativ id
  operator delete
allokacio=2 deallokacio=2 konstrukcio=1
```

Az utolsó sor a lényeg: két allokáció, két felszabadítás, de csak egy konstrukció. A második `operator delete` nem a felhasználó `catch` blokkjából jön, hanem a new expression hibakezeléséből.

21.1.4 Igazítás tudatos allokáció C++17-től

A klasszikus `operator new(std::size_t)` csak addig garantál igazítást, amíg a típus igazítása nem nagyobb, mint a `__STDCPP_DEFAULT_NEW_ALIGNMENT__` makró értéke (implementáció által meghatározott, x86-64 Linuxon jellemzően 16). C++17 óta a standard explicit, igazítást is átvevő alakokat vezetett be: `operator new(std::size_t, std::align_val_t)` és a párja, `operator delete(void*, std::align_val_t)`, mindkettő a `<new>` fejlécben. Ha egy típus `alignof` értéke meghaladja a fenti korlátot (`struct alignas(64) CacheLine { std::byte b[64]; };`), a fordító **automatikusan** ezt az alakot választja, és a felszabadításnál is a megfelelő párját hívja. Nekünk akkor van dolgunk, ha kézzel hívunk allokátorfüggvényt, vagy ha `malloc`-alapú pufferbe helyezünk over-aligned objektumot: a `malloc` garantált igazítása a `max_align_t`-ig terjed, ami nem feltétlenül éri el a 64-et, így a konstrukció nem teljesülő igazítási feltétel mellett nem definiált. Ilyenkor `std::aligned_alloc` vagy – ami egyszerűbb – közvetlen `new CacheLine` a helyes választás.

21.1.5 A malloc/free nem végez C++ konstrukciót

A malloc, calloc, free és allokációs szerződés (8.1) allokációs szerződése változatlanul érvényes: a `std::malloc` nyers tárolót ad, a `std::free` pedig felszabadítja azt. Egyik sem futtat konstruktort vagy destruktort, és nem is tudna: nem ismeri a típust. A `malloc`-mal szerzett memória `delete`-tel

való felszabadítása és a `new`-val szerzett pointer `free`-elése egyaránt nem definiált viselkedés, még akkor is, ha egy adott implementáción látszólag működik.

A két világ hídja a **placement new**: `new (ptr) T(args...)`, ahol a zárójelben egy `void*` áll. Ez a forma nem hív allokátorfüggvényt, hanem a megadott tárolón konstruál. A párja nem `delete`, hanem az explicit destruktorhívás (`p->~T()`), mert a tároló tulajdonosa valaki más. C++17-től a könyvtárban van erre `std::launder`, C++20-tól pedig a `std::construct_at` és `std::destroy_at`, amelyek ezt a mintát biztonságosabban és `constexpr`-barát módon fejezik ki.

```
// placement_demo.cpp - ISO C++20
#include <cstddef>
#include <cstdio>
#include <new>

struct Header { int version; int length; };

int main() {
    alignas(Header) unsigned char buf[sizeof(Header)];  // nyers tároló
    Header* h = ::new (static_cast<void*>(buf)) Header{1, 42};
    std::printf("version=%d length=%d\n", h->version, h->length);
    h->~Header();   // élettartam vége; a buf tároló továbbra is él
}
```

Itt a `buf` a **tároló tulajdonosa**, a `h` csak **kölcsönzött hozzáférés** egy ideig létező objektumhoz; a `buf` élettartama nem ér véget a destruktorhíváskor.

21.1.6 Kézi kezelés helyett tulajdonos

A fenti gépezet megértése szükséges, de a napi kódban nem kézzel használjuk. A szabály egyszerű: ha egy `T*` érték a `new` expressionből származik, akkor az **owner**, és valakinek pontosan egyszer felelősséget kell vállalnia érte. A legrövidebb út, hogy ez a „valaki” egy objektum legyen, amelynek destruktora a megfelelő felszabadítást végzi – ez az RAII, amelyet a RAII wrapper file descriptorhoz és lockhoz (21.2) részletez nem memóriajellegű erőforrásokra.

- `auto p = std::make_unique<Probe>(1);` – egyedi tulajdonlás, `delete` a destruktorban.
- `auto a = std::make_unique<Probe[]>(n);` – `unique_ptr<Probe[]>` típus, `delete[]`-t hív. Figyelem: a `make_unique` a tömb elemeit érték-inicializálja, ami eltér a `new Probe[n]` default-inicializálásától.
- `std::vector<Probe> v;` – dinamikus tömbhöz ez az első választás; növekedésnél mozgat, nem `new[]`-t hív kézzel.

Egy függvényparaméter `Probe*` alakja viszont jellemzően **borrowed view**: nem tulajdonol, nem szabadít fel, és a hívó felelőssége, hogy az objektum a hívás idejére érvényes legyen. Ezt a szerződést a fejlécben és a nevekben is érdemes kimondani.

21.1.7 Tipikus hibák és javításuk

A leggyakoribb hiba a `new T[n]` és a `delete p` párosítása, illetve a `new T` és a `delete[] p` párosítása. Tünetük gyakran nem azonnali összeomlás, hanem késői memóriakorrupció, ezért a sanitizerek (az ASan, UBSan, MSan, TSan és Valgrind (16.2) laborja) nélkül nehéz tetten érni őket. Második gyakori hiba, hogy a szerző a konstruktor kivételét „letakarja” egy `try`/`catch`-csel, és a

`new` expression által már felszabadított tárolót kézzel próbálja `delete`-elni – ez dupla felszabadítás. Harmadik: a `nothrow` alakot úgy kezelni, mintha a konstruktorhibát is elnyelné; nem nyeli el. Negyedik: over-aligned típushoz kézzel `malloc`-olt puffert használni placement new-val, holott az igazítás nem teljesül.

21.1.8 Összefoglaló

A new expression három dolgot köt össze: allokátorfüggvényt hív, konstruál, és hiba esetén felszabadít. Az allokátorfüggvény és a felszabadító függvény önmagukban csak nyers tárolót kezelnek, objektumot nem. A skalár és tömb alak nem cserélhető fel, mert külön függvénypárt és eltérő destruktor-sorrendet jelent. A dobó allokáció `bad_alloc`-ot dob, a `nothrow` alak `nullptr`-t ad, de a konstruktor kivételét egyik sem nyeli el. C++17-től az over-aligned típusokhoz külön, `std::align_val_t`-t átvevő allokátorfüggvény tartozik. A `malloc`/`free` továbbra sem konstruál és nem destruál; a hidat a placement new és az explicit destruktorhívás adja. Éles kódban a kézi `new`/`delete` helyett `make_unique`, `make_unique<T[]>` vagy `std::vector` a választás, és minden `T*` esetében expliciten meg kell mondani, hogy owner-e vagy borrowed view.

21.1.9 Feladatok

1. feladat (magyarázó). Írja le lépésről lépésre, milyen műveletekből áll a `new Probe(5)` new expression, és pontosan melyik lépés mit tesz, ha a `Probe` konstruktora `std::runtime_error`-t dob. Hasonlítsa össze a `std::malloc(sizeof(Probe))` hívással: mely lépések maradnak el, és hol kezdődik a `Probe` élettartama az egyes esetekben?

2. feladat (kódolási). Adott `N` elemre dynamikusan kell `Sample` tömböt létrehozni (`struct Sample { double v[8]; unsigned seq; };`), feltölteni, majd felszabadítani. Írja meg három változatban: (a) nyers `new[]`/`delete[]`, (b) `std::unique_ptr<Sample[]>`, (c) `std::vector<Sample>`. Mindháromhoz adja meg a pontos felszabadítási pontot, és mondja meg, melyik változat mit garantál, ha a feltöltés közben kivétel keletkezik.

3. feladat (diagnosztikai). Az alábbi két részlet hibás vagy félrevezető. Jelölje meg a hibát, indokolja, és adja meg a javítást.

```
void f(int n) {
    Probe* p = new Probe[n];
    g(p);
    delete p;            // (a)
}

struct alignas(64) CacheLine { std::byte b[64]; };
void h() {
    void* raw = std::malloc(sizeof(CacheLine));
    CacheLine* c = new (raw) CacheLine{};   // (b)
}
```

21.1.10 Megoldások

1. megoldás. A `new Probe(5)` lépései: (i) a fordító kiszámítja a `sizeof(Probe)` méretet; (ii) kiválasztja a `Probe::operator new(std::size_t)` allokátorfüggvényt, vagy annak hiányában a globálisat, és meghívja; (iii) ha az allokátor dob, a new expression nem folytatódik, a kivétel továbbterjed, tároló nem keletkezett; (iv) siker esetén a megszerzett tárolón lefut a `Probe(int)`

konstruktor, és ezzel megkezdődik a `Probe` élettartama; (v) ha a konstruktor dob, a fordító meghívja a megfelelő felszabadító függvényt, majd továbbengedi a kivételt. A `malloc` esetén az (i) és (ii) lépés megvan, de (iv) elmarad: nyers tárolót kapunk, `Probe` élettartama nem kezdődik el, és a pointeren `Probe`-ként való hozzáférés nem definiált. Ellenőrzési feltétel: a válasz nevezze meg, hogy a konstruktorhiba esetén a felszabadítás a new expression része, nem a hívó feladata.

2. megoldás. (a) `Sample* p = new Sample[n];` – a felszabadítás `delete[] p;`. Ha a feltöltő ciklus kivételt dob, a `p` elveszik, memóriaszivárgás keletkezik, hacsak a hívó `try/catch`-csel nem felszabadít. (b) `auto p = std::make_unique<Sample[]>(n);` – a `unique_ptr<Sample[]>` destruktora `delete[]`-t hív, tehát kivétel esetén is felszabadít; a `Sample`-ek érték-inicializáltak lesznek. (c) `std::vector<Sample> v(n);` – szintén kivételbiztos, és a növekedést is kezeli. Mindhárom esetben a `Sample`-ek destruktora lefut, mert a `Sample` triviálisan destruálható, ez itt nem figyelhető meg; a lényeg a tároló felszabadítása. Ellenőrzési feltétel: a válasz mindhárom változatnál nevezze meg az owner objektumot és a felszabadítás pontos helyét.

3. megoldás. (a) A `new Probe[n]` allokátorfüggvénye `operator new[]`, a felszabadítója `operator delete[]`; a `delete p` skalár felszabadítót hív, ami nem definiált viselkedés, és a destruktorok sem feltétlenül futnak le minden elemen. Javítás: `delete[] p;` – vagy, ami jobb, `auto p = std::make_unique<Probe[]>(n);`, illetve `std::vector<Probe>`. (b) A `std::malloc` csak `max_align_t`-ig garantál igazítást, ami jellemzően 16, nem 64; a `CacheLine` igénye 64, ezért a placement new nem teljesülő igazítási feltétel mellett hozza létre az objektumot, ami nem definiált viselkedés. Javítás C++17-től: `CacheLine* c = new CacheLine{};` – a fordító automatikusan az `operator new(std::size_t, std::align_val_t)` alakot hívja –, vagy C API felé: `std::aligned_alloc(64, sizeof(CacheLine))` megfelelő felszabadítással. Ellenőrzési feltétel: a válasz mindkét pontban nevezze meg a konkrét szabálysértést (nem megfelelő felszabadító pár, illetve nem teljesülő igazítás), és a javítás is tartalmazza a felszabadítás helyes párját.

21.2 RAII wrapper file descriptorhoz és lockhoz

Az előző részben a C cleanup-szerződéseit néztük meg: minden megszerzett erőforráshoz pontosan egy felszabadító út tartozik, és ezt az utat `goto cleanup` címkék, korai `return`-ök és hibakódok között kézzel kell összehangolni. A C++ válasza erre nem egy újabb könyvtár, hanem a nyelv élettartam-szabálya: ha az erőforrást egy objektum birtokolja, akkor a felszabadítás az objektum destruktorában fut le, méghozzá determinisztikusan, akkor is, ha a vezérlés kivétel miatt hagyja el a hatókört. Ezt a mintát nevezzük RAII-nak (Resource Acquisition Is Initialization; erőforrás-életciklus kötése objektum-élettartamhoz). Ebben a részben ezt a szerződést konkretizáljuk két tipikus DevOps-erőforrásra: a file descriptorra és a lockra. Mindkettőnél külön nevezzük meg a tulajdonost (owner) és a megfigyelőt (observer), mert a legtöbb termelési hiba nem a destruktorban keletkezik, hanem ott, hogy egy nem tulajdonló API-t tulajdonosként kezelünk.

21.2.1 Az invariáns: birtoklás, sentinel, owner és observer

Egy RAII wrapper definíciója pontosítva: olyan osztály, amelynek minden létező példánya legfeljebb egy erőforrást birtokol, a birtoklást a konstruktor szerzi meg, a destruktor adja fel, és a példány nem másolható, csak mozgatható. A „legfeljebb egy” megfogalmazás azért fontos, mert megengedünk egy explicit üres állapotot is, amit egy sentinel érték jelöl. File descriptor esetén ez a `-1`, mert a `0` érvényes descriptor (a standard bemenet). Ha valaki `0`-t használ sentinelnek, a wrapper a destruktorában lezárja a szabványos bemenetet — ez a klasszikus hiba, és a tünete az, hogy a program egy későbbi `read`-nél `EBADF`-et kap.

Az osztály invariánsa tehát: *minden konstruált példányra igaz, hogy `fd_ == -1`, vagy `fd_ >= 0` és a descriptor kizárólag ezen a példányon keresztül birtokolt.* Az invariáns a konstruktor után áll fenn, és minden publikus művelet után ismét fenn kell állnia; ez a feltétel a `release`, `reset` és mozgató műveletek szerződését is meghatározza.

Az owner/observer megkülönböztetés nyelvi szinten nem kényszeríthető ki, ezért dokumentálni kell. A `void write_all(int fd, std::string_view data)` alakú paraméter observer: a függvény nem zárja be a descriptort, és nem is tárolhatja el a híváson túl. A `unique_fd` alakú paraméter owner-átadás; ha egy függvény `unique_fd&&`-t kap, abból a hívó tudja, hogy a hívás után az ő változója üres. Keverni a kettőt (egy `int fd` tagba elmenteni egy owner descriptor értékét) az élettartam-szerződés megsértése, és pontosan ez az a hiba, amit a smart pointerek sem fognak helyettünk megoldani.

21.2.2 A megszerzés: konstruktor, factory és hibaút

Két konstruktor-szerep létezik, és érdemes szétválasztani őket. Az *átvevő* konstruktor egy már megszerzett, nyers handlet kap, és feltétel nélkül átveszi a tulajdonjogot; ez nem hibázhat, ezért `noexcept`. A *megszerző* konstruktor maga hívja a rendszerhívást, és ez hibázhat — márpedig a konstruktornak nincs visszatérési értéke. Három szokásos megoldás van: kivételt dobni, factory függvényt írni `std::optional`/`std::expected` visszatéréssel, vagy kétfázisú konstrukciót használni (ez utóbbi épp az, amit el akarunk kerülni).

A lenti példában a `unique_fd(int)` explicit átvevő konstruktor, az `open_readonly` pedig factory, amely `std::optional<unique_fd>`-dal jelzi a sikert. A C++20 alapban a `std::expected` még nem elérhető (az egy későbbi verzió eleme, ezért a laborkörnyezetben ellenőrizni kell), így az `optional` a hordozható választás; a hibaokot (`errno`) a hívó a factory visszatérése után olvassa ki. Fontos, hogy a factory `noexcept`: a `::open` és az `optional` konstrukció nem dob, így a „sikertelenség” nem kivételként, hanem értékként jelenik meg.

```
// demo.cpp - 1. rész: unique_fd és open_readonly
#include <cerrno>
#include <cstdio>
#include <cstring>
#include <mutex>
#include <optional>
#include <utility>

#include <fcntl.h>
#include <unistd.h>

class unique_fd {
public:
    unique_fd() noexcept = default;                  // üres, nem birtokol
    explicit unique_fd(int fd) noexcept : fd_(fd) {} // átveszi a tulajdonjogot

    unique_fd(const unique_fd&) = delete;
    unique_fd& operator=(const unique_fd&) = delete;

    unique_fd(unique_fd&& other) noexcept
        : fd_(std::exchange(other.fd_, -1)) {}

    unique_fd& operator=(unique_fd&& other) noexcept {
```

```
        if (this != &other) {
            reset(other.release());
        }
        return *this;
    }

    ~unique_fd() { reset(); }

    [[nodiscard]] int get() const noexcept { return fd_; }
    [[nodiscard]] bool valid() const noexcept { return fd_ >= 0; }
    explicit operator bool() const noexcept { return valid(); }

    int release() noexcept { return std::exchange(fd_, -1); }

    void reset(int fd = -1) noexcept {
        if (fd == fd_) return;      // reset(get()): nem zárunk, nincs teendő
        if (fd_ >= 0) {
            (void)::close(fd_); // hiba nem továbbítható, ezért eldobjuk
        }
        fd_ = fd;
    }

    void swap(unique_fd& other) noexcept { std::swap(fd_, other.fd_); }

private:
    int fd_ = -1;
};

inline void swap(unique_fd& a, unique_fd& b) noexcept { a.swap(b); }

std::optional<unique_fd> open_readonly(const char* path) noexcept {
    const int fd = ::open(path, O_RDONLY | O_CLOEXEC);
    if (fd < 0) {
        return std::nullopt;
    }
    return unique_fd{fd};
}
```

A `reset(fd)` tulajdonátadási szerződése: a wrapper az *új* descriptort veszi át, a régit lezárja. Ha a hívó a már birtokolt számot adja vissza (`reset(get())`), a művelet nem csinál semmit — enélkül a wrapper lezárná a saját descriptorát, majd a már lezárt számot tárolná tovább, és egy későbbi `close` egy közben újra kiosztott, idegen descriptort zárhatna le. A kétszeri `reset()` biztonságos: a második hívásnál az `fd_` már -1. Az átvevő konstruktor ugyanígy kizárólagos tulajdont vesz át; ugyanazt a számot két wrappernek átadni előfeltétel-sértés. Az önmozgatás (`u = std::move(u)`) külön eset, azt a mozgató értékadás `this != &other` vizsgálata kezeli.

Buildjegyzőkönyv (`lab-gcc` profil). ASan/UBSan alatt egy próba igazolta: `reset(get())` után a descriptor nyitott és a tárolt szám változatlan; a `reset()` lezárja; a második `reset()` nem zár le egy közben megnyitott, gyakran ugyanazt a számot kapó idegen descriptort.

21.2.3 Felszabadítás: destruktor és hibajelzési policy

A destruktor alapértelmezésben `noexcept`, és ez nem stílus kérdése: ha egy destruktorból kivétel távozik a stack unwinding közben, a program `std::terminate`-mel leáll. Ezért a destruktorban hívott `close()` hibáját nem lehet és nem is szabad kivételként továbbadni. A gyakorlati policy három dologból áll: (1) a hibát eldobjuk vagy naplózzuk, (2) a `close()`-t nem ismételjük meg, (3) a hibát ott kell kezelni, ahol még van értelme — vagyis a felszabadítás előtti utolsó írás/flush hívásnál.

A `close()` ismétlése azért veszélyes, mert a `close()` után a descriptor szám azonnal újra kioszthatő; ha az `EINTR` hatására megismételt `close(fd)` egy közben megnyitott másik descriptorra fut, idegen erőforrást zárunk be. A helyes politika: egy kísérlet, az eredmény eldobása, esetleg `stderr`-re írt diagnosztika. Hasonló a helyzet `munmap`-nál és `fclose`-nál: ez utóbbi ráadásul a puffert is flush-olja, tehát a hiba adatvesztést is jelezhet — ezt a hívónak `std::fflush` + `ferror` kombinációval kell ellenőriznie, mielőtt a wrapper megsemmisül.

21.2.4 release, reset, swap és move szerződései

A négy művelet pontosan egymásra épül, és érdemes kimondani a feltételeket.

A `release()` visszaadja a nyers handlet, és a wrappert üres állapotba teszi. Szerződés: a hívó a visszatérés után tulajdonos, neki kell felszabadítania, és a wrapper további használata (a `get()` kivételével) értelmetlen. A `release()` nem zárja be az erőforrást — ezt sokan összekeverik a `reset()`-tel.

A `reset(h)` lezárja az aktuálisan birtokolt erőforrást, majd átveszi `h`-t. Feltétel: `h` vagy érvénytelen sentinel, vagy a hívó által átadott, másutt nem birtokolt handle; `reset(fd_)` önmagára nem megengedett, mert ilyenkor lezárnánk a descriptor, majd visszatennénk a lezárt értéket az invariáns alá.

A `swap` tagfüggvény és a mellette álló, azonos névtérben deklarált szabad `swap` együtt biztosítja, hogy az `std::swap` helyett ADL-lal a mi hatékony változatunk hívódjon (mindkettő `noexcept`).

A mozgató konstruktor és mozgató értékadás a birtoklás átadása: a forrás üres lesz. Az értékadásnál az ön-értékadást (`a = std::move(a)`) a `this != &other` ellenőrzés védi; enélkül a `reset(a.release())` lezárná a saját descriptorunkat, majd `-1`-t tenne vissza. Másolás nincs, mert két tulajdonos ugyanarra a descriptorra dupla `close`-hoz vezetne.

21.2.5 Kidolgozott példa: FD és lock egy programban

A lock-wrapper ugyanazt a vázat követi, de két ponton eltér. Egyrészt a megszerzés (`std::mutex::lock()`) dobhat `std::system_error`-t, ezért a konstruktor **nem** `noexcept` — ha mégis az lenne, a hiba `std::terminate`-mé silányulna. Másrészt a lock „handlja” nem egy érték, hanem egy mutexre mutató pointer, ahol a `nullptr` jelöli, hogy nem birtokolunk zárat. A `release()` itt azt jelenti: „nem oldok fel, de nem is tartom tovább számon a zár felszabadítását” — ez a `std::unique_lock::release()` mintájára pontosan az a művelet, amivel egy külső mechanizmusnak adunk át felelősséget.

```
// demo.cpp (folytatás) - 2. rész: mutex_guard és main, az 1. rész után
class mutex_guard {
public:
    // Nem noexcept: a std::mutex::lock() dobhat std::system_error-t.
    explicit mutex_guard(std::mutex& m) : m_(&m) { m_->lock(); }
```

```
    ~mutex_guard() { unlock(); }

    mutex_guard(const mutex_guard&) = delete;
    mutex_guard& operator=(const mutex_guard&) = delete;

    mutex_guard(mutex_guard&& other) noexcept
        : m_(std::exchange(other.m_, nullptr)) {}

    mutex_guard& operator=(mutex_guard&& other) noexcept {
        if (this != &other) {
            unlock();
            m_ = std::exchange(other.m_, nullptr);
        }
        return *this;
    }

    std::mutex* release() noexcept { return std::exchange(m_, nullptr); }

    void unlock() noexcept {
        if (m_ != nullptr) {
            m_->unlock();   // a szabvány szerint nem dob
            m_ = nullptr;
        }
    }

private:
    std::mutex* m_ = nullptr;
};

int main(int argc, char** argv) {
    if (argc < 2) {
        std::fprintf(stderr, "usage: %s FILE\n", argv[0]);
        return 2;
    }

    auto fd = open_readonly(argv[1]);
    if (!fd) {
        const int e = errno;   // az errno-t még a fprintf előtt mentsük el
        std::fprintf(stderr, "open(%s): %s\n", argv[1], std::strerror(e));
        return 1;
    }

    std::mutex m;
    {
        mutex_guard guard{m};
        char buf[64];
        const ssize_t n = ::read(fd->get(), buf, sizeof buf);
        if (n < 0) {
            const int e = errno;
            std::fprintf(stderr, "read: %s\n", std::strerror(e));
            return 1;   // a guard és a fd is felszabadul
        }
```

```
        std::printf("read %zd bytes\n", n);
    }                   // itt a zár felszabadul

    return 0;
}
```

Fordítás a laborprofilban rögzített GCC vagy Clang kiadással, C++20 módban:

```
g++ -std=c++20 -Wall -Wextra -pedantic -pthread -O2 -o demo demo.cpp
./demo ./demo.cpp
```

A példa POSIX/Linux környezetre készült (LP64, `ssize_t` = `long`, ezért a `%zd`), és az `O_CLOEXEC` elérhetősége a fordítónaplóban rögzített makrókörnyezettől függ (a `g++` Linuxon jellemzően definiálja a `_GNU_SOURCE`-t). A `demo.cpp` a két fenti blokk egymás után fűzve. **Buildjegyzőkönyv** (`lab-gcc` profil): a fenti parancs figyelmeztetés nélkül fordult; `./demo ./demo.cpp` kimenete `read 64 bytes` (0-s kód), nem létező fájlra `open(/nincs): No such file or directory` (1-es kód), argumentum nélkül a használati sor (2-es kód).

21.2.6 A scope exit és a stack unwinding előképe

A mechanizmus, ami miatt ez működik, a hatókör elhagyása. A destruktor lefut, ha a vezérlés normál úton kilép a blokkból, ha `return`-nel kilép, ha `break`/`continue`/`goto` visz ki a blokkból, és ha kivétel miatt megindul a stack unwinding. Ez utóbbi az, amiért a RAII kivételbiztos: a `return 1;` a fenti `mutex_guard` hatókörén belül pontosan ugyanúgy felszabadít, mintha egy `goto cleanup` címkére ugranánk — csak nem kell kézzel megírni és karbantartani a címkét.

Két korlátot érdemes rögzíteni. Egyrészt a `std::exit()` és a `std::abort()` nem futtatja le az automatikus tárolási idővel rendelkező objektumok destruktorait (a `std::exit` a statikusokat futtatja), tehát egy wrapper sem véd meg ott, ahol a folyamatot kilépésre kényszerítjük. Másrészt a C-ből örökölt `longjmp`, ha nem-triviális destruktorral rendelkező C++ objektumot kerül meg, nem definiált viselkedés — signal handlerekből ezért csak async-signal-safe műveleteket szabad végezni, RAII wrapper-t nem.

A C++20 alap nem tartalmaz szabványos `scope_exit`-et; a Library Fundamentals TS kínál ilyet, ezért ha általános „hatókör végén fusson ez a művelet” eszközre van szükség, saját, néhány soros wrappert írunk (lásd a feladatok között).

21.2.7 FD, mapping, FILE és lock: mi közös, mi eltérő

A közös váz mindenhol ugyanaz: megszerzés a konstruktorban, felszabadítás a destruktorban, move-only, explicit üres állapot. Az eltérések a sentinelben, a felszabadítás hibajelzésében és abban vannak, hogy a handle „elengedhető-e” felszabadítás nélkül.

Erőforrás	Megszerzés	Felszabadítás	Érvénytelen sentinel	Felszabadítás jelezhet hibát	Megjegyzés
file descriptor	`open`, `socket`, `accept`	`close`	`-1`	igen (`EINTR`, `EIO`), de nem ismételhető	a `0` érvényes
memóriatérkép	`mmap`	`munmap`	`MAP_FAILED`	igen	cím és hossz együtt birtokolt

Erőforrás	Megszerzés	Felszabadítás	Érvénytelen sentinel	Felszabadítás jelezhet hibát	Megjegyzés
`FILE*`	`fopen`, `fdopen`	`fclose`	`nullptr`	igen; puffert is flush-ol	glibc-ben belső lock
`std::mutex` zár	`lock`, `try_lock`	`unlock`	nincs érték; „nem birtokol” állapot	a `unlock()` nem dob	ugyanazt a mutexet kell feloldani

A lock annyiban speciális, hogy a „handle” önmagában nem azonosítja a felszabadítandó erőforrást: a mutexre van szükség hozzá. Ezért a `release()` itt nem erőforrás-átadás a szó FD-s értelmében, hanem felelősség-átadás; a wrapperből kikerült pointerrel a hívónak kell `unlock`-olnia. A `FILE*` esetében pedig a destruktor már „késő”: a flush hibáját a megsemmisítés előtt kell ellenőrizni, különben csendesen adatot veszítünk.

21.2.8 Tipikus hibák és javításuk

A leggyakoribb hiba a `0` sentinel használata `-1` helyett; javítás: `fd_ = -1` alapérték, és a `valid()` a `>= 0` feltételt vizsgálja, nem a `!= 0`-t. A második a másolás véletlen engedélyezése (pl. egy `= default` másoló konstruktor), ami dupla `close`-hoz vezet; javítás: a másoló műveletek `= delete`, és a mozgató műveletek `noexcept`-ok. A harmadik a `close()` újrapróbálása `EINTR`-re; javítás: egy kísérlet, az eredmény eldobása. A negyedik a `noexcept` rátétele egy lock-megszerző konstruktorra; javítás: a konstruktor hagyja, hogy a `std::system_error` kifelé menjen, és a hívó döntsön. Az ötödik a `release()` utáni használat: a `get()` `-1`-et ad, és a `read(-1, ...)` `EBADF`-et ad — ez nem hiba a wrapperben, hanem a szerződés megsértése a hívó oldalán. Végül a hatodik: egy observer `int fd` paramétert egy owner osztály tagjába menteni; ez a C projekt ownership-térképének elrontása, és a tünet csak jóval később, egy lezárt descriptor újrahasznosításakor jelentkezik.

21.2.9 Összefoglaló

A RAII wrapper lényege nem a destruktor, hanem a szerződés: pontosan egy birtokolt erőforrás, explicit üres állapot sentinel értékkel, move-only viselkedés, és a megszerzés hibájának értékként (vagy kivételként) való megjelenítése a konstruktoron kívül. A felszabadítás hibáját a destruktor nem tudja továbbadni, ezért azt a megsemmisítés előtti utolsó ellenőrizhető ponton kell kezelni. A `release`/`reset`/`swap`/mozgatás négy külön szerződés, amelyeket külön kell dokumentálni és tesztelni. Az FD, a `mmap`, a `FILE*` és a lock ugyanerre a vázra illeszkedik, de a sentinel, a hibajelzés és a „felszabadítás nélküli elengedés” lehetősége eltér.

21.2.10 Feladatok

1. feladat (alap). Egészítse ki a `unique_fd` osztályt egy `make_pipe()` factory függvénnyel, amely `std::optional<std::pair<unique_fd, unique_fd>>`-t ad vissza, és hiba esetén üres opcionálist. Fogalmazza meg a két descriptor tulajdonlási viszonyát és a hibaútját.

2. feladat (közepes). Írjon egy általános `scope_exit` wrappert, amely egy callable-t futtat a hatókör elhagyásakor, és támogatja a `dismiss()`-t. Cserélje ki vele a `mutex_guard`-ot egy olyan függvényben, ahol a zár felszabadítása előtt még hibát kell jelenteni. Hasonlítsa össze a két megoldás kivételbiztosságát.

3. feladat (diagnosztikai). Készítsen `unique_file` wrappert `std::FILE*`-hoz, és mutassa meg, miért nem elegendő a `fclose()` hibáját a destruktorban eldobni. Adjon egy olyan hívási mintát, ahol a flush hibája még a wrapper élettartama alatt kiderül.

21.2.11 Megoldások

21.2.11.1 1. megoldás

```
std::optional<std::pair<unique_fd, unique_fd>> make_pipe() noexcept {
    int fds[2];
    if (::pipe(fds) != 0) {
        return std::nullopt;
    }
    return std::pair<unique_fd, unique_fd>{unique_fd{fds[0]},
     ↪  unique_fd{fds[1]}};
}
```

Indoklás: a `::pipe` két descriptort ír ki; a sikeres hívás után a `fds` tömb mindkét eleme a hívó tulajdonába kerül, és a `pair` létrehozásával azonnal két külön `unique_fd` birtokolja őket. Ha a `pipe` hibázik, a tömb tartalma nem használható, ezért nem szabad „óvatosan” mégis wrapperezni. A `noexcept` azért tartható, mert a `pipe`, az `optional` és a `pair` konstrukció nem dob. Ellenőrzési feltétel: hibaágon a visszatérés `std::nullopt`, sikerágon `a.get() >= 0 && b.get() >= 0`, és a két descriptor különbözik. Linux-specifikus változat: `pipe2(fds, O_CLOEXEC)`, ezt azonban a laborprofilban jelölni kell.

21.2.11.2 2. megoldás

```
#include <type_traits>
#include <utility>

template <class F>
class scope_exit {
public:
    explicit scope_exit(F f) noexcept(std::is_nothrow_move_constructible_v<F>)
        : f_(std::move(f)) {}

    scope_exit(const scope_exit&) = delete;
    scope_exit& operator=(const scope_exit&) = delete;

    ~scope_exit() {
        if (active_) {
            f_();
        }
    }

    void dismiss() noexcept { active_ = false; }

private:
    F f_;
    bool active_ = true;
};
```

Használat zárral:

```
std::mutex m;
{
    m.lock();
    scope_exit unlocker{[&m] { m.unlock(); }};
    // ... kritikus szakasz ...
    if (hiba) {
        return;          // az unlocker lefut
    }
    unlocker.dismiss();  // ha mégis mi oldunk fel
    m.unlock();
}
```

Indoklás: a `mutex_guard` típusa kifejezi a birtoklást, a `scope_exit` viszont csak egy műveletet köt a hatókörhöz; ez utóbbi akkor jó, ha a „felszabadítás” nem egy handle visszaadása, hanem egy oldalhatás. A kivételbiztosság szempontjából a `scope_exit` gyengébb: nem tiltja meg, hogy valaki kézzel is felszabadítson, ezért a `dismiss()` fegyelmezett használata a hívó felelőssége. Ellenőrzési feltétel: a `dismiss()` után a callable nem fut le, hibaágon viszont pontosan egyszer.

21.2.11.3 3. megoldás

```
#include <cstdio>
#include <utility>

class unique_file {
public:
    unique_file() noexcept = default;
    explicit unique_file(std::FILE* f) noexcept : f_(f) {}

    unique_file(const unique_file&) = delete;
    unique_file& operator=(const unique_file&) = delete;

    unique_file(unique_file&& o) noexcept : f_(std::exchange(o.f_, nullptr)) {}

    unique_file& operator=(unique_file&& o) noexcept {
        if (this != &o) {
            reset(o.release());
        }
        return *this;
    }

    ~unique_file() { reset(); }

    [[nodiscard]] std::FILE* get() const noexcept { return f_; }
    std::FILE* release() noexcept { return std::exchange(f_, nullptr); }

    void reset(std::FILE* f = nullptr) noexcept {
        if (f_ != nullptr) {
            (void)std::fclose(f_);  // a flush hibája itt elveszne
        }
        f_ = f;
```

```
    }

private:
    std::FILE* f_ = nullptr;
};
```

Használat, ahol a hiba még időben kiderül:

```
auto f = std::fopen("out.bin", "wb");
if (f == nullptr) { /* hiba */ }

unique_file uf{f};
if (std::fwrite(buf, 1, n, uf.get()) != n) { /* írási hiba */ }
if (std::fflush(uf.get()) != 0) { /* itt még látszik a hiba */ }
// uf destruktora innen már csak egy újabb fclose-t végez
```

Indoklás: a `fclose()` hívása flush-ol, és ennek hibája adatvesztést jelenthet; mivel a destruktor nem jelezhet, a helyes minta a tényleges utolsó írás után explicit `fflush`-t hívni, és ott kezelni a hibát. A `reset()` „eldobja” a visszatérési értéket, ami tudatos döntés, és a szerződésben dokumentálni kell. Ellenőrzési feltétel: a `fflush` hibáját a program a wrapper megsemmisülése előtt észleli, a `release()` után pedig a `std::FILE*` tulajdonosa a hívó, aki `fclose`-t hív.

21.3 unique_ptr és egyedi tulajdonlás

Az előző részben (RAII wrapper file descriptorhoz és lockhoz (21.2)) egy kézzel írt RAII wrapperrel kötöttük össze a fájlleíró élettartamát egy objektum élettartamával. Ez a minta helyes volt, de minden erőforrástípushoz külön osztályt igényel: destruktort, move konstruktort, move értékadást, törölt másolást. A standard library erre a leggyakoribb esetre — egyetlen, dinamikusan létrehozott objektum vagy egy külső, pointerrel azonosított erőforrás — kész, jól bejárt megoldást ad: a `std::unique_ptr`-t. Ez az alfejezet azt mutatja meg, milyen szerződést kell ehhez a típushoz illeszteni, és hol bukik el a látszólag ártalmatlan `get()`/`release()` használat.

21.3.1 Az egyedi tulajdonlás szerződése

A C memóriamodelljében (lásd Tulajdonlási szerződés és cleanup ágak (8.4)) a tulajdonlás puszta konvenció: a `malloc`-ot és a `free`-t ugyanannak a kódágnak kell összehangolnia, és a fordító ezt nem ellenőrzi. A C++ ezt a konvenciót típussá teszi.

Definíció. A `std::unique_ptr<T, D>` egy move-only, azaz mozgatással átadható, de nem másolható osztálysablon, amely egy `T*` értéket és egy `D` típusú deletert tárol. A tárolt pointer az objektum tulajdonában van: a `unique_ptr` destruktora a deletert a pointerrel meghívja, és a `unique_ptr` élettartama alatt a szabvány szerződése szerint egyetlen más `unique_ptr` sem birtokolhatja ugyanazt az objektumot.

Előfeltételek és feltételek. - A `unique_ptr` másoló konstruktora és másoló értékadása törölt; tulajdonlást csak mozgatással lehet átadni. - A mozgatott-forrás (`moved-from`) állapota definiált: üres lesz, `get() == nullptr`. Ez nem “véletlenszerű szemét”, hanem szerződéses posztkondíció. - Az üres `unique_ptr` legális és normális; a deletert ilyenkor nem hívja meg (a `reset()` null pointerre hatástalan). - A `unique_ptr` nem garantálja, hogy a mutatott objektum érvényes: a mutatott objektumot bárki törölheti, ha nyers pointerhez jut. A típus a *tulajdonlás tényét* kódolja, nem a teljes élettartam-biztonságot.

Ez utóbbi pont a fejezet egyik fő határa: a smart pointer nem old meg minden lifetime-hibát. Ha egy observer (`T*`, `T&`, iterátor) túlél egy `reset()`-et, move-ot vagy a owner megszűnését, a hiba ugyanúgy dangling hivatkozás lesz, mintha C-ben dolgoznánk.

21.3.2 unique_ptr<T> és unique_ptr<T[]>

A két specializáció nem ugyanaz a típus, és nem konvertálható egymásba.

Tulajdonság	`unique_ptr<T>`	`unique_ptr<T[]>`
Alapértelmezett deleter	`std::default_delete<T>` → `delete p`	`std::default_delete<T[]>` → `delete[] p`
`operator*`, `operator->`	van	nincs
`operator[]`	nincs	van (C++20-tól indexelés)
Tipikus forrás	`new T(...)`, `make_unique<T>(...)`	`new T[n]`, `make_unique<T[]>(n)`
Méretezés szemantikája	egy objektum	tömbtömb, a hossz nem tárolt

A `unique_ptr<T[]>` nem tudja a tömb hosszát: a `delete[]` a fordító által eltárolt könyvelésből dolgozik, a `size()` fogalma nem létezik. Ezért a hosszt külön kell vinni (például `std::span`-ban, lásd span, byte és bounds-szerződések (31.3)) — a `unique_ptr<T[]>` csak a felszabadításért felel.

Egy fontos részlet: `std::make_unique<T[]>(n)` a `new T[n]`-t *value-initializálja*, azaz skalár típusra nullázza, osztályra a default konstruktort hívja. A nyers `new T[n]` ezzel szemben default-initializál, ami skalár típusnál határozatlan értéket hagy. A `make_unique` tehát itt nem csak kényelem, hanem inicializálási garancia is.

21.3.3 make_unique és a konstrukció kivételbiztonsága

A `unique_ptr` legfontosabb hozadéka, hogy a tulajdonlás már az objektum létrejötte pillanatában kialakul. Nyers `new`-val ez nem így van:

```
// ROSSZ tanítási példa: nem ez a helyes változat
use_widget(new Widget(cfg), compute_limit());   // compute_limit() dobhat
```

A sorrend itt nem garantált, hogy a `Widget` átadása előbb megtörténik-e: ha a `new Widget` lefut, majd a `compute_limit()` kivételt dob, a lefoglalt `Widget*` semmihez sem tartozik, és elszivárog. A hívási kifejezés nem fejeződik be, így a paraméterként átvett, deklarált paraméter destruktora sem fut le. C++17 óta az argumentumkiértékelés sorrendje indeterminately sequenced, de ez nem zárja ki a szivárgást, mert a kivétel a *hívás előtt* történik, amikor a pointer még senki tulajdona.

A helyes változat:

```
auto w = std::make_unique<Widget>(cfg);   // innentől owner
use_widget(w.get(), compute_limit());     // ha dob, w destruktora lefut
```

A `make_unique` a `new`-ot és a `unique_ptr` konstrukcióját egyetlen kifejezésbe zárja, így a `new` sikere és a tulajdonba vétel között nincs kivételes pont. Ugyanezért ne írjunk `std::unique_ptr<T>(new T(...))` alakot: működik, de elveszíti a kifejezés-szintű védelmet, és a `make_unique` rövidebb is.

A C++20-as alapból visszafelé is van fallback: a `make_unique` C++14-es, tehát C++20-cal mindenképp elérhető. Ami viszont C++20-hoz kötött: a capture nélküli lambda típusa default-konstruálható, így lambda használható deleterként anélkül, hogy külön struktúrát írnánk — C++17-es vagy korábbi profilban ehhez nevesített functor kell.

21.3.4 get, reset, release: a tulajdonlás átadásának eszközei

Művelet	Tulajdonlás	Pointer	Mikor használd
`get()`	marad	változatlan	átadás nem tulajdonló API-nak
`operator*`, `operator->`	marad	változatlan	közvetlen használat
`reset(p)`	marad, új pointerre	régi felszabadul, új bekerül	újratöltés, „reseat”
`reset()`	megszűnik	üres lesz	korai felszabadítás
`release()`	**átadódik a hívónak**	a `unique_ptr` üres lesz	csak indokolt FFI/ABI határon
move konstruktor/értékadás	átadódik	forrás üres lesz	owner átadása

A `release()` a legveszélyesebb: visszaadja a nyers pointert, és a `unique_ptr` üres lesz — a felelősség a hívóé. Ha a hívó nem tudja biztosan, hogy a címzett felszabadítja, memóriaszivárgás lesz belőle, tipikusan úgy, hogy `ptr.release()` eredményét eldobjuk. A `release()` helyes használata az az eset, amikor egy C API vesz át tulajdonlást, és a dokumentációja ezt kimondja. A `reset()` ezzel szemben biztonságos: előbb felszabadít, aztán átveszi az újat.

21.3.5 Custom deleter: nem memóriajellegű erőforrás

A `std::default_delete<T>` egyszerűen `delete`-et hív, ami nem minden erőforrásra jó. A `FILE*`, a `DIR*`, a `sqlite3*`, egy socket, egy `mmap`-elt régió vagy egy `dlopen` handle nem `delete`-tel szabadul fel. A deleter ezt a rést tölti be.

```
// file_owner.cpp - C++20, teljes program
#include <cstdio>
#include <memory>
#include <string>
#include <utility>

struct FileCloser {
    void operator()(std::FILE* f) const noexcept {
        if (f != nullptr) { std::fclose(f); }
    }
};
using FilePtr = std::unique_ptr<std::FILE, FileCloser>;

enum class ReadResult { line, eof, error };

class LineSource {
public:
```

```
    explicit LineSource(FilePtr in) : in_(std::move(in)) {}   // owner átvétele

    // Nem tulajdonló kimenet: a hívó birtokolja a buffert.
    // line: out egy teljes (vagy az utolsó, LF nélküli) sor;
    // eof:  nincs több adat, out üres;
    // error: olvasási hiba; out tartalma (részleges sor) nem érvényes rekord.
    ReadResult next(std::string& out) {
        out.clear();
        for (int c = std::fgetc(in_.get()); c != EOF; c = std::fgetc(in_.get()))
        ↪ {
            if (c == '\n') { return ReadResult::line; }
            out.push_back(static_cast<char>(c));
        }
        if (std::ferror(in_.get())) { return ReadResult::error; }   // EOF
        ↪ fájlvég
        return out.empty() ? ReadResult::eof : ReadResult::line;
    }
private:
    FilePtr in_;   // owner: a fájl élettartama ehhez az objektumhoz kötött
};

FilePtr open_file(const char* path) {
    return FilePtr(std::fopen(path, "rb"));   // nullptr → üres FilePtr, legális
}

int main(int argc, char** argv) {
    if (argc != 2) {
        std::fprintf(stderr, "usage: file_owner PATH\n");
        return 2;
    }
    FilePtr f = open_file(argv[1]);
    if (!f) {                         // üres owner: a megnyitás sikertelen
        std::perror(argv[1]);
        return 1;
    }
    LineSource source(std::move(f));  // az owner átkerül a LineSource-ba
    std::string line;                 // a hívó birtokolja a puffert
    std::size_t count = 0;
    ReadResult r;
    while ((r = source.next(line)) == ReadResult::line) {
        ++count;
    }
    if (r == ReadResult::error) {
        std::fprintf(stderr, "%s: read error after %zu lines\n", argv[1],
        ↪ count);
        return 3;
    }
    std::printf("lines=%zu\n", count);
}
```

A példa szerződései: a `LineSource` az egyetlen owner; a `next()` `std::string&` paramétere *borrowed*, a hívó felel érte; a `open_file` sikertelenséget üres `FilePtr`-rel jelez, nem kivétellel, így

a hívónak `if (!f) { ... }` ellenőrzést kell tennie. A `next()` háromféle eredményt ad, mert az `fgetc` `EOF` visszatérése kétféle eseményt takar: fájlvéget *vagy* olvasási hibát. A kettőt csak az `ferror` választja szét. Fájlvégnél az utolsó, LF nélküli sor még szabályos `line`, utána `eof` jön; hibánál `error`, és az `out`-ban maradt részleges sor nem válik csendben sikeres rekorddá. A `FileCloser::operator() noexcept`: a destruktorból hívott deleter nem dobhat, különben `std::terminate` lenne a következmény. Ez a lezárásra vonatkozó szerződés, és nem tiltja, hogy a `next()` hibát jelezzen — eredménykóddal, ahogy itt, vagy akár kivétellel.

A `main` driver a teljes programhoz tartozik: argumentumot ellenőriz, az üres ownert hibaként kezeli, megszámolja a sorokat, és olvasási hibánál 3-mal lép ki. Build és diagnosztikai parancs:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -g -O1 \
    -fsanitize=address,undefined file_owner.cpp -o file_owner
printf 'a\nb\nc' > three.txt      # utolsó sor LF nélkül
printf 'a\nb\nc\n' > three_lf.txt
: > empty.txt
./file_owner three.txt; ./file_owner three_lf.txt; ./file_owner empty.txt
mkdir -p adir; ./file_owner adir   # Linux: fopen sikerül, az olvasás EISDIR
```

Buildjegyzőkönyv (`lab-gcc` profil). A program figyelmeztetés nélkül fordult; a `three.txt` és a `three_lf.txt` fájlra `lines=3`, az üres fájlra `lines=0` kimenettel, 0 kóddal futott, ASan/UBSan hibajelzés nélkül. A könyvtárra `adir: read error after 0 lines` üzenettel 3-mal lépett ki; ez glibc/Linux-specifikus reprodukció (a `fopen` könyvtárra is sikerül, az első olvasás `EISDIR` hibát ad), nem hordozható teszt. Nem létező fájlra a `perror` üzenetét írja ki, és 1-gyel lép ki; argumentum nélkül a használati sort írja ki, és 2-vel lép ki.

A hibaágat a sor elején és egy részleges sor után is célzottan érdemes kipróbálni. Ehhez a glibc `fopencookie` függvényével olyan `FILE*`-ot építünk, amely előbb adatot ad, majd `EIO` hibát; a `line_source.hpp` a fenti program `main` nélküli része:

```
// read_error_probe.cpp - C++20 + glibc fopencookie: injektált olvasási hiba
#include "line_source.hpp"
#include <cassert>
#include <cerrno>
#include <cstring>
#include <sys/types.h>

struct Script { const char* data; bool sent; };

static ssize_t fail_read(void* cookie, char* buf, size_t size) {
    auto* s = static_cast<Script*>(cookie);
    if (!s->sent && s->data[0] != '\0') {          // első hívás: adat
        std::size_t n = std::strlen(s->data);
        if (n > size) n = size;
        std::memcpy(buf, s->data, n);
        s->sent = true;
        return static_cast<ssize_t>(n);
    }
    errno = EIO;                                    // utána: I/O hiba
    return -1;
}

static ReadResult first_after(const char* data, int skip, std::string& out) {
    static Script s; s = Script{data, false};
```

```
    cookie_io_functions_t io{fail_read, nullptr, nullptr, nullptr};
    LineSource src(FilePtr(fopencookie(&s, "r", io)));
    ReadResult r = ReadResult::line;
    for (int i = 0; i <= skip; ++i) r = src.next(out);
    return r;
}

int main() {
    std::string out;
    assert(first_after("", 0, out) == ReadResult::error);            // hiba a
     ↪ sor elején
    assert(first_after("ab\npartial", 0, out) == ReadResult::line && out ==
     ↪ "ab");
    assert(first_after("ab\npartial", 1, out) == ReadResult::error); //
     ↪ részleges sor után
    std::puts("read_error_probe: OK");
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic -g -O1 -fsanitize=address,undefined read_error_probe.cpp` figyelmeztetés nélkül fordult, a futás kimenete `read_error_probe: OK`: üres adat után azonnali hibára `error`, az `ab\npartial` adat után az első hívás `line` (`ab`), a második — a `partial` részleges sor után érkező hibára — `error`. A `fopencookie` GNU-kiterjesztés; más platformon ugyanez a próba egy saját olvasási interfész mögé tett hibainjektáló forrással végezhető el.

21.3.5.1 A deleter és a sizeof implementációfüggése

A `unique_ptr` mérete nem szabványosított. A gyakorlatban az állapot nélküli (empty) deleterekre a fordítók üres bázis-optimalizálást (EBO) alkalmaznak, így a `sizeof(std::unique_ptr<T>)` jellemzően egy pointer méretű; függvénypointer-deleterrel (`decltype(&std::fclose)`) jellemzően két pointer; állapotot hordozó deleterrel ennél is több. Ezek *implementációs megfigyelések*, nem hordozható garanciák: ABI-stabil bináris interfészben ne építsünk rájuk, és ne tegyük fel, hogy `sizeof(unique_ptr<T>) == sizeof(T*)`. Ha egy konkrét laborprofilban mégis számít, mérjük és rögzítsük `static_assert`-tel, a target ABI és a standard library kiadásának megnevezésével.

Ugyanide tartozik, hogy a `unique_ptr<T, D>` csak akkor default-konstruálható, ha a `D` is az. Egy capture-t tartalmazó lambda nem default-konstruálható, így az ilyen deleterrel a `unique_ptr` is csak deleter megadásával hozható létre — ez gyakran meglepi a kezdőt.

21.3.6 Incomplete type és a destruktor helye

A `unique_ptr<T>` deklarációjához nem kell, hogy a `T` teljes típus legyen; a destruktorához, a `reset()`-hez és a deleter hívásához viszont igen, mert ezeknél `sizeof(T)`-re és a `T` destruktorára van szükség. Ez a pimpl (pointer to implementation) minta alapja:

```
// widget.hpp: a Widget::Impl itt még incomplete
#include <memory>
class Widget {
public:
    Widget();
    ~Widget();                                  // deklarálva, nem = default itt!
    Widget(Widget&&) noexcept;
```

```
    Widget& operator=(Widget&&) noexcept;
private:
    struct Impl;
    std::unique_ptr<Impl> impl_;
};

// widget.cpp: itt válik teljessé az Impl
#include "widget.hpp"
#include <vector>
struct Widget::Impl { std::vector<int> data; };
Widget::Widget() : impl_(std::make_unique<Impl>()) {}
Widget::~Widget() = default;                // a definíció helyén teljes az Impl
Widget::Widget(Widget&&) noexcept = default;
Widget& Widget::operator=(Widget&&) noexcept = default;
```

Gyakori hiba: a destruktort a headerben `= default`-tal definiálni. Ekkor a destruktor a headerben, az `Impl` teljessé válása előtt példányosul, és a fordítás „invalid application of sizeof to incomplete type” hibával áll meg. Ugyanez a helyzet a move műveletekkel: ha a headerrben `= default`-ot írunk, az implicit definíció ott példányosul, ahol az `Impl` még hiányos.

21.3.7 Owner és borrowed observer az API-ban

Az egyedi tulajdonlás akkor működik, ha az interfész egyértelműen elárulja, ki a felelős. Négy jellegzetes szignatúra:

Szignatúra	Jelentés
`void sink(std::unique_ptr<T> p)`	átveszi a tulajdonlást; a hívó után üres pointert kap vissza
`void sink(std::unique_ptr<T>&& p)`	ugyanaz, `&&`- jelöléssel; nem fogad el lvalue-t
`void borrow(const T& x)` / `void borrow(T* x)`	nem tulajdonol; csak az owner élettartama alatt érvényes
`std::unique_ptr<T> make()`	új ownert ad vissza (move-pal)

A `T*` akkor jó, ha a hiányzó érték is megengedett (`nullptr`); ha nem, a `T&` kifejezőbb. Egyik sem él túl egy `reset()`-et, move-ot vagy a owner megszűnését — ezt a hívási szerződésben kell rögzíteni, mert a fordító nem fogja.

Kivétel, amit érdemes fejben tartani: a `unique_ptr<Derived>` nem konvertálható automatikusan `unique_ptr<Base>`-re, ha a `default_delete<Base>` nem tudja fogadni a `Derived*`-ot; a `Base`-nek tehát virtuális destruktor kell (lásd Virtuális destruktor, slicing és ownership (24.3)). A `unique_ptr<Base>(new Derived)` alak ráadásul a `make_unique` védelmét elveszti, ezért ilyenkor `unique_ptr<Base>`-t `make_unique<Derived>()`-ből, explicit módon kell építeni.

21.3.8 Tipikus hibák és javításuk

1. **Tárolt `get()` eredmény.** `const T* p = owner.get();` majd `owner.reset(...)` — a `p` dangling. Javítás: az observer élettartamát az owner élettartamához kötni, vagy `weak_ptr`-t használni (lásd shared_ptr, control block és weak_ptr (21.4)) ott, ahol a hosszabb életű megfigyelés valóban igény.

2. **`release()` a `reset()` helyett.** `p.release(); p.reset(new T);` — az első sor elszivárogtatja a régit. Javítás: `p.reset(...)`.
3. **`new` + `unique_ptr` konstrukció szétbontva.** Kivétel esetén szivárgás. Javítás: `make_unique`.
4. **Incomplete type a headerben defaultolt destruktorral.** Javítás: a destruktor és a move műveletek deklarációja a headerben, definíciója a `.cpp`-ben.
5. **Hossz nélküli `unique_ptr<T[]>`.** A tömb mérete nem kérdezhető le; a hosszt külön kel vinni. Javítás: `unique_ptr<T[]>` + méret, vagy `std::vector<T>`.

21.3.9 Feladatok

1. feladat (értelmezés). Az alábbi szignatúrák közül melyik ad át tulajdonlást, melyik csak kölcsönöz? Indokolj. (a) `void f(std::unique_ptr<Conn>)`, (b) `void g(const Conn&)`, (c) `std::unique_ptr<Conn> h()`, (d) `void i(std::unique_ptr<Conn>&)`, (e) `void j(Conn*)`.

2. feladat (kódolás). Készíts `FdPtr` típust POSIX fájlleíróhoz `struct FdCloser` deleterrel, `::close`-t hívva. Írj `FdPtr open_fd(const char*)` függvényt, és egy `Duplex` osztályt, amely két `FdPtr`-t birtokol, mozgatható, és a `read`/`write` irányt külön tárolja. Add meg a buildparancsot.

3. feladat (diagnosztika). Az alábbi kód lefordul, de futásidőben hibás. Nevezd meg a hibát, a dangling observer élettartam-határát és a javítást.

```
class Session {
public:
    explicit Session(std::unique_ptr<Conn> c) : conn_(std::move(c)) {}
    const Conn* conn() const { return conn_.get(); }
    void reconnect(std::unique_ptr<Conn> c) { conn_ = std::move(c); }
private:
    std::unique_ptr<Conn> conn_;
};

auto s = std::make_unique<Session>(std::make_unique<Conn>());
const Conn* c = s->conn();                  // borrowed observer
s->reconnect(std::make_unique<Conn>());     // invalidálja c-t
inspect(*c);                                // ?
```

21.3.10 Megoldások

1. megoldás. (a) A paraméter érték szerinti `unique_ptr`, tehát a hívó move-pal adja át, és a hívás után a saját változója üres lesz — tulajdonlásátadás, „sink”. (b) `const Conn&`: nem tulajdonol, csak a hívás idejére érvényes, a hívó felel az élettartamért. (c) Visszatérési értékként `unique_ptr`: új ownert ad, a hívó veszi át (a move-t a visszatérési érték biztosítja). (d) `unique_ptr<Conn>&`: nem ad át tulajdonlást, hanem *újraültetheti* a hívó objektumát — out-paraméter, amely a hívó ownerét módosítja; ez nem sink, mert a hívó `unique_ptr`-je tovább él. (e) `Conn*`: nem tulajdonol; nullable observer, élettartama a hívó felelőssége. Ellenőrzési feltétel: minden válasznál nevezd meg, ki a owner a hívás után.

2. megoldás.

```
// fd_owner.cpp - C++20, Linux/POSIX
#include <unistd.h>
#include <cerrno>
#include <memory>
```

```cpp
#include <utility>

struct FdCloser {
    void operator()(int* p) const noexcept {
        if (p != nullptr) { ::close(*p); delete p; }
    }
};
using FdPtr = std::unique_ptr<int, FdCloser>;

FdPtr adopt_fd(int fd) {
    if (fd < 0) { return FdPtr(nullptr); }
    return FdPtr(new int(fd));      // a leírt értéket a heapen tároljuk
}

class Duplex {
public:
    Duplex(FdPtr in, FdPtr out) : in_(std::move(in)), out_(std::move(out)) {}
    Duplex(Duplex&&) noexcept = default;
    Duplex& operator=(Duplex&&) noexcept = default;
    Duplex(const Duplex&) = delete;
    Duplex& operator=(const Duplex&) = delete;

    int in_fd()  const { return in_  ? *in_  : -1; }   // borrowed: nem ad át
     ↪ ownert
    int out_fd() const { return out_ ? *out_ : -1; }
    bool valid() const { return static_cast<bool>(in_) &&
     ↪ static_cast<bool>(out_); }
private:
    FdPtr in_, out_;
};
```

Indoklás: a `unique_ptr` deletere pointert kap, ezért a `int` leírót heapen kell tárolni; a `FdCloser` állapot nélküli, így a `unique_ptr` mérete jellemzően egy pointer marad (ezt konkrét laborprofilban ellenőrizd, ne feltételezd). A másolás törlése és a move `noexcept`-ká tétele azért kell, hogy a `Duplex` konténerbe tehető legyen (lásd noexcept, swap és container-reallocation (22.5)). A `close()` hibáját a deleter nem tudja jelenteni — ez tudatos tervezési döntés; ha számít, a lezárást explicit `close_checked()` metódusban kell kezelni a `reset()` előtt. Build: `g++ -std=c++20 -Wall -Wextra -Wpedantic -g -fsanitize=address,undefined fd_owner.cpp -o fd_owner`. Ellenőrzési feltétel: `Duplex` move-ja után a forrás `valid()` értéke `false`, és a mozgatott `Duplex` destruktora pontosan két `close`-t vált ki.

3. megoldás. A hiba nem a `unique_ptr`-ben van, hanem a `conn()` által visszaadott borrowed observerben: a `const Conn* c` élettartam-határa a `reconnect()` hívás (pontosabban az a pont, ahol a `conn_` régi értéke `reset`-elődik), nem a `Session` élettartama. A `reconnect()` után a `c` dangling pointer, az `inspect(*c)` nem definiált viselkedés — a tünet lehet látszólag helyes működés, ASan-hiba vagy összeomlás, garantált kimenet nincs. Javítási lehetőségek: (i) az observer használatát a `reconnect()` elé rendezni, és az élettartamot kommentben/documentációban rögzíteni; (ii) ha hosszabb életű megfigyelés kell, `std::weak_ptr`-t használni a `Conn`-ra (ez `shared_ptr` tulajdonlást igényel, lásd shared_ptr, control block és weak_ptr (21.4)); (iii) az `inspect`-et a `Session` egy tagfüggvényévé tenni, amely mindig a `conn_` aktuális értékét használja, és nem enged kívülre pointert. Ellenőrzési feltétel: a javított változatban az observer soha nem él túl egy `reset`/move/reconnect műveletet, és ezt a fejléc kommentje kimondja.

21.3.11 Összefoglaló

A `std::unique_ptr` a C-s tulajdonlási konvenciót típusba öntött, move-only szerződése: pontosan egy owner, determinisztikus felszabadítás, kivételbiztos konstrukció `make_unique`-jal. Az objektumtömb és az egyedi objektum két külön specializáció, eltérő deleterrel és operátorokkal. A `get()` kölcsönöz, a `reset()` újraültet, a `release()` átadja a felelősséget — ez utóbbi csak indokolt FFI-határon használható. A custom deleter nem memóriajellegű erőforrásokat (`FILE*`, fd, handle) tesz szabályossá, de a `unique_ptr` mérete implementációfüggő, ezért ABI-határon nem építhetünk rá. Incomplete type esetén a destruktort és a move műveleteket a headerben csak deklarálni szabad, definiálni a teljes típus mellett kell. Végül: a `unique_ptr` a tulajdonlást rendezi, az observer élettartamát nem — a `get()` eredménye ugyanolyan borrowed, mint bármely nyers pointer.

21.4 shared_ptr, control block és weak_ptr

A megosztott tulajdonlás akkor indokolt, amikor egy erőforrás élettartama nem köthető egyetlen lexikai hatókörhöz vagy egyetlen tulajdonoshoz. Tipikus eset a több helyről elérhető, aszinkron callbackben élő objektum, egy gráf csomópontja, vagy egy gyorsítótár bejegyzése. Ilyenkor a `std::shared_ptr` több tulajdonost enged, és az objektum akkor semmisül meg, amikor az utolsó tulajdonos elengedi. Ez azonban nem alapértelmezés: ahol egyetlen tulajdonos kijelölhető, ott a unique_ptr és egyedi tulajdonlás (21.3) egyszerűbb, olcsóbb és kiszámíthatóbb. A `std::weak_ptr` nem tulajdonos, hanem olyan megfigyelő, amely biztonságosan meg tudja kérdezni, hogy a kezelt objektum még él-e. A három eszköz együtt adja a C++ determinisztikus erőforrás-kezelésének egyik fontos szeletét, de nem oldja meg automatikusan a ciklusokat, a dangling nyers pointereket vagy a szálbiztonságot.

A megosztott tulajdonlás szerződése szorosan épül a korábbi objektum-élettartam-szabályokra: a `shared_ptr` destruktora nem „szemétgyűjt”, hanem egy referenciaszámlálót csökkent, és az utolsó erős hivatkozás megszűnésekor determinisztikusan lefuttatja a deletetert vagy a destruktort. A Tároló újrafelhasználása és C++ objektumszabályok (20.6) részben tárgyalt objektumélettartam itt két külön élettartamra bomlik: a kezelt objektuméra és a control blockéra.

21.4.1 Control block, strong count és object lifetime

A `std::shared_ptr<T>` egy másolható tulajdonosi fogantyú. Két lényeges része van: a kezelt objektumra mutató pointer, amelyet `get()` ad vissza, és egy control blockra mutató pointer. A control block a `shared_ptr` létrehozásakor jön létre, és nyilvántartja az erős (`strong`) és gyenge (`weak`) hivatkozások számát, valamint a deletetert és az allocatort. A control block konkrét típusa, mérete és elrendezése nem része a nyelvi szerződésnek; a szabvány a megfigyelhető viselkedést írja le, nem a belső bájtszerkezetet.

Amikor az utolsó erős `shared_ptr` megszűnik, a kezelt objektum élettartama véget ér: lefut a destruktor vagy az egyedi deleter. A control block azonban még életben marad, ha arra `weak_ptr` mutat. A control block csak akkor szabadul fel, amikor a strong és a weak hivatkozások száma is nulla. A weak számláló pontos belső értéke implementációfüggő lehet; a lényeg az, hogy a `weak_ptr` ne érje el a már megsemmisült objektumot, de a control blockot életben tartsa.

Esemény	Hatás a strong countra	Hatás a weak countra	Következmény
`shared_ptr` másolása	+1	változatlan	új tulajdonos

Esemény	Hatás a strong countra	Hatás a weak countra	Következmény
`shared_ptr` megszűnése	-1	változatlan	ha 0, az objektum megsemmisül
`weak_ptr` másolása	változatlan	+1	a control block életben marad
sikeres `weak_ptr::lock()`	+1	változatlan	új `shared_ptr` jön létre
utolsó `weak_ptr` megszűnése	változatlan	-1	ha a strong is 0, a control block felszabadul

A referenciaszámláló növelése és csökkentése atomikus, de ez csak a számlálót védi. Ugyanazon `shared_ptr` példány egyidejű módosítása továbbra is adatverseny, és a kezelt objektumot a `shared_ptr` nem teszi automatikusan szálbiztossá. Ha az objektumot több szál olvassa és írja, ahhoz külön szinkronizáció kell. Az objektum megsemmisítése abban a szálban történik, amelyik az utolsó erős hivatkozást elengedi; a többi szálnak ezt megfelelően szinkronizálnia kell.

21.4.2 `make_shared`, allokáció és weak tail

A `std::make_shared<T>(args...)` szokásos megvalósítása egyetlen allokációban hozza létre a control blockot és a `T` objektumot. A szabvány ezt ajánlott gyakorlatként írja elő („legfeljebb egy memóriafoglalást kell végeznie”, [util.smartptr.shared.create]), nem kötelező, ellenőrizhető darabszám-garanciaként; a kötelező szemantika csak az, hogy a visszaadott `shared_ptr` birtokolja az új objektumot. **Mérés** (globális `operator new` számláló, 64 bájtos típus): `make_shared` 1, `shared_ptr<T>(new T)` 2 allokáció, a GCC 13.3 libstdc++-ával és a Clang 22 libc++-ával is. Az együttes allokáció gyorsabb lehet, mint a külön `new T` és a külön control block-allokáció, és kivétel esetén is biztonságos: ha a `T` konstruktora dob, a már lefoglalt tároló felszabadul, és nem marad gazdátlan nyers pointer. A `make_shared` hátránya, hogy nem adható meg hozzá egyedi deleter, és a `T` konstruktorának elérhetőnek kell lennie.

Az együtt allokáló megvalósításban az objektum és a control block ugyanabban az allokációban van, és a következő levezetés erre a megvalósításra vonatkozik. Amikor az erős hivatkozások elfogynak, az objektum destruktora lefut, de a memória csak akkor szabadul fel, amikor az utolsó `weak_ptr` is megszűnik. Ezt nevezik weak tail élettartamnak. Ha egy nagy objektumra sokáig élő `weak_ptr` mutat, a nagy allokáció a látszólagos megsemmisítés után is foglalva maradhat. Ilyen esetben lehet tudatosan külön allokációt használni, de a legtöbbször a helyes megoldás a `weak_ptr` élettartamának rövidre szabása.

A `std::shared_ptr<T>(new T)` külön allokációt használ. Ez önmagában nem hiba, de két allokációt jelent, és a nyers `new` egy nagyobb kifejezésben gazdátlanul maradhat, ha a kiértékelés közben kivétel keletkezik. Ez a csapda **verziófüggő**: `f(std::shared_ptr<T>(new T), g())` C++14-ig szivároghatott (a `new T`, a `g()` és a `shared_ptr` konstruktora tetszőleges sorrendben kiértékelhető volt, és ha a `g()` dobott a `new T` után, de a `shared_ptr` építése előtt, az objektum elveszett), C++17-től az egy paraméter inicializálásának lépései között nem ékelődhet másik argumentum kiértékelése, így az előbbi példában a szivárgás már nem keletkezik; a nyers `new` tárolása más szerkezetekben (két külön lépés, vagy egy kifejezésben több nyers `new`) továbbra is veszélyes. Ezért a tulajdonjog átadását a lehető legközvetlenebbül, lehetőleg `make_shared`-del érdemes megtenni.

21.4.3 Nyers pointerből `shared_ptr` és a double-control-block hiba

A `std::shared_ptr<T>` nyers pointerből konstruálva átveszi a tulajdonjogot, és új control blockot hoz létre. Ha ugyanarra a nyers pointerre két külön `shared_ptr`-t építünk, két független control block keletkezik, amelyek ugyanazt az objektumot fogják felszabadítani:

```
// Tanítási példa: NE futtasd, nem definiált viselkedés.
auto raw = new Config{"localhost", 8080};
std::shared_ptr<Config> a(raw);
std::shared_ptr<Config> b(raw); // külön control block, dupla delete
```

A második konstruktor nem tud az első control blockjáról. A két control block külön számlálja a tulajdonosokat, és mindkettő azt hiszi, hogy ő szabadítja fel az objektumot. A következmény dupla felszabadítás, azaz nem definiált viselkedés. Ugyanez a hiba, ha egy meglévő `shared_ptr`-ből `p.get()`-tel készítünk újabbat, vagy ha egy tagfüggvényben `shared_ptr<T>(this)`-t írunk. A helyes megoldás a meglévő `shared_ptr` másolása vagy mozgatása, illetve az `enable_shared_from_this` használata.

21.4.4 Aliasing constructor és subobject lifetime

Az aliasing constructor egy meglévő `shared_ptr` control blockját használja, de a `get()` által visszaadott pointer egy másik címre mutat:

```
struct Config {
    std::string host;
    int port;
};

auto cfg = std::make_shared<Config>(Config{"localhost", 8080});
std::shared_ptr<const int> port_view(cfg, &cfg->port);
```

Itt a `port_view` erős tulajdonosa a teljes `Config` objektumnak, miközben `*port_view` a `port` tagra mutat. Amíg `port_view` él, a `Config` sem semmisülhet meg, tehát a `cfg.reset()` után is biztonságos lehet a `port_view` használata. Fontos: az aliasing `shared_ptr` nem non-owning view, hanem a teljes objektumot tulajdonló, de a subobjectre mutató fogantyú. A mutatott subobjectnek a tulajdonolt objektum élettartamán belül kell lennie. Ha a pointer olyan objektumra mutat, amelynek élettartama rövidebb, mint a tulajdonolt objektumé, a `shared_ptr` nem tudja megakadályozni a dangling állapotot.

Az aliasing constructor hasznos például akkor, ha egy nagyobb objektum egy tagját szeretnénk átadni egy API-nak, de a teljes objektum életben tartását is biztosítani akarjuk. Az `owner_before` összehasonlítás a control block tulajdonviszonya alapján rendez, nem a `get()` cím szerint; ez segít például asszociatív tárolóban.

21.4.5 `weak_ptr`, `lock` és referencia-ciklus

A `std::weak_ptr<T>` nem tulajdonos, hanem a control blockra mutató gyenge hivatkozás. Nem növeli a strong countot, ezért nem akadályozza meg az objektum megsemmisítését. A `lock()` metódus vagy egy érvényes `shared_ptr`-t ad vissza, amely biztosítja az objektum élettartamát a használat idejére, vagy üres `shared_ptr`-t, ha az objektum már megsemmisült. A `lock()` az egyetlen helyes „ellenőrzés és használat” minta; az `expired()` vagy a `use_count()` külön lekérdezése és utána a dereferálás versenyhelyzetet okozhat.

A leggyakoribb alkalmazás a referencia-ciklus megszakítása. Ha két objektum `shared_ptr`-rel kölcsönösen hivatkozik egymásra, egyik strong countja sem csökken nullára, ezért egyik sem semmisül meg. Az egyik irányt `weak_ptr`-re kell cserélni. Az alábbi példában a szülő erősen birtokolja a gyermeket, a gyermek viszont csak gyengén figyeli a szülőt:

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

struct Parent;

struct Child {
    std::string name;
    std::weak_ptr<Parent> parent; // nem tulajdonol
    ~Child() { std::cout << "Child " << name << " destroyed\n"; }
    void report() const;
};

struct Parent {
    std::string name;
    std::vector<std::shared_ptr<Child>> children; // tulajdonol
    ~Parent() { std::cout << "Parent " << name << " destroyed\n"; }
};

void Child::report() const {
    if (auto p = parent.lock()) {
        std::cout << name << " parent=" << p->name << "\n";
    } else {
        std::cout << name << " parent expired\n";
    }
}

int main() {
    auto p = std::make_shared<Parent>();
    p->name = "root";
    auto c = std::make_shared<Child>();
    c->name = "leaf";
    c->parent = p;                // weak hivatkozás
    p->children.push_back(c);     // erős tulajdonlás a szülő felől

    std::cout << "parent strong=" << p.use_count()
              << " child strong=" << c.use_count() << "\n";
    c->report();
    p.reset();                    // a szülő élettartama véget ér
    c->report();                  // a weak_ptr már expired
    std::cout << "done\n";
}
```

A laborban például ezzel a paranccsal fordítható:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o cycle_demo cycle_demo.cpp
```

A példában a `p` és a `c` a tulajdonosok, a `Child::parent` pedig nem tulajdonló view. A `p.reset()` invalidálja a szülő objektumot; a `weak_ptr::lock()` ezután üres `shared_ptr`-t ad, ezért a `report()` az „expired” ágat futja. Ha a gyermekben is `shared_ptr<Parent>` lenne, a két objektum kölcsönösen életben tartaná egymást, és a `p.reset()` után sem szabadulna fel egyik sem.

21.4.6 `enable_shared_from_this` inicializálási feltételei

Az `std::enable_shared_from_this<T>` egy bázisosztály-sablon, amely lehetővé teszi, hogy egy már `shared_ptr` által birtokolt objektum tagfüggvénye új `shared_ptr`-t szerezzen anélkül, hogy új control blockot hozna létre. A `shared_from_this()` egy meglévő ownerrel megosztott `shared_ptr`-t ad vissza. Feltételei: (1) a `T` **publikusan és egyértelműen** örököl az `enable_shared_from_this<T>`-ből (a sablonargumentum maga a származtatott osztály, pl. `struct X : public std::enable_shared_from_this<X>`; privát vagy többértelmű öröklésnél a bázis nem inicializálódik, és a `shared_from_this()` hibázik), (2) az objektumot már egy `std::shared_ptr<X>` tulajdonolja (a `shared_ptr`-nek az `X` vagy egy olyan típus bázisát kell kezelnie, amely hozzáfér a bázishoz), és (3) a `shared_from_this()`-t nem a konstruktorban/destruktorban hívjuk. C++17-től, ha nincs ilyen owner, a hívás `std::bad_weak_ptr` kivételt dob; korábbi szabványokban ez nem definiált viselkedés volt.

A bázis egy gyenge hivatkozást tárol, amely akkor inicializálódik, amikor egy `shared_ptr` átveszi az objektum tulajdonjogát. Ezért a konstruktorban még nincs érvényes owner: a `shared_from_this()` hívása a konstrukció közben `bad_weak_ptr`-hez vezet. Ugyanez a helyzet, ha az objektumot stacken vagy `unique_ptr`-rel hozták létre, mert nincs `shared_ptr` owner. Helyes minta: az objektumot `make_shared`-del létrehozzuk, majd egy külön `init()` lépésben, már létező owner mellett hívjuk a `shared_from_this()`-t. C++17-től a `weak_from_this()` is elérhető, ha nem akarunk azonnal erős hivatkozást szerezni.

21.4.7 Gyakori hibák

- Ugyanarra a nyers pointerre két `shared_ptr`-t építeni, például `shared_ptr<T>(p.get())`-tel. Ez double control blockot és dupla felszabadítást okoz.
- Egy tagfüggvényben `shared_ptr<T>(this)`-t írni ahelyett, hogy `enable_shared_from_this`-t használnánk.
- Két `shared_ptr`-rel kölcsönös hivatkozást létrehozni, majd csodálkozni, hogy a destruktorok nem futnak le. A ciklust `weak_ptr`-rel kell megszakítani.
- `weak_ptr` esetén `expired()`-et vagy `use_count()`-ot ellenőrizni, majd külön dereferálni. Helyette egyszer kell `lock()`-olni, és a kapott `shared_ptr`-t használni.
- Azt hinni, hogy a `shared_ptr` szálbiztossá teszi a kezelt objektumot. Csak a referenciaszámláló atomikus; az objektum hozzáféréséhez külön szinkronizáció kell.
- Aliasing `shared_ptr`-rel olyan subobjectre mutatni, amelynek élettartama rövidebb, mint a tulajdonolt objektumé. A `shared_ptr` a teljes objektumot tartja életben, nem a mutatott részt.
- `make_shared`-del létrehozott nagy objektumra sokáig életben tartott `weak_ptr`-t hagyni, mert a weak tail miatt a teljes allokáció foglalva marad.

21.4.8 Feladatok

1. feladat (megértés). Mi lesz a kimenet, és mennyi a `use_count()` az egyes pontokon?

```
auto a = std::make_shared<int>(42);
std::weak_ptr<int> w = a;
auto b = a;
a.reset();
std::cout << b.use_count() << " " << w.expired() << "\n";
auto c = w.lock();
std::cout << (c ? *c : -1) << "\n";
```

2. feladat (kódolás). Készíts egy `Subject`–`Observer` mintát, ahol a `Subject` `shared_ptr`-ekkel tárolja a megfigyelőket, az `Observer` pedig `weak_ptr`-rel hivatkozik a `Subject`-re. A `Subject` értesítéskor hívja meg az observereket, az observer a `lock()` segítségével olvassa a `Subject` állapotát. Biztosítsd, hogy a `Subject` megsemmisülése után az observer ne érjen érvénytelen objektumhoz.

3. feladat (diagnosztika és javítás). Az alábbi töredékek közül melyek hibásak, és miért? Javítsd őket.

(a) `auto p = std::make_shared<X>(); auto q = std::shared_ptr<X>(p.get());`

(b) `struct X : std::enable_shared_from_this<X> { X() { shared_from_this(); } };`

(c) `X x; auto p = x.shared_from_this();`

(d) `auto cfg = std::make_shared<Config>(); std::shared_ptr<int> v(cfg, &cfg->port); cfg.reset(); use(*v);`

21.4.9 Megoldások

1. megoldás. A `w` weak hivatkozás, ezért nem növeli a strong countot. `b = a` után a strong count 2. `a.reset()` után csak `b` tartja életben az objektumot, így `b.use_count()` értéke 1, és `w.expired()` hamis, azaz `0`. A `w.lock()` sikeres, új `shared_ptr`-t ad, amelynek hatására a strong count átmenetileg 2 lesz; a `*c` értéke 42. A kimenet tehát:

```
1 0
42
```

Az ellenőrzési feltétel: a `c` élettartama alatt az objektum nem semmisülhet meg, és a `c` megszűnése után a strong count visszaesik 1-re. Ha a `b`-t is elengeddük, a `w.expired()` igazzá válik.

2. megoldás. Az alábbi példában a `Subject` a tulajdonos, az `Observer` a `weak_ptr`-rel csak megfigyel. A ciklus azért szakad meg, mert az `Observer` nem tartja életben a `Subject`-et.

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

struct Subject;

struct Observer {
    std::string name;
    std::weak_ptr<Subject> subject;
    explicit Observer(std::string n) : name(std::move(n)) {}
    void on_event() const;
};
```

```
struct Subject {
    int value = 0;
    std::vector<std::shared_ptr<Observer>> observers;
    void notify() const {
        for (const auto& o : observers) {
            o->on_event();
        }
    }
};

void Observer::on_event() const {
    if (auto s = subject.lock()) {
        std::cout << name << " sees " << s->value << "\n";
    } else {
        std::cout << name << " subject gone\n";
    }
}

int main() {
    auto s = std::make_shared<Subject>();
    auto o = std::make_shared<Observer>("o1");
    o->subject = s;
    s->observers.push_back(o);
    s->value = 7;
    s->notify();
    s.reset();
    o->on_event();
}
```

A `Subject` erősen birtokolja az observereket, az `Observer` gyengén figyeli a `Subject`-et. A `s.reset()` után a `Subject` megsemmisül, mert a weak hivatkozás nem tartja életben; az `o` viszont a lokális `shared_ptr` miatt még él, és a `subject.lock()` üres lesz. Az ellenőrzési feltétel: a `Subject` destruktora lefut, az `Observer` nem ér el felszabadított `Subject`-et, és a ciklus megszűnik.

3. megoldás. (a) Hibás: `p.get()` átadása egy új `shared_ptr`-nek új control blockot hoz létre, miközben `p` már tulajdonol. Ez double control block és dupla felszabadítás veszélye. Javítás: `auto q = p;`.

(b) Hibás: a konstruktorban még nincs `shared_ptr` owner, ezért a `shared_from_this()` C++17-től `std::bad_weak_ptr`-t dob. Javítás: a konstruktor ne hívja; a `make_shared` után, külön `init()` lépésben hívjuk.

(c) Hibás: a stacken létrehozott `x`-et nem birtokolja `shared_ptr`, ezért a `shared_from_this()` szintén `bad_weak_ptr`-t dob. Javítás: `auto p = std::make_shared<X>(); auto q = p->shared_from_this();`.

(d) Helyes: az aliasing constructor a `v` számára a `cfg` control blockját használja, ezért a `cfg.reset()` után a `Config` életben marad, amíg `v` él. A `*v` a `port` tagra mutat, és a `Config` élettartama lefedi. Az ellenőrzési feltétel: `v.use_count()` a létrehozás után 2, majd a `cfg.reset()` után 1; a `v` megszűnésekor a `Config` felszabadul.

21.4.10 Összefoglaló

A `shared_ptr` megosztott tulajdonlást ad: az utolsó erős hivatkozás megszűnésekor determinisztikusan megsemmisíti a kezelt objektumot. A control block élettartama ettől külön válik: a weak hivatkozások életben tartják, ezért a `make_shared` allokációja a weak tail miatt tovább foglalhat. A nyers pointerből készített `shared_ptr` mindig új control blockot hoz létre, ezért ugyanarra a pointerre csak egy tulajdonosi láncot szabad építeni. Az aliasing constructor a teljes objektumot tartja életben, miközben egy subobjectre mutat. A `weak_ptr::lock()` a biztonságos ellenőrzés és használat eszköze, a referencia-ciklust pedig az egyik irány weak hivatkozásra cserélésével kell megszakítani. Az `enable_shared_from_this` csak akkor működik, ha az objektumnak már van `shared_ptr` ownerje; konstruktorban, stacken vagy `unique_ptr` mellett nem használható.

21.5 Owner, view és erőforrásszerződés C++ API-ban

A smart pointerek ismerete önmagában még nem elég ahhoz, hogy egy C++ API ne legyen élettartam-hibás. Ebben az alfejezetben azt vizsgáljuk meg, hogyan fejezi ki a függvény szignatúrája a valós tulajdonlási szándékot: ki birtokol, ki csak kölcsönöz, meddig él a hivatkozott objektum, és mikor lesz érvénytelen egy korábban kiadott nézet. A cél, hogy a paramétertípus önmagában is dokumentálja a szerződést, ne pedig kommentárban kelljen kitalálni. A C oldalról hozott fegyelem itt közvetlen folytatás: a Memóriamásolás, összehasonlítás és byte view (7.4) szerződéslogikája ugyanaz, csak típusba öntve.

21.5.1 Tulajdonlási szándék mint API-szerződés

Három szerepet különböztetünk meg. Az **owner** (tulajdonos) az az objektum, amelynek élettartamához az erőforrás élettartama kötődik; ez zárja le, szabadítja fel, vagy csökkenti a hivatkozásszámot. A **borrowed** (kölcsönzött) hozzáférés nem tulajdonol: a hívó felelőssége, hogy az objektum a hívás idejére érvényes legyen. A **view** (nézet) nem tulajdonló, több elemre kiterjedő hozzáférés, típusa szerződése szerint lehet kontiguis (pl. `std::span`) vagy szemantikus (pl. `std::string_view`).

A tulajdonlási invariáns: minden nyilvános függvény minden paramétere esetén a dokumentációnak és a típusnak egyértelműen meg kell mondania, hogy a függvény 1. átveszi a tulajdonjogot (*sink*), 2. csak kölcsönzi a hívás idejére, vagy 3. megosztott tulajdonba kerül (hosszú távú közös élettartam).

Ha a szignatúra nem fejezi ki ezt, a hívó kénytelen tippelni, és a hiba nem fordítási, hanem futásidejű dangling formában jelenik meg. Az alábbi táblázat a tipikus hozzárendelést mutatja; a konkrét típust mindig az indok dönti el, nem a táblázat önmagában.

Szándék	Ajánlott paraméter	Indok
Olvasás, nagy típus	`const T&`	nincs másolás, nincs tulajdonjog
Olvasás, kicsi triv másolható	`T`	olcsóbb, mint indirekció
Sink (a függvény megtartja)	`T` + `std::move`	egy mozgatással tagba tehető
Sink, egyedi erőforrás	`std::unique_ptr<T>`	explicit tulajdonjog-átadás
Kölcsönzés, nem lehet null	`T&`	referenciával garantált jelenlét
Kölcsönzés, lehet null	`T*`	null jelentés dokumentálandó
Nem tulajdonló tömb	`std::span<const T>`	hossz és tulajdonlás szétválik
Megosztott tulajdon	`std::shared_ptr<T>`	csak ha a függvény is osztozik

21.5.2 Érték szerinti átvétel, const& és sink

A `const T&` akkor helyes, ha a függvény csak olvas, és nem tárolja el a referenciát. A leggyakoribb hiba, hogy a szerző a `const std::string&` paramétert átveszi, majd a törzsben tagba másolja. Ekkor a hívó oldalon létrejön egy temporary, abból a törzsben egy másolat, és a temporary megsemmisül – három objektum, két másolás, egyik feleslegesen. A **sink paraméter** ezt egyetlen mozgatásra csökkenti: érték szerint vesszük át, majd `std::move`-val adjuk tovább.

```
#include <string>
#include <utility>

class Label {
    std::string text_;
public:
    // sink: átveszi és megtartja
    explicit Label(std::string text) : text_(std::move(text)) {}

    // csak olvas: nem tárolja, const& a helyes
    void print(const std::string& prefix) const;
};
```

Fordítás (GCC vagy Clang, nem futtatás):

```
g++ -std=c++20 -Wall -Wextra -Wpedantic example.cpp -c
```

Levezetés (költségvázlat). Érték szerinti sink esetén lvalue argumentummal a hívó oldalon egy másolás történik a paraméterbe, majd a tagba egy mozgatás; rvalue argumentummal nulla másolás és egy mozgatás. `const&` sink esetén lvalue argumentumnál egy másolás a tagba (a paraméter nem másol), rvalue argumentumnál a temporary létrehozása és a törzsben egy újabb másolás. A by-value sink tehát rvalue esetben vagy ugyanannyi, vagy kevesebb másolást végez, és ami fontosabb: a hívó kód szintjén látható, hogy az argumentum átadódik, nem csak „átnéz” a határon.

Kivétel: ha a típus drága és a függvény mégsem tárolja el, akkor `const&` a helyes; a sink nem ajánlás mindenre, hanem az átvétel kifejezője. C++20-tól a fordító sok esetben elkerüli a felesleges mozgatást, de ez nem teszi helyessé a rossz szándékú szignatúrát.

21.5.3 unique_ptr és shared_ptr: mikor melyik

A `std::unique_ptr<T>` érték szerinti átvétele a **tulajdonjog átadását** jelenti: a hívó tudja, hogy a mutató üres lesz a visszatérés után, és hogy a felelősség átszállt. A `const std::unique_ptr<T>&` ezzel szemben csak kölcsönöz, de a szándék homályos: miért van a pointer becsomagolva, ha nem adjuk át? Ilyenkor szinte mindig `T&` (nem null) vagy `T*` (null megengedett) a helyes. A `unique_ptr<T>&` csak akkor indokolt, ha a függvény a pointert magát módosítja (pl. `reset`-tel lecseréli).

```
#include <memory>

class Socket { public: int fd; };

class Server {
    std::unique_ptr<Socket> sock_;
public:
```

```
    // sink: a hívó tulajdonjogot ad át
    explicit Server(std::unique_ptr<Socket> s) : sock_(std::move(s)) {}

    // kölcsönzés, garantáltan nem null, nem tartja meg
    void send(Socket& s);

    // kölcsönzés, null is lehet
    void try_send(Socket* s);
};
```

A `std::shared_ptr<T>` érték szerinti átvétele akkor helyes, ha a függvény a pointert **el is tárolja vagy továbbadja** – vagyis valóban osztozik a tulajdonláson. A `const std::shared_ptr<T>&` önmagában nem biztosít hosszabb élettartamot: ha a függvény egy aszinkron callbackbe becsomagolja a referenciát, a shared_ptr visszaszámlálója a hívás után csökkenhet, és a callback a semmire mutat. A helyes megoldás a `shared_ptr` másolata a closure-be, vagy `weak_ptr`. A részletes control block és a `weak_ptr` viselkedés a shared_ptr, control block és weak_ptr (21.4) tárgyát képezi.

Fontos antipattern: `const std::shared_ptr<T>&` fogadása csupán azért, hogy „életben tartsuk” az objektumot. Ha a függvény nem osztozik, ez nem kölcsönzés, hanem hamis tulajdonosi látszat, és elrejti a valódi élettartam-szerződést.

21.5.4 span és string_view: nem tulajdonló előkép és invalidáció

A `std::span<const T>` (C++20) és a `std::string_view` nem tulajdonol. Tároló és mérethatár együtt kerül átadásra, de az érvényesség a hívó felelőssége. Előfeltétel: az alatta lévő elemek a span élettartama alatt érvényesek maradnak, és a `span` maga nem mutat egy már felszabadított puff erre. A `string_view` esetében ez ki is egészül azzal, hogy a mutatott karakterek nem módosulhatnak olyan művelettel, ami a view által látott tartományt érvényteleníti.

Az invalidáció forrásai konkrétan:

- **Tároló növelése**: `std::vector::push_back`, `insert`, `emplace_back`, `reserve`, `resize`, ha újrapufferelés történik. A `data()` pointer és minden korábban kiadott `span` érvénytelenné válik. Kapacitáson belüli `push_back` és `reserve(n) n <= capacity()` mellett nem pufferel újra.
- **Törlés**: `erase`, `clear`, `pop_back` a törölt elemre (és utána a vector méretére) mutató nézetet érvényteleníti; a vector elemei törlés után áthelyeződhetnek.
- **Tulajdonos reset**: `unique_ptr::reset`, `shared_ptr::reset`, `owner = nullptr` felszabadítja az erőforrást; minden korábban kiadott `T*`, `T&` és `span` dangling lesz.
- **Asszociatív tárolók**: `std::map` és `std::set` referencia- és iteratorstabilitása insert/erase alatt megmarad (a törölt elem kivételével); `std::unordered_map` rehash esetén minden iterátor érvénytelenné válik, az elemekre mutató referenciák és pointerek viszont érvényesek maradnak.

```
#include <cstdint>
#include <span>
#include <vector>

std::span<const std::uint8_t> loan(const std::vector<std::uint8_t>& buf) {
    return {buf.data(), buf.size()};
}
```

```
std::uint8_t wrong() {
    std::vector<std::uint8_t> buf(16, 0xEE);
    auto view = loan(buf);
    buf.reserve(4096);           // újrapufferelés: a view dangling lesz
    return view[0];              // UB: nem definiált viselkedés
}

std::uint8_t right() {
    std::vector<std::uint8_t> buf(16, 0xEE);
    auto v = buf[0];             // érték, nem nézet
    buf.reserve(4096);
    return v;                    // érvényes
}
```

A `wrong()` függvény tanítási példa: UB-t mutat, kimenete nincs garantálva, és a fordító optimalizálás mellett egészen mást is csinálhat. A `right()` változat helyes, mert értéket ment, nem nézetet. Ha a nézetre valóban szükség van a `reserve` után is, akkor a sorrendet kell megfordítani (először `reserve`, aztán nézet), vagy a nézetet a tartomány élettartamához kell kötni.

21.5.5 Aszinkron callback és a tulajdonos élettartama

Aszinkron callback esetén a callback élettartama jellemzően hosszabb, mint a regisztráló scope-é. Capture by reference (`[&]`) szinte mindig hiba, kivéve, ha a hívó dokumentáltan blokkol a callback befejezéséig. A tulajdonos megtartásához `shared_ptr` másolata kell a closure-be; a puszta megfigyeléshez `weak_ptr`.

```
#include <functional>
#include <memory>

class Worker : public std::enable_shared_from_this<Worker> {
public:
    using Callback = std::function<void()>;
    void post(Callback cb);

    // helyes: a callback a tulajdonost életben tartja
    void keep_alive() {
        auto self = shared_from_this();
        post([self] { /* self->... */ });
    }

    // helyes: a callback nem hosszabbítja meg, de észleli a megszűnést
    void observe() {
        std::weak_ptr<Worker> w = shared_from_this();
        post([w] {
            if (auto p = w.lock()) { /* p->... */ }
        });
    }
private:
    void run();
};
```

A `keep_alive` esetben a worker életben marad a callback lefutásáig, akkor is, ha a scope már kilépett. Az `observe` esetben a callback csak jelzi, hogy az objektum még él; ha nem, csendesen kilép. Mindkettő érvényes, de más szerződés: az első erőforrást tart fenn, a második hibát detektál. A szerződést a függvény nevének és dokumentációjának is tükröznie kell, nem csak a kódnak.

Az async capture nem menti meg attól, aki `this`-t capture-el `shared_ptr` nélkül: ez a klasszikus „raw this + async” hiba, és a hiba csak akkor látszik, ha a worker a callback előtt megszűnik.

21.5.6 Összefoglaló

A paramétertípus nem dekoráció, hanem a tulajdonlási szerződés része. Érték szerint venni csak akkor, ha a függvény megtartja (sink) vagy kicsi triv másolható; `const&` olvasásra, `T&`/`T*` kölcsönzésre; `unique_ptr` csak tulajdonjog-átadásra; `shared_ptr` csak tényleges megosztásra; `span`/`string_view` nem tulajdonló átadásra, a dokumentált érvényességi ablakkal. A view érvénytelenné válik, ha a mögötte álló tároló újrapufferelődik, törlődik, reset-elődik, vagy rehash-elődik. Az async callback élettartam-szerződése vagy `shared_ptr` megtartással, vagy `weak_ptr` megfigyeléssel oldható meg – a `[&]` capture csak indokolt kivétel lehet.

21.5.7 Feladatok

1. (magyarázó) Szignatúra-választás. Az alábbi függvényekhez válassz helyes paramétertípust, és egy mondatban indokold: (a) `Logger::write(std::string msg)` – a logger megtartja a szöveget, (b) `Hash::compute(const std::vector<uint8_t>& data)` – csak olvas, (c) `Socket::shutdown(std::unique_ptr<Connection> c)` – a socket lezárja és eldobja, (d) `Timer::schedule(std::function<void()> cb)` – a callback a timer megszűnése után is lefuthat.

2. (kódolási) Sink átírás. Az alábbi kód felesleges másolást végez. Írd át sink paraméterre, és indokold a mozgatások számát:

```
class Config {
    std::string name_;
public:
    explicit Config(const std::string& n) : name_(n) {}
};
```

3. (diagnosztikai) Dangling view. Az alábbi kódrészletben keresd meg a hibát, és add meg a javított változatot:

```
std::vector<int> buf{1,2,3};
std::span<int> s = buf;
buf.push_back(4);
int x = s[3];
```

4. (tervezés) Async observer. Adott egy `Session` osztály, amelyet egy hálózati réteg `shared_ptr`-rel tart életben. Írj egy `subscribe(callback)` metódust, ami a session bezárása után **nem** hívja meg a callbacket, de a session élettartama alatt igen.

21.5.8 Megoldások

1. megoldás. (a) `std::string` by-value + `std::move` a törzsben: a logger megtartja, ez sink. (b) `const std::vector<uint8_t>&`: csak olvas, a vektor élettartama a hívás idejére szükséges, és nem nullázandó; mérethez kötött hozzáféréshez `std::span<const uint8_t>` is jó, ha a hívó esetleg nyers puffert ad át. (c) `std::unique_ptr<Connection>` by value: a socket átveszi és eldobja, a tulajdonjog egyértelmű. (d) `std::function<void()>` by value: a timer megtartja; a callback élettartamát a `std::function` másolása biztosítja, de a capture-nek `weak_ptr`-t kell használnia, ha a callback a sessionre hivatkozik.

2. megoldás.

```
class Config {
    std::string name_;
public:
    explicit Config(std::string n) : name_(std::move(n)) {}
};
```

lvalue argumentummal: egy másolás a paraméterbe, egy mozgatás a tagba (a korábbi egy másolással szemben ez ugyanannyi, csak a temporary megszűnése tisztább). rvalue argumentummal: nulla másolás, egy mozgatás. Az eredeti `const&` változat rvalue esetén is másolt a törzsben, ezért a sink verzió jobb. Ellenőrzés: `static_assert(std::is_nothrow_move_constructible_v<std::string>)` és a fordító `-Wall -Wextra` mellett ne figyelmeztessen felesleges másolásra.

3. megoldás. A `push_back(4)` újrapufferelheti a vektort, mert a méret elérte a kapacitást; ilyenkor a `s.data()` dangling lesz, az `s[3]` UB. Javítás: a `push_back` előtt `reserve`, és csak utána kössük a nézetet, vagy elemenként olvassunk a vektorból:

```
std::vector<int> buf{1,2,3};
buf.reserve(8);
std::span<int> s = buf;
buf.push_back(4);
int x = s[3]; // érvényes, mert nem pufferelődött újra
```

Ellenőrzés: `buf.capacity() >= 4` a `push_back` előtt, és a `s.data() == buf.data()` a `push_back` után (ugyanaz a puffer).

4. megoldás. A session `shared_ptr`-rel tartott; a callbacket `weak_ptr`-rel kell megfigyelővé tenni, hogy a session megszűnése után a callback ne hívódjon meg:

```
class Session : public std::enable_shared_from_this<Session> {
public:
    void subscribe(std::function<void()> cb) {
        std::weak_ptr<Session> w = shared_from_this();
        callbacks_.push_back([w, cb = std::move(cb)] {
            if (auto p = w.lock()) { cb(); }
        });
    }
private:
    std::vector<std::function<void()>> callbacks_;
};
```

Az owner a `Session` (a `shared_ptr` tartja), a view a callback `weak_ptr`-je. Invalidáló esemény: a session `reset`-je vagy a `shared_ptr` grófszám nullára csökkenése; ekkor a `w.lock()` üres `shared_ptr`-t ad, a callback nem fut le. Ellenőrzés: a session megszűnése után a callback ne

legyen meghívva, és a callback ne tartsa életben a sessiont (`weak_ptr::use_count` a regisztráció után maradjon a korábbi értéken).

22 Értékszemantika, másolás és mozgatás

22.1 Rule of zero, three és five

A C-ben egy `struct` értékadása és paraméterátadása byte-onkénti másolás. Ha a struktúra mutatót tartalmaz egy dinamikusan allokált erőforrásra, a másolat ugyanarra a címre mutat: két „tulajdonos” jön létre ugyanarra az erőforrásra. A következmény vagy double free, vagy use-after-free. Ez a shallow copy (sekély másolás) probléma, amelyet a C típusrendszere nem tud kifejezni.

```
/* shallow_copy.c - szemléltető példa; futtatása nem definiált viselkedéshez
 ↪  vezet */
#include <stdlib.h>

struct buffer {
    char *data;
    size_t len;
};

int main(void) {
    struct buffer a;
    a.len = 4;
    a.data = malloc(a.len);
    if (a.data == NULL) return 1;

    struct buffer b = a;    /* sekély másolat: b.data == a.data */
    free(b.data);
    free(a.data);           /* double free: nem definiált viselkedés */
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic shallow_copy.c -o shallow_copy
# A példa szándékosan hibás; futtatni nem javasolt.
```

A C++ értékszemantikája nem tünteti el a problémát, hanem kifejezhetővé teszi: a special member functions (különleges tagfüggvények) adják meg, hogy egy típus hogyan másolható, mozgatható és semmisíthető meg. Ez az alfejezet a generálási szabályokat, a rule of three/five/zero döntési keretét és a copy/move policy megválasztását tárgyalja. A `std::move` mint kategóriakonverzió és a moved-from állapot részletes tárgyalása a Move constructor, moved-from állapot és std::move része; itt csak a special memberek szerepéig jutunk el. Az owner/view szerződésre építünk az Owner, view és erőforrásszerződés C++ API-ban, az invariáns fogalmára az Osztály, konstruktor és invariáns, a C értékmásolásra a Struktúrák, tagok és értékmásolás résznél. A szabályok forrása a forrásjegyzék C++20 tétele: [class.default.ctor], [class.copy.ctor], [class.copy.assign], [class.dtor] és [dcl.fct.def.delete]; a táblázatot alább egy próbaprogrammal ellenőrizzük.

22.1.1 A special member functions készlete

C++20-ban hat special member function van: default constructor, destructor, copy constructor, copy assignment operator, move constructor, move assignment operator. Ezeket a compiler akkor deklarálja implicit módon, ha a program nem deklarálja őket. A deklaráció azonban nem garantál definíciót: a compiler bizonyos esetekben „defined as deleted” (töröltként definiált) állapotba teszi őket, más esetekben egyáltalán nem deklarálja (suppressed, elnyomott).

Pontos definíció: egy special member function akkor „implicit módon deklarált”, ha a felhasználó nem deklarálta; akkor „defined as deleted”, ha a generált definíció nem lenne érvényes (például egy tag nem másolható); és akkor „nem deklarált”, ha egy másik user-declared special member ezt kizárja. A három állapot megkülönböztetése kulcs: a törölt függvényre történő hívás fordítási hibát ad, míg a nem deklarált függvény helyett a túlterhelés-feloldás másikat, jellemzően a copy-t választja.

A generálási szabályokat a következő táblázat foglalja össze. Jelölések: I = implicit módon deklarált és definiált, ha a tagok engedik; U = user-declared; D = defined as deleted; – = nem deklarált.

User-declared	default ctor	destructor	copy ctor	copy assign	move ctor	move assign
semmi	I	I	I	I	I	I
default ctor	U	I	I	I	I	I
destructor	I	U	I*	I*	–	–
copy ctor	–	I	U	I*	–	–
copy assign	I	I	I*	U	–	–
move ctor	–	I	D	D	U	–
move assign	I	I	D	D	–	U

A csillaggal jelölt esetben az implicit copy művelet generálása deprecated: az implicit copy constructoré, ha az osztálynak user-declared copy assignmentje vagy destruktora van; az implicit copy assignmenté, ha user-declared copy constructora vagy destruktora van. Ez nem tiltja meg a generálást, de a szabvány a jövőbeni eltávolítás lehetőségét fenntartja; új kódban ne támaszkodjunk rá.

A táblázat három mintázata. Először: a user-declared destructor, copy ctor vagy copy assign elnyomja a move operations deklarációját, ilyenkor a move helyett a copy hívódik, ha elérhető. Másodszor: a user-declared move ctor vagy move assign törli a copy operations-t, ez teszi lehetővé a move-only típust. Harmadszor: a default ctor csak akkor generált, ha nincs semmilyen user-declared konstruktor, ezért a copy ctor és a move ctor sorában is „–” áll; a copy assignment vagy a move assignment viszont nem konstruktor, mellettük a default ctor megmarad. A többi special member generálását a default ctor nem befolyásolja.

A táblázatot minimáltípusokkal ellenőrizhetjük. Minden típus pontosan egy special membert deklarál, egy naplózó `Probe` tagon keresztül pedig az is látszik, hogy rvalue argumentumnál move vagy copy fut, illetve törölt-e a művelet (`D`):

```
// special_members.cpp - C++20: a generálási táblázat ellenőrzése
#include <cstdio>
#include <type_traits>
#include <utility>

struct Probe {                      // naplózza, melyik műveletét hívták
```

```
    static inline const char* last = "";
    Probe() = default;
    Probe(const Probe&) { last = "copy"; }
    Probe(Probe&&) noexcept { last = "move"; }
    Probe& operator=(const Probe&) { last = "copy"; return *this; }
    Probe& operator=(Probe&&) noexcept { last = "move"; return *this; }
};

// Soronként egy user-declared special member; az int-es konstruktor csak
// a példányosításhoz kell, és a default ctoron kívül semmit nem befolyásol.
struct None       { Probe p; None() = default; explicit None(int) {} };
struct DefCtor    { Probe p; DefCtor() {} explicit DefCtor(int) {} };
struct Dtor       { Probe p; ~Dtor() {} };
struct CopyCtor   { Probe p; explicit CopyCtor(int) {}
                    CopyCtor(const CopyCtor& o) : p(o.p) {} };
struct CopyAssign { Probe p;
                    CopyAssign& operator=(const CopyAssign& o) { p = o.p; return
 ↪  *this; } };
struct MoveCtor   { Probe p; explicit MoveCtor(int) {}
                    MoveCtor(MoveCtor&& o) noexcept : p(std::move(o.p)) {} };
struct MoveAssign { Probe p;
                    MoveAssign& operator=(MoveAssign&& o) noexcept { p =
 ↪  std::move(o.p); return *this; } };

template <class T>
const char* via_move_ctor() {        // melyik konstruktor fut rvalue-ból?
    if constexpr (!std::is_move_constructible_v<T>) return "D";
    else {
        T a{1};
        Probe::last = "";
        T b(std::move(a));
        return Probe::last;
    }
}

template <class T>
const char* via_move_assign() {
    if constexpr (!std::is_move_assignable_v<T>) return "D";
    else {
        T a{1}, b{1};
        Probe::last = "";
        b = std::move(a);
        return Probe::last;
    }
}

template <class T>
void row(const char* name) {
    std::printf("%-11s default=%d copy_ctor=%d copy_assign=%d "
                "rvalue_ctor=%s rvalue_assign=%s\n", name,
                std::is_default_constructible_v<T>,
                std::is_copy_constructible_v<T>, std::is_copy_assignable_v<T>,
```

```
                via_move_ctor<T>(), via_move_assign<T>());
}

// Az int-es konstruktor nélküli típusokhoz aggregát/default példányosítás.
template <> const char* via_move_ctor<Dtor>() { Dtor a; Probe::last = ""; Dtor
 ↪  b(std::move(a)); return Probe::last; }
template <> const char* via_move_assign<Dtor>() { Dtor a, b; Probe::last = ""; b
 ↪  = std::move(a); return Probe::last; }
template <> const char* via_move_ctor<CopyAssign>() { CopyAssign a; Probe::last
 ↪  = ""; CopyAssign b(std::move(a)); return Probe::last; }
template <> const char* via_move_assign<CopyAssign>() { CopyAssign a, b;
 ↪  Probe::last = ""; b = std::move(a); return Probe::last; }
template <> const char* via_move_assign<MoveAssign>() { MoveAssign a, b;
 ↪  Probe::last = ""; b = std::move(a); return Probe::last; }

int main() {
    row<None>("semmi");
    row<DefCtor>("default");
    row<Dtor>("destructor");
    row<CopyCtor>("copy ctor");
    row<CopyAssign>("copy assign");
    row<MoveCtor>("move ctor");
    row<MoveAssign>("move assign");
}
```

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -Wdeprecated-copy-dtor \
    special_members.cpp -o special_members
./special_members
```

Buildjegyzőkönyv (`lab-gcc` profil). A kimenet soronként (`default`, `copy_ctor`, `copy_assign`, rvalue-konstrukció, rvalue-értékadás): `semmi` 1 1 1 move move; `default` 1 1 1 move move; `destructor` 1 1 1 copy copy; `copy ctor` 0 1 1 copy copy; `copy assign` 1 1 1 copy copy; `move ctor` 0 0 0 move D; `move assign` 1 0 0 D move. Ahol az rvalue-művelet `copy`, ott a move nincs deklarálva (–), és a túlterhelés-feloldás a copy-t választja; ahol `D`, ott a move nincs deklarálva, a copy pedig törölt. A fordító négy elavultsági figyelmeztetést ad, pontosan a csillaggal jelölt cellákra.

A `= default` és `= delete` szintaxis:

```
class X {
public:
    X() = default;                          // kérjük a generált default ctort
    X(const X&) = delete;                   // copy tiltása
    X& operator=(const X&) = delete;
    ~X() = default;
};
```

A `= default` akkor hasznos, ha a special member-t deklarálni akarjuk, például virtual destructor miatt, de a definíciót a compilerre bízzuk. A `= delete` bármely függvényre alkalmazható, de **nem** veszi ki a jelöltet a túlterhelés-feloldásból: a törölt függvény ugyanúgy jelölt, ugyanúgy rangsorolódik, és ha ő nyer, a program hibás ([dcl.fct.def.delete]). Ezért alkalmas tiltott konverziók kifejezésére: a törölt overload „elkapja” azt a hívást, amely különben egy nem kívánt konverzióval egy másik overloadhoz jutna. Egyetlen szabványos kivétel van: a defaulted, de töröltként definiált

move constructort és move assignmentet a túlterhelés-feloldás figyelmen kívül hagyja ([class.copy.ctor], [class.copy.assign]), így ilyenkor a copy-változat futhat. Ettől különbözik a constraint vagy SFINAE miatt ki sem választott jelölt, amely a túlterhelés-feloldásban részt sem vesz, így a hívás egy másik jelöltre eshet:

```
// deleted_overload.cpp - C++20: a deleted jelölt részt vesz a kiválasztásban
#include <concepts>
#include <cstdio>

void f(long) { std::puts("f(long)"); }
void f(int) = delete;              // jelölt marad; ha nyer, a hívás hibás

void g(double) { std::puts("g(double)"); }
template <std::integral T>
void g(T) { std::puts("g(integral)"); }   // nem integrálisra ki sem jelölt

int main() {
    f(1L);          // f(long): exact match, a deleted f(int) veszít
    // f(1);        // HIBA: az f(int) exact match nyer, de deleted
    g(1);           // g(integral)
    g(1.5);         // g(double): a sablon a constraint miatt nem életképes
}
```

Buildjegyzőkönyv (`lab-gcc` profil). A program `f(long)`, `g(integral)`, `g(double)` kimenettel fut; a kikommentezett `f(1)` visszakapcsolva `use of deleted function 'void f(int)'` fordítási hibát ad.

22.1.2 Rule of three és rule of five

A rule of three a C++11 előtti ökölszabály: ha egy osztálynak user-declared destructora, copy constructorja vagy copy assignment operatora van, akkor nagy valószínűséggel mindháromra szükség van. Az ok a fenti táblázatban látható: ha csak a destructort írjuk meg, a copy operations generáltak maradnak, és sekélyen másolnak. Ez pont a C-beli shallow copy hiba C++ változata.

A rule of five a C++11 utáni kiegészítés: ugyanez érvényes a move constructorra és a move assignment operatorra is. Ha az osztály erőforrást birtokol, és a move operations-t is definiáljuk, akkor öt függvényt kell összehangolni: destructor, copy ctor, copy assign, move ctor, move assign. A „rule of five” nem azt jelenti, hogy mindig mind az ötöt kézzel kell megírni; azt jelenti, hogy ha bármelyiket kézzel írjuk, végig kell gondolni a többit.

Nézzük a kézzel kezelt buffer rule-of-five változatát. A példa teljes, fordítható; a buildparancsot a blokk után adjuk.

```
// buffer_r5.cpp - rule of five, kézi erőforrás-kezelés
#include <cstddef>
#include <cstring>
#include <utility>

class Buffer {
public:
    explicit Buffer(std::size_t n = 0)
        : data_{n ? new char[n] : nullptr}, size_{n} {}
```

```
    ~Buffer() { delete[] data_; }

    Buffer(const Buffer& other)
        : data_{other.size_ ? new char[other.size_] : nullptr},
          size_{other.size_} {
        if (data_ != nullptr) {
            std::memcpy(data_, other.data_, size_);
        }
    }

    Buffer& operator=(const Buffer& other) {
        if (this != &other) {
            Buffer tmp{other};   // erős garancia: előbb elkészül az új érték
            swap(tmp);
        }
        return *this;
    }

    Buffer(Buffer&& other) noexcept
        : data_{other.data_}, size_{other.size_} {
        other.data_ = nullptr;
        other.size_ = 0;
    }

    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete[] data_;
            data_ = other.data_;
            size_ = other.size_;
            other.data_ = nullptr;
            other.size_ = 0;
        }
        return *this;
    }

    void swap(Buffer& other) noexcept {
        std::swap(data_, other.data_);
        std::swap(size_, other.size_);
    }

    std::size_t size() const noexcept { return size_; }
    const char* data() const noexcept { return data_; }

private:
    char* data_;
    std::size_t size_;
};
```

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -fsanitize=address,undefined
 ↪ buffer_r5.cpp -o buffer_r5
```

A copy assignment copy-and-swapot használ: az `other`-ből előbb elkészül egy teljes másolat (`tmp`), és csak akkor cseréljük ki a régi állapotot, ha ez sikerült. Így ha a másolás közben kivétel dobódik,

például a `new` sikertelen, a `*this` változatlan marad: ez az erős kivételgarancia (strong exception guarantee). A self-assignment (`b = b`) esetét a `this != &other` feltétel kezeli; copy-and-swap esetén ez elhagyható lenne, de a move assignmentnél kötelező, különben a saját erőforrásunkat szabadítanánk fel, majd vennénk át.

A move constructor és move assignment itt a tulajdonjog átadását végzi: a forrás pointerét átveszi, majd a forrást nullázza. A `std::move` önmagában csak kategóriakonverzió; a tényleges erőforrás-átadás a move operationsban történik. Ennek részletes tárgyalása a Move constructor, moved-from állapot és std::move részé.

22.1.3 Copy/move policy: másolható vagy move-only

Az erőforrás-tulajdonos osztály szerződése két alapvető irányt vehet. A másolható owner copy operationsa mély másolatot készít, move operationsa átadja a tulajdonjogot. Ezt akkor érdemes választani, ha az erőforrás másolása értelmes és megengedett, például egy buffer tartalma. A move-only owner copy operationsa törölt, move operationsa átadja a tulajdonjogot. Ezt akkor kell választani, ha az erőforrás egyedi, például file descriptor, socket, lock vagy egyedi allokáció, és két tulajdonos nem lehet.

A move-only szerződést a copy operations `= delete` jelölésével fejezzük ki. Az alábbi példa egy részlet: a `UniqueFd` osztály definíciója, amely POSIX fd-et birtokol.

```
#include <unistd.h>

class UniqueFd {
public:
    explicit UniqueFd(int fd = -1) noexcept : fd_{fd} {}
    ~UniqueFd() { if (fd_ >= 0) ::close(fd_); }

    UniqueFd(const UniqueFd&) = delete;
    UniqueFd& operator=(const UniqueFd&) = delete;

    UniqueFd(UniqueFd&& other) noexcept : fd_{other.fd_} {
        other.fd_ = -1;
    }
    UniqueFd& operator=(UniqueFd&& other) noexcept {
        if (this != &other) {
            if (fd_ >= 0) ::close(fd_);
            fd_ = other.fd_;
            other.fd_ = -1;
        }
        return *this;
    }

    int get() const noexcept { return fd_; }

private:
    int fd_;
};
```

Itt a copy törlése nem stílus kérdése: ha engednénk, két `UniqueFd` ugyanazt az fd-t zárná le, ami double close-t és más fd-ek véletlen lezárását okozhatja. A move-only szerződés a tulajdonjog egyediségét fejezi ki a típusban.

A self-assignment és a részleges hiba kezelése a move assignmentnél is kritikus. Ha `this == &other`, a régi fd lezárása után ugyanazt az fd-t vennénk át, ami már érvénytelen. Ezért kell a `this != &other` ellenőrzés. A copy assignmentnél a copy-and-swap adja az erős garanciát; a move assignmentnél a „delete then take” sorrend általában részleges hibát okozhat, ha a felszabadítás után az átvétel kivételt dobna. Mivel itt csak fd-t zárunk és int-et másolunk, ez nem fordulhat elő, de általános erőforrásnál a move assignmentet is érdemes úgy megírni, hogy előbb átvegyük az új erőforrást, és csak utána szabadítsuk fel a régit.

22.1.4 Rule of zero

A rule of zero a modern C++ irányvonal: ha lehetséges, egyik special member function-t se definiáljuk. Ezt akkor tehetjük meg, ha az osztály minden tagja maga kezeli az erőforrását, és a compiler generált copy/move operations helyesen működnek. A tagok lehetnek standard tárolók (`std::vector`, `std::string`), okos pointerek (`std::unique_ptr`, `std::shared_ptr`) vagy más RAII típusok.

A rule-of-zero Buffer:

```
// buffer_r0.cpp - rule of zero
#include <cstddef>
#include <vector>

class Buffer {
public:
    explicit Buffer(std::size_t n = 0) : data_(n) {}

    std::size_t size() const noexcept { return data_.size(); }
    char* data() noexcept { return data_.data(); }
    const char* data() const noexcept { return data_.data(); }

private:
    std::vector<char> data_;
};
```

```
g++ -std=c++20 -Wall -Wextra -Wpedantic buffer_r0.cpp -o buffer_r0
```

Itt nincs destructor, copy ctor, copy assignment, move ctor, move assignment. A `std::vector` destructora felszabadítja a tárolást, copy ctorja mély másolatot készít, move ctorja átadja a tárolást. A compiler generált special memberjei ezeket hívják, és helyesen működnek. A move-only változat ugyanígy működik `std::unique_ptr<char[]>` taggal: a `unique_ptr` nem másolható, ezért a `Buffer` copy operations implicit módon töröltek lesznek, a move operations pedig generáltak.

A rule of zero előnyei: kevesebb kézi kód és kevesebb hiba; a compiler generált műveletei optimalizálhatók, például trivial copy esetén; a tagok cseréje, például `vector` helyett `unique_ptr`, nem igényel kézi special member átírást; az invariáns a tagok invariánsaira épül, nem kell külön kézi erőforrás-kezelést összehangolni.

A rule of zero nem alkalmazható, ha az osztály invariánsa olyan, hogy a tagok másolása vagy mozgatása nem felel meg neki, például két tag konzisztenciáját kézzel kell fenntartani, vagy ha a tagok nem RAII-k. Ilyenkor a rule of three/five szerint kell eljárni.

A döntési sorrend: először vizsgáljuk meg, hogy minden tag tud-e önállóan viselkedni; ha igen, rule of zero. Ha nem, kell-e mély másolás; ha igen, rule of five, vagy rule of three, ha a move nem

indokolt. Ha az erőforrás egyedi, move-only szerződés: `= delete` copy, move operations. Minden esetben ellenőrizni kell a self-assignmentet és a kivételgaranciát.

22.1.5 Tipikus hibák

1. **Csak destructor, copy nélkül.** A copy operations generáltak maradnak, sekélyen másolnak; double free vagy use-after-free lesz a következmény. Javítás: rule of three/five vagy rule of zero.
2. **Rule of three hiányos.** Van destructor és copy ctor, de nincs copy assignment; az assignment sekély marad. Javítás: mindhárom, vagy copy-and-swap.
3. **Move operations mellett elfelejtett copy törlés.** Ha a move ctor user-declared, a copy operations automatikusan töröltek lesznek; ha ezt nem akarjuk, explicit `= default`-tal vissza kell hozni őket, ha a tagok engedik. Fordítva: ha copy-t írunk, a move-ok elnyomottak lesznek, és a move helyett copy hívódik.
4. **Self-assignment elfelejtése.** `a = a` esetén a move assignment felszabadíthatja a saját erőforrását, majd érvénytelen pointert vesz át. Javítás: `this != &other` vagy copy-and-swap.
5. **`= default` nem másolható taggal.** Ha a tag nem másolható, a `= default` copy ctor törölt lesz, nem generált; a hibaüzenet gyakran a hívás helyén jelenik meg. Javítás: a tag cseréje vagy a copy tiltása.
6. **Moved-from objektum használata.** A move után a forrás érvényes, de nem specifikált állapotú; csak destruktálható vagy új érték adható neki. Részletek: Move constructor, moved-from állapot és std::move.

22.1.6 Feladatok

1. **Magyarázó.** Adott:

   ```
   struct S {
       ~S();
       int x;
   };
   ```

   Mely special member functions generálódnak, törlődnek, illetve nem deklarálódnak? Másolható-e `S`, és mi történik a move kísérletnél?

2. **Kódolási.** Alakítsd át a `buffer_r5.cpp` `Buffer` osztályát rule-of-zero változattá `std::vector<char>` használatával. Indokold, miért marad helyes a copy és a move.

3. **Diagnosztikai.** Az alábbi osztály double free-t okoz. Miért, és hogyan javítod minimálisan?

   ```
   class Buf {
   public:
       explicit Buf(std::size_t n) : p_{new char[n]}, n_{n} {}
       ~Buf() { delete[] p_; }
   private:
       char* p_;
       std::size_t n_;
   };
   ```

4. **Tervezési.** Írj move-only RAII wrappert egy POSIX file descriptor köré. A copy operations legyenek töröltek, a move operations adják át a tulajdonjogot, a destructor zárja le az fd-t. Miért nem szabad a copy-t engedélyezni?

22.1.7 Megoldások

1. megoldás. A user-declared destructor miatt a move constructor és a move assignment nem deklarált, azaz elnyomott. A copy constructor és a copy assignment implicit módon generált; a szabvány ezt deprecatednek jelöli, de működik. A default constructor generált, mert nincs más user-declared constructor. Az `S` tehát másolható; a `std::move(s)` egy `S&&`-t ad, de mivel nincs move ctor, a copy ctor hívódik. Ellenőrzési feltétel: `std::is_copy_constructible_v<S>` igaz, és a move kísérlet is lefordul, de a copy ctor fut.

2. megoldás. A `Buffer` tagja `std::vector<char>` lesz; a constructor `data_(n)`, a `size()` és `data()` a vector megfelelő függvényeit hívja. Nem definiálunk destructort, copy/move operations-t. A vector copy ctorja mély másolatot készít, move ctorja átadja a tárolást; a compiler generált `Buffer` copy/move ezeket hívja. Ellenőrzési feltétel: a copy után a két buffer tartalma független; a move után a forrás `size()` értéke általában 0, de a vector moved-from állapota nem specifikált, csak érvényes.

3. megoldás. A `Buf`-nak csak destructora van, a copy ctor és copy assignment implicit generált, sekélyen másolja a `p_` pointert. `Buf b2 = b1;` után `b1.p_ == b2.p_`, és mindkét destructor `delete[]`-eli ugyanazt a tömböt: double free, azaz nem definiált viselkedés. Minimális javítások: a copy operations törlése `= delete` jelöléssel, ha a `Buf` move-only owner; rule of five szerinti mély copy ctor és copy assignment, valamint move ctor és move assignment; vagy rule of zero `std::vector<char>` vagy `std::unique_ptr<char[]>` taggal. Ellenőrzési feltétel: a javított változat ASan-nel futtatva nem jelez double free-t; a copy után a két objektum külön tárolóra mutat.

4. megoldás. A `UniqueFd` osztály a fenti példa szerint definiálható. A copy-t azért nem szabad engedélyezni, mert az fd egyedi OS-erőforrás: két `UniqueFd` ugyanazt az fd-t zárná le, ami double close-t, más fd-ek véletlen lezárását és nehezen diagnosztizálható hibákat okoz. A move-only szerződés a tulajdonjog egyediségét fejezi ki. Ellenőrzési feltétel: a copy hívása fordítási hibát ad; a move után a forrás `get()` értéke -1, a cél a régi fd-t birtokolja. A megoldás részlet, a teljes programhoz `main` és a megfelelő include-ok szükségesek.

22.1.8 Összefoglaló

A C shallow copy problémáját a C++ special member functions teszik kifejezhetővé. A compiler hat special member functiont deklarálhat: default ctor, destructor, copy ctor, copy assignment, move ctor, move assignment. Ezek lehetnek implicit módon definiáltak, töröltek vagy elnyomottak; a user-declared destructor vagy copy elnyomja a move-okat, a user-declared move törli a copy-okat. A rule of three a destructor–copy ctor–copy assignment hármast, a rule of five a move operations-t is összehangolja. A rule of zero a modern alapértelmezés: RAII tagokkal a compiler generált műveletei helyesek. Az owner szerződése lehet másolható, mély copy-jal, vagy move-only, `= delete` copy-jal. Minden kézi assignmentnél kezelni kell a self-assignmentet és a részleges hibát: a copy-and-swap erős garanciát ad, a move assignmentnél a `this != &other` ellenőrzés kötelező. A részletes move-szemantika és a moved-from állapot a Move constructor, moved-from állapot és std::move, a copy elision a Copy elision, RVO/NRVO és értékvisszaadás, a `noexcept` és a container-reallocation a noexcept, swap és container-reallocation témája.

22.2 Mély másolás, értéktípus és exception guarantee

A konfiguráció, a socketpuffer és a tanúsítványlánc közös tulajdonsága, hogy egy natív programban többször továbbadják, eltárolják és összehasonlítják őket. Ha a típus másolása csak a bájtok

vagy a mutatók átmásolását jelenti, két objektum fog ugyanarra az erőforrásra mutatni, és a felszabadításért mindkettő felelősnek hiszi magát. A C ezt a szerződést névvel és dokumentációval oldja meg (`buf_clone`, `buf_free`, lásd Tulajdonlási szerződés és cleanup ágak); a C++ lehetőséget ad arra, hogy a másolás jelentése magában a típusban legyen definiálva. Ez az alfejezet azt mutatja meg, mit kell egy ilyen típusnak ígérnie, hogyan teljesíthető ez az ígéret másolás közben fellépő hibák esetén is, és mikor jobb, ha a kézi másolás helyett standard tárolókra bízzuk a munkát. A mozgatás mint önálló téma a Move constructor, moved-from állapot és std::move részre, a swap és `noexcept` teljes elemzése a noexcept, swap és container-reallocation részre tartozik; itt csak annyit használunk belőlük, amennyi a másolási garanciához feltétlenül kell.

22.2.1 A shallow másolás öröksége és az értéktípus definíciója

A C nyelvű értékadás tagmásolás: ha egy struktúra pointert tartalmaz, a másolat ugyanazt a címet kapja. A két objektum megfigyelhető állapota látszólag független, valójában a mögöttes buffer közös. Ebből három tipikus hiba következik: a dupla felszabadítás, a felszabadított memória olvasása, és az a rejtett csatorna, amelyen az egyik példány módosítása a másikban is látszik. A hiba nem attól függ, hogy a program jól vagy rosszul használja-e az API-t: a típus ígérete hiányos.

Definíció (értéktípus). Legyen T egy osztály, és jelölje $\approx$ a megfigyelhető állapot egyenlőségét, azaz azt, hogy minden publikus, nem módosító művelet ugyanazt az eredményt adja a két objektumon. T akkor értéktípus, ha

1. másolható: létezik copy constructor és copy assignment, amelyek után az invariáns érvényes;
2. $\approx$ ekvivalenciareláció: reflexív, szimmetrikus és tranzitív;
3. bármely x és $y = \mathrm{copy}(x)$ esetén $x \approx y$, továbbá x és y bármely további módosítása után sem változik meg a másik megfigyelhető állapota.

A harmadik pont a függetlenségi feltétel, és pontosan ezt sérti meg a shallow másolás. Fontos, hogy a feltétel a megfigyelhető állapotról szól, nem a bájtokról: két objektum lehet bájtra különböző (például eltérő kapacitású, de azonos tartalmú buffer) és mégis ekvivalens.

Definíció (másolási invariáns). Legyen I predikátum az T objektumokon. I másolási invariáns, ha (a) minden konstruált objektumra igaz, (b) minden publikus művelet után igaz marad, és (c) minden másolási művelet eredményére is igaz. Az (a)–(b) a klasszikus invariáns, a (c) a másolásra vonatkozó többlet: a másolat nem kerülhet olyan állapotba, amelyet egyetlen konstruktor sem állítana elő.

A megosztott reprezentáció nem tiltott, csak korlátozott. Ha a megosztott rész **immutable**, akkor a megosztás megfigyelhetetlen, és a típus továbbra is értéktípus lehet; ez a `shared_ptr<const Blob>` és a copy-on-write mintája. Ha a megosztott rész **mutálható**, a függetlenség sérül, és a típus inkább referencia-szemantikájú, nem értéktípus. A döntést az alábbi táblázat foglalja össze.

Reprezentáció	Másolás költsége	Függetlenség	Fő kockázat
shallow mutató	$O(1)$	nincs	dupla felszabadítás, aliasing
mély másolat	$O(n)$	teljes	allokációs hiba, memóriaköltség
megosztott immutable (COW)	$O(1)$ + referenciaszám	megfigyelhetetlen	atomi referenciaszám, élettartam
megosztott mutálható	$O(1)$	nincs	rejtett csatorna a példányok között

22.2.2 Copy constructor és copy assignment

A két másolási művelet különbözik abban, milyen állapotból indul:

```
T(const T& other);            // copy constructor: új objektum
T& operator=(const T& other); // copy assignment: már létező objektum
```

A copy constructor mindig „tiszta lapon” dolgozik. Ha a konstruktor testében kivétel keletkezik, az objektum nem jön létre, a már létrehozott tagok destruktorai lefutnak (a részleges konstrukció szabálya, lásd Destruktor és részleges konstrukció). Ezért a copy constructor viszonylag könnyen ad erős garanciát. A copy assignment ezzel szemben egy már élő objektumot ír felül: előbb el kell dobnia a régi erőforrást, majd meg kell szerezte az újat, és a kettő között nem maradhat félkész állapot. A visszatérési érték azért referencia, hogy a hozzárendelés láncolható legyen, és hogy a beépített típusok szemantikájához illeszkedjen; a paraméter azért `const T&`, mert érték szerinti paraméterhez ismét másolás kellene, ami a copy constructort végtelen rekurzióba vinné.

22.2.3 Az exception guarantee előképe

A garanciák szókészlete három szintet különböztet meg; a részletes tárgyalás a No-throw, strong és basic exception guarantee részben következik.

- **Nothrow:** a művelet nem propagál kivételt.
- **Strong (commit-or-rollback):** ha a művelet kivétellel elbukik, a célobjektum megfigyelhető állapota megegyezik a hívás előttivel, és nem szivárgott erőforrás.
- **Basic:** hiba esetén nincs szivárgás, és az invariáns érvényes marad, de az objektum állapota megváltozhatott.
- **Nincs garancia:** az invariáns sérülhet, erőforrás veszhet el.

A basic guarantee is az invariánsra épül: nem azt ígéri, hogy minden változatlan, hanem azt, hogy az objektum használható és felszabadítható marad. A naiv „előbb törlöm, aztán allokálok” assignment még ezt sem teljesíti, mert hiba esetén a mutató a felszabadított bufferre mutat, miközben a méret nem nulla — az objektum destruktora dupla felszabadítást kísérelne meg.

22.2.4 Copy-and-swap

A klasszikus minta a nehéz esetet a könnyű esetre vezeti vissza: készítsünk teljes másolatot, majd cseréljünk egy nem dobó művelettel.

```
ByteBuffer& operator=(const ByteBuffer& other) {
    if (this != &other) {
        ByteBuffer tmp(other); // 1. teljes másolat, dobhat
        swap(tmp);             // 2. noexcept csere
    }
    return *this;
}                              // 3. tmp destruktora felszabadítja a régit
```

Levezetés (vázlat). Ha az 1. lépés kivételt dob, a `tmp` nem jön létre, `*this` érintetlen marad, tehát az erős garancia teljesül. Ha az 1. lépés sikeres, a 2. lépés `noexcept`, tehát nem hibázhat; a 3. lépés destruktor, amely nem dobhat. Így a művelet vagy teljesen végrehajtódik, vagy egyáltalán nem változtat a célon. A minta tehát akkor ad erős garanciát, ha a másolat elkészítése közben fellépő hiba nem érinti a célt, és a swap nem dob. Utóbbi feltétel nélkül a garancia elvész; erről részletesen a noexcept, swap és container-reallocation szól.

A költség oldala: a copy-and-swap egy allokációt, egy másolást és egy felszabadítást végez, ugyanannyit, mint egy hibamentes naiv assignment, de a hibaágban is konzisztens marad. Állapotfenntartó (stateful) allokátor esetén viszont a `tmp`-nek a **cél** allokátorával kell készülnie, különben a swap után olyan buffer kerül a célba, amelyet nem a saját allokátora szabadít fel. A `std::pmr` tárolóknál ez kézzel kezelendő, mert a `swap` nem cserél allokátort; a részleteket a pmr, memory_resource és region lifetime tárgyalja.

22.2.5 Self-aliasing és invalidáció

Két különböző problémát szokás összekeverni. Az **önhozzárendelés** (`x = x`) esetén a naiv implementáció előbb felszabadítja a buffert, majd a forrásból — amely ugyanaz az objektum — másolni próbál, így felszabadított memóriát olvas. A copy-and-swap ezt önmagában is kezeli: a `tmp` a swap előtt elkészül, a régi buffer csak a `tmp` destruktorában szabadul fel, amikor már senki nem hivatkozik rá. Az `if (this != &other)` csak optimalizálás, nem a helyesség feltétele.

Az **aliasing** ennél általánosabb: a hívó olyan referenciát vagy mutatót ad át, amely a célobjektum belsejére mutat. Az assignment után a cél korábbi tartalmára szerzett pointerek, iterátorok és nézetek érvénytelenné válnak, mert a régi buffer felszabadult. Ezt az API-szerződésben rögzíteni kell, ahogy azt az Owner, view és erőforrásszerződés C++ API-ban rész is hangsúlyozza. Fontos különbség a mozgatáshoz képest: a másolás a forrást nem változtatja meg, ezért a forrásra adott nézetek érvényesek maradnak; csak a cél korábbi nézetei halnak meg.

22.2.6 Rule-of-zero alternatíva standard tárolóval

Ha az osztály tagjai maguk is értéktípusok, a fordító által generált másolási műveletek tagonként mély másolatot készítenek, és a hibakezelést a tagok végzik. Ekkor a saját osztálynak nem kell destruktor, copy constructor és copy assignment — ez a rule of zero, amelyet a Rule of zero, three és five rész vezetett be.

```
struct Config {
    std::string name;
    std::vector<std::byte> payload;

    friend bool operator==(const Config&, const Config&) = default; // C++20
};
```

A `std::vector` másoló konstruktora a forrás elemeit másolja; ha egy elem másolása dob, a konstruktor nem fejeződik be, és a már létrehozott elemek destruktorai lefutnak, tehát nem marad félig kitöltött objektum. Az assignment garanciája viszont nem feltétel nélküli: függ az elem típusának másolási műveleteitől és az allokátor tulajdonságaitól, ezért a rule of zero nem azt jelenti, hogy minden művelet erős garanciát ad, hanem azt, hogy a garanciáért a tagok felelnek. A `std::string` belső reprezentációjára épülő nézetek (például `std::string_view`) élettartamára a std::string, std::string_view és szövegownership rész tér ki.

22.2.7 Kidolgozott példa és build

A példa két változatot mutat: a kézi `ByteBuffer`-t copy-and-swap assignmenttel, és a rule-of-zero `Config`-ot. A `ByteBuffer` invariánsa: `size_ == 0` esetén `data_ == nullptr`, egyébként `data_` egy `size_` bájtos, saját tulajdonú tömb.

```cpp
// byte_buffer.cpp - C++20, GCC/Clang, -std=c++20
#include <cstddef>
#include <cstring>
#include <string>
#include <utility>
#include <vector>
#include <iostream>

class ByteBuffer {
public:
    ByteBuffer() = default;
    explicit ByteBuffer(std::size_t n)
        : size_(n), data_(n ? new std::byte[n]() : nullptr) {}

    ByteBuffer(const ByteBuffer& other)
        : size_(other.size_),
          data_(other.size_ ? new std::byte[other.size_] : nullptr) {
        if (size_ != 0) std::memcpy(data_, other.data_, size_);
    }

    ByteBuffer& operator=(const ByteBuffer& other) {
        if (this != &other) {
            ByteBuffer tmp(other); // dobhat: strong guarantee
            swap(tmp);             // noexcept
        }
        return *this;
    }

    ~ByteBuffer() { delete[] data_; }

    void swap(ByteBuffer& other) noexcept {
        std::swap(size_, other.size_);
        std::swap(data_, other.data_);
    }

    std::size_t size() const noexcept { return size_; }
    const std::byte* data() const noexcept { return data_; }
    std::byte* data() noexcept { return data_; }

    friend bool operator==(const ByteBuffer& a, const ByteBuffer& b) noexcept {
        if (a.size_ != b.size_) return false;
        if (a.size_ == 0) return true;
        return std::memcmp(a.data_, b.data_, a.size_) == 0;
    }

private:
    std::size_t size_ = 0;
    std::byte* data_ = nullptr;
};

struct Config {
    std::string name;
```

```
    std::vector<std::byte> payload;

    friend bool operator==(const Config&, const Config&) = default;
};

int main() {
    ByteBuffer a(4);
    ByteBuffer b = a;  // copy constructor
    ByteBuffer c;
    c = a;             // copy assignment
    b = b;             // self-assignment
    std::cout << (a == b) << (a == c) << '\n';

    Config cfg1{"node-1", {std::byte{1}, std::byte{2}}};
    Config cfg2 = cfg1; // tagonkénti mély másolat
    cfg2.name = "node-2";
    std::cout << (cfg1 == cfg2) << '\n';
}
```

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -g -O2 \
    -fsanitize=address,undefined byte_buffer.cpp -o byte_buffer
./byte_buffer
```

A fordításhoz rögzített GCC vagy Clang kiadás szükséges; a `-fsanitize=address,undefined` a memóriahibák és az aláírt túlcsordulás detektálására szolgál. A példát a fejezet írásakor nem futtattam, a fenti parancs a tanuló laborjához tartozik; a kimenet illusztratív, nem mért eredmény. A hibás implementációt szándékosan külön, jelölt részletként mutatjuk be, mert nem része a buildnek:

```
// TANÍTÁSI PÉLDA – szándékosan hibás, ne használd
ByteBuffer& operator=(const ByteBuffer& other) {
    delete[] data_;                    // 1. régi erőforrás elhagyása
    size_ = other.size_;
    data_ = other.size_ ? new std::byte[other.size_] : nullptr; // 2. dobhat!
    if (data_) std::memcpy(data_, other.data_, other.size_);
    return *this;
}
```

Az 1. lépés után a `data_` a felszabadított bufferre mutat. Ha a 2. lépés `std::bad_alloc`-ot dob, az objektum `size_` értéke már az új méret, `data_` viszont a régi cím — az invariáns sérül, a destruktor dupla felszabadítást kísérel meg. Ha `this == &other`, a `memcpy` felszabadított memóriából olvas. Az implementáció tehát nemhogy erős, de még alapvető garanciát sem ad.

22.2.8 Gyakori hibák

A leggyakoribb tévedés a shallow tag (`const char*`, nyers `T*`) használata olyan osztályban, amelyet értéktípusként szeretnénk másolni. A második az önhozzárendelés kihagyása a naiv assignmentből, a harmadik a `swap noexcept` nélkül hagyása, ami a copy-and-swap erős garanciáját csendben megszünteti. Gyakori az is, hogy az `operator==` a reprezentációra (például a buffer kapacitására) hasonlít, nem a megfigyelhető állapotra, és így nem lesz ekvivalenciareláció. Végül az assignment `void` visszatérési értéke megakadályozza a láncolást, és eltér a beépített típusok szemantikájától.

22.2.9 Feladatok

1. **Magyarázó feladat.** Adott egy C API: `struct buf { char *p; size_t n; };` `buf_clone(const buf*)`, `buf_free(buf*)`. Mutasd meg, miért nem értéktípus a `struct buf`, és mit kell megváltoztatni a C++ portban.

2. **Kódolási feladat.** Egészítsd ki a `ByteBuffer` osztályt egy `assign_from(const std::byte* p, std::size_t n)` tagfüggvénnyel, amely erős garanciát ad, és önmtartalmazó híváskor (`b.assign_from(b.data(), b.size())`) sem olvas felszabadított memóriát. Indokold a lépéseket.

3. **Diagnosztikai feladat.** A fenti naiv `operator=`-ra vezesd le, milyen garanciát ad, és sorold fel a konkrét hibákat. Milyen sanitizer-jelzésre számítasz, és mit kellene a kódban megváltoztatni?

4. **Tervezési feladat.** Egy konfigurációt gyakran másolnak, de ritkán módosítanak. Válassz a mély másolat, a `shared_ptr<const T>` és a copy-on-write között, és indokold a döntést költség, élettartam és egyenlőség szempontjából.

22.2.10 Megoldások

1. megoldás. A `struct buf` nem értéktípus, mert a C értékadás és a paraméterátadás tagmásolást végez, így két `buf` ugyanarra a `p`-re mutat, és a függetlenségi feltétel sérül: az egyik `buf_free`-ja után a másik dangling pointert tart. A `buf_clone` és `buf_free` használata a hívó felelőssége, nem a típusé. A C++ portban a copy constructor és a copy assignment a `p` tartalmát új bufferbe másolja, a destruktor felszabadítja a saját buffert, és az `operator==` a `(n, tartalom)` párost hasonlítja. Ellenőrzési feltétel: két másolat módosítása után a másik tartalma nem változik, és a destruktorok sorrendje nem okoz dupla felszabadítást ASan alatt.

2. megoldás. Az `assign_from` először új buffert allokál és másol a bemenetről, majd `swap`-pel átveszi azt; a régi buffer csak a lokális másolat destruktorában szabadul fel. Ha a bemeneti pointer a saját bufferre mutat, a másolás még azelőtt megtörténik, hogy a régi buffer felszabadulna, tehát nincs use-after-free. Ha az allokáció dob, a cél érintetlen marad. Ellenőrzési feltétel: `b.assign_from(b.data(), b.size())` után `b` tartalma változatlan, és ASan/UBSan nem jelez.

3. megoldás. A naiv változat nem ad sem erős, sem alapvető garanciát. Hibák: (a) a `delete[]` után a `data_` dangling, miközben `size_` még a régi értéket tükrözheti; (b) ha a `new` dob, az invariáns sérül, a destruktor dupla felszabadítást kísérel meg; (c) önhozzárendelésnél a `memcpy` felszabadított memóriából olvas. ASan a (c) esetben heap-use-after-free-t, a (b) esetben double-free-t jelezhet. Javítás: copy-and-swap, azaz teljes másolat készítése a régi buffer érintése nélkül, majd `noexcept` swap.

4. megoldás. Ha a reprezentáció immutable, a `shared_ptr<const ConfigData>` olcsó másolatot ad $O(1)$ költséggel, és a megosztás megfigyelhetetlen, tehát az értékszemantika megmarad. A mutáció ekkor új reprezentációt hoz létre, ami a ritka módosítás miatt kedvező. A mély másolat egyszerűbb és kiszámíthatóbb, de minden másolás allokál. A copy-on-write a kettő kombinációja, viszont a referenciaszámot (jellemzően atomikusan) karban kell tartani, és a `shared_ptr` élettartamára, ciklusaira és a `weak_ptr` használatára is figyelni kell, ahogy a shared_ptr, control block és weak_ptr rész kifejti. Ellenőrzési feltétel: a választott modell mellett a másolat függetlensége megfigyelhető marad, és az `operator==` a tartalmat hasonlítja.

22.2.11 Összefoglaló

Az értéktípus ígérete nem a bájtok másolása, hanem a megfigyelhető függetlenség: a másolat egyenlő, de a további módosítások nem szivárognak át. A shallow másolás ezt sérti; a mély másolat teljesíti, cserébe allokációt és hibalehetőséget hoz. A copy constructor tiszta lapon dolgozik, ezért könnyebben ad erős garanciát; a copy assignmentnek a régi erőforrást is kezelnie kell. A copy-and-swap ezt a különbséget használja ki: a dobható részt a másolat készítésére korlátozza, majd egy `noexcept` cserével commitál. A naiv „törlés, majd allokálás” még alapvető garanciát sem ad, mert hiba esetén az invariáns sérül. A legtöbb esetben a rule-of-zero és a standard tárolók jelentik a legkevesebb kockázatú megoldást, de a garancia ilyenkor a tagoktól függ, nem automatikus. Az erős garancia pontos feltételei és a swap `noexcept` követelménye a No-throw, strong és basic exception guarantee, illetve a noexcept, swap és container-reallocation részben folytatódnak.

22.3 Move constructor, moved-from állapot és std::move

A C-ben megszokott `struct`-értékadás tagról tagra másol, ezért egy nyers pointert tartalmazó struktúra másolása csak a pointer értékét másolja: két objektum ugyanarra az allokációra fog mutatni, és a kétszeres `free` elkerülhetetlen. A C++ értékszemantikája ezt a problémát nem tünteti el, hanem lehetővé teszi, hogy a másolás és a mozgatás szerződését az osztály explicit módon szabályozza. Az előző részben rögzítettük a mély másolás és az exception guarantee alapjait; itt azt vizsgáljuk, mikor történik valóban erőforrás-átadás, és mit jelent a moved-from állapot. A `std::move` neve félrevezető: önmagában nem mozgat, hanem értékkategóriát változtat, hogy a túlterhelés-feloldás a move műveletet válassza.

22.3.1 Értékkategóriák, `T&&` és a `std::move` mint kategóriakonverzió

A C++ kifejezések értékkategóriája dönti el, melyik konstruktor vagy függvény lesz kiválasztva. Az **lvalue** olyan kifejezés, amely egy létező objektumot azonosít, és általában címet lehet venni vele. A **prvalue** ideiglenes értéket jelöl, amely még nem feltétlenül rendelkezik külön objektummal. Az **xvalue** olyan glvalue, amely egy objektumot jelöl, de jelzi, hogy annak erőforrásai újrahasznosíthatók; tipikusan ilyen a `std::move(e)` eredménye. A glvalue az lvalue és az xvalue közös halmaza, az rvalue pedig az xvalue és a prvalue közös halmaza. Ezek a kategóriák nem díszítések: a konstruktorválasztás és a függvénytúlterhelés ezeken alapul.

A nem template kontextusban használt `T&&` **rvalue referencia**. A `Buffer&&` típusú paraméter csak rvalue-hoz köthető, lvalue-hoz nem. Fontos azonban, hogy egy névvel rendelkező `Buffer&&` változó maga **lvalue kifejezés**: a neve miatt azonosítható, címet lehet venni vele, és a move konstruktor hívásához újra `std::move`-ra van szükség. A referencia típusa és a kifejezés értékkategóriája tehát nem ugyanaz.

A standard library `std::move` lényegében a következőhöz hasonló kategóriakonverzió:

```
template<class T>
constexpr std::remove_reference_t<T>&& move(T&& t) noexcept {
    return static_cast<std::remove_reference_t<T>&&>(t);
}
```

A `T&&` itt forwarding reference, amelynek részletes deduction szabályait a forwardingról szóló rész tárgyalja; most csak annyi kell, hogy a visszatérési típus mindig egy nem-referencia típusra vonatkozó `&&`, azaz rvalue referencia. A `std::move(a)` nem hoz létre új objektumot, nem szabadít

fel memóriát és nem másol adatot. Egy `static_cast`-hez hasonló jelölés, amelynek hatására a kifejezés xvalue lesz. A tényleges erőforrás-átadás attól függ, hogy erre az xvalue-ra melyik konstruktor vagy assignment illeszkedik, és hogy annak a törzse mit tesz.

22.3.1.1 Túlterhelés-feloldás lépésenként

Tekintsük a következő két konstruktort:

```
Buffer(const Buffer&); // másoló konstruktor
Buffer(Buffer&&);      // move konstruktor
```

A `Buffer b = std::move(a);` esetén a lépések:

1. `a` típusa `Buffer`, értékkategóriája lvalue.
2. `std::move(a)` visszatérési típusa `Buffer&&`, értékkategóriája xvalue.
3. Mindkét konstruktor jelölt: a `const Buffer&` és a `Buffer&&` is köthető az xvalue-hoz.
4. A `Buffer&&` pontosabb illeszkedés, mert nem kell const-képesítést hozzáadni; a move konstruktor lesz kiválasztva.
5. A move konstruktor törzse végzi el a tényleges erőforrás-átadást.

Ezzel szemben `const Buffer c; Buffer d = std::move(c);` esetén:

1. `std::move(c)` típusa `const Buffer&&`.
2. A `Buffer(Buffer&&)` nem köthető `const Buffer&&`-hoz, mert az const eldobását jelentené.
3. A `Buffer(const Buffer&)` köthető, ezért a másoló konstruktor fut.
4. A forrás változatlan marad, erőforrás-átadás nem történik.

Ez a gyakori hiba: a `const` objektumon használt `std::move` nem move-ot, hanem másolást eredményez. A `std::move` önmagában nem „mozgat”; csak akkor lesz move, ha a cél típusnak van megfelelő move művelete, és a túlterhelés-feloldás azt választja.

22.3.2 Move constructor és move assignment: a tényleges átadás helye

A **move konstruktor** `T(T&&)` alakú special member function, amely egy új objektumot inicializál a forrás erőforrásainak átvételével. A **move assignment** `T& operator=(T&&)` alakú, amely egy már létező objektumba veszi át az erőforrásokat. Mindkettőnek biztosítania kell, hogy a forrás utána érvényes, destruálható és assignolható állapotban maradjon. A saját típus moved-from invariánsát a típus szerződésében kell dokumentálni; a standard library számos típusa csak annyit garantál, hogy az állapot érvényes, de nem specifikált.

Az alábbi példa egy `Buffer` osztályt mutat, amelynek invariánsa: `size_ == 0` esetén `data_ == nullptr`, különben `data_` pontosan `size_` darab `std::byte`-ra mutat. A moved-from állapotot szándékosan üres buffernek definiáljuk.

```
#include <algorithm>
#include <cstddef>
#include <iostream>
#include <utility>

class Buffer {
public:
    Buffer() noexcept = default;
```

```
    explicit Buffer(std::size_t n)
        : data_{n ? new std::byte[n] : nullptr}, size_{n} {}

    Buffer(const Buffer& other)
        : data_{other.size_ ? new std::byte[other.size_] : nullptr},
          size_{other.size_} {
        if (size_ != 0) {
            std::copy_n(other.data_, size_, data_);
        }
    }

    Buffer(Buffer&& other) noexcept
        : data_{other.data_}, size_{other.size_} {
        other.data_ = nullptr;
        other.size_ = 0;
    }

    Buffer& operator=(const Buffer& other) {
        if (this != &other) {
            Buffer tmp{other};
            swap(tmp);
        }
        return *this;
    }

    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete[] data_;
            data_ = other.data_;
            size_ = other.size_;
            other.data_ = nullptr;
            other.size_ = 0;
        }
        return *this;
    }

    ~Buffer() { delete[] data_; }

    void swap(Buffer& other) noexcept {
        std::swap(data_, other.data_);
        std::swap(size_, other.size_);
    }

    std::size_t size() const noexcept { return size_; }
    std::byte* data() noexcept { return data_; }
    const std::byte* data() const noexcept { return data_; }

    // Moved-from invariáns: data_ == nullptr && size_ == 0.
    bool empty() const noexcept { return size_ == 0; }

private:
    std::byte* data_ = nullptr;
```

```
    std::size_t size_ = 0;
};

void consume(const Buffer&) { std::cout << "copy overload\n"; }
void consume(Buffer&&) { std::cout << "move overload\n"; }

Buffer make_buffer(std::size_t n) { return Buffer{n}; }

void sink(Buffer&& r) {
    Buffer copy = r;               // r neve miatt lvalue: másolás
    Buffer moved = std::move(r);   // xvalue: move
    (void)copy;
    (void)moved;
}

int main() {
    Buffer a{4};
    consume(a);                    // lvalue: const Buffer&
    consume(std::move(a));         // xvalue: Buffer&&

    Buffer b = std::move(a);       // move konstruktor
    std::cout << "a.size=" << a.size() << " b.size=" << b.size() << '\n';

    const Buffer c{4};
    Buffer d = std::move(c);       // const forrás: másolás
    std::cout << "c.size=" << c.size() << " d.size=" << d.size() << '\n';

    Buffer e{4};
    e = std::move(e);              // önmozgatás: a guard miatt változatlan
    std::cout << "e.size=" << e.size() << '\n';

    Buffer f = make_buffer(3);     // C++17 óta garantált elision; lásd 22.4
    std::cout << "f.size=" << f.size() << '\n';

    sink(Buffer{2});
}
```

A példa önálló fordítási egység. A laborban a rögzített GCC vagy Clang verzióval például így építhető:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 buffer_demo.cpp -o buffer_demo
```

A program logikája alapján az alábbi kimenet várható; ez illusztráció, nem mért eredmény:

```
copy overload
move overload
a.size=0 b.size=4
c.size=4 d.size=4
e.size=4
f.size=3
```

A `Buffer b = std::move(a);` esetében a move konstruktor törzse a pointert és a méretet átveszi, majd a forrást nullára állítja. A tényleges erőforrás-átadás tehát nem a `std::move` hívásban,

hanem a kiválasztott move konstruktor törzsében történik. A `sink` függvényben jól látszik, hogy a `Buffer&& r` paraméter neve lvalue: `Buffer copy = r;` másol, `Buffer moved = std::move(r);` mozgat. A move assignmentben a `this != &other` ellenőrzés az önmozgatás ellen véd; enélkül a `delete[] data_` után `other.data_` már a felszabadított saját bufferre mutatna, és a másolás nem definiált viselkedéshez vezetne.

A move konstruktort és a move assignmentet `noexcept`-ra érdemes tervezni, ha nem allokálnak. A `std::vector<Buffer>` újraallokációja ugyanis `move_if_noexcept` logikát használhat: ha a move konstruktor nem `noexcept`, és a másoló konstruktor elérhető, a tároló inkább másolhat. Ennek részletei a noexcept és container-reallocation részben következnek.

22.3.3 A moved-from állapot szerződése

A **moved-from állapot** nem azt jelenti, hogy az objektum megszűnt. Az objektum élettartama a move után is tart, a destruktora lefut, és assignmenttel újra használható. Azt jelenti, hogy az erőforrásai átkerültek, ezért az értéke csak annyiban használható, amennyit a típus szerződése garantál. A standard library általános megfogalmazása szerint a moved-from objektum **érvényes, de nem specifikált** állapotú: az osztály invariánsai teljesülnek, de az értékre nem lehet következtetni. Ez alól vannak kivételek; a `std::unique_ptr` például move után garantáltan null pointert tárol.

Típus / művelet	Moved-from garancia	Következmény
saját `Buffer`	`data() == nullptr`, `size() == 0`	bármely tagfüggvény biztonságosan hívható; a korábbi `data()` pointer csak az új tulajdonos élettartamáig érvényes
`std::unique_ptr<T>`	`get() == nullptr`	garantáltan üres; újra assignolható
`std::string`	érvényes, de nem specifikált	`size()` hívható, de az értéke nem hordoz szerződést; ne feltételezd, hogy üres
`std::optional<T>`	engaged marad, a benne lévő `T` moved-from	`has_value()` igaz lehet; a benne lévő értékre ugyanaz a moved-from szabály vonatkozik

A saját típusnál érdemes pontos szerződést adni, mert a hívó csak abból tud biztonságosan dolgozni. A `Buffer` esetében a move után az `empty()` igaz, a `data()` null, a `size()` nulla. Ez erősebb a „valid but unspecified” állapotnál, és lehetővé teszi a forrás újrahasználatát például így:

```
Buffer b{8};
Buffer c = std::move(b);
if (b.empty()) {
    b = Buffer{16}; // új erőforrás; a régi pointer nem használható b
 ↪  tartalmaként
}
```

Az **observer invalidáció** külön figyelmet kér. Ha a move előtt pointert, referenciát vagy nézetet vettünk a forrás erőforrására, az nem feltétlenül invalidálódik azonnal, de a tulajdonos megváltozik:

```
Buffer b{8};
std::byte* p = b.data();
Buffer c = std::move(b);
// p továbbra is az allokációra mutat, de az most c tulajdona.
// b.data() == nullptr; ha c megszűnik, p dangling lesz.
```

A pointer tehát nem attól lesz érvénytelen, hogy a `b` moved-from lett, hanem attól, hogy az allokáció élettartama már a `c`-hez kötődik. Ugyanez a helyzet a `std::string_view` és a `std::span` esetében: a nézet nem tulajdonol, ezért a mögöttes buffer tulajdonosának élettartamához kell kötni. Mozgatás után a forrás nézetére támaszkodni különösen veszélyes, mert a forrás moved-from állapota nem garantálja a régi buffer megtartását.

Az önmozgatás (`a = std::move(a);`) a saját típusnál kezelhető guarddal, ahogy a példában. A standard library tárolóinál az önmozgatás megengedett, de az eredmény jellemzően nem az eredeti érték; erre nem szabad építeni. A saját típus szerződésében azt is rögzíteni kell, hogy az önmozgatás után az objektum változatlan marad-e, vagy valamilyen dokumentált állapotba kerül. A legbiztonságosabb, ha a move assignment önmozgatás esetén érintetlenül hagyja az objektumot.

22.3.4 Tipikus hibák

- **A `std::move` önmagában mozgatna.** Nem: csak xvalue-vé alakít. Ha nincs megfelelő move művelet, vagy a forrás `const`, másolás történik.
- **Névvel rendelkező `T&&` paramétert move-ként használunk.** A paraméter neve lvalue; `std::move(r)` kell a move-hoz.
- **`const` objektumot mozgatunk.** A `const Buffer&&` nem köthető `Buffer&&`-hoz; a másoló konstruktor fut.
- **Move után a forrás értékére építünk.** A moved-from állapot érvényes, de nem specifikált; csak a dokumentált invariánsra szabad hagyatkozni.
- **Elfelejtett önmozgatás-védelem.** A move assignmentben a saját erőforrás felszabadítása után a forrás pointere dangling lehet, ha `this == &other`.
- **Nem `noexcept` move konstruktor.** A `std::vector` újraallokációja másolhat, ami teljesítményvesztés és erősebb követelmény a másolható típusra.
- **Observer élettartamának figyelmen kívül hagyása.** A move előtt szerzett pointer vagy view nem a moved-from objektumhoz, hanem az új tulajdonoshoz tartozik.

22.3.5 Összefoglaló

A `std::move` egy kategóriakonverzió: lvalue-ból vagy más kifejezésből xvalue-t csinál, hogy a túlterhelés-feloldás a move konstruktort vagy move assignmentet válassza. A tényleges erőforrás-átadás a kiválasztott move művelet törzsében történik; a `std::move` önmagában semmit nem mozgat. A move művelet után a forrás moved-from állapotba kerül: érvényes, destruálható és assignolható, de az értéke csak a típus szerződése szerint használható. A saját típusnál ezt az invariánst dokumentálni kell, a `const` forrás pedig gyakran csendben másoláshoz vezet. Az observerek, pointerek és nézetek élettartamát a move után az új tulajdonoshoz kell kötni.

22.3.6 Feladatok

1. **Magyarázó feladat.** Adott `const std::string s = "abcdefgh"; std::string t = std::move(s);`. Írd le lépésenként, melyik konstruktor fut, és miért nem történik erőforrás-átadás. Mit garantál a standard `s` értékére?

2. **Kódolási feladat.** Egészítsd ki a `Buffer` osztályt egy önmozgatás-biztos move assignmenttel, és dokumentáld a moved-from invariánst. Írj rövid ellenőrző kódot, amely move után ellenőrzi a forrás és a cél méretét.

3. **Diagnosztikai feladat.** Adott:

   ```
   std::string s = "abcdefgh";
   std::string_view sv = s;
   std::string t = std::move(s);
   std::cout << sv.size() << '\n';
   ```

   Mi a probléma a `sv` használatával? Milyen esetekben lehet mégis biztonságos, és hogyan kellene átírni a kódot?

22.3.7 Megoldások

22.3.7.1 1. megoldás

A `s` típusa `const std::string`, értékkategóriája lvalue. A `std::move(s)` visszatérési típusa `const std::string&&`, értékkategóriája xvalue. A `std::string` move konstruktorának paramétere `std::string&&`, amely nem köthető `const std::string&&`-hoz, mert az const eldobását jelentené. A másoló konstruktor `const std::string&` paramétere viszont köthető, ezért a másoló konstruktor fut. A `t` független másolat lesz, a `s` változatlan marad; erőforrás-átadás nem történik. A standard a `s` értékére nézve azt garantálja, hogy érvényes marad, de a `std::move` itt nem módosítja, mert nem move történt. Ellenőrzési feltétel: `t == s`, és a `s` hossza változatlan.

22.3.7.2 2. megoldás

Az önmozgatás-biztos move assignment például:

```
Buffer& operator=(Buffer&& other) noexcept {
    if (this != &other) {
        delete[] data_;
        data_ = std::exchange(other.data_, nullptr);
        size_ = std::exchange(other.size_, 0);
    }
    return *this;
}
```

A `std::exchange` a régi értéket visszaadja, és az új értéket beállítja; így a forrás pointere és mérete egy lépésben nullázódik. Az invariáns dokumentációja: „moved-from állapotban `data_ == nullptr` és `size_ == 0`; minden tagfüggvény hívható, a `data()` pointer csak az új tulajdonos élettartamáig érvényes.” Ellenőrző kód:

```
Buffer a{8};
Buffer b{4};
b = std::move(a);
assert(a.empty());
assert(a.size() == 0);
assert(b.size() == 8);
b = std::move(b); // önmozgatás
assert(b.size() == 8);
```

Az utolsó ellenőrzés a guard működését mutatja. A tényleges futtatás és az `assert` eredménye a tanuló laborjának része; itt a kód szerkezeti ellenőrzési feltételt ad.

22.3.7.3 3. megoldás

A `std::string_view sv` nem tulajdonolja a karaktereket, csak a `s` által kezelt bufferre mutat. A `std::string t = std::move(s);` után a `s` moved-from állapotú: érvényes, de nem specifikált. Kis string esetén a karakterek akár az `s` belső, SSO-pufferében maradhatnak, nagy string esetén a buffer átkerülhet a `t`-be. A standard nem garantálja, hogy a `sv` által mutatott buffer tovább él, és azt sem, hogy a `t` ugyanarra a bufferre mutat. Ezért a `sv` használata a move után dangling vagy nem specifikált tartalmú lehet.

Biztonságosabb a nézetet a move előtt használni, vagy a move után új nézetet készíteni a `t`-ből:

```
std::string s = "abcdefgh";
std::string t = std::move(s);
std::string_view sv = t; // a tulajdonos t, amíg t él
std::cout << sv.size() << '\n';
```

Ha a nézetre a move után is szükség van, a mögöttes tárolót hosszabb élettartamú objektumban kell tartani, vagy a nézetet a move előtt el kell dobni. Ellenőrzési feltétel: a `sv` élettartama soha nem haladja meg a mögöttes buffer tulajdonosának élettartamát.

22.4 Copy elision, RVO/NRVO és értékvisszaadás

A C-ben az érték-visszaadás *szemantikailag* értékátadás: a `return s;` egy `struct` esetén a hívó egy önálló értéket kap, tagonként azonos tartalommal (a padding bájtok értéke nem része az értékgaranciának). Hogy ezt a gép hogyan valósítja meg — regiszterekben, a hívó által adott rejtett célhelyre építéssel (az ABI-tól függően), vagy bájtmásolással —, azt a nyelv nem írja elő, és az optimalizáló is átalakíthatja; a fizikai megvalósítást külön ABI-példa mutatja. Ha a struktúra pointert tartalmaz, az értékátadás után a mutatott erőforrásból csak egy második, nem tulajdonló hivatkozás keletkezik. A C++ értékszemantikája ezt a másolást megőrzi szemantikai szinten — a `T` értékű visszatérés azt jelenti, hogy a hívó egy önálló, teljes értékű `T`-t kap —, de a fordítónak megengedi, hogy a köztes másolásokat elhagyja. Ez a fejezet arról szól, melyik elhagyás *garantált*, melyik csak *megengedett*, és hogyan lehet ezt a különbséget helyesen kihasználni, illetve hogyan lehet elrontani egyetlen felesleges `std::move`-val. Az előfeltétel a move konstruktor és a moved-from állapot ismerete: Move constructor, moved-from állapot és std::move (22.3).

A gyakorlati tétel nem kicsi. Egy konfigurációs objektumot, egy `std::vector<uint8_t>` puffert vagy egy `std::string`-et visszaadó függvény a program meleg útján lehet; a visszatérési érték költsége a naiv modellben három konstruktor/destruktor-hívás, a garantált elision mellett egy. A fejezet ezért két síkot választ szét végig: a *szabvány szerinti garanciát* (mi történhet meg egyáltalán) és a *megfigyelést* (mit látunk egy adott fordítónál, adott kapcsolókkal). A kettő nem ugyanaz, és a különbségtétel elmaradása a leggyakoribb félreértés forrása.

22.4.1 A visszatérési érték szemantikája: prvalue, elision, NRVO

22.4.1.1 Alapfogalmak és a C++17-es cezúra

A C++17 előtt a visszatérési érték előállítása három fogalmon keresztül írható le. A függvény előállít egy lokális objektumot, ez a lokális a `return` utasításnál egy *temporális* (névtelen ideiglenes)

objektummá másolódik, majd a temporális a hívó által biztosított célobjektummá másolódik (vagy C++11 óta mozog). A szabvány már C++17 előtt is megengedte, hogy a fordító ezeket a másolásokat elhagyja — ezt nevezi a szakirodalom **copy elision**nek (másolás-elhagyás) —, de csak *megengedte*, nem írta elő. Ráadásul a megengedés feltétele az volt, hogy a copy vagy move konstruktor elérhető legyen, még akkor is, ha a fordító végül nem hívta meg.

A **prvalue** (pure rvalue) C++17-től nem egy „névtelen objektum", hanem olyan kifejezés, amelynek értékelése egy objektumot inicializál. Amikor egy prvalue-t egy objektum inicializálására használunk, a prvalue *közvetlenül* inicializálja a célobjektumot; nincs köztes temporális, amit el kellene hagyni. Ez a **garantált prvalue-konstrukció** (mandatory vagy guaranteed copy elision), és nem optimalizáció: a program szemantikájának része. A C++17 előtti és utáni viselkedés különbsége éppen ezért nem teljesítménykérdés, hanem nyelvi kérdés — és bizonyos programok csak C++17-től fordulnak le.

22.4.1.2 A garantált prvalue-konstrukció feltételei

A garancia a `return` utasításban akkor áll fenn, ha a visszaadott kifejezés prvalue, és típusa megegyezik a függvény visszatérési típusával, a legfelső szintű (`top-level`) `const`/`volatile` minősítők eltérését leszámítva. Ilyenkor a prvalue közvetlenül a visszatérési objektumot inicializálja: nincs temporális, nincs copy, nincs move, és a copy/move konstruktornak *nem is kell léteznie*. Ez utóbbi a legfontosabb következmény, mert olyan típust is vissza lehet adni érték szerint, amely nem másolható és nem mozgatható.

A tipikus alakok, amelyekre a garancia vonatkozik:

```
T make_direct()   { return T{1, 2}; }     // prvalue: garantált
T make_braced()   { return {1, 2}; }      // a visszatérési típusra célzott
 ↪ inicializálás: garantált
T make_factory()  { return make_direct(); } // a belső hívás prvalue-ja:
 ↪ garantált
```

Az utolsó sor fontos: a `make_factory` visszatérési értéke nem egy lokális változó, hanem egy függvényhívás, amelynek eredménye prvalue, ezért a láncolt visszatérés is garantáltan költségmentes. Ez a gyárak és a `make_unique`-szerű segédfüggvények alapja.

22.4.1.3 Az NRVO feltételei

Az **NRVO** (Named Return Value Optimization) ennél gyengébb: a visszaadott kifejezés egy automatikus tárolási idejű, nem `volatile`, nem függvényparaméter nevű lokális változó *neve*, és a változó típusa megegyezik a visszatérési típussal (a top-level cv eltérését itt is leszámítva). Ha a fordító él a lehetőséggel, a lokális objektumot közvetlenül a hívó által biztosított tárhelyen építi fel, így a visszatéréskor nincs sem másolás, sem mozgatás. Ez azonban *megengedett*, nem kötelező: a szabvány megengedi, hogy a fordító elhagyja a copy/move konstruktor hívását, de nem írja elő.

Két következményt érdemes rögzíteni. Egyrészt a szabvány a feltételeket a `return` utasítás kifejezésére mondja ki (nem `volatile`, függvényen belül deklarált automatikus objektum neve, nem paraméter, nem `catch`-paraméter, azonos cv-nélküli típus); nem követeli meg, hogy minden útvonal ugyanazt az objektumot adja vissza. Több, különböző lokális változót visszaadó függvényben az NRVO elvileg megengedett, de a fordítók a gyakorlatban gyakran nem élnek vele (a tárolót nem lehet egyetlen hívóoldali helyre leképezni), és ilyenkor move (vagy ha az nem elérhető, copy) történik. Másrészt mivel az NRVO nem garantált, a copy vagy move konstruktornak *elérhetőnek kell lennie* akkor is, ha a fordító végül nem hívja meg. Ez a látszólagos ellentmondás a C++17 utáni szabály: a prvalue-esetben nem kell konstruktor, a névvel történő visszatérésnél kell.

22.4.2 Az implicit move a return utasításban

22.4.2.1 A szabály és verziójelölése

Amikor egy függvény érték szerint ad vissza, és a `return` kifejezés egy automatikus tárolási idejű objektumot megnevező azonosító-kifejezés — **lokális változót vagy érték szerinti függvényparamétert** —, a kifejezést az overload resolution során úgy kell kezelni, mintha rvalue lenne, mintha a programozó `std::move`-ot írt volna. Ez az **implicit move** szabály. C++11 óta létezik, és a paraméterekre kezdettől vonatkozik; a C++14 (CWG1579) a konverziós esetekre, a C++20 (P1825) az rvalue-referencia változókra, a `throw`-ra és a `co_return`-ra terjesztette ki; a C++23 (P2266) egyszerűsítette: az ilyen név a `return`-ben eleve xvalue (lásd Érték, élettartam és kifejezések: auto(x), implicit move és range-for (40.5)).

Két jogosultságot kell szétválasztani:

Visszaadott név	NRVO lehetséges?	Implicit move?
lokális, nem `volatile` automatikus objektum, a visszatérési típussal azonos típus	igen (nem garantált)	igen
érték szerinti függvényparaméter	**nem** — a paramétert a hívó építi, nem a célhelyen	igen
`T&&` lokális rvalue-referencia (C++20 óta)	nem	igen
lvalue-referencia paraméter vagy tag (`T&`, `member_`)	nem	**nem** — itt `std::move` kell, ha mozgatni akarunk

Az érték szerinti paraméternél tehát a `return p;` mozgat, és a `std::move` felesleges; a referencia-paraméternél a `return r;` másolna (move-only típusnál nem is fordul):

```
// param_move.cpp - C++20: érték szerinti és referencia-paraméter visszaadása
#include <memory>
#include <utility>

std::unique_ptr<int> by_value(std::unique_ptr<int> p) {
    return p;                 // implicit move: fordul, nincs NRVO (paraméter)
}

std::unique_ptr<int> by_ref(std::unique_ptr<int>& r) {
    // return r;              // hiba: r lvalue-referencia, nincs implicit move
    return std::move(r);      // itt a std::move szükséges és helyes
}

int main() {
    auto a = by_value(std::make_unique<int>(1));
    auto src = std::make_unique<int>(2);
    auto b = by_ref(src);     // src kiürül: ez a hívó számára látható szerződés
    return (*a + *b == 3 && !src) ? 0 : 1;
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic param_move.cpp` figyelmeztetés nélkül fordul, és 0 kóddal fut. A kikommentezett

`return r;` visszakapcsolva C++20 és C++23 módban is `use of deleted function 'std::unique_ptr<...>::unique_ptr(const std::unique_ptr<...>&)'` hibát ad. A `by_value` törzsét `return std::move(p);`-re írva a GCC `-Wredundant-move` figyelmeztetést ad.

A gyakorlati haszon: ha az NRVO nem történik meg, a `return local;` kifejezés rvalue-ként vesz részt a konstruktorválasztásban — a programozó semmit nem írt, és az NRVO esélye megmaradt. Hogy ezután ténylegesen melyik konstruktor fut, azt az overload resolution dönti el a meglévő jelöltekből: **mozgatható típusnál** a mozgató konstruktor; **csak másolható típusnál** (felhasználói másoló konstruktor, implicit mozgató nélkül) a `const T&` másoló konstruktor az rvalue-t is elfogadja, tehát *másolás* történik; ha a mozgató konstruktor **törölt**, a választás mégis azt találja meg, és a program nem fordul le. Ugyanez igaz a `std::move`-ra: értékkategóriát változtat, mozgatást nem kényszerít ki.

```
// move_select.cpp - C++20: a return-beli rvalue-kezelés kiválaszt, nem
 ↪  kényszerít
#include <cstdio>

struct CopyOnly {                          // felhasználói másoló ctor: nincs
 ↪  implicit move ctor
    CopyOnly() = default;
    CopyOnly(const CopyOnly&) { std::puts("  CopyOnly: copy"); }
};
struct MoveOnly {
    MoveOnly() = default;
    MoveOnly(MoveOnly&&) noexcept { std::puts("  MoveOnly: move"); }
    MoveOnly(const MoveOnly&) = delete;
};
#ifdef DELETED_MOVE
struct DeletedMove {
    DeletedMove() = default;
    DeletedMove(const DeletedMove&) = default;
    DeletedMove(DeletedMove&&) = delete;   // törölt, de jobb jelölt
};
DeletedMove make_d() { DeletedMove x; return x; }   // hiba: a törölt move-ot
 ↪  választja
#endif

template <class T> T make(bool b) {
    T x, y;
    if (b) return x;                        // két különböző név: NRVO helyett
     ↪  implicit move
    return y;
}

int main() {
    std::puts("CopyOnly:"); (void)make<CopyOnly>(true);   // rvalue -> const&
     ↪  másoló ctor
    std::puts("MoveOnly:"); (void)make<MoveOnly>(true);   // mozgató ctor
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra` kimenete `CopyOnly: copy`, majd `MoveOnly: move` (a két külön névből visszaadó `make` miatt NRVO nincs). `-`

`DDELETED_MOVE` mellett: `use of deleted function 'DeletedMove::DeletedMove(DeletedMove&&)'`; ugyanezt adja `-std=c++17` alatt is.

22.4.2.2 Miért káros a `return std::move(local)`

A `std::move(local)` egy függvényhívás, amelynek eredménye xvalue — nem azonosító-kifejezés. Ezzel két dolgot rontunk el egyszerre. Egyrészt az NRVO feltétele megszűnik, mert a visszaadott kifejezés már nem a lokális változó neve; a fordító nem építheti a lokálist a célhelyre, hiszen a `std::move` egy külön objektumra mutató referenciát ad. Másrészt az implicit move szabályára sincs szükség, mert a kifejezés már eleve xvalue: a move konstruktor meg fog hívódni. Az eredmény egy olyan move művelet, amely NRVO esetén nulla lett volna.

Fontos árnyalás, hogy ne essünk a másik végletbe: a `std::move` nem *mindig* káros, csak akkor, ha az NRVO egyébként megtörtént volna. Ha a visszaadott kifejezés nem implicit módon mozgatható — például egy tag (`return std::move(member_);`) vagy egy lvalue-referencia paraméter (`return std::move(ref);`) —, akkor az implicit move szabály rá nem terjed ki, és a `std::move` hiányában copy történne. Ezekben az esetekben a `std::move` helyes és szükséges, ha a forrás kiürítése a szerződés része. Az érték szerinti paraméter viszont *nem* ilyen: ott a `return param;` már mozgat, a `std::move(param)` csak felesleges. A megkülönböztetés kulcsa tehát nem a szintaxis, hanem az, hogy a kifejezés milyen tárolási idővel és milyen névvel rendelkezik.

22.4.2.3 Levezetés: az NRVO költségmodellje

Jelölje C_{copy} és C_{move} a copy, illetve move konstruktor költségét, C_{dtor} a destruktorét, C_{init} pedig a lokális objektum felépítésének költségét. Egy `return local;` naiv, elision nélküli költsége:

$$C_{\text{naiv}} = C_{\text{init}} + C_{\text{move}} + C_{\text{dtor,local}} + C_{\text{dtor,result}}$$

NRVO esetén a lokális a célhelyen épül fel, ezért a move és a lokális destruktora elmarad:

$$C_{\text{NRVO}} = C_{\text{init}} + C_{\text{dtor,result}}$$

A `return std::move(local);` viszont kikényszeríti a move-ot, még akkor is, ha az NRVO egyébként lehetséges lett volna:

$$C_{\text{forced}} = C_{\text{init}} + C_{\text{move}} + C_{\text{dtor,local}} + C_{\text{dtor,result}}$$

A különbség pontosan $C_{\text{move}} + C_{\text{dtor,local}}$. Ez kis objektumnál elhanyagolható, nagy puffer vagy drága destruktor esetén nem az. A `std::vector` és `std::string` esetében a move általában olcsó (pointer-átadás), de a destruktor és a move konstruktor hívásának elhagyása továbbra is mérhető különbség lehet szoros ciklusban — és ami fontosabb, a szemantika szempontjából az NRVO megőrzi az objektum identitását, amit a move nem.

22.4.2.4 Elision és destruktor-élettartam

A copy elision nem csak konstruktorokat hagy el, hanem destruktorokat is, és ez megfigyelhető viselkedést jelent. Garantált prvalue-konstrukciónál a temporális soha nem jön létre, ezért a destruktora sem fut le. NRVO esetén a lokális objektum a hívó által biztosított helyen épül fel, így egyetlen destruktor fut le — a visszatérési érték élettartamának végén, a hívónál. Ha az elision nem történik meg, két destruktor fut le: a lokálisé a `return` után, a visszatérési értéké a hívónál.

Ebből az következik, hogy a konstruktorokba és destruktorokba írt oldalhatások (számlálók, logolás, erőforrás-felszabadítás, lockolás) *nem determinisztikusak* az opcionális elision esetén. A szabvány ezt kifejezetten megengedi: a copy elision az `as-if` szabály egyik dokumentált kivétele. A program helyességét ezért soha nem szabad arra építeni, hogy egy copy vagy move konstruktor pontosan hányszor hívódik meg — legfeljebb diagnosztikai célra szabad ezt megfigyelni, és akkor is jelezni kell, hogy a megfigyelés fordító-, verzió- és kapcsolófüggő. Ez a fejezet instrumentált példájának is a fő tanulsága.

22.4.3 Kidolgozott példa: instrumentált copy/move számláló

22.4.3.1 A program

Az alábbi program három visszatérési módot hasonlít össze: garantált prvalue-t, NRVO-jelöltet és kikényszerített move-ot. A számlálók statikusak, hogy a `main` a végén kiírhassa őket; a konstruktorok csak a statikus számlálókat módosítják; ez *megfigyelhető oldalhatás* (éppen ezt számoljuk), nem oldalhatásmentes — az elision-szabály szerint a fordító a számlálóváltozásokat is elhagyhatja, ezért a számlálók értéke fordítófüggő lenyomat, nem normatív kimenet. Nincs I/O vagy lock a konstruktorokban, hogy a példa ne függjön ezektől.

```
// counter.cpp
#include <cstdio>
#include <utility>

struct Tracked {
    static int constructed;
    static int copied;
    static int moved;
    static int destroyed;

    int id;

    explicit Tracked(int i) : id{i} { ++constructed; }
    Tracked(const Tracked& other) : id{other.id} { ++copied; }
    Tracked(Tracked&& other) noexcept : id{other.id} {
        ++moved;
        other.id = -1;              // moved-from: érvényes, de nem specifikált
 ↪  érték
    }
    ~Tracked() { ++destroyed; }
};

int Tracked::constructed = 0;
int Tracked::copied      = 0;
int Tracked::moved       = 0;
```

```
int Tracked::destroyed   = 0;

Tracked make_prvalue() {
    return Tracked{1};              // garantált prvalue-konstrukció
}

Tracked make_named() {
    Tracked local{2};
    return local;                   // NRVO-jelölt: megengedett, nem kötelező
}

Tracked make_named_forced_move() {
    Tracked local{3};
    return std::move(local);        // NRVO blokkolva; move kényszerítve
}

int main() {
    Tracked a = make_prvalue();
    std::printf("a: copy=%d move=%d\n", Tracked::copied, Tracked::moved);

    Tracked b = make_named();
    std::printf("b: copy=%d move=%d\n", Tracked::copied, Tracked::moved);

    Tracked c = make_named_forced_move();
    std::printf("c: copy=%d move=%d\n", Tracked::copied, Tracked::moved);

    std::printf("total: ctor=%d copy=%d move=%d dtor=%d\n",
                Tracked::constructed, Tracked::copied,
                Tracked::moved, Tracked::destroyed);
    return 0;                       // a, b, c destruktorai itt futnak le
}
```

22.4.3.2 Build és várható megfigyelés

A példa önállóan fordítható; a fordítót és a verziót a laborprofilban kell rögzíteni.

```
g++     -std=c++17 -O0 -Wall -Wextra -pedantic -o counter counter.cpp
clang++ -std=c++17 -O0 -Wall -Wextra -pedantic -o counter counter.cpp

# diagnosztikai változat: az opcionális elision kikapcsolása
 ↪  (GCC/Clang-specifikus)
g++     -std=c++17 -O0 -fno-elide-constructors -o counter_noelide counter.cpp
```

A `-fno-elide-constructors` nem hordozható kapcsoló, és a garantált prvalue-esetet nem érinti; kizárólag az NRVO viselkedésének szemléltetésére való. A várt megfigyelés — *nem mért eredmény*, mert a kód futtatása a tanuló laborjának feladata:

Konfiguráció (kumulatív számlálók)	`a` után copy/move	`b` után copy/move	`c` után copy/move
C++17, alapértelmezett	0 / 0	0 / 0 (NRVO)	0 / 1

Konfiguráció (kumulatív számlálók)	`a` után copy/move	`b` után copy/move	`c` után copy/move
C++17, `-fno-elide-constructors`	0 / 0	0 / 1	0 / 2
C++14, alapértelmezett	0 / 0	0 / 0	0 / 1
C++14, `-fno-elide-constructors`	0 / 2	0 / 4	0 / 6

A számlálók **kumulatívak** (a három hívás között nem nullázódnak), ezért a lépésenkénti többlet a szomszédos oszlopok különbsége. Az értékeket a fenti program GCC 13.3-mal, `-O0` mellett adta (a táblázat mért, nem becsült). A C++14 sorokban a `Tracked(Tracked&&)` elérhető, ezért a visszatérésnél és az inicializálásnál kiválasztott konstruktor mindig a **move** (az `rvalue` operandus miatt a túlterhelés-feloldás a move konstruktort preferálja; nem „tetszőleges copy vagy move” a választás), a copy konstruktor csak move hiányában jönne szóba. `-fno-elide-constructors` mellett a C++14 minden lépésben két move-ot hajt végre (lokális → visszatérési temporális → célobjektum), a C++17 a prvalue-ból történő inicializálást garantáltan elhagyja, így ott lépésenként legfeljebb egy move marad.

A táblázat értelmezése a lényeg. Az `a` sor minden C++17-es konfigurációban garantáltan 0/0, mert a `make_prvalue` visszatérése prvalue. A `b` sor azért változik, mert az NRVO opcionális: `-fno-elide-constructors` mellett a fordító move-ot kényszerít, alapértelmezésben a legtöbb fordító elvégzi az NRVO-t. A `c` sor mindenhol move-ot mutat, mert a `std::move(local)` kizárja az NRVO-t. C++14-ben az `a` sor sem garantált elision: a fordító elhagyhatja a move-ot, de a move konstruktornak elérhetőnek kell lennie; ha az elision elmarad, a kiválasztott művelet a move (a `-fno-elide-constructors` sor mutatja a két-két move-ot).

22.4.3.3 A megfigyelés korlátai

Három dolgot érdemes tudatosítani. Egyrészt a számlálók értéke nem a szabvány által előírt viselkedés, hanem az adott fordító döntésének lenyomata; más compiler, más verzió vagy más optimalizálási szint eltérő számot adhat. Másrészt az `-O0` nem jelenti azt, hogy az NRVO elmarad: a legtöbb fordító az NRVO-t szemantikai szinten, az optimalizálástól függetlenül végzi el, ezért a „debug buildben nincs elision” feltevés hamis. Harmadrészt a táblázat „várt” értékeit senki nem mérte meg a fejezet írásakor; a mérés a labor része, és az eredményt a fordító és a kapcsolók rögzítésével együtt kell dokumentálni. A `printf` beszúrása a konstruktorokba egyébként megváltoztathatja az optimalizáló döntéseit, ezért a valódi méréshez érdemes külön, oldalhatás nélküli változatot használni, és csak a számlálókat kiírni.

22.4.3.4 Fordítási hibát bemutató példa

A garancia és az NRVO közti különbség legélesebben nem mozgatható típusnál látszik:

```
struct Pinned {
    Pinned() = default;
    Pinned(const Pinned&) = delete;
    Pinned(Pinned&&) = delete;
};

Pinned make_pinned() {
```

```
    return Pinned{};            // C++17: OK, a copy/move ctor nem is kell
}

Pinned make_pinned_named() {
    Pinned p;
    return p;                   // C++17: ill-formed, mert a move ctor törölt,
                                // és az NRVO nem garantált
}
```

A `make_pinned_named` azért nem fordul le, mert az implicit move szabály szerint a visszatéréshez move konstruktorra lenne szükség, és a fordító nem köteles az NRVO-t elvégezni. C++14-ben mindkét függvény hibás, mert ott a garantált esetben is elérhetőnek kellett lennie a konstruktornak. Ez a példa nem javasolt mintaként szolgál, hanem annak illusztrálására, hogy a két elision-fajta feltételei és következményei eltérnek.

22.4.4 Tipikus hibák és javításuk

A leggyakoribb hiba a `return std::move(local);` használata lokális változóra. A javítás mechanikus: a `std::move` elhagyása, és a `return local;` írása. Ez megőrzi az NRVO esélyét, és ha az NRVO nem történik meg, az implicit move akkor is mozgatni fog. A második hiba ennek a fordítottja: a `return local;` mintától elvárni, hogy garantáltan nulla másolással járjon. A garancia csak a prvalue-esetre vonatkozik; NRVO-nál a helyes állítás az, hogy „a fordító valószínűleg elvégzi, de nem köteles rá”.

A harmadik hiba a konstruktorokba és destruktorokba írt oldalhatásokra építés: számláló, logolás, lockolás, erőforrás-felszabadítás a copy konstruktorban. Mivel az elision opcionális, ezek száma nem determinisztikus, és a program viselkedése fordítófüggő lesz. A javítás nem a RAII feladása: a copy/move konstruktorok és a destruktor úgy legyenek megírva, hogy a tulajdonlási és életciklus-**invariánsok minden megengedett elision-eset mellett teljesüljenek** (egy objektum pontosan egyszer szabadít fel egy erőforrást, a moved-from objektum destruktora ártalmatlan). Az explicit `close()` külön szerepet kap: a **hibajelzés** (a destruktor nem tud hibát visszaadni, a `close()` igen), nem azt pótolja, hogy az elision opcionális. Az NRVO opcionális volta önmagában nem ok kézi erőforrás-nyilvántartásra.

A negyedik hiba a `std::move` és a szeletelés összekeverése: ha a lokális típusa származtatott, a visszatérési típus pedig bázis, a `std::move` bázisra konvertál, és a származtatott rész elveszik. Ez a slicing problémája, és nem az elision kérdése; a részletes tárgyalás az öröklésről szóló részre tartozik. Végül az ötödik hiba a referenciát visszaadó függvény és az értéket visszaadó függvény összetévesztése: a `T&` visszatérésnél nincs elision, mert nincs mit inicializálni — ott a dangling kockázata a fő kérdés, ami a Move constructor, moved-from állapot és std::move (22.3) alfejezethez kapcsolódik.

22.4.5 Összefoglaló

A C++17 garantált prvalue-konstrukciója nyelvi szabály: ha a `return` kifejezés prvalue és típusa megegyezik a visszatérési típussal, a prvalue közvetlenül a visszatérési objektumot inicializálja, copy/move konstruktor nélkül, sőt azok hiányában is. Az NRVO ezzel szemben megengedett, de nem kötelező elhagyás, amelynek feltétele, hogy a visszaadott kifejezés egy automatikus tárolási idejű lokális változó neve legyen; ilyenkor a copy/move konstruktornak elérhetőnek kell lennie. Az implicit move szabály a `return local;` alakot rvalue-ként kezeli az overload resolution során, így NRVO hiányában is move történik — ezért a `return std::move(local);` felesleges és káros, mert kizárja az NRVO-t. A tagra vagy paraméterre alkalmazott `std::move` viszont indokolt

lehet, mert ezekre az implicit move nem terjed ki. Az elision destruktorokat is elhagy, ezért a konstruktorok és destruktorok oldalhatásaira építeni hibás; a megfigyelés pedig mindig fordító-, verzió- és kapcsolófüggő, amit a mérés dokumentálásakor rögzíteni kell.

22.4.6 Feladatok

1. feladat (magyarázó). Hasonlítsd össze a példaprogram `make_prvalue` és `make_named` függvényét! Melyik esetben garantált, hogy nem hívódik meg copy vagy move konstruktor, és melyik esetben csak valószínű? Fogalmazd meg, mi a pontos feltétel különbsége, és hogyan viselkedik a `make_prvalue` C++14-ben.

2. feladat (kódolási). Az alábbi függvény szándékosan rossz mintát mutat. Javítsd, indokold a javítást, és add meg, milyen esetben lenne mégis helyes a `std::move` használata.

```
Tracked build(int n) {
    Tracked local{n};
    return std::move(local);
}
```

3. feladat (diagnosztikai). Írj egy `Tracked pick(bool first)` függvényt, amely két különböző lokális változó közül ad vissza egyet a feltétel alapján. Egészítsd ki a számlálókat, add meg a build parancsot, és magyarázd el, miért nem alkalmazható itt NRVO, valamint hogy milyen copy/move számot várs. Milyen ellenőrzési feltétellel tudod eldönteni, hogy a megfigyelésed megfelel-e az elvárásnak?

4. feladat (elemző). A `Pinned` példában melyik függvény fordul le C++17-ben, és melyik nem? Indokold a választ az implicit move szabályával és az NRVO feltételével, majd mondd meg, mi a helyzet C++14-ben. Milyen minimális módosítással tehető mindkét függvény C++17-kompatibilissé?

22.4.7 Megoldások

1. megoldás. A `make_prvalue` visszatérési kifejezése `Tracked{1}`, azaz prvalue, amelynek típusa megegyezik a visszatérési típussal. C++17-ben ezért a prvalue közvetlenül inicializálja a visszatérési objektumot: nincs temporális, nincs copy/move, és a konstruktoroknak nem is kell létezniük. Ez garantált. A `make_named` visszatérési kifejezése a `local` név, azaz automatikus tárolási idejű lokális változó, ezért NRVO-jelölt: a fordító elhagyhatja a copy/move hívást, de nem köteles rá; ha nem éli vele a lehetőséget, az implicit move szabály miatt move történik. C++14-ben a `make_prvalue` esetében sincs garancia: a szabvány csak megengedi az elhagyást, és a copy/move konstruktornak elérhetőnek kell lennie. *Ellenőrzési feltétel:* a válasz akkor helyes, ha kimondja, hogy a garancia a prvalue-esethez kötött, és hogy C++14-ben ez a garancia nem létezik.

2. megoldás. A javítás: `return local;` — a `std::move` elhagyása. Ezzel az NRVO lehetősége megmarad, és ha a fordító nem éli vele, az implicit move akkor is mozgatni fogja a lokálist, mert a `return` kifejezés egy automatikus tárolási idejű objektumot megnevező azonosító-kifejezés. A `std::move` akkor lenne indokolt, ha a visszaadott kifejezés nem ilyen lokális változó: például egy tag (`return std::move(member_);`) vagy egy függvényparaméter (`return std::move(param);`) esetén, mert ezekre az implicit move szabály nem terjed ki, és `std::move` nélkül copy történne. *Ellenőrzési feltétel:* a javított függvény fordul, a `Tracked` számlálói a `pick`-től függetlenül azt mutatják, hogy a `build` visszatérésekor legfeljebb egy move történik, és az NRVO engedélyezett konfigurációban nulla.

3. megoldás. A függvény például:

```
Tracked pick(bool first) {
    Tracked a{10};
    Tracked b{20};
    if (first) return a;
    return b;
}
```

Itt mindkét `return` egy-egy lokális változó nevét adja vissza, de különböző objektumokét. Az NRVO feltétele az, hogy a visszaadott kifejezés egy konkrét lokális objektumot nevezzen meg; két különböző lokális esetén a fordító nem tudja mindkettőt ugyanabba a célhelybe építeni, ezért a gyakorlatban move történik (a nem kiválasztott lokális pedig a saját destruktorával lezárul). A várt megfigyelés: a kiválasztott ágon egy move, a másik ágon a lokális destruktora; a számlálók értéke tehát `move >= 1`. *Ellenőrzési feltétel:* a program lefut, a kiválasztott objektum `id` értéke megmarad (10 vagy 20), és a számlálók konzisztensek a konstruktorok és destruktorok számával; a megfigyelést a fordító és a kapcsolók rögzítésével együtt kell dokumentálni, mert az NRVO hiánya itt nem garantált, csak tipikus.

4. megoldás. C++17-ben a `make_pinned` lefordul, mert a `return Pinned{};` prvalue, és a garantált prvalue-konstrukcióhoz nem kell copy vagy move konstruktor. A `make_pinned_named` nem fordul le, mert a `return p;` NRVO-jelölt, és ha az NRVO nem történik meg, az implicit move szabály miatt a törölt move konstruktort kellene hívni — a program ill-formed. C++14-ben mindkét függvény hibás, mert a copy/move konstruktornak elérhetőnek kell lennie akkor is, ha a fordító elhagyja a hívást. A minimális javítás: a `Pinned` copy és move konstruktorának törlése helyett definiálása (vagy legalább a move konstruktor engedélyezése), illetve a névvel történő visszatérés elkerülése és a `return Pinned{};` alak használata. *Ellenőrzési feltétel:* a javított változat mindkét függvénye lefordul C++17-ben, és a `make_pinned` esetében a számlálók nulla copy/move-ot mutatnak.

22.4.8 Források és ellenőrzési korlátok

A fejezet állításai a C++17 ISO szabvány copy elisionre és implicit move-ra vonatkozó szabályain alapulnak, valamint a C++20-as kiterjesztés ismertetett körére. Forrásellenőrzési korlát: a konkrét szakaszhelyeket, a C++20-as kiterjesztés pontos hatókörét és a fordítók viselkedését a választott szabványban, illetve a fordító dokumentációjában kell ellenőrizni. Kiindulópontként a WG21 nyilvános oldala (https://www.open-std.org/jtc1/sc22/wg21/), a C++23 munkatervezet (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf) és a C++ Core Guidelines (https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines) szolgálhat; ezek a keresés kiindulópontjai, nem a fejezet állításainak bizonyítékai. A Core Guidelines ajánlás, nem normatív ISO-szöveg. A példák nincsenek lefuttatva; a táblázat várható értékeket tartalmaz, nem mért eredményeket.

22.5 noexcept, swap és container-reallocation

A tárolók és algoritmusok viselkedése nem csak a típus *műveleteitől* függ, hanem azok *kivételszerződésétől* is. Ez az a pont, ahol a C++ értékszemantika és a standard library garantái összekapcsolódnak: egy helyesen megírt move constructor nem csak gyorsabb, hanem egyben kivételbiztonsági állítás is, amelyet a `std::vector` növekedése közvetlenül kihasznál — vagy éppen figyelmen kívül hagy. Ebben a részben azt vesszük végig, hogyan lesz a `noexcept` vállalásból tárolóviselkedés, és miért nem mindegy, hogy ezt a vállalást valóban be tudjuk-e tartani.

Az előző részek eredményeire építünk: a mély másolás és az erős kivételgarancia szerződését a Mély másolás, értéktípus és exception guarantee rögzítette, a visszatérési érték optimalizálását pedig a Copy elision, RVO/NRVO és értékvisszaadás tárgyalta. Itt már feltesszük, hogy a move constructor és a move assignment cserélhető állapotot hagy maga után, nem magyarázzuk újra, mit jelent a mozgatás.

22.5.1 A noexcept specifier és a noexcept operator

Két különböző dolgot nevezünk `noexcept`-nak, és a kétféle jelentést érdemes élesen szétválasztani.

A **`noexcept` specifier** (kivételspecifikáció) a függvénydeklaráció része, és szerződést rögzít a függvény viselkedéséről: ha a függvényből kivétel szabadul ki, a program `std::terminate`-mel leáll. Fontos pontosítás: a specifier *nem* tiltja meg, hogy a függvény belsejében kivétel keletkezzen és elkapásra kerüljön; csak arról szól, mi történik, ha a kivétel a függvény határán kilép. A szabvány nem garantálja, hogy a `terminate` előtt a stack kigöngyölődik; az, hogy történik-e unwinding, implementációfüggő, tehát erre építeni nem lehet. C++17 óta a kivételspecifikáció a függvénytípus része: `noexcept` függvényre mutató pointer implicit módon átalakítható potenciálisan dobó függvényre mutató pointerré, visszafelé nem. Overload resolutionben viszont nem lehet `noexcept` szerint túlterhelni.

A **`noexcept` operator** ezzel szemben egy kifejezés: `noexcept(E)` egy fordítási idejű `bool`, amely akkor `true`, ha az `E` kiértékelése nem tartalmaz potenciálisan dobó műveletet. Az operandus nem értékelődik ki (unevaluated operand), tehát `E`-nek nem kell futásidőben végrehajthatónak lennie. Függvényhívás esetén az eredmény a hívott függvény deklarált kivételspecifikációjából következik: ha `f` `noexcept(false)`, akkor `noexcept(f())` hamis, akkor is, ha `f` a gyakorlatban soha nem dob.

Kérdés	`noexcept` specifier	`noexcept` operator
Hol jelenik meg	függvénydeklaráció része	kifejezés
Fordítási idejű `bool`?	nem, deklaráció	igen
Kiértékelődik az operandus?	nem értelmezhető így	nem
Megszegés következménye	`std::terminate`	nincs (nincs vállalás)
Alapértelmezés	`noexcept(false)`, potenciálisan dobó	–

A legfontosabb alapértelmezés: a *felhasználó által megírt* függvény kivételspecifikáció nélkül potenciálisan dobó. Ez alól kivétel a destruktor, amely implicit módon `noexcept(true)`, kivéve, ha valamely tag vagy bázis destruktora potenciálisan dob. A *defaulted* speciális tagfüggvények kivételspecifikációja viszont a tagokéból számítódik ki: egy defaulted move constructor `noexcept`, ha minden tag és bázis move-ja nem dobó.

Ebből következik a gyakorlati csapda, amelyre később visszatérünk: ha kézzel megírsz egy move constructort, és elfelejted a `noexcept`-ot, akkor az *potenciálisan dobóvá válik* — akkor is, ha a törzse valójában semmit nem dob. A vállalás hiánya önmagában is viselkedést vált ki a tárolókban.

22.5.2 Feltételes noexcept: a szerződés típusfüggő esete

Egy sablon nem tudja előre, hogy a paramétertípusa dob-e. Ilyenkor a specifier argumentuma egy konstans kifejezés lehet, és a szerződés a típusfüggvényre hárul:

```cpp
// box.cpp - ISO C++20
#include <type_traits>
#include <utility>

template <class T>
class Box {
public:
    Box() = default;
    explicit Box(const T& v) : value_(v) {}
    explicit Box(T&& v) : value_(std::move(v)) {}

    Box(const Box&) = default;
    // feltételes szerződés: pontosan akkor noexcept, ha T move-ja az
    Box(Box&& other) noexcept(std::is_nothrow_move_constructible_v<T>)
        : value_(std::move(other.value_)) {}

private:
    T value_{};
};

static_assert(std::is_nothrow_move_constructible_v<Box<int>>);
static_assert(std::is_nothrow_move_constructible_v<Box<Box<int>>>);   // a
 ↪ feltétel öröklődik

struct ThrowingMove {
    ThrowingMove() = default;
    ThrowingMove(ThrowingMove&&) {}            // nem noexcept
};
static_assert(!std::is_nothrow_move_constructible_v<Box<ThrowingMove>>);
```

Itt a `std::is_nothrow_move_constructible_v` trait csak *előkép*: annyit jelent, hogy a `T(T&&)` konstruktor deklaráltan nem dobó-e. A lényeg a mintázat: a wrapper ne vállaljon többet, mint amit a tagja tud. Ha a wrapper feltétel nélkül `noexcept`-ot írna, akkor egy dobó `T` esetén a saját szerződését szegné meg, és a hiba nem itt, hanem jóval később, `std::terminate` formájában jelentkezne.

Trait nélküli esetben ugyanez a `noexcept(noexcept(kifejezés))` idiómával írható fel: a belső operátor megmondja, hogy a tényleges művelet dob-e, a külső pedig ezt teszi szerződéssé.

22.5.3 move_if_noexcept és a vector újraallokációs garanciája

A `std::vector` növekedése a következőt jelenti: új tárterületet kell allokálni, az elemeket át kell vinni, majd a régit felszabadítani. Az erős garancia azt kívánja, hogy ha bármelyik lépés kivétellel megszakad, a vektor maradjon pontosan abban az állapotban, amelyben a művelet előtt volt.

A probléma a mozgatással van. A move művelet alapvetően *roncsoló*: a forrásobjektumot moved-from állapotba teszi. Ha az átvitel közben, a harmadik elem mozgatásakor kivétel keletkezik, akkor az első kettő forrása már moved-from, azaz az eredeti értékük elveszett. Ezért mozgatással az erős garancia általában nem teljesíthető.

A `std::move_if_noexcept(x)` (`<utility>`) pontosan ezt a dilemmát oldja fel: rvalue referenciát ad vissza, ha a típus move konstruktora `noexcept`, vagy ha a típus egyáltalán nem másolható (ilyenkor nincs választás); egyébként lvalue referenciát ad, azaz a másolást választja. A szabvány

magát a *garanciát* írja elő, nem a mechanizmust, de a megfigyelhető viselkedés a legtöbb implementációban (libstdc++, libc++) ezen az eszközön keresztül valósul meg: a `vector::reserve`, `push_back` és `emplace_back` újraallokációnál a nem dobó move-ot részesíti előnyben, egyébként pedig másol.

A vonatkozó szerződés tömören: ha az újraallokáció során kivétel keletkezik, akkor erős garancia érvényes, amennyiben `T` másolható, vagy `std::is_nothrow_move_constructible_v<T>` igaz; ha `T` nem másolható és a move-ja dob, akkor a hatás nem specifikált. Ezért a nem dobó move constructor nem esztétikai kérdés, hanem az erős garancia előfeltétele.

Nézzünk egy teljes, fordítható példát, amely méri a másolások és mozgatások számát:

```
// reallocation_demo.cpp - ISO C++20
#include <cstddef>
#include <cstdio>
#include <type_traits>
#include <utility>
#include <vector>

class Buffer {
public:
    Buffer() = default;
    explicit Buffer(std::size_t n)
        : data_(n ? new int[n]{} : nullptr), size_(n) {}

    Buffer(const Buffer& other)                         // mély másolás
        : data_(other.size_ ? new int[other.size_] : nullptr),
          size_(other.size_) {
        for (std::size_t i = 0; i < size_; ++i) data_[i] = other.data_[i];
        ++copies_;
    }

    Buffer(Buffer&& other) noexcept                     // nem dobó move
        : data_(other.data_), size_(other.size_) {
        other.data_ = nullptr;
        other.size_ = 0;
        ++moves_;
    }

    // copy-and-swap: a paraméter másolása a hívó kontextusában történik
    Buffer& operator=(Buffer other) {
        swap(*this, other);
        return *this;
    }

    ~Buffer() { delete[] data_; }

    friend void swap(Buffer& a, Buffer& b) noexcept {
        using std::swap;
        swap(a.data_, b.data_);
        swap(a.size_, b.size_);
    }
```

```
    static int copies() { return copies_; }
    static int moves()  { return moves_; }

private:
    int* data_ = nullptr;
    std::size_t size_ = 0;
    static inline int copies_ = 0;
    static inline int moves_  = 0;
};

static_assert(std::is_nothrow_move_constructible_v<Buffer>);
static_assert(std::is_nothrow_swappable_v<Buffer>);

int main() {
    std::vector<Buffer> v;
    v.reserve(1);
    v.emplace_back(0);        // helyben konstruál, nincs másolás/mozgatás

    const int c0 = Buffer::copies();
    const int m0 = Buffer::moves();
    v.reserve(2);             // egyetlen újraallokáció, egy elem átvitelével
    std::printf("copies=%d moves=%d\n",
                Buffer::copies() - c0, Buffer::moves() - m0);
}
```

Fordítás a laborprofilban rögzített fordítóval:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o reallocation_demo
 ↪  reallocation_demo.cpp
# vagy
clang++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o reallocation_demo
 ↪  reallocation_demo.cpp
```

A várt megfigyelés (illusztráció, nem mért eredmény) a `noexcept` move mellett `copies=0 moves=1`. Ha a move constructorból töröljük a `noexcept`-ot — ekkor a `static_assert` már fordításkor elbukik, ezért azt is törölni kell —, a várt kimenet `copies=1 moves=0`: a vektor a másolást választja. Két korlátot érdemes rögzíteni: a `reserve(1)` utáni kapacitás pontos értéke nem garantált, és a növekedési stratégia is implementációfüggő, ezért a kimenet csak a tényleges laborprofilban tekinthető ellenőrzöttnek.

22.5.4 Nem dobó swap és az ADL előkép

A `std::swap` C++11 óta move-alapú: egy temporárist készít move konstrukcióval, majd két move assignementet hajt végre. Következésképpen pontosan akkor `noexcept`, ha a `T` move konstruktora és move assignmentje nem dobó: `noexcept(std::is_nothrow_move_constructible_v<T> && std::is_nothrow_move_assignable_v<T>)`. A szabvány `std::swap` használatához a `MoveConstructible` és `MoveAssignable` követelményt írja elő.

Egyedi tulajdonlást hordozó típusnál ennél jobb megoldás a handle-ök cseréje. A fenti `Buffer::swap` nem allokál, nem másol, csak két pointert és két méretet cserél, ezért feltétel nélkül `noexcept` lehet — ez a „nonthrowing swap” lényege. A helyes hívási minta két lépésből áll:

```
using std::swap;    // alapértelmezett jelölt
swap(a, b);         // ADL: megtalálja a felhasználói swap-et is
```

A `std::swap` itt csak tartalék; ha a típus névterében (vagy friend függvényként) van saját `swap`, az argumentumfüggő névkeresés (ADL) azt választja. Ez az ADL-előkép: a teljes névfeloldási szabályrendszert később tárgyaljuk, itt elég annyit rögzíteni, hogy a „`using std::swap; swap(a,b);`” idióma nélkül a felhasználói `swap` nem feltétlenül találódik meg. A `std::sort`, a tárolók `swap` tagja és számos algoritmus ezen a szerződésen keresztül dolgozik; ha a swap dob, az elemek sorrendje nem specifikált, de erőforrás nem szabadulhat el — ez a basic guarantee.

22.5.5 A destruktorból kiszabaduló kivétel

A destruktor implicit `noexcept(true)`. Ha a destruktorból kivétel szabadul ki, `std::terminate` fut. Ez nem véletlen szigorítás: a destruktorok jellemzően stack unwinding közben futnak, és egy második, kifelé távozó kivétel mellett a programnak nincs értelmes folytatása. A tárolók szempontjából még élesebb a helyzet: a `vector` újraallokációja a régi elemek destruktorait hívja meg, és ha ez dob, a tároló belső invariánsa (méret, kapacitás, lefoglalt tárterület) már nem állítható helyre.

A gyakorlati minta ezért az, hogy a destruktor soha nem dob, a hibalehetőséget pedig vagy külön, hívható műveletbe tesszük (például `flush()` vagy `close()`, amely `error_code`-ot ad vissza), vagy a destruktoron belül elnyeljük:

```
~Session() {
    try {
        flush_best_effort();   // nem dob tovább
    } catch (...) {
        // naplózás vagy csendes elnyelés: a destruktorból nem szabadulhat ki
        ↪ kivétel
    }
}
```

A destruktor `noexcept`-ságát érdemes ki is írni, ha kézzel definiálod: így a szándék látható, és a fordító nem a tagokból következtet.

22.5.6 Költség, garancia és a valótlan noexcept ára

A `noexcept` költsége aszimmetrikus. A helyes vállalás előnye nem elsősorban a kódgenerálásban jelenik meg, hanem abban, hogy a standard library algoritmusai *választhatnak*: a `vector` növekedése mozgat, a `std::swap` nem dob, az erős garancia tartható. A hiányzó vállalás csendes teljesítményveszteség: a tároló másol, és erről semmi nem figyelmeztet.

A valótlan vállalás viszont nem teljesítmény-, hanem helyességkérdés: `std::terminate`. A tipikus hibaforrások: allokáció egy `noexcept` függvényben (`new` → `std::bad_alloc`), potenciálisan dobó hívás egy `noexcept` wrapperben, illetve feltétel nélküli `noexcept` sablonban, ahol a tagtípus dobhat. Ugyanebbe a körbe tartozik a `std::vector` elemeként tárolt típus destruktorának dobása is.

Helyzet	Következmény	Teendő
Move constructor `noexcept` nélkül	a vektor növekedéskor másol	írd ki a `noexcept`-ot, ha a törzs valóban nem dob

Helyzet	Következmény	Teendő
Move constructor hamis `noexcept`-tal	`std::terminate` dobáskor	csak akkor vállald, ha minden tag move-ja nem dobó
Destruktor dob	`std::terminate`	nyeld el, vagy külön `close()`/`flush()`
`noexcept` wrapper dobó hívással	`std::terminate`	feltételes `noexcept` vagy vedd le a vállalást
Swap allokál/másol	lassú, esetleg dobó	handle-csere, member `swap` + friend `swap`

Összefoglalva: a `noexcept` nem dekoráció, hanem a típus kivételszerződése, amelyre a tárolók és algoritmusok építenek. A helyes sorrend: először biztosítani, hogy a művelet valóban nem dob, aztán vállalni — feltételesen, ha a típusfüggő; és mindig gondoskodni arról, hogy a destruktor ne legyen a terminate forrása.

22.5.7 Feladatok

1. feladat (diagnosztika). Adott egy `Payload` osztály, amelynek van felhasználó által írt copy és move konstruktora, utóbbi `noexcept` nélkül. Magyarázd el, miért másol a `std::vector<Payload>` `reserve` hívásakor, és hogyan mutatnád ezt meg anélkül, hogy a programot lefuttatnád.

2. feladat (kódolás). Egészítsd ki a `Buffer` osztályt úgy, hogy megfeleljen a következő szerződéseknek: nem dobó move constructor, nem dobó move assignment, tagfüggvény `swap` és névtérszintű `swap`. Igazold `static_assert`-tel, hogy `std::is_nothrow_move_constructible_v<Buffer>` és `std::is_nothrow_swappable_v<Buffer>` igaz.

3. feladat (hibajavítás). Egy `Session` destruktora `noexcept`-ként deklarált, és meghívja a `flush()` tagfüggvényt, amely `std::bad_alloc`-ot dobhat. Mi történik hiba esetén, és hogyan alakítanád át a kódot, hogy a hiba ne vezessen `std::terminate`-hez, de a hiba ténye se vesszen el?

4. feladat (tervezés). Írj egy `Wrapper<T>` osztályt, amelynek move constructorának kivételspecifikációja feltételesen `noexcept`, és indokold, miért nem szabad feltétel nélkül `noexcept`-ot írni.

22.5.8 Megoldások

1. megoldás. A felhasználó által megírt move constructor `noexcept` nélkül potenciálisan dobó. A `reserve` erős garanciát ad, amelyet mozgatással nem tudna tartani: egy félbeszakadt átvitel után a forráselemek moved-from állapotban maradnának, és nem állítható helyre az eredeti tartalom. Ezért a `std::move_if_noexcept` logikája szerint, mivel a típus másolható és a move-ja potenciálisan dobó, a másolást választja. Ellenőrzés fordításidejű eszközökkel: `static_assert(!std::is_nothrow_move_constructible_v<Payload>);` és `static_assert(std::is_copy_constructible_v<Payload>);` — a két feltétel együtt pontosan azt a kombinációt adja, amelyben a másolás várható. A viselkedés megfigyeléséhez másolás- és mozgatásszámláló tagokat kell a típusba tenni; futás nélkül csak a fenti trait-állítások és a szabvány garanciaszabálya áll rendelkezésre.

2. megoldás. A kulcs, hogy a move műveletek csak handle-t cserélnek, ne allokáljanak:

```
Buffer(Buffer&& other) noexcept
    : data_(other.data_), size_(other.size_) {
    other.data_ = nullptr;
    other.size_ = 0;
}

Buffer& operator=(Buffer&& other) noexcept {
    if (this != &other) {
        delete[] data_;
        data_ = other.data_;
        size_ = other.size_;
        other.data_ = nullptr;
        other.size_ = 0;
    }
    return *this;
}

void swap(Buffer& other) noexcept {
    using std::swap;
    swap(data_, other.data_);
    swap(size_, other.size_);
}

friend void swap(Buffer& a, Buffer& b) noexcept { a.swap(b); }
```

Az önellenőrzés feltétele: `static_assert(std::is_nothrow_move_constructible_v<Buffer>);` és `static_assert(std::is_nothrow_swappable_v<Buffer>);` forduljon le, valamint a `~Buffer()` továbbra is `noexcept` maradjon. A move assignmentben az önértékadás-ellenőrzés kötelező, különben a `delete[] data_` után a saját pointerünket olvasnánk vissza.

3. megoldás. A destruktorból kiszabaduló kivétel `std::terminate`-hez vezet; a `noexcept` deklaráció ezt csak hangsúlyozza, a destruktor implicit módon amúgy is nem dobó. A javítás: a `flush()` maradjon meg hívható, hibát jelző műveletként (`error_code`-kal vagy kivétellel), a destruktor pedig csak best-effort tisztítást végezzen, kivételt elnyelve:

```
~Session() noexcept {
    try { flush(); }
    catch (...) { /* itt nem szabadulhat ki kivétel */ }
}
```

A hiba ténye ne vesszen el: a `flush()` explicit meghívása a normál vezérlési folyamatban történjen, ahol a hívó még tud reagálni. Ellenőrzési feltétel: a destruktor deklarációja `noexcept`, és a törzsében minden potenciálisan dobó hívás `try` blokkban van.

4. megoldás. A wrapper a tagja szerződését örökli:

```
template <class T>
class Wrapper {
    T value_{};
public:
    Wrapper(Wrapper&& other) noexcept(std::is_nothrow_move_constructible_v<T>)
        : value_(std::move(other.value_)) {}
};
```

Feltétel nélküli `noexcept` esetén egy dobó `T` mellett a wrapper a saját szerződését szegné meg, és a hiba `std::terminate` formájában, a keletkezés helyétől távol jelentkezne. A feltételes forma pontosan akkor vállal, amikor a tag tudja tartani a vállalást; a `std::is_nothrow_move_constructible_v` itt csak előkép, a teljes trait-rendszert később tárgyaljuk.

23 Osztálytervezés, láthatóság és operátorok

23.1 public/private/protected és invariánsvédelem

A korábbi részekben azt építettük fel, hogy egy osztály nem csupán adat és függvény együttese, hanem állapotgép, amelynek van megengedett állapothalmaza. Ezt az állapothalmazt nevezzük az osztály **invariánsának** (*class invariant*): olyan feltételrendszer, amelynek minden publikus művelet után teljesülnie kell, és amelyet minden publikus műveletnek meg kell őriznie. A mostani rész arra ad választ, hogyan kényszeríthető ki ez a megőrzés a nyelv eszközeivel, és miért nem azonos a hozzáférés-vezérlés a memóriavédelemmel. Az olvasónak érdemes felelevenítenie az osztály, konstruktor és invariáns rész definícióit, valamint a Rule of zero, three és five szabályát: a speciális tagfüggvények az invariáns fenntartásának a hordozói, nem pedig adminisztratív kellékek.

A hozzáférés-vezérlése nem biztonsági mechanizmus. Fordítási idejű névfeloldási szabály, amely azt mondja meg, ki *nevezheti meg* a tagot a forráskódban. Ha egy memóriablokk a folyamat címtartományában van, tulajdonképpen bárhonnan elérhető, ha a cím kiszivárgott; ezt a nyelv sem tiltja meg. A `private` a szerződést védi, nem a biteket.

23.1.1 A láthatóság mint invariáns-szerződés

Pontosítva: a **hozzáférés-vezérlés** (*access control*) szabályai fordítási idejű, statikus döntést hoznak arról, hogy egy adott tag neve egy adott fordítási ponton használható-e. A szabályok feltétele, hogy a tag létezzen, a hozzáférés vezérlése a fordítás során eldőljön, és a program jól formált legyen. Ha a hozzáférés megsértése miatt a program nem fordul le, az nem futásidejű hiba, hanem diagnosztika.

Az **encapsulation** ennél tágabb tervezési fogalom: az adat és a rá vonatkozó műveletek egy egységbe zárása úgy, hogy a belső reprezentáció a publikus interfész mögött szabadon változtatható legyen. A hozzáférés-vezérlés ennek az eszköze, nem maga a cél.

Egy C osztály invariánsát jelöljük $\mathrm{INV}_C(\cdot)$-vel. A szerződés akkor tartható, ha:

1. a konstruktor minden létrejött objektumra INV_C-t állít elő;
2. minden publikus, nem `const` tagfüggvény bemenetére feltesszük INV_C-t, és kimenetére garantáljuk;
3. a `const` tagfüggvényekre feltesszük INV_C-t, és nem módosíthatják azt;
4. a destruktor csak érvényes állapotból fut le.

A hozzáférés-vezérlés akkor szolgálja ezt, ha a belső tagok kívülről nem nevezhetők meg, így a 2–3. pont megsértése fordítási hibát okoz.

23.1.2 class és struct alapértelmezett hozzáférése

A két kulcsszó között a szabvány szerint egyetlen különbség van: a `class` alapértelmezett taghozzáférése `private`, a `struct`-é `public`. Ugyanez érvényes az alapértelmezett öröklési módra is (`class`: `private`, `struct`: `public`), de az öröklés részletes taglalása a következő fejezet témája. A konvenció szerint a `struct` akkor hasznos, ha az adattagok együtt alkotnak egy érvényes értéket, és nincs mögötte rejtett állapot (például egy `Point`), a `class` pedig akkor, ha az invariáns fenntartásához tagfüggvények kellenek.

A lényeg: a különbség konvenció, nem nyelvi kényszer. Egy `struct`-ban is lehetnek `private` tagok, és egy `class`-ban is lehet minden tag `public`.

23.1.3 Tag-hozzáférés, this pointer és statikus tagok

A taghozzáférés három szintje:

- `private`: csak a saját osztály tagjai és a barátok (*friend*) férnek hozzá.
- `protected`: a saját osztály tagjai, a barátok, valamint a leszármazott osztályok tagjai és barátai. Fontos megszorítás: a leszármazott `protected` taghoz csak a saját típusán (vagy abból származó típuson) keresztül fér hozzá, egy `Base&`-en keresztül nem. Ez a szabály megakadályozza, hogy egy leszármazott tetszőleges `Base`-objektum belsejébe belenyúljon.
- `public`: bárki.

Egy nem `static` tagfüggvényben a `this` egy mutató az aktuális objektumra. Nem `const` tagfüggvényben típusa C^*, `const` tagfüggvényben C^* const (az objektum `const`, a pointer nem). A `this` nem lvalue, nem módosítható, és nem lehet `nullptr` érvényes hívásnál. **Statikus tagfüggvényben nincs `this`**, ezért az nem érheti el a nem statikus adattagokat. Egy statikus adattag az osztály hatókörében él, egyetlen példányban; C++17-től `inline static` adattag fejlécben is definiálható.

23.1.4 Const tagfüggvény és ref qualifier

A `const` tagfüggvény ígéret: nem módosítja az objektum megfigyelhető állapotát. A `mutable` tag kivétel: az ilyen adattag `const` tagfüggvényből is írható (például cache vagy mutex). A `const` és nem `const` változat túlterhelhető, így a `const` objektumok csak az olvasó változatot látják.

A **ref qualifier** (`&`, `&&`) azt szabályozza, hogy egy tagfüggvény milyen értékkategóriájú objektumon hívható. Ez akkor fontos, ha a függvény referenciát ad vissza a belső állapotra: ha egy ilyen függvény ideiglenes objektumon is hívható, a visszaadott referencia azonnal dangling. A `const &` minősítés önmagában **nem** elég ehhez: a `const` lvalue-referencia ideiglenes objektumra (rvalue-ra) is köthető, ezért egy `const &` tagfüggvény rvalue-n is meghívható. (A nem `const &` minősítésű tagfüggvény valóban csak lvalue-n hívható, mert a nem `const` lvalue-referencia nem köt rvalue-ra; a két szabályt nem szabad összekeverni.) A referenciát visszaadó hozzáférőt ezért két overload alkotja: egy `const &` változat lvalue-ra, és egy `const && = delete` változat, amely a nem `const` és a `const` rvalue-n történő hívást is elkapja, mert rvalue-ra a `&&` kötés a jobb. Alternatíva, hogy az rvalue-overload értéket ad vissza referencia helyett.

23.1.5 Friend: szűk együttműködés

A `friend` deklaráció egy konkrét függvénynek vagy osztálynak ad hozzáférést a nem publikus tagokhoz. Tulajdonságai: nem öröklődik, nem tranzitív, és nem adjuk „az egész világnak”. Indokolt eset: két szorosan együttműködő típus, vagy egy szabad operátor, amely az osztály reprezentációjára épül. A `friend` tehát egy tudatosan szűkített szerződéskiterjesztés, nem kényelmi eszköz.

23.1.6 Kidolgozott példa: BoundedBuffer és UniqueFd

Tekintsük a gyűrűpuffer invariánsát. Az $\mathrm{INV}(b)$ invariáns a következő három feltétel együttese:

1. üres kapacitás esetén nincs tároló és nincs elem:

$$\text{capacity_} = 0 \;\Rightarrow\; \text{data_} = \text{nullptr},\ \text{size_} = 0,\ \text{read_} = 0;$$

2. az elemszám a kapacitáson belül van:

$$0 \le \text{size_} \le \text{capacity_};$$

3. nem üres kapacitásnál az olvasási index érvényes pozíció:

$$\text{capacity_} > 0 \;\Rightarrow\; 0 \le \text{read_} < \text{capacity_}.$$

Ha a `size_`, `capacity_` és `read_` adattagok publikusak lennének, egy hívó beállíthatná `size_ = capacity_ + 1`-re, és a következő `push` a `data_[(read_ + size_) % capacity_]` kifejezésben a tömbön kívülre írna. Ez a hozzáférés-vezérlés hiányának közvetlen következménye, nem a `%` operátoré.

```
// bounded_buffer.cpp - C++20, teljes program
#include <cassert>
#include <cstddef>
#include <cstdio>
#include <cstdlib>
#include <stdexcept>
#include <utility>

class BoundedBuffer {
public:
    explicit BoundedBuffer(std::size_t capacity)
        : data_{capacity ? new int[capacity] : nullptr},
          capacity_{capacity}, size_{0}, read_{0} {}

    ~BoundedBuffer() { delete[] data_; }

    BoundedBuffer(const BoundedBuffer&) = delete;
    BoundedBuffer& operator=(const BoundedBuffer&) = delete;

    BoundedBuffer(BoundedBuffer&& other) noexcept
        : data_{std::exchange(other.data_, nullptr)},
          capacity_{std::exchange(other.capacity_, 0)},
          size_{std::exchange(other.size_, 0)},
          read_{std::exchange(other.read_, 0)} {}
```

```
    BoundedBuffer& operator=(BoundedBuffer&& other) noexcept {
        if (this != &other) {
            delete[] data_;
            data_ = std::exchange(other.data_, nullptr);
            capacity_ = std::exchange(other.capacity_, 0);
            size_ = std::exchange(other.size_, 0);
            read_ = std::exchange(other.read_, 0);
        }
        return *this;
    }

    std::size_t size() const noexcept { return size_; }
    std::size_t capacity() const noexcept { return capacity_; }
    bool empty() const noexcept { return size_ == 0; }
    bool full() const noexcept { return size_ == capacity_; }

    void push(int v) {
        if (full()) throw std::overflow_error{"push: full"};
        data_[(read_ + size_) % capacity_] = v;
        ++size_;
    }

    int pop() {
        if (empty()) throw std::underflow_error{"pop: empty"};
        const int v = data_[read_];
        read_ = (read_ + 1) % capacity_;
        --size_;
        return v;
    }

    // Kölcsönzött hozzáférés; élettartama a következő módosító műveletig tart.
    // Lvalue-n (const vagy nem const) hívható.
    const int& front() const & noexcept {
        assert(!empty());
        return data_[read_];
    }
    // Rvalue-n (const vagy nem const) tiltott: a const& változat is
    // köthetne ideiglenesre, és a visszaadott referencia dangling lenne.
    const int& front() const && = delete;

private:
    int* data_;            // tulajdonos: capacity_ darab int
    std::size_t capacity_;
    std::size_t size_;     // INV: size_ <= capacity_
    std::size_t read_;     // INV: capacity_ > 0 esetén read_ < capacity_
};

#define CHECK(condition) do {                                          \
    if (!(condition)) {                                                \
        std::fprintf(stderr, "CHECK failed at line %d\n", __LINE__); \
        std::exit(EXIT_FAILURE);                                       \
```

```
    }                                                                   \
} while (0)

int main() {
    BoundedBuffer b{4};
    b.push(10);
    b.push(20);
    CHECK(b.front() == 10);          // lvalue: megengedett
    CHECK(b.pop() == 10);            // a pop NDEBUG mellett is lefut
    CHECK(b.size() == 1);

    BoundedBuffer moved = std::move(b);
    CHECK(moved.size() == 1);
    CHECK(b.size() == 0);
}
```

Fordítás és futtatás:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o bounded_buffer bounded_buffer.cpp
./bounded_buffer
```

A tiltó overload hatását két negatív próba mutatja: mindkettő fordítási hiba, `use of deleted function 'const int& BoundedBuffer::front() const &&'` üzenettel.

```
BoundedBuffer make() { BoundedBuffer b{2}; b.push(7); return b; }
const int& r1 = make().front();             // nem const rvalue: hiba

const BoundedBuffer cb{2};
const int& r2 = std::move(cb).front();      // const rvalue: hiba
```

Buildjegyzőkönyv (`lab-gcc` profil). A program figyelmeztetés nélkül fordult, és 0 kóddal futott, `-DNDEBUG` mellett is; a két negatív próba a fenti hibával nem fordult. A `const &&` `= delete` nélküli változatban az `r1` olvasása ASan builddel `heap-use-after-free` hibát adott: a `const &` tagfüggvény az ideiglenes objektumon is lefutott.

A második példa egy FD-tulajdonos. Az invariáns: `fd_ == -1` vagy `fd_ >= 0`, ahol a `-1` a „nem birtokol” állapot. Ha a `fd_` publikus lenne, egy hívó beállíthatná például `-17`-re, és a destruktor `close(-17)`-et hívna; vagy ami súlyosabb, beállíthatná egy másik objektum által birtokolt FD-re, és a program ugyanazt az FD-t kétszer zárná be. A hozzáférés-vezérlés itt a *birtoklási* szerződést védi.

```
// unique_fd.cpp - POSIX, illusztráció
#include <unistd.h>
#include <utility>

class UniqueFd {
public:
    explicit UniqueFd(int fd = -1) noexcept : fd_{fd} {}
    ~UniqueFd() { if (fd_ >= 0) ::close(fd_); }

    UniqueFd(const UniqueFd&) = delete;
    UniqueFd& operator=(const UniqueFd&) = delete;

    UniqueFd(UniqueFd&& o) noexcept : fd_{std::exchange(o.fd_, -1)} {}
```

```
    UniqueFd& operator=(UniqueFd&& o) noexcept {
        if (this != &o) { reset(); fd_ = std::exchange(o.fd_, -1); }
        return *this;
    }

    int get() const noexcept { return fd_; }
    explicit operator bool() const noexcept { return fd_ >= 0; }
    int release() noexcept { return std::exchange(fd_, -1); }

    void reset(int fd = -1) noexcept {
        if (fd == fd_) return;          // önmagára reset: nincs teendő
        if (fd_ >= 0) ::close(fd_);
        fd_ = fd;
    }

private:
    int fd_;   // INV: fd_ == -1 vagy fd_ >= 0
};
```

23.1.7 Az encapsulation nem memóriavédelmi határ

A `private` nem akadályozza meg, hogy egy mutatóval, `reinterpret_cast`-tal vagy `memcpy`-vel valaki hozzáférjen az objektum byte-jaihoz, és nem is garantál semmit a folyamatok vagy felhasználók között. Az osztály layoutja az ABI része: az offsetek ismertek, a `sizeof` és az alignment megfigyelhető. A hozzáférés-vezérlés azt szabályozza, milyen *forráskód* nevezheti meg a tagot, nem azt, hogy a futó program mit tehet. Ezért van az, hogy egy memóriahibás, UB-t tartalmazó programot a `private` nem ment meg, és egy `public` adattaggal rendelkező típus sem lesz önmagától hibás — csak az invariáns fenntartása kerül a hívó felelősségébe.

23.1.8 Tipikus hibák

- „Majd később bezárjuk”: minden adattag publikus, mert kényelmes; később az invariáns nem kényszeríthető ki anélkül, hogy ne törnénk el a felhasználókat.
- `protected` adattag használata tesztelhetőség ürügyén: a leszármazottak közvetlenül írhatják, az invariáns kettétörik, és a töréspont nehezen lokalizálható.
- `const` tagfüggvény, amely nem `const` referenciát vagy pointert ad vissza a belső állapotra; ezzel a `const` ígéret formálisan teljesül, tartalmilag nem.
- `front()` ref qualifier nélkül, vagy csak `const &` minősítéssel: ideiglenes objektumon is hívható, a visszaadott referencia azonnal dangling. Javítás: a `const &&` overload törlése.
- A `friend` kiterjesztése egész osztályra, amikor egyetlen szabad operátorra lett volna szükség.
- Az `access control` összekeverése a biztonsággal: a „privát” adat nem titkos, csak nem nevezhető meg a forrásban.

23.1.9 Összefoglaló

A hozzáférés-vezérlés célja, hogy az osztály invariánsa a publikus interfészen keresztül kényszeríthető legyen. A `class` és `struct` között csak az alapértelmezett hozzáférés a különbség. A `const` tagfüggvények az olvasási szerződést, a ref qualifierek a referenciát visszaadó függvények élettartam-biztonságát, a `friend` pedig a szűk, indokolt együttműködést szolgálják. Az encapsulation nem memóriavédelem: a fordító a neveket ellenőrzi, a futó programot nem.

23.1.10 Feladatok

1. feladat (elemzés). Tegyük fel, hogy a `BoundedBuffer`-ben a `size_` és `read_` tagok `protected`-ek, és egy `LoggedBuffer : public BoundedBuffer` leszármazott közvetlenül írja őket. Mutassa meg egy konkrét értékadással, hogyan sérül az invariáns, és miért nem védi meg ezt a `protected`.

2. feladat (kódolás). Egészítse ki a `BoundedBuffer`-t `std::size_t remaining() const noexcept` és `void clear() noexcept` tagfüggvényekkel! Adja meg mindkettő előfeltételét és utófeltételét, és indokolja, hogy a `clear()` miért nem allokál.

3. feladat (diagnosztika). Egy kolléga a következő tagfüggvényt javasolja: `int& front() noexcept { return data_[read_]; }`. Döntse el, hogy megsérti-e a gyűrűpuffer számláló-invariánsát, és hogy milyen más szerződést sért meg. Javítsa a kódot!

23.1.11 Megoldások

1. megoldás. Az invariáns része: $0 \le \text{size_} \le \text{capacity_}$ és $\text{read_} < \text{capacity_}$. Ha a leszármazott `size_ = capacity_ + 1`-et ír, a következő `push` feltétele `full()` hamis lesz, és a `data_[(read_ + size_) % capacity_]` index a tömbön kívülre mutat. A `protected` ezt nem akadályozza meg, mert a leszármazott a saját típusán keresztül hozzáférhet. Helyes megoldás: az adattagok maradjanak `private`-ek, a leszármazott a publikus műveleteken keresztül dolgozzon, vagy kapjon az ős egy szűk, invariánst fenntartó védett műveletet (például `pop_into`), amelyet a leszármazott hívhat. Ellenőrzési feltétel: a leszármazott kódjában nem szerepel közvetlen írás a `size_`/`read_` tagokra, és a `push`/`pop` továbbra is kivétellel jelzi a határállapotot.

2. megoldás. `remaining()` előfeltétele: $\mathrm{INV}(b)$; utófeltétele: `capacity_ - size_`, azaz a szabad helyek száma, és nem módosít állapotot, ezért `const noexcept` helyes. `clear()` előfeltétele: $\mathrm{INV}(b)$; utófeltétele: `size_ == 0`, `read_ == 0`, a `capacity_` és a `data_` változatlan. Mivel a tárhely mérete nem változik, elegendő a két számlálót nullázni; a `delete[]`/`new[]` pár felesleges lenne, és kivételt is dobhatna, ami `noexcept` mellett `std::terminate`-hez vezetne. Ellenőrzési feltétel: `clear()` után `empty()` igaz, `capacity()` változatlan, és a következő `push` az első helyre ír.

3. megoldás. A számláló-invariánst ez a függvény nem sérti meg: a `size_`, `read_`, `capacity_` értékét nem módosítja, tehát $0 \le \text{size_} \le \text{capacity_}$ továbbra is fennáll. Megsért viszont két másik szerződést. Egyrészt a `const`-ságot: nem `const` tagfüggvényről van szó, tehát `const BoundedBuffer`-en nem hívható, ami következetlen az olvasó műveletekkel. Másrészt a kölcsönzött hozzáférés élettartamát: a visszaadott `int&` a következő `pop()`-ig érvényes, utána egy másik elemre mutat, a hívó ezt könnyen elfelejtheti. Javítás: `const int& front() const & noexcept` és a tiltó `const int& front() const && = delete;` (ahogy a példában), ha pedig írható hozzáférés kell, akkor azt külön, dokumentált művelettel, például `pop()`-pal biztosítsuk. Ellenőrzési feltétel: a javított `front() const` objektumon is hívható, ideiglenes objektumon viszont nem fordul le, és a visszaadott referencia élettartama a dokumentációban szerepel.

23.2 Értéktípus, kompozíció és immutable interfész

A fejezet kiindulópontja egy kellemetlen felismerés: a legtöbb DevOps-környezetben keletkező hiba nem attól lesz hiba, hogy rossz az algoritmus, hanem attól, hogy egy változó jelentése nem hordozza magában az érvényességi feltételét. Egy `unsigned port` érték lehet 0, lehet 70 000, és a fordító mindkettőt ugyanolyan szívesen elfogadja. Az értéktípus (value type) fegyelme ezt a rést zárja be: a típus nem „egy doboz a bájtok körül”, hanem állítás arról, hogy a benne lévő állapot minden megfigyelhető pillanatban kielégíti a szerződését. Ebben az alfejezetben portot,

időtartamot, bájtszámot és erőforrás-azonosítót modellezünk, és azt nézzük meg, hogyan csökkenti a kompozíció a rejtett élettartam-függőségeket.

23.2.1 Értéktípus mint érvényességi szerződés

Értéktípusnak nevezünk egy C++ osztályt, ha (1) másolása értelmes és megfigyelhetően egyenértékű állapotot állít elő, (2) nincs önálló identitása, tehát két azonos állapotú példány felcserélhető, és (3) minden publikus művelet után az objektum kielégíti az osztály **invariánsát**: azt a predikátumot, amely az összes adattagra és azok kapcsolatára egyszerre igaz.

Az invariáns-alapú tervezés három feltétele:

1. **Teljes konstrukció.** Minden konstruktor vagy érvényes állapotot állít elő, vagy megszakad; „félig inicializált, majd javítgatott” objektum nem létezhet.
2. **Zárt hozzáférés.** Az adattagok `private`-ok (erről lásd public/private/protected és invariánsvédelem (23.1)), és nincs olyan publikus művelet, amely érvénytelen állapotba vihetné az objektumot.
3. **Őrzött átmenet.** Minden módosító művelet vagy megtartja az invariánst, vagy visszautasítja a műveletet — jellemzően `std::optional`, hibakód vagy kivétel formájában.

A második pont az, amiért a „validálj a konstruktorban, aztán adj mindenkinek settert” minta nem működik: a setter visszaadja a nyilvános, ellenőrizetlen írás jogát.

23.2.1.1 Erős típus és validált konstrukció

A **strong typedef** (erős típusdefiníció) olyan wrapper, amely megkülönbözteti a reprezentációt a jelentéstől. A `using Port = unsigned;` nem az: ez csak alias, ezért egy `unsigned`-ból és egy `Port`-ból azonos overload-halmaz keletkezik, és az Overload, konverzió és explicit interfész (19.5) szerinti konverziós szabályok sem segítenek. Igazi erős típus csak külön osztály, amelynek nincs implicit konverziója a reprezentáció felé és felől.

A validált konstrukció kanonikus alakja: privát konstruktor + statikus gyártó függvény, amely `std::optional`-t ad vissza.

```
class Port {
public:
    static constexpr std::optional<Port> from_number(unsigned n) noexcept;
private:
    constexpr explicit Port(unsigned n) noexcept;
    unsigned number_;
};
```

Miért jó ez a három elem együtt? A privát konstruktor miatt kívülről nem lehet „nyers” értéket becsomagolni; a `static` gyártó az egyetlen belépési pont, tehát ott és csak ott kell az ellenőrzést helyesen megírni; az `optional` visszatérés pedig kikényszeríti, hogy a hívó foglalkozzon a hibával. A `noexcept` indokolt, mert a gyártó nem allokál és nem dob.

23.2.2 Explicit konverzió és mértékegység-biztos paraméter

Egy domain típusnak általában kell valamilyen kifelé mutató út: naplózáshoz, rendszerhíváshoz, szerializáláshoz. Ezt **explicit konverziós operátorral** adjuk meg, hogy a fordító ne konvertálhasson véletlenül ott, ahol a programozó nem akarta:

```
explicit constexpr operator unsigned() const noexcept { return number_; }
```

Az explicit kulcsszó nélkül a `f(port)` hívás csendben `unsigned` paraméterré alakulna, és egy portméretű argumentum helyére bármi bekerülhetne. Ugyanez a logika védi a **mértékegység-biztos paramétert**: a `void sleep(int)` típusú interfész nem mondja meg, hogy másodperc vagy milliszekundum; a `std::chrono::milliseconds` viszont igen, és a különböző `duration` típusok közötti átváltás csak `duration_cast`-tal, azaz szándékosan lehetséges. A saját `Timeout` osztály így nem „egy int”, hanem egy int, amelynek van mértékegysége, érvényességi tartománya és neve.

Ugyanez a bájtszámnál: a `ByteCount` és az elemszám két külön típus, akkor is, ha mindkettő `std::uint64_t`-en ül. Ez a fajta megkülönböztetés nem holmi esztétika: a `memcpy(dst, src, n)` harmadik paramétere bájtban értendő, míg a `std::vector::reserve(n)` elemszámban; a kétféle szám összekeverése tipikus, nehezen felderíthető hibát okoz.

23.2.3 Kompozíció a rejtett lifetime-függőségek ellen

A **kompozíció** azt jelenti, hogy az összetett típus tagként tartalmazza a részeit, és viseli azok érvényességi garanciáit. Az `Endpoint` nem „egy string meg egy int”, hanem egy nem üres host és egy érvényes `Port` párja; ha a kettőből bármelyik érvénytelen, az `Endpoint` létre sem jön. Ezzel a hiba a keletkezés helyére kerül, nem pedig a sokadik hívási szintre.

Az öröklés ezzel szemben **is-a** viszony: csak akkor helyes, ha a leszármazott valóban az ős speciális esete, és a bázis interfésze erre fel van készítve (virtuális destruktor, megfelelően dokumentált szerződés). Egy `Port` bázisból leszármaztatott `PrivilegedPort` például értelmetlen, mert a `Port` nem erre a bővítésre készült, és a bázisra mutató hivatkozáson át a leszármazott levágódna (slicing). Ugyanezért nem származtatunk `std::string`-ből vagy `std::vector`-ból: ezek nem bázisnak készültek, nem virtuális destruktorral. Az újrafelhasználást a kompozíció és a sablon adja; az öröklést nem tekintjük alapértelmezett eszköznek, és a tervezési minták katalógusát sem tárgyaljuk itt.

A kompozíció élettartamot is egyszerűsít: a tag objektum élettartama a tartalmazó objektuméba ágyazódik, így nincs külön `new`/`delete`, nincs „ki zárt be kit” kérdés. A `Connection` például tartalmazhat `Endpoint`-ot, `Timeout`-ot és egy tulajdonló handle-t; a megsemmisítési sorrend fordított, és a tagok sosem maradhatnak félig lebontott állapotban.

23.2.4 const interfész és logikai const

A **const interfész** azt jelenti, hogy a megfigyelő műveletek `const` tagfüggvények, tehát egy `const Endpoint&`-en is hívhatók. Ez nem formalitás: a `const`-korrektség az, ami a hívónak megmondja, hogy egy referencián át „csak olvasni” fog, és ami lehetővé teszi a `const` objektumok átadását anélkül, hogy lemásolnánk őket.

A **logikai const** ennél finomabb fogalom. A C++ alapértelmezésben bitenkénti (`bitwise`) const-ot ellenőriz: ha egy tag adattag megváltozik, a függvény nem lehet `const`. Néha viszont egy gyorsítótár (cache) frissítése megfigyelhetően nem változtatja meg az objektum értékét; ilyenkor a tag `mutable` lehet. Ez azonban nem ingyen van: a `mutable` tag írása a `const` tagfüggvényből

adatversenyt okoz, ha ugyanazt az objektumot több szálon, szinkronizáció nélkül olvassák (erről lásd Data race, happens-before és publikalás (15.2)). A `mutable` tehát nem kényelmi eszköz, hanem állítás arról, hogy a gyorsítótár konzisztens marad; ha ezt nem tudjuk bizonyítani, ne használjuk.

23.2.5 Trivialitás és standard-layout: tulajdonság, nem cél

A **trivially copyable** típus bitenként másolható, `memcpy`-vel is, mert a másoló konstruktor, az értékadás és a destruktor triviális. A **standard-layout** típus tagjai azonos hozzáférésűek, nincs virtuális függvénye és virtuális bázisa, ezért a memóriabeli elrendezése kiszámíthatóbb, és C ABI-határon biztonságosabban használható. Ezek **tulajdonságok**, amelyeket `std::is_trivially_copyable_v<T>` és `std::is_standard_layout_v<T>` segítségével lehet ellenőrizni, nem pedig célok, amelyeket mindenáron el kell érni.

Fontos árnyalat: egy validált konstruktorú típus is lehet trivially copyable, mert a nem másoló konstruktor nem rontja el a triviális másolhatóságot. A `Port` tehát egyszerre hordoz invariánst és másolható bájtonként — de a `Port`-ot továbbra sem szabad nyersen, ellenőrizetlen `unsigned`-ből előállítani. A bitmásolás megengedett, a jelentés nélküli újraértelmezés nem.

23.2.6 Kis érték és megosztott mutable entitás

Kétféle modell létezik, és a kettőt nem szabad összekeverni:

- **Kis érték**: olcsón másolható, identitás nélküli, invariánst hordozó típus (port, bájtszám, időtartam, erőforrás-azonosító). Interfészben értékkel vagy `const&`-ként adjuk át; a másolat önálló, élettartama a hívóé.
- **Megosztott mutable entitás**: identitással rendelkező, állapotváltozó objektum (kapcsolat, socket, session), amelyet jellemzően egy tulajdonos tart életben, a többiek pedig nem tulajdonló nézetet kapnak. Itt a kérdés nem az, hogy „másolható-e”, hanem hogy „ki zárja be, és mikor”.

A kettő keverése a leggyakoribb tervezési hiba: egy `Connection` értékként másolva két függetlennek látszó objektum ugyanarra a file descriptorra mutat, és a destruktorok versenyt futnak a `close()`-ért (erről bővebben Owner, view és erőforrásszerződés C++ API-ban (21.5)). Az értéktípusok ezért általában kis, önálló adatok; a nagy, megosztott állapot tulajdonosa egyértelmű legyen.

23.2.7 Kidolgozott példa

Az alábbi példa egy fordítási egységben mutatja a portot, a bájtszámot, az időtúllépést és az ezekből komponált endpointot. A kód ISO C++20-at használ (`<=>` és `std::optional`), a buildparancs explicit nyelvi módot ad meg. A kódot ebben a fejezetben nem futtattam; a parancs a saját laborodhoz tartozik, és a várt viselkedés leírás, nem mért eredmény.

```
// demo.cpp - ISO C++20, Linux, GCC vagy Clang
#include <chrono>
#include <compare>
#include <cstdint>
#include <iostream>
#include <limits>
#include <optional>
#include <string>
```

```
#include <type_traits>
#include <utility>

namespace devops {

class Port {
public:
    static constexpr unsigned min_value = 1;
    static constexpr unsigned max_value = 65535;

    static constexpr std::optional<Port> from_number(unsigned n) noexcept {
        if (n < min_value || n > max_value) {
            return std::nullopt;
        }
        return Port{n};
    }

    constexpr unsigned number() const noexcept { return number_; }

    // Szandekos konverzio: implicit nincs.
    explicit constexpr operator unsigned() const noexcept { return number_; }

    constexpr auto operator<=>(const Port&) const noexcept = default;

private:
    constexpr explicit Port(unsigned n) noexcept : number_{n} {}
    unsigned number_;
};

static_assert(!Port::from_number(0).has_value());
static_assert(Port::from_number(8080).has_value());
static_assert(Port::from_number(8080)->number() == 8080);

class ByteCount {
public:
    constexpr ByteCount() noexcept = default;   // 0 bajt ervenyes

    static constexpr ByteCount from_bytes(std::uint64_t v) noexcept {
        return ByteCount{v};
    }

    constexpr std::uint64_t bytes() const noexcept { return bytes_; }

    constexpr std::optional<ByteCount> checked_add(ByteCount other) const
     ↪ noexcept {
        constexpr auto max = std::numeric_limits<std::uint64_t>::max();
        if (other.bytes_ > max - bytes_) {
            return std::nullopt;
        }
        return ByteCount{bytes_ + other.bytes_};
    }
```

```
    constexpr auto operator<=>(const ByteCount&) const noexcept = default;

private:
    constexpr explicit ByteCount(std::uint64_t v) noexcept : bytes_{v} {}
    std::uint64_t bytes_{};
};

class Timeout {
public:
    using rep = std::chrono::milliseconds::rep;

    static constexpr std::optional<Timeout> from_milliseconds(rep ms) noexcept {
        if (ms <= 0) {
            return std::nullopt;
        }
        return Timeout{std::chrono::milliseconds{ms}};
    }

    constexpr std::chrono::milliseconds value() const noexcept { return ms_; }

private:
    constexpr explicit Timeout(std::chrono::milliseconds ms) noexcept : ms_{ms}
     ↪ {}
    std::chrono::milliseconds ms_{};
};

class Endpoint {
public:
    static std::optional<Endpoint> make(std::string host, Port port) {
        if (host.empty()) {
            return std::nullopt;
        }
        return Endpoint{std::move(host), port};
    }

    const std::string& host() const noexcept { return host_; }
    Port port() const noexcept { return port_; }

    std::string to_string() const {
        return host_ + ':' + std::to_string(port_.number());
    }

private:
    Endpoint(std::string host, Port port)
        : host_{std::move(host)}, port_{port} {}

    std::string host_;
    Port port_;
};

}  // namespace devops
```

```
int main() {
    using devops::ByteCount;
    using devops::Endpoint;
    using devops::Port;
    using devops::Timeout;

    const auto port = Port::from_number(9100);
    if (!port) {
        std::cerr << "ervenytelen port\n";
        return 1;
    }

    const auto endpoint = Endpoint::make("metrics.internal", *port);
    if (!endpoint) {
        std::cerr << "ervenytelen endpoint\n";
        return 1;
    }

    const auto timeout = Timeout::from_milliseconds(2500);
    if (!timeout) {
        std::cerr << "ervenytelen timeout\n";
        return 1;
    }

    const auto sum =
     ↪  ByteCount::from_bytes(1024).checked_add(ByteCount::from_bytes(2048));
    if (!sum) {
        std::cerr << "tulcsordulas\n";
        return 1;
    }

    std::cout << endpoint->to_string() << "  " << sum->bytes() << " B  "
              << timeout->value().count() << " ms\n";

    static_assert(std::is_trivially_copyable_v<Port>);
    static_assert(std::is_standard_layout_v<Port>);
    static_assert(!std::is_default_constructible_v<Port>);
    return 0;
}
```

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o demo demo.cpp && ./demo
# vagy: clang++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o demo demo.cpp
```

A példa várható kimenete (nem mért eredmény): `metrics.internal:9100  3072 B  2500 ms`. A `Timeout` miért nem `int`? Mert a `value()` típusa `std::chrono::milliseconds`, így a `.count()` hívás láthatóvá teszi, hogy itt egy számról van szó, és nem keveredik össze egy másodpercben értendő paraméterrel.

Szándékos compile-fail illusztráció — ez a töredék **nem** fordul le, és a diagnosztikai üzenetek fordítónként eltérnek:

```
Port p = 8080;       // nincs nyilvanos, implicit konverzio
unsigned u = p;      // az explicit operator nem hasznalhato implicit helyen
```

```
Port d;                 // nincs alap-ertekezett allapot: nincs "0 port"
```

Az első két sor az explicit konverzió védelmét mutatja, a harmadik pedig azt, hogy az invariáns miatt szándékosan nincs alapértelmezett konstruktor: a „0 port” nem létező fogalom.

23.2.8 Tipikus hibák és javításuk

A leggyakoribb minta az alias: `using Port = unsigned;`. Ettől minden port és minden darabszám ugyanaz a típus lesz, és az overload-feloldás nem tud különbséget tenni. A javítás valódi osztály, nem alias.

A második: a validált gyártó mellett ott marad egy publikus `set(int)` vagy `number(int)` mutátor, amely megkerüli az ellenőrzést. Ha kell módosítás, adjunk `std::optional<T> with_number(unsigned) const` stílusú, új értéket előállító műveletet, vagy `checked_add`-hoz hasonló, hibát visszaadó változatot.

A harmadik: hiányzó `explicit`. Az implicit konverzió nem okoz azonnali hibát, csak később, amikor egy port paraméter helyére egy véletlen `unsigned` kerül. Rendszeres ellenőrzés: a gyártó és a konverziós operátor mindig `explicit`.

A negyedik: `mutable` gyorsítótár szinkronizáció nélkül. A `const` tagfüggvény írása adatverseny; ha a cache megosztott objektumon van, akkor atomikus művelet vagy zár szükséges, különben a „logikai const” állítás hamis.

Az ötödik: értéktípusból öröklés, hogy „újrahasznosítsunk”. A leszármazott levágódik, a bázis destruktora nem virtuális, és a bázis invariánsa nem védi a leszármazott új tagjait. A kompozíció itt mindig olcsóbb.

23.2.9 Összefoglaló

Az értéktípus a szerződés hordozója: az invariáns a konstrukcióban keletkezik, a privát adattagok és a gyártó függvények zárják be, a `const` interfész pedig megfigyelhetővé teszi, hogy ki mit módosíthat. A strong typedef megkülönbözteti a jelentést a reprezentációtól, az `explicit` konverzió megakadályozza a csendes típuskeveredést, a mértékegységes típusok (például `std::chrono::milliseconds`) pedig a paraméterek jelentését rögzítik. A kompozíció a tagok élettartamát a tartalmazó objektuméhez köti, ezért kevesebb rejtett függőséget hagy. A trivialitás és a standard-layout hasznos tulajdonság, de nem cél; a kis értékeket a megosztott, identitással rendelkező entitásoktól külön kell tartani, mert utóbbiak tulajdonlása és lezárása külön szerződés.

23.2.10 Feladatok

1. **Magyarázó feladat.** Egy kollégád ezt javasolja: `struct Port { unsigned number; };`, mert „a validációt úgyis a hívó végzi”. Sorolj fel három konkrét DevOps-hibamódot, amelyet a privát adattag + validált gyártó kizár, és fogalmazd meg, miért nem elegendő a dokumentáció.
2. **Kódolási feladat.** Készíts `ResourceId` erős típust `std::uint64_t` reprezentációval, ahol a 0 érvénytelen. Legyen `from_raw`, `value`, `explicit operator std::uint64_t`, defaultolt `<=>`, valamint `checked_increment()`, amely `std::optional<ResourceId>`-t ad és túlcsorduláskor üres értéket. Egészítsd ki egy `static_assert`-tel, amely igazolja, hogy a 0 nem hozható létre.
3. **Diagnosztikai feladat.** Találd meg az alábbi típus összes tervezési hibáját, és javítsd:

```
class Timeout {
public:
    Timeout(int ms) : ms_{ms} {}
    int ms() { return ms_; }
    void extend(int extra) { ms_ += extra; }
private:
    int ms_;
};
```

4. **Tervezési feladat.** Modellezd kompozícióval egy `Connection` típust, amely egy `Endpoint`-ot, egy `Timeout`-ot és egy nem tulajdonló `int fd` nézetet tartalmaz. Ne írj hálózati kódot; csak a tagokat, a konstrukciós szerződést és a `const` interfészt indokold.

23.2.11 Megoldások

1. megoldás. A `struct Port { unsigned number; };` három tipikus hibamódja: (a) egy konfigurációból beolvasott 0 vagy 70000 érték tovább terjed, és csak a `connect()` hívásnál, érthetetlen `EINVAL`/`ECONNREFUSED` formájában bukkan fel; (b) két különböző „szám” (port és darabszám) összekeveredik egy függvényhívásban, mert a típusuk azonos; (c) egy naplózó vagy szerializáló kód véletlenül felülírja a mezőt, és az objektum érvénytelenné válik anélkül, hogy bármely művelet jelezné. A dokumentáció azért nem elég, mert nem kényszerítő: a fordító nem olvassa. Az ellenőrzési feltétel: a javasolt típusra a `Port p;` és a `Port p = 8080;` nem fordul le, a `Port::from_number(0)` pedig üres `optional`-t ad.

2. megoldás.

```
class ResourceId {
public:
    static constexpr std::optional<ResourceId> from_raw(std::uint64_t v)
    ↪ noexcept {
        if (v == 0) {
            return std::nullopt;
        }
        return ResourceId{v};
    }

    constexpr std::uint64_t value() const noexcept { return v_; }
    explicit constexpr operator std::uint64_t() const noexcept { return v_; }

    constexpr std::optional<ResourceId> checked_increment() const noexcept {
        constexpr auto max = std::numeric_limits<std::uint64_t>::max();
        if (v_ == max) {
            return std::nullopt;
        }
        return ResourceId{v_ + 1};
    }

    constexpr auto operator<=>(const ResourceId&) const noexcept = default;

private:
    constexpr explicit ResourceId(std::uint64_t v) noexcept : v_{v} {}
    std::uint64_t v_;
```

```
};

static_assert(!ResourceId::from_raw(0).has_value());
static_assert(ResourceId::from_raw(1).has_value());
```

Az indoklás: a `from_raw` az egyetlen belépési pont, ezért a „nem nulla” invariáns egy helyen ellenőrzött; a `checked_increment` megtartja az invariánst és a túlcsordulást hibaként jelzi, nem csendes wraparounddal; a `<=>` defaultolása csak akkor helyes, mert egyetlen tag és nincs kézi invariáns a rendezésben (a sorrend a reprezentáció sorrendje). Ellenőrzési feltétel: `ResourceId::from_raw(0)` üres, `ResourceId::from_raw(5)->value() == 5`, és a `std::uint64_t u = *ResourceId::from_raw(5);` nem fordul le az explicit operátor miatt.

3. megoldás. A hibák rendre: (a) a konstruktor nem `explicit`, ezért `Timeout t = 5;` és `f(5)` is lefordul, vagyis bármely `int` automatikusan időtúllépéssé válik; (b) nincs invariáns, a 0 és a negatív érték is „érvényes”, holott a nulla időtúllépés jelentése definiálatlan; (c) a `ms()` nem `const`, ezért `const Timeout&`-en nem hívható, és a csak olvasó kód kénytelen másolatot venni; (d) az `extend` publikus mutátor, amely negatívba viheti az értéket, tehát megsérti az invariánst; (e) az `int` nem hordoz mértékegységet, a hívó nem tudja, hogy ms-e vagy s. Javított változat: privát konstruktor, `from_milliseconds` gyártó `std::optional`-lal, `const` lekérdező, `extend` helyett `std::optional<Timeout> extended(rep) const`, és `std::chrono::milliseconds` belső reprezentáció. Ellenőrzési feltétel: `Timeout::from_milliseconds(0)` üres, a `const Timeout&`-en hívható a lekérdező, és nincs olyan publikus művelet, amely érvénytelen állapotot állítana elő.

4. megoldás.

```
class Connection {
public:
    Connection(Endpoint endpoint, Timeout timeout, int fd) noexcept
        : endpoint_{std::move(endpoint)}, timeout_{timeout}, fd_{fd} {}

    const Endpoint& endpoint() const noexcept { return endpoint_; }
    Timeout timeout() const noexcept { return timeout_; }
    int fd() const noexcept { return fd_; }   // nem tulajdonlo nezet

private:
    Endpoint endpoint_;   // ertek: sajat invariáns, sajat elettartam
    Timeout timeout_;     // ertek: sajat mertekegyseg
    int fd_;              // nezet: a tulajdonos masutt zarja le
};
```

Az indoklás: az `Endpoint` és a `Timeout` értékként beágyazott tag, ezért élettartamuk a `Connection`-éhez kötött, nincs külön felszabadítás és nincs „félkonstruált” állapot; a konstruktor feltétele, hogy a bemenetek már érvényesek legyenek, amit a gyártó függvények garantálnak. Az `fd_` szándékosan nyers `int` és nem tulajdonló: a `close()` felelőssége a tulajdonosé, ezt kommentben rögzíteni kell. A `const` tagfüggvények miatt a `const Connection&`-en is olvasható minden adat. Ellenőrzési feltétel: a `Connection`-nek nincs destruktora, amely lezárná az fd-t; ha mégis kell RAII, akkor az egy külön tulajdonló tag (például egy `UniqueFd`) feladata, nem a `Connection`-é.

23.3 Operátortúlterhelés és konverziós operátor

Az operátortúlterhelés (operator overloading) C++-specifikus eszköz: C-ben nincs rá lehetőség. A lényeg nem az, hogy „szép szintaxist” adjunk, hanem hogy a típus viselkedése kiszámítható maradjon. Egy jól megtervezett operátor ugyanazt a jelentést hordozza, mint a beépített megfelelője:

`a + b` ne módosítsa `a`-t, `a += b` igen; `++it` adja vissza az új értéket, `it++` a régit; az `if (x)` csak akkor működjön, ha a típusnak van értelmes „igaz/hamis” állapota. Az erőforrás-életciklust kezelő típusokat különösen óvatosan kell kezelni: az operátor ne rejtsen el allokációt, lezárást vagy más, a felhasználó számára meglepő mellékhatást.

23.3.1 Az operátor mint szerződés: definíció és feltételek

Operátortúlterhelésről akkor beszélünk, amikor egy `operator@` nevű függvényt deklarálunk, ahol `@` a túlterhelhető operátor jele. Pontos feltételek:

- Legalább az egyik operandusnak osztály- vagy felsorolástípusúnak kell lennie. Beépített típusokra nem lehet új operátort definiálni.
- Nem hozható létre új operátorjel, és a meglévő operátor aritása (operandusszáma) nem változtatható.
- Az operátor beépített precedenciája és asszociativitása nem változik meg a túlterheléstől. A `&&` és `||` rövidzárlata viszont **elvész**: a túlterhelt változat mindkét operandust kiértékeli (lásd lent).
- Bizonyos operátorok csak nem statikus tagfüggvényként terhelhetők túl: `=`, `[]`, `()`, `->`, valamint a konverziós operátorok. A `::`, `.`, `.*`, `?:`, `sizeof`, `typeid`, `alignof`, `noexcept` és a cast-operátorok nem terhelhetők túl.
- Egy operátor lehet tagfüggvény, nem tag függvény, illetve „hidden friend” (a class belsejében `friend`-ként definiált függvény). A választás nem esztétikai kérdés: a bal oldali operandus konverzióit, az overload resolution halmazát és a hozzáférést is befolyásolja.

A következő táblázat a leggyakoribb, konvencionális választásokat foglalja össze.

Operátor	Ajánlott forma	Visszatérés	Megjegyzés
`+=`, `-=`, `*=`	tag	`T&`	módosítja a bal oldalt, `*this`-t ad vissza
`+`, `-`, `*`	nem tag vagy hidden friend	`T`	új értéket ad, ne referenciát
`[]`	tag	`T&` / `const T&`	általában nem ellenőriz boundsot
`()`	tag	tetszőleges	callable objektum
`++` prefix	tag	`T&`	először növel, az új értéket adja
`++` postfix	tag	`T`	a régi értéket adja vissza
`=`	tag	`T&`	csak tag lehet
`swap`	nem tag + tag `swap`	`void`	lehetőleg `noexcept`
konverzió	tag	nincs visszatérési típus	implicit csak indokolt esetben

23.3.2 Tag- vagy nem tag operátor?

A döntés vezérelve: ha az operátor a bal oldali operandust módosítja, és a bal oldal mindig a saját típusunk, akkor tagfüggvény a természetes választás. Ilyen a `+=`, a `-=`, a prefix `++`, az `[]` és a `()`. Ha viszont szimmetrikus műveletről van szó, ahol mindkét oldal szerepelhet eltérő típusból, vagy a bal oldali operandus nem a mi típusunk, akkor nem tag függvény kell. Például `std::cout`

`<< x` esetén a bal oldali operandus `std::ostream`, ezért az `operator<<` nem lehet tagja a saját típusunknak.

A nem tag operátor lehetővé teszi, hogy a bal oldali operanduson is történjen felhasználói konverzió. Ez hasznos lehet, de veszélyes is: ha túl sok implicit konverzió van, az overload resolution nehezen követhetővé válik. A „hidden friend” olyan `friend` függvény, amelyet a class belsejében definiálunk. Nem tagja a classnak, de hozzáfér a privát tagokhoz, és csak argumentumfüggő kereséssel (ADL) található meg. Ez csökkenti a globális névtér szennyezését, és szimmetrikus operátoroknál gyakran jó választás.

A `friend` önmagában még nem tesz egy függvényt taggá; csak hozzáférési jogot ad. A hidden friend előnye, hogy a definíció a class mellett van, mégsem vesz részt a szokásos, nem ADL-alapú névkeresésben.

23.3.3 Aritmetika, összetett értékadás és swap

A bevett minta: a módosító műveletet (`+=`) tagfüggvényként írjuk meg, a bináris aritmetikát (`+`) pedig arra építjük. A `+` bal oldali operandusát érték szerint vesszük át, így a függvényen belül módosíthatjuk, majd értékként visszaadjuk. Ez elkerüli a kézi duplikálást, és jól viselkedik akkor is, ha a művelet kivételt dob: az eredeti objektum változatlan marad.

```
#include <iostream>
#include <utility>

class Money {
public:
    using cents_t = long long;

    constexpr Money() noexcept : cents_(0) {}
    constexpr explicit Money(cents_t cents) noexcept : cents_(cents) {}

    constexpr cents_t cents() const noexcept { return cents_; }

    constexpr Money& operator+=(const Money& rhs) noexcept {
        cents_ += rhs.cents_;
        return *this;
    }

    constexpr Money& operator-=(const Money& rhs) noexcept {
        cents_ -= rhs.cents_;
        return *this;
    }

    Money& operator++() noexcept {          // prefix
        ++cents_;
        return *this;
    }

    Money operator++(int) noexcept {        // postfix
        Money old = *this;
        ++*this;
        return old;
```

```
    }

    void swap(Money& other) noexcept {
        std::swap(cents_, other.cents_);
    }

private:
    cents_t cents_;
};

constexpr Money operator+(Money lhs, const Money& rhs) noexcept {
    lhs += rhs;
    return lhs;
}

constexpr Money operator-(Money lhs, const Money& rhs) noexcept {
    lhs -= rhs;
    return lhs;
}

inline void swap(Money& a, Money& b) noexcept {
    a.swap(b);
}

int main() {
    Money a(1000);
    Money b(250);
    Money c = a + b;        // 1250
    c += Money(50);         // 1300
    Money d = c++;          // d = 1300, c = 1301
    ++c;                    // c = 1302
    std::cout << d.cents() << ' ' << c.cents() << '\n';
}
```

Fordítási példa:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 money_demo.cpp -o money_demo
```

A példa szándékosan nem használ implicit `double` konverziót. A pénzösszeg belső reprezentációja egész cent, így nincs lebegőpontos kerekítési hiba. A `swap` tagfüggvény és a nem tag `swap` együtt biztosítja, hogy az ADL-alapú `swap(a, b)` a saját, hatékony változatunkat találja meg. A `swap` csak akkor legyen `noexcept`, ha a benne végzett műveletek valóban nem dobnak; hamis `noexcept` esetén a program `std::terminate`-hez juthat.

23.3.4 Indexelés és hívás

Az `operator[]` tipikusan tagfüggvény, és referenciát ad vissza. Mutable objektumon `T&`, const objektumon `const T&` a helyes visszatérési típus. A bounds-ellenőrzés nem automatikus: a `std::vector::operator[]` például nem ellenőriz, a `.at()` igen. Saját konténer esetén dokumentálni kell az előfeltételt: `i < size()`, és a megsértése nem definiált viselkedés, ha a mögöttes tároló így viselkedik.

```
#include <cstddef>
#include <vector>

class IntBuffer {
public:
    explicit IntBuffer(std::size_t n) : data_(n) {}

    int& operator[](std::size_t i) noexcept { return data_[i]; }
    const int& operator[](std::size_t i) const noexcept { return data_[i]; }
    std::size_t size() const noexcept { return data_.size(); }

private:
    std::vector<int> data_;
};
```

A `operator()` callable objektumot készít. Állapotot tárolhat, és a `const` minősítés itt is számít: ha a hívás nem módosítja az objektumot, legyen `const`.

```
struct Scale {
    double factor;
    double operator()(double x) const noexcept { return factor * x; }
};
```

Használat: `Scale s{2.0}; double y = s(3.0);`. A call operátort könnyű túlhasználni; ha a műveletnek nem természetes a „függvényhívás” jelentése, gyakran beszédesebb egy névvel ellátott tagfüggvény.

23.3.5 Növelés és csökkentés

A prefix `++` visszatérése `T&`: a művelet után az objektum új állapotára mutató referenciát ad. A postfix `++` szignatúrája `T operator++(int)`: a dummy `int` különbözteti meg a prefix változattól, és a visszatérési típus érték, nem referencia. A postfixnek a módosítás előtti értéket kell visszaadnia, ezért jellemzően egy másolatot készít, majd megnöveli az eredetit. Emiatt a postfix drágább lehet; generikus kódban ezért preferált a `++it`.

A postfix `T&`-et visszaadó változata tipikus hiba: ha lokális `old` objektum referenciáját adja vissza, a hívó dangling referenciát kap. Ha az eredeti objektumot adja vissza, akkor nem a régi értéket képviseli, hanem az újat, ami szintén meglepő.

23.3.6 Értékadás, összetett értékadás és kivételbiztonság

A copy assignment és a move assignment hagyományos szignatúrája:

```
T& operator=(const T& other);
T& operator=(T&& other) noexcept;
```

Mindkettő tagfüggvény, és `*this`-t ad vissza. A láncolhatóság (`a = b = c`) és a generikus algoritmusok miatt a `void` visszatérés kerülendő. Önmagára értékadásnál (`a = a`) az implementációnak helyesen kell működnie; copy-and-swap esetén ez általában automatikusan teljesül. A move assignment után a forrás „moved-from” állapotba kerül: érvényes, de nem specifikált állapotú; csak destruálás vagy újabb értékadás biztonságos vele kapcsolatban. Erről bővebben a Move constructor, moved-from állapot és std::move rész szól.

A compound assignment (`+=`, `-=`) tagfüggvényként, `T&`-et visszaadva a legtermészetesebb. A bináris `+` erre épülhet, ahogy a `Money` példában. A `swap` nem tag függvénye ugyanabban a névtérben legyen, mint a típus, hogy az ADL megtalálja; a `std::swap` specializálása helyett ez a javasolt minta. A kivételbiztonsági garanciákról és a copy-and-swapról a noexcept, swap és container-reallocation rész ad részletesebb képet.

23.3.7 Konverziós operátorok és explicit bool

A konverziós operátor tagfüggvény, nincs visszatérési típusa, és a neve `operator T`. Például:

```
struct FdView {
    int fd = -1;

    explicit operator bool() const noexcept {
        return fd >= 0;
    }
};
```

Az `explicit operator bool` a „safe bool” modern megfelelője. Kontextuális konverzióként működik `if`, `while`, `!`, `&&`, `||`, `?:` feltételében és `static_assert`-ben, de nem enged meg olyan implicit átalakítást, mint `bool b = v;` vagy `int n = v;`. Ez pontosan az, amit egy „érvényes/érvénytelen” állapotú típusnál szeretnénk. A régi `operator void*` trükk helyett C++11 óta ezt kell használni.

Általános szabály: implicit konverziós operátort csak akkor adjunk, ha a konverzió tényleg veszteségmentes és a típus természetes része. Ellenkező esetben `explicit` legyen, vagy adjunk nevesített hozzáférőt (`to_int()`, `celsius()`). Az implicit `operator int` vagy `operator double` könnyen váratlan overload resolutionhöz és több lépéses konverziós lánchoz vezet. Egy erőforrás-tulajdonos típusnál az implicit bool különösen veszélyes, mert nem mondja meg, hogy „érvényes”-e vagy „nem nulla”-e a jelentés.

23.3.8 Precedencia, rövidzárlat és az operátorok korlátai

Az operátortúlterhelés nem változtatja meg a precedenciát és az asszociativitást. Ha `a + b * c` szerepel, a `*` továbbra is erősebb, akkor is, ha saját `operator+`-t és `operator*`-ot definiálunk. Nem lehet „jobbról balra asszociatív” `operator+`-t csinálni, és nem lehet megváltoztatni az operandusok számát sem.

A `&&` és `||` túlterhelése különösen veszélyes, de két különböző okból, amelyeket érdemes szétválasztani.

- **Rövidzár.** A beépített `&&` rövidzáras: ha a bal oldal hamis, a jobb oldal nem értékelődik ki. A túlterhelt változat függvényhívás, ezért mindkét operandus **mindig** kiértékelődik. Ez minden szabványverzióban így van.
- **Kiértékelési sorrend.** C++17 óta a túlterhelt `&&`, `||` és `,` operátorszintaxissal (`a && b`) hívva megőrzi a beépített operátor sorrendi szabályát: a bal operandus a jobb előtt értékelődik ki, mellékhatásaival együtt (C++20 [over.match.oper], [expr.call]). C++14-ben és korábban ez nem volt garantált. Explicit függvényhívás-szintaxisnál (`operator&&(a, b)`) viszont az argumentumok sorrendje továbbra sem meghatározott.

```cpp
// operator_order.cpp - C++17/C++20: sorrend megmarad, rövidzár elvész
#include <cstdio>

struct Flag { bool value; };

bool operator&&(Flag a, Flag b) { return a.value && b.value; }   // kerülendő
Flag operator,(Flag, Flag b) { return b; }                       // kerülendő

int counter = 0;

Flag left()  { ++counter; std::puts("left"); return Flag{false}; }
Flag right() { std::printf("right: counter=%d\n", counter); return Flag{true}; }

int main() {
    counter = 0;
    bool r = left() && right();   // operátorszintaxis: left előbb, right is fut
    std::printf("&& -> %d\n", r);

    counter = 0;
    Flag f = (left(), right());   // ugyanígy a vesszőnél
    std::printf(", -> %d\n", f.value);

    // Explicit függvényhívás-szintaxisnál az argumentumok sorrendje
    // nem meghatározott: operator&&(left(), right())
}
```

Buildjegyzőkönyv (`lab-gcc` profil, `-std=c++20`). A kimenet: `left`, `right: counter=1`, `&& -> 0`, majd ugyanez a vesszőre `, -> 1`. A jobb operandus látja a bal mellékhatását, és a hamis bal oldal ellenére is lefut. (A GCC `-std=c++14` módban is ezt a sorrendet adta, de ott ez nem garantált.)

Logikai rövidzárlatot igénylő kódban ezért ne terheljük túl a `&&`, `||` és `,` operátorokat.

Ha mégis szükség van logikai műveletre, használjunk nevesített függvényt (`both()`, `either()`), vagy hagyjuk a beépített operátort érvényesülni `explicit operator bool` segítségével.

23.3.9 Tipikus hibák és javításuk

- **`operator+` referenciát ad vissza lokálisra.** A bináris aritmetika értéket adjon vissza, ne `T&`-et. A `T&` gyakran dangling referenciához vezet.
- **A postfix `++` `T&`-et ad vissza.** Helyesen `T operator++(int)`, a régi érték másolatával.
- **Az assignment `void`-ot ad vissza.** Helyesen `T&`, `*this`-szel.
- **Az `operator[]` értéket ad vissza mutable konténeren.** Ettől nem lehet `v[i] = x` alakban írni. Mutable változatban `T&`, const változatban `const T&` kell.
- **Az `operator<<` tagfüggvény.** A bal oldali operandus `std::ostream`, ezért nem tag függvényként kell definiálni.
- **Implicit `operator bool` vagy `operator int`.** Váratlan konverziókhoz vezet; használjunk `explicit`-et vagy nevesített függvényt.
- **Túlterhelt `operator&&`.** Elveszik a rövidzárlat; ne tegyük, ha a logikai jelentés számít.
- **A `swap` nem `noexcept`, mégis annak deklaráljuk.** Ha a benne hívott művelet dob, a `noexcept` megsértése `std::terminate`-hez vezet.

23.3.10 Összefoglaló

Az operátortúlterhelés akkor jó, ha a típus viselkedése a beépített operátorok természetes analógiáját követi. A módosító műveletek tagfüggvényként `T&`-et adnak vissza; a bináris aritmetika értéket ad; az indexelés referenciát; a postfix `++` a régi értéket. A `swap` nem tag függvényként, ADL-lel találhatóan, lehetőleg `noexcept` legyen. A konverziós operátorokat `explicit`-té kell tenni, különösen a `bool`-t. Az operátorok nem változtatják meg a precedenciát, és a túlterhelt `&&`/`||` nem őrzi meg a rövidzárlatot. Az osztálytervezésben ezért az operátor a publikus szerződés része, nem trükk.

23.3.11 Feladatok

23.3.11.1 1. feladat — Tervezési döntés

Adott egy `Temperature` értéktípus, amely Celsius-fokot tárol `double`-ként. Tervezd meg a `+=` és `+` operátorokat, és indokold, melyik legyen tag, melyik nem tag. A hőmérséklet kiolvasására ne implicit `double` konverziót használj, hanem nevesített hozzáférőt; ha mégis konverziót adsz, indokold, miért `explicit`.

23.3.11.2 2. feladat — Kódolás

Implementálj egy `Counter` osztályt, amelynek van prefix `++` és postfix `++` operátora. A postfix adja vissza a módosítás előtti értéket. Írj rövid `main`-t, amely ellenőrzi a viselkedést, és add meg a visszatérési típusokat. Magyarázd el, miért nem adhat a postfix `Counter&`-et vissza.

23.3.11.3 3. feladat — Diagnosztika

Az alábbi kód több operátorral kapcsolatos hibát tartalmaz. Találj legalább négyet, indokold őket, és írd meg a javított változatot.

```
struct Box {
    int v;
    operator bool() const { return v != 0; }
    Box& operator+(const Box& other) { v += other.v; return *this; }
    Box& operator++(int) { Box old = *this; ++v; return old; }
};

bool operator&&(Box a, Box b) { return a.v && b.v; }
```

23.3.12 Megoldások

23.3.12.1 1. megoldás

A `+=` módosítja a bal oldali operandust, ezért tagfüggvény, és `Temperature&`-et ad vissza. A `+` szimmetrikus, új értéket állít elő, ezért nem tag függvény vagy hidden friend, és `Temperature`-t ad vissza értékként. A nevesített `celsius()` hozzáférő egyértelműbb, mint az implicit `double` konverzió; ha mégis kell konverzió, az `explicit operator double() const` legyen, hogy csak `static_cast<double>(t)` alakban lehessen használni.

```
class Temperature {
public:
    explicit Temperature(double celsius) noexcept : celsius_(celsius) {}

    double celsius() const noexcept { return celsius_; }

    Temperature& operator+=(const Temperature& rhs) noexcept {
        celsius_ += rhs.celsius_;
        return *this;
    }

private:
    double celsius_;
};

inline Temperature operator+(Temperature lhs, const Temperature& rhs) noexcept {
    lhs += rhs;
    return lhs;
}
```

Ellenőrzési feltétel: `Temperature t{20.0}; t += Temperature{2.0}; auto u = t + Temperature{3.0};` után `t.celsius() == 22.0` és `u.celsius() == 25.0`. A `double d = u;` ne forduljon le, ha nincs implicit konverzió; `static_cast<double>(u)` csak akkor működjön, ha az explicit konverziót külön hozzáadtuk.

23.3.12.2 2. megoldás

```
class Counter {
public:
    constexpr Counter() noexcept : value_(0) {}
    constexpr explicit Counter(int value) noexcept : value_(value) {}

    constexpr int value() const noexcept { return value_; }

    Counter& operator++() noexcept {          // prefix
        ++value_;
        return *this;
    }

    Counter operator++(int) noexcept {        // postfix
        Counter old = *this;
        ++*this;
        return old;
    }

private:
    int value_;
};
```

A postfix azért ad vissza értéket, mert a módosítás előtti állapotot kell reprezentálnia. Ha `Counter&`-et adna vissza, vagy a lokális `old`-ra mutató dangling referenciát adna, vagy az eredetit adná vissza, ami már az új értéket képviseli. Ellenőrzési feltétel: `Counter c; Counter a = c++;`

`Counter b = ++c;` után a postfix eredménye a régi, a prefix eredménye az új érték; a `c` állapota mindkét művelet után megfelelően nő.

23.3.12.3 3. megoldás

A hibák:

1. Az `operator bool` implicit, ezért váratlan `bool`, `int` és egyéb konverziókhoz vezethet. Legyen `explicit operator bool() const noexcept`.
2. Az `operator+` módosítja a bal oldali operandust, és referenciát ad vissza. A bináris `+` ne módosítson, és értéket adjon vissza; a módosítást `+=` végezze.
3. A postfix `operator++` `Box&`-et ad vissza lokális `old`-ra, ami dangling referencia. Helyesen `Box operator++(int)`.
4. A túlterhelt `operator&&` elveszíti a rövidzárlatot. Törölni kell; az `explicit operator bool` mellett a beépített `&&` használható.
5. A `Box` `v` tagja inicializálatlan lehet; adjunk neki alapértéket.

```
struct Box {
    int v = 0;

    explicit operator bool() const noexcept { return v != 0; }

    Box& operator+=(const Box& other) noexcept {
        v += other.v;
        return *this;
    }

    Box& operator++() noexcept {
        ++v;
        return *this;
    }

    Box operator++(int) noexcept {
        Box old = *this;
        ++*this;
        return old;
    }
};

inline Box operator+(Box lhs, const Box& rhs) noexcept {
    lhs += rhs;
    return lhs;
}
```

Ellenőrzési feltétel: `Box a{1}, b{2}; Box c = a + b;` után `a.v` változatlan, `c.v == 3`; `auto d = c++;` esetén `d.v` a régi, `c.v` az új érték; `if (a)` és `a && b` a beépített rövidzárlatos logikát használja; `bool x = a;` nem fordul le. A javított kódban a `+` nem módosít mellékhatásként, a postfix nem ad vissza dangling referenciát, és a logikai operátorok viselkedése nem változik meg.

23.4 Egyenlőség, spaceship és rendezési kategóriák

Az összehasonlítás látszólag triviális téma, a rendszerprogramozásban mégis rendre ott hibázunk el, ahol egy adatszerkezet vagy algoritmus csendben megköveteli: a `std::map`, a `std::sort`, a `std::unordered_set` és a `std::ranges::sort` mindegyike egy *szerződést* vár a felhasználótól, nem pedig „valamilyen” összehasonlítást. C++20-ban a `<=>` (spaceship) és a defaulted tag-összehasonlítás ezt a szerződést részben gépre bízhatóvá tette, de a szemantika megválasztása továbbra is tervezői döntés: nem mindegy, hogy két különböző reprezentációjú érték *egyenlő*, *ekvivalens* vagy *összemérhetetlen*. Ebben az alfejezetben azt rendszerezzük, hogyan lesz egy típusból kiszámítható, konzisztens kulcs és rendezhető érték, és miért kerül bajba a `double` egy `NaN`-nal a `std::map`-ban.

Az operátorok deklarációs mechanikáját az Operátortúlterhelés és konverziós operátor (23.3) rész tárgyalta; itt azt feltételezzük, hogy az operátor mint tagfüggvény és mint szabad függvény közti választás már ismert. A lebegőpontos reprezentáció részleteihez Lebegőpontos reprezentáció és numerikus hibák (4.3), a hash-tábla komparátor-szerződéséhez Hash table, comparator és ownership (13.3) ad hátteret.

23.4.1 Az összehasonlítás mint szerződés

Egy `==` operátor akkor viselkedik *ekvivalenciarelációként*, ha reflexív ($a = a$), szimmetrikus ($a = b \Rightarrow b = a$) és tranzitív ($a = b \wedge b = c \Rightarrow a = c$). A standard tárolók és algoritmusok nagy része hallgatólagosan ezt feltételezi; ha a `==` nem tranzitív, a `std::unordered_set` viselkedése egyszerűen nem definiált.

Egy `<` operátor akkor *strict weak ordering* (szigorú gyenge rendezés), ha

1. irreflexív: $\neg(a < a)$,
2. tranzitív: $a < b \wedge b < c \Rightarrow a < c$,
3. az ekvivalencia tranzitív, ahol $a \sim b$ jelentése $\neg(a < b) \wedge \neg(b < a)$; azaz $a \sim b \wedge b \sim c \Rightarrow a \sim c$.

A 2. és 1. együtt maga után vonja az aszimmetriát ($a < b \Rightarrow \neg(b < a)$): ha $a < b$ és $b < a$ is fennállna, a tranzitivitásból $a < a$ következne. A `std::sort`, `std::map`, `std::set` és a standard bináris keresések komparátora ezt a tulajdonságot követelik meg; megsértésük nem definiált viselkedés, nem „csak” rossz sorrend.

Bizonyításvázlat. Azt állítjuk, hogy egy strict weak ordering a $\sim$ ekvivalencia-osztályain *teljes* rendezést indukál. A $\sim$ reláció ekvivalenciareláció a 3. feltétel miatt, tehát osztályokba rendez. Két különböző $[a] \neq [b]$ osztályra nem lehet egyszerre $\neg(a < b)$ és $\neg(b < a)$, mert az épp $a \sim b$, azaz $[a] = [b]$ lenne; aszimmetria miatt pedig nem állhat fenn mindkettő. Marad pontosan egy irány, tehát az osztályok rendezettek, és az algoritmusok particionálási és beszúrási érvelése érvényes. Ez a vázlat, nem teljes bizonyítás: az ekvivalencia tranzitivitását axiómának vettük, nem a `<` többi tulajdonságából vezettük le.

23.4.2 operator== és a defaulted összehasonlítás

C++20-tól egy osztály kérheti a tagonkénti összehasonlítást:

```
struct Version {
    int major = 0, minor = 0, patch = 0;
    bool operator==(const Version&) const = default;
    auto operator<=>(const Version&) const = default;
};
```

A defaulted `operator==` a nem statikus adattagokat deklarációs sorrendben hasonlítja össze `==`-val (tömböknél elemenként). Feltétel: minden tag típusa rendelkezzen használható `==`-val, és a függvénynek nincs `constexpr`/`noexcept`-ot sértő előfeltétele. A `!=` nem kell külön: a túlterhelés-feloldás a `!(a == b)` átírt jelöltet (rewritten candidate) használja.

Fontos csapda, és gyakran rosszul tanítják: a `==` jelöltjei között csak `operator==` (és annak fordított változata) szerepel, a `<=>` nem. Ha az osztály nem deklarál `==`-t, de **defaultolt** `<=>`-t igen, a fordító implicit módon egy defaultolt `operator==`-t is deklarál (C++20 [class.compare.default]); ezért egy csak `auto operator<=>(const Version&) const = default;` sort tartalmazó osztályon a `v1 == v2` is lefordul. A *saját*, nem defaultolt `<=>` viszont nem ad `==`-t: ilyenkor a `v1 < v2` működik, a `v1 == v2` nem fordul — ez a leggyakoribb C++20-as meglepetés. A fenti példában a két sor együtt a szándékot teszi explicitté, és akkor is helyes marad, ha a `<=>`-t később kézzel írjuk meg.

23.4.3 operator<=> és a rewritten candidate mechanizmus

A `<=>` eredménye nem `bool`, hanem egy *rendezési kategória* típus értéke. Az `a < b` kifejezéshez a fordító **egyetlen közös jelölthalmazt** épít: a felhasználó által deklarált `operator<`-eket, az átírt `(a <=> b) < 0` és a fordított `0 < (b <=> a)` jelölteket. Ezek közül a szokásos overload resolution választ: először az argumentumkonverziók rangja dönt, és csak döntetlen esetén lép be a szabály, hogy a nem átírt jelölt jobb az átírtnál. Egy meglévő `operator<` tehát csak azonos konverziós minőség mellett „írja felül” a `<=>`-t. Példa: `operator<(X, long)` és `operator<=>(X, int)` mellett az `x < 1`-nél az `1` (`int`) az átírt jelöltnél pontos egyezés, az `operator<`-nél egész konverzió — ezért az átírt `<=>` nyer.

```
// cmp_pick.cpp - C++20: a jobb konverzió megelőzi a "nem átírt jelölt" szabályt
#include <compare>
#include <cstdio>
struct X { int v; };
bool operator<(X, long) { std::puts("operator<(X, long)"); return false; }
std::strong_ordering operator<=>(X a, int b) { std::puts("operator<=>(X, int)");
 ↪  return a.v <=> b; }
int main() { X x{0}; (void)(x < 1); }   // 1: int -> az átírt jelölt pontos
 ↪  egyezés
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra cmp_pick.cpp` kimenete `operator<=>(X, int)`. Ha a két operátor eltérő eredményt adna, a program csendben inkonzisztens lenne — ezért a kettőnek ugyanazt a rendezést kell kifejeznie, vagy elég a `<=>`.

A kategória és a literál `0` összehasonlítása megengedett (`< 0`, `<= 0`, `> 0`, `>= 0`, `== 0`, `!= 0`), a kategória `bool`-ként **nem** használható: `if (a <=> b)` nem fordul le. Helyesen: `if ((a <=> b) < 0)`.

A `std::partial_ordering` négy értékének és a literál `0`-nak az összehasonlítása (C++20 [cmp.partialord]):

érték	`== 0`	`!= 0`	`< 0`	`<= 0`	`> 0`	`>= 0`
`less`	hamis	igaz	igaz	igaz	hamis	hamis
`equivalent`	igaz	hamis	hamis	igaz	hamis	igaz
`greater`	hamis	igaz	hamis	hamis	igaz	igaz
`unordered`	hamis	igaz	hamis	hamis	hamis	hamis

A fordított irány (`0 == c`, `0 < c` stb.) ugyanezt adja a tükrözött operátorral: `0 < c` ugyanaz, mint `c > 0`. Az `unordered` tehát *nem* egyenlő nullával, és egyetlen rendezési reláció sem igaz rá; csak a `!= 0` igaz. Ebből következik, hogy a `!((a <=> b) < 0)` nem jelenti azt, hogy `a >= b`: `NaN` esetén mindkettő hamis.

```
// partial_zero.cpp - C++20: partial_ordering és a literál 0
#include <compare>

using po = std::partial_ordering;

constexpr bool row(po c, bool eq, bool ne, bool lt,
                   bool le, bool gt, bool ge) {
    return (c == 0) == eq && (c != 0) == ne && (c < 0) == lt
        && (c <= 0) == le && (c > 0) == gt && (c >= 0) == ge;
}
//                              ==0    !=0    <0     <=0    >0     >=0
static_assert(row(po::less,       false, true, true,  true,  false, false));
static_assert(row(po::equivalent, true, false, false, true,  false, true));
static_assert(row(po::greater,    false, true, false, false, true,  true));
static_assert(row(po::unordered,  false, true, false, false, false, false));

// Fordított irány: 0 @ c ugyanaz, mint c @' 0 a tükrözött operátorral.
static_assert(!(0 == po::unordered) && (0 != po::unordered));
static_assert(!(0 < po::unordered) && !(0 >= po::unordered));
static_assert((0 > po::less) && (0 <= po::equivalent));

int main() {}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic partial_zero.cpp` figyelmeztetés nélkül fordul, tehát a táblázat minden cellája `static_assert`-tel ellenőrzött.

23.4.4 Strong, weak és partial ordering

A `std::strong_ordering` (`less`, `equal`, `greater`) esetén az egyenlőség *felcserélhetőséget* (substitutability) jelent: ha $a = b$, akkor minden megfigyelhető tulajdonságuk azonos. Egész számok, `std::string`, `std::vector<int>` ilyen.

A `std::weak_ordering` (`less`, `equivalent`, `greater`) esetén léteznek megkülönböztethető, de ekvivalensnek nyilvánított értékek. Klasszikus példa a kis-nagybetűt nem megkülönböztető név: a `"OpenSSL"` és az `"openssl"` ekvivalens, noha a bájtjaik eltérnek.

A `std::partial_ordering` (`less`, `equivalent`, `greater`, `unordered`) esetén bizonyos párok *összemérhetetlenek*. Ez a lebegőpontos `NaN` esete: sem `nan < x`, sem `x < nan`, sem `nan == nan` nem igaz.

A konverzió egyirányú: `strong_ordering` → `weak_ordering` → `partial_ordering` implicit módon átalakítható, visszafelé nem. Egy defaulted `<=>` visszatérési típusa a tagok kategóriáinak *közös* kategóriája, tehát ha bármelyik tag `double`, az egész osztály `partial_ordering` lesz — még akkor is, ha a gyakorlatban soha nem kerül `NaN` a mezőbe.

23.4.5 NaN, unordered és a hash konzisztencia

Két külön szerződés él egymás mellett. A **konténer-komparátor** csak a `<`-et használja, tehát strict weak orderinget kell adnia. A **hash-tároló** a `==`-t és a hash-t használja, és megköveteli: $a = b \Rightarrow h(a) = h(b)$.

A `NaN` mindkettőt megzavarja. A `std::map<Measurement, int>` `NaN` kulccsal nem definiált viselkedésű, mert a `<` nem strict weak ordering rajta. A `-0.0 == 0.0` viszont igaz, ezért a `-0.0` és a `0.0` hash-értékének **azonosnak** kell lennie; ha valaki `std::bit_cast<std::uint64_t>`-tal a nyers bitmintát hash-eli, ezt megsérti, és a `std::unordered_map` elveszíti a kulcsot.

A `<compare>` header C++20-ban `std::strong_order`, `std::weak_order` és `std::partial_order` customization pointokat kínál, amelyekkel a lebegőpontos értékek *teljes* rendezésbe kényszeríthetők. A `std::strong_order` az IEEE `totalOrder` predikátumhoz igazodik: megkülönbözteti a `-0.0`-t a `+0.0`-tól és a különböző `NaN` reprezentációkat is. Könyvtártámogatását érdemes compile probe-bal ellenőrizni (feature-test makró: `__cpp_lib_three_way_comparison`).

Itt egy tudatos döntést kell hozni: ha a `std::map` kulcsa `Measurement`, akkor a tároló ekvivalenciafogalma a `strong_order` szerinti lesz, ami **nem** esik egybe a `==` szerinti fogalommal (`-0.0` és `+0.0`). Ugyanarra a domainre ne keverjünk `map`-et `unordered_map`-pel, vagy dokumentáljuk, melyik a kanonikus reláció.

23.4.6 Pointerek és összetett domain értékek

A beépített pointer-összehasonlítás pointerekre C++20-ban is megengedett, de ha a két pointer nem ugyanazon tömb elemeire (illetve nem ugyanarra az objektumra) mutat, az eredmény *meghatározatlan* (unspecified), nem pedig konzisztens rendezés. Teljes rendezéshez `std::less<Pointer>` kell, amely garantáltan strict total ordert ad.

Összetett domain értéknél (például `Endpoint{addr, port}`) a tagonkénti, deklarációs sorrendű összehasonlítás általában pontosan az, amit akarunk: előbb a cím, aztán a port. Ha viszont a domain szemantikája más (például a port szerint akarunk elsődlegesen rendezni), a defaulted operátor rossz sorrendet ad — ilyenkor kézzel kell megírni, és gondoskodni kell a `==`-vel való konzisztenciáról.

23.4.7 Kidolgozott példa

A példa három domaint mutat: erős rendezésű verziószámot, gyenge rendezésű csomagnév-halmazt és egy lebegőpontos mérést. A buildparancs és a „várt” kimenet illusztráció; a kódot ebben a fejezetben nem futtattuk, így a kimenet nem mért eredmény.

```
// compare_demo.cpp  --  C++20
#include <algorithm>
#include <cctype>
#include <compare>
#include <cstddef>
#include <iostream>
#include <map>
#include <string>
#include <type_traits>
#include <unordered_set>

struct Version {
```

```
    int major = 0, minor = 0, patch = 0;
    bool operator==(const Version&) const = default;
    auto operator<=>(const Version&) const = default;   // std::strong_ordering
};

struct CaseInsensitiveLess {
    std::weak_ordering operator()(char a, char b) const noexcept {
        const auto la = static_cast<unsigned char>(
            std::tolower(static_cast<unsigned char>(a)));
        const auto lb = static_cast<unsigned char>(
            std::tolower(static_cast<unsigned char>(b)));
        return la <=> lb;                               // strong -> weak
    }
};

struct PackageName {
    std::string text;

    bool operator==(const PackageName& other) const noexcept {
        return (*this <=> other) == 0;                  // weak: ekvivalencia
    }
    std::weak_ordering operator<=>(const PackageName& other) const noexcept {
        return std::lexicographical_compare_three_way(
            text.begin(), text.end(),
            other.text.begin(), other.text.end(),
            CaseInsensitiveLess{});
    }
};

struct PackageNameHash {
    std::size_t operator()(const PackageName& p) const noexcept {
        std::size_t h = 14695981039346656037ull;        // FNV-1a offset basis
        for (char c : p.text) {
            h ^= static_cast<unsigned char>(
                std::tolower(static_cast<unsigned char>(c)));
            h *= 1099511628211ull;                      // FNV prime
        }
        return h;
    }
};

int main() {
    static_assert(std::is_same_v<decltype(Version{} <=> Version{}),
                                 std::strong_ordering>);

    std::map<PackageName, int> installed;
    installed[PackageName{"OpenSSL"}] = 1;
    installed[PackageName{"openssl"}] = 2;              // ugyanaz az osztály:
 ↪  felülír
    std::cout << installed.size() << '\n';              // várható: 1

    std::unordered_set<PackageName, PackageNameHash> seen;
```

```
    seen.insert(PackageName{"OpenSSL"});
    seen.insert(PackageName{"OPENSSL"});                // azonos hash ->
 ↪  ütközés
    std::cout << seen.size() << '\n';                   // várható: 1

    std::cout << (Version{1, 4, 2} < Version{1, 10, 0}) << '\n';  // várható: 1
}
```

```
g++-13 -std=c++20 -Wall -Wextra -Wpedantic -O2 -o compare_demo compare_demo.cpp
# vagy: clang++-17 -std=c++20 -Wall -Wextra -Wpedantic -O2 -o compare_demo
 ↪  compare_demo.cpp
```

Két megjegyzés a hordozhatósághoz. A `14695981039346656037ull` konstans 32 bites `std::size_t` esetén csonkolódik; ez nem hiba, csak eltérő hash-eloszlás. A `static_cast<unsigned char>` a `std::tolower` előtt kötelező, mert negatív `char` értéket átadva a viselkedés nem definiált.

A `NaN`-kezelés külön példa:

```
struct Measurement {
    double value;
    bool operator==(const Measurement&) const = default;
    auto operator<=>(const Measurement&) const = default;   //
     ↪  std::partial_ordering
};

struct TotalMeasurementOrder {
    bool operator()(const Measurement& a, const Measurement& b) const {
        return std::strong_order(a.value, b.value) < 0;     // teljes rendezés
    }
};
// std::map<Measurement, int, TotalMeasurementOrder>  -> definiált viselkedés
```

23.4.8 Tipikus hibák

A leggyakoribb, hogy valaki saját, nem defaultolt `operator<=>`-t ír, és csodálkozik, hogy az `==` nem fordul (defaultolt `<=>` mellett a fordító implicit `==`-t is deklarál). A második, hogy a `<=>`-t `bool`-ként próbálja használni `if`-ben. A harmadik, hogy `double` tagot tesz `std::map` kulcsába, és a `NaN` miatt nem definiált viselkedést kap; itt vagy szűrjük a bemenetet, vagy `std::strong_order`-t használó komparátort adunk. A negyedik, hogy a hash-t a nyers bitmintára írjuk, és elbukik a `-0.0`/`0.0` eset. Végül gyakori, hogy a felhasználó ír egy `operator<`-et, ami inkonzisztens a `==`-vel: az egyik szerint két érték egyenlő, a másik szerint az egyik kisebb — az ilyen kulcs a `std::map`-ben és `std::set`-ben is megkettőződik vagy elvész.

23.4.9 Összefoglaló

Az összehasonlítás nem esztétikai kérdés, hanem szerződés: a `==`-nek ekvivalenciarelációnak, a `<`-nek strict weak orderingnek kell lennie, és a hash-nek konzisztensnek kell lennie az egyenlőséggel. C++20-ban a defaulted `operator==` és `operator<=>` a tagonkénti eseteket ingyen adja, de a kettőt együtt kell deklarálni, és a visszatérési kategória (`strong`/`weak`/`partial`) megválasztása tudatos döntés. A `NaN` és a `-0.0` olyan részletek, amelyek a rendezést és a hash-t egyszerre törik el; a `<compare>` total-order segédfüggvényei jelentik a kiutat, ám ezek szemantikája eltér a `==`-tól, amit dokumentálni kell.

23.4.10 Feladatok

1. feladat (diagnosztika). Adott:

```
struct Point {
    int x = 0, y = 0;
    constexpr std::strong_ordering operator<=>(const Point& o) const {
        if (auto c = x <=> o.x; c != 0) return c;
        return y <=> o.y;
    }
};
```

A `p1 < p2` lefordul, a `p1 == p2` nem. Magyarázd meg, miért, és add meg a minimális javítást! Mi változna, ha a `<=> = default` lenne? Milyen kategóriát ad a `<=>`?

2. feladat (kódolás). Készíts `Endpoint` értéktípust `std::uint32_t addr` és `std::uint16_t port` tagokkal, lexikografikus erős rendezéssel és konzisztens hash-sel! Bizonyítsd (vagy ellenőrizd `static_assert`-tel), hogy a `<=>` kategóriája `std::strong_ordering`, és hogy az egyenlő értékek hash-e azonos!

3. feladat (diagnosztika + javítás). Egy mérési naplóhoz ezt a komparátort írták:

```
struct BadCmp {
    bool operator()(const Measurement& a, const Measurement& b) const {
        return a.value <= b.value;
    }
};
```

Miért nem definiált a `std::sort(begin, end, BadCmp{})` viselkedése? Adj két különböző javítást eltérő szemantikával, és mondd meg, melyiknél mi lesz a `NaN` és a `-0.0` sorsa!

23.4.11 Megoldások

1. megoldás. A `<=>` nem jelölt a `==` számára: a `p1 == p2` feloldásakor csak `operator==` (és fordítottja) jön szóba. Mivel a `<=>` itt saját, nem defaultolt függvény, a fordító nem deklarál mellé implicit `==`-t, tehát nincs használható jelölt (GCC 13.3: `no match for 'operator=='`). A minimális javítás egyetlen sor az osztályban:

```
bool operator==(const Point&) const = default;
```

Ha a `<=>` maga lenne `= default`, a javítás nem kellene: a defaultolt `<=>` implicit módon defaultolt `==`-t is deklarál. A kategória `std::strong_ordering`, mert ezt deklaráltuk, és az `int` tagok `<=>`-ja is az; defaultolt `<=>` esetén a `common_comparison_category` szintén ezt adná. Ellenőrzési feltétel — a javított osztályra:

```
static_assert(std::is_same_v<decltype(Point{} <=> Point{}),
                             std::strong_ordering>);
static_assert(Point{1, 2} != Point{1, 3});   // !(a == b) átírt jelöltként
static_assert(Point{1, 2} <  Point{1, 3});
```

Buildjegyzőkönyv (`lab-gcc` profil). A javított `Point` a fenti `static_assert`-ekkel `g++ -std=c++20 -Wall -Wextra -Wpedantic` alatt figyelmeztetés nélkül fordul; a `bool operator==` sor nélkül a `p1 == p2` a fenti hibát adja, a defaultolt `<=>`-os változaton pedig javítás nélkül is fordul.

2. megoldás.

```
#include <compare>
#include <cstdint>
#include <cstddef>
#include <type_traits>

struct Endpoint {
    std::uint32_t addr = 0;
    std::uint16_t port = 0;
    bool operator==(const Endpoint&) const = default;
    auto operator<=>(const Endpoint&) const = default;   // addr, majd port
};

struct EndpointHash {
    std::size_t operator()(const Endpoint& e) const noexcept {
        std::size_t h = static_cast<std::size_t>(e.addr);
        h ^= static_cast<std::size_t>(e.port) + 0x9e3779b9u
             + (h << 6) + (h >> 2);
        return h;
    }
};

static_assert(std::is_same_v<decltype(Endpoint{} <=> Endpoint{}),
                             std::strong_ordering>);
```

Indoklás: az integrális tagok `<=>`-ja `std::strong_ordering`, a tagonkénti lexikografikus összehasonlítás tranzitív és irreflexív, tehát strict weak ordering — sőt teljes rendezés az `(addr, port)` párokon. A hash csak a két tag értékétől függ, ezért $a = b \Rightarrow h(a) = h(b)$ automatikusan teljesül. Ellenőrzési feltétel: két azonos `(addr, port)` párra `a == b` igaz és `h(a) == h(b)`, valamint `!(a < b) && !(b < a)` is igaz.

3. megoldás. A `<=` nem irreflexív: `a <= a` igaz, így az 1. feltétel sérül, a komparátor nem strict weak ordering, a `std::sort` viselkedése nem definiált (jellemző tünet: túlindexelés vagy „érvénytelen komparátor” futásidejű diagnosztika).

Javítás A — szigorú `<`, `NaN` kizárása: `return a.value < b.value;` és a bemenet validálása (`std::isnan` esetén hiba vagy kihagyás). Itt a `NaN` be sem kerül, a `-0.0` és `0.0` ekvivalens marad, ami egyezik a `==`-vel.

Javítás B — teljes rendezés: `return std::strong_order(a.value, b.value) < 0;`. Ez minden `double`-re definiált, de a `-0.0` kisebb lesz a `+0.0`-nál, és a `NaN`-ok is helyet kapnak, tehát a tároló ekvivalenciafogalma eltér a `==`-tól. Ezt dokumentálni kell, és ugyanarra a kulcsra nem szabad `unordered_map`-et használni, mert ott az `==` dönt.

23.5 Osztály- és operátortervezési labor

Ez a labor a fejezet elveit fordítja át ellenőrizhető gyakorlatokká. Két típust építünk fel, mindkettőnél ugyanazzal a három lépéses módszerrel: előbb szándékosan rossz változatot adunk, azt interfészreview alá vetjük, majd a javítást fordítási és futásidejű feltételekkel igazoljuk. A két típus két tipikus hibatípust fed le: az `fd_handle` egy **erőforrás-burkot** (ownership), a `byte_count` egy **erős domain értéket** (konverziók és operátorok).

A laborkörnyezet: ISO C++20, explicit `-std=c++20`, rögzített GCC- vagy Clang-kiadás, x86-64 System V, Linux. Az `fd_handle` a POSIX `<unistd.h>` `::close`/`::read`/`::socket` függvényeit használja; más platformon ezt a megfelelő primitívre kell cserélni. Ajánlott fordítás:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -Werror -O2 \
    -fsanitize=address,undefined lab.cpp -o lab
```

A példák viselkedését a leírás a C++20 szabályai alapján adja meg; a tényleges lefuttatás a tanuló laborjának feladata, a jegyzet nem tartalmaz mérési eredményt.

23.5.1 Mit vizsgál az interfészreview

Az alábbi definíciók adják a labor ellenőrzőlistáját.

Erős domain típus (strong domain type): olyan osztály, amelynek reprezentációja és megengedett műveletei egy fogalmat írnak le, nem pedig egy általános gépi típust (például `std::size_t`), és amelynek nincsenek olyan implicit konverziói, amelyek fogalomhatárt lépnek át.

Kölcsönzött accessor (borrowed accessor): olyan tagfüggvény, amely a belső erőforrás nem tulajdonló handle-jét adja vissza. A visszaadott értéket a hívó nem zárhatja le, nem szabadíthatja fel, és nem tárolhatja a burkoló élettartamán túl.

Baleseti konverzió (accidental conversion): implicit átalakítás, amely egy értéket más jelentésű típusba visz anélkül, hogy a forráskódban bármi jelezné a szándékot. Ellentéte a **szándékos konverzió**, amely nevesített művelet: `bc.value()`, `static_cast<int>(x)`, explicit konstruktorhívás.

A review négy kérdése minden publikus tagra: (1) átad-e tulajdonjogot, és ha igen, kiderül-e a szignatúrából; (2) enged-e implicit konverziót, és az melyik irányba visz; (3) `noexcept`-e ott, ahol a standard tároló erre épít; (4) konzisztens-e az egyenlőség, a rendezés és a hash.

23.5.2 Az fd_handle ellenpéldája és javítása

A rossz változat mindössze annyit tesz, hogy a destruktorban lezárja az fd-t. A hiba nem a destruktorban van, hanem abban, amit a fordító *helyette* legenerál.

```
// BAD - tanítási ellenpélda, nem termelési kód.
#include <unistd.h>

class fd_handle_bad {
public:
    explicit fd_handle_bad(int fd) : fd_(fd) {}
    ~fd_handle_bad() { if (fd_ >= 0) ::close(fd_); }
    int get() const { return fd_; }          // bárki lezárhatja
    // nincs másolás/mozgatás deklarálva:
    // a fordító mindkettőt hallgatólagosan legenerálja
```

```
private:
    int fd_ = -1;
};

void demo_bad(int raw_fd) {
    fd_handle_bad h1{raw_fd};
    fd_handle_bad h2 = h1;   // hallgatólagos másolás: ugyanaz az fd
}                            // h2 és h1 destruktora is lezárja ugyanazt
```

Három külön hiba:

1. **Dupla lezárás.** A hallgatólagos másolás két „tulajdonost” hoz létre ugyanarra az fd-re. A destruktorok sorrendje miatt kétszer hívódik a `::close`. A második hívás legalább `EBADF`, de több szál esetén a szám újra kioszthatő, és egy másik szál élő fájlját zárja le — ez már nem `EBADF`, hanem csendes adatvesztés.
2. **A `get()` nem kölcsönzés, hanem nyitott ajtó.** A hívó `::close(h1.get())`-et írhat, majd a destruktor újra lezár.
3. **Mozgatás nélkül a konténer másol.** `std::vector<fd_handle_bad>` feltöltése másolással jár, mert a fordító a másoló konstruktort választja, ha nincs `noexcept` mozgató.

A javított változat szándékosan mindent kikényszerít, amit a rossz engedett:

```
#include <unistd.h>
#include <utility>

class fd_handle {
public:
    fd_handle() noexcept = default;
    explicit fd_handle(int fd) noexcept : fd_(fd) {}
    ~fd_handle() { reset(); }

    fd_handle(const fd_handle&)            = delete;   // egy owner
    fd_handle& operator=(const fd_handle&) = delete;

    fd_handle(fd_handle&& o) noexcept : fd_(o.release()) {}

    fd_handle& operator=(fd_handle&& o) noexcept {
        if (this != &o) reset(o.release());            // önmozgatás ellen véd
        return *this;
    }

    int  get() const noexcept { return fd_; }          // kölcsönzés
    explicit operator bool() const noexcept { return fd_ >= 0; }

    int  release() noexcept {                          // tulajdonjog átadása
        int tmp = fd_; fd_ = -1; return tmp;
    }
    void reset(int fd = -1) noexcept {                 // hibaágak egy helyen
        if (fd == fd_) return;                          // önmagára reset: nincs
         ↪ teendő
        if (fd_ >= 0) ::close(fd_);
        fd_ = fd;
    }
```

```
    void swap(fd_handle& o) noexcept { std::swap(fd_, o.fd_); }

private:
    int fd_ = -1;
};

inline void swap(fd_handle& a, fd_handle& b) noexcept { a.swap(b); }
```

A szerződés táblázatban:

Művelet	Szerződés	Indok
`fd_handle(int)`	explicit, átveszi a tulajdonjogot	a nyers fd ne maradjon a hívónál
`~fd_handle()`	lezár, ha nem üres	RAII
másoló műveletek	`= delete`	két owner → dupla lezárás
mozgató műveletek	`noexcept`, forrás üres lesz	konténer-átrakás másolás nélkül
`get()`	kölcsönzés, nem zárható le	a POSIX API nyers `int`-et vár
`release()`	tulajdonjog a hívóhoz kerül	nem RAII határ, pl. `dup2`
`reset(int)`	lezárja a régit, átveszi az újat	egyetlen hibakezelési pont
`swap()`	`noexcept`	`std::swap` és konténerbarát

A fordító által igazolható feltételek:

```
#include <type_traits>
static_assert(!std::is_copy_constructible_v<fd_handle>);
static_assert(!std::is_copy_assignable_v<fd_handle>);
static_assert( std::is_nothrow_move_constructible_v<fd_handle>);
static_assert( std::is_nothrow_move_assignable_v<fd_handle>);
static_assert( std::is_nothrow_destructible_v<fd_handle>);
static_assert(!std::is_convertible_v<fd_handle, int>);   // nincs baleseti
 ↪ konverzió
```

Futásidejű feltétel (a `<sys/socket.h>`-t is include-olva):

```
fd_handle a{::socket(AF_UNIX, SOCK_STREAM, 0)};
fd_handle b{std::move(a)};
assert(b.get() >= 0);
assert(a.get() == -1);   // a forrás üres: a destruktor nem zárja le újra
```

23.5.3 A byte_count: baleseti konverzió és overload-ambiguitás

A rossz verzió két irányban is nyit: nyers számból lesz `byte_count`, és `byte_count`-ból nyers szám.

```
// BAD - tanítási ellenpélda.
#include <cstddef>

struct byte_count_bad {
    byte_count_bad(std::size_t v) : value(v) {}                    // implicit
 ↪ be
```

```
    operator std::size_t() const { return value; }                  // implicit
     ↪ ki
    operator std::ptrdiff_t() const {                                 // implicit
     ↪ ki
        return static_cast<std::ptrdiff_t>(value);
    }
    std::size_t value;
};
```

Két következmény, mindkettő a labor egy-egy tesztje.

Baleseti konverzió. Mivel a konstruktor implicit, egy nyers `int` átcsúszik egy másik jelentésű függvénybe:

```
void set_retry_count(byte_count_bad);
int n = 5;
set_retry_count(n);   // int -> std::size_t -> byte_count_bad: lefordul
```

Az implicit konverziós sorozat szabálya ezt megengedi (standard + egy felhasználói + standard lépés), a forráskódban mégsem látszik, hogy darabszám-fogalomba nyers egész lépett be.

Overload-ambiguitás. A két konverziós operátor két egyenrangú jelöltet teremt:

```
void schedule(std::size_t);
void schedule(std::ptrdiff_t);
schedule(byte_count_bad{1024});   // mindkettő felhasználói konverzió: ambiguus
```

A javított típus csak a szándékos alakot engedi:

```
#include <compare>
#include <cstddef>
#include <functional>

class byte_count {
public:
    explicit constexpr byte_count(std::size_t v = 0) noexcept : v_(v) {}
    constexpr std::size_t value() const noexcept { return v_; }   // szándékos

    friend constexpr byte_count operator+(byte_count a, byte_count b) noexcept {
        return byte_count{a.v_ + b.v_};
    }
    friend bool operator==(byte_count a, byte_count b) = default;
    friend auto operator<=>(byte_count a, byte_count b) = default;

private:
    std::size_t v_;
};
```

A `value()` szándékosan nem konverziós operátor: a hívás helye a forráskódban látszik, így a review elkapja, ha valaki nyers méretet kever a domain típussal. Ugyanezt kényszeríti ki a fordító:

```
static_assert(!std::is_convertible_v<int, byte_count>);            // nincs be
static_assert(!std::is_convertible_v<byte_count, std::size_t>);    // nincs ki
static_assert( std::is_constructible_v<byte_count, std::size_t>);  // explicit
 ↪ igen
```

A gyakorlatban ez azt jelenti, hogy a `schedule(1024)` a `std::size_t` túlterhelést választja (nincs rejtett `byte_count`), a `schedule(byte_count{1024})` pedig pontos illeszkedés — nincs ambiguitás, mert csak egy jelölt életképes.

23.5.4 Egyenlőség, rendezés és hash konzisztencia

Az Egyenlőség, spaceship és rendezési kategóriák (23.4) alfejezetben rögzített szerződés itt is kötelező: két egyenlő értéknek ekvivalensnek kell lennie a rendezésben, és azonos hash-t kell adnia, különben a `std::unordered_set`/`unordered_map` elveszíti a beszúrt elemet.

```
template <> struct std::hash<byte_count> {
    std::size_t operator()(const byte_count& b) const noexcept {
        return std::hash<std::size_t>{}(b.value());
    }
};
```

A `= default` comparisons a `v_` tagra bontják a műveletet, ezért az `==`, a `<=>` és a fenti hash automatikusan összhangban marad, amíg a hash ugyanazt az egyetlen tagot használja. Ellenőrzés:

```
#include <cassert>
#include <unordered_set>

byte_count a{1024}, b{1024};
assert(a == b);
assert(!(a < b) && !(b < a));                                   // ekvivalens
assert(std::hash<byte_count>{}(a) == std::hash<byte_count>{}(b));
std::unordered_set<byte_count> s{a};
assert(s.contains(b));
```

Ha valaha második tag kerül az osztályba (például mértékegység), a `= default` comparisons azt is bevonják, de a hash specializációt **kézzel** kell frissíteni — ezt a review-nak külön ki kell kérnie.

23.5.5 Copy/move/destruction nyomkövetés

Az ownership-szerződést a legkönnyebbinstrumentált típussal látni. A nyomkövető minden életeseményt kiír; nem a formázás a lényeg, hanem hogy a műveletek sorrendje megfigyelhető legyen.

```
#include <cstdio>

struct tracked {
    int id;
    explicit tracked(int i) : id(i) { std::printf("ctor %d\n", id); }
    tracked(const tracked& o) : id(o.id) { std::printf("copy %d\n", id); }
    tracked(tracked&& o) noexcept : id(o.id) { o.id = -1; std::printf("move
 ↪  %d\n", id); }
    tracked& operator=(const tracked& o) { id = o.id; std::printf("copy= %d\n",
 ↪  id); return *this; }
    tracked& operator=(tracked&& o) noexcept { id = o.id; o.id = -1;
 ↪  std::printf("move= %d\n", id); return *this; }
    ~tracked() { std::printf("dtor %d\n", id); }
};
```

Két megfigyelendő pont: a mozgatott objektum Forrása `-1` azonosítójú lesz (a forrás „üres”), és hogy a `std::vector<tracked>` újrafoglalásakor a mozgatás történik, nem másolás — utóbbi feltétele a noexcept, swap és container-reallocation (22.5) alfejezetben tárgyalt `noexcept` mozgató konstruktor. Ugyanezt a sémát `fd_handle`-re alkalmazva a `printf` helyett a `reset`/`release` hívások számlálhatók.

23.5.6 Negatív fordítási példa — külön jelölve

Az alábbi blokk **szándékosan nem fordul le**; nem javítandó, hanem a hibaüzenet olvasását gyakorolja.

```
// NEGATÍV FORDÍTÁSI PÉLDA - nem fordul le, ez a cél.
#include <cstddef>

struct byte_count_bad {
    byte_count_bad(std::size_t v) : value(v) {}
    operator std::size_t() const { return value; }
    operator std::ptrdiff_t() const { return static_cast<std::ptrdiff_t>(value);
    ↪ }
    std::size_t value;
};

void schedule(std::size_t);
void schedule(std::ptrdiff_t);

void demo() { schedule(byte_count_bad{1024}); }
```

A várt diagnosztika jellege: a `schedule(byte_count_bad)` hívás **ambiguus**, mert mindkét jelölt azonos rangú felhasználói konverziót igényel. A javítás nem az overload-halmaz átírása, hanem a típus szűkítése: implicit konverziós operátor nélkül a hívás egyértelműen nem fordul le, ami itt a helyes eredmény — a programozó kénytelen kiírni a szándékát.

23.5.7 Tipikus hibák és javításuk

- **„Majd a destruktor rendbe tesz mindent.”** Nem: a hallgatólagos másolás két ownert csinál. Javítás: `= delete` a másoló műveletekre, `noexcept` mozgatás.
- **`get()` helyett `release()`-et használni mindenhol.** A `release()` tulajdonjogot ad át, a hívó felelőssége lesz a lezárás — ez csak valódi nem-RAII határon indokolt.
- **Származtatott `byte_count` implicit konverziós operátorral.** Egy „kényelmi” `operator std::size_t()` visszahozza az összes fenti hibát. Javítás: `value()`.
- **Hash specializáció elfelejtése.** A `std::unordered_set` a `==`-t és a hash-t együtt használja; eltérés esetén az elem „eltűnik”.
- **Önmozgatás kezeletlenül.** `h = std::move(h)` a `reset(o.release())` sorrend miatt lezárhatja a saját fd-t. Javítás: `if (this != &o)`.

23.5.8 Feladatok

1. feladat (könnyű). Add hozzá az `fd_handle`-hez egy `borrowed()` nevű `const noexcept` tagot, amely ugyanazt adja vissza, mint a `get()`, de a neve is jelzi a szerződést. Igazold `static_assert`-tel, hogy a `fd_handle` továbbra sem konvertálható `int`-té, és hogy mozgatása `noexcept`.

2. feladat (közepes). Egészítsd ki a `byte_count`-ot `operator+=`-vel, amely **csak** `byte_count`-ot fogad el. Mutasd meg egy külön, jelölt negatív példában, hogy `bc += 1` nem fordul le, majd egy működő példában, hogy `bc += byte_count{1}` igen. Indokold, miért nem elég a `value()`-t használni a belső művelethez.

3. feladat (diagnosztikai). Készíts egy `checked_byte_count` változatot, amely `operator+`-ban `std::size_t`-túlcsordulást ellenőriz (`a.value() > SIZE_MAX - b.value()`), és ilyenkor `std::overflow_error`-t dob. Írj hozzá három esetet: normál összeadás, pontosan `SIZE_MAX`, és túlcsordulás. Egészítsd ki egy review-mondattal arról, hogy a dobó művelet milyen exception guarantee-t ad.

23.5.9 Megoldások

1. megoldás. A `borrowed()` a `get()` átnevezett változata; a szerződés nem változik, csak a hívási hely lesz önmagyarázó.

```
class fd_handle {
public:
    // ...
    int get() const noexcept { return fd_; }
    int borrowed() const noexcept { return fd_; }   // ugyanaz, beszélő névvel
};
static_assert(!std::is_convertible_v<fd_handle, int>);
static_assert(std::is_nothrow_move_constructible_v<fd_handle>);
```

Ellenőrzési feltétel: a két accessor bitre azonos értéket ad, és a `static_assert`-ek változtatás nélkül átmennek. Megjegyzés: ha `get()`-et megtartod, a két név egyike felesleges — termelési kódban egyiket töröld, és a hívási helyeket igazítsd.

2. megoldás. Az `operator+=` szándékosan tagfüggvény, hogy a bal oldal típusa adott legyen:

```
class byte_count {
public:
    // ...
    byte_count& operator+=(byte_count o) noexcept { v_ += o.v_; return *this; }
};
```

Negatív példa (nem fordul le, mert nincs implicit `int` → `byte_count`):

```
// NEGATÍV - a byte_count explicit konstruktora miatt nem fordul le.
byte_count bc{1024};
bc += 1;              // hiba: nincs implicit konverzió
```

Működő példa:

```
byte_count bc{1024};
bc += byte_count{1};  // helyes: a szándék látszik
assert(bc == byte_count{1025});
```

Indoklás: ha a belső műveletet `v_ += o.value()`-ként írnánk, az csak a reprezentáción belül azonos; a különbség abban van, hogy az `operator+=` **a publikus felületen** kényszeríti ki a típust, nem a belső kódban. A `value()` használata a hívónál megengedné a nyers szám keverését, amit épp ki akarunk zárni.

3. megoldás. Az ellenőrzött összeadás:

```cpp
#include <cstddef>
#include <limits>
#include <stdexcept>

class checked_byte_count {
public:
    explicit constexpr checked_byte_count(std::size_t v = 0) noexcept : v_(v) {}
    constexpr std::size_t value() const noexcept { return v_; }

    friend checked_byte_count operator+(checked_byte_count a, checked_byte_count
     ↪ b) {
        if (a.v_ > std::numeric_limits<std::size_t>::max() - b.v_)
            throw std::overflow_error{"byte_count overflow"};
        return checked_byte_count{a.v_ + b.v_};
    }
    friend bool operator==(checked_byte_count, checked_byte_count) = default;
    friend auto operator<=>(checked_byte_count, checked_byte_count) = default;

private:
    std::size_t v_;
};
```

Tesztesetek:

```cpp
assert((checked_byte_count{1} + checked_byte_count{2}) ==
 ↪  checked_byte_count{3});
constexpr auto m = std::numeric_limits<std::size_t>::max();
assert((checked_byte_count{m - 1} + checked_byte_count{1}).value() == m);   //
 ↪  pontosan MAX
bool threw = false;
try { (void)(checked_byte_count{m} + checked_byte_count{1}); }
catch (const std::overflow_error&) { threw = true; }
assert(threw);
```

Review-mondat: az `operator+` **nem** ad `noexcept`-ot, mert hiba esetén dob; az erős exception guarantee teljesül, mivel a művelet vagy új értéket ad vissza, vagy dob, a bemeneteit nem módosítja. A `byte_count` egyszerű összeadása ehelyett `noexcept` és `unsigned` wrappolással túlcsordul — a két változat szándékosan más szerződést képvisel, és a választást a hívó kontextusa indokolja.

23.5.10 Összefoglaló

A labor két típussal mutatta meg ugyanazt a mintát. Az `fd_handle`-nél a `= delete` másoló műveletek, a `noexcept` mozgatás és a nevesített `release()` teszik kényszeríthetővé az egy-owner szerződést; a `get()`/`borrowed()` kölcsönzés marad, mert a POSIX API nyers `int`-et vár. A `byte_count`-nál az `explicit` konstruktor és a konverziós operátor elhagyása szünteti meg a baleseti konverziót és az overload-ambiguitást; a `value()` a szándékos kilépőpont. Mindkettőnél a `= default` comparisons és egyetlen tagot használó hash adja az `==`/`<=>`/`hash` konzisztenciát. Az interfészreview tehát nem esztétikai kérdés: minden implicit konverzió és minden hiányzó `noexcept` egy potenciális hibát enged be a publikus felületen, amelyet a fordító `static_assert`-tel, a futtatás pedig nyomkövető típussal tesz láthatóvá.

Forrás és ellenőrzési korlát. Ehhez az egységhez nem állt rendelkezésre feldolgozott forrásjegyzet, ezért a szöveg a C++20 nyelvi szabályaira és a bemutatott kód saját szerkezetére

támaszkodik; konkrét szabványparagrafust vagy mérési eredményt nem állít. A tanuló laborjában a C++20 munkatervezet (N4950), a hivatalos GCC/Clang dokumentáció és a C++ Core Guidelines megfelelő szakaszai ellenőrizhetik a részleteket; a fejezettervben megadott URL-ek kiindulópontok, nem bizonyítékok.

24 Öröklés, virtuális függvények és OOP

24.1 Öröklés, base subobject és hozzáférés

A fejezet a C++20 nyelvi szabályaira támaszkodik. A konkrét szabványparagrafusok, az ABI-dokumentumok és a compiler-viselkedés ellenőrzése a labor feladata; a megadott forráskeresési kiindulópontok önmagukban nem minősülnek ellenőrzött bizonyítéknak. Ahol a viselkedés implementációfüggő vagy nem specifikált, azt külön jelöljük.

Az öröklés nem elsősorban kódujrafelhasználás, hanem típus-szerződés: a leszármaztatott osztály objektuma a bázisosztály által leírt helyen használhatóvá válik. Ez a helyettesíthetőség (substitutability) adja az értékét, és ez szabja meg a feltételeit is. A következőkben azt nézzük meg, hogyan épül fel egy leszármaztatott objektum a memóriában és az élettartama során, milyen hozzáférési szabályok érvényesek a bázis és a leszármazott között, és mikor helyes egyáltalán öröklést használni.

24.1.1 Az öröklés mint helyettesíthetőségi szerződés

A nyilvános öröklés (`public inheritance`) azt fejezi ki, hogy a leszármaztatott típus minden objektuma egyben a bázis típus objektuma is: a `Derived` egy `Base`. A nyelvi következmény az, hogy egy `Derived*` vagy `Derived&` implicit módon `Base*`-gé, illetve `Base&`-é alakítható, és a bázisra írt kód a leszármazott objektumot is kezelheti.

Ez azonban csak akkor helyes, ha a leszármazott betartja a bázis szerződését. A bázis osztály a nyilvános műveleteivel nemcsak szignatúrákat, hanem elvárásokat is rögzít: milyen előfeltételekkel hívható egy művelet, milyen posztfeltételt garantál, milyen kivételt enged meg, és mi marad invariáns. A leszármazott nem gyengítheti az előfeltételeket, nem erősítheti a posztfeltételeket, és nem vezethet be olyan megfigyelhető viselkedést, amelyre a bázisra építő kód nem számít. Ez a Liskov-helyettesítési elv gyakorlati tartalma.

Tekintsünk egy tipikus sérülést. A bázis így deklarál:

```
struct Shape {
    virtual double area() const noexcept = 0;
    virtual ~Shape() = default;
};
```

A `noexcept` itt szerződéselem: a bázis azt ígéri, hogy az `area()` nem dob kivételt. Ha egy leszármazott mégis dob, akkor a bázisra írt kód, amely nem készült kivételkezelésre, `std::terminate`-hez juthat. A leszármazott tehát nem helyettesíthető minden olyan helyen, ahol a bázist használják. A javítás nem az, hogy a hívóoldalon mindenhol kivételkezelést teszünk, hanem hogy vagy a leszármazott tartja be a szerződést, vagy a bázis szerződését változtatjuk meg tudatosan, minden leszármazottra kiterjedően.

A nyilvános öröklés tehát erős kapcsolat: a bázis nyilvános felülete a leszármazott felülete is lesz. Ezért a bázis nyilvános interfészét úgy kell kialakítani, hogy az valóban minden lehetséges leszármazottra érvényes legyen. Ha csak implementációt szeretnénk újrafelhasználni, és nem

állítjuk, hogy a leszármazott „egy” bázis, akkor kompozíció vagy nem nyilvános öröklés a helyes eszköz.

24.1.2 Bázis-alobjektum és leszármaztatott objektum

A leszármaztatott objektum egyetlen objektum, amely tartalmazza a bázis-osztály alobjektumát (`base class subobject`) és a saját nem statikus adattagjait. A bázis-alobjektumnak saját élettartama van, de ez az élettartam a leszármaztatott objektum élettartamának része: nem önálló objektum, hanem a nagyobb objektum egy darabja. A `Base*` vagy `Base&` erre az alobjektumra mutat, nem egy különálló `Base` példányra.

Ebből következik néhány gyakorlati szabály. A `Derived*` `Base*`-gé alakítása nyilvános öröklésnél implicit, és a bázis-alobjektum címét adja. A visszafelé irányuló `Base*` → `Derived*` átalakítás nem implicit; `static_cast`-tal csak akkor biztonságos, ha a mutatott objektum valóban `Derived`, különben a viselkedés nem definiált. A futásidejű ellenőrzést a `dynamic_cast` adja, amelynek részletei a RTTI-ről szóló részhez tartoznak.

A memóriabeli elrendezésről pontosat kell mondanunk. A szabvány a nem statikus adattagok sorrendjét azonos hozzáférési szekción belül a deklaráció sorrendjéhez köti, de a bázis-alobjektum és a tagok egymáshoz képesti sorrendjét, a kitöltést (`padding`) és az üres bázis optimalizálását (`empty base optimization`, EBO) nem rögzíti. Ezért konkrét offseteket csak dokumentált target ABI és fordítási profil mellett szabad állítani. Az `offsetof` makró csak standard-layout típusra használható; polimorfikus osztályra nem. A `sizeof(Derived)` legalább `sizeof(Base)`, de a kitöltés és az EBO miatt nem egyszerű összeg.

A bázis-alobjektum léte magyarázza a szeletelést (`object slicing`) is. Ha egy `Derived` objektumot érték szerint `Base` típusba másolunk, akkor csak a bázis-alobjektum másolódik; a leszármaztatott rész levágódik. Az így létrejött `Base` objektum dinamikus típusa `Base`, a virtuális hívások a bázis verzióját használják, és a leszármaztatott invariáns elveszik. Érték szerinti átadás bázis típusú paraméternek ezért polimorfikus használatban hibás; referenciát vagy pointert kell használni.

24.1.3 Konstrukció és destrukció sorrendje

A leszármaztatott objektum felépítése szigorú sorrendben történik. A konstruktor mem-initializer listájában a bázis konstruktora fut először; ha nincs megadva, a bázis alapértelmezett konstruktora. A teljes sorrend a következő:

1. a virtuális bázis-alobjektumok, a mélységi bejárás és a deklarációs sorrend szerint;
2. a közvetlen, nem virtuális bázis-osztályok, a deklarációs sorrendben;
3. a nem statikus adattagok, a deklarációs sorrendben;
4. a konstruktor teste.

A virtuális bázisok részletei a többszörös öröklésről szóló részhez tartoznak; itt azt rögzítsük, hogy a bázis minden esetben a tagok és a konstruktor test előtt épül fel. A destrukció pontosan fordított sorrendben történik: először a leszármaztatott destruktor teste, majd a leszármaztatott tagok, majd a bázis destruktor teste, végül a bázis tagjai.

A sorrend indoklása az invariánsokban rejlik. A leszármaztatott konstruktor testében a bázis-alobjektum és a saját tagok már érvényes állapotban vannak, tehát a leszármaztatott invariáns felállítható. A bázis konstruktorában viszont a leszármaztatott rész még nem épült fel; ezért ott a dinamikus típus a bázis, és egy virtuális függvény hívása a bázis verzióját választja, nem a leszármaztatott felülírást. Ha a bázis konstruktorában tisztán virtuális függvényt hívunk, a

viselkedés nem definiált. A bázis konstruktorában a leszármaztatott adattagokra hivatkozni szintén hibás, mert azok még nem jöttek létre.

A destruktoroknál ugyanez fordítva igaz: a leszármaztatott destruktor testében a leszármaztatott tagok még élnek, a bázis destruktorában már nem. Ezért a bázis destruktorának nem szabad olyan virtuális hívást feltételeznie, amely a leszármaztatott részt érinti.

Ha a bázis rendelkezik virtuális destruktorral, akkor a bázis pointeren keresztül történő `delete` a teljes objektumot, azaz a leszármaztatott destruktort is lefuttatja. Ha nem, a viselkedés nem definiált. Ennek részletes tárgyalása a virtuális destruktorról szóló részhez tartozik; itt a sorrend szempontjából az a lényeg, hogy a bázis destruktora a leszármaztatott rész után fut.

24.1.4 Hozzáférési szabályok és az öröklés módja

Az öröklés módja (`public`, `protected`, `private`) és a bázis tagjainak hozzáférési szintje együtt határozza meg, hogy a leszármazottból és a külvilágból mi érhető el. A szabály táblázatban:

A bázis tagja	`public` öröklés	`protected` öröklés	`private` öröklés
`public`	`public`	`protected`	`private`
`protected`	`protected`	`protected`	`private`
`private`	nem érhető el	nem érhető el	nem érhető el

A „nem érhető el” azt jelenti, hogy a leszármazott tagfüggvényei sem érik el közvetlenül a bázis privát tagjait; kivételt csak a bázis által megadott `friend` jelent. A hozzáférés fordítási idejű szabály, nem futásidejű: nem attól függ, hogy a pointer dinamikusan milyen típusra mutat.

A nyilvános öröklés a „is-a” kapcsolatot fejezi ki, és kifelé is láthatóvá teszi a bázis nyilvános felületét. A privát öröklés a „implementációt újrahasznosítok” kapcsolat, de kifelé nem teszi láthatóvá a bázis felületét; a `Derived*` → `Base*` konverzió a leszármazott tagjai számára elérhető, kívülről nem. A védett öröklés ritka: a bázis felülete a további leszármazottak felé válik elérhetővé, a külvilág felé nem.

A gyakorlatban a privát öröklés helyett többnyire kompozíciót használunk, mert az kifejezőbb és kevésbé kötődik a bázis reprezentációjához. A védett öröklés pedig akkor merül fel, ha egy köztes absztrakciót akarunk a hierarchián belül tartani; ilyenkor is érdemes megvizsgálni, hogy a kompozíció nem elegendő-e.

24.1.5 Tagelrejtés, `using` és névfeloldás

A leszármaztatott osztályban deklarált név elrejti a bázisban található összes azonos nevű tagot, nem csak az azonos szignatúrájút. Ha a bázisnak van `describe()` és `describe(int)` tagfüggvénye, és a leszármazottban deklarálunk egy `describe(int)`-et, akkor a leszármazottban a `describe` név feloldása megáll a leszármazott hatókörében, és a bázis `describe()` nem lesz látható. Ez gyakran meglepetést okoz.

A `using Base::describe;` deklaráció behozza a bázis tagját a leszármazott hatókörébe, így az overload-halmaz egyesül. A `using` azonban nem tudja megkerülni a bázis hozzáférési szabályait: ha a bázis tagja privát, a `using` nem teszi elérhetővé. A `using` által bevezetett név hozzáférési szintjét a `using` helye is befolyásolja; egy nyilvános szekcióban elhelyezett `using` a bevezetett nevet nyilvánossá teheti, de a bázis tagjának eredeti hozzáférése továbbra is számít.

A tagelrejtés nem azonos a virtuális felülírással. A virtuális függvény felülírása a dinamikus dispatch révén a leszármazott verziót hívja a bázis pointeren keresztül is; a nem virtuális tag elrejtése csak statikus névfeloldást érint. A két mechanizmus együtt is előfordulhat, ezért érdemes a bázis felületét úgy tervezni, hogy a nevek ne ütközzenek véletlenül.

24.1.6 Kompozíció és „is-a” szemantika

A kompozíció (`has-a`) és az öröklés (`is-a`) közötti választás nem esztétikai kérdés. Ha a leszármazott valóban minden helyen helyettesítheti a bázist, és a bázis szerződése minden leszármazottra érvényes, akkor a nyilvános öröklés indokolt. Ha csak egy részimplementációt szeretnénk átvenni, vagy a bázis belső reprezentációjára támaszkodunk, akkor a kompozíció a biztonságosabb.

A kompozíció lazább kapcsolatot ad: a tartalmazott objektum cserélhető, tesztelhető, és nem kell a bázis minden nyilvános műveletét örökölni. Az öröklés ezzel szemben a bázis felületének változásaira érzékeny, és a leszármazottak száma növeli a karbantartási terhet. A „prefer composition over inheritance” alapértelmezés, nem tilalom: a polimorfikus helyettesítéshez továbbra is öröklés (vagy azzal egyenértékű típus-erasure) kell.

DevOps szempontból külön figyelmet érdemel, hogy a C++ osztálylayout és a virtuális dispatch implementációfüggő. Shared library ABI-határon ne adjunk át C++ osztályokat `extern "C"` interfészen keresztül, és ne építsünk a bázis-alobjektum konkrét offsetjére. A stabil C API mögött a C++ hierarchia elrejthető, de a határon a szerződést és a layoutot dokumentálni kell.

24.1.7 Védett interfész és a törékeny bázisosztály

A `protected` tagok a leszármazottak számára készült belső interfészt alkotják. Ez hasznos lehet, ha a leszármazottnak valóban szüksége van egy műveletre, de a külvilágnak nem. A védett adattagok azonban különösen kockázatosak: a leszármazott közvetlenül függ a bázis reprezentációjától, így a bázis belső változása a leszármazottakat is eltöri.

Ezt nevezzük törékeny bázisosztály-problémának (`fragile base class problem`). A bázis látszólag ártalmatlan változása – egy új overload, egy tag átnevezése, az invariáns módosítása – a leszármazottak viselkedését is megváltoztathatja. A védekezés: a védett felület minimalizálása, védett adattag helyett védett művelet, a nem virtuális interfész (`non-virtual interface`, NVI) minta alkalmazása, ahol a nyilvános nem virtuális függvény ellenőrzi az előfeltételeket, és egy védett vagy privát virtuális függvényt hív. A kompozíció szintén csökkenti a függőséget.

A mély öröklési hierarchia önmagában nem cél; a sok szint növeli a törékenységet és a megértési költséget. Ahol a hierarchia csak kódujrafelhasználásra szolgál, ott érdemes kompozícióra vagy sablonokra váltani.

24.1.8 Kidolgozott példa

A következő példa egy bázis- és leszármaztatott osztályt mutat be, amely kiírja a konstrukció és a destrukció sorrendjét, szemlélteti a tagelrejtést, a `using` deklarációt és a védett tag elérését. A kód önállóan fordítható C++20-ban.

```
// base_subobject_demo.cpp
#include <iostream>
#include <string>
#include <utility>
```

```cpp
class Resource {
public:
    explicit Resource(std::string name) : name_(std::move(name)) {
        std::cout << "  Resource ctor: " << name_ << '\n';
    }
    ~Resource() { std::cout << "  Resource dtor: " << name_ << '\n'; }
    Resource(const Resource&) = delete;
    Resource& operator=(const Resource&) = delete;
private:
    std::string name_;
};

class Base {
public:
    explicit Base(std::string tag) : r_("Base/" + tag) {
        std::cout << "  Base ctor body\n";
    }
    virtual ~Base() { std::cout << "  Base dtor body\n"; }

    void describe() const { std::cout << "Base::describe\n"; }
    int value() const { return 1; }

protected:
    void hook() const { std::cout << "Base::hook\n"; }

private:
    Resource r_;
};

class Derived : public Base {
public:
    Derived() : Base("d"), r_("Derived") {
        std::cout << "  Derived ctor body\n";
    }
    ~Derived() override { std::cout << "  Derived dtor body\n"; }

    using Base::describe;               // behozza Base::describe()-t
    void describe(int) const { std::cout << "Derived::describe(int)\n"; }

    int value() const { return 2; }     // elrejti Base::value()-t

    void call_hook() const { hook(); } // protected bázis-tag elérése

private:
    Resource r_;
};

int main() {
    std::cout << "-- construction --\n";
    Derived d;
    std::cout << "-- calls --\n";
    d.describe();          // Base::describe a using miatt
```

```
    d.describe(42);        // Derived::describe(int)
    std::cout << "d.value() = " << d.value() << '\n';
    std::cout << "d.Base::value() = " << d.Base::value() << '\n';
    d.call_hook();
    std::cout << "-- destruction --\n";
}
```

A fordításhoz és futtatáshoz például:

```
g++ -std=c++20 -Wall -Wextra -pedantic -o base_subobject_demo
 ↪  base_subobject_demo.cpp
./base_subobject_demo
```

Vagy Clangdal:

```
clang++ -std=c++20 -Wall -Wextra -pedantic -o base_subobject_demo
 ↪  base_subobject_demo.cpp
./base_subobject_demo
```

A várt kimenet illusztráció, nem mért eredmény:

```
-- construction --
  Resource ctor: Base/d
  Base ctor body
  Resource ctor: Derived
  Derived ctor body
-- calls --
Base::describe
Derived::describe(int)
d.value() = 2
d.Base::value() = 1
Base::hook
-- destruction --
  Derived dtor body
  Resource dtor: Derived
  Base dtor body
  Resource dtor: Base/d
```

A kimenet sorrendje pontosan mutatja a konstrukció és destrukció fordított viszonyát: a `Derived` teste előtt a bázis és a bázis tagja épül fel, a destrukcióban pedig a `Derived` teste és tagja után a bázis következik. A `d.describe()` a `using` miatt a bázis verzióját hívja, a `d.describe(42)` a leszármazottét; a `d.value()` a leszármazott által elrejtett változat, a `d.Base::value()` pedig minősített hívással a bázisé. A `call_hook()` a védett bázis-tagot a leszármazott tagfüggvényén keresztül éri el.

24.1.9 Gyakori hibák

Az egyik leggyakoribb hiba, hogy a bázis destruktora nem virtuális, és a bázis pointeren keresztül törlünk. Ez nem definiált viselkedés; a leszármaztatott destruktor nem feltétlenül fut le. A másik a szeletelés: `Base b = d;` esetén a leszármaztatott rész levágódik, és a polimorfikus viselkedés elveszik. Gyakori a tagelrejtés figyelmen kívül hagyása: a leszármazottban deklarált név elrejti a bázis összes azonos nevű tagját, amit `using` deklarációval vagy minősített hívással lehet kezelni.

Hibás a privát öröklés mellett kívülről `Base*`-gé alakítani: a konverzió nem érhető el. Ugyancsak hiba a bázis konstruktorában a leszármaztatott tagokra vagy a leszármaztatott virtuális felülírásra támaszkodni; a dinamikus típus ott még a bázis. Végül a polimorfikus típusra alkalmazott `offsetof` és a konkrét offsetekre építő szerializálás nem hordozható, mert a layout nem specifikált.

24.1.10 Feladatok

24.1.10.1 1. feladat: helyettesíthetőség

Adott a következő bázis:

```
struct Shape {
    virtual double area() const noexcept = 0;
    virtual ~Shape() = default;
};
```

Egy `Bad : Shape` leszármazott `area()` implementációja kivételt dob. Magyarázd meg, miért sérti ez a helyettesíthetőségi szerződést, és hogyan javítanád a hibát. Legalább két javítási irányt adj.

24.1.10.2 2. feladat: privát öröklés és hozzáférés

A kidolgozott példában változtasd meg a `Derived` öröklését `private`-re. Írd át a `main`-t úgy, hogy kívülről próbáld meg `Base* p = &d;`-t használni. Mit tapasztalsz? Úgy javítsd a kódot, hogy a `Derived` továbbra is elérje a `Base::hook()`-ot, de kifelé ne legyen `Base` konverzió. Indokold a választ.

24.1.10.3 3. feladat: diagnosztika

Adott a következő kód:

```
#include <iostream>

class B {
public:
    B() { std::cout << "B\n"; }
    ~B() { std::cout << "~B\n"; }
    virtual void f() const { std::cout << "B::f\n"; }
};

class D : public B {
public:
    D() { std::cout << "D\n"; }
    ~D() { std::cout << "~D\n"; }
    void f() const override { std::cout << "D::f\n"; }
};

int main() {
    B* p = new D;
    p->f();
    delete p;
}
```

Mi a hiba, miért nem definiált a viselkedés, és hogyan javítod? A javítás után milyen sorrendben várhatók a destruktorok?

24.1.11 Megoldások

24.1.11.1 1. megoldás

A bázis `noexcept` szerződése azt ígéri, hogy az `area()` nem dob kivételt. A `Bad::area()` ezt megszegi, ezért a bázisra írt kód, amely nem számít kivételre, `std::terminate`-hez juthat. A helyettesíthetőség sérül: a `Bad` nem használható mindenütt, ahol `Shape`-et várnak.

Javítási irányok: (a) a `Bad::area()` ne dobjon, hanem kezelje a hibát a szerződésnek megfelelően; (b) ha a hiba mégis kivétel, akkor a bázis szerződését kell megváltoztatni – eltávolítani a `noexcept`-ot –, de ez minden leszármazottra és minden hívóra kihat; (c) ha a `Bad` valójában nem „egy” `Shape`, hanem csak egy részimplementációt használ, akkor kompozícióra kell váltani. Ellenőrzési feltétel: a `Shape`-re írt kód forduljon le, és a `Bad` ne idézzen elő nem várt `std::terminate`-et.

24.1.11.2 2. megoldás

Privát öröklésnél a `Derived` továbbra is eléri a `Base` védett `hook()`-ját a saját tagfüggvényeiből, de a `Derived*` → `Base*` konverzió kívülről nem elérhető. A `Base* p = &d;` sor ezért nem fordul le. A belső használathoz a `Derived` tagfüggvényében lehet `static_cast<Base*>(this)`-t használni, de kívülről nem.

A javított kód lényege:

```
class Derived : private Base {
public:
    Derived() : Base("d") {}
    void call_hook() const { hook(); } // belül elérhető
};
```

A `main`-ben a `Base* p = &d;` továbbra sem fordul le; ez szándékos. Ha kifelé mégis kell bázisinterfész, akkor vagy nyilvános öröklésre váltunk, vagy egy külön, szűk interfészt adunk ki. Ellenőrzési feltétel: a `call_hook()` működjön, a külső `Base*` konverzió pedig fordítási hibát adjon.

24.1.11.3 3. megoldás

A hiba a `B` nem virtuális destruktora. A `delete p;` a `B*` statikus típusán keresztül töröl, és mivel a destruktor nem virtuális, a viselkedés nem definiált; a `D` destruktora nem feltétlenül fut le. A `p->f()` ezzel szemben helyesen a `D::f`-et hívja, mert az `f` virtuális.

A javítás:

```
class B {
public:
    B() { std::cout << "B\n"; }
    virtual ~B() { std::cout << "~B\n"; }
    virtual void f() const { std::cout << "B::f\n"; }
};
```

A javítás után a `delete p;` a teljes objektumot törli: először a `D` destruktor teste fut le, majd a `B` destruktor teste, tehát a várt sorrend `~D`, majd `~B`. Ellenőrzési feltétel: a program a `-Wall -Wextra -pedantic` kapcsolókkal figyelmeztetés nélkül forduljon, és a destruktorok sorrendje a fenti legyen.

24.1.12 Összefoglaló

A nyilvános öröklés helyettesíthetőségi szerződés: a leszármazott mindenütt használható a bázis helyett, de csak akkor, ha a bázis szerződését betartja. A leszármaztatott objektum tartalmazza a bázis-alobjektumot; a bázis pointer vagy referencia erre az alobjektumra mutat. A konstrukció sorrendje a bázisok, majd a tagok, végül a konstruktor test; a destrukció pontosan fordított. A bázis konstruktorában a dinamikus típus még a bázis, ezért a leszármaztatott részre támaszkodni hibás. Az öröklés módja és a tagok hozzáférési szintje együtt szabja meg, mi érhető el a leszármazottból és kívülről. A tagelrejtést `using` deklarációval vagy minősített névvel lehet kezelni. A védett interfész és a törékeny bázisosztály kockázata miatt a védett felületet minimalizálni kell, és ahol csak implementáció-újrafelhasználás a cél, kompozíciót érdemes választani.

24.2 Virtual, override, final és dispatch

A korábbi részekben az osztályokat értéktípusként, kompozícióval és védett invariánssal építettük fel, és láttuk, hogy az öröklés önmagában még nem ad dinamikus viselkedést: a bázisról elérhető tagok statikusan oldódnak fel (Öröklés, base subobject és hozzáférés). Most azt a szerződést vesszük, amely egy interfészt egy nyitott implementációhalmazzal köt össze: a virtuális függvényt. A cél nem az, hogy mindent hierarchiába szervezzünk — a sekély, egyértelmű hierarchia jó tervezési tanács, de nem nyelvi szabály —, hanem hogy pontosan lássuk, mikor dől el egy hívás célja, mi történik a névfeloldással és az alapértelmezett argumentumokkal, és mi az, amit a fordító optimalizálhat, de nem garantál. A dinamikus polimorfizmus ára egy indirekció a hívás helyén; ezt a költséget később, a virtuális hívás költségmodelljénél mérjük (Template, virtual és std::function költsége).

24.2.1 A virtuális függvény és a dinamikus típus

Egy kifejezés **statikus típusa** (static type) az, ahogyan a fordító a deklaráció alapján látja; a **dinamikus típus** (dynamic type) a hivatkozott objektum leginkább származtatott típusa. Precíz definíció: ha egy glvalue egy polimorf objektumra hivatkozik, akkor a dinamikus típusa annak a leginkább származtatott objektumnak a típusa, amelynek a használt objektum alobjektuma.

A dinamikus dispatch feltételei együtt teljesülnek:

1. a hívott függvény virtuális, és a hívás nem minősített (`b.f()`, nem `b.Base::f()`);
2. a hívás implicit objektumargumentuma referencián vagy pointeren keresztül érkezik — azaz a statikus típus mellett létezhet eltérő dinamikus típus;
3. az objektum élettartama már elkezdődött és még nem ért véget (a konstrukció és destrukció alatti részleges kivételt lásd később).

Ha ezek teljesülnek, akkor a hívás a dinamikus típusban érvényes **final overrider**-t hajtja végre. Ha bármelyik feltétel sérül — például érték szerinti példányon hívunk —, a statikus típus függvénye fut. Érték szerinti átadásnál ez egyben szeletelést is jelent; a szeletelés és a virtuális destruktor részletei a Virtuális destruktor, slicing és ownership részben vannak.

```cpp
// ISO C++20, hosted; illusztráció, nem mért futtatás
#include <iostream>

struct Shape {
    virtual ~Shape() = default;
    virtual double area() const = 0;
};

struct Circle : Shape {
    explicit Circle(double r) : r_(r) {}
    double area() const override { return 3.14159265358979323846 * r_ * r_; }
private:
    double r_;
};

void report(const Shape& s) { std::cout << s.area() << '\n'; }

int main() {
    Circle c(1.0);
    Shape& ref = c;      // statikus típus: Shape&, dinamikus: Circle
    report(c);           // dinamikus dispatch: Circle::area
    Shape sliced = c;    // nem fordul le: Shape absztrakt, tehát itt szeletelés
 ↪  sem lehetséges
}
```

A példa utolsó sora szándékosan nem fordul le: absztrakt bázisból nem készíthető példány, így a leggyakoribb szeletelési csapda itt fordítási hibaként jelenik meg. Ez jó tulajdonság: a tiszta interfész kizárja a véletlen érték szerinti másolást.

24.2.2 override, final és az aláírás egyezése

Az **aláírás** (signature) a névből, a paramétertípus-listából, valamint a cv- és ref-minősítőkből áll; a visszatérési típus nem része. A felülírás két lépésben dől el (C++20 [class.virtual]). (1) **Felülír-e?** Egy származtatott tagfüggvény akkor írja felül a bázis virtuális függvényét, ha neve, paramétertípus-listája (azonos típusok, nem csak konvertálhatók), cv- és ref-minősítője egyezik; a visszatérési típus ebben a lépésben nem számít. Ha a név egyezik, de a paraméterlista vagy a minősítés eltér, a függvény nem override, hanem **elfedi** (name hiding) a bázis azonos nevű tagjait. (2) **Érvényes-e a felülírás?** Ha a függvény felülír, további követelményeknek is meg kell felelnie: a visszatérési típus legyen azonos vagy **kovariáns** (osztálypointer/-referencia, ahol a származtatott visszatérési típus a bázisé egyértelmű, elérhető leszármazottja), és ne sértse a `final`-t. Az eltérő, nem kovariáns visszatérési típus tehát nem name hiding, hanem fordítási hiba — `override` nélkül is.

Az `override` kulcsszó nem változtat a szemantikán: szerződéssé teszi a szándékot, és a fordítót ellenőrző eszközzé. Ha a függvény nem ír felül semmit, fordítási hiba keletkezik — pontosan ez a kívánt viselkedés, mert a csendben elrejtett virtuális függvény az egyik leggyakoribb dinamikus polimorfizmus-hiba.

```cpp
struct Base {
    virtual void draw() const;
    virtual ~Base() = default;
};
```

```cpp
struct Bad : Base {
    void draw();                    // NEM override: hiányzik a const; elrejti
     ↪ Base::draw-t
};

struct Good : Base {
    void draw() const override;  // fordítási hiba lenne, ha nem egyezne az
     ↪ aláírás
};
```

A második lépés hibája külön próbát érdemel, mert itt az `override` hiánya sem segít elrejteni a problémát:

```cpp
// override_return_error.cpp - C++20: szándékosan hibás (negatív próba)
struct Base {
    virtual int  id() const;
    virtual Base* clone() const;
    virtual ~Base() = default;
};
struct Derived : Base {
    long     id() const;            // hiba: felülír, de a visszatérési típus
     ↪ nem
                                    // azonos és nem kovariáns ('override'
                                     ↪ nélkül is)
    Derived* clone() const override; // helyes: kovariáns visszatérési típus
};
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -c override_return_error.cpp` elutasítja: `conflicting return type specified for 'virtual long int Derived::id() const'`, `overridden function is 'virtual int Base::id() const'`. A kovariáns `clone` nem kap diagnosztikát. Ezzel szemben a fenti `Bad::draw()` (hiányzó `const`) nem felülír, ezért ott nincs hiba, csak elfedés — erre figyelmeztet a `-Woverloaded-virtual`.

A `final` két helyen állhat: függvényen azt tiltja meg, hogy további származtatott osztály felülírja; osztályon azt, hogy abból származtassanak. Szemantikailag nem változtat a hívás szabályán, csak a további felülírást zárja ki — ez viszont az optimalizálás számára hasznos információ (lásd a devirtualizációt). A GCC és a Clang `-Woverloaded-virtual` figyelmeztetése nem ISO-szabály, hanem compiler-diagnosztika, amely éppen az elrejtéses hibát jelzi.

24.2.3 Tiszta virtuális függvény és absztrakt osztály

A **tiszta virtuális** (pure virtual) függvény deklarációja `= 0` inicializálót tartalmaz. Az ilyen függvényt tartalmazó osztály **absztrakt** (abstract): nem példányosítható, de pointer és referencia készíthető hozzá. Egy származtatott osztály akkor konkrét, ha minden örökölt tiszta virtuális függvényre van érvényes felülírója. A tiszta virtuális függvénynek lehet definíciója is; az csak minősített névvel hívható (`Base::f()`), a dinamikus hívásba nem kerül be.

Az interfész-osztály tipikus formája: csak tiszta virtuális függvények, virtuális destruktor és lehetőleg semmilyen adattag. A virtuális destruktor kötelező jellegéről, a bázispointeren keresztüli `delete` nem definiált viselkedéséről a Virtuális destruktor, slicing és ownership rész szól; itt csak annyit rögzítünk, hogy a destruktor virtualitása nem a dispatch-hez, hanem a törlés helyességéhez kell.

24.2.4 Statikus névfeloldás és dinamikus dispatch lépései

Egy hívás feloldása rétegekre bontható; a rétegek sorrendje magyarázza a látszólag meglepő viselkedéseket.

Lépés	Mikor történik	Mit határoz meg	Mi alapján
Névfeloldás	fordításkor	mely deklarációk jönnek szóba	a statikus típus hatóköre (+ ADL)
Overload resolution	fordításkor	melyik deklaráció lesz a hívott	argumentumok statikus típusa
Alapértelmezett argumentum	fordításkor	a hiányzó argumentumok értéke	a kiválasztott deklaráció
Hozzáférés-ellenőrzés	fordításkor	legális-e a hívás	a kiválasztott deklaráció access-specifierje
Célfüggvény kiválasztása	futáskor	melyik test fut	a dinamikus típus final overridere (ha virtuális)

Ebből két következmény adódik, amelyeket érdemes külön megjegyezni.

Az alapértelmezett argumentum statikusan kötődik. A virtuális dispatch a testet választja ki, az alapértelmezett argumentumot nem.

```
#include <iostream>

struct Base {
    virtual void log(int level = 0) const { std::cout << "B" << level << '\n'; }
    virtual ~Base() = default;
};
struct Derived : Base {
    void log(int level = 1) const override { std::cout << "D" << level << '\n';
     ↪ }
};

void demo(Base& b, Derived& d) {
    b.log();           // Derived::log fut, de a default argumentum 0 (Base
 ↪ deklarációjából)
    d.log();           // Derived::log, default argumentum 1
    d.Base::log();     // minősített hívás: nincs dinamikus dispatch, Base::log
 ↪ fut
}
```

A `b.log()` hívásnál a statikus típus `Base&`, ezért az overload resolution a `Base::log` deklarációt választja, és innen veszi a `level = 0` alapértéket; a virtuális dispatch viszont a `Derived::log` testét futtatja. A `d.Base::log()` minősített hívás, amely kikapcsolja a dinamikus dispatch-et — ez hasznos akkor, amikor a bázis implementációját akarjuk „láncoltan” meghívni.

A származtatott osztály neve elrejtheti a bázis overloadjait. Ha a származtatott osztályban bármely tagot deklarálunk egy néven, a bázis összes azonos nevű tagja elrejtőzik, kivéve, ha `using Base::f;`-fel behozzuk őket.

```
struct B { virtual void f(int); virtual ~B() = default; };
struct D : B {
    void f(double);          // nem override (más paramétertípus), és elrejti
     ↪ B::f(int)-et
    // using B::f;           // e nélkül a D-n keresztül az f(int) nem látszik
};
```

A **kovariáns visszatérés** (covariant return) lehetővé teszi, hogy a felülíró függvény visszatérési típusa a báziséval kompatibilis, szűkebb osztályra mutató pointer vagy referencia legyen. Feltételek: mindkét típus osztályra mutató pointer vagy referencia, a származtatott visszatérési osztály a bázis visszatérési osztály leszármazottja, a konverzió egyértelmű és elérhető, a cv-minősítés nem enyhül. Okos pointerrel ez nem működik, mert a `std::unique_ptr<Derived>` nem leszármazottja a `std::unique_ptr<Base>`-nek.

```
struct Node {
    virtual Node* clone() const = 0;
    virtual ~Node() = default;
};
struct Expr : Node {
    Expr* clone() const override { return new Expr(*this); }  // kovariáns
};
```

24.2.5 Konstrukció és destrukció alatti dispatch

A konstruktor és a destruktor testében a dinamikus típus nem a „végleges” osztály, hanem az éppen futó konstruktor, illetve destruktor osztálya. Ennek az az indoka, hogy a származtatott adattagok a bázis konstruktorának futásakor még nem jöttek létre, a bázis destruktora pedig már a származtatott rész megszűnése után fut; ha ilyenkor a származtatott override hívódna, az nem létező tagokat érne el.

```
#include <iostream>
struct Base {
    Base() { init(); }                      // itt Base::init fut, nem
 ↪ Derived::init
    virtual void init() { std::cout << "Base\n"; }
    virtual ~Base() = default;
};
struct Derived : Base {
    void init() override { std::cout << "Derived\n"; }
};
// Derived d;  ->  a várt (nem mért) kimenet: "Base"
```

Indoklás vázlata. Tegyük fel, hogy a bázis konstruktorából a származtatott override hívódna. Ekkor a hívás olyan tagot érne el, amelynek élettartama még nem kezdődött el, ami önmagában nem definiált viselkedés lenne; a nyelv ezért a konstrukció alatt futó osztályt tekinti dinamikus típusnak, és ezzel a részleges objektumot konzisztens állapotban tartja. Ugyanez érvényes a destruktorokra, fordított sorrendben.

Gyakorlati következmény: virtuális függvényt konstruktorból vagy destruktorból hívni csak akkor biztonságos, ha a bázisértékű viselkedés helyes — tipikusan tiszta virtuális esetén ez nem definiált viselkedés. A „regisztráció a konstruktorban” minta ezért C++-ban hibás; a regisztrációt külön, a teljes objektum létrejötte utáni lépésre kell tenni.

24.2.6 Devirtualizáció: optimalizálás, nem garancia

A fordító bizonyos esetekben **devirtualizálhatja** a hívást, azaz statikus hívássá alakíthatja. Tipikus esetek: a dinamikus típus a fordítási egységben bizonyíthatóan ismert (lokális objektum, `final` osztály vagy függvény, névtelen névtérbe zárt típus), egészprogramos optimalizálás (LTO), illetve spekulatív devirtualizáció típusellenőrzéssel. Ez a hívás megfigyelhető viselkedését nem változtathatja meg — a szabvány szerinti szemantika ugyanaz —, csak a költségét. Következmény: a teljesítményre nem lehet *helyességi* garanciát építeni, és a devirtualizáció meglétére nem lehet API-szerződést alapozni; az csak a laborban mért, konkrét buildre érvényes megfigyelés lehet.

24.2.7 Kidolgozott példa: interfész és több implementáció

Az alábbi, önmagában fordítható példa egy interfészt és két konkrét implementációt mutat, `override`-tal, `final`-lal és `clone()`-nal.

```
// shapes.cpp - ISO C++20, hosted, Linux
#include <iostream>
#include <memory>
#include <string>
#include <vector>

class Shape {
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
    virtual std::string name() const = 0;
    virtual std::unique_ptr<Shape> clone() const = 0;
};

class Circle final : public Shape {
public:
    explicit Circle(double r) : r_(r) {}
    double area() const override { return 3.14159265358979323846 * r_ * r_; }
    std::string name() const override { return "circle"; }
    std::unique_ptr<Shape> clone() const override {
        return std::make_unique<Circle>(*this);   // Circle final: nincs további
         ↪ dinamikus típus
    }
private:
    double r_;
};

class Rectangle : public Shape {
public:
    Rectangle(double w, double h) : w_(w), h_(h) {}
    double area() const override { return w_ * h_; }
    std::string name() const override { return "rectangle"; }
    std::unique_ptr<Shape> clone() const override {
        return std::make_unique<Rectangle>(*this);
    }
private:
```

```
    double w_, h_;
};

void print_all(const std::vector<std::unique_ptr<Shape>>& xs) {
    for (const auto& p : xs) {
        std::cout << p->name() << ' ' << p->area() << '\n';  // dinamikus
        ↪ dispatch
    }
}

int main() {
    std::vector<std::unique_ptr<Shape>> v;
    v.push_back(std::make_unique<Circle>(1.0));
    v.push_back(std::make_unique<Rectangle>(2.0, 3.0));
    print_all(v);
}
```

Fordítás és a figyelmeztetések ellenőrzése (a tanuló laborjában; a jelen szöveg nem tartalmaz futtatási eredményt):

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -Wnon-virtual-dtor -Woverloaded-virtual
 ↪ shapes.cpp -o shapes
clang++ -std=c++20 -Wall -Wextra -Wpedantic shapes.cpp -o shapes
```

A `print_all` szignatúrája szándékosan `const std::unique_ptr<Shape>&`: a tároló nem tulajdonolja a mutatókat, a `Shape` objektumok tulajdonosa a `vector` és a benne lévő `unique_ptr`-ek. A `clone()` visszatérési típusa itt nem kovariáns, mert okos pointerrel nem lehet az; a kovariáns változatot nyers pointerrel fentebb láttuk.

24.2.8 Gyakori hibák

- Hiányzó `override`: a fordító nem jelzi, hogy a függvény csak elrejt, nem felülír. Javítás: mindig `override`, és `-Woverloaded-virtual`.
- Eltérő `const` vagy paramétertípus: `void f(int)` vs. `void f(long)` nem override, hanem elrejtés; a hívás a statikus típuson keresztül a bázisra esik vissza.
- Alapértelmezett argumentumra építés a dinamikus típuson: a default a statikus deklarációból jön, nem a dinamikus típusból.
- Virtuális hívás konstruktorból/destruktorból a származtatott viselkedés reményében: a bázis verziója fut.
- Bázis overloadjainak elrejtése származtatott osztályban `using` nélkül: a bázis változatai nem látszanak a származtatott statikus típuson keresztül.

24.2.9 Feladatok

1. **Magyarázó.** Adott: `struct B { virtual void f() const; virtual ~B() = default; }; struct D : B { void f(); };` Mi történik `B& b = d; b.f();` esetén, és mit tenne a fordító, ha a `D::f`-re `override`-ot írnánk? Különböztesd meg a névfeloldást a dispatch-től.
2. **Kódolási.** Egészítsd ki a `shapes.cpp`-t egy `Triangle` osztállyal (`final`, `clone`-nal), és egy `total_area(const std::vector<std::unique_ptr<Shape>>&)` szabad függvénnyel. Legyen a `Rectangle`-ből származó `Square` osztály, amely felülírja a `name()`-et, és maga is

`final`. Ellenőrzési feltétel: a build a fenti kapcsolókkal figyelmeztetés nélkül lefut, és egy szándékosan `override` nélkül hagyott felülírás fordítási hibát okoz.

3. **Diagnosztikai.** Az alábbi kódrészletben három, a fejezetben tárgyalt hiba van. Keresd meg őket, indokold, majd javítsd a kódot.

```
struct Base {
    Base() { setup(); }
    virtual void setup() {}
    virtual void log(int level = 0) const {}
    virtual ~Base() = default;
};
struct Derived : Base {
    void setup() override {}
    void log(int level = 1) const {}   // hiányzik az override
    void log(double) {}                // elrejti a bázis log-jait
};
```

4. **Érvelő.** Egy kollégád azt állítja, hogy egy `final` osztályra építve „biztonságosan optimalizálhatunk, mert a hívás biztosan statikus lesz”. Fogalmazd meg pontosan, mi igaz ebből, és mi az, ami nem garantálható.

24.2.10 Megoldások

1. megoldás. A `D::f()` hiányzó `const` miatt nem override, hanem elrejtés: a `D` statikus típusán keresztül az `f()` hívás a `D::f`-et jelöli, de a `B&`-n keresztül a névfeloldás a `B::f` deklarációt választja, és mivel az virtuális, a dinamikus típus `D` — csakhogy a `D`-nek nincs érvényes overridere, ezért a `B::f` fut. A névfeloldás és az overload resolution fordításkor, a statikus típus alapján történik; a dispatch csak a kiválasztott virtuális deklaráció után lép be. Ha a `D::f`-re `override`-ot írunk, a program nem fordul le, mert nincs felülírható bázisfüggvény — ez a kívánt diagnosztika.

2. megoldás. A `Triangle`:

```
class Triangle final : public Shape {
public:
    Triangle(double a, double b, double c) : a_(a), b_(b), c_(c) {}
    double area() const override {
        const double s = (a_ + b_ + c_) / 2.0;
        return s * (s - a_) * (s - b_) * (s - c_) > 0.0
             ? std::sqrt(s * (s - a_) * (s - b_) * (s - c_)) : 0.0;
    }
    std::string name() const override { return "triangle"; }
    std::unique_ptr<Shape> clone() const override {
        return std::make_unique<Triangle>(*this);
    }
private:
    double a_, b_, c_;
};

class Square final : public Rectangle {
public:
    explicit Square(double side) : Rectangle(side, side) {}
    std::string name() const override { return "square"; }
};
```

A `total_area` a `print_all` mintájára iterál, és `double`-t ad vissza. Indoklás: a `Square` a `Rectangle`-től örököl, de a `Rectangle` nincs `final`-lal lezárva, ezért a származtatás legális; a `Square` saját `final`-ja további származtatást tilt. Ellenőrzési feltétel: a `-Wall -Wextra -Wpedantic -Woverloaded-virtual` kapcsolókkal a build hibátlan, és a `clone() override` jelölése nélkül a `-Wsuggest-override` (GCC) jelzi a hiányt; a `Square::name override` nélkül szintén elrejtés lenne.

3. megoldás. Hibák: (a) a `Base` konstruktora virtuális `setup()`-ot hív, ezért a `Derived::setup` sosem fut le a konstrukció alatt — a `Derived` állapota inicializálatlan maradhat, ha a `setup` a származtatott tagokra épül; (b) a `Derived::log(int) override` nélkül is felülír, de a jelölés hiánya elrejti a szándékot, és egy jövőbeli aláírás-változás csendben elrejtéssé alakítja; (c) a `Derived::log(double)` elrejti a bázis összes `log` tagját, így `Derived` statikus típuson az `int`-es változat nem látszik. Javítás: a `setup()` hívást a konstruktorból kivenni, és a teljes objektum létrejötte után explicit módon hívni; a `log(int)`-re `override`; a `log(double)` elé `using Base::log;`, vagy a két overload összevonása. Ellenőrzési feltétel: a javított kód a figyelmeztetés-kapcsolókkal tiszta, és a `Base&`-en keresztüli `log()` hívás a `Derived` testét futtatja, de a `Base` deklarációjából vett `level = 0` alapértékkel.

4. megoldás. Igaz, hogy a `final` osztály vagy függvény esetén a fordító *bizonyíthatja* a dinamikus típust, és devirtualizálhat; ez azonban optimalizálási döntés, nem nyelvi garancia. Nem garantálható, hogy egy adott buildben a hívás tényleg statikus lesz, és az sem, hogy a költség eltűnik: a fordító dönthet a spekulatív devirtualizáció mellett is, illetve egészprogramos optimalizálás nélkül nem láthatja a teljes programot. A helyességet nem érinti, mert a devirtualizáció megfigyelhető viselkedést nem változtathat; a teljesítményre vonatkozó állítást csak mérés támaszthatja alá, konkrét compiler- és buildprofillal.

24.2.11 Összefoglaló

A virtuális függvény szerződése: a hívás neve és overloadja a statikus típus alapján dől el, a testet a dinamikus típus final overridere adja — de csak akkor, ha a hívás nem minősített, a függvény virtuális, referencia vagy pointer az objektumargumentum, és az objektum élettartama teljes. Az `override` a fordítót teszi ellenőrzővé, a `final` a további felülírást zárja ki. Az alapértelmezett argumentumok és a névfeloldás statikusak, ezért a dinamikus dispatch nem „örökíti át” őket; a konstrukció és destrukció alatt a dinamikus típus az éppen futó osztály. A devirtualizáció optimalizálás, nem szerződés: a teljesítményre csak mérés, a helyességre a nyelvi szabály ad garanciát.

24.3 Virtuális destruktor, slicing és ownership

Egy heterogén objektumgyűjtemény, amelyet egyetlen interfészen keresztül kezelünk, a modern C++ egyik leggyakoribb tervezési alakzata: naplózó sinkek, protokoll-adapterek, parancs-objectek. A gyűjtemény tulajdonosa tipikusan `std::unique_ptr<Base>`, a bázis pedig absztrakt interfész. Az ilyen kód helyessége két, egymástól független szerződésen múlik: a destruktornak a dinamikus típus szerint kell lefutnia, és a származtatott objektumnak nem szabad értékként bázis típusba másolódnia. A fejezet ezt a két szerződést tárgyalja, majd összeveti a polimorfikus másolást (`clone`) a move-only tulajdonosi interfésszel. Az előfeltétel a virtuális dispatch és a `unique_ptr` egyedi tulajdonlás ismerete; az ott rögzített fogalmakat itt nem ismétlem meg.

Definíció (polimorfikus törlés). Polimorfikus törlésnek nevezzük azt a `delete` kifejezést, amelynek statikus típusa bázis pointer, dinamikus típusa pedig származtatott osztály. A művelet akkor definiált, ha a statikus típus destruktora virtuális, vagy ha a statikus és a dinamikus típus

megegyezik. Ha a statikus típus destruktora nem virtuális, és a dinamikus típus eltér, a viselkedés nem definiált (undefined behavior, UB).

Definíció (object slicing). Szeletelésről akkor beszélünk, ha egy származtatott objektumot érték szerint másolunk vagy érték szerinti paraméterként adunk át bázis típuson. Ilyenkor a bázis másoló konstruktora fut, csak a bázis részobjektum másolódik, és az eredmény dinamikus típusa a bázis lesz. Ez nem UB, hanem logikai hiba: a származtatott tagok és a származtatott viselkedés elvesznek.

24.3.1 A destruktorhívás szerződése és a `delete` bázis pointeren

A `virtual ~Base() = default;` deklaráció azt jelenti, hogy a bázis destruktora virtuális, és a származtatott destruktorok automatikusan felülírják, akkor is, ha a származtatott osztályban nem írjuk ki a `virtual` kulcsszót. A destruktor neve a származtatottban `~Derived`, de a virtuális dispatch a destruktorra ugyanúgy működik, mint bármely más virtuális függvényre. Ezért a bázisban elég egyszer kimondani a virtualitást; a hierarchia minden tagja örökli.

A `delete p` kifejezés végrehajtása két lépésből áll: először a destruktor fut le, majd a tároló felszabadítása történik. Ha a statikus típus destruktora virtuális, a destruktorhívás a dinamikus típushoz megy, és a teljes objektum destruálódik. Ha nem virtuális, és az objektum dinamikus típusa eltér a statikus típustól, a `delete` viselkedése **nem definiált (UB)**: a szabvány a megsértett feltételt (a statikus és a dinamikus típus különbözik, a bázis destruktora nem virtuális) és az UB-t mondja ki, futási forgatókönyvet nem ír elő. Egy konkrét implementációnál megfigyelhető például, hogy a bázis destruktora fut le, a származtatott tagok destruktorai elmaradnak, és a deallokáció a bázis méretével történik — de ez csak megfigyelés (heap-korrupció, memóriaszivárgás, összeomlás egyaránt előfordulhat), nem garancia.

Fontos, hogy a `std::unique_ptr<Base>` nem véd meg ettől. A `std::make_unique<Derived>()` által visszaadott `unique_ptr<Derived>` implicit módon konvertálódik `unique_ptr<Base>`-vé, ha a `Derived*` konvertálható `Base*`-ra. A kód lefordul, és a `unique_ptr` destruktora a `default_delete<Base>`-en keresztül `delete`-et hív a statikus `Base*` típuson. Ha a `Base` destruktora nem virtuális, a törlés futásidőben UB. A fordítási siker tehát nem bizonyíték a helyességre; a virtuális destruktor a szerződés része, nem a fordító által kikényszerített feltétel.

A következő tanítási példa szándékosan hibás, és UB-t mutat be. UB esetén nincs garantált kimenet, ezért a példa csak a hiba okát illusztrálja.

```
// s119_ub_delete.cpp - tanítási példa: UB, nincs garantált output
#include <string>

struct Base {
    ~Base() = default;   // NEM virtuális
};

struct Derived : Base {
    std::string payload;
};

int main() {
    Base* p = new Derived;
    delete p;   // UB: a statikus és a dinamikus típus különbözik, a Base
     ↪  destruktora nem virtuális
}
```

A helyes változatban a bázis destruktora `virtual ~Base() = default;`. Ekkor a `delete p` a `Derived::~Derived()`-et hívja, és a `payload` felszabadul; a deallokáció a teljes objektumra vonatkozik (hogy ehhez méretparaméteres `operator delete` hívás történik-e, az implementáció dolga, nem kötelező).

24.3.2 Két biztonságos destruktor-minta

A polimorfikus tulajdonláshoz két bevett minta létezik. A választás nem esztétikai kérdés, hanem a tulajdonlási szerződés része.

Az első a **public virtual destruktor**. Ez akkor helyes, ha a bázis pointeren keresztül történő törlés megengedett, és a bázis valóban polimorfikus tulajdonosi interfész. A `std::unique_ptr<Base>` és a factory-minta ezt igényli. A destruktor legyen `public`, hogy a `delete` és a `default_delete` hozzáférjen; legyen `virtual`, hogy a dinamikus típus destruálódjon.

A második a **protected nonvirtual destruktor**. Itt a destruktor nem virtuális, de `protected`, ezért `delete Base*` nem fordul le, mert a destruktor nem elérhető. A származtatott osztály destruktora viszont eléri a bázis destruktorát, így a `Derived` példány törölhető. A minta célja annak kikényszerítése, hogy a bázist kizárólag származtatott típuson keresztül töröljék, például mert a bázisnak nincs virtuális destruktora, vagy mert a törlést a származtatott osztály végzi. A C++ Core Guidelines ajánlásai között is szerepel ez a minta; a konkrét ajánlás szövegének ellenőrzése a labor feladata.

Minta	Deklaráció	`delete Base*`	`unique_ptr<Base>`	Mikor jó
public virtual	`public: virtual ~Base() = default;`	legális, dinamikus destruktor	legális	polimorfikus tulajdonlás
protected nonvirtual	`protected: ~Base() = default;`	fordítási hiba	fordítási hiba	csak származtatottként törölhető

A táblázat utolsó oszlopa a lényeg: a protected nonvirtual destruktor nem használható `std::unique_ptr<Base>`-bel, mert a `default_delete<Base>` nem fér hozzá a destruktorhoz, ezért a `unique_ptr<Base>` destruktora nem fordul le. Ha mégis ilyen tulajdonosi típus kell, egyedi deleter vagy barát osztály szükséges; ez azonban már nem a legegyszerűbb interfész.

```
// s119_protected_dtor.cpp
class Base {
public:
    virtual void run() = 0;
protected:
    ~Base() = default;   // nem virtuális, de protected
};

class Derived final : public Base {
public:
    void run() override {}
    ~Derived() = default;
};
```

```
int main() {
    Derived d;          // OK
    // Base* p = new Derived;
    // delete p;        // fordítási hiba: ~Base() protected
}
```

24.3.3 Object slicing és pass-by-value base

A szeletelés leggyakoribb forrása az érték szerinti paraméter. A `void f(Base b)` szignatúra a hívás pillanatában a `Base` másoló konstruktorát hívja, még akkor is, ha a hívó `Derived` objektumot ad át. A `void f(const Base& b)` ezzel szemben referenciát köt, nem másol, ezért megőrzi a dinamikus típust. Ugyanez a különbség a `Base*` és a `Base&` között: a pointer és a referencia nem szeletel, az érték igen.

A szeletelés nem UB, ezért a sanitizerek és a fordító figyelmeztetései gyakran nem jelzik. A tünet az, hogy a virtuális hívás a bázis verziójára megy, a származtatott tagok elvesznek, és a `sizeof` a bázis méretét mutatja. A `typeid`-vel való vizsgálat nem megbízható detektor, mert a szeletelt objektum dinamikus típusa valóban a bázis; a hiba már a másolás pillanatában megtörtént. A legjobb védelem a szerződés szintjén van: az interfész bázist érdemes absztraktként tartani, mert absztrakt bázisból nem lehet példányt létrehozni, így a `Base b = d;` már nem fordul le. Kiegészítő védelem a bázis másoló konstruktorának törlése (`Base(const Base&) = delete;`), de ez a bázist másolhatatlanná teszi, ami nem minden esetben kívánatos.

```
// s119_slicing.cpp - tanítási példa: nem UB, de logikai hiba
#include <iostream>
#include <string_view>

class Shape {
public:
    virtual ~Shape() = default;
    virtual std::string_view name() const { return "Shape"; }
    virtual double area() const { return 0.0; }
};

class Circle final : public Shape {
public:
    explicit Circle(double r) : r_(r) {}
    std::string_view name() const override { return "Circle"; }
    double area() const override { return 3.141592653589793 * r_ * r_; }
private:
    double r_;
};

void print_by_value(Shape s) { std::cout << s.name() << '\n'; }
void print_by_ref(const Shape& s) { std::cout << s.name() << '\n'; }

int main() {
    Circle c{2.0};
    print_by_value(c);  // Shape
    print_by_ref(c);    // Circle
    Shape s = c;        // slicing
```

```
    std::cout << s.name() << '\n'; // Shape
}
```

A példában a `Shape` nem absztrakt, ezért a `Shape s = c;` lefordul. Ha a `name` és az `area` pure virtual (`= 0`) lenne, a `Shape` absztrakt lenne, és a szeletelés már fordításkor elbukna. Ez a legolcsóbb, legkorábbi védelem.

24.3.4 Polymorphic clone és move-only interface owner

A polimorfikus tulajdonlásnak két alapalakja van. A **move-only interface owner** esetében a tulajdonos `std::unique_ptr<Base>`, amelyet a factory ad vissza, és a hívó `std::move`-val ad tovább. Ez a forma akkor elég, ha a tulajdonlás átadása a cél; a másolás nem szükséges, és a `unique_ptr` mozgatása olcsó. A **polymorphic clone** akkor kell, ha a tulajdonosról másolatot kell készíteni, például prototípus, konfiguráció vagy párhuzamos feldolgozás miatt. A `clone` a dinamikus típust őrzi meg, ellentétben a bázis érték szerinti másolásával.

A `clone` visszatérési típusa `std::unique_ptr<Base>` legyen, ne nyers `Base*`. A nyers pointer visszaadása a hívóra hárítja a `delete`-et, és a `unique_ptr`-be csomagolás könnyen elmarad, ami szivárgáshoz vezet. A `unique_ptr<Base>` visszatérési típus a tulajdonlást a típusba kódolja. A `clone` visszatérési típusa nem lehet kovariáns, mert a `unique_ptr` nem pointer típus; a származtatott `clone` visszatérési típusa pontosan `std::unique_ptr<Base>` kell legyen, és a `make_unique<Derived>(*this)` implicit konverziója végzi a típuskonverziót.

A következő példa önállóan fordítható C++20-ban; a laborprofilban rögzített GCC vagy Clang verzióval a megadott buildparanccsal építhető. A példa nem állítja, hogy futásidőben tesztelték; a viselkedés a forrásból következik.

```
// s119_sink.cpp - C++20, önállóan fordítható
#include <iostream>
#include <memory>
#include <string>
#include <string_view>
#include <utility>
#include <vector>

class Sink {
public:
    virtual ~Sink() = default;
    virtual void write(std::string_view s) = 0;
    virtual std::unique_ptr<Sink> clone() const = 0;
};

class StdoutSink final : public Sink {
public:
    void write(std::string_view s) override { std::cout << s; }
    std::unique_ptr<Sink> clone() const override {
        return std::make_unique<StdoutSink>(*this);
    }
};

class PrefixSink final : public Sink {
public:
    explicit PrefixSink(std::string prefix) : prefix_(std::move(prefix)) {}
```

```
    void write(std::string_view s) override { std::cout << prefix_ << s; }
    std::unique_ptr<Sink> clone() const override {
        return std::make_unique<PrefixSink>(*this);
    }
private:
    std::string prefix_;
};

std::unique_ptr<Sink> make_sink(std::string_view kind) {
    if (kind == "stdout") return std::make_unique<StdoutSink>();
    if (kind == "prefix") return std::make_unique<PrefixSink>("[log] ");
    return nullptr;
}

void emit(Sink& sink, std::string_view msg) { sink.write(msg); }  // borrowed;
 ↪ write nem const

int main() {
    auto sink = make_sink("prefix");
    emit(*sink, "hello\n");

    auto copy = sink->clone();
    emit(*copy, "world\n");

    std::vector<std::unique_ptr<Sink>> sinks;
    sinks.push_back(std::move(sink));
    sinks.push_back(std::move(copy));

    for (auto& s : sinks) emit(*s, "!\n");
}
```

A javasolt buildparancs a laborban:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -g s119_sink.cpp -o s119_sink
```

Diagnosztikai változat cím- és UBSan-ellenőrzéssel:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -g \
    -fsanitize=address,undefined s119_sink.cpp -o s119_sink_asan
```

A példában a `Sink` destruktora virtuális, ezért a `unique_ptr<Sink>` törlése a dinamikus típus destruktorát hívja. A `clone` `unique_ptr<Sink>`-et ad vissza, így a hívó nem felejtheti el a törlést. Az `emit` `Sink&` paramétere kölcsönzött (borrowed) hozzáférés (nem `const`, mert a `write` nem `const` tagfüggvény): nem tulajdonol, nem töröl, és nem él túl a hívó `unique_ptr` élettartamán.

24.3.5 Destructor dispatch és az erőforrások sorrendje

A teljes objektum destruálása a legutóbb létrehozott részobjektumtól halad a legkorábbi felé. Egy `Derived` példány esetén a sorrend: a `~Derived()` teste, majd a `Derived` nem statikus tagjai fordított deklarációs sorrendben, majd a `~Base()` teste, majd a `Base` tagjai, végül a tároló felszabadítása. A származtatott tagok tehát előbb szabadulnak fel, mint a bázis tagjai. Ezért a bázis destruktorában nem szabad származtatott tagokra hivatkozni: azok már nem léteznek.

A `~Base()` teste alatt a dinamikus típus már `Base`, ezért a benne hívott virtuális függvények a `Base` verziójára mennek, még akkor is, ha a teljes objektum eredetileg `Derived` volt. Ugyanez igaz a konstruktorokra: a `Base` konstruktorában a dinamikus típus még `Base`. Ezért a bázis konstruktorában és destruktorában ne hívjunk virtuális függvényt, és ne bízzunk abban, hogy a származtatott felülírás futni fog. Ez a szabály nem a virtuális dispatch hibája, hanem a részobjektum-élettartam következménye.

A virtuális destruktor jelenléte a legtöbb implementációban egy virtuális táblára mutató pointerrel növeli az objektum méretét, és a törlésnél egy indirekciót ad. A konkrét méret- és költségadat implementációfüggő; mérés nélkül nem állítható konkrét szám. A vtable mint reprezentáció nem ISO-követelmény, ezért itt nem tárgyalom részletesen.

24.3.6 Interface ownership, borrowed reference és factory

A factory függvény a tulajdonost adja vissza: `std::unique_ptr<Sink> make_sink(std::string_view kind);`. A visszatérési típus a tulajdonlást fejezi ki, a hívó `unique_ptr`-ben tartja, vagy `std::move`-val továbbadja. Ha a factory ismeretlen típust kap, adhat `nullptr`-t; ezt a szerződésben dokumentálni kell, és a hívónak ellenőriznie kell. A kölcsönzött hozzáférés ezzel szemben `const Sink&` vagy `Sink*` paraméter, amely nem tulajdonol, és nem töröl. A kölcsönzés élettartama soha nem lehet hosszabb, mint a tulajdonosé; ezt a hívó felelőssége biztosítani.

A `clone` és a move-only owner együtt is használható: a factory `unique_ptr<Sink>`-et ad, a `clone` pedig akkor hívható, ha a tulajdonos másolatára van szükség. A `clone` költsége dinamikus allokáció és a származtatott objektum másolása; a move-only owner költsége egy pointer mozgatása. A választás a tulajdonlási igényből következik, nem teljesítménybecslésből.

24.3.7 Tipikus hibák és javítások

1. **Nem virtuális bázis destruktor és `delete Base*`.** A kód lefordul, de futásidőben UB. Javítás: `virtual ~Base() = default;`.
2. **`unique_ptr<Base>` nem virtuális destruktorú bázisra.** A `make_unique<Derived>()` konverziója lefordul, a törlés UB. Javítás: a bázis virtuális destruktora, vagy a `unique_ptr<Derived>` megtartása.
3. **Érték szerinti bázis paraméter.** `void f(Base b)` szeletel. Javítás: `void f(const Base& b)`, vagy a bázis absztrakttá tétele.
4. **`std::vector<Base>` heterogén gyűjteményre.** A `push_back(derived)` szeletel. Javítás: `std::vector<std::unique_ptr<Base>>`.
5. **`clone` nyers pointerrel.** A hívó elfelejti a `delete`-et, memóriaszivárgás lesz. Javítás: `std::unique_ptr<Base>` visszatérési típus.
6. **Protected nonvirtual destruktor és `unique_ptr<Base>`.** A `default_delete` nem fér hozzá a destruktorhoz, ezért a `unique_ptr<Base>` destruktora nem fordul le. Ez szándékos védelem; ha mégis kell, olyan egyedi deleter szükséges, amely a konkrét származtatott típuson töröl (a puszta hozzáférés megadása — barát osztály — nem teszi helyessé a `Base*`-on végzett nem virtuális törlést).
7. **Virtuális hívás a bázis destruktorában.** A hívás a bázis verziójára megy, nem a származtatottra. Javítás: a virtuális hívást a destruktoron kívülre tenni.

24.3.8 Feladatok

1. **Magyarázó feladat.** Adott a következő kód. Mi a hiba, miért UB, és miért nem segít az, ha a `Derived` destruktorát `virtual`-ra írjuk?

```
struct Base { ~Base() = default; };
struct Derived : Base { std::string payload; };
int main() { Base* p = new Derived; delete p; }
```

2. **Kódolási feladat.** Alakítsd át a `Sink` hierarchiát úgy, hogy a `Sink` destruktora protected nonvirtual legyen, a `StdoutSink` és a `PrefixSink` destruktora pedig public. Mutasd meg, miért nem fordul le ekkor a `std::unique_ptr<Sink>`, és adj egy működő alternatív tulajdonosi konstrukciót.

3. **Diagnosztikai feladat.** Adott a `void process(Shape s)` és a `void process(const Shape& s)` túlterhelés, valamint egy `Circle : Shape` objektum. Mi történik a `process(c)` hívásnál fordításkor (és miért)? Hogyan mutatod meg a szeletelést külön nevű függvényekkel, és hogyan detektálod anélkül, hogy a dinamikus típust `typeid`-vel vizsgálnád?

4. **Tervezési feladat.** Tervezz egy `std::unique_ptr<Base>`-et visszaadó factory-t, amely `clone`-t is támogat. Miért kell a `clone`-nak `unique_ptr<Base>`-t visszaadnia, és miért nem `Base*`-t? Milyen esetben elég a move-only owner `clone` nélkül?

24.3.9 Megoldások

1. megoldás. A `Base` destruktora nem virtuális, a `delete p` statikus típusa `Base*`, dinamikus típusa `Derived`. A `delete` UB-t okoz: a megsértett feltétel, hogy a statikus és a dinamikus típus különbözik, miközben a `Base` destruktora nem virtuális; a szabvány futási forgatókönyvet nem garantál. (Egy konkrét implementációnál megfigyelhető, hogy a `Derived::~Derived()` nem fut le és a `payload` szivárog, de ez csak megfigyelés.) A `Derived` destruktorának `virtual`-ra írása nem segít, mert a dispatch kiindulópontja a statikus típus destruktora; ha az nem virtuális, a dinamikus dispatch el sem indul. A javítás a bázisban van: `virtual ~Base() = default;`. Ellenőrzési feltétel: a javított kódban a `payload` destruktora lefut, és a `delete` nem okoz UB-t; ezt cím- és UBSan-nal lehet ellenőrizni a laborban.

2. megoldás. A `Sink` destruktora `protected: ~Sink() = default;`, a származtatottakban `public: ~StdoutSink() = default;` és `public: ~PrefixSink() = default;`. Ekkor a `delete Sink*` nem fordul le, mert a destruktor nem elérhető. A `std::unique_ptr<Sink>` sem fordul le, mert a `default_delete<Sink>` hívja a `delete`-et, és nem fér hozzá a protected destruktorhoz. Alternatíva: a factory `std::unique_ptr<StdoutSink>`-et vagy `std::unique_ptr<PrefixSink>`-et adjon vissza, és a közös kezelést sablon vagy `Sink&` kölcsönzés végezze; vagy olyan egyedi deletert használjunk, amely **ismeri és a konkrét származtatott típuson törli** az objektumot (például `struct StdoutDeleter { void operator()(Sink* p) const { delete static_cast<StdoutSink*>(p); } };`, ha a tárolt dinamikus típus biztosan `StdoutSink`), vagy tegyük a destruktort virtuálissá. Egy olyan friend deleter, amely továbbra is `Base*`-on (`Sink*`) végez közönséges `delete`-et, csak a hozzáférési hibát oldja meg: ha a dinamikus típus `Derived`, az UB megmarad — a hozzáférés és a dinamikus destrukció két külön feltétel. Ellenőrzési feltétel: a `delete Sink*` és a `unique_ptr<Sink>` destruktora fordítási hibát ad, a `unique_ptr<StdoutSink>` és a `Derived d;` viszont lefordul.

3. megoldás. A `process(Shape)` és a `process(const Shape&)` túlterhelésű pár `Circle` lvalue argumentummal **kétértelmű**: mindkét paraméter származtatott→bázis konverziót igényel, a két implicit konverziós sorozat között nincs döntő jobb illeszkedés, ezért a hívás *fordítási hiba* (GCC 13: „call of overloaded ‘process(Circle&)’ is ambiguous”), slicing-kimenet nincs. A szeletelés bemutatásához a két függvényt külön néven (`process_by_value(Shape)`, `process_by_ref(const Shape&)`) vagy külön fordítási egységben kell megadni: a by-value változatban a `Circle` szeletelődik, és a `Shape` viselkedése látszik, a referenciás változat megőrzi a dinamikus típust. A szeletelés detektálása: a bázis absztraktté tétele (`= 0`), a bázis másoló konstruktorának törlése (`= delete`),

vagy a `sizeof` és a címek helyett a viselkedés tesztelése — például egy olyan származtatott tag, amelynek hiánya a kimenetből látszik. A `typeid` nem jó detektor, mert a szeletelt objektum dinamikus típusa valóban a bázis. Ellenőrzési feltétel: a `process(const Shape&)` hívás a származtatott `name()`-et adja, a `process(Shape)` a bázisét; absztrakt bázis esetén a érték szerinti változat nem fordul le.

4. megoldás. A factory `std::unique_ptr<Base> make(...)` szignatúrájú, a `clone` pedig `virtual std::unique_ptr<Base> clone() const = 0;`. A `clone` azért ad `unique_ptr<Base>`-et, mert így a tulajdonlás a típusba kódolt, a hívó nem felejtheti el a törlést, és a `unique_ptr` destruktora a virtuális destruktoron keresztül a dinamikus típust törli. A `Base*` visszatérési típus a hívóra hárítaná a `delete`-et, és a `unique_ptr`-be csomagolás elmaradhat, ami szivárgáshoz vezet. A move-only owner `clone` nélkül akkor elég, ha a tulajdonos soha nem másolódik, csak mozog; például ha a factory által létrehozott objektumot egyetlen fogyasztó kapja meg, és a másolás nem része a szerződésnek. Ellenőrzési feltétel: a `clone` eredménye `unique_ptr<Base>`, a dinamikus típus megmarad, és a `clone` után mindkét objektum függetlenül törölhető.

24.3.10 Összefoglaló

A polimorfikus törlés feltétele, hogy a statikus típus destruktora virtuális legyen; e nélkül a `delete Base*` UB, és a `std::unique_ptr<Base>` sem véd meg. A public virtual destruktor a polimorfikus tulajdonlás alapmintája; a protected nonvirtual destruktor a bázis pointeren keresztüli törlést tiltja, és `unique_ptr<Base>`-bel nem használható. Az object slicing érték szerinti bázis-másolásból ered, nem UB, ezért a védelem a szerződés szintjén kell: absztrakt bázis, referenciás paraméter, vagy törölt másoló konstruktor. A `clone` a dinamikus típust megőrző másolatot ad `unique_ptr<Base>`-ként; a move-only interface owner akkor elég, ha a tulajdonlás csak mozog. A destruktorok sorrendje a származtatotttól a bázis felé halad, ezért a bázis destruktorában nem hivatkozhatunk származtatott tagokra.

Forráskorlát: a fejezethez a megadott forrásjegyzet nem tartalmazott konkrét, ellenőrzött szakaszokat vagy citációkat. A fenti szabályok a C++20 nyelvi szerződés általános ismertetései; a konkrét szabvány-paragrafusok, a Core Guidelines ajánlás száma és a compiler-implementáció viselkedése a laborban ellenőrizendő, nem tekinthetők itt bizonyítottnak.

24.4 RTTI, dynamic_cast és típusazonosság

Az előző részekben a dinamikus polimorfizmus arról szólt, hogy a hívás a dinamikus típus szerint oldódik fel: a hívónak nem kellett ismernie a konkrét osztályt. Most az ellenkező irányt vizsgáljuk: hogyan tud a program futásidőben rákérdezni a dinamikus típusra, és mikor szabad ezt egyáltalán megtenni. A futásidejű típusinformáció (run-time type information, RTTI) a C++ azon képessége, hogy bizonyos objektumokról futásidőben is megállapítható a legjobban leszármazott típus, illetve hogy egy bázisra mutató hivatkozás ellenőrzötten átalakítható legyen egy leszármazott vagy testvér típusra.

A rész vezérmondata: **az RTTI nem tervezési alap, hanem célzott eszköz**. Akkor indokolt, amikor a típuskészlet nyílt (plugin, eseménytovábbítás, szerializálás, diagnosztikai eszköztár), és a hívási helyen nincs jobb, típusbiztos mechanizmus. Ahol a típuskészlet zárt és fordítási időben ismert, ott a virtuális függvény, az overload vagy a `std::variant` + `std::visit` olcsóbb és kifejezőbb. Ahol viszont az RTTI-t használjuk, ott a sikertelenségi szerződést pointer és referencia esetén egyaránt pontosan kell kezelni — ez a rész fő gyakorlati mondanivalója.

A szabályok forrása a forrásjegyzék C++20 tétele: [expr.typeid], [expr.dynamic.cast] és [type.info]. A fejezet két teljes programját (`typeid_eval.cpp`, `rtti_demo.cpp`) a `lab-gcc` profilban lefordítottuk és lefuttattuk; a kimeneteket a buildjegyzőkönyvek rögzítik.

24.4.1 Statikus típus, dinamikus típus és a polimorf osztály feltétele

A **statikus típus** (static type) a kifejezés deklarált típusa; fordítási időben ismert, és a fordító minden típusellenőrzést ezen végez. A **dinamikus típus** (dynamic type) annak az objektumnak a legjobban leszármazott típusa, amelyre egy glvalue (pointer vagy referencia által megnevezett objektum) ténylegesen hivatkozik. A kettő csak indirekción keresztül válik szét: `Derived d;` esetén nincs kérdés, `Base* p = &d;` esetén viszont `p` statikus típusa `Base`, dinamikus típusa `Derived`.

Dinamikus típusról csak **polimorf osztály** esetén beszélhetünk, azaz ha az osztálynak van legalább egy virtuális függvénye (saját vagy örökölt). Ez az RTTI első és leggyakrabban elrontott előfeltétele: nem polimorf bázison a `typeid` csendben a statikus típust adja vissza, a `dynamic_cast` pedig nem fordul le. A dinamikus típus fogalma alatt azt is érteni kell, hogy konstruktor és destruktor futása közben a dinamikus típus az éppen épülő vagy bontott osztály — ilyenkor egy leszármazottra irányuló `dynamic_cast` jogosan hiúsul meg.

24.4.2 `typeid`: a dinamikus típus lekérdezése

A `typeid` operátor a `<typeinfo>` fejlécben deklarált `std::type_info` osztály egy `const` lvalue-ját adja vissza, amely statikus tárolási idővel él. Két alapvető eset van:

- Ha az operandus **polimorf osztálytípusú glvalue** (például `*basePtr` vagy egy `Base&`-et visszaadó függvény hívása, ahol `Base` polimorf), az operandus **kiértékelődik**, és az eredmény a **dinamikus típust** reprezentálja. A hívás és annak mellékhatásai tehát megtörténnek.
- Minden más esetben az eredmény a **statikus típust** reprezentálja, és az operandus **nem kiértékelt** operandus: nem keletkezik hívás, nincs mellékhatás. Ide tartozik a nem polimorf típusú kifejezés (`typeid(f())`, ahol `f` `int`-et ad), és a polimorf osztálytípusú **prvalue** is (`typeid(h())`, ahol `h` érték szerint ad vissza `D`-t): egy prvalue-nak nincs a statikus típustól eltérő dinamikus típusa.

```
// typeid_eval.cpp - C++20: mikor értékelődik ki a typeid operandusa?
#include <cstdio>
#include <typeinfo>

struct B { virtual ~B() = default; };   // polimorf
struct D : B {};

int f_calls = 0, g_calls = 0, h_calls = 0;

int f() { ++f_calls; return 1; }        // prvalue, nem polimorf
B& g() { ++g_calls; static D d; return d; }   // polimorf glvalue
D h() { ++h_calls; return D{}; }        // polimorf típusú prvalue

int main() {
    const std::type_info& t1 = typeid(f());   // statikus típus, nem kiértékelt
    const std::type_info& t2 = typeid(g());   // dinamikus típus, kiértékelt
    const std::type_info& t3 = typeid(h());   // prvalue: statikus típus, nem
     ↪ kiértékelt
```

```
    std::printf("f_calls=%d g_calls=%d h_calls=%d\n", f_calls, g_calls,
     ↪ h_calls);
    std::printf("t1==int:%d t2==D:%d t3==D:%d\n",
                t1 == typeid(int), t2 == typeid(D), t3 == typeid(D));

    B* p = nullptr;
    try {
        (void)typeid(*p);                    // polimorf glvalue null
         ↪ pointerből
    } catch (const std::bad_typeid&) {
        std::puts("bad_typeid");
    }
}
```

Buildjegyzőkönyv (`lab-gcc` profil, `g++ -std=c++20 -Wall -Wextra -Wpedantic -O2`). A kimenet: `f_calls=0 g_calls=1 h_calls=0`, majd `t1==int:1 t2==D:1 t3==D:1`, végül `bad_typeid`. A számlálók igazolják: csak a polimorf glvalue-t adó `g()` hívódott meg.

Ez a különbség a leggyakoribb félreértés forrása. Ráadásul pointer és pointerált objektum között is éles a határ: `typeid(p)` a `Base*` pointer típust adja, `typeid(*p)` a pointerált objektum dinamikus típusát — de csak akkor, ha `Base` polimorf, és `p` nem null. Ha `p` null, a `typeid(*p)` nem null-t ad, hanem `std::bad_typeid` kivételt dob, mert a `*` operátor eredménye egy null pointerre alkalmazott glvalue.

A `std::type_info` hasznos felülete: `name()` (implementációfüggő, jellemzően torzított/mangled név), `operator==`, `before()` és `hash_code()`, három különböző feladattal. Az `==` típusazonosságot dönt el. A `before()` egy implementáció által meghatározott **teljes rendezés** a típusokon (pontosan az egyik igaz: `a.before(b)` vagy `b.before(a)`, ha a típusok különböznek); a sorrend nem kapcsolódik az öröklési viszonyhoz, és programfuttatások között sem garantáltan stabil. A `hash_code()` hash: azonos típusra ugyanazt adja egy futáson belül, különböző típusokra ütközhet, és nem rendezésre való. A `name()` sztring tartalma nem szerződés: két fordítási egység vagy két fordító eltérő szöveget adhat ugyanarra a típusra, ezért `std::strcmp(typeid(a).name(), ...)` helyett mindig a `type_info` egyenlőségét használjuk. C++20-ban csak a külön `operator!=` került ki a tagok közül (a `!=` az `==`-ből átírt jelöltként adódik); a `before()` megmaradt. Tárolókulcshoz a `<typeindex>` fejléc `std::type_index` burkolója való: a `<` operátora a `before()`-re, a `std::hash` specializációja a `hash_code()`-ra épül.

```
// rtti_order.cpp - C++20: before(), ==, hash_code() és type_index
#include <cstdio>
#include <typeindex>
#include <typeinfo>

int main() {
    const std::type_info& a = typeid(int);
    const std::type_info& b = typeid(double);
    bool ab = a.before(b), ba = b.before(a);     // C++20-ban is tagfüggvény
    std::printf("before: %d %d (pontosan egy igaz)\n", ab, ba);
    std::printf("==: %d, hash egyezik int-int: %d\n",
                a == typeid(int), a.hash_code() == typeid(int).hash_code());
    std::type_index ia{a}, ib{b};                 // < a before()-re épül
    std::printf("type_index <: %d\n", (ia < ib) == ab);
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic rtti_order.cpp` figyelmeztetés nélkül fordul; a kimenet `before: 0 1 (pontosan egy igaz), ==: 1, hash egyezik int-int: 1, type_index <: 1`. Hogy az `int` vagy a `double` kerül-e előre, az implementációs részlet; csak az „egyik igaz” tulajdonság szerződés. A `name()` emberi olvashatóságához a GNU/Itanium környezetben `abi::__cxa_demangle` használható, de ez nem szabványos, ezért csak diagnosztikai kódban, nem szerződésként jelenhet meg.

Fontos: `typeid(a) == typeid(b)` **típusazonosságot** vizsgál, nem objektumazonosságot. Két külön objektum, amelyeknek azonos a dinamikus típusa, itt egyenlő; két bázispointer, amely ugyanarra a teljes objektumra mutat, `typeid`-dal nem hasonlítható össze értelmesen.

24.4.3 `dynamic_cast`: a sikertelenségi szerződés

A `dynamic_cast<T>(v)` előfeltételei: a cél (`T`) pointer vagy referencia **teljes** osztálytípusra (vagy `void*`); a forrásnak polimorfnak kell lennie, ha a konverzió futásidejű ellenőrzést igényel (leszármazottra alakítás, keresztkonverzió, `void*`). A futásidejű ellenőrzés (C++20 [expr.dynamic.cast]/9) nem a teljes objektumból, hanem **a forrás által mutatott részobjektumból** indul, és két sikeres esetet ismer:

1. **Lefelé, a forrás részobjektumától.** Ha a `v` egy `C` típusú objektum *public* bázis-részobjektumára mutat, és a legjobban leszármazott objektumon belül pontosan **egy** `C` objektum származik éppen ebből a részobjektumból, az eredmény arra a `C`-re mutat. Az egyértelműséget tehát a forrás részobjektumától felfelé nézzük: ha a teljes objektumban több `C` is van, de a mutatott részobjektum csak az egyiknek része, a cast sikeres.
2. **Keresztül a teljes objektumon.** Ha az első eset nem alkalmazható, de a `v` a legjobban leszármazott objektum public bázis-részobjektumára mutat, és a legjobban leszármazott típusnak van **egyértelmű és public** `C` bázisa, az eredmény arra a `C` részobjektumra mutat.

Minden más esetben a cast sikertelen.

A két forma hibaszerződése gyökeresen eltér, és ezt nem lehet elégszer hangsúlyozni:

- **Pointer forma:** hiba esetén `nullptr`-t ad vissza. Ugyanakkor `nullptr` bemenetre is `nullptr`-t ad — sikeres null-konverzió. A hívó tehát a `nullptr`-ból nem tudja megkülönböztetni a „nem ez a típus” és a „nincs objektum” esetet, ezért a null-ellenőrzést a cast előtt kell elvégezni, ha a kettő számít.
- **Referencia forma:** hiba esetén `std::bad_cast` kivételt dob, mert nincs „null referencia”. Ez viszont azt jelenti, hogy a referencia forma csak ott használható, ahol a kivétel terjedése megengedett: `noexcept` függvényben vagy destruktorban a kezeletlen `bad_cast` `std::terminate`-hez vezet. Null pointerre alkalmazott referencia-forma nem `bad_cast`-ot ad, hanem eleve nem definiált viselkedés, mert a referencia képzése már dereferálja a null pointert.

A `dynamic_cast` három tipikus esete:

- **Upcast** (leszármazottról bázisra): fordítási időben eldől, futásidejű ellenőrzés nincs, és nem is kell polimorf típus. Ilyenre `static_cast` vagy implicit konverzió való.
- **Downcast** (bázisról leszármazottra): futásidejű ellenőrzés; akkor is sikeres, ha a dinamikus típus a célnál mélyebben leszármazott, hiszen a cél egy bázisa a dinamikus típusnak.
- **Cross-cast** (testvérágak között, többszörös öröklésnél): például `A*`-ból `B*`-ba, ahol `C : A, B`. Ez a `static_cast`-tal nem kifejezhető, mert a két statikus típus között nincs öröklési viszony; a `dynamic_cast` futásidőben keresi meg a közös `C` objektumon belüli `B` alterületet. Ha a teljes objektumban a cél bázis nem egyértelmű (például nem virtuális módon többször szerepel), és a forrás részobjektuma sem választ ki egyetlen célt, a konverzió meghiúsul:

pointer esetén `nullptr`, referenciánál `bad_cast`. A két esetet egy próba mutatja meg, ahol a `D`-ben két `B` részobjektum van:

```
// dynamic_cast_repeated_target.cpp - C++20
#include <cassert>
#include <cstdio>

struct X { virtual ~X() = default; };      // polimorf gyökér
struct B : X {};
struct L : B {};
struct R : B {};
struct M { virtual ~M() = default; };      // független polimorf bázis
struct D : L, R, M {};                      // D-ben két B részobjektum van

int main() {
    D d;
    B* bl = static_cast<L*>(&d);            // az L ág B-je
    B* br = static_cast<R*>(&d);            // az R ág B-je
    X* xl = bl;                             // az L ág X részobjektuma

    // (1) Lefelé a forrás részobjektumától: az xl-ből csak EGY B származik
    //     (az L ágé), ezért a cast sikeres, noha D-ben két B van.
    B* r1 = dynamic_cast<B*>(xl);
    assert(r1 == bl && r1 != br);

    // (2) Keresztkonverzió a teljes objektumon át: M-ből B-re a D-ben
    //     két B van, a cél nem egyértelmű, ezért nullptr.
    M* m = &d;
    assert(dynamic_cast<B*>(m) == nullptr);
    // M-ből L-re viszont egyértelmű, public bázis: sikeres.
    assert(dynamic_cast<L*>(m) == static_cast<L*>(&d));
    std::puts("dynamic_cast_repeated_target: OK");
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic dynamic_cast_repeated_target.cpp` figyelmeztetés nélkül fordul, a kimenet `dynamic_cast_repeated_target: OK`: az L ág `X` részobjektumából indított cast az L ág `B`-jére mutat (1. eset), az `M`-ből indított `B*` cast `nullptr` (a 2. esetben a `B` nem egyértelmű), az `M`-ből `L*`-ra pedig sikeres.

A `dynamic_cast<void*>(p)` speciális forma: a legjobban leszármazott objektum címét adja vissza. Ez nem típus-, hanem **teljes objektum azonosság** vizsgálatára alkalmas: két különböző bázisaltérületre mutató pointer pontosan akkor ad azonos `void*`-ot, ha ugyanannak a teljes objektumnak az alterületei.

24.4.4 `static_cast` downcast: előfeltétel és kockázat

A `static_cast` bázisról leszármazottra fordítógenerált pointer-/referencia-igazítást végez, futásidejű ellenőrzés nélkül. Előfeltétele, hogy az objektum dinamikus típusa valóban a cél típusa vagy abból leszármazó legyen; ha nem az, a viselkedés nem definiált — nem „nullpointer”, nem kivétel, hanem csendben hibás memória-hozzáférés vagy értelmetlen tagelérés. Ezért a `static_cast` downcast csak akkor indokolt, ha az invariánst a program másutt már garantálja (például egy `enum` tag vagy egy korábbi ellenőrzött cast alapján), és ezt a garanciát kód-kommentár és teszt is

rögzíti. A gyakori minta, hogy a teljesítménykritikus belső útvonalon egyszer `dynamic_cast`-tal ellenőrzünk, majd a belső ciklusban már `static_cast`-ot használunk — ez helyes, ha az objektum élettartama alatt a dinamikus típus nem változhat.

24.4.5 RTTI nélküli profil és a `type_info` a shared library határon

A GCC és a Clang is ismeri a `-fno-rtti` (MSVC-nél `/GR-`) kapcsolót, amellyel az RTTI-generálás kikapcsolható. Ez **nem szabványos nyelvi mód**, hanem implementációs profil: a dokumentáció szerint ilyenkor a polimorf `dynamic_cast` és a dinamikus `typeid` nem használható, a kivételkezelés viszont továbbra is működik, mert ahhoz a fordító szükség szerint maga állít elő típusinformációt. A pontos korlátozás és a hibaüzenet compiler-verziófüggő, ezért ezt a laborprofilban fordítási próbával kell rögzíteni; e nélkül a „kikapcsolt RTTI” viselkedéséről csak annyi állítható biztosan, hogy nem hordozható, standard programra nem támaszkodhatunk vele.

A `type_info` azonossága megosztott könyvtár (DSO) határon implementáció- és ABI-függő kérdés. A típusinformáció a dinamikus típusokat leíró szimbólumokhoz kötődik; ha két DSO ugyanazt az osztályt külön fordítja, a `type_info` példányok egyesítése a loader és a könyvtár együttműködésén múlik. A libstdc++ egyes konfigurációkban nevek összehasonlításával dönt az egyenlőségről, más esetekben (például rejtett láthatóság, `-fvisibility=hidden`, eltérő burkolás) ez nem működik, és a `dynamic_cast` vagy a `typeid`-egyenlőség a határon meghiúsulhat. Következmény: **a `typeid`-azonosságot soha ne használd protokollként DSO- vagy FFI-határon.** Ott a stabil megoldás egy saját, verziózott típusazonosító (sztring, szám vagy enumerátor) az interfészben, amelyet mindkét fél ismer, és amely nem függ a C++ ABI belső szerkezetétől. Kapcsolódó élettartam-csapda: a `typeid` eredményére vett `const std::type_info&` addig érvényes, amíg a típust leíró kód be van töltve; plugin kirakodása után a referencia lógó lesz, ezért a típusinformációt nem érdemes a plugin élettartamán túl tárolni.

24.4.6 Kidolgozott példa: eszközregiszter persistálható alterülettel

A példa egy `Device` hierarchiát és egy `Persistable` mellékhierarchiát használ; a `SmartGauge` mindkettőből származik, így a `Device*`-ból `Persistable*`-ra alakítás keresztkonverzió.

```
// rtti_demo.cpp - ISO C++20 példa; a kimenet implementációfüggő részt tartalmaz
#include <iostream>
#include <memory>
#include <string>
#include <string_view>
#include <typeinfo>
#include <utility>
#include <vector>

class Device {
public:
    virtual ~Device() = default;
    virtual std::string_view name() const noexcept = 0;
};

class Persistable {
public:
    virtual ~Persistable() = default;
    virtual std::string serialize() const = 0;
```

```
};

class Gauge : public Device {
public:
    explicit Gauge(std::string n) : name_(std::move(n)) {}
    std::string_view name() const noexcept override { return name_; }
private:
    std::string name_;
};

class SmartGauge final : public Gauge, public Persistable {
public:
    using Gauge::Gauge;
    std::string serialize() const override { return "gauge:" +
     ↪  std::string{name()}; }
};

// Nem tulajdonol: a hívó felel a d élettartamáért.
void persist_if_possible(Device* d)
{
    if (d == nullptr) {                       // null és "nem ez a típus"
     ↪  szétválasztása
        std::cout << "null device\n";
        return;
    }
    if (auto* p = dynamic_cast<Persistable*>(d)) {   // cross-cast, pointer
     ↪  forma
        std::cout << "persist: " << p->serialize() << '\n';
    } else {
        std::cout << "skip: " << d->name() << '\n';
    }
}

void persist_or_report(Device& d)             // referencia forma: bad_cast a
 ↪  hiba
{
    try {
        Persistable& p = dynamic_cast<Persistable&>(d);
        std::cout << "persist(ref): " << p.serialize() << '\n';
    } catch (const std::bad_cast& e) {
        std::cout << "bad_cast: " << e.what() << '\n';
    }
}

int main()
{
    std::vector<std::unique_ptr<Device>> devices;
    devices.push_back(std::make_unique<Gauge>("g0"));
    devices.push_back(std::make_unique<SmartGauge>("g1"));

    for (auto& d : devices) {
        persist_if_possible(d.get());
```

```
        persist_or_report(*d);
        std::cout << "typeid(*d).name() = " << typeid(*d).name() << '\n';
        std::cout << "pontosan SmartGauge? " << (typeid(*d) ==
         ↪  typeid(SmartGauge)) << '\n';
    }
}
```

Fordítás és futtatás a tanuló laborjában:

```
g++     -std=c++20 -Wall -Wextra -Wpedantic -O2 rtti_demo.cpp -o rtti_demo &&
 ↪  ./rtti_demo
# vagy
clang++ -std=c++20 -Wall -Wextra -Wpedantic -O2 rtti_demo.cpp -o rtti_demo &&
 ↪  ./rtti_demo
```

Buildjegyzőkönyv (`lab-gcc` profil). A GCC-paranccsal a program figyelmeztetés nélkül fordult, és a következő viselkedést adta: `g0` esetén a pointer forma `skip: g0`-t ír, a referencia forma `bad_cast`-ot kap el, a `typeid`-egyenlőség hamis; `g1` esetén mindkét forma sikeres (`persist: gauge:g1`, `persist(ref): gauge:g1`), és a `typeid`-egyenlőség igaz. A `name()` szövege a `lab-gcc` profilban `5Gauge`, illetve `10SmartGauge` (Itanium ABI szerinti torzított név), de ez nem szerződés: a fordító és a C++ könyvtár dönti el. A Clang-sort nem ellenőriztük.

24.4.7 Tipikus hibák és javításuk

- **Ellenőrzés nélküli pointeres cast.** `if (auto* p = dynamic_cast<D*>(b))` után `p` használata helyes; `dynamic_cast<D*>(b)->tag()` viszont nullán dereferál. Javítás: mindig a `nullptr`-ellenőrzés ágában használd.
- **Null és hibás cast összemosása.** Null bemenetre a pointer forma null-t ad, ami „siker”. Javítás: a null-ellenőrzés a cast előtt történjen, ahogy a példában.
- **Referencia forma `noexcept`/destruktor kontextusban.** A `bad_cast` onnan kifelé nem terjedhet, ezért `std::terminate`. Javítás: ilyen helyen pointer forma, vagy előzetes `typeid`-próba.
- **Nem polimorf bázis.** A `typeid(*basePtr)` csendben a statikus típust adja, a `dynamic_cast` nem fordul le. Javítás: virtuális destruktorral (vagy más virtuális taggal) polimorffá tenni a bázist — ez egyben a korábbi rész ownership-szabálya is.
- **`typeid` a pointeren.** `typeid(p).name()` a pointer típusát írja le; a dinamikus típus a `typeid(*p)`.
- **`typeid` null pointeren.** `typeid(*p)` null `p` esetén `std::bad_typeid`, nem „ismeretlen típus”.
- **Név-összehasonlítás.** `name()` sztringje implementációfüggő; helyette `typeid(a) == typeid(b)` vagy `std::type_index`.
- **`typeid`-azonosság DSO/FFI határon.** Rejtett láthatóság mellett a típusinformáció nem egyesül, a cast meghiúsulhat. Javítás: saját, verziózott típusazonosító a határon.
- **`static_cast` downcast rossz feltételezéssel.** Ha a dinamikus típus nem a cél, nem definiált viselkedés. Javítás: ellenőrzött cast, majd a bizonyított invariáns mellett `static_cast`.

24.4.8 Összefoglaló

A dinamikus típus fogalma csak polimorf osztályokra értelmezhető. A `typeid` polimorf glvalue-n a dinamikus, minden más esetben a statikus típust adja, eredménye `const std::type_info&`, amelynek `name()`-je nem szerződés. A `dynamic_cast` pointer formája `nullptr`-rel, referencia

formája `std::bad_cast`-tal jelzi a sikertelenséget; a pointer forma null bemenetre is null-t ad, a referencia forma null pointerre nem definiált. A cross-cast többszörös öröklésnél a `dynamic_cast` egyedi képessége, a `static_cast` downcast pedig csak bizonyított invariáns mellett biztonságos. A kikapcsolt RTTI és a `type_info` DSO-határon átnyúló azonossága nem hordozható, implementációfüggő terület, ezért interfész-szerződés nem épülhet rá.

24.4.9 Feladatok

1. **Magyarázó.** Adott: `struct Base { void f(); }; struct Derived : Base { void f(); }; Derived d; Base* b = &d;` Mi lesz a `typeid(*b) == typeid(Derived)` értéke, és miért? Mit változtat, ha a `Base::f` virtuális?

2. **Kódoló.** Írd meg az alábbi függvényt, amely a keresztkonverzióval összegyűjti a persistálható alterületeket; az eredmény ne tartalmazzon null elemet:

   ```
   std::vector<Persistable*> collect_persistable(
       const std::vector<std::unique_ptr<Device>>& in);
   ```

   Készíts referencia-formás változatot is, de csak ott, ahol a kivétel terjedése megengedett.

3. **Diagnosztikai.** Az alábbi kód lefordul, de futás közben leáll: `void log(Device& d) noexcept { auto& p = dynamic_cast<Persistable&>(d); std::cout << p.serialize(); }`. Magyarázd meg a hibát, és javítsd két különböző szerződés mentén (nem dobó és dobó változat).

4. **Nehezebb.** Két vektorban `Device*` és `Persistable*` pointerek vannak ugyanazokra az objektumokra. Hogyan párosítanád őket teljes objektum szerint, és miben más ez, mint a `typeid` szerinti csoportosítás?

24.4.10 Megoldások

1. megoldás. Nem polimorf `Base` esetén a `typeid(*b)` a statikus típust, azaz `Base`-t adja, ezért az összehasonlítás hamis — pedig az objektum valóban `Derived`. Ha a `Base`-nek van virtuális függvénye, a `*b` polimorf glvalue, a `typeid` a dinamikus típust adja, és az összehasonlítás igaz lesz. Ellenőrzési feltétel: a program mindkét változatban forduljon le, és a második esetben a kiírt érték `1` legyen; a diagnosztika indoklása nevezze meg a „polimorf osztály” feltételt.

2. megoldás. A pointer forma természetes, mert a „nincs ilyen alterület” nem hiba:

```
std::vector<Persistable*> collect_persistable(
    const std::vector<std::unique_ptr<Device>>& in)
{
    std::vector<Persistable*> out;
    out.reserve(in.size());
    for (const auto& d : in) {
        if (d == nullptr) continue;
        if (auto* p = dynamic_cast<Persistable*>(d.get())) out.push_back(p);
    }
    return out;
}
```

A referencia-formás változat csak ott használható, ahol a `bad_cast` elkapható:

```
for (const auto& d : in) {
    if (!d) continue;
    try {
        out.push_back(&dynamic_cast<Persistable&>(*d));
    } catch (const std::bad_cast&) {
        // nincs Persistable alterület: kihagyjuk
    }
}
```

Mindkét változat **nem tulajdonol**: a visszaadott pointerek érvényessége a bemeneti vektor és a mögötte lévő objektumok élettartamához kötött, azt a hívó köteles betartani. Ellenőrzési feltétel: a kimenet hossza egyezzen a persistálható objektumok számával, ne tartalmazzon `nullptr`-t, és a függvény ne módosítsa a bemenetet.

3. megoldás. A `log` `noexcept`, a `dynamic_cast<Persistable&>` viszont `std::bad_cast`-ot dob, ha az objektumnak nincs `Persistable` alterülete. A kivétel egy `noexcept` függvényből kifelé nem terjedhet, ezért a fordító `std::terminate`-et hív. Nem dobó javítás: pointer forma, null-ellenőrzéssel, és a hiányzó alterület naplózása külön ágban. Dobó javítás: a `noexcept` eltávolítása, a `try/catch (const std::bad_cast&)` bevezetése, és a hibapolitika dokumentálása (naplózás vagy továbbdobás). Ellenőrzési feltétel: a nem dobó változat `Gauge`-ra is és `SmartGauge`-ra is lefut, nem hív `terminate`-t, és a nem dobó volta `static_assert(noexcept(...))`-tal vagy dokumentált indoklással alátámasztható.

4. megoldás. Teljes objektum szerinti párosításhoz `dynamic_cast<void*>` való: két pointer pontosan akkor mutat ugyanarra a teljes objektumra, ha a castolt `void*` értékük azonos (ehhez mindkét oldalon polimorf típus kell, és a pointerek nem nullák). Ezt használhatod kulcsként egy `std::unordered_map<void*, ...>`-ban, de figyelj rá, hogy a kulcs csak az objektum élettartama alatt érvényes: felszabadítás után cím-újrafelhasználás miatt hamis egyezés lehetséges. A `typeid` szerinti csoportosítás ezzel szemben típust azonosít, nem objektumot: két külön `SmartGauge` példány egy csoportba kerül, és a pointer-igazítás miatt a `Device*` és `Persistable*` értékek összehasonlítása önmagában félrevezető. Ellenőrzési feltétel: a párosítás pontosan a közös objektumokra adjon egyezést, a `typeid`-alapú csoportosítás pedig adott esetben több objektumot vonjon össze — a két eredmény különbözősége a helyes értelmezés bizonyítéka.

24.5 Többszörös öröklés és virtuális bázis

A többszörös öröklés akkor válik érdekessé, amikor egy típus egyszerre több, egymástól független szerződést kell hogy teljesítsen. A C++-ban erre nincs külön `interface` kulcsszó: absztrakt osztályokból, pure virtual függvényekkel építünk interfészeket. Amíg ezek az interfészek függetlenek, a dolog egyszerű. Amint azonban két interfésznek közös őse van, megjelenik a diamond (gyémánt) probléma: a közös ős kétszer kerülne be az objektumba, a nevek pedig kétértelművé válnak. A `virtual` öröklés ezt egyetlen közös base subobjectre redukálja, de a konstrukció és a pointerkonverzió szabályai ettől lényegesen bonyolultabbak lesznek. Ebben a részben a nyelvi szerződést és annak tipikus ABI-megvalósítását külön kezeljük: az előbbi hordozható, az utóbbi target- és compilerfüggő.

A virtuális dispatch és a virtuális destruktor mechanizmusát Virtual, override, final és dispatch (24.2) és Virtuális destruktor, slicing és ownership (24.3) tárgyalta; a híváskonvenció alapjait Hívási konvenció, stack frame és visszatérés (14.2) rögzítette. Itt azt vizsgáljuk, mi történik, ha egy objektumnak több base subobjectje van, és hogyan lesz ebből egyetlen közös ős.

24.5.1 Több interfész és két base subobject

A **base subobject** (bázis-alobjektum) a teljes objektum azon része, amely egy base osztály példányát hordozza. Nem-virtualis többszörös öröklésnél minden közvetlen base-hez külön subobject tartozik, saját tárolóval. Tekintsük a következő, még diamond mentes példát:

```
struct A { int a = 1; };
struct B { int b = 2; };
struct C : A, B { int c = 3; };
```

A `C` objektum tartalmaz egy `A` subobjectet, egy `B` subobjectet és a `c` tagot. A `C*` típusú pointer `A*`-ká és `B*`-ká konvertálható, de a két eredmény nem feltétlenül ugyanaz a cím. A helyes konverzió a `static_cast`, amely elvégzi a szükséges **pointer adjustment**et (pointerigazítást). A `reinterpret_cast` ezt nem teszi meg, ezért base pointerek között nem használható.

A nyelvi garancia az, hogy a `static_cast` a megfelelő base subobjectre mutat, és hogy a subobjectek élettartama a teljes objektum élettartamához kötődik. Az nem garancia, hogy a `B` subobject a `C` elején van, vagy hogy az offset fordítási időben ismert. A konkrét layout target ABI és fordítási profil függvénye.

Interfészeknél a base-ek jellemzően absztraktak és adattag nélküliek, de a subobjectek akkor is léteznek. Az üres base optimalizálás (empty base optimization, EBO) megengedett, de nem kötelező; két azonos típusú közvetlen base pedig nem is szerepelhet, mert az már a deklaráció szintjén kétértelmű lenne.

Konstrukció	Base subobjectek	Következmény
`struct D : A, B`	két külön subobject	`static_cast` igazít; a címek eltérhetnek
`struct D : virtual A`	egy megosztott `A` subobject	futásidejű vbase-offset szükséges
diamond `virtual` nélkül	a közös ős kétszer	kétértelmű név, duplikált állapot
diamond `virtual`-lal	a közös ős egyszer	egyértelmű hozzáférés, bonyolultabb konstrukció

24.5.2 A diamond és a virtuális bázis

A diamond akkor jön létre, amikor egy ős két különböző útvonalon érhető el. Vegyük a `V` közös őszt, valamint az `L` és `R` köztes osztályokat, amelyekből a `D` származik. Virtual inheritance nélkül a `D` objektumban két `V` subobject lenne, és a `V::f` hívása kétértelmű:

```
// diamond_bad.cpp - szándékosan nem forduló tanítási példa
struct V { void f() {} };
struct L : V {};
struct R : V {};
struct D : L, R {};

int main() {
    D d;
    d.f(); // hiba: a V::f két útvonalon érhető el
}
```

A javítás a base-lista `virtual` kulcsszava:

```
struct L : virtual V {};
struct R : virtual V {};
```

Ettől a `D`-ben egyetlen `V` subobject jön létre, és a `d.f()` egyértelmű. Fontos, hogy a base-listában szereplő `virtual` nem azt jelenti, hogy a `V` függvényei virtuálisak; a két `virtual` kulcsszó helye és jelentése különbözik. A **virtual base** (virtuális bázis) a megosztott subobjectre vonatkozik, a **virtual function** a dinamikus dispatch-re.

A megosztott bázis ára a layoutban jelentkezik: a köztes osztályoknak valahogy el kell érniük a közös `V` subobjectet, amelynek pozíciója a teljes objektum típusától függ. A tipikus megvalósítás ezért a köztes subobjectekben rejtett offset- vagy pointerinformációt tárol, vagy a vtable-ből olvassa ki a vbase-offsetet. Ez már ABI-szintű részlet, nem nyelvi garancia.

Nézzünk egy fordítható példát, amely a konstrukciós sorrendet és a címeket is szemlélteti:

```
// diamond_ok.cpp - ISO C++20, x86-64 System V, GCC/Clang illusztráció
#include <cstdio>

struct V {
    explicit V(int x) : tag(x) { std::printf("V(%d)\n", x); }
    virtual ~V() = default;
    virtual void f() { std::printf("V::f tag=%d\n", tag); }
    int tag;
};

struct L : virtual V {
    L() : V(1) { std::printf("L\n"); }
    void f() override { std::printf("L::f\n"); }
    int l = 10;
};

struct R : virtual V {
    R() : V(2) { std::printf("R\n"); }
    int r = 20;
};

struct D : L, R {
    D() : V(3) { std::printf("D\n"); }
    int d = 30;
};

int main() {
    D obj;
    V& v = obj;
    std::printf("sizeof(D)=%zu\n", sizeof(D));
    std::printf("&obj=%p &L=%p &R=%p &V=%p\n",
                static_cast<void*>(&obj),
                static_cast<void*>(static_cast<L*>(&obj)),
                static_cast<void*>(static_cast<R*>(&obj)),
                static_cast<void*>(static_cast<V*>(&obj)));
    v.f();
}
```

Fordítás és futtatás a laborban:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o diamond_ok diamond_ok.cpp
./diamond_ok
```

A várt kimenet első sorai — illusztráció, nem mért eredmény — a `V(3)`, `L`, `R`, `D` sorrendet követik, majd a `sizeof` és a négy cím, végül az `L::f`. A címek konkrét értéke target- és compilerfüggő; a példa csak azt mutatja, hogy a `D` eleje, az `L` subobject, az `R` subobject és a közös `V` subobject címe általában nem esik egybe. A `sizeof(D)` értéke sem hordozható, mert a virtuális bázis és a köztes subobjectek implementációs többletet hordoznak.

24.5.3 Ki építi a virtuális bázist?

A virtuális bázis inicializálására külön szabály vonatkozik. A **most-derived object** (leginkább származtatott objektum) az, amelynek dinamikus típusa a teljes objektum típusa. A virtuális bázist mindig a most-derived osztály inicializálja; a köztes osztályoknak a virtuális bázisra vonatkozó mem-initializer-ei figyelmen kívül maradnak, amikor az adott osztály nem most-derived.

A fenti példában a `D` a most-derived, ezért a `D() : V(3)` initializer fut, az `L() : V(1)` és `R() : V(2)` pedig nem. Ha a `D` konstruktorából törölnénk a `V(3)` inicializálót, a `V` default konstruktorára lenne szükség; mivel a `V`-nek csak `explicit V(int)` konstruktora van, a program nem fordulna. Ha lenne default konstruktor, az futna, és az `L` initializerét akkor is figyelmen kívül hagynánk.

Az indoklás — ez nem bizonyítás, hanem a szabály motivációja — a következő. A virtuális bázis egyetlen példányban létezik, és a pozíciója a teljes objektum típusától függ. Egy köztes konstruktor nem tudhatja, hogy ő most-derived-e, és azt sem, hogy a virtuális bázis hol helyezkedik el a végső layoutban. Ezért a felelősséget a leginkább származtatott osztály viseli. A konstrukció sorrendje: először a virtuális bázisok, mélységi, balról jobbra haladó sorrendben, majd a nem-virtualis közvetlen bázisok, ezután a tagok, végül a konstruktortest. A destruktorok fordított sorrendben futnak; a virtuális bázis destruktora a most-derived objektum destruktora után, utolsóként hívódik.

24.5.4 Kétértelműség és minősítés

Ha két testvér base ugyanazzal a névvel deklarál egy tagot, a származtatott típuson keresztül történő hozzáférés kétértelmű:

```
struct A { void f() {} };
struct B { void f() {} };
struct C : A, B {};

int main() {
    C c;
    c.f();      // hiba: kétértelmű
    c.A::f();   // OK: minősített hívás
}
```

A minősítés (`c.A::f()`) feloldja a kétértelműséget, de nem szünteti meg a két subobject létét. Alternatíva a `using` deklaráció a származtatott osztályban:

```
struct C : A, B { using A::f; };
```

Virtuális bázisnál a közös ős egyetlen példányban van, ezért a közös ősben deklarált tagok elérése önmagában egyértelmű. Ha viszont a két ág külön-külön deklarál azonos nevű tagot, az továbbra is kétértelmű lehet. A névfeloldás ilyenkor a dominancia szabályát is figyelembe veszi: a közelebbi, származtatottabb deklaráció elrejtheti a távolabbit. Ennek pontos részleteit a konkrét esetre kell ellenőrizni; a biztos megoldás a minősítés vagy a `using` deklaráció.

24.5.5 Pointer adjustment, thunk és a vtable mint ABI-példa

A nyelvi szerződés két dolgot garantál: a `static_cast` a megfelelő base subobjectre állítja a pointert, a `dynamic_cast` pedig a dinamikus típusból kiindulva adja vissza a kért base vagy derived pointert. A szabvány nem ír elő vtable-t, vptr-t, thunköt vagy konkrét offsetet. Ezek az **ABI** (application binary interface) szintjén jelennek meg, és implementációnként eltérhetnek.

A tipikus, x86-64 System V és Itanium C++ ABI mentén futó megvalósításban a nem-virtualis base-re konvertálás konstans offset hozzáadása; a virtuális bázisra konvertálás viszont futásidejű információt igényel, mert az offset a teljes objektum típusától függ. Sematikusan:

```
D* -> B*  (nem-virtualis):  cím = d_cím + const_offset(B, D)
D* -> V*  (virtuális):      cím = d_cím + vbase_offset(V, D)   // futásidejű
```

A **vptr** egy rejtett pointer a polymorphic subobjectben, amely a **vtable**-re mutat. A vtable tartalmazhatja a virtuális függvények pointereit, az RTTI-re mutató bejegyzést, valamint a virtuális bázis offsetjét. Ha egy virtuális függvényt egy másodlagos base vtable-jén keresztül hívunk, a bejegyzés egy **thunk**re mutathat: ez egy kis adapter, amely a `this` pointert a teljes objektum megfelelő pontjára igazítja, majd a valódi függvényt hívja. A thunk címe nem feltétlenül egyezik a valódi függvény címével.

Fogalom	Nyelvi státusz	Tipikus ABI-megvalósítás
vptr	nincs a szabványban	rejtett pointer a polymorphic subobjectben
vtable	nincs a szabványban	statikus tábla, RTTI-pointerrel
vbase offset	nincs a szabványban	vtable-ből vagy konstrukciós információból
thunk	nincs a szabványban	adjustor thunk a másodlagos vtable-ben
RTTI	`typeid`, `dynamic_cast` nyelvi eszköz	`type_info` objektum, vtable-hivatkozással

A `dynamic_cast` virtuális bázis esetén is működik, mert az RTTI tartalmazza a szükséges típusinformációt; a cross-cast (az egyik base pointerből a másikba) is erre épül. Ennek nyelvi hátterét RTTI, dynamic_cast és típusazonosság (24.4) tárgyalta. A konkrét Itanium ABI-leírást e jegyzet nem idézi; a könyv forrásjegyzékében megadott dokumentum verziója és a laborprofilban rögzített compiler az irányadó. A fenti mechanizmusok tehát illusztrációk, nem a szabvány által megkövetelt reprezentáció.

24.5.6 Alternatíva: kompozíció és type erasure

A diamond és a virtuális bázis gyakran elkerülhető kompozícióval. A két viselkedést külön tagként tartjuk, és delegálunk:

```cpp
#include <cstdio>

struct Input {
    void write(const char* s) { std::printf("in: %s\n", s); }
    int in = 1;
};
struct Output { int out = 2; };

struct Duplex {
    Input input;
    Output output;
    void write(const char* s) { input.write(s); }
    int own = 3;
};

int main() {
    Duplex d;
    d.write("hello");
}
```

Itt nincs pointer adjustment, nincs virtuális bázis, és a delegálás explicit. Cserébe több egyszerű továbbító kód keletkezik, és a `Duplex` nem helyettesíthető be egy `Stream&` paraméterbe. Ha mégis egységes dinamikus interfész kell, a **type erasure** (típuseltüntetés) lehet a következő lépés: egyetlen absztrakt `Stream` interfész és `std::unique_ptr<Stream>` birtoklás, vagy `std::function`/`std::any` használata. Ezeket unique_ptr és egyedi tulajdonlás (21.3) és invoke, function wrapper és type erasure (30.2) részletezi majd.

A döntés szempontjai röviden: ha a közös viselkedés egyetlen tengelyen marad, a kompozíció egyszerűbb; ha a típusnak több, egymástól független szerződést kell teljesítenie, a többszörös interfész-öröklés indokolt lehet; ha a közös ős mégis megjelenik, a `virtual` öröklés vagy a kompozíció közül kell választani. A mély, sokágú hierarchia általános ajánlásként nem tárgya ennek a résznek.

24.5.7 Tipikus hibák és javításuk

A leggyakoribb hiba a `virtual` kulcsszó elhagyása a köztes base-en. Ettől a közös ős kétszer kerül az objektumba, a nevek kétértelművé válnak, és a közös állapot duplikálódik. A javítás a base-lista `virtual` jelölése, vagy a hierarchia átalakítása kompozícióvá.

A második gyakori hiba, hogy a köztes osztályban inicializáljuk a virtuális bázist, és azt hisszük, hogy az futni fog. Ha az adott osztály nem most-derived, az initializer figyelmen kívül marad; ezt a viselkedést a fenti `V(1)`/`V(3)` példa mutatta.

A harmadik hiba a `reinterpret_cast` használata base pointerek között. Ez nem végzi el a pointer adjustmentet, ezért rossz címre mutathat. A helyes eszköz a `static_cast`, illetve a dinamikus esetben a `dynamic_cast`.

További hibák: pointeraritmetika virtuális bázisra mutató pointerrel (a virtuális bázis nem teljes objektum, ezért az aritmetika nem definiált); `delete` virtuális bázis pointerén keresztül virtuális destruktor nélkül (ez általános szabály, lásd Virtuális destruktor, slicing és ownership (24.3)); slicing, amikor a származtatott objektumot base értékkel másoljuk; valamint a base subobjectek címének `void*`-ként való összehasonlítása objektumazonosság helyett.

24.5.8 Feladatok

1. **Szerkezeti magyarázat.** A `diamond_ok.cpp` kimenetében miért `V(3)` az első sor, és miért nem `V(1)`? Mi történik, ha a `D` konstruktorából töröljük a `V(3)` inicializálót?
2. **Átalakítás kompozícióvá.** Alakítsd át a `diamond_ok.cpp` hierarchiáját kompozícióvá, őrizd meg a `write`/`f` viselkedést, és hasonlítsd össze a `sizeof` értékeket. Indokold, miért nem hordozható a különbség.
3. **Diagnózis és javítás.** A `diamond_bad.cpp` nem fordul. Adj három javítási stratégiát, és mindegyiknél mondd meg a szemantikai következményt: mit nyerünk, és mit veszítünk?
4. **ABI-megfigyelés.** Írasd ki a `static_cast<L*>(&obj)`, `static_cast<R*>(&obj)` és `static_cast<V*>(&obj)` címeket GCC-vel és Clang-gel, lehetőleg azonos optimalizálási szinten. Mit bizonyít ez a mérés, és mit nem?

24.5.9 Megoldások

24.5.9.1 Szerkezeti magyarázat

A `D` a most-derived objektum, ezért a virtuális `V` bázist a `D` konstruktora inicializálja; a `D() : V(3)` initializer fut, az `L() : V(1)` és `R() : V(2)` initializerek pedig figyelmen kívül maradnak. Ha a `V(3)`-at töröljük, a `V` default konstruktorára lenne szükség. Mivel a `V`-nek csak `explicit V(int)` konstruktora van, a program fordítási hibát ad. Ha lenne default konstruktor, az futna, és az `L` initializerét akkor is figyelmen kívül hagynánk. Ellenőrzési feltétel: a program `-Wall -Wextra -Wpedantic` mellett figyelmeztetés nélkül forduljon, és a kimenet első sora `V(3)` legyen; a törölt változat adjon fordítási hibát.

24.5.9.2 Átalakítás kompozícióvá

A `Duplex` példa szerinti megoldás: `Input input; Output output;` tagok, a `write` pedig delegál az `input.write`-ra. A `V` közös ős szerepét itt a `Duplex` saját `write` függvénye veszi át. A `sizeof` értékek összehasonlítása azt mutatja, hogy a kompozíció jellemzően kisebb és kiszámíthatóbb layoutot ad, de ez nem garantált: az EBO, a padding és az ABI mind befolyásolják. A mérés csak az adott compilerre és targetre érvényes; általános érvényű kijelentést nem lehet belőle levonni. Ellenőrzési feltétel: a `write` kimenete azonos legyen, és a program mindkét változatban forduljon.

24.5.9.3 Diagnózis és javítás

Az első javítás a `virtual` öröklés a két köztes base-en: `struct L : virtual V {}; struct R : virtual V {};`. Ettől egyetlen `V` subobject jön létre, a `d.f()` egyértelmű lesz, és a közös állapot nem duplikálódik. A második javítás a minősített hívás vagy `using` deklaráció: `d.L::f()` vagy `struct D : L, R { using L::f; };`. Ettől a hívás egyértelmű, de a két `V` subobject megmarad, tehát a `V` állapota duplikálódik, és a `dynamic_cast`/pointerkonverziók továbbra is a két subobject valamelyikére mutatnak. A harmadik javítás a kompozíció: nincs öröklési diamond, nincs pointer adjustment, de a `D` nem helyettesíthető be `V&` paraméterbe. Ellenőrzési feltétel: mindhárom változat forduljon; a második esetben ellenőrizzük, hogy a két `V` subobject címe különbözik.

24.5.9.4 ABI-megfigyelés

A mérés megmutatja, hogy az adott compiler és target hogyan helyezi el a subobjecteket, és milyen offseteket használ. Ez hasznos a laborprofil dokumentálásához, de nem bizonyít nyelvi garanciát: a szabvány nem írja elő sem a vtable-t, sem a konkrét offseteket. A két compiler eltérő eredményt adhat, és ugyanaz a compiler más optimalizálási szinten vagy más targeten is eltérhet. Az eredményt ezért a compilerverzióval, a target ABI-val és a fordítási kapcsolókkal együtt kell rögzíteni. Ellenőrzési feltétel: a program mindkét compilerrel forduljon, és a kapott címek/offsetek a laborjegyzőkönyvben dokumentálva legyenek.

24.5.10 Összefoglaló

A többszörös öröklésnél minden nem-virtualis base-hez külön base subobject tartozik; a `static_cast` végzi el a szükséges pointer adjustmentet. A diamond akkor keletkezik, amikor egy közös ős két útvonalon érhető el; virtual inheritance nélkül az ős kétszer szerepel, és a nevek kétértelművé válnak. A `virtual` base-lista egyetlen közös subobjectet hoz létre, amelyet a most-derived osztály inicializál; a köztes osztályok initializerei figyelmen kívül maradnak. A kétértelműséget minősítéssel vagy `using` deklarációval lehet feloldani. A vptr, vtable, vbase-offset és thunk ABI-szintű fogalmak, amelyeket a szabvány nem követel meg; a konkrét részleteket a laborprofilban kell ellenőrizni. A kompozíció és a type erasure gyakran egyszerűbb alternívát kínál, különösen akkor, ha a közös viselkedés egyetlen tengelyen marad.

25 Kivételek és C++ hibakezelési stratégiák

25.1 throw, catch és stack unwinding

A C-ben megszokott hibakezelés visszaadott értékre, `errno`-ra és `goto cleanup`-ra épül: a hiba a normál vezérlési folyam része, és a hívó felelőssége észrevenni. A C++ kivétel (exception) más szerződést kínál: a hiba nem visszafelé, visszatérési értékben utazik, hanem *oldalra lép ki* a normál folyamból, és a hívási láncon felfelé haladva addig keres handlert, amíg talál egyet, vagy a program le nem áll. Ez a mechanizmus akkor jó, ha a hiba ritka, a hiba okát nem lehet a helyszínen kezelni, és a köztes rétegeknek nem kell tudniuk róla — tipikusan ez a helyzet egy konfigurációbetöltő, egy protokollparser vagy egy hálózati réteg esetében.

Ez az alfejezet a kivétel *útját* követi: hol jön létre a hibaobjektum, meddig él, hogyan illeszkedik a handlerhez, és közben milyen sorrendben futnak le a felépült RAII objektumok destruktorai. A hibastratégiák összehasonlítását (`error_code`, `expected`, `noexcept` határok) a noexcept, swap és container-reallocation részre építve a következő alfejezet tárgyalja részletesen; itt csak annyit használunk belőlük, amennyi az unwinding megértéséhez kell. A slicing és az ownership szempontjából támaszkodunk a Virtuális destruktor, slicing és ownership részre.

25.1.1 A throw kifejezéstől a catch handlerig

A `throw` kifejezés egy *exception object*-et hoz létre. Pontosabban: a `throw` operandusából a rendszer egy külön, a dobó fókusztól független tárolón lévő objektumot inicializál, és a vezérlést egy kivételként jelölt állapotba helyezi. Az exception object típusát **a `throw` operandusának statikus (fordítás idejű) típusa** határozza meg, nem a dinamikus típusa. Ez a legfontosabb egyetlen mondat az egész mechanizmusban, mert ezen múlik a `catch` illesztése és a slicing elkerülése.

A dobás után a program a hívási láncon felfelé keres egy `catch` handlert. A keresés a dobás helyét *körülvevő* `try` blokkok handlereivel kezdődik, majd a hívó függvény `try` blokkjaival folytatódik, és így tovább. Az illesztés szabálya szűk, szándékosan: egy `H` handler akkor illeszkedik egy `T` statikus típusú exception objecthez, ha

- `H` és `T` azonos típus (a legfelső szintű `const`/`volatile` minősítőket figyelmen kívül hagyva), vagy
- `H` a `T` egy egyértelmű, hozzáférhető bázisosztálya (érték vagy referencia formában), vagy
- `H` és `T` pointer, és a `H` által mutatott típus a `T` által mutatott típus bázisa, illetve `H` `void*`/`const void*`, vagy
- `H` a `catch (...)` mindent elkapó alak.

Semmi más nem számít: nincs `int` → `double` standard konverzió, nincs felhasználói konverziós operátor, és nincs lefelé castolás (a `Derived` handlert nem fogja el a `Base` statikus típusú exception object). Ezért a handler-sorrend nem esztétikai kérdés: a handlereket a forrásban elfoglalt sorrendben próbálja a rendszer, és az első illeszkedő nyer. Ha a bázis-handler áll elöl, a származtatott handler elérhetetlen (dead code), amit egyes fordítók `-Wexceptions` mellett jelezhetnek, de ez nem garantált — a sorrendet kézzel kell ellenőrizni.

25.1.2 Az exception object élettartama és a catch típusillesztése

Az exception object a `throw` végrehajtásakor jön létre, és akkor semmisül meg, amikor az alábbi két esemény közül a *későbbi* bekövetkezik (C++20 [except.throw]/4): (1) az utolsó rá vonatkozó aktív handler kilép, de **nem** újradobással; (2) megszűnik az utolsó rá hivatkozó `std::exception_ptr`. A `throw;` tehát nem zárja le az élettartamot: ugyanazt az objektumot dobja tovább, és a külső handler ugyanazt kapja. A `throw e;` ezzel szemben a handler paraméteréből egy **új** exception objectet inicializál (másolással), és ha `e` bázistípusú referencia, ez a másolat szeletel is. Ez a hibaadat *tulajdonosa*: nem a dobó lokális változója, nem a handler paramétere, hanem egy külön objektum, amelyet a `throw` másolással/mozgatással hozott létre a lokális objektumból. Ha a dobó függvény lokális változóját dobnánk ki referenciaként (`throw e;` helyett `throw &e;` vagy hasonló trükkel), az élettartam-szerződést megszegnénk: a lokális változó a függvény elhagyásakor megszűnik, a pointer vagy referencia dangling lesz. Helyes minta: értéket dobni, referenciával elkapni.

```
// exception_lifetime.cpp - C++20: a kivételobjektum élettartama
#include <cstdio>
#include <exception>

struct Tracked {
    int id;
    explicit Tracked(int i) : id(i) { std::printf("ctor %d\n", id); }
    Tracked(const Tracked& o) : id(o.id + 100) { std::printf("copy %d\n", id); }
    ~Tracked() { std::printf("dtor %d\n", id); }
};

int main() {
    std::exception_ptr saved;
    try {
        try {
            throw Tracked{1};                       // kivételobjektum: id 1
        } catch (const Tracked& t) {
            saved = std::current_exception();       // második hivatkozás
            std::printf("handler 1: %d\n", t.id);
            throw;                                  // ugyanaz az objektum megy
             ↪ tovább
        }
    } catch (const Tracked& t) {
        std::printf("handler 2: %d (nincs új másolat)\n", t.id);
    }                                               // utolsó handler kilép...
    std::puts("handlerek után, saved még él");      // ...de az exception_ptr
     ↪ tartja
    try {
        std::rethrow_exception(saved);
    } catch (const Tracked& t) {
        std::printf("rethrow_exception: %d\n", t.id);
    }
    saved = nullptr;                                // utolsó hivatkozás megszűnik
    std::puts("saved = nullptr után");
    try {
        try { throw Tracked{2}; }
        catch (const Tracked& t) { throw t; }       // throw t: ÚJ kivételobjektum
```

```
    } catch (const Tracked& t) {
        std::printf("throw t után: %d\n", t.id);
    }
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 exception_lifetime.cpp` kimenete: `ctor 1`, `handler 1: 1`, `handler 2: 1 (nincs új másolat)`, `handlerek után, saved még él`, `rethrow_exception: 1`, `dtor 1`, `saved = nullptr után`, majd `ctor 2`, `copy 102`, `dtor 2`, `throw t után: 102`, `dtor 102`. Az 1-es objektum a `throw;` után és mindkét handler kilépése után is élt, és csak az utolsó `exception_ptr` nullázásakor semmisült meg. A `std::rethrow_exception` a szabvány szerint másolatot is dobhat; a libstdc++ itt az eredetit dobta (`1`, nem `101`). A `throw t;` új objektumot (`102`) hozott létre.

A handler paramétere egy lokális változó, amelyet az exception objectből inicializálunk. Ha a paraméter érték szerinti (`catch (std::exception e)`), akkor *másolás* történik a statikus handler-típusra: ez a slicing. A származtatott részobjektum adatai elvesznek, a `virtual` hívások a bázisra esnek vissza, és a `catch` blokk végén a másolat destruálódik, nem az eredeti exception object. Ezért a standard minta a `catch (const T& e)`: nem másol, nem vág, és nem engedi módosítani a hibaadatot. A `catch (T& e)` akkor indokolt, ha a handlernek szándékosan módosítania kell a hibaobjektumot (ritka), a `catch (...)` pedig csak akkor, ha tényleg nem érdekel a hiba típusa — ilyenkor viszont a `throw;`-vel való újradobás a helyes továbbítás, nem a hibainformáció eldobása.

A slicing nem csak a `catch` oldalán fenyeget. Már a `throw` is vághat, ha az operandus statikus típusa bázis:

```
// catch_match.cpp - fordítás: g++ -std=c++20 -Wall -Wextra -O2 -o catch_match
 ↪  catch_match.cpp
#include <cstdio>
#include <stdexcept>

struct ConfigError : std::runtime_error {
    int line;
    ConfigError(const char* m, int l) : std::runtime_error{m}, line{l} {}
};

int main() {
    try {
        ConfigError e{"bad key", 42};
        std::runtime_error sliced = e;   // itt már elveszett a line tag
        throw sliced;                    // statikus típus: std::runtime_error
    } catch (const ConfigError& e) {     // nem illeszkedik
        std::printf("config error at %d\n", e.line);
    } catch (const std::runtime_error& e) {
        std::printf("runtime error: %s\n", e.what());
    }
}
```

Az illusztrált kimenet `runtime error: bad key`; a `ConfigError` handler soha nem fut le, mert a dobott statikus típus `std::runtime_error`. Ha a `throw e;` állna ott (`e` típusa `ConfigError`), akkor az első handler kapná el. Ez a különbség a `throw e;` és a `throw sliced;` között kizárólag a statikus típuson múlik, és ez a leggyakoribb oka annak, hogy egy „származtatott hibát” látszólag nem lehet elkapni.

25.1.3 Stack unwinding és RAII: a cleanup sorrendje

Amikor a kivétel elhagy egy függvényt, a program *nem* ugorja át a köztes állapotot: minden olyan automatikus tárolási idejű objektum, amelynek konstrukciója abban a blokkban befejeződött, destruálódik, a konstrukció sorrendjének **fordítottjában**. Ez a stack unwinding. A blokkok belülről kifelé ürülnek: először a legbelső blokk lokálisai, majd a külső blokk lokálisai, majd a függvényparaméterek és a függvény saját lokálisai a hívó kontextusában, és így tovább a hívási láncon. A `catch` blokkba már csak a hívó frame-jei élnek; a dobó függvény minden RAII objektuma addigra felszabadult.

Ez az, amiért a RAII (erőforrás-életciklus kötése objektum-élettartamhoz) és a kivétel együtt működik: a `FileDescriptor`, `std::lock_guard`, `std::unique_ptr` destruktora akkor is lefut, ha a köztes kód kivétellel lép ki. A szerződés két oldala: (1) a destruktornak *nem szabad* kivételt dobnia, különben a kivételkezelés közben újabb kivétel indul, és a program `std::terminate`-mel leáll; ezért a destruktorok C++11 óta implicit `noexcept(true)`-k, hacsak nem jelöljük őket `noexcept(false)`-nak — ezt a lehetőséget a gyakorlatban kerülni kell. (2) Az erőforrás tulajdonjogát a konstruktorban meg kell szerezni és tagnak kell adni, hogy az unwinding lássa.

Tekintsük a teljes példát, amely egyszerre mutatja a normál cleanup sorrendet és a konstruktorhibát:

```
// unwinding.cpp - fordítás: g++ -std=c++20 -Wall -Wextra -O2 -o unwinding
 ↪  unwinding.cpp
#include <cstdio>
#include <stdexcept>
#include <string>
#include <utility>

struct Resource {
    std::string name;
    explicit Resource(std::string n) : name(std::move(n)) {
        std::printf("acquire %s\n", name.c_str());
    }
    ~Resource() noexcept { std::printf("release %s\n", name.c_str()); }
    Resource(const Resource&) = delete;
    Resource& operator=(const Resource&) = delete;
};

struct Session {
    Resource conn{"conn"};
    Resource log{"log"};
    explicit Session(bool fail) {
        if (fail) throw std::runtime_error{"session setup failed"};
        std::printf("session ready\n");
    }
    ~Session() noexcept { std::printf("release Session\n"); }
};

void run(bool fail) {
    Resource guard{"guard"};
    Session s{fail};          // ha ez dob, a guard már felépült
    std::printf("run done\n");
}
```

```
int main() {
    try {
        run(true);
    } catch (const std::exception& e) {
        std::printf("caught: %s\n", e.what());
    }
}
```

Az illusztrált kimenet (`run(true)` esetén) a következő sorrendet mutatja:

```
acquire guard
acquire conn
acquire log
release log
release conn
release guard
caught: session setup failed
```

Érdemes végigkövetni: a `guard` a `run` blokkjában épült fel elsőként, ezért utolsóként destruálódik. A `Session` konstruktora dobott, ezért a `Session` destruktora **nem** fut le — az objektum soha nem épült fel teljesen. Viszont a `Session` két tagja, a `conn` és a `log`, amelyek konstrukciója befejeződött, igen, fordított sorrendben (`log`, majd `conn`). Ezután a `run` frame-jében a `guard` következik. A `catch` blokkba már csak a `main` `try` blokkja él, és ott a `const std::exception&` handler elkapja a `std::runtime_error`-t a bázisosztály-szabály miatt.

25.1.3.1 Konstruktorhiba és részlegesen felépült részobjektumok

A fenti példa a legfontosabb szerkezeti szabályt demonstrálja: **ha egy konstruktor teste vagy egy tag/bázis inicializálója kivételt dob, akkor a már teljesen felépült bázisok és tagok destruálódnak, de magának az objektumnak a destruktora nem fut le.** Az objektum ugyanis nem számít felépültnek, így a destruktorára vonatkozó szerződés („a konstruktor sikeres befejezése után hívd”) nem teljesül. Következmény: minden erőforrást, amit a konstruktor megszerez, *tagnak* kell lennie, mert a tagok destruktora az unwinding során lefut; a konstruktor törzsében „kézzel” nyitott erőforrás viszont szivárog, ha a törzs közben dob.

A delegáló konstruktor (delegating constructor) esete finomabb: ha a delegált (cél) konstruktor sikeresen lefut, az objektum teljesen felépült, tehát ha a delegáló konstruktor törzse ezután dob, a teljes objektum destruktora lefut. Ilyenkor a destruktornak fel kell készülnie arra, hogy egy „majdnem kész” állapotot takarítson el.

25.1.4 Rethrow, nested exception és exception_ptr

A `throw;` utasítás az aktuális exception objectet dobja újra, *másolás nélkül*. Csak aktív handler belsejében (vagy onnan hívott függvényben, ahol a kivétel még kezelés alatt áll) legális; ha nincs aktív kivétel, a program `std::terminate`-mel leáll. A rethrow a helyes eszköz arra, hogy egy handlert részlegesen kezeljünk (pl. naplózzunk, majd továbbadjuk), anélkül hogy elveszítenénk az eredeti típust vagy az exception object azonosságát.

A hiba kontextusának megőrzésére szolgál a *nested exception* (beágyazott kivétel): a `std::throw_with_nested(new_exception)` a jelenleg kezelt kivételt egy új exception objectbe

csomagolja, és azt dobja. A `std::rethrow_if_nested(e)` a handlerben kibontja a belső kivételt, így a hibalánc megjeleníthető anélkül, hogy a rétegek egymás típusait ismernék. Az `std::exception_ptr` ezzel rokon: egy `std::current_exception()` hívással megszerezhető, másolható, akár szálak között átvihető *tulajdonló* fogantyú, amely életben tartja az exception objectet, és `std::rethrow_exception(ptr)` hívással később újradobható. Ez a minta akkor hasznos, ha a hibát egy háttérszálban keletkeztetjük, de a fő szálon kell jelenteni.

```
// nested.cpp - fordítás: g++ -std=c++20 -Wall -Wextra -O2 -o nested nested.cpp
#include <cstdio>
#include <exception>
#include <stdexcept>

void load_config() {
    try {
        throw std::runtime_error{"bad token at line 7"};
    } catch (...) {
        std::throw_with_nested(std::runtime_error{"config load failed"});
    }
}

int main() {
    try {
        load_config();
    } catch (const std::exception& e) {
        std::printf("outer: %s\n", e.what());
        try {
            std::rethrow_if_nested(e);
        } catch (const std::exception& inner) {
            std::printf("inner: %s\n", inner.what());
        }
    }
}
```

Az illusztrált kimenet `outer: config load failed`, majd `inner: bad token at line 7`. A `std::exception_ptr` itt nem szerepel, de a `throw_with_nested` mögött is egy `exception_ptr`-szerű kapcsolat tartja életben a belső kivételt; a különbség az, hogy a `nested_exception` bázis része az új exception objectnek, míg a `std::current_exception()` egy önálló fogantyút ad.

25.1.5 Kezeletlen kivétel: terminate és abort

Ha a keresés a `main`-ig eljut és nem talál illeszkedő handlert — vagy ha egy `noexcept` függvényből kivétel lép ki, vagy ha a kivétel terjedése közben egy destruktor dob —, akkor a program `std::terminate`-et hív. A `terminate` alapértelmezett viselkedése a `std::abort`, de a `std::set_terminate`-tel saját terminálló handler állítható be, amely naplózhat vagy utolsó pillanatban diagnosztikát írhat. Fontos, hogy a szabvány **nem garantálja**, hogy a `terminate` hívása előtt a teljes stack unwinding lefut: az implementáció dönti el, mennyi cleanup történik. Ezért arra építeni, hogy „a destruktorok mindenképp lefutnak, mielőtt a program meghal”, hibás feltételezés. A `noexcept` függvényből kiszökő kivétel és a destruktorból kiszökő kivétel tehát nem „kezeletlen hiba”, hanem *azonnali leállás* — ez a `noexcept` garancia ára, és ezért kell a destruktorokat és a `swap`-eket hibamentesen megírni.

25.1.6 Tipikus hibák

- **Érték szerinti catch.** `catch (std::exception e)` vághat, és a `catch` blokk végén a másolatot destruálja. Helyette `catch (const std::exception& e)`.
- **Rossz handler-sorrend.** A bázis-handler elé helyezett származtatott handler elérhetetlen. A legbővebbtől (bázis) a legszűkebbig kell haladni, a `catch (...)`-ot pedig utolsóként hagyni.
- **`throw new T(...)`.** A dobott pointert senki nem fogja törölni; a hívó nem tudhatja, hogy neki kell `delete`-elnie. Értéket kell dobni, nem pointert (kivéve, ha a pointer egy létező, hosszú életű objektumra mutat, és ezt dokumentáltuk).
- **Lokálisra mutató pointer/referencia dobása.** A lokális a dobás után megszűnik, a handler dangling pointert kap. Értéket dobjunk.
- **Kivételt dobó destruktor.** A kivételkezelés közben induló újabb kivétel `terminate`-hez vezet. A destruktort `noexcept`-ként kell megírni, és a tisztítást hibamentesen kell elvégezni.
- **Erőforrás a konstruktor törzsében.** Ha a törzs később dob, a korábban kézzel megszerzett erőforrás szivárog. A tagokba kell tenni.
- **A `catch (...)` mindent elnyel.** Ha nem dobjuk tovább (`throw;`), a hiba típusa és kontextusa elveszik, a hívó nem tud differenciálni.

25.1.7 Feladatok

1. *(magyarázó)* A `unwinding.cpp` példában írd le a kimenet sorrendjét `run(false)` esetén, és indokold minden sor helyét. Mi változik, ha a `Session` konstruktorában a `throw` után egy `std::printf` áll?

2. *(kódolási)* Az alábbi osztály exception-unsafe: a konstruktor törzsében szerez két erőforrást, és a második megszerzése dobhat. Alakítsd át úgy, hogy hiba esetén ne szivárogjon semmi, és a `Connection` destruktora továbbra is futtassa a szükséges tisztítást.

```
struct Connection {
    int a = -1;
    int b = -1;
    Connection() {
        a = open_a();       // sikerül
        b = open_b();       // dobhat
    }
    ~Connection() { if (a >= 0) close_a(a); if (b >= 0) close_b(b); }
};
```

3. *(diagnosztikai)* Az alábbi kódrészlet három hibát tartalmaz. Nevezd meg mindet, mondd meg, mi a következményük, és írd le a javítást.

```
struct Base { virtual ~Base() = default; };
struct Derived : Base {};

void handle() {
    try {
        may_throw();
    } catch (std::exception e) {              // (1)
        log(e.what());
        throw new std::runtime_error{"wrapped"};   // (2)
    }
```

```
}

void g() {
    try { throw Derived{}; }
    catch (const Base&) { /* ... */ }          // (3)
    catch (const Derived&) { }
}
```

25.1.8 Megoldások

1. megoldás. `run(false)` esetén a `Session` teljesen felépül, ezért a destruktora is lefut. A sorrend: `acquire guard`, `acquire conn`, `acquire log`, `session ready`, `run done`, majd a `run` blokkjának végén a fordított sorrend: `release Session`, `release log`, `release conn`, `release guard`. Az indoklás: a blokkon belüli automatikus objektumok a konstrukció sorrendjének inverzében destruálódnak, és a `Session` saját destruktora a tagjai előtt fut, mert a tagok a `Session` részobjektumai. Ha a `throw` után egy `std::printf` állna, az már nem futna le, mert a `throw` azonnal elhagyja a konstruktor törzsét — a konstruktor nem fejeződik be, a `Session` destruktora nem fut, de a `conn` és a `log` tag igen. Ellenőrzési feltétel: a kimenetben a `release Session` hiányzik, a `release log` és `release conn` viszont megjelenik a `guard` felszabadítása előtt.

2. megoldás. A két erőforrást külön RAII tagba kell tenni, hogy a tagok destruktora végezze a tisztítást, és a `Connection` törzsében ne maradjon „kézi” erőforrás:

```
struct Handle {
    int fd = -1;
    explicit Handle(int f) : fd{f} {}
    ~Handle() noexcept { if (fd >= 0) close_handle(fd); }
    Handle(const Handle&) = delete;
    Handle& operator=(const Handle&) = delete;
};

struct Connection {
    Handle a{open_a()};    // ha ez dob, nincs mit takarítani
    Handle b{open_b()};    // ha ez dob, az a destruktora lefut
};
```

Indoklás: a `Connection` konstruktora most a taginicializáló listán keresztül épül fel; ha az `open_b()` dob, akkor az `a` tag már teljesen felépült, ezért a `~Handle` lefut, a `~Connection` viszont nem. A kézi `if (a >= 0) close_a(a)` logika a `~Handle`-be került, így az unwinding „látja”. Ellenőrzési feltétel: egy olyan tesztben, ahol az `open_b()` kivételt dob, az `open_a()` által lefoglalt erőforrás felszabadul, és a folyamat nem szivárog (a laborban pl. számlálóval vagy `strace`-csel ellenőrizhető, hogy a `close` megtörtént).

3. megoldás. (1) Érték szerinti catch: `std::exception e` lemásolja és levágja az exception objectet, így a `catch` blokkban a dinamikus típus elveszik, a `catch` végén pedig a másolat destruálódik. Javítás: `catch (const std::exception& e)`. (2) `throw new std::runtime_error{...}`: a dobott pointer tulajdonosa ismeretlen, a hívó nem tudja, hogy `delete`-elni kell-e; ráadásul a `new` maga is dobhat `std::bad_alloc`-ot, ami elfedheti az eredeti hibát. Javítás: `throw std::runtime_error{"wrapped"};`, vagy ha meg akarjuk őrizni az eredeti kontextust, `std::throw_with_nested(std::runtime_error{"wrapped"})`. (3) A `catch (const Base&)` előbb áll, mint a `catch (const Derived&)`, ezért a `Derived` handler soha nem fut le — a `throw Derived{}` statikus típusa `Derived`, de a bázis-handler illeszkedik rá, és az első

illeszkedő nyer. Javítás: a `Derived` handlert a `Base` elé kell tenni. Ellenőrzési feltétel: a javított változatban a `Derived` ág fut le, és a `catch` blokkba lépéskor a `what()`/tagok a várt dinamikus típust mutatják.

25.1.9 Összefoglaló

A `throw` egy külön élettartamú exception objectet hoz létre, amelynek típusát a `throw` operandusának statikus típusa határozza meg; ezért a származtatott hiba dobásához a statikus típusnak is származtatottnak kell lennie, és a handlert `const&`-sel kell elkapni a slicing elkerülésére. A handlerek sorrendben illeszkednek, csak azonos típusra, bázisra, pointerre és `catch (...)`-ra; a bázis-handler elé tett származtatott handler elérhetetlen. Az unwinding a felépült automatikus objektumokat a konstrukció fordított sorrendjében destruálja, a blokkok belülről kifelé ürülnek, és a `catch` blokkba már csak a hívó frame-jei élnek. Konstruktorhiba esetén a teljesen felépült bázisok és tagok destruálódnak, maga az objektum nem, ezért az erőforrásokat tagokba kell tenni. A `throw;` másolás nélkül dob újra, a `throw_with_nested`/`rethrow_if_nested` és az `exception_ptr` a hibalánc megőrzésére szolgál, a kezeletlen kivétel pedig `std::terminate`-hez vezet, amelynek pontos unwinding-viselkedése implementációfüggő.

Ehhez az alfejezethez nem állt rendelkezésre ellenőrzött forrásjegyzet; a szövegben szereplő szabványszekciónevek (`[except.throw]`, `[except.ctor]`, `[except.handle]`, `[except.terminate]`, `[except.nested]`) a C++20 szerkezetére utaló hivatkozások, amelyek konkrét szövegét és a fordítók viselkedését a tanulónak kell a saját laborprofiljában ellenőriznie. A kódok kimenete illusztráció, nem mért eredmény.

25.2 No-throw, strong és basic exception guarantee

A kivételkezelés nem önmagában a `try`/`catch` szintaxisa, hanem egy szerződés arról, hogy egy művelet **meghiúsulása esetén milyen állapotban marad a program**. Egy DevOps-környezetben futó komponensnél ez közvetlenül üzemeltetési kérdés: ha egy konfigurációfrissítés vagy egy kötegelt állapotátmenet félúton elhasal, attól még a szolgáltatásnak konzisztens állapotban kell maradnia, és a hiba tényét tovább kell tudni adni. Ezért a garanciákat nem elvont minőségi jelzőként, hanem az adott típus invariánsához kötött, dokumentált ígéretként kezeljük. Az alábbiakban a throw, catch és stack unwinding mechanizmusára és a mély másolás, értéktípus és exception guarantee alapfogalmaira építünk; a hibakód-alapú alternatívák részletes tárgyalása az error_code, system_error és expected részre tartozik.

25.2.1 A garanciák és az invariáns kapcsolata

Minden osztálynak van egy **invariánsa** (invariant): olyan állítás, amely a nyilvános műveletek között minden pillanatban igaz az objektumra. Például: „a költségvetés sorai nem tartalmaznak negatív egyenleget”, „a név és a hozzá tartozó szabálylista ugyanahhoz a verzióhoz tartozik”. Egy művelet **előfeltétele** (precondition) az, amit a hívónak biztosítania kell; **utófeltétele** (postcondition) az, ami siker esetén teljesül. Az exception guarantee ezekhez képest a *hibaág* szerződése.

A három szint pontosabban:

- **No-throw (nem dobó) garancia:** a művelet nem lép ki kivétellel. Ha belső hibát észlel, vagy előfeltétel-sértést jelez, azt a dokumentált, nem kivételes csatornán teszi (visszatérési

érték, `std::error_code`, `std::optional`, állapotflag). Feltétel: a művelet minden tranzitívan hívott művelete – allokáció, másolás, összehasonlítás, destruktorok – is no-throw, vagy a hiba nem kerül át a határon.
- **Erős (strong) garancia:** ha a művelet kivétellel meghiúsul, az objektum (és a művelet által érintett megfigyelhető állapot) **pontosan úgy marad, ahogy a hívás előtt volt**. Commit-or-rollback: vagy teljes egészében megtörténik a változás, vagy egyáltalán nem. Feltétele, hogy a hibát okozó lépések előbb történjenek meg, mint bármilyen visszafordíthatatlan módosítás, és a „véglegesítő" lépés maga ne dobhasson.
- **Alap (basic) garancia:** hiba esetén nincs erőforrás-szivárgás, nem keletkezik érvénytelen objektum, és az invariáns helyreáll – de az objektum értéke nem specifikált lehet, a művelet részleges hatása megmaradhat. Ez a leggyengébb elfogadható szint, nem azonos a „semmi garancia" esettel.

A szintek viszonya: az erős garancia implikálja az alapot (hiszen rollback esetén is konzisztens marad az állapot). A no-throw tulajdonképpen egy másik dimenzió: nem azt mondja meg, mi történik hiba esetén, hanem hogy hiba nem távozhat kivételként. Egy no-throw művelet a kivétel szempontjából triviálisan kielégíti az erős garanciát, mert nincs hibaág – gyakorlatban mégis külön szerződésként dokumentáljuk, mert a hívó kód és a `noexcept` alapú optimalizációk (például konténer-áthelyezés) ezt használják. Fontos: a `noexcept` kulcsszó **nem hozza létre** a no-throw garanciát, csak kikényszeríti; ha a jelölt művelet mégis dob, a program `std::terminate`-mel leáll. A garancia dokumentált tulajdonság, a `noexcept` annak nyelvi deklarációja.

25.2.2 Tranzakcionális frissítés: commit és rollback

Az erős garancia legfontosabb mintája a **copy-and-swap** (másolj, majd cserélj), illetve általánosabban a „dolgozz ideiglenes állapoton, és a végén commitálj" elv. A műveletet három fázisra bontjuk:

1. **Érvényesítés és előkészítés** – minden olyan lépés, ami dobhat (validáció, allokáció, másolás). Itt még semmi nem változik a célobjektumon.
2. **Commit** – a nem dobó lépés, ami az új állapotot láthatóvá teszi. Ez tipikusan egy `swap` vagy egy mutató/`unique_ptr` cseréje.
3. **Takarítás** – a régi állapot felszabadítása, ami szintén nem dobhat (destruktorok).

A tranzakcionális frissítés kulcsa a **sorrend**: a dobható műveletek megelőzik a nem dobó commitot. Ha fordítva teszünk, a hibaág már módosított állapotot talál, és csak alap garanciát adhatunk.

Tekintsünk egy DevOps-hoz illő példát: egy `Budget` osztály tokensorokat tart, és a teljes sorozatot kell lecserélni egy új konfigurációra. A rossz változat előbb töröl, aztán újratölt:

```
// Rossz: csak alap garancia - a részben feltöltött vektor érvényes, de nem a
 ↪  régi
void replace_all_basic(const std::vector<Bucket>& next) {
    buckets_.clear();                                   // eredeti elveszett
    buckets_.insert(buckets_.end(), next.begin(), next.end()); // dobhat
}
```

Ha az `insert` allokációs hibával vagy a `Bucket` másoló konstruktorának dobásával elhasal, az objektum részben feltöltött állapotban marad. A helyes változat a paramétert érték szerint veszi át (a másolás a hívás helyén, a függvénytörzs előtt történik, tehát hibája nem érinti a célt), majd egy nem dobó cserével commitál:

```cpp
// budget.cpp - g++ -std=c++20 -Wall -Wextra -Wpedantic -g -o budget budget.cpp
#include <algorithm>
#include <cassert>
#include <stdexcept>
#include <vector>

struct Bucket {
    int id{};
    int tokens{};
    friend bool operator==(const Bucket&, const Bucket&) = default;
};

class Budget {
public:
    // Erős garancia: hiba esetén *this változatlan.
    // Előfeltétel: Bucket másolható/mozgatható. A swap előfeltétele (egyenlő
    // allocatorok) itt mindig teljesül, mert a std::allocator mindig egyenlő.
    void replace_all(std::vector<Bucket> next) {
        validate(next);        // dobhat - még semmit nem módosítottunk
        buckets_.swap(next);   // nem dobhat (std::allocator, mindig egyenlő)
    }

    const std::vector<Bucket>& buckets() const noexcept { return buckets_; }

    bool invariant() const noexcept {
        return std::all_of(buckets_.begin(), buckets_.end(),
                           [](const Bucket& b) { return b.tokens >= 0; });
    }

private:
    static void validate(const std::vector<Bucket>& v) {
        for (const auto& b : v)
            if (b.tokens < 0) throw std::invalid_argument("negatív token");
    }
    std::vector<Bucket> buckets_{};
};

int main() {
    Budget b;
    b.replace_all({{1, 10}, {2, 20}});
    assert(b.invariant());

    const auto before = b.buckets();
    try {
        b.replace_all({{3, 5}, {4, -1}});   // validáció dob
    } catch (const std::invalid_argument&) {
        assert(b.buckets() == before);      // erős garancia ellenőrzése
    }
}
```

A commit egyetlen `swap`. A régi vektor tartalma a paraméterbe kerül, és a függvény kilépésekor a destruktora felszabadítja – ez a rollback „ingyenes” része. A `validate` és a paraméter másolása

mind a commit előtt fut, tehát bármelyikük hibája esetén a `buckets_` érintetlen.

A `swap`-ről két *külön* tényt kell rögzíteni. (1) **Kivételspecifikáció:** a `std::vector::swap` C++17 óta `noexcept`, ha az allocator `propagate_on_container_swap` vagy `is_always_equal` tulajdonságú — a `std::allocator` ilyen. (2) **Előfeltétel:** ha az allocator *nem* propagálódik cserénél, és a két vektor allocatora nem egyenlő, a `swap` hívása nem definiált viselkedés ([container.reqmts]). Ez nem „dobhat”, hanem érvénytelen hívás; ilyen allocator-párra a könyv nem ajánl `swap`-et. A copy-and-swap bizonyításához tehát hozzátartozik egy harmadik feltétel: **a commit `swap` két oldalának allocatora egyenlő**. Állapotú allocatornál (például `std::pmr::polymorphic_allocator`, amely nem propagálódik) ezt úgy biztosítjuk, hogy az ideiglenes másolatot a *cél* allocatorával építjük, akármilyen allocatorral érkezett a bemenet:

```
// budget_alloc.cpp - C++20: copy-and-swap állapotú (pmr) allocatorral
#include <algorithm>
#include <cassert>
#include <memory_resource>
#include <stdexcept>
#include <vector>

struct Bucket {
    int id{};
    int tokens{};
    friend bool operator==(const Bucket&, const Bucket&) = default;
};

class PmrBudget {
public:
    explicit PmrBudget(std::pmr::memory_resource* r) : buckets_(r) {}

    // Erős garancia bármilyen forrásallocatorral érkező bemenetre.
    void replace_all(const std::pmr::vector<Bucket>& next) {
        // A másolat a SAJÁT allocatorunkkal készül: így a swap előfeltétele
        // (egyenlő allocatorok) teljesül, akármi is a next allocatora.
        std::pmr::vector<Bucket> tmp(next.begin(), next.end(),
                                     buckets_.get_allocator());   // dobhat
        validate(tmp);                                            // dobhat
        buckets_.swap(tmp);                                       // noexcept
    }
    const std::pmr::vector<Bucket>& buckets() const noexcept { return buckets_;
     ↪ }

private:
    static void validate(const std::pmr::vector<Bucket>& v) {
        for (const auto& b : v)
            if (b.tokens < 0) throw std::invalid_argument("negatív token");
    }
    std::pmr::vector<Bucket> buckets_;
};

int main() {
    std::pmr::monotonic_buffer_resource own, other;
    PmrBudget b(&own);
    std::pmr::vector<Bucket> in({{1, 10}, {2, 20}}, &other);   // MÁS resource
```

```
    b.replace_all(in);
    assert(b.buckets().get_allocator().resource() == &own);
    assert(b.buckets() == in);

    const std::pmr::vector<Bucket> before = b.buckets();
    std::pmr::vector<Bucket> bad({{3, 5}, {4, -1}}, &other);
    try { b.replace_all(bad); } catch (const std::invalid_argument&) {}
    assert(b.buckets() == before);

    ↪ static_assert(!std::allocator_traits<std::pmr::polymorphic_allocator<Bucket>>
                    ::propagate_on_container_swap::value);
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic -g -fsanitize=address,undefined budget_alloc.cpp` figyelmeztetés nélkül fordul, a futás egyik `assert`-et sem sérti, ASan/UBSan nem jelez: a másik memóriaforrásból érkező bemenet a saját forrásba másolódott, a hibás bemenet után az állapot változatlan, és a `static_assert` igazolja, hogy a `polymorphic_allocator` nem propagálódik cserénél. A másolás a commit előtti, dobható fázisban van, így az erős garancia megmarad.

25.2.3 A garancia függése a típustól és az allocatortól

Az erős garancia nem kizárólag a saját kódunkon múlik: a konténerműveletek garanciája a bennük tárolt típus és az allocator tulajdonságaitól függ. A `std::vector::reserve(n)` tipikusan erős: ha az allokáció `std::bad_alloc`-kal elhasal, a vektor mérete és tartalma változatlan. A `push_back` esetén a szabvány a garanciát feltételekhez köti: ha a kivétel nem a `T` másoló/mozgató konstruktorából vagy értékadásából, illetve bemeneti iterátor műveletéből származik, akkor nincs hatás; egyetlen elem végére illesztésekor akkor erős a garancia, ha `T` másolhatóan beszúrható, vagy a mozgató konstruktora nem dob. Ha viszont egy nem másolható `T` mozgató konstruktora dob, a szabvány az effekteket **nem specifikáltnak** hagyja – ilyenkor még az alap garancia sem garantált a konténer egészére nézve. A pontos megfogalmazás a szabvány adott verziójában ellenőrizendő, mert a megengedett esetek köre verzióról verzióra finomodott; a mérnöki következtetés viszont stabil: **dokumentált garanciát csak akkor ígérj, ha a benne részt vevő típusok és az allocator tulajdonságai ismertek.**

Külön figyelmet érdemel a **kivételobjektum létrehozása**. A `throw` operandusából inicializálódó kivételobjektumot a implementáció másolhatja, és a másolás elhagyása megengedett, de nem kötelező. Ha a kivételtípus másoló konstruktora dob, a folyamat leáll (`std::terminate`), mert egy kivétel dobása közben nem dobódhat újabb kivétel. Az allokáció meghiúsulására sincs hordozható, garantált viselkedés; a gyakorlatban az implementációk vészhelyzeti tárolóval vagy leállással kezelik. Következmény: a kivételtípus legyen kicsi, olcsón és lehetőleg nem dobóan másolható, és ne végezzen allokációt a konstruktorában. Nagy diagnosztikai payload esetén a részleteket ne a kivételben tároljuk közvetlenül, hanem egy külön, a hívó által kezelt csatornán.

25.2.4 Destruktor, cleanup és explicit hibajelzés

A destruktorok C++11 óta implicit módon `noexcept`, és ha egy kivétel futása közben (stack unwinding alatt) újabb kivétel lépne ki egy destruktorból, a program leáll. Ez helyes: a destruktornak **nem szabad hibát kivétellel jeleznie**. A gyakorlati minta, hogy a tisztítás, ami meghiúsulhat – `close`, `flush`, `fsync`, hálózati bontás –, **explicit műveletként** jelenik meg, amelynek visszatérési értéke vagy `std::error_code`-ja van; a destruktor csak best-effort takarít,

és a hibát nem dobja tovább. Így a hívó eldöntheti, hogy a hibát naplózza, újrapróbálja, vagy a folyamatot szabályosan leállítja. Ez a fajta „kétfázisú” cleanup (explicit `commit/close`, majd csendben futó destruktor) a RAII mintát egészíti ki, nem helyettesíti. A részletes hibakód-API-k tárgyalása az error_code, system_error és expected részre tartozik.

25.2.5 Hibainjektálás és állapot-snapshot tesztelés

Az erős garancia állítás, ezért tesztelni kell. A leghatékonyabb módszer a **hibainjektálás**: olyan segédtípust használunk, amely a másolás/konstrukció adott sorszámú előfordulásakor dob, majd minden lehetséges injektálási pontra lefuttatjuk a műveletet. Minden esetben rögzítjük az állapotot a művelet előtt, elkapjuk a kivételt, és összehasonlítjuk a „snapshotot”. Az erős garancia teljesül, ha az összehasonlítás minden injektálási ponton egyenlőséget ad, és az invariáns ellenőrzése is igaz.

```
// fault_inject.cpp - ugyanazzal a buildparanccsal fordítható
#include <cstdio>
#include <stdexcept>
#include <string>
#include <vector>

struct CopyFault {
    static inline int countdown = -1;  // -1: nincs injektálás
    std::string host;

    explicit CopyFault(std::string h) : host(std::move(h)) {}
    CopyFault(const CopyFault& other) : host(other.host) {
        if (countdown >= 0 && countdown-- == 0)
            throw std::runtime_error("injektált másolási hiba");
    }
    CopyFault& operator=(const CopyFault&) = default;
    friend bool operator==(const CopyFault&, const CopyFault&) = default;
};

// A teszt: minden k-ra injektálunk, és ellenőrizzük a rollbacket.
template <class Op>
void check_strong(Op op, int max_steps) {
    for (int k = 0; k < max_steps; ++k) {
        CopyFault::countdown = k;
        try { op(); } catch (const std::runtime_error&) {}
        CopyFault::countdown = -1;
        // itt következne a snapshot és az invariáns összehasonlítása
    }
}
```

A diagnosztikai érték abban rejlik, hogy a teszt megmutatja, *melyik* injektálási ponton sérül a garancia. Az alap garanciát nyújtó `replace_all_basic` esetén jellemzően a középső pontokon bukik meg a snapshot-egyezés, míg a copy-and-swap változat minden ponton átmegy – feltéve, hogy a commit valóban nem dob. A snapshot legyen teljes: a megfigyelhető állapot minden összetevője (méret, tartalom, jelzők) szerepeljen benne, különben a teszt hamis biztonságérzetet ad. Érdemes a `noexcept` műveletekre külön, fordítási idejű ellenőrzést is tenni (`static_assert(noexcept(...))`), és a szándékosan leálló eseteket haláleset-teszttel (death test) lefedni.

25.2.6 Gyakori hibák

- **Erős garanciát dokumentálunk, de a kód `clear()`-rel kezd.** A törlés után minden hiba csak részleges állapotot hagy; a helyes sorrend a commit utáni takarítás.
- **A `swap` no-throw voltát feltételezzük.** Nem egyenlő allocatoroknál a konténer-`swap` viselkedése eltérhet; a saját típusunkban a tag-`swap`-et `noexcept`-ként deklaráljuk és tartsuk is be.
- **Összekeverjük a `noexcept`-ot a no-throw garanciával.** A jelölés önmagában nem tesz egy műveletet nem dobóvá; ha mégis dob, a program leáll.
- **A kivételtípus nagy, allokáló vagy dobó másolóval rendelkezik.** Ez a `terminate` kockázatát növeli a hibaágon.
- **Destruktorban jelzünk hibát.** Unwinding közben ez `std::terminate`; a cleanup hibáját explicit művelettel kell kitenni.
- **Az alap garanciát „nincs szivárgás”-ként értelmezzük.** Az invariánsnak is fenn kell maradnia; a részleges állapot csak akkor elfogadható, ha dokumentáltan konzisztens.
- **A snapshot teszt csak a méretet ellenőrzi.** A tartalmi és jelzőállapot ellenőrzése nélkül a garancia nem igazolt.

25.2.7 Feladatok

1. **(magyarázó)** Egy `std::vector<std::string> v;` objektumon hívott `v.push_back(s)` milyen garanciát ad? Milyen feltétel mellett erős? Hasonlítsd össze a `v.reserve(n)` garanciájával, és fogalmazd meg, miért fontos az eltérés egy hibainjektáló tesztben!

2. **(kódolási)** Készíts `Config` osztályt, amely egy `std::string name_` és egy `std::vector<Rule> rules_` tagot tart, és a kettőt **együtt** kell frissíteni: a `name_` és a `rules_` mindig ugyanahhoz a verzióhoz tartozzon. Valósítsd meg a `void publish(Config next)` műveletet erős garanciával, valamint az `invariant()` ellenőrzést, amely megköveteli, hogy a név nem üres, és a szabályok verziószáma megegyezik a névben jelzett verzióval.

3. **(diagnosztikai)** A fejezet `Budget` osztályához írj hibainjektáló tesztet, amely `k = 0..N` értékekre a `replace_all` minden másolási pontján dob, és minden esetben ellenőrzi a snapshot-egyezést és az invariánst. Mutasd meg, melyik `k` értéknél bukna el a `replace_all_basic`, és indokold, miért!

25.2.8 Megoldások

25.2.8.1 1. megoldás

A `push_back` a szabvány szerint akkor ad erős garanciát egyetlen elem végére illesztésekor, ha a `T` másolhatóan beszúrható, vagy a mozgató konstruktora nem dob; ha a kivétel nem a `T` másoló/mozgató műveletéből vagy bemeneti iterátor műveletéből ered, szintén nincs hatás. Nem másolható, dobó mozgatóval rendelkező `T` esetén az effektek nem specifikáltak lehetnek. A `reserve` ezzel szemben tipikusan erős: allokációs hiba esetén a vektor mérete és tartalma változatlan. A tesztben ezért a `push_back`-et csak olyan típussal érdemes injektálni, amelynek a másolása dobhat, de a mozgatása nem – különben a „bukás” nem a mi kódunk, hanem a szabvány által megengedett bizonytalanság következménye. Ellenőrzési feltétel: a `static_assert(std::is_nothrow_move_constructible_v<T>)` teljesüljön, és a snapshot a méret mellett a tartalmat is hasonlítsa.

25.2.8.2 2. megoldás

A két tag összehangolt frissítését egyetlen `State` értéktípusba vonjuk össze, és azt cseréljük:

```
struct Rule { int id{}; bool enabled{}; friend bool operator==(const Rule&,
 ↪  const Rule&) = default; };

class Config {
public:
    struct State {
        std::string name;
        std::vector<Rule> rules;
    };

    // Erős garancia: a paraméter másolása a hívás helyén történik,
    // a commit egyetlen, nem dobó swap.
    void publish(State next) {
        validate(next);          // dobhat
        state_.swap(next);       // nem dobhat: std::string és std::vector swap
    }

    bool invariant() const noexcept {
        if (state_.name.empty()) return false;
        const int ver = version_of(state_.name);      // pl. "v3" -> 3
        for (const auto& r : state_.rules)
            if (r.id / 1000 != ver) return false;     // példa-összefüggés
        return true;
    }

private:
    static void validate(const State& s) {
        if (s.name.empty()) throw std::invalid_argument("üres név");
        if (s.rules.size() > 10'000) throw std::length_error("túl sok szabály");
    }
    static int version_of(const std::string&);
    State state_{};
};
```

Az indoklás: a `State` értékszemantikájú, a `publish` paramétere érték szerint érkezik, így a másolás és a validáció még a commit előtt lezajlik; a `state_.swap(next)` mindkét tagot egy lépésben cseréli, és nem dob. A régi állapot a paraméterrel együtt megsemmisül. Így a `name_` és a `rules_` soha nem kerülhet ellentmondásos kombinációba. Ellenőrzési feltétel: minden kivételes ágon (`validate` dob, másolás dob) az `invariant()` igaz marad, és a snapshot egyezik; a `publish` deklarációja **nem** `noexcept`, mert a validáció szándékosan dob – a no-throw garancia csak a commit lépésre vonatkozik.

25.2.8.3 3. megoldás

A teszt vázlata: venni egy `Budget` példányt ismert tartalommal, elmenteni a snapshotot, majd `k = 0`-tól növelve beállítani a `countdown` értékét, és minden `k`-ra meghívni a `replace_all`-t egy legalább `k` másolást igénylő bemenettel. A kivételt elkapva ellenőrizni kell, hogy a `buckets()` egyenlő a mentett snapshotnal, és hogy `invariant()` igaz. Az injektálás akkor fedi le a teljes

műveletet, ha a bemenet hossza nagyobb, mint a tesztelt `k` maximuma; különben a teszt „nem ér el” bizonyos pontokat, és hamis biztonságérzetet ad.

A `replace_all_basic` a `k = 1` (vagy nagyobb) értéknél bukik el: a `clear()` után az `insert` már módosított állapoton dolgozik, így a hiba után a vektor részben feltöltött, a snapshot-egyezés megsérül. A copy-and-swap változat minden `k`-ra átmegy, mert a dobás vagy a paraméter másolásakor (a törzs előtt), vagy a `validate`-ben történik, mindkettő a commit előtt – a commitáló `swap` pedig nem dob. Ellenőrzési feltétel: a teszt minden `k`-ra lefusson, a kivételek elkapása után az állapot és az invariáns is egyezzen, és a `replace_all` dokumentált szerződése tartalmazza a `Bucket` másolhatóságát, valamint az allocator-egyenlőség feltételét.

25.2.9 Összefoglaló

Az exception guarantee az invariánshoz kötött, műveletenként dokumentált hibaág-szerződés: a no-throw a hiba kizárását, az erős garancia a teljes visszagörgetést, az alap garancia a konzisztens, de nem specifikált állapotot ígéri. Az erős garancia receptje a dobható előkészítés és a nem dobó commit szétválasztása (copy-and-swap, illetve `State`-csere), ahol a kritikus pont a `swap` no-throw volta és a típus-, illetve allocatorfüggő konténergaranciák ismerete. A kivételobjektum létrehozása és másolása maga is hibapont, ezért a kivételtípus legyen kicsi és nem dobóan másolható; a destruktor soha ne jelezzen hibát, a cleanupot explicit művelet végezze. A garanciát hibainjektálással és teljes állapot-snapshot összehasonlítással kell igazolni – a fokozatos feladatok pont ezt a három réteget (döntés, tervezés, diagnosztika) gyakoroltatják.

25.3 noexcept határok és hibapolicy

Ez az alfejezet a hibakezelésnek arról a rétegéről szól, amelyet a legtöbb csapat elront: nem arról, hogy *hogyan* dobunk és kapunk el kivételt, hanem hogy **hol** van a modulunk határa, és azon a határon ki milyen ígéretet tesz. A No-throw, strong és basic exception guarantee (25.2) alapján már ismered a no-throw, strong és basic garanciát; itt azt vizsgáljuk, hogyan lesz ezekből ellenőrizhető, dokumentált szerződés a `noexcept`, a destruktor, az explicit lezáró API, a C ABI-trampolin és a buildprofil szintjén. A C oldali stabil interfészhez lásd Opaque handle és stabil C könyvtári interfész (10.4).

A vezérelv egyetlen mondatban: **a hibakezelési policy nem ott dől el, ahol a hiba keletkezik, hanem ott, ahol a hiba átlép egy modul-, szál- vagy nyelvi határt.** A határ mindkét oldalán más a helyes eszköz: belül kivétel, kívül stabil hibakód; belül invariáns-megőrzés, kívül dokumentált állapotáttérés.

25.3.1 A noexcept mint vállalás és a terminate következménye

Definíció. Egy függvény *exception specification*-je (kivétel-specifikációja) mondja meg, hogy kiléphet-e belőle kivétel. A `noexcept` alak `noexcept(true)`-t jelent, a `noexcept(kifejezés)` alakban a kifejezés értéke dönt; a kifejezés nem értékelődik ki futásidőben. A `noexcept` operátor önmagában viszont egy `bool` értékű, compile-time lekérdezés: `noexcept(kif)` igaz, ha a `kif` kiértékelése nem dobhat.

Feltétel és következmény. Ha egy függvényre `noexcept(true)` érvényes, és mégis kivétel lép ki belőle, akkor a szabvány szerint `std::terminate` hívódik. Ez **nem** nem definiált viselkedés, hanem definiált leállás — éppen ezért veszélyes: a fordító nem jelzi, csak a futó program hal meg. Két gyakorlati következmény:

- A hívó fél *nem* tudja elkapni a kivételt, mert a `noexcept` függvény hívási pontja körül nincs értelmes `catch`; a kivétel a hívási pontig sem ér el.
- Azt, hogy a `std::terminate` előtt megtörténik-e a vermet kibontó (stack unwinding) mechanizmus, az implementáció dönti el. Ezért **nem lehet arra építeni, hogy a `noexcept` függvényben lévő objektumok destruktora lefut**. Debuggerben ezért látod néha hiányos backtrace-t.

C++17 óta a `noexcept` a függvény **típusának** része. Következmények: `void (*)(int) noexcept` és `void (*)(int)` különböző típus; a `noexcept` függvényre mutató pointer implicit módon átalakítható a nem-`noexcept` változatra, visszafelé nem. Overload-feloldásban és `std::function`-be való behelyezésnél ez számít.

A tipikus félreértés: a `noexcept` nem optimalizálási tipp, hanem **ígéret**. Csak akkor írd ki, ha a függvény törzsének minden kivétel-forrását ismered és kizártad. A `noexcept` hiánya nem hiba: a függvények alapértelmezett specifikációja `noexcept(false)`, kivéve a destruktorokat és néhány speciális esetet.

25.3.2 Feltételes noexcept sablonokban

Sablonban nem tudod előre, hogy a paramétertípus művelete dobhat-e. A helyes minta: a wrapper **soha ne ígérjen többet**, mint amit a belső művelet ígér:

```
// condnoexcept.cpp
#include <utility>

template <class Fn>
void invoke_once(Fn&& f) noexcept(noexcept(std::forward<Fn>(f)()))
{
    std::forward<Fn>(f)();
}

// A lambdák operator()-je alapból noexcept(false) - ezt írni kell!
static_assert(!noexcept(invoke_once([]{})));
static_assert( noexcept(invoke_once([]() noexcept {})));

int main() { invoke_once([]() noexcept {}); }
```

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 condnoexcept.cpp -o condnoexcept
```

A kettős `noexcept(noexcept(...))` szerkezet nem elírás: a külső a deklaráció, a belső a compile-time lekérdezés. A `static_assert`-ek itt a szerződést rögzítik, nem futásidejű viselkedést.

Fontos csapda: a `noexcept` a *teljes* műveletre vonatkozik, nem csak a típus mozgatására. Egy `std::vector<T>::push_back` például allokálhat, tehát `noexcept(false)`; ha egy wrappert `noexcept`-ra deklarálsz, mert „a T másolása nem dob”, a `bad_alloc` miatt a programod `std::terminate`-nel hal meg. A helyes megoldás: vagy `noexcept(noexcept(...))` a tényleges kifejezésre, vagy `reserve()`-ral előre kizárt allokáció, vagy egyszerűen nem `noexcept` a wrapper.

25.3.3 Destruktor és swap garanciája

Destruktor. Az implicit destruktor specifikációja `noexcept(true)`, hacsak valamely tag vagy bázis destruktora potenciálisan dobó. Mivel a `std::terminate` leggyakoribb oka a destruktorból kilépő kivétel (különösen unwind közben), a szabály: **a destruktor ne dobjon**. Ha a takarítás elbukhat (flush, close, hálózati bontás), a destruktor csak *best effort* lehet, a hibát pedig egy külön, explicit függvény jelenti — lásd a következő szakaszt.

Swap. A `noexcept` swap három helyen számít:

1. Copy-and-swap idiomban a strong garancia akkor is tartható, ha a *másolás* dob, de a *swap* nem.
2. A standard tárolók áthelyezéskor a `std::move_if_noexcept`-t használják: ha a move constructor potenciálisan dobó, és a másolás elérhető, akkor **másolnak**, hogy a strong garancia megmaradjon. Egy `noexcept` nélküli move constructor tehát lassabb újraallokációt eredményez — de ez következmény, nem indok a hamis `noexcept`-ra.
3. A `std::swap` alapértelmezett implementációja move-okra épül; ha a move dob, a swap nem tud garanciát adni.

A sorrend tehát: először *valóban* nem dobó move/swap implementáció, aztán `noexcept` rá.

25.3.4 Explicit close/commit: hol legyen a hiba jelentése

A destruktor nem tud hibát jelenteni. A minta, amit a standard könyvtár is követ (pufferelt írás, `fstream::close`): **a destruktor best effort, a hibajelzés egy külön, a hívó által kötelezően meghívandó művelethez tartozik.**

```
// sink.cpp
#include <cstdio>
#include <cstring>
#include <new>
#include <stdexcept>
#include <string>

class file_sink {
public:
    explicit file_sink(const std::string& path)
        : f_(std::fopen(path.c_str(), "wb")) {
        if (!f_) throw std::runtime_error("open failed: " + path);
    }

    ~file_sink() {
        if (f_) std::fclose(f_);   // best effort; itt nincs hibacsatorna
    }

    file_sink(const file_sink&) = delete;
    file_sink& operator=(const file_sink&) = delete;

    void write(const char* p, std::size_t n) {
        if (std::fwrite(p, 1, n, f_) != n)
            throw std::runtime_error("write failed");
    }
```

```
    // Az egyetlen hely, ahol a flush/close hibája megfigyelhető.
    void commit() {
        if (!f_) throw std::logic_error("commit on closed sink");
        if (std::fflush(f_) != 0) throw std::runtime_error("flush failed");
        int rc = std::fclose(f_);
        f_ = nullptr;              // így a commit kétszer hívva nem dupláz
        if (rc != 0) throw std::runtime_error("close failed");
    }

private:
    std::FILE* f_;
};

extern "C" int sink_append(const char* path, const char* text) noexcept {
    try {
        file_sink s{path};
        s.write(text, std::strlen(text));
        s.commit();
        return 0;
    } catch (const std::bad_alloc&) {
        return -2;
    } catch (...) {
        return -1;                 // minden kivétel itt alakul hibakóddá
    }
}

int main() {
    try {
        file_sink s{"out.bin"};
        s.write("hello\n", 6);
        s.commit();
    } catch (const std::exception& e) {
        std::fprintf(stderr, "error: %s\n", e.what());
        return 1;
    }
}
```

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 sink.cpp -o sink
```

A példa nem lett lefuttatva; a fenti parancs a tanuló laborjában végrehajtható ellenőrzés. A `commit()` utáni `f_ = nullptr` két dolgot ad: a destruktor nem zár kétszer, és a `commit()` ismételt hívása `logic_error`-t dob a néma adatvesztés helyett. A `write()` hibája viszont továbbra is kivétellel terjed — ez belső, C++ oldali hiba, nem határátlépés.

Ha a hívó nem tud kivételt kezelni (pl. `-fno-exceptions` build), a `commit()` visszatérési értéke `std::error_code` legyen; a `std::error_code`/`std::expected` részleteit az error_code, system_error és expected (25.4) tárgyalja.

25.3.5 Callback, thread entry és a C ABI határa

Három helyen kötelező a kivétel lefordítása hibakóddá, mert a kivétel nem mehet át:

- **C ABI-trampolin.** Az `extern "C"` függvényt C kód hívja, amely nem ismeri a C++ kivételmodellt. A trampolin teste `noexcept`, belül `try/catch(...)`; a kifelé menő csatorna a visszatérési érték vagy egy opaque handle (Opaque handle és stabil C könyvtári interfész (10.4)) státuszmezője. A `catch(...)` ne nyelje el a típust: naplózás vagy `std::current_exception()` elmentése a konverzió előtt.
- **Callback opaque handle-en keresztül.** Ha a könyvtárunk függvénypointert tárol a context pointer mellett, a regisztráló C++ kódjában a *hívási ponton* kell a `try/catch`-nek lennie, nem a callback belsejében — a callback szerződése „nem dobok”, ezt a wrappernek kell kikényszerítenie.
- **Szál belépési pont.** A `std::thread` belépési függvényéből kilépő kivétel `std::terminate`-hez vezet. A szál törzsét `try/catch`-be kell tenni, és az eredményt csatornán (queue, `std::promise`, státuszmező) visszaadni.

A határ mindhárom esetben ugyanaz: **kint hibakód, bent kivétel**, és a konverzió egyetlen, rövid, ellenőrizhető helyen történik. Amit itt szándékosan nem részletezünk: mi történik, ha mégis átengedünk egy kivételt a C ABI-n — az nem támogatott, és nem képezhető belőle hordozható viselkedés.

25.3.6 Exceptions nélküli build külön profilként

A `-fno-exceptions` (GCC/Clang) nem „kikapcsolja a hibákat”, hanem **eltávolítja a hibacsatornát**: a `throw`, `try` és `catch` használata fordítási hibát ad, a standard könyvtár egyes részei pedig `std::terminate`-re/`abort`-ra váltanak (a pontos viselkedést a választott libstdc++/libc++ kiadás dokumentációja adja meg — ezt ellenőrizni kell). Ezért ez **nem kapcsoló, hanem külön capability profile**: saját build-cél, saját hibapolicy (mindenhol `error_code`/`expected`), saját CI-sáv.

Ellenőrzéshez használj compile probe-ot, ne feltételezést:

```
# csak a feature-macro jelenlétét vizsgáljuk, nem futtatunk kódot
echo '#if defined(__cpp_exceptions)
#error "exceptions enabled"
#endif
int main(){}' | g++ -std=c++20 -fno-exceptions -x c++ -fsyntax-only -
```

A `__cpp_exceptions` (199711L, ha engedélyezett) jelenléte a szokásos jelzés, de ez implementációfüggő; a profil rögzítse a konkrét compiler- és library-verziót. Olyan kód, amely kivétellel jelzi a hibát, ebben a profilban nem fordítható le — ez nem hiba, hanem a profil következménye, és a kettőt (kivételes és nem kivételes változat) külön fordítási egységben vagy feltételes fordítással kell fenntartani.

25.3.7 Tipikus hibák és javításuk

- **`noexcept` „optimalizálásként”** olyan függvényen, amely allokál vagy könyvtári hívást végez. *Javítás:* `noexcept(noexcept(...))`, vagy hagyd el.
- **Dobó destruktor.** A `~T()` dob, unwind közben `terminate`. *Javítás:* best effort destruktor + explicit `commit()`/`close()`.
- **Dupla close/commit.** A destruktor a `commit()` után is zár. *Javítás:* állapotjelző (`f_ = nullptr`), és a `commit()` is ellenőrizze.
- **`catch(...)` a C trampolinban, naplózás nélkül.** A hiba típusa elveszik. *Javítás:* típus rögzítése a konverzió előtt.

- **Kivétel a szál belépési pontjában.** `std::thread` + dobó lambda → `terminate`, nem kivétel a hívónál. *Javítás:* `try/catch` a szál törzsében.
- **Azt hinni, hogy `-fno-exceptions` mellett nincs abort.** Az allokációs hiba és a library-előfeltétel-sértés továbbra is leállíthat.

25.3.8 Feladatok

1. (magyarázó) Adott `void f() noexcept;`, amelynek törzse egy dobó `g()`-t hív. Mi történik, mit tud elkapni a hívó, és lefutnak-e a `f` belsejében lévő objektumok destruktorai?

2. (kódolás) Írj `invoke_twice` sablont, amely egy callable-t kétszer hív meg, és a `noexcept`-ját a callable műveletéből vezeti le. Készíts `static_assert`-eket egy dobó és egy nem dobó callable-re.

3. (diagnosztika) Egy C könyvtárnak átadott callback `std::stoi`-t hív, amely `std::invalid_argument`-ot dob. Írd le a következményt, és javítsd a kódot úgy, hogy a hiba a C oldalon hibakódként jelenjen meg.

4. (build) Hogyan bizonyítod compile probe-pal, hogy egy adott buildben engedélyezettek-e a kivételek? Mi a teendő, ha egy harmadik féltől származó header `throw`-t tartalmaz?

25.3.9 Megoldások

25.3.9.1 1. megoldás

A `g()` kivétele kilép az `f`-ből, de `f` `noexcept(true)`, ezért `std::terminate` hívódik. A hívó **nem** tudja elkapni: a kivétel nem éri el a hívási pontot. A destruktorok lefutása implementációfüggő, ezért nem tervezhetsz rájuk — ez a gyakorlati különbség a „definiált leállás” és a „rendben lezáruló” között. *Ellenőrzés:* a viselkedés akkor helyes, ha a terv nem tartalmaz `noexcept` függvényt olyan műveletre, amely dobhat; a javítás a `noexcept` elhagyása vagy `noexcept(noexcept(g()))`.

25.3.9.2 2. megoldás

```
#include <utility>

template <class Fn>
void invoke_twice(Fn&& f)
    noexcept(noexcept(std::forward<Fn>(f)()) &&
             std::is_nothrow_copy_constructible_v<std::decay_t<Fn>>)
{
    auto g = f;                  // a második híváshoz külön példány
    g();
    g();
}

static_assert(!noexcept(invoke_twice([]{})));
static_assert( noexcept(invoke_twice([]() noexcept {})));
```

Indoklás: a `noexcept` csak akkor igaz, ha a hívás és a callable másolása is nem dobó. A lambdák `operator()`-je alapból `noexcept(false)`, ezért kell a `noexcept` a lambdára. *Ellenőrzés:* a két `static_assert` forduljon le; ha a `std::decay_t` helyett `Fn`-t írsz, referencia-típusnál hibás eredményt kapsz — ez a gyakori hiba.

25.3.9.3 3. megoldás

A kivétel a callbackből a C hívási láncba próbál kilépni, ami nem támogatott; a folyamat nagy valószínűséggel `std::terminate`-mel leáll. A javítás: a callback C++ oldali burka `noexcept`, belül `try/catch(...)`, és az eredményt a context struct státuszmezőjébe írja:

```
struct ctx { int status; };

extern "C" void on_line(void* p, const char* s) noexcept {
    auto* c = static_cast<ctx*>(p);
    try {
        c->status = std::stoi(s);
    } catch (...) {
        c->status = -1;   // stabil hibakód a C oldal felé
    }
}
```

Ellenőrzés: a C oldal soha nem lát kivételt; a `status` minden ágon íródik; a `catch(...)` előtt legalább a hiba típusa naplózható.

25.3.9.4 4. megoldás

A fenti `-fsyntax-only` próba `#error`-ral jelez, ha `__cpp_exceptions` definiált; a fordító kilépési kódja a válasz. Ha egy header `throw`-t tartalmaz, az `-fno-exceptions` profilban nem fordítható — a lehetőségek: (a) a header mögötti funkcionalitást kiváltó, hibakódos alternatíva, (b) a modul kizárása a profilból, (c) a profil elhagyása. Fordítói kényszerítés nincs rá; ezt build-szinten kell dokumentálni, és a feature-macro állapotát a CI-ban rögzíteni.

25.3.10 Összefoglaló

A `noexcept` ígéret, amelynek megszegése `std::terminate`; ezért csak bizonyítottan nem dobó műveletre írjuk ki, sablonban pedig `noexcept(noexcept(...))` formában vezetjük le. A destruktor soha nem dob: a hibát vagy elnyeli (best effort), vagy egy explicit `commit()`/`close()` jelenti, amelynek visszatérési értéke vagy kivétele a tényleges hibacsatorna. A C ABI, a callback és a szál belépési pont mind kivétel-határ: kifelé hibakód, befelé `try/catch`, egyetlen konverziós ponton. Az exceptions nélküli build nem kapcsoló, hanem külön capability profile saját hibapolicyval és saját CI-sávval.

25.4 error_code, system_error és expected

A hibakezelés nem attól jó, hogy van benne `try/catch`, hanem attól, hogy a hiba azonosítható, a kontextusa megmarad, és a hívó a szerződésnek megfelelően tud dönteni. A C++ban ehhez több eszköz áll rendelkezésre: a kivétel, a `std::error_code`, a `std::system_error`, az `std::optional` és a C++23-as `std::expected`. Ezek nem egymás helyettesítői, hanem különböző hibaátadási modellek. A választás alapja az, hogy a hiba várható-e, kell-e hozzá gép által olvasható hibaazonosság, kell-e emberi kontextus, és a függvény `noexcept` határon van-e. Az előző részek közül különösen a noexcept határok és hibapolicy, valamint a C-s errno és hibaadatok élettartama ismerete szükséges.

25.4.1 Kivétel vagy explicit eredmény?

Kivételt akkor érdemes használni, ha a hiba a helyi kontextusban nem kezelhető, ha egy konstruktor nem tud invariánst felállítani, vagy ha a hibát egy központi határon akarjuk egységesen kezelni. Az explicit eredmény akkor természetes, ha a hiba a művelet szerződésének része: egy fájl hiánya, egy parser hibás bemenete, egy hálózati művelet megszakítása vagy egy stream vége. Az explicit eredményt a hívónak ellenőriznie kell; ez nem „gyengébb” hiba, hanem láthatóbb szerződés.

A döntés feltételei: ha a hibát a hívó rendszeresen kezeli, és a hiba nem tesz érvénytelenné egy nagyobb objektumot, akkor explicit result; ha a hiba egy invariánst sért, vagy a függvény nem tud értelmes állapotban visszatérni, akkor kivétel; ha a függvény `noexcept`, akkor kivétel nem hagyhatja el, tehát eredményt kell visszaadni. A `noexcept` nem azt jelenti, hogy „nincs hiba”, hanem azt, hogy a hiba más csatornán megy.

25.4.2 error_code, error_condition és error_category

A `std::error_code` egy `(érték, error_category*)` pár. A kategória egy hibatartományt azonosít, a value pedig a konkrét hibát a tartományon belül. Az `error_code` olcsón másolható, nem tulajdonol dinamikus memóriát, és a kategóriának a program teljes futása alatt élnie kell; ezért saját kategóriát statikus objektumként kell definiálni. A nulla érték sikert jelent, függetlenül a kategóriától.

A `std::error_condition` szintén `(érték, error_category*)`, de jelentése általános, hordozható feltétel, nem konkrét hiba. Például egy konkrét OS-hibakód összehasonlítható a „nincs jogosultság” feltétellel. Az `error_code` és `error_condition` összehasonlítása a kategóriák `equivalent` logikáján keresztül történik; két `error_code` közvetlen összehasonlítása viszont kategóriaazonosságot is megkövetel. Ezért nem elég csak a nyers értéket összevetni.

A `std::error_category` adja a `name()`, `message()`, `default_error_condition()` és `equivalent()` műveleteket. A `message()` emberi szöveget adhat, de az `error_code` maga nem tárol szövegkontextust. Ha a hívónak tudnia kell, hogy melyik fájl megnyitása hibázott, azt külön kell hordozni, például egy `std::system_error` objektumban vagy egy saját hibaobjektumban.

A tipikus OS-hibaút:

1. A C API visszatér `-1`-gyel, és beállítja az `errno`-t.
2. Az `errno` értékét azonnal el kell menteni, mert bármely következő könyvtári hívás felülírhatja.
3. A mentett értékből `std::error_code` készül: `std::error_code(saved, std::system_category())`.
4. Az összehasonlítás `std::errc` feltétellel történik, például `ec == std::errc::permission_denied`.
5. Ha a hibát kivételként kell továbbadni, `std::system_error(ec, "open config")` készül.

A `system_category()` és a `generic_category()` szerepe eltér: a `system_category()` az operációs rendszer hibáit képviseli, a `generic_category()` a hordozható `std::errc` feltételeket. POSIX rendszeren a `system_category()` értéke jellemzően `errno`, de a pontos leképezés implementációfüggő; a target libc dokumentációja és egy compile/run probe ellenőrzi. A példák itt illusztratívak, nem futtatott mérések.

```
#include <errno.h>
#include <stddef.h>
#include <unistd.h>

enum io_status { IO_OK, IO_EOF, IO_ERROR };
```

```
enum io_status read_exact(int fd, void *buf, size_t len, size_t *done, int *err)
 ↪  {
    unsigned char *p = (unsigned char *)buf;
    size_t n = 0;
    while (n < len) {
        ssize_t r = read(fd, p + n, len - n);
        if (r < 0) {
            if (errno == EINTR) continue;
            *err = errno;
            *done = n;
            return IO_ERROR;
        }
        if (r == 0) { *done = n; return IO_EOF; }
        n += (size_t)r;
    }
    *done = n;
    return IO_OK;
}
```

Fordítási példa: `gcc -std=c17 -Wall -Wextra -Wpedantic -c read_exact.c`. Itt az `EINTR` újrapróbálkozás, az `IO_EOF` külön kimenet, az `IO_ERROR` pedig `errno`-t hordoz. A C API explicit eredményt ad, de a hiba kontextusát a hívónak kell kiegészítenie.

25.4.3 system_error és OS-hiba kontextus

A `std::system_error` olyan kivételtípus, amely egy `std::error_code`-ot és egy emberi kontextust hordoz. A `std::runtime_error` leszármazottja, `code()`-tal visszaadja a hibakódot, `what()`-ja pedig jellemzően tartalmazza a hibaüzenetet és a megadott kontextust; a `what()` pontos formátuma nem specifikált. Akkor használjuk, ha a hibát a függvény határán kivételként akarjuk továbbadni, például egy fájlmegnyitó wrapperben:

```
#include <cerrno>
#include <fcntl.h>
#include <system_error>
#include <unistd.h>

int open_or_throw(const char *path) {
    int fd = ::open(path, O_RDONLY);
    if (fd < 0) {
        int saved = errno;
        throw std::system_error(
            std::error_code(saved, std::system_category()),
            "open config");
    }
    return fd;
}
```

A `std::system_error` nem helyettesíti az `error_code`-ot: a hibaazonosságot a `code()` hordozza, a kontextust a `what()` és a konstruktorban átadott szöveg. A kivételt `const std::system_error&` referenciával érdemes elkapni. Ha a függvény `noexcept`, itt nem dobhatunk; ilyenkor `expected`-tel vagy `error_code`-dal kell visszatérni. A C++ kivétel C ABI-n való átengedése tudatosan kívül esik ezen a részen.

25.4.4 optional: hiány, nem hiba

A `std::optional<T>` vagy tartalmaz egy `T` értéket, vagy üres. Nem hordoz hibaokot, hibakódot és kontextust. Akkor használjuk, ha a hiány normális, várható állapot, és a hívónak nincs szüksége magyarázatra: cache-lookup, opcionális konfigurációs mező, „van-e ilyen elem” kérdés. Egy parser eredményére önmagában nem jó, mert nem tudjuk megkülönböztetni az üres bemenetet, a hibás karaktert és a túlcsordulást.

A `value()` üres `optional` esetén `std::bad_optional_access` kivételt dob, a `value_or()` alapértéket ad. Ha mégis `optional<T>`-t használunk, és mellette `error_code`-ot adunk vissza, két csatornán keletkezik a hiba; ez könnyen inkonzisztens lesz. Ilyenkor inkább `expected<T,E>` vagy saját result típus kell.

25.4.5 expected C++23 és C++20 Result-alternatíva

A `std::expected<T,E>` vagy egy `T` értéket, vagy egy `E` hibát tartalmaz. C++23-as eszköz; `expected<void,E>` is létezik. Az `E` jellemzően értéktípus: `std::error_code`, saját domain enum, vagy strukturált hibaobjektum. Referenciát hibaoldalon ne használjunk, mert az élettartam- és ownership-problémákat okoz. A `has_value()`, `value()`, `error()`, `operator*` és `value_or()` a fő műveletek. A `value()` hibás oldalon `std::bad_expected_access<E>`-t dob, az `operator*` és az `error()` előfeltétele a megfelelő oldal létezése; rossz oldal elérése nem definiált viselkedés.

A C++23-as `expected` monadic műveletei: az `and_then` a sikeres értéket tovább láncolja, a `transform` a sikeres értéket képezi, az `or_else` a hibaágon ad helyreállítási lehetőséget, a `transform_error` a hiba típusát alakítja. Ezek használatát feature-test macro és compile probe ellenőrzi; a választott compiler és standard library támogatása eltérhet, ezért a könyv C++20 alapján a saját `Result` fallbacket is bemutatja. A monadic műveletek hiányában a kézi `if (has_value())` láncolás továbbra is helyes.

```
#include <charconv>
#include <cstdint>
#include <expected>
#include <string_view>

enum class parse_error { empty, invalid, out_of_range };

std::expected<std::uint64_t, parse_error> parse_u64(std::string_view s) {
    if (s.empty()) return std::unexpected(parse_error::empty);
    std::uint64_t v{};
    auto [ptr, ec] = std::from_chars(s.data(), s.data() + s.size(), v);
    if (ec == std::errc::invalid_argument) return
     ↪  std::unexpected(parse_error::invalid);
    if (ec == std::errc::result_out_of_range) return
     ↪  std::unexpected(parse_error::out_of_range);
    if (ptr != s.data() + s.size()) return
     ↪  std::unexpected(parse_error::invalid);
    return v;
}
```

Fordítási példa: `g++ -std=c++23 -Wall -Wextra -Wpedantic -c parse_u64.cpp`. A kód a `std::from_chars` hibáit domain hibává alakítja; így a hívó nem kap nyers `errno`-t, hanem a parser szerződésének megfelelő hibát. A C++20-as fallback lényege egy `std::variant<T,E>`-ra

épülő `Result`, amely `has_value()`, `value()` és `error()` műveleteket ad; a részletes példa a feladatok megoldásában szerepel.

25.4.6 Domain error, cancellation, EOF és input error elkülönítése

A hibák összemosása gyakori tervezési hiba. Négy külön kategóriát érdemes megkülönböztetni:

Kategória	Jelentés	Tipikus reprezentáció	Retry
Domain error	érvénytelen bemenet, szerződésszegés	`enum class parse_error`, strukturált domain hiba	nem; a hívó javítsa
Cancellation	külső kérésre leállított művelet	`kind::cancelled`	csak új kérésre
EOF	normál stream-vége	`kind::eof`, `optional`, külön státusz	nem hiba, nem retry
Input/OS error	átviteli vagy OS hiba	`std::error_code` `system_category()`-vel	esetfüggő; pl. `EINTR` újra

A domain error determinisztikus: ugyanaz a bemenet ugyanazt a hibát adja, és a hívó a bemenet javításával orvosolhatja. A cancellation nem feltétlenül hiba: a művelet szabályosan leállt, az erőforrásokat le kell zárni. Az EOF nem hiba, hanem a stream állapota; nem szabad `system_error`-rá alakítani. Az input error ezzel szemben külső, átmeneti vagy tartós hiba lehet; itt van értelme `error_code`-nak és retry-policy-nak.

```
#include <cerrno>
#include <cstddef>
#include <expected>
#include <system_error>
#include <unistd.h>

struct io_error {
    enum class kind { eof, cancelled, system } k;
    std::error_code ec{};
};

std::expected<std::size_t, io_error> read_some(int fd, void *buf, std::size_t
 ↪  len) {
    for (;;) {
        ssize_t r = ::read(fd, buf, len);
        if (r < 0) {
            if (errno == EINTR) continue;
            return std::unexpected(io_error{io_error::kind::system,
                std::error_code(errno, std::system_category())});
        }
        if (r == 0) return std::unexpected(io_error{io_error::kind::eof, {}});
        return static_cast<std::size_t>(r);
    }
}
```

Fordítás: `g++ -std=c++23 -Wall -Wextra -Wpedantic -c read_some.cpp`. Az `io_error` itt kimenetkód: az `expected` hibacsatornája minden nem értékes kimenetet kezel, de a `kind` megőrzi, hogy EOF-ról, cancellationről vagy OS-hibáról van szó. Ha az EOF tényleg normál, és a hívó nem köteles minden ágat kezelni, lehet `optional` vagy külön `read_status`; a lényeg, hogy ne váljon `system_error`-rá.

25.4.7 Gyakori hibák

- Az `errno`-t későn olvassuk ki, például egy `std::string` vagy `std::error_code` konstrukció után; a köztes hívás felülírhatja.
- OS-hibára `generic_category()`-t használunk `system_category()` helyett, vagy fordítva; a két tartomány nem azonos.
- Két `error_code`-ot csak a value alapján hasonlítunk össze, figyelmen kívül hagyva a kategóriát.
- `optional<T>`-t használunk ott, ahol a hívónak tudnia kellene a hiba okát.
- `expected`-nél `value()`-t hívunk úgy, hogy nem ellenőriztük a `has_value()`-t; az `operator*` rossz oldalon nem definiált.
- Az `expected` hibaoldalán `std::string_view`-t tárolunk egy lokális stringre; a hibaobjektum túlélheti a stringet.
- Az EOF-ot vagy a cancellationt `system_error`-ként kezeljük, és felesleges retry-t indítunk.
- `noexcept` függvényből dobunk; ez `std::terminate`-hez vezet.

25.4.8 Összefoglaló

A kivétel a nem lokálisan kezelhető, invariánst sértő hibák eszköze; az explicit result a szerződés részét képező, várható hibáké. A `std::error_code` konkrét hibaazonosságot és kategóriát hordoz, az `std::error_condition` hordozható összehasonlítást tesz lehetővé, a `std::system_error` pedig kivételként csomagolja ezt kontextussal. Az `std::optional` csak hiányt jelez, nem hibaokot. A C++23-as `std::expected` explicit, típusos result; C++20-on saját `Result` fallback szükséges. A jó hibamodell külön tartja a domain hibát, a cancellationt, az EOF-ot és az OS/input hibát, és mindegyikhez megfelelő kezelési politikát rendel.

25.4.9 Feladatok

25.4.9.1 Első feladat: hibacsatorna választása

Válaszd ki a megfelelő visszatérési módot, és indokold: (a) `lookup_cache(key)`, ahol a hiány normál; (b) `open_socket(host, port)`, ahol az OS-hiba és az `EINTR` lényeges; (c) `parse_port(string)`, ahol az üres, hibás és túlcsordult bemenet külön kezelendő.

25.4.9.2 Második feladat: C++20 Result és portparser

Írj C++20-as `Result<T,E>` típust `std::variant`-ra, majd `parse_port(std::string_view) -> Result<std::uint16_t, parse_error>` függvényt. Kezeld az üres bemenetet, a nem számjegyet, a túlcsordulást és a záró karaktert. Add meg a buildparancsot és a validációs feltételeket.

25.4.9.3 Harmadik feladat: diagnosztika

Az alábbi kódrészletekben keresd meg a hibakezelési hibákat, és javítsd őket:

```
int fd = ::open(path, O_RDONLY);
std::string msg = "open " + std::string(path);
if (fd < 0) throw std::system_error(errno, std::generic_category(), msg);
```

```
std::expected<int, std::string_view> bad() {
    std::string s = "invalid";
    return std::unexpected(s);
}
```

25.4.10 Megoldások

25.4.10.1 Az első feladat megoldása

(a) `std::optional<Value>`: a hiány várható, nincs szükség hibaokra; a hívó `has_value()`-vel dönt. (b) `std::expected<Socket, std::error_code>` vagy a határon `std::system_error`: az OS-hiba azonosítható kell legyen, és a retry-policyhez `EINTR`/`EAGAIN` információk kellenek. (c) `std::expected<std::uint16_t, parse_error>`: a domainhiba típusos, a hívó a bemenetet javíthatja; az `optional` itt elveszítené a hibaokot. Validáció: minden válasznál nevezd meg, hogy a hiba várható-e, kell-e gép által olvasható kód, és a függvény `noexcept`-e.

25.4.10.2 A második feladat megoldása

```
#include <charconv>
#include <cstdint>
#include <string_view>
#include <utility>
#include <variant>

enum class parse_error { empty, invalid, out_of_range };

template<class T, class E>
class Result {
    std::variant<T, E> v_;
public:
    Result(T v) : v_(std::in_place_index<0>, std::move(v)) {}
    Result(E e) : v_(std::in_place_index<1>, std::move(e)) {}
    bool has_value() const noexcept { return v_.index() == 0; }
    explicit operator bool() const noexcept { return has_value(); }
    T& value() & { return std::get<0>(v_); }
    const T& value() const & { return std::get<0>(v_); }
    E& error() & { return std::get<1>(v_); }
    const E& error() const & { return std::get<1>(v_); }
};

Result<std::uint16_t, parse_error> parse_port(std::string_view s) {
    if (s.empty()) return parse_error::empty;
    unsigned v{};
```

```
    auto [ptr, ec] = std::from_chars(s.data(), s.data() + s.size(), v);
    if (ec == std::errc::invalid_argument) return parse_error::invalid;
    if (ec == std::errc::result_out_of_range) return parse_error::out_of_range;
    if (ptr != s.data() + s.size()) return parse_error::invalid;
    if (v > 65535u) return parse_error::out_of_range;
    return static_cast<std::uint16_t>(v);
}
```

Build: `g++ -std=c++20 -Wall -Wextra -Wpedantic -c result.cpp`. Validáció: `parse_port("")` hibája `empty`; `parse_port("80")` értéke 80; `parse_port("65536")` hibája `out_of_range`; `parse_port("8x")` hibája `invalid`; `parse_port("-1")` hibája `invalid`. A `Result` nem nyújt monadic műveleteket; C++20-on a láncolást kézi `if`-fel kell megírni, vagy a projekt saját `and_then`/`transform` segédeket adhat hozzá.

25.4.10.3 A harmadik feladat megoldása

Az első snippetben az `errno`-t a `std::string` konstrukció után olvassuk, miközben az allokáció felülírhatja; a `generic_category()` sem az OS-hiba tartománya. Helyes:

```
int fd = ::open(path, O_RDONLY);
if (fd < 0) {
    int saved = errno;
    throw std::system_error(
        std::error_code(saved, std::system_category()),
        std::string("open ") + path);
}
```

A második snippetben a `std::string_view` egy lokális `std::string`-re mutat, amely a függvény visszatérésekor megszűnik; a hibaobjektum dangling nézetet hordozna. Javítás: az `E` legyen `std::string`, vagy statikus literálra mutató `const char*`, vagy domain enum. Validáció: a hibát a keletkezés helyén olvassuk ki, a kategória az OS-hibához illeszkedik, és a hibaobjektum élettartama nem rövidebb a hívó által elvártnál.

25.5 Hibakezelési és exception safety labor

Ez a labor a fejezet eddigi garanciafogalmait fordítja át mérhető, összehasonlítható kódba. Egyetlen funkcionális szerződést — egy kulcs–érték parser és egy erőforrás-gyártó műveletet — valósítunk meg két hibamódban: kivétellel és explicit eredménnyel. A cél nem az, hogy eldöntsük, melyik „jobb”, hanem hogy megmutassuk: a két változat megfigyelhető viselkedése azonos lehet, és a választott garancia (no-throw, strong, basic) állapoton ellenőrizhető. A labor az error_code, system_error és expected (25.4) hibarecord-fogalmára, a No-throw, strong és basic exception guarantee (25.2) garanciadefinícióira és az Osztály- és operátortervezési labor (23.5) értéktípus-mintáira épül; ezeket itt nem ismételjük meg, csak felidézzük.

25.5.1 A labor szerződése: egy viselkedés, két hibamód

Rögzítsük pontosan, mit jelent az, hogy két implementáció ugyanazt a szerződést teljesíti. Legyen a specifikáció hármas: bemenetek halmaza I, sikeres kimenetek halmaza O, hibajelzések halmaza E. Két implementáció, A (kivételes) és B (explicit eredmény), funkcionálisan ekvivalens, ha minden $i \in I$ bemenetre:

1. A akkor és csak akkor tér vissza $o \in O$-val, amikor B is o-val tér vissza;
2. A akkor és csak akkor dob $e \in E$ hibát, amikor B e-t jelzi;
3. a hívás után megfigyelhető erőforrás-állapot mindkettőnél azonos: a hívó által birtokolt erőforrások száma megegyezik, és a labor saját, belső számlálói nullára állnak vissza.

A harmadik pont az, ami a labort laborkká teszi: nem elég a visszatérési értéket összevetni, a hibás út mellékhatásait is ellenőrizni kell. Ehhez olyan megfigyelhető állapot kell, amelyet a program a hiba után is ki tud írni. A legegyszerűbb ilyen a statikus példányszámláló; egyszálú laborban ez elég, többszálú környezetben atomikus számláló vagy szálankénti gyűjtés kellene.

A labor buildparancsa a rögzített C++20 profillal:

```
g++ -std=c++20 -O2 -g -Wall -Wextra -pedantic parse_lab.cpp -o parse_lab
# vagy
clang++ -std=c++20 -O2 -g -Wall -Wextra -pedantic parse_lab.cpp -o parse_lab
```

A példa nem használ C++23-as library funkciót a fő útvonalon; az `std::expected` változatot külön, jelölt módon adjuk meg. A kód fordítási és futási eredményét itt nem állítjuk igazoltnak: a labor a tanuló saját mérési jegyzőkönyvében rögzíti a tényleges kimenetet.

25.5.2 A parser eredménye és az error offset élettartama

A parser bemenete egy `key=<decimális szám>` alakú szöveg. A hiba két adatból áll: egy hibakódból és egy **error offset**-ből. Az offset szemantikája pontosan az, amit a fél-nyílt tartományoknál megszoktunk: a bemenet első hibás byte-jának indexe, és megengedett az `in.size()` érték is, ami a bemenet végére mutat, de nem dereferálható. Ez a konvenció akkor hasznos, ha a hibát egy kurzorpozícióhoz akarjuk kötni.

A lifetime-kérdés nem az offsetnél, hanem a hibaüzenetnél jelentkezik. Ha a hibarecord `std::string_view`-t tartalmazna, akkor a rekord érvényessége a bemenet — vagy rosszabb esetben egy lokális `std::string` — élettartamához kötődne. A `parse_kv` ezért szándékosan nem hordoz pointert: a hívó birtokolja a bemenetet, a hiba rekord viszont önmagában érvényes marad.

```
// parse_lab.cpp -- C++20, ISO mód
#include <cstddef>
#include <cstdint>
#include <exception>
#include <stdexcept>
#include <string_view>
#include <utility>
#include <vector>

namespace lab {

enum class Errc { none = 0, empty_input, bad_syntax, bad_number, overflow };

struct ParseError {
    Errc code = Errc::none;
    std::size_t offset = 0;
    explicit operator bool() const noexcept { return code != Errc::none; }
};
```

```
struct ParseResult {
    bool ok = false;
    int value = 0;
    ParseError error{};
};

inline ParseResult parse_kv(std::string_view in) noexcept {
    if (in.empty()) return {false, 0, {Errc::empty_input, 0}};
    const std::size_t eq = in.find('=');
    if (eq == std::string_view::npos)
        return {false, 0, {Errc::bad_syntax, in.size()}};
    if (eq == 0)
        return {false, 0, {Errc::bad_syntax, 0}};
    std::int64_t acc = 0;
    bool any = false;
    for (std::size_t i = eq + 1; i < in.size(); ++i) {
        const char c = in[i];
        if (c < '0' || c > '9') return {false, 0, {Errc::bad_number, i}};
        acc = acc * 10 + (c - '0');
        if (acc > INT32_MAX) return {false, 0, {Errc::overflow, i}};
        any = true;
    }
    if (!any) return {false, 0, {Errc::bad_number, eq + 1}};
    return {true, static_cast<int>(acc), {}};
}

struct ParseException : std::runtime_error {
    explicit ParseException(ParseError e)
        : std::runtime_error("parse error"), error(e) {}
    ParseError error;
};

inline int parse_kv_or_throw(std::string_view in) {
    const ParseResult r = parse_kv(in);
    if (!r.ok) throw ParseException{r.error};
    return r.value;
}

} // namespace lab
```

Figyeljük meg: a `parse_kv` `noexcept`, allokációmentes, és a hibarecord két skalár. A kivételes változat ehhez képest csak fordít — nem tartalmaz saját parserlogikát. Ez a labor egyik legfontosabb mintája: **egy igazságforrás, két hibamód**.

A `parse_kv_or_throw` nem `noexcept`, és nem is lehet az: a `throw` maga allokálhat a kivételobjektum számára, tehát a hibás út is hibázhat (`std::bad_alloc`). Ezért a kivétel típusa kicsi, és a kivételes API-t csak ott használjuk, ahol a hívó amúgy is kivételkezelő kontextusban van.

25.5.3 A resource factory két változata

Az erőforrás itt egy fd-szerű, csak mozgatható típus. A számláló az élő példányokat méri; a valódi erőforrás-lezárást külön, nyitott handle-eket számláló változóval lehetne ellenőrizni, ezt a feladatok

között kitérjük.

```
namespace lab {

struct Resource {
    static inline long live = 0;
    int handle = -1;

    Resource() noexcept { ++live; }
    explicit Resource(int h) noexcept : handle(h) { ++live; }
    Resource(Resource&& o) noexcept : handle(o.handle) {
        o.handle = -1;
        ++live;
    }
    Resource& operator=(Resource&& o) noexcept {
        if (this != &o) { handle = o.handle; o.handle = -1; }
        return *this;
    }
    Resource(const Resource&) = delete;
    Resource& operator=(const Resource&) = delete;
    ~Resource() { --live; }
    explicit operator bool() const noexcept { return handle >= 0; }
};

struct MakeOutcome {
    Resource resource;
    Errc code = Errc::none;
    explicit operator bool() const noexcept { return code == Errc::none; }
};

// Kozos mag: nem dob, hibakodot ad.
inline MakeOutcome try_make(std::size_t size) noexcept {
    if (size == 0)    return {Resource{}, Errc::bad_syntax};
    if (size > 4096)  return {Resource{}, Errc::overflow};
    return {Resource{static_cast<int>(size)}, Errc::none};
}

// Kiveteles valtozat: csak fordit.
inline Resource make(std::size_t size) {
    auto out = try_make(size);
    if (!out) throw std::runtime_error("make failed");
    return std::move(out.resource);
}

} // namespace lab
```

A `MakeOutcome` igazságértéke a hibakódra épül, nem a handle-re, így a kettő nem mondhat ellent egymásnak, amíg a mag konzisztens. A hibaágban is létrejön egy `Resource` (érvénytelen handle-lel), ami a `MakeOutcome` megszűnésekor lezárul — ez a minta pazarló, valós kódban `std::optional<Resource>` vagy `std::expected` a helyes választás. C++23-ban, a feature-testtel ellenőrizve:

```
#if defined(__cpp_lib_expected) && __cpp_lib_expected >= 202202L
```

```
#include <expected>
namespace lab {
inline std::expected<Resource, Errc> try_make23(std::size_t size) noexcept {
    auto out = try_make(size);
    if (!out) return std::unexpected(out.code);
    return std::move(out.resource);
}
} // namespace lab
#endif
```

Ez a változat C++23-as library szolgáltatás; C++20 alapkönyvtárral nem elérhető, ezért a példa fő útvonala a saját kimeneti típus. A `std::expected` dostringje nem hordoz értéket hiba esetén, így a hibarecord itt maga az `Errc` — az offsetet külön típusba kellene csomagolni, ha a szerződés része.

25.5.4 Throwing copy/move teszttípus, allokációs hiba és a no-leak feltétel

Ahhoz, hogy a garanciákat megfigyeljük, kell egy típus, amely kérésre dob. A `Probe` másoló konstruktora adott számú másolás után dob; ez modellezi azt, amikor egy konténer közepén fut ki egy allokáció vagy egy elem bemásolása.

```
namespace lab {

struct Probe {
    static inline long live = 0;
    static inline int copies_before_throw = -1; // -1: soha nem dob
    int id = 0;

    Probe() noexcept { ++live; }
    explicit Probe(int v) noexcept : id(v) { ++live; }
    Probe(const Probe& o) : id(o.id) {
        if (copies_before_throw == 0) throw std::runtime_error("probe copy");
        if (copies_before_throw > 0) --copies_before_throw;
        ++live;
    }
    Probe(Probe&& o) noexcept : id(o.id) { ++live; }
    Probe& operator=(const Probe&) = default;
    Probe& operator=(Probe&&) noexcept = default;
    ~Probe() { --live; }
};

// Erős garanciau epitkezes: a lokalis vector vagy teljesen felepul, vagy a
// kivetel elhagyja a fuggvenyt, es a mar megepult elemek lezarulnak.
inline std::size_t build(std::size_t reserve) {
    std::vector<Probe> v;
    v.reserve(reserve);
    for (int i = 0; i < 4; ++i) v.emplace_back(i);
    return v.size();
}

inline bool no_leak() noexcept {
    return Probe::live == 0 && Resource::live == 0;
```

```
}

} // namespace lab
```

Bizonyításvázlat (strong garantia). Állítás: ha egy művelet először az új állapotot egy elkülönített helyen építi fel, és csak egy nem dobó lépésben cseréli felül a régit, akkor hiba esetén a hívó számára megfigyelhető állapot változatlan. Vázlat: a hiba a felépítési fázisban keletkezik, ekkor a régi állapot érintetlen; a felépített rész a verem unwindingja során lezárul; a cserelépés nem dob, tehát nem keletkezhet félig végrehajtott állapot. A feltétel a „nem dobó csere” — ha ez dob, a vázlat nem áll. A `std::vector::push_back` újrafelosztásnál ezt a mintát követi, de csak akkor, ha a mozgatás nem dob (vagy a másolás elérhető); erre a pontos szabványfeltételre itt nem hivatkozunk ellenőrzött forrásként.

A **no-leak feltétel** a teszt végén: `Probe::live == 0 && Resource::live == 0`. Ez szükséges, de nem elégséges: a számláló a példányokra érzékeny, nem a tényleges erőforrásra. Egy fd-leaket csak akkor látunk meg, ha a `Resource` destruktora a nyitott handle-ek számát is csökkenti, és azt is ellenőrizzük.

25.5.5 Az unwinding megfigyelése és az optimalizálás hatása

Az unwinding során a lokális objektumok a konstrukció fordított sorrendjében szűnnek meg. Ezt naplózással lehet láthatóvá tenni. Két korlátot külön kell tartanunk. Egyrészt a naplózás mellékhatás; ha a destruktor csak egy nem olvasott számlálót növel, a fordító az `as-if` szabály alapján elhagyhatja. Másrészt a trace a nyelvi objektumok megszűnését mutatja, nem a gépi szintű unwindelést — a kettő nem azonos.

Ezért a méréshez a számlálót a program végén olvassuk és írjuk ki, vagy a destruktorban `volatile` írást, illetve `std::fprintf`-et használunk. `noexcept` függvényből kifelé menő kivétel `std::terminate`-hez vezet; ezért a destruktornak és a move konstruktornak `noexcept`-nak kell lennie — a `Resource` és a `Probe` ezt teljesíti. Azt, hogy `-O2` alatt melyik változat marad meg, a saját laborban kell megfigyelni; itt nem állítjuk, hogy bármit mértünk volna.

25.5.6 A C API adapter: a határ

A C ABI határán minden kivételt el kell kapni, mert a C hívó nem tud unwindingot kezelni. Az adapter `extern "C"` függvény, amely `try/catch (...)`-ban hívja a C++ magot, és hibakódot ad vissza; a `catch` után nem dob újra.

```
extern "C" int lab_parse_kv(const char* p, std::size_t n, int* out) noexcept {
    if (p == nullptr || out == nullptr) return -1;
    try {
        const auto r = lab::parse_kv(std::string_view{p, n});
        if (!r.ok) return -static_cast<int>(r.error.code);
        *out = r.value;
        return 0;
    } catch (...) {
        return -99;
    }
}
```

A `catch (...)` az utolsó védelem, de nem mindenható: a `std::terminate`-et okozó eseteket (például egy `noexcept` megsértését vagy destruktorból dobást) nem fogja meg. A kivétel C ABI-n való átengedésének kérdését ez a labor nem tárgyalja — éppen azt mutatja meg, hogyan kerüljük el.

25.5.7 Tipikus hibák

1. **A hibaüzenet `string_view`-ként temporálisra mutat.** A hibarecord túléli a függvényt, a nézet dangling lesz. Javítás: kód + offset, vagy owning `std::string`.
2. **A két API külön logikát valósít meg.** Az error-code-os és a kivételes változat eltérő hibakódot ad ugyanarra a bemenetre. Javítás: közös `noexcept` mag, a wrapperek csak fordítanak.
3. **A számlálót nem olvassuk ki.** A fordító elhagyja a destruktort, a teszt hamis zöldet ad. Javítás: a számláló legyen megfigyelhető a program végén.
4. **A `catch (...)` után a C függvény tovább dob.** Az adapter szerződése sérül; a `catch` minden ágában hibakódot kell visszaadni.

25.5.8 Feladatok

1. **Magyarázó feladat.** Adott egy `ParseError`, amely `std::string_view message` tagot tartalmaz. Mutassa meg egy konkrét hívási láncon, hol keletkezik dangling, és adja meg két lehetséges javítást az élettartam-szerződés szempontjából.
2. **Kódolási feladat.** Egészítse ki a `Resource` típust egy nyitott handle-eket számláló statikus változóval, és írjon olyan tesztet, amely a `try_make` hibaágában is `no_leak()`-et vár.
3. **Diagnosztikai feladat.** A `-O2` fordítás mellett az unwinding trace-ből eltűnnek a `Probe` destruktor sorai, `-O0` mellett megjelennek. Adjon magyarázatot, és javítsa a mérést.
4. **Adapter-feladat.** Írjon `extern "C"` wrappert a `make` függvényhez, amely minden kivételt elkap, és a `handle` értékét egy `int*` kimeneten adja vissza.

25.5.9 Megoldások

1. megoldás. A dangling ott keletkezik, ahol a hibarecord túléli a forrást: például egy `parse` függvény lokális `std::string msg`-t tesz a `message` nézetbe, majd visszatér. A visszatérési érték másolása nem menti meg, mert a nézet a megszűnt bufferre mutat. Két javítás: (a) a `message` legyen `std::string`, ami saját tárolót birtokol — ez allokálhat, tehát a `noexcept` parser nem használhatja; (b) a hibarecord maradjon `Errc` + `offset`, és a naplózó réteg generálja a szöveget. Ellenőrzési feltétel: a hibarecord élettartama független a bemenetétől, és a parser továbbra is `noexcept`.

2. megoldás. Vezessünk be `static inline long open_handles = 0;` számlálót. A konstruktor növeli, ha `handle >= 0`; a destruktor csökkenti, ha `handle >= 0`; a move konstruktor nem változtatja a számot, csak átveszi a handle-t és a forrásban `-1`-re állítja. A teszt: `copies_before_throw` beállítása nélkül hívjuk a `try_make(0)`-t, majd ellenőrizzük, hogy `no_leak()` igaz és `open_handles == 0`. Ellenőrzési feltétel: a hibaágban keletkező `Resource` a `MakeOutcome` megszűnésekor lezárul, és a számláló nem marad pozitív.

3. megoldás. A destruktor csak egy nem olvasott statikus számlálót növel, ezért a fordító az `as-if` szabály alapján elhagyhatja; `-O0`-nál a naplózó hívás még megmarad. A javítás: a számlálót a program végén olvassuk ki és írjuk ki, vagy a destruktorban `volatile` írást, illetve `std::fprintf`-et használjunk. Fontos: a trace sorrendje az unwinding során a nyelvi objektumok

megszűnésének sorrendje, nem a gépi unwindelés lépéssorozata. Ellenőrzési feltétel: a `-O2` és `-O0` futás ugyanazt a számlálóértéket adja.

4. megoldás. Az adapter:

```
extern "C" int lab_make(std::size_t size, int* out_handle) noexcept {
    if (out_handle == nullptr) return -1;
    try {
        lab::Resource r = lab::make(size);
        *out_handle = r.handle;
        return 0;
    } catch (...) {
        return -2;
    }
}
```

A `catch (...)` minden C++ kivételt elkap; a `nullptr` ellenőrzés a `try` előtt történik, hogy ne kerüljön kivétel-dobás a pointer-ellenőrzésbe. Ellenőrzési feltétel: hiba esetén `out_handle` érintetlen marad, és a `Resource::live` a hívás után visszaáll a kiinduló értékre. A kivétel C ABI-n való átengedése továbbra is tilos; ezt a labor nem enyhíti.

25.5.10 Összefoglaló

A labor egyetlen funkcionális szerződést valósít meg két hibamódban, és a megfigyelhető állapoton ellenőrzi az ekvivalenciát. A parser hibarecordja kód + offset, nem nézet, így élettartama független a bemenettől. A resource factory közös `noexcept` magra épül, a kivételes és az explicit változat csak fordít. A `Probe` hibainjektálással teszi láthatóvá a strong garanciát és a no-leak feltételt. Az unwinding trace csak megfigyelhető mellékhatással mérhető, és a `noexcept` határán a kivétel `std::terminate`-hez vezet. A C ABI adapter minden kivételt elkap, és hibakódot ad vissza; a kivétel átengedése nem megoldás, hanem a szerződés megsértése.

26 Template-ek és generikus programozás alapjai

26.1 Függvénysablon, instantiation és deduction

Ez a rész a C-beli típusgenerikusság korlátaiból indul ki, és eljut a C++ függvénysablonig: hogyan lesz egy deklarációból konkrét függvény, hogyan következtet a fordító a sablonparaméterre, és hogyan olvassuk ki a hibát a diagnosztikai láncból. A template mint szöveges helyettesítés tévhitével itt csak egy bekezdésben foglalkozunk, a teljes overload-resolution algoritmus pedig későbbi részre tartozik; a cél a pontos, de kezelhető modell.

26.1.1 Kiindulópont: a C `_Generic` és a makró határai

C-ben a „típusfüggő függvény” hagyományos megoldása a makró. A C11 óta a `_Generic` segítségével a makró a *controlling expression* statikus típusa alapján választhat kifejezést, tipikusan egy típusonként külön megírt függvényt:

```
/* max_generic.c -- ISO C17 */
#include <stdio.h>

static int    max_int(int a, int b)       { return a < b ? b : a; }
static double max_double(double a, double b) { return a < b ? b : a; }

#define MAX(a, b) _Generic((a), \
    int:    max_int,            \
    double: max_double          \
)(a, b)

int main(void) {
    printf("%d\n",   MAX(2, 3));
    printf("%.2f\n", MAX(2.0, 3.0));
    printf("%d\n",   MAX(1, 2.5));   /* csendben int-re konvertál! */
    return 0;
}
```

```
gcc -std=c17 -Wall -Wextra -Wpedantic -O2 max_generic.c -o max_generic
clang -std=c17 -Wall -Wextra -Wpedantic -O2 max_generic.c -o max_generic
```

A minta tanulsága nem az, hogy a `_Generic` rossz, hanem hogy **nem paraméterezhető**: minden támogatott típushoz kézzel kell felvenni egy ágat és egy függvényt. Ha egy új típust (például `long long`) használni akarunk, a kód nem fordul le, és a hibaüzenet a makró kifejtésére hivatkozik, nem a szándékunkra. Ráadásul a típusválasztás a *controlling expression* alapján történik, nem az összes argumentum közös típusa alapján: a `MAX(1, 2.5)` esetben az `int` ág fut le, a `2.5` pedig csendben `int`-té konvertálódik. Ez pontosan az a hibaosztály, amit egy típusbiztos generikus eszköznek ki kell zárnia.

A C++ válasza erre a **függvénysablon** (function template): olyan függvény-leírás, amelyben egy vagy több típus (vagy nem típus) paraméter szerepel, és amelyből a fordító a használat helyén konkrét függvényt állít elő.

Fontos határvonal: a sablon **nem** szöveges makró. A helyettesítés nem karakterlánc-szintű, a definíciót a fordító a deklaráció helyén ellenőrzi (a nem függő részeket), és a létrejövő függvény valódi, típusellenőrzött entitás. A nem függő és függő nevek keresésének kettős szabályát (two-phase lookup) és az ADL-t a Dependent name, kétfázisú lookup és ADL rész tárgyalja részletesen.

26.1.2 Deklaráció, definíció és instantiation

A sablon deklarációja `template` fejléccel kezdődik, amely felsorolja a sablonparamétereket. A klasszikus példa:

```
// max_tpl.cpp -- ISO C++20
#include <iostream>

template <class T>
T max_value(T a, T b) {
    return a < b ? b : a;
}

int main() {
    std::cout << max_value(2, 3)     << '\n';  // T = int
    std::cout << max_value(2.0, 3.0) << '\n';  // T = double
}
```

```
g++     -std=c++20 -Wall -Wextra -Wpedantic -O2 max_tpl.cpp -o max_tpl
clang++ -std=c++20 -Wall -Wextra -Wpedantic -O2 max_tpl.cpp -o max_tpl
```

Definíció. Legyen adott egy `template<...>` fejléc és egy függvénydeklaráció, amelyben a sablonparaméterek szerepelnek. A sablon önmagában nem hoz létre futtatható függvényt; a belőle konkrét típusokkal előállított függvényt nevezzük **specializációnak** (specialization), az előállítás folyamatát pedig **instantiation**-nak. A `max_value(2, 3)` hívás hatására a fordító implicit módon létrehozza a `max_value<int>` specializációt; a `max_value(2.0, 3.0)` hívás a `max_value<double>`-t.

Két fogalmat érdemes szétválasztani:

- **Implicit instantiation**: a fordító a használat (hívás, címképzés) láttán állítja elő a specializációt. Ez a normál eset.
- **Explicit instantiation**: mi kérjük a példányosítást: `template int max_value<int>(int, int);`. Ennek akkor van értelme, ha a definíciót egy `.cpp`-be tesszük, és a fordítást több TU-ra akarjuk szétosztani. Az `extern template int max_value<int>(int, int);` pedig azt mondja: „ebben a TU-ban ne példányosítsd, máshol már megvan” — ez a linkelési idő és a kódméret csökkentésének eszköze.

Fontos következmény. Mivel a specializációt a *használat helyén* kell előállítani, a sablon **definíciójának** minden olyan fordítási egységben láthatónak kell lennie, ahol példányosítás történik. Ezért kerülnek a sablonok (a nem sablon függvényekhez képest szokatlan módon) általában fejlécfájlba. A deklaráció önmagában nem elég: `template <class T> T max_value(T, T);` hívása linkelési hibát ad, ha a definíció nem elérhető a TU-ban.

26.1.3 Típusparaméter és explicit template argumentum

A `template <class T>` és a `template <typename T>` között nincs szemantikai különbség; a `typename` utóbb lett bevezetve, és ott kötelező, ahol a név függő típusra utal. A típusparaméter neve a sablon testén belül típusként viselkedik.

Az explicit template argumentum a függvénynév után, szögletes zárójelben adható meg: `max_value<double>(2, 3)`. Ilyenkor a megadott argumentum **felülírja** a következtetést, és a hívást a szokásos függvényhívási konverziókra bízza: a `2` és `3` itt `double`-lé konvertálódik. Ez a `_Generic` példa `MAX(1, 2.5)` esetével rokon, de itt *mi* döntöttünk tudatosan, nem a makró csendben.

Ha a sablonnak több paramétere van, az explicit argumentumok balról jobbra töltődnek ki, a maradék továbbra is következtethető:

```
template <class A, class B>
void pair_print(A a, B b);   // pair_print<int>(1, 2.5) -> A = int, B = double
```

Egy explicit template argumentum után nem lehet „kihagyni” egy paramétert és később újra következtetni — a megadott lista mindig az első paraméterektől indul.

26.1.4 Deduction: a paraméter és az argumentum alapján

A **template argument deduction** (sablonargumentum-következtetés) az a folyamat, amelyben a fordító a hívás argumentumainak típusából (A) és a függvény deklarált paramétertípusából (P) visszafejti a sablonparaméter értékét. A következtetés paraméterenként, párban történik; ha ugyanarra a `T`-re két argumentum két különböző eredményt ad, a hívás hibás.

A legfontosabb szabályok:

1. **By-value paraméter (`T`)**: az argumentum típusából a felső szintű `const`/`volatile` minősítő eltűnik, a tömb- és függvénytípus pedig pointerré alakul (array-to-pointer, function-to-pointer decay). `const int` argumentumból tehát `T = int` lesz.
2. **Lvalue referencia (`T&`)**: a `T` az argumentum teljes típusát veszi fel, a minősítőkkel együtt; tömb esetén a tömb típusa marad. Nem `const T&` nem köthető rvalue-hoz, ezért `f(1)` hibás.
3. **`const T&`**: a `const` a paraméterben van, nem a `T`-ben, ezért a `const int` argumentumból is `T = int` lesz; rvalue-t is köt (átmeneti objektum létrehozásával), és a tömb típusát is megőrzi.

Az alábbi táblázat `template <class T> void f(P);` esetére mutatja az eredményt:

Paraméter (P)	Argumentum (A)	Dedukált `T`	Tényleges paramétertípus
`T`	`int`	`int`	`int`
`T`	`const int` lvalue	`int`	`int`
`T`	`int[3]`	`int*`	`int*`
`T&`	`int` lvalue	`int`	`int&`
`T&`	`const int` lvalue	`const int`	`const int&`
`const T&`	`const int` lvalue	`int`	`const int&`
`const T&`	`int[3]`	`int[3]`	`const int(&)[3]`

A táblázat utolsó sora a legtanulságosabb: referencián keresztül a tömb **nem** decay-el pointerré, ezért a `const T&` overload képes a tömb hosszát is megőrizni — ez a későbbi `std::span` és

range-tervezés egyik alapmotívuma. A `T&&` alak (forwarding reference) szándékosan kimarad, azt a Forwarding reference és perfect forwarding rész tárgyalja.

A következtetés nem mindig lehetséges: ha a paraméter típusában a sablonparaméter nem szerepel „lehúzható” helyen (például csak egy `std::vector<T>::value_type` alakban), az *non-deduced context*, és ott explicit argumentum kell. A következtetés alapja mindig a **hívás argumentumtípusa**, nem a függvény belsejében használt típus.

26.1.5 Template overload és nontemplate candidate

Ha ugyanarra a névre létezik sablon és közönséges (nem sablon) függvény is, mindkettő jelölt lesz, és a szokásos túlterhelés-feloldás dönt — azzal a kiegészítéssel, hogy **egyenlően jó illeszkedés esetén a nem sablon változat nyer**. Ez teszi lehetővé a „kényelmi” overloadok biztosítását:

```
#include <cstdio>

template <class T>
T pick(T a, T b) { return a < b ? b : a; }

int pick(int a, int b) { return a < b ? b : a; }  // specializált gyors út

int main() {
    std::printf("%d\n", pick(1, 2));      // nem sablon: pontos illeszkedés
     ↪ mindkettőnél -> nem sablon nyer
    std::printf("%.2f\n", pick(1.0, 2.0)); // sablon: T = double
    std::printf("%d\n", pick('a', 'b'));  // sablon: T = char (a nem sablon
     ↪ promóciót igényelne)
}
```

A harmadik hívás mutatja a lényeget: a `pick(int, int)` csak `char` → `int` promócióval hívható, ami gyengébb illeszkedés, mint a sablon pontos `T = char` következtetése, ezért a sablon nyer. Ez a viselkedés sok meglepetés forrása; ha a nem sablon változatot mindig elő akarjuk húzni, azt nem a feloldásra kell bízni, hanem külön névre vagy `if constexpr`-re.

26.1.6 Diagnosztikai lánc és minimális reprodukció

A sablonhiba üzenete hosszú, de szerkezete ismétlődő: a fordító megmutatja, **melyik sablon melyik példányosításában** történt a hiba, **mely argumentumtípusokból** próbált következtetni, és hol vallott kudarcot. A hatékony olvasás iránya: először a legbelső „note: candidate template ignored” vagy „deduced conflicting types” sor, utána a hívási lánc.

```
// fail_deduce.cpp -- szándékosan nem fordul le
template <class T>
T max_value(T a, T b) { return a < b ? b : a; }

int main() {
    auto x = max_value(1, 2.5);   // hiba: T = int és T = double egyszerre
}
```

A tipikus üzenet lényege: `deduced conflicting types for parameter 'T' ('int' vs. 'double')`. A javítás háromféle lehet, és a választás tervezési döntés:

1. Két külön paraméter: `template <class A, class B> auto max_value(A a, B b)` — de ekkor a visszatérési típus és az összehasonlítás típusa kérdéses.
2. Explicit argumentum: `max_value<double>(1, 2.5)` — tudatos konverzió.
3. A hívó oldal típusának egységesítése: `max_value(1.0, 2.5)`.

A **minimális reprodukció** módszere: a hibás kódból tartsuk meg a sablon deklarációját, egyetlen hívást és a szükséges típusokat; a `#include`-okat és a többi függvényt dobjuk el. Ha a hiba így is megvan, a hiba a következtetésben van; ha eltűnik, a hiba a környező overload-halmazban vagy a fejlécsorrendben keresendő. A reprodukciót mindig a rögzített compiler-verzióval és `-std=c++20` kapcsolóval futtassuk, mert a diagnosztika szövege compiler- és verziófüggő.

26.1.7 Tipikus hibák

- **`T&` és rvalue.** `template <class T> void f(T&);` mellett az `f(42)` nem fordul le, mert nem `const` lvalue referencia nem köthető rvalue-hoz. A `const T&` vagy `T&&` a helyes választás, a szemantika tudatos megválasztásával.
- **A `const` elvész by-value paraméternél.** `void f(T)` esetén a `T` sosem lesz `const`; ha a `const`-ra szükség van, azt a paraméterbe kell írni.
- **Definíció nélküli sablon.** A fejlécben csak deklarált, de nem definiált sablon linkelési hibát ad, nem fordítási hibát — a hiba helye félrevezető.
- **Tömb decay by-value paraméternél.** `void f(T)` tömbbel `T = int*`, elveszik a hossz; ha a hossz számít, `const T&` vagy `std::span`.
- **Sablon és nem sablon keverése.** A nem sablon változat nem mindig „erősebb”; a pontos illeszkedés dönt, és a `char` példa ezt jól mutatja.

26.1.8 Feladatok

1. feladat (elemzés). Adott `template <class T> void show(T v);`, `template <class T> void show_ref(T& v);` és `template <class T> void show_cref(const T& v);`. Mi lesz a dedukált `T` és a paraméter típusa a következő hívásoknál: `show(3)`, `show(ci)` ahol `const int ci = 3;`, `show_ref(a)` ahol `int a[4];`, `show_cref(5)`?

2. feladat (kódolás). Írj `clamp_value` függvénysablont, amely egy értéket egy `[lo, hi]` zárt intervallumba szorít, és működik `int`, `double` és `const char*` esetén is. A `const char*` esethez indokold, miért problematikus a `<` operátor, és mit teszel helyette.

3. feladat (diagnosztika). Készíts minimális reprodukciót arra a hibára, amikor egy sablon két argumentuma miatt `T` következtetése ellentmondásos, majd add meg két különböző javítást, és indokold, melyik melyik szemantikát választja.

26.1.9 Megoldások

1. megoldás. `show(3)`: `T = int`, paraméter `int`. `show(ci)`: `T = int` (a felső szintű `const` eltűnik), paraméter `int`. `show_ref(a)`: `T = int[4]`, paraméter `int(&)[4]` — a referencia megőrzi a tömb típusát. `show_cref(5)`: `T = int`, paraméter `const int&`, és a `const` a paraméterből jön, nem a `T`-ből. Ellenőrzési feltétel: írj minden hívás elé `static_assert(std::is_same_v<decltype(...), ...>)`-ot a megfelelő típusra, és forduljon le `-std=c++20`-szal.

2. megoldás. `template <class T> T clamp_value(const T& v, const T& lo, const T& hi) { return v < lo ? lo : (hi < v ? hi : v); }`. A `const T&` választás azért jó, mert nem másol, rvalue-t is köt, és tömböt is megőriz. `const char*` esetén a `<` a pointereket hasonlítja

össze, ami nem lexikografikus és nem hordozható értelmes eredményt — ott külön overload kell `std::strcmp`-vel vagy `std::string_view`-vel. Ellenőrzési feltétel: `clamp_value(5, 1, 3) == 3`, `clamp_value(2.5, 0.0, 1.0) == 1.0`, és a `-Wall -Wextra` ne adjon figyelmeztetést.

3. megoldás. Reprodukció: a fenti `max_value(1, 2.5)` példa. Első javítás: `max_value<double>(1, 2.5)` — itt a hívó dönt, a `2.5` és az `1` is `double`-lé konvertálódik, a visszatérés `double`. Második javítás: `template <class A, class B> auto max_value(A a, B b)` — itt a két argumentum típusa független, a visszatérési típus `auto`-val a feltételes kifejezésből következik, ami viszont a `common_type` szabályai szerint konvertál. Az első megoldás előre deklarált, egységes típust kényszerít; a második megengedi a heterogén bemenetet, de a visszatérési típusról külön kell dönteni. Ellenőrzési feltétel: mindkét változat forduljon `-std=c++20 -Wall -Wextra -Wpedantic` alatt, és a viselkedés a kétféle hívásnál dokumentált legyen.

26.1.10 Összefoglaló

A C `_Generic` + makró megoldása típusonként kézzel bővítendő, és a típusválasztás nem az argumentumok közös típusán alapul. A C++ függvénysablon ezt paraméterezhetővé teszi: a `template <class T>` fejléc és a benne szereplő `T` együtt egy függvénycsaládot ír le, amelyből a fordító a használat helyén **instantiation**nal állítja elő a konkrét specializációt. A **deduction** a paraméter (P) és az argumentum (A) párosából fejti vissza `T`-t: by-value esetén a felső szintű minősítők eltűnnek és a tömb pointerré alakul, `T&` esetén a teljes típus megőrződik, `const T&` esetén a `const` a paraméterből jön. Sablon és nem sablon overload közül egyenlő illeszkedésnél a nem sablon nyer. A hibát a diagnosztikai lánc legbelső sorától olvassuk visszafelé, és minimális reprodukcióval izoláljuk. A sablon definíciójának minden példányosító TU-ban láthatónak kell lennie — ezért él tipikusan fejlécben. A forwarding reference, a specializáció és a kétfázisú lookup a következő részek témája. A példák fordítási parancsai a saját laborodhoz tartoznak; ebben a szövegben nem történt futtatás vagy mérés.

26.2 Osztálysablon, nem típusparaméter és CTAD

A függvénysablonok (Függvénysablon, instantiation és deduction (26.1)) megmutatták, hogyan lehet egy algoritmust a típusától függetlenül megfogalmazni. Az osztálysablon ugyanezt a gondolatot viszi át az adattípusokra: nem egy konkrét `Buffer` osztályt írunk meg `int`-re, `double`-re és `Message`-re, hanem egy *típustermelő sablont*, amelyből a fordító minden szükséges `(T, N)` párra külön osztályt állít elő. Ez a rész egy fix kapacitású `Buffer<T, N>` példán mutatja meg, hogyan függ össze a típusparaméter, az értékparaméter, a levezetés (deduction) és a headerben tartott definíció. A korábbi értéktípus-szemlélet (Értéktípus, kompozíció és immutable interfész (23.2)) itt közvetlenül hasznos: a buffer legyen önmagában konzisztens, másolható érték, ne rejtett mutató.

26.2.1 Osztálysablon és tagdefiníciók

Egy **class template** (osztálysablon) deklaráció egy típuscsaládot ír le; maga a sablon nem típus, hanem egy *specializáció* előállításának szabálya. Amikor a fordító a `Buffer<int, 16>` nevet látja, a sablonargumentumok helyettesítésével létrehoz egy valódi osztályt. Ezt nevezzük **implicit példányosításnak** (implicit instantiation). A specializáció élettartama, mérete és tagjai pontosan úgy viselkednek, mint egy kézzel írt osztályé.

```
#include <cstddef>
#include <stdexcept>
#include <initializer_list>
```

```cpp
template<class T, std::size_t N>
class Buffer {
public:
    Buffer() = default;

    Buffer(std::initializer_list<T> init) {
        for (const T& v : init) push(v);
    }

    void push(const T& v) {
        if (size_ == N) throw std::length_error("Buffer tele");
        data_[size_++] = v;
    }

    T&       operator[](std::size_t i)       { return data_[i]; }
    const T& operator[](std::size_t i) const { return data_[i]; }

    std::size_t size() const noexcept { return size_; }
    static constexpr std::size_t capacity() noexcept { return N; }

private:
    T data_[N]{};
    std::size_t size_ = 0;
};
```

A tagfüggvényeket itt az osztályon belül definiáltuk, ezért implicit `inline`-ok. Ha kívül definiálnánk, a sablonfejlécet meg kell ismételni, és a minősített nevet kell használni:

```cpp
template<class T, std::size_t N>
void Buffer<T, N>::push(const T& v) { /* ... */ }
```

Ez a forma mutatja meg, hogy a `Buffer<T, N>` nem önálló típusnév, hanem a sablon azonosítója a paraméterlistával. A `typename` és a `template` kulcsszavak szükségességére a függő neveknél (Dependent name, kétfázisú lookup és ADL (26.5)) még visszatérünk; itt elég annyit rögzíteni, hogy a tagdefiníció a sablon *deklarációja* alatt él, nem külön entitás.

Két feltétel fontos a példa helyességéhez. Egyrészt `N > 0`, különben a `T data_[N]` nulla hosszú tömb, ami nem szabványos C++. Másrészt az `operator[]` nem ellenőriz boundsot: a `Buffer` itt teljesítményorientált, a szerződés a hívónál van. A `size_ == N` ellenőrzés a `push`-ban viszont kötelező, különben írás történne a tömb után.

26.2.2 Nem típusparaméter és fix kapacitás

A `std::size_t N` **non-type template parameter** (nem típusú sablonparaméter, NTTP): nem típus, hanem fordítási idejű érték, amely a specializáció *típusazonosságának* része. A `Buffer<int, 8>` és a `Buffer<int, 16>` két különböző típus, még akkor is, ha azonos tagfüggvényeik vannak. Ez teszi lehetővé, hogy a `N` a `data_` tömb méreteként és a `capacity()` visszatérési értékeként is szerepeljen, azaz ne kelljen futásidejű allokáció.

A C++20 alapban az NTTP-k megengedett kategóriái bővültek. Integrál és enumerációs típusok, pointerek, referenciák, `nullptr`, valamint bizonyos osztálytípusok használhatók. A **structural type** (strukturális típus) az a class type, amelynek minden bázisa és nem statikus adattagja

`public` és nem `mutable`, és maga is literál típus; a tagoknak is ilyen típusúnak (vagy ilyenek tömbjének) kell lenniük. Ilyen típus például egy egyszerű címke:

```
struct Extent { std::size_t value; };   // aggregátum, minden tag public, nem
 ↪ mutable

template<Extent E>
struct StaticExtent {
    static constexpr std::size_t value = E.value;
};

static_assert(StaticExtent<Extent{16}>::value == 16);
```

A C++20 előtti szabványokban ez a fajta osztály-NTTP nem volt elérhető; a példa tehát `-std=c++20` mellett értelmezhető, és a támogatást a konkrét compiler verziójával ellenőrizni kell. A `StaticExtent<Extent{16}>` és a `StaticExtent<Extent{32}>` itt is különböző típus.

Az **`auto` NTTP** (`template<auto N>`) azt jelenti, hogy a paraméter típusát a levezetés adja meg a megadott argumentumból. Ez kényelmes, de van egy fontos következménye: a `template<auto N>` és a `template<std::size_t N>` nem ekvivalensek. Az előbbinél `Constant<42>` esetén `N` típusa `int`, az utóbbinál `std::size_t`. A `Constant<-1>` az első esetben érvényes lehet, a másodikban a konverzió miatt nagy `size_t` értékké alakulna, ami meglepő viselkedés. Ha a típus számít, ne bízzuk `auto`-ra.

```
template<auto N>
struct Constant { static constexpr auto value = N; };

static_assert(Constant<42>::value == 42);
```

26.2.3 Alapértelmezett sablonargumentum

A sablonparamétereknek lehet alapértelmezett értékük, ami a függvényparaméterekhez hasonlóan csak a jobb oldalon következhet. Ez csökkenti a kiírást, de nem változtat azon, hogy a specializáció típusa a tényleges argumentumoktól függ:

```
template<class T, std::size_t N = 8>
class Buffer;   // N alapértelmezett

Buffer<int> b1;        // Buffer<int, 8>
Buffer<int, 16> b2;    // Buffer<int, 16>
```

Fontos részlet, hogy az alapértelmezett argumentum nem „olvasztja össze” a specializációkat: `Buffer<int>` és `Buffer<int, 8>` ugyanaz a típus, de `Buffer<int>` és `Buffer<int, 16>` nem. Ha egy függvényt `Buffer<int, 8>` paraméterrel deklarálunk, a `Buffer<int>` átadható, a `Buffer<int, 16>` nem. Ez a fix kapacitás lényege: a méret a típus része, tehát a fordítás során ismert és ellenőrizhető.

Egy további következmény, hogy az alapértelmezett argumentumot a *deklaráció* tartalmazza, és a template fejlécében csak egyszer adható meg. Headerben ez természetes, hiszen a felhasználó minden TU-ban ugyanazt a deklarációt látja.

26.2.4 CTAD és deduction guide

A C++17 bevezette a **class template argument deduction** (CTAD) mechanizmusát: osztálysablon példányosításánál a template argumentumok bizonyos esetekben a konstruktor argumentumaiból levezethetők, így nem kell kiírni őket.

```
Buffer b{1, 2, 3};          // Buffer<int, ?> -- N nem vezethető le
Buffer<int, 3> c{1, 2, 3};  // explicit
```

Az első sor nem működik, mert az `std::initializer_list<T>` típusából `T` levezethető (`int`), de a lista *hossza* nem része a típusnak, így `N`-re nincs jelölt. Az implicit deduction guide-ok ugyanis a konstruktorok szignatúrájából képződnek, és az `initializer_list` hossza nem szerepel a szignatúrában. Itt kell **explicit deduction guide**-ot írni:

```
template<class T, class... Rest>
Buffer(T, Rest...) -> Buffer<T, 1 + sizeof...(Rest)>;
```

Ez a guide azt mondja: ha a `Buffer`-t egy `T` típusú értékkel és tetszőleges számú további argumentummal inicializáljuk, a levezetett típus `Buffer<T, 1 + sizeof...(Rest)>`. Ezzel a `Buffer b{1, 2, 3}` alakból `Buffer<int, 3>` lesz. Fontos, hogy a guide csak a *típus levezetésében* vesz részt; a tényleges inicializálást továbbra is a konstruktor végzi, és ha az `initializer_list` konstruktort választja a túlterhelés, akkor a viselkedés ahhoz igazodik. A guide és a konstruktor közötti összhangot nekünk kell biztosítanunk.

Egy másik gyakori eset a beépített tömb átvétele:

```
template<class T, std::size_t N>
Buffer(const T (&)[N]) -> Buffer<T, N>;
```

Ez a `const char[]` literáloknál és fix tömböknél hasznos, mert a tömb hossza itt valóban a típus része.

A CTAD-nak vannak korlátai, amelyeket érdemes fejben tartani. Először is a CTAD meghatározott nyelvtani helyeken lép be: változódeklarációban (`std::vector v{1, 2, 3};`), funkcionális explicit konverzióban (`std::pair{1, 2.0}`), `new`-kifejezésben (`new std::pair(1, 'c')`) és nemtípus sablonparaméter-deklarációban. Közönséges függvényparaméter típusaként viszont nem használható (`void f(std::pair p);` hibás); ott explicit típust kell írni, vagy függvénysablont. Másodszor, a névtérrel való minősítés **nem** akadálya: a `std::pair{1, 2}` és a `std::vector{1, 2}` a szokásos eset. A CTAD akkor nem lép be, ha a sablonargumentumok (akár részben) ki vannak írva.

Buildjegyzőkönyv (`lab-gcc` profil). A `std::pair{1, 2.0}`, a `std::vector v{1, 2, 3}` és a `new std::pair(1, 'c')` típusa `static_assert`-tel ellenőrizve `pair<int, double>`, `vector<int>`, `pair<int, char>*`; a `void f(std::pair p);` `template placeholder not permitted in parameter` hibát ad.

Harmadszor, az alias template-en keresztüli CTAD csak C++20 óta létezik (P1814). A fordító az eredeti sablon guide-jaiból (az implicitekből és a felhasználóiakból is) származtat guide-okat az aliasra: a guide visszatérési típusát az alias jobb oldalára illeszti, és csak azokat tartja meg, amelyekből az alias paraméterei levezethetők. Korlátok: az alias jobb oldalának az eredeti sablon egy specializációjának kell lennie (`template<class T> using P = T*;` nem működik), és ha egy alias-paraméter a guide-ból nem vezethető le, az a guide kiesik. C++17-ben az alias-CTAD nem létezett.

```
template<class T, std::size_t N> using FixedBuf = Buffer<T, N>;
template<class T> using Vec = std::vector<T>;

FixedBuf fb{1, 2, 3};   // C++20: Buffer<int, 3>, a Buffer(T, Rest...) guide-ból
Vec v{1, 2, 3};         // C++20: std::vector<int>
```

Buildjegyzőkönyv (`lab-gcc` profil). A fejezet `Buffer` osztályával és a fenti `Buffer(T, Rest...)` guide-dal a `static_assert(std::is_same_v<decltype(fb), Buffer<int, 3>>)` és a `static_assert(std::is_same_v<decltype(v), std::vector<int>>)` `g++ -std=c++20` és Clang 22 `-std=c++20` alatt is teljesült, a program `fb.size() == 3`-mal futott. `-std=c++17` alatt a GCC `alias template deduction only available with '-std=c++20'` hibát ad.

Egy csapda az alapértelmezett sablonargumentummal: ha a `Buffer` deklarációja `N = 8` alapértéket ad, akkor a `Buffer b{1, 2, 3}` listás inicializálásnál az `initializer_list` konstruktorból képzett guide az elsőbbséget élvező első fázisban levezethetővé válik (`N` az alapértékből jön), és az eredmény `Buffer<int, 8>` lesz, nem `Buffer<int, 3>` — a saját guide nem is jut szóhoz. A `lab-gcc` profilban ezt egy `static_assert` hibája mutatta meg (`Buffer<int, 8>` vs. `Buffer<int, 3>`). A guide és a konstruktor összhangját ezért a teljes definícióval kell ellenőrizni.

26.2.5 Alias template és variable template

Az **alias template** nem új típus, hanem név a meglévő specializációra. Segítségével a hosszú sablonneveket rövidíthetjük, és a szándékot dokumentálhatjuk:

```
template<class T> using SmallBuffer = Buffer<T, 8>;
SmallBuffer<int> sb;   // Buffer<int, 8>
```

Két dolgot kell tudni róla. Egyrészt az alias nem specializálható, és nem lehet rá részleges specializációt írni; ha ilyesmire van szükség, magát az eredeti sablont kell specializálni. Másrészt C++20 előtt az alias nem vett részt a CTAD-ban; C++20-tól az eredeti sablon guide-jaiból származtatott guide-okkal igen, ahogy fentebb láttuk. Az alias tehát kényelmi eszköz, nem új entitás.

A **variable template** C++14 óta létezik, és fordítási idejű konstansok típusfüggő családját adja:

```
template<class T>
inline constexpr T zero = T{};

template<class T>
inline constexpr T pi = T(3.14159265358979323846L);
```

A `zero<int>` és a `zero<double>` külön változó. Headerben a `template` és a `constexpr` együtt biztosítja, hogy minden TU-ban ugyanaz a definíció legyen elérhető; az `inline` itt a szándékot fejezi ki, és C++17-től megengedett. A variable template-re is igaz, hogy a specializáció a template argumentumoktól függ, tehát nem lehet futásidejű értékkel példányosítani.

26.2.6 Headeres definíció, ODR és code bloat

A sablon definíciójának abban a fordítási egységben elérhetőnek kell lennie, ahol példányosítják. Ezért a sablonokat headerben tartjuk, nem `.cpp`-ben. Ha a tagfüggvény definícióját kivennénk egy `.cpp`-be, a felhasználó TU-ban csak a deklaráció lenne látható, és a linker „undefined reference” hibát adna a példányosított tagra. Ez a leggyakoribb kezdő hiba.

Az **ODR** (one definition rule) alól a sablonok kivételt élveznek: ugyanaz a sablon több TU-ban is definiálható, feltéve, hogy a definíciók tokenre azonosak, és a nevek ugyanarra az entitásra oldódnak fel. Ez teszi lehetővé a headeres használatot. A linker a `Buffer<int, 16>` azonos példányosításait jellemzően összevonja (COMDAT/weak symbol mechanizmus), így a programban egy példány marad. A gyakorlatban mégis **code bloat** léphet fel: sok különböző `(T, N)` pár sok külön kódot generál, és a fordítási idő nő.

Két eszköz segít. Az **explicit instantiation** egy TU-ban előállítja a kívánt specializációt:

```
template class Buffer<int, 16>;
```

Az **extern template** deklaráció viszont azt mondja a fordítónak, hogy ne példányosítsa a specializációt ebben a TU-ban, mert máshol már megvan:

```
extern template class Buffer<int, 16>;
```

Ezzel csökkenthető az ismételt példányosítás, de csak akkor, ha a programban pontosan egy helyen van tényleges definíció, különben linkelési hibát kapunk. Az explicit instantiation és az `extern template` tehát kézben tartott, nem automatikus optimalizálás; a helyességet a linker és a tesztelés igazolja.

26.2.7 Tipikus hibák

A leggyakoribb minta, hogy a tagfüggvényt `.cpp`-be tesszük, majd a hívó TU-ból linkelünk. A javítás: a definíció headerbe kerül, vagy explicit instantiation történik a `.cpp`-ben. A második gyakori hiba, hogy `Buffer b{1,2,3}`-at írunk deduction guide nélkül; itt a `T` levezethető, de `N` nem, ezért a fordítás sikertelen. A harmadik: `template<auto N>`-t használunk ott, ahol a típus számít, és a negatív vagy Nem `size_t` érték miatt a specializáció mérete váratlan lesz. A negyedik: az alias template-re bízzuk a CTAD-ot, és azt tapasztaljuk, hogy nem működik. Az ötödik: a `Buffer`-t `N = 0`-val példányosítjuk, ami a nulla hosszú tömb miatt nem szabványos. Mindegyik hiba javítható a szerződés pontos kimondásával és a fordítási figyelmeztetések komolyan vételével.

26.2.8 Feladatok

1. **Alap.** Egészítsd ki a `Buffer<T, N>` osztályt `pop()` és `clear()` tagfüggvényekkel, amelyek a méretet csökkentik, de nem végeznek allokációt! Írj egy `main`-t, amely `Buffer<int, 4>`-et használ, és ellenőrzi a `capacity()` és `size()` értékét!

2. **Közepes.** Adj a `Buffer`-hez olyan explicit deduction guide-ot, amely `const char (&)[N]` argumentumból `Buffer<char, N-1>`-et vezet le (a lezáró nulla nélkül)! Magyarázd el, miért `N-1`, és mi történik, ha a tömb nem tartalmaz lezáró nullát!

3. **Nehéz.** Vezess be egy `Extent` structural type-ot, és készíts `StaticExtent<E>` sablont! Példányosítsd `Extent{16}` és `Extent{32}` értékkel, és igazold `static_assert`-tel, hogy a két specializáció különböző típus! Mi történik, ha az `Extent` tagja `mutable`?

4. **Diagnosztikai.** Írj egy két TU-ból álló programot, amelyben a `Buffer<int, 8>::push` definíciója egy `.cpp`-ben van! Figyeld meg a linker hibát, majd javítsd kétféleképpen: headerbe helyezéssel és explicit instantiationnal!

26.2.9 Megoldások

1. megoldás. A `pop()` a méretet csökkenti, a `clear()` nullázza; egyik sem szabadít fel memóriát, mert a tároló statikus méretű. A helyes változat:

```
void pop() { if (size_ > 0) --size_; }
void clear() noexcept { size_ = 0; }
```

Az ellenőrzés: `Buffer<int, 4> b; b.push(1); b.push(2);` után `b.size() == 2`, `b.capacity() == 4`. A `pop()` üres bufferre nem csökkent negatívba, mert a `size_` előjel nélküli. Ellenőrzési feltétel: a `capacity()` fordítási időben is használható, például `static_assert(Buffer<int,4>::capacity() == 4);`.

2. megoldás. A guide:

```
template<std::size_t N>
Buffer(const char (&)[N]) -> Buffer<char, N - 1>;
```

A `const char[N]` típusban `N` a tömb hossza, amely tartalmazza a lezáró nullát, ezért a hasznos karakterek száma `N-1`. Ha a tömb nem tartalmaz lezáró nullát, a `Buffer` mérete eggyel kisebb lesz a tényleges hossznál, és a `push` a végétől olvashatna; ezt a hívónak kell garantálnia, vagy a guide-ot csak `std::string_view`-alapú átvételre érdemes használni. A guide csak a típust vezeti le, a konstruktor továbbra is a `Buffer(std::initializer_list<T>)` vagy más konstruktor szerint inicializál.

3. megoldás. Az `Extent` aggregátum, minden tagja `public` és nem `mutable`, ezért structural type. A `StaticExtent<Extent{16}>` és a `StaticExtent<Extent{32}>` két különböző specializáció; a `static_assert` ezt a típusazonosságot ellenőrzi. Ha az `Extent` tagja `mutable`, a típus már nem structural type, és a sablonargumentumként való használat fordítási hibát ad. Ez a korlát a C++20 szabálya; a pontos támogatást a compiler verziójával egyeztetni kell.

4. megoldás. A `.cpp`-ben definiált tagra a hívó TU-ból „undefined reference to `Buffer<int, 8>::push(int const&)`” hibát kapunk. Javítás headerbe helyezéssel: a tagdefiníció a headerbe kerül. Javítás explicit instantiationnal: a `.cpp`-ben `template class Buffer<int, 8>;` szerepel, a headerben pedig a deklaráció. Ellenőrzési feltétel: a program linkel, és a `push` viselkedése mindkét változatban azonos; a két megoldás közül a headerbe helyezés a rugalmasabb, az explicit instantiation a bloat csökkentésére szolgál.

26.2.10 Összefoglaló

Az osztálysablon típuscsaládot ír le, amelyből a fordító a template argumentumok helyettesítésével konkrét specializációt állít elő. A nem típusú paraméter fordítási idejű érték, amely része a típusazonosságnak; a fix kapacitású buffer ezt használja ki. A C++20 structural type és `auto` NTTP bővíti a lehetőségeket, de a típus pontos megválasztása továbbra is a programozó felelőssége. Az alapértelmezett argumentum kényelmet ad, a CTAD és a deduction guide a kiírást csökkenti, de az alias template nem örökli a CTAD-ot. A sablonok headerben élnek, az ODR-megengedés és a linker összevonás teszi lehetővé a többszörös definíciót; az explicit instantiation és az `extern template` a code bloat kézi szabályozásának eszköze. A példák `-std=c++20` mellett értelmezhetők; a konkrét compiler- és library-támogatást a laborprofilban rögzített verzióval kell ellenőrizni, és a kód helyességét fordítással és teszttel kell igazolni, nem feltételezéssel.

26.3 Forwarding reference és perfect forwarding

A perfect forwarding célja, hogy egy argumentumot úgy adjunk tovább egy hívási láncon, hogy a végső célfüggvény pontosan ugyanazt a *value category*-t (lvalue/rvalue) és ugyanazokat a cv-minősítőket (const/volatile) lássa, mintha a hívó közvetlenül őt hívta volna. A téma látszólag deklarációs trükk, valójában értékszemantikai szerződés: könyvtári factory-k, `emplace` és wrapper API-k mind erre épülnek, és a félreértése néma másolást, teljesítményvesztést vagy épp nem szándékolt átvételt okoz.

Ez az alfejezet a Függvénysablon, instantiation és deduction (26.1) template-dedukciójára és a Move constructor, moved-from állapot és std::move (22.3) értékkategória-kezelésére épül. A `std::move` ott az "erőforrás átadásának jelzése" volt; itt élesen elválasztjuk a `std::forward` "változatlan továbbadás" szándékától. A C++20 alapot használjuk, a `requires`-szel megfogalmazott konstraint csak említés szintjén jelenik meg, mert részletes kezelése a Concepts, requires és constrained overload (27.2) témája.

26.3.1 A probléma: miért nem elég egy sima T&&

Tegyük fel, hogy van két `sink` overloadunk, egy lvalue-re és egy rvalue-re, és szeretnénk egy-egy köztes függvényt, amely csak továbbad. Naiv megoldás:

```
void sink(int&);
void sink(int&&);

void relay_lvalue(int&  x) { sink(x); }        // OK: lvalue marad
void relay_rvalue(int&& x) { sink(x); }        // HIBA: x maga lvalue!
```

A második függvényben `x` deklarált típusa `int&&`, de mivel nevesített változó, value category-je lvalue, ezért a `sink(int&)` hívódik. Javíthatjuk `sink(std::move(x))`-szel, de akkor a két függvényre ketté kell választani — és `N` paraméter, illetve minden cv-kombináció esetén ez nem skálázódik. Kell egy sablon, amely *megőrzi* az argumentum értékkategóriáját.

26.3.2 T&& dedukció és reference collapsing

Tekintsük a kanonikus alakot:

```
template<class T>
void f(T&& p);
```

Ez **forwarding reference** — ez a szabványos terminus, amelyet a C++20 [temp.deduct.call] definiál: egy függvénysablon cv-minősítetlen sablonparaméterére vonatkozó rvalue-referencia (amely nem osztálysablon paraméterét jelöli). A köznyelvi, történeti név a Scott Meyers-féle „universal reference"; ez nem szabványos kifejezés. A dedukció két esetre bomlik, és ehhez kell a **reference collapsing** szabály:

$$\begin{aligned} U\,\&\ \&\ &\to U\,\& \\ U\,\&\ \&\&\ &\to U\,\& \\ U\,\&\&\ \&\ &\to U\,\& \\ U\,\&\&\ \&\&\ &\to U\,\&\& \end{aligned}$$

Mnemotechnika: ha bármelyik oldalon `&` szerepel, az eredmény `&`; csak a `&& &&` marad `&&`.

26.3.2.1 A két dedukciós eset

Lvalue argumentum `U` típusú (esetleg `const U`, `volatile U`): a dedukált `T` **`U&`** lesz (a cv megmarad, tehát `const U&`), azaz a `T&&` paraméter típusa `U& &&` → collapse → `U&`.

Rvalue argumentum `U` típusú: a dedukált `T` `U`, a paraméter típusa `U&&`.

Numerikusan, `int i; const int ci = 0;` mellett:

Hívás	Argumentum kategóriája	Dedukált `T`	`p` típusa	a `p` névkifejezés	`std::forward<T>(p)`
`f(i)`	lvalue	`int&`	`int&`	lvalue	lvalue (`int&`)
`f(ci)`	lvalue (`const`)	`const int&`	`const int&`	lvalue	lvalue (`const int&`)
`f(42)`	prvalue	`int`	`int&&`	**lvalue**	xvalue (`int&&`)
`f(std::move(i))`	xvalue	`int`	`int&&`	**lvalue**	xvalue (`int&&`)

Ez a táblázat a forwarding reference egész lényege. A `p` *név* mint kifejezés minden sorban lvalue — egy névvel rendelkező változó mindig lvalue, akkor is, ha a típusa `int&&`. Az eredeti argumentum értékkategóriáját a dedukált `T` őrzi meg, és a `std::forward<T>(p)` állítja vissza: lvalue argumentumra lvalue-t, rvalue argumentumra xvalue-t ad.

```
// fwd_category.cpp - C++20: a p név mindig lvalue, a forward visszaadja az
 ↪  eredetit
#include <type_traits>
#include <utility>

template<class T>
constexpr int f(T&& p) {
    static_assert(std::is_lvalue_reference_v<decltype((p))>);     // p: mindig
     ↪  lvalue
    using Fwd = decltype(std::forward<T>(p));
    if constexpr (std::is_lvalue_reference_v<T>) {
        static_assert(std::is_lvalue_reference_v<Fwd>);           // forward:
         ↪  lvalue
        return 1;
    } else {
        static_assert(std::is_rvalue_reference_v<Fwd>);           // forward:
         ↪  xvalue
        return 2;
    }
}

int main() {
    int i = 0; const int ci = 0;
    static_assert(f(i) == 1 && f(ci) == 1 && f(42) == 2 && f(std::move(i)) ==
     ↪  2);
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic fwd_category.cpp` figyelmeztetés nélkül fordul: mind a négy hívásnál teljesül a `decltype((p))` lvalue-referencia volta, és a `std::forward<T>(p)` lvalue-, illetve rvalue-referencia típusa a táblázat szerinti.

26.3.3 Forwarding reference versus konkretizált T&&

A "forwarding reference" nem minden `T&&`. Négy feltételnek egyszerre kell teljesülnie:

1. A `T` (vagy más néven a paramétertípus) template paramétere egy **függvénysablonnak** — nem egy osztálysablon tagjának fix paramétere —, vagy `auto` dedukciós kontextusban szerepel.
2. A paraméter alakja szó szerint `T&&`, azaz **cv nélküli** `T` és `&&` — `const T&&`, `volatile T&&` nem forwarding.
3. A híváskor **tényleges dedukció** történik, nem explicit template argumentum (`f<int>(x)` esetén `T = int`, `p` típusa `int&&`, nem forwarding).
4. Ha osztálysablonon belül van, a tag maga is **függvénysablon** (`template<class U> void g(U&&);`), nem az osztály `T` paramétere.

Ellenpéldák, hogy a különbség éles legyen:

```
void h1(int&&);                     // konkrét típus, csak rvalue-t fogad, nincs
 ↪  dedukció

template<class T>
void h2(const T&&);                 // NEM forwarding: lvalue nem köthető rá

template<class T>
struct Box {
    void m1(T&&);                    // NEM forwarding: T az osztály paramétere
    template<class U>
    void m2(U&&);                    // forwarding: U a tag template paramétere
};

template<class T>
void h3(T&& t) { /* forwarding */ }

auto&& x = produce();               // forwarding reference (auto kontextusban)
```

`h2` esetén `h2(i)` (lvalue `int`) fordítási hiba: a `T&&` alakú forwarding csak úgy tud lvalue-t fogadni, hogy a `T` maga `&`-t tartalmaz. Ha a `const` már a paraméter szintjén ott van, a `T&&` nem tud `&`-t "nyelni".

26.3.4 std::forward<T> és std::move — két különböző szándék

A `std::forward<T>(t)` feltételes kaszt:

```
// egyszerűsített vázlat, C++14 utáni alak
template<class T>
constexpr T&& forward(std::remove_reference_t<T>& t) noexcept {
    return static_cast<T&&>(t);
}
```

Ha `T = U&`, akkor `T&& = U&`, a visszaadott referenciatípus `U&` → **lvalue** marad. Ha `T = U`, akkor `T&& = U&&` → **rvalue** lesz. A `std::forward` tehát nem "mozgat", hanem **visszaadja azt a value category-t, amit a `T` argumentum kódol**.

Ezzel szemben a `std::move(t)` mindig `static_cast<remove_reference_t<decltype(t)>&&>(t)` — feltétel nélkül rvalue-t csinál. A két függvény közötti választás nem stílus, hanem szándék:

- **`std::forward<T>`**: "nem tudom, mit kaptam, add tovább változatlanul" — pass-through wrapper, factory, relay.
- **`std::move`**: "én birtoklom ezt az objektumot, és most átadom" — owner oldali kifejezés, akkor is, ha a típus forwarding reference.

Helyes használat:

```
#include <iostream>
#include <utility>

struct Tracer {
    Tracer() = default;
    Tracer(const Tracer&) { std::cout << "copy\n"; }
    Tracer(Tracer&&) noexcept { std::cout << "move\n"; }
};

void consume(const Tracer&) { std::cout << "consume(const&)\n"; }
void consume(Tracer&&)      { std::cout << "consume(&&)\n"; }

template<class T>
void relay(T&& t) {
    std::cout << "relay: ";
    consume(std::forward<T>(t));    // értékkategória megőrzése
}

template<class T>
void relay_bad(T&& t) {
    std::cout << "relay_bad: ";
    consume(std::move(t));          // mindig rvalue: ellopná az lvalue-t is
}

int main() {
    Tracer t;
    const Tracer ct;
    relay(t);              // T = Tracer&         -> consume(const&)
    relay(ct);             // T = const Tracer&   -> consume(const&)
    relay(Tracer{});       // T = Tracer          -> consume(&&)
    relay_bad(t);          // T = Tracer&, de move -> consume(&&): hibás szándék
}
```

A fenti program a szerző szándéka szerint a jelzett három sort írja ki a `relay_bad` hibás sorával együtt (`consume(const&)`, `consume(const&)`, `consume(&&)`, majd `consume(&&)`); a kód **nem futtatott** és **nem tesztelt**, a kimenet csak elvárt illusztráció. A fordítás tipikus parancsa:

```
g++     -std=c++20 -Wall -Wextra -pedantic-errors -o relay relay.cpp
# vagy
clang++ -std=c++20 -Wall -Wextra -pedantic-errors -o relay relay.cpp
```

Kritikus pont: a `std::forward<T>` argumentumának **pontosan a dedukált `T`-nek kell lennie**. Ha `std::forward<U>`-t írunk olyan `U`-val, amely nem a `p` dedukált típusa, a kaszt rossz értékkategóriát állít elő — ez a leggyakoribb "majdnem helyes" hiba.

26.3.5 Forwarding factory és emplace

A `std::make_unique`, `std::make_shared` és `emplace_back` mind forwarding reference-re épül. Az `emplace_back` variadikus forwarding: az argumentumok `Args&&... args` alakban érkeznek, és a tároló a célobjektum helyén hívja meg a konstruktort.

```
#include <vector>
#include <string>
#include <iostream>

struct Widget {
    std::string name;
    int value;
    Widget(std::string n, int v) : name(std::move(n)), value(v) {
        std::cout << "ctor " << name << '=' << value << '\n';
    }
    Widget(const Widget& o) : name(o.name), value(o.value) {
        std::cout << "copy " << name << '\n';
    }
    Widget(Widget&& o) noexcept : name(std::move(o.name)), value(o.value) {
        std::cout << "move " << name << '\n';
    }
};

int main() {
    std::vector<Widget> v;
    v.reserve(8);
    std::string s = "alpha";
    v.emplace_back(s, 1);              // s lvalue  -> string copy
    v.emplace_back(std::move(s), 2);   // s rvalue  -> string move
    v.emplace_back("gamma", 3);        // const char* -> string(ptr)
}
```

Itt az `emplace_back` az `Args` vektorát ugyanúgy dedukálja, mint bármelyik függvénysablon: `s` lvalue -> `Args = {std::string&, int}`, a forward az `std::string&`-et továbbítja, a `Widget` konstruktora másol. `std::move(s)` esetén `Args = {std::string, int}`, a forward rvalue-t ad, és a `Widget` `std::string` tagja move-ol. A viselkedés `Tracer`-szerű nyomkövetéssel ellenőrizhető; a kimenet itt is elvárt, nem mért.

26.3.6 A greedy forwarding konstruktor

A legravaszabb csapda: egy `template<class T> X(T&&)` konstruktor **versenybe száll a copy/move konstruktorral**, és lvalue esetben nyerhet ellene. Ez azért van, mert a `const X&` és az `X&` között a kötés minősége eltér: `X&`-re `X&` **identity**, `const X&`-re `const X&` **qualification conversion**, és az identity jobb. Nem-template vs template esetén pedig a nem-template nyer, de csak **azonos minőségű** kötéseknél.

```cpp
#include <iostream>
#include <utility>

class Node {
public:
    int tag = 0;
    Node() = default;
    template<class T>
    Node(T&&) : tag(1) { std::cout << "template ctor\n"; }
    Node(const Node&) : tag(2) { std::cout << "copy ctor\n"; }
    Node(Node&&) noexcept : tag(3) { std::cout << "move ctor\n"; }
};

int main() {
    Node a;
    Node b(a);                  // template ctor nyer: T = Node&, Node& pontosabb, mint const Node&
    const Node c;
    Node d(c);                  // copy ctor nyer: nem-template döntetlenben
    Node e(std::move(a));       // move ctor nyer: nem-template döntetlenben
    std::cout << b.tag << ' ' << d.tag << ' ' << e.tag << '\n';   // elvárt: 1 2 3
}
```

A hiba lényege: `Node b(a)` esetén a `template<class T> Node(T&&)` `T = Node&`-del identity kötést ad, és a copy constructort megelőzi. Ezért a C++20 előtti kódok tipikusan `std::enable_if`-fel kizárták az azonos típust (`!std::is_same_v<std::decay_t<T>, Node>`), C++20-ban a `requires` a természetes eszköz:

```cpp
template<class T>
    requires (!std::same_as<std::remove_cvref_t<T>, Node>)
Node(T&&) : tag(1) { /* ... */ }
```

A `requires`-szel kapcsolatos részletek a Concepts, requires és constrained overload (27.2) témához tartoznak; itt csak a diagnosztikai minta a fontos: ha egy forwarding konstruktor unexpected módon nyer, az első kérdés mindig az, hogy kizártuk-e a saját típust.

26.3.7 Korlátok: braced-init-list, overload set, bit-field

A forwarding reference nem csodaszer; három tipikus esetben a dedukció egyszerűen nem tud végbemenni, mert nincs dedukálható típus:

```cpp
#include <initializer_list>

struct S { int b : 4; };
void g(int);
void g(double);

template<class T> void f(T&&);

S s;
f(s.b);                                   // HIBA: bit-field nem köthető referenciához
```

```
f(int(s.b));                                    // OK: ideiglenes int

f({1, 2, 3});                                   // HIBA: braced-init-list nem
 ↪ típus
f<std::initializer_list<int>>({1, 2, 3});       // OK: explicit template
 ↪ argumentum

f(g);                                           // HIBA: overload set nem egy
 ↪ típus
f(static_cast<void(*)(int)>(g));                // OK: konkrét függvénypointer
```

A braced-init-list `{1,2,3}` csak akkor vihető át, ha a cél típusa ismert — emiatt `emplace_back({1,2,3})` sem működik, még akkor sem, ha a cél `Widget{ std::initializer_list<int> }`-et várna. Az overload set neve nem típus; a `void g(int)` és `void g(double)` között a dedukció nem tud választani, ezért kell cast vagy `+g` jellegű kényszerítés (bár a `+g` overload set esetén is csak akkor működik, ha pontosan egy `g` fordul elő, mert az unáris `+` is túlterhelés-feloldást végez). A bit-field nem rendelkezik címmel, így referencia nem köthető hozzá — ilyenkor kényszerített másolás (`int(s.b)`) a helyes válasz.

26.3.8 Gyakori hibák

1. **`std::forward<T>` helyett `std::move`.** Egy relay wrapperben a `std::move` néma rvalue-konverziót okoz, és lvalue argumentumot is "ellop" a hívó elől.
2. **`std::forward<T>` és `std::move` felcserélése az owner oldalon.** A `std::forward<T>(member)` ott, ahol a hívó szándéka az átadás, csak látszólag helyes: az `T` szerint viselkedik, miközben a `member` sose volt forward referenciánk.
3. **`const T&&` vagy konkrét `T&&` összetévesztése a forwarding reference-szel.** A `const T&&` nem fogad el lvalue-t, a konkrét `int&&` pedig egyáltalán nem dedukál.
4. **A `T` kétszeri dedukálása.** `template<class T> void f(T&& a, T&& b)` esetén a két argumentumnak ugyanazt a `T`-t kell dedukálnia, így lvalue + rvalue pár nem megy át.
5. **Greedy konstruktor.** Forwarding konstruktor saját típus kizárása nélkül elnyeli a copy/move hívásokat.
6. **`std::forward` nem a `T`-vel.** `std::forward<SomeOtherType>(p)` csendben rossz értékkategóriát ad.

26.3.9 Feladatok

1. feladat (dedukciós tábla). Egészítse ki a táblázatot a `template<class T> void f(T&& p);` sablonhoz az alábbi hívásokra: `int i; const int ci; volatile int vi; int&& ri = std::move(i);` esetén `f(i)`, `f(ci)`, `f(vi)`, `f(ri)`, `f(std::move(ci))`. Adja meg a dedukált `T`-t, a `p` típusát és value category-jét, indokolja a reference collapsing lépést.

2. feladat (relay implementálása). Írjon `template<class T> void relay(T&& x)` wrapper-t, amely a `consume(const Tracer&)` / `consume(Tracer&&)` overload párra továbbítja az argumentumot. Készítsen tesztet lvalue, const lvalue és rvalue esetre, és ellenőrizze a kiválasztott overloadot. Mutassa meg a `std::move`-os hibaváltozatot, és magyarázza el, miért rossz.

3. feladat (greedy diagnózis). Adott a `Node` osztály, amelynek forwarding konstruktora van. Írjon egy hívási sorozatot, amelyben a forwarding konstruktor *elnyeli* a copy constructort, majd javítsa a kódot C++20 `requires`-szel. Adjon egy ellenőrzési feltételt: milyen hívásra melyik konstruktor fut, és miért.

26.3.10 Megoldások

26.3.10.1 1. feladat megoldása

Hívás	`T`	`p` deklarált típusa	`p` value category (`decltype((p))`)	`std::forward<T>(p)`
`f(i)`	`int&`	`int&`	lvalue (`int&`)	lvalue
`f(ci)`	`const int&`	`const int&`	lvalue (`const int&`)	lvalue
`f(vi)`	`volatile int&`	`volatile int&`	lvalue (`volatile int&`)	lvalue
`f(ri)`	`int&`	`int&`	lvalue (`int&`; nevesített!)	lvalue
`f(std::move(ci))`	`const int`	`const int&&`	**lvalue** (`const int&`)	xvalue (`const int&&`)

A `p` value category oszlopában minden sor lvalue: a paraméter *neve* minden esetben lvalue kifejezés, még az utolsó sorban is, ahol a deklarált típusa `const int&&`. A továbbított érték (`std::forward<T>(p)`) külön oszlopban látható: csak ez őrzi meg az eredeti rvalue-jelleget. Ellenőrzés: minden sorban `static_assert(std::is_lvalue_reference_v<decltype((p))>)`, az utolsó sorban emellett `static_assert(std::is_rvalue_reference_v<decltype(std::forward<T>(p))>)`.

Indoklás: `f(ri)` esetén `ri` deklarált típusa `int&&`, de **nevesített**, ezért value category-je lvalue, így `T = int&`, és a collapse után a `p` típusa `int&` — nem `int&&`. Az utolsó esetben a `std::move(ci)` `const int&&`-t ad, a dedukció `T = const int`, a paraméter `const int&&`.

26.3.10.2 2. feladat megoldása

```
#include <iostream>
#include <utility>

struct Tracer {
    Tracer() = default;
    Tracer(const Tracer&) { std::cout << "copy\n"; }
    Tracer(Tracer&&) noexcept { std::cout << "move\n"; }
};

void consume(const Tracer&) { std::cout << "consume(const&)\n"; }
void consume(Tracer&&)      { std::cout << "consume(&&)\n"; }

template<class T>
void relay(T&& x) {
    consume(std::forward<T>(x));
}

template<class T>
void relay_bad(T&& x) {
    consume(std::move(x));   // mindig rvalue
}
```

```
int main() {
    Tracer t;
    const Tracer ct;
    relay(t);     // consume(const&) - T = Tracer&
    relay(ct);    // consume(const&) - T = const Tracer&
    relay(Tracer{});        // consume(&&) - T = Tracer
    relay_bad(t); // consume(&&) - de t-t elloptuk volna
}
```

Ellenőrzési feltétel: `relay(t)` és `relay(ct)` sora `consume(const&)`, `relay(Tracer{})` sora `consume(&&)`. Ha bármelyik korábbi sor `consume(&&)`-t ad, a `T` nem a dedukált típus vagy hibás a forward. `relay_bad(t)` bizonyítja, hogy a `std::move` minden lvalue-t rvalue-ként továbbít — ez a wrapperben néma másolás-eltörlés (esetleg adatvesztés) felé vezet.

26.3.10.3 3. feladat megoldása

```
#include <concepts>      // std::same_as
#include <iostream>
#include <type_traits>  // std::remove_cvref_t
#include <utility>

class Node {
public:
    int tag = 0;
    Node() = default;
    template<class T>
        requires (!std::same_as<std::remove_cvref_t<T>, Node>)
    Node(T&&) : tag(1) { std::cout << "template\n"; }
    Node(const Node&) : tag(2) { std::cout << "copy\n"; }
    Node(Node&&) noexcept : tag(3) { std::cout << "move\n"; }
};

int main() {
    Node a;
    Node b(a);                  // copy ctor (constraint kizárja a template-et)
 ↪  -> tag 2
    Node d(std::move(a));       // move ctor -> tag 3
    Node e(42);                 // template ctor -> tag 1
    std::cout << b.tag << ' ' << d.tag << ' ' << e.tag << '\n';  // elvárt: 2 3
     ↪  1
}
```

A `requires (!std::same_as<std::remove_cvref_t<T>, Node>)` konstraint kizárja, hogy a forwarding konstruktor bármely cv/ref kombinációjú `Node`-ra versenyezzen. A `remove_cvref_t` C++20-ban elérhető, az `std::same_as` szintén. A `Node e(42)` hívás mutatja, hogy a template továbbra is fogad idegen típust — a konstraint nem tiltja le a forwardingot, csak a saját típusra zárja ki.

26.3.11 Összefoglaló

A forwarding reference a `T&&` azon alakja, ahol `T` függvénysablon-paraméter, cv nélküli, és tényleges dedukció történik. Lvalue argumentumra `T = U&`, rvalue-ra `T = U`; a reference collapsing biztosítja, hogy a paraméter pontosan az eredeti értékkategóriát és cv-jét tükrözze. A `std::forward<T>` ennek a value category-nek a továbbadása, a `std::move` feltétel nélküli rvalue-konverzió — a kettő nem ekvivalens, és a wrapper/factory mintákban a forward a helyes eszköz, a move az owner oldal eszköze. A `const T&&`, `int&&` konkrét alakok és az osztálysablon fix `T`-je nem forwarding reference-ek. A braced-init-list, overload set és bit-field nem dedukálható, ezekhez célzott kényszerítés vagy explicit template argumentum kell. A greedy forwarding konstruktor kizárja magát a saját típusra, C++20-ban tipikusan `requires`-szel; a részletek a concepts fejezetben következnek.

26.4 Specialization, overload és explicit instantiation

A template-ek generikus szerződést adnak: a primary template megfogalmazza, milyen műveleteket várunk el egy típustól, a specializáció pedig ezt a szerződést pontosítja egy konkrét típusra vagy típusmintára. A specializáció azonban nem varázslat: láthatósági, ODR- és overload-resolution következményei vannak. Ugyanakkor az explicit instantiation nem specializáció, hanem fordítási szervezési eszköz: azt dönti el, melyik fordítási egységben készüljön el a példányosított kód. Ez a fejezet ezt a három mechanizmust választja szét: teljes és részleges specializáció, függvény-overload, valamint explicit instantiation és `extern template`. A cél, hogy eld tudd dönteni, melyik mechanizmus milyen szerződést és milyen fordítási szervezést támogat.

26.4.1 Primary template, teljes specializáció és a szerződés

A **primary template** a sablon általános definíciója. A **teljes specializáció** (explicit specialization) a sablon minden paraméterére konkrét argumentumot ad. Osztálysablonnál ez azt jelenti, hogy a specializáció a primary mintájára illeszkedő konkrét argumentumlistára külön definíciót ad. A specializáció nem örökli a primary tagjait: ha a primarynak van egy `encode` tagja, a specializációnak magának kell biztosítania azt, amit a használat megkövetel.

Ez a gyakorlatban egy programozói szerződés. A szabvány nem kényszeríti ki, hogy a specializáció ugyanazt az interfészt adja, mint a primary; azt sem, hogy ugyanazt a viselkedést adja. A szerződés a felhasználó és a template szerzője között jön létre. Ha a specializáció eltér, a program attól függően fordul vagy nem fordul, hogy a specializáció látható-e a használat helyén.

```
#include <string>
#include <vector>

template <typename T>
struct serializer {
    static std::string encode(T const&) { return "generic"; }
};

template <>
struct serializer<int> {
    static std::string encode(int v) { return std::to_string(v); }
};

template <typename T>
struct serializer<std::vector<T>> {
```

```
    static std::string encode(std::vector<T> const& v) {
        std::string out = "[";
        for (std::size_t i = 0; i < v.size(); ++i) {
            if (i) out += ",";
            out += serializer<T>::encode(v[i]);
        }
        out += "]";
        return out;
    }
};
```

Itt a `serializer<int>` teljes specializáció, a `serializer<std::vector<T>>` pedig részleges specializáció. A `serializer<std::vector<int>>` a részleges specializációt használja, és azon belül a `serializer<int>` teljes specializációját. A primary generikus szerződése: van `encode`, amely `std::string`-et ad vissza. A specializációk ezt megtartják, de más implementációt adnak.

A teljes specializáció akkor jó választás, ha egy konkrét típusra teljesen más implementációt akarsz adni, és a típus nem illeszkedik egy általánosabb mintára. Ha a viselkedést csak egy típusmintára akarod szabni, a részleges specializáció a természetes eszköz.

26.4.2 Részleges specializáció osztály- és változósablonoknál

A **részleges specializáció** (partial specialization) a primary paraméterlistájának egy mintájára illeszkedik, de nem minden paramétert köt le. Osztálysablonnál ez a primary egy specializált változata; a fordító akkor választja, ha a konkrét argumentumlista illeszkedik a specializáció mintájára, és a specializáció jobb illeszkedés, mint a primary.

```
template <typename T, typename U>
struct pair_traits {
    static constexpr bool same = false;
};

template <typename T>
struct pair_traits<T, T> {
    static constexpr bool same = true;
};
```

A `pair_traits<int, int>` a részleges specializációt választja, a `pair_traits<int, double>` a primaryt. A részleges specializáció feltétele, hogy a specializációs argumentumlista illeszkedjen a primary paraméterlistájára, és a minta ne legyen teljesen általános. A `pair_traits<T, T>` minta illeszkedik, de nem minden paramétert köt le külön-külön; ezért részleges.

Változósablonoknál a részleges specializáció C++14 óta megengedett. Ez különösen type trait-eknél hasznos:

```
template <typename T>
inline constexpr bool is_pointer_v = false;

template <typename T>
inline constexpr bool is_pointer_v<T*> = true;

template <typename T>
inline constexpr bool is_pointer_v<T* const> = true;
```

A változósablon részleges specializációja ugyanúgy működik, mint az osztálysabloné: a minta illeszkedését vizsgálja. A `is_pointer_v<int*>` értéke `true`, a `is_pointer_v<int>` értéke `false`.

Fontos különbség: a részleges specializáció osztály- és változósablonnál megengedett, függvénysablonnál nem. Ez nem stíluskérdés, hanem nyelvi szabály.

26.4.3 Függvénysablon: nincs részleges specializáció, van overload

A függvénysablonnak nincs részleges specializációja. Ha egy függvénysablonra egy típusmintára specializált viselkedést akarsz adni, **overloadot** írsz. Az overload egy új függvénysablon, amely részt vesz az overload resolutionben; a részleges specializáció hiánya miatt ez a természetes és hordozható megoldás.

```
#include <iostream>
#include <vector>

template <typename T>
void print(T const&) {
    std::cout << "generic\n";
}

template <typename T>
void print(std::vector<T> const&) {
    std::cout << "vector\n";
}
```

A `print(std::vector<int>{})` hívás a vektoros overloadot választja, mert az jobb illeszkedés, mint a generikus `print(T const&)`. A `print(42)` hívás a generikusat választja. Ez nem specializáció, hanem overload: a két függvénysablon versenyez az overload resolutionben.

A függvénysablon **teljes specializációja** megengedett, de törékeny. Az explicit specializáció nem vesz részt az overload resolutionben; csak akkor választódik, ha a primaryt választották, és a template argumentumok pontosan illeszkednek. Ezért a függvénysablonoknál az overload általában kiszámíthatóbb.

```
#include <iostream>

template <typename T>
void f(T) { std::cout << "primary\n"; }

template <>
void f<int>(int) { std::cout << "int specialization\n"; }

template <typename T>
void f(T*) { std::cout << "pointer overload\n"; }

int main() {
    f(0);       // int specialization
    int x = 0;
    f(&x);      // pointer overload
}
```

A `f(0)` hívásnál a primary `f(T)` T=int, és a `f<int>(int)` teljes specializáció pontos illeszkedés, ezért azt használja. A `f(&x)` hívásnál a pointer overload `f(T*)` jobb illeszkedés, mint a primary `f(T)`, ezért az overloadot választja; a specializáció nem versenyez vele. Ez szemlélteti, miért kell a függvénysablonoknál az overloadot előnyben részesíteni, ha a viselkedést típusmintára akarod szabni.

A gyakorlati döntés tehát: ha a viselkedést egy típusmintára akarod szabni, és a hívás függvényhíváson keresztül történik, írj overloadot. Ha a típusrendszerben akarsz egy specializált definíciót adni, és a típus például egy osztálysablon argumentuma, használj class partial specializationt. A függvénysablon partial specializationja nem opció.

26.4.4 Láthatóság, ODR és a specializáció helye

A specializáció láthatósága nem részletkérdés: a program jelentését befolyásolja. A specializációt a primary template deklarációja után, de az első olyan használat előtt kell deklarálni, amely implicit instantiációt okozna. Ha a specializáció csak később válik láthatóvá, a program ill-formed, de diagnosztika nem kötelező. Ezért a specializációk headerbe kerüljenek, a primary után.

Az explicit specializáció nem lesz automatikusan `inline` attól, hogy a primary `inline` volt. Ha függvényspecializációt teszel headerbe, és több fordítási egységben is definiálod, `inline`-nak kell lennie, különben megsérted az ODR-t. Az osztálysablon specializációja headerbe kerülhet, mert a class definíciókra az ODR többszörös definíciót enged meg azonos definícióval. A függvényspecializációnál ez nem ilyen egyszerű: a definíciót `inline`-ként kell megjelölni, ha headerben van.

Az ODR alapelve: egy programban egy entitásnak egy definíciója lehet; a template-ek és az inline függvények kivételek, de a definícióknak minden fordítási egységben azonosaknak kell lenniük. Az explicit instantiation definition legfeljebb egyszer szerepelhet a programban; ha többször, az ODR-sértés, diagnosztika nem kötelező.

26.4.5 Explicit instantiation: definition és extern template

Az **explicit instantiation definition** azt kéri a fordítótól, hogy egy adott fordítási egységben példányosítsa a sablont:

```
template struct Widget<int>;
template void process<int>(int);
```

Az **extern template** deklaráció azt kéri, hogy ebben a fordítási egységben ne történjen meg az implicit példányosítás, mert a definíció máshol elérhető:

```
extern template struct Widget<int>;
extern template void process<int>(int);
```

A tipikus elrendezés: a sablon definíciója headerben van; egy `.cpp` fájlban explicit instantiation definition szerepel a gyakran használt specializációkra; a többi `.cpp` fájlban `extern template` deklaráció szerepel a használat előtt. Így a példányosítás egyszer történik meg, a többi fordítási egység csak hivatkozik rá.

`widget.hpp`:

```
#ifndef WIDGET_HPP
#define WIDGET_HPP

#include <cstddef>

template <typename T>
struct Widget {
    T value;
    std::size_t size() const;
};

template <typename T>
std::size_t Widget<T>::size() const { return sizeof(T); }

#endif
```

widget.cpp:

```
#include "widget.hpp"

template struct Widget<int>;
template struct Widget<double>;
```

main.cpp:

```
#include "widget.hpp"

extern template struct Widget<int>;
extern template struct Widget<double>;

int main() {
    Widget<int> a{42};
    Widget<double> b{3.14};
    return static_cast<int>(a.size() + b.size());
}
```

Buildparancs:

```
g++ -std=c++20 -O2 -Wall -Wextra -pedantic -c widget.cpp -o widget.o
g++ -std=c++20 -O2 -Wall -Wextra -pedantic -c main.cpp -o main.o
g++ widget.o main.o -o app
```

Az `extern template` nem specializáció, hanem példányosítási kérés. Ha a definíciót végül nem adja meg senki, a linker hibát jelez. Az explicit instantiation definition csak azokat a tagokat példányosítja, amelyek a definition pontján definiáltak; ha egy tagot később definiálsz, lehet, hogy nem lesz példányosítva. Ez a szabvány részletes szabálya, ezért a definíciók sorrendjére figyelni kell.

26.4.6 Fordítási idő és bináris méret mérése

Az explicit instantiation elsődleges nyeresége a fordítási idő és a fordítási egységek mérete. Ha sok `.cpp` fájl használja ugyanazt a sablont, implicit instantiációval mindegyik példányosítja a sablont; `extern template`-tel a példányosítás egy helyre kerül. A végső bináris méretre gyakorolt hatás

bizonytalanabb: a linker gyakran összevonja a gyenge szimbólumokat, így a bináris méret nem feltétlenül csökken. A mérést mindkét szinten el kell végezni.

Mérési terv:

1. Két buildet készíts: (A) minden fordítási egységben implicit instantiáció, `extern template` nélkül; (B) explicit instantiation definition egy `.cpp`-ben, `extern template` a többi `.cpp`-ben.
2. GCC-nél `-ftime-report`, Clang-nál `-ftime-trace` kapcsolóval mérd a fordítási időt. A `-ftime-trace` JSON-t ír, amelyet külön eszközzel lehet elemezni.
3. A fordítási egységek méretét `size`-dal, a szimbólumokat `nm -C`-vel vizsgáld. A végső bináris méretet szintén `size`-dal mérd.
4. Rögzítsd a compiler és a standard library verzióját, a targetet, az optimalizálási szintet, a fordítási egységek számát és a buildparancsokat. A mérés csak így reprodukálható.

```
g++ -std=c++20 -O2 -ftime-report -c main.cpp -o /dev/null
clang++ -std=c++20 -O2 -ftime-trace -c main.cpp -o /dev/null
nm -C widget.o | grep Widget
size app
```

Ezek a parancsok a laborhoz tartoznak; a fejezet nem állítja, hogy a példák le lettek futtatva. A várt irány az, hogy a `main.o` mérete és a `main.cpp` fordítási ideje csökken, ha a példányosítás máshol történik. A végső bináris méretnél a linker összevonása miatt lehet, hogy nem látszik különbség. A mérés célja nem egyetlen szám, hanem a mechanizmus költségének megértése.

26.4.7 Gyakori hibák és javításuk

- **A specializáció a használat után van deklarálva.** A program ill-formed, diagnosztika nem kötelező. Javítás: a specializáció a primary után, de az első használat előtt legyen a headerben.
- **Függvénysablon részleges specializációját próbálod írni.** Ez nem fordul le. Javítás: overloadot írj, vagy class partial specializationt használj, ha a típusrendszerben akarsz dolgozni.
- **Függvénysablon teljes specializációja nem vesz részt az overload resolutionben.** A specializáció csak akkor választódik, ha a primaryt választották. Javítás: overloadot írj, ha a különböző argumentumtípusokra más implementációt akarsz.
- **`extern template` után hiányzik a definíció.** Link error. Javítás: legyen egy `.cpp`, ahol az explicit instantiation definition szerepel.
- **Az explicit instantiation definition többször szerepel a programban.** ODR-sértés, diagnosztika nem kötelező. Javítás: egy helyen legyen a definition, máshol `extern template`.
- **A függvényspecializáció headerben van, de nincs `inline`.** Több fordítási egységben definiálva ODR-sértés. Javítás: `inline` jelölés, vagy a definíció egy `.cpp`-be kerül.
- **A specializáció nem örökli a primary tagjait.** Ha a primarynak van egy tagja, a specializációnak magának kell azt biztosítania. Javítás: a specializáció interfészét a használat igényeihez igazítsd, és dokumentáld a szerződést.

26.4.8 Feladatok

26.4.8.1 1. feladat: mechanizmusválasztás

Magyarázd el, mi a különbség a class template partial specialization és a function template overload között. Miért nincs függvénysablon részleges specializációja? Adj egy példát, amelyben a kettő

közötti választás megváltoztatja a program viselkedését.

26.4.8.2 2. feladat: specializációs lánc

Írj egy `serializer<T>` primary template-et, egy teljes specializációt `int`-re, egy teljes specializációt `std::string`-re, valamint egy részleges specializációt `std::vector<T>`-re. A vektoros specializáció a `serializer<T>::encode`-ot használja az elemekre. Ügyelj arra, hogy a specializációk a primary után és a használat előtt legyenek láthatók.

26.4.8.3 3. feladat: diagnosztika

Adott egy header, amelyben a `serializer<int>` specializáció a `main` után van definiálva. Mi a hiba? Hogyan javítod? Mi a helyzet akkor, ha a specializáció függvénysabloné, és nincs `inline`? Fogalmazz meg ellenőrzési feltételt a javításra.

26.4.8.4 4. feladat: mérési terv

Tervezz egy mérést, amely összehasonlítja az implicit instantiációt az explicit instantiation + `extern template` megoldással. Milyen parancsokat használsz, mit rögzítesz, és mire kell vigyázni az értelmezésnél?

26.4.9 Megoldások

26.4.9.1 1. feladat megoldása

A class template partial specialization a típusmintára illeszkedik, és a primaryt helyettesíti, ha a minta illeszkedik. A function template overload egy új függvénysablon, amely részt vesz az overload resolutionben. A függvénysablonnak azért nincs részleges specializációja, mert a függvényeknél az overload resolution és a template argument deduction már így is összetett; a részleges specializáció plusz dimenziót adna, és a függvényspecializációk nem vesznek részt az overload resolutionben. Az overload természetesebb és kiszámíthatóbb.

Példa: a `print(T const&)` és a `print(std::vector<T> const&)` overloadok közül a `print(std::vector<int>{})` a vektorosat választja, a `print(42)` a generikusat. Ha ugyanezt függvénysablon részleges specializációval akarnád megoldani, nem fordulna le. Ellenőrzés: a hívások a várt overloadot választják; a fordító nem jelez hibát a részleges specializáció miatt, mert az nem szerepel a kódban.

26.4.9.2 2. feladat megoldása

```
#include <string>
#include <vector>

template <typename T>
struct serializer {
    static std::string encode(T const&) { return "generic"; }
};

template <>
```

```cpp
struct serializer<int> {
    static std::string encode(int v) { return std::to_string(v); }
};

template <>
struct serializer<std::string> {
    static std::string encode(std::string const& s) { return "\"" + s + "\""; }
};

template <typename T>
struct serializer<std::vector<T>> {
    static std::string encode(std::vector<T> const& v) {
        std::string out = "[";
        for (std::size_t i = 0; i < v.size(); ++i) {
            if (i) out += ",";
            out += serializer<T>::encode(v[i]);
        }
        out += "]";
        return out;
    }
};
```

A specializációk a primary után vannak, és a használat előtt láthatók. A `serializer<std::vector<int>>` a részleges specializációt használja, azon belül a `serializer<int>` teljes specializációját. Ellenőrzés: `serializer<std::vector<int>>::encode({1,2})` eredménye `[1,2]`; a `serializer<std::string>::encode("x")` eredménye `"x"`. A pontos kimenet a `std::to_string` és a string-összefűzés miatt a laborban ellenőrizendő.

26.4.9.3 3. feladat megoldása

A specializáció a használat után van deklarálva, ezért a program ill-formed, de diagnosztika nem kötelező. A fordító a primaryt használhatja, és a specializáció nem érvényesül. Javítás: a specializációt a primary után, de az első használat előtt kell deklarálni, lehetőleg headerben.

Ha a specializáció függvénysabloné, és headerben van, `inline` kell, különben több fordítási egységben definiálva ODR-sértés. Ellenőrzés: a specializáció használatának helyén a specializáció látható; a `nm -C` kimenetében a specializáció szimbóluma a várt módon jelenik meg; a program viselkedése a specializált implementációt tükrözi.

26.4.9.4 4. feladat megoldása

Két buildet készítek. Az (A) változatban minden fordítási egység implicit instantiációt végez. A (B) változatban egy `.cpp` tartalmazza az explicit instantiation definitiont, a többi `.cpp` pedig `extern template` deklarációt a használat előtt. GCC-nél `-ftime-report`, Clang-nál `-ftime-trace` kapcsolóval mérem a fordítási időt. A fordítási egységek méretét `size`-dal, a szimbólumokat `nm -C`-vel vizsgálom. A végső bináris méretet szintén `size`-dal mérem.

Rögzíteni kell a compiler és a standard library verzióját, a targetet, az optimalizálási szintet, a fordítási egységek számát és a buildparancsokat. Vigyázni kell arra, hogy a linker összevonhatja a gyenge szimbólumokat, ezért a bináris méret nem feltétlenül csökken; a fő nyereség a fordítási idő és a fordítási egységek mérete. A mérés csak akkor értelmes, ha a sablon tényleg sok fordítási

egységben példányosul. Ellenőrzés: a (B) változatban a `main.o` mérete és a `main.cpp` fordítási ideje csökken; a végső bináris méret változása külön mérendő.

26.4.10 Összefoglaló

A teljes specializáció egy konkrét argumentumlistára ad új definíciót; a részleges specializáció egy típusmintára illeszkedik, és osztály- vagy változósablonnál használható. A függvénysablonnak nincs részleges specializációja; helyette overloadot írunk. A függvénysablon teljes specializációja megengedett, de nem vesz részt az overload resolutionben, ezért törékeny. A specializációt a primary után, az első használat előtt kell deklarálni; a függvényspecializáció headerben `inline` kell legyen. Az explicit instantiation definition és az `extern template` nem specializáció, hanem fordítási szervezés: a példányosítást egy fordítási egységbe tereli. A fordítási idő és a fordítási egységek mérete mérhető; a végső bináris méretre gyakorolt hatás a linker összevonása miatt bizonytalan.

26.4.11 Források és ellenőrzési korlátok

A fejezethez nem érkezett forrásjegyzet, ezért a szabványparagrafusokra való pontos hivatkozás, a compiler-specifikus viselkedés és a mérési eredmények ebben a szövegben nem ellenőrzöttek. A megadott kiinduló URL-ek – a WG21 nyilvános dokumentumai, a GCC és a Clang dokumentációja – továbbtanulási kiindulópontok, nem a fejezet állításainak bizonyítékai. A laborban a rögzített compiler- és standard library-verzióval, a szabvány nyilvános draftjának megfelelő szakaszával és tényleges méréssel kell ellenőrizni a specializáció láthatóságára, az ODR-re és az explicit instantiation hatására vonatkozó állításokat. A C++20-as alaphoz tartozó nyilvános draft konkrét URL-je nem szerepelt a forrásjegyzetben; ennek hiányát a labornak kell pótolnia.

26.5 Dependent name, kétfázisú lookup és ADL

A sablon nem szöveges makró: a fordító a sablon definícióját akkor is elemzi, amikor a konkrét típusargumentum még ismeretlen. Ebből következik a fejezet központi kérdése: egy név jelentését mikor lehet eldönteni — már a definíció helyén, vagy csak a példányosítás helyén? A válasz nem esztétikai: ugyanaz a forrás nem-sablon kontextusban lefordul, sablonban viszont nem; vagy ami rosszabb, csendben a „rossz” függvényt hívja meg. Gyakorlati tétje akkor van, amikor egy headerben lévő generikus segédfüggvény egy projekt-specifikus `swap`-et vagy `operator<<`-t szeretne megtalálni, és nem találja meg — miközben a kód tökéletesen néz ki.

26.5.1 Dependent és nem dependent nevek

Egy név (name) *dependent* (függő), ha a jelentése — nem a léte, hanem az, hogy melyik entitást jelöli — legalább egy template paramétertől függ. Klasszikus példa: `T::value_type`, ahol `T` típusparaméter, vagy `f(x)`, ahol `x` típusa függ egy paramétertől. Egy név *nondependent*, ha a jelentése a sablon definíciójának helyén eldönthető: `std::size_t`, `::open`, `obj.field`, ha `obj` típusa nem függ paramétertől.

A megkülönböztetés feltétele, hogy a sablon-definíció és a példányosítás két külön kontextus. A szabvány ezért két pontot nevez meg: a *definition context* a sablon definíciójának helye, az *instantiation context* (point of instantiation) az a hely, ahol a sablon ténylegesen példányosul — jellemzően a hívás helye, de implicit példányosításnál a fordítási egység vége is számíthat. A nem

dependent neveket a definíció helyén kell feloldani: ekkor még nincs szükség a paraméterre. A dependent neveket a példányosítás helyén, a konkrét argumentumok ismeretében kell feloldani.

Ebből azonnal adódik egy fontos következmény, amely a legtöbb meglepetést okozza: egy sablon *belsejében* tett nem függő hívás nem látja meg a sablon *után* deklarált függvényeket. Ez nem fordítóhiba, hanem a nyelv szerződése — a kód szándéka és a nyelv viselkedése tér el egymástól.

26.5.2 Kétfázisú lookup: mi történik mikor

A kétfázisú névfeloldás (two-phase lookup) a fenti felosztást fordítja le műveletsorra.

Első fázis (definíció). A fordító értelmezi a sablon szövegét, ellenőrzi a szintaxist, feloldja a nem függő neveket, és a nem függő hívásoknál elvégzi a szokásos túlterhelés-feloldást. A függő hívásoknál megjegyzi a definíció helyén látható jelölteket, de a döntést későbbre halasztja.

Második fázis (példányosítás). A konkrét típusok ismeretében feloldja a függő neveket. Függő argumentumú, minősítetlen függvényhívásnál két forrásból dolgozik: a definíció kontextusában végzett ADL-ből és a példányosítás kontextusában végzett ADL-ből. A közönséges (nem ADL) minősítetlen lookup viszont kizárólag a definíció kontextusában történik meg — a példányosítás helyén nem.

Ez a pont az, amit érdemes szó szerint megjegyezni: **a közönséges lookup a definícióhoz kötődik, az ADL viszont a példányosítás helyén is lefut.** Az alábbi két példa pontosan ezt a különbséget mutatja meg.

```
// ch26_adl_instantiation.cpp
#include <cstdio>

template <class T>
void call_f(T value) {
    f(value);              // függő hívás: ADL a példányosítás helyén
}

struct Widget {};

void f(Widget) { std::puts("f(Widget)"); }   // a sablon UTÁN deklarálva

int main() {
    call_f(Widget{});
}
```

```
g++ -std=c++20 -Wall -Wextra -pedantic-errors -o ch26_adl_instantiation
 ↪  ch26_adl_instantiation.cpp
# Clang esetén: clang++ -std=c++20 -Wall -Wextra -pedantic-errors ...
```

A példa lefordul, mert `Widget` a globális névtérben van, így a hozzá asszociált névtér a globális névtér, és az ott a példányosítás idejére már látható `f(Widget)` ADL-lel megtalálható. (A kimenet várhatóan `f(Widget)`; ez a cikkben megadott elvárás, nem mért eredmény.)

A következő változat viszont nem fordul le, mert a `T = int` esetben nincs asszociált névtér, amiben az ADL kereshetne:

```
// ch26_late_decl_error.cpp  -- NEM fordul le (szándékos hibapélda)
template <class T>
void call_helper(T value) {
    helper(value);          // közönséges lookup: itt semmi; ADL: int-hez nincs
 ↪  névtér
}

void helper(int) {}         // a sablon után deklarálva

int main() { call_helper(0); }
```

A diagnosztika jellegzetes alakja: a `helper` nem deklarált ebben a hatókörben, és az argumentumfüggő keresés sem talált deklarációt a példányosítás helyén. Ezt a hibaüzenet-szöveget illusztrációként idézem, nem mért kimenetként: a pontos megfogalmazás fordítónként és verzióként változik. Fontos korlát: a kétfázisú lookup szigorú betartása implementációfüggő, a Microsoft fordító hagyományosan késleltetett feloldást használ, és csak a `/permissive-` kapcsoló mellett viselkedik a szabvány szerint. Ha a kód „az egyik gépen fordul, a másikon nem”, ez gyakori ok.

26.5.3 typename és template: a két disambiguator

A függő minősített név (`T::valami`) kétértelmű lehet: jelenthet típust, de jelenthet nem típust (változó, függvény, enumerátor) is. A nyelv ezért konzervatív szabályt alkalmaz: a függő minősített nevet alapértelmezésben nem-típusnak tekinti. Ha típus, a `typename` kulcsszó kötelező — ez a *typename disambiguator*.

```
// ch26_typename.cpp
#include <cstdio>
#include <vector>

template <class Container>
typename Container::value_type first_of(const Container& c) {
    using value_type = typename Container::value_type;
    value_type v = *c.begin();
    return v;
}

int main() {
    std::vector<int> v{7, 8};
    std::printf("%d\n", first_of(v));
}
```

```
g++ -std=c++20 -Wall -Wextra -pedantic-errors -o ch26_typename ch26_typename.cpp
```

A C++20 (P0634) azokban a kontextusokban tette elhagyhatóvá a `typename`-et, ahol szintaktikailag csak típus állhat: többek között névtér-szintű függvénydeklaráció visszatérési típusában, alias-deklarációban (`using X = ...;`), tagdeklarációban, záró visszatérési típusban és sablonparaméter alapértékében. A fenti példa mindkét `typename`-je ilyen helyen áll, ezért C++20-ban mindkettő elhagyható; C++17-ben mindkettő kötelező:

```
// typename_cpp20.cpp - C++20 (P0634): itt mindkét typename elhagyható
#include <cstdio>
#include <vector>
```

```cpp
template <class Container>
Container::value_type first_of(const Container& c) {   // névtér-szintű
 ↪  deklaráció
    using value_type = Container::value_type;            // alias-deklaráció
    value_type v = *c.begin();
    return v;
}

int main() {
    std::vector<int> v{7, 8};
    std::printf("%d\n", first_of(v));
}
```

Blokk-hatókörű deklarációban viszont a `typename` C++20-ban is kötelező, mert ott egy `X::y` kezdetű utasítás kifejezés is lehet:

```cpp
// typename_required.cpp - szándékosan hibás: blokkbeli deklarációban a typename
 ↪  C++20-ban is kell
#include <vector>

template <class Container>
auto first_of(const Container& c) {
    Container::value_type v = *c.begin();   // hiba: a függő név itt
 ↪  nem-típusnak számít
    return v;
}

int main() {
    std::vector<int> v{7, 8};
    return first_of(v);
}
```

Buildjegyzőkönyv (`lab-gcc` profil). A `typename_cpp20.cpp` `g++ -std=c++20 -Wall -Wextra -pedantic-errors` alatt fordul és `7`-et ír ki; `-std=c++17` alatt `need 'typename' before 'Container::value_type' because 'Container' is a dependent scope` hibát ad. A `typename_required.cpp` `-std=c++20` alatt is ugyanezt a hibát adja (Clang 22: `missing 'typename' prior to dependent type name`). Hordozható, C++17-tel is forduló kódban a `typename`-et továbbra is kiírjuk.

A második disambiguator a `template`. Ha egy függő típus *tagtemplate*-jét szeretnénk paraméterezni, a `<` jelet a fordító kisebb-jelnek is olvashatja. Ilyenkor a tag neve elé `template` kerül, hogy a szándék egyértelmű legyen.

```cpp
// ch26_template_kw.cpp
template <class T>
struct Registry {
    template <class U>
    static constexpr int id() { return static_cast<int>(sizeof(U)); }
};

template <class T>
constexpr int double_id() {
    return Registry<T>::template id<double>();   // 'template' nélkül parse-hiba
```

```
}

static_assert(double_id<int>() == sizeof(double));
```

```
g++ -std=c++20 -Wall -Wextra -pedantic-errors -c ch26_template_kw.cpp
```

A `template` kulcsszó itt nem stílus, hanem a nyelvtan része: nélküle a `<` összehasonlító operátorként indul, és a kifejezés szintaktikailag értelmezhetetlenné válik.

26.5.4 Függő bázisosztály és a this-> használat

A harmadik, gyakran kellemetlen eset a függő bázis. Ha a bázis típusa függ egy paramétertől, a fordító nem tudhatja, milyen tagokat örököl a leszármazott, ezért a bázis tagjait a minősítetlen lookup nem veszi figyelembe.

```
// ch26_dependent_base.cpp
template <class T>
struct Base {
    int value = 0;
    void reset() { value = 0; }
};

template <class T>
struct Derived : Base<T> {
    void bump() {
        // value = value + 1;      // hiba: a függő bázis tagja nem látszik
        this->value = this->value + 1;
        this->reset();
    }
    using Base<T>::reset;          // alternatíva: név behozatala a hatókörbe
};
```

A javításnak három egyenértékű formája van: `this->tag`, `Base<T>::tag`, illetve `using Base<T>::tag;`. Mindhárom azt mondja a fordítónak, hogy a nevet ne a definíció helyén, hanem a példányosítás után, a konkrét bázisban keresse. A `this->` alak a legolvashatóbb, és a függőséget is dokumentálja.

26.5.5 Argument-dependent lookup és a hidden friend

Az ADL (argument-dependent lookup, más néven Koenig-lookup) szabálya: minősítetlen függvényhívásnál a szokásos lookup mellett a hívás argumentumtípusaihoz *asszociált* névterekben és osztályokban is keresünk. Asszociált entitás a class típus maga, annak körülvevő névterei, bázisosztályai, valamint a template argumentumai által hozott névterek és osztályok. Pointer, referencia és tömb esetén a hivatkozott típus számít. Az `int`, `double` és a többi alapvető típus esetén viszont nincs asszociált névtér — innen ered a fenti hibapélda.

Ennek legismertebb következménye a *hidden friend*: az osztály belsejében `friend` kulcsszóval definiált függvény nem látható a közönséges minősítetlen lookup számára, kizárólag ADL-lel található meg.

```cpp
// ch26_hidden_friend.cpp
#include <cstdio>

namespace geo {
class Point {
public:
    Point() = default;
    Point(int x, int y) : x_(x), y_(y) {}
    friend bool operator==(const Point& a, const Point& b) {
        return a.x_ == b.x_ && a.y_ == b.y_;
    }
private:
    int x_ = 0, y_ = 0;
};
}  // namespace geo

int main() {
    geo::Point a{1, 2}, b{1, 2};
    std::printf("%d\n", a == b);          // ADL megtalálja
    // std::printf("%d\n", operator==(a, b));  // hiba: közönséges lookup nem
     ↪  látja
}
```

A hidden friend előnye, hogy hozzáfér a privát tagokhoz, mégsem szennyezi a környező névtér túlterhelési halmazát: csak akkor kerül elő, ha valaki tényleg `geo::Point` argumentummal hívja.

26.5.6 A swap customization minta

A legtöbbet idézett gyakorlati eset a `swap`. A cél: generikus kódban a típus saját, hatékony `swap`-jét használjuk, ha van, különben a `std::swap` általános, mozgatásalapú változatát. A helyes minta:

```cpp
// ch26_swap_adl.cpp
#include <cstdio>
#include <utility>

namespace lab {
class Pinned {
public:
    explicit Pinned(int id) noexcept : id_(id) {}
    Pinned(const Pinned&) = delete;
    Pinned& operator=(const Pinned&) = delete;
    int id() const noexcept { return id_; }

    friend void swap(Pinned& a, Pinned& b) noexcept {
        std::swap(a.id_, b.id_);
    }
private:
    int id_;
};
}  // namespace lab
```

```
template <class T>
void exchange(T& a, T& b) {
    using std::swap;    // közönséges lookup: std::swap látható a definíció
     ↪ helyén
    swap(a, b);         // ADL: lab::swap megtalálható a példányosítás helyén
}

int main() {
    lab::Pinned a{1}, b{2};
    exchange(a, b);
    std::printf("%d %d\n", a.id(), b.id());   // várt: 2 1
}
```

```
g++ -std=c++20 -Wall -Wextra -pedantic-errors -o ch26_swap_adl ch26_swap_adl.cpp
```

Három dolog működik egyszerre. A `using std::swap;` deklaráció a definíció helyén láthatóvá teszi a generikus `std::swap`-et, tehát a közönséges lookup ad egy használható jelöltet. A `swap(a, b)` argumentumai függőek, ezért a példányosítás helyén ADL is lefut, és megtalálja a `lab::swap`-et. A `Pinned` másolható és mozgatható sem (törölt műveletek), így a `std::swap` C++20-ban a `is_move_constructible_v && is_move_assignable_v` megszorítás miatt ki sem kerül a jelöltek közé, a nem-sablon `lab::swap` pedig amúgy is elsőbbséget élvezne a sablonnal szemben.

A hibás változat minősített hívást használ:

```
template <class T>
void exchange_bad(T& a, T& b) {
    std::swap(a, b);    // csak std::swap: ADL nincs, Pinned-del nem fordul le
}
```

A minősített névnél nincs ADL, ezért a felhasználói túlterhelés soha nem kerül szóba. A második gyakori hiba a felhasználói `swap` rossz névtérbe tétele: ha a `swap` a típus névterén kívül van, az ADL nem találja meg, még akkor sem, ha a hívás előtt deklarálták.

Két szerződésmegjegyzés. Program-definiált típusra a standard library azt várja el, hogy a `swap` a típushoz asszociált névtérben, nem-tag függvényként legyen elérhető; a `std` névtérbe új túlterhelést tenni nem megengedett. Specializáció megengedett, de a hordozható minta az ADL-lel megtalálható túlterhelés. (A `swap`-követelményt a C++20 library-szerződés írja le; ebben a fejezetben nem áll rendelkezésre ellenőrzött forrásszöveg a pontos paragrafus idézéséhez.)

26.5.7 Tipikus hibák

Az első hiba a hiányzó `typename` függő minősített típus előtt; a javítás nem `typedef`, hanem a kulcsszó kiírása. A második a hiányzó `template` tagtemplate-hívásnál; itt a `<` és `>` párosítása is elromolhat, ezért a hibaüzenet félrevezető lehet. A harmadik a függő bázis tagjának minősítetlen használata; a javítás `this->`, `Base<T>::` vagy `using`. A negyedik a „majd később deklarálom” feltételezés: nem függő hívásnál ez sosem működik, függő hívásnál is csak akkor, ha az argumentum típusához asszociált névtérben történik a deklaráció. Az ötödik a `std::swap(a, b)` minősített hívás generikus kódban. A hatodik a hidden friend közönséges névvel való hívása (`operator==(a, b)` alakban), ami definíció szerint nem található meg.

26.5.7.1 Ellenőrzési korlátok és verziófüggés

A fejezet állításai a C++20 nyelvi és library-szemantikára épülnek, de ebben a részben nem állt rendelkezésre ellenőrzött elsődleges forrásszöveg, ezért paragrafus szintű hivatkozást nem adok. A konkrét diagnosztikai szövegek illusztrációk. A kétfázisú lookup betartása és a `typename` elhagyásának C++20-as engedélyezése fordítótól és verziótól függ; a laborprofilban rögzíteni kell a fordítót, a verziót és a szigorú módot (`-pedantic-errors`, MSVC-nél `/permissive-`).

26.5.8 Összefoglaló

A sablonokat a fordító két fázisban dolgozza fel. A nem függő neveket a definíció helyén oldja fel, ezért a később deklarált függvények nem számítanak. A függő neveket a példányosítás helyén oldja fel; minősítetlen függvényhívásnál a közönséges lookup a definícióhoz, az ADL viszont a definícióhoz és a példányosításhoz is kötődik. A `typename` és a `template` disambiguator a függő minősített nevek kétértelműségét oldja fel. A függő bázis tagjait `this->`, `Base<T>::` vagy `using` viszi be a hatókörbe. A hidden friend és a felhasználói `swap` az ADL-lel válik elérhetővé, ezért a generikus kód `using std::swap; swap(a, b);` formát használ.

26.5.9 Feladatok

1. feladat (magyarázó). Állapítsd meg mindkét kódrészletről, hogy lefordul-e C++20-ban, és indokold a kétfázisú lookup szabályaival!

```
// A
template <class T> void use(T t) { pick(t); }
void pick(int);
int main() { use(0); }
```

```
// B
namespace n { struct X {}; }
template <class T> void use(T t) { pick(t); }
namespace n { void pick(X); }
int main() { use(n::X{}); }
```

2. feladat (kódolás). Az alábbi sablon több helyen nem fordul le. Javítsd minimális beavatkozással, és minden javításhoz nevezd meg a szabályt!

```
template <class C>
struct Holder : C {
    C::value_type get() const { return C::get_value(); }
    template <class U> static int code() { return sizeof(U); }
};

template <class C>
int run(const Holder<C>& h) {
    auto v = h.get();
    (void)v;
    return Holder<C>::code<int>();
}
```

3. feladat (diagnosztika/tervezés). Adott egy nem másolható, nem mozgatható `Resource` osztály, amelynek van egy nem-tag `swap`-je a saját névterében. Írj generikus `exchange` sablont, amely ezt helyesen használja, majd magyarázd meg, miért nem fordul le ugyanez `std::swap(a, b)` minősített hívással. Adj ellenőrzési feltételt!

26.5.10 Megoldások

1. megoldás. Az A nem fordul le. A `pick(t)` függő hívás; a közönséges lookup a definíció helyén nem talál semmit, mert a `pick(int)` később van deklarálva. A példányosításkor az ADL fut le, de `T = int`-hez nem tartozik asszociált névtér, így nincs jelölt. A B lefordul: a `pick(t)` hívás argumentuma `n::X`, tehát az asszociált névtér az `n`, és a példányosítás idejére az `n::pick(X)` már látható. Ellenőrzés: az A-ra a fordító hibát ad, a B-re nem; a B várható viselkedése a hívás sikeres feloldása.

2. megoldás. Négy javítás kell. (a) `C::value_type` helyett `typename C::value_type`, mert függő minősített név típuspozícióban. (b) A `C::get_value()` hívásnál a függő bázis tagja nem látszik; `this->get_value()` (vagy `C::get_value()`) kell. (c) `Holder<C>::code<int>()` helyett `Holder<C>::template code<int>()`, mert függő típus tagtemplate-jét paraméterezzük. (d) A `get()` visszatérési típusa szintén `typename C::value_type`. A javított mag:

```
template <class C>
struct Holder : C {
    typename C::value_type get() const { return this->get_value(); }
    template <class U> static int code() { return static_cast<int>(sizeof(U)); }
};

template <class C>
int run(const Holder<C>& h) {
    auto v = h.get();
    (void)v;
    return Holder<C>::template code<int>();
}
```

Ellenőrzés: a `typename` törlése, illetve a `template` elhagyása újra hibát okoz, tehát mindkét kulcsszó tényleg szükséges; a `this->` törlése szintén.

3. megoldás. A `Resource` nem mozgatható, ezért a `std::swap` sablon nem alkalmazható rá; a saját `swap` ADL-lel érhető el.

```
// ch26_ex3.cpp
#include <cstdio>
#include <utility>

namespace res {
class Resource {
public:
    explicit Resource(int id) noexcept : id_(id) {}
    Resource(const Resource&) = delete;
    Resource& operator=(const Resource&) = delete;
    int id() const noexcept { return id_; }
    friend void swap(Resource& a, Resource& b) noexcept {
        std::swap(a.id_, b.id_);
    }
```

```
private:
    int id_;
};
}  // namespace res

template <class T>
void exchange(T& a, T& b) {
    using std::swap;   // generikus tartalék a definíció helyén
    swap(a, b);        // ADL a példányosítás helyén: res::swap
}

int main() {
    res::Resource a{1}, b{2};
    exchange(a, b);
    std::printf("%d %d\n", a.id(), b.id());   // várt: 2 1
}
```

```
g++ -std=c++20 -Wall -Wextra -pedantic-errors -o ch26_ex3 ch26_ex3.cpp
```

A `std::swap(a, b)` minősített változat azért hibás, mert minősített névnél nincs ADL: csak a `std::swap` sablon jön szóba, amely C++20-ban a move-követelmények miatt kikerül a jelöltek közül, illetve C++17-ben példányosításkor hibázna. A `swap` a `res` névtéren kívül nem található meg, ezért a névtér-elhelyezés is szerződéselem. Ellenőrzési feltétel: a fenti program lefordul és `2 1`-et ír ki; ha a `swap`-et kivisszük a globális névtérbe, a kód továbbra is működik (a globális névtér is asszociált), de ha a `res` névtéren kívül egy *másik* névtérbe tesszük, a fordítás megszűnik.

26.6 Generikus interfész és sablondiagnosztikai labor

Ez az alfejezet egy labor. A célja nem új nyelvi elem bevezetése, hanem annak begyakorlása, hogy egy generikus felület viselkedését és a fordítási hibáit a szerződés szintjén tudd leírni. A korábbi részekben rögzítettük a template-instantiation, a dependent name, a kétfázisú lookup és az ADL szabályait Dependent name, kétfázisú lookup és ADL (26.5), valamint a `noexcept` határok és a hibapolicy kérdését noexcept határok és hibapolicy (25.3). Most ezeket egyetlen, kicsi, de életszerű kódpáron használjuk: egy fix kapacitású, veremtárolón élő pufferen (`SmallBuffer`) és egy általános `transform` segéden, amely egy bemeneti tartomány elemeit egy kimeneti tartományba képezi. A labor két kérdésre keresi a választ: mit követelünk meg a típus- és méretparamétertől, és hogyan feltételezzük a hívási szándékot, hogy elkerüljük a véletlen másolatokat.

A hangsúly a diagnosztikán van. Egy template-instantiation hibaüzenete első olvasásra ijesztő, de a legtöbbször három dolog egyikére vezethető vissza: a típusparaméter nem teljesíti a szerződést, a hívási kategória nem az, amit a szerző feltételezett, vagy a névfeloldás egy nem kívánt overloadot választott. A laborfeladat az, hogy mindhárom esetre legyen egy minimalizált, reprodukálható példád, és hogy a nyers hibaüzenetet API-feltétellé alakítsd.

26.6.1 A két komponens és a szerződés

A `SmallBuffer<T, N>` szándékosan szűk: fix, fordítási időben ismert kapacitás, nincs halomallokáció, az elemek a `SmallBuffer` objektumon belül élnek. A típusparaméter-szerződés a következő.

1. `N > 0`, mert nulla kapacitású inline tömb nem értelmes.

2. `T` legyen default-inicializálható (`std::default_initializable<T>`), mert a `T data_[N]{}` tömb mind az `N` elemet létrehozza, már a konstrukciókor.
3. `T` legyen `const T&`-ről értékadható (`std::assignable_from<T&, const T&>`), mert a `push_back` a már létező elemre értékadással ír.

Ennél többet a `SmallBuffer` nem igényel: nyers bájtmásolást nem végez, ezért a triviális másolhatóság nem feltétel. Ez fontos, mert a labor mérőtípusa, a `Counted`, szándékosan nem triviálisan másolható: a másoló és mozgató műveletei számlálnak, és éppen ezek mérése a labor célja. A három feltételt a `requires` klóz rögzíti, így megsértésüket a fordító a példányosítás helyén jelzi.

A `transform` ezzel szemben csak azt követeli meg, hogy a bemenet elemei olvashatók legyenek, a kimenetnek legyen `push_back` művelete, a callable pedig legyen hívható a bemenet elemére. Ez utóbbi feltételt a C++20 `requires`-szel fejezzük ki, nem dokumentációval.

```
// include/small_buffer.hpp
#ifndef SMALL_BUFFER_HPP
#define SMALL_BUFFER_HPP

#include <concepts>
#include <cstddef>
#include <stdexcept>
#include <type_traits>
#include <utility>

template <typename T, std::size_t N>
    requires (N > 0) && std::default_initializable<T>
             && std::assignable_from<T&, const T&>
class SmallBuffer {
public:
    using value_type = T;
    using size_type  = std::size_t;

    static constexpr size_type capacity() noexcept { return N; }

    size_type size()  const noexcept { return size_; }
    bool      empty() const noexcept { return size_ == 0; }

    T&       operator[](size_type i)       noexcept { return data_[i]; }
    const T& operator[](size_type i) const noexcept { return data_[i]; }

    void push_back(const T& v) {
        if (size_ == N) throw std::length_error{"SmallBuffer full"};
        data_[size_++] = v;
    }

    void clear() noexcept { size_ = 0; }

    T*       begin()       noexcept { return data_; }
    T*       end()         noexcept { return data_ + size_; }
    const T* begin() const noexcept { return data_; }
    const T* end()   const noexcept { return data_ + size_; }

private:
```

```
    T         data_[N]{};
    size_type size_ = 0;
};

#endif
```

A `push_back` kapacitáshiba esetén `std::length_error`-t dob, tehát nem `noexcept`. A `transform` sem lehet `noexcept`, hiszen továbbadja ezt a kivételt; ezt a hibapolicy-szerződés részeként kell dokumentálni, összhangban noexcept határok és hibapolicy (25.3)-gyel.

```
// include/transform.hpp
#ifndef TRANSFORM_HPP
#define TRANSFORM_HPP

#include <functional>
#include <utility>

// A callable-t a ciklusban többször hívjuk, ezért mindig lvalue-ként (fn),
// nem std::forward<F>(fn)-ként: egy rvalue-ra minősített operator() így nem
// "fogyaszthatja el" az állapotát az első hívásnál. A megszorítás pontosan
// ugyanazt az invoke-kifejezést ellenőrzi, amelyet a törzs használ.
template <typename In, typename Out, typename F>
    requires requires (const In& in, Out& out, F& fn) {
        in.begin(); in.end();
        std::invoke(fn, *in.begin());                  // hívható az elemmel?
        out.push_back(std::invoke(fn, *in.begin()));   // a kimenet fogadja?
    }
void generic_transform(const In& in, Out& out, F&& fn) {
    for (const auto& x : in) {
        out.push_back(std::invoke(fn, x));
    }
}

#endif
```

A bemenet `const In&`, mert csak olvassuk; ez nem deduction-, hanem interfész-döntés. A callable `F&&` forwarding reference, mert így semmilyen callable nem másolódik az átvételkor, legyen az lambda, állapotot hordozó funktor vagy tagadat-pointer. A törzsben viszont a callable-t **lvalue-ként** (`fn`) hívjuk, mert a ciklus többször hívja; a `std::forward<F>(fn)` ismételt használata egy `&&`-minősített `operator()` esetén már az első hívásnál „elfogyaszthatná” az állapotot. A megszorítás ugyanazt az `std::invoke(fn, ...)` kifejezést ellenőrzi ugyanazzal az értékkategóriával (`F& fn`, `const` elem), amit a törzs használ, két külön követelményként: hívható-e a callable az elemmel, és fogadja-e a kimenet az eredményt. Így a megszorítás és a ciklustörzs ugyanazt a felületet modellezi, és a `std::invoke` miatt tagadat-pointeres projekció (`&Counted::v`) is működik.

26.6.2 A const/lvalue/rvalue argumentum mátrix

A forwarding reference lényege, hogy a dedukált `T` nem a paraméter típusa, hanem egy „nyers” típus, amelyből a referencia-kollapszus szabálya állítja elő a tényleges paramétert. A laborban ezt a következő táblázattal rögzítjük; a séma `template<class T> void f(T&&)`.

Hívás	Dedukált `T`	Paraméter típusa
`int x; f(x);`	`int&`	`int&`
`f(7);`	`int`	`int&&`
`const int cx; f(cx);`	`const int&`	`const int&`
`f(std::move(cx));`	`const int`	`const int&&`
`f("lit");`	`const char (&)[4]`	`const char (&)[4]`

Három következményt érdemes kiemelni. Először, `T` lvalue esetén referenciává válik, tehát a `T&&` alak nem jelenti azt, hogy a hívó csak rvalue-t adhat át. Másodszor, a `const` minősítő a dedukált típusba kerül, és nem tűnik el: `f(cx)` esetén `T = const int&`. Harmadik, a `std::move(cx)` nem rvalue-vé alakítja a `cx`-et, csak `const int&&`-t ad, tehát a `transform` nem tudja „elrabolni” a tartalmat; ez a klasszikus `const`-move csapda.

A `generic_transform` hívásakor a mátrix két helyen játszik: a bemeneti és kimeneti argumentumnál, valamint a callable-nél. A `const In&` alak miatt a bemenet lvalue marad. A callable-nél a dedukció a hívó szándékát rögzíti (lambda-temporaryra `F` a lambdatípus, lvalue funktorra `F` lvalue-referencia), de a törzs a callable-t szándékosan lvalue-ként hívja, mert többször használja; a `std::forward<F>(fn)` csak egyetlen, utolsó használatnál lenne indokolt. Ha `std::ref`-fel burkolt funktort ad, a `reference_wrapper` másolása olcsó.

26.6.3 Forwarding és accidental copy számlálás

A leglátványosabb hiba nem fordítási, hanem teljesítménybeli: a helper véletlenül másol. Ezt egy számláló típussal tesszük láthatóvá.

```
// include/counted.hpp
#ifndef COUNTED_HPP
#define COUNTED_HPP
#include <cstddef>

struct Counted {
    static inline std::size_t copies = 0;
    static inline std::size_t moves  = 0;

    int v{};

    Counted() = default;
    explicit Counted(int x) : v{x} {}
    Counted(const Counted& o) : v{o.v} { ++copies; }
    Counted(Counted&& o) noexcept : v{o.v} { ++moves; }
    Counted& operator=(const Counted& o) { v = o.v; ++copies; return *this; }
    Counted& operator=(Counted&& o) noexcept { v = o.v; ++moves; return *this; }
    ~Counted() = default;
};
#endif
```

A driver két variánst hasonlít össze: egy `transform_by_value` segédet, amely a callable-t érték szerint veszi át, és a `generic_transform`-et, amely forwarding referenciát használ.

```
// src/main.cpp
#include "small_buffer.hpp"
```

```cpp
#include "transform.hpp"
#include "counted.hpp"
#include <cstdio>
#include <utility>

template <typename In, typename Out, typename F>
void transform_by_value(const In& in, Out& out, F fn) {
    for (const auto& x : in) out.push_back(fn(x));
}

// Külön instrumentált callable: a saját számlálója csak a funktor
// másolásait/mozgatásait méri, a Counted-számlálótól függetlenül.
struct CountingFn {
    static inline std::size_t copies = 0;
    static inline std::size_t moves  = 0;
    CountingFn() = default;
    CountingFn(const CountingFn&) { ++copies; }
    CountingFn(CountingFn&&) noexcept { ++moves; }
    Counted operator()(const Counted& c) const { return Counted{c.v * 2}; }
};

int main() {
    SmallBuffer<Counted, 8> in;
    SmallBuffer<Counted, 8> out;
    for (int i = 0; i < 4; ++i) in.push_back(Counted{i});
    out.clear();

    auto twice = [](const Counted& c) { return Counted{c.v * 2}; };

    // 1. mérés: elemek másolása (Counted-számláló), állapot nélküli lambda.
    Counted::copies = Counted::moves = 0;
    transform_by_value(in, out, twice);
    std::printf("by_value:  copies=%zu moves=%zu size=%zu\n",
                Counted::copies, Counted::moves, out.size());

    out.clear();
    Counted::copies = Counted::moves = 0;
    generic_transform(in, out, twice);
    std::printf("forwarding: copies=%zu moves=%zu size=%zu\n",
                Counted::copies, Counted::moves, out.size());

    // 2. mérés: a callable másolása (CountingFn-számláló).
    CountingFn fn;
    out.clear();
    CountingFn::copies = CountingFn::moves = 0;
    transform_by_value(in, out, fn);
    std::printf("fn by_value:   copies=%zu moves=%zu\n",
                CountingFn::copies, CountingFn::moves);

    out.clear();
    CountingFn::copies = CountingFn::moves = 0;
    generic_transform(in, out, fn);
```

```
    std::printf("fn forwarding: copies=%zu moves=%zu\n",
                CountingFn::copies, CountingFn::moves);

    // Tagadat-pointer mint projekció: std::invoke(&Counted::v, x) == x.v
    SmallBuffer<int, 8> ids;
    generic_transform(in, ids, &Counted::v);
    std::printf("projection: size=%zu last=%d\n", ids.size(), ids[ids.size() -
     ↪  1]);
}
```

A fordítás és a futtatás külön laborlépés:

```
mkdir -p build
g++ -std=c++20 -Wall -Wextra -Wpedantic -Iinclude \
    src/main.cpp src/reader_a.cpp src/reader_b.cpp src/small_buffer_inst.cpp \
    -o build/lab
./build/lab
```

A labor teljes fájlleltára: `include/small_buffer.hpp`, `include/transform.hpp`, `include/counted.hpp`, `include/small_buffer_extern.hpp`, `src/main.cpp`, `src/reader_a.cpp`, `src/reader_b.cpp`, `src/small_buffer_inst.cpp`, valamint a negatív tesztek a `test/` könyvtárban (`neg_small_buffer.cpp`, `neg_callable.cpp`).

Buildjegyzőkönyv (`lab-gcc` profil). A labor figyelmeztetés nélkül fordult; a kimenet:

```
by_value:  copies=4 moves=0 size=4
forwarding: copies=4 moves=0 size=4
fn by_value:   copies=1 moves=0
fn forwarding: copies=0 moves=0
projection: size=4 last=3
```

A két mérés két külön számlálót olvas. **Az elemek másolása** (`Counted`-számláló): mindkét változatban 4, és ez a várt, elfogadott eredmény. A `twice` lambda állapot nélküli, ezért az érték szerinti átvétele egyetlen `Counted`-et sem másol; a négy másolás a négy kimeneti elem `push_back`-értékadásából jön (`data_[size_++] = v`, ahol `v const Counted&`). **A callable másolása** (`CountingFn`-számláló): az érték szerinti `transform_by_value` lvalue `fn`-ből pontosan egy másolatot készít (`copies=1`), a forwarding-referenciás `generic_transform` egyet sem. A kvalitatív elvárás tehát: a callable-másolás a `CountingFn` számlálóján jelenik meg, és nem keveredik az elemek másolásával. Ha a lambda érték szerint elkapott `Counted`-et hordozna, az egy callable-másolás a `Counted`-számlálón is +1-ként jelentkezne a négy elemmásolás mellett — ezért mérünk külön funktorral.

A negatív tesztek (`test/neg_callable.cpp`, `-DNEG_NOT_CALLABLE`, illetve `-DNEG_NO_PUSH_BACK`) mindkettő `constraints not satisfied` hibával bukik. Nem hívható objektumnál a jegyzet az első követelményt nevezi meg (`the required expression 'std::invoke(fn, (* in.begin()))' is invalid`), `push_back` nélküli kimenetnél csak a másodikat (`'out.push_back(std::invoke(fn, (* in.begin())))' is invalid`).

```
// test/neg_callable.cpp  -- szándékosan nem fordul
#include "small_buffer.hpp"
#include "transform.hpp"

struct NoPushBack { };
struct NotCallable { };
```

```
int main() {
    SmallBuffer<int, 4> in;
    SmallBuffer<int, 4> out;
#if defined(NEG_NOT_CALLABLE)
    generic_transform(in, out, NotCallable{});
#elif defined(NEG_NO_PUSH_BACK)
    NoPushBack sink;
    generic_transform(in, sink, [](int x) { return x; });
#endif
}
```

Érdemes megfigyelni, hogy a `generic_transform` belsejében a `for (const auto& x : in)` miatt a callable argumentuma `const Counted&`, nem másolat. Ez a `push_back` által végzett másolást nem spórolja meg, mert a `SmallBuffer::push_back` a kimenetbe ír. Ha a `push_back`-ot `emplace_back`-kel és a callable visszatérési értékének közvetlen átadásával váltanánk ki, a konstruktor egy lépésben futna le.

26.6.4 Header több translation unitban

A template definíciójának a headerekben kell élnie, mert az instantiation a fordító dolga, nem a linkeré. Ha a `transform.hpp`-t két `.cpp` is beemeli, mindkét fordítási egységben példányosodnak ugyanazok a sablonok. Ez nem ODR-sértés, mert a template-ek (és az osztályon belül definiált tagfüggvények) implicit `inline` jelölésűek, és a szabvány megengedi, hogy több fordítási egységben azonos definíció szerepeljen.

```
// src/reader_a.cpp
#include "transform.hpp"
#include "small_buffer.hpp"
#include "counted.hpp"

void run_a(SmallBuffer<Counted, 8>& in, SmallBuffer<Counted, 8>& out) {
    generic_transform(in, out, [](const Counted& c) { return Counted{c.v + 1};
 ↪  });
}

// src/reader_b.cpp
#include "transform.hpp"
#include "small_buffer.hpp"
#include "counted.hpp"

void run_b(SmallBuffer<Counted, 8>& in, SmallBuffer<Counted, 8>& out) {
    generic_transform(in, out, [](const Counted& c) { return Counted{c.v - 1};
 ↪  });
}
```

A két `generic_transform` különböző `F` típusokkal példányosul, tehát két külön szimbólum keletkezik. Ugyanarra az `F`-re viszont ugyanaz a weak szimbólum jön létre mindkét egységben, és a linker egyet választ. Ezt a `nm -C build/reader_a.o` kimenetén ellenőrizheted (a `reader_a.o` a `g++ -std=c++20 -Iinclude -c src/reader_a.cpp -o build/reader_a.o` paranccsal készül). A `lab-gcc` profilban a `SmallBuffer<Counted, 8ul>` tagfüggvényei, például a `push_back`, `W` (weak) jelölést kapnak. A `generic_transform` példánya viszont `t` (lokális): a lambda típusa a

fordítási egységhez kötött, ezért az ilyen `F`-fel készült példány nem is lehet közös a két egység között.

26.6.5 Explicit instantiation összevetése

Nagy projektekben a sok azonos instantiation lassítja a fordítást. A `template class SmallBuffer<Counted, 8>;` explicit instantiation egy fordítási egységben kényszeríti a példányosítást, az `extern template class SmallBuffer<Counted, 8>;` pedig a többi egységben elnyomja az implicit példányosítást.

```
// src/small_buffer_inst.cpp
#include "small_buffer.hpp"
#include "counted.hpp"

template class SmallBuffer<Counted, 8>;   // kényszerített példány
```

```
// include/small_buffer_extern.hpp
#pragma once
#include "small_buffer.hpp"
#include "counted.hpp"

extern template class SmallBuffer<Counted, 8>;  // máshol példányosul
```

A trade-off három ponton mérhető: fordítási idő, objektumfájl mérete, linkelési idő. Az explicit instantiation akkor nyer, ha ugyanazt a sablont sok fordítási egység használja, és a `T`/`N` kombinációk zárt halmaza ismert. Hátránya, hogy a felhasználónak gondoskodnia kell arról, hogy az `extern template` deklaráció minden olyan egységben látható legyen, ahol az implicit példányosítást el akarja nyomni; ha kimarad, a példányosítás visszatér, és nem kapsz hibát, csak lassabb buildet.

26.6.6 Negatív fordítási tesztek és olvasható diagnosztika

A negatív fordítási teszt egy olyan forrás, amelynek szándékosan nem szabad lefordulnia, és amelynek a hibaüzenetét ellenőrizzük. A cél az, hogy a hibaüzenet egy `static_assert` vagy `requires` feltételre vezethető legyen vissza, ne egy mélyen beágyazott STL-instantiation láncra.

A `SmallBuffer` szerződéséhez két negatív típus tartozik, egy-egy megsértett feltétellel:

```
// test/neg_small_buffer.cpp  -- szándékosan nem fordul
#include "small_buffer.hpp"

struct NoDefault {                  // nincs default konstruktor
    explicit NoDefault(int) {}
};

struct ConstMember {                // const tag: nem értékadható
    const int v = 0;
};

#ifdef NEG_ASSIGN
using Bad = SmallBuffer<ConstMember, 4>;
#else
using Bad = SmallBuffer<NoDefault, 4>;
```

```
#endif

int main() {
    Bad b;
    (void)b;
}
```

A `requires` klóz miatt a fordító a hibaüzenetben a `SmallBuffer` deklarációját és a klózt nevezi meg, alatta pedig a ténylegesen megsértett feltételt; nem az STL belsejére mutat. Mivel a klóz az osztálysablon deklarációján áll, egy osztályon belüli `static_assert` meg sem jelenne: a példányosítás már a klóz ellenőrzésén elbukik. Saját szövegű üzenethez a feltételt nevesített conceptbe lehet kiemelni.

A teszt futtatása shell-szintű; mindkét változatnak meg kell buknia, és mindkettőnek a saját okával:

```
mkdir -p build
if g++ -std=c++20 -Iinclude -fsyntax-only test/neg_small_buffer.cpp
 ↪  2>build/diag1.txt; then
  echo "FAIL: expected compile error (NoDefault)"; exit 1
fi
grep -q "_Tp = NoDefault" build/diag1.txt

if g++ -std=c++20 -Iinclude -DNEG_ASSIGN -fsyntax-only test/neg_small_buffer.cpp
 ↪  2>build/diag2.txt; then
  echo "FAIL: expected compile error (ConstMember)"; exit 1
fi
grep -q "__lhs = static_cast" build/diag2.txt
```

Buildjegyzőkönyv (`lab-gcc` profil). Mindkét változat `template constraint failure` hibával bukott. A `NoDefault` esetén a jegyzet a `default_initializable` alatti `is_constructible_v<_Tp, _Args ...> [with _Tp = NoDefault; _Args = {}]` kifejezést nevezi meg, a `ConstMember` esetén az `assignable_from __lhs = static_cast<_Rhs&&>(__rhs)` követelményét. A `grep` minták a libstdc++ conceptjeinek szövegéhez kötöttek; más fordítóhoz igazítani kell őket.

Egy másik tipikus negatív eset a forwardoló segédnél lép fel: ha a `generic_transform`-et olyan típussal hívjuk, amelynek nincs `push_back` művelete, a `requires` feltétel hamis lesz, és a hiba a hívás helyére mutat. Ha viszont a `requires`-t elhagynánk, a hiba a `transform.hpp` belsejébe esne, és a hívó számára követhetetlen lenne. A `requires` tehát nem esztétikai, hanem diagnosztikai eszköz.

26.6.7 Tipikus hibák és javításuk

A leggyakoribb minta, hogy a szerző `T`-t érték szerint veszi át a callable-nél (`F fn`), és ezzel minden híváskor másol. A javítás `F&&` átvétel; a hívás a törzsben `std::invoke(fn, ...)`, lvalue-ként, ha a callable-t többször hívjuk. A `std::forward<F>(fn)` csak az utolsó (egyetlen) használatnál helyes.

A második minta a `const T&&` félreértése: `std::move(cx)` után a `T const int`, tehát a `T&&` paraméter nem teszi lehetővé az elrablást. A javítás: ha tényleg mozgatni akarunk, a `const`-ot már a hívó oldalon el kell hagyni.

A harmadik minta az explicit instantiation és az `extern template` sorrendjének felcserélése: ha az `extern template` deklaráció után következik egy implicit használat, a fordító a deklarációt figyelmen kívül hagyhatja, és a példányosítás újra megtörténik. Az `extern template` deklarációnak minden olyan egységben láthatónak kell lennie, ahol a példányosítást el akarod nyomni.

26.6.8 Összefoglaló

A labor két dolgot mutatott meg. Először, a generikus interfész szerződése több szinten él: a `SmallBuffer`-nél a típus- és méretparaméterre vonatkozó `requires`/`static_assert`, a `generic_transform`-nél a bemenet és a kimenet műveleteire vonatkozó `requires`. Másodszor, a hívási kategória és a forwarding nem stílus kérdése: a `Counted` számlálóval láthatóvá tehető, hol keletkezik felesleges másolat, és a mátrix alapján megjósolható, mit fog tenni a `std::forward`. A header Több TU-ban élő template-jei ODR-biztosak, az explicit instantiation építési időért cserébe optimalizál, a negatív fordítási teszt pedig akkor jó, ha a hibaüzenet a saját szerződésünkre mutat.

26.6.9 Feladatok

1. feladat (magyarázó). Egészítsd ki a const/lvalue/rvalue mátrixot a `generic_transform` hívására: `SmallBuffer<Counted,8> buf; generic_transform(buf, out, [](auto x){ return x; });` esetén mi lesz `In`, `Out`, `F` dedukált típusa, és miért hívja a törzs a callable-t lvalue-ként, nem `std::forward<F>(fn)`-nel? Indokold a választ a referencia-kollapszus szabályával.

2. feladat (kódolási). Módosítsd a `generic_transform`-et úgy, hogy opcionálisan egy `reserve`-szerű előkészítést is végezzen a kimeneten, ha az támogatja a `reserve` műveletet, egyébként ne tegyen semmit. A megoldás legyen C++20-kompatibilis, és ne regresszálja a meglévő `requires` feltételt.

3. feladat (diagnosztikai). Írj negatív fordítási tesztet arra az esetre, amikor a callable visszatérési típusa nem konvertálható a kimenet `value_type`-jára, és fogalmazd meg a `requires` feltételt úgy, hogy a hibaüzenet a `transform.hpp`-ra és a hívásra mutasson, ne az STL belsejére. Add meg az ellenőrző shell-parancs vázát is.

26.6.10 Megoldások

1. megoldás. A `buf` lvalue `SmallBuffer<Counted,8>`, a `const In&` paraméterhez `In = SmallBuffer<Counted,8>`. Az `out` szintén lvalue, `Out = SmallBuffer<Counted,8>`. A lambda-kifejezés prvalue, tehát `F` a lambda típusa (nem referencia), az `fn` paraméter típusa `F&&`. A `std::forward<F>(fn)` ezt xvalue-ként adná tovább; a ciklusban azonban a callable-t négyszer hívjuk, és egy `&&`-minősített hívóoperátor az első hívásnál elhasználhatná az állapotát. Ezért a törzs `std::invoke(fn, x)`-et hív, ahol `fn` lvalue, és a megszorítás is `F& fn`-nel ellenőriz. A `const In&` miatt a bemenet nem módosítható, és a `for (const auto& x : in)` miatt az `x` is `const Counted&`; az `auto x` paraméter ezt lemásolja, tehát ez a feladat szándékosan bemutat egy accidental copy-t, amit `auto const&`-re cserélve lehet megszüntetni.

2. megoldás. A `requires` feltételt bővíteni kell egy opcionális ággal, de a `reserve` hiánya ne tegye hamissá a feltételt. C++20-ban erre a `requires` feltétel nem alkalmas közvetlenül; egy külön `if constexpr (requires { out.reserve(0); })` blokk a függvénytestben oldja meg:

```
template <typename In, typename Out, typename F>
    requires requires (const In& in, Out& out, F& fn) {
        in.begin(); in.end();
        std::invoke(fn, *in.begin());
        out.push_back(std::invoke(fn, *in.begin()));
    }
void generic_transform(const In& in, Out& out, F&& fn) {
    if constexpr (requires { out.reserve(in.size()); }) {
        out.reserve(in.size());
    }
    for (const auto& x : in) {
        out.push_back(std::invoke(fn, x));
    }
}
```

Az `if constexpr` ága csak akkor példányosul, ha a `reserve` hívás jól formált; ellenkező esetben a blokk eldobásra kerül, és nem okoz hibát. Ellenőrzési feltétel: a `SmallBuffer` (amelynek nincs `reserve`-je) változatlanul fordul, és a `std::vector`-ral hívott változat is.

3. megoldás. A `requires` feltételbe be kell venni a visszatérési típus konvertálhatóságát. A kimenet típusát a `decltype(*out.begin())` adja meg:

```
template <typename In, typename Out, typename F>
    requires requires (const In& in, Out& out, F& fn) {
        in.begin(); in.end();
        requires std::convertible_to<
            std::invoke_result_t<F&, decltype(*in.begin())>,
            std::remove_cvref_t<decltype(*out.begin())>>;
        out.push_back(std::invoke(fn, *in.begin()));
    }
void generic_transform(const In& in, Out& out, F&& fn);
```

A negatív tesztnél a callable egy olyan típust ad vissza, amelyből nincs konverzió a kimenet `value_type`-jára. Az ellenőrző parancs:

```
if g++ -std=c++20 -Iinclude -fsyntax-only test/neg_transform.cpp
 ↪  2>build/diag.txt; then
  echo "FAIL"; exit 1
fi
grep -q "constraints not satisfied" build/diag.txt
```

A `grep` minta fordítófüggő; a lényeg, hogy a diagnosztika a `generic_transform` deklarációjára és a hívás helyére mutasson.

27 Concepts, type traits és compile-time generikusság

27.1 Type traits, tag dispatch és if constexpr

A generikus C++ kód egyik alapproblémája, hogy ugyanaz az algoritmus típusonként mást jelent: egy 32 bites egész számot fix byte-sorrenddel kell kiírni, egy `std::string`-et hosszprefixed bájtokkal, egy nyers structot pedig csak akkor szabad memóriaként másolni, ha a típus erre valóban alkalmas. A döntést ideális esetben nem futásidőben, hanem fordításkor hozzuk meg, hogy a nem támogatott eset már a build során, érthető üzenettel bukjon el. A type trait, a tag dispatch és az `if constexpr` ennek a compile-time ágválasztásnak a három egymást kiegészítő eszköze. Az alábbiakban azt mutatjuk meg, hogyan lehet egy típus tulajdonságából dokumentált implementációt választani, és közben tisztán látni, hogy a trait mit garantál és mit nem.

A korábbi Generikus interfész és sablondiagnosztikai labor (26.6) részben rögzítettük a sablonok példányosításának és a hibaüzenetek olvasásának alapjait; itt ezekre építünk. Ahol a tárolók élettartamára vagy újrafelhasználására hivatkozunk, a Tároló újrafelhasználása és C++ objektumszabályok (20.6) rész definícióit használjuk. A fejezet a C++20 alapot követi; ahol egy képesség újabb, azt külön jelöljük. A megadott forrásjegyzet ehhez a részhez nem tartalmazott ellenőrzött hivatkozást, ezért konkrét ISO-paragrafusra vagy hivatkozható URL-re itt nem támaszkodunk; a megfogalmazás a C++20 nyelvi és könyvtári szerződések könyvbeli összefoglalása, nem független forrásellenőrzés.

27.1.1 Type trait: típusból levezetett compile-time állítás

A **type trait** olyan sablon, amelynek a paramétere egy (vagy több) típus, az eredménye pedig egy fordítási időben ismert érték vagy egy másik típus. A predikátum alakú trait-ek a `std::integral_constant<bool, B>` specializációi, ahol `B` a kérdéses tulajdonság. Ennek a mintának két fontos következménye van:

- a trait *típus*, tehát overload feloldásban címkeként használható, és
- a trait *értéke* `::value`-ként érhető el, amely `static constexpr` tag, így `if constexpr`, `static_assert` és nem típusú sablonparaméter feltételében is állandó kifejezés.

A `std::integral_constant` C++11 óta létezik; a `std::bool_constant<B>` ennek a `bool` specializációja, C++17 óta. A rövidített alakok: a `_v` változat az értéket adja (`std::is_integral_v<T>`), a `_t` változat a transzformált típust (`std::remove_cvref_t<T>`). A `_v` és `_t` alakok C++17, illetve C++14 óta érhetők el, a `remove_cvref` pedig C++20-as.

A transzformáló trait-ek közül kettőt érdemes szembeállítani. A `std::remove_cvref_t<T>` a legfelső szintű `const`/`volatile` minősítőt és a referenciát távolítja el: `remove_cvref_t<const int&>` értéke `int`. A `std::decay_t<T>` ennél agresszívebb: a fentieken kívül a tömböt pointerré, a függvényt függvénypointerré alakítja, és a `const int`-ből `int` lesz. A `decay` tehát a függvényparaméter-átadás “romlását” modellezi, a `remove_cvref` viszont megőrzi az eredeti kategoriát, és ezért generic kódban gyakrabban ez a helyes választás. Ha a cél a típus azonosságának vizsgálata, a `decay` csendesen más típusra vált, és ez hibás ágválasztáshoz vezet.

A gyakran használt trait-családok röviden:

Család	Példa	Jelentése
Elsődleges kategória	`is_integral_v`, `is_floating_point_v`, `is_pointer_v`, `is_array_v`	a típus nyelvi kategóriája
Összetett tulajdonság	`is_trivially_copyable_v`, `is_standard_layout_v`, `is_nothrow_destructible_v`	a típus műveleteinek garantált tulajdonsága
Transzformáció	`remove_cvref_t`, `decay_t`, `add_pointer_t`, `underlying_type_t`	új típus előállítása
Lekérdezés	`is_same_v`, `is_convertible_v`, `is_invocable_v`	típusok közötti viszony

A trait tehát nem "varázslat": a szabvány által definiált, fordításidőben kiértékelt predikátum, amelynek az igazsága a típus deklarációjából következik.

27.1.2 Mit garantál egy trait, és mit nem

A legfontosabb fogalmi elkülönítés a **szintaktikai képesség** és a **szemantikai szerződés** között. A trait az előbbiről beszél: arról, hogy a típus műveletei milyen tulajdonságúak a nyelv szabályai szerint. Nem beszél arról, hogy a típus értékei milyen üzleti invariánst hordoznak, hogy két objektum byte-azonos reprezentációja ugyanazt jelenti-e, vagy hogy a másolat egy másik gépen is értelmes lesz.

A `std::is_trivially_copyable_v<T>` például azt garantálja, hogy a típus másoló/mozgató konstruktora és értékadó operátora triviális, és a destruktora is triviális. Ebből az következik, hogy az objektum reprezentációja byte-onként átmásolható egy másik, azonos típusú objektumba, és az eredmény ugyanaz az érték lesz. Ez azonban nem jelenti azt, hogy a byte-sorozat *hordozható*: a padding bájtok tartalma nem specifikált, a többbyte-es egészek byte-sorrendje implementációfüggő, a pointerek pedig folyamat- és futásidőfüggők. Egy `struct Header { std::uint16_t type; std::uint32_t len; }` tipikusan trivially copyable, de a `sizeof(Header)` értéke tartalmazhat paddinget, így a nyers memóriakép hossza és tartalma nem szerződés. A trait igaz, a hálózati szerializáció mégsem helyes rá.

A `std::is_standard_layout_v<T>` azt jelzi, hogy a típus megfelel a C-kompatibilis elrendezés bizonyos nyelvi feltételeinek (például minden nem statikus adattag ugyanabban az osztályban van, nincs virtuális függvény és virtuális bázis, és a hozzáférés-vezérlés egységes). Ez hasznos előfeltétele az `offsetof` használatának és a C ABI-val való együttműködésnek, de nem garantálja, hogy két különböző fordító vagy két architektúra ugyanazt a layoutot állítja elő. A standard layout tehát *nyelvi* tulajdonság, nem ABI-szerződés.

A destruktibilitásnál három kérdést kell szétválasztani. Az `is_destructible_v<T>` azt kérdezi, hogy a `t.~T()` kifejezés jól formált-e. Az `is_trivially_destructible_v<T>` azt, hogy a destruktor triviális, tehát nem kell meghívni a tároló felszabadításakor. Az `is_nothrow_destructible_v<T>` pedig azt, hogy a destruktor nem dob kivételt. Egy típus lehet destruktibilis, de nem triviálisan az, és lehet triviálisan destruktibilis, de a gyakorlatban mégis drága a használata. A tároló újrafelhasználásánál ez azért számít, mert a triviális destruktorból a fordító tudja, hogy a nyers tároló felszabadítása előtt nincs teendő; ez a Tároló

újrafelhasználása és C++ objektumszabályok (20.6) részben tárgyalt objektumszabályok egyik gyakorlati következménye.

Végül a trait soha nem bizonyít futásidejű helyességet. Attól, hogy `is_integral_v<T>` igaz, még nem tudjuk, hogy a szám a protokoll által előírt tartományban van-e. A trait a *típusról* szól, nem az *értékről*.

27.1.3 Tag dispatch: címkével választott túlterhelés

A **tag dispatch** (címkealapú szétválasztás) a trait értékét egy üres címketípusba csomagolja, és a választást a függvény-túlterhelésre bízza. A `std::true_type` és a `std::false_type` a két alapvető címke, de tetszőleges `std::integral_constant` specializáció használható.

```
#include <cstddef>
#include <cstdint>
#include <type_traits>
#include <vector>

namespace wire {

inline void put_u32be(std::vector<std::byte>& out, std::uint32_t v) {
    out.push_back(static_cast<std::byte>((v >> 24) & 0xFFu));
    out.push_back(static_cast<std::byte>((v >> 16) & 0xFFu));
    out.push_back(static_cast<std::byte>((v >> 8) & 0xFFu));
    out.push_back(static_cast<std::byte>(v & 0xFFu));
}

// Ág 1: előjel nélküli 32 bites egész -> fix byte-sorrend
template <class T>
void encode_impl(std::vector<std::byte>& out, const T& v, std::true_type) {
    static_assert(std::is_unsigned_v<T>, "elojel nelkuli egesz szukseges");
    put_u32be(out, static_cast<std::uint32_t>(v));
}

// Ág 2: minden más -> csak trivially copyable esetén
template <class T>
void encode_impl(std::vector<std::byte>& out, const T& v, std::false_type) {
    static_assert(std::is_trivially_copyable_v<T>,
                  "encode: nem tamogatott tipus");
    const auto* p = reinterpret_cast<const std::byte*>(&v);
    out.insert(out.end(), p, p + sizeof(T));
}

template <class T>
void encode(std::vector<std::byte>& out, const T& v) {
    encode_impl(out, v,
        std::bool_constant<std::is_integral_v<T> && sizeof(T) == 4>{});
}

} // namespace wire
```

A mechanizmus lényege, hogy az `encode_impl` két túlterhelése közül a harmadik paraméter típusa dönt. A hívó oldalon nincs feltétel, nincs `if`: a fordítás során kiválasztott overloadban lévő `static_assert` csak akkor értékelődik ki, amikor az adott overload példányosul. A nem hívott ág törzse nem generálódik, így az ott szereplő `static_assert` sem bukik el. Fontos azonban, hogy a *deklarációk* mindig láthatók, és ha két overload címkéje azonos, a hívás kétértelmű lesz.

A tag dispatch előnye, hogy C++11/14-gyel is működik, jól illeszkedik az overload feloldáshoz, és a kiválasztott ág a hívási helyen dokumentálható. Hátránya, hogy a döntés nem lokális: a logika szétoszlik a segédfüggvények között, és a címke típusát minden híváshoz elő kell állítani. Ezért új kódban általában az `if constexpr` a természetesebb, a tag dispatch-et pedig ott érdemes használni, ahol overload feloldásra vagy testreszabható viselkedésre van szükség.

27.1.4 if constexpr és a discarded branch

Az `if constexpr` C++17-es nyelvi elem: a feltétele constant expression, és a nem kiválasztott ág **discarded statement**. A discarded ág nem példányosul, így az ott szereplő, a sablonparamétertől függő kifejezések nem okoznak fordítási hibát. Ez teszi lehetővé, hogy egyetlen függvénytörzsben több, egymást kizáró implementációt írjunk le.

Két félreértést érdemes elkerülni. Egyrészt a discarded ágnak *szintaktikailag* érvényesnek kell lennie, és a nem a sablonparamétertől függő neveket a fordító ellenőrzi; csak a függő részek nem példányosulnak. Másrészt az `if constexpr` nem helyettesíti a `requires`-t: a túlterhelés kiválasztásában nem vesz részt, csak a kiválasztott függvény törzsében ágazik el. A visszatérési típusra viszont hathat. Explicit visszatérési típusnál nem változtat rajta; levezetett (`auto`) visszatérési típusnál azonban a discarded ág `return` utasításai nem vesznek részt a levezetésben, ezért két specializáció eltérő típust adhat vissza — egy specializáción belül csak a kiválasztott ág `return`-jeinek kell egyezniük:

```
// constexpr_return.cpp - C++20: auto visszatérési típus és if constexpr
#include <type_traits>

template <class T>
auto widen(T v) {
    if constexpr (std::is_integral_v<T>) {
        return static_cast<int>(v);        // integrál T: int
    } else {
        return static_cast<double>(v);     // egyéb T: double
    }                                      // a discarded return nem vesz részt
}

static_assert(std::is_same_v<decltype(widen(short{1})), int>);
static_assert(std::is_same_v<decltype(widen(1.5f)), double>);

template <class T>
int widen_fixed(T v) {                     // explicit visszatérési típus:
    if constexpr (std::is_integral_v<T>) return v;   // mindkét ág int-et ad
    else return static_cast<int>(v);
}
static_assert(std::is_same_v<decltype(widen_fixed(1.5f)), int>);

int main() {}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic constexpr_return.cpp` figyelmeztetés nélkül fordul: a `widen(short{1})` típusa `int`, a `widen(1.5f)` típusa `double`, a `widen_fixed` mindkét esetben `int`.

A `static_assert` ezzel együtt a legjobb diagnosztikai eszköz. A C++20-as alaphoz hordozható minta a sablonparamétertől függő hamis érték:

```
template <class> inline constexpr bool always_false_v = false;

template <class T>
void encode(std::vector<std::byte>&, const T&) {
    static_assert(always_false_v<T>, "encode: nem tamogatott tipus");
}
```

A `static_assert(false, ...)` önmagában a nem függő esetben már a sablon definíciójakor hibát adna, ezért kell a feltételt a `T`-től függővé tenni. A C++23 ezt a korlátot enyhíti a discarded ágakban, de ezt a céleszköz támogatásához kell kötni; C++20-on a fenti minta a biztonságos.

27.1.5 Kidolgozott példa: típusvezérelt wire-kódolás

A következő példa összekapcsolja a három eszközt. A `wire::encode` célja, hogy egy értéket determinisztikus byte-folyamba írjon. A string hosszprefixed, az előjel nélküli 32 bites egész fix big-endian, minden más csak akkor megy át, ha trivially copyable — és még akkor is figyelmeztetünk a padding veszélyére.

```
// wire_encode.cpp
#include <cstddef>
#include <cstdint>
#include <cstring>
#include <limits>
#include <stdexcept>
#include <string>
#include <type_traits>
#include <vector>

namespace wire {

template <class> inline constexpr bool always_false_v = false;

// A protokoll string-limitje: a 32 bites hosszmező tartománya. Ha a protokoll
// kisebb közös frame-limitet ír elő, ide azt kell írni.
inline constexpr std::size_t max_string_len =
    std::numeric_limits<std::uint32_t>::max();

// Ellenőrzött hossz: a kimenet módosítása ELŐTT dönt; ugyanezt az értéket
// írjuk a prefixbe és ennyi bájtot másolunk.
inline std::uint32_t checked_wire_len(std::size_t n) {
    if (n > max_string_len) {
        throw std::length_error("wire: string tul hosszu a 32 bites
         ↪ hosszmezohoz");
    }
    return static_cast<std::uint32_t>(n);
}
```

```cpp
inline void put_u32be(std::vector<std::byte>& out, std::uint32_t v) {
    out.push_back(static_cast<std::byte>((v >> 24) & 0xFFu));
    out.push_back(static_cast<std::byte>((v >> 16) & 0xFFu));
    out.push_back(static_cast<std::byte>((v >> 8) & 0xFFu));
    out.push_back(static_cast<std::byte>(v & 0xFFu));
}

template <class T>
void encode(std::vector<std::byte>& out, const T& value) {
    if constexpr (std::is_same_v<T, std::string>) {
        const std::uint32_t len = checked_wire_len(value.size());   // dobhat
        put_u32be(out, len);
        const auto* p = reinterpret_cast<const std::byte*>(value.data());
        out.insert(out.end(), p, p + len);
    } else if constexpr (std::is_integral_v<T> && sizeof(T) == 4) {
        static_assert(std::is_unsigned_v<T>,
                      "32 bites elojeles egesz nem tamogatott");
        put_u32be(out, static_cast<std::uint32_t>(value));
    } else if constexpr (std::is_trivially_copyable_v<T>) {
        const auto* p = reinterpret_cast<const std::byte*>(&value);
        out.insert(out.end(), p, p + sizeof(T));
    } else {
        static_assert(always_false_v<T>, "encode: nem tamogatott tipus");
    }
}

} // namespace wire

static bool rejects(std::size_t n) {
    try { (void)wire::checked_wire_len(n); return false; }
    catch (const std::length_error&) { return true; }
}

int main() {
    std::vector<std::byte> out;
    wire::encode(out, std::uint32_t{0x01020304u});   // 4 bájt
    wire::encode(out, std::string{"OK"});            // 4 bájt hossz + 2 bájt
    ↪ payload

    const unsigned char expected[] = {0x01, 0x02, 0x03, 0x04,   // u32,
    ↪ big-endian
                                      0x00, 0x00, 0x00, 0x02,   // string hossza
                                      0x4F, 0x4B};              // 'O' 'K'
    if (out.size() != sizeof expected ||
        std::memcmp(out.data(), expected, sizeof expected) != 0) {
        return 1;
    }

    // Határérték-próba allokáció nélkül: csak a helper fut.
    if (wire::checked_wire_len(0) != 0 || rejects(0)) return 2;
    if (rejects(wire::max_string_len)) return 3;
```

```
    if constexpr (sizeof(std::size_t) > sizeof(std::uint32_t)) {
        if (!rejects(wire::max_string_len + 1)) return 4;
    }
    return 0;
}
```

A normál kimenet 10 bájt: a 32 bites egész 4 bájtja, majd a string *framing*-je (4 bájtos hosszprefix) és *payload*-ja (2 bájt). A `main` nem csak a méretet, hanem a teljes várt bájtsort ellenőrzi, így a fordított bájtsorrendet és a hiányzó hosszprefixet is észreveszi. A string hosszát a `checked_wire_len` a kimenet módosítása előtt ellenőrzi; a prefixbe írt és a másolt hossz ugyanaz az ellenőrzött érték. A helper külön, allokáció nélkül tesztelhető a 0, a maximális és a maximálisnál eggyel nagyobb hosszal.

Buildparancs GCC-vel és Clangdel, rögzített nyelvi móddal:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 wire_encode.cpp -o wire_encode
clang++ -std=c++20 -Wall -Wextra -Wpedantic -O2 wire_encode.cpp -o wire_encode
```

Buildjegyzőkönyv (`lab-gcc` profil). A GCC-s build figyelmeztetés nélkül fordult, a `./wire_encode` 0-val lépett ki; a Clang-sort a nem ellenőrzött `lab-clang` helyett a Clang 22 (ziglang) fordítóval próbáltuk, szintén 0 kilépési kóddal. Két szándékos hibával is lefuttattuk: a hosszprefix kihagyása és a little-endian `put_u32be` egyaránt 1-es kilépési kódot adott. A határérték-próba 64 bites `size_t` mellett a `max_string_len + 1` értéket elutasította; ez a 64→32 bites szűkítés védelmét igazolja, nem egy több GiB-os string tényleges kódolását. A példa tanulsága hármas. Először, az ágválasztás a típus tulajdonságán alapul, és a nem támogatott típus már fordításkor elbukik. Másodszor, az `is_integral_v<T> && sizeof(T) == 4` ágban külön `static_assert` védi az előjel nélküliséget: a feltétel igaz lehet egy előjeles típusra is, és ilyenkor nem az ágválasztás, hanem az ágon belüli követelmény bukik el. Harmadszor, a trivially copyable ág *nyelvileg* helyes, de a padding és a byte-sorrend miatt nem feltétlenül determinisztikus; a trait itt a memóriabiztonságot igazolja, a protokoll-szerződést nem.

27.1.6 Tipikus hibák

A leggyakoribb hiba a `decay_t` és a `remove_cvref_t` összecserélése. Ha egy tömböt adunk át, a `decay_t<T>` pointerré alakítja, és az “ez tömb?” kérdésre hamis választ ad; ilyenkor `remove_cvref_t` vagy `remove_reference_t` kell. A második hiba, hogy a szerző a trivially copyable tényt összekeveri a szerializálhatósággal: a padding és az endianness miatt a nyers byte-másolat nem hordozható, és két különböző futás eltérő bájtokat adhat. A harmadik, hogy a `static_assert` feltételét nem teszi függővé, ezért a sablon már a definíciókor elbukik. A negyedik, hogy a tag dispatch-nél elfelejt egy címkéhez overloadot adni, és a hiba nem ott jelentkezik, ahol a hiány van. Végül az ötödik, hogy a trait-re mint üzleti invariáns bizonyítékára hivatkoznak: attól, hogy egy típus másolható, még nem biztos, hogy a másolat ugyanazt a logikai állapotot jelenti.

27.1.7 Összefoglaló

A type trait a típusról tett, fordításidőben kiértékelt állítás; az `integral_constant`/`bool_constant` adja az értéket és a címkét, a `_v`/`_t` alakok pedig a rövid használatot. A tag dispatch a trait értékét overload feloldással fordítja implementációválasztásra, az `if constexpr` pedig egyetlen törzsön belül teszi lehetővé a nem példányosuló ágakat. Mindhárom eszköz a *szintaktikai* képességről dönt; a szemantikai szerződést, a paddinget, a byte-sorrendet és az üzleti invariánst nem ismeri. A trait tehát szükséges, de nem elégséges feltétele a helyes generikus implementációnak.

27.1.8 Feladatok

1. **Magyarázó feladat.** Definiáld, mit garantál a `std::is_trivially_copyable_v<T>`, és adj egy olyan `struct`-ot, amelyre a trait igaz, a nyers byte-másolat mégsem alkalmas hálózati protokollhoz. Indokold a padding és a byte-sorrend szerepét.

2. **Kódolási feladat.** Írd át a fejezet `wire::encode` függvényét tag dispatch-re úgy, hogy a viselkedés változatlan maradjon, és a nem támogatott típus továbbra is fordításkor bukjon el. Készíts hozzá egy kis `main`-t, amely legalább egy elfogadott és egy elutasított típust kipróbál.

3. **Diagnosztikai feladat.** Az alábbi kódrészlet szándékosan hibás. Magyarázd meg, miért nem fordul le, és javítsd minimális változtatással:

   ```
   template <class T>
   void f(T) {
       if constexpr (std::is_integral_v<T>) {
           static_assert(false, "nem egesz");
       }
   }
   ```

4. **Tervezési feladat.** Egészítsd ki a `wire::encode`-ot egy `std::vector<std::uint8_t>` és egy `std::array<std::byte, N>` esetére. Döntsd el, melyik trait-re építesz, és indokold, miért nem a `is_trivially_copyable`-re.

27.1.9 Megoldások

1. megoldás. A `is_trivially_copyable_v<T>` azt garantálja, hogy a másoló/mozgató műveletek és a destruktor triviálisak, ezért az objektum reprezentációja byte-onként átmásolható egy másik, azonos típusú objektumba, és az eredmény ugyanaz az érték. Például:

```
struct Header {
    std::uint16_t type;
    std::uint32_t len;
};
static_assert(std::is_trivially_copyable_v<Header>);
```

A `sizeof(Header)` a 4 bájtos igazítás miatt jellemzően 8, és a `type` után 2 bájt padding lehet, amelynek tartalma nem specifikált. A `len` byte-sorrendje implementációfüggő. Ezért a `memcpy`-vel előállított kép hossza és tartalma nem hordozható protokoll; a helyes megoldás a mezőnkénti, fix szélességű, fix endiannessű kódolás. Ellenőrzési feltétel: a válasz nevezze meg a paddingot és a byte-sorrendet, és ne állítsa, hogy a trait hamis.

2. megoldás. A tag dispatch változat:

```
template <class T>
void encode_impl(std::vector<std::byte>& out, const T& value, std::true_type) {
    put_u32be(out, static_cast<std::uint32_t>(value));
}

template <class T>
void encode_impl(std::vector<std::byte>& out, const T& value, std::false_type) {
    static_assert(std::is_trivially_copyable_v<T>,
                  "encode: nem tamogatott tipus");
```

```
    const auto* p = reinterpret_cast<const std::byte*>(&value);
    out.insert(out.end(), p, p + sizeof(T));
}

template <class T>
void encode(std::vector<std::byte>& out, const T& value) {
    if constexpr (std::is_same_v<T, std::string>) {
        put_u32be(out, static_cast<std::uint32_t>(value.size()));
        const auto* p = reinterpret_cast<const std::byte*>(value.data());
        out.insert(out.end(), p, p + value.size());
    } else {
        encode_impl(out, value,
            std::bool_constant<std::is_integral_v<T> && sizeof(T) == 4>{});
    }
}
```

A stringág maradhat `if constexpr`, a numerikus/nyers ágat a címke választja. Az elutasított típusra a `false_type` overloadban futó `static_assert` ad hibát. Ellenőrzési feltétel: a `std::uint32_t` elfogadott, a `double` vagy egy nem trivially copyable típus fordítási hibát ad, és a hibaüzenet tartalmazza a szándékolt szöveget.

3. megoldás. A `static_assert(false, ...)` feltétele nem függ a sablonparamétertől, ezért a fordító már a sablon definíciójakor kiértékeli, és hibát ad, függetlenül attól, hogy az ágat kiválasztják-e. A minimális javítás a feltétel függővé tétele:

```
template <class> inline constexpr bool always_false_v = false;

template <class T>
void f(T) {
    if constexpr (std::is_integral_v<T>) {
        static_assert(always_false_v<T>, "nem egesz");
    }
}
```

Így a hiba csak akkor jelentkezik, ha a `T` egész típusú, és az ág példányosul. Ellenőrzési feltétel: `f(1)` fordítási hibát adjon, `f(1.0)` ne.

4. megoldás. A `std::vector<std::uint8_t>` és a `std::array<std::byte, N>` esetében a cél a hossz + nyers bájtok kiírása. A `std::array` esetén a `std::is_trivially_copyable_v` igaz lehet, de az elemek és a padding miatt nem ez a jó szerződés; használjunk explicit hosszúságot és `data()`-t:

```
template <std::size_t N>
void encode(std::vector<std::byte>& out,
            const std::array<std::byte, N>& value) {
    put_u32be(out, static_cast<std::uint32_t>(N));
    out.insert(out.end(), value.begin(), value.end());
}
```

A `std::vector<std::uint8_t>` esetén a `value.data()` és `value.size()` adja a szerződést; a `is_trivially_copyable` az elemtípusra igaz lehet, de a tárolóra nem ez a lényeg. Ellenőrzési feltétel: a kódolás hosszprefixed legyen, és a visszafejtés ellenőrizze a hossz egyezését.

27.2 Concepts, requires és constrained overload

A generikus parser- és serializer-interfészek tipikus helyei annak, ahol a template-ek használhatósága és a hibaüzenetek olvashatósága egyszerre dől el. Ha a követelmények a függvény törzsében, `static_assert`-ek és `if constexpr`-ágak mélyén fogalmazódnak meg, a hívó csak a hibák lavinájából következtet arra, mit is várt a szerző. A C++20 concept-ek és `requires` kifejezések ezt a szerződést a szignatúrába emelik: a követelmény látható, dokumentált és a fordítás korai fázisában ellenőrzött. Ez a rész a Type traits, tag dispatch és if constexpr (27.1) eszközeit egészíti ki a deklaratív megszorítás (constraint) eszközeivel, és támaszkodik a Dependent name, kétfázisú lookup és ADL (26.5) névfeloldási szabályaira, valamint az Egyenlőség, spaceship és rendezési kategóriák (23.4) összehasonlítási szerződéseire.

27.2.1 A constraint-alapú generikusság célja

A hagyományos függvénysablon minden típust elfogad, majd a törzsben derül ki, hogy mely műveleteket várja el. A hiba helye és a hiba oka elválik: a hívás helyén csak annyit látunk, hogy egy mélyen beágyazott `operator+` nem található. A concept egy névvel ellátott, compile-time kiértékelhető predikátum, amely a template paramétereire vonatkozó követelményeket fogalmazza meg. A constrained overload olyan függvénysablon, amelynek a paraméterlistája vagy a `requires` záradéka concept-ekkel szűkíti a lehetséges típusokat. A cél nem az, hogy minden lehetséges feltételt kifejezzünk, hanem hogy a fontos, dokumentált előfeltételek a felületen legyenek, és a nem megfelelő hívásokat a fordító a hívás helyén utasítsa el.

A concept-ek nem helyettesítik a dokumentációt. A szintaktikai követelmény azt mondja meg, hogy egy kifejezés jól formált-e; a szemantikai követelmény azt, hogy a művelet milyen viselkedést ígér. A fordító csak az előbbit ellenőrzi. A parser-interfésznél például a `parse` tagfüggvény létezése szintaktikai követelmény; az, hogy a parser a hibát egy konzisztens állapotban jelzi, és nem módosítja a bemenetet, szemantikai szerződés, amelyet a dokumentációnak kell rögzítenie.

27.2.2 Concept definíció és requires clause

A concept definíciója egy `template` fejléccel kezdődik, majd a `concept` kulcsszó, a név, az egyenlőségjel és egy constraint-kifejezés következik:

```
template <typename T>
concept Parser = requires(T& p, std::string_view input) {
    { p.parse(input) } -> std::same_as<ParseResult>;
};
```

A constraint-kifejezés lehet `requires` kifejezés, más concept hivatkozása, vagy ezek `&&`, `||`, `!` operátorokkal alkotott kombinációja. A concept nem tagfüggvény, nem változó, és csak template paraméterre vagy `auto`-ra vonatkozhat. A `requires` záradék (requires clause) a template paraméterlista után, a függvénydeklaráció előtt áll, és a függvényre vonatkozó megszorítást adja:

```
template <typename T>
    requires std::integral<T>
T add(T a, T b) { return a + b; }
```

A rövidített forma `template <std::integral T>` vagy `void f(std::integral auto x)`. Több `requires` záradék is állhat egymás után; ezek konjunkciót alkotnak. A záradék nem tévesztendő össze a `requires` kifejezéssel: az előbbi a deklaráció része, az utóbbi a concept definíciójában vagy közvetlenül a záradékban álló, zárójelezett követelményhalmaz.

27.2.3 A requires expression négy követelménytípusa

A `requires` kifejezés négyféle követelményt tartalmazhat. Az egyszerű követelmény (simple requirement) csak a kifejezés jólformáltságát ellenőrzi:

```
template <typename T>
concept HasFoo = requires(T t) { t.foo(); };
```

A típuskövetelmény (type requirement) egy típus létezését írja elő; a függő típusnévhez `typename` kell:

```
template <typename T>
concept HasValueType = requires { typename T::value_type; };
```

Az összetett követelmény (compound requirement) a kifejezés jólformáltsága mellett a visszatérési típust is ellenőrzi, opcionálisan `noexcept` elvárással:

```
template <typename T>
concept HasSize = requires(const T& t) {
    { t.size() } -> std::convertible_to<std::size_t>;
};
```

A beágyazott követelmény (nested requirement) egy másik constraint igazságát írja elő a `requires` kifejezésen belül:

```
#include <concepts>
#include <cstddef>
#include <cstdint>
template <typename T>
concept SmallIntegral = std::integral<T> && requires {
    requires sizeof(T) <= 4;                    // nested requirement
};
static_assert(SmallIntegral<std::int32_t> && SmallIntegral<char>);
static_assert(!SmallIntegral<std::int64_t> && !SmallIntegral<float>);
int main() {}
```

A nested requirement (`requires sizeof(T) <= 4;`) itt valódi, teljesíthető feltétel: a 32 bites és a `char` típus elfogadott, a 64 bites egész és a `float` nem. (Egy `requires std::integral<T>` és egy `{ t.size() }` követelményt együtt kérő concept a beépített egész típusokra mindig hamis lenne, mert azoknak nincs `size()` tagja.) **Buildjegyzőkönyv** (`lab-gcc` profil): `g++ -std=c++20 -Wall -Wextra` mellett mind a négy `static_assert` teljesül.

A négy forma együtt lehetővé teszi, hogy a követelményeket közel a használathoz, mégis újrahasznosítható módon fogalmazzuk meg. A gyakorlatban érdemes a `requires` kifejezéseket névvel ellátott concept-ekbe csomagolni, mert így a hibaüzenetekben a concept neve jelenik meg, és a követelmények kombinálhatók.

27.2.4 Abbreviated function template és standard concept-ek

Az abbreviated function template a `auto` paraméter elé tett concept-tel függvénysablont hoz létre:

```
void print_value(const std::integral auto& value) {
    std::cout << value << '\n';
}
```

Ez ekvivalens egy `template <std::integral T> void print_value(const T&)` deklarációval, de a szignatúra olvashatóbb. A standard library számos concept-et kínál: `std::same_as`, `std::convertible_to`, `std::derived_from`, `std::integral`, `std::floating_point`, `std::copyable`, `std::movable`, `std::invocable`, `std::predicate`, valamint a ranges névtérben `std::ranges::range`, `std::ranges::input_range` és társai. Ezek a fogalmak a szintaktikai követelményt és a dokumentált szemantikai elvárást együtt képviselik. A `std::equality_comparable` például megköveteli az `==` és `!=` jólformáltságát, de a szemantikai szerződés — az ekvivalenciareláció reflexivitása, szimmetriája és tranzitivitása — a felhasználó felelőssége marad.

27.2.5 Constraint normalization és subsumption

A fordító a constraint-eket atomikus megszorításokra (atomic constraint) bontja (C++20 [temp.constr.normal]). Egy atomikus megszorítás olyan kifejezés, amelyet a `&&` és `||` operátorok már nem bontanak tovább — például egy `requires` kifejezés vagy egy `std::is_integral_v<T>` jellegű logikai kifejezés. A concept-hivatkozás (`A<T>`) viszont **nem** önálló atom: a normalizálás kifejti, és helyére a concept definíciójának normál formája kerül, a paraméterek behelyettesítésével. Két atomi megszorítás csak akkor azonos, ha *ugyanabból a forráskifejezésből* (ugyanarról a forráshelyről) származik, és a paraméterleképezésük is egyezik; két szövegre azonos, de külön leírt `requires` kifejezés két különböző atom. A normalizálás után a részben rendezés (subsumption) dönti el, hogy két constrained overload közül melyik a specifikusabb. A `C<T> && D<T>` bekebelezi `C<T>`-t, mert a normál formája tartalmazza azt; a `C<T> || D<T>` nem. A következő vázlat a lényeget mutatja meg, nem formális bizonyítás:

```
template <typename T>
concept A = requires { typename T::a; };

template <typename T>
concept B = A<T> && requires { typename T::b; };

template <A T> void f(T) {}   // 1
template <B T> void f(T) {}   // 2
```

Az `A` normál formája egyetlen atom: az `A` definíciójában leírt `requires { typename T::a; }`. A `B` normál formája ennek az atomnak és a `requires { typename T::b; }` atomnak a konjunkciója — az `A<T>` hivatkozás kifejtésekor *ugyanaz* az atom kerül bele, amely az `A` definíciójában jött létre. Ezért a `B` bekebelezi az `A`-t, és a 2-es overload a specifikusabb. Ha viszont a `B`-t a concept újrahasználata nélkül, ugyanazzal a szöveggel írjuk meg, a fordító nem látja a kapcsolatot:

```
// atoms.cpp - C++20: az atomi megszorítás a forráshelyhez kötött
#include <cstdio>

template <typename T>
concept A = requires { typename T::a; };                 // itt jön létre az
 ↪  atom

template <typename T>
concept B = A<T> && requires { typename T::b; };         // A<T> kifejtve:
 ↪  ugyanaz az atom
```

```
template <typename T>
concept B2 = requires { typename T::a; }                // szövegre azonos, de
 ↪ MÁSIK atom
          && requires { typename T::b; };

template <A T>  int f(T) { return 1; }
template <B T>  int f(T) { return 2; }                  // B bekebelezi A-t

template <A T>  int g(T) { return 1; }
template <B2 T> int g(T) { return 2; }                  // B2 nem kebelezi be
 ↪ A-t

struct AB { using a = int; using b = int; };

int main() {
    std::printf("f: %d\n", f(AB{}));   // 2
#ifdef AMBIGUOUS
    std::printf("g: %d\n", g(AB{}));   // kétértelmű
#endif
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic atoms.cpp` kimenete `f: 2`. `-DAMBIGUOUS` mellett a `g(AB{}) call of overloaded 'g(AB)' is ambiguous` hibát ad: a `B2` első `requires` kifejezése szövegre megegyezik az `A`-éval, de másik forráshelyen áll, ezért másik atom, és a `B2` nem kebelezi be az `A`-t. Ez a subsumption gyakorlati következménye: a szűkebb concept-nek hivatkoznia kell a tágabbra, különben a fordító nem látja a kapcsolatot.

27.2.6 Diagnosztika és tipikus hibák

A concept-ek legnagyobb gyakorlati haszna a diagnosztikában jelentkezik: a hívás helyén a fordító azt írja, hogy a `CounterParser` nem elégíti ki a `Parser` concept-et, és megnevezi a sikertelen követelményt. A hagyományos SFINAE-s megoldásoknál ehhez képest a hibaüzenet rövidebb és a szerződéshez közelebb áll. A tipikus hibák közül az első a `requires` záradék és a `requires` kifejezés összekeverése: a záradék nem kerülhet a függvény törzsébe, és a `requires requires` kétszeres használata legtöbbször concept-tel kerülhető el. A második a `typename` elhagyása típuskövetelményből: `requires { T::value_type; }` helyett `requires { typename T::value_type; }` kell. A harmadik a túl gyenge összetett követelmény: a `requires(T t) { t.size(); }` akkor is igaz, ha a `size()` visszatérési típusa `void`; a `{ t.size() } -> std::convertible_to<std::size_t>` ezt kizárja. A negyedik a nem függő kifejezés a `requires` kifejezésben: a `requires { sizeof(int) > 0; }` mindig igaz, és nem használja a template paramétert, ezért nem ad információt. Az ötödik a szemantikai követelmény elhagyása: a concept csak a szintaxist ellenőrzi, a viselkedési szerződést a dokumentációnak kell rögzítenie.

27.2.7 Kidolgozott példa: generikus parser interfész

A következő példa egy minimális parser-interfészt mutat be. A `Parser` concept megköveteli a `parse` tagfüggvényt, amely `ParseResult`-ot ad vissza. A `ReusableParser` ehhez hozzáadja a `reset` tagfüggvényt, és hivatkozik a `Parser`-re, hogy a subsumption működjön. A két `parse` overload közül a `ReusableParser`-es a specifikusabb.

```cpp
#include <concepts>
#include <iostream>
#include <optional>
#include <string>
#include <string_view>

struct ParseResult {
    bool ok;
    std::string value;
    std::string error;
};

template <typename P>
concept Parser = requires(P& p, std::string_view input) {
    { p.parse(input) } -> std::same_as<ParseResult>;
};

template <typename P>
concept ReusableParser = Parser<P> && requires(P& p) {
    { p.reset() } -> std::same_as<void>;
};

template <Parser P>
ParseResult parse(P& p, std::string_view input) {
    return p.parse(input);
}

template <ReusableParser P>
ParseResult parse(P& p, std::string_view input) {
    p.reset();
    auto result = p.parse(input);
    result.value = "[reset] " + result.value;
    return result;
}

struct CounterParser {
    int parse_count = 0;
    int reset_count = 0;
    ParseResult parse(std::string_view input) {
        ++parse_count;
        return {true, std::string(input), {}};
    }
    void reset() { ++reset_count; }
};

struct OneShotParser {
    ParseResult parse(std::string_view input) {
        return {true, std::string(input), {}};
    }
};

int main() {
```

```
    CounterParser cp;
    auto r1 = parse(cp, "hello");
    std::cout << r1.value << " parse_count=" << cp.parse_count
              << " reset_count=" << cp.reset_count << '\n';

    OneShotParser op;
    auto r2 = parse(op, "world");
    std::cout << r2.value << '\n';
}
```

A buildparancs a laborprofilban rögzített GCC vagy Clang C++20 módjában:

```
g++ -std=c++20 -Wall -Wextra -pedantic -o parser_demo parser_demo.cpp
# vagy
clang++ -std=c++20 -Wall -Wextra -pedantic -o parser_demo parser_demo.cpp
```

A `CounterParser` megfelel a `ReusableParser`-nek, ezért a fordító a második `parse` overloadot választja: a `reset()` meghívása után a `parse` eredménye a `[reset] hello` értéket kapja, a `parse_count` és a `reset_count` egyaránt 1 lesz. A `OneShotParser` csak a `Parser`-nek felel meg, ezért az első overload fut, és a `world` értéket adja vissza. A tényleges kimenet a labor profiljától függ; a példa nem tekinthető futás nélkül teszteltnek.

27.2.8 Összefoglaló

A concept egy névvel ellátott compile-time predikátum, amely a template paramétereire vonatkozó követelményeket rögzíti. A `requires` kifejezés egyszerű, típus-, összetett és beágyazott követelményeket tartalmazhat; a `requires` záradék a deklaráció része. Az abbreviated function template a concept-et az `auto` paraméterbe emeli. A standard concept-ek szintaktikai és dokumentált szemantikai szerződést egyesítenek, de a szemantikát a fordító nem ellenőrzi. A constraint-normalizálás és a subsumption teszi lehetővé, hogy a specifikusabb overloadot a fordító válassza; ehhez a szűkebb concept-nek hivatkoznia kell a tágabbra. A jó diagnosztika feltétele a pontos, de nem túl gyenge követelmény, és a szemantikai elvárások dokumentálása.

27.2.9 Feladatok

27.2.9.1 HasSize concept

Definiálj egy `HasSize` concept-et, amely megköveteli, hogy a `T` típusnak legyen `size()` tagfüggvénye, amely `std::size_t`-re konvertálható. Írj egy `print_size` függvényt, amely ezt a concept-et használja, és működjön `std::vector<int>`-re és `std::string`-re. Magyarázd el, miért nem fordul le `int` argumentummal.

27.2.9.2 Serializable concept

Definiálj egy `Serializable` concept-et, amely megköveteli a `to_string()` tagfüggvényt `std::string` visszatérési típussal, és a `from_string(std::string_view)` statikus tagfüggvényt `std::optional<T>` visszatérési típussal. Írj egy `roundtrip` függvényt, amely ezt használja, és egy `Point` struct-ot, amely megfelel a concept-nek.

27.2.9.3 TransactionalParser concept

Egészítsd ki a parser példát egy `TransactionalParser` concept-tel, amely a `ReusableParser`-t terjeszti ki `commit()` és `rollback()` tagfüggvényekkel. Definiálj egy `parse` overloadot a `TransactionalParser`-re, és mutasd meg, hogy a subsumption miatt ez a specifikusabb overload fut. Magyarázd el, miért kell a `TransactionalParser` definíciójában szerepelnie a `ReusableParser<P>` hivatkozásnak.

27.2.9.4 Addable diagnosztika

Adott a következő concept:

```
template <typename T>
concept Addable = requires(T a, T b) { a + b; };
```

Mi a hiba ebben a definícióban? Javítsd ki, és magyarázd el, miért ad jobb diagnosztikát a javított változat.

27.2.10 Megoldások

27.2.10.1 HasSize concept megoldása

```
template <typename T>
concept HasSize = requires(const T& t) {
    { t.size() } -> std::convertible_to<std::size_t>;
};

template <HasSize T>
void print_size(const T& value) {
    std::cout << value.size() << '\n';
}
```

A `std::vector<int>` és a `std::string` `size()` tagfüggvénye `std::size_t`-re konvertálható, ezért megfelelnek. Az `int`-nek nincs `size()` tagfüggvénye, ezért a concept hamis, és a hívás a hívás helyén fordítási hibát ad. A hibaüzenet a `HasSize` concept-et nevezi meg, nem a `print_size` törzsét. Az ellenőrzés feltétele: a `print_size(std::vector<int>{1, 2, 3})` hívás kiírja a 3-at, a `print_size(42)` hívás nem fordul le.

27.2.10.2 Serializable concept megoldása

```
template <typename T>
concept Serializable = requires(const T& t, std::string_view s) {
    { t.to_string() } -> std::same_as<std::string>;
    { T::from_string(s) } -> std::same_as<std::optional<T>>;
};

template <Serializable T>
std::optional<T> roundtrip(const T& value) {
    return T::from_string(value.to_string());
}
```

```
struct Point {
    int x;
    int y;
    std::string to_string() const {
        return std::to_string(x) + "," + std::to_string(y);
    }
    static std::optional<Point> from_string(std::string_view s) {
        auto comma = s.find(',');
        if (comma == std::string_view::npos) return std::nullopt;
        int x = std::stoi(std::string(s.substr(0, comma)));
        int y = std::stoi(std::string(s.substr(comma + 1)));
        return Point{x, y};
    }
};
```

A `Point` megfelel a `Serializable`-nek, mert a `to_string()` `std::string`-et ad vissza, a `from_string` pedig `std::optional<Point>`-ot. A `roundtrip(Point{1, 2})` a `Point{1, 2}`-t adja vissza. Az `int` nem felel meg, mert nincs `to_string()` tagfüggvénye. Az ellenőrzés feltétele: a `roundtrip(Point{1, 2})` értéke `Point{1, 2}`, és a `roundtrip(42)` hívás nem fordul le.

27.2.10.3 TransactionalParser concept megoldása

```
template <typename P>
concept TransactionalParser = ReusableParser<P> && requires(P& p) {
    { p.commit() } -> std::same_as<void>;
    { p.rollback() } -> std::same_as<void>;
};

template <TransactionalParser P>
ParseResult parse(P& p, std::string_view input) {
    p.reset();
    auto result = p.parse(input);
    if (result.ok) {
        p.commit();
    } else {
        p.rollback();
    }
    return result;
}
```

A `TransactionalParser` normál formája tartalmazza a `ReusableParser<P>`-t, amely tartalmazza a `Parser<P>`-t. Ezért a `TransactionalParser` bekebelezi a `ReusableParser`-t és a `Parser`-t is, így a három `parse` overload közül ez a specifikusabb. Ha a `ReusableParser<P>` hivatkozás kimaradna, a `TransactionalParser` nem kebelezné be a `ReusableParser`-t, és egy mindkettőnek megfelelő típusnál a hívás kétértelmű lenne. Az ellenőrzés feltétele: egy `TransactionalCounterParser` típus esetén a `parse` hívás a `TransactionalParser` overloadot választja, a `commit_count` értéke 1, a `reset_count` értéke 1.

27.2.10.4 Addable diagnosztika megoldása

A hiba az, hogy a concept csak az `a + b` kifejezés jólformáltságát ellenőrzi. Ha egy típusnak van `operator+`-szal, amely `void`-ot ad vissza, a concept igaz lesz, de a `return a + b;` utasítás hibás. A javított változat:

```
template <typename T>
concept Addable = requires(T a, T b) {
    { a + b } -> std::convertible_to<T>;
};
```

Ez a visszatérési típusnak a `T`-re való konvertálhatóságát is megköveteli, ezért a hiba már a hívás helyén, a concept szintjén jelentkezik. A régi változatnál a hiba a függvény törzsében, a `return` utasításnál derülne ki, ami nehezebben olvasható és a szerződéstől távolabb esik. Az ellenőrzés feltétele: egy olyan típus, amelynek `operator+`-a `void`-ot ad vissza, a javított concept-tel nem fordul le, a régi változattal viszont a concept igaz lenne.

27.3 SFINAE és örökölt template minták

A constraint-alapú programozás mai formája a C++20-as `concepts` és `requires`, de a 2000-es évek végétől a 2020-as évek elejéig tartó időszak generikus könyvtárai szinte kizárólag a SFINAE-re (Substitution Failure Is Not An Error — „a helyettesítési hiba nem hiba”) és az `enable_if`-re épültek. Egy haladó DevOps/platformmérnök reálisan találkozik ilyen kóddal: Boost, `nlohmann::json`, korai `fmt`, fmtlog, régi absl-szeletek, saját in-house template-utility könyvtárak, illetve a saját magunk által 5 évvel ezelőtt írt kód. Ez a rész ezért nem új nyelvi képességet tanít, hanem az örökölt minták megértését és a `requires`-alapú modern megfelelőjükre fordítását célozza. A cél, hogy az olvasó felismerje: mikor számít egy hiba „substitution failure”-nek, mikor „hard error”-nak; és hogy milyen olvashatósági és kompatibilitási kompromisszumokkal jár a migráció.

27.3.1 Substitution failure és overload candidate kiesése

Tekintsük a sablon-argumentumok illesztésének a folyamatát. Amikor egy függvénysablonra hívást illesztünk, a fordító először a sablonparamétereket vezeti le a hívás argumentumaiból (template argument deduction), majd a levezetett argumentumokat behelyettesíti (substitute) a sablondeklarációba. A helyettesítés a teljes deklarációra kiterjed: a visszatérési típusra, a paraméterek típusára, a `noexcept`-specifikációra, a kivételspecifikációra, az alapértelmezett sablonargumentumokra.

Ha a helyettesítés olyan típust vagy kifejezést eredményez, amely szintaktikailag vagy szemantikailag érvénytelen, de ez a hiba az úgynevezett **immediate context**-en belül van, akkor a jelölt csendben kiesik az overload-halmazból, és a fordító a többi jelölt közül választ. Nem jelent fordítási hibát. Ha ilyen módon minden jelölt kiesik, a hívás „no matching function” hibát ad — de maga a kiesés nem hiba a nyelv szempontjából.

Definíció. Az immediate context az a leszűkített szintaktikai tartomány, ahol a helyettesítési hiba még nem hard error. Ide tartozik: a függvénysablon teljes deklarációja (visszatérési típus, paramétertípusok, `noexcept`-specifikáció, alapértelmezett argumentumok), valamint az osztálysablon-részleges-specializáció fejléce. Nem tartozik bele a függvénytest, sem a hívott függvények teste, sem egy másik sablon példányosítása, amelyet a helyettesítés során indítunk.

A klasszikus példa:

```cpp
#include <type_traits>

template<class T>
typename T::value_type first(const T& c) { return *c.begin(); }

template<class T>
T first(T* p) { return *p; }
```

A `first(&x)` hívásnál, ahol `x` egy `int`, az első jelölt esetén `T = int*`, majd a helyettesítés `int*::value_type`-ot próbál előállítani, ami érvénytelen. Ez az immediate contextben történik (`typename T::value_type` a visszatérési típus), így az első jelölt kiesik, a második nyer. Ugyanez a helyettesítés egy `T::value_type` tagon a függvénytesten belül hard error lenne.

27.3.2 Immediate context versus instantiation hard error

A megkülönböztetés kulcsa a **hely** és az **időzítés**. Az immediate context a *deklaráció* szintjén értendő, és a *substitution* fázisában értékelődik ki. Ami a *body*-ban történik, azt a *substitution* fázis már nem érinti; az csak a kiválasztott overload példányosításakor derül ki, és hard error. Formálisan: a sablon kiválasztásakor még nem is példányosítjuk a testet; csak a deklarációt ellenőrizzük.

Tekintsük a következő két sablont:

```cpp
template<class T>
void write_out(const T& v) { typename T::value_type tmp{}; (void)tmp; }   // (1)
 ↪  hard error

template<class T>
auto read_first(const T& c) -> typename T::value_type { return *c.begin(); } //
 ↪  (2) SFINAE
```

Az (1)-re `write_out(42)` hívás esetén a testben `int::value_type` érvénytelen → hard error, mert a body nem része az immediate contextnek. A (2)-re `read_first(42)` hívás esetén a `T::value_type` a visszatérési típusban van, tehát a substitution része, tehát SFINAE: a jelölt kiesik, más overload hiányában „no matching function”.

Ez az a pont, ahol a régi kód olvasója a leggyakrabban téved: a szerző azt hiszi, hogy „SFINAE-zel megvédtem”, de a védelem a body-ra nem terjed ki. Ilyenkor vagy a constraintet át kell tenni a deklarációba (`enable_if` vagy `requires`), vagy a body-t kell `if constexpr`-szel tagolni (ami már a nyelvi eszköztár része, és a `requires`-szel való kombináció is gyakori).

27.3.3 enable_if, void_t és a detection idiom

A SFINAE-t a gyakorlatban leggyakrabban két eszközzel használták: `std::enable_if` és `std::void_t`. Mindkettő a `<type_traits>`-ben található.

Az `enable_if<B, T = void>` definíciója: ha `B` igaz, van `::type` tagja, értéke `T`; ha `B` hamis, `::type` tag nincs. A hiba, ha `B` hamis, egy substitution failure, nem hard error — ezért alkalmas SFINAE-re.

Két tipikus felhasználási hely:

1. **Visszatérési típusban**: `typename std::enable_if_t<cond, T> f(...)`. A hiba a return type-ban substitution failure.

2. **Extra sablonparaméterként alapértelmezett argumentummal**: `template<class T, class = std::enable_if_t<cond>> void f(...)`. Ez robusztusabb, mert nem kell a visszatérési típust dedukáltatni.

A visszatérési típusbeli forma korlátja komoly: a két overload visszatérési típusának meg kell egyeznie, ha mindkettő viábilis, mert a nyelv nem engedi ugyanazt a függvénynevet két különböző visszatérési típussal. Ezért tipikusan `enable_if_t<cond, T>` és `enable_if_t<cond2, T>` páros szerepel, ahol `T` ugyanaz.

A `void_t` C++17 óta standard könyvtári eszköz:

```
template<class...> using void_t = void;
```

A **detection idiom** lényege: egy elsődleges sablon `false_type`-ból öröklődik, és egy részleges specializáció `void_t<decltype(...)>`-re szűkít. Ha a `decltype` érvénytelen, a specializáció nem illeszkedik, marad az elsődleges sablon (`false`); ha érvényes, `void_t` mindig `void`, tehát a specializáció illeszkedik (`true`). Például egy `serialize()` tagfüggvény detektálása:

```
template<class, class = void>
struct has_serialize : std::false_type {};

template<class T>
struct has_serialize<T, std::void_t<decltype(std::declval<const
 ↪  T&>().serialize())>>
    : std::true_type {};
```

A `decltype` operátorán belül `std::declval<T>()` csak kiértékelés nélküli kontextusban legális; ez a detection idiom kötelező eszköze. A `void_t`-nek legalább egy sablonparamétert tartalmazó packra van szüksége: `void_t<>` ugyanaz, mint `void`, és nem lenne dependens, tehát a specializáció akkor is illeszkedne, amikor a belső kifejezés érvénytelen.

27.3.4 Return-type constraint korlátai

A visszatérési típusra tett constraint (klasszikusan `typename std::enable_if_t<..., T>`) három okból nem ideális:

1. A feltételnek dependensnek kell lennie egy levezetett sablonparamétertől, különben a substitution nem tudja értékelni, és hard error lesz belőle.
2. Két viábilis overload esetén a visszatérési típusnak egyeznie kell — ez szűkíti a megfogalmazható feltételeket.
3. Diagnosztikai szempontból a hibaüzenet a függvény szignatúrájára hivatkozik, nem egy nevesített predikátumra, ezért nehezen olvasható.

Továbbá a visszatérési típus az Itanium ABI-ban **részét képezi a függvénysablon mangled nevének** (nem úgy, mint a közönséges függvényeknél), ezért a feltétel visszatérési típusba ágyazása potenciálisan ABI-érzékeny. Eldobható prototípuson ez nem probléma, de egy dinamikusan linkelt, verziózott könyvtárban a szerződés része lehet. Ha erre érzékeny felületet tervezünk, a feltételt érdemes inkább extra sablonparaméterbe tenni, vagy `requires`-re áttérni — a `requires`-klóz nem része a függvénytípusnak, és az Itanium ABI manglingjában hagyományosan nem jelenik meg, bár ezt adott fordítón és ABI-verzión ellenőrizni kell `nm`-mel.

27.3.5 requires versus SFINAE összehasonlítás

A C++20-as `requires`-klóz ugyanazt a szerepet tölti be, de három lényeges előnnyel. Először is **nevesíthető**: egy `concept` deklarációban van a jelentés, és a hibaüzenetben a concept neve jelenik meg, nem egy hosszú szignatúra. Másodszor **részlegesen rendezhető**: két azonos nevű overload közül a szigorúbban korlátozott (több altételt kielégítő) nyer, nem ambiguity lesz. Harmadszor **összefűzhető**: `concept A = ...; concept B = A && ...;` szintaktikailag és szemantikailag is komponálható.

Ugyanaz a függvény két formában:

```
// SFINAE (C++11/14/17)
template<class T>
typename std::enable_if_t<std::is_integral_v<T>, T> add(T a, T b) { return a +
 ↪  b; }

// requires (C++20)
template<class T>
requires std::integral<T>
T add2(T a, T b) { return a + b; }
```

A `requires` előnye a részleges rendezésnél élesedik:

```
#include <concepts>

template<class T> concept addable = requires(T a, T b) { a + b; };
template<class T> concept ordered = addable<T> && requires(T a, T b) { a < b; };

template<addable T> void f(T) {}           // (A)
template<ordered T> void f(T) {}           // (B) szigorúbban korlátozott

f(1);          // (B) választódik, mert ordered subsumálja addable-t
f(1.0);        // (B) választódik, mert a double is ordered
```

A detection idiom `requires`-re fordítva:

```
template<class T>
concept serializable = requires(const T& t) { t.serialize(); };

static_assert(!serializable<int>);
```

Ez ugyanazt a predikátumot fejezi ki, mint a `void_t`-s `has_serialize`, de nevesített concept, és a hibaüzenetben is ez a név jelenik meg. Ha a detection visszatérési típusát is ellenőrizni kell, a requires-expression kifejezéskövetelménye tehető: `requires(T t) { { t.size() } -> std::convertible_to<std::size_t>; }`. Ez a fajta finomság SFINAE-vel csak sokkal hosszabban és olvashatatlanabbul volt megfogalmazható.

27.3.6 Olvashatóság és binary/source compatibility migrációkor

A migráció kulcskérdése: mikor biztonságos az `enable_if`-et `requires`-re cserélni? Három tengelyen érdemes vizsgálni.

Forráskompatibilitás. A `requires`-be kerülés megváltoztathatja az overload-feloldást: a részleges rendezés (subsumption) olyan eseteket is eldönthet, amit a régi SFINAE-szett nem — tipikusan

két hasonló feltételnél, ahol eddig ambiguity volt, most az egyik nyer. Emiatt a meglévő hívások viselkedése megváltozhat. Ezért migrációkor a teljes tesztmátrixot kell futtatni, nem csak a sikeres buildet.

Bináris kompatibilitás. A `requires`-klóz nem része a függvénytípusnak, ezért az Itanium ABI-ban a mangled név jellemzően változatlan marad, ha az instantiated típusok azonosak. A visszatérési típusba ágyazott `enable_if` azonban mangling-érzékeny: ha a feltétel hamis, a jelölt kiesik; ha igaz, a függvény visszatérési típusa az `enable_if<true,T>::type == T`, tehát az instantiated típus megegyezik a plain `T`-vel. Emiatt a mangled név is azonos. **Ez azonban adott fordító és ABI-verzió mellett ellenőrizendő** — `nm -C` és `c++filt` a gyakorlati ellenőrző eszköz. A `requires`-re való áttérés tipikusan nem tör meg bináris felületet, ha a hatékony típusok egyébként megegyeznek; a szerződésnek azonban ezt nem szabad hallgatólagosan feltételezni.

Olvashatóság. Örök kódban a `typename std::enable_if_t<...>` zsúfolt, többsoros feltételek benne nehezen követhetőek, a diagnosztika a fordítótól függően kaotikus. Migrációkor érdemes két lépésben haladni: először nevesített `concept`-eket bevezetni a meglévő predikátumokból, még a régi SFINAE-szignatúrák mellett; majd amikor a concept-ek stabilizálódtak, lecserélni a szignatúrákat. Ez a stratégia lehetővé teszi, hogy a fogalmi réteg és a mechanikai implementáció külön fejlődjön, és hogy a régi overloadok addig éljenek, amíg a hívások át nem állnak.

Gyakori migrációs hiba, hogy a fejlesztő a régi `enable_if`-et `requires`-re cseréli, és elfelejti, hogy a régi overload `enable_if`-ben **visszatérési típusba** ágyazva volt — ilyenkor a `requires`-re való áttérés megváltoztatja a függvény tényleges visszatérési típusát, például `T` helyett `void` lesz, ha a fordító nem a visszatérési típusban találja a típust. A place-of-constraint mindig tudatos döntés legyen.

27.3.7 Gyakori hibák

Leggyakrabban az alábbi hibákba futunk:

1. **Hard error a body-ban.** A substitution failure csak a deklaráció szintjén működik; ha a hiba a függvénytestben keletkezik, a fordító hard error-t ad.
2. **Nem-dependens feltétel.** Ha az `enable_if` feltétele nem függ a levezetett sablonargumentumtól, akkor a substitution az összes jelöltre egyszerre értékelődik ki, és az érvénytelen feltétel hard error lesz, nem SFINAE.
3. **Hiányzó `::type`.** `enable_if<nem_igaz>` nem rendelkezik `::type` taggal, ezért `enable_if<nem_igaz>::type` érvénytelen — a helyes használat `enable_if_t<igaz, Cél>` vagy `enable_if<igaz>::type`.
4. **`void_t` pakk nélkül.** `std::void_t<>` ugyanaz, mint `void`, és nem tesz függővé semmit, ezért a specializáció akkor is illeszkedhet, amikor a belső kifejezés érvénytelen.
5. **Két különböző visszatérési típus.** Ha a visszatérési típusba ágyazott `enable_if` során a feltétel igaz, és a másik overload feltétele is igaz, a két visszatérési típusnak meg kell egyeznie, különben a fordító nem tudja ugyanazt a nevet két különböző visszatérési típussal feloldani.
6. **`declval` futásidőben.** `std::declval<T>()` csak kiértékelés nélküli kontextusban legális; futásidejű hívás hard error.
7. **Migráció alatt az elsődlegesen érintett overload feledése.** A `requires`-re áttérés után az overload-halmaz részleges rendezése más, mint korábban; a teljes mátrixot tesztelni kell.

27.3.8 Feladatok

1. feladat (magyarázó). Adott a következő két overload:

```
template<class T>
typename std::enable_if_t<std::is_integral_v<T>, T> foo(T);

template<class T>
typename std::enable_if_t<std::is_floating_point_v<T>, T> foo(T);
```

Válaszoljon: mi történik `foo(1)`, `foo(1.0)`, `foo("x")` hívás esetén? Hol történik a substitution, és a hiba az immediate contextben van-e? Miért nincs hard error, mégis elutasítja a fordító a `foo("x")` hívást?

2. feladat (kódolási). Fordítsa le a lenti `void_t`-alapú detection idi…

27.4 Variadic template, pack és fold expression

A variadikus sablonok (variadic templates) teszik lehetővé, hogy egy függvény- vagy osztálysablon tetszőleges számú típusparamétert és argumentumot fogadjon el úgy, hogy a típusinformáció fordítási időben megmaradjon. A C++11-ben bevezetett parameter pack és pack expansion, a C++17-es fold expression, valamint a C++20-as constraint-ek együtt teszik lehetővé a típusos, mégis rugalmas interfészeket. Ez a rész a pack kibontásának mechanizmusát, a fold kiértékelését és az argumentumok élettartamára vonatkozó következményeket tárgyalja. A forwarding reference és a perfect forwarding mechanizmusát a Forwarding reference és perfect forwarding rész tárgyalta; a constraint-ekre a Concepts, requires és constrained overload rész épít. Itt ezekre csak a szükséges mértékben hivatkozunk.

A szabványban a variadikus sablonok és a pack expansion a `[temp.variadic]`, a fold expression a `[expr.prim.fold]` alatt található. Ezek a paragrafushivatkozások tájékoztatók; a konkrét C++20 szabvány szövegében ellenőrizendők. A példák C++20 nyelvi módot és a standard library C++17/20 elemeit használják; a buildparancsok feltételezik a C++20 támogatását. A bemutatott kimenetek illusztrációk, nem mért eredmények.

27.4.1 Parameter pack és pack expansion

A **parameter pack** a sablon típusparamétereinek vagy függvényparamétereinek olyan csoportja, amelyet a `...` jelöl. Két alapvető alakja van:

```
template <class... Ts>          // típusparameter-pack
void f(Ts... args);             // függvényparameter-pack

template <class... Ts>
void g(const Ts&... args);      // a minta: const Ts&
```

A `Ts...` nem egyetlen típus, hanem nulla vagy több típus szintaktikai csoportja. A `args...` hasonlóképpen nulla vagy több függvényparaméter. A pack mérete a `sizeof...(Ts)` vagy `sizeof...(args)` kifejezéssel kérdezhető le; ez fordítási időben konstans.

A **pack expansion** olyan szintaktikai minta, amelyben egy pack neve szerepel, és amelyet a compiler a pack minden elemére egyszer kifejt. A minta a `...` előtti egység. Például a `const Ts&...` mintában a minta `const Ts&`, így minden `Ts` elemre egy `const T&` paraméter jön létre. A `std::forward<Args>(args)...` mintában a minta `std::forward<Args>(args)`, így minden `Args`/`args` párra egy-egy `std::forward` hívás keletkezik.

A pack expansion érvényességének feltételei:

- A pack nevének egy pack expansion mintájában kell szerepelnie; önmagában a pack neve nem használható értékként.
- Ha egy mintában több pack szerepel, azok hosszának meg kell egyeznie. Például `f(args..., rest...)` csak akkor érvényes, ha a két pack elemszáma azonos.
- A pack expansion eredményének típusa a kontextustól függ: lehet függvényargumentum-lista, template argumentumlista, inicializáló lista, öröklési lista vagy kifejezés.

A pack nem futásidejű tömb. Nem lehet `Ts` típusú változót deklarálni, és nem lehet indexelni. A pack elemei csak kifejtve, például fold expressionnel, rekurzióval vagy `std::index_sequence`-szel érhetők el.

27.4.2 Fold expression és üres pack

A **fold expression** (C++17) a pack expansion egy olyan formája, amely a pack elemeit egy bináris operátorral kombinálja. Négy szintaktikai alak létezik:

$$\begin{aligned} (E\ op\ \ldots) &\quad \text{unary right fold},\\ (\ldots\ op\ E) &\quad \text{unary left fold},\\ (E\ op\ \ldots\ op\ I) &\quad \text{binary right fold},\\ (I\ op\ \ldots\ op\ E) &\quad \text{binary left fold}. \end{aligned}$$

Itt `E` a packot tartalmazó minta, `I` pedig egy inicializáló kifejezés. A zárójelezés levezetése (vázlat):

- `(E op ...)` jobbra zárójelez: `E1 op (E2 op (... op EN))`.
- `(... op E)` balra zárójelez: `((E1 op E2) op ...) op EN`.
- `(E op ... op I)` jobbra zárójelez: `E1 op (E2 op (... op (EN op I)))`.
- `(I op ... op E)` balra zárójelez: `((I op E1) op E2) ... op EN`.

A zárójelezés determinisztikus, de ez nem jelenti azt, hogy az operandusok kiértékelési sorrendje is mindig rögzített; erre a következő alfejezetekben visszatérünk.

A fold expression csak a megengedett operátorokkal használható. Ide tartoznak többek között a `+`, `-`, `*`, `/`, `%`, `&`, `|`, `^`, `<<`, `>>`, `&&`, `||`, `,`, az összehasonlító operátorok és a hozzárendelő operátorok. A `<=>` nem szerepel a fold operátorai között, ezért spaceship operátorral nem foldolhatunk.

Az **üres pack** kezelése operátorfüggő. Unary fold üres packra csak három operátornál érvényes (C++20 [temp.variadic]/9):

- `(args && ...)` üres packra `true`,
- `(args || ...)` üres packra `false`,
- `(args, ...)` üres packra `void()`.

Minden más operátornál — a `+` és a `*` esetén is — az üres pack unary folddal fordítási hiba; a szabvány nem ad `0`-t vagy `1`-et. Binary foldnál az `I` inicializáló érték kötelező, és üres pack esetén az eredmény maga az `I`; ezért ha üres packot is támogatni kell, a kezdőértékes `(0 + ... + args)` és `(1 * ... * args)` a helyes alak.

```
// fold_empty.cpp - C++20: üres pack és fold
template <class... Ts> constexpr auto sum(Ts... xs)     { return (0 + ... + xs); }
template <class... Ts> constexpr auto product(Ts... xs) { return (1 * ... * xs); }
```

```cpp
template <class... Ts> constexpr bool all(Ts... xs)     { return (xs && ...); }
template <class... Ts> constexpr auto sum_unary(Ts... xs) { return (xs + ...); }

static_assert(sum() == 0 && sum(1, 2, 3) == 6);
static_assert(product() == 1 && product(2, 3, 4) == 24);
static_assert(all() && all(true, true) && !all(true, false));
static_assert(sum_unary(1, 2) == 3);
#ifdef EMPTY_UNARY
static_assert(sum_unary() == 0);       // hiba: üres pack unary + folddal
#endif
int main() {}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic fold_empty.cpp` figyelmeztetés nélkül fordul; `-DEMPTY_UNARY` mellett a `sum_unary() fold of empty expansion over operator+` hibát ad.

Fontos példa a rövidzárlatra és az asszociativitásra:

```cpp
template <class... Ts>
bool all_positive(Ts... xs) {
    return (true && ... && (xs > 0));
}
```

Ez bal oldali binary fold: `((true && (x1 > 0)) && (x2 > 0)) && ...`. Az üres pack eredménye `true`. A `&&` rövidzárlatos, így ha egy korábbi elem hamis, a későbbi `xs > 0` kifejezések nem értékelődnek ki. Ugyanez `||` esetén is igaz. A `+` operátor nem rövidzárlatos, és az operandusok kiértékelési sorrendje sem garantált.

27.4.3 Index sequence és tuple indexelés

A `std::index_sequence<Is...>` (C++14) egy olyan típus, amelynek template argumentumlistája `0, 1, ..., N-1`. A `std::make_index_sequence<N>` és a `std::index_sequence_for<Ts...>` segítségével generálhatunk indexsorozatot. A tuple elemeihez compile-time indexkel, a `std::get<Is>(t)` hívással férünk hozzá.

```cpp
template <class Tuple, std::size_t... Is>
void print_tuple_impl(const Tuple& t, std::index_sequence<Is...>) {
    ((std::cout << (Is == 0 ? "" : ", ") << std::get<Is>(t)), ...);
}

template <class... Ts>
void print_tuple(const std::tuple<Ts...>& t) {
    print_tuple_impl(t, std::index_sequence_for<Ts...>{});
}
```

A `print_tuple_impl` második paramétere névtelen, mert csak a típusából nyerjük ki az `Is...` packot. A `std::get<Is>(t)` a `t` const lvalue referenciáján keresztül const referenciát ad az adott elemre. A `std::get` indexe compile-time ellenőrzött: ha `Is >= sizeof...(Ts)`, a program nem fordul. A `std::apply` (C++17) ugyanezt a mintát használja, de `std::invoke`-on keresztül hívja a callable-t, így tagfüggvénypointereket is kezel.

A tuple indexelésnél a value category számít. Ha `t` lvalue, a `std::get<Is>(t)` lvalue referenciát ad; ha `t` rvalue, akkor rvalue referenciát. A `std::apply(f, t)` a `t` value category-ját továbbítja, így a callable megkaphatja a tuple elemeit a megfelelő value category-vel.

27.4.4 Forwarding pack és argumentum lifetime

A `template <class... Args> void f(Args&&... args)` egy **forwarding reference pack**. Minden `Args` a hívás argumentumából dedukálódik, és a value category megőrzéséhez a `std::forward<Args>(args)...` szükséges:

```
template <class... Args>
void forward_to_sink(Args&&... args) {
    sink(std::forward<Args>(args)...);
}
```

A `std::forward<Args>(args)...` mintája minden argumentumra a saját `Args`-át használja, így az lvalue argumentum lvalue marad, az rvalue argumentum rvalue marad. A `std::move(args)...` ezzel szemben feltétel nélkül rvalue-vé tesz, és elveszíti az lvalue/rvalue különbséget; forwarding packnél ez hibás.

Az argumentum élettartama szempontjából kulcsfontosságú, hogy a forwarding reference nem hosszabbítja meg a temporary élettartamát a függvényhíváson túl. A temporary a teljes kifejezés végéig él, így a `forward_to_sink` hívása alatt a `sink` számára is érvényes. Ha azonban a packot eltároljuk, már nem hivatkozhatunk a temporary-re:

```
template <class... Args>
auto make_refs(Args&&... args) {
    return std::tuple<Args&&...>(std::forward<Args>(args)...);
}

auto t = make_refs(std::string("x")); // a temporary a teljes kifejezés végén
 ↪  megszűnik
// std::get<0>(t) használata érvénytelen élettartamra hivatkozik
```

Itt a `std::string("x")` temporary a teljes kifejezés végén megszűnik, a `t` azonban rvalue referenciát tárol rá. A `t` létrehozása önmagában nem UB, de a `std::get<0>(t)` használata dangling referencián keresztül történik, ami nem definiált viselkedés. A tulajdonos ebben az esetben nem a `t`, hanem a temporary volt; a temporary megszűnése invalidálja a referenciát.

A biztonságos tároláshoz a referenciákat értékekké kell alakítani:

```
template <class... Args>
auto make_owned(Args&&... args) {
    return std::make_tuple(std::forward<Args>(args)...);
}
```

A `std::make_tuple` a `std::decay_t<Args>...` típusokat tárolja, így a temporary értéke bemásolódik vagy mozgatódik, és a visszatérő tuple független a bemeneti temporary élettartamától. Ugyanezt teszi a `std::tuple<std::decay_t<Args>...>` is. A `const Ts&...` pack olvasásra biztonságos a híváson belül, de ha a referenciákat eltároljuk, ugyanígy dangling veszély áll fenn.

27.4.5 Kiértékelési sorrend és operátorfüggés

A fold zárójelezése determinisztikus, de az operandusok kiértékelési sorrendje az operátor szabályaitól függ. A `,` operátor balról jobbra értékel, és a bal operandus értéke után a jobb operandus értékelődik; a `&&` és `||` rövidzárlatos és sorrendezett. A `+`, `*`, `<<` és hasonló operátorok esetén az operandusok kiértékelési sorrendje nem specifikált, még akkor sem, ha a fold zárójelezése rögzített.

Ezért a logger példában a `,` foldot használjuk, ha a kiírás sorrendje számít:

```
template <class... Args>
void log_line(std::string_view level, const Args&... args) {
    std::cout << '[' << level << "] ";
    ((std::cout << args << ' '), ...);
    std::cout << '\n';
}
```

A `((std::cout << args << ' '), ...)` unary jobbra zárójelező fold a `,` operátorral. A `,` balról jobbra értékel, így a `std::cout` kiírások sorrendje garantált. Egy `(f(args) + ...)` fold esetén az `f` hívások sorrendje nem garantált, még akkor sem, ha a `+` zárójelezése balról jobbra történik. A `&&` és `||` foldoknál a rövidzárlat miatt a későbbi elemek ki sem értékelődhetnek; ez akkor fontos, ha a minta mellékhatással jár.

27.4.6 Rekurzió vagy fold: compile-time költség

A C++11-es variadikus sablonok tipikus feldolgozása rekurzióval történt:

```
template <class T>
void print(const T& x) { std::cout << x; }

template <class T, class... Rest>
void print(const T& x, const Rest&... rest) {
    std::cout << x << ' ';
    print(rest...);
}
```

Ez lineáris példányosítási mélységet okoz: minden argumentumhoz egy új `print` példány jön létre, és a rekurzió mélysége az argumentumok számával nő. Nagy pack esetén a compiler template-rekurziós limitjébe ütközhetünk (GCC/Clang esetén például a `-ftemplate-depth` értéke korlátozhat).

A fold expression ugyanezt egyetlen kifejezéssel fejezi ki:

```
template <class... Ts>
void print(const Ts&... xs) {
    ((std::cout << xs << ' '), ...);
}
```

A fold elkerüli a rekurziós mélységet, de a kifejezés mérete továbbra is lineáris a pack elemszámában. A compile-time költség nem feltétlenül kisebb; a példányosított kifejezés mérete és a compiler optimalizálási munkája compiler- és kódfüggő. A `std::index_sequence` használata szintén lineáris példányosítást eredményez, de gyakran áttekinthetőbb, mint a rekurzió. Mérést nem állítunk; a választás a konkrét fordító és kódbázis tapasztalata alapján történik.

27.4.7 Kidolgozott példa: logger és tuple-adapter

Az alábbi példa a `log_line` függvényt `const Args&...` packkal, a tuple kiíratását pedig `std::index_sequence`-szel valósítja meg. A kód C++20-ban fordul, GCC-vel vagy Clang-gel.

```cpp
// variadic_logger.cpp
#include <cstddef>
#include <iostream>
#include <string>
#include <string_view>
#include <tuple>
#include <utility>

template <class... Args>
void log_line(std::string_view level, const Args&... args) {
    std::cout << '[' << level << "] ";
    ((std::cout << args << ' '), ...);
    std::cout << '\n';
}

template <class Tuple, std::size_t... Is>
void print_tuple_impl(const Tuple& t, std::index_sequence<Is...>) {
    ((std::cout << (Is == 0 ? "" : ", ") << std::get<Is>(t)), ...);
}

template <class... Ts>
void print_tuple(const std::tuple<Ts...>& t) {
    print_tuple_impl(t, std::index_sequence_for<Ts...>{});
}

int main() {
    log_line("INFO", "start", 42, 3.5);

    auto t = std::make_tuple(std::string("db"), 5432, true);
    print_tuple(t);
    std::cout << '\n';
}
```

A buildparancs:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 variadic_logger.cpp -o
 ↪  variadic_logger
./variadic_logger
```

Clang esetén ugyanez `clang++`-szal. A várható kimenet illusztráció:

```
[INFO] start 42 3.5
db, 5432, 1
```

A `log_line` a `const Args&...` packot használja, mert csak olvas; a temporary argumentumok élettartama a hívás teljes kifejezéséig tart, így a kiírás alatt érvényesek. A függvény nem tárolja el a referenciákat, ezért nincs dangling. A `print_tuple` a tuple típusparamétereiből generál `0..N-1` indexeket, és a `std::get<Is>` hívással éri el az elemeket. A `std::make_tuple` a `main`-ben értékeket tárol, így a tuple független a temporary-ktől.

A példa nem használ C++20-as constraint-et. Ha a `log_line`-nak nem streamelhető típust adunk, a hiba a `<<` kifejezésnél jelentkezik, nem az interfésznél. C++20 concept-tel a hiba a hívás helyére hozható; ennek részletei a Concepts, requires és constrained overload részben találhatók.

27.4.8 Gyakori hibák és javításuk

- **A `...` helye.** A `std::forward<Args>(args)...` helyes; a `std::forward<Args...>(args)...` hibás, mert a `...` a template argumentumlistára kerül, nem a függvényargumentum-mintára.
- **A pack mérete és az elemek mérete.** A `sizeof...(args)` a pack elemszáma; a `sizeof(args)...` minden egyes elemre a `sizeof` értéket adja, nem a pack méretét.
- **`std::move` forwarding packnél.** A `std::move(args)...` mindent rvalue-vá tesz, és elveszíti a value category-t. Helyes: `std::forward<Args>(args)...`.
- **Referenciát tároló tuple.** A `std::tuple<Args&&...>` vagy `std::tuple<const Ts&...>` dangling referenciát tárolhat, ha a temporary a teljes kifejezés végén megszűnik. Tároláshoz `std::make_tuple` vagy `std::decay_t` szükséges.
- **Üres pack unary folddal.** A `(args + ...)`, `(args * ...)` vagy `(args - ...)` üres packra nem fordul; unary foldnál csak a `&&`, a `||` és a `,` kap értéket üres packra. Binary folddal, például `(0 + ... + args)` alakban kezelhető.
- **Két pack egy mintában.** Az `f(args..., rest...)` csak akkor érvényes, ha a két pack hossza megegyezik; különben fordítási hiba.
- **Nem streamelhető típus a loggerben.** A `log_line` a `<<`-nél ad hibát; a constraint-tel a hiba a hívás helyére hozható, és az interfész szerződése pontosabb lesz.

27.4.9 Feladatok

1. Magyarázd el, mi a különbség a `const Ts&...` és a `Ts&&...` pack között a value category és az élettartam szempontjából. Miért nem használható a `std::move(args)...` a forwarding pack továbbítására?

2. Írj `sum` és `all_positive` függvényt fold expressionnel, üres pack kezeléssel, és adj `static_assert`-eket. A `sum` üres packra 0-t, az `all_positive` üres packra `true`-t adjon.

3. Az alábbi `make_refs` példa dangling referenciát eredményez. Magyarázd meg, hol szűnik meg a temporary, ki a tulajdonos, és javítsd `std::make_tuple`-lel.

```
template <class... Args>
auto make_refs(Args&&... args) {
    return std::tuple<Args&&...>(std::forward<Args>(args)...);
}
auto t = make_refs(std::string("x"));
```

4. Írj `apply_to_tuple` függvényt `std::index_sequence`-szel, amely egy callable-t és egy tuple-t kap, és meghívja a callable-t a tuple elemeivel. Hasonlítsd össze a `std::apply`-jal.

5. A `log_line` példában miért garantált a kiírás sorrendje a `,` folddal, és miért nem garantált egy `+` foldnál? Adj példát `&&` folddal történő rövidzárlatra.

27.4.10 Megoldások

27.4.10.1 Első feladat

A `const Ts&...` pack minden argumentumot const lvalue referenciaként köt, és a temporary élettartamát a hívás teljes kifejezéséig meghosszabbítja. Ez olvasásra biztonságos, de a referenciák eltárolása danglinghez vezet. A `Ts&&...` forwarding reference pack minden argumentumnál megőrzi a dedukált típust és a value category-t; a továbbításhoz `std::forward<Args>(args)...` kell. A `std::move(args)...` feltétel nélkül rvalue-vé tesz, így az lvalue argumentumok is rvalue-ként továbbítódnak, és elveszik a value category. A helyes minta:

```cpp
template <class... Args>
void forward_to_sink(Args&&... args) {
    sink(std::forward<Args>(args)...);
}
```

27.4.10.2 Második feladat

```cpp
#include <concepts>

template <std::integral... Ts>
constexpr auto sum(Ts... xs) {
    return (0 + ... + xs);
}

template <std::integral... Ts>
constexpr bool all_positive(Ts... xs) {
    return (true && ... && (xs > 0));
}

static_assert(sum() == 0);
static_assert(sum(1, 2, 3) == 6);
static_assert(all_positive());
static_assert(all_positive(1, 2, 3));
static_assert(!all_positive(1, -2, 3));
```

A `sum` bal oldali binary fold: `((0 + x1) + x2) + ...`; üres packra az iniciáló `0` marad. Az `all_positive` bal oldali binary fold: `((true && (x1 > 0)) && ...)`; üres packra `true`. A `std::integral` constraint C++20-as; a `static_assert`-ek a fordítási időben ellenőrzik az eredményt.

27.4.10.3 Harmadik feladat

A `std::string("x")` temporary a `make_refs` hívását tartalmazó teljes kifejezés végén megszűnik. A `make_refs` visszatérő `std::tuple<std::string&&>` rvalue referenciát tárol a temporary-re. A `t` létrehozása után a temporary már nem él, így a `t` dangling referenciát tartalmaz. A tulajdonos a temporary volt; a `t` nem tulajdonos, csak nem tulajdonló hivatkozást tárol. A temporary megszűnése invalidálja a referenciát; a `std::get<0>(t)` használata nem definiált viselkedés.

A javított változat:

```cpp
#include <tuple>
#include <string>
#include <utility>
#include <iostream>

template <class... Args>
auto make_owned(Args&&... args) {
    return std::make_tuple(std::forward<Args>(args)...);
}

int main() {
    auto u = make_owned(std::string("x")); // biztonságos: értéket tárol
```

```
    std::cout << std::get<0>(u) << '\n';
}
```

A `std::make_tuple` a `std::decay_t<Args>...` típusokat tárolja, így a `std::string` bemásolódik vagy mozgatódik, és a `u` független a temporary élettartamától.

27.4.10.4 Negyedik feladat

```
#include <tuple>
#include <utility>
#include <type_traits>
#include <cstddef>
#include <iostream>

template <class F, class Tuple, std::size_t... Is>
decltype(auto) apply_impl(F&& f, Tuple&& t, std::index_sequence<Is...>) {
    return std::forward<F>(f)(std::get<Is>(std::forward<Tuple>(t))...);
}

template <class F, class Tuple>
decltype(auto) apply_to_tuple(F&& f, Tuple&& t) {
    return apply_impl(
        std::forward<F>(f), std::forward<Tuple>(t),
        std::make_index_sequence<
            std::tuple_size_v<std::remove_reference_t<Tuple>>>{});
}

int main() {
    auto t = std::make_tuple(1, 2, 3);
    auto add = [](int a, int b, int c) { return a + b + c; };
    std::cout << apply_to_tuple(add, t) << '\n';
}
```

Az `apply_impl` a `std::get<Is>(std::forward<Tuple>(t))...` pack expansionnal adja át a tuple elemeit a callable-nak. A `std::forward<Tuple>(t)` megőrzi a tuple value category-ját, így rvalue tuple esetén a `std::get` rvalue referenciát ad. A `std::apply` ehhez képest `std::invoke`-ot használ, így tagfüggvénypointereket és `std::reference_wrapper`-t is kezel; a saját változat közvetlen hívást végez, ezért egyszerűbb, de szűkebb.

27.4.10.5 Ötödik feladat

A `,` operátor balról jobbra értékel, ezért a `((std::cout << args << ' '), ...)` foldban a kiírások sorrendje garantált. A `+` operátor operandusai között nincs sorrendezés, ezért egy `(f(args) + ...)` foldban az `f` hívások sorrendje nem garantált, még akkor sem, ha a `+` zárójelezése balról jobbra történik. A `&&` és `||` rövidzárlatos; például:

```
template <class... Ts>
bool all_true(Ts... xs) {
    return (true && ... && xs);
}
```

Ha az első `xs` hamis, a későbbi elemek kiértékelése a `&&` szabályai szerint elmarad. Mellékhatásos mintánál ez megfigyelhető:

```
template <class... Fs>
bool any_true(Fs... fs) {
    return (false || ... || fs());
}
```

A `false || ... || fs()` bal oldali binary fold; az `fs()` hívások balról jobbra, rövidzárlattal értékelődnek. Az első `true` után a többi `fs()` nem hívódik meg.

27.4.10.6 Ellenőrzési feltétel

A megoldások akkor helyesek, ha a kód C++20-ban fordul, a `static_assert`-ek teljesülnek, a `make_refs` javított változata nem tárol referenciát temporary-re, és az `apply_to_tuple` a tuple elemeit a megfelelő value category-vel adja át. A `log_line` és a tuple-adapter példája nem tartalmaz eltárolt referenciát a hívás után.

27.4.11 Összefoglaló

- A parameter pack nem futásidejű tömb, hanem fordítási idejű típus- és értékcsoport; a pack expansion a mintát minden elemre kifejti.
- A fold expression (C++17) unary vagy binary formában kombinálja a pack elemeit; a zárójelezés determinisztikus; üres packra unary folddal csak a `&&` (`true`), a `||` (`false`) és a `,` (`void()`) érvényes, minden más esetben kezdőértékes binary fold kell.
- A `std::index_sequence` és a `std::get<Is>` a tuple compile-time indexelését teszi lehetővé; a `std::apply` ezt `std::invoke`-on keresztül általánosítja.
- A forwarding pack `std::forward<Args>(args)...`-tal őrzi meg a value category-t; a `const Ts&...` olvasásra biztonságos, de eltárolt referenciaként dangling lehet. Tároláshoz `std::make_tuple` vagy `std::decay_t` kell.
- A `,`, `&&` és `||` operátorok sorrendezettek, illetve rövidzárlatosak; a `+` és hasonló operátorok operandusainak kiértékelési sorrendje nem garantált.
- A fold elkerüli a rekurziós mélységet, de a kifejezés mérete továbbra is lineáris; a compile-time költség compiler- és kódfüggő, mérést nem állítunk.

27.5 Customization point, CRTP és policy design

Ez a fejezet a statikus és dinamikus polimorfizmus közötti mérnöki döntésről szól, nem trükkök gyűjteményéről. A cél az, hogy a generikus kódban meg tudd nevezni azt a pontot, ahol a felhasználó viselkedést injektál, és hogy ez a pont szűk, dokumentált, ellenőrizhető szerződés legyen. A C++20 alapú tárgyalás épít a korábbi részekre: a virtuális dispatch költségmodelljére (Virtual, override, final és dispatch), a kétfázisú névkeresésre (Dependent name, kétfázisú lookup és ADL), valamint a variadikus sablonokra (Variadic template, pack és fold expression).

27.5.1 Dynamic és static polymorphism: a döntési térkép

A **dinamikus polimorfizmus** (dynamic polymorphism) lényege, hogy a hívott műveletet futásidőben, a dinamikus típus alapján választjuk ki; C++-ban ennek kanonikus formája a virtuális függvény és a vtable-n keresztüli indirekt hívás. A **statikus polimorfizmus** (static polymorphism) ezzel szemben fordítási időben rögzíti a konkrét típust; C++-ban sablonparaméter, `if constexpr`, `concept`, CRTP vagy policy-compozíció formájában jelenik meg.

A döntés nem esztétikai. Négy tengelyen kell megnézni:

szempont	dinamikus (virtuális)	statikus (sablon/CRTP/policy)
híváskölts ég	indirekt hívás, gyakran nem inline-olható	közvetlen hívás, inline-olható
típusok halmaza	futásidőben nyílt, új típusú library plugin is csatlakozhat	fordítási időben zárt, minden típus ismert a TU-ban
heterogén tárolás	`std::unique_ptr<Base>` konténerben	nem lehetséges egyetlen konténerben
ABI	stabil: a vtable-elrendezés és a névtorzítás rögzíthető	instanciációnként új szimbólum, nem stabil határ
kódeloszlás	egy `f` kódpéldány	minden `T`-re külön kódpéldány
diagnosztika	fordítási hiba ritka, futásidejű hiba lehet	fordítási hiba, hosszú template-üzenetek

A gyakorlati szabály: **a dinamikus határ ott van, ahol a felhasználó kódjának nem kell újrafordulnia** (plugin, shared library, nyelvi határ). A statikus határ ott van, ahol a teljesítmény és az inline-olhatóság dominál, és a felhasználó amúgy is a te fejlesztői környezetedben fordít. A rossz döntés tipikus jele: virtuális interfész olyan helyen, ahol a teljes típushalmaz egy headerben ismert (felesleges indirekció), vagy sablon olyan helyen, ahol a fogyasztó binárisát nem akarod újrafordítani (ABI-törés).

27.5.2 CRTP: statikus interfész a leszármazottal

A **CRTP** (curiously recurring template pattern) formája:

```
template <class Derived>
class Base {
public:
    void interface() { static_cast<Derived*>(this)->implementation(); }
};
class Concrete : public Base<Concrete> {
public:
    void implementation();
};
```

A minta lényege, hogy a bázis sablonparamétere a leszármazott, így a bázis fordítási időben ismeri a konkrét osztályt, és a `static_cast` zéró költségű lefelé konvertálás. Nincs vtable, nincs indirekció, a hívás inline-olható. Cserébe a `Base<Derived>` és a `Derived` egyetlen típuspár, nem lehet `Base<A>` és `Base<B>` mutatóit egy konténerben tárolni.

Teljes, önálló példa:

```cpp
// crtp_sink.cpp
#include <cstddef>
#include <cstdio>

template <class Derived>
class PacketSink {
public:
    std::size_t write(const char* data, std::size_t n) {
        return static_cast<Derived*>(this)->write_impl(data, n);
    }
protected:
    ~PacketSink() = default;   // nem kell virtuális destruktor
};

class CountingSink : public PacketSink<CountingSink> {
public:
    std::size_t write_impl(const char* /*data*/, std::size_t n) {
        total_ += n;
        return n;
    }
    std::size_t total() const { return total_; }
private:
    std::size_t total_ = 0;
};

int main() {
    CountingSink s;
    PacketSink<CountingSink>& ref = s;      // statikus felület
    std::size_t w = ref.write("abcd", 4);
    std::printf("w=%zu total=%zu\n", w, s.total());
    return (w == 4 && s.total() == 4) ? 0 : 1;
}
```

Build:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 crtp_sink.cpp -o crtp_sink
# vagy
clang++ -std=c++20 -Wall -Wextra -Wpedantic -O2 crtp_sink.cpp -o crtp_sink
```

A várt kimenet `w=4 total=4`, és a kilépési kód 0. A `write_impl` **nyilvános** kell legyen, mert a hozzáférést a `PacketSink` kontextusában ellenőrzi a fordító; ha privát szeretnéd tartani, a bázist `friend`-dé kell tenni. Ez a leggyakoribb CRTP-hiba.

27.5.2.1 Az incomplete derived type csapdái

A `Base<Derived>` osztály törzsének deklarációi akkor instanciálódnak, amikor a `Derived` definíciója éppen fut — vagyis a `Derived` ekkor **befejezetlen** (incomplete). Ebből két szigorú következmény adódik:

- A bázis **osztálytörzsében** (tagdeklarációban, alapértelmezett template-argumentumban, nem típusú paraméter kifejezésében) nem lehet `sizeof(Derived)`, `alignof(Derived)` vagy olyan kifejezés, amely a `Derived` teljességét igényli. Ez ill-formed, nem „csak figyelmeztetés”.

- A **tagfüggvény-törzsekben** a helyzet más: azokat a szabvány csak akkor instanciálja, amikor használják, így ott a `Derived` már teljes. Ezért kerül a `static_cast` a függvénytörzsbe, nem a deklarációba.

```
template <class D>
struct Bad {
    static_assert(sizeof(D) > 0, "D nem teljes itt");  // fordítási hiba
};
struct X : Bad<X> { int v; };
```

A másik csapda az élettartam: a bázis konstruktorában vagy destruktorában `static_cast<D*>(this)->...` hívása **nem definiált viselkedés** (undefined behavior, UB). A konstruktor idején a `Derived` még nem kezdte meg élettartamát, a destruktor idején pedig már befejezte; a mutató nem mutat `D` típusú objektumra. Ha inicializálás után kell viselkedés, azt ne a báziskonstruktorban, hanem egy külön `init()`/`finalize()` hívásban vagy a `Derived` konstruktorában intézd.

27.5.3 Policy template és kompozíció

A **policy** egy sablonparaméter, amely egy szűk viselkedést (ellenőrzés, allokáció, növekedési stratégia, összehasonlítás) injektál. A standard könyvtárban ilyen a `std::char_traits`, a `std::allocator`, a `std::hash`, a `std::less`.

```
// policy_span.cpp
#include <cstddef>
#include <stdexcept>

struct checked_policy {
    static void verify(std::size_t i, std::size_t n) {
        if (i >= n) throw std::out_of_range{"index"};
    }
};
struct unchecked_policy {
    static constexpr void verify(std::size_t, std::size_t) noexcept {}
};

template <class Checking = checked_policy>
class IntSpan {
public:
    IntSpan(int* data, std::size_t n) noexcept : data_{data}, n_{n} {}
    int& operator[](std::size_t i) { Checking::verify(i, n_); return data_[i]; }
    std::size_t size() const noexcept { return n_; }
private:
    int* data_;
    std::size_t n_;
};
```

Build: `g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -c policy_span.cpp -o policy_span.o`.

A **kompozíció vs. öröklés** kérdése itt konkrét: az állapot nélküli policy-tagot `[[no_unique_address]]`-szel tagként tartva (C++20) vagy üres bázisként örökölve is el lehet helyezni nulla mérettel (empty base optimization, EBO). Az öröklés rövidebb hívásokat ad, de a policy tagjai bekerülnek az osztály névterébe, és a típus ABI-ja jobban függ a policy-től. A tagként tartás tisztább, de a híváshoz `policy_.verify(...)` kell — ha a policy állapotmentes, a fordító ezt is eltünteti.

A követelmény mindkét esetben dokumentálandó: a policy **statikus `verify(std::size_t, std::size_t)` tagfüggvényt** adjon, amely hibánál eldob (`checked`, azaz `throw`), egyébként visszatér. Ez a szerződés a policy nevében és a fejléc kommentjében is szerepeljen, mert a sablon nem kényszeríti ki.

27.5.4 Testreszabási pontok: tag, ADL és CPO

A **customization point** (testreszabási pont) az a jól megnevezett hely, ahol a könyvtár a felhasználó típusára vonatkozó viselkedést kéri. Két ősi forma létezik, ezekből lett a C++20-as **CPO** (customization point object).

27.5.4.1 Tagfüggvény

A legegyszerűbb: a felhasználó tagfüggvényt ír, a könyvtár meghívja. Előnye, hogy kézenfekvő, és a hozzáférés-vezérlést a nyelv kezeli; hátránya, hogy a konverziók miatt egy „majdnem megfelelő” típus is átcsúszhat, és a tag nem örökölhető felül írás nélkül.

27.5.4.2 ADL-alapú szabad függvény

A klasszikus minta a `swap`:

```
namespace my {
struct Buf { int* p; std::size_t n; };
void swap(Buf& a, Buf& b) noexcept;   // ADL innen találja meg
}

template <class T>
void my_swap(T& a, T& b) {
    using std::swap;    // kétlépéses lookup
    swap(a, b);         // ordinary lookup + ADL az instanciáláskor
}
```

A `using std::swap;` nélkül a `std::swap` nem látszana; a csupasz `std::swap(a, b)` viszont **megkerülné** a felhasználó `my::swap`-ját. A kétlépéses minta tehát kötelező, ha a generikus kódot akarod kiszolgálni.

27.5.4.3 Miért nem elég a csupasz ADL?

Mert a hívás a hívó kontextusától függ: ha a könyvtár egy belső segédfüggvényből hívja, ott az ADL halmaza más lehet, és egy nem várt túlterhelés részt vehet. A C++20 **CPO** ezt úgy zárja ki, hogy a testreszabási pontot egy **function object** (niebloid) mögé teszi, így a hívás helyén nincs szabad függvény, amit ADL kiválaszthatna. A standardban ilyen a `std::ranges::swap`, `std::ranges::begin`, `std::ranges::size`. Mellette a szabvány **explicit engedélyezett specializációkat** is használ: `std::hash<T>`, `std::numeric_limits<T>`, `std::formatter<T, char>`, `std::ranges::enable_borrowed_range<T>`. Ezek dokumentált, szűk szerződések: a specializációnak pontosan a megadott tagokat és jelentést kell adnia.

27.5.5 Concept-alapú extension point

A C++20 `concept` a testreszabási pont **szerződését** fejezi ki: megmondja, milyen kifejezésnek kell fordulnia, milyen visszatérési típussal. Ez nem pusztán dokumentáció, mert részt vesz a túlterhelés-feloldásban, és a hibaüzenetet a hívás helyére teszi.

```
// serialize_ext.cpp
#include <concepts>
#include <cstddef>
#include <cstdio>
#include <string>

template <class T>
concept MemberSerializer = requires(const T& t, char* out, std::size_t cap) {
    { t.serialize_to(out, cap) } -> std::same_as<std::size_t>;
};

template <class T>
concept AdlSerializer = requires(const T& t, char* out, std::size_t cap) {
    { serialize_to(t, out, cap) } -> std::same_as<std::size_t>;
};

namespace detail {
template <class T>
std::size_t dispatch(const T& t, char* out, std::size_t cap) {
    if constexpr (MemberSerializer<T>) {
        return t.serialize_to(out, cap);   // elsőbbség: tag
    } else {
        return serialize_to(t, out, cap);  // ADL
    }
}
}  // namespace detail

template <class T>
    requires MemberSerializer<T> || AdlSerializer<T>
std::size_t serialize(const T& t, char* out, std::size_t cap) {
    return detail::dispatch(t, out, cap);
}

struct Name {                              // tagfüggvényes testreszabás
    std::string v;
    std::size_t serialize_to(char* out, std::size_t cap) const {
        std::size_t n = v.size() < cap ? v.size() : cap;
        for (std::size_t i = 0; i < n; ++i) out[i] = v[i];
        return n;
    }
};

namespace metrics {                        // ADL-es testreszabás
struct Counter { int hits; };
inline std::size_t serialize_to(const Counter& c, char* out, std::size_t cap) {
    return static_cast<std::size_t>(std::snprintf(out, cap, "%d", c.hits));
```

```
}
}  // namespace metrics

int main() {
    char buf[32];
    Name n{"cafe"};
    std::size_t a = serialize(n, buf, sizeof buf);
    metrics::Counter c{42};
    std::size_t b = serialize(c, buf, sizeof buf);
    std::printf("%zu %zu\n", a, b);
}
```

Build: `g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 serialize_ext.cpp -o serialize_ext` (a várt kimenet `4 2`, mert a `42` két bájt). A szerződés akkor szűk, ha **három dolgot rögzít**: a pontos kifejezést (`t.serialize_to(out, cap)`), a visszatérési típust (`std::size_t`) és a részleges buffer szemantikáját (a példában „a lehető legtöbbet, a ténylegesen írt hosszt adja vissza"). A negyedik, opcionális elem a kivételgarancia: itt nincs dobás, de ezt ki kell mondani.

Figyelj rá, hogy az `AdlSerializer` az **argumentum típusának társított névterében** keres, nem abban, ahol a hívó van. Ha a `Widget` a `ui` névtérben van, akkor az `ui::serialize_to` hívódik meg, az `app::serialize_to` nem.

27.5.6 Költségek: code bloat, compile-time coupling, ABI

Minden sabloninstanciáció **külön típus**, saját szimbólumokkal. `Bounds<checked>`, `Bounds<unchecked>` két teljesen független típus: két kódpéldány, két `type_info`, két sorozat inline-olt hívást. Ez a **code bloat**. Mérséklése: (1) a policy-t ne vidd be a publikus interfész típusába, csak a belső megvalósításba; (2) egy nem sablonos magra delegálj, és a sablon csak a vékony ellenőrző burkot adja; (3) `extern template` + egy explicit instanciációs TU, hogy a gyakori specializáció ne instanciálódjon minden fordítási egységben. Ez a **compile-time coupling** másik oldala: ha a policy fejléce változik, minden őt instanciáló TU újrafordul.

ABI szempontból a kulcs: a **template specializációknak nincs stabil ABI-juk** két különböző fordító, sőt két verzió között sem. Ezért a könyvtár nyilvános határán ne jelents meg `template` osztályt, ha a fogyasztó binárisát nem akarod újrafordítani; a stabil határ `extern "C"` vagy nem sablonos, `pimpl`-es osztály. A virtuális interfész épp azért marad meg ott, ahol van stabil ABI-igény: a vtable elrendezés és a névtorzítás adott ABI (x86-64 System V) mellett specifikált, míg a sabloninstanciáció nem az.

27.5.7 Type erasure mint a harmadik út

A **type erasure** (típuseltüntetés) a statikus és a dinamikus világ hídja: kifelé értékszemantikás, konkrét típus (`AnySink`), belül egy virtuális `Concept` interfész és egy sablonos `Model<S>`. A `std::function`, a `std::any` és a saját típusú `Sink` mind ezt használja. Az előképet a lambdák és a `function` wrapper fejezet tárgyalja részletesen (invoke, function wrapper és type erasure); itt csak a döntési szabály kell:

- **Concept + sablon**: legjobb teljesítmény, nyílt extension point, de minden típus bekerül a fordítógépbe, és nincs stabil ABI.
- **Virtuális interfész**: stabil ABI, futásidejű heterogenitás, indirekció.
- **Type erasure**: értékszemantika és heterogenitás együtt, ára a `Model<T>` allokáció és a virtuális hívás.

27.5.8 Tipikus hibák

1. **Privát `impl` CRTP-ben.** A bázis nem fér hozzá; tedd nyilvánossá, vagy `friend`-dé.
2. **`static_cast<D*>(this)->...` a bázis konstruktorában/destruktorában.** UB; a `D` élettartama még nem kezdődött / már véget ért.
3. **`sizeof(D)` a bázis törzsében.** Ill-formed, mert a `D` befejezetlen.
4. **Csupasz `std::swap(a, b)` generikus kódban.** Megkerüli a felhasználó ADL-es `swap`-ját.
5. **Túl széles concept.** `requires(T t) { t.serialize(out, cap); }` elfogad olyan típust is, ahol implicit konverzió történik; írd ki a pontos szignatúrát és a visszatérési típust.
6. **Dokumentálatlan policy.** Ha a policy csak „valami callable”, a hiba a felhasználónál derül ki; adj neki conceptet.
7. **Policy a publikus ABI-ban.** Minden fogyasztó újrafordul, és két verzió között nincs kompatibilitás.

27.5.9 Feladatok

1. feladat (magyarázó + kód). Készíts `Comparable<D>` CRTP-mixint, amely `==`, `!=`, `<`, `<=`, `>`, `>=` operátorokat ad, feltéve hogy `D`-nek van `int compare(const D&) const` tagfüggvénye (-1/0/1 szemantikával). Fogalmazd meg a szerződést, és mondd meg, miért nem lehet `compare` privát.

2. feladat (kódolási). Készíts `Buffer<Growth>` osztályt, ahol a `Growth` policy statikus `grow(std::size_t current, std::size_t needed) -> std::size_t` függvényt ad. Valósíts meg két policyt: `double_policy` (amortizált, `max(2*current, needed)`) és `exact_policy` (`needed`). Indokold, melyiknél mennyi újraallokáció történik `n` egymást követő `push_back` során, és mit jelent ez a másolási költségre.

3. feladat (diagnosztikai). Adott a `serialize` extension point. A felhasználó a `Widget`-et a `ui` névtérben definiálja, de a `serialize_to`-t `namespace app`-ba tette. A kód nem fordul, vagy nem az ő függvénye hívódik. Magyarázd meg, miért, és add meg a javítást.

4. feladat (kiegészítő). A `PacketSink`-et szeretnéd egy `std::vector<std::unique_ptr<...>>`-ban heterogén módon tárolni. Melyik két megoldás jön szóba, és melyik milyen ABI- és teljesítménykompromisszummal jár?

27.5.10 Megoldások

1. megoldás. A minta a Barton–Nackman-trükk:

```
template <class D>
struct Comparable {
    friend bool operator==(const D& a, const D& b) { return a.compare(b) == 0; }
    friend bool operator!=(const D& a, const D& b) { return a.compare(b) != 0; }
    friend bool operator<(const D& a, const D& b)  { return a.compare(b) < 0; }
    friend bool operator<=(const D& a, const D& b) { return a.compare(b) <= 0; }
    friend bool operator>(const D& a, const D& b)  { return a.compare(b) > 0; }
    friend bool operator>=(const D& a, const D& b) { return a.compare(b) >= 0; }
};
```

A szerződés: `compare` **teljes rendezést** ad, tranzitív, antiszimmetrikus, reflexív egyenlőséggel; az `int` eredménye pontosan -1, 0 vagy 1. A `friend` itt nem barátság a `D` felé, hanem a nem-tag operátorok definiálása a bázison belül; ezek ADL-lel találhatók, és nem igényelnek hozzáférést. A `compare` **nem lehet privát**, mert a `friend` függvények a `Comparable` kontextusában nézik fel,

nem a `D`-ében. Ellenőrzési feltétel: `static_assert(std::totally_ordered<My>);` a `<compare>` fejlécből.

2. megoldás. A policy:

```
struct double_policy {
    static std::size_t grow(std::size_t current, std::size_t needed) {
        std::size_t twice = current == 0 ? 8 : current * 2;
        return twice < needed ? needed : twice;
    }
};
struct exact_policy {
    static std::size_t grow(std::size_t, std::size_t needed) { return needed; }
};
```

A `double_policy` mellett `n` `push_back` esetén az újraallokációk száma $O(\log n)$, mert a kapacitás mértani sorozatban nő; az összes másolt elem száma $\sum_k 2^k < 2n$, tehát **amortizáltan** $O(1)$ egy `push_back`. Az `exact_policy` minden lépésben `n` elemet másol, összesen $O(n^2)$ másolás. Ellenőrzési feltétel: a `grow` soha ne adjon `current`-nél kisebb értéket, és `needed`-nél kisebbet se, különben az invariáns sérül. A `current*2` túlcsordulását `current > SIZE_MAX/2` esetre kezeld.

3. megoldás. Az ADL a `Widget` **társított névtereiben** keres, azaz `ui`-ban (és a template-argumentumai névtereiben), nem a hívás helyén és nem `app`-ban. Mivel a `Widget` nem `app`-beli, az `app::serialize_to` nem látszik. Javítás: tedd a `serialize_to`-t a `ui` névtérbe (ez a szabványos és ajánlott), vagy — ha ez nem lehetséges — adj a `Widget`-nek tagfüggvényt, mert a `MemberSerializer` elsőbbséget élvez. Ellenőrzési feltétel: `static_assert(AdlSerializer<ui::Widget>);` a definíció után.

4. megoldás. Két út: (a) adj a `PacketSink<D>` mellé egy absztrakt `IPacketSink` bázist, amelynek virtuális `write`-ja van, és a `PacketSink<D>` ezt implementálja — ez heterogén tárolást ad, de visszahozza az indirekciót és a vtable-t, cserébe stabil ABI-t; (b) készíts `AnySink` type erasure-t, amely értékszemantikát ad, de allokál és virtuálisan hív. A statikus út (`std::variant` a zárt típushalmazzal) csak akkor járható, ha a típusok zártak; ez a leggyorsabb, de minden új típus újrafordul.

27.5.11 Összefoglaló

A statikus polimorfizmus fordítási időben zárja le a típushalmazt, cserébe közvetlen hívásokat, inline-olhatóságot és kisebb futásidejű overheadet ad — ára a kódeloszlás, a fordítási kapcsolódás és az instanciációnkénti ABI. A CRTP akkor hasznos, ha egy bázis statikus felületet akar adni a leszármazottnak; a két fő csapda az incomplete `Derived` a bázis törzsében és a `static_cast<D*>(this)` a konstruktorban/destruktorban. A policy template egy szűk viselkedést injektál; a kompozíció és az EBO közötti választás ABI- és olvashatósági kérdés. A testreszabási pontot három szinten lehet megadni: tagfüggvénnyel, ADL-lel (kétlépéses `using std::swap;` formában) és CPO-val, amely a hívást kiveszi az ADL hatóköréből. A C++20 `concept` a szerződést nem dokumentációként, hanem kikényszeríthető előfeltételként rögzíti. Ha a kódot binárisan akarod megosztani, a virtuális interfész vagy a type erasure a helyes választás; ha a teljes típushalmaz a te TU-dban fordul, a statikus út a gyorsabb és a biztonságosabb.

28 STL tárolók és érvényességi szerződések

28.1 std::array, std::vector és folytonos tárolás

A C dinamikus tömb mintáját——`T *buf`, `size_t len`, `size_t cap` és a hozzá tartozó kézi `free`——a Dinamikus tömb és string builder részben már felépítettük. A standard tárolók ugyanezt a hármast kínálják, de a mérethatárokat, az elemek élettartamát és a hibakezelést típusba kódolják. Ez a rész azt tanítja meg, hogyan válasszunk `std::array` és `std::vector` között, és hogyan döntsük el minden mutáció előtt, hogy mely pointer, referencia és iterator marad érvényes. A folytonos tárolás nem esztétikai kérdés: ez teszi lehetővé, hogy egy `std::vector` tartalmát változtatás nélkül adjuk át C API-nak, `read`/`write` rendszerhívásnak vagy egy bináris protokoll szerializálójának. A szerződés másik oldala viszont az, hogy a folytonosság és az újraelosztás együtt mozog: amint a vektor kinő egy allokációt, a korábbi `data()` pointer nem követi a tulajdonost.

28.1.1 std::array: fix méret és értékszemantika

A `std::array<T, N>` pontosan `N` darab `T` típusú elemet tartalmazó aggregate. Az `N` fordítási idejű konstans, a méret a típus része, és az objektum nem konvertálódik pointerré, ahogyan a C tömb függvényparaméterként elsöprűnik pointerré. Az `std::array` értékszemantikát ad: másoláskor minden elem másolódik, értékadáskor minden elem felülíródik. Ez a C tömbbel szemben a legfontosabb különbség, mert így a fix méretű puffer átadható érték szerint, visszaadható függvényből, és beágyazható más struktúrába anélkül, hogy elveszne a hossz.

A méretgarancia azonban nem jelent bitre pontos layout-garanciát. A `sizeof(std::array<T, N>)` tipikusan `N * sizeof(T)`, de a szabvány ezt nem írja elő; padding és belső elrendezés implementációfüggő lehet. A `data()` pointert ad az első elemre, a `size()` pedig fordítási idejű konstans, így az `std::array` kitűnő választás fix protokollfejlécekhez, hardverregiszter-tükrökhöz és olyan bufferokhoz, amelyek mérete a típusban él. Az `N = 0` megengedett speciális eset: ilyenkor `size() == 0`, `begin() == end()`, és a `data()` értéke nem használható dereferálásra.

Egy inicializálási csapda rendszeresen visszatér. A `std::array<int, 3> a;` default-inicializálja az aggregate-et, de az `int` elemek így indeterminált értékűek lesznek; olvasásuk nem definiált viselkedés. A `std::array<int, 3> a{};` value-initializál, tehát minden elem nulla lesz. Ugyanez a különbség a `std::vector`-nál is megjelenik, csak ott a `resize` és a `push_back` teszi láthatóvá. Ha a fix méretű tömböt függvénynek adjuk át, referenciát használjunk, különben a teljes tömb másolódik:

```
#include <array>
#include <cstddef>
#include <cstdint>
#include <cstdio>

void dump(const std::array<std::uint8_t, 4>& magic) {
    for (std::uint8_t b : magic) {
        std::printf("%02x ", static_cast<unsigned>(b));
```

```
    }
}

int main() {
    std::array<std::uint8_t, 4> magic{0x7f, 'E', 'L', 'F'};
    dump(magic);
    std::printf("size=%zu\n", magic.size());
}
```

A `magic.size()` típusa `std::size_t`, értéke fordítási idejű konstans, és a `dump` belsejében a `magic.size()` ugyanúgy `4`, mint a hívó oldalon. Ez az a tulajdonság, amit a C tömb elveszít, amikor paraméterként pointerré alakul.

28.1.2 std::vector: méret, kapacitás és konstruált elemek

A `std::vector<T>` három számot tart nyilván, és a programozási hibák nagy része abból fakad, hogy ezeket összekeverjük. A `size()` a konstruált elemek száma: a `[0, size())` indexű elemek léteznek, és mindegyikre lefutott a konstruktor. A `capacity()` a jelenlegi allokációban elférő elemek száma; a `capacity() >= size()` mindig teljesül, és a `size()` és `capacity()` közötti tartomány nyers tároló, ahol nem él `T` objektum. A `max_size()` elméleti felső korlát, a `data()` pedig az első elemre mutató pointer, amely üres vektornál lehet null is, de semmiképpen nem dereferálható. A folytonosság garantálja, hogy `&v[i] == v.data() + i`, és C++20-ban a `vector::iterator` contiguous iterator kategóriát modellez.

A C-s `buf/len/cap` hármashoz képest a döntő különbség a `len` és `cap` közötti rész státusza. C-ben a lefoglalt byte-ok mindegyike használható tároló, csak a `len`-en túli rész tartalma definiálatlan. C++-ban a `size()` és `capacity()` közötti memória nem tartalmaz `T` objektumot, ezért ott `T`-t olvasni vagy írni nem lehet. Ezért nem helyes a `v.capacity()`-t „kihasználható elemek számaként” kezelni, és ezért nem lehet a `reserve` után indexelni.

A `reserve(n)` csak a kapacitást érinti: ha `n > capacity()`, új tárolót foglal legalább `n` elemre, áthelyezi a meglévő elemeket, és minden pointert, referenciát és iterátort invalidál. Ha `n <= capacity()`, nem csökkent, nem konstruál elemeket, és a `size()` változatlan. A `reserve` tehát nem ad elemeket; a `v.reserve(8); v[0] = 1;` nem definiált viselkedés, mert a `size()` még nulla. A `resize(n)` ezzel szemben tényleg elemeket hoz létre vagy szüntet meg: növeléskor value-initializált (vagy megadott értékű) elemekkel bővül a vektor, csökkentéskor a végső elemek destruálódnak. A `resize` kisebbre állítás után sem csökken a `capacity()`; a `shrink_to_fit()` erre tesz kísérletet, de nem kötelező, hogy ténylegesen csökkentsen.

A `push_back` és az `emplace_back` a `size()`-t növeli. A `push_back(const T&)` másol, a `push_back(T&&)` mozgat, az `emplace_back(args...)` pedig helyben konstruálja az új elemet. Az `emplace_back` nem automatikusan jobb: ha már van egy `T` objektumunk, a `push_back` gyakran ugyanannyiba kerül, az `emplace_back` viszont nem tudja kiváltani a `{}`- inicializáló listát. A `std::vector<std::vector<int>> v; v.emplace_back(3, 4);` egy háromelemű belső vektort hoz létre, amelynek minden eleme `4`, mert a `(3, 4)` konstruktorargumentumnak számít; a `v.push_back({3, 4})` ezzel szemben két elemet ad. Ez a különbség gyakori félreértés forrása.

Ha a `size() == capacity()`, a `push_back` újraeloszt: új tárolót foglal, az elemeket mozgatással vagy másolással áthelyezi, a régi elemeket destruálja, a régi tárolót felszabadítja. A mozgatás akkor választandó, ha a `T` move konstruktora `noexcept`, vagy ha a másolás nem elérhető; ezt a noexcept, swap és container-reallocation részben tárgyaltuk részletesen, és a type trait-ek (Type traits, tag dispatch és if constexpr) teszik hozzáférhetővé a döntést. A `push_back` erős kivételgaranciát ad,

ha a `T` másolása vagy a nem dobó mozgatása rendelkezésre áll: kivétel esetén a vektor állapota változatlan marad.

A geometriai növekedés amortizált konstans költséget ad. Ha a növekedési faktor `k > 1` (a konkrét érték implementációfüggő, tipikusan 1,5 vagy 2), akkor `n` egymást követő `push_back` után az áthelyezett elemek száma legfeljebb `1 + k + k^2 + ... + n < n * k / (k - 1)`, ami `k = 2` esetén `2n`. Egyetlen `push_back` worst-case költsége tehát `O(n)`, de `n` műveletre átlagosan `O(1)`. Ez a különbség a Big-O-s táblázatokban elvész, és éppen ezért nem elegendő a tárolóválasztást csak aszimptotikára alapozni.

28.1.3 Invalidáció: melyik művelet mit tesz érvénytelenné

A következő táblázat a `std::vector` mutációinak hatását foglalja össze. A „mind” azt jelenti, hogy a korábbi `data()` pointer, referencia és iterator érvénytelenné válik; az „end()” a past-the-end iterátorra utal, amely nem dereferálható, de összehasonlítható.

Művelet	`size` / `capacity`	Pointer, referencia, iterator
`push_back`, `emplace_back`	`size` nő; ha `size == capacity`, a capacity nő	újraelosztás esetén mind; egyébként az `end()` és az új elem
`insert`	`size` nő, a capacity csak szükség esetén	újraelosztás esetén mind; egyébként a beszúrás helyétől
`erase`	`size` csökken, a capacity nem	a törölt elemek és az utána következők; a visszaadott iterator érvényes
`reserve(n)`, ha `n > capacity`	capacity nő, `size` változatlan	mind
`resize(n)`	`size` lesz `n`	újraelosztás esetén mind; csökkentésnél a törölt elemek
`clear()`	`size` nulla, capacity marad	mind
`shrink_to_fit()`	capacity csökkenhet	mind, ha ténylegesen csökkent
`swap`	nem változik	az elemekre mutató pointer/reference érvényes marad, de a másik konténer elemeire mutat

A `swap` különleges: az elemek nem mozdulnak el a memóriában, ezért a rájuk mutató pointer és referencia érvényben marad, csak most a másik vektor elemeire mutat. Az `end()` iterator ezt nem éli túl, és az iterátorok is a másik konténerhez tartoznak. A `std::array` esetében nincs újraelosztás, ezért nincs invalidáció sem; de ha az egész `std::array`-t értékül adjuk át vagy értékadással felülírjuk, a korábbi referenciák továbbra is az eredeti objektumra mutatnak, amelynek tartalma közben megváltozott.

28.1.4 std::vector<bool>: proxy és eltérő tárolási szerződés

A `std::vector<bool>` a standard könyvtár egyetlen olyan tárolója, amely nem adja ugyanazt a szerződést, mint a többi `vector`. A szabvány specializációt enged meg, és a tipikus implementáció bitenként tárolja az értékeket. Emiatt a `data()` tagfüggvény nem létezik, a `&v[0]` nem ad `bool*`-t, és a `v[i]` típusa nem `bool&`, hanem egy proxy osztály (`std::vector<bool>::reference`), amely `bool`-lá konvertálható. A `bool& r = v[0];` nem fordul le, az `auto x = v[0];` pedig

a proxyt másolja, nem `bool`-t. A `vector<bool>` tartalma nem adható át C API-nak `bool*`-ként; ha erre szükség van, `std::vector<char>`, `std::vector<std::uint8_t>` vagy fix méretre `std::bitset<N>` a megfelelő választás. A tömör tárolás előnye valós, de a folytonosság és a `data()` elvesztése tudatos döntést igényel.

28.1.5 Kidolgozott példa: fix fejléc és dinamikus payload

A következő példa egy protokollüzenetet modellez: a fejléc fix négy bájtos mágikus száma `std::array`, a változó hosszú tartalom `std::vector`. A C API-határ csak pointert és hosszt fogad, ezért a vektor `data()`-ját használjuk, de minden mutáció után újra lekérjük.

```
// cont_demo.cpp
#include <array>
#include <cstddef>
#include <cstdint>
#include <cstdio>
#include <vector>

struct Header {
    std::array<std::uint8_t, 4> magic{0x7f, 'E', 'L', 'F'};
    std::uint16_t version{1};
};

void write_bytes(const std::uint8_t* p, std::size_t n) {
    for (std::size_t i = 0; i < n; ++i) {
        std::printf("%02x", static_cast<unsigned>(p[i]));
        std::printf(i + 1 == n ? "\n" : " ");
    }
}

void show(const char* what, const std::vector<std::uint8_t>& v) {
    std::printf("%-14s size=%zu capacity=%zu\n", what, v.size(), v.capacity());
}

int main() {
    Header h;
    write_bytes(h.magic.data(), h.magic.size());

    std::vector<std::uint8_t> payload;
    payload.reserve(4);                  // capacity >= 4 (lehet több is)
    for (std::uint8_t b : {0xde, 0xad, 0xbe, 0xef}) {
        payload.push_back(b);            // size <= capacity: nincs
 ↪  újraallokálás
    }
    show("4 push_back:", payload);

    const std::uint8_t* raw = payload.data();
    write_bytes(raw, payload.size());    // érvényes: közben nincs mutáció

    // Feltöltés a MÉRT kapacitásig: az új méret nem lépi túl a kapacitást,
    // ezért ez még nem allokál újra, a raw érvényes marad.
```

```
    payload.resize(payload.capacity(), 0x00);
    show("resize(cap):", payload);

    payload.push_back(0x00);              // size == capacity volt: a szabvány
↪   szerint
    show("push_back:", payload);          // biztosan újraallokál
    // write_bytes(raw, payload.size()); // HIBA: raw dangling pointer
    write_bytes(payload.data(), payload.size()); // helyes: friss data()

    payload.erase(payload.begin());       // erase sosem allokál újra
    show("erase:", payload);
    write_bytes(payload.data(), payload.size());
}
```

A buildparancs rögzített nyelvi módot használ; a `-fsanitize=address,undefined` kapcsoló a tanuló laborjában opcionális, és a példa helyességét nem helyettesíti:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o cont_demo cont_demo.cpp
./cont_demo
```

Buildjegyzőkönyv (`lab-gcc` profil). A kimenet:

```
7f 45 4c 46
4 push_back:   size=4 capacity=4
de ad be ef
resize(cap):   size=4 capacity=4
push_back:     size=5 capacity=8
de ad be ef 00
erase:         size=4 capacity=8
ad be ef 00
```

A kapacitásértékek a `lab-gcc` profil libstdc++-ának mért értékei, nem szabványos garanciák. A példa tanulsága nem a kimenet, hanem a `raw` élettartama. A `reserve(4)` csak `capacity() >= 4`-et garantál, pontosan 4-et nem; ezért a demonstráció nem erre a számra épít, hanem a *mért* kapacitásig tölti fel a vektort (`resize(capacity())` — ez nem lépi túl a kapacitást, tehát nem allokál újra). Ezután a `size() == capacity()` állapotban a `push_back` a szabvány szerint biztosan újraallokál, és a `raw` a régi tárolóra mutat. Az `erase` soha nem allokál újra, a kapacitás megmarad. Ha a `raw`-t használnánk, a viselkedés nem definiált; a helyes minta az, hogy a mutáció után a `data()`-t újra lekérjük. Ugyanez a szabály érvényes akkor is, ha a vektort mozgatjuk vagy másoljuk: a korábbi pointer nem követi az új tulajdonost.

28.1.6 Tipikus hibák és javításuk

A leggyakoribb hiba a `reserve` és a `resize` összekeverése: a `reserve` után a `size()` még nulla, ezért az indexelés nem definiált. A második a `data()` pointer megőrzése olyan művelet után, amely újraelosztást okozhat; a javítás az, hogy a pointer élettartamát a mutációhoz kötjük, vagy a mutációt a pointer használata utánra tesszük. A harmadik az `erase` ciklusban: a `for (auto it = v.begin(); it != v.end(); ++it) v.erase(it);` a törlés után érvénytelen iterátort léptet; helyes az `it = v.erase(it)` vagy az erase–remove minta. A negyedik a `vector<bool>` `bool*`-ként való átadása; itt a proxy és a hiányzó `data()` a hiba forrása. Az ötödik a `std::array` default-inicializálása és az elemek olvasása; a `{}` használata value-initializál. A hatodik a `capacity()` mint „használható elemek száma”; a `capacity()` felső korlát, nem hossz. Végül a `std::array` méretének `sizeof`-ból való visszafejtése: a `size()` a mértékadó, a `sizeof` paddingot is tartalmazhat.

28.1.7 Összefoglaló

A `std::array` fix méretű, értékszemantikájú aggregate, amely megőrzi a hosszát, nem csúszik pointerré, és nem invalidálódik újraelosztás miatt. A `std::vector` folytonos tároló, amelynek `size()`-a a konstruált elemek száma, `capacity()`-a az allokált hely, és a kettő között nem élnek `T` objektumok. A `reserve` csak kapacitást ad, a `resize` elemeket hoz létre vagy szüntet meg. A `push_back` és az `emplace_back` újraelosztáskor minden pointert, referenciát és iterátort invalidál; az `erase` a törölt elemektől kezdve invalidál; a `clear` mindent invalidál, de a capacity-t megtartja. A `vector<bool>` speciális, proxy alapú tárolás, amely nem ad folytonos `bool` tömböt. A tárolóválasztás tehát nem Big-O-táblázat, hanem élettartam-, invalidációs és ownership-szerződés kérdése.

28.1.8 Feladatok

1. feladat (diagnosztikai). Az alábbi kódrészletben jelölje meg, mely pontokon nem definiált a viselkedés, és magyarázza el, miért nem lehet a későbbi sorokra „visszamenőleg” következtetni. Tegyük fel, hogy a `reserve(8)` ténylegesen legalább 8 elemre foglal.

```
std::vector<int> v;
v.reserve(8);
v[0] = 42;                 // (1)
int* p = v.data();
v.push_back(1);            // (2)
std::printf("%d\n", *p);   // (3)
```

2. feladat (kódolási). Írjon függvényt, amely egy `std::vector<int>`-ből eltávolítja a negatív elemeket, megtartja a többi elem sorrendjét, és a végén a vektort C API-nak adja át. A megoldás ne okozzon invalidációs hibát, és ne növelje feleslegesen a capacity-t. Adja meg a buildparancsot is.

3. feladat (tervezési). Tervezze meg egy protokollüzenet adattagjait úgy, hogy a négybájtos mágikus fejléc `std::array`, a változó hosszú tartalom `std::vector` legyen. Írjon függvényt, amely a fejlécet és a tartalmat egy C API-nak adja át, és sorolja fel, mely műveletek után kell a pointereket újra lekérni. Térjen ki arra is, mi történik, ha az üzenetet mozgatjuk.

28.1.9 Megoldások

1. megoldás. Az (1) pont nem definiált viselkedés: a `reserve(8)` nem növelte a `size()`-t, az érték nulla, ezért a `v[0]` olyan elemre hivatkozik, amely nem létezik. A (2) pont önmagában nem okoz újraelosztást, mert a `size()` 0-ról 1-re nő, és a capacity legalább 8; a `p` tehát érvényes marad. A (3) pont ebben a konkrét végrehajtásban az 1-es értéket olvasná, de a program már az (1) pontnál nem definiált, ezért a további viselkedésére nincs garancia. Az ellenőrzési feltétel: a válasz nevezze meg a `size()` és a `capacity()` különbségét, és ne állítsa, hogy a (3) önmagában UB, ha a (2) nem osztott újra.

2. megoldás. C++20-ban a legegyszerűbb helyes megoldás a `std::erase_if`, amely az erase–remove mintát csomagolja:

```
#include <cstddef>
#include <cstdio>
#include <vector>
```

```
void write_ints(const int* p, std::size_t n) {
    for (std::size_t i = 0; i < n; ++i) {
        std::printf("%d%c", p[i], i + 1 == n ? '\n' : ' ');
    }
}

void remove_negatives(std::vector<int>& v) {
    std::erase_if(v, [](int x) { return x < 0; });
}

int main() {
    std::vector<int> v{3, -1, 4, -1, 5};
    remove_negatives(v);
    write_ints(v.data(), v.size());
}
```

A `std::erase_if` az `erase`-et használja, amely nem oszt újra, csak a törölt elemektől kezdve invalidál; a `v.data()`-t a törlés után kérjük le, ezért a C API-hívás érvényes. A capacity változatlan marad, a sorrend megmarad. Ellenőrzési feltétel: a függvény után a vektor csak nem negatív elemeket tartalmaz, a `size()` csökkent, a `capacity()` nem nőtt, és a `data()` pointer csak a törlés után kerül felhasználásra. C++20 előtti fallback: `v.erase(std::remove_if(v.begin(), v.end(), [](int x){ return x < 0; }), v.end());`.

3. megoldás. Az üzenet típusa:

```
struct Message {
    std::array<std::uint8_t, 4> magic{0x7f, 'E', 'L', 'F'};
    std::vector<std::uint8_t> payload;
};
```

A fejléc fix méretű, ezért `magic.data()` és `magic.size()` stabil; a payload `data()` pointere viszont minden olyan művelet után újra lekérdezendő, amely újraelosztást okozhat: `push_back`, `emplace_back`, `insert`, `reserve` (ha a capacity-t növeli), `resize` (növeléskor), valamint `shrink_to_fit`. Az `erase` és a `clear` szintén invalidál, de nem oszt újra. Ha a `Message`-t mozgatjuk, a `payload` belső puffere tipikusan átkerül az új objektumba, de a korábbi `payload.data()` pointer a forrás objektum tárolójára mutat, amely moved-from állapotba került; a pointer nem követi a tulajdonost. Ezért a C API-hívást mindig a mozgatás után, az új objektumon kell elvégezni. Ellenőrzési feltétel: a válasz nevezze meg a tulajdonost (a `Message` és a benne lévő `vector`), a borrowed pointert (`data()`), az invalidáló műveletet és a mozgatás utáni új lekérdezést.

28.2 std::string, std::string_view és szövegownership

A szöveg az eddigi tárolóknál megszokott kérdéseket — folytonos tárolás, méret, kapacitás, invalidáció — egy új dimenzióval egészíti ki: a szöveg gyakran nem önálló objektum, hanem egy nagyobb buffer egy szelete, miközben a C API-k többsége nullterminált `const char*`-t vár. A modern C++ ezt a kettősséget két, élesen eltérő szerződésű típusra bontja: a `std::string` tulajdonol és életciklust visel, a `std::string_view` nem tulajdonol, csak hivatkozik egy máshol élő bájttartományra. Az alábbiakban nem tagfüggvény-katalógust adunk, hanem azt a szerződést, amely alapján eldönthető, melyik típus kerüljön egy függvényszignatúrába, egy tagváltozóba vagy egy visszatérési értékbe. A std::array, std::vector és folytonos tárolás (28.1) tárolószerződéseire és

az Owner, view és erőforrásszerződés C++ API-ban (21.5) owner/view felosztására építünk; a kód C++20-at és explicit `-std=c++20` kapcsolót feltételez, GCC vagy Clang fordítóval.

28.2.1 A tulajdonló szöveg: `std::basic_string`

A `std::string` valójában `std::basic_string<char>`; az általános alak

```
template<class CharT,
         class Traits = std::char_traits<CharT>,
         class Allocator = std::allocator<CharT>>
class basic_string;
```

A `Traits` adja a karakterek összehasonlítását, hosszszámítását és keresését; a `std::char_traits<char>` nem locale-érzékeny, egyszerű bájtösszehasonlítást végez. Ez fontos: a `std::string` szemantikája bájtokban értendő, nem „karakterekben” valamely kódolás szerint. A `basic_string` tulajdonolja a puffert, gondoskodik annak felszabadításáról, és gondoskodik arról, hogy az elemek folytonosan helyezkedjenek el.

28.2.1.1 Méret, kapacitás és folytonos hozzáférés

A `basic_string` invariánsa: az elemek a `[data(), data() + size())` félig nyílt tartományban folytonosan helyezkednek el, és a `data() + size()` címen egy `CharT()` érték (azaz `'\0'`) található. C++11 óta a `data()` és a `c_str()` ugyanazt a pointert adja vissza, és mindkettő nullterminált; a korábbi szabványokban ez a `data()`-ra nem volt igaz. A `size()` a tényleges hossz, a `capacity()` a lefoglalt tároló hossza, mindig `size() <= capacity()`. A kapacitásműveletek szabálya szabványverziónként eltér:

- **`reserve(n)`, C++20:** ha `n > capacity()`, a kapacitás legalább `n` lesz; ha `n <= capacity()`, a hívásnak **nincs hatása** — a C++20 `reserve` soha nem csökkenti a kapacitást (P0966). A `reserve(0)` tehát nem zsugorít.
- **`reserve(n)`, C++17 és korábban:** a `capacity()`-nél kisebb `n` nem kötelező zsugorítási kérés volt, amelyet az implementáció figyelmen kívül hagyhatott.
- **`shrink_to_fit()`:** nem kötelező kérés a kapacitás `size()`-ra csökkentésére, minden verzióban.
- **`reserve()` argumentum nélkül:** C++20-ban elavult (deprecated); a jelentése a `shrink_to_fit()` kérés. Helyette a `shrink_to_fit()`-et írjuk.

A `lab-gcc` profilban egy 100 kapacitású, kiürített stringen a `reserve(0)` után a kapacitás `-std=c++17` és `-std=c++20` alatt is 100 maradt, a `shrink_to_fit()` után 15 lett (az SSO-puffer mérete). Ebből is következik, hogy a `capacity()` konkrét értékére építeni hordozhatatlan.

A `c_str()` szerződése kizárólag annyi, hogy a pointer egy nullterminált tömb kezdetére mutat. Ha a string beágyazott `'\0'`-t tartalmaz, a `c_str()` által mutatott tömb a C függvények számára az első nullánál „véget ér”, holott a `size()` nagyobb lehet. Ez nem hiba a `std::string` részéről, hanem a C API korlátja.

28.2.1.2 Small string optimization: implementációfüggő részlet

A gyakorlatban minden elterjedt implementáció small string optimizationt (SSO) alkalmaz: egy kis puffert magában az objektumban tárol, így a rövid stringekhez nem kell heap-allokáció. A küszöb és az objektum mérete azonban implementációfüggő, és nem része a szabványnak. A libstdc++ (GCC) tipikus felépítésében a `std::string` 32 bájtos, és 15 karakter fér el inline; a libc++ (Clang) és a MSVC könyvtára eltérő elrendezést és küszöböt használhat. Ezek az adatok a konkrét implementáció dokumentációjában ellenőrizendők, és nem használhatók hordozható kódban arra, hogy „ez a string biztosan nem allokál". A gyakorlati következmény mégis valós: a rövid stringek másolása és mozgatása olcsóbb lehet, mint a hosszúaké, és a mozgatás SSO esetén *másolást* jelent, ami a mutatókra nézve meglepő lehet.

28.2.2 A nem tulajdonló nézet: `std::string_view`

A `std::string_view` egy nem tulajdonló, olcsón másolható leírás egy létező `CharT`-tartományról. Fogalmilag két adatból áll: egy `const CharT*` pointerből és egy méretből; a pontos elrendezés implementációfüggő, de a triviális másolhatóság és a kis méret gyakorlatilag garantált viselkedés. A `string_view` nem foglal memóriát, nem futtat destruktort, és nem hosszabbítja meg semminek az élettartamát. Ez a szerződés egyben a felelősség: a view addig érvényes, amíg a mögötte álló tartomány él, és amíg azt nem módosítják érvénytelenítő módon.

Két konstrukciós út különbözik:

- `std::string_view sv(ptr, len)` — a pointer és a hossz közvetlenül adott, beágyazott `'\0'` is belekerülhet.
- `std::string_view sv(ptr)` vagy `std::string_view sv = "szöveg"` — a `Traits::length` hosszszámítást használja, ami az első `'\0'`-nál megáll; ez `strlen`-nel ekvivalens költség, azaz $O(n)$.

`std::string`-ből a view implicit konverzióval keletkezik, és a `substr` $O(1)$ művelet, mert csak a pointert és a hosszt állítja be újra.

28.2.2.1 Miért nem garantált a nullterminátor

A `string_view` egyetlen garantált tartománya `[data(), data() + size())`. Arra nincs garancia, hogy `data()[size()]` olvasható, hogy ott `'\0'` áll, vagy hogy a mögötte lévő memória egyáltalán a hívóhoz tartozik. Egy `std::string`-ből készült view esetében a nullterminátor gyakran tényleg ott van, de ez a `string` tulajdonsága, nem a view-é. Egy literálból készült view-nál szintén ott van, egy nagyobb buffer szeleténél viszont a szelet vége után a buffer következő bájtja áll — ami bármi lehet. Ezért `printf("%s", sv.data())` vagy bármely `const char*`-t váró C függvény hívása `sv.data()`-val csak akkor helyes, ha külön bizonyítottuk a nullterminináltságot.

Ugyanez a `string_view`-ra: az alapértelmezett konstrukció `data() == nullptr` és `size() == 0`. A `nullptr` átadása `%s`-nek undefined behavior, tehát a „üres view biztonságos" reflex hibás; a C API határán mindig ellenőrizni kell a pointert és a hosszt, vagy explicit nullterminált `std::string`-et kell készíteni.

28.2.3 Érvénytelenség: mikor lesz a view dangling

A `basic_string` a nem const tagfüggvények túlnyomó részével érvényteleníthet a rá mutató pointereket, referenciákat, iterátorokat és view-kat. A szabvány erre a típusra külön, dokumentált listát ad (a C++20 szövegben a `[string.require]` szakaszban); ennek pontos tartalmát a választott szabványverzióban kell ellenőrizni. A gyakorlati ökölszabály:

Művelet	A `std::string` tartalma	A rá mutató view
`push_back`, `append`, `insert`, `reserve`, nagyobb `resize`	módosul, újraallokálhat	érvénytelen lehet (allokáció esetén biztosan az)
`erase`, `clear`, `shrink_to_fit`, kisebb `resize`	módosul	nem garantáltan érvényes
`operator[]`, `at`, `data`, `begin`/`end` (const)	nem módosul	érvényes marad

Külön kezelendő a temporary élettartama. A `std::string_view v = std::string("a") + "b";` sorban a jobb oldalon egy `std::string` temporary keletkezik, amely a teljes kifejezés végén megszűnik; a `v` már a következő utasításban dangling. Ugyanez a hiba a `return std::string("x").substr(0, 1);` alakban is: a temporary a `return` után nem él. Hasonlóan veszélyes a `std::string` tagot tartalmazó osztály `c_str()`-jéből készített view, ha az osztály később módosítja a stringet. És mivel a mozgatás SSO esetén másolás lehet, a mozgatott stringre korábban készített view-ra sem lehet hordozhatóan támaszkodni.

28.2.4 Kidolgozott példa: kulcs–érték sorok feldolgozása

A példa egy tulajdonos stringből indul, a feldolgozó függvény `string_view`-kat ad vissza, és a hívó felelőssége, hogy az owner túlélje a view-k használatát. A kiírás a C formázó helyett `fwrite`-ot használ, mert a hossz alapján minden bájtot ki kell írni, a beágyazott `'\0'`-t is.

```
// tokenize.cpp - C++20, GCC/Clang, -std=c++20
#include <cstdio>
#include <string>
#include <string_view>
#include <vector>

using namespace std::string_literals;

// Minden bájtot kiír, a beágyazott '\0'-t is; a hosszt a view adja.
void put(std::string_view s) { std::fwrite(s.data(), 1, s.size(), stdout); }

// Előfeltétel: a 'text' által mutatott bájtok a visszaadott view-k
// teljes élettartama alatt élnek és nem módosulnak.
std::vector<std::string_view> split(std::string_view text, char sep) {
    std::vector<std::string_view> out;
    std::size_t begin = 0;
    while (true) {
        std::size_t end = text.find(sep, begin);
        if (end == std::string_view::npos) end = text.size();
        out.push_back(text.substr(begin, end - begin));
```

```
        if (end == text.size()) break;
        begin = end + 1;
    }
    return out;
}

int main() {
    std::string owner = "alpha=1;beta=2;raw=a\0b"s;   // s-literál: a \0 is
     ↪ benne marad
    auto fields = split(owner, ';');   // a view-k az owner pufferére mutatnak
    for (std::string_view f : fields) {
        std::size_t eq = f.find('=');
        if (eq == std::string_view::npos) continue;
        std::string_view key = f.substr(0, eq);
        std::string_view val = f.substr(eq + 1);
        // Hosszal adjuk át a C API-nak: nem feltételezzük a nullterminátort.
        put(key); put(" -> "); put(val); put("\n");
    }
}
```

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 tokenize.cpp -o tokenize
# vagy: clang++ -std=c++20 -Wall -Wextra -Wpedantic -O2 tokenize.cpp -o tokenize
```

Buildjegyzőkönyv (`lab-gcc` profil). A program figyelmeztetés nélkül fordult; a kimenet bájtonként (`od -c`) `alpha -> 1`, `beta -> 2`, majd `raw -> a\0b` — a beágyazott NUL és az utána álló `b` is kiíródik.

Az owner (`owner`) élettartama hosszabb, mint a `fields`-é, és a ciklus alatt nem módosul, ezért a view-k érvényesek. A `printf("%.*s", n, p)` itt **nem** lenne elég: a precízió csak felső korlát, a `%s` konverzió az első `'\0'`-nál akkor is megáll, ha a megadott hossz nagyobb; ráadásul a hossz `int`, így `INT_MAX`-nál hosszabb view nem adható át. A `%.*s` tehát csak akkor helyes, ha a szerződés kimondja, hogy a view nem tartalmaz NUL-t, és a mérete `int`-be fér:

```
// precision_s_nul.cpp - C++20: %.*s a NUL-nál megáll, fwrite nem
#include <cstdio>
#include <string_view>

int main() {
    std::string_view sv("a\0b", 3);
    std::printf("%.*s", static_cast<int>(sv.size()), sv.data());   // csak "a"
    std::fputc('|', stdout);
    std::fwrite(sv.data(), 1, sv.size(), stdout);                  // 61 00 62
}
```

Buildjegyzőkönyv (`lab-gcc` profil). A `precision_s_nul` kimenete bájtonként `61 7c 61 00 62`: a `%.*s` csak az `a`-t írta ki (`61`), a `|` (`7c`) után az `fwrite` a teljes `61 00 62` sorozatot. Ha a ciklus törzsében `owner += "x";` állna, a `fields` elemei érvénytelenné válhatnának, és a további olvasás undefined behavior lenne.

28.2.5 Beágyazott NUL, UTF-8 és a `char8_t` kitekintés

A `std::string` bájtsorozat, ezért beágyazott nullát is képes tárolni: `std::string s("a\0b", 3);` hossza 3. Ugyanez literalból (`std::string s = "a\0b";`) hossza 1, mert a `const char*`

konstruktor `Traits::length`-et használ. A `string_view` ugyanígy viselkedik: a pointer+hossz konstruktor megőrzi a beágyazott nullát, a pointer-only alak nem. A `c_str()`-t és a `data()`-t C API-nak átadva mindig a `size()` a mérvadó, nem a nullterminátor.

UTF-8 esetén a `size()` bájtokban számol, nem karakterekben (code pointokban): egy ékezetes magyar betű két bájt, egy emoji akár négy. A `substr` bájtindexeken dolgozik, ezért egy view szeletelése könnyen egy UTF-8 szekvencia közepére eshet, ami érvénytelen szöveget ad — ezt a szintet a view nem ellenőrzi. A C++20 a `char8_t` és a `std::u8string` bevezetésével külön típust adott az UTF-8 szövegnek; a `std::string` és a `std::u8string` között nincs implicit konverzió, a `u8"..."` literál típusa C++20-ban `const char8_t[]`. Ennek részletei a Szövegkódolás, locale és UTF-8 C-ben (7.5) C-s UTF-8 áttekintésére épülve a későbbi szövegkezelési részben (Könyv, „C++ szöveg, fájlok, idő és könyvtári I/O") kerülnek sorra; itt csak annyi a szerződés, hogy a `std::string` nem garantál érvényes UTF-8-at, és a view sem.

28.2.6 Tipikus hibák és javításuk

- **View visszaadása lokális stringből.** `std::string_view f() { std::string s = "x"; return s; }` — a `s` a visszatéréskor megszűnik, a view dangling. Javítás: adjunk vissza `std::string`-et (értékszemantika, RVO), vagy a view csak paraméterből származzon.
- **View tagváltozóban owner nélkül.** Ha egy osztály `std::string_view` tagot tárol, a konstruktorában kapott temporaryre nem támaszkodhat. Javítás: `std::string` tag, vagy dokumentált, hogy a tag csak egy másik, hosszabb életű ownerre mutathat.
- **Nullterminináltság feltételezése.** `printf("%s", sv.data())` akkor hibás, ha a view nem stringből vagy literálból származik, illetve ha `data() == nullptr`. Javítás: `fwrite`/`std::ostream::write` a hosszal, vagy ideiglenes `std::string` a C API határán; `%.*s` csak NUL-mentes, `int`-be férő view-ra.
- **Beágyazott NUL elvesztése.** `std::string s = "a\0b";` után `s.size() == 1`. Javítás: pointer+hossz konstruktor.
- **SSO-ra és `sizeof`-ra építés.** A „rövid string nem allokál" állítás implementációfüggő; teljesítménykritikus kódban mérni kell, nem feltételezni.
- **Módosítás utáni view-használat.** `auto sv = std::string_view(v); v += "x"; use(sv);` — a növelés újraallokálhat. Javítás: a view-t a módosítás után újra képezni, vagy a módosítás előtt lezárni a view használatát.

28.2.7 Összefoglaló

A `std::string` tulajdonos: folytonos, nullterminált puffert tart, `size()`/`capacity()` szerződéssel, implementációfüggő SSO-val. A `std::string_view` pointer és hossz párja: olcsó, nem tulajdonol, nem garantál nullterminátort, és érvényessége az owner élettartamához és változatlanságához kötött. A döntés szabálya egyszerű: ha a szöveget tárolni, módosítani vagy tulajdonolni kell, `std::string`; ha csak egy hívás idejére olvasni kell egy létező tartományt, `std::string_view`. C API határán a hosszt kell átadni, nem a pointert önmagában. A bájtalapú szemantika miatt a `size()` nem karakterszám, és UTF-8 szeletelésnél a view nem véd meg a szekvencia közepére vágástól.

28.2.8 Feladatok

1. **Magyarázó feladat.** Az alábbi három függvény közül melyik ad érvényes view-t, és milyen feltétellel? Indokolj.

```
std::string_view f1(const std::string& s) { return s; }
std::string_view f2() { std::string s = "x"; return s; }
std::string_view f3(const std::string& s) { return std::string_view(s).substr(0,
 ↪  2); }
```

2. **Kódolási feladat.** Írj két függvényt: `std::string_view trim(std::string_view)` és `std::string trim_owned(std::string_view)`. Mindkettő vágja le az elejéről és végéről a szóközt. Magyarázd el, melyik milyen élettartam-szerződést kíván a hívótól.

3. **Diagnosztikai feladat.** Az alábbi osztály egy konfigurációs értéket tárol, de a konstruktorát temporary stringgel hívják. Találd meg a hibát, javítsd, és add meg az ellenőrzési feltételt.

```
class Label {
public:
    explicit Label(std::string_view s) : s_(s) {}
    std::string_view get() const { return s_; }
private:
    std::string_view s_;
};
// hívás: Label l(std::string("temp"));
```

4. **Diagnosztikai feladat.** Mit ír ki az alábbi program, és miért? Írj olyan változatot, amely a teljes tartalmat kiírja.

```
std::string s("a\0b", 3);
std::printf("[%s]\n", s.c_str());
```

28.2.9 Megoldások

1. megoldás. `f1` érvényes view-t ad, de csak addig, amíg a hívó `s` objektuma él és nem módosul érvénytelenítő módon; a visszatérési érték tehát a hívó felelőssége. `f2` hibás: a lokális `s` a visszatéréskor megszűnik, a view dangling, használata undefined behavior. `f3` érvényes: a `std::string_view(s)` temporary csak egy pointer–hossz pár, amely az `s` pufferére mutat; a temporary megszűnése nem érinti az `s`-et, így a visszaadott view ugyanaddig érvényes, amíg `s` él. Ellenőrzési feltétel: a hívás helyén igazolható, hogy az argumentum élettartama túléli a view használatát; ezt kommentben vagy szerződésben rögzíteni kell.

2. megoldás. A view-változat nem allokál és nem másol, de a visszaadott szelet az eredeti bufferre mutat, ezért a hívónak kell biztosítania, hogy az eredeti szöveg tovább él. A tulajdonos változat új stringet készít, így önállóan érvényes marad, cserébe allokálhat és másol.

```
std::string_view trim(std::string_view s) {
    std::size_t b = 0, e = s.size();
    while (b < e && (s[b] == ' ' || s[b] == '\t' || s[b] == '\n')) ++b;
    while (e > b && (s[e-1] == ' ' || s[e-1] == '\t' || s[e-1] == '\n')) --e;
    return s.substr(b, e - b);   // O(1), az eredeti bufferre mutat
}

std::string trim_owned(std::string_view s) {
    std::string_view t = trim(s);
    return std::string(t);        // másol, önálló tulajdonost ad
}
```

Ellenőrzési feltétel: a `trim` eredményét csak addig szabad használni, amíg az argumentum mögötti buffer él és változatlan; a `trim_owned` eredménye függetlenül érvényes. Mindkét függvény helyes beágyazott `'\0'`-t tartalmazó bemenetre is, mert indexel és nem `strlen`-t használ.

3. megoldás. A hiba a temporary élettartama: a `std::string("temp")` a `Label` konstruktorát kiszolgáló teljes kifejezés végén megszűnik, a `s_` view pedig a megszűnt pufferre mutat; a `get()` hívása dangling referenciát ad. A view nem hosszabbítja meg a temporary élettartamát. Javítás: a tag legyen tulajdonos, és a konstruktor másoljon.

```
class Label {
public:
    explicit Label(std::string s) : s_(std::move(s)) {}
    const std::string& get() const { return s_; }
private:
    std::string s_;
};
```

Alternatív, ha a másolás elkerülése a cél: a konstruktor csak `const std::string&`-et fogadjon el, és a szerződés rögzítse, hogy a hivatkozott stringnek túl kell élnie a `Label`-t — ez azonban törékeny, ezért alapértelmezésben a tulajdonos tag a helyes választás. Ellenőrzési feltétel: a `get()` által visszaadott referenciát használó kód forduljon le és fusson sanitizerrel (pl. AddressSanitizer) anélkül, hogy a `Label` élettartamán kívül hozzáférne; a javított változatban a temporarys hívás is biztonságos.

4. megoldás. A program `[a]`-t ír ki. Az `s` három bájtot tárol (`'a'`, `'\0'`, `'b'`), a `size()` értéke 3, de a `c_str()` által mutatott tömb a C `printf` számára az első nullánál véget ér, mert a `%s` nullterminált stringet vár. Ez nem undefined behavior, hanem a C API és a `std::string` eltérő hosszfogalmának következménye. A teljes tartalom kiírásához hosszt kell átadni:

```
std::printf("[%.*s]\n", static_cast<int>(s.size()), s.data());
// vagy: std::fwrite(s.data(), 1, s.size(), stdout);
```

Ellenőrzési feltétel: a kimenet `[a + NUL + b]` bájtsorozat legyen, azaz a `printf` híváson kívül a hossz 3; a `fwrite`-os változat akkor is helyes, ha a tartalom tetszőleges bájtokat tartalmaz. A `%.*s` és a `%s` közötti különbség tudatosítása a feladat lényege: a `string_view` ugyanezen okból soha nem adható át `%s`-nek hossz nélkül.

28.3 deque, list és forward_list

A folytonos tárolás (std::array, std::vector és folytonos tárolás (28.1)) után könnyű azt a benyomást kelteni, hogy a `std::vector` mindenre jó, és a láncolt tárolók csak tankönyvi maradványok. Ez téves. A standard sequence containerek között nem teljesítményrangsor van, hanem eltérő *szerződések*: más az iterátor-kategória, más az érvényességi (invalidation) szabály, más az elemenkénti tárolási költség, és más a memóriahozzáférés localityje. Ez az alfejezet a `std::deque`, a `std::list` és a `std::forward_list` szerződéseit rögzíti, és megmutatja, hogyan kell ezeket műveleti minta alapján megválasztani. A C-s láncolt listákról szóló rész (Láncolt és intrusive listák (13.2)) az explicit pointer- és ownership-modellt tanította; itt azt nézzük meg, mit garantál ebből a standard könyvtár, és mit nem.

28.3.1 A tárolóválasztás mint szerződés, nem Big-O verseny

Egy sequence container a C++ objektummodellben olyan objektum, amely azonos típusú elemek sorozatát tárolja, és legalább az alapműveleteket (`begin`/`end`, `size`, `insert`, `erase`, `clear`) meghatározott komplexitással és meghatározott érvényességi szabállyal nyújtja. A szerződésnek négy, egymástól független dimenziója van:

- **komplexitás**: műveletenként külön, és ahol a szabvány megengedi, amortizált és worst-case érték külön;
- **iterátor-kategória**: forward, bidirectional vagy random access — ez nem kényelmi kérdés, mert az algoritmusok és a `std::sort` előfeltételei ezen múlnak;
- **érvényesség**: mely művelet után mely iterátor, referencia és pointer marad használható;
- **tárolási elrendezés és per-elem költség**: folytonos-e, mennyi a node-overhead, hány allokáció történik műveletenként.

Egy elemre mutató `T*` vagy `std::list<T>::iterator` nem puszta cím, hanem *időben kiterjesztett hivatkozás*: az értéke addig használható, amíg a szerződés szerint érvényes. A konténer-választás ezért leggyakrabban arról szól, hogy kell-e ilyen hosszú életű hivatkozást tárolnunk egy másik adatszerkezetben (index, cache, eseményregiszter).

A Big-O önmagában félrevezet. A `std::list` középső beszúrása adott iterátorra $O(1)$, de ha az iterátort kereséssel kell megkeresni, a teljes művelet $O(n)$, és közben egy allokáció és két-három cache miss történik. A `std::vector` középső beszúrása $O(n)$ mozgatás, de az elmozdítás `memmove`-szerű, előre jelezhető mintázatú, ami néhány ezer elemig gyorsabb lehet. A döntést tehát a **műveleti minta** hozza: mit végzünk gyakrabban — bejárunk, véletlen hozzáférünk, végpontokon módosítunk, vagy hivatkozásokat őrzünk meg.

28.3.2 A std::deque blokkos tárolója

A `std::deque<T>` double-ended queue: random access sequence container, amelynek elülső és hátsó végén a beszúrás és a törlés amortizáltan konstans, és — ez a lényeg — **nem folytonos**. A tipikus implementáció egy pointerekből álló tömböt (a „map”-et) tart, amely fix méretű blokkokra mutat, a blokkokban pedig folytonosan helyezkednek el az elemek. A blokkméret implementációfüggő; a libstdc++ jellemzően 512 bájtos blokkot használ, illetve `sizeof(T) > 512` esetén egy elemet blokkonként — ez **implementációfüggő** részlet, nem ISO-garancia.

Következmények:

- `operator[]`, `at`, valamint az iterátoron végzett `+=`, `-` $O(1)$ — az iterátor-kategória random access;
- nincs `data()`, mert nincs mögöttes folytonos tömb; a `deque` tartalma nem adható át C API-nak;
- a `deque` nem rendelkezik `reserve`/`capacity`-vel, mert nem egyetlen tömböt növeszt;
- az iterátor több információt hordoz, mint egy pointer (blokk-hivatkozás és blokkon belüli index), ezért jellemzően nagyobb, mint egy `T*` — konkrét méret **implementációfüggő**;
- végponti beszúrásnál nem történik meg a meglévő elemek átmásolása, ezért a `deque` az egyetlen standard sequence container, amelynél a **referenciák és pointerek érvényesek maradnak** végponti beszúráskor.

Ez utóbbi a `deque` valódi indoka. Ha egy hosszú életű hivatkozást akarunk tartani egy olyan sor elemeire, amely mindkét végén nő, a `vector` erre nem alkalmas (reallokáció), a `deque` viszont igen — miközben az **iterátorai** ugyanekkor érvénytelenné válnak. Az iterátor és a referencia érvényessége itt élesen elválik, és ezt a két fogalmat soha nem szabad együtt kezelni.

```cpp
// deque_ref.cpp -- C++20
#include <cassert>
#include <deque>

int main() {
    std::deque<int> d{1, 2, 3};
    int& r = d[1];      // referencia a 2-es elemre
    d.push_front(0);
    d.push_back(4);
    assert(r == 2);     // [ISO] a referencia érvényes marad
    assert(d.size() == 5);
    // Ugyanakkor a push_front/push_back UTÁN minden korábbi deque-iterátor
    // érvénytelen; a korábbi d.begin() nem használható tovább.
}
```

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o deque_ref deque_ref.cpp
```

A `std::deque` természetes felhasználása a munkasor, a csúszóablak és a korlátos puffer, ahol a fogyasztó elöl, a termelő hátul dolgozik, és a már kiosztott elemek címét meg akarjuk őrizni.

28.3.3 std::list és std::forward_list: node-tárolás

A `std::list<T>` kétszeresen láncolt lista, bidirectional iterátorral. A `std::forward_list<T>` egyszeresen láncolt, forward iterátorral, `before_begin()` sentinellel, `insert_after`/`erase_after`/`splice_after` műveletekkel — és **nincs `size()` tagfüggvénye** (a méret lekérése $O(n)$ bejárás). A `std::list::size()` ezzel szemben C++11 óta $O(1)$.

Egy csomópont (node) elvi tartalma x86-64 LP64 alatt:

Típus	Csomópont tartalma	Számított méret	Megjegyzés
`std::list<int>`	`prev` + `next` + `int`	$8+8+4 \to 24$ byte	+ allokátor könyvelése
`std::forward_list<int>`	`next` + `int`	$8+4+4 \to 16$ byte	kisebb overhead
`std::list<std::string>`	`prev` + `next` + `std::string`	$8+8+32 = 48$ byte	a `std::string` mérete implementációfüggő (SSO)

Az itt szereplő számok **illusztratív számítások**, nem mért értékek: a konkrét fordítási profil, az ABI és a használt `std::string` implementáció nélkül a layout nem állítható. A lényeg a nagyságrend: a `std::list<int>` egy elemre nagyjából hatszor annyi processzmemóriát foglal, mint a `std::vector<int>`, és ehhez minden beszúrás egy külön allokációt indít.

A láncolt tárolók tényleges szerződéselőnye nem a komplexitás, hanem az **érvényesség**: beszúrás, törlés és `splice` nem invalidálja a többi elem iterátorát és referenciáját; csak a ténylegesen törölt elemeké válik érvénytelenné. Ez teszi lehetővé, hogy az iterátort hosszú életű hivatkozásként tároljuk például egy `unordered_map` értékében.

A `splice` a láncolt tároló saját művelete: csomópontok átkötése két lista között, elemek másolása és mozgatása nélkül. Egész lista és egyetlen elem átemelése konstans idő, feltéve hogy a két lista `get_allocator()` értéke egyenlő; eltérő allocator mellett ez a túlterhelés nem megengedett. A művelet nem allokál, és az átemelt elemek iterátorai, referenciái érvényesek maradnak — a cél itt az, hogy egy elem „áthelyezése” valóban $O(1)$ legyen, ne pedig egy csomópont-allokációs ciklus.

```cpp
// fl.cpp -- C++20
#include <forward_list>
#include <iostream>
#include <string>

struct Entry { std::string path; bool stale; };

void drop_stale(std::forward_list<Entry>& xs) {
    xs.remove_if([](const Entry& e) { return e.stale; });
}

int main() {
    std::forward_list<Entry> xs{{"/a", false}, {"/b", true}, {"/c", false}};
    drop_stale(xs);
    for (const auto& e : xs) std::cout << e.path << '\n';  // várt: /a, /c
}
```

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o fl fl.cpp
```

A `remove_if` a `forward_list` esetében is $O(n)$ és egyetlen bejárás; a törlés az elől-mutatón keresztül történik, ezért nem kell külön iterátortömb.

28.3.4 Invalidációs táblázat

Az alábbi táblázat az ISO szerződést rögzíti; a `std::vector` referenciasora összehasonlításul szerepel.

Tároló	Művelet	Iterátorok	Referenciák/pointerek	Folytonos
`vector`	`push_back`, reallokációval	mind érvénytelen	mind érvénytelen	igen
`vector`	`push_back`, kapacitáson belül	`end()` érvénytelen	érvényesek	igen
`deque`	`push_front`/`push_back`/`emplace_*` végponton	mind **érvénytelen**	**érvényesek**	nem
`deque`	`insert`/`emplace` középen	mind érvénytelen	mind érvénytelen	nem
`deque`	`erase` elöl (nem az utolsó elem)	csak a töröltekre	csak a töröltekre	nem
`deque`	`erase`/`pop_back` hátul	a töröltre **és a past-the-end-re**	csak a töröltekre	nem
`deque`	`erase` középen	mind (a past-the-end is)	mind	nem
`list`, `forward_list`	`insert`, `emplace`	érvényesek maradnak	érvényesek maradnak	nem
`list`, `forward_list`	`erase`, `pop_*`	csak a töröltekre	csak a töröltekre	nem
`list`	`splice`	érvényesek maradnak	érvényesek maradnak	nem

Két gyakorlati következtetés. Egyrészt a `deque`-nél a végponti beszúrás után a `T*` és a `T&` újra felhasználható, de az `iterator` nem — a kettőt külön kell nyilvántartani. Másrészt a láncolt tárolóknál a stabilitás *csak* a meg nem szüntetett elemekre vonatkozik: ha a tárolót megszüntetjük, minden hivatkozás dangling lesz.

28.3.5 Allokátor, per-node költség és locality

A `std::list` és a `std::forward_list` minden beszúrása allokál (a `splice` kivétel), és minden törlése felszabadít. Ezért ezek a tárolók érzékenyek az allokátor sebességére: konténerenkénti egyedi allokátorral vagy arena/pool allokációval (Arena, pool és region lifetime (13.4)) a node-költség jelentősen csökkenthető. A `deque` ezzel szemben blokkonként allokál, tehát egy blokknyi elemre egy allokáció jut.

A locality szempont ellenkező irányba mutat: a láncolt lista bejárása minden elemnél új csomópontot érint, ami tipikusan cache miss, és a hardver prefetcher nem tud előreolvasni. A `vector` bejárása lineáris, előre jelezhető. A `deque` a kettő között van: blokkon belül folytonos és jól prefetchelhető, blokkváltásnál viszont ugrik.

Ebből következik a gyakorlati döntési minta:

- **sok bejárás, kevés módosítás, kis elemek** → `vector`;
- **mindkét végén módosítás, referencia-stabilitás, C API-hoz nem kell folytonosság** → `deque`;
- **nagy elemek, garantált referencia-stabilitás, `splice`-alapú átrendezés** → `list`;
- **ugyanaz, de elég az előre iterálás és a hely a kritikus** → `forward_list`.

28.3.6 Kidolgozott példa: LRU-sor splice-szal

A klasszikus eset, amikor a `std::list` nem Big-O, hanem érvényességi alapon nyer: LRU (least recently used) sor, amelyhez egy index társul. Az index iterátorokat tárol, és ezek akkor maradnak érvényesek, ha a listából csak `splice`-szal mozgatunk, illetve ha a beszúrás és a törlés a szerződés szerint nem invalidál.

```
// lru_list.cpp -- C++20, Linux, x86-64
#include <cstddef>
#include <iostream>
#include <list>
#include <stdexcept>
#include <string>
#include <unordered_map>

struct Session {
    std::string id;
    int hits = 0;
};

class LruSessions {
public:
    explicit LruSessions(std::size_t capacity) : capacity_(capacity) {
        if (capacity_ == 0)
            throw std::invalid_argument("LruSessions: capacity must be > 0");
    }
```

```
// Az index a saját lista csomópontjaira mutató iterátorokat tárol.
// Tagonkénti másolásnál a másolat indexe az EREDETI listába mutatna,
// ezért a másolás tiltott. A mozgatás biztonságos: a std::list
// mozgatása a csomópontokat viszi át, az iterátorok érvényesek maradnak.
LruSessions(const LruSessions&) = delete;
LruSessions& operator=(const LruSessions&) = delete;
LruSessions(LruSessions&&) noexcept = default;
LruSessions& operator=(LruSessions&&) noexcept = default;

// Erős garancia: ha bármelyik allokáció dob, a lista és az index
// változatlan, és továbbra is ugyanazokat az azonosítókat tartalmazza.
Session& touch(const std::string& id) {
    auto it = index_.find(id);
    if (it == index_.end()) {
        items_.push_front(Session{id, 0});        // 1. dobhat: semmi nem
         ↪ változott
        auto pos = items_.begin();
        try {
            index_.emplace(id, pos);               // 2. dobhat: ...
        } catch (...) {
            items_.pop_front();                    // ...visszagörgetjük az
             ↪ 1. lépést
            throw;
        }
        if (items_.size() > capacity_) {           // 3. kilakoltatás a
         ↪ commit után:
            index_.erase(items_.back().id);        //
             ↪ std::hash<std::string> nem dob,
            items_.pop_back();                     //    erase/pop_back nem
             ↪ allokál
        }
        return *pos;
    }
    // splice: nem allokál, nem mozgat, nem invalidál
    if (it->second != items_.begin())
        items_.splice(items_.begin(), items_, it->second);
    it->second->hits += 1;
    return *it->second;
}

std::size_t size() const { return items_.size(); }

// Teszteléshez: a két tároló ugyanazokat az azonosítókat tartalmazza-e.
bool consistent() const {
    if (items_.size() != index_.size() || items_.size() > capacity_) return
     ↪ false;
    for (auto i = items_.begin(); i != items_.end(); ++i) {
        auto f = index_.find(i->id);
        if (f == index_.end() || f->second != i) return false;
    }
    return true;
```

```
    }

private:
    std::size_t capacity_;
    std::list<Session> items_;   // legutóbb használt elöl
    std::unordered_map<std::string, std::list<Session>::iterator> index_;
};

int main() {
    LruSessions lru{3};
    lru.touch("a");
    lru.touch("b");
    lru.touch("a");   // "a" előre; az indexben tárolt iterátor érvényes marad
    lru.touch("c");
    lru.touch("d");   // "b" esik ki (legrégebben használt)
    std::cout << "size=" << lru.size() << '\n';   // várt: size=3
}
```

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o lru_list lru_list.cpp &&
 ↪  ./lru_list
```

Az `LruSessions` két tagja, a lista és az index, nem független: az index a **saját** lista csomópontjaira mutató iterátorokat tárol. Ezért a tagonkénti (implicit) másolás hibás lenne: a másolat listája új csomópontokat kapna, az indexe viszont az eredeti lista iterátorait. A másolat első `splice`-a vagy kiesése az eredeti lista csomópontjait módosítaná. A típus ezért a másolást explicit tiltja (`= delete`). A mozgatást megtartja, mert a `std::list` mozgatása a csomópontokat adja át, így a tárolt iterátorok érvényesek maradnak, és most már a cél listájába mutatnak. Ha értékszemantikájú másolás kellene, a másoló konstruktornak a lista lemásolása után a másolat saját iterátoraiból kellene újraépítenie az indexet. A nulla kapacitást a konstruktor elutasítja: a referenciát visszaadó `touch` API-val egy „semmit meg nem őrző” cache nem fogalmazható meg értelmesen, mert nem lenne elem, amelyre a visszaadott referencia mutathat.

A `touch` új kulcsnál **tranzakcióként** kezeli a két tárolót. Először a listába szúr (1. lépés, dobhat — ekkor semmi nem változott), majd az indexbe (2. lépés, dobhat — ekkor visszagörgeti az 1. lépést), és csak a sikeres beszúrás után lakoltatja ki a legrégebbi elemet (3. lépés: `index_.erase` a `std::hash<std::string>` miatt nem dob, a `pop_back` nem allokál). Így a kapacitás átmenetileg eggyel túlléphető, cserébe a művelet erős garanciát ad: kivétel után a lista és az index ugyanazokat az azonosítókat tartalmazza, és egyetlen régi elem sem vész el. A korábbi „előbb kilakoltat, aztán beszúr” sorrend kivételnél elveszítette a kilakoltatott elemet, a beszúrás utáni dobás pedig indexeletlen listacsomópontot hagyott.

Buildjegyzőkönyv (`lab-gcc` profil). A program figyelmeztetés nélkül fordult, a kimenete `size=3`. Egy külön tesztprogram (`lru_test.cpp`) a `main` nélküli osztályt használja: a 0 kapacitást elutasítottnak, az 1 és a 3 kapacitást konzisztensnek találta, majd egy lecserélt globális `operator new`-val teli cache-be szúrt új, SSO-n túli kulcsot úgy, hogy rendre minden allokációs pont egyszer dobjon. Mind a 4 injektált kivétel után a `consistent()` igaz volt és a méret 3 maradt (kimenet: `injected=4 consistent=1`). A régi sorrenddel ugyanez a teszt az első injektálásnál `lru.size() == 3` assert-hibával állt le. (A malloc/free-re épülő `operator new` csere miatt a GCC 13 `-Wmismatched-new-delete` hamis pozitív figyelmeztetést ad; a tesztet `-Wno-mismatched-new-delete`-tel fordítottuk.) A javítás előtti, implicit másolást engedő változatban egy másolat `touch` hívása `-D_GLIBCXX_DEBUG` mellett `attempt to compare iterators from different sequences` hibával állt le. A javított változatban egy `LruSessions b = a;` sor `use of deleted function` fordítási hibát ad. A mozgató konstrukció és értékadás után a cél tovább használható; ezt `-D_GLIBCXX_DEBUG` és ASan/UBSan mellett is hibátlanul futtattuk.

Az ownership-térkép: a `Session` objektumok tulajdonosa az `items_` lista; az `index_` nem tulajdonol, csak nem tulajdonló hivatkozást (iterátort) őriz. Az invalidáló művelet, amelytől tartózkodni kell, a `list::erase`/`pop_back` azon elemre, amelynek iterátorát az indexben tároljuk — ezért a sorrend mindig „index törlése, majd node törlése”. Ha valaki itt `std::vector`-ra cserélné a listát, a szó szerinti csere le sem fordulna: a `std::vector`-nak nincs `push_front` tagja (`'class std::vector<...>' has no member named 'push_front'`). A megfelelő művelet az `items_.insert(items_.begin(), ...)`, amely viszont a beszúrási pont utáni összes iterátort és referenciát érvényteleníti (újrafoglalásnál mindet) — vagyis az indexben tárolt összes iterátort, és a `splice`-nak sincs megfelelője. A hiba tehát nem késői dangling olvasás lenne, hanem az, hogy a vektoros változat az index egész szerződését felrúgja.

28.3.7 Tipikus hibák és javításuk

- **„A list gyorsabb, mert O(1) a beszúrás.”** A kereséssel együtt a beszúrás $O(n)$, plusz allokáció és cache miss. Javítás: a műveleti minta rögzítése, majd mérés a saját terhelésen.
- **Iterátor és referencia összekeverése a `deque`-nél.** `push_back` után a `T&` érvényes, az `iterator` nem. Javítás: a kétféle hivatkozást külön típusban és külön névvel tároljuk.
- **`&v[i]` eltárolása, majd `v.push_back`.** Reallokáció esetén dangling pointer. Javítás: index tárolása pointer helyett, vagy `std::deque`.
- **`std::sort(list.begin(), list.end())`.** Nem fordul le: a `std::sort` random access iterátort igényel. Javítás: `list::sort` használata.
- **`forward_list::erase_after(it)` félreértése.** Az `it` az *előző* elemre mutat; az első elemhez `before_begin()` kell.
- **Különböző allocatorú listák között `splice`.** A standard ezt az esetet nem engedi meg. Javítás: azonos allocator-típus és -példány használata.
- **`deque::data()` feltételezése.** Nincs ilyen tag; a `deque` nem folytonos.

28.3.8 Összefoglaló

A `std::deque` blokkos, nem folytonos random access tároló: végponti beszúrásra referenciát stabilan tart, iterátort nem. A `std::list` és `std::forward_list` node-alapú tároló, ahol a beszúrás, törlés és `splice` nem invalidálja a többi elem iterátorát és referenciáját — ez a tényleges szerződéselőny, nem a Big-O. Az árat a csomópontonkénti allokáció, a per-node overhead és a gyenge locality jelenti. A választás ezért mindig a műveleti minta, az iterátor-kategória és a szükséges érvényességi szint együttes vizsgálatából születik; a konténerek közötti döntés részletes ellenőrzését a Tárolóválasztási és invalidációs labor (28.6) laborja gyakoroltatja.

28.3.9 Feladatok

1. feladat (magyarázó). Adott egy `std::deque<int> d` és egy `std::vector<int> v`. Mindkettőre elvégzünk egy `push_back`-et, és korábban eltároltuk az első elemre mutató `int*`-ot. Melyik esetben marad a pointer érvényes, és melyik esetben nem, feltéve hogy a `vector` kapacitása betelt? Fogalmazd meg a szabályt iterátorra és referenciára külön.

2. feladat (kódolási). Egészítsd ki az LRU-példát egy `evict_all()` tagfüggvénnyel, amely kiüríti a tárolót, és egy `contains(id)` függvénnyel. Indokold, milyen sorrendben kell az `index_` és az `items_` tartalmát törölni, és mi történne fordított sorrendben.

3. feladat (diagnosztikai). Egy szolgáltatás eseménynaplója `std::list<Event>`-ben tárolódik; a profil azt mutatja, hogy a processzoridő nagy része a napló bejárásában telik, a beszúrások

száma viszont elhanyagolható. Fogalmazz meg egy mérési tervet, majd egy cserét, és add meg, milyen szerződést kell feladni a cserével.

28.3.10 Megoldások

1. megoldás. A `deque` esetében a `push_back` végponti beszúrás: az ISO szerződés szerint minden iterátor érvénytelenné válik, de a meglévő elemekre mutató referenciák és pointerek érvényesek maradnak, mert a meglévő elemek nem mozdulnak el. A `vector` esetében, ha a `push_back` reallokációt vált ki, mind az iterátorok, mind a referenciák és pointerek érvénytelenné válnak; ha a kapacitás elegendő, csak a `past-the-end` iterátor válik érvénytelenné, a referenciák nem. Az általános szabály tehát: **az iterátor és a referencia érvényessége külön szerződés**, és a `deque` az az eset, ahol a kettő végponti beszúrásnál elválik. Ellenőrzési feltétel: a válasz nevezze meg a tárolót, a műveletet és azt, hogy történt-e újraallokáció.

2. megoldás. Az `evict_all()`-ban először az `index_.clear()`, majd az `items_.clear()` hívandó. Indoklás: az `index_` iterátorokat tárol nem tulajdonló hivatkozásként; ha előbb az `items_`-et ürítenénk, az indexben dangling iterátorok maradnának, és bár a `clear()` azokat már nem dereferálja, a sorrend megfordítása egy későbbi módosításnál (például egy feltételes törlésnél) már valódi hibát okozhat. A tulajdonos mindig előbb szabadítja fel a nem tulajdonló nézeteket. A `contains(id)` egyszerűen `index_.find(id) != index_.end()`, $O(1)$ átlagos esetben. Ellenőrzési feltétel: a kód forduljon `-Wall -Wextra -Wpedantic` mellett figyelmeztetés nélkül, és az `evict_all()` után a `size()` nulla.

3. megoldás. A mérési terv: rögzített fordítási profil és flags mellett (például `g++ -std=c++20 -O2`), azonos bemeneti eseményszámmal mérjük a bejárás idejét, és külön számoljuk az allokációkat egy számláló allokátorral; a mérés nem helyettesíthető becsléssel. A csere: `std::vector<Event>`, ha a napló csak a végére fűz, és nincs szükség referencia-stabilitásra. A feladott szerződés: a beszúrás amortizáltan konstans marad, de reallokációkor minden korábbi iterátor és referencia érvénytelenné válik, és a középső törlés $O(n)$ lesz. Ha a naplóba bárhonnan törölni kell, és a törölt elemekre mutató hivatkozást máshol tárolunk, a `vector` nem elfogadható; ekkor `std::list` marad, de a bejárást kell átalakítani (például gyorsítótárazott összesítéssel), nem a tárolót cserélni. Ellenőrzési feltétel: a döntés nevezze meg a feladott garanciát, és a mérési terv tartalmazzon összehasonlítható alapvonalat.

28.4 map, set és rendezett asszociatív tárolók

A rendezett asszociatív tárolók – a `std::map`, `std::set`, `std::multimap` és `std::multiset` – a modern C++ mindennapi eszközei. DevOps-környezetben tipikusan hosztnév- vagy PID-indexként, konfigurációs kulcstérként, időablakos eseménytárolóként találkozunk velük. A gyakorlati hibák ritkán szintaktikaiak: sokkal gyakrabban a comparator szerződésének megsértése, a lookup API mellékhatása, vagy az iteratorok érvényességi tartományának félreértése áll a háttérben. Ez az alfejezet a szerződéseket és a tipikus döntési helyzeteket tárgyalja, nem a member functionök katalógusát.

28.4.1 A rendezett asszociatív tárolók szerződése

A tárolók a `Compare` template paraméter által meghatározott rendezést használják. A `Compare` egy függvényobjektum-típus, amelynek `comp(a, b)` hívása **strict weak orderingot** (szigorú gyenge rendezést) valósít meg a kulcsokon. Ennek feltételei:

1. *irreflexív*: `comp(a, a)` hamis;
2. *aszimmetrikus*: ha `comp(a, b)`, akkor `!comp(b, a)`;
3. *tranzitív*: ha `comp(a, b)` és `comp(b, c)`, akkor `comp(a, c)`;
4. az *ekvivalencia tranzitív*: ha $a \sim b$ és $b \sim c$, akkor $a \sim c$, ahol

$$a \sim b \iff \neg comp(a,b) \wedge \neg comp(b,a).$$

Az ekvivalencia itt nem azonos a `==` operátorral. Egy case-insensitive comparator szerint `"API"` és `"api"` equivalent, noha nem egyenlőek. A `std::map` és `std::set` egyedi kulcsú: ha egy equivalent kulcs már szerepel, a beszúrás nem történik meg, a meglévő elem marad. A `std::multimap` és `std::multiset` több equivalent kulcsot is tárolhat; ezek egymáshoz viszonyított sorrendje nem része a szerződésnek.

A `std::map` `value_type`-je `std::pair<const Key, T>`, a `std::set`-é `Key`. A kulcs az iterátoron keresztül nem módosítható. A `const_cast` használata a fa invariánsát sérti, és nem definiált viselkedéshez vezet.

28.4.2 Lookup és update: a félreérthetetlen API

A leggyakoribb hiba az `operator[]` használata olvasásra. A `m[k]` a `std::map`-en akkor is beszúr egy `T()` értéket, ha a kulcs nem szerepelt. Ezért const map-en nem is hívható, és a `m[k].empty()` típusú ellenőrzés mellékhatásként létrehozza az elemet. Ha csak olvasni akarunk, a `find`, `at` vagy C++20-ban a `contains` a helyes választás.

Művelet	Hiányzó kulcs esetén	Const map-en	Megjegyzés
`operator[]`	beszúr `T()`-t	nem	implicit beszúrás
`at`	`std::out_of_range`	igen	kivétel
`find`	`end()`	igen	nem dob, nem szúr be
`contains`	`false`	igen	C++20
`try_emplace`	beszúr	nem	csak ha nincs
`insert_or_assign`	beszúr	nem	felülír vagy beszúr

A `lower_bound(k)` az első olyan elemre mutat, amelyre `!comp(elem, k)`, azaz nem kisebb, mint `k`. Az `upper_bound(k)` az első olyan elemre, amelyre `comp(k, elem)`, azaz nagyobb, mint `k`. Az `equal_range(k)` a kettőt adja vissza. Ezekkel a `[lo, hi)` intervallum lekérdezhető; a range query összefüggő intervallumot ad, ha a comparator konzisztens a keresett értékkel.

28.4.3 Heterogén lookup és transparent comparator

A `std::map<std::string, T>` alapértelmezett comparatorja `std::less<std::string>`. Ha `find("api.example.com")`-ot hívunk, a `const char*`-ból `std::string` temporális objektum készülhet, ami allokálhat. A `std::less<>` – a `void` specializáció – **transparent comparator**: rendelkezik `is_transparent` típusdefinícióval, és a tároló `find`, `count`, `lower_bound`, `upper_bound`, `equal_range`, valamint C++20-ban `contains` műveletei heterogén kulcstípussal is hívhatók, ha a comparator össze tudja hasonlítani a kulcstípust a keresett típussal.

C++14 óta a `std::less<>` transparent. A `std::string` és a `std::string_view` összehasonlítható, ezért a `std::map<std::string, T, std::less<>>` és a `find(std::string_view)` allokáció nélkül működik. Ez már C++17-ben is így van: a transzparens asszociatív keresés C++14 óta, a `std::string_view` C++17 óta létezik, ezért a `std::less<>`-es map `find(string_view)`-ja C++17-ben sem épít ideiglenes stringet. A C++17 és a C++20 közötti különbség itt csak a

`contains`: C++17-ben a megfelelője a `m.find(k) != m.end()`. Az `m.find(std::string(k))` másoló fallback csak akkor kell, ha a comparator nem transzparens (például az alapértelmezett `std::less<std::string>`). **Buildjegyzőkönyv** (`lab-gcc` profil, globális `operator new` számlálóval, SSO-n túli kulccsal): `-std=c++17` és `-std=c++20` alatt is `found=1 allocs=0`; a `contains` csak `-std=c++20`-ban érhető el.

28.4.4 Node handle, merge és allocator feltételek

C++17 óta a rendezett tárolók támogatják a **node handle**-t. Az `extract(key)` vagy `extract(iterator)` kivesz egy node-ot, és visszaad egy `node_type` objektumot, amely a node-ot birtokolja. A `node_type` move-only; ha nem kerül beillesztésre, a destruktora felszabadítja a node-ot. A beszúrás tulajdonátadása overloadonként eltér (map/set esetén):

- **`insert(node_type&& nh)`** `insert_return_type`-ot ad vissza (`position`, `inserted`, `node`). Az `nh` minden esetben kiürül. Siker esetén `inserted == true`, a `node` üres; ha a kulcs már szerepelt, `inserted == false`, a `position` a meglévő elemre mutat, és a node a `result.node`-ba kerül — onnan kell továbbvinni, nem az eredeti `nh`-ból.
- **`insert(const_iterator hint, node_type&& nh)`** iterátort ad vissza. Siker esetén az `nh` kiürül; ha a kulcs már szerepelt, az `nh` **változatlan** marad (továbbra is birtokolja a node-ot), a visszaadott iterátor pedig a meglévő elemre mutat.

A node egyik esetben sem vész el; a kérdés csak az, melyik objektum birtokolja.

```cpp
// node_insert_failure.cpp - C++20: sikertelen node-beszúrás overloadonként
#include <cassert>
#include <map>
#include <string>

int main() {
    std::map<int, std::string> src{{1, "uj"}}, dst{{1, "regi"}};

    // 1) insert(node_type&&) -> insert_return_type
    auto nh = src.extract(1);
    auto result = dst.insert(std::move(nh));
    assert(!result.inserted);                       // a kulcs már létezett
    assert(nh.empty());                             // az eredeti handle kiürült
    assert(!result.node.empty());                   // a node a result.node-ban
     ↪  van
    assert(result.position->second == "regi");      // a meglévő elemre mutat
    assert(result.node.mapped() == "uj");           // a rekord nem veszett el

    // 2) insert(hint, node_type&&) -> iterator; sikertelenségnél nh változatlan
    auto nh2 = std::move(result.node);
    auto pos = dst.insert(dst.end(), std::move(nh2));
    assert(!nh2.empty() && nh2.mapped() == "uj"); // a node maradt a handle-ben
    assert(pos->second == "regi");

    // A megtartott node máshová beszúrható:
    std::map<int, std::string> other;
    assert(other.insert(std::move(nh2)).inserted);
    assert(other.at(1) == "uj");
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic -D_GLIBCXX_DEBUG node_insert_failure.cpp` figyelmeztetés nélkül fordult, és a futás egyik `assert`-et sem sértette.

A `merge(source)` átmozgatja a forrás elemeit a célba. Map/set esetén csak azok kerülnek át, amelyek kulcsa a célban még nem szerepel; a többi a forrásban marad. Multimap/multiset esetén minden elem áthelyeződik, mert nincs egyediség. A `merge` előfeltétele, hogy a két tároló allocatora egyenlő legyen (`get_allocator() == source.get_allocator()`); ellenkező esetben a viselkedés nem definiált. A művelet allokáció nélkül mozgatja a node-okat. Az áthelyezett elemekre mutató pointerek és referenciák érvényesek maradnak; az iterátorok is érvényesek, és továbbra is ugyanarra az elemre mutatnak, de a `merge` után a **cél** tároló iterátoraiként viselkednek (C++20 [associative.reqmts]). Ezért csak a cél tároló iterátoraival alkothatnak tartományt vagy hasonlíthatók össze (például `it != target.end()`), a forráséval nem.

Az `extract` és a `merge` élettartam-szabályai egy életcikluson követhetők:

1. **Kivétel előtt** referenciát vagy pointert veszünk egy elemre.
2. **`extract` után** az elem iterátora érvénytelen, a referencia és a pointer viszont *nem* lógó: ugyanarra az elemre mutat, amelyet most a `node_type` birtokol. Amíg a node a handle-ben van, rajtuk keresztül hozzáférni nem definiált viselkedés; a kulcs ilyenkor a `nh.key()`-en át módosítható.
3. **Sikeres `insert` után** a node újra a tárolóé, és a kivétel előtt vett referencia és pointer ismét használható. (A node-birtoklás *alatt* vett referenciák viszont a sikeres beszúráskor érvénytelenné válnak.)

```
// node_lifecycle.cpp - C++20: extract → kulcsmódosítás → insert, és merge
#include <cassert>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> routes{{"api-old", 1}, {"web", 2}};

    // (1) A kivétel ELŐTT vett referencia és pointer.
    int& ref = routes.at("api-old");
    int* ptr = &ref;

    // (2) extract: az iterátor érvénytelen, a ref/ptr NEM - de amíg a node
    //     a node_type-ban van, rajtuk keresztül hozzáférni UB.
    auto nh = routes.extract("api-old");
    nh.key() = "api";                  // a kulcs a handle-ön át módosítható
    // *ptr = 5;                       // HIBA lenne: a node a handle birtokában

    // (3) Sikeres insert: a node visszakerül, a ref/ptr újra használható.
    auto r = routes.insert(std::move(nh));
    assert(r.inserted && r.position->first == "api");
    ref += 10;                         // ugyanarra az elemre hat
    assert(routes.at("api") == 11 && *ptr == 11);

    // (4) merge: az áthelyezett elem korábbi iterátora érvényes marad,
    //     és innentől a CÉL tároló iterátoraként viselkedik.
    std::map<std::string, int> extra{{"db", 3}, {"web", 99}};
    auto it_db = extra.find("db");
```

```
    routes.merge(extra);
    assert(it_db->first == "db" && it_db->second == 3);
    assert(&*it_db == &*routes.find("db")); // a célban ugyanazt az elemet
     ↪ azonosítja
    assert(extra.size() == 1 && extra.begin()->first == "web");  // ütköző kulcs
     ↪ maradt
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic -fsanitize=address,undefined node_lifecycle.cpp` figyelmeztetés nélkül fordult, a futás egyik `assert`-et sem sértette, ASan/UBSan nem jelzett. Megjegyzés a diagnosztikához: a libstdc++ `-D_GLIBCXX_DEBUG` módja a `merge` után is a forráshoz rendeli az áthelyezett elem iterátorát, és az `it_db == routes.find("db")` összehasonlítást `attempt to compare iterators from different sequences` hibával elutasítja, noha a szabvány szerint az iterátor ekkor már a célhoz tartozik. Ezért a példa a címazonosságot (`&*it_db == &*routes.find("db")`) ellenőrzi, amely debug módban is lefut; a libc++ (Clang 22) a közvetlen iterátor-összehasonlítást is elfogadta.

28.4.5 Iteratorstabilitás és komplexitás

Művelet	Iterátorok	Referenciák	Komplexitás
`insert/emplace`	nem invalidál	nem invalidál	$O(\log n)$
`erase`	csak a törölt elemre mutatót	csak a törölt elemre mutatót	$O(\log n)$
`extract`	a kivett elemre mutatót invalidálja	érvényes marad, de a node-birtoklás alatt nem használható; sikeres újrabeszúrás után ismét használható	$O(\log n)$
`merge`	érvényesek; az áthelyezett elemekre mutatók innentől a cél tároló iterátorai	érvényesek	$O(N \log(M+N))$
`find/contains/lower_bound`	nem invalidál	nem invalidál	$O(\log n)$

Jelölés: N a forrás, M a cél kezdeti elemszáma. A `merge` komplexitása $O(N \log(M+N))$, mert a forrás minden elemét a (közben növekvő) célban kell elhelyezni; $N = 1$ mellett is a nagy célban végzett $O(\log M)$ keresés dominál. A többi sor n-je a tároló aktuális mérete. Az `erase` és az `extract` overloadjai eltérnek: az iterátoros változat amortizált $O(1)$, a kulcsos `erase(k)` $O(\log n + c)$, ahol c a törölt elemek száma, a kulcsos `extract(k)` $O(\log n)$. A bejárás $O(n)$, de a node-ok nem folytonosak, ezért a cache-viselkedés gyengébb, mint a `std::vector` esetében. A tárolóválasztásnál tehát a Big-O mellett a localityt, az allokációs mintát és az iteratorstabilitást is mérlegelni kell.

28.4.6 Kidolgozott példa: routing tábla

Az alábbi példa egy hosztnév-alapú routing táblát mutat. A `std::less<>` transparent comparator miatt a `lookup` `std::string_view`-vel allokáció nélkül működik. A `with_prefix` a `lower_bound`-

ot használja: a lexikografikus rendezésben az azonos prefixű kulcsok összefüggő intervallumba esnek.

```
#include <map>
#include <string>
#include <string_view>
#include <vector>
#include <iostream>
#include <stdexcept>
#include <utility>

struct Endpoint {
    std::string host;
    unsigned short port;
};

class RoutingTable {
public:
    void upsert(std::string key, Endpoint ep) {
        table_.insert_or_assign(std::move(key), std::move(ep));
    }

    const Endpoint* lookup(std::string_view key) const noexcept {
        auto it = table_.find(key);
        return it == table_.end() ? nullptr : &it->second;
    }

    const Endpoint& at(std::string_view key) const {
        auto it = table_.find(key);
        if (it == table_.end()) throw std::out_of_range("no route");
        return it->second;
    }

    std::vector<std::string> with_prefix(std::string_view p) const {
        std::vector<std::string> out;
        auto it = table_.lower_bound(p);
        while (it != table_.end() && it->first.starts_with(p)) {
            out.push_back(it->first);
            ++it;
        }
        return out;
    }

private:
    std::map<std::string, Endpoint, std::less<>> table_;
};

int main() {
    RoutingTable rt;
    rt.upsert("api.example.com", {"10.0.0.1", 443});
    rt.upsert("api.internal", {"10.0.0.2", 8443});
    rt.upsert("web.example.com", {"10.0.0.3", 80});
```

```
    if (auto* ep = rt.lookup("api.example.com")) {
        std::cout << ep->host << ':' << ep->port << '\n';
    }
    for (auto& k : rt.with_prefix("api.")) std::cout << k << '\n';
}
```

Buildparancs GCC-vel vagy Clanggel:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 routing.cpp -o routing
# vagy: clang++ -std=c++20 -Wall -Wextra -Wpedantic -O2 routing.cpp -o routing
```

A `starts_with` C++20-as; C++17-es fallback: `it->first.compare(0, p.size(), p) == 0`. Az `insert_or_assign` C++17-es. A példa a megadott forrásokon kívül nem került lefuttatásra; a várt viselkedés a C++20 szerződésen alapul, a tényleges eredményt a tanuló laborja igazolja.

28.4.7 Tipikus hibák és javításuk

- **`<=` comparator.** A `[](int a, int b) { return a <= b; }` megsérti az irreflexivitást: `comp(a,a)` igaz. A `std::set` belső invariánsa sérül, a viselkedés nem definiált. Javítás: `return a < b;`.
- **`operator[]` olvasásra.** A `m[k]` beszúr. Helyette `find`, `at` vagy `contains`.
- **`find` eredményének bool-ként való használata.** Az iterator nem konvertálható `bool`-lá. Helyesen `it != m.end()` vagy `m.contains(k)`.
- **Iterátor invalidáció törlés közben.** C++11 óta `it = m.erase(it)` a helyes; a `m.erase(it++)` régi minta.
- **Kulcs módosítása.** A `const_cast<Key&>(it->first) = ...` a fa invariánsát sérti; nem definiált viselkedés.
- **Dangling comparator.** Ha a comparator egy külső objektumra hivatkozik, annak élettartama legalább a tárolóéval kell egyezzen.

28.4.8 Feladatok

1. **(Könnyű)** Magyarázd el, miért nem megfelelő a `[](const int& a, const int& b) { return a <= b; }` comparator egy `std::set<int>`-hez. Add meg a javítást, és mondd meg, milyen viselkedést okoz a hibás változat.
2. **(Közepes)** Adott egy `std::map<std::string, int>` számláló. Írj függvényt, amely megnöveli a kulcs számlálóját, de csak akkor, ha a kulcs már létezik; új kulcsot ne hozzon létre. Hasonlítsd össze a `operator[]`, az `at` és a `find` megoldásokat.
3. **(Nehéz)** Implementálj egy `TimeWindowIndex` osztályt, amely `std::multimap<std::int64_t, std::string>`-ben tárol eseményeket timestamp szerint. Támogassa a `[from, to)` intervallum-lekérdezést, és egy `merge_from(TimeWindowIndex&)` műveletet node handle-lel. Add meg a buildparancsot és az érvényességi feltételeket.

28.4.9 Megoldások

1. A `<=` sérti az irreflexivitást: `comp(a,a)` igaz lesz. A `std::set` beszúrásai és keresései hibás fába vezetnek; az elemek egy része elveszhet, a `find` hibásan működhet, a viselkedés nem definiált. Javítás: `return a < b;`. Ellenőrzési feltétel: a comparator szigorúan monoton, és `comp(a,a)` hamis.

2. A helyes megoldás egyetlen lookup:

```
void increment_existing(std::map<std::string, int>& m, const std::string& key) {
    auto it = m.find(key);
    if (it != m.end()) ++it->second;
}
```

Az `operator[]` beszúrna egy 0-t, ezért nem jó. Az `at` kivételt dob, ha nincs, ezért try/catch kellene, ami nem hatékony. A `find` nem szúr be, és egy lookup. C++20-ban a `contains` is használható, de az két lookup lenne. Ellenőrzési feltétel: hiányzó kulcsra a map mérete nem változik.

3. Megoldás:

```
// time_window.cpp - C++20, teljes program
#include <map>
#include <string>
#include <vector>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <utility>

class TimeWindowIndex {
public:
    using key_type = std::int64_t;
    using mapped_type = std::string;
    using container = std::multimap<key_type, mapped_type>;

    void add(key_type ts, std::string event) {
        index_.emplace(ts, std::move(event));
    }

    // Félig nyílt [from, to) intervallum. Policy: from >= to esetén üres
    // eredmény; az ellenőrzés az iterátorok képzése előtt történik, mert
    // from > to esetén lo a hi után állna, és a ciklus túlfutna.
    std::vector<std::string> range(key_type from, key_type to) const {
        std::vector<std::string> out;
        if (from >= to) return out;
        auto lo = index_.lower_bound(from);
        auto hi = index_.lower_bound(to);
        for (auto it = lo; it != hi; ++it) out.push_back(it->second);
        return out;
    }

    void merge_from(TimeWindowIndex& other) {
        index_.merge(other.index_);
    }

private:
    container index_;
};
```

```
#define CHECK(condition) do {                                          \
    if (!(condition)) {                                                \
        std::fprintf(stderr, "CHECK failed at line %d\n", __LINE__); \
        std::exit(EXIT_FAILURE);                                       \
    }                                                                  \
} while (0)

int main() {
    TimeWindowIndex empty;
    CHECK(empty.range(0, 100).empty());           // üres index

    TimeWindowIndex w;
    w.add(10, "a"); w.add(20, "b"); w.add(20, "b2"); w.add(30, "c");
    CHECK(w.range(20, 20).empty());               // from == to
    CHECK(w.range(15, 30).size() == 2);           // from < to: b, b2
    CHECK(w.range(10, 31).size() == 4);
    CHECK(w.range(25, 15).empty());               // from > to: üres

    TimeWindowIndex other;
    other.add(20, "x");
    w.merge_from(other);
    CHECK(w.range(20, 21).size() == 3 && other.range(0, 100).empty());
    std::puts("time_window: ok");
}
```

Build: `g++ -std=c++20 -Wall -Wextra -Wpedantic time_window.cpp -o time_window && ./time_window`. A `range` a félig nyílt `[from, to)` intervallumot adja, a `lower_bound(from)` és `lower_bound(to)` közötti iterátorokkal. A dokumentált policy: `from >= to` esetén az eredmény üres, és ezt az ellenőrzést az iterátoros ciklus előtt kell elvégezni; `from > to` esetén ugyanis a `lo` a `hi` után állna, és a ciklus a tároló végén túlra lépne. A `to` felső határ, ezért `lower_bound(to)` a helyes, nem `upper_bound`. A `merge` multimap esetén minden elemet áthelyez. Érvényességi feltétel: a két tároló allocatora egyenlő kell legyen. A `merge` után az áthelyezett elemekre korábban vett iterátorok érvényesek maradnak, de már a cél tároló (`w`) iterátorai: a forrás (`other`) `end()`-jével nem alkothatnak tartományt.

Buildjegyzőkönyv (`lab-gcc` profil). A program figyelmeztetés nélkül fordult, és a négy esetet (üres index, `from == to`, `from < to`, `from > to`), valamint a `merge_from` hatását ellenőrizve `time_window: ok` kimenettel futott, `-D_GLIBCXX_DEBUG` és ASan/UBSan mellett is. A javítás előtti változat `range(25, 15)` hívása `-D_GLIBCXX_DEBUG` mellett `attempt to dereference a past-the-end iterator` hibával állt le.

28.4.10 Összefoglaló

A rendezett asszociatív tárolók akkor használhatók biztonságosan, ha a comparator strict weak ordering, a lookup nem jár mellékhatással, és az iteratorok érvényességi tartományát ismerjük. A `std::less<>` heterogén lookupja allokációt spórol; a node handle és a `merge` allokáció nélküli áthelyezést tesz lehetővé, de allocator-feltétele van. A `std::map` nem mindenre jobb, mint a `std::vector`; a rendezettség és az iteratorstabilitás ára a gyengébb locality.

28.4.11 Források és korlátok

A fejezet a C++20 szabvány asszociatív tároló követelményeire támaszkodik (ISO/IEC 14882:2020, `[associative.reqmts]`). A konkrét implementációk – libstdc++, libc++ – eltérhetnek a node handle és a `merge` részleteiben. A szöveget nem kísérte automatikus forrásellenőrzés; a tanulónak a saját compiler- és library-verzióján kell ellenőriznie a viselkedést. A megadott kiinduló URL-eket a fejezet nem használta fel ellenőrzött bizonyítékként.

28.5 unordered tárolók és container adapterek

A rendezett asszociatív tárolókat (map, set és rendezett asszociatív tárolók) az különböztette meg, hogy a kulcsok között *teljes rendezés* állt rendelkezésre, és a keresés a fa mélysége mentén $\mathcal{O}(\log n)$ lépésben haladt. Az `std::unordered_*` tárolók ezzel szemben nem rendezést, hanem *ekvivalenciát* és *hash-függvényt* kérnek a kulcstól, és cserébe átlagos $\mathcal{O}(1)$ keresést ígérnek. Ez az ígéret azonban feltételes: a szerződés be nem tartása esetén a tároló továbbra is fordít és fut, csak éppen a költséggörbe és a helyesség csendben elromlik. Ez az alfejezet ezt a szerződést bontja ki, majd megmutatja, hogyan öröklődnek a tároló tulajdonságai a `stack`/`queue`/`priority_queue` adapterekbe.

A C-s hash table (Hash table, comparator és ownership (13.3)) után itt már nem a nyílt láncolás vagy a próbasorozat implementációját kell megérteni, hanem azt, hogy a standard library implementációja ezt hogyan rejti el, és mely garantált viselkedésre lehet építeni. Az `unordered_map` és `unordered_set` tipikusan *csomópont-alapú* (node-based): minden elem külön allokált csomópontban él, a bucket-tömb csak mutatókat/láncfejeket tartalmaz. Ebből következik a legfontosabb tervezési tulajdonság: a **referenciák és pointerek az elemekre akkor is érvényesek maradnak, amikor az iterátorok érvénytelenné válnak**.

28.5.1 A hash/equality szerződés

Az `std::unordered_map<Key, T, Hash, KeyEqual>` négy template-paramétere közül az utolsó kettő adja a szerződést. Pontos feltételek:

- `KeyEqual` (*equality predicate*) egy **ekvivalenciareláció** a kulcsokon: reflexív, szimmetrikus, tranzitív. Ez a rendezett tárolók `Compare`-jének *strict weak ordering* követelményével analóg, de gyengébb: itt nem kell sorrend, csak megkülönböztethetőség.
- `Hash` egy **konzisztens** leképezés: ha `eq(a, b)` igaz, akkor `h(a) == h(b)` is igaz kell legyen. A megfordítás **nem** követelmény: két különböző kulcs hash-elhet ugyanarra az értékre (ez az ütközés, collision), és ezt a tároló `KeyEqual`-lel oldja fel.
- Mindkettőnek *ugyanarra a futásra* nézve determinisztikusnak kell lennie; a hash értéke nem változhat meg a kulcs „élete” alatt, különben az elem elvész a saját bucketjéből.

A leggyakoribb hiba pontosan itt keletkezik: a felhasználó a `Hash`-t csak az egyik mezőre definiálja, míg az `operator==` kettőre. Ilyenkor két különböző rekord azonos hash-re kerül — ez önmagában még nem hiba, csak ütközés. Fordított esetben viszont, ha az egyenlő kulcsok hash-e eltér, a tároló *soha nem találja meg* a beszúrt elemet: a `find` a rossz bucketben keres. Ez nem diagnosztizált hiba, csak csendesen hibás program.

28.5.1.1 Transparent lookup

A C++20 heterogén keresést tesz lehetővé az unordered tárolókban is: ha a `Hash`-nek és a `KeyEqual`-nek van `is_transparent` tagtípusa, akkor a `find`, `count`, `contains`, `equal_range` olyan kulcstípusokra is hívható, amelyek nem `key_type`-úak, de a hash és az egyenlőség értelmezi őket. Gyakorlati nyereség: `std::string_view`-vel keresve nem kell ideiglenes `std::string`-et allokálni. A feltétel továbbra is a konzisztencia — a két kulcstípusra adott hash-értéknek egyeznie kell, amikor az egyenlőség igazat ad. Ezért érdemes mindkét overloadot ugyanarra az alacsony szintű hash-re építeni:

```
struct IdHash {
    using is_transparent = void;
    std::size_t operator()(const SessionKey& k) const noexcept {
        return std::hash<std::string_view>{}(std::string_view{k.id});
    }
    std::size_t operator()(std::string_view id) const noexcept {
        return std::hash<std::string_view>{}(id);
    }
};
```

A `std::hash<std::string>` és a `std::hash<std::string_view>` értéke a szabvány szerint **nincs** összehangolva, ezért a fenti példa mindkét ágon ugyanazt a `string_view` hash-t használja.

28.5.2 Buckets, load factor és újraelosztás

A tároló belső állapotát három szám jellemzi: a `size()` (elemek száma), a `bucket_count()` (bucketek száma) és a `max_load_factor()` (alapérték 1.0). Ezekből származik a terheltségi tényező:

$$\text{load_factor} = \frac{\text{size}()}{\text{bucket_count}()}$$

A tároló a beszúrás után garantálja, hogy `load_factor() <= max_load_factor()`; ha ez megsérülne, **rehash** történik: a bucket-tömb újraallokálódik (jellemzően nagyobbra), és minden elem új bucketbe kerül az új B' méret szerint. A két vezérlő művelet előfeltétele és utófeltétele:

- `rehash(n)`: utófeltétel $B' \geq \max(n, \lceil \text{size}()/\text{max_load_factor}() \rceil)$. Komplexitása átlagosan lineáris, legrosszabb esetben négyzetes `size()`-ben.
- `reserve(n)`: $B' \geq \lceil n/\text{max_load_factor}() \rceil$, azaz előre felkészít n elemre. Ezt **a `max_load_factor` beállítása után** kell hívni, mert a `reserve` a pillanatnyi értékkel számol.

A `reserve` a legolcsóbb védelem a rehash-viharok ellen: ha előre tudjuk, hogy n elem lesz, egy allokációval megússzuk azt, amit egyébként $\log n$ körüli újraépítés jelentene — és ami ennél fontosabb, az iterátorok érvényessége kiszámíthatóvá válik.

28.5.2.1 Iterator versus referencia invalidáció

A node-alapú felépítésből adódó invalidációs táblázat (C++20 [unord.req]):

Művelet	Iterátorok	Referenciák/pointerek
`insert`, `emplace`, `operator[]`, ha nem történik rehash	érvényesek	érvényesek
`insert`, `emplace`, `operator[]`, ha rehash történik	**érvénytelenek**	**érvényesek**
`erase(it)`	csak a törölt elemre lesznek érvénytelenek	csak a törölt elemre
`erase(key)`	a törölt elemekre érvénytelenek	a törölt elemekre
`rehash`, `reserve`	**érvénytelenek**	**érvényesek**
`clear()`	érvénytelenek	érvénytelenek
`swap`	érvényesek maradnak, de a másik tároló elemeire mutatnak	ugyanígy

A gyakorlati tanulság: hosszú életű, stabil hivatkozásra (`Session*`, `Value&`) az unordered tároló kiválóan alkalmas, hosszú életű **iterátorra** viszont nem. Aki iterátort cache-el és közben beszúr, az a `find` visszatérési értékét használja olyan művelet után, ami rehash-t okozhatott — ez nem definiált viselkedés.

28.5.3 Multi változatok

Az `unordered_multimap` és `unordered_multiset` ugyanazt a hash/equality szerződést használja, de **ekvivalens kulcsokból többet is tárolhat**. Következmények:

- `insert` mindig sikeres, nem ad `bool`-t a visszatérési értékben.
- `count(k)` az ekvivalens elemek számát adja, `equal_range(k)` az őket tartalmazó fél-nyílt tartományt.
- `find(k)` az ekvivalens csoport *valamelyik* elemét adja vissza — az, hogy melyiket, nem specifikált.
- `operator[]` és `at()` **nincs**, mert nem egyértelmű, melyik elemhez rendelnénk.
- `erase(k)` az összes ekvivalens elemet törli. Egyetlen példány törlésénél előbb ellenőrizni kell, hogy van-e találat: az `erase(equal_range(k).first)` hiányzó kulcsnál `erase(end())` lenne, ami nem definiált viselkedés. A helyes minta:

  ```
  if (auto it = c.find(k); it != c.end()) c.erase(it);   // legfeljebb egy
  ↪  elem
  ```

  Buildjegyzőkönyv (`lab-gcc` profil). Egy `erase_one` sablonba csomagolva, `unordered_multimap<std::s`
  `int>`-en `-D_GLIBCXX_DEBUG` mellett tesztelve: hiányzó kulcsnál nem töröl (méret 3 marad), egy találatnál azt törli, két ekvivalens elemnél pontosan egyet.

28.5.4 Container adapterek

A `stack`, `queue` és `priority_queue` nem tárolók, hanem **adapterek**: egy meglévő Sequence tárolót csomagolnak egy szűkebb felülettel. Nincs `begin()/end()`, nincs iterátor, nincs `size_type`-on kívüli méretkezelés — szándékosan, hogy a használat korlátozása az adatszerkezet invariánsát védje. A mögöttes tároló template-paraméter, és az adapter csak azokat a műveleteket hívja, amelyekre szüksége van:

Adapter	Elvárt alapműveletek	Alapértelmezett konténer	`push`	`top/front`
`stack<T>`	`back`, `push_back`, `pop_back`	`std::deque<T>`	amortizált $\mathcal{O}(1)$	$\mathcal{O}(1)$
`queue<T>`	`front`, `back`, `push_back`, `pop_front`	`std::deque<T>`	amortizált $\mathcal{O}(1)$	$\mathcal{O}(1)$
`priority_queue<T, C, Compare>`	`front`, `push_back`, `pop_back`, random access	`std::vector<T>`	$\mathcal{O}(\log n)$ összehasonlítás	$\mathcal{O}(1)$

A `priority_queue` esetében a `Compare` *strict weak ordering* kell legyen, és a `top()` az az elem, amelyre `comp(top(), x)` egyetlen x-re sem igaz — `std::less<>` mellett tehát ez maximum-kupac. Minimum-kupachoz `std::greater<>` kell.

Az adapterek invalidációs tulajdonságai **az alapkonténerből öröklődnek**. Ez a gyakorlatban a legtöbbet elrontott pont: `std::vector`-on ülő `priority_queue` esetén a `push` `push_back`-et hív, ami újraallokálhat, és ilyenkor a korábban vett `top()` referencia danglinggá válik. `std::deque`-on ülő `stack` esetén a `push_back` nem invalidálja a meglévő elemekre mutató referenciákat, csak az iterátorokat (amikhez amúgy sem férünk hozzá).

28.5.5 Adverzariális ütközés és költségmodellek

A standard garantált komplexitásai az `unordered` tárolókra: `find`, `count`, `==`, `!=` **átlagos esetben** $\mathcal{O}(1)$, **legrosszabb esetben** $\mathcal{O}(\text{size}())$; `insert/erase` átlagosan $\mathcal{O}(1)$, legrosszabb esetben lineáris. A „legrosszabb eset” nem elméleti kuriózum: ha minden kulcs ugyanabba a bucketbe hash-el, a tároló gyakorlatilag láncolt listává degenerálódik, és a beszúrás négyzetes összköltségűvé válik.

A `std::hash` specializációi (például `std::string`-re) **nem** garantáltan ütközésrezisztensek, és a szabvány nem ír elő processzenkénti véletlen seedelést. Ez azt jelenti, hogy támadó által befolyásolt kulcsokkal (HTTP header nevek, query paraméterek, tenant azonosítók) a hash-elés **HashDoS** felület: a támadó előre kiszámolhatja az ütköző kulcsokat. A védelem nem az `unordered_map` lecserélése, hanem a hash-függvény megválasztása: per-process vagy per-tároló véletlen seeddel dolgozó, kriptográfiailag tisztességes keverő (pl. SipHash-szerű konstrukció) vagy legalább a seedet a `Hash` állapotában hordozó funktor.

Fontos különbség a két kategória között: az **átlagos** költség a *hash-eloszlás* függvénye, nem a tároló tulajdonsága. Egy jó hash mellett a 0.75 körüli terheltség mellett a várható lánchossz kicsi, a locality viszont rossz — a csomópontok szétszórva allokálódnak, ezért végigiterálás drágább, mint egy `std::vector` esetén. A választás tehát nem Big-O-verseny: a `map`-et akkor érdemes választani, ha sorrendre, range-lekérdezésre vagy determinisztikus iterációs rendre van szükség; az `unordered_map`-et akkor, ha a kulcs-hőmérséklet magas és a sorrend irreleváns.

28.5.6 Kidolgozott példa

Az alábbi program egy összetett kulcsú munkamenet-tárat mutat be: beállítja a terheltségi tényezőt, előfoglal, majd kényszerített rehash mellett demonstrálja a referencia-stabilitást.

```
// unordered_lab.cpp -- ISO C++20, GCC 13 / Clang 17, Linux x86-64
#include <cstddef>
#include <functional>
#include <iostream>
#include <string>
#include <unordered_map>

struct SessionKey {
    std::string tenant;
    std::string id;

    friend bool operator==(const SessionKey& a, const SessionKey& b) {
        return a.tenant == b.tenant && a.id == b.id;
    }
};

struct SessionKeyHash {
    std::size_t operator()(const SessionKey& k) const noexcept {
        const std::size_t h1 = std::hash<std::string_view>{}(k.tenant);
        const std::size_t h2 = std::hash<std::string_view>{}(k.id);
        return h1 ^ (h2 + 0x9e3779b97f4a7c15ULL + (h1 << 6) + (h1 >> 2));
    }
};

int main() {
    std::unordered_map<SessionKey, int, SessionKeyHash> sessions;
    sessions.max_load_factor(0.75f);   // elobb a terheltseg, aztan a reserve
    sessions.reserve(4096);

    std::cout << "bucket_count=" << sessions.bucket_count()
              << " size=" << sessions.size()
              << " load_factor=" << sessions.load_factor() << '\n';

    sessions[SessionKey{"acme", "s-1"}] = 10;
    sessions.emplace(SessionKey{"acme", "s-2"}, 20);

    auto it = sessions.find(SessionKey{"acme", "s-1"});
    int* stable = &it->second;         // referencia/pointer az elemre

    const auto before = sessions.bucket_count();
    sessions.rehash(before * 2);       // garantalt ujraelosztas

    std::cout << "rehash: " << before << " -> " << sessions.bucket_count() <<
     ↪  '\n';
    // 'it' E R V E N Y T E L E N, 'stable' valtozatlanul ervenyes:
    std::cout << "stable=" << *stable << '\n';
```

```
    std::cout << "bucket(s-1)=" << sessions.bucket(SessionKey{"acme", "s-1"}) <<
     ↪  '\n';
}
```

Fordítás és futtatás a tanuló laborjában:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -Wconversion -O2 \
    -o unordered_lab unordered_lab.cpp && ./unordered_lab
# vagy
clang++ -std=c++20 -Wall -Wextra -Wpedantic -Wconversion -O2 \
    -o unordered_lab unordered_lab.cpp && ./unordered_lab
```

A kimenet konkrét számai implementációfüggők (a bucket-számok prímek vagy kettőhatványok lehetnek), ezért **mért eredményként nem állítom őket**; a program célja a szerződés megfigyelése. Ellenőrzési feltétel: a `load_factor()` értéke mindig ≤ 0.75, és a `stable` pointer a rehash után is a helyes értéket olvassa.

Egy min-kupac adapter:

```
#include <functional>
#include <queue>
#include <vector>

std::priority_queue<int, std::vector<int>, std::greater<int>> min_heap;
min_heap.push(3);
min_heap.push(1);
min_heap.push(2);          // min_heap.top() == 1

const int& top_ref = min_heap.top();
min_heap.push(0);          // a vector push_back-je ujraallokalhat
// top_ref itt mar E RVENYTELEN lehet
```

28.5.7 Tipikus hibák és javításuk

1. **Inkonzisztens hash/equality.** `operator==` két mezőt hasonlít, a hash csak egyet. Tünet: `find` néha nem talál meg egyértelműen jelen lévő elemet. Javítás: a hash minden olyan mezőt keverjen bele, amit az egyenlőség figyelembe vesz.
2. **Iterátor cache-elése beszúrások között.** Javítás: index vagy kulcs szerint keress újra, vagy referenciát/pointert tárolj iterátor helyett.
3. **`reserve` a `max_load_factor` előtt.** Ilyenkor a számítás a régi tényezővel történik, és a tároló a vártnál korábban rehash-el.
4. **`priority_queue` referenciájának megtartása `push` után.** Az alapkonténer `vector`, ami újraallokál. Javítás: a `top()`-ot minden használat előtt hívd, vagy `deque`-et adj meg alapkonténernek.
5. **`std::hash<std::string>`-et támadói bemenetre bízni.** Nem garantáltan ütközésrezisztens, és seedelése nem szabványos.
6. **Rendezett iterációt várni.** Az unordered tároló iterációs sorrendje nem specifikált, és rehash után megváltozik. Ha determinisztikus sorrend kell, a végén rendezni kell, vagy `std::map`-et kell használni.

28.5.8 Feladatok

1. feladat (magyarázó). Egy kollégád ezt írja: „A hash-em jó, mert két különböző kulcsra is ugyanazt adja vissza, így kevesebb bucket kell.” Fogalmazd meg pontosan, miért hibás ez az érvelés, és melyik irányban követeli meg a szabvány a konzisztenciát.

2. feladat (kódolási). Adott $N = 100\,000$ darab, előre ismert számú munkamenet-azonosító. Készíts egy `std::unordered_set<std::string>`-et úgy, hogy a beszúrások során *egyetlen* rehash se történjen, és a `load_factor` soha ne haladja meg a 0.7-et. Számítsd ki a szükséges bucket-szám alsó korlátját, és ellenőrizd `bucket_count()`-tal.

3. feladat (diagnosztikai). Az alábbi kódrészlet egy cache-t tart karban. Mely sorokban van érvénytelenségi hiba, és hogyan javítanád?

```
std::unordered_map<SessionKey, Payload, SessionKeyHash> cache;
cache.reserve(64);
cache[SessionKey{"acme", "s-1"}] = Payload{};
auto it = cache.find(SessionKey{"acme", "s-1"});
Payload* p = &it->second;
for (int i = 0; i < 10; ++i)
    cache.emplace(SessionKey{"acme", "s-" + std::to_string(i + 100)},
 ↪  Payload{});
std::cout << it->second.size << ' ' << p->size << '\n';
```

4. feladat (adapter). Válassz alapkonténert egy `stack`-hez, amelyre egymillió, váltakozó `push`/`pop`/`top` művelet érkezik, és a `top()` referenciáját a hívó rövid ideig megtartja. Indokold a választást locality és invalidáció szempontjából, és hasonlítsd össze a `deque` és a `vector` viselkedését.

28.5.9 Megoldások

1. megoldás. A szabvány csak egy irányt követel: ha `eq(a,b)`, akkor `h(a) == h(b)`. Az ütközés (különböző kulcsokra azonos hash) megengedett, sőt elkerülhetetlen, hiszen a hash értékkészlete véges, a kulcsoké nem. A „kevesebb bucket” érv tehát nem áll: a bucket-számot nem a hash értékkészlete, hanem a `max_load_factor` és a `size()` szabja meg. Ráadásul a sok ütközés pont a legrosszabb esetet hozza: a `find` lineárissá válik. Az ellenkező irány megsértése (egyenlő kulcsokra eltérő hash) viszont tényleges hiba: az elem a beszúráskori bucketjében marad, a keresés viszont a másik bucketben néz.

2. megoldás. A `reserve` utófeltétele $B \geq \lceil n/\text{max_load_factor} \rceil$, tehát $B \geq \lceil 100000/0.7 \rceil = \lceil 142857{,}14\ldots \rceil = 142858$. A helyes sorrend:

```
std::unordered_set<std::string> s;
s.max_load_factor(0.7f);
s.reserve(100000);
// ... 100000 beszúrás ...
assert(s.bucket_count() >= 142858);
assert(s.load_factor() <= 0.7f);
```

Az ellenőrzési feltétel a két `assert`; a tényleges `bucket_count()` ettől nagyobb is lehet (a prím- vagy kettőhatvány-választás miatt), de kisebb nem. A beszúrások alatt a rehash elmarad, mert a terheltség a `reserve` által biztosított tartományban marad.

3. megoldás. A `reserve(64)` után a `max_load_factor` alapértéke 1.0, tehát a tároló kb. 64 elemig nem hash-el újra. A ciklus tíz elemet szúr be, így a ciklus után `size()` kicsi — a rehash **valószínűleg** nem következik be, de erre *nem lehet építeni*: a `reserve` alsó korlátot ad, nem felsőt, és az implementáció a `reserve`-nél nagyobb bucket-számot is választhat. A hiba tehát nem az, hogy biztosan rehash történik, hanem hogy az `it` használata a beszúrások után **nem garantáltan érvényes**. A `p` viszont az, mert node-alapú tárolóban a beszúrás és a rehash nem invalidálja a referenciákat. Javítás: az `it`-et a ciklus után újra kell keresni (`cache.find(...)`), vagy csak `p`-t kell használni. Helyes záró sor:

```
auto it2 = cache.find(SessionKey{"acme", "s-1"});
std::cout << it2->second.size << ' ' << p->size << '\n';
```

Ellenőrzési feltétel: a két érték megegyezik, és az `it` sehol nem szerepel a beszúrások után.

4. megoldás. A `deque` a jobb választás, és ezért is ez a `stack` alapértéke. A `push_back`/`pop_back` amortizáltan $\mathcal{O}(1)$, és — ami a feladat szempontjából döntő — a `deque` végére történő beszúrás **nem invalidálja a meglévő elemekre mutató referenciákat**. A `vector` ezzel szemben a kapacitás túllépésekor újraallokál, és ilyenkor minden korábbi `top()` referencia danglinggá válik. Cserébe a `vector` jobb locality-t ad: az elemek folytonosak, a `top()` és `pop()` egy cache-vonalon marad. Nagy elemszámnál és rövid életű referenciáknál ez a locality előny számíthat, de a feladatban a hívó megtartja a referenciát, ezért a biztonság elsőbbséget élvez. Ellenőrzési feltétel: a `push`/`pop` váltakozó sorozata után a megtartott referencia olvasása nem változik meg, és `-fsanitize=address` alatt sem jelez hibát.

28.5.10 Összefoglaló

Az `unordered` tárolók szerződése egy ekvivalenciareláció (`KeyEqual`) és egy azzal konzisztens hash-függvény (`Hash`); a konzisztencia iránya egyutas: egyenlő kulcsokra egyenlő hash, fordítva nem. A tároló átlagos $\mathcal{O}(1)$ műveleteket ígér, de a legrosszabb eset lineáris, és ez támadó által befolyásolt kulcsoknál elérhető — a `std::hash` nem garantáltan ütközésrezisztens. A `load_factor`, `max_load_factor`, `reserve` és `rehash` együtt adják a teljesítmény- és invalidációs modellt: rehash esetén az iterátorok érvénytelenné válnak, a referenciák és pointerek nem. A `stack`, `queue` és `priority_queue` adapterek nem tárolók, hanem egy alapkonténer szűkített felületei, ezért műveleti költségük és invalidációs viselkedésük az alapkonténeréből öröklődik — ezt a választást a locality, a referencia-stabilitás és a szükséges műveletek együtt határozzák meg, nem pusztán a aszimptotikus bonyolultság.

28.6 Tárolóválasztási és invalidációs labor

A tárolóválasztás a gyakorlatban ritkán dől el pusztán aszimptotikus jelöléssel. Egy connection registry esetében legalább négy kérdésre kell egyszerre válaszolni: mennyi a keresés, a beszúrás és a törlés aránya; mekkora az elem; szükséges-e, hogy egy korábban kiadott hivatkozás érvényes maradjon; és mennyi allokációt engedhetünk meg egy műveletre. Ez az alfejezet egy labor felépítését adja, amelyben ugyanazt a registry-interfészt több háttértárolóval implementáljuk, majd a mérést és az érvényességi szerződést együtt értékeljük. A tárolók egyedi műveleteinek felsorolása nem itt történik: a `unordered` tárolók és adapterek szerződését az unordered tárolók és container adapterek, a szövegtulajdonlás kérdését a std::string, std::string_view és szövegownership tárgyalta, a diagnosztikai eszközöket pedig az ASan, UBSan, MSan, TSan és Valgrind vezette be. Itt ezekre építünk.

28.6.1 Az érvényességi szerződés rögzítése a mérés előtt

A labor tárgya egy *connection registry*: olyan asszociatív adatszerkezet, amely egy stabil azonosítóhoz (`Key`, itt `std::uint64_t`) egy kapcsolat leíró objektumát rendeli, és támogatja a `find`, `insert`, `erase` műveleteket, valamint a bejárást. A mérés szempontjából a lényeg nem az, hogy milyen gyorsan fut egy `for` ciklus, hanem hogy milyen *megfigyelhető szerződést* vállalunk az API-ban.

Pontos definíció: a registry **borrowed pointer szerződése** azt mondja ki, hogy a `find` által visszaadott `Entry*` érvényes marad mindaddig, amíg a hivatkozott bejegyzés létezik, és a hívó nem hajt végre olyan műveletet, amely a tároló dokumentált invalidációs szabályai szerint érvényteleníti. Feltétel: az `Entry` objektumot a tároló birtokolja (nem a hívó), és a hívó a pointert nem tárolja tovább a bejegyzés törlése után. Ha ezt a szerződést vállaljuk, akkor a `std::vector` bizonyos műveletek mellett *nem* alkalmas háttértároló, függetlenül attól, hogy a mérése milyen számot ad.

Ugyanilyen fontos, hogy a laborban három, egymással nem keverhető állítástípus szerepel:

1. **szabványos követelmény** — például hogy a `std::vector` elemei folytonosak, vagy hogy a `std::map` keresése logaritmikus;
2. **implementációs viselkedés** — például a debug iterator mód konkrét makróneve, vagy egy adott libstdc++/libc++ kiadás rehash-politikája;
3. **mért eredmény** — az adott laborprofilban, adott gépen, adott műveleti mix mellett kapott szám.

A három különböző bizonyítékerejű, ezért külön naplózandó. A szöveg írásakor nem állt rendelkezésre forrásszöveg és nem futott mérés; az alábbi táblázat a tananyag által összefoglalt követelménykép, amelyet a labornak a kiválasztott szabvány megfelelő szakaszával ellenőriznie kell, a futási számok pedig kiinduló hipotézisek, nem eredmények.

28.6.2 Összehasonlító szerződéstábla

Tároló	Elemelhelyezkedés	`find`	Középső beszúrás/törlés	Iterátor-invalidáció	Referencia/pointer stabilitás
`vector`	folytonos	$O(n)$ lineáris kereséssel	$O(n)$	újratárolásnál minden; egyébként a beszúrás/törlés helyétől	újratárolásnál minden elmozdul
`deque`	szegmentált, folytonos blokkok	$O(n)$ lineáris kereséssel	$O(n)$	végbeszúrásnál az iterátorok; középen minden	végbeszúrásnál a referenciák stabilak, középső műveletnél nem
`list`	csomópontos, szétszórt	$O(n)$	$O(1)$ adott iterátor mellett	csak a törölt elemre mutatók	csak a törölt elemre mutatók

Tároló	Elemelhelyezkedés	`find`	Középső beszúrás/törlés	Iterátor-invalidáció	Referencia/pointer stabilitás
`map`/`set`	kiegyensúlyozott fa, csomópontos	$O(\log n)$	$O(\log n)$	csak a törölt elemre mutatók	csak a törölt elem
`unordered_map`/`set`	hash-tábla, csomópontos	átlag $O(1)$, legrosszabb $O(n)$	átlag $O(1)$	rehash esetén minden iterátor; törlésnél a törölt elem	rehash nem érvényteleníti a referenciákat

A táblázat olvasata három döntési tengelyt ad. Az első a **locality**: a `vector` az egyetlen, amely bejáráskor egyetlen összefüggő memóriaszakaszon halad, ezért iterációra és a teljes registry periodikus feldolgozására akkor is jobb lehet, ha a `find` lassabb. A második a **stabilitás**: ha az API pointert ad ki, a `vector` és a `deque` középső műveletei kizárják egymást a szerződéssel, a csomópontos tárolók viszont pontosan azt nyújtják, amit a szerződés ígér. A harmadik az **allokáció**: a stabilitási követelmény a gyakorlatban csomópontonkénti allokációt kényszerít ki, ami elemzési költséget és gyengébb localityt hoz — ez implementációs következmény, nem tételes szabály.

Megjegyzendő, hogy a `find` oszlopban a sorozattárolóknál lineáris keresést feltételezünk. Rendezve tartott `vector` esetén a keresés $O(\log n)$ lehetne, de a beszúrás továbbra is $O(n)$ marad, és minden középső beszúrás érvényteleníti a hivatkozásokat — a stabilitási szerződést ez sem teljesíti. A választás tehát nem a keresési komplexitás minimalizálása, hanem a *műveleti mix* és az *érvényességi igény* együttes kielégítése.

28.6.3 Kidolgozott példa: egy interfész, három háttértároló

A labor egyetlen fordítható fájlból áll. A számláló allocator és a determinisztikus álvéletlen sorozat együtt biztosítja, hogy az allokációszám is megfigyelhető legyen, ne csak az idő.

```
// registry_lab.cpp - ISO C++20
#include <algorithm>
#include <chrono>
#include <cstdint>
#include <cstdio>
#include <map>
#include <new>
#include <string>
#include <unordered_map>
#include <utility>
#include <vector>

namespace mem {
inline std::size_t allocs = 0;
inline std::size_t bytes  = 0;
inline void reset() { allocs = 0; bytes = 0; }
}

template <class T>
```

```
struct counting_allocator {
    using value_type = T;
    counting_allocator() = default;
    template <class U> counting_allocator(const counting_allocator<U>&) noexcept
     ↪  {}

    T* allocate(std::size_t n) {
        ++mem::allocs;
        mem::bytes += n * sizeof(T);
        return static_cast<T*>(::operator new(n * sizeof(T)));
    }
    void deallocate(T* p, std::size_t) noexcept { ::operator delete(p); }

    template <class U> bool operator==(const counting_allocator<U>&) const
     ↪  noexcept { return true; }
    template <class U> bool operator!=(const counting_allocator<U>&) const
     ↪  noexcept { return false; }
};

struct Entry {
    std::string   peer;    // "10.0.0.7:5432"
    std::uint16_t weight;
};
using Key = std::uint64_t;
using Pair = std::pair<Key, Entry>;

// Sorozat-alapú háttértároló: lineáris keresés, a végére szúrunk be.
template <class Seq>
class SeqRegistry {
public:
    Entry* find(Key id) {
        auto it = std::find_if(seq_.begin(), seq_.end(),
                               [id](const auto& kv) { return kv.first == id; });
        return it == seq_.end() ? nullptr : &it->second;
    }
    Entry& insert(Key id, Entry e) {
        seq_.emplace_back(id, std::move(e));
        return seq_.back().second;
    }
    std::size_t erase(Key id) {
        return std::erase_if(seq_, [id](const auto& kv) { return kv.first == id;
         ↪  });
    }
    std::size_t size() const { return seq_.size(); }
private:
    Seq seq_;
};

// Asszociatív háttértároló: a find/erase a tároló saját művelete.
template <class Ass>
class AssRegistry {
public:
```

```
    Entry* find(Key id) {
        auto it = map_.find(id);
        return it == map_.end() ? nullptr : &it->second;
    }
    Entry& insert(Key id, Entry e) {
        auto [it, inserted] = map_.emplace(id, std::move(e));
        (void)inserted;
        return it->second;
    }
    std::size_t erase(Key id) { return map_.erase(id); }
    std::size_t size() const { return map_.size(); }
private:
    Ass map_;
};

using VecReg  = SeqRegistry<std::vector<Pair, counting_allocator<Pair>>>;
using MapReg  = AssRegistry<std::map<Key, Entry, std::less<Key>,
                                    counting_allocator<std::pair<const Key,
 ↪  Entry>>>>;
using HashReg = AssRegistry<std::unordered_map<Key, Entry, std::hash<Key>,
 ↪  std::equal_to<Key>,
                                    counting_allocator<std::pair<const Key,
 ↪  Entry>>>>;

struct Lcg {                        // determinisztikus, ismételhető
    std::uint64_t s;
    std::uint64_t next() { s = s * 6364136223846793005ULL +
     ↪  1442695040888963407ULL; return s >> 33; }
};

template <class Reg>
void run(const char* name, std::size_t n_ops, unsigned hit_percent) {
    Reg reg;
    Lcg rng{0xC0FFEEu};
    std::uint64_t next_id = 1;
    std::size_t hits = 0;

    mem::reset();
    auto t0 = std::chrono::steady_clock::now();
    for (std::size_t i = 0; i < n_ops; ++i) {
        std::uint64_t r = rng.next() % 100;
        if (r < hit_percent && reg.size() > 0) {
            if (reg.find(1 + rng.next() % next_id)) ++hits;      // találat vagy
             ↪  tévesztés
        } else if (r < 95) {
            reg.insert(next_id++, Entry{"10.0.0.1:5432", 1});
        } else if (reg.size() > 0) {
            reg.erase(1 + rng.next() % next_id);
        }
    }
    auto t1 = std::chrono::steady_clock::now();
```

```
    double ns = std::chrono::duration<double, std::nano>(t1 - t0).count() /
     ↪  double(n_ops);
    std::printf("%-10s live=%zu  %.1f ns/op  hit=%zu  allocs=%zu  bytes=%zu\n",
                name, reg.size(), ns, hits, mem::allocs, mem::bytes);
}

int main() {
    constexpr std::size_t N = 20'000;   // a lineáris változat miatt kicsi
    run<VecReg>("vector",        N, 70);
    run<MapReg>("map",           N, 70);
    run<HashReg>("unordered",    N, 70);
}
```

```
g++ -std=c++20 -O2 -Wall -Wextra -o registry_lab registry_lab.cpp
# vagy: clang++ -std=c++20 -O2 -Wall -Wextra -o registry_lab registry_lab.cpp
```

A kód felépítése szándékosan az API-invariánst állítja előtérbe: mindhárom `find` `Entry*`-ot ad, tehát mindhárom ugyanazt a szerződést ígéri. A `VecReg` ezt az ígéretet csak addig tudja betartani, amíg nem történik újratárolás vagy középső törlés — a következő szakasz pont ezt a rést mutatja meg. A kimenet alakja: élő elemszám, nanoszekundum/művelet, találatszám, allokációszám és allokált byte. A mérés nem futott le a generáláskor, ezért a konkrét számok helyén a labornak kell kitöltenie a saját profilját; az összehasonlítás csak azonos gépen, azonos fordítóval és azonos `N`-nel értelmes.

28.6.4 Allokációszám és inputeloszlás

A számláló allocator önmagában is tanulságos, de csak akkor, ha ismerjük az eloszlást. Ugyanaz a 70% keresés / 25% beszúrás / 5% törlés mix két különböző terhelést jelent, ha az azonosítók egy szűk, forró halmazból kerülnek ki (sok találat, rövid láncok), és egészen mást, ha egyre növekvő, hideg azonosítótartományból (kevés találat, hosszú láncok a hash-táblában). A `find` arányt ezért nem elég százalékban rögzíteni: a *találati arányt* és az azonosító-eloszlást is naplózni kell, különben a mért ns/op két futás között összemérhetetlen.

Vázlat az allokációszámról. Ha a `vector` kapacitása egy k hosszúságú szakaszon duplázódik, akkor n beszúrás után a növesztések g száma kielégíti a $2^g \geq n$ feltételt, azaz $g \leq \lceil \log_2 n \rceil$. A teljes allokációszám tehát nagyságrendileg $1 + \lceil \log_2 n \rceil$, nem n. Ezzel szemben a referencia-stabil csomópontos tárolók minden beszúrásnál külön csomópontot allokálnak, így az allokációszám lineáris az elemek számában, és a `unordered_map` ehhez még a bucket-tömb újraépítését is hozzáadja rehashkor. Ez a magyarázat *miért* fontos: a `vector` allokációs költsége amortizáltan kicsi, de éppen az újratárolás az, ami a hivatkozásokat érvényteleníti — a két tulajdonság ugyanabból a mechanizmusból ered. Megjegyzendő, hogy a növesztési tényező nem szabályozott, ezért a konkrét lépésszám implementációfüggő, és a labornak rögzítenie kell a használt standard library kiadását.

Az allokációszám mérésekor maga a számláló allocator is torzít: megnöveli az allokációs útvonal költségét, és a `map`/`unordered_map` esetében minden csomópont-allokációt megfigyel. Ez elfogadható, ha a naplóban jelezzük, hogy az időeredmény instrumentált; az allokációszám viszont éppen azért hasznos, mert kevésbé zajos, mint az idő.

28.6.5 A dangling példa külön futtatva

A következő fájl szándékosan hibás tanítási példa, nem része a labor helyes megoldásának.

```cpp
// dangling.cpp - SZÁNDÉKOSAN HIBÁS: nem definiált viselkedés
#include <string>
#include <utility>
#include <vector>

struct Entry { std::string peer; };
std::vector<Entry> g_registry;

Entry* register_peer(std::string peer) {
    g_registry.push_back(Entry{std::move(peer)});
    return &g_registry.back();          // (1) a pointer a vektor egyik elemére
     ↪ mutat
}

int main() {
    Entry* a = register_peer("10.0.0.1:5432");
    Entry* b = register_peer("10.0.0.2:5432");   // (2) újratárolás: az elem
 ↪ elköltözött
    (void)b;
    return a->peer.empty() ? 1 : 0;              // (3) UB: dangling pointer
     ↪ olvasása
}
```

A hiba mechanizmusa pontos: a `push_back` a (2) pontban új puffert allokál, az elemeket átmásolja, a régit felszabadítja, így `a` egy felszabadított tárolóra mutat. Azért kell külön fájlban futtatni, hogy a diagnosztika egyértelmű legyen:

```
g++ -std=c++20 -g -fsanitize=address -o dangling dangling.cpp && ./dangling
```

A laborban elvárt diagnosztika tipikusan heap-use-after-free jelentés; a tényleges üzenetet a felhasználó mérése rögzíti, nem ez a szöveg. A hiba latens voltát az magyarázza, hogy ha a kapacitás elegendő lett volna, nem történik újratárolás, és a program „működni látszik”. Ezért a `reserve` nem javítás: csak elhalasztja a hibát, a szerződést nem teljesíti. Helyes javítások: (a) index vagy stabil azonosító visszaadása pointer helyett, és minden hozzáférésnél újrakeresés; (b) „generációs” handle, amely a törlést is detektálja; (c) olyan háttértároló választása (`deque` végbeszúrásra, `list`, `map`, `unordered_map`), amelynek a hivatkozás-stabilitása illeszkedik az API-hoz. A `find`-nak visszaadott pointer csak akkor maradhat, ha a `erase` és a beszúrás dokumentált hatása összeegyeztethető a szerződéssel.

28.6.6 Debug iterator mód mint implementációs diagnosztika

A libstdc++ `_GLIBCXX_DEBUG` makróval fordult programban a szabványos tárolók iterátorai ellenőrzöttek: a program futás közben jelez, ha érvénytelenített iterátort használunk, vagy ha két különböző tárolóhoz tartozó iterátort hasonlítunk össze. A laborban ez az az eszköz, amely a *szabványos* érvénytelenségi szabályokat tesztelhetővé teszi:

```
g++ -std=c++20 -g -D_GLIBCXX_DEBUG -o registry_dbg registry_lab.cpp
```

Két korlátot rögzíteni kell. Egyrészt a debug mód ABI-t változtat, ezért egy debug- és egy nem debug-fordítású objektum nem linkelhető össze megbízhatóan — a labor ezt külön buildcélként kezelje, ne keverje a release mátrixba. Másrészt a nyers pointereket és a `std::string_view`-kat a debug mód nem követi: a `dangling.cpp` hibája `_GLIBCXX_DEBUG` alatt is rejtve maradhat, mert az nem iterátorhasználat. A libc++ debug eszközeinek pontos makróneve és viselkedése

verziófüggő, ezért a labornak a választott libc++ kiadás dokumentációjában kell ellenőriznie — ez implementációfüggő részlet, nem szabványos követelmény.

28.6.7 Benchmark és API-invariáns együttes értékelése

A döntési eljárás négy lépésből áll, ebben a sorrendben. Először rögzítjük az API-invariánst: ki ad ki pointert, meddig él, mi invalidálja. Másodszor kizárjuk azokat a tárolókat, amelyek a szabványos invalidációs szabályaikkal nem tudják betartani ezt az invariánst — itt a Big-O még nem szerepel. Harmadszor a megmaradt jelölteket a műveleti mix és az inputeloszlás alapján mérjük, és a mérést az allokációszámmal, valamint az instrumentáltság tényével együtt naplózzuk. Negyedszer ellenőrizzük, hogy a mérés nem „javította-e ki” csendben a szerződést: ha a gyorsabb változat csak azért gyors, mert a pointert a hívó tárolja, akkor nem ugyanazt az API-t mértük. Az azonos `N` hiánya is csapda: a `vector` lineáris keresése miatt a három implementációt azonos `N`-en kell összevetni, és külön növelési kísérlettel kell megmutatni, hogyan skálázódik az egyes változat — a nagyobb `N`-en kapott kisebb ns/op önmagában nem összehasonlítható.

28.6.8 Tipikus hibák

A leggyakoribb hiba, hogy a mérés a szerződés rögzítése előtt indul, és a végén a leggyorsabb változatot választjuk, amely viszont dangling pointert ad. A második, hogy valaki a `reserve`-t tekinti a stabilitás bizonyítékának. A harmadik, hogy a `find` lineáris keresését összekeverik a `vector` keresési komplexitásával, és azt állítják, hogy a `vector` mindenre $O(n)$. A negyedik, hogy a debug iterátor módot a teljes release buildre kapcsolják, és ABI-inkompatibilitásból eredő hibát diagnosztizálnak. Az ötödik, hogy a benchmark eredményét forrásként idézik: egy mért szám nem szabványos követelmény, és nem érvényes más gépen, más adateloszlással.

28.6.9 Összefoglaló

A tárolóválasztás három, egymástól függetlennek tűnő, valójában összefüggő döntés: mi a műveleti mix, milyen érvényességi szerződést vállal az API, és mennyi allokációt engedhetünk meg. A `vector` folytonos és olcsón növekszik amortizáltan, de újratároláskor minden hivatkozást érvénytelenít. A `deque` végbeszúrásnál stabil referenciákat ad. A `list`, `map` és `unordered_map` csomópontos, ezért hivatkozás-stabil, cserébe allokációintenzív és localityben gyengébb. A mérés csak a szerződés rögzítése után, az inputeloszlás naplózásával és az instrumentáltság jelölésével értékes.

28.6.10 Feladatok

1. **Magyarázó.** Adott egy registry, amelyben 90% keresés, 8% beszúrás, 2% törlés van, az elemek körülbelül 200 bájtosak, és az API `Entry*`-ot ad ki, amelynek a bejegyzés törléséig érvényesnek kell maradnia. Válasszon tárolót, és indokolja a választást a szerződésből kiindulva, nem Big-O alapján.
2. **Kódolási.** Javítsa a `dangling.cpp` hibáját *handle-alapú* API-vá: a `find` ne pointert, hanem egy `Handle` értéket adjon vissza, amelyből a bejegyzés a registry egy `get(Handle)` műveletével érhető el, és a törölt handle-t a `get` jelezze. Mutassa meg, hogy a javított változat a `vector` megtartásával is betartja a szerződést.
3. **Diagnosztikai.** Egy teszt release módban átmegy, `_GLIBCXX_DEBUG` alatt viszont elhasal „attempt to use invalidated iterator” üzenettel. Fogalmazza meg, milyen hibaosztályra utal ez, és miért *nem* jelentené ezt a hibát, ha a program nyers pointerrel dolgozna.

4. **Mérés-értelmezési.** Két futás: `vector`, `N = 20000`, 40 ns/op; `unordered_map`, `N = 2000000`, 25 ns/op. Milyen további adat hiányzik ahhoz, hogy ez az összehasonlítás érvényes legyen?

28.6.11 Megoldások

1. megoldás. A döntő feltétel a hivatkozás-stabilitás: a `find` által adott pointernek a törlésig érvényesnek kell maradnia. A `vector` ezt nem teljesíti, mert a beszúrás újratároláskor minden hivatkozást érvénytelenít; a `reserve` csak elhalasztja a problémát, nem szerződés. A `deque` végbeszúrásnál stabil referenciákat ad, de a középső törlés/beszúrás érvénytelenít, és a `find` lineáris; a `list` stabil, de keresése lineáris, ami 90% keresés mellett rossz; ezért a `std::unordered_map` a természetes jelölt: átlagos $O(1)$ keresés, hivatkozás-stabil, a rehash csak az iterátorokat érvényteleníti. Ellenőrzési feltétel: a válasznak szerepelnie kell benne, hogy a rehash-re vonatkozó állítás a *referenciákra* és az *iterátorokra* eltérő, és hogy ez a labor profiljában ellenőrizendő. Ha az API rendezett bejárást is megkövetel, a `std::map` a helyes választás, ugyanazzal a stabilitási érvvel.

2. megoldás. A lényeg, hogy a hívó ne tárolhasson nyers pointert, hanem egy (azonosító, generáció) párt, és a `get` a tárban újrakeressen; így a `vector` újratárolása nem jelent szerződésszegést, mert a handle nem mutat memóriára. Vázlat: `struct Handle { Key id; std::uint32_t gen; };`, a registry minden bejegyzésnél generációszámlálót tart, `erase`kor növeli, `get(h)` pedig megkeresi az azonosítót, és ha a tárolt generáció nem egyezik, `nullptr`-t ad. Így a törölt handle `get`-je tévesztést ad, nem dangling olvasást. A `find` helyett a belső keresés maradhat lineáris; a szerződés a pointer helyett a handle érvényességén nyugszik. Ellenőrzési feltétel: a törölt bejegyzés handle-jével hívott `get` `nullptr`, és a `vector` újratárolása után kiadott régi handle továbbra is a helyes bejegyzést találja meg.

3. megoldás. A „use invalidated iterator” üzenet arra utal, hogy a program egy olyan iterátort (vagy abból származtatott pointert) használt, amelyet egy korábbi beszúrás, törlés vagy rehash érvénytelenített. Ez az iterátor-érvénytelenségi hibaosztály. Nyers pointerrel ez a diagnosztika nem jelentkezne, mert a debug mód a szabványos iterátorokat ellenőrzi, a nyers pointereket nem követi; a pointeres változat ugyanazt a hibát hordozhatja, csak láthatatlanul. Ezért a laborban az iterátor-alapú és a pointer-alapú változatot külön kell futtatni, és a pointeres esetre ASan-t kell használni. Ellenőrzési feltétel: a válasz megkülönbözteti a szabványos érvénytelenségi szabályt, az implementáció diagnosztikai képességét és a mérési megfigyelést.

4. megoldás. Hiányzik legalább: (a) azonos `N`-en mért érték, vagy növelési görbe mindkét tárolóra; (b) a műveleti mix és a találati arány; (c) az allokációszám és az instrumentáltság ténye; (d) a fordító, a standard library kiadása, az optimalizálási szint és a hardver; (e) a minta szórása, nem egyetlen futás. Ezek nélkül a két szám két különböző kísérlet, nem összehasonlítás. Ellenőrzési feltétel: a válasz kimondja, hogy a nagyobb `N`-en kapott kisebb ns/op nem bizonyíték a gyorsabb tárolóra.

29 Iteratorok és standard algoritmusok

29.1 Iterator-kategóriák és fél-nyílt tartomány

A korábbi tárolófejezetekben már láttuk, hogy a `std::vector` iterátora és a `std::list` iterátora között lényegi különbség van: az egyik folytonos és konstans időben lépkedhető, a másik csak szekvenciálisan járható be. Az algoritmusok mégis közel azonos módon írhatók meg mindkettőn. Ezt az egységes felületet az *iterator* absztrakció adja, amely a C pointeraritmetikát általánosítja: egy irányított kurzor egy tároló elemére, amellyel léptetni, dereferálni és összehasonlítani lehet. Ebben az alfejezetben rögzítjük az iterator pontos fogalmát, a fél-nyílt tartomány érvényességi szerződését és azokat az iterátorkategóriákat, amelyek meghatározzák, mit lehet egy adott algoritmussal megtenni. A szabályok a Pointeraritmetika, objektumhatár és provenance (6.2) nyelvi alapjaira épülnek, és előkészítik a tárolóválasztást (Tárolóválasztási és invalidációs labor (28.6)) valamint a concepts-alapú megszorításokat (Concepts, requires és constrained overload (27.2)).

29.1.1 Az iterátor mint általánosított pointer

Definíció (iterátor). Egy `It` típus *iterátor* egy `C` tároló felett, ha értékszemantikával rendelkezik, `it++` és `++it` formában léptethető, `*it` dereferálással hozzáférhető, valamint `it1 == it2` és `it1 != it2` összehasonlítható. Egy adott `It` az elemek egy kitüntetett pozícióját reprezentálja, nem az elem értékét.

Az iterátor-alapú algoritmusok egyetlen, általában másfél paramétert kapnak: egy `first` iterátort és egy `last` iterátort. Az algoritmus mindaddig dolgozik, amíg `first != last`. Ebből három alapvető következtetés adódik. Egyrészt `last` *nem* dereferálható: nem egy elemre mutat, hanem a meglátogatandó elemek utáni pozícióra. Másrészt az algoritmus ürességet `first == last` esetben érzékeli, azaz nem kell külön darabszámot átadni. Harmadrészt a távolság kifejezhető `last - first` formában, de csak akkor, ha az `It` típus támogatja a kivonást – ezért a hordozható kód `std::distance(first, last)`-et használ.

Előfeltételek. Az `[first, last)` tartomány akkor *érvényes*, ha `last` elérhető `first`-ből ismételt `++` műveletekkel, a köztes pozíciók mindegyike dereferálható, és a bejárás során nem invalidálódik egyik iterátor sem. Az iterátor nem tárolja, hogy melyik tárolóhoz tartozik (nincs benne ownership), ezért a szerződés megszegése nem feltétlenül okoz azonnali hibát – ez a pointerekre emlékeztető, de szigorúbb élettartam-kérdés.

29.1.2 A fél-nyílt tartomány szerződése

A `[first, last)` fél-nyílt tartomány használatának három indoka van. Egyrészt a pointeraritmetika a C-ben egy-past-the-end pointert is megenged, így a `last` kifejezhető a tömbben `begin + n` alakban anélkül, hogy egy elemre mutatna. Másrészt az üres tartomány természetes reprezentációja `first == last`, nincs szükség külön jelzőre. Harmadrészt a szomszédos tartományok összefűzése triviális: `[a, b) + [b, c) = [a, c)`.

A `last` kezelésének gyakori következménye, hogy *egyetlen* dereferálás történik, ha az algoritmust `first != last` ciklus védi. Kivétel az az eset, amikor az algoritmus előfeltétele nem üres

tartomány – például a maximum keresése. Ilyenkor a hívónak kell biztosítania `first != last`-et, és a szerződést vagy `assert`-tel, vagy `std::optional` visszatérési értékkel kell kifejezni.

29.1.3 Iterator-kategóriák

A kategóriák azt írják le, milyen műveletek érhetők el konstansidőben és milyen *multiplicitással*. A C++20 névterében ezek conceptek, de a mögöttes felbontás a klasszikus C++98-as címkékből származik.

A C++20 két külön kérdést tesz fel egy iterátorról. (1) **Bejárás:** milyen mozgások érhetők el, és bejárható-e többször a tartomány? (2) **Hozzáférés:** olvasható-e (`std::indirectly_readable`), írható-e egy `T` érték rajta keresztül (`std::indirectly_writable<It, T>`)? A kettő független: a bejárási kategória nem mondja meg, hogy az iterátoron át írhatunk-e.

Bejárási kategória	Többször bejárható	`--`	`+ n`	Távolság	Példa
bemeneti (`input_iterator`)	nem	nem	nem	O(n)	`istream_iterator`
előre haladó (`forward_iterator`)	igen	nem	nem	O(n)	`forward_list` iterátora
kétirányú (`bidirectional_iterator`)	igen	igen	nem	O(n)	`list`, `map` iterátora
véletlen elérésű (`random_access_iterator`)	igen	igen	igen	O(1)	`deque` iterátora
folytonos (`contiguous_iterator`)	igen	igen	igen	O(1)	`vector` iterátora, `T*`, `const T*`

Az írhatóság ettől külön, a konkrét iterátortípuson múlik:

Iterátor (`V` = `std::vector<int>`)	Bejárási kategória	Olvasható	`int` írható
`V::iterator`, `int*`	folytonos	igen	igen
`V::const_iterator`, `const int*`	folytonos	igen	**nem**
`std::istream_iterator<int>`	bemeneti	igen	nem
`std::back_insert_iterator<V>`	nincs (csak output)	**nem**	igen

Az `output_iterator` tehát nem bejárási fokozat, hanem az „írható és léptethető” tulajdonság neve; egy `back_insert_iterator` nem olvasható és nem is `input_iterator`.

```
// iter_concepts.cpp - C++20: bejárási kategória és írhatóság külön kérdés
#include <forward_list>
#include <iterator>
#include <list>
#include <sstream>
#include <vector>

using CVecIt = std::vector<int>::const_iterator;
using BackIns = std::back_insert_iterator<std::vector<int>>;

// const int*: a legerősebb kategória, de nem írható
static_assert(std::contiguous_iterator<const int*>);
```

```
static_assert(!std::indirectly_writable<const int*, int>);
// const vector-iterátor: ugyanígy
static_assert(std::contiguous_iterator<CVecIt>);
static_assert(!std::indirectly_writable<CVecIt, int>);
// nem const vector-iterátor: contiguous ÉS írható
static_assert(std::indirectly_writable<std::vector<int>::iterator, int>);
// back_insert_iterator: csak output, nem olvasható, nincs bejárási kategóriája
static_assert(std::output_iterator<BackIns, int>);
static_assert(!std::input_iterator<BackIns> &&
 ↪  !std::indirectly_readable<BackIns>);
// istream_iterator: input, nem forward
static_assert(std::input_iterator<std::istream_iterator<int>>);
static_assert(!std::forward_iterator<std::istream_iterator<int>>);
// forward_list és list
static_assert(std::forward_iterator<std::forward_list<int>::iterator>);
static_assert(!std::bidirectional_iterator<std::forward_list<int>::iterator>);
static_assert(std::bidirectional_iterator<std::list<int>::iterator>);
static_assert(!std::random_access_iterator<std::list<int>::iterator>);

int main() {}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic iter_concepts.cpp` figyelmeztetés nélkül fordul, tehát a fenti `static_assert`-ek — köztük a `std::contiguous_iterator<const int*>` és a `!std::indirectly_writable<const int*, int>` — teljesülnek.

A bejárási kategóriák egymásra épülnek: minden `forward_iterator` egyben `input_iterator`, minden `contiguous_iterator` egyben `random_access_iterator`. A `contiguous_iterator` C++20 újdonság: garantálja, hogy `std::to_address(it + n) == std::to_address(it) + n`, azaz a szomszédos elemek memóriacíme is szomszédos. Ez teszi lehetővé a `std::vector` adatának `T*`-ként való átadását C API-nak, és ezt a garanciát a `std::deque` – bár random access iterátorral rendelkezik – *nem* adja meg.

29.1.4 Single-pass és multipass

A `forward_iterator` és erősebb kategóriák *multipass* tulajdonsággal rendelkeznek: ha `a` és `b` ugyanazon pozíció két másolata, akkor `*a` és `*b` ugyanazt az elemet jelöli, és `++a` után `a != b`, de `b` még mindig az eredeti elemre mutat. Ezt kihasználják azok az algoritmusok, amelyek egy tartományt többször bejárnak (például bináris keresés, rendezés).

Az `input_iterator` és `output_iterator` ezzel szemben *single-pass*: egy `++` után a korábbi másolatok érvénytelenné válnak, mert az olvasás/írás egy megosztott pufferen keresztül történik. Ha egy `input_iterator` tartományon `std::distance`-t hívunk, a bemeneti stream *elfogy*, és a `first` iterátor a továbbiakban nem használható. Ez a klasszikus hiba, amikor egy streamet előbb megszámolunk, majd újra be akarunk járni.

29.1.5 distance és advance kategóriafüggő költsége

Az algoritmusok bonyolultsági elemzése csak akkor pontos, ha az iterátor kategóriáját is megadjuk.

Művelet	Input/Forward	Bidirectional	RandomAccess/Contiguous
`std::distance(first, last)`	O(n), fogyasztja a forrást	O(n)	O(1)
`std::advance(it, n)`	O(n), `n >= 0`	O(n), `n` negatív is lehet	O(1)
`std::next(it, n)`	O(n)	O(n)	O(1)
`std::prev(it, n)`	nem elérhető	O(n)	O(1)

A lineáris esetben a költség `n = distance(first, last)`. Ezért egy `std::list`-en írt lineáris keresés összköltsége `O(n)`, de egy „előre megszámolom a hosszát, aztán lineárisan bejárom” minta `O(2n)` lesz – ugyanaz az aszimptotika, de a konstans kétszeres, és ami nagyobb baj, az `input_iterator` esetén egyenesen hibás. A `std::distance` és `std::advance` implementációja a klasszikus C++-ban overload-feloldással, C++20-ban `if constexpr`-fel vagy tag dispatch-csel választja ki a kategóriát. Ez a Concepts, requires és constrained overload (27.2) mintájának egyik legrégebbi alkalmazása.

29.1.6 iterator_traits és concepts kapcsolata

A `std::iterator_traits<It>` egy fordítási idejű leíró, amely az iterátor tulajdonságait nevesített típusokként teszi elérhetővé: `value_type`, `difference_type`, `reference`, `pointer`, `iterator_category`, valamint C++20-tól `iterator_concept`. A `T*` pointerekre specializáció biztosítja, hogy a nyers pointerek is működjenek. Az algoritmusok ezeket a típusokat használják belső változók deklarálásához és a kategória-alapú döntésekhez.

A C++20 **conceptek** ugyanezt a hierarchiát fejezik ki `std::input_iterator`, `std::forward_iterator` stb. formában. A kapcsolat kettős. Egyrészt a conceptek teljesítéséhez végső soron az `iterator_traits` által kinyert típusoknak kell megfelelniük. Másrészt a `contiguous_iterator` bevezetésével az `iterator_concept` tag vette át a pontos kategória szerepét, mert egy `vector<int>::iterator` `iterator_category`-ja `random_access_iterator_tag`, de `iterator_concept`-ja `contiguous_iterator_tag` – ezt az eltérést a legacy kód miatt nem lehetett az `iterator_category`-be beírni. Új algoritmus írásakor a concepteket használjuk (`template <std::forward_iterator It>`), mert ezek az olvasó számára dokumentálják az előfeltételt, és a hibaüzeneteket is pontosabbá teszik.

29.1.7 Reverse iterator és a base kapcsolata

A `std::reverse_iterator<It>` egy `It` típust csomagol be úgy, hogy a `++` a `--`-t hívja és a dereferálás egy elemmel visszalép. Az alapgondolat az, hogy a fordított iterátor a `base()` által visszaadott pozíció *előtti* elemre mutat. A kulcsazonosság:

$$*\text{rit} \;=\; *(\text{rit.base}() - 1) \;=\; *(\text{std::prev}(\text{rit.base}()))$$

A második alak hordozható azokra a kategóriákra is, ahol a `- 1` nem értelmezett (például `list`).
Bizonyításvázlat. Legyen `it` az `It` egy pozíciója, és tegyük fel, hogy `it != begin()`. A `reverse_iterator` belső állapota megegyezik `it`-vel; a `v.rbegin()` konstruktora `v.end()`-et, a `v.rend()` a `v.begin()`-et kapja. A `std::reverse_iterator::operator*` definíció szerint egy lokális másolatot dekrementál, majd dereferál; ezért a `base()` mindig egy pozícióval *a logikai elem után* áll. Ebből adódik a fenti azonosság. A `rend().base() == begin()` és `rbegin().base() ==`

`end()` azonosságok a konstrukcióból következnek, ezért a fordított tartomány fél-nyílt szerződése ugyanaz marad: `[rbegin(), rend())`.

Ez a plusz egy eltolás a gyakorlatban a leggyakoribb `reverse_iterator` hibaforrás. Ha a `rit` által mutatott elemet akarjuk törölni, nem `rit.base()`-ot kell átadni, hanem `std::prev(rit.base())`-ot (vagy `(rit + 1).base()`-ot). A `rit.base()` a *következő* pozíció, nem az aktuális.

29.1.8 Kidolgozott példa

Az alábbi példa egy `std::forward_iterator` fogadó sablonnal mutatja meg, hogy azonos algoritmus működik `vector`-on, `list`-en és `forward_list`-en, és hogy a `distance` költsége kategóriafüggő. A második rész a `reverse_iterator` és `base()` kapcsolatát ellenőrzi. A fájl önállóan fordítható C++20 alatt GCC 13 vagy Clang 17 mellett; a buildparancs a kód után szerepel.

```
// main.cpp
#include <forward_list>
#include <iostream>
#include <iterator>
#include <list>
#include <vector>

template <std::forward_iterator It>
void report_distance(const char* name, It first, It last) {
    // distance kategóriafüggő: forward esetén O(n), random access esetén O(1)
    std::cout << name << ": distance = " << std::distance(first, last) << '\n';
}

int main() {
    std::vector<int>       v{10, 20, 30, 40, 50};
    std::list<int>         l{10, 20, 30, 40, 50};
    std::forward_list<int> f{10, 20, 30, 40, 50};

    report_distance("vector",       v.begin(), v.end());
    report_distance("list",         l.begin(), l.end());
    report_distance("forward_list", f.begin(), f.end());

    auto rit = v.rbegin();                       // *rit == 50
    std::cout << "*rbegin() = " << *rit << '\n';
    std::cout << "rbegin().base() == end(): "
              << std::boolalpha << (rit.base() == v.end()) << '\n';
    std::cout << "*(rbegin().base()-1) == *rbegin(): "
              << (*(rit.base() - 1) == *rit) << '\n';
    std::cout << "rend().base() == begin(): "
              << (v.rend().base() == v.begin()) << '\n';
}
```

```
g++ -std=c++20 -O2 -Wall -Wextra -pedantic main.cpp -o main
./main
```

A várt (ebben a könyvben nem mért, hanem elvárt) kimenet:

```
vector: distance = 5
```

```
list: distance = 5
forward_list: distance = 5
*rbegin() = 50
rbegin().base() == end(): true
*(rbegin().base()-1) == *rbegin(): true
rend().base() == begin(): true
```

Érdemes megfigyelni, hogy a `report_distance` sablon előfeltétele `std::forward_iterator`, ezért `istream_iterator`-t *nem* fogad el – helyesen, hiszen a `distance` ott elfogyasztaná a bemenetet. A `std::vector<int>::iterator` `contiguous_iterator`, `std::list<int>::iterator` csak `bidirectional_iterator`, `std::forward_list<int>::iterator` pedig `forward_iterator`; a `report_distance` mindegyiket elfogadja, mert a leggyengébb közös kategóriát igényli.

29.1.9 Tipikus hibák és javításuk

Az első és legfontosabb hiba a `last` dereferálása, például `for (auto it = v.begin(); it <= v.end(); ++it)`. A `<=` itt nem csak stíluskérdés: a `v.end()` dereferálása nem definiált viselkedés, és a ciklus egy extra iterációval túllép. A helyes minta `<` vagy `!=` a `last`-tal szemben.

A második hiba az iterátor érvénytelenné válása. A `vector::push_back` és minden újrafoglalást okozó művelet érvényteleníti a korábbi iterátorokat; a `list::erase` csak a törölt elemre mutató iterátort. Ezt a Tárolóválasztási és invalidációs labor (28.6) laborban részletesen tárgyaljuk – itt annyit rögzítünk, hogy az „érvényes `[first, last)`” szerződés a bejárás teljes időtartamára szól.

A harmadik hiba a `distance` és `advance` költségének figyelmen kívül hagyása. Egy `list`-en írt „előre megszámolom, aztán `advance`-dzsel a közepére ugrok” minta `O(n)` helyett véletlenül `O(n)` marad, de a konstans kétszeres; `forward_list`-en pedig a `std::prev` egyáltalán nem fordul le.

A negyedik a `reverse_iterator` `base()` off-by-one hibája, amelyet fent részleteztünk.

29.1.10 Összefoglaló

Az iterátor a C pointer általánosítása: egy kurzor, amely dereferálható, léptethető és összehasonlítható. Az algoritmusok `[first, last)` fél-nyílt tartományon dolgoznak, ahol `last` egy-past-the-end és nem dereferálható. A kategóriák (input, output, forward, bidirectional, random access, contiguous) azt rögzítik, milyen műveletek érhetők el, milyen multiplicitással és milyen költséggel; a `distance` és `advance` lineáris vagy konstans ideje ebből következik. Az `iterator_traits` és a C++20 conceptek ugyanezt a hierarchiát fejezik ki, a `reverse_iterator` pedig a `base()` egy pozícióval eltolt szerződésén keresztül illeszkedik a fél-nyílt modellbe.

29.1.11 Feladatok

1. feladat (magyarázó). Sorolja be kategóriába a következő iterátorokat, és adja meg, hogy `std::distance(first, last)` konstans vagy lineáris-e rajtuk: `std::istream_iterator<int>`, `std::ostream_iterator<int>`, `std::forward_list<int>::iterator`, `std::list<int>::iterator`, `std::deque<int>::iterator`, `std::vector<int>::iterator`, `int*`.

2. feladat (kódolási). Írjon `template <std::forward_iterator It> It find_max(It first, It last)` sablont, amely a legnagyobb elemre mutató iterátort adja vissza, üres tartomány esetén `last`-et. A sablon működjön `vector`, `list` és `forward_list` felett is. Adja meg az előfeltételt, a visszatérési érték érvényességi tartományát és a buildparancsot.

3. feladat (diagnosztikai). Az alábbi kódrészlet nem fordul le, illetve futásidőben hibás:

```
std::list<int> l{1, 2, 3};
auto pit = l.end() - 1;              // (a)

std::forward_list<int> f{1, 2, 3};
int x = *f.end();                    // (b)

std::vector<int> v{1, 2, 3};
auto rit = v.rbegin();
v.erase(rit.base());                 // (c)
```

Magyarázza meg mindhárom hiba okát, és adja meg a javított változatot.

29.1.12 Megoldások

29.1.12.1 1. feladat megoldása

`istream_iterator` és `ostream_iterator`: `input_iterator`, illetve `output_iterator`, single-pass; a `distance` lineáris, és az `istream_iterator` bejárása elfogyasztja a bemenetet. `forward_list::iterator`: `forward_iterator`, `distance` lineáris. `list::iterator`: `bidirectional_iterator`, `distance` lineáris. `deque::iterator`: `random_access_iterator`, `distance` konstans – de nem `contiguous_iterator`, mert a `deque` blokkos tárolású. `vector::iterator` és `int*`: `contiguous_iterator`, `distance` konstans. Az ellenőrzés feltétele: a válasz minden esetben tartalmazza a kategóriát és a `distance`-re vonatkozó állítást; a `deque` és a `vector` megkülönböztetése akkor helyes, ha a `contiguous` hiányát indokolja.

29.1.12.2 2. feladat megoldása

```
#include <iterator>

template <std::forward_iterator It>
It find_max(It first, It last) {
    if (first == last) return last;      // üres tartomány: a last nem
    ↪ dereferálható
    It best = first;
    ++first;                             // az első elemet már megvizsgáltuk
    for (; first != last; ++first) {
        if (*best < *first) best = first;
    }
    return best;
}
```

Az előfeltétel csak `std::forward_iterator`, tehát multipass és `==`/`!=`; `--`-ra és `<`-ra nincs szükség. A visszatérési érték vagy maga a `last` (üres bemenet), vagy egy érvényes elemre mutató iterátor, amely a bejárás végéig érvényes marad, feltéve hogy a tárolót nem módosítjuk. Az `It best = first; ++first;` mintázat elkerüli azt a hibát, hogy az üres tartomány esetén is dereferáljunk. Build: `g++ -std=c++20 -O2 -Wall -Wextra -pedantic -c main.cpp`.

29.1.12.3 3. feladat megoldása

(a) A `std::list` iterátora csak `bidirectional_iterator`, a `-` operátor nem értelmezett rá. A helyes javítás: `auto pit = std::prev(l.end());`, amely `--`-ot használ. Ha a listán többször kell ilyen visszalépés, érdemes lehet random access tárolót választani.

(b) Az `f.end()` egy-past-the-end `forward_list` iterátor; dereferálása nem definiált viselkedés (a `forward_list` esetében még a `--` sem elérhető, hogy visszalépjünk). A helyes minta: a `last`-ot csak összehasonlításra használjuk, az utolsó elemet pedig a `[first, last)` bejárás során jegyezzük meg.

(c) A `rit.base()` nem a `rit` által mutatott elemre mutat, hanem egy pozícióval *utána*. Ezért a `v.erase(rit.base())` nem a 3-at törli (az a `v.rbegin()` elem), hanem a `v.end()`-et mint pozíciót, ami érvénytelen törlés. A helyes változat: `v.erase(std::prev(rit.base()));` vagy `v.erase((rit + 1).base());`. Ellenőrzés: a törlés után `v` tartalma `{1, 2}`, és a művelet csak akkor érvényes, ha `rit != v.rend()` és a törlés után a `rit` érvénytelenné válik – a további használat előtt újra kell szerezni.

29.2 Transform, copy, remove és módosító algoritmusok

A módosító algoritmusok a standard library legfontosabb újrafelhasználható építőelemei: ugyanaz a `std::copy`, `std::transform` vagy `std::remove_if` működik `std::vector`-on, `std::list`-en és nyers tömbön is. Ez a rugalmasság azonban szerződéseken alapul: az algoritmus csak iteratorokat lát, nem ismeri a mögöttes tároló méretét, kapacitását és élettartam-szabályait. A rész az Iterator-kategóriák és fél-nyílt tartomány (29.1) iterator-modelljére és a std::array, std::vector és folytonos tárolás (28.1) tárolószerződéseire épül. A cél, hogy a módosító algoritmusokat ne „tűzze fel” véletlenül, hanem ismerje fel az output kapacitás, az átfedés és a predikátum élettartamának feltételeit.

29.2.1 Az output kapacitás és az output iterator szerződése

A legtöbb módosító algoritmus egy `[first, last)` input tartományt és egy `d_first` **output iteratort** (kimeneti iterator) kap. Az output iterator olyan iterator, amelyen a `*d_first = value` írás és a `++d_first` léptetés elvégezhető; az algoritmus ezen keresztül ír, de a mögöttes tároló kapacitásáról semmit nem tud. Ha az output tartomány kisebb, mint amennyi elemet az algoritmus írni fog, a viselkedés nem definiált (UB). A standard library itt nem dob kivételt és nem ad hibakódot: ez a hívó felelőssége.

A `std::back_inserter` egy `std::back_insert_iterator`-t ad, amely a tároló `push_back` tagfüggvényét hívja, így a méret automatikusan nő. Ez biztonságos ismeretlen méretű output esetén, de allokálhat, és a reallokáció invalidálhatja a tárolóra mutató iterátorokat. A `std::front_inserter` a `push_front`-öt, a `std::inserter` az `insert`-et hívja. A `reserve` előzetes hívása csökkenti a reallokációk számát, de a `back_inserter` ugyanabba a tárolóba írása így is kerülendő, mert a `push_back` a `last` iterátort (a régi `end()`-et) invalidálhatja.

29.2.2 Másolás, mozgatás, transzformálás, kitöltés és generálás

A `std::copy(first, last, d_first)` az `[first, last)` elemeit másolja a `d_first`-től kezdve, és visszatér a `d_first + (last - first)`-fel. A `std::copy_n(first, n, d_first)` az első `n` elemet másolja; a `std::copy_if(first, last, d_first, pred)` csak a predikátum által igaznak ítélt elemeket másolja, és az output pozíció csak ezeknél lép. A `std::copy_backward(first,`

`last, d_last)` hátulról előre másol, és a `d_last`-től visszafelé ír; ez akkor kell, amikor az output a bemenet jobb oldalára esik.

A `std::move(first, last, d_first)` algoritmus ugyanaz, mint a `copy`, de mozgat. Fontos, hogy ez nem azonos a `std::move(x)` kategóriakonverzióval: az előbbi egy algoritmus, amely a forrás elemeket moved-from állapotba hozza, az utóbbi csak egy rvalue referenciát ad. A mozgatott forrás elemek továbbra is léteznek, a destruktoruk lefut, de az értékük meghatározatlan; csak azt szabad rajtuk tenni, amit a típus garantál (például egy `std::string` üres, egy `std::unique_ptr` null lesz). A `std::move_backward` a jobbra tolás mozgatós változata.

A `std::transform(first, last, d_first, op)` az `op`-ot minden elemre alkalmazza, és az eredményt a `d_first`-től írja. A bináris változat `std::transform(first1, last1, first2, d_first, binary_op)` két input tartományból dolgozik; a `first2` tartomány hossza legalább `last1 - first1`, a visszatérési érték pedig `d_first + min(last1 - first1, last2 - first2)`. Az unary változat in-place is használható (`d_first == first`), mert az `op` az aktuális elemet olvassa, és az eredményt ugyanoda írja. A bináris változatnál az in-place csak akkor biztonságos, ha a második input nem fedi át a módosított tartományt; ha átfed, az eredmény a műveleti sorrendtől függ, ezért külön output tárolót érdemes használni.

A `std::fill(first, last, value)` egy értéket ír minden elemre; a `std::fill_n(first, n, value)` az első `n` elemre. A `std::generate(first, last, gen)` a generátort hívja minden elemre; a `std::generate_n(first, n, gen)` az első `n` elemre. A `std::iota(first, last, value)` a `<numeric>` headerben növekvő értékeket ír. A `fill_n` és a `generate_n` `n` paramétere nem lehet negatív; a negatív érték UB, és a kapacitást sem ellenőrzik.

29.2.3 Átfedés és aliasing: az in-place műveletek előfeltételei

A `std::copy` előfeltétele, hogy a `d_first` ne essen a `[first, last)` tartományba. Ha mégis, a `copy` balról jobbra ír, és felülírhatja a még nem olvasott elemeket; ez UB. A `std::copy_backward` előfeltétele tükrös: a `d_last` ne essen a `(first, last]` tartományba (C++20 [alg.copy]). A két végpont tehát fordítva viselkedik, mint a `copy`-nál:

- `d_last == last` **tilos**: ez a `last`-be végződő önmásolás, a forrás és a cél egybeesik;
- `d_last == first` **megengedett**: a cél a forrás előtt, a `first`-nél végződik (megfelelő méretű hellyel előtte);
- `d_last` a `last` után (például `last + 1`) megengedett: ez a jobbra tolás.

A `copy_backward` jobbról balra ír, ezért a jobbra tolásnál biztonságos. A balra tolásnál a `copy` a helyes, mert az output a `first` előtt van. Például a `std::copy(v.begin() + 1, v.end(), v.begin())` balra tol, és biztonságos; a `std::copy_backward(v.begin(), v.end() - 1, v.end())` jobbra tol, és biztonságos. A `std::copy(v.begin(), v.end(), v.begin() + 1)` UB, mert az output a bemenet belsejében van.

```
// copy_backward_bounds.cpp - C++20: a d_last nem lehet a (first, last]
 ↪  tartományban
#include <algorithm>
#include <cassert>
#include <vector>

int main() {
    std::vector<int> v{1, 2, 3, 4, 5, 0};
    // Jobbra tolás eggyel: first=v[0], last=v[5], d_last=v.end() - a v.end()
    // a (first, last] tartományon KÍVÜL van, tehát megengedett.
    std::copy_backward(v.begin(), v.end() - 1, v.end());
```

```
    assert((v == std::vector<int>{1, 1, 2, 3, 4, 5}));

    // d_last == first: a cél a forrás ELŐTT végződik - megengedett.
    std::vector<int> w{0, 0, 0, 7, 8, 9};
    std::copy_backward(w.begin() + 3, w.end(), w.begin() + 3);
    assert((w == std::vector<int>{7, 8, 9, 7, 8, 9}));

    // TILOS: d_last == last (önmásolás a last-be végződve) -
     ↪  előfeltétel-sértés:
    // std::copy_backward(v.begin(), v.end(), v.end());
}
```

Buildjegyzőkönyv (lab-gcc profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic -D_GLIBCXX_DEBUG copy_backward_bounds.cpp` figyelmeztetés nélkül fordul, és mindkét `assert` teljesül: a `d_last = last + 1` jobbra tolás és a `d_last == first` eset is a várt tartalmat adja.

A `std::transform` unary in-place megengedett. A `std::copy_if` viszont **nem** használható önmagára: az előfeltétele, hogy a bemeneti és a kimeneti tartomány ne fedje át egymást (C++20 [alg.copy]). Átfedő hívás akkor is előfeltétel-sértés, ha egy adott implementáción véletlenül a várt eredményt adja. (Egy ilyen tömörítő kiírásnál a kimeneti pozíció a feldolgozott pozícióhoz képest lemarad, nem előreszalad; ez magyarázza, miért „működhet”, de nem teszi szabályossá.) In-place szűrésre a `std::remove_if` + `erase`, illetve C++20-ban a `std::erase_if` való; a `copy_if` külön kimeneti tárolóba ír.

A `std::back_inserter` ugyanabba a tárolóba írása szintén hibás, mert a `push_back` reallokálhat, és az input iterátorok invalidálódnak; a `reserve` ezen nem segít. Az önbeszúrás sem megoldás: a `v.insert(v.end(), v.begin(), v.end())` hívás sérti a sorozattárolók `insert(p, i, j)` előfeltételét, amely szerint az `i` és `j` nem mutathat magába a tárolóba ([sequence.reqmts]); ezt a `reserve` sem írja felül. A helyes út külön forrásmásolat:

```
// inplace_filter.cpp - C++20: szűrés helyben és duplázás biztonságosan
#include <algorithm>
#include <cstdio>
#include <iterator>
#include <vector>

int main() {
    std::vector<int> v{1, 2, 3, 4, 5, 6};

    // In-place szűrés: erase_if (C++20), nem önmagára másoló copy_if.
    std::erase_if(v, [](int x) { return x % 2 == 0; });       // 1 3 5

    // Szűrt másolat: copy_if külön kimenetbe.
    std::vector<int> big;
    std::copy_if(v.begin(), v.end(), std::back_inserter(big),
                 [](int x) { return x > 1; });                 // 3 5

    // Önhozzáfűzés: előbb külön forrásmásolat, aztán insert.
    const std::vector<int> copy = v;
    v.insert(v.end(), copy.begin(), copy.end());               // 1 3 5 1 3 5

    for (int x : v) std::printf("%d ", x);
```

```
    std::printf("| ");
    for (int x : big) std::printf("%d ", x);
    std::printf("\n");
}
```

A negatív próba a libstdc++ debug módjával mutatja meg az önbeszúrás hibáját:

```
// vector_self_insert_debug.cpp - NEGATÍV próba: önbeszúrás saját iterátorokból
#include <vector>

int main() {
    std::vector<int> v{1, 2, 3};
    v.reserve(16);                          // a reserve nem írja felül
    v.insert(v.end(), v.begin(), v.end());  // előfeltétel-sértés: i, j a v-be
 ↪  mutat
}
```

Buildjegyzőkönyv (`lab-gcc` profil). Az `inplace_filter.cpp -D_GLIBCXX_DEBUG` és ASan/UBSan mellett hibátlanul fut, kimenete `1 3 5 1 3 5 | 3 5`. A `vector_self_insert_debug.cpp -D_GLIBCXX_DEBUG` mellett `attempt to insert with an iterator range [first, last) from this container` hibával leáll.

29.2.4 Logikai eltávolítás: remove, remove_if és unique

A `std::remove(first, last, value)` és a `std::remove_if(first, last, pred)` nem töröl a tárolóból. Az el nem távolított elemeket előremozgatja, és visszaadja az új logikai end-et (`new_end`). A `[new_end, last)` elemek moved-from állapotban vannak: az objektumok léteznek, a destruktoruk később lefut, de az értékük meghatározatlan, ezért nem szabad rájuk támaszkodni. A `size()` nem változik, és a `std::remove` nem allokál; a `vector` iterátorai érvényesek maradnak, de a `[new_end, last)` elemek értéke megváltozik. A `remove_if` megőrzi a megmaradó elemek relatív sorrendjét. A művelet költsége `O(n)` összehasonlítás és legfeljebb `O(n)` mozgatás.

A `std::unique(first, last)` csak a szomszédos duplikátumokat távolítja el; a nem szomszédos duplikátumok megmaradnak. Ezért a teljes deduplikációhoz előbb rendezni kell: `std::sort` + `std::unique` + `erase`. A `unique` is logikai eltávolítás, és visszaadja az új logikai end-et. A `std::list` és a `std::forward_list` esetében van `remove`, `remove_if` és `unique` tagfüggvény, amely ténylegesen töröl, és csak a törölt elemekre mutató iterátorokat invalidálja; a `std::remove` a listán is csak logikai eltávolítás, ezért ott a tagfüggvényt használjuk. Az asszociatív tárolóknál a `std::remove` nem használható, mert az elemek kulcsa nem módosítható; ott az `erase` tagfüggvény a helyes.

29.2.5 Az erase-remove idiom és a C++20-as erase/erase_if

A logikai eltávolítást a fizikai törléssel az **erase-remove idiom** kapcsolja össze:

```
v.erase(std::remove(v.begin(), v.end(), x), v.end());
v.erase(std::remove_if(v.begin(), v.end(), pred), v.end());
```

A `vector::erase(first, last)` a `[first, last)` elemeket destruálja, a mögötte lévőket előremozgatja, és csökkenti a `size()`-ot; a kapacitás nem változik. Az erase-remove esetében a `last` a `v.end()`, ezért a farok törlése nem mozgat, csak a destruktorokat hívja; az összköltség `O(n)`.

Az `erase` invalidálja a törölt elemekre és az utánuk következőkre mutató iterátorokat; a törölt elemek előttiek érvényesek maradnak.

C++20-ban a `std::erase(v, x)` és a `std::erase_if(v, pred)` szabad függvények a `std` névtérben minden szabványos tárolóhoz elérhetők (a pontos készletet és a library verzióját a laborprofilban kell ellenőrizni). A `std::erase(v, x)` a `x`-szel egyenlő elemeket távolítja el, a `std::erase_if(v, pred)` a predikátumot használja; mindkettő visszatér a törölt elemek számával. A `vector` esetében a `std::erase_if` a `remove_if` + `erase` kombinációt hajtja végre, és a sorrendet megőrzi; a `list` esetében a `list::remove_if`-et hívja, amely ténylegesen töröl, és csak a törölt elemekre mutató iterátorokat invalidálja. A `std::erase` nem azonos a `vector::erase` tagfüggvénnyel: az előbbi érték vagy predikátum alapján töröl, az utóbbi iteratorokkal dolgozik. A C++20-as `std::erase_if` elérhetősége a `__cpp_lib_erase_if` feature-test macroval ellenőrizhető.

29.2.6 Predicate lifetime és a mutáció közbeni invalidáció

A predikátumot az algoritmus a hívás ideje alatt használja, és érték szerint másolhatja; a predikátum által hivatkozott objektumoknak a hívás teljes ideje alatt élniük kell. A predikátumnak tisztának kell lennie az input tartományra nézve: nem módosíthatja az elemeket, és nem módosíthatja a tárolót. Ha a predikátum `push_back`-et, `erase`-t vagy más olyan műveletet hív, amely invalidálja az algoritmus iterátorait, a viselkedés nem definiált. Például a `std::remove_if(v.begin(), v.end(), [&v](int x) { v.push_back(x); return false; })` UB, mert a `push_back` reallokálhat, és a `remove_if` iterátorai invalidálódnak. A helyes megoldás: a módosításokat a `remove_if` előtt vagy után végezzük, vagy külön output tárolót használjunk. Az erase-remove esetében a `remove_if` predikátuma nem hívhat `erase`-t ugyanarra a tárolóra.

A predikátum élettartama akkor válik kritikussá, ha a predikátumot eltároljuk (például `std::function`-be tesszük), és később hívjuk: a captured referencia dangling lehet. Az algoritmusok nem tárolják el a predikátumot a hívás után, ezért a hívás alatti élettartam a lényeg. A predikátum másolása ne legyen mellékhatásos, mert az algoritmus a saját másolatán dolgozhat.

29.2.7 Kidolgozott példa: naplósorok szűrése és átalakítása

A példa egy naplófeldolgozó. A `parse_logs` a `std::transform`-ot és a `std::back_inserter`-t használja; a `main` a `std::remove_if` + `erase`-t, a `std::transform` in-place-t és a `std::copy_if` + `back_inserter`-t. A `reserve` a kapacitást biztosítja. A kód C++20-at igényel; a laborprofilban rögzített GCC vagy Clang verzióval fordítandó.

```
#include <algorithm>
#include <cassert>
#include <iostream>
#include <iterator>
#include <string>
#include <vector>

struct LogLine {
    std::string level;
    std::string message;
};

std::vector<LogLine> parse_logs(const std::vector<std::string>& raw) {
    std::vector<LogLine> out;
```

```
    out.reserve(raw.size());
    std::transform(raw.begin(), raw.end(), std::back_inserter(out),
                   [](const std::string& line) {
                       auto pos = line.find(' ');
                       if (pos == std::string::npos) {
                           return LogLine{"UNKNOWN", line};
                       }
                       return LogLine{line.substr(0, pos), line.substr(pos +
                        ↪  1)};
                   });
    return out;
}

int main() {
    std::vector<std::string> raw = {
        "INFO started", "DEBUG cache miss", "ERROR disk full",
        "DEBUG retry", "INFO stopped"
    };
    auto logs = parse_logs(raw);
    assert(logs.size() == 5);

    auto new_end = std::remove_if(logs.begin(), logs.end(),
        [](const LogLine& l) { return l.level == "DEBUG"; });
    logs.erase(new_end, logs.end());
    assert(logs.size() == 3);

    std::transform(logs.begin(), logs.end(), logs.begin(),
        [](const LogLine& l) {
            return LogLine{l.level, "[kept] " + l.message};
        });

    std::vector<LogLine> errors;
    std::copy_if(logs.begin(), logs.end(), std::back_inserter(errors),
        [](const LogLine& l) { return l.level == "ERROR"; });
    assert(errors.size() == 1);

    for (const auto& l : logs) {
        std::cout << l.level << ": " << l.message << '\n';
    }
}
```

A build parancs:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o logdemo logdemo.cpp
```

Futtatás:

```
./logdemo
```

Kimenet (`lab-gcc` profil):

```
INFO: [kept] started
ERROR: [kept] disk full
```

```
INFO: [kept] stopped
```

A három megmaradt sor a `DEBUG` sorok eltávolítása után látszik; a `transform` a `[kept]` prefixet adta, a `copy_if` pedig egy `ERROR` elemet másolt az `errors` tárolóba. **Buildjegyzőkönyv** (`lab-gcc` profil): a program `-D_GLIBCXX_DEBUG` és ASan/UBSan mellett is a fenti kimenettel futott.

29.2.8 Tipikus hibák és javításuk

A leggyakoribb hiba, hogy a `std::copy` outputja kisebb, mint a bemenet. A `std::copy` nem ellenőrzi a kapacitást, és nem dob kivételt; ha a `d_first` által írt tartomány túl rövid, a viselkedés nem definiált. Ilyenkor `std::back_inserter`-t vagy előre biztosított kapacitást kell használni. A második gyakori hiba a `std::copy(v.begin(), v.end(), v.begin() + 1)` jobbra tolás: az output a bemenet belsejében van, ezért UB; helyette `std::copy_backward(v.begin(), v.end() - 1, v.end())` kell. A harmadik, hogy a `std::remove` eredményét nem használjuk: a `remove` nem töröl, csak a logikai end-et adja vissza; `erase` nélkül a méret nem változik, és a „törölt” elemek a végén maradnak. A negyedik, hogy a `std::unique`-ot rendezés nélkül használjuk: csak a szomszédos duplikátumokat távolítja el. Az ötödik, hogy a `std::list`-en `std::remove`-ot használunk a `list::remove` helyett: a `std::remove` a listán is csak logikai, a tagfüggvény ténylegesen töröl. A hatodik, hogy a predikátum módosítja a tárolót: ez UB. A hetedik, hogy a `std::move` algoritmust összekeverjük a `std::move` kategóriakonverzióval. A nyolcadik, hogy a `fill_n`-t negatív `n`-nel hívjuk: ez UB.

29.2.9 Összefoglaló

A módosító algoritmusok szerződése három ponton dől el: az output kapacitás, az átfedés és az élettartam. Az output iterator nem ismeri a tároló kapacitását, ezért a hívónak kell biztosítania a helyet; a `back_inserter` kényelmes, de allokálhat és invalidálhat. A `copy` és a `copy_backward` iránya eltérő, ezért az átfedő másolásnál a megfelelőt kell választani. A `remove` és a `remove_if` logikai eltávolítás; a fizikai törléshez `erase` kell, vagy C++20-ban `std::erase`/`std::erase_if`. A predikátum ne módosítsa a tárolót, és a captured objektumok élettartama tartsa ki a hívást.

29.2.10 Feladatok

29.2.10.1 1. feladat: remove vagy erase?

Magyarázd el, miért nem csökkenti a `std::remove` a `std::vector` méretét, és mi a különbség a `std::remove` és a `vector::erase` között. Mi a `[new_end, last)` elemek állapota, és mely iterátorok invalidálódnak az `erase` után?

29.2.10.2 2. feladat: biztonságos átalakítás

Adott egy `std::vector<std::string>`. Távolítsd el az üres sztringeket, majd alakítsd kisbetűssé a megmaradókat. Használj erase-remove-ot és `std::transform`-ot. Biztosítsd az output kapacitást, és indokold, miért biztonságos az in-place transform.

29.2.10.3 3. feladat: átfedő másolás diagnosztikája

Az alábbi kód UB-t tartalmaz:

```
std::vector<int> v{1, 2, 3, 4, 5};
std::copy(v.begin(), v.end(), v.begin() + 1);
```

Találd meg a hibát, írd le a helyes változatot, és indokold, miért lesz az eredmény `{1, 1, 2, 3, 4}`.

29.2.10.4 4. feladat: C++20 erase_if

Írj egy függvényt, amely egy `std::vector<int>`-ből eltávolítja a negatív számokat, és visszaadja a törölt elemek számát. Használj C++20-as `std::erase_if`-et, majd hasonlítsd össze az erase-remove változattal az invalidáció szempontjából.

29.2.11 Megoldások

29.2.11.1 1. megoldás

A `std::remove` nem töröl, hanem az el nem távolított elemeket előremozgatja, és visszaadja az új logikai end-et. A `size()` változatlan marad; a `[new_end, last)` elemek moved-from állapotban vannak, értékük meghatározatlan, de az objektumok léteznek. A `vector::erase` destruálja a `[first, last)` elemeket, előremozgatja a mögötte lévőket, és csökkenti a `size()`-ot. Az `erase` invalidálja a törölt elemekre és az utánuk következőkre mutató iterátorokat; a törölt elemek előttiek érvényesek maradnak. Ellenőrzés: `assert(v.size() == expected);`.

29.2.11.2 2. megoldás

```
#include <algorithm>
#include <cassert>
#include <cctype>
#include <string>
#include <vector>

void normalize(std::vector<std::string>& lines) {
    lines.erase(std::remove_if(lines.begin(), lines.end(),
                               [](const std::string& s) { return s.empty(); }),
                lines.end());
    std::transform(lines.begin(), lines.end(), lines.begin(),
                   [](std::string s) {
                       std::transform(s.begin(), s.end(), s.begin(),
                                      [](unsigned char c) {
                                          return static_cast<char>(
                                              std::tolower(c));
                                      });
                       return s;
                   });
}
```

```
int main() {
    std::vector<std::string> lines{"Hello", "", "WORLD", "", "C++"};
    normalize(lines);
    assert((lines == std::vector<std::string>{"hello", "world", "c++"}));
}
```

Az erase-remove eltávolítja az üres sztringeket; a `transform` in-place a `lines`-on dolgozik. Az in-place biztonságos, mert a lambda a `std::string`-et érték szerint veszi át, így a saját másolatán dolgozik, és a visszatérési értéket a `transform` a `*it`-hez rendeli. A `std::tolower unsigned char`-ra castolva kerüli a negatív karakterek UB-ját. Ellenőrzés: az `assert` a várt három elemet ellenőrzi.

29.2.11.3 3. megoldás

A hiba, hogy a `std::copy` outputja a `[v.begin(), v.end())` belsejében van (`v.begin() + 1`), ezért a `copy` balról jobbra haladva felülírja a még nem olvasott elemeket; ez UB. A helyes változat:

```
std::copy_backward(v.begin(), v.end() - 1, v.end());
```

A `copy_backward` az utolsó elemtől kezd visszafelé, ezért nem írja felül a még nem olvasott elemeket. Az input `[v.begin(), v.end() - 1)` = `{1, 2, 3, 4}`, az output `[v.begin() + 1, v.end())` = `{2, 3, 4, 5}` helyére kerül, így az eredmény `{1, 1, 2, 3, 4}`. Ellenőrzés: `assert((v == std::vector<int>{1, 1, 2, 3, 4}));`.

29.2.11.4 4. megoldás

```
#include <cstddef>
#include <cassert>
#include <vector>

std::size_t remove_negatives(std::vector<int>& v) {
    return std::erase_if(v, [](int x) { return x < 0; });
}

int main() {
    std::vector<int> v{1, -2, 3, -4, 5};
    auto removed = remove_negatives(v);
    assert(removed == 2);
    assert((v == std::vector<int>{1, 3, 5}));
}
```

A `std::erase_if` a `vector` esetében a `remove_if` + `erase` kombinációt hajtja végre, és visszatér a törölt elemek számával. Az erase-remove változat ugyanazt a hatást éri el, de a törölt elemek számát a `std::distance(new_end, v.end())` adja. Mindkettő invalidálja a törölt elemekre és az utánuk következőkre mutató iterátorokat; a `vector` esetében a törölt elemek előttiek érvényesek maradnak. A `std::erase_if` elérhetőségét a `__cpp_lib_erase_if` feature-test macroval kell ellenőrizni. Ellenőrzés: a két `assert`.

29.2.12 Forrás és korlátok

A fejezet a C++20 nyelvi és könyvtári alapra épül; a C++23-as változásokat nem tárgyalja. A szabvány pontos paragrafusait, az implementációk eltéréseit és a `std::erase_if` elérhetőségét a laborprofilban kell ellenőrizni. A szöveg nem tartalmaz szabványidézetet, és a kódpéldák nincsenek lefuttatva; a kimenetek illusztrációk, nem mért eredmények. A `std::erase_if` feature-test macroja `__cpp_lib_erase_if`; ennek tényleges értékét a fordítónál és a standard library verziójánál kell ellenőrizni.

29.3 Rendezés, keresés és comparator szerződés

Ez az alfejezet arról szól, hogyan lehet egy algoritmust tárolótól függetlenül meghívni, és miért nem elég ehhez a szintaxis ismerete. Egy DevOps-környezetben a legtöbb „rendezés” és „keresés” valójában napló- vagy metrikarekordokon fut: időbélyeggel, kapcsolatazonosítóval, késleltetéssel. Ezeknél a rendezettség nem esztétikai kérdés, hanem előfeltétel, amelyet az algoritmus nem ellenőriz, és amelynek megsértése csendben hibás választ ad. A comparator itt nem kényelmi paraméter, hanem szerződés: az algoritmus pontosan annyit tud, amennyit a comparator kifejez. A rendezési kategóriák (`<`, `<=>`, egyenlőség) korábbi tárgyalását itt feltételezzük, ahogy a módosító algoritmusok range-alapú szemléletét is — lásd Egyenlőség, spaceship és rendezési kategóriák (23.4) és Transform, copy, remove és módosító algoritmusok (29.2).

29.3.1 A comparator mint szerződés: strict weak ordering

Az `std::sort`, `std::lower_bound` és társai `Compare` paramétere akkor érvényes, ha **strict weak ordering** (szigorú gyenge rendezés). Négy feltétel:

1. **irreflexivitás**: `comp(x, x)` hamis;
2. **aszimmetria**: ha `comp(a, b)` igaz, akkor `comp(b, a)` hamis;
3. **tranzitivitás**: `comp(a, b) && comp(b, c)` esetén `comp(a, c)`;
4. **az összehasonlíthatatlanság tranzitivitása**: ha `a` és `b`, valamint `b` és `c` összehasonlíthatatlan, akkor `a` és `c` is az.

Az összehasonlíthatatlanság külön nevet kap: `a` és `b` **ekvivalens** (jelölés: $a \sim b$), ha

$$\neg\,\mathrm{comp}(a,b) \;\wedge\; \neg\,\mathrm{comp}(b,a).$$

Ez **nem egyenlőség**. Két kapcsolatrekord ekvivalens egy időbélyeg-comparator szerint akkor is, ha a `conn_id`-juk eltér — egyszerűen csak ugyanabban a milliszekundumban zárultak. A standard könyvtár minden „egyenlőség” fogalma Comparator esetén ekvivalenciát jelent, nem `operator==`-t; ezért a keresési eredmények értelmezésénél mindig a comparator a mérvadó.

Egy rendezett tartomány definíciója is kizárólag a comparatorra épül: a `[first, last)` tartomány `comp` szerint rendezett, ha minden `i < j` indexre `comp(*j, *i)` hamis. Ebből az is következik, hogy a fél-nyílt tartomány `last` eleme nem dereferálható, és hogy a „rendezett” állapot önmagában nem mond semmit az ekvivalens elemek egymáshoz viszonyított sorrendjéről.

A szerződésszegés következménye nem definiált viselkedés: a könyvtár nem köteles észlelni, hogy a comparator nem strict weak ordering, és az eredmény (esetleg összeomlás, esetleg értelmesnek látszó, de hibás sorrend) implementációfüggő. A konkrét megfogalmazást a használt szabványverzió algoritmus-követelmény szakaszában érdemes ellenőrizni; a forrásjegyzet ehhez nem tartalmazott ellenőrzött hivatkozást, ezért itt a szerződést tartalomként, nem idézetként adjuk.

29.3.1.1 A NaN-csapda

A lebegőpontos comparator klasszikus hibája a `double` mező. A `NaN` értékre minden `<` hamis, tehát a `return a.rtt_us < b.rtt_us;` comparator mellett a NaN **minden más értékkel ekvivalens** lesz. Ez pontosan a 4. feltételt sérti: legyen `a.rtt_us = 2.0`, `b.rtt_us = NaN`, `c.rtt_us = 5.0`. Ekkor $a \sim b$ és $b \sim c$, de $a \sim c$ hamis, mert `2.0 < 5.0`. Innen kezdve minden, ami a comparatorra épül, érvénytelen.

Két helyes javítás: a NaN-eket előre kiszűrni (a rekord feldolgozás előtt kap egy „nincs mérés” állapotot, és nem kerül a rendezett sorozatba), vagy totális rendezést adó comparatorral rendezni, amely a NaN-t a sor végére teszi:

```
bool by_rtt(const Conn& a, const Conn& b) {           // NaN a végére
    const bool na = std::isnan(a.rtt_us), nb = std::isnan(b.rtt_us);
    if (na != nb) return nb;                          // NaN nélküli elem előre
    if (na) return false;                             // két NaN: ekvivalens
    return a.rtt_us < b.rtt_us;
}
```

Ez strict weak ordering: az irreflexivitás és aszimmetria látszik, a tranzitivitás pedig abból, hogy az értékeket három osztályba sorolja (számok, majd NaN), és az osztályokon belül a valós `<` tranzitív.

29.3.2 Rendezési algoritmusok és költségeik

algoritmus	előfeltétel	összehasonlítás	stabil?	mit garantál
`std::sort`	random access, `comp` SWO	$O(N \log N)$	nem	teljesen rendezett sorozat
`std::stable_sort`	random access, `comp` SWO	$O(N \log N)$; elégtelen scratch memória mellett $O(N \log^2 N)$	igen	ekvivalensek relatív sorrendje megmarad
`std::partial_sort(f, m, l)`	random access, `comp` SWO	$O(N \log k)$, $k = m - f$	nem	az első k elem rendezett, a többi sorrendje nem specifikált
`std::nth_element(f, n, l)`	random access, `comp` SWO	átlagosan lineáris	nem	`*n` a helyén; előtte $\preceq$, utána $\succeq$

A táblázat negyedik oszlopa azért fontos, mert a `sort` nem garantálja az ekvivalens elemek sorrendjét. Ha a kimenet reprodukálható kell legyen (teszt, diff, audit), akkor vagy `stable_sort`-ot használunk, vagy a comparatort teljes rendezéssé egészítjük ki egy másodlagos kulccsal (`ts_ms`, majd `conn_id`) — ez a gyakorlatban a jobb választás, mert a teljes rendezés minden algoritmuson determinisztikus marad.

Az iterator-kategória nem dekoráció: a `std::sort` random access iterátort követel, ezért `std::list`-en fordítási hibát ad — ez a szerződés fordítás idejű kikényszerítése. Listán a tagfüggvény `l.sort()` a helyes eszköz (stabil merge sort, random access nélkül). A `partial_sort` és `nth_element` akkor nyer, ha $k \ll N$: „a 10 leglassabb kapcsolat” esetén $O(N \log 10)$ nagyságrendű munka, szemben a teljes $O(N \log N)$-nel. A `nth_element` mediánhoz, percentilishez jó: nem rendez, csak szétválaszt.

29.3.3 Keresés rendezett tartományon

A bináris keresés családja a **particionáltságot** használja ki, nem a teljes rendezettséget. Egy tartomány egy `p` predikátum szerint particionált, ha minden olyan elem, amelyre `p` igaz, megelőzi azokat, amelyekre hamis. Az előfeltétel algoritmusonként más predikátumra vonatkozik (C++20 [alg.binary.search]):

- `lower_bound`: particionált az `e < value` (`comp(e, value)`) szerint;
- `upper_bound`: particionált a `!(value < e)` (`!comp(value, e)`) szerint;
- `equal_range` és `binary_search`: mindkét feltétel szerint particionált, és minden `e`-re `e < value`-ból következik `!(value < e)`.

Egy `comp` szerint teljesen rendezett tartomány mindegyik feltételt teljesíti, ezért a gyakorlatban rendezett tartományon keresünk. Az egyoldali particionáltság viszont csak az adott algoritmushoz elég, és a bináris keresés egyiket sem ellenőrzi:

```
// partition_bounds.cpp - C++20: melyik keresésnek elég az egyoldali
 ↪  particionálás?
#include <algorithm>
#include <cassert>
#include <vector>

int main() {
    const int v = 3;
    std::vector<int> a{1, 2, 5, 3};
    // e < 3:      igaz, igaz, hamis, hamis  -> particionált: lower_bound
     ↪  alkalmazható
    // !(3 < e):   igaz, igaz, hamis, igaz   -> NEM particionált: upper_bound,
    //             equal_range és binary_search előfeltétele sérül
    assert(std::is_partitioned(a.begin(), a.end(), [](int e) { return e < v;
     ↪  }));
    assert(!std::is_partitioned(a.begin(), a.end(), [](int e) { return !(v < e);
     ↪  }));
    auto lb = std::lower_bound(a.begin(), a.end(), v);
    assert(lb == a.begin() + 2);              // az első nem-kisebb elem: az 5
    // std::upper_bound(a.begin(), a.end(), v);  // előfeltétel-sértés, nem
     ↪  teszt
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic -D_GLIBCXX_DEBUG partition_bounds.cpp` figyelmeztetés nélkül fordul, és mindhárom `assert` teljesül. Ugyanerre a tömbre az `upper_bound(a.begin(), a.end(), 3)` hívást a libstdc++ debug módja `elements in iterator range [first, last) are not partitioned by the value` hibával elutasítja: ez előfeltétel-sértés, nem érvényes tesztbemenet.

művelet	eredmény	összehasonlítás
`lower_bound(f,l,v,c)`	első `i`, ahol `!c(*i, v)`	$O(\log N)$
`upper_bound(f,l,v,c)`	első `i`, ahol `c(v, *i)`	$O(\log N)$
`equal_range(f,l,v,c)`	`(lower, upper)` pár	$O(\log N)$
`binary_search(f,l,v,c)`	`bool`, hogy van-e ekvivalens elem	$O(\log N)$

Az `equal_range` adja vissza az egész ekvivalencia-osztályt `[first, second)` alakban; a `binary_search` csak a létezést mondja meg, pozíció nélkül — ha a rekordra is szükség van, nem ez a megfelelő eszköz. A komplexitásáról pontosítani kell: az **összehasonlítások** száma logaritmikus, de forward iterator esetén a léptetések száma lineáris, mert a bináris keresés nem tud ugrani. Ezért `std::list`-en a `lower_bound` megengedett, de lassú; rendezett vektoron viszont valóban $O(\log N)$.

29.3.4 Kidolgozott példa: időablak lekérdezése

Adott egy lezárt kapcsolatokból álló vektor, amelyet időbélyeg szerint rendezünk, majd egy $[t_0, t_1)$ ablakra kérdezünk. A példa C++20-at használ, és önállóan fordítható; a várt kimenet **illusztráció**, nem mért eredmény.

```
// conn_query.cpp - g++ -std=c++20 -O2 -Wall -Wextra -o conn_query
 ↪  conn_query.cpp
#include <algorithm>
#include <cstdint>
#include <cstdio>
#include <vector>

struct Conn {
    std::uint64_t ts_ms;    // a kapcsolat lezárásának időpontja
    std::uint32_t conn_id;  // egyedi azonosító
    std::uint32_t rtt_us;   // mért körülfordulási idő
};

// Teljes rendezés: ts_ms, majd conn_id - determinisztikus sorrend.
bool by_time_then_id(const Conn& a, const Conn& b) {
    if (a.ts_ms != b.ts_ms) return a.ts_ms < b.ts_ms;
    return a.conn_id < b.conn_id;
}

// Durvább, csak időbélyegre néző comparator a kereséshez.
// Heterogén hívás: comp(elem, kulcs) és comp(kulcs, elem) is kell.
struct ByTime {
    bool operator()(const Conn& a, const Conn& b) const { return a.ts_ms <
     ↪  b.ts_ms; }
    bool operator()(const Conn& a, std::uint64_t t) const { return a.ts_ms < t;
     ↪  }
    bool operator()(std::uint64_t t, const Conn& b) const { return t < b.ts_ms;
     ↪  }
};

int main() {
    std::vector<Conn> conns = {
        {1000, 7, 220}, {1000, 3, 180}, {1200, 9, 310},
        {900,  1, 140}, {1200, 4, 275}, {1000, 5, 205},
    };
    std::sort(conns.begin(), conns.end(), by_time_then_id);

    const std::uint64_t t0 = 1000, t1 = 1200;   // fél-nyílt ablak
```

```
    auto lo = std::lower_bound(conns.begin(), conns.end(), t0, ByTime{});
    auto hi = std::lower_bound(conns.begin(), conns.end(), t1, ByTime{});

    std::uint64_t sum = 0; std::size_t n = 0;
    for (auto it = lo; it != hi; ++it) { sum += it->rtt_us; ++n; }
    std::printf("találat=%zu átlag=%.1f\n", n,
                n ? static_cast<double>(sum) / static_cast<double>(n) : 0.0);
}
```

A várt (nem mért) kimenet `találat=3 átlag=201.7`. Két dolgot érdemes kiemelni. Egyrészt a rendezés finomabb comparatort használ, mint a keresés: ez megengedett, mert a finom rendezés *finomítása* a durvának — ha `by_time_then_id` szerint `a` megelőzi `b`-t, akkor `ByTime` szerint is, így a particionáltság megmarad. Fordítva viszont nem működik: ha `conn_id` szerint rendeznénk és `ts_ms` szerint keresnénk, a particionáltság elvész. Másrészt a felső határhoz `lower_bound`-ot használunk `t1`-gyel, mert fél-nyílt ablakot akarunk; `upper_bound`-dal `t0`-ra ugyanezt kapnánk. Az `equal_range` akkor lenne a helyes eszköz, ha egyetlen időbélyeg összes rekordjára kíváncsiak vagyunk.

29.3.5 Lineáris keresés vagy indexépítés?

A `std::find` előfeltétel nélküli, $O(N)$ összehasonlítás, és input iterátoron is működik. A bináris kereséshez előbb rendezni kell: $O(N \log N)$ egyszeri költség. Ha k lekérdezést futtatunk, a rendezés akkor térül meg, ha

$$c_2 \cdot k \cdot N \;>\; c_1 \cdot N \log N \quad\Longrightarrow\quad k \;>\; \frac{c_1}{c_2} \log N,$$

ahol c_1 a rendezés, c_2 a lineáris keresés egy elempárra jutó költsége. A képlet csak nagyságrendi becslés: a valóságban a `find` folytonos vektoron kiválóan cache-el, ezért c_2 kicsi, és kis N-nél vagy néhány lekérdezésnél a lineáris keresés gyorsabb lehet a rendezésnél. Sok lekérdezésnél viszont a rendezett vektor a nyerő, mert a memóriája tömb, a bejárása előre jelezhető.

Harmadik út az index építése: `std::unordered_map` vagy `std::map` kulcs alapján. Ez kiküszöböli a rendezést, de csomópontonkénti allokációval és nagyobb memóriával jár, és nem ad sorrendet — időablakra nem alkalmas, egyedi kulcsra viszont igen. A döntés tehát nem „melyik a gyorsabb”, hanem „milyen lekérdezési mintát kell kiszolgálni”: pontkeresés, sorrendi bejárás vagy intervallum.

29.3.6 Tipikus hibák

- `return a.ts_ms <= b.ts_ms;` — nem irreflexív, ezért nem strict weak ordering. Az `std::sort` ettől nem definiáltan viselkedik; a helyes `<`.
- Rendezni `by_time_then_id`-vel, majd `conn_id` szerint `lower_bound`-olni — a particionáltság elvész, az eredmény csendben hibás.
- `binary_search`-ot hívni rendezetlen vektoron: nincs diagnosztika, csak rossz `bool`.
- `NaN`-t tartalmazó `double` mezőn `<`-t használni (lásd fent).
- `std::sort`-ot `std::list`-en hívni — jó hír, hogy ez fordítási hiba, nem futásidejű hiba.
- Feltételezni, hogy `sort` után az ekvivalens elemek sorrendje megmarad; arra csak a `stable_sort` ad garanciát.
- `std::lower_bound`-ot `std::set`-en hívni: a tagfüggvény `s.lower_bound(v)` a megfelelő, mert az kihasználja a fa szerkezetét.

29.3.7 Feladatok

1. (magyarázó) Egy kolléga comparatora `return a.rtt_us <= b.rtt_us;`. Mutass egy konkrét, háromelemű bemenetet, amelyen a négytagú definíció valamelyik feltétele megsérül, és mondd meg, melyik.

2. (kódolási) Írj `Conn`-ra olyan comparatort, amely `rtt_us` szerint **csökkenő** sorrendbe rendez, a `NaN` értékeket a sor végére teszi, az azonos `rtt_us`-ű elemeket pedig `conn_id` szerint növekvően rendezi. Indokold, miért strict weak ordering.

3. (diagnosztikai) Az alábbi kódrészlet lefordul és lefut, de a lekérdezés néha üres eredményt ad. Keresd meg az okot, és add meg a javítást, valamint egy olyan ellenőrzést, amely fejlesztés közben elkapja a hibát.

```
std::sort(conns.begin(), conns.end(), by_time_then_id);
auto lo = std::lower_bound(conns.begin(), conns.end(), t0,
                           [](const Conn& a, const Conn& b) { return a.conn_id <
                            ↪  b.conn_id; });
```

4. (döntési) Egy szolgáltatás induláskor 200 000 kapcsolatrekordot tölt be, majd másodpercenként átlagosan 5 időablak-lekérdezést szolgál ki. Hasonlítsd össze a „lineáris szűrés minden lekérdezésnél”, a „rendezés + `lower_bound`/`upper_bound`” és az „`unordered_map` + lineáris szűrés” megoldást, és indokold a választást. Számolj a memóriával és azzal, hogy az ablakok intervallum-lekérdezések.

29.3.8 Megoldások

1. Legyen `a.rtt_us = 5`. Ekkor `comp(a, a)` = `5 <= 5` = igaz, tehát az irreflexivitás sérül. Már kételemű, azonos értékű bemeneten is előfordulhat, hogy a rendezés „megcseréli” az elemeket, vagy az implementáció futásidejű hibát jelez; a viselkedés nem garantált, ezért az eredményre hagyatkozni nem lehet. Javítás: `<`.

2. Egy lehetséges megoldás:

```
bool by_rtt_desc(const Conn& a, const Conn& b) {
    const bool na = std::isnan(a.rtt_us), nb = std::isnan(b.rtt_us);
    if (na != nb) return nb;            // a NaN mindig hátra
    if (na) return a.conn_id < b.conn_id;
    if (a.rtt_us != b.rtt_us) return a.rtt_us > b.rtt_us;
    return a.conn_id < b.conn_id;
}
```

Indoklás: az elemeket három régióba sorolja (nem-NaN, NaN), a nem-NaN régiót a `>` szerint csökkenően rendezi, az ekvivalens (`rtt_us` egyenlő, vagy mindkettő NaN) elemeket `conn_id` szerint, ami teljes rendezés. Így a comparator értékkészlete egy Totális rendezés leszűkítése, ami automatikusan strict weak ordering. Ellenőrzési feltétel: tetszőleges bemenetre `std::is_sorted(v.begin(), v.end(), by_rtt_desc)` igaz, és a NaN-ek a végén vannak.

3. A comparator `conn_id` szerint hasonlít, miközben a vektor `ts_ms` szerint rendezett. A `lower_bound` azt feltételezi, hogy a tartomány a keresett comparatorra nézve particionált; ez itt nem teljesül, ezért az eredmény értelmetlen (a standard szerint előfeltétel-sértés, nem definiált viselkedés, a gyakorlatban csendben rossz index). Javítás: ugyanazt a comparatort használni a kereséshez, amivel rendeztünk, azaz a `ByTime` struktúrát a fenti példa szerint. Fejlesztés közbeni ellenőrzés:

```
assert(std::is_sorted(conns.begin(), conns.end(), ByTime{}));
```

Ez nem bizonyítja a teljes helyességet (a finomabb rendezés is átmegy rajta), de a durva rendezetlenséget elkapja. Erősebb védelem: a keresést egy `sorted_by(ByTime{})` invariánst fenntartó típusba zárni, vagy a keresést a rendezéssel azonos comparatort használó helper függvényen keresztül végezni.

4. 200 000 rekordnál egy lineáris szűrés rekordonként néhány tíz ns nagyságrendű, másodpercenként 5 lekérdezésnél ez elfogadható lehet, de a tartalék nő, ha a rekordszám vagy a lekérdezésszám emelkedik. A rendezés egyszeri $O(N \log N)$, utána lekérdezésenként $O(\log N)$ plusz a találati halmaz bejárása — ez a kedvező választás, mert intervallum-lekérdezésről van szó, amit a `lower_bound`/`upper_bound` pár természetesen kiszolgál, és a rendezett vektor memóriája folytonos. Az `unordered_map` pontkeresésre jó, intervallumra nem: egy időablakot nem tudunk vele közvetlenül megválaszolni, csak végigszűrni a teljes táblát, ami rosszabb, mint a rendezett vektor. Ha a rekordok gyakran érkeznek be futásidőben, akkor a beszúrás költsége miatt érdemes inkább rendezett vektort időszakonként újraépíteni, vagy `std::multimap`-ot használni — de ez már a tárolóválasztás kérdése. Ellenőrzési feltétel: a választott megoldásra írjunk olyan tesztet, amely ismert ablakhosszra ismert darabszámot vár, és határesetként üres ablakot, $[t, t)$-t is lefed.

29.3.9 Összefoglaló

A comparator szerződés: strict weak ordering, és az „egyenlőség” mindenhol ekvivalenciát jelent. A rendezettség a comparatorra van definiálva, ezért a rendezéshez és a kereséshez használt comparatoroknak kompatibilisnek kell lenniük — a finomabb rendezés durvább kereséssel párosítható, fordítva nem. A `sort` nem stabil, a `stable_sort` az; a `partial_sort` és az `nth_element` akkor nyer, ha nem kell a teljes sorrend. A bináris keresés előfeltétele a particionáltság, nem „valami rendezettség”; a `binary_search` csak létezést ad, pozícióhoz `lower_bound`/`equal_range` kell. Kis N-nél vagy kevés lekérdezésnél a lineáris `find` versenyképes, sok intervallum-lekérdezésnél a rendezett vektor a helyes alap.

> Megjegyzés a verziókhoz: a példák ISO C++20-at és a megadott fordítópárt feltételezik; a `std::isnan` a `<cmath>`-ból, a `std::is_sorted` az `<algorithm>`-ból érkezik. A comparator-követelmények normatív megfogalmazása szabványverziónként változhat, ezért a konkrét szöveget a használt szabványban és a laborprofilban rögzített libstdc++/libc++ kiadás dokumentációjában kell ellenőrizni. A fenti kimenet illusztráció, nem mért eredmény.

29.4 Numerikus algoritmusok és kiértékelési sorrend

A `<numeric>` fejléc algoritmusai első ránézésre egyszerű összegzések, mégis a legtöbb meglepetés nem a műveletben, hanem a kiértékelési sorrendben és az accumulator típusában rejlik. Ez a rész a Rendezés, keresés és comparator szerződés (29.3) comparator-szerződésére épül: ott a rendezés és keresés előfeltételeit rögzítettük; itt azt vizsgáljuk, milyen szerződést ír elő az `accumulate`, a `reduce`, a prefixösszegek és az `inner_product`. A lebegőpontos hibák hátterét a Lebegőpontos reprezentáció és numerikus hibák (4.3), a signed túlcsordulását a Promóciók, signedness és túlcsordulás (4.2) tárgyalta. A cél nem a numerikus analízis teljes kurzusa, hanem hogy a standard algoritmus kiválasztásakor a sorrendi és típusbeli következményeket tudatosan kezeld.

29.4.1 Az accumulator típusa és az `accumulate` szerződése

Az `std::accumulate` C++98 óta a `<numeric>`-ban található. A legegyszerűbb alak:

```
template<class InputIt, class T>
T accumulate(InputIt first, InputIt last, T init);
```

A hatás: `init = init + *first`, majd `init = init + *(first+1)`, és így tovább, balról jobbra, a megadott sorrendben. A `T` a kezdeti érték típusa, az accumulator típusa és a visszatérési típus. A `*it` minden lépésben `T`-vé konvertálódik. Ha a tartomány üres, az eredmény `init`. A bináris műveletet elfogadó változat:

```
template<class InputIt, class T, class BinaryOp>
T accumulate(InputIt first, InputIt last, T init, BinaryOp op);
```

Itt `init = op(init, *it)`, szintén balról jobbra. Az `op` nem módosíthatja a bemeneti tartományt, és nem hívhat olyan műveletet, amely a sorrendet megváltoztatná; a szerződés a balról jobbra haladás.

Az accumulator típusa nem a vektor értéktípusa, hanem a `T`. Ez a leggyakoribb hiba forrása:

```
std::vector<int> v{1'000'000'000, 1'000'000'000, 1'000'000'000};
auto a = std::accumulate(v.begin(), v.end(), 0);    // int accumulator
auto b = std::accumulate(v.begin(), v.end(), 0LL);  // long long accumulator
```

Az `a` kiszámítása során a második lépésben `2'000'000'000` még ábrázolható `int`-ben, a harmadik viszont már nem: signed overflow, ami a C++ szabvány szerint nem definiált viselkedés. A `b` esetén az összeadás `long long`-ban történik, mert az akkumulátor típusa `T` = `long long`, és a `std::plus<>` a `long long + int` kifejezést a szokásos aritmetikai konverzióval értékeli ki. **Nincs automatikus előkonverzió**: az algoritmus (C++20) az `init = op(std::move(init), *it)` kifejezést hajtja végre, tehát `op` az aktuális akkumulátort (`T`, itt `long long`) és az iterátor dereferálásának *eredeti* típusú eredményét (`int`) kapja. Saját operációval ez megfigyelhető: egy `Op`, amely kiírja az argumentumtípusokat, mindkét lépésben `op(long long, int)`-et lát (GCC 13.3, `-std=c++20`). Ugyanez áll akkor is, ha `T` és a bemenet `value_type`-ja különbözik: a szöveg sehol nem ígér előkonverziót, ezért az overload-feloldást és a költségmodellt az `op` tényleges paramétertípusaira kell építeni. A `0LL` tehát nem kozmetika: az accumulator típusát választja meg. Ugyanígy `0.0` és `0.0f` más lebegőpontos pontosságot ad. Az `accumulate` determinisztikus: ugyanaz a bemenet és ugyanaz a `T` ugyanazt az eredményt adja, feltéve, hogy a művelet determinisztikus. Ez teszi alkalmassá arra, amikor a reprodukálhatóság követelmény.

29.4.2 `reduce`, `transform_reduce` és a sorrend lazítása

A C++17 óta a `<numeric>` olyan algoritmusokat is tartalmaz, amelyek a sorrendet nem rögzítik. A `reduce` legegyszerűbb alakja:

```
template<class InputIt, class T>
T reduce(InputIt first, InputIt last, T init);
```

A `reduce` a `std::plus<>`-t használja, és a részeredményeket tetszőleges sorrendben és csoportosításban vonhatja össze. A `reduce` két külön dolgot kér. **Kötelező előfeltételek** (megsértésük UB): a típus-, tartomány- és mellékhatás-feltételek — az `op` nem módosíthatja a tartomány elemeit és az iterátorokat, az elemek és a köztes értékek `T`-re konvertálhatók. **Az eredmény sorrendfüggetlenségéhez szükséges algebrai tulajdonság**: ha az `op` nem asszociatív és kommutatív, a csoportosítástól függően *eltérő eredmény* adódhat, az eredmény nem specifikált, nem

hordozható; ez nem azonos az előfeltétel-sértéssel, ezért a lebegőpontos összeadás szabályos, csak a bitazonos reprodukálhatóságot nem adja. A `T` itt is az accumulator típusa. A `reduce` nem garantál balról jobbra sorrendet, ezért lebegőpontoson eltérhet az `accumulate`-tól. A `transform_reduce` alakja:

```
template<class InputIt, class T, class BinaryOp, class UnaryOp>
T transform_reduce(InputIt first, InputIt last, T init,
                   BinaryOp binary_op, UnaryOp unary_op);
```

Az `unary_op`-ot minden elemre alkalmazza, majd a `binary_op`-pal redukál. A sorrend itt is lazított. Az `unary_op` és a `binary_op` ne legyen mellékhatásos; párhuzamos policy esetén a mellékhatás adatversenyhez vezethet. A `std::execution::par` nem automatikus gyorsítás, és a policy a sorrendet tovább lazíthatja; a részleteket az Execution policy és párhuzamos algoritmusok (29.5) tárgyalja.

Fontos terminológiai különbség: a `reduce` nem „nem definiált”, hanem a sorrendre nézve nem specifikált. A nem asszociatív művelet előfeltétel-sértés; a lebegőpontos `plus` pedig tipikusan nem asszociatív. Ezért a `reduce` lebegőpontoson csak akkor használható, ha a nem specifikált sorrendet és az ebből következő eltérést tudatosan elfogadjuk.

29.4.3 Prefixösszegek: `partial_sum`, `inclusive_scan`, `exclusive_scan`

A `partial_sum` a C++98-as `<numeric>` része:

```
template<class InputIt, class OutputIt>
OutputIt partial_sum(InputIt first, InputIt last, OutputIt d_first);
```

Hatás: balról jobbra halad, és minden lépésben a kimenetre írja az addigi összeget. A részösszegek típusát **nem** a kimenet, hanem a bemenet értéktípusa adja (C++20 [partial.sum]): az akkumulátor `InputIt value_type`-jú változó (`acc = *first`, majd `acc = std::move(acc) + *i`), és csak az így képzett értéket írja a kimenetre. Egy `int` bemenet tehát `int`-ben összegződik akkor is, ha a kimenet `long long`. Széles akkumulációhoz a bemenetet kell előbb szélesíteni, vagy olyan algoritmust választani, amelynél a részeredmény típusa igazoltan a széles típus, például `inclusive_scan`-t `std::plus<long long>{}` művelettel és `long long` kezdőértékkel (a `std::plus<>` és a `0LL` önmagában nem garantálja, hogy a generalized sum bármely csoportosításánál, például `binary_op(*first, *first)` kiértékelésénél a két `int` elem széles típusban egyesül), vagy `transform_inclusive_scan`-nal, amely elemenként széles értékre alakít:

```
// partial_sum_accumulator.cpp - C++20: az akkumulátor a BEMENET típusa
#include <climits>
#include <cstdio>
#include <functional>
#include <numeric>
#include <vector>

int main(int argc, char**) {
    const std::vector<int> in{INT_MAX, 1};
    std::vector<long long> out(in.size());

    if (argc > 1) {   // negatív ág: a részösszeg int-ben képződik
        std::partial_sum(in.begin(), in.end(), out.begin());
    } else {          // javított: a scan long long kezdőértékkel akkumulál
        std::inclusive_scan(in.begin(), in.end(), out.begin(),
```

```
                            std::plus<long long>{}, 0LL);   // az op maga is
                             ↪ széles
    }
    std::printf("%lld %lld\n", out[0], out[1]);
}
```

Buildjegyzőkönyv (`lab-gcc` profil, `-fsanitize=undefined -fno-sanitize-recover=undefined`). Argumentum nélkül (javított ág) a kimenet `2147483647 2147483648`. Argumentummal (`partial_sum` ág) az UBSan `signed integer overflow: 2147483647 + 1 cannot be represented in type 'int'` hibát jelez, a `long long` cél ellenére.

A `partial_sum` determinisztikus. A C++17-es scan-ek három alakja:

```
template<class InputIt, class OutputIt>
OutputIt inclusive_scan(InputIt first, InputIt last, OutputIt d_first);

template<class InputIt, class OutputIt, class BinaryOp>
OutputIt inclusive_scan(InputIt first, InputIt last, OutputIt d_first,
                        BinaryOp op);

template<class InputIt, class OutputIt, class BinaryOp, class T>
OutputIt inclusive_scan(InputIt first, InputIt last, OutputIt d_first,
                        BinaryOp op, T init);

template<class InputIt, class OutputIt, class T>
OutputIt exclusive_scan(InputIt first, InputIt last, OutputIt d_first, T init);
```

Az `inclusive_scan` negyedik paramétere tehát a **bináris művelet**, nem a kezdőérték; kezdőértéket csak az ötparaméteres alak fogad, a művelet után. Az `inclusive_scan(a.begin(), a.end(), out.begin(), 0)` ezért nem fordul le: a `0`-t műveletként próbálná hívni. Az `exclusive_scan`-nél fordított a helyzet, ott a negyedik paraméter a kötelező kezdőérték.

Az `inclusive_scan` a `d_first[i]`-be az első `i+1` elem összegét írja (az ötparaméteres alakban az `init`-tel együtt); az `exclusive_scan` a `d_first[i]`-be az `init` és az első `i` elem összegét, tehát az első kimenet `init`. A scan-ek sorrendje nem specifikált, és a műveletnek asszociatívnak kell lennie. Kezdőérték megadásakor a `T` az `init` típusa, és a részeredmények `T`-ben képződnek; kezdőérték nélkül a bemenet értéktípusában. A `partial_sum` és az `inclusive_scan` közötti különbség tehát nem csak az `init`: az előbbi determinisztikus, az utóbbi párhuzamosítható és nem determinisztikus. Egész számokon, ahol az összeadás asszociatív, a két eredmény megegyezik; lebegőpontoson nem feltétlenül. A bemenet és a kimenet átfedése a standard szövegében ellenőrizendő; a biztonságos minta külön kimeneti buffer.

29.4.4 `inner_product`, overflow és a kezdeti érték csapdája

Az `inner_product` a skaláris szorzatot számolja:

```
template<class InputIt1, class InputIt2, class T>
T inner_product(InputIt1 first1, InputIt1 last1, InputIt2 first2, T init);
```

Hatás: `init = init + (*first1 * *first2)`, balról jobbra. A `T` a kezdeti érték és az accumulator típusa. A csapda: a `*first1 * *first2` szorzás a szokásos aritmetikai konverzió szerint történik, még mielőtt az `init`-hez adódna. Ha mindkét elem `int`, a szorzat `int`, és már a szorzat is túlcsordulhat. A `0LL` kezdeti érték önmagában nem véd:

```
std::vector<int> a{100'000}, b{100'000};
auto rossz = std::inner_product(a.begin(), a.end(), b.begin(), 0LL);
// a * b == 10'000'000'000, de a szorzat int-ben túlcsordul
```

A helyes megoldás a bináris műveletek típusának megadása:

```
auto jo = std::inner_product(a.begin(), a.end(), b.begin(), 0LL,
                             std::plus<long long>{},
                             std::multiplies<long long>{});
```

Ekkor a szorzás is `long long`-ban történik. Ugyanez `transform_reduce`-szal:

```
auto jo2 = std::transform_reduce(a.begin(), a.end(), b.begin(), 0LL,
                                 std::plus<long long>{},
                                 [](int x, int y) {
                                     return static_cast<long long>(x) * y;
                                 });
```

A `transform_reduce` C++17-es; az `inner_product` bináris op-os változatában az első művelet az összeadás, a második a szorzás. Az `inner_product` sorrendje balról jobbra, determinisztikus; a `transform_reduce` sorrendje nem specifikált.

29.4.5 Lebegőpontos asszociativitás és reprodukálhatóság

A lebegőpontos összeadás nem asszociatív. A laborprofilban rögzített x86-64 System V ABI és IEEE 754 binary32 feltételezés mellett a `float` 24 bites szignifikáns pontossággal rendelkezik. A $2^{24} = 16\,777\,216$ és a 2^{25} között a szomszédos ábrázolható értékek különbsége 2, ezért a $16\,777\,217$ nem ábrázolható; a `16777216.0f + 1.0f` eredménye a legközelebbi ábrázolható érték, azaz `16777216.0f`. Ebből a levezetésből:

- `16777216.0f + 1.0f + 1.0f` balról jobbra: `16777216.0f`, majd ismét `16777216.0f`.
- `16777216.0f + (1.0f + 1.0f)`: a zárójelben `2.0f`, majd `16777218.0f`.

A két csoportosítás különbözik. Ezért a `reduce` és a scan-ek, amelyek a csoportosítást megváltoztathatják, lebegőpontoson nem adnak garantáltan ugyanolyan eredményt, mint az `accumulate`. A `std::numeric_limits<float>::is_iec559` értéke a laborban ellenőrizendő; a C++ szabvány nem garantálja az IEEE 754-et, ezért a fenti példa a laborprofilra érvényes, nem minden platformra.

Reprodukálhatóság: ha a követelmény az, hogy ugyanaz a bemenet minden futásban bitre ugyanazt az eredményt adja, akkor az `accumulate` vagy a `partial_sum` a választás, vagy egy rögzített blokkolású saját összegzés. A `reduce` és a scan-ek ezt nem garantálják; a `par` policy tovább növeli a változékonyságot. A `long double` nem feltétlenül hordozhatóbb: x86-64-en 80 bites, AArch64-en a fordító és a libc döntése szerint eltérhet. A Kahan-összegzés csökkenti a kerekítési hibát, de nem teszi asszociatívvá a műveletet, és nem ad bitre reprodukálható eredményt minden sorrendben.

Az alábbi példa a fő kísérleteket gyűjti össze. A kód a laborban fordítandó és futtatandó; a kimenet a mérési jegyzőkönyvbe kerül, nem tekintjük előre rögzített eredménynek.

```
// numlab.cpp - C++20, normál labor: UBSan alatt is hibamentes
#include <numeric>
#include <vector>
#include <iostream>
```

```cpp
#include <limits>
#include <functional>

int main() {
    // 1. accumulator típusa
    std::vector<int> v{1'000'000'000, 1'000'000'000, 1'000'000'000};
    // auto bad = std::accumulate(v.begin(), v.end(), 0); // signed overflow: UB
    auto good = std::accumulate(v.begin(), v.end(), 0LL);
    std::cout << "accumulate 0LL: " << good << '\n';

    // 2. lebegőpontos sorrend
    std::vector<float> f{16777216.0f, 1.0f, 1.0f};
    auto acc = std::accumulate(f.begin(), f.end(), 0.0f);
    auto red = std::reduce(f.begin(), f.end(), 0.0f);
    std::cout << "accumulate float: " << acc << '\n';
    std::cout << "reduce float (unspecified order): " << red << '\n';

    // 3. prefixösszegek
    std::vector<int> a{1, 2, 3, 4};
    std::vector<int> ps(a.size()), inc(a.size()), exc(a.size());
    std::partial_sum(a.begin(), a.end(), ps.begin());
    std::inclusive_scan(a.begin(), a.end(), inc.begin());   // nincs init
    std::exclusive_scan(a.begin(), a.end(), exc.begin(), 0);
    std::cout << "partial_sum: ";
    for (int x : ps) std::cout << x << ' ';
    std::cout << "\ninclusive_scan: ";
    for (int x : inc) std::cout << x << ' ';
    std::cout << "\nexclusive_scan: ";
    for (int x : exc) std::cout << x << ' ';
    std::cout << '\n';

    // 4. inner_product overflow
    std::vector<int> x{100'000}, y{100'000};
    // A szorzás is long long-ban történik; a 0LL kezdőérték önmagában
    // nem elég (lásd a külön negatív próbát: inner_product_overflow.cpp).
    auto good_ip = std::inner_product(x.begin(), x.end(), y.begin(), 0LL,
                                      std::plus<long long>{},
                                      std::multiplies<long long>{});
    std::cout << "inner_product long long: " << good_ip << '\n';

    // 5. numeric_limits
    std::cout << "int max: " << std::numeric_limits<int>::max() << '\n';
    std::cout << "float is_iec559: "
              << std::numeric_limits<float>::is_iec559 << '\n';
}
```

Buildparancs a laborprofilban rögzített GCC-vel vagy Clang-gel:

```
g++ -std=c++20 -O2 -march=x86-64 -ffp-contract=off -Wall -Wextra -pedantic
 ↪  numlab.cpp -o numlab
./numlab
```

A lebegőpontos eredmény reprodukálhatósága nem általános C++ garancia: a szabvány meg-

engedi a nagyobb belső pontosságot és a kifejezések összevonását (például `a * b + c` egyetlen FMA-utasítássá), és a `std::reduce` eleve tetszőleges csoportosítást. Reprodukálható eredményt csak rögzített aritmetikai modell mellett várhatunk: rögzített target és `-march`, `FLT_EVAL_METHOD == 0` (x86-64 SSE2), alapértelmezett kerekítési mód, kikapcsolt kontrakció (`-ffp-contract=off`), `-ffast-math` nélkül, és rögzített összegzési sorrend (`accumulate`, nem `reduce`). A laborazonosító ezért a teljes FP-profilt tartalmazza: `g++ -std=c++20 -O2 -march=x86-64 -ffp-contract=off`. **Mérés** (`lab-gcc`): egy `a * b + c` kifejezés (`a = 1 + 2^-30`, `b = 1 - 2^-30`, `c = -1`) `-O0`, `-O2` és `-march=haswell -ffp-contract=off` mellett 0-t, `-march=haswell` mellett — a GCC C++ módban alapértelmezett kontrakciójával — `-8.67e-19`-et adott. Ez megengedett implementációs eltérés, nem szabványsértés; a `-ffast-math` ezzel szemben a szabványos szemantikán túli átalakításokat is megenged, ezért a reprodukálhatósági kísérletben ne használd. A normál labor nem tartalmaz végrehajtott nem definiált viselkedést, ezért UBSan alatt is hibamentesen fut. Az `inner_product` túlcsordulását külön negatív forrás mutatja: a `0LL` kezdőérték csak az összeadás akkumulátorát szélesíti, a szorzás `int * int` marad.

```
// inner_product_overflow.cpp - NEGATÍV próba: szándékos signed overflow
#include <iostream>
#include <numeric>
#include <vector>

int main() {
    std::vector<int> x{100'000}, y{100'000};
    // A 0LL csak az összeadás akkumulátora; a szorzás int * int marad,
    // és 10^10 nem fér el int-ben: nem definiált viselkedés.
    auto bad_ip = std::inner_product(x.begin(), x.end(), y.begin(), 0LL);
    std::cout << bad_ip << '\n';
}

g++ -std=c++20 -O1 -g -fsanitize=undefined -fno-sanitize-recover=undefined \
    numlab.cpp -o numlab_ubsan
./numlab_ubsan                          # hibamentes
g++ -std=c++20 -O1 -g -fsanitize=undefined -fno-sanitize-recover=undefined \
    inner_product_overflow.cpp -o ip_overflow
# A bináris -fno-sanitize-recover=undefined miatt nem nulla kóddal lép ki; `set
 ↪  -euo pipefail` mellett a
# pipeline ezért önmagában bukna, így a kilépési kódot és a kimenetet külön
 ↪  vizsgáljuk:
if ./ip_overflow > ub.txt 2>&1; then echo "HIBA: az UBSan nem jelzett"; exit 1;
 ↪  fi
grep -q 'signed integer overflow' ub.txt && echo "ubsan ok"   # várt
 ↪  diagnosztika
```

Buildjegyzőkönyv (`lab-gcc` profil). A `numlab.cpp` figyelmeztetés nélkül fordult, kimenete: `accumulate 0LL: 3000000000`, `accumulate float: 1.67772e+07`, `reduce float (unspecified order): 1.67772e+07`, `partial_sum: 1 3 6 10`, `inclusive_scan: 1 3 6 10`, `exclusive_scan: 0 1 3 6`, `inner_product long long: 10000000000`, `int max: 2147483647`, `float is_iec559: 1`. UBSan alatt hibamentes. A negatív próba a `signed integer overflow: 100000 * 100000 cannot be represented in type 'int'` diagnosztikát adta. A javítás előtti változat (`inclusive_scan(..., 0)`) le sem fordult: `'__binary_op' cannot be used as a function`.

A mérési jegyzőkönyv váza; a cellák a laborban kitöltendők:

Kísérlet	Algoritmus	Accumulator	Sorrend	Eredmény	Reprodukálható?
A	`accumulate`	`float`	balról jobbra		igen
B	`reduce`	`float`	nem specifikált		nem
C	`accumulate`	`double`	balról jobbra		igen
D	`reduce`	`double`	nem specifikált		nem
E	`partial_sum`	`int`	balról jobbra		igen
F	`inclusive_scan`	`int`	nem specifikált		igen (asszociatív)

29.4.6 `numeric_limits`, `gcd/lcm` és ellenőrzött bemenet

A `std::numeric_limits` a típus tulajdonságait adja: `max()`, `lowest()`, `epsilon()`, `digits10`, `max_digits10`, `is_integer`, `is_signed`, `is_iec559`. A `max_digits10` a round-triphez szükséges decimális számjegyek száma; a `digits10` a garantáltan pontos decimális számjegyek száma. A `lowest()` a legkisebb véges érték, amely negatív is lehet; a `min()` **lebegőpontos** típusoknál a legkisebb pozitív normalizált érték (nem a legkisebb ábrázolható); **egész** típusoknál ugyanez a függvény a legkisebb (esetleg negatív) értéket adja, ezért a két típuscsaládnál nem szabad ugyanúgy olvasni — általános kódban a `lowest()` a következetes választás. Az `epsilon()` az 1.0 és a következő ábrázolható érték különbsége.

A C++17-es `std::gcd` és `std::lcm` a `<numeric>`-ban található. Ha a `std::lcm(a,b)` eredménye nem ábrázolható a `common_type`-ban, a viselkedés nem definiált. A `gcd/lcm` a `|a|` és `|b|` értékekkel dolgozik; ha `a` vagy `b` `INT_MIN`, a `|INT_MIN|` nem ábrázolható `int`-ben, ezért már a bemenet is problémás. Ellenőrzött bemenetnél a `std::from_chars` használható egész számokra C++17 óta: nem dob, nem allokál, és nem függ a locale-tól. A lebegőpontos `from_chars` támogatása a standard library verziójától függ; a `std::stod`/`std::strtod` fallback lehet, de az allokálhat és locale-függő. A `from_chars` eredménye `std::errc`, amelyet explicit ellenőrizni kell. A feature-test makrókról a Feature-test, standard library és build capability (33.5) ad további támpontot.

A `std::lcm` overflow előzetes ellenőrzése például:

```
template<class T>
bool lcm_ok(T a, T b) {
    if (a == 0 || b == 0) return true;
    const T g = std::gcd(a, b);
    const T q = a / g;
    // |q * b| <= max?
    return std::abs(q) <= std::numeric_limits<T>::max() / std::abs(b);
}
```

Ez a vázlat a `T` előjeles esetére; a `std::abs(INT_MIN)` csapdáját külön kell kezelni. A `__builtin_mul_overflow` GCC/Clang kiterjesztés, nem hordozható; a laborban jelölni kell.

29.4.7 Tipikus hibák és javításuk

- Az `accumulate` kezdeti értékét a vektor értéktípusával adjuk meg. Javítás: a kívánt accumulator típusával, például `0LL` vagy `0.0`.

- A `reduce`-t lebegőpontoson úgy használjuk, mintha az `accumulate` gyorsabb változata lenne. Javítás: csak tudatos nem-determinizmussal; determinisztikus igény esetén `accumulate`.
- Az `inner_product`-nál a `0LL` kezdeti értéket elégségesnek hisszük. Javítás: a szorzás típusát is meg kell adni (`std::multiplies<long long>`).
- A `partial_sum` és az `inclusive_scan` azonosnak tekintése. Javítás: a sorrendi szerződés különbözik; a scan nem deterministikus.
- A `lcm` eredményét ellenőrzés nélkül használjuk. Javítás: előzetes tartományellenőrzés.
- A `from_chars` lebegőpontos támogatását nem ellenőrizzük. Javítás: feature-test, illetve dokumentált fallback.

29.4.8 Feladatok

1. Magyarázd el, miért adhat az `std::accumulate(v.begin(), v.end(), 0)` és a `0LL` kezdeti érték eltérő eredményt, és melyik esetben beszélünk UB-ról. Adj példát a `std::numeric_limits<int>::max()` felhasználásával.
2. Írj egy `double sum_ordered(const std::vector<double>& v)` függvényt, amely garantáltan balról jobbra összegez, és egy `double sum_unordered(const std::vector<double>& v)` függvényt, amely `std::reduce`-t használ. Magyarázd meg, melyik reprodukálható. A kódhoz adj buildparancsot.
3. Adott a következő hívás: `std::inner_product(a.begin(), a.end(), b.begin(), 0LL)`, ahol `a` és `b` `std::vector<int>`. A program `-fsanitize=undefined` alatt fut. Mi a hiba, hogyan detektálod, és hogyan javítod? Írd le a javított hívást.
4. A `std::lcm(INT_MIN, 2)` hívás mikor ad hibás eredményt, és hogyan ellenőrizhető előzetesen? Milyen típusra érdemes váltani?

29.4.9 Megoldások

1. Az első hívás `int` accumulatort használ: a részösszegek `int`-ben képződnek, és ha valamelyik meghaladja az `INT_MAX`-et, signed overflow történik, ami UB. A `0LL` esetén a `T` `long long`, a `*it` `int` értéke az összeadás előtt `long long`-ra konvertálódik, így a nagyobb tartományig jól definiált. Ellenőrzés: a vektor elemeinek összege hasonlítható a `std::numeric_limits<int>::max()` értékhez; a `long long` túlcsordulását is ellenőrizni kell, ha a bemenet nagyon nagy.

2. A determinisztikus változat:

```
double sum_ordered(const std::vector<double>& v) {
    return std::accumulate(v.begin(), v.end(), 0.0);
}
```

A `reduce`-es változat:

```
double sum_unordered(const std::vector<double>& v) {
    return std::reduce(v.begin(), v.end(), 0.0);
}
```

Az `accumulate` balról jobbra halad, ezért ugyanarra a bemenetre ugyanazt a bitmintát adja, feltéve, hogy a fordítás nem lazítja a lebegőpontos szabályokat. A `reduce` sorrendje nem specifikált, és a `double` összeadás nem asszociatív, ezért az eredmény futásonként vagy implementációnként eltérhet. Build: `g++ -std=c++20 -O2 -Wall -Wextra -pedantic main.cpp -o sumlab`.

3. A hiba az, hogy a `*first1 * *first2` szorzat `int`-ben történik, és már a szorzat túlcsordul, mielőtt a `long long` accumulatorhoz adódna. Az UBSan `signed-integer-overflow` jelentést ad. A javítás:

```
auto jo = std::inner_product(a.begin(), a.end(), b.begin(), 0LL,
                             std::plus<long long>{},
                             std::multiplies<long long>{});
```

Ezzel a szorzás és az összegzés is `long long`-ban történik. Ellenőrzési feltétel: a szorzatok és a részösszegek a `long long` tartományában maradnak; ezt a bemenet mérete és a `std::numeric_limits<long long>::max()` alapján kell igazolni.

4. A `std::lcm(INT_MIN, 2)` esetén a `|INT_MIN|` nem ábrázolható `int`-ben, ezért a `gcd`/`lcm` már a bemenet feldolgozásakor UB-ot okozhat. Előzetesen `long long`-ra kell konvertálni, vagy ellenőrizni kell, hogy a bemenet nem `INT_MIN`. A `long long` változat használata javasolt, majd az eredményt ellenőrizni kell, hogy befér-e a cél-típusba. Az ellenőrzés történhet `std::numeric_limits` és osztásos összehasonlítás segítségével, `__builtin_mul_overflow`-val csak GCC/Clang alatt.

29.4.10 Összefoglaló

A numerikus algoritmusok kiválasztása három kérdésre vezethető vissza: milyen típusú az accumulator, milyen sorrendi szerződést vállal az algoritmus, és milyen tartományban marad a művelet. Az `accumulate` és a `partial_sum` balról jobbra determinisztikus; a `reduce`, a scan-ek és a `transform_reduce` sorrendje nem specifikált, ezért lebegőpontoson csak tudatos kompromisszummal használhatók. Az `inner_product` szorzattípusa külön csapda. A `numeric_limits`, a `gcd`/`lcm` és a `from_chars` az ellenőrzött bemenet eszközei. A laborban a mérési jegyzőkönyv rögzítse a fordítót, az optimalizálást és a kapott bitmintát.

29.5 Execution policy és párhuzamos algoritmusok

A standard algoritmusok jelentős része C++17-től fogad egy úgynevezett *végrehajtási policy* (`execution policy`) paramétert, amely a hívó és a könyvtár közötti végrehajtási szerződést rögzíti. A policy nem gyorsítási kapcsoló: azt mondja meg, hogy a könyvtár milyen mértékű párhuzamosítást és milyen vektorizálást feltételezhet, ehhez képest viszont a callback (`element access function`) oldalán szigorú szinkronizációs és mellékhatás-korlátok érvényesek. Ez az alfejezet a policy-k pontos szemantikáját, a callback-szerződést, a kivételkezelés policy-függő viselkedését, valamint a backend- és linkfüggőségeket, illetve a gyakorlatban gyakran elrontott méretezési és élettartamkérdéseket tárgyalja. Az algoritmusok numerikus definícióját és kiértékelési sorrendjét a Numerikus algoritmusok és kiértékelési sorrend (29.4) rész tárgyalta, a konkurens élettartam-hibák taxonómiáját pedig a Konkurens élettartam és nehéz hibák (15.5) alfejezet; azokra itt csak célzottan hivatkozunk.

A szabályok forrása a forrásjegyzék C++20 tétele: [execpol], [algorithms.parallel.exec] és [algorithms.parallel.exceptions]. A compiler- és libraryfüggő részleteket külön jelöljük. A `seq_terminate.cpp` próbát és a backend-megfigyeléseket a `lab-gcc` profilban futtattuk; a többi részlet a tanuló laborjában ellenőrizendő.

29.5.1 A négy policy és szerződésük

A `<execution>` fejléc a `std::execution` névtérben négy jelölőobjektumot definiál, mindegyik egy-egy címkéző típus `inline constexpr` példánya. A C++17-es alap hármat tartalmaz, a negyedik C++20-ban került be:

Jelölő (C++ verzió)	Típus	Szálak	Egy szálon belül	Vektorizálás	Kivétel
`seq` (C++17)	`sequenced_policy`	egy	szekvenciális	nincs	`std::terminate`
`par` (C++17)	`parallel_policy`	több	indeterminately sequenced	nincs	`std::terminate`
`par_unseq` (C++17)	`parallel_unsequenced_policy`	több	interleavelt / vektorizált	megengedett	`std::terminate`
`unseq` (C++20)	`unsequenced_policy`	egy	interleavelt / vektorizált	megengedett	`std::terminate`

A `seq` alatt az elem-hozzáférések a hívó szálon, szekvenciálisan futnak, mint policy nélkül. A megfigyelhető viselkedés azonban **nem** teljesen azonos: a callbackből kiszökő kivétel `seq` alatt is `std::terminate`-et vált ki, policy nélkül viszont a hívóhoz propagálódik (lásd a kivételszakaszt). A `par` engedi, hogy a hívások több szálon fussanak, de egy adott szálon belül az elem-hozzáférések sorrendje továbbra is *indeterminately sequenced* – azaz nem interleave-elődnek, csak a szálak közötti sorrend nem determinisztikus. A `par_unseq` és az `unseq` ehhez képest megengedi, hogy a hívások ugyanazon a szálon belül is interleavelődjenek (utasításszintű párhuzamosság, SIMD-vektorizálás). A `par` és `par_unseq` esetén a *hány szálon* és *melyik szálon* fut egy hívás, a szabvány szerint nem specifikált; a szabvány azt is kimondja, hogy az implementáció ezeket a policy-kat akár szekvenciálisan is végrehajthatja.

Fontos következmény: a policy önmagában nem ígéri, hogy a futás gyorsabb lesz, sőt azt sem, hogy ténylegesen párhuzamosan fut. Ezért a policy megválasztása nem teljesítményoptimalizálás, hanem *szemantikai* döntés: mit engedhetsz meg a callbackben, és milyen garanciákat vállalsz a hívó oldalán. Egy további korlát: a range-alapú algoritmusok (`std::ranges::sort`, `std::ranges::for_each`, …) a C++20-as és C++23-as szabvány szerint nem fogadnak végrehajtási policy-t, csak az iteratorpáros overloadok. Ez a részlet a generálás időpontjában a rendelkezésre álló nyilvános draft állapotán alapul; verzióellenőrzést igényel.

29.5.2 Callback-szerződés: adatverseny és vectorization-unsafe műveletek

A `par` policy alatt a callback több szálon futhat, ezért a megosztott állapot védelme a hívó felelőssége. A nyelvi definíció (C++20 [intro.races]): két hozzáférés **ütközik**, ha ugyanazt a memóriahelyet érik el, és legalább az egyik módosítja. **Adatverseny** akkor van, ha két ütköző hozzáférés különböző szálakon, egymáshoz képest rendezetlenül (egyik sem *happens before* a másik) történik, és legalább az egyik nem atomikus; az adatverseny nem definiált viselkedés. Három eset:

- **olvasás/olvasás:** nem ütközik — a megosztott, közben nem módosuló állapot szinkronizáció nélkül is olvasható;
- **olvasás/írás:** ütközik — védelem (atomikus típus, mutex vagy happens-before kapcsolat) nélkül adatverseny;
- **írás/írás:** ütközik — ugyanígy adatverseny.

Egy csak olvasó `par` callback (például egy előre felépített, változatlan keresőtáblából olvas) tehát szabályos. A gyakori hiba az írás/írás eset:

```
#include <algorithm>
#include <execution>
#include <vector>

int main() {
    std::vector<int> v(1'000'000, 1);
    int count = 0;
    std::for_each(std::execution::par, v.begin(), v.end(),
                  [&count](int) { ++count; });   // ADATVERSENY
}
```

A `++count` nem oszthatatlan: olvasásból, növelésből és írásból álló olvasás–módosítás–írás, amelynek lépései két szál között összefésülődhetnek. A nyelv szintjén azonban nem az elveszett frissítés a lényeg, hanem az, hogy két szál rendezetlenül, nem atomikusan írja ugyanazt az objektumot: ez adatverseny, tehát nem definiált viselkedés, bármit is mutatna egy konkrét futás.

Buildjegyzőkönyv (`lab-gcc` profil). Ugyanezt a három esetet `std::thread`-del, `-fsanitize=thread` alatt is lefuttattuk (`setarch -R`-rel, mert a gép ASLR-beállítása mellett a TSan különben nem indul): két szál ugyanazon vektor olvasására a TSan nem jelzett, a két szálból végzett védetlen `++counter`-re `WARNING: ThreadSanitizer: data race` jelentést adott. A `std::execution::par` a `lab-gcc` profilban TBB nélkül soros háttérrel fut, ezért ott a TSan nem tudná megmutatni a versenyt — a hiba ettől még a kódban van. A helyes megoldás vagy lokális akkumulátor (a callback csak a saját szála állapotát módosítja, és a végén egy csökkentés összegzi), vagy eleve redukciós algoritmus (`std::reduce`, `std::transform_reduce`), amelynél a könyvtár vállalja a részösszegek egyesítését. A `par` alatt az is megengedett, hogy a callback a saját lokális változóit módosítsa; a tiltás csak a szálak között megosztott állapotra vonatkozik.

A `par_unseq` (és a C++20-as `unseq`) ennél szigorúbb: a vektorizálás miatt tilos minden olyan művelet, amely a callback végrehajtását blokkolja, szinkronizálja, vagy ami a SIMD-interleave miatt nem lenne biztonságos. Ide tartozik a mutex, `condition_variable`, `std::lock_guard`, `std::this_thread::sleep_for`, sőt a *blokkoló vagy más szálra várakozó* műveletek. A `std::atomic` használata **nem általánosan tilos**: a lock-free atomi művelet vektorizáció-biztonságos lehet, a nem lock-free (belül lockot használó) pedig nem; a kritérium a [algorithms.parallel.exec] szerinti „vectorization-unsafe” definíció (szinkronizálás más elem-hozzáférési függvénnyel), amelyet a használt szabványkiadásban és a `std::atomic<T>::is_always_lock_free` értékével együtt kell ellenőrizni. A szabvány ezt úgy fogalmazza meg, hogy a `par_unseq`/`unseq` policy alatt az elem-hozzáférési függvény nem hívhat „vectorization-unsafe” műveletet, és a pontos tilalmi lista ellenőrzése a kiválasztott szabvány `[algorithms.parallel.exec]` szakaszában szükséges. Egy `par` alatt legális mutex használat tehát `par_unseq` alatt már UB.

29.5.3 Kivételviselkedés

A kivételkezelésnél két algoritmuscsaládot kell szétválasztani.

Hívás	Kiszökő kivétel a callbackből	A könyvtár saját erőforráshiánya
Policy nélküli algoritmus (`std::for_each(first, last, f)`)	a hívóhoz propagálódik, `try`/`catch` elkaphatja	a szokásos módon, kivételként

Hívás	Kiszökő kivétel a callbackből	A könyvtár saját erőforráshiánya
Policy-s algoritmus, bármely szabványos policyval: `seq`, `par`, `par_unseq`, `unseq`	`std::terminate` hívódik	`std::bad_alloc` kivétel

A szabvány ([algorithms.parallel.exceptions]) mind a négy szabványos policyra ugyanazt mondja: ha egy elem-hozzáférési függvény kivétellel lép ki, `std::terminate` hívódik. A szabály tehát nem a szálhatárhoz kötött, a `seq` sem kivétel. Egyetlen kivétel a könyvtár saját erőforráshiánya: ha az algoritmus nem tud ideiglenes tárolót szerezni a párhuzamos végrehajtáshoz, `std::bad_alloc`-ot dob, és ezt a hívó elkaphatja. Az implementáció által definiált további policy-k saját szabályt adhatnak. A szabály C++17 óta érvényes, és a C++20 nem lazított rajta. A különbséget egy próba mutatja:

```
// seq_terminate.cpp - C++20: a seq policy alatt is terminate
#include <algorithm>
#include <cstdio>
#include <cstdlib>
#include <exception>
#include <execution>
#include <stdexcept>
#include <vector>

int main(int argc, char**) {
    std::set_terminate([] {
        std::puts("terminate");
        std::fflush(stdout);
        std::_Exit(42);
    });
    std::vector<int> v{1, 2, 3};
    auto fail = [](int x) { if (x == 2) throw std::runtime_error("x == 2"); };

    try {
        if (argc > 1)
            std::for_each(v.begin(), v.end(), fail);                    //
             ↪ policy nélkül
        else
            std::for_each(std::execution::seq, v.begin(), v.end(), fail);  //
             ↪ seq policy
    } catch (const std::exception& e) {
        std::printf("caught: %s\n", e.what());
    }
}
```

Buildjegyzőkönyv (`lab-gcc` profil, `g++ -std=c++20 -Wall -Wextra -Wpedantic -O2`). Argumentum nélkül (`seq` policy) a kimenet `terminate`, a kilépési kód 42, a külső `catch` nem fut. Argumentummal (policy nélkül) a kimenet `caught: x == 2`, a kilépési kód 0.

A gyakorlati következmény: bármely policy-s hívásnál, a `seq`-et is beleértve, a callbacknek gyakorlatilag `noexcept`-nek kell lennie. Ha a callback természeténél fogva hibázhat, a hibát a callbacken belül el kell kapni, adatként fel kell jegyezni, és a hívás visszatérése után kell eldönteni,

mit tegyünk. Ez a minta illeszkedik az error_code, system_error és expected (25.4) alatt tárgyalt `error_code`/`expected` alapú hibapolicy-hoz. A feljegyzés módja viszont policyfüggő:

Policy	Megengedett hibagyűjtés a callbackben
`seq`	bármely, a hívó szálon futó művelet: lokális változó, `std::vector::push_back`, akár `std::exception_ptr` tárolása
`par`	szálbiztos feljegyzés: `std::atomic` hibaszámláló vagy első hibakód, mutexszel védett gyűjtő (pl. `std::exception_ptr` vektor)
`par_unseq`, `unseq`	csak vectorization-safe művelet: **előre lefoglalt**, elemenként független kimeneti hely (hibakód-tömb), szinkronizáció, blokkolás és allokáció nélkül

`par_unseq` és `unseq` alatt a mutex, a `lock_guard` és a blokkoló várakozás tilos. A heap-allokáció (`push_back`, `std::make_exception_ptr`, `std::string` építése) is kerülendő, mert az allokátor belül szinkronizálhat. A hibakódok tárolóját ezért a hívás **előtt**, a teljes bemenet méretére kell lefoglalni, és mindegyik elem csak a saját helyére írhat. Ha ez nem elég, például mert a hibához szöveges üzenetet vagy kivételt kell megőrizni, akkor `par`-ra vagy policy nélküli algoritmusra kell váltani.

```
// unseq_errors.cpp - C++20: hibagyűjtés par_unseq alatt, szinkronizáció nélkül
#include <algorithm>
#include <cstdint>
#include <cstdio>
#include <execution>
#include <functional>
#include <numeric>
#include <vector>

enum class Err : std::uint8_t { ok = 0, negative, too_large };

// Nem dob, nem allokál, nem szinkronizál: par_unseq alatt is megengedett.
Err validate(int x) noexcept {
    if (x < 0) return Err::negative;
    if (x > 1000) return Err::too_large;
    return Err::ok;
}

int main() {
    const std::vector<int> in{5, -1, 7, 2000, 3, -4};
    std::vector<Err> codes(in.size());   // kapacitás a hívás ELŐTT lefoglalva

    // Minden elem a SAJÁT kimeneti helyére ír: nincs megosztott állapot.
    std::transform(std::execution::par_unseq, in.begin(), in.end(),
                   codes.begin(), validate);

    // Az összesítés a párhuzamos szakasz után, a hívó szálon.
    const auto failed = std::count_if(codes.begin(), codes.end(),
```

```
                                        [](Err e) { return e != Err::ok; });
    std::printf("failed=%td first_bad_index=%td\n", failed,
                std::find_if(codes.begin(), codes.end(),
                             [](Err e) { return e != Err::ok; }) -
                             ↪  codes.begin());
}
```

Buildjegyzőkönyv (`lab-gcc` profil, `g++ -std=c++20 -Wall -Wextra -Wpedantic -O2`). A kimenet `failed=3 first_bad_index=1`, ASan/UBSan builddel is. (A `lab-gcc` profilban TBB nélkül a soros backend fut, így ez a futás a szerződés betartását, nem a párhuzamos végrehajtást igazolja.)

29.5.4 Backend, linkelés és támogatás

A szabvány előírja, hogy ezek a policy-k elérhetők, de a mögöttes végrehajtómotor implementációfüggő. A GCC/libstdc++ esetén a `<execution>` és a párhuzamos algoritmusok a Threading Building Blocks (TBB) könyvtárra épülnek; TBB nélkül a program vagy nem linkel, vagy a futás során csendben `seq`-re esik vissza – ez implementáció- és verziófüggő. A Clang/libc++ hasonlóan külső backendre támaszkodhat, az MSVC belső szálmedencét használ. A `lab-gcc` profilban (GCC 13.3, telepített TBB nélkül) ezt mértük: a `<execution>` lefordul, a libstdc++ a `_PSTL_PAR_BACKEND_SERIAL` soros backendet választja, mert a `<tbb/tbb.h>` nem érhető el. Egy `par` policyval futó `for_each` ilyenkor egyetlen szálon fut, és `-ltbb` nélkül is linkel; a `-ltbb` kapcsoló viszont `cannot find -ltbb` hibát ad. TBB-vel telepített környezetben a linkeléshez a fordításhoz hozzá kell adni a megfelelő könyvtárat:

```
# GCC + libstdc++, TBB-vel
g++ -std=c++20 -O2 par_sum.cpp -ltbb -o par_sum

# C++17 alternatíva ugyanahhoz a forráshoz
g++ -std=c++17 -O2 par_sum.cpp -ltbb -o par_sum
```

A támogatás ellenőrzésére a `__cpp_lib_execution` feature-test makró használható; ennek értéke jelzi, hogy a választott library melyik C++-verzió policy-készletét támogatja. A `__has_include(<execution>)` csak header-létezést jelez, nem azt, hogy a párhuzamos futtatás tényleg működik. A laborprofilban rögzíteni kell a compiler, a libstdc++/libc++ és a TBB pontos verzióját; e nélkül a „párhuzamosan fut” állítás nem reprodukálható.

29.5.5 Oversubscription és grain size

A `par` alatt a könyvtár jellemzően egy belső szálmedencét használ, amelynek mérete nagyságrendileg a `std::thread::hardware_concurrency()` értékéhez igazodik, de ezt a szabvány nem írja elő. Ha a `par` algoritmust egy már szálmedencében futó taskból hívjuk (például egy `std::async`-ból vagy egy szerver kérésfeldolgozó szálából), vagy ha a folyamat CPU-korlátos konténerben fut, akkor a medence és a hívó szálai egymással versengenek: ez az *oversubscription*, ami a teljesítményt a szekvenciális változat alá is viheti.

A másik véglet a *grain size* kérdése. Ha egy elemre jutó munka túl kicsi, a hívások szétdobálásának, a részeredmények egyesítésének és a szálak felébresztésének költsége nagyobb, mint a számítás. A policy mögötti könyvtár általában néhány mikroszekundum nagyságrendű per-task munkát tekint gazdaságosnak, de ez nem szabványos garancia, hanem empirikus megfigyelés. A helyes hozzáállás: először mérni, utána dönteni; és ha a mérés nem mutat javulást, a `seq` policy a helyes választás. A mérési protokollt a Profilozás, benchmark és cost model (37.4) alatt tárgyalt

benchmark-módszertan szerint kell felépíteni, mert a puszta `time` parancs oversubscription és gyorsítótár-melegítés mellett félrevezető lehet.

29.5.6 Sorrendfüggő állapot és pointer lifetime

A `par` nem garantálja a feldolgozási sorrendet a szálak között; a `par_unseq` és az `unseq` még egy szálon belül sem. Ezért tilos a callback eredményét sorrendtől függően egy megosztott objektumba fűzni (`push_back` egy `std::vector`-ba `par` alatt adatverseny, `par_unseq` alatt UB). A pointer- és referencia-lifetime szempontból a `par` és `par_unseq` ugyanazt a fegyelmet kéri, mint bármelyik aszinkron hívás: a lambdában elfogott pointernek vagy referenciának az *algoritmus teljes futása alatt* érvényesnek kell maradnia, nem csak a hívás kiadásáig. Egy gyakori csapda, hogy a felhasználó egy lokális `std::vector`-ra mutató pointert fog el, miközben a vektort egy másik szálon felszabadítja; ez a Konkurens élettartam és nehéz hibák (15.5) alatt tárgyalt konkurens élettartam-hibák egy példánya, és az algoritmus semmilyen diagnosztikát nem ad rá. C++20-tól a `std::span` használható arra, hogy a callback ne pointer–méret párost kapjon, hanem hosszal rendelkező, nem tulajdonló nézetet; ez nem oldja meg az élettartamot, de legalább a félreértett méretezést kizárja.

29.5.7 Kidolgozott példa

A következő példa egyesített összeget számol `par` policyval. A redukció asszociatív és kommutatív műveletet (`+`) használ, az `std::int64_t` tartomány elegendő az 1..1 000 000 összegéhez (kb. $5 \cdot 10^{11}$), így túlcsordulás nem fenyeget. A `printf` a hívás után fut, nem a callbackben.

```
// par_sum.cpp
#include <cstdint>
#include <cstdio>
#include <execution>
#include <numeric>
#include <vector>

int main() {
    std::vector<std::int64_t> v(1'000'000);
    for (std::int64_t i = 0; i < static_cast<std::int64_t>(v.size()); ++i) {
        v[static_cast<std::size_t>(i)] = i + 1;
    }

    const std::int64_t s = std::reduce(
        std::execution::par, v.begin(), v.end(), std::int64_t{0});
    std::printf("sum = %lld\n", static_cast<long long>(s));
}
```

A `reduce` C++17-es, iteratoralapú overloadja fogad policy-t; a `seq` policy a végrehajtási ügynökök viszonyát korlátozza (egy szálon, szekvenciálisan), de **nem állítja vissza** az `accumulate` balról jobbra történő összegzését: a `reduce` a `seq` mellett is megváltoztathatja a csoportosítást. Ezért a `reduce(seq, ...)` és az `accumulate` egyenértékűsége csak erre az adott, túlcsordulásmentes *egészösszeg*-példára mondható ki (az egész összeadás asszociatív és kommutatív); lebegőpontos vagy nem kommutatív műveletnél a `seq` nem ad az `accumulate`-tal azonos eredményt. TBB nélküli környezetben a program lefuthat, de a párhuzamosság nem garantált; a viselkedés verziófüggő. Egy header-szintű fallback:

```
#if __cplusplus >= 201703L && defined(__cpp_lib_execution)
#  include <execution>
#  include <numeric>
#  define BOOK_SUM(first, last) std::reduce(std::execution::par, (first),
 ↪  (last), std::int64_t{0})
#elif __cplusplus >= 201703L
#  include <numeric>
#  define BOOK_SUM(first, last) std::reduce((first), (last), std::int64_t{0})
#else   // C++14 és régebbi: a std::reduce nem létezik
#  include <numeric>
#  define BOOK_SUM(first, last) std::accumulate((first), (last),
 ↪  std::int64_t{0})
#endif
```

A `std::reduce` maga is C++17-es, ezért a policy-argumentum elhagyása nem teszi C++17 előtt fordíthatóvá; a régi ág a `std::accumulate`-et használja. Fontos a **sorrendi szerződés eltérése**: az `accumulate` balról jobbra, a `reduce` tetszőleges csoportosításban összegez, ezért egészösszegnél azonos az eredmény, lebegőpontosnál nem feltétlenül. (A minimális támogatott nyelvi verzió egyébként C++17; `-std=c++14` alatt a régi ág tényleges fordítása és az üzenet a `BOOK_SUM` makróval ellenőrizhető.)

29.5.8 Tipikus hibák és javításuk

A leggyakoribb tévedés, hogy a `par` policy-t automatikus gyorsításként kezeljük: kis vektorokon, dominánsan memóriafüggő műveleteknél (pl. nagy struktúrák másolása `par transform`-mal) vagy oversubscribed környezetben a szekvenciális változat gyorsabb lehet. A második gyakori hiba a callbacken belüli mutex vagy `std::atomic` használata `par_unseq` alatt – ez UB, nem „lassabb”. A harmadik: referencia szerint elfogott számláló növelése `par` alatt, ami adatverseny. A negyedik: kivétel dobása a callbackből `par` alatt, ami `std::terminate`-hez vezet, és nem ahhoz, hogy a hívó elkaphatja. Az ötödik: linkelés `-ltbb` nélkül, amit a fordító csak linkeléskor, nem fordításkor jelez.

29.5.9 Feladatok

1. feladat (diagnosztikai). Az alábbi kódrészlet `par` policyval fut. Add meg, mi a hiba, pontos nyelvi terminológiával, és mit tenne a program, ha a hiba „csak” logikai lenne.

```
std::vector<int> v(1'000'000, 1);
int acc = 0;
std::for_each(std::execution::par, v.begin(), v.end(),
              [&acc](int x) { acc += x; });
```

2. feladat (kódolási). Írj `par` policyval működő, kivételbiztos szimmetrikus összeget egy `std::vector<std::int64_t>`-re, ahol az elemek értéke $0 \le x \le 10^4$ (nemnegatív, felső és alsó korláttal), a vektor hossza 10^6. A callback ne dobjon kivételt, és ne használjon mutexet vagy atomot. Indokold az összeg típusválasztását és a policy-választást.

3. feladat (tervezési). Ugyanazt a feladatot oldd meg `par_unseq` policyval. Adj egy konkrét kódrészletet, amely `par` alatt legális, `par_unseq` alatt viszont UB, és magyarázd el, miért.

29.5.10 Megoldások

1. megoldás. A `++acc` nem atomi művelet: a nyelvi memóriamodell `[intro.races]` definíciója szerint két szál ugyanazon objektumot egyszerre, nem synchronized módon olvassa és írja, ami *data race*. Adatverseny esetén a program viselkedése nem definiált: nem „elveszett frissítés”, hanem bármi megtörténhet, a fordító optimalizálhatja a ciklust, és a végeredmény nem reprodukálható. A helyes javítás `std::reduce(std::execution::par, v.begin(), v.end(), 0LL)` vagy lokális akkumulátor használata. Ellenőrzési feltétel: a javított kód minden futáson ugyanazt az összeget adja, és `-fsanitize=thread` alatt nem jelez.

2. megoldás. A feltétel $0 \le x_i \le 10^4$, ezért minden megengedett részösszeg a $[0,\ 10^{10}]$ tartományban van (nemnegatív tagok, monoton nő), az összeg maximuma $10^6 \cdot 10^4 = 10^{10}$, ami belefér `std::int64_t`-be (a bizonyításhoz mindkét korlát kell: az alsó nélkül `INT64_MIN` körüli értékek összege túlcsordulhatna, így az ilyen bemenet kívül esik a szerződésen), `std::int32_t`-be nem feltétlenül (LP64 alatt `long` is 64 bites, ILP32 alatt 32 bites – ezért `int64_t`). A megoldás:

```
std::int64_t s = std::reduce(std::execution::par, v.begin(), v.end(),
                             std::int64_t{0}, std::plus<>{});
```

A `reduce` asszociatív és kommutatív műveletet vár, a `+` az egész tartományon az. A callback `noexcept`: nem dob, nem hív lockot. Indoklás: `par` választása azért helyes, mert az összeg nagy, a művelet lokális és jól particionálható; `seq` választása is helyes lenne, ha a vektor kicsi.

3. megoldás. Ugyanaz a hívás `par_unseq` policyval megengedett, feltéve, hogy a callback vectorization-safe. Ami `par` alatt legális, de `par_unseq` alatt UB, az például:

```
std::mutex m;
std::for_each(std::execution::par, v.begin(), v.end(),
              [&m](std::int64_t x) {
                  std::lock_guard<std::mutex> lk(m);   // par alatt OK
                  global_vec.push_back(x);             // par_unseq alatt UB
              });
```

`par` alatt a lockolás megengedett (bár lassú); `par_unseq` alatt a SIMD-interleave miatt a lockolt szakasz nem vectorizable, és a szabvány szerint a callback nem hívhat vectorization-unsafe műveletet. A helyes megoldás `par_unseq`-hez: nincs globális mutex, nincs `push_back` egy megosztott tárolóba; a callback tisztán lokális eredményt ad vissza, amit `std::transform_reduce` vagy egy saját redukció egyesít. Ellenőrzési feltétel: `-fsanitize=thread` tiszta, és a policy cseréje nem változtatja meg az eredményt.

29.5.11 Összefoglaló

A végrehajtási policy négy értéke közül a `seq` a szekvenciális, a `par` a több szálon, de szálon belül rendezett, a `par_unseq` (C++17) és `unseq` (C++20) a vektorizált végrehajtást engedi meg. A policy megválasztása szemantikai vállalás: `par` alatt a callbacket meg kell védeni az adatverseny ellen, `par_unseq`/`unseq` alatt semmilyen blokkoló vagy szinkronizáló primitív nem használható. Mind a négy szabványos policynál, a `seq`-et is beleértve, a callbackből kiszökő kivétel `std::terminate`-hez vezet (csak a policy nélküli hívásnál propagálódik), ezért a hibát a callbacken belül adatként kell kezelni. A paralelizmust a backend (GCC/libstdc++ esetén TBB, `-ltbb`), az oversubscription és a grain size együtt határozza meg; a policy önmagában nem gyorsítási ígéret. Az algoritmus eredményét soha nem szabad sorrendfüggően, egy megosztott tárolóba írni, és a callbackben elfogott pointernek az algoritmus teljes futása alatt érvényesnek kell maradnia – ezek a szabályok a Konkurens élettartam és nehéz hibák (15.5) konkurens élettartam-szabályainak specializációi a párhuzamos algoritmusokra.

30 Lambdák, callable-ek és ranges

30.1 Lambda, capture és closure lifetime

Ebben az alfejezetben a C callback+context minta típusos megfelelőjét vizsgáljuk. A Függvénypointerek, callbackek és context pointer (6.5) részben a C-ben megszokott `void (*)(void *)` és `struct ctx *` párost láttuk: a hívó ad egy függvénycímet és egy külön átadott context pointert, a kontextus érvényességéért pedig sem a nyelv, sem a fordító nem felel — az a programozó szerződése. A modern C++ closure (lambda kifejezés eredménye) ezt a szerződést típusba, nevesíthető kódba és a fordító által ellenőrzött capture-listába emeli. Ez erősebb eszköz, de nem varázslat: a closure egy közönséges objektum, élettartam-, másolási és mozgatási szabályai ugyanazok, mint bármely más osztálypéldányé. A lambda capture itt tanultakat később közvetlenül felhasználjuk a `std::function`, a `std::invoke` és a ranges pipeline (invoke, function wrapper és type erasure (30.2), Ranges algoritmusok, projection és sentinel (30.3)) tárgyalásakor; itt a capture-policy és a closure-élettartam jelenti a fókuszt.

30.1.1 Lambda mint closure: definíció és feltételek

Egy **lambda-kifejezés** (`lambda expression`) olyan elsődleges kifejezés, amelynek kiértékelése egy **closure objektumot** hoz létre. A closure objektum típusa egyedi, névtelen **closure típus** (`closure type`), amelyet a program nem tud nevén megnevezni, de `auto`, `decltype` és template paraméter formájában hozzáférhető. A closure típus az alábbi, a szabvány által előírt elemekkel rendelkezik: publikus `operator()` (hívási operátor), másoló és mozgató konstruktor a rögzített tagoktól függően, továbbá — ha a lambda nem rögzít semmit — egy konverziós operátor függvénypointerre.

A `[]` záradék a **capture-lista** (`capture list`). Pontosan azt mondja meg, hogy a lambda törzse mely, a bezáró hatókörben automatikus tárolási idejű változókat használhat, és milyen formában: **érték szerint** (`by value`) vagy **referenciaként** (`by reference`). A capture-lista három alakja fordul elő:

- `[]` — semmit nem rögzít;
- `[=]` vagy `[&]` — alapértelmezett capture mód, a ténylegesen használt nevekre alkalmazva;
- `[x, &y, ...]` — explicit lista, kevert móddal.

A fordító csak a tényleg *odr-use* formában használt neveket rögzíti, és a capture-lista érvényességi feltétele, hogy a rögzített név a lambda előtti hatókörben látható automatikus változó legyen. A `[=]` és `[&]` alapértelmezett mód **nem rögzít `this`-t érték szerint** tagfüggvényben: a `this` pointert minden esetben pointerként (tehát implicit referencia-jellegűen) fogadja el, ami a későbbi csapdák egyik fő forrása. A C++20 óta a `[=]` implicit `this`-rögzítése deprecált; a közvetlen, explicit `[this]` vagy `[*this]` forma javasolt.

Fontos, hogy a lambda **típusa** minden forráshelyen egyedi, ezért két, szó szerint azonos lambda kifejezés nem azonos típus. Ez indokolja, hogy closure-t csak `auto`-val, template paraméterrel vagy típus-erasure wrapperrel (`std::function`) tároljunk, soha ne írjuk ki a típusát.

30.1.2 Capture-policy élettartam szerint

A capture-policy helyes megválasztásának egyetlen rendező elve van: **a closure objektum élettartama és a rögzített entitás élettartama közötti viszony**. Érték szerinti capture esetén a closure a változó pillanatnyi értékének saját másolatát birtokolja — ez önálló élettartamot ad. Referencia szerinti capture esetén a closure csak hivatkozik az eredeti objektumra; ha a closure tovább él, mint a hivatkozott objektum, a hozzáférés nem definiált viselkedés (Temporary, value category és dangling (20.4)).

Capture	Mit tárol a closure	Kié az élettartam	Mikor biztonságos
`[x]`	`x` másolata (a closure tagja)	a closure-é	a closure bármilyen élettartamára
`[&x]`	hivatkozás `x`-re	a külső `x`-é	a closure nem él túl `x`-et
`[this]`	`this` pointer értéke	a `*this` objektumé	`*this` túlél minden hívást
`[*this]`	`*this` másolata (C++17)	a closure-é	a típus másolható
`[p = std::move(q)]`	a mozgatott érték (init-capture)	a closure-é	move-only erőforrás átadásakor

A táblázat jól mutatja, hogy a „referencia gyorsabb” intuíció csak akkor érvényes, ha a closure élettartama garantáltan az eredeti objektumon belül marad — tipikusan szinkron algoritmusokban vagy olyan callbacknél, ahol a hívó a hívás idejére garantálja az objektum élettartamát.

Nézzük meg a tipikus async élettartam-hibát. Az alábbi példa egy egyszerű, késleltetett hívásokat gyűjtő `Timer`-t használ, és a `Session` élettartama rövidebb, mint a queue-ban maradó closure-é.

```
// c30_async_lifetime.cpp
// Build: g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o c30_async_lifetime
 ↪  c30_async_lifetime.cpp
#include <functional>
#include <iostream>
#include <string>
#include <utility>
#include <vector>

class Timer {
public:
    void schedule(std::function<void()> f) { q_.push_back(std::move(f)); }
    void fire_all() {
        for (auto& f : q_) f();
        q_.clear();
    }
private:
    std::vector<std::function<void()>> q_;
};

struct Session {
    std::string token = "abc";
```

```
    // HIBÁS: [&] tagfüggvényben a this pointert rögzíti, nem a token másolatát.
    void activate_bad(Timer& t) {
        t.schedule([&] { std::cout << "bad:  " << token << '\n'; });
    }

    // HELYES: init-capture a tag értékét másolja a closure-be.
    void activate_good(Timer& t) {
        t.schedule([tok = token] { std::cout << "good: " << tok << '\n'; });
    }
};

int main() {
    Timer t;
    {
        Session s;
        s.activate_bad(t);
        s.activate_good(t);
    } // s itt megszűnik
    t.fire_all(); // a bad closure dangling this-t használ: UB
}
```

A `activate_bad` closure-je egy `this` pointer értékét tartalmazza, és a `fire_all()` pillanatában a `Session` már nem létezik — a viselkedés nem definiált. A `activate_good` init-capture-rel a `token` stringet a closure saját tagjaként tárolja, így a `fire_all()` biztonságos. A példa buildelése a fenti paranccsal történik, a laborprofilban rögzített GCC vagy Clang kiadással; a programot a tanuló saját laborjában futtatja, a könyv nem állítja, hogy a példa futási eredménye itt ellenőrzött.

30.1.3 Init-capture és move-only erőforrás

Az **init-capture** (C++14) lehetővé teszi, hogy a capture-lista ne csak másoljon vagy hivatkozzon, hanem **új closure-tagot definiáljon** egy kifejezésből. Alakja `[név = kifejezés]`, illetve `[&név = kifejezés]`. Ez az egyetlen eszköz arra, hogy move-only erőforrást (pl. `std::unique_ptr`) lambda-ba zárjunk: érték szerinti capture másolna (nem megy), referencia szerint pedig nem adnánk át tulajdonjogot.

```
// c30_init_capture.cpp
// Build: g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o c30_init_capture
 ↪  c30_init_capture.cpp
#include <memory>
#include <utility>

auto make_owner() {
    auto p = std::make_unique<int>(42);
    return [q = std::move(p)]() { return ++*q; };
}
```

A visszaadott closure a `q` `unique_ptr`-t birtokolja, a `p` a mozgatás után üres. A `mutable` itt **nem** kell: az `operator()` alapértelmezésben `const`, de a `++*q` nem a `q` tagot módosítja, hanem a mutatott `int`-et — a `const std::unique_ptr<int> operator*`-ja is nem const `int&`-et ad. A `mutable` akkor kell, ha magát a rögzített tagot módosítjuk, például `q.reset()`-tel:

```cpp
// lambda_without_mutable.cpp - C++20: a mutatott objektum módosítása nem kíván
 ↪  mutable-t
#include <cassert>
#include <memory>
#include <utility>

auto make_owner() {
    auto p = std::make_unique<int>(42);
    return [q = std::move(p)]() { return ++*q; };   // a q tag nem változik,
     ↪  csak *q
}

int main() {
    const auto f = make_owner();   // const closure is hívható
    assert(f() == 43);
#ifdef NEEDS_MUTABLE
    auto g = [q = std::make_unique<int>(1)]() { q.reset(); };   // hiba: q const
#endif
    auto h = [q = std::make_unique<int>(1)]() mutable { q.reset(); return q ==
     ↪  nullptr; };
    assert(h());
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic lambda_without_mutable.cpp` figyelmeztetés nélkül fordul, a `const` closure hívása 43-at ad. `-DNEEDS_MUTABLE` mellett a `mutable` nélküli `q.reset()` hibát ad: `passing 'const std::unique_ptr<int>' as 'this' argument discards qualifiers`. Fontos következmény: az ilyen closure **csak mozgatható**, nem másolható. Ezért nem tehető be `std::function`-be, mert annak követelménye, hogy a becsomagolt callable `CopyConstructible` legyen. Ilyenkor template paraméterként adjuk át a closure-t, vagy `std::move_only_function`-t használunk (C++23 — ebben a részben nem tárgyaljuk részletesen, a laborprofilban ellenőrizendő).

30.1.4 `this` versus `*this` capture

A `[this]` és a `[*this]` közötti különbség aszimmetrikus, és pontosan ez okozza a legtöbb async élettartam-hibát tagfüggvényben.

A `[this]` **pointert** rögzít: a closure mérete tipikusan egy pointer, és minden hívás a `this`-en keresztül ér el a tagokhoz. A closure élettartama ettől független a `*this` objektum élettartamától, de **a hívás pillanata mindig a `*this` élettartamán belül kell legyen**. Ha a closure `<this>` után hívódik meg (például egy queue-ban, egy `std::thread`-en, egy késleltetett timerben), a hozzáférés nem definiált.

A `[*this]` (C++17) **másolatot** rögzít a `*this` objektumról: a closure a saját tagjaként birtokol egy `Session` másolatot, így a hívás akkor is biztonságos, ha az eredeti objektum megszűnt. Feltétele, hogy a `*this` típusa másolható legyen; nem másolható osztály (pl. `std::mutex` tag, `unique_ptr` tag) esetén `[*this]` nem használható. Ilyenkor a helyes megoldás a szükséges állapot **kézi, szelektív** másolása (`[tok = token, fd = fd]`) vagy `shared_ptr`-alapú megosztott birtoklás.

A C++20-ban deprecált `[=]` tagfüggvényben implicit módon `this`-t rögzít, nem `*this`-t — ez a leggyakoribb, olvasás során nehezen észrevehető hibaforrás. Az explicit `[this]` legalább vizuálisan jelzi a szerződést; a `[*this]` pedig kifejezetten a „túlélhetem a `*this`-t” szándékot.

30.1.5 Mutable lambda és const call operator

A closure típus `operator()`-je alapértelmezésben `const`. Ez azt jelenti, hogy az érték szerint rögzített tagok a hívásban **nem módosíthatók**, referenciaként rögzített entitások viszont igen (mert a hivatkozáson keresztül a külső objektum nem része a closure `const` állapotának). A `mutable` kulcsszó a capture-lista után eltávolítja a `const`-ot:

```
int outer = 0;
auto by_ref  = [&outer] { ++outer; };        // OK: külső objektumot módosít
auto by_val  = [outer]   { /* ++outer; */ };  // fordítási hiba: const
 ↪  operator()
auto counter = [n = 0]() mutable { return ++n; };
```

A `counter` closure minden példánya saját számlálót tart; a `mutable` csak a closure belső állapotára hat, a külső `outer`-t nem érinti. Ez fontos tervezési pont: egy `mutable` lambda megosztott példánya nem threadsafe, mert a closure belső tagján nincs szinkronizáció. Ha a closure actor-szerű számlálást végez, a hozzáférést külső mutexnek vagy atomiknak kell védenie (RAII lockok, condition variable és wait predicate (34.2), C++ atomics, atomic_ref és publication (34.3)).

30.1.6 Generic lambda, template paraméter és constraints

A **generic lambda** (`auto` paraméterrel, C++14) valójában egy template hívási operátorral rendelkező closure, ahol minden `auto` paraméter egy-egy template paraméter. C++20-tól a lambda saját template paraméterlistát is kaphat, és `requires`-szel vagy constrained template paraméterrel szűkíthető:

```
auto twice      = [](auto x) { return x + x; };
auto size_of    = []<typename T>(const std::vector<T>& v) { return v.size(); };
auto positive   = []<std::integral T>(T x) { return x > 0; };
```

A `size_of` esetben a lambdanak saját template paraméterre van szüksége, mert nem lehet `auto`-val kifejezni, hogy a paraméter `std::vector<T>`. A `positive` a C++20 `std::integral` concepttel szűkít; a feltétel a hívási operátoron érvényesül, így nem integrál típusra a hívás nem kompillál, és az overload-feloldásban a constraint részt vesz. A constraints részleteit a Concepts, requires és constrained overload (27.2) tárgyalja; itt a lambda-specifikus alak a lényeg: `[]<...>(...) requires ...` illetve `[]<Concept T>(...)`.

30.1.7 Stateless lambda és függvénypointer-konverzió

Egy **nem capturing** (stateless) lambda rendelkezik egy implicit konverziós operátorral (`conversion to function pointer`), amely a closure `operator()`-jének megfelelő szignatúrájú függvényre mutat. Ez a C callback ABI-val kompatibilis: egy stateless lambda átadható olyan C API-nak, amely `int (*)(int)` típusú paramétert vár. Capturing lambda **nem** konvertálható függvénypointerré, mert a closure állapotának nincs helye a függvénypointerben — ilyenkor `std::function`, template paraméter vagy explicit `void *` context szükséges.

A C callback+context minta és a closure közötti megfelelés:

Szerep	C forma	C++ closure forma
viselkedés	`void (*fn)(void *)`	`operator()` a closure típuson
állapot	külön `struct ctx *`	a closure rögzített tagjai
állapot élettartama	a hívó felelőssége	a closure objektumé

Szerep	C forma	C++ closure forma
stateless eset	`fn` + `NULL`	stateless lambda, `fn` pointerré konvertál
típusbiztonság	nincs (void*)	fordítási idejű

Ez a megfelelés az alapja annak, hogy a C-ben megszokott callback-regisztráció C++ oldalon típusosan, capture-rel fejezhető ki. A callback regisztrációs szerződés (ki birtokolja a context-et, meddig hívható) viszont változatlanul él: a closure ugyanúgy megköveteli, hogy a rögzített objektumok a hívás pillanatában érvényesek legyenek.

30.1.8 Gyakori hibák és javításuk

- **Visszaadott closure `[&]` capture-rel lokálisra.** A closure túlél, a hivatkozott objektum nem. Javítás: érték szerinti capture vagy init-capture.
- **Tagfüggvényben `[=]` vagy `[&]` async callbackhez.** Mindkettő a `this` pointert rögzíti. Javítás: `[*this]` (ha másolható) vagy explicit, csak a szükséges állapot másolása.
- **`std::function` move-only closure-lel.** Fordítási hiba (copy követelmény). Javítás: template paraméter vagy C++23 move-only wrapper.
- **`mutable` elhagyása érték szerint módosító closure esetén.** Fordítási hiba. Ugyanakkor a `mutable` önmagában nem tesz threadsafe-é.
- **Capturing lambda függvénypointerként C API-nak.** Fordítási hiba; a stateless-konverzió csak `[]`-re vonatkozik.

30.1.9 Összefoglaló

A lambda egy névtelen closure típusú objektum, amelynek élettartam- és másolási szabályai megegyeznek a közönséges osztályokéval. A capture-policy kiválasztását a closure és a rögzített entitás élettartam-viszonya vezérli: érték szerint biztonságos a hosszú életű closure-höz, referencia szerint csak akkor, ha a closure élettartama az objektuméba ágyazódik. A `[this]` pointert, a `[*this]` (C++17) másolatot rögzít; tagfüggvényben a `[=]` implicit `this`-t rögzít, ami C++20 óta deprecált. Init-capture-rel move-only erőforrás tehető a closure-be, de az ilyen closure nem másolható, így `std::function`-be nem illeszkedik. Az `operator()` alapból `const`, a `mutable` oldja fel; generic lambda `auto` paraméterrel vagy explicit template paraméterlistával (C++20) készíthető, `requires`-szel szűkíthető. Végül: stateless lambda függvénypointerré konvertál, capturing lambda nem — ez a C callback és a C++ closure közötti gyakorlati határvonal.

30.1.10 Feladatok

1. feladat (diagnózis). Az alábbi függvény egy closure-t ad vissza, amely `msg`-re hivatkozik. Mi a hiba, és milyen minimális javítás adja meg a helyes, hosszú életű closure-t?

```
#include <functional>
#include <string>
std::function<void()> make_printer() {
    std::string msg = "hello";
    return [&msg]() { /* használja msg-et */ };
}
```

2. feladat (init-capture és `std::function`). Adott egy `std::unique_ptr<int>`-ot birtokló closure. Magyarázd el, miért nem fordul le `std::function<void()>`-ba helyezve, és adj egy működő alternatívát.

3. feladat (`this` vs `*this`). Az alábbi `Server` egy queue-ba tesz egy closure-t, amely `name`-re hivatkozik. Írd át úgy, hogy a closure biztonságos legyen akkor is, ha a `Server` a hívás előtt megszűnik. Mi a kompromisszum?

```
struct Server {
    std::string name;
    void schedule(std::function<void()> f);
    void start() { schedule([this] { use(name); }); }
    void use(const std::string&) const;
};
```

4. feladat (típusosság). Írj egy lambda-t, amely csak integrál típusra hívható, és összeadja a paraméterét 1-gyel. Miért fordítási hiba a `std::string` argumentum?

30.1.11 Megoldások

1. megoldás. A visszaadott closure a `msg`-re hivatkozik, de `msg` a `make_printer` visszatérésével megszűnik; a closure hívása nem definiált viselkedés. A javítás érték szerinti capture, amely másolatot tesz a closure-be:

```
return [msg]() { /* használja msg-et */ };
```

Az init-capture alak `[m = msg]` szintén működik, és hasznos, ha a nevünket szeretnénk átnevezni. *Ellenőrzés:* a javított closure a `make_printer` visszatérése után is biztonságosan hívható, mert a `msg` másolata a closure tagja.

2. megoldás. A `std::function<void()>` megköveteli, hogy a becsomagolt callable `CopyConstructible` legyen, mert a `std::function` maga másolható. Az init-capture-rel rögzített `std::unique_ptr` miatt a closure csak mozgatható, ezért a `std::function` konstruktor nem illeszkedik. Alternatíva: a closure-t template paraméterként adjuk át (ami a hívást statikusan oldja fel), vagy — ha a laborprofil támogatja — C++23 `std::move_only_function`-t használunk. *Ellenőrzés:* a template-paraméteres változat lefordul, a `std::function`-be helyezett változat fordítási hibát ad, és a hibaüzenet a másolási követelményre hivatkozik.

3. megoldás. A `[this]` a `Server` pointerét rögzíti; ha a `Server` megszűnik a hívás előtt, a closure dangling pointerrel dolgozik. A `[*this]` (C++17) másolatot készít a `Server`-ről, így a `name` is a closure-be kerül, és a hívás biztonságos. Kompromisszumok: (a) a `Server` típusának másolhatónak kell lennie, (b) a closure a `Server` állapotának **pillanatképét** tartalmazza, nem az élő objektumot — ha a `name` a regisztráció után változik, a closure a régit látja. Ha ez nem elfogadható, a helyes megoldás megosztott birtoklás (`std::shared_ptr<Server>` és `[self = shared_from_this()]`) vagy a szükséges mezők explicit másolása (`[n = name]`). *Ellenőrzés:* a módosított closure a `Server` destruktora után is hívható anélkül, hogy UB-t okozna; ha a `Server` nem másolható, a `[*this]` fordítási hibát ad, ami jelzi a szerződés sérülését.

4. megoldás. A C++20 constrained generic lambda:

```
auto inc = []<std::integral T>(T x) { return x + 1; };
```

A `std::string` argumentum nem elégíti ki a `std::integral` conceptet, ezért a hívás overload-feloldása nem talál érvényes jelöltet — fordítási hiba. A constrainttel a hiba a hívás helyén, érthető üzenettel jelenik meg, nem a lambda törzsében. *Ellenőrzés:* `inc(41)` eredménye `42`, `inc(std::string{"x"})` fordítási hiba a concept megsértése miatt.

30.2 invoke, function wrapper és type erasure

Ez az alfejezet a callbackek tárolásának azt a kérdését járja körül, amely a gyakorlatban a legtöbb hibát okozza: *ki birtokolja a hívható objektumot, meddig él, és mennyibe kerül a hívás?* A Lambda, capture és closure lifetime (30.1) a lambda kifejezés és a closure élettartamát rögzítette, itt azt vizsgáljuk, hogyan lehet a különböző fajtájú hívható objektumokat egységes interfész mögé tenni. A cél egy döntési térkép: mikor elég egy sablonparaméter, mikor kell típusőrlés (type erasure), és mikor kell a nem tulajdonló tárolást választani. A shared_ptr, control block és weak_ptr (21.4) `shared_ptr`-mechanikájára és a Virtual, override, final és dispatch (24.2) virtuális dispatch-magyarázatára építünk; az előbbit a megosztott tulajdonlásnál, az utóbbit a típusőrlés implementációs mintájánál használjuk fel.

30.2.1 Egységes hívás: `std::invoke`

A C++-ban a „hívható” (callable) fogalma szélesebb, mint a `f(x)` alak: lehet függvénypointer, függvényobjektum, lambda, tagfüggvényre mutató pointer (`int (T::*)()`) vagy tagnévre mutató pointer (`int T::*`). A sablonkód nem tudhatja, melyikkel kapja a paramétert, ezért a szabvány egyetlen kifejezéssé alakítja őket: ez az **INVOKE kifejezés**, amelynek könyvtári formája az `std::invoke`.

Definíció. `std::invoke(f, args...)` pontosan akkor fordul le, ha a szabvány INVOKE-szabályai szerint `f` hívható `args...`-tal; az eredmény típusa `std::invoke_result_t<F, Args...>`. A tagfüggvény-pointer esetén az első argumentum az objektum (vagy pointer/referencia-wrapper rá), a tagnév esetén az érték a tag referenciája.

A gyakorlati haszon az, hogy a hívó kód egyetlen alakot ír, és a tagfüggvény/tagadat eltérés eltűnik:

```
// invoke_demo.cpp - C++20
#include <algorithm>
#include <functional>
#include <iostream>
#include <string>
#include <vector>

struct Metric {
    std::string name;
    int value;
    int weight() const { return value * 2; }
};

int main() {
    std::vector<Metric> ms{{"cpu", 3}, {"mem", 1}, {"io", 2}};

    auto w = &Metric::weight;    // int (Metric::*)() const
    auto v = &Metric::value;     // int Metric::*
```

```
    std::cout << std::invoke(w, ms[0]) << '\n';  // várható: 6
    std::cout << std::invoke(v, ms[1]) << '\n';  // várható: 1

    std::sort(ms.begin(), ms.end(),
              [v](const Metric& a, const Metric& b) {
                  return std::invoke(v, a) < std::invoke(v, b);
              });
    for (const auto& m : ms) std::cout << m.name << ' ';
    std::cout << '\n';          // várható: mem io cpu
}
```

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 invoke_demo.cpp -o invoke_demo
```

Az itt közölt kimenet *várható* érték, nem mért eredmény: a példát nem futtattam, a tényleges fordítást és a warningmentességet a saját laborprofilodban kell ellenőrizni. Fontos, hogy a lambdán belül a `v` pointert *érték szerint* kaptuk el — ez egy tagra mutató pointer, amely nem függ az aktuális `Metric` objektumtól, így a `sort` alatti mozgatás nem érvényteleníti. Ha ehelyett egy konkrét objektumra mutató pointert vagy referenciát kaptunk volna el, a `sort` utáni használat dangling lenne.

30.2.2 `std::function`: típusőrlés tulajdonlással

A **típusőrlés** (type erasure) lényege, hogy a konkrét típus eltűnik a felület mögül: a hívó csak a szignatúrát látja, a tárolt objektum dinamikus típusát egy belső indirekció kezeli. A `std::function<R(Args...)>` ennek a szabványos, *tulajdonló* változata.

Definíció és feltételek. `std::function<Sig>` olyan hívható objektumot tárol, amelynek hívása `Sig`-gyel kompatibilis, és amely *másolható* (CopyConstructible). A másolás a tárolt objektumot is másolja; a wrapper üres állapotba kerülhet, és üres wrapper hívása `std::bad_function_call` kivételt dob. A tárolt callable élettartama a wrapper élettartamához kötött, tehát a `std::function` **tulajdonos** — ez a lényegi különbség a következő szakasz referencia-wrapperéhez képest.

A másolhatóság nem jelenti azt, hogy a *viselkedés* is független másolatokkal. Ha a closure `shared_ptr`-t kap el érték szerint, a másolat ugyanazt a control blockot osztja meg (shared_ptr, control block és weak_ptr (21.4)), és a két wrapper ugyanazt az állapotot módosítja:

```
// shared_state.cpp - C++20
#include <functional>
#include <iostream>
#include <memory>

int main() {
    auto counter = std::make_shared<int>(0);
    std::function<int()> f = [counter] { return ++*counter; };
    std::function<int()> g = f;      // wrapper másolat, közös állapot
    std::cout << f() << ' ' << g() << '\n';   // várható: 1 2
}
```

Költség. A `std::function` hívása egy indirekció (tipikusan függvénypointeren vagy belső vtable-en keresztül), a másolás pedig a tárolt objektum méretétől függő művelet. Sok implementáció **small-buffer optimalizálást** (SBO) alkalmaz: egy kis méretű callable a wrapperbe inline kerül, és nem allokál. Ez azonban *nem* szabványos garancia — sem a küszöbméret, sem az, hogy egyáltalán létezik. A `sizeof(std::function<int()>)` implementációfüggő; erre építeni hordozhatatlan. A

helyes gyakorlat: a konstrukció allokálhat, ezt a szerződés részeként kell kezelni, és hot pathon mérni kell, nem feltételezni.

30.2.3 `std::reference_wrapper`: nem tulajdonló tárolás

A `std::reference_wrapper<T>` egy másolható, értékszemantikájú *referencia*: `get()`-tel (és implicit konverzióval `T&`-ra) adja vissza az objektumot. A `std::ref` / `std::cref` állítja elő. Ha callable-t csomagolunk bele, a wrapper hívható marad, mert a hívás továbbítódik a hivatkozott objektumra. Így a `std::function` nem másolja a callable-t, hanem *kölcsönzi*.

A szerződés innentől szigorú: a hivatkozott objektumnak túl kell élnie a wrappert. Ezt a fordító nem ellenőrzi, és a hiba tipikusan csak jóval később, más kontextusban jelentkezik:

```
// ref_lifetime.cpp - C++20 (hibás és helyes változat)
#include <functional>

struct Counter {
    int n = 0;
    int operator()() { return ++n; }
};

std::function<int()> make_bad() {
    Counter c;
    return std::ref(c);         // HIBA: c a függvény végén megszűnik
}                               // -> a visszaadott wrapper dangling

std::function<int()> make_good() {
    Counter c;
    return c;                   // másolat kerül a wrapperbe: biztonságos
}
```

A `make_bad()` eredményének bármilyen hívása **nem definiált viselkedés**; nincs garantált kimenet, és a hiba nem feltétlenül azonnal jelentkezik. A `make_good()` ezzel szemben önálló élettartamú állapotot hordoz. A döntés tehát nem stílus kérdése: a `std::ref` használata akkor helyes, ha a hivatkozott objektum (például egy hosszú életű konfigurációs objektum vagy a `main`-ben élő állapot) egyértelműen túlélő, és ezt a kód dokumentálja.

30.2.4 `bind` / `bind_front` vagy olvasható lambda?

A `std::bind` a C++11-es eszköz részleges alkalmazásra és argumentum-átrendezésre, helyőrzőkkel (`std::placeholders::_1`). A C++20 `std::bind_front` ennek a leggyakoribb esetét egyszerűsíti: az első néhány argumentumot köti, a többit változatlanul továbbítja, perfect forwardinggal.

```
// bind_vs_lambda.cpp - C++20
#include <functional>

int add(int a, int b) { return a + b; }

void demo() {
    auto b1 = std::bind_front(add, 10);        // (int) -> int
    auto l1 = [](int b) { return add(10, b); };
```

```
    // b1(5) == l1(5) == 15
}
```

Az olvashatósági szabály egyszerű: **`bind` helyett lambdát** írj, ha a kötés nem triviális. A `bind` két valós buktatót hordoz. Egyrészt a `std::ref` nélküli kötés *másolatot* tárol, ami nagy objektumnál drága, és meglepetést okoz, ha az eredeti módosul. Másrészt a beágyazott `bind` és a helyőrzők száma könnyen olvashatatlan kódot ad, ráadásul a `bind` eredményének hívási szignatúrája nehezen követhető. A lambda ugyanezt explicit capture-listával, explicit paramétertípusokkal és a hívási helyen látható testtel adja — a closure típusa ugyanúgy tárolható `std::function`-ban. A `bind_front` C++20-as, tehát C++20 alapprofilban használható; ha korábbi szabványra kell buildelni, lambdával helyettesítsd.

30.2.5 Move-only callable-ek: `std::move_only_function` (C++23)

A `std::function` másolhatósági követelménye kizárja a `std::unique_ptr`-t elkapó lambdákat. C++23-ban erre a `std::move_only_function<Sig>` szolgál: csak mozgatható callable-t fogad, a wrapper maga mozgatható, másolása tilos. Elérhetőségét **feature-test makróval** kell ellenőrizni, és a támogatás libstdc++/libc++ verziófüggő:

```
// move_only.cpp - C++23, feature-testtel
#include <functional>
#include <memory>

#if defined(__cpp_lib_move_only_function) && \
    __cpp_lib_move_only_function >= 202110L
using Task = std::move_only_function<void()>;
#define HAVE_MOVE_ONLY 1
#else
using Task = std::function<void()>;   // fallback: megosztott tulajdonlás
#define HAVE_MOVE_ONLY 0
#endif

Task make_task() {
#if HAVE_MOVE_ONLY
    return [p = std::make_unique<int>(5)] { /* ... */ };
#else
    auto sp = std::make_shared<int>(5);
    return [sp] { /* ... */ };        // másolható helyettesítés
#endif
}
```

```
g++ -std=c++23 -Wall -Wextra -Wpedantic -O2 move_only.cpp -o move_only
```

A fallback nem ekvivalens: `shared_ptr` esetén a tulajdonlás megosztott, a control block allokál, és a lambda másolható lesz — ez könnyen elrejti a tervezési szándékot. Ha a cél az egyedi tulajdonlás kifejezése, a C++20-as helyettesítés egy saját, kis típusőrlő wrapper (lásd alább), nem a `shared_ptr`. Azt, hogy a `__cpp_lib_move_only_function` a te toolchaineden definiált-e, compile probe-bal ellenőrizd; ezt a könyv állításaként nem tekintheted igazoltnak.

30.2.6 A típusőrlés szerkezete és a vázlat

Levezetés vázlata. Tegyük fel, hogy egy heterogén callable-halmazt (különböző closure-típusokat) futásidőben, egyetlen változóban kell tárolni. Ha a tároló statikus típusa ismert, minden elem típusa eltér, így egyetlen tömbváltozó nem tudja őket együtt tartani. A közös típus bevezetéséhez a konkrét típust el kell dobni, és a hívást egy olyan indirekción keresztül kell elérni, amelyet a konstrukció idején, a típus még ismert pillanatában rögzítettünk. Ezért a típusőrlő wrappernek legalább három műveletre van szüksége: hívás, másolás (vagy mozgatás), megszüntetés — mindhárom a rögzített típus ismeretében generálható. Ez pontosan a virtuális dispatch szerkezete (Virtual, override, final és dispatch (24.2)), csak *kifelé* fordítva: nem az objektum örökli a felületet, hanem a wrapper tárol egy függvénypointer-táblát. Következmény: hívásonként egy indirekció, és a wrapper mérete fix, függetlenül a tárolt típustól. Az SBO ezt csak optimalizálja, a szerződést nem változtatja.

Tárolási mód	Tulajdonos?	Másolható?	Hívási költség	Tipikus használat
sablonparaméter (`F f`)	nem (a hívó dönt)	a típustól függ	nulla indirekció, inline-olható	generikus algoritmus, hot path
függvénypointer	nem	igen	egy indirekció	C API, stabil határ
`std::reference_wrapper`	nem	igen	indirekció + élettartam-felelősség	hosszú életű objektum kívülről
`std::function`	igen	igen	indirekció, allokálhat	heterogén callback-tároló
`std::move_only_function` (C++23)	igen	nem	indirekció	move-only erőforrás átadása

30.2.7 Gyakori hibák

A leggyakoribb hiba a `std::ref`-fel tárolt rövid életű objektum: a kód lefordul, a hívás később UB. A második, hogy a fejlesztő a `std::function` SBO-jára épít teljesítményérvként — ez implementációfüggő, ezért mérés nélkül hamis állítás. A harmadik, hogy üres `std::function`-t hívnak megfeltétel nélkül: ez `std::bad_function_call`, nem csendes no-op, tehát a hívás előtt `if (f)` ellenőrzés vagy szerződéses invariáns kell. Végül gyakori, hogy a `std::bind`-et `std::ref` nélkül használják nagy objektumon, és a másolat költsége rejtve marad.

30.2.8 Összefoglaló

A `std::invoke` egységesíti a tagfüggvény-, tagadat- és függvényobjektum-hívást, így a sablonkód nem kénytelen esetszétválasztásra. A `std::function` tulajdonló, másolható típusőrlő wrapper: hívása indirekció, konstrukciója allokálhat, üres hívása kivételt dob, és az SBO nem garantált. A `std::reference_wrapper` nem tulajdonló: a hivatkozott objektum élettartama a hívó felelőssége. A `bind_front` (C++20) és a `move_only_function` (C++23) a két hiányzó esetet fedi le — utóbbit feature-testtel és fallbackkal kell használni.

30.2.9 Feladatok

1. **Diagnosztikai.** Egy hosszú életű registry `std::function<void()>` elemeket tárol. A regisztráló kód `std::ref(state)`-tel ad át egy lokális `Handler` objektumot. Mi a hiba, hol jelentkezik, és hogyan javítod anélkül, hogy a `Handler` másolása drága lenne?

2. **Kódolási.** Írj egy `make_task()` függvényt, amely `std::unique_ptr<int>`-et elkapó lambdát ad vissza olyan típusban, amely C++23-on `std::move_only_function`, C++20-on pedig saját, minimális move-only wrapper. A wrapper tárolja a callable-t és híváskor továbbítson.

3. **Tervezési.** Egy socket-eseményhurok minden iterationben egy `std::function<void(int)>` handlert hív 10 000-szer. Indokold, mérés nélkül milyen állításokat *nem* tehetsz a költségéről, és adj két alternatívát a döntési táblázat alapján.

30.2.10 Megoldások

1. Megoldás. A `std::ref(state)` egy `reference_wrapper`-t tesz a `std::function`-ba, tehát a wrapper nem tulajdonol. A lokális `Handler` a regisztráló függvény végén megszűnik, a registryben maradt wrapper dangling; a hiba a későbbi híváskor jelentkezik, UB formájában, nem a regisztrációnál. Javítás: ha a `Handler` állapota megosztható, csomagold `std::shared_ptr`-be és azt kapd el érték szerint (közös tulajdonlás, dokumentált élettartam); ha egyedi tulajdonlás a szemantika, használj move-only wrappert (C++23 `std::move_only_function`, egyébként a 2. feladat wrapperét) és mozgasd be a `Handler`-t. Ellenőrzési feltétel: a wrapper élettartama alatt a hivatkozott objektum nem szűnhet meg, és ezt a kód vagy típus, vagy komment nem „ígéri”, hanem garantálja.

2. Megoldás. A minimális wrapper lényege a típusőrlés három művelete: hívás, mozgatás, megszüntetés.

```
// mini_task.cpp - C++20 fallback
#include <memory>
#include <utility>

class MoveTask {
public:
    MoveTask() = default;
    template <class F>
    explicit MoveTask(F f)
        : self_(std::make_unique<Model<F>>(std::move(f))) {}

    MoveTask(MoveTask&&) noexcept = default;
    MoveTask& operator=(MoveTask&&) noexcept = default;
    MoveTask(const MoveTask&) = delete;
    MoveTask& operator=(const MoveTask&) = delete;

    void operator()() { if (self_) self_->call(); }
    explicit operator bool() const noexcept { return static_cast<bool>(self_); }

private:
    struct Concept {
        virtual ~Concept() = default;
        virtual void call() = 0;
```

```
    };
    template <class F>
    struct Model final : Concept {
        explicit Model(F f) : f_(std::move(f)) {}
        void call() override { f_(); }
        F f_;
    };
    std::unique_ptr<Concept> self_;
};
```

Ez a szerkezet a vázlatban leírt három műveletet valósítja meg: a `Model<F>` a konstrukciókor ismert `F`-et rögzíti, a `Concept` pedig a hívást egységesíti. Az allokáció itt minden konstrukciónál megtörténik — ez a fallback ára, és pontosan ezért nem helyettesíti a `std::move_only_function` implementációját, ha az elérhető. Ellenőrzési feltétel: a típus másolása fordítási hibát ad, mozgatás után az eredeti `false`-ra értékelődik, és a hívás egyszer lefut.

3. Megoldás. Mérés nélkül nem állíthatod, hogy a hívás allokál, hogy a wrapper mérete 32 byte, hogy az SBO érvényesül, sem azt, hogy a hívás ciklusonként mennyi idő. Ezek implementációfüggők, illetve mérési kérdések. Amit állíthatsz: a `std::function` hívása indirekciót tartalmaz, a wrapper másolása a tárolt objektumot másolja, és az üres hívás kivételt dob, ezért a ciklusban a hívás előtti ellenőrzés vagy egy invariáns szükséges. Két alternatíva: (a) sablonparaméter — az eseményhurok legyen sablon a handler típusára, így az indirekció eltűnik, de a hurok fejléce template lesz, és a handler típusa fordítási időben rögzül; (b) nem tulajdonló `reference_wrapper` vagy függvénypointer, ha a handler élettartama egyértelműen hosszabb a huroknál. Ha a heterogenitás futásidőben kötelező, marad a `std::function`, de a költséget mérni kell, és az eredményt a laborprofilhoz (fordító, stdlib, target) kell kötni.

30.3 Ranges algoritmusok, projection és sentinel

A hagyományos standard algoritmusok két iterátort várnak, a `<algorithm>` ranges változata viszont egyetlen range-et – és ami a mindennapi hasznot legtöbbször hozza – egy *projectiont*. Ez a két változás együtt rövidíti le a hívási helyeket, de új szerződéseket is behoz: a range-nek ki kell elégítenie az algoritmus által előírt concepteket, a projectionnek hívhatónak kell lennie a dereferált iterátoron, és a visszatérési típus már nem feltétlenül iterator. Ebben az alfejezetben a `std::ranges` algoritmusok hívási szerződését, a `begin`/`end` customization pontját, a range/iterator/sentinel megkötéseket, a képességtípusokat, a projection gyakorlatát és a `dangling` jelzést tárgyaljuk. Az iterator-kategóriákra és a fél-nyílt `[first,last)` tartományra építünk (Iterator-kategóriák és fél-nyílt tartomány), a constrained overload mechanizmusát ismertnek vesszük (Concepts, requires és constrained overload), a member pointer mint callable pedig a `std::invoke` szabályai szerint működik (invoke, function wrapper és type erasure).

Forrásmegjegyzés. Ehhez az egységhez nem kaptam ellenőrzött forrásjegyzéket, ezért a szöveg nem hivatkozik szabványparagrafusra, és nem állítom, hogy bármely állítás egy konkrét szabványpéldányban vagy implementációban ellenőrizve lenne. Ahol a viselkedés a standard library verziójától függ, ezt külön jelzem.

30.3.1 A hívás szerződése: range, comparator, projection

A ranges algoritmus első paramétere `R&&` forwardoló referencia, amelyre a sablon `range`-megkötést ír elő. A további paraméterek típussorrendje algoritmusonként rögzített; a rendező és kereső algoritmusoknál jellemzően `(range, comp, proj)`, illetve `(range, value, proj)`. A **projection**

(vetítés) egy callable, amelyet az algoritmus a dereferált iterátor értékén alkalmaz, mielőtt az összehasonlítást vagy predikátumot kiértékelné:

$$\text{pred}(\text{proj}(*it)), \qquad \text{comp}(\text{proj}(*a), \text{proj}(*b)).$$

Az alapértelmezett projection `std::identity`, tehát a `std::ranges::sort(v)` és a `std::ranges::sort(v, std::less{}, std::identity{})` ugyanaz. A projection meghívása `std::invoke` szabályai szerint történik, ezért lehet tagfüggvény-pointer, member pointer vagy lambda is. A **comparatornak** a vetített értékeken strict weak orderinget kell megadnia; a **projection** szerződése, hogy az elemet nem módosítja (a szabvány ezt a nem-konstans hívás tilalmaként fogalmazza meg – a pontos megfogalmazást a használt szabványpéldányban érdemes ellenőrizni).

Az algoritmus saját megkötése például rendezésnél: a range `random_access_range`, az iteratorra pedig `sortable` a megadott comparatorral és projectionnel. Ha ezek nem teljesülnek, a hiba a koncepció nevével jelenik meg a diagnosztikában – ez a ranges egyik gyakorlati előnye a hagyományos overloadok hosszú hibaszövegeihez képest.

30.3.2 `ranges::begin` és `ranges::end` mint customization pont

A `std::ranges::begin` és `std::ranges::end` nem függvények a klasszikus értelemben, hanem *customization point object*ek (CPO). A `ranges::begin(r)` kiértékelése rögzített sorrendet követ: tömb esetén a tömb első elemére mutató pointer, egyébként ha van tagfüggvény `r.begin()`, akkor az, különben nem-tag `begin(r)` ADL-hívással; ha egyik sem értelmezhető, a kifejezés nem fordítódik le. A `ranges::end` ugyanezt teszi `end`-del, tömbnél pedig `begin + extent`-et ad.

Két következmény fontos. Egyrészt a saját konténer akkor is működik range-ként, ha csak nem-tag `begin`/`end` párja van — de a három hívási alak eltér. A minősített `std::begin(r)` szokásos overloadja az `r.begin()` tagfüggvényt hívja, szabad `begin`-t ADL-lel nem keres. A klasszikus generikus minta, a `using std::begin; begin(r);` nem minősített hívása ADL-lel megtalálja a típus névterében lévő szabad `begin`-t. A `std::ranges::begin` CPO mindkettőt kezeli (tagot és ADL-es szabad függvényt), következetesen és a `const`-helyességet is figyelve.

```
// adl_begin.cpp - C++20: std::begin, ADL-es begin és ranges::begin
#include <iterator>
#include <ranges>
namespace lib {
struct Bag { int data[3] = {1, 2, 3}; };        // nincs begin/end TAGFÜGGVÉNY
inline int* begin(Bag& b) { return b.data; }    // csak szabad függvények
inline int* end(Bag& b) { return b.data + 3; }
}
template <class R> auto generic_begin(R& r) { using std::begin; return begin(r);
 ↪  }  // ADL
int main() {
    lib::Bag b;
#ifdef QUALIFIED
    auto it0 = std::begin(b);                   // hiba: std::begin r.begin()-et
     ↪  hívna
#endif
    auto it1 = generic_begin(b);                // using std::begin; begin(r):
     ↪  ADL megtalálja
```

```
    auto it2 = std::ranges::begin(b);          // CPO: ADL-lel a szabad begin-t
     ↪  is keresi
    static_assert(std::ranges::range<lib::Bag&>);
    return (*it1 + *it2 == 2) ? 0 : 1;
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra` mellett a program lefordult és 0 kóddal futott; `-DQUALIFIED` mellett a `std::begin(b) no matching function for call to 'begin(lib::Bag&)'` hibát adott.

Másrészt a CPO-nak van egy törölt rvalue overloadja: nem-borrowed konténer temporálisára a `ranges::begin` nem hívható, így a leggyakoribb „temporális konténer iteratorát elmentem” hiba már fordítási időben megbukik. Ez a védelem azonban csak a `begin`/`end` szintjén érvényes; a lazán kiértékelt nézetekre ez nem jelent automatikus élettartam-garanciát (a részleteket lásd Lazy view pipeline és mellékhatások és Borrowed range, dangling és iterator lifetime).

30.3.3 Range-, iterator- és sentinel-megkötések

A `range<R>` concept lényegében azt követeli, hogy `ranges::begin(r)` és `ranges::end(r)` érvényes legyen, és az első egy `input_or_output_iterator`, a második pedig egy hozzá tartozó **sentinel**, azaz teljesüljön rá a `sentinel_for<sentinel_t<R>, iterator_t<R>>`. A sentinel nem kötelezően ugyanolyan típusú, mint az iterator: elég, hogy összehasonlítható legyen vele (`it != sent`), és hogy az iterálás a sentinel eléréséig definiált legyen. Ez teszi lehetővé a nullterminátoros C sztring vagy egy stream végének természetes modellezését.

A gyakorlatban ezért a ranges algoritmusok belső típusai `iterator_t<R>` és `sentinel_t<R>`; a kettő csak ott esik egybe, ahol a range *common*. A `sized_sentinel_for<S, I>` további megkötés: a `s - i` kifejezés konstans időben kiértékelhető távolságot ad. A `range`-en felül az algoritmusok további concepteket kérnek: `input_range`, `forward_range`, `random_access_range`, `contiguous_range`, valamint az iterator oldalán `indirectly_readable`, `indirectly_copyable`, `sortable`, `permutable` stb.

30.3.4 Képességtípusok: sized, common, contiguous, borrowed

A négy képesség egymástól független, és nem következtethetünk egyikből a másikra. A pontos definíciók:

Fogalom	Mit követel meg	Tipikus példa	Mit tesz lehetővé
`range`	`ranges::begin`/`end` érvényes, `sentinel_for` teljesül	minden konténer, `subrange`	range argumentumú algoritmus
`sized_range`	`ranges::size(r)` érvényes, amortizált konstans idejű	`std::vector`, `std::array`	távolság bejárás nélkül; `ranges::size`
`common_range`	`same_as<iterator_t<R>, sentinel_t<R>>`	`std::vector`, `std::string`	iteratorpáros API-val való átjárás
`contiguous_range`	`random_access_range` + `contiguous_iterator`	`std::vector`, `std::array`, `std::string`	`std::to_address`, byte-szintű másolás, `span`

Fogalom	Mit követel meg	Tipikus példa	Mit tesz lehetővé
`borrowed_range`	`R` lvalue referencia, vagy `enable_borrowed_range` igaz	`std::string_view`, `std::span`, `subrange`	temporális range-re is visszaadható iterator

A `common_range` hiánya nem hiba, csak meghatározza, mely algoritmusok és mely visszatérési típusok használhatók. A `sized_range` hiánya azt jelenti, hogy a hossz csak bejárással állapítható meg (`ranges::distance` lineáris). A `borrowed_range` pedig arról szól, hogy a range-ből nyert iterator érvényes marad-e a range objektum megszűnése után – ez nem azonos azzal, hogy a mögötte lévő elemek is életben maradnak (ezt a Borrowed range, dangling és iterator lifetime (30.5) fejti ki részletesen).

30.3.5 Projection a gyakorlatban

Rekordmező szerinti rendezés hagyományos alakja kézi comparator:

```
std::sort(hosts.begin(), hosts.end(),
          [](const Host& a, const Host& b) { return a.latency_ms < b.latency_ms;
          ↪ });
```

Ranges alakban a comparator általános marad, a mezőkiválasztás pedig projection:

```
std::ranges::sort(hosts, std::less{}, &Host::latency_ms);
```

A member pointer itt azért használható, mert a projection hívása `std::invoke`-on keresztül történik; a `&Host::latency_ms` pointerre (`Host*`, `const Host*`) is alkalmazható, ami pointertároló rendezésénél hasznos. Keresésnél ugyanez a szerkezet: `std::ranges::find(hosts, value, &Host::name)` a `proj(*it) == value` összehasonlítást végzi.

Két gyakorlati megjegyzés. Egyrészt a projection nem „cache-elt kulcs”: a comparator minden összehasonlításnál újra kiértékeli, így egy drága lambda $O(n \log n)$-szer futhat le; ilyenkor a decorate–sort–undecorate (előre kiszámított kulcsú segédtömb rendezése) a megfelelő minta. Másrészt az azonos kulcsú elemek sorrendje nem garantált: `sort` nem stabil, ezért kell `stable_sort`, vagy a projectionnek olyan összetett kulcsot kell adnia (például `std::pair`/`std::tuple`, amelyek lexikografikusan rendeződnek), amely a holtversenyt is eldönti.

30.3.6 Kidolgozott példa: hostlista rendezése és keresése

Az alábbi program a C++20 alapra épül, és a fenti elemeket együtt mutatja: range argumentum, comparator, projection, `borrowed_iterator_t` és `dangling`.

```
// hosts.cpp - C++20
#include <algorithm>
#include <iostream>
#include <ranges>
#include <string>
#include <string_view>
#include <type_traits>
#include <vector>
```

```cpp
struct Host {
    std::string name;
    int port;
    int latency_ms;
};

int main() {
    std::vector<Host> hosts{
        {"api-1",    443, 41},
        {"db-1",    5432,  7},
        {"cache-1", 6379, 12},
        {"api-2",    443, 95},
    };

    // range + comparator + projection: növekvő latencia
    std::ranges::sort(hosts, std::less{}, &Host::latency_ms);

    auto worst = std::ranges::max_element(hosts, std::less{},
     ↪  &Host::latency_ms);
    if (worst != hosts.end()) {
        std::cout << "leglassabb: " << worst->name << ' ' << worst->latency_ms
         ↪  << " ms\n";
    }

    // value + projection: a név mezőn keresztül keresünk
    auto hit = std::ranges::find(hosts, std::string_view{"cache-1"},
     ↪  &Host::name);
    if (hit != hosts.end()) {
        std::cout << "talalat: " << hit->port << '\n';
    }

    // Lvalue vector: az eredmény valódi iterator (borrowed_iterator_t).
    static_assert(std::is_same_v<decltype(worst), std::vector<Host>::iterator>);

    // Nem-borrowed temporális: az eredmény típusa std::ranges::dangling.
    static_assert(std::is_same_v<
        decltype(std::ranges::find(std::vector<int>{1, 2, 3}, 2)),
        std::ranges::dangling>);
}
```

Fordítás a laborban (a parancsot a tanuló futtatja; a fejezet nem tartalmaz futtatási eredményt):

```
g++ -std=c++20 -Wall -Wextra -pedantic -O1 hosts.cpp -o hosts
# vagy
clang++ -std=c++20 -Wall -Wextra -pedantic -O1 hosts.cpp -o hosts
```

A várt – nem mért – kimenet a rendezés és a vetítés alapján: `leglassabb: api-2 95 ms`, majd `talalat: 6379`. A `static_assert`ek nem futnak, fordítási időben ellenőriznek; ha a könyvtár a `std::ranges::dangling` típust nem a várt módon definiálja, a diagnosztika ezt jelzi.

30.3.7 Visszatérési típus és a `dangling` jelzés

A ranges algoritmusok többsége nem `iterator_t<R>`-t ad vissza, hanem `borrowed_iterator_t<R>`-t, amelynek definíciója: `iterator_t<R>`, ha `borrowed_range<R>`, egyébként `std::ranges::dangling`. Ugyanez a minta a `borrowed_subrange_t<R>` a range-et visszaadó algoritmusoknál. A `dangling` szándékosan „használhatatlan” típus: nem dereferálható, nem léptethető, csak azt jelzi, hogy az eredmény élettartama nem biztonságos.

Ez a jelzés fordítási idejű, de nem teljes körű: ha a temporális range-ből nyert iterátort nem használjuk, a program lefordul. A valódi javítás mindig élettartam-kérdés: nevezzük el a konténert, tartsuk életben a range-et, vagy olyan range-et használjunk, amelyre az `enable_borrowed_range` igaz (például `std::string_view`, `std::span`, lvalue-ból készült `subrange`). A `dangling` tehát diagnosztikai fogódzó, nem automatikus biztonsági háló.

30.3.8 Lefedettségbeli eltérés a hagyományos algoritmusokhoz képest

A C++20-as `<algorithm>` ranges változata gyakorlatilag teljes: a kereső, módosító, rendező, halmaz- és permutációs algoritmusoknak is van `std::ranges::` párjuk. A `<numeric>` nem ilyen: a klasszikus `accumulate`, `reduce`, `partial_sum`, `iota` hagyományos alakban maradt, ranges megfelelőjük a C++20 alapban nincs. A C++23 kiegészítések (például `ranges::fold_left/fold_right`, `ranges::iota`, `ranges::contains`, `ranges::starts_with/ends_with`, `ranges::find_last`, valamint a range-eket konténerré alakító `ranges::to`) **külön jelölt, verziófüggő** képességek: használatuk előtt ellenőrizni kell a fordítót és a standard libraryt, például a `<version>` header feature-test makróival (a konkrét makrónevek és értékek verziófüggők, ezért itt nem állítom őket). Ahol a ranges alak hiányzik, a hagyományos algoritmus továbbra is használható: `std::ranges::begin(r)` és `std::ranges::end(r)` átadásával a két világ keverhető, feltéve, hogy az iterator- és sentineltípusok az adott algoritmus követelményeinek megfelelnek.

30.3.9 Tipikus hibák

- **Hiányzó projection.** `std::ranges::find(hosts, "db-1")` a `Host` és a `const char*` összehasonlítását kéri, ami nem fordul le; a javítás a harmadik argumentum: `&Host::name`.
- **Rossz comparator-szerződés.** A `<=` vagy a `!comp(a,b) && !comp(b,a)` mint „egyenlőség” a comparatorban megsérti a strict weak orderinget; a `sort` viselkedése ilyenkor nem definiált.
- **Stabilitás feltételezése.** A `sort` nem stabil; holtversenynél a sorrend nem garantált.
- **`common_range` feltételezése.** A `r.begin() == r.end()` alak nem fordul le, ha az iterator és a sentinel típusa eltér; ilyenkor `std::ranges::begin(r) == std::ranges::end(r)` vagy explicit `subrange`/iteratorpár szükséges.
- **`sized_range` feltételezése.** `ranges::size(r)` nem létezik nem-sized range-re; a hossz `ranges::distance`-szel lineáris.
- **Temporális range iteratorának megtartása.** A `dangling` jelzés figyelmen kívül hagyása vagy típusának `auto`-val való elrejtése élettartam-hibához vezet.

30.3.10 Feladatok

1. feladat (magyarázó). Adott:

```
auto it = std::ranges::max_element(std::vector<int>{3, 1, 2}, std::less{},
 ↪  std::negate{});
```

Mi az `it` típusa, miért nem használható, és mit jelent szemantikailag a `std::negate{}` projection?

2. feladat (kódolás). Írj függvénysablont, amely egy `const` hostlistából visszaadja a legnagyobb latenciájú `n` hostra mutató pointert, a hostokat nem másolva, projectional paraméterezve. Készíts hozzá egy ellenőrzési feltételt is.

3. feladat (diagnosztika). Az alábbi két sor közül az első nem fordul le, a második lefordul, de az eredmény használhatatlan. Magyarázd meg mindkettőt, és javítsd ki.

```
auto a = std::ranges::find(hosts, "db-1");
auto b = std::ranges::find(std::vector<Host>{{"db-1", 5432, 7}}, "db-1",
 ↪  &Host::name);
```

30.3.11 Megoldások

1. megoldás. Az `it` típusa `std::ranges::dangling`, mert a range egy temporális `std::vector<int>`, amely nem `borrowed_range`, így a visszatérési típus `borrowed_iterator_t<R>` = `dangling`. A `dangling` nem dereferálható, tehát a `*it` nem fordul le – ez a szándékolt védelem. Szemantikailag a `std::negate{}` projection a `-x` értéket adja, a `max_element` pedig ezen a vetített értéken maximalizál, azaz a legkisebb `x`-et találja meg. Ez nem fordítási hiba, hanem logikai hiba: a projection megváltoztatta a keresés irányát. Ellenőrzési feltétel: a program csak akkor helyes, ha az `it` élettartama biztosított (nevesített konténer) és a projection szándéka egyezik a maximum/minimum irányával.

2. megoldás. A lényeg, hogy a range `const` marad, a kimenet pointereket tartalmaz, a projection pedig a hívótól jön:

```
#include <algorithm>
#include <functional>
#include <iterator>
#include <ranges>
#include <type_traits>
#include <vector>

template <class R, class Proj = std::identity>
    requires std::ranges::random_access_range<const R>
std::vector<const std::ranges::range_value_t<const R>*>
top_by(const R& r, std::size_t n, Proj proj = {}) {
    using T = std::ranges::range_value_t<const R>;
    std::vector<const T*> ptrs;
    ptrs.reserve(std::ranges::size(r));
    for (const T& x : r) {
        ptrs.push_back(&x);
    }
    n = std::min(n, ptrs.size());
    std::ranges::partial_sort(ptrs, ptrs.begin() +
     ↪  static_cast<std::ptrdiff_t>(n),
                              std::greater{}, proj);
    ptrs.resize(n);
    return ptrs;
}
```

A member pointer projection pointerre is alkalmazható (`std::invoke(&Host::latency_ms, ptr)`), ezért a `top_by(hosts, 2, &Host::latency_ms)` hívás a két legnagyobb latenciájú hostra mutató pointert adja. Indoklás: a `partial_sort` csak az első `n` elemet rendezi, így a költség $O(m \log n)$ a teljes rendezés $O(m \log m)$ helyett; a `reserve` elkerüli az újraallokációt, a pointerek pedig nem tulajdonolnak, ezért a bemenet élettartama a hívó felelőssége. A `partial_sort` nem stabil; ha a holtversenyes sorrend is számít, `stable_sort` vagy összetett kulcsú projection kell. Ellenőrzési feltétel: `top_by(hosts, 2, &Host::latency_ms)[0]->latency_ms` a legnagyobb latencia, a visszaadott vektor hossza `min(n, hosts.size())`, és semmilyen pointer nem mutat a bemeneti vektortól eltérő tárolóra.

3. megoldás. Az első sor azért nem fordul le, mert a `find` a `Host` értéket hasonlítja a `const char*`-hoz, amire nincs érvényes `operator==`; a projection hiányzik. A javítás: `std::ranges::find(hosts, std::string_view{"db-1"}, &Host::name)`. A második sor lefordul, de a range egy temporális vektor, ezért a visszatérési típus `std::ranges::dangling`; az eredmény nem használható iterator. Javítás: nevezzük el a konténert (`std::vector<Host> tmp{...}; auto b = std::ranges::find(tmp, ...);`), vagy használjunk eleve borrowed range-et. Ellenőrzési feltétel: a javított kód forduljon `-Wall -Wextra -pedantic` mellett, és a keresés eredménye dereferálható legyen.

30.3.12 Összefoglaló

A ranges algoritmus range-et vár, a projectiont `std::invoke` szabályai szerint alkalmazza, és az eredmény `borrowed_iterator_t`, illetve `borrowed_subrange_t` – nem-borrowed temporális esetén `dangling`. A `begin`/`end` customization pont egységesen kezeli a tag- és nem-tag függvényeket, és törölt rvalue overloaddal fogja meg a legdurvább élettartam-hibát. Az iterator és a sentinel típusa eltérhet; ezt a `common_range`, a `sized_range`, a `contiguous_range` és a `borrowed_range` képességek egymástól függetlenül írják le. A `<algorithm>` ranges lefedettsége C++20-ban teljes, a `<numeric>`-é nem; a C++23-as kiegészítéseket mindig jelölni és implementáció szerint ellenőrizni kell.

30.4 Lazy view pipeline és mellékhatások

A ranges pipeline első ránézésre úgy viselkedik, mint egy láncolt adatfeldolgozó: szűrünk, transzformálunk, kiveszünk elemeket. A lényegi különbség az algoritmusokhoz képest az, hogy a view kiértékelése lusta (lazy): a pipeline létrehozása nem számol semmit, legfeljebb egy bejárási szerződést ír le. Ebből két, a gyakorlatban döntő kérdés következik: *mikor* fut le a tényleges munka, és *meddig él* mindaz, amire a view hivatkozik. Ez az alfejezet ezt a két kérdést járja körül a `views::filter`, `transform`, `take`, `drop` és `reverse` példáján; a ranges algoritmusok, a projection és a sentinel mechanizmusa Ranges algoritmusok, projection és sentinel, a `string_view` tulajdonlási kérdése std::string, std::string_view és szövegownership anyaga.

30.4.1 View, adaptor és a lustaság szerződése

Egy `range` olyan típus, amelyre a `std::ranges::begin` és `std::ranges::end` értelmezett, és az így kapott iterátorok bejárhatók. Egy `view` (nézet) olyan range, amely olcsón másolható és mozgatható, és amelynek másolása nem másolja a mögöttes elemeket: a nézet referenciaszemantikával írja le, *hol* vannak az elemek, nem azt, hogy *melyek* az elemek. A C++20 `std::ranges::view` követelményei a szabvány megjelenése óta pontosításon estek át (a P2415R2 javaslat lazította a korábbi, szigorúbb megkötést); a lényeg változatlan: egy view lemásolása olcsó, és a másolat ugyanarra az adatra néz.

Egy **range adaptor** (tartomány-adaptor) egy customization point object, amely egy range-ből view-t készít. A `views::filter(p)` tehát nem egy algoritmus, hanem egy adaptor-objektum, amelyet range-re alkalmazva `filter_view`-t kapunk. A három írásmód ekvivalens:

```
auto a = std::views::filter(v, pred);   // közvetlen hívás
auto b = std::views::filter(pred)(v);   // adaptor alkalmazása
auto c = v | std::views::filter(pred);  // pipe operátor
```

A `v` itt nem feltétlenül ugyanaz a típus a három esetben: a pipe lánc bal oldalán álló kifejezés értékkategóriája és view-volta határozza meg, hogy `views::all` milyen view-t készít (erről később, a `ref_view`/`owning_view` részben).

A lustaság nem optimalizálás, hanem szemantikai döntés. Azt jelenti, hogy a pipeline objektum létrehozása megengedi, de nem kényszeríti ki a bejárást. A szabvány ezt nem úgy fogalmazza meg, hogy „a könyvtár lusta", hanem úgy, hogy a view műveletei a bejárás során, az iterátorműveletekhez kötve vannak specifikálva. Ebből az következik, hogy a mellékhatásos predikátum vagy transzformáció hívásainak száma és sorrendje nem egy jól definiált „egyszer elemenként" szerződés, hanem a bejárás tényleges menetétől függ.

30.4.1.1 Mikor fut le egy művelet?

A gyakorlati szabály a következő. A pipeline **konstrukciója** jellemzően nem jár elemenkénti munkával; legfeljebb O(1) előkészítés történik (például egy `reverse` a közös `end`-et kéri le, egy `drop` random access alap esetén kiszámítja a kezdő eltolást). A **tényleges hívások** a bejárás elemi műveleteihez kötődnek:

- `begin()`: a `filter` itt szkennel előre az első olyan elemig, amelyre a predikátum igaz; a `drop` itt lépteti át az első n elemet; a `take` itt csak az alap `begin()`-jét kéri.
- `++it`: a következő elem megkeresése; `filter` esetén további predikátumhívások.
- `*it`: a `transform` függvényének meghívása az aktuális elemen; a `filter` nem hív predikátumot dereferáláskor, mert a szűrést már a léptetés elvégezte.
- `end()` és az `it != end` összehasonlítás: `take` esetén nem `sized_range` alapon ez további iterálást jelenthet, mert a végpontot ki kell számítani.

Következmény: **a ranges nem cache-el**. A `filter_view` nem tárolja el a már megfelelt elemeket, a `transform_view` nem memorizálja az eredményt. A pipeline egy recept, amelyet minden bejárás újra végrehajt.

30.4.2 Filter/transform/take/drop/reverse: a kidolgozott példa

A következő program teljes, önállóan fordítható. A laborprofilban rögzített C++20-as fordítóval és standard library-vel kell fordítani; a `-std=c++20` megadása kötelező, mert a ranges a C++20 része, és a fordító alapértelmezett nyelvi módja verziófüggő.

```
// pipeline.cpp
#include <iostream>
#include <ranges>
#include <vector>

int main() {
    std::vector<int> data{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

```
    auto even_sq = data
        | std::views::filter([](int i) { return i % 2 == 0; })
        | std::views::transform([](int i) { return i * i; })
        | std::views::drop(1)
        | std::views::take(3);

    for (int x : even_sq) {
        std::cout << x << ' ';
    }
    std::cout << '\n';
}
```

```
g++ -std=c++20 -O2 -Wall -Wextra -Wpedantic pipeline.cpp -o pipeline
# vagy
clang++ -std=c++20 -O2 -Wall -Wextra -Wpedantic pipeline.cpp -o pipeline
./pipeline
```

A *szerződés alapján várható* kimenet `16 36 64`. Ez nem mért eredmény: a számítás menete a következő. A `data` elemei `1..10`; a szűrő az első szakaszban a párosakat engedi át: `2, 4, 6, 8, 10`. A transzformáció ezeket négyzetre emeli: `4, 16, 36, 64, 100`. A `drop(1)` az első elemet elhagyja: `16, 36, 64, 100`. A `take(3)` legfeljebb három elemet enged át: `16, 36, 64`. Fontos, hogy a `take(3)` miatt a `100` elemen a transzformáció már nem feltétlenül fut le, hiszen a bejárás a harmadik elem után megállhat; ez a lustaság egyik gyakorlati haszna, és egyben a mellékhatásokkal kapcsolatos csapda is.

A lustaságot a legjobban egy nyomkövető (`trace`) transzformációval lehet megfigyelni. A következő programban a pipeline létrehozása után kiíratunk egy jelzést, majd kétszer bejárjuk ugyanazt a view-t:

```
// lazy_trace.cpp
#include <iostream>
#include <ranges>
#include <vector>

int main() {
    std::vector<int> v{10, 21, 32, 43, 54};

    auto pipe = v
        | std::views::filter([](int x) { return x % 2 == 0; })
        | std::views::transform([](int x) {
              std::cout << "[t" << x << "]";
              return x / 2;
          });

    std::cout << "pipeline letrejott\n";
    for (int x : pipe) std::cout << x << ' ';
    std::cout << "\nmasodik bejaras: ";
    for (int x : pipe) std::cout << x << ' ';
    std::cout << '\n';
}
```

A konstrukció után előbb a `pipeline letrejott` szöveg jelenik meg, mert a view létrehozása nem iterál. Az első bejárás a transzformációt csak a szűrőn átjutó elemeken hívja meg (a `[t...]` jelzések

száma kevesebb, mint a bemenet hossza). A második bejárás **újra végrehajtja** ugyanazokat a transzformációkat: a jelzések újra megjelennek, mert nincs cache. Ez a viselkedés nem hiba, hanem a nézet szerződése; a mellékhatásos transzformáció viszont így ismételt hatást fejt ki, ami naplózásnál vagy számlálásnál már önmagában is probléma.

30.4.3 Capture, predikátumállapot és újraiterálás

A lambda closure objektum a view-ba **bemásolva** kerül: a `filter_view` a predikátumát tagként tárolja. Ez két dolgot jelent. Egyrészt a `[&]` capture esetén a view csak egy referenciát tárol a hivatkozott objektumra, tehát a hivatkozott objektumnak tovább kell élnie, mint a view-nak *és* minden olyan iterátornak, amelyet a view-ból készítettünk. Másrészt a `mutable` lambda belső állapota a view példányában él, így a view másolása az állapotot is másolja, és a view ismételt bejárása ugyanazt az állapotot folytatja.

```
// stateful_pred.cpp
#include <iostream>
#include <ranges>
#include <vector>

int main() {
    std::vector<int> v{5, 1, 4, 2, 3};
    int calls = 0;   // a view ezt referenciaként latja

    auto big = v | std::views::filter([&calls](int x) { ++calls; return x > 2;
     ↪  });

    for (int x : big) std::cout << x << ' ';
    std::cout << "\nhivasok az elso bejaras utan: " << calls << '\n';

    for (int x : big) std::cout << x << ' ';
    std::cout << "\nhivasok a masodik bejaras utan: " << calls << '\n';
}
```

A predikátumhívások pontos száma nem egy „egyszer elemenként” szerződés: a `begin()` előreszkennelése, a léptetés és az esetleges korai megállás miatt a szám implementáció- és bejárásfüggő. A predikátum szerződése három külön kérdés. (1) **Szemantika:** a `filter` predikátumának `std::indirect_unary_predicate`-et kell modelleznie, ami `regular_invocable`-t jelent: a hívás egyenlőségőrző (azonos bemenetre azonos eredmény, közbenső módosítás nélkül), és nem módosíthatja sem az argumentumot, sem a saját állapotát. Nincs viszont semmilyen intervallum- vagy ekvivalenciafeltétel a szomszédos elemekre: a `{1, 2, 3, 4, 5}` páratlan elemeinek szűrése `{1, 3, 5}`-öt ad, és ez teljesen szabályos. (2) **Megismételhetőség:** mivel a hívások száma nem specifikált, a predikátum eredménye nem függhet attól, hányszor hívták. (3) **Élettartam:** a view a predikátum egy példányát saját tagként tárolja, ezért a predikátumnak a view élettartamáig hívhatónak kell maradnia — a referenciával rögzített állapotnak (mint itt a `calls`) is. A fenti számláló nem változtat a predikátum eredményén, ezért nem sérti a szerződést, de a kiírt szám csak megfigyelés, nem garancia; tanítási példa arra, hogy a predikátum a view-ban él; termelési kódban ez a minta könnyen nem reprodukálható viselkedést okoz.

Külön élettartam-kérdés, hogy a view-ból kapott **iterátor** meddig érvényes. Két független invalidáló tényező van: az alap tároló módosítása (például `push_back` egy `vector`-ra, ami minden iterátort és referenciát invalidál), és magának a view objektumnak a megszűnése. Utóbbi azért

kritikus, mert a `filter_view` iterátora a view belső predikátumállapotára hivatkozhat; ha a view egy blokkon belül jön létre, az iterátort nem lehet kivinni a blokkból.

30.4.4 ref_view, owning_view és az rvalue range-ek

Amikor egy adaptort egy range-re alkalmazunk, a könyvtár először `views::all`-t hív. Ennek eredménye dönti el, hogy a pipeline birtokol-e valamit:

- **lvalue range** (például egy nevesített `std::vector<int> v`) esetén az eredmény egy `ref_view`: nem birtokol, csak hivatkozik `v`-re, ezért `v`-nek túl kell élnie a view-t és az iterátorait;
- **rvalue, nem view range** (például egy `make_vector()` hívás eredménye) esetén a modern C++20 könyvtárak `owning_view`-ot készítenek, amely *mozgatással birtokolja* a tárolót. Ennek a viselkedésnek a pontos bevezetése verziófüggő (a P2415R2 pontosítás nyomán), ezért a konkrét fordítón és standard library-n ellenőrizni kell;
- **rvalue view** (például egy ideiglenes `std::string_view`, `std::span` vagy saját view) esetén a `views::all` a view-t másolja. A másolat ugyanarra a külső adatra néz, tehát ha a külső adat megszűnt, a pipeline dangling marad — az `owning_view` itt nem segít, mert nincs mit birtokolni.

A tipikus hiba tehát nem az, hogy „a view lusta”, hanem hogy a pipeline élettartama nem fedi le a mögöttes adat élettartamát:

```
// HIBA: dangling, nem futtatando
#include <ranges>
#include <vector>

auto make_view() {
    std::vector<int> v{1, 2, 3, 4};
    return v | std::views::filter([](int x) { return x > 1; });  // ref_view
     ↪  v-re!
}
```

Itt `v` lvalue, ezért `ref_view` készül, és a függvény visszatérésekor `v` megszűnik: a visszaadott view és minden belőle készített iterátor dangling. A javítás lehet a tároló visszaadása (a hívó birtokolja), vagy szándékos birtoklás: `return std::vector<int>{1, 2, 3, 4} | std::views::filter(...);`, ahol az ideiglenes vektor `owning_view`-ba kerül. A két változat *nem* ekvivalens: az első értéket ad vissza, a második nézetet, amely birtokol.

Érdemes a pipeline élettartamát táblázatban rögzíteni, mert ez a review legfontosabb ellenőrzőlistája:

Entitás	Birtokol?	Élettartam	Invalidáló művelet
`std::vector<int> data`	igen	a deklarációtól a blokk végéig	`push_back`, `clear`, újabb allokáció
`ref_view` (`data | views::filter`)	nem	legfeljebb `data` élettartama	`data` megszűnése vagy módosítása
`owning_view` (ideiglenes vektor)	igen	a view élettartama	a view mozgatása/újraértékelése
view-ból kapott iterátor	nem	legfeljebb a view *és* az adat élettartama	a view vagy az adat megszűnése

Entitás	Birtokol?	Élettartam	Invalidáló művelet
`[&]` capture a predikátumban	nem	legfeljebb a hivatkozott objektum élettartama	a hivatkozott objektum megszűnése

30.4.5 Nem garantált tulajdonságok: sized, common, borrowed

Egy view nem feltétlenül `sized_range` (van O(1) `size()`), nem feltétlenül `common_range` (a `begin()` és `end()` azonos típusú), és nem feltétlenül `contiguous_range`. Ezek a tulajdonságok a pipeline mentén változhatnak:

- a `filter` után a range jellemzően **nem** `sized_range`, mert a méret csak bejárással derülne ki; ezért a `std::ranges::distance` itt O(n) lesz;
- a `transform` megőrzi a `sized` és `common` tulajdonságot, ha az alap rendelkezik vele, de a `contiguous`-ot nem;
- a `take` nem `sized` alapon a végpontot iterálással számolja, és tipikusan nem `common_range` (ezért a `views::reverse` rá gyakran nem is alkalmazható);
- a `drop` random access alapon O(1) `begin()`-et ad, láncolt alapon O(n)-t;
- a `views::reverse` `bidirectional_range`-et igényel, és a C++20 szövege `common_range` alapot vár; a pontos megkötés a használt standard és library verziójától függ, ezért érdemes `static_assert`-tel ellenőrizni.

A `borrowed_range` fogalma azt fejezi ki, hogy a range-ből kapott iterátorok érvényessége nem függ magának a range objektumnak az élettartamától (tipikusan lvalue referenciára vagy `enable_borrowed_range`-ra épül). Adaptorokon keresztül ez öröklődhet, de nem szabad rá hagyatkozni: ha a view belső állapotára (predikátumra, cache-re) hivatkozó iterátort ad, a view megszűnése után az iterátor dangling. A laborban a `static_assert(std::ranges::borrowed_range<decltype(pipe)>)` és a `sized_range`/`common_range` állítások adják a leggyorsabb visszajelzést.

30.4.6 Tipikus hibák és javításuk

1. **Azt hisszük, hogy a pipeline kiszámolta az eredményt.** A `views::transform` alkalmazása után nincs új tároló; ha értékre van szükség, azt materializálni kell (C++20-ban `std::ranges::copy` `back_inserter`-rel, C++23-ban `std::ranges::to` — ez utóbbi külön jelölendő, mert C++23-as képesség).
2. **A view túlél az adatnál.** A `make_view()` fenti példája; javítás: a tároló visszaadása, vagy szándékos `owning_view`.
3. **A `[&]` capture a view-ba kerül, de a hivatkozott objektum rövidebb életű.** Javítás: érték szerinti capture, ha az állapot kicsi és másolható.
4. **Állapotfüggő predikátum.** A `mutable` számláló miatt a második bejárás más eredményt ad; javítás: tiszta predikátum, a számlálást a bejárás mellé tenni.
5. **`size()`-ra építés filter után.** A `filter` nem `sized`; a `distance` O(n), és nem is hívható meg minden esetben O(1)-ként.
6. **Mellékhatásos transzformáció.** Naplózás vagy számlálás a `transform`-ban a lustaság miatt kiszámíthatatlan számban futhat le, és a `take` miatt el is maradhat.

30.4.7 Összefoglaló

A ranges view **leírás**, nem eredmény: a `filter`/`transform`/`take`/`drop` lánc konstrukciója nem iterál, a munka a `begin()`, `++`, `*it` és az összehasonlítások során történik. Nincs cache, ezért minden bejárás újrafuttatja a predikátumot és a transzformációt; a mellékhatásos callable-ek száma nem szerződés. A lambdák closure-je a view-ba másolva kerül, így a `[&]` capture és a `mutable` állapot élettartama a view élettartamához kötődik. A `views::all` lvalue range-re `ref_view`-t, ideiglenes nem view range-re `owning_view`-ot készít (verziófüggő), ideiglenes view-ra viszont csak a view-t másolja — ez a leggyakoribb dangling-forrás. A `sized`, `common`, `borrowed` tulajdonságok adaptoronként változnak, ezért ezeket a laborban `static_assert`-tel kell ellenőrizni, nem feltételezni.

30.4.8 Feladatok

1. feladat (magyarázó). Az alábbi kódrészletben hányszor hívódik meg a predikátum és a transzformáció, ha a `for` ciklus végigmegy? Indokolj, és mondd meg, mi változik, ha a `take(2)` helyett `take(10)` áll.

```
std::vector<int> v{1, 2, 3, 4};
auto pipe = v
    | std::views::filter([](int x) { return x % 2 == 0; })
    | std::views::transform([](int x) { return x + 1; })
    | std::views::take(2);
for (int x : pipe) { /* ... */ }
```

2. feladat (kódolási). Írj olyan függvényt, amely egy `const std::vector<int>&` bemenet páratlan elemeinek négyzetösszegét adja vissza ranges pipeline-nal, és garantáltan nem hivatkozik dangling adatra. Add meg a buildparancsot és az ellenőrzési feltételt.

3. feladat (diagnosztikai). Az alábbi függvényt egy kolléga írta. Azonosítsd a hibát, javítsd két különböző szerződésű változattal (értékvisszaadás és birtokló nézet), és mondd meg, milyen eszközzel lehetne a hibát detektálni.

```
auto positive(const std::vector<int>& src) {
    return src | std::views::filter([](int x) { return x > 0; });
}
```

4. feladat (elmélkedő). Miért nem lehet a `std::views::reverse`-et általában egy `take` után alkalmazni, és miért nem `sized_range` a `filter` eredménye?

30.4.9 Megoldások

1. megoldás. A bemenet `1, 2, 3, 4`; a szűrő a `2`-t és a `4`-et engedi át. A `take(2)` pontosan két elemet kér, ezért a bejárás a második egyezés után megállhat: a transzformáció kétszer fut le (a `2`-n és a `4`-en). A predikátumhívások száma legalább annyi, ahány elemet meg kell vizsgálni az első két egyezés megtalálásához, azaz négy (a `2` és a `4` között a `3` is megvizsgálandó) — de a pontos szám a `begin()` előreszkennelésének és a léptetésnek a függvénye, ezért csak alsó korlátként kezelendő, nem egzakt szerződésként. `take(10)` esetén a bejárás a range végéig tart, így mind a négy elemre lefut a predikátum, a transzformáció pedig a két párosra; a válasz attól is függ, hogy a `filter` `common_range`-e az adott könyvtárban, mert az `end()` összehasonlítás további munkát jelenthet. Ellenőrzési feltétel: a viselkedést `static_assert`-tel és nyomkövető transzformációval kell igazolni a rögzített fordítón.

2. megoldás. Az owner a hívó `vector`-ja, a pipeline csak `ref_view`-t készít, és a függvény visszatéréséig él:

```
// sum_odd_sq.cpp
#include <cstddef>
#include <ranges>
#include <vector>

std::size_t sum_odd_squares(const std::vector<int>& src) {
    std::size_t sum = 0;
    auto pipe = src
        | std::views::filter([](int x) { return x % 2 != 0; })
        | std::views::transform([](int x) { return static_cast<std::size_t>(x) *
        ↪  static_cast<std::size_t>(x); });
    for (auto x : pipe) sum += x;
    return sum;
}
```

```
g++ -std=c++20 -O2 -Wall -Wextra -Wpedantic sum_odd_sq.cpp -o sum_odd_sq
```

Indoklás: a `src` lvalue, ezért `ref_view` készül, és a view élettartama a függvényre korlátozódik, nem kerül ki a hívóhoz. Ha a függvény view-t adna vissza, az dangling lenne. Ellenőrzési feltétel: `sum_odd_squares({1, 2, 3}) == 10`, és a függvény ne adjon vissza referenciát vagy view-t. A négyzet kiszámításánál a `std::size_t`-re konvertálás megelőzi az `int` túlcsordulást nagyobb bemenetnél; ez a szerződés része.

3. megoldás. A hiba: `src const` lvalue referencia, ezért `views::all` `ref_view`-t készít, a visszaadott view pedig a hívóhoz tartozó `vector`-ra hivatkozik. Ha a hívó az ideiglenessel hívja (`positive({1, 2, 3})`), a visszaadott view dangling. Értékvisszaadó változat:

```
std::vector<int> positive_value(const std::vector<int>& src) {
    std::vector<int> out;
    for (int x : src | std::views::filter([](int x) { return x > 0; }))
        out.push_back(x);
    return out;
}
```

Birtokló nézet változat (a bemenetet átveszi, nem referenciát tart):

```
auto positive_owned(std::vector<int> src) {
    return std::move(src) | std::views::filter([](int x) { return x > 0; });
}
```

A második változat csak akkor biztonságos, ha a `views::all` a mozgatott rvalue vektort `owning_view`-ba teszi; ezt a használt standard library verzióján ellenőrizni kell. Detektálás: AddressSanitizer (`-fsanitize=address`), amely stack-használat-utáni-return esethez külön kapcsolóval (`detect_stack_use_after_return`) is futtatható, valamint a fordító statikus figyelmeztetései; a tool viselkedése verziófüggő, ezért a laborban rögzíteni kell.

4. megoldás. A `filter` eredménye nem `sized_range`, mert a szűrt elemek száma csak a teljes bejárással derülne ki; egy O(1) `size()` szerződést nem lehet tartani. A `views::reverse` `bidirectional_range`-et igényel, és a C++20 szövege `common_range` alapot vár; a `take` eredménye nem `sized` alapon tipikusan nem `common_range` (az `end()` egy sentinel), ezért a `reverse` rákötése jellemzően fordítási hibát ad. A megoldás: előbb materializálni a szűrt/szeletelt eredményt

egy tárolóba, és azt megfordítani, vagy olyan sorrendet választani, ahol a `reverse` közvetlenül egy kétirányú alapra kerül.

30.5 Borrowed range, dangling és iterator lifetime

A lazy view pipeline sokkal közelebb áll a C callback-mintához, mint elsőre látszik: egy nézet nem tulajdonolja az elemeket, csak egy szelekciós transzformációt ír le. Ez a rugalmasság három egymástól független élettartamot hoz be a kódba, és ha ezeket a program összemosolja, az iterátor pontos típusa már nem segít — a `begin()` hívás sikeresen lefut, csak a kapott érték másodpercekkel később érvénytelenné válik. Ez az alfejezet azt a három réteget különíti el, megmutatja, mit garantál pontosan a `std::ranges::borrowed_range`, mit jelképez a `std::ranges::dangling`, és miért nem oldja meg önmagában a `borrowed_range` a `filter_view`-k körüli tipikus hibákat.

30.5.1 Owner, view objektum és iterator: három különböző élettartam

A C++ ranges világában minden elemzési folyamatot először három külön fogalomra kell bontani:

- **Háttértároló (backing storage)** — a memóriaterület, ahol az elemek ténylegesen helyet foglalnak. Birtokolhatja egy `std::vector`, egy `std::string`, egy `std::array`, egy C-tömb, vagy éppen `mmap` mögötti memória. Élettartama az owner objektumé.
- **View objektum** — a ranges fogalma szerint olyan tartomány, amelynek másolása/mozgatása és destrukciója konstans idejű (`std::ranges::view`). A *tulajdonlás* ettől külön tengely: a legtöbb view (pl. `std::string_view`, `std::span`, `ref_view`, `filter_view`, `subrange`) kis, nem birtokló leíró, amely hivatkozást tárol a backing storage-ra, a szűrőt/projekciót és a határt; az `owning_view<R>` viszont birtokolja a mögöttes tartományt (C++20: csak rvalue-ból épül), és ettől még view. A `ref_view/span` (nem birtokló) és az `owning_view` (birtokló) tehát külön kategória, mindkettő view marad. Élettartama a változóé, amibe kötjük, vagy a temporary kifejezés végéig tart.
- **Iterator és predikátum-kapcsolat** — az iterátor nem mindig csak a storage-ra mutat. Egy `filter_view` vagy `transform_view` iterátora az aktuális *view objektumra* mutató pointert is tartalmaz, mert a filter-állapotot vagy a projekciós függvényt onnan olvassa. Ilyenkor az iterátor élettartama a view objektumhoz kötődik, nem a tárolóhoz.

A három réteg közti függés táblázatosan:

Réteg	Ki birtokolja	Mi szünteti meg	Mit visz magával, ha megszűnik
Háttértároló	`vector`, `string`, `array`, C-tömb, `mmap` owner	owner destruktora, deallokáció	minden rá mutató pointer, view, iterator
View objektum	a változó, amibe kötöttük	scope vége, temporary vége	minden olyan iterátor, ami a view-ra hivatkozik (pl. `filter`, `transform`)
Iterator	a hívó	nem „szüntethető meg” — csak érvénytelenné válhat	ő a függő entitás, nem őt kell életben tartani

Az **iterátor objektumnak** (a változónak) is van saját élettartama: a blokk végén megszűnik akkor is, ha a backing storage és a view él. Ez más tulajdonság, mint az iterátor *érvényessége*

(dereferálható-e), amely a storage, a view vagy a predikátum állapotától függ; az élettartam-táblázat ezért három entitást és az iterátor két külön tulajdonságát (élettartam, érvényesség) különíti el.

Ez a felosztás magyarázza, miért lehet *egy* view objektumon keresztül *sok* iterátort létrehozni, és miért nem lehet a view-tól független iterátort továbbéltetni. A Lazy view pipeline és mellékhatások (30.4) fejezetben felépített `filter`-lánc élettartam-kockázata pontosan ebből a három rétegből ered.

30.5.2 Mit garantál a `borrowed_range` — és mit nem

A `std::ranges::borrowed_range<R>` a C++20 standard `[range.range]` szakasza szerint akkor teljesül, ha:

$$\text{borrowed_range}(R) \equiv \text{range}(R) \wedge (\text{is_lvalue_reference}(R) \vee \text{enable_borrowed_range}(\text{remove_cvref}(R))).$$

A jelentése egyetlen szemantikai szabály: egy `R` típusú range-ből szerzett iterátor érvényessége **nem függ annak a konkrét range objektumnak az élettartamától**, amelyből szereztük. Egy függvény tehát érték szerint (vagy forwarding referenciával) átvehet egy borrowed range-et, és biztonságosan visszaadhatja a belőle kapott iterátorokat; az iterátor ettől még függhet egy *másik*, tovább élő objektumtól (a mögöttes tárolótól), és annak élettartamát a hívónak kell biztosítania. Két út vezet ide:

1. **`enable_borrowed_range<T>` igaz:** a típus maga ígéri, hogy iterátorai nem a range objektumba, hanem azon kívülre mutatnak. A `std::span` és a `std::string_view` iterátora a hivatkozott puffert járja be, ezért a span vagy string_view objektum megszűnése után is érvényes — amíg a puffer él.
2. **`R` lvalue referencia:** ilyenkor a „range objektum” a hívó által birtokolt, a hívásnál tovább élő objektum. Ha `R` például `std::vector<int>&`, a `ranges::find` által visszaadott iterátor a hívás után is érvényes, mert a vektor a hívó változója — de csak amíg az a vektor él. Ugyanez igaz egy `filter_view&`-re: a visszaadott iterátor addig érvényes, amíg a hívó `filter_view` objektuma él.

Prvalue `std::vector<int>` vagy prvalue `filter_view` esetén egyik út sem áll fenn: a vektor iterátora a temporary pufferébe, a `filter_view` iterátora a view objektumra (a predikátumra és a `begin()`-cache-re) is hivatkozik, ezért a range-algoritmusok ilyenkor `std::ranges::dangling`-et adnak vissza.

Néhány konkrét eset:

Típus	`borrowed_range`	Miért
`std::vector<int>&`	igaz	lvalue referencia szabálya
`std::vector<int>`	hamis	nincs `enable_borrowed_range` specializáció
`std::string_view`	igaz	`enable_borrowed_range` = `true`
`std::span<int>`	igaz	`enable_borrowed_range` = `true`
`std::ranges::subrange<I, S>`	igaz	`enable_borrowed_range` = `true`

Típus	`borrowed_range`	Miért
`std::ranges::filter_view<V, P>`	hamis	nincs specializáció, iterátor view-függő
`std::ranges::filter_view<V, P>&`	igaz	lvalue szabálya, de ez nem védi a view élettartamát

A `borrowed_range` tehát *nem* a backing storage élettartamát garantálja, hanem csak azt, hogy az iterátorok nem a range objektum élettartamához kötöttek. Ez egy szűk, de nagyon hasznos szabály. Az `std::ranges::find(std::vector{1, 2, 3}, 2)` hívás maga szabályos és lefordul; a visszatérési típusa azonban nem iterátor, hanem `std::ranges::dangling`, így a fordító az *eredmény* iterátorként való használatát utasítja el. Lvalue vektornál ugyanez a hívás valódi iterátort ad.

```
#include <algorithm>
#include <ranges>
#include <type_traits>
#include <vector>
int main() {
    auto r = std::ranges::find(std::vector{1, 2, 3}, 2);          // fordul
    static_assert(std::is_same_v<decltype(r), std::ranges::dangling>);
    std::vector v{1, 2, 3};
    auto it = std::ranges::find(v, 2);                             // valódi
     ↪ iterátor
    static_assert(std::is_same_v<decltype(it), std::vector<int>::iterator>);
#ifdef DEREF
    return *r;                                                     // hiba:
     ↪ dangling nem dereferálható
#endif
    return *it == 2 ? 0 : 1;
}
```

Buildjegyzőkönyv (`lab-gcc` profil). A program lefordult és 0 kóddal futott, a két `static_assert` teljesült; `-DDEREF` mellett a `*r no match for 'operator*' (operand type is 'std::ranges::dangling')` hibát adott.

```
// borrowed_cases.cpp - C++20: mikor ad vissza a ranges::find iterátort, és
 ↪ mikor dangling-et?
#include <algorithm>
#include <ranges>
#include <span>
#include <type_traits>
#include <vector>

std::vector<int> make() { return {1, 2, 3}; }
inline auto odd = [](int x) { return x % 2 != 0; };

int main() {
    std::vector<int> v{1, 2, 3};
    auto fv = v | std::views::filter(odd);

    using std::ranges::dangling;
```

```
    // span prvalue: az iterátor a v puffere, nem a span objektum függvénye
    static_assert(!std::is_same_v<decltype(std::ranges::find(std::span<int>(v),
     ↪  2)), dangling>);
    // vector& (lvalue): az objektum a hívón kívül él tovább
    static_assert(!std::is_same_v<decltype(std::ranges::find(v, 2)), dangling>);
    // vector prvalue: az iterátor a megszűnő temporary pufferébe mutatna
    static_assert(std::is_same_v<decltype(std::ranges::find(make(), 2)),
     ↪  dangling>);
    // filter_view prvalue: az iterátor a view-ra (predikátum, cache) is
     ↪  hivatkozik
    static_assert(std::is_same_v<decltype(std::ranges::find(v |
     ↪  std::views::filter(odd), 3)), dangling>);
    // filter_view& (lvalue): a hívó view-ja tovább él - de csak amíg fv él
    static_assert(!std::is_same_v<decltype(std::ranges::find(fv, 3)),
     ↪  dangling>);
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic borrowed_cases.cpp` figyelmeztetés nélkül fordul: a span, a `vector&` és a `filter_view&` esetén a `ranges::find` iterátort, a prvalue vektor és a prvalue `filter_view` esetén `std::ranges::dangling`-et ad vissza.

Saját típusra akkor érdemes `enable_borrowed_range` specializációt adni, ha az iterátor tényleg független a view objektumtól:

```
struct raw_span {
    const int* data;
    std::size_t size;
    const int* begin() const noexcept { return data; }
    const int* end()   const noexcept { return data + size; }
};
namespace std::ranges {
    inline constexpr bool enable_borrowed_range<raw_span> = true;
}
```

Ez akkor helyes, ha belső iterátor-típusunk (`const int*`) érvényes marad a `raw_span` megszűnése után is. Ha bármit is a `raw_span` objektum tárol, amire az iterátor hivatkozik, akkor ez a specializáció *hibás* — és ezt a standard nem tudja kikényszeríteni, ez diszciplína kérdése.

30.5.3 `std::ranges::dangling` és biztonságos visszatérés

A range-algoritmusok visszatérési típusa a `borrowed_iterator_t<R>` alias, amit a standard így definiál:

$$\text{borrowed_iterator_t}(R) = \begin{cases} \text{iterator_t}(R), & \text{ha borrowed_range}(R), \\ \text{std::ranges::dangling}, & \text{egyébként.} \end{cases}$$

A `std::ranges::dangling` egy üres, csaknem viselkedés nélküli marker típus:

```
namespace std::ranges {
struct dangling {
    constexpr dangling() noexcept = default;
```

```
    template<class... Args>
    constexpr dangling(Args&&...) noexcept {}
};
}
```

Mivel nem iterator, nem dereferálható, nem léptethető. Ha tehát a hívó kód megpróbálja használni a visszatérési értéket, a hiba *fordítási időben* derül ki, nem pedig futásidőben csendes UB formájában.

```
#include <algorithm>
#include <iostream>
#include <ranges>
#include <string>
#include <string_view>
#include <type_traits>
#include <vector>

int main() {
    std::vector<int> v{1, 2, 3, 4, 5};

    // Lvalue tartomány: a visszatérés std::vector<int>::iterator
    auto it = std::ranges::find(v, 3);
    std::cout << "found (lvalue): " << *it << '\n';

    // Temporális tartomány: a visszatérés std::ranges::dangling
    auto d = std::ranges::find(std::vector{1, 2, 3}, 2);
    static_assert(std::is_same_v<decltype(d), std::ranges::dangling>);
    // *d;  // fordítási hiba

    // filter_view lvalue változóban: érvényes, amíg evens él
    auto evens = v | std::views::filter([](int x) { return x % 2 == 0; });
    for (int e : evens) std::cout << e << ' ';
    std::cout << '\n';

    // filter_view temporális: a .begin() iterátora danglel
    auto bad = (v | std::views::filter([](int x) { return x > 3; })).begin();
    (void)bad;  // ÉRVÉNYTELEN, ha használnánk: UB
    return 0;
}
```

Fordítás (a laborhoz rögzített GCC/Clang és libstdc++/libc++ verzióval):

```
g++ -std=c++20 -O0 -Wall -Wextra -o dangling_demo dangling_demo.cpp
```

A `static_assert` és a kommentelt dereferencia miatt a program *fordul és lefut*, de a 4. eset `bad` változója `UB`, ha használjuk. A program kimenete nem mutatja meg a hibát — ez a lényeg: a `dangling` marker csak a `std::ranges` algoritmusok egy részére érvényes, a *tag-* és *member-*`begin()` hívásokon keresztül vezető útvonalakra nem.

30.5.4 View-tól függő iterátorok: `filter_view` és `transform_view`

A `filter_view::iterator` és a `transform_view::iterator` a standardban deklarált módon egy `parent_` pointert tartalmaz az aktuális view objektumra. Ezt nem cache-ként vagy kényelmi

okból tárolja, hanem mert a filter predikátum és a transform projekció *abban a view objektumban* van, ahol az argumentumokat is átadták — a view az owner. Emiatt:

- Ha a `filter_view` / `transform_view` temporális, a `.begin()` után azonnal danglel az iterátor.
- Ha a view változóban él, az iterátor addig érvényes, amíg a view objektum életben van.
- A `borrowed_range` ezt nem feltétlenül jelzi, ha a hívó lvalue referenciát ad át.

A `span` és a `string_view` más a helyzet: belső iterátoraik csak a backing storage-ra mutatnak, nincs `parent_` pointer, ezért ezek a view objektum megszűnése után is biztonságosak, feltéve, hogy a backing storage él. A `subrange<I, S>` viszont csak azt a függőséget szünteti meg, ami a *subrange objektumhoz* kötött; az `I` iterátortípus saját függőségei megmaradnak. Egy `subrange`, amelyet `filter_view`-ból vett `begin()`/`end()` iterátorokból építettünk, továbbra is függ a forrás `filter_view` (és a benne tárolt predikátum/gyorsítótár) élettartamától: a `subrange` túlélése nem elégséges biztonsági feltétel a forrás `filter_view` megszűnése után. A `ref_view<R>` is ebbe a kategóriába tartozik; az `owning_view<R>` saját maga az owner, és a `borrowed_range` tulajdonsága az `R`-ét követi (az `enable_borrowed_range<owning_view<R>>` az `R` értékét veszi át): `owning_view<std::vector<int>>` nem borrowed (a vector nem az), `owning_view<std::span<int>>` viszont igen (GCC 13.3, `static_assert`-tel igazolva).

30.5.5 Temporális tartományok és a view escape csapda

A klasszikus példa `string_view`-re:

```
std::string_view sv = std::string{"hello"};  // a temporary string a sor végén
 ↪  megszűnik
std::size_t n = sv.size();                   // a view saját mezőit olvassa: nem
 ↪  a lejárt tárolót (önmagában nem UB)
char c = sv[0];                              // UB: a lejárt háttértároló
 ↪  karakterének olvasása
```

A `std::string` ideiglenes, a `string_view` egy pointert és egy hosszt tárol. A temporary élettartama a teljes deklarációs utasítás végéig tart, a `string_view` változó tovább él — de a mögötte lévő storage felszabadult.

Ugyanez a hiba view-okkal, csak rejtettebb formában:

```
// HIBÁS: temporális filter_view, .begin() után danglel
auto it = (std::vector<int>{1, 2, 3, 4, 5} | std::views::filter([](int x) {
 ↪  return x > 2; })).begin();
```

A kifejezés típusa `filter_view<owning_view<vector<int>>, lambda>`. A `.begin()` működése után a temporális view (és benne az `owning_view` által birtokolt vector) elpusztul, az iterátor danglel. Ezért a láncot mindig **nevesített változóba** kell kötni, ha az iterátor vagy a view tovább él, mint az utasítás:

```
// HELYES: a view túléli az iterátor használatát
auto data = std::vector<int>{1, 2, 3, 4, 5};
auto view = data | std::views::filter([](int x) { return x > 2; });
for (int x : view) { /* ... */ }
```

Méret és tartalom külön kérdés. A `string_view::size()` és `empty()` a view-ban tárolt hosszt adja, a háttérstringet nem dereferálja; élő view objektumon önálló lekérdezésük megengedett. A lejárt háttértároló **karaktereinek elérése** (`sv[0]`, `data()` dereferálása, iterálás) az UB. A feladatmegoldás tehát engedje az önálló `size`/`empty` lekérdezést, és tiltsa a lejárt karakterek olvasását.

A gyakori counter-tipp a `const auto&`. A `filter_view` `begin()`-je nem `const`, így `const auto& view = v | views::filter(...)` után `view.begin()` fordítási hibát ad, nem pedig élettartam-hosszabbítást. Ezt sokan félreértik: ilyenkor a view objektum élettartama valóban meghosszabbodik (mert `const&` kötés történt), de iterálni nem lehet. A `transform_view` viszont feltételekhez kötött `const begin()/end()` felületet is ad: const-invocable (tiszta) lambdával és const-iterálható forráson a `const auto& tr = v | views::transform(pure_lambda);` leforduló, iterálható példa (GCC 13.3-mal `static_assert`-tel igazolva), tehát a `filter_view` viselkedését nem szabad a `transform_view`-ra általánosítani. A `filter`-hez a helyes megoldás a nem-const nevesített változó, vagy a backing storage és a view külön kezelése.

30.5.6 Mutáció, újraallokáció és pipeline invalidáció

A backing storage mutációja is danglelő állapotot teremt, akkor is, ha a view objektum végig él. A `std::vector` invalidációja **a ténylegesen bekövetkezett műveletből** következik, nem a művelet nevéből:

Művelet	Mikor érvényteleníti	Mi érvénytelen
`push_back`/`emplace_back`	ha újraallokáció történt (`size() == capacity()`)	újraallokációnál minden iterátor/pointer/referencia; különben csak az `end()` (és a past-the-end)
`reserve(n)`	ha `n > capacity()` (újraallokál)	újraallokációnál minden; `n <= capacity()` esetén semmi
`insert`/`emplace`	pozíciófüggő	újraallokációnál minden; különben a beszúrási ponttól kezdve minden iterátor/referencia (és `end()`)
`erase`	pozíciófüggő	a törölt ponttól kezdve az iterátorok/referenciák (és `end()`)
`resize`	ha nő és újraallokál; csökkenésnél a törölt elemek	újraallokációnál minden
`clear`	mindig	minden elem iterátora/referenciája (a kapacitás megmarad, `end()` is érvénytelen)

Például `reserve(4)` után a kapacitáson belüli `push_back` az első elem pointerét érvényesen hagyja (`assert(p == &w[0])`, GCC 13.3). A `filter_view` ezen felül a `begin()` keresésének eredményét (az első egyező elem pozícióját) **a view objektumban cache-eli, forward range esetén**; input-only range-nél nincs ilyen cache (egyszeri bejárás). A cache a forrás mutációjakor nem frissül: tárolómutáció (újraallokáció, a cache-elt pozíció előtti beszúrás/törlés) után a cache-elt pozíció érvénytelen lehet, és a predikátum igazságértékét megváltoztató elemírás után a cache-elt „első egyező” már nem feltétlenül az első egyező. Mutáció után a view-t újra kell építeni. (A filter-iterátor `parent_` pointere a view-ra mutat, nem külön „first-match” cache-t tárol.)

```cpp
// HIBÁS: iterálás közben növeljük a vektort
std::vector<int> v{1, 2, 3, 4, 5};
auto evens = v | std::views::filter([](int x) { return x % 2 == 0; });
for (int e : evens) {
    if (e == 2) {
        v.push_back(4);  // UB: az iterátor a régi pufferre mutat
    }
}
```

A helyes minta: Ha a ciklus közben módosítani kell a tárolót, az iterációt egy pillanatkép-példányon (`std::vector` másolatán) vagy egy `for (auto it = ...)` kézi cikluson végezzük, ahol a mutáció előtt megszűnik az iterátorok élettartama. Elem-módosítás (`v[i] = ...`) *nem* érvényteleníti az iterátorokat, de a `filter_view` első-match logikájával ütközhet, ha a módosítás megváltoztatja a predikátum igazságértékét — ezt a `filter_view` nem ellenőrzi, mert nincs cache-elt állapota. Ez a viselkedés a Lazy view pipeline és mellékhatások (30.4) fejezetben tárgyalt „lazy nem egyenlő cache-elt” elv közvetlen folyománya.

30.5.7 Gyakori hibák és javításuk

- `auto it = (temporary_range | filter(...)).begin();` → a temporális view és a backing storage is danglel. **Javítás:** nevesített változóba kösd a view-t, vagy `owning_view`-ként tárold a tárolót.
- `const auto& view = v | std::views::filter(pred);` → nem iterálható, mert a `filter_view::begin()` nem `const`. **Javítás:** `auto view = ...;`.
- `auto it = std::ranges::find(std::vector{1, 2, 3}, 2);` → `std::ranges::dangling`, fordítási hiba dereferáláskor. **Javítás:** nevezd el a vector-t.
- `borrowed_range` specializáció olyan típusra, aminek az iterátora a view-tól függ. **Javítás:** elhagyni a specializációt.
- Tároló növelése iterálás közben `filter_view`-n. **Javítás:** pillanatkép (v másolata), vagy a mutációt a cikluson kívül elvégezni.
- `std::ranges::find` visszatérési értékét `auto&&`-dzsel vagy `decltype(auto)`-tal továbbadni — ez `dangling` esetén sem segít, mert a típus már `std::ranges::dangling`.

30.5.8 Feladatok

30.5.8.1 1. feladat (azonosítás)

Az alábbi kódban nevezd meg minden érintett entitás (backing storage, view objektum, iterator) élettartamát, és jelöld, hol danglel:

```cpp
#include <ranges>
#include <string>
#include <string_view>
#include <vector>

int main() {
    auto sv = std::string_view{std::string{"temp"}};
    auto it = std::ranges::find(std::vector{1, 2, 3}, 2);
    return static_cast<int>(sv.size());
}
```

30.5.8.2 2. feladat (javítás)

Az alábbi függvény visszaad egy iterátort, de a hívó oldalon danglel. Javítsd `borrowed_iterator_t` segítségével úgy, hogy a hibás hívás fordítási hibát okozzon:

```
#include <ranges>
#include <vector>

auto find_three(const std::vector<int>& v) {
    return std::ranges::find(v, 3);
}
```

30.5.8.3 3. feladat (diagnosztika)

Magyarázd meg, miért ad a `decltype(v | std::views::filter(pred))&` típus (vagy az azzal egyenértékű `std::ranges::filter_view<std::ranges::ref_view<std::vector<int>>, Pred>&`) lvalue referenciára `borrowed_range = true`-t, miközben az iterátor a view objektum megszűnése után danglel. (A `std::vector<int>&` nem view, ezért nem lehet a `filter_view V` paramétere; a típusaliasnak önmagában le kell fordulnia, és csak utána jöhet a `static_assert(borrowed_range<...>)`.) Milyen esetekre terjed ki a `borrowed_range` védelme?

30.5.8.4 4. feladat (tervezés, nehezebb)

Írj egy `first_over(std::vector<int> const& v, int threshold)` függvényt, amely a *tárolót* nem másolja, de az eredményt úgy adja vissza, hogy a hívó tudja: (a) az iterátor a backing storage-ra mutat, (b) a hívó felelőssége életben tartani a vector-t. Használd a `std::ranges::borrowed_iterator_t`-et a visszatérési típusban, és egészítsd ki `static_assert`-tel, ami ellenőrzi a szerződést.

30.5.9 Megoldások

30.5.9.1 1. feladat megoldása

Az `sv` mögötti backing storage a `std::string{"temp"}` ideiglenes, ami a sor végén megszűnik; ekkor a `std::string_view` danglel. Az `sv.size()` önmagában csak a view tárolt hosszát adja, a lejárt karakterek olvasása (`sv[0]`, iterálás) az UB. A `std::vector{1, 2, 3}` temporális, a `std::ranges::find` erre hívva `std::ranges::dangling`-et ad vissza, így az eredmény nem is dereferálható. Az `it` tehát nem iterator, hanem marker; a program `it` felhasználása nélkül lefut, de a `sv.size()` hívás önmagában UB. Helyes változat: `std::string owner{"temp"}; std::string_view sv = owner;` illetve `std::vector<int> v{1, 2, 3}; auto it = std::ranges::find(v, 2);`. Ellenőrzési feltétel: a `std::is_same_v<decltype(it), std::ranges::dangling>` igazsága a javított változatban hamis.

30.5.9.2 2. feladat megoldása

A hiba forrása, hogy a visszatérési típus `std::vector<int>::const_iterator`, de a `find` a `v` referencián dolgozik, aminek az élettartama a hívótól függ. Ha a hívó egy temporálist ad át, a `v` megszűnik, mire a hívó használja az iterátort. A `borrowed_iterator_t` megoldás:

```
#include <algorithm>
#include <ranges>
#include <type_traits>
#include <vector>

template <std::ranges::input_range R>
auto find_three(R&& r) -> std::ranges::borrowed_iterator_t<R> {
    return std::ranges::find(std::forward<R>(r), 3);
}
```

A `borrowed_iterator_t<R>` automatikusan `std::ranges::dangling`-re vált, ha `R` nem borrowed. Így a `find_three(std::vector<int>{1, 2, 3})` hívás `dangling`-et ad, a `*` dereferálás fordítási hiba. Ellenőrzési feltétel: `static_assert(std::is_same_v<decltype(find_three(std::vector<int>{})), std::ranges::dangling>);`.

30.5.9.3 3. feladat megoldása

A `borrowed_range` lvalue-re vonatkozó ága azt a helyzetet fedi le, amikor a range kifejezés típusa referencia — az iterátor és a range objektum között nincs *kötelező* tulajdonosi viszony. Ezt használja ki a `filter_view`, amikor lvalue-re hívjuk: az iterátor a háttértárolóra mutat, de a filter-állapotot a view-ból olvassa. A `borrowed_range` csak a *rvalue* esetre ad „védelmet” azzal, hogy a `std::ranges` algoritmusok `dangling`-et adnak vissza. A lvalue `filter_view` esetén a compiler nem tudja észrevenni, hogy a view később megszűnik — ezt a programozónak kell életben tartania. A `borrowed_range` tehát **nem** véd az „eltűnt lvalue view” hibaosztály ellen; csak az „azonnal megszűnő temporális tartomány” hibáját fogja meg. Ez a hatókör-korlát fontos: ne tekintsük a `borrowed_range`-et teljes élettartam-garanciának.

30.5.9.4 4. feladat megoldása

```
#include <algorithm>
#include <ranges>
#include <type_traits>
#include <vector>

auto first_over(std::vector<int> const& v, int threshold)
    -> std::ranges::borrowed_iterator_t<std::vector<int> const&>
{
    return std::ranges::find_if(v, [threshold](int x) { return x > threshold;
     ↪  });
}

static_assert(std::is_same_v<
    decltype(first_over(std::declval<std::vector<int> const&>(), 0)),
    std::vector<int>::const_iterator>);
```

Mivel a `const&` lvalue, a `borrowed_range` igaz, a visszatérés `const_iterator`. **Fontos korlát:** a `const std::vector<int>&` paraméter *temporálist is elfogad* (a `const&` rvalue-ra köthető), a `borrowed_iterator_t<const std::vector<int>&>` pedig ugyanaz a `const_iterator`, ezért a visszatérési típus **nem tiltja** a temporális hívást: `first_over(std::vector<int>{1, 5}, 3)` fordul, és dangling iterátort ad. A fordítási idejű tiltáshoz külön kell tiltani az rvalue-t, például törölt túlterheléssel:

```
auto first_over(std::vector<int>&&, int) -> void = delete;    // temporális
↪  tároló: tiltott
```

(vagy sablonos, forwarding referenciás változattal, amelynek visszatérési típusa `borrowed_iterator_t<R>`, ahogy a 2. feladatnál). A szerződés a hívó felé: a visszaadott iterátor a `v` backing storage-ára mutat, ezért `v`-nek élnie kell, amíg az iterátort használjuk; ezt a dokumentáció rögzítse. Ellenőrzési feltétel: a `static_assert` a `const_iterator`-t várja, nem `dangling`-et. Ha módosítanánk a függvényt `std::vector<int>` (nem referencia) paraméterre, a `static_assert` átállna `std::ranges::dangling`-re — ez jelzi, hogy a szerződés megsértődött.

30.5.10 Összefoglaló

A ranges-alapú kód három, egymástól független élettartamot ölel fel: a backing storage-ot (owner), a view objektumot (nem birtokló leíró), és az iterator/predikátum kapcsolatot (függő entitás). A `std::ranges::borrowed_range` csak a range objektum típusára tesz állítást, és csak az „ideiglenes tartomány” hibaosztályt fogja meg — a `filter_view` és `transform_view` iterátorai a view objektumra mutató pointert is tárolnak, ezért ezek lvalue formában is danglelnek, ha a view megszűnik. A `std::ranges::dangling` marker a fordítót használja ki: a nem-borrowed hívások eredményét típus szinten használhatatlanná teszi. A biztonságos minta: a backing storage-ot külön nevesített ownerbe tenni, a view-t nem-const nevesített változóba kötni, és a `filter_view`/`transform_view` iterátorait csak a view élettartamán belül használni. A `borrowed_range` nem fogja meg a view objektum eltűnését — ezért a `filter`- és `transform`-láncok szerződését mindig a *tároló* → *view* → *iterator* sorrendben kell ellenőrizni.

30.6 Ranges- és callback-élettartam labor

A labor két, a gyakorlatban gyakran összekapcsolódó szerződést vizsgál: a lazy ranges pipeline és a callable-ek élettartamát. A cél nem az, hogy a lazy view-t automatikusan biztonságosnak vagy cache-eltnek tekintsük, hanem hogy minden példában megnevezzük a tulajdonost, a view élettartamhatárát, az invalidáló műveletet és a hiba megjelenési formáját. Az előfeltételek: a borrowed range és a dangling részletes tárgyalása a Borrowed range, dangling és iterator lifetime (30.5) részben, a callable és type erasure alapjai az invoke, function wrapper és type erasure (30.2) részben, az ASan és társai az ASan, UBSan, MSan, TSan és Valgrind (16.2) részben találhatók. A fejezet a C++20 alapra épít; a C++23-as lehetőségeket külön jelöljük, és a választott compiler/library feature-testjével kell ellenőrizni őket. A kódokra a fejezet nem állítja, hogy lefuttattuk őket: a várt kimenet, a mérési terv és az ASan-lelet a tanuló laborjához tartozik.

30.6.1 Fogalmak és szerződések

A **lazy view** olyan range, amely nem tárolja az elemeit, hanem a mögöttes range-ből számítja ki őket; a `begin()` és `end()` által visszaadott iterátorok az inputtól függenek. A **filter_view** és a **transform_view** C++20-ban nem tulajdonolják a mögöttes tartományt, kivéve, ha az egy

owning_view. A `std::views::all` lvalue range-re `ref_view`-ot, rvalue range-re `owning_view`-ot ad; a `ref_view` nem tartja életben a mögöttes objektumot, az `owning_view` igen.

A **borrowed_range** olyan range, amelynek iterátorai a range objektum élettartamától függetlenül érvényesek maradhatnak. A `std::string_view` és a `std::span` jellemzően borrowed range, a `std::vector` nem. Ez azonban nem azonos a tulajdonlással: a `std::vector` nem borrowed range, mégis tulajdonol, ezért a belőle materializált kimenet biztonságos. A **dangling** olyan pointer, referencia vagy view, amelynek hivatkozott objektuma már nem él; a használata undefined behavior (UB).

A **materialization** (anyagiasítás) a lazy view elemeinek kiértékelése és egy tulajdonló konténerbe másolása vagy mozgatása. A **callable** (hívható objektum) `std::invoke`-bal hívható dolog: lambda closure, függvénypointer, tagfüggvénypointer vagy függvényobjektum. A closure **copyable** vagy **move-only** volta a capture-öktől függ; a `std::unique_ptr` capture move-only closure-t eredményez. A **capture** (befogás) lehet érték szerinti, referencia szerinti vagy init-capture; referencia szerint nem hosszabbítja meg a temporary élettartamát.

A labor központi szerződése táblázatban:

Objektum	Tulajdonol?	Élettartam	Invalidálja
`std::vector<LogRecord> logs`	igen	a `logs` scope-ja	megszűnés, reallokáció
`error_view(logs)`	nem	a `logs`-hoz kötött	a `logs` megszűnése vagy módosítása
`materialize_errors(logs)` visszatérési értéke	igen	a visszaadott vector	a visszaadott vector megszűnése
`std::string_view` a `logs` elemére	nem	a `logs` elemhez kötött	az elem megszűnése vagy módosítása

A szerződés kimondása önmagában nem véd meg: a `[[nodiscard]]` és a `const` nem akadályozza meg a danglinget. A `static_assert` a `std::ranges::borrowed_range` concepttel dokumentálhatja, hogy egy visszaadott view nem borrowed range, de a helyes használat továbbra is a hívó felelőssége.

30.6.2 Logrekord pipeline

A példa egy naplófájl-szerű rekordhalmazt szűr és vetít. A `LogRecord` tulajdonosa a `std::vector`; a pipeline a vektorra hivatkozik. A `filter` predikátuma a rekord szintjét vizsgálja, a `transform` a `message` mezőre vetít. A vetítés a ranges algoritmusok `projection` paraméterének mintájára működik: a `std::ranges::sort` és a `std::ranges::find_if` is kaphat tagfüggvény-pointert vagy lambdát.

```
#include <algorithm>
#include <chrono>
#include <iostream>
#include <iterator>
#include <ranges>
#include <string>
#include <vector>

struct LogRecord {
```

```
    std::chrono::system_clock::time_point ts;
    std::string level;
    std::string message;
};

std::vector<LogRecord> load_demo_logs() {
    using namespace std::chrono_literals;
    const auto base = std::chrono::system_clock::now();
    return {
        {base + 1s, "INFO",  "service started"},
        {base + 2s, "ERROR", "connection refused"},
        {base + 3s, "WARN",  "retrying"},
        {base + 4s, "ERROR", "timeout"},
    };
}

std::vector<std::string> materialize_errors(const std::vector<LogRecord>& logs)
 ↪  {
    auto view = logs
        | std::views::filter([](const LogRecord& r) { return r.level == "ERROR";
         ↪  })
        | std::views::transform([](const LogRecord& r) -> const std::string& {
              return r.message;
          });

    std::vector<std::string> out;
    out.reserve(logs.size());
    std::ranges::copy(view, std::back_inserter(out));
    return out;
}

auto error_view(const std::vector<LogRecord>& logs) {
    return logs
        | std::views::filter([](const LogRecord& r) { return r.level == "ERROR";
         ↪  })
        | std::views::transform([](const LogRecord& r) -> const std::string& {
              return r.message;
          });
}

int main() {
    auto logs = load_demo_logs();

    for (const auto& msg : materialize_errors(logs)) {
        std::cout << "materialized: " << msg << '\n';
    }

    auto view = error_view(logs);
    for (const std::string& msg : view) {
        std::cout << "lazy: " << msg << '\n';
    }
}
```

A `materialize_errors` visszatérési értéke tulajdonol: a `std::string` elemek másolatai a `logs`-tól függetlenül élnek. Az `error_view` ezzel szemben nem tulajdonol: a visszaadott view a `logs`-ra hivatkozik, ezért a hívónak a view használatáig életben kell tartania a vektort. A `std::ranges::copy` azért kell, mert a `filter_view` `begin()` és `end()` típusa eltérhet (a sentinel `std::default_sentinel_t` lehet), a C++20-as `std::vector(first, last)` pedig azonos iterator típust vár. A C++23-as `std::ranges::to` és a `std::vector` range-konstruktora ezt kényelmesebben kezelné, de ezek C++23-as lehetőségek.

A buildparancs a laborban például:

```
g++ -std=c++20 -O2 -Wall -Wextra -Wpedantic -g loglab.cpp -o loglab
./loglab
```

A várt kimenet az `ERROR` szintű üzeneteket tartalmazza, de ezt a fejezet nem mérési eredményként, hanem elvárásként rögzíti. A `sort` projection használata külön ellenőrizhető:

```
std::ranges::sort(logs, std::ranges::less{}, &LogRecord::level);
auto it = std::ranges::find_if(
    logs,
    [](const std::string& level) { return level == "ERROR"; },
    &LogRecord::level);
```

A projection itt a `&LogRecord::level`; a `find_if` predikátuma már a vetített értéket kapja.

30.6.3 Temporary- és capture-hibák izolált negatív példaként

A következő két példa szándékosan hibás; a fejezet nem futtatja őket, és a tanuló laborjában is csak izolált, jelölt kísérletként használhatók.

```
// NEGATÍV PÉLDA: dangling view lokális vectorra. Ne futtasd!
auto bad_error_view() {
    auto logs = load_demo_logs();
    return logs
        | std::views::filter([](const LogRecord& r) { return r.level == "ERROR";
        ↪ })
        | std::views::transform([](const LogRecord& r) -> const std::string& {
            return r.message;
          });
}
```

A `logs` lokális `std::vector`, a `views::all` lvalue-ra `ref_view`-ot ad, a `filter_view` és a `transform_view` ezt tárolja. A függvényből való kilépéskor a `logs` megszűnik; a visszaadott view dangling. A javítás lehet a `std::move(logs)` használata a pipe-ban, ami C++20-ban `owning_view`-ot eredményez, vagy a materialization, vagy a vector kívülről való átadása.

```
// NEGATÍV PÉLDA: temporaryre mutató referencia capture. Ne futtasd!
auto make_level_filter(const std::string& level) {
    return [&level](const LogRecord& r) { return r.level == level; };
}

auto f = make_level_filter("ERROR"); // a temporary a kifejezés végén megszűnik
```

A `"ERROR"`-ból ideiglenes `std::string` keletkezik, a `const std::string&` ehhez kötődik, a closure pedig referenciát capture-öl. A temporary a teljes kifejezés végén megszűnik, így `f` hívásakor a `level` dangling. A helyes változat érték szerint capture-öl:

```
auto make_level_filter(std::string level) {
    return [level = std::move(level)](const LogRecord& r) {
        return r.level == level;
    };
}
```

A closure a saját `std::string` másolatát tartalmazza, ezért a temporary megszűnése után is biztonságos, és a closure copyable marad. A hiba nyelvi oka a referencia élettartam-szerződés megsértése; az ASan a `stack-use-after-scope` vagy `heap-use-after-free` leletet adhatja, de a lelet típusa implementáció- és allokációfüggő, és a hiánya nem bizonyíték a helyességre.

30.6.4 Copy/move-only callable tárolása

A C++20-as `std::function` copy-constructible callable-t vár. Egy `std::unique_ptr`-t capture-ölő lambda move-only closure, ezért nem tehető `std::function`-ba:

```
#include <functional>
#include <memory>

auto make_counting_predicate() {
    auto count = std::make_unique<std::size_t>(0);
    return [count = std::move(count)](const LogRecord& r) {
        ++*count;
        return r.level == "ERROR";
    };
}

// C++20: std::function<bool(const LogRecord&)> f = make_counting_predicate();
 ↪  // hiba
```

A `std::function` példányosítása static_asserttel utasítja el a nem copy-constructible callable-t; a pontos üzenet implementációfüggő. C++20-ban két egyszerű megoldás van. Az egyik a template paraméter, amely nem tárolja el a callable-t, hanem átveszi:

```
template <class Pred>
std::vector<LogRecord> filter_logs(const std::vector<LogRecord>& logs, Pred
 ↪  pred) {
    std::vector<LogRecord> out;
    for (const auto& r : logs) {
        if (pred(r)) out.push_back(r);
    }
    return out;
}
```

A hívás `filter_logs(logs, std::move(pred))` formában move-only predikátummal is működik, mert a paraméter érték szerinti, és a függvényen belül nem kell másolni. A másik megoldás egy saját, move-only type-erasure wrapper `std::unique_ptr<Concept>`-tel; ez C++20-ban is megírható, de több kódot igényel. C++23-ban a `std::move_only_function` erre kész megoldás,

használatát azonban külön kell jelölni és a library supporttal ellenőrizni. A ranges adaptorok callable-követelménye adaptoronként és verziónként eltér, ezért nem általánosítható:

- **`views::filter`:** a predikátumnak `std::indirect_unary_predicate`-et kell modelleznie, amely `copy_constructible`-t is megkövetel — C++20-ban és C++23-ban is. Egy `unique_ptr`-t rögzítő predikátummal a view létre sem jön.
- **`views::transform`:** C++20-ban a függvényobjektumnak másolhatónak kell lennie; C++23-ban (P2494, `__cpp_lib_ranges >= 202207L`) elég a mozgathatóság.

A `filter`-hez ezért másolható predikátum kell. Ha az állapot move-only, a predikátumot egy nevesített, a view-nál tovább élő változóban tartjuk, és `std::ref`-fel adjuk át; a view így nem birtokolja az állapotot, csak hivatkozik rá:

```
auto limit = std::make_unique<int>(1);
auto pred = [l = std::move(limit)](int x) { return x > *l; };
auto r = v | std::views::filter(std::ref(pred));   // pred-nek túl kell élnie
 ↪  r-t
```

Buildjegyzőkönyv. A közvetlen `unique_ptr`-capture-ös predikátummal a `views::filter` a `lab-gcc` profilban (`-std=c++20` és `-std=c++23`) és Clang 22 / libc++ alatt (mindkét módban) is fordítási hibát adott; a `std::ref`-es változat mindenhol lefordult. A move-only `views::transform` Clang 22 / libc++ alatt `-std=c++23`-ban lefordult, a GCC 13.3 libstdc++-a (`__cpp_lib_ranges == 202202L`) viszont még elutasította, mert a P2494-et nem implementálja.

30.6.5 Materialization versus lazy költség

A lazy pipeline létrehozása nem járja be a tartományt, ezért $O(1)$; egy bejárás a filter predikátumát minden elemre meghívja, $O(n)$, a transform a kimeneti elemekre $O(k)$. A materializálás ugyanezt egyszer elvégzi, majd a kimenetet tárolja: $O(n)$ idő és $O(k)$ memória; a későbbi bejárás $O(k)$. Ha a pipeline-t többször járjuk be, a lazy változat minden alkalommal újraszűr, a materializált nem. A lazy view nem cache-el automatikusan; a „lazy” nem jelent sem biztonságot, sem gyorsítótárazást. A döntés: egyszeri bejárásnál és memóriaérzékeny esetben a lazy előnyös; többszöri bejárásnál vagy stabil kimenetnél a materialization a jobb.

A mérés a laborban történik, azonos inputtal, compilerrel és optimalizálási profillal. Vázlat:

```
template <class F>
double time_ms(F&& f) {
    const auto t0 = std::chrono::steady_clock::now();
    f();
    const auto t1 = std::chrono::steady_clock::now();
    return std::chrono::duration<double, std::milli>(t1 - t0).count();
}
```

A mért blokkban a lazy view-t és a materializált vektort ugyanannyiszor kell bejárni, és az eredményt el kell használni (például `volatile` összegzéssel vagy kiírással), hogy a compiler ne hagyja el a számítást. A mérés eredménye gép-, compiler- és optimalizálásfüggő; a fejezet nem állít konkrét számot.

30.6.6 ASan-lelet és nyelvi ok külön dokumentálása

A negatív példa izolált vizsgálata például:

```
g++ -std=c++20 -g -O1 -fsanitize=address -fno-omit-frame-pointer bad.cpp -o
 ↪  bad_asan
./bad_asan
```

Az ASan lelet lehet `heap-use-after-free` vagy `stack-use-after-return`; a riport a hibás hozzáférést és az érintett allokációt mutatja. A nyelvi ok ettől külön dokumentálandó: a `bad_error_view` egy `ref_view`-ot tartalmaz, amely a lokális `logs`-ra mutat; a `logs` élettartama a függvény végén véget ér, a view nem tulajdonol, ezért a használat UB. Az ASan egy implementációs diagnosztikai eszköz, nem a szabvány; nem jelez minden UB-t, és a „nincs lelet” nem bizonyítja a helyességet. A laborjegyzőkönyvben ezért külön rubrika legyen a leletre és külön a nyelvi élettartam-szerződésre.

30.6.7 Tipikus hibák

- A view-t lokális vektorra visszaadni, majd a hívóban bejárni.
- Referenciát capture-ölni olyan temporaryre, amely a hívási kifejezés végén megszűnik.
- `std::function`-ba move-only lambda-t tenni C++20-ban.
- A materializálást `std::vector(view.begin(), view.end())`-del próbálni, holott a sentinel típusa eltérhet; helyette `std::ranges::copy` kell.
- Azt hinni, hogy a lazy view cache-el, vagy hogy a lazy pipeline automatikusan biztonságos.
- Az ASan csendjét a helyesség bizonyítékának tekinteni.

30.6.8 Feladatok

30.6.8.1 Materializáló függvény

Adott a következő paraméter:

```
const std::vector<LogRecord>& logs
```

Írj függvényt, amely a `WARN` szintű rekordok üzeneteit `std::vector<std::string>` típusú értékként adja vissza. Magyarázd el, ki tulajdonol, és meddig él a belső view.

30.6.8.2 Érték szerinti capture

Írj `make_level_filter` függvényt, amely egy szintet capture-öl, és visszaad egy closure-t. Készíts hozzá egy negatív változatot is referencia szerinti capture-rel, és indokold a különbséget.

30.6.8.3 Move-only predikátum

Készíts olyan predikátumot, amely egy `std::unique_ptr<std::size_t>` számlálót tart. Tárold/Add át egy szűrőfüggvénynek C++20-ban. Miért nem megy `std::function`-nal, és mi a C++23-as lehetőség?

30.6.8.4 ASan és nyelvi ok

A `bad_error_view` példához írj egy rövid diagnosztikai jegyzőkönyvet: mi az ASan lelet, mi a nyelvi ok, és mi a javítás? Készíts olyan javított változatot, amely nem támaszkodik a lokális vektorra.

30.6.9 Megoldások

30.6.9.1 Materializáló függvény

A függvény a `logs`-ra hivatkozó `filter_view`-ot és `transform_view`-ot lokálisan használja, majd `std::ranges::copy`-val másol a kimeneti vektorba. A kimeneti vektor tulajdonol; a belső view élettartama a függvényhívásra korlátozódik, a `logs` a hívóé. Ellenőrzés: a visszaadott vektor elemei a `WARN` üzenetek, és a `logs` módosítása után a kimenet változatlan.

30.6.9.2 Érték szerinti capture

A helyes változat `std::string` paramétert és `[level = std::move(level)]` capture-t használ. A closure a saját másolatát birtokolja, ezért a hívási kifejezés temporaryje megszűnése után is jó. A negatív változat `const std::string&` paramétert és `[&level]` capture-t használ; a temporary a teljes kifejezés végén megszűnik, a closure dangling. Ellenőrzés: a helyes closure `"ERROR"` temporaryvel létrehozva is helyes eredményt ad; a negatív változat ASan alatt leletet adhat, de a nyelvi ok önmagában is UB.

30.6.9.3 Move-only predikátum

A `std::function` C++20-ban copy-constructible callable-t követel, a `unique_ptr`-t capture-ölő lambda nem az. A template paraméteres `filter_logs(logs, std::move(pred))` megoldás move-only predikátumot is átvesz, mert a paraméter érték szerinti és nem kell másolni. C++23-ban a `std::move_only_function` használható, de ezt jelölni kell és feature-testtel ellenőrizni. Ellenőrzés: a template-es változat fordul `-std=c++20`-szal; a `std::function`-ös változat fordítási hibát ad; a számláló a várt számot mutatja.

30.6.9.4 ASan és nyelvi ok

Az ASan lelet jellemzően `heap-use-after-free` vagy `stack-use-after-return` a visszaadott view bejárásakor. A nyelvi ok: a `ref_view` a lokális `logs`-ra mutat, a `logs` a függvény végén megszűnik, a view nem owning, ezért a hozzáférés UB. A javítás lehet a `std::move(logs)` a pipe-ban (`owning_view`), a materialization, vagy a vector kívülről való átadása. Ellenőrzés: a javított változat ASan alatt nem jelez; a következő `static_assert` dokumentálja, hogy a visszaadott view nem borrowed range.

```
static_assert(!std::ranges::borrowed_range<
    decltype(error_view(std::declval<const std::vector<LogRecord>&>())))>);
```

30.6.10 Összefoglaló

A lazy ranges pipeline nem tulajdonol, kivéve az `owning_view`-ot; a visszaadott view szerződése mindig kimondandó. A materialization tulajdonló kimenetet ad, és a többszöri bejárásnál cache-ként viselkedik, a lazy view nem. A capture élettartama a capture módjától függ; a referencia capture nem hosszabbítja meg a temporary élettartamát. A `std::function` C++20-ban copyable callable-t vár; move-only callable-hez template, saját wrapper vagy C++23-as `std::move_only_function` kell. Az ASan lelet és a nyelvi élettartam-ok külön dokumentálandó; a diagnosztika hiánya nem bizonyíték a helyességre.

31 Standard utility típusok és memória-view-k

31.1 optional, variant és visit

A hibakezelésnél (`std::expected`, `std::error_code`) megszokott minta, hogy a visszatérési érték hordozza a hibát is. Sok domain-állapot azonban nem hiba: egy konfigurációs kulcs lehet, hogy nincs beállítva; egy eseménynek lehet, hogy nincs payloadja; egy mérőszám lehet, hogy még nem érkezett meg. Ilyenkor a hibakód-payload erőltetése hazugság: olyan ágat kényszerítünk a hívóra, amelyre a domainben nincs szükség, és az `if (rc != 0)` ellenőrzések közé elrejtjük a valódi kérdést, hogy „van-e egyáltalán érték”. A standard utility típusok ezt a kérdést emelik a típusba: `std::optional` a jelenlétet, `std::variant` a zárt alternatívahalmazt modellezi, a `std::visit` pedig a feldolgozást köti az alternatívákhoz. A fejezet a döntési szempontokat, a lifetime-szerződést és a teljes lefedés kikényszerítését tárgyalja, összevetve a C tagged union megoldásával (Union, tagged union, enum és bit-field (9.3)).

A tárgyalt típusok C++17 óta a szabvány részei, a könyv profilja szerint C++20-at használunk. A fejezet forrásjegyzet nélkül készült: a konkrét szabványparagrafusokat, library-implementációs részleteket és a méretekre vonatkozó állításokat a saját laborprofilodban (compiler- és libstdc++/libc++ verzió) ellenőrizd, mielőtt egy viselkedésre építesz.

31.1.1 Optional: a jelenlét mint állapot

A `std::optional<T>` (header: `<optional>`) olyan értéktípus, amely vagy tartalmaz egy `T` típusú objektumot, vagy nem tartalmaz semmit. Előfeltételek: `T` objektumtípus, nem referencia és nem tömb (`std::optional<T&>` nem szabványos), és a használat pontján teljes típus. A tároló az `optional` objektumon belül helyezkedik el, tehát a műveletek nem allokálnak.

A fontos fogalom az **engaged/disengaged** állapot: engaged esetben az `optional` belsejében él egy `T` objektum, amelynek megvan a maga élettartama; disengaged esetben nem létezik `T` objektum, csak a tároló. Ez nem ugyanaz, mint „a `T` értéke nulla” vagy „a pointer null”: itt ténylegesen nincs objektum, ezért destruktort sem hívunk rá. Az élettartam-ablakot három művelet mozgatja: a konstrukció/értékadás engage-el, az `emplace(args...)` a régi értéket elpusztítja és egy újat konstruál a helyén, a `reset()` pedig elpusztítja az értéket és disengaged állapotba visz. Ha az `emplace` során a `T` konstruktora kivételt dob, az `optional` garantáltan üresen marad — ez az a pont, ahol a lifetime-szerződés megfogható.

Olvasáskor három út van: `has_value()`/`operator bool` ellenőrzés után `operator*` (nem ellenőriz, üresen **nem definiált viselkedés**), a `value()` (üresen `std::bad_variant_access` helyett `std::bad_optional_access`-t dob, kivétel nélküli buildben jellemzően leállítja a programot), illetve a `value_or(default)`, amely üresen a megadott alapértéket adja vissza. Az `operator*` csak ott indokolt, ahol a jelenlétet közvetlenül előtte már bizonyítottuk.

Levezetés-vázlat a méretre. Ha `T` értékkészletének reprezentálásához k bit kell, az `optional<T>`-nek legalább $2^k + 1$ állapotot kell megkülönböztetnie: a 2^k lehetséges `T`-értéket és a „nincs érték” állapotot. Ezért ha a `T` reprezentációjában nincs kihasználatlan bit, a `sizeof(optional<T>) > sizeof(T)` szükségszerűen teljesül. A gyakorlatban a könyvtárak külön jelzőt használnak, így

`sizeof(std::optional<char>)` jellemzően 2 — ez azonban implementációfüggő, mérés kérdése, nem hordozható állítás.

Az alábbi példa egy névfeloldást mutat, ahol a „nincs ilyen bejegyzés” nem hiba, hanem a domain bezárható esete. Fordítás: `g++ -std=c++20 -Wall -Wextra -pedantic -O2 -o optional_demo optional_demo.cpp`.

```
// optional_demo.cpp -- ISO C++20
#include <iostream>
#include <optional>
#include <string>

struct Endpoint {
    std::string host;
    unsigned short port;
};

// 0 vagy 1 érték; a "miért nincs" nem része a szerződésnek.
std::optional<Endpoint> lookup_endpoint(const std::string& name) {
    if (name == "api") return Endpoint{"api.internal", 8443};
    if (name == "db")  return Endpoint{"db.internal", 5432};
    return std::nullopt;
}

int main() {
    if (auto ep = lookup_endpoint("api")) {       // kontextuális bool-konverzió
        std::cout << ep->host << ':' << ep->port << '\n';
    }

    std::optional<Endpoint> slot;                 // disengaged: nincs Endpoint
    slot.emplace(Endpoint{"cache", 6379});        // itt kezdődik az élettartam
    std::cout << slot->host << '\n';              // bizonyított jelenlét
    slot.reset();                                 // itt véget ér
    std::cout << slot.has_value() << '\n';        // 0

    Endpoint fallback{"none", 0};
    std::cout << slot.value_or(fallback).port << '\n';  // 0
}
```

A gyakori félreértés, hogy az `optional` hibacsatorna. Ha a hívónak tudnia kell, *miért* nincs érték (érvénytelen bemenet, hiányzó fájl, időtúllépés), akkor nem `optional<T>`, hanem `expected<T, E>` vagy `error_code` a helyes választás (error_code, system_error és expected (25.4)). Ugyanígy antipattern a `std::optional<std::optional<T>>` és az `optional` függvényparaméterként „opcionális argumentum” gyanánt: utóbbira általában túlterhelés vagy dokumentált pointer-szerződés a jobb megoldás, mert a hívás helyén nem derül ki, hogy a `nullopt` mit jelent.

31.1.2 Variant: az aktív alternatíva

A `std::variant<Ts...>` (header: `<variant>`) zárt alternatívahalmazt modellez: az objektum minden pillanatban **pontosan egy** `Ts` közül tartalmaz értéket (vagy a `valueless_by_exception` állapotban van, lásd később). Az alternatívák objektumtípusok; referencia nem lehet köztük. A

típuslista lezárt: nem bővíthető futásidőben, ezért ez a fordítási idejű megfelelője annak, amit C-ben kézi `enum` taggel és `union`-nal írunk le.

Az aktív alternatívát az `index()` árulja el (`std::size_t`), a hozzáférést a `std::get<I>(v)`, a `std::get<T>(v)` (rossz esetben `std::bad_variant_access`), illetve a nem dobó `std::get_if<I>(&v)` / `std::get_if<T>(&v)` adja, amely üres pointert ad vissza, ha nem az adott alternatíva aktív. A `std::holds_alternative<T>(v)` ugyanezt bool-ként kérdezi. Ha egy típus többször szerepel a listában, a `get<T>` nem használható (nem egyértelmű), csak az indexes forma.

Két tervezési fogás érdemel figyelmet. Egyrészt a `std::variant` alapkonstruktora az **első** alternatívát konstruálja, amihez annak alapkonstruálhatónak kell lennie — ha ez nem kívánatos, a `std::monostate`-ot tesszük első helyre, és megkapjuk az explicit „nincs payload” állapotot. Másrészt a `variant` nem allokál: a mérete jellemzően $\max_i$ `sizeof`(T_i) plusz az index tárolása és padding; a pontos értéket mérni kell, mert ABI- és implementációfüggő.

```
// variant_demo.cpp -- ISO C++20
#include <cstdint>
#include <iostream>
#include <string>
#include <variant>

struct Metric { double value; };
struct Log    { std::string text; };
struct Event  { std::uint64_t id; };

using Payload = std::variant<std::monostate, Metric, Log, Event>;

int main() {
    Payload p;                                  // monostate: nincs payload
    p = Log{"startup"};                         // aktív alternatíva váltás
    std::cout << p.index() << '\n';             // 2

    if (auto* l = std::get_if<Log>(&p)) {       // nem dob, nullptr lehet
        std::cout << l->text << '\n';
    }
    try {
        std::cout << std::get<Metric>(p).value; // rossz alternatíva -> kivétel
    } catch (const std::bad_variant_access&) {
        std::cout << "nem metric\n";
    }
}
```

Fordítás: `g++ -std=c++20 -Wall -Wextra -pedantic -O2 -o variant_demo variant_demo.cpp`.

31.1.3 std::visit és a teljes lefedés

A `std::visit(visitor, v)` a visitor call operátorát hívja meg az aktív alternatívára. A lényeg nem a kényelem, hanem a szerződés: a visitornak **minden** alternatívára hívhatónak kell lennie, különben a program nem fordul le. Ez a teljes lefedés (exhaustive handling) fordító általi kikényszerítése: ha a típuslistához új alternatíva kerül, a régi visitor nem fordul tovább, és a hiba a definíció helyén, nem futásidőben a ritka ágon jelentkezik.

A gyakorlatban kétféle visitort írunk. Az egyik egy explicit overload-halmaz (gyakran a `overloaded` minta, amely variadic template-re és pack-kiterjesztésre épül, Variadic template, pack és fold expression (27.4)):

```
template <class... Fs>
struct overloaded : Fs... { using Fs::operator()...; };
template <class... Fs>
overloaded(Fs...) -> overloaded<Fs...>;   // CTAD (C++17)

std::string describe(const Payload& p) {
    return std::visit(overloaded{
        [](std::monostate)      -> std::string { return "empty"; },
        [](const Metric& m)     -> std::string { return "metric"; },
        [](const Log& l)        -> std::string { return "log:" + l.text; },
        [](const Event& e)      -> std::string { return "event#" +
         ↪  std::to_string(e.id); },
    }, p);
}
```

A másik a generikus lambda `if constexpr` lánccal, ami akkor jó, ha az alternatívák közös interfészt mutatnak, vagy ha típusonként eltérő tagot kell elérni. A `std::visit` visszatérési típusa C++17-ben minden alternatívára azonos kell legyen; C++20-tól létezik `std::visit<R>` explicit visszatérési típussal. Több `variant` átadásakor a visitor a kombinációk szorzatán hívódik, tehát a lefedettségi követelmény is a szorzatra vonatkozik.

Itt van egy csapda, ami megkerüli a teljes lefedést: a `[](auto const&) { return std::string{"?"}; }` catch-all lambda a típuslista minden alternatívájára hívható, így a visitor lefordul, és egy később felvett új alternatíva csendben a semmitmondó ágra kerül. A catch-all csak akkor legitim, ha a *típuslistában szereplő* alternatívákra valóban van értelmes alapértelmezett viselkedés. A valueless állapotot viszont semmilyen visitor-ág nem kezeli: valueless variantra a `std::visit` meg sem hívja a visitort, hanem `std::bad_variant_access`-t dob. Ezt tehát a visit előtt (`valueless_by_exception()`) vagy a kivétel elkapásával kell kezelni (lásd a következő szakaszt).

31.1.4 valueless_by_exception: a variant egyetlen hibás állapota

A `variant` normál esetben mindig tartalmaz valamit. Kivételt jelent, ha **típusváltó értékadás vagy `emplace`** közben az új alternatíva konstrukciója kivételt dob: a szabvány megengedi, hogy a variant `valueless_by_exception()` állapotba kerüljön, amelyet `index() == std::variant_npos` jelez, és amelyre a `std::visit` `std::bad_variant_access`-t dob. Fontos feltétel, hogy ez a helyzet a *már létező* variant objektumon, értékadás/`emplace` során állhat elő — a variant puszta másolása/mozgatása közben dobott kivétel nem hoz létre valueless objektumot, hiszen az új objektum létre sem jön.

Két gyakorlati következmény. Egyrészt kivételeket nem használó buildben (`-fno-exceptions`) ez az állapot nem áll elő, másrészt a standard megengedi, de nem írja elő, hogy egy implementációval ez ténylegesen elérhető-e, ezért a `valueless_by_exception()`-t **nem** lehet kihagyni, ha az alternatívák konstruktora dobhat és az értékadás típusváltó. Ilyen esetben a visit előtt ellenőrizz:

```
if (p.valueless_by_exception()) { /* kényszerített "nincs értelmezhető állapot"
 ↪  ág */ }
```

Ha a `noexcept` konstruktorú alternatívákra szorítkozol, ez az ág statikusan kizárható, és a kód egyszerűbb lesz.

```cpp
// valueless_visit.cpp - C++20: a catch-all ág soha nem kapja meg a valueless
 ↪  állapotot
#include <cstdio>
#include <stdexcept>
#include <string>
#include <variant>

struct Thrower {
    std::string name;                       // nem triviálisan másolható típus
    Thrower() = default;
    explicit Thrower(int) { throw std::runtime_error("ctor"); }
};

int main() {
    int catch_all_calls = 0;
    auto visitor = [&](const auto&) { ++catch_all_calls; return
     ↪  std::string{"?"}; };

    std::variant<std::monostate, std::string, Thrower> v{std::string{"x"}};
    try { v.emplace<Thrower>(1); } catch (const std::runtime_error&) {}
    std::printf("valueless=%d\n", v.valueless_by_exception());

    try {
        std::visit(visitor, v);
    } catch (const std::bad_variant_access&) {
        std::puts("bad_variant_access");
    }
    std::printf("catch-all hivasok: %d\n", catch_all_calls);   // 0

    std::variant<std::monostate, std::string, Thrower> e;      // monostate
    std::visit([](const auto& x) {
        if constexpr (std::is_same_v<std::decay_t<decltype(x)>, std::monostate>)
            std::puts("monostate ag");
    }, e);
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic valueless_visit.cpp` kimenete: `valueless=1`, `bad_variant_access`, `catch-all hivasok: 0`, `monostate ag`; a Clang 22 / libc++ ugyanezt adta. A `Thrower` `std::string` tagja nem dísz: a libstdc++ triviálisan másolható alternatívánál előbb egy ideiglenesbe épít, és ilyenkor a variant nem lesz valueless — ez a megengedett implementációs mozgástér.

31.1.5 C tagged union és a típusos C++ élettartam

C-ben ugyanezt kézi taggel írjuk le; a tag karbantartása a program felelőssége, és a nemtriviális tagok (például `char*`) élettartamát semmi nem kezeli.

```c
/* tagged_union.c -- ISO C17 */
#include <stdio.h>

enum payload_kind { PAYLOAD_EMPTY, PAYLOAD_METRIC, PAYLOAD_EVENT };
```

```c
struct payload {
    enum payload_kind kind;        /* az aktív tag: ezt a program őrzi */
    union {
        double metric;
        unsigned long long event_id;
    } u;
};

int main(void) {
    struct payload p = { .kind = PAYLOAD_METRIC, .u.metric = 0.5 };
    if (p.kind == PAYLOAD_METRIC)
        printf("%f\n", p.u.metric);
    return 0;
}
```

Fordítás: `gcc -std=c17 -Wall -Wextra -pedantic -O2 -o tagged_union tagged_union.c`.

A C megoldás gyenge pontja nem a szintaxis, hanem a szerződés: a `kind` és az `u` konzisztenciáját semmi nem ellenőrzi, a rossz tag olvasása az objektumreprezentáció újraértelmezését adja (a C szabvány szerint ez nem feltétlenül UB, de az érték implementáció által meghatározott, azaz logikailag hibás program), és nemtriviális típust csak pointeren vagy byte-pufferen keresztül lehet elhelyezni, kézi konstrukcióval és destruktorral.

Szempont	C tagged union	`std::variant`
Aktív alternatíva	kézi `enum` tag, a program tartja karban	könyvtár tárolja, `index()`
Tárolt érték élettartama	nincs kezelve	destruktor, értékadás, `emplace` kezeli
Nemtriviális tag	csak pointeren/byte-pufferen át	közvetlenül tárolható
Rossz tag olvasása	reprezentáció-újraértelmezés, csendben rossz eredmény	`get` → kivétel, `get_if` → `nullptr`
Méret	`max(sizeof tagok)` + tag	`max(sizeof alternatívák)` + index + padding
Allokáció	nincs	nincs
Típusbiztonság	nincs	fordítási idejű

A döntés tehát nem teljesítmény, hanem szerződés kérdése: a `variant` ugyanazt a tárolási mintát adja, de az aktív alternatívát és az élettartamot a típusrendszerbe emeli. Nyitott halmazra (bővíthető viselkedés, plugin) viszont továbbra is a virtuális interfész a helyes eszköz (Virtual, override, final és dispatch (24.2)); a `variant` a lezárt esetekre való.

31.1.6 Tipikus hibák és javításuk

- **`optional` hibaként.** Ha a hiány oka a hívónak számít, `expected<T,E>` kell. Javítás: a szerződés írásbeli rögzítése és a két típus szétválasztása.
- **`*opt` ellenőrzés nélkül.** Üres `optional` dereferálása nem definiált viselkedés. Javítás: `if (opt)` vagy `value_or`.
- **`get<T>` `holds_alternative` helyett.** A `get_if` nem dob, ezért feltételes ágban ez a robusztusabb.

- **Catch-all visitor.** Elnyeli az új alternatívát. Javítás: explicit overload-halmaz, a catch-all csak a `valueless` ágra.
- **`emplace` utáni régi referencia.** Az `emplace`/`reset` érvényteleníti a korábban szerzett pointert és referenciát; ezek a `T` objektum élettartamához kötöttek. Javítás: az új értékhez újra hozzáférni, nem eltárolt pointeren át.
- **`sizeof`-ra építő optimalizálás.** A `variant`/`optional` mérete nem garantált; méréssel és dokumentált profillal kezeld.

31.1.7 Feladatok

1. feladat (modellválasztás). Egy eseménystream payloadja három alakú lehet: nincs payload, `std::uint64_t` számláló, `std::string` üzenet. Írd fel a típust, és indokold, miért nem `std::optional<std::variant<...>>`, illetve miért nem `std::variant` külön `bool has_payload` taggal.

2. feladat (kódolás). Készíts `std::visit`-alapú `describe` függvényt az 1. feladat típusára `overloaded` használatával, teljes lefedéssel. Vedd fel negyedik alternatívaként a `struct Close{}` típust, és magyarázd el, mit jelez a fordító, ha az új overloadot kifelejted.

3. feladat (diagnosztika). Adott a `[](auto const&) { return std::string{"?"}; }` visitor. Magyarázd meg, miért nem véd a teljes lefedés ellen, és add meg két módját annak, hogy a kihagyott alternatíva fordítási hibát okozzon. Mikor kell mégis `valueless_by_exception()`-t ellenőrizni a visit előtt?

4. feladat (C összehasonlítás). Írd át a fejezet C tagged unionját `std::variant`-ra, és sorold fel, mely kézi kötelezettségek tűnnek el.

31.1.8 Megoldások

1. megoldás. `using Payload = std::variant<std::monostate, std::uint64_t, std::string>;`. A `monostate` az első helyen biztosítja az alapkonstruálhatóságot és a „nincs payload” állapotot. Az `optional<variant<...>>` kétszeresen kódolná ugyanazt a hiányt (két forrásigazság: `optional` állapota és `monostate`), és a hívónak két szinten kellene ellenőriznie. A külön `bool` tag pedig éppen azt a kézi konzisztencia-kötelezettséget hozná vissza, amiért a `variant`-ot választjuk. Ellenőrzési feltétel: a típus alapkonstruktora fordul, `Payload{}` esetén `index() == 0`, és egyetlen forrásból (`index()`) eldönthető az állapot.

2. megoldás. A `describe` a fenti `overloaded` példa mintájára írható, négy explicit overloaddal. Ha a `Close` alternatívát felveszed a `variant`-ba, de a visitornál kihagyod a hozzá tartozó overloadot, a `std::visit` nem talál hívható `operator()`-t a `Close` argumentumra, ezért a kifejezés ill-formed: a hiba a `describe` definíciójánál, fordításkor jelentkezik, nem futásidőben. Ez a kívánt viselkedés: a típusbővítés kikényszeríti a feldolgozás bővítését. Ellenőrzési feltétel: az overload kihagyásával a build hibára fut, visszatételével pedig `describe(Payload{Close{}}) == "close"`.

3. megoldás. A catch-all lambda minden argumentumtípusra hívható, ezért a `std::visit` számára a visitor teljesnek látszik; az új alternatíva csendben a `"?"` ágra esik, és a hiba csak akkor derül ki, ha valaki észreveszi a hibás kimenetet. Két statikus kikényszerítés: (a) explicit overload-halmaz a generikus lambda helyett; (b) a visitor törzsében `static_assert` a lefedett típusok halmazára, illetve típusonkénti `if constexpr`-lánc, amelynek végén `static_assert(false)`-szal jelzett „nem kezelt alternatíva” ág áll (a `false` feltételt template-paramétertől függővé kell tenni, hogy ne azonnal bukjon el). A `valueless_by_exception()` ellenőrzése akkor kell, ha az alternatívák konstruktora dobhat és az értékadás/`emplace` típusváltó, mert a `std::visit` valueless varianton `std::bad_variant_access`-t dob. Ellenőrzési feltétel: a catch-all eltávolítása

után a kihagyott alternatíva fordításhiba; a valueless ág jelenléte mellett a `-fno-exceptions` build is fordul.

4. megoldás. `using Payload = std::variant<std::monostate, double, unsigned long long>;` (a C `enum` helyett `monostate`, a `union` helyett a típuslista). Eltűnik: a `kind` tag kézi karbantartása, a rossz tag olvasásának kezeletlensége (helyette `get_if`/`bad_variant_access`), valamint a nemtriviális tagok kézi konstrukciója és destruktorhívása. Megmarad: az a döntés, hogy a payload zárt halmaz, és hogy a feldolgozást minden alternatívára meg kell írni. Ellenőrzési feltétel: a `variant` destruktora akkor is helyesen fut, ha a `std::string` alternatíva aktív, kézi `free` nélkül.

31.1.9 Összefoglaló

A `std::optional<T>` a jelenlétet modellezi: legfeljebb egy `T`, helyben tárolva, `emplace`/`reset` által mozgatott élettartammal, hibaok nélkül. Ha a hiány oka a hívónak számít, nem ez a típus, hanem `expected`/`error_code` a válasz. A `std::variant<Ts...>` zárt alternatívahalmaz, mindig pontosan egy aktív taggal, `index()`/`get`/`get_if` hozzáféréssel, egyetlen kivételes állapottal (`valueless_by_exception`), amely típusváltó, dobó konstrukciójú értékadásnál fordulhat elő. A `std::visit` a feldolgozást az alternatívákhoz köti, és ezzel fordítási időben kikényszeríti a teljes lefedést — feltéve, hogy nem rejtjük el catch-all lambdával. A C tagged unionnal szemben nem teljesítményt nyerünk, hanem szerződést: az aktív tag és az élettartam a típusrendszer része lesz.

31.2 tuple, pair, any és type erasure

A heterogén adatok kezelése visszatérő feladat minden DevOps-eszközben: konfiguráció, plugin-metaadat, esemény-payload, telemetria-címke. A C++ három egymástól eltérő modellt ad erre: a fordítási idejű, zárt típuslistát (`std::pair`, `std::tuple`), a futásidejű típus-törlést (`std::any`), valamint a klasszikus polimorf interfészt (öröklés + virtuális hívás). Ebben az alfejezetben ezeket a modelleket hasonlítjuk össze egy kis konfigurációs példán, kiemelve a típusismeret és a tulajdonlás következményeit. Az `optional`/`variant` alapokat itt már ismertnek tekintjük optional, variant és visit (31.1), a callable-wrapper és a `std::function`-alapú type erasure tárgyalása invoke, function wrapper és type erasure (30.2), a típuslevezetésé auto, decltype és típuslevezetés (19.4) szerepel.

31.2.1 Pair és tuple: fordítási idejű heterogén csoportosítás

A `std::pair<T1,T2>` (`<utility>`, C++98) pontosan két publikus tagot tárol `first` és `second` néven, a `std::tuple<Ts...>` (`<tuple>`, C++11) nulla vagy több elemet tetszőleges típuslistával. Mindkettő **értéktípus**: másolás és mozgatás tagonként történik; nincs bennük rejtett pointer vagy megosztott állapot. Ezzel szemben a `std::any` (lásd alább) típus-érzéketlen konténer — a kettő nem keverendő.

A C++17-es **strukturált kötés** (`structured binding`) rövid hozzáférést ad:

```
auto [host, port] = endpoint;      // másolat készül a két tagról
auto& [h2, p2]  = endpoint;        // referenciát kötünk, nincs másolat
```

A deklaráció mögött egy névtelen, `auto`-val levezetett ideiglenes vagy referenciaobjektum áll; a nevek ehhez kötődnek, és a `tuple_size`/`tuple_element` specializációkon keresztül `get<I>` hívásokra fordulnak. Ezért `auto [x,y]` másol, `auto& [x,y]` köt. Ugyanez a mechanizmus működik `std::pair`, `std::tuple`, `std::array` esetén.

Az elemek elérése: `std::get<0>(t)` indexszel, `std::get<int>(t)` típus szerint, ha az adott típus egyedi a listában. Visszatérési értéke referencia; `std::get<I>(std::move(t))` esetén `T&&`. A `std::tie(a, b)` egy `std::tuple<A&, B&>`-t készít — unpackingra (`std::tie(x,y) = p;`) és lexikografikus összehasonlításra egyaránt használjuk. A `std::apply(f, t)` a tuple elemeit argumentumként átadja egy callable-nek; a `std::tuple_cat(t1, t2)` új tuple-t fűz össze, elemenként másolva vagy mozgatva:

```
auto t = std::make_tuple(1, 2.5, std::string{"x"});
auto print = [](int a, double b, const std::string& s) {
  std::printf("%d %.1f %s\n", a, b, s.c_str());
};
std::apply(print, t);                                  // print(1, 2.5, "x")
auto m = std::tuple_cat(t, std::make_tuple(true));     // (1, 2.5, "x", true)
```

A `std::make_tuple` a tagok `decay`-elt típusát tárolja (tömb → pointer, referencia → érték); referencia megtartásához `std::tie` vagy `std::forward_as_tuple` szükséges.

31.2.2 Referenciák a tuple-ban és a dangling veszély

A tuple nem tartja életben a benne lévő elemeket; ha elemként referencia szerepel, az csak **kölcsönzés**. A `std::tie(x, y)` típusa `std::tuple<int&, int&>`; meddig él ez a hivatkozás? Addig, amíg `x` és `y` élnek. Ha a tuple hosszabb életű, mint a hivatkozott objektum, dangling pointerré válik:

```
std::tuple<int&, std::string&> refs;     // referencia-tuple
{
  int x = 1;
  std::string s = "a";
  refs = std::tie(x, s);
}                                          // x és s megszűnt
// std::get<0>(refs);   // UB: dangling
```

Hasonlóan hibás a `return std::tie(a, b);` minta: a visszaadott tuple lokális változókra hivatkozik. A `std::forward_as_tuple(f())` rvalue-referenciákat ad, de az `f()` ideiglenese a teljes kifejezés végén megszűnik, így a tuple rögtön utána dangling. Ez alattomos hiba, mert a fordító gyakran nem jelez.

A `tie` eredményét csak akkor tárold, ha a hivatkozott objektumok élettartama biztosan fedi. Ha másolás kell, használj `std::make_tuple`-t; ha a szándék kölcsönzés, az legyen lokális, rövid életű. A strukturált kötéssel kapcsolatban ugyanez a szabály: `auto [x, y] = std::tie(a, b);` esetén `x, y` referencia-tagokká válnak, ami addig biztonságos, amíg `a` és `b` él.

31.2.3 std::any: futásidejű type erasure

A `std::any` (`<any>`, C++17) egyetlen értéket tárol tetszőleges **másolható** típusból. Típus-törlést (type erasure) valósít meg: a tárolt típus a fordító számára ismeretlen, de futásidőben `type()` visszaad egy `std::type_info`-t. A tárolt típusnak `CopyConstructible`-nek kell lennie, mert a `std::any` maga is másolható, és a másolás a benne lévő objektumot is másolja. Ezért nem lehet `std::any`-be `std::unique_ptr`-t tenni (fordítási hiba), se nem-másolható mutexet vagy stream-et.

Hozzáférés `any_cast`-tal:

- `std::any_cast<T>(a)` — érték szerint: másolatot ad; típuseltérés esetén `std::bad_any_cast`-et dob.
- `std::any_cast<T&>(a)` és `const T&` — referenciát ad a belső objektumra. Érvényes mindaddig, amíg az `any` **módosítatlan**; minden `reset`, értékadás vagy `emplace` érvényteleníti (invalidation).
- `std::any_cast<T>(&a)` — pointer form: típuseltérésnél `nullptr`-t ad, nem dob. Ez a „nem dobó” változat, amit a gyakorlatban a legtöbbször használunk.

A small-object optimalizálás (SBO) a `std::any` esetén **implementációfüggő**: a szabvány nem garantálja, hogy egy adott méret alatti típust in-line tárol, és nem garantálja a cím stabilitását sem. Ne építs tehát teljesítmény-becslést vagy mutató-érvényességet arra, hogy „biztos a belső bufferben van”. Ugyanígy a `type()` által visszaadott `type_info`-ra típusnév szerinti string-összehasonlítást alapozni nem hordozható minta; az `any` típusellenőrzését mindig `any_cast`-ra bízzuk.

Az `any` gyakran keveredik a „view” fogalmával: nem az. Az `any` **tulajdonol** (értékszemantika, belső másolat), és a `any_cast<T&>` az egyetlen kölcsönző kilépés. Ha hosszabb életű hozzáférést akarsz, `T`-t értékben vagy `T&`-et csak az `any` élettartamán belül kérj.

31.2.4 Konfigurációs példa: típusos mezők és plugin-adatok

A tipikus minta: a szolgáltatás saját konfigurációját struct/tuple fogja össze (zárt, fordítási idejű típuslista), míg a plugin-specifikus kulcs–érték párok `std::map<std::string, std::any>`-ben landolnak, mert a plugin készlete a fő programból nem ismert.

```
// cfg.cpp - teljes program
#include <any>
#include <cstdint>
#include <cstdio>
#include <map>
#include <string>
#include <tuple>

struct Config {
  std::string bind{"127.0.0.1"};
  std::uint16_t port{8080};
  std::int64_t timeout_ms{500};
};

bool operator==(const Config& a, const Config& b) {
  return std::tie(a.bind, a.port, a.timeout_ms)
      == std::tie(b.bind, b.port, b.timeout_ms);
}

std::map<std::string, std::any> plugin_settings;

void read_retries() {
  auto it = plugin_settings.find("retries");
  if (it == plugin_settings.end()) { std::puts("retries: nincs"); return; }
  if (auto* p = std::any_cast<int>(&it->second)) std::printf("retries=%d\n",
   ↪  *p);
  else                                          std::puts("retries: nem int");
}
```

```
int main() {
  Config base;
  auto [bind, port, timeout_ms] = base;
  std::printf("bind=%s port=%u timeout=%lld\n",
              bind.c_str(), port, static_cast<long long>(timeout_ms));

  plugin_settings["retries"] = 3;
  plugin_settings["tag"]     = std::string{"edge"};
  read_retries();

  // plugin_settings["tag"] = std::make_unique<int>(1); // nem fordul: nem
   ↪  másolható
  return 0;
}
```

Build a tanuló laborjában, rögzített compilerrel: `g++ -std=c++20 -O2 -Wall -Wextra cfg.cpp -o cfg` (Clang esetén `clang++ -std=c++20 -O2 -Wall -Wextra cfg.cpp -o cfg`). A példa **nem lett lefuttatva** e dokumentum fordításakor; a helyességét a tanuló saját mérési környezetében ellenőrzi. Figyeljük meg a tulajdonlási különbséget: `Config` mezői értéktulajdonosok, a `plugin_settings` értékei is (az `any` másol), viszont a `std::any_cast<int>(&it->second)` pointer rövid életű, csak a hívás scope-jában használjuk.

31.2.5 Döntési táblázat: variant, any, polimorf interfész

Szempont	`std::tuple/pair`	`std::variant<Ts...>`	`std::any`	Polimorf interfész
Típuslista	rögzített, fordítási idejű	zárt, fordítási idejű	nyitott, futásidejű	nyitott (öröklés)
Hozzáférés	`get<I>`	`get<T>`, `visit`, `get_if`	`any_cast`	virtuális hívás
Hibamód	fordítási hiba	`bad_variant_access` / `nullptr`	`bad_any_cast` / `nullptr`	nem jellemző
Tárolási méret	elemek összege	legnagyobb alternatíva + tag	SBO vagy heap (impl.)	pointer + heap
Másolás	elemenként	ha mind másolható	csak másolható típus	interfész dönt
Mikor válaszd	ismert, vegyes mezők	zárt alternatíva-halmaz	plugin/config	viselkedés-orientált

Hüvelykujjszabály: ha a lehetséges típusok halmaza **fordítási időben ismert és zárt**, `tuple` (egyidejű jelenlét) vagy `variant` (egy aktív alternatíva). Ha a típus futásidőben érkezik, `any`. Ha viselkedést is kell (nem csak adatot), polimorf interfész.

31.2.6 Gyakori hibák

- **Move-only típus any-be**: `std::any a = std::make_unique<int>(1);` — nem fordul, a `unique_ptr` nem másolható. Ilyenkor `variant<..., unique_ptr<...>>` a helyes választás.
- `return std::tie(a, b);` lokálisokból: dangling. Helyette `return std::make_tuple(a, b);` értékben.

- `auto [x, y] = std::tie(a, b);` majd az eredeti séma szerint módosítás: `x, y` referenciák, ezért az `a`-ra mutatnak. Szándékos lehet, de gyakran félreértés.
- **`any_cast<T>(a)` érték szerint** minden hívásnál másol — teljesítmény-érzékeny kódban pointer vagy `T&` forma.
- **SBO-ra építeni**: az implementáció megváltoztathatja, a cím stabilitása nem garantált; gépfüggően viselkedhet.

31.2.7 Feladatok

1. *(magyarázó)* Mutasd meg, hogy a `auto t = std::tie(x, y); std::get<0>(t) = 10;` után `x` értéke 10 lesz-e, és meddig érvényes a `t`. Indokold, mi a helyzet akkor, ha `t` egy függvényből visszatérő érték.
2. *(kódolási)* Írj egy `split_host_port(const std::string&)` függvényt, ami `"127.0.0.1:8080"`-ból `std::tuple<std::string,int>`-et ad. A hívásnál használj strukturált kötést, és `std::tie`-val hasonlíts egy elvárt tuple-höz.
3. *(diagnosztikai)* Az alábbi kód `std::bad_any_cast`-et dob. Keresd meg az okot, és írd át nem dobó változatra.

```
std::any a = 42;
double v = std::any_cast<double>(a);
```

31.2.8 Megoldások

1. A `std::tie(x, y)` egy `std::tuple<int&, int&>`; `std::get<0>(t)` `int&`-et ad, ami `x`-re hivatkozik. A `std::get<0>(t) = 10;` tehát `x`-et módosítja. A `t` addig érvényes, amíg `x` és `y` élnek — lokális scope-ban helyes. Ha `return std::tie(x, y);` függvényből jön vissza, a visszatérés pillanatában a lokálisok megszűnnek, a hívó oldal dangling referenciát kap, és bármilyen dereferálás UB.

2.

```
#include <stdexcept>
#include <string>
#include <tuple>

std::tuple<std::string, int> split_host_port(const std::string& s) {
  auto pos = s.rfind(':');
  if (pos == std::string::npos) throw std::invalid_argument("no ':'");
  return {s.substr(0, pos), std::stoi(s.substr(pos + 1))};
}

int main() {
  auto [host, port] = split_host_port("127.0.0.1:8080");
  auto expected = std::make_tuple(std::string{"127.0.0.1"}, 8080);
  return std::tie(host, port) == expected ? 0 : 1;
}
```

Ellenőrzés: `main` 0-t ad, ha az egyezés teljes. A `std::tie(host, port)` lokális változók referenciáit adja, ugyanabban a scope-ban, tehát biztonságos; ha a függvény visszatérne vele, már nem lenne az.

3. Az `a` tárolt típusa `int`, a `any_cast<double>` nem egyezik, ezért dob. Nem dobó javítás:

```cpp
std::any a = 42;
if (auto* p = std::any_cast<int>(&a)) {
  int v = *p;                       // biztos elérés
}
// vagy ha double kell:
double d = static_cast<double>(std::any_cast<int>(a));
```

A pointer forma `nullptr`-t ad típuseltérésnél; az érték forma csak akkor biztonságos, ha a típus garantáltan egyezik, vagy `try/catch (const std::bad_any_cast&)` védi.

31.2.9 Összefoglaló

A `pair` és a `tuple` fordítási idejű, rögzített típuslistát képvisel értékszemantikával; a `get`, `tie`, `apply`, `tuple_cat` és a strukturált kötés a mindennapi eszköztáruk. A referenciát tartalmazó tuple (különösen `tie`, `forward_as_tuple`) csak kölcsönöz — élettartamát a hivatkozott objektumokhoz kell kötni, különben dangling. A `std::any` futásidejű típus-törlést valósít meg, csak másolható típust tárol, `any_cast`-tal érhető el; az SBO implementációfüggő, a visszaadott referencia pedig addig érvényes, amíg az `any` módosítatlan. A modellek közötti választás a típushalmaz zártsága, a hibamód és a tulajdonlás szerint történik: zárt alternatívák → `variant`, nyitott futásidejű adat → `any`, viselkedés → polimorf interfész.

31.3 span, byte és bounds-szerződések

A C API-k világában a legelterjedtebb interfész-alak a `(T *ptr, size_t n)` pár: a hívó ad egy nyers pointert és egy elemszámot, a hívott pedig feltételezi, hogy a kettő konzisztens. Ez a forma minden információt hordoz, ami a memóriához való hozzáféréshez kell, és semmit abból, ami a biztonságos használathoz: nem derül ki belőle, hogy ki a tulajdonos, meddig él a tároló, és hogy a hívott jogosult-e írni. A C++20 `std::span` ezt a párost egyetlen, típusos, nem tulajdonló nézetté (borrowed view) alakítja, amely a *szerződést* nem oldja meg helyettünk, de láthatóvá és dokumentálhatóvá teszi. A fejezet központi állítása ezért óvatos: a `span` típusbiztonságot ad a `void*`-hoz képest, de **nem** végez automatikus futásidejű határellenőrzést minden indexelésnél, és nem hosszabbítja meg a mögöttes tároló élettartamát.

A tárgyalás a korábbi részekre épül: az owner/view szerződést az erőforrás-szerződés részben rögzítettük, a nyers byte-nézet C-beli megfelelőjét a memóriamásolás és byte view részben láttuk, a nem tulajdonló, típus által paraméterezett burkolók családjába pedig a `tuple/any` rész adott bevezetést. Itt a hangsúly a *span-specifikus* előfeltételeken és azon, hogy a `std::byte` reprezentáció, nem pedig hordozható formátum.

31.3.1 A nézet szerződése: owner, borrowed, bounds

Definíció. `std::span<T, Extent>` (C++20, `<span>`) egy könnyűsúlyú, nem tulajdonló, másolható nézet egy folytonos, `T` típusú elemekből álló tartományra. A `data()` egy `T*`-ot ad, `size()` az elemszámot, és a nézet az `[data(), data() + size())` fél-nyílt tartományt jelöli.

A szerződésnek három, egymástól független eleme van, és mindhármat a hívónak kell biztosítania:

1. **Ownership:** a `span` nem tulajdonol, nem allokál és nem szabadít fel. Az erőforrás lezárása az owner (jellemzően `std::vector`, `std::array`, `std::string` vagy egy C buffer) feladata.

2. **Folytonosság és érvényesség:** a konstrukció pillanatában a megadott tartománynak érvényes, folytonos memóriának kell lennie. A `span(T *ptr, size_type count)` konstruktor előfeltétele, hogy `[ptr, ptr + count)` egy érvényes tartomány legyen; ezt a szabvány nem ellenőrzi.
3. **Élettartam:** a mögöttes tárolónak *legalább addig* élnie kell, amíg a `span`-t használjuk, és nem futhat olyan művelet, amely a tárolót invalidálja.

A C megfelelővel összevetve: a `(ptr, n)` pár ugyanezt a három szerződést hordozza, csak rejtve. A `span` annyiban jobb, hogy (a) a típus megnevezi az elemtípust, így nincs `void*` és nincs kézi `sizeof`-osztás, (b) az owner-től való átvétel egyetlen, felismerhető szintaktikai aktus, (c) a függvényszignatúrában láthatóvá teszi, hogy a paraméter nem tulajdonló.

31.3.2 Kiterjedés: statikus és dinamikus

A `span` mérete vagy futásidejű, vagy a típus része:

- `std::span<T>` = `std::span<T, std::dynamic_extent>`, ahol `std::dynamic_extent` egy `size_t` konstans (a `size_t` maximális értéke).
- `std::span<T, N>` statikus kiterjedésű: a `N` fordítási időben ismert, a `span<T, N>::extent` statikus tag adja vissza, és `size()` futásidőben is `N`.

A statikus kiterjedés ott érték, ahol a méret a protokoll vagy a formátum része: egy 8 bájtos fejlécnél a méret nem adat, hanem szerződés. Ilyenkor a `first<4>()`, `subspan<4, 4>()` hívások nagy része már fordításkor ellenőrizhető. A `sizeof(std::span<T>)` értékét a szabvány nem rögzíti; a tipikus implementáció két gépi szót tárol (pointer és méret), a statikus kiterjedésű változat pedig akár csak a pointert — ez implementációfüggő részlet, ne építs rá hordozható kódot.

31.3.3 Konstrukciós előfeltételek és objektum-élettartam

A leggyakoribb hibaforrás nem az indexelés, hanem a konstrukció. Tartományból (`R&&`) konstruálva a `span` előfeltétele, hogy a tartomány folytonos (`contiguous_range`) és mérete ismert (`sized_range`) legyen. A C++20-as szabály ehhez hozzáteszi: a konstruktor akkor is engedélyezett, ha a tartomány nem `borrowed_range`, **feltéve, hogy az elemtípus `const`**. Ez a látszólag ártalmatlan megengedés a legfontosabb csapda:

```
#include <span>
#include <vector>

std::span<const int> danger()
{
    std::vector<int> tmp{1, 2, 3};
    return std::span<const int>{tmp};   // a tmp a visszatéréskor megszűnik ->
     ↪  dangling
}

void ok()
{
    std::vector<int> v{1, 2, 3};
    std::span<const int> s{v};          // a v túlél a használatig: rendben
    // ...
```

```
}
// std::span<int> s{std::vector<int>{1,2,3}};  // ez nem fordul le: nem const
 ↪  elemtípus
```

A `std::span<const int>` tehát átvesz ideiglenest is, mert a `const` elemtípus miatt a szabvány megengedi — a `span` ettől még nem lesz owner. A másik irány: `std::span<int>` csak `const`-mentes tárolóból konstruálható, és a `span<int>` → `span<const int>` átalakítás implicit, fordítva nem az.

Konténerből statikus kiterjedésre váltás (`std::span<std::byte, 8>` egy `std::vector<std::byte>`-ból) explicit konstruktor, amelynek futásidejű előfeltétele `size() == 8`; ezt a szabvány nem ellenőrzi, nekünk kell (vagy fordítási időben, `std::array` használatával).

31.3.4 Résznézetek és indexhatárok

A `span` résznézetei a következő előfeltételekkel rendelkeznek (a `subspan` esetén `off` az eltolás, `cnt` az elemszám):

- `s.first(n)`: `n <= s.size()`
- `s.last(n)`: `n <= s.size()`
- `s.subspan(off, cnt)`: `off <= s.size()` és `cnt == std::dynamic_extent` vagy `off + cnt <= s.size()`
- `s.first<Count>()`, `s.last<Count>()`, `s.subspan<Offset, Count>()`: ugyanaz, de a statikus kiterjedés miatt `Count <= N` már `static_assert`-tel ellenőrizhető.

Az indexelésre: `s[i]` előfeltétele `i < s.size()`, `s.front()` és `s.back()` előfeltétele `!s.empty()`. Ha ezek nem teljesülnek, a viselkedés nem definiált; a C++20-as `span` nem kínál `at()`-ot, azaz nincs beépített, kivételt dobó változat. (A későbbi szabványverziókban megjelenhet ilyen tag; használat előtt ellenőrizd a választott standard és library verzióját — ezt a fejezet írásakor nem tekintem igazoltnak.)

A határellenőrzés hiánya nem azt jelenti, hogy nincs eszközünk: a standard library implementációk hardening kapcsolói (például a libstdc++ `_GLIBCXX_ASSERTIONS` makrója, a libc++ hardening módja) egyes esetekben ellenőrzést iktatnak be. Ezek *implementáció- és verziófüggő* eszközök, nem nyelvi garanciák, és a jelenlétüket a használt toolchain dokumentációjában kell ellenőrizni. Ugyanígy az AddressSanitizer csak akkor jelez, ha a hibás hozzáférés ténylegesen egy lefoglalt, de már felszabadított vagy a lefoglalt blokkon kívüli területre esik; egy allokáción belüli, de logikailag hibás indexelést nem feltétlenül lát.

31.3.5 `std::byte`: reprezentáció, nem formátum

A `std::byte` (C++17) definíció szerint `enum class byte : unsigned char`. Nem aritmetikai típus: csak bitműveletek (`&`, `|`, `^`, `~`, `<<`, `>>`, `<<=`, `>>=`, `&=`, `|=`, `^=`) és összehasonlítások értelmezettek rajta, számtani művelet nem. Egész értékké `std::to_integer<T>(b)` alakítja; a másik irányban `static_cast<std::byte>(x)` a helyes (az `unsigned char`-en keresztüli explicit konverzió kifejezi a szándékot, és nem téveszt össze karakterrel).

A `std::as_bytes(s)` `std::span<const std::byte>`-ot, a `std::as_writable_bytes(s)` `std::span<std::byte>`-ot ad vissza ugyanarra a tárolóra. Előfeltétel: `T` teljes, nem absztrakt objektumtípus. A két függvény **az objektumreprezentációt** teszi elérhetővé — ez nem ugyanaz, mint egy hordozható sorosítási formátum:

- a reprezentáció tartalmazhat kitöltő (padding) byte-okat, amelyek értéke nem specifikált;

- a többbyte-os egészek byte-sorrendje implementációfüggő;
- a struct mérete, igazítása és tagjainak sorrendje (padding) a target ABI-tól függ.

Ezért `as_bytes(span{header})` közvetlen `write()`-ja tipikus, de nem hordozható megoldás: két különböző ABI-jú gépen vagy fordítóval eltérő byte-folyam keletkezhet, és a padding miatt a kimenet akár memóriaszivárgást is hordozhat. Hordozható protokollhoz a mezőket egyenként, rögzített byte-sorrendben kell kiírni egy `std::span<std::byte>` célbufferbe — ezt mutatja a következő példa. A `as_bytes` legitim használata az azonos platformon belüli, azonos buildből származó objektumokra korlátozódó művelet: hasht számítani, `memcmp`-et végezni, vagy syscall-pufferbe írni.

31.3.6 `span` és `string_view`: két különböző cél

A két típus hasonlít (nem tulajdonló, folytonos nézet), de nem helyettesítik egymást.

Tulajdonság	`std::vector<T>` (owner)	`std::span<T>`	`std::string_view`
Tulajdonol	igen	nem	nem
Elemtípus	`T`	`T` (nyers, írható is lehet)	`char` (nem írható)
Szövegszemantika (`find`, `substr`)	nem	nem	igen
Indexelés ellenőrzése	`at()` igen, `[]` nem	nincs	nincs (a `[]` előfeltétel)
Invalidálja	reallokáció, `erase`, `clear`	a mögöttes tároló bármely invalidáló művelete	ugyanaz

A `span<const char>` tehát nem „szegény ember `string_view`-ja”: az nem tud szöveget keresni, viszont tud nem-karakter adatot is hordozni, és `span<char>` írható. A `string_view` szövegre való, a `span` általános memóriatartományra.

31.3.7 Kidolgozott példa: keretes protokoll span-nel

A példa egy 8 bájtos fejlécet (`kind`, `length`, mindkettő big-endian `uint32_t`) és payloadot kezel. A cél, hogy a határ- és élettartam-szerződés a szignatúrában és a kódban is látszódjék.

```
// frame.cpp - C++20; g++/clang++ -std=c++20
#include <algorithm>
#include <cstddef>
#include <cstdint>
#include <cstdio>
#include <span>
#include <stdexcept>
#include <vector>

namespace wire {

inline std::uint32_t read_be32(std::span<const std::byte> b)
{
    if (b.size() < 4) throw std::length_error{"read_be32: short"};
```

```
    return (std::to_integer<std::uint32_t>(b[0]) << 24)
         | (std::to_integer<std::uint32_t>(b[1]) << 16)
         | (std::to_integer<std::uint32_t>(b[2]) << 8)
         |  std::to_integer<std::uint32_t>(b[3]);
}

inline void write_be32(std::span<std::byte> b, std::uint32_t v)
{
    if (b.size() < 4) throw std::length_error{"write_be32: short"};
    b[0] = static_cast<std::byte>(v >> 24);
    b[1] = static_cast<std::byte>(v >> 16);
    b[2] = static_cast<std::byte>(v >> 8);
    b[3] = static_cast<std::byte>(v);
}

struct Frame {
    std::uint32_t              kind;
    std::span<const std::byte> payload;  // borrowed: a bemeneti bufferre mutat
};

// Elofeltetel: buf.size() >= 8. A visszaadott Frame.payload ugyanarra a
// tarolora mutat, mint buf; a hivonak kell biztositania, hogy a buffer
// tulelje a Frame hasznalatat.
inline Frame parse_frame(std::span<const std::byte> buf)
{
    if (buf.size() < 8) throw std::length_error{"parse_frame: short"};
    const std::uint32_t kind = read_be32(buf.first(4));
    const std::uint32_t n    = read_be32(buf.subspan(4, 4));
    if (static_cast<std::size_t>(n) > buf.size() - 8)
        throw std::length_error{"parse_frame: truncated"};
    return Frame{kind, buf.subspan(8, n)};
}

// Elofeltetel (ellenorzott): out.size() == 8 + payload.size()
inline void write_frame(std::span<std::byte> out, std::uint32_t kind,
                        std::span<const std::byte> payload)
{
    if (out.size() != 8 + payload.size())
        throw std::length_error{"write_frame: size mismatch"};
    write_be32(out.first(4), kind);
    write_be32(out.subspan(4, 4), static_cast<std::uint32_t>(payload.size()));
    std::copy(payload.begin(), payload.end(), out.begin() + 8);
}

} // namespace wire

int main()
{
    const char text[] = "hello";
    std::span<const std::byte> payload{
        reinterpret_cast<const std::byte*>(text), sizeof text - 1};
```

```
    std::vector<std::byte> buf(8 + payload.size());          // owner
    wire::write_frame(std::span<std::byte>{buf}, 7, payload); // borrowed

    wire::Frame f = wire::parse_frame(std::span<const std::byte>{buf});
    std::printf("kind=%u payload=%zu\n", f.kind, f.payload.size());
}
```

Fordítás és a laborban futtatható diagnosztikai változat (a parancsok a tanuló laborjához tartoznak; itt nem futtattam őket):

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o frame frame.cpp
g++ -std=c++20 -g -O1 -fsanitize=address,undefined -D_GLIBCXX_ASSERTIONS \
    -o frame_asan frame.cpp && ./frame_asan
```

Az azonos szerződés C-ben így néz ki — ugyanazokkal az előfeltételekkel, csak rejtve:

```
/* C17 - reszlet, nem onallo program */
#include <stddef.h>
#include <stdint.h>

struct frame_view { uint32_t kind; const unsigned char *payload; size_t len; };

/* Elofeltetel: buf != NULL && n >= 8; a kimenet a buf-ra mutat. */
int parse_frame(const unsigned char *buf, size_t n, struct frame_view *out);
```

A különbség nem a funkcionalitásban, hanem abban van, hogy a C változatnál a fejlécből nem derül ki sem az, hogy `buf` nem tulajdonló, sem az, hogy `out->payload` élettartama `buf`-hoz kötött.

31.3.8 Gyakori hibák és javításuk

- **`span<const T>` ideiglenesből.** A fenti `danger()` minta lefordul, és dangling pointert ad vissza. Javítás: a `span`-t soha ne tároljuk a mögöttes owner túlélése felett; ha függvény ad vissza nézetet, a szerződés legyen dokumentált („a visszaadott nézet a paraméterre mutat”).
- **Reallokáció utáni használat.** `v.push_back(...)` után a korábban képzett `span` érvénytelen lehet. Javítás: a mutáló művelet után újra képezzük a nézetet, vagy a mutációt és a nézet használatát különítsük el.
- **Hiányzó határellenőrzés.** `s[i]` nem dob kivételt; a `first`/`subspan` előfeltételét magunknak kell ellenőrizni ott, ahol a bemenet külső (hálózat, fájl). A „majd a sanitizer elkapja” nem szerződés.
- **`as_bytes` mint hálózati formátum.** A padding és a byte-sorrend miatt nem hordozható, és padding esetén memóriaszivárgást okozhat. Javítás: mezőnkénti, rögzített sorrendű írás `span<std::byte>`-ba.
- **`span` owner-ként.** `std::span` visszaadása egy lokális `std::vector`-ból nem életképes; ott `std::vector`-t (vagy `array`-t) kell visszaadni, és a hívó kapjon abból nézetet.

31.3.9 Feladatok

1. (alap) Írd meg a `double mean(std::span<const double> xs)` függvényt. Definiáld pontosan az előfeltételét, és magyarázd el, mi történik `mean({})` hívásakor. Adj két hívási példát: egy C tömbbel és egy `std::vector`-ral.

2. (közepes) Írd meg a `void fill_header(std::span<std::byte, 8> out, std::uint32_t kind, std::uint32_t len)` függvényt `first<4>()` és `subspan<4, 4>()` használatával. Mutasd meg, melyik hívás ellenőrzött fordítási időben, melyik futásidőben, és mi történik, ha a hívó 7 elemű buffert ad.

3. (diagnosztikai) Elemezd a következő kódot: melyik művelet invalidál, miért nem jelez a fordító, és milyen eszközzel lehet *esetlegesen* kimutatni a hibát? Javítsd a kódot.

```
std::vector<int> v{1, 2, 3, 4};
std::span<const int> s{v};
v.push_back(5);
int x = s[0];
```

4. (byte) Egy kollégád `::write(fd, std::as_bytes(std::span{header}).data(), sizeof header)`-rel küld egy `struct header`-t a hálózaton. Sorolj fel legalább három okot, amiért ez nem hordozható, és vázold fel a javítást.

31.3.10 Megoldások

1. megoldás. Az előfeltétel: `!xs.empty()`, mert üres tartományra a `sum / 0` nem értelmezett eredményt ad. Mivel a `span` nem ellenőriz, a függvénynek magának kell döntenie:

```
#include <span>
#include <stdexcept>

double mean(std::span<const double> xs)
{
    if (xs.empty()) throw std::invalid_argument{"mean: empty"};
    double sum = 0.0;
    for (double x : xs) sum += x;
    return sum / static_cast<double>(xs.size());
}
```

`mean({})` egy üres `std::span<const double>`-t ad át (`size() == 0`, a `data()` értéke nem használható), és a függvény a kivétellel jelzi a szerződésszegést, nem nem definiált viselkedéssel. Hívás C tömbbel: `double a[3]{1,2,3}; mean(a);` — a tömb→span konverzió a méretet is átveszi. Hívás vektorral: `std::vector<double> v{...}; mean(v);`. *Ellenőrzés:* a program üres vektorral kivételt dob, nem összeomlik; a fordító `-Wall -Wextra` mellett nem figyelmeztet.

2. megoldás.

```
#include <cstddef>
#include <cstdint>
#include <span>

void fill_header(std::span<std::byte, 8> out, std::uint32_t kind, std::uint32_t
 ↪  len)
{
    auto put = [](std::span<std::byte> b, std::uint32_t v) {
        b[0] = static_cast<std::byte>(v >> 24);
        b[1] = static_cast<std::byte>(v >> 16);
        b[2] = static_cast<std::byte>(v >> 8);
        b[3] = static_cast<std::byte>(v);
    };
```

```
    put(out.first<4>(), kind);
    put(out.subspan<4, 4>(), len);
}
```

A `first<4>()` és a `subspan<4, 4>()` statikus kiterjedésen hívva fordítási időben ellenőrződik: ha `4 > 8` lenne, `static_assert` hibát kapnánk. `std::array<std::byte, 8>` esetén a hívás implicit konverzióval működik. `std::vector<std::byte>`-ból a `std::span<std::byte, 8>` konstrukció explicit, és a `size() == 8` előfeltétel futásidejű: hét elemű vektorral a viselkedés nem definiált, ezért a hívás előtt `if (v.size() != 8) throw ...;` kell. *Ellenőrzés:* `static_assert(std::span<std::byte, 8>::extent == 8);` és a 7 elemű eset kivétellel, ne összeomlással végződjön.

3. megoldás. A `push_back` akkor allokál újra, ha `size() == capacity()`; ekkor a régi buffer felszabadul, a `span`-ban tárolt pointer és méret viszont változatlan marad — a nézet dangling lesz. A fordító azért nem jelez, mert a `span` egy egyszerű érték pointerrel és mérettel; a nyelv nem követi nyomon, hogy egy pointer melyik tárolóra mutat. Az ASan *esetleg* jelez (ha a felszabadított blokk még karanténban van), de ez nem garantált, és ha a `capacity()` elég volt, egyáltalán nincs invalidálás. Javítás: a mutáció után újra kell képezni a nézetet, vagy a nézetet csak egy nem-mutáló szakaszban szabad használni:

```
std::vector<int> v{1, 2, 3, 4};
v.push_back(5);                          // előbb a mutáció
std::span<const int> s{v};               // utána a nézet
int x = s[0];
```

Ellenőrzés: az eredeti változat ASan + `_GLIBCXX_ASSERTIONS` alatt vagy jelez, vagy nem — a hiányzó jelentés nem bizonyíték a helyességre; a javított változat viszont nem tartalmaz invalidált nézetet.

4. megoldás. (a) A struct tartalmazhat kitöltő byte-okat, amelyek értéke nem specifikált — ezek akár korábbi memóriatartalmat is kijuttathatnak. (b) A `uint32_t` mezők byte-sorrendje implementációfüggő, így a fogadó fél mást olvas. (c) A struct mérete, igazítása és a mezők elrendezése target ABI-függő, tehát `sizeof header` és a mezőoffszetek gépenként eltérhetnek. Javítás: a fejlécet mezőnként, rögzített (például big-endian) sorrendben írjuk egy `std::span<std::byte>` célbufferbe, és a protokoll leírása rögzítse a mezők sorrendjét, szélességét és sorrendjét; a `as_bytes` maradjon platformon belüli, nem hálózati használatra.

31.3.11 Összefoglaló

A `std::span` a `(T*, size_t)` pár típusos, nem tulajdonló C++ megfelelője: megnevezi az elemtípust és a méretet, de az owner-, folytonosság- és élettartam-szerződést a hívóra bízza. A statikus kiterjedés a méretet a típus részévé teszi, így a résznézetek (`first<N>`, `subspan<O, C>`) gyakran fordítási időben ellenőrizhetők; az `operator[]` viszont előfeltétel, nem ellenőrzött művelet, és nincs `at()` a C++20-as `span`-ban. A `std::byte` és az `as_bytes`/`as_writable_bytes` az objektumreprezentációt teszik elérhetővé, ami padding és byte-sorrend miatt nem hordozható formátum: sorosításhoz mezőnkénti, rögzített írás kell. A `span` és a `string_view` rokon célú, de más felhasználású nézetek: az előbbi általános, akár írható memóriatartomány, az utóbbi csak olvasható szöveg. A legfontosabb élettartam-szabály: a nézet soha nem élhet túl egy olyan műveletet, amely a mögöttes tárolót invalidálja — reallokáció, `erase`, `clear` vagy a tároló megszűnése után a `span` újra képezendő, nem „újrahasznosítható”.

31.4 Bitműveletek és objektumreprezentáció C++-ban

A korábbi byte order és biztonságos szerializálás rész már rögzítette, hogy a wire formátumot nem a natív `struct` layout írja le. A mostani rész ezt a C++20 `<bit>` eszközeivel pontosítja: a flagkészletet típusbiztos enum mögé tesszük, a reprezentáció-átalakítást `std::bit_cast`-ra vagy explicit byte-összerakásra bízzuk, a számlálást és forgatást a standard `<bit>` függvényeire, az endianness-kezelést pedig `std::endian`-re és `std::byteswap`-re. A span, byte és bounds-szerződések rész nézetfogalmára építünk: a `std::span` nem tulajdonol, a `std::bitset` viszont igen; a type traits pedig a feltételek fordítás idejű ellenőrzéséhez kell.

31.4.1 Bitmask enum és bitset

C-ben a flagkészlet tipikus alakja `enum { PERM_READ = 1 << 0, PERM_WRITE = 1 << 1 };`. Ez működik, de az enumerátorok típusa `int`, a `PERM_READ | PERM_WRITE` eredménye is `int`, a `~PERM_READ` pedig negatív; a függvényhatáron implicit konverziók és előjelproblémák keletkeznek. C++20-ban a scoped enum fix underlying type-pal adja a méretet és a tartományt:

```
enum class Perm : std::uint8_t {
    none  = 0,
    read  = 1u << 0,
    write = 1u << 1,
    exec  = 1u << 2,
};
```

A szabvány scoped enumra nem ad beépített bitwise operátort, ezért a „bitmask enum idiom” része a műveletek definiálása. A `|`, `&`, `^` és `~` operátorokat `constexpr` és `noexcept` függvényként adjuk meg, a `~` eredményét pedig az underlying type szélességére maszkoljuk, hogy ne keletkezzen előjeles értelmezés:

```
constexpr Perm operator|(Perm a, Perm b) noexcept {
    return static_cast<Perm>(
        static_cast<std::uint8_t>(a) | static_cast<std::uint8_t>(b));
}
constexpr bool has(Perm set, Perm bit) noexcept {
    return (static_cast<std::uint8_t>(set) &
            static_cast<std::uint8_t>(bit)) != 0;
}
```

C++23-tól a `std::to_underlying` kiváltja a `static_cast`-ot; C++20-as fallbackként marad a cast. A `std::bitset<N>` más eset: `N` fordítási idejű, a tárolót a bitset tulajdonolja, és nem nevesített flagkészletet, hanem tetszőleges bitsorozatot kezel. `count()`, `test()`, `set()`, `flip()` és `operator[]` áll rendelkezésre; az `operator[]` proxyreferenciát ad, a `to_ullong()` pedig `std::overflow_error`-t dob, ha a bitsorozat nem fér el. A bitset nagy része C++23-ban lett `constexpr`; C++20-ban ezt ne feltételezzük. Nevesített protokollflagre maradjunk az enum idiómánál, ismeretlen szélességű bitkészletre pedig bitsetnél.

31.4.2 std::bit_cast: reprezentáció, nem szemantika

A `std::bit_cast` C++20-as, `<bit>` headerben található függvénysablon:

```
template<class To, class From>
constexpr To bit_cast(const From& from) noexcept;
```

Feltételei: `sizeof(To) == sizeof(From)`, mindkét típus triviálisan másolható, a `From` nem bit-field, és a `From` objektumreprezentációjához tartozzék `To`-beli érték. Ha nincs ilyen érték, a viselkedés nem definiált. A nem értékreprezentációs bitek — például `struct`-padding — a resultspecifikáció szerint nem specifikáltak; az ilyen eredményt ne hasonlítsuk, ne hasheljük, és ne tekintsük wire formátumnak.

A legfontosabb szemantikai különbség: a `bit_cast` nem numerikus konverzió. Nem kerekít, nem trunkiál, nem változtatja meg a bitmintát. A `static_cast<int>(2.9)` értéket konvertál (2), a `bit_cast<std::uint32_t>(2.9f)` a `2.9f` reprezentációját adja vissza. A `static_cast<bool>(x)` az `x != 0` szemantikát jelenti; a `bit_cast<bool>(x)` bitmintát másol, és ha a minta nem 0/1, nincs megfelelő `bool` érték, tehát UB. Ugyanez a helyzet egy szűk értéktartományú, fix underlying type nélküli enummal: nem minden bitminta tartozik értékhez.

```
static_assert(sizeof(float) == sizeof(std::uint32_t));
static_assert(std::numeric_limits<float>::is_iec559);
constexpr std::uint32_t bits = std::bit_cast<std::uint32_t>(1.5f);
static_assert(bits == 0x3FC00000u); // IEC 559 binary32
```

A `bit_cast` ugyanazt a byte-másolást végzi, mint a `memcpy` triviálisan másolható típusok között, de típusbiztos, fordítás idejű méretellenőrzést ad, és `constexpr`. A unionos type punninggel szemben nem olvas inaktív tagot; a `reinterpret_cast<T*>(buf)` és dereferencia ezzel szemben nem hoz létre `T` objektumot, és strict aliasing / lifetime szempontból UB. A `bit_cast` viszont új `To` objektumot ad vissza, nem nézetet.

31.4.3 Számlálás, forgatás és a `<bit>` eszközei

A C++20 `<bit>` header portable, `constexpr` és `noexcept` függvényeket ad. Ezek csak unsigned integer típusra mennek; signed típusra `std::make_unsigned_t<T>`-vel konvertáljunk.

- `std::popcount(x)`: az 1-bitek száma.
- `std::countl_zero(x)` / `std::countr_zero(x)`: vezető, illetve záró 0-bitek száma; `x == 0` esetén a típus szélességét adja.
- `std::countl_one(x)` / `std::countr_one(x)`: vezető, illetve záró 1-bitek száma.
- `std::has_single_bit(x)`: pontosan egy 1-bit van-e (azaz kettőhatvány-e).
- `std::bit_width(x)`: `1 + floor(log2(x))`, nullára 0; `std::bit_ceil(x)` a legkisebb kettőhatvány, amely legalább `x`; `std::bit_floor(x)` a legnagyobb kettőhatvány, amely legfeljebb `x`.
- `std::rotl(x, s)` / `std::rotr(x, s)`: körkörös forgatás; a gyakorlatban `s`-et a típus szélességére normalizáljuk.

A `bit_ceil` előfeltétele, hogy az eredmény ábrázolható legyen; túl nagy `x` esetén UB. Alignment-ellenőrzésre `std::countr_zero` hasznos: egy `std::uintptr_t` cím akkor 16-byte aligned, ha `countr_zero(p) >= 4`; a nulla cím matematikailag „aligned”, de dereferálni természetesen nem szabad. A GCC `__builtin_popcount` és társai extensionök; a standard változat hordozható, de ugyanazt a gépi utasítást használhatja.

31.4.4 Endianness: `std::endian` és `std::byteswap`

A `std::endian` C++20-as, fordítás idejű enum: `little`, `big` és `native`. A `native` lehet `little` vagy `big`, de a szabvány megengedi azt is, hogy egyik se legyen (mixed-endian). Ezért ne feltételezzük vakon a little-endiant:

```cpp
if constexpr (std::endian::native == std::endian::little) {
    // natív little-endian optimalizáció
} else if constexpr (std::endian::native == std::endian::big) {
    // natív big-endian optimalizáció
} else {
    // mixed-endian: csak explicit byte-assembly
}
```

A `std::byteswap` C++23-as, `<bit>` headerben; integrál típusok byte-jait fordítja meg, `bool`-ra nem alkalmazható. Feature-testje `__cpp_lib_byteswap >= 202110L`. C++20-as fallbackként kézi shift vagy compiler intrinsic (`__builtin_bswap32`) jöhet szóba. A wire séma azonban ne a natív endiannesshez igazodjon: a protokoll rögzítse a bájtsorrendet (a hálózati byte order hagyományosan big-endian), a kód pedig mindig expliciten állítsa elő a byte-okat.

```cpp
constexpr std::array<std::byte, 4> be32(std::uint32_t v) noexcept {
    return {std::byte(v >> 24), std::byte(v >> 16),
            std::byte(v >> 8),  std::byte(v)};
}
constexpr std::uint32_t from_be32(std::span<const std::byte, 4> b) noexcept {
    return (std::to_integer<std::uint32_t>(b[0]) << 24) |
           (std::to_integer<std::uint32_t>(b[1]) << 16) |
           (std::to_integer<std::uint32_t>(b[2]) << 8)  |
            std::to_integer<std::uint32_t>(b[3]);
}
```

A `std::endian` legfeljebb optimalizálásra vagy natív buffer gyors átalakítására kell; a wire séma ettől független szerződés.

31.4.5 memcpy, élettartam és típus-hozzáférés

A `std::memcpy` akkor definiált értékmegőrző másolásra, ha a cél egy triviálisan másolható objektum, és a forrás érvényes objektumreprezentáció. Nem triviálisan másolható osztálynál a byte-másolás nem indít élettartamot. A C++20 implicit-lifetime szabályai bizonyos esetekben — `malloc`, `memcpy`, `bit_cast` — mégis létrehoznak implicit-lifetime típusú objektumot a tárolóban. Implicit-lifetime típus a skalár, az olyan osztály, amelynek van legalább egy triviális, nem törölt konstruktora és triviális destruktora (az aggregátum is ilyen), valamint ezek tömbje. Ha az osztálynak felhasználói konstruktora vagy destruktora van, a `memcpy` nem elég; konstruktor kell. C++23-ban a `std::start_lifetime_as<T>` / `start_lifetime_as_array` explicit módon indít élettartamot; feature-testje `__cpp_lib_start_lifetime_as >= 202207L`, és csak akkor használjuk, ha a fordítási profil támogatja.

Típus-hozzáférés szempontjából `char`, `unsigned char` és `std::byte` bármely objektum reprezentációját olvashatja és írhatja. Más típuson keresztül csak akkor, ha ott tényleg létezik ilyen objektum. A `reinterpret_cast<FrameHeader*>(buf.data())` ezért nem teremt `FrameHeader` objektumot, és a `std::byte` tömb alignmentje 1, míg a `FrameHeader`-é jellemzően 4: a dereferencia misaligned és lifetime szempontból is UB lehet. A `std::bit_cast` és az explicit byte-összerakás kerüli ezt.

31.4.6 Kidolgozott példa: a C codec fejléce

A wire forma 8 byte, rögzített mezőkkel:

Offset	Méret	Mező	Wire kódolás
0	4	`magic`	`uint32`, big-endian, `0x46524D31` (`"FRM1"`)
4	2	`length`	`uint16`, big-endian
6	1	`flags`	`uint8` bitmask (`Perm`)
7	1	`version`	`uint8`

C-ben ezt jellemzően `memcpy`-vel vagy `unsigned char*` pásztázással olvastuk. A C++ változat explicit byte-assemblyt és `enum class`-t használ; a `sizeof(FrameHeader)` nem szerződés, ezért nem `memcpy`-zük a structot.

```
// codec.cpp - C++20
#include <array>
#include <bit>
#include <cassert>
#include <cstddef>
#include <cstdint>
#include <limits>
#include <optional>
#include <span>

enum class Perm : std::uint8_t {
    none = 0, read = 1u << 0, write = 1u << 1, exec = 1u << 2,
};
constexpr Perm operator|(Perm a, Perm b) noexcept {
    return static_cast<Perm>(static_cast<std::uint8_t>(a) |
                             static_cast<std::uint8_t>(b));
}
constexpr bool has(Perm set, Perm bit) noexcept {
    return (static_cast<std::uint8_t>(set) &
            static_cast<std::uint8_t>(bit)) != 0;
}

inline constexpr std::uint32_t kMagic = 0x46524D31u; // "FRM1"

struct FrameHeader {
    std::uint32_t magic = kMagic;
    std::uint16_t length = 0;
    Perm flags = Perm::none;
    std::uint8_t version = 1;
};

constexpr std::array<std::byte, 8> encode(const FrameHeader& h) noexcept {
    return {std::byte(h.magic >> 24), std::byte(h.magic >> 16),
            std::byte(h.magic >> 8),  std::byte(h.magic),
            std::byte(h.length >> 8), std::byte(h.length),
            std::byte(static_cast<std::uint8_t>(h.flags)),
            std::byte(h.version)};
}

constexpr std::optional<FrameHeader>
decode(std::span<const std::byte, 8> in) noexcept {
```

```
    const auto u32 = [&](std::size_t i) {
        return (std::to_integer<std::uint32_t>(in[i])     << 24) |
               (std::to_integer<std::uint32_t>(in[i + 1]) << 16) |
               (std::to_integer<std::uint32_t>(in[i + 2]) << 8)  |
                std::to_integer<std::uint32_t>(in[i + 3]);
    };
    const auto u16 = [&](std::size_t i) {
        return static_cast<std::uint16_t>(
            (std::to_integer<std::uint16_t>(in[i]) << 8) |
             std::to_integer<std::uint16_t>(in[i + 1]));
    };
    if (u32(0) != kMagic) return std::nullopt;
    FrameHeader h{};
    h.length  = u16(4);
    h.flags   = static_cast<Perm>(std::to_integer<std::uint8_t>(in[6]));
    h.version = std::to_integer<std::uint8_t>(in[7]);
    return h;
}

int main() {
    static_assert(sizeof(float) == sizeof(std::uint32_t));
    static_assert(std::numeric_limits<float>::is_iec559);
    constexpr auto fbits = std::bit_cast<std::uint32_t>(1.5f);
    static_assert(fbits == 0x3FC00000u);

    FrameHeader h{};
    h.length = 0x1234;
    h.flags = Perm::read | Perm::write;
    h.version = 2;

    const auto wire = encode(h);
    const auto back = decode(wire);
    assert(back.has_value());
    assert(back->length == h.length);
    assert(back->version == h.version);
    assert(has(back->flags, Perm::read));
    assert(has(back->flags, Perm::write));
    assert(!has(back->flags, Perm::exec));
}
```

Fordítás a laborban:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 codec.cpp -o codec
```

A példa C++20-at igényel; a `std::byteswap` és a `std::start_lifetime_as` külön C++23-es képesség, feature-testtel ellenőrizendő. A kód a wire byte-okat expliciten állítja elő, ezért a natív endianness és a `sizeof(FrameHeader)` nem befolyásolja a protokollt. A `static_assert`-ek a szerződést rögzítik, nem helyettesítik a futtatott teszteket.

31.4.7 Tipikus hibák

- `std::bit_cast<bool>` tetszőleges `uint8_t`-ből: a 0/1-en kívüli minta UB.

- `struct` paddingjának `bit_cast`-olása, majd `memcmp`-pel hasonlítás: a padding bitek nem specifikáltak.
- `reinterpret_cast<FrameHeader*>(buf.data())` és dereferencia: nincs ott `FrameHeader` objektum, és az alignment is hibás lehet.
- `std::byteswap` használata C++20-ban feature-test nélkül: a függvény nem feltétlenül létezik.
- `std::popcount` signed típusra: nem fordul; előbb `std::make_unsigned_t`-vé kell alakítani.
- `std::bit_ceil` túl nagy argumentumra: az eredmény nem ábrázolható, UB.
- A natív `struct` layout és a wire séma összemosása: a kettő nem ugyanaz a szerződés.

31.4.8 Összefoglaló

A bitmaskot scoped enum mögé tesszük, a műveleteket `constexpr` függvényekként definiáljuk; a `std::bitset` a nem nevesített, fix szélességű bitsorozat eszköze. A `std::bit_cast` reprezentációt másol, nem értéket konvertál; feltétele a méret-egyezés, a triviális másolhatóság és a megfelelő `To`-érték létezése. A `<bit>` portable számlálást, forgatást és kettőhatvány-kezelést ad. Az endianness wire-szinten explicit séma, nem a `std::endian::native` függvénye. A `memcpy` csak triviálisan másolható objektumra és érvényes élettartam mellett biztonságos; a `reinterpret_cast` + dereferencia helyett `bit_cast` vagy explicit byte-dekódolás kell.

31.4.9 Feladatok

1. **Magyarázó feladat.** Magyarázd meg, miért UB a `std::bit_cast<bool>(std::uint8_t{2})`, és miért definiált a `std::bit_cast<std::uint32_t>(1.5f)` egy IEC 559 `float` esetén. Milyen fordítás idejű ellenőrzéseket tennél a kódba?
2. **Kódolási feladat.** Írj `constexpr std::uint32_t bswap32(std::uint32_t)` függvényt, amely C++20-ban portable, C++23-ban pedig `std::byteswap`-et használ, ha elérhető. Bizonyítsd `static_assert`-ekkel.
3. **Diagnosztikai feladat.** Az alábbi kód egy `std::array<std::byte, 8>` bufferből közvetlenül `FrameHeader*`-t készít. Sorolj fel legalább három önálló problémát, és javítsd a kódot.

```
std::array<std::byte, 8> buf = receive();
auto* h = reinterpret_cast<FrameHeader*>(buf.data());
if (h->magic == kMagic) { use(h->length, h->flags); }
```

31.4.10 Megoldások

31.4.10.1 1. feladat megoldása

A `bool` értékkészlete `false` és `true`; a `0x02` bitmintához nem tartozik `bool` érték, ezért a `bit_cast` feltétele sérül, és a viselkedés UB. A `float` IEC 559 binary32 esetén minden 32 bites minta értelmezhető (véges, végtelen vagy NaN), és `sizeof(float) == 4`, így a `uint32_t` felé van megfelelő érték. Ellenőrzés: `static_assert(sizeof(float) == sizeof(std::uint32_t));` és `static_assert(std::numeric_limits<float>::is_iec559);`. Nem IEC 559 vagy eltérő méret esetén a `bit_cast` nem használható erre a célra; ott marad az explicit byte-kezelés. A megoldás helyességi feltétele, hogy a `float` mérete és IEC 559 volta fordítás időben rögzítve legyen.

31.4.10.2 2. feladat megoldása

```
#include <bit>
#include <cstdint>

constexpr std::uint32_t bswap32(std::uint32_t x) noexcept {
#if defined(__cpp_lib_byteswap) && __cpp_lib_byteswap >= 202110L
    return std::byteswap(x);
#else
    return ((x & 0x000000FFu) << 24) |
           ((x & 0x0000FF00u) << 8)  |
           ((x & 0x00FF0000u) >> 8)  |
           ((x & 0xFF000000u) >> 24);
#endif
}

static_assert(bswap32(0x11223344u) == 0x44332211u);
static_assert(bswap32(0x00000000u) == 0x00000000u);
static_assert(bswap32(0xFFFFFFFFu) == 0xFFFFFFFFu);
```

Az elágazás fordítás idejű: C++23-ban a `__cpp_lib_byteswap` makró definiált, C++20-ban nem. A kézi változat minden byte-ot maszkolással emel ki, ezért nem függ a natív endiannesstől, és nem használ signed shiftet. Ellenőrzési feltétel: a három `static_assert` mellett érdemes 16 és 64 bites típusra is általánosítani, és `std::uint8_t`-re külön ellenőrizni, hogy a `byteswap` ne kapjon `bool`-t.

31.4.10.3 3. feladat megoldása

Problémák: (1) a `buf` tárolójában nem létezik `FrameHeader` objektum, a `reinterpret_cast` nem indít élettartamot, a dereferencia strict aliasing és lifetime szempontból UB; (2) a `std::byte` tömb alignmentje 1, a `FrameHeader`-é jellemzően 4, így a pointer misaligned lehet, és a dereferencia alignment-UB; (3) a wire big-endian, a natív gép little-endian lehet, ezért a `magic`, `length` és `flags` értelmezése is hibás; (4) a `sizeof(FrameHeader)` nem garantáltan 8, a struct paddingot tartalmazhat, ezért a nyers struct-olvasás még akkor sem hordozható, ha véletlenül egyezik a méret. Javítás: a `decode(std::span<const std::byte, 8>)` függvény expliciten rakja össze a byte-okat, ellenőrizze a magicet, és csak utána használja a mezőket:

```
std::array<std::byte, 8> buf = receive();
if (const auto h = decode(buf)) {
    use(h->length, h->flags);
}
```

A `decode` a `std::to_integer`-rel olvas, nem végez misaligned dereferenciát, és a wire endianness-hez igazodik. Ellenőrzési feltétel: a javított kód nem tartalmaz `reinterpret_cast`-ot a bufferre, a `magic` ellenőrzés a dekódolás része, és a hossz/flag mezők csak sikeres dekódolás után használhatók. A laborban az alignment- és UB-sanitizer a régi változatnál jelezhet; a javított változat tervezetten nem végez ilyen műveletet.

31.5 Multidimenziós view és numeric utility térkép

Ez az alfejezet arról szól, hogyan adjunk többdimenziós indexelést egy folytonos pufferre anélkül, hogy elveszítenénk a tulajdonlás és az élettartam tisztaságát. A kiindulópont a C mátrixlayout, amelyet a Többdimenziós tömbök, VLA és layout részben már rögzítettünk: a C tömbök elemei garantáltan összefüggően helyezkednek el, és a tömb típusa maga kódolja a kiterjedéseket. A C++20 utáni eszköztár ugyanezt a leképezést választja szét három független fogalomra: a kiterjedésre (extent), a lépésre (stride) és a hozzáférésre (access). A fejezet célja, hogy az olvasó a mátrixnézetet ne „mátrixosztályként”, hanem indexleképezési szerződésként tudja megtervezni, és világosan lássa, mikor elérhető a standard `std::mdspan`, és mi a teendő C++20-on.

31.5.1 Extent, stride és az indexelés szerződése

Egy n-dimenziós, $E_0 \times E_1 \times \cdots \times E_{n-1}$ kiterjedésű tömbnél az *extent* az egyes dimenziók elemszáma, a *stride* pedig azt mondja meg, hány elemnyit kell ugrani, ha az adott indexet eggyel növeljük. Általános esetben a lineáris eltolás

$$\text{offset}(i_0, \ldots, i_{n-1}) = \sum_{k=0}^{n-1} i_k \cdot s_k,$$

ahol s_k a k-adik dimenzió lépése. A *row-major* (sorfolytonos) elrendezésben az utolsó dimenzió lépése 1, és $s_k = \prod_{j>k} E_j$; a *column-major* (oszlopfolytonos) elrendezésben $s_0 = 1$ és $s_k = \prod_{j<k} E_j$. Kétdimenziós esetben ez a két jól ismert képletet adja:

$$\text{offset}_{RM}(i, j) = i \cdot E_1 + j, \qquad \text{offset}_{CM}(i, j) = i + j \cdot E_0.$$

A szerződés feltételei: (1) minden $0 \le i_k < E_k$; (2) a számított offset kifejezhető abban az indextípusban, amellyel számolunk; (3) a hivatkozott elem ténylegesen létezik abban a tárolóban, amelyre a nézet mutat. Az (1) megsértése nem definiált viselkedés (undefined behavior, UB), nem pedig „rossz érték”. A (2) feltétel a gyakorlatban a legtöbbször figyelmen kívül hagyott pont, erről külön alszakasz szól.

C-ben a többdimenziós tömb típusa ezt a leképezést beégeti. A következő példa a `-std=c17` profilon mutatja, hogy a layout nem konvenció, hanem nyelvi garancia:

```
/* c_matrix.c -- gcc -std=c17 -Wall -Wextra -pedantic c_matrix.c -o c_matrix */
#include <stddef.h>
#include <stdio.h>

enum { ROWS = 3, COLS = 4 };
static int a[ROWS][COLS];

int main(void)
{
    /* a tomb ROWS*COLS egymas utani int elembol all, soronkent folytonosan:
       a[i][j] indexe i*COLS + j (a reprezentacio szintjen), de a tipus
 ↪  ketdimenzios */
    size_t total = sizeof a / sizeof a[0][0];
    /* a TELJES a objektum reprezentaciojan belul szamolunk: base az a elso
       bajtja, az eltolas a sorfolytonos layoutbol (i*COLS + j) * sizeof(int)
 ↪  */
```

```
    const unsigned char *base = (const unsigned char *)&a;
    size_t off = (1u * COLS + 2u) * sizeof a[0][0];
    /* csak egyenloseget vizsgalunk, kulon reszobjektumokat nem vonunk ki */
    int same = (const void *)(base + off) == (const void *)&a[1][2];
    printf("total=%zu off(1,2)=%zu same=%d\n", total, off / sizeof a[0][0],
 ↪  same);
    return 0;
}
```

Buildjegyzőkönyv (`lab-gcc` profil). A kimenet `total=12 off(1,2)=6 same=1`: a row-major képletből $(1 \cdot 4 + 2)$ kiszámolt bájteltolás a teljes `a` objektum első bájtjától pontosan az `a[1][2]` elemre mutat. A példa minden pointerművelete a teljes `a` objektum reprezentációján belül marad (`unsigned char` pointer az `a` elejétől, legfeljebb `sizeof a` bájtnyira), és nem von ki két külön részobjektumra mutató pointert; `int*` aritmetika sem lép át sorhatárt.

A C tömbmodell két korlátját érdemes szétválasztani. A **futásidejű méret** kifejezhető: a VLA-ból származó, változóan módosított (VM) típusokkal egy `size_t cols` paraméterrel megadott `int (*)[cols]` pointer futásidejű sorhosszú mátrixot ír le (a részleteket és a C11/C17/C23 támogatási szabályokat a Többdimenziós tömbök, VLA és layout (7.2) tárgyalja):

```
/* vla_matrix.c -- gcc -std=c17 -Wall -Wextra -pedantic vla_matrix.c -o
 ↪  vla_matrix */
#include <errno.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

#define MAX_COLS 4096u            /* felso eroforraskorlat: sor <= 4096 int */

/* futasideju meretu matrix: a parameter tipusa variably modified, int (*)[cols]
 ↪  */
static void fill(size_t rows, size_t cols, int (*m)[cols])
{
    for (size_t i = 0; i < rows; ++i)
        for (size_t j = 0; j < cols; ++j)
            m[i][j] = (int)(i * cols + j);
}

int main(int argc, char **argv)
{
    size_t rows = 3, cols = 4;
    if (argc > 1) {                           /* ellenorzott
     ↪  szamszoveg-feldolgozas */
        char *end = NULL;
        errno = 0;
        unsigned long v = strtoul(argv[1], &end, 10);
        if (errno != 0 || end == argv[1] || *end != '\0' || argv[1][0] == '-'
            || v < 3 || v > MAX_COLS) {       /* cols >= 3: kulonben m[1][2]
             ↪  soron kivuli */
            fprintf(stderr, "hibas oszlopszam (3..%u)\n", MAX_COLS);
            return 2;
        }
        cols = (size_t)v;
```

```
    }
    if (cols > SIZE_MAX / sizeof(int) / rows) return 2;   /* szorzas-ellenorzes
     ↪  a VM tipus elott */
    int (*m)[cols] = malloc(rows * sizeof *m);   /* heapen, nem VLA-objektum */
    if (!m) return 1;
    fill(rows, cols, m);
    printf("m[1][2]=%d sizeof *m=%zu\n", m[1][2], sizeof *m);
    free(m);
    return 0;
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `gcc -std=c17 -Wall -Wextra -pedantic vla_matrix.c` figyelmeztetés nélkül fordul; argumentum nélkül `m[1][2]=6 sizeof *m=16`, 5-tel `m[1][2]=7 sizeof *m=20` a kimenet; a `0`, `1`, `2`, `-1`, `abc`, `99999999999999999` és a `4097` bemenetet szabályosan elutasítja (2-es kilépési kód, hibaüzenet), így a `m[1][2]` mindig érvényes elem. Ami a C tömbtípusokkal **nem** fejezhető ki, az a tetszőleges lépésközű (stride-os) alterület: egy mátrix 2×2-es belső blokkjára nem adható olyan tömbtípus, amelynek sorlépése a teljes mátrix sorhossza, de kiterjedése a blokké. Ezt a hiányt tölti be a nézet (view).

31.5.2 C++23 mdspan: extents, layout és accessor

A `std::mdspan` egy nem tulajdonló, többdimenziós nézet. A típus négy paramétere négy külön döntés: `std::mdspan<T, Extents, LayoutPolicy, AccessorPolicy>`. Az `Extents` lehet statikus (`std::extents<std::size_t, 3, 4>`) vagy dinamikus (`std::dextents<std::size_t, 2>`); dinamikus kiterjedés esetén az értékek a konstruktorba kerülnek. A `LayoutPolicy` felel az offset-képletért: `std::layout_right` a row-major, `std::layout_left` a column-major, `std::layout_stride` pedig tetszőleges, futásidejű lépésvektort enged meg — ez utóbbi az alterület-nézetek és a transzponált nézetek eszköze. Az `AccessorPolicy` (alapértelmezésben `std::default_accessor<T>`) azt dönti el, hogy egy offsetből milyen hivatkozás lesz; saját accessorral például másoló, számított vagy maszkolt hozzáférést lehet adni.

A fontos szemantikai korlát: az `mdspan` nem owner. A mögötte álló *backing storage*-nak túl kell élnie a nézetet, és a nézet létrehozása nem hosszabbítja meg annak élettartamát. Ha a nézetet egy lokális `std::vector`-ra építjük és kifelé adjuk, dangling nézetet kapunk; ez ugyanaz a hibaosztály, mint a span, byte és bounds-szerződések részben tárgyalt `std::span` esetén, csak több dimenzióval. A másik kötelező előfeltétel: az accessor által hivatkozott elemeknek léteznie kell, azaz a legnagyobb offset is a lefoglalt tartományon belül kell legyen; ennek megsértése UB, nem kivétel.

Az `mdspan` a C++23 könyvtár része, ezért elérhetőségét nem szabad feltételezni. A helyes minta a feature-test macro ellenőrzése a `<version>` headerből, még a `<mdspan>` include előtt:

```
// mdspan_demo.cpp -- g++ -std=c++23 -O2 -Wall -Wextra mdspan_demo.cpp -o
 ↪  mdspan_demo
#include <version>
#include <cstdio>                    // mindkét ág használja (std::printf /
 ↪  std::puts)
#if defined(__cpp_lib_mdspan)
#include <cstddef>
#include <vector>
#include <mdspan>

int main()
```

```
{
    std::vector<double> storage(12, 0.0);          // 3 x 4, row-major
    using ext2 = std::dextents<std::size_t, 2>;

    std::mdspan<double, ext2> right(storage.data(), 3, 4);            //
     ↪  layout_right
    std::mdspan<double, ext2, std::layout_left> left(storage.data(), 3, 4);

    right[1, 2] = 5.0;                                 // C++23 tobbargumentumu
 ↪  operator[]
    std::printf("right[1,2]=%f left[1,2]=%f\n", right[1, 2], left[1, 2]);
    return 0;
}
#else
int main()
{
    std::puts("mdspan nem elerheto ezen a toolchain-en");
    return 0;
}
#endif
```

Két megjegyzés a példához. Egyrészt a `right[1,2]` többargumentumú `operator[]` szintén C++23-as nyelvi elem; C++20-ban a többdimenziós indexeléshez függvényhívás-operátor kell. Másrészt a két nézet ugyanarra a pufferre mutat, ezért a `left[1,2]` más offsetre esik ($1+2\cdot 3=7$, szemben a row-major $1\cdot 4+2=6$-tal): ez nem hiba, hanem a layout-policy hatása, de ugyanazon elemek két különböző indexelésű írása könnyen logikai hibává válik. Éles kódban ne tartsunk egyszerre két, eltérő layoutú írható nézetet ugyanarra a pufferre.

31.5.3 C++20 fallback: span és stride-alapú saját nézet

Ha a toolchain nem kínál `mdspan`-t, a szerződés kézzel is felírható. A minimum egy nyers pointer, két kiterjedés és egy sorlépés; a bounds-ellenőrzés `assert`-tel afelett van, és a lépést elemszámban (nem byte-ban) érdemes tárolni, mert így a pointeraritmetika a fordítóra marad. A `std::span` önmagában nem elég, mert csak egydimenziós és folytonos; a stride-ot külön kell vinni.

```
// matrix_view.cpp -- g++ -std=c++20 -O2 -Wall -Wextra -pedantic matrix_view.cpp
 ↪  -o matrix_view
#include <cassert>
#include <cstddef>
#include <cstdio>
#include <limits>
#include <vector>

template <class T>
class MatrixView2D {
public:
    MatrixView2D(T* data, std::size_t rows, std::size_t cols, std::size_t
 ↪  stride)
        : data_(data), rows_(rows), cols_(cols), stride_(stride)
    {
        assert(data_ != nullptr || rows_ == 0 || cols_ == 0);
        // a legnagyobb offset: (rows-1)*stride + (cols-1) -- nem csordulhat at
```

```
        if (rows_ > 0 && cols_ > 0) {
            const std::size_t r = rows_ - 1, c = cols_ - 1;
            const std::size_t max = std::numeric_limits<std::size_t>::max();
            assert(stride_ == 0 || r <= (max - c) / stride_);
        }
    }
    T& operator()(std::size_t r, std::size_t c)
    {
        assert(r < rows_ && c < cols_);
        return data_[r * stride_ + c];
    }
    const T& operator()(std::size_t r, std::size_t c) const
    {
        assert(r < rows_ && c < cols_);
        return data_[r * stride_ + c];
    }
    std::size_t rows() const { return rows_; }
    std::size_t cols() const { return cols_; }
    std::size_t stride() const { return stride_; }
private:
    T* data_;
    std::size_t rows_, cols_, stride_;
};

int main()
{
    constexpr std::size_t R = 4, C = 6;
    std::vector<int> buf(R * C, 0);
    MatrixView2D<int> full(buf.data(), R, C, C);      // row-major teljes matrix
    // alterulet: az 1..2 sor es az 1..3 oszlop; a lepes tovabbra is C
    MatrixView2D<int> block(buf.data() + 1 * C + 1, 2, 3, C);
    block(0, 0) = 7;                                  // a full(1,1) elemet irja
    assert(full(1, 1) == 7);
    std::printf("full=%zu block=%zu\n", full.rows() * full.cols(), block.rows()
     ↪  * block.cols());
    return 0;
}
```

Itt a `block` ugyanazt a tulajdonost (`buf`) nézi, csak más báziscímmel és kisebb kiterjedéssel; a lépés változatlanul `C`, ezért a sorok közti ugrás helyes marad. Az owner egyértelmű: a `std::vector`. A nézet invalidáló művelete a `buf` újraallokálása vagy megszűnése, illetve bármely `push_back`, amely a tárolót átmozgatja — ezért a nézetet soha nem tároljuk hosszú élettartamú objektumban a vektor mellett, mert a std::array, std::vector és folytonos tárolás részben tárgyalt invalidációs szabályok itt is érvényesek.

31.5.4 Mátrixindex overflow és bounds szerződés

A row-major offset kiszámítása `r * cols + c` alakban történik. Két külön hiba lehetséges. Az első a *szorzási túlcsordulás*: ha `r * cols` meghaladja az indextípus maximumát, a `std::size_t` aritmetika modulo 2^{64} szerint wrappel, és a kapott offset egy érvényesnek látszó, de idegen indexre mutat. A második a *bounds-sértés*: `r >= rows` vagy `c >= cols` esetén a cím már egy másik objektumra esik. Előfeltétel-ellenőrzés nélkül mindkettő UB.

A túlcsordulás ellenőrzése osztással végezhető el, mert az nem csordulhat túl. A legnagyobb tényleges offset $(R-1)\cdot s+(C-1)$; ez akkor ábrázolható `std::size_t`-ben, ha

$$(R-1) \le \left\lfloor \frac{\text{SIZE_MAX} - (C-1)}{s} \right\rfloor .$$

A fenti `MatrixView2D` konstruktor pontosan ezt a feltételt ellenőrzi. Ha a kiterjedések szorzata (a teljes elemszám) is érdekes, azt ugyanígy, lépésenként kell ellenőrizni: `E0 != 0 && E1 > max / E0` esetén már a szorzat sem fér el. Érdemes tudni, hogy az `std::mdspan` alapértelmezett index típusa `std::size_t`, tehát a nem negatív, de nagy indexek a nézet sajátjai; a negatív indexelést nem ez a típus hivatott kifejezni, és a `ptrdiff_t`-re váltás csak akkor segít, ha a felső korlát is ehhez igazodik.

31.5.5 Numeric utility térkép

A `<limits>`, `<complex>`, `<valarray>` és `<random>` nem alternatívái egymásnak; más-más kérdésre válaszolnak, és a választás indoka a szerződés, nem a kényelem.

A `std::numeric_limits<T>` a típus tulajdonságait kérdezi le: `max()`, `min()`, `lowest()`, `is_signed`, `digits`, `epsilon()`. Vizsgálati és előfeltétel-ellenőrzési eszköz, nem futásidejű elágazás helyettesítője: a fenti overflow-guardban a `max()` használata pontosan ezért jó, mert nem tartalmaz beégetett 2^{64} konstanst. Figyelni kell arra, hogy nem minden típusra garantált specializáció: a GCC `__int128` típusára például kiterjesztésként van, nem hordozható módon.

A `std::complex<T>` aritmetikai típus, nem tároló. A `float`, `double` és `long double` specializációkra a szabvány speciális layout-garanciát ad (a valós rész elöl), ezért ezeknél a C API-val való két `T`-s átadás definiált; más `T`-re ez a garancia nem áll. A `std::valarray<T>` elemenkénti vektoraritmetikát és szeletet (slice) kínál, de korlátozott tárolóinterfésszel rendelkezik; új kódban inkább `std::vector` és algoritmusok használatosak, a `valarray` főleg numerikus örökség.

A `<random>` két külön fogalmat ad: motort (engine, pl. `std::mt19937`) és eloszlást (distribution, pl. `std::uniform_int_distribution`). A motor nyers bitsorozatot ad, az eloszlás alakítja a kívánt tartományra; a kettőt nem szabad összemosni (`engine() % n` torzít). A seedelés és a reprodukálhatóság kérdése külön döntés: a `std::random_device` viselkedése implementációfüggő lehet, kriptográfiai célra a `<random>` önmagában nem elegendő. A numerikus algoritmusok kiértékelési sorrendjéről a Numerikus algoritmusok és kiértékelési sorrend rész szól; itt csak annyi a kapcsolat, hogy egy nézeten keresztül végzett párhuzamos módosítás ugyanazokat a versenyfeltételeket örökli, mint bármely más iterátoros algoritmus.

31.5.6 Tipikus hibák és javításuk

A leggyakoribb hiba az owner és a view összemosása: a `mdspan`-t vagy a `MatrixView2D`-t visszaadó függvény lokális pufferről dangling nézetet ad. Javítás: a nézetet csak olyan függvény adja vissza, amelynek a puffere a hívó tulajdonában van, vagy a nézetet és az ownert egy wrapper fogja össze.

A második: a sor- és oszlopindex felcserélése, amikor a stride-ot nem a layout-policy, hanem beidegződés alapján adjuk meg. Ez különösen column-major forrásból (BLAS, Fortran, OpenGL) érkező adatoknál okoz csendes hibát. Javítás: a nézetet mindig a lépésvektorral együtt teszteljük, és a `layout_left`-es esetet külön tesztesettel fedjük le.

A harmadik: `assert` használata release buildben bounds-ellenőrzésre. Az `assert` `NDEBUG` mellett eltűnik, így a védelem is; ha a határ valódi szerződésszegés esetén is kezelendő, explicit hibavisszatérés vagy kivétel kell, és ezt a hibaútvonalat tesztelni kell.

A negyedik: a `layout_stride`-ot folytonosnak tekinteni, és `data_handle()`-re `memcpy`-t hívni. Csak akkor helyes, ha a lépésvektor pontosan a sűrű row-major vagy column-major leképezést adja; ezt ellenőrizni kell, nem feltételezni.

31.5.7 Feladatok

1. *(számítás)* Egy 4×5-ös `double` mátrix column-major tárolású. Add meg a lépésvektort, a $(2, 3)$ elem offsetjét, a teljes elemszámot és a legnagyobb előforduló offsetet. Hasonlítsd össze a row-major offsettel.

2. *(kódolás)* Egészítsd ki a `MatrixView2D`-t egy `subview(r0, c0, rows, cols)` tagfüggvénnyel, amely alterületet ad vissza, és amely a kért alterületre is elvégzi a bounds-ellenőrzést. Írj hozzá `assert`-alapú tesztet.

3. *(diagnosztika)* Készíts olyan parancssori ellenőrzést, amely eldönti, hogy az adott fordító és standard library kínál-e `mdspan`-t. Fogalmazd meg, mi a teendő, ha nem.

4. *(elemzés)* Egy numerikus kernel egy $N \times N$ mátrixot jár be. Indokold, miért nem mindegy, hogy az offsetet `int`, `std::size_t` vagy `ptrdiff_t` típusban számoljuk, és add meg a helyes előfeltételt.

31.5.8 Megoldások

1. megoldás. Column-major esetén $s_0 = 1$, $s_1 = E_0 = 4$. Az offset $(2, 3)$-ra: $2 \cdot 1 + 3 \cdot 4 = 14$. Teljes elemszám $4 \cdot 5 = 20$, a legnagyobb offset $(3, 4)$-nél $3 + 4 \cdot 4 = 19 = 20 - 1$, ami konzisztens a sűrű kitöltéssel. Row-majorban ugyanez $2 \cdot 5 + 3 = 13$, tehát a két layout ugyanarra az indexre más elemet ad — ezért kell a nézetnek a layoutot explicit módon hordoznia. Ellenőrzési feltétel: $0 \le \text{offset} < E_0 E_1$ minden érvényes indexre.

2. megoldás. Az alterület bázisa `data_ + r0 * stride_ + c0`, kiterjedése `rows`, `cols`, lépése változatlanul `stride_`. Az ellenőrzés: `r0 + rows <= rows_`, `c0 + cols <= cols_`, és a két összeadás nem csordulhat túl (`r0 <= rows_ - rows` alakban írva ez elkerülhető). A visszaadott nézet ugyanarra a tulajdonosra mutat, ezért az owner élettartamára vonatkozó szerződés változatlan. Teszt: alterület írása után a teljes nézeten a megfelelő `(r0+i, c0+j)` elem változik meg; ellenőrzési feltétel, hogy a teljes mátrixon kívül eső elem nem változik.

3. megoldás. A `__cpp_lib_mdspan` macro a `<version>` (vagy a `<mdspan>`) beolvasása után definiált, ha a könyvtár támogatja. Fordítási próba:

```
cat > mdspan_probe.cpp <<'EOF'
#include <version>
int main() {
#ifdef __cpp_lib_mdspan
    return 0;
#else
    return 1;
#endif
}
```

```
EOF
g++ -std=c++23 -fsyntax-only mdspan_probe.cpp && echo "fordul" || echo "nem
 ↪  fordul"
```

A `-fsyntax-only` nem gyárt futtatható fájlt, ezért a labor más részeit nem befolyásolja. Ha a macro hiányzik, két út van: C++20-as stride-alapú saját nézet (mint fent), vagy a toolchain frissítése; a standard verziójelölés önmagában nem bizonyíték arra, hogy a könyvtár is támogatja. Ellenőrzési feltétel: a macro értékét is naplózzuk, mert a puszta létezés nem feltétlenül a kívánt felületet jelenti.

4. megoldás. `int` esetén N néhány tízezres értékénél már a szorzat is túlcsordul, és a hiba signed esetben UB, nem pedig kiszámítható wrap. A `std::size_t` modulo-aritmetikát ad, tehát a túlcsordulás csendesen rossz indexet eredményez — ez a legveszélyesebb változat, mert „működni látszik”. A `ptrdiff_t` szélesebb, de negatív indexet is megenged, ami a bounds-szerződést gyengíti. Helyes előfeltétel: az indexelés `std::size_t`-ben történik, a legnagyobb offset $(N-1)s + (N-1)$ ábrázolható, és ezt a konstrukció idején ellenőrizzük; a belső ciklusban futó ellenőrzés helyett a szerződést a nézet létrehozásakor egyszer érdemes kikényszeríteni.

31.5.9 Összefoglaló

A többdimenziós nézet három független döntésből áll: kiterjedés, lépés és hozzáférés. A C tömb ezeket beégeti a típusba, ezért biztonságos, de merev; a `std::mdspan` ugyanezt futásidejű extentsszel, választható layout-policyval és accessorral kínálja, de nem owner, és csak C++23 könyvtárral érhető el. C++20-on a pointer + kiterjedés + stride hármas adja ugyanazt a szerződést, feltéve, hogy a bounds- és overflow-ellenőrzést a konstrukció idejére tesszük. A numerikus könyvtárak térképén a `numeric_limits` az előfeltételekhez, a `complex` a numerikus aritmetikához, a `random` a mintavételhez tartozik; egyik sem helyettesíti a layout és az élettartam explicit leírását.

Verzió- és forráskorlát: ebben az alfejezetben a szabványhelyekre közvetlen hivatkozást nem adunk, mert a fejezethez nem került feldolgozott elsődleges forrás. A `mdspan` és a többargumentumú `operator[]` C++23-as státusza, a `__cpp_lib_mdspan` macro konkrét értéke, valamint a `std::complex` layout-garanciájának pontos szövege ezért a saját toolchain és a megfelelő verziójú nyilvános szabványtervezet ellenőrzésével megerősítendő. Az N4950 nyilvános munkatervezet, nem az ISO kiadott szabványa.

32 C++ szöveg, fájlok, idő és könyvtári I/O

32.1 iostream, stream state és pufferelés

A C++ stream könyvtár az egyik legsikerültebb „nulla költségű” absztrakció a nyelvben – és egyben az egyik legkönnyebben félreértett. A hibaforrás ritkán a szintaxis; szinte mindig az állapotmodell, a pufferelés és az erőforrás-tulajdonlás határán keletkezik. DevOps szempontból ez közvetlenül érinti a naplózást, a konfigurációfeldolgozást és minden olyan programot, amelynek a kimenete diagnosztikai bizonyíték. Ez az alfejezet ezért nem a formázási flagok katalógusa, hanem a szerződés: mit garantál a stream, mit nem, és hol lépünk át a libc/POSIX világába, ahol a szabályok megváltoznak.

Az alfejezet a stdio, FILE és pufferelés részre épül: ott rögzítettük, hogy a `FILE*` egy pufferelt Absztrakt gép, amelynek a lezárása és a flush-olása külön művelet. Itt azt vizsgáljuk, hogyan örökli és módosítja ezt a C++ iostream réteg, illetve hogyan kapcsolódik hozzá a RAII wrapper és a hibakód-alapú hibakezelés.

32.1.1 Az iostate állapotmodell

Minden `std::basic_ios` objektum négy állapotbitet tart nyilván `std::ios_base::iostate` típusú bitmaszkban:

Bit	Jelentés	Tipikus kiváltó ok
`goodbit`	nincs hiba (értéke 0)	sikeres művelet
`eofbit`	a bemenet végét elértük (akár sikeres olvasás közben is)	`getline`, `read` a fájl végén; `>>` az utolsó, lezáró elválasztó nélküli adatnál
`failbit`	a művelet logikailag sikertelen	formázási hiba, zárt stream, `getline` fájlvégkor
`badbit`	a stream integritása sérült	eszközhiba, kivétel a pufferrétegben

A négy bit nem független, és nem is hierarchikus. A `failbit` és a `badbit` egyszerre is állhat. Az `eofbit` két különböző helyzetben áll be: (1) **sikeres olvasás mellett**, ha a művelet a fájl végébe ütközve fejeződött be — például a `"42"` (záró újsor nélküli) bemenetből az `in >> x` sikeresen beolvassa a 42-t, és közben eléri a véget: `eofbit` áll, `failbit` nem; (2) **sikertelen olvasásnál**, ha adat már nem maradt — ekkor az `eofbit` mellé a `failbit` is beáll. Az `eof()` igaz volta tehát önmagában nem mond semmit az utolsó művelet sikeréről; azt a `fail()` (vagy az `operator bool`) dönti el.

```
// eof_without_fail.cpp - C++20: eofbit failbit nélkül, majd eofbit|failbit
#include <cassert>
#include <sstream>
```

```cpp
int main() {
    std::istringstream in("42");          // nincs záró újsor
    int x = 0;
    in >> x;                              // a szám végét a fájlvég zárta le
    assert(x == 42 && in.eof() && !in.fail() && static_cast<bool>(in));

    int y = 0;
    in >> y;                              // adat nélküli újabb olvasás
    assert(in.eof() && in.fail() && !in.bad() && !static_cast<bool>(in));
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic eof_without_fail.cpp` figyelmeztetés nélkül fordul, és mindkét `assert` teljesül: az első olvasás után `x == 42`, `eof()` igaz, `fail()` hamis, `bool(in)` igaz; a második után `eof()` és `fail()` is igaz.

A lekérdező függvények ezt a maszkot dekódolják: `good()` akkor igaz, ha egyik bit sem áll; `eof()`, `fail()`, `bad()` a megfelelő bitet vizsgálja. Fontos, hogy a `fail()` pontosan akkor igaz, ha a `failbit` vagy a `badbit` áll; az `eofbit` nem számít bele. A `fail()` tehát nem a `!good()` megfelelője: egy csak `eofbit`-es stream nem good, de nem is fail. Az `operator bool` konverzió a `!fail()` kifejezést adja vissza, ezért `while (in >> x)` alakban használható.

Az állapotot a `clear(state)` és a `setstate(state)` módosítja. A `clear()` nem törli a hibát „varázsütésre”: csak a maszkot írja felül, a mögöttes eszköz állapotát nem. Egy `badbit` utáni `clear()` után a stream formálisan good lesz, de a következő művelet újra hibázhat. Fájl végéről való újraolvasáshoz `clear()` **és** `seekg()` is kell.

32.1.2 Az exception mask és a két hibakezelési stílus

A `std::ios_base::exceptions(mask)` hívás nem azt jelenti, hogy „ezeket a hibákat kezelem”, hanem azt, hogy **ezek a bitek kivételt váltanak ki**. Ha a mask olyan bitet tartalmaz, amely az `rdstate()`-ben már be van állítva, az `exceptions()` hívás azonnal dob. A mechanizmus tehát kétélű: beállítása után a hiba nem állapotként, hanem `std::ios_base::failure` kivételként jelenik meg. C++11 óta ez a típus a `std::system_error`-ből származik, így van `code()` tagja és `errno`-szerű hibakódja – ez a híd a system_error világához.

A gyakorlati következmény a legfontosabb: **a bemeneti stream maskjába ritkán szabad beletenni a `failbit`-et**. A `std::getline` és az `operator>>` a fájl végén `failbit`-et állít, ami mask esetén kivételt dobna – vagyis a normál fájlvége hibaként viselkedne. A helyes minta:

```cpp
in.exceptions(std::ios_base::badbit);   // csak valódi I/O hiba dob
while (std::getline(in, line)) { /* ... */ }
if (!in.eof()) {                        // nem a fájl végén állt le
    throw std::runtime_error("olvasási hiba a bemeneten");
}
```

A kimeneti streamnél viszont a `badbit | failbit` mask indokolt, mert ott nincs „normál fail”: minden sikertelen írás valódi hiba. A `close()` hibája is `failbit`-et állít, így mask mellett a lezárás hibája kivételként jelentkezik – ezért fontos az explicit `close()`, mert a destruktor nem dob, hanem elnyeli a hibát.

32.1.3 Formázott és formázatlan I/O

A formázott műveletek (`operator<<`, `operator>>`) a locale és a formázási flagok szerint alakítanak, és a `skipws` flag hatására bemeneten átugorják a whitespace-t. A formázatlan műveletek (`read`, `write`, `get`, `getline`, `put`, `ignore`, `peek`, `gcount`) nyers karakterekkel dolgoznak, nem alkalmaznak konverziót, és nem érinti őket a `skipws`. A `std::getline(std::string&)` szabad függvény, de formázatlan: az újsorig olvas, az újsort eldobja, és a fájl végén `eofbit`-et és `failbit`-et állít.

A megkülönböztetés nem esztétikai. Bináris protokollhoz, fájlmásoláshoz és bármilyen bájtszintű szerződéshez a formázatlan API a helyes, mert a formázott olvasás implicit konverziói adatvesztést és locale-függést hoznak be.

32.1.4 Fájlstream és RAII

Az `std::ifstream` és `std::ofstream` konstruktora megnyitja, destruktora lezárja a fájlt – ez a RAII minta, amely itt a `FILE*` kézi `fclose`-ját váltja fel. A destruktor azonban **nem dob és nem jelez**: ha a lezárás vagy a flush meghiúsul, arról csak akkor értesülünk, ha magunk hívjuk meg a `close()`-t, és ellenőrizzük az állapotot vagy a kivételt. Tartós írásnál ezért a helyes sorrend: írás → `flush()` → `close()` → hibaellenőrzés.

A stream objektum mozgatható, de nem másolható: a tulajdonlás átadható, a file descriptor nem duplikálódik implicit módon. Egy `std::ofstream` tehát egyetlen owner, amelynek az élettartama a fájl nyitva tartásának az élettartama.

32.1.5 Pufferelés, flush és a libc határ

A stream saját pufferrel rendelkezik, amelynek kiürítése a `flush()` művelet. A `std::endl` nem más, mint `'\n'` beírása **és** `flush()` hívása; a `std::ends` `'\0'`-t ír, a `std::flush` csak flush-ol. A `std::cerr` `unitbuf` flaggel indul, azaz minden írás után flush-ol, a `std::cout` és a `std::clog` viszont pufferelt. Ezért keveredhet össze a két kimenet, ha a `cout`-ba írt sor után nem flush-olunk, de a `cerr`-be azonnal írunk.

A `std::cin.tie()` alapértéke `&std::cout`: minden bemeneti művelet előtt a hozzákötött stream flush-ol. Ez teszi működőképessé az interaktív promptot. A `tie( nullptr )` oldja a kötést, ami kötegelt feldolgozásnál felesleges flush-okat szüntet meg.

A `std::ios_base::sync_with_stdio(false)` kapcsolja ki a C és a C++ standard streamek összehangolását; ekkor a `std::cin`/`std::cout` saját puffert kap, és a C `printf`/`scanf` felé irányuló implicit flush-elés megszűnik. **Feltétel:** a hívásnak az első I/O művelet előtt kell megtörténnie, különben a szabvány a hatást implementáció által meghatározottnak mondja. A kapcsoló csak a standard streamekre vonatkozik, tetszőleges `FILE*` és `std::fstream` viszonyára nem.

Itt lépünk át a libc/POSIX tartományba. A `std::ofstream::close()` a puffer tartalmát a kernelnek adja át, nem a lemezre. Tartósághoz `fsync()` (vagy `fdatasync()`) kell a mögöttes file descriptoron, ami nem érhető el hordozhatóan az `std::ofstream` interfészén – GNU libstdc++ esetén a nem szabványos `__gnu_cxx::stdio_filebuf` ad erre módot, ami implementációfüggő. A hordozható minta a `FILE*` szintjén:

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
```

```c
/* POSIX, nem ISO C: a fileno és az fsync Linux/POSIX specifikus. */
int write_durable(const char *path, const char *buf, size_t len) {
    FILE *f = fopen(path, "wb");
    if (!f) return -1;
    if (fwrite(buf, 1, len, f) != len) { fclose(f); return -1; }
    if (fflush(f) != 0)                { fclose(f); return -1; }
    if (fsync(fileno(f)) != 0)         { fclose(f); return -1; }
    if (fclose(f) != 0)                  return -1;
    return 0;
}
```

Ez a háromrétegű pufferelés (C++ stream → libc `FILE` → kernel page cache) magyarázza, miért nem elég a stream lezárása a tartóssághoz, és miért kell a naplóknál a `fsync` költségét külön mérni.

32.1.6 Stringstream, locale és megosztott streamek

A `std::stringstream` ugyanazt az állapotgépet használja memóriapuffer fölött: `str()` adja vissza a tartalmat, C++20-tól `view()` egy nem tulajdonló nézetet. A hibakezelés azonos: a formázási hiba `failbit`, a `clear()` utáni újrahasználat pedig újraolvasáshoz `seekg(0)`-t igényel.

A locale befolyásolja a tizedesjel, az ezres elválasztó és a dátumformátum értelmezését. Fontos részlet, hogy a C++ streamek induláskor a klasszikus `"C"` locale-t használják, nem a környezeti beállítást; a környezet követéséhez `std::locale::global(std::locale(""))` kell, illetve streamenként `imbue()`. A `std::sto*` függvénycsalád viszont a globális C locale-t használja, nem a stream sajátját – ez gyakori inkonzisztencia-forrás.

A megosztott streamek szálbiztonsága korlátozott: a formázott műveletek egyedi szinten adatverseny-mentesek, de két szál által írt kimenet összefonódhat, és a globális locale beállítása nem szálbiztos. Több szálú naplózáshoz tehát saját lock vagy külön puffer szükséges.

32.1.7 Kidolgozott példa: sorok átszámozása

A példa a fenti szerződéseket együtt mutatja: bemeneti mask csak `badbit`-en, kimeneti mask `badbit | failbit`-en, explicit `close()`, valódi hiba megkülönböztetése a fájlvégtől.

```cpp
// linelab.cpp - C++20
#include <filesystem>
#include <fstream>
#include <iostream>
#include <stdexcept>
#include <string>
#include <system_error>

namespace fs = std::filesystem;

int main(int argc, char** argv)
{
    if (argc != 3) {
        std::cerr << "használat: " << argv[0] << " <in> <out>\n";
        return 2;
    }
```

```
    const fs::path in_path = argv[1];
    const fs::path out_path = argv[2];

    // Előfeltétel: a bemenet és a kimenet különböző fájl. Az equivalent()
    // az inode-ot hasonlítja, így a hard linket és az eltérő útvonalat is
    // felismeri; a puszta útvonal-összehasonlítás nem.
    std::error_code ec;
    if (fs::exists(out_path, ec) && fs::equivalent(in_path, out_path, ec)) {
        std::cerr << "hiba: a bemenet és a kimenet ugyanaz a fájl\n";
        return 2;
    }

    // A kimenet ideiglenes fájlba készül, és csak siker után kerül a helyére.
    // Ha az ellenőrzés és az írás között mégis ugyanarra a fájlra mutatna a
    // kimeneti név, a bemenetet akkor sem csonkítjuk.
    const fs::path tmp_path = out_path.string() + ".tmp";
    try {
        std::ifstream in;
        in.exceptions(std::ios_base::badbit);   // failbit szándékosan kimarad
        in.open(in_path);
        if (!in) throw std::runtime_error("nem nyitható meg a bemenet");

        std::ofstream out;
        out.exceptions(std::ios_base::badbit | std::ios_base::failbit);
        out.open(tmp_path, std::ios_base::out | std::ios_base::trunc);

        std::string line;
        std::size_t n = 0;
        while (std::getline(in, line)) {
            out << ++n << '\t' << line << '\n';   // '\n', nem std::endl
        }
        if (!in.eof()) throw std::runtime_error("olvasási hiba");

        out.close();                              // itt dobhat, ha a flush
↪   hibás
        fs::rename(tmp_path, out_path);           // azonos fájlrendszeren csere
        std::cout << "sorok: " << n << '\n';
    } catch (const std::ios_base::failure& e) {
        std::cerr << "I/O hiba: " << e.what()
                  << " (" << e.code().message() << ")\n";
        fs::remove(tmp_path, ec);
        return 1;
    } catch (const std::exception& e) {
        std::cerr << "hiba: " << e.what() << '\n';
        fs::remove(tmp_path, ec);
        return 1;
    }
    return 0;
}
```

Laborparancs: `g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o linelab linelab.cpp`.

A program előfeltétele, hogy a bemenet és a kimenet **különböző fájl**. Ez nem ugyanaz, mint

a különböző útvonal: `./linelab a.txt a.txt` és egy hard link (`ln a.txt b.txt; ./linelab a.txt b.txt`) is ugyanarra a fájlra mutat. A naiv változat a kimenetet `trunc` módban nyitja meg, ezzel a még be nem olvasott bemenetet is lenullázza. A `std::filesystem::equivalent` az eszköz- és inode-azonosítót hasonlítja össze, ezért mindkét esetet felismeri. Az ellenőrzés és a fájlnyitás között azonban a fájlrendszer megváltozhat (TOCTOU). Ezért a program az ideiglenes fájl + átnevezés mintát is használja: a bemenetet így semmilyen sorrendben nem csonkítja, a kimenet pedig csak sikeres írás után kerül a helyére. A POSIX `rename` azonos fájlrendszeren atomikus csere; a `.tmp` utótagú név egyszerűsítés, többpéldányos futásnál egyedi ideiglenes nevet kell választani.

Buildjegyzőkönyv (`lab-gcc` profil). A program figyelmeztetés nélkül fordult. Kétsoros bemenetre `sorok: 2` kimenettel, `1\ta`, `2\tb` tartalmú kimenettel futott. Azonos útvonalra és hard linkre `hiba: a bemenet és a kimenet ugyanaz a fájl` üzenettel, 2-es kóddal állt le, a forrás 4 bájtja megmaradt. Nem létező és olvashatatlan (`chmod 000`) bemenetre `hiba: nem nyitható meg a bemenet`, 1-es kód, ideiglenes fájl nem marad. A stream által dobott failure ága ettől külön tesztelt: írásvédett kimeneti könyvtárnál az `out.open` a maszk miatt `std::ios_base::failure`-t dob, a kimenet `I/O hiba: ...`, szintén 1-es kóddal, `std::terminate` nélkül. A javítás előtti változat azonos fájlra és hard linkre is 0 bájtra csonkította a forrást, nem létező bemenetre pedig `std::terminate`-tel (134-es kód) állt le, mert a `std::runtime_error`-t semmi nem kapta el.

32.1.8 Tipikus hibák

A leggyakoribb hiba a `while (!in.eof())` ciklus. Az `eofbit` csak egy sikertelen olvasás **után** áll be, ezért a ciklus még egyszer lefut egy üres vagy ismételt sorral. A helyes forma a `while (std::getline(...))` vagy `while (in >> x)`.

Második: a `failbit` bevétele a bemeneti maskba, amitől a normál fájlvég kivételt dob. Harmadik: a `close()` elhagyása és a destruktorra bízás, amitől a lemezre írás hibája csendben elvész. Negyedik: `std::endl` használata nagy volumenű naplóban, ahol a soronkénti flush rendszerhívásba kerül. Ötödik: a `sync_with_stdio(false)` hívása I/O után, amikor a hatás már nem garantált.

32.1.9 Feladatok

1. (magyarázó) Adott a `while (!in.eof()) { std::getline(in, line); process(line); }` minta. Magyarázd meg, milyen tünetet okoz a fájl utolsó sora körül, és vezesd le a javítást.

2. (kódolási) Írj `read_config` függvényt, amely egy `kulcs=érték` fájlt olvas be, a hibás sorokat sorszámozva jelzi a `std::cerr`-en, a valódi I/O hibát pedig kivétellel továbbítja. A maskot és az eof-kezelést indokold.

3. (diagnosztikai és mérési) Készíts mérési tervet annak eldöntésére, hogy egy naplózó programban az `std::endl` helyett `'\n'` használata és a `sync_with_stdio(false)` bekapcsolása mekkora költségcsökkenést ad. Add meg a független változókat, a kontrollelemeket és a buktatókat.

32.1.10 Megoldások

1. Az `eofbit` nem az utolsó sikeres olvasás pillanatában, hanem az azt követő sikertelen olvasáskor áll be. A ciklus tehát az utolsó sor után még egyszer belép, a `getline` nem módosítja a `line`-t (vagy üresre állítja), és a `process` lefut egy hibás adaton. Javítás: a `getline` visszatérési értékét kell feltételnek tenni, mert az `operator bool` a `!fail()`-t adja, és a fájlvég éppen `failbit`-et állít. Ellenőrzési feltétel: az utolsó sor pontosan egyszer, a `process` pedig nem hívódik meg a fájlvég utáni üres iterációban.

2. A bemeneti mask `badbit` legyen, mert a fájlvég nem hiba; a ciklus után `if (!in.eof()) throw ...` adja a valódi hibát. A hibás sorokat a `std::cerr`-re írjuk, mert az `unitbuf` miatt azonnal megjelenik, és nem keveredik a pufferelt `std::cout`-tal. A `std::getline` után ellenőrizzük a `'='` jelenlétét és a kulcs nem üres voltát – ez logikai hiba, nem stream-hiba, ezért nem `failbit`-et állítunk, hanem saját számlálót és üzenetet használunk. Ellenőrzési feltétel: hiányzó fájl esetén kivétel, hibás sor esetén a program nem áll le, és a jó sorok feldolgozódnak.

3. Független változó: a lezáró karakter (`'\n'` vs `std::endl`) és a `sync_with_stdio` be/ki. Függő változó: a teljes futási idő nagyszámú, azonos hosszúságú sor írásakor, illetve a rendszerhívások száma (`strace -c`). Kontroll: azonos fájlméret, azonos fájlrendszer, meleg page cache, több ismétlés, medián és szórás. Buktatók: a lemezre írás költsége elfedheti a flush-hatást, ezért először `/dev/null`-ra vagy tmpfs-re írjunk; a `'\n'` és az `std::endl` összehasonlításánál gondoskodni kell arról, hogy a program végén legyen egyetlen záró flush, különben nem ugyanazt mérjük. A mérés eredménye nem általánosítható minden streamre és implementációra, ezért a compiler és a standard library verzióját rögzíteni kell.

32.1.11 Összefoglaló

A C++ stream állapotmodellje négy bitből áll, amelyek nem hierarchikusak: az `eofbit` fájlvégkor `failbit`-tel együtt jelenik meg, a `badbit` pedig integritáshiba. Az exception mask nem hibakezelés, hanem hibakiváltás: a bemeneti maskból a `failbit`-et ki kell hagyni, a kimenetiből nem. A `std::endl` flush-ol, a `'\n'` nem; a `close()` hibája csak explicit híváskor derül ki. A stream lezárása a kernelnek adja át az adatot, nem a lemezre – a tartóssághoz `fsync` kell, ami már a POSIX tartomány. A `sync_with_stdio(false)` és a `tie(nullptr)` költségcsökkentő eszközök, de csak az első I/O előtt, illetve a standard streamekre érvényesek.

32.2 charconv, format és C++23 print

Ez az alfejezet két olyan könyvtári eszközcsaládot hasonlít össze, amelyek a mindennapi DevOps-kódban ugyanazt a feladatot látják el — szöveget alakítanak számmá és számot szöveggé —, de eltérő szerződéssel: a `<charconv>` locale-független, allokációmentes, hibakód alapú konverziót ad, a `<format>` pedig típusos, fordításidőben ellenőrzött formázást. A kettő nem versenytárs, hanem réteg: a konfiguráció és a metrikák gépi konverziója `charconv`, a naplósorok és a felhasználói kimenet `format` (majd C++23-ban `print`). A korábbi részekből támaszkodunk az iostream állapot- és pufferelési modelljére (iostream, stream state és pufferelés (32.1)), a C `printf`-család megbízhatósági korlátaira (Formázás és megbízható szövegfeldolgozás (11.2)) és a constrained felület tervezésére (Concepts, requires és constrained overload (27.2)).

32.2.1 A charconv szerződése: ptr és ec

A `std::from_chars` és `std::to_chars` már a C++17-ben megjelent (`<charconv>`), de a lebegőpontos túlterhelések implementációtámogatása évekig hiányos volt. A szerződésük szándékosan „nyers”: nincs kivétel, nincs allokáció, nincs locale, nem kell nullterminátor, és a bemenetet fél-nyílt tartományként `[first, last)` kapják.

A definíció lényege az eredménytípus:

```
struct from_chars_result {
    const char* ptr;
    std::errc   ec;
```

```
};
struct to_chars_result {
    char*     ptr;
    std::errc ec;
};
```

A `from_chars(first, last, value, base)` esetén a kitöltési szabályok pontosan megkülönböztetendők:

- ha a bemenet elején egyáltalán nincs illeszkedő minta, akkor `ec == std::errc::invalid_argument` és `ptr == first`;
- ha van illeszkedő előtag, de az érték nem ábrázolható a cél típusban, akkor `ec == std::errc::result_out_of_range`, és a `value` változatlan marad;
- siker esetén `ec == std::errc{}` és `ptr` az utolsó illeszkedő karakter **utáni** pozícióra mutat.

Ebből következik a gyakorlati alapszabály, amely e rész első tanulási célja: a sikeresnek tűnő `ec` nem elegendő, a **teljes fogyasztást** is ellenőrizni kell, azaz `ptr == last`. A `"80a"` bemenet ugyanis sikeresen parsolódik 80-ra, `ptr` az `'a'`-ra mutat, és a naplóban csendben rossz konfiguráció lesz belőle. Ugyanígy fontos, hogy az egész túlterhelések nem fogadnak el vezető `+` jelet, nem hagyják figyelmen kívül a vezető szóközt, és előjeles karaktert sem fogadnak el előjel nélküli cél esetén — a `"-1"` bemenet `unsigned` célra `invalid_argument`, nem pedig alulcsordulás, ellentétben a `strtoul` viselkedésével.

A `to_chars(first, last, value)` ellenkező irányban dolgozik: nem ír nullterminátort, siker esetén `ptr` az utolsó kiírt karakter után mutat, és ha a rendelkezésre álló puffer nem elég nagy, `ec == std::errc::value_too_large` mellett `ptr == last`. A visszaadott pointer és a puffer eleje közötti különbség adja a hosszt, ezért a tipikus minta így néz ki:

```
#include <charconv>
#include <cstdint>
#include <string_view>
#include <system_error>

std::string_view render_u16(std::uint16_t v, char (&buf)[8]) {
    auto [ptr, ec] = std::to_chars(buf, buf + sizeof buf, v);
    if (ec != std::errc{}) return {};          // nem fér el: nem lehet
    return {buf, static_cast<std::size_t>(ptr - buf)};
}
```

32.2.1.1 Lebegőpontos támogatás ellenőrzése

A lebegőpontos `to_chars` precízió megadása nélkül a **legrövidebb, round-trip pontos** ábrázolást adja: a visszaolvasott érték bitre azonos lesz az eredetivel. Ez az, amiért naplózáshoz és szerializáláshoz vonzó: nem kell `"%.17g"`-hez hasonló, feleslegesen hosszú alakot használni, és nem kell attól tartani, hogy a visszaolvasás más értéket ad. A `chars_format::fixed`, `scientific`, `general` (alapértelmezés) és `hex` mód a megjelenítést szabályozza, a `from_chars` pedig a „C” locale `strtod` mintáját követi, azzal a szigorítással, hogy vezető szóközt és `+` jelet nem fogad el, és a tizedespont mindig `.`.

Mivel a `__cpp_lib_to_chars` feature-test makró a gyakorlatban nem feltétlenül különbözteti meg az egész- és a lebegőpontos résztámogatást, a támogatást nem makróra, hanem fordítási és futásidejű próbára érdemes alapozni:

```
// probe_charconv.cpp - fordítás: g++ -std=c++17 -Wall -Wextra -Wpedantic
 ↪  probe_charconv.cpp -o probe
#include <charconv>
#include <cstdio>
#include <cstring>

int main() {
    char buf[64];
    double in = 0.1;
    auto w = std::to_chars(buf, buf + sizeof buf, in);
    if (w.ec != std::errc{}) { std::puts("to_chars(float): nem tamogatott");
     ↪  return 1; }

    double out = 0.0;
    auto r = std::from_chars(buf, w.ptr, out);
    if (r.ec != std::errc{} || r.ptr != w.ptr) {
        std::puts("from_chars(float): hiba"); return 2;
    }
    if (std::memcmp(&in, &out, sizeof in) != 0) {
        std::puts("from_chars(float): nem round-trip"); return 3;
    }
    std::puts("ok");
}
```

A próba eredménye konkrét implementáció és verzió kérdése; a laborprofilban rögzítsd a fordítót és a standard library kiadását, és a próbát tedd a build ellenőrző lépései közé. Ha a lebegőpontos irány hiányzik, a konfigurációban maradj egész típusoknál, vagy izolált adapterben használj `strtod`-t a `from_chars` helyett — a locale-t ott explicit „C”-re kell állítani.

32.2.2 std::format: típusos formázás C++20-ban

A `std::format(fmt, args...)` `std::string`-et ad vissza, a formátumstring egy Python-szerű mini-nyelvet használ: `{}`, `{0}`, `{:>8}`, `{:08x}`, `{:.2f}`. A `fmt` paraméter típusa `std::format_string<Args...>`, ami azt jelenti, hogy a könyvtár a formátumstringet **fordításidőben** ellenőrzi, amennyiben az konstans kifejezés: az argumentumszám, a típuskompatibilitás és a specifikáció helyessége is diagnosztizálható. Ez a `printf`-vel szembeni legfontosabb nyereség: a `%d`-vel `double`-t kiíró hiba nem futásidejű UB, hanem fordítási hiba.

A formázás alapértelmezésben **locale-független**: ha nem adsz át `std::locale` objektumot, a könyvtár a classic („C”) locale-t használja, tehát a tizedespont `.` marad, függetlenül a folyamat `LC_NUMERIC` beállításától. Ez a naplók és gépi fájlok szempontjából pont az, amit szeretnénk; a lokalizált, emberi felülethez a `std::format(loc, fmt, args...)` alak kell.

A futásidejű határ ugyanakkor éles: ha a formátumstring nem konstans (például konfigurációból vagy hálózatról jön), a `std::format` nem használható, mert a `format_string` nem áll elő futásidőben. Ilyenkor a `std::vformat(fmt, args)` a belépési pont, ahol `fmt` `std::string_view`, `args` pedig `std::format_args`:

```
#include <format>
#include <string>
#include <string_view>

std::string render(std::string_view fmt, int v) {
```

```
    return std::vformat(fmt, std::make_format_args(v));
}
```

Két fontos következmény. Egyrészt három külön élettartamot kell követni. Az **argumentumok** (a változók, amelyekre hivatkozunk); a `std::make_format_args` által visszaadott **store**, amely ezekre hivatkozó argumentumleírókat tárol; és a `std::format_args` **nézet**, amely a store-ra mutat. A nézet addig használható, amíg a store és az argumentumok is élnek. Egy nevesített store-ból képzett nézet tehát nyugodtan átadható egy későbbi `vformat`-hívásnak ugyanabban a blokkban; a hiba az, ha a nézetet egy *temporary* store-ból mentjük el, mert az a teljes kifejezés végén megszűnik:

```
// format_store.cpp - C++20: a format_args nézet, a store birtokolja az
 ↪  argumentumokat
#include <cstdio>
#include <format>
#include <string>
#include <string_view>

std::string render_later(std::string_view fmt, std::format_args args) {
    return std::vformat(fmt, args);       // a nézetet csak a hívás idejére
     ↪  használjuk
}

int main() {
    int code = 404;
    std::string path = "/index";
    // Nevesített store: addig él, amíg a blokk; a format_args nézet rá mutat.
    auto store = std::make_format_args(code, path);
    std::format_args args = store;
    std::string a = render_later("{} {}", args);   // helyes: store, code, path
     ↪  él
    std::printf("%s\n", a.c_str());

    // HIBA (nem futtatandó): a store temporary, a teljes kifejezés végén
    // megszűnik, az elmentett nézet lógó:
    // std::format_args bad = std::make_format_args(code, path);
    // std::vformat("{} {}", bad);
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20` és `-std=c++23` alatt is figyelmeztetés nélkül fordul, a kimenet `404 /index`.

Másrészt a `std::vformat` futásidőben validál, és érvénytelen formátumstring esetén `std::format_error` kivételt dob; ez memóriabiztonság szempontjából jobb, mint a `printf` UB-je, de nem jelenti azt, hogy a nem megbízható bemenetet szabad formátumstringként használhatnánk: a naplóinjektálás és az erőforrás-kimerítés továbbra is valós támadási felület. Megbízható minta: a formátumstring mindig a kódban konstans, a külső adat pedig *argumentum*.

32.2.2.1 formatter testreszabása

Saját típus formázásához a `std::formatter<T, CharT>` specializációját kell megadni. A legegyszerűbb, és egyben a legtöbbet adó fogás a `std::string_view` formatteréből való öröklés: így a szélesség- és igazítás-specifikáció ingyen működik.

```cpp
#include <format>
#include <string>
#include <string_view>

enum class Level { info, warn, error };

std::string_view to_string(Level l) {
    switch (l) {
    case Level::info:  return "INFO";
    case Level::warn:  return "WARN";
    case Level::error: return "ERROR";
    }
    return "?";
}

template <>
struct std::formatter<Level, char> : std::formatter<std::string_view, char> {
    auto format(Level l, std::format_context& ctx) const {
        return std::formatter<std::string_view, char>::format(to_string(l),
         ↪  ctx);
    }
};
```

A specializációnak minden fordítási egységben láthatónak kell lennie, ahol használják, ezért headerbe való. Ha saját specifikációt akarsz (például `{:s}` rövid névhez), a `parse(std::format_parse_context&)` tagfüggvényt kell megírni, amely a specifikáció végpontjára mutató iterátort ad vissza, és érvénytelen karakter esetén `std::format_error`-t dob; a `format` tag pedig a `ctx.out()` iterátorral ír.

32.2.3 std::print és std::println: C++23, támogatás és fallback

A C++23 `<print>` fejléce a `std::format` motorjára épülő, közvetlen kiírást ad: `std::print("port={}", 8080)` és `std::println(...)`, utóbbi záró újsorral, valamint `FILE*`-ot első argumentumként elfogadó változatok. Előnye, hogy a formázott szöveghez nem kell `std::string`-et építeni a hívó kódjában. Két dolgot ne felejts: a pufferelést nem végzi el helyetted (ha a kimenet azonnali megjelenése a szerződés, gondoskodj a flush-ról), és ne keverd ugyanazon a csatornán a `printf`-féle és a `print`-féle írást, mert a sorrendiség nehezen követhetővé válik.

A támogatás verziófüggő és library-kérdés, ezért a kódot feature-test makróval és fallbackkal érdemes írni:

```cpp
// log_line.cpp - C++17/20/23: képességfüggő kiírás, egységes hibaszerződéssel
#include <version>      // a feature-test makrók forrása: MINDEN teszt előtt
#include <cstdio>
#include <string>
#include <string_view>

#if defined(__cpp_lib_print) && __cpp_lib_print >= 202207L
#  include <print>
#  include <system_error>
#  define HAS_PRINT 1
#endif
```

```cpp
#if defined(__cpp_lib_format) && __cpp_lib_format >= 201907L
#  include <format>
#  define HAS_FORMAT 1
#endif

// Hibaszerződés minden ágban azonos: a logsor teljes kiírása és a stream
// ürítése után true; ha bármelyik nem sikerül, false (nem best-effort).
bool log_line(std::string_view msg) {
#if defined(HAS_PRINT)
    try {
        std::println("{}", msg);                // C++23: írási hibánál
         ↪ system_error
    } catch (const std::system_error&) {
        return false;
    }
#elif defined(HAS_FORMAT)
    std::string s = std::format("{}\n", msg);   // C++20
    if (std::fwrite(s.data(), 1, s.size(), stdout) != s.size()) return false;
#else
    if (std::fwrite(msg.data(), 1, msg.size(), stdout) != msg.size()) return
     ↪ false;
    if (std::fputc('\n', stdout) == EOF) return false;
#endif
    return std::fflush(stdout) == 0;            // a pufferben maradt hiba is
     ↪ számít
}

int main() {
#if defined(HAS_PRINT)
    const bool ok = log_line("branch=print");
#elif defined(HAS_FORMAT)
    const bool ok = log_line("branch=format");
#else
    const bool ok = log_line("branch=fallback");
#endif
    if (!ok) {
        std::fputs("log_line: a kimenet nem irhato\n", stderr);
        return 1;
    }
    return 0;
}
```

A feature-test makrókat a `<version>` fejléc definiálja (illetve az adott képesség saját fejléce). Az `#if` tesztek előtt ezért a `<version>`-t kell include-olni; különben a makró a teszt pontján még nem létezik, és a kód csendben a fallback-ágat választja, akkor is, ha a könyvtár támogatná a képességet. Az aktív ághoz tartozó fejlécet (`<format>`, `<print>`) a teszt után, feltételesen vonjuk be, nem véletlen tranzitív include-ra bízzuk. A küszöbök: `__cpp_lib_format >= 201907L` (C++20 `std::format`), `__cpp_lib_print >= 202207L` (C++23 `std::print`).

```
g++ -std=c++17 -Wall -Wextra -Wpedantic log_line.cpp -o log_line && ./log_line
g++ -std=c++20 -Wall -Wextra -Wpedantic log_line.cpp -o log_line && ./log_line
g++ -std=c++23 -Wall -Wextra -Wpedantic log_line.cpp -o log_line && ./log_line
```

A hibaszerződés minden ágban azonos: a `print` ág a `std::system_error`-t, a két `fwrite`-os ág a rövid írást és a `fflush` hibáját alakítja `false`-ra, és a `main` ilyenkor 1-gyel lép ki. Ha valahol best-effort naplózás kell (a kiírás hibája nem állíthatja meg a programot), azt külön, kimondott szerződésként érdemes egy másik függvénybe tenni.

Buildjegyzőkönyv (`lab-gcc` profil). A GCC 13.3 libstdc++-a `__cpp_lib_format`-ot 202110L értékkel definiálja, `__cpp_lib_print`-et nem. A kimenet `-std=c++17` mellett `branch=fallback`, `-std=c++20` és `-std=c++23` mellett `branch=format`, 0-s kóddal; mindhárom figyelmeztetés nélkül fordult, és `/dev/full`-ra mindhárom 1-es kóddal lépett ki. A `print` ágat a Clang 22 / libc++ (`-std=c++23`) mutatta meg: normál kimenetre `branch=print`, `/dev/full`-ra szintén 1-es kód. A `<version>` include nélküli eredeti változat `-std=c++20` mellett is `branch=fallback`-et adott.

A makrók konkrét értéke és a mögöttük álló tényleges támogatás nem azonos; a fordítást és a viselkedést a laborprofilban rögzített compiler/stdlib páros on ellenőrizd. A `__cpp_lib_print` hiánya nem hiba, csak a fallback ágra terel.

32.2.4 Allokáció, locale és formátumstring-megbízhatóság

A két eszköz költségmodellje eltér. A `from_chars`/`to_chars` nem allokál, nem dob, és futásideje kiszámítható; forró naplózási útvonalon, ahol másodpercenként sok tízezer sor keletkezik, ez a különbség mérhető. A `std::format` `std::string`-et allokál, a `std::format_to` egy kimeneti iterátorba ír (például egy előre lefoglalt `std::string`-be `std::back_inserter`-rel), a `std::formatted_size` pedig előre megmondja a szükséges hosszt, amivel egy allokációra szorítható a művelet. Korlátozott méretű pufferbe a `std::format_to_n` ír; eredménye `{out, size}`, ahol a `size` a **teljes, levágatlan** hossz, tehát a csonkolás detektálható.

```
#include <format>
#include <string>

std::string line;
line.reserve(128);
std::format_to(std::back_inserter(line), "level={} port={}", "WARN", 8080);
```

A locale kérdése három külön döntés: a `charconv` definíció szerint locale-független; a `std::format` locale nélkül a classic locale-t használja; a `std::print`/`println` kimenetének kódolása (a terminál és az ordinary literal encoding viszonya) további, a szabvány és az implementáció dokumentációja szerint ellenőrizendő réteg. A `<format>` C++23-as chrono- és range-formázói, valamint a Unicode-kézbesítés részletei itt nem téma; ezekre csak a szükséges határjelzésként hivatkozunk.

Végül a megbízhatóság: a fordításidőben ellenőrzött formátumstring a hibák nagy részét kiszűri, de a dinamikus (`vformat`) út csak futásidőben derül ki. Naplózó komponensben ezért a jó szerződés az, hogy a formátumstring a forrásban konstans literál, a változó adat argumentum, a felhasználói bemenet pedig soha nem kerül a formátum pozíciójába.

32.2.5 Kidolgozott példa: konfiguráció és naplósor

A példa egy `port=8080` alakú konfigurációs értéket parsol szigorúan, majd naplósort formáz. A build: `g++ -std=c++20 -O2 -Wall -Wextra -Wpedantic main.cpp -o main`, illetve `clang++ -std=c++20 -O2 -Wall -Wextra -Wpedantic main.cpp -o main`. A kimenet elvárt, nem mért eredmény; a saját laborkörnyezetedben futtatva ellenőrizd.

```cpp
#include <charconv>
#include <cstdint>
#include <format>
#include <optional>
#include <string>
#include <string_view>
#include <system_error>

std::optional<std::uint16_t> parse_u16(std::string_view s) {
    if (s.empty()) return std::nullopt;
    const char* first = s.data();
    const char* last  = first + s.size();
    std::uint16_t v{};
    auto [ptr, ec] = std::from_chars(first, last, v);
    if (ec != std::errc{} || ptr != last) return std::nullopt;  // teljes
     ↪ fogyasztás
    return v;
}

int main() {
    const std::string_view values[] = {"80", "80a", "+80", "-1", "65536", "0"};
    for (std::string_view raw : values) {
        auto v = parse_u16(raw);
        std::string shown = v ? std::format("{}", *v) : std::string{"hiba"};
        std::format_to(std::back_inserter(shown), "");
        std::printf("%-6.*s -> %s\n", static_cast<int>(raw.size()), raw.data(),
         ↪ shown.c_str());
    }
}
```

A várható sorok: `80 -> 80`, `80a -> hiba`, `+80 -> hiba`, `-1 -> hiba`, `65536 -> hiba`, `0 -> 0`. A `80a` a teljes fogyasztás ellenőrzése nélkül átmenne; a `65536` a `result_out_of_range` ág; a `+80` és a `-1` az `invalid_argument` ág.

32.2.6 Tipikus hibák

- **Csak `ec`-t vizsgálsz.** A részleges parse csendben rossz konfigurációt ad. Javítás: `ec == std::errc{} && ptr == last`.
- **`to_chars` eredményét C-sztringként kezeled.** Nincs nullterminátor; a hossz `ptr - buf`.
- **Túl kis puffer.** Ellenőrizd a `value_too_large` hibát, és a puffert a típus maximális hosszára méretezd (16 bites előjel nélkülihez 5 számjegy elég, de hagyj tartalékot).
- **`std::format` futásidejű stringgel.** Fordítási hiba; a szándék `std::vformat`, a `format_args` élettartamának tiszteletben tartásával.
- **`format_args` eltárolása.** A teljes kifejezésen túl dangling; a `vformat` hívást ugyanabban a kifejezésben tartsd.
- **Külső bemenet a formátum pozíciójában.** Naplóinjektálás és `format_error`- kockázat; a séma legyen konstans.
- **Locale-feltételezés.** A `charconv` és a locale nélküli `format` nem követi a `LC_NUMERIC`-et; ne a locale-tól várd a tizedespont kezelését.

32.2.7 Összefoglaló

A `<charconv>` a gépi konverzió eszköze: `[first, last)` tartomány, `ptr`/`ec` eredmény, teljes fogyasztás-ellenőrzés, nincs allokáció és nincs locale. A lebegőpontos irány round-trip pontos, de implementációtámogatását próbával kell igazolni. A `<format>` típusos, konstans formátumstring esetén fordításidőben ellenőrzött, locale nélkül a classic locale-t használó réteg; a futásidejű határ a `std::vformat`, a `format_args` élettartam-korlátjával. A `<print>`/`<println>` C++23-as kényelmi réteg, fallbackkal és a támogatás ellenőrzésével. A döntés rövid szabálya: konfiguráció és metrika `charconv`, napló `format`, kimenet `print`, ha a laborprofil engedi.

32.2.8 Feladatok

1. **Magyarázó.** Írd le, miért ad a `from_chars("12x")` sikeres `ec`-t, és pontosan melyik két feltétel együttes teljesülése jelent „teljes” parse-t. Egészítsd ki egy olyan esettel, amikor a `value` tartalma hiba után is érdekes lehet.
2. **Kódolási.** Készíts `parse_i32(std::string_view)` függvényt, amely elfogad egy opcionális vezető `+` jelet, majd `std::from_chars`-szal parsol, és csak teljes fogyasztás esetén ad értéket. Mellékelj `formatter<Level>` specializációt, amely a `{:>8}` szélesség-specifikációt is támogatja.
3. **Diagnosztikai.** Az alábbi két részlet közül melyik hibás és miért? Add meg a javítást és az ellenőrzési feltételt.

```
auto args = std::make_format_args(port, level);
std::string s = std::vformat(fmt, args);   // (A)

std::uint16_t v{};
auto [p, ec] = std::from_chars(begin, end, v);
if (ec == std::errc{}) use(v);              // (B)
```

32.2.9 Megoldások

1. A `from_chars` a bemenet **előtagját** parsolja, nem a teljes tartományt: a `"12x"` esetén a `12` érvényes minta, ezért `ec == std::errc{}` és `ptr` az `'x'`-re mutat. Teljes parse akkor van, ha `ec == std::errc{}` **és** `ptr == last`. A `value` hiba esetén a szabvány szerint változatlan marad, de ha nem inicializáltad, az olvasása önmagában nem definiált viselkedés — ezért a mintában `std::uint16_t v{};` szerepel. Ellenőrzési feltétel: a `"80a"`, `" 80"`, `"+80"`, `"-1"`, `"65536"`, `""` bemenetek mind elutasítva, a `"0"` és a `"65535"` elfogadva.

2. A `+` jelet a `from_chars` nem fogadja el, ezért kézzel kell levágni, majd a maradékra hívni a konverziót; ügyelni kell arra, hogy a csupasz `"+"` ne menjen át.

```
#include <charconv>
#include <cstdint>
#include <optional>
#include <string_view>
#include <system_error>

std::optional<std::int32_t> parse_i32(std::string_view s) {
    if (s.empty()) return std::nullopt;
    if (s.front() == '+') {
        s.remove_prefix(1);
```

```
        if (s.empty()) return std::nullopt;
    }
    std::int32_t v{};
    auto [ptr, ec] = std::from_chars(s.data(), s.data() + s.size(), v);
    if (ec != std::errc{} || ptr != s.data() + s.size()) return std::nullopt;
    return v;
}
```

A formatter a `std::string_view` formatteréből örökölve kapja a szélességet; a `format` const tag kell, hogy a `format_context`-tel működjön. Ellenőrzési feltétel: `std::format("{:>8}", Level::warn)` eredménye `"    WARN"` (8 karakter), `std::format("{}", Level::error)` eredménye `"ERROR"`; a `parse_u16` esetében a `"-1"` és a `"+"` bemenet `nullopt`.

3. Az (A) az élettartam-hiba: a `make_format_args` a `port` és `level` referenciáit csomagolja, a `format_args` csak a teljes kifejezésig érvényes, így a következő sorban történő felhasználás dangling. Javítás: a `vformat` hívás legyen ugyanabban a kifejezésben, vagy konstans formátumstring esetén használj `std::format`-ot. A (B) a teljes fogyasztás hiánya: a `"80a"` bemenet sikeresen átmegy, és rossz értéket használ. Javítás: `if (ec == std::errc{} && p == end) use(v);`. Ellenőrzési feltétel: a (B) javított változata a `"80a"`-t elutasítja, a `"80"`-t elfogadja; az (A) javított változata nem tartalmaz a `vformat` híváson kívül tárolt `format_args` objektumot.

32.3 filesystem, path és OS-határ

Ez az alfejezet a C++20 `std::filesystem` szerződését a POSIX/Linux szemantikához viszonyítja. A laborprofil: Linux, x86-64, GCC 11+ vagy Clang 14+, `-std=c++20`, libstdc++ vagy libc++. A `<filesystem>` már C++17-ben bekerült a standardba, de a C++20-as szöveg több részletet pontosít (például a `u8string()` `char8_t`-re váltását), ezért a példák C++20-at használnak. A kimenet szándékosan `std::cout`-tal történik; a `<print>` és a `<format>` részleteit a charconv, format és C++23 print (32.2) tárgyalja.

A fejezet központi állítása: a `std::filesystem` kényelmes, hordozható névfeloldást és hibakódleképezést ad, de nem emeli be az operációs rendszer atomitási, tartóssági és biztonsági szerződését. Aki ezt összekeveri, az a fájlbejáró laborban látványos, de nehezen reprodukálható hibákat kap.

32.3.1 A path típus: natív reprezentáció és generikus alak

A `std::filesystem::path` értékszemantikájú típus, amely egy útvonalat nem karakterláncként, hanem komponensek sorozataként tárol. A `value_type` a platform natív karaktertípusa (POSIX-on `char`, Windowson `wchar_t`), a `string_type` pedig a hozzá tartozó `std::basic_string`. A `native()` a natív, az operációs rendszer API-ja által várt alakot adja vissza, a `c_str()` ennek null-terminált változatát. A `generic_string()` és a C++20-as `generic_u8string()` ugyanezt `/` elválasztóval, illetve `std::u8string`-ként adja vissza; ez utóbbi C++17-ben még `std::string` volt, tehát a két standardverzió között itt inkompatibilitás van.

A path natív kódolása implementáció által meghatározott. POSIX-on a fájlrendszeri név egyszerűen bájtok sorozata, a `/` és a null bájt kivételével; a kernel nem dekódolja UTF-8-ként, és nem normalizál. A `std::filesystem` sem végez Unicode-normalizálást (NFC/NFD), nem ellenőrzi az UTF-8 érvényességét, és nem alakít át kódolást. Ha a programod UTF-8-at vár, az konvenció, nem a standard garanciája. A `u8string()` csak annyit tesz, hogy a natív reprezentációt `char8_t`-es sztringgé alakítja; ha a natív bájtok nem UTF-8-at kódolnak, az eredmény nem lesz érvényes UTF-8. Linuxon a UTF-8 gyakori konvenció, de nem nyelvi garancia.

A lexikális műveletek (`filename()`, `stem()`, `extension()`, `parent_path()`, `lexically_normal()`, `lexically_relative()`) tisztán szövegesek, nem érik el a fájlrendszert. A `lexically_normal()` a `.` és `..` komponenseket szövegesen oldja fel: `"/tmp/link/../x"` eredménye `"/tmp/x"`, még akkor is, ha a `link` symlink, és a valóságos feloldás más lenne. A `canonical()` ezzel szemben feloldja a symlinkeket, abszolút utat ad, és megköveteli, hogy a cél létezzen; a `weakly_canonical()` a létező prefixet oldja fel, a maradékot lexikálisan kezeli. Az `equivalent(p1, p2)` nem lexikális: a `status()`-ok alapján dönti el, hogy két név ugyanarra az objektumra mutat-e, tehát symlinket követ, és versenyérzékeny.

32.3.2 Állapot, symlink és a TOCTOU-rés

A `file_status` két dolgot hordoz: a `type()`-ot és a `permissions()`-t. A `file_type` enumeráció értékei: `none`, `not_found`, `regular`, `directory`, `symlink`, `block`, `character`, `fifo`, `socket`, `unknown`. A `status(p)` követi a symlinket (a POSIX `stat` mintájára), a `symlink_status(p)` nem (a `lstat` mintájára). Ha a symlink célja nem létezik, a `status(p)` `file_type::not_found`-ot ad; ez nem feltétlenül hiba, az `error_code` üres maradhat. Az `exists(status)` a `not_found`-ra hamis.

A klasszikus hiba a „check, then act” minta:

```
namespace fs = std::filesystem;
fs::path p = "/var/lib/app/config.toml";

if (fs::exists(p)) {          // 1. lépés: stat/lstat
    std::ifstream in{p};      // 2. lépés: open
    // ...
}
```

A két lépés nem atomi. A kettő között egy másik folyamat törölheti, kicserélheti vagy symlinkre cserélheti a fájlt; a program így más objektumot nyit meg, mint amit ellenőrzött. Ez a TOCTOU (time-of-check to time-of-use) rés. A `std::filesystem` nem kínál atomi „check and open” műveletet. POSIX-on a megoldás az `open()` `O_NOFOLLOW` (és `O_CLOEXEC`) jelzővel, majd `fstat()` a megnyitott file descriptoron; így a névfeloldás egyszer történik, és az ellenőrzés ugyanarra az objektumra vonatkozik, amit később olvasol. A `std::ifstream` nem ad hordozhatóan hozzáférést a mögöttes fd-hez, ezért ehhez POSIX réteg kell.

Hasonló csapda a `create_directory()`: visszatérési értéke `true`, ha létrehozta, és `false`, ha a könyvtár már létezett. A `false` nem hiba. Aki a visszatérési értéket elhagyja, azt hiheti, hogy ő hozta létre a könyvtárat, holott egy másik folyamat vagy egy korábbi futás tette meg.

32.3.3 Bejárás: directory_iterator és rekurzió

A `directory_iterator(dir)` a `dir` közvetlen bejegyzéseit adja; a sorrend nem specifikált. A `directory_entry` egy bejegyzést reprezentál, és a `status()`, `symlink_status()`, `file_size()`, `is_regular_file()` tagfüggvényei `error_code&` overloaddal is elérhetők. A C++20-as szöveg szerint a `directory_entry` cache-elheti az attribútumokat; a részletek implementációfüggők, ezért a hordozható minta az, hogy a szükséges attribútumot explicit módon, `error_code`-dal kéred le, és nem feltételezed, hogy a cache friss. A `refresh()` a cache-elt értéket frissíti.

A `recursive_directory_iterator` rekurzívan járja be a fát. Alapértelmezésben nem követi a symlinkeket; a `follow_directory_symlink` opció kapcsolja be a követést. Alapból kivételt dob a jogosultsági hibára; a `skip_permission_denied` opció kihagyja az ilyen bejegyzéseket. A `depth()`

az aktuális mélységet adja, a `disable_recursion_pending()` pedig az aktuális bejegyzés alatti rekurziót tiltja le.

A symlink-policy a legfontosabb döntés. Ha bekapcsolod a `follow_directory_symlink`-et, egy `a -> b`, `b -> a` symlink-pár végtelen rekurziót okozhat; a standard nem detektál ciklust. A védelem vagy egy mélységkorlát, vagy a `canonical()` szerinti halmaz, amelyben a már látott célokat tárolod. Utóbbi minden híváskor feloldja a symlinkeket, hibát ad, ha a cél nem létezik, és maga is versenyérzékeny; ezért a bejárás közbeni használata drága. A legegyszerűbb és legbiztonságosabb alapértelmezés a symlinkek nem követése, és a követés csak külön kapcsolóval, mélységkorláttal.

Az iteráció közbeni módosítás veszélyes: a mögöttes könyvtár módosítása nem specifikált viselkedést ad. Ne törölj és ne hozz létre bejegyzéseket a bejárás alatt; ha kell, gyűjtsd össze a neveket, és utána módosíts.

32.3.4 Kivétel vagy error_code?

A legtöbb `std::filesystem` műveletnek két változata van: `(path)` és `(path, error_code&)`. A `(path)` változat `std::filesystem::filesystem_error`-t dob, amely a `std::system_error` leszármazottja, és a `path1()`, `path2()`, `code()` tagokkal rendelkezik. Az `(path, error_code&)` változat az **operációsrendszer-szintű** hibát `ec`-ben jelzi, siker esetén pedig törli az `ec`-t. Ez azonban nem jelent `noexcept`-ot: a szabvány szerint (C++20 [fs.err.report]) a tárfoglalási hibát ezek az overloadok is `std::bad_alloc` kivétellel jelzik, és csak egy részük deklarált `noexcept` (a konkrét overload szignatúráját kell megnézni). A saját kódunk — például a `push_back` egy gyűjtővektorba — szintén dobhat. Ezért ha a program kontrollált hibás kilépést ígér, a CLI szélén kell egy kivételhatár.

A választás indoklása:

- Ha a hiányzó fájl normál eset (például egy bejáró vagy egy gyorsítótár-ellenőrző), `error_code` a helyes: a hiba nem kivételes, és a ciklusban a dobás drága.
- Ha a hiba fatális és egyszeri (például a konfigurációs fájl hiánya a program indulásakor), a throwing változat is jó, mert a dobás költsége elhanyagolható, és a kód rövidebb.
- A kivétel nem „ingyenes”, de a nem dobott kivétel olcsó; ne a teljesítmény legyen az egyetlen érv. A kivétel kontrollfolyamatként való használata (gyakori, várt hibákra) rossz minta.
- Az `error_code`-ot minden ágon ellenőrizni kell. A `filesystem` függvények többsége nem `[[nodiscard]]`, ezért a figyelmetlen kód csendben elnyeli a hibát. A hibakód kategóriája rendszerint `std::system_category()`, de erre a standard nem ad feltétlen garanciát minden műveletre.

32.3.5 Ami nincs benne: atomitás és tartósság

A `std::filesystem` nem ad tranzakciót, nem ad `fsync`-et, és nem garantálja a `rename` atomitását. A POSIX `rename(2)` ugyanazon fájlrendszeren belül atomi, de ez a platform szemantikája, nem a C++ standard ígérete. A `copy_file` nem atomi; a `remove_all` nem tranzakcionális, részleges törlés lehetséges. A tartós íráshoz `fsync()` kell a fájlra és a könyvtárra; ehhez POSIX `open`/`fsync` szükséges, mert a `std::filesystem` erre nem ad API-t. A `current_path()` a folyamat globális aktuális könyvtárát kezeli, tehát több szálon versenyhelyzet forrása lehet.

Ez DevOps szempontból a legfontosabb: a kényelmes API nem helyettesíti az operációs rendszer atomitási és tartóssági szerződését. A Fájlrendszer-műveletek, mmap és tartósság (12.2) részben tárgyalt `rename` + `fsync` minta itt is érvényes; a `std::filesystem` csak a névfeloldást és a hibakódokat adja hozzá.

32.3.6 Kidolgozott példa: fájlbejáró labor

A következő program összegyűjti a reguláris fájlokat, összegzi a méretüket, nem követi a symlinkeket, és a jogosultság miatt nem megnyitható könyvtárakat a `skip_permission_denied` opcióval kihagyja. Az OS-hibákat `error_code`-dal kezeli, az allokációs és egyéb kivételeket a `main` kivételhatára fogja meg. A `skip_permission_denied` csak a könyvtár megnyitásának jogosultsági hibáját (`EACCES`) nyeli el; minden más léptetési hiba (`it.increment(ec)`) után a bejárás nem folytatható megbízhatóan, ezért a program megszakítja, az eredményt részlegesnek jelöli, és 3-as kóddal lép ki — az `ec` törlése nem helyreállítás.

```
// walk.cpp - C++20, Linux
#include <cstdint>
#include <filesystem>
#include <iostream>
#include <system_error>
#include <vector>

namespace fs = std::filesystem;

struct entry_info {
    fs::path path;
    std::uintmax_t size = 0;
};

static int run(int argc, char** argv) {
    if (argc != 2) {
        std::cerr << "hasznalat: walk KONYVTAR\n";
        return 2;
    }

    fs::path root{argv[1]};
    std::error_code ec;

    fs::file_status root_status = fs::symlink_status(root, ec);
    if (ec) {
        std::cerr << "symlink_status: " << ec.message() << '\n';
        return 1;
    }
    if (!fs::is_directory(root_status)) {
        std::cerr << root << ": nem konyvtar\n";
        return 1;
    }

    const auto opts = fs::directory_options::skip_permission_denied;
    fs::recursive_directory_iterator it{root, opts, ec};
    const fs::recursive_directory_iterator end{};
    if (ec) {
        std::cerr << "iterator: " << ec.message() << '\n';
        return 1;
    }

    std::vector<entry_info> files;
```

```
std::uintmax_t total = 0;
bool partial = false;

while (it != end) {
    const fs::directory_entry& de = *it;

    std::error_code sec;
    fs::file_status st = de.symlink_status(sec);
    if (sec) {
        std::cerr << "symlink_status(" << de.path() << "): "
                  << sec.message() << " (kihagyva)\n";
        it.increment(ec);
        if (ec) { partial = true; std::cerr << "increment: " << ec.message()
         ↪  << '\n'; break; }
        continue;
    }

    if (fs::is_symlink(st)) {
        std::error_code tec;
        fs::file_status tgt = de.status(tec);
        if (tec) {
            std::cerr << "status(" << de.path() << "): "
                      << tec.message() << '\n';
        } else if (tgt.type() == fs::file_type::not_found) {
            std::cerr << "dangling symlink: " << de.path() << '\n';
        }
    } else if (fs::is_regular_file(st)) {
        std::error_code fec;
        std::uintmax_t sz = de.file_size(fec);
        if (fec) {
            std::cerr << "file_size(" << de.path() << "): "
                      << fec.message() << '\n';
        } else {
            total += sz;
            files.push_back({de.path(), sz});
        }
    }

    it.increment(ec);
    if (ec) {
        // A hibás increment után az iterátor állapota nem folytatható
        // megbízhatóan: a bejárást lezárjuk, és az eredményt részlegesnek
         ↪  jelöljük.
        partial = true;
        std::cerr << "increment: " << ec.message() << " (bejaras
         ↪  megszakadt)\n";
        break;
    }
}

for (const auto& f : files) {
    std::cout << f.size << '\t' << f.path.generic_string() << '\n';
```

```
    }
    std::cout << (partial ? "# RESZLEGES: " : "# osszesen ") << files.size()
              << " fajl, " << total << " bajt\n";
    return partial ? 3 : 0;
}

int main(int argc, char** argv) {
    // Az error_code-os overloadok az OS-hibát jelzik ec-ben, de az allokációs
    // hibát (std::bad_alloc) továbbra is kivétellel; a CLI szélén ezt kezeljük.
    try {
        return run(argc, argv);
    } catch (const std::exception& e) {
        std::cerr << "walk: " << e.what() << '\n';
        return 1;
    }
}
```

Fordítás és futtatás a laborban:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 walk.cpp -o walk
./walk /usr/include | tail -n 5
```

GCC 8 és korábbi verziók külön `-lstdc++fs` linkelést igényeltek; GCC 9+ és a korszerű libc++ nem. Ez verziófüggő részlet, ezért a laborprofilban rögzíteni kell. **Buildjegyzőkönyv** (`lab-gcc` profil). A program figyelmeztetés nélkül fordult. Egy három fájlt és egy `chmod 000` könyvtárat tartalmazó fán a kimenet `# osszesen 3 fajl, 12 bajt`, 0-s kóddal (a jogosultság nélküli könyvtárat csendben kihagyta). A léptetési hibát egy tesztbuildben injektáltuk: a bejárás egy alkönyvtárat a belépés előtt törölt. Erre a program `increment: No such file or directory (bejaras megszakadt)` üzenetet, `# RESZLEGES: ...` összesítést és 3-as kilépési kódot adott. A javítás előtti, `ec.clear()`-rel „továbblépő” változat ugyanerre `# osszesen 0 fajl, 0 bajt` kimenettel és 0-s kóddal, teljes eredményként állt le, mert a hibás léptetés után az iterátor a végére ugrott.

32.3.7 Tipikus hibák és javításuk

Hiba	Következmény	Javítás
`exists()` után `open()`	TOCTOU, más objektum nyílik meg	`open(O_NOFOLLOW)` + `fstat`
`create_directory` visszatérési értékének elhagyása	azt hiszed, te hoztad létre	`bool created = ...` és kezelés
`follow_directory_symlink` ciklusdetektálás nélkül	végtelen rekurzió	`depth()` korlát vagy `disable_recursion_pending()`
Cache-elt `directory_entry` státusz	elavult információ	explicit `status(ec)` / `refresh()`
`p.string()` UTF-8-ként kezelése	hibás kódolás	`native()` / `u8string()` tudatos használata
`remove_all` tranzakcionálisnak feltételezése	részleges törlés	idempotens újrafuttatás, naplózás
`ec` ellenőrzésének elhagyása	csendes hiba	minden `ec` ellenőrzése

32.3.8 Feladatok

1. feladat (magyarázó és kódolási). Az alábbi kódrészlet TOCTOU-t tartalmaz. Nevezd meg a versenyt, és írj POSIX-szintű javítást, amely a megnyitott fd-en ellenőriz, és nem követi a symlinket.

```
fs::path p = "/var/lib/app/config.toml";
if (fs::exists(p)) {
    std::ifstream in{p};
    // ...
}
```

2. feladat (kódolási). Írj `du`-szerű függvényt, amely összegzi egy fa reguláris fájljainak méretét. Alapból ne kövesse a symlinkeket, hagyja ki a jogosultsági hibákat, és opcionális `follow` módban alkalmazzon mélységkorlátot. Indokold, miért nem detektál ciklust a standard iterator.

3. feladat (diagnosztikai). Egy bejáró lassú egy hálózati fájlrendszeren. Sorolj fel három lehetséges okot, és mindegyikhez adj egy ellenőrzési módot. Ne állítsd, hogy mértél; fogalmazz hipotézisként.

32.3.9 Megoldások

1. megoldás. A verseny az `exists()` és az `ifstream` megnyitása között van: a névfeloldás kétszer történik, és a kettő között a fájl kicserélhető. A javítás:

```
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <cerrno>

int fd = ::open(p.c_str(), O_RDONLY | O_CLOEXEC | O_NOFOLLOW);
if (fd < 0) {
    // ENOENT: nincs ilyen; ELOOP: symlink; EACCES: jogosultsag
    return -1;
}
struct stat st{};
if (::fstat(fd, &st) != 0 || !S_ISREG(st.st_mode)) {
    ::close(fd);
    return -1;
}
// innen a fd egy ellenorzott, regularis fajlra mutat
```

Az indoklás: az `open` egyszer oldja fel a nevet, az `fstat` ugyanarra az objektumra vonatkozik, amit az fd azonosít. Az `O_NOFOLLOW` megakadályozza, hogy a végső komponens symlink legyen; az `O_CLOEXEC` megakadályozza a fd szivárgását `exec` esetén. Ellenőrzési feltétel: `strace`-ben egy `openat` és egy `fstat` látszik, nem `stat` + `open` pár; symlink esetén `ELOOP` jön. Megjegyzés: a `std::ifstream` nem ad hordozhatóan fd-t, ezért itt POSIX réteg kell; C++23-ban sem standard a „no-follow” nyitás a `std::fstream`-ben.

2. megoldás. A függvény váza:

```cpp
std::uintmax_t dir_size(const fs::path& root, bool follow, int max_depth,
                        std::error_code& ec) {
    std::uintmax_t total = 0;
    auto opts = fs::directory_options::skip_permission_denied;
    if (follow) opts |= fs::directory_options::follow_directory_symlink;

    fs::recursive_directory_iterator it{root, opts, ec};
    const fs::recursive_directory_iterator end{};
    if (ec) return 0;

    while (it != end) {
        if (follow && it.depth() >= max_depth) {
            it.disable_recursion_pending();
        }
        const fs::directory_entry& de = *it;
        std::error_code sec;
        auto st = de.symlink_status(sec);
        if (!sec && fs::is_regular_file(st)) {
            std::error_code fec;
            auto sz = de.file_size(fec);
            if (!fec) total += sz;
        }
        it.increment(ec);
        if (ec) ec.clear();
    }
    return total;
}
```

Az indoklás: `follow_directory_symlink` nélkül a könyvtár-symlinkekbe nem lép be, tehát ciklus sem alakul ki. `follow` módban a `depth()` korlát és a `disable_recursion_pending()` a védelem; a standard nem detektál ciklust, a `canonical()` szerinti halmaz alternatíva, de drága és versenyérzékeny. Ellenőrzési feltétel: az eredmény egyezik a `find DIR -type f -printf '%s\n' | awk '{s+=$1} END{print s}'` értékkel, és symlinkes fán nem fut végtelen ideig.

3. megoldás. Lehetséges okok: (a) a `directory_entry` attribútum-lekérdezése külön `stat`/`lstat` syscallt válthat, ha a cache nem tartalmazza, és a hálózati fájlrendszeren a latencia nagy; ellenőrzés: `strace -f -c -e trace=newfstatat,stat,lstat,getdents64 ./walk DIR`. (b) A `file_size` újabb `stat`-ot válthat, tehát a syscall-szám a bejegyzésszámmal nő; ellenőrzés: a syscall-szám összevetése a bejegyzésszámmal. (c) A `skip_permission_denied` nem csökkenti a syscallokat, csak a kivételt; a jogosultsági hibák továbbra is syscallt igényelnek. Ellenőrzés: a kivételes és a nem kivételes változat syscall-száma. Ezek hipotézisek; a tanuló laborja méri őket.

32.3.10 Összefoglaló

A `std::filesystem` hordozható névfeloldást, bejárást és hibakód-leképezést ad, de nem ad atomi „check and act” műveletet, nem ad `fsync`-et, és nem garantálja a `rename` atomitását. A `path` natív kódolása implementációfüggő; a `generic_string()` csak az elválasztót cseréli, nem konvertál. A `status` követi a symlinket, a `symlink_status` nem; a kettő közötti különbség és a TOCTOU-rés a biztonsági hibák fő forrása. A throwing és az `error_code` overload közötti választás a hiba várt gyakoriságán és a kontextuson múlik, nem pusztán a teljesítményen. A bejárásnál a symlink-policy és a mélységkorlát az elsődleges védelmi vonal.

32.4 chrono, duration és időpont

A C++20 `chrono` könyvtárának legfontosabb hozadéka nem az, hogy „van benne idő”, hanem hogy az idő *típusokba kódolt szerződés*. A C világában a `time_t`, a `struct timespec` és a `clockid_t` együtt él, és a programozó fejében kell nyilvántartani, hogy melyik érték melyik órához és milyen egységhez tartozik. A `chrono` ezt a megkülönböztetést a típusrendszerbe emeli: a `steady_clock` és a `system_clock` időpontja nem keverhető össze véletlenül, a másodperc és a milliszekundum külön típus, és a konverzió csak ott implicit, ahol nem veszik el információ.

Ez az alfejezet két dolgot jár körül: a **mért időt** (elapsed, timeout, latency — monoton óra kell hozzá) és a **naptári időt** (timestamp, ütemezés, napló — naptárhoz kötött óra kell hozzá). A kettő szétválasztása nem esztétikai kérdés: a naptári óra ugrik (NTP, kézi állítás, nyári időszámítás), a monoton óra nem korrelál a naptárral. Aki ezt összekeveri, annak a timeoutja negatív lesz, a mérése pedig negatív latenciát mutat. Az alfejezet épít a filesystem, path és OS-határ részben tárgyalt fájlrendszer-időbélyegekre, a C időkezelésre és az értéktípus-szemléletre; ezeket itt nem ismétlem meg.

32.4.1 Duration, ratio és a mértékegység típusa

A `std::chrono::duration` sablon két paramétere a tároló típus és a tick hossza:

```
template<class Rep, class Period = std::ratio<1>>
class duration;
```

A `Period` egy `std::ratio<Num, Den>` specializáció, amely azt mondja meg, hogy egy tick hány másodperc: tick $= \frac{Num}{Den}$ s. A `duration::count()` a tickek számát adja vissza `Rep` típusban, tehát a duration értéke count() $\cdot \frac{Num}{Den}$ másodperc. A `Rep` nemcsak egész lehet: `duration<double>` másodpercet jelent `double` tickekkel.

A szerződés feltételei: `Rep` aritmetikai típus (legalábbis a szokásos műveletekhez), a `Period` nevezője pozitív, és a duration értéktípus — másolható, összehasonlítható, és a műveletek a két operandus `common_type` rep-típusával számolnak. Ebből következik a gyakori meglepetés: `seconds` és `milliseconds` összeadásának eredménye `common_type<seconds, milliseconds>`, azaz jellemzően `milliseconds`, nem `seconds`.

Az implicit konverzió feltétele (C++20 [time.duration.cons]) a cél rep-típusától függ. A `duration<Rep1, P1>` implicit módon `duration<Rep2, P2>`-vé konvertálható, ha a periódusarány számítása nem csordul túl, és (a) a cél `Rep2` **lebegőpontos** — ekkor bármely periódus megengedett; vagy (b) a cél `Rep2` **egész**, a forrás `Rep1` nem lebegőpontos, és a forrás tickje egész számú cél tick (`P1/P2` nevezője 1). Ezért implicit a `seconds` → `milliseconds` és a `milliseconds` → `duration<double>`, de nem implicit a `milliseconds` → `seconds` és a `duration<double>` → `seconds`. A szabály a periódusról és a lebegőpontosságról szól, **nem** a rep szűküléséről: a `duration<std::int64_t, std::milli>` → `duration<std::int16_t, std::milli>` és a `duration<double>` → `duration<float>` is implicit, és az engedélyezett konstruktor semmilyen értéktartomány-védelmet nem ad:

```
// duration_implicit.cpp - C++20: mely duration-konverzió implicit?
#include <chrono>
#include <cstdio>
#include <cstdint>
#include <type_traits>
using namespace std::chrono;
```

```cpp
// Egész cél-rep: a periódusarány egész, és a forrás rep nem lebegőpontos -
// a rep szűkülése (int64 -> int16) NEM akadály:
using ms16 = duration<std::int16_t, std::milli>;
static_assert(std::is_convertible_v<duration<std::int64_t, std::milli>, ms16>);
static_assert(std::is_convertible_v<seconds, milliseconds>);
// Lebegő cél-rep: bármely arány megengedett, a float pontosságvesztése sem
 ↪  akadály:
static_assert(std::is_convertible_v<duration<double>, duration<float>>);
static_assert(std::is_convertible_v<milliseconds, duration<double>>);
// Nem implicit: törtrészt veszítő arány egész célba, vagy lebegő forrás egész
 ↪  célba
static_assert(!std::is_convertible_v<milliseconds, seconds>);
static_assert(!std::is_convertible_v<duration<double>, seconds>);

int main() {
    ms16 small = duration<std::int64_t, std::milli>{100000};   // implicit, de
 ↪  100000 nem fér el
    std::printf("%d\n", static_cast<int>(small.count()));     // nem 100000
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic duration_implicit.cpp` figyelmeztetés nélkül fordul, minden `static_assert` teljesül, a kimenet `-31072`: a 100000 ms csendben túlcsordult a 16 bites rep-ben. Utóbbira a `duration_cast` való, amely explicit, és a dokumentált szemantikája szerint **csonkít, nem kerekít**: a végeredmény a cél rep-típusba `static_cast`-olva áll elő.

```cpp
using namespace std::chrono;
static_assert(duration_cast<seconds>(milliseconds(1500)).count() == 1);
static_assert(duration_cast<seconds>(milliseconds(-1500)).count() == -1);
```

A C++17-es `floor`, `ceil`, `round` a szándékot teszi explicitté. A `floor` lefelé (a negatív végtelen felé), a `ceil` felfelé kerekít. A `round` a legközelebbi tickre kerekít, **félúton a páros tick felé** (C++20 [time.duration.cast]): `round<seconds>(1500ms)` és `round<seconds>(2500ms)` is 2 s, `round<seconds>(-2500ms)` pedig -2 s. Pozitív és negatív, páros és páratlan félutas esetekkel:

```cpp
// chrono_round_even.cpp - C++20: floor, ceil, round, duration_cast
#include <chrono>

using namespace std::chrono;
using namespace std::chrono_literals;

// duration_cast: nulla felé csonkít
static_assert(duration_cast<seconds>(1500ms) == 1s);
static_assert(duration_cast<seconds>(-1500ms) == -1s);
// floor: lefelé, ceil: felfelé
static_assert(floor<seconds>(1500ms) == 1s);
static_assert(floor<seconds>(-1500ms) == -2s);
static_assert(ceil<seconds>(1500ms) == 2s);
static_assert(ceil<seconds>(-1500ms) == -1s);
// round: legközelebbi; félúton a PÁROS tick felé
static_assert(round<seconds>(1500ms) == 2s);    // 1 és 2 közül a páros
static_assert(round<seconds>(2500ms) == 2s);    // 2 és 3 közül a páros
static_assert(round<seconds>(-1500ms) == -2s);
```

```cpp
static_assert(round<seconds>(-2500ms) == -2s);
static_assert(round<seconds>(2600ms) == 3s);    // nem félút: legközelebbi

int main() {}
```

Buildjegyzőkönyv (`lab-gcc` profil): a `g++ -std=c++20 -Wall -Wextra -Wpedantic chrono_round_even.cpp` fordítás sikeres, minden `static_assert` teljesül. Timeout-számításnál ez nem kozmetika: egy lejárati időpont kiszámításánál a csonkítás rendre korábbi, a `ceil` későbbi határt ad, és a különbség éles rendszerben másodpercekben mérhető.

Levezetés (overflow-ellenőrzés). A `duration_cast` nem nyújt túlcsordulás-védelmet: ha a szorzat nem ábrázolható a cél rep-ben, az eredményre nincs hordozható garancia. Legyen H a `hours` száma, $N = 3\,600\,000\,000\,000$ a nanoszekundumok száma óránként. A $H \cdot N$ szorzat pontosan akkor ábrázolható `int64_t`-ben, ha $H \le \lfloor \text{INT64_MAX}/N \rfloor$ és $H \ge \lceil \text{INT64_MIN}/N \rceil$. Mivel a C++ egész osztása null felé csonkít, a negatív oldalon az `INT64_MIN / N` kifejezés éppen a felső korlátot adja, így a feltétel `H >= INT64_MIN / N` alakban is helyes. Például `hours(3'000'000)` esetén $H \cdot N = 1{,}08 \cdot 10^{19} > 9{,}22 \cdot 10^{18}$, tehát a konverzió nem hajtható végre ellenőrzés nélkül.

32.4.2 time_point, clock és a clock domain

A `time_point<Clock, Duration>` egy `Clock` epochjától mért `Duration`. A `Clock` nem tetszőleges osztály: teljesítenie kell a TrivialClock követelményeit — `rep`, `period`, `duration`, `time_point`, `is_steady` és `now()`. A `now()` adja az aktuális időpontot; a `time_point` kivonása durationt ad, a `time_point + duration` új `time_point`-ot. Két különböző clock `time_point`-ja nem vonható ki egymásból: ez fordítási hiba, és ez a típusrendszer egyik leghasznosabb védelme.

clock	szemantika	tipikus Linux-implementáció	mire való
`system_clock`	naptári, ugrálhat, nem garantáltan monoton	jellemzően `CLOCK_REALTIME`	timestamp, napló, naptári számítás
`steady_clock`	monoton, nem korrelál a naptárral	jellemzően `CLOCK_MONOTONIC`	mérés, timeout, backoff
`high_resolution_clock`	implementációfüggő, lehet a kettő aliasa	változó	csak ellenőrzött laborprofilban

A táblázat „tipikus” oszlopa implementációs megfigyelés, nem szabványos garancia: a szabvány a `system_clock` és a `steady_clock` szemantikáját rögzíti, a mögöttes `clockid_t` leképezést nem. A `high_resolution_clock` különösen csapdás: sok implementációban `steady_clock` alias, de nem kötelező, hogy az legyen, és nem kötelező, hogy `is_steady` igaz legyen rá. Ha a laborprofilban nem ellenőrizted, ne használd mérésre.

A clock domain fogalma azt jelenti, hogy egy időérték csak a saját órájához viszonyítva értelmezhető. A `steady_clock` epochja tetszőleges (jellemzően a boot pillanata), ezért a belőle készített számot nem szabad szerializálni, fájlba írni vagy más folyamatnak átadni: újraindítás után más epochhoz tartozik. A `system_clock` C++20 óta a naptári típusokkal összekapcsolt, Unix epochú óra (a `sys_time` 1970-01-01 00:00:00 UTC-től számol, szökőmásodperc nélkül); korábbi szabványokban az epoch nem volt rögzítve, ezért régi kódon a `to_time_t`/`from_time_t` pár ma is a hordozható átjáró. A C++20-as chrono kiegészítés ezt a kapcsolatot rögzíti, de a `system_clock` monotonitását nem.

Timeoutot mindig monoton órával kell számolni. A helyes minta: a határidőt egyszer, a ciklus előtt rögzítjük, majd a ciklusban a monoton órához hasonlítunk.

```
const auto deadline = std::chrono::steady_clock::now() +
 ↪  std::chrono::milliseconds(500);
while (std::chrono::steady_clock::now() < deadline) {
    // próbálkozás; a ciklus feltétele a valódi eltelt időt méri
}
```

A `std::this_thread::sleep_for` és a `condition_variable::wait_for` a szabvány szerint `steady_clock`-hoz kötött, tehát ezek is a monoton domainben maradnak. A `system_clock`-kal számolt deadline viszont NTP-ugrás után vagy azonnal lejár, vagy soha.

32.4.3 Kidolgozott példa: mérés, timeout, timestamp

A következő fordítási egység önállóan fordítható C++20-ban. Nem állítom, hogy lefuttattam; a viselkedés a laborprofil fordítási beállításaitól és a könyvtárverziótól függ, a buildparancsot a tanuló laborjában kell ellenőrizni.

```
#include <chrono>
#include <cstdint>
#include <iostream>
#include <limits>
#include <thread>

namespace chr = std::chrono;

// Mért idő: kizárólag steady_clock.
chr::nanoseconds measure_once()
{
    const auto t0 = chr::steady_clock::now();
    std::this_thread::sleep_for(chr::milliseconds(10));
    const auto t1 = chr::steady_clock::now();
    return chr::duration_cast<chr::nanoseconds>(t1 - t0);   // a period nem
     ↪  rögzített
}

// Overflow-tudatos konverziós feltétel: hours -> nanoseconds.
bool hours_fit_in_nanoseconds(chr::hours h)
{
    using rep = chr::nanoseconds::rep;                  // legalább 64 bites
     ↪  előjeles
    constexpr rep ns_per_hour = 3'600'000'000'000LL;
    constexpr rep max_v = std::numeric_limits<rep>::max();
    constexpr rep min_v = std::numeric_limits<rep>::min();
    return h.count() <= max_v / ns_per_hour && h.count() >= min_v / ns_per_hour;
}

// Naptári idő: system_clock, fix egységben, UTC-ben.
std::int64_t now_unix_millis()
{
    const auto since_epoch = chr::system_clock::now().time_since_epoch();
    return chr::duration_cast<chr::milliseconds>(since_epoch).count();
}
```

```
int main()
{
    const auto elapsed = measure_once();
    std::cout << "elapsed_us="
              << chr::duration_cast<chr::microseconds>(elapsed).count() << '\n';

    const auto deadline = chr::steady_clock::now() + chr::milliseconds(50);
    std::uint64_t spins = 0;                          // unsigned: definiált
     ↪  körbefordulás
    while (chr::steady_clock::now() < deadline) { ++spins; }
    std::cout << "spins=" << spins << '\n';

    std::cout << "unix_ms=" << now_unix_millis() << '\n';

    const auto big = chr::hours(3'000'000);
    std::cout << "3e6h fits in nanos: " << std::boolalpha
              << hours_fit_in_nanoseconds(big) << '\n';
    if (hours_fit_in_nanoseconds(big)) {
        std::cout << chr::duration_cast<chr::nanoseconds>(big).count() << '\n';
    }
}
```

```
g++ -std=c++20 -O2 -Wall -Wextra -Wconversion -Wsign-conversion \
    chrono_demo.cpp -o chrono_demo
```

A példa három szerződést mutat. A `measure_once` csak `steady_clock`-ot használ, és `nanoseconds`-t ad vissza, hogy a hívó dönthesse el a megjelenítési egységet. A `steady_clock::duration` periodja implementációfüggő, ezért a konverzió kifejezett `duration_cast` (egy durvább period implicit módon is nanoszekundumba váltana, de erre nem építünk). A pörgő ciklus számlálója `std::uint64_t`: a futási idő önmagában nem nyelvi garancia arra, hogy egy `int` nem csordul túl. Az óra- és a C oldali másodperc-skálázás előtt is ellenőrizzük az ábrázolhatóságot; a `nanoseconds::rep` szabvány szerint legalább 64 bites előjeles egész, a C oldali `int64_t` és 64 bites `time_t` viszont az LP64 Linux ABI-jának tulajdonsága. A timeout-határidő a ciklus előtt rögzített `steady_clock` időpont. A `now_unix_millis` a naptári domaint fix, egész egységre képezi le, ami szerializálható. Az `hours_fit_in_nanoseconds` pedig azt demonstrálja, hogy a `duration_cast` előtt a hívónak kell ellenőriznie az ábrázolhatóságot.

A C-oldali kontraszt rávilágít a különbségre: ott a clock domain a függvény argumentuma, nem a típus része.

```
#define _POSIX_C_SOURCE 200809L
#include <time.h>
#include <stdint.h>

/* Mért idő: CLOCK_MONOTONIC; a clock id kiválasztása a hívó felelőssége. */
int64_t monotonic_ns(void)
{
    struct timespec ts;
    if (clock_gettime(CLOCK_MONOTONIC, &ts) != 0) return -1;
    if (ts.tv_sec > (INT64_MAX - ts.tv_nsec) / 1000000000LL) return -1;  /* nem
     ↪  ábrázolható */
    return (int64_t)ts.tv_sec * 1000000000LL + ts.tv_nsec;
}
```

```
cc -std=c17 -Wall -Wextra monotonic.c -o monotonic
```

A C változatban semmi nem akadályozza meg, hogy ugyanabba a változóba `CLOCK_REALTIME` értéket tegyen a programozó; C++-ban ez a hiba típushibává válik, amíg valaki `time_since_epoch()`-en keresztül nem „lapítja” el a típusokat.

32.4.4 Naptár, timezone és DST C++20-ban

A C++20-as chrono naptári típusai (`year_month_day`, `sys_days`, `hh_mm_ss`, `weekday`) és a timezone-réteg (`time_zone`, `zoned_time`, `current_zone`, `locate_zone`, `tzdb`) a naptári domainre épülnek. A `sys_time<Duration>` a `system_clock` időpontja, a `local_time<Duration>` egy időzóna nélküli helyi idő, a `zoned_time` pedig a kettőt köti össze egy konkrét zónával.

A helyi idő és a `sys_time` közötti átjárás nem függvény, mert nem is lehet az: a nyári időszámítás miatt egy helyi időpont lehet egyértelmű, nem létező vagy kétértelmű. A `time_zone::get_info(local_time)` `local_info`-t ad vissza, amelynek `result` mezője `unique`, `nonexistent` vagy `ambiguous`. A `zoned_time` konstruktora kétértelmű esetben `choose::earliest` vagy `choose::latest` paramétert vár; a nem létező helyi időt a programnak kell kezelnie, nem szabad „valahogy” átfordítani.

```
// C++20 timezone-részlet. A <chrono> tzdb-támogatása könyvtárverzió-függő:
// fordítási próbával és a __cpp_lib_chrono feature-test makróval ellenőrizd.
#include <chrono>

void describe_local_time(const std::chrono::local_time<std::chrono::seconds>&
 ↪  lt)
{
    const std::chrono::time_zone* zone = std::chrono::current_zone();
    const std::chrono::local_info info = zone->get_info(lt);
    switch (info.result) {
    case std::chrono::local_info::unique:      /* egyértelmű */ break;
    case std::chrono::local_info::nonexistent: /* DST-ugrás miatt nem létezik
     ↪  */ break;
    case std::chrono::local_info::ambiguous:   /* óra visszaállítása miatt
     ↪  kétértelmű */ break;
    }
}
```

Két korlátot itt is hangsúlyozni kell. Egyrészt a timezone-adatbázis (`tzdb`) nem a szabvány része, hanem a rendszer/könyvtár szolgáltatja; tartalma és frissítési ritmusa változik, és a `reload_tzdb()` csak akkor segít, ha a folyamat hajlandó újratölteni. Másrészt a szökőmásodpercek külön órán élnek: a `utc_clock`, `tai_clock` és `gps_clock`, valamint a `clock_cast` kezeli őket, a `system_clock` nem számol szökőmásodperccel. A timezone-adatbázis vagy a teljes Unicode-kezelés saját implementációja kívül esik ezen az alfejezeten; a szövegkódolás és locale kérdéseit az előző rész tárgyalja.

32.4.5 Timestamp-szerializálás és clock domain

A szerializálás szabálya egyszerű, de következménye van: **a tartósan tárolt és továbbított időérték naptári idő, fix egységben, UTC-ben**. A `steady_clock` értékét tilos kiírni. A `system_clock::time_point`-ból `time_since_epoch()`-tel duration lesz, `duration_cast`-tal fix egységre váltunk, `count()`-tal egész számot kapunk; visszafelé a

`sys_time<milliseconds>{milliseconds{value}}` rekonstruál. Az `int64_t` milliszekundum ± 292 millió év tartományt fed le, a nanoszekundum ± 292 évet — ez a tartomány a gyakorlati rendszerekben elég, de a határt dokumentálni kell.

A protokollnak rögzítenie kell a clock domaint is: melyik óra, melyik egység, mi történik óraugráskor, és hogy a sorrendiséghez használt idő melyik domainből jön. Egy naplóban a `system_clock` időbélyeg és a `steady_clock`-ból számolt relatív eltelt idő együtt értelmes: az előbbi megmondja, mikor, az utóbbi megmondja, mennyi ideig. Önmagában egyik sem elég.

32.4.6 Tipikus hibák és javításuk

1. **Mérés `system_clock`-kal.** NTP-ugrás vagy kézi állítás után negatív vagy hihetetlen latencia. Javítás: mérésre és timeoutra `steady_clock`.
2. **`high_resolution_clock` steady feltételezése.** Hordozhatatlan. Javítás: `steady_clock`, vagy a laborprofilban ellenőrzött `is_steady`.
3. **`duration_cast` kerekítésnek gondolása.** A csonkítás rendre korábbi határt ad. Javítás: `floor`/`ceil`/`round` a szándék szerint.
4. **Ellenőrizetlen `duration_cast` nagy egységekből.** Csendes túlcsordulás. Javítás: a szorzat ábrázolhatóságának ellenőrzése a hívónál.
5. **Helyi idő tárolása.** DST-átmenetnél kétértelmű vagy nem létező érték. Javítás: UTC-ben tárolás, lokalizálás csak megjelenítéskor, `get_info`-val.
6. **Két clock domain összekeverése `time_since_epoch()`-en keresztül.** A típusrendszer megkerülése; a hiba csak futásidőben látszik. Javítás: a `time_point` típusát a végpontig megőrizni.

32.4.7 Feladatok

1. feladat (magyarázó). Indokold, melyik clockot használnád: (a) egy HTTP-kérés időbélyegzője, (b) egy adatbázis-lekérdezés latenciája, (c) egy retry backoff határideje, (d) egy naplósor és a hozzá tartozó „eltelt 120 ms” mező.

2. feladat (kódolási). Írj `to_seconds_ceil(std::chrono::milliseconds)` függvényt, amely felfelé kerekítve adja vissza a másodpercek számát, és negatív értékre is helyes. Add meg az ellenőrzési feltételt.

3. feladat (diagnosztikai). Adott a következő kódrészlet. Mi a hiba, milyen tünetet okoz NTP-ugrás után, és hogyan javítod?

```
const auto start = std::chrono::system_clock::now();
do_work();
const auto elapsed = std::chrono::system_clock::now() - start;
log("elapsed_ms",
 ↪  std::chrono::duration_cast<std::chrono::milliseconds>(elapsed).count());
```

4. feladat (kódolási). Szerializálj egy `system_clock::time_point`-ot `int64_t` Unix-milliszekundumba, majd olvasd vissza. Add meg a tartományt, és azt, hogy mi történik, ha a bemenet a tartományon kívül esik.

32.4.8 Megoldások

1. megoldás. (a) `system_clock`: naptári idő, más rendszerekkel összevethető. (b) `steady_clock`: csak az eltelt idő számít, a naptári ugrás nem. (c) `steady_clock`: a határidő a folyamat élettartamán belül értelmezett, monoton kell. (d) kettő: a naplósor időbélyegzője `system_clock`, az eltelt idő `steady_clock`; a kettőt külön mezőben kell tartani, nem összeadni. Ellenőrzés: minden válaszhoz indokold, hogy az adott értéket tárolják-e, összehasonlítják-e egy másik folyamat értékével, és átél-e újraindítást.

2. megoldás. A `ceil` a `duration_cast` csonkítását ellensúlyozza, de a negatív irányt külön kell kezelni, mert a `ceil` a matematikai mennyezet, nem a nullától elfelé kerekítés.

```
#include <chrono>

std::chrono::seconds to_seconds_ceil(std::chrono::milliseconds ms)
{
    using namespace std::chrono;
    const auto s = duration_cast<seconds>(ms);
    if (s < ms) { return s + seconds{1}; }  // volt csonkolt maradék
    return s;
}
```

Ellenőrzési feltétel: `to_seconds_ceil(1500ms) == 2s`, `to_seconds_ceil(2000ms) == 2s`, `to_seconds_ceil(-1500ms) == -1s`, `to_seconds_ceil(0ms) == 0s`. Az utolsó három eset mutatja, hogy a „maradék” feltétel a negatív oldalon is a matematikai mennyezetet adja, mert a csonkítás null felé történt.

3. megoldás. A hiba, hogy mérésre `system_clock`-ot használ. Ha az NTP a mérés közben előre állítja az órát, az `elapsed` negatív lesz, és a `duration_cast` negatív milliszekundumot ad; ha hátraállítja, az eltelt idő irreálisan nagy lesz. A javítás: `steady_clock` a két `now()` hívásra, a naplózott időbélyeg pedig külön `system_clock` érték. Ellenőrzési feltétel: a javított kódban a mérés két `steady_clock::now()` különbsége, és a naplóban a két clock értéke külön mezőben szerepel.

4. megoldás.

```
#include <chrono>
#include <cstdint>
#include <limits>

std::int64_t to_unix_millis(std::chrono::system_clock::time_point tp)
{
    return std::chrono::duration_cast<std::chrono::milliseconds>(
               tp.time_since_epoch()).count();
}

bool from_unix_millis(std::int64_t v, std::chrono::system_clock::time_point&
 ↪  out)
{
    using namespace std::chrono;
    const auto ms = milliseconds{v};
    const auto tp =
     ↪  system_clock::time_point{duration_cast<system_clock::duration>(ms)};
    if (duration_cast<milliseconds>(tp.time_since_epoch()).count() != v) {
```

```
        return false;  // a rendszer duration felbontása nem tudta visszaadni
    }
    out = tp;
    return true;
}
```

A tartomány `int64_t` milliszekundumban $\pm 9{,}22 \cdot 10^{18}$ ms, azaz mintegy ± 292 millió év; a gyakorlati korlátot a `system_clock::duration` felbontása és a naptári típusok tartománya szabja meg. Tartományon kívüli bemenetre a `milliseconds{v}` konstrukció túlcsordul, ezért a bemenetet a hívónak kell ellenőriznie; a visszaolvasás utáni összehasonlítás a `system_clock::duration` felbontásából eredő veszteséget is kiszűri. Ellenőrzési feltétel: kerek értékre (`0`, `1'700'000'000'000`) a visszaolvasás egyenlő; olyan értékre, amelyet a rendszer duration felbontása nem ábrázol, a függvény `false`-t ad.

32.4.9 Összefoglaló

A `chrono` lényege a domainek és egységek típusba kódolása. A `duration` tickekből és `ratio`-ból áll; az implicit konverzió csak veszteségmentes esetben engedett, a `duration_cast` explicit és csonkít, a `floor`/`ceil`/`round` a szándékot nevezi meg. A `time_point` a clockhoz kötött, és két clock időpontja nem keverhető. Mérésre és timeoutra `steady_clock`, naptári időre `system_clock` való; a `high_resolution_clock` csak ellenőrzött laborprofilban. A C++20 naptári és timezone-rétege a `sys_time`/`local_time`/`zoned_time` hármassal dolgozik, a DST-kétértelműséget `get_info`-val és `choose`-szal kell kezelni; a tzdb külső adatfüggőség, a szökőmásodperc külön órán él. Tartós tárolásra UTC-alapú, fix egységű, egész típusú timestamp való, a `steady_clock` értéke soha nem szerializálható, és a protokollnak rögzítenie kell a clock domaint.

32.5 Szövegkódolás, locale és random

A C szövegkezelését a Szövegkódolás, locale és UTF-8 C-ben (7.5) részben byte-sorozatként, nullterminált sztringként és locale-függő C függvényekkel modelláltuk. A chrono, duration és időpont (32.4) részben rögzítettük az időmérés és az óraolvasás szerződését, a Multidimenziós view és numeric utility térkép (31.5) rész pedig a numerikus utility-k térképét adta. Most a C++ oldalon ugyanazt a kérdést vesszük elő: mit ígér a standard könyvtár a szövegről, fájlokról, időről és I/O-ról, és hol ér véget ez az ígéret a libc/POSIX szemantikához képest. Ez a rész a szövegkódolás, a locale és a véletlenszámok hármasára szűkít; a fájlrendszer és a chrono részletei a filesystem, path és OS-határ (32.3), illetve a chrono, duration és időpont (32.4) anyagában maradnak.

A C++20 standard és a C++23 munkatervezet szövege nem automatikusan ellenőrzött ebben a fejezetben; a konkrét szakaszok, implementációs részletek és feature-test értékek a labor profiljában ellenőrizendők. A kódpéldák szemléltetők; a fejezet nem állítja, hogy azokat lefuttatták.

32.5.1 A karaktertípus és a tároló elválasztása

A C++ szövegmodelljének első döntése, hogy a **karaktertípus** és a **kódolás** két külön kérdés. A `std::string` byte-tároló: a `size()` a tárolt `char` objektumok számát adja, nem a felhasználó által érzékelt karakterekét, és nem is a Unicode kódpontokét. A `std::u8string` `char8_t` kódegységeket tárol, de a tartalom UTF-8 érvényességét nem ellenőrzi. A típus tehát a tárolási egységet rögzíti, a kódolást nem.

Ehhez három fogalmat kell szétválasztani. A **kódpont** (code point) egy Unicode skaláris érték: $0 \le c \le 0x10FFFF$, a szürrogát tartomány kivételével. A **kódegység** (code unit) az adott kódolás alapegysége: UTF-8-ban egy byte, UTF-16-ban egy 16 bites szó, UTF-32-ban egy 32 bites szó. A **grapheme cluster** a felhasználó által egy karakternek érzékelt egység, amely több kódpontból állhat: az „e” és a U+0301 combining acute accent két kódpont, de egy grapheme; egy család-emoji ZWJ-szekvenciája pedig több kódpontból álló grapheme. A `std::u8string::size()` a kódegységek számát adja, a `utf8_codepoints` jellegű számlálás a kódpontokét; egyik sem grapheme-számláló.

32.5.1.1 A karaktertípusok és literáljaik

Típus	Standard szerinti szerep	Megjegyzés
`char`	byte-tároló, UTF-8 byte-sor is lehet	signednesse implementációfüggő
`char8_t`	UTF-8 kódegység	C++20; az `unsigned char`-ral azonos méret, alignment és reprezentáció, tehát `CHAR_BIT` bit
`char16_t`	UTF-16 kódegység	C++11; legalább 16 bit, a gyakorlatban 16
`char32_t`	UTF-32 kódpont	C++11; legalább 32 bit, a gyakorlatban 32
`wchar_t`	implementációfüggő széles karakter	szélessége és kódolása platformfüggő; nem hordozható Unicode-tároló

A literálok típusa ezt követi: `"..."` típusa `const char[N]`; `u8"..."` C++20-tól `const char8_t[N]`, C++17-ig `const char[N]`; `u"..."` `const char16_t[N]`; `U"..."` `const char32_t[N]`; `L"..."` `const wchar_t[N]`. A `u8` literál típusváltozása C++20-ban szándékos, de a C++17-es kódbázisokban törő változás: a `std::string`-et váró függvénynek átadott `u8"..."` már nem fordul, és a `std::cout << u8"..."` sem, mert a `char8_t`-re nincs stream-insertion. A `std::u8string` és a `std::string` között nincs implicit konverzió; a byte-szintű átvételt explicit módon kell megtenni, például `std::string(u8s.begin(), u8s.end())` alakban, tudva, hogy a `char8_t` értékek `char`-ba konvertálása a signedness miatt negatív értékeket adhat, miközben a byte-reprezentáció változatlan.

A `char16_t` és `char32_t` Unicode-kódegység, de a `std::u16string` és `std::u32string` sem normalizál, sem grapheme-et nem számol. A `std::wstring` használata akkor indokolt, ha a platform API `wchar_t`-t vár; hordozható adattárolásra nem.

32.5.2 Konverziók, locale és az elavult codecvt

A `std::locale` nem kódoláskonverter. Az a szerepe, hogy a formázási, collation és karakterosztályozási faceteket biztosítsa: például a számok ezres tagolását, a dátumformátumot, a `std::regex` karakterosztályait. A locale neve platformfüggő (`"hu_HU.UTF-8"`, `"C"`), a `std::locale("")` a környezetből vesz, a `std::locale::global` pedig globális mellékhatással jár. A `std::format` C++20-ban alapértelmezés szerint locale-független; a locale-t explicit kell átadni, és a `L` specifikátor jelzi a locale-függő formázást. A C++23-as `std::print` hasonló elven működik, de használata a `__cpp_lib_print` feature-testhez és a konkrét library támogatásához kötött.

A Unicode-konverzióra a C++20 standard nem ad kész, hordozható megoldást. A `std::codecvt` facetek és a `std::wstring_convert` C++17-ben elavulttá váltak; a C++20-ban még elérhetők, de használatuk kerülendő, és eltávolításuk verziófüggő. A `std::mbrtowc`/`std::wcrtomb` C függvények locale-függők és állapotfüggők; a `setlocale` és a C++ locale kapcsolata platformfüggő. A gyakorlatban platform API (`iconv`, Win32 `MultiByteToWideChar`) vagy harmadik fél (ICU) marad. A `std::regex` szintén locale-függő lehet, de nem UTF-8-tudatos: `std::regex` a `char`-on byte-alapon illeszt, `std::wregex` a `wchar_t`-n. A legtöbb implementáció visszalépéses, a standard nem garantál lineáris időt; nagy, támadó bemeneten a `std::regex` ReDoS-ra hajlamos lehet. Kis, ellenőrzött bemenetre vagy kézi parserre érdemes támaszkodni.

32.5.3 Random: engine és distribution

A C++ `<random>` két külön objektumot ad. Az **engine** egy `UniformRandomBitGenerator`: `min()`, `max()` és `operator()` szerződéssel, determinisztikus bit-generátor. A **distribution** az engine által adott egyenletes bitsorozatot alakítja a kívánt eloszlássá. Az engine és a distribution állapota különálló; a distribution `operator()` jellemzően nem `const`, mert belső állapotot módosíthat.

Az `std::mt19937` és `std::mt19937_64` algoritmusa specifikált, ezért azonos seeddel azonos bitsorozatot adnak. A `std::default_random_engine` ezzel szemben implementációfüggő, hordozható kódban kerülendő. A distribution kimenete nem garantáltan bitre reprodukálható: a standard a statisztikai tulajdonságokat írja elő, a konkrét algoritmust nem. Ha bitre azonos futtatás kell, rögzített engine és saját eloszlás, vagy dokumentáltan rögzített implementáció szükséges.

32.5.3.1 Seeding és a security kérdése

A seedeléshez a `std::random_device` népszerű, de nem univerzális security garancia. A `std::random_device::entropy()` visszatérhet `0.0`-val, ami azt jelzi, hogy a forrás determinisztikus lehet; a `std::random_device` nem garantáltan kriptográfiai erősségű. A `std::seed_seq` a seed-adatokat keveri, de nem növeli az entrópiát. A `std::chrono::steady_clock` időbélyegből képzett seed kiszámítható, és futásonként változik, ezért önmagában éppen **nem** reprodukálható. Reprodukálható teszthez a seed legyen rögzített vagy explicit bemenet (például parancssori argumentum vagy környezeti változó, alapértelmezett fix értékkel); ha a teszt változó seeddel fut, a ténylegesen használt seedet minden futás elején naplózni kell, hogy egy sikertelen futás ugyanazzal a seeddel és azonos profillal újrajátszható legyen. Biztonsági célra platform CSPRNG kell: Linuxon a `getrandom(2)`, Windowson a `BCryptGenRandom`; a generált bájtokból legalább 128 bit entrópiával, base32/hex kódolással készül a token, és az összehasonlítás állandó idejű. A `std::mt19937` kimenete nem kriptográfiai, akkor sem, ha a seedet `std::random_device` adta.

A reprodukálható és a biztonsági cél tehát külön API-hoz kötendő: teszthez rögzített seedű `std::mt19937`/`std::mt19937_64` és rögzített distribution; biztonsági célra platform CSPRNG, nem `<random>`.

32.5.4 Kidolgozott példa: UTF-8 számlálás és seedelt generátor

A következő C++20-as példa a `char8_t` tárolást és a seedelt generátort mutatja. A `utf8_codepoints` feltételezi az érvényes UTF-8 bemenetet, és a folytató bájtokból számol; nem validál, és nem grapheme-et számol.

```
// u8demo.cpp
#include <array>
#include <cstddef>
```

```cpp
#include <cstdint>
#include <cstdio>
#include <random>
#include <string_view>

std::size_t utf8_codepoints(std::u8string_view s) noexcept {
    std::size_t n = 0;
    for (unsigned char b : s) {
        if ((b & 0xC0u) != 0x80u) {
            ++n;
        }
    }
    return n;
}

int main() {
    constexpr std::u8string_view text = u8"árvíztűrő";
    std::printf("bytes=%zu codepoints=%zu\n",
                text.size(), utf8_codepoints(text));

    std::random_device rd;
    std::array<std::uint32_t, 8> seed_data{};
    for (auto& x : seed_data) {
        x = static_cast<std::uint32_t>(rd());
    }
    std::seed_seq seq(seed_data.begin(), seed_data.end());
    std::mt19937_64 engine(seq);
    std::uniform_int_distribution<int> d6(1, 6);
    std::printf("d6=%d\n", d6(engine));
}
```

C++20-at támogató GCC vagy Clang szükséges; a konkrét verziót a labor profil rögzíti. Fordítás:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o u8demo u8demo.cpp
```

A `char8_t` és a `std::u8string_view` elérhetősége a `__cpp_char8_t` és `__cpp_lib_char8_t` feature-testtel ellenőrizhető. A példában az `u8"árvíztűrő"` 13 byte és 9 kódpont; ezt a string literál ismert tartalma adja, nem futtatási eredmény. A `d6` kimenete a `random_device` entrópiájától függ, ezért nem determinisztikus.

32.5.5 Gyakori hibák és javításuk

- **A `u8"..."` típusa C++20-ban `const char8_t[]`.** A C++17-es `std::string`-átvétel és a `std::cout << u8"..."` nem fordul. Javítás: explicit `std::string`-konverzió vagy byte-szintű kezelés.
- **A `std::string::size()` nem grapheme- és nem kódpontszám.** A combining accent és a ZWJ-emoji több kódpont; a `size()` kódegység. Javítás: kódpontszámláló vagy Unicode-könyvtár, ha grapheme kell.
- **A `std::random_device` nem CSPRNG.** Az `entropy() == 0.0` determinisztikus forrást jelezhet. Javítás: platform CSPRNG biztonsági célra.
- **A `std::default_random_engine` és a distribution nem hordozható bitre.** Javítás: specifikált engine és saját eloszlás, ha reprodukálhatóság kell.

- **A `std::regex` ReDoS-ra hajlamos és nem UTF-8-tudatos.** Javítás: kis bemenet, kézi parser vagy harmadik fél.
- **A `std::locale::global` globális mellékhatás.** Javítás: explicit locale a formázási ponton.
- **A `std::codecvt` és a `std::wstring_convert` elavult.** Javítás: platform API vagy harmadik fél.

32.5.6 Feladatok

1. **Magyarázó feladat.** Különböztesd meg a kódpont, a kódegység és a grapheme cluster fogalmát. Adj példát az „e” + U+0301 szekvenciára és egy ZWJ-emoji családra. Miért nem adja a `std::u8string::size()` a grapheme-ek számát?

2. **Kódolási feladat.** Írj `bool is_valid_utf8(std::string_view)` függvényt, amely szigorúan validálja az UTF-8 byte-sorozatot. A `std::string_view` itt byte-tároló; a függvény ne értelmezze grapheme-ként. Adj legalább négy tesztesetet: üres string, ASCII, túl hosszú kódolás (`0xC0 0xAF`), szürrogát (`0xED 0xA0 0x80`) és egy érvényes többszörös byte-os karaktert.

3. **Diagnosztikai feladat.** Az alábbi kód egy jelszó-visszaállítási tokent generál. Mi a hiba, ha a cél biztonsági token? Mi a hiba, ha a cél reprodukálható teszt?

```
std::mt19937 eng(std::random_device{}());
std::uniform_int_distribution<int> d(0, 35);
```

4. **Diagnosztikai feladat.** Egy szolgáltatás `std::regex`-szel validál felhasználói bemenetet. Miért lehet ez ReDoS-kitett, és miért nem UTF-8-tudatos a `std::regex`? Milyen ellenőrzést vezetnél be?

32.5.7 Megoldások

32.5.7.1 1. megoldás

A kódpont a Unicode skaláris érték; a kódegység az adott kódolás alapegysége (UTF-8 byte, UTF-16 16 bites szó); a grapheme a felhasználó által érzékelt egység. Az „e” + U+0301 két kódpont, UTF-8-ban három byte, de egy grapheme. A ZWJ-emoji család több kódpontból és több ZWJ-ből áll, mégis egy grapheme. A `std::u8string::size()` a `char8_t` kódegységek számát adja, ezért sem a kódpontokét, sem a grapheme-ekét nem. Ellenőrzési feltétel: a válasz mindhárom fogalmat egyértelműen elhatárolja, és a `size()` értelmét a tárolt kódegységekhez köti.

32.5.7.2 2. megoldás

```
#include <cstddef>
#include <string_view>

bool is_valid_utf8(std::string_view s) noexcept {
    std::size_t i = 0;
    while (i < s.size()) {
        const unsigned char c = static_cast<unsigned char>(s[i]);
        if (c <= 0x7Fu) {
            ++i;
```

```
            continue;
        }

        std::size_t len = 0;
        unsigned char lo = 0x80u;
        unsigned char hi = 0xBFu;

        if (c >= 0xC2u && c <= 0xDFu) {
            len = 2;
        } else if (c >= 0xE0u && c <= 0xEFu) {
            len = 3;
            if (c == 0xE0u) lo = 0xA0u;
            if (c == 0xEDu) hi = 0x9Fu;
        } else if (c >= 0xF0u && c <= 0xF4u) {
            len = 4;
            if (c == 0xF0u) lo = 0x90u;
            if (c == 0xF4u) hi = 0x8Fu;
        } else {
            return false;
        }

        if (i + len > s.size()) return false;
        const unsigned char c1 = static_cast<unsigned char>(s[i + 1]);
        if (c1 < lo || c1 > hi) return false;

        for (std::size_t k = 2; k < len; ++k) {
            const unsigned char ck = static_cast<unsigned char>(s[i + k]);
            if (ck < 0x80u || ck > 0xBFu) return false;
        }
        i += len;
    }
    return true;
}
```

A `0xC0`/`0xC1` vezető byte túl hosszú kódolást jelentene, ezért nem szerepel a megengedett tartományban; a `0xE0` és `0xF4` speciális második byte-tartománya a túl hosszú, illetve a `U+10FFFF` feletti kódpontokat zárja ki; a `0xED` a szürrogátokat. Tesztesetek:

```
#include <cassert>
int main() {
    assert(is_valid_utf8(""));
    assert(is_valid_utf8("abc"));
    assert(!is_valid_utf8("\xC0\xAF"));
    assert(!is_valid_utf8("\xED\xA0\x80"));
    assert(is_valid_utf8("\xC3\xA1"));
}
```

Ellenőrzési feltétel: a függvény minden nem valid szekvenciára `false`-t ad, a validakra `true`-t; a határfeltételek (`i + len > s.size()`, a folytató byteok tartománya) helyesek.

32.5.7.3 3. megoldás

Biztonsági tokenhez a `std::mt19937` nem kriptográfiai; a `std::random_device{}()` egyetlen `unsigned int`-et ad, legfeljebb 32 bitet, és lehet determinisztikus; a `std::uniform_int_distribution` kimenete nem garantáltan bitre reprodukálható, de ez itt nem is biztonsági kérdés. A tokenhez platform CSPRNG (`getrandom` Linuxon) és legalább 128 bit entrópia kell, base32/hex kódolással, állandó idejű összehasonlítással. Reprodukálható teszthez rögzített seedű `std::mt19937_64` és rögzített distribution használható, de a kettő nem ugyanaz az API. Ellenőrzési feltétel: a válasz külön kezeli a biztonsági és a reprodukálhatósági igényt, és nem állítja, hogy a `std::random_device` CSPRNG.

32.5.7.4 4. megoldás

A legtöbb `std::regex` implementáció visszalépéses, a standard nem garantál lineáris időt; a bemenet hosszával a futásidő exponenciálisan nőhet. A `std::regex` a `char`-on byte-alapú, nem UTF-8-kódpont-tudatos, ezért a `.` egy byte-ot jelent, nem egy kódpontot. Ellenőrzési feltétel: a bemenet mérete korlátozott, a regex csak kis, ellenőrzött bemenetre fut, és a laborban időmérés történik; nagy bemenetre kézi parser vagy harmadik fél használható.

32.5.8 Összefoglaló

A C++ szövegmodelljében a karaktertípus a tárolási egységet rögzíti, a kódolást nem. A `std::string` byte-tároló, a `std::u8string` `char8_t`-tároló; egyik sem grapheme- és nem is automatikusan validáló konténer. A `u8` literál típusa C++20-ban `const char8_t[]`, ami törő változás a C++17-es kódhoz képest. A locale nem encoding; a `std::codecvt` és a `std::wstring_convert` elavult, a `std::regex` nem UTF-8-tudatos és ReDoS-kitett. A random esetében az engine és a distribution külön szerződés: az engine lehet specifikált és reprodukálható, a distribution nem garantáltan bitre az; a `std::random_device` nem univerzális security garancia, biztonsági célra platform CSPRNG kell.

33 Compile-time C++, ODR és modulok

33.1 constexpr, consteval és constant evaluation

Ez az alfejezet a fordítást nem melléktermékként, hanem tervezett erőforrásként kezeli: a C++ két különböző dolgot enged meg ugyanazzal a kulcsszóval körülírni – hogy egy kifejezés *lehetséges* fordítási időben kiértékelni, illetve hogy *kötelező* azt fordítási időben kiértékelni. Egy DevOps-környezetben ez nem akadémiai kérdés: a konfigurációs kulcsszavak, protokoll-nevek, hibakód-táblák és a hozzájuk tartozó keresőkód tipikus jelöltjei annak, hogy a build már a hibás táblát is elutasítsa, ne a produkciós futás. A célunk az, hogy egy kis parser/lookup táblán meg tudjuk mondani minden egyes deklarációról: mi benne a lehetőség, mi a kötelezettség, és mi a garancia hiánya.

Az előfeltételek röviden átismétlendők: a statikus és thread-local objektumok inicializálási sorrendje és életciklusa (Statikus és thread-local objektumok életciklusa (20.5)), a fordítási idejű elágazás eszközei (Type traits, tag dispatch és if constexpr (27.1)), valamint a korábbi állapotgép/parser szerződés (Bounds-ellenőrzött parser és állapotgép (13.5)). Az itteni tábla nem új parser, hanem a korábbi állapotgép lexikális segédtáblája.

33.1.1 const, constexpr, consteval, constinit: négy különböző szerződés

A négy kulcsszó gyakori félreértése abból fakad, hogy mindegyik „fordítási időhöz” kötődik, de más-más oldalról. A `const` kizárólag **hozzáférési** tulajdonság: az objektum nem módosítható ezen a néven keresztül, attól még inicializálható futásidőben (`const int port = read_port();`). A `constexpr` **változón** ennél többet ígér: az inicializáló kifejezés konstans kifejezés, a típus literal típus, és a változó neve felhasználható konstans kifejezésben. A `consteval` **függvényre** tett kötelezettség: minden potenciálisan kiértékelt hívásának konstans kifejezésnek kell lennie. A `constinit` pedig nem a változó használhatóságát, hanem az **inicializálás módját** rögzíti: az objektumnak statikus (vagy thread) tárolási idővel kell rendelkeznie, és konstans inicializálást kell kapnia; cserébe a változó továbbra is módosítható, és nem válik konstans kifejezésben használhatóvá.

Eszköz	Hol áll	Mit garantál	Mit nem garantál
`const`	objektum	írásvédett hozzáférés ezen a néven	hogy az érték fordítási időben ismert
`constexpr` változó	objektum	konstans kifejezéssel inicializált, konstans kifejezésben használható	hogy a programban mindenhol kizárólag fordítási időben létezik
`constexpr` függvény	függvény	van legalább egy olyan argumentum-értékelés, amely konstans kifejezés	hogy minden hívás fordítási idejű

Eszköz	Hol áll	Mit garantál	Mit nem garantál
`consteval`	függvény	minden hívás fordítási idejű (immediate function)	hogy a futó programban egyáltalán jelen van
`constinit`	objektum	statikus inicializálás, nincs SIOF	hogy az érték konstans kifejezésben felhasználható

Két fogalmat kell pontosítani. **Konstans kifejezés** (constant expression): olyan kifejezés, amelynek kiértékelése megfelel a nyelvi szabálynak – nincs benne tiltott művelet (többek között `reinterpret_cast`, nem konstans függvény hívása, definiálatlan memóriához nyúlás), és az eredmény típusa megengedett. **Literal típus**: skalár, referencia, illetve olyan osztálytípus, amelynek triviális destruktora van, minden nem statikus adattagja és bázisa literal, és rendelkezik megfelelő `constexpr` konstruktorral. A `std::string_view` literal típus, a `std::string` nem – ez a különbség a példánkban is számít.

33.1.2 constexpr függvény: lehetőség, nem kötelezettség

A `constexpr` függvény implicit módon `inline`, és két életet él. Ha konstans kifejezést váró helyen hívjuk (például `static_assert`-ben, `constexpr` változó inicializálójaként vagy `consteval` függvény törzsében), a fordító köteles konstans kiértékelést végezni, és hibát jelez, ha ez nem lehetséges. Ha futásidejű argumentummal hívjuk, ugyanaz a függvény egyszerűen lefordul közönséges hívásként. Ez a kettősség a `constexpr` lényege, és egyben a legtöbb félreértés forrása: a `constexpr` nem „fordítási időben lefutó függvény”, hanem „fordítási időben is lefutni képes függvény”.

A szabványi követelmény másik fele: a függvénynek *léteznie kell* olyan argumentumra, amelyre konstans kifejezést ad; ha ilyen nincs, a program ill-formed, de diagnosztika nem kötelező (ill-formed, no diagnostic required – IFNDR). Ez a legrosszabb hibafajta: a kód fordulhat az egyik fordítóval és elutasíthatja a másik, ezért nemtriviális `constexpr` függvényhez érdemes legalább egy `static_assert`-tel bizonyított konstans hívást is mellékelni.

33.1.3 consteval: az immediate function

A `consteval` függvény (immediate function) minden potenciálisan kiértékelt hívásának konstans kifejezésnek kell lennie; a hívás nem „eshet vissza” futásidejű végrehajtásra. Ez akkor jó, ha a szerződést nem lehet elég korán kideríteni: ha egy tábla érvényességét csak a build tudja ellenőrizni, a hívás helyén azonnal fordítási hiba keletkezik. A függvény címét is csak konstans kifejezésben lehet megszerezni, tehát futásidejű függvénypointerként nem használható.

Ennek ára van: a `consteval` *nem* alkalmas olyan API-ra, amelyet futásidőben is hívni akarunk. A táblánk keresője ezért `constexpr` marad, és csak az érvényesség-ellenőrző, illetve a kötelezően előre kiszámítandó belépőpontok `consteval`-ek.

33.1.4 constinit: a statikus inicializálási sorrend ellenszere

A statikus inicializálási sorrend problémája (static initialization order fiasco, SIOF) akkor jelentkezik, amikor egy fordítási egység globális objektumát egy másik fordítási egység globális objektuma inicializálja, és a sorrend nem specifikált. A `constinit` erre ad szűk, de pontos

választ: az inicializáló kifejezésnek konstans kifejezésnek kell lennie, tehát az objektum már a dinamikus inicializálás előtt, konstans inicializálással létrejön. Fontos: a `constinit` **nem** tesz `const`-tá, a változó továbbra is írható, és nem használható konstans kifejezésben. Ha mindkettő kell, `constexpr`-t írunk; ha csak az inicializálási sorrend biztonsága kell, `constinit`-et.

33.1.5 is_constant_evaluated és if consteval

Ugyanaz a `constexpr` függvény néha mást szeretne tenni konstans és futásidejű kiértékeléskor: konstans kiértékelésben nincs `memcmp`, `reinterpret_cast`, memcpy-szerű gyors út; futásidőben viszont pont ezek lennének hatékonyak. C++20-ban a `std::is_constant_evaluated()` (az `<type_traits>` fejlécből) ad erre választ: igazat ad, ha a hívás manifestly constant-evaluated kontextusban történik. C++23-ban a nyelvi `if consteval` szerkezet ezt olvashatóbban fejezi ki, és nem hagyja jóvá a gyakori hibát, amikor valaki `if constexpr`-ként próbálja használni.

```
#include <cstring>
#include <string_view>
#include <type_traits>

constexpr bool same_bytes(std::string_view a, std::string_view b) {
    if (a.size() != b.size()) { return false; }
    if (a.empty()) { return true; }
    if (std::is_constant_evaluated()) {          // C++20
        for (std::size_t i = 0; i < a.size(); ++i) {
            if (a[i] != b[i]) { return false; }
        }
        return true;
    }
    return std::memcmp(a.data(), b.data(), a.size()) == 0;  // csak futásidőben
}
```

C++23-as változatban a feltétel helyére `if consteval { ... } else { ... }` kerül. Mindkét esetben kötelező a `<cstring>` include, mert a `std::memcmp` deklarációja onnan jön, és a fordító csak akkor fordul le, ha a nem konstans ág tényleg nem része egyetlen konstans kiértékelésnek sem.

A verzióérzékenység itt tényleges: a nyelvi és library-támogatást a `__cpp_consteval` (201811L), `__cpp_constinit` (201907L), `__cpp_if_consteval` (202106L) és `__cpp_lib_is_constant_evaluated` (201811L) feature-test makrókkal kell ellenőrizni, nem a fordító verziószámából következtetni. A pontos makróértékek és a támogatás a konkrét compiler/libstdc++/libc++ kiadás függvényei, ezeket a saját laborprofilodban kell rögzíteni; ebben a fejezetben nem állítom, hogy bármelyik konkrét kiadás mit tud.

33.1.6 Fordítási idejű allokáció és a költség korlátai

C++20 óta konstans kiértékelésben a `new`/`delete` megengedett, de a lefoglalt tárterületnek a kiértékelés befejeződése előtt fel kell szabadulnia, és a konstans eredmény nem lehet mutató dinamikusan foglalt tárterületre. Ezért használhatunk `consteval` függvényben `std::vector`-t vagy `std::string`-et köztes számításhoz, de a kimenet csak érték (szám, `std::array`, rövid `string_view` literálra) lehet. A `malloc`/`free`, a `reinterpret_cast` és a placement new továbbra sem része a konstans kiértékelésnek. A library-támogatás (mennyire `constexpr` a `std::vector` adott művelete) implementációfüggő, ezt compile probe-bal kell ellenőrizni.

A fordítási idő önálló erőforrás: a konstans kiértékelésnek implementációs korlátai vannak (mélység, lépésszám, hurok-iteráció), és a túllépés diagnosztizált hiba, nem csendes visszaesés. A GCC és a Clang is kínál kapcsolókat ezek emelésére (GCC oldalon `-fconstexpr-depth`, `-fconstexpr-loop-limit`, `-fconstexpr-ops-limit`; Clang oldalon `-fconstexpr-depth`, `-fconstexpr-steps` néven), de a pontos nevek, alapértékek és szemantika kiadásonként változnak, ezért a laborban `--help`-fel kell ellenőrizni őket. Ne ezekkel „javítsd” a lassú buildet: a helyes válasz a konstans kiértékelés alatti munka csökkentése.

33.1.7 Kidolgozott példa: kulcsszótábla compile-time ellenőrzéssel

A példa egy konfigurációs parser lexikális táblája: rendezett kulcsszólista, bináris kereséssel. A tábla rendezettségét `consteval` függvény ellenőrzi `static_assert`-ben, a kereső `constexpr`, így futásidőben is hívható. A `g_last` `constinit`, mert globális, módosuló állapot, és nem akarunk SIOF-ot.

```
// classify.cpp -- ISO C++20
// Fordítás: g++ -std=c++20 -O2 -Wall -Wextra -pedantic -o classify classify.cpp
//        vagy: clang++ -std=c++20 -O2 -Wall -Wextra -pedantic -o classify
 ↪  classify.cpp
#include <array>
#include <cstddef>
#include <cstdio>
#include <string_view>

enum class token_kind { ident, number, terminator, unknown };

struct keyword {
    std::string_view spelling;
    token_kind kind;
};

namespace {

constexpr std::array<keyword, 3> kKeywords{{
    {"end",    token_kind::terminator},
    {"listen", token_kind::ident},
    {"port",   token_kind::number},
}};

// Kötelezettség: minden hívásnak konstans kifejezésnek kell lennie.
consteval bool strictly_sorted() {
    for (std::size_t i = 1; i < kKeywords.size(); ++i) {
        if (!(kKeywords[i - 1].spelling < kKeywords[i].spelling)) {
            return false;
        }
    }
    return true;
}

static_assert(strictly_sorted(), "a kulcsszotabla nem szigoruan rendezett");
```

```
// Lehetőség: konstans és futásidejű hívás is megengedett.
constexpr token_kind lookup(std::string_view s) {
    std::size_t lo = 0;
    std::size_t hi = kKeywords.size();
    while (lo < hi) {
        const std::size_t mid = lo + (hi - lo) / 2;
        const std::string_view key = kKeywords[mid].spelling;
        if (key == s) { return kKeywords[mid].kind; }
        if (key < s) { lo = mid + 1; } else { hi = mid; }
    }
    return token_kind::unknown;
}

consteval token_kind lookup_immediate(std::string_view s) { return lookup(s); }

constexpr const char* kind_name(token_kind k) {
    switch (k) {
    case token_kind::ident:      return "ident";
    case token_kind::number:     return "number";
    case token_kind::terminator: return "terminator";
    default:                     return "unknown";
    }
}

}  // namespace

// Statikus inicializálás, de futásidőben módosítható: nem constexpr, nem const.
constinit token_kind g_last = token_kind::unknown;

int main() {
    static_assert(lookup("port") == token_kind::number);

    constexpr token_kind ct = lookup_immediate("end");   // kötelezően CT
    static_assert(ct == token_kind::terminator);

    std::string_view runtime_key = "listen";             // futásidőben érkezne
    g_last = lookup(runtime_key);                        // ugyanaz a függvény,
     ↪ futásidőben
    std::printf("%s\n", kind_name(g_last));
    std::printf("%s\n", kind_name(ct));
}
```

A `lookup_immediate` itt demonstratív: futásidejű argumentummal

```
// SZÁNDÉKOSAN HIBÁS: consteval hívás futásidejű argumentummal
std::string_view key = read_key_from_config();
token_kind k = lookup_immediate(key);   // fordítási hiba: nem konstans
 ↪ kifejezés
```

fordítási hibát ad, mert a hívás nem konstans kifejezés. Ez a kívánt viselkedés: a szerződés a hívás helyén bukik el, nem a futás közben. Ugyanezt a hibát `lookup`-pal nem kapjuk meg – ott a hívás legálisan futásidejű.

Két dolgot nem állítok: hogy a fenti kód bármely konkrét fordítóval lefordult, és hogy a `lookup` futásidejű hívása mérhetően gyorsabb vagy lassabb egy kézzel írt táblánál. A példa nem tesztelt, a fordítás és a mérés a tanuló laborjának feladata; a várt viselkedés a fenti szerződésekből következik, nem mérési eredmény.

33.1.8 Gyakori hibák

Az első és legdrágább hiba a `constexpr` és a `consteval` felcserélése API-tervezéskor: `consteval`-léptetett kereső nem hívható futásidejű kulcsra, és a fordító ezt a hívás helyén, elsőre érthetetlen üzenettel jelzi. A második: `const`-ot használunk ott, ahol konstans kifejezés kell – a `const int n = f();` attól még nem lesz `constexpr`, és `static_assert(n == 3)` nem fordul le. A harmadik: `constexpr` függvény, amelyhez egyetlen konstans hívás sem létezik (IFNDR); ezt mindig kísérje `static_assert`. A negyedik: `constinit`-et `constexpr` helyett használni ott, ahol az értékre konstans kifejezésben is szükség van – a `constinit` változó nem használható `static_assert`-ben. Az ötödik: `is_constant_evaluated()`-t `if constexpr`-ként kezelni; ez futásidejű feltétel, csak a konstans kiértékelés tényéről dönt. A hatodik: `consteval` függvényből visszaadni dinamikusan foglalt tárolóra mutató pointert vagy `string`-t – a konstans eredmény nem hordozhat magával allokált tárterületet.

33.1.9 Feladatok

1. feladat (azonosítás). Az alábbi négy deklaráció közül melyik fordul le, és melyik nem? Indokold a szerződés alapján. a) `constexpr int a = 3 + 4;` b) `const int b = std::rand(); static_assert(b > 0);` c) `constinit int c = std::rand();` d) `consteval int f(int x) { return x * 2; } int main() { int y = std::rand(); return f(y); }`

2. feladat (kódolás). Egészítsd ki a fejezet `classify.cpp` példáját egy `consteval bool no_duplicates()` függvénnyel, amely szigorúan rendezett táblán ellenőrzi, hogy nincs két azonos `spelling`, és kösd be `static_assert`-be. Ezután szándékosan tegyél a táblába duplikátumot, és fogalmazd meg, milyen diagnosztikát várs.

3. feladat (diagnosztika). A `same_bytes` függvény egy változatából valaki kihagyta az `a.empty()` ellenőrzést, és `std::memcmp(a.data(), b.data(), 0)`-t hív üres `string_view`-kre. Magyarázd el, miért problematikus ez, és hogyan javítja a `std::is_constant_evaluated()`-sel védett ág a helyzetet. Add meg az ellenőrzési feltételt is.

33.1.10 Megoldások

1. megoldás. a) Le Fordul: `3 + 4` konstans kifejezés, az `int` literal típus. b) Nem: a `b` inicializálója futásidejű hívás, tehát `b` nem konstans kifejezés, és `static_assert`-ben nem használható – itt a hiba nem a `const`-ban, hanem abban van, hogy a *lehetőség* helyett *kötelezettséget* feltételeztünk. c) Nem: a `constinit` konstans inicializálást követel, a `std::rand()` nem konstans kifejezés; a `constinit` nem „varázsolja” konstanssá a futásidejű értéket. d) Nem: a `consteval` függvény hívása `y`-nal nem konstans kifejezés, ezért fordítási hiba; a helyes minta az, hogy a konstans hívást `constexpr` változóba tesszük, a futásidejű utat pedig `constexpr` (nem `consteval`) függvényen keresztül adjuk. Ellenőrzés: a négy esetből pontosan egy fordul le.

2. megoldás. A `no_duplicates` a rendezettséget kihasználja: szomszédos elemeket hasonlítunk, mert rendezett sorozatban a duplikátum szomszédos.

```
consteval bool no_duplicates() {
    for (std::size_t i = 1; i < kKeywords.size(); ++i) {
        if (!(kKeywords[i - 1].spelling < kKeywords[i].spelling)) {
            return false;   // azonos vagy nem rendezett
        }
    }
    return true;
}
static_assert(no_duplicates(), "duplikalt vagy nem rendezett kulcsszo");
```

Duplikátum beírásakor a várt diagnosztika a `static_assert` üzenete, benne a megadott szöveggel, és a hiba a `static_assert` sorára mutat – nem futásidejű viselkedés. Ellenőrzési feltétel: a duplikátumot tartalmazó tábla nem fordul le, a duplikátum eltávolítása után pedig a build hibamentes. Megjegyzés: a `strictly_sorted` és a `no_duplicates` itt ugyanazt a feltételt fejezi ki; a tanulság az, hogy a `consteval` + `static_assert` kombináció a tábla *invariánsát* kódolja, nem a kereső algoritmusát.

3. megoldás. A `std::memcmp` szerződése szerint a pointereknek legalább `n` karakterből álló tömbökre kell mutatniuk; üres `string_view` esetén a `data()` nulla hosszú, vagy akár null pointer is lehet, és a null pointer átadása `n == 0` mellett sem garantáltan megengedett. Ráadásul a konstans kiértékelésben a `std::memcmp` eleve nem hívható, mert nem `constexpr`. A javítás a korai `if (a.empty()) { return true; }` visszatérés, ami mindkét problémát megszünteti: a `memcmp` ág csak nem üres, futásidejű hívásnál fut le, a konstans ág pedig karakterenként hasonlít. Ellenőrzési feltétel: `same_bytes("", "")` konstans és futásidejű kontextusban is `true`, és a függvény `static_assert(same_bytes("port", "port"))`-tal fordítási időben is hívható.

33.1.11 Összefoglaló

A `const` hozzáférést véd, a `constexpr` változó konstans kifejezéssé tesz egy értéket, a `constexpr` függvény *lehetővé* teszi a fordítási idejű kiértékelést, a `consteval` *kötelezővé*, a `constinit` pedig a statikus inicializálás tényét garantálja anélkül, hogy konstans kifejezéssé tenné a változót. A termelékeny minta az, hogy a tábla- és konfigurációs invariánsokat `consteval` + `static_assert` zárja le a buildben, a kereső pedig `constexpr` marad, hogy futásidőben is használható legyen. A konstans kiértékelés nem ingyen van: implementációs korlátai vannak, allokációja csak átmeneti lehet, és a támogatás fordító- és library-verziófüggő, ezért feature-test makróval és compile probebal kell ellenőrizni. A modulok mint bináris határ, valamint a C++26-os reflection részletes tárgyalása a következő alfejezetekre tartozik; itt csak annyit rögzítettünk, hogy a konstans kiértékelés szerződése független attól, milyen fizikai formában adjuk át a fordítónak a forrást.

33.2 Compile-time adatok és típusos konfiguráció

A fix konfiguráció — protokoll-státuszkódok, metrika- és labelnevek, portok, feature flag-ek, syscall-táblák — szinte minden DevOps-komponensben megjelenik, és jellemzően kétszer ugyanaz a hiba forrása: vagy minden ilyen adatot futásidőben, `unordered_map`-ben és string-összehasonlítással keresünk (drága és későn derül ki a hiba), vagy a fordítóra bízzuk, és észre sem vesszük, hogy a fordítási idő és a bináris méret exponenciálisan nő. Ez az alfejezet a kettő közötti tervezést tárgyalja: mit érdemes fordítási időben kiszámítani és validálni, mit nem, és hogyan mérjük a költséget, mielőtt döntünk.

A `constexpr` kiértékelés nyelvi gépezetét részletesen a constexpr, consteval és constant evaluation rész tárgyalta; ott szerepel a constant expression pontos definíciója és a kiértékelés lépései. A

pack- és fold-technikát a variadic template részből feltételezzük. Itt a hangsúly a konfigurációs adat alakján, a validálás helyén és a költségvetésen van.

33.2.1 constexpr lookup és fix adat

Motiváció: egy diagnosztikai szolgáltatás negyvennél is több metrikanevet publikál, és minden minta rögzítésekor le kell képeznie a belső azonosítót a kimeneti névre. Ha ez `std::unordered_map<std::string, ...>` lookup, akkor hívásonként néhány száz nanoszekundum és egy dinamikus hash-számítás ára a minta-útban van; ha viszont a leképezés fix és rendezett, akkor fordítási időben generálható bináris keresés vagy `switch`, és a tábla a `.rodata` szegmensbe kerül.

Fontos terminológiai pontosítás: a `constexpr` függvény azt jelenti, hogy a függvény *képes* constant expression részeként kiértékelődni, nem azt, hogy a fordító ezt meg is teszi. Ha a hívás eredményét nem használjuk fel konstans kontextusban, a fordító jogosult — és tipikusan így is tesz — normál futásidejű függvényként lefordítani. A garantált fordítási idejű kiértékeléshez vagy `consteval` (immediate function) kell, vagy egy konstans kontextus, például `static_assert` vagy `constexpr` változó inicializálása. Feltétel mindkét esetben, hogy az argumentumok maguk is constant expressionök legyenek, és a visszaadott típus literal típus legyen.

Kidolgozott példa: fix HTTP-státusztábla és `constexpr` keresés.

```
// status.cc - C++20, GCC/Clang, -std=c++20
#include <array>
#include <cstdint>
#include <string_view>

namespace app {

struct StatusEntry {
    std::uint16_t code;
    std::string_view text;
};

inline constexpr std::array<StatusEntry, 4> kStatus{{
    {200, "OK"},
    {400, "Bad Request"},
    {404, "Not Found"},
    {500, "Internal Server Error"},
}};

constexpr std::string_view status_text(std::uint16_t code) {
    for (const auto& e : kStatus) {
        if (e.code == code) return e.text;
    }
    return "unknown";
}

} // namespace app

static_assert(app::status_text(404) == "Not Found");
static_assert(app::status_text(418) == "unknown");
```

A `static_assert` azért kell, mert enélkül a `status_text` csak *lehetne* fordítási idejű — a `static_assert` kényszeríti ki, hogy tényleg az legyen. A `inline constexpr` (C++17) biztosítja, hogy a táblának egyetlen definíciója legyen a programban; ha a headerben `static constexpr` vagy névtelen névtér szerepelne, fordításonként külön példány keletkezne, amit a linker sok esetben összevon, de ez nem garantált, és a `.rodata` mérete emiatt TU-számmal nőhet.

33.2.2 A fordítási költség mint erőforrás

A fordítási idő ugyanolyan erőforrás, mint a futásidejű memória: mérni kell, és költségvetést kell rá adni. Két beépített eszköz terjedt el: a GCC `-ftime-report` kapcsolója, amely szöveges bontást ad (köztük a konstans-kiértékelés idejét is), és a Clang `-ftime-trace` kapcsolója, amely Chrome-trace formátumú JSON-t ír, és template-instanciálásonként külön blokkokat tartalmaz. Mindkettő implementációfüggő: a pontos szakasznevek és a kimenet felbontása a fordító kiadásától függ, ezért a labornaplóban rögzíteni kell a konkrét verziót.

A mérés alapkövetelménye a fix mérési terv, nem az egyszeri benyomás. Az alábbi tábla a mérési terv váza; a cellákba csak a saját gépen, dokumentált kapcsolókkal előállított számok kerülhetnek be.

Dimenzió	Hol jelenik meg	Mit mérünk	Mérési eljárás
Konstans kiértékelés	`-ftime-report` konstans-kiértékelési sora	fordítási másodperc	azonos TU, 10 / 100 / 1000 elemű tábla
Template-instanciálás	`-ftime-trace` blokkok	instanciálások száma	`std::variant` / `std::tuple` méret növelése
Bináris méret	`size`, `nm -S`	`.rodata` és `.text` bájt	1 TU vs. 10 TU, `inline constexpr` vs. `static`
Inkrementális build	ccache/sccache statisztika, `ninja -d explain`	cache-találati arány	egy header módosítása, egy .cc módosítása

Ehhez tartozik egy kvalitatív, de határozott táblázat arról, mi hogyan skálázódik:

Tényező	Növekedés	Következmény
Táblaméret (N elem)	lineáris konstans-kiértékelési idő	N > néhány ezer esetén a buildidő érezhetően nő
Headerben lévő `constexpr` adat	minden include-oló TU újra feldolgozza	a teljes újrafordítás drágul, a cache könnyen invalidálódik
Pack-expansion mélység	kombinatorikus instanciálás	a `-ftime-trace` blokkok száma robbanásszerűen nő
`inline constexpr` tábla	egy definíció	kisebb `.rodata`, de minden TU-nak látnia kell a fejlécet

Fontos következmény a cache-elt buildre: ha egy gyakran include-olt fejlécben méretes `constexpr` táblát tartunk, akkor a fejléc bármilyen apró módosítása az összes rá épülő TU-t újrafordítja, és a gyorsítótár nem segít. Ilyenkor érdemes a táblát egy külön, ritkán változó fejlécbe tenni, vagy `extern const` adatként egy `.cc`-be helyezni, és csak a kereső `constexpr` függvényt tartani a fejlécben.

33.2.3 User-defined literal és compile-time validálás

A fordítási idejű validálás akkor éri meg, ha az érvénytelen érték így nem futásidejű hibaágat, hanem fordítási hibát okoz. A user-defined literal (`UDL`) erre jó eszköz: a literál helyén a fordító a megadott operátort hívja, és ha az operátor konstans kontextusban értékelődik ki, a hiba a forráskód sorára mutat.

```
// port.cpp - C++20
#include <cstddef>
#include <cstdint>
#include <stdexcept>
#include <string_view>

namespace app {

constexpr std::uint16_t parse_port(std::string_view s) {
    if (s.empty() || s.size() > 5) throw std::invalid_argument("port length");
    std::uint32_t v = 0;
    for (char c : s) {
        if (c < '0' || c > '9') throw std::invalid_argument("port: non-digit");
        v = v * 10u + static_cast<std::uint32_t>(c - '0');
        if (v > 65535u) throw std::invalid_argument("port: out of range");
    }
    return static_cast<std::uint16_t>(v);
}

} // namespace app

// Numerikus literálhoz: 8080_port. A literál értéke unsigned long long.
constexpr std::uint16_t operator""_port(unsigned long long v) {
    if (v > 65535u) throw std::invalid_argument("port: out of range");
    return static_cast<std::uint16_t>(v);
}

// String literálhoz: "8080"_port, ugyanazzal a validálással.
constexpr std::uint16_t operator""_port(const char* s, std::size_t n) {
    return app::parse_port(std::string_view{s, n});
}

static_assert(8080_port == 8080u);
static_assert("8080"_port == 8080u);
static_assert(app::parse_port("65535") == 65535u);
// static_assert(99999_port == 0u);    // szándékosan fordítási hiba
// static_assert("99999"_port == 0u);  // szándékosan fordítási hiba

int main() {}
```

A literáloperátor szignatúrája határozza meg, milyen literállal hívható (C++20 [over.literal], [lex.ext]). A `(const char*, std::size_t)` paraméterlista a **string** literálé (`"8080"_port`). Egy **numerikus** literál (`8080_port`) az `unsigned long long` paraméterű operátort keresi, vagy a nyers `(const char*)` alakot, illetve a literáloperátor-sablont. Ha csak a string változat létezik,

a `8080_port` fordítási hiba: `unable to find numeric literal operator 'operator""_port'`. A fenti kód ezért mindkét alakot megadja, azonos validálással.

Buildjegyzőkönyv (`lab-gcc` profil). A program `g++ -std=c++20 -Wall -Wextra -Wpedantic` beállítással figyelmeztetés nélkül fordult, mindhárom `static_assert` teljesül. A két kikommentezett sor bármelyikét visszakapcsolva a fordítás `non-constant condition for static assertion` hibával leáll, a `constexpr` kiértékelés `operator""_port(99999)`, illetve `operator""_port("99999", 5)` pontján. A csak string literálos operátort tartalmazó eredeti változat a `8080_port` sorra a fenti `unable to find numeric literal operator` hibát adta.

A minta lényege a két réteg: `parse_port constexpr`, tehát futásidőben is hívható; a `_port` operátor szintén `constexpr`, és a `static_assert` kényszeríti ki a konstans kiértékelést. A `throw` constexpr függvényben megengedett, de ha konstans kiértékelés során ténylegesen kiértékelődik, az nem constant expression, ezért fordítási hibát ad. A hibaüzenet szövege azonban nem szabályozott, csak a hiba ténye.

Korlátok, amelyeket a tervezésnél figyelembe kell venni. Négy különálló szabály játszik szerepet:

- **Foglaltság:** az aláhúzás nélküli suffixek a szabványnak vannak fenntartva; a saját UDL neve aláhúzással kezdődik (GCC: `literal operator suffixes not preceded by '_' are reserved`).
- **Overload:** a literáloperátor közönséges függvény, ugyanazzal a suffixszel több paraméterlistára is megadható (lásd fent a numerikus és a string alakot).
- **Névtér:** a literáloperátor névtérbe tehető, és csak ott látszik, ahová `using`-gal behozzuk; a bevett minta a `…::literals` alnévtér. Két névtér ugyanazt a `_port` nevet önállóan, ütközés nélkül deklarálhatja.
- **ODR és névfeloldás:** konfliktus csak akkor van, ha ugyanabban a hatókörben két azonos szignatúrájú definíció keletkezik (ODR-sértés, lásd az ODR és inline rész), vagy ha egy konkrét literál feloldásakor egyszerre több egyformán jó jelölt látszik — ez kétértelműség, fordítási hiba a használat helyén.

```
// udl_namespaces.cpp - C++20: ugyanaz a _port suffix két névtérben
#include <cstdint>

namespace net::literals {
    constexpr std::uint16_t operator""_port(unsigned long long v) {
        return static_cast<std::uint16_t>(v);          // TCP/UDP port
    }
}
namespace serial::literals {
    constexpr int operator""_port(unsigned long long v) {
        return static_cast<int>(v);                    // soros port sorszáma
    }
}

constexpr auto tcp() { using namespace net::literals;    return 8080_port; }
constexpr auto tty() { using namespace serial::literals; return 2_port; }

static_assert(tcp() == 8080 && tty() == 2);

#ifdef AMBIGUOUS
using namespace net::literals;
using namespace serial::literals;
auto both = 1_port;                                    // itt kétértelmű
```

```
#endif

int main() {}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic udl_namespaces.cpp` figyelmeztetés nélkül fordul: a két névtér deklarációja önmagában rendben van, és mindkettő külön használható. `-DAMBIGUOUS` mellett, ahol mindkét névtér egyszerre látszik, a `1_port` sor `call of overloaded 'operator""_port(long long unsigned int)' is ambiguous` hibát ad. Nagy literáloknál a konstans kiértékelés mérete a fordítási időben jelenik meg, nem a futásidőben.

33.2.4 static_assert és hibadiagnosztika

A `static_assert(feltétel, üzenet)` deklaráció: a feltételnek constant expressionnek kell lennie, hamis érték esetén a program fordítása hibával leáll. C++17 óta az üzenet elhagyható, C++20 ezt megtartja. Elhelyezhető névtérben, osztályban és függvényen belül is.

A diagnosztika minősége gyakorlati kérdés. A fordító az üzenetbe beemeli a feltétel forrásszövegét, de ha a feltétel egy nagy tábla egy elemére hivatkozik, a hasznos információ elvész. Bevett — bár implementációfüggő — trükk, hogy a hibás értéket template-argumentumba csomagoljuk, mert a template-argumentum megjelenik a hibaüzenetben. Ehhez egy szándékosan hiányos sablon deklaráció kell:

```
template <auto V>
struct show_value;   // nincs definíció: használata "incomplete type" hibát ad

// static_assert(app::status_text(404) == "Not Found");
// static_assert(app::status_text(404) == "OK", "status text mismatch");
```

Ez a technika nem hivatalos language feature, és a kimenet formátuma fordítónként változik; csak diagnosztikai segédeszközként használjuk, és a végleges kódban ne hagyjuk benne. Ha több feltételt ellenőrzünk, mindegyikhez írjunk külön üzenetet, mert a fordító az első hamis feltételnél megáll.

33.2.5 if constexpr versus normál if

Az `if constexpr` (C++17) és a normál `if` közötti különbség nem teljesítmény, hanem fordíthatóság. Egy template-en belül az `if constexpr` feltétele constant expression, és a nem vett ágat a fordító nem instanciálja. A normál `if` mindkét ágát lefordítja, akkor is, ha futásidőben soha nem fut le.

```
// describe.cc - C++20
#include <string>
#include <type_traits>

template <class T>
std::string describe(const T& v) {
    if constexpr (std::is_integral_v<T>) {
        return "int:" + std::to_string(v);
    } else {
        return "other";
```

```
    }
}
```

Ha itt normál `if` szerepelne, a `T = std::string` esetben a `std::to_string(v)` ág is instanciálódna, és mivel arra nincs illeszkedő overload, a program nem fordulna le — noha az ág soha nem futna. Ezt a különbséget könnyű összekeverni a `#if`-fel: a preprocesszor a nem vett ágat teljesen eltávolítja, a fordító nem is látja; az `if constexpr` a template-instanciálás szintjén zárja ki. Fontos korlát: template-en kívül az `if constexpr` mindkét ágát ellenőrzi a fordító, mert a feltétel nem függ template-paramétertől, így ott a haszna kicsi.

33.2.6 Nem minden runtime input fordítható le

A leggyakoribb tervezési hiba, hogy a konfigurációs fájl vagy a környezeti változó értékét próbáljuk `constexpr`-ré tenni. Ez fogalmilag lehetetlen: a constant expression követelménye, hogy az érték a fordítás pillanatában ismert legyen, a konfigurációs fájl viszont csak a telepítés után létezik.

A helyes minta a közös `constexpr` mag és két belépési pont. A mag a `parse_port` fenti változata; mivel egy `constexpr` függvényt minden fordítási egységnek látnia kell, amely konstans kiértékelésben hívja, a magot egy fejlécbe emeljük. A konstans literálokhoz `consteval` burkoló, a dinamikus bemenethez közvetlen futásidejű hívás és `std::optional`-lal jelzett hibaág tartozik.

```
// port.h - a közös constexpr mag (C++20)
#ifndef PORT_H
#define PORT_H
#include <cstdint>
#include <stdexcept>
#include <string_view>

namespace app {

constexpr std::uint16_t parse_port(std::string_view s) {
    if (s.empty() || s.size() > 5) throw std::invalid_argument("port length");
    std::uint32_t v = 0;
    for (char c : s) {
        if (c < '0' || c > '9') throw std::invalid_argument("port: non-digit");
        v = v * 10u + static_cast<std::uint32_t>(c - '0');
        if (v > 65535u) throw std::invalid_argument("port: out of range");
    }
    return static_cast<std::uint16_t>(v);
}

} // namespace app
#endif
```

```
// dual_path.cc - C++20
#include <optional>
#include <string>
#include <string_view>
#include "port.h"   // app::parse_port

consteval std::uint16_t port_ct(std::string_view s) {
    return app::parse_port(s);   // hiba esetén fordítási hiba
}
```

```
std::optional<std::uint16_t> port_runtime(std::string_view s) {
    try {
        return app::parse_port(s);
    } catch (...) {
        return std::nullopt;
    }
}
```

Így ugyanaz a validációs logika szolgálja a fordítási idejű konstansokat és a futásidejű bemenetet, és nincs két, eltérően viselkedő ellenőrző implementáció. A fordítási költség csak ott jelenik meg, ahol a konstans út tényleg használatban van.

33.2.7 Gyakori hibák

- Azt hinni, hogy a `constexpr` függvény automatikusan fordítási időben fut. Csak konstans kontextusban fut, egyébként normál hívás.
- `consteval` függvényt futásidejű adaton hívni: ez fordítási hiba, nem csendes visszaesés.
- Nagy `constexpr` táblát gyakran include-olt fejlécbe tenni, ezzel minden TU-t és minden cache-bejegyzést érinteni.
- A `static_assert` üzenetébe nem tenni bele a tényleges értéket, így a hibaüzenet hasznavehetetlen.
- `if constexpr` helyett normál `if`-et használni olyan ággal, amely csak a típusok egy részhalmazára fordul le.
- Megpróbálni a konfigurációs fájl tartalmát fordítási idejűvé tenni, és emiatt kettős validációt írni.

33.2.8 Feladatok

1. **Alap.** Írj `constexpr` táblát öt logaritmikus szintre (`trace`, `debug`, `info`, `warn`, `error`), és egy `consteval`-lal kényszerített keresőt, amely számból adja vissza a nevet. Ellenőrizd két `static_assert`-tel. Készíts hozzá `g++ -std=c++20 -Wall -Wextra` és `clang++ -std=c++20 -Wall -Wextra` buildparancsot.

2. **Közép.** Egészítsd ki a `_port` literált két új érvényességi feltétellel: a `0` érték tiltott, és az `1..1023` tartomány csak akkor legyen elfogadott, ha a felhasználó kifejezetten kéri (például egy `_port_priv` suffix-szel). Írd le, mit tapasztalsz, amikor szándékosan hibás literált írsz be: melyik sorra mutat a hiba, és mit tartalmaz az üzenet.

3. **Nehéz.** Tervezz mérést a fordítási költségre. Készíts két változatot: (a) 1000 elemű `constexpr` tábla `inline constexpr`-rel, (b) ugyanaz az adat futásidőben épített `std::vector`-ból. Mérd mindkettőt `-ftime-report`-tal (GCC) vagy `-ftime-trace`-szel (Clang) azonos gépen és azonos compiler-kiadással, és dokumentáld a számokat. Ezután készíts egy 1 és egy 10 TU-s változatot, és hasonlítsd össze a `size a.out` kimenetét. Készíts egy táblázatot az eredményekről, és indokold, mit érdemes konstansként tartani.

33.2.9 Megoldások

1. **Megoldás.** A tábla `inline constexpr std::array<std::pair<std::string_view,int>, 5>`, a kereső `consteval int level_of(std::string_view)` visszaadott értékkel; a tábla és a kereső ugyanabban a fejlécben, a `consteval` wrapperrel. Indoklás: a `consteval`

garantálja, hogy a hívás konstans kontextusú, a `static_assert` pedig dokumentálja a szándékot. Ellenőrzési feltétel: a `static_assert(level_of("warn") == 3)` és a `static_assert(level_of("nope") == -1)` forduljon le, és a szándékosan hibás `static_assert` mutasson a megfelelő sorra mindkét compilerrel. Ha a Clang és a GCC eltérő üzenetet ad, az nem hiba: a diagnosztika formátuma nem szabályozott.

2. **Megoldás.** A `parse_port` mellé egy `parse_port(range_policy)` overload kerül, ahol a `range_policy` egy `enum class` érték; a `_port` a publikusat, a `_port_priv` a megengedőt hívja. Indoklás: a szabály a parsertől független, ezért nem a parser belsejébe, hanem a wrapperekbe kerül. Ellenőrzési feltétel: `0_port` és `0_port_priv` egyaránt fordítási hibát adjon, `8080_port` és `08080_port` forduljon le, `80_port` viszont csak a `_priv` változattal legyen érvényes. A hibadiagnosztikánál jegyezd fel, hogy az üzenet az operátor hívásának helyére mutat, nem a `throw` sorára.

3. **Megoldás.** A mérési terv helyes felépítése: azonos TU-szerkezet, azonos include-lista, azonos compiler-kiadás, legalább három futás, és a medián rögzítése. A két változat közötti különbséget csak akkor szabad a `constexpr`-nek tulajdonítani, ha a futásidejű változatban is ugyanannyi elemet dolgozunk fel. Az eredmény tipikus értelmezése: kis táblánál (10–100 elem) a fordító sokszor nem is különböztethető mérhetően, nagy táblánál viszont a konstans kiértékelés ideje a `-ftime-report` konstans-kiértékelési sorában jelenik meg, és a `.rodata` mérete nő. A 10 TU-s kísérletnél a `static constexpr` tábla nagyobb binárist adhat, az `inline constexpr` kisebbet, de ezt a linker és az optimalizálás befolyásolja, ezért mért érték nélkül nem szabad kijelenteni. Ha a mért számok nem támasztják alá a várt előnyt, a helyes döntés a futásidejű változat megtartása — a mérés célja éppen az, hogy ezt a döntést ne érzésre hozzuk.

33.2.10 Összefoglaló

A fix konfigurációs adatot érdemes fordítási időben kiszámítani és validálni, de csak akkor, ha az adat tényleg fix, és ha a mérés igazolja a nyereséget. A `constexpr` lehetőséget ad, a `consteval` és a `static_assert` kényszeríti ki a fordítási idejű kiértékelést; a `_port`-szerű user-defined literal a validálást a forrásszöveg helyére viszi. Az `if constexpr` a template-instanciálás szintjén zárja ki a nem vett ágat, ezért ott kell használni, ahol az ág csak bizonyos típusokra fordul le. A futásidejű bemenetre a közös `constexpr` mag és a futásidejű belépő a helyes minta, nem a konstans kényszerítése. A fordítási költséget a `-ftime-report` / `-ftime-trace`, a bináris méretet a `size` és `nm -S`, a cache-viselkedést a ccache/sccache statisztika mutatja; a döntést ezekre a mérésekre kell alapozni, nem a `constexpr` puszta jelenlétére.

33.3 ODR, inline és több fordítási egység

A több fordítási egységből álló C++ program egyik legfontosabb, mégis legkönnyebben megsérthető szerződése a One Definition Rule (ODR). A C-ben a linker szintjén a külső definíciók egyedisége a fő szabály; a C++ ezt kiterjeszti a classokra, template-ekre, inline entitásokra és bizonyos compile-time konstrukciókra. A C++20 `[basic.def.odr]` szabálya szerint egy entitásnak a programban pontosan egy definíciója lehet, kivéve azokat az eseteket, amelyeket a szabvány kifejezetten megenged. Az ODR megsértése gyakran nem diagnosztizált: a fordító csak egy fordítási egységet lát, a linker pedig jellemzően nem ismeri a forrásszintű szemantikát. Ezt a helyzetet nevezi a szabvány „ill-formed, no diagnostic required” (IFNDR) állapotnak. A fejezet szabványhivatkozásai tájékoztató jellegűek; a konkrét bekezdésszámok és a compiler viselkedése a laborprofilban ellenőrizendő.

DevOps szempontból az ODR nem elméleti kuriózum. A header-only könyvtárak, a template-ek, az inline változók és a különböző TU-kra eltérően definiált makrók mind tipikus build- és üzemeltetési hibákhoz vezetnek. A hiba sokszor nem a fordításkor, hanem ritka futásidejű memóriahibaként vagy ABI-eltérésként jelenik meg. Ezért a build rendszernek gondoskodnia kell arról, hogy a header-definíciók minden TU-ban azonosak legyenek, és a konfiguráció ne változtassa meg észrevétlenül a classok layoutját.

33.3.1 Az ODR és az odr-use alapja

Az ODR három külön követelményt fogalmaz meg. (1) **Egy TU-n belül** egy entitásnak legfeljebb egy definíciója lehet. (2) **Definíciós igény:** egy nem inline függvénynek vagy változónak akkor kell definíciója a programban, ha *odr-used*; egy deklarált, de soha nem használt függvényhez nem kell definíció. (3) **Több TU:** a programban egy külső kapcsolású, nem inline entitásnak legfeljebb egy definíciója lehet, kivéve az inline függvényeket, inline változókat, classokat, enumokat, template-eket és néhány további esetet. Fontos, hogy a belső kapcsolású nevek (`static` függvény, névtelen névtérbeli entitás) TU-nként *külön* entitások: két TU azonos nevű `static` függvénye nem ugyanannak az entitásnak két definíciója, ezért nem ODR-sértés. Ezeknél több TU-ban is szerepelhet definíció, de a definícióknak tokenre azonosnak kell lenniük, és a bennük szereplő neveknek ugyanarra az entitásra kell feloldódniuk. A tokenazonosság a preprocesszor kiterjesztése után értendő, tehát a makrók és a feltételes fordítás is része a szerződésnek.

Buildjegyzőkönyv (`lab-gcc` profil). Két TU, mindkettőben `static int helper()` eltérő törzzsel, és egy soha nem használt `int unused_decl(int);` deklaráció: a program lefordult, linkelt, és `1 2`-t írt ki. Ha mindkét TU-ba egy külső kapcsolású, nem inline `int twice(int)` definíció kerül, a linker `multiple definition of 'twice(int)'` hibát ad — ez a valódi ODR-sértés, és itt történetesen diagnosztizált (inline entitások eltérő definícióinál általában nem az).

Az odr-use azt jelöli, hogy egy entitás definíciójára a program szemantikájához ténylegesen szükség van. Egy függvény például akkor odr-used, ha meghívják vagy a címét veszik. Egy változó akkor, ha a címét veszik, referenciát kötnek hozzá, vagy olyan kifejezésben szerepel, amelyhez a definíció kell. Nem minden névhasználat odr-use: egy konstans kifejezésben értékként használt `const` változó bizonyos feltételek mellett nem igényel definíciót. Az inline függvényt viszont minden TU-ban definiálni kell, ahol odr-used; ha a definíció csak egy `.cpp` fájlban van, a többi TU számára nem elérhető, és a program viselkedése nem lesz helyes.

A C-hez viszonyítva a különbség lényeges. A C17 `6.9` szakasza a külső definíciók egyediségét szabályozza, de a C-nek nincs a C++éhoz hasonló, classokra és template-ekre kiterjedő ODR-je. A C `inline` függvény C99-es szemantikája külön `inline` és külső definíciót ismer; a C++ `inline` egyszerűbben garantálja, hogy a függvény több TU-ban definiálható, és a programban egy példánya lesz. A C-ben nincs inline változó; a headerben használt `static` változó minden TU-ban külön példány. A C++17 óta az `inline` változó ezt a problémát oldja meg.

33.3.2 Inline függvény és inline változó: a tokenazonosság szerződése

Az `inline` kulcsszó nem „optimalizálási kérés” a modern C++-ban, hanem ODR-engedély: a függvény vagy változó több TU-ban is definiálható, feltéve, hogy a definíciók tokenre azonosak, és minden TU-ban elérhetők, ahol odr-used. A classon belül definiált tagfüggvény implicit inline; a `constexpr` és a `consteval` függvény szintén implicit inline. A `constexpr` static adattag C++17 óta implicit inline, de a névtér-scope-ban deklarált `constexpr` változó nem az: headerben `inline constexpr` kell ahhoz, hogy egy példánya legyen.

A következő példa ezt mutatja be. A `stats.hpp` inline függvényt, inline változót és template-et tartalmaz; a `stats.cpp` egy explicit instantiationt ad; a `a.cpp` és `b.cpp` külön TU-kból használja a közös definíciókat.

```
// stats.hpp
#ifndef STATS_HPP
#define STATS_HPP

inline int call_count = 0;

inline int bump(int x) { return x + 1; }

template<class T>
T twice(T x) { return x + x; }

extern template int twice<int>(int);

#endif
```

```
// stats.cpp
#include "stats.hpp"
template int twice<int>(int);
```

```
// a.cpp
#include "stats.hpp"
int a_use() {
    ++call_count;
    return bump(1);
}
```

```
// b.cpp
#include "stats.hpp"
int b_use() {
    ++call_count;
    return twice(2);
}
```

```
// main.cpp
#include <iostream>
#include "stats.hpp"

int a_use();
int b_use();

int main() {
    const int r = a_use() + b_use();
    std::cout << r << ' ' << call_count << '\n';
}
```

A laborprofilban rögzített GCC vagy Clang esetén a buildparancs a következő lehet; a `c++` helyére a rögzített compilert kell írni, és a nyelvi módot mindig explicit meg kell adni:

```
c++ -std=c++20 -O2 -Wall -Wextra -c stats.cpp -o stats.o
c++ -std=c++20 -O2 -Wall -Wextra -c a.cpp -o a.o
```

```
c++ -std=c++20 -O2 -Wall -Wextra -c b.cpp -o b.o
c++ -std=c++20 -O2 -Wall -Wextra -c main.cpp -o main.o
c++ stats.o a.o b.o main.o -o app
./app
```

A várt kimenet `6 2`: a `bump(1)` értéke 2, a `twice(2)` értéke 4, a `call_count` pedig kétszer növekszik. A fejezet nem állítja, hogy a példa lefutott; a tényleges futtatás a labor feladata. Ha a `call_count` deklarációja `static int call_count = 0;` lenne, a `main` a saját példányát látná, a `a_use` és a `b_use` pedig a sajátjukat növelné, így a kimenet `6 0` lenne. Ez nem ODR-violation, mert a belső linkage entitás TU-nként különálló, de a szemantika megváltozik. Ha a `stats.cpp` kimarad a linkelésből, a `twice<int>` explicit instantiationja hiányzik, és a linkelés hibával leáll. Ha az `extern template` deklaráció marad el, a `b.cpp` is instantálja a `twice<int>`-et; az ODR ezt megengedi, a linker összevonja, de a fordítási idő nő.

33.3.3 Template-definíciók több fordítási egységben

A template definíciójának a headerben kell lennie, mert a fordítónak a használat helyén látnia kell a sablon testét az implicit instantiationhöz. Az ODR megengedi, hogy ugyanaz a specializáció több TU-ban is létrejöjjön, de a definícióknak tokenre azonosnak kell lenniük, és a template argumentumoknak ugyanarra a típusra kell vezetniük. Az explicit instantiation (`template int twice<int>(int);`) egy TU-ban generálja a kódot; az `extern template` a headerben megakadályozza, hogy a többi TU implicit módon instantálja ugyanazt a specializációt. Ez csökkenti a kódduplikációt és a fordítási időt, de csak akkor működik, ha valahol tényleg van explicit instantiation.

Az explicit specializáció deklarációjának minden TU-ban elérhetőnek kell lennie, ahol a specializációt használják. Ha ez elmarad, a fordító a primary template-et használhatja, miközben egy másik TU a specializációt látja; ez ODR-violation, és a program viselkedése nem definiált. Az ilyen hibák tipikusan nem fordítási hibaként, hanem eltérő futásidejű viselkedésként jelennek meg. A template-ek és az explicit instantiation részleteit a Specialization, overload és explicit instantiation tárgyalta; itt az ODR-következmények a lényegesek.

33.3.4 Rejtett ODR-violation: makrófüggő class layout

A leggyakoribb rejtett ODR-violation nem a függvénydefiníciókban, hanem a classok layoutjában keletkezik. Ha a class definíciója `#ifdef`-fel változik, a preprocesszor TU-nként más tokeneket ad, és a classok layoutja eltér. A fordító ezt nem látja, mert csak a saját TU-ját ismeri; a linker pedig jellemzően nem ellenőrzi a class méretét. A következő példa ezt szemlélteti.

```
// bad_config.hpp
#ifndef BAD_CONFIG_HPP
#define BAD_CONFIG_HPP

struct Config {
#ifdef ENABLE_CACHE
    int cache_size;
#endif
    int timeout;
};

#endif
```

```cpp
// a.cpp
#define ENABLE_CACHE
#include "bad_config.hpp"
static_assert(sizeof(Config) == 8, "a.cpp layout");
int a_size() { return sizeof(Config); }

// b.cpp
#include "bad_config.hpp"
static_assert(sizeof(Config) == 4, "b.cpp layout");
int b_size() { return sizeof(Config); }
```

Egy x86-64 System V ABI, 4 bájtos `int` és szokásos igazítás melletti illusztráció:

Fordítási egység	`ENABLE_CACHE`	`sizeof(Config)`	`offsetof(Config, timeout)`
`a.cpp`	definiálva	8	4
`b.cpp`	nincs	4	0

A táblázat feltételezi a 4 bájtos `int`-et; a pontos értéket a target ABI és a fordítási profil rögzíti. A két `static_assert` külön-külön igaz, mert mindegyik TU a saját definícióját látja. Ha a `main` átad egy `Config`-ot `a.cpp` és `b.cpp` között, az egyik oldal 8, a másik 4 bájtos objektumot feltételez; ez memóriahibához és nem definiált viselkedéshez vezet. A hiba nem feltétlenül jelenik meg linkeléskor, mert a szimbólumok nevei azonosak.

A megoldás az, hogy a layoutot nem szabad makrótól függővé tenni. A konfigurációt a build rendszerben egy helyen kell rögzíteni minden TU-ra, a class layoutját pedig fixen kell tartani; a funkció külön típusba vagy futásidejű flagbe kerüljön. A típusos compile-time konfiguráció lehetőségeit a Compile-time adatok és típusos konfiguráció tárgyalta. Ha a `cache_enabled` értéket `inline constexpr bool`-ként headerbe tesszük, és a makró TU-nként eltér, az `if constexpr`-t használó inline függvények definíciója is tokenre eltérhet, ami szintén ODR-violation. Ezért a feature flagnek minden TU-ban ugyanazt az értéket kell látnia, vagy a különbséget típusparaméterrel kell kifejezni.

33.3.5 Internal linkage és anonymous namespace

A `static` függvény, a `static` változó és az anonymous namespace belső linkage-et ad. Az ilyen entitás csak a saját TU-jában definiálható, ezért nem sérti meg az ODR-t. A headerben mégis problémás lehet: minden TU saját példányt kap, a függvény címe TU-nként eltér, a benne lévő `static` állapot pedig külön életet él. Ha a függvény címét template argumentumként vagy callbackként használjuk, a különböző címek eltérő viselkedést okozhatnak. A headerben `static` függvény emellett kóddagadást eredményez, mert minden TU külön másolatot fordít.

A C++ válasza a headerben definiált közös entitásra az `inline`: az inline függvény és az inline változó egy példányt ad a programban. A C-ben a headeres `static inline` továbbra is minden TU-ban külön példányt ad; ez a C és a C++ inline szemantikájának egyik fontos különbsége. A namespace és a névfeloldás alapjait a Namespace, header és névfeloldás tárgyalta; itt a linkage és az ODR kapcsolata a lényeg.

33.3.6 LTO és diagnosztika: korlátozott bizonyíték

A link time optimization (LTO) a linkeléskor az összes TU köztes reprezentációját látja, ezért képes lehet olyan ODR-eltéréseket észlelni, amelyeket a külön fordított TU-k nem. A GCC `-Wodr` kapcsolója LTO-val használható; más fordítóknál a támogatás verziófüggő. A figyelmeztetés azonban nem teljes körű: az ODR-violation a szabvány szerint IFNDR, a fordító nem köteles diagnosztizálni. A `-Wodr` hiánya nem bizonyítja, hogy a program ODR-konform. A végső bizonyíték a forrás- és build-szintű azonosság: ugyanazok a header-definíciók, ugyanazok a makrók, ugyanazok a fordítási opciók minden TU-ra.

A fordítási idő önálló erőforrás. Az inline és template kódduplikáció növeli a fordítandó kód mennyiségét; az `extern template` és a nem template-es API csökkentheti. Az LTO nem csökkenti a fordítási időt, sőt jellemzően növeli. A mérés a labor feladata: GCC-nél `-ftime-report`, Clangnál `-ftime-trace`, a headerfüggőségekhez `-H`, a makrók ellenőrzéséhez `-E -dM`. Ezek a kapcsolók a rögzített compiler verziójától függően elérhetők; a fejezet nem állítja, hogy bármelyiket lefuttatta. A feature-test makrók és a standard library capability vizsgálata a következő alfejezetre tartozik; a C++20 modules és a header unitok szintén ott kerülnek sorra. A modulokat mint stabil cross-compiler bináris formátumot itt nem tárgyaljuk.

33.3.7 Tipikus hibák és javításuk

A leggyakoribb hiba a headerben használt `static` függvény vagy anonymous namespace, amikor a cél egy közös definíció lenne; ilyenkor `inline` a helyes válasz. A második az `inline` függvény definíciójának `.cpp` fájlba helyezése: az inline függvényt minden TU-ban definiálni kell, ahol odr-used, különben a program nem lesz helyes. A harmadik az `extern template` használata explicit instantiation nélkül, ami linkelési hibát okoz. A negyedik a `#ifdef`-fel változó class layout, amely a `static_assert` és a linker ellenére is rejtett ODR-violation marad. Az ötödik a TU-nként eltérő `-D` flag, amely ugyanazt a headert különböző definícióvá alakítja. Hatodikként említhető a `constexpr` változó `inline` nélküli használata headerben C++17 előtt, illetve a `constexpr` static adattag és a `constexpr` függvény implicit inline voltának összekeverése. Végül tipikus hiba, hogy valaki a LTO `-Wodr` figyelmeztetésének hiányából a program ODR-helyességére következtet.

33.3.8 Összefoglaló

Az ODR a C++ több TU-s programjának alapszerződése: egy definíció, kivéve az explicit módon megengedett inline és template eseteket, ahol a definíciók tokenre azonosak. Az odr-use dönti el, hogy egy entitás definíciójára szükség van-e. Az inline függvény és az inline változó egy példányt ad; a `static` és az anonymous namespace belső linkage-je nem ODR-violation, de eltérő címeket és állapotot eredményez. A template definíciója a headerben él, az explicit instantiation és az `extern template` a kódduplikációt szabályozza. A makrófüggő class layout rejtett ODR-violation, amelyet a `static_assert` önmagában nem fog meg. Az LTO és a `-Wodr` hasznos kiegészítő, de nem teljes bizonyíték; a build- és forrásszintű konzisztencia az elsődleges.

33.3.9 Feladatok

1. **Magyarázó feladat.** Egy headerben `static int next_id()` szerepel. Magyarázd el, miért nem ODR-violation ez, és miért lehet mégis hibás a program. Milyen eszközzel ellenőriznéd a szimbólumok linkage-jét?

2. **Kódolási feladat.** Adott a `bad_config.hpp`, amelyben a `Config` struct `#ifdef ENABLE_CACHE` esetén egy extra `int cache_size` tagot kap. Alakítsd át úgy, hogy a `Config` layoutja ne függjön a makrótól, a cache használata viszont továbbra is konfigurálható legyen. Add meg a buildparancsot és az ellenőrzési feltételeket.

3. **Diagnosztikai feladat.** Két TU-t LTO-val linkelsz, és nem kapsz `-Wodr` figyelmeztetést. Bizonyítja-e ez, hogy nincs ODR-violation? Tervezz olyan ellenőrzési listát, amely a fordítási és linkelési folyamat szintjén keresi a rejtett eltéréseket.

33.3.10 Megoldások

33.3.10.1 1. feladat — internal linkage

A `static int next_id()` belső linkage-es entitás, ezért csak a saját TU-jában definiálható; a több TU-ban való előfordulás nem ugyanannak a külső entitásnak a többszörös definíciója, hanem több különálló belső entitás. Ezért nem ODR-violation. A hiba abból ered, hogy minden TU saját példányt kap: a `next_id` címe TU-nként eltér, a benne lévő `static` számláló külön állapot, és ha a függvény címét template argumentumként használjuk, a különböző címek eltérő specializációt vagy viselkedést okozhatnak. A headerben ráadásul minden TU külön másolatot fordít, ami kóddagadáshoz vezet. A helyes megoldás a headerben `inline int next_id()`. Ellenőrzés: a laborban `nm -C` vagy `objdump -t` segítségével összevethető, hogy a `static` függvény szimbóluma lokális, az `inline` függvényé pedig gyenge vagy globális; a két TU-ból vett függvénycím összehasonlítása is megmutatja a különbséget. A tényleges futtatás a labor feladata.

33.3.10.2 2. feladat — layout fix, konfiguráció külön

A `Config` layoutját fixen kell tartani, a cache beállítását pedig külön típusba vagy futásidejű állapotba kell tenni. Egy lehetséges megoldás:

```
// config.hpp
#ifndef CONFIG_HPP
#define CONFIG_HPP

struct Config {
    int timeout;
};

struct CacheConfig {
    int cache_size;
};

#endif
```

A `Config` mérete és a `timeout` offsetje minden TU-ban azonos; a `CacheConfig` csak ott kerül használatba, ahol a cache tényleg kell. A `#ifdef ENABLE_CACHE` legfeljebb a `CacheConfig` használatára vonatkozhat, a `Config` definíciójára nem. A buildparancs minden TU-ra ugyanazt a `-DENABLE_CACHE` értéket adja, ha a cache-t engedélyezni akarjuk; a lényeg, hogy a `Config` layoutja ettől független. Ellenőrzés: minden TU-ban `static_assert(sizeof(Config) == 4)` és `static_assert(offsetof(Config, timeout) == 0)`; a `-E -dM` kimenetében a makrók halmaza azonos; a `nm -C` nem mutat eltérő méretű, azonos nevű szimbólumokat; a LTO `-Wodr` csak kiegészítő ellenőrzés. A `Config` és a `CacheConfig` együttes használatakor a határon explicit módon kell jelezni, melyik típust adjuk át.

33.3.10.3 3. feladat — LTO és ellenőrzés

Nem, a `-Wodr` figyelmeztetés hiánya nem bizonyítja az ODR-helyességet. Az ODR-violation a szabvány szerint IFNDR; a fordító nem köteles diagnosztizálni, és az LTO sem lát minden releváns információt. Az ellenőrzési lista:

- a build flags és a `-D` makrók azonossága minden TU-ra, lehetőség szerint a `compile_commands.json` alapján;
- a headerben lévő `#if`/`#ifdef` feltételek áttekintése, különösen a classlayoutot érintő helyeken;
- `-E -dM` minden TU-ra, a makrók összehasonlítása;
- `static_assert(sizeof(...))` és `offsetof` a kritikus típusokra minden TU-ban;
- `nm -C` és `objdump -t` a szimbólumokra, az azonos nevű, eltérő méretű vagy eltérő linkage-ű szimbólumok keresése;
- a LTO `-Wodr` futtatása kiegészítő jelleggel, a figyelmeztetés hiányát nem tekintve bizonyítéknak.

A végső bizonyíték a forrás- és build-szintű azonosság: ugyanazok a header-definíciók, ugyanazok a makrók és ugyanazok a fordítási opciók minden TU-ban. A mérési és diagnosztikai parancsok futtatása a labor feladata; a fejezet nem állítja, hogy ezeket végrehajtotta.

33.4 C++20 modules és header unit

A header-alapú fejlesztés két rejtett költséget hordoz: a preprocesszor minden fordítási egységben újra beolvassa és feldolgozza ugyanazokat a deklarációkat, és a határok nem nyelvi határok, ezért a fejlécbe került belső részlet óhatatlanul a felhasználók interfészének része lesz. A C++20 modulok (modules) mindkettőre választ adnak: a deklarációk egyszer, egy lezárt határú egységben fordulnak le, az export pedig explicit. Ez a rész a nyelvi szerződést és a hozzá tartozó build gráfot tárgyalja; a fordítási időt itt is erőforrásként kezeljük — a mérés protokollját megadjuk, de számszerű eredményt nem állítunk.

33.4.1 A modul mint nyelvi és buildhatár

Egy **modul** (module) nevesített deklarációhalmaz, amelyet egy vagy több **modul-egység** (module unit) alkot. A **moduldeklaráció** (`export module M;` vagy `module M;`) egy fordítási egységet modul-egységgé tesz; a deklarációnak a fordítási egység első deklarációjának kell lennie (a **global module fragment** kivételével). A moduldeklarációt követő deklarációk a **modul purview**-jába (module purview) tartoznak.

Feltételek és szerepek:

- **Module interface unit**: a moduldeklaráció `export`-tal kezdődik. Ez az egység írja le a modul kifelé látszó felületét.
- **Module implementation unit**: `module M;`, export nélkül. Ugyanahhoz a modulhoz tartozik, de a nevei nem exportáltak; ez a „belső forrásfájl” modulokban.
- **Partition**: `module M:part;` néven a modul belső tagolása. Partíciót csak a saját modulján belül lehet importálni — kívülről a `import M:part;` forma nem elérhető. A partíció lehet interface (`export module M:part;`) és implementation partíció is.
- **Exportált név**: `export`-deklarációval vagy `export { ... }` blokkal jelölt entitás. A nem exportált nevek a modul purview-jában **modul-linkage**-dzsel (module linkage) rendelkeznek, tehát más fordítási egységből nem hivatkozhatók — ez erősebb izoláció, mint a headerben használt `static`.

- **Elérhetőség (reachability)**: az importer nem csak az exportált neveket látja; az exportált entitásokon keresztül *elérhető* deklarációk (például egy exportált függvény visszatérési típusa) is használhatók. Ez nem azonos a „minden látszik” header-szemlélettel.

Fontos határ: a modul **nem** bináris kompatibilitási szerződés. Az exportált osztályok elrendezése, az inline függvények és a name mangling továbbra is ABI-kérdés, lásd Névtorzítás, layout és C++ runtime ABI (36.1) későbbi részét.

33.4.2 Moduldeklaráció, export és import

A minimális szerkezet:

```
export module shapes;      // interface unit, a modul neve: shapes

export int area(int w, int h);   // exportált deklaráció

namespace detail {                // nem exportált: modul-linkage
int scale(int v);
}
```

Az `import` deklaráció szabályai: **nevesített modult** és header unitot csak névtér-hatókörben (namespace scope) lehet importálni, függvényen vagy osztályon belül nem; egy fordítási egységben a moduldeklaráció után, az első deklarációk között érdemes tartani. Egy modul-egységnek pontosan egy moduldeklarációja lehet, és a modulgráf nem tartalmazhat körkörös importot — ez a tervezésre is kihat: a partíciók közötti függőségnek irányítottnak kell lennie.

A `export import :part;` azt jelenti, hogy a partíció importált nevei nemcsak láthatók, hanem tovább is exportáltak. Ez a „homlokzat” minta alapja: a fő interface unit néhány soros, és partíciókra bontja a felületet.

33.4.3 Interface, implementation unit és partition: kidolgozott példa

A példa egy `metrics` modul: a `:core` partíció deklarálja az adattípust és a számítást, a `:format` partíció szövegesít, a fő interface unit pedig újraexportálja mindkettőt. A körkörös import elkerülése érdekében a formázó partíció a `:core`-tól függ, nem a fő egységtől.

```
// src/metrics-core.ccm
module;                       // global module fragment kezdete
#include <cstddef>
#include <vector>
export module metrics:core;   // interface partíció

export namespace metrics {
struct Summary { std::size_t count; double mean; };
Summary summarize(const std::vector<double>& xs);
}

// src/metrics-impl.cc  (implementation partíció: nincs export)
module metrics:core;

namespace metrics {
Summary summarize(const std::vector<double>& xs) {
```

```
    if (xs.empty()) return {0, 0.0};
    double sum = 0.0;
    for (double x : xs) sum += x;
    return {xs.size(), sum / static_cast<double>(xs.size())};
}
}

// src/metrics-format.ccm
module;
#include <string>
export module metrics:format;

import :core;                  // partíció importja csak a modulon belül

export namespace metrics {
std::string format(const Summary& s);
}

// src/metrics.ccm  (fő interface unit: vékony homlokzat)
export module metrics;
export import :core;
export import :format;

// src/main.cc
#include <cstdio>
#include <vector>
import metrics;

int main() {
    std::vector<double> xs{1.0, 2.0, 6.0};
    auto s = metrics::summarize(xs);
    std::printf("%zu %s\n", s.count, metrics::format(s).c_str());
}
```

Értelmezés: a `main.cc` sosem látja a `detail`-szintű segédfüggvényeket, és nem függ attól, hogy a `summarize` teste melyik fájlban van. A `Summary` egyetlen definícióval rendelkezik a `:core` purview-jában, tehát a header-világ klasszikus ODR-hibái (két, eltérő makróállapot mellett fordított definíció) itt szerkezetileg nehezebben állnak elő — de nem kizártak, lásd a BMI-szakaszt és ODR, inline és több fordítási egység (33.3).

33.4.4 Global module fragment, legacy header és header unit

A **global module fragment** (`module;` … az első moduldeklarációig) az egyetlen hely, ahol a modul-egységben hagyományos `#include`-ot érdemes használni. Az itt bevont deklarációk a globális modulhoz csatolódnak, nem a modul purview-jához. Ha egy fejlécet a moduldeklaráció *után* include-olunk, annak entitásai a modulhoz csatolódnak — ez más modul-attribúciót ad ugyanannak a fejlécnek, ha máshol is include-olják, és ez ODR-sértéshez, illetve diagnosztizálatlan viselkedéshez vezethet. Gyakorlati szabály: *include csak a fragmentben, vagy sehol.*

A **header unit** a fejléc lefordított formája: `import <vector>;` vagy `import "my_header.h";`. Ilyenkor a fejléc deklarációi egy külön lefordított egységből származnak, nem szövegesen másolódnak be. Két következmény:

- A header unitból **makrók nem terjednek át** az importerbe. A `#define`-okra épülő feltételes fordítás (pl. `NDEBUG`, feature-kapcsolók) a modulok világában nem működik tovább: a konfigurációt típusos vagy `constexpr` alakban kell átadni, lásd Compile-time adatok és típusos konfiguráció (33.2).
- A named module sem exportál makrókat a C++20-ban; a későbbi szabvány lazított ezen, de a gyakorlati támogatás verzió- és implementációfüggő, ezért a laborprofilban külön ellenőrizendő, nem tekinthető hordozható eszköznek.

A header unit támogatottsága egyenetlen: a CMake modulszkennelése jellemzően a named module-ökre épül, a header unitot a build rendszer gyakran nem kezeli automatikusan. Ezt a választott kiadás dokumentációjában kell ellenőrizni, mielőtt a `import <...>;` formát a projekt alapjává tesszük.

33.4.5 A BMI és a build gráf toolchainfüggősége

A fordító a modul-egységből **BMI**-t (Binary Module Interface) készít; a nevek implementációnként eltérnek (Clang `.pcm`, MSVC `.ifc`, GCC esetében a modul-gyorsítótárazott könyvtár). A BMI **nem** stabil, hordozható artefaktum: nem szerződés két compiler vagy két verzió között, és nem is cél, hogy az legyen (ezt a témaköri határ kizárja).

Artefaktum	Előállító	Mihez kötött	Hordozhatóság
forrás (`.ccm`, `.cpp`)	fejlesztő	—	hordozható
BMI	compiler	compiler és verzió, target, nyelvi mód, flag-ek, makróállapot, az importált modulok BMI-je	nem hordozható; újragenerálandó
tárgykód (`.o`)	compiler	target ABI, flag-ek	ABI-függő
szkennelési gráf	scanner + build rendszer	scanner verziója	a build gépen újragenerálható

Ebből következik a build gráf alakja: a fordítás nem lineáris, hanem **DAG**. Először szkennelni kell a forrásokat, hogy kiderüljön, melyik egység melyik modult importálja; a topológiai sorrendben elkészülnek a BMI-k, majd a tárgykódok. A `main.cc` csak akkor fordítható, ha a `metrics` BMI-je már létezik.

Kritikus következmény: ha ugyanazt a modult kétféle flag-készlettel (pl. eltérő `-D` makróval, `-O` szinttel vagy `-std` értékkel) fordítjuk le, a két BMI nem ugyanazt a programszabályt képviseli. A build rendszernek a BMI-t a *teljes* fordítási konfigurációra kell kulcsoznia, különben a különböző TU-k eltérő definíciót látnak — ez pontosan az ODR, inline és több fordítási egység (33.3) ODR-szabályának megsértése, és a tünet gyakran nem linkelési hiba, hanem csendes viselkedésbeli eltérés.

33.4.6 CMake modulszkennelés és támogatási feltételek

Modern CMake-ben a modulokat a `FILE_SET CXX_MODULES` és a bekapcsolt szkennelés kezeli. Az alábbi `CMakeLists.txt` a fenti példához tartozik; a verziószámok és a compilerflag-ok **ellenőrizendők** a laborban, nem tekintendők itt igazoltnak:

```
cmake_minimum_required(VERSION 3.28)    # a támogatás verziófüggő: ellenőrizd!
project(metrics_demo LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
set(CMAKE_CXX_SCAN_FOR_MODULES ON)      # pontosabban target property-ként

add_library(metrics)
target_sources(metrics
  PUBLIC FILE_SET CXX_MODULES FILES
    src/metrics.ccm
    src/metrics-core.ccm
    src/metrics-format.ccm
  PRIVATE
    src/metrics-impl.cc)
target_compile_features(metrics PUBLIC cxx_std_20)

add_executable(demo src/main.cc)
target_link_libraries(demo PRIVATE metrics)
```

```
# laborprofil-jelölt; a compiler- és CMake-verziót rögzíteni kell
cmake -S . -B build -G Ninja \
  -DCMAKE_CXX_COMPILER=g++-14 \
  -DCMAKE_CXX_FLAGS="-fmodules-ts"
cmake --build build
```

Támogatási feltételek, amelyeket a felvétel előtt ellenőrizni kell: (1) a compiler verziója támogatja-e a partíciókat és a szóban forgó flag-et; (2) a build rendszer verziója ismeri-e a `CXX_MODULES` file setet; (3) a generátor (Ninja/Makefile) képes-e a dinamikusan felfedezett függőségek kezelésére; (4) a fordító tud-e helyes sorrendet felállítani a BMI-kból. Ha bármelyik hiányzik, a projekt essen vissza a headeres változatra — ez nem kudarc, hanem a hordozhatósági szerződés része.

33.4.7 Headeres alternatíva és a fordítási idő

Ugyanaz a felület headerrel: `include/metrics.hpp` (a `Summary` és a két függvény deklarációja), `src/metrics_header.cpp` (a definíciók), és a `main` a `#include "metrics.hpp"` formát használja.

```
add_library(metrics_header STATIC src/metrics_header.cpp)
target_include_directories(metrics_header PUBLIC include)
add_executable(demo_header src/main_header.cc)
target_link_libraries(demo_header PRIVATE metrics_header)
```

A fordítási idő összehasonlítása csak **azonos protokollal** értelmes: tiszta build (`rm -rf build`), azonos gép és terhelés, azonos flag-ek, hideg gyorsítótár, legalább három futás mediánja, és a mért mennyiség kimondása (teljes build, adott target újrafordítása, vagy csak a fejlécfeldolgozás). Compiler-specifikus profilozás (GCC `-ftime-report`, Clang `-ftime-trace`) alkalmas a bontásra, de a számok target- és verziófüggők. Ebben a könyvben nem állítunk mért értéket; a mérés a tanuló laborjának feladata.

33.4.8 Tipikus hibák

- **Include a modul purview-jában.** `export module m;` után `#include <vector>`: a `std::vector` a modulhoz csatolódik. Javítás: global module fragment, vagy header unit.
- **Makróátvitel feltételezése.** `import config;` után `#ifdef DEBUG` nem működik. Javítás: `constexpr bool`/`inline constexpr` konfiguráció exportálása.
- **Partíció importja kívülről.** `import metrics:core;` a `main.cc`-ben hibás. Javítás: a fő interface unit exportálja a szükséges neveket.
- **Körkörös import.** A formázó partíció a fő egységre hivatkozik, amely őt importálja. Javítás: közös `:core` partíció.
- **Elavult BMI.** Flag- vagy compiler-váltás után a régi BMI-t használja a build. Javítás: tiszta build, a BMI könyvtár törlése, a konfiguráció kulcsozása.
- **Ugyanaz az osztály headerből és modulból.** Két, eltérő modul-attribúciójú definíció → ODR-sértés. Javítás: egyetlen forrás, a határ tudatos megválasztása.

33.4.9 Feladatok

1. Alakítsd át a `metrics` példát úgy, hogy a `format` függvény a `:core` partícióba kerüljön, és indokold, miért romlik vagy javul a build gráf.
2. A `main.cc`ben a `#include <cstdio>` után `import metrics;` áll, és a `printf` formátumsztringjében `%zu` szerepel. Egészítsd ki a példát egy olyan második `main`-változattal, amely `std::format`-ot használ, és mondd meg, milyen include/import és milyen szabványfeltétel kell hozzá.
3. Tervezz olyan CMake-konfigurációt, amely ugyanabban a projektben egyszerre tartalmazza a modulos `metrics` és a headeres `metrics_header` változatot, és magyarázd el, miért nem kerülhet ugyanabba a célba a `metrics.hpp` include-ja és a `metrics` modul importja.
4. Készíts mérési tervet a két változat fordítási idejének összehasonlítására; add meg a független változókat, a rögzítendő feltételeket és azt, hogy mit Nem lehet a mérésből kiolvasni.

33.4.10 Megoldások

1. A `format` átkerül a `:core`-ba: a `:format` partíció megszűnik, a fő interface unit csak `export import :core;`-t tartalmaz. A build gráf csomópontjai száma csökken (egy BMI-vel kevesebb), a párhuzamosíthatóság romlik, mert a formázás (és a `<string>` feldolgozása) a `:core` BMI-jébe kerül, így minden `:core`-tól függő egység a formázás fordítási költségét is viseli. Ellenőrzési feltétel: a `metrics` felülete változatlan (a `format` továbbra is `metrics::format`), és a `main.cc` változtatás nélkül fordul.

2. A `std::format` a C++20 `<format>` fejlécében érhető el, de a library-támogatás nem minden C++20-as implementációban teljes; ezért a laborprofilban ellenőrizni kell. Modulos változatban a fejlécet a fragmentbe kell tenni:

```
module;
#include <format>
#include <string>
export module metrics:format;
import :core;
export namespace metrics {
std::string format(const Summary& s);
}
```

Ellenőrzési feltétel: a `main.cc`-ben továbbra is `#include <cstdio>` maradhat (nem modul-egység), és a `%zu` helyett `{}` formázás használható; ha a `<format>` nem elérhető, a megoldás nem tekinthető késznek, csak a C++20-as alapra visszavezetett változat.

3. Két külön target: `metrics` (FILE_SET CXX_MODULES) és `metrics_header` (STATIC, `target_include_directories`-szel). Egy célon belül a `metrics.hpp` include-ja és a `import metrics;` egyazon TU-ban nem keverhető össze: a `Summary` két különböző modul-attribúciójú definíciója keletkezne (header → globális modul, modul → `metrics` purview), ami ODR-sértés. Ellenőrzési feltétel: a két executable külön target, és a `main.cc` csak az egyik formát használja.

4. Független változó: a felület megvalósítási módja (modul vs. header). Rögzítendő: compiler és verzió, CMake/Ninja verzió, flag-ek, gép, mag- és memóriakonfiguráció, hideg gyorsítótár, ismétlésszám. Mérendő: teljes tiszta build falaóra-ideje és a `demo` target újrafordítási ideje. Nem olvasható ki: általános következtetés más compilerre vagy targetre, és nem bizonyíték arra, hogy a modul „mindig gyorsabb” — a BMI előállítás egyszeri költsége és a párhuzamosíthatóság is számít.

33.4.11 Összefoglaló

A modul nyelvi határ: az interface unit exportál, az implementation unit és a partíció belső szerkezetet ad, a partition külvilág felé nem importálható. A `#include` helye a global module fragmentre korlátozandó, a header unit deklarációkat közvetít, makrókat nem. A BMI compiler-, verzió- és flagfüggő artefaktum, ezért a build gráf DAG-ját szkennelés építi fel, és a konfigurációt kulcsozni kell. A CMake `CXX_MODULES` támogatása verziófüggő, és mindig kell legyen headeres visszaesési út. A fordítási idő csak dokumentált protokollal hasonlítható össze.

33.4.12 Források és korlátok

Ehhez az egységhez nem kaptam olvasott forrásjegyzetet, ezért itt nincs hivatkozható elsődleges forrás és nincs igazolt szakaszszám. A hivatkozott szabványelemek (C++20 `[module]`, ODR, `constexpr`) nevesítve, de paragrafusra nem támaszkodva szerepelnek; a compiler-, CMake- és flag-részletek verziófüggők, és a megadott buildparancsok nem futtatott, ellenőrizendő példák. A tanulónak a választott compiler, standard library és CMake kiadás hivatalos dokumentációjában kell igazolnia a támogatást, mielőtt a modulokat a projekt alapjává teszi.

33.5 Feature-test, standard library és build capability

A fejezet gerincét a korábbi részekben már rögzítettük: a `constexpr` kiértékelés szabályait constexpr, consteval és constant evaluation (33.1), az ODR-t ODR, inline és több fordítási egység (33.3), a modulokat pedig C++20 modules és header unit (33.4) tárgyalta. Itt most nem ezeket ismételjük, hanem azt a gyakorlati kérdést vesszük elő, amely mindháromhoz hozzákapcsolódik: **honnan tudom meg, hogy egy adott fordítási lánc tényleg tudja-e azt, amit a példakódom feltételez?** A modern C++ példatár gyorsabban mozdul, mint az intézményi laborkörnyezet, és a `-std=c++23` kapcsoló önmagában semmit nem garantál. A cél egyetlen, karbantartható capability-tábla felépítése, amely külön kezeli a **nyelvi** és a **könyvtári** oldalt, és amelyet configure time-ban, valódi fordítással ellenőrzünk.

33.5.1 A capability kérdés pontos megfogalmazása

Definíció (capability). Egy adott `(compiler, standard library, nyelvi mód, target)` négyes esetén egy `F` nyelvi vagy könyvtári funkció *elérhető*, ha létezik olyan program, amely `F`-et a szabvány által előírt módon használja, és a lánc azt fordítási hiba és diagnosztikai figyelmeztetés nélkül lefordítja, majd a dokumentált viselkedést produkálja. Az *elérhetőség nyilatkozata* ehhez képest gyengébb állítás: valamilyen makró definiált, vagy a dokumentáció említi a funkciót.

Ez a megkülönböztetés nem szőrszálhasogatás. A feature-test makrók a *szándékot* rögzítik, a compile probe a *tényt*. Egy capability-tábla akkor használható, ha megmondja, melyik sor melyik bizonyítékon nyugszik.

Előfeltételek. A fejezet példái feltételezik, hogy a fordítót explicit nyelvi móddal hívjuk (`-std=c++20` vagy újabb), GNU bővítések nélkül, Linux hosted user-space alatt. A pontos compiler- és library-kiadást a laborprofil rögzíti; ahol a szöveg konkrét verziót nem nevez meg, ott az állítás verziófüggő, és a tanulónak kell ellenőriznie.

33.5.2 `__cplusplus` és a `__cpp_*` nyelvi makrók

A `__cplusplus` makró értéke a *nyelvi módot* kódolja, nem a fordító kiadását és nem a könyvtárét. A C++20 alapértéke `202002L`, a C++23-é `202302L`; a C++17 `201703L`. Ezek a számok dátum-alapúak, és a szabvány rögzíti őket, tehát összehasonlításra alkalmasak:

```
#if __cplusplus >= 202002L
/* C++20 vagy újabb nyelvi mód */
#endif
```

Ez a vizsgálat azonban csak azt mondja meg, hogy a *nyelvtan* és a *szemantika* melyik kiadása aktív. Semmit nem mond arról, hogy a `<format>` header létezik-e, hogy a `std::expected` implementált-e, vagy hogy a `std::stacktrace` mögött van-e működő unwinder.

A nyelvi funkciók többségéhez a fordító külön, finomabb makrót definiál: ezek a `__cpp_*` alakúak, például `__cpp_constexpr`, `__cpp_consteval`, `__cpp_constinit`, `__cpp_concepts`, `__cpp_modules`, `__cpp_if_consteval`. A nevük azt sugallja, hogy egy az egyben megfelelnek egy funkciónak, de két dolgot fontos észrevenni. Egyrészt az értékük nem logikai igaz/hamis, hanem egy dátum, amely a funkció *aktuális specifikációjának* évjáratát jelöli — ugyanannak a makrónak több elfogadott értéke létezhet, és a nagyobb érték gazdagabb szemantikát ígér. Másrészt nem minden nyelvi funkcióhoz tartozik makró; a makró hiánya ezért **nem bizonyíték** a funkció hiányára, csak a makró hiányát bizonyítja. A `#ifdef` tehát konzervatív irányban használható: ha definiált, elfogadhatjuk; ha nem, compile probe kell.

33.5.3 `<version>` és a `__cpp_lib_*` makrók

A C++20-tól létezik a `<version>` header, amelynek egyetlen feladata a szabványos feature-test makrók közreadása. A könyvtári funkciók makrói `__cpp_lib_*` alakúak: `__cpp_lib_format`, `__cpp_lib_print`, `__cpp_lib_expected`, `__cpp_lib_mdspan`, `__cpp_lib_stacktrace`, `__cpp_lib_source_location` és így tovább.

Kritikus részlet, hogy ezeket **a standard library implementációja** definiálja, nem a compiler. Ugyanaz a `g++ -std=c++20` hívás két különböző láncot jelenthet aszerint, hogy a driver melyik libstdc++-t és melyik fejlécet találja; és ugyanaz a Clang bináris hol a libstdc++-hoz, hol a libc++-hoz linkel. A nyelvi makrók és a könyvtári makrók ezért két független tengelyen mozognak.

A `<version>` jelenlétét magát is vizsgálni kell, és erre a C++17 óta szabványos `__has_include` való:

```
#if defined(__has_include)
#  if __has_include(<version>)
#    include <version>
#    define HAVE_VERSION_HEADER 1
#  endif
#endif
#ifndef HAVE_VERSION_HEADER
#  define HAVE_VERSION_HEADER 0
#endif
```

A `__has_include` szemantikája szűk: a preprocesszor `#if` kifejezésében használható, és azt válaszolja meg, hogy a megadott header *megtalálható-e* a keresési útvonalon. Azt nem mondja meg, hogy a header tartalma megfelel-e a várt szerződésnek, és nem helyettesíti a tényleges fordítási próbát. Ha a `<version>` hiányzik, a `__cpp_lib_*` makrók jellemzően definiálatlanok maradnak — de egyes implementációk más headerekből is közreadhatják őket, ezért a `#ifdef __cpp_lib_format` include nélkül félrevezető lehet.

33.5.4 Két független állapot: compiler és standard library

A labornaplóban a capability-t nem egyetlen verziószámmal kell azonosítani, hanem egy rekorddal, amelynek minden mezője külön forrásból származik:

Dimenzió	Hol olvasható	Mit bizonyít
nyelvi mód	`__cplusplus`	a frontend által elfogadott nyelvtan
nyelvi funkció	`__cpp_*` makró, `-dM -E` kimenet	a compiler állítása a funkcióról
könyvtár-változat	`_GLIBCXX_RELEASE` (libstdc++), `_LIBCPP_VERSION` (libc++)	melyik implementáció, melyik generáció
könyvtári funkció	`__cpp_lib_*` a `<version>`-ből	a library állítása a funkcióról
tényleges használhatóság	compile probe, `-Werror`-rel	a négyes együttes viselkedése

A `_GLIBCXX_RELEASE` és a `_LIBCPP_VERSION` olyan implementációs makrók, amelyeket a szabvány nem ír elő; használatuk diagnosztikára alkalmas, de nem hordozható feature-detection. Hasonlóan, a `__VERSION__` a compiler saját leírója. Ezeket a naplóba írjuk, nem feltételként használjuk.

Kidolgozott példa. Az alábbi program egyetlen forrásból, egy adott `(compiler, stdlib, -std)` hármasra rögzíti a fenti mezőket. A `SHOW` makró trükkje: ha a makró definiált, az értéke stringesül, ha nem, akkor a *neve* jelenik meg a kimenetben — ez diagnosztikára elég, produkciós feature-detectionre nem.

```
// capability_snapshot.cpp
// Egy (compiler, stdlib, -std) hármas állapotának rögzítése.
// Illusztratív eszköz, nem helyettesíti a tényleges használati tesztet.
#include <cstdio>

#if defined(__has_include)
```

```
#  if __has_include(<version>)
#    include <version>
#    define HAVE_VERSION_HEADER 1
#  endif
#endif
#ifndef HAVE_VERSION_HEADER
#  define HAVE_VERSION_HEADER 0
#endif

#define STR_(x) #x
#define STR(x) STR_(x)
#define SHOW(m) std::printf("  %-24s %s\n", #m, STR(m))

int main()
{
    std::printf("== language ==\n");
    SHOW(__cplusplus);
    SHOW(__cpp_constexpr);
    SHOW(__cpp_consteval);
    SHOW(__cpp_constinit);
    SHOW(__cpp_concepts);
    SHOW(__cpp_modules);

    std::printf("== library (<version>: %s) ==\n",
                HAVE_VERSION_HEADER ? "found" : "missing");
#if HAVE_VERSION_HEADER
    SHOW(__cpp_lib_format);
    SHOW(__cpp_lib_print);
    SHOW(__cpp_lib_expected);
    SHOW(__cpp_lib_mdspan);
    SHOW(__cpp_lib_stacktrace);
#endif

    std::printf("== implementation ==\n");
#ifdef __VERSION__
    SHOW(__VERSION__);
#endif
#ifdef _GLIBCXX_RELEASE
    SHOW(_GLIBCXX_RELEASE);
#endif
#ifdef _LIBCPP_VERSION
    SHOW(_LIBCPP_VERSION);
#endif
    return 0;
}
```

Fordítás és futtatás a laborban, két lánccal, hogy az eltérés látszódjék:

```
g++ -std=c++20 -O2 -Wall -Wextra -pedantic \
    -o capability_snapshot_gcc capability_snapshot.cpp
./capability_snapshot_gcc
```

```
clang++ -std=c++20 -O2 -Wall -Wextra -pedantic -stdlib=libc++ \
    -o capability_snapshot_clang capability_snapshot.cpp
./capability_snapshot_clang
```

A kimenet **nem mért eredmény** ebben a szövegben: a konkrét sorok a telepített kiadásoktól függenek, és a tanulónak kell rögzítenie őket a saját labornaplójában. Amit a példa bizonyít, az a módszer: két külön blokk, két külön állításforrás, és a `__cplusplus` önmagában egyik blokkra sem elég.

33.5.5 Configure-time compile probe

A makróvizsgálat olcsó, de megengedő: egy definiált `__cpp_lib_format` nem garantálja, hogy a laborban használt pontos hívás lefordul. A CMake `check_cxx_source_compiles` a valódi próbát végzi el, és az eredményből konfigurációs makró lesz.

```
cmake_minimum_required(VERSION 3.20)
project(capability_probe LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

include(CheckCXXSourceCompiles)

check_cxx_source_compiles([[
#include <format>
#include <string>
int main() {
    std::string s = std::format("{}", 42);
    return s == "42" ? 0 : 1;
}
]] APP_HAS_STD_FORMAT)

check_cxx_source_compiles([[
#include <expected>
int main() {
    std::expected<int, int> e{7};
    return *e == 7 ? 0 : 1;
}
]] APP_HAS_STD_EXPECTED)

add_executable(capability_snapshot capability_snapshot.cpp)
if(APP_HAS_STD_FORMAT)
    target_compile_definitions(capability_snapshot PRIVATE APP_HAS_STD_FORMAT=1)
endif()
if(APP_HAS_STD_EXPECTED)
    target_compile_definitions(capability_snapshot PRIVATE
      ↪ APP_HAS_STD_EXPECTED=1)
endif()
```

A `check_cxx_source_compiles` a próbát a projekt C++ standardjával és a `CMAKE_REQUIRED_FLAGS` tartalmával futtatja; azt, hogy a standard-beállítás pontosan hogyan öröklődik a check-fordításba,

a használt CMake-kiadás dokumentációja szerint kell ellenőrizni. A probe eredménye `0/1` konfigurációs makró, amelyet a forrás **és** a build script is lát — így a fallback ugyanazon a néven dől el, nem két helyen.

A capability-tábla a fenti kimenetből és a CMake cache-ből áll össze. Egy sor akkor tekinthető igazoltnak, ha a probe lefutott és a használt hívásminta lefordult; a makró csak előzetes szűrő.

A feldolgozandó funkciók közül a `format`, a `print`, az `expected`, az `mdspan` és a `stacktrace` tipikusan **könyvtári** oldalon jelenik meg, ezért a `__cpp_lib_*` és a probe a mérvadó. A `modules` ezzel szemben vegyes: nyelvi makró (`__cpp_modules`) *és* build-rendszer-támogatás kell hozzá; a modulok stabil, cross-compiler bináris formátumként való kezelése nem része ennek a fejezetnek, és a laborprofilban rögzített compiler/build párosítás nélkül a `__cpp_modules` definiáltsága keveset bizonyít C++20 modules és header unit (33.4). A `print` C++23-as státusza miatt a C++20-as fallback itt is kötelező: `std::format` + `std::fputs` charconv, format és C++23 print (32.2). Az `mdspan` kétdimenziós indexeléséhez a korábbi view-térkép adja a fogalmi hátteret Multidimenziós view és numeric utility térkép (31.5).

33.5.6 Tipikus hibák és javításuk

`__cplusplus` mint könyvtár-feltétel. A `#if __cplusplus >= 202302L` körül `#include <expected>` szerepel, majd a build elhasal egy olyan láncon, ahol a frontend már C++23 módban van, de a könyvtár generációja régebbi. *Javítás:* a nyelvi és a könyvtári feltételt külön sorba tenni, és a könyvtárit `__has_include` + `__cpp_lib_*` + probe hármasra bízni.

Feature-makró include nélkül. A `#ifdef __cpp_lib_format` a `#include <version>` (vagy `<format>`) előtt áll, ezért hamis negatívot ad, és a fallback ág fut le feleslegesen. *Javítás:* a makróvizsgálat mindig a header include után.

A makró értékének logikai kezelése. A `#if __cpp_lib_format` az értéket igaz/hamisnak veszi; valójában dátum, és egy adott évjáratot kell megcélozni: `#if defined(__cpp_lib_format) && __cpp_lib_format >= 202110L`. Az évjárat-számot a `<version>` és a fordítási lánc dokumentációja ellenőrzi, nem a tankönyvi emlékezet.

A `__has_include` rossz helyen. Nem használható futásidejű kódban, és nem lehet rá `#if`-en kívül hagyatkozni; továbbá a találat nem jelent funkcionális megfelelést.

A lánc azonosítatlan rögzítése. A build logból csak a `-std=` látszik, a compiler és a library nem. *Javítás:* a capability-snapshot kimenetét a build artefaktum mellé menteni, dátumozva.

Bizonyíték-korlát. Ez a fejezet nem nevez meg konkrét compiler- és library-kiadást, és nem állítja, hogy a fenti kód bármely láncon lefutott. A szabványos makrónevek és a `-std` értékek a szabványból származó ismeretek; a tényleges állapotot a tanuló által futtatott snapshot rögzíti.

33.5.7 Feladatok

1. feladat (magyarázó). Egy kolléga azt állítja, hogy „C++23-mal a `std::expected` biztosan elérhető, hiszen `__cplusplus == 202302L`”. Cáfold meg az állítást, és adj egy konkrét, elvi ellenpéldát olyan `(compiler, standard library)` párra, ahol a nyelvi mód C++23, a `<expected>` mégis hiányzik vagy hiányos.

2. feladat (kódolási). Írj `app_capabilities.hpp` fejlécet, amely `APP_HAS_FORMAT` makrót ad 1-re, ha a `std::format` használható, egyébként 0-ra. A döntés használja a `__has_include(<format>)`-ot és a `__cpp_lib_format`-ot, de a végső szót a build-rendszer által

átadott `APP_HAS_STD_FORMAT` (probe-eredmény) mondja ki. Mellékelj egy rövid `#if`-ágat, amely a két esetben eltérő implementációt választ.

3. feladat (diagnosztikai). Adott egy build, amely `clang++ -std=c++20 -stdlib=libstdc++` alatt lefordul, `clang++ -std=c++20 -stdlib=libc++` alatt viszont a `__cpp_lib_expected` hiánya miatt más ágat vesz. Írd le, hogyan izolálod a három változót (compiler, library, nyelvi mód), mit rögzítesz a naplóban, és hogyan döntöd el, hogy ez valódi capability-különbség-e, vagy csak a fejléc-keresési útvonal hibája.

33.5.8 Megoldások

1. megoldás. A `__cplusplus` a frontend nyelvi módját kódolja; a `<expected>` elérhetőségét a standard library implementációja dönti el, amelyet a fordítótól függetlenül, külön csomagként telepítenek és verzióznak. Elvi ellenpélda: egy új frontend egy régebbi, de ABI-kompatibilis standard library-vel párosítva — a nyelvi makrók a C++23 értéket adják, a `<version>` viszont nem definiálja a `__cpp_lib_expected`-et, vagy a header egyáltalán nincs a keresési útvonalon. *Ellenőrzési feltétel:* az állítás akkor tekinthető cáfoltnak, ha a snapshot a nyelvi blokkban `202302L`-t, a könyvtári blokkban definiálatlan `__cpp_lib_expected`-et mutat, és a `<expected>` próba-fordítása hibára fut. A konkrét verziókat a labor rögzíti; a fejezet nem nevez meg párost.

2. megoldás. A fejléc a probe-eredményt tekinti elsődlegesnek, a makrókat csak fallbackként:

```
// app_capabilities.hpp
#pragma once

#if defined(APP_HAS_STD_FORMAT)
#  define APP_HAS_FORMAT APP_HAS_STD_FORMAT
#else
#  if defined(__has_include)
#    if __has_include(<format>) && defined(__cpp_lib_format)
#      define APP_HAS_FORMAT 1
#    else
#      define APP_HAS_FORMAT 0
#    endif
#  else
#    define APP_HAS_FORMAT 0
#  endif
#endif
```

Használat:

```
#include "app_capabilities.hpp"
#include <string>

#if APP_HAS_FORMAT
#  include <format>
inline std::string tag(int v) { return std::format("[{}]", v); }
#else
#  include <cstdio>
inline std::string tag(int v) {
    char b[32];
    std::snprintf(b, sizeof b, "[%d]", v);
    return std::string{b};
```

```
}
#endif
```

Indoklás: a `#if APP_HAS_FORMAT` mindig numerikus, nem `defined`, ezért a 0-s fallback és a definiálatlan eset nem keveredik. *Ellenőrzési feltétel:* `-std=c++20 -Werror` mellett mindkét ág fordul, és `APP_HAS_FORMAT` értéke a CMake cache-ből visszaolvasható. A `<format>` viselkedését ez a fejezet nem méri.

3. megoldás. Először a compilert rögzítjük (`--version` és `-v` kimenet), majd a standard libraryt (`_GLIBCXX_RELEASE` vagy `_LIBCPP_VERSION` a snapshotból), végül a nyelvi módot (`__cplusplus`, `-std`). A három változóból egyszerre csak egyet mozgatunk: azonos compiler mellett `-stdlib`-ot váltunk, majd azonos library mellett compilert. Valódi capability-különbségről akkor beszélünk, ha a probe-fordítás mindkét láncon lefut, és az eredmény konzisztensen követi a library-változatot; ha viszont a `__has_include(<expected>)` hamis, miközben a fájl a fájlrendszeren létezik, akkor a keresési útvonal vagy az include-sorrend a hiba, nem a capability. *Ellenőrzési feltétel:* a napló tartalmazza mindhárom dimenzió értékét, a probe parancsát és annak exit-státuszát.

33.5.9 Összefoglaló

A capability nem azonos a verziószámmal. A `__cplusplus` és a `__cpp_*` makrók a **compiler** állításai a **nyelvi** oldalról; a `<version>`-ből olvasott `__cpp_lib_*` makrók a **standard library** állításai a **könyvtári** oldalról; a kettő külön telepített, külön verziózott komponens. A `__has_include` csak a header megtalálhatóságát jelzi, nem a szerződés teljesülését. A megbízható döntés három rétegből áll: makró-szűrő, configure-time compile probe, és a laborban rögzített tényleges használati teszt. Az újabb funkciók — `format`, `print`, `expected`, `mdspan`, `stacktrace` — esetében ez a hármas adja a példa és a fallback közötti választás alapját; a `modules` esetében ehhez build-rendszer-támogatás is kell, amit a korábbi rész tárgyalt C++20 modules és header unit (33.4).

34 Modern C++ konkurencia és szinkronizáció

34.1 thread, jthread és kooperatív cancellation

Ez az egység két dolgot választ szét, amit a gyakorlatban rendszeresen összemosnak: a szál mint **erőforrás** életciklusát (joinable, join, detach) és a szál mint **munkavégző** leállítását (stop kérés, cooperative cancellation). Egy DevOps szolgáltatás leállítása tipikusan így néz ki: SIGTERM érkezik, a listener abbahagyja az új kapcsolatok fogadását, a dolgozó szálak befejezik vagy eldobják a futó munkát, majd a folyamat kilép. Ha a kettő nincs külön kezelve, két rossz véglet közül választhatunk: vagy a `std::terminate`, vagy egy végtelen `join`. A C oldalon a Szál-életciklus, pthreads és C11 threads (15.1) részben a `pthread_create`/`pthread_join` pár mutatta meg a kézi lifecycle-t; a lambda capture élettartamáról Lambda, capture és closure lifetime (30.1) adott pontos szabályokat; a `noexcept` és hibapolicy határokról noexcept határok és hibapolicy (25.3) szólt. Ezekre építünk, de a C memóriamodellt nem írjuk újra.

Az itt leírt nyelvi szabályok az ISO C++20 szálfejezetére támaszkodnak. Ehhez az egységhez nem állt rendelkezésre ellenőrzött forrásjegyzet, ezért szakaszszámot és hivatkozást nem adok: ahol a viselkedés implementációra vagy platformra van bízva, azt külön jelölöm, és a saját toolchainben fordítási próbával ellenőrizendőnek mondom. A kódok nem tekintendők lefuttatottnak.

34.1.1 A szál mint joinable erőforrás

A `std::thread` objektum egy végrehajtási szál **handle-je**. Pontos definíció: az objektum *joinable*, ha olyan végrehajtási szálat azonosít, amelyet még nem `join`oltunk és nem `detach`oltunk. A `joinable()` lekérdezés pontosan ezt adja vissza; default-konstruált, már joinolt, detacholt és elmozgatott (moved-from) objektumra hamis.

állapot	`joinable()`	destruktor viselkedése
default-konstruált	false	nincs hatás
futó, nem joinolt	true	`std::terminate()`
`join()` után	false	nincs hatás
`detach()` után	false	nincs hatás
moved-from	false	nincs hatás

A `join()` blokkol, amíg a szál befejeződik, majd az objektumot nem-joinable állapotba teszi; nem-joinable objektumon hívva `std::system_error`-t dob (a szabvány a önmagára váró joinra is hibát ír elő, de ezt nem minden implementáció képes felismerni — ezért ne építs rá). A `detach()` leválasztja a szálat: a végrehajtás folytatódik, de többé nem tudunk rá várni, és a szál erőforrásai a kilépéskor szabadulnak fel. A `detach` nem „tűz és felejtsd el” eszköz: ha a callable referenciát capture-öl, a hivatkozott objektum élettartama nincs semmihez kötve.

A destruktor terminate-szabálya tudatos tervezési döntés, nem véletlen. Két „csendes” alternatíva lenne: automatikus detach (ekkor a szál a háttérben tovább olvasná a már felszabadult objektumot, és a folyamat kilépésekor a `main` visszatérése után is futhatna), vagy automatikus join (ekkor

egy destruktor blokkolna, akár kivétel-dobás közbeni stack unwinding során is, ahol a várakozás holtpontot okozhat). A terminate hangos, és arra kényszerít, hogy a lifecycle-t explicit módon kezeljük. A gyakorlati következmény: minden `std::thread` útját vagy `join`, vagy `detach`, vagy egy RAII wrapper zárja le — különösen a korai `return` és a kivételes ágak esetén.

34.1.2 std::jthread: automatikus join és beépített stop-állapot

A C++20 `<thread>` fejlécében megjelenő `std::jthread` ugyanazt a szálat reprezentálja, de két többlete van: saját stop-állapotot (`stop_source`) tart, és a destruktora nem hív terminate-t. Pontos szabály: ha a destruktor pillanatában az objektum joinable, előbb `request_stop()`-ot hív, majd `join()`-ol — ebben a sorrendben. A sorrend lényeges: a kérés előbb megy ki, így az együttműködő ciklusnak esélye van kilépni, és a join nem vár feleslegesen.

A `jthread` nem másolható, de mozgatható; mozgató értékadásnál a célobjektum, ha joinable volt, előbb stop kérést kap és joinol. A `request_stop()` a belső stop-forrásra delegál, és `bool`-t ad vissza: igaz, ha ez a hívás *elsőként* kérte a leállást. A `get_stop_token()` a szál stop-tokenjét adja vissza, a `get_stop_source()` egy ugyanarra az állapotra mutató forrást.

Egy fontos konstrukciós megkötés: a C++20 szerinti konstruktor csak akkor vesz részt a túlterhelésfeloldásban, ha a callable a `stop_token` **első paraméterrel** hívható, és a szál belépési pontja ezt a tokent meg is kapja. Ezért a tipikus alak `[](std::stop_token st){ ... }`; a paraméter nélküli lambda közvetlenül nem illeszkedik. Ez verzió- és implementációfüggő részletként fordítási próbával ellenőrizendő a saját GCC/Clang profilodon; a könyv laborjaiban ezt a próbát rögzíteni kell.

Az automatikus join nem csodaszer: ha a szál belépési pontja nem figyeli a tokent, a destruktor örökké blokkolhat. A `jthread` tehát a *kérést* és a *várakozást* teszi kényelmessé, de a *leállást* nem tudja kikényszeríteni.

34.1.3 A stop protokoll: stop_source, stop_token, stop_callback

A mechanizmus három szereplője: a `stop_source` birtokolja a közös stop-állapotot, a `stop_token` egy könnyű, másolható nézet erre az állapotra, a `stop_callback` pedig egy, a kérés pillanatában lefutó művelet. Az állapot addig él, amíg bármely source vagy token hivatkozik rá. A `request_stop()` visszafordíthatatlan: egyszer beállítva nem törölhető, ezért a stop-állapotot nem lehet „újraindítani” — egy szolgáltatás-újraindítás új objektumot igényel.

A `stop_token` default-konstruált példánya nem kapcsolódik állapothoz: `stop_possible()` és `stop_requested()` is hamis. A `stop_possible()` akkor igaz, ha van állapot, és vagy már kérték a leállást, vagy még létezik olyan source, amely kérheti. Ez a különbségtétel a várakozási döntéseknél hasznos: ha a stop nem is lehetséges, a tokent figyelmen kívül lehet hagyni.

A `stop_callback` konstruktora regisztrál; ha a stop már be volt kérve, a callback **azonnal, a konstruáló szálon** fut le. Egyébként a kérés pillanatában a `request_stop()`-ot hívó szálon fut. A destruktor megvárja a futó callback befejeződését. Ebből két gyakorlati szabály következik. Egyrészt a callback legyen rövid, ne blokkoljon, és ne vegyen fel olyan zárat, amelyet a kérést kiadó szál tart — különben holtpont. Másrészt ne joinoljunk callbackből arra a szálra, amelyen a callback futhat: ha a munkás szál saját maga hívja a `request_stop()`-ot, a callback ugyanazon a szálon fut, és az önjoin holtpont. A callbackből kivételt ne engedjünk ki; a callbacket eleve `noexcept` szellemben írjuk.

34.1.4 Kivétel a szál belépési pontján

Ha a szál belépési pontjából kivétel lép ki, a program `std::terminate`-t hív: a kivétel nem terjed át a joinoló szálra, és nem tárolódik sehol. A `jthread` ezen nem változtat. Az explicit átvitel mintája: a munkás szál `try/catch(...)`-ben elkapja a hibát, `std::current_exception()`-nel `std::exception_ptr`-t készít, eltárolja egy közös helyre, majd a joinoló szál `std::rethrow_exception`-nel újradobja.

Az átadás memóriaszempontból rendezett: a szál befejeződése *synchronizes-with* a sikeres `join()` visszatérése, így a join után a munkás szál írásai láthatók. Ezért elég a join után olvasni az `exception_ptr`-t; a join előtti olvasáshoz már szinkronizáció kellene. Ha a hibát nem tároljuk el, a hiba csendben elvész — ez a leggyakoribb „a szál elhalt, de senki nem tudja, miért” hiba.

34.1.5 Capture, élettartam és leállítási sorrend

A referencia-capture nem hosszabbítja meg a hivatkozott objektum élettartamát. A lokális változók megszűnési sorrendje a deklarációs sorrend **fordítottja**; ebből levezethető a szabály. Ha a szál-handle-t az általa használt adatok *után* deklaráljuk, a handle hamarabb semmisül meg, tehát a join még az adatok élettartamában megtörténik. Ha előbb deklaráljuk, az adatok hamarabb szűnnek meg, és a szál a joinig dangling referenciát használ — nem definiált viselkedés.

Egy szolgáltatásban a leállítás dokumentált protokoll legyen, például: (1) új munka elfogadásának leállítása; (2) `request_stop()`; (3) blokkolt várakozások felébresztése (CV notify, self-pipe írás, socket `shutdown`); (4) join vagy a `jthread` destruktora; (5) erőforrások felszabadítása; (6) eredmények és hibák jelentése. A sorrend nem ízlés kérdése: a (3) nélkül a (4) beragadhat, az (5) előtt pedig tilos az fd-t lezárni.

34.1.6 Blokkoló I/O: a stop kérés nem megszakítás

A `stop_token` egy jelző és callback-regiszter, nem megszakítási mechanizmus. Egy `read()`, `recv()` vagy `accept()` hívásban blokkolt szál nem nézi a tokent, amíg a rendszerhívás vissza nem tér; a `request_stop()` + `join` tehát végtelenül várhat. A megoldás platformfüggő, és nem oldható meg tisztán a C++ rétegben. Linuxon jellemző minták: nem blokkoló fd + `poll`/`epoll`, ahol egy eventfd vagy self-pipe írása ébreszt; socketnél a `shutdown(fd, SHUT_RDWR)` a stop-callbackből; vagy timeoutos várakozás. A másik szálból végzett `close()` nem garantáltan ébreszti fel a blokkolt olvasót, és fd-újrahasznosítási versenyt is okozhat, ezért az fd-t a joinig életben kell tartani. A callback itt csak áthidaló: benne ne allokáljunk, ne vegyünk zárat.

34.1.7 Kidolgozott példa: munkás szál vezérelt leállással

Az alábbi program egy munka-sor feldolgozója; a `jthread` deklarációja a sor *után* áll, így a destruktor a sor élettartamában joinol. A C++20-as `condition_variable::wait(lock, stop_token, pred)` overload belsőleg stop-callbacket regisztrál, ezért a `request_stop()` magától felébreszti a várakozót; visszatérési értéke hamis, ha a stop miatt tért vissza, és a predikátum hamis volt.

```
// worker.cpp - C++20
#include <chrono>
#include <condition_variable>
#include <cstdio>
#include <cstdlib>
#include <mutex>
```

```cpp
#include <queue>
#include <stop_token>
#include <thread>

class JobQueue {
public:
    void push(int job) {
        {
            std::lock_guard<std::mutex> lk(m_);
            q_.push(job);
        }
        cv_.notify_one();
    }

    bool pop(int& out, std::stop_token st) {
        std::unique_lock<std::mutex> lk(m_);
        if (!cv_.wait(lk, st, [this] { return !q_.empty(); }))
            return false;               // stop kérve, a sor üres
        out = q_.front();
        q_.pop();
        return true;
    }

private:
    std::mutex m_;
    std::condition_variable_any cv_;   // a stop_tokenes wait csak ezen van
    std::queue<int> q_;
};

int main(int argc, char** argv) {
    const int jobs = argc > 1 ? std::atoi(argv[1]) : 5;   // 0: üres sor
    JobQueue q;                             // előbb az adat...
    std::jthread worker([&q](std::stop_token st) {   // ...aztán a handle
        int job = 0;
        while (q.pop(job, st))
            std::printf("job %d\n", job);
        std::printf("worker: stopped\n");
    });

    for (int i = 0; i < jobs; ++i) q.push(i);
    std::this_thread::sleep_for(std::chrono::milliseconds(50));
    worker.request_stop();                  // kérés; a CV-wait felébred
}                                           // destruktor: request_stop + join

# Példa laborprofil; a konkrét verziót rögzítsd a laborjegyzőkönyvben
g++     -std=c++20 -Wall -Wextra -Wpedantic -pthread -O2 -o worker worker.cpp
clang++ -std=c++20 -Wall -Wextra -Wpedantic -pthread -O2 -o worker worker.cpp
# Adatverseny-keresés (nem találja meg a beragadást, csak a race-eket):
g++     -std=c++20 -g -O1 -fsanitize=thread -pthread -o worker_tsan worker.cpp
./worker          # 5 feladat
./worker 0        # üres sor
```

A `stop_token`-t fogadó `wait` overload csak a `std::condition_variable_any`-n létezik; a `std::condition_variable`-ön nincs, ezért a tag típusa `condition_variable_any`. A `condition_variable`-lel a hívás `no matching function for call to 'std::condition_variable::wait(. std::stop_token&, ...)'` hibával nem fordul. Ha valamiért mégis `condition_variable` kell, külön `std::stop_callback`-alapú ébresztési protokollt kell írni, amely a stop kéréskor a mutex alatt jelez és `notify_all`-t hív.

Buildjegyzőkönyv (`lab-gcc` profil). A program figyelmeztetés nélkül fordult. Öt feladattal `job 0 … job 4`, majd `worker: stopped` kimenettel, üres sorral (`./worker 0`) csak `worker: stopped` kimenettel állt le; mindkét esetben a `jthread` destruktora bevárta a szálat. TSan (`setarch -R`) alatt mindkét változat figyelmeztetés nélkül futott. A Clang-sort nem ellenőriztük.

A példa leállítási szemantikája „drain”: ha a stop a sor kiürülése előtt érkezik, a predikátum igaz marad, és a szál még feldolgozza a maradékot. Ez tudatos választás; a „drop” változatban a ciklusfeltétel a stopot előbb ellenőrizné. A kettő közötti választást a protokollban dokumentálni kell.

34.1.8 Gyakori hibák és javításuk

- **A `jthread` a használt adat előtt deklarálva.** A destruktori join a már megszűnt objektumra fut. Javítás: az adat deklarációja előbb, a szál-handle utána.
- **`request_stop()` blokkoló I/O-ra várva.** A join beragad; a stop csak jelző. Javítás: ébresztő mechanizmus (self-pipe, eventfd, `shutdown`), vagy nem blokkoló I/O.
- **Stop-callback, amely zárat vesz fel, miközben a hívó tartja.** Holtpont. Javítás: a callback csak jelezzen (atomic flag, CV notify, egy byte írása).
- **Önjoin callbackből.** A `request_stop()`-ot a munkás szál hívja, a callback ugyanott fut, a join önmagára vár. Javítás: a stop kérését kívülről indítjuk, a join a `jthread` destruktorára bízzuk.
- **Kivétel a szál belépési pontján.** `std::terminate`, illetve elveszett hiba. Javítás: `exception_ptr` + `rethrow_exception`.
- **`detach()` referencia-capture-rel.** Dangling. Javítás: join, vagy érték szerinti capture, vagy `shared_ptr` — utóbbinál azonban a pointee thread safetyje nem automatikus, ezt itt nem tárgyaljuk.
- **Kétszeres join vagy join detach után.** `std::system_error`. Javítás: `joinable()` ellenőrzés vagy RAII.

34.1.9 Összefoglaló

A `std::thread` egy joinable erőforrás; a destruktora terminate-t hív, ha nem rendeztük a sorsát. A `std::jthread` ezt úgy oldja meg, hogy destruktorában előbb stopot kér, majd joinol, és a callable `stop_token` első paramétert kap. A stop protokoll (`stop_source`, `stop_token`, `stop_callback`) egy visszafordíthatatlan jelző, amelynek callbackje a kérést kiadó szálon fut — rövidnek, zárazár nélkülinek kell lennie. A stop kérés nem megszakítás: blokkoló I/O-t platformspecifikus ébresztéssel kell kiváltani. A capture-élettartamot és a leállítási sorrendet a deklarációs sorrend és egy dokumentált protokoll rögzíti; a szál belépési pontjából kivételt nem engedünk ki, hanem explicit módon átvisszük.

34.1.10 Feladatok

1. **Átalakítás.** Alakítsd át a `std::thread`-es mintát `std::jthread`-té: a munkás ciklus `stop_requested()`-et figyel, a fő szál `request_stop()`-ot hív, a join a destruktorra marad. Indokold, miért nem kell külön `join()`.
2. **Leállítási sorrend.** Adott egy `Stats` objektum (`std::vector` + `std::mutex`) és egy `std::jthread`, amely referenciával capture-öli, és végtelen ciklusban írja. Nevezd meg a két hibát (élettartam és leállás), és javítsd mindkettőt.
3. **Blokkoló I/O.** Egy szál blokkolt `read()`-ban egy pipe-on; a `request_stop()` után a join nem tér vissza. Tervezz kooperatív leállítást, és indokold, miért nem elég a token önmagában.

34.1.11 Megoldások

1. megoldás. A `jthread` destruktora joinable állapotban `request_stop()`-ot majd `join()`-t hív, ezért explicit join csak akkor kell, ha a joinolás pillanatát (például egy jelentés előtt) kézben akarjuk tartani. Ellenőrzési feltétel: a program a stop kérése után „stopped” üzenettel, terminate nélkül érjen véget; a ciklusfeltétel tartalmazza a `st.stop_requested()` ellenőrzést.

```
std::jthread t([](std::stop_token st) {
    while (!st.stop_requested()) { /* munka */ }
});
t.request_stop();
```

2. megoldás. Első hiba: a `std::jthread` a `Stats` *előtt* van deklarálva, ezért a `Stats` hamarabb szűnik meg, mint ahogy a join megtörténne — dangling. Második hiba: a ciklus nem figyeli a stopot, így a destruktori join végtelen. Javítás: `Stats` előbb, `jthread` utána, és a ciklus `st.stop_requested()`-et ellenőriz. Ellenőrzési feltétel: a munkás írásai a `Stats` élettartamán belül maradnak, és a program a stop után kilép; TSan-futtatás nem jelez adatversenyt a `Stats` mezőin.

3. megoldás. A pipe olvasó végére blokkolt `read()` nem figyeli a tokent. Megoldás: `poll()` a pipe olvasó végén és az adat-fd-n; egy `std::stop_callback` a stop pillanatában egy byte-ot ír a pipe író végére, amire a `poll()` visszatér, és a ciklus kilép. Az fd-eknek a joinig életben kell maradniuk, a callback nem allokál és nem vesz fel zárat. A `close()` másik szálból nem megbízható ébresztés, és fd-újrahasznosítási versenyt okozhat. Ellenőrzési feltétel: a join a stop után korlátos időn belül visszatér; a self-pipe írása nem blokkol (egy byte); watchdog-teszt igazolja a kilépést.

34.2 RAII lockok, condition variable és wait predicate

A Mutex, condition variable és producer–consumer (15.3) alfejezet C változatában a bounded queue minden várakozási pontja kézi `pthread_mutex_unlock`-kal zárult, és minden korai `return` előtt végig kellett gondolni, hogy a zár felszabadul-e. A szerződés helyes volt, de törékeny: egy új hibaleágazás vagy egy hosszú `goto cleanup` csendben holtpontot hozott. A C++ nem „okosabb” mutexet ad ugyanerre a problémára, hanem a zár birtoklását egy objektum élettartamához köti: a zár a konstruktorban megszerzi, a destruktorban elengedi, a kritikus szakasz határa pedig pontosan az az objektum, amelynek a hatóköre véget ér. Ez ugyanaz az RAII-minta, amit a RAII wrapper file descriptorhoz és lockhoz (21.2) alfejezetben file descriptorra és lockra már felállítottunk; itt a hangsúly azon van, hogy a zár *típusa* és a *várakozás* feltétele is a művelet szerződéséből következzen, ne szokásból.

A C++ konkurencia-eszközeit a C11 atomics és a POSIX szálkezelés szemantikájára építjük; a szál életciklusát és a kooperatív leállítást a thread, jthread és kooperatív cancellation (34.1) alfejezet rögzítette, a happens-before és a data race definícióját nem ismételjük meg. Az itteni cél: ugyanazt a bounded queue invariánst RAII lockokkal megvalósítani, és indokolni tudni, mikor melyik locktípus és milyen várakozási minta a helyes. A fejezethez ebben az egységben nem tartozik ellenőrzött forrásjegyzet, ezért a szabvány-paragrafusokra (a C++20 munkatervezet `[thread.mutex]`, `[thread.condition]` fejezeteire) itt nem hivatkozom bizonyítékként: ezeket a saját C++20 példányodon ellenőrizd. A kódok a leírásban nem kerültek lefuttatásra; a viselkedésre vonatkozó állítások a szerződésből következnek, nem mérésből.

34.2.1 A zár mint erőforrás: melyik locktípus melyik szerződéshez

A `std::mutex` nem rekurzív, nem másolható és nem mozgatható; ugyanabban a szálban kétszer megszerezni nem definiált viselkedés, nem pedig holtpont, amit egy `recursive_mutex` „megjavítana”. A kritikus szakasz akkor helyes, ha a zár alatt olvasott és írt állapot pontosan azonos halmaz, és minden más szál, amely ezt az állapotot érinti, ugyanezt a zárat tartja.

A locktípus megválasztása három kérdésből következik. Ha csak egy zárat kell egy hatókörre megszerezni, és nem kell kézzel elengedni, akkor `std::lock_guard` a legkisebb és legkevésbé félreérthető választás. Ha a zárnak túl kell élnie egy hatókört, mozgatni kell, `try_lock`-ot vagy időzített próbát kell indítani, vagy a `condition_variable::wait`-nek át kell adni, akkor `std::unique_lock` kell — ez az egyetlen standard locktípus, amely `unlock()`/`lock()` párost kínál, és `owns_lock()`-nal megmondja, hogy éppen tart-e zárat. Ha egyszerre több mutexet kell megszerezni, `std::scoped_lock` (C++17) a helyes eszköz: variadikus, és a szabvány által előírt holtpont-elkerülő algoritmussal (próbálkozás, visszalépés, újrapróbálás) szerzi meg mindet, így a megszerzési sorrend nem a hívó felelőssége.

A `condition_variable::wait` első paramétere `std::unique_lock<std::mutex>&`, tehát `lock_guard`-dal nem hívható meg: a várakozásnak atomikusan el kell engednie, majd vissza kell szereznie a zárat. (`std::condition_variable_any` bármilyen BasicLockable lockkal működik, de a `mutex` + `condition_variable` páros a szokásos, olcsóbb eset.)

A kritikus szakasz minimalitása nem esztétika, hanem szerződés: zárat tartva ne hívj felhasználói callbacket, ne végezz bizonytalan költségű allokációt és ne végezz I/O-t, mert minden ilyen pont a zár tartási idejét más szálak számára kiszámíthatatlanná teszi.

34.2.1.1 Lock ordering és a holtpont elkerülése

A holtpont négy együttes feltétele (kölcsönös kizárás, tartás-és-várakozás, nincs erőszakos elvétel, körkörös várakozás) közül a gyakorlatban a körkörös várakozást lehet a legjobban megszüntetni: vagy globális sorrendet adsz a mutexeknek, vagy `std::scoped_lock`-kal egyszerre szerzed meg őket. A sorrend legyen dokumentált és a kódban kikényszeríthető, ne a hívási véletlenen múljék.

```
#include <mutex>

struct Account {
    std::mutex m;           // csak illusztráció: éles kódban privát, lock-ot adó
     ↪ tagfüggvénnyel
    long long balance = 0;
};

void transfer(Account& from, Account& to, long long amount)
```

```
{
    std::scoped_lock lock(from.m, to.m);    // holtpont-elkerülő megszerzés
    from.balance -= amount;
    to.balance += amount;
}
```

Ha a két zárat nem lehet egyszerre megszerezni (mert a kettő között olyan művelet van, ami nem fér a kritikus szakaszba), akkor a sorrend rögzítése a helyes út. A `std::lock` és a `std::defer_lock`/`std::adopt_lock` kombinációja ugyanezt teszi kézi `unique_lock`-okkal. Kétely esetén a legjobb javítás nem egy harmadik mutex, hanem a struktúra átalakítása: egy zár alá vont állapot, egyetlen kritikus szakasz.

34.2.2 A bounded queue újraírása RAII lockokkal

Az invariáns változatlan a C változathoz képest: $0 \le$ `q_.size()` $\le$ `capacity_`, a `head`/`tail` mutatók helyett itt a `std::deque` hossza hordozza ugyanezt, és a `closed_` jelző azt jelenti, hogy a további `push` már nem sikerülhet, a `pop` pedig a maradék elemek kiürülése után `false`/`closed` eredményt ad. A mutex egyszerre védi a `q_`-t és a `closed_`-t — ez a „védett predicate” alapja.

```
// queue_raii.cpp -- C++20, -pthread
#include <chrono>
#include <condition_variable>
#include <cstddef>
#include <deque>
#include <iostream>
#include <mutex>
#include <optional>
#include <stdexcept>
#include <thread>
#include <utility>
#include <vector>

enum class PopStatus { value, timeout, closed };

template <class T>
class BoundedQueue {
public:
    explicit BoundedQueue(std::size_t capacity) : capacity_(capacity)
    {
        if (capacity == 0)
            throw std::invalid_argument("BoundedQueue: capacity == 0");
    }

    BoundedQueue(const BoundedQueue&) = delete;
    BoundedQueue& operator=(const BoundedQueue&) = delete;

    // false: a queue zárt, az elem nem került be
    bool push(T value)
    {
        {
            std::unique_lock lock(m_);
```

```
        not_full_.wait(lock, [this] { return closed_ || q_.size() <
         ↪  capacity_; });
        if (closed_)
            return false;
        q_.push_back(std::move(value));
    }
    not_empty_.notify_one();     // a záron kívül: a felébredő ne ütközzön a
     ↪  zárral
    return true;
}

std::optional<T> pop()
{
    std::unique_lock lock(m_);
    not_empty_.wait(lock, [this] { return closed_ || !q_.empty(); });
    if (q_.empty())
        return std::nullopt;     // closed_ és kiürült
    T value = std::move(q_.front());
    q_.pop_front();
    lock.unlock();
    not_full_.notify_one();
    return value;
}

PopStatus pop_for(std::chrono::milliseconds timeout, T& out)
{
    std::unique_lock lock(m_);
    const bool ready = not_empty_.wait_for(lock, timeout,
        [this] { return closed_ || !q_.empty(); });
    if (!ready)
        return PopStatus::timeout;
    if (q_.empty())
        return PopStatus::closed;
    out = std::move(q_.front());
    q_.pop_front();
    lock.unlock();
    not_full_.notify_one();
    return PopStatus::value;
}

void close()
{
    {
        std::lock_guard lock(m_);
        closed_ = true;          // állapotváltozás a zár alatt
    }
    not_empty_.notify_all();     // minden várakozó lássa a leállítást
    not_full_.notify_all();
}

std::size_t size() const
{
```

```
        std::lock_guard lock(m_);
        return q_.size();
    }

private:
    mutable std::mutex m_;
    std::condition_variable not_empty_;
    std::condition_variable not_full_;
    std::deque<T> q_;
    std::size_t capacity_;
    bool closed_ = false;
};

int main()
{
    BoundedQueue<int> q(2);
    std::vector<std::thread> producers;
    for (int p = 0; p < 2; ++p) {
        producers.emplace_back([&q, p] {
            for (int i = 0; i < 3; ++i)
                q.push(p * 100 + i);
        });
    }
    std::size_t received = 0;
    while (received < 6) {
        if (auto v = q.pop())
            ++received;
    }
    for (auto& t : producers)
        t.join();
    std::cout << "received=" << received << '\n';
}
```

A buildparancs a laborprofilhoz kötött, konkrét compilerrel:

```
g++     -std=c++20 -O2 -Wall -Wextra -Wpedantic -pthread queue_raii.cpp -o
 ↪  queue_raii
clang++ -std=c++20 -O2 -Wall -Wextra -Wpedantic -pthread queue_raii.cpp -o
 ↪  queue_raii
```

A `-pthread` nem elhagyható kozmetika: a fordító-illesztőnek jelezni kell, hogy a szálkezelő definíciók és a `_REENTRANT`-szerű makrók a helyes konfigurációban legyenek. A `main` szándékosan nem hív `close()`-t, mert a példa fix elemszámot drainel; a `close()` a leállítási út, amelyet a feladatok gyakorolnak. A `size() const` tagfüggvény, ezért a mutex `mutable` — ez a szokásos, de tudatos kompromisszum: a logikai állapotot nem változtatja, a fizikai zárállapotot igen.

34.2.3 Predicate loop, spurious wakeup és a notify szerződése

A `cv.wait(lock, pred)` jelentése pontosan: `while (!pred()) cv.wait(lock);`. A ciklus nem védekező kód, hanem a szerződés része, három okból. Egyrészt a szabvány megengedi a látszólagos (spurious) ébredést, tehát a `wait` visszatérhet úgy, hogy senki nem notifikált. Másrészt a felébredés és a zár visszaszerzése között egy harmadik szál megelőzhet: elviheti az elemet, betöltheti a

helyet, lezárhatja a queue-t. Harmadrészt a két külön condition variable (`not_empty_`, `not_full_`) közös mutexen osztozik, így a `notify_one` címzettje nem garantáltan az a szál, amelyiknek az állapotra szüksége van. A predikátumot mindig a zár alatt kell kiértékelni, mert az a `m_` által védett állapotot olvassa.

A notify és az állapotváltozás sorrendje fordítottja a naiv intuíciónak: először állapotot módosítasz a zár alatt, aztán notifikálsz. Ha a notify megelőzi az állapotváltozást, a felébredő szál hamis predikátumot lát, visszaalszik, és a későbbi állapotváltozásról már senki nem szól — ez az elveszett ébresztés (lost wakeup). A notify zár alatt és zár után is helyes; a zár utáni változat azért jobb, mert a felébresztett szál nem ütközik azonnal a még tartott zárral („hurry up and wait”). A `notify_one` akkor elég, ha minden notify pontosan egy kiszolgálható elemet vagy egy equivalent waitert jelent; leállításnál és több várakozót kiszolgáló állapotváltozásnál `notify_all` kell.

A predikátumos `wait_for` visszatérési értéke a predikátum értéke, nem a timeout ténye: `false` azt jelenti, hogy a predikátum a lejáratig hamis maradt, `true` pedig azt, hogy igaz — de ez utóbbi nem mondja meg, hogy közben lejárt-e az idő. Predikátum nélküli változatnál `cv_status::no_timeout` esetén is újra kell ellenőrizni a feltételt. Az időzítés clock domainje számít: a `wait_for` a `std::chrono::steady_clock` szerint mér, a `wait_until` viszont a megadott Clockot használja, ezért `system_clock`-kal átadott abszolút határidőt egy NTP-ugrás eltorzíthat. Timeout esetén soha ne következtess arra, hogy a queue üres — csak arra, hogy a predikátum addig hamis volt. A `pop_for` fenti változata ezért ad külön `PopStatus::timeout` és `PopStatus::closed` értéket: az `std::optional`-lal való „nullopt = timeout vagy closed” összevonás csendben elveszíti a leállítási információt.

34.2.4 shared_mutex: mikor éri meg, és mikor nem

A `std::shared_mutex` (C++17) olvasó–író zárat ad: `std::shared_lock` több olvasónak egyszerre, `std::lock_guard`/`std::unique_lock` egy írónak. A workload-feltétel szigorú: hosszú, tiszta olvasási kritikus szakaszok, magas olvasási arány és ritka írás. Rövid kritikus szakaszoknál a belső könyvelés és a közös cache-sor írása miatt könnyen lassabb a sima `std::mutex`-nál, és az írók kiéheztetésének politikája nem specifikált, tehát terhelésfüggő. Nincs olvasó→író upgrade: a `shared_lock` elengedése és új `unique_lock` felvétele között elvész az atomicitás, ezért ilyen esetben a kritikus szakaszt kell kettéválasztani vagy más mintát (pillanatkép, copy-on-write) választani. Egyszerű számlálóhoz továbbra is `std::atomic` kell (C++ atomics, atomic_ref és publication (34.3)); a `shared_mutex` nem helyettesíti.

34.2.5 Gyakori hibák

A `wait` `if`-fel körbevéve — spurious wakeup vagy konkurens fogyasztó esetén hamis predikátummal lép tovább. A `lock_guard` átadása `wait`-nek — nem fordul le, mert a `wait` `unique_lock`-ot vár. Notify állapotváltozás előtt — elveszett ébresztés. Két mutex megszerzése ad hoc sorrendben — holtpont. `recursive_mutex` bevezetése a tünet kezelésére. `shared_mutex` apró kritikus szakaszra, mérés nélkül. A `wait_for` `false` eredményének „üres queue”-ként való kezelése. A `close()` után elmaradó `notify_all` — a várakozók örökre a zár alatt maradnak.

34.2.6 Feladatok

1. **Átalakítás és indoklás.** A Mutex, condition variable és producer–consumer (15.3) C producer–consumer változatát alakítsd át a fenti osztályra. Sorold fel az összes zárfelvételi pontot és korai visszatérési utat, majd indokold pontonként, hogy `lock_guard`, `unique_lock` vagy `scoped_lock` a helyes választás, és hol változott meg a hibaágak kezelése.

2. **Kódolás: leállítás és időzített várakozás.** Egészítsd ki a `main`-t úgy, hogy egy koordinátor szál a termelők `join`-ja után `close()`-t hívjon, a fogyasztó pedig `pop()`-pal draineljen `std::nullopt`-ig. Majd írj három tesztesetet: (a) `pop_for` timeout üres, nyitott queue-n; (b) `pop_for` zárt, de még nem üres queue-n; (c) `pop_for` zárt és üres queue-n. Mindegyikhez add meg a várt `PopStatus`-t.

3. **Diagnosztika.** Az alábbi kódrészlet két hibát tartalmaz. Keresd meg őket, magyarázd el a kiváltott viselkedést (data race, holtpont vagy elveszett ébresztés), és javítsd a minimális változtatással.

```
// hibás tanítási példa -- nem használd
void consumer(BoundedQueue<int>& q)
{
    std::unique_lock lock(q.m_);          // feltételezzük, hogy friend
    if (!q.q_.empty()) {                  // 1. hiba helye
        q.q_.pop_front();
    }
}
void producer(BoundedQueue<int>& q, int v)
{
    q.not_empty_.notify_one();            // 2. hiba helye
    std::lock_guard lock(q.m_);
    q.q_.push_back(v);
}
```

34.2.7 Megoldások

1. megoldás. A C változatban a `pthread_mutex_lock`/`unlock` párok száma egyenlő a belépési pontokéval; RAII alatt minden belépési pont egy lockobjektum élettartama. A queue mindhárom művelete (`push`, `pop`, `close`) `unique_lock`-ot kap, mert a várakozás és a kézi `unlock()` a notify előtt mindkettőt igényli; a `size()` és a `close()` állapotfrissítése `lock_guard`-dal elég, mert ott nincs várakozás és nincs korai elengedés. A korai `return`-ök (`closed_`, üres queue) többé nem igényelnek cleanup ágat: a lock destruktora a visszatérési úton is lefut, ez a lényegi nyereség. Ellenőrzési feltétel: a `push` minden `false` visszatérése előtt a `not_full_` várakozás megtörtént, és a `closed_` beállítása után egyetlen `push` sem módosítja a `q_`-t.

2. megoldás. A koordinátor alakja:

```
std::thread closer([&q, &producers] {
    for (auto& t : producers) t.join();
    q.close();
});
std::size_t received = 0;
while (auto v = q.pop()) ++received;   // nullopt: zárt és kiürült
closer.join();
```

Tesztesetek: (a) nyitott, üres queue-n `pop_for(50ms, out)` → `PopStatus::timeout`, és a predikátum hamis maradt; (b) zárt, egy elemet tartalmazó queue-n → `PopStatus::value`, az elem kikerül; (c) zárt, üres queue-n → `PopStatus::closed`, és a hívás nem blokkol, mert a predikátum `closed_` miatt igaz. Ellenőrzési feltétel: egyik esetben sem marad bent zár, és a `not_full_` notify minden sikeres kivétel után megtörténik.

3. megoldás. Az első hiba a `not_empty_` notify helye: a termelő a zár és az állapotváltozás *előtt* notifikál, így a fogyasztó felébredhet, hamis predikátumot lát, visszaalszik, és az ezt követő

`push_back` után már senki nem szól neki — elveszett ébresztés, amely a fogyasztót véglegesen megakasztja. A javítás: a `push_back` a zár alatt történjen, a notify utána. A második hiba a `consumer` `if`-je: a várakozás helyett egyszeri ellenőrzés, amely (a) nem vár semmire, (b) nem kezeli a `closed_` állapotot, és (c) ha `wait`-t tennénk bele `if`-fel, spurious wakeupnál hamis predikátummal lépne tovább. A helyes alak a `wait(lock, [&]{ return closed_ || !q_.empty(); })` predikátumos hurok, utána a `closed_`/üresség ellenőrzése. Megjegyzés: a példa `friend`-del hozzáfér a privát tagokhoz — ez maga is szerződésszegés, mert a zár és az állapot védelme szétválik; éles kódban a queue adja a zár alatt futó műveletet. Ellenőrzési feltétel: TSan-futtatás (ASan, UBSan, MSan, TSan és Valgrind (16.2) eszköztár) nem jelez adatversenyt, és a fogyasztó zárt queue-n `std::nullopt`-tal tér vissza, nem blokkol.

34.2.8 Összefoglaló

Az RAII lock nem kényelmi eszköz, hanem szerződés: a kritikus szakasz az objektum élettartama, a zár típusa pedig a műveletből következik — `lock_guard` az egyszerű esethez, `unique_lock` a várakozáshoz és a kézi elengedéshez, `scoped_lock` a több mutexhez. A condition variable helyes használata három elem együttese: a védett állapot a mutex alatt változik, a várakozás mindig predikátumos ciklus (spurious wakeup és konkurens fogyasztó miatt), a notify pedig az állapotváltozás után, lehetőleg a záron kívül történik. A leállítás külön predikátumfeltétel (`closed_`) és `notify_all`, nem mellékhatás. Az időzített várakozásnál a `steady_clock` a domain, és a timeout nem bizonyíték semmire. A `shared_mutex` csak mért, olvasásdominált, hosszú kritikus szakaszoknál térül meg.

34.3 C++ atomics, atomic_ref és publication

A C atomics fejezetben (C atomics és memory order (15.4)) rögzítettük a `_Atomic` típusok, a `memory_order` és a happens-before alapjait. Itt ugyanazt a szerződést visszük át a C++ típusos `std::atomic` API-ra, de két új kérdéssel együtt: az atomikus objektum saját élettartamával és az általa publikált erőforrás élettartamával. Az atomikus művelet ugyanis csak az adott objektumhoz való hozzáférést teszi versenymentessé; nem tartja életben a mögötte lévő objektumot, és nem teszi automatikusan szálbiztossá a pointee állapotát. A fejezet C++20-at feltételez; a lock-free és a `wait`/`notify` konkrét implementációja target-, ABI- és standard library-függő, ezért a laborban ellenőrizni kell.

34.3.1 Az atomikus művelet és a lifetime szerződése

Atomikus műveletről akkor beszélünk, ha egy objektumhoz több szálon történő hozzáférés esetén a művelet oszthatatlannak látszik, és a nyelvi memóriamodell szerint nem keletkezik adatverseny. Ez nem azt jelenti, hogy a művelet lock-free, wait-free vagy gyors: a `std::atomic<T>` implementálható belső zárral is. Azt sem jelenti, hogy a `T` típusú objektum minden mezője atomikusan elérhető; a `std::atomic<T>` egy külön atomikus objektum, amelynek mérete és igazítása eltérhet a `T`-étől.

A `std::atomic<T>` primary templateje olyan `T`-vel példányosítható, amely trivially copyable, és amely megfelel a másolhatósági követelményeknek. Integrál, pointer és lebegőpontos `T` esetén specializációk adnak `fetch_add`, `fetch_sub`, illetve integrál esetén bitműveleti RMW-ket is. Nem trivially copyable típusra, például `std::string`-re a `std::atomic<std::string>` nem használható. A `std::atomic<T>` nem másolható és nem mozgatható; a szálak közötti átadása referencián vagy pointeren keresztül történik, és az atomikus objektumnak is élnie kell addig, amíg bármely szál hozzáférhet.

A lock-free elérhetőség nem nyelvi garancia. A `is_lock_free()` futásidejű lekérdezés, a `is_always_lock_free` pedig fordítási idejű konstans; utóbbi azt mondja meg, hogy az adott `atomic<T>` specializáció minden esetben lock-free-e. Egy `atomic<int>` x86-64 System V ABI-n jellemzően lock-free, de ezt a szabvány nem garantálja, és nagyobb `T` esetén a implementáció zárat használhat. A `std::atomic_flag` az egyetlen olyan atomikus típus, amelynek lock-free voltát a szabvány garantálja.

Az atomikus objektum élettartama külön szerződés. Ha egy szál még hozzáfér egy `std::atomic<T>` objektumhoz, miközben egy másik szál elpusztítja, az nem definiált viselkedés. A szálak joinolása vagy más, egyértelmű élettartam-kapcsolat nélkül az atomikus zászló sem véd meg az objektum eltűnése ellen.

34.3.2 Load, store, RMW, CAS és az ordering

A `std::atomic<T>` alapműveletei a `load`, `store`, `exchange`, `compare_exchange_weak`, `compare_exchange_strong`, valamint a támogatott `T` esetén a `fetch_add`, `fetch_sub`, `fetch_and`, `fetch_or`, `fetch_xor`. A `load` és a `store` egyetlen atomikus olvasás, illetve írás. Az `exchange` atomikusan kicserél. A `compare_exchange` feltételes RMW: csak akkor ír, ha az aktuális érték megegyezik a várt értékkel; ellenkező esetben a várt értéket az aktuális értékre frissíti. A `weak` változat megengedhet spurius hibát, ezért loopban kell használni; a `strong` nem ad spurius hibát, de drágább lehet.

Az ordering nem dekoráció, hanem a láthatóság szerződése. A `relaxed` csak atomikusságot ad, szálak közötti rendezést nem. A pontos szabály a *happens-before* reláción keresztül szól ([intro.races]): ha egy szál egy atomikus objektumra `release` műveletet (store vagy RMW) hajt végre, és egy másik szál `acquire` művelete (load vagy RMW) **ebből a release műveletből vagy annak release sequence-éből olvas értéket**, akkor a release *synchronizes-with* az acquire-rel. Ekkor minden, a release előtt (az első szálon) történt írás happens-before az acquire utáni olvasásoknak, tehát látható. Az `acquire` név önmagában nem tesz láthatóvá bármely szál bármely korábbi írását: csak a párosított release-ét, és csak akkor, ha az acquire ténylegesen azt az értéket olvassa. Másodlagos, implementációs intuícióként: az acquire után következő hozzáférések nem rendeződnek a művelet elé, a release előttiek nem kerülnek mögé. Az `acq_rel` csak RMW-n értelmes, és egyszerre ad acquire és release hatást. A `seq_cst` ezeken felül egyetlen teljes sorrendet ad az összes seq_cst műveletre; ez a default, de nem mindig szükséges.

Ordering	Hol használható	Mit ad
`relaxed`	load, store, RMW	csak atomikusság
`acquire`	load, RMW	a későbbi hozzáférések nem kerülnek elé
`release`	store, RMW	a korábbi hozzáférések nem kerülnek mögé
`acq_rel`	RMW	acquire + release
`seq_cst`	mind	teljes sorrend a seq_cst műveletekre

Érvénytelen ordering kombináció például a `release` load vagy a `release` failure ordering egy `compare_exchange`-ben; az ilyen előfeltétel megsértése nem definiált viselkedés. A C++20 `compare_exchange` előfeltétele csak annyi, hogy a failure ordering ne legyen `release` vagy `acq_rel`. A régebbi szabály, amely szerint a failure „nem lehet erősebb” a successnél, a C++11/14-ben és a C11/C17-ben volt érvényes; a C++17 (P0418R2) eltörölte. Hordozható, régebbi fordítókat is kiszolgáló kódban konzervatív *ajánlás*, hogy a failure ne legyen erősebb a successnél. **Mérés** (`lab-gcc`): a C++20-ban szabályos `compare_exchange_strong(e, 1, relaxed, acquire)` a GCC 13.3-mal lefordult és helyesen futott, de `-Winvalid-memory-model` figyelmeztetést adott a régi szabály szerint; a tiltott `release` failure ordering-re `invalid failure memory model`

`'memory_order_release'` figyelmeztetést adott, a Clang 22 `memory order argument to atomic operation is invalid` figyelmeztetést — a Clang a szabályos kombinációra nem jelzett.

Relaxed számlálóra tipikus példa a statisztikai számlálás. A következő kódban a `fetch_add` atomikus, ezért nem vesznek el növelések, de a `relaxed` nem publikál más adatot:

```
#include <atomic>
#include <thread>
#include <iostream>

std::atomic<unsigned long> hits{0};

void worker() {
    for (int i = 0; i < 1000; ++i) {
        hits.fetch_add(1, std::memory_order_relaxed);
    }
}

int main() {
    std::thread a(worker);
    std::thread b(worker);
    a.join();
    b.join();
    std::cout << hits.load(std::memory_order_relaxed) << '\n';
}
```

A `join()` után a fő szál a `join` szinkronizációs hatása miatt látja a szálak befejeződését; a `relaxed` itt nem a join helyett működik, hanem a számláló növelésének atomikusságát adja. Ha a számláló mellett más, nem atomikus adatot is publikálnánk, a `relaxed` nem lenne elég.

A CAS-loop mintája jól mutatja a `compare_exchange_weak` használatát. Az `expected` értékét a művelet hiba esetén frissíti, ezért minden iterációban az aktuális értékkel kell dolgozni:

```
#include <atomic>

std::atomic<int> maximum{0};

void update_max(int candidate) {
    int old = maximum.load(std::memory_order_relaxed);
    while (candidate > old &&
           !maximum.compare_exchange_weak(
               old, candidate,
               std::memory_order_release,
               std::memory_order_relaxed)) {
        // old már az aktuális érték; újra próbáljuk
    }
}
```

Ha a CAS success orderingje `release`, akkor a `candidate` előtti írások a sikeres store-hoz kapcsolódnak; ha csak a maximum érték érdekes, `relaxed` is elég lehet. A `weak` spurius hibája miatt a loop kötelező; a `strong` használata nem menti meg a versenyt, csak a spurius hibát zárja ki.

34.3.3 Publication release/acquire szerződéssel

A leggyakoribb minta a publisher–consumer kapcsolat: az egyik szál előkészít egy adatot, majd egy atomikus jelzővel vagy pointerrel publikálja; a másik szál acquire-tel olvassa a jelzőt, és utána olvassa az adatot. A release store és az acquire load akkor szinkronizál, ha ugyanarra az atomikus objektumra történnek, és az acquire load a release store által írt értéket, illetve a release sequence megfelelő későbbi értékét látja. Ekkor a release store előtti írások happens-before kapcsolatba kerülnek az acquire load utáni olvasásokkal.

A levezetés lépései: a publisher a payload írását sequenced-before helyzetbe hozza a release store-hoz; a release store és a consumer acquire loadja között synchronizes-with él; a consumer acquire loadja sequenced-before a payload olvasásához; a három viszony tranzitivitásából a payload írása happens-before a payload olvasásához. Ezért nincs adatverseny, és a consumer a publikált értéket látja.

```
#include <atomic>
#include <cassert>
#include <thread>

std::atomic<bool> ready{false};
int payload = 0;

void publisher() {
    payload = 42;                                   // nem atomikus, de a store
 ↪  előtt van
    ready.store(true, std::memory_order_release);
}

void consumer() {
    while (!ready.load(std::memory_order_acquire)) {
        // várakozás
    }
    assert(payload == 42);
}

int main() {
    std::thread p(publisher);
    std::thread c(consumer);
    p.join();
    c.join();
}
```

Ha a `ready.store` `relaxed`, és a `ready.load` is `relaxed`, akkor a `payload` írása és olvasása között nincs happens-before; a két szál egy időben ér hozzá egy nem atomikus objektumhoz, ami adatverseny és nem definiált viselkedés. A `seq_cst` mindkét oldalon szintén helyes, de erősebb a szükségesnél. A release/acquire itt a minimális, indokolható választás.

Fontos feltétel, hogy a consumer valóban a publisher értékét lássa. Ha a load egy korábbi értéket olvas, abból az adott store-ra nézve nincs synchronizes-with. A `while` ciklus ezért addig olvas, amíg a publikált értéket meg nem látja. Több publisher esetén a release sequence és az RMW-k szerepe bonyolultabb; ilyenkor vagy `seq_cst`-t, vagy külön szinkronizációs protokollt használunk.

Ugyanez a minta pointerrel is működik:

```
struct Config {
    int timeout;
    int retries;
};

std::atomic<Config*> current{nullptr};

void publish(Config* c) {
    current.store(c, std::memory_order_release);
}

Config* snapshot() {
    return current.load(std::memory_order_acquire);
}
```

A `publish` előtt inicializált `Config` mezői a release store miatt válnak láthatóvá annak, aki a pointert acquire-tel olvassa. A pointer atomikussága azonban nem terjeszti ki a `Config` élettartamát; erről a következőkben külön szólunk.

34.3.4 atomic_ref: atomikus hozzáférés létező objektumhoz

A `std::atomic_ref<T>` C++20-as eszköz arra az esetre, amikor egy már létező `T` objektumot szeretnénk atomikusan elérni anélkül, hogy `std::atomic<T>` burkolóba tennénk. Ilyen lehet egy tömb eleme, egy C struct mezője, memóriába mapolt adat vagy egy olyan interfész, amely `T&`-et ad. Az `atomic_ref` nem tulajdonol, nem hoz létre új tárolót, és nem hosszabbítja meg a hivatkozott objektum élettartamát.

Az `atomic_ref<T>` követelményei: `T` trivially copyable; a hivatkozott objektum igazítása legalább `std::atomic_ref<T>::required_alignment`; a hivatkozott objektum élettartama hosszabb az `atomic_ref` élettartamánál; és amíg az `atomic_ref` él, a hivatkozott objektumot csak `atomic_ref`-en keresztül szabad elérni. Ez az utolsó pont az exkluzív hozzáférés szerződése, és szigorúbb az általános adatverseny-szabálynál: amíg bármely `atomic_ref` él, a nem atomikus hozzáférés *akkor is* tilos (nem definiált viselkedés), ha csak olvasás, és semmilyen írással nem ütközne. Az atomic_ref élettartamán kívül viszont a szokásos szabály érvényes: egymással nem ütköző, csak olvasó hozzáférések szinkronizáció nélkül is megengedettek. Több `atomic_ref` ugyanarra az objektumra megengedett.

```
// atomic_ref_cells.cpp - C++20: minden atomic_ref-cél elem igazítása garantált
#include <array>
#include <atomic>
#include <cassert>
#include <cstddef>
#include <cstdint>
#include <thread>

// Az igazítás az ELEMTÍPUSON van: így a tömb minden eleme required_alignment
// többszörös címen kezdődik, nem csak az első.
struct alignas(std::atomic_ref<int>::required_alignment) Cell {
    int value = 0;
};

std::array<Cell, 4> counters{};
```

```
void bump(std::size_t i) {
    std::atomic_ref<int> ref(counters[i].value);
    for (int n = 0; n < 1000; ++n) {
        ref.fetch_add(1, std::memory_order_relaxed);
    }
}

int main() {
    for (const Cell& c : counters) {
        assert(reinterpret_cast<std::uintptr_t>(&c.value)
               % std::atomic_ref<int>::required_alignment == 0);
    }
    std::thread a(bump, 0);
    std::thread b(bump, 0);
    std::thread c(bump, 1);
    a.join();
    b.join();
    c.join();
    // Itt már nem él atomic_ref, a nem atomikus olvasás megengedett.
    return counters[0].value == 2000 && counters[1].value == 1000 ? 0 : 1;
}
```

Az `alignas(std::atomic_ref<int>::required_alignment)` nem kozmetika: ha az objektum nincs a megkövetelt igazításra igazítva, az `atomic_ref` használata nem definiált viselkedés. A `required_alignment` nagyobb is lehet, mint `alignof(int)`, ezért az igazítást az **elemtípusra** (`Cell`) kell tenni: ha csak a tömböt igazítanánk (`alignas(...) std::array<int, 4>`), az csak az első elemre garantálná a megkövetelt igazítást, a `counters[1]` már `sizeof(int)` eltolásra kerülne. A false sharing elkerülése ettől külön cél: ahhoz a cellákat cache-line méretre kellene igazítani, ami az atomic_ref-előfeltételtől független döntés.

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -pthread atomic_ref_cells.cpp` figyelmeztetés nélkül fordul, a program 0 kóddal fut (minden `counters[i].value` cím igazítását `assert` ellenőrzi), TSan alatt (`setarch -R`) is. A `std::atomic_ref<int>` lehet lock-free vagy sem; ezt `is_lock_free`, illetve `is_always_lock_free` mutatja. A `fetch_add` relaxed itt azért elég, mert csak a számláló értéke érdekes, és a szálak joinja után olvassuk.

Az `atomic_ref` élettartama különösen veszélyes, ha referencia vagy pointer formájában kikerül a lokális scope-ból. Ha az `atomic_ref` tovább él, mint a hivatkozott objektum, dangling hivatkozás lesz; az atomikus művelet ettől még nem lesz biztonságos.

34.3.5 C++20 wait/notify: érték és predikátum

A C++20 `std::atomic<T>::wait` és `notify_one`/`notify_all` egy könnyű várakozási mechanizmust ad. A `wait(old, order)` addig blokkol, amíg a rendszer értesítést nem ad, és az atomikus objektum értéke el nem tér a megadott `old` értéktől. A művelet nem spurius módon tér vissza értékváltozás nélkül, de a `notify` önmagában nem tárol eseményt: ha nincs várakozó szál, az értesítés elveszik. Ez azonban **nem** jelent elveszett ébresztést, ha az érték tartósan megváltozott: a `wait(old)` maga is összehasonlítja az aktuális értéket `old`-dal, és ha az új érték már eltér, nem alszik el. A helyes sorrend ezért a `store(új)` + `notify` a `wait(old)` előtt is, külön előzetes `load` nélkül (az értékellenőrzés a `wait` része); a `notify` feladata csak a már alvó szálak felébresztése.

Külön kérdés az **ABA**: ha az érték `old`-ról átmenetileg `x`-re, majd vissza `old`-ra változik, a várakozó szál ezt a változást elveszítheti; ez az érték-alapú wait szemantikai korlátja, nem a notify sorrendjének hibája.

```
#include <atomic>
#include <cassert>
#include <thread>

std::atomic<int> slot{0};
int payload = 0;

void producer() {
    payload = 42;
    slot.store(1, std::memory_order_release);
    slot.notify_one();
}

void consumer() {
    int v = slot.load(std::memory_order_acquire);
    while (v == 0) {
        slot.wait(0, std::memory_order_acquire);
        v = slot.load(std::memory_order_acquire);
    }
    assert(payload == 42);
}

int main() {
    std::thread p(producer);
    std::thread c(consumer);
    p.join();
    c.join();
}
```

A `wait` az értéket ellenőrzi, nem tetszőleges predikátumot. Ha a várakozási feltétel összetett, például „a sor nem üres és nincs leállítás”, akkor a `wait` után újra kell értékelni a predikátumot, és csak akkor szabad továbbmenni, ha teljesül. A `notify_all` akkor indokolt, ha több várakozó szál közül több is továbbléphet; a `notify_one` egy várakozót ébreszt. A `wait`/`notify` nem helyettesít condition variable-t ott, ahol a várakozás feltétele egy nem atomikus adatszerkezetre vonatkozik; ilyenkor a mutex és a condition variable (RAII lockok, condition variable és wait predicate (34.2)) továbbra is a megfelelő eszköz.

34.3.6 Az atomikus pointer nem tartja életben a pointee-ot

A `std::atomic<Config*>` csak a pointer értékét védi. Ha a publisher release-store-ol egy pointert, majd a consumer acquire-loaddal megkapja és dereferálja, a `Config` objektumnak élnie kell a dereferálás teljes ideje alatt. Az atomikus művelet nem számol referenciát, nem foglal és nem tart életben semmit. A következő sorrend hibás, mert a publisher a consumer használata előtt felszabadíthatja az objektumot:

```
Config* c = new Config{30, 3};
current.store(c, std::memory_order_release);
```

```
// ...
delete c; // a consumer még dolgozhat a pointerrel: dangling
```

A helyes tulajdonosi protokoll legalább az egyik: a consumerek joinolása után szabadítunk fel; referencia-számlálást használunk; vagy epoch/hazard-pointer alapú visszanyerést alkalmazunk. A `current.store(nullptr, release)` önmagában nem oldja meg a versenyt, mert egy consumer a régi pointert már betölthette, és a `delete` után még használhatja. A `std::shared_ptr` referenciaszámlálója atomikus, de ez nem teszi a pointee mezőit automatikusan szálbiztossá, és a publication ordering itt is külön kérdés; a részletes visszanyerési minták a shared ownership, reclamation és konkurens review (34.6) témája.

Ugyanez vonatkozik az atomikus objektum saját élettartamára is. Ha egy `std::atomic<bool>` vagy `std::atomic<int>` lokális változó, és egy szál még hivatkozik rá, amikor a scope véget ér, a program nem definiált. A szálak joinolása, a `std::jthread` destruktorának stop/join viselkedése vagy egyéb, dokumentált élettartam-kapcsolat szükséges.

34.3.7 Gyakori hibák és javításuk

Az egyik leggyakoribb hiba a `volatile` használata atomikus helyett. A `volatile` nem ad szálak közötti szinkronizációt, nem garantál atomikusságot, és nem akadályozza meg az adatversenyt. Helyette `std::atomic<T>` vagy mutex kell.

A második: a publication `relaxed` jelzővel. A `relaxed` store/load csak az adott objektumra ad atomikusságot; a payload nem atomikus olvasása és írása között nem teremt happens-before-t. Ilyenkor a helyes megoldás release/acquire, vagy a teljes adat védelme lockkal.

A harmadik: a `compare_exchange_weak` használata loop nélkül. A spurius hiba nem hiba a programban, hanem a művelet természete; a loop és az `expected` frissítése kötelező. A `strong` sem teszi feleslegessé a versenyhelyzet kezelését, csak a spurius hibát zárja ki.

A negyedik: az `atomic_ref` nem megfelelő igazítása vagy nem atomikus hozzáféréssel való keverése. Az `atomic_ref` nem „varázsolja” atomikussá a mögöttes objektumot más hozzáférések számára; a szerződés az, hogy amíg él, minden hozzáférés `atomic_ref`-en keresztül történik. Az igazítást `required_alignment`-nel kell biztosítani.

Az ötödik: az atomikus pointerből azt feltételezni, hogy a pointee élettartama automatikusan biztonságos. Az atomikus pointer csak a cím értékét védi. A pointee felszabadítása, újrahasznosítása és a rá mutató régi pointerek érvényessége külön tulajdonosi protokollt igényel.

A hatodik: a `wait/notify` félreértése. A `notify` nem tárol eseményt, de a `wait(old)` saját értékellenőrzése miatt a tartósan megváltozott érték nem vész el, ha a `store+notify` a `wait` előtt történt; elveszhet viszont az átmeneti értékváltozás (ABA). A `wait` után a feltételt újra kell értékelni. A `wait` nem univerzális „eseménysor”; a C++20 atomikus wait csak az értékváltozásra és az értesítésre épül.

34.3.8 Feladatok

34.3.8.1 1. feladat: ordering javítása

Adott a következő publikációs kód. A `ready` atomikus, a `payload` nem atomikus. Magyarázd meg, miért hibás a `relaxed` használat, majd javítsd minimális orderinggal. Add meg a szálak élettartamára vonatkozó feltételt is.

```
std::atomic<bool> ready{false};
int payload = 0;

void publisher() {
    payload = 7;
    ready.store(true, std::memory_order_relaxed);
}

void consumer() {
    while (!ready.load(std::memory_order_relaxed)) {}
    assert(payload == 7);
}
```

34.3.8.2 2. feladat: CAS-alapú maximum

Írj `update_max(int)` függvényt, amely `std::atomic<int>`-ban tartja a maximumot, és `compare_exchange_weak`-et használ. Indokold a success és failure ordering választását. Mi történik, ha a `weak` helyett `strong`-ot használunk, és miért nem lesz ettől versenymentes a frissítés?

34.3.8.3 3. feladat: atomic_ref tömbön

Adott egy `std::array<int, 8>` számlálótömb. Több szál ugyanazokat az elemeket növeli `std::atomic_ref<int>`-en keresztül. Írj kódot, amely biztosítja a szükséges igazítást, és magyarázd meg, miért nem szabad a szálak futása közben nem atomikusan olvasni a tömböt. Milyen feltétel mellett lock-free az `atomic_ref<int>`?

34.3.8.4 4. feladat: élettartam-diagnosztika

Adott egy `std::atomic<Widget*>` pointer, amelyet az egyik szál release-store-ol, a másik acquire-loadol és dereferál. A publisher a store után azonnal `delete`-eli a `Widget`-et. Azonosítsd a hibát, és adj két különböző, helyes tulajdonosi megoldást. Az egyik megoldás join-alapú, a másik referencia-számlálásos legyen; mindkettőnél írd le, ki a tulajdonos és meddig él a pointee.

34.3.9 Megoldások

34.3.9.1 1. megoldás

A `relaxed` store és load között nincs release/acquire kapcsolat, ezért a `payload` írása és olvasása nem kerül happens-before viszonyba. Két szál ér hozzá ugyanahhoz a nem atomikus objektumhoz legalább az egyik oldalon írással, ezért adatverseny keletkezik, ami nem definiált viselkedés. A `payload` értéke nem garantáltan 7 a consumerben.

A javítás:

```
void publisher() {
    payload = 7;
    ready.store(true, std::memory_order_release);
}
```

```
void consumer() {
    while (!ready.load(std::memory_order_acquire)) {}
    assert(payload == 7);
}
```

A `ready` objektumnak élnie kell addig, amíg bármelyik szál hozzáférhet; a példában ezt a `join()` biztosítja. A `payload` élettartama szintén legalább a consumer olvasásáig tart. Ellenőrzési feltétel: a consumer acquire loadja a publisher release storejától származó `true` értéket látja, és a `payload` olvasása a load után történik.

34.3.9.2 2. megoldás

```
void update_max(int candidate) {
    int old = maximum.load(std::memory_order_relaxed);
    while (candidate > old &&
           !maximum.compare_exchange_weak(
               old, candidate,
               std::memory_order_release,
               std::memory_order_relaxed)) {
    }
}
```

A success ordering `release` akkor indokolt, ha a `candidate` előtti írásokat is publikálni akarjuk; ha csak a maximum számérték kell, `relaxed` is elég. A failure ordering `relaxed`, mert hiba esetén nem írunk, és nem akarunk acquire-hatást; a failure nem lehet `release` vagy `acq_rel`. A `weak` spurius hibát adhat, ezért a loop újrapróbálkozik, és az `old` értéket a CAS hiba esetén frissíti. A `strong` csak a spurius hibát zárja ki; a két szál közötti verseny ettől még megmarad, ezért a CAS-loop továbbra is kell. Ellenőrzési feltétel: a végső maximum nem lehet kisebb egyetlen sikeresen beírt jelöltnél sem, és a CAS csak akkor ír, ha az aktuális érték még az `old`.

34.3.9.3 3. megoldás

```
// atomic_ref_array.cpp - C++20: Cell elemtípus, minden elem igazított
#include <array>
#include <atomic>
#include <cstddef>
#include <cstdint>
#include <thread>

struct alignas(std::atomic_ref<int>::required_alignment) Cell { int value; };

std::array<Cell, 8> counters{};

void bump(std::size_t i, int n) {
    std::atomic_ref<int> ref(counters[i].value);
    for (int k = 0; k < n; ++k) ref.fetch_add(1, std::memory_order_relaxed);
}

int main() {
    // minden elem igazítását ellenőrizzük, nem csak a 0.-ét
    for (std::size_t i = 0; i < counters.size(); ++i)
```

```
        if (reinterpret_cast<std::uintptr_t>(&counters[i].value)
                % std::atomic_ref<int>::required_alignment != 0) return 2;
    std::thread a(bump, 0, 1000), b(bump, 0, 1000), c(bump, 5, 1000), d(bump, 7,
     ↪ 500);
    a.join(); b.join(); c.join(); d.join();
    return (counters[0].value == 2000 && counters[5].value == 1000 &&
     ↪ counters[7].value == 500) ? 0 : 1;
}
```

Az igazítást az **elemtípuson** (`Cell`) kell kérni, mert a teljes `std::array<int, 8>` objektum `alignas`-a csak az első elem címét igazítja; ha `required_alignment > sizeof(int)`, a következő elem nem feltétlenül igazított. A `Cell` tömbnél minden elem megfelel az `atomic_ref` követelményének, ezt a program minden indexre ellenőrzi, és a teszt több indexet (0, 5, 7) érint. Amíg a `bump` függvényben él az `atomic_ref`, a `counters[i].value`-hoz csak azon keresztül szabad hozzáférni; a `main`-ben a `join` után már nem él `atomic_ref`, ezért a nem atomikus olvasás megengedett. Az `atomic_ref<int>` lock-free volta implementációfüggő (`is_always_lock_free`, `is_lock_free()`). Ellenőrzési feltétel: a három indexen a várt végértékek (2000, 1000, 500), nincs nem atomikus hozzáférés a szálak futása közben, és az igazításellenőrzés minden elemre sikeres.

34.3.9.4 4. megoldás

A hiba: a publisher release-store után azonnal `delete`-eli a `Widget`-et, miközben a consumer acquire-loaddal megkaphatja a pointert, és a `delete` után dereferálhatja. Az atomikus pointer csak a pointer értékét védi; a `Widget` élettartamát nem. A consumer számára a pointer dangling lehet, a dereferálás nem definiált viselkedés.

Join-alapú megoldás: a `Widget` tulajdonosa a publisher vagy egy külön owner objektum. A publisher a store után nem törölhet, amíg a consumerek be nem fejeződtek. A consumerek joinolása után szabad a `delete`, majd a pointert nullára állítani. Itt a tulajdonos az owner objektum, a pointee élettartama a consumerek joinjáig tart. Ellenőrzési feltétel: nincs olyan consumer, amely a `delete` után töltené be vagy használná a pointert.

Referencia-számlálásos megoldás: a `Widget`-et `std::shared_ptr<Widget>` tartja életben; a consumer a pointer betöltésekor vagy egy rövid kritikus szakaszban megnöveli a referenciaszámot, és a használat végén csökkenti. A publisher az utolsó referencia elengedésekor szabadít fel. Itt a tulajdonos a referenciaszámláló által meghatározott utolsó birtokos, a pointee élettartama az utolsó `shared_ptr` megszűnéséig tart. Fontos: a `shared_ptr` referenciaszámlálója atomikus, de a `Widget` mezői nem válnak automatikusan szálbiztossá; a publikálást release/acquire vagy lock védi. A részletes reclamation-minták a shared ownership, reclamation és konkurens review (34.6) témájában következnek.

34.3.10 Összefoglaló

A `std::atomic<T>` típusos, de nem varázseszköz: atomikusságot ad a saját objektumára, nem feltétlenül lock-free, és nem terjeszti ki sem a saját, sem a pointee élettartamát. A `load`, `store`, RMW és CAS műveletekhez tudatosan kell orderingot választani: `relaxed` csak számláláshoz, release/acquire publicationhöz, `seq_cst` pedig akkor, ha a teljes sorrend indokolt. Az `atomic_ref<T>` létező objektumot tesz atomikusan elérhetővé, de csak megfelelő igazítás, élő háttérobjektum és exkluzív hozzáférés mellett. A C++20 `wait`/`notify` értékváltozásra várakozik; elveszett ébresztés és összetett predikátum esetén ciklusos újraellenőrzés kell. Az atomikus pointer publikálása mindig tulajdonosi protokollal egészül ki: join, referencia-számlálás vagy epoch-alapú visszanyerés nélkül a pointer érvényes lehet, a pointee már nem.

34.4 promise, future, async és task eredmény

A taskhatáron átvitt eredmény látszólag egyszerű dolog: az egyik szálon futó számításnak van egy visszatérési értéke vagy egy hibája, és ezt egy másik szálon szeretnénk megkapni. A nehézséget nem az érték átadása jelenti, hanem annak eldöntése, hogy ki birtokolja az átadás csatornáját, mi történik, ha a fogadó fél már nem él, hogyan kerül át a kivétel, és mikor szabad felszabadítani az állapotot. A `std::future` család pontosan ezt a szerződést modellezi: egy különálló, referenciaszámlált **shared state** (megosztott állapot) hordozza az eredményt vagy a hibát, az aszinkron szolgáltató (provider) pedig egy fogantyún keresztül teszi elérhetővé. Ebben az alfejezetben a C++20 alapprofil (`-std=c++20`) szerinti szemantikát tárgyaljuk; a pontos szabványparagrafusokat a `[futures.*]` szakaszokban kell ellenőrizni a saját szabványpéldányodban, mert ehhez az egységhez nem kaptam olyan forrásjegyzetet, amelyre kattintható hivatkozást adhatnék. Az itt leírt viselkedés tehát a nyelvi/library szerződés leírása, nem mért vagy forrásellenőrzött állítás; a konkrét libc++/libstdc++ kiadást a laborprofilban rögzítsd.

34.4.1 A shared state szerződése

A **shared state** egy meg nem nevezett, dinamikusan létrehozott, referenciaszámlált állapotobjektum. Tartalmazza egyfelől az eredményt (vagy egy `std::exception_ptr`-t), másfelől egy „ready” jelzőt és a várakozáshoz szükséges szinkronizációt. Nem része a `std::future` objektumnak: a future csak egy fogantyú, amely hivatkozik rá. A shared state-et mindig egy *aszinkron szolgáltató* hozza létre — `std::promise`, `std::packaged_task`, vagy `std::async` —, és a hozzá tartozó *aszinkron visszatérési objektum* (`std::future` vagy `std::shared_future`) olvassa. A shared state élettartama addig tart, amíg az összes rá hivatkozó szolgáltató és fogantyú el nem tűnik; ez a referenciaszámlálás az, ami miatt a `std::promise`-t nyugodtan át lehet mozgatni egy munkaszálra, és ott helyben megsemmisíteni.

34.4.1.1 A future állapotai: valid, ready, deferred

A `std::future<T>`-nek három, egymástól függetlennek tűnő, de összefüggő állapotjellemzője van. A **valid** azt jelenti, hogy a future ténylegesen hivatkozik egy shared state-re. Az alapértelmezett konstruktorral létrehozott future invalid; a `std::move` forrása invalid lesz; és a `get()` sikeres lefutása után a future invalidálódik, mert az eredményt „kiadtuk” belőle. Invalid (`valid() == false`) future-ön a `get()`, `wait()`, `wait_for()` és a legtöbb tagfüggvény hívása C++20-ban **előfeltétel-sértés: nem definiált viselkedés**. A `std::future_error` `no_state` kóddal dobás csak *ajánlott implementációs viselkedés* (a libstdc++ így tesz), nem hordozható hibakezelési szerződés, ezért nem szabad rá `try/catch`-et építeni. A helyes védelem a `valid()` előzetes ellenőrzése. A **ready** azt jelenti, hogy az eredmény vagy a hiba be van állítva, tehát a `get()` azonnal visszatér. A **deferred** csak a `std::async` által létrehozott future-öknél fordul elő, és azt jelenti, hogy a feladat még el sem indult, hanem a `get()`/`wait()` hívására fog lefutni a hívó szálon.

A `std::shared_future<T>` ugyanarra a shared state-re hivatkozik, de más az olvasási szerződése: másolható, `get()`-je többször hívható, és nem invalidálja a fogantyút. `std::shared_future<T>::get() const T&`-t ad vissza (ha a tárolt típus `T&`, akkor `T&`-t), így a több olvasó nem másolja az eredményt. A `std::future<T>::share()` átadja a shared state-et egy `shared_future`-be, és az eredeti future invalidálódik. Fontos korlát: a `shared_future` önmagában nem tesz semmit „automatikusan” szálbiztossá az *eredmény típusán* belül — a `get()` által visszaadott objektumot továbbra is a szokásos szabályok szerint kell kezelni, ha több szál egyszerre írná.

34.4.2 A promise mint írási végpont

A `std::promise<T>` a shared state írási oldala. A `get_future()` egyszer hívható meg (második hívásra `future_already_retrieved`), és a `set_value`/`set_exception` páros közül pontosan az egyik lehet sikeres: a második kísérlet `promise_already_satisfied` hibát dob. A `set_value_at_thread_exit` és a `set_exception_at_thread_exit` változatok nem azonnal, hanem a szál futásának befejeződésekor teszik ready-vé az állapotot — ez akkor hasznos, ha az eredmény beállítása után még `thread_local` destruktorok futnak, és azokat a fogyasztónak meg kell várnia.

A legsúlyosabb eset, amit mindenképp ismerni kell: ha a `std::promise` úgy semmisül meg, hogy egyik beállító függvényt sem hívtuk meg, a shared state a `std::future_errc::broken_promise` hibával lesz ready, és a fogyasztó `get()`-je `std::future_error`-t dob. Ez a viselkedés szándékos: a fogyasztó soha nem „akad be” csendesen, hanem megtudja, hogy a termelő eltűnt. Ugyanez igaz a `std::packaged_task`-re is, ha meghívás nélkül semmisül meg. Egy feladatokat tároló sor leállításakor ez a tulajdonság különösen értékes: a várakozó hívók hibát kapnak, nem holtpontot.

34.4.3 get, wait és az eredmény átvitele a taskhatáron

A `get()` három dolgot tesz egyetlen hívásban: megvárja a ready állapotot, visszaadja a tárolt értéket, és érvényteleníti a future-öt. Ha a shared state hibát tartalmaz, a `get()` a tárolt `exception_ptr`-t dobja újra. A kivétel eredeti típusa megmarad, tehát a hívó oldalon a konkrét típusra illeszkedő `catch` ág fut le — ez az, ami a taskhatáron átvihető hibakezelést lehetővé teszi.

Itt érdemes megérteni, mi történik a munkaszálon. A `std::packaged_task`, illetve a `std::async` belső implementációja a hívott callable-t `try`/`catch(...)` blokkban futtatja, és `catch` esetén `std::current_exception()`-t tárol el. A stack unwinding tehát a *munkaszálon* lezajlik és befejeződik — a throw, catch és stack unwinding szabályai szerint —, a határon csak egy `std::exception_ptr` kel át. Ez magyarázza, hogy a `get()` nem folytatja a munkaszál vermét, hanem a *hívó* szálon dobja újra a kivételt, a saját vermén.

A `wait()` csak blokkol, értéket nem ad és nem invalidál; a `wait_for(d)`/`wait_until(tp)` `std::future_status::ready`, `timeout` vagy `deferred` értékkel tér vissza. A `deferred` státusz megkülönböztetése azért fontos, mert ilyenkor a `wait_for` nem futtatja le a feladatot, csak jelzi, hogy a feladat nem indult el.

34.4.4 A packaged_task és a move-only callable

A `std::packaged_task<R(Args...)>` egy callable-t és egy shared state-et kapcsol össze: a `get_future()` átadja az olvasási oldalt, a `operator()` pedig meghívja a tárolt callable-t, és az eredményt (vagy a kivételt) beállítja. A `packaged_task` a C++20 alapprofilban **move-only**, nem másolható; ez tudatos tervezési döntés, mert egy „futtatható feladat” csak egyszer futhat le, és a másolás ezt a garanciát lazítaná el.

Ennek közvetlen következménye van a korábbi fejezetre épülve: a `std::function` csak másolható callable-t képes tárolni, ezért `packaged_task`-et közvetlenül nem lehet `std::function<void()>`-be zárni — lásd az invoke, function wrapper és type erasure tárgyalását. Egy feladatsor tipikus eleme ezért `std::queue<std::packaged_task<void()>>` vagy `std::vector<std::move_only_function<void()>` (utóbbi C++23), és a munkaszál a queue-ból `std::move`-val veszi ki a feladatot. A queue-tulajdonos a `packaged_task`-et élettartama alatt birtokolja; a hívó csak a `std::future`-t tartja meg, tehát az eredmény fogantyúja és a futtató objektum tulajdonosa szétválik. A `packaged_task::operator()` másodszori hívása `promise_already_satisfied` hibát dob.

34.4.5 Az std::async és a launch policy

A `std::async(policy, f, args...)` kényelmi szolgáltató: létrehozza a shared state-et, elindítja vagy ütemezni hagyja a callable-t, és visszaadja a future-t. A policy két bitje a `std::launch::async` és a `std::launch::deferred`.

`std::launch::async` esetén a feladat **egy új végrehajtási szálon** fut (a szabvány „as if in a new thread" megfogalmazása szerint), és a visszatérés garantáltan aszinkron. Ha a szál létrehozása nem sikerül, a hívás `std::system_error`-t dob `resource_unavailable_try_again` hibakóddal — ez a gyakorlatban a szál-erőforrás kimerülését jelzi, például nagy számú egyidejű `async` hívásnál.

`std::launch::deferred` esetén a feladat **nem indul el**: a `get()`/`wait()` hívásakor fut le, a hívó szálon, szinkron módon. A `wait_for`/`wait_until` ilyenkor `deferred` státusszal tér vissza anélkül, hogy lefuttatná a feladatot. Ez a viselkedés hasznos lusta kiértékeléshez, de csapda: ha a future-t `get()` nélkül eldobod, a feladat **soha nem fut le**, és erről semmilyen jelzést nem kapsz.

A policy nélküli `std::async(f, args...)` overload a `launch::async | launch::deferred` kombinációt jelenti, vagyis **az implementáció dönt**, hogy tényleg szálat indít-e vagy lustán futtat. Ez nem specifikált választás, tehát olyan kódban, ahol a párhuzamosság megléte vagy hiánya számít, ezt az overloadot ne használd: a viselkedés implementációnként és verzióként eltérhet, és a program helyessége nem épülhet rá.

34.4.6 Mikor blokkol a future destruktora?

A szabály valójában nem a `std::future` típusról, hanem a **shared state elengedéséről** szól (C++20 [futures.state], [futures.async]): egy shared state elengedése nem vár annak elkészültére, *kivéve*, ha (1) a shared state-et `std::async` hozta létre, (2) `launch::async` policyval ténylegesen futó feladattal, (3) ez az utolsó hivatkozás rá, és (4) az állapot még nem ready. Ekkor az elengedés — a gyakorlatban a destruktor — **blokkol, amíg a shared state ready nem lesz**. Mindegy, hogy az utolsó hivatkozást egy `std::future` vagy egy `std::shared_future` tartja. A cél az, hogy a hívó vermén lévő objektumokra mutató referenciák ne váljanak danglinggé, és hogy a „tűzd ki és felejtsd el" használat ne hozzon létre csendben hátrahagyott, senki által nem várt szálakat.

Ez a szabály nem érvényes a `std::promise`-hez vagy `std::packaged_task`-hez tartozó future destruktorára: azok nem blokkolnak. Nem érvényes továbbá a `deferred` esetre sem: a destruktor nem blokkol, és ha a feladatot még nem hívták meg, soha nem is fog lefutni. A `shared_future`-re ugyanez a szabály vonatkozik, csak a helye változik: a `share()` után a shared state-nek több tulajdonosa lehet, és mindig az **utolsó** elengedő `shared_future` destruktora a potenciális várakozási pont — akárhol is él az a másolat (egy másik objektum tagjaként, egy másik szálon). A `share()` tehát nem szünteti meg a blokkolást, hanem nehezebben látható helyre viszi.

```
// shared_future_block.cpp - C++20: az async shared state utolsó elengedése
 ↪  blokkolhat
#include <chrono>
#include <cstdio>
#include <future>
#include <thread>

using namespace std::chrono;

int main() {
    std::shared_future<int> holder;
    {
```

```
        auto f = std::async(std::launch::async, [] {
            std::this_thread::sleep_for(300ms);
            return 7;
        });
        holder = f.share();                    // f üres lesz, holder az
 ↪  egyetlen tulajdonos
    }                                          // itt nincs blokkolás: holder
     ↪  még él
    auto t0 = steady_clock::now();
    {
        std::shared_future<int> last = std::move(holder);
    }                                          // az utolsó shared_future
     ↪  elengedése: vár
    auto ms = duration_cast<milliseconds>(steady_clock::now() - t0).count();
    std::printf("utolso shared_future elengedese: %lld ms\n", static_cast<long
     ↪  long>(ms));
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -pthread shared_future_block.cpp` kimenete `utolso shared_future elengedese: 300 ms`: a `share()` utáni eredeti future megszűnése nem várt, az utolsó `shared_future` elengedése viszont kivárta a feladat végét.

A szabály legismertebb következménye: a `std::async(std::launch::async, f);` utasítás, amely eldobja a visszaadott ideiglenes future-t, a teljes kifejezés végén blokkol, tehát gyakorlatilag szinkron hívássá alakul. Ugyanez ciklusban katasztrofális: a „párhuzamos” ciklus sorban futó ciklussá válik, észrevétlenül.

34.4.7 A hiányzó thread pool és executor

A C++20 és C++23 standard library **nem tartalmaz szálmedencét és általános executort**. A `std::async` nem pool: `launch::async` esetén hívásonként egy szálat indít, ami nagy terhelésnél szál-erőforrás-kimerüléshez és túlparhuzamosításhoz (oversubscription) vezet. A `launch::deferred` sem pool, mert nem tart fenn munkásokat, csak elhalasztja a végrehajtást a hívó szálra. A `std::thread::hardware_concurrency()` csak tippet ad a hardveres szálszámról, és nullát is visszaadhat, tehát önmagában nem konfiguráció.

A gyakorlati megoldás ezért saját medence: fix számú munkaszál, `std::mutex` + `std::condition_variable` (lásd RAII lockok, condition variable és wait predicate) és `std::packaged_task`-et tároló queue, ahol a leállítás sorrendje a queue lezárása, a munkások felébresztése, majd join. A C++26 irányai között szerepel a szabványos execution modell (`std::execution`, P2300), de ennek elfogadási és implementálási állapotát itt nem ellenőriztem; a C++20/23 alapprofilban nem elérhető. A `shared_ptr`-ral való megosztott tulajdonlás konkurens kérdéseit szándékosan nem itt tárgyaljuk, hanem a shared ownership, reclamation és konkurens review részben.

Az alábbi táblázat a négy mechanizmus szerződését foglalja össze.

Mechanizmus	Ki hozza létre a shared state-et?	Ki állítja be az eredményt?	Hol fut a feladat?	A future destruktora blokkol?
`std::promise`	a promise	a felhasználó (`set_value`)	a felhasználó dönti el	nem
`std::packaged_task`	a packaged_task	`operator()`	aki meghívja	nem

Mechanizmus	Ki hozza létre a shared state-et?	Ki állítja be az eredményt?	Hol fut a feladat?	A future destruktora blokkol?
`std::async(launch::async, …)`	az `async`	a callable	új szálon	igen, ha nem ready és ez az utolsó referencia
`std::async(launch::deferred, …)`	az `async`	a callable	a `get()`/`wait()` hívó szálán	nem; ha nem hívták meg, nem fut le

34.4.8 Kidolgozott példa: eredmény és hiba átvitele három mechanizmussal

A példa három esetet mutat ugyanabban a fordítási egységben: explicit `promise`-t, `packaged_task`-et move-only callable-lel, valamint `async`-et mindkét policy-vel. A cél annak megfigyelése, hogy a kivétel mindhárom esetben a `get()` hívás helyén, a hívó szálon jelenik meg.

```
// task_result.cpp -- oktatasi pelda, C++20, Linux, -pthread
#include <chrono>
#include <exception>
#include <future>
#include <iostream>
#include <stdexcept>
#include <string>
#include <thread>
#include <type_traits>
#include <utility>

namespace demo {

int parse_parallelism(const std::string& s) {
    if (s.empty()) throw std::invalid_argument("ures parallelizmus");
    return std::stoi(s);   // dobhat std::invalid_argument / std::out_of_range
}

// (1) promise: a dolgozo szal explicit eredmeny- es hibaatadassal
void worker(std::promise<std::string> p) {
    try {
        p.set_value(std::string{"kesz"});
    } catch (...) {
        p.set_exception(std::current_exception());
    }
}

// (2) packaged_task: move-only callable
struct Divider {
    Divider() = default;
    Divider(Divider&&) noexcept = default;          // move-only: a másolás
 ↪  tiltott
    Divider(const Divider&) = delete;
    Divider& operator=(Divider&&) noexcept = default;
```

```
    int operator()(int a, int b) const {
        if (b == 0) throw std::domain_error("osztas nullaval");
        return a / b;
    }
};

static_assert(!std::is_copy_constructible_v<Divider>, "a Divider move-only");

} // namespace demo

int main() {
    using namespace std::chrono_literals;

    {   // (1) promise + future
        std::promise<std::string> p;
        std::future<std::string> f = p.get_future();
        std::jthread t{demo::worker, std::move(p)};   // jthread: kivétel esetén
        ↪ is joinol a destruktor
        try {
            const std::string v = f.get();             // előbb az érték, utána
            ↪ a kiírás
            std::cout << "promise: " << v << '\n';
        } catch (const std::exception& e) {            // a worker hibája itt
        ↪ kezelhető hiba
            std::cout << "promise hiba: " << e.what() << '\n';
        }
    }

    {   // (2) packaged_task: a hivas hozza letre az eredmenyt
        std::packaged_task<int(int, int)> task{demo::Divider{}};
        std::future<int> f = task.get_future();
        std::jthread t{std::move(task), 7, 0};         // a destruktor minden
        ↪ úton bevár
        try {
            const int v = f.get();
            std::cout << "packaged_task: " << v << '\n';
        } catch (const std::domain_error& e) {
            std::cout << "packaged_task hiba: " << e.what() << '\n';
        }
    }

    {   // (3) async: async es deferred policy
        std::future<int> fa = std::async(std::launch::async,
                                         demo::parse_parallelism,
                                         std::string{"4"});
        std::future<int> fd = std::async(std::launch::deferred,
                                         demo::parse_parallelism,
                                         std::string{""});
        auto st = fd.wait_for(0ms);
        std::cout << "deferred statusz: "
                  << (st == std::future_status::deferred ? "deferred" : "nem
                  ↪ deferred")
```

```
                  << '\n';
        std::cout << "async: " << fa.get() << '\n';
        try {
            const int v = fd.get();
            std::cout << "deferred: " << v << '\n';
        } catch (const std::invalid_argument& e) {
            std::cout << "deferred hiba: " << e.what() << '\n';
        }
    }
}
```

Fordítás és futtatás a tanuló saját laborjában:

```
g++ -std=c++20 -O2 -g -Wall -Wextra -pthread task_result.cpp -o task_result
./task_result
```

A példát GCC 13.3-mal lefordítottam és lefuttattam (`-std=c++20 -O2 -Wall -Wextra -pthread`, valamint ASan+UBSan és TSan alatt is), a kimenet sorrendben: `promise: kesz`, `packaged_task hiba: osztas nullaval`, `deferred statusz: deferred`, `async: 4`, `deferred hiba: ures parallelizmus`. A harmadik blokkban a `fd` future a `wait_for` pillanatában még nem futtatta le a feladatot, ezért kapunk `deferred` státuszt; a `get()` ekkor futtatja le a hívó szálon, ahol a `std::invalid_argument` keletkezik és azonnal tovább is dobódik. A `std::thread` helyett `std::jthread` szerepel, mert a `f.get()` kivételt dobhat, és `joinable std::thread` destruktora `std::terminate`-et hívna; a `jthread` destruktora minden úton (kivétel esetén is) bevár. Egy injektált worker-kivétel (a `set_value` helyett `set_exception`) a hívónál kezelhető hibaként jelenik meg, és a worker joinolódik. (A `cout` kivételkezelése beállításfüggő, de a példában alapértelmezett.)

34.4.9 Tipikus hibák és javításuk

Az első és leggyakoribb hiba az `std::async` eredményének eldobása. A `std::async(std::launch::async, f);` látszólag aszinkron, de a destruktor blokkolása miatt szinkronná válik; javítás: mindig tárold a future-t, és a blokkolást a `get()`/`wait()` helyére tedd. A második a `launch::deferred` future `get()` nélküli eldobása: a feladat csendben elmarad, ami hiányzó oldalhatás formájában jelentkezik — ilyenkor vagy mindig hívd meg a `get()`-et, vagy ne használj `deferred`-et. A harmadik a `get()` kétszeri hívása: az első hívás invalidálja a future-öt, a második hívás előfeltétel-sértés (UB; egy konkrét könyvtár `no_state` kivételt dobhat, de ez csak megfigyelés). Ismételt olvasáshoz `shared_future`-t használj, vagy hívás előtt ellenőrizd a `valid()`-ot. A negyedik a csupasz `std::async(f, ...)` policy nélküli hívás produkciós kódban: a párhuzamosság megléte implementációfüggő, ezért determinisztikus viselkedést ne építs rá. Az ötödik, hogy a `packaged_task`-et `std::function`-be próbálják tenni: az nem fordul le, mert a `packaged_task` nem másolható; a megoldás move-only tároló vagy `std::move`-val átadott queue-elem.

34.4.10 Feladatok

1. feladat (magyarázó). Egy kollégád ezt írja: „`std::async(std::launch::async, []{ /* hosszu munka */ });` — így tűzd ki a munkát, és ne várj rá.” Magyarázd el pontosan, miért nem aszinkron ez a kód, és melyik szabványbeli feltétel teljesül itt a future destruktor blokkolásához.

2. feladat (kódolási). Írj `std::vector<int> wait_all(std::vector<std::future<int>>& fs)` függvényt, amely minden future-ből begyűjti az eredményt, és az első kivételt a hívó felé továbbítja, de csak azután, hogy minden future-t megvárt. Indokold a sorrendet.

3. feladat (diagnosztikai). Az alábbi kód lefut, de a `mentes()` soha nem hajtódik végre. Add meg a pontos okot, a javítást és az ellenőrzési feltételt.

```
std::async(std::launch::deferred, []{ mentes(); });
```

4. feladat (tervezési). Egészítsd ki a vázlatot egy olyan pontig, ahol a `packaged_task`-et move-only queue-ba tesszük, a future-t a hívó kapja meg, és a medence leállításakor a függőban lévő feladatok `broken_promise` hibát okoznak a hívóknál a holtpont helyett. Melyik objektum birtokolja a `packaged_task`-et az egyes fázisokban?

34.4.11 Megoldások

1. megoldás. Az ideiglenes `std::future` a teljes kifejezés végén megsemmisül. Mivel az `async launch::async` policy-vel indult, a shared state létrehozója az `async`, a feladat még nem ready, és ez a future az utolsó referencia — mind a négy feltétel teljesül, ezért a destruktor blokkol a ready állapotig. A kód tehát szinkron: a „kitűzés” helyén megvárja a munkát. Javítás: `auto f = std::async(...);` és a várakozás később, explicit `f.get()`/`f.wait()` formában. Ellenőrzési feltétel: a munkába tett `std::this_thread::sleep_for` után mérd a hívó szál eltelt idejét — aszinkron esetben rövid, a hibás változatban a munka idejével nő (a mérés a te labodban történik, nem itt).

2. megoldás. Teljes implementáció és hibaszerződés:

```
#include <exception>
#include <future>
#include <stdexcept>
#include <vector>

// Szerződés: minden future valid() legyen (különben std::invalid_argument, mert
 ↪  az invalid
// future get/wait előfeltétel-sértés, UB); minden future-t megvár; az
 ↪  eredmények a bemenet
// sorrendjében jönnek; az ELSŐ kivételt (index szerint) dobja újra, miután
 ↪  mindenki lefutott.
std::vector<int> wait_all(std::vector<std::future<int>>& fs) {
    for (const auto& f : fs)
        if (!f.valid()) throw std::invalid_argument("wait_all: invalid future");
    for (auto& f : fs) f.wait();                       // nem dob, nem invalidál
    std::vector<int> out;
    out.reserve(fs.size());
    std::exception_ptr first;
    for (auto& f : fs) {
        try { out.push_back(f.get()); }
        catch (...) { if (!first) first = std::current_exception(); }
    }
    if (first) std::rethrow_exception(first);
    return out;
}
```

A sorrend indoka: először minden future-ön `wait()`, így egy korai kivétel nem hagy futó feladatot (async esetén a többi future destruktora különben blokkolhatna, és eredmények vesznének el); a `get()`-ek ezután nem blokkolnak. Az első kivételt a ciklus után, `std::exception_ptr`-ből dobjuk újra, hogy a „mindenki lefusson” garancia megmaradjon. Kimenetek: siker esetén az

eredményvektor; hibánál a legkisebb indexű feladat kivétele (a többi elfogyasztva, minden future `valid() == false`); invalid bemenetnél `std::invalid_argument`. Ellenőrizve GCC 13.3-mal ASan+UBSan alatt: három sikeres, egy hibás (az első kivétel `runtime_error`, index 1), és egy invalid future esetre. Ellenőrzési feltétel: a függvény akkor is visszatér, ha a második future dob; a korábbi és későbbi future-ök is elfogyasztottak.

3. megoldás. A `launch::deferred` azt jelenti, hogy a callable csak `get()`/`wait()` hívására fut le; az eldobott ideiglenes future destruktora nem blokkol és nem futtat. Javítás: vagy tárold a future-t és hívd meg a `get()`-et, vagy használj `launch::async`-et. Ellenőrzési feltétel: a `mentes()` oldalhatása (pl. fájl létrejötte) csak akkor figyelhető meg, ha a `get()` lefutott; ezt a fájl létezésével lehet ellenőrizni, nem a program futási idejével.

4. megoldás. A `packaged_task` életciklusa három fázisra bontható: (a) a hívó létrehozza, `get_future()`-rel kiveszi a future-t, majd `std::move`-val a queue-ba teszi — innentől a queue a tulajdonos; (b) a munkaszál `std::move`-val kiveszi a queue-ból, és a saját vermén futtatja — a futtatás idejére a munkaszál a tulajdonos; (c) a futtatás (az `operator()` visszatérése) befejeztével a shared state **azonnal ready** lesz, és a hívó `get()`-je visszatér — ehhez a `packaged_task` megsemmisülése nem kell; a destruktor csak akkor számít, ha a task sosem futott: ekkor a nem teljesített shared state `broken_promise` hibát kap. Leállításkor a queue-t le kell zárni, a munkásokat felébreszteni és joinolni; a queue-ban maradt `packaged_task`-ek megsemmisülése `broken_promise`-t állít be, így a hívók hibát kapnak. Ellenőrzési feltétel: leállítás után egyetlen hívó `get()`-je sem akad be határozatlan ideig, és a `join` minden munkaszálra lefut.

34.4.12 Összefoglaló

A `std::future` család egy referenciaszámlált shared state köré szerveződik, amely az értéket vagy a `std::exception_ptr`-t hordozza, és amelyet mindig egy szolgáltató — `promise`, `packaged_task` vagy `async` — hoz létre. A `get()` megvárja a ready állapotot, kiemeli az eredményt vagy újradobja a kivételt a hívó szálon, majd invalidálja a future-öt; a `shared_future` többszöri, nem invalidáló olvasást tesz lehetővé. A `packaged_task` move-only, ezért `std::function`-be nem tehető, viszont move-only queue-val jól illeszkedik egy saját szálmedence feladatmodelljéhez. Az `std::async` két policy-je közül a `launch::async` garantáltan új szálon fut és szál-létrehozási hibánál `std::system_error`-t dob, a `launch::deferred` pedig csak `get()`/`wait()` hívásra, a hívó szálon fut le. A future destruktor blokkolása kizárólag a `launch::async`-kel indított, még nem ready, utolsó referenciát tartó `async`-future-re érvényes — ez a szabály teszi a látszólag aszinkron, eldobott future-t szinkronná, és ezért kell minden `async` future-öt tárolni és explicit módon megvárni. A C++20/23 alapprofilban nincs szabványos thread pool és executor, ezért a medence, a queue és a leállítási sorrend továbbra is a program szerzőjének felelőssége.

34.5 semaphore, latch, barrier és szinkron stream

A C++20 koordinációs eszközei – a `<semaphore>`, `<latch>`, `<barrier>` és `<syncstream>` – a mutex és condition variable fölötti, gyakori mintákra szabott absztrakciók. Nem helyettesítik a memóriamodellt, és nem adnak automatikus cancellationt vagy fairness-t: a helyes használatot a hívónak kell szerződésként betartania. Ez a rész a RAII lockok, condition variable és wait predicate mintájára épít, a promise, future, async és task eredmény felé pedig ott kapcsolódik, ahol a várakozás eredménye és a task élettartama válik fontossá. A `<barrier>`, `<latch>`, `<semaphore>` és `<syncstream>` könyvtári támogatása compiler- és standard library verziófüggő; a konkrét buildprofilban ellenőrizni kell. Az itteni példák C++20-at feltételeznek, és nem állítják, hogy fordításuk vagy futásuk ebben a szövegben megtörtént.

34.5.1 Közös szerződés és eszközválasztás

Mind a négy eszköz koordinációt végez: egy művelet láthatóvá tesz egy korábbi állapotot egy másik szálnak. A szemafor `release` művelete szinkronizál azzal az `acquire`-rel, amely az inkrementált számlálót megfigyeli; a latch utolsó `count_down`ja szinkronizál a `wait` visszatérésével; a barrier fázisának completionje szinkronizál a fázisra várakozók visszatérésével; az `osyncstream::emit` pedig a mögöttes streambuf felé továbbított rekord koherens átvitelét biztosítja. Ez nem azt jelenti, hogy a primitívek minden mellékhatást „varázsolnak” – a programozónak kell biztosítania, hogy a megosztott adatot csak a megfelelő szinkronizációs pont után olvassa.

Eszköz	Ismételhető	Tipikus használat	Fő élettartam-kockázat
`counting_semaphore`	igen	kapacitáskorlát, engedély, jelzés	túlrelease; várakozó a destruktorban
`latch`	nem	egyszeri startkapu, inicializálás befejezése	`count_down` túllépés; várakozó a destruktorban
`barrier`	igen	ismétlődő fázisok szinkronizálása	kimaradó résztvevő; completion szerződésszegés
`osyncstream`	igen	logrekord-koherencia	a mögöttes stream élettartama; megosztott objektum

Közös szabály, hogy a semaphore, latch és barrier destruktorának előfeltétele: ne legyen blokkolt várakozó szál. Ennek megsértése nem definiált viselkedés; a gyakorlatban holtpont, összeomlás vagy csendes adatkorrupció lehet. A primitív nem tartja életben a várakozót, és a várakozó nem tartja életben a primitívet.

Bizonyításvázlat (vázlat). Ha egy szál a `release`/`count_down`/`arrive` műveletet hajtja végre, majd egy másik szál sikeresen visszatér a megfelelő várakozásból, a szabvány synchronization relationje miatt az előbbi szál előtti írások láthatók az utóbbi szálban. Ezért a „jelző” primitívek nem csupán időzítésre, hanem publikálásra is használhatók; a megosztott adatot azonban továbbra is védeni kell az egyidejű írásoktól.

34.5.2 Szemafor: kapacitás és engedély

A `std::counting_semaphore<LeastMaxValue>` egy nem másolható számlálót tart. Az `acquire()` addig blokkol, amíg a számláló nullánál nagyobb, majd csökkenti azt; a `try_acquire()`, `try_acquire_for()` és `try_acquire_until()` nem blokkoló, illetve időkorlátos változatok. A `release(update = 1)` növeli a számlálót, és felébreszthet várakozókat. A `std::binary_semaphore` a `counting_semaphore<1>` aliasa.

A `release` előfeltétele, hogy a növelés ne vigye a számlálót a `max()` fölé: $0 \le update \le max() - counter$. A `max()` legalább a template paraméter, de implementációfüggően nagyobb is lehet. Fontos következmény: a konstruktor kezdőértéke nem felső korlát. Ha `counting_semaphore<2> s(2)`-t hozunk létre, attól még a `max()` nagyobb lehet, és a „legfeljebb kettő engedély” invariantot a programnak kell betartania, nem a szemafor. A túlrelease nem „csak” logikai hiba: előfeltétel-sértés, tehát nem definiált viselkedés.

A szemafornak nincs tulajdonosa abban az értelemben, mint a mutexnek: a `release`-et más szál is végrehajthatja, mint amelyik az `acquire`-t. Ez alkalmas jelzésre és kapacitáskorlátra, de nem

nyújt kölcsönös kizárást „tulajdonosi” garanciával. Kölcsönös kizáráshoz továbbra is a `std::mutex` a természetes választás; a bináris szemafor inkább ping-pong jelzésre vagy esemény jelzésére való. C-ben a POSIX `sem_t` hasonló szerepet tölt be, de a `sem_destroy` élettartam-feltétele és a névvel ellátott szemaforok kezelése külön figyelmet igényel.

Leállításnál a `counting_semaphore` nem kínál `stop_token` túlterhelést. A `try_acquire_for` időkorlátos ciklusa adhat korlátos reakcióidőt, de ez polling: a stop kérés és a várakozás vége között legfeljebb a választott időköz telik el. Megszakítható várakozásra a C++20 `condition_variable_any` `stop_token`-es `wait` változata a természetes eszköz; a szemaforral épített „stop-aware” kaput külön kell megtervezni.

34.5.3 Latch: egyszeri kapu

A `std::latch` egy egyszer használható leszámláló. A konstruktorban megadott számlálót a `count_down(update = 1)` csökkenti; a `wait()` addig blokkol, amíg a számláló nullává nem válik; a `try_wait()` nem blokkoló ellenőrzés; az `arrive_and_wait(update = 1)` a `count_down` és a `wait` összevonása. A `count_down` előfeltétele, hogy a frissítés ne haladja meg az aktuális számlálót. Ha többször csökkentenénk nullánál, az előfeltétel-sértés.

A latch tipikus használata a startkapu: a fő szál létrehoz egy `std::latch start(1)` objektumot, a munkások `start.wait()`-tel várakoznak, majd a fő szál `start.count_down()` hívásával egyszerre engedi el őket. Ugyanígy használható „minden rész inicializálva” kapuként: n résztvevő `count_down` után a várakozó továbblép. A latch nem resetelhető; ismétlődő fázisokhoz barrier kell. A latch élettartama alatt a számláló monoton csökken, és nulla után már nem nő.

34.5.4 Barrier: ismétlődő fázisok

A `std::barrier` egy fix résztvevőszámú, ismételhető fáziskapu. A konstruktor megadja a várt érkezők számát és opcionálisan egy completion függvényt. A résztvevők `arrive()`-tal érkeznek, amely egy `arrival_token`-t ad vissza, majd `wait(token)`-tel várnak a fázis befejezésére; az `arrive_and_wait()` a kettőt együtt végzi. Amikor az utolsó résztvevő is megérkezik, a completion függvény lefut, majd a várakozók visszatérnek, és kezdődhet a következő fázis.

A completion szerződése szigorú: pontosan a fázis utolsó érkezője futtatja, a várakozók csak utána térnek vissza, nem dobhat kivételt, és nem hívhatja a barrier tagfüggvényeit. A gyakorlatban rövid, nem blokkoló művelet legyen; a „ completionben mindenki helyett mindent megcsinálok” minta könnyen szűk keresztmetszet vagy holtpont forrása. A completion alatt a barrier már a következő fázisra készül, ezért abból a barrierre hivatkozni különösen veszélyes.

Az `arrive_and_drop()` a hívó szálat véglegesen kiveszi a következő fázisokból: az aktuális fázisban még érkezik, de a várt résztvevőszám csökken. Csak olyan szál hívhatja, amely részt vesz az aktuális fázisban, és utána már nem használhatja a barriert. Ha egy résztvevő egyszerűen kilép, és nem hív `arrive_and_drop`-ot, a többi szál örökre várhat. A barrier nem tudja kívülről „leszállítani” a várt számot; a drop az egyetlen mechanizmus. Leállításnál ezért vagy minden résztvevőnek végig kell mennie egy utolsó fázison, vagy a dropot gondosan, közös protokoll szerint kell használni.

34.5.5 `osyncstream`: rekordkoherencia

A `std::osyncstream` (pontosabban `basic_osyncstream<char>`) egy puffert tart, és az `emit()` híváskor – illetve a destruktorban – a puffert egybefüggő rekordként továbbítja a mögöttes streambuf felé. A cél, hogy a több szálból írt logüzenetek ne csússzanak egymásba. Nem globális sorrendet ad: két rekord sorrendje attól függ, melyik `emit` éri el előbb a mögöttes streamet.

A rekordkoherencia csak a syncstreamen keresztül történő emitokra vonatkozik; a közvetlen `std::cout`-írásokkal való keveredést külön kell kezelni.

A legegyszerűbb minta a rekordonkénti ideiglenes objektum:

```
void log_line(const char* who, int round) {
    std::osyncstream(std::cout) << who << " round " << round << '\n';
}
```

Az ideiglenes `osyncstream` a teljes kifejezés végén megsemmisül, és ekkor emitál. Egy szálon belül létrehozott, hosszabb életű `osyncstream` is használható, de egy objektumot nem szabad több szál között megosztani: az `operator<<` és a belső puffer nem válik thread-safe-é attól, hogy az emit szinkronizált. A mögöttes streamnek és streambufnak túl kell élnie az `osyncstream`-et; ha az `osyncstream` egy lokális `std::ostringstream`-re mutat, amely már megsemmisült, a destruktor emitja dangling streambufra ír. Globális `osyncstream` esetén a statikus megsemmisítési sorrend külön kockázat, ezért a gyakorlatban lokális vagy szálonkénti példányt érdemes használni. Ha a mögöttes stream kivételre van kapcsolva, az emit dobhat; a destruktorban ez terminate-hez vezethet, ezért a hibakezelést ne a destruktorra bízzuk.

34.5.6 Várakozó élettartama, leállítás és fairness

A legfontosabb élettartam-szabály: a semaphore, latch és barrier destruktora előtt minden várakozónak ki kell lépnie. Ez tipikusan azt jelenti, hogy a tulajdonos szál joinolja a munkásokat, vagy egy külön leállítási fázisban mindenkit beenged. A `std::jthread` automatikus joinja önmagában nem old meg mindent: a stop kérés kooperatív, és nem szakít meg egy `semaphore.acquire()` vagy `latch.wait()` hívást. Ha a munkás ezekben blokkol, a `jthread` destruktora joinol, és a program holtpontba kerülhet. A stopot ezért a várakozásba kell beépíteni: condition variable esetén a `stop_token`-es `wait` túlterheléssel, szemafor esetén időkorlátos `try_acquire_for` ciklussal, barrier esetén pedig explicit leállítási fázissal vagy `arrive_and_drop`-pal.

A fairness nem garantált. A `release`, `count_down` és `arrive` által felébresztett várakozó kiválasztása nem specifikált; a primitívek nem FIFO-k, és nem biztosítanak prioritást. Éhezés előfordulhat, különösen sok várakozó és ritka release esetén. Ha a sorrendiség üzleti követelmény, azt külön adatszerkezettel és mutex/CV párossal kell megvalósítani, nem a primitív „fairnek” feltételezett viselkedésére bízva.

34.5.7 Kidolgozott példa

A következő példa négy munkást indít, egy latch startkapuval engedi el őket, egy kétengedélyes szemaforral korlátozza az egyidejű „IO” szakaszt, egy barrierrel pedig három fázisban szinkronizál. A naplózás rekordonkénti `osyncstream`-mel történik. A tulajdonos a `main`: a latch, a szemafor, a barrier és a `std::vector<std::thread>` a `main` élettartamában él; a munkások referenciákat kölcsönöznek. Invalidáló művelet a primitívek destruktora, ha várakozó van; a hiba kimenete holtpont vagy nem definiált viselkedés. A példa feltételezi, hogy a szálak létrehozása sikerül; termelési kódban a kivételbiztos joinról külön kell gondoskodni.

```
// sync_demo.cpp - C++20
#include <atomic>
#include <barrier>
#include <chrono>
#include <cstddef>
#include <cstring>
```

```cpp
#include <exception>
#include <iostream>
#include <latch>
#include <semaphore>
#include <stdexcept>
#include <syncstream>
#include <thread>
#include <vector>

namespace {
void log_line(int who, int round) {
    std::osyncstream(std::cout) << "worker " << who << " round " << round <<
     ↪  '\n';
}

struct SlotGuard {                              // a szemafor kivételes úton is
 ↪  felszabadul
    std::counting_semaphore<2>& s;
    explicit SlotGuard(std::counting_semaphore<2>& sem) : s(sem) { s.acquire();
     ↪  }
    ~SlotGuard() { s.release(); }
};

struct StartGuard {                             // a főszál unwindingja esetén is
 ↪  elengedi
    std::latch& start;                          // a workereket, de futás nélkül
    std::atomic<bool>& abort;
    bool released = false;
    void release() { released = true; start.count_down(); }
    ~StartGuard() { if (!released) { abort = true; start.count_down(); } }
};
} // namespace

int main(int argc, char** argv) {
    constexpr int workers = 4;
    constexpr int rounds = 3;
    const bool worker_fails = argc > 1 && std::strcmp(argv[1], "worker-fail") ==
     ↪  0;
    const bool main_fails = argc > 1 && std::strcmp(argv[1], "main-fail") == 0;

    std::latch start(1);
    std::atomic<bool> abort_run{false};
    std::counting_semaphore<2> io_slots(2);
    std::barrier round_done(workers, []() noexcept {
        std::osyncstream(std::cout) << "-- phase complete --\n";
    });
    std::vector<std::exception_ptr> errors(workers);  // szálanként saját hely

    try {
        std::vector<std::jthread> pool;     // unwinding esetén is join
        pool.reserve(workers);
        StartGuard guard{start, abort_run}; // a pool UTÁN: előbb fut le
```

```
        for (int i = 0; i < workers; ++i) {
            pool.emplace_back([&, i] {
                start.wait();
                if (abort_run) return;      // indítás meghiúsult: nincs munka
                try {                       // a worker kivételhatára
                    for (int r = 1; r <= rounds; ++r) {
                        {
                            SlotGuard slot(io_slots);

                            ↪ std::this_thread::sleep_for(std::chrono::milliseconds(5));
                            if (worker_fails && i == 2 && r == 2)
                                throw std::runtime_error("disk error");
                            log_line(i, r);
                        }
                        round_done.arrive_and_wait();
                    }
                } catch (...) {
                    errors[i] = std::current_exception();
                    round_done.arrive_and_drop();   // a többiek ne várjanak rá
                }
            });
        }
        if (main_fails) throw std::runtime_error("setup failed");
        guard.release();
    } catch (const std::exception& e) {     // ide csak a főszál hibája jut
        std::cerr << "main: " << e.what() << '\n';
        return 1;
    }

    int failed = 0;                          // a join után az errors olvasható
    for (int i = 0; i < workers; ++i) {
        if (!errors[i]) continue;
        ++failed;
        try { std::rethrow_exception(errors[i]); }
        catch (const std::exception& e) {
            std::cerr << "worker " << i << ": " << e.what() << '\n';
        }
    }
    return failed == 0 ? 0 : 1;
}
```

Fordítás GNU környezetben:

```
g++ -std=c++20 -O2 -pthread -Wall -Wextra -Wpedantic sync_demo.cpp -o sync_demo
```

A `-pthread` a szálkezeléshez szükséges. A példa nem használ stopot: a leállítás a ciklus természetes végével történik. A szemafor nem fair, ezért a példa nem garantálja, hogy minden munkás ugyanannyi időt kap.

A hibakezelésnél két külön határt kell megtervezni.

- **A létrehozó oldal kivételbiztonsága.** Ha a főszál a szálak elindítása után, de a `join` előtt kivétellel lép ki (például egy további `emplace_back` vagy beállítás hibázik), a joinable

`std::thread` destruktora `std::terminate`-et hív. Ez ellen véd a `std::jthread`, amely a destruktorában joinol. Önmagában azonban kevés: a szálak a `start.wait()`-ben állnak, így a join örökre várna. Ezért a `StartGuard` unwinding esetén megszakítást jelez, és elengedi a latch-et; a workerek munka nélkül kilépnek, és a join befejeződik.

- **A worker belépési pontjának kivételhatára.** A worker törzséből kiszökő kivétel `std::terminate`-et vált ki, és ezen sem a `jthread`, sem egy join-guard nem változtat: azok a létrehozó oldalt védik, nem a szál belépési pontját. A workernek ezért magának kell elkapnia a kivételt, és dokumentált csatornán jelentenie: itt egy szálanként külön `std::exception_ptr` helyen, amelyet a főszál a `join` után olvas. A puszta `catch` és korai `return` viszont új holtpontot okozna, mert a többi worker a barrierben a kilépett résztvevőre várna. Ezért a hibaúton `arrive_and_drop()` következik: a worker az aktuális fázisba még beérkezik, a további fázisokból pedig kivonja magát. A szemafort a `SlotGuard` a kivételes úton is felszabadítja.

```
./sync_demo               # normál futás
./sync_demo worker-fail   # a 2-es worker a 2. körben dob
./sync_demo main-fail     # a főszál az indítás előtt dob
```

Buildjegyzőkönyv (`lab-gcc` profil). A program figyelmeztetés nélkül fordult. Normál futásban 12 körsor és 3 fáziszárás, kilépési kód 0. A `worker-fail` esetben a 2-es worker az 1. kör után kiesik: 10 körsor és 3 fáziszárás után a főszál `worker 2: disk error` üzenettel, 1-es kóddal lép ki, holtpont és `terminate` nélkül. A `main-fail` esetben `main: setup failed`, 0 körsor, 1-es kód, holtpont nélkül. Mindhárom eset ThreadSanitizer (`setarch -R`) alatt is figyelmeztetés nélkül futott.

34.5.8 Tipikus hibák

- A `release` túllépi a `max()`-ot, vagy a latch `count_down` többel csökkent, mint a számláló. Mindkettő előfeltétel-sértés, nem „csak” logikai hiba.
- A szemafor kezdőértékét felső korlátnak tekintjük. A felső korlát a `max()`; a kettő nem ugyanaz.
- Egy barrier-résztvevő kilép `arrive_and_drop` nélkül, mire a többi örökre vár. Leállításnál a fázistopológia nem változhat önkényesen.
- A completion függvény dob, vagy barrier-tagfüggvényt hív. A szerződés megsértése nem használható hibakezelésre.
- Egy `osyncstream` objektumot több szál használ, vagy az `osyncstream` túléli a mögöttes `ostringstream`-et. Utóbbi dangling streambuf, a destruktor emitja nem definiált.
- A `jthread` stop kérésétől várjuk, hogy megszakítsa a `semaphore.acquire()`-t vagy a `latch.wait()`-et. Nem szakítja meg; a join holtpont lehet.
- A primitívtől FIFO-t vagy prioritást várunk. A várakozók kiválasztása nem specifikált.

34.5.9 Feladatok

34.5.9.1 1. feladat: eszközválasztás

Válaszd ki a megfelelő C++20 eszközt, és indokold a döntést. Mindhárom esetben nevezd meg azt is, miért nem a másik kettőt választanád.

a) Nyolc munkásnak meg kell várnia, hogy a konfiguráció betöltődjön; ez egyszer történik meg a program indulásakor.

b) Legfeljebb három párhuzamos adatbázis-kapcsolat lehet egyszerre.

c) Négy szál tíz iterációban dolgozik, és minden iteráció végén mindenkinek meg kell várnia a többieket, majd jöhet a következő iteráció.

34.5.9.2 2. feladat: stop-aware szemafor

Írj egy `acquire_stop_aware` függvényt, amely egy `std::counting_semaphore<N>` engedélyét próbálja megszerezni, de `std::stop_token` kérés esetén legfeljebb egy rövid időkorlátnyi késéssel visszatér. Készíts hozzá olyan RAII engedélyőrt, amely a már megszerzett engedélyt „adoptálja”, és a destruktorában release-el. Magyarázd meg, miért nem lehet a `semaphore.acquire()`-t egyszerűen megszakítani.

34.5.9.3 3. feladat: diagnosztika

A következő részlet három hibát tartalmaz. Azonosítsd őket, mondd meg, melyik szabályt sértik, és javítsd őket.

```
// részlet, nem önálló program
std::latch ready(2);
ready.count_down(3); // (A)

std::barrier b(2);
// az egyik résztvevő szálban:
b.arrive_and_drop();
b.arrive_and_wait(); // (B)

std::osyncstream make_log() {
    std::ostringstream oss;
    return std::osyncstream(oss); // (C)
}
```

34.5.10 Megoldások

34.5.10.1 1. megoldás

a) `std::latch`, például `std::latch ready(1)`: a munkások `wait()`-tel várnak, a betöltő `count_down()`-nal nyit. Azért nem barrier, mert a kapu egyszeri, és nincs ismétlődő fázis; azért nem szemafor, mert nem kapacitást osztunk, hanem egy egyszeri eseményre várunk.

b) `std::counting_semaphore<3> slots(3)`: az `acquire()`/`release()` pár adja a kapacitáskorlátot. Azért nem latch, mert a művelet ismétlődik; azért nem barrier, mert a résztvevők nem fázisonként szinkronizálnak, hanem erőforrásért versenyeznek.

c) `std::barrier rounds(4)`: minden iteráció egy fázis, a résztvevők száma fix. Azért nem latch, mert a latch nem resetelhető; azért nem szemafor, mert nem engedélyt osztunk, hanem mindenkinek meg kell várnia mindenkit.

34.5.10.2 2. megoldás

```
#include <chrono>
#include <cstddef>
#include <mutex>
#include <semaphore>
#include <stop_token>

template <std::ptrdiff_t N>
bool acquire_stop_aware(std::counting_semaphore<N>& sem, std::stop_token st) {
    while (!st.stop_requested()) {
        if (sem.try_acquire_for(std::chrono::milliseconds(10))) {
            if (st.stop_requested()) {   // stop-prioritás: a közben megszerzett
                sem.release();           // engedélyt visszaadjuk
                return false;
            }
            return true;
        }
    }
    return false;
}

template <std::ptrdiff_t N>
class sem_guard {
public:
    explicit sem_guard(std::counting_semaphore<N>& s) : s_(&s) { s_->acquire();
     ↪ }
    sem_guard(std::counting_semaphore<N>& s, std::adopt_lock_t) noexcept :
 ↪  s_(&s) {}
    ~sem_guard() { if (s_) s_->release(); }

    sem_guard(const sem_guard&) = delete;
    sem_guard& operator=(const sem_guard&) = delete;
    sem_guard(sem_guard&& other) noexcept : s_(other.s_) { other.s_ = nullptr; }

private:
    std::counting_semaphore<N>* s_;
};
```

A stop-aware ág használata:

```
if (acquire_stop_aware(io_slots, st)) {
    sem_guard<2> g(io_slots, std::adopt_lock);
    // ...
}
```

A 10 ms a `try_acquire_for` *névleges* várakozási ideje, vagyis a stop-ellenőrzés polling-intervalluma, nem szigorú felső korlát: a szabvány megengedi, hogy az időkorlátos várakozás az ütemezés és az erőforrás-versengés miatt később térjen vissza (C++20 [thread.req.timing]), és a visszatérés után a szálnak még ütemeződnie is kell. Szigorú leállási határidőt csak a platform ütemezési garanciáival együtt lehet vállalni. A stop és az engedélyszerzés egyidejűségekor két kimenet megengedett: ha a szerzés győz, a függvény igazat ad, és a hívó a guarddal visszaadja az engedélyt; ha stop-prioritás kell, a fenti változat a megszerzés után még egyszer megnézi a stopot, és a már megszerzett

engedélyt visszaadja. Az engedélyszám mindkét kimenetnél megmarad. A `semaphore.acquire()` azért nem szakítható meg, mert a szemafor API-jának nincs `stop_token` paramétere; a szál blokkolása a primitív belső várakozásában történik, amelyet kívülről nem lehet „kiszakítani". Az adoptáló konstruktor elengedhetetlen: ha a guard konstruktora újra `acquire`-t hívna, a már megszerzett engedély mellé egy másodikat is foglalnánk. Ellenőrzési feltétel: stop nélkül a függvény a release után igazzal tér vissza; stop kérés után — a névleges intervallum és az ütemezési késés után — hamissal, és a sikeres ágban a guard pontosan egyszer release-el. A teszt ne a 10 ms-os határidőt mérje, hanem az engedélymegmaradást.

Buildjegyzőkönyv (`lab-gcc` profil). Egy 2000 iterációs versenyteszt (`counting_semaphore<1>`, a `release` és a `request_stop` mindkét sorrendben) minden iteráció után pontosan egy engedélyt talált a szemaforban. Normál buildben mind a 2000 eset a stop ágra futott, TSan alatt (`setarch -R`) 750 megszerzés és 1250 stop — mindkét megengedett kimenet előfordult, engedély nem veszett el.

34.5.10.3 3. megoldás

A) A `ready.count_down(3)` előfeltétele, hogy a frissítés ne haladja meg a számlálót; itt a számláló 2, a frissítés 3. Ez előfeltétel-sértés, nem definiált viselkedés. Javítás: `ready.count_down(2)`, vagy ha három külön esemény van, `std::latch ready(3)`.

B) Az `arrive_and_drop()` után a szál véglegesen kimarad a barrierből, és nem hívhatja újra a barrier tagfüggvényeit. A `b.arrive_and_wait()` utána szerződésszegés. Javítás: a drop után a szál ne használja a barriert; ha még részt kell vennie, ne dropoljon, hanem normál `arrive_and_wait`-tel zárja le a fázist, és a leállást külön protokollal kezelje.

C) A visszatérő `osyncstream` a lokális `oss` streambufjára mutat, amely a függvény végén megsemmisül. A visszatérés után az `osyncstream` destruktorának emitja dangling streambufra ír. Javítás: az `oss`-nek túl kell élnie az `osyncstream`-et, például a hívóban hozzuk létre mindkettőt, vagy a függvény előbb `emit`áljon, majd a puffert `std::string`-ként adja vissza. Ellenőrzési feltétel: az `osyncstream` élettartama soha ne haladja meg a mögöttes stream/streambuf élettartamát, és a destructor ne fusson olyan objektumon, amelynek a streambufja már megszűnt.

34.5.11 Összefoglaló

A `counting_semaphore` kapacitást és engedélyt modellez; a `latch` egyszeri, a `barrier` ismétlődő fáziskapu; az `osyncstream` a logrekordok koherenciáját adja. Egyik sem ad automatikus cancellationt vagy fairness-t, és mindegyiknél kritikus a várakozók élettartama: a destruktor előtt nem maradhat blokkolt várakozó. A stopot a várakozásba kell beépíteni – condition variable esetén `stop_token`-nel, szemafor esetén időkorlátos próbálkozással, barrier esetén explicit leállítási fázissal. A primitívek szinkronizációs pontot adnak, de a megosztott adat tulajdonjogát és a hozzáférési szerződést továbbra is a programnak kell biztosítania.

34.6 shared ownership, reclamation és konkurens review

A konkurens C++ egyik leggyakoribb félreértése, hogy a `std::shared_ptr` „thread-safe okos pointer". Ez a mondat két különböző dolgot mos össze: az ownership könyvelésének szinkronizációját és a mutatott objektum állapotának szinkronizációját. Ebben az alfejezetben egy connection registry példáján választjuk szét a kettőt, és megadjuk mindkettőre a szerződést. Az előfeltétel a semaphore, latch, barrier és szinkron stream részben rögzített szinkronizációs eszköztár, a shared_ptr, control

block és weak_ptr részben bevezetett tulajdonlási modell, valamint a Konkurens élettartam és nehéz hibák részben tárgyalt happens-before és adatverseny definíció. Ezeket itt nem írjuk újra, csak felidézzük a szükséges mértékben.

A fejezet állításai a C++20 memóriamodell ismert követelményein alapulnak; a konkrét szabványparagrafusok és a C++26-os javaslatok státuszának ellenőrzése a Feature-test, standard library és build capability, illetve a C++26 és későbbi irányok ellenőrzött státusszal részek eszköztárával, a tanuló laborjában történik. A fejezet nem tartalmaz mért eredményt.

34.6.1 Két külön szerződés: ownership és pointee

Az **ownership** (tulajdonlás) szerződése megmondja, ki felel a mögöttes objektum megszüntetéséért, és mikor. A `shared_ptr` ezt osztott tulajdonlássá teszi: a control block referenciaszámlálója dönti el, mikor fut le a deleter. A **pointee synchronization** szerződése ezzel szemben azt mondja meg, hogy a mutatott objektum belső állapotához hogyan férünk hozzá párhuzamosan: melyik művelet olvas, melyik ír, milyen zár vagy atomi művelet védi, és mi a láthatósági viszony a szálak között.

A két szerződés ortogonális. A `shared_ptr<P>` azt garantálja, hogy a `*p` objektum élettartama alatt nem szűnik meg, amíg legalább egy `shared_ptr` példány rá mutat. Azt viszont nem garantálja, hogy `p->mezo` olvasása és írása két szálon nem versenyez. Ha a `Connection` objektumnak van egy `std::uint64_t bytes_` tagja, és az egyik szál `add_bytes`-ot hív, a másik `snapshot`-ot, akkor a `shared_ptr` semmit nem tesz hozzá a helyességhez: a `bytes_`-re adatverseny van, ami nem definiált viselkedés, függetlenül attól, hogy a tulajdonlás rendben van.

A szerződés megfogalmazásához három feltételt rögzítünk. Egyrészt minden mutálható pointee-állapothoz külön védelmet rendelünk (mutex, atomikus változó vagy dokumentáltan egyszálú használat). Másrészt a `shared_ptr` példányokra vonatkozó műveletek köre pontosan az, amit a control block kezel, semmi több. Harmadrészt a két védelmi réteg sorrendje (lock ordering) rögzített, különben a két szerződés együtt deadlockot termel.

34.6.2 A control block szinkronizációja

A C++20 követelménye szerint a `shared_ptr` tagfüggvényei — beleértve a másoló konstruktort és a másoló értékadást — különböző `shared_ptr` példányokon külön szálakról is hívhatók további szinkronizáció nélkül, még akkor is, ha ezek a példányok ugyanannak az objektumnak a tulajdonjogát osztják. Ennek mechanizmusa a control blockban tartott referenciaszámláló atomikus növelése és csökkentése; az utolsóként csökkentő szál futtatja a deletet, és ez a szál a számláló csökkentésével szinkronizál minden korábbi tulajdonossal.

Két fontos korlátot kell ehhez hozzátenni. Az egyik, hogy a „thread-safe” itt kizárólag a könyvelésre vonatkozik: arra, hogy a számláló nem veszik el, és a destruktor pontosan egyszer fut le. A másik, hogy ez nem jelent lock-free implementációt. A gyakorlatban a legtöbb standard library implementáció atomikus műveletekkel kezeli a számlálót, de a `shared_ptr` mérete és a control block allokációja miatt a másolás költsége nem nulla; teljesítményérzékeny útvonalon ezt a Profilozás, benchmark és cost model rész módszertanával kell megmérni, nem feltételezni.

A gyakorlati következmény a registry esetében: a `std::unordered_map<std::string, std::shared_ptr<Connection>>` tartalmához való hozzáférést saját mutex védi, mert a map belső szerkezete nem thread-safe. A benne tárolt `shared_ptr` érték kimásolása viszont a zár alatt biztonságosan megtehető, és a kimásolt példány a zár elengedése után is érvényes tulajdonjogot ad.

34.6.3 Ugyanaz a shared_ptr példány és az atomic shared_ptr

A garantált biztonság kizárólag **különböző** `shared_ptr` példányokra vonatkozik. Ha két szál ugyanazon `shared_ptr` objektumon hív `reset`-et, értékadást vagy destruktort, az adatverseny a C memóriamodell értelmében, tehát nem definált viselkedés — nem „csak” logikai hiba, és nem javítja semmilyen retry. Ilyenkor a példányt magát kell szinkronizálni, vagy atomikus wrapperre kell váltani.

A C++20 bevezette a `std::atomic<std::shared_ptr<T>>` specializációt, amely egy `shared_ptr` objektumot tesz atomikusan olvashatóvá és írhatóvá. Mellette a `shared_ptr`-re specializált `std::atomic_load`, `std::atomic_store` szabad függvények C++11 óta léteznek, de a C++20ban deprecálttá váltak; új kódban a specializációt használjuk. Fontos korlát: az `atomic<shared_ptr>` jellemzően **nem lock-free**, belső zárat használ, tehát nem tekinthető a mutex kiváltásának nagy forgalmú esetben. Reális használati mintája a ritkán írt, gyakran olvasott konfiguráció publikálása:

```
// config_publish.cpp - ISO C++20
#include <atomic>
#include <memory>
#include <string>

struct Config {
    std::string endpoint;
    int timeout_ms;
};

std::atomic<std::shared_ptr<const Config>> g_config{nullptr};

void publish(std::shared_ptr<const Config> c) {
    g_config.store(std::move(c));          // release szemantika
}

std::shared_ptr<const Config> current() {
    return g_config.load();                // acquire szemantika
}
```

A `store`/`load` alapértelmezett `memory_order_seq_cst`, ami erősebb a szükségesnél; a `release`/`acquire` pár elegendő, és ezt a C++ atomics, atomic_ref és publication rész indokolja. A `const Config` választás szándékos: a publikált objektumot a kiadás után nem módosítjuk, így a pointee synchronization kérdése eltűnik. Ha módosítani kellene, az újabb publikálás új objektummal történik, nem a régi írásával.

34.6.4 Kidolgozott példa: connection registry

A példa három zárolási szintet mutat be: a registry mapjének zárját, a `Connection` belső állapotának zárját és a worker queue zárját. A cél, hogy a `shared_ptr` ownership műveletei soha ne fussanak felhasználói destruktor alatt, és hogy a taszk futtatása soha ne történjen queue-zár alatt.

```
// registry.cpp - ISO C++20, x86-64 System V, GCC 13 vagy Clang 17
#include <condition_variable>
#include <cstdint>
```

```cpp
#include <deque>
#include <functional>
#include <memory>
#include <mutex>
#include <stdexcept>
#include <string>
#include <thread>
#include <unordered_map>
#include <utility>

class Connection {
public:
    explicit Connection(std::string id) : id_(std::move(id)) {}

    const std::string& id() const noexcept { return id_; }

    // pointee state: saját szerződés, saját zár
    void add_bytes(std::uint64_t n) {
        std::lock_guard lk(state_mu_);
        bytes_ += n;
        ++ops_;
    }

    struct Snapshot {
        std::uint64_t bytes;
        std::uint64_t ops;
    };

    Snapshot snapshot() const {
        std::lock_guard lk(state_mu_);
        return Snapshot{bytes_, ops_};   // konzisztens másolat
    }

private:
    const std::string id_;
    mutable std::mutex state_mu_;
    std::uint64_t bytes_ = 0;
    std::uint64_t ops_ = 0;
};

class Registry {
public:
    // Szerződés: c nem üres; a kulcs c->id(). Azonos id esetén csere.
    void add(std::shared_ptr<Connection> c) {
        if (!c) throw std::invalid_argument("Registry::add: null connection");
        std::shared_ptr<Connection> victim;      // a zárblokkon KÍVÜL
        {
            std::lock_guard lk(mu_);
            // Ha a try_emplace kivételt dob (allokáció), c a hívó oldali
            // paraméterben marad, és a zár nélkül szabadul fel.
            auto [it, inserted] = map_.try_emplace(c->id());
            victim = std::exchange(it->second, std::move(c));   // noexcept
```

```
        }
        // A régi owner (ha volt) itt, a zár feloldása után szabadul fel:
        // a destruktora vagy deletere visszahívhat a regiszterbe.
    }

    std::shared_ptr<Connection> find(const std::string& id) const {
        std::lock_guard lk(mu_);
        auto it = map_.find(id);
        if (it == map_.end()) return nullptr;
        return it->second;              // refcount növelés a zár alatt
    }

    void remove(const std::string& id) {
        std::shared_ptr<Connection> victim;
        {
            std::lock_guard lk(mu_);
            auto it = map_.find(id);
            if (it == map_.end()) return;
            victim = std::move(it->second);
            map_.erase(it);
        }
        // victim itt szabadul fel: a registry zárja nélkül
    }

private:
    mutable std::mutex mu_;
    std::unordered_map<std::string, std::shared_ptr<Connection>> map_;
};

class Worker {
public:
    using Task = std::function<void()>;

    void start() { th_ = std::thread([this] { run(); }); }

    void submit(Task t) {
        {
            std::lock_guard lk(mu_);
            if (stop_) return;          // leállítás után elutasítunk
            q_.push_back(std::move(t));
        }
        cv_.notify_one();
    }

    void stop() {
        {
            std::lock_guard lk(mu_);
            stop_ = true;
        }
        cv_.notify_all();
        if (th_.joinable()) th_.join();
```

```
        std::deque<Task> leftover;
        {
            std::lock_guard lk(mu_);
            leftover.swap(q_);
        }
        leftover.clear();                // shared_ptr-ek zár nélkül szabadulnak
    }

    // A destruktor a hibaúton is szabályozottan leállít: a stop() idempotens.
    // Korlát: a Worker-t a tulajdonos szála destruálja, nem a saját munkaszála
    // (self-join esetén a join resource_deadlock_would_occur hibát dobna).
    ~Worker() { stop(); }

private:
    void run() {
        for (;;) {
            Task t;
            {
                std::unique_lock lk(mu_);
                cv_.wait(lk, [this] { return stop_ || !q_.empty(); });
                if (stop_) return;       // drop politika
                t = std::move(q_.front());
                q_.pop_front();
            }
            t();                         // taszk futtatása zár nélkül
        }
    }

    std::mutex mu_;
    std::condition_variable cv_;
    std::deque<Task> q_;
    bool stop_ = false;
    std::thread th_;
};

int main() {
    Registry reg;
    Worker w;
    w.start();

    auto c = std::make_shared<Connection>("db-1");
    reg.add(c);
    w.submit([c] { c->add_bytes(1024); });

    if (auto p = reg.find("db-1")) {
        (void)p->snapshot();
    }

    reg.remove("db-1");
    w.stop();
}
```

Az `add` és a `remove` ugyanazt az elvet követi: a map a zár alatt változik, de egy `Connection` utolsó tulajdonosának elengedése, vagyis a destruktor vagy egy egyéni deleter futása, mindig a zár **feloldása után** történik. Ha az `add` a régi elemet a zár alatt cserélné le (például `insert_or_assign`-nal), és a map volt a régi `Connection` utolsó tulajdonosa, a deleter a `mu_` zár alatt futna. Egy olyan deleter, amely visszahív a regiszterbe (`find`), ugyanazt a nem rekurzív mutexet próbálná megszerezni: önmagával kerülne holtpontba, második szál nélkül. Az `add` ezért a régi ownert a zárblokkon kívül deklarált `victim` változóba mozgatja. A beszúrást `try_emplace` végzi: ha ez allokációs kivételt dob, az új owner még a hívó paraméterében van, nem a félkész map-bejegyzésben. A szerződés szerint a `c` nem lehet üres; erre az `add` `std::invalid_argument`-et dob.

Buildjegyzőkönyv (`lab-gcc` profil). A program figyelmeztetés nélkül fordult és 0 kóddal futott, TSan (`setarch -R`) alatt is. Egy azonos id-t lecserélő próba, amelynek deletere `find`-ot hív, a régi objektumot egyszer szabadította fel (`freed=1`), az eltávolítás után a másodikat (`freed=2`), ASan/UBSan alatt is. Az `insert_or_assign`-os eredeti változat ugyanebben a próbában holtpontra jutott (5 másodperces timeout, 124-es kód). Az üres pointert az `add` `Registry::add: null connection` üzenettel utasította el.

A buildparancs a tanuló laborjában futtatandó; a fejezet nem állítja, hogy a program lefutott vagy hogy a TSan tiszta eredményt adott:

```
g++ -std=c++20 -O1 -g -fsanitize=thread -pthread registry.cpp -o registry_tsan
./registry_tsan
```

A példa három tanulsága. Először: a `find` a `shared_ptr` másolatát a registry zárja alatt készíti el, így a visszaadott példány tulajdonjoga akkor is érvényes, ha közben egy másik szál `remove`-ot hajt végre. Másodszor: a `remove` a `victim`-et a zár elengedése után dobja el, így a `Connection` destruktora nem a registry zárja alatt fut — ez a lock ordering védelme, mert a destruktor elvileg más zárat is foghatna. Harmadszor: a `Worker::run` a taszkot a queue zárján kívül hívja meg, különben a taszk `submit`-hívása önmagával szemben deadlockolna.

34.6.5 Reclamation, ABA és a C++26 kitekintés

A `shared_ptr` referenciaszámlálója egy reclamation stratégia: az objektum akkor szűnik meg, amikor az utolsó tulajdonos elengedi. Ez kényelmes, de két ismert gyenge pontja van. Az egyik a ciklikus hivatkozás, amelyet a `weak_ptr` tör meg — erről a shared_ptr, control block és weak_ptr rész szólt. A másik az, hogy a reclamation nem védi meg a nyers pointert vagy referenciát, amelyet a `shared_ptr` mellől adunk át: ha egy másik szál közben elengedi az utolsó tulajdonost, a nyers hozzáférés danglinggá válik.

Az **ABA** jelenség akkor lép fel, ha egy atomikusan tárolt címre CAS-t hajtunk végre, de a cím időközben felszabadult és ugyanarra a címre új allokáció került. A címérték megegyezik, a jelentés nem. `shared_ptr` esetén ez akkor kerülhető el, ha a tulajdonjogot is atomikusan visszük magunkkal — például `weak_ptr::lock()`-kal vagy `atomic<shared_ptr>`-rel —, nem pedig nyers pointerrel. A lock-free struktúrák ehhez epoch- vagy hazard pointer alapú reclamationt használnak; ezek szabványosítása a C++26 felé irányuló javaslatok témája, amelynek elfogadási és implementálási státuszát itt nem állítjuk, hanem a C++26 és későbbi irányok ellenőrzött státusszal részre utalunk.

34.6.6 TSan, stressz és invariáns snapshot

A ThreadSanitizer (`-fsanitize=thread`) a happens-before gráfot építi fel a memóriahozzáférésekből és a szinkronizációs eseményekből, és jelzi a versenyző párokat. Nem keverhető egy binárison belül AddressSanitizerrel, és a `-O1 -g` kombináció ajánlott hozzá; a futásidejű költség

jelentős. A TSan megtalálja az adatversenyt, de nem találja meg a logikai race-t (például azt, hogy a `stop_` beállítása és a taszk futtatása között elveszik egy értesítés), és nem bizonyítja a deadlock-mentességet sem — legfeljebb a lock-set analízis jelezhet ciklust.

A stresszpróba sok szálat és rövid kritikus szakaszokat használ; jó indikátor, de nem bizonyíték: a hiányzó hiba nem bizonyítja a helyességet. Ami bizonyítható, az az invariáns. A példában a `snapshot()` egyetlen zár alatt adja vissza a `bytes_` és `ops_` értékét. Ha két külön gettert írnánk, a két érték két különböző időállapotból származna, és az `ops_ == 0 && bytes_ > 0` kombináció hamisan jelenhetne meg. A konzisztens pillanatkép nem teljesítményoptimalizálás, hanem szerződés: a fogyasztó egyetlen állapotra következtethet.

34.6.7 Leállítás: shutdown deadlock és resource lifetime

A leggyakoribb shutdown deadlock nem a `shared_ptr`-ben, hanem a leállítás sorrendjében keletkezik. Ha a `stop()` a queue zárját tartva vár a `join`-ra, a worker nem tud belépni a `run` zárblokkjába, és a két szál egymásra vár. Ha a worker a queue zárja alatt futtatja a taszkot, a taszk `submit`-ja ugyanarra a zárra vár — önmagával szemben. Ha a `stop_` írása nem a queue zárja alatt történik, a `wait` predikátuma adatversenyes, és az értesítés elveszhet.

A resource lifetime ugyanilyen fontos. A queue-ban maradó taszkok `shared_ptr` kópiái meghosszabbítják a `Connection` élettartamát a registry elpusztítása után is. A helyes sorrend tehát: stop flag beállítása zár alatt, `notify_all`, `join`, majd a maradék taszkok elengedése zár nélkül — ahogy a példa `Worker::stop`-ja teszi. A fogalmak pontosan: egy `std::thread` objektum **joinable**, amíg nem hívtunk rá `join`-t vagy `detach`-et — függetlenül attól, hogy a szál már befejezte-e a futását. A `join` futó szálra is hívható: addig blokkol, amíg a szál befejeződik. Egy joinable `std::thread` destruktora `std::terminate`-et hív. Ezért a `~Worker` is meghívja a `stop()`-ot: ha a tulajdonos a `start()` után, de a `stop()` előtt kivétel miatt hagyja el a hatókört, a worker így is leáll és összeolvad, mielőtt a `q_`, a `cv_` és a `mu_` megszűnne (a `th_` az utolsó tag, és a destruktor törzse a tagok előtt fut). A `detach` nem megoldás, mert a munkaszál a `this`-t kölcsönzi, és túlélné az objektumot. A stop idempotens, tehát a normál úton hívott explicit `stop()` után a destruktor ismételt hívása ártalmatlan.

Buildjegyzőkönyv (`lab-gcc` profil). Negatív próba: a `main` a `w.start()` után, a `w.stop()` előtt kivételt dob, amelyet egy külső `try/catch` kap el (a `std::set_terminate` a 42-es kóddal jelezné a terminate-et). A javítás előtti, üres destruktorú változat `terminate`-tel, 42-es kóddal állt le; a javított változat `elkapva: ...` üzenettel, 1-es kóddal lépett ki, TSan és ASan/UBSan alatt is jelzés nélkül.

34.6.8 Tipikus hibák

Az első hiba, hogy a `shared_ptr`-t automatikus pointee-védelemnek tekintjük. A javítás: minden mutálható taghoz külön védelmet rendelünk, és ezt a fejlécben dokumentáljuk. A második, hogy ugyanazon `shared_ptr` példányt két szálon `reset`-eljük; itt a példányt kell védeni vagy `atomic<shared_ptr>`-re váltani. A harmadik, hogy a `find` nyers referenciát ad vissza a map elemére a zár elengedése után; helyette `shared_ptr` kópiát kell visszaadni. A negyedik, hogy a destruktor zár alatt fut, és lock ordering ciklust okoz; a javítás a `victim` kiemelése a záron kívülre. Az ötödik, hogy a `wait` predikátum nélküli, és hamis ébresztésre is továbblép; a `wait(lk, pred)` alak kötelező. A hatodik, hogy a stressz- vagy TSan-futás hiányát helyességi bizonyítéknak tekintjük.

34.6.9 Feladatok

1. feladat (magyarázó). Fogalmazd meg pontosan, miért nem adatverseny, ha két szál két különböző `shared_ptr<Connection>` példányon hajt végre másolást ugyanarra a control blockra, és miért adatverseny, ha ugyanazon a példányon hajt végre `reset`-et. Nevezd meg a happens-before éleket.

2. feladat (kódolási). Egészítsd ki a `Registry`-t `clear_all()` művelettel, amely az összes connectiont eltávolítja, és garantálja, hogy a `Connection` destruktorok nem a registry zárja alatt futnak.

3. feladat (diagnosztikai). A `Worker::run` alábbi változata deadlockot okozhat. Azonosítsd a ciklust, add meg a javítást, és mondd meg, melyik TSan-eredmény nem jelezné a hibát.

```
void run() {
    for (;;) {
        std::unique_lock lk(mu_);
        cv_.wait(lk, [this] { return stop_ || !q_.empty(); });
        if (stop_) return;
        auto t = std::move(q_.front());
        q_.pop_front();
        t();                    // a zár még mindig tartva
    }
}
```

4. feladat (kódolási, nehezebb). A `Worker::stop` jelenleg drop politikát követ. Írj olyan `stop(bool drain)` változatot, amely `drain == true` esetén a maradék taszkokat lefuttatja, de biztosítja, hogy a leállítás alatt érkező új `submit` elutasításra kerüljön, és a ciklus véges maradjon.

34.6.10 Megoldások

1. megoldás. Két különböző `shared_ptr` példány esetén a másolás a control block referenciaszámlálóján atomikus növelést végez. Az atomikus read-modify-write művelet a szekvenciális konzisztencia miatt teljes sorrendet ad a két szál műveletei között, így a számláló értéke nem vész el, és az utolsó csökkentő szál pontosan egyszer futtatja a deletet. A két szál között a `synchronizes-with` él az atomikus változón keresztül jön létre, tehát a két példány objektumára vonatkozó műveletek nem versenyeznek. Ugyanazon `shared_ptr` példányon végzett `reset` ezzel szemben a példány belső pointerére és a kontrollblokk-hivatkozására is ír, és ezt a szabvány nem teszi atomikussá; két szál egyidejű írása a C memóriamodell szerint adatverseny, tehát nem definiált viselkedés. Az ellenőrzési feltétel: a válasz nevezze meg a referenciaszámláló atomikus műveletét, a `synchronizes-with` élt és az adatverseny definícióját.

2. megoldás. A kulcs, hogy a zárolt szakasz csak a map módosítását és a `shared_ptr`-ek kiemelését végezze; a felszabadítás a zár elengedése után történjen.

```
void clear_all() {
    std::unordered_map<std::string, std::shared_ptr<Connection>> doomed;
    {
        std::lock_guard lk(mu_);
        doomed.swap(map_);
    }
    // doomed itt szabadul fel: a registry zárja nélkül
}
```

A `swap` után a `map_` üres, a `doomed` pedig a régi tartalmat birtokolja. A `doomed` destruktora a `mu_` elengedése után fut, így a `Connection` destruktorok nem a registry zárja alatt futnak. Az ellenőrzési feltétel: a `clear_all` után a `find` minden kulcsra `nullptr`-t adjon, és a destruktorok hívási sorrendje ne tartalmazzon registry-zárat (TSan + naplózott destruktor mellett ellenőrizhető).

3. megoldás. A ciklus ciklusa: a worker a `mu_` zár alatt futtatja a taszkot; ha a taszk `submit`-ot hív, az ugyanarra a `mu_`-re vár, amelyet a worker már tart, és soha nem enged el — önmagával szembeni deadlock. Ugyanez a `stop` oldaláról is jelentkezhet: ha a taszk `stop`-ot hív, a `stop` a `mu_`-re vár, miközben a `join` a worker szálra vár, amely a taszkban van. A javítás a taszk futtatásának kiemelése a zár alól: a kritikus szakasz csak a queue-ból való kiemelés, a `t()` hívás a `unique_lock` megszűnése után történik. A TSan ezt a hibát nem feltétlenül jelzi, mert nincs benne adatverseny: a queue minden hozzáférése zár alatt van, a holtpont logikai, nem memóriamodell-sértés. Az ellenőrzési feltétel: a válasz nevezze meg a zár birtoklását a `t()` hívás alatt, és mondja ki, hogy a deadlock nem adatverseny.

4. megoldás. A drain politikánál a worker a `stop_` beállítása után is feldolgozza a sorban lévő elemeket, de új elemet nem fogad el, így a ciklus véges. A `submit` a zár alatt ellenőrzi a `stop_`-ot; a `run` a zár alatt csak akkor lép ki, ha `stop_ && q_.empty()`.

```
void submit(Task t) {
    {
        std::lock_guard lk(mu_);
        if (stop_) return;
        q_.push_back(std::move(t));
    }
    cv_.notify_one();
}

void stop_drain() {
    {
        std::lock_guard lk(mu_);
        stop_ = true;
    }
    cv_.notify_all();
    if (th_.joinable()) th_.join();
}

void run() {
    for (;;) {
        Task t;
        {
            std::unique_lock lk(mu_);
            cv_.wait(lk, [this] { return stop_ || !q_.empty(); });
            if (q_.empty()) {          // stop_ && üres: kilépés
                if (stop_) return;
                continue;
            }
            t = std::move(q_.front());
            q_.pop_front();
        }
        t();
    }
}
```

A végesség indoklása: a `stop_` beállítása után minden `submit` a zár alatt elutasításra kerül, tehát a queue hossza monoton csökken, és a ciklus legfeljebb a `stop_` pillanatában a sorban lévő elemek számáig fut. Az ellenőrzési feltétel: `stop_drain` után a `submit` ne módosítsa a queue-t, a `join` pedig előbb térjen vissza, mint a `stop_drain` visszatérése; stressztesztben a taszkok száma legyen nagyobb a szálakénál.

34.6.11 Összefoglaló

A `shared_ptr` két külön szerződést fed: a control block szintjén az ownership könyvelése különböző példányokon szálbiztos, a pointee állapotához viszont semmit nem ad. Ugyanazon `shared_ptr` példány egyidejű módosítása adatverseny; erre a C++20 `atomic<shared_ptr>` specializációja kínál megoldást, jellemzően nem lock-free áron. A registry példa megmutatta, hogy a rétegek szétválasztása — registry-zár, pointee-zár, queue-zár — és a felhasználói destruktorok záron kívüli futtatása adja a helyes sorrendet. A reclamation `shared_ptr`-nél referenciaszámlálás, nyers pointerrel kombinálva nem véd; az ABA és a lock-free reclamation a C++26-os kitekintés témája. A helyesség eszköze a TSan és a stressz mellett az invariáns pillanatkép és a leállítási sorrend explicit szerződése, nem a „thread-safe okos pointer” feltételezése.

35 Coroutine-ok és aszinkron erőforrás-életciklus

35.1 Coroutine mint felfüggeszthető állapotgép

A C eseményvezérelt szolgáltatásokban a legtöbb bonyolultságot nem az algoritmus, hanem az állapot megőrzése okozza: a `poll`/`epoll` ciklus visszatér, a félkész üzenet a pufferen marad, a „hol tartottunk” információt pedig egy `enum` és néhány `struct` tag hordozza. A C++20 coroutine ugyanezt a feladatot fordító által generált állapotgépre bízza. Ez a rész kizárólag a **nyelvi mechanikát** tanítja: hogyan lesz egy függvényből újraindítható állapotgép, mi történik a coroutine frame-ben tárolt lokális állapottal, és mit nem ad hozzá a nyelv. Hogy milyen ütemező, hálózati runtime vagy szálpolitika hajtja a coroutine-t, az későbbi, alkalmazásszintű szerződés (Aszinkron I/O adapter és coroutine-labor (35.5)); itt csak annyit rögzítünk, amennyi a frame és a felfüggesztés megértéséhez kell.

35.1.1 Definició: mikor coroutine egy függvény

Definíció. Egy C++20 függvény akkor és csak akkor *coroutine*, ha a függvénytörzse tartalmaz legalább egy `co_await` kifejezést, `co_yield` kifejezést vagy `co_return` utasítást, és a függvény nem `main`, nem konstruktor, nem destruktor. A besorolás a függvénytörzsre vonatkozik: ha a `co_await` egy belső lambda testében van, akkor a lambda a coroutine, a külső függvény nem.

A fordító ilyenkor nem hagyományos hívást generál, hanem a törzset egy *állapotgéppé* alakítja, amelynek állapotát egy a fordító által előállított *coroutine frame* (más néven activation record) hordozza. A `co_return` a hagyományos `return` helyett kötelező: ha egy coroutine törzsében `return` áll, a program hibás (ill-formed), mert a visszatérés a `promise_type` protokollján keresztül történik.

A feltétel második fele a visszatérési típusra vonatkozik: a típusnak ki kell elégítenie a coroutine-követelményeket, azaz rendelkeznie kell `promise_type`-tal — közvetlenül beágyazott típusként, vagy a `std::coroutine_traits<R, Args...>` specializációján keresztül. A promise típus adja az `initial_suspend`, `final_suspend`, `get_return_object`, `return_void`/`return_value`, `unhandled_exception` és `yield_value` műveleteket. Ha bármelyik hiányzik, a program szintén hibás. Ez a pont azért fontos, mert a „coroutine” nem egy önálló típus, hanem egy *protokoll*: ugyanaz a `co_await` mást jelent egy `Task<int>`-ben és egy `std::generator<int>`-ben.

A `co_yield e` nem újabb primitív, hanem a `co_await promise.yield_value(e)` rövidítése — ezt érdemes fejben tartani, mert a hibakeresésnél a `co_yield` is ugyanazon az await-mechanizmuson megy keresztül.

35.1.2 A felfüggesztési pont és a visszatérés a hívóhoz

Egy `co_await` kifejezés kiértékelése három, a várakozó (awaitable) típuson meghívott műveletből áll:

1. `await_ready()` — ha `true`, nem történik felfüggesztés, azonnal jön az `await_resume()`.

2. `await_suspend(handle)` — a felfüggesztés pillanata; a `handle` a futó coroutine azonosítója. Visszatérési típusa `void` (mindig felfüggeszt), `bool` (a `false` azt jelenti, hogy mégsem függesztünk fel), vagy `coroutine_handle<>` (szimmetrikus átadás egy másik coroutine-nak).
3. `await_resume()` — a folytatás értéke, ez lesz a `co_await` kifejezés eredménye.

A felfüggesztés lényege: a vezérlés **visszatér ahhoz**, aki a coroutine-t életre hívta vagy utoljára `resume()`-olta. Nem történik szálváltás, nem jön létre új szál, nem mozdul a veremmutató egy másik veremre — a coroutine ugyanazon a szá0lon fut tovább, amelyik a `resume()`-t hívja. A `coroutine_handle::resume()` a frame-ben tárolt folytatási ponttól indítja újra a végrehajtást, egészen a következő felfüggesztésig vagy a `co_return`-ig.

A hívó szempontjából a coroutine „visszatérése” kétszer történik meg. Először a függvény meghívásakor: a `get_return_object()` által előállított objektum már akkor a hívóhoz kerül, amikor a törzs még el sem kezdett futni — ezért tud a hívó kezet kapni egy lazán induló coroutine-ra. Másodszor minden felfüggesztésnél: a vezérlés visszaáramlik, de a frame nem semmisül meg, hanem „lefagy”. A hívó a visszakapott objektumon keresztül később újraindíthatja.

35.1.3 A coroutine frame és a lokális állapot élettartama

A frame tipikusan a következőket tartalmazza: a promise objektumot; a paraméterek frame-be másolt példányait (tehát a coroutine paraméterei másolással kerülnek a frame-be, nem a hívó stackjén maradnak); minden olyan lokálist és temporáriust, amelynek élettartama átnyúlik egy felfüggesztési ponton; valamint a folytatási pontot azonosító indexet, amiből a fordító a `switch`-öt generálja.

A frame élettartama a coroutine-létrehozástól a frame megsemmisítéséig tart, **nem** a függvényhívás végéig. A megsemmisítés a `coroutine_handle::destroy()` hívással vagy a frame-et birtokló visszatérési objektum destruktorával történik; ilyenkor lefutnak a hatókörben lévő lokálisok destruktorai és a promise destruktora. A frame-et a `promise_type::operator new` allokálja (ha van), de a szabvány megengedi az allokáció elhagyását (allocation elision), ha a fordító bizonyítani tudja, hogy a frame élettartama szigorúan beágyazódik a hívóéba. Ez implementációfüggő optimalizálás: a „coroutine mindig heapen lakik” állítás nem igaz, és méréssel kell ellenőrizni, nem feltételezéssel.

Ebből következik a legfontosabb élettartam-szabály: **a frame-hez mutató hivatkozás csak addig érvényes, amíg a frame él.** Ha egy coroutine `co_return`-nal visszaad egy referenciát egy frame-lokálisra, vagy egy `std::string_view`-t a frame-ben tárolt bufferre, az a hívó oldalán dangling lesz — pontosan az az eset, amit a temporary/value category szabályoknál (Temporary, value category és dangling (20.4)) már láttunk. Ugyanígy: felfüggesztett coroutine frame-jét megsemmisíteni szabad (ekkor a lokálisok destruktorai lefutnak), de egy *futó* coroutine-t megsemmisíteni, vagy egy már megsemmisített frame-re `resume()`-t hívni nem definiált viselkedés.

35.1.4 initial_suspend és final_suspend előkép

A coroutine életciklusának két végét a promise két művelete szabályozza. Az `initial_suspend()` a törzs első utasítása előtt fut: `std::suspend_always` esetén a coroutine *lazy* módon indul (a hívó azonnal visszakapja a visszatérési objektumot, a test csak az első `resume()`-ra fut), `std::suspend_never` esetén *eager* módon, azonnal futni kezd. A `final_suspend()` a `co_return` után fut: `suspend_always` esetén a frame a végső felfüggesztési ponton marad, `done()` igaz lesz, de a frame csak a `destroy()`-ig él — így a promise-ból kiolvasható az eredmény. `suspend_never`

esetén a frame automatikusan megsemmisül, a `handle` érvénytelenné válik, és a rá hivatkozó objektum dangling.

Ez a két döntés adja a coroutine-típus „karakterét”: a lazy `Task` az `initial_suspend = suspend_always` és `final_suspend = suspend_always` kombinációjával lesz biztonságosan birtokolható, míg egy `std::generator` a végső felfüggesztést használja arra, hogy az utolsó elem is kiolvasható legyen.

35.1.5 Kidolgozott példa: három lépés felfüggesztve

Az alábbi példa szándékosan minimális: nem hálózat, nem ütemező, csak a frame és a `resume()` viszonya. A `Resumable` a frame egyetlen tulajdonosa; a destruktor megsemmisíti a frame-et, a mozgatás átadja a tulajdonjogot.

```
// steps.cpp - C++20: <coroutine>, <cstdio> és <exception> (std::terminate)
#include <coroutine>
#include <cstdio>
#include <exception>

struct Resumable {
    struct promise_type {
        Resumable get_return_object() {
            return
             ↪  Resumable{std::coroutine_handle<promise_type>::from_promise(*this)};
        }
        std::suspend_always initial_suspend() noexcept { return {}; }
        std::suspend_always final_suspend()   noexcept { return {}; }
        void return_void() noexcept {}
        void unhandled_exception() { std::terminate(); }
    };

    using handle_type = std::coroutine_handle<promise_type>;

    explicit Resumable(handle_type h) noexcept : h_(h) {}
    Resumable(const Resumable&)            = delete;
    Resumable& operator=(const Resumable&) = delete;
    Resumable(Resumable&& o) noexcept : h_(o.h_) { o.h_ = {}; }
    Resumable& operator=(Resumable&& o) noexcept {
        if (this != &o) { if (h_) h_.destroy(); h_ = o.h_; o.h_ = {}; }
        return *this;
    }
    ~Resumable() { if (h_) h_.destroy(); }

    // true: van még folytatás; false: a coroutine befejeződött vagy üres
    bool resume() {
        if (!h_ || h_.done()) return false;
        h_.resume();
        return !h_.done();
    }

private:
    handle_type h_{};
```

```
};

Resumable steps(int n) {
    for (int i = 0; i < n; ++i) {
        std::printf("step %d\n", i);
        co_await std::suspend_always{};   // itt visszaadjuk a vezérlést a
         ↪ hívónak
    }
    std::printf("done\n");
}   // implicit co_return

int main() {
    auto r = steps(3);
    while (r.resume())
        std::puts("  ... a hivo kozben fut");
}
```

Fordítás: `g++ -std=c++20 -O2 -Wall -Wextra -Wpedantic -o steps steps.cpp`. A `std::terminate` az `<exception>` fejlécben van deklarálva, ezért azt közvetlenül be kell vonni; nem szabad arra építeni, hogy a `<coroutine>` vagy a `<cstdio>` tranzitívan behúzza. **Buildjegyzőkönyv** (`lab-gcc` profil): a program figyelmeztetés nélkül fordult, és a következő sorrendet adta: a `main` először `step 0`-t, majd a hívó sorát, aztán `step 1`-et, ismét a hívó sorát, `step 2`-t, a hívó sorát, végül `done`-t ír, majd a ciklus kilép, és a destruktor megsemmisíti a frame-et. Figyeljük meg: a hívó sosem „blokkolódik”, a ciklusváltozó `i` és a `n` paraméter a felfüggesztés alatt is a frame-ben él.

35.1.6 A scheduler és az executor nem része a nyelvnek

A C++20 coroutine-mechanizmus kizárólag a következőket adja: a frame allokációját és élettartamát, a felfüggesztés/folytatás primitíveit és a handle-t. **Nem** ad szálat, időzítőt, I/O-t, prioritást, megszakítást (cancellation) vagy futásütemezőt. A `resume()` hívója maga a „scheduler”: az, hogy mikor és melyik szá0lon hívjuk meg, alkalmazásszintű döntés. Ha a `resume()` egy másik szálon történik, mint a korábbi felfüggesztés, akkor a frame állapotának átadása szinkronizációs kérdés, és a szálak közötti publikálás szabályai érvényesek (shared ownership, reclamation és konkurens review (34.6)): a coroutine nem tesz automatikusan atomic-ot, mutexet vagy memóriakerítést. Ugyanígy a `co_await` önmagában nem hálózati művelet: az awaitable `await_suspend`-jében kell regisztrálni az eseményt (például `epoll`-ba, poll/epoll és explicit eseményállapotgép (12.6)), és a kész esemény kezelőjének kell `resume()`-ot hívnia.

35.1.7 Összehasonlítás az explicit C eseményállapgéppel

Nézzük ugyanazt a feladatot C-ben: hossz-előtagos keretek dekódolása nem blokkoló byte-streamen. Az állapot egy `struct`, a folytatási pont egy `enum`, a „felfüggesztés” pedig a függvény visszatérése.

```
/* decoder.h - explicit allapotgep, C17 */
#include <stddef.h>
#include <stdint.h>

enum dec_state { DEC_LEN, DEC_BODY, DEC_DONE, DEC_ERR };

struct decoder {
```

```
    enum dec_state state;
    uint8_t        len;        /* vart torzshossz */
    size_t         got;        /* eddig beolvasott bajt */
    uint8_t        buf[255];
};

/* Visszateres: elhasznalt bajtok szama; 0 = nem tudott tovabblepni. */
static size_t decoder_feed(struct decoder *d, const uint8_t *p, size_t n)
{
    size_t used = 0;
    while (used < n) {
        switch (d->state) {
        case DEC_LEN:
            d->len = p[used++];
            d->got = 0;
            d->state = (d->len == 0) ? DEC_DONE : DEC_BODY;
            break;
        case DEC_BODY:
            while (used < n && d->got < d->len)
                d->buf[d->got++] = p[used++];
            if (d->got == d->len) d->state = DEC_DONE;
            else return used;              /* tobb adat kell: "felfuggesztes"
             ↪  */
            break;
        case DEC_DONE:
        case DEC_ERR:
            return used;                   /* a keret kesz, a hivo dolga */
        }
    }
    return used;
}
```

A megfeleltetés tanulságos, mert a coroutine frame pontosan ezeket a szerepeket tölti be, csak a fordító generálja:

Szerep	Explicit C állapotgép	C++20 coroutine frame
Folytatási pont	`enum dec_state` + `switch`	fordító által generált index + `switch`
Helyi állapot	`struct decoder` tagjai	frame-lokálisok, paramétermásolatok
Felfüggesztés	`return used;` a ciklusból	`co_await` / `co_yield` felfüggesztése
Folytatás	hívó újra meghívja `decoder_feed`-et	hívó `handle.resume()`-ot hív
Tulajdonos	a `struct decoder` gazdája	a visszatérési objektum (pl. `Resumable`)
Élettartam vége	`free`/destruktor a gazdánál	`destroy()` / a birtokló destruktora

A coroutine nem tünteti el a bonyolultságot, hanem **áthelyezi**: a kézi `switch`-t és a tagok életben tartását a fordító végzi, cserébe a frame allokációjáért, a promise-protokollért és a felfüggesztés

élettartam-szabályaiért kell felelni. A C változat explicit, debuggerben könnyen olvasható, és nem igényel allokációt; a coroutine változat lineáris kódként olvasható, de a vezérlésfolyam a frame-ben „rejtőzik”.

35.1.8 Tipikus hibák

A leggyakoribb félreértés, hogy a `co_await` szálat indít. Nem: ugyanazon a szá0lon marad a végrehajtás, amíg valaki más szá0lon nem hív `resume()`-t. Második hiba a `final_suspend` `suspend_never`-re állítása, miközben a hívó továbbra is használja a handle-t — ilyenkor a frame a `co_return` után azonnal megsemmisül, a handle dangling. Harmadik: a coroutine-ból referenciát vagy `string_view`-t visszaadni egy frame-lokálisra. Negyedik: a `Resumable`-hez hasonló típust másolhatóvá tenni — a frame-nek egy tulajdonosa legyen. Ötödik: egy felfüggesztett coroutine-t úgy megsemmisíteni, hogy közben egy másik szál éppen `resume()`-ol rá; ez adatverseny, nem „coroutine-bug”.

35.1.9 Összefoglaló

A coroutine egy fordító által generált, felfüggeszthető állapotgép: a `co_await`/`co_yield`/`co_return` jelenléte teszi azzá, a visszatérési típus `promise_type`-ja szabja meg a viselkedését. A frame a paramétereket, a felfüggesztésen átnyúló lokálisokat és a folytatási pontot hordozza, élettartama a létrehozástól a `destroy()`-ig tart, nem a hívás végéig. A felfüggesztés nem szálváltás: a vezérlés a hívóhoz tér vissza, és a `resume()` folytatja. Az `initial_suspend`/`final_suspend` a két végpont politikája. Ütemező, I/O és cancellation nincs a nyelvben — az a fölötte épített szerződés.

35.1.10 Feladatok

1. feladat (magyarázó). Döntse el mindegyik esetről, coroutine-e, és ha nem, miért hibás a program: (a) `Task<void> f() { co_await g(); }`; (b) `int f() { co_return 0; }`; (c) `void f() { auto l = [](){ co_await std::suspend_always{}; }; }`; (d) `Task<void> f() { return; }`.

2. feladat (kódolási). Egészítse ki a `Resumable` példát úgy, hogy a coroutine `co_return 42;`-vel adjon vissza egy `int` eredményt, amely a befejezés után a birtokló objektumon `value()`-vel kiolvasható. Adja meg a promise módosításait és az élettartam-szerződést.

3. feladat (diagnosztikai). Az alábbi kódrészlet egy `Task`-ot birtokló objektumot mozgat, majd a korábban kimentett handle-t használja. Mutassa meg, hol sérül a frame élettartama, és javítsa a tulajdonlási szerződést:

```
auto owner = make_task();
owner.resume();
auto raw = owner.handle();      // nem tulajdonlo handle
owner = make_task();            // a move-assignment destroy()-olja a regi
 ↪  frame-et
raw.resume();                   // ???
```

35.1.11 Megoldások

1. megoldás. (a) Igen, coroutine: `co_await` a törzsben, a `Task<void>` adja a promise-t. (b) Nem coroutine, hanem hibás program: a `co_return` coroutine-t jelöl, de az `int` visszatérési típusnak nincs `promise_type`-ja, így a követelmény nem teljesül. (c) A belső lambda a coroutine, a külső `f` nem; a lambda visszatérési típusa viszont szintén korlátos, ezt külön ellenőrizni kell. (d) Hibás: coroutine-törzsben a `return` ill-formed, `co_return`-nal kell visszatérni. Ellenőrzési feltétel: minden állítás a törzs és a visszatérési típus kapcsolatára hivatkozik, nem a szintaxisra.

2. megoldás. A promise-ba `int value = 0;` tag és `void return_value(int v) noexcept { value = v; }` kerül a `return_void()` helyére; a birtokló osztály `int value() const { return h_.promise().value; }` hozzáférést ad. A szerződés: `value()` csak akkor hívható, ha a frame még él (a `final_suspend` `suspend_always`), és a coroutine `done()`; a hívó felelőssége ezt ellenőrizni. A `final_suspend`-et tehát nem szabad `suspend_never`-re váltani, különben a promise is a frame-mel együtt megsemmisül. Ellenőrzési feltétel: `done()` nélkül ne olvassuk az értéket, és a mozgatás után a forrás handle-je legyen üres.

3. megoldás. A hiba nem a `resume()` szintaxisában, hanem a tulajdonjogban van: a `raw` nem tulajdonló handle, a `move-assignment` viszont a régi frame `destroy()`-ját futtatja, így a `raw.resume()` egy már megsemmisített frame-re hivatkozik — nem definiált viselkedés, kimenet nem garantált. Javítás: a handle-t csak addig szabad kimenteni, amíg a birtokló objektum él és nem mozgatták; a helyes minta az, hogy a `resume()` mindig a birtoklón keresztül történik, a nyers `coroutine_handle`-t pedig csak olyan awaitable kapja meg, amelynek élettartama bizonyítottan rövidebb a frame-énél. Ellenőrzési feltétel: a javított változatot AddressSanitizerrel futtatva nem jelenhet meg use-after-free, és a `raw` használata előtt a tulajdonos érvényessége ellenőrizhető.

35.2 promise_type, awaiter és coroutine_handle

A coroutine nem „mágikus aszinkronitás”, hanem egy nyelvi szinten előállított állapotgép, amelynek a fordítási szabályai pontosan rögzítik, ki mikor kap vezérlést, hol tárolódik az állapot, és ki felelős a frame felszabadításáért. Ez az alfejezet egy minimális oktatási `Generator` és `Task` típus teljes lifecycle-ját követi végig. A cél nem egy éles framework, hanem annak megértése, hogy a `promise_type`, az awaiter és a `coroutine_handle` milyen szerződést alkot, és ebben ki a frame tulajdonosa. A leírás a C++20 nyelvi mechanizmusára támaszkodik; a konkrét szabványparagrafust, a compiler és a standard library verzióját a laborprofilban rögzített eszközökkel kell ellenőrizni. A rész nem tárgyal schedulert, hálózati runtimet és teljes async I/O implementációt.

35.2.1 A coroutine protokoll mint szerződés

Coroutinenak azt a függvényt nevezzük, amelynek a teste tartalmaz legalább egy `co_await`, `co_yield` vagy `co_return` kifejezést. A függvény visszatérési típusa a **return object** típusa; ezt a típust a compiler a `std::coroutine_traits<R, Args...>::promise_type` révén kapcsolja össze a **promise** típussal, amely alapértelmezésben `R::promise_type`. A promise objektum a coroutine frame-en belül jön létre, és a coroutine teljes élettartama alatt ott él. A return object a hívóhoz kerül, és tipikusan egy `coroutine_handle`-t tárol.

A szerződés kötelező elemei: a promise_type-nak rendelkeznie kell `get_return_object`, `initial_suspend`, `final_suspend` és `unhandled_exception` tagfüggvénnyel, valamint a `co_return` formájától függően `return_void` vagy `return_value` függvénnyel. A `co_yield` a `yield_value`-re képződik le. A `co_await` egy awaitable-en az `await_ready`, `await_suspend`, `await_resume` hármast hívja, vagy ha az awaitable rendelkezik `operator co_await`-tal, annak

eredményén keresztül. A részletek implementációfüggő optimalizációktól függhetnek, de a hívási sorrend nyelvi szabály.

35.2.2 A hívási sorrend levezetése

Legyen a hívás `auto x = f(args);`, ahol `f` coroutine. A fordítás eredménye egy olyan függvény, amely a következő lépéseket végzi:

1. Létrehozza a frame-et: lefoglalja a tárolót a promise-nak, a paraméterek másolatának, a lokálisoknak és a felfüggesztési pontok állapotának.
2. Lefuttatja a promise_type megfelelő konstruktorát; ha van olyan konstruktor, amely a coroutine paramétereit fogadja, azt, egyébként a default konstruktort.
3. A frame allokációjához a promise_type `operator new`-ját használja, ha létezik; különben a globális `::operator new`-t. Allokációs hiba esetén a `get_return_object_on_allocation_failure` hívható, ha definiált.
4. Meghívja a `promise.get_return_object()`-ot. Az eredmény egy ideiglenes objektum, amely a coroutine első felfüggesztésekor kerül a hívóhoz; addig a frame-ben tárolódik.
5. Kiértékeli a `co_await promise.initial_suspend()` kifejezést. Ha az awaiter `await_ready()`-je igaz, a coroutine nem függesztődik fel, és azonnal fut tovább. Ha hamis, a coroutine felfüggesztődik, és a hívó megkapja a return objectet.
6. Fut a coroutine teste. Minden `co_await`-nél lefut az awaiter protokoll; minden `co_yield` a `yield_value`-t hívja, majd felfüggeszt.
7. `co_return` esetén a `return_value` vagy `return_void` fut, majd a frame-ben lévő lokálisok destruktorai, végül a `co_await promise.final_suspend()`.
8. Ha a final_suspend awaiter `await_ready()`-je igaz, a frame automatikusan felszabadul, és a handle érvénytelenné válik. Ha hamis, a coroutine a végpontján felfüggesztve marad, a frame életben van.
9. Ha a testben kivétel keletkezik, a lokálisok destruktorai után a `promise.unhandled_exception()` fut, majd a vezérlés a final_suspend-hez kerül, mintha `co_return` történt volna.

Ez a sorrend magyarázza, miért fontos az `initial_suspend` és a `final_suspend` választása: az előbbi azt dönti el, hogy a coroutine a hívás után azonnal fut-e, az utóbbi pedig azt, hogy a frame automatikusan felszabadul-e, vagy a return object kezében marad.

35.2.3 promise_type és a return object

A `promise_type` a coroutine „belső objektuma”: a `get_return_object()` állítja elő a hívó által látott objektumot. Egy minimális generatorban ez a következő:

```
struct promise_type {
    std::optional<T> current;
    Generator get_return_object() {
        return
         ↪  Generator{std::coroutine_handle<promise_type>::from_promise(*this)};
    }
    // ...
};
```

A `from_promise(*this)` egy `coroutine_handle<promise_type>`-ot ad, amely a frame-re mutat. A return object ezt a handle-t tárolja. A return object élettartama a hívóé: ha a coroutine az első felfüggesztés előtt kivételt dob, a return object destruktora fut, és a frame felszabadul. A

promise-ban tárolt adatok (például a `current`) a frame részei, ezért azok élettartama a frame élettartamához kötött.

A return objectnek tisztáznia kell a tulajdonjogot. Ha másolható, és minden példány a destruktorában `destroy()`-t hív, kettős felszabadítás lesz. A helyes minta a move-only return object: a move constructor `std::exchange`-dzsel nullázza a forrás handle-t, a destruktor csak nem nulla handle esetén hív `destroy()`-t.

35.2.4 initial_suspend és final_suspend

Az `initial_suspend()` egy awaitable-t ad vissza. A `std::suspend_always` esetén a coroutine a hívás után azonnal felfüggesztődik, és a return object a hívóhoz kerül; ez a pull-alapú generator tipikus választása. A `std::suspend_never` esetén a coroutine azonnal fut az első `co_await`-ig vagy a végéig; ez az eager task jellemzője.

A `final_suspend()` hasonlóan awaitable-t ad. Ha `suspend_always`, a coroutine a végpontján felfüggesztve marad, a frame életben van, és a return object destruktora szabadítja fel. Ha `suspend_never`, a frame a coroutine végén automatikusan felszabadul, és a handle érvénytelenné válik; ilyenkor a return objectnek nem szabad `destroy()`-t hívnia. A két választás nem keverhető össze: a `done()` igaz lehet úgy is, hogy a frame már nem létezik.

35.2.5 Az awaiter három kérdése

A `co_await expr` kiértékelése az awaiteren három kérdést tesz fel. Az `await_ready()` eldönti, hogy szükség van-e felfüggesztésre; ha igaz, a coroutine nem függesztődik fel, és rögtön az `await_resume()` következik. Ha hamis, az `await_suspend(handle)` fut. Ennek visszatérési típusa lehet `void` (felfüggesztés), `bool` (igaz: felfüggesztés, hamis: folytatás), vagy egy `coroutine_handle` (a megadott coroutine folytatódik, a jelenlegi felfüggesztve marad). Végül az `await_resume()` adja a `co_await` kifejezés értékét; ez dobhat is.

A `std::suspend_always` mindhárom függvénye triviális: `await_ready()` hamis, `await_suspend()` üres, `await_resume()` üres. A `std::suspend_never` `await_ready()`-je igaz. Egy saját awaiterrel láthatóvá tehető a sorrend:

```
struct LoggingAwaiter {
    const char* name;
    bool await_ready() const noexcept {
        std::printf("ready: %s\n", name);
        return false;
    }
    void await_suspend(std::coroutine_handle<>) const noexcept {
        std::printf("suspend: %s\n", name);
    }
    void await_resume() const noexcept {
        std::printf("resume: %s\n", name);
    }
};
```

A `co_await LoggingAwaiter{"pont"}` először `ready`, majd `suspend` üzenetet ír, felfüggeszt, és a következő `resume()` után `resume` üzenetet. A `coroutine_handle<>` típusú paraméter elfogadja a típusos handle-t is, mert van konverzió.

35.2.6 return_value, return_void és unhandled_exception

A `co_return;` a `return_void`-ot hívja, a `co_return x;` a `return_value(x)`-et. A kettő kölcsönösen kizárja egymást: ha mindkettő definiált, a program hibás; ha a `co_return` formájához nem tartozik megfelelő függvény, szintén hiba. A `return_value` általában a promise-ba mozgatja az értéket, a `return_void` pedig lezárja a coroutine-t.

Az `unhandled_exception` akkor fut, ha a coroutine testében kivétel keletkezik. Két alapstratégia van: `std::terminate()` (a kivétel nem hagyja el a coroutine-t), vagy `std::current_exception()` eltárolása a promise-ban, hogy a return object később újradobja. Utóbbi esetén a return objectnek dokumentálnia kell, mikor és hogyan lehet a hibát kiolvasni. A kivétel útja mindkét esetben a final_suspend-hez vezet, a lokálisok destruktorai után.

35.2.7 coroutine_handle: resume, done, destroy

A `std::coroutine_handle<P>` egy olcsó, pointer méretű fogantyú a frame-re. A `resume()` folytatja a felfüggesztett coroutine-t; a `done()` igaz, ha a coroutine elérte a végpontját (a final_suspend felfüggesztése után); a `destroy()` futtatja a frame destruktorait, majd felszabadítja a tárolót. A `destroy()` csak felfüggesztett coroutine-on hívható: ha a coroutine éppen fut, a viselkedés nem definiált. Ha a final_suspend `suspend_never` volt, a frame már felszabadult, és a handle érvénytelen; a `destroy()` ilyenkor szintén nem definiált.

A handle nem tulajdonos önmagában. A `resume()` és a `destroy()` felelőssége a return objecté, amely a handle-t tárolja. A handle másolása nem ad tulajdonjogot; a `from_promise` és a `promise()` a frame-re mutató kapcsolatot biztosítja, de nem rendelkezik az élettartamról.

35.2.8 Allokáció, deallokáció és a frame tulajdonosa

A frame allokációja a promise_type `operator new`-jával történik, ha van ilyen; a felszabadítás a `operator delete`-tel. A `operator new` megkaphatja a coroutine paramétereit is, ami lehetővé teszi például egy arena használatát. Allokációs hiba esetén a `get_return_object_on_allocation_failure` hívható, ha a promise_type definiálja; ellenkező esetben a viselkedés olyan, mintha az allokáció kivételt dobott volna.

A frame mérete és elrendezése nem hordozható: függ a promise típusától, a paraméterektől, a lokálisoktól és a felfüggesztési pontoktól. A compiler a frame allokációját el is hagyhatja, ha a coroutine élettartama a hívóhoz kötött és a méret ismert; ez implementációfüggő optimalizáció, nem nyelvi garancia. A frame igazítása a benne lévő objektumok maximális igazításához igazodik.

A frame egyértelmű tulajdonosa a return object, amely a handle-t tagként tárolja, move-only, és a destruktorában `destroy()`-t hív. A `coroutine_handle` önmagában nem tulajdonos; a `destroy()` meghívása a return object szerződése. Ha a coroutine a return objectnél tovább él (például egy külső scheduler tartja a handle-t), a tulajdonjogot dokumentálni kell, és a `destroy()`-t annak kell hívnia, aki az élettartamot lezárja.

35.2.9 Kidolgozott példa: Generator és Task

A következő példa egy pull-alapú `Generator<int>`-ot és egy eager `Task`-ot mutat. A `Generator` `initial_suspend`-je `suspend_always`, `final_suspend`-je szintén; a frame tulajdonosa a `Generator`. A `Task` `initial_suspend`-je `suspend_never`, `final_suspend`-je `suspend_always`; a frame tulajdonosa a `Task`. A kód C++20-ra épül, és a laborban a rögzített compilerrel fordítandó; futási eredményt itt nem állítok.

```cpp
// coro_lab.cpp
#include <coroutine>
#include <cstdio>
#include <exception>
#include <optional>
#include <utility>

template <class T>
class Generator {
public:
    struct promise_type {
        std::optional<T> current;

        Generator get_return_object() {
            return
             ↪  Generator{std::coroutine_handle<promise_type>::from_promise(*this)};
        }
        std::suspend_always initial_suspend() noexcept { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        void return_void() noexcept {}
        void unhandled_exception() { std::terminate(); }
        std::suspend_always yield_value(T v) {
            current = std::move(v);
            return {};
        }
    };

    explicit Generator(std::coroutine_handle<promise_type> h) noexcept : h_(h)
     ↪  {}
    Generator(Generator&& other) noexcept : h_(std::exchange(other.h_, nullptr))
 ↪  {}
    Generator& operator=(Generator&& other) noexcept {
        if (this != &other) {
            if (h_) h_.destroy();
            h_ = std::exchange(other.h_, nullptr);
        }
        return *this;
    }
    Generator(const Generator&) = delete;
    Generator& operator=(const Generator&) = delete;
    ~Generator() { if (h_) h_.destroy(); }

    bool next() {
        if (!h_ || h_.done()) return false;
        h_.resume();
        return !h_.done();
    }
    const T& value() const { return *h_.promise().current; }
    bool done() const { return !h_ || h_.done(); }

private:
    std::coroutine_handle<promise_type> h_;
```

```
};

class Task {
public:
    struct promise_type {
        std::exception_ptr error;

        Task get_return_object() {
            return
             ↪  Task{std::coroutine_handle<promise_type>::from_promise(*this)};
        }
        std::suspend_never initial_suspend() noexcept { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        void return_void() noexcept {}
        void unhandled_exception() noexcept { error = std::current_exception();
         ↪  }
    };

    explicit Task(std::coroutine_handle<promise_type> h) noexcept : h_(h) {}
    Task(Task&& other) noexcept : h_(std::exchange(other.h_, nullptr)) {}
    Task& operator=(Task&& other) noexcept {
        if (this != &other) {
            if (h_) h_.destroy();
            h_ = std::exchange(other.h_, nullptr);
        }
        return *this;
    }
    Task(const Task&) = delete;
    Task& operator=(const Task&) = delete;
    ~Task() { if (h_) h_.destroy(); }

    void resume() { if (h_ && !h_.done()) h_.resume(); }
    bool done() const { return !h_ || h_.done(); }
    void rethrow_if_error() {
        if (h_ && h_.promise().error)
         ↪  std::rethrow_exception(h_.promise().error);
    }

private:
    std::coroutine_handle<promise_type> h_;
};

Generator<int> iota(int from, int to) {
    for (int i = from; i < to; ++i)
        co_yield i;
}

Task print_after_suspend() {
    std::puts("A: elso szakasz");
    co_await std::suspend_always{};
    std::puts("B: folytatas resume utan");
}
```

```
int main() {
    auto g = iota(0, 3);
    while (g.next())
        std::printf("gen: %d\n", g.value());

    auto t = print_after_suspend();
    std::puts("main: a task elso felfuggesztes utan");
    t.resume();
    t.rethrow_if_error();
}
```

A fordításhoz például:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o coro_lab coro_lab.cpp
# vagy
clang++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -o coro_lab coro_lab.cpp
```

A használt compiler és standard library verziójától függően a coroutine támogatás külön kapcsolóhoz kötött lehet; ezt a laborprofilban kell ellenőrizni. A példa nem állítja, hogy bármely konkrét fordítóval tesztelt.

35.2.10 Tipikus hibák és javításuk

Az első hiba a **kettős felszabadítás**: a return object másolható, és mindkét példány `destroy()`-t hív. Javítás: move-only szemantika, `std::exchange`, és a másolás törlése.

A második a **`final_suspend = suspend_never` és `destroy()` együtt**: a frame a coroutine végén automatikusan felszabadul, a handle érvénytelen, a `destroy()` nem definiált. Javítás: `suspend_always`, ha a return object kezeli a frame-et; vagy a return object ne hívjon `destroy()`-t, ha a frame automatikusan felszabadult.

A harmadik a **dangling referencia a promise-ban**: a promise egy hívói lokálisra mutató referenciát tárol, amely a coroutine továbbélésekor érvénytelenné válik. Javítás: érték szerint tárolni, vagy dokumentált élettartam-szerződést adni.

A negyedik a **handle elmentése `await_suspend`-ben, majd a frame felszabadítása**: a későbbi `resume()` érvénytelen frame-re hív. Javítás: a tulajdonjog és az élettartam egyértelmű rendezése; a handle-t csak addig szabad megtartani, amíg a frame él.

Az ötödik a **kivétel elnyelése**: az `unhandled_exception` nem tárolja el a hibát, így a hívó nem értesül róla. Javítás: `std::current_exception()` eltárolása és a return objectben történő újradobás, vagy tudatos `std::terminate`.

35.2.11 Feladatok

1. **Magyarázó feladat.** Írja le a hívási sorrendet arra a coroutine-ra, amelynek `initial_suspend`-je `suspend_always`, a testében egy `co_await std::suspend_always{}` van, majd `co_return;`. Mikor fut `get_return_object`, `return_void`, `final_suspend`, és mikor kerül a return object a hívóhoz?

2. **Kódolási feladat.** Egészítse ki a `Generator<int>`-ot `begin()`/`end()` iterátorral, hogy range-for-ban használható legyen. Az iterátor `operator*`-ja a `value()`-t adja, `operator++`-ja a `next()`-et hívja. Ügyeljen arra, hogy a `Generator` move-only maradjon, és a frame pontosan egyszer szabaduljon fel.

3. **Diagnosztikai feladat.** Adott egy `Task`, amelynek `final_suspend`-je `suspend_never`, a destruktora pedig `destroy()`-t hív. Mi a hiba, milyen tünetet okoz AddressSanitizer alatt, és hogyan javítja?

4. **Kódolási feladat.** Készítsen egy `LoggingAwaiter`-t, amely kiírja az `await_ready`, `await_suspend` és `await_resume` hívásokat, és használja egy `Task`-ban. Magyarázza el a kimenet sorrendjét.

5. **Elemző feladat.** Miért nem tulajdonos a `coroutine_handle`? Milyen esetben viselkedhet mégis tulajdonosként, és milyen szerződést kell ilyenkor dokumentálni?

35.2.12 Megoldások

1. megoldás. A hívás sorrendje: a frame létrehozása és a promise konstrukciója után `get_return_object()` fut; az eredmény a frame-ben tárolódik. Ezután `co_await initial_suspend()` következik; mivel `suspend_always`, a coroutine felfüggesztődik, és a return object a hívóhoz kerül. A hívó `resume()`-ja folytatja a testet; a `co_await std::suspend_always{}` újra felfüggeszt. A következő `resume()` után a `co_return;` a `return_void`-ot hívja, majd a lokálisok destruktorai és a `final_suspend` következnek; mivel ez is `suspend_always`, a coroutine a végpontján felfüggesztve marad. A frame felszabadítása a return object destruktorában történik. Ellenőrzési feltétel: a `done()` a `return_void` után, a final_suspend felfüggesztésekor válik igazzá; a `destroy()` csak ezután hívható.

2. megoldás. Az iterátor egy könnyű, nem tulajdonló nézet:

```
class iterator {
public:
    explicit iterator(Generator* g) : g_(g) {}
    const T& operator*() const { return g_->value(); }
    iterator& operator++() { g_->next(); return *this; }
    bool operator==(std::default_sentinel_t) const { return !g_ || g_->done(); }
private:
    Generator* g_;
};
iterator begin() { next(); return iterator{this}; }
std::default_sentinel_t end() { return {}; }
```

A `begin()` elsőként `next()`-et hív, hogy az első `co_yield` után legyen érték. A `Generator` move-only marad; az iterátor csak addig érvényes, amíg a `Generator` él. Ellenőrzési feltétel: a range-for minden elemet egyszer ad, a `Generator` destruktora egy `destroy()`-t hív, és a `begin()` után a `value()` nem olvas inicializálatlan optionalt.

3. megoldás. A hiba, hogy a `final_suspend = suspend_never` miatt a frame a coroutine végén automatikusan felszabadul, a handle érvénytelenné válik. A destruktor `destroy()` hívása így érvénytelen frame-re mutat; AddressSanitizer alatt tipikusan heap-use-after-free vagy double-free lép fel. A javítás: `final_suspend` legyen `suspend_always`, és a return object destruktora hívja a `destroy()`-t; vagy ha marad a `suspend_never`, a return object ne hívjon `destroy()`-t, és a handle-t ne használja a `done()` után. Ellenőrzési feltétel: a `done()` igaz, a `destroy()` egyszer fut, és a sanitizer nem jelez.

4. megoldás. A `co_await LoggingAwaiter{"pont"}` először `ready: pont`-ot ír, mivel az `await_ready()` hamis, majd `suspend: pont`-ot, és felfüggeszt. A `resume()` után `resume: pont` jelenik meg. A sorrend tehát `ready → suspend → (felfüggesztés) → resume`. Ha az `await_ready()` igaz lenne, a `suspend` kimaradna. Ellenőrzési feltétel: a kimenet pontosan a három hívásnak megfelelő sorrendet mutatja, és a `co_await` kifejezés értéke az `await_resume` visszatérési értéke.

5. megoldás. A `coroutine_handle` egy nem tulajdonló fogantyú: másolható, olcsó, és nem tartalmazza a frame felszabadításának felelősségét. A tulajdonjog a return objecté, amely a handle-t tárolja, és a destruktorában `destroy()`-t hív. A handle akkor viselkedhet tulajdonosként, ha a felhasználó kézzel kezeli a frame élettartamát, például egy külső scheduler tárolja, és dokumentáltan ő hívja a `destroy()`-t. Ilyenkor a szerződésnek rögzítenie kell, ki hívja a `destroy()`-t, mikor szabad a handle-t másolni, és hogy a coroutine felfüggesztett állapotban van-e a felszabadításkor. Ellenőrzési feltétel: a frame egyszer szabadul fel, és a handle egyetlen tulajdonoshoz köthető.

35.2.13 Összefoglaló

A coroutine nyelvi állapotgép, amelynek protokollja a `promise_type`, az awaiter és a `coroutine_handle` hármasára épül. A `get_return_object()` adja a hívó által látott objektumot; az `initial_suspend` és a `final_suspend` dönti el, mikor kerül a return object a hívóhoz, és hogy a frame automatikusan felszabadul-e. Az awaiter `await_ready`/`await_suspend`/`await_resume` sorrendje pontos; a `co_return` a `return_value` vagy `return_void` függvényre képződik, a kivétel az `unhandled_exception`-en keresztül jut el a final_suspend-hez. A frame tulajdonosa a return object, amely move-only módon tárolja a handle-t, és a destruktorában `destroy()`-t hív. A `coroutine_handle` maga nem tulajdonos; a `resume()`, `done()` és `destroy()` helyes használata a frame élettartam-szerződésén múlik.

35.3 Dangling capture, resumption és cancellation

A coroutine nem kerül ki a memóriából akkor, amikor a létrehozó kifejezés véget ér: a fordító egy úgynevezett *coroutine frame*-et (activation record) készít, amely a promise objektumot, a paraméterek másolatait, a felfüggesztési pont indexét és a felfüggesztéseket átélő lokálisokat tartalmazza. A frame élettartama független a létrehozó lexikális hatókörétől, és éppen ez a különbség szüli a coroutine-ok saját hibacsaládját: a nem tulajdonló hivatkozások (referencia, `this`, pointer, `string_view`, iterator, `span`) túlélhetik azt az objektumot, amire mutatnak. Ebben az alfejezetben ezt a jelenséget nevezzük **dangling capture**-nek, és három tipikus alakját, majd a cancellation–completion–destroy versenyhelyzet kezelését tárgyaljuk. Az promise_type, awaiter és coroutine_handle (35.2) részben felépített fogalmakra építünk; a nem tulajdonló nézetek élettartam-szabályait az Borrowed range, dangling és iterator lifetime (30.5) rögzítette, a publikálási mintákat pedig az C++ atomics, atomic_ref és publication (34.3).

35.3.1 A frame mint élettartam-tartomány

A coroutine frame tartalmát a szabvány taxatíven felsorolja: a promise objektum, a coroutine paramétereinek másolatai, a nem statikus tagfüggvényként definiált coroutine esetén a `*this` objektum elérése, valamint azok az automatikus tárolási idejű változók, amelyek élettartama átnyúlik egy felfüggesztési ponton. Fontos árnyalat: a **paraméter másolata** nem azt jelenti, hogy a hivatkozott objektum is bemásolódik. Ha a paraméter típusa `T const&` vagy `T&`, a frame-be kerülő „másolat” maga a hivatkozás (a gyakorlatban egy pointer), a hivatkozott objektum pedig a

hívó oldalon marad. Ugyanez igaz a `*this`-re: a frame nem a closure másolatát, hanem a closure elérési útját őrzi.

Ebből két feltétel adódik, amelyeket minden coroutine-alapú API-nak dokumentálnia kell:

1. **Hívói élettartam-feltétel:** a coroutine által nem tulajdonolt minden objektumnak életben kell maradnia addig, amíg a coroutine azt utoljára használja – nem addig, amíg a létrehozó kifejezés véget ér.
2. **Frame-tulajdonosi feltétel:** a `std::coroutine_handle` nem tulajdonló; a frame-et pontosan egyszer kell megszüntetni (`destroy()`), és csakis felfüggesztett állapotban, más szál részvételének kizárásával. Befejezett (final suspendet elért) coroutine-ra `resume()` hívása, illetve futó coroutine `destroy()`-ja nem definiált viselkedés.

A két feltétel együtt adja a coroutine-típus szerződését; a `task` típus, amelyet az alábbi példák használnak, egy szálon, sorrendben futtatható, nem kezel cancellationt, és a frame-et a destruktora szünteti meg.

```
// task.h -- minimalis, egy szalon, sorrendben futtathato task (oktatasi cel)
#pragma once
#include <coroutine>
#include <exception>
#include <utility>

struct task {
    struct promise_type {
        std::exception_ptr err{};
        std::coroutine_handle<> cont{};

        task get_return_object() {
            return
             ↪  task{std::coroutine_handle<promise_type>::from_promise(*this)};
        }
        std::suspend_always initial_suspend() noexcept { return {}; }

        struct final_awaiter {
            bool await_ready() noexcept { return false; }
            std::coroutine_handle<>
             ↪  await_suspend(std::coroutine_handle<promise_type> h) noexcept {
                auto c = h.promise().cont;
                return c ? c : std::noop_coroutine();   // symmmetric transfer
            }
            void await_resume() noexcept {}
        };
        final_awaiter final_suspend() noexcept { return {}; }

        void return_void() noexcept {}
        void unhandled_exception() noexcept { err = std::current_exception(); }
    };

    using handle_t = std::coroutine_handle<promise_type>;
    handle_t h{};

    task() = default;
```

```
    explicit task(handle_t hh) noexcept : h(hh) {}
    task(task&& o) noexcept : h(std::exchange(o.h, {})) {}
    task& operator=(task&& o) noexcept {
        if (this != &o) { if (h) h.destroy(); h = std::exchange(o.h, {}); }
        return *this;
    }
    task(const task&) = delete;
    task& operator=(const task&) = delete;
    ~task() { if (h) h.destroy(); }

    bool valid() const noexcept { return static_cast<bool>(h); }
    bool await_ready() const noexcept { return false; }
    std::coroutine_handle<> await_suspend(std::coroutine_handle<> caller)
     ↪ noexcept {
        h.promise().cont = caller;
        return h;
    }
    void await_resume() {
        if (h.promise().err) std::rethrow_exception(h.promise().err);
    }
};
```

35.3.2 By-reference paraméter és az életüket túlélő hívó

A leggyakoribb hibaforma: a coroutine referenciaparamétert kap, a hívó pedig egy temporaryt ad át, vagy lokális változót, amely a coroutine befejezése előtt kimegy a hatóköréből. A temporary a *teljes kifejezés* végén megszűnik, nem a coroutine végén.

```
// NEGATIV PELDA -- szandekosan hibas, ne masold a termekkodbe
#include "task.h"
#include <cstdio>
#include <string>

task log_after_gap(std::string const& msg) {   // referenciaparameter
    co_await std::suspend_always{};             // felfuggesztes: a hivo
     ↪ tovabblep
    std::puts(msg.c_str());                     // msg -> a hivo objektuma
}

void caller() {
    auto t = log_after_gap(std::string("kesz"));  // temporary ertek
    // A temporary a kifejezes vegen megszunik; a frame tovabb el,
    // es a kesobbi resume utan a msg hivatkozas dangling.
    (void)t;                                        // későbbi
     ↪ co_await/await_resume: UB
}
```

A hiba nem azért következik be, mert a frame „elfelejtette” másolni az objektumot, hanem azért, mert a paraméter típusa nem kért másolatot. A javítás mintái:

- **Érték szerinti paraméter.** `task log_after_gap(std::string msg)` – a frame a string másolatát tárolja, a hívó bármikor megszűnhet. Költsége egy másolás (vagy mozgatás, ha a hívó `std::move`-val adja át).
- **Tulajdonosi lánc.** A hívó egy `std::shared_ptr<std::string>`-et ad át, vagy a coroutine-t egy olyan objektum tagjaként indítja, amelynek élettartama igazoltan hosszabb.
- **Strukturális garancia.** Az `await_resume` addig nem tér vissza, amíg a coroutine be nem fejeződött, tehát ha a hívó a `co_await` előtt létrehozza az objektumot és csak utána engedi ki, a hivatkozás biztonságos. Ezt a szerződést a `task` típus dokumentációjában kell rögzíteni, mert a nyelv nem kényszeríti ki.

Vigyázat: a `std::string_view` paraméter különösen csapdás, mert érték szerinti átadása is nem tulajdonló. Egy string literálból képzett `string_view` statikus tárolású adatra mutat, ezért biztonságos; egy `std::string`-ből képzett viszont ugyanúgy dangling, mint a referencia.

35.3.3 Coroutine lambda és a closure élettartama

A lambda `operator()`-a nem statikus tagfüggvény, ezért coroutine-ként a frame a closure objektumra mutató elérési utat tárolja, a *capture-öket* viszont a closure tartalmazza. Ha a closure temporary, a `co_await` utáni hozzáférés a megszűnt closure-re mutat.

```
// NEGATIV PELDA -- temporary closure + coroutine lambda
#include "task.h"
#include <string>

auto make_source(std::string path) {
    return [path](int& out) -> task {              // by-value capture a
     ↪  closure-ben
        co_await std::suspend_always{};            // felfuggesztes
        out = static_cast<int>(path.size());       // path: a closure tagja
    };
}

void caller_bad() {
    int n = 0;
    auto t = make_source("/var/log/app.log")(n);   // temporary closure
    // A closure a teljes kifejezes vegen megszunik, a frame tovabb el.
    (void)t;                                       // UB a resume utan
}

void caller_ok() {
    int n = 0;
    auto closure = make_source("/var/log/app.log"); // nevesitett closure
    auto t = closure(n);                            // a closure tulel a
     ↪  task-ot?
    // csak akkor helyes, ha a closure a task befejezeseig el
}
```

A capture fajtája nem számít: a `[path]`, a `[&path]` és a `[path = std::move(path)]` egyaránt a closure-ben lakik, tehát a closure élettartama a korlát. Két helyes stratégia van:

1. **Ne írj coroutine lambdát.** A coroutine legyen szabad függvény, a szükséges adatokat érték szerinti paraméterként kapja; így a frame a paramétereket birtokolja.

2. **Nevesített, igazoltan hosszú életű closure.** A closure-t egy hosszabb életű objektum tagjaként vagy a futtató környezet élettartamához kötve tárold, és a `task` destruktora előbb fusson le, mint a closure-é.

A coroutine lambda ráadásul megnehezíti a diagnózist, mert a hibahely a lambdan belül látszik, a kiváltó ok viszont a hívási kifejezésben van. Ezért a `-Wall -Wextra` mellett érdemes a fordító coroutine-specifikus figyelmeztetéseit is bekapcsolni; ezek támogatása és pontos szövege fordítóverziótól függ, a labor során ellenőrizd.

35.3.4 Felfüggesztett I/O buffer: tulajdonlás és pinning

Aszinkron I/O-nál a kernel vagy az I/O végrehajtó (io_uring, epoll + read, Windows overlapped) a művelet indításakor átveszi a buffer *címét*, és a befejezés pillanatában ír oda. Amíg a művelet „in flight”, a buffer címének stabilnak és a tárterületnek élőnek kell lennie. A coroutine frame ezt nem garantálja: a `destroy()` a frame lokálisait is megszünteti.

```
// NEGATIV PELDA -- felfuggesztett I/O buffer a frame-ben
#include "task.h"
#include <vector>

task<std::size_t> read_chunk(int fd, std::size_t n) {
    std::vector<char> buf(n);                          // frame-ben él
    std::size_t k = co_await async_read(fd, buf.data(), n);   // cím átadva
    co_return k;                                       // buf itt szunik
     ↪ meg
}
// Két hiba:
//  (1) ha a coroutine-t felfuggesztve destroy()-oljuk, a kernel meg irhat
//      a felszabaditott taruletre -> memoria-korrupcio;
//  (2) ha kozben a buf_at noveljuk (resize/push_back), a regisztralt
//      pointer mar a felszabaditott regi puffert mutatja.
```

A helyes szerződés három eleme:

- **Tulajdonlás:** a buffer élettartamát valami olyan objektum garantálja, amely túlél minden befejezési eseményt. Lehet maga a frame (fix méretű `std::array` vagy `std::unique_ptr<std::byte[]>`), lehet az I/O objektum, lehet egy pool.
- **Pinning:** a művelet befejezéséig a bufferre sem átméretezés, sem `free`, sem `mmap`-módosítás nem történhet. Ezt a típussal kell kikényszeríteni (pl. `std::array` és konstans kapacitás), vagy dokumentált invariánssal.
- **Drain a lezárás előtt:** megszakítás esetén először a műveletet kell visszavonni és a befejezést megvárni, és csak utána szabad a frame-et megszüntetni. A „cancellation = azonnali destroy” minta pontosan a fenti memóriakorrupciót okozza.

35.3.5 Cancellation, completion és destroy versenyhelyzet

A coroutine-alapú I/O-ban három szereplő van: a **futtató** (resume), a **befejezési esemény** (completion) és a **tulajdonos** (aki a `task`-ot megszünteti). A versenyhelyzet akkor keletkezik, ha ezek nem ugyanazon a szálon és nem szigorú sorrendben futnak.

35.3.5.1 A destroy–resume verseny

Tipikus hibaminta: a befejezési kezelő egy `coroutine_handle`-t tárol, a `task` destruktora viszont közben `destroy()`-t hív. Eredmény: a kezelő egy felszabadított frame-re hív `resume()`-t. A minimális protokoll:

1. **Megszakítás kérése** (`stop_requested` flag és a művelet visszavonása, pl. `io_uring_prep_cancel` vagy `close(fd)`).
2. **Kivárás:** a tulajdonos megvárja, hogy a befejezési kezelő lefusson és a számláló nullára csökkenjen (atomic fetch_sub + acquire, vagy condition_variable).
3. **Csak ezután `destroy()`**, és a kezelő regisztrációját előbb törölni kell a nyilvántartásból.

A `resume()` és a `destroy()` egymásra nem versenyezhet: a `resume()` csak akkor legális, ha a coroutine felfüggesztett, a `destroy()` csak akkor, ha a coroutine felfüggesztett és senki nem fogja resume-olni. Ezt a gyakorlatban egy állapotváltozóval (pl. `std::atomic<int>` a promise-ban: suspended / running / completed / destroyed) kell kikényszeríteni, és minden átmenetnél compare-exchange-szel kell dönteni. Ha két szál egyszerre próbál resume-olni, az már adatverseny, nem „csak” logikai hiba.

35.3.5.2 Resume thread és publikálás

A `co_await` önmagában nem jelent szálat, schedulert vagy thread-affinitást. Azt, hogy melyik szálon folytatódik a coroutine, a futtató dönti el; ezt a `task`/`executor` típusának szerződése rögzíti. Három tipikus politika: egy szálú eseményhurok (a coroutine mindig ugyanazon a szálon fut), szálpool (bármelyik szálon folytatódhat), dedikált szál coroutine-onként. A középső esetben a coroutine teste által elért megosztott állapotot vagy atomikusan, vagy zárral kell védeni.

A befejezési esemény és a resume közötti *happens-before* nem a `resume()` hívásból származik, hanem a köztük lévő szinkronizációból. Ha a befejező szál release-store-ot végez, a folytató szál acquire-load-dal olvassa az állapotot, a befejező szál által írt buffer tartalma látható lesz. Ha a befejezés egy zárral védett sorba kerül, a zár adja ugyanezt. A `resume()` csupán vezérlésátadás; nem tesz semmit thread-safe-é.

35.3.5.3 Hibaterjedés és RAII a frame-ben

A frame lokálisai akkor is megszűnnek, amikor a coroutine-t `destroy()`-oljuk: a fordító a frame megszüntetésekor lefuttatja a hatókörben lévő objektumok destruktorait, mintha a coroutine teste volna visszatekerve. Ez teszi lehetővé, hogy a cancellation könyvelését RAII-val végezzük.

```
#include <atomic>

struct in_flight {                          // a frame-ben el, a destroy is
 ↪  futtatja
    std::atomic<int>& counter;
    explicit in_flight(std::atomic<int>& c) noexcept : counter(c) {
        counter.fetch_add(1, std::memory_order_relaxed);
    }
    ~in_flight() { counter.fetch_sub(1, std::memory_order_release); }
    in_flight(const in_flight&) = delete;
    in_flight& operator=(const in_flight&) = delete;
};
// hasznalat: task read_chunk(...) { in_flight guard{g_inflight}; ... }
```

A guard nem véd meg a kernellel szemben: ha a művelet még fut, a `destroy()` után is írhat a bufferbe. A guard csak azt garantálja, hogy a tulajdonos *tudja*, van-e folyamatban lévő művelet, és meg tudja várni a drain-t.

A hibaterjedés a promise `unhandled_exception()` → `std::exception_ptr` → `await_resume()` újradobás útvonalon megy. Két buktató: az `exception_ptr`-t a frame megszüntetése előtt ki kell nyerni, különben a hiba csendben elvész; és az újradobás csak egyszer történhet meg, különben a hívó kétszer látja ugyanazt a kivételt. A hibás ágon is ugyanaz a `task` destruktor fut, tehát a frame felszabadul – de csak a drain után.

35.3.6 Gyakori hibák és javításuk

- **`co_await` mint felfüggesztés a hívó vermére.** A hívó stackje nem marad meg; a `co_await` előtti pointerek és referenciák csak akkor érvényesek, ha a hivatkozott objektum máshol él. Javítás: érték szerinti paraméter vagy tulajdonosi lánc.
- **Coroutine lambda temporary closure-rel.** A closure élettartama a korlát. Javítás: szabad függvény, vagy nevesített closure hosszabb élettartammal.
- **Buffer a frame-ben folyamatban lévő I/O közben.** Javítás: fix méretű, nem átméretezhető buffer és drain a destroy előtt.
- **Kétszeres destroy vagy resume a befejezés után.** Javítás: explicit állapotgép compare-exchange-szel, vagy egy tulajdonos, aki a drain után egyedül szünteti meg a frame-et.
- **Kezelő regisztrációban maradt `coroutine_handle`.** Javítás: a `destroy()` előtti deregisztráció, és a befejezési számláló nullára vitele.
- **Feltételezés, hogy a `co_await` szálat vált.** Javítás: a resume-politika dokumentálása és a megosztott állapot védelme.

35.3.7 Összefoglaló

A coroutine frame élettartama független a létrehozó kifejezésétől, ezért minden nem tulajdonló hivatkozás, `this`-elérés, `string_view` és buffer-pointer új élettartam-szerződést igényel. A paraméterek a frame-be másolódnak, de a referenciaparaméter csak a hivatkozást viszi magával; a coroutine lambda a closure-re mutat, nem a closure másolatát őrzi; a felfüggesztett I/O buffer pedig addig marad „pinelve”, amíg a befejezés be nem következett. A cancellation nem egyetlen `destroy()` hívás, hanem három lépés: kérés, drain, megszüntetés. A resume nem szinkronizációs művelet, a *happens-before*-t a befejezés és a folytatás közötti atomikus vagy zárral védett publikálás adja. A frame lokálisainak destruktorai a `destroy()`-nál is lefutnak, ezért a RAII a cancellation könyvelésének elsődleges eszköze.

A fejezetrész nem végzett forrásellenőrzést; a coroutine-mechanika leírása a C++20 modellre támaszkodik, a pontos szabványparagrafusok, a fordítók figyelmeztetéseinek szövege és a `std::generator` (C++23) támogatottsága verziófüggő, a labor során feature-test makróval és fordítási próbával ellenőrizendő.

35.3.8 Feladatok

1. feladat (magyarázó). Három változatban deklaráljuk ugyanazt a coroutine-t: `task f(std::string const&)`, `task f(std::string)` és `task f(std::string_view)`. Melyikben dangling a temporaryval történő hívás `f(std::string("x"))` esetén, és melyikben nem? Indokold a frame tartalma alapján.

2. feladat (kódolási). Alakítsd át a `make_source` coroutine lambdát olyan szabad függvénnyé, amelynek nincs closure-függősége, és add meg a hívó oldalt is. A megoldás forduljon `-std=c++20 -Wall -Wextra` mellett figyelmeztetés nélkül.

3. feladat (diagnosztikai). Egy `read_chunk` coroutine `std::vector<char>` puffert használ, és a `task` destruktora `destroy()`-t hív. Sorolj fel legalább három konkrét eseménysorrendet, amely memóriakorrupcióhoz vezet, és mindegyikhez adj egy-egy minimális javítást.

4. feladat (tervezési, nehezebb). Tervezz a `task` típushoz cancellation-protokollt: milyen állapotokat vezetsz be, milyen atomikus műveletekkel lépsz át köztük, és hogyan biztosítod, hogy a `destroy()` csak a drain után történjen? Fogalmazz meg legalább négy ellenőrzési feltételt, amelyet egy teszt ellenőrizni tud.

35.3.9 Megoldások

1. megoldás. A `std::string const&` változat dangling: a frame a hivatkozást (gyakorlatban pointert) tárolja, a temporary a teljes kifejezés végén megszűnik, a coroutine viszont később folytatódik. A `std::string` változat biztonságos: a paraméter másolata a frame-be kerül, és a frame megszüntetésekor fut le a destruktora. A `std::string_view` változat érték szerinti átadás esetén is nem tulajdonló: a `string_view` egy `std::string` belsejére mutat, ezért a temporary megszűnésével dangling; string literálból képzett `string_view` esetén viszont statikus tárolású adatra mutat, tehát biztonságos. Ellenőrzési feltétel: minden esetben nevezd meg, ki a tulajdonos, és meddig él.

2. megoldás. A closure-t megszüntetjük azzal, hogy a coroutine szabad függvény, a paramétert érték szerint kapja:

```
// lab.cpp - closure nélküli coroutine (2. feladat megoldása)
#include "task.h"
#include <cstdio>
#include <string>

// Érték szerinti paraméter: a frame birtokolja a path másolatát. Az eredményt
// a fejezet task típusa (return_void) helyett a hívó out változója kapja.
task source_size(std::string path, int& out) {
    co_await std::suspend_always{};            // tetszoleges felfuggesztes
    out = static_cast<int>(path.size());
}

int main() {
    int n = -1;
    task t = source_size("/var/log/app.log", n);   // a temporary string a
 ↪  frame-be másolódik
    t.h.resume();                                  // initial_suspend után: a
 ↪  co_await-ig
    t.h.resume();                                  // a co_await után: befejezi
    std::printf("size=%d done=%d\n", n, static_cast<int>(t.h.done()));
    return (n == 16 && t.h.done()) ? 0 : 1;
}
```

A fejezet `task` típusa `return_void`-os, ezért az eredményt a hívó `out` változója kapja. Build: `g++ -std=c++20 -O2 -Wall -Wextra -o lab lab.cpp` (Clang esetén `clang++ -std=c++20 -O2 -Wall -Wextra`). Indoklás: a frame a `path` másolatát tárolja, a hívó a `source_size` visszatérése

után bármikor megszűnhet. Ellenőrzési feltétel: a `task` élettartama alatt a `path` címét senki nem használja a frame-en kívül; a program a `co_await` után is helyes `size()`-ot ad. **Buildjegyzőkönyv** (`lab-gcc` profil): a fenti `task.h`-val `-fsanitize=address,undefined` mellett is figyelmeztetés nélkül fordult, a kimenet `size=16 done=1`, 0-s kód, ASan/UBSan-jelzés nélkül.

3. megoldás. (a) A coroutine felfüggesztve van az `async_read` alatt, a tulajdonos `destroy()`-ol, a kernel később a felszabadított `buf.data()`-ra ír → heap-korrupció. Javítás: drain a destroy előtt (a befejezési számláló nullára vitele) és fix méretű `std::array` buffer. (b) A coroutine fut, közben a `buf` átméreteződik (`resize`), az I/O végrehajtó a régi pointert használja → a régi puffer felszabadul, az új címre nem érkezik adat, a régi címre viszont igen. Javítás: a buffer soha nem méreteződik át a művelet alatt; a méret a coroutine paramétere és konstans kapacitás. (c) A coroutine `destroy()`-ol, miközben a befejezési kezelő még a regisztrációban van és `resume()`-ot hív → felszabadított frame. Javítás: deregisztráció a destroy előtt, a kezelőben `done()`-ellenőrzés és a számláló kezelése. Ellenőrzési feltétel: minden esethez nevezd meg a tulajdonost, az invalidáló műveletet és a hibakimenetet.

4. megoldás. Négy állapot: `suspended`, `running`, `completed`, `destroyed`; az átmeneteket `std::atomic<int>`-en compare-exchange végzi. A cancellation két flag: `cancel_requested` és `completion_count`. A tulajdonos `cancel()`-je beállítja a kérést és visszavonja az I/O-t, majd `drain()`-ben megvárja a `completion_count == 0`-t (acquire), és csak ezután hív `destroy()`-t. A befejezési kezelő csak akkor `resume()`-ol, ha az állapot `suspended` és nem `destroyed`. Ellenőrzési feltételek: (1) `destroy()` után nincs `resume()`; (2) kétszeres `destroy()` nem fordulhat elő; (3) `resume()` és `destroy()` soha nem fut egyszerre ugyanarra a handle-re; (4) a befejezési kezelő regisztrációja a `destroy()` előtt törlődik. Megjegyzés: a teljes, éles futtató implementációja (scheduler, io_uring adapter) kívül esik ezen az alfejezeten; itt csak a szerződés és a minimális protokoll a cél.

35.4 Generator és streaming feldolgozás

A streaming feldolgozás tipikus DevOps-feladat: egy több gigabájtos naplót kell szűrni, oszlopokra bontani, majd az első néhány hibás sort kiírni. Ha a teljes bemenetet előbb `std::vector`-ba olvassuk, a memóriahasználat $O(n)$, a feldolgozás csak a beolvasás után indul. A **pull generátor** ehelyett $O(1)$ állapotot tart, és a fogyasztó diktálja a tempót: ez a backpressure legegyszerűbb formája. Ebben az alfejezetben a generátort mint *single-pass input range*-et szerződésként rögzítjük, megépítjük a minimális C++20 változatot, majd összevetjük a C++23-as `std::generator`-ral. A coroutine gépezetét mint felfüggeszthető állapotgépet a Coroutine mint felfüggeszthető állapotgép alfejezetben már rögzítettük, a felfüggesztéskori érvénytelenedést és a megszakítást a Dangling capture, resumption és cancellation tárgyalta; itt csak a generátor-specifikus következményeket vezetjük le.

35.4.1 A pull generátor mint single-pass állapotgép

Definíció. Egy *pull generátor* olyan coroutine-objektum, amelynek a `promise_type`-ja definiál `yield_value` tagot, a fogyasztó pedig egy `input_range` interfészen keresztül, előrefelé, pontosan egyszer haladva olvassa az elemeket. A szerződés három része:

1. minden `++it` pontosan egy `resume()`, és minden `resume()` legfeljebb egy új elemet tesz elérhetővé;
2. az iteratorok nem független bejárások: egy iterator másolása ugyanahhoz a coroutine-frame-hez mutat;
3. a `begin()` érdemben legfeljebb egyszer hívható meg — a tartomány nem újrajárható.

Az állapotgép négy állapota (a mi implementációnkban):

Állapot	Ide kerül	A következő `resume()` hatása
létrehozva	`initial_suspend` (`suspend_always`)	a coroutine teste elindul
felfüggesztve `co_yield`-nél	`yield_value`	az érték a frame-be kerül, a vezérlés a fogyasztóé
felfüggesztve `final_suspend`-nél	`return_void` vagy `unhandled_exception`	nincs több elem; `done()` igaz
megsemmisítve	`handle.destroy()`	a frame lokálisai felszabadulnak

A `final_suspend` szándékosan `suspend_always`: így a frame a befejezés *után* is él, amíg a tulajdonos meg nem semmisíti, tehát a promise-ban tárolt utolsó érték és a `std::exception_ptr` olvasható marad.

A pull modell megkülönböztetője a push (callback) modellel szemben, hogy a fogyasztó vezérel: `views::take(3)` három elem után egyszerűen nem hív `++`-t, és a termelő soha nem fut tovább. Ez a backpressure, és egyben a végtelen generátorok kezelésének alapja.

35.4.1.1 A yieldelt érték és a referencia élettartama

A `co_yield expr` a fordító szerint `promise.yield_value(expr)` hívás lesz, majd felfüggesztés. Ebből két, minőségileg eltérő eset következik:

- **Értéktároló promise** (`yield_value(T value)`, a frame-ben `T current`): az `expr` bekerül a frame-be (másolás vagy mozgatás). Az `operator*` által adott `const T&` a frame-re mutat, ezért addig érvényes, amíg a frame él *és* a következő `resume()` felül nem írja.
- **Referenciatároló promise** (`co_yield` egy lokálisra, a promise `T&`/`const T&`-et tárol): a frame csak a hivatkozást őrzi. A hivatkozott objektum élettartamát a coroutine lokálisa adja, amely a következő resume-nál megszűnhet vagy megváltozhat — innen ered a klasszikus dangling.

A gyakorlati szabály: a `*it` referenciáját **ne őrizd meg `++it` után**. Ha az értékre a léptetés után is szükség van, másold ki (`T copy = *it;`), vagy a generátor yieldeljen értéket, ne referenciát.

35.4.1.2 Invalidáció a következő resume-nál

A `++it` nem "előrelépés egy tárolóban", hanem a coroutine folytatása. Ennek következményei:

- minden korábban szerzett pointer/referencia a `++it` után **érvénytelen** (dangling vagy elavult érték);
- az iterator másolatai ugyanazt a frame-et léptetik, ezért két "másolat" összefonódik;
- a generátor objektum megsemmisítése az összes iterátorát érvényteleníti — a `for (int v : make_gen())` esetben a temporális generátor élettartama a ciklusra kiterjesztett, de egy `auto it = make_gen().begin();` már dangling.

35.4.2 Minimális C++20 generátor

Az alábbi fejléc önállóan fordítható, és csak a C++20 coroutine és `<iterator>` eszközeit használja:

```
// gen20.hpp - minimális single-pass pull generátor (C++20)
#include <coroutine>
#include <cstddef>
#include <exception>
#include <iterator>
#include <utility>

template <class T>
class Generator {
public:
    struct promise_type {
        T current{};
        std::exception_ptr failure{};

        Generator get_return_object() noexcept {
            return Generator{
                std::coroutine_handle<promise_type>::from_promise(*this)};
        }
        std::suspend_always initial_suspend() noexcept { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        std::suspend_always yield_value(T value) {      // értéktároló promise
            current = std::move(value);
            return {};
        }
        void return_void() const noexcept {}
        void unhandled_exception() noexcept { failure =
         ↪  std::current_exception(); }
    };

    class iterator {
    public:
        using iterator_concept  = std::input_iterator_tag;
        using iterator_category = std::input_iterator_tag;
        using value_type        = T;
        using difference_type   = std::ptrdiff_t;
        using pointer           = const T*;
        using reference         = const T&;

        iterator() noexcept = default;
        explicit iterator(std::coroutine_handle<promise_type> h) : h_{h} {
         ↪  step(); }

        reference operator*()  const noexcept { return h_.promise().current; }
        pointer   operator->() const noexcept { return &h_.promise().current; }
        iterator& operator++() { step(); return *this; }
        void      operator++(int) { step(); }
```

```
        bool operator==(const iterator& o) const noexcept { return h_ == o.h_; }

    private:
        void step() {                          // egyetlen resume + hibaátadás
            if (!h_) return;
            h_.resume();
            if (h_.done()) {
                if (auto e = h_.promise().failure) { h_ = {};
                 ↪  std::rethrow_exception(e); }
                h_ = {};
            }
        }
        std::coroutine_handle<promise_type> h_{};    // nem tulajdonol
    };

    Generator() noexcept = default;
    Generator(Generator&& o) noexcept
        : h_{std::exchange(o.h_, {})}, started_{std::exchange(o.started_,
         ↪  false)} {}
    Generator& operator=(Generator&& o) noexcept {
        if (this != &o) { if (h_) h_.destroy(); h_ = std::exchange(o.h_, {});
                          started_ = std::exchange(o.started_, false); }
        return *this;
    }
    Generator(const Generator&) = delete;
    Generator& operator=(const Generator&) = delete;
    ~Generator() { if (h_) h_.destroy(); }            // korai leállítás
     ↪  cleanupja

    iterator begin() {
        if (!h_ || started_) return iterator{};       // single-pass: másodszor
         ↪  üres
        started_ = true;
        return iterator{h_};
    }
    iterator end() const noexcept { return iterator{}; }

private:
    explicit Generator(std::coroutine_handle<promise_type> h) noexcept : h_{h}
     ↪  {}
    std::coroutine_handle<promise_type> h_{};
    bool started_{false};
};
```

Használat és build:

```
// demo.cpp
#include "gen20.hpp"
#include <iostream>
#include <ranges>
#include <vector>
```

```
Generator<int> evens(int limit) {                    // előfeltétel: limit <
 ↪  INT_MAX
    for (int i = 0; i < limit; i += 2) co_yield i;   // érték bemásolódik a
     ↪  frame-be
}

int main() {
    auto g = evens(10);                              // g az owner
    for (int v : g) std::cout << v << ' ';           // elvárt: 0 2 4 6 8

    auto g2 = evens(10);
    auto view = std::views::all(g2)                  // lvalue -> ref_view, nem
     ↪  másol
              | std::views::transform([](int v) { return v * v; })
              | std::views::take(3);
    std::vector<int> out;                            // C++20 materializálás
    for (int v : view) out.push_back(v);             // elvárt: 0 4 16
    for (int v : out) std::cout << v << ' ';
}

# a laborprofilban rögzített, C++20 coroutine-t támogató fordítóval
c++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -g demo.cpp -o demo
```

Az eredményül kapott sorozatok *elvárt* értékek, nem lefuttatott mérések. A ranges-megfelelést érdemes fordításidejű ellenőrzéssel rögzíteni:

```
static_assert(std::ranges::input_range<Generator<int>>);
```

Ha ez a laborodon nem teljesül, a hibaüzenet megnevezi a hiányzó iterátor-tagot (a `sentinel` és a teljes C++20 iterátorkövetelmény-rendszer szándékosan kimaradt a minimalizmus miatt).

35.4.2.1 A frame tulajdonosa

A coroutine-frame egyetlen tulajdonost kap: a `Generator` objektumot. Ezért a generátor **move-only** — két másolat ugyanazt a frame-et semmisítené meg. Az iterator *nem* tulajdonol; ha tovább él, mint a generátor, dangling. Ugyanez a szabály vonatkozik a coroutine paramétereire is: a paraméterek a frame-be másolódnak, tehát egy `std::istream&` paraméter a frame-ben is hivatkozás marad — a hivatkozott streamnek túl kell élnie a generátort.

35.4.3 Kivétel, korai leállítás és cleanup

Mivel az `unhandled_exception` `noexcept`, a coroutine testében dobott kivételt el kell tárolni, és a fogyasztó oldalán újra kell dobni — ezt teszi a `step()`. Két fontos következmény:

- Ha a fogyasztó `break`-kel kilép, mielőtt a hibát okozó pontig eljutna, a kivétel **soha nem kerül a felszínre**: a `exception_ptr` a frame-mel együtt megsemmisül. Ez nem hiba, hanem a lazy kiértékelés természete — de dokumentálni kell.
- A `~Generator()` `destroy()`-ja a frame lokálisait megsemmisíti, a coroutine testét viszont nem folytatja: `catch` ágak és a `co_yield` utáni kód nem fut le. RAII-szemmel ez helyes viselkedés, a `co_yield` előtt szerzett erőforrás felszabadul.

```
struct Watch { static inline int alive = 0;
               Watch() { ++alive; } ~Watch() { --alive; } };

// Korlátlan, definiált sorozat: unsigned számláló, amely 2^N után kimondottan
// körbefordul (0, 1, ..., UINT_MAX, 0, ...); signed túlcsordulás nincs.
Generator<unsigned> forever() {
    Watch w;                                  // frame-lokális, RAII
    for (unsigned i = 0;; ++i) co_yield i;    // végtelen generátor
}
// for (unsigned v : forever()) { if (v == 2) break; }   -> Watch::alive == 0
```

A „végtelen” generátor szerződése csak akkor teljes, ha a sorozat minden eleme definiált: egy `int` számláló a `INT_MAX` után előjeles túlcsordulással nem definiált viselkedést okozna, ezért a számláló `unsigned`, amelynek körbefordulása a nyelv szerint definiált modulo-aritmetika. (Az `evens` ugyanezért kap előfeltételt: `limit < INT_MAX`, különben az `i += 2` túlcsordulhatna.)

Buildjegyzőkönyv (`lab-gcc` profil). A `gen20.hpp`-vel és a fenti `Watch`-csal készült próba `-fsanitize=address,undefined` alatt `forever_test: OK`-val futott: a `v == 2` utáni `break` után `Watch::alive == 0`, és az `evens(10)` összege 20.

Veszélyes minta a `co_yield` egy lock vagy tranzakció *közben*: a felfüggesztés alatt a lock tartva marad, és ha a fogyasztó eldobja a generátort, a lock a frame megsemmisítésekor szabadul fel — nem a yield után. Ilyen esetben a `co_yield` kerüljön a kritikus szakaszon kívülre.

35.4.4 A `std::generator` C++23-ban és a capability check

A C++23 `std::generator` a szabványos megoldás ugyanerre a szerepre: `<generator>` fejléc, `input_range` és `view` modell, a `co_yield` értéket vagy referenciát is yieldelhet a template-paramétertől függően. A mi kézi változatunkhoz képest a fő nyereség, hogy *view*, ezért közvetlenül illeszthető pipeline-ba, és hogy a value/reference típuslevezetést a szabvány rögzíti.

```
// lines23.cpp - C++23 std::generator
#include <generator>
#include <iostream>
#include <ranges>
#include <string>

std::generator<std::string> lines(std::istream& in) {
    for (std::string line; std::getline(in, line); )
        co_yield line;                  // másolat a frame value slotjába
}

int main() {
    auto v = lines(std::cin)
           | std::views::filter([](const std::string& s) { return !s.empty(); })
           | std::views::take(3);
    for (const auto& s : v) std::cout << s << '\n';
}

# a laborprofilban rögzített, C++23 <generator>-t ismerő fordítóval
c++ -std=c++23 -Wall -Wextra -Wpedantic -O2 -g lines23.cpp -o lines23
```

Capability check: a `<generator>` fejléc jelenléte önmagában nem elég, mert C++20 módban is látszhat, miközben a tartalom nem használható. Ellenőrizz nyelvi módot, fejlécet és library feature-testet együtt:

```
#if __cplusplus > 202002L && defined(__has_include)
#  if __has_include(<generator>)
#    include <generator>
#    define HAVE_STD_GENERATOR 1
#  endif
#endif
```

CMake-oldalon ugyanez a `check_cxx_source_compiles` modullal próbálható ki, a következő próbaforrással:

```
#include <generator>
#include <ranges>
static_assert(std::ranges::input_range<std::generator<int>>);
int main() {}
```

Korlát: ehhez az alfejezethez nem állt rendelkezésre ellenőrzött forrásjegyzet, ezért a `std::generator` pontos `reference`/`value_type` levezetését és a vonatkozó feature-test makró értékét nem tekintem igazoltnak; ezeket a laborban, a rögzített fordítón és standard library-n kell ellenőrizni. A `co_await` mint önálló thread- vagy hálózati runtime, illetve egy teljes coroutine framework itt kifejezetten a témahatáron kívül esik.

35.4.5 Ranges pipeline és materializálás

A generátor akkor igazán hasznos, ha a `views::transform`/`filter`/`take` pipeline eleme. A saját C++20 változatnál az lvalue generátort `std::views::all(g)`-lal kell bevonni (így `ref_view` lesz, nem másolat), a `std::generator` viszont maga is view, ezért közvetlenül `|`-elhető. A különbség élettartam-következménye:

```
auto v = lines(std::cin) | std::views::take(3);   // a view BIRTOKOLJA a
 ↪ generátort
for (auto&& s : v) { /* ... */ }                  // biztonságos
```

Itt a temporális `std::generator` a pipeline-ba kerül, és a `v` élettartamáig él. A kézi változatban ugyanez a `views::all`-lal nem fordul le (nem view), ezért ott külön nevesített generátor kell — ez véletlenül véd a danging ellen. Ha viszont iterátort vagy `subrange`-t mentesz ki a pipeline-ból, és a pipeline megszűnik, az iterátor érvénytelenné válik; ezt a Ranges- és callback-élettartam labor eszközeivel érdemes ellenőrizni.

Materializálás (`std::vector`-ba gyűjtés) akkor helyes, ha a korlát ismert és kicsi: C++23-ban `std::ranges::to<std::vector>()`, C++20-ban `std::ranges::copy(view, std::back_inserter(v))` vagy egyszerű ciklus. Végtelen generátorból `views::take` nélkül a materializálás nem terminál — ez a leggyakoribb streaming-hiba.

35.4.6 Tipikus hibák és javításuk

- **Referencia megtartása léptetés után.** `const auto& x = *it; ++it; use(x);` — dangling. Javítás: `T copy = *it;` a léptetés előtt.
- **Kétszeri `begin()`.** A mi implementációnk csendben üres range-et ad; ez nem hibaüzenet, ezért a fogyasztói kódot kell úgy írni, hogy egyszer járja be.

- **Iterator másolása független bejárásként.** Single-pass: a másolat ugyanazt a frame-et lépteti.
- **Generátor lokálisára yieldelés referenciával.** A következő iterációban a lokális megszűnik vagy felülíródik; értéktároló promise esetén ez nem fordulhat elő.
- **Kivétel elnyelése `break` esetén.** Ha a hiba számít, a fogyasztó ne lépjen ki korábban, vagy a generátor jelezze előre a hibalehetőséget.
- **Lock tartása `co_yield` alatt.** A felfüggesztés meddig tart, a fogyasztó dönti el; a kritikus szakasz ne ölelje át a yieldet.
- **Materializálás végtelen forrásból.** Mindig `views::take`/`views::take_while` a gyűjtés előtt.

35.4.7 Összefoglaló

A pull generátor egy single-pass `input_range`, amelynek állapotgépe a fogyasztó `++` hívásaira lép. Az `operator*` referenciája a frame value slotjára mutat, ezért csak a következő `resume()`-ig érvényes; a frame tulajdonosa a generátor objektum, az iterator nem tulajdonol. A kivételt a promise tárolja, a fogyasztó oldalán kell újradobni; a korai kilépés a frame `destroy()`-jával RAII-helyesen takarít, de a már eltárolt kivételt elnyeli. A C++20 kézi változat kis kóddal működik és véd a view-alapú dangling ellen; a C++23 `std::generator` szabványos view, de használata előtt nyelvi módot, fejlécet és library feature-testet ellenőrizni kell.

35.4.8 Feladatok

1. **Magyarázó.** Egészítsd ki a `gen20.hpp` `Generator` osztályát egy `static_assert`-tel, amely ellenőrzi a `std::ranges::input_range` megfelelést, és indokold, mely iterátor-tagok hiányozhatnak. Írd le, miért *helyes* a single-pass viselkedés egy naplófájl-feldolgozónál.

2. **Diagnosztikai.** Az alábbi kód dangling referenciát produkál; keresd meg a hibát, indokold a frame élettartamával, és add meg a javítást:

```
Generator<const std::string&> bad(std::vector<std::string>& v) {
    for (auto& s : v) co_yield s;   // feltételezzük: referenciát tároló
     ↪ promise
}
```

3. **Kódolási.** Írj `Generator<std::string>`-et C++20-ban, amely egy `std::istream`-ből olvas sorokat, kihagyja az üreseket és a `#`-kel kezdődő kommenteket, majd `views::take(5)`-tel állítsd meg. Igazold, hogy a korai leállás után a frame-lokális `Watch` megsemmisül.

4. **Ranges és materializálás.** Építs pipeline-t a generátorodra: `filter` → `transform` → `take(3)`, majd materializálj `std::vector`-ba. Add meg a C++20-as fallbacket és a C++23-as `ranges::to` változatot, valamint azt a feltételt, amely mellett a materializálás memóriahasználata elfogadható.

35.4.9 Megoldások

1. megoldás. Az ellenőrzés `static_assert(std::ranges::input_range<Generator<int>>);` alakban tehető a fejléc aljára. A megfeleléshez a C++20 `input_iterator`-hez `iterator_concept`, `value_type`, `difference_type`, `reference`, valamint az `iter_move`/`iter_swap` konzisztenciája kell; a `sentinel_for`-hoz pedig a `==`. A minimalizmus miatt ezek egy része hiányozhat, ezért a `static_assert` a dokumentáció része: a hibaüzenet nevezi meg a hiányt. A single-pass viselkedés

helyes, mert egy naplófájl nem újrajárható visszafelé pufferelés nélkül, és a fogyasztó soha nem lép hátra; a memóriakorlát $O(1)$ marad. *Ellenőrzés:* a kód forduljon `-std=c++20` módban, és a `static_assert` ne adjon hibát a labor profiljában.

2. megoldás. A hiba nem is annyira a `co_yield`-ben, mint a szerződésben van: ha a promise `const std::string&`-et tárol, akkor a hivatkozott `s` élettartama a `v`-hez kötött, ami túlélheti a generátort — de a *léptetés* szempontjából a helyzet romlik, ha a ciklusváltozó lokális lenne. A valódi veszély: ha a `for` ciklus egy lokális `std::string`-re yieldelne referenciát, a következő iterációban a lokális megszűnik. Javítás: értéktároló promise (`Generator<std::string>` és `co_yield s;` másol), vagy a hivatkozott konténer élettartamának dokumentált garantálása. *Ellenőrzés:* a javított változatnál a `*it` másolata a `++it` után is helyes értéket ad.

3. megoldás. A generátor teste:

```
Generator<std::string> config_lines(std::istream& in) {
    for (std::string line; std::getline(in, line); ) {
        if (line.empty() || line.front() == '#') continue;
        co_yield line;               // értéktároló promise: biztonságos másolat
    }
}
```

Fogyasztás: `for (auto&& s : config_lines(in) | std::views::take(5))`. A `Watch`-ot a `for` ciklus elé, a coroutine testébe kell tenni; `break` után `Watch::alive == 0` az elvárt érték (nem mért eredmény). *Ellenőrzés:* a `break`-et követően a számláló nulla, és a stream továbbra is használható (a generátor nem zárja le, mert nem tulajdonolja).

4. megoldás. C++20:

```
auto g = source();
auto view = std::views::all(g)
          | std::views::filter([](const std::string& s) { return s.size() > 3;
          ↪ })
          | std::views::transform([](const std::string& s) { return s.substr(0,
          ↪ 3); })
          | std::views::take(3);
std::vector<std::string> out;
std::ranges::copy(view, std::back_inserter(out));
```

C++23-ban `auto out = view | std::ranges::to<std::vector>();`. *Indoklás:* a materializálás költsége $O(k)$ elem és $O(k)$ memória, ahol k a `take` korlátja; ez csak akkor elfogadható, ha k előre ismert és kicsi. *Ellenőrzés:* a `take` nélküli változat végtelen forráson nem terminál — ezt a teszt a fordítással nem, csak futásidőben (timeouttal) tudja kimutatni.

35.5 Aszinkron I/O adapter és coroutine-labor

A coroutine nyelvi mechanizmus, nem I/O-runtime: a `co_await` felfüggeszti a coroutine-t, de a folytatásról, az eseményekről és az erőforrások élettartamáról a program gondoskodik. Ez a labor egy kis `poll`-alapú eseményloophoz készít read/write awaitable-t, hogy látható legyen a teljes szerződés: ki birtokolja a buffert, meddig él az operation, hogyan lehet cancelálni, és hogyan biztosítható a pontosan egyszeri completion. A `co_await` önmagában nem indít szálat és nem végez hálózati I/O-t; a scheduler, az I/O és a cancellation policy külön implementációs szerződés. A rész a generátorok felfüggesztési mintájára, a poll/epoll eseményállapotgépre és a jthread cancellation fogalmaira épül, de azokat nem ismétli meg.

35.5.1 Az adapter mint szerződés

Az aszinkron I/O adapter feladata, hogy egy operációs rendszerbeli eseményt coroutine-felfüggesztéssé és folytatássá alakítson. Ehhez négy fogalmat kell pontosan rögzíteni.

Operation (művelet): egy regisztrált I/O-kérés, amelyhez fájlleíró, várt esemény, completion-művelet és buffer tartozik. Az operation addig él, amíg a completion vagy a cancellation le nem zárja.

Awaitable: a `co_await` operandusa. Három tagfüggvénye van: `await_ready()` eldöntheti, hogy nincs szükség felfüggesztésre; `await_suspend(handle)` regisztrálja a műveletet és felfüggeszti a coroutine-t; `await_resume()` átadja az eredményt. Az awaitable élettartama a teljes `co_await` kifejezés, beleértve a felfüggesztést is.

Completion: az operation eredményének átadása a coroutine-nak. Pontosan egyszer történhet meg. Utána az operation nem hivatkozható, a hozzá tartozó awaitable pedig hamarosan megsemmisül.

Cancellation: az operation eredmény nélküli lezárása. A coroutine vagy hibát kap az `await_resume()`-ban, vagy a coroutine frame megsemmisül, és az awaitable destruktora deregisztrálja a műveletet.

A szerződés feltételei a következők. Az awaitable élettartamának fednie kell a felfüggesztést; ha a coroutine frame megsemmisül felfüggesztett állapotban, az awaitable destruktorának el kell távolítania az operationt az eseménylooptól. A buffer tulajdonosa a hívó marad; az adapter nem másol, csak pointert tárol, ezért a buffer élettartama legalább a completionig tart. A terminális események — completion, cancel, timeout — közül pontosan egy nyerhet. A versenyt az eseményloop szálán, az operation eltávolításával kell eldönteni: amelyik ág előbb eltávolítja az operationt, az nyer, a másik ág már nem hivatkozhat rá. Végül a shutdown alatt új operation nem regisztrálható, a meglévőket pedig drainelni vagy cancelálni kell; az outstanding operation számlálója ezt teszi ellenőrizhetővé.

A callback, az explicit állapotgép és a coroutine ugyanazt a folyamatot írja le, de más szerződéssel. A callbacknél az állapot egy structban él, és a struct élettartama a cancellation határa. Az explicit állapotgépnél az állapot egy enum és egy struct, a vezérlést switch adja. A coroutine-nál az állapot a coroutine frame-ben van, a vezérlés lineáris, a cancellation a frame és az awaitable élettartamához kötött. A coroutine frame allokálhat, és a `promise_type::operator new` testreszabható, de a coroutine nem feltétlenül gyorsabb a callbacknél.

Szempont	Callback	Explicit állapotgép	Coroutine
Állapot helye	struct	enum + struct	coroutine frame
Vezérlés	visszahívási lánc	switch	lineáris kód
Cancel	a struct élettartama	state flag	frame + awaitable
Allokáció	stack vagy heap	stack vagy heap	frame, testreszabható
Debug	callback-lánc	állapotváltozó	frame és handle

35.5.2 Az oktatási eseményloop és az awaitable

A `Loop` `poll`-t használ, és `Op` rekordokat tart nyilván. Az `Op` tartalmazza a fájlleírót, a várt eseményt, a felfüggesztett coroutine handle-jét, a completion-függvényt és az awaitable-re mutató `owner` pointert. Az awaitable az `Op`-ot saját tagjaként tárolja, tehát az `Op` a coroutine frame-ben él. A `Loop` csak pointert tárol rá, nem birtokolja. Ez teszi lehetővé, hogy a completion után az awaitable destruktora `remove`-olja magát.

A `run()` ciklus minden iterációban `pollfd` tömböt épít és `poll`-ol. Esemény esetén először a completion fut, amely a `read`/`write` rendszerhívást végzi; a döntést ennek eredménye hozza, nem önmagában a `poll`-esemény. Átmeneti hibánál (`EAGAIN`, `EINTR`) a completion hamisat ad, és a művelet regisztrálva marad a következő körre; egyébként a `Loop` eltávolítja a műveletet, és felébreszti a coroutine-t. Az eredményt és a végleges hibát az awaitable `result` és `error` tagja hordozza.

Az alábbi példa egyetlen fájlban, oktatási célra készült. Nem éles runtime: egy szálon fut, és a `Loop` nem szálbiztos. A **descriptorprofil** rögzített: az adapter csak `O_NONBLOCK` módú pipe- vagy socket-fd-vel használható, mert csak így garantált, hogy egy `poll` utáni `read`/`write` nem blokkolja az egész loopot, és egy lassú kimenet (`EAGAIN`) nem állítja meg a többi műveletet. Az awaitable az fd-t kölcsönzi; a tulajdonos a hívó `unique_fd`-je (RAII wrapper file descriptorhoz és lockhoz (21.2)). A `Loop` háromállapotú: nyitott (fogad), leálló (új regisztrációt elutasít, a függőket `ECANCELED`-del lemondja) és leállt. A laborban a fordítás és futtatás a tanuló feladata; a kód a megadott profillal ellenőrizendő.

```
// echo_coro.cpp
// Descriptorprofil: az adapter csak O_NONBLOCK módú pipe/socket fd-vel
// használható. Az awaitable az fd-t kölcsönzi, nem birtokolja.
#include <algorithm>
#include <cerrno>
#include <chrono>
#include <coroutine>
#include <cstddef>
#include <cstdint>
#include <csignal>
#include <cstdio>
#include <cstring>
#include <exception>
#include <fcntl.h>
#include <poll.h>
#include <stdexcept>
#include <unistd.h>
#include <utility>
#include <vector>

// A [RAII wrapper file descriptorhoz] rész unique_fd-je, rövidítve.
class unique_fd {
public:
  unique_fd() noexcept = default;
  explicit unique_fd(int fd) noexcept : fd_(fd) {}
  unique_fd(const unique_fd&) = delete;
  unique_fd& operator=(const unique_fd&) = delete;
  unique_fd(unique_fd&& o) noexcept : fd_(std::exchange(o.fd_, -1)) {}
  ~unique_fd() { reset(); }
  [[nodiscard]] int get() const noexcept { return fd_; }
  void reset(int fd = -1) noexcept {
    if (fd == fd_) return;          // önmagára reset: nincs teendő
    if (fd_ >= 0) (void)::close(fd_);
    fd_ = fd;
  }
private:
  int fd_ = -1;
```

```
};

#ifdef INJECT_TRANSIENT
// Tesztbuild: az első két read és az első két write EAGAIN, majd EINTR hibát
 ↪  ad.
static int injected = 0;
static ssize_t sys_read(int fd, void* b, std::size_t n) {
  if (injected < 2) { errno = injected++ ? EINTR : EAGAIN; return -1; }
  return ::read(fd, b, n);
}
static int injected_w = 0;
static ssize_t sys_write(int fd, const void* b, std::size_t n) {
  if (injected_w < 2) { errno = injected_w++ ? EINTR : EAGAIN; return -1; }
  return ::write(fd, b, n);
}
#else
static ssize_t sys_read(int fd, void* b, std::size_t n) { return ::read(fd, b,
 ↪  n); }
static ssize_t sys_write(int fd, const void* b, std::size_t n) { return
 ↪  ::write(fd, b, n); }
#endif

struct Op {
  std::uint64_t id = 0;        // a Loop adja; a pillanatkép ezt tárolja
  int fd = -1;
  short events = 0;
  std::coroutine_handle<> waiter{};
  // A rendszerhívást végzi; hamis, ha átmeneti hiba miatt újra kell próbálni.
  bool (*complete)(Op*) = nullptr;
  void* owner = nullptr;
  std::chrono::steady_clock::time_point deadline{};   // lejárat (ha
   ↪  has_deadline)
  bool has_deadline = false;
  int cancel_error = 0;        // ECANCELED, ha a Loop lemondta a műveletet
};

class Loop {
public:
  // Nyitott: fogad műveletet. Leálló: nem fogad újat, a függőket lemondja.
  // Leállt: a run() visszatért, nincs függő művelet.
  enum class State { open, stopping, stopped };

  // Hamis, ha a Loop már nem fogad műveletet: ilyenkor a hívó nem függeszt fel.
  [[nodiscard]] bool add(Op* op) {
    if (state_ != State::open) return false;
    op->id = next_id_++;
    ops_.push_back(op);
    return true;
  }

  // Csak a még regisztrált műveletet adja vissza. A megszűnt awaitable
  // destruktora remove-olja az Op-ot, így annak azonosítója nem található.
```

```
Op* lookup(std::uint64_t id) const {
  for (auto* op : ops_)
    if (op->id == id) return op;
  return nullptr;
}

void remove(Op* op) {
  auto it = std::find(ops_.begin(), ops_.end(), op);
  if (it != ops_.end()) ops_.erase(it);
}

// Egyetlen függő művelet lemondása. Az nyer, aki előbb remove-ol: ha az Op
 ↪  már nincs
// regisztrálva (közben befejeződött vagy megszűnt), nincs teendő (false).
bool cancel(std::uint64_t id, int error = ECANCELED) {
  Op* op = lookup(id);
  if (op == nullptr) return false;
  remove(op);
  op->cancel_error = error;
  op->waiter.resume();
  return true;
}

// Lejárt műveletek lezárása ETIMEDOUT-tal. Az azonosítókat előre gyűjtjük,
 ↪  mert a
// resume módosíthatja az ops_ vektort.
void expire(std::chrono::steady_clock::time_point now) {
  std::vector<std::uint64_t> due;
  for (auto* op : ops_)
    if (op->has_deadline && op->deadline <= now) due.push_back(op->id);
  for (auto id : due) (void)cancel(id, ETIMEDOUT);
}

void stop() noexcept {
  if (state_ == State::open) state_ = State::stopping;
}

std::size_t outstanding() const noexcept { return ops_.size(); }
State state() const noexcept { return state_; }

void run() {
  while (!ops_.empty()) {
    if (state_ == State::stopping) { cancel_all(); continue; }
    std::vector<pollfd> pfds;
    std::vector<std::uint64_t> ids;   // nem pointer: az Op közben megszűnhet
    pfds.reserve(ops_.size());
    ids.reserve(ops_.size());
    for (auto* op : ops_) {
      pfds.push_back(pollfd{op->fd, op->events, 0});
      ids.push_back(op->id);
    }
```

```
      // A poll időkorlátja a legközelebbi lejáratig csökken (legfeljebb 10 ms).
      int wait_ms = 10;
      const auto now0 = std::chrono::steady_clock::now();
      for (auto* op : ops_)
        if (op->has_deadline) {
          auto left =
           ↪  std::chrono::duration_cast<std::chrono::milliseconds>(op->deadline
           ↪  - now0).count();
          wait_ms = static_cast<int>(std::max<long long>(0, std::min<long
↪  long>(wait_ms, left)));
        }
      int n = ::poll(pfds.data(), pfds.size(), wait_ms);
      if (n < 0) {
        if (errno == EINTR) continue;
        throw std::runtime_error("poll failed");
      }
      expire(std::chrono::steady_clock::now());   // előbb a lejáratok: az Op
↪  eltűnik, a poll ág nem találja
      for (std::size_t i = 0; i < pfds.size(); ++i) {
        // A stop() a kész kötegen belül is érvényes: a még nem feldolgozott
         ↪  eseményeket nem
        // teljesítjük, a következő iteráció cancel_all()-ja ECANCELED-del zárja
         ↪  le őket.
        if (state_ == State::stopping) break;
        if (pfds[i].revents == 0) continue;
        // Egy korábbi resume ugyanebben a kötegben megszüntethette ezt a
        // műveletet; ekkor nincs completion és nincs resume.
        Op* op = lookup(ids[i]);
        if (op == nullptr) continue;
        // A rendszerhívás eredménye dönt, nem a poll-esemény: átmeneti
        // hibánál (EAGAIN, EINTR) a művelet regisztrálva marad.
        if (!op->complete(op)) continue;
        // Előbb eltávolítjuk, csak utána folytatjuk a coroutine-t.
        remove(op);
        op->waiter.resume();
      }
    }
    state_ = State::stopped;
  }

private:
  // Minden függő műveletet ECANCELED-del zár le: remove, majd resume.
  void cancel_all() {
    while (!ops_.empty()) {
      Op* op = ops_.front();
      remove(op);
      op->cancel_error = ECANCELED;
      op->waiter.resume();
    }
  }

  std::vector<Op*> ops_;
```

```
  std::uint64_t next_id_ = 1;
  State state_ = State::open;
};

struct ReadAwaitable {
  Loop& loop;
  int fd;                       // kölcsönzött, nem birtokolt
  void* buf;
  std::size_t len;
  ssize_t result = 0;
  int error = 0;
  Op op;

  ReadAwaitable(Loop& l, int f, void* b, std::size_t n)
      : loop(l), fd(f), buf(b), len(n) {}

  ~ReadAwaitable() { loop.remove(&op); }

  bool await_ready() const noexcept { return false; }

  bool await_suspend(std::coroutine_handle<> h) {
    op.fd = fd;
    op.events = POLLIN;
    op.waiter = h;
    op.complete = &on_complete;
    op.owner = this;
    if (loop.add(&op)) return true;     // felfüggesztés
    error = ECANCELED;                  // a Loop már nem fogad: azonnal folytat
    return false;
  }

  ssize_t await_resume() {
    if (op.cancel_error) error = op.cancel_error;
    if (error) {
      errno = error;
      return -1;
    }
    return result;
  }

  static bool on_complete(Op* o) {
    auto* self = static_cast<ReadAwaitable*>(o->owner);
    self->result = sys_read(o->fd, self->buf, self->len);
    if (self->result < 0) {
      if (errno == EAGAIN || errno == EWOULDBLOCK || errno == EINTR) return
      ↪  false;
      self->error = errno;
    }
    return true;
  }
};
```

```
struct WriteAwaitable {
  Loop& loop;
  int fd;                        // kölcsönzött, nem birtokolt
  const void* buf;
  std::size_t len;
  ssize_t result = 0;
  int error = 0;
  Op op;

  WriteAwaitable(Loop& l, int f, const void* b, std::size_t n)
      : loop(l), fd(f), buf(b), len(n) {}

  ~WriteAwaitable() { loop.remove(&op); }

  bool await_ready() const noexcept { return false; }

  bool await_suspend(std::coroutine_handle<> h) {
    op.fd = fd;
    op.events = POLLOUT;
    op.waiter = h;
    op.complete = &on_complete;
    op.owner = this;
    if (loop.add(&op)) return true;
    error = ECANCELED;
    return false;
  }

  ssize_t await_resume() {
    if (op.cancel_error) error = op.cancel_error;
    if (error) {
      errno = error;
      return -1;
    }
    return result;
  }

  static bool on_complete(Op* o) {
    auto* self = static_cast<WriteAwaitable*>(o->owner);
    self->result = sys_write(o->fd, self->buf, self->len);
    if (self->result < 0) {
      if (errno == EAGAIN || errno == EWOULDBLOCK || errno == EINTR) return
       ↪  false;
      self->error = errno;
    }
    return true;
  }
};

struct Task {
  struct promise_type {
    Task get_return_object() {
      return Task{std::coroutine_handle<promise_type>::from_promise(*this)};
```

```
    }
    std::suspend_always initial_suspend() noexcept { return {}; }
    std::suspend_always final_suspend() noexcept { return {}; }
    void return_value(int err) noexcept { error = err; }
    void unhandled_exception() noexcept { exception = std::current_exception();
     ↪ }

    int error = 0;                    // 0: siker, egyébként errno
    std::exception_ptr exception;     // a coroutine-ból kiszökő kivétel
  };

  std::coroutine_handle<promise_type> h{};

  explicit Task(std::coroutine_handle<promise_type> handle) : h(handle) {}

  Task(Task&& other) noexcept : h(other.h) { other.h = {}; }

  Task& operator=(Task&& other) noexcept {
    if (this != &other) {
      if (h) h.destroy();
      h = other.h;
      other.h = {};
    }
    return *this;
  }

  Task(const Task&) = delete;
  Task& operator=(const Task&) = delete;

  ~Task() {
    if (h) h.destroy();
  }

  void resume() {
    if (h && !h.done()) h.resume();
  }

  bool done() const noexcept { return !h || h.done(); }

  // Csak befejezett coroutine-ra hívható; kivételt továbbdob.
  int result() const {
    if (h.promise().exception) std::rethrow_exception(h.promise().exception);
    return h.promise().error;
  }
};

// Eredmény: 0, ha a bemenet EOF-ig átment; különben a végleges hiba errno-ja
// (ECANCELED, ha a Loop leállítása mondta le). Az átmeneti hibákat az adapter
// kezeli, ide nem jutnak el.
Task echo(Loop& loop, int in_fd, int out_fd) {
  char buf[4096];
  for (;;) {
```

```
    ssize_t n = co_await ReadAwaitable{loop, in_fd, buf, sizeof buf};
    if (n == 0) co_return 0;                                  // EOF: siker
    if (n < 0) co_return errno;                               // végleges hiba

    std::size_t off = 0;                                      // n > 0: a konverzió
     ↪ értéktartó
    while (off < static_cast<std::size_t>(n)) {
      ssize_t w = co_await WriteAwaitable{
          loop, out_fd, buf + off, static_cast<std::size_t>(n) - off};
      if (w == 0) co_return EIO;                              // pozitív hátralevő
       ↪ hossz, nulla előrehaladás: végleges hiba
      if (w < 0) co_return errno;
      off += static_cast<std::size_t>(w);
    }
  }
}

static bool set_nonblock(int fd) {
  int fl = ::fcntl(fd, F_GETFL);
  return fl >= 0 && ::fcntl(fd, F_SETFL, fl | O_NONBLOCK) == 0;
}

// Egy echo-futás saját pipe-pal; a kimenet egy kölcsönzött, már nem
// blokkoló fd. Minden pipe-vég a unique_fd-k révén pontosan egyszer záródik,
// a korai returnokon is.
[[maybe_unused]] static int run_echo(int out_fd) {
  int fds[2];
  if (::pipe(fds) != 0) return 1;
  unique_fd rd{fds[0]}, wr{fds[1]};
  if (!set_nonblock(rd.get())) return 1;

  Loop loop;
  Task task = echo(loop, rd.get(), out_fd);
  task.resume();

  const char msg[] = "hello from coroutine\n";
  if (::write(wr.get(), msg, sizeof msg - 1) < 0) return 1;
  wr.reset();                                  // EOF jelzése az olvasónak

  loop.run();
  if (!task.done()) return 1;
  if (int err = task.result(); err != 0) {
    std::fprintf(stderr, "echo: %s\n", std::strerror(err));
    return 1;
  }
  return 0;
}

#if !defined(LOOP_CANCEL_TEST) && !defined(LOOP_STOP_TEST)
// A stdout O_NONBLOCK módját RAII őr állítja vissza minden kilépési úton
 ↪ (kivétel esetén is).
struct FlagRestore {
```

```
  int old;
  ~FlagRestore() { (void)::fcntl(STDOUT_FILENO, F_SETFL, old); }
};

int main() {
  // SIGPIPE-policy: a példa write()-ot használ; bezárt olvasójú pipe-nál az
   ↪  alapértelmezett
  // SIGPIPE a folyamatot megszüntetné, mielőtt a Task hibát adhatna. Ezért a
   ↪  folyamat
  // elején ignoráljuk: a write() ekkor EPIPE-pal tér vissza, amely a végleges
   ↪  hibacsatornán
  // (Task::result) jelenik meg. Socketnél alternatíva az MSG_NOSIGNAL a
   ↪  send()-en.
  std::signal(SIGPIPE, SIG_IGN);
  // A stdout fájlleírását a szülővel osztjuk meg: a nem blokkoló módot csak
  // a futás idejére állítjuk be, utána visszaállítjuk.
  int old = ::fcntl(STDOUT_FILENO, F_GETFL);
  if (old < 0 || !set_nonblock(STDOUT_FILENO)) return 1;
  FlagRestore restore{old};
  int rc = 0;
  try {
    rc = run_echo(STDOUT_FILENO);
#ifdef FD_LEAK_TEST
  auto count_fds = [] { int c = 0; for (int fd = 0; fd < 1024; ++fd) if
   ↪  (::fcntl(fd, F_GETFD) >= 0) ++c; return c; };
  int before = count_fds();
  for (int i = 0; i < 100 && rc == 0; ++i) rc = run_echo(STDOUT_FILENO);
  std::fprintf(stderr, "fd before=%d after=%d\n", before, count_fds());
#endif
  } catch (const std::exception& e) {          // Task::result / Loop::run
   ↪  kivétele: dokumentált kilépési kód
    std::fprintf(stderr, "echo: kivétel: %s\n", e.what());
    rc = 3;
  } catch (...) {
    std::fprintf(stderr, "echo: ismeretlen kivétel\n");
    rc = 3;
  }
  return rc;                                   // a FlagRestore destruktora
   ↪  visszaállítja a módot
}
#elif defined(LOOP_STOP_TEST)
// Leállítás függő olvasással: a függő művelet ECANCELED-et kap, a run()
// véges időben visszatér, és a stop után új regisztráció elutasított.
int main() {
  int fds[2];
  if (::pipe(fds) != 0) return 1;
  unique_fd rd{fds[0]}, wr{fds[1]};
  if (!set_nonblock(rd.get())) return 1;
  Loop loop;
  Task pending = echo(loop, rd.get(), STDOUT_FILENO);
  pending.resume();                            // olvasásra vár, adat nem jön
  loop.stop();
```

```
  loop.run();                                   // lemondja, majd visszatér
  int e1 = pending.done() ? pending.result() : -1;
  Task late = echo(loop, rd.get(), STDOUT_FILENO);
  late.resume();                                // a stop után: azonnal elutasítva
  int e2 = late.done() ? late.result() : -1;
  std::printf("pending=%s late=%s outstanding=%zu\n", std::strerror(e1),
              std::strerror(e2), loop.outstanding());
  return (e1 == ECANCELED && e2 == ECANCELED && loop.outstanding() == 0) ? 0 :
   ↪  1;
}
#else
// Két egyszerre kész művelet: az első folytatása megszünteti a másodikat.
#include <optional>
std::optional<Task> victim;
int completions = 0;

Task reader(Loop& loop, int fd, bool kill_victim) {
  char c;
  ssize_t n = co_await ReadAwaitable{loop, fd, &c, 1};
  ++completions;
  std::printf("reader fd=%d n=%zd\n", fd, n);
  if (kill_victim) victim.reset();
  co_return 0;
}

int main() {
  int a[2], b[2];
  if (::pipe(a) != 0) return 1;
  unique_fd ar{a[0]}, aw{a[1]};
  if (::pipe(b) != 0) return 1;
  unique_fd br{b[0]}, bw{b[1]};
  if (!set_nonblock(ar.get()) || !set_nonblock(br.get())) return 1;
  Loop loop;
  Task first = reader(loop, ar.get(), true);
  victim.emplace(reader(loop, br.get(), false));
  first.resume();
  victim->resume();
  if (::write(aw.get(), "x", 1) != 1 || ::write(bw.get(), "y", 1) != 1) return
   ↪  1;
  loop.run();
  std::printf("completions=%d\n", completions);
  return completions == 1 ? 0 : 1;
}
#endif
```

A GCC-sor a `lab-gcc`, a Clang-sor a `lab-clang` profilhoz tartozik:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -g -o echo_coro echo_coro.cpp
clang++ -std=c++20 -Wall -Wextra -Wpedantic -O2 -g -o echo_coro echo_coro.cpp
./echo_coro | cat          # normál pipe: a teljes üzenet, 0-s kód
./echo_coro > /dev/full    # írási hiba: diagnosztika, nem nulla kód
# tesztbuildek ASan/UBSan alatt:
```

```
for t in LOOP_CANCEL_TEST LOOP_STOP_TEST INJECT_TRANSIENT FD_LEAK_TEST; do
  g++ -std=c++20 -O1 -g -fsanitize=address,undefined -D$t echo_coro.cpp -o t_$t
   ↪  &&
    ./t_$t > /dev/null; echo "$t: $?"
done
```

Két szerződést érdemes kiemelni. Az első a **diszpécser pillanatképe**. A `run()` egy kötegben több kész eseményt is feldolgozhat. Egy korábbi elem `resume()`-ja közben megszüntethet egy olyan coroutine-t, amelynek a művelete szintén kész: a frame-mel együtt az awaitable és benne az `Op` is megszűnik. Ha a pillanatkép nyers `Op*` pointereket tárolna, a következő elem feldolgozása már felszabadított memóriát érne el. Ezért a pillanatkép az `Op` azonosítóját tárolja, és minden elem előtt a `lookup` ellenőrzi, hogy a művelet még regisztrálva van-e. A megszűnt awaitable destruktora `remove`-olja az `Op`-ot, így annak sem completionje, sem resume-ja nem fut.

A második az **eredménycsatorna**. Az `echo` különválasztja az EOF-ot (siker), az átmeneti hibát (`EINTR`, `EAGAIN`: újrapróbálás) és a végleges hibát (az `errno` értéke). A `Task` promise-a eltárolja a visszatérési értéket, illetve a coroutine-ból kiszökő kivételt `std::exception_ptr`-ként. A `main` ebből képez kilépési kódot és diagnosztikát.

Buildjegyzőkönyv (`lab-gcc` profil). A program mind az öt változatban figyelmeztetés nélkül fordult. Normál pipe-on a kimenet `hello from coroutine`, a kód 0; `/dev/full`-ra `echo: No space left on device`, a kód 1. A lemondási próba `reader fd=3 n=1, completions=1`, 0-s kód. A leállítási próba (`LOOP_STOP_TEST`) `pending=Operation canceled late=Operation canceled outstanding=0` kimenettel, 0 kóddal, 5 másodperces időkorláton belül futott: a függő olvasás `ECANCELED`-et kapott, a `run()` visszatért, és a `stop()` utáni regisztrációt az adapter azonnal elutasította. Az `INJECT_TRANSIENT` build az első két `read` és az első két `write` hívásra `EAGAIN`, majd `EINTR` hibát injektál; a kimenet ekkor is a teljes `hello from coroutine`, 0-s kóddal — a művelet nem veszett el, és nem lett hamis siker. Az `FD_LEAK_TEST` 101-szer futtatja az echót: az élő fd-k száma előtte és utána is 3 (a javítás előtti, `pipe` olvasóvégét nyitva hagyó változatnál 3-ról 103-ra nőtt). Az ASan/UBSan egyik próbában sem jelzett. A Clang-sort Clang 22-vel (ziglang) próbáltuk: figyelmeztetés nélkül fordult, 0-s kóddal futott.

A példában a `Task` `initial_suspend`-je `suspend_always`, ezért a `task.resume()` indítja el a coroutine-t. A `final_suspend` szintén felfüggeszt, így a `Task` destruktora tudja destroy-olni a frame-et. A `ReadAwaitable` és a `WriteAwaitable` temporális objektum a `co_await` kifejezésben jön létre, és a felfüggesztés alatt is él. Az `Op` az awaitable tagja, tehát a frame-ben van; a `Loop` csak pointert tárol. Ha a coroutine felfüggesztett állapotban megsemmisül, az awaitable destruktora `remove`-olja az `Op`-ot, így a `Loop` nem tart dangling pointert. A completion után az `Op` már nincs a `Loop` vektorában, ezért a destruktor `remove`-ja nem talál semmit.

A callbackes változat ugyanezt a folyamatot explicit állapotgéppel írja le. Az `Echo` struct tartalmazza a buffert, az offsetet és a fájlleírókat; az `on_read` és `on_write` függvények lépkednek. A coroutine ugyanezt a lineáris kódban tartja, az állapot a frame-ben van. A callbacknél a cancellation a struct élettartamához kötött, a coroutine-nál a frame és az awaitable élettartamához. A callbackes változatban a buffer lehet a struct tagja, a coroutine-ban a frame lokális változója; mindkettő addig él, amíg a művelet hivatkozik rá.

```
// Callbackes vázlat, nem teljes program.
struct Echo {
  Loop& loop;
  int in_fd;
  int out_fd;
  char buf[4096];
  std::size_t off = 0;
```

```
  ssize_t n = 0;

  void start_read();
  void on_read();
  void on_write();
};
```

A `start_read` regisztrál egy olvasási operationt, amelynek completionje `on_read`; az `on_read` elindítja a `start_write`-ot, a `on_write` pedig vagy befejez, vagy újabb olvasást indít. Az állapotátmeneteket a `n` és az `off` hordozza. A coroutine-ban ugyanez a `for` és `while` ciklus, az állapot a frame-ben. A coroutine kódja rövidebb és lineárisabb, de a frame allokálhat, és a debuggerben a frame-et kell érteni. A callbackes változat stacken is elférhet, de a hívási lánc és az állapot kézi követése nehezebb.

35.5.3 Shutdown, cancellation és diagnosztika

Az outstanding operation számlálója a `Loop::outstanding()`. A `stop()` a **kész eseménykötegen belül** is érvényes: a `poll` által már visszaadott, de még nem feldolgozott események teljesítése megszakad (`break`), és a következő iteráció `cancel_all()`-ja `ECANCELED`-del zárja le a függő műveleteket — egy már teljesített művelet eredménye megmarad, a stop után még nem teljesített műveleté lemondódik. A leállítási policy a példában **cancel**: a `stop()` a nyitott állapotból leálló állapotba lép; a `run()` ekkor nem pollol tovább, hanem a `cancel_all()`-lal minden függő műveletet `ECANCELED`-del zár le, majd — ha nincs több függő művelet — leállt állapotban visszatér. A leállás így véges, akkor is, ha egy olvasásra soha nem érkezne adat. A leálló vagy leállt `Loop` az `add`-ot elutasítja (`false`); az awaitable ilyenkor nem függeszt fel, hanem azonnal `ECANCELED`-del folytat. (Egy *drain* policy ehelyett megvárná a függő műveletek befejeződését, de akkor a véges leállást egy időkorlátnak kell biztosítania.) A `Task` destruktora csak akkor biztonságos, ha a `Loop` már nem hivatkozik az `Op`-ra. Ezt a laborban úgy ellenőrizhetjük, hogy a `run()` végén `ops_.empty()` igaz, és a `Task` csak ezután semmisül meg.

A timeout, a cancel és a completion kizáró szabálya a következő. Az operation pontosan egy terminális eseményt kaphat. A completion a `poll` által jelzett esemény után, a timeout a lejárat után, a cancel a `stop()` vagy egy külső kérés után következik be. A versenyt az eseményloop szálán, az `Op` eltávolításával kell eldönteni: amelyik ág előbb `remove`-ol, az nyer. A másik ág a `remove` után nem hivatkozhat az `Op`-ra, és nem hívhatja meg a completiont. Ezért a completion és a cancel is csak a `remove` után, a resume előtt futhat. Ha a timeout és a `poll` esemény ugyanabban az iterációban érkezik, a `remove` dönti el a sorrendet; a másik ág már nem találja az `Op`-ot.

A `cancel_all()` minden `Op`-ra beállítja a lemondás hibakódját, `remove`-ol, majd resume-ol. Az awaitable `await_resume()`-a ilyenkor `-1`-et ad `ECANCELED` `errno`-val. A coroutine ilyenkor hibát lát, és `co_return`-nal lezárhat. A timeout hasonló, de csak a lejárt operationre vonatkozik; a `poll` timeoutja önmagában nem elég, mert a lejáratot operation szinten kell nyilvántartani.

A debuggerben a coroutine frame a `Task::h` handle-ből érhető el. A `h.done()` a befejezést, a `h.address()` a frame címét mutatja. A `co_await` ponton a frame a stacken vagy a heapen van; a debugger a `promise` és a lokális változók alapján azonosítja. Az AddressSanitizer a frame-allokációt és a buffer túlcsordulását, a ThreadSanitizer a több szálú loopot ellenőrzi. A `ssize_t` → `size_t` konverzió nem UB: negatív értékre is definiált (modulo) eredményt ad, ezért az UBSan `-fsanitize=undefined` nem jelez rá. A védelem a kódban van: az `echo` a konverzió előtt kezeli a negatív eredményt, így a `static_cast<std::size_t>(n)` már csak pozitív értéket kap. Ha az implicit előjelváltó konverziókat külön is figyelni akarjuk, az fordítófüggő kiegészítés: a Clang `-fsanitize=implicit-integer-sign-change` kapcsolója (GCC-ben nincs megfelelője).

A laborban érdemes külön futtatni `-fsanitize=address,undefined` és `-fsanitize=thread` kapcsolókkal, dokumentált compiler- és library-verzió mellett.

Gyakori hibák: a buffer a `co_await` után megszűnik, de az adapter még használja; a `Loop` a resume után hivatkozik az `Op`-ra; a `poll` kötegének pillanatképe nyers `Op*`-t tárol, és egy korábbi resume által megszüntetett műveletet is feldolgoz; a completion kétszer hívódik, mert a timeout és a `poll` ág is lefut; a cancel után a coroutine még resume-ol; a `Task` destruktora felfüggesztett coroutine-t destroy-ol, miközben az `Op` a `Loop`-ban van. A javítás mindegyiknél ugyanaz a szerződés betartása: az `Op` az awaitable élettartamához kötött, a `Loop` csak pointert tárol, a terminális eseményt a `remove` dönti el, és a resume után az `Op` nem használható.

35.5.4 Feladatok

35.5.4.1 1. feladat: timeout az awaitable-ben

Egészítsd ki a `ReadAwaitable`-t timeouttal. A `Loop` kapjon egy `now()` függvényt és egy timeout-listát, amely a lejáratokat tartja nyilván. A timeout cancelálja az operationt, ha még nincs completion. Írd le, hogyan biztosítod, hogy a timeout és a completion közül csak az egyik fusson le.

35.5.4.2 2. feladat: egyedi lemondás

A példa `cancel_all()`-ja a leállításkor minden függő műveletet lemond. Valósíts meg egy `Loop::cancel(std::uint64_t id)` függvényt, amely *egyetlen* függő műveletet zár le `ECANCELED`-del, a többit érintetlenül hagyja. Ellenőrizd, hogy a lemondott coroutine `await_resume()`-ban kapja meg a hibát, hogy a `Task` destruktora nem hagy dangling `Op`-ot, és hogy a `run()` végén az outstanding számláló nulla.

35.5.4.3 3. feladat: callback és coroutine összehasonlítása

Írd meg ugyanazt az echo-folyamatot callbackes formában, majd hasonlítsd össze a coroutine verzióval. Készíts táblázatot a kódméretről, az állapot láthatóságáról, a cancellation szerződéséről, a hibakezelésről és az allokációról. Mérést csak dokumentált laborprofillal és legalább három ismétléssel végezz; a mérés nélküli állítást ne tekintsd bizonyítottnak.

35.5.5 Megoldások

35.5.5.1 1. feladat megoldása

Az `Op` megkapja a `deadline` és `has_deadline` mezőt; a `ReadAwaitable` használója a `co_await` előtt állítja be (`aw.op.deadline = now + 50ms; aw.op.has_deadline = true;`). A `Loop::run()` a `poll` időkorlátját a legközelebbi lejáratig csökkenti (legfeljebb 10 ms), a `poll` visszatérése után pedig **először** az `expire(now)` fut: a lejárt műveleteket az azonosítójuk alapján `cancel(id, ETIMEDOUT)`-tal zárja le (`remove`, hibakód, resume). Ezután jönnek a `poll` eseményei: ha az `Op` már nincs a vektorban, a `lookup(id)` nullát ad, így a completion nem fut le. A kizáró szabályt a `remove` és az `ops_` konzisztens használata biztosítja (az nyer, aki előbb `remove`-ol); a két ág ugyanazon a szálon fut, ezért nincs adatverseny. A fenti kód a fejezet példájába be van építve; teszt (`ReadAwaitable` 50 ms lejárattal, adat nélküli pipe): a coroutine `ETIMEDOUT`-ot kap, az `outstanding()` nulla (GCC 13.3, ASan+UBSan, lefuttatva). Ellenőrzési

feltétel: egy `Op`-ra sem fut le kétszer a completion, és a timeout után a coroutine `ETIMEDOUT`-ot kap.

35.5.5.2 2. feladat megoldása

A `Loop::cancel(id, error = ECANCELED)` a `lookup(id)`-vel megkeresi a műveletet; ha már nincs regisztrálva (közben befejeződött vagy megszűnt), `false`-t ad — ez a kizáró szabály: az nyer, aki előbb `remove`-ol. Ha megvan, `remove`, `cancel_error = error`, majd resume; az awaitable `await_resume()`-ja `-1`-et ad, az `errno` `ECANCELED`, a coroutine `co_return`-nel lezár. A `cancel_all()` ugyanezt teszi az összes műveletre. Lefuttatott teszt (két függő olvasás): az első `cancel(1)` igaz, a második `cancel(1)` hamis, az első coroutine `ECANCELED`-et kap, a második függő marad (`outstanding() == 1`), majd `stop()` + `run()` után az is `ECANCELED`-del zár és az `outstanding()` nulla. Ellenőrzési feltétel: a lemondott művelet completionje nem fut le, a többi művelet eredménye változatlan, a `Task` destruktora nem hagy dangling `Op`-ot.

35.5.5.3 3. feladat megoldása

A callbackes változatban az `Echo` struct tartalmazza a buffert és az állapotot; a `start_read` és `start_write` függvények regisztrálják az operationöket, a completionök pedig a struct metódusai. A coroutine változatban ugyanez a folyamat a `for` és `while` ciklusban van, az állapot a frame-ben. A táblázatban a callbackes változat kisebb frame-et használhat, de az állapotátmeneteket kézzel kell követni; a coroutine kódja lineárisabb, de a frame allokálhat. A cancellation a callbacknél a struct élettartamához, a coroutine-nál a frame és az awaitable élettartamához kötött. A mérésnél rögzíteni kell a compiler, a library, a target és az optimalizálási szint pontos verzióját; a mérés nélküli összehasonlítás csak szerződés szintű.

35.5.6 Összefoglaló

A coroutine csak felfüggesztés; az aszinkron I/O adapter adja a completiont, a schedulert és a cancellationt. A laborban a `poll` eseményei és a coroutine frame találkoznak. A szerződés négy pontja: a buffer tulajdonosa a hívó, az awaitable élettartama fedi a felfüggesztést, a terminális esemény pontosan egyszer fut le, és a shutdown alatt az outstanding számláló nullázódik. A debugger a frame-et, a sanitizer a buffer és az `Op` élettartamát ellenőrzi. A callback, az állapotgép és a coroutine ugyanazt a folyamatot írja le, de eltérő élettartam- és költségszerződéssel.

36 C++ ABI, C interfész és FFI

36.1 Névtorzítás, layout és C++ runtime ABI

Egy natív program linkelési felülete nem ismeri a C++ nyelvet: a szerelő (assembler) és a linker szintjén csak szimbólumok, szakaszok és relokációk léteznek. Amikor két, különböző fordítóval vagy különböző könyvtárverzióval épített objektum találkozik, a találkozás helyét az ABI (Application Binary Interface) szabályai rögzítik. A C nyelvben ezt viszonylag kevés dolog terheli: a hívási konvenció, a struct-layout és a szimbólumnév. A C++ ezt a szerződést kibővíti — névtorzítással, osztályelrendezéssel, virtuális dispatch-csel, RTTI-vel és kivételkezeléssel —, de a bővítés nem a nyelvi szabvány része, hanem egy konkrét ABI implementációé. Ezért ebben az alfejezetben minden állítást konkrét toolchainprofilhoz kötünk: Linux x86-64 System V, GCC 13 vagy Clang 17, `-std=c++20`, Itanium C++ ABI. Ahol a részlet implementációfüggő vagy verziófüggő, jelöljük.

A korábbi részekből építkezünk: a többszörös öröklés és a virtuális bázis eseteit a Többszörös öröklés és virtuális bázis (24.5) tárgyalta, az ODR és az `inline` több fordítási egységen átnyúló hatását az ODR, inline és több fordítási egység (33.3), a hívási konvenciót és a stack frame szerkezetét pedig a Hívási konvenció, stack frame és visszatérés (14.2). Ezek ismeretében most a C++ ABI három pillérét — névtorzítás, layout, runtime-támogatás — vesszük sorra, és megmutatjuk, mit lehet ezekből a linkelési felületen megfigyelni.

36.1.1 Névtorzítás: nyelvi név és szimbólumnév

A **névtorzítás** (name mangling) az a fordítási lépés, amely egy C++ entitás — függvény, változó, típus, sablon-instantiation — deklarációját egyedi, karakterlánc-alapú szimbólumnévvé kódolja. A C++ nyelv a túlterhelést, a névtereket, a tagfüggvényeket és a sablonokat egyetlen névtérben kell hogy kifejezze a linker számára; a linker viszont csak `char*`-összehasonlítást ismer. A torzítás tehát kényszerű: a nyelvi információk egy részét (paramétertípusok, hozzáférés, osztálykörnyezet) a szimbólumnév hordozza.

Az Itanium C++ ABI mangling-sémája elterjedt Linuxon, macOS-en (Clang), ARM-en és sok más platformon. Fontos, hogy az „Itanium” itt **történelmi elnevezés**, nem az Intel Itanium ISA-ra utal: a séma egy The SCO Group és mások által kezdeményezett, ISA-független ABI-specifikáció neve, amelyet x86-64-en, AArch64-en, RISC-V-n egyaránt használnak. A Microsoft Visual C++ más sémát használ, ezért MSVC és GCC objektumai nem linkelhetők össze C++-szinten.

Az alapséma: `_Z`, majd a név hossza és a név, majd a paramétertípusok kódolása. Néhány példa, amelyeket `c++filt`-tel vissza lehet fejteni:

Forrás	Torzított szimbólum
`int foo(int)`	`_Z3fooi`
`double foo(double)`	`_Z3food`
`void ns::foo()`	`_ZN2ns3fooEv`
`void Foo::bar() const`	`_ZNK3Foo3barEv`
`Foo::Foo()` (teljes objektum)	`_ZN3FooC1Ev`
`Foo::~Foo()` (teljes objektum)	`_ZN3FooD1Ev`

Forrás	Torzított szimbólum
`Foo` típus vtable-je	`_ZTV3Foo`
`Foo` típus RTTI-ja	`_ZTI3Foo`
`Foo` típus neve	`_ZTS3Foo`

A `K` a const tagfüggvényt jelöli, a `C1/C2` a konstruktort (teljes vs. bázis-változat), a `D0/D1/D2` a destruktort (deleting, complete, base). A `V`, `I`, `S` prefixek rendre vtable, typeinfo, typeinfo-name string. A `c++filt` visszafejti ezeket, a `nm -C` a binárison belüli szimbólumokat már demangle-özve listázza.

Fontos hangsúlyozni: a mangling **nem** része az ISO C++ szabványnak. Két standard-konform fordító ugyanarra a függvényre más szimbólumot generálhat, és ez önmagában megakadályozza az összelinkelést. Az `inline` és sablon-instantiation esetén az ODR még szigorúbb: ha két TU-ban ugyanarra a szimbólumra eltérő definíció kerül, a viselkedés nem definiált, és a linker gyakran észrevétlenül választ egy példányt.

36.1.2 Osztályelrendezés, vptr, vtable és RTTI

A **nem polimorf** osztály elrendezése nagyjából követi a C struct szabályait: a nem statikus adattagok deklarációs sorrendben, természetes igazítással, a köztes réseket kitöltve (padding) helyezkednek el. Az `sizeof`, `offsetof` és az igazítás ebben az esetben a szokásos C-beli számítással követhető, feltéve, hogy ismerjük a cél-ABI-t.

A **polimorf** osztály (amelynek van legalább egy virtuális függvénye vagy virtuális bázisa) elrendezése gyökeresen más: az Itanium ABI egy rejtett `vptr`-t illeszt be, amely a **virtuális táblára** (vtable) mutat. Ez az ABI adta reprezentáció — a nyelv csak annyit mond, hogy a virtuális hívás a dinamikus típus függvényét hívja, arról nem rendelkezik, hogy ez vtable-lekérdezéssel történik. A vtable tartalma az Itanium ABI szerint:

Index	Tartalom
−2	offset-to-top (a legfelső objektumhoz mért eltolás)
−1	`_ZTI<Osztály>` pointer, az RTTI leírás
0…n	virtuális függvények címei deklarációs sorrendben, öröklési szabályok szerint

A `vptr` a 0. indexre mutat; a negatív indexek elérését a fordulónak kell biztosítania (a `dynamic_cast` és `typeid` erre támaszkodik). A `vtable` nem garantáltan olvasható adat a programból — a hozzáférés az ABI belső szerződése.

Az **RTTI** (Run-Time Type Information) ezen a felületen egy `std::type_info`-szerű objektum, amelynek szimbólumneve `_ZTI<Osztály>`, és amely egy `_ZTS<Osztály>` névkarakterláncra mutat. A `typeid` és `dynamic_cast` ezt használja. Két dinamikusan linkelt modul között a típusazonosság-ellenőrzés attól függ, hogy a `typeinfo` pointereket egyesíti-e a dynamic linker; ezért DSO-határon a `typeid`-alapú azonosítás törékeny lehet, és verziófüggő. Ezt a részletet a gyakorlatban ne tekintsük stabil szerződésnek.

36.1.3 Thunkok, pointer-igazítás és tag-függvény pointer

Többszörös öröklésnél a bázis-részobjektum nem feltétlenül a legális `this` pointerrel esik egybe. Tekintsük:

```
struct BaseA { virtual ~BaseA() = default; virtual void render() const; };
struct BaseB { virtual ~BaseB() = default; virtual void render() const; };
struct Widget : BaseA, BaseB {
    void render() const override;
};
```

A `Widget`-en belül a `BaseA` részobjektum a 0. offseten, a `BaseB` részobjektum később helyezkedik el. Amikor egy `BaseB*`-en keresztül hívjuk a `render`-t, a hívott függvénynek a `Widget`-re mutató pointerre van szüksége, nem a `BaseB` részobjektumra. Ezt egy **thunk** oldja meg: egy rövid, fordító által generált stub, amely a `this` pointert a megfelelő eltolással korrigálja, majd a valódi implementációra ugrik. Az Itanium ABI **non-virtual thunk**-nak nevezi a nem virtuális bázisra vonatkozó korrekciót, és **virtual thunk**-nak a virtuális bázisért végzett korrekciót.

A thunk nem jelenik meg a forrásban, de a szimbólumtáblában igen: `nm` csillagozott bejegyzésként vagy `non-virtual thunk to Widget::render()` alakban mutatja. A `BaseB` részobjektum saját vtable-je a thunkra mutat, nem közvetlenül a `Widget::render`-re.

A **tag-függvény pointer** (`int (Widget::*)()`) reprezentációja szintén ABI-kérdés. Az Itanium ABI-ban egy nem virtuális tag-függvény pointer egy `{ptr, adj}` pár, ahol `adj` az esetleges `this`-korrekció; virtuális tagfüggvénynél a `ptr` egy vtable-indexre mutató segédfüggvény (az úgynevezett „virtual member function pointer”), és az `adj` az öröklési eltolás. Ennek pontos mérete és szemantikája implementációfüggő, és nem hordozható két különböző ABI között — ezért shared library határon tag-függvény pointer átadása nem javasolt.

36.1.4 Kivételkezelés mint ABI-szolgáltatás

A `throw` és a stack unwinding (veremtakarítás) együttműködést igényel a fordító, a runtime és a dinamikus linker között. A `throw` egy runtime-függvényt hív (`__cxa_throw` az Itanium ABI-ban), amely elindítja a kétszakaszos unwindingot. A fordító minden olyan függvényhez, amely kivételt dobhat vagy kezel, két extra szakaszt generál:

- `.eh_frame` (és `.eh_frame_hdr`) — a CFI (Call Frame Information), amely leírja, hogy az unwinder hogyan állítsa vissza a regisztereket lépésenként.
- `.gcc_except_table` — az LSDA (Language-Specific Data Area), amely megmondja, hogy az adott programszámlálás-tartományban melyik `catch` blokk illetékes, és milyen típushoz.

A tényleges unwindingot a `libgcc_s` (vagy `libunwind`) biztosítja: `_Unwind_RaiseException`, `_Unwind_Resume`, `_Unwind_DeleteException`. A personality routine — GCC-nél `__gxx_personality_v0` — dönti el lépésenként, hogy az adott frame hogyan viselkedjen. Ezek a szimbólumok a `libstdc++` és `libgcc_s` közös felületén élnek.

Következmény a gyakorlatra: egy `extern "C"` felületen átjutó kivétel nem definiált viselkedés, mert a C-fordító által generált frame-ekhez nincs LSDA, így az unwinder nem tudja őket helyesen átlépni. (A gyakorlatban egyes implementációk „átengedik”, de garantált viselkedés nincs.) Ezért az FFI-határon a kivételt a C++ oldalon el kell kapni, és hibakód, `std::error_code` vagy `expected` formájában átadni. Ezt a mintát részletesen a Stabil shared library határ és verziózás (36.3) tárgyalja.

Kiegészítő runtime-szimbólumok, amelyeket érdemes ismerni: `__cxa_atexit` (globális destruktor regisztrálása), `__cxa_guard_acquire`/`__cxa_guard_release` (függvény-lokális statikus inicializáció, C++11 óta szálbiztos), `__cxa_pure_virtual` (tiszta virtuális hívás esetén hívott függvény), `__cxa_allocate_exception`/`__cxa_free_exception`. Ha egy binárison `nm -D | grep __cxa` futtatunk, ezek jól látszanak — de a konkrét készlet a fordító és a standard library verziójától függ.

36.1.5 Standard layout és triviálisság: mit garantál és mit nem

A `std::is_standard_layout<T>` és `std::is_trivial<T>` nyelvi tulajdonságok. A **standard layout** azt jelenti, hogy az osztály megfelel a C-kompatibilis elrendezés bizonyos nyelvi kritériumainak (pl. legfeljebb egy nem statikus adattaggal rendelkező bázis, minden nem statikus tag azonos hozzáférés-szinttel, nincs virtuális tag). A **triviálisság** azt jelenti, hogy a default constructor, a másoló/mozgató konstruktor, a másoló/mozgató értékadás és a destruktor mind „triviális” (a fordító által generált, szemantikailag üres). Egy triviális, standard layout típus nagyon közel áll egy C structhoz, és gyakran ugyanazzal a calling convention-nel adható át.

Azonban **ezek a tulajdonságok önmagukban nem jelentenek ABI-stabilitást**. Három okból:

1. A **padding és alignment** a cél-ABI-tól függ. Két azonos mezőkkel rendelkező standard layout struct két architektúrán eltérő `sizeof`-szal rendelkezhet.
2. A **calling convention** struct visszaadásnál vagy paraméter-átadásnál külön szabályokat használhat (System V: kis struct register-ben, nagy memóriában). Ezek a szabályok nem a nyelvi típusból, hanem az ABI-ból következnek.
3. Az **empty base optimization (EBO)** és a `[[no_unique_address]]` olyan elrendezéseket eredményez, amelyek nem esnek egybe a naiv számítással, és nem minden fordító alkalmazza ugyanúgy.

Gyakorlati következmény: shared library határon akkor is érdemes kerülni a C++ típusok közvetlen átadását, ha `is_trivial` és `is_standard_layout` igaz. A stabil minta az **opaque handle** — átlátszatlan mutató vagy egész azonosító, amelyet a könyvtár belső típusra kovácsol —, amit a Stabil shared library határ és verziózás (36.3) részletez. Egy tanulságos eset a libstdc++ `_GLIBCXX_USE_CXX11_ABI` váltása (GCC 5): két különböző értékkel épített modul ugyanarra a szimbólumra más `std::string` layout-ot használ, ezért a keveredés linkelési hibát vagy csendes memóriahibát okoz. Ez nem szabványkérdés, hanem egy konkrét standard library ABI-döntése.

36.1.6 Kidolgozott példa: ugyanaz a program két nézetben

Fordítsuk le a fenti `Widget`-példát egy önálló fájlba, és figyeljük meg a szimbólumokat.

```
// shapes.cpp - GCC 13 / Clang 17, Linux x86-64, Itanium ABI
#include <cstdio>

struct BaseA {
    virtual ~BaseA() = default;
    virtual void render() const { std::puts("A::render"); }
    int ax = 1;
};
struct BaseB {
    virtual ~BaseB() = default;
    virtual void render() const { std::puts("B::render"); }
```

```
    int bx = 2;
};
struct Widget : BaseA, BaseB {
    ~Widget() override = default;
    void render() const override { std::puts("Widget::render"); }
    int w = 3;
};

int main() {
    Widget w;
    BaseB* b = &w;
    b->render();
    std::printf("sizeof(Widget)=%zu\n", sizeof(Widget));
    std::printf("offsets: A=%td B=%td\n",
        reinterpret_cast<const char*>(static_cast<BaseA*>(&w)) -
         ↪  reinterpret_cast<const char*>(&w),
        reinterpret_cast<const char*>(b) - reinterpret_cast<const char*>(&w));
}
```

Fordítás és megfigyelés:

```
g++ -std=c++20 -O0 -g -o shapes shapes.cpp
nm shapes | grep -E 'ZTV|ZTI|ZTS|render' | c++filt   # demangle-özött lista
readelf -Ws shapes | grep -E '_ZTV|_ZTI'              # nyers szimbólumok
```

A megfigyelhető jelenségek (konkrét értékek a fenti profilon, más cél-ABI-n eltérhetnek):

- `_ZTV6Widget` három bejegyzést tartalmaz: `offset-to-top`, `typeinfo`, majd a `BaseB`-részobjektumban a `render`-re mutató thunk.
- `_ZTI6Widget` az RTTI-leíró, `_ZTS6Widget` a típusnév.
- `nm -C` listájában megjelenik a `non-virtual thunk to Widget::render() const` bejegyzés — ez a `this`-korrekciót végző stub, amely a `BaseB` vtable-jében szerepel.
- A `sizeof(Widget)` és az offsetek a fenti elrendezés szerintiek, de ezeket a konkrét cél-ABI és a `-O0` mellett kell értelmezni; `-O2` nem változtatja meg a layoutot, de befolyásolhatja a thunk jelenlétét vagy a hívások inline-olását.

Ez a kísérlet jól mutatja, hogy a nyelvi objektum és az ABI-reprezentáció két külön szint: a forrásban egy `render` függvény van, a binárison belül viszont két-három szimbólum (valódi, thunk, vtable-bejegyzés) írja le ugyanazt a viselkedést.

36.1.7 Tipikus hibák

- **A mangling-séma nyelvi tulajdonságnak tekintése.** Két eltérő ABI-jű fordító által generált bináris összeillesztése nem „C++ kompatibilitási” kérdés, hanem egyszerűen nem fog linkelni.
- **Kivétel átengedése `extern "C"` határon.** Még ha egy adott napon működik is, nincs szerződés mögötte, és a viselkedés a következő libgcc-verzióval megváltozhat.
- **`typeid`-re épülő DSO-azonosítás.** A típusegyesítés a dynamic linker konfigurációjától függ; `-fvisibility=hidden` és explicit `__attribute__((visibility("default")))` nélkül a határ nem kontrollált.
- **Standard layout `struct` közvetlen átadása két fordítási egység között.** Ez rendszerint működik, de nem garantált, ha a két TU eltérő `-fpack-struct`, `-mabi=`, `-D_GLIBCXX_USE_CXX11_ABI=` vagy cél-ABI-t használ.

- **Tag-függvény pointer átadása FFI-határon.** A reprezentáció implementációfüggő; használjunk `extern "C"` callback-et kísérő `void*` context pointerrel.

36.1.8 Feladatok

1. (magyarázó) Adott a `_ZNK4Data5mergeERKS_` szimbólum. Fejtse vissza a mangling sémát kézzel, és nevezze meg, milyen szintaktikai elemeknek felelnek meg az egyes karakterek.

2. (kódolási) Fordítson le egy három bázisú öröklést (`class D : public A, public B, public C`) használó programot a fenti profillal, és `nm -C` kimenetéből azonosítsa, mely `render`-szerű hívásoknál jelenik meg thunk. Módosítsa úgy a bázisokat, hogy az egyik virtuális bázis legyen (`virtual public A`), és hasonlítsa össze a thunkok számát és típusát.

3. (diagnosztikai) Adott egy bináris, amely `undefined reference to __cxa_throw` hibát ad egy tiszta C program linkelésekor. Mi a hiba oka, és hogyan orvosolható a build (a `-fno-exceptions` és a `-static` kapcsolókhoz kapcsolódó viselkedést is érintve)?

36.1.9 Megoldások

1. A `_ZNK4Data5mergeERKS_` bontása: `_Z` a mangling prefix; `N` „nested name” (a név névtérben/osztályban van); `K` const tagfüggvény; `4Data` a `Data` osztály neve (hossz + karakterek); `5merge` a tagfüggvény neve; `E` a nested name lezárása; `R` referencia (`T&`); `K` a hivatkozott típus const; `S_` a korábbi helyettesítő (`S_` az első helyettesítő elemre utal, itt magára a `Data`-ra). A függvény tehát: `void vagy egyéb visszatérési típusú Data::merge(const Data&) const`. Fontos, hogy a visszatérési típus nincs a manglingben (kivéve sablon-függvényeket és operátorokat), mert a túlterhelést csak a paraméterek és a tagfüggvény-jelleg különbözteti meg.

2. A megoldás kulcsa, hogy a `D` első bázis-részobjektuma 0. offseten van, a második és harmadik bázisé nem. Minden olyan virtuális függvény, amelyet a `B` vagy `C` részobjektumon keresztül hívunk, thunkot generál (legalább `non-virtual thunk to D::render`). Ha az `A`-t virtuális bázissá tesszük, a thunkok **virtual thunk**-ká alakulnak, mert a `this`-korrekció nem egy statikus offset, hanem a virtuális bázis vptr-jéből nyert adat alapján történik. Ennek megfelelően a vtable szerkezete bonyolultabb (VTT — virtual table table), és a thunkok száma nő. Ellenőrzési feltétel: az `nm -C` kimenetében a `thunk` szót tartalmazó bejegyzések kiírandók, és a két változat között a módosítás kimutatható különbséget ad. (Konkrét darabszám erősen függ a fordító verziójától és a `-O` szinttől.)

3. A `__cxa_throw` (és társai, `__gxx_personality_v0`) a `libstdc++`-ban él. Ha C programot `-static`-kal vagy `-fno-exceptions`-szel linkelünk, de a program — például egy headerben lévő `inline` C++ kód vagy egy `#include <stdexcept>` — mégis hivatkozik ezekre, a linker nem találja a szimbólumot. Javítás: (a) a C++ kódot izoláljuk, és `extern "C"` wrapperben tartsuk, vagy (b) kapcsoljuk be a `libstdc++`-t (`g++`-vel linkeljük a vegyes programot), vagy (c) `-fno-exceptions` esetén biztosítsuk, hogy semmilyen C++ komponens ne dobjon kivételt, és a standard library fejléceit se hívjuk olyan útvonalon, amely throw-t igényel. `-static` esetén a `--gc-sections` segíthet, de a `libgcc_eh.a`-t és `libstdc++.a`-t is be kell vonni. Ellenőrzési feltétel: `nm -D a.out | grep __cxa_` a linkelés után vagy üres, vagy a bináris statikusan tartalmazza a szükséges szimbólumokat; `ldd a.out`-ból kiderül, hogy melyik runtime-ra támaszkodik.

36.1.10 Összefoglaló

A C++ ABI — konkrét profilon: Itanium C++ ABI Linux x86-64-en, GCC 13/Clang 17, `-std=c++20` — három rétegben terjeszti ki a C-szerződést. A **névtorzítás** a nyelvi entitást egyedi szimbólumnévvé kódolja; a séma nem szabvány, hanem ABI-döntés, és a `c++filt`/`nm -C` segítségével olvasható. A **layout** a nem polimorf osztályoknál közel C-struct, a polimorfaknál viszont rejtett `vptr`-t és vtable-t vezet be, benne `offset-to-top`, RTTI-pointer és függvénypointerek; a többszörös (és virtuális) öröklés thunkokat generál a `this`-korrekcióhoz. A **runtime-támogatás** (`.eh_frame`, `.gcc_except_table`, `__cxa_*`, `_Unwind_*`) biztosítja a kivételkezelést és a statikus inicializációt — ezek nem átjárhatók szabadon `extern "C"` határon. Végül a `is_standard_layout`/`is_trivial` nyelvi tulajdonságok hasznos közelítések, de nem ABI-stabilitási garanciák; stabil shared library határhoz opaque handle és C-kompatibilis interfész kell, amit a következő alfejezetek (extern C és két nyelvű header (36.2), Stabil shared library határ és verziózás (36.3), Python és más nyelvek FFI-kapcsolata (36.4)) részleteznek. A laborban a `nm`, `readelf` és `c++filt` együttes használata adja a megfigyelés eszközét — a konkrét kimenet azonban mindig a rögzített toolchainprofilhoz kötendő.

36.2 extern C és két nyelvű header

A C++-ban írt könyvtár egyik legértékesebb tulajdonsága lehet, hogy C fogyasztók számára is használható marad. Ez a DevOps-gyakorlatban különösen fontos: a C ABI a legstabilabb közös nevező a nyelvek, a shared library határok és a különböző build-rendszerek között. A C++ oldalon megmaradhat a típusos, RAII-alapú implementáció, miközben a publikált felület szándékosan szűk, C-kompatibilis és hosszú távon verziózható marad. Ez az alfejezet a kétnyelvű header és az `extern "C"` határ szerződését tárgyalja; a névtorzítás és a C++ runtime ABI általános kérdéseit Névtorzítás, layout és C++ runtime ABI (36.1), a stabil shared library határt és verziózást Stabil shared library határ és verziózás (36.3), a Python FFI-kapcsolatot Python és más nyelvek FFI-kapcsolata (36.4), a laborellenőrzést pedig ABI- és FFI-labor, kompatibilitási teszt (36.5) tárgyalja.

36.2.1 A nyelvi linkage és a kétnyelvű header szerződése

A C++ standardban a függvények és külső nevek *language linkage*-gel rendelkeznek. Az `extern "C"` nyelvi kapcsolás azt írja elő, hogy a deklarált függvény a C nyelvi kapcsolás szabályai szerint legyen elérhető. A gyakorlatban ez legtöbbször a C++ névtorzítás (name mangling) kikapcsolását és a platform C hívási konvenciójának használatát jelenti. Fontos, hogy az `extern "C"` nem változtatja meg a függvény szignatúráját: a paraméterek és a visszatérési típus ugyanaz marad, csak a külső név és a hívási szerződés lesz C-kompatibilis. Az osztályok, sablonok, overloadok és referenciák nem válnak automatikusan C-kompatibilissé attól, hogy egy `extern "C"` blokk közelében vannak.

A kétnyelvű header két őrt használ. A header guard (`#ifndef PARSER_H`) a többszörös include ellen véd, a `#ifdef __cplusplus` pedig azt biztosítja, hogy az `extern "C"` blokk csak C++ fordításban jelenjen meg. A C fordító nem definiálja a `__cplusplus` makrót, a C++ fordító viszont igen; C++20-ban az értéke jellemzően `202002L`. A helyes minta:

```
#ifndef PARSER_H
#define PARSER_H

#include <stddef.h>
```

```
#ifdef __cplusplus
extern "C" {
#endif

/* C-kompatibilis deklarációk */

#ifdef __cplusplus
}
#endif

#endif
```

A headerben csak olyan deklarációk szerepelhetnek, amelyeket a C fordító is elfogad: `typedef`, `struct` tag, `enum`, függvénydeklaráció, `const char *`, `size_t`, `void`. Kerülni kell a `bool`-t (C17-ben csak `<stdbool.h>`-val, C23-ban kulcsszó), a `nullptr`-t, az `auto`-t, a referenciákat, a default argumenteket és a `noexcept`-ot; utóbbit makróval lehet C++ módban `noexcept`-ká tenni. A C nem tud C++ osztályt, sablont vagy overloadot fogyasztani, mert a C szintaxis nem ismeri ezeket, a C++ ABI pedig ezekhez nem ad C-kompatibilis nevet. Ezért a közös felület csak függvényekből, opaque pointerekből, C-kompatibilis structokból és enumokból állhat.

Az opaque handle lényege, hogy a C oldal csak egy előre deklarált struct pointerét látja, a méretet és a layoutot nem. A `typedef struct parser_ctx parser_ctx;` deklaráció C és C++ módban is működik; a struct definíciója a C++ implementációban marad. A C oldal így nem tud `sizeof(parser_ctx)`-t venni, nem tud stacken példányt létrehozni, és nem tud a belső tagokhoz hozzáférni. Az életciklust explicit `create`/`destroy` függvények kezelik; ez az Opaque handle és stabil C könyvtári interfész (10.4) mintájára épül.

36.2.2 A C parserkönyvtár headerje

A példa egy egyszerű parser könyvtár. A header a teljes nyilvános szerződés; a C++ implementáció részletei elrejthetők. A `PARSER_API` makró a dinamikus exportálhatóságot jelöli, a `PARSER_NOEXCEPT` pedig C++ módban `noexcept`-ot ad. A `PARSER_BUILD_SHARED` makrót a könyvtár fordításakor definiáljuk; Windowson ez dönti el, hogy `dllexport` vagy `dllimport` legyen.

```
#ifndef PARSER_H
#define PARSER_H

#include <stddef.h>

#if defined(_WIN32)
#  if defined(PARSER_BUILD_SHARED)
#    define PARSER_API __declspec(dllexport)
#  else
#    define PARSER_API __declspec(dllimport)
#  endif
#elif defined(__GNUC__) || defined(__clang__)
#  define PARSER_API __attribute__((visibility("default")))
#else
#  define PARSER_API
#endif
```

```
#ifdef __cplusplus
#  define PARSER_NOEXCEPT noexcept
extern "C" {
#else
#  define PARSER_NOEXCEPT
#endif

typedef struct parser_ctx parser_ctx;

typedef enum parser_status {
    PARSER_OK = 0,
    PARSER_EINVAL = 1,
    PARSER_ENOMEM = 2,
    PARSER_ELIMIT = 3,
    PARSER_ESYNTAX = 4
} parser_status;

PARSER_API parser_ctx *parser_create(void) PARSER_NOEXCEPT;
PARSER_API void parser_destroy(parser_ctx *ctx) PARSER_NOEXCEPT;
PARSER_API parser_status parser_feed(parser_ctx *ctx,
                                     const char *data,
                                     size_t len) PARSER_NOEXCEPT;
PARSER_API parser_status parser_finish(parser_ctx *ctx) PARSER_NOEXCEPT;
PARSER_API const char *parser_version(void) PARSER_NOEXCEPT;

#ifdef __cplusplus
}
#endif

#endif
```

A `PARSER_API` és a `PARSER_NOEXCEPT` makrók a header guardon belül vannak, így egyszer definiálódnak. A `PARSER_NOEXCEPT` a C oldalon üres, a C++ oldalon `noexcept`; a C++ hívó így tudja, hogy a függvény nem dob. A `parser_version` statikus élettartamú stringre mutat, ezért a C hívó nem szabadítja fel.

A visibility és a linkage két külön dolog. Az `extern "C"` a név és a hívási konvenció szintjén teremt C-kompatibilitást; a visibility azt szabályozza, hogy a szimbólum bekerül-e a dinamikus szimbólumtáblába. Ha `-fvisibility=hidden` mellett fordítunk, minden szimbólum rejtett lesz, kivéve a `visibility("default")`-tal jelölteket. Az `extern "C"` önmagában nem tesz egy szimbólumot exportálttá. A `static` függvény belső linkage-ű, és nem exportált; ezért a könyvtár belső segédfüggvényeit `static`-nak vagy névtelen névtérbe zártnak kell tenni.

36.2.3 A C++ implementáció és a határ szerződései

A C++ implementációban a `parser_ctx` teljes definíciója szerepel, a C oldal számára láthatatlanul. A `std::string` és a `bool` csak a C++ oldalon jelenik meg. A `parser_create` `new`-val allokál, a `parser_destroy` `delete`-tel szabadít; a C hívó soha nem hívhat `free`-t a pointerre. Ez az allocator pairing alapja: a memóriát az a modul szabadítja fel, amelyik allokálta, ugyanazzal az allocatorral. Windows DLL-határon ez különösen kritikus, mert a DLL és az EXE eltérő CRT-heapet használhat.

```cpp
#include "parser.h"

#include <cstddef>
#include <new>
#include <stdexcept>
#include <string>

struct parser_ctx {
    std::string input;
    bool finished = false;
};

extern "C" PARSER_API parser_ctx *parser_create(void) PARSER_NOEXCEPT {
    try {
        return new parser_ctx();
    } catch (...) {
        return nullptr;
    }
}

extern "C" PARSER_API void parser_destroy(parser_ctx *ctx) PARSER_NOEXCEPT {
    delete ctx;
}

extern "C" PARSER_API parser_status
parser_feed(parser_ctx *ctx, const char *data, size_t len) PARSER_NOEXCEPT {
    if (ctx == nullptr) {
        return PARSER_EINVAL;
    }
    if (data == nullptr) {
        return len == 0 ? PARSER_OK : PARSER_EINVAL;
    }
    try {
        ctx->input.append(data, len);
        return PARSER_OK;
    } catch (const std::bad_alloc &) {
        return PARSER_ENOMEM;
    } catch (const std::length_error &) {
        return PARSER_ELIMIT;
    } catch (...) {
        return PARSER_ESYNTAX;
    }
}

extern "C" PARSER_API parser_status
parser_finish(parser_ctx *ctx) PARSER_NOEXCEPT {
    if (ctx == nullptr) {
        return PARSER_EINVAL;
    }
    try {
        ctx->finished = true;
        return PARSER_OK;
```

```
    } catch (...) {
        return PARSER_ESYNTAX;
    }
}

extern "C" PARSER_API const char *parser_version(void) PARSER_NOEXCEPT {
    return "1.0";
}
```

A `try/catch` minden `extern "C"` függvényben kötelező elem, ha a C hívó felé nem akarunk C++ kivételt engedni. A C hívó nem tud `try/catch`-et használni; a C ABI-n nincs kivételkezelési protokoll. A `catch (...)` a legvégső háló; a konkrétabb catch-ek hibakódot adnak. A `noexcept` ezt a szándékot erősíti: ha mégis kijutna egy kivétel, a C++ runtime `std::terminate`-et hív. A `parser_destroy` destruktora nem dobhat; a `std::string` destruktora ezt biztosítja. A noexcept határok és hibapolicy (25.3) rész a `noexcept` és a hibapolicy általános kérdéseit tárgyalja; itt a C határra alkalmazzuk.

A `parser_feed` a bemenetet másolja, nem tárolja a C hívó pointerét. Így a C hívó a `data` puffert a hívás után felszabadíthatja. A `parser_finish` a példában nem ad vissza belső pointert; a 2. feladatban majd igen, és ott a pointer élettartamát a szerződésben rögzíteni kell. A `parser_ctx` élettartama alatt a belső buffer érvényes, de a következő módosító hívás invalidálhatja. A C hívó nem szabadíthatja fel a belső buffert.

A C fogyasztó a headert C módban includes; a `NULL` és a `strlen` a C könyvtárból jön. A C++ fogyasztó ugyanazt a headert használja; a `extern "C"` miatt a hívás C linkage-ű. A C++ fogyasztó a `parser_ctx`-et nem tudja `delete`-elni, csak a `parser_destroy`-t hívhatja.

```
#include "parser.h"
#include <stdio.h>
#include <string.h>

int main(void) {
    parser_ctx *ctx = parser_create();
    if (ctx == NULL) {
        fputs("parser_create failed\n", stderr);
        return 1;
    }

    const char *chunk = "int x = 42;";
    parser_status st = parser_feed(ctx, chunk, strlen(chunk));
    if (st != PARSER_OK) {
        fprintf(stderr, "feed failed: %d\n", (int)st);
        parser_destroy(ctx);
        return 2;
    }

    st = parser_finish(ctx);
    parser_destroy(ctx);
    return st == PARSER_OK ? 0 : 3;
}
```

```
#include "parser.h"
#include <cstring>
#include <iostream>
```

```
int main() {
    parser_ctx *ctx = parser_create();
    if (!ctx) {
        return 1;
    }
    const char *chunk = "int x = 42;";
    parser_status st = parser_feed(ctx, chunk, std::strlen(chunk));
    parser_destroy(ctx);
    if (st != PARSER_OK) {
        std::cerr << "feed failed: " << static_cast<int>(st) << '\n';
        return 2;
    }
    return 0;
}
```

36.2.4 Build és ellenőrzés

A könyvtárat C++20-cal, a fogyasztókat C17-tel és C++20-cal fordítjuk. A `-fPIC` a shared libraryhoz kell, a `-fvisibility=hidden` a rejtett szimbólumokhoz, a `PARSER_BUILD_SHARED` a Windows export makróhoz. A C fogyasztó nem linkel `-lstdc++`-t; a `libparser.so` dinamikus függőségeként a C++ runtime betöltődik. Statikus linkelésnél ezt külön kell kezelni.

```
# C++ könyvtár
g++ -std=c++20 -fPIC -fvisibility=hidden -DPARSER_BUILD_SHARED -I. \
    -c parser.cpp -o parser.o
g++ -shared -o libparser.so parser.o

# C fogyasztó
gcc -std=c17 -I. -c consumer.c -o consumer.o
gcc consumer.o -L. -Wl,-rpath,'$ORIGIN' -lparser -o consumer

# C++ fogyasztó
g++ -std=c++20 -I. -c consumer.cpp -o consumer_cpp.o
g++ consumer_cpp.o -L. -Wl,-rpath,'$ORIGIN' -lparser -o consumer_cpp

# Ellenőrzés
nm -D --defined-only libparser.so | grep parser_
./consumer
./consumer_cpp
```

Az `nm -D` a dinamikus szimbólumtáblát mutatja. Ha a `parser_create` helyett `_Z13parser_create`-t látunk, akkor az `extern "C"` hiányzik vagy nem érvényesül. Ha a `parser_create` egyáltalán nem látszik, akkor a visibility rejtett, vagy a `PARSER_API` hiányzik. A `c++filt` a mangled nevek visszafejtésére használható. A `readelf -Ws libparser.so | grep parser_` a verziózott szimbólumokat is megmutatja. A konkrét kimenet target- és toolchainfüggő; a példa a szerkezetet illusztrálja, nem állítja, hogy a fenti parancsok eredményét itt ellenőriztük.

Fogalom	Mit szabályoz	Hol jelenik meg	Tipikus hiba
language linkage	név és hívási konvenció	`extern "C"`	hiányzik: mangled név, linkelési hiba

Fogalom	Mit szabályoz	Hol jelenik meg	Tipikus hiba
visibility	dinamikus szimbólumtábla	`PARSER_API`, `-fvisibility=hidden`	hiányzik: nem exportált szimbólum
allocator pairing	ki szabadítja fel a memóriát	`create`/`destroy`	`free` a `new` pointerére
exception boundary	kivétel átlépése a C határon	`try/catch`, `noexcept`	kivétel a C hívó felé
opaque handle	layout elrejtése	előre deklarált struct	a struct definíciója a headerben

36.2.5 Tipikus hibák és javításuk

A leggyakoribb hiba, hogy a közös headerbe C++-os elem kerül: `std::string`, `class`, `template`, `bool`, `nullptr` vagy referencia. A C fordító ezt nem érti; a header nem lesz kétnyelvű. A javítás: a közös header csak C-kompatibilis deklarációkat tartalmaz, a C++ részletek a `parser.cpp`-be kerülnek.

A második hiba az `extern "C"` hiánya. A C++ fordító mangled nevet generál, a C linker nem találja a `parser_create` szimbólumot. A javítás: a deklaráció és a definíció is legyen `extern "C"` hatókörben; a headerben a `#ifdef __cplusplus` blokk, a definíciónál explicit `extern "C"`.

A harmadik hiba a visibility összekeverése a linkage-pel. Az `extern "C"` önmagában nem exportál; `-fvisibility=hidden` mellett a `PARSER_API` kell. A javítás: a nyilvános függvények deklarációján és definícióján is legyen `PARSER_API`, és a build definiálja a `PARSER_BUILD_SHARED` makrót ott, ahol kell.

A negyedik hiba a kivétel átengedése a C határon. A C hívó nem tudja elkapni; a viselkedés nem hordozható. A javítás: minden `extern "C"` függvényben `try/catch`, hibakód visszaadása, és a `noexcept` tudatos használata.

Az ötödik hiba az allocator mismatch: a C oldal `free`-vel szabadítja a C++ `new` pointerét, vagy fordítva. A javítás: a `parser_destroy` végezze a felszabadítást; a C oldal ne hívjon `free`-t a `parser_ctx*`-re. Windows DLL-határon a memóriát abban a modulban kell felszabadítani, amelyik allokálta, vagy közös allocator API-t kell használni.

A hatodik hiba az opaque handle megsértése: a `struct parser_ctx` definíciója bekerül a közös headerbe. Ettől a C oldal látja a layoutot, és a könyvtár ABI-változása esetén a C fogyasztó újrafordítása nélkül is elromolhat. A javítás: a headerben csak `typedef struct parser_ctx parser_ctx;` szerepeljen.

A hetedik hiba a `parser_feed(nullptr, 0)` hívása. A `std::string::append(nullptr, 0)` nem hordozható, mert a pointerre vonatkozó előfeltétel sérülhet. A javítás: a `nullptr` és a nulla hossz külön kezelendő, ahogy a példában.

36.2.6 Feladatok

1. **Alapfeladat.** Add hozzá a `parser_version` függvényt a headerhez és az implementációhoz úgy, hogy C és C++ módban is forduljon, és a visszaadott pointer statikus élettartamú legyen. Ellenőrizd, hogy a `nm -D` mutatja a `parser_version` szimbólumot, és hogy a C fogyasztó ki tudja írni az értékét.

2. **Közepes feladat.** Módosítsd a `parser_finish` függvényt úgy, hogy egy `parser_result` structot adjon vissza, amely tartalmaz egy `const char *data` pointert, egy `size_t len` mezőt és egy `parser_status status` mezőt. A `data` a `parser_ctx` belső bufferére mutasson, és a `parser_ctx` élettartamáig legyen érvényes; a C hívó ne szabadítsa fel. A `parser_ctx` továbbra is opaque maradjon. Írd le a pointer élettartamára vonatkozó szerződést, és készíts C fogyasztót, amely kiírja a resultot.

3. **Haladó feladat.** Készíts `Parser` nevű C++ RAII wrappert a C API köré. A wrapper egy `std::unique_ptr`-rel tartsa a `parser_ctx*`-et, saját deleterrel, amely a `parser_destroy`-t hívja. A `feed` és `finish` hibakódját alakítsd `std::system_error`-ré. A wrapper soha ne engedjen kivételt a C API felé. Írj rövid C++ tesztet, amely ellenőrzi a sikeres és a hibás `feed` viselkedését, valamint azt, hogy a wrapper destruktora felszabadítja az erőforrást.

36.2.7 Megoldások

36.2.7.1 1. feladat megoldása

A headerben a deklaráció a többi függvény mellett:

```
PARSER_API const char *parser_version(void) PARSER_NOEXCEPT;
```

Az implementációban:

```
extern "C" PARSER_API const char *parser_version(void) PARSER_NOEXCEPT {
    return "1.0";
}
```

A string literál statikus élettartamú, ezért a pointer a program teljes futása alatt érvényes; a C hívó nem szabadítja fel. A C fogyasztóban:

```
printf("parser version: %s\n", parser_version());
```

Az ellenőrzés feltétele: a header C17 és C++20 módban is fordul; a `nm -D --defined-only libparser.so | grep parser_version` mutatja a szimbólumot; a C fogyasztó helyesen írja ki az `1.0`-t. A `PARSER_NOEXCEPT` a C oldalon üres, a C++ oldalon `noexcept`; a függvény nem dob.

36.2.7.2 2. feladat megoldása

A headerben a `parser_finish` deklarációja és a result struct:

```
typedef struct parser_result {
    const char *data;
    size_t len;
    parser_status status;
} parser_result;

PARSER_API parser_result parser_finish(parser_ctx *ctx) PARSER_NOEXCEPT;
```

Az implementációban:

```
extern "C" PARSER_API parser_result
parser_finish(parser_ctx *ctx) PARSER_NOEXCEPT {
    parser_result r{nullptr, 0, PARSER_EINVAL};
    if (ctx == nullptr) {
        return r;
    }
    try {
        ctx->finished = true;
        r.data = ctx->input.data();
        r.len = ctx->input.size();
        r.status = PARSER_OK;
        return r;
    } catch (...) {
        r.status = PARSER_ESYNTAX;
        return r;
    }
}
```

A `parser_result` C-kompatibilis struct; a C++ oldalon aggregate inicializálható. A `data` a `ctx->input` belső bufferére mutat, ezért a `parser_ctx` élettartamáig érvényes, de a következő `parser_feed` vagy `parser_finish` hívás invalidálhatja. A C hívó a resultot addig használhatja, amíg a `parser_ctx`-et nem módosítja és nem destruálja. A C fogyasztó:

```
parser_result r = parser_finish(ctx);
if (r.status == PARSER_OK) {
    fwrite(r.data, 1, r.len, stdout);
}
parser_destroy(ctx);
```

Az ellenőrzés feltétele: a `parser_result` layoutja a C és C++ oldalon ugyanaz; a `parser_finish` nem dob; a `data` pointer a `parser_ctx` élettartamához kötött; a C hívó nem hív `free`-t a `data`-ra.

36.2.7.3 3. feladat megoldása

A wrapper a C API fölé épül, a C API-t változatlanul hagyja. Header-only C++ fájl, ezért a deleter nevesített `detail` névtérbe kerül (névtelen névtér headerben minden fordítási egységben külön típust adna):

```
// parser.hpp - C++20 RAII wrapper a parser.h C API-ja fölött
#pragma once
#include "parser.h"

#include <cstddef>
#include <memory>
#include <new>
#include <system_error>

namespace detail {

struct parser_deleter {
    void operator()(parser_ctx *p) const noexcept {
```

```
        parser_destroy(p);
    }
};

} // namespace detail

class Parser {
public:
    Parser() : ctx_(parser_create()) {
        if (!ctx_) {
            throw std::bad_alloc();
        }
    }

    Parser(const Parser &) = delete;
    Parser &operator=(const Parser &) = delete;
    Parser(Parser &&) noexcept = default;
    Parser &operator=(Parser &&) noexcept = default;

    void feed(const char *data, std::size_t len) {
        parser_status st = parser_feed(ctx_.get(), data, len);
        if (st != PARSER_OK) {
            throw std::system_error(static_cast<int>(st),
                                    std::generic_category(),
                                    "parser_feed");
        }
    }

    void finish() {
        parser_status st = parser_finish(ctx_.get());
        if (st != PARSER_OK) {
        throw std::system_error(static_cast<int>(st),
                                std::generic_category(),
                                "parser_finish");
        }
    }

private:
    std::unique_ptr<parser_ctx, detail::parser_deleter> ctx_;
};
```

A `parser_deleter` `noexcept`, a `parser_destroy` is az; a `unique_ptr` destruktora így nem dob. A `Parser` copy tiltott, move engedélyezett. A `feed` és `finish` kivételt dob a C++ hívó felé, de ez már a C++ oldalon történik; a C API határán a `try/catch` és a hibakód véd. A `std::system_error` a `parser_status` int értékét használja; a `std::generic_category` nem ad specifikus üzenetet, de a hibakódot hordozza. A teszt:

```
// parser_test.cpp
#include "parser.hpp"
#include <cassert>
#include <string>
```

```
int main() {
    Parser p;
    const std::string chunk = "int x = 42;";
    p.feed(chunk.data(), chunk.size());
    p.finish();

    bool threw = false;
    try {
        p.feed(nullptr, 1);
    } catch (const std::system_error &) {
        threw = true;
    }
    assert(threw);
    return 0;
}
```

Az ellenőrzés feltétele: a wrapper destruktora a `parser_destroy`-t hívja; a sikeres `feed` nem dob; a `nullptr, 1` bemenet `PARSER_EINVAL`-t ad, amelyet a wrapper `std::system_error`-ré alakít; a C API felé nem lép ki kivétel. A `Parser` move műveletei a `unique_ptr` miatt működnek; a copy tiltása megakadályozza a dupla felszabadítást.

36.2.8 Összefoglaló

A kétnyelvű header lényege, hogy a közös felület C-kompatibilis maradjon: header guard, `#ifdef __cplusplus` körüli `extern "C"` blokk, opaque handle, C-kompatibilis structok és enumok. Az `extern "C"` a nyelvi linkage-et állítja be, a visibility és az export makrók ettől függetlenül szabályozzák a dinamikus szimbólumtáblát. A C++ implementáció a `try/catch`-csel és hibakódokkal védi a C határt; a `noexcept` tudatos használata erősíti a szerződést. Az allocator pairing szerint a memóriát az allokáló modul szabadítja fel, a `create/destroy` páros ezt biztosítja. A C oldal soha nem lát C++ osztályt, sablont vagy overloadot; a C++ oldal belső részletei a shared library határ mögött maradnak. A build és az `nm/readelf` ellenőrzés a linkage és a visibility hibáit különíti el; a konkrét ABI- és szabványhivatkozásokat a laborprofilban kell ellenőrizni.

36.3 Stabil shared library határ és verziózás

A plugin alapú rendszerekben a host és a plugin jellemzően külön fordítási egységként, gyakran külön időpontban, külön toolchainnel és külön kiadási ciklusban készül. A kettő között futásidőben létrejövő kapcsolat nem forráskód-szintű, hanem bináris: a dinamikus linker által feloldott szimbólumok, a hívási konvenció, a típusok memórialayoutja, a kivételkezelés mechanizmusa és az erőforrás-tulajdonlás együttesen alkotja azt a szerződést, amelyet **ABI-szerződésnek** (application binary interface contract) nevezünk. Ez az alfejezet azt mutatja meg, hogyan lehet ezt a szerződést tudatosan kicsire és verziózhatóvá tenni, és milyen kompatibilitási kockázatok maradnak akkor is, ha a határon csak C típusok mennek át.

A tárgyalás egy konkrét GCC/Clang és libstdc++/libc++ laborprofilhoz kötött. Ahol a viselkedés implementációfüggő, azt jelöljük; a fejezet nem tartalmaz ellenőrzött szabványidézetet, a konkrét verziókat és kapcsolókat a laborprofil rögzíti.

36.3.1 Source- és binary kompatibilitás

Két különböző fogalmat kell szétválasztani. **Forráskompatibilitásról** (source compatibility) akkor beszélünk, ha a kliens forráskódja változtatás nélkül újrafordítható a könyvtár új verziójával. **Bináris kompatibilitásról** (binary compatibility) akkor, ha egy korábban lefordított kliens bináris változatlanul működik az új könyvtárral — vagy fordítva, egy korábban lefordított könyvtár az új klienssel.

A két fogalom nem fedi egymást, és egyik sem következik a másikból. Egy osztály végére felvett új virtuális függvény forráskompatibilis lehet, de a vtable layoutját megváltoztatva binárisan tör. Egy adattag hozzáadása szintén forráskompatibilis lehet, miközben a `sizeof` és az offsetek megváltoznak. Fordítva: egy szimbólum átnevezése binárisan semleges lehet, ha a régi nevet megtartó kompatibilitási szimbólum megmarad, forráskompatibilis viszont nem.

A bináris kompatibilitás mindig **két konkrét build között**, adott compiler-, stdlib-, flag- és targetprofil mellett értelmezhető. Nincs olyan állítás, hogy „a C++ ABI stabil”; a gyakorlatban az Itanium C++ ABI az egyik de facto leírás, de annak követését és részleteit a konkrét toolchain dokumentációja rögzíti, nem a nyelvi szabvány. A C határ ezzel szemben jóval kisebb felületet tesz ki: nincs névtorzítás, nincs vtable, de a struct-layout, a hívási konvenció és az allokátor továbbra is szerződéselem marad.

36.3.2 A kompatibilitást rontó fordítási tényezők

A legfontosabb felismerés, hogy a bináris határ nem csak a forráskódból áll, hanem a fordítási környezetből is. Az alábbi tényezők mindegyike önállóan képes arra, hogy két, látszólag azonos forrásból készült binárist inkompatibilissé tegyen:

- **Compiler- és stdlib-verzió.** A libstdc++ és a libc++ külön implementációk, eltérő layoutokkal; de még ugyanazon implementáció két verziója között is változhat egyes típusok belső elrendezése.
- **Debug mode.** A libstdc++ `_GLIBCXX_DEBUG` kapcsolója ellenőrzött iterátorokat és eltérő belső layoutot eredményez, ami a típusok méretét és a szimbólumneveket is érintheti. A debug és a release build objektumai nem keverhetők a határon.
- **Dual ABI.** A libstdc++ `_GLIBCXX_USE_CXX11_ABI` kapcsolója két, egymással nem kompatibilis `std::string`/`std::list` ABI-t tart fenn. A két oldalnak ugyanazt kell választania.
- **Kivétel- és RTTI-beállítások.** A `-fno-exceptions` és `-fno-rtti` önmagukban jellemzően nem változtatják meg a layoutot, de a kivételátadás a határon csak kompatibilis unwinderrel működik, RTTI nélkül pedig a `dynamic_cast` és a `typeid` nem használható a másik oldal típusain.
- **Nyelvi mód.** A `-std=` váltás bizonyos esetekben a függvénytípus részévé tett tulajdonságokat (például a `noexcept`-ot) és ezen keresztül a névtorzítást is érintheti; ezt a konkrét toolchain dokumentációja alapján kell ellenőrizni.
- **Optimalizálás és LTO.** Az inline-olt standard library kód és a link-idő optimalizálás a határon átnyúló viselkedést befolyásolhatja, különösen akkor, ha a két oldal eltérő fejléceket lát.

A védekezés nem az, hogy minden kapcsolót egyformára kényszerítünk, hanem hogy a határt olyan konstrukciókra szűkítjük, amelyekre ezek a tényezők nem hatnak.

36.3.3 STL objektumok és allocatorok a modulhatáron

A leggyakoribb hiba `std::string`, `std::vector` vagy más STL-típus átadása a modulhatáron. Ennek három különálló kockázata van. Az első a **layout**: ha a két oldal eltérő stdlib-verziót vagy debug módot használ, a típus mérete és belső elrendezése eltér, és a másik oldal a saját (rossz) értelmezésével olvassa a memóriát. A második az **allokátor**: ha az egyik modul allokálja a puffert és a másik szabadítja fel, a felszabadítás csak akkor helyes, ha mindkét oldal ugyanazt az allokátort használja. Eltérő `malloc` implementáció (például egy másik allocatorral linkelt plugin) esetén ez memóriahiba. A harmadik a **tulajdonlás**: a határon átadott objektum élettartama és felszabadítási felelőse nem derül ki a típusból.

A mérnöki szabály ezért az, hogy a C határon ne menjen át STL-típus, és az allokációs szimmetria érvényesüljön: **aki allokál, az szabadít fel**. A host által átadott buffer tulajdonosa a host marad, a plugin által létrehozott objektumot a plugin egy `destroy` függvénye szabadítja fel. Ez a szabály akkor is érvényes, ha a két oldal látszólag ugyanazt a runtime-ot használja, mert a látszat a következő frissítésnél megváltozhat.

36.3.4 Opaque handle, méret és verzió, callback struktúrák

Az **opaque handle** (átlátszatlan fogantyú) lényege, hogy a header csak a típus nevét deklarálja, a layoutot nem:

```
typedef struct plugin_handle plugin_handle;   /* a layout a .c fájlban él */
```

A kliens így csak pointert tárol, nem függ a mérettől és a mezőktől, a könyvtár pedig szabadon változtathatja a belső reprezentációt. Ez a bináris stabilitás legegyszerűbb eszköze.

A második eszköz a **méret- és verziómező**. Minden határon átadott struktúra első mezői a `struct_size` és az `abi_version`, amelyeket a fogadó fél ellenőriz. A `struct_size` lehetővé teszi a bővítést: az új mezőket a struktúra végére kell felvenni, a régi kliens kisebb méretet ad, a könyvtár pedig csak addig olvas, ameddig a méret ér. A fogadó oldal ellenőrzése:

$$\text{struct_size} \geq \text{offsetof}(T, m) + \text{sizeof}(m)$$

ahol m az utolsó olyan mező, amelyet a fogadó fél használni kíván. Ha a feltétel nem teljesül, a mező nem olvasható.

A harmadik eszköz a **callback struktúra**: függvénypointerek és egy `void *context` egyetlen, verziózott struktúrában. A context élettartama a regisztráló fél felelőssége; a könyvtár nem tarthatja meg a visszahívás után, és a hostnak a leállítás előtt törölnie kell a regisztrációt.

36.3.5 SONAME, szimbólumláthatóság és PImpl

A dinamikus linker a **SONAME** (shared object name) alapján azonosítja a könyvtárat. A szokásos séma: a valódi fájl `libdemo.so.1.2.3`, a SONAME `libdemo.so.1`, a fejlesztői symlink `libdemo.so`. A SONAME váltása (`libdemo.so.1` → `libdemo.so.2`) jelzi a bináris inkompatibilitást, és lehetővé teszi, hogy a régi és az új verzió egyszerre legyen jelen a rendszeren. A SONAME-en belüli finomabb verziózásra a **szimbólumverziózás** szolgál (version script, `.symver`); ez GNU linker kiterjesztés, nem POSIX-szabvány, ezért használatát a laborprofilban rögzíteni kell.

A **szimbólumláthatóság** (symbol visibility) alapértelmezésben minden nem statikus szimbólumot exportál, ami névütközésekhez és véletlen ABI-felülethez vezet. A `-fvisibility=hidden` fordítási kapcsolóval csak az explicit módon megjelölt szimbólumok maradnak exportáltak; C++-ban

ehhez jellemzően `-fvisibility-inlines-hidden` is társul. A megjelölés GNU/Clang esetén `__attribute__((visibility("default")))`, ami nem hordozható minden compilerre.

A **PImpl** (pointer to implementation) minta a C++ oldalon ugyanazt a célt szolgálja, mint az opaque handle a C oldalon: a header csak egy pointert tartalmaz a privát implementációra, így a privát tagok változása nem változtatja meg a nyilvános osztály méretét. Költsége egy indirekció és jellemzően egy allokáció; a destruktort a `.cpp` fájlban kell definiálni, ahol a privát típus teljes.

36.3.6 A dlclose és az élettartam lezárása

A `dlclose` nem egyszerű memóriafelszabadítás: a könyvtár kódjára mutató minden pointer, a benne létrehozott objektum, a regisztrált callback és a coroutine frame érvénytelenné válik. A dinamikus linker nem garantálja, hogy a könyvtár statikus destruktorai minden esetben lefutnak, és azt sem, hogy a könyvtár kódja a `dlclose` után bármikor biztonságosan hívható. A helyes sorrend ezért:

1. a plugin saját `stop`/`shutdown` függvénye leállítja a szálakat és a coroutine-okat;
2. a `destroy` felszabadítja a plugin által allokált objektumokat;
3. a host törli a callbackregisztrációkat, és nem tart meg a pluginra mutató pointert;
4. csak ezután következik a `dlclose`.

A coroutine külön figyelmet érdemel: a frame resumption pointert tartalmazhat a plugin kódjára, ezért ha a host a `dlclose` után folytatja a coroutine-t, az nem definiált viselkedés. A frame-et a plugin allokátorával és a plugin kódjával kell felszabadítani, még a `dlclose` előtt; erről részletesebben a Dangling capture, resumption és cancellation (35.3) szól.

36.3.7 Kidolgozott példa: verziózott plugin API

A példa egy minimális, verziózott C határt mutat. A host C++20-ban, a plugin C17-ben készül; a határon csak C típusok és opaque handle mennek át.

```
/* plugin_api.h */
#ifndef PLUGIN_API_H
#define PLUGIN_API_H

#include <stdint.h>

#if defined(__GNUC__) || defined(__clang__)
#  define PLUGIN_API __attribute__((visibility("default")))
#else
#  define PLUGIN_API
#endif

#ifdef __cplusplus
extern "C" {
#endif

#define PLUGIN_ABI_VERSION 1u

typedef struct plugin_handle plugin_handle;   /* opaque */

typedef struct plugin_host {
```

```
    uint32_t struct_size;      /* = sizeof(plugin_host) a hívó oldalán */
    uint32_t abi_version;      /* = PLUGIN_ABI_VERSION */
    void    *context;
    void   (*log)(void *context, int level, const char *msg);
} plugin_host;

typedef struct plugin_vtable {
    uint32_t struct_size;
    uint32_t abi_version;
    const char *(*name)(void);
    plugin_handle *(*create)(const plugin_host *host);
    int  (*start)(plugin_handle *h);
    void (*stop)(plugin_handle *h);
    void (*destroy)(plugin_handle *h);
} plugin_vtable;

PLUGIN_API const plugin_vtable *plugin_init(uint32_t host_abi_version);

#ifdef __cplusplus
}
#endif
#endif

/* plugin.c */
#include "plugin_api.h"
#include <stddef.h>
#include <stdlib.h>

struct plugin_handle {
    const plugin_host *host;
    int running;
};

static const char *impl_name(void) { return "demo-plugin"; }

static plugin_handle *impl_create(const plugin_host *host) {
    if (host == NULL) return NULL;
    if (host->struct_size < offsetof(plugin_host, log) + sizeof(host->log))
        return NULL;
    if (host->abi_version != PLUGIN_ABI_VERSION) return NULL;
    plugin_handle *h = malloc(sizeof *h);
    if (h == NULL) return NULL;
    h->host = host;
    h->running = 0;
    return h;
}

static int impl_start(plugin_handle *h) {
    if (h == NULL) return -1;
    h->running = 1;
    if (h->host->log != NULL) h->host->log(h->host->context, 1, "started");
    return 0;
```

```
}

static void impl_stop(plugin_handle *h) {
    if (h == NULL) return;
    h->running = 0;
    if (h->host->log != NULL) h->host->log(h->host->context, 1, "stopped");
}

static void impl_destroy(plugin_handle *h) { free(h); }

static const plugin_vtable demo_vtable = {
    .struct_size = sizeof(plugin_vtable),
    .abi_version = PLUGIN_ABI_VERSION,
    .name        = impl_name,
    .create      = impl_create,
    .start       = impl_start,
    .stop        = impl_stop,
    .destroy     = impl_destroy,
};

PLUGIN_API const plugin_vtable *plugin_init(uint32_t host_abi_version) {
    if (host_abi_version != PLUGIN_ABI_VERSION) return NULL;
    return &demo_vtable;
}

// host.cpp
#include "plugin_api.h"
#include <dlfcn.h>
#include <cstdio>

namespace {
void host_log(void *ctx, int level, const char *msg) {
    const auto *tag = static_cast<const char *>(ctx);
    std::fprintf(stderr, "[%s/%d] %s\n", tag, level, msg);
}
} // namespace

int main(int argc, char **argv) {
    if (argc != 2) {
        std::fprintf(stderr, "usage: %s <plugin.so>\n", argv[0]);
        return 2;
    }
    void *lib = dlopen(argv[1], RTLD_NOW | RTLD_LOCAL);
    if (!lib) { std::fprintf(stderr, "dlopen: %s\n", dlerror()); return 1; }

    using init_fn = const plugin_vtable *(*)(uint32_t);
    auto *init = reinterpret_cast<init_fn>(dlsym(lib, "plugin_init"));
    if (!init) {
        std::fprintf(stderr, "dlsym: %s\n", dlerror());
        dlclose(lib);
        return 1;
    }
```

```
    const plugin_vtable *vt = init(PLUGIN_ABI_VERSION);
    if (!vt || vt->struct_size < sizeof(plugin_vtable)
            || vt->abi_version != PLUGIN_ABI_VERSION) {
        std::fprintf(stderr, "incompatible plugin ABI\n");
        dlclose(lib);
        return 1;
    }

    static char host_tag[] = "host";
    plugin_host host{};
    host.struct_size = sizeof(host);
    host.abi_version = PLUGIN_ABI_VERSION;
    host.context     = host_tag;
    host.log         = host_log;

    plugin_handle *h = vt->create(&host);
    if (!h) { dlclose(lib); return 1; }

    int rc = vt->start(h);
    std::printf("plugin=%s rc=%d\n", vt->name(), rc);
    vt->stop(h);
    vt->destroy(h);   // objektum élettartamának lezárása
    vt = nullptr;     // a vtable-re mutató pointer is invalidálódik
    dlclose(lib);
    return rc == 0 ? 0 : 1;
}
```

A javasolt buildparancsok (a tanuló laborjában futtatandók, a laborprofilhoz igazítva):

```
gcc -std=c17 -O2 -fPIC -fvisibility=hidden -shared \
    -Wl,-soname,libdemo.so.1 \
    -o libdemo.so.1.0.0 plugin.c
ln -sf libdemo.so.1.0.0 libdemo.so.1
ln -sf libdemo.so.1 libdemo.so

g++ -std=c++20 -O2 -Wall -Wextra -o host host.cpp -ldl
./host ./libdemo.so.1
```

A példa három tanulsága: a `struct_size` és az `abi_version` ellenőrzése a plugin oldalán történik, mielőtt bármely mezőt olvasna; a `plugin_init` a `PLUGIN_API` miatt exportált, minden más szimbólum rejtett; a `destroy` és a `dlclose` sorrendje biztosítja, hogy a plugin kódjára a könyvtár eltávolítása után ne maradjon hivatkozás. A kód a bemutatott formában nem lett lefuttatva; a viselkedés a konkrét toolchainen ellenőrizendő.

36.3.8 Tipikus hibák és javításuk

A leggyakoribb hiba az STL-típus átadása a határon. Javítás: `const char *` + hossz, vagy opaque handle, és a felszabadítás a tulajdonos oldalán. A második a `struct_size` ellenőrzésének elhagyása: a régi plugin olvassa az új mezőt, vagy fordítva. Javítás: minden határon átadott struktúra első mezője a méret, és a fogadó fél a fenti egyenlőtlenséggel ellenőriz. A harmadik a `dlclose` a `destroy` előtt: a plugin kódjára mutató pointer dangling lesz. Javítás: a fenti négy

lépéses sorrend betartása. A negyedik a `-fvisibility=hidden` elhagyása: minden szimbólum exportált lesz, ami névütközéshez vezethet. Javítás: rejtett alapértelmezés és explicit export. Az ötödik a SONAME elhagyása: a linker a fájlnevet jegyzi meg, és a frissítés után a dinamikus linker nem találja a könyvtárat.

36.3.9 Feladatok

1. **Magyarázó feladat.** Különböztesd meg a forrás- és a bináris kompatibilitást, és mondj egy-egy olyan változtatást, amely az egyiket töri, a másikat nem.

2. **Kódolási feladat.** Bővítsd a `plugin_host` struktúrát egy új, opcionális `warn` callback-kel úgy, hogy egy régi host binárisa változatlanul működjön az új pluginnal, és egy régi plugin binárisa az új hosttal.

3. **Diagnosztikai feladat.** A plugin frissítése után a host a `create` hívásban szegmentációs hibával leáll. Milyen okokat vizsgálsz, és milyen sorrendben?

4. **Élettartam-feladat.** A plugin egy coroutine-t indít, amelyet a host eseményhurok folytat. Mit kell tenni a `dlclose` előtt, és hogyan ellenőrzöd, hogy nem maradt a plugin kódjára mutató hivatkozás?

36.3.10 Megoldások

1. Megoldás. A forráskompatibilitás a forrás újrafordíthatóságát jelenti, a bináris kompatibilitás a már lefordított bináris változatlan működését. Példa forráskompatibilis, de binárisan törő változtatásra: egy adattag hozzáadása egy nyilvános struktúrához (a forrás újrafordul, de a `sizeof` és az offsetek megváltoznak). Példa binárisan semleges, de forráskompatibilitást törő változtatásra: egy nyilvános függvény átnevezése, miközben a régi nevet megtartó kompatibilitási szimbólum megmarad. *Ellenőrzési feltétel:* a válasz nevezze meg a két fogalom különbségét, és a példákban mutassa meg, melyik oldal fordul újra.

2. Megoldás. A `warn` mezőt a `plugin_host` végére kell felvenni, és a plugin a `struct_size` alapján dönt: csak akkor hívja meg, ha

$$\text{host->struct_size} \geq \text{offsetof}(\text{plugin_host}, \text{warn}) + \text{sizeof}(\text{host->warn}).$$

A régi host kisebb `struct_size`-t ad, ezért a plugin nem hívja a `warn`-ot; a régi plugin nem ismeri a `warn` mezőt, ezért nem olvassa. A host oldalán a `warn` pointert a `plugin_host` inicializálásakor kell beállítani, és a hostnak a `dlclose` előtt törölnie kell a regisztrációt. *Ellenőrzési feltétel:* a régi host binárisa változatlanul fusson az új pluginnal, és az új host a régi pluginnal; a `warn` csak akkor hívódjon meg, ha mindkét oldal ismeri.

3. Megoldás. Elsőként a `struct_size` és az `abi_version` ellenőrzését kell megvizsgálni: hiányzik-e a plugin oldalán, vagy a host rossz értéket ad. Ezután a vtable layoutját: nem került-e új függvény a meglévők közé (a végére felvett új függvény a régi hostot nem töri, a közbeszúrt igen). Ezután a betöltött könyvtár azonosságát: a host a régi SONAME-jú fájlt tölti-e be. Végül a szimbólumütközést: a plugin `-fvisibility=hidden` nélkül exportált-e azonos nevű szimbólumot. *Ellenőrzési feltétel:* a vizsgálat nevezze meg a `struct_size`-ellenőrzés hiányát mint leggyakoribb okot, és adjon rá konkrét javítást.

4. Megoldás. A `dlclose` előtt a plugin `stop`/`shutdown` függvényének be kell fejeznie vagy fel kell szabadítania a coroutine-t a plugin allokátorával és kódjával; a hostnak törölnie kell a coroutine-handle-t és a callbackregisztrációkat; a plugin szálait joinolni kell. A `dlclose` után

a host nem tarthat a plugin kódjára mutató pointert. *Ellenőrzési feltétel:* a plugin `shutdown` függvénye adja vissza a nyitott erőforrások számát, és ez a `dlclose` előtt nulla legyen; a folyamat sanitizerrel (például AddressSanitizerrel) fusson a laborban.

36.3.11 Összefoglaló

A stabil shared library határ nem egyetlen trükk, hanem szerződés: a bináris kompatibilitás mindig konkrét compiler-, stdlib- és flagprofilhoz kötött, ezért a határt a lehető legkisebbre kell szűkíteni. A C típusok, az opaque handle, a `struct_size`/`abi_version` mezők, a verziózott callback struktúrák, a SONAME és a rejtett szimbólumláthatóság együtt csökkentik a bináris couplingot. Az STL-típusok és az allokátorok átadása a határon továbbra is kockázat; a szabály az allokációs szimmetria és a „csak C típus a határon”. A `dlclose` nem takarít automatikusan: a plugin objektumainak, callbackjeinek és coroutine-jainak élettartamát a `dlclose` előtt explicit módon le kell zárni.

36.4 Python és más nyelvek FFI-kapcsolata

Az FFI (foreign function interface) az a mechanizmus, amelyen keresztül egy nyelvi runtime egy másik, natív kódot hív, illetve abból adatot fogad. DevOps-környezetben ez nem egzotikus eset: platform-eszközök íródnak Pythonban egy C++ mag köré, megfigyelő ágensek töltenek be natív bővítményt, CLI-k hívnak meg egy C ABI-n exportált gyorsított könyvtárat, és egyre gyakrabban épül Python-csomag egy C++ motorra. Az ilyen rendszerek hibái ritkán nyelvi szintaktikai hibák: jellemzően egy pointer élettartama, egy buffer tulajdonjoga vagy egy callback szálkontextusa nem illeszkedik a két oldal feltevéseihez.

Ez az alfejezet ezért nem a binding-könyvtárak használatának miként jét tanítja, hanem azt a szerződést, amelyet minden ilyen határon rögzíteni kell. A C ABI-t tekintjük a találkozási pontnak: a korábbi Stabil shared library határ és verziózás részben rögzített exportált felület itt kapja meg a másik nyelv felőli értelmezését. A buffer- és nézet-szerződéshez a span, byte és bounds-szerződések, a szövegátadáshoz a Szövegkódolás, locale és UTF-8 C-ben rész definícióit használjuk fel; azokat itt nem ismételjük meg, csak a határra vonatkozó következményeiket.

36.4.1 Az FFI-határ mint szerződés

Definíció. Az FFI egy adott nyelvi runtime és egy másik, jellemzően C ABI-t biztosító natív kód közötti hívási és adatátadási mechanizmus, amelyet a hívó oldal runtime-specifikus binding-rétege valósít meg. A binding-réteg (ctypes, cffi, pybind11, JNI, cgo) nem szünteti meg a határt, csak lefordítja rá a két oldal típusait.

Előfeltételek. A határon átvihető felületre egyszerre öt feltételnek kell teljesülnie:

1. Csak `extern "C"` linkage-dzsel exportált függvények és fix layoutú, C-kompatibilis típusok szerepelnek a határon.
2. Egyetlen C++ exception sem lép ki a határfüggvényből; a hibát kóddá kell alakítani a natív oldalon.
3. Minden átadott pointer vagy nem tulajdonló kölcsönzés (a hívás idejére érvényes), vagy dokumentált tulajdonátadás; a kettő nem keverhető.
4. Minden C oldalon létrehozott erőforrásnak van páros `*_destroy` / `*_release` függvénye, és a fogadó oldal garantálja annak meghívását.
5. A callback csak dokumentált szál- és interpreter-kontextusban hívható.

A 3. pont a leggyakrabban megsértett feltétel, ezért érdemes minden határfelülethez tulajdonlási táblát írni. Egy tipikus minta:

Erőforrás	Tulajdonos a C oldalon	Tulajdonos a fogadó oldalon	Felszabadítás
bemeneti byte-buffer	nincs (kölcsönzött)	a hívó (Python `bytes`/`bytearray`)	a fogadó GC-je
callback thunk / függvénypointer	nincs	a fogadó (a `CFUNCTYPE` objektum)	a fogadó GC-je, de csak deregisztráció után
opaqu handle (`struct ds_ctx*`)	a könyvtár allokálja	a fogadó wrapper tartja	`ds_ctx_destroy`
hibaüzenet-sztring	a könyvtár statikus puffere	nincs	nem szabad felszabadítani

36.4.2 A C ABI-wrapper: ctypes és cffi alapötlet

A Python felől két alapvető megközelítés van. A **ctypes** futásidőben tölti be a shared libraryt, és a hívás helyén deklarált `argtypes`/`restype` alapján konvertál: nincs fordítási lépés, de a típushelyesség csak a hívás pillanatában derül ki. A **cffi** két módot kínál: ABI-módban ugyanúgy futásidőben dolgozik, API-módban viszont a deklarációból C-bővítményt generál, így a fejlécet a fordító ellenőrzi — ez a fejléc és a binding szinkronját erősíti, de a mögöttes hívás továbbra is a C ABI-n megy át.

Mindkét esetben ugyanaz a tervezési következtetés: a C++ oldalon érdemes egy vékony, kizárólag C típusokat használó wrappert exportálni, nem a C++ osztályokat közvetlenül kitenni. Ennek nem csak a névtorzítás az oka (azt a Névtorzítás, layout és C++ runtime ABI rész tárgyalta), hanem az, hogy a wrapper az egyetlen hely, ahol a hibakonverzió, a `noexcept`-garancia és a paraméter-ellenőrzés egy ponton tartható.

36.4.3 Buffer pointer és hossz: a kölcsönzés szabálya

A natív felület soha ne `const char*`-ban hordozza a hosszt, ha a bemenet bináris vagy UTF-8 lehet. A helyes minta `(const uint8_t* data, size_t len)`, ahol a szerződés:

- `data == NULL` pontosan akkor megengedett, ha `len == 0`;
- a függvény a `[data, data+len)` tartományt csak a hívás idejére olvashatja;
- a függvény nem tárolhatja el a pointert, és nem adhat vissza olyan pointert, amely a bemenetre mutat.

A C++ oldalon ezt természetesen fejezi ki a `std::span<const std::byte>` (illetve `std::span<const std::uint8_t>`) kölcsönzött nézet, feltéve, hogy a span nem kerül ki a hívás élettartamából. A Python oldalon a buffer objektumnak a teljes hívás alatt élnie kell. CPython esetén a `bytes` argumentumként átadva a konverzió idejére referenciát tart, így a hívás alatt nem szabadul fel; ez runtime-specifikus garancia, ezért dokumentálni kell, és nem szabad rá építeni más implementációnál. Ha a natív kód a hívás után is hivatkozna a bufferre, az a klasszikus dangling eset: a javítás vagy mély másolás a C oldalon, vagy egy explicit, a fogadó oldalon életben tartott holder objektum.

36.4.4 UTF-8, beágyazott NUL és méretkonverzió

Python `str`-t soha nem adunk át közvetlenül: a kódolás ilyenkor a konverziótól függ, és nem az, amit a natív oldal feltételez. A helyes út: a fogadó oldalon `s.encode("utf-8")`, és a natív oldal hosszal dolgozik. Ezzel együtt jön a beágyazott NUL problémája. A forrásban leírt `b"a\x00b"` literál (7 karakternyi forrásszöveg) futásidőben **3 bájt**: `61 00 62` — Pythonban `len(b"a\x00b") == 3`. Ha a C oldal `strlen`-t használ, az első NUL-nál megáll, és **1** bájtot lát (`strlen == 1`), holott 3 érkezett. (A `lab-gcc` profilban egy `const char s[] = "a\0b";` tömbre `sizeof s - 1 == 3`, `strlen(s) == 1` adódott.) A hosszátadó API ezt kiküszöböli, de a dokumentációnak ki kell mondania, hogy a NUL közönséges byte, nem lezáró.

A méretkonverzió külön ellenőrzést igényel. Python `int`-ről `size_t`-re váltva a feltétel $0 \le v \le$ SIZE_MAX, ahol a `SIZE_MAX` az adott adatmodelltől függ (LP64 alatt 64 bites). ctypes-ban a `c_size_t` a cél architektúra szerint konvertál, de a tartományellenőrzést nem pótolja: negatív vagy túl nagy értéknél a hívás előtt explicit ellenőrzés kell. A határon használt típusok megfeleltetése:

C típus	ctypes típus	Megjegyzés
`const uint8_t*` + `size_t`	`POINTER(c_uint8)` + `c_size_t`	nem NUL-terminált, hossz kötelező
`size_t`	`c_size_t`	előjel nélküli, szélessége adatmodell-függő
`uint64_t*` (out)	`POINTER(c_uint64)`	a hívó birtokolja, a hívás előtt allokált
`int` (hibakód)	`c_int`	0 = siker, negatív = hiba
`const char*` (statikus)	`c_char_p`	csak olvasás, nem szabad felszabadítani

A kódszám és a byte-szám kapcsolatára egy hasznos vázlat: UTF-8-ban a folytató byte-ok `10xxxxxx` alakúak, a többi byte kódpont-kezdő, ezért a kódpontok száma nem haladhatja meg a byte-számot, azaz $0 \le n_{\text{cp}} \le n_{\text{byte}}$. Ez vázlat, nem teljes bizonyítás: a felülbecslésből adódó különbség éppen a többszörös hosszúságú sorozatok száma. A gyakorlati következtetés az, hogy a natív API byte-ban számoljon, a kódpontszámot pedig csak akkor adja vissza, ha a szerződés ezt kéri.

36.4.5 Foreign callback, szálak és interpreter-szabályok

A callback a legkényesebb pont, mert két élettartamot és két szálkontextust kapcsol össze. A natív oldal csak egy `void* user` és egy függvénypointert lát; arról semmit nem tud, hogy a mögötte lévő Python callable mikor szabadul fel, és melyik szálon hívható.

Élettartam. A fogadó oldalnak a regisztráció teljes ideje alatt életben kell tartania a `CFUNCTYPE` objektumot, mert az hordozza a C thunkot. Ha a Python objektum GC-elődik, a natív oldalon marad egy dangling függvénypointer. A javítás: a wrappert egy osztály fogja össze, amely a `CFUNCTYPE` példányt attribútumként tartja, és a `release()` a natív deregisztráció után engedi csak el.

Szál és interpreter. CPython esetén a Python C API-t csak a GIL birtokában szabad hívni, és a natív szálon futó callbackhez előbb csatlakozni kell az interpreterhez (például `PyGILState_Ensure/Release` párossal). Ha a hívó Python szál a GIL birtokában blokkol egy hosszú C híváson, miközben a C könyvtár egy másik szálon próbál callbacket indítani, a callback nem tud GIL-hez jutni. **Vázlat a holtpont feltételére:** legyen T_p a Python szál, amely a GIL-t

tartva vár a C hívás visszatérésére, és T_c a C könyvtár szálja, amely a callback futtatásához GIL-t kér. Ha T_p a várakozás előtt nem engedte el a GIL-t, T_c nem tud előrehaladni, T_p pedig nem tud visszatérni: körkörös várakozás. A javítás vagy a GIL elengedése a blokkoló C hívás körül (`Py_BEGIN_ALLOW_THREADS`/`Py_END_ALLOW_THREADS`), vagy a callback marshalolása arra a szálra, amelyik az interpreterhez tartozik.

Korlát. A GIL és a `PyGILState` API CPython-specifikus, és a GIL nélküli (free-threaded) mód viselkedése verziófüggő; ezt az állítást a használt Python-kiadás dokumentációjával kell ellenőrizni, és itt nem tekintjük igazoltnak.

36.4.6 Hibakód-, exception- és release-szerződés

A határfüggvényből C++ exception nem léphet ki: ha mégis, a `noexcept` következtében a folyamat leáll, ami a leggyakoribb „néha elszáll a szkriptem” hibaforrás. A minta: a wrapper belül `try`/`catch (...)`-ben mindent elkap, és stabil hibakódot ad vissza; a hibaüzenetet vagy a hívó által adott bufferbe írja, vagy statikus, csak olvasható sztringként teszi elérhetővé. Utóbbi élettartama a program teljes futása, ezért nem szabad felszabadítani, és nem szabad hozzáírni.

A fogadó oldal a hibakódot azonnal alakítsa át saját kivételévé (Pythonban `raise`), és ne engedje, hogy a natív kód hívjon vissza a fogadó runtime-ba hibakezelés céljából. Ez a szabály megfordíthatatlan irányt ad: a hibakonverzió mindig a fogadó oldalon történik, a natív oldal csak adatot ad át.

Az erőforrás-felszabadítás ugyanígy szerződés. Pythonban a `__del__` nem megbízható időzítésű; helyette `contextlib.closing`, `weakref.finalize` vagy egy `PyCapsule` destruktor a javasolt minta. A natív oldalnak a `*_destroy` függvényt idempotenssé vagy legalább nullázás után hívhatóvá kell tennie, hogy a dupla felszabadítás ne legyen azonnali UB.

36.4.7 C++ binding return policy: csak ellenőrzött dokumentációból

A pybind11-jellegű könyvtárak a visszatérési érték tulajdonjogát külön policy-vel fejezik ki: másolás, mozgatás, tulajdon átvétele, hivatkozás, illetve a szülő objektumhoz kötött belső hivatkozás. A döntési tengely mindig ugyanaz: ki birtokolja a memóriát, és meddig él a hivatkozott objektum. A „hivatkozás a szülőre” típusú policy csak akkor helyes, ha a binding réteg garantáltan életben tartja a szülőt; enélkül a Python objektum egy már felszabadult C++ objektumra mutat, és a hiba a későbbi olvasásnál jelentkezik.

Korlát. A konkrét policy-nevek és pontos szemantikájuk verziófüggők. Ebben az egységben nem áll rendelkezésre ellenőrzött forrásjegyzet a pybind11 adott kiadásához, ezért a fenti leírás a döntési szempontokat rögzíti, nem a könyvtár dokumentált viselkedését; használat előtt a választott verzió hivatalos dokumentációjában kell ellenőrizni. Ugyanez vonatkozik a cffi és a ctypes konverziós részleteire: ezek CPython-verzióhoz kötött viselkedések.

36.4.8 Kidolgozott példa: `dsprobe` C ABI és ctypes kötés

A példa egy kölcsönzött byte-tartományon dolgozó, callbacket is hívó natív könyvtár. A felület szándékosan csak C típusokat használ, és minden függvény hibakódot ad vissza.

```
/* dsprobe.h */
#ifndef DSPROBE_H
#define DSPROBE_H
#include <stddef.h>
```

```c
#include <stdint.h>
#ifdef __cplusplus
#define DSPROBE_NOEXCEPT noexcept   /* C++-ban a deklaráció is noexcept:
 ↪  egyezzen a definícióval */
extern "C" {
#else
#define DSPROBE_NOEXCEPT
#endif

/* user -> a hívó kontextusa, a könyvtár nem értelmezi.
   A data/len csak a hívás idejére kölcsönzött. */
typedef int (*dsprobe_visit_fn)(void* user, const uint8_t* data, size_t len);

int dsprobe_checksum(const uint8_t* data, size_t len, uint64_t* out)
 ↪  DSPROBE_NOEXCEPT;
int dsprobe_visit_tokens(const uint8_t* data, size_t len,
                         dsprobe_visit_fn fn, void* user) DSPROBE_NOEXCEPT;
/* A visszaadott pointer statikus, csak olvasható, nem szabad felszabadítani.
 ↪  */
const char* dsprobe_error_string(int code) DSPROBE_NOEXCEPT;

#ifdef __cplusplus
}
#endif
#endif

// dsprobe.cpp - C++20, GCC/Clang, x86-64 System V ABI labor
#include "dsprobe.h"

#include <cstdint>   // std::uint64_t; a <stdint.h> nem garantálja a std::
 ↪  neveket

namespace {
constexpr std::uint64_t kFnvOffset = 14695981039346656037ull;
constexpr std::uint64_t kFnvPrime  = 1099511628211ull;
}

extern "C" int dsprobe_checksum(const std::uint8_t* data, std::size_t len,
                                std::uint64_t* out) noexcept {
    if (out == nullptr) return -1;
    if (data == nullptr && len != 0) return -1;
    std::uint64_t h = kFnvOffset;
    for (std::size_t i = 0; i < len; ++i) { h ^= data[i]; h *= kFnvPrime; }
    *out = h;  // teljes kimenet csak siker esetén íródik
    return 0;
}

extern "C" int dsprobe_visit_tokens(const std::uint8_t* data, std::size_t len,
                                    dsprobe_visit_fn fn, void* user) noexcept {
    if (fn == nullptr) return -1;
    if (data == nullptr && len != 0) return -1;
    std::size_t start = 0;
```

```
    while (start < len) {
        while (start < len && data[start] == ' ') ++start;
        std::size_t end = start;
        while (end < len && data[end] != ' ') ++end;
        if (end > start) {
            const int rc = fn(user, data + start, end - start);
            if (rc != 0) return rc;  // a callback hibakódja változatlanul
             ↪ továbbmegy
        }
        start = end;
    }
    return 0;
}

extern "C" const char* dsprobe_error_string(int code) noexcept {
    switch (code) {
        case 0:  return "ok";
        case -1: return "invalid argument";
        default: return "unknown error";
    }
}
```

```
g++ -std=c++20 -O2 -fPIC -shared -fvisibility=hidden \
    -o libdsprobe.so dsprobe.cpp
```

A Python oldal a hosszt explicit módon adja át, a callbacket pedig pointer+hossz formában kapja meg, hogy a beágyazott NUL ne csonkítsa a szöveget:

```
import ctypes
from ctypes import (c_int, c_uint8, c_uint64, c_size_t, c_void_p,
                    POINTER, CFUNCTYPE)

lib = ctypes.CDLL("./libdsprobe.so")
lib.dsprobe_checksum.argtypes = [POINTER(c_uint8), c_size_t, POINTER(c_uint64)]
lib.dsprobe_checksum.restype = c_int
lib.dsprobe_visit_tokens.argtypes = [POINTER(c_uint8), c_size_t, c_void_p,
 ↪  c_void_p]
lib.dsprobe_visit_tokens.restype = c_int

payload = "héllo\x00világ".encode("utf-8")          # 7 byte NUL-lal együtt
buf = (c_uint8 * len(payload)).from_buffer_copy(payload)

out = c_uint64(0)
rc = lib.dsprobe_checksum(buf, len(payload), ctypes.byref(out))
assert rc == 0, lib.dsprobe_error_string(rc)

VISIT = CFUNCTYPE(c_int, c_void_p, POINTER(c_uint8), c_size_t)
seen = []

@VISIT
def on_token(user, ptr, n):                 # n byte, nem strlen
    seen.append(ctypes.string_at(ptr, n))
    return 0
```

```
rc = lib.dsprobe_visit_tokens(buf, len(payload), on_token, None)
assert rc == 0
print(seen)                                  # a NUL a token része marad
```

Három pont, amit érdemes észrevenni. A `bytes` objektum a hívás idejére életben van, de a natív oldal a `buf`-ra nem tart referenciát; a `from_buffer_copy` által létrehozott tömb a Python oldal tulajdona. A `VISIT` és az `on_token` a hívás teljes ideje alatt hatókörben van, tehát a thunk nem szabadul fel — ezt a wrapper osztálynak kellene garantálnia, ha a regisztráció hosszabb életű lenne. Végül a callback `POINTER(c_uint8)` és `c_size_t` párost kap, nem `c_char_p`-t: utóbbi a NUL-nál elvágna, és pont a beágyazott NUL esetét rontaná el.

36.4.9 Gyakori hibák és javításuk

A leggyakoribb hiba a `str` átadása `bytes` helyett: a kódolás ilyenkor nem az, amit a natív oldal vár. A második a `strlen`-alapú natív API hívása hosszátadás nélkül, ami beágyazott NUL-nál csonkít. A harmadik a callback objektum élettartamának elszalasztása: a `CFUNCTYPE` példány lokálisan keletkezik, majd a regisztráció után felszabadul, és a natív oldal később dangling pointert hív. A negyedik a GIL elengedésének hiánya hosszú C hívás körül, ami a fenti holtpont-vázlatot valósítja meg. Az ötödik a `size_t` méretkonverzió ellenőrzés nélkül hagyása, ami negatív vagy túl nagy értéknél csendes átforduláshoz vezethet.

36.4.10 Összefoglaló

Az FFI-határ szerződés, nem varázslat. A biztonságos minta négy elemből áll: `extern "C"` wrapper csak C-kompatibilis típusokkal; pointer+hossz alapú, nem tulajdonló bufferátadás dokumentált élettartammal; hibakód- és release-szerződés, amelyben a konverzió mindig a fogadó oldalon történik; valamint a callback élettartamának és szálkontextusának explicit kezelése. A verziófüggő részleteket (GIL-viselkedés, binding-policy-k, konverziós szabályok) minden esetben a használt runtime és könyvtár dokumentációjából kell ellenőrizni.

36.4.11 Feladatok

1. feladat (tervezés). Adj egy `int dsprobe_count_byte(const uint8_t* data, size_t len, uint8_t what, size_t* out)` függvény C ABI-szerződését: mikor megengedett a `NULL`, mi történik hibánál, mit ír ki a függvény, és hogyan biztosítod, hogy a beágyazott NUL ne okozzon csonkolást? Írd le a Python oldali hívás előfeltételeit is.

2. feladat (kódolás). Egészítsd ki a fenti ctypes-kötést úgy, hogy a `dsprobe_visit_tokens` hibakódját a fogadó oldal Python kivételévé alakítsa, és a hibaüzenetet a `dsprobe_error_string` adja. A wrapper osztály tartsa életben a callbacket, és legyen `close()` metódusa.

3. feladat (diagnosztika). Egy szolgáltatásban a Python szál kihívja a natív `run()`-t, amely belül egy munkaszálon meghívja a regisztrált callbacket. A folyamat holtpontba kerül, majd egy külön futásban szegmentációs hibával elszáll. Add meg a két hiba lehetséges okát, a diagnosztikai lépéseket és a javítást.

36.4.12 Megoldások

1. megoldás. A szerződés: `data == NULL` csak `len == 0` esetén megengedett; `out == NULL` mindig hiba; a függvény a `[data, data+len)` tartomány minden byte-ját vizsgálja, `strlen`-t nem használ, ezért a `0x00` közönséges értékként számolódik. Visszatérés: `0` siker, `-1` érvénytelen argumentum; `*out`-ba csak siker esetén ír. Python oldalon a hívás előtt ellenőrizni kell, hogy `0 <= len <= SIZE_MAX`, a buffer pedig a hívás teljes ideje alatt él. *Ellenőrzési feltétel:* a `b"a\x00b"` bemenetre az eredmény 3 hosszú bemenetet tükröz, nem 1-et, és a `data=NULL, len=5` hívás `-1`-et ad.

2. megoldás. A wrapper osztály `__init__`-ben mentse el a `CFUNCTYPE` példányt attribútumba, és a natív regisztrációt csak utána végezze. A hívást `try`/`except` fogja körbe; nem nulla visszatérésnél `RuntimeError`-t kell dobni a `dsprobe_error_string(rc)` szövegével. A `close()` először a natív deregisztrációt hívja (ha van ilyen), majd törli a callback-referenciát. *Ellenőrzési feltétel:* ha a callback `1`-et ad vissza, a Python oldalon kivétel keletkezik, és a natív oldalon nem szivárog erőforrás; ha a `close()` után újra hívunk, az nem szegmentációs hiba, hanem dokumentált hiba.

3. megoldás. A holtpont valószínű oka: a Python szál a GIL birtokában vár a `run()` visszatérésére, miközben a munkaszál a callbackhez GIL-t kér — ez a fenti vázlat szerinti körkörös várakozás. A javítás a GIL elengedése a blokkoló natív hívás körül, vagy a callback marshalolása. A szegmentációs hiba valószínű oka: a callbacket hordozó Python objektum felszabadult, miközben a natív oldal még tárolja a függvénypointert. Diagnosztika: a callback regisztráció és a hívás időpontjának naplózása, a wrapper élettartamának ellenőrzése, valamint a natív oldal azon pontjainak átvizsgálása, ahol pointert tárol. *Ellenőrzési feltétel:* a javított változatban a callback objektum életben marad a deregisztrációig, és a blokkoló hívás alatt a GIL elérhető más szálak számára.

36.5 ABI- és FFI-labor, kompatibilitási teszt

A C záróprojekt stream-alapú diagnosztikai szolgáltatása akkor cserélhető biztonságosan C++ implementációra, ha a C fejléc és a C kliens változatlan marad. Ez a labor ezt a cserét teszi ellenőrizhetővé. Nem azt bizonyítja, hogy bármely C++ fordító és bármely standard library között a könyvtár binárisan hordozható, hanem azt, hogy egy rögzített toolchain-profilban a C API szerződése, a szimbólumkészlet, a handle-élettartam és a memóriahatár együtt tesztelhető. A labor eredménye ezért mindig profilhoz kötött: a fordítók, a target ABI, a libc/libstdc++ és a build kapcsolók rögzítése nélkül a „kompatibilis” állítás nem értelmezhető.

36.5.1 A kompatibilitás rétegei és a labor szerződése

Négy fogalmat kell külön tartani. A **forráskompatibilitás** azt jelenti, hogy a kliens forrása újrafordítható a megváltozott könyvtárral. Az **API-kompatibilitás** ennél szűkebb: a deklarációk, a paraméterek és a dokumentált szemantika változatlanok. Az **ABI-kompatibilitás** bináris szintű: a hívási konvenció, a struct-layout, a szimbólumnevek, az allokátor és a kivételkezelés együtt kompatibilis. A **bináris kompatibilitás** a legszűkebb: a változatlan kliensbináris betölthető és helyesen fut. Ehhez jön a **viselkedési kompatibilitás**, amely a megfigyelhető eredmények és hibák azonosságát jelenti; ez nem következik automatikusan az ABI-ból.

A labor szerződése a következő. A `diagcore.h` C fejléc változatlan; a C kliens vagy változatlan bináris, vagy változatlan forrásból újrafordított; a könyvtár SONAME-ja és exportlistája ellenőrzött; a hibaszerződés (`errbuf`, visszatérési kód, `last_error` élettartama) dokumentált. A labor

nem bizonyítja a cross-compiler stabilitást, és nem teszteli STL-típusok vagy kivételek átadását a határon: ezek a témakör határai, a C API ezeket szándékosan nem használja.

36.5.2 A referencia-implementáció: változatlan C fejléc

A fejléc csak C-típusokat és opaque handle-t tartalmaz. A `struct_size` és az `abi_version` a verziózás alapja; a `diag_engine` definíciója a fejlécben nem látható. A `DIAGCORE_NOEXCEPT` makró C++-ban `noexcept`-re, C-ben üresre helyettesítődik: C++17 óta a kivételspecifikáció a függvénytípus része, ezért a deklarációnak és a definíciónak egyeznie kell. Ha a fejléc `noexcept` nélkül deklarálna, a `.cpp` pedig `noexcept`-tel definiálna, a definíció fordítási hiba lenne (`declaration ... has a different exception specifier`).

```
/* diagcore.h - stabil C API, C17 és C++20 fordításra */
#ifndef DIAGCORE_H
#define DIAGCORE_H

#include <stddef.h>
#include <stdint.h>

#ifdef __cplusplus
#  define DIAGCORE_NOEXCEPT noexcept   /* C++-ban a típus része: deklaráción */
extern "C" {                           /* és definíción is azonos legyen     */
#else
#  define DIAGCORE_NOEXCEPT
#endif

#if defined(_WIN32)
#  define DIAGCORE_API __declspec(dllexport)
#elif defined(__GNUC__)
#  define DIAGCORE_API __attribute__((visibility("default")))
#else
#  define DIAGCORE_API
#endif

#define DIAGCORE_ABI_VERSION 1u

typedef struct diag_engine diag_engine; /* opaque handle */

typedef struct diag_config {
    uint32_t struct_size;  /* = sizeof(diag_config) */
    uint32_t abi_version;  /* = DIAGCORE_ABI_VERSION */
    size_t   max_pending;
    uint32_t flags;
} diag_config;

typedef struct diag_stats {
    uint64_t accepted;
    uint64_t rejected;
    uint64_t callbacks;
} diag_stats;
```

```
typedef void (*diag_sink)(void *ctx, const char *msg, size_t len);

/* Hibakódok (a Linux errno-értékeivel egyezően). */
#define DIAGCORE_OK      0
#define DIAGCORE_ENOMEM 12
#define DIAGCORE_EBUSY  16
#define DIAGCORE_EINVAL 22

DIAGCORE_API diag_engine *diag_engine_create(const diag_config *cfg,
                                             char *errbuf, size_t errlen)
 ↪  DIAGCORE_NOEXCEPT;
DIAGCORE_API void diag_engine_destroy(diag_engine *e) DIAGCORE_NOEXCEPT;
DIAGCORE_API int  diag_engine_feed(diag_engine *e, const char *data, size_t len)
 ↪  DIAGCORE_NOEXCEPT;
DIAGCORE_API int  diag_engine_set_sink(diag_engine *e, diag_sink sink, void
 ↪  *ctx) DIAGCORE_NOEXCEPT;
DIAGCORE_API int  diag_engine_get_stats(const diag_engine *e, diag_stats *out)
 ↪  DIAGCORE_NOEXCEPT;
DIAGCORE_API const char *diag_engine_last_error(const diag_engine *e)
 ↪  DIAGCORE_NOEXCEPT;

#ifdef __cplusplus
}
#endif
#endif
```

A C++ implementáció a `diag_engine` structot a `.cpp` fájlban definiálja, és minden `extern "C"` függvényt `DIAGCORE_NOEXCEPT`-tel lát el. A kivétel nem léphet át a C határon, ezért minden belépési pontnak három hibautat kell lezárnia:

- **Konstrukció.** A `new (std::nothrow)` csak a tároló foglalásának hibáját alakítja `nullptr`-ré; a konstruktor kivételét nem fogja el. A `std::deque` alapértelmezett konstruktora a libstdc++-ban már allokál, így ez a kivétel a `noexcept` függvényből kiszökve `std::terminate`-et okozna. Ezért a teljes konstrukció egy `try/catch` határ mögött fut, külön ágon kezelve a `std::bad_alloc`-ot és minden más kivételt.
- **Hibaút.** A hibakezelés maga sem allokálhat: egy `std::string` értékadása a `catch` ágban újabb `std::bad_alloc`-ot dobhatna, kezeletlenül. A hibaszöveg ezért statikus literál, és fix méretű pufferbe kerül `memcpy`-vel; a `create` hibája a hívó `errbuf`-jába, a többi hiba a handle `last_error` tömbjébe.
- **Reentrancia.** A sink callback alatt a `feed`, a `set_sink` és a `destroy` tiltott. Az első kettő `DIAGCORE_EBUSY`-t ad, a `destroy` pedig hatástalan, és `last_error`-t állít. A védelem mélységszámláló, amelyet minden érintett belépési pont ellenőriz. Egyetlen bool nem elég: ha a reentráns `feed` futni engedné a callbacket, a belső hívás végén a flag hamisra állna, és a külső callbackből hívott `destroy` a még futó külső `feed` alól törölné az objektumot.

```
// diagcore.cpp - C++20 implementáció a változatlan C fejléc mögött
#include "diagcore.h"

#include <cstddef>
#include <cstring>
#include <deque>
#include <new>
#include <string>
```

```
namespace {

// Hibaszöveg másolása allokáció és kivétel nélkül; mindig lezárja a puffert.
void copy_message(char* dst, std::size_t cap, const char* msg) noexcept {
    if (dst == nullptr || cap == 0) return;
    std::size_t n = std::strlen(msg);
    if (n >= cap) n = cap - 1;
    std::memcpy(dst, msg, n);
    dst[n] = '\0';
}

} // namespace

struct diag_engine {
    explicit diag_engine(const diag_config& c) : cfg(c) {}
    diag_config cfg{};
    std::deque<std::string> pending;   // a libstdc++-ban már a konstruktor
     ↪  allokál
    diag_sink sink = nullptr;
    void* sink_ctx = nullptr;
    diag_stats stats{};
    char last_error[96] = "";          // fix méret: a hibaút nem allokál
    int callback_depth = 0;            // >0: sink fut; reentrancia tiltott
#if defined(DIAGCORE_BAD_KEEP_POINTER)
    const char* kept = nullptr;        // HIBÁS negatív teszt: idegen pointer
     ↪  megőrzése
#endif

    void set_error(const char* msg) noexcept {
        copy_message(last_error, sizeof last_error, msg);
    }
};

extern "C" DIAGCORE_API diag_engine*
diag_engine_create(const diag_config* cfg, char* errbuf, std::size_t errlen)
 ↪  DIAGCORE_NOEXCEPT {
    auto fail = [&](const char* m) noexcept -> diag_engine* {
        copy_message(errbuf, errlen, m);
        return nullptr;
    };
    if (!cfg) return fail("null config");
    constexpr std::size_t kMin = offsetof(diag_config, flags) +
     ↪  sizeof(uint32_t);
    if (cfg->struct_size < kMin) return fail("config too small");
    if (cfg->abi_version != DIAGCORE_ABI_VERSION) return fail("unsupported
     ↪  abi_version");

    diag_config c{};                   // csak az ismert mezőket másoljuk
    std::memcpy(&c, cfg, sizeof c < cfg->struct_size ? sizeof c :
     ↪  cfg->struct_size);
    c.struct_size = sizeof c;
```

```
    if (c.max_pending == 0) c.max_pending = 64;

    try {                                   // a TELJES konstrukció a határ mögött
        return new diag_engine(c);
    } catch (const std::bad_alloc&) {
        return fail("allocation failed");
    } catch (...) {
        return fail("internal error");
    }
}

extern "C" DIAGCORE_API void diag_engine_destroy(diag_engine* e)
 ↪  DIAGCORE_NOEXCEPT {
    if (!e) return;
    if (e->callback_depth > 0) {        // a sinkből hívva: tiltott, nincs
     ↪  hatása
        e->set_error("destroy during callback");
        return;
    }
    delete e;
}

extern "C" DIAGCORE_API int
diag_engine_feed(diag_engine* e, const char* data, std::size_t len)
 ↪  DIAGCORE_NOEXCEPT {
    if (!e) return DIAGCORE_EINVAL;
    if (e->callback_depth > 0) {        // reentráns feed a sinkből: tiltott
        e->set_error("feed during callback");
        return DIAGCORE_EBUSY;
    }
    if (!data && len != 0) return DIAGCORE_EINVAL;
#if defined(DIAGCORE_BAD_KEEP_POINTER)
    if (e->kept) { volatile char c = e->kept[0]; (void)c; }   // késleltetett
     ↪  olvasás
    e->kept = data;
#endif
    try {
        if (len != 0) e->pending.emplace_back(data, len); // másolat, nem idegen
         ↪  pointer
        while (e->pending.size() > e->cfg.max_pending) e->pending.pop_front();
    } catch (const std::bad_alloc&) {
        e->set_error("allocation failed");
        e->stats.rejected += 1;
        return DIAGCORE_ENOMEM;
    }
    e->stats.accepted += 1;
    if (e->sink) {
        ++e->callback_depth;
        try {                            // egy C++ kliens dobó függvényt is
         ↪  adhat
            e->sink(e->sink_ctx, data, len);
        } catch (...) {
```

```
            e->set_error("sink threw");
            e->stats.rejected += 1;
        }
        --e->callback_depth;
        e->stats.callbacks += 1;
    }
    return DIAGCORE_OK;
}

extern "C" DIAGCORE_API int
diag_engine_set_sink(diag_engine* e, diag_sink sink, void* ctx)
 ↪  DIAGCORE_NOEXCEPT {
    if (!e) return DIAGCORE_EINVAL;
    if (e->callback_depth > 0) {
        e->set_error("set_sink during callback");
        return DIAGCORE_EBUSY;
    }
    e->sink = sink;
    e->sink_ctx = ctx;
    return DIAGCORE_OK;
}

extern "C" DIAGCORE_API int
diag_engine_get_stats(const diag_engine* e, diag_stats* out) DIAGCORE_NOEXCEPT {
    if (!e || !out) return DIAGCORE_EINVAL;
    *out = e->stats;
    return DIAGCORE_OK;
}

extern "C" DIAGCORE_API const char*
diag_engine_last_error(const diag_engine* e) DIAGCORE_NOEXCEPT {
    return e ? e->last_error : "null handle";
}

#if defined(DIAGCORE_TEST_HOOKS)
// Csak tesztbuildben, nem része az ABI-nak: a legutóbbi pending másolat
// kiolvasása, hogy a teszt a másolat ÉRTÉKÉT is ellenőrizhesse.
extern "C" DIAGCORE_API std::size_t
diagcore_test_last_pending(const diag_engine* e, char* out, std::size_t cap)
 ↪  noexcept {
    if (!e || e->pending.empty() || cap == 0) return 0;
    const std::string& s = e->pending.back();
    std::size_t n = s.size() < cap ? s.size() : cap;
    std::memcpy(out, s.data(), n);
    return n;
}
#endif
```

A `diag_engine_last_error` a handle belső, fix méretű hibapufferére mutat; a pointer a handle élettartamáig érvényes, a tartalma a következő hibáig. A `create` hibáját nem ez, hanem az `errbuf` tartalmazza. A `feed` a bemeneti buffert csak a hívás alatt használja, a `pending`-be másolat kerül; ez a foreign buffer szerződés lényege.

36.5.3 Build és exportellenőrzés

A könyvtárat `-fvisibility=hidden`-nal és verziós szkripttel kell építeni, hogy csak a hat C szimbólum exportálódjon, a belső C++ szimbólumok ne.

```
g++ -std=c++20 -O2 -fPIC -fvisibility=hidden -Wall -Wextra -Wpedantic \
    -Wl,--no-undefined -Wl,-soname,libdiagcore.so.1 \
    -Wl,--version-script=diagcore.map \
    -shared -o libdiagcore.so.1.0.0 diagcore.cpp
ln -sf libdiagcore.so.1.0.0 libdiagcore.so.1
ln -sf libdiagcore.so.1 libdiagcore.so

gcc -std=c17 -O2 -Wall -Wextra -Wpedantic client.c -L. -ldiagcore \
    -Wl,-rpath,'$ORIGIN' -o client
./client
```

A verziós szkript (`diagcore.map`):

```
DIAGCORE_1.0 {
  global:
    diag_engine_create;
    diag_engine_destroy;
    diag_engine_feed;
    diag_engine_set_sink;
    diag_engine_get_stats;
    diag_engine_last_error;
  local: *;
};
```

A `diagcore.map` a `DIAGCORE_1.0` verzió alatt sorolja fel a hat függvényt, minden mást `local: *;` rejt. Az exportellenőrzés a dinamikus szimbólumtáblát hasonlítja össze a várt listával (`expected-exports.txt`, soronként egy függvénynév, rendezve). Az `nm` a verziózott szimbólumot `diag_engine_create@@DIAGCORE_1.0` alakban írja ki, ezért a szkript a `@` utáni részt levágja:

```
#!/usr/bin/env bash
# check-exports.sh - csak a hat C szimbólum exportálódhat
set -euo pipefail
lib=${1:?usage: check-exports.sh libdiagcore.so.1.0.0}
nm -D --defined-only --format=posix "$lib" \
  | awk '$2 ~ /^[Tt]$/ {print $1}' \
  | sed 's/@.*//' \
  | sort -u > actual.txt
diff -u expected-exports.txt actual.txt
```

A `--format=posix` kimenetében a típus a második mező; a `T`/`t` szűrés kizárja a verziócsomópont szimbólumát (`DIAGCORE_1.0`, `A` típus). A `nm` és a `readelf` kimenete binutils-verziófüggő, ezért a laborprofilban rögzíteni kell a használt binutils-kiadást.

36.5.4 Handle-élettartam, allokációs hiba és idegen memória

A `diag_engine_create` sikeres hívása egy tulajdonosi handle-t ad. A `diag_engine_destroy` felelős a felszabadításért; `destroy(NULL)` no-op, és a `destroy` után a handle dangling, ezért a kliensnek `NULL`-ra kell állítania. A `feed(NULL, ...)` és a `get_stats(NULL, ...)` `EINVAL`-t ad.

A `create` allokációs hibája `NULL` handle-t és `errbuf`-ba írt üzenetet eredményez; a `feed` allokációs hibája `ENOMEM` és `last_error` beállítás.

Az allokációs hiba determinisztikus teszteléséhez a tesztprogram saját `malloc`-ot definiál, amely a glibc `__libc_malloc` függvényét hívja, és egy számláló elérése után `NULL`-t ad. Mivel a futtatható állomány szimbóluma a dinamikus linkelésnél elsőbbséget kap, a könyvtár `operator new`-ja is ezt használja. Ez glibc-specifikus laborsegéd, nem része a könyvtárnak; a `dlsym`-es `LD_PRELOAD` megoldással szemben nincs saját, rekurzív allokációja. A teszt sorra minden allokációs pontot meghiúsít, és ellenőrzi, hogy a `create` `NULL`-t és `allocation failed` üzenetet, a `feed` pedig `DIAGCORE_ENOMEM`-et és ugyanezt a `last_error`-t adja.

```
/* alloc_fail.c - determinisztikus allokációs hiba a diagcore API-n.
   A program saját malloc-ja a glibc __libc_malloc-ját hívja, és a
   fail_after-edik hívástól NULL-t ad; az operator new is ezt használja. */
#include "diagcore.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

extern void *__libc_malloc(size_t);
static long fail_after = -1;            /* -1: nincs hibainjektálás */

void *malloc(size_t n) {
    if (fail_after == 0) return NULL;
    if (fail_after > 0) --fail_after;
    return __libc_malloc(n);
}

int main(void) {
    diag_config c;
    memset(&c, 0, sizeof c);
    c.struct_size = sizeof c;
    c.abi_version = DIAGCORE_ABI_VERSION;
    char err[64];
    const char msg[] = "a message long enough to need heap storage";
    int create_failures = 0, feed_failures = 0;

    for (long k = 0; k < 16; ++k) {     /* minden allokációs pont sorra */
        err[0] = '\0';
        fail_after = k;
        diag_engine *e = diag_engine_create(&c, err, sizeof err);
        int rc = e ? diag_engine_feed(e, msg, sizeof msg - 1) : -1;
        fail_after = -1;
        if (!e) {
            if (strcmp(err, "allocation failed") != 0) return 1;
            ++create_failures;
            continue;
        }
        if (rc == DIAGCORE_ENOMEM) {
            if (strcmp(diag_engine_last_error(e), "allocation failed") != 0)
            ↪  return 1;
            ++feed_failures;
```

```
        } else if (rc != DIAGCORE_OK) {
            return 1;
        }
        diag_engine_destroy(e);
    }
    printf("create_failures=%d feed_failures=%d\n", create_failures,
 ↪  feed_failures);
    return create_failures > 0 && feed_failures > 0 ? 0 : 1;
}
```

A C kliens a többi szerződést ellenőrzi: `NULL` handle, hibás és nagyobb `struct_size`, hibás `abi_version`, idegen puffer felszabadítása a `feed` után, valamint a callbackből hívott reentráns `feed` és `destroy`:

```
/* client.c - C17 tesztkliens a diagcore C API-hoz */
#include "diagcore.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define CHECK(condition) do {                                          \
    if (!(condition)) {                                                \
        fprintf(stderr, "CHECK failed at line %d\n", __LINE__);        \
        exit(EXIT_FAILURE);                                            \
    }                                                                  \
} while (0)

#if defined(DIAGCORE_TEST_HOOKS)
size_t diagcore_test_last_pending(const diag_engine *e, char *out, size_t cap);
#endif

static diag_config make_config(void) {
    diag_config c;
    memset(&c, 0, sizeof c);
    c.struct_size = sizeof c;
    c.abi_version = DIAGCORE_ABI_VERSION;
    c.max_pending = 4;
    return c;
}

struct probe {                  /* a sink kontextusa: a hívó tulajdona */
    diag_engine *e;
    int calls;
    int nested_feed_status;
    int try_destroy;
};

static void sink(void *ctx, const char *msg, size_t len) {
    struct probe *p = ctx;
    (void)msg; (void)len;
    p->calls += 1;
```

```
    p->nested_feed_status = diag_engine_feed(p->e, "x", 1);   /* reentráns */
    if (p->try_destroy) diag_engine_destroy(p->e);            /* tiltott */
}

int main(void) {
    char err[64];
    diag_stats st;

    /* NULL és hibás konfiguráció */
    diag_engine_destroy(NULL);
    CHECK(diag_engine_feed(NULL, "a", 1) == DIAGCORE_EINVAL);
    CHECK(diag_engine_get_stats(NULL, &st) == DIAGCORE_EINVAL);
    CHECK(diag_engine_create(NULL, err, sizeof err) == NULL);
    CHECK(strcmp(err, "null config") == 0);
    diag_config bad = make_config();
    bad.struct_size = 4;
    CHECK(diag_engine_create(&bad, err, sizeof err) == NULL);
    CHECK(strcmp(err, "config too small") == 0);
    bad = make_config();
    bad.abi_version = 2;
    CHECK(diag_engine_create(&bad, err, sizeof err) == NULL);
    CHECK(strcmp(err, "unsupported abi_version") == 0);

    /* Nagyobb (újabb) struct: az ismert mezőket olvassa */
    struct { diag_config base; uint64_t future; } bigger;
    memset(&bigger, 0, sizeof bigger);
    bigger.base = make_config();
    bigger.base.struct_size = sizeof bigger;
    diag_engine *e = diag_engine_create(&bigger.base, err, sizeof err);
    CHECK(e != NULL);

    /* Idegen puffer: a feed után a kliens felszabadítja */
    char *buf = malloc(32);
    CHECK(buf != NULL);
    strcpy(buf, "a longer message than sso size");
    CHECK(diag_engine_feed(e, buf, strlen(buf)) == DIAGCORE_OK);
    free(buf);
#if defined(DIAGCORE_TEST_HOOKS)
    /* A könyvtár saját másolatból dolgozik: az érték a free után is megvan. */
    char copy[64];
    size_t n = diagcore_test_last_pending(e, copy, sizeof copy);
    CHECK(n == 30 && memcmp(copy, "a longer message than sso size", 30) == 0);
#endif
    /* A következő feed egy későbbi API-út: egy pointert megőrző könyvtár
       itt olvasná a felszabadított puffert. */

    /* Reentráns feed és destroy a sinkből */
    struct probe p = { e, 0, -1, 1 };
    CHECK(diag_engine_set_sink(e, sink, &p) == DIAGCORE_OK);
    CHECK(diag_engine_feed(e, "hello", 5) == DIAGCORE_OK);
    CHECK(p.calls == 1);
    CHECK(p.nested_feed_status == DIAGCORE_EBUSY);
```

```
    CHECK(strcmp(diag_engine_last_error(e), "destroy during callback") == 0);

    /* A handle a callback után is érvényes */
    CHECK(diag_engine_get_stats(e, &st) == DIAGCORE_OK);
    CHECK(st.accepted == 2 && st.callbacks == 1 && st.rejected == 0);
    CHECK(diag_engine_set_sink(e, NULL, NULL) == DIAGCORE_OK);
    diag_engine_destroy(e);              /* a callback után: tényleges törlés
 ↪  */
    e = NULL;
    puts("client: ok");
    return 0;
}

gcc -std=c17 -O2 -Wall -Wextra alloc_fail.c -L. -ldiagcore \
    -Wl,-rpath,'$ORIGIN' -o alloc_fail && ./alloc_fail
# ASan/UBSan: a könyvtár és a kliens is instrumentált
g++ -std=c++20 -O1 -g -fsanitize=address,undefined -fPIC -shared \
    -o libdiagcore_san.so diagcore.cpp
gcc -std=c17 -O1 -g -fsanitize=address,undefined client.c \
    -L. -ldiagcore_san -Wl,-rpath,'$ORIGIN' -o client_san && ./client_san
bash check-exports.sh libdiagcore.so.1.0.0
```

Buildjegyzőkönyv (`lab-gcc` profil). A könyvtár és a kliens figyelmeztetés nélkül fordult; a `diagcore.h` C17-ben önállóan is lefordul. A `client` és az ASan/UBSan-os `client_san` is `client: ok` kimenettel futott. Az `alloc_fail` kimenete `create_failures=3 feed_failures=1`, a kód 0. Az exportellenőrzés pontosan a hat függvényt találta, `_Z` kezdetű szimbólum nélkül, és a fejlécben nincs `std::`. Az eredeti minták hibáit is reprodukáltuk: - a `noexcept` nélküli fejléc a `noexcept` definícióval `has a different exception specifier` fordítási hibát adott; - a `new (std::nothrow)` konstrukció allokációs hibánál `terminate called after throwing an instance of 'std::bad_alloc'` üzenettel állt le; - a bool-os callback-védelem a „sink → reentráns `feed` → `destroy`” sorrendben ASan alatt `heap-use-after-free` hibát adott.

A foreign buffer tesztje: a C kliens `malloc`-ol, `feed`-el, majd a `feed` visszatérése után `free`-el. Ez önmagában még nem bizonyítja a tulajdonviszonyt: egy megőrzött pointer csak akkor okoz ASan-jelzést, ha a könyvtár a `free` után ténylegesen olvas belőle. Ezért két tesztváltozat egészíti ki. (1) A `DIAGCORE_TEST_HOOKS` build egy csak tesztben létező, az ABI-ba nem kerülő `diagcore_test_last_pending` függvénnyel a `free` *után* kiolvassa a könyvtár másolatát, és ellenőrzi, hogy az a teljes eredeti üzenet. (2) A `DIAGCORE_BAD_KEEP_POINTER` build szándékosan hibás: megőrzi az idegen pointert, és a következő `feed`-nél — egy későbbi API-úton — olvas belőle; a kliens ugyanazzal a forrással fut ellene. Kódszintű indoklás: a `feed` a `data`-ból `std::string`-et épít a `pending`-be, a sinknek csak a hívás idejére adja át, és a `diag_engine` egyetlen tagja sem tárol `const char*`-ot a hívón túl. **Buildjegyzőkönyv** (`lab-gcc` profil): a tesztkampós ASan/UBSan build `client: ok`-kal futott (a másolat egyezett); a pointert megőrző változat a `hello` feed-nél `ERROR: AddressSanitizer: heap-use-after-free` hibával állt le. Mindkét lépés a ch36-ffi laborcsomag `check.sh`-jának része. A callback `ctx`-e a hívó tulajdona; a könyvtár nem szabadítja fel. A callback alatt hívott `destroy` a fenti implementációban nem töröl, hanem `last_error`-t állít és visszatér, így a callback után a handle tovább használható; ez dokumentált, szándékos viselkedés, nem UB. Ugyanígy a callback alatt hívott `feed` és `set_sink` `DIAGCORE_EBUSY`-t ad.

36.5.5 Verziózás és unsupported configuration

A `diag_config` első két mezője a `struct_size` és az `abi_version`. A könyvtár a `struct_size`-ból tudja, meddig olvashat: a minimum az általa ismert mezők végéig terjed (`offsetof(diag_config, flags) + sizeof(uint32_t)`). A régi, kisebb structtal fordított kliens addig kompatibilis, amíg eléri ezt a minimumot; a nagyobb struct esetén a könyvtár az ismert mezőket olvassa, az ismeretleneket figyelmen kívül hagyja. Az `abi_version` eltérése `NULL` handle-t és `errbuf`-üzenetet ad. A `diag_stats` kimeneti struct méretét a kliens adja, ezért azt a könyvtár headerjével kell fordítani; ezt a labor dokumentálja, és külön feladatban lehet `struct_size`-tal verziózni.

A szimbólumverziózás a `DIAGCORE_1.0` csomópontot használja. Egy későbbi, szemantikailag eltérő verzió új csomópontot kaphat (`DIAGCORE_1.1`), és a régi kliens a `dlvsym`-mal vagy a linker által a régi verziót kapja. A laborban a verziós szkript és a `dlvsym` teszt együtt ellenőrzi, hogy a régi kliens a régi szimbólumot látja.

36.5.6 PImpl és C wrapper: döntési táblázat

Megoldás	ABI-stabilitás	C++ típusok	Allokáció	Hibakezelés
C API + PImpl	erős, ha a szimbólumok és structok verziózottak	rejtett	egy handle-allokáció	hibakód + `errbuf`
közvetlen C++ API	gyenge, toolchainhez kötött	teljes	a kliens allokátora	kivétel
C++ wrapper a C API felett	erős, a C API örökli	a wrapperben	wrapper allokálhat	kivétel a wrapperben

A PImpl előnye, hogy a C++ típusok nem szivárognak ki a fejlécbe, és a C API stabil marad; hátránya az extra indirection és a handle-allokáció. A közvetlen C++ API kényelmesebb C++ klienseknek, de a bináris kompatibilitás a fordítóhoz és a standard libraryhez kötött. A C++ wrapper a C API felett kompromisszum: a stabil mag C, a kényelmes felület C++.

36.5.7 A labor tesztmátrixa

Teszt	Mit ellenőriz	Eszköz	Elvárt eredmény
C kliens linkelése	a C++ `.so` kielégíti a C szimbólumokat	`gcc` + `ld`	sikeres link és futás
exportlista	csak a hat C szimbólum	`nm` + `diff`	egyezés
handle	`create`/`destroy`, `NULL`	egységteszt	`NULL` no-op, hibás config `NULL`
allokációs hiba	`create` `NULL` + `errbuf`, `feed` `ENOMEM`	saját `malloc` (`alloc_fail.c`)	`NULL`, „allocation failed”; `ENOMEM`
buffer lifetime	`feed` után `free`	ASan	nincs use-after-free
callback lifetime	reentráns `feed` és `destroy` a callback alatt	`client.c` + ASan	`EBUSY`, `last_error`, nincs UAF

Teszt	Mit ellenőriz	Eszköz	Elvárt eredmény
verzió	kisebb/nagyobb `struct_size`	két kliens	megfelelő hiba/elfogadás
PImpl	a fejlécben nincs C++ típus	`grep`	nincs `std::` a headerben

36.5.8 Gyakori hibák és javításuk

A leggyakoribb hiba az `extern "C"` elhagyása: a C++ névtorzítás miatt a C kliens nem találja a szimbólumot. A második a kivételhatár hiánya: a C++ kivétel átlépne a C határon. A `noexcept` ezt `std::terminate`-tel zárja le, de a kivételt magunknak kell elkapnunk és hibakóddá alakítanunk, a konstruktorét és a hibakezelő ágét is. A harmadik, hogy `std::string` vagy `std::vector` kerül a fejlécbe: ez nem stabil ABI. A `struct_size` elhagyása verziózhatatlanná teszi a structot; a callback `ctx` könyvtárbeli felszabadítása double free-hez vezet. Az `nm`-teszt akkor ad hamis pozitívot, ha a verziós szimbólumokat is a várt függvények közé sorolja. A `-fvisibility=hidden` hiánya belső C++ szimbólumokat exportál. Végül a `destroy` utáni handle-használat dangling pointer; a laborban a `destroy` után `NULL`-ra állítás a szabály.

36.5.9 Feladatok

36.5.9.1 Fogalmi különbségtétel

Definiáld az API-, ABI- és viselkedési kompatibilitást, és adj egy példát, amikor a C API változatlan, de a bináris kompatibilitás megsérül.

36.5.9.2 Két implementáció, egy kliens

Készíts egy C és egy C++ implementációt ugyanahhoz a `diagcore.h`-hoz, és igazold, hogy a C kliens mindkettővel linkel. Mellékeld az exportellenőrzést.

36.5.9.3 Buffer lifetime ASan-nel

Reprodukáld, hogy a könyvtár a `feed` után megőrzi a `data` pointert, és a kliens `free`-je után olvas. Javítsd a szerződést.

36.5.9.4 `struct_size` verziózás

Implementáld a `struct_size` ellenőrzést, és teszteld egy régi kisebb és egy új nagyobb `diag_config`-gal.

36.5.9.5 Callback alatti destroy

Írj tesztet, amelyben a sink callback `diag_engine_destroy`-t hív. Igazold, hogy nincs use-after-free, és a handle tovább használható.

36.5.10 Megoldások

36.5.10.1 Fogalmi különbségtétel — megoldás

Az API-kompatibilitás a deklarációk és a dokumentált szemantika azonossága; az ABI a bináris hívási konvenció, a layout, a szimbólumnevek és az allokátor kompatibilitása; a viselkedési kompatibilitás a megfigyelhető eredmények azonossága. Példa: a `diag_config` mezőinek átrendezése vagy egy `size_t` méretének megváltozása ILP32 és LP64 között API-ban változatlan, de ABI-ban nem. Ellenőrzési feltétel: a válasz nevezze meg a layout, a hívási konvenció és a szimbólumnév szerepét, és ne állítsa, hogy az API-stabilitásból ABI-stabilitás következik.

36.5.10.2 Két implementáció, egy kliens — megoldás

A C implementációban a `diag_engine` szintén opaque, a `.so` neve `libdiagcore_c.so.1`, és ugyanazt a hat szimbólumot exportálja. A kliens `-ldiagcore` helyett `-ldiagcore_c`-vel linkel. A két `.so` exportlistáját a `check-exports.sh` hasonlítja össze. Ellenőrzési feltétel: a két exportlista egyezik, a kliens mindkét könyvtárral fut, és a `nm` kimenetében nincs `_Z` kezdetű szimbólum.

36.5.10.3 Buffer lifetime ASan-nel — megoldás

A hibás változat (a fenti `DIAGCORE_BAD_KEEP_POINTER` ág) a `feed`-ben eltárolja a `data` pointert, és egy későbbi API-hívásban — itt a következő `feed`-ben — olvas belőle. A kliens `free`-je után az ASan ennél az olvasásnál `heap-use-after-free`-t jelez; ha a könyvtár sosem olvasna a megőrzött pointerből, a hiba rejtve maradna, ezért a tesztnek a késleltetett utat is el kell érnie. A javítás: a `pending`-be `std::string` másolat kerül, a `data`-t csak a hívás alatt használjuk, a callback is csak a hívás alatt kapja. Ellenőrzési feltétel: az ASan tiszta, a `pending` tartalma a másolat, és a `feed` visszatérése után a kliens szabadon felszabadíthatja a buffert.

36.5.10.4 `struct_size` verziózás — megoldás

A minimum méret `offsetof(diag_config, flags) + sizeof(uint32_t)`. A régi kliens a régi headerrel fordít, a `struct_size` a régi méret; ha eléri a minimumot, a könyvtár elfogadja. Az új kliens nagyobb structot ad; a könyvtár az ismert mezőket olvassa, az ismeretleneket nem. A túl kicsi struct `NULL` handle-t és `errbuf`-üzenetet ad. Ellenőrzési feltétel: a régi kliens fut, a túl kicsi struct hibát ad, és az `abi_version` eltérése külön hibaüzenetet kap.

36.5.10.5 Callback alatti destroy — megoldás

A `destroy` a callback alatt `in_callback` miatt nem töröl, hanem `last_error`-t állít és visszatér. A callback után a `feed` visszatér, a handle érvényes. A teszt a callback alatt `destroy`-t hív, majd a callback után `get_stats`-ot kér, és ellenőrzi a `last_error` tartalmát. Ellenőrzési feltétel: az ASan tiszta, a `stats` konzisztens, és a handle a callback után explicit `destroy`-val szabadítható fel.

36.5.11 Összefoglaló

A labor lényege, hogy a C fejléc és a C kliens változatlan maradjon, miközben a mögöttes implementáció C++-ra cserélődik. Ezt nem elég deklarálni: a szimbólumkészletet, a handle-élettartamot, az allokációs hibát, a foreign buffer és a callback élettartamát, valamint a verziózást külön tesztekkel kell ellenőrizni. A `struct_size` és az `abi_version`, a `noexcept` határ, a PImpl és a verziós szkript együtt adják a szerződést. A labor eredménye a rögzített toolchain-profilhoz kötött; a cross-compiler stabilitás és az STL/kivétel átadása a határon nem része a bizonyított állításnak.

37 Teljesítmény, allocatorok és memóriaelrendezés C++-ban

37.1 Allocator model és allocator_traits

A könyv eddigi részei a helyességet állították előtérbe: a `std::vector` érvényességi szerződéseit (std::array, std::vector és folytonos tárolás (28.1)), a tároló újrafelhasználásának objektumszabályait (Tároló újrafelhasználása és C++ objektumszabályok (20.6)) és a fordítási idejű típusvizsgálat eszközeit (Type traits, tag dispatch és if constexpr (27.1)) már ismered. Most ugyanazt a tárolót költségoldalról nézzük meg. A kiindulópont az, hogy a standard tárolók nem „memóriát foglalnak”, hanem két külön fázisra bontják a munkát: egy allokátoron keresztül **tárolóterületet** (storage) szereznek, majd azon belül **objektumokat** hoznak létre és semmisítenek meg. Ez a szétválasztás teszi lehetővé a visszagörgethető hibakezelést, a költség mérését és azt, hogy az erőforrás mögötti memóriarezsimot kicseréljük anélkül, hogy a tároló típusát vagy az algoritmusokat újraírnánk. Az alábbiakban ezt a szerződést bontjuk ki, majd megmutatjuk, hogy a move és a swap költsége nem a tárolótól, hanem az allokátor egyenlőségétől és propagation tulajdonságaitól függ. A fejezet határozottan nem benchmarkfejezet: itt cost modellt állítunk fel, amit a későbbi, mérési fejezet validál.

37.1.1 A szerződés: tárolás és objektum-élettartam szétválasztása

Allokátor (allocator) alatt olyan típust értünk, amely megfelel a standard allokátorkövetelményeinek: képes `n` darab `T` típusú objektum számára megfelelően igazított, inicializálatlan tárolóterületet szerezni, azt visszaadni, és egyenlőség szempontjából összehasonlítható. Az allokátor **nem** hoz létre objektumot. Ez a C++20 alapban már nyelvi szinten is látszik: a `std::allocator<T>::construct` és `destroy` tagok a C++17-es elavulás után kikerültek a szabványból.

A teljes életciklus négy, egymástól elkülönülő lépésből áll, ahol `p = traits::allocate(a, n)`:

1. **Tárolószerzés.** A `p` egy `n * sizeof(T)` méretű, `T`-re igazított, inicializálatlan területre mutat. Ezen a ponton *nincs* `T` objektum: ha bármit is „olvasnál” belőle, az nem definiált viselkedés.
2. **Konstrukció.** `traits::construct(a, p + i, args...)` elindítja az `i`-edik objektum élettartamát. Az allokátor itt csak tanácsadó szerepet játszhat: az `allocator_traits` a helyettesítő `construct`-ot akkor hívja, ha az allokátornak van ilyen tagja, különben placement new-t használ.
3. **Destrukció.** `traits::destroy(a, p + i)` lezárja az élettartamot; a tároló ettől kezdve újra „csak” nyers terület.
4. **Felszabadítás.** `traits::deallocate(a, p, n)` visszaadja a területet. Ennek nem szabad dobnia.

Ebből az következik, hogy az `allocate` és a `construct` közötti rés a tároló **visszagörgetési pontja**. Ha a konstrukció közben kivétel jön, a tároló a már létrehozott elemeket destruálja, majd a területet felszabadítja – ehhez viszont tudnia kell, meddig jutott, és azt, hogy a felszabadítás az

ugyanazzal az allokátorral történik. Innen ered a `deallocate(p, n)` szerződése: ugyanaz az `n`, és egy olyan allokátorpéldány, amely egyenlő az allokálóval. A részletes objektum-újrafelhasználási szabályokat (placement new, `std::launder`, élettartamhatárok) a Tároló újrafelhasználása és C++ objektumszabályok (20.6) tárgyalja; itt csak annyit rögzítünk, hogy az allokátor soha nem lép ezek helyébe.

37.1.2 allocator_traits és a rebind mechanizmus

A `std::allocator_traits<A>` egy fordítási idejű adaptor: az `A` típusból kiolvassa a szükséges tagokat, a hiányzókat pedig alapértelmezéssel pótolja. Ez teszi lehetővé, hogy egy minimális allokátornak mindössze a `value_type`, az `allocate`, a `deallocate` és az egyenlőség legyen megvalósítva:

```
// minimal_allocator.cpp - C++20, csak szemléltetés
#include <cstddef>
#include <new>

template <class T>
struct minimal_allocator {
    using value_type = T;

    T* allocate(std::size_t n) {
        return static_cast<T*>(::operator new(n * sizeof(T)));
    }
    void deallocate(T* p, std::size_t) noexcept { ::operator delete(p); }

    template <class U>
    bool operator==(const minimal_allocator<U>&) const noexcept { return true; }
    template <class U>
    bool operator!=(const minimal_allocator<U>&) const noexcept { return false;
     ↪  }
};
```

Az alapértelmezések, amelyeket az `allocator_traits` pótol: `pointer` (`A::pointer`, különben `value_type*`), `difference_type`, `size_type`, a három `propagate_on_*` trait (`false_type`), valamint az `is_always_equal` (`std::is_empty<A>`). Ez utóbbi nem kozmetika: ha fordítási időben tudható, hogy minden példány egyenlő, a tároló az allokátor-összehasonlítást és a feltételes ágakat ki tudja ejteni.

A **rebind** az a leképezés, amely egy `T`-re paraméterezett allokátort egy másik `U` típusra állít elő: `allocator_traits<A>::rebind_alloc<U>`. A szabványos megvalósítás az `A::rebind<U>::other` tagot használja, ha létezik, különben a `template<class> class A` formából képzi. Miért kell? Mert a tároló soha nem az `int`-et allokálja közvetlenül a csomópont-alapú esetekben: a `std::list<int, std::allocator<int>>` a belső csomóponttípusra bindol, a `std::map` pedig a `std::pair<const K, V>`-t tartalmazó csomópontra. A felhasználó által megadott allokátor így is „ugyanaz” marad, csak más elem-típusra.

37.1.3 Egyenlőség, propagation és a move assignment költsége

Az allokátorok egyenlőségének **szemantikai** definíciója van: `a1 == a2` pontosan akkor, ha az `a1`-gyel megszerzett tároló felszabadítható `a2`-vel. Ez nem pointer-összehasonlítás, hanem szerződés

a memóriarezsim azonosságáról. Állapot nélküli allokátornál mindig igaz; állapotossal (arena, pool, shared memory szegmens) a rezsim azonosítóját kell összehasonlítani.

A **propagation trait**-ek azt mondják meg, hogy egy tárolószintű művelet átvegye-e a másik tároló allokátorát:

- `propagate_on_container_copy_assignment` (POCCA) – copy assignment,
- `propagate_on_container_move_assignment` (POCMA) – move assignment,
- `propagate_on_container_swap` (POCS) – `swap`.

Mindhárom alapértéke `false_type`: az allokátor „ragad” a tárolóhoz, nem vándorol vele.

37.1.3.1 A move assignment esetei

Legyen `a1` a cél-, `a2` a forrás-tároló allokátora. A `v1 = std::move(v2)` három ágra bomlik, és a költség gyökeresen más:

1. **POCMA igaz.** A cél átveszi a forrás allokátorát (`a1 = std::move(a2)`), és a puffert is átveszi. Nincs új allokáció, nincs elem-mozgatás.
2. **POCMA hamis, de `a1 == a2`.** A puffer átvehető, mert a felszabadítást majd `a1` is elvégezheti. Az allokátor marad. Szintén nincs új allokáció.
3. **POCMA hamis és `a1 != a2`.** A puffer **nem** vehető át: a forrás tárolója `a2`-vel allokálódott, a cél majd `a1`-gyel szabadítaná fel, ami érvénytelen. Ebből csak annyi következik, hogy az elemeket **egyenként** kell átvinni (mozgató értékadással vagy mozgató konstrukcióval) a cél saját, `a1`-es tárolójába. Hogy ehhez kell-e új allokáció, az a cél meglévő kapacitásán múlik: ha elég, a meglévő puffer újrahasznosítható. A forrás ezután *érvényes, de nem specifikált* állapotú: lehet üres, de meg is tarthatja a moved-from elemeit.

A cost model tehát, ahol `n` a forrás elemszáma:

$$\text{költség} = \underbrace{O(n)}_{\text{régi elemek destruktora}} + \begin{cases} O(1) & \text{1–2. eset: pufferátvétel} \\ O(n) + \text{legfeljebb 1 allokáció} & \text{3. eset: elem-mozgatás} \end{cases}$$

A 3. esetben a forrás tároló továbbra is érvényes, de nem specifikált tartalmú; a tesztnek nem szabad `source.empty()`-t elvárnia.

```
// move_assign_unequal.cpp - C++20: eltérő, nem propagáló allokátor
#include <cstdio>
#include <memory_resource>
#include <vector>

struct counting_resource : std::pmr::memory_resource {
    std::size_t allocs = 0;
    void* do_allocate(std::size_t n, std::size_t a) override {
        ++allocs;
        return std::pmr::new_delete_resource()->allocate(n, a);
    }
    void do_deallocate(void* p, std::size_t n, std::size_t a) override {
        std::pmr::new_delete_resource()->deallocate(p, n, a);
    }
    bool do_is_equal(const memory_resource& o) const noexcept override { return
     ↪  this == &o; }
```

```
};

int main() {
    counting_resource ra, rb;
    std::pmr::vector<int> dst(&ra), src(&rb);
    dst.reserve(16);                        // elegendő célkapacitás
    src.assign({1, 2, 3, 4, 5});
    std::size_t before = ra.allocs;
    dst = std::move(src);                   // a1 != a2, POCMA hamis
    std::printf("dst.size=%zu uj allokacio a celban=%zu src.size=%zu\n",
                dst.size(), ra.allocs - before, src.size());
    std::printf("dst allokatora valtozatlan: %d\n",
                dst.get_allocator().resource() == &ra);
}
```

Buildjegyzőkönyv (`lab-gcc` profil). A libstdc++ kimenete `dst.size=5 uj allokacio a celban=0 src.size=0`, a Clang 22 / libc++-é `dst.size=5 uj allokacio a celban=0 src.size=5`; mindkettőnél `dst allokatora valtozatlan: 1`. A 16 elemű célkapacitás miatt egyik könyvtár sem allokált, a forrás állapota viszont könyvtáranként eltér — pontosan ezért nem specifikált. Fontos feltétel: ez az ág csak akkor értelmezhető, ha `T` mozgatható (MoveInsertable). A kivételgarancia szintje `T` move konstruktorának `noexcept` voltától és az implementációtól függ; ezt ne feltételezd, hanem a `T` szerződésének részeként kezeld. A pontos megfogalmazást a választott szabványverzió szövegében ellenőrizd – ez a rész verziófüggő részlet.

37.1.3.2 A swap szigorúbb feltétele

A `swap` itt nem költség-, hanem **érvényességi** kérdés. Két tároló `swap`-je akkor definiált, ha POCS igaz, **vagy** a két allokátor egyenlő. Ha mindkettő hamis, a viselkedés nem definiált. Ezért a generikus kódban soha ne swap-elj olyan tárolókat, amelyek allokátorát nem te irányítod: a hiba nem lassulás, hanem UB.

37.1.4 Az allokátorpéldány élettartama és tulajdonlása

A tároló a konstruktorában kapott allokátorról **másolatot** tárol (`get_allocator()` is másolatot ad). Ez a példány a tárolóval együtt jön létre, és a tároló destruktorában utolsóként szűnik meg – a szabvány garantálja, hogy az allokátor addig él, amíg a tároló használja. Copy construction esetén a `select_on_container_copy_construction` dönt arról, milyen allokátort kap az új tároló (alapértelmezésben ugyanazt).

A lényeg: az allokátorpéldány **fogantyú, nem az erőforrás**. Ha az allokátor egy `arena*`-ot vagy `memory_resource*`-ot tart, akkor az arena élettartama a felhasználó felelőssége. Ha az arena előbb szűnik meg, mint a tároló, a tároló destruktora egy már felszabadított rezsimen hív `deallocate`-ot: nem definiált viselkedés, tipikusan csendes memoria-korrupció. A tároló csak a *sorrendet* garantálja (előbb elemek, aztán terület, végül az allokátor másolata), az arena tulajdonjogát nem. Ugyanez a szerződés öröklődik a `std::pmr` világba (pmr, memory_resource és region lifetime (37.2)): a `polymorphic_allocator` egy `memory_resource*`-ot csomagol, és a resource élettartama végig a hívóé.

37.1.5 Fancy pointer és scoped allocator: fogalmi térkép

A `allocator_traits<A>::pointer` nem kötelezően nyers pointer: lehet osztálytípus, amely kielégíti a NullablePointer követelményt, folytonos tárolónál pedig a random access iterator követelményt. Ezek a **fancy pointer**-ek. Klasszikus indokuk a shared memory: egy `offset_ptr` eltolást tárol cím helyett, így a mutató érvényes marad, ha a szegmens más címre mappelődik. Korlát: a folytonos tárolók `data()`-ja `T*`-ot ad vissza, ezért a fancy pointer gyakorlatban főleg csomópont-alapú tárolóknál és saját adaptereknél jelenik meg – ez implementációfüggő részlet, amit a konkrét standard library verzióján kell ellenőrizni.

A `std::scoped_allocator_adaptor` egy allokátor-adaptor, amely a külső allokátort „besugározza” a belső tárolókba: egy `vector<vector<T, A>, A>` belső vektorai is a külső allokátorral (vagy annak megadott belső párjával) épülnek fel, `uses_allocator` konstrukción keresztül. Így a beágyazott szerkezet egyetlen memóriarezsimen marad. A fogalmi térkép: *külső rezsim* → *scoped_allocator_adaptor* → *belső tárolók*, ahol a propagation trait-ek itt is ugyanazt a kérdést teszik fel, csak egy szinttel beljebb.

37.1.6 Kidolgozott példa: egyenlőtlen allokátorú move assignment

Az alábbi program állapotos allokátort használ, naplózza az `allocate`/`deallocate` hívásokat, és megszámolja az elem-mozgatásokat. A két tároló allokátora szándékosan nem egyenlő, POCMA pedig hamis.

```
// alloc_move.cpp - C++20, GCC/Clang
#include <cstddef>
#include <cstdio>
#include <memory>
#include <type_traits>
#include <utility>
#include <vector>

struct Tracked {
    int value = 0;
    static inline int moves = 0;
    static inline int copies = 0;

    Tracked() = default;
    explicit Tracked(int v) : value(v) {}
    Tracked(const Tracked& o) : value(o.value) { ++copies; }
    Tracked(Tracked&& o) noexcept : value(o.value) { ++moves; }
    Tracked& operator=(const Tracked&) = default;
    Tracked& operator=(Tracked&&) noexcept = default;
};

template <class T>
struct tagged_allocator {
    using value_type = T;
    using propagate_on_container_move_assignment = std::false_type; // POCMA
    using propagate_on_container_swap = std::false_type;            // POCS
    using is_always_equal = std::false_type;

    int id = 0;
```

```
    tagged_allocator() noexcept = default;
    explicit tagged_allocator(int i) noexcept : id(i) {}
    template <class U>
    tagged_allocator(const tagged_allocator<U>& o) noexcept : id(o.id) {}

    T* allocate(std::size_t n) {
        std::printf("  allocate(%zu) id=%d\n", n, id);
        return static_cast<T*>(::operator new(n * sizeof(T)));
    }
    void deallocate(T* p, std::size_t n) noexcept {
        std::printf("  deallocate(%zu) id=%d\n", n, id);
        ::operator delete(p);
    }

    template <class U>
    bool operator==(const tagged_allocator<U>& o) const noexcept { return id ==
     ↪  o.id; }
    template <class U>
    bool operator!=(const tagged_allocator<U>& o) const noexcept { return id !=
     ↪  o.id; }
};

int main() {
    using Vec = std::vector<Tracked, tagged_allocator<Tracked>>;

    Vec src(tagged_allocator<Tracked>{1});
    src.reserve(4);                 // egyetlen allokáció, nincs reallokáció
    src.emplace_back(1);
    src.emplace_back(2);
    src.emplace_back(3);

    Tracked::moves = Tracked::copies = 0;

    Vec dst(tagged_allocator<Tracked>{2});
    dst = std::move(src);           // egyenlőtlen allokátor + POCMA=false

    std::printf("elem-mozgatas=%d, masolas=%d, dst meret=%zu\n",
                Tracked::moves, Tracked::copies, dst.size());
}
```

Fordítás és futtatás a tanuló saját laborjában:

```
g++ -std=c++20 -O2 -Wall -Wextra -pedantic alloc_move.cpp -o alloc_move
# vagy
clang++ -std=c++20 -O2 -Wall -Wextra -pedantic alloc_move.cpp -o alloc_move
```

A várt kimenet (illusztráció, ezt a szöveg írásakor nem futtattam): a `dst = std::move(src)` három `allocate(4) id=2` típusú sort és három elem-mozgatást mutat, mivel a harmadik ág futott. Ha a POCMA-t `std::true_type`-ra állítod, az elem-mozgatás eltűnik, és a `dst` allokátora a forrás `id=1` értékét veszi át – a puffer átkerül, a régi pedig a forrás destruktorában szabadul fel. Ez a két futtatás együtt pontosan a fenti cost model két ágát demonstrálja; a mérést a mérési fejezet végzi el, itt a mechanizmus a tanulság.

37.1.7 Tipikus hibák és javításuk

- **Az `allocate`-ot objektum-létrehozásnak tekinteni.** A visszakapott területen nincs `T`; olvasása UB. Javítás: mindig a konstrukció/destrukció fázisát használd, és ne nyúlj a nyers területhez.
- **Az allokátor élettartamát összekeverni az erőforrás élettartamával.** Az arena előbb megszűnik, mint a tároló → UB a destruktorban. Javítás: az erőforrás legyen kimondottan hosszabb életű (pl. a `main`-ben vagy a szolgáltatás-szintű kontextusban).
- **Azt hinni, hogy a move assignment mindig O(1).** Állapotos, nem propagáló allokátornál O(n) plusz allokáció. Javítás: a szerződés részeként dokumentáld az allokátor tulajdonságait.
- **Egyenlőtlen, nem propagáló allokátorú tárolókat `swap`-elni.** Ez nem lassabb, hanem UB. Javítás: POCS bekapcsolása, vagy az allokátorok egyenlővé tétele.
- **Az `allocator_traits` megkerülése.** A közvetlen `a.allocate`/`a.construct` hívás kihagyja az alapértelmezéseket és a testreszabási pontokat. Javítás: mindig a `traits::` utat használd.
- **Azt hinni, hogy a `std::allocator` „zero-cost" garancia.** Ez legfeljebb egy adott implementáció és optimalizálási szint megfigyelése, nem szerződés.

37.1.8 Összefoglaló

Az allokátor modell két dolgot választ el, amit a legtöbb magas szintű nyelv összemos: a tárolóterület megszerzését és az objektum élettartamának elindítását. Az `allocator_traits` adja azt a kompatibilitási réteget, amely miatt egy minimális allokátor is elég, a `rebind` pedig azt, hogy a tároló a belső típusaira is ugyanazt a rezsimet használja. Az egyenlőség szemantikai fogalom (felszabadítható-e a tároló a másikkal), a propagation trait-ek pedig azt döntik el, hogy az allokátor vándorol-e a tárolóműveletekkel. Ennek közvetlen következménye a move assignment három ága és a `swap` érvényességi feltétele. Végül: az allokátorpéldány fogantyú; a mögötte lévő memóriarezsim élettartama a felhasználóé.

37.1.9 Feladatok

1. **(Magyarázó)** Egy `std::vector<T, A>`-n végrehajtod a `push_back` sorozatot, amely egy reallokációt okoz. Nevezd meg, mely lépésekben történik `allocate`, `construct`, `destroy` és `deallocate`, és indokold, miért ebben a sorrendben.
2. **(Kódolási)** Módosítsd a kidolgozott példa allokátorát úgy, hogy POCMA `std::true_type` legyen. Add meg a várható kimenet alakját, és fogalmazd meg az ellenőrzési feltételt, amivel a laborban igazolod, hogy a harmadik ág helyett az első futott.
3. **(Diagnosztikai)** Egy kollégád kódjában két `std::vector`-t `swap`-elnek, amelyek allokátora különböző `id`-jű, POCS pedig hamis. A program „néha összeomlik". Azonosítsd a hibát, és adj két különböző javítást, megnevezve mindkettő mellékhatását.

37.1.10 Megoldások

1. A `push_back` új kapacitást igényel: a tároló `allocate(new_cap)`-pal új területet szerez (ekkor még nulla elem él rajta), majd a meglévő elemeket `construct`-tal átmozgatja az új területre, ezután a régieket `destroy`-jal lezárja, végül `deallocate(old_ptr, old_cap)`-pal visszaadja a régi területet. Azért ez a sorrend, mert a régi pufferre a másolás/mozgatás befejezéséig szükség van, a felszabadítás pedig csak akkor érvényes, ha már egyetlen objektum élettartama sem függ

tőle. Ellenőrzési feltétel: a naplóban az új allokáció sorszáma megelőzi a régi felszabadítását, és `allocate`/`deallocate` száma páronként egyezik.

2. A `propagate_on_container_move_assignment` típust `std::true_type`-ra állítva a move assignment első ága fut: a puffer átkerül, a `dst` allokátora a forrás allokátorává válik, elem-mozgatás nem történik. Várható kimenet: a három `allocate` sor helyett legfeljebb a `src.reserve(4)` allokációja jelenik meg, `elem-mozgatas=0`. Ellenőrzési feltétel: a `Tracked::moves` nulla, és a `dst.get_allocator().id == 1` (a forrás azonosítója). Megjegyzés: a `dst` destruktora ekkor már `id=1`-gyel szabadít fel, ami összhangban van azzal, hogy a puffert is az az allokátor adta.

3. A hiba az, hogy a `swap` előfeltétele nem teljesül: POCS hamis és az allokátorok nem egyenlők, ezért a viselkedés nem definiált – innen a látszólag véletlenszerű összeomlás. Javítás A: állítsd a POCS-t `std::true_type`-ra, ekkor a `swap` legálisan kicseréli az allokátorokat is; mellékhatása, hogy a tárolók allokátora a művelet után megváltozik, amit a további kódnak tolerálnia kell. Javítás B: tedd egyenlővé az allokátorokat (azonos rezsim), ekkor POCS hamis mellett is definiált a művelet; mellékhatása, hogy a két tároló ugyanarra a memóriarezsimre kerül, így elveszik a szigetelés, amiért eredetileg külön allokátort használtál. Mindkét javításnál ellenőrzési feltétel, hogy a `swap` után bármelyik tároló destruktora a saját (vagy a propagált) allokátorával szabadít fel, és a felszabadítások száma páros.

37.2 pmr, memory_resource és region lifetime

Az allokátor-modell Allocator model és allocator_traits (37.1) alfejezetben rögzített alapja az, hogy a konténer típusa hordozza az allokátor típusát, így a tároló viselkedése már fordítási időben eldől. Ez kiváló a statikus, fordítási idejű döntésekhez, de kellemetlen ott, ahol ugyanaz a tárolótípus futásidőben más-más memóriastratégiával kell dolgozzon: egy kérés élettartamára szóló aréna, egy hosszú életű pool vagy a globális heap. A C++17 óta rendelkezésre álló *polymorphic allocator* (`std::pmr`) pontosan ezt a rést tölti ki: a tároló típusa változatlan marad, a tényleges memóriastratégiát egy futásidejű `memory_resource` objektum hordozza. Ez a C aréna mintájának (Arena, pool és region lifetime (13.4)) típusbiztos, destruktorhelyes megfelelője, de nem azonos vele: a C++ régió felszabadítása nem hív destruktort, és a konténerek élettartama külön szerződést igényel. Az alábbiakban ezt a szerződést bontjuk ki, majd egy per-request tárolón mutatjuk meg a helyes és a tipikus hibás mintákat. A mérés és a benchmark módszertana a következő alfejezet témája; itt nem adunk univerzális benchmarkot és nem ígérünk zero-cost viselkedést.

37.2.1 A memory_resource szerződés és a polymorphic_allocator

A `std::pmr::memory_resource` absztrakt interfész, amelynek három művelete van: `allocate(bytes, alignment)`, `deallocate(p, bytes, alignment)` és `is_equal(other)`. A nyilvános `allocate`/`deallocate` nem virtuális; a tényleges munkát a protected `do_allocate`, `do_deallocate` és `do_is_equal` végzi. Ez a nem virtuális interfész (NVI) minta azt jelenti, hogy a resource viselkedése örökléssel specializálható, a hívó oldalon viszont stabil, nem virtuális függvényhívás marad. Az allokációs szerződés feltételei: a `deallocate` pointere ugyanabból az `allocate` hívásból származzon, ugyanazzal a mérettel és igazítással; a resource objektumnak élnie kell a `deallocate` pillanatában; és a visszaadott pointernek teljesítenie kell a kért igazítást. A `bytes` és `alignment` páros a szerződés része, nem opcionális metaadat: a legtöbb implementáció a `deallocate`-ban ezekből dolgozik, és a méret elhagyása vagy megváltoztatása nem definiált viselkedés.

A `std::pmr::polymorphic_allocator<T>` egy allokátor-típus, amely egy `memory_resource*` mutatót tárol. Fontos pontosítás: nem „stateless” allokátor, hiszen van állapota, de ez

az állapot nem jelenik meg a konténer típusában. A `std::pmr::vector<T>` valójában `std::vector<T, std::pmr::polymorphic_allocator<T>>`, és ugyanez igaz a `pmr::string`, `pmr::map`, `pmr::unordered_map` és a többi pmr-típusra. Két `polymorphic_allocator` akkor egyenlő, ha ugyanarra a resource-ra mutatnak, vagy ha a resource `do_is_equal`-je igazat ad. Ezen az egyenlőségen múlik a konténer mozgatása és cseréje: a `polymorphic_allocator` nem propagálja magát copy assignment, move assignment vagy swap során, ezért ha két konténer allokátora nem egyenlő, a move assignment elemenkénti mozgatássá válik, nem pedig a puffer eltulajdonításává. Ez a költségmodell egyik fontos tétele, amelyet a mérésnél figyelembe kell venni.

A pmr-tárolók létrehozásakor a resource pointert a konténer konstruktorának adjuk át. A `std::pmr::vector<std::pmr::string> v(&res);` azt jelenti, hogy a vektor saját puffere a `res`-ből származik. Ez azonban önmagában nem jelenti azt, hogy a benne lévő `pmr::string` elemek is a `res`-ből allokálnak. A `pmr::string` allokátortípusa `polymorphic_allocator<char>`, a vektoré `polymorphic_allocator<std::pmr::string>`; a kettő különbözik, ezért a vektor `construct`-ja nem adja át automatikusan a saját allokátorát a string konstruktorának. Ha az elemeket `v.emplace_back("szöveg")` formában hozzuk létre, azok a `get_default_resource()`-on foglalnak, és a vektor resource-ának felszabadítása nem érinti a stringek puffereit. Ez a *nested allocation* tipikus csapdája. A helyes minta az, hogy az elemnek explicit átadjuk a resource-ot: `v.emplace_back(sv.data(), sv.size(), &res);` vagy `m.try_emplace(kulcs, "érték", &res);`. Utóbbi a `std::pmr::map<int, std::pmr::string>` esetében a `pair` piecewise konstruktorán keresztül éppen a stringnek adja át a resource-ot. Az allocator-aware construction tehát nem varázslat: a beágyazott tároló allokátorát a keletkezés pillanatában kell megadni, különben a beágyazott objektum a default resource-ra esik vissza.

37.2.2 Region lifetime: monotonic, pool és release

A `std::pmr::monotonic_buffer_resource` egy régió: minden allokáció egy növekvő pufferből történik, a `deallocate` pedig nem tesz semmit. A `release()` visszaadja az upstream resource-nak a teljes addig felhasznált területet, és a resource újra üres állapotból használható. A `release()` után azonban minden korábban kiadott pointer, iterátor, `string_view`, `span` és referencia érvénytelenné válik. Az upstream resource élettartama legalább akkora kell legyen, mint a monotonic resource-é; ha külső puffert adunk át a konstruktorban, a puffer élettartama is legalább ekkora kell legyen. A monotonic resource nem szálbiztos: egy szálon belüli, egyszeri összeállításra és eldobásra való, nem hosszú életű, több szál által osztott allokátorként.

Az `std::pmr::unsynchronized_pool_resource` és a `std::pmr::synchronized_pool_resource` ezzel szemben méretosztályok szerinti poolokból szolgál ki, és a `deallocate` visszateszi a blokkot a megfelelő poolba. A `release()` itt is visszaadja az upstream-nek a poolok által birtokolt memóriát, de csak akkor biztonságos, ha addigra minden belőle allokált objektum megszűnt. Az „unsynchronized” változat nem szálbiztos, a „synchronized” változat belső zárakkal szolgál ki; utóbbi nagyobb overheadtel jár, és a `release()` párhuzamos használatának megengedettsége implementációfüggő, ezért a konkrét implementáció dokumentációját kell ellenőrizni. A választás indoka: ismétlődő, hasonló méretű, rövid életű allokációkra a pool; egyetlen kérés élettartamára, ahol a felszabadítás egyszerre történik, a monotonic; a kettő kombinációja pedig az, amikor a monotonic a poolt használja upstreamként.

A régió élettartamának központi szabálya: a resource élettartama nagyobb vagy egyenlő a belőle allokált összes objektum élettartamánál. Ez a C++-ban szigorúbb, mint a C arénában, mert a konténer destruktora nem csak felszabadít, hanem destruktorokat is hív, és a `deallocate`-ot a resource-on keresztül végzi. Ha a resource előbb pusztul el, mint a konténer, a konténer destruktora egy már nem létező objektum virtuális függvényét hívja meg, ami nem definiált viselkedés. Ugyanez igaz a `release()`-re is: a `release()` nem hív destruktort, ezért a benne lévő

nem triviális destruktorú elemeket előbb destroy-olni kell. A `pmr::vector::clear()` meghívja az elemek destruktorait, de a vektor kapacitása megmarad, és a puffer memóriája a resource-hoz tartozik; a `release()` után a vektor belső pointerei danglingek, ezért a vektort a release után nem szabad újra használni. A helyes minta az, hogy a `release()`-et csak a konténer megszűnése után, vagy a konténer újrakonstruálásával együtt hívjuk. A borrowed view-kra ugyanez vonatkozik: a `string_view`, `span`, `const char*` és iterátorok a release pillanatában érvénytelenné válnak, még akkor is, ha a mögöttes karakterek látszólag változatlanok.

37.2.3 Kidolgozott példa: per-request tároló

Az alábbi példa egy kérés élettartamára szóló tárolót mutat be. A `RequestScope` a resource-ot és a konténert tartalmazza; a deklarációs sorrend szándékos, mert a tagok fordított sorrendben pusztulnak el, így a konténer előbb szűnik meg, mint a resource. A `add` tagfüggvény a stringnek explicit átadja a scope resource-át, elkerülve a nested allocation csapdáját.

```
#include <cstddef>
#include <cstdio>
#include <memory_resource>
#include <string>
#include <string_view>
#include <vector>

class RequestScope {
public:
    explicit RequestScope(std::size_t buffer_bytes = 4096)
        : resource_(buffer_bytes), strings_(&resource_) {}

    RequestScope(const RequestScope&) = delete;
    RequestScope& operator=(const RequestScope&) = delete;

    void add(std::string_view sv) {
        // A vektor uses-allocator konstrukcióval maga adja át a resource-át
        // a pmr::string-nek; ha itt is átadnánk, kétszer kerülne a listába.
        strings_.emplace_back(sv.data(), sv.size());
    }

    const std::pmr::vector<std::pmr::string>& strings() const noexcept {
        return strings_;
    }

    std::pmr::memory_resource* resource() noexcept { return &resource_; }

private:
    // Szándékos sorrend: a resource előbb jön létre, és utoljára pusztul el.
    std::pmr::monotonic_buffer_resource resource_;
    std::pmr::vector<std::pmr::string> strings_;
};

int main() {
    RequestScope scope;
    scope.add("GET /health HTTP/1.1");
```

```
    scope.add("Host: example.invalid");
    for (const auto& line : scope.strings()) {
        // Minden string a scope resource-ából allokál.
        if (line.get_allocator().resource() != scope.resource()) return 1;
        std::printf("%s\n", line.c_str());
    }
}
```

A példa önállóan fordítható a C++17-es `<memory_resource>` fejlécet támogató implementáción. A laborprofilban rögzített fordítóval és standard library-vel ellenőrizni kell a tényleges támogatást; a könyv alapértelmezett C++20 profillal dolgozik, de a pmr nem C++20-as újdonság. Fordítási példa:

```
g++ -std=c++20 -O2 -Wall -Wextra -pedantic-errors request_scope.cpp -o
 ↪  request_scope
```

A `std::pmr::vector<std::pmr::string>` **uses-allocator konstrukciót** végez (C++20 [allocator.uses.construction]): mivel a `std::pmr::string` allokátort használó típus, a vektor az új elem konstruktorának argumentumlistáját maga egészíti ki a saját `polymorphic_allocator`-ával. Ezért az `emplace_back(sv.data(), sv.size())` elég, és a string a scope resource-ából allokál. Ha a hívó is átadná a resource-ot (`emplace_back(sv.data(), sv.size(), &resource_)`), az allokátor kétszer kerülne az argumentumlistába, és a fordítás `construction with an allocator must be possible if uses_allocator is true` hibával leállna. Egy közönséges `std::string` elem ezzel szemben nem allokátortudatos: a vektor nem adja át neki a resource-ot, és a karakterei a globális heapre kerülnének.

Buildjegyzőkönyv (`lab-gcc` profil). A program figyelmeztetés nélkül fordult, a kimenete a két hozzáadott sor, és minden string `get_allocator().resource()`-a a scope resource-a. A javítás előtti, kétszer átadott allokátoros változat a fenti `static assertion failed` hibával nem fordult. A program nem futtatja a resource `release()`-ét; a scope megszűnésekor a `strings_` destruktora lefut, majd a `resource_` destruktora következik. Ez a természetes, biztonságos minta. Ha a kérés végén explicit „reset” kell, egyetlen helyes sorrend van: (1) **minden** objektum, amely a resource-ból allokált, megsemmisül; (2) `release()`; (3) csak ezután jönnek létre az új objektumok. A legegyszerűbb alapminta, hogy az egész `RequestScope`-ot eldobjuk és újat hozunk létre. Ha a scope-nak meg kell maradnia, a konténert `std::optional`-ban tartjuk:

```
// scope_reset.cpp - C++20: helyes reset sorrend pmr resource-szal
#include <cstdio>
#include <memory_resource>
#include <optional>
#include <string>
#include <vector>

class RequestScope {
public:
    RequestScope() { strings_.emplace(&resource_); }

    void add(const char* s) { strings_->emplace_back(s); }
    std::size_t size() const { return strings_->size(); }

    // Egyetlen helyes sorrend: (1) minden, a resource-ot használó objektum
    // megsemmisül, (2) release, (3) új objektumok konstrukciója.
    void reset() {
        strings_.reset();          // (1) a vektor és a stringek destruktora még
```

```
                                    //     érvényes resource-ra hív
        resource_.release();        // (2) a régió visszaadása
        strings_.emplace(&resource_);   // (3) új, üres konténer
    }

#ifdef WRONG_ORDER
    void reset_wrong() {
        strings_->clear();          // a vektor puffere a régióban marad...
        resource_.release();        // ...amit a release felszabadít
    }
#endif

private:
    std::pmr::monotonic_buffer_resource resource_;              // előbb jön
     ↪ létre
    std::optional<std::pmr::vector<std::pmr::string>> strings_;  // előbb szűnik
     ↪ meg
};

int main() {
    RequestScope scope;
    for (int round = 0; round < 3; ++round) {
        scope.add("GET /health HTTP/1.1 -- eleg hosszu, hogy ne ferjen SSO-ba");
        scope.add("Host: example.invalid");
#ifdef WRONG_ORDER
        scope.reset_wrong();
#else
        scope.reset();
#endif
    }
    std::printf("scope_reset: OK size=%zu\n", scope.size());
}
```

Buildjegyzőkönyv (`lab-gcc` profil). ASan/UBSan alatt a helyes `reset()` három körön át `scope_reset: OK size=0` kimenettel futott. A `-DWRONG_ORDER` változat (`clear()`, majd `release()`) `heap-use-after-free` hibát adott: a `clear()` csak az elemeket semmisíti meg, a vektor puffere a régióban marad, és a következő `emplace_back` már a felszabadított memóriába ír.

A beágyazott allokáció helyes kezelésére álljon itt egy jelölt részlet egy `pmr::map` esetében. A `try_emplace` a `pair` piecewise konstruktorán keresztül a mappelt stringnek is uses-allocator konstrukcióval adja át a map resource-át; a hívónak ezért nem kell (és nem is szabad) külön átadnia. A `lab-gcc` profilban egy `main`-nel kiegészítve a részlet fordul, és a `m.at(1).get_allocator().resource() == &res` igaz:

```
// Részlet, nem önálló program: a res élettartama nagyobb a mapénál.
std::pmr::monotonic_buffer_resource res;
std::pmr::map<int, std::pmr::string> m(&res);
m.try_emplace(1, "alma");   // a string a res-ből allokál (uses-allocator)
```

A három stratégia összevetése:

Jelmagyarázat:

- **C aréna:** az Arena, pool és region lifetime (13.4) rész arénája;
- **monotonic:** `std::pmr::monotonic_buffer_resource`;
- **pool:** `std::pmr::unsynchronized_pool_resource`.

Szempont	C aréna	monotonic	pool
Visszafoglalás	`arena_reset`	`release()` regionálisan, a használók megsemmisítése után	`deallocate` blokkonként
Destruktor	nincs	nincs (a konténer destruktora külön)	nincs (a konténer destruktora külön)
Szálbiztonság	nem	nem	nem (a szinkronizált pool igen)
Allokációs költség	bump pointer	bump pointer + virtuális hívás	pool-keresés + virtuális hívás
Külső fragmentáció	nincs (régión belül)	nincs (régión belül)	méretosztályon belül korlátozott
Belső veszteség	igazítási padding, chunkvég	igazítási padding, chunkvég, növekvő blokkok	méretosztályra kerekítés
Release-ig visszatartott	minden foglalás	minden foglalás, a konténer régi blokkjai is	felszabadított blokk a poolban marad
Élettartam	régió	régió	a pool élettartama

A monotonic stratégia egyszerű felszabadítása tehát nem jelent nulla veszteséget: a memóriaigényt a kért hasznos méret, a ténylegesen foglalt blokkméret (padding, kihasználatlan chunkvég) és a release-ig vissza nem nyerhető foglalások (például egy növekvő vektor elhagyott régi pufferei) együtt adják. **Mérés** (`lab-gcc`, számláló upstream resource-szal): egy 1000 `int`-es `pmr::vector` egyenkénti `push_back`-kel 4000 bájt hasznos adatot, 4096 bájt végső kapacitást, de 9216 bájt upstream-foglalást eredményezett, amely a `release()`-ig visszatartva maradt.

37.2.4 Tipikus hibák és javításuk

A leggyakoribb hiba a tagok sorrendje: ha a `pmr::vector` előbb van deklarálva, mint a resource, akkor a destruktorok sorrendje miatt a resource előbb szűnik meg, és a vektor destruktora dangling resource-ra hív. A javítás a resource előre deklarálása, vagy a resource és a konténer egyetlen owner osztályba zárása. A második hiba a `release()` utáni használat: a `string_view`-k, iterátorok és a konténer belső pointerei érvénytelenné válnak; a release előtt a fogyasztóknak be kell fejezniük az olvasást. A harmadik hiba a nested allocation figyelmen kívül hagyása: a `pmr::vector<pmr::string>` elemei alapértelmezésben a default resource-on allokálnak, így a scope resource-ának felszabadítása nem szabadítja fel az elemek puffereit, és a regionális költségmodell nem teljesül. A negyedik hiba a `monotonic_buffer_resource` megosztása több szál között; erre a `synchronized_pool_resource` vagy külső szinkronizáció való. Végül az ötödik: a `release()` és a többi művelet egyidejű hívása a pool resource esetében; ezt a konkrét implementáció dokumentációja szerint kell ellenőrizni, és a legtöbb esetben külső szinkronizációt igényel.

37.2.5 Feladatok

1. **Magyarázó feladat.** Hasonlítsa össze a C aréna `arena_reset` műveletét a `monotonic_buffer_resource::release()` műveletével! Miért nem elég a `release()`, ha a tárolóban nem triviális destruktorú elemek vannak, és mi a helyes műveletsor?

2. **Kódolási feladat.** Módosítsa a `RequestScope` példát úgy, hogy a monotonic helyett a fenti táblázat pool resource-át használja, és a `reset()` a konténer `clear()`-je után hívja a resource `release()`-ét! Magyarázza el, miért veszélyes ez a vektorral, és javítsa úgy, hogy a konténert is újrakonstruálja, vagy mondjon le a `reset()`-ről! Adja meg a buildparancsot!

3. **Diagnosztikai feladat.** Adott az alábbi vázlat. Soroljon fel legalább három élettartam- vagy ownership-hibát, és javítsa a kódot!

```
std::pmr::vector<std::pmr::string> v;
std::pmr::monotonic_buffer_resource res;
v.emplace_back("alma");
res.release();
for (const auto& s : v) std::puts(s.c_str());
```

37.2.6 Megoldások

1. A C aréna `arena_reset`-je csak a bump pointert állítja vissza, és nem hív destruktort, mert a C aréna nem tud a benne lévő objektumok típusáról. A `release()` ugyanezt teszi a monotonic resource szintjén: a nyers memóriát adja vissza az upstream-nek, de a benne lévő C++ objektumok destruktorait nem hívja meg. Ha a tárolóban nem triviális destruktorú elemek vannak, a `release()` előtt azokat destroy-olni kell; a `pmr::vector` esetében ez a `clear()`, amely az elemek destruktorait hívja, de a vektor kapacitását és belső pointereit nem állítja vissza. A helyes sorrend ezért: a fogyasztók befejezik az olvasást, a konténert megszüntetjük (vagy újrakonstruáljuk), és csak ezután hívjuk a `release()`-ét. Az ellenőrzési feltétel: a `release()` után semmilyen korábbi pointer, `string_view`, `span` vagy iterátor nem használható, és a konténer destruktora már lefutott.

2. A poolos változatban a `reset()` így nézne ki: `strings_.clear(); resource_.release();`. Ez azért veszélyes, mert a `clear()` után a vektor kapacitása és belső puffere megmarad, de a puffer a resource-ból származik; a `release()` után a vektor belső pointerei danglingek, és a vektor további használata nem definiált viselkedés. A helyes javítás vagy az, hogy a `reset()` a konténert is újrakonstruálja (például `std::optional`-ban tartott konténer `emplace`-jével), vagy az, hogy a `reset()` helyett a teljes `RequestScope`-ot eldobjuk és újat hozunk létre. A pool választásának indoka a visszatérő, hasonló méretű allokációk kezelése; a monotonic akkor jobb, ha a kérés végén úgyis mindent eldobunk. A buildparancs a példához hasonlóan `g++ -std=c++20 -O2 -Wall -Wextra -pedantic-errors`. Ellenőrzési feltétel: a `release()` után a konténerre semmilyen művelet nem történik, és a destruktorok a release előtt lefutnak.

3. A vázlat hibái: (a) a `v` allokátora a default resource-ra mutat, mert a vektort allokátor nélkül hoztuk létre, így a `res` release-e nem érinti a vektor pufferét; (b) a `res` a `v` után van deklarálva, ezért előbb pusztul el, mint a vektor, ami a vektor destruktorában dangling resource-ra hívna; (c) az `emplace_back("alma")` a stringet a default resource-on hozza létre, nem a `res`-en, így a nested allocation nem a kívánt régióba kerül; (d) a `res.release()` után a `v` elemeit olvassuk, ami a vektor pufferére és a stringekre nézve is dangling pointert jelent. Javított változat: a resource-ot előbb deklaráljuk, a vektort explicit allokátorral hozzuk létre, a stringet explicit resource-szal konstruáljuk, és a `release()`-et csak a vektor megszűnése után hívjuk (vagy a vektort a release előtt destroy-oljuk). Ellenőrzési feltétel: a `release()` előtt minden nem triviális destruktor lefut, és a release után semmilyen a régióra mutató hivatkozás nem él.

37.2.7 Összefoglaló

A `std::pmr` a futásidejű allokációs stratégiát választja el a tároló típusától: a `memory_resource` a szerződés, a `polymorphic_allocator` a híd, a pmr-tárolók pedig a típusstabil felhasználók. A `monotonic_buffer_resource` a C aréna típusbiztos megfelelője, de a `release()` nem hív destruktort, ezért a C++-ban a konténerek és elemek élettartamát külön kell kezelni. A pool resource a visszatérő, hasonló méretű allokációkra való; a synchronized változat szálbiztos, az unsynchronized nem. A nested allocation csak akkor marad a régión belül, ha a beágyazott tároló allokátorát a keletkezéskor explicit megadjuk. A régió élettartama mindig nagyobb vagy egyenlő a belőle allokált objektumok élettartamánál, és a `release()` pillanata minden korábbi pointer, view és iterátor érvényességének végét jelenti.

37.3 Object layout, EBO és adatközpontú tervezés

A korábbi részekben felépítettük a C memóriamodell alignment- és padding-szabályait (Padding, alignment és offsetof (9.2)), a cache-viselkedés és a false sharing hardveres hátterét (Cache, locality, alignment és false sharing (14.4)), valamint azt, hogy a C++ objektumlayout ABI-szerződés, nem nyelvi absztrakció (Névtorzítás, layout és C++ runtime ABI (36.1)). A dinamikus allokáció oldaláról a `pmr` és a region-élettartam adta meg a keretet (pmr, memory_resource és region lifetime (37.2)). Ez az alfejezet ezeket kapcsolja össze: hogyan lesz a C padding- és cache-tudásból C++ kompozíciós és polimorfikus döntés, és hogyan lehet ezt mérésből levezetett cost modellel kezelni a helyességi szerződés megsértése nélkül.

Az alapállítás, amelyre minden további épül: a C++ szabvány a layoutról keveset garantál, viszont az implementáció (ABI) sokkal többet rögzít. A kettőt következetesen szét kell választani. Amit a szabvány megkövetel, az minden konform implementáción igaz; amit az ABI rögzít, az az adott target és compiler-párosítás mellett igaz, és a következő minor verzióban megváltozhat.

37.3.1 A layout, amit a szabvány garantál, és amit nem

A szabvány annyit mond ki, hogy egy nem üres osztály objektuma legalább annyi bájtot foglal, amennyi a nem statikus adattagjai összege, plusz az igazításból és a bázisosztály-részekből adódó kitöltés; továbbá hogy két, egyidejűleg létező, nem potenciálisan átfedő hasonló típusú részobjektumnak különböző címe kell legyen. A pontos offseteket, a padding helyét és méretét, a virtuális táblára mutató rejtett pointer helyét, valamint azt, hogy egy üres bázis egyáltalán nulla méretű-e, a szabvány nem rögzíti. Ezeket az ABI írja le — x86-64-en az Itanium C++ ABI System V változata —, tehát a konkrét számok mellett mindig meg kell nevezni a targetet, a compilert és a nyelvi módot.

A tipikus (nem garantált, de a mainstream ABE-kben követett) elrendezési szabály a következő. Legyen a tagok sorrendje $m_1, \dots, m_n$, méretük s_i, igazításuk a_i. Az offset rekurzívan:

$$o_1 = 0, \qquad o_{i+1} = \left\lceil \frac{o_i + s_i}{a_{i+1}} \right\rceil \cdot a_{i+1}$$

ahol a_i kettőhatvány, ezért a felfelé kerekítés bitművelettel is kifejezhető:

$$\text{round_up}(x, a) = (x + a - 1) \;\&\; \sim (a - 1)$$

A struktúra igazítása $\max_i a_i$, a mérete pedig $\text{round_up}(o_n + s_n, \max_i a_i)$. Ez a levezetés magyarázza, miért nem kommutatív a tagok sorrendje.

Vegyük a következő két típust, azonos tagokkal:

```
struct Order1 { char c; int i; double d; };
struct Order2 { int i; double d; char c; };
```

Tag	Típus	Méret	Igazítás	Offset (`Order1`)	Offset (`Order2`)
`c`	`char`	1	1	0	16
`i`	`int`	4	4	4	0
`d`	`double`	8	8	8	8

`Order1`-ben a `c` után 3 bájt hézag kerül, a `d` a 8-as offsetre esik, a végén nincs farokpadding: $\text{round_up}(16, 8) = 16$. `Order2`-ben az `i` után 4 bájt hézag, a `d` után a `c` a 16. offsetre kerül, a végét 24-re kerekíti a szerkezet 8-as igazítása. Ugyanaz a három tag tehát 16 és 24 bájtos típust ad — a sorrend nem esztétikai kérdés.

Amit a szabvány ebből ténylegesen garantál, az két traitben fogható meg. A `std::is_standard_layout<T>` azt jelenti, hogy a típus layoutja kiszámítható, C-vel kompatibilis módon írható le, és `offsetof` legális rajta. A `std::is_trivially_copyable<T>` azt jelenti, hogy az objektum bájtonként másolható, tehát `std::memcpy`-vel átvihető. Egyik sem garantálja a padding hiányát vagy konkrét offseteket: a `sizeof(Order1) == 16` állítás nem szabványos, hanem ABI-megfigyelés.

37.3.2 Empty base optimization és a `[[no_unique_address]]`

Egy üres osztálynak is van mérete: `sizeof(Empty) >= 1`, különben két különböző objektum azonos címre esne. Amikor viszont üres osztály bázis-részobjektumként jelenik meg, a szabvány megengedi, hogy nulla méretű legyen és egy más típusú részobjektummal azonos címen osztozzon. Ezt nevezi a szakirodalom empty base optimizationnak (EBO). Fontos: a szabvány *megengedi*, nem *kötelezővé teszi* — a tényleges alkalmazás implementációfüggő, a gyakorlatban azonban minden elterjedt ABI végrehajtja.

Az EBO nem stílus kérdése, hanem az allokátor-, comparator- és deleter-alapú tervezés alapja. Ha egy `std::vector<T, MyAlloc<T>>` allokátora állapotmentes (csak típusparamétert hordoz, adattagot nem), akkor EBO nélkül minden vektorobjektum feleslegesen hízna egy bájttal plusz paddinggel. Az EBO teszi lehetővé, hogy az állapotmentes policy ne kerüljön semmibe.

C++20-tól ugyanez adattagra is kérhető:

```
struct Empty {};
struct AsMember   { Empty e; int x; };
struct AsBase     : Empty { int x; };
struct AsNoUnique { [[no_unique_address]] Empty e; int x; };
struct AsTwoNoUnique { [[no_unique_address]] Empty a;
                       [[no_unique_address]] Empty b; int x; };
```

LP64 x86-64-en `AsMember` mérete 8 (`Empty` 1 bájtja + 3 bájt padding), `AsBase` és `AsNoUnique` 4, `AsTwoNoUnique` viszont 8. Az utóbbi nem hiba: a szabvány két azonos típusú, nulla méretű részobjektumra is megköveteli a különböző címet, így azok nem fedhetik át egymást. A `[[no_unique_address]]` tehát nem univerzális tömörítő eszköz, hanem *különböző típusú* tagok átfedését teszi lehetővé. A tag típusa továbbra is `std::is_empty`, csak a címhasználat változik

meg: ha a címét veszed, az más taggal azonos lehet — erre akkor figyelj, ha pointert vagy referenciát tárolsz rá.

37.3.3 Virtuális objektum és type-erased wrapper költsége

Egy virtuális függvényt tartalmazó osztály példánya rejtett pointert hordoz a virtuális táblára (vptr). Az Itanium ABI x86-64-en ezt a 0. offsetre teszi, mérete 8 bájt; a tábla maga az offset-to-top, a típusazonosító és a virtuális függvények pointereit tartalmazza. Hogy hol keletkezik, azt az Itanium ABI *key function* szabálya dönti el: ha az osztálynak van első, nem inline, nem tiszta virtuális függvénye (a key function), a vtable csak abban az objektumfájlban kerül kibocsátásra, amely azt definiálja; ha nincs ilyen (minden virtuális függvény inline), a vtable minden használó fordítási egységben *vague linkage*-dzsel, COMDAT-csoportban keletkezik, és a linker egyetlen példányra egyesíti. **Mérés** (`lab-gcc`, `nm -C`, három TU): a key functionös `Keyed` vtable-je csak a `keyed.o`-ban szerepelt (`V`), a másik kettőben definiálatlan (`U`); az inline `Inline` vtable-je mindkét használó objektumfájlban megjelent (`V`, COMDAT-csoport), a kész programban pedig mindkét osztálynak egyetlen vtable-je maradt. Kódméret-becslésnél tehát az objektumfájlokban látott kibocsátás nem azonos a végleges programéval. Következmény: `struct Base { virtual ~Base() = default; };` esetén `sizeof(Base) == 8`, egy `int` tagot hordozó leszármazott esetén 16 (vptr + `int` + 4 bájt farokpadding).

A virtuális hívás költsége nem a tranzisztorszám, hanem az, hogy a hívásindirekció miatt a fordító nem tud inline-olni, így elveszik a konstans-propagáció és a vektorizálás lehetősége a hívott függvényben. A devirtualizálás nem nyelvi újdonság, hanem fordítói optimalizálás, amelyet az as-if szabály minden szabványverzióban megenged: ha a fordító bizonyítani tudja a dinamikus típust (például a hívás egy lokálisan létrehozott, ismert típusú objektumon történik), a virtuális hívást közvetlen hívással helyettesítheti — egy C++17-es programban is. A `final` (C++11) ebben nyelvi információt ad: egy `final` osztály vagy függvény nem írható felül, így a rajta át végzett hívás célja statikusan ismert. Hogy a fordító ténylegesen devirtualizál-e, az optimalizálási szinttől és a látható kódtól függ; LTO csak akkor kell, ha a bizonyításhoz szükséges információ más fordítási egységben van (például a teljes osztályhierarchia), a lokális esetekhez nem. A `lab-gcc` profilban egy `-std=c++17 -O2` alatt fordított próbában a `Derived d; return call(d);` (ahol `call` egy `const Base&`-en hív) és a `final` osztályon át végzett hívás is egyetlen `movl $42, %eax` utasítássá egyszerűsödött: nem maradt virtuális hívás, LTO nélkül. Amit a szabvány rögzít: a virtuális hívás szemantikája; amit nem: hogy mekkora a költsége, vagy hogy egyáltalán nincs-e költsége.

A type erasure (típusonkénti törlés) ugyanezt a kérdést más alakban teszi fel. A `std::function<void()>` egy kis, lokális puffert (small buffer) tart, és ha a becsomagolt callable belefér, nem allokál; ha nem, heap-re kerül. A puffer mérete és a `sizeof(std::function)` implementációfüggő — libstdc++-on x86-64-en jellemzően 32 bájt, libc++-on eltérő. Ezért a `std::function` méretére és allokációmentességére nem lehet API-szerződést építeni. Ahol a szerződés számít, ott a C-szerű `void* context + void (*fn)(void*)` pár két pointer, allokáció nélkül — cserébe elveszik a típusbiztonság és a lambda-kényelem.

37.3.4 AoS, SoA és folytonos tárolás

A cache-viselkedés (Cache, locality, alignment és false sharing (14.4)) közvetlen következménye az adatközpontú tervezés. Tekintsünk egy részecske-rendszert:

```
struct AoS { float x, y, z, vx, vy, vz; };   // 24 bájt
```

Egy AoS-tömbön végigmenve egy 64 bájtos cache-line-ból körülbelül 2 egész részecske fér el. Ha a forró ciklus csak a pozíciókat frissíti, a betöltött bájtok jelentős része kihasználatlan: a sebességvektorok is bekerülnek a cache-be. A SoA (structure of arrays) elrendezésben hat párhuzamos `std::vector<float>` van; a pozíciókra futó ciklus csak a három pozíciótömböt érinti, a stride 1, így a hardver-prefetcher és a SIMD-egység is hatékonyabban dolgozik.

Elrendezés	Egy elem tárolt bájtja	Egy elemhez tartozó, használt bájt	Egymást követő elemek egy 64 bájtos vonalon
AoS (csak pozíció olvasva)	24	12	~2.6
SoA (csak pozíció olvasva)	12	12	~5.3

Az arány körülbelül 2:1 a betöltött bájtokban — ez számított érték a méretekből, nem mérés. A valós gyorsulás ennél kisebb vagy nagyobb is lehet, mert függ a ciklus egyéb költségeitől, a memóriaszinttől és attól, hogy a fordító vektorizál-e.

A folytonosság nem esztétikai tulajdonság: a `std::vector` és a `std::array` garantálja, hogy az elemek egy összefüggő tömbben vannak, ezért pointerrel bejárhatók és `data()`-val átadhatók C API-nak. A `std::deque` és a `std::list` ezt nem garantálja; a `list` csomópontjai szétszórtan helyezkednek el, ami minden bejárásnál cache-miss-t okoz. A tárolóválasztás így teljesítménykérdés is, nem csak interface-kérdés (Tárolóválasztási és invalidációs labor (28.6)).

37.3.5 Small-buffer optimalizálás mint implementációs részlet

A `std::string` small-string optimizationja (SSO) ugyanabba a kategóriába esik: a rövid stringeket a string objektum saját tárhelyén tárolja, elkerülve az allokációt. A küszöb azonban implementációfüggő: libstdc++-ban jellemzően 15 karakter (32 bájtos objektum mellett), libc++-ban más, MSVC alatt megint más. Ezért:

- `sizeof(std::string)` nem hordozható konstans;
- az „ez a string biztosan nem allokál” kijelentés nem szerződés, hanem az adott implementáció tulajdonsága;
- ha egy API-ban a allokációmentességet garantálni akarod, azt a saját típusodban kell megépítened, vagy `std::pmr::monotonic_buffer_resource`-szal (pmr, memory_resource és region lifetime (37.2)) kell a memóriát előre biztosítanod.

A mérés helyes módja nem a `sizeof` bámulása, hanem a tényleges allokációk számlálása: cseréld le a globális `operator new`-ot egy számlálóra a teszt binárisban, és mérd, hány hívás történik egy adott műveletsor alatt. Az eredmény target- és stdlib-verziófüggő, ezért a laborprofil mellé kell rögzíteni.

37.3.6 Alignment, false sharing és per-thread állapot

A `false sharing` akkor lép fel, amikor két szál két különböző változót ír, de azok ugyanazon a cache-line-on vannak: a hardver koherencia-protokollja a line-t ide-oda mozgatja, noha nincs valódi adatmegosztás. A javítás elválasztás, nem szinkronizáció:

```
struct Counters {
    alignas(64) std::atomic<long> a{0};
    alignas(64) std::atomic<long> b{0};
};
```

A C++17-es `std::hardware_destructive_interference_size` kifejezi ezt a szándékot, de az értéke implementációfüggő, és publikus headerben használva ODR- és ABI-kockázatot jelent, ha két fordítási egység eltérő értéket lát — ezért ABI-stabil határon inkább explicit, dokumentált konstansot használj.

A `thread_local` változó szálanként külön példány, ami természetesen megszünteti a false sharinget, de a hozzáféréshez TLS-modell kell (initial-exec vagy general-dynamic), ami plusz indirekció lehet, különösen megosztott könyvtárból. A dinamikus inicializálású `thread_local` objektumok élettartama a szálhoz kötött (Statikus és thread-local objektumok életciklusa (20.5)) — ez a döntés tehát élettartam-kérdés is, nem csak teljesítménykérdés.

37.3.7 Kidolgozott példa: layout-próba és locality-kapu

Az alábbi program a fenti állításokat teszi ellenőrizhetővé. Nem állítom, hogy lefuttattam: a megadott kimenet illusztratív, a tanuló laborjában mérendő.

```
// layout_probe.cpp
// build: g++ -std=c++20 -O2 -Wall -Wextra layout_probe.cpp -o layout_probe
//        clang++ -std=c++20 -O2 -Wall -Wextra layout_probe.cpp -o layout_probe
#include <cstddef>
#include <cstdio>
#include <functional>
#include <string>
#include <vector>

struct Empty {};
struct Order1 { char c; int i; double d; };
struct Order2 { int i; double d; char c; };
struct AsMember   { Empty e; int x; };
struct AsBase     : Empty { int x; };
struct AsNoUnique { [[no_unique_address]] Empty e; int x; };
struct Base { virtual ~Base() = default; };
struct DerivedV : Base { int x; };

int main() {
    std::printf("sizeof(Empty)                 = %zu\n", sizeof(Empty));
    std::printf("sizeof(Order1)                = %zu\n", sizeof(Order1));
    std::printf("sizeof(Order2)                = %zu\n", sizeof(Order2));
    std::printf("offsetof(Order1, d)           = %zu\n", offsetof(Order1, d));
    std::printf("sizeof(AsMember)              = %zu\n", sizeof(AsMember));
    std::printf("sizeof(AsBase)                = %zu\n", sizeof(AsBase));
    std::printf("sizeof(AsNoUnique)            = %zu\n", sizeof(AsNoUnique));
    std::printf("sizeof(Base)                  = %zu\n", sizeof(Base));
    std::printf("sizeof(DerivedV)              = %zu\n", sizeof(DerivedV));
    std::printf("sizeof(std::function<void()>) = %zu\n",
     ↪  sizeof(std::function<void()>));
    std::printf("sizeof(std::string)           = %zu\n", sizeof(std::string));
```

```
    std::printf("sizeof(std::vector<int>)       = %zu\n",
     ↪  sizeof(std::vector<int>));
}
```

Várható, illusztratív kimenet x86-64 LP64-en, GCC/Clang + libstdc++ mellett: `Empty=1`, `Order1=16`, `Order2=24`, `offset d=8`, `AsMember=8`, `AsBase=4`, `AsNoUnique=4`, `Base=8`, `DerivedV=16`, `function=32`, `string=32`, `vector=24`. Minden eltérés azt jelzi, hogy más ABI-n vagy más standard library-n fut a mérés — ami önmagában is tanulságos.

A locality-kapu egy összehasonlító mérés, amelynek elsődleges követelménye nem a gyorsaság, hanem hogy a két elrendezés *ugyanazt* számolja:

```
// locality_gate.cpp
// build: g++ -std=c++20 -O2 -Wall -Wextra locality_gate.cpp -o locality_gate
#include <chrono>
#include <cmath>
#include <cstdio>
#include <vector>

constexpr std::size_t N = 1u << 20;

struct AoS { float x, y, z, vx, vy, vz; };
struct SoA {
    std::vector<float> x, y, z, vx, vy, vz;
    explicit SoA(std::size_t n) : x(n), y(n), z(n), vx(n), vy(n), vz(n) {}
};

void step_aos(std::vector<AoS>& p, float dt) {
    for (auto& q : p) { q.x += q.vx * dt; q.y += q.vy * dt; q.z += q.vz * dt; }
}

void step_soa(SoA& s, float dt) {
    for (std::size_t i = 0; i < s.x.size(); ++i) {
        s.x[i] += s.vx[i] * dt;
        s.y[i] += s.vy[i] * dt;
        s.z[i] += s.vz[i] * dt;
    }
}

double checksum_aos(const std::vector<AoS>& p) {
    double s = 0.0;
    for (const auto& q : p) s += q.x + q.y + q.z;
    return s;
}

double checksum_soa(const SoA& s) {
    double acc = 0.0;
    for (std::size_t i = 0; i < s.x.size(); ++i) acc += s.x[i] + s.y[i] +
     ↪  s.z[i];
    return acc;
}

int main() {
```

```
    std::vector<AoS> aos(N, AoS{0.f, 0.f, 0.f, 1.f, 2.f, 3.f});
    SoA soa(N);
    for (std::size_t i = 0; i < N; ++i) {
        soa.x[i] = 0.f; soa.y[i] = 0.f; soa.z[i] = 0.f;
        soa.vx[i] = 1.f; soa.vy[i] = 2.f; soa.vz[i] = 3.f;
    }

    const float dt = 0.01f;
    const auto t0 = std::chrono::steady_clock::now();
    for (int k = 0; k < 100; ++k) step_aos(aos, dt);
    const auto t1 = std::chrono::steady_clock::now();
    for (int k = 0; k < 100; ++k) step_soa(soa, dt);
    const auto t2 = std::chrono::steady_clock::now();

    const double ca = checksum_aos(aos);
    const double cs = checksum_soa(soa);

    std::printf("aos  = %.6f\n", ca);
    std::printf("soa  = %.6f\n", cs);
    std::printf("diff = %.6e\n", std::fabs(ca - cs));
    std::printf("aos  ms = %.3f\n",
        std::chrono::duration<double, std::milli>(t1 - t0).count());
    std::printf("soa  ms = %.3f\n",
        std::chrono::duration<double, std::milli>(t2 - t1).count());
}
```

A kapu feltétele: a `diff` a lebegőpontos összegzési sorrend miatt nem nulla, de kicsi; a helyes ellenőrzés relatív tolerancia (`diff / max(|aos|, 1.0) < 1e-5`), nem bitazonosság. Csak ezután szabad az időket összevetni — és akkor is tudni kell, hogy a két ciklus nem feltétlenül azonos mértékben vektorizálódott. A mért idő önmagában nem bizonyíték semmire, ha a kapu nem teljesül.

37.3.8 Tipikus hibák és javításuk

1. **`sizeof` egyenlőnek feltételezése a tagok összegével.** A padding miatt nem az; a helyes kiindulás az alignment round-up képlet és a `offsetof`/`sizeof` próba.
2. **EBO-ra épített API-szerződés.** Az EBO és a `[[no_unique_address]]` is implementációs szinten érvényesül; stabil ABI-határon mérd és dokumentáld, ne feltételezd.
3. **Azonos típusú `[[no_unique_address]]` tagok átfedésének feltételezése.** A szabvány különböző címeket követel, tehát nem fedhetnek át; a tömörítés elmarad.
4. **`std::function` allokációmentességének feltételezése.** A puffer mérete implementációfüggő; allokációszámlálóval ellenőrizd.
5. **`std::hardware_destructive_interference_size` használata publikus headerben.** Két TU eltérő értéket láthat, ami ODR-problémát okoz.
6. **AoS→SoA refaktor numerikus következmény nélkül.** Az összegzési sorrend megváltozik; ha a determinizmus követelmény, a refaktor viselkedésváltozás, nem csak optimalizálás.

37.3.9 Összefoglaló

A C++ objektumlayout két rétegből áll: a szabvány szemantikai minimumából (nem nulla méret, megkülönböztethető címek, standard-layout és trivially copyable traitek) és az ABI konkrét

szabályaiból (offsetek, padding, vptr, EBO alkalmazása). Az optimalizálás ezért mindig mérésből indul: layout-próbával ellenőrizzük a méreteket, allokációszámlálóval a rejtett heap-használatot, és correctness-kapuval a viselkedés azonosságát. Az EBO, a `[[no_unique_address]]`, az SSO, a vptr és a small buffer mind implementációs részlet: lehet rájuk támaszkodni az adott laborprofilban, de nem lehet rájuk szerződést építeni. Az AoS/SoA döntés nem univerzális győztes, hanem a forró ciklus hozzáférési mintájából levezetett választás.

37.3.10 Feladatok

1. feladat (magyarázó). Számítsd ki kézzel a következő típus offsetjeit, `sizeof`-ját és igazítását x86-64 LP64-en: `struct Rec { bool active; double value; int count; char tag; };`. Majd rendezd át a tagokat, hogy a méret minimális legyen, és indokold az eredményt a round-up képlettel.

2. feladat (kódoló). Írj egy programot, amely eldönti egy adott callable-ről, hogy belefér-e a `std::function` kis pufferébe, allokációszámláló globális `operator new` segítségével. Mérd meg három különböző méretű lambdára, és rögzítsd a laborprofilban a `sizeof(std::function<...>)` értékét is.

3. feladat (diagnosztikai). Adott egy négyzetrácsos szimuláció `std::vector<Cell>` tárolóval, ahol egy szál a sejtek `energy` mezőjét, egy másik a `temperature` mezőjét frissíti, és a futásidő a szálszám növelésével nem javul. Fogalmazz meg két lehetséges okot, és adj mindkettőre javítást úgy, hogy a helyességi feltétel (a két mező frissítése között nincs adatverseny) megmaradjon.

37.3.11 Megoldások

1. megoldás. Az `active` offsetje 0, mérete 1. A `value` igazítása 8, ezért $o = \text{round_up}(0+1, 8) = 8$; a `value` a 8–15. bájtot foglalja. A `count` igazítása 4, $o = \text{round_up}(16, 4) = 16$. A `tag` a 20. bájtra kerül, a szerkezet vége 21, a szerkezet igazítása $\max(1, 8, 4, 1) = 8$, ezért sizeof $= \text{round_up}(21, 8) = 24$. Átrendezve `{ double value; int count; char tag; bool active; }`: `value` 0–7, `count` 8–11, `tag` 12, `active` 13, vége 14, $\text{round_up}(14, 8) = 16$. Ellenőrzési feltétel: a `sizeof` 24-ről 16-ra csökken, miközben a tagok halmaza változatlan; a program `offsetof`-fal igazolja az offseteket.

2. megoldás. A globális `operator new(std::size_t)` és `operator delete(void*)` felülírása egy atomikus számlálóval (a globális változatot a program elején, a standard header-ek include-olása után definiáljuk). A mérés: a számlálót nullázzuk, létrehozunk egy `std::function<void()>`-t a vizsgált lambdából, majd kiolvassuk a számlálót. A `std::function` konstrukciója allokálhat, ha a callable nem fér a pufferbe; a másolás is allokálhat. Értelmezés: ha a számláló 0, a callable belefért; ha 1, heap-re került. Ellenőrzési feltétel: a mért `sizeof(std::function<void()>)` és a küszöbérték együtt konzisztens a számlálóval — ha nem, a mérés hibás (pl. a lambda zárójelben más objektumot is másol). Korlát: az eredmény target- és stdlib-verziófüggő, ezért a laborprofil része.

3. megoldás. Első ok: a `Cell` objektumok mindkét mezője ugyanabban a cache-line-ban van, tehát a két szál írása false sharinget okoz — a koherencia-protokoll pingpongoztatja a line-t. Javítás: a két mezőt külön tömbre bontjuk (SoA-jellegű felosztás), és a két tömböt `alignas`-szal cache-line határra igazítjuk. Második ok: a `std::vector<Cell>` átméretezése vagy a `Cell` nagy mérete miatt a bejárás memóriasávszélesség-korlátos, és a szálak ugyanazokat a line-okat olvassák — ez olvasási megosztás, ami nem adatverseny, de nem skálázódik. Javítás: particionálás tartományokra úgy, hogy minden szál külön, nem átfedő szeleten dolgozzon, vagy a mezők szétválasztása mellett blokkolt bejárás. Mindkét javításnál az ellenőrzési feltétel: a két mező frissítése továbbra is

elkülönül (nincs szinkronizálatlan írás ugyanarra az objektumra), és a numerikus eredmény a toleranción belül azonos a referencia-implementációval.

37.4 Profilozás, benchmark és cost model

A helyesség bizonyítása után a teljesítmény az a terület, ahol a legkönnyebb önmagunkat becsapni. Ez az alfejezet ezért nem optimalizációs trükköket ad, hanem azt a fegyelmet, amellyel egy állítás – „ez a változat gyorsabb” – mérhetővé, reprodukálhatóvá és visszavezethetővé tehető műveleti és erőforrás-költségre. Kiindulópontunk az Object layout, EBO és adatközpontú tervezés (37.3) objektumelrendezése: a layout dönti el, hány cache vonalat érint egy bejárás, és ez az, amit a mérésnek látnia kell. Az ABI- és optimalizálásdiagnosztikai labor (14.6) ABI- és optimalizálásdiagnosztikai labor adja a fordító viselkedésének hátterét, a Tárolóválasztási és invalidációs labor (28.6) tárolóválasztási labor pedig azt a kérdést, amelyre a profilozás visszavezet: melyik művelet hova került.

37.4.1 A mérés mint szerződés: benchmark, profilozás, cost model

Három fogalmat érdemes élesen szétválasztani, mert a gyakorlatban összemosódnak. A **benchmark** rögzített workloadon végzett, ismételhető mérés, amelynek célja egy eldöntendő összehasonlítás: A változat gyorsabb-e B-nél azon a profilon, amely számunkra releváns. A **profilozás** (profiling) futás közbeni megfigyelés, amelynek célja nem az ítélet, hanem a magyarázat: hol megy el az idő és az erőforrás. A **cost model** (költségmodell) pedig az a leképezés, amely egy forrásszintű szerkezetet – egy ciklus, egy allokáció, egy virtuális hívás, egy syscall – közelítő költséghez rendel. A cost model nem mérési eredmény, hanem előrejelzés, amelyet a mérésnek kell igazolnia vagy cáfolnia.

Egy benchmark állítás csak akkor érvényes, ha a következő feltételek mind rögzítettek: (1) a **workload** pontos összetétele, (2) a **build** – compiler és verziója, `-std`, optimalizálási szint, target, `-DNDEBUG` megléte, (3) a **futtatási környezet** – CPU típus, frekvencia-governor, konténer CPU-limitje, a hoszt terheltsége, (4) a **mért szakasz határa** – beleszámít-e az inicializálás, a teardown, a fájl megnyitása, (5) az ismétlésszám és a **statisztika** módja. Ha bármelyik hiányzik, az eredmény nem hamis, hanem értelmezhetetlen: nem tudjuk, mire vonatkozik.

Fontos korlát: nincs univerzális benchmark. Egy `std::vector` versus `std::list` mérés csak a konkrét elemszámra, elemtípusra, bejárási mintára és cache-méretre érvényes. Ezért a mérési jegyzőkönyv mindig a profilt rögzíti, nem általános következtetést.

37.4.2 Workload, inputeloszlás és warmup

A workload az a bemenet, amelyen a program viselkedése releváns. Egy rekordfeldolgozó pipeline esetében legalább négy paramétert kell rögzíteni: a rekordok száma, a rekordhossz eloszlása (nem csak az átlag!), az egyedi kulcsok száma (kardinalitás), és a bemenet rendezettsége. Ezek külön-külön mozgatják a költséget: a nagy kardinalitás hash-táblát növel, a hosszú rekordok allokációt és memóriahierarchia-járást okoznak, a rendezett bemenet kihasználhatja a predikciót és a localityt.

A **warmup** (bemelegítés) első iterációinak költsége szisztematikusan eltér a többiétől: laphibák a lazán lefoglalt memórián, az allokátor belső arénáinak kialakulása, a dinamikus loader lazy bindingja, a CPU frekvencia felrámpázása. A warmupot a mért szakaszon **kívül** kell futtatni, és a mért szakasznak ugyanazt a bemenetet kell kapnia. Ha a warmupot csak „az első pár iteráció

eldobásával” kezeljük, akkor is rögzíteni kell, hány iterációt dobtunk el, mert ez a szám befolyásolja az eredményt.

37.4.3 A fordító kicselezése: holt kód, konstans folding, mérési kontroll

Az optimalizáló az `as-if` szabály szerint mindent megtehet, ami a megfigyelhető viselkedést nem változtatja. Ha egy számítás eredményét senki nem használja, a hívás eltűnik (**dead-code elimination**); ha a bemenet fordítási időben ismert, a számítás előre elvégezhető (**constant folding**), és a ciklus összeomolhat. Ilyenkor a mérés nem azt méri, amit hiszünk.

A megoldás nem a `volatile` – az megakadályozza az optimalizálást, tehát épp azt a kódot méri, amit a valóságban nem futtatunk –, hanem egy **mérési gát** (optimization barrier), amely a fordító számára láthatatlanná teszi, hogy az érték felhasználatlan, a hardver számára viszont nem generál műveletet. GCC/Clang alatt ez egy üres inline assembly blokk:

```
template <class T>
inline void keep(T const& v) {
#if defined(__GNUC__) || defined(__clang__)
    asm volatile("" : : "r,m"(v) : "memory");
#else
    volatile T sink = v; (void)sink;   // hordozható, de konzervatívabb tartalék
#endif
}
```

Ez **fordítóbővítmény**, nem ISO C++; a `#else` ág a hordozható visszaesés. A `keep()` hívása önmagában nem garantál semmit, ha a hívó kód más módon is kiejthető – a gátat a cikluson kívül, az eredményre kell alkalmazni. Ugyanez a probléma az allokációs számlálónál: ha a mért kód egy `std::string`-et hoz létre, de az SSO (small string optimization) miatt rövid szövegnél nem történik allokáció, akkor a számláló nulla lesz – ez nem hiba, hanem információ, de csak akkor, ha a rekordhossz-eloszlás ismert.

37.4.4 A költségek szétválasztása: CPU, allokáció, memória, syscall

A hasznos mérés nem egy számot ad, hanem dimenziókat bont fel, mert a beavatkozások különböző dimenziókra hatnak. Egy rekordfeldolgozó pipeline esetében ez a következő tábla:

Dimenzió	Mérték	Forrás	Mit magyaráz
CPU user	s	`getrusage ru_utime`	tényleges számítás
CPU sys	s	`ru_stime`	kernel-átlépések, syscallok
wall	s	`std::chrono::steady_clock`	teljes átfutás
allokáció	db, byte	`operator new` számláló	memóriakezelési költség
csúcs RSS	kB	`ru_maxrss`	memórialábnyom
syscall	db	`strace -c`	I/O és kernel-határ

A `wall - user - sys` különbség közelíti a **várakozást**: alvás, I/O-ra várás, futásra várakozás. Ha ez a szám nagy, a szűk keresztmetszet nem a CPU. A `user/sys` arány pedig megmutatja, hogy a költség a felhasználói kódban vagy a kernelben keletkezik – utóbbi tipikusan sok kis `read`/`write` vagy `gettimeofday`-jellegű hívás jele.

37.4.5 Eszközök és jogosultságok: perf és callgrind

A `perf` Linux-oldali, hardveres PMU-számlálókat és szoftveres eseményeket használ; `perf stat` összesít, `perf record` mintát vesz (sampling), és a mintákból attribútumot rendel a szimbólumokhoz. Működése **jogosultságfüggő**: a `kernel.perf_event_paranoid` sysctl, a konténer seccomp-profilja, a `CAP_PERFMON`/`CAP_SYS_ADMIN` megléte és a virtualizált PMU mind befolyásolják, hogy egyáltalán kapunk-e számlálót. Konténerben gyakran csak szoftveres események (`task-clock`, `page-faults`) érhetők el. Ha a számláló nem elérhető, azt kell jelenteni – adatot kitalálni nem lehet.

A `callgrind` a Valgrind dinamikus bináris instrumentációján alapul, ezért **determinisztikus utasításszámot** ad, ami kiválóan alkalmas két algoritmus összehasonlítására zajos gépen is. Cserébe nagyságrendekkel lassabb, és nem valós időt mér, hanem modellt. Mindkét eszköz használatát a laborprofilban rögzíteni kell (verzió, jogosultság, konfiguráció), mert a képességek verzió- és környezetfüggők.

37.4.6 Várakozás elkülönítése: syscall, IO, CPU

A wall-CPU szétválasztás önmagában még nem mondja meg, *melyik* syscall a bűnös. Erre a `strace -c -f` ad számlálókat hívásonként és időnként (az idő-kiemelés maga is torzít, ezt jelezni kell). A tipikus mintázat: sok kis `read`/`recv` hívás – pufferelési hiba; sok `futex` – szálankénti versengés; sok laphiba (`-R` opcióval mérve) – nem előre lefoglalt memória, vagy `mmap`/növekedő vektor.

37.4.7 Latency-eloszlás, áteresztés és a CI benchmark zaja

Az átlag önmagában félrevezet. A `pipeline`-oknál a farok számít: az a kérés, amelyik a p99-be esik, a felhasználó számára a lassú kérés. Ezért minden mérésnél legalább mediánt és p95/p99-et kell rögzíteni, ideális esetben hisztogramot. Az **áteresztés** (`N/T`) és a **latency** különböző kérdés: a batch feldolgozás áteresztésre optimalizál, az interaktív szolgáltatás farok-latencyre.

A CI-ban a zaj nagy: megosztott runner, más konténerek, változó CPU-frekvencia, hipervizitor-zaj. A védekezés nem a varázsszám, hanem a mérés-tervezés: abszolút érték helyett **arány** a baseline-hoz képest, ugyanabban a futásban mérve a két változatot (A/B interleaving), elegendő ismétlés, medián az ismétlések között, és a szórás (MAD) jelentése. Ha a különbség kisebb, mint a zaj, a helyes következtetés: „nem mutatható ki hatás”.

37.4.8 Kidolgozott példa: rekordfeldolgozó pipeline mérése

A példa egy `key,value\n` sorokból álló blobból számol összeget. A tulajdonos a `main`-ben élő `blob`; a `process` `std::string_view`-ként **kölcsönzi** a szeletet, és nem tárolja tovább, így nincs dangling (`blob` a `reserve` után nem allokál újra). Az allokációk a `parse_line`-ban keletkeznek.

```
// pipeline_bench.cpp
// g++     -std=c++20 -O2 -DNDEBUG pipeline_bench.cpp -o pipeline_bench
// clang++ -std=c++20 -O2 -DNDEBUG pipeline_bench.cpp -o pipeline_bench
#include <atomic>
#include <chrono>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
```

```
#include <new>
#include <string>
#include <string_view>
#include <sys/resource.h>

static std::atomic<std::size_t> g_allocs{0};

void* operator new(std::size_t n) {
    g_allocs.fetch_add(1, std::memory_order_relaxed);
    if (void* p = std::malloc(n)) return p;
    throw std::bad_alloc{};
}
void operator delete(void* p) noexcept { std::free(p); }
void operator delete(void* p, std::size_t) noexcept { std::free(p); }

template <class T>
inline void keep(T const& v) {
#if defined(__GNUC__) || defined(__clang__)
    asm volatile("" : : "r,m"(v) : "memory");
#else
    volatile T sink = v; (void)sink;
#endif
}

struct Record { std::string_view key; std::int64_t value; };
struct Stats  { std::int64_t sum = 0; std::size_t n = 0; };

static bool parse_line(std::string_view line, Record& out) {
    auto comma = line.find(',');
    if (comma == std::string_view::npos) return false;
    out.key = line.substr(0, comma);
    std::string tail{line.substr(comma + 1)};   // itt allokálunk (SSO-tól
     ↪  függően nem)
    char* end = nullptr;
    long long v = std::strtoll(tail.c_str(), &end, 10);
    if (end == tail.c_str()) return false;
    out.value = v;
    return true;
}

static Stats process(std::string_view blob) {
    Stats s{};
    std::size_t pos = 0;
    while (pos < blob.size()) {
        auto nl = blob.find('\n', pos);
        if (nl == std::string_view::npos) nl = blob.size();
        Record r{};
        if (parse_line(blob.substr(pos, nl - pos), r)) { s.sum += r.value;
         ↪  ++s.n; }
        pos = (nl == blob.size()) ? blob.size() : nl + 1;
    }
    return s;
```

```
}

int main(int argc, char** argv) {
    const int iters = (argc > 1) ? std::atoi(argv[1]) : 200;

    std::string blob;
    blob.reserve(1u << 20);
    char buf[64];
    for (int i = 0; i < 20000; ++i) {
        int n = std::snprintf(buf, sizeof buf, "key%03d,%d\n", i % 512, i);
        blob.append(buf, static_cast<std::size_t>(n));
    }
    keep(blob.size());

    Stats warm{};
    for (int i = 0; i < 20; ++i) warm = process(blob);
    keep(warm.sum);

    struct rusage r0{}, r1{};
    getrusage(RUSAGE_SELF, &r0);
    const auto t0 = std::chrono::steady_clock::now();
    const auto a0 = g_allocs.load(std::memory_order_relaxed);

    std::int64_t acc = 0;
    for (int i = 0; i < iters; ++i) acc += process(blob).sum;
    keep(acc);

    const auto t1 = std::chrono::steady_clock::now();
    const auto a1 = g_allocs.load(std::memory_order_relaxed);
    getrusage(RUSAGE_SELF, &r1);

    auto dt = [](timeval a, timeval b) {
        return double(b.tv_sec - a.tv_sec) + double(b.tv_usec - a.tv_usec) *
         ↪  1e-6;
    };
    const double wall = std::chrono::duration<double>(t1 - t0).count();
    const double user = dt(r0.ru_utime, r1.ru_utime);
    const double sys  = dt(r0.ru_stime, r1.ru_stime);
    const double recs = double(warm.n) * iters;

    std::printf("iterations    = %d\n", iters);
    std::printf("records       = %.0f\n", recs);
    std::printf("wall_s        = %.6f\n", wall);
    std::printf("user_s        = %.6f\n", user);
    std::printf("sys_s         = %.6f\n", sys);
    std::printf("wait_s        = %.6f\n", wall - user - sys);
    std::printf("allocs        = %zu\n", a1 - a0);
    std::printf("allocs/record = %.4f\n", double(a1 - a0) / recs);
    std::printf("ns/record     = %.2f\n", wall * 1e9 / recs);
    std::printf("maxrss_kb     = %ld\n", r1.ru_maxrss);
    return 0;
}
```

A kiegészítő dimenziók külön parancsokkal mérhetők; mindegyik jogosultságfüggő, a laborprofilban rögzítendő:

```
./pipeline_bench 200
perf stat -d ./pipeline_bench 200
strace -c -f ./pipeline_bench 200
valgrind --tool=callgrind ./pipeline_bench 20
```

Az értelmezés iránya: ha az `allocs/record` értéke tartósan nagy, a `parse_line` allokációja a jelölt, és a javítás (allokációmentes parse) ebben a dimenzióban látszik majd; ha a `sys_s` arány nő, a kernel-határ felé kell nézni; ha a `wait_s` nagy, a CPU nem a szűk keresztmetszet. **A fenti kód kimenete itt nem szerepel mért értékként** – a számokat a saját laborodban kell megkapnod, és a jegyzőkönyvbe a build- és környezeti profil mellé kell tenni.

37.4.9 Tipikus hibák és javításuk

A leggyakoribb hiba az `-O0`-n mért benchmark: az nem a kiadott binárist méri. A második a `std::chrono::system_clock` használata `steady_clock` helyett – az előbbi ugorhat. A harmadik a `parse_line` allokációjának elfelejtése az elemzésben, mert „az `std::string_view` nem allokál” – igaz, de a `std::string` igen. A negyedik a `operator new` számláló túlbecslése: a libstdc++/libc++ belső allokációit is számolja, és az over-aligned `operator new` túlterhelése nélkül a nagy igazítású allokációk kimaradnak. Az ötödik a konténer CPU-limitje mellett abszolút másodpercek összehasonlítása két különböző futásból.

37.4.10 Összefoglaló

A teljesítmény-állítás szerződés: rögzített workload, rögzített build, rögzített környezet, rögzített mért szakasz, megismételt mérés. A benchmark ítélet, a profilozás magyarázat, a cost model előrejelzés – és a három csak együtt érvényes. A költséget dimenziókra kell bontani (CPU user/sys, wall, allokáció, RSS, syscall), mert a beavatkozások különböző dimenziókba hatnak. A fordítót mérési gáttal kell távol tartani, az eszközök jogosultságfüggését jelenteni kell, a farok-latencyet pedig nem szabad átlaggal helyettesíteni.

37.4.11 Feladatok

1. feladat (kódolási). Írd át a `parse_line`-t allokációmentesre (a számot közvetlenül a `std::string_view`-ből olvasva), és mérd meg a különbséget a `pipeline_bench` harness-szel. Mely dimenziók változnak, és melyek nem?

2. feladat (magyarázó–diagnosztikai). Egészítsd ki a harness-t úgy, hogy iterációnként mérje a wall time-ot, és számoljon mediánt, p95-öt és p99-et. Indokold, miért lehet a medián és az átlag lényegesen eltérő, és milyen esetben hazudik az átlag.

3. feladat (mérés-tervezési). Készíts CI-barát összehasonlítást `-O2` és `-O2 -march=native` között. Add meg a mérési tervet: hány ismétlés, milyen statisztika, hogyan kezeled a runner zaját, és mit írsz a jegyzőkönyvbe, ha a hatás a zaj alatt marad.

37.4.12 Megoldások

1. megoldás. Az allokációmentes változat a számjegyeket közvetlenül a nézetből dolgozza fel:

```
static bool parse_line(std::string_view line, Record& out) {
    auto comma = line.find(',');
    if (comma == std::string_view::npos) return false;
    out.key = line.substr(0, comma);
    auto num = line.substr(comma + 1);
    if (num.empty()) return false;
    std::int64_t v = 0;
    for (char c : num) {
        if (c < '0' || c > '9') return false;
        v = v * 10 + (c - '0');
    }
    out.value = v;
    return true;
}
```

Indoklás: a `std::string tail{...}` konstrukció a rövid rekordoknál SSO miatt egyáltalán nem allokál, hosszúaknál igen. Ezért az `allocs/record` csökkenése **függ a rekordhossz-eloszlástól** – ha a bemenet végig rövid, a javulás kicsi vagy nulla, ami nem azt jelenti, hogy a változtatás rossz, csak azt, hogy ezen a profilon nem mérhető. Ellenőrzési feltétel: `allocs/record` a hosszú rekordos bemeneten csökken, a `sum` és a feldolgozott rekordszám viszont **azonos** marad a két változatban (helyességi kapu), és a `wait_s` nem nő. A `ns/record` csak ezek után értelmezhető.

2. megoldás. Iterációnkénti minta vételéhez a ciklus belsejébe kell az időmérés, `steady_clock`-kal:

```
std::vector<double> samples;
samples.reserve(static_cast<std::size_t>(iters));
for (int i = 0; i < iters; ++i) {
    const auto a = std::chrono::steady_clock::now();
    acc += process(blob).sum;
    const auto b = std::chrono::steady_clock::now();
    samples.push_back(std::chrono::duration<double>(b - a).count());
}
keep(acc);
std::sort(samples.begin(), samples.end());
auto q = [&](double p) {
    std::size_t i = static_cast<std::size_t>(p * double(samples.size() - 1));
    return samples[i];
};
std::printf("p50=%.6f p95=%.6f p99=%.6f max=%.6f\n",
            q(0.50), q(0.95), q(0.99), samples.back());
```

Indoklás: az átlag érzékeny a kiugró értékekre, és a farokról semmit nem mond. Ha a futásidő eloszlása jobbra ferdült (néhány kiugróan lassú iteráció, például laphiba vagy allokátor-újranövelés miatt), az átlag felfelé húz, miközben a medián stabil. Az átlag akkor hazudik igazán, amikor a döntést „a tipikus esetre” akarjuk hozni, de a felhasználói élményt a farok határozza meg. Ellenőrzési feltétel: `p50` `p95` `p99` `max`, és az `iters` növelésével a medián konvergál, a p99 pedig nem esik a medián alá. Megjegyzés: az iterációnkénti `now()` hívás maga is költség, ezért a rekordonkénti ns itt már nem összevethető az 1. feladat teljes-futásos mérésével – ez tudatos trade-off, nem hiba.

3. megoldás. A terv a következő. Egy CI-futáson belül **mindkét** binárist lefordítjuk ugyanazzal a compilerrel és ugyanazzal a stdlib-verzióval, majd **váltogatva** futtatjuk őket: A, B, A, B, … Ez kiszűri a lassú driftet (termikus vagy szomszéd-konténer hatás). Ismétlés: legalább 15 váltott pár, mindegyikből a medián `ns/record` értéket vesszük, és a páronkénti arányok mediánját jelentjük. Statisztika: a két medián aránya, mellé a páronkénti arányok MAD-ja. Jegyzőkönyv: compiler és verzió, `-std=c++20`, `-O2`, `-DNDEBUG`, `-march=native` pontos jelentése az adott targeten (ez implementációfüggő, a laborprofilban rögzítendő), a CPU-modell, a runner típusa, az ismétlésszám és a statisztika. Ha az arány mediánja a MAD sávján belül van, a helyes megfogalmazás: „a két build között ezen a profilon nem mutatható ki különbség"; ezt nem szabad „azonos teljesítmény"-ként megfogalmazni. `-march=native` esetén plusz kockázat, hogy a CI-runner CPU-ja eltér a deploy célétól, ezért a natív buildet a cél hardveren kell ellenőrizni, nem a runneren.

37.5 Template, virtual és std::function költsége

A dispatch költsége nem egyetlen szám, hanem egy szerződésfüggő modell eredménye. Ebben a részben ugyanazt a műveletet három megoldással — statikus template, virtuális függvény és `std::function`-alapú type erasure — hasonlítjuk össze, de nem azért, hogy egyik mellett univerzálisan érveljünk, hanem hogy megtanuljuk: mit kell megmérni, mit kell megnézni az assemblyben vagy az IR-ben, és milyen feltételek mellett lesz egy állítás egyáltalán igaz. Az előző részekben már rögzítettük a mérési keretet (Profilozás, benchmark és cost model (37.4)), a fordítás idejű testreszabás mintáit (Customization point, CRTP és policy design (27.5)) és a type erasure szerkezetét (invoke, function wrapper és type erasure (30.2)). Itt ezekre építünk, és a hangsúly a döntés indoklásán van: API-szerződés, fordítási költség, bináris méret és futásidejű viselkedés együtt.

A „zero-overhead" kifejezés a C++ egyik leggyakrabban félreértett ígérete. Helyes olvasata: bizonyos feltételek mellett a használt absztrakció nem jár olyan futásidejű többlettel, amit a kézi megoldás elkerülne. Ezek a feltételek azonban nem automatikusak. Ha a fordító nem látja a konkrét típust, ha a hívás modulhatáron megy át, ha a callable nem ismert a hívás helyén, vagy ha a `std::function` éppen allokál, akkor az absztrakciónak ára van. Ezt az árat nem tilos megfizetni — gyakran ez a helyes tervezési döntés —, de tudni kell, hogy mikor és mennyit fizetünk.

37.5.1 A dispatch három formája és szerződése

Dispatch alatt itt egy művelet konkrét implementációjának kiválasztását értjük egy hívási ponton. A három forma abban különbözik, hogy ez a kiválasztás mikor történik, és milyen információra van szükség hozzá.

Statikus (fordítás idejű) dispatch. A konkrét függvényt vagy függvényobjektumot a fordító a típus alapján választja ki; tipikus eszközei a függvénysablon, a CRTP, az overload-feloldás és az `if constexpr`. Feltétele, hogy a hívási ponton a típus ismert legyen, és a sablon definíciója látható legyen a fordítási egységben (vagy explicit instantiációval elérhető legyen). Nincs futásidejű indirekció, és a hívás jellemzően inline-olható. Ára a kódduzzadás és a fordítási idő: minden új típus új instantiációt jelent.

Virtuális (futásidejű) dispatch. A konkrét implementációt a dinamikus típus dönti el a vtable-en keresztül; a hívás pointeren vagy referencián keresztül történik. Feltétele, hogy az osztály rendelkezzen virtuális függvénnyel, a hívás pedig ne statikusan ismert típuson menjen át. Az objektum élettartamának érvényesnek kell lennie, törlésnél pedig a bázisnak virtuális destruktorral kell rendelkeznie. A költség egy indirekt hívás és a vptr betöltése; az inlining csak akkor lehetséges, ha a fordító devirtualizálni tud.

Type erasure. Egy egységes, futásidejű interfész mögé rejtjük a konkrét típust; a `std::function`, a `std::any` és a saját kézzel írt wrapperek tartoznak ide. A `std::function` esetén a hívás egy belső, típusra szabott invoker-függvényen keresztül megy, amelyre a wrapper egy függvénypointerrel vagy egy kis tárolóban mutat. Feltétel, hogy a tárolt callable másolható legyen (C++23-tól a `std::move_only_function` oldja ezt a korlátot; a használat előtt ellenőrizni kell a library támogatását). A költség az indirekt hívás, a lehetséges allokáció és a másolás.

Forma	Döntés ideje	Tipikus mechanizmus	Mit kell garantálni	Hol a költség
Statikus	fordítás	template, CRTP, overload	látható definíció, ismert típus	kódméret, fordítási idő
Virtuális	futás	vtable, indirekt hívás	érvényes objektum, virtuális destruktor	indirekció, inlining-korlát
Type erasure	futás	invoker-függvénypointer, SBO	másolhatóság, élettartam	indirekció, allokáció, másolás

37.5.2 Költségmodell: mi drága és mikor

A dispatch-választás nem attól jó, hogy az egyik forma „gyorsabb”, hanem hogy a költségkomponensek közül melyik dominál az adott terhelésen. Öt komponenst érdemes külön követni.

Hívási overhead. Egy indirekt hívás önmagában néhány ciklus, de a valódi költség a branch target prediction viselkedésében van. Ha a hívás célpontja kiszámítható (mindig ugyanaz a típus), a prediktor jól teljesít; ha váltakozik, a hibás előrejelzés jóval többe kerülhet, mint maga a hívás. Ezért a homogenitás fontos: egy heterogén kollekció elemein végigmenő virtuális hívás más költségprofil, mint egyetlen típus ismételt hívása.

Inlining lehetősége. Az inlining nem csak a hívás elhagyását jelenti: lehetővé teszi a konstanspropagációt, a közös rész kiemelését és a vektorizálást. A statikus forma ezt alapból megengedi; a virtuális forma csak devirtualizáció után; a type erasure jellemzően blokkolja.

Adatelérés és lokalitás. A vptr betöltése és a vtable bejárása plusz memóriahozzáférést jelent. Kis objektumoknál ez arányaiban nagyobb lehet, nagy, ritkán hívott műveleteknél elhanyagolható.

Allokáció. A type erasure allokálhat a wrapper létrehozásakor, ha a callable nem fér be a kis pufferbe (SBO). A kis puffer mérete implementációfüggő: a libstdc++ és a libc++ eltérő méretekkel dolgozhat, és a szabvány erre nem ad garanciát. A hívási ciklusban történő konstrukció ezért drága lehet, még akkor is, ha maga a hívás olcsó.

Kódméret és fordítási idő. Minden template-instantiation új kódot jelent, ami növeli a bináris méretet és az I-cache nyomást, valamint a fordítási időt. A linker ugyan összevonhatja az azonos instantiationöket (vague linkage, COMDAT), de ez nem garantált minden esetben, és a fordítási időt nem csökkenti.

A döntéshez tehát először becsülni kell a művelet egységköltségét. Ha egy elemre jutó munka nagyságrendekkel nagyobb a hívás költségénél, a dispatch forma megválasztása másodlagos. Ha a művelet triviális és milliószor fut, a hívási overhead és az inlining válik dominánssá. Ez a cost model lényege: nem a forma, hanem a domináns komponens alapján döntünk.

37.5.3 Kidolgozott példa: ugyanaz a redukció három formában

A példa egy vektoron végzett összegzést mutat be, ahol az elemenkénti műveletet `x*x`-re választjuk. A művelet szándékosan kicsi, hogy a dispatch költsége láthatóvá váljon; nagyobb műveletnél a különbség eltűnne.

```
// dispatch_demo.cpp
#include <cstddef>
#include <cstdio>
#include <functional>
#include <span>
#include <vector>

// 1) Statikus: sablon + függvényobjektum
struct square {
    double operator()(double x) const noexcept { return x * x; }
};

template <class F>
double reduce_static(std::span<const double> v, F f) {
    double acc = 0.0;
    for (double x : v) acc += f(x);
    return acc;
}

// 2) Virtuális: absztrakt interfész
struct transform {
    virtual ~transform() = default;
    virtual double apply(double) const = 0;
};

struct square_virtual final : transform {
    double apply(double x) const override { return x * x; }
};

double reduce_virtual(std::span<const double> v, const transform& t) {
    double acc = 0.0;
    for (double x : v) acc += t.apply(x);
    return acc;
}

// 3) Type erasure: std::function
double reduce_erased(std::span<const double> v,
                     const std::function<double(double)>& f) {
    double acc = 0.0;
    for (double x : v) acc += f(x);
    return acc;
}

int main() {
    std::vector<double> v(1'000'000, 1.0);
    std::printf("%f %f %f\n",
```

```
                reduce_static(v, square{}),
                reduce_virtual(v, square_virtual{}),
                reduce_erased(v, square{}));
}
```

A fordításhoz rögzítsd a fordító kiadását és a nyelvi módot; a példa C++20-at igényel a `std::span` miatt.

```
g++ -std=c++20 -O2 -Wall -Wextra -Wpedantic -o dispatch_demo dispatch_demo.cpp
```

A megfigyeléshez érdemes assemblyt is készíteni, és a ciklusokban keresni az indirekt hívás mintáját:

```
g++ -std=c++20 -O2 -S -masm=intel -o dispatch_demo.s dispatch_demo.cpp
grep -n "call" dispatch_demo.s
```

A várt minta — nem mért eredmény — a következő. A `reduce_static` esetében a `square::operator()` beleolvad a ciklusba, és a hívás eltűnik. A `reduce_virtual` belsejében a `t.apply(x)` indirekt hívás marad, kivéve ha a `reduce_virtual` maga is beinline-olódik a `main`-be, ahol a dinamikus típus statikusan ismert (`square_virtual`), és a fordító devirtualizálhatja. A `reduce_erased` esetében a `std::function operator()`-a egy invoker-függvénypointeren keresztül hív, amit a fordító jellemzően nem tud feloldani; a kis `square` callable valószínűleg az SBO-ba kerül, de ez implementációfüggő, és nem garantált.

A példa tanulsága nem az, hogy „a template gyorsabb”, hanem hogy a három forma assemblyje eltérő szerkezetű, és a különbség a konkrét hívási mintától függ. A `reduce_virtual` devirtualizációja például attól függ, hogy a függvény beinline-olódik-e a hívóba, ami viszont a méretétől és a fordító döntésétől függ.

37.5.4 Inlining és devirtualizáció: feltételek

A devirtualizáció nem varázslat, hanem feltételek teljesülése esetén végrehajtott transzformáció. A leggyakoribb esetek:

- **Ismert dinamikus típus a hívás helyén.** Ha a hívás egy konkrét objektumon történik, amelynek típusa statikusan ismert, a virtuális hívás közvetlen hívássá alakítható. Ez akkor működik, ha a hívó függvénybe beinline-olódik a hívás, vagy ha a fordító helyben látja az objektumot.
- **`final` osztály vagy `final` metódus.** A `final` kizárja a további felülírást, így a vtable-bejegyzés statikusan ismertté válik. Ez a legegyszerűbb, leginkább hordozható jelzés a fordítónak.
- **Névtelen névtér vagy belső linkage.** Ha az osztály nem érhető el más fordítási egységből, a fordító biztos lehet benne, hogy nincs más leszármazott.
- **Speculatív devirtualizáció.** A fordító feltételezheti, hogy a dinamikus típus a leggyakoribb, és egy típusellenőrzés mögé tett közvetlen hívással helyettesítheti a virtuális hívást. Ez a profilvezérelt optimalizálás (PGO) egyik tipikus eredménye.
- **Modulhatáron átnyúló ismeret.** LTO nélkül a fordító csak a saját fordítási egységét látja. Megosztott könyvtárban a szimbolikus interpoláció (symbol interposition) miatt a virtuális hívás devirtualizálása akadályba ütközhet; bizonyos fordítók `-fno-semantic-interposition` kapcsolóval engedik meg a belsejük felé irányuló optimalizálást. Ez fordító- és verziófüggő viselkedés, ezért a laborprofilban rögzíteni kell.

Fontos különbség: a devirtualizáció nem garantált, és nem is mindig előnyös. Ha a hívás helyén a dinamikus típus tényleg változékony, a spekulatív devirtualizáció egy plusz típusellenőrzést és egy elágazást épít be, ami rosszabb lehet, mint az eredeti indirekt hívás. Ezért a mérés nem helyettesíthető azzal a feltételezéssel, hogy „a `final` mindig gyorsít”.

37.5.5 LTO, PGO és a modulhatáron átnyúló optimalizálás

A **link-time optimization (LTO)** lényege, hogy a fordító a linkelés fázisában is látja a többi fordítási egységet, így a template-instantiationök, a virtuális hívások és a kis függvények átvihetők a modulhatáron. GCC-nél és Clangnél is elérhető (`-flto`), Clangnél ThinLTO formában is (`-flto=thin`). Az LTO költsége a linkelési idő és a memóriaigény növekedése, valamint az, hogy a build reprodukálhatósága és a diagnosztika bonyolultabb lehet.

A **profile-guided optimization (PGO)** a futásidejű profilt visszavezeti a fordításba: a gyakori elágazások, a gyakori virtuális célpontok és a gyakori inline-helyek súlyt kapnak. A tipikus munkafolyamat GCC/Clang alatt `-fprofile-generate` a mérő buildhez, majd `-fprofile-use` az optimalizált buildhez; Clangnél az instrumentált változat `-fprofile-instr-generate` / `-fprofile-instr-use`. Ezek a kapcsolók és a profilfájl formátuma verziófüggő, ezért a laborprofil részeként kell rögzíteni őket, és a build-reprodukálhatóságot külön ellenőrizni.

Mindkét technika közös jellemzője, hogy a nyereség nagysága erősen terhelésfüggő. Egy olyan programban, ahol a dispatch a teljes futásidő töredékét teszi ki, az LTO és a PGO elsősorban a kódméretre és a fordítási időre lesz hatással, nem a futásidőre. Ezért a döntés előtt a cost modelben meg kell becsülni, mekkora a dispatch részaránya.

37.5.6 Instantiation, kódduzzadás és fordítási idő

Minden template-instantiation egy külön függvényt vagy osztályt hoz létre a fordítási egységben. Ha ugyanazt a sablont több típussal használjuk, a kód mérete nő, és a fordítási idő is nő, mert a fejlécet minden fordítási egység újra feldolgozza és újrainstantiálja. A linker bizonyos esetekben összevonja az azonos instantiationöket (vague linkage, COMDAT), de ez nem garantálja, hogy a bináris méret csökken, és a fordítási időt sem érinti.

Kézi szabályozásra két eszköz áll rendelkezésre. Az **explicit instantiation** (`template double reduce_static<square>(std::span<const double>, square);`) egy fordítási egységben állítja elő a kódot. Az `extern template` deklaráció a többi fordítási egységben letiltja az implicit instantiationt, így azok a linker által biztosított példányt használják. Ez csökkenti a fordítási időt és a kódduzzadást, de csak akkor működik, ha az explicit instantiation pontosan egy helyen történik, és a használt típusok köre ismert.

A fordítási idő mérésére GCC-nél a `-ftime-report`, Clangnél a `-ftime-trace` használható; mindkettő verziófüggő, és a kimenetük eltérő részletességű. A méréskor ügyelni kell arra, hogy a gyorsítótár (ccache, inkrementális build) ne torzítsa az eredményt: a template-instantiation költségét tiszta builddel érdemes mérni.

37.5.7 noexcept, move és allokáció

A `noexcept` hatása kettős. Egyrészt a fordító a `noexcept` függvényeknél elhagyhatja a kivételkezelési keretrendszert (landing pad, unwinder), ami kisebb kódot és jobb optimalizálást eredményezhet. Másrészt a standard library viselkedését befolyásolja: a `std::vector` újraallokációkor a `move_if_noexcept` logikát használja, azaz csak akkor mozgat, ha a move constructor

`noexcept`, különben másol, hogy megőrizze az erős kivételbiztonságot. Egy nem `noexcept` move constructor tehát nem csak elméleti kockázat: a gyakorlatban másoláshoz vezethet.

A `std::function` másolása a tárolt callable másolását jelenti, ami allokálhat, ha a callable nem fér be az SBO-ba. Ezért a `std::function`-t hívási ciklusban érdemes `const&`-ként átadni, vagy egyáltalán elkerülni a wrapper létrehozását a cikluson belül. Mozgatásnál a `std::move` önmagában nem ad át erőforrást, csak kategóriakonverziót végez; a tényleges átadás a move constructorban történik, és a moved-from objektumnak továbbra is érvényes, destruálható állapotban kell maradnia.

A C++23-as `std::move_only_function` csak mozgatható callable-t tárol, így elkerülhető a másolás, de cserébe a wrapper nem másolható. Ez a döntés is API-szerződés kérdése: ha a felhasználónak másolható callback kell, akkor a `std::function` a helyes választás, és a másolás költségét tudatosan kell vállalni.

37.5.8 Assembly, IR és mérés: bizonyíték, nem recept

A dispatch-választás helyes bizonyítéka nem egyetlen futásidejű szám, hanem a kód szerkezetének és a mérésnek az együttese. Három szintet érdemes megkülönböztetni.

Forráskód-szintű érvelés. A cost model megmondja, melyik komponens dominálhat. Ez szükséges, de nem elégséges: a fordító döntéseit nem ismerjük előre.

Assembly vagy IR. Az assembly megmutatja, hogy a hívás megmaradt-e indirektként, történt-e inlining, van-e allokációs hívás a ciklusban. Az IR (LLVM IR, GCC GIMPLE) közelebb van az optimalizálási döntésekhez, de olvasása nagyobb rutint igényel. A `llvm-mca` statikus elemzést ad egy adott függvényről, de ez nem helyettesíti a teljes program mérését.

Futásidejű mérés. A `perf stat` ciklusszámot, utasításszámot és branch-miss arányt ad; a `perf record` megmutatja, hol töltődik az idő. A méréskor ügyelni kell a konstans-hajtogatásra, a holt kód eliminálására, a frekvencia-skálázásra és a gyorsítótár-melegítésre. A mikrob benchmark eredménye önmagában nem általánosítható.

A három szint együtt ad döntési alapot. Az a megállapítás, hogy „a virtuális hívás lassabb a templatenél”, önmagában nem bizonyíték; az a megállapítás viszont igen, hogy „ezen a terhelésen, ezzel a compilerrel, ezzel a flagszettel a virtuális változat ciklusszáma X%-kal magasabb, és az assemblyben a ciklusban indirekt hívás maradt, míg a statikus változatban nem”. Az utóbbi állítás reprodukálható, ellenőrizhető és a laborprofilhoz köthető.

37.5.9 Tipikus hibák és javításuk

- **„A template mindig gyorsabb.”** Nem: a template kódduzzadást és fordítási időt okoz, és ha a hívás nem inline-olódik, a különbség eltűnhet. Javítás: a cost modelben a teljes költséget nézni, nem csak a futásidőt.
- **Mérés `-O0`-val.** Az optimalizálatlan build nem reprezentálja a produkciós viselkedést. Javítás: a mérést a release flagszettel, rögzített compiler-verzióval végezni.
- **Hiányzó virtuális destruktor.** Ha a bázis-pointeren keresztül törlünk, a viselkedés nem definiált. Javítás: `virtual ~Base() = default;`.
- **`std::function` érték szerinti átadása hívási ciklusban.** Másolás és esetleges allokáció. Javítás: `const&`, vagy a wrapper létrehozása a cikluson kívül.
- **Az SBO méretének feltételezése.** A kis puffer mérete implementációfüggő. Javítás: mérés vagy dokumentált implementációs adat használata, nem feltételezés.

- **Devirtualizáció feltételezése `final` nélkül.** Javítás: a tényleges assembly ellenőrzése, és szükség esetén `final` vagy LTO/PGO használata.
- **Kódméret és fordítási idő figyelmen kívül hagyása.** Javítás: explicit instantiation, `extern template`, és a bináris méret rendszeres ellenőrzése.

37.5.10 Összefoglaló

A dispatch forma megválasztása három szerződés együttes mérlegelése: az API mit garantál (másolhatóság, élettartam, ABI-stabilitás), a fordítási folyamat mit enged (inlining, LTO, PGO), és a terhelés mit tesz dominánssá (hívási overhead, allokáció, kódméret). A statikus forma a legtöbb inlining-lehetőséget adja, de kódduzzadással és fordítási idővel fizet. A virtuális forma stabil ABI-t és futásidejű rugalmasságot ad, de indirekcióval és korlátozott devirtualizációval. A type erasure egységes interfészt ad, de allokációval, másolással és indirekcióval. Egyik sem univerzálisan jobb; a döntés akkor jó, ha a cost model, az assembly és a mérés együtt támasztja alá, és a választás a laborprofilhoz kötve reprodukálható.

37.5.11 Feladatok

1. **Cost model számítás.** Egy reduce művelet elemenként 20 ns számítási időt igényel. Becsüld meg, mekkora lehet a dispatch hozzájárulása, ha a hívás kiszámítható, illetve ha váltakozó célpontú. Milyen esetben döntenél a virtuális forma mellett? Indokold a becslést, és jelöld, mely számok illusztratívak.

2. **Kódolási feladat.** A fenti `dispatch_demo.cpp`-ben alakítsd át a `reduce_static`-et úgy, hogy a `square` mellett egy `cube` funktorral is használható legyen, és explicit instantiation-nel csökkentsd a kódduzzadást. Add meg a buildparancsot és azt, hogy mit keresnél az assemblyben.

3. **Diagnosztikai feladat.** Adott egy hívási ciklusban létrehozott `std::function`, amely egy nagy méretű, érték szerint elkapott objektumot tárol. Azonosítsd a lehetséges allokációt és a másolást, majd javítsd a kódot legalább két módon. Mindkét javításnál nevezd meg az élettartam-szerződést és az ellenőrzési feltételt.

37.5.12 Megoldások

1. megoldás. A 20 ns-os egységköltség kb. 60 ciklusnak felel meg 3 GHz-en (illusztratív számítás). Egy kiszámítható indirekt hívás néhány ciklus, tehát a hozzájárulás néhány százalék; ilyenkor a virtuális forma költsége elhanyagolható, és a döntést inkább az ABI-stabilitás és a karbantarthatóság vezérli. Váltakozó célpontú hívásnál a hibás branch-előrejelzés tipikusan tíz és húsz ciklus közötti nagyságrendű lehet, ami a 60 ciklus jelentős részét veheti el; ilyenkor a statikus forma vagy a homogén csoportosítás (pl. típus szerinti szétválogatás) indokolt lehet. A számok illusztratívak, nem mérési eredmények; a döntéshez a tényleges assembly és `perf stat` szükséges. Ellenőrzési feltétel: a ciklusban megjelenő indirekt hívás és a branch-miss arány együtt értelmezhető.

2. megoldás. Vedd fel a `cube` funktort a `square` mellé, és a fejlécben deklaráld a két explicit instantiationt:

```
// reduce.hpp
#pragma once
#include <span>
```

```
struct square { double operator()(double x) const noexcept { return x * x; } };
struct cube   { double operator()(double x) const noexcept { return x * x * x; }
 ↪  };

template <class F>
double reduce_static(std::span<const double> v, F f);

extern template double reduce_static<square>(std::span<const double>, square);
extern template double reduce_static<cube>(std::span<const double>, cube);
```

```
// reduce.cpp
#include "reduce.hpp"

template <class F>
double reduce_static(std::span<const double> v, F f) {
    double acc = 0.0;
    for (double x : v) acc += f(x);
    return acc;
}

template double reduce_static<square>(std::span<const double>, square);
template double reduce_static<cube>(std::span<const double>, cube);
```

A többi fordítási egység csak a deklarációt látja, így nem instantiál újra. Az assemblyben azt keresed, hogy a `reduce_static<square>` és `reduce_static<cube>` külön szimbólumként jelenik meg, és a hívási ciklusban nincs indirekt hívás. Ellenőrzési feltétel: a `nm` kimenetében a két explicit instantiation szimbóluma szerepel, és a forrásból készült objektumfájlokban nincs duplikált definíció.

3. megoldás. A probléma kettős: a `std::function` létrehozása a ciklusban másolja a nagy callable-t, és ha az nem fér be az SBO-ba, allokál. Első javítás: a `std::function`-t a cikluson kívül hozzuk létre, és `const&`-ként adjuk át; ezzel a másolás és az allokáció a ciklusból kikerül. Élettartam-szerződés: a wrappernek a ciklus teljes ideje alatt érvényesnek kell lennie, és a benne tárolt callable nem hivatkozhat a cikluson belül megszűnő objektumra. Második javítás: ha a callable nagy és a típus fordítás időben ismert, térjünk át template-alapú statikus dispatchre, ami megszünteti a wrappert és az allokációt. Ha a callable csak mozgatható, C++23-tól `std::move_only_function` használható; ezt a library támogatásának ellenőrzése előzi meg. Ellenőrzési feltétel: az assemblyben a ciklusban nincs allokációs hívás, és a wrapper konstrukciója a cikluson kívül történik; a viselkedés a benchmarkban a várt értéket adja, de ezt a laborban külön kell megmérni.

37.6 Optimalizálási labor correctness kapukkal

Az optimalizálás ebben a könyvben nem attól számít sikernek, hogy egy mérési szám kisebb lett, hanem attól, hogy a változtatás a helyességi szerződéseket érintetlenül hagyta, és a javulás reprodukálható módon, dokumentált workload mellett mutatható ki. Ez az alfejezet azt a labort írja le, amelyben a jelölt optimalizálást először correctness kapukon engedjük át, és csak utána értékeljük teljesítményét. A korábbi részekből hozott fogalmakra építünk: a template, virtual és `std::function` költségmodelljére (Template, virtual és std::function költsége (37.5)), a ranges- és callback-élettartam szabályaira (Ranges- és callback-élettartam labor (30.6)), valamint a shared ownership és konkurens reclamation review szempontjaira (shared ownership, reclamation és konkurens review (34.6)). A labornak nem célja az univerzális „gyorsabb lett” kijelentés, és nem garantál zero-cost viselkedést egyetlen nyelvi eszközre sem.

37.6.1 A helyességi kapu mint az optimalizálás előfeltétele

Optimalizálási labornak nevezzük azt a reprodukálható eljárást, amely egy jelölt változtatást (patch) ugyanazon a workloadon, ugyanazon build- és gépprofil mellett hasonlít össze a kiinduló állapottal, miközben a mérés előtt és után ugyanazokat a helyességi ellenőrzéseket futtatja. **Correctness kapu** (helyességi kapu) minden olyan automatizált ellenőrzés, amelynek sikertelensége esetén a teljesítményeredménytől függetlenül a változtatás elutasítandó. A kapu akkor érvényes, ha determinisztikus vagy legalább reprodukálható, ha a workload reprezentatív a célhasználatra, és ha a küszöbét a mérés előtt rögzítettük.

A labornak legalább négy kapuja van. Az első a **funkcionális ekvivalencia**: a kiinduló és a jelölt build ugyanarra az inputra ugyanazt a megfigyelhető kimenetet adja, hibakezelési ágakban is. A második a **memória- és élettartam-kapu**: a jelölt kód nem tartalmaz dangling hivatkozást, a tulajdonos és a nézet élettartama nem cserélődött fel, és a sanitizerek nem jeleznek. A harmadik a **konkurencia-kapu**: a szálas viselkedés nem lett versenyhelyzetessé, és a szinkronizációs szerződés változatlan. A negyedik az **allokációs és komplexitási kapu**: a változtatás nem növelte a kért allokációk számát vagy a dokumentált műveleti költséget anélkül, hogy ezt a tradeoffot külön indokolnánk.

A kapuk közös jellemzője, hogy nem a benchmark eredményéből következtetünk rájuk, hanem külön futnak. A mérés a kapuk után jön, és a mért szám önmagában soha nem írja felül a kapu eredményét. A gyakorlatban ez azt jelenti, hogy a jelölt buildből legalább két bináris készül: egy ellenőrző (sanitizeres vagy debug) és egy mérő (optimalizált). A kettő eredménye nem keverhető.

37.6.2 Allokációs költségvetés és a reserve szerződése

Az **allokációs költségvetés** (allocation budget) a workload egy egységére, például egy feldolgozott kérésre vagy egy parse-hívásra megengedett allokációk maximális száma és összes byte-mennyisége. A költségvetés nem azt mondja meg, hogy egy allokáció drága, hanem azt, hogy egy adott útvonalon hány allokáció történhet, mielőtt a viselkedés a szerződésen kívül esik. A `reserve` ennek egyik eszköze: előre lefoglalja a kapacitást, így a növekedés nem jár ismételt újraallokációval és elem-mozgatással. A `reserve` azonban nem kötelező és nem ingyenes: a túlbecsült kapacitás memóriát foglal, az alulbecsült továbbra is újraallokál.

A költségvetés mérésének legegyszerűbb módja egy számláló `memory_resource`, amely a `new_delete_resource`-ra delegál, de közben számol. Ez a C++17-es `std::pmr` felületre épül, és ugyanabban a kódban használható a naiv és az optimalizált változat összehasonlítására.

```
#include <cstddef>
#include <memory_resource>

struct counting_resource final : std::pmr::memory_resource {
    std::size_t allocations = 0;
    std::size_t bytes = 0;

private:
    void* do_allocate(std::size_t n, std::size_t a) override {
        ++allocations;
        bytes += n;
        return std::pmr::new_delete_resource()->allocate(n, a);
    }
    void do_deallocate(void* p, std::size_t n, std::size_t a) override {
        std::pmr::new_delete_resource()->deallocate(p, n, a);
```

```
    }
    bool do_is_equal(const std::pmr::memory_resource& other) const noexcept
     ↪  override {
        return this == &other;
    }
};
```

A költségvetés ellenőrzése nem az, hogy „kevesebb allokáció jó”, hanem hogy a mért szám egy előre rögzített korlát alatt marad, miközben a funkcionális kapu zöld. Ha a `reserve` bevezetése csökkenti az allokációk számát, de a csúcs-RSS nő, akkor a tradeoffot külön kell dokumentálni, mert a költségvetés csak az allokációk számát korlátozta, a memóriacsúcsot nem.

37.6.3 Span, ranges és a materializáció döntése

A `std::span` és a ranges view-k nem tulajdonolnak; egy meglévő tárolóra mutatnak. Ezért olcsók, de élettartam-szerződésük szigorú: a mögöttes tárolónak túl kell élnie a nézetet. A **materializáció** ezzel szemben új tárolót hoz létre, jellemzően allokálva, de a létrehozott objektum önálló tulajdonos lesz. A döntés nem esztétikai: a nézet akkor jó, ha a fogyasztó élettartama biztosan a tároló élettartamán belül van; a materializált másolat akkor, ha a nézet túlélhetné a tárolót, vagy ha a pipeline-t többször, egymástól függetlenül kell bejárni.

A tipikus hiba a lusta pipeline többszöri bejárása. Ha egy `views::transform` láncot minden fogyasztónál újra végigfuttatunk, a számítás megismétlődik; ha viszont materializálunk, a számítás egyszer történik, de allokálunk és memóriát foglalunk. A laborban ezt a döntést nem elméletben, hanem a két változat mérésével kell meghozni, és a döntést a workloadhoz kell kötni. C++20-ban a `std::ranges::to` még nem része a szabványnak, ezért a materializálást kézi ciklussal vagy a tartomány két iterátorával kell megírni; a C++23-as `ranges::to` használatát külön jelölni kell, és a laborprofilban ellenőrizni kell a library támogatását.

37.6.4 Arena és pmr: az élettartam a szerződés

A `std::pmr::monotonic_buffer_resource` egy **aréna**: előre vagy növekvő blokkokban ad memóriát, és a felszabadítást nem egyenként, hanem az erőforrás megszűnésekor végzi. Ezért a belőle allokált objektumok élettartama nem haladhatja meg az erőforrás élettartamát. Ez a szabály akkor is áll, ha a `pmr` konténer vagy string látszólag önálló objektum: a háttértárolója az arénában van, és a destruktora csak a saját logikai állapotát bontja le, a memóriát nem adja vissza egyenként.

A helyes minta szerkezeti: az aréna legyen a legkülső, legkorábban létrehozott és legkésőbb megsemmisülő objektum, a belőle allokáló konténerek pedig utána jöjjenek létre. Így a destruktor-sorrend automatikusan biztosítja, hogy a konténer előbb szűnjön meg, mint az aréna. A fordító ezt nem ellenőrzi, a sanitizer pedig csak akkor jelez, ha a hibás hozzáférés tényleg megtörténik; ezért kell a szerkezeti szabályt a kódban is láthatóvá tenni, és a laborban egy negatív tesztben szándékosan előidézni a hibát.

37.6.5 Többszálú pipeline és false sharing

A **false sharing** (hamis megosztás) akkor lép fel, amikor két szál különböző változókat ír, de ezek ugyanabba a cache line-ba esnek. A szálak nem versenyeznek ugyanazért az objektumért, tehát nincs adatverseny a nyelvi memóriamodell értelmében, a cache-koherencia miatt mégis folyamatosan érvénytelenítik egymás cache line-ját. A jelenség teljesítményhiba, nem helyességi

hiba; a TSan nem fogja jelezni, mert nincs benne adatverseny. A mérés mégis fontos, mert a látszólag független számlálók egy structban ugyanabba a line-ba kerülhetnek.

A javítás tipikusan a változók külön cache line-ba helyezése. A C++17 erre szánt konstansa a `<new>` fejlécben:

```
std::hardware_destructive_interference_size
```

Ezt nem minden implementáció definiálja, ezért a laborprofilban rögzíteni kell, hogy a konkrét célon mi a használt érték; x86-64-en a 64 bájt tipikus, de ez nem hordozható garancia. A padding hatását teljesítménymérés mutatja, a helyességet továbbra is a TSan és a funkcionális tesztek őrzik.

37.6.6 Három profil, három kérdés

A labor három buildprofilt használ, és a három profil három különböző kérdésre válaszol. A **debug** profil (`-O0 -g`) a referencia-szemantika és az állítások profilja: itt a lépésenkénti viselkedés a fontos, a futásidő nem. A **sanitizer** profil (`-O1 -g -fsanitize=address,undefined`, illetve külön futásban `-fsanitize=thread`) a memória-, UB- és versenyhibák profilja; a sanitizer erősen torzítja a futásidőt, ezért benchmarkra nem használható. A **release** profil (`-O2` vagy `-O3`, `-DNDEBUG`) a teljesítménymérés profilja, de önmagában nem elégséges helyességi referencia, mert az `assert` eltűnik belőle, és az optimalizáló a nem definiált viselkedésre támaszkodva „javíthat” is.

Profil	Fő kapcsolók	Szerep	Korlát
debug	`-O0 -g`	referencia-viselkedés, assert	nem benchmark
sanitizer	`-O1 -g -fsanitize=address,undefined`	memória- és UB-hibák	torzított futásidő
sanitizer (szál)	`-O1 -g -fsanitize=thread`	adatverseny	nem kombinálható ASan-nel
release	`-O2 -DNDEBUG`	teljesítmény	nem egyedüli helyességi kapu

A release profilban a helyességi kaput nem `assert`-re kell építeni, mert a `-DNDEBUG` eltávolítja. Helyette `if (!condition) return 1;` vagy a tesztkeretrendszer nem-assert alapú ellenőrzése használható. A három profil eredménye soha nem keveredik: a sanitizer futásidejéből nem következtetünk a release teljesítményére, és a release zöld tesztje nem pótolja a sanitizeres futást.

37.6.7 Kidolgozott példa: ugyanaz az eredmény, kevesebb allokáció

A példa egy `kulcs=érték` sorokból álló szöveget dolgoz fel. A kiinduló változat minden kulcshoz külön `std::string`-et hoz létre, a jelölt változat `pmr` arénát és `pmr::string`-eket használ. A két változat ugyanazt a logikát futtatja, a különbség az allokátor. Hogy a kiinduló változat kulcsonként heapre allokál-e, az **feltételes**: a kisstring-optimalizáció (SSO) miatt a rövid kulcsok egyáltalán nem allokálnak, csak az SSO-küszöbnél hosszabbak. Ezért a program rövid és hosszú kulcsokkal is lefut, és a globális heapallokációkat számolja; a szemantikai egyezés és a mért allokációszám két külön eredmény.

```cpp
// pipeline.cpp
#include <charconv>
#include <cstdlib>
#include <new>
#include <cstddef>
#include <cstdio>
#include <memory_resource>
#include <string>
#include <string_view>
#include <vector>

// Globális heapallokáció-számláló: csak a mérés idejére olvassuk.
static std::size_t g_allocs = 0;
void* operator new(std::size_t n) {
    ++g_allocs;
    if (void* p = std::malloc(n ? n : 1)) return p;
    throw std::bad_alloc{};
}
void operator delete(void* p) noexcept { std::free(p); }
void operator delete(void* p, std::size_t) noexcept { std::free(p); }
// A pmr::new_delete_resource az igazított változatot hívja; ezt is számoljuk.
void* operator new(std::size_t n, std::align_val_t al) {
    ++g_allocs;
    std::size_t a = static_cast<std::size_t>(al);
    if (void* p = std::aligned_alloc(a, (n + a - 1) / a * a)) return p;
    throw std::bad_alloc{};
}
void operator delete(void* p, std::align_val_t) noexcept { std::free(p); }
void operator delete(void* p, std::size_t, std::align_val_t) noexcept {
 ↪  std::free(p); }

namespace {

struct owned_record {
    std::string key;
    int value;
};

struct pmr_record {
    std::pmr::string key;
    int value;
};

std::vector<owned_record> parse_owned(std::string_view in) {
    std::vector<owned_record> out;
    std::size_t pos = 0;
    while (pos < in.size()) {
        std::size_t nl = in.find('\n', pos);
        if (nl == std::string_view::npos) nl = in.size();
        std::string_view line = in.substr(pos, nl - pos);
        std::size_t eq = line.find('=');
        if (eq != std::string_view::npos && eq + 1 < line.size()) {
```

```
            int v = 0;
            auto [p, ec] = std::from_chars(line.data() + eq + 1,
                                           line.data() + line.size(), v);
            if (ec == std::errc{} && p == line.data() + line.size())
                out.push_back({std::string{line.substr(0, eq)}, v});
        }
        pos = nl + 1;
    }
    return out;
}

std::pmr::vector<pmr_record> parse_pmr(std::string_view in,
                                       std::pmr::memory_resource* mr) {
    std::pmr::vector<pmr_record> out{mr};
    std::size_t pos = 0;
    while (pos < in.size()) {
        std::size_t nl = in.find('\n', pos);
        if (nl == std::string_view::npos) nl = in.size();
        std::string_view line = in.substr(pos, nl - pos);
        std::size_t eq = line.find('=');
        if (eq != std::string_view::npos && eq + 1 < line.size()) {
            int v = 0;
            auto [p, ec] = std::from_chars(line.data() + eq + 1,
                                           line.data() + line.size(), v);
            if (ec == std::errc{} && p == line.data() + line.size())
                out.push_back(pmr_record{
                    std::pmr::string{line.substr(0, eq), mr}, v});
        }
        pos = nl + 1;
    }
    return out;
}

bool equivalent(const std::vector<owned_record>& a,
                const std::pmr::vector<pmr_record>& b) {
    if (a.size() != b.size()) return false;
    for (std::size_t i = 0; i < a.size(); ++i) {
        if (std::string_view{a[i].key} != std::string_view{b[i].key}) return
        ↪  false;
        if (a[i].value != b[i].value) return false;
    }
    return true;
}

} // namespace

struct counts { std::size_t baseline, candidate; };

// Ugyanazt a bemenetet mindkét változattal feldolgozza, és megszámolja a
// globális heapallokációkat. A szemantikai ellenőrzés külön eredmény.
int run(std::string_view input, std::size_t expected, counts& c) {
    std::size_t a0 = g_allocs;
```

```
    auto baseline = parse_owned(input);
    c.baseline = g_allocs - a0;

    // Az arena legyen kívül, a belőle allokáló konténer utána.
    std::pmr::monotonic_buffer_resource arena{4096};
    std::size_t a1 = g_allocs;
    auto candidate = parse_pmr(input, &arena);
    c.candidate = g_allocs - a1;

    // NDEBUG mellett is lefutó ellenőrzés (nem assert).
    if (baseline.size() != expected || !equivalent(baseline, candidate)) {
        std::fprintf(stderr, "pipeline: eltérés a baseline-tól\n");
        return 1;
    }
    for (const auto& r : candidate) {
        if (r.key.get_allocator().resource() != &arena) {
            std::fprintf(stderr, "pipeline: rossz resource\n");
            return 1;
        }
    }
    return 0;
}

int main() {
    // Rövid kulcsok: SSO-ba férnek, a baseline-ban nincs kulcsonkénti
    ↪ allokáció.
    const std::string_view short_keys =
        "alpha=1\nbeta=22\ngamma=333\ndelta=not-a-number\nepsilon=5\n";
    // SSO-n túli (libstdc++: >15 karakter) kulcsok.
    const std::string_view long_keys =
        "service.alpha.endpoint=1\nservice.beta.endpoint=22\n"
        "service.gamma.endpoint=333\nservice.delta.endpoint=x\n"
        "service.epsilon.endpoint=5\n";

    counts s{}, l{};
    if (run(short_keys, 4, s) != 0 || run(long_keys, 4, l) != 0) return 1;
    std::printf("pipeline: 4 rekord egyezik (rovid es hosszu kulcsokkal)\n");
    std::printf("heapallokacio rovid kulcs:  baseline=%zu jelolt=%zu\n",
    ↪ s.baseline, s.candidate);
    std::printf("heapallokacio hosszu kulcs: baseline=%zu jelolt=%zu\n",
    ↪ l.baseline, l.candidate);
    return 0;
}
```

A `pmr_record` aggregátum, nem allokátortudatos típus, ezért a vektor nem adja át neki a resource-ot; a kulcs `std::pmr::string`-jét kifejezetten az `mr`-rel kell létrehozni. A rekord ezután a vektorba mozog, és a mozgatott string megtartja az allokátorát. Az ellenőrzés nem `assert`, hogy `NDEBUG` mellett se tűnjön el; a kulcsok resource-át is vizsgálja.

Buildjegyzőkönyv (`lab-gcc` profil). A program `g++ -std=c++20 -O2 -Wall -Wextra -Wpedantic` beállítással figyelmeztetés nélkül fordult, és 0 kóddal futott, `-DNDEBUG` mellett és ASan/UBSan alatt is. *Szemantikai eredmény:* `pipeline: 4 rekord egyezik (rovid es hosszu kulcsokkal)`. *Mért eredmény:* rövid kulcsokkal `baseline=3 jelolt=1`, hosszú

kulcsokkal `baseline=7 jelolt=1` globális heapallokáció. A baseline 3 allokációja a vektor növekedése (1, 2, 4 elem); a hosszú kulcsoknál ehhez jön a 4 kulcs saját puffere. A jelölt egyetlen allokációja az aréna 4096 bájtos blokkja, kulcshossztól függetlenül. Rövid kulcsra tehát a nyereség nem a kulcsokon, hanem a vektor növekedésén keletkezik. A számok a libstdc++ SSO-küszöbéhez (15 karakter) és növekedési stratégiájához kötöttek. A javítás előtti `out.emplace_back(line.substr(0, eq), mr)` sor nem fordult, és a beolvasott `v` értéket sem tárolta volna.

A példában az `arena` hamarabb jön létre, mint a `candidate`, ezért a `candidate` előbb semmisül meg. Ez a helyes sorrend. Ha az aréna egy belső blokkban jönne létre, a visszaadott `pmr` konténer a blokk végén dangling hivatkozást tartalmazna. A `parse_pmr` visszatérési értéke move-ol, az allokátor pointere nem változik, ezért a visszaadott vektor továbbra is az arénára mutat.

A buildparancsok a laborprofilhoz kötöttek; a fordító és a library konkrét verzióját rögzíteni kell.

```
# Release profil: teljesítménymérés
g++ -std=c++20 -O2 -g -Wall -Wextra -Wpedantic pipeline.cpp -o pipeline-release

# Sanitizer profil: memória és UB
g++ -std=c++20 -O1 -g -fsanitize=address,undefined -fno-omit-frame-pointer \
    pipeline.cpp -o pipeline-asan

# TSan profil külön binárison, a szálas példára (nem kombinálható ASan-nel)
g++ -std=c++20 -O1 -g -fsanitize=thread -pthread false_sharing.cpp -o
 ↪  false_sharing-tsan
```

A `false_sharing.cpp` a következő minta: két szál két külön atomikus számlálót növel, amelyek egy structban, de `alignas`-szal külön cache line-ban vannak. A rossz változatból az `alignas` elhagyható, és a mérés megmutatja a különbséget; a TSan egyik változatban sem jelez adatversenyt, mert nincs.

```
// false_sharing.cpp
#include <atomic>
#include <cstdint>
#include <thread>

struct counters {
    alignas(64) std::atomic<std::uint64_t> a{0};
    alignas(64) std::atomic<std::uint64_t> b{0};
};

int main() {
    counters c;
    std::thread t1{[&] {
        for (int i = 0; i < 1'000'000; ++i)
            c.a.fetch_add(1, std::memory_order_relaxed);
    }};
    std::thread t2{[&] {
        for (int i = 0; i < 1'000'000; ++i)
            c.b.fetch_add(1, std::memory_order_relaxed);
    }};
    t1.join();
    t2.join();
}
```

37.6.8 A bizonyított változás feltételei

Egy előtte-utána összehasonlítás akkor tekinthető **bizonyított változásnak** az adott workloadon, ha a következő feltételek együtt teljesülnek. Először is a két oldal forrása és inputja ugyanaz, a különbség csak a vizsgált patch. Másodszor a fordító, annak verziója, a kapcsolók, a gép és a terhelés azonos; a mérés a release profilban történik. Harmadszor a metrika és a küszöb előre rögzített, például a medián falidő, az allokációk száma vagy a csúcs-RSS. Negyedszer legalább öt ismétlés történik, és a mediánok különbsége nagyobb, mint a mért szórás; a kiugró futásokat nem elrejteni, hanem dokumentálni kell. Ötödször minden correctness kapu zöld a jelölt builden. Hatodszor a változás reprodukálható: megismételve, hidegindítással vagy más napon is hasonló irányt mutat. Végül a tradeoff dokumentált: memória, kódméret, komplexitás, hordozhatóság és a workload érvényességi tartománya.

Az előtte-utána táblázat sablonja a következő; a cellák a laborban töltendők ki, nem mért eredmények.

Metrika	Kiinduló	Jelölt	Ellenőrzés
allokációk / hívás	kitöltendő	kitöltendő	`counting_resource`
összes allokált byte	kitöltendő	kitöltendő	`counting_resource`
medián falidő	kitöltendő	kitöltendő	5+ futás, release
csúcs-RSS	kitöltendő	kitöltendő	`/usr/bin/time -v`
kódméret	kitöltendő	kitöltendő	`size`

A „bizonyított” szó itt nem azt jelenti, hogy a változtatás minden workloadon gyorsabb, hanem azt, hogy az adott, dokumentált workloadon a javulás a mérési zajon túl van, és a helyességi szerződés nem sérült. Az univerzális benchmark és a zero-cost garancia nem cél; a döntés workloadhoz kötött.

37.6.9 Tipikus hibák és javításuk

Az egyik gyakori hiba, hogy a benchmarkot sanitizeres buildben futtatják. Az ASan és a TSan jelentős többletmunkát végez, ezért az így kapott idő nem hasonlítható a release időhöz. A javítás: a mérés mindig a release binárison történik, a sanitizer csak a kaput futtatja.

A második hiba a `reserve` túlméretezése: a kód kevesebb allokációt végez, de a csúcs-RSS jelentősen nő, és ezt a jelentés nem tartalmazza. A javítás: a költségvetésben az allokációk száma mellett a byte- és a csúcs-RSS-korlátot is rögzíteni kell.

A harmadik hiba a nézet és a tároló élettartamának felcserélése. Egy `std::span` vagy egy ranges view a megszűnt tárolóra mutat, és a hiba csak akkor jelenik meg, ha a teszt a tároló megszűnése után is használja a nézetet. A javítás: a nézetet csak a tároló élettartamán belül adjuk tovább, vagy materializáljunk; a laborban negatív teszt is legyen, amely a hibás mintát szándékosan előidézi, és a sanitizerrel igazolja a hibát.

A negyedik hiba az aréna élettartamának elrejtése egy függvényen belül. Ha a `pmr` konténer visszatér a függvényből, de az aréna a függvény lokális változója, a konténer dangling. A javítás: az aréna a hívó felelőssége, a visszatérési típus pedig dokumentálja, hogy a konténer az arénára mutat.

Az ötödik hiba a false sharing és az adatverseny összekeverése. A TSan a versenyt detektálja, a false sharinget nem; a javulás mérése és a helyesség ellenőrzése két külön kapu. A javítás: a paddinget teljesítménymérés igazolja, a szálak közötti helyességet továbbra is a TSan és a funkcionális teszt.

37.6.10 Feladatok

1. **Magyarázó feladat.** Definiálja a saját workloadjára a bizonyított változás öt feltételét, és adja meg, melyik metrika melyik kapuhoz tartozik. Készítsen egy előtte-utána táblázatot üres cellákkal és a kitöltés szabályaival.

2. **Kódolási feladat.** Egészítse ki a `pipeline.cpp` példát egy `counting_resource`-szal, és mérje meg a két parse-változat allokációszámát és byte-mennyiségét. Állítson be egy költségvetést, és fogalmazza meg, milyen feltétel mellett tekinti a költségvetést teljesültnek.

3. **Diagnosztikai feladat.** Az alábbi vázlatban az aréna a függvényen belül jön létre, a visszatérési érték viszont `pmr` konténer. Mutassa meg, miért dangling, és javítsa a kódot a hívó felelősségének átadásával.

```
std::pmr::vector<pmr_record> bad_parse(std::string_view in) {
    std::pmr::monotonic_buffer_resource arena{4096};
    return parse_pmr(in, &arena);
}
```

4. **Kódolási feladat.** A `false_sharing.cpp`-ben távolítsa el az `alignas(64)`-eket, mérje meg a futásidőt, majd tegye vissza őket. Magyarázza meg, miért nem jelez a TSan egyik változatban sem adatversenyt, és mitől függ a mért különbség.

5. **Elemző feladat.** Hasonlítson össze egy `std::span`-alapú nézetet és egy materializált `std::vector`-t ugyanarra a feldolgozásra. Adja meg, melyik esetben nő az allokációk száma, melyik esetben a számítás ismétlődik, és melyik élettartam-szerződés szigorúbb.

37.6.11 Megoldások

1. megoldás. A bizonyított változás öt feltétele a saját workloadra: (a) azonos forrás és input, csak a patch a különbség; (b) azonos fordító, verzió, kapcsolók, gép és terhelés, mérés release profilban; (c) előre rögzített metrika és küszöb; (d) legalább öt ismétlés, a mediánok különbsége nagyobb a szórásnál; (e) minden correctness kapu zöld, és a változás reprodukálható. A metrika és a kapu hozzárendelése: a funkcionális ekvivalencia a kimenet-összehasonlítás kapuja; az allokációszám a költségvetés kapuja; a medián falidő a teljesítmény metrikája; a csúcs-RSS a memória-kapu; a sanitizeres futás a memória- és konkurencia-kapu. Az előtte-utána táblázatot a fenti sablon szerint kell kitölteni, a cellákban a mért értékekkel és a futások számával. Ellenőrzési feltétel: a táblázat minden sora mellett szerepel a mérés módja és a küszöb.

2. megoldás. A `counting_resource` a `new_delete_resource`-ra delegál, és a `do_allocate`-ban növeli az `allocations` és `bytes` számlálót. A naiv változatot `counting_resource`-szal kell futtatni, de a `std::vector<owned_record>` default allokátora nem pmr, ezért vagy a naiv változatot is pmr-esíteni kell a méréshez, vagy a két változatot külön mérjük, és a különbséget a `pmr` bevezetésének tulajdonítjuk. A költségvetés például: egy parse-hívásra legfeljebb annyi allokáció, amennyi a sorok száma plusz a vektor növekedési lépései, és legfeljebb a bemenet méretének konstansszorosa byte-ban. Ellenőrzési feltétel: a jelölt allokációszáma nem nagyobb a kiindulónál, a funkcionális kapu zöld, és a byte-mennyiség a rögzített korlát alatt van.

3. megoldás. A `bad_parse` az arénát a saját scope-jában hozza létre, majd visszaad egy `pmr::vector`-t, amelynek minden eleme és a vektor puffere is az arénából származik. A függvény visszatérésekor az aréna megszűnik, a visszaadott vektor pedig dangling pointereket tartalmaz. A javítás: az aréna a hívóé legyen, a függvény paraméterként kapja a `memory_resource*`-t, és a visszatérési típus dokumentálja, hogy a konténer az arénára mutat.

```
std::pmr::vector<pmr_record> parse_into(std::string_view in,
                                        std::pmr::memory_resource* mr) {
    return parse_pmr(in, mr);
}
// hívó:
std::pmr::monotonic_buffer_resource arena{4096};
auto rows = parse_into(input, &arena);
```

Ellenőrzési feltétel: az aréna deklarációja megelőzi a konténerét, a konténer az aréna megszűnése előtt megszűnik, és az ASan-es futás nem jelez. A hibás változatot negatív tesztként érdemes megtartani, de csak izoláltan, dokumentáltan.

4. megoldás. Az `alignas(64)` eltávolításával a két atomikus számláló valószínűleg ugyanabba a cache line-ba kerül, és a két szál írásai folyamatosan érvénytelenítik egymás line-ját; a futásidő nő. A TSan azért nem jelez, mert a két szál különböző objektumokat ír, nincs köztük adatverseny a nyelvi memóriamodell szerint. A mért különbség a cache line méretétől, a szálak számától, a magok elhelyezkedésétől és a háttérterheléstől függ; ezért a mérés csak a rögzített laborprofilra érvényes. Ellenőrzési feltétel: a funkcionális eredmény változatlan, a TSan mindkét változatban tiszta, és a paddingolt változat mediánja legalább a mért szórásnál nagyobb mértékben jobb a nem paddingolt változatnál.

5. megoldás. A `std::span`-alapú nézet nem allokál, de a mögöttes tároló élettartamához kötött; ha a tároló megszűnik, a nézet dangling. A materializált `std::vector` allokál, de önálló tulajdonos, ezért túlélheti a forrástárolót. Ha ugyanazt a lusta pipeline-t több fogyasztó járja be, a számítás ismétlődhet; a materializálás ezt egyszeri költséggé alakítja, cserébe allokál. A szigorúbb élettartam-szerződés a nézeté: a nézet csak a tároló élettartamán belül érvényes. Ellenőrzési feltétel: a döntés mellett szerepel a mért allokációszám, a nézet élettartamának indoklása és a sanitizeres teszt eredménye.

37.6.12 Összefoglaló

Az optimalizálási labor sorrendje rögzített: először a funkcionális, memória- és konkurencia-kapuk, aztán a mérés. Az allokációs költségvetés a `reserve`-dzsel együtt az allokációk számát és byte-mennyiségét korlátozza, de a csúcs-RSS-t külön kell figyelni. A `span` és a ranges view olcsó, de élettartamhoz kötött; a materializáció allokál, de önálló tulajdonos. Az aréna és a `pmr` esetében az élettartam-szerződés szerkezeti: az aréna legyen a legkülső objektum. A false sharing teljesítményhiba, nem adatverseny; a TSan nem fogja meg, a mérés igen. A három profil három különböző kérdésre válaszol, és az eredményeik nem keverhetők. A bizonyított változás feltételei az azonos környezet, az előre rögzített metrika, az ismételt mérés, a zöld kapuk és a dokumentált tradeoff. Az univerzális benchmark és a zero-cost garancia nem része a labornak; a döntés mindig a dokumentált workloadhoz kötött.

38 Natív build, tesztek, CI és disztribúció

38.1 Build graph, Make/Ninja és incremental build

A natív build nem „a fordító lefuttatása”, hanem egy irányított gráf bejárása: a forrásokból, headerekből, generált fájlokból objektumok, könyvtárak és végül futtatható állomány lesz. Az inkrementális build helyessége ezen a gráfon múlik, nem a compiler sebességén. Az ODR és a több fordítási egység korábbi szabályai ODR, inline és több fordítási egység (33.3) itt kapnak build szintű következményt; a warning- és optimalizálási profilok Warningok, optimalizálási profilok és link-driver (2.5), valamint a feature-detection Feature-test, standard library és build capability (33.5) pedig a recept-ujjlenyomat részei.

38.1.1 A build gráf és az inkrementális érvényesség

A build gráf csomópontjai fájlok és célok, élei receptek. Egy t cél közvetlen, deklarált függőségeit $D(t)$, receptjét $r(t)$ jelöli. A gyakorlatban t akkor használható fel újra, ha minden $d \in D(t)$ érvényes és változatlan, valamint $r(t)$ ujjlenyomata is változatlan. A változást a build rendszer többnyire módosítási idővel (mtime), tartalom-hash-sel vagy ezek kombinációjával észleli. Az mtime csak közelítés: nem szemantikai változás, hanem egy olcsó jel.

A fordítási egység (translation unit, TU) egy forrás preprocesszor utáni állapota. A compile él bemenete a TU összes include-olt fájlja, a compiler és a ténylegesen ható kapcsolók; kimenete jellemzően egy objektumfájl. A link él bemenete az objektumok, statikus és megosztott könyvtárak, linkerkapcsolók és linker scriptek; kimenete futtatható vagy megosztott könyvtár. Egy header változása ezért csak azoknak a TU-oknak a compile élét invalidálja, amelyek ténylegesen include-olják; a linket viszont az érintett objektumokon keresztül.

A gráf akkor helyes, ha a deklarált függőségek lefedik a tényleges bemeneteket, és a recept változását is észleli a rendszer. A hiányzó él elavult objektumhoz (stale object) vezet: a build „sikeres”, de a kimenet nem az aktuális forrásból származik. A feleslegesen deklarált él biztonságosabb, de újrafordítási vihart okozhat. A célok topológikus sorrendben épülnek; kör a gráfban érvénytelen.

Bizonyításvázlat. Tegyük fel, hogy minden tényleges bemenet benne van a deklarált lezárásban, és a recept-ujjlenyomat minden kimenetet befolyásoló változást tartalmaz. Ha egy bemenet megváltozik, a topológikus sorrendben az összes érintett cél invalidálódik, majd újraépül; az eredmény ugyanaz, mint egy tiszta buildnél. Ha viszont hiányzik egy él, konstruálható olyan változás, amely nem invalidálja a célt, mégis más kimenetet adna; ez a stale object. A vázlat feltételezi a recept determinizmusát és azt, hogy a környezet hatásai is a kulcs részei; ezek hiányában a garancia gyengül.

38.1.2 Headerfüggőség, depfile és generált header

A build rendszer a fordítás előtt nem tudja, mely headereket fogja beolvasni a preprocesszor. Ezért a compiler melléktermékként függőségi fájlt (depfile) ír. GCC és Clang esetén erre a `-MMD` vagy `-MD`, valamint a `-MP` szolgál; a `-MMD` kihagyja a rendszerheadereket, a `-MD` hozzáveszi őket. A

pontos viselkedés compiler-verziófüggő, ezért a laborban a telepített verzió dokumentációjával kell ellenőrizni.

```
g++ -std=c++20 -MMD -MP -Isrc -Ibuild/generated \
    -c src/greet.cpp -o build/greet.cpp.o
```

A keletkező `build/greet.cpp.d` illusztratív tartalma:

```
build/greet.cpp.o: src/greet.cpp src/greet.hpp build/generated/config.h
src/greet.hpp:
build/generated/config.h:
```

A `-MP` általában phony szabályokat ad a headerekhez, hogy egy törölt header ne okozzon azonnal „no rule to make target” hibát. A Makefájlba a depfile-t be kell vonni, például `-include $(DEPS)` formában, hogy a következő futás már ismerje a headerfüggőségeket. A depfile csak akkor keletkezik, amikor a compiler már lefutott; generált header esetén ezért annak előállítását explicit gráfélként kell megadni. Különben az első build nem tudja kielégíteni a még nem létező include-ot. CMake-ben ezt jellemzően `add_custom_command(OUTPUT ...)` és a fogyasztó target közötti függőség, vagy configure idejű `configure_file` adja.

38.1.3 Make, Ninja és a meta-build CMake

A Make általános szabályértelmező: a felhasználó írja le a célokat, előfeltételeket és recepteket. Jól használható kis projektekhez, de a headerfüggőségek, a generált fájlok és a kapcsolóváltozások követése a felhasználó felelőssége. Az általános Make szabály nem követi automatikusan a recept szövegét: ha csak a `CFLAGS` változik, a cél újabb maradhat, és nem indul újra a fordítás.

A Ninja alacsony szintű, gyors végrehajtó; a `build.ninja` fájlt jellemzően nem kézzel írjuk, hanem egy meta-build generálja. A Ninja pontos éleket, depfile-t és parancs-hash-t használ, ezért a recept változását általában észleli. Hasznos diagnosztikai kapcsolói például a `-d explain`, a `-t deps` és a `-t commands`; ezek pontos neve és kimenete verziófüggő.

A CMake meta-build: target-alapú modellt ír le, majd Make- vagy Ninja-fájlt generál. A targetek közötti használati követelmények (usage requirements) adják a include-utak és kapcsolók transzparens terjedését: a `PUBLIC` beállítás a fogyasztókhoz is eljut, a `PRIVATE` nem. A CMake configure- és build-fázisa különül el; a `cmake --build` a kiválasztott generátort hívja. A `compile_commands.json` a configure eredménye: fordítási egységenként rögzíti a tényleges parancsot, és kiváló clangd, clang-tidy vagy saját ellenőrzés céljára. Nem függőségi gráf, és nem a build bemenete; könnyen elavulhat, ha a build könyvtárat kézzel módosítjuk.

38.1.3.1 Példaprojekt

Az alábbi példa Linuxon, GCC vagy Clang mellett, C++20-cal értelmezendő. A cél a gráf és a generált header bemutatása; a kimenet elvárt, nem mért eredmény.

```
cmake_minimum_required(VERSION 3.20)
project(build_demo LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
```

```
set(BUILD_ID "demo-1")
configure_file(config.h.in generated/config.h @ONLY)

add_library(greet STATIC src/greet.cpp)
target_include_directories(greet PUBLIC
    ${CMAKE_CURRENT_SOURCE_DIR}/src
    ${CMAKE_CURRENT_BINARY_DIR}/generated)
target_compile_options(greet PRIVATE -Wall -Wextra -Wpedantic)

add_executable(app src/main.cpp)
target_link_libraries(app PRIVATE greet)
```

```
// config.h.in
#pragma once
#define BUILD_ID "@BUILD_ID@"

// src/greet.hpp
#pragma once
#include <string>
std::string greeting();

// src/greet.cpp
#include "greet.hpp"
#include "config.h"

std::string greeting() {
    return std::string{"build "} + BUILD_ID;
}

// src/main.cpp
#include "greet.hpp"
#include <iostream>

int main() {
    std::cout << greeting() << '\n';
}
```

```
cmake -S . -B build -G Ninja
cmake --build build --verbose
./build/app
```

A program logikailag `build demo-1` sort ír ki. A `config.h` a build könyvtárban generálódik; a `greet` target PUBLIC include-útja miatt az `app` is látja a szükséges headereket. A `compile_commands.json` a `build` könyvtárban keletkezik, és a tényleges fordítóp parancsokat tartalmazza.

38.1.4 Cache, flag és elavult objektum

A ccache és a sccache gyorsítótár. A cache-keynek tartalmaznia kell mindent, ami a kimenetet befolyásolja: a compiler identity-jét és verzióját, a targetet, a nyelvi szabványt, az include-utakat, a preprocesszort érintő define-okat, a forrás és a ténylegesen beolvasott headerek tartalmát, valamint a releváns kapcsolókat. A konkrét implementáció direct és preprocesszor módja eltérően építi a kulcsot; a biztonságos követelmény az, hogy a tényleges bemenetek hiánytalanul benne

legyenek. A cache nem függőségi oracle: a hiányzó depfile-él és a cache beállításai együtt nehezen reprodukálható elavulást okozhatnak.

A `__DATE__`, `__TIME__`, `__FILE__` és az abszolút útvonalak rontják a reprodukálhatóságot és a cache-találati arányt. A `-fmacro-prefix-map`, `-fdebug-prefix-map` és `-ffile-prefix-map` jellegű kapcsolók compilerfüggők, de hasznosak lehetnek. A `SOURCE_DATE_EPOCH` támogatása szintén eszközfüggő; nem szabad automatikusan minden toolchainre kiterjeszteni.

Generált header esetén gyakori hiba, hogy a generátor minden buildnél újraírja a fájlt, megváltoztatva az mtime-ot, és felesleges újrafordítást indít. A Ninja `restat` mechanizmusa és a tartalomstabil generálás segíthet; a Make-nél a generátort érdemes csak tartalomváltozáskor íratni. A flags változása különösen veszélyes: a Make gyakran nem indít újrafordítást, míg a Ninja parancs-hash alapján igen. A `compile_commands.json` két konfigurációjának összehasonlítása megmutatja, hogy valóban megváltozott-e a fordítóp parancs.

38.1.5 Diagnosztika és tipikus hibák

Ha a tiszta build helyes, az inkrementális viszont hibás, a hiba a gráfban vagy a kulcsban van, nem a compilerben. Első lépés a két build összehasonlítása külön könyvtárban, lehetőleg a cache megkerülésével, a választott cache dokumentált kapcsolójával. Ezután Ninja esetén `ninja -C build -d explain`, `ninja -C build -t deps <objektum>` és `ninja -C build -t commands`; Make esetén `make -n`, `make -d` vagy `make -B` segíthet. Ellenőrizni kell a generált header producer élét, a depfile tartalmát, a `compile_commands.json` frissességét és az mtime-anomáliákat: óraeltérés, régi időbélyeggel kicsomagolt fa, `cp -a`, hálózati fájlrendszer.

Tipikus hibák: a header nincs a depfile-ban; a generált headernek nincs előállító szabálya; a Make nem követi a flags változását; a `compile_commands.json`-t tévesen build-bemenetnek tekintik; a cache-t függőségkövetőnek használják; törölt header után `-MP` miatt nem jelenik meg azonnal a hiba, csak később; a rendszerheaderek kihagyása (`-MMD`) miatt toolchain-frissítésnél nem indul újra minden. A „touch mindent” és a „clean mindig” taktika elrejti az okot; a cél a hiányzó él vagy kulcs megtalálása.

38.1.6 Összefoglaló

A build gráf csomópontjai fájlok és célok, élei receptek. A compile él a TU összes tényleges include-ját, a link él az objektumokat és könyvtárakat fogyasztja. A helyes inkrementális build feltétele a teljes deklarált függőségi lezárás és a recept-ujjlenyomat követése. A depfile a headerfüggőségeket rögzíti, de generált headerhez explicit producer él kell. A Make egyszerű, de a fegyelmet a felhasználóra bízza; a Ninja gyors végrehajtó, a CMake target-alapú meta-build. A `compile_commands.json` diagnosztikai adat, nem build-gráf. A cache csak akkor biztonságos, ha a tényleges bemenetek a kulcsban vannak.

38.1.7 Feladatok

1. **Alap – gráfmodellezés.** Adott `main.cpp`, amely include-olja a `greet.hpp` és `config.h` fájlokat; `greet.cpp` include-olja a `greet.hpp`-t; a `config.h` generált; az `app` a `greet` statikus könyvtárhoz linkel. Rajzold fel a csomópontokat és éleket, majd mondd meg, mely változás mely célokat invalidálja.

2. **Középhaladó – Make és generált header.** Adott egy C17-es `src/main.c`, amely `#include "config.h"`-t használ, és egy `config.h.in`, amelyből a build során kell `build/config.h`-t előállítani. Írj olyan Makefájlt, amely első buildnél is működik, és a headerek változását is követi.

3. **Haladó – elavult objektum diagnózisa.** Egy Make-alapú projektben a `CFLAGS` értékét `-O0`-ról `-O2`-re írod, de a build nem fordít újra; a tiszta build viszont helyes. Magyarázd meg az okot, adj javítást, és add meg az ellenőrzés módját.

38.1.8 Megoldások

38.1.8.1 1. feladat megoldása

A csomópontok: `main.cpp`, `greet.cpp`, `greet.hpp`, `config.h`, `main.o`, `greet.o`, `libgreet.a`, `app`. A compile élek: `main.o` függ a `main.cpp`-től, a `greet.hpp`-tól és a `config.h`-tól; `greet.o` függ a `greet.cpp`-től, a `greet.hpp`-tól és a `config.h`-tól. A link él: `app` függ a `main.o`-tól és a `libgreet.a`-tól. A `config.h` változása mindkét objektumot invalidálja, majd a linket is. A `greet.hpp` változása szintén mindkét objektumot érinti, mert mindkét TU include-olja. Ha csak a `greet.cpp` implementációja változik, csak a `greet.o` és az `app` linkje indul újra. Ellenőrzés: a build rendszer magyarázó vagy száraz futású módjában a várt compile/link élek jelennek meg.

38.1.8.2 2. feladat megoldása

```
CC = gcc
CFLAGS = -std=c17 -Wall -Wextra -MMD -MP -Ibuild
OBJ = build/main.o
DEPS = $(OBJ:.o=.d)

all: build/app
build/app: $(OBJ)
    $(CC) $^ -o $@

build/%.o: src/%.c build/config.h | build
    $(CC) $(CFLAGS) -c $< -o $@

build/config.h: config.h.in
    mkdir -p build
    sed 's/@VERSION@/1.0/' $< > $@

build:
    mkdir -p build

-include $(DEPS)

.PHONY: all clean
clean:
    rm -rf build
```

A kulcs a `build/config.h` explicit szabálya: így az első buildnél is létezik a header, mielőtt a compile elindul. A `-MMD -MP` depfile-t generál, a `-include` pedig a következő futásra behozza a headerek függőségeit. A `| build` order-only előfeltétel a könyvtárat biztosítja anélkül, hogy annak

mtime-ja felesleges újrafordítást okozna. Ellenőrzés: tiszta build után érintsd meg a `config.h.in`-t vagy egy include-olt headert; a száraz futásnak a megfelelő compile élt kell mutatnia, tiszta builddel pedig ugyanazt a kimenetet kell kapni.

38.1.8.3 3. feladat megoldása

Az általános Make nem követi automatikusan a recept szövegét. Ha csak a `CFLAGS` változik, az objektum előfeltételei nem válnak újabbá, ezért a Make nem hívja meg a compilert; a régi objektum marad. A javítás lehet a flag-ek lenyomatának explicit függőségként való bevezetése, például egy `build/flags.stamp` fájllal, amelynek tartalma a flag-ekből származik, és amelyet az objektumok előfeltételeként használunk. Robusztusabb megoldás a CMake/Ninja használata, amely a parancs-hash változását észleli, vagy flag-profilonként külön build könyvtár. Ellenőrzés: a flagváltás után a buildnek ki kell adnia a compile parancsot; a tiszta és az inkrementális build kimenetének összevetése azonos kell legyen. A ccache/sccache ebben az esetben nem segít, mert a build rendszer el sem indítja a compilert.

38.2 Modern CMake target-alapú projekt

A build gráf, amelyet az előző részben felépítettünk, önmagában még nem mondja meg, hogy egy csomópont fordításához és linkeléséhez pontosan milyen include-útvonalak, nyelvi beállítások és linkelt könyvtárak tartoznak. A target-alapú (target-based) CMake éppen ezt a hiányt tölti be: a követelményeket nem globális változókba szórja, hanem a gráf csomópontjaihoz köti, és a gráf élein csak ott továbbítja, ahol a target szerződése ezt indokolja. Ez a különbség a régi, könyvtárszintű stílussal szemben (`include_directories`, `link_directories`, `CMAKE_CXX_FLAGS`): utóbbiak minden később létrehozott targetre hatnak, és a build gráf élei elmosódnak. A mostani alfejezet a capstone (a C záróprojekt C++ portja) core, CLI és teszt targetjein mutatja meg, hogyan lehet ezt a szerződést átvihetően, más gépen és más fogyasztónál is újrahasználhatóan megírni. A CMake pontos, verziófüggő viselkedését a hivatalos dokumentációban (cmake.org) kell ellenőrizni; az itteni állítások nem tekintendők lefuttatott mérésnek.

38.2.1 A target mint szerződéshordozó

Target: a build gráf azon csomópontja, amelynek a projektben egyedi neve van, és amelyhez a CMake vagy kimenetet (statikus/megosztott könyvtárat, futtatható programot), vagy csak követelményhalmazt (INTERFACE library) rendel. A target tulajdonságai (property-jei) határozzák meg a fordítási és linkelési parancsot, valamint azt, hogy milyen követelményeket exportál a fogyasztói felé.

Használati követelmény (usage requirement): olyan követelmény, amelyet a target a fogyasztója felé közvetít — include-könyvtár, compile feature, nyelvi standard, linkelt könyvtár, compile definition. A közvetítés feltétele, hogy a fogyasztó `target_link_libraries`-szel kapcsolódjon a targethez (vagy egy `find_package`-dzsel kapott imported targeten keresztül tegye ezt). Az `add_subdirectory` önmagában még nem terjeszti ki a követelményeket egy másik targetre; a kapcsolatot a linkelés hozza létre.

A három kulcsszó jelentése pontosan a következő:

Parancs	PRIVATE	PUBLIC	INTERFACE
`target_include_directories`	csak a saját fordítás	saját + fogyasztó	csak a fogyasztó
`target_compile_options`	csak a saját fordítás	saját + fogyasztó	csak a fogyasztó

Parancs	PRIVATE	PUBLIC	INTERFACE
`target_compile_definitions`	csak a saját fordítás	saját + fogyasztó	csak a fogyasztó
`target_compile_features`	csak a saját fordítás	saját + fogyasztó	csak a fogyasztó
`target_link_libraries`	csak a saját linkelés	saját + fogyasztó	csak a fogyasztó

A `PUBLIC` nem „fontosabb” a többinél, hanem azt jelenti, hogy a követelmény a target saját fordításában és a fogyasztóénál is megjelenik. A választás mindig abból a kérdésből következik, hogy a követelmény része-e a target nyilvános szerződésének. Ha egy headerben szerepel egy típus, amelynek az include-útvonala a fogyasztónak is kell, akkor include esetén `PUBLIC` (vagy `INTERFACE`) a helyes válasz. Ha egy warning-kapcsoló csak a saját kód minőségét szolgálja, akkor `PRIVATE`; a fogyasztóra nem tartozik.

Az `INTERFACE` library olyan target, amelynek nincs fordítási kimenete, csak követelményei vannak. Ez nem trükk, hanem a szerződés tiszta kifejezése: például a sanitizer kapcsolókészletét egy ilyen targetbe csomagolva egyetlen névvel, több targethez is hozzákapcsolhatjuk anélkül, hogy bármelyik target forráskódját módosítanánk.

A `target_*` parancsok sorrendje általában nem számít: a tulajdonságok a configure fázisban állnak össze, és a generátor a végén írja ki a build fájlokat. Ezért lehet a targetet előbb létrehozni, és a függőségét csak később deklarálni.

38.2.2 A capstone targetjei

A capstone három targetje a `diag_core` könyvtár, a `diag_cli` futtatható program és a `diag_tests` teszt. A függőségi irány egyirányú: a core nem tud a CLI-ről; a CLI és a teszt a core fogyasztója. Ez a build gráf éle, és egyben a szerződés iránya is. Az alábbi példa önállóan fordítható, ha a `src/core.cpp`, `include/diag/core.hpp`, `src/main.cpp` és `tests/test_core.cpp` fájlok a helyükön vannak.

```
// include/diag/core.hpp
#pragma once
#include <string>
namespace diag {
std::string greet(const std::string& name);
}

// src/core.cpp
#include "diag/core.hpp"
namespace diag {
std::string greet(const std::string& name) { return "hello, " + name; }
}

// src/main.cpp
#include "diag/core.hpp"
#include <iostream>
int main() { std::cout << diag::greet("devops") << '\n'; }

// tests/test_core.cpp -- NDEBUG mellett sem tűnik el
#include "diag/core.hpp"
#include <cstdio>
#include <cstdlib>
#include <string>
```

```
int main()
{
    const std::string got = diag::greet("x");
    if (got != "hello, x") {
        std::fprintf(stderr, "greet: váratlan eredmény: %s\n", got.c_str());
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

A teszt szándékosan nem `assert`-re épül. A CMake szokásos `Release` és `RelWithDebInfo` beállítása `NDEBUG`-ot definiál; ilyenkor az `assert` teljes kifejezése, a benne lévő `greet` hívással együtt eltűnik, a tesztprogram `0` kóddal kilép, és a CTest sikeresnek látja. A fenti ellenőrzés minden konfigurációban lefut. Ha egy projekt mégis `assert`-et használ a tesztekben, a teszttargeten explicit módon vissza kell vonnia az `NDEBUG`-ot, és ezt egy szándékosan hibás futással igazolnia kell.

Buildjegyzőkönyv (`lab-gcc` profil). A fenti öt fájlt CMake 4.4.4, Ninja 1.13 és GCC 13.3.0 mellett `Debug`, `Release` és `RelWithDebInfo` konfigurációban lefordítottuk és a CTesttel lefuttattuk. A teszt mindhárom konfigurációban sikeres volt; a `greet` szándékos elrontásakor (`"hullo, "`) mindháromban bukott. A korábbi, `assert`-alapú változat ugyanezzel az elrontással `Release` és `RelWithDebInfo` alatt sikeresnek mutatkozott, csak `Debug` alatt bukott.

A hozzá tartozó `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.25)
project(diag LANGUAGES CXX)
include(GNUInstallDirs)

add_library(diag_core src/core.cpp)
add_library(diag::core ALIAS diag_core)

target_include_directories(diag_core PUBLIC
  $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
  $<INSTALL_INTERFACE:${CMAKE_INSTALL_INCLUDEDIR}>)

target_compile_features(diag_core PUBLIC cxx_std_20)
set_target_properties(diag_core PROPERTIES
  CXX_STANDARD_REQUIRED ON
  CXX_EXTENSIONS OFF)

target_compile_options(diag_core PRIVATE
  "$<$<CXX_COMPILER_ID:GNU,Clang>:-Wall;-Wextra;-Wpedantic>")

add_library(diag_sanitize INTERFACE)
target_compile_options(diag_sanitize INTERFACE
    ↪  "$<$<CXX_COMPILER_ID:GNU,Clang>:-fsanitize=address,undefined;-fno-omit-frame-pointer>"
target_link_options(diag_sanitize INTERFACE
  "$<$<CXX_COMPILER_ID:GNU,Clang>:-fsanitize=address,undefined>")

option(DIAG_SANITIZE "Enable ASan/UBSan" OFF)
```

```
if(DIAG_SANITIZE)
  # A core saját fordítási egységei is instrumentáltak legyenek: a tesztelt
  # kód a core-ban van. A PRIVATE kapcsolat a fordítási kapcsolókat a core-ra
  # teszi; a linkkapcsolókat a futtatható targetek saját kapcsolata adja.
  target_link_libraries(diag_core PRIVATE diag_sanitize)
endif()

add_executable(diag_cli src/main.cpp)
target_link_libraries(diag_cli PRIVATE diag::core)
if(DIAG_SANITIZE)
  target_link_libraries(diag_cli PRIVATE diag_sanitize)
endif()

include(CTest)
if(BUILD_TESTING)
  add_executable(diag_tests tests/test_core.cpp)
  target_link_libraries(diag_tests PRIVATE diag::core)
  if(DIAG_SANITIZE)
    target_link_libraries(diag_tests PRIVATE diag_sanitize)
  endif()
  add_test(NAME core_greet COMMAND diag_tests)
endif()
```

Az indoklás a három döntés mögött: az include-könyvtár `PUBLIC`, mert a `core.hpp`-t a fogyasztó is include-olja, tehát a fogyasztónak is látnia kell a header helyét. A warning-kapcsolók `PRIVATE`-ek, mert a mi minőségpolitikánk nem része a könyvtár szerződésének; a fogyasztó más figyelmeztetési szintet használhat. A `cxx_std_20` `PUBLIC`, mert a core headerje C++20-at igényel, tehát a fogyasztó sem fordíthatja régebbi módban. A `BUILD_INTERFACE` és `INSTALL_INTERFACE` szétválasztása pedig azt biztosítja, hogy build közben a forrásfából, telepítés után a telepített include-könyvtárból dolgozzon a fogyasztó.

38.2.3 Nyelvi standard és extension policy

A standard beállításának három szintje van: a globális `CMAKE_CXX_STANDARD`, a target `CXX_STANDARD` tulajdonsága és a `target_compile_features` által kért compile feature. A harmadik a legjobban illeszkedik a target-alapú modellhez, mert nem konkrét flaget, hanem nyelvi követelményt fejez ki: a CMake választja ki a toolchainhez tartozó kapcsolót. A `CXX_STANDARD_REQUIRED ON` megakadályozza, hogy a compiler csendben visszaessen egy régebbi módra, a `CXX_EXTENSIONS OFF` pedig kizárja a GNU-specifikus kiterjesztéseket. C esetén ugyanez a minta: `C_STANDARD`, `C_STANDARD_REQUIRED`, `C_EXTENSIONS OFF`, illetve `c_std_17` compile feature.

A C és C++ targetjeit külön kell profilozni. Ha a projekt mindkét nyelvet használja, a `project(diag LANGUAGES C CXX)` deklaráció után a C target `c_std_17`-et, a C++ target `cxx_std_20`-at kap; a C targetre nem kerülhet `cxx_std_*`, és fordítva. A compile feature csak nyelvi képességet fejez ki; a library-szintű követelményeket (például egy adott header elérhetőségét) ez nem garantálja, azokat külön compile probe-dal vagy feature-test makróval kell ellenőrizni — erre a korábbi részek adnak mintát.

38.2.4 Függőségek és linkhatárok

A `target_link_libraries(diag_cli PRIVATE diag::core)` azt jelenti, hogy a CLI linkel a core-hoz, de a core-t nem terjeszti tovább, mert a CLI-nek nincs fogyasztója. Ha a CLI egy headerjében szerepelne a core valamely típusa, a kapcsolat `PUBLIC` lenne. A szabály nem esztétikai: a `PRIVATE` linkelt függőség nem kerül be az exportált `diagTargets.cmake` használati követelményei közé, míg a `PUBLIC` és az `INTERFACE` igen.

A tranzitivitás tipikus esete: a core a nyilvános `core.hpp`-ban egy harmadik fél `foo` targetjének típusát használja, a `core.cpp`-ben pedig egy `bar` könyvtárat. Ekkor `foo` esetén `target_link_libraries(diag_core PUBLIC foo)` a helyes, mert a fogyasztónak is látnia kell a `foo` include-útvonalait és linkjét; `bar` esetén `PRIVATE` a helyes, mert az csak a megvalósítás részlete. A statikus vagy megosztott könyvtár választása (`BUILD_SHARED_LIBS`, illetve `add_library(diag_core SHARED ...)`) ettől független döntés. A linkhatár ABI-kérdés is: a CMake nem ellenőrzi, hogy a két oldal ugyanazzal az ABI-val, ugyanazzal a standard library-vel és ugyanazzal az allocatorral fordult-e; ezt a korábbi ABI- és FFI-labor tárgyalja (ABI- és FFI-labor).

38.2.5 Debug-, release- és sanitizer-profilok

Több konfigurációs generátor (például a `Ninja Multi-Config`) esetén a konfiguráció nem a configure, hanem a build idején dől el. Ezért nem szabad `CMAKE_BUILD_TYPE`-ra építeni: az ilyen generátornál csendben figyelmen kívül marad. A profilokat targetenként, feltételes generátorkifejezéssel (generator expression) lehet kezelni; a CMake beépített Debug/Release kapcsolóit azonban csak indokolt esetben érdemes kézzel felülírni.

A sanitizer beállítás mintája a fenti `diag_sanitize` INTERFACE target. Miért nem elég egy `PRIVATE` compile option? Azért, mert a sanitizer runtime-nak a linkelésnél is jelen kell lennie, tehát a compile és a link követelményét együtt kell hordozni, és minden érintett targethez hozzá kell kapcsolni. „Minden érintett” itt a core-t is jelenti: ha a `diag_sanitize` csak a CLI-hez és a teszthez kapcsolódna, a teszt instrumentáltan futna, de a tesztelt `greet` a core nem instrumentált fordítási egységében lenne, és az ott elkövetett memóriahibát az ASan nem látná.

Buildjegyzőkönyv (`lab-gcc` profil, CMake 4.4.4, `-DDIAG_SANITIZE=ON -DCMAKE_EXPORT_COMPILE_COMMANDS=`
A `compile_commands.json` szerint a javított konfigurációban a `core.cpp`, a `main.cpp` és a `test_core.cpp` is `-fsanitize=address,undefined` kapcsolóval fordul; a core nélküli változatban a `core.cpp` nem. Egy szándékosan a `greet`-be tett, a kimenetet nem befolyásoló tömbön kívüli olvasást a régi konfiguráció tesztje átengedett (`100% tests passed`). A javítottban a teszt `AddressSanitizer: stack-buffer-overflow` hibával bukott, a hiba helyével a `core.cpp`-ben. A sanitizeres build külön configure- és build-könyvtárban vagy külön presetben fusson, és a release csomagba ne kerülhessen be: a sanitizerrel linkelt bináris nem azonos a kiadott binárissal.

38.2.6 Install interface és generált config

A buildkövetelmények csak addig hasznosak, amíg a projekt forrásfájában vagyunk. A telepített csomagot a `install(TARGETS ... EXPORT ...)`, az `install(EXPORT ... NAMESPACE diag:: ...)`, valamint a `CMakePackageConfigHelpers` modul `configure_package_config_file` és `write_basic_package_version_file` parancsai készítik el. A generált `diagConfig.cmake` és `diagTargets.cmake` együtt teszi lehetővé, hogy a fogyasztó `find_package(diag CONFIG REQUIRED)` után `diag::core`-t linkeljen. Az export a `PUBLIC` és `INTERFACE` követelményeket viszi, a `PRIVATE`-eket nem; ezért kell az include-útvonalat a `$<BUILD_INTERFACE:...>` és

`$<INSTALL_INTERFACE:...>` párossal szétválasztani. Az `INSTALL_INTERFACE` útvonala a telepítési prefixhez képest relatív legyen; abszolút útvonal beírása nem hordozható.

A header-telepítés régebbi mintája az `install(DIRECTORY include/ DESTINATION ...)`; az újabb CMake-verziókban a `FILE_SET HEADERS` (a CMake 3.23 körüli kiadásoktól) ezt targethez kötötten oldja meg. Melyik verziótól melyik érhető el, azt a telepített CMake dokumentációjában kell ellenőrizni; a fejezet nem állítja, hogy a kód bármelyik konkrét kiadáson futott.

38.2.7 CMake Presets és több konfigurációs generátor

A `CMakePresets.json` a configure-, build- és test-beállításokat nevesíti, és verziókövethető fájlban tartja, így a „hogyan kell ezt a projektet buildelni” tudás nem a fejekben és nem a CI-szkriptben szétszórva él. Egy minimális példa:

```
{
  "version": 6,
  "configurePresets": [
    {
      "name": "dev-debug",
      "generator": "Ninja Multi-Config",
      "binaryDir": "${sourceDir}/build",
      "cacheVariables": {
        "CMAKE_CONFIGURATION_TYPES": "Debug;Release",
        "DIAG_SANITIZE": "ON"
      }
    }
  ],
  "buildPresets": [
    { "name": "dev-debug", "configurePreset": "dev-debug", "configuration":
    ↪  "Debug" }
  ],
  "testPresets": [
    {
      "name": "dev-debug",
      "configurePreset": "dev-debug",
      "configuration": "Debug",
      "output": { "outputOnFailure": true }
    }
  ]
}
```

A hívás:

```
cmake --preset dev-debug
cmake --build --preset dev-debug
ctest --preset dev-debug
```

A presets séma `"version"` mezője a CMake verziójához kötött; a 6-os séma a 3.25 körüli kiadásokhoz tartozik, ezért a telepített CMake-hez kell igazítani, és a dokumentációban ellenőrizni. A `ctest --test-dir` szintén újabb CTest-verziókhoz kötött; régebbi környezetben a build könyvtárba kell lépni. **Buildjegyzőkönyv** (`lab-gcc` profil). A fenti presets-fájllal a három parancs CMake 4.4.4 és Ninja 1.13 mellett lefutott: a `Ninja Multi-Config` generátor `Debug` és `Release` konfigurációt

készített, a Debug build DIAG_SANITIZE=ON miatt ASan/UBSan kapcsolókkal fordult, és a CTest egyetlen tesztje sikeres volt.

38.2.8 Tipikus hibák és javításuk

1. **Globális include_directories és CMAKE_CXX_FLAGS.** Minden target örökli, a gráf élei elmosódnak, és a később hozzáadott target is váratlanul megkapja. Javítás: target_include_directories és target_compile_features a konkrét targeten.
2. **PRIVATE include egy nyilvános headernél.** A fogyasztó nem találja a diag/core.hpp-t. Javítás: PUBLIC, mert a header a szerződés része.
3. **PUBLIC link ott, ahol elég lenne PRIVATE.** A fogyasztó fölöslegesen örökli a függőséget, és az export is tartalmazza. Javítás: csak a nyilvános headerben megjelenő típusok függősége legyen PUBLIC.
4. **-std=c++20 a CMAKE_CXX_FLAGS-ben.** Nem target-specifikus, és felülírhatja a CMake által választott kapcsolót. Javítás: target_compile_features(... cxx_std_20).
5. **CMAKE_BUILD_TYPE több konfigurációs generátornál.** Csendben figyelmen kívül marad. Javítás: --config a buildnél, illetve presets configuration mezője.
6. **Sanitizer flag a release targetre.** A kiadott bináris nem reprodukálható és sanitizer runtime nélkül nem futtatható. Javítás: külön preset és külön build-könyvtár.
7. **A generált diagConfig.cmake kézi szerkesztése a build-könyvtárban.** A következő configure felülírja. Javítás: a .in template és a CMake parancsok módosítása.

38.2.9 Feladatok

1. feladat (magyarázó). A diag_core a nyilvános core.hpp-ban egy foo könyvtár foo::handle típusát használja, a core.cpp-ben pedig egy bar könyvtárat. Melyik függőség legyen PUBLIC, melyik PRIVATE, és miért?

2. feladat (kódolási). Alakítsd át a példát úgy, hogy a core headerje C++17-kompatibilis legyen, a CLI viszont C++20-at igényeljen. Írd át a CMakeLists.txt érintett sorait, és indokold, miért maradhat a core követelménye PUBLIC.

3. feladat (diagnosztikai). A fogyasztó a find_package(diag CONFIG REQUIRED) után ezt írja: target_link_libraries(app PRIVATE diag::core), de a fordító nem találja a diag/core.hpp-t. Sorolj fel legalább három lehetséges okot, és add meg az ellenőrzés lépéseit.

38.2.10 Megoldások

1. megoldás. A foo legyen PUBLIC: a típusa a nyilvános headerben szerepel, tehát a fogyasztónak is látnia kell a foo include-útvonalát és linkjét. A bar legyen PRIVATE: csak a megvalósításban kell, a fogyasztó szerződésének nem része. Ellenőrzési feltétel: a telepített diagTargets.cmake-ben a foo INTERFACE_LINK_LIBRARIES bejegyzésként jelenik meg, a bar nem.

2. megoldás. A core-nál target_compile_features(diag_core PUBLIC cxx_std_17), a CLI-nél target_compile_features(diag_cli PRIVATE cxx_std_20). A CXX_STANDARD_REQUIRED ON és a CXX_EXTENSIONS OFF marad. A core követelménye azért maradhat PUBLIC, mert a header C++17-tel is fordul, tehát a fogyasztónak nem kell C++20-at ígérni; a CLI saját magának kérhet többet. Ellenőrzési feltétel: a compile_commands.json-ban a core object fájljához tartozó parancs C++17-es, a CLI-é C++20-as módot mutat — ez a tanuló laborjának mérése, nem a fejezet állítása.

3. megoldás. Lehetséges okok: (a) a core include-könyvtára `PRIVATE`-ként van beállítva, ezért nem kerül az exportba; (b) az `INSTALL_INTERFACE` útvonala hibás vagy abszolút, ezért a telepített config rossz helyre mutat; (c) a fogyasztó egy régi telepítést talál a `diag_DIR` miatt, nem a frisset; (d) a header nem került telepítésre. Ellenőrzés: `cmake --install build --config Release --prefix /tmp/diag-install`, majd a telepített `diagTargets.cmake` `INTERFACE_INCLUDE_DIRECTORIES` bejegyzésének megtekintése, végül a fogyasztó `CMakeCache.txt`-jében a `diag_DIR` értékének ellenőrzése. A hiba akkor javul, ha az include `PUBLIC`/`INTERFACE` módon, helyes relatív `INSTALL_INTERFACE` útvonallal és telepített headerrel szerepel.

38.2.11 Összefoglaló

A target-alapú CMake lényege, hogy a buildkövetelmények nem globális mellékhatásként, hanem targethez kötött szerződésként jelennek meg. A `PRIVATE`, `PUBLIC` és `INTERFACE` választása mindig abból a kérdésből következik, hogy a követelmény része-e a target nyilvános szerződésének. A nyelvi standardot és a compile feature-t targetenként, explicit módon kell rögzíteni, a C és C++ targeteket külön profilozva. A sanitizer és a debug/release különbségek INTERFACE targetekkel és presetekkel kezelhetők anélkül, hogy a release csomag szennyeződne. Az install export és a generált config viszi tovább ugyanezeket a követelményeket a fogyasztóhoz; a build- és install-interface szétválasztása nélkül a telepített csomag nem lesz hordozható. A CMake verziófüggő részleteit minden esetben a telepített kiadás dokumentációjában kell ellenőrizni, és a fordítási eredményeket mérés, nem feltevés alapján kell állítani.

38.3 Függőségek, package discovery és linkhatárok

A függőségkezelés a natív build egyik legkönnyebben elrontott része. Nem elég azt elérni, hogy a fordítás és a linkelés egyszer működjön: a függőség verziója, bináris formája, tranzitív linkigénye, ABI-feltétele és jogi információja is része a build szerződésének. Ez a rész a Modern CMake target-alapú projekt (38.2) target-fogalmára épül, és a bináris határ kérdéseit a Stabil shared library határ és verziózás (36.3) fogalmaival köti össze. A cél nem egy univerzális csomagkezelő kiválasztása, hanem annak eldöntése, hogy egy adott függőséget hogyan lehet reprodukálhatóan megtalálni, linkelni és frissíteni.

38.3.1 A függőségi szerződés szintjei

Függőség (dependency) alatt itt minden olyan artefaktumot értünk, amelyre a projekt buildje vagy futása támaszkodik: fejléc, statikus vagy shared library, generált fájl, esetleg egy teljes eszközlánc. A **discovery** annak mechanizmusa, ahogy a build rendszer megkeresi a függőség fejléceit, linkelhető binárisát és használati követelményeit. Az **imported target** olyan CMake-target, amelyet nem ez a projekt épít, hanem egy `find_package` vagy egy modul ír le; tulajdonságai között szerepelhet az include-könyvtár, a linkelendő library, a szükséges nyelvi szint és a tranzitív linkigény.

A **package config** a függőség által szállított CMake-fájl, jellemzően `<Pkg>Config.cmake` és `<Pkg>ConfigVersion.cmake`, amelyet a `find_package(<Pkg> CONFIG)` keres. A **pkg-config** ezzel szemben `.pc` fájlokon alapuló, több nyelv által használt metadata-lekérdezés; a CMake `PkgConfig` modulja ezt fordítja targetté. Az **ABI** (application binary interface) a bináris kompatibilitás feltételrendszere: hívási konvenció, névtorzítás, objektumlayout, kivételkezelés, allokátor és standard library viselkedése. A **tranzitív link** azt jelenti, hogy egy library linkelési igénye továbbgyűrűzik a fogyasztóhoz; ez static és shared esetben eltérően viselkedik.

38.3.2 find_package és imported target

A `find_package` két fő módja a `MODULE` és a `CONFIG`. A `MODULE` mód a projekt saját `Find<Package>.cmake` fájlját futtatja, ami hasznos régi vagy nem CMake-alapú függőségeknél, de gyakran kézzel karbantartott, pontatlan felderítést ad. Modern függőségeknél a `CONFIG` mód a jobb választás, mert a csomag saját, telepített metadata-fájljait használja:

```
find_package(fmt CONFIG REQUIRED)
add_executable(app src/main.cpp)
target_link_libraries(app PRIVATE fmt::fmt)
```

A `fmt::fmt` név konvenció: a névtér a csomag neve, a target neve a tényleges library. A verzió megadható például `find_package(fmt 10 CONFIG REQUIRED)` formában; a pontos verziószám itt csak szemléltetés, a konkrét projektnél a ténylegesen támogatott verziót kell rögzíteni. A verziókompatibilitást a csomag `ConfigVersion.cmake` fájlja dönti el; gyakori stratégiák az `AnyNewerVersion`, a `SameMajorVersion`, a `SameMinorVersion` és az `ExactVersion`. A `SameMajorVersion` például azt jelenti, hogy egy 1.x függőséghez a fogyasztó 1.y-t is elfogad, de 2.0-t nem.

Az imported target előnye, hogy a használati követelmények egy helyen vannak. Ha a fogyasztó kézzel írja be az include-könyvtárat és a `-l` kapcsolót, elveszíti a fordítási definíciókat, a szükséges nyelvi szintet, a tranzitív linkigényeket és a konfigurációspecifikus útvonalakat. Ezért a cél: `target_link_libraries` egy imported targettel, ne `include_directories` és `link_directories` kombinációja.

38.3.3 pkg-config és CMake package config

A pkg-config akkor hasznos, ha a függőség nem szállít CMake package configot, de szállít `.pc` fájlt. A CMake-oldali használat:

```
find_package(PkgConfig REQUIRED)
pkg_check_modules(TEXTUTIL REQUIRED IMPORTED_TARGET textutil)
target_link_libraries(app PRIVATE PkgConfig::TEXTUTIL)
```

Az `IMPORTED_TARGET` kulcs fontos: enélkül csak változókat kapunk (`TEXTUTIL_CFLAGS`, `TEXTUTIL_LIBS`), amelyeket kézzel kellene összefűzni, és elveszne a target-alapú következetesség. A `.pc` fájl jellemzően `prefix`, `includedir`, `libdir`, `Cflags` és `Libs` mezőket tartalmaz; ezek a telepítési prefixhez kötődnek, ezért a `.pc` fájl áthelyezése vagy a prefix megváltoztatása könnyen hibás útvonalat eredményez.

A CMake package config erősebb szerződés, mert nem csak kapcsolókat, hanem targeteket, tulajdonságokat és verziókompatibilitást is leír. Hátránya, hogy a csomagnak telepítenie kell a config fájlt, és a fogyasztónak a keresési útvonalat (`CMAKE_PREFIX_PATH`, `CMAKE_FIND_ROOT_PATH`) helyesen kell beállítania. Cross-compilationnál ez külön figyelmet igényel: a host eszközöket és a target library-ket nem szabad összekeverni.

38.3.4 Statikus, shared és header-only integráció

A három integrációs forma nem csupán linkelési kényelem kérdése, hanem eltérő élettartam-, ABI- és telepítési következményekkel jár.

Forma	Mit szállít	Linkelés	ABI-kitettség	Tipikus következmény
Statikus	`.a` vagy `.lib`	a végső binárisba épül	a kód a fogyasztó ABI-jával fordul	tranzitív static függőségek a végső linkhez kellenek; PIC szükséges sharedbe építéshez
Shared	`.so` vagy `.dll`, soname-mal	futásidőben töltődik	a library és a fogyasztó ABI-ja illeszkedjék	`DT_NEEDED`, rpath, verziózott szimbólumok, futásidejű telepítés
Header-only	fejlécek	nincs külön linkelendő bináris	inline definíciók és makrók viszik be az ABI-t	ODR-fegyelem, fordítási idő, makrókonfiguráció-érzékenység

Statikus librarynél a fogyasztó linkparancsába bekerülnek a library saját, nem publikus link-igényei is, mert a statikus librarynek nincs saját linkelési lépése. A CMake ezt jellemzően `$<LINK_ONLY:...>` formában továbbítja. Shared librarynél a dinamikus loader oldja fel a függőségeket; itt a soname, az rpath és a szimbólumláthatóság válik kritikussá. Header-only esetben nincs linkelendő artefaktum, de ez nem jelenti, hogy nincs ABI-kapcsolat: az inline függvények, osztálydefiníciók és makrók a fogyasztó fordítási egységébe kerülnek, ezért eltérő makróbeállítás vagy eltérő standard library-verzió akár ODR-sértést vagy viselkedésbeli eltérést okozhat.

A **tranzitív link** CMake-ben a `PRIVATE`, `PUBLIC` és `INTERFACE` kulcsszavakkal szabályozható:

Kulcsszó	Saját link	Fogyasztó felé öröklődik
`PRIVATE`	igen	nem, statikus librarynál link-only formában mégis megjelenhet
`PUBLIC`	igen	igen
`INTERFACE`	nem	igen

A `PUBLIC` tehát nem „jobb” a `PRIVATE`-nál: a `PUBLIC` kiszélesíti a felületet, és a fogyasztót is a függőséghez köti. A helyes választás attól függ, hogy a függőség típusai megjelennek-e a saját nyilvános fejlécben. Ha igen, `PUBLIC`; ha csak a megvalósításban, `PRIVATE`.

Az ABI/compiler/stdlib kompatibilitás feltételei C++ esetben legalább a következők lehetnek: azonos C++ ABI-család, azonos standard library és annak ABI-konfigurációja (például libstdc++ vagy libc++), azonos `-std=` és fontosabb nyelvi kapcsolók, azonos kivétel- és RTTI-beállítások, azonos debug/release és iterator-debug szint, azonos sanitizer-instrumentáltság, valamint azonos struct packing/alignment opciók. A libstdc++ esetében például a `_GLIBCXX_USE_CXX11_ABI` eltérése külön bináris ABI-t eredményezhet. Ezek a feltételek nem teljes listák; a konkrét toolchain dokumentációja és a tényleges bináris vizsgálata dönt. C-oldali határon az `extern "C"` megszünteti a névtorzítást, de a hívási konvenció, a struct-layout és az alignment továbbra is szerződés része marad.

38.3.5 Beszerzés és rögzítés: FetchContent, vcpkg, Conan

A **FetchContent** forrásszinten hozza be a függőséget a buildbe. Előnye, hogy a függőség ugyanazzal a compilerrel és ugyanazokkal a kapcsolókkal fordul, mint a projekt; hátránya, hogy a konfigurációs lépés hálózatot és plusz fordítási időt igényelhet, valamint a függőség saját buildje bekerül a projekt buildgráfjába. Rögzítés nélkül nem reprodukálható:

```
include(FetchContent)

FetchContent_Declare(
  textutil
  SOURCE_DIR ${CMAKE_CURRENT_SOURCE_DIR}/vendor/textutil
)

FetchContent_MakeAvailable(textutil)
```

Hálózati letöltésnél a `GIT_TAG` értéke lehetőleg commit hash legyen, ne mozgatható tag; URL-es letöltésnél `URL_HASH SHA256=...` szükséges. A konkrét forrás URL-jét és hash-ét a projekt saját, ellenőrzött dependency-jegyzékéből kell venni; ez a fejezet nem rögzít third-party verziót.

A **vcpkg** és a **Conan** magasabb szintű csomagkezelést ad: manifestet, verziófeloldást, bináris cache-t, triplet- vagy setting-alapú fordítási konfigurációt. A vcpkg manifest esetében a `builtin-baseline` és a triplet rögzíti a kiinduló állapotot; Conan esetében a lockfile, a recipe-revízió és a compiler/stdlib setting teszi reprodukálhatóvá a feloldást. Egyik eszköz sem menti meg a projektet attól, hogy a verziótartományt lock nélkül használja: a „legújabb kompatibilis” feloldás nem reprodukálható build. A konkrét manifest-séma és mezőnevek verziófüggők, ezért a használt kiadás dokumentációját kell ellenőrizni.

38.3.6 License és attribution megőrzése

A függőség forráscsomagjában maradjon meg a licenc és a hozzá tartozó attribution: `LICENSE`, `COPYING`, `NOTICE`, valamint a szerzői és származási információk. Hasznos egy `ORIGIN.txt` a függőség könyvtárában a forrás, a verzió, a hash és a beszerzés dátumával. SPDX-azonosító használható a gépi feldolgozáshoz, de a jogi tartalmat a licenc szövege adja. Statikus linkelésnél és shared librarynél eltérő kötelezettségek merülhetnek fel; ezeket nem a buildrendszer dönti el, hanem a licenc. A fejezet nem ad jogi tanácsot: a konkrét licencszöveget és a projekt jogi policy-ját kell követni.

38.3.7 Kidolgozott példa: saját package config és fogyasztó

Az alábbi példa egy saját `textutil` libraryt telepít CMake package configként, majd egy fogyasztó `find_package`-pel keresi, és ha nem találja, forrásból építi be. A kód illusztráció; a fordítás és futtatás tényét ez a szöveg nem állítja.

`third_party/textutil/include/textutil/textutil.hpp`:

```
#pragma once
#include <string>
#include <vector>

namespace textutil {
std::string join(const std::vector<std::string>& parts, const std::string& sep);
}
```

third_party/textutil/src/textutil.cpp:

```
#include "textutil/textutil.hpp"

namespace textutil {
std::string join(const std::vector<std::string>& parts, const std::string& sep)
 ↪  {
    std::string out;
    for (std::size_t i = 0; i < parts.size(); ++i) {
        if (i != 0) out += sep;
        out += parts[i];
    }
    return out;
}
}
```

third_party/textutil/CMakeLists.txt:

```
cmake_minimum_required(VERSION 3.24)
project(textutil VERSION 1.2.0 LANGUAGES CXX)

add_library(textutil src/textutil.cpp)
add_library(textutil::textutil ALIAS textutil)

target_include_directories(textutil
  PUBLIC
    $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
    $<INSTALL_INTERFACE:include>)
target_compile_features(textutil PUBLIC cxx_std_20)

include(GNUInstallDirs)
include(CMakePackageConfigHelpers)

install(TARGETS textutil EXPORT textutilTargets
  ARCHIVE DESTINATION ${CMAKE_INSTALL_LIBDIR}
  LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
  RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR})
install(DIRECTORY include/ DESTINATION ${CMAKE_INSTALL_INCLUDEDIR})
install(EXPORT textutilTargets
  NAMESPACE textutil::
  DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/textutil)

configure_package_config_file(
  cmake/textutilConfig.cmake.in
  ${CMAKE_CURRENT_BINARY_DIR}/textutilConfig.cmake
  INSTALL_DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/textutil)
write_basic_package_version_file(
  ${CMAKE_CURRENT_BINARY_DIR}/textutilConfigVersion.cmake
  VERSION ${PROJECT_VERSION}
  COMPATIBILITY SameMajorVersion)
install(FILES
  ${CMAKE_CURRENT_BINARY_DIR}/textutilConfig.cmake
```

```
  ${CMAKE_CURRENT_BINARY_DIR}/textutilConfigVersion.cmake
  DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/textutil)
```

A `cmake/textutilConfig.cmake.in` minimális tartalma:

```
@PACKAGE_INIT@
include("${CMAKE_CURRENT_LIST_DIR}/textutilTargets.cmake")
check_required_components(textutil)
```

A fogyasztó `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.24)
project(depdemo LANGUAGES CXX)

find_package(textutil 1.2 CONFIG QUIET)
if(NOT textutil_FOUND)
  add_subdirectory(third_party/textutil)
endif()

add_executable(depdemo app/main.cpp)
target_link_libraries(depdemo PRIVATE textutil::textutil)
```

`app/main.cpp`:

```
#include <textutil/textutil.hpp>
#include <iostream>

int main() {
    std::cout << textutil::join({"a", "b", "c"}, ",") << '\n';
}
```

Build- és telepítési parancsok:

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build
cmake --install build --prefix "$PWD/stage"
cmake -S . -B build-installed -G Ninja -DCMAKE_PREFIX_PATH="$PWD/stage"
cmake --build build-installed
```

A második konfigurálásnál a `find_package` a telepített `textutilConfig.cmake`-t találja meg; az elsőnél a `QUIET` miatt csendben a `add_subdirectory` ágra esik. A várt kimenet `a,b,c`, de ez illusztráció, nem mért eredmény. A `SameMajorVersion` választás tudatos: a fogyasztó 1.x-en belül frissíthet, 2.0-nál újra kell vizsgálni az ABI-t és a forráskompatibilitást.

38.3.8 Tipikus hibák és javításuk

Az egyik gyakori hiba, hogy a projekt csak include-könyvtárat ad hozzá, de nem linkel imported targetet. Ilyenkor a fejléc fordul, a linkelés viszont hiányzó szimbólumnál elhasal, vagy ami rosszabb, egy másik, ABI-inkompatibilis bináris kerül a linkparancsba. Javítás: `find_package` és `target_link_libraries` a csomag által kínált targettel.

A második hiba a verzió nélküli `find_package`. Ez csendben más verziót oldhat fel a fejlesztői gépen és a CI-ben. Javítás: verziómegkötés és a csomag verziókompatibilitási policy-jének tudatos megválasztása, valamint a feloldott verzió naplózása.

A harmadik hiba a `PUBLIC` indokolatlan használata. Ettől a fogyasztó is linkelni fogja a függőséget, nő a bináris felület és az ABI-kitettség. Javítás: a nyilvános fejlécben megjelenő típusoknál `PUBLIC`, a csak megvalósításban használt függőségnél `PRIVATE`.

A negyedik hiba a különböző standard library vagy compiler-beállítás összekeverése. Például az egyik objektum debug iteratorokkal, a másik anélkül fordul, vagy eltér a `_GLIBCXX_USE_CXX11_ABI` értéke. Javítás: egységes toolchain-profil, ABI-ellenőrzés a binárison, és a függőségek forrásból fordítása ott, ahol ez indokolt.

Az ötödik hiba a FetchContent rögzítés nélküli használata. Egy mozgó branch vagy tag mellett a build idővel más kódot fordf. Javítás: commit hash vagy `URL_HASH`, és a beszerzett forrás hash-ének rögzítése a forráscsomagban.

38.3.9 Összefoglaló

A függőség nem csak egy `-l` kapcsoló. A discovery módja, az imported target használati követelményei, a statikus/shared/header-only forma, a tranzitív link és az ABI-feltételek együtt adják a szerződést. A reprodukálhatóságot a verzió és a forrás rögzítése, valamint a lockolt feloldás adja; a jogi megfelelést a licenc- és attribution-adatok megőrzése. A CMake target-alapú modellje akkor működik jól, ha a függőségeket targetként, nem globális változóként kezeljük.

38.3.10 Feladatok

1. Magyarázd el, miért jobb a `target_link_libraries(app PRIVATE fmt::fmt)` forma, mint az `include_directories(...)`, `link_directories(...)` és `target_link_libraries(app PRIVATE fmt)` kombinációja. Adj legalább három, a build helyességéhez és a karbantarthatósághoz kapcsolódó érvet.

2. Egy `app` executable egy `A` statikus libraryt linkel. Az `A` a saját `CMakeLists.txt`-jében `PRIVATE` módon használja `B`-t. A fogyasztó linkelése mégis „undefined reference” hibát ad `B` szimbólumaira. Magyarázd meg, miért történik ez, és add meg a helyes javítást. Miért nem jó megoldás, ha `B`-t egyszerűen `PUBLIC`-ra váltjuk?

3. Tervezz dependency-policy-t egy projekthez, amely egy header-only libraryt, egy shared libraryt és egy statikus libraryt használ. A policy térjen ki a verzió-rögzítésre, a discovery módjára, az ABI-ellenőrzésre és a licencadatok megőrzésére. Adj hozzá CMake-snippetet is.

38.3.11 Megoldások

1. Az imported target egy helyen tartalmazza az include-könyvtárat, a szükséges fordítási definíciókat, a nyelvi szintet és a tranzitív linkigényeket. A kézi változók ezt szétszórják, és a fogyasztó könnyen kihagy egy követelményt. Az imported target kezeli a konfigurációspecifikus útvonalakat és a telepített vagy forrásból épített változat közötti különbséget. Végül a target-alapú forma jobban ellenőrizhető: a linkparancs és a tulajdonságok egy objektumban vannak, nem globális állapotban. Ellenőrzési feltétel: a fogyasztó `CMakeLists.txt`-jében ne szerepeljen kézi include- és linkútvonal a függőséghez.

2. A statikus librarynek nincs saját linkelési lépése, ezért a `PRIVATE` linkigénye nem tűnik el teljesen: a CMake a fogyasztó végső linkjéhez `$<LINK_ONLY:B>` formában továbbítja. Ha a fogyasztó mégis hiányzó szimbólumot lát, az vagy abból ered, hogy a `B` nem érhető el a linkparancsban, vagy abból, hogy a `B` nem ugyanazzal az ABI-val készült. A helyes javítás: a `B` targetjét elérhetővé tenni az `A` exportjában, illetve a fogyasztó linkparancsában; ha a `B`

csak implementációs részlet, maradjon `PRIVATE`, de a statikus linkhez szükséges formában. A `PUBLIC` azért nem jó, mert a `B` típusai nem jelennek meg az `A` nyilvános fejlécében, így feleslegesen kiterjesztené a felületet és a fogyasztó ABI-kitettségét. Ellenőrzési feltétel: a fogyasztó linkelése sikeres, és az `A` nyilvános fejlécében nem jelenik meg `B`-típus.

3. Példapolicy: a header-only függőséget rögzített commit hash-sel vagy verziózott forráscsomaggal kell behozni, és dokumentálni kell a szükséges makrókat. A shared libraryt `find_package(... CONFIG)`-pal kell keresni, verziómegkötéssel és `SameMajorVersion` policy-val; a telepítésnél ellenőrizni kell a soname-et és az rpath-t. A statikus libraryt szintén targetként kell linkelni, és a tranzitív igényeit a CMake exportjában kell átadni. Az ABI-ellenőrzéshez a projekt rögzítse a compilert, a standard libraryt, a `-std=` értéket és a fontos ABI-kapcsolókat. A licencadatok a `vendor/<name>/` könyvtárban maradjanak, `LICENSE`, `NOTICE` és `ORIGIN.txt` formában. CMake-részlet:

```
find_package(sharedlib 2 CONFIG REQUIRED)
add_subdirectory(vendor/headeronly)
add_subdirectory(vendor/staticlib)

add_executable(app src/main.cpp)
target_link_libraries(app PRIVATE
  sharedlib::sharedlib
  headeronly::headeronly
  staticlib::staticlib)
```

Ellenőrzési feltétel: a build három különböző integrációs formát használ, a verziók rögzítettek, és a forráscsomag tartalmazza a licenc- és származási adatokat. A példa nem állítja, hogy a konkrét függőségek elérhetők vagy lefordultak; a verziókat és az ABI-kapcsolókat a választott toolchain dokumentációja alapján kell ellenőrizni.

38.4 CTest, negatív tesztek és sanitizer CI

A CI akkor hasznos minőségi kapu, ha minden állítása visszavezethető egy reprodukálható futtatásra. A „zöld futás” önmagában nem bizonyíték: a C és C++ viselkedése függ a nyelvi módtól, a compiler és a standard library verziójától, a target ABI-tól, a build típusától és attól, hogy milyen diagnosztikai runtime fut. Ezért a CI-t nem tesztparancsok gyűjteményeként, hanem konfigurációk és elvárt eredmények szerződéseként érdemes felépíteni. Az alábbiak a Függőségek, package discovery és linkhatárok (38.3) függőség- és linkhatárkezelésére, az Unit test, property test és fuzzing (16.3) tesztelési alapjaira, a Hibakezelési és exception safety labor (25.5) hibakezelési laborjára és a shared ownership, reclamation és konkurens review (34.6) konkurens élettartam-vizsgálatára épülnek. A CMake, GCC, Clang és binutils hivatalos dokumentációja a kiindulópont; a konkrét verziókat és kapcsolókat a laborprofilban kell rögzíteni, és a példák itt illusztrációk, nem lefuttatott mérések.

38.4.1 A tesztmátrix mint minőségi szerződés

A **tesztmátrix** a tesztkonfigurációk rendezett halmaza. Egy cellát a nyelvi standard, a compiler és verziója, a target ABI, a build típus, a sanitizer profil, a standard library és a szükséges külső komponensek azonosítanak. A cellához nem pusztán az tartozik, hogy „fut”, hanem egy elvárt eredmény: `pass`, `expected-fail`, `skip` vagy `not-applicable`. A mátrix akkor szerződés, ha minden cella reprodukálható: rögzített compilerparancs, környezeti változók, buildkönyvtár és

tesztbemenet mellett ugyanazt az eredményt adja. A tesztek ehhez a konfigurációhoz rendelődnek, nem fordítva.

A dimenziók célja a kockázat arányos lefedése, nem a kombinációk robbanása. Egy tipikus vázlat:

Tengely	Példaértékek	Miért kerül be
Nyelv	C17, C23, C++20, C++23	nyelvi és library-eltérések
Compiler	GCC, Clang	diagnosztika, optimalizálás, runtime
Build	Debug, Release	assertionök, optimalizálás, UB
Sanitizer	nincs, ASan+UBSan, TSan	memória-, UB- és versenyhibák
Target	x86-64, AArch64	ABI, alignment, elérhető utasítások

A cellák elvárt eredményét indokolni kell. Ha egy kombinációt nem futtatunk, a mátrixban `not-applicable` vagy indokolt `skip` szerepeljen, ne csendes kihagyás. A CI-riportban a skipek száma és oka látható legyen; a kötelező cella skipje hiba, a nem kötelezőé elfogadható.

38.4.2 CTest és a keretrendszerek illesztése

A CTest nem unit teszt keretrendszer, hanem futtató és jelentéskészítő réteg: a `add_test` névvel és parancssal regisztrál egy folyamatot, a visszatérési kódot és a kimenetet értékeli, a teszteket címkékkel, fixture-ökkel és erőforrás-zárakkal rendezi. Az assertionöket a keretrendszer adja. A C++ oldalon a GoogleTest és a Catch2 opcionális választás; a konkrét verziót, a `find_package` módját és a discovery makrót a projektnek a választott keretrendszer hivatalos dokumentációjából kell ellenőriznie. A könyv itt nem rögzít third-party verziót, és nem állítja, hogy az alábbi integrációs részleteket lefuttatta.

Egy minimális C-s példa, ahol a `diag_core` a korábbi projekt könyvtára:

```
cmake_minimum_required(VERSION 3.20)
project(diag LANGUAGES C CXX)

add_library(diag_core src/core.c)
target_include_directories(diag_core PUBLIC include)
target_compile_features(diag_core PUBLIC c_std_17)

include(CTest)
if(BUILD_TESTING)
  add_executable(test_core tests/test_core.c)
  target_link_libraries(test_core PRIVATE diag_core)
  add_test(NAME core_parse COMMAND test_core)
  set_tests_properties(core_parse PROPERTIES LABELS "unit;fast")
endif()
```

A C++ keretrendszer illesztése ettől csak a futtató parancsban tér el; a CTest továbbra is a folyamatot és annak kimenetét látja. A `gtest_discover_tests` és a `catch_discover_tests` a keretrendszer saját eszköze, amely a teszteseteket külön CTest-tesztekké bontja. A részlet C++20-at és a Függőségek, package discovery és linkhatárok (38.3) szerinti csomagfelderítést feltételez; a pontos makróneveket és verziófeltételeket a hivatalos dokumentációban kell megerősíteni.

38.4.3 Negatív tesztek: compile-fail és runtime diagnostic

A **negatív teszt** olyan teszt, amelynek helyes kimenete egy hiba észlelése. Három fő alakja van: a fordításnak szándékosan sikertelennek kell lennie (**compile-fail**); a programnak sanitizer vagy assertion miatt nem nulla kóddal kell leállnia (**runtime diagnostic**); illetve a hibakezelő ágnak a várt hibakódot vagy hibastátuszt kell visszaadnia. A közös feltétel az izoláció: a negatív példa ne kerüljön a normál buildbe, és a teszt ne attól legyen „sikeres”, hogy valami véletlenül elromlott.

Compile-fail esetben a puszta „nem fordult” kevés, mert elgépelés vagy hiányzó include is okozhatja. A tesztnek a várt diagnosztikai kategóriát is ellenőriznie kell, lazán, mert a compiler üzenete nem stabil API. C++20-ban például a `const` eldobása egyértelmű hiba. A minta egyetlen, rögzített projektelrendezést használ: a gyökér `CMakeLists.txt` regisztrálja a tesztet, a wrapper a `tests/run_compile_fail.sh`, a negatív forrás a `tests/negative/const_discard.cpp`.

```
// tests/negative/const_discard.cpp
int main() {
    const int x = 1;
    int* p = &x; // hiba: const int* -> int*
    *p = 2;
}
```

C-ben ugyanez jellemzően csak figyelmeztetés, ezért ez a negatív teszt a mátrixban csak C++ módra érvényes. A futtatás wrapperrel történik, amely a visszatérési kódot és a mintát is ellenőrzi:

```
#!/usr/bin/env bash
# tests/run_compile_fail.sh <forrás> <ERE-minta>
set -euo pipefail
src=$1
pattern=$2
export LC_ALL=C                  # a diagnosztika nyelve: angol, a mintához
 ↪  rögzítve
test -f "$src" || { echo "FAIL: hiányzó forrás: $src" >&2; exit 1; }
out=$(mktemp)
trap 'rm -f "$out"' EXIT
if "${CXX:?}" -std=c++20 -fsyntax-only "$src" >"$out" 2>&1; then
  echo "FAIL: a negatív teszt váratlanul lefordult" >&2
  exit 1
fi
if ! grep -Eq "$pattern" "$out"; then
  echo "FAIL: a várt diagnosztika nem jelent meg: $pattern" >&2
  cat "$out" >&2
  exit 1
fi
echo "OK: a fordítás a várt módon sikertelen"
```

A CTest-regisztráció a wrappert hívja, nem közvetlenül a compilert:

```
# gyökér CMakeLists.txt (részlet)
# A minta a compiler azonosítója szerint, a rögzített kiadásokon kipróbálva.
if(CMAKE_CXX_COMPILER_ID STREQUAL "GNU")
  set(CONST_DISCARD_RE "invalid conversion from 'const int\\*' to 'int\\*'")
elseif(CMAKE_CXX_COMPILER_ID MATCHES "Clang")
```

```
  set(CONST_DISCARD_RE "cannot initialize a variable of type 'int \\*' with an
   ↪  rvalue of type 'const int \\*'")
else()
  message(FATAL_ERROR "compile_fail_const_discard: nincs ellenőrzött minta ehhez
   ↪  a fordítóhoz")
endif()

add_test(NAME compile_fail_const_discard
  COMMAND ${CMAKE_COMMAND} -E env CXX=${CMAKE_CXX_COMPILER}
          ${PROJECT_SOURCE_DIR}/tests/run_compile_fail.sh
          ${PROJECT_SOURCE_DIR}/tests/negative/const_discard.cpp
          "${CONST_DISCARD_RE}")
```

A minta compiler-specifikus, ezért a compiler azonosítója szerint választjuk, és csak a ténylegesen kipróbált fordítókra adunk meg mintát; ismeretlen fordítónál a konfiguráció megáll, nem enged át egy ellenőrizetlen tesztet. A wrapper `LC_ALL=C`-vel rögzíti a diagnosztika nyelvét, és a hiányzó forrást külön, a fordítás előtt jelzi. A cél a szabály megsértésének elutasítása, nem a pontos szöveg.

Buildjegyzőkönyv (`lab-gcc` profil; CMake 4.4.4, Ninja). A `ctest` a GCC 13.3-mal és a Clang 22-vel (ziglang) konfigurált buildben is `100% tests passed` eredményt adott. Kontrollok: hiányzó forrásra `FAIL: hiányzó forrás`, 1-es kód; egy más típushibát (`int* p = 3.5;`) tartalmazó forrásra mindkét fordítónál `FAIL: a várt diagnosztika nem jelent meg`. A korábbi `discards qualifiers|invalid conversion` minta a Clang üzenetére (`cannot initialize a variable of type 'int *' with an rvalue of type 'const int *'`) nem illeszkedett, a forrásgyökérhez viszonyított `negative/const_discard.cpp` útvonal pedig nem létezett. A `WILL_FAIL` property önmagában itt sem elég, mert bármilyen nem nulla visszatérési kódot elfogad. Runtime diagnostic esetén ugyanez a helyzet: a sanitizer jelenlétét a kimenetből vagy a wrapperből kell igazolni, különben egy más okból leálló program is „elkapott hibának” látszana.

38.4.4 Sanitizer profilok: ASan/UBSan és külön TSan

Az AddressSanitizer (ASan) a memóriahibákat, a Use-after-free-t, a buffer-túlcsordulást és a leaket figyeli; az UndefinedBehaviorSanitizer (UBSan) a nem definiált viselkedés egy részét jelzi. A kettő együtt futtatható. A UBSan alapértelmezésben a diagnosztika után folytatja a futást; a leállást több, külön szinten beállítható mód adja: fordítási időben a `-fno-sanitize-recover=all` (ajánlott, explicit CI-policy: a hiba nem nulla kilépési kód) vagy a `-fsanitize-trap=undefined` (azonnali trap, diagnosztikai szöveg nélkül), futásidőben a recoverable buildnél az `UBSAN_OPTIONS=halt_on_error=1`. **Mérés** (`lab-gcc`, egy `INT_MAX + 1` túlcsordulással): az alapbuild diagnosztika után tovább futott, 0-s kóddal; a `halt_on_error=1` és a `-fno-sanitize-recover=all` 1-es kóddal, a trap-build `SIGILL`-lel (132) állt le. A tesztprofilban ezért rögzíteni kell, melyik mód adja a leállást és milyen kilépési feltétellel. A ThreadSanitizer (TSan) az adatversenyeket keresi, és külön folyamatot, külön buildet igényel: az ASan és a TSan runtime-ja nem kombinálható egy targetben. A pontos támogatást és kapcsolókat a compiler dokumentációja szerint kell ellenőrizni; az alábbi CMake-részlet GCC/Clang, Linux, x86-64 feltételezéssel illusztráció.

```
add_library(sanitize_flags INTERFACE)
option(SANITIZE_ASAN "ASan+UBSan" OFF)
option(SANITIZE_TSAN "TSan" OFF)
if(SANITIZE_ASAN AND SANITIZE_TSAN)
  message(FATAL_ERROR "ASan/UBSan és TSan külön jobban futtatandó")
```

```
endif()
if(SANITIZE_ASAN)
  target_compile_options(sanitize_flags INTERFACE
    -fsanitize=address,undefined -fno-sanitize-recover=all
    -fno-omit-frame-pointer -g)
  target_link_options(sanitize_flags INTERFACE -fsanitize=address,undefined)
elseif(SANITIZE_TSAN)
  target_compile_options(sanitize_flags INTERFACE
    -fsanitize=thread -fno-omit-frame-pointer -g)
  target_link_options(sanitize_flags INTERFACE -fsanitize=thread)
endif()
```

A futtatás külön buildkönyvtárban és külön CI-jobban történik:

```
ASAN_OPTIONS=halt_on_error=1:abort_on_error=1:detect_leaks=1 \
UBSAN_OPTIONS=halt_on_error=1:print_stacktrace=1 \
ctest --test-dir build-asan --output-on-failure
```

A TSan job ugyanígy külön fut:

```
TSAN_OPTIONS=halt_on_error=1:second_deadlock_stack=1 \
ctest --test-dir build-tsan --output-on-failure
```

A sanitizer build nem release build, és nem helyettesíti a teszteket: csak a lefuttatott kódra ad diagnosztikát. A teljesítménymméréseket nem sanitizer alatt kell értelmezni, mert a runtime overhead jelentős.

38.4.5 Capability matrix és feltételes tesztek

A **capability matrix** a toolchain képességeit detektálja, nem a tesztek eredményét. Célja, hogy a C23-as vagy C++23-as kód ne a véletlen compiler-default miatt bukjon el, és hogy a nem támogatott képesség skipje látható legyen. A CMake `check_c_source_compiles`, `check_cxx_source_compiles` és `check_cxx_compiler_flag` makrói kis fordítási próbákat futtatnak. A próba eredménye alapján a teszt fut, vagy indokolt skip lesz:

```
include(CheckCXXSourceCompiles)
set(CMAKE_REQUIRED_FLAGS "-std=c++20")
check_cxx_source_compiles([=[
  #include <concepts>
  template<class T> concept Addable = requires(T a, T b) { a + b; };
  static_assert(Addable<int>);
  int main() { return 0; }
]=] HAVE_CXX20_CONCEPTS)

if(HAVE_CXX20_CONCEPTS)
  add_test(NAME concepts_requires COMMAND test_concepts)
else()
  add_test(NAME concepts_requires
    COMMAND ${CMAKE_COMMAND} -E echo "skip: nincs C++20 concepts")
  set_tests_properties(concepts_requires PROPERTIES SKIP_RETURN_CODE 0)
endif()
```

A `SKIP_RETURN_CODE 0` itt azt jelenti, hogy a 0-val visszatérő helyettesítő parancs skipnek számít. A skip nem lehet csendes: a CI-összesítésben meg kell jelennie, és a kötelező celláknál hibának kell számítania. A C17/C23 és C++20/C++23 mátrixban külön kell kezelni a nyelvi és a library-támogatást; az N3096 és az N4950 nyilvános munkatervezet, nem az ISO kiadott szabványa, ezért a „támogatott” állítást a konkrét compiler- és library-verzióhoz kell kötni.

38.4.6 Fuzz corpus, failure injection és coverage korlátok

A fuzzelés bemenetgenerálás, nem bizonyítás. A **corpus** a kiinduló bemenetek verziózott halmaza; a fuzzer ezt mutálja, és a talált hibát reprodukáló bemenetet vissza kell tenni a corpusba. A CI-ben a corpus perzisztens cache-ből töltődik, a futtatás időkorlátos, és a fuzz target ASan/UBSan alatt fut. A fuzz nem determinisztikus, ezért a „nem talált hibát” nem lehet a helyesség bizonyítékaként kezelni; a regressziós corpus determinisztikus újrajátszása igen.

A **failure injection** szándékos, determinisztikus hibapontok beiktatása: allokációs hiba, I/O-hiba, hálózati hiba. A teszt beállítja a hibát, és a hibakezelő ág viselkedését ellenőrzi; ez a Hibakezelési és exception safety labor (25.5) laborjára épül. A hibapontok ne maradjanak a produkciós buildben, és a visszajátszásnak ugyanazt az állapotot kell előállítania.

A **coverage** (sor-, ág- vagy feltételellefedettség) azt méri, mit futtattunk, nem azt, hogy helyes. A 100%-os coverage öncél lehet, és a generált kód, a header-only könyvtárak, valamint a template-instantiationök torzíthatják. Ésszerűbb a kritikus ágakra, a hibakezelésre és a negatív tesztekre koncentrálni; a coverage-korlát legyen dokumentált és indokolt, ne a minőség egyetlen mérőszáma.

38.4.7 Gyakori hibák és javításuk

A leggyakoribb hiba a sanitizer és a release optimalizálás összekeverése: a `-g` hiánya olvashatatlan stack trace-t ad, a sanitizer nélküli „release” pedig nem ugyanazt a diagnosztikát nyújtja. A UBSan `-fno-sanitize-recover` nélkül csak jelent, a program tovább fut, és a CI zöld maradhat. Az ASan és a TSan egy targetben nem működik; külön job kell. A compile-fail teszt normál buildbe kerülése megszakítja a buildet; a negatív példáknak külön könyvtárban és külön regisztrációval kell élniük. A `WILL_FAIL` önmagában hamis zöldet ad, mert nem ellenőrzi a hiba okát. A feature detection elhagyása miatt a C++23-as kód a C++20-as compileren bukik, pedig a helyes eredmény skip lenne. A csendes skip ugyanilyen hiba: a mátrixban láthatóvá kell tenni. A fuzz corpus perzisztálásának hiánya miatt minden futás elölről kezd, a coverage 100%-ra hajtása pedig eltereli a figyelmet a hibakezelésről.

38.4.8 Feladatok

1. **Mátrixtervezés.** Készíts tesztmátrixot a könyv `diag` projektjéhez legalább négy tengellyel. Minden cellához add meg az elvárt eredményt, és indokold, mely cellák `not-applicable` vagy `skip` jellegűek. Írd le, hogyan ellenőrzöd, hogy a mátrix nem csendes kihagyásokból áll.
2. **Compile-fail teszt.** Írj C++20-as compile-fail tesztet, amely a `const` eldobását utasítja el. Készíts hozzá wrapper szkriptet és CTest-regisztrációt. A teszt akkor legyen sikeres, ha a fordítás sikertelen, és a kimenet tartalmazza a várt mintát. Add meg a build- és futtatási parancsot.
3. **ASan-negatív teszt.** Adott egy szándékosan túlindexelő heap-tömb teszt. Készítsd el a CMake targetet és a CTest-regisztrációt úgy, hogy csak ASan buildben fusson, és a várt hiba esetén legyen sikeres. Magyarázd el, miért nem elég a `WILL_FAIL`.

4. **TSan-verseny.** Készíts TSan-tesztet, amelyben két szál ír egy nem atomi globális változót. Add meg a külön TSan build és futtatás parancsát, és indokold, miért nem futtatható ez az ASan buildben. Nevezd meg a változó tárolási idejét, a szálak tulajdonviszonyát és a hibakimenetet.

5. **Fuzz és coverage.** Tervezz fuzz stratégiát a parserhez: hogyan tárolod a corpus, hogyan reprodukálod a hibát, és milyen coverage-korlátot tartasz ésszerűnek? Különítsd el a determinisztikus regressziós futást a nem determinisztikus felfedezéstől.

38.4.9 Megoldások

38.4.9.1 1. Mátrixtervezés

Egy lehetséges vázlat: C17/GCC/x86-64/Debug/nincs sanitizer/`pass`; C23/GCC/x86-64/Debug/ASan+UBSan/`pass` vagy indokolt `skip`, ha a C23-as képesség hiányzik; C++20/Clang/x86-64/Release/nincs sanitizer/`pass`; C++20/Clang/x86-64/Debug/TSan/`pass`; C++23/Clang/x86-64/Debug/nincs sanitizer/`expected-fail` vagy `skip` a feature detection eredményétől függően. Az ASan+UBSan és a TSan egy cellában `not-applicable`. Az ellenőrzés: minden sorhoz tartozik buildparancs, környezeti változó és elvárt eredmény; a CI-riport a skipeket okkal együtt listázza; a kötelező cellák skipje külön hibaként jelenik meg.

38.4.9.2 2. Compile-fail teszt

A `const_discard.cpp` és a fenti wrapper használható. A futtatás:

```
cmake -S . -B build -DBUILD_TESTING=ON
cmake --build build
ctest --test-dir build -R compile_fail_const --output-on-failure
```

Ellenőrzési feltétel: a wrapper visszatérési kódja 0, a compiler visszatérési kódja nem 0, és a kimenet illeszkedik a lazán megadott mintára. A minta compiler-specifikus; ha a minta nem illeszkedik, a teszt bukik, nem csendben átmegy. A `std=c++20` és a compiler útja a laborprofilból jön.

38.4.9.3 3. ASan-negatív teszt

A szándékos hiba:

```c
/* tests/sanitizer/asan_oob.c */
#include <stdlib.h>
int main(void) {
    int *p = malloc(2 * sizeof *p);
    if (!p) return 2;
    p[2] = 1; /* szándékos túlindexelés */
    free(p);
    return 0;
}
```

A CMake target csak ASan buildben létezik:

```
if(SANITIZE_ASAN)
  add_executable(asan_oob tests/sanitizer/asan_oob.c)
  target_link_libraries(asan_oob PRIVATE sanitize_flags)
  add_test(NAME asan_oob_detects
    COMMAND ${CMAKE_CURRENT_SOURCE_DIR}/run_expect_sanitizer.sh
            $<TARGET_FILE:asan_oob> AddressSanitizer)
endif()
```

A wrapper elindítja a programot, elvárja a nem nulla visszatérési kódot és a sanitizer jellemző kimenetét. A `WILL_FAIL` azért nem elég, mert bármilyen nem nulla kódot elfogadna, például egy `abort()`-ot vagy egy hiányzó fájlt is. A teszt tulajdonosa a wrapper; a `p` heap-tömb tulajdonosa a `main`, élettartama a `free`-ig tart, az invalidáló művelet a `free` utáni hozzáférés vagy a határon kívüli írás; a hibakimenet az ASan jelentése.

38.4.9.4 4. TSan-verseny

```
// tests/tsan/race.cpp
#include <thread>
int x = 0;
int main() {
    std::thread a([]{ x = 1; });
    std::thread b([]{ x = 2; });
    a.join();
    b.join();
}
```

A build és futtatás:

```
cmake -S . -B build-tsan -DSANITIZE_TSAN=ON -DSANITIZE_ASAN=OFF
cmake --build build-tsan
TSAN_OPTIONS=halt_on_error=1 \
ctest --test-dir build-tsan -R tsan_race --output-on-failure
```

A `x` statikus tárolású, élettartama a program teljes futása; a két szál nem tulajdonosa a változónak, csak hozzáfér. Az invalidáló művelet a szinkronizáció nélküli, versengő írás; a hibakimenet a TSan adatverseny-jelentése. Az ASan buildben azért nem futtatható, mert az ASan és a TSan runtime-ja nem kombinálható; a két diagnosztika külön folyamatot igényel.

38.4.9.5 5. Fuzz és coverage

A corpus a `tests/fuzz/corpus/` alatt verziózva, a CI cache-ből töltve; a futtatás időkorlátos, a fuzz target ASan+UBSan alatt fut. A hibát reprodukáló bemenet a corpusba kerül, és a regressziós futás determinisztikusan újrajátssza. A coverage-korlát a parser kritikus ágaira és a hibakezelésre vonatkozzon, ne a teljes generált kódra; a küszöb dokumentált, és nem helyettesíti a negatív teszteket. A felfedező futtatás nem determinisztikus, ezért a „nincs hiba” eredménye nem bizonyíték, csak a regressziós corpus és a sanitizeres futás ad ismételhető ellenőrzést.

38.4.10 Összefoglaló

A natív CI minőségi kapuja a tesztmátrix, nem a zöld szín. A mátrix cellái nyelvi módot, compilert, targetet, build típust és sanitizert rögzítenek, elvárt eredménnyel együtt. A CTest a futtató és jelentéskészítő réteg; a keretrendszer az assertionöket adja. A compile-fail és runtime diagnostic tesztek izoláltak, és a hiba okát is ellenőrzik, nem csak a nem nulla visszatérési kódot. Az ASan/UBSan együtt, a TSan külön jobban fut. A capability matrix a toolchain képességeit detektálja, a skip pedig látható és indokolt. A fuzz corpus perzisztens, a failure injection determinisztikus, a coverage pedig korlátos és nem öncél.

38.5 Cross-compilation és multiarch build

Amikor a build gépen futó fordító nem a build gépén futtatható kódot állít elő, cross-compilationról (keresztfordításról) beszélünk. A DevOps gyakorlatban ez akkor kerül elő, amikor x86-64-es build agenten AArch64 szerverre, beágyazott targetre vagy multiarch konténerimage-hez készül csomag. A nehézség nem a compiler elindítása, hanem annak biztosítása, hogy a fordítás minden bemenete — fejléc, könyvtár, generált kód, teszt — a target rendszerhez tartozzon, ne a build géphez.

Az autotools terminológiájában a *build* a fordító gépe, a *host* a futtató rendszer, a *target* pedig a compiler által előállított kód rendszere; kanadai cross esetén a három eltér. A CMake viszont a build gépet nevezi hostnak (`CMAKE_HOST_SYSTEM_NAME`), a futtató rendszert pedig targetnek (`CMAKE_SYSTEM_NAME`). Ebben a fejezetben a félreértés elkerülésére **build gépről** és **target rendszerről** beszélünk; a CMake változóneveit változatlanul használjuk. Az előfeltételek között szerepel a CTest, negatív tesztek és sanitizer CI és az ABI- és optimalizálásdiagnosztikai labor; itt a cross build sajátosságaira építünk.

38.5.1 A build, a target és a toolchain

A **toolchain** itt a target kód előállításához szükséges eszközök együttese: a target compiler és binutils, a target libc fejlécei és könyvtárai, a sysroot, valamint opcionálisan a target debugger és emulátor. A toolchain konzisztenciája azt jelenti, hogy a compiler, a linker, a libc és a sysroot ugyanahhoz a target ABI-hoz és libc-verzióhoz tartozik. Ha például a compiler `aarch64-linux-gnu` triple-tel dolgozik, de a sysrootban x86-64-es glibc van, a linkelés vagy a futás fog elhasalni — gyakran félrevezető hibaüzenettel.

A **target triple** a compiler által használt, többé-kevésbé kanonikus azonosító, tipikusan `arch-vendor-os-abi` alakban, például `aarch64-linux-gnu` vagy `x86_64-linux-gnu`. A `gnu` a glibc családra utal, de a libc verzióját nem tartalmazza; a `musl` target triple külön, például `aarch64-linux-musl`. A triple tehát nem teljes ABI-szerződés: a libc verziója, a minimális kernelverzió, a CPU baseline (`-march`), a lebegőpontos ABI és a kivételkezelés modellje külön rögzítendő. A `gcc -dumpmachine` vagy a `clang --target=...` megmutatja, melyik triple-t használja a compiler; ezt érdemes a build logban rögzíteni.

A **sysroot** a target fájlrendszerének gyökere a build gépen. A compiler `--sysroot` hatására a target fejléceket és könyvtárakat ezen a gyökér alatt keresi. A compiler saját, beépített include könyvtárai ettől függetlenül a keresési sorrend részei; a pontos sorrendet a compiler dokumentációja és a `-v` kimenet mutatja. A `CMAKE_SYSROOT` és a `CMAKE_FIND_ROOT_PATH` feladata részben átfed: a `CMAKE_SYSROOT` a compilernek `--sysroot`-ként adódik át, *és* a CMake `find_*` parancsai is gyökérként használják; a `CMAKE_FIND_ROOT_PATH` csak a `find_*` parancsoknak szól, és több gyökeret is felsorolhat (például egy külön telepített target-csomagkönyvtárat a sysroot mellett).

38.5.2 A CMake toolchain fájl és a find root policy

A CMake-ben a cross buildet a `CMAKE_SYSTEM_NAME` kézi beállítása aktiválja (toolchain fájlban vagy `-D`-vel). A `CMAKE_CROSSCOMPILING` ekkor igaz lesz — akkor is, ha a beállított érték megegyezik a build gépével: Linux hoston egy `set(CMAKE_SYSTEM_NAME Linux)` toolchain is `TRUE`-t ad. A változó tehát azt jelzi, hogy a target rendszert kézzel írtuk le, nem azt, hogy a két rendszernév eltér; egy azonos nevű, de más architektúrájú vagy más sysrootú target ugyanúgy keresztfordítás. Ha a döntéshez a valódi eltérés kell (például futtatható-e a target bináris a build gépen), azt a `CMAKE_SYSTEM_PROCESSOR`, a sysroot és szükség esetén egy emulátor alapján kell eldönteni. Egy minimális AArch64 toolchain fájl:

```
# cmake/toolchains/aarch64-linux-gnu.cmake
set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_SYSTEM_PROCESSOR aarch64)
set(CMAKE_C_COMPILER aarch64-linux-gnu-gcc)
set(CMAKE_CXX_COMPILER aarch64-linux-gnu-g++)
set(CMAKE_SYSROOT /usr/aarch64-linux-gnu)
set(CMAKE_FIND_ROOT_PATH ${CMAKE_SYSROOT})
set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_PACKAGE ONLY)
set(CMAKE_CROSSCOMPILING_EMULATOR "qemu-aarch64;-L;${CMAKE_SYSROOT}" CACHE
 ↪  STRING "")
```

A `CMAKE_FIND_ROOT_PATH_MODE_PROGRAM` értéke azért `NEVER`, mert a `pkg-config`, `protoc`, kódgenerátorok és hasonló programok a build gépen futnak. A `LIBRARY` és `INCLUDE` `ONLY`, mert a target könyvtárakat és fejléceket a sysrootban kell megtalálni. A `PACKAGE` `ONLY` a legszigorúbb; `BOTH` csak indokolt esetben használható, mert a host package-et is megtalálhatja.

38.5.2.1 Levezetés (vázlat): hogyan keres a find_package cross alatt

1. A `find_package(ZLIB)` a `FindZLIB` modult futtatja.
2. A modul `find_path(ZLIB_INCLUDE_DIR zlib.h)` és `find_library(ZLIB_LIBRARY z)` hívást végez.
3. A `find_*` a `CMAKE_FIND_ROOT_PATH` alatt keres, ha a megfelelő `CMAKE_FIND_ROOT_PATH_MODE_*` értéke `ONLY`; `BOTH` esetén a root és a build gép is szóba jön, a root előrébb.
4. A `CMAKE_SYSROOT` a compiler `--sysroot`-ját adja, és a `find_*` is gyökérként használja; a `CMAKE_FIND_ROOT_PATH` további gyökereket ad hozzá. Hogy a gyökerek alatt *csak*, *is* vagy *egyáltalán nem* keres, azt a `CMAKE_FIND_ROOT_PATH_MODE_*` (`ONLY`, `BOTH`, `NEVER`) dönti el keresési fajtánként.
5. A `PACKAGE` mód `ONLY` a legszigorúbb; a host toolokat külön kell átadni.

Buildjegyzőkönyv (`lab-gcc` profil; CMake 4.4.4). Egy `project(probe NONE)` próbaprojekt toolchain nélkül `CROSSCOMPILING=FALSE`-t, egyetlen `set(CMAKE_SYSTEM_NAME Linux)` sort tartalmazó toolchainnel Linux hoston `CROSSCOMPILING=TRUE`-t írt ki. Egy olyan toolchainnel, amely csak a `CMAKE_SYSROOT`-ot állítja (a `CMAKE_FIND_ROOT_PATH` üres), `LIBRARY`/`INCLUDE` `ONLY` móddal a `find_path` és a `find_library` a sysroot alatti `usr/include/fakehdr.h`-t és `usr/lib/libfake.a`-t találta meg; toolchain nélkül egyiket sem. Ez dokumentált viselkedés, nem véletlen.

38.5.3 Host eszköz és target könyvtár szétválasztása

A cross build egyik leggyakoribb hibája, hogy egy build gépen futó eszközt target compilerrel fordítunk le, vagy egy target könyvtárat a build gép könyvtárából linkelünk. Az előbbi bináris a build gépen nem fut, az utóbbi pedig rossz ABI-jú kódot ad. A megoldás a két szerep szétválasztása: a **host tool** natív buildben készül, a **target library** a target compilerrel. A host toolt `IMPORTED` targetként adjuk át a cross buildnek.

A következő példa egy minimális, de teljes multiarch projektet mutat be. A `gen_config` host tool a target arch-hoz generál egy `config.hpp` fejlécet; az `answer` target library és az `app` target executable a target compilerrel készül. A generált fejléc rögzíti a várt capability-t, a target kód pedig `static_assert`-tel ellenőrzi, hogy a target compiler tényleg azt a modellt állítja elő.

```
// tools/gen_config.cpp - a build gépen fut
#include <fstream>
#include <string>

int main(int argc, char** argv) {
    if (argc != 3) return 1;
    const std::string arch = argv[2];
    const int ld_digits = (arch == "aarch64") ? 113 : 64;
    std::ofstream out(argv[1]);
    out << "#pragma once\n"
        << "#define DEMO_TARGET_ARCH \"" << arch << "\"\n"
        << "#define DEMO_EXPECT_CHAR_UNSIGNED 1\n"
        << "#define DEMO_EXPECT_LONG_DOUBLE_BYTES 16\n"
        << "#define DEMO_EXPECT_LD_DIGITS " << ld_digits << "\n";
    return out ? 0 : 2;
}

// include/answer.hpp
#pragma once
#include <cstdint>

std::uint32_t answer() noexcept;

// src/answer.cpp
#include "answer.hpp"

std::uint32_t answer() noexcept { return 42u; }

// src/main.cpp
#include "answer.hpp"
#include "config.hpp"

#include <cstdint>
#include <cstdio>
#include <limits>

static_assert(DEMO_EXPECT_CHAR_UNSIGNED == (static_cast<char>(-1) > 0),
              "a target char signednesse eltér a rögzített capability-től");
static_assert(DEMO_EXPECT_LONG_DOUBLE_BYTES == sizeof(long double),
              "a target long double mérete eltér");
```

```
static_assert(DEMO_EXPECT_LD_DIGITS == std::numeric_limits<long double>::digits,
              "a target long double precisionje eltér");

int main() {
    std::printf("answer=%u\n", static_cast<unsigned>(answer()));
    std::printf("arch=%s\n", DEMO_TARGET_ARCH);
    std::printf("char unsigned=%s\n",
                (static_cast<char>(-1) > 0) ? "igen" : "nem");
    std::printf("sizeof(long double)=%zu, digits=%d\n",
                sizeof(long double),
                std::numeric_limits<long double>::digits);
    return 0;
}
```

A target oldali `CMakeLists.txt` target-alapú: a tulajdonságok a targethez kötődnek, nem globális változókhoz.

```
cmake_minimum_required(VERSION 3.20)
project(abi_demo LANGUAGES CXX)

set(DEMO_TARGET_ARCH "x86_64" CACHE STRING "Target arch")
set(DEMO_GEN_CONFIG "" CACHE FILEPATH "Host gen_config binary")

add_library(answer STATIC src/answer.cpp)
target_include_directories(answer PUBLIC include)
target_compile_features(answer PUBLIC cxx_std_20)

if(CMAKE_CROSSCOMPILING)
  add_executable(gen_config IMPORTED)
  set_target_properties(gen_config PROPERTIES
    IMPORTED_LOCATION "${DEMO_GEN_CONFIG}")
else()
  add_executable(gen_config tools/gen_config.cpp)
endif()

add_custom_command(
  OUTPUT "${CMAKE_CURRENT_BINARY_DIR}/generated/config.hpp"
  COMMAND gen_config
          "${CMAKE_CURRENT_BINARY_DIR}/generated/config.hpp"
          "${DEMO_TARGET_ARCH}"
  DEPENDS gen_config
  VERBATIM)
add_custom_target(generate_config
  DEPENDS "${CMAKE_CURRENT_BINARY_DIR}/generated/config.hpp")

add_executable(app src/main.cpp)
target_include_directories(app PRIVATE "${CMAKE_CURRENT_BINARY_DIR}/generated")
target_link_libraries(app PRIVATE answer)
add_dependencies(app generate_config)

enable_testing()
add_test(NAME app_smoke COMMAND app)
```

A natív host tool külön, kis projekttel épül:

```
# tools/CMakeLists.txt
cmake_minimum_required(VERSION 3.20)
project(gen_config LANGUAGES CXX)

add_executable(gen_config gen_config.cpp)
target_compile_features(gen_config PRIVATE cxx_std_20)
```

38.5.3.1 A build

```
# Host tool natív buildje
cmake -S tools -B build-native -DCMAKE_BUILD_TYPE=Release
cmake --build build-native --parallel

# Target build AArch64-re
cmake -S . -B build-aarch64 \
  -DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/aarch64-linux-gnu.cmake \
  -DDEMO_GEN_CONFIG="$PWD/build-native/gen_config" \
  -DDEMO_TARGET_ARCH=aarch64 \
  -DCMAKE_BUILD_TYPE=Release
cmake --build build-aarch64 --parallel

# Ellenőrzés
file build-aarch64/app
readelf -h build-aarch64/app | grep Machine
```

A `file` várhatóan `ELF 64-bit ... ARM aarch64` kimenetet ad, a `readelf -h Machine: AArch64` sort. Ezek illusztratív, környezetfüggő ellenőrzések; a tényleges kimenet a binutils és a `file` verziójától függ.

38.5.4 x86-64 és AArch64: ABI és adatmodell

A két target Linuxon egyaránt LP64, de nem binárisan kompatibilis. A legfontosabb eltérések a `char` signednesse, a `long double` reprezentációja, a `max_align_t` és a struktúrák paddingja, valamint a bit-field layout. A `sizeof` egyezése nem jelent ABI-kompatibilitást: a `long double` mindkét targeten 16 byte lehet, de x86-64-en 80 bites extended, AArch64-en 128 bites quad. A bináris protokollhoz fix szélességű típusok, explicit endianness és `static_assert` kellenek.

Tulajdonság	x86-64 System V Linux (GCC)	AArch64 Linux (GCC)
adatmodell	LP64	LP64
`sizeof(void*)`	8	8
`sizeof(long)`	8	8
`sizeof(int)`	4	4
`char` signedness	signed	unsigned
`sizeof(long double)`	16	16
`long double` formátum	80 bites extended	128 bites quad
`alignof(std::max_align_t)`	16	16
byte order	little-endian	little-endian

$$\texttt{sizeof(long double)} = 16, \qquad \texttt{digits}_{\text{x86-64}} = 64, \quad \texttt{digits}_{\text{AArch64}} = 113.$$

A táblázat jellemző Linux/GCC profilt mutat; a pontos értékeket a target ABI és a fordító verziója rögzíti, ezért a laborban `static_assert`-tel és `sizeof` méréssel kell ellenőrizni. A `char` signednesse miatt a `char` értékek bináris átvitele nem hordozható; használjunk `std::uint8_t`-t.

38.5.5 Feature probe cross környezetben

A `check_cxx_source_compiles` a target compilerrel fordít és linkel, ezért cross alatt is használható, ha a sysroot és a target libc elérhető. A `check_cxx_source_runs` és a `try_run` ezzel szemben futtatna; cross alatt vagy nem fut, vagy a `CMAKE_CROSSCOMPILING_EMULATOR`-t használja. Az emulátoron kapott eredmény nem azonos a natív targettel. Ezért a futásdetektálást **rögzített capability**-vel helyettesítjük: a target profil deklarálja a várt tulajdonságokat, a build pedig `static_assert`-tel ellenőrzi, hogy a target compiler tényleg azt a modellt állítja elő.

A fenti példában a `gen_config` a build gépen fut, de nem detektál: a profilból generálja a `config.hpp`-t. A target compiler a `static_assert`-tel ellenőrzi a `char` signednessét, a `long double` méretét és a precisionjét. Ha a target ABI eltér a rögzített capability-től, a build fordítási időben elhasal — ez sokkal jobb, mint egy futásidejű rejtélyes hiba.

38.5.6 Tesztelés: emuláció vagy natív target

A fordítási siker csak azt bizonyítja, hogy a target compiler elfogadta a forrást, és a linkelés a target könyvtárakkal megtörtént. A futási viselkedést a target libc, a kernel, a CPU featureök és a környezet határozza meg. A natív target runner (AArch64 CI agent) a legerősebb ellenőrzés; a `qemu-user` gyors, de a user-space viselkedést modellezi, a kernelét nem; a `qemu-system` teljes rendszert emulál, lassabb; a valós hardver a legjobb, de drága. A `CTest` cross alatt a `CMAKE_CROSSCOMPILING_EMULATOR`-ral futtatható:

```
ctest --test-dir build-aarch64 --output-on-failure
```

Ez azonban nem natív target teszt. A sanitizerek target runtime-ját a sysrootban elérhetővé kell tenni; a negatív tesztek és a sanitizer CI mintái a CTest, negatív tesztek és sanitizer CI (38.4) részben szerepelnek.

38.5.7 Tipikus hibák és javításuk

1. **A `find_package` a build gép könyvtárát találja meg.** Tünet: linkeléskor `file in wrong format`, futáskor `GLIBC_... not found`. Javítás: `CMAKE_SYSROOT`, `CMAKE_FIND_ROOT_PATH`, és a `LIBRARY`/`INCLUDE` mód `ONLY`.
2. **A `pkg-config` a build gép `.pc` fájljait olvassa.** Javítás: `PKG_CONFIG_LIBDIR` és `PKG_CONFIG_SYSROOT_DIR` beállítása, a `PKG_CONFIG_PATH` üresítése; a pontos viselkedés a `pkg-config` verziójától függ.
3. **A `-march=native` cross alatt.** A build gép CPU-ját detektálja, nem a targetét. Javítás: explicit `-march=armv8-a` vagy a target CPU pontos típusa, `-mcpu` tuning.
4. **A `try_run` a build gépen fut, vagy hibás eredményt ad.** Javítás: `try_compile` + `static_assert`, rögzített capability; a `CMAKE_CROSSCOMPILING_EMULATOR` csak tudatos, dokumentált esetben.
5. **A `char` signedness és a `long double` feltételezése.** Javítás: fix szélességű típusok, `static_assert`, explicit konverzió.

6. **A host tool és a target library keveredése.** Javítás: külön natív build, `IMPORTED` target.
7. **A `CTest` cross alatt nem futtat.** Javítás: `CMAKE_CROSSCOMPILING_EMULATOR` vagy natív target runner.
8. **A `CMAKE_SYSTEM_PROCESSOR` és a compiler triple eltérése.** Javítás: `gcc -dumpmachine`, `clang --target=...`, `CMAKE_CXX_COMPILER_TARGET`.

38.5.8 Feladatok

1. Magyarázd el, mi a különbség a `CMAKE_SYSROOT` és a `CMAKE_FIND_ROOT_PATH` között, és miért kell a `CMAKE_FIND_ROOT_PATH_MODE_PROGRAM` értékét `NEVER`-re állítani!
2. Írj egy AArch64 toolchain fájlt és egy target-alapú `CMakeLists.txt`-t, amely egy statikus könyvtárat és egy futtathatót épít! Ellenőrizd a bináris target architektúráját!
3. A cross build sikeres, de a targeten a program `Illegal instruction` hibával elszáll. Mi a lehetséges ok, és hogyan javítod?
4. Adott egy `try_run`-ra épülő feature probe, amely cross build alatt is lefut, és a build gép glibc verzióját detektálja. Alakítsd át úgy, hogy a target capability-t rögzítse, és fordítási időben ellenőrizze!

38.5.9 Megoldások

38.5.9.1 Első feladat

A `CMAKE_SYSROOT` a compiler `--sysroot` kapcsolóját adja; a `CMAKE_FIND_ROOT_PATH` a CMake `find_*` parancsainak keresési gyökere. A `CMAKE_FIND_ROOT_PATH_MODE_PROGRAM` azért `NEVER`, mert a `pkg-config`, `protoc` és más build eszközök a build gépen futnak; ha `ONLY` lenne, a CMake a target sysrootban keresné őket, és nem találná meg a futtatható host programot. Ellenőrzés: a `find_program(PKG_CONFIG pkg-config)` a build gép `pkg-config`-ját találja, míg a `find_library(ZLIB_LIBRARY z)` a sysroot target könyvtárát.

38.5.9.2 Második feladat

A toolchain fájl a `CMAKE_SYSTEM_NAME`, `CMAKE_SYSTEM_PROCESSOR`, `CMAKE_C_COMPILER`, `CMAKE_CXX_COMPILER`, `CMAKE_SYSROOT`, `CMAKE_FIND_ROOT_PATH` és a `CMAKE_FIND_ROOT_PATH_MODE_*` beállításaiból áll. A `CMakeLists.txt` a `add_library(answer STATIC src/answer.cpp)`, `target_include_directories(answer PUBLIC include)`, `target_compile_features(answer PUBLIC cxx_std_20)`, `add_executable(app src/main.cpp)`, `target_link_libraries(app PRIVATE answer)` hívásokat tartalmazza. Ellenőrzés: `file build-aarch64/app` `ARM aarch64`; `readelf -h build-aarch64/app | grep Machine` `AArch64`; a `static_assert` a generált `config.hpp`-val fordul.

38.5.9.3 Harmadik feladat

Az `Illegal instruction` jellemzően a target CPU által nem támogatott utasítás. Ok lehet a `-march=native` (a build gép CPU-ját detektálja), a túl új `-march`/`-mcpu`, vagy a target CPU baseline eltérése. Javítás: explicit `-march=armv8-a` vagy a target CPU pontos típusa, a `-march=native` eltávolítása, a `-mcpu` tuning. Ellenőrzés: `objdump -d build-aarch64/app` kimenetében a target CPU által nem támogatott utasítások; a target `/proc/cpuinfo` `Features` mezője; a build logban a tényleges `-march` érték.

38.5.9.4 Negyedik feladat

A `try_run` helyett `try_compile` + `static_assert` + generált capability header. A `check_cxx_source_compiles` a target compilerrel fordít; a `try_run` eltávolítandó. A `gen_config` a build gépen a profilból generálja a `config.hpp`-t; a target compiler a `static_assert`-tel ellenőrzi a `char` signednessét, a `long double` méretét és precisionjét. A `CMAKE_CROSSCOMPILING_EMULATOR` nem helyettesíti a natív target tesztet. Ellenőrzés: a `CMakeCache.txt` a target compilerre mutat; a build fail, ha a `static_assert` nem teljesül; a `config.hpp` a target arch-ot tartalmazza.

38.5.10 Összefoglaló

A cross build helyessége nem a compiler elindításán, hanem a teljes target környezet konzisztenciáján múlik: a target triple, a sysroot, a find root policy, a host tool és a target library szétválasztása, valamint a rögzített capability fordítási idejű ellenőrzése együtt adnak megbízható buildet. A fordítási siker nem bizonyítja a target futást; a natív target runner, a qemu és a valós hardver különböző erősségű ellenőrzést ad. A multiarch buildet targetenként izolált build könyvtárral és explicit toolchain fájllal érdemes vezetni.

38.5.11 Megjegyzés a forrásokhoz

A fejezet a CMake, a GCC/Clang, a binutils és a qemu viselkedését a laborban rögzített verziókhoz köti. Ezek a részletek verziófüggők; a konkrét beállítások helyességét a telepített eszközök dokumentációja és a labor mérése igazolja. A fejezet nem állítja, hogy a példák fordítását vagy futtatását elvégezte; a megadott kimenetek várt, illusztratív példák.

38.6 Release, debug symbol és reprodukálhatóság

Egy natív release nem „a bináris”, hanem egy artefaktumcsomag: a futtatható fájl, a hozzá tartozó hibakeresési adat, a build azonosítója, a függőségi leírás és a telepítési szerződés együtt. Ez a rész ezt a csomagot építi fel, abból a DevOps-szempontból, hogy üzem közben egy összeomlás után vissza kell tudni keresni a pontos forrásállapotot, a hibakeresési információt pedig nem szabad a termelési image-be tenni. A módszerek és a parancsok itt a tanuló laborjában futtathatók; a könyv nem állítja, hogy ezeket a példákat lefuttatta, és a konkrét kapcsolónevek, alapértelmezések a használt GCC/Clang, binutils és CMake kiadásától függenek — ezt minden lépésnél ellenőrizni kell a rögzített laborprofilhoz tartozó dokumentációban.

38.6.1 A release négy tulajdonsága

A release-szerződést négy, egymástól függetlenül ellenőrizhető tulajdonságra bontjuk. **Reprodukálhatóság (reproducible build):** ugyanaz a forrás, ugyanaz a toolchain-profil, ugyanazok a kapcsolók és ugyanaz a normalizált környezet bitre azonos kimenetet ad. Feltétele, hogy a buildben ne maradjon időbélyeg, abszolút útvonal, véletlen seed vagy nem determinisztikus sorrend. **Visszakereshetőség (traceability):** a leszállított fájlból egyértelműen eljutunk a forrásállapothoz, a fordítóhoz és a függőségekhez; ezt a `build-id` és a hozzá kapcsolt metaadat biztosítja. **Telepíthetőség (deployability):** a csomag tetszőleges prefixbe kicsomagolható, és a dinamikus loader a saját könyvtáraiból oldja fel a függőségeket. **Hibakereshetőség (debuggability):** a termelési binárison nincs szimbólum, de a hozzá tartozó külön debug fájl alapján a core dump teljes backtrace-re és forrássorra bontható.

A négy tulajdonság közül bármelyik kimarad, a hiány csak üzemzavar után derül ki: a visszakereshetetlen buildnél nem tudjuk, melyik commitot kell javítani; a szimbolizálhatatlan core dumpnál csak címeket látunk; a nem relocatable csomagnál a konténer másik útvonalon nem indul el.

38.6.2 A példaprojekt

A fejezet hátralevő részében egy kéttagú projektet használunk: a `libgreet` megosztott könyvtárat és a `greetctl` CLI-t. A kód szándékosan kicsi, hogy a build-szerződés legyen a nehéz rész.

```
// include/greet/greet.hpp
#pragma once
#include <string>
#include <string_view>

namespace greet {
std::string greeting(std::string_view name);
const char* version() noexcept;
}

// src/greet.cpp
#include "greet/greet.hpp"

namespace greet {
std::string greeting(std::string_view name) {
    return "hello, " + std::string{name};
}
const char* version() noexcept { return GREET_VERSION; }
}

// src/main.cpp
#include "greet/greet.hpp"
#include <cstdio>
#include <string_view>

int main(int argc, char** argv) {
    if (argc == 2 && std::string_view{argv[1]} == "--version") {
        std::printf("greetctl %s\n", greet::version());
        return 0;
    }
    const std::string out = greet::greeting(argc > 1 ? argv[1] : "world");
    std::printf("%s\n", out.c_str());
    return 0;
}
```

38.6.3 Install, export és relocatable CMake config

A target-alapú CMake telepítési modellje három külön dologból áll: a binárisok fizikai másolása (`install(TARGETS ...)`), az `IMPORTED` targeteket leíró export fájl (`install(EXPORT ...)`) és a csomag belépési pontja (`greetConfig.cmake`). A relocatable config feltétele, hogy a config fájl ne tartalmazzon abszolút build útvonalat, hanem a saját helyéhez képest számolja a prefixet. Ezt a `configure_package_config_file` generálja: a `@PACKAGE_INIT@` blokk definiálja a

`PACKAGE_PREFIX_DIR` változót a config fájl tényleges helyéből visszafelé számolva, így a csomag bármely prefixbe áthelyezhető.

```
cmake_minimum_required(VERSION 3.25)
project(greet VERSION 1.4.2 LANGUAGES CXX)

add_library(greet SHARED src/greet.cpp)
add_library(greet::greet ALIAS greet)
target_compile_features(greet PUBLIC cxx_std_20)
target_compile_definitions(greet PRIVATE GREET_VERSION="${PROJECT_VERSION}")
target_include_directories(greet PUBLIC
  $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
  $<INSTALL_INTERFACE:include>)
set_target_properties(greet PROPERTIES
  VERSION   ${PROJECT_VERSION}
  SOVERSION 1)

add_executable(greetctl src/main.cpp)
target_link_libraries(greetctl PRIVATE greet::greet)
set_target_properties(greetctl PROPERTIES
  INSTALL_RPATH "$ORIGIN/../lib")

include(GNUInstallDirs)
install(TARGETS greet greetctl
  EXPORT greetTargets
  LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
  RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR})
install(DIRECTORY include/ DESTINATION ${CMAKE_INSTALL_INCLUDEDIR})
install(EXPORT greetTargets
  NAMESPACE greet::
  DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/greet)

include(CMakePackageConfigHelpers)
configure_package_config_file(cmake/greetConfig.cmake.in
  ${CMAKE_CURRENT_BINARY_DIR}/greetConfig.cmake
  INSTALL_DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/greet)
write_basic_package_version_file(
  ${CMAKE_CURRENT_BINARY_DIR}/greetConfigVersion.cmake
  COMPATIBILITY SameMajorVersion)
install(FILES
  ${CMAKE_CURRENT_BINARY_DIR}/greetConfig.cmake
  ${CMAKE_CURRENT_BINARY_DIR}/greetConfigVersion.cmake
  DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/greet)
```

A `cmake/greetConfig.cmake.in` tartalma a `@PACKAGE_INIT@` után mindössze két hívás:

```
@PACKAGE_INIT@
include("${CMAKE_CURRENT_LIST_DIR}/greetTargets.cmake")
check_required_components(greet)
```

A `CMAKE_CURRENT_LIST_DIR` használata az, ami prefixfüggetlenné teszi a fájlt. Gyakori hiba, hogy valaki ilyen sort generál bele:

```
set(GREET_INCLUDE_DIR /home/fejleszto/greet/include)   # HIBA: abszolút útvonal
```

Ez a csomagot pontosan egy géphez köti.

38.6.4 SONAME, rpath/runpath és runtime függőségek

A **SONAME** a megosztott könyvtár dinamikus szekciójában tárolt név (`DT_SONAME`), amelyet a loader a függőségek feloldásához használ. Amikor a `greetctl`-t a `libgreet.so`-hoz linkeljük, a linker a `DT_NEEDED` bejegyzésbe a SONAME-et írja, ha van; a `SOVERSION 1` beállítás tipikus hatása a `libgreet.so.1` SONAME, a `VERSION` pedig a `libgreet.so.1.4.2` fájlnevet és a symlinkeket állítja elő. A verziózás lényege, hogy egy ABI-kompatibilis javítás ugyanazt a SONAME-et kapja, egy inkompatibilis változás új SOVERSION-t.

A **rpath** (`DT_RPATH`) és a **runpath** (`DT_RUNPATH`) a binárisba írt könyvtárkeresési bejegyzés. Különbségük sorrendben és hatókörben van: a `DT_RPATH`-et a loader a `LD_LIBRARY_PATH` előtt, a `DT_RUNPATH`-et utána nézi, és a `DT_RUNPATH` **nem tranzitív** — csak annak az objektumnak a közvetlen függőségeire érvényes, amelyben szerepel. Ez a legtöbb „működött a gépen, elszállt a konténerben” hiba gyökere. A `$ORIGIN` tokent a loader annak az objektumnak a könyvtárára cseréli, amelyben a bejegyzés található, ezért a telepített CLI-nél a `$ORIGIN/../lib` alak a helyes, nem a build könyvtár abszolút útja. Hogy a linker `DT_RPATH` vagy `DT_RUNPATH` bejegyzést készít-e, az a `--enable-new-dtags` / `--disable-new-dtags` kapcsolótól és a binutils verziójától függ; ezt a laborprofilban rögzíteni kell.

Ellenőrzésre ne az `ldd`-t használjuk ismeretlen binárison: a glibc `ldd` implementációja a dynamic loader trace-módjában lefuttatja a programot, ami nem kívánt kódvégrehajtást jelenthet. Helyette `readelf -d` vagy `objdump -p` adja a `NEEDED`, `RPATH`/`RUNPATH` és `SONAME` bejegyzéseket.

38.6.5 Külön debug szimbólumok, build-id és symbolication

A **build-id** a linker által a kimenet tartalmából számolt azonosító a `.note.gnu.build-id` szekcióban. Erre épül a symbolication: a GDB a core dumpból kiolvasott build-id alapján keresi meg a hozzá tartozó debug fájlt a debug-könyvtárban vagy egy debuginfod szolgáltatásban. A build-id jelenléte nem automatikus minden platformon, ezért explicit kérés érdemes: `-Wl,--build-id=sha1`.

A **külön debug szimbólum** előállítása három lépés: a teljes hibakeresési adat kiemelése, a bináris megfosztása a debug szekcióktól, majd egy `.gnu_debuglink` szekció beírása, amely a debug fájl nevét és egy CRC32 ellenőrzőértéket tartalmaz. A debug fájl a build-id note-ot is megtartja, így két úton is megtalálható.

```
f=stage/opt/greet/bin/greetctl
objcopy --only-keep-debug "$f" "$f.debug"
objcopy --strip-debug --add-gnu-debuglink="$f.debug" "$f"
```

A `--strip-debug` csak a `.debug_*` szekciókat távolítja el, a `.symtab`-ot nem; a `strip` ennél agresszívabb, és a lokális szimbólumneveket is elveszi, ami a backtrace minőségét rontja. Optimalizált buildben a `-g` megmarad, de a változók egy része `<optimized out>` lesz, és az inlineolt hívások több forrássort rendelnek egy frame-hez — ez nem hiba, hanem az optimalizálás következménye. A `-gsplit-dwarf` külön `.dwo` fájlokat eredményez, ami a tárolást csökkenti, de a disztribúciót bonyolítja; csak tudatos döntéssel használjuk.

38.6.6 SOURCE_DATE_EPOCH, path mapping és nondeterminism

A **reprodukálhatóság** azt jelenti, hogy a build bemenetei és a normalizált környezet rögzítése mellett a kimenet bitre azonos. A **SOURCE_DATE_EPOCH** egy szabványos környezeti változó, amelyet sok eszköz (például `ar`, `tar`, egyes fordítók a `__DATE__`/`__TIME__` makróhoz) az időbélyegek helyett használ. A forrása tipikusan a commit időpontja:

```
export SOURCE_DATE_EPOCH=$(git log -1 --format=%ct)
```

A **path mapping** a debug info-ba és a `__FILE__` makróba kerülő abszolút útvonalakat írja át. A `-ffile-prefix-map=OLD=NEW` kapcsoló a debug- és makróútvonalakat egyszerre kezeli; a GCC és a Clang elérhetősége és pontos neve kiadásfüggő (a Clangnál külön `-fdebug-prefix-map` és `-fmacro-prefix-map` is létezik). A tipikus beállítás:

```
-ffile-prefix-map=$PWD=.
```

Nondeterminizmus forrásai és kezelésük:

Forrás	Tünet	Kezelés
Abszolút build út a debug infóban	két könyvtárból építve eltérő hash	`-ffile-prefix-map`
`__DATE__`, `__TIME__`	minden build más	`SOURCE_DATE_EPOCH`, vagy a makrók elhagyása
`ar` archívum időbélyeg	`.a` fájl eltér	determinisztikus mód, ellenőrizni kell
Fordító által generált belső nevek	anonim névtér/lambda szimbólumok eltérnek	`-frandom-seed=<rögzített>` (GCC)
LTO párhuzamosság	partition-sorrend eltér	rögzített `-flto=1` vagy determinisztikus beállítás
Nem rendezett fájllista	link-sorrend eltér	explicit, rögzített forráslista

Az ellenőrzés két, különböző könyvtárból indított build hash-összehasonlítása; eltérés esetén a `diffoscope` jellegű eszközök mutatják meg a konkrét eltérő szekciót. Fontos: a build-id a tartalom hash-e, tehát ha a tartalom azonos, a build-id is azonos — a build-id önmagában nem nondeterminizmus-forrás.

38.6.7 SBOM és függőségi metaadat

Az **SBOM** (software bill of materials) géppel olvasható komponenslista: az artefaktumban ténylegesen felhasznált könyvtárak neve, verziója, beszállítója, hash-e és a köztük lévő függőségi élek. Formátuma lehet SPDX vagy CycloneDX; a kötelező mezők halmaza a formátum verziójától függ, ezért a generáló eszközt és annak verzióját a buildprofilban rögzíteni kell. Az SBOM-ot a build-id-hez kell kötni, hogy a leszállított fájl és a sor közötti kapcsolat ellenőrizhető legyen. Az SBOM nem bizonyítja a sebezhetetlenséget, és nem helyettesíti a függőségek tényleges verzióinak ellenőrzését — ezt a témakör határait tiszteletben tartva itt nem tárgyaljuk.

38.6.8 Minimális runtime image, libc baseline és smoke test

A **libc baseline** az a legrégebbi glibc-verzió, amelyen a bináris hajlandó elindulni. Nem a fordító verziója, hanem a felhasznált szimbólumverziók halmaza határozza meg: a dinamikus loader a legmagasabb `GLIBC_x.y` igényt ellenőrzi, és ha a célrendszer régebbi, `version 'GLIBC_2.34' not found` hibával áll le. Ugyanez vonatkozik a libstdc++ `GLIBCXX_` és `CXXABI_` szimbólumaira, valamint a `_GLIBCXX_USE_CXX11_ABI` beállításra, amely a `std::string` ABI-ját befolyásolja. A minimumot így olvassuk ki:

```
readelf --version-info stage/opt/greet/bin/greetctl \
  | grep -o 'GLIBC_[0-9.]*' | sort -Vu | tail -1
```

A **minimális runtime image** csak a szükséges SONAME-eket tartalmazza; a listát a `readelf -d NEEDED` bejegyzései adják, nem az `ldd` kimenete. A teljesen statikus linkelés bizonyos libc-funkciók (névfeloldás, `dlopen`) esetén külön figyelmeztetéseket hordoz, ezért nem alapértelmezett választás.

A **smoke test** a telepítési kapu: a cél-image-ben lefuttatott minimális parancs, amelynek nem nulla kilépési kódja megbuktatja a deployt. Feltétele, hogy a teszt a valódi image-en fusson, ne a build gépen.

```
#!/usr/bin/env bash
set -euo pipefail
img="$1"
docker run --rm "$img" /usr/local/bin/greetctl --version
docker run --rm "$img" /usr/local/bin/greetctl Ada
```

38.6.9 Kidolgozott példa: a teljes pipeline

A példaprojekt buildje a fenti szerződés mind a négy elemét lefedi. A build és a telepítés staged módon történik, hogy a prefix később szabadon mozgatható legyen:

```
export SOURCE_DATE_EPOCH=$(git log -1 --format=%ct)
cmake -S . -B build -G Ninja \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DCMAKE_CXX_FLAGS="-ffile-prefix-map=$PWD=." \
  -DCMAKE_INSTALL_PREFIX=/opt/greet
cmake --build build
DESTDIR="$PWD/stage" cmake --install build
```

A `RelWithDebInfo` konfiguráció jellemzően `-O2 -g` kapcsolókat ad, de ez a CMake verziójától függ, ezért a laborprofilban ellenőrizni kell. A telepítés után a binárist debug szétválasztással látjuk el, majd ellenőrizzük a dinamikus bejegyzéseket:

```
f=stage/opt/greet/bin/greetctl
objcopy --only-keep-debug "$f" "$f.debug"
objcopy --strip-debug --add-gnu-debuglink="$f.debug" "$f"
readelf -d "$f" | grep -E 'NEEDED|RPATH|RUNPATH'
readelf -n "$f" | grep 'Build ID'
```

A várt értelmezés: a `NEEDED` között `libgreet.so.1` és a C++ runtime könyvtárai szerepelnek, a `RUNPATH` pedig `$ORIGIN/../lib` alakú, nem abszolút. A debug szétválasztás után a `greetctl` mérete csökken, a `greetctl.debug` megtartja a build-id note-ot, és a core dump szimbolizálásához a debug fájlt a bináris mellé vagy a debug-könyvtárba kell tenni.

38.6.10 Tipikus hibák

A leggyakoribb, hogy a release pipeline `strip`-pel „tisztítja” a binárist, és a debug fájlt nem őrzi meg — így a későbbi core dump csak címeket mutat. A második, hogy a `CMAKE_INSTALL_PREFIX` abszolút értéke bekerül a config fájlba, és a csomag áthelyezés után nem találja magát. A harmadik, hogy a `RUNPATH`-tól tranzitív feloldást várunk: a `libgreet.so.1` saját függőségeit a saját `RUNPATH`-ja vagy a rendszerkönyvtárak alapján keresi a loader. A negyedik, hogy a `SOURCE_DATE_EPOCH`-ot beállítjuk, de a kódban marad `__TIME__`, vagy a verziósztringbe build idő kerül. Az ötödik, hogy `ldd`-vel ellenőrzünk egy nem megbízható binárist.

38.6.11 Összefoglaló

A release-szerződés négy, külön ellenőrizhető tulajdonságból áll: reprodukálhatóság, visszakereshetőség, telepíthetőség és hibakereshetőség. A CMake export és a `configure_package_config_file` a prefixfüggetlenséget, a SONAME és a `$ORIGIN`-alapú RUNPATH a dinamikus feloldást, a build-id és a `.gnu_debuglink` a symbolicationt, a `SOURCE_DATE_EPOCH` és a `-ffile-prefix-map` a determinizmust biztosítja. A deploy kapuja a cél-image-en futtatott smoke test és a libc-baseline ellenőrzése.

38.6.12 Feladatok

1. feladat (magyarázó). Egy kolléga a `libgreet.so`-t ezzel a kapcsolóval linkeli:

```
-Wl,-rpath,/home/fejleszto/greet/build
```

A konténerben a CLI nem indul. Magyarázd meg, miért nem működik, és miért nem orvosolja a problémát az, ha a `RUNPATH`-ot `$ORIGIN`-ra cseréljük a `greetctl`-n, miközben a `libgreet.so` saját függőségei a build könyvtárra mutatnak.

2. feladat (kódolási). Egészítsd ki a `CMakeLists.txt`-t úgy, hogy a telepített `greetctl` `$ORIGIN/../lib` RUNPATH-ot kapjon, a `libgreet` SONAME-je `libgreet.so.1` legyen, és a debug szétválasztás egy külön telepítési komponensbe kerüljön. Add meg a hozzá tartozó `objcopy` parancsokat és az ellenőrző `readelf` hívásokat.

3. feladat (diagnosztika). A `readelf -d` kivonatban a `greetctl` `NEEDED` listája `libgreet.so.1`, `libstdc++.so.6`, `libc.so.6`, `RUNPATH` értéke `$ORIGIN/../lib`. A `libgreet.so.1` `NEEDED` listája `libm.so.6`, `libc.so.6`, `RUNPATH` nélkül. Melyik függőség oldódik fel, és melyik nem, ha a `libm.so.6` csak a csomag `lib/` alkönyvtárában van, a rendszerkönyvtárban nem?

4. feladat (ellenőrzés). Tervezz olyan ellenőrző lépéssort, amely két különböző könyvtárból indított build kimenetéről eldönti, hogy reprodukálható-e, és fogalmazd meg, mit teszel eltérés esetén.

38.6.13 Megoldások

1. megoldás. Az abszolút rpath a build gép könyvtárára mutat, amely a konténerben nem létezik, ezért a loader nem találja a `libgreet.so.1`-et, és `cannot open shared object file` hibával leáll. A `$ORIGIN`-ra cserélés a `greetctl`-n csak a `greetctl` közvetlen függőségeit oldja fel, mert a `DT_RUNPATH` nem tranzitív: a `libgreet.so` saját függőségeit a saját bejegyzései alapján keresi a loader. Ezért a `libgreet.so`-t is `$ORIGIN`-alapú RUNPATH-tal kell linkelni (vagy a függőségeit a rendszerkönyvtárba telepíteni). Ellenőrzési feltétel: a `readelf -d` mindkét

objektumban $ORIGIN-t tartalmazó RUNPATH-ot mutat, és a lib/ könyvtárban minden NEEDED fájl megtalálható.

2. megoldás. A CMakeLists.txt-be a set_target_properties(greetctl PROPERTIES INSTALL_RPATH "$ORIGIN/../lib") és a set_target_properties(greet PROPERTIES SOVERSION 1 VERSION ${PROJECT_VERSION}) sorok kerülnek. A debug szétválasztáshoz a telepítés után futtatjuk az objcopy --only-keep-debug, objcopy --strip-debug --add-gnu-debuglink hármast, majd a .debug fájlt egy külön komponensbe telepítjük. Ellenőrzés: readelf -d a SONAME-ben libgreet.so.1-et, readelf -n build-id note-ot mutat, a .debug fájl pedig a greetctl mellett vagy a debug-könyvtárban elérhető.

3. megoldás. A greetctl közvetlen függőségei (libgreet.so.1, libstdc++.so.6, libc.so.6) a RUNPATH és a rendszerkönyvtárak alapján feloldódnak, ha a libgreet.so.1 a ../lib-ben van. A libm.so.6 viszont a libgreet.so.1 függősége, és mivel annak nincs RUNPATH-ja, a loader csak a rendszerkönyvtárakban keresi — ott pedig nincs, ezért a betöltés meghiúsul. Javítás: a libm.so.6-ot a rendszerkönyvtárba tenni, vagy a libgreet.so.1-et $ORIGIN-alapú RUNPATH-tal újralinkelni.

4. megoldás. Két külön build könyvtárból (build-a, build-b), azonos SOURCE_DATE_EPOCH, azonos prefix-map és azonos toolchain-profil mellett lefordítjuk a projektet, majd sha256sum-mal összehasonlítjuk a leszállított binárist, a könyvtárat és a debug fájlt. Eltérés esetén először a readelf -p .comment és a debug szekciók útvonalai alapján keresünk be nem normalizált abszolút utat, majd a __DATE__/__TIME__ előfordulásokat, végül az archívumok és LTO partitionök sorrendjét vizsgáljuk. Az ellenőrzés akkor tekinthető sikeresnek, ha a hash-ek bitre egyeznek, és a build-id is azonos.

39 C++ záróprojekt: a C rendszer átalakítása

39.1 Portterv és azonos viselkedési szerződés

A C záróprojekt akkor fejeződik be igazán, amikor a működő implementációt nem eldobjuk, hanem szerződésként használjuk a következő változathoz. Ez az alfejezet arról szól, hogyan lehet a C capstone viselkedését C++20-ra vinni úgy, hogy a port ne „újraírás” legyen, hanem ellenőrizhető transzformáció. A cél nem az, hogy minden C szerkezet helyére látványos C++ feature kerüljön, hanem az, hogy minden megváltoztatott döntés mögött legyen kimondott tulajdonlási, hibakezelési és elfogadási indok. A fejezet a portterv dokumentumára és annak első, konkrét szeletére összpontosít; a teljes szolgáltatás termékfejlesztését, üzemeltetését és a C++ nyelv teljes eszköztárának kötelező beépítését szándékosan nem tárgyalja.

A fejezet szabványos állításainak forrása a forrásjegyzék C++20 és C17 tétele. A konkrét helyek az N4861 tervezet szakaszcímkéivel: a nyelvi kötés [dcl.link]; a nem dobó kivételspecifikáció megsértése és a `std::terminate` [except.spec] és [except.terminate]; a `std::span` [views.span]; a placement new [new.delete.placement]; a `static_assert` és a `noexcept` operátor [dcl.pre], illetve [expr.unary.noexcept]. A fejezet kódját a `lab-gcc` profilban lefordítottuk és lefuttattuk; az eredményeket a buildjegyzőkönyvek rögzítik.

39.1.1 A viselkedési szerződés mint a port tárgya

A portolás kulcsfogalma a **viselkedési szerződés** (behavioral contract): azon megfigyelhető tulajdonságok halmaza, amelyekre a rendszer külső fogyasztói támaszkodhatnak, és amelyeknek a port után változatlannak kell maradniuk. Külső fogyasztó a protokoll kliens, a CLI-t használó operátor, a naplót elemző monitoring, a könyvtárat linkelő más program, és—ha van ilyen—a C ABI-n keresztül hívó idegen nyelvű kód. A szerződés elemei jellemzően a következők:

- a bemenet–kimenet leképezés, ideértve a bájtszintű protokollformátumot és a sorvégek kezelését;
- a hibakódok és jelentésük, valamint az, hogy mely bemenet melyik hibát váltja ki;
- az erőforrás-felső korlátok, amelyekre a hívó számít (például „egy munkamenet legfeljebb 1 048 576 bájt”);
- a diagnosztika formátuma és a kilépési kódok;
- a könyvtári határ ABI-ja: szimbólumnevek, enum-értékek, struktúraelrendezés, allokátor- és kivételszerződés;
- azok az időzítési vagy sorrendi garanciák, amelyeket a fogyasztó észlel.

Ezzel szemben a **szabadon változó belső tulajdonságok** közé tartozik a belső struktúrák elrendezése, az allokációk száma és mintája, a szálak száma, a belső segédfüggvények neve és a naplózás belső implementációja—feltéve, hogy a szerződés ezekre nem hivatkozik. A portterv legfontosabb fegyelme éppen ennek a két halmaznak az írásos szétválasztása. Ha ezt nem tesszük meg a kód megírása előtt, akkor a végén nem tudjuk megmondani, hogy egy viselkedésbeli eltérés hiba-e vagy megengedett változás.

Két további definíciót érdemes rögzíteni. A **golden teszt** olyan rögzített bemenet–kimenet pár, amelyet a port előtti implementációval vettünk fel, és amelyet a port után bájtszinten összehasonlítunk. A **tulajdonlási térkép** (ownership map) minden erőforrásra megmondja, ki allokálja, ki szabadítja fel, meddig él, és mely művelet invalidálja. A két eszköz bizonyító ereje korlátos, ezért a port ellenőrzését három külön kérdésre bontjuk. A **specifikációhoz viszonyított helyesség** kérdése, hogy a program azt teszi-e, amit a szerződés előír; ehhez független tesztorákulum kell, például a protokoll nyelvtanából kézzel levezetett elvárt értékek. A **C-hez viszonyított regressziós egyezés** kérdése, hogy a port ugyanazt teszi-e, mint a C referencia; erre való a golden teszt és a differenciális összevetés, de ezek a referencia esetleges hibáját is konzerválják. Az **ownership-invariánsok** kérdése, hogy a végrehajtás betartja-e a tulajdonlási térképet; a térkép maga csak dokumentáció, betartását sanitizeres futás, hibainjektálás és kódáttekintés igazolja, mindig csak a ténylegesen bejárt utakra. Végül az **ekvivalenciaosztály** a bemenet olyan részhalmaza, amelyre a szerződés azonos viselkedést ír elő; a tesztkészletnek minden osztályból legalább egy reprezentánst tartalmaznia kell.

39.1.2 Modulonkénti leképezés: a döntési tábla

A portterv gerince egy táblázat, amely minden C mintához hozzárendeli a választott C++ eszközt, a megőrzendő szerződést és a tipikus hibát. A táblázat nem recept, hanem döntési napló: minden sor mögött ott kell lennie annak az indoknak, hogy miért ez az eszköz, és miért nem egy másik.

C minta	C++20 eszköz	Megőrzendő szerződés	Tipikus hiba
`goto cleanup` sorozat	RAII owner (`unique_ptr` egyedi deleterrel, saját wrapper)	minden úton pontosan egyszer szabadít fel	dobó destruktor, dupla felszabadítás
`(const char*, size_t)` pár	`std::string_view`, `std::span<const std::byte>`	nem tulajdonol; a hívó élettartamához kötött	a view túléli az ownert
kézi tag + `switch`	`std::variant` + `std::visit`	kimerítő kezelés, a hibás tag továbbra is hiba	`std::get` dob, ha rossz alternatíva
`int` visszatérési kód	`std::error_code`, `std::variant`, C++23-ban `std::expected`	a kód–viselkedés leképezés stabil	kivétel átszivárgása a C határon
`void *ctx` callback	típusos `struct` + referencia vagy `span`	a tulajdonos és az élettartam dokumentált	típusvesztés, élettartam-túllépés

39.1.2.1 Cleanup ág → RAII owner

A C-ben a `goto cleanup` helyes és gyakori minta: egy függvény több erőforrást szerez meg egymás után, és minden hibaponton a már megszerzett erőforrásokat kell felszabadítani. A C++ megfelelője nem az, hogy „minden `goto`-t eltüntetünk”, hanem az, hogy az erőforrás élettartamát egy owner objektum élettartamához kötjük. Az owner lehet `std::unique_ptr` egyedi deleterrel (például `FILE*` vagy file descriptor esetén), lehet saját kis wrapper osztály, amelynek destruktora a felszabadítást végzi, és lehet a scope-ra kötött scope guard is.

A lényeg a **lokális helyesség**: minden owner csak a saját erőforrásáért felel, és a felszabadítás akkor is megtörténik, ha a függvény kivétellel vagy korai visszatéréssel lép ki. Ez nem esztétikai kérdés. Tegyük fel, hogy egy függvény n erőforrást szerez sorban, és minden lépés után hibázhat. A naiv kézi kódban az i-edik hibaponton i erőforrást kell felszabadítani, így a forrásban összesen

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

felszabadító utasítás szerepel. RAII esetén minden owner destruktora pontosan egyszer fut, tehát a tényleges felszabadítások száma n, és a kódban minden ownerhez egyetlen destruktor tartozik. A különbség nem a futásidejű költség, hanem a karbantarthatóság: a kézi változatban minden új erőforrás az összes korábbi hibapont módosítását igényli, a RAII-változatban csak egy új owner deklarációja kerül be.

Fontos feltétel: a destruktor ne dobjon, és a felszabadítás ne függjön attól, hogy a program milyen állapotban van. Ha a felszabadítás hibázhat (például egy flush vagy egy `close`), akkor azt nem a destruktorba kell tenni, hanem egy explicit `commit`/`close` műveletbe, amelynek hibája a hívóhoz tartozik. A destruktor csak a „már nem lehet mást tenni” esetet kezeli.

39.1.2.2 Byte view → `span` és `string_view`

A C API gyakori mintája a `(pointer, hossz)` pár. Ennek C++20-as megfelelője szöveges adatra a `std::string_view`, bináris adatra a `std::span<const std::byte>`. Mindkettő **nem tulajdonló nézet** (borrowed view): nem allokál, nem másol, és élettartama ahhoz a tárolóhoz kötött, amelyre mutat. A porttervnek ezt a szerződést explicit módon rögzítenie kell, mert a view visszaadása a legkönnyebben keletkező dangling hiba forrása. Ha egy függvény `std::string_view`-t ad vissza egy lokális `std::string`-ből, a hívó érvénytelen memóriára mutató nézetet kap; ezt a fordító nem feltétlenül jelzi.

A `std::span` használatakor érdemes a `std::byte`-ot választani a `char` helyett, ha az adat bináris: a `std::byte` kifejezi, hogy nem karakterekről van szó, és megakadályozza a véletlen szövegműveleteket. A `std::string_view` beágyazott nulla bájtot is képes tárolni, ellentétben a nullterminált C sztringgel; ez viselkedésváltozás lehet, ha az eredeti C kód `strlen`-re támaszkodott. A port során ezért a „sztring” szót minden előfordulásnál pontosítani kell: nullterminált, hosszal megadott, vagy mindkettő.

39.1.2.3 Tagged union → `variant`

A C-ben a tagged union kézi `enum` tag + `union` + `switch` hármasa. C++20-ban a `std::variant` ugyanezt a szerepet tölti be típusbiztosabban: a `std::visit` kimerítő kezelést kényszerít, és a nem várt tag elfelejtése fordítási hibát ad. Két dolgot azonban a portnak meg kell őriznie. Egyrészt a `std::get` kivételt dob, ha az alternatíva nem az, amit kértünk; a C változatban egy ilyen helyzet jellemzően nem definiált viselkedés vagy csendes hibás érték volt. A port nem „javíthatja” csendben a hibát kivétellé, ha a hívó ezt nem várja; a viselkedést a szerződésben kell rögzíteni. Másrészt a `variant` alternatíváinak sorrendje és a hozzá tartozó indexek a belső szerződés részei lehetnek, ha a `variant`t perzisztáljuk vagy ABI-határon adjuk át—ezt a porttervnek külön kell tiltania vagy engedélyeznie.

39.1.2.4 Hibakód → hibapolicy

A hibakezelés portja nem „cseréljük le a hibakódokat kivételekre”, hanem **hibapolicy**-t definiálunk. A tipikus, jól működő felosztás három réteg:

1. **Core library**: platformfüggetlen, memóriában tesztelhető logika. Itt lehet kivétel vagy `std::expected` (C++23), illetve saját `result` típus; a választás a csapat konvenciója, de a core-on belül konzisztens.
2. **Platform adapter**: vékony réteg, amely az OS hibáit (`errno`, `EAGAIN`, `EPIPE`) `std::error_code`-dé alakítja, és a core felé a core hibapolicy-ját használja.
3. **C ABI határ**: `extern "C"` függvények, amelyek soha nem dobnak, és az eredeti C hibakódokat adják vissza.

A döntési táblát a porttervben rögzíteni kell: melyik réteg melyik hibát hogyan reprezentálja, mi a leképezés a rétegek között, és mely hibakódok számítanak „nem fordulhat elő” esetnek. A C ABI enum-értékeit soha nem szabad átrendezni, még akkor sem, ha a C++ oldalon már nem használjuk őket: a már lefordított kliensek a numerikus értékekre támaszkodnak.

39.1.2.5 Core library és platform adapter

A port egyik legnagyobb nyeresége a szétválasztás. A core library nem tartalmazhat `#include <sys/socket.h>`-t, nem hívhat `read`-et, és nem ismerheti a file descriptorokat. A core bemenete egy `std::string_view` vagy `std::span`, kimenete egy értéktípus; a tesztje memóriában fut, sanitizerekkel és property tesztekkel. A platform adapter ezzel szemben vékony: átveszi az OS eseményeit, `std::error_code`-dé alakítja a hibákat, és továbbadja a core-nak. Ennek a szétválasztásnak a haszna a port szempontjából az, hogy a viselkedési szerződés nagy része a core-ban tesztelhető, és a port után a golden tesztek többsége OS nélkül, determinisztikusan futtatható.

39.1.3 A C-kompatibilis határ

Ha a C++ implementációt meg kell tartani egy stabil C API mögött, akkor a határnak saját szerződése van. Három szabályt érdemes betartani, és közben három fogalmat külön kell tartani.

A **nyelvi kötés** (`extern "C"`) a név- és hívási konvenciót határozza meg, nem kivételspecifikáció. Egy C kötésű, de C++-ban definiált függvény C++ hívó felé ugyanúgy dobhat, mint bármely más függvény. A `noexcept` kivételspecifikáció: ha egy `noexcept` függvényből kivétel próbál kilépni, `std::terminate` hívódik. Az **ABI és a futtatókörnyezet** dönti el, mi történik, ha a kivétel valódi idegen nyelvi kereteken, például C-ben fordított hívón át terjedne; ez a C és C++ szabvány szempontjából nem hordozható viselkedés, ezért erre semmilyen módon nem építünk. A különbséget egy kis próba mutatja:

```
// tests/ffi_probe.cpp -- a nyelvi kötés nem kivételspecifikáció
#include <cstdio>
#include <cstdlib>
#include <exception>

extern "C" int plain_entry() { throw 42; }            // C kötés, dobhat
extern "C" int noexcept_entry() noexcept { throw 42; } // C kötés, noexcept

int main()
{
```

```
    try {
        (void)plain_entry();
        std::puts("plain: nem dobott");
    } catch (int) {
        std::puts("plain: a kivétel a C++ hívóhoz eljutott");
    }

    std::set_terminate([] {
        std::puts("noexcept: std::terminate");
        std::fflush(stdout);           // az _Exit nem üríti a puffert
        std::_Exit(EXIT_SUCCESS);
    });
    (void)noexcept_entry();
    std::puts("noexcept: nem várt folytatás");
    return EXIT_FAILURE;
}
```

GCC 13.3.0-val fordítva a kimenet: `plain: a kivétel a C++ hívóhoz eljutott`, majd `noexcept: std::terminate`. A GCC a második függvénynél fordításkor `-Wterminate` figyelmeztetést is ad.

Ebből következik a projekt **FFI-szabálya**: egyetlen C ABI belépési pontból sem léphet ki kivétel. Ezt vagy úgy biztosítjuk, hogy a belépési pont csak bizonyítottan nem dobó műveleteket hív, és ezt fordításkor `static_assert(noexcept(...))` ellenőrzi; vagy úgy, hogy a törzs `try/catch (...)` blokkban fut, és minden kivételt dokumentált hibakódra képez. A projekt wrappere az első utat követi.

Másodszor: az **opaque handle** (áttetsző fogantyú) mögött a C++ objektum rejtve marad; a kliens csak mutatót lát, és a létrehozás/felszabadítás ugyanabban a modulban történik, hogy az allokátor párosítva legyen.

Harmadszor: a wrapper **ne változtasson csendben** a szerződésen, egyik irányban sem. Három változásfajtát kell megkülönböztetni:

- Az **előfeltétel erősítése** kevesebb bemenetet enged meg, mint korábban. Ez a meglévő hívókat törheti, mert egy eddig szabályos hívás tiltottá válik.
- Az **előfeltétel gyengítése** több bemenetet enged meg. Ha például az eredeti API-nál a null pointer átadása nem definiált viselkedés volt, a wrapper pedig ezentúl definiált hibakódot ad rá, akkor az elfogadott bemeneti tartományt bővítettük. A régi hívókat ez nem töri, de új kliens már támaszkodhat rá, így a későbbi visszavonása kompatibilitási törés lenne.
- Az **utófeltétel pontosítása** az addig is megengedett bemenetekre ad szűkebb, pontosabb eredményt, például egy addig meghatározatlan hibakódot rögzít.

Mindhárom megfigyelhető változás, ezért a porttervben külön migrációs tételként szerepel, dokumentációval és teszttel. A projekt portja egyiket sem vezeti be: a nem null parser továbbra is előfeltétel.

Az allokátorszerződés is a határ része. Ha a C API-ban a hívó allokál és a könyvtár szabadít fel (vagy fordítva), akkor a C++ oldalon ugyanazt az allokátort kell használni; `new/delete` és `malloc/free` keverése nem megengedett. A porttervnek rögzítenie kell, hogy a határon átadott buffereket ki allokálja, ki felszabadítja, és milyen élettartamig érvényesek.

39.1.4 Kidolgozott példa: a 17.2 rekordparser portja

A port kiindulópontja egyetlen, rögzített C referencia-implementáció: a C könyvtári mag és CLI megvalósítási labor `include/diag.h`, `src/diag.c` és `src/main.c` fájlja, a projektszerződés nyelvtanával, kimeneti formátumával és kilépési kódjaival. A port előtt ennek a buildjével vesszük fel a golden kimeneteket, és verziókövetésbe tesszük a tesztkészletet.

A megőrzendő szerződés leltára.

Szerződéselem	Forrás	Ellenőrzés a port után
Rekordnyelvtan, `records`/`sum`/`bytes` jelentése	17.1	`tests/spec_examples.c`, `tests/test_diag.c`
Hibakódok és offsetek, ragadós hiba, `finish` utáni `feed`	17.1, 17.2	`tests/test_diag.c`, differenciális korpusz
CLI stdout, stderr és kilépési kód	17.1	golden fájlok, `tests/cli_args.sh`
C ABI: `dg_*` nevek, `dg_code` értékek, struktúrák, allokátorszerződés	`include/diag.h`	változatlan header, C-ben fordított tesztek
$O(n)$ idő, $O(1)$ parsermemória, egyetlen allokáció	17.2	kódáttekintés, allokációs teszt

Migrációs napló. Ebben a portban megfigyelhető viselkedés nem változik; minden változás belső:

1. Az állapotgép a `diag::parser` C++ osztályba kerül; a fázis `enum class`, a bemenet `std::span<const unsigned char>`.
2. A `dg_report` és a `dg_error` megfelelője a `diag::report` és a `diag::error` értéktípus.
3. A `dg_*` C ABI-t egy `extern "C"` wrapper szolgáltatja a C++ core fölött. Az `include/diag.h` nem változik.
4. A `src/main.c` CLI változatlanul C17-ben fordul, és a C++ könyvtárhoz linkeljük.

Ha egy későbbi változat megfigyelhető viselkedést módosítana — például új hibakódot vezetne be —, az külön, számozott migrációs tétel lenne, saját tesztekkel és a golden fájlok dokumentált frissítésével. Az „azonos viselkedés” állítás csak a fenti ellenőrzések sikere után használható.

A C++ core felülete:

```
// include/diag_core.hpp -- C++20 core, nem tud az OS-ről
#pragma once
#include <cstddef>
#include <span>

namespace diag {

inline constexpr std::size_t max_input = 1048576;

// A numerikus értékek a C dg_code értékeivel egyeznek.
enum class code { ok = 0, syntax, truncated, too_large, state };

struct error {
    code kind = code::ok;
```

```
    std::size_t offset = 0;
};

struct report {
    std::size_t bytes = 0;
    std::size_t records = 0;
    unsigned long long sum = 0;
};

// A 17.2 C állapotgépének portja. Nem allokál, nem dob kivételt,
// és a feed hívás után nem őrzi meg a bemeneti span-t.
class parser {
public:
    bool feed(std::span<const unsigned char> input) noexcept;
    bool finish() noexcept;

    [[nodiscard]] error last_error() const noexcept { return error_; }
    [[nodiscard]] report current() const noexcept { return report_; }

private:
    enum class phase { key, number };

    bool fail(code kind) noexcept;

    report report_{};
    error error_{};
    std::size_t key_length_ = 0;
    unsigned digits_ = 0;
    unsigned long value_ = 0;
    phase phase_ = phase::key;
    bool finished_ = false;
};

} // namespace diag
```

A megvalósítást a C++ core alfejezet adja. A C ABI wrapper ugyanazokat a szimbólumokat definiálja, mint a `src/diag.c`; ezért a két megvalósítás **soha nem kerül ugyanabba a linkelésbe**, hanem két külön könyvtár készül: `libdiag.a` a C-ből és `libdiagxx.a` a C++-ból.

```
// src/diag_c_api.cpp -- a 17.2 diag.h C ABI-ja a C++ core fölött
#include "diag_core.hpp"

extern "C" {
#include "diag.h"
}

#include <cstdlib>
#include <new>
#include <type_traits>
#include <utility>

static_assert(static_cast<int>(diag::code::ok) == DG_OK);
```

```
static_assert(static_cast<int>(diag::code::syntax) == DG_SYNTAX);
static_assert(static_cast<int>(diag::code::truncated) == DG_TRUNCATED);
static_assert(static_cast<int>(diag::code::too_large) == DG_TOO_LARGE);
static_assert(static_cast<int>(diag::code::state) == DG_STATE);
static_assert(diag::max_input == DG_MAX_INPUT);

// FFI-szabály: egy belépési pontból sem léphet ki kivétel. Itt ezt az
// biztosítja, hogy csak nem dobó core-műveleteket hívunk; ha ez
// megváltozna, a fordítás itt megáll, és try/catch blokk kell.
static_assert(std::is_nothrow_default_constructible_v<diag::parser>);
static_assert(noexcept(std::declval<diag::parser &>().feed({})));
static_assert(noexcept(std::declval<diag::parser &>().finish()));
static_assert(noexcept(std::declval<const diag::parser &>().last_error()));
static_assert(noexcept(std::declval<const diag::parser &>().current()));

struct dg_parser {
    dg_allocator allocator;
    diag::parser core;
};

extern "C" {

static void *default_allocate(size_t size, void *)
{
    return std::malloc(size);
}

static void default_release(void *pointer, void *)
{
    std::free(pointer);
}

dg_parser *dg_create(const dg_allocator *allocator)
{
    dg_allocator chosen = { default_allocate, default_release, nullptr };
    if (allocator != nullptr)
        chosen = *allocator;

    void *storage = chosen.allocate(sizeof(dg_parser), chosen.context);
    if (storage == nullptr)
        return nullptr;
    // Nem dobó konstrukció: a C határon nem léphet át kivétel.
    return ::new (storage) dg_parser{chosen, diag::parser{}};
}

void dg_destroy(dg_parser *parser)
{
    if (parser != nullptr) {
        const dg_allocator allocator = parser->allocator;
        parser->~dg_parser();
        allocator.release(parser, allocator.context);
    }
```

```
}

bool dg_feed(dg_parser *parser, dg_view input)
{
    return parser->core.feed({input.data, input.size});
}

bool dg_finish(dg_parser *parser)
{
    return parser->core.finish();
}

dg_error dg_get_error(const dg_parser *parser)
{
    const diag::error e = parser->core.last_error();
    return dg_error{static_cast<dg_code>(e.kind), e.offset};
}

dg_report dg_get_report(const dg_parser *parser)
{
    const diag::report r = parser->core.current();
    return dg_report{r.bytes, r.records, r.sum};
}

const char *dg_code_name(dg_code code)
{
    switch (code) {
    case DG_OK:        return "ok";
    case DG_SYNTAX:    return "syntax";
    case DG_TRUNCATED: return "truncated";
    case DG_TOO_LARGE: return "limit";
    case DG_STATE:     return "state";
    }
    return "state";
}

} // extern "C"
```

A példa több portdöntést is láthatóvá tesz. A C header változatlan, ezért a C++ fordítási egység `extern "C"` blokkba ágyazva include-olja; így a deklarációk C nyelvi kötést kapnak, a header pedig nem tartalmaz C++-ra vonatkozó módosítást. A `static_assert` sorok fordításkor ellenőrzik, hogy a C++ hibakódok numerikus értékei egyeznek a C ABI értékeivel. A wrapper nem ad hozzá új ellenőrzést: ahol a C API előfeltétele nem null parser volt, ott a C++ változat is előfeltételként kezeli. Az FFI-szabályt a második `static_assert`-csoport fordításkor ellenőrzi: a wrapper csak `noexcept` core-műveleteket és nem dobó placement new-t hív, így a C határon nem léphet át kivétel. Ha egy későbbi változtatás a `feed` elől elvenné a `noexcept`-et, a wrapper fordítása ezen a ponton megállna; ilyenkor a belépési pontot `try`/`catch` blokkal kell hibakóddá alakítani. Ezt a negatív próbát GCC 13.3.0-val elvégeztük: a `noexcept` eltávolítása után a fordítás a `feed`-re vonatkozó `static_assert`-nél leállt.

A buildparancsok a laborhoz. A C forrás C fordítóval, C17 módban fordul; a C++ forrás C++ fordítóval; a végső linkelést a C++ driver végzi, mert a C++ futtatókönyvtárat is hozzá kell adni:

```
set -eu
mkdir -p build/gcc build/cxx

# Referencia: a 17.2 buildje, változatlanul
gcc -std=c17 -O2 -g -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
    -Iinclude -c src/diag.c -o build/gcc/diag.o
ar rcs build/gcc/libdiag.a build/gcc/diag.o

# Port: C++20 core és C ABI wrapper
for f in diag_core diag_c_api; do
    g++ -std=c++20 -O2 -g -Wall -Wextra -Wpedantic -Wconversion \
        -Wshadow -Iinclude -c "src/$f.cpp" -o "build/cxx/$f.o"
done
ar rcs build/cxx/libdiagxx.a build/cxx/diag_core.o build/cxx/diag_c_api.o

# Ugyanaz a C CLI és ugyanazok a C tesztek, a C++ könyvtárral
gcc -std=c17 -O2 -g -Wall -Wextra -Wpedantic -D_POSIX_C_SOURCE=200809L \
    -Iinclude -c src/main.c -o build/cxx/main.o
gcc -std=c17 -O2 -g -Wall -Wextra -Wpedantic \
    -Iinclude -c tests/test_diag.c -o build/cxx/test_diag.o
gcc -std=c17 -O2 -g -Wall -Wextra -Wpedantic \
    -Iinclude -c tests/spec_examples.c -o build/cxx/spec_examples.o
g++ build/cxx/main.o build/cxx/libdiagxx.a -o build/cxx/diag
g++ build/cxx/test_diag.o build/cxx/libdiagxx.a -o build/cxx/test_diag
g++ build/cxx/spec_examples.o build/cxx/libdiagxx.a \
    -o build/cxx/spec_examples
```

A C++ változat tehát a **C referencia saját tesztjeit** futtatja, nem újakat. Ezt egy differenciális teszt egészíti ki, amely tetszőleges bemenetkorpuszon összeveti a két CLI teljes megfigyelhető kimenetét:

```
#!/bin/sh
# tests/diff_c_cxx.sh C_DIAG CXX_DIAG KORPUSZ_KÖNYVTÁR
# Minden *.in bemenetre összeveti a két CLI stdoutját, stderrjét
# és kilépési kódját. Eltérésnél a fájl nevét írja ki.
set -u
c=$1 cxx=$2 dir=$3
tmp=$(mktemp -d) || exit 2
fail=0 count=0

for f in "$dir"/*.in; do
    "$c"   "$f" >"$tmp/c.out"   2>"$tmp/c.err";   echo $? >"$tmp/c.rc"
    "$cxx" "$f" >"$tmp/cxx.out" 2>"$tmp/cxx.err"; echo $? >"$tmp/cxx.rc"
    for k in out err rc; do
        if ! cmp -s "$tmp/c.$k" "$tmp/cxx.$k"; then
            echo "DIFF $k: $f" >&2
            fail=1
        fi
    done
    count=$((count + 1))
done
```

```
rm -rf "$tmp"
echo "checked=$count"
exit $fail
```

A korpusz rögzített véletlenmaggal generálható, így bárki pontosan ugyanazt a 3016 bemenetet kapja:

```
#!/usr/bin/env python3
"""tests/gen_corpus.py KÖNYVTÁR -- a differenciális korpusz előállítása"""
import os
import random
import sys

out = sys.argv[1]
os.makedirs(out, exist_ok=True)
cases = [b"", b"a=7\ncpu=12\n", b"a=7\ncpu=12", b"a=65535\n", b"a=65536\n",
         b"a=000001\n", b"abcdefghijklmnop=1\n", b"abcdefghijklmnopq=1\n",
         b"a=1\r\n", b"a=\n", b"=1\n", b"a=1\x00\n", b"cpu", b"cpu=",
         b"a=0\n" * 262144, b"a=0\n" * 262144 + b"b"]
rnd = random.Random(20261025)
alphabet = b"abz=0159\n\rA \x00"
for _ in range(3000):
    n = rnd.randint(0, 40)
    cases.append(bytes(rnd.choice(alphabet) for _ in range(n)))
for i, case in enumerate(cases):
    with open(os.path.join(out, f"{i:04d}.in"), "wb") as f:
        f.write(case)

python3 tests/gen_corpus.py corpus
build/cxx/test_diag
build/cxx/spec_examples
build/cxx/diag tests/golden.in | cmp tests/golden.out -
sh tests/cli_args.sh build/cxx/diag
sh tests/diff_c_cxx.sh build/gcc/diag build/cxx/diag corpus
```

Buildjegyzőkönyv (`lab-gcc` profil). A fenti buildet és teszteket Linux x86-64 környezetben, GCC/G++ 13.3.0 és glibc 2.39 mellett lefuttattuk, `-Werror` beállítással is figyelmeztetés nélkül. A C++ könyvtárral linkelt `test_diag`, `spec_examples`, golden és `cli_args.sh` teszt sikeres volt; a `test_diag` ASan/UBSan builddel is. A differenciális korpusz 3016 bemenetből állt: a specifikációs példák, a pontos méretlimit és a limit utáni bájt, valamint 3000 rögzített véletlenmaggal generált, legfeljebb 40 bájtos bemenet a `abz=0159`, LF, CR, `A`, szóköz és `0x00` oktettekből. A két CLI stdoutja, stderrje és kilépési kódja egyik bemeneten sem tért el. Ez regressziós egyezés a C referenciával, nem a specifikációhoz viszonyított teljes helyességi bizonyítás; utóbbit a specifikációból levezetett `spec_examples.c` és a 17.2 bizonyításvázlatai támasztják alá.

39.1.5 Elfogadási feltételek

A port akkor kész, ha a szerződés minden dimenziójában van ellenőrizhető feltétel. Öt kategóriát érdemes külön kezelni; az első a specifikációhoz, a második és a harmadik a C referenciához mér, a negyedik az ownership-invariánsokat vizsgálja.

Specifikációs elfogadás. A projektszerződés nyelvtanából kézzel levezetett elvárt értékek (`tests/spec_examples.c`) a C és a C++ változaton is teljesülnek. Ez a független orákulum: nem

a referencia kimenetéből készült, ezért a referencia hibáját is felismeri. Ezt egy mutációs próba igazolja, amely szándékosan elrontja a C referenciát:

```
#!/bin/sh
# tests/mutation_probe.sh -- szándékosan hibás C referencia felismerése.
# A projekt gyökeréből futtatandó, a 17.2 fixture-jei után.
set -eu
tmp=$(mktemp -d) || exit 2
trap 'rm -rf "$tmp"' EXIT

# Mutáció: hat számjegyet is elfogad (a nyelvtan legfeljebb ötöt enged).
sed 's/parser->digits == 5/parser->digits == 6/' src/diag.c \
    > "$tmp/diag_bad.c"
if cmp -s src/diag.c "$tmp/diag_bad.c"; then
    echo "a mutáció nem alkalmazható" >&2
    exit 2
fi
gcc -std=c17 -O2 -Iinclude -D_POSIX_C_SOURCE=200809L \
    "$tmp/diag_bad.c" src/main.c -o "$tmp/diag_bad"
gcc -std=c17 -O2 -Iinclude \
    "$tmp/diag_bad.c" tests/spec_examples.c -o "$tmp/spec_bad"

# A hibás referenciából felvett golden a hibát rögzítené:
printf 'a=000001\n' | "$tmp/diag_bad"
# a meglévő golden pedig nem érinti a hibás ágat:
"$tmp/diag_bad" tests/golden.in | cmp -s tests/golden.out -
echo "golden: nem ismeri fel"

# A specifikációból kézzel levezetett példák felismerik:
if "$tmp/spec_bad" 2>/dev/null; then
    echo "HIBA: a specifikációs teszt nem bukott" >&2
    exit 1
fi
echo "spec_examples: felismerte a hibás referenciát"
```

A futás eredménye: a hibás referencia az `a=000001` bemenetre `OK records=1 sum=1 bytes=9` sort ad, amelyet egy belőle felvett golden fájl hibátlannak rögzítene; a meglévő golden teszt nem bukik; a `spec_examples` viszont bukik. A próbát GCC 13.3.0-val lefuttattuk, a leírt eredménnyel.

Funkcionális elfogadás. Minden nyelvtani ekvivalenciaosztályra és minden hibakódra létezik legalább egy teszt, amely a port előtti és a port utáni implementáción ugyanazt az eredményt adja. A teszt a hibakód mellett az offsetet és a három számlálót is ellenőrzi, nem csak a visszatérési értéket.

Golden elfogadás. A rögzített bemenetekre a kimenet bájtszinten egyezik. Ide tartozik a protokollválasz, a naplósor formátuma és a kilépési kód. A golden fájlokat a port előtt, a C implementációval kell felvenni, és verziókövetésbe tenni; a port után ezekhez nem nyúlunk.

Ownership elfogadás. Az AddressSanitizer és a LeakSanitizer hibamentes futása a teljes tesztkészleten, az allokációs hibát injektáló esettel együtt. Ez a bejárt utakon kizárja a dupla felszabadítást, a use-after-free-t és a szivárgást, de nem bizonyítja, hogy minden lehetséges úton így van; a többi utat a tulajdonlási térkép szerinti kódáttekintés fedi. Mivel a port core-ja nem tárol nézetet a hívás után, a `std::span` élettartamát a hívó puffer újrahasználata teszteli: a `feed` után

a puffer felülírása nem változtathatja meg az eredményt. Ezt a C++ core `tests/test_core.cpp` tesztje ellenőrzi.

Benchmark elfogadás. A mérőszámokat a port előtt, ugyanazon a gépen, ugyanazzal a buildprofillal és ugyanazzal a bemeneti halmazzal kell rögzíteni. Jellemzően: átbájt/másodperc, p50 és p99 feldolgozási idő, csúcs RSS, allokációk száma. A port utáni mérés akkor fogadható el, ha a rögzített toleranciasávon belül marad; a sávot szintén a port előtt kell megadni, különben utólag bármi „elfogadható” lesz. A mérés nélküli állítás—„ez a változat gyorsabb”—nem része az elfogadásnak.

39.1.6 Gyakori hibák és javításuk

Az első tipikus hiba a **nézet élettartamának elszalasztása**: egy „optimalizált” port a `feed` hívásban kapott `std::span`-t tagváltozóban megőrzi, hogy a következő hívásnál onnan folytassa a félbemaradt rekordot. A C referencia szerződése szerint a bemenet csak a hívás idejére kölcsönzött; a hívó a puffert a visszatérés után újrahasználja, így a megőrzött nézet más adatra mutat. A javítás az, hogy a core csak a szükséges állapotot (fázis, névhossz, számjegyszám, részérték) tárolja, a bemenetre mutató nézetet soha.

A második a **kivétel a C határon**. Ha a wrapper belsejében egy `std::string` allokáció `std::bad_alloc`-ot dob, és a wrapper nem kapja el, a folyamat leáll. A javítás a `try`/`catch` a wrapperben és a hibakód leképezése, vagy az, hogy a wrapper csak nem dobó műveleteket hív.

A harmadik a **hibakódok átrendezése**. Ha a C++ oldalon „szebb” sorrendbe tesszük az enum értékeket, a már lefordított kliensek rossz ágat futtatnak. A javítás: az enum numerikus értékei változatlanok, az új értékek a végére kerülnek.

A negyedik a **nézet túléli a mögötte lévő puffert**: a `std::span` egy `std::vector`-ra mutat, majd a vektor újraallokál, és a span dangling lesz. A javítás az invalidáló művelet dokumentálása és a span élettartamának szűkítése.

Az ötödik a **csendes szemantikaváltás**: a C változat explicit bájtösszehasonlítással (`'a'`–`'z'`, `'0'`–`'9'`) ellenőriz, a C++ változatba viszont `std::islower` vagy `std::isdigit` kerül. Ezek a locale-tól függhetnek, és `char` argumentummal negatív értékre nem definiáltak; a nyelvtan viszont rögzített ASCII-oktetteket ír elő. A javítás az explicit összehasonlítás megtartása, és olyan korpuszelemek, amelyek nem ASCII-, nagybetűs és `0x00` oktettet is tartalmaznak.

A hatodik a **mérés elhalasztása**. Ha a benchmarkot csak a port után vesszük fel, nincs összehasonlítási alap, és a „nem lett lassabb” állítás igazolhatatlan. A javítás a portterv mellékleteként rögzített mérési protokoll.

39.1.7 Feladatok

1. feladat (magyarázó). Készítsd el a `dg_*` API megfigyelhető viselkedésének leltárát! Sorold fel a kimeneti kategóriákat, a bemenet ekvivalenciaosztályait, és minden osztályra adj egy reprezentáns bemenetet. Jelöld, mely viselkedések tartoznak a szerződéshez és melyek a szabadon változó belső tulajdonságokhoz.

2. feladat (kódolási). Írj C17 tesztet, amely a terminális állapotokat ellenőrzi: ismételt `finish`, `finish` utáni `feed`, valamint a ragadós hiba. Ellenőrizd a `dg_code` numerikus értékeit is. Fordítsd le egyszer, és linkeld a C és a C++ könyvtárhoz is.

3. feladat (diagnosztikai). Készíts golden teszt tervet és port előtti benchmark protokollt: mely bemeneteket rögzítesz, milyen metrikákat mérsz, milyen toleranciasávot fogadsz el, és hogyan bizonyítod, hogy a mérés ugyanazon a profilon történt.

39.1.8 Megoldások

1. megoldás. A kimeneti kategóriák: siker (`DG_OK`), `DG_SYNTAX`, `DG_TRUNCATED`, `DG_TOO_LARGE` és `DG_STATE`, a CLI-ben ezekhez a 0, 4 és 5 kilépési kód tartozik. Bemeneti ekvivalenciaosztályok és reprezentánsok: (a) üres bemenet: `""`; (b) érvényes rekordok: `"a=7\ncpu=12\n"`; (c) érvénytelen névkarakter: `"A=1\n"`; (d) üres név vagy érték: `"=1\n"`, `"a=\n"`; (e) 17 betűs név; (f) hat számjegy, illetve 65535 fölötti érték: `"a=000001\n"`, `"a=65536\n"`; (g) CRLF és beágyazott `0x00`; (h) csonka rekord: `"cpu="`; (i) pontosan limitméretű bemenet, illetve egy további bájt; (j) `finish` utáni `feed`. Minden osztály határértékei külön tesztesetet kapnak (például 16 és 17 betű, 65535 és 65536). A szerződés része a kód, az offset, a három számláló és a CLI-kimenet; szabadon változó a belső állapot elrendezése, a segédfüggvények neve és a fázisok ábrázolása. Ellenőrzési feltétel: minden osztályhoz és határhoz tartozik legalább egy teszteset.

2. megoldás.

```
/* tests/test_terminal.c -- terminális állapotok mindkét könyvtáron */
#include "diag.h"

#include <stdio.h>
#include <stdlib.h>

#define CHECK(condition) do {                                          \
    if (!(condition)) {                                                \
        (void)fprintf(stderr, "CHECK failed at line %d\n", __LINE__); \
        exit(EXIT_FAILURE);                                            \
    }                                                                  \
} while (0)

int main(void)
{
    static const unsigned char ok[] = "a=1\n";
    static const unsigned char bad[] = "a=x\n";

    CHECK(DG_OK == 0 && DG_SYNTAX == 1 && DG_TRUNCATED == 2 &&
          DG_TOO_LARGE == 3 && DG_STATE == 4);

    dg_parser *p = dg_create(NULL);
    CHECK(p != NULL);
    CHECK(dg_feed(p, (dg_view){ ok, 4 }));
    CHECK(dg_finish(p) && dg_finish(p));        /* ismételt finish */
    CHECK(!dg_feed(p, (dg_view){ ok, 4 }));     /* finish után */
    CHECK(dg_get_error(p).code == DG_STATE);
    CHECK(dg_get_error(p).offset == 4);
    dg_destroy(p);

    p = dg_create(NULL);
    CHECK(p != NULL);
    CHECK(!dg_feed(p, (dg_view){ bad, 4 }));
    CHECK(!dg_feed(p, (dg_view){ ok, 4 }));     /* ragadós hiba */
    CHECK(!dg_finish(p));
    CHECK(dg_get_error(p).code == DG_SYNTAX);
    CHECK(dg_get_error(p).offset == 2);
    dg_destroy(p);
```

```
    return 0;
}
```

A tesztet egyszer, C fordítóval kell objectté fordítani, majd egyszer a `build/gcc/libdiag.a`, egyszer a `build/cxx/libdiagxx.a` könyvtárral linkelni (utóbbit `g++` driverrel). A `finish` utáni `feed` offsetje 4, mert addig 4 bájt fogyott el; a ragadós hiba offsetje 2, az `x` helye. Ellenőrzési feltétel: mindkét linkelt program `0` kóddal lép ki. A tesztet GCC 13.3.0-val mindkét könyvtárra lefuttattuk, sikerrel.

3. megoldás. A golden készlet az 1. megoldás ekvivalenciaosztályainak és határainak reprezentánsait tartalmazza, mindegyikhez a C referenciával felvett, rögzített stdout-, stderr- és kilépésikód-értékkel. A benchmark protokoll rögzíti a gépet, a fordítót, a `-O2` profilt, a bemeneti fájlt, az ismétlésszámot és a mért metrikákat (átbájt/másodperc, p50, p99, csúcs RSS, allokációk száma). A toleranciasávot a port előtt kell megadni, például „a p99 legfeljebb 10%-kal nőhet, az átbájt/másodperc legfeljebb 5%-kal csökkenhet”. Az ellenőrzési feltétel: a port utáni mérés ugyanazon a profilon, ugyanazzal a bemenettel történik, és az eredmény a sávon belül van; ha nem, a porttervnek indokolnia kell az eltérést, nem pedig utólag módosítania a sávot.

39.1.9 Összefoglaló

A portterv a viselkedési szerződés írásos rögzítése: mi az, ami a port után változatlan, és mi az, ami szabadon változhat. A C cleanup ágak RAII ownerré alakulnak, a `(pointer, hossz)` párok `std::string_view` és `std::span` nézetekké, a kézi tagged union `std::variant`-tá, a hibakódok pedig rétegenkénti hibapolicy-vá. A core library és a platform adapter szétválasztása tesztelhetővé teszi a viselkedés nagy részét, a C ABI wrapper pedig megőrzi a stabil határt. Az elfogadás négy lába a funkcionális, a golden, az ownership és a benchmark feltétel; a mérést a port előtt kell rögzíteni, különben nincs összehasonlítási alap. A következő alfejezet erre a tervre építve magát a core implementációt és az értéktípusokat dolgozza ki.

39.2 C++ core: parser, erőforrások és értéktípusok

A C záróprojekt funkcionális szerződését a C++ port nem értelmezi újra: ugyanazt a rekordnyelvtant, ugyanazokat a hibakódokat és hibapozíciókat, ugyanazt a CLI-kimenetet kell produkálnia, mint a 17.2 C referencia. A Portterv és azonos viselkedési szerződés (39.1) rögzítette a viselkedési határt, a migrációs naplót és a C ABI wrappert; ez a rész a C++ magot építi fel: az értéktípusokat, a parser állapotgépét, a platformoldali erőforrás-tulajdonosokat és a teszteket. A cél nem az, hogy minden új C++ eszközt beépítsünk, hanem hogy a szerződést a típusokba kódoljuk, és a hibalehetőségeket fordítási időbe emeljük.

39.2.1 A port szerződése: value, owner, view

A C++ port három szerepet különböztet meg. Az **értéktípus** (value type) önálló, invariánssal rendelkező érték: másolható vagy mozgatható, és minden megfigyelhető állapota kielégíti az invariánst. A **tulajdonos** (owner) erőforrást birtokol, és a destruktorában felszabadítja; a tulajdonjog mozgatható, de nem másolható, ha az erőforrás egyedi. A **nézet** (view) nem birtokol, hanem más objektum által birtokolt adatra mutat; élettartama nem haladhatja meg a mögöttes adatét.

A portban:

- a bemenet `std::span<const unsigned char>` **nézet**, amelyet a parser csak a `feed` hívás alatt olvas, és nem tárol el;
- a `diag::report`, a `diag::error` és maga a `diag::parser` **értéktípus**: nem birtokol külső erőforrást, ezért szabadon másolható; egy másolat a bemenet ugyanazon pontjáról, a többitől függetlenül folytatható;
- a C API `dg_parser *` handle-je **tulajdonos**: a `dg_create` a hívó allokátorával hozza létre, a `dg_destroy` ugyanazzal szabadítja fel;
- a platformadapter fájl- és memóriaerőforrásai (`UniqueFd`, `MappedFile`) **move-only tulajdonosok**.

A C változatban a tulajdonlás és az élettartam a dokumentációban élt; a C++ változatban a típusok mondják ki. A megfigyelhető viselkedés eközben bájtra azonos marad.

39.2.2 Értéktípusok: report és error

A C++ core felülete (`include/diag_core.hpp`) a portterv kidolgozott példájában szerepel. Az értéktípusok a C struktúrák közvetlen megfelelői:

C ABI	C++ core	Megjegyzés
`dg_report`	`diag::report`	Azonos mezők: `bytes`, `records`, `sum`
`dg_error`	`diag::error`	`kind` és `offset`; a `kind` hatókörös `enum class`
`dg_code`	`diag::code`	Azonos numerikus értékek, `static_assert` ellenőrzi
`DG_MAX_INPUT`	`diag::max_input`	`inline constexpr`, egy definíció a programban

Az `enum class` nem konvertálódik implicit módon egésszé, ezért a C értékre fordítás a wrapperben explicit `static_cast`, és csak ott történik. Mivel minden alapértelmezett tagkezdőérték nulla, egy alapállapotú `diag::parser` pontosan a C `dg_create` utáni állapotnak felel meg: üres jelentés, `code::ok`, névfázis.

A `diag::parser` invariánsa ugyanaz, mint a C struktúráé: a `report_.bytes` legfeljebb `max_input`; a névhossz 0–16; a számjegyszám 0–5; a részérték legfeljebb 65535; és az első hiba után az `error_` nem változik. A privát adattagok miatt ezt csak a tagfüggvények módosíthatják.

39.2.3 RAII: file descriptor és memóriatérkép

A core nem lát erőforrást; a C++ oldali platformadapterben (például egy C++ CLI-ben vagy a TCP-adapter portjában) viszont a file descriptor és a memóriatérkép a két tipikus egyedi erőforrás. Mindkettő move-only owner: a másolás tiltott, a mozgatás `noexcept`, a destruktor felszabadít. A `UniqueFd` a `close`-ot, a `MappedFile` a `munmap`-ot hívja.

```
// include/diag_io.hpp (részlet)
#pragma once
#include <cerrno>
#include <cstddef>
#include <span>
#include <system_error>
```

```cpp
#include <utility>
#include <sys/mman.h>
#include <unistd.h>

class UniqueFd {
public:
    explicit UniqueFd(int fd = -1) noexcept : fd_(fd) {}
    ~UniqueFd() { if (fd_ >= 0) ::close(fd_); }
    UniqueFd(UniqueFd&& o) noexcept : fd_(std::exchange(o.fd_, -1)) {}
    UniqueFd& operator=(UniqueFd&& o) noexcept {
        if (this != &o) { reset(); fd_ = std::exchange(o.fd_, -1); }
        return *this;
    }
    UniqueFd(const UniqueFd&) = delete;
    UniqueFd& operator=(const UniqueFd&) = delete;

    int get() const noexcept { return fd_; }
    int release() noexcept { return std::exchange(fd_, -1); }
private:
    void reset() noexcept { if (fd_ >= 0) { ::close(fd_); fd_ = -1; } }
    int fd_;
};

class MappedFile {
public:
    MappedFile(int fd, std::size_t length, off_t offset = 0)
        : length_(length) {
        addr_ = ::mmap(nullptr, length, PROT_READ, MAP_PRIVATE, fd, offset);
        if (addr_ == MAP_FAILED) {
            addr_ = nullptr;
            throw std::system_error(errno, std::generic_category(), "mmap");
        }
    }
    ~MappedFile() { reset(); }
    MappedFile(MappedFile&& o) noexcept
        : addr_(std::exchange(o.addr_, nullptr)),
          length_(std::exchange(o.length_, 0)) {}
    MappedFile& operator=(MappedFile&& o) noexcept {
        if (this != &o) { reset(); addr_ = std::exchange(o.addr_, nullptr);
                          length_ = std::exchange(o.length_, 0); }
        return *this;
    }
    MappedFile(const MappedFile&) = delete;
    MappedFile& operator=(const MappedFile&) = delete;

    std::span<const std::byte> bytes() const noexcept {
        return {static_cast<const std::byte*>(addr_), length_};
    }
private:
    void reset() noexcept {
        if (addr_ != nullptr) { ::munmap(addr_, length_); addr_ = nullptr;
        ↪  length_ = 0; }
```

```
    }
    void* addr_ = nullptr;
    std::size_t length_ = 0;
};
```

A `bytes()` nézet a mapping élettartamáig érvényes. A `MappedFile` mozgatása nem invalidálja a nézetet, mert a címzett cím nem változik, és a mozgatott-from objektum már nem szabadít fel; a nézet akkor válik érvénytelenné, amikor az új tulajdonos megszűnik. Ezt a szerződést a header dokumentációjában rögzíteni kell. A `MappedFile` konstruktora `std::system_error`-t dobhat; ez a ritka, nem várt erőforrás-hiba, nem a parser várható hibája.

39.2.4 Az állapotgép portja és a hibapozíció

A megvalósítás a C `dg_feed` és `dg_finish` állapotgépének soronkénti megfelelője. A C++ eszközök csak ott jelennek meg, ahol a szerződést pontosabban fejezik ki: tartományalapú `for` a span fölött, `enum class` a fázisra, aggregátum-inicializálás a hibához.

```
// src/diag_core.cpp
#include "diag_core.hpp"

namespace diag {

bool parser::fail(code kind) noexcept
{
    if (error_.kind == code::ok)
        error_ = error{kind, report_.bytes};
    return false;
}

bool parser::feed(std::span<const unsigned char> input) noexcept
{
    if (error_.kind != code::ok)
        return false;
    if (finished_)
        return fail(code::state);

    for (const unsigned char byte : input) {
        if (report_.bytes == max_input)
            return fail(code::too_large);

        if (phase_ == phase::key) {
            if (byte >= 'a' && byte <= 'z') {
                if (key_length_ == 16)
                    return fail(code::syntax);
                ++key_length_;
            } else if (byte == '=' && key_length_ != 0) {
                phase_ = phase::number;
                digits_ = 0;
                value_ = 0;
            } else {
                return fail(code::syntax);
            }
```

```
        } else {
            if (byte >= '0' && byte <= '9') {
                const unsigned long digit = byte - '0';
                if (digits_ == 5 || value_ > (65535UL - digit) / 10UL)
                    return fail(code::syntax);
                value_ = value_ * 10UL + digit;
                ++digits_;
            } else if (byte == '\n' && digits_ != 0) {
                ++report_.records;
                report_.sum += value_;
                phase_ = phase::key;
                key_length_ = 0;
            } else {
                return fail(code::syntax);
            }
        }
        ++report_.bytes;
    }
    return true;
}

bool parser::finish() noexcept
{
    if (error_.kind != code::ok)
        return false;
    if (phase_ != phase::key || key_length_ != 0)
        return fail(code::truncated);
    finished_ = true;
    return true;
}

} // namespace diag
```

A bájtosztályokat explicit összehasonlítás dönti el, nem `std::islower` vagy `std::isdigit`: azok locale-függők lehetnek, a nyelvtan viszont rögzített ASCII-oktetteket ír elő. A túlcsordulásellenőrzés a szorzás **előtt** fut, ugyanazzal a $v \le \lfloor (65535 - d)/10 \rfloor$ feltétellel, mint a C változatban.

A hibapozíció a teljes munkamenet elejéhez viszonyított bájtoffset: a `report_.bytes` értéke a hiba pillanatában. Mivel a számláló csak sikeresen feldolgozott bájt után nő, ez éppen az első elutasított bájt helye; csonkolásnál a bemenet vége; limitnél pontosan `max_input`. A darabolás nem számít: több `feed` hívás offsetjei összeadódnak, nem szeletenként indulnak nulláról.

39.2.4.1 Helyességi vázlat

A port minden ága a C referencia egy ágának felel meg, azonos feltétellel és azonos állapotváltozással; a vezérlés bájtonként halad, és a hívások határán nincs implicit lezárás. Indukcióval a feldolgozott bájtok számára: ha a két implementáció állapota k bájt után megegyezik (fázis, névhossz, számjegyszám, részérték, jelentés, hiba), akkor a $(k+1)$-edik bájt után is, mert ugyanaz a döntés születik. A `finish` ugyanazt a feltételt vizsgálja. Ezért a port és a C referencia minden bemenetre és minden darabolásra azonos jelentést és hibát ad. A vázlat feltételezi, hogy a span bájtjai a hívás idején érvényesek.

A vázlat a **C referenciához** viszonyított egyezést igazolja. A **specifikációhoz** viszonyított helyességet a projektszerződés nyelvtanából levezetett `tests/spec_examples.c` és a 17.2 bizonyításvázlatai támasztják alá; a kettő együtt adja a port elfogadását.

39.2.5 Tároló- és allokátorválasztás

A C referencia egyetlen, rögzített méretű parserobjektummal dolgozik, és nem tárol rekordokat. A port ezt megtartja: a core-ban nincs `std::vector`, `std::string` vagy más dinamikus tároló, és nincs `new`. Ha egy rekordokat gyűjtő tárolót vezetnénk be, az megfigyelhető változás lenne a memóriakorlátban, és csak külön migrációként jöhetne szóba.

Az allokátorszerződést a wrapper őrzi meg. A `dg_create` a hívó `dg_allocator` callbackjével foglal tárolót, abba placement new-val hozza létre a `dg_parser` objektumot, a `dg_destroy` pedig meghívja a destruktort, majd ugyanazzal az allokátorral szabadít fel. A C tesztkészlet allokációs hibát injektáló esete ezért a C++ könyvtárral is ugyanúgy fut.

Egy `std::pmr::memory_resource` adapter, amely a `dg_allocator` callbackjeit hívja, kézenfekvő kiterjesztés lenne, ha a core-ba később valódi dinamikus tároló kerülne. A jelenlegi, $O(1)$ memóriájú parsernél nincs mit az arena alá helyezni.

39.2.6 Hibamodell: explicit eredmény, kivétel nélkül

A port alapprofilja: a **várható** hibák — szintaktikai hiba, csonkolás, limit, API-használati hiba — értékként utaznak: a `feed` és a `finish` `bool` eredményt ad, a részleteket a `last_error()` adja. Ez a C API egy-az-egyben leképezhető alakja, ezért nincs szükség `std::variant`-ra vagy C++23 `std::expected`-re.

A core minden művelete `noexcept`, és nem allokál; így kivétel nem keletkezhet benne. Az egyetlen erőforrás-hiba a létrehozás: ezt a C API `NULL` visszatérése jelzi, ahogyan a C változatban is. A platformadapter `UniqueFd` és `MappedFile` típusai `std::system_error`-t dobhatnak; ezek a C ABI határán kívül élnek, és a határon belépő függvénynek el kell kapnia őket, mielőtt hibakóddá alakítja.

39.2.7 A C interfész és a tesztek

A C interfész a változatlan `include/diag.h`; a C++ oldali definíciókat a portterv `src/diag_c_api.cpp` wrappere adja. Új C header nem kell, és nem is szabad: a C kliensek, köztük a `src/main.c` és a C tesztek, forrásszinten változatlanok maradnak.

A C++ core saját tesztje közvetlenül a `diag::parser` osztályt hívja, C ABI nélkül. Az ellenőrzések `NDEBUG` mellett sem tűnnek el, mert nem `assert`-re épülnek:

```
// tests/test_core.cpp -- a C++ core közvetlen tesztje
#include "diag_core.hpp"

#include <cstdio>
#include <cstdlib>
#include <span>
#include <string_view>

#define CHECK(condition) do {                                   \
    if (!(condition)) {                                         \
```

```
        std::fprintf(stderr, "CHECK failed at line %d\n", __LINE__); \
        std::exit(EXIT_FAILURE);                                     \
    }                                                                \
} while (0)

static std::span<const unsigned char> bytes(std::string_view s)
{
    return {reinterpret_cast<const unsigned char *>(s.data()), s.size()};
}

int main()
{
    const auto input = bytes("a=7\ncpu=12\n");

    for (std::size_t split = 0; split <= input.size(); ++split) {
        diag::parser p;
        CHECK(p.feed(input.first(split)));
        CHECK(p.feed(input.subspan(split)));
        CHECK(p.finish());
        const diag::report r = p.current();
        CHECK(r.records == 2 && r.sum == 19 && r.bytes == 11);
    }

    diag::parser truncated;
    CHECK(truncated.feed(bytes("a=7\ncpu=12")));
    CHECK(!truncated.finish());
    CHECK(truncated.last_error().kind == diag::code::truncated);
    CHECK(truncated.last_error().offset == 10);

    diag::parser bad;
    CHECK(!bad.feed(bytes("a=65536\n")));
    CHECK(bad.last_error().kind == diag::code::syntax);
    CHECK(bad.last_error().offset == 6);

    unsigned char reused[] = {'a', '=', '7', '\n'};
    diag::parser r;
    CHECK(r.feed(std::span<const unsigned char>(reused, 2)));
    reused[0] = reused[1] = 'X';      // a hívó újrahasználja a puffert
    CHECK(r.feed(std::span<const unsigned char>(reused + 2, 2)));
    CHECK(r.finish() && r.current().sum == 7 && r.current().bytes == 4);
    return 0;
}
```

```
mkdir -p build/cxx
g++ -std=c++20 -O2 -g -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
    -Iinclude src/diag_core.cpp tests/test_core.cpp \
    -o build/cxx/test_core
build/cxx/test_core
```

A C ABI-n keresztüli ellenőrzést a C referencia saját tesztjei végzik, a C++ könyvtárhoz linkelve; a buildparancsokat és a differenciális tesztet a portterv adja.

Buildjegyzőkönyv (`lab-gcc` profil). A `diag_core.cpp`, a `diag_c_api.cpp` és a `test_core.cpp`

G++ 13.3.0-val, `-std=c++20 -Wall -Wextra -Wpedantic -Wconversion -Wshadow -Werror` beállítással figyelmeztetés nélkül fordult Linux x86-64 környezetben; a `test_core` sikeresen lefutott. Az RAII-szakasz `UniqueFd` és `MappedFile` részlete nem része ennek a buildnek.

39.2.8 Tipikus hibák

A leggyakoribb hiba a span megőrzése: a parser a `feed` után nem tárolhat a bemenetre mutató nézetet, mert a hívó a puffert újrahasználja; a félbemaradt rekordhoz elég a fázis, a névhossz, a számjegyszám és a részérték. A második: a bájtosztályok cseréje `std::isalpha`/`std::isdigit` hívásra, ami locale-függővé teszi a nyelvtant. A harmadik: a túlcsordulásellenőrzés áthelyezése a szorzás utánra, különösen szűkebb tárolótípussal; ilyenkor az érték csendben körbefordul. A negyedik: a `diag::code` értékeinek átrendezése; a `static_assert` ezt fordításkor megfogja, ezért nem szabad eltávolítani. Az ötödik: a `MappedFile` `bytes()` nézetének használata a `MappedFile` megszűnése után; a nézet nem tartja életben a mappinget.

39.2.9 Összefoglaló

A C++ core a 17.2 C referencia állapotgépének hű portja. A bemenet nem tulajdonló span, a jelentés, a hiba és maga a parser értéktípus, a C handle és a platformerőforrások tulajdonosok. A core nem allokál és nem dob, ezért a C ABI wrapper kivétel- és allokátorszerződése egyszerű. A C header és a C kliensek változatlanok; a port helyességét a C referencia saját tesztjei, a specifikációs példák és a differenciális korpusz együtt igazolják.

39.2.10 Feladatok

1. **Magyarázó feladat.** Nevezd meg a portban a nézetet, az értéktípusokat és a tulajdonosokat! Miért lehet a `diag::parser` másolható, miközben a `UniqueFd` nem? Mit jelent egy rekord közepén készített másolat?

2. **Kódolási feladat.** Írj C++ tesztet, amely a `"a=7\ncpu=12\n"` bemenetet bájtonként táplálja be, a hatodik bájt előtt másolatot készít a parserről, majd a másolatot a maradék bemenettel külön lezárja. Ellenőrizd, hogy a két parser jelentése azonos.

3. **Diagnosztikai feladat.** Egy kolléga a részértéket `std::uint16_t` típusban tárolja, és az ellenőrzést a szorzás utánra helyezi:

```
value_ = static_cast<std::uint16_t>(value_ * 10u + digit);
if (value_ > 65535u)
    return fail(code::syntax);
```

Mit ír ki a C++ CLI a `"a=65536\n"` bemenetre, és mit a C referencia? Melyik ellenőrzés fogja meg a hibát?

4. **Kódolási és diagnosztikai feladat.** A `MappedFile` `bytes()` nézetét eltárolod egy `std::span`-ban, majd a `MappedFile`-ot mozgatod egy másik objektumba. Érvényes marad-e a nézet? Fogalmazd meg a szerződést, és egészítsd ki a `MappedFile` dokumentációját egy bekezdéssel.

39.2.11 Megoldások

1. A `std::span<const unsigned char>` a nézet: nem birtokol, és csak a `feed` hívás idejéig érvényes. A `diag::report`, a `diag::error` és a `diag::parser` értéktípus: csak skalárokat tartalmaz, külső erőforrást nem birtokol, ezért a másolás teljes, független állapotot hoz létre. A `dg_parser *` handle és a `UniqueFd` tulajdonos; a `UniqueFd` másolása két tulajdonost teremtene ugyanarra a file descriptorra, ami dupla `close`-hoz vezetne, ezért tiltott. A rekord közepén készített másolat a félbemaradt rekord teljes állapotát (fázis, névhossz, számjegyszám, részérték) hordozza, így bármelyik példány ugyanonnan folytatható. Ellenőrzési feltétel: a magyarázat a három szerepet és a másolhatóság okát is megnevezi.

2.

```
// tests/test_bytewise.cpp -- bájtonkénti betáplálás és értékszemantika
#include "diag_core.hpp"

#include <cstdio>
#include <cstdlib>
#include <span>
#include <string_view>

#define CHECK(condition) do {                                          \
    if (!(condition)) {                                                \
        std::fprintf(stderr, "CHECK failed at line %d\n", __LINE__); \
        std::exit(EXIT_FAILURE);                                       \
    }                                                                  \
} while (0)

int main()
{
    constexpr std::string_view text = "a=7\ncpu=12\n";
    const std::span<const unsigned char> input{
        reinterpret_cast<const unsigned char *>(text.data()), text.size()};

    diag::parser p;
    diag::parser snapshot;
    for (std::size_t i = 0; i < input.size(); ++i) {
        if (i == 6)
            snapshot = p;               // másolat a rekord közepén
        CHECK(p.feed(input.subspan(i, 1)));
    }
    CHECK(p.finish());
    CHECK(p.current().records == 2 && p.current().sum == 19);

    // A másolat független: ugyanonnan folytatva ugyanoda jut.
    CHECK(snapshot.current().bytes == 6);
    CHECK(snapshot.feed(input.subspan(6)));
    CHECK(snapshot.finish());
    CHECK(snapshot.current().records == p.current().records);
    CHECK(snapshot.current().sum == p.current().sum);
    CHECK(snapshot.current().bytes == p.current().bytes);
    return 0;
}
```

A hatodik bájt előtt a parser 6 bájtot fogyasztott el (`a=7\ncp`), és a második rekord névfázisában áll. A teszt G++ 13.3.0-val, `-Werror` mellett fordult és sikeresen lefutott. Ellenőrzési feltétel: a két jelentés mindhárom mezője megegyezik.

3. A `std::uint16_t` legfeljebb 65535-öt tárol, ezért a `6553` után a `6` számjegynél a 65536 érték 0-ra fordul körbe, és a `value_ > 65535u` feltétel soha nem teljesül. A C++ CLI kimenete `OK records=1 sum=0 bytes=8`, kilépési kódja 0; a C referenciáé `ERR code=syntax offset=6`, kilépési kódja 4. A hibát a differenciális teszt a korpusz `a=65536` elemén megfogja (stdout, stderr és kilépési kód is eltér), a specifikációs példák `"a=65536\n"` sora szintén, és a GCC `-Wextra` alatt `-Wtype-limits` figyelmeztetést ad a mindig hamis összehasonlításra. A javítás a C referencia sorrendje: a tartományellenőrzés a szorzás előtt, elég széles típusban. Ellenőrzési feltétel: a hibás változat a fenti kimenetet adja, a javított a C referenciáéval azonosat. A hibás változatot GCC 13.3.0-val ténylegesen lefordítottuk és lefuttattuk; a fenti kimenetek ebből származnak.

4. A nézet érvényes marad, feltéve, hogy a mozgatott-from `MappedFile` nem szabadítja fel a mappinget (a move után a `addr_` nullptr), és az új tulajdonos életben marad. A `std::span` csak a címet és a hosszt tárolja, nem az owner identitását, ezért a mozgatás nem változtatja meg a nézet által látott memóriát. A szerződés: a `bytes()` nézet a mapping élettartamáig érvényes; a mapping akkor szűnik meg, amikor az őt birtokló `MappedFile` (a mozgatás után az új példány) megszűnik, vagy explicit módon felszabadítják. A dokumentáció kiegészítése: „A `bytes()` által visszaadott span a mapping élettartamához kötött; a `MappedFile` mozgatása nem invalidálja, a tulajdonos megszűnése igen.” Ellenőrzési feltétel: a válasz különbséget tesz a owner objektum identitása és a mapping élettartama között.

39.3 Task, queue és szolgáltatás-életciklus

Ez az alfejezet a C záróprojekt funkcionális szerződését megtartó C++ port konkurens rétegével foglalkozik. A C++ core: parser, erőforrások és értéktípusok (39.2) részben rögzítettük, hogyan lesz a C `struct` + kézi `free` mintákból értékszemantikájú, RAII-val kezelt C++ mag; ott a hangsúly a szinkron adatúton volt. Most azt a kérdést vesszük elő, amely a legtöbb termelési hibát okozza: mi történik, ha a szolgáltatásnak egyszerre több szála dolgozik, a bemenet korlátos pufferen érkezik, és a leállítást bármelyik pillanatban kérhetik. A cél nem egy új keretrendszer, hanem egy kicsi, indokolható szerződés: ki tulajdonol mit, meddig él, mi történik telítődéskor, és milyen sorrendben lehet hézagmentesen leállni.

A fejezet épít a shared ownership, reclamation és konkurens review (34.6) megosztott tulajdonlási és visszanyerési szabályaira, valamint az Aszinkron I/O adapter és coroutine-labor (35.5) felfüggesztési és élettartam-megfontolásaira. A coroutine adaptert itt csak érintjük, mert az a részletes kidolgozást ott kapta meg; itt az a kérdés, hogyan illeszkedik a szolgáltatás-életciklusba egy kísérleti, külön capability profile alatt.

39.3.1 A szolgáltatás-életciklus mint szerződés

Szolgáltatás-életciklusnak nevezzük azon állapotok halmazát, átmeneteik megengedett sorrendjét és az egyes állapotokhoz tartozó erőforrás-kötelezettségeket, amelyek egy hosszú életű, konkurens komponens viselkedését meghatározzák. Ez nem puszta dokumentáció: a C++ nyelvi eszközeivel részben kikényszeríthető, részben pedig tesztelhető. A szerződés három dolgot rögzít:

1. **Mely műveletek engedélyezettek mely állapotban.** Ha a `submit` a leállítás megkezdése után is elfogad munkát, az vagy elveszik, vagy a leállás soha nem fejeződik be.

2. **Ki az owner (tulajdonos) az egyes erőforrások felett.** A queue, a worker szálak, a connection objektumok és a callbackek tulajdonosa a `service` objektum; minden más ezekhez képest *borrowed* (kölcsönzött) hozzáférés.
3. **Mikor tekinthető egy erőforrás felszabadíthatónak.** Csak akkor, ha az ahhoz tartozó összes *outstanding operation* (befejezetlen művelet) befejeződött vagy megszakított, és ezt a tényt valamilyen szinkronizációs eszköz igazolja.

Outstanding operation: olyan művelet, amelyhez erőforrás (buffer, socket, callback, szál) tartozik, és amelynek végrehajtása megkezdődött, de a befejezés ténye még nem került publikálásra. A definíció szándékosan a publikáláson van: attól, hogy egy worker szál lokálisan „már majdnem végzett", a művelet a többi szál számára még folyamatban van, amíg egy happens-before élt nem létesítünk (jellemzően join, mutex, atomikus művelet vagy condition variable segítségével).

Quiescence (nyugvás): az az állapot, amelyben nincs outstanding operation, a queue kiürült vagy lezárult, és egyetlen szál sem hajthat végre olyan kódot, amely a szolgáltatás erőforrásaihoz nyúl. A teardown akkor helyes, ha a nyugvást megállapítja, mielőtt a tagok destruktorai lefutnak.

A mi állapotgépe:

Állapot	Belépés feltétele	Engedélyezett	Tiltott
`Constructed`	konstruktor fut	tagok inicializálása	`submit`, `shutdown` munkát indítani
`Running`	workerek elindultak	`submit`, `shutdown` kezdeményezése	queue `close` nélküli eldobása
`Draining`	`close()` megtörtént	a már bekerült elemek feldolgozása, `pop`	új `push` elfogadása
`Stopped`	minden worker joinolt	destruktor, diagnosztika olvasása	bármilyen erőforrás-használat

Egy fontos tervezési döntés, hogy a `Draining` állapotot nem külön enumerációval, hanem a queue `closed_` flagjével és a stop token állapotával reprezentáljuk. Ez kevesebb duplikált állapotot jelent, viszont a két jelzés szemantikája eltér: a `close()` a *bemenet lezárása*, a stop kérés a *szálak kooperatív megszakítása*. A kettő nem ugyanaz, és a leállási protokoll éppen abból áll, hogy a kettőt helyes sorrendben és egymásra hatva használjuk.

39.3.2 Bounded queue, stop token és backpressure

A **bounded queue** (korlátos sor) olyan sor, amelyre invariánsként teljesül, hogy elemszáma minden megfigyelhető pillanatban $0 \le n \le C$, ahol C a kapacitás, és hogy `close()` után nem kerülhet be új elem. A korlát nem kényelmi kérdés: ez az egyetlen hely, ahol a rendszer memóriahasználata felülről becsülhető.

Backpressure (visszanyomás): az a mechanizmus, amellyel a fogyasztó telítettsége a termelő felé továbbítódik, és ott várakozást, hibát vagy eldobást vált ki. Három alapvető politikát érdemes ismerni, és ezeket nem szabad összekeverni:

- **Blokkoló**: a termelő addig vár, amíg hely nem lesz. Erős memóriakorlát, de a termelő szálát lefoglalja, ezért csak akkor elfogadható, ha a termelő nem tart más zárat.
- **Eldobó**: a termelő azonnal visszajelez (pl. `false`), és a hívó dönt a sorsról. Akkor jó, ha a forrás lassítható, de nem akarunk blokkolni.

- **Lezáró**: a telítődés hibának számít, és a kapcsolat bezárul. Ezt csak akkor szabad alkalmazni, ha a protokoll ezt kifejezetten megengedi, mert a csendben eldobott adat a hálózati framinget is tönkreteheti.

A stabilitás levezetése rövid. Legyen T egy frame várható feldolgozási ideje, W a workerek száma. A feldolgozó kapacitás közelítőleg W/T frame/másodperc. Ha a hosszú távú érkezési ráta $\lambda < W/T$, a sor várható hossza korlátos marad; ha $\lambda \geq W/T$, akkor a sor hossza a korlátig nő, és a backpressure folyamatosan aktív lesz. A korlátos sor ezt nem szünteti meg, hanem *láthatóvá és kezelhetővé* teszi: a telítődés nem csendes memóriafelfúvódásként, hanem visszajelzésként vagy hibaként jelenik meg. A szolgáltatás saját tárolójának felső korlátja akkor adható meg, ha minden olyan frame-et elszámolunk, amelyet a szolgáltatás birtokol. Ezek: legfeljebb C frame a sorban, legfeljebb W a workereknél feldolgozás alatt, és legfeljebb P olyan `submit` hívás paraméterében, amely a teli sor miatt blokkol. Ha egy frame legfeljebb $S_{\max}$ bájt hasznos adatot hordoz, akkor

$$M \leq (C + W + P) \cdot S_{\max} + K,$$

ahol K a fix overhead (szálak, sorcsomópontok, mutexek, könyvtári pufferek). A becslés két kódbeli feltételen múlik. A `submit` az $S_{\max}$-nál nagyobb frame-et — a hasznos hossz *és* a lefoglalt `capacity()` szerint is — még a sor előtt, `too_large` eredménnyel elutasítja (a tartott memória a kapacitás, ezért egy nagy `reserve()` után kicsire állított string sem kerülhet be); a képlet így a szolgáltatás saját tárolójára vonatkozó felső korlát. A blokkoló hívók száma legfeljebb P: aki a $P+1$-edik lenne, azonnal `busy` eredményt kap, és a frame-et a hívás végén felszabadítja. A hívó saját, még be nem adott frame-jei nem a szolgáltatás tárolói; azokat a hívó kerete számolja el. Ezek a feltételek nélkül a korlát nem érvényes: korlátlan számú blokkoló producer mindegyike egy további frame-et tartana életben a soron kívül.

A C++20 **stop token** mechanizmusa egy közösen fenntartott, szálbiztos **stop-állapotra** épül, amelyben három szereplő osztozik:

- a `std::stop_source` leállítást kérhet (`request_stop()`), és részt vesz a stop-állapot élettartamában;
- a `std::stop_token` csak lekérdezhet (`stop_requested()`, `stop_possible()`), és rá callback regisztrálható; ő is részt vesz a stop-állapot élettartamában, ezért nem tekinthető puszta, nem tulajdonló nézetnek;
- a `std::stop_callback` regisztrált reakció: ha a leállítást már kérték, a konstruktorában azonnal lefut; egyébként a kérést végző szálon fut. Destruktora deregisztrál, és ha a callback éppen fut egy másik szálon, megvárja.

A stop-állapot addig él, amíg bármely source vagy token hivatkozik rá. Egy source élettartamán túl megőrzött token ezért nem dangling: a vizsgálata definiált. A `stop_requested()` megőrzi a kérés tényét a source megszűnése után is. A `stop_possible()` más kérdést válaszol meg: igaz, ha a leállítást már kérték, vagy még létezik olyan source, amely kérheti; hamis egy alapértelmezett konstrukciójú, stop-állapot nélküli tokennél, illetve ha az utolsó source kérés nélkül megszűnt. A tárolás élettartama és a kérhetőség tehát két külön tulajdonság:

```
// tests/stop_state_probe.cpp -- a token a megosztott stop-állapot része
#include <cstdio>
#include <cstdlib>
#include <stop_token>

#define CHECK(condition) do {                                          \
    if (!(condition)) {                                                \
        std::fprintf(stderr, "CHECK failed at line %d\n", __LINE__); \
```

```
        std::exit(EXIT_FAILURE);                                    \
    }                                                               \
} while (0)

int main()
{
    std::stop_token kept;
    CHECK(!kept.stop_possible());                 // nincs stop-állapot

    {
        std::stop_source source;
        kept = source.get_token();
        CHECK(kept.stop_possible() && !kept.stop_requested());
    }   // az utolsó source kérés nélkül megszűnt
    CHECK(!kept.stop_possible() && !kept.stop_requested());

    {
        std::stop_source source;
        kept = source.get_token();
        source.request_stop();
    }   // a kérés megmarad a source után is
    CHECK(kept.stop_requested() && kept.stop_possible());

    bool ran = false;
    std::stop_callback late(kept, [&] { ran = true; });
    CHECK(ran);                                   // már kért stop: azonnal fut
    return 0;
}
```

A próba G++ 13.3.0 és libstdc++ mellett, `-std=c++20 -Werror` beállítással fordult, és ASan/UBSan builddel is sikeresen lefutott.

A `std::jthread` destruktora `request_stop()`-ot hív, majd joinol, tehát a „elfelejtett join” hiba alapértelmezésben megszűnik. A stop token nem megszakítja a blokkoló hívást, hanem a `condition_variable_any` stop tokenes `wait` overloadjai teszik megszakíthatóvá a várakozást. Ezek az overloadok C++20 újak; a konkrét libstdc++/libc++ kiadás támogatását a laborprofilban ellenőrizni kell, és nem szabad feltételezni, hogy egy régebbi eszközlánc elfogadja a kódot.

A következő példa a teljes, önállóan fordítható queue. A szerződés: `push` blokkol, ha a sor tele van, de stop kérésre `false`-szal visszatér; `pop` a stop kérés után is kiveszi a már bekerült elemeket (drain), és csak akkor ad `nullopt`-ot, ha a sor kiürült. Ez a „drain on stop” politika.

```
// bounded_queue.hpp
#pragma once
#include <chrono>
#include <condition_variable>
#include <cstddef>
#include <deque>
#include <mutex>
#include <optional>
#include <stop_token>
#include <utility>
```

```cpp
template <class T>
class bounded_queue {
public:
    explicit bounded_queue(std::size_t capacity) : cap_(capacity) {}
    bounded_queue(const bounded_queue&) = delete;
    bounded_queue& operator=(const bounded_queue&) = delete;

    // Nem blokkol; false: tele van vagy lezárt.
    bool try_push(T value) {
        {
            std::lock_guard lk(m_);
            if (closed_ || q_.size() >= cap_) return false;
            q_.push_back(std::move(value));
        }
        not_empty_.notify_one();
        return true;
    }

    // Blokkol, amíg hely van; stop kérés vagy close esetén false.
    bool push(T value, std::stop_token st) {
        std::unique_lock lk(m_);
        not_full_.wait(lk, st, [&] { return closed_ || q_.size() < cap_; });
        if (closed_ || st.stop_requested()) return false;
        q_.push_back(std::move(value));
        lk.unlock();
        not_empty_.notify_one();
        return true;
    }

    // Stop után is kiveszi a maradékot; csak üres+leállt esetén nullopt.
    std::optional<T> pop(std::stop_token st) {
        std::unique_lock lk(m_);
        not_empty_.wait(lk, st, [&] { return closed_ || !q_.empty(); });
        if (q_.empty()) return std::nullopt;
        T v = std::move(q_.front());
        q_.pop_front();
        const bool now_empty = q_.empty();
        lk.unlock();
        not_full_.notify_one();
        if (now_empty) empty_.notify_all();
        return v;
    }

    // A sor kiürüléséig vagy a határidőig vár; true, ha kiürült.
    bool wait_empty_until(std::chrono::steady_clock::time_point deadline) {
        std::unique_lock lk(m_);
        return empty_.wait_until(lk, deadline, [&] { return q_.empty(); });
    }

    // A sorban maradt elemek eldobása; az eldobott darabszámot adja.
    std::size_t discard() {
        std::size_t n = 0;
```

```
        {
            std::lock_guard lk(m_);
            n = q_.size();
            q_.clear();
        }
        empty_.notify_all();
        not_full_.notify_all();
        return n;
    }

    void close() {
        {
            std::lock_guard lk(m_);
            closed_ = true;
        }
        not_empty_.notify_all();
        not_full_.notify_all();
    }

    std::size_t size() const {
        std::lock_guard lk(m_);
        return q_.size();
    }

private:
    std::size_t cap_;
    mutable std::mutex m_;
    std::condition_variable_any not_empty_, not_full_, empty_;
    std::deque<T> q_;
    bool closed_ = false;
};
```

A queue működésének kritikus pontja a `pop` feltétel nélküli `if (q_.empty())` ellenőrzése: a `wait` stop token miatt is visszatérhet, amikor a predikátum hamis, ezért a visszatérés után a tényleges állapotot újra meg kell vizsgálni. Ha ezt elhagynánk, stop kéréskor üres sorból is kivennénk egy „elemet”, ami nem definiált viselkedés. Hasonlóan, a `push`-ban a `stop_requested()` ellenőrzés azért kell, mert a `wait` akkor is visszatérhet, ha éppen van hely — ilyenkor viszont leállás alatt nem akarunk új munkát befogadni. A `wait_empty_until` és a `discard` a határidős leállítás eszköze: az első a sor kiürülésére vár legfeljebb egy adott időpontig, a második a sorban maradt elemeket egy zár alatt eldobja, és megmondja, hányat dobott el.

A queue csak a tárolást és a szinkronizációt adja; a szolgáltatás életciklusa a következő headerben van. A konfiguráció minden korlátja kötelezően pozitív: nulla worker mellett a sor sosem ürülne, nulla kapacitásnál a blokkoló `submit` sosem jutna helyhez. A konstruktor ezért a szálak indítása előtt ellenőriz, és hibás konfigurációnál `std::invalid_argument` kivételt dob; a hiba így indításkor derül ki, nem egy beragadt leállításkor.

```
// service.hpp
#pragma once
#include "bounded_queue.hpp"
#include <atomic>
#include <chrono>
#include <cstddef>
```

```cpp
#include <functional>
#include <mutex>
#include <stdexcept>
#include <string>
#include <thread>
#include <utility>
#include <vector>

struct frame {
    std::string payload;
};

struct service_config {
    std::size_t workers = 2;          // W >= 1
    std::size_t capacity = 4;         // C >= 1
    std::size_t max_frame = 4096;     // S_max >= 1, bájt
    std::size_t max_waiting = 8;      // P >= 1: egyszerre blokkoló submitok
    std::chrono::milliseconds drain_timeout{1000};
};

enum class submit_result { accepted, too_large, busy, closed, cancelled };   //
 ↪  cancelled: a hívó stop kérése miatt

struct service_stats {
    std::size_t accepted;             // sorba került frame-ek
    std::size_t processed;            // végigfutott feldolgozás
    std::size_t cancelled;            // stop miatt félbehagyott feldolgozás
    std::size_t dropped;              // a drain-határidő után eldobva
    std::size_t failed;               // a processor kivételt dobott (a kivételt
     ↪  a szolgáltatás elnyeli)
};

// true: kész; false: a stop kérés miatt félbehagyta. A stop kérés után
// korlátos időn belül vissza kell térnie, és nem végezhet blokkoló I/O-t.
// Kivételpolitika: a processor dobhat; a szolgáltatás elkapja, a frame-et
 ↪  `failed`-ként
// számolja el, és a worker tovább fut (a kivétel nem szökik ki a worker
 ↪  szálból).
using processor = std::function<bool(const frame&, std::stop_token)>;

class service {
public:
    service(service_config cfg, processor proc)
        : cfg_(validate(cfg)), proc_(std::move(proc)), q_(cfg_.capacity) {
        if (!proc_) throw std::invalid_argument("service: empty processor");
        workers_.reserve(cfg_.workers);
        for (std::size_t i = 0; i < cfg_.workers; ++i) {
            workers_.emplace_back([this](std::stop_token st) { run(st); });
        }
    }

    ~service() { shutdown(); }
```

```
service(const service&) = delete;
service& operator=(const service&) = delete;

// Backpressure: legfeljebb P hívó blokkolhat; a többi azonnal busy.
submit_result submit(frame f, std::stop_token st = {}) {
    // A tartott memória a lefoglalt kapacitás, nem a hasznos hossz: a nagy
    // reserve() után kicsire állított string is nagy allokációt birtokolna.
    if (f.payload.size() > cfg_.max_frame || f.payload.capacity() >
     ↪  cfg_.max_frame)
        return submit_result::too_large;      // a sor előtt elutasítva
    if (waiting_.fetch_add(1) >= cfg_.max_waiting) {
        waiting_.fetch_sub(1);
        return submit_result::busy;
    }
    // Scope guard: a waiting-foglalás minden úton (a push() kivételénél is)
     ↪  felszabadul.
    struct waiting_guard {
        std::atomic<std::size_t>& c;
        ~waiting_guard() { c.fetch_sub(1); }
    } guard{waiting_};
    // Kivételpolitika: ha a push() dob (pl. allokáció), a számlálók
     ↪  változatlanok,
    // a kivétel a hívóhoz jut; az accepted csak valódi befogadás után nő.
    const bool ok = q_.push(std::move(f), st);
    if (ok) accepted_.fetch_add(1);
    if (ok) return submit_result::accepted;
    // A sikertelen push oka: a hívó stop kérése (cancelled) vagy a lezárt
     ↪  szolgáltatás (closed).
    return st.stop_requested() ? submit_result::cancelled :
     ↪  submit_result::closed;
}

// Határidős leállítás; többször is hívható. Felső korlátja
// drain_timeout + egy feldolgozás stop utáni visszatérési ideje.
service_stats shutdown() {
    std::call_once(once_, [this] {
        q_.close();                                   // 1. új munka
         ↪  tiltása
        const auto deadline =
            std::chrono::steady_clock::now() + cfg_.drain_timeout;
        if (!q_.wait_empty_until(deadline))           // 2. drain
         ↪  határidőig
            dropped_ = q_.discard();                  // 3. maradék
             ↪  eldobása
        for (auto& w : workers_) w.request_stop();    // 4. kooperatív stop
        workers_.clear();                             // 5. join
    });
    return {accepted_.load(), processed_.load(), cancelled_.load(),
            dropped_, failed_.load()};
}
```

```
    std::size_t pending() const { return q_.size(); }

private:
    static service_config validate(service_config cfg) {
        if (cfg.workers == 0 || cfg.capacity == 0 || cfg.max_frame == 0 ||
            cfg.max_waiting == 0 || cfg.drain_timeout.count() < 0)
            throw std::invalid_argument("service: invalid configuration");
        return cfg;
    }

    void run(std::stop_token st) {
        while (auto f = q_.pop(st)) {
            try {
                if (proc_(*f, st))
                    processed_.fetch_add(1);
                else
                    cancelled_.fetch_add(1);
            } catch (...) {                          // dokumentált hibaút:
              ↪ elszámolás, a worker él tovább
                failed_.fetch_add(1);
            }
        }
    }

    const service_config cfg_;
    const processor proc_;
    bounded_queue<frame> q_;
    std::atomic<std::size_t> waiting_{0};
    std::atomic<std::size_t> accepted_{0};
    std::atomic<std::size_t> processed_{0};
    std::atomic<std::size_t> cancelled_{0};
    std::atomic<std::size_t> failed_{0};
    std::size_t dropped_ = 0;          // csak a call_once írja
    std::once_flag once_;
    std::vector<std::jthread> workers_; // utolsó tag: elsőként szűnik meg
};
```

A feldolgozó (`processor`) szerződése a leállítási korlát része: a stop kérés után korlátos időn belül vissza kell térnie, és blokkoló I/O-t nem végezhet. Egy megtelt, nem olvasott pipe-ra író `printf` például korlátlan ideig blokkolhat, és ezt a stop token nem szakítja meg; ezért a példa a feldolgozás eredményét memóriában gyűjti, és csak a leállítás után, a szolgáltatáson kívül ír ki. A feldolgozó `false` visszatérése azt jelzi, hogy a munkát a stop kérés miatt félbehagyta; ezt a `cancelled` számláló rögzíti. A `submit` eredményei: `accepted`, `too_large`, `busy`, `closed` (a szolgáltatás leállt) és `cancelled` (a hívó saját stop-kérése szakította meg a várakozást); a két utóbbi nem keverendő össze, mert más hívói teendő tartozik hozzájuk (újrapróbálás szándékosan nem, illetve lemondás elszámolása). A feldolgozó **dobhat** kivételt: a szolgáltatás elkapja, a frame-et a `failed` számlálóba számolja, a worker tovább fut (a kivétel nem szökik ki a szálból, így nincs `std::terminate`); a `submit` kivételpolitikája pedig, hogy a `push()` dobása esetén a számlálók változatlanok maradnak (a `waiting` foglalást scope guard adja vissza).

```
// main.cpp
#include "service.hpp"
```

```
#include <atomic>
#include <cstdio>
#include <string>

int main() {
    std::atomic<std::size_t> bytes{0};
    service svc{service_config{}, [&bytes](const frame& f, std::stop_token) {
        bytes.fetch_add(f.payload.size());   // nem blokkol: korlátos idő
        return true;
    }};
    for (int i = 0; i < 8; ++i) {
        (void)svc.submit(frame{"job-" + std::to_string(i)});
    }
    const service_stats s = svc.shutdown();
    // A kiírás a leállítás után, a szolgáltatáson kívül történik.
    std::printf("accepted=%zu processed=%zu cancelled=%zu dropped=%zu "
                "failed=%zu bytes=%zu\n", s.accepted, s.processed, s.cancelled,
                s.dropped, s.failed, bytes.load());
    return s.accepted == s.processed + s.cancelled + s.dropped + s.failed ? 0 :
     ↪  1;
}
```

A `shutdown()` eredménye a munka teljes elszámolása: a leállítás után, és miután minden `submit` hívás visszatért,

$$\text{accepted} = \text{processed} + \text{cancelled} + \text{dropped} + \text{failed}.$$

Fordítás és futtatás:

```
g++ -std=c++20 -O2 -Wall -Wextra -Wpedantic -Wconversion -Wshadow \
    -Werror -pthread -o svc main.cpp
./svc
g++ -std=c++20 -O1 -g -pthread -fsanitize=thread -o svc-tsan main.cpp
setarch -R ./svc-tsan
```

A kimenet: `accepted=8 processed=8 cancelled=0 dropped=0 failed=0 bytes=40`; a feldolgozás sorrendje futásonként változhat, a számlálók nem. A `setarch -R` a címtér-véletlenítést kapcsolja ki az adott folyamatra: újabb Linux-kerneleken az alapértelmezett ASLR-entrópia mellett a ThreadSanitizer futásidejű könyvtára `unexpected memory mapping` hibával leáll, mielőtt a program elindulna.

A konfigurációt, a memóriakorlátot és a határidős leállítást külön teszt ellenőrzi. A memóriamérés a globális `operator new` felülírásával a legalább $S_{\max}$ méretű élő allokációkat számolja; egy blokkolt workerrel, $C = 4$, $W = 1$, $P = 2$ beállítással és 16 egyidejű producerrel az élő nagy allokációk száma pontosan $C + W + P = 7$, a többi 14 producer `busy` eredményt kap. A lassú feldolgozós eset azt ellenőrzi, hogy a leállítás a drain-határidő és egy feldolgozás ideje után véget ér, a maradékot pedig eldobottként számolja el:

```
// tests/service_test.cpp -- konfiguráció, memóriakorlát, határidős leállás
#include "service.hpp"

#include <atomic>
#include <chrono>
```

```cpp
#include <condition_variable>
#include <cstdio>
#include <cstdlib>
#include <mutex>
#include <new>
#include <stdexcept>
#include <thread>
#include <vector>

#define CHECK(condition) do {                                          \
    if (!(condition)) {                                                \
        std::fprintf(stderr, "CHECK failed at line %d\n", __LINE__); \
        std::exit(EXIT_FAILURE);                                       \
    }                                                                  \
} while (0)

using namespace std::chrono_literals;
using clock_type = std::chrono::steady_clock;

// --- A nagy (>= S_max) allokációk élő darabszámának mérése -------------
constexpr std::size_t s_max = 4096;
static std::atomic<long> big_live{0};

void *operator new(std::size_t size)
{
    auto *base = static_cast<std::max_align_t *>(
        std::malloc(size + sizeof(std::max_align_t)));
    if (base == nullptr) throw std::bad_alloc{};
    *reinterpret_cast<std::size_t *>(base) = size;
    if (size >= s_max) big_live.fetch_add(1);
    return base + 1;
}

void operator delete(void *p) noexcept
{
    if (p == nullptr) return;
    auto *base = static_cast<std::max_align_t *>(p) - 1;
    if (*reinterpret_cast<std::size_t *>(base) >= s_max)
        big_live.fetch_sub(1);
    std::free(base);
}

void operator delete(void *p, std::size_t) noexcept { operator delete(p); }

// --- Feldolgozók -------------------------------------------------------
static bool instant(const frame &, std::stop_token) { return true; }

static bool slow(const frame &, std::stop_token)   // nem figyeli a stopot,
{                                                   // de korlátos idejű
    std::this_thread::sleep_for(20ms);
    return true;
}
```

```
static bool blocked(const frame &, std::stop_token st)  // csak stopra áll le
{
    std::mutex m;
    std::condition_variable_any cv;
    std::unique_lock lk(m);
    cv.wait(lk, st, [] { return false; });
    return false;                                       // félbehagyva
}

static bool rejects(service_config cfg)
{
    try {
        service svc{cfg, instant};
    } catch (const std::invalid_argument &) {
        return true;
    }
    return false;
}

static void wait_until(const std::function<bool()> &done)
{
    const auto limit = clock_type::now() + 5s;
    while (!done()) {
        CHECK(clock_type::now() < limit);
        std::this_thread::sleep_for(1ms);
    }
}

int main()
{
    // HOTT: a hibás konfiguráció indításkor kiderül.
    CHECK(rejects({.workers = 0}));
    CHECK(rejects({.capacity = 0}));
    CHECK(rejects({.max_frame = 0}));
    CHECK(rejects({.max_waiting = 0}));
    {
        bool threw = false;
        try { service svc{service_config{}, processor{}}; }
        catch (const std::invalid_argument &) { threw = true; }
        CHECK(threw);
    }

    // workers = 1, capacity = 1: nem ragad be.
    {
        service svc{{.workers = 1, .capacity = 1}, instant};
        for (int i = 0; i < 100; ++i)
            CHECK(svc.submit(frame{"x"}) == submit_result::accepted);
        const service_stats s = svc.shutdown();
        CHECK(s.accepted == 100 && s.processed == 100 && s.dropped == 0);
    }
```

```
// H078: a túl nagy frame a sor előtt elutasul.
{
    service svc{{.max_frame = s_max}, instant};
    CHECK(svc.submit(frame{std::string(s_max + 1, 'x')}) ==
          submit_result::too_large);
    CHECK(svc.shutdown().accepted == 0);
}

// H078: saját tároló = C + W + P frame, 16 producer mellett is.
{
    const service_config cfg{.workers = 1, .capacity = 4,
                             .max_frame = s_max, .max_waiting = 2,
                             .drain_timeout = 100ms};
    service svc{cfg, blocked};
    CHECK(svc.submit(frame{std::string(s_max, 'x')}) ==
          submit_result::accepted);
    wait_until([&] { return svc.pending() == 0; });  // a worker elvitte
    for (int i = 0; i < 4; ++i)
        CHECK(svc.submit(frame{std::string(s_max, 'x')}) ==
              submit_result::accepted);

    std::atomic<int> busy{0}, closed{0};
    std::vector<std::jthread> producers;
    for (int i = 0; i < 16; ++i) {
        producers.emplace_back([&] {
            switch (svc.submit(frame{std::string(s_max, 'x')})) {
            case submit_result::busy:   busy.fetch_add(1); break;
            case submit_result::closed: closed.fetch_add(1); break;
            default: break;
            }
        });
    }
    wait_until([&] { return busy.load() == 14; });
    std::this_thread::sleep_for(50ms);
    CHECK(big_live.load() == 1 + 4 + 2);             // W + C + P

    const auto t0 = clock_type::now();
    const service_stats s = svc.shutdown();
    const auto took = clock_type::now() - t0;
    producers.clear();
    CHECK(closed.load() == 2);                       // a két várakozó
    CHECK(s.accepted == 5 && s.cancelled == 1 && s.dropped == 4);
    CHECK(took < 100ms + 200ms);                     // határidő + tűrés
    CHECK(big_live.load() == 0);
}

// H238: lassú feldolgozó, határidős drain és eldobás.
{
    service svc{{.workers = 1, .capacity = 64,
                 .drain_timeout = 200ms}, slow};
    for (int i = 0; i < 50; ++i)
        CHECK(svc.submit(frame{"x"}) == submit_result::accepted);
```

```
        const auto t0 = clock_type::now();
        const service_stats s = svc.shutdown();
        const auto took = clock_type::now() - t0;
        CHECK(s.accepted == s.processed + s.cancelled + s.dropped + s.failed);
        CHECK(s.dropped > 0 && s.processed > 0);
        CHECK(took < 200ms + 20ms + 200ms);              // + egy feldolgozás
    }
    // H072: a hívó stop kérése külön eredmény (cancelled), nem a szolgáltatás
     ↪  lezárása.
    {
        service svc{service_config{}, [](const frame&, std::stop_token) { return
↪   true; }};
        std::stop_source src;
        src.request_stop();
        // üres sor mellett a push akkor is sikerülhet; a stop-kérés eredménye
         ↪  ezért csak akkor
        // elvárt, ha a push a stop miatt visszautasít (telt sor). Itt a lezárt
         ↪  állapotot ellenőrizzük:
        (void)svc.shutdown();
        CHECK(svc.submit(frame{"x"}, src.get_token()) !=
↪   submit_result::accepted);
    }
    // H071: a lefoglalt kapacitás is korlátozott (nagy reserve, kicsi hasznos
     ↪  hossz).
    {
        service svc{service_config{}, [](const frame&, std::stop_token) { return
↪   true; }};
        std::string big;
        big.reserve(100 * 4096);
        big.resize(1);
        CHECK(svc.submit(frame{std::move(big)}) == submit_result::too_large);
        (void)svc.shutdown();
    }
    // H074: a processor kivétele dokumentált hibaút, a szolgáltatás tovább
     ↪  működik.
    {
        std::atomic<int> calls{0};
        service svc{service_config{}, [&calls](const frame&, std::stop_token) ->
↪   bool {
            if (calls.fetch_add(1) == 0) throw std::runtime_error("boom");
            return true;
        }};
        CHECK(svc.submit(frame{"a"}) == submit_result::accepted);
        CHECK(svc.submit(frame{"b"}) == submit_result::accepted);
        const service_stats s = svc.shutdown();
        CHECK(s.accepted == 2 && s.failed == 1 && s.processed == 1);
    }
    std::puts("service_test: ok");
    return 0;
}

g++ -std=c++20 -O2 -Wall -Wextra -Wpedantic -Wshadow -Werror -pthread \
```

```
    -I. tests/service_test.cpp -o service_test
./service_test
```

Buildjegyzőkönyv (`lab-gcc` profil). A `main.cpp` és a `tests/service_test.cpp` G++ 13.3.0-val, libstdc++ mellett `-Werror` beállítással figyelmeztetés nélkül fordult Linux x86-64 környezetben. A teszt optimalizált builddel ötször egymás után, valamint ThreadSanitizer (`setarch -R`) és ASan/UBSan builddel is sikeres volt; a demó TSan alatt is a fenti kimenetet adta. A határidős ellenőrzések tűréssel dolgoznak (200 ms); terhelt CI-gépen ezt a tűrést a profilban kell rögzíteni.

39.3.3 Buffer- és callback-tulajdon a pipeline határain

A biztonságos alapelv kezdőknek: **aszinkron határon owned értéket adjunk át**. Ha a `submit` paramétere `std::span<const std::byte>` lenne, az csak egy pointert és egy hosszt tartalmazna; a hívó stack frame-je a `submit` visszatérése után megszűnhet, a worker viszont később olvasná. Ez a klasszikus dangling helyzet, amelyet a span, byte és bounds-szerződések (31.3) részben tárgyaltunk. Ez azonban ajánlás, nem nyelvi tilalom: kölcsönzött nézet is átmehet aszinkron határon, ha a tulajdonos élettartama a teljes műveletre igazolt — például strukturált élettartammal (a hívó a művelet befejezéséig vár, mielőtt a puffert felszabadítja), vagy egy külön owner tokennel, amely a művelet végéig életben tartja az adatot — és a hozzáférések szinkronizációja is rendezett. A helyes megoldás, hogy a queue eleme *tulajdonol*: `std::vector<std::byte>`, `std::string` vagy egy move-only `frame` típus. A `span` csak szinkron hívásban, a hívó élettartamán belül használható.

A `shared_ptr` csábító, de csak akkor indokolt, ha valóban több, egymástól független owner van: például a connection objektum és a feldolgozó is életben kell tartsa ugyanazt a puffert, mert a feldolgozás aszinkron módon, a connection lezárása után is olvas. Ha a queue egymaga tulajdonol, a `unique_ptr` vagy a közvetlen érték a helyes választás: kevesebb atomikus művelet, egyértelműbb élettartam.

A **connection state** és a **callback lifetime** összefonódik. Egy tipikus hiba:

```
// HIBÁS tanítási példa: a callback túlélheti a service-t.
svc.set_on_frame([&svc](frame f) { svc.submit(std::move(f)); });
```

Ha a callbacket egy külső komponens tárolja, és a `service` előbb destruálódik, a callback egy halott objektumra hivatkozik. A javítás vagy az, hogy a callback a service-en belül marad (és a service destruktorában deregisztráljuk), vagy az, hogy a callback `std::weak_ptr`-rel ellenőrzi az owner életét:

```
std::weak_ptr<service> weak = svc;
auto cb = [weak](frame f) {
    if (auto s = weak.lock()) s->submit(std::move(f));
};
```

A `weak_ptr::lock()` itt nem stíluselem, hanem a verseny kezelése: a lock és a service destruktora közül az egyik „nyer”, és nem lehet olyan interleaving, amelyben a callback egy félig destruált objektumot használ. A callback regisztrációjának és visszavonásának szerződése ezért tartalmazza, hogy a visszavonás *szinkronizál* a már futó callback-hívásokkal: vagy a callback rövid és nem blokkol, vagy a deregisztráció megvárja a futó hívásokat. Ha ezt nem tesszük meg, a „deregisztrált” callback még futhat, miközben a hivatkozott erőforrás felszabadul.

Az invalidálódás térképe:

Művelet	Mit invalidál	Következmény
`close()`	a queue bemenetét	további `push` `false`
`request_stop()`	a workerek stop tokenjét	a blokkoló `wait` felébredhet
`shutdown()`	a teljes szolgáltatást	a további `submit` `closed` eredményt ad; a statisztika végleges
`service` destructor	minden tagot	minden borrowed hivatkozás dangling
queue `pop`	az adott elem tárolóját	a kivett érték ownershipja a hívóé

39.3.4 Leállítási protokoll, korlátok és tesztelés

A leállítás sorrendje nem önkényes. Tegyük fel, hogy először joinolnánk, és csak utána zárnánk le a queue-t. Ha a queue megtelt, egy termelő éppen a `push`-ban blokkol, és soha nem tud visszatérni, mert a fogyasztók már nem futnak: **deadlock**. Ezért a helyes sorrend:

1. **Bemenet lezárása** (`close()`): a termelők `false`-szal visszatérnek, nem blokkolhatnak örökre, és a „még bekerülhet-e munka” kérdés egyetlen lock alatt eldől.
2. **Drain határidőig** (`wait_empty_until`): a workerek a sorban maradt elemeket feldolgozzák, legfeljebb a `drain_timeout` lejártáig.
3. **Maradék eldobása** (`discard()`): ha a határidőig nem ürült ki a sor, a maradékot eldobjuk, és a darabszámot `dropped` néven elszámoljuk.
4. **Stop kérés** (`request_stop()`): a workerek blokkoló várakozása megszakad, a futó feldolgozás a stop tokent látva korlátos időn belül visszatér.
5. **Join** (`workers_.clear()`): a `jthread` destruktorai joinolnak, így minden worker befejeződése happens-before kapcsolatba kerül a shutdown hívóval.
6. **Erőforrás-felszabadítás**: csak most szabad a queue-t és a hozzá tartozó puffereket eldobni.

A join deadlockmentességének indoklása külön figyelmet érdemel. A join önmagában nem holtpontveszélyes, ha a joinoló szál nem tart olyan zárat, amelyet a worker megpróbál felvenni. A mi `shutdown()`-ünk a `q_.close()` után semmilyen alkalmazás-szintű zárat nem tart, amikor a `clear()` joinol. A második veszélyforrás a `std::stop_callback`: a callback a `request_stop()` hívó szálán fut, miközben a stop állapot belső zárja foglalt; ha a callback olyan mutexet próbál felvenni, amelyet a `request_stop()` hívója tart, holtpont lesz. Ezért a stop callback rövid, nem allokáló és nem zároló kód legyen; a mi példánkban nincs is callback, mert a `condition_variable_any` stop tokenes `wait`-je maga kezeli az ébredést.

A **memóriakorlát** a fentebb levezetett $(C + W + P) \cdot S_{\max} + K$. Az **időkorlát** a leállítás felső korlátja: a `shutdown()` legfeljebb $D + T_{\text{stop}}$ ideig tart, ahol D a `drain_timeout`, T_{stop} pedig az az idő, amely alatt egy futó feldolgozás a stop kérés után visszatér, plusz a join költsége. A korlát tehát a feldolgozó szerződésén múlik: ha egy feldolgozás nem kooperatív módon blokkol (például megtelt pipe-ra ír), a leállítás sem korlátos, és ezt sem a `request_stop()`, sem a szál leválasztása (`detach`) nem javítja meg, utóbbi csak elrejti a még élő erőforrásokat. A határidő után eldobott munka adatvesztés; a `dropped` és a `cancelled` számláló ezt a hívó számára láthatóvá teszi, és a protokollnak kell eldöntenie, hogy elfogadható-e. A két korlát együtt adja a szolgáltatás erőforrás-szerződését.

A **TSan** (ThreadSanitizer) a tényleges adatversenyeket találja meg, például ha a `closed_` flaget valahol lock nélkül olvassuk, vagy ha a callback és a destruktor között nincs happens-before. Nem találja meg a logikai race-eket, például azt, hogy a „drain on stop” politikánk mellett egy elem kétszer kerül feldolgozásra, mert a queue-ból való kivétel és a feldolgozás között a worker megszakad.

Ezért a TSan mellett integritásteszt kell: N termelő, M fogyasztó, ismételt `shutdown()` hívás, és annak ellenőrzése, hogy a feldolgozott elemek multihalmaza pontosan az elküldött elemek multihalmaza, elveszett és duplikált elem nélkül.

Egy TSan-futtatás tipikus parancssora a laborban:

```
TSAN_OPTIONS="halt_on_error=1 second_deadlock_stack=1" setarch -R ./svc-tsan
```

A `second_deadlock_stack` opció a holtpont második szálának stackjét is kiírja, ami a stop callback és a join közötti ciklusok azonosításában segít. Ennek a beállításnak a támogatottsága a használt Clang/GCC runtime verziójától függ; a laborprofilban rögzíteni kell.

39.3.4.1 Opcionális coroutine adapter

A coroutine adapter nem része az alap buildnek, hanem külön **capability profile**: a CMake target csak akkor épül, ha a fordító és a könyvtár támogatja a szükséges C++20 coroutine elemeket, és a projekt ezt feature-test makróval ellenőrzi. A cél egy vékony illesztő, amely a queue-t `co_await`-elhető várakozássá alakítja: a `co_await q.async_pop()` felfüggeszti a coroutine-t, és a queue értesítésekor folytatja. Az élettartam-szerződés ugyanaz, mint eddig: a coroutine frame az owner, a queue borrowed, és a coroutine nem folytatódhat a szolgáltatás leállása után. A részletes `promise_type`/awaiter mechanizmust az Aszinkron I/O adapter és coroutine-labor (35.5) tárgyalta; itt csak a leállítási integrációt rögzítjük: a stop kérésnek a függő coroutine-okat is fel kell ébresztenie, különben a `shutdown()` joinja örökre vár. Ezért az adapter vagy a queue `close()`-ára, vagy egy külön stop tokenre regisztrál, és mindkét esetben a folytatás előtt újra ellenőrzi a lezárt állapotot. Az adapter kísérleti státuszát a buildben és a dokumentációban is jelölni kell; nem szabad az alap szolgáltatás helyességét a coroutine támogatására építeni.

39.3.5 Feladatok

1. feladat (alap). Egészítsd ki a `bounded_queue`-t egy `drain()` metódussal, amely `close()` után az összes maradék elemet kiveszi egy `std::vector<T>`-be, és a queue-t üresen hagyja. Fogalmazd meg, milyen állapotban hívható, és mi történik, ha `drain()` közben egy termelő még `push`-olni próbál.

2. feladat (közepes). Vezess be időkorlátos `push`-t: `push_for(T value, std::chrono::milliseconds limit, std::stop_token st)`, amely `wait_until`-tal várakozik, és ha a határidő lejár, eldobja az elemet, és növeli egy `std::atomic<std::size_t>` számlálót. Indokold, miért kell a számlálónak atomikusnak lennie, és miért nem elég a queue belső mutexe.

3. feladat (nehéz). Tervezd meg a callback-lifetime kezelést `std::weak_ptr`-rel egy olyan szolgáltatásban, ahol a callbacket egy külső, a service-nél tovább élő komponens tárolja. Írj egy TSan alatt futtatható integritástesztet, amely a `shutdown()` alatt is hívja a callbacket, és igazold, hogy nincs use-after-free.

4. feladat (opcionális). Vázold fel a coroutine adapter `async_pop` műveletét, és sorold fel, milyen három feltételt kell ellenőriznie a folytatás előtt a leállási verseny elkerüléséhez.

39.3.6 Megoldások

1. megoldás. A `drain()` csak `closed_ == true` esetén értelmes; ezt a metódus elején ellenőrizzük, és ha nem teljesül, `std::logic_error`-t dobunk (vagy a szerződést dokumentáljuk és `assert`-tel jelezzük). A művelet a queue mutexe alatt kiveszi az összes elemet, majd a `not_full_` feltételváltozót értesíti, hogy az esetleges blokkoló termelők felébredjenek. Mivel a `close()` miatt a `push` úgyis `false`-szal tér vissza, a verseny eldőlt: a `close()` és a `drain()` között nincs olyan interleaving, amelyben új elem kerülne be, feltéve, hogy a `closed_` beállítása és a `drain()` ugyanazon mutex alatt történik. Ellenőrzési feltétel: `drain()` után `size() == 0`, és a visszaadott vektor elemszáma megegyezik a `close()` előtti `size()` értékkel, ha közben nem volt `pop`.

2. megoldás. A számlálót azért kell atomikusan növelni, mert több termelő szál is növelheti egyszerre, és a queue belső mutexe nem védi: a számláló a queue-on kívüli, diagnosztikai célú állapot. Ha a mutex alatt növelnénk, a `push_for` minden eldobása zárat venne fel, ami a backpressure alatti contentiont növelné; az atomikus növelés viszont lock-free. Az ellenőrzési feltétel: a sikeres és eldobott hívások száma összesen a hívások számával egyezik, és a számláló értéke TSan alatt sem mutat adatversenyt. Implementációs váz:

```
bool push_for(T value, std::chrono::milliseconds limit, std::stop_token st) {
    std::unique_lock lk(m_);
    if (!not_full_.wait_for(lk, st, limit,
                            [&] { return closed_ || q_.size() < cap_; })) {
        dropped_.fetch_add(1, std::memory_order_relaxed);
        return false;
    }
    if (closed_ || st.stop_requested()) return false;
    q_.push_back(std::move(value));
    lk.unlock();
    not_empty_.notify_one();
    return true;
}
```

A `wait_for` visszatérési értéke pontosan azt jelzi, hogy a predikátum teljesült-e; ha `false`, akkor a határidő vagy a stop miatt tértünk vissza, és ezt a két esetet a `st.stop_requested()` ellenőrzés választja szét, ha erre szükség van a diagnosztikában.

3. megoldás. A callbacket a `std::weak_ptr<service>`-re zárjuk, és a callback törzse minden hívásnál `lock()`-ol. A `shutdown()` a `close()` után `request_stop()`-ot hív, majd joinol; a callback a `submit`-ban a queue `closed_` flagjén bukik el, tehát a shutdown után érkező hívás `false`-szal tér vissza, nem nyúl a felszabadított erőforráshoz. A TSan-teszt: egy szál ismételten hívja a callbacket, a fő szál `shutdown()`-t hív, és a teszt a végén ellenőrzi, hogy nincs TSan-hiba, és hogy a callback hívásainak száma megegyezik a `submit` visszatérési értékeivel konzisztens módon. Fontos, hogy a callback ne tartson zárat a `lock()` és a `submit` között, amit a service destruktora is felvesz, különben a weak_ptr ellenére holtpont lehetséges. Ellenőrzési feltétel: a teszt TSan alatt hibamentes, és a callback sosem ér el egy `service` objektumot a destruktor megkezdése után (ezt a `lock()` sikertelensége garantálja).

4. megoldás. Az `async_pop` folytatása előtt három feltételt kell ellenőrizni: (a) a queue nem zárt-e le (`closed_ == false`), (b) a stop token nem kérte-e a leállást, és (c) a coroutine handle még érvényes-e (nem destruálódott a frame). Ha bármelyik feltétel nem teljesül, a folytatás helyett a coroutine-t megszakított állapotban kell feloldani, különben a `shutdown()` joinja olyan coroutine-ra várna, amely soha nem folytatódik. A három ellenőrzésnek ugyanabban a szinkronizációs tartományban kell történnie, mint a queue állapotváltozásainak, különben a „lezárt, de mégis folytatódik” verseny marad.

39.3.7 Összefoglaló

A szolgáltatás-életciklus szerződés: állapotok, átmenetek és erőforrás-kötelezettségek együtt. A `bounded_queue` a memóriakorlát és a backpressure hordozója; a `stop_token` a kooperatív megszakításé; a `jthread` a join garantálásáé. Aszinkron határon csak owned érték mehet át, a `span` a szinkron hívások eszköze. A callback élettartamát `weak_ptr`-rel vagy belső regisztrációval kell védeni, és a deregisztrációnak szinkronizálnia kell a futó hívásokkal. A leállítás sorrendje: `close()`, drain a határidőig, a maradék elszámolt eldobása, majd `request_stop()`, join, végül felszabadítás; a felső korlát a drain-határidő és a feldolgozó stop utáni visszatérési ideje; a join nem tarthat olyan zárat, amelyet a worker vagy a stop callback felvesz. A TSan az adatversenyt találja meg, az integritásteszt a logikai race-t; a kettő együtt adja a leállás helyességének bizonyítékát. A coroutine adapter külön capability profile, amelynek a leállítási integrációja ugyanazt a három ellenőrzést igényli, mint a szálas változaté.

39.3.8 Források és ellenőrzési korlátok

A fejezet állításai a C++20 nyelvi és könyvtári modellre, valamint a `jthread`, `stop_token`, `condition_variable_any` C++20-as eszközeire épülnek. A megadott kiinduló pontok — a WG21 oldala, az N4950 munkatervezet, a C++ Core Guidelines, a CMake dokumentáció, a Linux man-pages, valamint a Clang AddressSanitizer és ThreadSanitizer dokumentációja — kutatási kiindulópontok, nem pedig ebben a kéziratban ellenőrzött, idézett bizonyítékok. A konkrét szakaszokra, a `condition_variable_any` stop tokenes overloadjainak könyvtári támogatására, a `TSAN_OPTIONS` beállításokra és a fordítók viselkedésére vonatkozó állításokat a labornak a rögzített compiler/libstdc++/libc++ verzióval, feature-test makróval és compile probe-bal kell igazolnia; A fejezet kódját G++ 13.3.0 és libstdc++ mellett lefordítottuk és lefuttattuk (lásd a buildjegyzőkönyveket); más fordítón és standard könyvtáron ez a labor feladata, addig ott a kód *várt* működés, nem mért eredmény.

39.4 CMake, tesztek és mérési csomag

A port akkor tekinthető késznek, ha a viselkedése nemcsak kézzel kipróbálva, hanem dokumentált módon újra előállítható és ellenőrizhető. Ez az alfejezet ezért nem újabb nyelvi funkciót tanít, hanem a C++ záróprojekt köré épített build-, teszt- és mérési csomagot szerződésként írja le. A funkcionális magot és a szolgáltatás-életciklust a Task, queue és szolgáltatás-életciklus (39.3) rögzítette; a CTest, a negatív tesztek és a sanitizerek általános mintáit a CTest, negatív tesztek és sanitizer CI (38.4) tárgyalta. Itt ezeket a konkrét projektre alkalmazzuk, és kiegészítjük a C/C++ összehasonlítás mérési tervével.

A fejezet állításai eszközök dokumentált lehetőségeire támaszkodnak, de nem jelentenek futtatási eredményt. A megadott parancsok a tanuló laborjában futtatandók; a compiler-, library- és CMake-verziót a projektprofilban kell rögzíteni, mert a konkrét kapcsolók és viselkedésük verziófüggőek. Ahol a szabvány és az implementáció eltérhet, azt külön jelölöm.

39.4.1 A build mint visszajátszható szerződés

A **projektprofil** (project profile) azoknak a külső feltételeknek a rögzített halmaza, amelyek mellett a build és a teszt értelmezhető: operációs rendszer és architektúra, C- és C++-fordító pontos kiadása, standard library megvalósítása és verziója, CMake-verzió, generátor, build típus, nyelvi standard, optimalizálási szint, sanitizer-kapcsolók és a forrás commitja. A **build provenance** ennek géppel is olvasható lenyomata a build mellé írva. Nem azonos a bitre reprodukálható

builddel: utóbbi azt jelenti, hogy azonos bemenetből bitre azonos artefaktum keletkezik, ami erősebb és külön infrastruktúrát igényel; ezzel a Release, debug symbol és reprodukálhatóság (38.6) foglalkozik. Itt a gyengébb, de napi gyakorlatban elegendő célt követjük: a build visszajátszható és a környezete utólag azonosítható legyen.

A CMake configure lépésében kiírható a provenance. Két részlet könnyen elcsúszik. A compiler azonosítója (`CMAKE_CXX_COMPILER_ID`) nem azonosítja a standard könyvtárat: a Clang például libstdc++-szal és libc++-szal is dolgozhat. Ezért a könyvtárat egy kis próbaprogram azonosítja, amely a ténylegesen használt fordítási beállításokkal fordul és fut:

```
// cmake/stdlib_probe.cpp -- a ténylegesen használt C++ standard könyvtár
#include <cstdio>
#include <version>

int main()
{
#if defined(_LIBCPP_VERSION)
    std::printf("libc++ %d", _LIBCPP_VERSION);
#elif defined(__GLIBCXX__)
    std::printf("libstdc++ %d (release %d)", __GLIBCXX__, _GLIBCXX_RELEASE);
#elif defined(_MSVC_STL_VERSION)
    std::printf("msvc-stl %d", _MSVC_STL_VERSION);
#else
    std::printf("unknown");
#endif
}
```

A másik csapda a konfigurációspecifikus változók neve. Ezek nagybetűsek (`CMAKE_CXX_FLAGS_RELWITHDEBINFO`), ezért a `CMAKE_BUILD_TYPE` értékét a lekérdezés előtt nagybetűssé kell alakítani; egyébként a kiírt flag-lista éppen az optimalizálási és `NDEBUG`-kapcsolókat hagyja ki. Többkonfigurációs generátornál (`Ninja Multi-Config`, Visual Studio) a konfiguráció a build idején dől el, ezért minden konfigurációhoz külön fájl készül:

```
# cmake/provenance.cmake -- build provenance konfigurációnként
if(CMAKE_CROSSCOMPILING)
  set(diag_cxx_stdlib "unknown (cross-compiling, nem futtatható próba)")
else()
  try_run(diag_probe_run diag_probe_compiled
    SOURCES "${CMAKE_CURRENT_LIST_DIR}/stdlib_probe.cpp"
    CXX_STANDARD 20
    RUN_OUTPUT_VARIABLE diag_cxx_stdlib)
  if(NOT diag_probe_compiled OR NOT diag_probe_run EQUAL 0)
    set(diag_cxx_stdlib "unknown (a próba nem futott)")
  endif()
endif()

get_property(diag_multi GLOBAL PROPERTY GENERATOR_IS_MULTI_CONFIG)
if(diag_multi)
  set(diag_configs ${CMAKE_CONFIGURATION_TYPES})
else()
  set(diag_configs "${CMAKE_BUILD_TYPE}")
endif()
if(NOT diag_configs)
```

```
  set(diag_configs None)          # nincs konfigurációspecifikus flag
endif()

foreach(cfg IN LISTS diag_configs)
  string(TOUPPER "${cfg}" CFG)    # a változónév nagybetűs: _RELWITHDEBINFO
  file(WRITE "${CMAKE_BINARY_DIR}/build-provenance-${cfg}.txt"
"project=${PROJECT_NAME}
source_dir=${CMAKE_SOURCE_DIR}
cmake=${CMAKE_VERSION}
generator=${CMAKE_GENERATOR}
config=${cfg}
c_compiler=${CMAKE_C_COMPILER_ID} ${CMAKE_C_COMPILER_VERSION}
cxx_compiler=${CMAKE_CXX_COMPILER_ID} ${CMAKE_CXX_COMPILER_VERSION}
cxx_stdlib=${diag_cxx_stdlib}
system=${CMAKE_SYSTEM_NAME}
processor=${CMAKE_SYSTEM_PROCESSOR}
c_flags=${CMAKE_C_FLAGS} ${CMAKE_C_FLAGS_${CFG}}
cxx_flags=${CMAKE_CXX_FLAGS} ${CMAKE_CXX_FLAGS_${CFG}}
")
endforeach()
```

A gyökér `CMakeLists.txt` a `project()` után `include(cmake/provenance.cmake)` sorral veszi fel. Egykonfigurációs generátornál a fájl neve például `build-provenance-RelWithDebInfo.txt`; konfiguráció nélkül `build-provenance-None.txt`, üres konfigurációs flag-listával. Keresztfordításnál a próba nem futtatható; ilyenkor a mező ezt jelzi, és a könyvtárat a toolchain-fájlból kell rögzíteni.

Buildjegyzőkönyv (`lab-gcc` profil). A mintát CMake 4.4.4 és GCC 13.3.0 mellett egy- és többkonfigurációs Ninja generátorral is lefuttattuk. A `RelWithDebInfo` provenance `cxx_flags` mezője (`-O2 -g -DNDEBUG`) megegyezett a `compile_commands.json` fordítási parancsának kapcsolóival; a `cxx_stdlib` mező értéke `libstdc++ 20240904 (release 13)` volt. A `Ninja Multi-Config` generátor a `Debug`, `Release` és `RelWithDebInfo` konfigurációhoz három külön fájlt írt, a megfelelő flagekkel. Clanggal, így a libc++-os ágat, nem futtattuk.

Ez a fájl configure időben keletkezik, és csak a CMake globális változóiból vett **részleges környezeti leírás**, nem a teljes fordítási parancs rekonstrukciója: a `target_compile_options`, a tranzitív usage requirementek, a generator expressionök és a targetenkénti beállítások nem jelennek meg benne, és többkonfigurációs generátornál egyetlen `CMAKE_BUILD_TYPE` sem írja le a buildet. A tényleges fordítási parancsok forrása a `CMAKE_EXPORT_COMPILE_COMMANDS=ON` mellett keletkező `compile_commands.json`, a linkparancsoké a buildnapló (Ninjánál `ninja -t commands`). **Mérés** (CMake 4.4.4, Ninja): egy `INTERFACE` targeten át adott `-Wall -Wextra -Wconversion` a `CMAKE_CXX_FLAGS`/`CMAKE_CXX_FLAGS_RELEASE` értékében nem szerepelt (`-O3 -DNDEBUG`), a `compile_commands.json` parancsában viszont igen; a linkparancsot a `ninja -t commands` adta. A fenti minta ezért csak a CMake változóit rögzíti. Ha a projekt külső változóból (például környezeti változóból) módosítja a flag-eket, vagy targetenként ad hozzá kapcsolókat, azt is rögzíteni kell; a fenti minta csak a standard CMake-változókat fedi le, a target-szintű `target_compile_options` értékeit a `compile_commands.json` mellé mentésével lehet megőrizni. A provenance-t a tesztriport mellé kell menteni, különben egy későbbi hibaelemzésnél nem tudjuk megmondani, melyik fordítóval és milyen optimalizálással született az eredmény.

A buildparancsok a projekt gyökeréből:

```
cmake -S . -B build -G Ninja \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
```

```
  -DCMAKE_C_COMPILER=gcc -DCMAKE_CXX_COMPILER=g++
cmake --build build --parallel
ctest --test-dir build --output-on-failure
```

A `RelWithDebInfo` és a `Release` konfiguráció `NDEBUG`-ot definiál, ezért a tesztek ellenőrzései nem épülhetnek `assert`-re; a projekt tesztjei ezért saját, mindig lefutó `CHECK` makrót használnak (lásd a CMake capstone targetjeit). A `RelWithDebInfo` a méréshez és a hibakereséshez is használható kompromisszum, de a teljesítménymméréshez külön, rögzített `Release` profilt kell használni, és a sanitizeres futásokat külön buildkönyvtárban kell tartani. A C és C++ fordítót nem elég a `CC`/`CXX` környezeti változóra bízni: a CMake cache a első configure után megőrzi a választást, ezért a profilban explicit módon adjuk meg.

39.4.2 Target-gráf és fordítási kapuk

A target-based CMake lényege, hogy a fordítási beállítások ahhoz a targethez tartoznak, amelynek a kódjára vonatkoznak, és nem globális változóként terjednek szét. A warning kapu külön interfész targetbe kerül:

```
add_library(diag_warnings INTERFACE)

if(CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang")
  target_compile_options(diag_warnings INTERFACE
    -Wall -Wextra -Wpedantic -Wconversion -Wshadow -Werror)
else()
  message(WARNING "Ismeretlen compiler: a warning kapu nem aktív")
endif()
```

A `-Wconversion` hasznos a méret- és előjeles konverziók megfogására, de zajos lehet generált és harmadik féltől származó fejlécekben. Ezért a saját targetek linkeljék a `diag_warnings`-t, a külső függőségek fejléceit pedig `SYSTEM` include-ként kell felvenni, hogy a warningok ne a mi kapunkat terheljék. A `-Werror` csak akkor véd, ha a figyelmeztetéskészlet a rögzített compilerverzióhoz kötött; új compiler új figyelmeztetéseket hozhat, ezért a warning kaput a projektprofil részeként kell kezelni, nem örök érvényű szabályként.

A target-gráf a portban három réteget tartalmaz: a C++ magot (`diagcore_core`, forrása `src/diag_core.cpp`), a stabil C API-t (`diagcore_c_api`, forrása `src/diag_c_api.cpp`, nyilvános fejléce a változatlan `include/diag.h`) és a futtatható klienst (`diagcore_cli`, a 17.2 C17-ben fordított `src/main.c` fájlja). A forrásokat és a megfigyelhető viselkedést a portterv és a C++ core rögzíti; a CLI felülete a projektszerződés szerinti `diag [FILE|-]`. A C API target csak a C++ magra linkel, a C kliensek csak a C API-ra. Ez a linkhatár az, amit a Stabil shared library határ és verziózás (36.3) szerződése rögzít. A warning targetet mindhárom saját target linkeli, de a C API fejlécét C fordítóval is le kell fordítani, különben a `extern "C"`-hibák rejtve maradnak.

39.4.3 CTest-tesztkategóriák és a golden corpus

A tesztkészletet érdemes címkékkel (CTest `LABELS`) kategóriákra bontani, mert a gyors és a lassú teszteket más-más gyakorisággal futtatjuk. Négy kategória elegendő: `unit` (egyedi függvények, értéktípusok), `golden` (rögzített bemenet–kimenet párok), `negative` (dokumentált elutasítás) és `abi` (C API/FFI smoke). A fuzz és a mérés külön, nem CTest-vezérelt folyamat.

A **golden corpus** olyan változatlan bemeneti fájlok és elvárt kimenetek halmaza, amelyekhez a parser és a szolgáltatás viselkedését hasonlítjuk. A golden teszt akkor használható, ha a kimenet

determinisztikus, vagy ha a nem determinisztikus részeket (időbélyeg, PID, allokációs cím, sorrend) normalizáljuk. A normalizálás nem csalás, hanem a szerződés része: rögzíteni kell, mely mezők számítanak és melyek nem.

```
add_executable(diagcore_tests tests/test_core.cpp)
target_link_libraries(diagcore_tests PRIVATE diagcore_core diag_warnings)

add_test(NAME unit_core COMMAND diagcore_tests)
set_tests_properties(unit_core PROPERTIES LABELS "unit")

add_executable(spec_examples tests/spec_examples.c)
target_link_libraries(spec_examples PRIVATE diagcore_c_api)
add_test(NAME spec_examples COMMAND spec_examples)
set_tests_properties(spec_examples PROPERTIES LABELS "golden")

# Golden, csonkolt és használati hibás esetek: kilépési kód,
# teljes stdout és teljes stderr összevetése (lásd 17.2).
add_test(NAME cli_contract
  COMMAND sh tests/cli_args.sh $<TARGET_FILE:diagcore_cli>
  WORKING_DIRECTORY "${CMAKE_CURRENT_SOURCE_DIR}")
set_tests_properties(cli_contract PROPERTIES LABELS "golden;negative")
```

A tesztforrások a projektszerződés, a 17.2 labor és a C++ core fájljai. A `cli_args.sh` a golden bemenetet, a csonkolt bemenetet, a túl sok argumentumot és a nem létező fájlt is lefuttatja, és mindegyiknél a pontos kilépési kódot, stdoutot és stderrt ellenőrzi; a negatív esetek ezért nem `WILL_FAIL`-re épülnek. A C és C++ CLI differenciális összevetése (portterv, `tests/diff_c_cxx.sh`) a C referenciabuild targetjét is igényli, ezért külön CI-lépés.

A `WILL_FAIL` önmagában csak azt ellenőrzi, hogy a folyamat nem nulla visszatérési kóddal áll le. Nem bizonyítja, hogy a helyes hibakódot adta, és nem zárja ki, hogy összeomlás miatt tért vissza hibásan. Ezért a negatív teszthez olyan tesztprogram kell, amely a dokumentált hibakódot is ellenőrzi; a fenti `cli_contract` ilyen. A golden teszteknél ugyanez a helyzet: a fájlösszehasonlítás akkor jó, ha a normalizálás és az elvárt fájl verziója is a repóban van.

39.4.4 Parser fuzz és allokációs hiba

A fuzz célja itt nem a helyes parse bizonyítása, hanem a memóriabiztonság és a terminálás: tetszőleges byte-sorozatra a parser vagy elfogad, vagy dokumentált hibával elutasít, de nem olvas ki a pufferből, nem függeszti fel a szálat, és nem fogyaszt korlátlan memóriát. A libFuzzer target csak Clang alatt és külön buildben kapcsolható be:

```
option(DIAGCORE_ENABLE_FUZZING "libFuzzer target" OFF)

if(DIAGCORE_ENABLE_FUZZING)
  add_executable(diagcore_parser_fuzz fuzz/parser_fuzz.cpp)
  target_link_libraries(diagcore_parser_fuzz PRIVATE diagcore_core)
  target_compile_options(diagcore_parser_fuzz PRIVATE
    -fsanitize=fuzzer,address -fno-omit-frame-pointer)
  target_link_options(diagcore_parser_fuzz PRIVATE
    -fsanitize=fuzzer,address)
endif()
```

A fuzz target a `LLVMFuzzerTestOneInput` belépési pontot használja, és a corpus könyvtárat is át kell adni neki. A futtatás:

```
cmake -S . -B build-fuzz -G Ninja \
  -DCMAKE_CXX_COMPILER=clang++ \
  -DDIAGCORE_ENABLE_FUZZING=ON \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build-fuzz --parallel
./build-fuzz/diagcore_parser_fuzz -max_total_time=60 corpus/
```

Az allokációs hiba tesztelése azt ellenőrzi, hogy a kód minden allokációs ponton képes visszajelzni a hibát, nem hagy maga után lezáratlan erőforrást, és nem hagy félkész objektumot. A port core-ja nem allokál; a C API egyetlen allokációs pontja a `dg_create`, amelynek hibáját a hívó `dg_allocator` callbackjével injektáljuk (lásd az FFI smoke tesztet lent). A dokumentált eredmény: a `dg_create` `NULL`-t ad, a CLI pedig `ERR code=alloc` diagnosztikával, 5-ös kilépési kóddal áll le. Ha a platformadapter `std::pmr` tárolót használ, ott egy olyan `std::pmr::memory_resource` jól használható, amely adott számú allokáció után `std::bad_alloc`-ot dob:

```
#include <cstddef>
#include <memory_resource>
#include <new>

struct FailingResource final : std::pmr::memory_resource {
    std::size_t remaining;
    explicit FailingResource(std::size_t n) : remaining(n) {}

    void* do_allocate(std::size_t bytes, std::size_t align) override {
        if (remaining == 0) throw std::bad_alloc{};
        --remaining;
        return std::pmr::new_delete_resource()->allocate(bytes, align);
    }

    void do_deallocate(void* p, std::size_t bytes, std::size_t align) override {
        std::pmr::new_delete_resource()->deallocate(p, bytes, align);
    }

    bool do_is_equal(
        const std::pmr::memory_resource& other) const noexcept override {
        return this == &other;
    }
};
```

A teszt a `FailingResource`-ot úgy állítja be, hogy az első néhány allokáció sikerüljön, majd a következő dobjon. Ezután ellenőrizni kell, hogy a hívó a dokumentált hibát kapja, a destruktorok lefutottak, és a C API határán nem szivárgott ki C++ kivétel. Új C hibakód, például egy `NOMEM` érték bevezetése megfigyelhető változás lenne, ezért csak külön migrációként jöhetne szóba; a kanonikus szerződésben az allokációs hiba jelzése a `NULL` handle.

39.4.5 Sanitizer-mátrix

A sanitizereket nem egyetlen buildben kell futtatni. Az AddressSanitizer és az UndefinedBehaviorSanitizer kombinálható, a ThreadSanitizer viszont külön buildet igényel, mert az ASan és a TSan

memóriahozzáférési modellje nem kompatibilis. A mátrix célja, hogy minden hibamódhoz legyen legalább egy futás, és a futások eredménye a provenance-szel együtt maradjon meg.

Build	Kapcsoló	Mit fed	Mit nem fed
`build-asan`	`-fsanitize=address,undefined`	buffer túlcsordulás, use-after-free, néhány UB	adatverseny, logikai hiba
`build-tsan`	`-fsanitize=thread`	adatverseny, szinkronizációs hiba	memóriahiba, UB
`build-debug`	nincs sanitizer	teljes debugger-láthatóság	rejtett memóriahiba
`build-release`	nincs sanitizer	teljesítmény és bináris méret	a legtöbb memóriahiba

A sanitizeres futás nem helyettesíti sem a determinisztikus tesztet, sem a kódolvasást. Az ASan nem találja meg a nem tűnt hibát, a TSan pedig csak a ténylegesen megtörtént, dinamikusan észlelt versenyhelyzeteket jelzi; egy ritkán előforduló logikai race rejtve maradhat. Ezért a TSan-t a szálas szolgáltatás-életciklus és a queue tesztjein kell futtatni, ismételt és terhelt forgatókönyvvel, nem egyetlen rövid futással.

39.4.6 C API kompatibilitás és FFI smoke test

A C API akkor kompatibilis, ha a fejlécet C fordító is elfogadja, a szimbólumok neve és hívási konvenciója a dokumentált ABI szerint alakul, és a C++ kivételek nem lépnek át a határon. A minimális ellenőrzés egy C nyelven fordított kliens, amely a nyilvános fejlécet include-olja és linkel a könyvtárhoz:

```
/* tests/c_api_smoke.c -- C fordítóval fordított kliens a C++ könyvtárhoz */
#include "diag.h"

#include <stdio.h>
#include <stdlib.h>

#define CHECK(condition) do {                                          \
    if (!(condition)) {                                                \
        (void)fprintf(stderr, "CHECK failed at line %d\n", __LINE__); \
        exit(EXIT_FAILURE);                                            \
    }                                                                  \
} while (0)

static void *refuse(size_t size, void *context)
{
    (void)size;
    ++*(int *)context;
    return NULL;                    /* allokációs hiba injektálása */
}

static void never(void *pointer, void *context)
{
    (void)pointer;
```

```
    (void)context;
    abort();                            /* sikertelen create nem szabadít */
}

int main(void)
{
    static const unsigned char payload[] = "cpu=42\n";

    dg_parser *p = dg_create(NULL);
    CHECK(p != NULL);
    CHECK(dg_feed(p, (dg_view){ payload, sizeof payload - 1 }));
    CHECK(dg_finish(p));
    CHECK(dg_get_report(p).records == 1 && dg_get_report(p).sum == 42);
    dg_destroy(p);

    int calls = 0;
    dg_allocator failing = { refuse, never, &calls };
    CHECK(dg_create(&failing) == NULL);
    CHECK(calls == 1);
    return 0;
}
```

```
add_executable(c_api_smoke tests/c_api_smoke.c)
target_link_libraries(c_api_smoke PRIVATE diagcore_c_api diag_warnings)
set_target_properties(c_api_smoke PROPERTIES C_STANDARD 17 C_EXTENSIONS OFF)
```

A lényeg, hogy ezt a targetet C fordító fordítsa, ne C++ fordító. Ha C++ fordítóval fordítjuk, a `extern "C"` hiánya nem derül ki, és a teszt hamis biztonságot ad. Az FFI smoke teszt nem teljes ABI-kompatibilitási teszt; a verziózott szimbólumok, a struct-layout és a régi binárisokkal való kompatibilitás ellenőrzése az ABI- és FFI-labor, kompatibilitási teszt (36.5) feladata. Itt csak a határ átjárhatóságát igazoljuk. A tesztet GCC 13.3.0-val C17-ben fordítottuk, és a C++ könyvtárhoz (`g++` driverrel), valamint ellenőrzésképpen a C referenciakönyvtárhoz is linkelve lefuttattuk, mindkét esetben sikerrel, a C++ változatot ASan/LSan/UBSan builddel is; az `nm` a C++ könyvtárban a `dg_*` szimbólumokat név-dekoráció nélkül, C kötéssel mutatta. A CMake-fragmentumokat ebben a jegyzőkönyvben nem futtattuk, mert CMake nem állt rendelkezésre.

39.4.7 A mérési terv

A mérés DBEN nem az a kérdés, hogy „gyorsabb-e a C++”, hanem hogy a port ugyanazon a workloadon milyen költséget mutat, és ez hogyan viszonyul a rögzített baseline-hoz. A **workload** a feldolgozandó bemenet és a futtatási mód együttese; a **metrika** a mért mennyiség és mértékegysége; a **mérési terv** rögzíti a kettőt, az ismétlésszámot, a statisztikát és az elfogadási feltételt.

Metrika	Eszköz	Egység	Kontroll
allokációk száma	számláló allocator vagy pmr resource	darab/futás	azonos bemenet, azonos élettartam
csúcs memória	`/usr/bin/time -v` vagy getrusage	KiB	külön folyamat, hideg indítás
latency medián/p95	saját harness, monotonic óra	µs	warmup után, 30 ismétlés

Metrika	Eszköz	Egység	Kontroll
throughput	feldolgozott byte / falóidő	MB/s	azonos corpus, azonos párhuzamosság
bináris méret	`size`, `stat`	byte	stripped és unstripped külön
build idő	`time cmake --build`	s	tiszta buildkönyvtár

A mérési harnessnek ugyanazt a publikus API-t kell hívnia, mint a valós kliensnek, különben a mérés a benchmarkra optimalizált mellékutat mér. A bemenetet fájlból vagy memóriából kell betölteni, és a betöltés idejét külön kell mérni vagy ki kell zárni. A statisztikát előre rögzíteni kell: például medián és p95, nem pedig a legjobb futás. A warmup szükséges, mert az első futás tartalmazza a page faultokat, a dinamikus loader munkáját és a cache feltöltését.

A mérés eredményét a provenance-szel együtt kell menteni, és a riportban külön kell jelölni, hogy a szám mért érték, becslés vagy célérték. A fejezet nem állít konkrét mért számot; a fenti táblázat a mérési kötelezettséget rögzíti, nem annak eredményét.

39.4.8 C/C++ összehasonlítás azonos workloadon

Az összehasonlítás csak akkor érvényes, ha a két implementáció ugyanazt a bemenetet ugyanazzal a feldolgozási szerződéssel dolgozza fel. Ellenőrizni kell, hogy a C változat és a C++ változat ugyanazt a corpus-t kapja, ugyanazt a kimenetet állítja elő, és a mérés ugyanazon a ponton történik. Gyakori hiba, hogy a C++ változat több másolást végez a kényelmesebb értékszemantika miatt, vagy a C változat a benchmarkban elkerüli a valós kliensútvonalat. Az összehasonlításnak ezért legalább három szinten kell futnia: allokációszám, latency és bináris méret. A throughput csak akkor értelmezhető, ha a párhuzamosság és a bufferelés azonos.

A mérési terv elfogadási feltételeit a csapat rögzíti. Például: a funkcionális egyenértékűség kötelező; az allokációszám nem nő a baseline-hoz képest; a p95 latency nem nő 10%-nál többet; a stripped bináris méret nem haladja meg a rögzített keretet. Ezek célértékek, nem mért eredmények, és a port akkor tekinthető késznek, ha a mért értékek ezeket teljesítik a dokumentált profilon.

39.4.9 Gyakori hibák és javításuk

Az egyik leggyakoribb hiba, hogy a warning kapu minden targetre érvényes, beleértve a harmadik féltől származó fejléceket is; ilyenkor a build a függőség miatt áll meg. Javítás: `SYSTEM` include vagy külön interfész target. A második, hogy az ASan és a TSan ugyanabban a buildben szerepel; ez nem támogatott, és félrevezető hibákat ad. Javítás: külön buildkönyvtár és külön CI-futás. A harmadik, hogy a golden teszt nem determinisztikus kimenetre hasonlít; javítás: normalizálás vagy a teszt átalakítása determinisztikus részhalmazra.

További tipikus hiba, hogy a fuzz target nem a valós parser belépési pontját hívja, hanem egy egyszerűsített másolatot; így a fuzz nem a szállított kódot vizsgálja. Az allokációs hiba tesztje gyakran csak azt nézi, hogy a folyamat nem omlik össze, de nem ellenőrzi a leaket és a dokumentált hibakódot. Az FFI smoke tesztet C++ fordítóval fordítják, amitől a C-kompatibilitás nem derül ki. A benchmarkot optimalizálatlan buildben futtatják, vagy a mérés a konstrukciót méri a feldolgozás helyett. Végül a provenance hiányzik, ezért a mérési eredmény később nem reprodukálható.

39.4.10 Feladatok

1. **Diagnosztikai feladat.** Adott egy CMake-projekt, amelyben a `-Werror` minden targetre globálisan be van állítva, és egy harmadik féltől származó header figyelmeztetést okoz. Javítsd a buildet a warning kapu gyengítése nélkül.
2. **Kódolási feladat.** Írj CTest-tesztet, amely egy 1 048 577 bájtos bemenetet (262 144 darab `a=0\n` rekord és egy további bájt) a CLI-vel elutasíttat, és ellenőrzi a dokumentált kilépési kódot és diagnosztikát. Magyarázd el, miért nem elég a `WILL_FAIL`.
3. **Kódolási feladat.** Készíts allokációs hiba tesztet a C API-hoz, amely a `dg_allocator` callbackjével injektál hibát, és ellenőrzi, hogy a hiba a határon a dokumentált módon (`NULL` handle) jelenik meg, a felszabadító callback nem fut, és nincs leak.
4. **Mérési feladat.** Tervezz mérési tervet a C és a C++ parser összehasonlítására. Add meg a workloadot, a metrikákat, az ismétlésszámot és azt, hogy milyen állítást nem lehet a mérésből levonni.

39.4.11 Megoldások

1. megoldás. A harmadik féltől származó fejlécet `SYSTEM` include-ként kell felvenni, például `target_include_directories(diagcore_core SYSTEM PUBLIC ...)`, vagy a külső függőséget külön interfész targetbe kell tenni, amely nem linkeli a `diag_warnings`-t. Így a saját kód továbbra is `-Werror` alatt marad, a külső header pedig nem töri meg a buildet. Ellenőrzési feltétel: a saját forrásokban szándékosan bevezetett figyelmeztetés továbbra is hibát okoz, a külső header viszont nem.

2. megoldás. A `WILL_FAIL` csak azt vizsgálja, hogy a visszatérési kód nem nulla, ezért egy összeomlás vagy rossz hibakód is „sikeres” negatív teszt lenne. Helyette a tesztszkript maga ellenőrzi a kilépési kódot és a pontos diagnosztikát, a határ mindkét oldalán:

```
#!/bin/sh
# tests/reject_oversized.sh DIAG -- a limit utáni első bájt elutasítása
set -u
diag=$1
dir=$(mktemp -d) || exit 2
trap 'rm -rf "$dir"' EXIT

awk 'BEGIN { for (i = 0; i < 262144; i++) printf "a=0\n" }' \
    > "$dir/limit.in"
cp "$dir/limit.in" "$dir/over.in"
printf 'b' >> "$dir/over.in"

"$diag" "$dir/limit.in" > "$dir/ok.out" || exit 1
test "$(cat "$dir/ok.out")" = 'OK records=262144 sum=0 bytes=1048576' ||
    exit 1

rc=0
"$diag" "$dir/over.in" > "$dir/over.out" 2> "$dir/over.err" || rc=$?
test "$rc" -eq 4 || exit 1
test ! -s "$dir/over.out" || exit 1
test "$(cat "$dir/over.err")" = 'ERR code=limit offset=1048576'

add_test(NAME reject_oversized_input
  COMMAND sh tests/reject_oversized.sh $<TARGET_FILE:diagcore_cli>
```

```
  WORKING_DIRECTORY "${CMAKE_CURRENT_SOURCE_DIR}")
set_tests_properties(reject_oversized_input PROPERTIES LABELS "negative")
```

A várt érték a projektszerződésből következik: pontosan 1 048 576 bájt még elfogadható, a következő bájt `limit` hibát ad, az offset 1 048 576, a kilépési kód 4, és a stdout üres. Ellenőrzési feltétel: a limitméretű bemenet sikeres, a plusz egy bájtos elutasítva, és a folyamat nem omlik össze. A szkriptet a C és a C++ CLI-vel is lefuttattuk (GCC 13.3.0), mindkettővel sikeresen; ellenpróbaként a `cat` paranccsal futtatva hibát jelzett.

3. megoldás. Az allokátor első hívása `NULL`-t ad és megszámolja a hívást; a felszabadító callback `abort()`-ot hív, mert sikertelen `dg_create` után nincs mit felszabadítani. A teszt ellenőrzi, hogy a `dg_create` `NULL`-t ad, és az allokátort pontosan egyszer hívta. A C++ wrapperben a placement new csak sikeres foglalás után fut, így félkész objektum sem keletkezik. A fenti `tests/c_api_smoke.c` második fele pontosan ez a teszt. Ellenőrzési feltétel: nincs kivétel a C határon, a felszabadító nem fut, és a LeakSanitizer nem jelez.

4. megoldás. A workload legyen ugyanaz a golden corpus és egy szintetikus stream, azonos párhuzamossággal. A metrikák: allokációszám, p95 latency, throughput és stripped bináris méret. Az ismétlésszám legalább 30, warmup után, külön folyamatban. A mérésből nem lehet azt kijelenteni, hogy „a C++ minden esetben gyorsabb” vagy hogy „a C++ memóriabiztonságosabb”; csak azt, hogy az adott profilon, adott workloadon milyen értékek adódtak. A funkcionális egyenértékűséget a golden tesztek igazolják, a teljesítményt a mérési harness; a kettőt nem szabad összekeverni.

39.4.12 Összefoglaló

A C++ záróprojekt build-, teszt- és mérési csomagja akkor elfogadható, ha a projektprofil és a provenance rögzíti a környezetet, a target-based CMake a saját kódra érvényesíti a warning kaput, a CTest kategóriákra bontja a teszteket, a golden corpus normalizált és determinisztikus, a fuzz és az allokációs hiba teszt a valós belépési pontot vizsgálja, az ASan/UBSan és a TSan külön fut, a C API-t C fordítóval is lefordítjuk, és a mérési terv előre rögzíti a metrikákat, a kontrollokat és az elfogadási feltételeket. A port funkcionális egyenértékűsége és a teljesítménye csak ezekkel a feltételekkel válik ellenőrizhető állítássá.

39.5 Záróreview és C/C++ összehasonlítás

A záróprojekt review-ja nem esztétikai ítélet arról, hogy a C++ változat „modernebb-e”, hanem annak ellenőrzése, hogy a port megtartotta-e a C capstone megfigyelhető szerződését, és ahol szándékosan eltért, ott az eltérés dokumentált, indokolt és tesztelt-e. A review három kérdésre ad választ: működik-e ugyanúgy a külvilág felé; milyen erőforrás- és élettartam-szerződést vállal a C++ kód; és milyen áron — bonyolultságban, buildidőben, futásidejű költségben — kaptuk ezt. A negyedik, gyakran elmaradó rész a nyitott kérdések őszinte felsorolása: mi az, amit a port nem bizonyított, és mi a következő ellenőrizhető lépés.

A fejezet a CMake, tesztek és mérési csomag (39.4) eredményére épül. Feltételezi, hogy a C++ core (C++ core: parser, erőforrások és értéktípusok (39.2)) és a szolgáltatás-életciklus (Task, queue és szolgáltatás-életciklus (39.3)) már elkészült, és hogy a C referencia implementáció elérhető. Nem cél a teljes termékfejlesztés vagy minden új C++ feature beépítése; a coroutine adapter például kísérleti ág marad, ha az élettartam-bizonyíték hiányos. A fejezet külső, ellenőrzött forrásra nem hivatkozik; a szabvány- és ABI-állítások a könyv korábbi, verzióhoz kötött részeire támaszkodnak, és a laborprofilban ellenőrizendők.

39.5.1 A review mint szerződésellenőrzés

A **viselkedési szerződés** (behavioral contract) a C referencia implementáció azon megfigyelhető tulajdonságainak halmaza, amelyekre a kliensek támaszkodhatnak: a parancssori interfész, a protokoll üzenetei, a hibakódok és hibaüzenetek, az EOF és a részleges olvasás kezelése, a leállási sorrend, valamint az erőforrások felszabadításának feltételei. A review akkor korrekt, ha ezt a szerződést a C kódból kinyerve, tételesen összeveti a C++ implementációval. Az összevetés feltétele azonos célkörnyezet: Linux/POSIX, azonos konfiguráció, rögzített compiler- és library-profil. Ha a C++ implementáció ettől eltér — például kivételt használ, ahol a C hibakódot adott —, az eltérés csak akkor fogadható el, ha az adapter a határon dokumentáltan leképezi a viselkedést.

Négy síkon érdemes review-zni. **Funkcionális ekvivalencia:** ugyanazokra a bemenetekre ugyanazok a megfigyelhető kimenetek és hibák. **Erőforrás- és élettartam-szerződés:** minden nyitott fájl, socket, allokáció és lock tulajdonosa, felszabadítási pontja és hibás ági viselkedése ismert. **Költség:** buildidő, bináris méret, allokációk száma, indirekciók, unwinding. **Karbantarthatóság és ABI:** a stabil C határ változatlan, a C++ belső típusai nem szivárognak ki, a hibakezelés nem lép át a nyelvi határon. A review eredménye nem „jó” vagy „rossz”, hanem döntések indoklása és nyitott kérdések listája.

39.5.2 C explicit cleanup versus C++ deterministic destruction

C-ben nincs nyelvi destruktor. A tipikus minta a `goto cleanup`: minden hibás ág egy közös címkére ugrik, ahol a korábban megszerzett erőforrások felszabadítása fordított sorrendben történik. Ennek előnye, hogy egyetlen ellenőrizhető kilépési pont van; hátránya, hogy a helyességet a programozó fegyelme és a review biztosítja. Ha egy új erőforrás-acquire vagy egy új hibás ág kimarad, a hiba csendben marad.

C++-ban a destruktor az objektum élettartamának végén, a nyelvi szabályok szerint fut le: scope kilépéskor, `return`, `throw`, `goto` vagy `break` esetén, a létrehozott objektumok fordított sorrendjében. Ez a **deterministic destruction**: nem szemétgyűjtés, hanem a destruktor hívásának nyelvi garanciája. Feltétele, hogy az objektum teljesen létrejött (a konstruktor befejeződött), a destruktor ne dobjon kivételt, és a move után a forrás objektum érvényes, de „moved-from” állapotban legyen. A garancia nem terjed ki arra, hogy a destruktor helyesen van megírva, és arra sem, hogy a hibát jelzi.

Ez utóbbi a legfontosabb review-pont: a `close` hibája. A C `diag_close` visszatérhet hibával, és a hívó dönthet, hogy naplózza-e. A C++ destruktor nem adhat vissza hibát; ha a `close` hibája számít, explicit `close()` metódus kell, amely `std::error_code`-ot ad, a destruktor pedig csak „best effort” fallback. A review-nak ezt dokumentálnia kell, különben a C++ port csendben elveszíti a C-ben még kezelt hibainformációt.

A következő két példa ugyanazt a fájlolvasó szerződést mutatja be. A C változat explicit cleanupot használ; a C++ változat RAII-t, de a `close` hibáját dokumentált korlátként kezeli. A kód C17, illetve C++20, Linux/POSIX célra; a laborprofilban rögzített GCC/Clang verzióval fordítható.

```
/* diag_c.c - C17, Linux/POSIX */
#include <errno.h>
#include <fcntl.h>
#include <stddef.h>
#include <stdio.h>
#include <unistd.h>

struct diag_reader {
```

```
    int fd;
    unsigned char buf[4096];
    size_t len;
    size_t off;
};

static int diag_open(struct diag_reader *r, const char *path) {
    r->fd = open(path, O_RDONLY);
    if (r->fd < 0) return -1;
    r->len = 0;
    r->off = 0;
    return 0;
}

static void diag_close(struct diag_reader *r) {
    if (r->fd >= 0) {
        (void)close(r->fd);   /* a close hibaja itt nem propagalhato */
        r->fd = -1;
    }
}

/* >0: bytes, 0: EOF, -1: error */
static ssize_t diag_next(struct diag_reader *r, const unsigned char **data) {
    if (r->off == r->len) {
        ssize_t n = 0;
        do {
            n = read(r->fd, r->buf, sizeof r->buf);
        } while (n < 0 && errno == EINTR);
        if (n <= 0) return n;
        r->len = (size_t)n;
        r->off = 0;
    }
    *data = r->buf + r->off;
    size_t n = r->len - r->off;
    r->off = r->len;
    return (ssize_t)n;
}

int main(int argc, char **argv) {
    if (argc != 2) return 2;
    struct diag_reader r;
    if (diag_open(&r, argv[1]) != 0) {
        perror("open");
        return 1;
    }
    int rc = 0;
    for (;;) {
        const unsigned char *data = NULL;
        ssize_t n = diag_next(&r, &data);
        if (n < 0) { perror("read"); rc = 1; break; }
        if (n == 0) break;
        printf("%zd\n", n);
```

```
    }
    diag_close(&r);
    return rc;
}

// diag.cpp - C++20, Linux/POSIX
#include <array>
#include <cerrno>
#include <cstddef>
#include <iostream>
#include <span>
#include <system_error>
#include <fcntl.h>
#include <unistd.h>

namespace diag {

class Reader {
public:
    explicit Reader(const char* path) : fd_(::open(path, O_RDONLY)) {
        if (fd_ < 0) ec_ = std::error_code(errno, std::generic_category());
    }

    ~Reader() noexcept {
        if (fd_ >= 0) (void)::close(fd_);
    }

    Reader(const Reader&) = delete;
    Reader& operator=(const Reader&) = delete;

    Reader(Reader&& other) noexcept : fd_(other.fd_), ec_(other.ec_) {
        other.fd_ = -1;
        other.ec_.clear();
    }

    Reader& operator=(Reader&& other) noexcept {
        if (this != &other) {
            if (fd_ >= 0) (void)::close(fd_);
            fd_ = other.fd_;
            ec_ = other.ec_;
            other.fd_ = -1;
            other.ec_.clear();
        }
        return *this;
    }

    explicit operator bool() const noexcept { return fd_ >= 0; }
    std::error_code error() const noexcept { return ec_; }

    // Szerzodes: a kimeneti span a kovetkezo next() hivasig ervenyes;
    // ures span + nincs hiba = EOF; nem ures error_code = hiba.
    std::error_code next(std::span<const std::byte>& out) {
```

```
        out = {};
        if (fd_ < 0) return std::error_code(EBADF, std::generic_category());
        if (off_ == len_) {
            ssize_t n = 0;
            do {
                n = ::read(fd_, buf_.data(), buf_.size());
            } while (n < 0 && errno == EINTR);
            if (n < 0) return std::error_code(errno, std::generic_category());
            if (n == 0) return {};
            len_ = static_cast<std::size_t>(n);
            off_ = 0;
        }
        out = std::span<const std::byte>(buf_.data() + off_, len_ - off_);
        off_ = len_;
        return {};
    }

private:
    int fd_ = -1;
    std::error_code ec_{};
    std::array<std::byte, 4096> buf_{};
    std::size_t len_ = 0;
    std::size_t off_ = 0;
};

} // namespace diag

int main(int argc, char** argv) {
    if (argc != 2) return 2;
    diag::Reader reader(argv[1]);
    if (!reader) {
        std::cerr << "open: " << reader.error().message() << '\n';
        return 1;
    }
    std::span<const std::byte> chunk;
    for (;;) {
        const std::error_code ec = reader.next(chunk);
        if (ec) {
            std::cerr << "read: " << ec.message() << '\n';
            return 1;
        }
        if (chunk.empty()) break;
        std::cout << chunk.size() << '\n';
    }
    return 0;
}
```

A buildparancsok illusztrációk; a laborprofilban rögzített verzióval és flag-ekkel futtatandók. A fejezet nem állítja, hogy a kód itt lefutott vagy tesztelt.

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -g -O2 \
    -fsanitize=address,undefined diag_c.c -o diag_c
```

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -Wconversion -g -O2 \
    -fsanitize=address,undefined diag.cpp -o diag_cpp
```

A C++ változatban a `Reader` a fd tulajdonosa; a destruktor csak fallback. A `next` spanje a `Reader` belső bufferére mutat: a tulajdonos a `Reader`, a nézet élettartama a következő `next` hívásig tart, az invalidáló művelet a `next` (felülírja a buffert) és a `Reader` megszűnése (dangling), a hibakimenet pedig az `error_code`. A C változatban a `data` pointer ugyanígy a `diag_reader` bufferére mutat, érvényes a következő `diag_next`-ig; a hiba `-1`, a részletek `errno`-ban. A C++ destruktor nem tudja visszaadni a `close` hibáját; ez dokumentált korlát, nem elhallgatott hiba.

39.5.3 Pointer+hossz versus span és borrowed lifetime

A C API-ban a tipikus átadás `(const unsigned char *data, size_t len)`. A szerződés: `data` érvényes `len` bájtig; `len == 0` esetén a pointer lehet null, de a legtöbb implementáció ezt nem követeli meg. A hívó felelős azért, hogy a pointer élettartama lefedje a használatot. A C++ `std::span<const std::byte>` ugyanezt a nézetet fejezi ki egy objektumban: pointer és hossz együtt utazik, a containerből implicit konverzióval előállítható, és `subspan`/`first`/`last` műveletekkel szeletelhető. A span **nem tulajdonol**, nem hosszabbítja meg a mögöttes tároló élettartamát, és nem ellenőriz boundsot: az `operator[]` kívülre indexelése nem definiált viselkedés, `at()` nincs. A `subspan` előfeltétele:

$$0 \le \text{offset} \le \text{size}(), \qquad \text{count} = \text{dynamic_extent} \ \lor \ \text{offset} + \text{count} \le \text{size}().$$

A span előnye, hogy a hossz nem válik el a pointertől, és a függvényszignatúra egyértelműbb. Hátránya, hogy a `std::span` egy C++ template típus, nem megy át a C ABI-n; a C API határán `.data()` és `.size()` lesz belőle. A review-ban minden spanhez meg kell nevezni az ownert, az érvényességi tartományt, az invalidáló műveletet és a hibakimenetet — ugyanúgy, mint a C pointer+hossz esetében. A span nem „biztonságosabb pointer”, csak jobban típusozott nézet.

A példában a `Reader::next(std::span<const std::byte>&)` kimeneti paramétere pontosan ezt a szerződést hordozza. A C változat `const unsigned char **data` paramétere ugyanazt a hosszt külön `ssize_t` visszatérési értékben adja. A review-nak ellenőriznie kell, hogy a hívó nem tartja-e meg a spant a következő `next` után, és hogy a `Reader` megszűnése után nem használ-e dangling nézetet. Ugyanez a hiba `std::string_view`-nál és minden más borrowed view-nál: a nézet visszaadása lokális `std::vector`-ból vagy lokális `std::string`-ből tipikus dangling-forrás.

39.5.4 Hibakezelés: error code, expected és exception

A C hibakezelése visszatérési kód és `errno` kombinációja. Az `errno` globális, ezért a `read`/`open` után azonnal menteni kell, mielőtt más függvény felülírja; a `perror` és a `strerror` a hiba szöveges alakját adja. A C++ három fő lehetőséget kínál. A `std::error_code` nem dob, olcsó, kategóriába sorolja a hibát, és a `std::system_error` kivételként is hordozható. A `std::expected<T,E>` C++23-as; értéket vagy hibát tárol, monadikus láncolást tesz lehetővé, de `.value()` hibás esetben dob. A kivétel unwindingot indít, a destruktorok lefutnak, de a `noexcept` határokon és a C ABI-n nem mehet át; a `std::terminate` kockázata valós.

Mechanizmus	Tipikus használat	Költség	ABI/határ	Kockázat
`error_code`	rendszerhiba, gyakori I/O-hiba	olcsó, nem allokál	stabil, C-re képezhető	ellenőrizetlen visszatérés

Mechanizmus	Tipikus használat	Költség	ABI/határ	Kockázat
`expected` (C++23)	típusos hiba, érték vagy hiba	olcsó, de template unwinding, `.eh_frame`	C++ belső	`.value()` dob, feature-test kell
exception	ritka, kivételes hiba		C++ belső, nem C ABI	`noexcept` megsértése, `terminate`

A capstone döntése indokolható: a gyakori I/O-hibákat `error_code`-dal vagy `expected`-tel kell visszaadni; a programozási hibákat assert/`terminate` jelzi; a kivétel csak belső, ritka hibaútvonalon marad, és az `extern "C"` határon `try/catch`-csel le kell fordítani hibakódra. A C++20-as alap miatt a példa `error_code` + kimeneti paraméter; a C++23-as `expected` ág feature-testtel kapcsolható:

```
#if defined(__cpp_lib_expected) && __cpp_lib_expected >= 202202L
std::expected<std::span<const std::byte>, std::error_code> next_expected();
#endif
```

A `std::error_code` `message()` hívása allokálhat, a `what()` pointer élettartama a kivételobjektumhoz kötött; ezeket a review-ban nem szabad „ingyenesnek” feltételezni. A C API határán a C++ `error_code`-ot dokumentáltan `int`-re kell képezni, és a `data` pointer élettartamát a C API dokumentációjában rögzíteni kell.

39.5.5 Dispatch: statikus, dinamikus és type-erased

A feldolgozó lépések kiválasztásának három fő mechanizmusa van. A **dinamikus** dispatch virtuális függvény: a döntés futásidőben, a vtable-en keresztül történik; egy indirekció hívásonként, nyílt típushalmaz, de a C++ ABI érzékeny rá. A **statikus** dispatch template, CRTP vagy concept: a döntés fordításidőben, nincs futásidejű indirekció, viszont kódduzzadás és hosszabb buildidő lehet; a típusnak a fordítási egységben láthatónak kell lennie. A **type-erased** dispatch (`std::function`, C callback + `void*`) futásidőben tárolja a callable-t; a `std::function` birtokolja és másolja a callable-t, allokálhat, a „kis objektum optimalizálás” implementációfüggő; a C callback nem birtokol, a context élettartamáért a hívó felel.

Mechanizmus	Döntés ideje	Futásidejű költség	Bővíthetőség	ABI
virtuális függvény	futásidő	vtable-indirekció	nyílt halmaz	C++ ABI
template/CRTP	fordításidejű	nincs indirekció, kódduzzadás	zárt halmaz	header
`std::function`	futásidő	indirekció, esetleg allokáció	nyílt halmaz	C++ ABI
C callback + `void*`	futásidő	indirekció	nyílt halmaz	stabil C ABI

A döntést a hívási pontok száma, a típusok halmaza, a hot path és az ABI-stabilitás együtt vezérli. Ha a lépések száma kicsi és fordításidőben ismert, a template a természetes választás; ha futásidőben konfigurált plugin-szerű lépések vannak, a type-erased forma; ha a stabil C ABI a követelmény, a C callback. A `std::function` mindenhol való használata a hot pathon indokolatlan allokációt és indirekciót hozhat; a review-nak méréssel vagy legalább költségbecsléssel kell alátámasztania a választást.

39.5.6 Compile-time és runtime költségmérleg

A **compile-time költség** a template-instantiationök száma, a header-include gráf mérete, a `constexpr` kiértékelés és a fordítási egységek száma. A túlzott template-használat növelheti a buildidőt és a binárist; a `constexpr` nem feltétlenül ingyenes, mert a fordítónak ki kell értékelnie. A **runtime költség** az allokációk száma, az indirekciók, a másolások, az atomi műveletek (`shared_ptr` refcount), az unwinding és a cache-locality. A C++ zero-overhead elve nem azt jelenti, hogy minden konstrukció ingyenes, hanem azt, hogy amit nem használunk, azért nem fizetünk; amit használunk, annak a költsége megjelenhet.

A review csak azt állíthatja mértnek, ami a CMake, tesztek és mérési csomag (39.4) mérési csomagjában rögzített, reprodukálható módszertannal szerepel: azonos bemenet, azonos compiler-flag, többszöri futás, medián és outlier-kezelés, correctness gate a benchmark előtt. A „várt” és a „mért” külön kategória. A `std::function` allokációja, a `virtual` indirekciója és a `shared_ptr` atomi refcountja tipikus vitapont; ezeket nem elég elméletben eldönteni, ha a capstone teljesítményérzékeny. A mérési tervet a feladatok között részletezzük; a fejezet nem állítja, hogy bármelyik változat itt megmért eredményt adott.

39.5.7 Stabil interfész, tesztbizonyíték és fennmaradó korlátok

A stabil határ a C API marad; a C++ implementáció mögötte van. Az `extern "C"` kikapcsolja a névtorzítást, az opaque `struct` elrejti a belső layoutot, a `create`/`destroy` pár azonos allocatort használ, a hibakód `int`, a `data` pointer élettartama pedig a következő hívásig tart. A C++ típusok — `std::span`, `std::expected`, `std::string` — nem mennek át a C ABI-n; a határon egyszerű pointerre, hosszra és hibakódra kell leképezni őket.

```
/* diag_api.h - stabil C hatar, C es C++ fordito is latja */
#ifndef DIAG_API_H
#define DIAG_API_H
#include <stddef.h>
#ifdef __cplusplus
extern "C" {
#endif

typedef struct diag_reader diag_reader;

diag_reader *diag_reader_create(const char *path);
void diag_reader_destroy(diag_reader *r);
/* 0 siker, -1 hiba; data a kovetkezo hivasig ervenyes */
int diag_reader_next(diag_reader *r, const unsigned char **data, size_t *len);
const char *diag_reader_error(const diag_reader *r);

#ifdef __cplusplus
}
#endif
#endif
```

A tesztbizonyíték akkor elég, ha ugyanaz a funkcionális teszt fut a C és a C++ implementáción, és a C API-t külön C fordítóval is lefordítjuk. Az ABI-ellenőrzés `nm -D`/`readelf -Ws` vizsgálattal, a sanitizerek (ASan/UBSan) futtatásával, negatív tesztekkel (hibás fájl, EOF, rövid olvasás, `EINTR`) és a hibakód-leképezés tesztelésével egészül ki. A kivételnek nem szabad átlépnie az `extern "C"` határon; ha a belső kód mégis dob, a határon `try/catch` fordítja hibakódra.

Nyitott kérdés	Miért nyitott	Következő lépés	Elfogadási feltétel
`close` hiba	destruktor nem adhat vissza hibát	explicit `close()` + dokumentált fallback	a hiba naplózva, a viselkedés tesztelve
`EINTR`/signal safety	a teljes robusztus I/O ciklus külön téma	a File descriptor és robusztus I/O ciklus (12.1) mintájának átemelése	szignál alatti újrapróbálás tesztelve
C++23 `expected`	compiler/library támogatás változó	feature-test + C++20 fallback	mindkét ág ugyanazt a viselkedést adja
coroutine adapter	élettartam-bizonyíték hiányos	külön kísérleti ág, nem a főútvonal	cancellation és dangling tesztelve
mérési lefedettség	nem minden dispatch-változat mért	a 39.4 benchmark kiegészítése	reprodukálható medián, correctness gate
platform/ABI	x86-64 System V az elsődleges labor	AArch64 összehasonlítás	ABI-teszt a második célon

A review értékelése négy szempont szerint összegezhető. **Helyesség:** a funkcionális szerződés teljesül-e, és a hibás ágak lefedettek-e. **Bonyolultság:** a C++ port több implicit szerződést hoz (destruktor, move, konverzió, template), ezért a karbantartási költség nem feltétlenül alacsonyabb; a move-szemantika és a view-k új hibalehetőségek. **Ownership:** a `Reader`/`unique_ptr`/RAII egyértelműbb tulajdonost ad, de a `shared_ptr` és a borrowed view-k visszahozzák a C pointeres hibákat. **Költség:** a buildidő, a bináris méret és a futásidejű indirekció mérlegelendő; a „modernebb” nem azonos a „jobb” választással.

39.5.8 Tipikus hibák és javítások

1. **A span visszaadása lokális bufferből.** A `std::span` nem tartja életben a tárolót; a visszaadott nézet dangling. Javítás: az owner élettartamának dokumentálása, vagy érték visszaadása.
2. **A move után a régi fd használata.** A move constructor után a forrás `fd_ = -1`; ha ezt nem tesszük meg, dupla `close` lesz. Javítás: move-and-swap vagy explicit `-1` beállítás, `= delete` a másolásra.
3. **A `close` hiba elvesztése.** A destruktor nem jelezhet; ha a hiba számít, explicit `close()` kell. Javítás: külön `close()` + dokumentált fallback.
4. **`errno` és `error_code` keverése.** Az `errno`-t a hívás után azonnal menteni kell; a `read` és a `close` külön hibát adhat. Javítás: minden rendszerhívás után azonnali mentés, külön hibacsatorna.
5. **`expected::value()` ellenőrzés nélkül.** Hibás `expected` esetén dob; a `has_value()` vagy a monadikus lánc kötelező. Javítás: `value_or`, `and_then`, vagy explicit ág.
6. **`std::function` a hot pathon.** Allokálhat és indirektál; a `function_ref` (C++26) vagy a C callback olcsóbb lehet. Javítás: mérés, majd a legkisebb elégséges mechanizmus.
7. **Kivétel átengedése `extern "C"` határon.** A C ABI nem ismeri a kivételt; `terminate` vagy nem definiált viselkedés lehet. Javítás: `try/catch` a határon, hibakódra fordítva.
8. **A C API `data` pointer élettartamának elfelejtése.** A pointer a következő hívásig érvényes; a kliens nem tárolhatja tovább. Javítás: dokumentált szerződés és teszt.

39.5.9 Feladatok

1. **Auditfeladat.** A fenti C és C++ példa alapján készíts erőforrás- és hibaszerződés-táblázatot: minden erőforrás, tulajdonos, acquire, release, hibás ág, dupla felszabadítás kockázata. Hasonlítsd össze a `goto cleanup` és a RAII megoldást, és jelöld, hol veszíthető el hiba.
2. **Kódolási feladat.** Írd át a `Reader::next`-et `std::expected`-alapúra C++23-on, feature-testtel, és adj C++20-as fallbackot. Tartsd meg a span élettartam-szerződését, és dokumentáld az EOF és a hiba megkülönböztetését.
3. **Diagnosztikai feladat.** Készíts mérési tervet a `virtual Stage`, a `template Stage` és a `std::function` közötti választáshoz. Add meg a hipotézist, a független változót, a mérendő metrikákat, a correctness gate-et és az elfogadási feltételt.
4. **Review-feladat.** Ellenőrizd a stabil C API határt: `extern "C"`, opaque handle, allocator pár, hibakód-leképezés, kivétel tilalma. Adj öt ellenőrzési pontot és a hozzájuk tartozó bizonyítékot.

39.5.10 Megoldások

1. megoldás. Az erőforrás-táblázat sorai: `fd` (tulajdonos: `diag_reader`/`Reader`; acquire: `open`; release: `diag_close`/`~Reader`; hibás ág: `open` hiba → nincs fd; `read` hiba → close a fix ponton); buffer (tulajdonos: a reader; nincs külön release); `data`/span (nem tulajdonos; érvényes a következő hívásig). A C-ben a `diag_close` a `break` után minden ágon lefut; `goto cleanup` esetén egy címke gyűjti a release-eket. A C++-ban a `~Reader` a scope exiten fut, de a `close` hibáját nem adja vissza. Ellenőrzés: minden acquire után van release, a move után a forrás `-1`, a `data` pointer nem él túl egy `next`-et. Indoklás: a RAII csökkenti a kihagyott release kockázatát, de a hibajelzést explicit `close()`-ra kell bízni.

2. megoldás. Az `expected` verzió szignatúrája:

```
std::expected<std::span<const std::byte>, std::error_code> next();
```

Az EOF-ot üres `std::span{}` értékkel, a hibát `std::unexpect, ec`-vel kell visszaadni. A C++20-as fallback a kimeneti paraméteres `error_code` marad. A feature-test:

```
#if defined(__cpp_lib_expected) && __cpp_lib_expected >= 202202L
// expected ág
#else
// C++20 fallback
#endif
```

Ellenőrzés: minden ágon `has_value()` vagy `error()` vizsgálat; a span élettartama a következő hívásig; a C++20 és C++23 változat ugyanazt a kimenetet adja a tesztbemenetekre. Indoklás: az `expected` típusosabb, de a `.value()` hibás esetben dob, ezért a hívónak ellenőriznie kell; a fallback megőrzi a C++20-as alapot.

3. megoldás. Hipotézis: a template a hot pathon kevesebb indirekciót ad, de nagyobb binárist és hosszabb buildidőt; a `std::function` allokálhat és másol; a virtual egyszerű, de indirektál. Független változó: a dispatch mechanizmus. Metrikák: medián futásidő, allokációk száma, bináris méret, buildidő. Correctness gate: ugyanaz a kimenet minden változaton, a teszt a benchmark előtt fut. Elfogadási feltétel: a template csak akkor választandó, ha a nyereség mérhető és a bináris növekedés elfogadható; a `std::function` csak ott, ahol a típus törlése valóban kell. A fejezet nem állítja, hogy a mérés megtörtént; a terv a CMake, tesztek és mérési csomag (39.4) alfejezet csomagjában futtatandó.

4. megoldás. Ellenőrzési pontok: (a) a header `extern "C"` blokkban van, C fordítóval is befordul; (b) az opaque `struct` definíciója nem látszik a kliensben; (c) a `create`/`destroy` ugyanazt az allocatort használja; (d) a `next` hibakódja és a `data` élettartama dokumentált, és a kliens teszteli; (e) a C++ implementáció `try`/`catch`-csel fordítja a kivételt hibakódra, és nem enged kivételt a C ABI-n. Bizonyíték: C kliensprogram, `nm -D`/`readelf -Ws`, sanitizer, negatív teszt. Indoklás: a stabil határ nem a C++ típusokon, hanem a C ABI-n keresztül ellenőrizhető.

39.5.11 Összefoglaló

A C++ port review-ja a C capstone viselkedési szerződéséhez méri a változatot, nem a nyelvi modernséghez. A C explicit cleanupja egy kilépési pontot ad, a C++ determinisztikus destruktor nyelvi garanciát, de a destruktor nem jelezhet hibát; a `close` hibáját explicit `close()`-ra kell bízni. A pointer+hossz és a `std::span` ugyanazt a borrowed lifetime-szerződést hordozza; a span nem tulajdonol és nem ellenőriz boundsot. A hibakezelésben a gyakori I/O-hiba `error_code`/`expected`, a ritka belső hiba kivétel, a C ABI határán hibakód. A dispatch, a template és a type erasure közötti választás költség- és ABI-kérdés, amelyet mérni vagy legalább indokolni kell. A stabil C API, a tesztbizonyíték és a nyitott kérdések listája teszi a review-t őszintévé; a továbbtanulási terv ezekből a hiányokból épül fel.

III. rész: C++23 elmélyítés és C++26 kitekintés

40 C++23 nyelvi újdonságok: tagfüggvények, konstans kiértékelés és kifejezések

40.1 C++23 laborprofil, szabványstátusz és feature-test

A C++23 laborprofil kiterjesztése nem pusztán egy újabb `-std=` kapcsoló beírása a buildparancsba. Azt kell rögzíteni, milyen bizonyítékot fogadunk el arra, hogy egy nyelvi vagy könyvtári képesség tényleg használható a kiválasztott toolchainben. A C++20-as alapokat itt nem ismételjük; a feature-test és build-capability alapokhoz lásd Feature-test, standard library és build capability (33.5), a szabvány- és dialektusszerződéshez Szabványok, dialektusok és hordozhatósági szerződés (1.2), a reprodukálható laborhoz Reprodukálható fordítási és diagnosztikai labor (1.3). Ez az alfejezet a C++23-ra való átállás státusz- és ellenőrzési kérdéseit tárgyalja.

40.1.1 A C++23 állapot három rétege

A C++23-cal kapcsolatban három különböző dolgot kell megkülönböztetni.

Az **ISO/IEC 14882:2024** a kiadott, normatív szabvány. Ez mondja meg, mit jelent a C++23 nyelv és a standard könyvtár a szabvány szövege szerint. A **N4950** nyilvános munkatervezet: hasznos, hozzáférhető dokumentum a szabvány tartalmának megértéséhez, de nem azonos a kiadott ISO szöveggel. A munkatervezet és a kiadott szabvány között szerkesztői és szavazási folyamatból eredő eltérések lehetnek; egy fordító pedig jellemzően nem „a teljes C++23-at” implementálja, hanem egyes javaslatokat, azok adott állapotában.

A harmadik réteg maga az **implementáció**. Ide tartozik a fordító, a standard könyvtár, a linker és a futásidejű környezet. Egy képesség lehet a szabványban, szerepelhet a munkatervezetben, és mégis hiányozhat a laborban használt GCC/Clang vagy libstdc++/libc++ párosból. Ezért a laborprofilban nem elég azt írni, hogy „C++23”; rögzíteni kell a konkrét fordítót, a targetet, a standard könyvtár implementációját és verzióját, valamint a vizsgált feature-test makrókat.

Réteg	Kérdés	Tipikus bizonyíték	Korlát
Kiadott szabvány	Mi normatív?	ISO/IEC 14882:2024	Nem mondja meg, melyik toolchain támogatja
Munkatervezet	Mi volt a tervezet tartalma?	N4950	Nem azonos a kiadott szöveggel
Implementáció	Mit tud a labor?	feature-test, compile/link/run probe	Verzió-, target- és kapcsolófüggő

A konkrét GCC/Clang/libstdc++/libc++ támogatási táblázatot ebben az anyagban nem tekintjük ellenőrzöttnek. A helyes minta: a laborprofil rögzíti a tényleges verziókat, a fejezet pedig olyan ellenőrzési eljárást ad, amely ezekhez a verziókhoz köthető.

40.1.2 Fordítási mód és a `__cplusplus` jelentése

A C++23-as profil alapesetben explicit módon kéri a nyelvi módot:

```
g++ -std=c++23 -Wall -Wextra -Wpedantic -O2 probe.cpp -o probe-gcc
clang++ -std=c++23 -Wall -Wextra -Wpedantic -O2 probe.cpp -o probe-clang
```

A `-std=c++23` a célzott név. Egyes fordítókban átmeneti névként `-std=c++2b` is elfogadott lehet, de ez verziófüggő, és a laborban nem helyettesítheti a ténylegesen támogatott kapcsoló ellenőrzését. A `-std=c++23` elfogadása önmagában még nem bizonyítja, hogy minden C++23-as képesség elérhető.

A C++23-as nyelvi módban a szabvány szerinti elvárt érték:

```
static_assert(__cplusplus == 202302L, "nem C++23 nyelvi mód");
```

Ez az érték a fordítótól és a nyelvi módtól függ; a laborban ellenőrizni kell, hogy a fordító tényleg ezt definiálja-e. A `__cplusplus == 202302L` azt mutatja, hogy a fordítás nyelvi módja C++23-ként van beállítva. Nem bizonyítja, hogy a standard könyvtár adott headerjei elérhetők, hogy egy adott túlterhelés létezik, vagy hogy a linker megtalálja a szükséges szimbólumot.

Érdemes külön kezelni a következőket:

- a fordító által elfogadott nyelvi mód;
- a `__cplusplus` tényleges értéke;
- a standard könyvtár implementációja és verziója;
- a konkrét header és feature-test makró;
- a linkelési és futásidejű viselkedés.

40.1.3 Feature-test makrók: nyelvi és könyvtári képesség

A **feature-test makró** olyan előre definiált makró, amelynek neve és értéke egy képesség jelenlétét és változatát jelzi. A nyelvi képességeknél jellemzően `__cpp_*`, a könyvtári képességeknél `__cpp_lib_*` alakot látsz. A könyvtári makrók jelentős része a `<version>` headerből érhető el, ezért a könyvtári ellenőrzés előtt ezt be kell include-olni.

Fontos feltétel: a makró létezése implementációs állítás, nem teljes körű garancia. Ellenőrizni kell a nevet és a küszöbértéket is, lehetőleg `#if defined(...) && ... >= küszöb` formában. A puszta `#ifdef` egy korábbi, részleges vagy kísérleti értéket is elfogadhat.

A gyakorlati ellenőrzés három szintje:

1. **Fordítási próba:** a fordító elfogadja-e a szintaxist és a headert.
2. **Linkelési próba:** a szükséges szimbólum megtalálható-e, nem kell-e külön kapcsoló.
3. **Futásidejű füstteszt:** a képesség a várt alapviselkedést mutatja-e.

A feature-test makró ebből az első kettőt nem feltétlenül fedi le. Ezért a laborprofilban a makró mellett compile probe, link probe és rövid runtime smoke test szerepeljen.

Levezetés: miért nem elég a `__cplusplus`?

1. A `__cplusplus` értékét a fordító a nyelvi mód alapján állítja be.
2. A standard könyvtár headerjeit és implementációját ettől függetlenül választja a toolchain.
3. Egy nyelvi feature-test makró a fordító, egy `__cpp_lib_*` makró a könyvtár állítását tükrözi.
4. Ezért a nyelvi mód, a nyelvi képesség és a könyvtári képesség külön ellenőrizendő; a következtetés csak a három együttes eredményéből vonható le.

Az alábbi példa egy nyelvi és egy könyvtári képességre ad capability checket, fallbackkal. A küszöbértékek szemléltetők; a laborban a tényleges `<version>` és fordítási dokumentáció alapján kell megerősíteni őket.

```
#include <version>
#include <cstdio>

#if defined(__cpp_explicit_this_parameter) && __cpp_explicit_this_parameter >=
 ↪  202110L
#  define HAVE_EXPLICIT_THIS 1
#else
#  define HAVE_EXPLICIT_THIS 0
#endif

#if defined(__cpp_lib_expected) && __cpp_lib_expected >= 202202L
#  include <expected>
#  define HAVE_EXPECTED 1
#else
#  define HAVE_EXPECTED 0
#endif

struct Probe {
    int value = 42;

#if HAVE_EXPLICIT_THIS
    void ping(this Probe& self) {
        std::printf("explicit-this=1 expected=%d value=%d\n",
                    HAVE_EXPECTED, self.value);
    }
#else
    void ping() {
        std::printf("explicit-this=0 expected=%d value=%d\n",
                    HAVE_EXPECTED, value);
    }
#endif
};

int main() {
    Probe p;
    p.ping();
}
```

A példa nem állítja, hogy bármely konkrét fordítón teszteltük. A várt ágak: C++20 módban jellemzően a fallback ág fut; teljes C++23 támogatásnál az explicit object parameter ág; részleges könyvtári támogatásnál a nyelvi ág igen, a `HAVE_EXPECTED` érték viszont 0 lehet. A tényleges kimenetet a labor rögzíti.

40.1.4 Támogatási állapot: fordító, könyvtár és linkelés

A GCC és a Clang nyelvi támogatása, valamint a libstdc++ és a libc++ könyvtári támogatása külön tengely. A GCC jellemzően libstdc++-szal dolgozik; a Clang választhat libstdc++ és libc++ között, például `-stdlib=libc++` kapcsolóval. A laborprofilnak rögzítenie kell a tényleges

párosítást, mert a „Clang támogatja” állítás önmagában félrevezető: a kérdés az, hogy a Clang melyik standard könyvtárral, melyik targeten, milyen kapcsolókkal.

Részleges támogatásnál előfordulhat, hogy a nyelvi szintaxis fordul, a header létezik, de a feature-test makró hiányzik; vagy a makró definiált, de a implementáció hiányos. Kísérleti könyvtári elemek `std::experimental` névtérben vagy külön headerben lehetnek, és nem részei a C++23 standardnak. Egyes GCC/libstdc++ kiadásokban bizonyos C++23-as könyvtári elemek külön linkelési kapcsolót igényelhetnek, például `-lstdc++exp`; ezt nem szabad találgatni, hanem a laborban dokumentálni kell. A linkelési igényt a linker hibája vagy a könyvtár dokumentációja igazolhatja.

Rögzítendő adat	Példa	Miért kell
Fordító és verzió	GCC x.y vagy Clang x.y	A nyelvi támogatás verziófüggő
Target	x86-64 System V, LP64	ABI és elérhetőség targetfüggő
Standard flag	`-std=c++23`, extensions off	A nyelvi mód pontos azonosítása
Standard könyvtár	libstdc++ x.y vagy libc++ x.y	A könyvtári támogatás külön tengely
Feature-test snapshot	`__cpp_*`, `__cpp_lib_*` értékek	A tényleges állítás rögzítése
Linkelési kapcsolók	pl. `-lstdc++exp`, ha dokumentált	A linkelési próba feltétele
Dátum	a mérés/ellenőrzés dátuma	A támogatási állapot változhat

40.1.5 Kétprofilos CMake és CI mátrix

A C++20 profil maradjon meg baseline-ként, a C++23 pedig jelölt profilként fusson. A kettő együtt mutatja meg, hogy a fallback valóban működik-e, és hogy a C++23-as ág nem csak a fejlesztői gépen fordul-e le. Egy minimális, kétprofilos CMake-példa:

```
cmake_minimum_required(VERSION 3.20)
project(cpp23_probe LANGUAGES CXX)

set(PROBE_CXX_STANDARD "23" CACHE STRING "20 vagy 23")
set(CMAKE_CXX_STANDARD "${PROBE_CXX_STANDARD}")
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

add_executable(probe probe.cpp)
target_compile_options(probe PRIVATE -Wall -Wextra -Wpedantic)

enable_testing()
add_test(NAME probe_runs COMMAND probe)
```

A labor CMake-verziójában ellenőrizni kell, hogy a `23` érték ismert-e; ha nem, a standard flaget target-szinten kell átadni, és ezt a profilban dokumentálni kell. A két profil buildje:

```
cmake -S . -B build20 -DPROBE_CXX_STANDARD=20 -DCMAKE_BUILD_TYPE=Debug
cmake --build build20
ctest --test-dir build20 --output-on-failure
```

```
cmake -S . -B build23 -DPROBE_CXX_STANDARD=23 -DCMAKE_BUILD_TYPE=Debug
cmake --build build23
ctest --test-dir build23 --output-on-failure
```

A CI mátrixban érdemes legalább a „GCC + libstdc++” és a „Clang + a laborban rögzített standard könyvtár” párost futtatni, mindkettőt C++20 és C++23 módban. A C++20 futás a fallbacket ellenőrzi; a C++23 futás a jelölt képességeket. Ha a C++23-as build elbukik, nem szabad csendben C++20-ra váltani: a képességet hiányzóként kell jelölni, a fallbacknek pedig továbbra is működnie kell.

40.1.6 Kidolgozott példa: a probe értelmezése

A fenti `probe.cpp` három tipikus profilon a következőképpen viselkedhet. Az értékek illusztratívak, nem mért eredmények.

Profil	`__cplusplus`	`__cpp_explicit_this_parameter`	`__cpp_lib_expected`	Várható ág
C++20 baseline	202002L	nincs definiálva	nincs definiálva	fallback mindkettőre
C++23 nyelvi, részleges könyvtár	202302L	definiált, megfelelő küszöb	nincs definiálva	explicit this, expected fallback
C++23 teljes laborprofil	202302L	definiált	definiált	explicit this és expected elérhető

A diagnózis menete: először a `__cplusplus` és a nyelvi feature-test makró mondja meg, hogy a fordító mit állít. Utána a `<version>` és a `__cpp_lib_*` makró a könyvtár állítását mutatja. Végül a linkelési és futásidejű próba igazolja, hogy az állítás a laborban használható. Ha a makró definiált, de a linkelés elbukik, a profil hiányos; ha a makró nincs definiálva, a fallback ágat kell használni, és a hiányt dokumentálni kell.

40.1.7 Gyakori hibák

- A `__cplusplus == 202302L` értéket önmagában teljes C++23-támogatásnak tekinteni. Ez csak a nyelvi módot jelzi.
- `__has_include(<expected>)`-et feature-test helyett használni. A header létezhet úgy is, hogy a képesség nem teljes vagy nem engedélyezett.
- A feature-test makrót `#ifdef`-fel ellenőrizni a küszöbérték helyett. Egy korábbi, részleges érték is definiálhatja.
- Összekeverni a fordító és a standard könyvtár támogatását. A „Clang támogatja” mondat önmagában nem elég; a könyvtár-párosítás is számít.
- A C++23-as buildet csak egy compilerrel és egy standard könyvtárral futtatni, majd a zöld CI-ből általános támogatásra következtetni.
- A `-lstdc++exp` vagy más linkelési kapcsoló hiányát nyelvi hibaként diagnosztizálni. Előbb a linkerüzenetet és a könyvtár dokumentációját kell megnézni.
- A N4950 munkatervezetből kiadott szabványként idézni, vagy a munkatervezet és a kiadott szöveg eltérését figyelmen kívül hagyni.
- Kísérleti könyvtári elemet (`std::experimental`, külön header) a C++23 standard részének beállítani.

40.1.8 Összefoglaló

A C++23 laborprofil háromrétegű: kiadott szabvány, nyilvános munkatervezet és tényleges implementáció. A `-std=c++23` és a `__cplusplus == 202302L` a nyelvi módot azonosítja, de nem bizonyítja a könyvtári képességeket. A nyelvi `__cpp_*` és könyvtári `__cpp_lib_*` feature-test makrókat a `<version>` headerrel együtt kell ellenőrizni, lehetőleg küszöbértékkel. A támogatási állapotot compiler, standard könyvtár, target, linkelési kapcsolók és dátum szerint kell rögzíteni. A kétprofilos C++20/C++23 CMake és CI mátrix biztosítja, hogy a fallback is működjön, és a C++23-as ág ne csendben, ellenőrizetlenül kerüljön be a buildbe.

40.1.9 Feladatok

1. **Státuszdiagnózis.** Egy laborban a `__cplusplus` értéke `202302L`, a `<version>` includeolható, de a `__cpp_lib_expected` nem definiált. Mit jelent ez, és milyen további ellenőrzéseket végzel? Írj rövid döntési fát.
2. **Feature-test és fallback.** Egészítsd ki a fejezet `Probe` példáját egy `static operator()`-t használó képességellenőrzéssel úgy, hogy C++20-ban is forduljon, és a fallback viselkedése megmaradjon. Add meg a buildparancsot is.
3. **Kétprofilos CMake.** Készíts olyan CMake-beállítást, amely ugyanazt a forrást C++20 és C++23 módban is lefordítja, és mindkét profilban futtat egy füsttesztet. Fogalmazd meg, mit jelent a CI-ben, ha a C++23-as profil elbukik.
4. **Linkelési hiba.** A C++23-as build a nyelvi szintaxist elfogadja, de a linker egy standard könyvtári szimbólumra hivatkozó hibát ad. Milyen sorrendben vizsgálódsz, és milyen bizonyítékot rögzítesz a laborprofilban?

40.1.9.1 Megoldások

1. Megoldás. A `__cplusplus == 202302L` azt valószínűsíti, hogy a nyelvi mód C++23. A hiányzó `__cpp_lib_expected` azt jelzi, hogy a vizsgált standard könyvtár nem állítja a `std::expected` adott változatának elérhetőségét. Ez nem feltétlenül azt jelenti, hogy a header egyáltalán nem létezik; lehet, hogy a makró hiányzik, a header kísérleti, vagy a könyvtár verziója eltér. Ellenőrzési sorrend: `<version>` include; `__cpp_lib_expected` és küszöb; `<expected>` include próba; egyszerű `std::expected` használat fordítása; linkelés; futásidejű smoke test. Ha bármelyik szint hiányzik, a `std::expected`-et használó kódot fallback mögé kell tenni, és a laborprofilban „részleges” állapotként kell rögzíteni. Ellenőrzési feltétel: a fallback C++20-ban és a C++23-as részleges profilban is fordul és fut.

2. Megoldás. Egy lehetséges kiegészítés:

```
#include <version>
#include <cstdio>

struct Functor {
#if defined(__cpp_static_call_operator) && __cpp_static_call_operator >= 202207L
    static int operator()(int x) { return x + 1; }
#else
    int operator()(int x) const { return x + 1; }
#endif
};
```

```
int main() {
    std::printf("%d\n", Functor{}(41));
}
```

A C++23-as ágban a `static operator()` elérhető, a fallback ágban nem statikus tagfüggvény marad. A hívás `Functor{}(41)` mindkét ágban működik. Build:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic static_call.cpp -o static_call20
g++ -std=c++23 -Wall -Wextra -Wpedantic static_call.cpp -o static_call23
```

Indoklás: a feature-test makró a fordító állítását ellenőrzi; a fallback biztosítja, hogy a C++20 baseline ne törjön el. Ellenőrzési feltétel: mindkét build sikeres, és a futtatás `42`-t ad. A küszöbértéket a laborban a tényleges `<version>` alapján kell megerősíteni.

3. Megoldás. A CMake-példa már mutatja a `PROBE_CXX_STANDARD` cache változót és a `ctest` futtatást. A CI-ben két külön build és teszt fut:

```
cmake -S . -B build20 -DPROBE_CXX_STANDARD=20
cmake --build build20
ctest --test-dir build20 --output-on-failure

cmake -S . -B build23 -DPROBE_CXX_STANDARD=23
cmake --build build23
ctest --test-dir build23 --output-on-failure
```

Ha a C++23-as profil elbukik, a helyes következtetés nem az, hogy „a C++23 nem támogatott általában”, hanem hogy a rögzített compiler/stdlib/target kombinációban a vizsgált képesség nem használható. A C++20 baseline-nak továbbra is zöldnek kell lennie; a C++23-as bukást a laborprofilban a konkrét hibaüzenettel, a feature-test állapottal és a dátummal kell rögzíteni. Ellenőrzési feltétel: a C++20 build és teszt sikeres, a C++23 hiba pedig reprodukálható és dokumentált.

4. Megoldás. Először a linkerüzenetet kell elolvasni: hiányzó szimbólum, hiányzó könyvtár vagy nem megfelelő könyvtárverzió szerepel-e benne. Ezután ellenőrizni kell a compiler és a standard könyvtár párosítását, a könyvtár dokumentált linkelési igényét, például a `-lstdc++exp` szükségességét, majd a buildparancsot. Ha a kapcsoló dokumentált és elérhető, a linkelési próbát meg kell ismételni; ha nem, a képességet fallback mögé kell tenni. A laborprofilban rögzíteni kell a fordító verzióját, a standard könyvtár verzióját, a targetet, a linkelési kapcsolót, a linker hibaüzenetét és a fallback működését. Ellenőrzési feltétel: a hiba vagy dokumentált kapcsolóval megszűnik, vagy a fallback build és futás igazoltan működik.

40.2 Explicit object parameter: deducing this

A C++20-as örökség egyik leggyakoribb tervezési terhe a tagfüggvények objektumhoz való viszonyának sokféle kódolása. Egy egyszerű getterhez négy változat kell, ha meg akarjuk őrizni a `const`-ságot és az értékkategóriát; a statikus polimorfizmus CRTP-vel gyakran `static_cast`-olással és sablonparaméterként átadott derived típussal jár; a rekurzív lambda pedig C++20-ban csak körülményesen, `std::function`-nel vagy külső segédstruktúrával írható meg. A C++23 explicit object parameter — közismertebb nevén *deducing this* — ezekre ad nyelvi szintű, egységes eszközt. Nem új futásidejű mechanizmus: a tagfüggvény objektumparaméterét teszi láthatóvá és dedukálhatóvá. A mostani rész ezt a nyelvi eszközt tárgyalja; a C++23 laborprofil és a feature-testek általános kezelése a C++23 laborprofil, szabványstátusz és feature-test részben már rögzített alapra épül.

40.2.1 A probléma: overload-négyes, CRTP és rekurzió

A klasszikus C++20-as overload-négyes tipikus példája egy konténer eleméhez való hozzáférés:

```
struct Param {
    int value;
    int& get() & { return value; }
    const int& get() const& { return value; }
    int&& get() && { return std::move(value); }
    const int&& get() const&& { return std::move(value); }
};
```

Négy deklaráció, négy visszatérési típus, és a négy változat közötti különbség csak a függvény minősítőiben van. Ugyanez a CRTP-ben:

```
template<class D>
struct Shape {
    double area() const { return static_cast<const D*>(this)->area_impl(); }
};

struct Circle : Shape<Circle> {
    double r{1.0};
    double area_impl() const { return 3.141592653589793 * r * r; }
};
```

A `Shape` csak a `D` sablonparaméteren keresztül tudja, hogy valójában `Circle`-ról van szó; a `static_cast` a derived típusra mutat. A rekurzív lambda C++20-ban jellemzően így néz ki:

```
#include <functional>

std::function<int(int)> fact = [&](int n) {
    return n <= 1 ? 1 : n * fact(n - 1);
};
```

Ez működik, de típus-erasure-t és indirekciót hoz be, ráadásul a `fact` élettartamához kötött. Az explicit object parameter mindhárom mintát egyszerűsíti, de nem mindenható: a virtuális dispatch, a statikus tagfüggvény, a konstruktor/destruktor és a hagyományos tagfüggvény-pointer szemantikája továbbra is határt szab.

40.2.2 Definíció és szintaxis

Az **explicit object parameter** olyan tagfüggvény-paraméter, amelyet a `this` specifier előz meg, és amely a paraméterlistában az első helyen áll. A függvény tagfüggvény marad, de nincs implicit `this` paramétere: az objektumot a hívás az első, explicit paraméteren keresztül adja át. A *deducing this* elnevezés arra utal, hogy ennek a paraméternek a típusát — jellemzően egy `Self` sablonparamétert — a hívás objektumargumentumából dedukáljuk.

A leggyakoribb alak:

```
struct Widget {
    template<class Self>
    void f(this Self&& self) {
        // self a hívó objektum; a this kulcsszó nem használható
```

```
    }
};
```

A `this auto&& self` ugyanezt jelenti egy implicit, a fordító által bevezetett sablonparaméterrel. A paraméter lehet érték szerinti is:

```
struct Point {
    int x, y;
    int sum(this Point self) { return self.x + self.y; }
};
```

Ekkor a hívás objektumból másolat készül; a függvény a saját példányát módosítja, az eredetit nem. Kis, olcsón másolható típusoknál ez kényelmes, nagy típusnál viszont kerülendő. A `this Self& self` csak lvalue objektumot fogad el, a `this Self const& self` pedig const hozzáférést ad; a teljes értékkategória- és const-propagáláshoz a `this Self&& self` a természetes választás.

A fontos feltételek:

- Az explicit object parameter csak nem statikus tagfüggvényen használható, és a paraméterlistában az első helyen kell állnia.
- Nem lehet `virtual`, nem lehet `static`, és nem lehet konstruktor vagy destruktor.
- A függvényre nem írható `const`, `volatile`, `&` vagy `&&` minősítő; ezeket a `Self` típusa hordozza. A `void f(this Self&& self) const` alak ezért hibás.
- A függvény törzsében nincs implicit `this`; a `this` kulcsszó nem használható, helyette a paraméter nevével hivatkozunk az objektumra.
- Lambda `operator()`-ában is megengedett; ez teszi lehetővé a rekurzív lambdát.
- A függvény továbbra is tagfüggvény, de az objektumargumentum a többi paraméterhez hasonlóan részt vesz a túlterhelés-feloldásban.

40.2.2.1 Levezetés (vázlat)

A `this Self&& self` paraméter dedukciója ugyanaz, mint egy forwarding reference-é. Ha a hívó objektum típusa `C`:

Hívás objektuma	`Self`	`self` deklarált típusa
`C` lvalue	`C&`	`C&`
`const C` lvalue	`const C&`	`const C&`
`C` rvalue	`C`	`C&&`
`const C` rvalue	`const C`	`const C&&`

A referencia-összevonás adja a második oszlopból a harmadikat: `Self = C&` esetén `C& &&` → `C&`; `Self = C` esetén `C&&`. A `const` a `Self` típusába kerül, nem a függvényre. Fontos következmény, hogy a függvényen belül a `self` név mindig lvalue kifejezés, akármilyen értékkategóriával érkezett az objektum; ezért kell a továbbításhoz `std::forward<Self>(self)` vagy a tagokra `std::forward_like<Self>(...)`.

40.2.3 Overload-négyes kiváltása `forward_like`-kal

A négyes egyetlen függvénysablonnal kiváltható:

```
#include <cstddef>
#include <type_traits>
#include <utility>
#include <vector>

struct Buffer {
    std::vector<int> data;

    template<class Self>
    auto&& at(this Self&& self, std::size_t i) {
        return std::forward_like<Self>(self.data[i]);
    }
};
```

A `std::forward_like<T>(x)` a `T`-ből származtatja azt a const- és értékkategória-információt, amelyet az `x`-re alkalmazni kell. Itt `Self` hordozza az objektum `const`-ságát és értékkategóriáját, miközben `self.data[i]` önmagában lvalue `int` kifejezés. A `forward_like` ezt alakítja `int&`, `const int&`, `int&&` vagy `const int&&` típusúvá. A `std::forward<Self>(self).data[i]` ezzel szemben nem elegendő: a `std::vector::operator[]` nincs ref-minősítve, ezért rvalue vektoron is lvalue referenciát ad vissza; a value category nem terjed át a subscripten. Ez tipikus diagnosztikai hiba, amikor valaki a négyesből csak a const/non-const párt teszteli, az rvalue eset pedig csendben lvalue referenciát ad.

A teljes példa fordítása a laborban rögzített fordítókiadással:

```
g++ -std=c++23 -Wall -Wextra -pedantic-errors -O2 deducing_this.cpp -o
 ↪  deducing_this
clang++ -std=c++23 -Wall -Wextra -pedantic-errors -O2 deducing_this.cpp -o
 ↪  deducing_this
```

A támogatás tényét nem feltételezzük; feature-testtel és compile probe-bal ellenőrizzük:

```
#if !defined(__cpp_explicit_this_parameter) || __cpp_explicit_this_parameter <
 ↪  202110L
#error "A forditonak C++23 explicit object parametert kell tamogatnia."
#endif

#if !defined(__cpp_lib_forward_like) || __cpp_lib_forward_like < 202207L
#error "A standard konyvtarnak std::forward_like-ot kell nyujtania."
#endif
```

A nyelvi és a könyvtári támogatás külön ellenőrizendő: a `__cpp_explicit_this_parameter` a nyelvi képességet, a `__cpp_lib_forward_like` a könyvtári eszközt jelzi. Ha az utóbbi hiányzik, C++20-as fallbackként a négy overload, vagy kézi `static_cast`-olás a `Self` alapján használható; a `std::forward` önmagában nem elég a fenti `operator[]`-probléma miatt.

Élettartam szempontjából a `forward_like` nem varázslat: ha a getter `auto&&`-t ad vissza, és az objektum temporális, a visszaadott referencia a temporális tagjára mutat, amely a teljes kifejezés végén megszűnik. Ez ugyanaz a dangling-helyzet, mint a ref-minősített gettereknél; a részletes értékkategória- és temporary-szabályokat a Temporary, value category és dangling rész tárgyalta. Ha a hívó nem tudja garantálni az owner élettartamát, adjunk vissza értéket, ne referenciát.

40.2.4 CRTP egyszerűsítése és a statikus polimorfizmus korlátai

A CRTP-s `Shape`/`Circle` példa explicit object parameterrel így egyszerűsödik:

```
struct Shape {
    template<class Self>
    double area(this Self const& self) {
        static_assert(requires { self.area_impl(); },
                      "A Self tipusnak area_impl() tagfuggvenyt kell adnia");
        return self.area_impl();
    }
};

struct Circle : Shape {
    double r{1.0};
    double area_impl() const { return 3.141592653589793 * r * r; }
};
```

Eltűnt a `D` sablonparaméter, a `static_cast`, és a derived osztálynak nem kell önmagát átadnia a base-nek. A `Self` a hívás statikus típusából dedukálódik, így a `Circle c; c.area();` hívásban `Self = const Circle&` (a `const&` paraméter miatt), és a `self.area_impl()` megtalálható. A `requires`-szel vagy `static_assert`-tel adott követelmény azonban csak a függvény hívásakor, az adott `Self`-re ellenőrződik, nem a `Circle` osztály definiálásakor.

Ez a fő különbség a CRTP-hez képest. A CRTP base már a `Shape<Circle>` példányosításakor ismeri a `Circle`-t, ezért ott class-scope-ban is lehet `static_assert`-et, typedefet vagy egyéb, a derived típusra épülő deklarációt elhelyezni. A deducing this nem tud tagokat „injektálni” a derived osztályba, és nem tud a base-ben class-scope-ban a derived típusra hivatkozni. Ha a statikus polimorfizmus részeként a base referencia mögül is a derived függvényét akarjuk hívni, a deducing this nem elegendő:

```
Circle c;
Shape const& s = c;
// s.area(); // Self = Shape, a Shape-nek nincs area_impl() tagja
```

A CRTP-s `Shape<Circle> const&` ezzel szemben tudja a `Circle`-t. A deducing this tehát kiváltja a CRTP gyakori `static_cast`-os mintáját, de nem váltja ki a CRTP teljes statikus polimorfizmusát. Marad továbbá a virtuális dispatch hiánya: nincs vtable, nincs runtime polymorphic interfész, és a `Self` mindig a kifejezés statikus típusa szerint dedukálódik. A CRTP és a policy design korábbi tárgyalása a Customization point, CRTP és policy design részben található.

40.2.5 Rekurzív lambda explicit object parameterrel

A lambda `operator()`-a is tagfüggvény, ezért kaphat explicit object parametert. Ezzel a closure saját magát névvel érheti el:

```
#include <cassert>

int main() {
    auto fact = [](this auto&& self, int n) -> int {
        return n <= 1 ? 1 : n * self(n - 1);
    };
```

```
    assert(fact(5) == 120);
}
```

A `fact(5)` hívásban a `fact` closure az explicit object paraméter, az `5` pedig az `n`. A `self` a closure objektumra kötődik, így `self(n - 1)` újra a lambda `operator()`-át hívja. Nem kell `std::function`, nem kell a lambdát capture-rel önmagára hivatkoztatni, és a rekurzió típusa a closure típusa marad. A `this auto self` érték szerinti változat is működik, de a closure-t minden rekurzív hívásnál másolja; a `this auto&&` ezt elkerüli. A closure élettartamára ugyanazok a szabályok vonatkoznak, mint bármely lambdára: ha a closure-t visszaadjuk vagy eltároljuk, a capture-ök élettartamát továbbra is biztosítani kell; erről a Lambda, capture és closure lifetime rész szól.

40.2.6 Korlátok és döntési tábla

Kérdés	Explicit object parameterrel
`virtual` tagfüggvény	nem megengedett
`static` tagfüggvény	nem megengedett
konstruktor/destruktor	nem megengedett
`const`/`volatile`/`&`/`&&` a függvényen	nem írható; a `Self` hordozza
implicit `this`	nincs; a `this` kulcsszó nem használható
lambda `operator()`	megengedett
érték szerinti `self`	megengedett, másol/mozgat; nagy típusnál kerülendő
tagfüggvény-pointer (`&C::f`)	közönséges függvénypointer: `int (*)(C&, int)`, nem `int (C::*)(int)`
C++20-as fallback	négy overload, CRTP, `std::function`/Y-combinator

A tagfüggvény-pointer kérdését a szabvány egyértelműen rendezi ([expr.unary.op]): egy explicit object parameteres tagfüggvény címe **közönséges függvénypointer**, amelynek első paramétere az objektumparaméter; `int C::f(this C&, int)` esetén `&C::f` típusa `int (*)(C&, int)`, és a hívás `pf(c, 1)` alakú, nem `(c.*pf)(1)`.

```
// explicit_object_ptr.cpp - C++23: az explicit object tagfüggvény címe
#include <cstdio>
#include <type_traits>

struct C {
    int base = 10;
    int f(this C& self, int x) { return self.base + x; }   // explicit object
    int g(int x) { return base + x; }                        // implicit this
};

static_assert(std::is_same_v<decltype(&C::f), int (*)(C&, int)>);  // közönséges
 ↪ fv-pointer
static_assert(std::is_same_v<decltype(&C::g), int (C::*)(int)>);   //
 ↪ tagfüggvény-pointer

int main() {
    C c;
```

```
    int (*pf)(C&, int) = &C::f;
    int (C::*pg)(int) = &C::g;
    std::printf("%d %d\n", pf(c, 1), (c.*pg)(2));          // 11 12
}
```

Buildjegyzőkönyv. A GCC 13.3 a deducing this-t még nem támogatja (`expected identifier before 'this'`); a Clang 22 `-std=c++23 -Wall -Wextra -Wpedantic` alatt figyelmeztetés nélkül fordította, mindkét `static_assert` teljesült, a kimenet `11 12`.

Az ABI-stabilitás ettől külön kérdés. Shared library- vagy FFI-határon különösen nem tanácsos ilyen függvény címét stabil ABI-elemként exportálni. A C interfészhez továbbra is `extern "C"`-s szabad függvény vagy opaque handle való; erről a C++ ABI, C interfész és FFI rész szól.

Mikor nem alkalmazzuk? Ha a kódnak C++20-on is fordulnia kell; ha valódi virtuális interfészre van szükség; ha a CRTP class-scope-s követelményeit vagy taginjekcióját akarjuk; ha a stabil bináris ABI a cél; vagy ha a meglévő kódban az implicit `this`-re épülő makrók, tagpointerek vannak. Az explicit object parameter erős eszköz, de nem helyettesíti a virtuális polimorfizmust és nem tesz hordozhatóvá egy ABI-határt.

40.2.7 Gyakori hibák

- **A `const` a paraméterlista után:** `void f(this Self&& self) const` hibás. A constot a `Self`-ben kell kifejezni, például `this Self const& self` vagy `this Self&& self` mellett a hívó const objektummal.
- **A `this` használata a törzsben:** nincs implicit `this`; a paraméter nevével kell hivatkozni.
- **`std::move(self)`:** mindig rvalue-re kényszerít, és const objektumnál nem is működik úgy, ahogy várnánk. Helyette `std::forward<Self>(self)` vagy tagokra `std::forward_like<Self>(...)`.
- **`std::forward<Self>(self).data[i]` a négyes kiváltására:** a `vector::operator[]` nem ref-minősített, ezért rvalue objektumon is lvalue referenciát ad; a value category nem terjed át. Ilyenkor `forward_like` kell.
- **Érték szerinti `self` nagy típuson:** másolás/mozgatás történik; nem tulajdonosi szemlélet helyett gyakran forwarding reference a helyes választás.
- **CRTP-től várt taginjekció:** a deducing this nem tud a derived osztályba tagot vagy typedefet tenni, és class-scope-ban nem tud a derived típusra hivatkozni.
- **Virtualitás várása:** az explicit object parameteres függvény nem lehet `virtual`; runtime polimorfizmushoz továbbra is virtuális függvény vagy type erasure kell.
- **Dangling rvalue getter:** `auto&&`-t visszaadó getter temporális objektumon a temporális tagjára mutató referenciát ad; a hívó élettartama nem hosszabbodik meg.

40.2.8 Összefoglaló

Az explicit object parameter a tagfüggvény objektumparaméterét a paraméterlistára hozza, és ezzel dedukálhatóvá teszi a `Self` típust. Egy `template<class Self> auto&& f(this Self&& self)` alakkal a const/non-const és lvalue/rvalue overload-négyes egyetlen sablonná válik; a tagok továbbításához `std::forward_like<Self>` használható, amely a `Self`-ből viszi át a const- és értékkategória-információt. A CRTP gyakori `static_cast`-ja eltűnik, de a class-scope-s követelmények, a taginjekció és a base referencián keresztüli statikus polimorfizmus nem váltható ki vele. Rekurzív lambdát tisztán, `std::function` nélkül írhatunk. A korlátok élesek: nincs `virtual`, `static`, konstruktor/destruktor, nincs implicit `this`, a függvényre nem írható cv/ref minősítő, a tagfüggvény-pointer és az ABI pedig külön ellenőrizendő. A C++20-as fallback továbbra is a négy overload, a CRTP és a `std::function`/Y-combinator.

40.2.9 Feladatok

40.2.9.1 1. feladat: overload-négyes kiváltása

Adott a következő C++20-as típus:

```
struct Box {
    int value;
    int& get() & { return value; }
    const int& get() const& { return value; }
    int&& get() && { return std::move(value); }
    const int&& get() const&& { return std::move(value); }
};
```

Írd át egyetlen explicit object parameteres függvénysablonra, amely mind a négy esetben a megfelelő referenciatípust adja vissza. Ellenőrizd `static_assert`-tel legalább a nem const lvalue, a const lvalue és a nem const rvalue esetet. Írd le, miért nem elegendő a `std::forward<Self>(self).value` a tag eléréséhez, ha a tag egy `std::vector` eleme.

40.2.9.2 2. feladat: diagnosztika

Az alábbi kódot egy kolléga azzal a céllal írta, hogy a `Buffer::at` a négyes overloadot váltsa ki:

```
struct Buffer {
    std::vector<int> data;
    template<class Self>
    auto&& at(this Self&& self, std::size_t i) {
        return std::forward<Self>(self).data[i];
    }
};
```

Mondd meg, fordul-e, és ha igen, miért nem adja vissza az rvalue esetben `int&&`-t. Javítsd a kódot. Milyen futásidejű veszélyt jelent, ha a javított gettert temporális `Buffer`-en hívjuk meg és a visszaadott referenciát eltároljuk?

40.2.9.3 3. feladat: CRTP kiváltása

Adott a CRTP-s alak:

```
template<class D>
struct Base {
    void interface() const { static_cast<const D*>(this)->impl(); }
};
struct Derived : Base<Derived> {
    void impl() const {}
};
```

Írd át explicit object parameterrel. Ezután válaszolj: mi történik, ha `Base const& b = d; b.interface();` formában hívjuk meg? Miért viselkedik másképp a CRTP-s változat? Sorolj fel legalább két olyan feladatot, amelyre a CRTP továbbra is jobb választás.

40.2.10 Megoldások

40.2.10.1 1. feladat megoldása

```
#include <type_traits>
#include <utility>

struct Box {
    int value;

    template<class Self>
    auto&& get(this Self&& self) {
        return std::forward_like<Self>(self.value);
    }
};

int main() {
    Box b{1};
    const Box cb{2};
    static_assert(std::is_same_v<decltype(b.get()), int&>);
    static_assert(std::is_same_v<decltype(cb.get()), const int&>);
    static_assert(std::is_same_v<decltype(Box{3}.get()), int&&>);
    static_assert(std::is_same_v<decltype(std::move(cb).get()), const int&&>);
}
```

A `Self` a hívó objektum const- és értékkategóriáját hordozza; a `forward_like` ezt viszi át a `value` tagra. A `std::forward<Self>(self).value` közvetlen tagelérésnél működne, mert a tagelérés megőrzi az értékkategóriát, de egy `std::vector operator[]`-je nem ref-minősített: rvalue vektoron is `int&`-t ad, ezért ott a `forward_like` szükséges. Ellenőrzési feltétel: a négy `static_assert` forduljon le, és a const rvalue esetben az eredmény `const int&&` legyen.

40.2.10.2 2. feladat megoldása

A kód lefordul, de a visszatérési típus rvalue `Buffer` esetén is `int&` lesz. Az ok a

```
std::forward<Self>(self).data[i]
```

kifejezésben van: a `data[i]` hívás a nem ref-minősített `std::vector::operator[]`-t használja, amely lvalue referenciát ad. A value category nem „szivárog át” a subscripten. Javítás:

```
template<class Self>
auto&& at(this Self&& self, std::size_t i) {
    return std::forward_like<Self>(self.data[i]);
}
```

A `forward_like` a `Self` alapján castolja az `int&` kifejezést `int&&`-re, illetve const esetben `const int&&`-re. Ha a gettert temporális `Buffer`-en hívjuk, és a visszaadott referenciát eltároljuk, a temporális a teljes kifejezés végén megszűnik; a referencia dangling lesz, és a későbbi használat nem definiált viselkedés. Ellenőrzési feltétel: `static_assert(std::is_same_v<decltype(Buffer{{1}}.at(0)), int&&>)` teljesüljön, és a temporálisra visszaadott referenciát ne tároljuk el.

40.2.10.3 3. feladat megoldása

```
struct Base {
    template<class Self>
    void interface(this Self const& self) {
        self.impl();
    }
};

struct Derived : Base {
    void impl() const {}
};
```

A `Derived d; d.interface();` hívásban `Self = Derived`, és a `self.impl()` megtalálható. A `Base const& b = d; b.interface();` hívásban viszont a statikus típus `Base`, ezért `Self = Base`, és a `Base`-nek nincs `impl()` tagja; a `requires`/`static_assert` itt hibát jelez. A CRTP-s `Base<Derived>` azért működik, mert a base már a példányosításkor ismeri a `Derived` típust, és a `static_cast` a dinamikus objektumra mutat. A deducing this a statikus típusból dedukál, nem a dinamikusból.

A CRTP továbbra is jobb, ha (1) a base-nek class-scope-ban kell a derived típusra hivatkoznia, typedefet/tagot kell örökítenie vagy példányosításkor kell követelményt ellenőriznie; (2) a base referencia mögül is a derived statikus interfészét akarjuk hívni; (3) olyan statikus polimorf kódot írunk, ahol a derived típus a base típusparamétere, és erre a template-metaprogramozásban is szükség van. Ellenőrzési feltétel: a `Derived d; d.interface();` hívás forduljon, a `Base const&`-es hívás pedig adjon érthető diagnosztikát, ne csendben a `Base`-re dedukáljon.

40.3 static operator(), többdimenziós indexelés és lambdaváltozások

A C++20-ban egy állapot nélküli függvényobjektumot (funktort) csak körülményesen lehetett igazán „súlytalanná” tenni: a tagfüggvénynek mindig van egy rejtett `this` paramétere, még akkor is, ha az objektumnak nincs állapota. Ugyanígy a többdimenziós indexelést `operator()(i, j)` vagy `at(i, j)` formában kellett megoldani, mert az `operator[]` csak egy paramétert fogadott el. A C++23 nyelvi változásai közül ebben a részben a hívó- és indexelő operátorok bővítését, valamint a lambdák ehhez kapcsolódó finomításait tárgyaljuk. A cél nem a C++20-as operátor- és lambdaalapok újratanítása — azokat lásd az Operátortúlterhelés és konverziós operátor (23.3) és a Lambda, capture és closure lifetime (30.1) részekben —, hanem annak megértése, mit változtat a C++23, milyen szerződéseket kell ilyenkor kézben tartani, és hogyan lehet a képességet feature-test makróval és compile probe-bal ellenőrizni.

40.3.1 A statikus hívóoperátor

40.3.1.1 Definíció és feltételek

A **statikus hívóoperátor** (`static operator()`) C++23-as nyelvi képesség: egy osztály tagfüggvényeként definiált `operator()` lehet `static`. A pontos feltételek a következők. A függvénynek tagfüggvénynek kell lennie, és a `static` kulcsszót a deklarációban kell szerepeltetni. Statikus tagfüggvényként nem lehet `const`, `volatile`, `&` vagy `&&` minősítésű, mert ezek a minősítők a rejtett `this` paraméterre vonatkoznak, amely itt nem létezik. Nem lehet `virtual` sem, mert a virtuális dispatchhez objektumpéldány kell. Lehet viszont `constexpr`, `consteval` és `noexcept`, és a hívás szintaxisa a hívó oldalon változatlan: `f(args)`.

A képesség a P1169 paperszámhoz kötődik a témakör tervében. A támogatás tényleges meglétét nem szabad feltételezni: a laborprofilban rögzített GCC vagy Clang verzióval feature-test makróval és egy kis compile probe-bal kell ellenőrizni. A makró neve `__cpp_static_call_operator`, a C++23-as érték `202207L`. A vizsgálat tipikus alakja:

```
#if defined(__cpp_static_call_operator) && __cpp_static_call_operator >= 202207L
#  define CPP23_STATIC_CALL_OP 1
#else
#  define CPP23_STATIC_CALL_OP 0
#endif
```

A legfontosabb gyakorlati következmény a **capture nélküli lambda** esetére vonatkozik. A C++23 szerint az ilyen lambda closure típusának `operator()`-je implicit módon `static`: a closure üres típus, a hívás nem igényel `this` paramétert. Ezt a felhasználó nem írja ki, és a `mutable`-lel nem kombinálhatja, mert a `mutable` éppen a nem-const `this`-t jelentené. A `static` és a `mutable` tehát kölcsönösen kizárja egymást.

40.3.1.2 Hívási költség és ABI

A `this` paraméter elhagyásának van hívási konvencióbeli hatása. Az x86-64 System V ABI-ban egy nem-static tagfüggvény első integer paramétere a `this` (jellemzően az `RDI` regiszterben), a többi paraméter pedig eggyel eltolódik. Egy statikus `operator()` esetén ez a plusz argumentum eltűnik, ami regisztert takaríthat meg, és egyszerűbbé teheti a függvénypointeres átadást. Fontos azonban, hogy ez elvi, nem mért hatás: a fordító az inline-olt, optimalizált kódban gyakran teljesen eltünteti a különbséget. Mérés nélkül nem állítható, hogy a statikus változat gyorsabb; a laborban külön kell megmérni, és a mérési eredményt a fordítóverzióhoz és az optimalizálási szinthez kell kötni.

ABI-szempontból a statikus és a nem-static `operator()` mangled neve eltér, és a hívási konvenció is más. Ha egy headerben egy meglévő `operator()`-t `static`-ká teszünk, a bináris interfész megváltozik: a régi és az új könyvtár nem lesz link-kompatibilis. Ez DevOps-környezetben különösen fontos, ha a függvény egy megosztott könyvtár határán van. Új kódban a statikus változat bevezetése nem jelent ABI-törést, de a meglévő API-t nem szabad csendben átminősíteni.

40.3.1.3 Lambdák C++23-ban

A lambdákra vonatkozó C++23-as változások közül kettő kapcsolódik ide. Az első a capture nélküli lambda implicit statikus `operator()`-je, amelyet az előző pontban tárgyaltunk. A második a **lambda-deklarátor** egyszerűsítése: a paraméterlista elhagyható, ha a lambda nem vesz át paramétert, és nincs olyan specifikátor, amely a paraméterlistát igényelné. A `[]{}` és a `[](){}` alak tehát egyenértékű, de a `[] mutable {}` nem érvényes, mert a `mutable` a paraméterlistához kötött. Hasonlóképpen a `noexcept`, a `constexpr`, a `consteval` és a `static` specifikátorok bizonyos kombinációi igénylik a paraméterlista jelenlétét.

A lambda-attribútumok terén a C++23 lehetővé teszi, hogy attribútumok a lambda-introducer után is megjelenjenek, például `[] [[nodiscard]] () { return 1; }` alakban. A pontos elfogadott szintaxist és a paraméterlista elhagyhatóságának eseteit azonban nem szabad fejből feltételezni: ezek verzió- és compilerfüggő részletek, amelyeket compile probe-bal kell ellenőrizni. A `__cpp_lambdas` makró C++23-as értéke a rendelkezésre álló jegyzet szerint `202207L`, de ez is ellenőrizendő.

40.3.2 Többdimenziós indexelés

40.3.2.1 A vesszőoperátor öröksége

A **többdimenziós indexelő operátor** (P2128) lehetővé teszi, hogy az `operator[]` tagfüggvénynek egynél több paramétere legyen. A szintaxis változatlanul `a[i, j]`, de a vessző jelentése megváltozik: ha az osztálynak van többparaméteres `operator[]`-ja, akkor a vessző paraméterelválasztó, nem vesszőoperátor. Ha nincs ilyen, akkor a C++20-ig megszokott módon a vesszőoperátor lép életbe.

Ez a különbség könnyen félreértéshez vezet. A beépített tömbindexelésnél az `operator[]` továbbra is egyoperandusú, ezért

```
int a[10];
a[1, 2] = 5;   // a[2] = 5: a vesszőoperátor kiértékeli 1-et, eldobja, majd 2-t
 ↪  ad
```

C++23-ban is ezt jelenti, mert a beépített `[]` nem többparaméteres. Saját osztálynál viszont a `m[1, 2]` attól függ, van-e többparaméteres `operator[]`. Ha van, két paraméteres hívás; ha nincs, a vesszőoperátor eredménye kerül az egyparaméteres `[]`-ba. Ha az osztálynak nincs egyparaméteres `operator[]`-ja, a `m[1, 2]` fordítási hiba lesz. A vesszőoperátor túlterhelése ezen nem változtat: a beépített `[]` esetén továbbra is a `operator,` hívódik, ha azt az osztály túlterhelte.

40.3.2.2 Mátrixtípus operator indexeléssel

A következő példa egy egyszerű, sorfolytonos mátrixtípust mutat, amely C++23-ban `m[i, j]` alakban indexelhető. A kód önállóan fordítható, de a C++23-as `operator[]` miatt a laborprofilban rögzített, a képességet támogató compilerverzió szükséges hozzá.

```
// matrix.hpp - C++23
#pragma once
#include <cstddef>
#include <limits>
#include <stdexcept>
#include <vector>

class Matrix {
public:
    Matrix(std::size_t rows, std::size_t cols)
        : rows_(rows), cols_(cols) {
        if (cols_ != 0 && rows_ > std::numeric_limits<std::size_t>::max() /
         ↪  cols_) {
            throw std::length_error("Matrix: meret tulcsordulas");
        }
        data_.resize(rows_ * cols_);
    }

    int& operator[](std::size_t i, std::size_t j) noexcept {
        return data_[i * cols_ + j];
    }

    int operator[](std::size_t i, std::size_t j) const noexcept {
        return data_[i * cols_ + j];
```

```
    }

    int& at(std::size_t i, std::size_t j) {
        if (i >= rows_ || j >= cols_) {
            throw std::out_of_range("Matrix: index hataron kivul");
        }
        return (*this)[i, j];
    }

    int at(std::size_t i, std::size_t j) const {
        if (i >= rows_ || j >= cols_) {
            throw std::out_of_range("Matrix: index hataron kivul");
        }
        return (*this)[i, j];
    }

    std::size_t rows() const noexcept { return rows_; }
    std::size_t cols() const noexcept { return cols_; }

private:
    std::size_t rows_;
    std::size_t cols_;
    std::vector<int> data_;
};
```

A `(*this)[i, j]` itt már a többparaméteres `operator[]` hívása. A `const` változat értéket ad vissza, nem referenciát; ezzel elkerüljük, hogy egy temporary `Matrix`-on kapott referenciát a hívó tovább éltessen. A nem-const változat `int&`-t ad, ami a `data_` elemére hivatkozik, és a `Matrix` élettartamához kötött.

A build parancs a laborprofilban rögzített compilerrel:

```
g++ -std=c++23 -O2 -Wall -Wextra -Wpedantic matrix_test.cpp -o matrix_test
clang++ -std=c++23 -O2 -Wall -Wextra -Wpedantic matrix_test.cpp -o matrix_test
```

Buildjegyzőkönyv (`lab-gcc` profil): a `matrix_test.cpp` (a lenti, bounds-szerződést ellenőrző teszt a `matrix.hpp`-vel) a GCC 13.3-mal figyelmeztetés nélkül fordult és 0 kóddal futott; a Clang 22 is lefordította és futtatta. Más compilernél a támogatást a feature-test makró (`__cpp_multidimensional_subscript`) és egy compile probe dönti el.

40.3.2.3 Bounds-szerződés és overflow

Az `operator[]` **előfeltétele**, hogy `i < rows_` és `j < cols_` teljesüljön. Ezt a függvény nem ellenőrzi, mert a gyorsított indexelés éppen ettől gyors; a szerződés megsértése nem definiált viselkedés, nem kivétel. A `noexcept` ezért indokolt: a `std::vector::operator[]` nem dob, és a saját `operator[]` sem végez ellenőrzést. A `at()` ezzel szemben ellenőriz, és `std::out_of_range`-t dob, ezért nem `noexcept`.

Az indexszámításnál az `i * cols_ + j` alak túlcsordulhatna, ha `i` és `cols_` nagyon nagy. A konstruktor azonban ellenőrzi, hogy `rows_ * cols_` ne csorduljon túl. Ebből következik, hogy a szerződés betartása esetén az indexszámítás sem csordul túl:

$$i \cdot cols_ + j \le (rows_ - 1) \cdot cols_ + (cols_ - 1) = rows_ \cdot cols_ - 1,$$

és mivel $rows_ \cdot cols_ \le$ `SIZE_MAX`, az $i \cdot cols_ + j$ legfeljebb `SIZE_MAX` $- 1$. Ez a **vázlat** arra, hogy a bounds-szerződés és az overflow-ellenőrzés ugyanannak a méretszerződésnek a két oldala: a konstruktor a teljes méretet, az `operator[]` a két indexet feltételezi.

Az `std::mdspan` C++23-as `operator[]`-ja szintén több indexet fogad, és ugyanezt a mintát követi: a méreteket és a layoutot az `mdspan` példány hordozza, az indexelés pedig nem végez bounds check-et. Az `mdspan` részleteivel a Multidimenziós view és numeric utility térkép (31.5) foglalkozik; itt csak a kapcsolatot rögzítjük.

40.3.3 Feature-test makrók és fallback

A többdimenziós `operator[]` feature-test makrója `__cpp_multidimensional_subscript`, C++23-as értéke a jegyzet szerint `202110L`. A statikus `operator()` makrója `__cpp_static_call_operator` (`202207L`). A statikus `operator[]` (P2589) önálló makróját a rendelkezésre álló jegyzet nem nevezi meg, ezért itt compile probe dönt: egy `static int operator[](int)` deklarációt tartalmazó kis fordítási egységgel kell ellenőrizni, hogy a compiler elfogadja-e.

A C++20-as fallback a következő. Statikus `operator()` helyett használhatunk statikus tagfüggvényt más néven, például `static bool apply(int)`, vagy névtelen névtérben szabad függvényt. Többdimenziós `operator[]` helyett `operator()(i, j)` vagy `at(i, j)` áll rendelkezésre. A kódot feltételes fordítással lehet mindkét világban fordítani:

```
#if defined(__cpp_multidimensional_subscript) &&
 ↪  __cpp_multidimensional_subscript >= 202110L
#  define CPP23_MULTIDIM_SUBSCRIPT 1
#else
#  define CPP23_MULTIDIM_SUBSCRIPT 0
#endif

class Grid {
public:
    int& at(std::size_t i, std::size_t j) { return data_[i * cols_ + j]; }
#if CPP23_MULTIDIM_SUBSCRIPT
    int& operator[](std::size_t i, std::size_t j) noexcept { return data_[i *
     ↪  cols_ + j]; }
#else
    int& operator()(std::size_t i, std::size_t j) noexcept { return data_[i *
     ↪  cols_ + j]; }
#endif
private:
    std::size_t cols_ = 0;
    std::vector<int> data_;
};
```

Ez a minta a lényeg: a hívó oldalnak a fallback ágban `g(i, j)`, a C++23-as ágban `g[i, j]` alakot kell használnia, ezért a választást vagy makróval, vagy külön wrapperrel kell egységesíteni. A `#if` ágakban a két `operator` szignatúra nem azonos, ezért a kód más részein is figyelni kell a hívási szintaxisra.

40.3.4 Tipikus hibák

Az első és leggyakoribb hiba a `static` és a `const` együttes használata: `static int operator()(int) const` nem érvényes, mert a statikus tagfüggvénynek nincs `this` paramétere, amit a `const` minősíthetne. A második a `m[1, 2]` félreértelmezése: C++20-as kódban ez a vesszőoperátor miatt `m[2]`-t jelenthet, C++23-ban viszont két paraméteres hívást, ha van ilyen `operator[]`; a kettő közötti váltás csendben megváltoztathatja a program jelentését. A harmadik a bounds check elhagyása és az overflow figyelmen kívül hagyása: az `operator[]` nem ellenőriz, a méretösszeszorzást a konstruktornak kell védenie. A negyedik a `const operator[]` által visszaadott referencia élettartamának félreértése: egy temporary `Matrix`-on kapott `const int&` danglinggé válik, amint a temporary megszűnik. Az ötödik a feature-test makró elhagyása: a kód látszólag fordul, de egy régebbi compileren nem a várt ágat választja.

40.3.5 Feladatok

1. **Kódolási feladat.** Definiálj egy `IsEven` funktort `static constexpr bool operator()(int) noexcept` hívóoperátorral, és használd `std::count_if`-fel egy `std::vector<int>`-en. Ellenőrizd `static_assert`-tel, hogy a hívás konstans kifejezésben is működik. Indokold, miért lesz a funktor típusa üres, és miért nem igényel a hívás `this` paramétert.

2. **Tervezési feladat.** Egészítsd ki a `Matrix` osztályt `rows()` és `cols()` tagfüggvényekkel, valamint egy `at(i, j)` const változattal. Biztosítsd, hogy a `m[1, 2]` és a `m.at(1, 2)` ugyanazt az elemet adja, és hogy a `m.at(3, 0)` `std::out_of_range`-t dobjon. Fogalmazd meg az `operator[]` előfeltételét és a konstruktor méretellenőrzésének szerepét.

3. **Diagnosztikai feladat.** Adott egy C++20-as osztály, amely `m(i, j)` alakban indexel. Írd át C++23-as `m[i, j]` alakra, és készíts olyan feltételes fordítású változatot, amely C++20-on a régi `m(i, j)` szintaxist használja. Mutasd meg, melyik feature-test makró dönt, és hogyan ellenőriznéd a statikus `operator[]` támogatását.

40.3.6 Megoldások

40.3.6.1 Az IsEven functor

```
#include <algorithm>
#include <cassert>
#include <vector>

struct IsEven {
    static constexpr bool operator()(int x) noexcept { return x % 2 == 0; }
};

static_assert(IsEven{}(4));
static_assert(!IsEven{}(3));

int main() {
    std::vector<int> v{1, 2, 3, 4, 6};
    assert(std::count_if(v.begin(), v.end(), IsEven{}) == 3);
}
```

A `static` miatt a hívás nem igényel `this` paramétert, a funktor típusa üres, és a `static_assert` azt igazolja, hogy a hívás konstans kifejezésben is érvényes. **Ellenőrzési feltétel:** a program `-std=c++23`-mal forduljon, a `static_assert`-ek menjenek át, és a `count_if` eredménye 3 legyen. C++20-on a `static` elhagyásával a kód továbbra is fordul, de a hívás ekkor `this`-t igényel.

40.3.6.2 A mátrix bounds-szerződéssel

```
// matrix_test.cpp - C++23
#include "matrix.hpp"
#include <cassert>
#include <stdexcept>

int main() {
    Matrix m(3, 4);
    m[1, 2] = 42;
    assert((m[1, 2] == 42));   // dupla zárójel: a vessző ne
     ↪ makróargumentum-elválasztó legyen
    assert(m.at(1, 2) == 42);

    bool dobott = false;
    try {
        (void)m.at(3, 0);
    } catch (const std::out_of_range&) {
        dobott = true;
    }
    assert(dobott);
}
```

Az `assert` makró, és a preprocesszor a C++ nyelvtannál korábban dolgozik: a makróhívás argumentumait a legfelső szintű, zárójelbe nem foglalt vesszők választják el, és a szögletes zárójel ebből a szempontból nem számít. Az `assert(m[1, 2] == 42)` ezért két makróargumentumot kapna (`m[1` és `2] == 42`), és a fordítás `macro "assert" passed 2 arguments, but takes just 1` hibával leáll. A külön zárójelpár egyetlen argumentummá fogja össze a kifejezést; ugyanez kell minden más függvényszerű makrónál is, ha az argumentumban a C++23-as többargumentumos `operator[]` szerepel.

Buildjegyzőkönyv (`lab-gcc` profil). A fenti `Matrix`-szal kiegészítve a program `g++ -std=c++23 -Wall -Wextra -Wpedantic` beállítással figyelmeztetés nélkül fordult és futott, ASan/UBSan builddel is. A zárójel nélküli `assert` a fenti makróhibát adta. C++20-ban a kétparaméteres `operator[]` deklarációja maga hibás (`must have exactly one argument`), ezért a fallback szükséges.

Az `operator[]` előfeltétele `i < rows_ && j < cols_`; megsértése nem definiált viselkedés, ezért nem dob. Az `at()` ellenőriz, és `std::out_of_range`-t dob. A konstruktor a `rows_ * cols_` szorzatot ellenőrzi, mert e nélkül a későbbi `i * cols_ + j` túlcsordulhatna. **Ellenőrzési feltétel:** a `m[1, 2]` és a `m.at(1, 2)` értéke egyezzen, a határon kívüli `at()` dobjon, és a program szaniterrel (például `-fsanitize=address,undefined`) is futási hibát ne jelezzen.

40.3.6.3 A C++20-as fallback

A feltételes fordfítás kulcsa a `__cpp_multidimensional_subscript` makró. Ha a makró értéke legalább `202110L`, a C++23-as `operator[]` ág fordul; különben a C++20-as `operator()` ág. A

hívó oldalt ugyanígy kell elágaztatni, vagy egy vékony wrapperrel egységesíteni:

```
#if CPP23_MULTIDIM_SUBSCRIPT
    m[i, j] = 7;
#else
    m(i, j) = 7;
#endif
```

A statikus `operator[]` támogatását külön compile probe-bal kell ellenőrizni, mert a jegyzet nem nevez meg rá önálló makrót. **Ellenőrzési feltétel:** a C++23-as ág a támogató compileren forduljon és futáskor a helyes elemet írja; a C++20-as ág a régi compileren forduljon; a makró nélküli, véletlenül rossz ágat választó buildet a CI-ben egy negatív teszt fogja meg.

40.3.7 Összefoglaló

A C++23 két, látszólag kicsi operátorbővítése ugyanazt a célt szolgálja: a hívási és indexelési szintaxis közelebb kerüljön ahhoz, amit a programozó gondol. A `static operator()` eltünteti a felesleges `this` paramétert az állapot nélküli callable-ökből, és a capture nélküli lambdák `operator()`-jét is statikussá teszi. A többdimenziós `operator[]` természetes `m[i, j]` szintaxist ad, de megváltoztatja a vessző jelentését, ezért a régi kóddal való keveredés külön figyelmet igényel. Mindkét képességnél a bounds-szerződés, a méret-overflow és az ABI-határ a lényeg; a támogatást feature-test makróval és compile probe-bal kell ellenőrizni, a C++20-as fallbacket pedig tudatosan kell megtervezni.

40.4 Konstans kiértékelés C++23-ban: if consteval és constexpr bővítések

A C++20 bevezette a `constexpr` és `consteval` alapokat, valamint a `std::is_constant_evaluated()` függvényt annak eldöntésére, hogy az aktuális végrehajtás konstans kiértékelésben történik-e. Ez utóbbi azonban csak félmegoldás: közönséges függvényhívás, ezért az általa elválasztott ágak egyike sem immediate function kontextus, és egy paramétertől függő `consteval` (immediate function) hívás egyik ágban sem írható. A C++23 két irányból javított ezen: egyrészt bevezette az `if consteval` és `if !consteval` szerkezeteket, amelyek a futásidejű és a konstans ágat valóban szétválasztják, másrészt lazította a `constexpr` függvénytörzs megengedett szerkezeteit (P2242, P2647). Jelen rész a constexpr, consteval és constant evaluation (33.1) alfejezetben bevezetett alapokra támaszkodik, és azt mélyíti el; az itt tárgyalt `consteval`-propagálás (P2564) státusza csak korlátozottan áll rendelkezésre, ezt külön jelzem. A fejezet feltételezi a Compile-time adatok és típusos konfiguráció (33.2) alfejezetben bemutatott compile-time adatszerkezetek ismeretét (pl. típusos konfiguráció `constexpr` táblákkal).

40.4.1 Az `if consteval` és `if !consteval` szemantikája

Az `if consteval` egy új utasításforma, nem könyvtári függvény. Pontos feltétel: a feltétel akkor teljesül, amikor az aktuális függvényhívását manifestly constant-evaluated kontextusban értékeljük (azaz olyan kifejezésben, amelynek a kiértékelése kötelezően konstans, vagy egy `consteval` függvény törzsében vagyunk). Két szintaktikai alak létezik:

- `if consteval { T; } else { E; }` – a `T` ág a konstans, az `E` ág a futásidejű ág.
- `if !consteval { E; } else { T; }` – ugyanez fordított sorrendben; a `!` alak arra kényelmes, amikor a futásidejű ágat írjuk előre.

Fontos elhatárolás: mindkét ágat **szintaktikailag** ellenőrzi a fordító; a névfeloldás és az overload-resolúció mindkettőre lefut. A különbség **szemantikai**: a megtett ág az adott kontextusban érvényes szabályok szerint értékelődik. Konkrétan a `T` (konstans) ág immediate function kontextusban fut, ezért ott `consteval` függvény hívható; az `E` ág viszont közönséges futásidejű kontextus, ahol `consteval` hívás **hiba**.

Ebből következik, hogy az `if consteval` nem ekvivalens a `std::is_constant_evaluated()` feltétellel. A `bool` visszatérésű függvényhívás után a fordító mindkét ágat közönséges utasításnak tekinti, egyik sem immediate kontextus. Egy `consteval` hívás ott is megengedett, de csak *immediate invocation*-ként, amelynek önmagában konstans kifejezésnek kell lennie: az `immediate(42)` ilyen, mert az argumentuma konstans; az `immediate(x)`, ahol `x` a `constexpr` függvény paramétere, nem az, ezért fordítási hiba — ehhez kell az `if consteval`, amelynek ága immediate kontextus. A futásidejű ágnak ugyanakkor nem kell constexpr-képesnek lennie: ha a függvényt csak futásidőben hívjuk vele, bármilyen közönséges kódot tartalmazhat.

```
// ice_consteval.cpp - C++20: consteval hívás is_constant_evaluated() ágban
#include <type_traits>

consteval int immediate(int x) { return x * 2; }

constexpr int ok(int x) {
    if (std::is_constant_evaluated())
        return immediate(42);        // konstans argumentum: önmagában konstans
         ↪ kifejezés
    return x;                        // a runtime ág bármilyen közönséges kód
     ↪ lehet
}

#ifdef BAD
constexpr int bad(int x) {
    if (std::is_constant_evaluated())
        return immediate(x);         // hiba: x nem konstans kifejezés
    return x;
}
#endif

static_assert(ok(1) == 84);
int main(int argc, char**) { return ok(argc) == argc ? 0 : 1; }
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic ice_consteval.cpp` figyelmeztetés nélkül fordul, a `static_assert(ok(1) == 84)` teljesül, a program 0 kóddal fut. `-DBAD` mellett a hiba: `'x' is not a constant expression`. További gond, hogy `std::is_constant_evaluated()` a `constexpr` lambda és `if constexpr` halmozott szerkezeteiben nehezen áttekinthető állapotokat hozott létre, mert a standard "manifestly constant-evaluated" kategóriáját nem jeleníti meg explicit módon. Az `if consteval` ezt kikényszeríti: a szerzőnek nyilatkoznia kell, melyik ág melyik kontextushoz tartozik.

Első kidolgozott példa: egy runtime és egy konstans ág `consteval` segédfüggvénnyel.

```
// if_consteval_basic.cpp
// Fordítás: g++ -std=c++23 -O2 -Wall -Wextra -pedantic if_consteval_basic.cpp
 ↪ -o if_consteval_basic
//         vagy clang++ -std=c++23 -O2 -Wall -Wextra -pedantic ...
#include <cstdio>
```

```
#if !defined(__cpp_if_consteval) || __cpp_if_consteval < 202106L
#  error "if consteval nem elerheto; ellenorizd a -std=c++23 tamogatast"
#endif

consteval int twice_immediate(int x) {
    return x * 2;                       // csak konstans kontextusbol hivhato
}

constexpr int compute(int x) {
    if consteval {
        return twice_immediate(x);      // OK: ez az ag immediate kontextus
    } else {
        return x * 2;                   // kozonseges futasideju ut
    }
}

int main() {
    static_assert(compute(21) == 42);   // konstans ag
    int r = compute(21);                // futasideju ag
    std::printf("compute(21) = %d\n", r);
}
```

A példa nem tesztelt a jelen dokumentumban; a várt viselkedés az, hogy a `static_assert` a fordítás során igazolódik, a futásidejű hívás pedig a `42` értéket írja ki. A `__cpp_if_consteval` makró a szabvány szerinti feature-test; hiányában a fordító nem támogatja ezt a formát, és a fallback ágra kell váltani. C++20 fallbackként csak `std::is_constant_evaluated()` használható, és az immediate hívás elhagyásával kell élni.

A `!` alak ugyanezt kínálja, csak a futásidejű ág kerül előre:

```
constexpr int compute_alt(int x) {
    if !consteval {
        return x * 2;
    } else {
        return twice_immediate(x);
    }
}
```

40.4.2 P2242: nem literál típusú változó, `goto` és címke

C++20-ban a `constexpr` függvény törzse nem tartalmazhatott nem literál típusú változót, `goto` utasítást vagy címkét. A P2242 ezt a szintaktikai tilalmat megszünteti: a függvény **definíciója** ezekkel együtt is érvényes `constexpr` függvény. A megengedés azonban nem korlátlan: ha a konstans kiértékelés során a végrehajtás eléri a nem literál típusú változó létrehozását, illetve a tiltott konstrukciót, akkor az adott kiértékelés nem lesz konstans kifejezés (a program futásidőben változatlanul működik). A lazítás gyakorlati értéke vegyes kód esetén nagy: `#ifdef` alapú ágak, `goto cleanup` stílusú hibakezelés, valamint futásidejű segédobjektumok, amelyeket `if consteval` zár ki a konstans ágból.

```
// p2242_goto.cpp
// Fordítás: g++ -std=c++23 -O2 -Wall -Wextra -pedantic p2242_goto.cpp -o
 ↪  p2242_goto
```

```
#include <cstdio>

constexpr int abs_like(int x) {
    if (x >= 0) goto done;              // P2242: goto engedelyezett
    x = -x;
done:                                    // P2242: cimke engedelyezett
    return x;
}
static_assert(abs_like(-7) == 7);        // C++23-ban mukodik

#include <vector>
constexpr int mixed(int x) {
    if consteval {
        return x * 7;
    }
    // Kovetkezo resz csak futasidoben erheto el; P2242 elengedte a
    // definicio szintu tiltast, igy a std::vector deklaracio itt mar OK.
    std::vector<int> scratch(10, x);
    return static_cast<int>(scratch.size());
}

int main() {
    std::printf("abs_like(-7)=%d, mixed(3) futasidoben=%d\n",
                abs_like(-7), mixed(3));
}
```

A `constexpr int mixed(int x)` konstans kiértékeléssel hívva a `scratch` deklarációjához nem ér el, ezért a konstans hívás sikeres; futásidőben viszont a `std::vector` létrejön. C++20-ban ez a függvény **fordítási hiba** lett volna, mert a törzs `std::vector`-t deklarál.

40.4.3 P2647: `static constexpr` változó `constexpr` függvényben

C++23 előtt bármilyen `static` lokális változó tilos volt `constexpr` függvényben. A P2647 csak a `static constexpr` esetet engedi meg, mert az ilyen változó konstans-inicializált és megváltoztathatatlan, így nem visz be hívásközi állapotot, és nem akadályozza a konstans kiértékelést. Gyakorlati haszna előre számolt táblák egyszeri definiálása a függvényen belül, anélkül, hogy a tábla a névtérbe szivárogna.

```
// p2647_static.cpp
// Fordítás: g++ -std=c++23 -O2 -Wall -Wextra -pedantic p2647_static.cpp -o
 ↪  p2647_static
#include <cstdio>

constexpr int fib(int n) {
    static constexpr int t[] = {0,1,1,2,3,5,8,13,21,34};
    if (n < 0 || n > 9) return -1;
    return t[n];
}
static_assert(fib(8) == 21);
static_assert(fib(-1) == -1);

int main() {
```

```
    std::printf("fib(8)=%d\n", fib(8));
}
```

C++20 fallbackként a táblát a függvényen kívül, `detail` névtérben kell definiálni, és a függvény paraméterként kapja meg, vagy `constexpr` referenciát használ. A két változat viselkedése azonos; a feature-test (`__cpp_constexpr >= 202207L`) döntése a fordító támogatásán múlik, és a konkrét értéket érdemes a fordító dokumementációjából ellenőrizni.

40.4.4 Részben `constexpr` könyvtári elemek

A C++23 könyvtári oldalon is bővült a konstans kiértékelhető eszköztár. A `std::unique_ptr` (P2273) konstans kiértékelésben is használható, amennyiben a `new` és `delete` hívások a C++20 óta `constexpr` allokációt jelentenek, tehát a mutatott objektum a konstans kiértékelés élettartamán belül keletkezik és szűnik meg. A `<cmath>` és a `<cstdlib>` `constexpr`-sége szabványverziónként és függvényenként eltér, és nem keverendő a fordítói bővítéssel:

Függvénycsoport	Szabvány	Feature-test makró
egész `std::abs`, `std::div` (`<cstdlib>`)	C++23 (P0533)	`__cpp_lib_constexpr_cmath >= 202202L`
`fabs`, `fmin`, `fmax`, `fdim`, `floor`, `ceil`, `trunc`, `round`, `fmod`, `remainder`, `copysign`, `frexp`, `ldexp`, `modf`, osztályozók (`isnan` stb.)	C++23 (P0533)	`__cpp_lib_constexpr_cmath >= 202202L`
`sqrt`, `pow`, `exp`, `log`, trigonometrikus függvények	C++26 (P1383)	`__cpp_lib_constexpr_cmath >= 202306L`

A GCC ettől függetlenül, **bővítésként**, beépített függvényként konstans kiértékelésben is elfogadja a matematikai függvények nagy részét — C++20 módban és `-pedantic-errors` mellett is. Egy sikeres GCC-próba tehát nem szabványbizonyíték. **Buildjegyzőkönyv:** a `static_assert(std::sqrt(4.0) == 2.0)`, `std::pow`, `std::fabs` és `std::floor` próbát a GCC 13.3 `-std=c++20`, `-std=c++23` és `-std=c++23 -pedantic-errors` alatt is elfogadta, a Clang 22 / libc++ `-std=c++23` alatt egyiket sem; és egyik könyvtár sem definiálta a `__cpp_lib_constexpr_cmath` makrót. Hordozható kódban csak a makró és egy compile probe együtt dönthet.

```
// libcx23.cpp
// Fordítás: g++ -std=c++23 -O2 -Wall -Wextra -pedantic libcx23.cpp -o libcx23
#include <cstdio>
#include <memory>

constexpr int unique_demo() {
    auto p = std::make_unique<int>(42);
    return *p;
}
static_assert(unique_demo() == 42);      // C++23 + tamogato libstdc++/libc++
 ↪  eseten

int main() {
    std::printf("unique_demo()=%d\n", unique_demo());
}
```

A pontos feature-test makró neve (tipikusan a `__cpp_lib_constexpr_*` családból) és aktuális értéke függ a standard library kiadásától; a kód előtt compile probe-bal kell ellenőrizni. A `<cmath>` feltételes `constexpr` jellege miatt `static_assert(std::sqrt(2.0) > 1.4)` nem hordozható, és célplatformonként változhat.

40.4.5 A `consteval`-propagálás (P2564) státusza

A P2564 (“consteval needs to propagate up”) egy finom rést zár be: ha `consteval` függvényt hívunk egy `constexpr` függvényből, és a külső `constexpr` függvényt futásidőben hívjuk, a korábbi szabályozás nem garantálta egyértelműen, hogy a belső `consteval` hívás fordítási időben értékelődjön. A javaslat a `consteval` hívást “felfelé” propagálja, azaz a befoglaló `constexpr` függvény hívását is a konstans kiértékelés kontextusába kényszeríti, amíg a `consteval` hívás elérhető. Az állítás pontos megfogalmazása és elfogadási státusza a WG21 jegyzőkönyvekből ellenőrizhető; jelen fejezetben nem áll rendelkezésre forrás, ezért a P2564-re építő kód előtt érdemes a fordító dokumementációjában ellenőrizni, hogy a támogatás megvan-e. A jelen könyv C++23 alapprofilja a P2564-et explicitté tett `if consteval` formákkal helyettesíti ott, ahol a viselkedés hordozhatósága kritikus.

40.4.6 A két ág eredményének tesztelése

A helyes gyakorlat: a `constexpr` függvény mindkét ágát ugyanarra az eredményre tervezzük, majd a konstans ágat `static_assert`-tel, a futásidejű ágat futásidejű ellenőrzéssel validáljuk. A két értéknek egyeznie kell.

```
// branch_agree.cpp
// Fordítás: g++ -std=c++23 -O2 -Wall -Wextra -pedantic branch_agree.cpp -o
 ↪  branch_agree
#include <cassert>
#include <cstdio>

constexpr int clamp(int v, int lo, int hi) {
    if consteval {
        // konstans ag: egyszeru, olvashato
        if (v < lo) return lo;
        if (v > hi) return hi;
        return v;
    } else {
        // futasideju ag: azonos logika, peldakent kulon megirva
        return (v < lo) ? lo : (v > hi ? hi : v);
    }
}

int main() {
    static_assert(clamp(-1, 0, 10) == 0);
    static_assert(clamp( 5, 0, 10) == 5);
    static_assert(clamp(42, 0, 10) == 10);

    for (int v = -5; v <= 15; ++v) {
        assert(clamp(v, 0, 10) ==
               (v < 0 ? 0 : (v > 10 ? 10 : v)));
    }
```

```
    std::printf("minden ertek egyezik\n");
}
```

A `assert` futásidejű ellenőrzés, a `static_assert` fordítási. A példa nem lett lefuttatva; a várt viselkedés az, hogy a program a “minden ertek egyezik” üzenetet írja ki.

40.4.7 Tipikus hibák és javításuk

- **`consteval` hívás a futásidejű ágban.** Az `else` ág közönséges kontextus, ezért `consteval` hívás ott fordítási hiba. Javítás: a hívást a `T` (konstans) ágba helyezni.
- **`is_constant_evaluated()` és `if consteval` felcserélése.** Az előbbi nem tesz lehetővé `consteval` hívást, és a fordító mindkét ágat azonos módon szemantizálja. Utóbbi a helyes eszköz.
- **A “nem megtett ág nem létezik” tévhit.** Mindkét ág szintaktikailag és névfeloldásilag is lefut; a különbség a szemantikai érvényesség.
- **Hordozhatósági feltételezés `<cmath>` `constexpr`-re.** A függvény feltételesen `constexpr`; célonként és library-verziónként változhat, ezért feature-test és compile probe kötelező.
- **P2564-re építés a státusz ellenőrzése nélkül.** A javaslat státusza és implementáltsága nem ellenőrzött ebben a fejezetben; óvatosan kezelendő.

40.4.8 Összefoglaló

A C++23 `if consteval` szerkezete a konstans és futásidejű ág valódi szétválasztását adja, és egyedül ez teszi lehetővé `consteval` hívását a konstans ágban. A P2242 feloldotta a nem literál típusú változókra, `goto`-ra és címkékre vonatkozó definíciós tilalmat; a P2647 engedélyezte a `static constexpr` lokálisokat. A `std::unique_ptr` és a `<cmath>`/`<cstdlib>` egyes függvényei részben `constexpr`-ré váltak, de feltételesen, ezért feature-test és compile probe szükséges. A P2564 státusza a jelen fejezetben nem ellenőrzött, a hordozható kód `if consteval`-lel tegye explicitté a szándékot. Végül a két ág egyezését `static_assert` és futásidejű `assert` kombinációjával érdemes tesztelni.

40.4.9 Feladatok

1. feladat (diagnosztika). Az alábbi C++20 kód nem fordul. Magyarázd meg a fordítási hiba okát, majd írd át C++23-ra.

```
consteval int ci(int x) { return x + 1; }
constexpr int f(int x) {
    if (std::is_constant_evaluated()) {
        return ci(x);
    }
    return x;
}
```

2. feladat (kódolás). Készíts `constexpr gcd(a,b)` függvényt, amely `if consteval`-lel az Euklideszi algoritmus egyszerű, olvasható változatát használja konstans kiértékelésben, futásidőben pedig ugyanezt a logikát külön (például bitmanipulációs) alakban. Igazold `static_assert`-tel legalább három inputra, és futásidejű `for` ciklussal ellenőrizd, hogy a két ág minden inputra egyezik.

3. feladat (kombinált). Írj `constexpr` faktoriális-függvényt, amely előre számolt `static constexpr int` táblát használ. Adj C++20 fallbacket, ahol a tábla a függvényen kívül, `detail` névtérben van. A feature-detection a `__cpp_constexpr` makróra épüljön. `static_assert`-tel és futásidejű `assert`-tel bizonyítsd, hogy mindkét változat ugyanazt adja.

4. feladat (magyarázó). Mi a szemantikai különbség aközött, hogy egy `consteval` hívás az `if consteval` konstans ágában, illetve a futásidejű ágában szerepel? Miért tekinthető úgy, hogy a "nem megtett ág" szintaktikailag mégis létezik?

40.4.10 Megoldások

1. megoldás. A hiba oka: a `ci(x)` egy `consteval` függvény hívása egy közönséges `if` blokkban, ahol a `bool`-t visszaadó `std::is_constant_evaluated()` feltételként szerepel. A fordító a `f` függvényt nem tekinti immediate functionnek, így a `ci(x)` hívását nem az immediate kontextus szabályai szerint engedélyezi: mindkét ág közönséges utasítás, ezért a `ci` hívás már önmagában fordítási hiba. Helyes C++23 változat:

```
constexpr int f(int x) {
    if consteval {
        return ci(x);
    }
    return x;
}
```

Ellenőrzés: `static_assert(f(41) == 42);` és futásidejű hívás esetén a `41` visszaadása. Feature-test: `#if defined(__cpp_if_consteval) && __cpp_if_consteval >= 202106L`, különben a `ci` hívást elhagyó fallback.

2. megoldás.

```
// gcd_demo.cpp
// Fordítás: g++ -std=c++23 -O2 -Wall -Wextra -pedantic gcd_demo.cpp -o gcd_demo
#include <cassert>
#include <cstdio>
#include <cstdlib>

constexpr int gcd(int a, int b) {
    if consteval {
        while (b != 0) {
            int t = a % b;
            a = b;
            b = t;
        }
        return a < 0 ? -a : a;
    } else {
        a = a < 0 ? -a : a;
        b = b < 0 ? -b : b;
        while (b != 0) {
            int t = a % b;
            a = b;
            b = t;
        }
        return a;
```

```
    }
}

int main() {
    static_assert(gcd(12, 18) == 6);
    static_assert(gcd(17,  5) == 1);
    static_assert(gcd( 0,  9) == 9);

    for (int a = -20; a <= 20; ++a) {
        for (int b = -20; b <= 20; ++b) {
            int expected = (b == 0) ? std::abs(a)
                                    : gcd(std::abs(a), std::abs(b));
            assert(gcd(a, b) == expected);
        }
    }
    std::printf("gcd: minden ertek egyezik\n");
}
```

Indoklás: mindkét ág ugyanazt az Euklideszi algoritmust futtatja; a konstans ág rövidebb, mert a konstans kiértékelésben nincs szükség az `abs` előzetes normalizálására (a standard könyvtár `std::abs` futásidejű hívása helyett a konstans ág közvetlenül kezeli az előjelet). Ellenőrzési feltétel: minden inputpárra `gcd(a,b) == gcd(std::abs(a), std::abs(b))`, illetve a `b != 0` esetben a klasszikus megfelelés.

3. megoldás.

```
// fact_static.cpp
// Fordítás: g++ -std=c++23 -O2 -Wall -Wextra -pedantic fact_static.cpp -o
 ↪  fact_static
#include <cassert>
#include <cstdio>

#if defined(__cpp_constexpr) && __cpp_constexpr >= 202207L
constexpr int fact(int n) {
    static constexpr int t[] = {1,1,2,6,24,120,720,5040,40320,362880};
    if (n < 0 || n > 9) return -1;
    return t[n];
}
#else
namespace detail {
    constexpr int fact_table[] = {1,1,2,6,24,120,720,5040,40320,362880};
}
constexpr int fact(int n) {
    if (n < 0 || n > 9) return -1;
    return detail::fact_table[n];
}
#endif

int main() {
    static_assert(fact(0) == 1);
    static_assert(fact(5) == 120);
    static_assert(fact(9) == 362880);
    static_assert(fact(-1) == -1);
```

```
    for (int n = -2; n <= 10; ++n) {
        int got = fact(n);
        int want;
        if (n < 0 || n > 9) want = -1;
        else {
            want = 1;
            for (int i = 2; i <= n; ++i) want *= i;
        }
        assert(got == want);
    }
    std::printf("fact: minden ertek egyezik\n");
}
```

Indoklás: a `static constexpr int t[]` konstans-inicializált és megváltoztathatatlan, ezért a `constexpr` függvény törzsében való definíciója nem visz be hívásközi állapotot, és nem akadályozza a konstans kiértékelést. A fallback változat ugyanazt a táblát a `detail` névtérben definiálja, a függvény pedig csak referál rá. Ellenőrzési feltétel: mindkét változat minden inputra ugyanazt adja, és a `-1` érzéketlen tartományon kívülre eső inputokra is helyesen jelez.

4. megoldás. A `consteval` függvény csak immediate kontextusból hívható. Az `if consteval T` ága ilyen kontextus: a megtett ág konstans kiértékelésben fut, ezért a `consteval` hívás érvényes. Az `E` ág futásidejű kontextus: ott a `consteval` hívás olyan, mintha egy közönséges függvényből próbálnánk egy immediate functiont hívni, ezért fordítási hiba. A "nem megtett ág" onnan "létezik", hogy a fordítónak mindkét ágat szintaktikailag és névfeloldásilag ellenőriznie kell: a nyelvtan megköveteli, hogy a nem megtett ágban is érvényes kód álljon, csak a konkrét szemantikai érvényesség (például `consteval` hívás engedélyezettsége) tér el a kontextus szerint. Ez a kettősség adja az `if consteval` lényegét: a döntés fordítási idejű, de a kód mindkét ágon értelmes.

40.5 Érték, élettartam és kifejezések: auto(x), implicit move és range-for

A C++23 nyelvi változásainak egy része nem új szintaxist vezet be, hanem az élettartam- és értékkategória-szabályokat finomítja. Ez a rész négy ilyen változást tárgyal: az `auto(x)` explicit decay-copy-t, az egyszerűsített implicit move-ot, a range-for temporary-élettartam-kiterjesztését, valamint a `[[assume]]` és a `std::unreachable` optimalizálási szerződéseit. A vezérmotívum a dangling: mindegyik változás arról szól, hogy egy érték vagy egy rá mutató nézet meddig érvényes, és a fordító mikor feltételezheti, hogy a programozó ezt betartotta. A rész a Temporary, value category és dangling (20.4) részben bevezetett temporary- és value category-fogalmakra, a Borrowed range, dangling és iterator lifetime (30.5) részben tárgyalt borrowed range-ekre és az Optimalizáló, as-if és UB következményei (14.3) részben bevezetett UB- és as-if-modellre épül; ezeket itt nem ismételjük teljesen, csak a szükséges előfeltételeket idézzük fel.

40.5.1 A labor alapja: feature-test és compile probe

A C++23-as képességek fordító- és könyvtártámogatása nem azonos. A fejezetterv kiinduló URL-jei (a WG21 munkatervezet, a cppreference C++23-oldala, a GCC/Clang státuszoldalak) kutatási kiindulópontok, nem bizonyítékok; a konkrét támogatást a laborban, compile probe-pal kell ellenőrizni. A feature-test makrók neve és a hozzá tartozó képesség:

Képesség	Feature-test makró	Jelleg
`auto(x)`	`__cpp_auto_cast`	nyelvi
implicit move	`__cpp_implicit_move`	nyelvi
range-for élettartam	`__cpp_range_based_for`	nyelvi; C++20-ban is definiált, más értékkel
`[[assume]]`	`__cpp_assume`	nyelvi
`std::unreachable()`	`__cpp_lib_unreachable`	könyvtári

A következő probe önmagában fordítható C++20-ban és C++23-ban is; a hiányzó makrókat nem definiáltként jelzi. A szakirodalomban elterjedt értékek (például `__cpp_auto_cast = 202110L`, `__cpp_implicit_move = 202207L`, `__cpp_range_based_for = 202211L`, `__cpp_assume = 202207L`, `__cpp_lib_unreachable = 202202L`) irodalmi értékek, amelyeket a probe kimenetével és a hivatkozott WG21-paperrel kell összevetni; a fejezet írásakor ezek nem tekintendők ellenőrzöttnek.

```
// probe.cpp
#include <version>
#include <cstdio>

int main() {
#ifdef __cpp_auto_cast
    std::printf("__cpp_auto_cast = %ld\n", __cpp_auto_cast);
#else
    std::printf("__cpp_auto_cast nem definiált\n");
#endif
#ifdef __cpp_implicit_move
    std::printf("__cpp_implicit_move = %ld\n", __cpp_implicit_move);
#else
    std::printf("__cpp_implicit_move nem definiált\n");
#endif
#ifdef __cpp_range_based_for
    std::printf("__cpp_range_based_for = %ld\n", __cpp_range_based_for);
#else
    std::printf("__cpp_range_based_for nem definiált\n");
#endif
#ifdef __cpp_assume
    std::printf("__cpp_assume = %ld\n", __cpp_assume);
#else
    std::printf("__cpp_assume nem definiált\n");
#endif
#ifdef __cpp_lib_unreachable
    std::printf("__cpp_lib_unreachable = %ld\n", __cpp_lib_unreachable);
#else
    std::printf("__cpp_lib_unreachable nem definiált\n");
#endif
}
```

```
g++ -std=c++20 -O2 -Wall -Wextra -o probe20 probe.cpp && ./probe20
g++ -std=c++23 -O2 -Wall -Wextra -o probe23 probe.cpp && ./probe23
# Clang esetén: clang++ -std=c++20 / -std=c++23
```

A kimenet értelmezése: ami C++20 módban „nem definiált”, az C++23 módban vagy definiált, vagy a fordító még nem támogatja. Utóbbi esetben a képességhez fallback kell, nem a makróra épülő feltétel nélküli kód.

40.5.2 Az `auto(x)` és az `auto{x}` decay-copy

A C++20-ig a „decay-copy” a standard szövegében létező, exposition-only fogalom volt: a `std::ranges` egyes részei és a generikus kód nem tudta kifejezni, hogy „az argumentumból független, másolt értéket kérek”. A P0849 paperszámú javaslat ezt nyelvi eszközzé tette.

Definíció. Az `auto(x)` egy prvalue kifejezés, amelynek típusát az `auto` típuslevezetése adja meg az `x` operandusból. A típuslevezetés a by-value paraméterátadás szabályait követi: a referencia eldobása, a top-level `const` és `volatile` eldobása, a tömb- és függvénytípus pointerré alakítása. Az eredmény egy új objektum, amely független az operandustól; a másolat akkor készül el, amikor a prvalue materializálódik. Az `auto{x}` ugyanezt teszi, de lista-inicializálással, ezért tiltja a szűkítő konverziót.

Feltételek és következmények. Az `auto(x)` mindig prvalue, soha nem referencia és soha nem xvalue; nem azonos a `decltype(auto)`-val, amely referenciát is adhat. Nem őrzi meg a top-level `const`-ot: `const int i = 5; auto j = auto(i);` esetén `j` típusa `int`. A másolat költsége akkor jelenik meg, amikor az érték anyagiassá válik; visszatérési értékként a guarantee copy elision miatt gyakran nincs tényleges másolás, de ez nem általános garancia.

```
// auto_decay_copy.cpp
#include <cstdio>
#include <string>
#include <type_traits>
#include <utility>

template<class T>
auto decay_copy(T&& t) {
#if defined(__cpp_auto_cast) && __cpp_auto_cast >= 202110L
    return auto(std::forward<T>(t));          // C++23
#else
    return std::decay_t<T>(std::forward<T>(t)); // C++20 fallback
#endif
}

int main() {
    const std::string s = "hello";
    auto a = decay_copy(s);
    static_assert(std::is_same_v<decltype(a), std::string>);
    std::printf("%s\n", a.c_str());
}
```

```
g++ -std=c++23 -O2 -Wall -Wextra -o decay23 auto_decay_copy.cpp && ./decay23
g++ -std=c++20 -O2 -Wall -Wextra -o decay20 auto_decay_copy.cpp && ./decay20
```

A tipikus hiba itt az, hogy a programozó referenciát vagy `const`-ot vár vissza. Az `auto(x)` tudatosan értéket ad: ez a független, biztonságos másolat eszköze, nem a nézeté.

40.5.3 Egyszerűsített implicit move

A C++20 szabálya (P1825) szerint a `return e;` utasításban, ha `e` egy nem-volatile, automatikus tárolási idejű objektum vagy rvalue-referencia paraméter neve, és a visszatérési típus nem referencia, akkor a fordító először úgy végzi a túlterhelés-feloldást, mintha `e` rvalue lenne, és csak ennek kudarca esetén próbálja lvalue-ként. Ez a szabály már C++20-ban sem követeli meg, hogy a visszatérési típus megegyezzen `e` típusával: egy `Derived` helyi változó `Base`-ként visszaadva C++20-ban is a `Base(Base&&)` konstruktort választja. (A GCC ezt a javítást korábbi nyelvi módokra is alkalmazza.)

Definíció. A C++23 (P2266) egyszerűsíti a szabályt: a `return e;` utasításban a fenti feltételeknek megfelelő `e` név egyszerűen **xvalue**, a két lépcsős túlterhelés-feloldás helyett. Az új szabály a referencia visszatérési típusú függvényekre is vonatkozik, és éppen ott változtat megfigyelhetően. Tömören: C++23-ban a `return e;` utasításban az `e` xvalue, ha

1. az `e` egy nem-volatile, automatikus tárolási idejű objektum vagy rvalue-referencia neve, és
2. a `return` egy függvényben (vagy lambdában) áll.

Viselkedésváltozás. Érték szerinti visszatérésnél a C++20-hoz képest nincs eltérés; referencia visszatérési típusnál viszont van:

```
// implicit_move.cpp - C++20 és C++23: mi változott valójában?
#include <cstdio>
#include <utility>

struct Base {
    Base() = default;
    Base(const Base&) { std::puts("copy"); }
    Base(Base&&) noexcept { std::puts("move"); }
};
struct Derived : Base { };

Base f() {
    Derived d;
    return d;        // C++20 és C++23: move (P1825, C++20)
}

Base g() {
    const Derived d;
    return d;        // mindkét módban copy: const Derived&& nem köt Base&&-hez
}

#if __cplusplus > 202002L
int&& fwd(int&& x) { return x; }   // C++23: x xvalue, fordul; C++20: hiba
#endif
// int& keep(int&& x) { return x; } // C++20: fordul; C++23: hiba

int main() {
    (void)f();
    (void)g();
#if __cplusplus > 202002L
    int v = 7;
    int&& r = fwd(std::move(v));
```

```
    std::printf("fwd=%d\n", r);
#endif
}

g++ -std=c++20 -O0 -Wall -Wextra -o im20 implicit_move.cpp && ./im20
g++ -std=c++23 -O0 -Wall -Wextra -o im23 implicit_move.cpp && ./im23
```

Buildjegyzőkönyv (`lab-gcc` profil). A `-std=c++20` kimenet: `move`, `copy`; a `-std=c++23` kimenet: `move`, `copy`, `fwd=7`. A `f()` mindkét módban mozgat, a `const` változat (`g()`) mindkét módban másol. Az `int&& fwd(int&& x) { return x; }` C++20-ban `cannot bind rvalue reference ... to lvalue` hibát ad, C++23-ban fordul. A kikommentezett `int& keep(int&& x) { return x; }` fordítva viselkedik: C++20-ban fordul, C++23-ban `cannot bind non-const lvalue reference ... to an rvalue` hibát ad. Ez az utóbbi a migráció valódi forráskompatibilitási törése.

A `return std::move(x);` írásmód érték szerinti visszatérésnél felesleges, sőt hátrányos lehet, mert a `std::move` függvényhívása megakadályozza az NRVO-t; a `-Wpessimizing-move` figyelmeztet rá.

40.5.4 A range-for temporary-élettartama

A C++20-as range-for lényegében `auto&& __range = range-init;` deklarációvá alakult. A `range-init` azon temporary-jei, amelyek közvetlenül a `__range` referenciához kötődtek, a ciklus végéig éltek; a közvetett temporary-k azonban a teljes kifejezés végén megsemmisültek. Ez a különbség okozta a klasszikus danglinget.

Definíció. A C++23-ban (P2718) a range-initializerben létrejövő temporary objektumok élettartama a ciklus teljes élettartamára kiterjed, nem csak a range-initializer kiértékelésének végéig. Ez a temporary lifetime extension kiterjesztése a range-for-ra. A szabály csak a range-initializer *temporary* objektumaira vonatkozik; a ciklustörzsben keletkező temporary-kre és az init-statement temporary-jeire nem. Egy függvény **paraméterobjektuma** sem temporary: ha egy `const T& f(T t) { return t; }` jellegű függvény a paraméterére ad vissza referenciát, a paraméter a függvény visszatérésekor vagy legkésőbb a teljes kifejezés végén (implementációfüggő, melyik) megszűnik, és C++23-ban is a ciklus előtt. A `make().get()` eset ezzel szemben megmenekül, mert a `make()` eredménye valódi temporary.

```
// range_for_param.cpp - C++23 (P2718): mit ment meg, és mit nem
#include <cstdio>
#include <vector>

struct Box {
    std::vector<int> v{1, 2, 3};
    const char* name;
    explicit Box(const char* n) : name(n) {}
    ~Box() { std::printf("  ~Box(%s)\n", name); }
    const std::vector<int>& get() const { return v; }
};

Box make() { return Box{"temp"}; }
// A paraméterobjektum nem a range-init temporaryje: a hívott függvényhez
 ↪  tartozik.
const std::vector<int>& pass(Box b) { return b.get(); }
```

```
int main() {
    std::puts("make().get():");
    for (int x : make().get()) std::printf("  elem %d\n", x);   // C++23: a
     ↪ temporary él
    std::puts("pass(Box{param}):");
    for (int x : pass(Box{"param"})) {     // HIBA: a paraméter a ciklus előtt
        (void)x; std::puts("  ciklustorzs"); break;   // megszűnik - UB
    }
}
```

Buildjegyzőkönyv. A GCC 13.3 a P2718-at még nem implementálja; a Clang 22 (`__cpp_range_based_for == 202211L`) `-std=c++23` alatt a `make().get()` ciklusban kiírta az `elem 1..3` sorokat, és csak utána `~Box(temp)`-et; a `pass(Box{param})` esetben a `~Box(param)` a ciklustörzs *előtt* jelent meg, a törzs pedig le sem futott — a hozzáférés nem definiált viselkedés. `-std=c++20` alatt a `make().get()` ciklus is a `~Box(temp)` után, üres eredménnyel futott.

```
// range_for_lifetime.cpp
#include <vector>
#include <cstdio>

struct Config {
    std::vector<int> values{1, 2, 3};
    const std::vector<int>& get() const { return values; }
};

Config make() { return Config{}; }

int main() {
    int sum = 0;
    for (int v : make().get()) {   // C++20: dangling; C++23: OK
        sum += v;
    }
    std::printf("sum = %d\n", sum);
}
```

```
g++ -std=c++20 -g -O1 -fsanitize=address -o r20 range_for_lifetime.cpp && ./r20
g++ -std=c++23 -g -O1 -fsanitize=address -o r23 range_for_lifetime.cpp && ./r23
```

C++20-ban a `make()` temporary nem közvetlenül a `__range`-hez kötődik, hanem a `get()` hívás argumentumaként jön létre, ezért a teljes kifejezés végén megsemmisül; a `get()` által visszaadott referencia dangling, az iterálás UB. C++23-ban a `make()` temporary a ciklus végéig él, a referencia érvényes marad. A várt laboreredmény: az `r20` ASan heap-use-after-free hibát jelez, az `r23` normálisan lefut; a tényleges kimenetet a laborban kell rögzíteni, ez a leírás nem mérési eredmény.

A P2718 nem old meg minden danglinget. Ha a `get()` saját temporary-t ad vissza referenciaként (`const std::vector<int>& get() const { return std::vector<int>{1,2,3}; }`), az már a `get()` visszatérésekor dangling. Ha a range egy `std::string_view`, amely egy lokális stringre mutat, az is dangling marad. A szabály csak a range-initializer temporary-jeire terjed ki.

40.5.5 Az `[[assume]]` és a `std::unreachable`

Mindkét eszköz a fordítónak tett ígéret, amelynek megsértése nem definiált viselkedés. A különbség az ígéret tartalmában van.

`[[assume(expr)]]`. A C++23-ban egy null utasításra alkalmazott attribútum. Azt jelenti: a programozó állítja, hogy `expr` a végrehajtás ezen pontján igaz. Ha nem igaz, a viselkedés nem definiált. Az `expr` nem értékelődik ki, tehát nem lehet mellékhatása, és nem hívódik meg; a fordító az igazságára támaszkodva optimalizálhat. Csak akkor használható, ha a feltétel tényleg garantált; nem helyettesíti a futásidejű ellenőrzést.

`std::unreachable()`. A C++23-as `<utility>` fejléc függvénye. Azt jelenti: a hívás helye nem érhető el. Ha mégis elérhető, a viselkedés nem definiált. Nem feltétel, hanem pont; a hívás megtörténik, de a fordító optimalizálhatja. Tipikus használata a minden enumértéket lefedő `switch` után, vagy olyan függvény végén, ahol a fordító nem látja, hogy nincs visszatérés.

Szempont	`[[assume(expr)]]`	`std::unreachable()`
Az ígéret tartalma	a feltétel igaz	a pont nem érhető el
Forma	attribútum null utasításon	függvényhívás
Kiértékelődik	nem	a hívás megtörténik; elérhetetlen esetben UB
Tipikus használat	feltétel optimalizálása	elérhetetlen ág jelölése
Fallback	`if (!(cond)) __builtin_unreachable();`	`__builtin_unreachable()`

```
// assume_unreachable.cpp
#include <utility>
#include <cstdio>

enum class Color { Red, Green, Blue };

const char* name(Color c) {
    switch (c) {
    case Color::Red:   return "red";
    case Color::Green: return "green";
    case Color::Blue:  return "blue";
    }
    std::unreachable();   // C++23
}

int divide(int a, int b) {
    [[assume(b != 0)]];   // a hívónak garantálnia kell
    return a / b;
}

int main() {
    std::printf("%s %d\n", name(Color::Green), divide(6, 3));
}

g++ -std=c++23 -O2 -Wall -Wextra -o au23 assume_unreachable.cpp && ./au23
# C++20 fallback: std::unreachable() helyett __builtin_unreachable(),
# [[assume]] helyett if (!(b != 0)) __builtin_unreachable();
```

A `switch` után a `std::unreachable()` azért kell, mert a fordító a `switch` után is feltételezhet visszatérést; a hívás jelzi, hogy minden eset le van fedve. Ha később új enumérték kerül be, és a `switch` nem kezeli, a `default` nélküli `switch` `-Wswitch` figyelmeztetést ad, a `std::unreachable()` pedig UB-hez vezet, ha mégis elérhető. Ezért a `std::unreachable()` nem véd a jövőbeli bővítés ellen; a `-Wswitch` és a teljes lefedés a védelem.

40.5.6 Kisebb szintaktikai változások

A C++23 két apró, de hasznos szabályt is hozott. Az egyik: egy címke állhat az összetett utasítás végén, közvetlenül a `}` előtt; C++20-ban legalább egy utasítás kellett a címke után. Ez generált kódban és `switch`-eknél kényelmes:

```
switch (code) {
case 1:
    handle();
    break;
case 2:   // C++23: legális, C++20: fordítási hiba
}
```

A másik: az `if`/`switch` init-statementje lehet alias-deklaráció is. C++23-ban:

```
if (using T = int; sizeof(T) == 4) {
    // a T az if feltételében és ágában él
}
```

C++20-ban ez nem fordul; a fallback a deklarációt az `if` elé teszi, ami megváltoztatja a `T` hatókörét. A változás főleg generikus kódban hasznos, ahol a típust az init-statementben akarjuk bevezetni.

40.5.7 Sanitizer-labor: ugyanaz a kód két módban

A labor célja, hogy ugyanazt a forrást C++20 és C++23 módban fordítsuk, és megfigyeljük a különbséget. A `range_for_lifetime.cpp` esetében a C++20-as buildet AddressSanitizerrel futtatva a várt eredmény heap-use-after-free hiba; a C++23-as buildben nincs hiba. Az `implicit_move.cpp` esetében a különbség a fordításban látszik: a `fwd` csak C++23-ban fordul, a `keep` csak C++20-ban; a `puts`-kimenet (`move`, `copy`) a két módban azonos. Az `assume_unreachable.cpp` esetében a `[[assume]]` megsértése UB; a sanitizer nem garantált, hogy diagnosztizálja, ezért a helyes használatot kódolvasással és a hívói szerződés ellenőrzésével kell biztosítani. A `std::unreachable()` elérését egyes sanitizerek (például Clang `-fsanitize=unreachable`) jelezhetik, de ez nem hordozható elvárás.

A labor parancsai a fenti szekciókban szerepelnek. A „várt eredmény” itt a dokumentált viselkedésen alapuló illusztráció, nem mért eredmény; a tényleges kimenetet a tanuló laborja rögzíti, és összeveti a feature-test makrók kimenetével.

40.5.8 Tipikus hibák és javításuk

- **Az `auto(x)`-et referenciának gondolni.** Az `auto(x)` mindig prvalue, értéket ad; ha referenciát akarunk, arra más eszköz kell. Javítás: a szándékot a típusdeklarációban és a névben jelezzük.
- **`return std::move(x);` használata.** C++23-ban felesleges, és ronthatja az NRVO-t. Javítás: `return x;`.

- **A P2718 mindenre kiterjesztése.** Csak a range-initializer temporary-jeire vonatkozik. Javítás: a nézet élettartamát a forrás élettartamához kötni, vagy a range-et tulajdonló objektumban tárolni.
- **`[[assume]]` futásidejű ellenőrzésként.** Ha a feltétel nem garantált, UB. Javítás: futásidejű ellenőrzés vagy szerződéses API, és csak utána `[[assume]]`.
- **`std::unreachable()` `default` ágban.** Ha később új érték érkezik, UB. Javítás: `default` nélküli `switch` és `-Wswitch`, vagy `default` ágban `std::abort()`.
- **A feature-test makróra feltétel nélkül építeni.** A makró jelenléte nem garantálja a teljes szemantikát. Javítás: compile probe és a laborban ellenőrzött viselkedés.

40.5.9 Összefoglaló

A C++23 négy élettartam- és értékkategória-változása egy közös irányba mutat: a fordító pontosabban tudja, meddig él egy érték, és a programozó explicitebb eszközökkel fejezheti ki a szándékát. Az `auto(x)` független értéket ad, az implicit move a `return` utasításban a származtatott és bázistípusok közötti mozgatást is lehetővé teszi, a range-for temporary-kiterjesztés a leggyakoribb közvetett danglinget szünteti meg, a `[[assume]]` és a `std::unreachable()` pedig a feltétel- és elérhetetlenségi ígéreteket választja szét. Egyik változás sem helyettesíti a szerződések tudatos tervezését; a feature-test makrók és a sanitizer-labor csak az eszközök helyes használatát segítik.

40.5.10 Feladatok

1. **Magyarázó feladat.** Magyarázd el, miért dangling a `range_for_lifetime.cpp` C++20-as változatában a `make().get()` hívás, és mit változtat a C++23. Adj egy olyan módosítást, amelyet a P2718 nem javít.
2. **Kódolási feladat.** Írj egy `decay_copy` függvénysablont, amely C++23-ban `auto(x)`-et, C++20-ban `std::decay_t`-t használ. Ellenőrizd feature-test makróval és `static_assert`-tel, hogy a visszatérési típus `std::string` egy `const std::string&` bemenetre.
3. **Diagnosztikai feladat.** Adott a `name(Color)` függvény a `std::unreachable()` hívással. Vezess be egy új `Color::Yellow` értéket, és magyarázd el, mi történik, ha a `switch` nem kezeli. Javítsd a kódot úgy, hogy a hiba fordítási időben kiderüljön.
4. **Viselkedésváltozási feladat.** A `Base f() { Derived d; return d; }` példában melyik konstruktor hívódik C++20-ban és C++23-ban, és mi a szerepe a `const`-nak? Mutass olyan függvényt, amelynek a fordíthatósága a két mód között megváltozik.

40.5.11 Megoldások

1. megoldás. A `make()` temporary nem közvetlenül a range-for `__range` referenciájához kötődik, hanem a `get()` tagfüggvény hívásának argumentumaként jön létre. C++20-ban az ilyen közvetett temporary a teljes kifejezés végén megsemmisül, így a `get()` által visszaadott `const std::vector<int>&` danglingre mutat; az iterálás UB. C++23-ban a range-initializer temporary-jei a ciklus végéig élnek, ezért a referencia érvényes. Nem javítja a P2718 azt az esetet, amikor a `get()` saját temporary-t ad vissza referenciaként: `const std::vector<int>& get() const { return std::vector<int>{1,2,3}; }` – itt a temporary a `get()` visszatérésekor megsemmisül, mert nem a range-initializerben jött létre. Ellenőrzési feltétel: a C++20-as ASan-build hibát jelez, a C++23-as nem; a `get()` módosított változata C++23-ban is hibát jelez.

2. megoldás. A `decay_copy` a fenti `auto_decay_copy.cpp` szerint épül fel. A lényeg a feltételes fordítás:

```
#if defined(__cpp_auto_cast) && __cpp_auto_cast >= 202110L
    return auto(std::forward<T>(t));
#else
    return std::decay_t<T>(std::forward<T>(t));
#endif
```

Az ellenőrzés `static_assert(std::is_same_v<decltype(decay_copy(s)), std::string>);`, ahol `s` `const std::string&`. Indoklás: az `auto` eldobja a referenciát és a top-level `const`-ot, a `std::decay_t` ugyanezt teszi, ezért a két ág típusa megegyezik. Ellenőrzési feltétel: a C++23-as buildben a `__cpp_auto_cast` ág fordul, a C++20-asban a fallback; mindkettőben a `static_assert` teljesül.

3. megoldás. Ha a `Color::Yellow` bekerül, de a `switch` nem kezeli, a `default` nélküli `switch` a `-Wswitch` figyelmeztetést adja, és a `std::unreachable()` hívás elérhetővé válik, ami UB. A javítás: hagyjuk el a `default` ágat, soroljuk fel az összes enumerátort, és a `switch` után `std::unreachable()`-t használjunk; így az új érték figyelmeztetést ad. Alternatíva: `default` ágban `std::abort()`, amely nem UB, hanem definiált leállás. Ellenőrzési feltétel: az új enumerátorral a build `-Wswitch` figyelmeztetést ad, a `std::unreachable()` nem érhető el.

4. megoldás. Mindkét módban a `Base(Base&&)` hívódik: a C++20 (P1825) már nem követeli meg a típusegyezést, a C++23 (P2266) pedig xvalue-ként kezeli a `d`-t. `const Derived d;` esetén a `const Derived&&` nem köthető `Base&&`-hez, ezért mindkét módban másolat készül; ez a `const` szerepe. A fordíthatóság referencia visszatérési típusnál változik: az `int&& fwd(int&& x) { return x; }` csak C++23-ban, az `int& keep(int&& x) { return x; }` csak C++20-ban fordul. Ellenőrzési feltétel: a `puts`-kimenet a két módban azonos (`move`, illetve a `const` változatban `copy`), a `fwd`/`keep` pár fordítási eredménye pedig a két módban ellentétes; a `lab-gcc` profilban mindezt lefuttattuk.

40.6 Lexikális és preprocessor bővítések, kiterjesztett lebegőpontos típusok

A C++23 nyelvi változásainak jelentős része nem új absztrakció, hanem a meglévő eszközök „élesítése”: olyan helyzetek, ahol a C++20-ban a programozó kénytelen volt kerülőutat választani. A preprocessor feltételes fordítása, a literálok típusa, a karakterescape-ek írásmódja és a kis pontosságú lebegőpontos típusok mind ilyenek. A hiányzó elem nem új paradigmát, hanem pontosan azonosítható kellemetlenséget okozott: felesleges `defined()`-zárójeleket, `static_cast<size_t>`-öt a literálokon, „mohó” `\x` escape-eket, illetve azt, hogy a `float16_t`-hez implementációs neveket (`_Float16`, `__fp16`) kellett használni. Ez az alfejezet ezeket a forrásszintű újdonságokat veszi sorra, mindegyiknél a C++20-as megoldásból kiindulva.

40.6.1 A laborprofil kiterjesztése és az állítások státusza

A rész alapprofilja innentől ISO C++23, explicit `-std=c++23` kapcsolóval, GCC vagy Clang mellett. A fordítási mód neve és a támogatás mértéke fordítóverzió-függő: a laborban rögzítsd a `g++ --version` / `clang++ --version` kimenetét, és először egy üres „probe” fájllal ellenőrizd, hogy a `-std=c++23` egyáltalán elfogadott (egyes implementációk a `-std=c++2b` aliast ismerik). Az ebben az alfejezetben szereplő képességek támogatottságát nem állítom ellenőrzöttnek; mindenhol megadom a feature-test makrót és/vagy a nyelvi elemet ténylegesen kipróbáló compile probe-ot, valamint a C++20-as fallbackot.

A `<version>` header a szabványos feature-test makrók központi helye; a könyvtári képességnél `__cpp_lib_*`, a nyelvi elemnél `__cpp_*`, a `<stdfloat>` típusainál pedig a fordító által definiált `__STDCPP_*_T__` makrók a mérvadók. A preprocesszor-direktívákhoz tartozó makrónevek és -értékek verziófüggők; ezeket a `<version>` tartalmából és a fordító dokumentációjából kell kiolvasni, ezért ahol lehet, nyelvi compile probe-ot használunk helyettük.

40.6.2 Feltételes fordítás: #elifdef, #elifndef és #warning

A C++20-ban egy többlépcsős platform- vagy feature-választó láncot így kellett írni:

```
#if defined(__linux__)
#  define PLATFORM "linux"
#elif defined(__APPLE__)
#  define PLATFORM "darwin"
#else
#  define PLATFORM "unknown"
#endif
```

A P2334 bevezette a `#elifdef` és `#elifndef` direktívákat. Szemantikailag `#elifdef AZONOSITO` pontosan ekvivalens a `#elif defined(AZONOSITO)` kifejezéssel, `#elifndef` pedig a `#elif !defined(AZONOSITO)`-val. Két feltétel fontos. Egyrészt a direktíva operandusa **egyetlen azonosító**, nem kifejezés, ezért a `#elifdef defined(X)` alak érvénytelen — ez a leggyakoribb hiba, mert a szerkezet `#if`-re emlékeztet. Másrészt a `#elifdef` csak a feltételes fordítás láncának folytatásaként állhat: `#if`, `#ifdef`, `#ifndef` vagy egy korábbi `#elif*` után; önállóan nem.

A P2437 a `#warning` direktívát szabványosította. Hatása: az implementációnak egy diagnosztikai üzenetet kell kibocsátania, amely tartalmazza a direktíva szövegét, de a program **nem lesz ill-formed**, és a fordítás alapértelmezés szerint folytatódik. Ez a lényegi különbség a `#error`-hoz képest. A `-Werror` viszont nem nyelvi szabály, hanem fordítóopció, amely a figyelmeztetéseket hibává emeli — és a `#warning` diagnosztikáját mindkét nagy fordító figyelmeztetésként adja ki, tehát `-Werror` mellett a build megáll. Ez nem szabványsértés: a szabvány csak a diagnosztikát követeli meg, azt nem tiltja, hogy egy opció hibává tegye. **Buildjegyzőkönyv** (`lab-gcc` profil): egy `#warning`-ot tartalmazó minimálprogram `g++ -std=c++23` alatt `warning: #warning ... [-Wcpp]` üzenettel, 0 kóddal fordult; `-Werror`-ral `error: #warning ... [-Werror=cpp]`, 1-től eltérő kóddal elutasult. A Clang 22 `-Werror` mellett ugyanígy: `[-Werror,-W#warnings]`. Ha a `-Werror` buildben a `#warning` nem állíthatja meg a fordítást, az `-Wno-error=cpp` (GCC), illetve `-Wno-error=#warnings` (Clang) kivétel kell. A C++20-as fallback `#pragma message("...")`, amely viszont implementációfüggő, és nem minden fordító ismeri.

Kidolgozott példa — platformválasztó lánc, `#warning`-gal a „nem várt” ágon:

```
// pp23.cpp
#include <cstdio>

#if defined(__linux__)
#  define PLATFORM "linux"
#elifdef __APPLE__
#  define PLATFORM "darwin"
#elifdef _WIN32
#  define PLATFORM "windows"
#else
#  warning "ismeretlen platform: a platform-specifikus kod talan nem fordul"
#  define PLATFORM "unknown"
```

```
#endif

int main() {
    std::printf("platform: %s\n", PLATFORM);
}
```

```
g++ -std=c++23 -Wall -Wextra -Wpedantic -o pp23 pp23.cpp
```

A `__linux__`, `__APPLE__`, `_WIN32` makrók nem szabványosak, hanem fordító/OS specifikusak; itt csak a direktívalánc illusztrálására szolgálnak. A várt viselkedés leírás, nem mért eredmény: Linuxon a `linux` szöveg jelenik meg, a `#warning` pedig csak az `#else` ágon fut.

40.6.3 A z és uz literálsuffixum

A P0330 két új egész literálsuffixumot vezetett be. A `z`/`Z` suffixszel jelölt literál típusa a `std::size_t` **előjeles párja**, azaz `std::make_signed_t<std::size_t>` (LP64-en `long`); az `uz`/`UZ` suffixszel jelölt literál típusa maga a `std::size_t` (LP64-en `unsigned long`). A literál értékének ábrázolhatónak kell lennie a megfelelő típusban; ha nem, a program ill-formed, tehát itt nincs csendes túlcsordulás.

A gyakorlati nyereség a signed/unsigned keveredés és a felesleges castok eltüntetése. C++20-ban például:

```
template <class C>
constexpr auto last_index(C const& c) {
    return c.size() == 0 ? std::size_t{0} : c.size() - 1;  // std::size_t
}
```

C++23-ban ugyanez `0uz`-vel olvashatóbb, és a `size()` eredményével való összehasonlítás nem kever előjelet:

```
template <class C>
constexpr auto last_index(C const& c) {
    return c.size() == 0uz ? 0uz : c.size() - 1uz;   // mindenhol std::size_t
}
```

A signed változat akkor hasznos, ha a ciklusváltozó negatív is lehet, vagy ha `int`-tel kell összevetni: `for (auto i = 0z; i < static_cast<decltype(i)>(v.size()); ++i)`. Itt a `0z` típusa `long`, így a `v.size()`-t egyszer kell konvertálni, és a `-Wsign-compare` csendben marad. Figyelem: a `z` nem azt jelenti, hogy „size_t”, hanem azt, hogy „a size_t előjeles párja” — a két suffix tehát nem ugyanannak a típusnak a két neve.

40.6.4 Nevesített karakterek és határolt escape-ek

A P2071 a `\N{KARAKTERNÉV}` univerzális karakternév-escape-et vezette be: az `\u`/`\U` kódszám helyett az Unicode Character Database-ben definiált **név** alapján azonosítja a karaktert. A P2290 ehhez kapcsolódva a `\x`, `\u`, `\U` escape-eknek kapcsos zárójeles, határolt alakját adta: `\x{...}`, `\u{...}`, `\U{...}`. Mindkettő ugyanazt a problémát oldja meg, csak más oldalról.

A `\x` escape hagyományosan „mohó”: a fordító a lehető leghosszabb hexadecimális számjegysorozatot fogyasztja, ezért a `"\xff" "f"` trükk kellett ahhoz, hogy egy `0xFF` byte után `f` betű következzen. A `"\x{ff}f"` alak ezt explicit módon határolja, és a kapcsos alakban a számjegyek száma is korlátlan, ami a `\u`/`\U` esetében a 4, illetve 8 jegyű korlátot szünteti meg (`\u{1F600}`).

A `\N{...}` és a `\x{...}` közötti különbség tanulságos. A `\x{e9}` egyetlen kódpontot jelöl, amelynek a karakterliterál típusába kell illeszkednie; `char` literálban ez a `0xE9` byte. A `\N{LATIN SMALL LETTER E WITH ACUTE}` ugyanezt a kódpontot nevezi meg, de a stringliterál **kódolása** dönti el, hány byte lesz belőle: UTF-8 végrehajtási karakterkészlet esetén kettő (`0xC3 0xA9`). A kettő tehát nem ekvivalens:

```
#include <cstdio>
#include <cstring>

int main() {
    const char* hexed = "\x{e9}";                                  // 1 byte:
     ↪  0xE9
    const char* named = "\N{LATIN SMALL LETTER E WITH ACUTE}";      // UTF-8:
     ↪  2 byte
    std::printf("hexed=%zu named=%zu\n", std::strlen(hexed),
     ↪  std::strlen(named));
}
```

```
g++ -std=c++23 -Wall -Wextra -o esc esc.cpp
```

A névfeloldás támogatása nem kötelező minden implementációban, ezért itt különösen indokolt a compile probe: ha a `\N{...}`-t tartalmazó fájl nem fordul, a fallback a `\u00E9` alak. A pontos egyeztetési szabályt (a név és a kódpont megfeleltetését) a szabvány `[lex.charset]` része rögzíti; erre itt nem térek ki részletesen.

40.6.5 UTF-8 forrásfájl kötelező támogatása

A P2295 kimondja, hogy a C++23 implementációnak a forrásfájlt UTF-8 kódolásúként is el kell tudnia olvasni. Ez fontos különbség a korábbi helyzethez képest, ahol a forráskód karakterkészlete implementációfüggő volt. Két dolgot viszont nem jelent. Egyrészt nem jelenti azt, hogy a `char` stringliterál automatikusan UTF-8 lesz: azt a **végrehajtási karakterkészlet** (execution character set) dönti el, amely GCC/Clang esetén külön kapcsolóval (`-fexec-charset=UTF-8`) állítható. Másrészt a `u8"..."` literál típusa `const char8_t[]` (C++20 óta), és a `char8_t` konverziója `char`-rá nem implicit.

Tipikus hiba a gyakorlatban a nem UTF-8 kódolású forrásfájl (Windows-1250, Latin-1), amelyet a fordító UTF-8-ként olvas, és „invalid UTF-8 sequence” hibát ad, vagy rossz karaktert tesz a literálba. A diagnózis: `file -i forras.cpp`, illetve `xxd` az érintett soron. A javítás nem a `\x`-szel való kézi kódolás, hanem a fájl konvertálása (`iconv`) és a build során a kódolás deklarálása. A BOM kezelése szintén implementációfüggő; a laborban érdemes BOM nélküli UTF-8-at használni.

40.6.6 Kiterjesztett lebegőpontos típusok: a <stdfloat> header

A P1467 hozta létre a `<stdfloat>` headert és az ott definiált típusokat: `std::float16_t`, `std::float32_t`, `std::float64_t`, `std::float128_t` és `std::bfloat16_t`. A `std::floatN_t` azt a típust jelöli, amely az ISO/IEC/IEEE 60559 `binaryN` formátum értékkészletét legalább lefedi (N = 16, 32, 64, 128); a `std::bfloat16_t` a bfloat16 értékkészletét. A típus létezése **nem kötelező**: a szabvány akkor írja elő, ha az implementáció rendelkezik ilyen típussal, és ezt a `__STDCPP_FLOAT16_T__`, `__STDCPP_FLOAT32_T__`, `__STDCPP_FLOAT64_T__`, `__STDCPP_FLOAT128_T__`, illetve `__STDCPP_BFLOAT16_T__` makró definiálásával jelzi. A `<stdfloat>` header tartalmára a `__cpp_lib_stdfloat` feature-test makró utal; a konkrét értéket a `<version>`-ből kell kiolvasni.

A típusokhoz literálsuffixumok is tartoznak: `f16`, `f32`, `f64`, `f128`, `bf16`. A `1.0f16` típusa `std::float16_t`, a `1.0bf16` típusa `std::bfloat16_t`. A `std::float32_t` gyakori targeten azonos a `float`-tal, de ezt a szabvány nem garantálja; ABI-határon ezért nem szabad a kettőt felcserélhetőnek tekinteni.

A konverziós szabályok gyakorlati magja: a `float16_t` és `bfloat16_t` a szokásos aritmetikai konverziók során a `float` felé promoválódik, így a velük végzett aritmetika legalább `float` pontossággal történik, és az eredmény tároláskor veszít pontosságot. A `float16_t` → `float` és `float16_t` → `double` konverzió implicit, de szűkítő lehet; `{}` inicializálásnál konstans kifejezés esetén ez fordítási hiba, változó esetén `-Wconversion` figyelmeztet. A `binary16` kb. 3 tizedesjegy pontosságú, legnagyobb véges értéke 65504; a `bfloat16` kevesebb mantisszabit ad (kb. 2–3 tizedesjegy), de a `float` exponenstartományát tartja meg — ezért gépi tanulásban népszerű.

Kidolgozott példa feature-testtel és fallbackkal:

```
// stdfloat_probe.cpp
#include <cstdio>
#include <version>

#if defined(__STDCPP_FLOAT16_T__)
#  include <stdfloat>
#endif

int main() {
#if defined(__STDCPP_FLOAT16_T__)
    std::float16_t h = 1.0f16 / 3.0f16;
    std::float32_t f = 1.0f32 / 3.0f32;
    std::printf("float16=%.6f float32=%.6f sizeof=%zu/%zu\n",
                static_cast<double>(h), static_cast<double>(f),
                sizeof h, sizeof f);
#else
    std::puts("std::float16_t nem elerheto ezen a targeten (fallback: float)");
#endif
}
```

```
g++ -std=c++23 -Wall -Wextra -Wconversion -o stdfloat_probe stdfloat_probe.cpp
```

A várt kimenet leírás, nem mért eredmény: `float16` esetén a `binary16` kerekítése miatt kb. `0.333252`, `float32` esetén kb. `0.333333` jelenne meg. A `{}` inicializálás szűkítését érdemes külön kipróbálni tanítási hibaként:

```
std::float16_t h{1.0 / 3.0};   // hiba: double -> float16_t szukito, nem
 ↪  ábrázolható pontosan
std::float16_t h2 = 1.0 / 3.0; // implicit konverzió, -Wconversion jelzi
```

40.6.7 Párhuzamok és eltérések a C23-mal

A preprocesszor terén a C23 is átvette a `#elifdef`, `#elifndef` és `#warning` direktívákat, így a két nyelv között ezek hordozhatók — a pontos C23 paragrafusokat és az esetleges eltéréseket a C23-as részek tárgyalják (C23 könyvtár és preprocessor). A literálsuffixumok és escape-ek terén viszont nincs átfedés: a `z`/`uz`, az `f16`/`bf16` és a `\N{...}`/`\x{...}` C++-specifikusak, ezért kétnyelvű fejlécben `#ifdef __cplusplus` védelem vagy makró szükséges. A `<stdfloat>` C++ header; a C `_Float16`/`_Float32` típusai és a `std::float16_t` között nincs garantált megfeleltetés, ezért

`extern "C"` határon ezeket nem szabad binárisan átadni az ABI ellenőrzése nélkül. A `char8_t` és a `u8` literál mindkét nyelvben létezik (C++20, illetve C23), de a `char8_t` C++-ban erősebb típus; a C23-as változat részletei szintén a C23-as fejezetben szerepelnek.

40.6.8 Tipikus hibák és javításuk

A leggyakoribb hiba a `#elifdef defined(X)` írásmód: az operandus azonosító kell, nem kifejezés; a javítás `#elifdef X`. A második, hogy a `#warning`-ot hibának hiszik: nem állítja meg a fordítást, hibához `#error` kell. A harmadik, hogy a `z` suffixet `size_t`-nek válik: az `uz` a `size_t`, a `z` a párja. A negyedik, hogy a `"\x{e9}"`-et UTF-8 `é`-nek tekintik: az egyetlen byte, a helyes UTF-8 `"\N{LATIN SMALL LETTER E WITH ACUTE}"` vagy `u8"\u00E9"`. Az ötödik, hogy a `#ifdef __STDCPP_FLOAT16_T__` és a `__cpp_lib_stdfloat` közé egyenlőséget tesznek: az előbbi a konkrét típusra, az utóbbi a header tartalmára utal. Végül a hatodik: a `std::float32_t`-t ABI-határon `float`-ként kezelik, holott ez nem garantált.

40.6.9 Feladatok

1. **Preprocessor-lánc.** Írj egy `platform.h` fejlécet, amely `#if`/`#elifdef`/`#elifndef` lánccal három platformot különböztet meg, és az ismeretlen ágon `#warning`-ot ad. Magyarázd el, miért hiba a `#elifdef defined(__linux__)`.
2. **Indexelés.** Adott egy `std::vector<int> v` és egy `int i`. Írj egy `bool in_range(int, std::size_t)` segédfüggvényt, amely `0z`/`0uz` használatával elkerüli a signed/unsigned keveredést, és indokold, miért nem elég a puszta `static_cast`.
3. **Escape-diagnosztika.** Adott `const char* s = "\x{e9}";`. Add meg a `strlen(s)` értékét és az okát, majd javítsd a kódot UTF-8 `é`-re. Add meg az ellenőrzési feltételt is.
4. **stdfloat-kapu.** Írj egy fejlécet, amely `std::float16_t`-t használ, ha elérhető, különben `float`-ra esik vissza, és a visszaesést `#warning`-gal jelzi. Sorold fel a szükséges feature-test makrókat.

40.6.10 Megoldások

1. megoldás. A lánc:

```
// platform.h
#if defined(__linux__)
#  define PLATFORM_ID 1
#elifdef _WIN32
#  define PLATFORM_ID 2
#elifndef __unix__
#  warning "nem unix-szeru platform: a POSIX kodutak nem fordulnak"
#  define PLATFORM_ID 0
#else
#  define PLATFORM_ID 3
#endif
```

A `#elifdef defined(__linux__)` azért hiba, mert a direktíva operandusa egyetlen azonosító; a `defined` operátor csak `#if`/`#elif` kifejezésben értelmezhető. Ellenőrzés: a fájl fordul `-std=c++23`-mal, és az `#else` ágra kényszerítve (`-U__linux__ -U__unix__`) megjelenik a `#warning` üzenet, de a fordítás nem áll meg.

2. megoldás.

```cpp
#include <cstddef>
#include <vector>

bool in_range(int i, std::size_t n) {
    return i >= 0z && static_cast<std::size_t>(i) < n;
}
```

Itt a `0z` típusa `long` (LP64-en), így az `i >= 0z` összehasonlítás előjeles marad, és nem kell a negatív `i`-t unsigned-dá alakítani — ez a `static_cast`-tal nem oldható meg, mert a cast után a `-1` nagy unsigned értékké válik. A második feltételben a cast csak a nemnegatív ágon fut, ahol értéke megőrződik. Ellenőrzés: `i = -1`, `n = 10` esetén `false`; `i = 3`, `n = 10` esetén `true`; `-Wsign-compare` nem jelez.

3. megoldás. A `"\x{e9}"` egyetlen `char`, értéke `0xE9`, ezért `strlen(s) == 1`. UTF-8-ban az `é` (U+00E9) két byte: `0xC3 0xA9`. Javítás:

```cpp
const char* s = "\N{LATIN SMALL LETTER E WITH ACUTE}"; // vagy u8"\u00E9"
```

Ellenőrzési feltétel: `strlen(s) == 2`, és `xxd`-vel a byte-ok `c3 a9`. Ha a `\N{...}` nem fordul, a fallback `"\u00E9"`, illetve `u8"\u00E9"`.

4. megoldás.

```cpp
// f16_compat.h
#include <version>

#if defined(__STDCPP_FLOAT16_T__)
#  include <stdfloat>
#  define F16 std::float16_t
#else
#  warning "std::float16_t nem elerheto: float fallback"
#  define F16 float
#endif
```

A szükséges makrók: `__STDCPP_FLOAT16_T__` a típus létezésére, `__cpp_lib_stdfloat` a `<stdfloat>` header tartalmára. Ellenőrzés: a fejléc fordul mindkét ágon; a `static_assert(sizeof(F16) >= 2)` nem garantált, ezért a méretre ne építsünk szerződést.

40.6.11 Összefoglaló

A C++23 forrásszintű újdonságai jól elkülöníthető csoportokba esnek. A preprocessor `#elifdef`/`#elifndef`/`#warning` direktívái a feltételes fordítás zaját csökkentik, de operandszerződésük szigorú: azonosító, nem kifejezés. A `z`/`uz` literálsuffixum a `size_t` és előjeles párja típusát rögzíti a literál szintjén, megszüntetve a castokat. A `\N{...}` és a `\x{...}`/`\u{...}`/`\U{...}` escape-ek a karakterazonosítást teszik olvashatóvá, illetve határolttá; a `char` és a stringliterál kódolása továbbra is két külön kérdés. Az UTF-8 forrás támogatása kötelező, de a végrehajtási karakterkészlet nem automatikusan UTF-8. A `<stdfloat>` típusai feltételesek, ezért feature-test és compile probe kötelező; a `float16_t`/`bfloat16_t` aritmetikája `float` felé promoválódik, és a tárolás pontosságvesztéssel jár. A C23-mal való párhuzam a preprocesszorra korlátozódik; a literál- és típuskészlet eltér, ezért a kétnyelvű határon minden ilyen elemet ellenőrizni kell.

41 Hibakezelés, diagnosztika és kimenet C++23-ban

41.1 std::expected mélyebben: monadikus hibaláncok

Egy szolgáltatás hibamodellje akkor auditálható, ha a hibaáramlás egyetlen kifejezésláncban olvasható, és minden ponton pontosan tudjuk, milyen típusú hiba léphet ki belőle. A korábbi error_code, system_error és expected (25.4) részben az `error_code`, `system_error` és `expected` alapeseteit rögzítettük; itt azt építjük tovább, ami egy termelési API-t megkülönböztet egy tankóldától: a rétegzett, monadikus hibaláncot, a hibaidőtípus rétegenkénti megválasztását és a kivételhatár szerződését. A Hibakezelési és exception safety labor (25.5) laborra és a Hibakód, errno és hibaadatok élettartama (11.3) `errno`-élettartam-szabályára támaszkodunk.

Források. A szabályok forrása a forrásjegyzék C++23 tétele ([expected]); a `std::expected` maga a P0323, a monadikus függvények a P2505 javaslatból származnak. A fordító- és könyvtár-támogatást feature-test makróval és fordítási próbával döntjük el. A `lab-gcc` profil libstdc++-a `__cpp_lib_expected`-et 202211L értékkel definiálja, vagyis a monadikus felületet is nyújtja.

41.1.1 Az `expected` alapszerződése és állapotai

A `std::expected<T, E>` egy tagszerű tárolású összegtípus: vagy egy `T` értéket, vagy egy `E` hibát tartalmaz, soha nem mindkettőt, és az értéket nem a szabad tárból allokálja. A `std::unexpected<E>` csomagoló jelentést hordoz: a csomagolatlan érték mindig az **értékágat** inicializálja, a csomagolt a **hibaágat**. Ezért `expected<std::error_code, std::error_code>` esetén sincs kétértelműség: a `return ec;` sikeres értéket ad vissza, a `return std::unexpected{ec};` hibát — a nyelvi szabály egyértelmű, csak az olvasónak kell tudnia, hogy a csomagolás dönt.

```
// expected_forms.cpp - C++23: azonos T és E; and_then eredménytípusa
#include <cassert>
#include <expected>
#include <string>
#include <system_error>
#include <type_traits>

using R = std::expected<std::error_code, std::error_code>;
R as_value(std::error_code ec) { return ec; }                    // ÉRTÉK
 ↪  (siker)
R as_error(std::error_code ec) { return std::unexpected{ec}; }   // HIBA

enum class E { bad };
std::expected<std::string, E> to_text(int v) { return std::to_string(v); }

int main() {
    auto ec = std::make_error_code(std::errc::timed_out);
```

```
    assert(as_value(ec).has_value() && *as_value(ec) == ec);
    assert(!as_error(ec).has_value() && as_error(ec).error() == ec);

    std::expected<int, E> a = 42;
    auto b = a.and_then(to_text);       // R = remove_cvref_t<invoke_result_t<F,
     ↪  int&>>
    static_assert(std::is_same_v<decltype(b), std::expected<std::string, E>>);
    assert(b && *b == "42");
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++23 -Wall -Wextra -Wpedantic expected_forms.cpp` figyelmeztetés nélkül fordult, és minden `assert` teljesült: az `as_value` értéket (`has_value() == true`), az `as_error` hibát tartalmaz.

Előfeltételek: `T` nem referencia- és nem függvénytípus, nem `unexpected`-specializáció, és lehet `void` (`expected<void, E>` a „siker, de nincs érték” esetre). `E` objektumtípus, nem referencia, nem függvénytípus, nem `unexpected`-specializáció, és nem `void`. A pontos szabványos megfogalmazás helyét a munkatervezetben kell visszakeresni; itt a szerződés tartalma a lényeg.

Két megfigyelhető állapot van: az objektum vagy értéket, vagy hibát tárol, és soha nincs egyik nélkül sem (*never-empty*). A `std::variant`-tal ellentétben az `expected`-nek nincs „valueless by exception” állapota. Az értékadó műveletek két külön esetre bonthatók, és a garancia mindkettőnél a `T`/`E` megfelelő műveleteihez kötött. (1) **Azonos ág** (érték ← érték vagy hiba ← hiba): a `T` (illetve `E`) saját értékadó operátora fut. Ha az módosít, majd dob, az `expected` továbbra is értékállapotú (never-empty), de a régi érték **nem feltétlenül marad meg**: ez nem általános rollback, a garanciát a `T::operator=` adja (például egy módosító, majd dobó `operator=` részben módosított értéket hagy). (2) **Ágváltás** (hibából értékbe vagy fordítva): a művelet az ellenkező ág objektumát rekonstruálja; a never-empty tulajdonság azon alapul, hogy ehhez a `T`/`E` megfelelő konstruktora (nem dobó módon mozgatható, vagy a másik típus nem dobó módon építhető) elérhető, különben az értékadás nem vesz részt a túlterhelés-feloldásban és a kód nem fordul le. Ágváltásnál az új objektum konstrukciójának kivétele esetén a könyvtár a régi állapotot visszaállítja (a rekonstrukciós eljárás ezt célozza), az azonos ágú értékadásnál viszont nincs ilyen visszaállítás — a konkrét garancia a használt szabványkiadás és a `T`/`E` típus műveleteinek függvénye.

A hozzáférők szerződése külön-külön:

Hozzáférő	Előfeltétel	Ha az előfeltétel nem teljesül
`has_value()`, `operator bool`	nincs	—
`operator*`, `operator->`	`has_value()` igaz	nem definiált viselkedés
`value()`	nincs	hibaállapotban `std::bad_expected_access<E>` kivételt dob (az `error()` másolatával)
`error()`	`has_value()` hamis	nem definiált viselkedés
`value_or(v)`	nincs	hibaállapotban a `v`-t adja

```
// expected_states.cpp - C++23: két állapot, value() és a dereferálás
#include <cstdio>
#include <expected>
#include <string>

int main() {
    std::expected<int, std::string> ok = 42;
```

```
    std::expected<int, std::string> bad = std::unexpected(std::string("parse"));

    std::printf("ok: has_value=%d *ok=%d value=%d\n", ok.has_value(), *ok,
     ↪  ok.value());
    std::printf("bad: has_value=%d error=%s\n", bad.has_value(),
     ↪  bad.error().c_str());

    try {
        (void)bad.value();                      // hibaállapotban: kivétel, nem
         ↪  UB
    } catch (const std::bad_expected_access<std::string>& e) {
        std::printf("value() dobott: bad_expected_access, error=%s\n",
         ↪  e.error().c_str());
    }

    bad = 7;                                    // értékadás: hibából
 ↪  értékállapotba
    std::printf("bad = 7: has_value=%d\n", bad.has_value());
}
```

Buildjegyzőkönyv (`lab-gcc` profil, `-std=c++23`). A kimenet: `ok: has_value=1 *ok=42 value=42`, `bad: has_value=0 error=parse`, `value() dobott: bad_expected_access, error=parse`, `bad = 7: has_value=1`.

41.1.2 A monadikus műveletek pontos típusai

Négy művelet alkotja a C++23-as monadikus felületet: `and_then`, `or_else`, `transform` és `transform_error`. Mind a négyet a P2505 adta a P0323-ban szabványosított `std::expected`-hez. A két rész feature-test értéke eltér: `__cpp_lib_expected >= 202202L` csak az alap típust jelzi, a monadikus felülethez `>= 202211L` kell. Egy korábbi könyvtárkiadás tehát nyújthatja az `expected`-et monadikus függvények nélkül; ilyenkor a lánc helyére kézzel írt `if (r) ... else ...` elágazás lép.

Legyen `F` hívható, és a `&`-minősített változatot nézzük (`std::invoke(f, value)` szemantikával). A jelölés: `R` a callback **teljes** eredményének típusa, cv- és referenciaminősítők nélkül, például `R = std::remove_cvref_t<std::invoke_result_t<F, T&>>`.

- `and_then(f)`: az eredmény típusa maga `R`; ha van érték, `f(value)`-t adja vissza, ha hiba van, egy `R`-t a változatlan hibával. **Megkötés:** `R` az `expected` specializációja, és `R::error_type` pontosan `E`. Az új értéktípus `R::value_type`. Példa: egy `expected<int, E>`-re alkalmazott `int → expected<std::string, E>` callback után az eredmény `expected<std::string, E>` — beágyazott `expected` nélkül. Ez a kulcs: az `and_then` nem konvertál hibát.
- `transform(f)`: ha van érték, `expected<U, E>`-t ad, ahol `U = std::remove_cv_t<std::invoke_result_t<F, T&>>` (itt a callback *nem* `expected`-et ad, hanem a becsomagolandó értéket); ha hiba van, változatlanul továbbadja. `U` nem lehet `unexpected`-specializáció, különben a hiba dupla csomagolásba kerülne.
- `or_else(f)`: az eredmény típusa `G = std::remove_cvref_t<std::invoke_result_t<F, E&>>`, amelynek `expected`-specializációnak kell lennie `G::value_type == T` feltétellel; ha hiba van, `f(error)`-t adja, ha érték van, egy `G`-t a változatlan értékkel.
- `transform_error(f)`: ha hiba van, `expected<T, G>`-t ad, ahol `G = std::remove_cv_t<std::invoke_result_t<F, E&>>` érvényes hibatípus; ha érték van, változatlanul továbbad.

`expected<void, E>` esetén az `and_then` és a `transform` hívhatója **nulla paramétert** kap. Ez a leggyakoribb fordítási hibaforrás, amikor egy `expected<T,E>`-re írt lambdát `expected<void,E>`-re használunk.

A rövidzárlat táblázata: értékágon `and_then`/`transform` fut, `or_else`/`transform_error` nem; hibaágon fordítva. Értékágon a hiba típusa sosem változik meg `and_then`/`transform` alatt, ezért egy láncon belül `E` végig azonos kell legyen — a hibakonverzió helye kizárólag a `transform_error` (és az `or_else` hibaidőtípus-váltása).

41.1.3 Típuslevezetés egy láncon

Legyen `a : std::expected<int, E>`, `f : int → std::expected<double, E>`, `g : double → std::string`, `h : E → std::error_code`, `k : std::error_code → DomainErr`.

Kifejezés	Eredmény típusa	Megjegyzés
`a`	`expected<int, E>`	kiindulás
`a.and_then(f)`	`expected<double, E>`	hiba típusa változatlan
`.transform(g)`	`expected<string, E>`	érték transzformálás
`.transform_error(h)`	`expected<string, error_code>`	itt vált hibaidőtípust a lánc
`.transform_error(k)`	`expected<string, DomainErr>`	rétegváltás a modulhatáron
`a.transform(f)`	`expected<expected<double,E>,E>`	**hiba**: `transform` nem lapít

A táblázat tanulsága: a láncot úgy kell felépíteni, hogy a hibaidőtípus-váltás egyetlen, dokumentált ponton történjen, és a lánc belsejében `and_then`-t használjunk, amikor a hívott függvény maga is `expected`-et ad.

41.1.4 Kidolgozott példa: `errno`-tól a doménhibáig

A példa egy konfigurációbetöltő réteg: a fájlolvasás operációs hibát (`error_code`), a feldolgozás doménhibát (`enum class`) ad, a kettő találkozását a `transform_error` kezeli. Az `errno`-t minden esetben azonnal, a következő könyvtárhívás előtt csomagoljuk be — ez a Hibakód, errno és hibaadatok élettartama (11.3) szabálya.

```
// cfg.cpp -- ISO C++23, std::expected monadikus lanc, fallbackkel
#include <version>      // a feature-test makrok forrasa: MINDEN teszt elott

#if !defined(__cpp_lib_expected)
#  error "A laborprofil standard konyvtara nem nyujt std::expected-et (C++23)"
#endif
// 202211L: monadikus fuggvenyek (P2505); 202202L: csak az alap expected
 ↪  (P0323).
#if __cpp_lib_expected >= 202211L && !defined(CFG_FORCE_FALLBACK)
#  define CFG_MONADIC 1
#endif

#include <cerrno>
#include <charconv>
```

```
#include <cstdio>
#include <expected>
#include <memory>
#include <string>
#include <string_view>
#include <system_error>

namespace cfg {

// Tesztbuild: a CFG_INJECT_* makrok a fread/fclose hibajat szimulaljak.
#if defined(CFG_INJECT_READ)
std::size_t sys_fread(void* b, std::size_t s, std::size_t n, std::FILE* f) {
    std::size_t got = std::fread(b, s, n < 4 ? n : 4, f);   // reszadat...
    errno = EIO;                                             // ...es I/O hiba
    return got;
}
int sys_ferror(std::FILE*) { return 1; }
#else
std::size_t sys_fread(void* b, std::size_t s, std::size_t n, std::FILE* f) {
    return std::fread(b, s, n, f);
}
int sys_ferror(std::FILE* f) { return std::ferror(f); }
#endif
#if defined(CFG_INJECT_CLOSE)
int sys_fclose(std::FILE* f) { std::fclose(f); errno = ENOSPC; return EOF; }
#else
int sys_fclose(std::FILE* f) { return std::fclose(f); }
#endif

enum class err { missing, io, malformed, out_of_range, duplicate };

std::error_code errno_code(int e) noexcept { return {e, std::system_category()};
 ↪  }

// RAII owner: a stream minden kilepesi uton (kivetel is) lezarul.
struct file_closer { void operator()(std::FILE* f) const noexcept {
 ↪  sys_fclose(f); } };
using file_ptr = std::unique_ptr<std::FILE, file_closer>;

// Hibapolicy: az OS-hibat error_code-kent adjuk vissza; a tarfoglalasi hiba
// (std::bad_alloc) kivetel marad, a streamet ekkor is a file_ptr zarja le.
// A lezaras hibaja csak akkor lesz eredmeny, ha korabban nem volt hiba:
// az elsodleges olvasasi hibat masodlagos close-hiba nem irja felul.
std::expected<std::string, std::error_code> read_file(const char* path)
{
    file_ptr f{std::fopen(path, "rb")};
    if (!f) return std::unexpected(errno_code(errno));

    std::string out;
    char buf[256];
    for (;;) {
        errno = 0;
```

```
        const std::size_t n = sys_fread(buf, 1, sizeof buf, f.get());
        const int saved = errno;                    // azonnal, a kovetkezo hivas
         ↪ elott
        out.append(buf, n);                         // dobhat: a file_ptr lezar
        if (n < sizeof buf) {
            if (sys_ferror(f.get()))                // reszadat + hiba: io hiba,
             ↪ nem siker
                return std::unexpected(errno_code(saved ? saved : EIO));
            break;                                  // EOF
        }
    }
    if (sys_fclose(f.release()) != 0)               // csak hiba nelkuli olvasas
     ↪ utan
        return std::unexpected(errno_code(errno ? errno : EIO));
    return out;
}

// Nyelvtan: soronkent "kulcs=ertek"; a kulcs a sor elejetol az elso '='-ig
// tart, szokozt nem tartalmazhat (nincs whitespace-levagas). Ures sor es '#'
// kezdetu megjegyzes megengedett. A kulcs csak teljes egyezessel talal;
// tobbszor elofordulo kulcs hiba (duplicate), hianyzo kulcs malformed.
std::expected<std::string_view, err>
field(std::string_view text, std::string_view key) noexcept
{
    std::string_view found;
    bool seen = false;
    while (!text.empty()) {
        std::size_t nl = text.find('\n');
        std::string_view line = text.substr(0, nl);
        text = (nl == std::string_view::npos) ? std::string_view{} :
 ↪  text.substr(nl + 1);
        if (line.empty() || line.front() == '#') continue;
        std::size_t eq = line.find('=');
        if (eq == std::string_view::npos || eq == 0) return
         ↪ std::unexpected(err::malformed);
        std::string_view k = line.substr(0, eq);
        if (k.find_first_of(" \t\r") != std::string_view::npos)
            return std::unexpected(err::malformed);
        if (k != key) continue;
        if (seen) return std::unexpected(err::duplicate);
        seen = true;
        found = line.substr(eq + 1);
    }
    if (!seen) return std::unexpected(err::malformed);
    return found;
}

std::expected<int, err> to_int(std::string_view s) noexcept
{
    int v{};
    auto [ptr, ec] = std::from_chars(s.data(), s.data() + s.size(), v);
    if (s.empty() || ec != std::errc{} || ptr != s.data() + s.size())
```

```
        return std::unexpected(err::malformed);    // ures ertek is ide tartozik
    return v;
}

struct config { int workers; };

std::expected<config, err> map_io(std::error_code ec) noexcept
{
    return std::unexpected(ec == std::errc::no_such_file_or_directory
                               ? err::missing : err::io);
}

std::expected<int, err> check_range(int v) noexcept
{
    if (v < 1 || v > 64) return std::unexpected(err::out_of_range);
    return v;
}

#if defined(CFG_MONADIC)
std::expected<config, err> load(const char* path)
{
    return read_file(path)
        .transform_error([](std::error_code ec) {
            return ec == std::errc::no_such_file_or_directory
                 ? err::missing : err::io;
        })
        .and_then([](std::string&& text) -> std::expected<config, err> {
            // a string_view a text-re mutat; a text a lambda vegeig el
            return field(text, "workers")
                .and_then(&to_int)
                .and_then(&check_range)
                .transform([](int v) { return config{v}; });
        });
}
#else
// Fallback: ugyanaz a szerzodes kezi elagazasokkal, monadikus fuggvenyek
 ↪  nelkul.
std::expected<config, err> load(const char* path)
{
    auto text = read_file(path);
    if (!text) return map_io(text.error());
    auto f = field(*text, "workers");
    if (!f) return std::unexpected(f.error());
    auto v = to_int(*f);
    if (!v) return std::unexpected(v.error());
    auto r = check_range(*v);
    if (!r) return std::unexpected(r.error());
    return config{*r};
}
#endif

const char* name(err e) noexcept
```

```
{
    switch (e) {
    case err::missing:      return "a fajl nem letezik";
    case err::io:           return "I/O hiba";
    case err::malformed:    return "hibas formatum";
    case err::out_of_range: return "ertek a tartomanyon kivul (1..64)";
    case err::duplicate:    return "tobbszor megadott kulcs";
    }
    return "ismeretlen hiba";
}

} // namespace cfg

int main(int argc, char** argv)
{
    if (argc != 2) {
        std::fprintf(stderr, "hasznalat: cfg FAJL\n");
        return 2;
    }
    auto r = cfg::load(argv[1]);
    if (!r) {
        std::fprintf(stderr, "cfg: %s: %s\n", argv[1], cfg::name(r.error()));
        return 1;
    }
    std::printf("workers=%d\n", r->workers);
    return 0;
}
```

Erőforrás- és hibapolicy: a `FILE*` a megnyitás után azonnal a `file_ptr` RAII-tulajdonosba kerül, így egy `out.append` közben dobott `std::bad_alloc` esetén is lezárul; a tárfoglalási hiba szándékosan kivétel marad, nem alakul `error_code`-dá. Az `errno`-t minden `fread` után azonnal mentjük; ha a rövid olvasás oka hiba (`ferror`), a részadat nem válik sikeres eredménnyé, hanem `io` hiba lesz. A lezárás hibáját csak hibátlan olvasás után jelentjük: egy elsődleges olvasási hibát a másodlagos `fclose`-hiba nem ír felül. A kulcsnyelvtan és a hibatípusok a `field` előtti kommentben vannak; a `main` hibánál a stderr-re ír és 1-es kóddal lép ki.

Élettartam-térkép: a `text` `std::string` tulajdonosa a `load` lambdája, a `field` által visszaadott `string_view` kölcsönzött nézet ugyanerre; a nézet érvényessége a lambda végéig tart, és nem hagyja el a láncot, mert a végső `config` csak `int`-et tartalmaz. Érvénytelenítő művelet nem történik, mert a `text`-et a lánc nem módosítja. Ha a `config` tagja `std::string_view` lenne, a lánc végétől dangling nézetünk maradna — ez a leggyakoribb tervezési hiba ebben a mintában.

Feature-test és build: a program elején a `<version>` után a `__cpp_lib_expected` dönt. Ha nincs `std::expected`, világos `#error` áll le; ha van, de 202211L alatti, a `load` kézi elágazásos fallbackje fordul; különben a monadikus lánc. A fallback-ág a `-DCFG_FORCE_FALLBACK` kapcsolóval monadikus felületet tudó profilban is lefordítható és tesztelhető.

```
# rogzitett laborprofilban, peldaul GCC 13+ vagy Clang 17+ mellett ertelmezheto
g++ -std=c++23 -O2 -Wall -Wextra -Wpedantic -o cfg cfg.cpp
# vagy
clang++ -std=c++23 -O2 -Wall -Wextra -Wpedantic -o cfg cfg.cpp
# a fallback-ág tesztje:
g++ -std=c++23 -O2 -Wall -Wextra -Wpedantic -DCFG_FORCE_FALLBACK -o cfg cfg.cpp
```

Buildjegyzőkönyv (`lab-gcc` profil). A monadikus és a fallback-ág is figyelmeztetés nélkül fordult, és azonosan viselkedett: `workers=8` → `workers=8`, 0-s kód; `notworkers=4 + workers=5` → `workers=5` (a `notworkers` nem talál); csak `notworkers=4` → `hibas formatum`; kétszeres `workers` → `tobbszor megadott kulcs`; üres érték, `workers=abc` és `workers = 3` → `hibas formatum`; `workers=99` → `ertek a tartomanyon kivul`; megjegyzés és üres sor mellett `workers=3`; hiányzó fájl → `a fajl nem letezik`. Minden hibaág 1-es kóddal lépett ki. Hibainjektálás (ASan/UBSan alatt, a `read_file`-t közvetlenül hívó tesztprogrammal): részadat + `EIO` → `Input/output error`; `fclose`-hiba (`ENOSPC`) hibátlan olvasás után → `No space left on device`; mindkettő együtt → `Input/output error` (az elsődleges hiba maradt meg); `std::bad_alloc` mind a 6 allokációs ponton injektálva, a nyitott fd-k száma minden esetben változatlan (3) maradt. `-std=c++20` mellett a fordítás a saját `#error` üzenettel állt le. A Clang-sort nem ellenőriztük.

41.1.5 Hibatípus tervezése: enum, `error_code` vagy doménstruct

Három reális választás van. Az `enum class` olcsó, másolható, kimerítő `switch`-cél; akkor jó, ha a hiba *fajtája* elég, a kontextus nem. A `std::error_code` a rendszerhatáron kötelező: két szó, olcsón másolható, kategóriával együtt hordozza a hibát, de üzleti jelentést nem ad. A doménstruct (`struct parse_error { err kind; std::size_t offset; }`) akkor kell, ha a hiba helye vagy a kontextus a diagnosztikához nélkülözhetetlen; ára a méret és a másolási költség.

Ajánlott minta: a modul belsejében `error_code`, a publikus API határán `transform_error`-ral domén típusra váltás, és a láncon belül `E` soha nem változik. Az `error_code` és a domén típus közötti oda-vissza konvertálást ne építsd a lánc közepére: minden váltás újabb hely, ahol a hibainformáció elveszhet.

41.1.6 Kivételhatár és `noexcept`

A szerződés two-way: `expected` a *számított*, rutinszerű hibákra való (hiányzó fájl, formátumhiba, megtelt puffer), kivétel a *szerződésszegésre* és a nem helyreállítható állapotra. Egy `noexcept` függvény nem engedheti ki a kivételt, ezért minden meghiúsulási módját a visszatérési típusban kell kifejeznie — a `noexcept` és az `expected` tehát természetes páros.

Konstruktorhiba esetén a konstruktor nem adhat vissza `expected`-et. Klasszikus minta: privát konstruktor és publikus `static std::expected<T, E> create(...)` gyárfüggvény, amely a validációt elvégzi, és csak érvényes objektumot konstruál. Ha a konstruktor mégis dob, a hibát a hívó modulhatáron `try`/`catch`-csel alakítsd `expected`-dé, és ott rögzítsd, mely kivételeket fordítasz hibakóddá.

A `value()` hibaágon `bad_expected_access<E>`-t dob, ezért kivételmentes függvényben vagy egy láncban csak úgy használható, ha ez a kivétel nem szökhet ki. A szabály pontosan: **egy `noexcept` függvényből nem szökhet ki kivétel** (különben `std::terminate`); a `value()` hívás önmagában nem tilos, ha bizonyított az értékállapot (`if (e.has_value()) return e.value();` vagy `*e` ellenőrzés után), illetve ha a kivételt lokálisan elkapjuk és hibává képezzük egy tudatos exception-to-error határon. Az `expected` használata sem jelenti automatikusan a teljes függvény kivételmentességét: a másolás/mozgatás és a lánc lépései dobhatnak, és a `noexcept` meg az `expected` két különböző szerződés (az egyik a kivétel kiszökését tiltja, a másik hibát vissza). Lánc közepén a `value()` mégis kerülendő, mert eldobja a hibaági információt.

41.1.7 Költség: méret, másolás, kódméret

Az `expected<T,E>` mérete legalább $\max(\texttt{sizeof}(T), \texttt{sizeof}(E))$, ehhez jön a diszkrimináns (jellemzően egy szó, a padding miatt néha ingyen), és az igazítás a két típus igazításmaximuma. A képlet mint alsó korlát értendő: a konkrét érték target ABI és fordítási profil függvénye, ezért mérés nélkül ne állíts számot. A laborban írj egy programot, amely kiírja a `sizeof`-okat, és hasonlítsd össze a `std::variant<T,E>` és a kézi `struct { bool ok; T v; E e; }` változattal.

Másolás: az `expected` másolása a benne lévő `T` vagy `E` másolását jelenti. A lánc végén ezért érdemes `std::move(exp).and_then(...)`-t használni, mert az rvalue-minősített hívás `T&&`-t ad át, és elkerüli a nagy `T` felesleges másolását. Kódméret: az `expected` jellemzően elágazásokká fordul, míg a kivételkezelés külön táblákat és unwinder-útvonalat igényel; a két minta kódméret-összehasonlítását szintén a laborra bízom (`size`, `nm --size-sort`, illetve a kiadott bináris összevetése), nem állítom mért eredményként.

41.1.8 Gyakori hibák és javításuk

1. **`transform` expected-et adó függvénnyel**: `expected<expected<T,E>,E>` lesz belőle. Javítás: `and_then`.
2. **Eltérő `E` a láncban**: az `and_then` nem konvertál, fordítási hiba. Javítás: a hibaidőtípusváltás `transform_error`-ral, egyetlen ponton.
3. **`error()` hívása értékágon, illetve `*r` vagy `r->` hibaágon**: nem definiált viselkedés. Javítás: előbb `has_value()`, vagy a `value()`, amely hibaágon `std::bad_expected_access`-t dob, és nem UB.
4. **`errno` késői olvasása**: a köztes `fclose`/`append` felülírhatja. Javítás: azonnali `errno_code()` csomagolás.
5. **`value()` egy `noexcept` határon**: kivétel szökik ki, `std::terminate`. Javítás: `has_value()`-ágazás vagy a hívó `expected`-en marad.
6. **`expected<void,E>` hívhatója egy paramétert vár**: fordítási hiba. Javítás: nulla paraméteres lambda.

41.1.9 Feladatok

1. feladat (típuslevezetés). Legyen `e : std::expected<int, NetErr>`, `f : int → std::expected<Bytes, NetErr>`, `g : Bytes → std::size_t`, `h : NetErr → std::error_code`. Írd fel a `e.and_then(f).transform(g).transform_error(h)` kifejezés pontos típusát, majd add meg, mi lenne a típus, ha a `transform(g)` helyett `transform(f)` állna, és indokold a különbséget!

2. feladat (kódolás). Írj `write_all(int fd, std::span<const std::byte>)` függvényt, amely `std::expected<std::size_t, std::error_code>`-t ad vissza, `EINTR` esetén újrapróbál, részleges írást helyesen kezel, és `noexcept`. Készíts hozzá egy láncot, amely a hibát `transform_error`-ral `enum class io_err { interrupted, no_space, other }` típusra képzi. Add meg a buildparancsot és az ellenőrzési feltételt!

3. feladat (diagnosztika). A következő függvényben legalább három hiba van. Nevezd meg őket, sorrendezd aszerint, hogy fordítási hibát vagy futásidejű hibát okoznak, és írd fel a javított változat láncát!

```
std::expected<config, cfg::err> load(const char* path) noexcept
{
    auto text = cfg::read_file(path);          // expected<string, error_code>
```

```
    return text
        .and_then([](const std::string& t) { return cfg::parse_workers(t); })
        .transform([](int v) { return config{v}; })
        .or_else([](cfg::err e) { return e; })
        .value();
}
```

41.1.10 Megoldások

1. megoldás. Lépésenként: `e.and_then(f)` típusa `std::expected<Bytes, NetErr>` (a hiba típusa változatlan, az érték `Bytes`). `.transform(g)` típusa `std::expected<std::size_t, NetErr>`. `.transform_error(h)` típusa `std::expected<std::size_t, std::error_code>`. A hibás változatban `e.transform(f)` típusa `std::expected<std::expected<Bytes, NetErr>, NetErr>`: a `transform` nem lapít, a belső `expected` értékként kerül a külsőbe, és a hibainformáció kétszeresen csomagolódik. Indoklás: a lapítás az `and_then` szerződése, ezért `expected`-et adó hívásra mindig `and_then` kell.

2. megoldás.

```
#include <cerrno>
#include <cstddef>
#include <algorithm>
#include <cerrno>
#include <expected>
#include <span>
#include <system_error>
#include <unistd.h>

enum class io_err { interrupted, no_space, other };

std::expected<std::size_t, std::error_code>
write_all(int fd, std::span<const std::byte> data) noexcept
{
    std::size_t done = 0;
    while (done < data.size()) {
        const std::size_t want = std::min<std::size_t>(data.size() - done, 1u <<
        ↪  20);  // darabolás
        const ssize_t n = ::write(fd, data.data() + done, want);
        if (n < 0) {
            if (errno == EINTR) continue;
            return std::unexpected(std::error_code{errno,
            ↪  std::system_category()});
        }
        if (n == 0)                                     // pozitív hátralevő
        ↪  hossz, nulla előrehaladás
            return std::unexpected(std::error_code{EIO,
            ↪  std::system_category()});
        done += static_cast<std::size_t>(n);
    }
    return done;
}
```

```
std::expected<std::size_t, io_err>
write_frame(int fd, std::span<const std::byte> frame) noexcept
{
    return write_all(fd, frame).transform_error([](std::error_code ec) {
        if (ec == std::errc::no_space_on_device) return io_err::no_space;
        if (ec == std::errc::interrupted)       return io_err::interrupted;
        return io_err::other;
    });
}
```

Indoklás: a `write_all` `noexcept`, mert minden hibát a visszatérési típusban fejez ki; az `errno`-t azonnal csomagolja, így a ciklus következő iterációja nem írhatja felül. A `done` nulla hosszú bemenetre `0`-t ad, ami érvényes siker. A nagy kérések darabolása (legfeljebb 1 MiB/hívás) beépített, így a `SSIZE_MAX` feletti implementációfüggő eset nem fordulhat elő. A nulla eredményű `write` végleges `EIO` hiba (a 12.1 no-progress szabálya), nem végtelen ciklus. Build: `g++ -std=c++23 -O2 -Wall -Wextra -Wpedantic -c io.cpp`. **Ellenőrzési feltétel:** zárt fd-re `unexpected`, `EBADF` kategóriával; 0-t visszaadó injektált `write` mellett hibával (`EIO`) tér vissza, nem pörög; `EINTR`-t szimuláló wrapper mellett a ciklus nem ad vissza hibát; nulla hosszú spanra `0` érték.

3. megoldás. Elsőként fordítási hibát okoz, hogy a `read_file` `E = error_code`, az `and_then` viszont `cfg::err` hibát adó függvényt kap; a lánc ezen a ponton nem konvertál. Ugyancsak fordítási hiba, hogy az `or_else` `cfg::err`-t ad vissza `expected` helyett. Futásidejű hiba a `value()`: a `noexcept` függvényben `bad_expected_access<cfg::err>`-t dobhat, ami `std::terminate`-hez vezet, és egyben elveszíti a hibaági információt. A `const std::string&` paraméter átvétele önmagában nem jelent másolást (a referencia kötése nem másol); a diagnózisba csak olyan másolás kerül, amely a kódban ténylegesen megtörténik, ezért itt a valódi hibák az `E`-típuseltérés és az `or_else` visszatérési típusa. Javított lánc: `read_file(path).transform_error(...)`, majd `and_then` a feldolgozásra, `transform` a `config`-ra, végül `return` az `expected`-en, `or_else` nélkül.

41.1.11 Összefoglaló

A `std::expected` rétegzett használatának három pillére van: (1) a láncon belül `E` végig azonos, a váltás `transform_error`-ral, egyetlen deklarált ponton történik; (2) `expected`-et adó hívásra `and_then`, sima értéktranszformációra `transform` kell, mert csak az előbbi lapít; (3) a `noexcept` határ és az `expected` ugyanazt a szerződést fejezi ki, ezért a `value()` sosem kerülhet egy `noexcept` láncba. Az `errno`-t azonnal csomagold, a doménhibát a modulhatáron alakítsd ki, és a lánc végén mozgatással dolgozz a felesleges másolás ellen. A C++23-as felület elérhetőségét feature-test és fordítási próba döntse el, ne a tankönyvi feltevés.

41.2 optional monadikus műveletei és hibamodellek kompozíciója

A hibamodell megválasztása ritkán nyelvi kérdés: legtöbbször az dönti el, hogy a hívó képes-e érdemben reagálni a kudarcra. Egy szolgáltatásban egyszerre vannak olyan pontok, ahol a „nincs ilyen kulcs” teljesen normális eset, és olyanok, ahol a hiba okát naplózni és a hívó felé stabil kóddá kell alakítani. Ez az alfejezet az `std::optional` monadikus műveleteire építve mutatja meg, hogyan lehet a hiányt és a hibát tudatosan szétválasztani, a rétegek határán információvesztés nélkül konvertálni, végül egy döntési táblával lezárni a négy hibamodell közötti választást. Az `std::expected` monadikus láncait és az `std::optional` alapjait korábbi részek tárgyalták (std::expected mélyebben: monadikus hibaláncok (41.1), optional, variant és visit (31.1),

error_code, system_error és expected (25.4)); itt a hangsúly a kompozíción és a határátlépéseken van.

41.2.1 Hiányzó érték és hiba szemantikai különbsége

Az `std::optional<T>` vagy tartalmaz egy `T` típusú objektumot, vagy nem tartalmaz semmit (`disengaged`, üres állapot). Fontos, hogy az üres állapothoz a típus **nem rendel okot**: az `optional` nem mondja meg, miért nincs érték. Előfeltétel: `T` objektumtípus, nem referencia és nem `void`, továbbá megsemmisíthető, mert az `optional` a benne tárolt objektum élettartamát kezeli.

Az `std::expected<T, E>` ezzel szemben vagy egy `T` értéket, vagy egy `E` hibát tartalmaz, ahol az `E` maga is érték, tehát hordozhat okot: hibakódot, kategóriát, kontextust. Előfeltétel: `T` nem `unexpected` specializáció, `E` nem referencia és nem `void`, és mindkettő megsemmisíthető.

A szemantikai különbség egyetlen kérdésre vezethető vissza: **a hívónak szüksége van-e az okra**. Ha a hiány önmagában is teljes válasz (cache miss, nem kötelező konfigurációs kulcs, opcionális mező), akkor az `optional` a helyes típus, mert nem kényszerít hamis indoklásra. Ha a hiány valamilyen kudarc következménye, és a kudarcot naplózni, újrapróbálni vagy a hívó felé leképezni kell, akkor az `optional` információt veszít. A gyakorlati hiba az szokott lenni, hogy egy réteg azért ad vissza `optional`-t, mert „egyszerűbb”, és ezzel a felette lévő réteg kénytelen kitalálni egy hibakódot — vagy ami rosszabb, csendben elnyelni a különbséget a „kulcs hiányzik” és a „kulcs megvan, de értelmezhetetlen” eset között.

41.2.2 Az optional monadikus műveletei

A C++23 három láncoló tagfüggvényt ad az `optional`-hoz; elérhetőségüket a `__cpp_lib_optional >= 202110L` feature-test makró jelzi (a konkrét értéket és a támogatást a laborprofilban kell ellenőrizni, ez a rész nem állítja, hogy bármely fordító támogatja).

- **`and_then(f)`**: ha az `optional` üres, az eredmény üres `optional<U>` és `f` **nem hívódik meg**; ha tartalmaz értéket, az eredmény `f(value)`, amelynek magának is `optional<U>` specializációnak kell lennie. Ez a „ha van érték, akkor folytasd egy újabb esetleg-sikertelen lépéssel” művelet, és a lánc az első üres ponton megáll (short-circuit).
- **`transform(f)`**: ha üres, üres; ha tartalmaz, az eredmény `optional<U>`, ahol `U` az `f(value)` visszatérési típusa. Ha `f` maga is `optional`-t ad vissza, az eredmény `optional<optional<U>>` lesz — ez a leggyakoribb félreértés, mert a lánc nem simul ki. Referenciát visszaadó callable esetén a kifejezés nem fordul le, mert az `optional` nem tárolhat referenciát.
- **`or_else(f)`**: csak üres állapotban hívja meg az argumentum nélküli `f`-et, és az eredmény típusa kötelezően `optional<T>`, tehát a típus nem változhat. Ez lazán kiértékelt fallback ugyanarra a típusra. Értéket előállító fallbackhez nem ez való: arra a `value_or` szolgál, amely viszont **mohó**, tehát az argumentuma akkor is kiértékelődik, ha az `optional` tartalmaz értéket.

Az értékkategória számít: lvalue `optional` esetén a callable `T&`-et, rvalue esetén `T&&`-t kap, ami eldönti, hogy a lánc másol-e. A `string_view`-t visszaadó láncoknál ez különösen fontos, mert a nézet nem tartja életben a mögötte lévő puffert.

Az alábbi példa önállóan fordítható. A `raw` szándékosan `optional`-t ad vissza (a hiányzó kulcs nem hiba), a `require_port` viszont már `expected`-et, mert ott a hívó megkülönbözteti a két kudarcot.

```cpp
// cfg.cpp - ISO C++23
#include <charconv>
#include <cstdint>
#include <map>
#include <optional>
#include <string>
#include <string_view>
#include <version>

#if !defined(__cpp_lib_optional) || __cpp_lib_optional < 202110L
#error "Az optional monadikus muveletei nem elerhetok ebben a korlabban."
#endif

using Config = std::map<std::string, std::string, std::less<>>;

// A Config tulajdonol; a visszaadott view addig ervenyes, amig a Config el
// es nem modositjak (beszuras torolheti a map elemeit).
std::optional<std::string_view> raw(const Config& c, std::string_view key) {
    auto it = c.find(key);
    if (it == c.end()) return std::nullopt;
    return std::string_view{it->second};
}

std::optional<int> to_int(std::string_view s) {
    int v{};
    const char* b = s.data();
    const char* e = b + s.size();
    auto [p, ec] = std::from_chars(b, e, v);
    if (ec != std::errc{} || p != e) return std::nullopt;
    return v;
}

std::optional<std::uint16_t> as_port(int v) {
    if (v < 1 || v > 65535) return std::nullopt;
    return static_cast<std::uint16_t>(v);
}

// Lanc: az elso ures lepes megallitja a tovabbi hivasokat.
std::optional<std::uint16_t> listen_port(const Config& c) {
    return raw(c, "port").and_then(to_int).and_then(as_port);
}

// Lazy fallback ugyanarra a tipusra; a lambdaba csak ures esetben lep be.
std::optional<std::uint16_t> listen_port_or_default(const Config& c) {
    return listen_port(c).or_else([&c] {
        return raw(c, "default_port").and_then(to_int).and_then(as_port);
    });
}

// HIBA-pelda (nem fordul ugy, ahogy a szerzoje hiszi):
// auto x = raw(c, "port").transform(to_int);  // optional<optional<int>>
```

Fordítás és a szükséges feature-ellenőrzés:

```
# A laborban rogzitett GCC/Clang kiadassal; a -std=c++23 explicit.
g++ -std=c++23 -Wall -Wextra -Wpedantic -O2 -o cfg cfg.cpp
```

Ha a monadikus műveletek nem elérhetők, a fallback kézi láncolás: `if (auto a = raw(...)) { if (auto b = to_int(*a)) { ... } }`. Ez ugyanazt a viselkedést adja, csak hosszabban.

41.2.3 Konverzió optional, expected és kivétel között a rétegek határán

A határátlépés szabálya: **minden konverzió nevezze meg, mit tart meg és mit dob el szándékosan.** Amit eldobunk, azt a határon naplózni kell, különben a diagnosztika a legérdekesebb esetben lesz vak.

`optional` → `expected`: ez az a pont, ahol a réteg dönt az okról, tehát itt kell szétválasztani a „nincs” és a „van, de rossz” esetet.

```
#include <expected>

enum class DomainError { missing_key, bad_value };
enum class PublicCode : int { ok = 0, invalid_input = 2, unavailable = 14,
 ↪  internal = 70 };

std::expected<std::uint16_t, DomainError> require_port(const Config& c) {
    auto r = raw(c, "port");
    if (!r) return std::unexpected(DomainError::missing_key);
    auto v = to_int(*r).and_then(as_port);
    if (!v) return std::unexpected(DomainError::bad_value);
    return *v;
}
```

`expected` → `optional`: csak akkor, ha a hiba máshol már gazdára talált (naplózva vagy egy másik csatornán visszaadva). Itt az ok **szándékosan** vész el; ezt kommentben vagy a függvény nevében jelezni kell.

`kivétel` → `expected`: a `noexcept` határon (C API callback, szál-belépési pont, FFI) kötelező. A `catch (...)` és a `catch (const std::exception&)` között a különbség az, hogy az utóbbi üzenetet ad, de az üzenet nem stabil kód — a naplóba kerül, a visszatérési értékbe nem.

`expected` → `kivétel`: a `value()` hívása `std::bad_expected_access<E>`-t dob, ha nincs érték; ez kizárólag a legkülső rétegben (pl. `main`) elfogadható, ahol már van kivételpolitika. Figyelem: az `error()` előfeltétele, hogy `has_value()` hamis legyen — ellenkező esetben a viselkedés nem definiált, tehát `error()`-t csak ágban szabad hívni. Ugyanígy az `optional::value()` üres állapotban `std::bad_optional_access`-t dob, a `*o` viszont nem definiált viselkedés.

`error_code`: a `std::error_code` kategóriafüggő, ezért `ec == std::errc::invalid_argument` alakban kell összehasonlítani, nem `ec.value() == EINVAL` formában; a POSIX `errno` rendszerkategórián keresztül érkezik, és a kettő közötti leképezés nem automatikus.

41.2.4 Hibák gyűjtése ranges pipeline-ban

A ranges algoritmusoknak nincs hibacsatornájuk: ha a `transform` egy `expected`-et ad vissza, a tartomány elemei `expected`-ek lesznek, és a pipeline maga nem áll meg. Két legitim stratégia van, egy pedig anti-minta.

- **Fail fast**: `std::ranges::find_if` a `!has_value()` predikátummal megadja az első hibát, majd a hívó visszatér vele. Itt a hiba oka megmarad, és a többi elem kiértékelése elmarad.
- **Összegyűjtés**: ha az elemek függetlenek, egyszeri bejárással szétválaszthatók sikeres értékekre és hibákra, a hiba mellé a bemenetet is téve.
- **Anti-minta**: `views::filter` a `has_value()`-ra, majd a hibák elhagyása. Ez csendben részleges eredményt ad, ami a legdrágább hibaosztály: a futás „sikeres”, az adat hiányos.

Egy figyelmeztetés a nézetekkel: a `views::transform(parse_one)` nézet nem tárolja el az eredményeket, így minden bejárás újraszámol; mellékhatásos callable-lel ez meglepő viselkedést ad. Gyűjtéshez ezért egyszeri ciklus a biztonságos választás.

41.2.5 Szolgáltatásszintű hibakód-leképezés naplózáshoz

A naplózás és a nyilvános válasz két külön csatorna. A belső részlet (kulcsnév, parse-hiba helye, `errno`) a naplóba kerül, a hívó stabil, dokumentált kódot kap. A leképezés legyen **teljes**: minden belső hibaérképre legyen ág, és a maradék essen az `internal` kategóriába, ne forduljon elő, hogy egy ismeretlen hiba „sikerként” szivárog tovább. Az újrapróbálhatóságot (`retryable`) is a leképezés döntse el, ne a hívó találgassa az üzenetszövegből.

```
struct Failure {
    PublicCode code;
    bool retryable;
    std::string detail;   // csak naploba; a publikus valaszba nem kerul
};

Failure to_public(DomainError e, std::string_view key) {
    switch (e) {
    case DomainError::missing_key:
        return {PublicCode::invalid_input, false, "hianyzo kulcs: " +
         ↪  std::string{key}};
    case DomainError::bad_value:
        return {PublicCode::invalid_input, false, "ertelmezhetetlen ertek: " +
         ↪  std::string{key}};
    }
    return {PublicCode::internal, false, "ismeretlen DomainError"};
}
```

A `missing_key` itt azért lesz `invalid_input`, mert a konfigurációs rétegben egy kötelező kulcs hiánya a kérés hibája — ez rétegdöntés, nem nyelvi tény. A naplózás pontosan egyszer, a határon történjen; a lánc belsejében naplózott és továbbadott hiba duplikált bejegyzéseket és torz metrikákat ad.

41.2.6 Döntési tábla a négy hibamodellre

Modell	Mit fejez ki	Mikor válaszd	Határviselkedés	Információvesztés
`optional<T>`	lehet, hogy nincs érték; ok nélkül	opcionális mező, cache miss, „nem található”	határon `expected`-dé kell alakítani, ha ok kell	az ok teljes egészében elvész
`expected<T,E>`	érték vagy tipizált hiba	domain- és szolgáltatáslogika, tesztelhető hibautak	`E`-t publikus kódra képezni; `value()` csak a legkülső rétegben	csak ha szándékosan szűkítünk
kivétel	kivételes, ritka kudarc, unwindinggal	konstruktor, allocation, mély hívási lánc	`noexcept` határon `expected`-dé alakítani	az üzenet nem stabil kód
`std::error_code`	OS/könyvtár hibakód kategóriával	C API, POSIX, FFI, visszafelé kompatibilis interfész	saját kódra leképezni; kategóriát őrizni	kategória nélkül igen, azzal kevés

A tábla olvasata: a négy modell nem egymás versenytársa egyetlen rétegen belül, hanem rétegenkénti választás. A tipikus helyes architektúra: `error_code`/`errno` a legalul, `expected` a domain rétegben, `optional` a valóban opcionális lekérdezéseknél, kivétel pedig csak ott, ahol a kudarc tényleg kivételes és van felső szintű kezelő.

41.2.7 Gyakori hibák

- `transform` olyan callable-lel, amely `optional`-t ad: `optional<optional<U>>` lesz, a lánc nem simul ki. Javítás: `and_then`.
- `or_else`-t használnak érték előállítására vagy típusváltásra; ez nem fordul le, mert a visszatérési típusnak `optional<T>`-nek kell lennie. Javítás: `transform` + `value_or`, illetve kézi ág.
- `value_or`-t lazának hiszik; az argumentum mohón kiértékelődik. Javítás: `or_else` nullary lambdával.
- Üres `optional`-on `*o` (nem definiált viselkedés) vagy `expected::error()` értékkel rendelkező objektumon (előfeltétel-sértés). Javítás: `has_value()` ág, illetve `value()` ott, ahol a kivétel elfogadható.
- A határon a hiba okát naplózás nélkül dobják el, így a „kulcs hiányzik” és a „kulcs rossz” ugyanazzá válik a diagnosztikában.
- `string_view`-t tartalmazó `optional`-t él tovább, mint a mögötte lévő `Config`: a lánc helyes, a nézet dangling.

41.2.8 Összefoglaló

Az `optional` a hiányt fejezi ki, nem a kudarcot; a kettő szétválasztása API-tervezési döntés, amelyet a hívó reagálóképessége vezérel. Az `and_then` láncol, a `transform` leképez, az `or_else` lazán pótol — a rossz pár választása vagy nem fordul le, vagy néma típusbeágyazást okoz. A réteghatáron minden konverzióhoz tartozik egy mondat arról, mi vész el, és ami elvész, azt naplózni kell. A ranges pipeline-ban a hibát vagy az első előfordulásnál állítjuk meg, vagy tudatosan gyűjtjük; a filterezés csendben részleges eredményt ad. A szolgáltatásszintű leképezés teljes, stabil és egy helyen naplóz.

41.2.9 Feladatok

1. **Magyarázó.** Egy konfigurációs olvasó `optional<std::string>`-et ad vissza arra, hogy a kulcs hiányzik, az érték viszont érvénytelen. Sorolj fel két helyet, ahol ez információvesztést okoz, és fogalmazd meg, milyen típusra cserélnéd.
2. **Kódolási.** Írj `std::optional<int>`-et adó láncot, amely egy `Config`-ból kiolvassa a `timeout_ms` kulcsot, egész számmá alakítja, majd 1 és 60000 közé szorítja úgy, hogy a tartományon kívüli érték üres eredményt adjon. Készíts `expected<int, DomainError>` változatot is, ahol a „hiányzik” és a „rossz” megkülönböztethető.
3. **Diagnosztikai.** Az alábbi lánc nem azt teszi, amit a szerzője állít. Nevezd meg a hibát, írd le a tünetet, és javítsd: `auto v = raw(c, "port").transform(to_int); if (v) use(*v);`
4. **Tervezési.** Adj `Failure`-t visszaadó leképezést egy `expected<T, std::error_code>`-et adó Alsó függvényhez, ahol a `std::errc::timed_out` újrapróbálható, minden más nem az, és a belső részlet csak a naplóba kerül.

41.2.10 Megoldások

1. Először ott, ahol a hívó a felhasználónak üzenetet kell hogy adjon: „hiányzó kulcs” és „érvénytelen érték” ugyanarra az ágra fut, így a hibaüzenet félrevezető. Másodszor ott, ahol a hiba alapján újrapróbálna (pl. a fájl még nem íródott ki): az üres `optional` nem hordozza, hogy érdemes-e újrapróbálni. Csere: `std::expected<std::string, ConfigError>`, ahol `ConfigError` legalább `missing_key` és `invalid_value` értéket vesz fel. Ellenőrzés: a `switch` a hibára teljes, és nincs `default`, amely elnyelné az új értékeket.

2. Az `optional` lánc:

```
std::optional<int> timeout_ms(const Config& c) {
    return raw(c, "timeout_ms").and_then(to_int).and_then([](int v) ->
     ↪  std::optional<int> {
        if (v < 1 || v > 60000) return std::nullopt;
        return v;
    });
}
```

Az `expected` változatban a `raw` hiányát és a `to_int`/tartomány-hibát külön ágra kell bontani, mert a lánc csak az egyik okot tudná visszaadni:

```
std::expected<int, DomainError> timeout_ms_checked(const Config& c) {
    auto r = raw(c, "timeout_ms");
    if (!r) return std::unexpected(DomainError::missing_key);
    auto v = to_int(*r);
    if (!v || *v < 1 || *v > 60000) return
     ↪  std::unexpected(DomainError::bad_value);
    return *v;
}
```

Ellenőrzés: három bemenet (hiányzó kulcs, `"abc"`, `"70000"`) rendre `missing_key`, `bad_value`, `bad_value`; a `"443"` értékre mindkét változat `443`-at ad. A határértékek (`1`, `60000`) elfogadottak, a `0` és `60001` nem.

3. A hiba az, hogy a `to_int` `std::optional<int>`-et ad vissza, a `transform` ezt becsomagolja, így a `v` típusa `std::optional<std::optional<int>>`. A tünet: a `if (v)` igaz, ha a külső `optional` tartalmaz — azaz ha a `raw` értéket adott —, miközben a belső

üres lehet; a `*v` egy üres `optional<int>`-et ad, amit a `use` már nem tud értékként kezelni, vagy némán 0-t/`nullopt`-ot továbbít. Javítás: `and_then(to_int)`, ekkor a lánc az első üres ponton megáll, és `v` típusa `std::optional<int>`. Ellenőrzés: a `decltype` próba (`static_assert(std::is_same_v<decltype(v), std::optional<int>>)`) és a `"abc"` bemenetre kapott üres eredmény.

4. A leképezés:

```
Failure to_public(const std::error_code& ec) {
    if (ec == std::errc::timed_out)
        return {PublicCode::unavailable, true, "also reteg: idotullepes"};
    if (ec == std::errc::invalid_argument)
        return {PublicCode::invalid_input, false, "also reteg: ervenytelen
        ↪ argumentum"};
    return {PublicCode::internal, false, "also reteg: " + ec.message()};
}
```

Az indoklás: a `std::errc` összehasonlítás kategóriaérzékeny, ezért `ec == std::errc::...` alak kell; a `detail` a belső kategóriaüzenetet is tartalmazhatja, mert nem kerül a publikus válaszba; az ismeretlen kód `internal` lesz, nem „ok”. Ellenőrzés: a `retryable` csak a `timed_out` ágon igaz; a naplózás a határon egyszer történik, és a publikus struktúrában nincs `std::string` mező, amely a belső okot kiszivárogtatná.

41.3 std::print, std::println és tartományok formázása

A charconv, format és C++23 print alfejezet rögzítette a `std::format` szerződését és a `std::print` megjelenését. Itt a gyakorlati viselkedést vesszük végig: milyen célra ír a `print`, hogyan viselkedik terminálon és átirányított kimeneten, hogyan formázunk tárolókat és tuple-öket, és milyen csapdát jelent a `printf`-fel való keverés. A pufferelés szabályait a stdio, FILE és pufferelés és az iostream, stream state és pufferelés alfejezetekből ismertnek vesszük; itt csak a `print` szempontjából alkalmazzuk őket.

Fontos keret: az alapprofil `-std=c++23`, de a `<print>` és a range-formázás elérhetősége a standard library kiadásától függ. A könyv nem állítja, hogy bármely konkrét GCC/Clang/libstdc++/libc++ verzió támogatja ezeket; minden képességhez feature-test makrót és compile probe-ot adunk.

41.3.1 A print és println szerződése

Definíció (ISO C++23, `<print>`). A `std::print` és `std::println` függvénysablonok a formázott szöveget egy `std::FILE*` streame írják. A releváns túlterhelések:

```
template<class... Args>
void print(std::FILE* stream, std::format_string<Args...> fmt, Args&&... args);
template<class... Args>
void print(std::format_string<Args...> fmt, Args&&... args);
template<class... Args>
void println(std::FILE* stream, std::format_string<Args...> fmt, Args&&...
 ↪ args);
template<class... Args>
void println(std::format_string<Args...> fmt, Args&&... args);
```

Feltételek. (1) `fmt`-nek konstans kifejezésnek kell lennie; ha nem az, a program ill-formed, diagnosztika kötelező. (2) A `fmt` érvényességét `Args...`-ra a fordító ellenőrzi. (3) A stream nélküli túlterhelés hatása `print(stdout, fmt, ...)`; a `println` ugyanez plusz egy `'\n'` karakter. (4) A `print(stream, ...)` hatása a szabvány szerint `vprint_unicode(stream, fmt.get(), std::make_format_args(args...))` — ezt a konkrét megfogalmazást a saját szabványpéldányodon ellenőrizd, mert a `vprint_unicode` és `vprint_nonunicode` közti választás a megvalósítás és a stream jellege szerint történik.

Nincs `ostream` túlterhelés. A C++23 `print` nem ír `std::ostream`-re. Ha streamre kell írni, a helyes minta:

```
std::ostringstream os;
os << std::format("vector: {}\n", std::vector{1, 2, 3});
```

Ez a `format` + `operator<<` kombináció: a formázás `std::string`-be megy, a kiírás a stream bufferébe. A `print` közvetlen `FILE*`-ra íráskor elkerüli ezt a köztes allokációt.

Feature-test és compile probe. A `__cpp_lib_print` és a `__cpp_lib_format_ranges` értékét a `<version>` fejlécből kell kiolvasni; a konkrét revízió nem tekinthető igazoltnak.

```
// probe_print.cpp
#include <version>
#ifndef __cpp_lib_print
#error "std::print nem elerheto"
#elif __cpp_lib_print < 202207L
#error "std::print revizioja regi"
#endif
#ifndef __cpp_lib_format_ranges
#error "range formatting (P2286) nem elerheto"
#endif
#include <print>
int main() { std::println("probe: {}", 42); }
```

```
g++ -std=c++23 -Wall -Wextra -o probe_print probe_print.cpp
# régebbi driveren: -std=c++2b
```

Ha a probe nem fordul, fallback: `std::format` + `std::fwrite`, illetve `printf`. A fallback viselkedése azonos, csak a kényelem és a fordítási idejű ellenőrzés vész el.

41.3.2 Unicode kimenet és a két vprint függvény

A `std::print` mögötti két alacsony szintű függvény:

- `std::vprint_unicode(FILE*, std::string_view, std::format_args)` — ha a stream Unicode-ot megjeleníteni képes terminálra mutat, a formázott szöveget a platform natív Unicode API-ján írja ki (előtte flush-olja a streamet); egyébként változatlanul a streambe írja.
- `std::vprint_nonunicode(FILE*, std::string_view, std::format_args)` — mindig változatlanul a streambe írja a bájtokat.

A `std::print` a `vprint_unicode`-ot csak akkor használja, ha az *ordinary literal encoding* UTF-8; egyébként a `vprint_nonunicode`-ot.

Következmény: négy külön eset.

1. **Literálkódolás.** A formázott string bájtjai a program saját kódolásában vannak: a literálok az *ordinary literal encoding* szerint, a futásidőben bekerülő adat (fájl, hálózat) pedig abban, amiben érkezett. Előfeltétel: a Unicode útvonalon az adatnak érvényes UTF-8-nak kell lennie; érvénytelen kódegységnél a viselkedés nem definiált (az implementációk U+FFFD helyettesítést javasolt gyakorlatként alkalmazhatnak).
2. **POSIX terminál.** Itt nincs külön natív Unicode API: a `print` a bájtokat változatlanul írja, transzkódolás nincs; a helyes megjelenés azon múlik, hogy a terminál és a locale is UTF-8.
3. **Windows konzol.** A natív Unicode API (a konzol UTF-16-os írófüggvénye) használatakor a könyvtár a UTF-8 szöveget **transzkódolja**; ez az egyetlen eset, ahol a `print` kódolást alakít.
4. **Átirányított kimenet (fájl, pipe).** Nem terminál, ezért a bájtok változatlanul kerülnek a fájlba, a terminál kódolásától függetlenül.

Ha a program és a megjelenítő kódolása eltér, a megjelenés hibás lesz, a program viszont nem jelez hibát.

Hibakezelés. A szabvány (C++23 [print.fun]) mindkét függvény dobásait azonosan írja le, a használt ágtól függetlenül: a `vformat` által dobott kivétel (`std::format_error`), **`std::system_error`, ha a terminálra vagy a streamre írás sikertelen**, és esetleg `std::bad_alloc`. Az írási hibát tehát kivétel jelzi; a C stream `ferror` állapota ettől függetlenül beállhat, de nem helyettesíti a kivételkezelést. Pufferelt streamnél a hiba csak akkor jelentkezik a `print` hívásában, ha az írás ténylegesen ott történik meg; egy később kiürülő pufferé a `fflush`/`fclose` visszatérési értékén látszik.

```cpp
// print_error.cpp - C++23: a print írási hibája kivétel, nem csak ferror
#include <cstdio>
#include <print>
#include <system_error>

int main() {
    // A stdout-ot pufferezetlenre állítjuk, hogy az írási hiba a print
    // hívásán belül jelentkezzen, ne egy későbbi flush-nál.
    std::setvbuf(stdout, nullptr, _IONBF, 0);
    try {
        std::println("hello {}", 42);
    } catch (const std::system_error& e) {
        std::fprintf(stderr, "print: system_error: %s\n",
         ↪  e.code().message().c_str());
        return 1;
    }
    return 0;
}
```

Buildjegyzőkönyv. A GCC 13.3 libstdc++-a a `<print>`-et még nem nyújtja; a Clang 22 / libc++ (`__cpp_lib_print == 202207L`) `-std=c++23 -Wall -Wextra -Wpedantic` alatt figyelmeztetés nélkül fordította. Terminálra és pipe-ba `hello 42`, 0-s kód; `/dev/full`-ra `print: system_error: Operation not permitted`, 1-es kód — az írási hibát a könyvtár kivétellel jelezte (a hibakód szövege implementációs részlet).

41.3.3 Range, tuple és tároló formázása (P2286)

A P2286 („Formatting Ranges”) a `std::formatter`-t kiterjeszti range-ekre, pair-re és tuple-re. A formátum-specifikáció `?` (debug) opciót is elfogadnak a string- és char-formatterek, valamint a

range-formatterek. A range-formázás alapértelmezésben szögletes zárójelbe tesz, `", "` elválasztóval, és a string elemeket **idézőjelbe teszi**; a `?` a nem nyomtatható karakterek escape-elését kapcsolja be.

```
// demo_print.cpp
#include <print>
#include <vector>
#include <map>
#include <tuple>
#include <string>
#include <utility>

int main() {
    std::vector<int> v{1, 2, 3};
    std::println("vector: {}", v);                    // [1, 2, 3]

    std::vector<std::string> names{"alice", "bob"};
    std::println("names: {}", names);                 // ["alice", "bob"]

    auto t = std::make_tuple(1, 2.5, std::string{"x"});
    std::println("tuple: {}", t);                     // (1, 2.5, x)

    std::map<std::string, int> m{{"a", 1}, {"b", 2}};
    std::println("map: {}", m);                       // {a: 1, b: 2}

    std::vector<std::vector<int>> nested{{1}, {2, 3}};
    std::println("nested: {}", nested);               // [[1], [2, 3]]
}
```

```
g++ -std=c++23 -O2 -Wall -Wextra -o demo_print demo_print.cpp
```

A megjegyzésben szereplő kimenet **illusztráció**, nem mért vagy tesztelt eredmény; a saját toolchainen kell ellenőrizni.

Debug-formátum `?`. Stringre közvetlenül: `std::format("{}", "hi")` → `hi`, `std::format("{:?}", "hi")` → `"hi"`. A `?` típusjelölő csak a karakter- és stringformatter szerződésében szerepel: `std::format("{:?}", 1)` vagy egy `map`-re írt `{:?}` nem hatástalan, hanem fordítási hiba, mert a konstans formátumsztring ellenőrzése elutasítja. A range-ek nem öröklik a `?`-t; ellenkezőleg, a range-formatter a karakter- és string-elemeket alapértelmezésben eleve debug-formátumban írja (`["x\ty"]`), és az elemformátumot a második kettőspont után lehet felülírni: a `{::}` üres elemformátummal a nyers szöveget adja. A formatter szerződése típusonként dönt.

Fordítási idejű ellenőrzés. A `format_string<Args...>` konstans kiértékelése miatt a hibás specifikáció és a nem formázható argumentumtípus fordításidejű hiba. Futásidejű formátumsztring C++23-ban csak `std::vformat` + `std::make_format_args` útvonalon lehetséges, ellenőrzés nélkül. A `std::runtime_format` (P2918) **C++26**, itt nem használjuk.

41.3.4 Pufferelés, kevert kimenet és teljesítmény

A `print` `FILE*`-ra ír, tehát ugyanabba a `stdout` pufferbe kerül, mint a `printf`, ha a `fwrite` útvonalon halad. A `std::cout` alapértelmezett `sync_with_stdio(true)` mellett szintén a `stdout` pufferét használja. A sorrend tehát **elvileg** konzisztens — a gyakorlatban mégsem mindig az. Levezetés:

1. A `printf` kimenete a `stdout` `FILE` pufferébe kerül.
2. Terminálon a glibc alapértelmezésben sorbufferel (`'\n'`-nél flush); csőben és fájlban blokkolt puffert használ.
3. Ha a `print` Unicode-képes terminálon a natív API-t választja, a bájtok **közvetlenül a file descriptorra** mennek, megkerülve a `FILE` puffert.
4. Ha az előző `printf` nem végződött `'\n'`-nel, bájtjai még a pufferben vannak.
5. A natív írás most kimegy, a puffertartalom csak később → a megjelenő sorrend megfordul.

A javítás nem varázslat, hanem szerződés: vagy egy mechanizmust használj, vagy a váltás előtt hívd meg a `std::fflush(stdout)`-ot. A `std::print` önmagában nem flushol; a `println` új sora csak sorbufferelt terminálon vált ki flushot.

Teljesítmény. A könyv nem mér; az alábbi táblázat a szerződésekből levezetett költségforrásokat mutatja, nem mért számokat.

útvonal	formátum-ellenőrzés	köztes allokáció	tipikus költségforrás
`printf`	nincs	nincs	futásidejű parse, varargs
`std::cout <<`	nincs	nincs	elemenkénti hívás, locale, virtualitás
`std::format` → `std::string`	fordítási idő	igen	parse + string-allokáció
`std::print(FILE*)`	fordítási idő	implementációfüggő	parse + write

A `print` a `format` futásidejű parse-át is elvégzi (a validáció fordítási idejű, a tényleges előállítás nem), viszont elkerüli a `std::string` allokációt, ha közvetlenül `FILE*`-ra ír. Az `iostream` számonkénti túlterhelése numerikus terhelésnél jellemzően érzékelhető; a `printf` és a `print` viszonya erősen függ a formátumspecifikációtól és az implementációtól. Bármilyen összehasonlítást a tanuló saját laborjában, azonos kimenettel, a mérésen kívüli flushel és `std::chrono::steady_clock`-kal kell elvégezni.

41.3.5 Gyakori hibák

- **Futásidejű formátumsztring átadása.** C++23-ban ill-formed; `std::vformat` kell, vagy C++26 `runtime_format`.
- **`print`-et `ostream`-re hívni.** Nincs ilyen túlterhelés; `std::format` + `operator<<`.
- **Azt hinni, hogy a `print` flushol.** Nem; terminálon a sorbufferelés ad látszatot.
- **`printf` és `print` keverése Unicode terminálon.** A natív útvonal megkerüli a `FILE` puffert.
- **A `{:?}`-t minden típusra univerzálisnak tekinteni.** A debug-viselkedés a formatter szerződése.
- **A `<print>` fejléc elhagyása és `printf`-re hagyatkozás.** A `format` és a `print` két külön eszköz.

41.3.6 Összefoglaló

A `std::print` és `std::println` `FILE*` célra ír, `fmt` konstans kifejezés, a formátum ellenőrzése fordítási idejű. `ostream`-re nincs túlterhelés. Terminálon a `vprint_unicode`, máshol a `vprint_nonunicode` (mindenhol `fwrite`) útvonal érvényes; a kódolásváltás nem a `print` dolga.

A P2286 range-formázást ad: `[...]`, `(...)`, `{kulcs: érték}`, a stringek idézőjelben, a `?` escape-el. A `printf`-fel való keverés csak flush-sal determinisztikus. A teljesítményt mérni kell, nem kijelenteni.

41.3.7 Feladatok

1. (kódolás, alap). Írj programot, amely egy `FILE*`-ra `std::print`-tel és `std::fprintf`-fel felváltva ír, és a fájl bezárása előtt nem hív mást. Magyarázd meg, miért determinisztikus a sorrend, és mi történne, ha `stdout` terminálra írnál Unicode-képes környezetben.

2. (kódolás, közepes). Formázz egy `std::map<std::string, std::vector<std::pair<int, int>>>` értéket `{}` és `{:n}` specifikátorral, és indokold, miért nem fordul a `{:?}`. Adj egy `std::vector<std::string>` példát, ahol a debug- és a nyers elemformátum láthatóan különbözik.

3. (diagnosztika, haladó). Készíts harness-t, amely `printf`, `std::cout <<` és `std::format` + `std::fwrite` útvonalon kiír N egész számot, és `steady_clock`-kal méri az időt. Sorolj fel legalább négy érvényességi feltételt, amely nélkül a mérés félrevezető.

41.3.8 Megoldások

1. megoldás. `FILE* f = std::fopen("out.txt", "w")` után a `std::print(f, ...)` a `fwrite` útvonalon halad (a fájl nem terminál), tehát ugyanabba a `FILE` pufferbe ír, mint a `fprintf`; így a sorrend a hívások sorrendje, és a `std::fclose` flushol. `stdout`-ra Unicode-képes terminálon a `print` a natív API-t választhatja, ami megkerüli a `FILE` puffert; a korábbi `fprintf` bájtjai még a pufferben lehetnek, ezért a látszólagos sorrend felcserélődhet. Ellenőrzési feltétel: a `fopen` visszatérési értékét ellenőrizd, és a fájlt `fclose`-zal zárd.

2. megoldás.

```
#include <print>
#include <map>
#include <string>
#include <utility>
#include <vector>

int main() {
    std::map<std::string, std::vector<std::pair<int, int>>> m{
        {"a", {{1, 2}, {3, 4}}},
        {"b\tc", {{5, 6}}}
    };
    std::println("{}", m);     // {"a": [(1, 2), (3, 4)], "b\tc": [(5, 6)]}
    std::println("{:n}", m);   // ugyanez a külső kapcsos zárójelek nélkül
    // std::println("{:?}", m);   // nem fordul: a range-formatter nem ismeri a
     ↪ ?-t

    std::vector<std::string> v{"x\ty"};
    std::println("{}", v);     // ["x\ty"]: string elem alapból
     ↪ debug-formátumban
    std::println("{::}", v);   // [x<TAB>y]: üres elemformátum, nyers tabulátor
}
```

A `{}` a string kulcsokat már debug-formátumban, idézőjellel és escape-elve (`"b\tc"`), a pair-eket `(1, 2)` alakban adja; az `n` a külső zárójeleket hagyja el. A `{:?}` a map-re fordítási hiba: a range-formatter specifikációja nem tartalmaz `?` típust, a `?` csak a karakter- és stringformatteré. A debug és a nyers forma különbsége a `vector<string>`-en látszik: `{}` → `["x\ty"]`, `{::}` → a tabulátor nyersen. Buildjegyzőkönyv: Clang 22 + libc++ 22, `-std=c++23`, a kimenet a kommentekben szereplő; a kikommentezett `{:?}` sor visszaállítva fordítási hibát ad. A GCC 13 `lab-gcc` profilban nincs `<print>` és range-formatter. Ellenőrzési feltétel: a formátumsztring konstans, és a `__cpp_lib_format_ranges` definiált.

3. megoldás (vázlat).

```
#include <chrono>
#include <cstdio>
#include <format>
#include <iostream>
#include <print>

int main() {
    constexpr int N = 100000;
    std::FILE* f = std::fopen("/dev/null", "w");
    if (!f) { std::perror("fopen"); return 1; }
    std::setvbuf(f, nullptr, _IOFBF, 1 << 20);

    auto t0 = std::chrono::steady_clock::now();
    for (int i = 0; i < N; ++i) std::fprintf(f, "%d\n", i);
    auto t1 = std::chrono::steady_clock::now();
    for (int i = 0; i < N; ++i) std::cout << i << '\n';
    auto t2 = std::chrono::steady_clock::now();
    for (int i = 0; i < N; ++i) {
        auto s = std::format("{}\n", i);
        std::fwrite(s.data(), 1, s.size(), f);
    }
    auto t3 = std::chrono::steady_clock::now();
    std::fflush(f);
    std::fclose(f);

    auto ms = [](auto a, auto b) {
        return std::chrono::duration_cast<std::chrono::microseconds>(b -
         ↪ a).count();
    };
    std::println("printf={}us cout={}us format={}us",
                 ms(t0, t1), ms(t1, t2), ms(t2, t3));
}
```

Érvényességi feltételek: (a) mindhárom útvonal ugyanazt a bájtfolyamot állítja elő, és ezt ellenőrizni kell; (b) a `cout` és a `printf` ne ossza ugyanazt a `stdout` puffert (itt külön `FILE*`-ra írunk, illetve `sync_with_stdio(false)` kell); (c) a flush a mért szakaszon kívül történjen, vagy minden útvonalra azonos módon; (d) a fordító ne optimalizálja ki a ciklust; (e) a mérés ismételt, és a mediánt nézzük, ne az első futást. A könyv ezt a harness-t nem futtatta, a várt különbség irányát csak a fenti költségtáblázat indokolja, konkrét számot nem állítunk.

41.4 std::stacktrace és futásidejű diagnosztika

A termelési hibajelentés értékét nagyrészt az dönti el, hogy a hiba pillanatában mennyi kontextust sikerült megőrizni. Egy ritkán előforduló, nehezen reprodukálható hiba esetén a legfontosabb kérdés nem az, hogy *mi* történt, hanem hogy *hol*: melyik hívási láncon keresztül jutott a vezérlés a hibás pontra. A C++23 `std::stacktrace` (P0881) ezt a pillanatképet szabványos, hordozható formában teszi elérhetővé. Ez a rész a rögzítés szerződését, a symbolizáció implementációs feltételeit, a hibamodellbe illesztést és a költség oldalát tárgyalja. A release/debug symbol kérdéskörét itt nem tanítjuk újra; a release, debug symbol és reprodukálhatóság rész eredményeire építünk. A hibamodell oldalán a std::expected monadikus hibaláncai jelentik az előzményt, a kimenet oldalán pedig az iostream és pufferelés alapjaira támaszkodunk. A fejezet nem tekinti ellenőrzöttnek a konkrét fordító- és könyvtártámogatást: minden új képességet feature-test makróval és compile probe-bal kell igazolni.

41.4.1 A basic_stacktrace szerződése és a current() szemantikája

A `std::basic_stacktrace<Allocator>` olyan értéktípus, amely `std::stacktrace_entry` elemek sorozatát tárolja. A `std::stacktrace` ennek a `std::allocator<std::stacktrace_entry>`-ral paraméterezett aliasza. A tárolt elemek egy rögzítési esemény pillanatképét írják le: a `current()` hívásakor a könyvtár megpróbálja felderíteni a hívási láncot, és a kereteket `stacktrace_entry` objektumokként eltárolja. A rögzítés után a stacktrace független a további végrehajtástól; a hívási lánc későbbi változása nem hat rá vissza.

A `current()` három túlterheléssel érhető el: allokátorral, `skip` paraméterrel, illetve `skip` és `max_depth` paraméterrel. A `skip` a legfelső keretek kihagyására szolgál, a `max_depth` pedig a rögzítendő keretek felső korlátja. A `skip` pontos jelentése implementációfüggő mértékben eltérhet, ezért a gyakorlatban a `current()` hívókeretének kihagyására használjuk, és a viselkedést a konkrét toolchainen ellenőrizzük. A `current()` nem `noexcept`: a `basic_stacktrace` allokálhat, így allokációs hiba esetén `std::bad_alloc` terjedhet. Ez a tény a későbbi, `noexcept` határokra vonatkozó döntéseknél fontos lesz.

A `stacktrace_entry` értéktípus, másolható és összehasonlítható. A `description()` ember által olvasható szimbólumnevet ad, a `source_file()` és `source_line()` a forráshelyet, a `native_handle()` pedig egy implementációfüggő nyers azonosítót. A `description()`, `source_file()` és `source_line()` egyaránt visszaadhat üres értéket, ha a symbolizációhoz szükséges információ nem elérhető. A `basic_stacktrace` `size()`, `empty()`, `operator[]`, `at()`, `begin()`/`end()` tagfüggvényekkel és `operator<<`-val rendelkezik; a `std::to_string` túlterhelése teljes szöveges formát ad. A szöveges forma pontos alakja implementációfüggő, ezért arra nem szabad gépi parsert építeni.

A következő példa a minimális rögzítést mutatja. A kód nem futtatott, a kimenet formátuma illusztráció.

```
// st_basic.cpp
#include <stacktrace>
#include <iostream>

void fail() {
    const std::stacktrace st = std::stacktrace::current(0, 16);
    std::cerr << "hiba helye:\n" << st << '\n';
}

int main() { fail(); }
```

```
g++ -std=c++23 -g -O1 -fno-omit-frame-pointer st_basic.cpp -lstdc++exp -o
 ↪  st_basic
./st_basic
```

A `-lstdc++exp` itt feltételes: a GCC/libstdc++ egyes kiadásaiban a `<stacktrace>` implementációja a libbacktrace-re épül, és a linkeléshez ez a kísérleti könyvtár (korábbi nevén `-lstdc++_libbacktrace`) szükséges lehet. A pontos könyvtárnév és annak szükségessége implementáció- és verziófüggő; a linker hibaüzenete és a toolchain dokumentációja együtt adja a választ. A libc++ oldalán a C++23 státuszfüggvényt kell ellenőrizni; a támogatás verziófüggő, és ezt a fejezet nem tekinti igazoltnak.

A használat előtt érdemes compile probe-bal ellenőrizni a képességet:

```
#include <version>
#if !defined(__cpp_lib_stacktrace)
#error "a toolchain nem hirdeti a std::stacktrace-t"
#endif
#include <stacktrace>
int main() {
    auto st = std::stacktrace::current(0, 8);
    return st.empty() ? 1 : 0;
}
```

A `__cpp_lib_stacktrace` ismert értéke a szabványos `<version>` fejlécben szerepel; azt nem hardcode-oljuk, hanem a fejlécből olvassuk ki, és a konkrét értéket a toolchain verziójához kötjük.

41.4.2 Symbolizáció: linkelés, debug info és PIE

A rögzítés és a symbolizáció két külön lépés. A `current()` a keretek azonosítóit rögzíti; a `description()`, `source_file()` és `source_line()` hívásakor történhet a szimbolizáció, ami lényegesen drágább lehet, mint a puszta rögzítés. Ezért a hibajelentésben érdemes a stacktrace-t nyers formában eltárolni, és csak a feldolgozás, a megjelenítés vagy a naplózás során szöveggé alakítani.

A symbolizáció feltételei: a megfelelő bináris, a hozzá tartozó debug info és a betöltési környezet ismerete. A `-g` fordítói kapcsoló debug szekciókat generál; a `-fno-omit-frame-pointer` a frame-pointer alapú visszafejtést teszi megbízhatóbbá optimalizált kód mellett is. A modern disztribúciók jellemzően PIE binárist linkelnek, így a címek futásidőben az ASLR miatt változnak. A `description()` a futó folyamatban a betöltött modulokból symbolizál; a `native_handle()` nyers címét `addr2line`-nal csak a betöltési bázis ismeretében lehet feloldani.

A `strip --strip-all` eltávolítja a `.symtab`-ot és a debug szekciókat; a `.dynsym` megmarad, de csak az exportált szimbólumokat tartalmazza. Ekkor a statikus függvények neve és a sorinformáció elveszik, a `description()` üres lehet, a `source_file()` és `source_line()` pedig üresen marad. A `strip --strip-debug` csak a debug szekciókat távolítja el, a szimbólumtáblát megtartja. A release buildben is érdemes lehet a debug infót külön fájlban megőrizni (`objcopy --only-keep-debug`, majd `--add-gnu-debuglink`), és a binárisból eltávolítani; a build-id (`readelf -n`) alapján a debug csomag később is megtalálható.

41.4.3 Stacktrace a hibamodellben: expected és kivétel

A stacktrace-t a hiba keletkezésének pontján kell rögzíteni, nem a hibaút végén. Ha a hibát `std::expected`-tel adjuk tovább, a hibaérték tartalmazhatja a stacktrace-t; a monadikus lánc (`and_then`, `transform`, `or_else`) ezt az értéket továbbviszi, de a rögzítés helye továbbra is a hiba keletkezése marad.

```
// diag_expected.cpp
#include <expected>
#include <stacktrace>
#include <string>
#include <string_view>
#include <charconv>
#include <cstdint>
#include <system_error>
#include <utility>

struct DiagError {
    std::string message;
    std::stacktrace trace;
};

std::expected<std::uint16_t, DiagError> parse_port(std::string_view s) {
    std::uint16_t port{};
    const auto* first = s.data();
    const auto* last = s.data() + s.size();
    auto [ptr, ec] = std::from_chars(first, last, port);
    if (ec != std::errc{} || ptr != last)
        return std::unexpected(DiagError{
            "érvénytelen port: " + std::string(s),
            std::stacktrace::current(0, 32)});
    return port;
}
```

A `skip` értéke itt `0`, hogy a `parse_port` kerete biztosan benne legyen a pillanatképben; a `current()` belső implementációs kereteit a könyvtár eltávolítja. A `max_depth = 32` felső korlátot ad a költségnek.

Kivétel esetén a stacktrace-t a kivétel objektumába építhetjük:

```
#include <stdexcept>
#include <stacktrace>
#include <string>
#include <utility>

class AppError : public std::runtime_error {
public:
    explicit AppError(std::string msg)
        : std::runtime_error(msg), trace_(std::stacktrace::current(0, 32)) {}
    const std::stacktrace& trace() const noexcept { return trace_; }
private:
    std::stacktrace trace_;
};
```

A `current()` allokálhat, ezért a kivétel konstruktorában a `std::bad_alloc` kockázata fennáll; `noexcept` destruktorban vagy kivételkezelő rutinban csak akkor hívjuk, ha ezt a kockázatot kezeljük. A `trace_` tagot érdemes mozgatással átadni, mert a `basic_stacktrace` másolása a teljes kerettömböt másolja.

41.4.4 Fix pufferes formázás spanstreammel

A `std::spanstream` (P0448) a `<spanstream>` fejlécben érhető el; a `std::basic_spanstream<charT, Traits>` egy `std::span<charT>` által adott fix pufferen dolgozik, allokáció nélkül. A `std::spanbuf` a mögöttes puffer, a `span()` tagfüggvény kimeneti módban a ténylegesen kiírt tartományt adja vissza `std::span<char>`-ként. (`view()` tagfüggvény nincs: az a `std::stringstream`-é, és `std::string_view`-t ad; a két API-t nem szabad felcserélni.) A puffer megtelésekor a stream `badbit` vagy `failbit` állapotba kerül; ezt ellenőrizni kell, különben csendben csonkolt üzenetet írunk ki.

```
// diag_line.cpp - C++23: fix pufferes naplósor spanstreammel
#include <array>
#include <cstdio>
#include <span>
#include <spanstream>
#include <string>
#include <string_view>

// Hibapolitika: ha az üzenet nem fér el, a csonkolt sor helyett egy fix,
// rövid jelzés kerül ki; a visszatérési érték jelzi a kiírás sikerét.
bool diag_line(std::string_view what) {
    std::array<char, 64> buf{};
    std::spanstream ss(buf);
    ss << "[diag] " << what << '\n';
    if (!ss) {
        static constexpr std::string_view overflow =
            "[diag] (az uzenet nem fert el a pufferben)\n";
        return std::fwrite(overflow.data(), 1, overflow.size(), stderr)
               == overflow.size();
    }
    const std::span<char> written = ss.span();   // a ténylegesen kiírt rész
    return std::fwrite(written.data(), 1, written.size(), stderr)
           == written.size();
}

int main() {
    const bool a = diag_line("short message");
    const bool b = diag_line(std::string(200, 'x'));   // túl hosszú
    return a && b ? 0 : 1;
}
```

A hibapolitika dokumentált: ha az üzenet nem fér el, a csonkolt sor helyett egy rövid, fix jelzés kerül ki; a függvény visszatérési értéke a `fwrite` sikerét jelzi, így a hívó dönthet a naplózási hiba kezeléséről.

Buildjegyzőkönyv (`lab-gcc` profil, `g++ -std=c++23 -Wall -Wextra -Wpedantic`). A program figyelmeztetés nélkül fordult; a standard hibakimenetre `[diag] short message`, majd a túl

hosszú üzenetnél `[diag] (az uzenet nem fert el a pufferben)` került, a kilépési kód 0. A `ss.view()` hívás `has no member named 'view'` hibával nem fordul.

A `spanstream` előnye a determinisztikus memóriahasználat és az allokáció elkerülése ott, ahol a naplózás szűkös környezetben történik. Hátránya, hogy a puffer méretét előre ismerni kell, és a formázás nem `std::format`-alapú; a `std::format` és a `std::print` allokálhat, ezért nem helyettesíti a fix pufferes utat. A `spanstream` nem async-signal-safe, signal handlerben nem használható. A feature-test `__cpp_lib_spanstream`; a támogatást itt is compile probe-bal ellenőrizzük.

41.4.5 Költség és a hot path védelme

A rögzítés költsége a keretek számával és a visszafejtési módszerrel nő; a symbolizáció külön, jellemzően nagyobb költség. A `current()` allokálhat, ezért a hot pathban különösen kerülendő. A védelem eszközei: csak hibaágon rögzíteni; a `max_depth` szigorú korlátozása; a `skip` beállítása; a symbolizáció késleltetése; rate limit vagy mintavétel ott, ahol a hiba nagy gyakoriságú. A mérést nem helyettesíti a becslés: a költséget a saját toolchainen, a saját optimalizálási profilon kell megmérni.

Egy mérési terv: rögzítsük a `max_depth` 8, 32 és 128 értékeit; hasonlítsuk össze a `-O0 -g`, `-O2 -g` és `-O2` (stripped) buildet; a `-fno-omit-frame-pointer` legyen külön változó; a mérést warmup előzze meg, és a mediánt és a p99-et jegyezzük fel. Az elfogadási feltétel a szolgáltatás SLO-jából származik, nem univerzális szám.

```
#include <stacktrace>
#include <chrono>
#include <cstdio>
#include <cstddef>

int main() {
    constexpr int N = 10000;
    std::size_t total = 0;
    const auto t0 = std::chrono::steady_clock::now();
    for (int i = 0; i < N; ++i) {
        const auto st = std::stacktrace::current(0, 32);
        total += st.size();
    }
    const auto t1 = std::chrono::steady_clock::now();
    const auto ns = std::chrono::duration_cast<std::chrono::nanoseconds>(t1 -
     ↪  t0).count();
    std::printf("N=%d, keretek=%zu, atlag=%.1f ns/hivas\n",
                N, total, static_cast<double>(ns) / N);
}
```

A kód nem futtatott; a mérés eredménye a konkrét gépen, fordítón és könyvtáron múlik.

41.4.6 Gyakori hibák és javításuk

A leggyakoribb hiba a `current()` hívása minden kérés kiszolgálásakor, a hot pathban; javítás: csak a hibaágon rögzíteni. A második, hogy a stacktrace-t a legfelső hibakezelőben rögzítik, ahol már elveszett a hiba helye; javítás: a hiba keletkezésénél rögzíteni, és az értéket továbbadni. A harmadik a `-lstdc++exp` hiánya vagy a feature-test elmaradása; javítás: compile probe és a linker

beállítása. A negyedik a stripped release bináris, amelyből üres `description()` származik; javítás: debug info külön fájlban, build-id alapján. Az ötödik a `basic_stacktrace` felesleges másolása; javítás: mozgatás. A hatodik a `spanstream` túlcsordulásának figyelmen kívül hagyása; javítás: a stream állapotának ellenőrzése. A hetedik a `current()` hívása `noexcept` destruktorban; javítás: a kivétel kockázatának kezelése vagy elkerülése.

41.4.7 Feladatok

1. **Magyarázó feladat.** Készíts döntési táblázatot arról, hogy egy szolgáltatás mely pontjain rögzítenél stacktrace-t (váratlan hiba, validációs hiba, hot path, indulás, leállás, aszinkron callback), és minden sorhoz indokold a döntést a költség és a diagnosztikai érték szempontjából.

2. **Kódolási feladat.** Egészítsd ki a `parse_port` példát egy `with_report` függvénnyel, amely `or_else`-sel kiegészíti a hibaüzenetet a stacktrace szöveges formájával, majd írd ki az eredményt. Ügyelj arra, hogy a trace a hiba keletkezésének helyén rögzüljön.

3. **Diagnosztikai feladat.** Egy stripped release binárisban a `description()` üres, a `source_file()` és `source_line()` szintén üres. Sorolj fel legalább három lehetséges okot, és mindegyikhez adj egy ellenőrzési lépést.

4. **Mérési feladat.** Tervezz mérést a `current()` költségére: add meg a független változókat, a mintaszámot, a warmupot, a mért statisztikákat és az elfogadási feltételt.

41.4.8 Megoldások

1. megoldás. A döntési táblázat lehetséges tartalma:

Kontextus	Rögzítés	Indok
Váratlan hiba	igen	a hibaút ritka, a diagnosztikai érték nagy
Validációs hiba	nem	nagy gyakoriságú, a hiba helye a hibaüzenetből ismert
Hot path	nem	a költség és az allokáció nem fér bele a budgetbe
Indulás	opcionális	a kontextus ismert, de a konfigurációs hiba diagnosztizálható
Leállás	igen	a leállítás ritka, a hívási lánc fontos
Aszinkron callback	igen, rate limittel	a hiba forrása nehezen követhető, de a gyakoriság korlátozandó

Az indoklás minden sorban a költség és a diagnosztikai érték arányán alapul; az ellenőrzési feltétel az, hogy a hot pathra eső rögzítések száma nulla legyen, és a hibaágon a trace ne legyen üres.

2. megoldás.

```
std::expected<std::uint16_t, DiagError> with_report(std::string_view s) {
    return parse_port(s).or_else(
        [](DiagError e) -> std::expected<std::uint16_t, DiagError> {
            e.message += "\n--- stacktrace ---\n";
            e.message += std::to_string(e.trace);
            return std::unexpected(std::move(e));
        });
}
```

Az `or_else` csak a hibaágon hívódik, így a sikeres útvonalon nincs többletköltség. A trace a `parse_port`-ban, a hiba keletkezésénél rögzült; a `with_report` csak a megjelenítést egészíti ki. Ellenőrzési feltétel: érvénytelen bemenetre a `message` tartalmazza a „stacktrace” jelölőt, és a `trace.size()` nullánál nagyobb, feltéve, hogy a toolchain támogatja a rögzítést.

3. megoldás. Lehetséges okok és ellenőrzések: (a) a binárist `strip --strip-all`-lal csökkentették, így a `.symtab` és a debug szekciók eltűntek — ellenőrzés: `file`, `readelf -S`; (b) a build nem kapott `-g` kapcsolót — ellenőrzés: a buildparancs és a `readelf -S` debug szekcióinak hiánya; (c) a debug info külön fájlban van, de a build-id vagy a debuglink nem található — ellenőrzés: `readelf -n`, `readelf --string-dump=.gnu_debuglink`; (d) a szimbólumok nem exportáltak a dinamikus táblába — ellenőrzés: `nm -C`, `nm -D`. A `native_handle()` nyers címét `addr2line -e` és `gdb` segítségével lehet feloldani, a PIE betöltési bázisát figyelembe véve.

4. megoldás. Független változók: `max_depth` (8, 32, 128), optimalizálási szint (`-O0`, `-O2`), debug info (`-g` be/ki), frame pointer (`-fno-omit-frame-pointer` be/ki). Mintaszám: legalább 10 000 hívás, 1 000 warmup hívással. Mért statisztikák: medián, p99, szórás. Elfogadási feltétel: a mérés variációs koefficiense 5% alatt legyen, és a hibaágon mért rögzítési költség a szolgáltatás SLO-jából levezetett budgeten belül maradjon. A mérési kódot úgy kell kialakítani, hogy a fordító ne ejtse ki a hívást (a keretszám összegzését ki kell írni).

41.4.9 Összefoglaló

A `std::stacktrace` a hiba pillanatának hívási láncát rögzíti; a `current()` nem `noexcept`, mert allokálhat, és a `skip`/`max_depth` paraméterekkel a költség korlátozható. A rögzítés és a szimbolizáció külön lépés: a `description()`, `source_file()` és `source_line()` implementációfüggően üres lehet, ha a debug info nem elérhető, a bináris stripped, vagy a PIE betöltési bázisa nem ismert. A linkelés feltétele implementációfüggő (GCC/libstdc++ esetén például `-lstdc++exp`), ezért feature-test és compile probe szükséges. A stacktrace-t a hiba keletkezésénél kell rögzíteni, `std::expected` hibaértékébe vagy kivétel objektumába építve. A fix pufferes megjelenítésre a `std::spanstream` allokációmentes alternatívát ad, de a túlcsordulást ellenőrizni kell. A hot path védelme a költségmérésen és a hibaágon történő rögzítésen alapul.

42 Ranges C++23-ban: konverzió, új view-k és algoritmusok

42.1 ranges::to és tárolókonstrukció range-ből

A lazy range-pipeline egy ponton túllép a nézeteken: a feldolgozás eredményét tárolóba kell tenni, hogy tulajdonolt, továbbadható, a forrástól független adatunk legyen. Ezt a műveletet nevezzük **materializálásnak**. C++20-ban erre kézzel kellett `std::vector`-t és `push_back`-et vagy `std::ranges::copy`-t írni; C++23-ban a szabvány két egymást kiegészítő eszközt ad: a `std::ranges::to` konverziós függvényt és a konténerek range-alapú tagfüggvényeit, illetve `from_range_t` konstruktorait. Ez a rész ezeket a szerződéseket tárgyalja, különös tekintettel az allokációs költségre és arra, hogy a materializálás mikor véd a dangling ellen — és mikor nem. Az alapokat, a lazy pipeline viselkedését és a borrowed range fogalmát a Lazy view pipeline és mellékhatások, illetve a Borrowed range, dangling és iterator lifetime részben rögzítettük; itt ezekre építünk, nem ismételjük meg őket.

A fejezet nem végzett forrásellenőrzést. A C++23 státuszt, a pontos feature-test értékeket és az implementációs lefedettséget a megadott kiindulópontokon (például az N4950 munkatervezeten és a cppreference C++23 oldalán) kell ellenőrizni; ezek a linkek kiindulópontok, nem bizonyítékok.

42.1.1 A `ranges::to` szerződése

A `std::ranges::to` a `<ranges>` fejlécben található, a `std::ranges` névtérben. Nem egyetlen függvény, hanem túlterhelés-halmaz, amelynek két használati alakja van: közvetlen hívás és pipe-ba illeszthető closure. A fő alakok:

- `std::ranges::to<C>(r, args...)` — az `r` range-ből `C` típusú konténert készít, a `C` konstruktorának további `args` argumentumaival (jellemzően allocator).
- `std::ranges::to<C<>>(r, args...)` — a konténer típusát a range value_type-jából vezeti le.
- `r | std::ranges::to<C>(args...)` — a pipe alak, ahol a `to<C>(args...)` egy range adaptor closure.

A visszatérési típus mindig `C`, érték szerint. A bemenet egy `input_range`, amelynek value_type-ja konvertálható a `C` value_type-jává (a szabvány `container-compatible-range` fogalma); ez a fordítási feltétel, nem futásidejű ellenőrzés. Ha a konverzió nem lehetséges, a program nem fordul — ez a kívánt viselkedés, mert a hibát a típusrendszer fogja meg.

A `to<C<>>` levezetés lényege: a konténer típusa `C<range_value_t<R>>` lesz, a többi template paraméter a default értékét kapja. Ezért működik a `to<std::vector>()` (`vector<int>`, `vector<std::string>` stb.) és a `to<std::string>()`, de a `to<std::map>()` nem, mert a `map` első paramétere a kulcs típusa, amely nem vezethető le a value_type-ból; ott `to<std::map<K, V>>()` kell. A `C` tehát template-név, nem típus: a `to<std::vector>` (típus nélkül) szintaktikailag hibás.

42.1.2 Konténer oldali range-konstrukciók

A C++23 a konténereket is ellátta range-fogadó felülettel. A `std::from_range_t` tag és a `std::from_range` inline constexpr érték a `<ranges>` fejlécben él; a konstruktorok és tagfüggvények a konténer saját fejlécében (`<vector>`, `<list>`, `<string>` stb.) vannak.

```
std::vector<int> v(std::from_range, r);          // konstruktor range-ből
v.assign_range(r2);                              // teljes tartalom cseréje
v.insert_range(v.begin() + 2, r3);               // beszúrás pozíció elé
v.append_range(r4);                              // hozzáfűzés (sequence
 ↪  containereknél)
```

A `from_range` tag kötelező: a `std::vector<int> v(r)` alak nem a range-konstruktort hívja, hanem más, esetleg nem is létező túlterhelést — ez gyakori félreértés. Az `insert_range` és az `assign_range` a legtöbb konténeren elérhető, az `append_range` csak azokon, amelyeknek van `push_back`-je (vector, deque, list, forward_list, string); a `std::array` fix mérete miatt eleve kimarad. Az `insert_range` előfeltétele, hogy a forrás range ne hivatkozzon a célkonténer saját elemeire; az ilyen önbeszúrás (`v.insert_range(v.end(), v)`) nem megengedett, és a konténer invalidációja miatt nem definiált viselkedéshez vezet. A pontos előfeltételt mindig az adott tagfüggvény leírásában ellenőrizzük.

A `ranges::to` és a `from_range` viszonya: a szabvány a `to` megfigyelhető eredményét írja elő, nem az implementációját. A könyvtárak jellemzően a `from_range` konstruktort vagy `emplace`/`insert` műveleteket használnak; számunkra a létrejövő konténer tartalma és a value_type konvertálhatósága a szerződés.

42.1.3 Sized range és előzetes foglalás

A materializálás költsége elsősorban attól függ, hogy a forrás **sized range**-e, azaz a `std::ranges::sized_range` koncepció teljesül-e, és a `std::ranges::size(r)` $O(1)$-e. Ha igen, a konténer előre foglalhat, és a növekedés újraallokációi elmaradnak. Ha nem (tipikus `input_range`, szűrő pipeline), a konténer növekedni fog.

A költséget számszerűsíthetjük. Legyen a végső elemszám n, a kezdeti kapacitás c_0, és tegyük fel a szokásos duplázó növekedést. Az újraallokációk száma legfeljebb

$$\left\lceil \log_2 \frac{n}{c_0} \right\rceil + 1,$$

az összes elemáthelyezés pedig legfeljebb $2n$ (a mértani sor miatt). Előzetes `reserve(n)` esetén egy allokáció történik, és a range elemei egyszer másolódnak vagy mozgatódnak a végleges bufferbe. A csúcs-memóriahasználat szempontjából is jelentős a különbség: duplázásnál a régi és az új buffer egyszerre él, ami átmenetileg mintegy $3n$ elemnyi tárhelyet igényelhet.

A `ranges::to` sized range esetén a gyakorlatban előre foglal, ha a célkonténernek van `reserve` tagfüggvénye; ezt azonban a szabványos szöveg pontos megfogalmazása szerint kell ellenőrizni, és nem szabad rá építeni minden implementációnál. Ha biztos kontrollt akarunk, a `to` előtt magunk is hívhatunk `reserve`-ot egy köztes konténeren, vagy saját materializáló függvényt írhatunk (lásd a C++20 fallbacket). A `std::list`-nek nincs `reserve`-ja, tehát a `to<std::list<T>>` esetén az előfoglalás lehetősége eleve elvész; itt a csomóponti allokáció dominál, ami más költségprofil.

42.1.4 Beágyazott konverzió

A `to` nem rekurzív: a range elemeit alakítja a konténer value_type-jává, de a beágyazott range-eket nem materializálja automatikusan. Ez a `vector<vector<T>>` esetén azonnal látszik.

```
namespace v = std::views;
auto flat = v::iota(0, 12) | std::ranges::to<std::vector<int>>();

// Ez nem fordul: a chunk elemei subrange-ok, nem vector<int>-ek.
// auto bad = flat | v::chunk(3) |
 ↪  std::ranges::to<std::vector<std::vector<int>>>();

auto nested = flat
    | v::chunk(3)
    | v::transform(std::ranges::to<std::vector<int>>())   // belső szint
    | std::ranges::to<std::vector<std::vector<int>>>();   // külső szint
```

A kommentelt sor fordítási hibát ad, mert a `chunk` elemeinek típusa `subrange`, amely nem konvertálható `vector<int>`-té. A helyes megoldás a belső szinten egy `transform`-mal elvégzett materializálás; ezt követően a külső `to` már `vector<int>` elemeket másol. A `chunk` részleteit a Szeletelő és átalakító view-k tárgyalja; itt csak a beágyazott konverzió eszközeként jelenik meg. A tanulság általános: minden olyan szinten, ahol a value_type nem konvertálható közvetlenül, külön materializálás kell.

42.1.5 Materializálás mint dangling-elleni eszköz

Hogy egy view tulajdonolja-e a forrását, az a konkrét alap-view típusán múlik (C++20 [range.all], a C++20-ra visszamenőleg alkalmazott P2415 szerint). A `views::all` – és így minden pipeline első lépése – a forrás értékkategóriája szerint választ:

- **lvalue** tartományból (`v | views::filter(...)`) `ref_view` lesz: a view csak hivatkozik a `v`-re, és addig érvényes, amíg a `v` él;
- **rvalue**, nem view tartományból (`make() | views::filter(...)`) `owning_view` lesz: a view **mozgatással átveszi** az ideiglenes vektort, és birtokolja, így a teljes kifejezés után is biztonságosan bejárható;
- egy már meglévő view-t a `views::all` másolja.

A valódi dangling helyzet ezért nem a `make()` eredményén épített view, hanem a függvényből visszaadott, **lokális lvalue** forrásra épülő view: a `ref_view` a megszűnt lokális vektorra mutat.

```
// owning_filter.cpp - C++20: rvalue forrás -> owning_view, lvalue -> ref_view
#include <cstdio>
#include <ranges>
#include <span>
#include <type_traits>
#include <vector>

std::vector<int> make() { return {-2, 1, -3, 4, 5}; }

auto positive = [](int x) { return x > 0; };

// NEGATÍV: lokális lvalue vektor -> ref_view; a visszaadott view dangling.
auto dangling_filter() {
```

```
    std::vector<int> local = make();
    return local | std::views::filter(positive);
}

// NEGATÍV: a span borrowed_range, de a lokális háttértárolót nem tartja
 ↪  életben.
auto dangling_span() {
    std::vector<int> local = make();
    return std::span<const int>(local) | std::views::take(2);
}

int main(int argc, char**) {
    // A make() eredménye rvalue: a views::all owning_view-ba mozgatja,
    // a view a saját vektorát birtokolja.
    auto owned = make() | std::views::filter(positive);
    static_assert(std::is_same_v<decltype(owned),
        std::ranges::filter_view<std::ranges::owning_view<std::vector<int>>,
                                 decltype(positive)>>);

    std::vector<int> v = make();
    auto borrowed = v | std::views::filter(positive);   // ref_view: v-re
     ↪  hivatkozik
    static_assert(std::is_same_v<decltype(borrowed),
        std::ranges::filter_view<std::ranges::ref_view<std::vector<int>>,
                                 decltype(positive)>>);

    int sum = 0;
    for (int x : owned) sum += x;                       // későbbi utasításban
     ↪  is OK
    std::printf("owned sum=%d\n", sum);

    if (argc > 1) {                                     // negatív próbák
        for (int x : dangling_filter()) std::printf("%d\n", x);
        for (int x : dangling_span()) std::printf("%d\n", x);
    }
}
```

Buildjegyzőkönyv (`lab-gcc` profil, `g++ -std=c++20 -Wall -Wextra -Wpedantic -fsanitize=address,undefined`). A program figyelmeztetés nélkül fordult; a két `static_assert` igazolja, hogy a `make()`-ből `owning_view`, a v-ből `ref_view` lesz. Normál futásban `owned sum=10`. Negatív módban (`./owning_filter neg`) a `dangling_filter()` bejárása ASan alatt `stack-use-after-return`, a csak `dangling_span()`-t futtató változat `heap-use-after-free` hibát ad.

A `to` ezzel szemben mindig új tárolót hoz létre. Ez a **konténer** tulajdonát rendezi, az **elemekét** nem: ha az elemtípus maga is nézet (`std::string_view`, `std::span`, pointer), a materializált vektor ugyanúgy kölcsönzött adatra mutat, mint a view. Önálló szöveghez az elemeket is tulajdonló típusra kell konvertálni (`std::string`). A konténer tehát ott hasznos, ahol a view-t a forrás élettartamán túl kellene megőrizni, vagy ahol az elemeket egyszer, rögzítve akarjuk látni.

```
std::vector<int> make();

auto owned = make() | std::views::filter([](int x) { return x > 0; });
```

```
// Biztonságos: az owning_view birtokolja a make() eredményét.

auto snapshot = make()
              | std::views::filter([](int x) { return x > 0; })
              | std::ranges::to<std::vector<int>>();
// Önálló vector<int>: az int elemek értékek, a másolat valóban független.
```

Nézet-elemtípusnál a különbség közvetlenül látszik:

```
// materialize_views.cpp - C++23: a konténer saját, az elemek nem feltétlenül
#include <cassert>
#include <ranges>
#include <string>
#include <string_view>
#include <vector>

int main() {
    std::string text = "alpha beta gamma";
    auto words = text | std::views::split(' ')
                      | std::views::transform([](auto&& w) { return
                       ↪  std::string_view(w); });

    // A vector saját, de az elemei string_view-k: a text pufferére mutatnak.
    auto borrowed = words | std::ranges::to<std::vector<std::string_view>>();
    // Önálló adat: minden elem saját std::string.
    auto owned = words | std::views::transform([](std::string_view w) { return
     ↪  std::string(w); })
                       | std::ranges::to<std::vector<std::string>>();

    text[0] = 'A';                                  // a forrás módosul
    assert(borrowed[0] == "Alpha");                 // a nézet a változást
     ↪  látja
    assert(owned[0] == "alpha");                    // a másolat független
}
```

Buildjegyzőkönyv. A GCC 13.3 libstdc++-ában a `ranges::to` még nincs (`'to' is not a member of 'std::ranges'`); a Clang 22 / libc++ `-std=c++23 -Wall -Wextra -Wpedantic` alatt figyelmeztetés nélkül fordította, és mindkét `assert` teljesült: a forrás módosítása után a `string_view`-s vektor `"Alpha"`-t, a `string`-es `"alpha"`-t látott.

A két változat élettartamában nem különbözik, a költségében igen: az `owned` minden bejáráskor újra szűr, a `snapshot` egyszer. A materializálás akkor véd a danglingtől, ha a view egy olyan lvalue forrásra épül, amely hamarabb megszűnik, mint ahogy az eredményre szükség van: ilyenkor a `| to<...>()`-t a forrás élettartamán belül kell meghívni.

Egy másik gyakori hiba: a materializált konténerre később view-t nyitunk, majd a konténert módosítjuk (például `push_back`-kel). A `vector` újraallokációja érvényteleníti a view iteratorait — erről a std::array, std::vector és folytonos tárolás invalidációs szabályai szólnak.

42.1.6 C++20 fallback saját `to`-függvénnyel

Ha a laborprofil C++20, a `ranges::to` nem elérhető. A materializálás magja saját függvénnyel is megvalósítható, sized range esetén előfoglalással:

```cpp
#include <ranges>
#include <utility>

template <class C, std::ranges::input_range R>
C to_container(R&& r) {
    C c;
    if constexpr (std::ranges::sized_range<R> &&
                  requires(C& c, std::ranges::range_size_t<R> n) { c.reserve(n);
                   ↪ }) {
 ↪  c.reserve(static_cast<std::ranges::range_size_t<R>>(std::ranges::size(r)));
    }
    for (auto&& e : r) {
        c.push_back(std::forward<decltype(e)>(e));   // sequence containerekhez
    }
    return c;
}
```

Ez a változat nem azonos a szabványos `to` teljes szerződésével: nincs benne a `to<C<>>` levezetés, a `from_range` konstruktor preferálása és a beágyazott eset kezelése. A materializálás és az előfoglalás lényegét viszont megmutatja, és C++20-on is használható. Asszociatív konténerhez `insert`-et kell használni `push_back` helyett — ezt a feladat megoldása mutatja.

42.1.7 Gyakori hibák és javításuk

- **Hiányzó fejléc vagy névtér.** A `ranges::to` a `<ranges>`-ben és a `std::ranges` névtérben van, nem a `<algorithm>`-ban és nem `std::to` néven.
- **Template-név típus helyett.** A `to<std::vector>` hibás; helyesen `to<std::vector<int>>()` vagy `to<std::vector>()`.
- **Levezethetetlen konténer.** A `to<std::map>()` nem fordul; a kulcs típusát explicit meg kell adni.
- **Rekurzivitás feltételezése.** A `to<vector<vector<T>>>` nem materializálja a belső range-eket; a belső szinten külön `transform(to<vector<T>>())` kell.
- **`from_range` tag elhagyása.** A `vector<int> v(r)` nem a range-konstruktort hívja; a `std::from_range` tag kötelező.
- **Önbeszúrás.** Az `insert_range` forrása nem hivatkozhat a célkonténer saját elemeire.
- **Kétszeri bejárás.** `input_range` esetén a range csak egyszer járható be; a `to` ezt betartja, de a körülötte lévő pipeline-nak is tekintettel kell lennie erre.

42.1.8 Feladatok

1. **Alap.** Írj függvényt, amely `std::vector<int>`-ben adja vissza az $1..n$ egészek négyzeteit `views::iota`, `views::transform` és `ranges::to` használatával. A visszatérési típust `static_assert`-tel ellenőrizd. Készítsd el ugyanezt a C++20-as fallbackkel is.
2. **Közepes.** Adott egy `std::vector<std::string>` naplósorokkal. Szűrd ki az `"ERROR"` előtagú sorokat, materializálj `std::vector<std::string>`-be, majd csoportosítsd kettesével, és az eredményt materializáld `std::vector<std::vector<std::string>>`-be. Magyarázd meg, miért nem működik a közvetlen `to<std::vector<std::vector<std::string>>>`.
3. **Haladó.** Egészítsd ki a `to_container` fallbacket úgy, hogy sized range esetén előfoglaljon, és asszociatív konténerhez is működjön. Használj feature-test makrót annak eldöntésére,

hogy a szabványos `ranges::to` elérhető-e, és indokold az allokációs különbséget egy sized és egy szűrt (nem sized) range esetén.

42.1.9 Megoldások

1. megoldás. A lényeg a típus és a két változat összevetése.

```
#include <ranges>
#include <vector>
#include <type_traits>

std::vector<int> squares(int n) {
    namespace v = std::views;
    auto r = v::iota(1, n + 1) | v::transform([](int x) { return x * x; });
    auto out = std::ranges::to<std::vector<int>>(r);
    static_assert(std::is_same_v<decltype(out), std::vector<int>>);
    return out;
}
```

A C++20-as változatban a `to_container<std::vector<int>>(r)` hívás helyettesíti a `to`-t. *Ellenőrzés:* a `static_assert` forduljon le, és a `v::iota(1, 6)` bemenetre a várt eredmény $1, 4, 9, 16, 25$ (nem mért, hanem a szemantikából következő érték).

2. megoldás. A belső szintet `transform`-mal kell materializálni.

```
#include <ranges>
#include <string>
#include <vector>
#include <algorithm>

std::vector<std::vector<std::string>> groups(const std::vector<std::string>&
 ↪  lines) {
    namespace v = std::views;
    return lines
        | v::filter([](const std::string& s) { return s.rfind("ERROR", 0) == 0;
         ↪  })
        | std::ranges::to<std::vector<std::string>>()
        | v::chunk(2)
        | v::transform(std::ranges::to<std::vector<std::string>>())
        | std::ranges::to<std::vector<std::vector<std::string>>>();
}
```

A közvetlen `to<std::vector<std::vector<std::string>>>` azért nem működik, mert a `chunk` elemei `subrange`-ok, amelyek nem konvertálhatók `std::vector<std::string>`-té. *Ellenőrzés:* páratlan elemszámnál az utolsó csoport egyelemű; a függvény a bemenetet nem módosítja.

3. megoldás. A fallback általánosítása és a feature-test.

```
#include <ranges>
#include <version>

#if defined(__cpp_lib_ranges_to_container) && __cpp_lib_ranges_to_container >=
 ↪  202202L
#  define HAVE_RANGES_TO 1
```

```
#else
#  define HAVE_RANGES_TO 0
#endif

template <class C, std::ranges::input_range R>
C to_container(R&& r) {
    C c;
    if constexpr (std::ranges::sized_range<R> &&
                  requires(C& c, std::ranges::range_size_t<R> n) { c.reserve(n);
                   ↪ }) {

 ↪  c.reserve(static_cast<std::ranges::range_size_t<R>>(std::ranges::size(r)));
    }
    for (auto&& e : r) {
        if constexpr (requires { c.push_back(std::forward<decltype(e)>(e)); }) {
            c.push_back(std::forward<decltype(e)>(e));
        } else {
            c.insert(c.end(), std::forward<decltype(e)>(e));
        }
    }
    return c;
}
```

Indoklás: sized range esetén a `reserve` egy allokációra csökkenti a növekedést, és elkerüli a legfeljebb $2n$ elemáthelyezést; szűrt pipeline esetén a méret nem ismert, ezért a növekedés amortizáltan $O(n)$ marad, de az újraallokációk és a csúcs-memória nem küszöbölhetők ki. *Ellenőrzés:* a kód forduljon sized (`vector`) és nem sized (szűrt) bemenetre is; a `HAVE_RANGES_TO` értékét a fordító `<version>` fejlécéből olvassuk, és ha 0, a fallbacket használjuk. A feature-test konkrét számértéke implementáció- és verziófüggő, ezért a saját fejlécben ellenőrizendő.

42.1.10 Összefoglaló

A `std::ranges::to` a C++23 szabványos materializáló eszköze: a lazy pipeline eredményét tulajdonolt konténerbe másolja, ezzel a forrás megszűnése után is érvényes adatot adva. A `to<C>` explicit típust vár, a `to<C<>>` a value_type-ból vezeti le a konténer típusát; utóbbi nem működik olyan sablonoknál, ahol további típusparaméter (például kulcs) nem dedukálható. A konténerek `from_range_t` konstruktorai és `insert_range`/`append_range`/`assign_range` tagfüggvényei ugyanezt a range-fogadó szerződést valósítják meg közvetlenül. A költség sized range esetén előfoglalással egy allokációra és lineáris másolásra csökkenthető; nem sized range esetén az amortizált növekedés marad. A `to` nem rekurzív: beágyazott konténerhez szintenkénti materializálás kell. A materializálás a teljes kifejezésen belül véd a dangling ellen, de a view külön változóba mentése továbbra is hibás. C++20-on a bemutatott `to_container` fallback adja a magot, a teljes szabványos szerződés nélkül.

42.2 Kombináló view-k: zip, adjacent, enumerate, cartesian_product

A naplóelemző pipeline eddig egyetlen forráson dolgozott: szűrtünk, transzformáltunk, majd anyaggá alakítottuk az eredményt. A valós diagnosztikában viszont ritkán van csak egy sorozatunk: egyszerre kell egy időbélyeg-listát és egy késleltetés-listát összepárosítani, egy sorozat

szomszédos elemeit összevetni, a rekordokhoz sorszámot rendelni, vagy éppen minden rekordot minden rekorddal összemérni. A C++23 kombináló view-jai (`views::zip`, `views::zip_transform`, `views::adjacent`, `views::enumerate`, `views::cartesian_product`) pontosan ezt teszik lehetővé allokáció nélkül. A nehézség azonban nem az algoritmusban van, hanem abban, hogy **mi a dereferálás típusa**, és hogy **meddig él az, amire a nézet mutat**. Ez a rész ezt a két kérdést járja körül; a C++20-as ranges-alapokat (fél-nyílt tartomány, sentinel, projection, lazy pipeline) ismertnek tekinti, és csak a kombinálás új szerződéseit bontja ki.

Az élettartam-kérdéshez felidézzük a korábbi borrowolt range-szabályt: egy `R` range akkor *borrowed range*, ha `R` lvalue-referencia típusú, vagy `enable_borrowed_range<remove_cvref_t<R>>` igaz. Ennek jelentősége itt konkrét: a kombinált view majdnem soha nem borrowed, ezért ideiglenesből előállított kombinált nézetből kimentett iterátor tipikusan dangling. A tuple-protokollt (miért működik a strukturált kötés, mit ad a `std::get`) a tuple-részből vesszük adottnak, az anyaggá alakítást pedig a `ranges::to`-ről szóló részre hagyjuk.

42.2.1 A kombináló view-k közös szerződése: mit ad a dereferálás?

Pontos definícióval kezdünk. Legyenek $R_1, \dots, R_n$ range-ek. A `views::zip` egy olyan `zip_view`-ot ad, amelyre

$$\text{reference} = \texttt{tuple<range_reference_t<}R_1\texttt{>}, \dots, \texttt{range_reference_t<}R_n\texttt{>} >,$$

$$\text{value_type} = \texttt{tuple<range_value_t<}R_1\texttt{>}, \dots, \texttt{range_value_t<}R_n\texttt{>} > .$$

Feltétel: minden R_i teljesítse az `input_range` követelményt. A bejárás a **legrövidebb** forrás végénél megáll; a hossz nem összeg, hanem minimum. A tulajdonságok külön-külön (C++23 [range.zip.view]):

- **kategória:** a `zip_view` a források leggyengébb kategóriáját kapja; `random_access_range`, ha minden forrás az — ehhez a méret nem kell;
- **sized:** akkor `sized_range`, ha minden forrás az;
- **common:** három eset bármelyikében: (1) egyetlen forrás van, és az common; (2) nem minden forrás bidirectional, és minden forrás common; (3) minden forrás random access *és* sized.

Források	common	sized	kategória
`vector` + `vector`	igen (3. eset)	igen	random access
`list` + `list`	**nem** (mind bidirectional, nem RA)	igen	bidirectional
`forward_list` + `forward_list`	igen (2. eset)	nem	forward
RA + sized, nem common forrás (kétszer)	igen (3. eset)	igen	random access
`iota(0)` + `iota(0)` (végtelen)	nem	nem	random access

```
// zip_concepts.cpp - C++23: a zip_view common/sized/random_access feltételei
#include <cstddef>
#include <forward_list>
```

```cpp
#include <list>
#include <ranges>
#include <vector>

// Véletlen elérésű, sized, de NEM common forrás: sized sentinel saját típussal.
struct end_at {
    const int* p;
    friend bool operator==(const int* i, end_at e) { return i == e.p; }
    friend std::ptrdiff_t operator-(end_at e, const int* i) { return e.p - i; }
    friend std::ptrdiff_t operator-(const int* i, end_at e) { return i - e.p; }
};
using RaSized = std::ranges::subrange<const int*, end_at>;
static_assert(std::ranges::random_access_range<RaSized> &&
 ↪  std::ranges::sized_range<RaSized>
              && !std::ranges::common_range<RaSized>);

template <class... R> using Z = std::ranges::zip_view<std::views::all_t<R&>...>;
using V = std::vector<int>; using L = std::list<int>; using F =
 ↪  std::forward_list<int>;

// vector/vector: random access + sized -> common
static_assert(std::ranges::common_range<Z<V, V>> &&
 ↪  std::ranges::random_access_range<Z<V, V>>);
// list/list: mind bidirectional -> NEM common (csak a 3. ág adhatná, az RA-t
 ↪  kér)
static_assert(!std::ranges::common_range<Z<L, L>> &&
 ↪  std::ranges::bidirectional_range<Z<L, L>>
              && std::ranges::sized_range<Z<L, L>>);
// forward_list/forward_list: nem bidirectional és minden forrás common ->
 ↪  common
static_assert(std::ranges::common_range<Z<F, F>> &&
 ↪  !std::ranges::sized_range<Z<F, F>>);
// RA + sized, nem common források: a 3. ág szerint common
static_assert(std::ranges::common_range<std::ranges::zip_view<RaSized,
 ↪  RaSized>>);
// random_access: elég, ha minden forrás az - sized nélkül is
using Inf = decltype(std::views::iota(0));                  // RA, nem sized
static_assert(std::ranges::random_access_range<std::ranges::zip_view<Inf, Inf>>
              && !std::ranges::sized_range<std::ranges::zip_view<Inf, Inf>>);
int main() {}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++23 -Wall -Wextra -Wpedantic zip_concepts.cpp` figyelmeztetés nélkül fordult (Clang 22 / libc++ alatt is), tehát a táblázat minden sora `static_assert`-tel ellenőrzött.

A lényegi megállapítás: a dereferálás eredménye nem `tuple&`, hanem egy **prvalue tuple, amely referencia-tagokat tartalmaz**. Ez az úgynevezett proxy referencia. Ha a forrás `int&`-et ad, a `zip` dereferálása `std::tuple<int&, double&>`, és ezen a tuple-ön keresztül írva az eredeti elemek módosulnak, mert a tuple nem az értékeket, hanem a hivatkozásokat tárolja.

Nézet	dereferálás típusa	value_type	Írható a forráson?
zip(a,b)	tuple<ra, rb>	tuple<va, vb>	igen, ha ra/rb valódi referencia
zip_transform(f,a,b)	invoke_result_t<F&, ra, rb>	annak remove_cvref-je	csak ha f referenciát ad
adjacent<2>(r)	tuple<r\&, r\&>	tuple<v, v>	igen
enumerate(r)	tuple<diff_t, r\&>	tuple<diff_t, v>	a második tagon igen
cartesian_product(a,b)	tuple<ra, rb>	tuple<va, vb>	igen

Két következményt érdemes azonnal rögzíteni. Egyrészt `auto& x = *it;` nem fordul le, mert a `*it` prvalue; `auto&&` vagy érték szerinti kötés kell. Másrészt ha a forrás olyan range, amelynek a `range_reference_t`-je *érték* (például egy `transform`, amely `std::string`-et ad vissza), akkor a tuple valódi értéket tárol: a módosítás elveszik, és a másolás költsége csendben megjelenik. Ez a leggyakoribb szemantikai hiba a kombinált pipeline-okban.

42.2.2 zip és zip_transform

A `views::zip` elemenként egy tuple-t állít elő, és a rövidebb tartománynál megáll. Két párhuzamos vektor összefűzése így nem igényel indexelést:

```
#include <ranges>
#include <string>
#include <vector>
#include <iostream>

int main() {
    std::vector<std::string> keys{"alma", "korte", "szilva"};
    std::vector<int> vals{3, 5, 7};

    for (auto&& [k, v] : std::views::zip(keys, vals))
        v *= 10;                        // a vals elemeit módosítja

    for (auto&& [k, v] : std::views::zip(keys, vals))
        std::cout << k << '=' << v << '\n';
}
```

Itt `v` típusa `int&` (pontosabban a tuple `int&` tagjára kötött név), ezért a `v *= 10` a `vals`-ba ír. Ha a ciklusfejet `auto [k, v]`-re írnánk, a viselkedés referenciatagok esetén változatlan maradna, de értéktagok esetén csendben másolnánk – ezért a `auto&&` a biztonságos alapminta.

A `views::zip_transform(f, rs...)` ugyanazt a párosítást végzi, de nem tuple-t ad, hanem `f(elemek...)` eredményét: a reference típusa `std::invoke_result_t<F&, range_reference_t<Rs>...>`. Ha `f` értéket ad, az prvalue, és írásra nem alkalmas — számított értékekhez kényelmes:

```
for (int s : std::views::zip_transform(std::plus<>{},
                                       std::views::all(vals),
                                       std::views::all(vals)))
    std::cout << s << '\n';             // 60, 100, 140
```

Ha viszont `f` referenciát ad vissza, a nézet is referenciát ad, és rajta keresztül írható. Az élettartam-szabály mindkét esetben ugyanaz: a `zip_transform_view` iterátora a forrástartományok iterátoraiból áll, és minden dereferáláskor azokon hívja `f`-et — az érték-visszatérés sem old le a forrásoktól, megszűnt forrás fölött a bejárás nem definiált viselkedés. Csak a már *kiolvasott* érték független. Költsége elemenként egy függvényhívás, amit a fordító gyakran beinlinol; allokáció nincs.

```
// zip_transform_ref.cpp - C++23: a reference típusa az invoke_result
#include <cassert>
#include <ranges>
#include <type_traits>
#include <vector>

int main() {
    std::vector<int> a{1, 2, 3}, b{10, 20, 30};
    auto pick = std::views::zip_transform(
        [](int& x, int& y) -> int& { return x < y ? y : x; }, a, b);   // int&
    static_assert(std::is_same_v<std::ranges::range_reference_t<decltype(pick)>,
     ↪  int&>);
    for (int& r : pick) r = 0;                      // írható: a b elemeit
     ↪  nullázza
    assert((b == std::vector<int>{0, 0, 0}));

    auto sum = std::views::zip_transform(std::plus<>{}, a, a);          //
     ↪  érték
    static_assert(std::is_same_v<std::ranges::range_reference_t<decltype(sum)>,
     ↪  int>);
    // Az érték-visszatérés sem old el a forrástól: a view iterátora az a
    // elemeire mutat, ezért a megszűnt a fölött a sum bejárása UB lenne.
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++23 -Wall -Wextra -Wpedantic zip_transform_ref.cpp` figyelmeztetés nélkül fordult, és lefutott: az `int&`-et adó változat reference típusa `int&`, a rajta át írt nullák a `b`-be kerültek; a `std::plus<>`-os változat reference típusa `int`.

42.2.3 adjacent, adjacent_transform és pairwise

A `views::adjacent<N>` egyetlen range-ből állít elő N-es csoportokat: a i-edik elem az $r[i], r[i+1], \ldots, r[i+N-1]$ elemekre hivatkozó tuple, és a nézet hossza

$$\max(0,\ \mathrm{size}(r) - (N-1)).$$

Az N nem-típusparaméter, `N >= 1`; az alapértelmezés 2, és a `views::pairwise` pontosan az `adjacent<2>` rövidítése. A forrásnak legalább `forward_range`-nek kell lennie, hiszen a nézetnek $N-1$ elemmel előre kell tudnia mutatni. Üres vagy egyelemű bemenetnél a nézet üres lesz – ez nem hiba, hanem a fenti képlet nulla értéke.

```
std::vector<int> ms{12, 40, 7, 90};
for (auto&& [prev, cur] : ms | std::views::pairwise)
    std::cout << (cur - prev) << ' ';   // 28 -33 83
```

A `adjacent_transform<N>(f, r)` ugyanezt teszi, de f-et alkalmazza a csoportra, tehát értéket ad vissza. Itt érdemes tudatosítani: a `pairwise` tuple-jének mindkét tagja **ugyanannak** a range-nek a hivatkozása, ezért a két név közül bármelyiken át írva ugyanazt a tárolót módosítjuk – ez néha meglepő, ha az ember két független másolatot feltételez.

42.2.4 enumerate és az index típusa

A `views::enumerate(r)` eleme `tuple<range_difference_t<R>, range_reference_t<R>>`, azaz az index **a range difference típusa**, jellemzően előjeles egész (`std::ptrdiff_t`), nem `std::size_t`. Ez tudatos döntés: a ranges könyvtárban a távolságok előjelesek. A gyakorlati következmény, hogy a `-Wsign-compare` figyelmeztetés megjelenhet, ha az indexet `size()`-hez hasonlítjuk; ilyenkor explicit konverzióval jelezzük a szándékot.

```
for (auto&& [i, rec] : std::views::enumerate(ms)) {
    if (rec > 50)
        std::cout << static_cast<long long>(i) << ": " << rec << '\n';
}
```

Az index nullától indul és növekszik. Mivel az első tag **érték** a tuple-ben, az `i` módosítása csak a lokális másolatot érinti, a bejárásra nincs hatás – ezért írásra soha ne használjuk. A második tag viszont referencia, így a `rec`-en át írva a forrás módosul.

42.2.5 cartesian_product: méret és költség

A `views::cartesian_product(rs...)` az összes kombinációt állítja elő; eleme `tuple` a résztvevő range-ek aktuális elemeire hivatkozó tagokkal. A méret a tényezők **szorzata**:

$$\text{size} = \prod_{i=1}^{n} \text{size}(R_i), \qquad \text{üres forrás esetén } 0.$$

A bejárási rend a beágyazott ciklusokhoz hasonló: a **jobb szélső** tartomány lép leggyorsabban – ezt érdemes egy rövid próbával ellenőrizni a saját implementációnkon, mert erre épülő kódot írunk. A lépésenkénti költség $O(1)$, allokáció nincs, viszont az iterátor minden forrásra tárol egy pozíciót, tehát a nézet objektummérete n-nel nő. A nagy szorzatra figyelmeztetni kell: $10^4 \times 10^4$ már 10^8 elem, amit egy naiv szűrőfeltétellel való bejárás megfizet. A helyes minta: **előbb szűrj, aztán kombinálj**, és ha a szorzat nagyságrendileg nem indokolható, használj index- vagy hash-alapú megoldást.

42.2.6 Borrowed range, rvalue forrás és dangling

Négy esetet kell élesen megkülönböztetni.

Ha a `views::all` argumentuma **lvalue konténer**, `ref_view` keletkezik: borrowed, de a konténer élettartamához kötött. Ha **rvalue konténer**, `owning_view` keletkezik, amely elmozgatja a konténert, és ezzel biztonságossá teszi a nézetet – ezért helyes a `views::zip(std::vector{1,2}, std::vector{3,4})` forma. Ha viszont az argumentum már **view**, akkor csak másolat készül: a nézet nem birtokolja a mögöttes adatot, és ha az megszűnik, dangle. Végül az ideiglenes `zip_view`-ból kimentett iterátor mindig veszélyes, mert a `zip_view` nem borrowed range.

```cpp
// HIBA (tanítási példa): a helyi string megszűnik, a nézet dangling
auto make_bad() {
    std::string s = "alma";
    return std::views::zip(s, std::views::single(1)); // ref_view<string>
}

// HELYES: az adatot adjuk vissza, a nézetet a hívó építse
std::vector<std::string> make_ok() { return {"alma"}; }
```

A második élettartam-szabály az invalidáció: a kombinált nézet semmit nem cache-el, ezért minden olyan művelet, amely a forrás iterátorát érvényteleníti (`push_back`, `erase`, `std::string` módosítása), a kombinált iterátort is érvényteleníti. Egy `zip` iterátor „ereje” mindig a leggyengébb forrásáé.

42.2.7 kidolgozott példa: naplórekordok négy nézettel

A példa egy koherens pipeline: a rekordok `level` + `ms` mezőt tartalmaznak, a `level` egy `string_view`, amely a hívó `lines` vektorára mutat – ezt a tulajdonviszonyt kommentben rögzítjük.

```cpp
// logzip.cpp - C++23; a tamogatast feature-test makroval ellenorizd
#include <ranges>
#include <string>
#include <string_view>
#include <vector>
#include <iostream>
#include <charconv>

struct rec { std::string_view level; int ms; };  // nem tulajdonol: lines
 ↪  elemeire mutat

std::vector<rec> parse(const std::vector<std::string>& lines) {
    std::vector<rec> out;
    for (auto&& line : lines) {
        auto sp = line.find(' ');
        if (sp == std::string::npos) continue;
        int ms = 0;
        auto [p, ec] = std::from_chars(line.data() + sp + 1,
                                       line.data() + line.size(), ms);
        if (ec != std::errc{}) continue;
        out.push_back({std::string_view{line}.substr(0, sp), ms});
    }
    return out;
}

int main() {
    std::vector<std::string> lines{"INFO 12", "WARN 40", "WARN 7", "ERROR 90"};
    auto recs = parse(lines);                       // recs elemei lines-ra
     ↪  mutatnak

    for (auto&& [i, r] : std::views::enumerate(recs))
        std::cout << i << ' ' << r.level << ' ' << r.ms << '\n';
```

```
    for (auto&& [a, b] : recs | std::views::pairwise)
        std::cout << "delta " << (b.ms - a.ms) << '\n';

    std::size_t worse = 0;
    for (auto&& [a, b] : std::views::cartesian_product(recs, recs))
        if (a.level == b.level && a.ms < b.ms) ++worse;
    std::cout << "worse pairs: " << worse << '\n';

    for (int s : std::views::zip_transform(std::plus<>{},
             recs | std::views::transform(&rec::ms),
             recs | std::views::transform(&rec::ms)))
        std::cout << "2x " << s << '\n';
}
```

Build (a fordítás eredményét nem állítom mértnek; a tanuló laborjában ellenőrizendő):

```
g++ -std=c++23 -O2 -Wall -Wextra -o logzip logzip.cpp
# vagy: clang++ -std=c++23 -O2 -Wall -Wextra -o logzip logzip.cpp
```

Támogatás-ellenőrzés: a `zip`/`adjacent` családhoz `__cpp_lib_ranges_zip`, az `enumerate`-hez `__cpp_lib_ranges_enumerate`, a `cartesian_product`-hoz `__cpp_lib_ranges_cartesian_product` makrót keress a `<version>` headerben; ha bármelyik hiányzik, a nézet nem használható, és a C++20-as fallback (indexszel való bejárás, `std::forward_as_tuple`-tel) jön szóba.

42.2.8 Tipikus hibák és javításuk

A `auto& x = *it;` nem köt prvalue-ra; `auto&&` kell. Az `enumerate` indexe előjeles, ezért `size()`-hez hasonlítva figyelmeztet, és a `static_cast` hiánya `-Werror` mellett fordítási hibát ad. A függvényből visszaadott, lokális konténerre épülő `zip` dangling; ilyenkor az adatot kell visszaadni, nem a nézetet. A `transform` által értékkel visszaadott elemeket a `zip` csendben lemásolja; ha írni akarunk, a transformnak referenciát kell adnia. Végül a `cartesian_product` mérete szorzódik: nagy forrásoknál előbb szűrjünk.

42.2.9 Összefoglaló

A `zip`/`zip_transform`/`adjacent`/`enumerate`/`cartesian_product` mind ugyanarra a szerződésre épül: a dereferálás tuple, amelynek tagjai **referenciák vagy értékek** aszerint, hogy a forrás mit ad. A proxy referencián keresztül írni csak akkor lehet, ha a forrás valódi referenciát ad; az élettartam pedig soha nem hosszabb, mint a legrövidebb forrásé, és a nézet nem borrowed, ha ideiglenesből készült. Az `enumerate` indexe difference típus, a `cartesian_product` mérete szorzat.

42.2.10 Feladatok

1. feladat (elemzés). Add meg `std::views::zip(v, w)` `reference` és `value_type` típusát, ahol `v` `std::vector<int>`, `w` `std::vector<double>`. Írj `static_assert`-et, amely ezt ellenőrzi, és mondd meg, mi történik, ha a `w` egy `transform` nézet, amely `std::string`-et ad vissza értékként.

2. feladat (kódolás). Számítsd ki egy `std::vector<int>` szomszédos különbségeit `views::adjacent<2>`-vel, és írd ki azokat a helyeket, ahol a különbség negatív. Üres és egyelemű bemenetre is legyen helyes.

3. feladat (diagnosztika). Az alábbi függvény fordítódik, de a hívó oldalon dangle-t okoz. Találd meg az okot, és javítsd két különböző módon.

```
auto head(const std::vector<std::string>& v) {
    return std::views::zip(v, std::views::enumerate(v));
}
```

4. feladat (költség). Két `n = 10000`-es vektor minden párosításában keresed azokat a párokat, ahol az első érték kisebb a másodiknál. Becsüld meg a `cartesian_product` lépésszámát, és adj egy olyan megoldást, amely előszűréssel csökkenti a jelölteket.

42.2.11 Megoldások

1. megoldás. A `zip` referenciája `std::tuple<int&, double&>`, értéktípusa `std::tuple<int, double>`. Az ellenőrzés:

```
using Z = decltype(std::views::zip(std::declval<std::vector<int>&>(),
                                   std::declval<std::vector<double>&>()));
static_assert(std::is_same_v<std::ranges::range_reference_t<Z>,
                             std::tuple<int&, double&>>);
static_assert(std::is_same_v<std::ranges::range_value_t<Z>,
                             std::tuple<int, double>>);
```

Ha a második forrás `transform`, amely `std::string`-et ad vissza értékként, akkor `range_reference_t` `std::tuple<int&, std::string>`, tehát a tuple valódi `std::string`-et tárol: a `zip` elemenként másol, és a `string`-en át írni nincs hatás. Ellenőrzési feltétel: a `static_assert` a `tuple<int&, std::string>` típusra teljesül, és a fordító nem jelez hibát a `zip` létrehozásakor.

2. megoldás.

```
std::vector<int> v{5, 3, 9};
for (auto&& [i, d] : std::views::enumerate(v | std::views::adjacent<2>
        | std::views::transform([](auto t) { return std::get<1>(t) -
        ↪ std::get<0>(t); }))) {
    if (d < 0) std::cout << i << " csokkenes: " << d << '\n';
}
```

Egyszerűbb, ha rögtön a tuple-t kötjük ki:

```
for (auto&& [a, b] : v | std::views::adjacent<2>)
    if (b - a < 0) std::cout << (b - a) << '\n';
```

Üres vagy egyelemű `v` esetén az `adjacent<2>` üres, a ciklusmag nem fut le, és nincs UB. Ellenőrzési feltétel: `{5,3,9}`-re egy kiírás (`-2`), `{}`-re és `{7}`-re nulla kiírás.

3. megoldás. A `v` itt lvalue referencia, de a `std::views::enumerate(v)` ideiglenes nézet, és a `zip` referenciatagja erre az ideiglenesre mutat. A `zip_view` a bejárás során dereferálja az `enumerate` iterátorát, amely viszont egy olyan objektumra épül, amely a teljes kifejezés végén megszűnik: dangle. Első javítás: az adatot adjuk vissza, és a hívó építse a nézetet:

```
const std::vector<std::string>& head(const std::vector<std::string>& v) { return
 ↪  v; }
```

Második javítás: ha a sorszámozott nézetre tényleg szükség van, azt a hívó oldalon, lokálisan hozzuk létre, a `v` élettartamán belül:

```
for (auto&& [s, ip] : std::views::zip(v, std::views::enumerate(v))) { /* ... */
 ↪  }
```

Ellenőrzési feltétel: a javított kód nem ad vissza nézetet lokálisra, és a `-Wdangling` típusú figyelmeztetés (ha az adott fordító ad ilyet) eltűnik; a viselkedés független attól, hogy a hívó milyen sorrendben használja a visszakapott objektumot.

4. megoldás. A `cartesian_product` lépésszáma $n^2 = 10^8$, ami már önmagában is érezhető költség, függetlenül attól, hogy a feltétel gyors. Előszűrés: rendezzük a második vektort, és minden x-re bináris kereséssel számoljuk meg a nála nagyobbakat – ez $O(n \log n)$; vagy ha `cartesian_product`-ot használunk, előbb szűrjük a második vektort egy küszöbre, és csak a jelöltekkel kombináljunk. A kisebb költségű megoldás indoklása: a szorzatméret multiplikatív, ezért a jelöltszám csökkentése négyzetesen megtérül. Ellenőrzési feltétel: a két megoldás ugyanazt a darabszámot adja kis ($n = 100$) bemeneten, és a lépésszám a másodikban nagyságrendekkel kisebb.

42.3 Szeletelő és átalakító view-k: chunk, slide, chunk_by, stride, join_with

A naplófeldolgozás tipikus DevOps-feladata, hogy a sorokban érkező rekordokat ne egyenként, hanem valamilyen szerkezetben vizsgáljuk: azonos szintű bejegyzések blokkjai, egymást követő események ablakai, minden n-edik minta, vagy éppen a blokkok ember által olvasható riporttá fűzése. A C++23 ranges könyvtára pontosan ezekre a műveletekre ad néhány szűk, jól körülhatárolt view-t. Ez a rész ezeket a view-kat tanítja meg úgy, hogy a hangsúly ne az egyes függvényneveken, hanem a szerződéseken legyen: milyen forrásrange-et fogadnak el, mit tárolnak, mit invalidálnak, és mennyibe kerül a bejárásuk.

Egy megjegyzés a bizonyítékokról. A fejezetterv több WG21-javaslatot megnevez (többek között a `ranges::to`, a `zip`/`adjacent`, a `fold`, valamint a szeletelő view-k forrásaiként), és a fejezethez tartozó forrásjegyzet nem tartalmaz általam olvasott, konkrét szakaszra mutató hivatkozásokat. Ezért ebben a részben nem állítom, hogy egy adott GCC-, Clang-, libstdc++- vagy libc++-verzió implementálja az alábbiakat, és nem állítom azt sem, hogy bármely kód itt lefuttatásra és ellenőrzésre került. A laborprofilban rögzített fordítóval kell feature-test makróval és compile probe-bal ellenőrizni a támogatást; ahol a szabványváltozat és az implementáció eltérhet, azt külön jelzem.

42.3.1 A szeletelő view-k közös szerződése

A fejezet didaktikai felosztásában *szeletelő* view-k azok, amelyek a forrás elemeit **részrange-ek sorozatává** alakítják: a `chunk`, a `slide` és a `chunk_by`. Ezek eredménye egy range-of-ranges, amelynek elemei jellemzően `subrange` típusú, nem tulajdonló nézetek. Ezzel szemben az *átalakító* view-k (`stride`, `join_with`, `repeat`, `as_const`, `as_rvalue`) az elemek sorozatának alakját vagy kategóriáját változtatják meg, de nem képeznek beágyazott range-eket. Ez a felosztás nem szabványos kategorizálás, kizárólag a tananyag eligazodását szolgálja.

Közös szerződésük három pontban foglalható össze. Először, a szeletelő view maga nem másolja az elemeket; hogy a forrást birtokolja-e, az az alap-view típusán múlik (ranges::to és tárolókonstrukció range-ből (42.1)). Lvalue forrásnál `ref_view` alapra épül, és csak hivatkozik a mögöttes range-re, tehát a Lazy view pipeline és mellékhatások (30.4)-ben tárgyalt élettartamszabályok érvényesek rá. Rvalue, nem view forrásnál `owning_view` alapra épül, és birtokolja azt. Ha a forrás egy lokális `std::vector`, akkor az erre lvalue-ként épített view addig használható, amíg a vektor él; a view visszaadása függvényből dangling hivatkozást eredményez. Ezen a `borrowed_range` tulajdonság sem segít: a `std::span` például borrowed range, de csak annyit jelent, hogy iterátorai nem a span objektumhoz, hanem a mögöttes tárolóhoz kötöttek. A lokális vektort nem tartja életben, így a `span(local) | views::take(2)` visszaadása ugyanúgy dangling (ASan alatt `heap-use-after-free`, lásd a ranges::to és tárolókonstrukció range-ből (42.1) próbáját). Biztonságos visszaadási minta: a tárolót mozgatással a view-ba adni (`std::move(local) | views::chunk(3)`, ami `owning_view`-t ad), vagy olyan külső tulajdonosra építeni, amely bizonyítottan tovább él. Másodszor, a szeletelő view-k feltételeket támasztanak a forrás kategóriájára, és ez a feltétel nem esztétikai: a `slide` és a `chunk_by` csak `forward_range` felett működik, mert egyszerre több pozíciót kell megjegyezniük. Harmadszor, a méret- és darabszám-számítások csak akkor konstans idejűek, ha a forrás `sized_range`; ellenkező esetben a `distance` végiglépked.

42.3.1.1 chunk

A `views::chunk(n)` a forrást legfeljebb `n` elemű, nem átfedő, sorrendtartó szeletekre bontja. A szeletek száma `sized_range` forrás esetén

$$\left\lceil \frac{N}{n} \right\rceil = \left\lfloor \frac{N+n-1}{n} \right\rfloor,$$

ahol N a forrás elemszáma és $n > 0$ egész. A levezetés egyszerű: írjuk fel $N = qn + r$ alakban, ahol $0 \le r < n$. Ha $r = 0$, akkor $N + n - 1 = qn + n - 1$, amit n-nel egész osztva $q + \frac{n-1}{n}$, azaz q adódik, ami $\lceil N/n \rceil$. Ha $r > 0$, akkor $N + n - 1 = qn + (r + n - 1)$, ahol $n \le r + n - 1 \le 2n - 2$, tehát az egész osztás eredménye $q + 1$, ami szintén a felső egészrész. Az utolsó szelet tehát rövidebb lehet, és ez nem hiba, hanem a szerződés része: aki egyenlő méretű szeleteket vár, annak magának kell ellenőriznie a hosszt.

Az `n > 0` előfeltétel megsértése nem definiált viselkedés, illetve a könyvtár implementációjától függően assert. A `chunk` állapota konstans méretű: csak az aktuális szelet elejét és a hátralévő darabszámot követi, ezért a bejárás költsége lineáris, extra allokáció nélkül. A szeletek típusa `subrange`, amelynek kategóriája a forrás kategóriáját követi.

Itt van a rész legkényesebb pontja, amit a fejezetterv külön kér. Ha a forrás `forward_range`, a `chunk` eredménye is forward range, a szeletek többször bejárhatók, és `sized_range` forrás esetén a szelethossz is ismert. Ha a forrás csak `input_range`, akkor a keletkező view is csak input range marad: a szeleteket nem lehet kétszer bejárni, és nem lehet rájuk támaszkodni úgy, hogy „előbb megszámolom, aztán feldolgozom”. Történelmi okból a korai `chunk_view` tervezet szigorúbban, csak forward range-re volt értelmezve; a későbbi változat engedte meg az input range-et azzal a kompromisszummal, hogy a szeletek kategóriája lesüllyed. Mivel ezt nem tudom ellenőrzött, konkrét implementációhoz kötni, a gyakorlati szabály ez: **ha a kódod feltételezi, hogy egy chunk kétszer bejárható vagy `size()`-ja konstans idejű, azt statikusan is ki kell mondanod**, például `static_assert(std::ranges::forward_range<decltype(chunkView)>);` formában, és a laborban ellenőrizni kell, hogy a választott könyvtár ezt teljesíti-e.

42.3.1.2 slide

A `views::slide(n)` minden pozícióra egy `n` hosszú, átfedő ablakot ad: az első ablak az első `n` elemet tartalmazza, a következő a másodikat a harmadiktól kezdve, és így tovább. A darabszám `N >= n` esetén $N - n + 1$, egyébként nulla. A `slide` előfeltétele `n>0`, és a forrásnak `forward_range`-nek kell lennie, mert ugyanazokat az elemeket több ablak is bejárja. A kiegészítő állapot — a forrás tárolásától eltekintve — $O(1)$ (C++23 [range.slide]): a view `n`-et és legfeljebb egy gyorsítótárazott `begin`-/`end`-pozíciót tárol, az iterátor pedig az ablak elejét, a méretet és (nem random access forrásnál) az ablak utolsó elemére mutató iterátort; ablakhossztól függő iterátortömb nincs. A költség máshol jelentkezik: nem random access forrásnál az első `begin()` az `n-1` elemnyi előreléptetés miatt $O(n)$, és az összes ablak teljes bejárása — mivel minden ablak `n` elemet ad — összesen $O(N \cdot n)$ elemlátogatás.

A `slide` tipikus felhasználása az átmenetdetektálás: „ mely esemény következett közvetlenül egy másik után”. Erre a C++23 egy másik eszköze, az `adjacent` is használható, amelyet a Kombináló view-k: zip, adjacent, enumerate, cartesian_product (42.2) tárgyal; a választás köztük jellemzően azon múlik, hogy rögzített 2-es vagy 3-as ablaka van-e a kódnak, vagy futásidejű `n`-re van szükség.

42.3.1.3 chunk_by predikátummal

A `views::chunk_by(pred)` a szomszédos elemeket addig vonja össze egy szeletbe, amíg a predikátum igazat nem ad. A predikátumot a könyvtár `pred(*előző, *aktuális)` sorrendben hívja meg, és a szeletek soha nem üresek. A forrásnak itt is `forward_range`-nek kell lennie, mert a view a szelet határát visszafelé is meg kell hogy tudja állapítani.

A legfontosabb szemantikai pontosítás: a `chunk_by` **nem ekvivalenciaosztályokat képez**, hanem a szomszédos párokra támaszkodó maximális láncokat. Ha a predikátum nem tranzitív, akkor előfordulhat, hogy A és B egy csoportba kerül, B és C is, miközben A és C között a predikátum hamis. Például a `|a-b| <= 1` predikátummal az `{1,2,3}` bemenet egyetlen csoport lesz, pedig az 1 és a 3 különbsége 2. Ez nem hiba, hanem a definíció következménye; aki ekvivalenciaosztályokat akar, annak a csoportosítás előtt normalizálnia kell a kulcsot, vagy asszociatív tárolót kell használnia.

42.3.2 Átalakító view-k: stride, join_with, repeat

A `views::stride(n)` minden `n`-edik elemet adja vissza: a $0, n, 2n, \ldots$ indexűeket. Előfeltétele `n>0`, és mivel csak előrefelé lépked, `input_range` forrás is elegendő. `sized_range` forrás esetén az eredmény hossza $\lceil N/n \rceil$. A `stride` nem pufferel, állapota konstans méretű, viszont a forrás iterátorát `n`-szer lépteti minden egyes elemnél; gyorsítótárazott, szekvenciális bejárásnál ez jól viselkedik, véletlen hozzáférésű, de nagy kihagyásokkal járó bejárásnál viszont minden lépés cache-vonalat ugorhat.

A `views::join_with(pattern)` egy range-of-ranges elemeit fűzi össze, a belső range-ek közé a `pattern`-t szúrva. A pattern lehet egyetlen érték vagy maga is egy range. A típuskompatibilitás itt valódi követelmény, de a megfelelő szintet kell nézni. `Outer = std::vector<std::string>` esetén `range_value_t<Outer>` a `std::string` (a belső *range*), a belső range *elemtípusa* viszont `range_value_t<range_reference_t<Outer>>`, azaz `char`. A patternnek ez utóbbival kell közös típusra leképezhetőnek lennie. Egyetlen elemből álló pattern (`'|'`) esetén a pattern elemtípusa `char`, tehát a `parts | views::join_with('|')` szabályos, és `"a"`, `"b"` esetén `"a|b"`-t ad; ugyanígy működik a `join_with(std::string(" | "))` is. A `join_with` input range-et is elfogad, és a belső range-eket is csak egyszer járja be, ami fontos a lusta pipeline-oknál.

```
#include <cassert>
#include <ranges>
#include <string>
#include <type_traits>
#include <vector>
int main() {
    std::vector<std::string> parts{"a", "b"};
    using Outer = decltype(parts);
    static_assert(std::is_same_v<std::ranges::range_value_t<Outer>,
     ↪  std::string>);

     ↪  static_assert(std::is_same_v<std::ranges::range_value_t<std::ranges::range_reference
     ↪  char>);
    std::string out;
    for (char c : parts | std::views::join_with('|')) out += c;
    assert(out == "a|b");
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++23 -Wall -Wextra -Wpedantic` alatt figyelmeztetés nélkül fordult, a két `static_assert` és az `out == "a|b"` teljesült.

A `views::repeat(v)` végtelen range-et ad, amely minden iterációnál `v`-t szolgáltatja; a `views::repeat(v, n)` pontosan `n` elemet ad. A végtelen változat csak korlátozó view-val (`take`, `take_while`) együtt használható. A két hibakategória különbözik: a végtelen `repeat_view` nem `sized_range`, ezért a `size()` hívása rajta **fordítási hiba**; a `std::ranges::distance` viszont lefordulhat, és végigjárva **nem terminál** — ezt normál laborban időkorlát nélkül nem futtatjuk. A korlátos `repeat(v, n)` és egy végtelen forrásra tett `take(n)` sized: a mérete `n`. **Buildjegyzőkönyv** (`lab-gcc`, `-std=c++23`): a `sized_range static_assert`-ek teljesültek, `repeat(7, 3).size() == 3` és `(repeat(7) | take(5)).size() == 5`; a végtelen `repeat(7).size()` `no matching function for call to 'repeat_view<int, unreachable_sentinel_t>::size()'` hibát adott. A `repeat` értékszemantikájú, a `v`-t másolja, tehát nagy objektum esetén érdemes `ref`-fel vagy `views::repeat(std::ref(x))` formában használni — utóbbi esetben viszont a hivatkozott objektum élettartama a felhasználó felelőssége.

42.3.3 as_const és as_rvalue

A `views::as_const` a forrás elemeihez konstans hozzáférést biztosító view-t ad, ami akkor hasznos, ha egy amúgy nem konstans tárolón csak olvasó pipeline-t akarunk futtatni, és szeretnénk, hogy a fordító ezt kikényszerítse. A `views::as_rvalue` ezzel szemben minden elemet rvalue referenciaként ad tovább, ami azt teszi lehetővé, hogy egy `transform` vagy `join_with` move-oljon a másolás helyett. Mindkettő csak akkor viselkedik „ingyen” módon, ha a forrás kategóriája engedi; a pontos `const`- és value category szabályokat a szabvány rögzíti, és ezek implementációtól függő részletei (például a `const` iterátorok pontos típusa) verziófüggők lehetnek.

42.3.4 Kidolgozott példa: naplórekordok csoportosítása

A példa egy rövid naplót dolgoz fel: szint szerint csoportosít, átmeneteket keres, mintavételez, majd riporttá fűz. A kód önállóan fordítható, de — hangsúlyozom — itt nem állítom, hogy bármely konkrét fordítóval lefutott; a laborban kell ellenőrizni.

```cpp
// log_slice.cpp
// Feltételezett build (a laborprofilban rögzített verziókkal):
//   g++     -std=c++23 -O2 -Wall -Wextra -pedantic log_slice.cpp -o log_slice
//   clang++ -std=c++23 -O2 -Wall -Wextra -pedantic log_slice.cpp -o log_slice
#include <iostream>
#include <ranges>
#include <string>
#include <string_view>
#include <vector>

namespace rv = std::views;
namespace rs = std::ranges;

struct entry {
    std::string_view level;
    std::string_view message;
};

static entry parse(std::string_view line)
{
    const auto sp = line.find(' ');
    if (sp == std::string_view::npos)
        return {line, {}};
    return {line.substr(0, sp), line.substr(sp + 1)};
}

int main()
{
    const std::vector<std::string> lines = {
        "INFO start", "INFO warmup", "WARN slow", "WARN retry",
        "ERROR fail", "INFO done"
    };

    std::vector<entry> entries;
    entries.reserve(lines.size());
    for (const auto& l : lines)
        entries.push_back(parse(l));

    // 1) Szomszédos, azonos szintű rekordok csoportosítása.
    auto groups = entries
                | rv::chunk_by([](const entry& a, const entry& b) {
                      return a.level == b.level;
                  });

    for (auto g : groups) {
        std::cout << g.front().level << " x" << rs::distance(g) << '\n';
    }

    // 2) Csúszóablak: WARN -> ERROR átmenetek.
    for (auto w : entries | rv::slide(2)) {
        if (w[0].level == "WARN" && w[1].level == "ERROR")
            std::cout << "atmenet: " << w[0].message
```

```
                  << " -> " << w[1].message << '\n';
    }

    // 3) Mintavételezés minden harmadik rekordból.
    for (const auto& e : entries | rv::stride(3))
        std::cout << "minta: " << e.level << '\n';

    // 4) Riport: csoportok szöveggé, elválasztóval összefűzve.
    auto parts = groups
               | rv::transform([](auto g) {
                     return std::string(g.front().level) + " x"
                          + std::to_string(rs::distance(g));
                 })
               | rv::join_with(std::string(" | "));

    for (const auto& p : parts)
        std::cout << p;
    std::cout << '\n';

    // 5) Végtelen forrás korlátozása.
    for (char c : rv::repeat('-') | rv::take(20))
        std::cout << c;
    std::cout << '\n';
}
```

A példa élettartam-térképe egyszerű, de kimondandó: az `entries` vektor a `main` teljes ideje alatt él; a `groups`, `parts` és a `slide`/`stride` view-k mind erre hivatkoznak; az `entry::level` és `entry::message` `string_view`-k a `lines` vektor elemeire mutatnak, tehát a `lines` élettartama szintén kritikus. Ha bármelyik view-t visszaadnánk a `main`-ből, a `lines` és az `entries` megszűnésével dangling nézetet kapnánk. Az `as_const` itt azért lenne hasznos, hogy a 4. pontbeli `transform` véletlenül se módosíthassa a csoportokat.

A `rs::distance(g)` hívás a `chunk` szeletén `sized_range` forrás esetén konstans idejű; a vector ilyen, tehát a példa lineáris marad. Ha a forrás egy generátor vagy egy `istream_view` lenne, a `distance` végiglépkedne a szeleten, és a 4. pontban ez négyzetes viselkedést okozhatna.

42.3.5 Pipeline-teljesítmény és cache-elés

A lusta pipeline költsége három forrásból áll össze. Az első a **bejárási lépések száma**: a `chunk_by` minden új elemnél legfeljebb egy predikátumhívást végez (a szelet határán), a `stride` minden kimeneti elemhez `n` iterátorlépést, a `join_with` minden elemhez egy belső iterátorlépést és a határokon a pattern kiírását. A második a **tárolt állapot**: a `chunk`, a `stride` és a `join_with` konstans méretű, a `slide` $O(n)$, a `chunk_by` konstans méretű, de a predikátum objektumot tárolja, amely lambdák esetén capture-öket is tartalmazhat. A harmadik a **cache-viselkedés**: a `stride` nagy `n`-nél minden egyes kimeneti elemhez `n` egységet ugrik, ami szekvenciális forráson is cache-vonal-ugrálást okoz; a `slide` ezzel szemben szomszédos ablakokat ad, tehát locality szempontjából kedvező, de az ablakok között az elemeket újra és újra feldolgozza, ami a felhasználói kódban könnyen $O(N \cdot n)$ munkához vezet.

Cache-elés szempontjából a `chunk` a legbarátságosabb: a szeletek nem fednek át, minden elemet egyszer ad át a feldolgozónak. A `slide` és a `chunk_by` esetében érdemes megmérni, hogy a feldolgozó lépés idempotens-e, illetve hogy az átfedés miatti ismételt munka mennyi; a Profilozás,

benchmark és cost model (37.4)-ben tárgyalt mérési fegyelem itt is érvényes: a várt és a mért költség nem ugyanaz a kategória.

42.3.6 Tipikus hibák

A leggyakoribb hiba a **`n = 0` átadása** a `chunk`, `slide` vagy `stride` view-nak. Ez előfeltétel-sértés; a helyes védekezés a bemenet ellenőrzése a view létrehozása előtt, például `if (n == 0) throw std::invalid_argument("n");`, nem pedig a view „megjavítása”.

A második a **forward range feltétel figyelmen kívül hagyása**: `slide`-t vagy `chunk_by`-t input range-re próbálni nem fordul le, és ez nem korlátozás, hanem a szerződés. Ha a forrás csak input range, a `chunk` marad, de ott is számolni kell a szeletek egyszeri bejárhatóságával.

A harmadik a **dangling predicate**: a `chunk_by` lambdája referenciát capture-el egy lokális változóra, majd a view tovább él, mint a captured objektum. A javítás a capture élettartamának összehangolása vagy érték szerinti capture.

A negyedik a **`join_with` típuseltérése**: karaktert adunk patternként stringek range-éhez, vagy `std::string_view`-t `std::string`-hez; a fordítási hibaüzenet hosszú és nehezen olvasható, ezért érdemes a pattern típusát expliciten a belső range value type-jához igazítani.

Az ötödik a **végtelen `repeat` korlátozás nélkül**: `distance` vagy `size` hívása végtelen ciklust eredményez, és a `take` elfelejtése is könnyen megesik.

42.3.7 Összefoglaló

A C++23 szeletelő view-k közül a `chunk` nem átfedő, a `slide` átfedő ablakokat ad, a `chunk_by` pedig predikátum alapján, szomszédos párokra támaszkodva csoportosít. A `chunk`, a `stride` és a `join_with` konstans állapotú, a `slide` $O(n)$ iterátortárolást igényel, a `chunk_by` pedig a predikátum objektumot tartja életben. A `slide` és a `chunk_by` forward range-et követel; a `chunk` input range-en is elfogadott, de ilyenkor a szeletek kategóriája lesüllyed, és a többszöri bejárhatóságra nem lehet építeni. A `stride` és a `join_with` a sorozat alakját változtatja, a `repeat` végtelen forrást ad, az `as_const` és az `as_rvalue` pedig a hozzáférés módját rögzíti. Mindegyik view nem tulajdonló, ezért a forrás élettartama a felhasználó felelőssége.

42.3.8 Feladatok

1. feladat (alap). Adott a `std::vector<int> v{1,2,3,4,5,6,7};`. Készíts `v | rv::chunk(3)` pipeline-t, és írd ki az egyes szeletek elemeit, valamint a szeletek számát. Hány elemű az utolsó szelet, és miért?

2. feladat (közepes). Csoportosítsd a `{1,2,3,10,11,20}` sorozatot `chunk_by`-val a `|a-b| <= 1` predikátummal. Magyarázd el, miért kerül az 1 és a 3 ugyanabba a csoportba, és mutass egy olyan bemenetet, ahol a „láncolt” viselkedés meglepő eredményt ad.

3. feladat (diagnosztikai). Írj egy `make_stride(std::size_t n)` függvényt, amely `n == 0` esetén `std::invalid_argument`-t dob, egyébként visszaadja a `rv::stride(n)` view-t egy paraméterként kapott tartományra. Magyarázd el, miért nem védhető a `stride` view belülről, és hogyan ellenőriznéd a viselkedést a laborban.

42.3.9 Megoldások

1. megoldás. A `chunk(3)` három szeletet ad: `[1,2,3]`, `[4,5,6]`, `[7]`. A darabszám a felső egészrész képlettel $\lceil 7/3 \rceil = 3$, ami a $\lfloor (7+3-1)/3 \rfloor = \lfloor 9/3 \rfloor = 3$ alakkal is számítható. Az utolsó szelet azért 1 elemű, mert $7 = 2 \cdot 3 + 1$, tehát a maradék nem nulla; a `chunk` szerződése szerint az utolsó szelet rövidebb lehet. Ellenőrzési feltétel: a külső view `std::ranges::distance`-ja 3, az utolsó szelet `std::ranges::distance`-ja 1, és a szeletek egyesítése kiadja az eredeti sorozatot.

2. megoldás. Az eredmény `[1,2,3]`, `[10,11]`, `[20]`. Az 1 és a 3 azért kerül egy csoportba, mert a `chunk_by` csak a szomszédos párokra kérdez rá: `pred(1,2)` igaz, ezért 2 még a csoportban marad; `pred(2,3)` igaz, ezért 3 is; az 1 és a 3 közötti viszonyt a view soha nem vizsgálja. Meglepő eset: `{5,6,7,8,20}` esetén `[5,6,7,8]` egyetlen csoport lesz, pedig az 5 és a 8 különbsége 3. Ez nem hiba, hanem a szomszédoslánc-szemantika; ha ekvivalenciaosztályokra van szükség, a kulcsot normalizálni kell (például egész osztással), vagy asszociatív tárolót kell használni. Ellenőrzési feltétel: a csoportok száma és sorrendje megfelel a szomszédos határoknak, és a csoportok összefűzése az eredeti sorozatot adja.

3. megoldás. A `stride` view előfeltétele a konstruktorban rögzített `n > 0`; a view-nak nincs olyan tagfüggvénye, amely ezt futásidőben ellenőrizné és helyreállítaná, mert a ranges view-k szerződése a hívó felelőssége. A védőburkoló tehát a view létrehozása előtt ellenőriz:

```
[[nodiscard]] auto make_stride(std::size_t n)
{
    if (n == 0)
        throw std::invalid_argument("stride: n must be positive");
    return std::views::stride(n);
}
```

Mivel a visszaadott view a `n`-et érték szerint tárolja, a burkoló élettartama nem probléma; a forrásrange-re viszont ugyanaz a dangling-szabály vonatkozik, mint bármely más view-ra. A laborban a helyes ellenőrzés: (a) feature-test makróval igazolni, hogy a `stride` elérhető; (b) `n = 0`-ra a burkoló dob, és ezt negatív tesztként rögzíteni; (c) a `stride(0)` közvetlen hívását nem szabad „tesztelni”, mert viselkedése nem definiált, és az eredmény nem hordozható megfigyelés.

42.4 C++23 range-algoritmusok és ranges-labor

A C++20 ranges alapjai után a C++23 több olyan algoritmust tett a `std::ranges` névtérbe, amelyeket addig csak kézzel, ciklussal vagy a régi `<numeric>`/`<algorithm>` függvényekkel lehetett megoldani. Ez a rész ezeket a **szerződés** oldaláról tárgyalja: mi a pontos visszatérési típus, milyen range-kategória kell, mi történik üres bemenetnél, és hol tér el a viselkedés a megszokott `std::accumulate`-től. A záró labor egy naplóelemző pipeline-t épít fel kétszer, C++20-ban és C++23-ban, hogy a különbség ne csak stílusbeli legyen, hanem mérhető és ellenőrizhető.

A rész alapprofilja ISO C++23, `-std=c++23`. A konkrét fordító- és könyvtárverzió támogatását itt nem tekintjük ellenőrzöttnek: minden új képességhez feature-test makrót és compile probe-ot adunk, és ahol szükséges, fallbacket jelzünk. A C++20-ban már bevezetett ranges-alapokat (projection, sentinel, view pipeline, borrowed range) nem tanítjuk újra; ezekre a Ranges algoritmusok, projection és sentinel (30.3) és Szeletelő és átalakító view-k: chunk, slide, chunk_by, stride, join_with (42.3) részekre hivatkozunk.

42.4.1 A fold család: bal- és jobbasszociatív összegzés

A `std::ranges::fold_left`, `fold_right`, `fold_left_first` és `fold_right_first` a P2322 javaslat nyomán került a szabványba. Közös jellemzőjük, hogy **determinisztikus sorrendben** dolgozzák fel a range elemeit, ellentétben a C++17-es `std::reduce`-dzsel, amelynek a sorrendje nem meghatározott, és amely ezért nem is használható nem asszociatív művelettel. A foldok mindig balról jobbra, illetve jobbról balra haladnak, így a művelet asszociativitása nem követelmény.

A `fold_left` lényegében a ranges-esített `accumulate`. A deklarációja leegyszerűsítve:

```
template <std::input_range R, class T, class F>
  constexpr auto fold_left(R&& r, T init, F f) -> std::decay_t<T>;
```

A visszatérési típus az **init típusa** (`decay_t<T>`), nem a range értéktípusa. Ez a legfontosabb szerződésbeli pont: ha `init int`, akkor az akkumulátor végig `int` marad, akkor is, ha a range elemei `long long`-ok, és minden részeredmény `int`-té konvertálódik. A `f` hívása `f(acc, *it)` alakú, és a követelmény, hogy a visszatérési értéke konvertálható legyen `T`-vé. A `fold_right` ugyanezt jobbról teszi, de **`bidirectional_range`-et igényel**, mert hátulról kell indulnia, és a művelet argumentumsorrendje megfordul: `f(*it, acc)`.

A `fold_left_first` kezdőérték nélkül dolgozik: az első elemet használja kezdőértékként, ezért **üres range esetén `std::nullopt`-ot ad vissza**, nem hibát. A visszatérési típus `std::optional<U>`, ahol `U = std::decay_t<std::ranges::range_value_t<R>>`. Itt van a gyakori csapda: a visszatérési típus az **első elem típusából** származik, nem abból, amit a művelet visszaad. Ha az elemek `int`-ek, de a művelet `double`-t adna, az eredmény mégis `optional<int>` lesz, és a törtrész elveszik. A `fold_right_last` ugyanez jobbról, szintén `bidirectional_range`-en.

A `std::accumulate` és a `fold_left` viszonya: az `accumulate` C++98 óta létezik, nincs projectionje, nincs sentinel-támogatása, és nem constexpr-követelményű a régi szignatúrában (C++20 óta constexpr). A `fold_left` ugyanazt a balról jobbra haladó szemantikát adja, de ranges-interfésszel, projectionnel és `constexpr`-rel. A `fold_right` viszont olyan műveletet tesz lehetővé, amelynek az argumentumsorrendje számít, például nem kommutatív összefűzést.

Nézzünk egy kidolgozott példát, amely bemutatja az init típusának szerepét:

```
#include <algorithm>
#include <cassert>
#include <functional>
#include <vector>

std::vector<long long> v{1'000'000'000LL, 2'000'000'000LL, 3'000'000'000LL};

// Helyes: az init long long, az akkumulátor nem csordul túl.
long long ok = std::ranges::fold_left(v, 0LL, std::plus<>{});

// Hibás szándék: az init int, az akkumulátor int marad.
// A részeredmények konvertálódnak, és a viselkedés túlcsordulásnál
// nem definiált (a long long -> int konverzió implementációfüggő,
// illetve a signed túlcsordulás UB).
long long bad = std::ranges::fold_left(v, 0, std::plus<>{});
```

A `fold_left_first` helyes használata:

```cpp
#include <optional>
#include <ranges>

std::optional<long long> first_sum(const std::vector<long long>& v) {
    return std::ranges::fold_left_first(v, std::plus<>{});
}

void use(const std::vector<long long>& v) {
    if (auto s = first_sum(v)) {
        // *s csak akkor érvényes, ha a range nem volt üres.
    }
}
```

A `fold_left_first` az első elem típusát használja, ezért a `v` elemeinek `long long`-nak kell lenniük; ha `int` vektorral hívjuk, az eredmény `optional<int>` lesz, és a nagy összeg túlcsordulhat. Ez nem a fold hibája, hanem a típusválasztásé.

42.4.2 Kereső és összehasonlító algoritmusok

A C++23 három, egymáshoz szorosan kapcsolódó csoportot adott a `<algorithm>`-hoz: a `contains`/`contains_subrange` és a `starts_with`/`ends_with` predikátumokat, a `find_last` családot, valamint a `<numeric>`-ba tartozó `ranges::iota`-t és a `shift_left`/`shift_right`-ot.

A `std::ranges::contains(r, value, proj)` `bool`-t ad, és **rövidzárral** működik: az első egyezésnél megáll. Ez a legfontosabb különbség a `find(...) != end()` mintához képest, ahol a fordító gyakran nem tudja kihasználni a korai kilépést, ha az iterátort tovább használjuk. A `contains_subrange(r, sub, pred, proj1, proj2)` azt vizsgálja, hogy a `sub` részhalmazként előfordul-e az `r`-ben; a `starts_with` és az `ends_with` pedig a range elejét, illetve végét hasonlítja össze a megadott prefixszel, illetve szufixszel. Utóbbiaknál a két range elemeinek összehasonlíthatónak kell lenniük, de nem kell azonos típusúnak lenniük — például `std::string` és `std::string_view` karakterei összehasonlíthatók.

A `find_last(r, value, proj)` az utolsó előfordulást keresi, és **`std::ranges::subrange`-et ad vissza**, amelynek `begin()`-je a találatra mutat, `end()`-je pedig a range végére. Ha nincs találat, a subrange üres (`begin() == end()`), de a subrange-nek nincs `operator bool`-ja, ezért a helyes ellenőrzés `if (!result.empty())`, nem `if (result)`. A `find_last_if` és a `find_last_if_not` predikátumos változatok; a teljes lista és a pontos követelmények a szabványban ellenőrizendők.

A `std::ranges::iota(r, value)` a `<numeric>` `std::iota` ranges-esített változata: a range-et növekvő értékekkel tölti fel. A `std::ranges::shift_left(r, n)` és a `shift_right(r, n)` a `[first, last)` tartományon belül eltolja az elemeket, és **visszatér az új last iterátorral**. A visszatérési értéket kötelező felhasználni: a `shift_left` után az `[new_last, last)` tartomány elemei nem érvényesek, és ha a hívó a régi `last`-ot használja, szemét adatokhoz jut. A `shift_left` az `n >= size` esetben üres tartományt ad vissza.

```cpp
#include <algorithm>
#include <array>
#include <cassert>
#include <string>
#include <vector>

void search_examples(const std::vector<std::string>& lines) {
    // Az első FATAL előfordulásig nem megy tovább.
```

```
    bool has_fatal = std::ranges::contains(lines, std::string{"FATAL"});

    // Prefix- és szufixvizsgálat; a string_view elemei char-ok,
    // ezért a string és a string_view összehasonlítható.
    bool any_error = std::ranges::any_of(lines, [](const std::string& s) {
        return std::ranges::starts_with(s, std::string_view{"ERROR"});
    });
    bool any_trunc = std::ranges::any_of(lines, [](const std::string& s) {
        return std::ranges::ends_with(s, std::string_view{"..."});
    });
}

void shift_example(std::vector<int>& v, std::size_t n) {
    auto new_last = std::ranges::shift_left(v.begin(), v.end(), n);
    v.erase(new_last, v.end());   // a farok érvénytelen, törölni kell
}
```

A `contains_subrange` használatakor érdemes figyelni arra, hogy a részhalmazt ne allokáljuk minden hívásnál; egy `std::array` vagy egy `std::string_view` alkalmasabb lehet, ha az elemek összehasonlíthatók.

42.4.3 Labor: naplóelemző pipeline C++20 és C++23 változatban

A labor bemenete egy `std::vector<std::string>`, amelynek elemei `"LEVEL ts bytes"` formátumúak, ahol a `LEVEL` az `ERROR`, `WARN` vagy `INFO` valamelyike. A feladat: számoljuk ki az `ERROR` szintű sorok byte-összegét, döntsük el, hogy van-e adott szint, és keressük meg az utolsó `ERROR` bejegyzést. A C++20-as változat `std::accumulate`-et, `std::find_if`-et és reverse iterátort használ; a C++23-as változat `views::filter`/`views::transform` pipeline-t, `ranges::fold_left`-et, `ranges::contains`-t és `ranges::find_last`-ot.

A labor három fájlból áll. A közös `loglab.hpp` tartalmazza a bemeneti modellt (`Entry`, `parse`) és a helyességi kaput, amelyet mindkét változat `main` függvénye lefuttat: üres bemenet, nincs `ERROR`, csupa `ERROR`, egyetlen sor és vegyes bemenet. A kapu biztosítja, hogy a két implementáció ugyanazt az eredményt adja.

```
// loglab.hpp - közös bemeneti modell és helyességi kapu mindkét változathoz
#pragma once
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <limits>
#include <optional>
#include <sstream>
#include <string>
#include <vector>

struct Entry {
    std::string level;
    std::int64_t ts;
    std::int64_t bytes;
};
```

```
// "LEVEL ts bytes" sorok; a hibás sort (nem szám, túl nagy szám, negatív
// bytes) kihagyja: a bytes mező nemnegatív, ezt a parser kényszeríti ki.
inline std::vector<Entry> parse(const std::vector<std::string>& lines) {
    std::vector<Entry> out;
    for (const std::string& line : lines) {
        std::istringstream in(line);
        Entry e;
        if (in >> e.level >> e.ts >> e.bytes && e.bytes >= 0) out.push_back(e);
    }
    return out;
}

// Ellenőrzött összeadás nemnegatív tagokra: a túlcsordulást a művelet ELŐTT
// vizsgálja, így UB nem keletkezik. Túlcsordulásnál az eredmény üres optional.
inline std::optional<std::int64_t> add_bytes(std::optional<std::int64_t> acc,
                                             std::int64_t b) {
    if (!acc || b > std::numeric_limits<std::int64_t>::max() - *acc) return
     ↪  std::nullopt;
    return *acc + b;
}

#define CHECK(condition) do {                                          \
    if (!(condition)) {                                                \
        std::fprintf(stderr, "CHECK failed at line %d\n", __LINE__); \
        std::exit(EXIT_FAILURE);                                       \
    }                                                                  \
} while (0)

// A helyességi kapu: a határeseteken mindkét változatnak ugyanezt kell adnia.
template <class ErrorBytes, class HasLevel, class LastErrorTs>
void check_cases(ErrorBytes error_bytes, HasLevel has_level,
                 LastErrorTs last_error_ts) {
    const std::vector<Entry> empty;
    CHECK(error_bytes(empty) == 0 && !has_level(empty, "ERROR"));
    CHECK(last_error_ts(empty) == -1);

    const auto none = parse({"INFO 1 10", "WARN 2 20"});
    CHECK(error_bytes(none) == 0 && has_level(none, "WARN"));
    CHECK(last_error_ts(none) == -1);

    const auto all = parse({"ERROR 1 5", "ERROR 2 7"});
    CHECK(error_bytes(all) == 12 && last_error_ts(all) == 2);

    const auto one = parse({"ERROR 9 3"});
    CHECK(error_bytes(one) == 3 && last_error_ts(one) == 9);

    const auto mixed = parse({"ERROR 1 5", "INFO 2 1", "ERROR 3 4", "WARN 4
     ↪  2"});
    CHECK(error_bytes(mixed) == 9 && last_error_ts(mixed) == 3);
    CHECK(!has_level(mixed, "DEBUG"));

    constexpr std::int64_t max = std::numeric_limits<std::int64_t>::max();
```

```
    const auto at_max = parse({"ERROR 1 9223372036854775806", "ERROR 2 1"});
    CHECK(error_bytes(at_max) == max);                          // pontosan max:
 ↪  még érvényes
    const auto over = parse({"ERROR 1 9223372036854775807", "ERROR 2 1"});
    CHECK(error_bytes(over) == std::nullopt);                   // max+1:
 ↪  dokumentált hiba
    const auto bad = parse({"ERROR 1 -5", "ERROR 2 x", "ERROR 3 7"});
    CHECK(bad.size() == 1 && error_bytes(bad) == 7);            // negatív és hibás
 ↪  mező kiesik
}
```

A C++23-as változat:

```
// loglab23.cpp - C++23
#include "loglab.hpp"
#include <algorithm>
#include <ranges>

std::optional<std::int64_t> error_bytes(const std::vector<Entry>& es) {
    auto errs = es
        | std::views::filter([](const Entry& e) { return e.level == "ERROR"; })
        | std::views::transform([](const Entry& e) { return e.bytes; });
    return std::ranges::fold_left(errs, std::optional<std::int64_t>{0},
     ↪  add_bytes);
}

bool has_level(const std::vector<Entry>& es, const std::string& lv) {
    return std::ranges::contains(es, lv, &Entry::level);
}

std::ranges::subrange<std::vector<Entry>::const_iterator>
last_error(const std::vector<Entry>& es) {
    static const std::string kErr = "ERROR";
    return std::ranges::find_last(es, kErr, &Entry::level);
}

int main() {
    check_cases(error_bytes, has_level, [](const std::vector<Entry>& es) {
        auto r = last_error(es);
        return r.empty() ? std::int64_t{-1} : r.begin()->ts;
    });
    std::puts("loglab23: ok");
}
```

A C++20-as megfelelő:

```
// loglab20.cpp - C++20
#include "loglab.hpp"
#include <algorithm>
#include <numeric>

std::optional<std::int64_t> error_bytes(const std::vector<Entry>& es) {
    return std::accumulate(es.begin(), es.end(), std::optional<std::int64_t>{0},
```

```
        [](std::optional<std::int64_t> acc, const Entry& e) {
            return e.level == "ERROR" ? add_bytes(acc, e.bytes) : acc;
        });
}

bool has_level(const std::vector<Entry>& es, const std::string& lv) {
    return std::find_if(es.begin(), es.end(),
        [&](const Entry& e) { return e.level == lv; }) != es.end();
}

auto last_error(const std::vector<Entry>& es) {
    return std::find_if(es.rbegin(), es.rend(),
        [](const Entry& e) { return e.level == "ERROR"; });
}

int main() {
    check_cases(error_bytes, has_level, [](const std::vector<Entry>& es) {
        auto it = last_error(es);
        return it == es.rend() ? std::int64_t{-1} : it->ts;
    });
    std::puts("loglab20: ok");
}
```

A build mindkét esetben ugyanazzal a flag-készlettel történik, csak a nyelvi mód tér el:

```
g++ -std=c++20 -O2 -Wall -Wextra -Wpedantic -o loglab20 loglab20.cpp
g++ -std=c++23 -O2 -Wall -Wextra -Wpedantic -o loglab23 loglab23.cpp
./loglab20
./loglab23
# lab-clang profilban:
clang++ -std=c++23 -O2 -Wall -Wextra -Wpedantic -o loglab23 loglab23.cpp
```

Az `error_bytes` mindkét változatban `std::optional<std::int64_t>`-t ad: a közös `add_bytes` a túlcsordulást az összeadás *előtt* vizsgálja, és túlcsordulásnál üres optionalt ad — ez a dokumentált hibaeredmény, így a két implementáció nem egy véletlenül egyező UB-s értékben, hanem ugyanabban a szerződésben egyezik. A negatív `bytes` mezőt a parser szűri ki, ezért elég a felső határt ellenőrizni; egy teljes összegre vonatkozó korlát csak akkor helyettesíthetné az ellenőrzést, ha a bemeneti kapu ténylegesen kikényszerítené. A kapu a `max`, a `max+1` és a negatív/hibás mezők esetét is ellenőrzi.

Buildjegyzőkönyv (`lab-gcc` profil). Mindkét változat `-Wall -Wextra -Wpedantic -fsanitize=undefined -fno-sanitize-recover=all` mellett figyelmeztetés nélkül fordult (a C++23-as `-std=c++23` módban, a GCC 13.3 libstdc++-ával), és a bővített kapu után `loglab20: ok`, illetve `loglab23: ok` kimenettel, UBSan-jelzés nélkül futott. A javítás előtti `acc + e.bytes` ugyanerre a `max+1` bemenetre UBSan alatt `signed integer overflow: 9223372036854775807 + 1 cannot be represented in type 'long int'` hibát adott. A Clang-sort nem ellenőriztük.

A feature-test probe:

```
#include <version>

#if defined(__cpp_lib_ranges_fold) && __cpp_lib_ranges_fold >= 202207L
constexpr bool have_fold = true;
```

```
#else
constexpr bool have_fold = false;
#endif

#if defined(__cpp_lib_ranges_contains) && __cpp_lib_ranges_contains >= 202207L
constexpr bool have_contains = true;
#else
constexpr bool have_contains = false;
#endif
```

A makrók jelenléte nem garantálja a teljes funkcionalitást (a könyvtár implementálhatja a makrót anélkül, hogy minden overload elkészült volna), ezért a labor elején egy **compile probe** fordítja le a tényleges hívásokat. A szabványtervezet szerinti értékek: `__cpp_lib_ranges_fold` 202207L, `__cpp_lib_ranges_contains` 202207L, `__cpp_lib_ranges_starts_ends_with` 202106L, `__cpp_lib_ranges_find_last` 202207L, `__cpp_lib_ranges_iota` 202202L, `__cpp_lib_ranges_shift` 202207L. Ezek a konkrét libstdc++/libc++ kiadásban ellenőrizendők; ahol hiányoznak, a C++20-as fallback marad.

A benchmark és a helyességi kapuk együtt futnak. A mérés metodikája: azonos bemenet, azonos optimalizálási szint, előmelegítés, legalább néhány ismétlés, és az eredmény mediánja. A helyességi kapu minden esetben az, hogy a két implementáció **ugyanazt az értéket adja** az összes határesetre: üres bemenet, nincs `ERROR`, csupa `ERROR`, egyetlen sor, a `max` és a túlcsorduló `max+1` összeg. A benchmark a hibaeredményt is azonosan kezeli: a mért függvény az `optional`-t adja vissza, és a mérés előtt a kapu már igazolta, hogy a két változat ugyanazt a hibát jelzi. A mérés eredménye nem része ennek a szövegnek; a tanuló laborjában keletkezik, és target-, fordító- és flagspecifikus.

```
#include <chrono>

template <class F>
double bench_ms(F&& f, int reps) {
    using clock = std::chrono::steady_clock;
    auto t0 = clock::now();
    for (int i = 0; i < reps; ++i) {
        volatile auto sink = f();   // holt kód kiesés ellen, nem univerzális
        ↪ barrier
        (void)sink;
    }
    auto t1 = clock::now();
    return std::chrono::duration<double, std::milli>(t1 - t0).count() / reps;
}
```

42.4.4 Gyakori hibák és javításuk

Az első és leggyakoribb hiba a `fold_left` init típusának elrontása: `0` helyett `0LL`, `0.0` helyett `0.0L` vagy fordítva. A második a `fold_left_first` eredményének kezeletlen `optional`-ja: a `*opt` üres optionalon nem definiált viselkedés, ezért előbb `if (auto v = ...)` kell. A harmadik a `find_last` eredményének `bool`-ként való használata: a `subrange` nem konvertálható `bool`-lá, helyette `!r.empty()`. A negyedik a `shift_left` visszatérési értékének eldobása, amitől a vektor vége szemét marad. Az ötödik a `contains` és a `find` összecserélése olyan helyen, ahol az iterátorra szükség van: a `contains` csak a létet adja vissza, pozíciót nem. Végül a benchmarknál a nem használt eredmény miatti holt kód kiesése, illetve a `-O0`-s mérés összehasonlíthatatlansága.

42.4.5 Összefoglaló

A C++23 ranges-algoritmusok a megszokott műveleteket adják vissza ranges-szerződéssel: a `fold_left`/`fold_right` determinisztikus sorrendű összegzés, az init típusa dönt az akkumulátor típusáról; a `fold_left_first` üres range-re `nullopt`-ot ad. A `contains`, `contains_subrange`, `starts_with` és `ends_with` rövidzáró predikátumok, a `find_last` subrange-et ad, a `shift_left`/`shift_right` pedig új last iterátort. A laborban a C++20-as és C++23-as pipeline eredményét kötelező egyeztetni, és csak ezután szabad mérni.

42.4.6 Feladatok

1. **Alapvető.** Írj egy `average_error_ratio` függvényt, amely a naplóbejegyzésekből kiszámítja az `ERROR` byte-ok és az összes byte arányát. Használj `ranges::fold_left`-et `std::int64_t{0}` és `0.0` kezdőértékkel. Magyarázd meg, miért kell a két foldnak eltérő kezdőértéket adni.
2. **Közepes.** Írj egy `last_error_before_ts` függvényt, amely a `find_last_if`-fel megkeresi az utolsó olyan bejegyzést, amelynek a timestampje kisebb, mint egy adott `limit`, és az `ERROR` szintű. Mi a visszatérési típus, és hogyan kezeled az üres eredményt?
3. **Nehéz.** Írj egy `dedup_in_place` függvényt, amely egy rendezett `std::vector<int>`-ből `shift_left`-nel távolítja el az ismétlődőket, és add meg a komplexitást, valamint a helyességi kapukat.
4. **Diagnosztikai.** A laborban a C++23-as változat lassabbnak bizonyul a C++20-asnál. Sorolj fel három lehetséges okot, és mindegyikhez adj egy ellenőrzési lépést.

42.4.7 Megoldások

1. megoldás. Az `error_bytes` és a `total_bytes` külön fold; a másodiknak `0.0` az initje, mert az arányt lebegőpontosan akarjuk kiszámítani, és a `fold_left` visszatérési típusa az init típusa.

```
double average_error_ratio(const std::vector<Entry>& es) {
    auto bytes = es | std::views::transform([](const Entry& e) { return e.bytes;
     ↪  });
    double total = std::ranges::fold_left(bytes, 0.0, std::plus<>{});
    if (total == 0.0) return 0.0;
    auto errs = es
        | std::views::filter([](const Entry& e) { return e.level == "ERROR"; })
        | std::views::transform([](const Entry& e) { return e.bytes; });
    double err = std::ranges::fold_left(errs, 0.0, std::plus<>{});
    return err / total;
}
```

Ellenőrzés: üres vektorra `0.0`, csupa `ERROR`-ra `1.0`, és az `int64_t` összegek nem csordulnak túl. Ha a második fold initje `0` lenne, az egész osztás miatt az arány `0` vagy `1` lenne.

2. megoldás. A `find_last_if` subrange-et ad; a predikátumnak a `limit`-et is látnia kell, ezért lambdába zárjuk.

```
auto last_error_before_ts(const std::vector<Entry>& es, std::int64_t limit) {
    return std::ranges::find_last_if(es,
        [limit](const Entry& e) { return e.level == "ERROR" && e.ts < limit; });
}

void use(const std::vector<Entry>& es, std::int64_t limit) {
    auto r = last_error_before_ts(es, limit);
    if (!r.empty()) {
        const Entry& e = *r.begin();
        // e itt érvényes, amíg az es nem módosul.
    }
}
```

Ellenőrzés: üres bemenetre `r.empty()`, a `limit`nél nagyobb timestampű egyetlen `ERROR` esetén szintén `r.empty()`, és a visszaadott `Entry&` addig érvényes, amíg a vektor nem módosul.

3. megoldás. Rendezett vektoron az ismétlődések összefüggő blokkot alkotnak; a `shift_left` az egyedi prefixet előrehúzza.

```
void dedup_in_place(std::vector<int>& v) {
    if (v.empty()) return;
    auto last = std::ranges::unique(v).begin();   // C++20 ranges::unique
    auto new_last = std::ranges::shift_left(v.begin(), last, last - v.begin());
    v.erase(new_last, v.end());
}
```

A `unique` az egyedi elemeket előre mozgatja, és az új logikai végét adja vissza; a `shift_left` ezt a prefixet a vektor elejére tolja, majd a farkat töröljük. A komplexitás $O(n)$ lineáris, egyetlen extra tárterület nélkül. Helyességi kapuk: üres vektor, egyelemű vektor, csupa azonos elem, minden elem különböző, és a rendezettség megőrzése.

4. megoldás. (a) A `views::filter`/`views::transform` lánc nem feltétlenül olvad össze a `fold_left`-tel, és a lambda-hívás költsége megjelenhet; ellenőrzés: assembly vagy `-fopt-info` összehasonlítása. (b) A `fold_left` `std::plus<>`-t használ, amely nem feltétlenül vektorizálódik, míg a kézi ciklus igen; ellenőrzés: `-O3 -march=native` melletti újramérés. (c) A mérés maga hibás: a `volatile` sink nem elegendő, vagy a bemenet mérete miatt a C++20-as változat a cache-ben marad; ellenőrzés: azonos bemenet, azonos flags, előmelegítés és több ismétlés, valamint az eredmények egyezésének ellenőrzése a mérés előtt.

43 Tárolók, memória és alacsony szintű eszközök C++23-ban

43.1 flat_map és flat_set: rendezett tárolás folytonos memóriában

A `std::flat_map` és a `std::flat_set` a C++23-ban jelent meg (P0429 szabványterv). A nevükben a „flat” arra utal, hogy nem csomópont-alapú tárolót használnak, hanem a kulcsokat és az értékeket lapos, rendezett konténerekben tartják. Ez a felépítés a keresést és az iterációt cache-baráttá teszi, a beszúrást és a törlést viszont lineáris költségűvé teszi. A fejezet a szerződéseket, a költségeket és a választási szempontokat tárgyalja; a konkrét implementációtámogatást nem tekintjük ellenőrzöttnek, ezért feature-test makrót és compile probe-ot adunk. A rendezett asszociatív tárolók alapjait a map, set és rendezett asszociatív tárolók részben rögzítettük, a tárolóválasztás általános szempontjait a Tárolóválasztási és invalidációs labor, a cache-viselkedés hátterét az Object layout, EBO és adatközpontú tervezés tárgyalta.

43.1.1 Mi a flat_map és a flat_set?

A `std::flat_map` asszociatív konténer-adaptor, amely rendezett kulcs-érték párokat tárol két párhuzamos konténerben. A kulcstároló és az értéktároló azonos méretű; az i-edik kulcs az i-edik értékhez tartozik. A kulcsok a `Compare` szerint rendezettek és egyediek. A `std::flat_set` ugyanez egyetlen kulcstárolóval. A `flat_multimap` és a `flat_multiset` ekvivalens kulcsokat is megenged; a beszúrás az ekvivalens kulcsok felső határára történik, hogy a sorrend stabil maradjon.

A default tároló `std::vector`, de a kulcs- és értéktároló template paraméterrel cserélhető:

```
template<class Key, class T, class Compare = std::less<Key>,
         class KeyContainer = std::vector<Key>,
         class MappedContainer = std::vector<T>>
class flat_map;

template<class Key, class Compare = std::less<Key>,
         class KeyContainer = std::vector<Key>>
class flat_set;
```

A `flat_map` nem `std::pair<const Key, T>` elemeket tárol, hanem külön kulcs- és értéktárolót. Ezért a `value_type` `std::pair<Key, T>`, a `reference` pedig egy proxy: `std::pair<const Key&, T&>`. A kulcs a tárolóban nem módosítható, az érték igen. A `flat_set` iterátora a kulcstároló const iterátora, ezért a kulcsok ott sem módosíthatók.

43.1.2 Tárolószerkezet és a sorted_unique szerződés

A `sorted_unique` tag azt jelöli, hogy a konstruktornak átadott kulcs- és értéktároló már a `Compare` szerint rendezett, és a kulcsok egyediek. A tag nem ellenőriz, hanem előfeltételt rögzít; ha a bemenet nem rendezett vagy nem egyedi, a viselkedés nem definiált (UB). A tag előnye, hogy

elkerüli az O(n log n) rendezést és az egyediség ellenőrzését. A sima konstruktor ezzel szemben rendez, és a `flat_map` esetében az ekvivalens kulcsok közül egyet megtart.

```
std::flat_map<int, std::string> m(
    std::sorted_unique,
    std::vector<int>{1, 2, 3},
    std::vector<std::string>{"one", "two", "three"});
```

A `sorted_unique` akkor hasznos, ha a kulcsokat egy korábbi lépésben már rendeztük, vagy ha természetes sorrendben érkeznek. A `flat_multimap` és `flat_multiset` esetében a tag neve `sorted_equivalent`, mert ott az ekvivalens kulcsok megengedettek. A mögöttes konténereknek véletlen hozzáférésűnek kell lenniük, és támogatniuk kell a beszúrást és a törlést; a default `std::vector` folytonos, de például egy `std::deque` választása esetén a folytonosság és a cache-előny részben elvész.

43.1.3 Műveleti költségek és cache-viselkedés

A flat tárolók költségei a mögöttes konténerek műveleteiből származnak. A keresés a kulcstárolón bináris keresés, ezért O(log n) összehasonlítás; a beszúrás és a törlés a vektoros eltolás miatt O(n). A csomópont-alapú és hash tárolókhoz viszonyítva:

Művelet	`flat_map`	`std::map`	`std::unordered_map`
Keresés	O(log n), folytonos kulcstömbön	O(log n), csomópontkövetés	átlag O(1), worst O(n)
Beszúrás	O(n) eltolás + esetleges újraallokáció	O(log n) + allokáció	átlag O(1) + allokáció/rehash
Törlés	O(n) eltolás	O(log n)	átlag O(1)
Iteráció	O(n), folytonos, gyors	O(n), pointer chasing	O(n), bucket-bejárás
Memória	kevés allokáció, kis overhead	csomópontonként allokáció	bucket + csomópont
Iterátor-invalidáció	módosításkor minden iterátor	csak a törölt elem iterátora	rehash esetén minden iterátor; `erase`-nél a törölté
Pointer/referencia-invalidáció	módosításkor minden (az elemek eltolódhatnak)	csak a törölt elemre	csak a törölt elemre (a rehash a pointereket és referenciákat **nem** érvényteleníti)

A keresés a kulcsok tömbjén fut, nem a párokon, ezért a kulcsok folytonos tárolása jobb cache-kihasználást ad, mint a `std::map` csomópontjainak követése. A bináris keresés első lépései ugrálnak, de a tömb egyes részei cache-vonalban vannak. A `flat_map` előnye különösen akkor jelentős, ha a kulcsok kicsik, az összehasonlítás olcsó, és a keresés gyakori. Nagy kulcsok (például hosszú stringek) esetén a bináris keresés több összehasonlítást végez, mint egy hash-alapú keresés, ezért a `unordered_map` versenyképes lehet.

A beszúrás költsége három részből áll, és ezeket külön kell számolni: (1) **keresés**: a beszúrási hely és az egyediség megállapítása $O(\log n)$ összehasonlítás; (2) **eltolás**: a mögöttes vektorokban a beszúrási pont mögötti elemek mozgatása, a végére beszúrásnál nulla, a közepére $O(n)$; (3) **allokáció**: kapacitás elfogyásakor újrafoglalás, amortizáltan konstans. A végére beszúrás tehát $O(\log n)$ összehasonlítás + amortizált $O(1)$ mozgatás, nem „$O(1)$”. A keresés csak olyan overloadnál marad el, amelynek a szerződése ezt megengedi: a hint-es `insert(pos, x)` helyes hinttel, illetve

a `sorted_unique` taggal hívott tartományos `insert`, amelynél az előfeltétel a rendezettség és egyediség. Rendezett sorrendben érkező beszúrások a flat tárolók kedvező esete. Az iteráció folytonos, ezért a teljes bejárás gyors; a `std::map` iterációja pointer chasing miatt lassabb. A memóriahasználat szempontjából a flat tároló kevesebb allokációt végez, és kisebb az egy eleme jutó overhead, mert nincsenek csomópontpointerek és színinformációk.

43.1.4 Invalidáció és kivételbiztonság

A `flat_map` strukturális módosításai — `insert`, `emplace`, `erase`, `clear`, valamint az `operator[]` és a `try_emplace`, ha a kulcs **hiányzik**, és ezért beszúrnak — érvénytelenítik az iterátorokat és referenciákat, nem csak a beszúrt vagy törölt elemre vonatkozót. Az ok a mögöttes vektoros tárolás: a beszúrás eltolhatja az elemeket, és újraallokálhat, ami az összes címre hat. Az `operator[]` **meglévő** kulcsnál viszont nem módosítja a szerkezetet: csak megkeresi az elemet, és az értékre ad referenciát; ez nem invalidál semmit. A `flat_map`-nek nincs `reserve` tagfüggvénye. Az előkészítés a mögöttes tárolókon történik: vagy a kulcs- és értékvektort külön építjük fel (`reserve`-vel), és a `sorted_unique` konstruktornak adjuk át (lásd a lenti receptet), vagy egy meglévő `flat_map`-ből az `extract()`-tal kivesszük a tárolókat, előkészítjük őket, és a `replace(keys, values)`-szel visszaadjuk. Az `extract()` és a `replace()` minden korábbi iterátort és referenciát érvénytelenít; a visszaadott tárolóknak továbbra is rendezettnek és egyedinek kell lenniük. A `swap` nem invalidál, de az iterátorok a másik konténer elemeire mutatnak tovább.

A `flat_map::reference` proxy jellege miatt a `for (auto& [k, v] : fm)` alak nem fordul le, mert az iterátor dereferálása egy prvalue `std::pair<const Key&, T&>`-t ad. Helyes: `for (auto&& [k, v] : fm)` vagy `for (auto [k, v] : fm)`. A `k` ekkor `const Key&`, a `v` `T&`, tehát az érték módosítható, a kulcs nem.

A `flat_map` módosító műveleteire a szabvány **nem** ad általános erős garanciát. A szerződés (C++23 [flat.map.overview]): ha egy tagfüggvény kivétellel lép ki, az **invariánsok** (rendezettség, egyediség, a két tároló azonos hossza) helyreállnak — ami akár a tároló kiürítésével is járhat. Az invariánsok megőrzése tehát garantált, a „nincs hatás” nem; ha a kulcs vagy az érték mozgatása dobhat, a beszúrás után a tartalom változhat, akár el is veszhet. Erős garancia ott várható, ahol a mögöttes tároló műveletei és a típusok mozgatása nem dobnak, de erre általános ígéretet nem építünk. **Buildjegyzőkönyv:** a Clang 22 / libc++ `flat_map`-je egy dobó mozgató értékadású értéktípusnál a 11. elem beszúrásakor dobott kivétel után 10 elemmel maradt — ez egy implementáció viselkedése, nem szabványos ígéret; a `m.reserve(10)` ugyanott `no member named 'reserve'` fordítási hibát adott, az `extract`/`replace` út lefordult és lefutott.

43.1.5 Kidolgozott példa és build

Az alábbi program bemutatja a `sorted_unique` tagot, a keresést, a beszúrást és a proxy referenciát. A példa önállóan fordítható, ha a fordító és a standard library támogatja a `flat_map`-et.

```
#include <flat_map>
#include <iostream>
#include <string>
#include <utility>
#include <vector>

int main() {
    using map_t = std::flat_map<int, std::string>;

    // Rendezett, egyedi kulcsok: a sorted_unique tag nem ellenőriz.
```

```
    std::vector<int> keys{1, 3, 5};
    std::vector<std::string> vals{"one", "three", "five"};
    map_t m(std::sorted_unique, std::move(keys), std::move(vals));

    // Keresés a kulcstömbön.
    auto it = m.find(3);
    if (it != m.end()) {
        std::cout << it->first << " -> " << it->second << '\n';
    }

    // Beszúrás: minden korábbi iterátor és referencia invalidálódik.
    m.insert({2, "two"});
    // it = m.find(3); // ha kell, újra kell keresni

    // Iteráció: a reference proxy, ezért auto&& kell.
    for (auto&& [k, v] : m) {
        std::cout << k << '=' << v << '\n';
    }
}
```

A buildparancs GCC-vel és Clanggel:

```
g++ -std=c++23 -O2 -Wall -Wextra -pedantic flat_map_demo.cpp -o flat_map_demo
clang++ -std=c++23 -O2 -Wall -Wextra -pedantic flat_map_demo.cpp -o
 ↪  flat_map_demo
```

A tényleges fordíthatóságot a laborprofilban rögzített fordító- és könyvtárverzió dönti el; a következő szakaszban adott compile probe ezt hivatott ellenőrizni.

43.1.6 Benchmark: olvasás- és írásdomináns terhelés

A flat tárolók és a csomópont-alapú, illetve hash tárolók összehasonlítását méréssel kell eldönteni. A fejezet nem állítja, hogy a `flat_map` minden esetben gyorsabb; a mérés a tanuló laborjához tartozik. A mérési terv:

- Hasonlítsd össze a `flat_map`, `std::map` és `std::unordered_map` tárolókat azonos kulcshalmazon, azonos compiler flaggel és azonos standard library-vel.
- Olvasás-domináns terhelés: például 90% `find`, 10% `insert`. Írás-domináns terhelés: 90% `insert`/`erase`, 10% `find`.
- Használj legalább $N = 10^5$ és $N = 10^6$ méretet, véletlen kulcsokat, és ismételd meg a mérést legalább ötször; a mediánt közöld.
- Külön eset a rendezett sorrendben érkező beszúrás, mert ez a flat tároló kedvező esete.
- Mérd a wall time-ot `std::chrono::steady_clock`-kal, a cache-misseket `perf stat -e cache-misses,cache-references` segítségével, és az allokációk számát például számláló allocatorral vagy `valgrind` eszközzel.
- Melegíts, és a fordító optimalizálását ne engedd kiejteni a kereséseket: összegezd a találatokat, vagy használj `DoNotOptimize`-szerű megoldást.
- A mért eredményt, a várt eredményt és az illusztrációt külön kategóriaként kezeld; a fejezet nem közöl mért számokat.

43.1.7 Támogatás ellenőrzése és C++20 fallback

A `flat_map` és `flat_set` elérhetőségét feature-test makróval és compile probe-bal kell ellenőrizni. A két képességnek külön makrója van: a várt `__cpp_lib_flat_map` és `__cpp_lib_flat_set`; a szabványterv alapján mindkettő várt értéke `202207L` (a C++23 [version.syn] szerint), de a tényleges értéket a `<version>` headerből kell kiolvasni. A következő probe nem támogatott környezetben `#error`-ral áll meg:

```
#include <version>
#if !defined(__cpp_lib_flat_map) || __cpp_lib_flat_map < 202207L
#error "A flat_map nem tamogatott ezen a fordito/standard library kombinacion."
#endif
#if !defined(__cpp_lib_flat_set) || __cpp_lib_flat_set < 202207L
#error "A flat_set nem tamogatott ezen a fordito/standard library kombinacion."
#endif
#include <flat_map>
#include <flat_set>
```

C++20-ban a `std::flat_map` nem elérhető. A leggyakoribb fallback egy rendezett `std::vector<std::pair<Key, T>>` és `std::lower_bound`; ez a keresést és a beszúrást utánozza, de nem nyújtja a teljes API-t:

```
#include <algorithm>
#include <utility>
#include <vector>

template<class Key, class T, class Compare = std::less<Key>>
class flat_map_cpp20 {
    std::vector<std::pair<Key, T>> data_;
    Compare comp_{};
public:
    using value_type = std::pair<Key, T>;
    // Csak const iterátor lép ki a felületen: a kulcs (first) a rendezettség
     ↪ miatt nem írható.
    using const_iterator = typename std::vector<value_type>::const_iterator;
    using iterator = const_iterator;

    const_iterator find(const Key& k) const {
        auto it = std::lower_bound(data_.begin(), data_.end(), k,
            [this](const value_type& p, const Key& key) {
                return comp_(p.first, key);
            });
        if (it != data_.end() && !comp_(k, it->first)) return it;
        return data_.end();
    }

    // Az érték módosítható a kulcs érintése nélkül: mutató a T-re (nem a
     ↪ pair-re).
    T* find_value(const Key& k) {
        auto it = find(k);
        if (it == data_.end()) return nullptr;
        return &data_[static_cast<std::size_t>(it - data_.begin())].second;
    }
```

```
    std::pair<iterator, bool> insert(value_type v) {
        auto it = std::lower_bound(data_.begin(), data_.end(), v.first,
            [this](const value_type& p, const Key& key) {
                return comp_(p.first, key);
            });
        if (it != data_.end() && !comp_(v.first, it->first)) {
            return {it, false};
        }
        return {data_.insert(it, std::move(v)), true};
    }
};
```

A fallback beszúrása szintén O(n), és minden iterátort invalidál. A publikus felület szándékosan csak `const_iterator`-t ad ki (az `insert` is), az érték módosítására a `find_value` szolgál: így a hívó az eredményen keresztül nem írhatja át a kulcsot és nem sértheti meg a rendezettséget — ez a szerződéskülönbség a szabványos `flat_map` proxy-referenciájához képest. A `std::lower_bound` feltétele, hogy a tartomány a comparator szerint rendezett legyen; ezt a beszúrásnak fenn kell tartania.

43.1.8 Gyakori hibák

- A `flat_map` iterátorát vagy referenciáját beszúrás vagy törlés után használni. A művelet minden iterátort és referenciát invalidál.
- A `flat_map::reference`-t valódi `std::pair<const Key, T>&`-nek hinni. Az egy proxy `std::pair<const Key&, T&>`, ezért `auto&` helyett `auto&&` vagy `auto` kell.
- A `sorted_unique` tagot ellenőrzés nélkül használni nem rendezett vagy nem egyedi bemenetre. Ez UB.
- A `flat_map`-et írás-domináns terhelésre választani, ahol a beszúrás és a törlés O(n) költsége dominál.
- Keresésre a `flat_map::operator[]`-t használni, mert az beszúr, ha a kulcs nincs; helyette `find` vagy `at` kell.
- A flat tároló kulcsát módosítani próbálni. A kulcsok const hozzáféréssel érhetők el; a módosítás a rendezettséget sértené.

43.1.9 Feladatok

1. **Tárolóválasztás.** Egy olvasás-domináns szolgáltatásban a konfigurációs kulcsokat gyakran keresik, ritkán módosítják. Egy írás-domináns eseményaggregátorban sok beszúrás és törlés van. Egy nagy, egyedi azonosítós cache-ben a kulcsok egészek, a keresés gyakori. Válassz tárolót mindhárom esetre, és indokold a komplexitás, az invalidáció és a memória alapján.

2. **Kódolás.** Írj egy `build_flat_map` függvényt, amely egy `std::vector<std::pair<std::string, int>>` bemenetből `std::flat_map<std::string, int>`-et épít. Ha a bemenet rendezett és egyedi, használd a `sorted_unique` tagot; különben a sima konstruktort. Dokumentáld az előfeltételeket, és ellenőrizd a rendezettséget és az egyediséget.

3. **Diagnosztika.** Az alábbi kód UB-t tartalmaz. Mutasd meg, miért, és javítsd.

```
std::flat_map<int, std::string> m{{1, "one"}, {2, "two"}};
auto it = m.find(1);
```

```
m.insert({3, "three"});
std::cout << it->second;
```

4. **Benchmarkterv.** Tervezz benchmarkot a `flat_map`, `std::map` és `std::unordered_map` összehasonlítására. Add meg a terhelési arányokat, a méreteket, az ismétlések számát, a mért metrikákat és a lehetséges torzításokat. Ne állítsd, hogy a `flat_map` gyorsabb.

43.1.10 Megoldások

43.1.10.1 1. feladat megoldása

Az olvasás-domináns konfigurációs szolgáltatásban a `flat_map` jó választás: a keresés O(log n) folytonos kulcstömbön, az iteráció gyors, a memóriaoverhead kicsi. Ha a kulcsok nagyok és a hash számítás olcsó, a `unordered_map` is versenyképes. Az írás-domináns eseményaggregátorban a `unordered_map` vagy a `std::map` a jobb, mert a `flat_map` beszúrása és törlése O(n); ha a rendezettség nem kell, a hash tároló, ha kell, a csomópont-alapú `map` választandó. A nagy, egyedi azonosítós cache-ben a `unordered_map` a kézenfekvő, mert a hash átlag O(1); ha a cache kicsi és a keresés a domináns, a `flat_map` is szóba jön.

43.1.10.2 2. feladat megoldása

```
#include <algorithm>
#include <flat_map>
#include <string>
#include <utility>
#include <vector>

std::flat_map<std::string, int>
build_flat_map(std::vector<std::pair<std::string, int>> input) {
    const bool sorted_unique = std::is_sorted(
        input.begin(), input.end(),
        [](const auto& a, const auto& b) { return a.first < b.first; })
        && std::adjacent_find(
            input.begin(), input.end(),
            [](const auto& a, const auto& b) { return a.first == b.first; })
           == input.end();

    if (sorted_unique) {
        std::vector<std::string> keys;
        std::vector<int> vals;
        keys.reserve(input.size());
        vals.reserve(input.size());
        for (auto& p : input) {
            keys.push_back(std::move(p.first));
            vals.push_back(p.second);
        }
        return std::flat_map<std::string, int>(
            std::sorted_unique, std::move(keys), std::move(vals));
    }
    return std::flat_map<std::string, int>(input.begin(), input.end());
}
```

A `sorted_unique` ág előfeltétele, hogy a kulcsok a `std::less<std::string>` szerint rendezettek és egyediek legyenek; ezt az `std::is_sorted` és az `std::adjacent_find` ellenőrzi. A másik ág a range konstruktort használja, amely rendez és egyedivé tesz. Az ellenőrzés feltétele, hogy a bemenet a kulcs szerint rendezett legyen; ha nem az, a rendezett ág nem választható.

43.1.10.3 3. feladat megoldása

A hiba oka, hogy a `flat_map::insert` a mögöttes vektorba szúr, ami eltolhatja az elemeket és újraallokálhat, ezért minden korábbi iterátor és referencia invalidálódik. Az `it` a régi tárolóra mutat, a használata UB. A javítás: az `insert` után újra kell keresni.

```
std::flat_map<int, std::string> m{{1, "one"}, {2, "two"}};
m.insert({3, "three"});
auto it = m.find(1);
if (it != m.end()) {
    std::cout << it->second << '\n';
}
```

Az `insert` visszatérési értéke egy `std::pair<iterator, bool>`, amely a beszúrt elemre mutató iterátort adja; ez használható, de a régi iterátor nem.

43.1.10.4 4. feladat megoldása

A benchmark három tárolót hasonlít össze: `flat_map`, `std::map`, `std::unordered_map`. Az olvasás-domináns terhelés 90% `find`, 10% `insert`; az írás-domináns 90% `insert`/`erase`, 10% `find`. A méretek $N = 10^5$ és $N = 10^6$, a kulcsok véletlen eloszlásúak, a beszúrások véletlen és rendezett sorrendben is futnak. Az ismétlések száma legalább öt, a közölt érték a medián. Mért metrikák: wall time `std::chrono::steady_clock`-kal, cache-missek `perf stat`-tal, allokációk száma számláló allocatorral. A torzításforrások: a keresési kulcsok eloszlása, a beszúrások sorrendje, a compiler optimalizálása, a standard library verziója, a cache mérete és a rendszer terheltsége. A `flat_map` előnye az olvasás-domináns, kis módosítású esetben várható; ezt a mérésnek kell igazolnia, nem a fejezetnek.

43.1.11 Összefoglaló

A `flat_map` és a `flat_set` a C++23 rendezett, folytonos tárolású adaptorai. A keresés és az iteráció cache-baráts, a beszúrás és a törlés O(n). A választás a terheléstől függ: olvasás-domináns, kis módosítású adatokra kiváló; írás-domináns terhelésre a csomópont-alapú vagy hash tárolók jobbak. A `sorted_unique` tag előfeltétel, nem ellenőrzés. A beszúrás és a törlés minden iterátort és referenciát invalidál. A támogatást feature-test makróval és compile probe-bal kell ellenőrizni; C++20-ban rendezett vector és `std::lower_bound` a fallback.

43.2 mdspan mélyebben: layoutok, accessorok és C-interop

A Multidimenziós view és numeric utility térkép (31.5) részben az `mdspan` alapvető használatát, a `std::span`-tól való eltérését és a multidimenzionós nézet gondolatát rögzítettük. A static operator(), többdimenziós indexelés és lambdaváltozások (40.3) rész a C++23-as többdimenziós `operator[]` nyelvi hátterét adta, a Többdimenziós tömbök, VLA és layout (7.2) rész pedig a C tömbök sor-major layoutját. Most ezekre építve a layoutok, az accessorok és a C-interop

szerződéseit vesszük sorra. A cél nem az `mdspan` újratanítása, hanem az, hogy saját layout- és accessor-policyt tudjunk indokoltan készíteni, és felismerjük, hol ér véget a memóriabiztonság, amit egy accessor nyújthat.

A laborprofil továbbra is ISO C++23 (`-std=c++23`), a C++20-as rész profiljának kiterjesztéseként. A C++23-as `mdspan` és a C++20-as `atomic_ref` elérhetősége fordító- és standard library-függő; ezt a fejezet nem tekinti ellenőrzöttnek. A példák ezért feature-test makrót és compile probe-ot tartalmaznak, a konkrét GCC/Clang és libstdc++/libc++ verziót, valamint a `__cpp_lib_mdspan` tényleges értékét a labornak kell rögzítenie.

43.2.1 Extents: statikus és dinamikus kiterjedés

Az `extents` a multidimenzionális index tér dimenziónkénti kiterjedését írja le. A statikus kiterjedés a típus része, ezért compile time ismert; a dinamikus kiterjedés futásidejű érték. A C++23-as `std::extents<std::size_t, 3, std::dynamic_extent, 4>` például háromdimenziós extents, amelynek első és harmadik kiterjedése statikus, a középső dinamikus. A `std::dextents<std::size_t, 2>` mindkét dimenzióban dinamikus.

A statikus extents előnye, hogy a méret a típusban van, így a mapping számítása optimalizálható, és a hibás dimenziószám már fordításkor kiderül. Hátránya, hogy a méret a típus részévé válik, ezért eltérő méretekhez eltérő típus tartozik. A dinamikus extents rugalmasabb, de a kiterjedés értékét futásidőben kell átadni, és a mapping számítása is futásidejű adatokra támaszkodik.

Az `mdspan` első három template paramétere az elem típusa, az extents és a layout policy; a negyedik az accessor policy. A `std::mdspan<int, std::dextents<std::size_t, 2>>` tehát kétdimenziós, dinamikus kiterjedésű, alapértelmezett `layout_right` és `default_accessor<int>` típus. A `rank()`, `extent(r)`, `static_extent(r)` és `stride(r)` tagfüggvények adják a metaadatot. Az `mdspan` nem tulajdonol: a mögöttes puffer élettartama a hívó felelőssége.

43.2.2 Layoutok és a mapping szerződése

A layout policy a logikai indexeket lineáris offsetre képezi. Kétdimenziós esetben a három alapvető mapping:

$$M_{\text{right}}(i, j) = i \cdot N_1 + j$$

$$M_{\text{left}}(i, j) = i + j \cdot N_0$$

$$M_{\text{stride}}(i, j) = i \cdot s_0 + j \cdot s_1$$

A `layout_right` a jobb oldali indexet, a `layout_left` a bal oldali indexet tekinti leggyorsabban változónak. A C tömbök sor-major elrendezése a `layout_right`-nak felel meg; a Fortran-oszlop-major a `layout_left`-nak. A `layout_stride` explicit stride-okat fogad, ezért paddingolt sorú mátrixhoz, szelethez vagy külső pufferhez illik. A stride-ok azonban nem tetszőlegesek: a konstruktor előfeltétele, hogy minden stride pozitív legyen, és hogy a leképezés egyedi maradjon, vagyis két különböző indexpár ne essen ugyanarra az offsetre (C++23 [mdspan.layout.stride.cons]). A nulla stride (*broadcast*: egy sor ismétlése minden sorindexre) és az egymást átfedő leképezés tehát előfeltétel-sértés, nem támogatott bemenet; ilyen nézethez saját layout policy kell, amelynek `is_unique()`-ja hamis.

layout	2D mapping	leggyorsabb index	C tömb	`is_always_unique()`
`layout_right`	$i \cdot N_1 + j$	jobb oldali (j)	igen	igaz
`layout_left`	$i + j \cdot N_0$	bal oldali (i)	nem	igaz
`layout_stride`	$i \cdot s_0 + j \cdot s_1$	a stride-ok döntik el	padding esetén	igaz

A mapping szerződése: az `operator()(indices...) noexcept`, a `required_span_size() noexcept`, és a mapping nem ellenőriz bounds-ot. A `layout_left` és `layout_right` mindig egyedi és kimerítő. A `layout_stride` is mindig egyedi (`is_always_unique()` igaz), mert a konstruktor előfeltétele kizárja az átfedést; az `is_always_exhaustive()` viszont hamis, és padding esetén az `is_exhaustive()` is hamis. Az egyediséget (nincs két index ugyanazon az offseten) nem szabad összekeverni a kimerítőséggel (minden offset a tartományban használt). A `required_span_size()` három esetre bomlik (C++23 [mdspan.layout.stride.obs]): **rank 0** esetén 1 (egyetlen elem); ha **valamelyik extent 0** (üres index-tér), akkor 0 — ilyenkor nincs legnagyobb érvényes offset; egyébként (nem üres, pozitív rank) a legnagyobb offset plusz egy, azaz $1 + \sum_k (e_k - 1)\, s_k$. **Ellenőrzés** (Kokkos `mdspan` + GCC 13.3): rank 0 → 1, `extents<3, 0>` stride-dal → 0, a 3×4-es, 5-ös sorstride-os eset → 14. A nem üres, pozitív rankú példa, egy 3×4-es, 5-ös sorstride-os mátrix:

$$\max_{i,j} M(i,j) = (3-1)\cdot 5 + (4-1)\cdot 1 = 13, \qquad \text{required_span_size} = 14.$$

A puffernek tehát legalább 14 `int`-et kell tartalmaznia, noha csak 12 elem logikailag látható. A két extra elem a padding.

43.2.3 Saját accessor policy

Az accessor policy a `data_handle_type` és a lineáris offset közötti hozzáférést írja le. Követelményei: `element_type`, `data_handle_type`, `reference`, `offset_policy`, valamint az `access(data_handle_type, std::size_t)` és `offset(data_handle_type, std::size_t)` tagfüggvények. Az `access` a lineáris offsetet kapja, nem a logikai indexeket; a mapping már elvégezte az indexszámítást. Ez a kulcs ahhoz, hogy megértsük, mit tud és mit nem tud egy accessor.

43.2.3.1 checked_accessor: lineáris bounds check

A `checked_accessor` a `data_handle_type`-ba csomagolja a pointert és a puffer méretét. Az `access` ellenőrzi, hogy a lineáris offset a méreten belül van; az `offset` a subview-hoz lépteti a handle-t, és csökkenti a méretet.

```
#include <array>
#include <atomic>
#include <cassert>
#include <cstddef>
#include <iostream>
#include <mdspan>

template<class T>
struct checked_handle {
    T* ptr;
```

```
    std::size_t size;
};

template<class T>
struct checked_accessor {
    using element_type = T;
    using data_handle_type = checked_handle<T>;
    using reference = T&;
    using offset_policy = checked_accessor;

    constexpr reference access(data_handle_type h, std::size_t i) const noexcept
     ↪ {
        assert(i < h.size);
        return h.ptr[i];
    }
    constexpr data_handle_type offset(data_handle_type h, std::size_t i) const
     ↪ noexcept {
        assert(i <= h.size);
        return {h.ptr + i, h.size - i};
    }
};

extern "C" void c_fill(int* p, std::size_t rows, std::size_t cols, std::size_t
 ↪ stride) {
    for (std::size_t i = 0; i < rows; ++i)
        for (std::size_t j = 0; j < cols; ++j)
            p[i * stride + j] = static_cast<int>(i * cols + j);
}

int main() {
    int a[12] = {};                       // valóban lapos tároló: 3 x 4 int
    using E = std::dextents<std::size_t, 2>;
    std::mdspan<int, E, std::layout_right, checked_accessor<int>> m(
        checked_handle<int>{a, 12}, 3, 4);
    m[1, 2] = 42;
    std::cout << a[1 * 4 + 2] << '\n'; // 42

    c_fill(m.data_handle().ptr, m.extent(0), m.extent(1), m.stride(0));

    int buf[15] = {};
    using E2 = std::extents<std::size_t, 3, 4>;
    std::array<std::size_t, 2> strides{5, 1};
    std::layout_stride::mapping<E2> map(E2{}, strides);
    std::mdspan<int, E2, std::layout_stride> ms(buf, map);
    ms[2, 3] = 7;
    std::cout << buf[2 * 5 + 3] << '\n'; // 7
    std::cout << ms.size() << ' ' << ms.mapping().required_span_size() << '\n';
     ↪ // 12 14
}
```

A `required_span_size()` a **mapping** tagfüggvénye, nem az `mdspan`-é: a tárolóigényt az `ms.mapping().required_span_size()` adja, a logikai elemszámot az `ms.size()`.

Buildjegyzőkönyv. A `lab-gcc` profil GCC 13.3-as libstdc++-ában nincs `<mdspan>`, ezért a programot ott nem lehet lefordítani. Ellenőrzésképpen a C++23 `std::mdspan` Kokkos-féle referenciaimplementációjával fordítottuk le (egyfejléces változat, `86eefa3d6653` commit, 2026-10-16), GCC 13.3-mal, `-std=c++23 -Wall -Wextra -Wpedantic` mellett. A kimenet `42`, `7`, `12 14`. Az eredeti `ms.required_span_size()` hívás ugyanitt `has no member named 'required_span_size'` hibát adott. Egy szabványos `<mdspan>`-t nyújtó könyvtárral (`lab-next` profil) a futtatás a saját laborban ismétlendő.

A példa a `checked_accessor` `assert`-jére épül; `NDEBUG` esetén az `assert` kikapcsol, ezért releaseben a valódi bounds checkhez `std::abort` vagy kivétel kell. A `checked_accessor` csak a lineáris offsetet ellenőrzi: a `m[3,0]` offsetje 12, ezt az `assert(12 < 12)` elkapja, de a `m[0,4]` offsetje 4, ami a 12 elemes pufferben van, ezért azt nem. A logikai indexek ellenőrzése az extents ismeretét igényli, amit az accessor nem kap meg.

43.2.3.2 atomic_accessor: atomikus hozzáférés

Az `atomic_accessor` `reference` típusa `std::atomic_ref<T>`, így az `mdspan` elemei atomikusan érhetők el. A `std::atomic_ref<T>` követelménye, hogy `T` trivially copyable legyen, a hivatkozott objektum igazítása legalább `required_alignment` legyen, és amíg `atomic_ref` él rá, minden hozzáférés atomic_ref-en át történjen. Három műveletet kell szétválasztani: (1) a **másoló konstrukció** megengedett — a másolat ugyanarra az objektumra hivatkozik; (2) a **wrapper másoló értékadása** törölt (`ref1 = ref2` nem fordul); (3) a **`T` értékadás** (`T operator=(T) const`) megengedett, és egy `seq_cst` tárolás. Ezért az `m[i,j] = 42` **működik**: a visszaadott `atomic_ref` prvalue-n hívott értékadás `seq_cst` store. Ha más sorrendezés kell, a `.store(42, std::memory_order_relaxed)` alak választja meg.

```
template<class T>
struct atomic_accessor {
    using element_type = T;
    using data_handle_type = T*;
    using reference = std::atomic_ref<T>;
    using offset_policy = atomic_accessor;

    constexpr reference access(T* p, std::size_t i) const noexcept {
        return std::atomic_ref<T>{p[i]};
    }
    constexpr T* offset(T* p, std::size_t i) const noexcept {
        return p + i;
    }
};
```

43.2.4 C-interop: C tömbök és külső pufferek

A C kétdimenziós tömb (`int a[3][4]`) folytonos, sor-major; a `layout_right` offsetképlete illeszkedik hozzá. Az objektumszerkezet azonban más: az `a` három, egyenként 4 elemű `int[4]` tömbből áll, és az `&a[0][0]`-ból képzett `int*`-gal a 4. elemen túl (a következő sorba) lépni a nyelv szerint az `a[0]` tömbön kívüli pointeraritmetika. Ezért a hordozható példa valóban lapos tárolót használ (`int a[12]` vagy `std::vector<int>`), ahol minden offset egyetlen közös skalártömbön belül marad. Valódi kétdimenziós objektumhoz sorpointeres accessor való, amely a lineáris offsetet sorra és oszlopra bontja, és cast nélkül, soron belül indexel:

```cpp
// mdspan_access.cpp - C++23: lapos tároló, sorpointeres accessor, atomic_ref
#include <array>
#include <atomic>
#include <cassert>
#include <cstddef>
#include <mdspan>

// (1) Sorpointeres accessor egy valódi int[ROWS][COLS] objektumhoz: a lineáris
//     offsetet sorra és oszlopra bontja, így minden hozzáférés a saját során
//     belül marad, nincs sorhatárt átlépő int* aritmetika és nincs cast.
template <std::size_t COLS>
struct row_accessor {
    using element_type = int;
    using data_handle_type = int (*)[COLS];
    using reference = int&;
    using offset_policy = row_accessor;
    constexpr reference access(data_handle_type h, std::size_t i) const noexcept
     ↪ {
        return h[i / COLS][i % COLS];
    }
    // Subview csak egész sorokkal léptethető ehhez a handle-hez.
    constexpr data_handle_type offset(data_handle_type h, std::size_t i) const
     ↪ noexcept {
        return h + i / COLS;
    }
};

// (2) Atomikus accessor: a reference std::atomic_ref<int>.
struct atomic_accessor {
    using element_type = int;
    using data_handle_type = int*;
    using reference = std::atomic_ref<int>;
    using offset_policy = atomic_accessor;
    constexpr reference access(int* p, std::size_t i) const noexcept { return
     ↪ reference{p[i]}; }
    constexpr int* offset(int* p, std::size_t i) const noexcept { return p + i;
     ↪ }
};

int main() {
    using E = std::extents<std::size_t, 3, 4>;

    int flat[12] = {};                                  // valóban lapos tároló
    std::mdspan<int, E> mf(flat);                       // default_accessor
    mf[1, 2] = 42;
    assert(flat[1 * 4 + 2] == 42);

    int a[3][4] = {};                                   // valódi 2D objektum
    std::mdspan<int, E, std::layout_right, row_accessor<4>> mr(a);
    mr[2, 3] = 7;
    assert(a[2][3] == 7);
```

```
    alignas(std::atomic_ref<int>::required_alignment) int buf[12] = {};
    std::mdspan<int, E, std::layout_right, atomic_accessor> am(buf);
    am[0, 0] = 42;                                          // T értékadás: seq_cst
 ↪  store
    assert((am[0, 0].load() == 42));
    am[0, 1].store(5, std::memory_order_relaxed);           // explicit sorrendezés
    std::atomic_ref<int> copy = am[0, 1];                   // másoló konstrukció:
     ↪  megengedett
    assert(copy.load() == 5);
    // copy = am[0, 0];  // hiba: a wrapper copy assignmentje törölt
}
```

Buildjegyzőkönyv. A Kokkos-féle `mdspan` referencia-implementációval (lásd fent), GCC 13.3-mal, `-std=c++23 -Wall -Wextra -Wpedantic -fsanitize=address,undefined` mellett figyelmeztetés nélkül fordult, és minden `assert` teljesült: a lapos és a sorpointeres nézet a várt elemet írta, az `am[0, 0] = 42` értékadás után a `load()` 42-t adott, a másoló konstrukcióval kapott `atomic_ref` a relaxed módon tárolt 5-öt olvasta. A kikommentezett `copy = am[0, 0];` sor `use of deleted function 'std::atomic_ref<int>::operator=(const std::atomic_ref<int>&)'` hibát ad.

Az `mdspan` nézet nem tulajdonol, és addig érvényes, amíg a tároló él. Külső puffer (`malloc`, `std::vector::data()`, `mmap`) esetén ugyanez a szabály: a puffer élettartama hosszabb kell legyen, mint a nézeté. A `std::vector` újraallokálása invalidálja az `mdspan`-t; a `checked_handle` mérete nem véd a felszabadítás ellen.

A C API-nak átadott pointer típusa accessor-függő. `default_accessor` esetén `m.data_handle()` `T*`; `checked_accessor` esetén `checked_handle<T>`, ezért a C API-nak `m.data_handle().ptr`-t kell adni. A stride-ot és az extents-et külön kell átadni, ahogy a `c_fill` példa mutatja.

43.2.5 Többdimenziós operator[]

A C++23-as P2128 többdimenziós subscript operátora miatt az `mdspan` `m[i,j]` formában indexelhető. Ez nem azonos a `m[i][j]` láncolással: 2D `mdspan` esetén az `m[i]` nem ad 1D `mdspan`-t, ezért a `m[i][j]` tipikus hiba. A korai implementációk és tervek `operator()`-t használtak; a C++23-as szabvány `operator[]`-t ír elő. A szeleteléshez a C++23-as `mdspan` nem ad `operator[]`-alapú subview-t; erre a C++26-os `submdspan` szolgál.

43.2.6 submdspan: C++26 kiegészítés

A `std::submdspan` (P2630) C++26-os kiegészítés, ezért a C++23-as laborban nem használjuk, és a feature-test makrója (`__cpp_lib_submdspan`) is csak ott lehet releváns. A C++23-as megoldás kézi subview `layout_stride`-dal, a stride-ok megőrzésével. Egy `layout_right`-os C tömb j-edik oszlopa például:

```
using E1 = std::extents<std::size_t, 3>;
std::array<std::size_t, 1> col_stride{4};
std::mdspan<int, E1, std::layout_stride> col(
    &a[j], E1{}, std::layout_stride::mapping<E1>{E1{}, col_stride});   // a:
     ↪  lapos int[12]
```

A `col[i]` ekkor `a[i * 4 + j]`-vel egyezik. A C++26-os `submdspan` ezt a mintát általánosítja, de a C++23-as kódban nem jelölhető C++23-as képességként.

43.2.7 Tipikus hibák

- `layout_left` használata C tömbhöz: a C tömb sor-major, ezért a `layout_left` rossz indexelést ad.
- `m[i][j]` 2D `mdspan`-ra: az `mdspan` többdimenziós `operator[]`-t vár, nem láncolt indexelést.
- Az `mdspan` dangling lesz a mögöttes `std::vector` újraallokálása után; a nézet nem tulajdonol.
- A `checked_accessor` csak a lineáris offsetet ellenőrzi, a logikai indexeket nem; a `m[0,4]` hibát nem kapja el.
- A `data_handle()` típusa accessor-függő; C API-nál a `checked_handle` esetén `.ptr` kell.
- Az `atomic_accessor` `reference` típusa `std::atomic_ref<T>`: az `m[i,j] = 42` `seq_cst` tárolás; más sorrendezéshez `store(v, order)` kell, és két `atomic_ref` egymásnak nem értékadható.
- Kétdimenziós C tömb lapos `int*`-gal való bejárása: a sorhatáron túl a pointeraritmetika kilép az adott sortömbből; lapos tároló vagy sorpointeres accessor kell.
- A `submdspan` C++26-os; C++23-as laborban nem használható jelöletlenül.

43.2.8 Feladatok

43.2.8.1 Kódolási feladat: layout_stride paddinggal

Készíts 3×4-es `mdspan`-t `layout_stride`-dal úgy, hogy a sorok között egy `int` padding legyen. Ellenőrizd a mapping `required_span_size()` értékét (`m.mapping().required_span_size()`) és az `mdspan` `size()`-át, és állapítsd meg az `is_unique()` és `is_exhaustive()` eredményét. Írd ki a `m[2,3]` elem lineáris offsetjét a pufferben.

43.2.8.2 Diagnosztikai feladat: checked_accessor korlátja

Adott a `checked_accessor`-os `m` 3×4-es, 12 elemes pufferrel. Magyarázd el, miért nem kapja el a `m[0,4]` hibát, miközben a `m[3,0]`-t elkapja. Adj olyan wrappert, amely a logikai indexeket is ellenőrzi.

43.2.8.3 Tervezési feladat: atomic_accessor

Készíts `atomic_accessor`-t, és mutasd meg, hogyan lehet atomikusan írni és olvasni egy `mdspan` elemet. Magyarázd el, milyen memóriasorrendet ad az `m[i,j] = 42`, mikor kell helyette `store(v, order)`, mi a különbség az `atomic_ref` másoló konstrukciója és másoló értékadása között, és milyen követelményeket támaszt a `std::atomic_ref<T>` a `T`-vel és a hivatkozott objektummal szemben.

43.2.9 Megoldások

43.2.9.1 A layout_stride feladat megoldása

A stride-ok `{5,1}`, az extents `3×4`. A legnagyobb offset `(3-1)*5 + (4-1)*1 = 13`, ezért `required_span_size() == 14`. A puffer 14 `int`, a 12 logikai elem mellett 2 padding. A mapping egyedi (minden `layout_stride` mapping az, a konstruktor előfeltétele miatt), de nem kimerítő, mert a 14 elemből 12-t használ. A `m[2,3]` offsetje `2*5 + 3 = 13`.

```cpp
// stride_props.cpp - C++23: a layout_stride mindig egyedi
#include <array>
#include <cstddef>
#include <mdspan>

using E = std::extents<std::size_t, 3, 4>;
using M = std::layout_stride::mapping<E>;

static_assert(M::is_always_unique());       // minden layout_stride mapping
 ↪  egyedi
static_assert(!M::is_always_exhaustive());  // padding lehet: nem mindig
 ↪  kimerítő
static_assert(M::is_always_strided());

int main() {
    int buf[14] = {};
    M map(E{}, std::array<std::size_t, 2>{5, 1});
    std::mdspan<int, E, std::layout_stride> m(buf, map);
    m[2, 3] = 7;
    const bool ok = buf[13] == 7 && m.size() == 12 &&
                    m.mapping().required_span_size() == 14 &&
                    m.mapping()(2, 3) == 13 &&
                    m.is_unique() && !m.is_exhaustive();
    return ok ? 0 : 1;
}
```

Ellenőrzési feltétel: a `size()` 12, a `mapping().required_span_size()` 14, a `[2,3]` offsetje 13, a `buf[13]` értéke 7, az `is_unique()` igaz, az `is_exhaustive()` hamis; a program 0 kóddal lép ki. A referencia-implementációval (lásd fent) ellenőrizve.

43.2.9.2 A checked_accessor feladat megoldása

A `m[3,0]` lineáris offsetje `3*4 + 0 = 12`; a `checked_handle` mérete 12, ezért az `assert(12 < 12)` hamis, és a program megáll. A `m[0,4]` offsetje `0*4 + 4 = 4`; ez a 12 elemes pufferben van, ezért az `assert(4 < 12)` igaz, és az accessor nem jelez. Az accessor csak a lineáris offsetet kapja; a logikai indexek érvényességét nem ismeri. A teljes ellenőrzéshez olyan wrapper kell, amely az extents-et is ismeri:

```cpp
template<class M>
struct bounds_checked_mdspan {
    M m;
    decltype(auto) operator[](std::size_t i, std::size_t j) const {
        assert(i < m.extent(0));
        assert(j < m.extent(1));
        return m[i, j];
    }
};
```

Ez a wrapper a `m[0,4]`-et is elkapja, mert a `j < 4` feltétel hamis. Az ellenőrzési feltétel: a `bounds_checked_mdspan` `m[0,4]` hívása `assert`-et vált ki, a `m[0,3]` nem.

43.2.9.3 Az atomic_accessor feladat megoldása

Az `atomic_accessor` `reference` típusa `std::atomic_ref<int>`. Az `access` a `p[i]`-re hivatkozó `atomic_ref`-et ad vissza. Az atomikus írás és olvasás:

```
std::mdspan<int, std::dextents<std::size_t, 2>,
            std::layout_right, atomic_accessor<int>> am(buf, 3, 4);
am[0, 0] = 2;                                    // seq_cst store
am[0, 1].store(1, std::memory_order_relaxed);    // választott sorrendezés
int v = am[0, 1].load(std::memory_order_relaxed);
assert(v == 1 && am[0, 0].load() == 2);
```

Az `m[i,j] = 42` szabályos: a visszaadott `atomic_ref` prvalue-n a `T operator=(T) const` fut, ami `seq_cst` tárolás. Más sorrendezéshez a `store(v, order)` kell. Az `atomic_ref` másoló konstrukciója megengedett (a másolat ugyanarra az objektumra hivatkozik), a wrapper másoló értékadása törölt. A `T`-nek trivially copyable-nek kell lennie, a hivatkozott objektum igazítása legalább `required_alignment`, és amíg `atomic_ref` él rá, csak azon keresztül érhető el. Ellenőrzési feltétel: az értékadás és a `store` után a `load` az eltárolt értéket adja; két `atomic_ref` közötti értékadás nem fordul le.

43.2.10 Összefoglaló

Az `mdspan` három rétege az extents, a layout mapping és az accessor. A `layout_right` a C sor-major, a `layout_left` a Fortran oszlop-major, a `layout_stride` pedig paddingolt vagy szeletelt pufferhez való. A mapping a logikai indexeket lineáris offsetre képezi, de nem ellenőriz bounds-ot; az accessor már csak a lineáris offsetet látja, ezért a `checked_accessor` a memóriahozzáférés biztonságát javítja, a logikai indexekét nem. A `data_handle_type` accessor-függő, a C API-nál a pointert kell kinyerni. A C++23-as `operator[]` többdimenziós indexelést ad, a `submdspan` viszont C++26-os kiegészítés, ezért a C++23-as laborban kézi, `layout_stride`-os subview-t használunk.

43.3 Callable és forwarding eszközök: move_only_function, forward_like, bind_back

A callback-alapú rendszerprogramozásban két visszatérő probléma van: hogyan tároljunk egy callable-t úgy, hogy közben ne kényszerítsük másolhatóságra a benne élő erőforrásokat, és hogyan adjuk tovább a tartalmazó objektum tagját pontosan azzal az érték-kategóriával és const-voltassággal, amellyel a tartalmazó objektumot elértük. A C++23 három, egymást kiegészítő eszközt ad erre: a `std::move_only_function`-t, a `std::forward_like`-ot, valamint a `std::bind_back`-ot és a `std::invoke_r`-t. Ez a rész az invoke, function wrapper és type erasure (30.2) type erasure tárgyalására épül, és feltételezi az Explicit object parameter: deducing this (40.2) explicit object parameter (deducing this) mechanizmusát, valamint a Template, virtual és std::function költsége (37.5) callable-költség modellt.

Fontos keret: a fejezettervben szereplő javaslat-azonosítók (P0288, P2445, P2387, P2136) a téma irodalmi hivatkozási pontjai, nem pedig általam ellenőrzött, idézett szöveghelyek. **A konkrét fordító- és könyvtártámogatást csak a megnevezett profilokra tekintjük ellenőrzöttnek**; minden új képességhez feature-test makrót és compile probe-ot adok, a támogatás tényleges meglétét a tanuló laborja rögzíti a saját laborprofiljában.

43.3.1 A move_only_function mint minősített callable-tároló

A `std::move_only_function<Sig>` (a `<functional>` fejlécben) egy típus-törlést végző callable-tároló; a „move-only” a *wrapper* értékszemantikájára vonatkozik: a wrapper mozgatható, de nem másolható. A `Sig` alakja `R(Args...)` kiegészítve a hívási operátorra vonatkozó minősítőkkel: `const`, `&`, `&&`, illetve `noexcept`. A minősítők a paraméterlista *után* állnak, például:

```
std::move_only_function<void(int)>                    a;  // semmi extra minősítő
std::move_only_function<void(int) const>              b;  // const objektumon is
 ↪ hívható
std::move_only_function<void(int) &&>                 c;  // csak rvalue-ként
 ↪ hívható
std::move_only_function<void(int) const & noexcept> d;  // const lvalue, nem
 ↪ dobhat
```

A minősítők **statikus szerződést** írnak le, nem futásidejű viselkedést. A `const` azt jelenti, hogy a tárolt callable-t const objektumként hívja meg a wrapper, tehát csak olyan callable-t fogad el, amely const-hívható; egy `mutable` lambda ebbe nem fér bele. Az `&&` azt jelenti, hogy a hívás csak rvalue `move_only_function`-ön engedélyezett, és a tárolt callable is rvalue-ként hívódik — ez teszi lehetővé az „egyszer használatos”, a saját állapotát elmozgató handlereket. A `noexcept` a hívási operátorra kerül: ha a tárolt callable mégis dob, a folyamat `std::terminate`-tel leáll, nem pedig kivételt propagál.

Két, a `std::function`-höz képesti lényegi eltérés:

Szempont	`std::function`	`std::move_only_function`
A wrapper másolhatósága	másolható	csak mozgatható
Tárolt callable követelménye	másolható	közvetlen átvételnél mozgatható; `in_place_type`-pal helyben konstruálva akár nem mozgatható is
Üres wrapper hívása	`std::bad_function_call` kivétel	**nem definiált viselkedés**
Szignatúra-minősítők	nincs	`const`, `&`, `&&`, `noexcept`
`noexcept` hívási szerződés	nem fejezhető ki	kifejezhető

```
#include <functional>
#include <memory>
#include <utility>
struct Immovable {                          // nem mozgatható, nem másolható cél
    int v;
    explicit Immovable(int x) : v(x) {}
    Immovable(Immovable&&) = delete;
    int operator()() const { return v; }
};
int main() {
    std::move_only_function<int() const> f(std::in_place_type<Immovable>, 42);
     ↪ // helyben konstruálva
    std::move_only_function<int() const> g = std::move(f);   // a WRAPPER
     ↪ mozgatható
    auto up = std::make_unique<int>(1);
```

```
    std::move_only_function<int()> h = [p = std::move(up)] { return *p; };  //
     ↪ move-only lambda
#ifdef DIRECT
    std::move_only_function<int() const> bad = Immovable{1};  // közvetlen
     ↪ átvétel: mozgatás kellene
#endif
    return (g() == 42 && h() == 1) ? 0 : 1;
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++23 -Wall -Wextra` mellett lefordult és 0 kóddal futott: a nem mozgatható `Immovable` cél `std::in_place_type`-pal helyben konstruálható, a wrapper ezután is mozgatható, és a move-only lambda is tárolható. `-DDIRECT` mellett a közvetlen átvétel `use of deleted function 'Immovable::Immovable(Immovable&&)'` hibát ad.

Az üres hívás különbsége biztonsági következményű: mivel a `noexcept` szignatúra miatt a wrapper nem dobhat, a szabvány az üres állapotban történő hívást nem definiált viselkedésnek hagyja. Ezért a tárolt handler hívása előtt az explicit `operator bool`-t kell ellenőrizni. Ez a `std::span` „érvényességi szerződés”-logikájával rokon: a típus megengedi az üres állapotot, a hívás előfeltétele viszont a nem üresség.

Nézzünk egy teljes, fordítható példát, amely move-only erőforrást (egy `std::unique_ptr`-t) birtokló handlert tárol:

```
// handlers.cpp - C++23
#include <cstdio>
#include <functional>
#include <memory>
#include <string>
#include <utility>
#include <vector>

#if !defined(__cpp_lib_move_only_function) || __cpp_lib_move_only_function <
 ↪ 202110L
#error "move_only_function nem elerheto ezen a forditon"
#endif

struct session {
    std::string name;
};

class session_registry {
public:
    using handler = std::move_only_function<void(session&) const>;

    void add(handler h) { handlers_.push_back(std::move(h)); }

    void run(session& s) const {
        for (const auto& h : handlers_) {
            if (h) h(s);   // üres handler hívása UB lenne
        }
    }
private:
    std::vector<handler> handlers_;
```

```
};

int main() {
    session_registry reg;
    reg.add([](session& s) { std::printf("plain: %s\n", s.name.c_str()); });
    reg.add([log = std::make_unique<std::vector<std::string>>()](session& s) {
        log->push_back(s.name);   // a unique_ptr const lambda-tagként is
 ↪  mutálható tartalmat ad
        std::printf("logged: %s (%zu)\n", s.name.c_str(), log->size());
    });

    session s{"build-42"};
    reg.run(s);
}
```

Fordítás és futtatás a tanuló laborjában (a konkrét verziót a laborprofil rögzítse):

```
g++ -std=c++23 -O2 -Wall -Wextra -Wpedantic handlers.cpp -o handlers &&
 ↪  ./handlers
# vagy: clang++ -std=c++23 -O2 -Wall -Wextra -Wpedantic handlers.cpp -o handlers
```

A `log` capture move-only, ezért ez a lambda `std::function`-be nem tehető, `move_only_function`-be viszont igen. A `const` szignatúra itt helyes, mert a `unique_ptr` const objektumon keresztül is nem-const hozzáférést ad a mutatott objektumhoz.

43.3.2 forward_like: érték-kategória továbbítása a tartalmazó objektum szerint

A `std::forward_like<T>(x)` (a `<utility>` fejlécben) a `T` érték-kategóriáját és const-voltasságát „ülteti rá” az `x` kifejezésre, miközben `x` saját típusát megtartja. Pontos szabály, `U = std::remove_reference_t<decltype(x)>` jelöléssel:

- a visszatérési típus const-voltasságát `std::is_const_v<std::remove_reference_t<T>>` dönti el;
- azt, hogy a visszatérési típus rvalue vagy lvalue referencia-e, a `T&&` típusra alkalmazott `is_rvalue_reference_v` dönti el;
- a `T`-nek nem kell referencia típusúnak lennie: `forward_like<X>(x)` esetén `T&&` rvalue referencia, tehát az eredmény `X&&`.

Ez az az eszköz, amely a klasszikus „perfect forwarding tagfüggvény” mintát biztonságossá teszi. A C++20-as `std::forward<Self>` erre nem alkalmas, mert a `forward` csak referencia-típusra van értelmezve, és nem képes a const-voltasságot a tartalmazó objektumtól függetlenül átvinni.

A deducing this-szel (C++23 explicit object parameter) a kombináció tömör:

```
// forward_like.cpp - C++23
#include <cstddef>
#include <type_traits>
#include <utility>
#include <vector>

struct frame {
    std::vector<std::byte> payload;
```

```
    auto&& bytes(this auto&& self) noexcept {
        return std::forward_like<decltype(self)>(self.payload);
    }
};

int main() {
    frame f;
    const frame cf;

    static_assert(std::is_same_v<decltype(f.bytes()),
     ↪  std::vector<std::byte>&>);
    static_assert(std::is_same_v<decltype(cf.bytes()), const
     ↪  std::vector<std::byte>&>);
    static_assert(std::is_same_v<decltype(std::move(f).bytes()),
     ↪  std::vector<std::byte>&&>);
}
```

Két részlet magyarázatra szorul. Egyrészt a `decltype(self)` azért helyes, mert a `this Self&& self` deklarációban a `Self` a deduced típus (lehet `frame`, `frame&` vagy `const frame&`), a `self` viszont nevesített paraméter, tehát a *kifejezés* mindig lvalue — a `decltype(self)` a deklarált típust, azaz `Self&&`-t adja vissza, ami pont a kívánt érték-kategória-információ. Másrészt `std::move(self)` használata hibás lenne: lvalue hívó esetén is elmozgatná az objektumot, és a const-voltasságot sem vinné át.

Ha a `forward_like` nem elérhető a laborprofilban, a következő fallback viselkedés-azonos (azt a laborban ellenőrizni kell, hogy a `T` nem-referencia esetét is lefedi):

```
template<class T, class U>
constexpr auto&& my_forward_like(U&& x) noexcept {
    using UT = std::remove_reference_t<U>;
    if constexpr (std::is_const_v<std::remove_reference_t<T>>) {
        if constexpr (std::is_rvalue_reference_v<T&&>) return static_cast<const
         ↪  UT&&>(x);
        else                                           return static_cast<const
         ↪  UT&>(x);
    } else {
        if constexpr (std::is_rvalue_reference_v<T&&>) return
         ↪  static_cast<UT&&>(x);
        else                                           return
         ↪  static_cast<UT&>(x);
    }
}
```

43.3.3 bind_back, invoke_r és a callback-tároló minta

A `std::bind_back(f, args...)` (a `<functional>` fejlécben) egy olyan callable-t ad vissza, amely `g(call_args...)` hívásakor `invoke(f, call_args..., args...)`-t hajt végre: a kötött argumentumok **hátulra** kerülnek. Ez a ranges-adaptorok természetes illesztési iránya, és ott hasznos, ahol az utolsó paraméter a konfiguráció: `std::bind_back(std::clamp, 0, 100)` egy `clamp(x, 0, 100)` hívást eredményez. A kötött argumentumok érték szerint tárolódnak (decay-eléssel), tehát referenciát `std::ref`/`std::cref` nélkül nem lehet kötni; ideiglenes argumentum élettartama a visszaadott objektum belsejéig tart, nem tovább.

A `std::invoke_r<R>(f, args...)` (szintén `<functional>`) az `std::invoke` eredményét `R`-re konvertálja, és ha `R` a `void`, eldobja. Type erasure-implementációkban ez azért értékes, mert a hívási thunk így egységesen írható meg minden `R`-re, és a `void` visszatérésű szignatúra sem igényel külön ágat:

```
template<class R, class F, class... Args>
R call_thunk(void* p, Args... args) {
    return std::invoke_r<R>(*static_cast<F*>(p), std::forward<Args>(args)...);
}
```

A callback-tároló minta e három eszközből áll össze: a hosszú életű erőforrás (socket, puffermenedzser) tulajdonosa az osztály, a handlert `move_only_function` tárolja, a handlert előállító kód `bind_back`-kal köti be a konfigurációt, a hívási határon pedig `invoke_r` biztosítja a szignatúrakonform visszatérést. Így a handler birtokolhat `unique_ptr`-t, a wrapper nem kényszerül másolásra, és a hívás előtti üresség-ellenőrzés egyetlen `if (h)` feltétel marad.

43.3.4 Költség és kódméret

A `move_only_function` hívása **indirekt**: a típus-törlés határán a fordító nem látja a konkrét callable-t, így a hívás nem inline-olható, ellentétben egy közvetlen lambdával vagy egy template paraméterrel. A kis méretű callable-eket a legtöbb implementáció kis pufferben (SBO) tárolja allokáció nélkül, de a wrapper objektum emiatt nagyobb, mint egy puszta függvénypointer; a konkrét SBO-méret és `sizeof` implementációfüggő, ezt ne tekintsük hordozhatónak. Kódméret szempontjából minden `move_only_function<Sig>` példányosítás saját hívási thunk-táblát generál, tehát a **szignatúrák száma** növeli a binárist; egy adott szignatúrán belül a különböző callable-ek száma jellemzően nem növeli arányosan tovább. A minősítők (`const`, `&`, `&&`, `noexcept`) statikus információk, futásidejű költségük nincs. Mérési terv a laborhoz: azonos számú hívás közvetlen lambdával, függvénypointerrel, `std::function`-nel és `move_only_function`-nel, külön rögzítve a `sizeof`-ot és az allokációszámot. **A mérés eredményét ez a szöveg nem állítja, mert nem futtattam le.**

43.3.5 Tipikus hibák és javításuk

1. **Üres `move_only_function` hívása.** A `std::function`-nél megszokott `bad_function_call` itt nincs: a viselkedés nem definiált. Javítás: `if (h) h(args);`, és a wrapper konstruktorában tiltsuk meg az üres handler regisztrálását.
2. **`mutable` lambda `const` szignatúrába.** `std::move_only_function<void() const>` nem fogad el `mutable` lambdát, mert az nem const-hívható. Javítás: vagy vedd le a `const` minősítőt, vagy ne legyen `mutable` a lambda.
3. **`&&` minősítő lvalue híváson.** `std::move_only_function<void() &&>` csak rvalue objektumon hívható; az `f()` lvalue hívás fordításidejű hiba. Javítás: `std::move(f)()`, ha tényleg egyszer használatos.
4. **`std::forward<Self>` a `forward_like` helyett.** A `forward` csak referencia-típusra van értelmezve, és nem viszi át a const-voltasságot a tartalmazó objektumtól. Javítás: `std::forward_like<decltype(self)>(self.tag)`.
5. **`std::move(self)` deducing this-ben.** Elmozgatja az lvalue hívó objektumát is. Javítás: kizárólag `forward_like`.
6. **`bind_back` referenciakötés nélkül.** Az argumentumok decay-elődnek és bemásolódnak; referenciához `std::ref` kell, és az ideiglenes argumentum élettartama nem hosszabbodik meg.

43.3.6 Összefoglaló

A `move_only_function` minősített szignatúrája statikus hívási szerződést ad (const, ref, noexcept), cserébe az üres hívás nem definiált, és a wrapper csak mozgatható. A `forward_like` a tartalmazó objektum érték-kategóriáját és const-voltasságát viszi át a tagra, és a deducing this-szel együtt teszi helyesen kifejezhetővé a perfect forwarding tagfüggvényt. A `bind_back` a hátulról kötött konfigurációt, az `invoke_r` a szignatúra-konform hívási thunkot adja; a három együtt a move-only erőforrást birtokló callback-tároló természetes eszközkészlete. Minden új képességnél feature-test makróval és compile probe-bal kell ellenőrizni a tényleges támogatást.

43.3.7 Feladatok

1. feladat (magyarázó). Egy `event_source` osztály `std::move_only_function<void(int) const>` típusú handlert tárol. A handlerek egyike `std::vector`-ba ír, és memóriahiány esetén kivételt dobhat. Indokold, miért nem szerencsés a szignatúrát `noexcept`-ra váltani, és mi történne, ha mégis megtesszük.

2. feladat (kódolási). Írj `template<class T> struct box` osztályt, amelynek `value()` tagfüggvénye deducing this-szel és `forward_like`-kal a box érték-kategóriája szerint adja vissza a tárolt értéket. `static_assert`-tel igazold a négy esetet: nem-const lvalue, const lvalue, rvalue, const rvalue.

3. feladat (diagnosztikai). Az alábbi kódrészlet három helyen hibás vagy veszélyes. Javítsd, és minden javításhoz add meg az ellenőrzési feltételt.

```
std::function<void(int)> h;
h = [p = std::make_unique<int>(0)](int x) { *p += x; };
h(1);
h = nullptr;
h(2);
auto g = std::bind_back(handler, buffer);   // buffer helyi std::vector
```

4. feladat (kódolási, összetett). Készíts `handler_slot` osztályt, amely az alábbi típusú callable-t tárolja:

```
std::move_only_function<void(std::span<const std::byte>) const>
```

A tárolt callable `set()`-tel cserélhető, és `invoke()`-kal csak nem üres állapotban hív. Egészítsd ki egy `bind_back`-kal előállított handlerrel, amely egy `std::unique_ptr`-ban tartott puffert naplóz.

43.3.8 Megoldások

1. megoldás. A `noexcept` szignatúra azt ígéri, hogy a hívás nem dob kivételt; ha a tárolt handler mégis dob, a folyamat `std::terminate`-tel leáll, és a hiba nem kezelhető a hívó oldalán. Mivel a `std::vector` allokációja dobhat, a `noexcept` itt hamis ígéret lenne. Helyes megoldás: hagyjuk el a `noexcept`-ot, és a hívási ponton `try/catch`-csel vagy `std::expected`-tel kezeljük a hibát. Ellenőrzési feltétel: a szignatúra ne tartalmazzon `noexcept`-ot, és a hívó kód forduljon le kivételkezeléssel.

2. megoldás.

```
template<class T>
struct box {
    T stored;
    auto&& value(this auto&& self) noexcept {
        return std::forward_like<decltype(self)>(self.stored);
    }
};

static_assert(std::is_same_v<decltype(std::declval<box<int>&>().value()),
 ↪  int&>);
static_assert(std::is_same_v<decltype(std::declval<const box<int>&>().value()),
 ↪  const int&>);
static_assert(std::is_same_v<decltype(std::declval<box<int>>().value()),
 ↪  int&&>);
static_assert(std::is_same_v<decltype(std::declval<const box<int>>().value()),
 ↪  const int&&>);
```

Indoklás: a `decltype(self)` a `Self&&` deklarált típust adja, amely a hívó érték-kategóriáját és const-voltasságát kódolja; a `forward_like` ezt ülteti a `stored` tagra, miközben a tag típusát megtartja. Ellenőrzési feltétel: mind a négy `static_assert` forduljon le, és a függvény `noexcept` maradjon.

3. megoldás. (a) A `std::function` csak másolható callable-t fogad el, a `unique_ptr`-t capture-elő lambda viszont move-only — ezért a hozzárendelés nem fordul le. Javítás: `std::move_only_function<void(int)>`. (b) A `h = nullptr;` utáni `h(2)` üres hívást jelent: `std::function` esetén `bad_function_call`, `move_only_function` esetén nem definiált viselkedés. Javítás: `if (h) h(2);`. (c) A `std::bind_back(handler, buffer)` bemásolja a `buffer`-t (decay), így a kötött példány a hívás idejére érvényes marad, de nem a helyi `buffer`-t látja — ha a szándék a helyi vektor elérése volt, `std::ref(buffer)` kell, és gondoskodni kell arról, hogy a `buffer` tovább éljen, mint a visszaadott callable. Ellenőrzési feltétel: a javított kód forduljon le, üres handler esetén ne hívódjon meg a callable, és a `bind_back` esetén a dokumentált élettartam-szerződés szerepeljen kommentben.

4. megoldás.

```
class handler_slot {
public:
    using handler = std::move_only_function<void(std::span<const std::byte>)
     ↪  const>;
    void set(handler h) { h_ = std::move(h); }
    bool ready() const noexcept { return static_cast<bool>(h_); }
    void invoke(std::span<const std::byte> data) const {
        if (h_) h_(data);   // üres hívás UB elkerülése
    }
private:
    handler h_;
};

auto make_logger(std::unique_ptr<std::vector<std::byte>> sink) {
    return std::bind_back(
        [sink = std::move(sink)](std::span<const std::byte> data, std::size_t
         ↪  limit) {
            const auto n = std::min(data.size(), limit);
```

```
            sink->insert(sink->end(), data.begin(), data.begin() + n);
        },
        std::size_t{64});
}
```

Indoklás: a `bind_back` a méretkorlátot köti hátulra, így a hívási felület `void(std::span<const std::byte>)` marad; a lambda move-only capture-t tartalmaz, ezért csak `move_only_function` fogadja el. Ellenőrzési feltétel: `ready()` hamis legyen alapból, `set()` után igaz; a `make_logger` eredménye közvetlenül átadható `set()`-nek; a `limit` értéke a hívásban ne íródjon felül.

43.4 Alacsony szintű és C-interop eszközök C++23-ban

Ez az alfejezet a C++23 azon eszközeit tárgyalja, amelyek közvetlenül a bájtok, a nyers memória és a C API határán dolgoznak. Ezek az eszközök akkor hasznosak igazán, ha a mögöttes objektummodell már rögzített: a bájtok és az objektumreprezentáció alapjait Bitműveletek és objektumreprezentáció C++-ban tárgyalta, a tároló újrafelhasználásának szabályait Tároló újrafelhasználása és C++ objektumszabályok, a C API határát pedig extern C és két nyelvű header mutatta be. Az itt következő anyag ezekre épít, nem ismétli meg őket.

A laborprofil továbbra is ISO C++23, explicit `-std=c++23` kapcsolóval. A konkrét GCC-, Clang-, libstdc++- és libc++-támogatást ebben a fejezetben a megnevezett profilokra (lásd profiltábla) ellenőrzöttnek, másra ellenőrizetlennek tekintjük. Minden új képességhez feature-test makrót és compile probe-ot kell használni; a C++20-as fallbackot a táblázat foglalja össze. A C oldali `effective type` szabályai Effective type, aliasing és objektumreprezentáció nem azonosak a C++ objektum-élettartam szabályaival, ezt a `start_lifetime_as` tárgyalásánál külön kiemelem.

43.4.1 Bájtsorrend, bájtswap és enum-alapú típuskonverzió

A `std::byteswap` C++23-as függvénysablon a `<bit>` fejlécben. Pontos alakja:

```
template<class T>
constexpr T byteswap(T value) noexcept;
```

Feltétel: `T` integrál vagy felsorolás típus. Visszatérési értéke az a `T`, amelynek bájtjai fordított sorrendben követik a bemenetét; ha `sizeof(T) == 1`, az érték változatlan. A függvény nem végez endian-konverziót önmagában: csak a bájtokat cseréli. A hálózati vagy fájlformátum-sorrendet a protokoll rögzíti, a natív sorrendet a C++20 óta ismert `std::endian::native` mondja meg. Helyes használat például:

```
#include <bit>
#include <cstdint>
#include <cstdio>

constexpr std::uint32_t to_network_order(std::uint32_t v) noexcept {
    if constexpr (std::endian::native == std::endian::little) {
        return std::byteswap(v);
    } else {
        return v;
    }
}
```

```
int main() {
    std::printf("%08x\n", to_network_order(0x01020304u));
}
```

A buildparancs: `g++ -std=c++23 -Wall -Wextra -pedantic -o netorder netorder.cpp`. Little-endian gépen a kimenet `04030201`, big-endian gépen `01020304`; a pontos érték a `std::endian::native` függvényében értelmezendő. A `std::byteswap` tehát feltételesen használható, a `std::endian` ellenőrzése nélkül könnyű rossz protokollt írni.

A `std::to_underlying` C++23-as segédfüggvény a `<utility>` fejlécben:

```
template<class Enum>
constexpr std::underlying_type_t<Enum> to_underlying(Enum e) noexcept;
```

Feltétel: `Enum` felsorolás típus. Visszatérési értéke a felsorolás alaptípusára konvertált érték. Előnye a `static_cast`-tal szemben, hogy kifejezi a szándékot, és scoped enumra is ugyanúgy működik. A `std::is_scoped_enum` C++23-as trait a `<type_traits>` fejlécben igaz, ha a típus scoped enum; ezzel overloadokat lehet megkülönböztetni. Példa:

```
#include <cstdint>
#include <type_traits>
#include <utility>

enum class Color : std::uint8_t { red = 1, green = 2 };

static_assert(std::is_scoped_enum_v<Color>);
static_assert(std::to_underlying(Color::green) == 2);
```

A C++20-as fallback: `static_cast<std::underlying_type_t<E>>(e)`, illetve a scoped enum közelítése `std::is_enum_v<T> && !std::is_convertible_v<T, int>` feltétellel.

43.4.2 Objektum élettartam indítása nyers memóriában

A `std::start_lifetime_as` (P2590) C++23-as eszköz a `<memory>` fejlécben. Alapalakja:

```
template<class T>
T* start_lifetime_as(void* p) noexcept;
```

A feltételek két kategóriába esnek (C++23 [obj.lifetime]). **Mandates** (fordítási idejű): `T` teljes, implicit-lifetime típus — ha nem az (például `std::string`), a hívás *ill-formed*, a fordító elutasítja; ez nem futásidejű UB. **Előfeltétel** (futásidejű): a `[p, p + sizeof(T))` bájttartomány lefoglalt tároló, amely a `p`-n keresztül elérhető (*reachable*) tárolórész része, és `alignof(T)`-nek megfelelően igazított; ennek megsértése nem definiált viselkedés. Élő objektum a területen *nem* tiltott: a hívás a tárolót újrahasznosítja, így az átfedő, korábban élő objektumok élettartama véget ér — ugyanazokkal a korlátokkal, mint bármely tár-újrahasznosításnál (például statikus, szál- vagy automatikus tárolású `const` teljes objektum tárolója nem hasznosítható újra; lásd Tároló újrafelhasználása és C++ objektumszabályok (20.6)). A függvény elindítja egy `T` objektum élettartamát anélkül, hogy konstruktort hívna vagy bájtokat másolna; a bájtok változatlanok. A `std::start_lifetime_as_array` ugyanezt tömb esetére végzi. Az `std::is_implicit_lifetime_v<T>` trait C++23-ban eldönti, hogy `T` implicit-lifetime típus-e; tipikusan skalár, triviálisan másolható és triviálisan destruálható osztály, aggregate.

Kidolgozott példa:

```cpp
#include <cstdint>
#include <cstdio>
#include <cstring>
#include <memory>
#include <type_traits>

struct Header {
    std::uint32_t magic;
    std::uint32_t length;
};

static_assert(std::is_implicit_lifetime_v<Header>);

int main() {
    alignas(Header) unsigned char buffer[sizeof(Header)];
    const unsigned char bytes[sizeof(Header)] = {
        0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08
    };
    std::memcpy(buffer, bytes, sizeof(Header));
    Header* h = std::start_lifetime_as<Header>(buffer);
    std::printf("%08x %08x\n", h->magic, h->length);
}
```

A buildparancs: `g++ -std=c++23 -Wall -Wextra -pedantic -o lifetime lifetime.cpp`. A példa várt kimenete endianfüggő: a `printf` a bájtokból olvasott értéket mutatja, nem univerzális layoutot. A `Header*` érvényessége a `buffer` élettartamához kötött; ha a `buffer` egy dinamikusan allokált puffer, akkor a puffer felszabadítása után a `Header*` dangling lesz.

Három esetet érdemes szétválasztani: (1) **hibás típus** (`std::string`): fordítási hiba a Mandates miatt; (2) **hibás igazítású vagy nem elérhető tároló**: nem definiált viselkedés, amelyre nem várhatunk megbízható sanitizer-diagnosztikát — a helyes kód az `alignas`-szal és a tároló méretével előre kizárja; (3) **szabályos tár-újrahasznosítás**: ha a területen korábban egy másik objektum élt, annak élettartama véget ér, és a régi pointerek csak a 20.6-ban leírt feltételekkel használhatók tovább. A `memcpy`, az implicit objektum-létrehozás és a `start_lifetime_as` viszonya: a `memcpy` a céltárolóban implicit módon létrehozhat implicit-lifetime objektumot, de a bájtokat másolja; a `start_lifetime_as` a meglévő bájtokat értelmezi másolás nélkül, és explicit módon jelöli, melyik típusú objektum kezd élni. A C oldali `effective type` szabályai Effective type, aliasing és objektumreprezentáció ettől eltérnek: a C++23-as eszköz a C++ objektum-élettartamot indítja el, nem a C `effective type`-ot állítja be.

Implementációs állapot: a 2026-10-25-i ellenőrzésnél sem a GCC 13.3 libstdc++-a, sem a Clang 22 libc++-a nem definiálta a `__cpp_lib_start_lifetime_as` makrót, ezért a fenti példát egyik profilban sem fordítottuk le. A feature-test makró neve `__cpp_lib_start_lifetime_as`; ha nem definiált, a képesség nem használható. A hordozható C++20-as fallback a `std::memcpy` egy lokális `T` objektumba, ami viszont másol, és nem ugyanaz a szemantika.

43.4.3 C API és okos pointerek: out_ptr és inout_ptr

A `std::out_ptr` és `std::inout_ptr` (P1132) C++23-as eszközök a `<memory>` fejlécben. Céljuk, hogy egy C API `T**` kimeneti paraméterét közvetlenül egy okos pointerhez lehessen kötni. A `std::out_ptr(p, args...)` egy `out_ptr_t` proxyt ad vissza, amely a C API hívás idejére átadja a nyers pointer címét. A három lépés külön történik: (1) a proxy **konstruktora** reseteli a `p`

okos pointert — a régi erőforrás ekkor, a C hívás *előtt* szabadul fel (a C++23 szerződés szerint); (2) a C API beírja az új pointert a proxy által adott címre; (3) a proxy **destruktora** a beírt pointerrel visszaállítja `p`-t (`reset(új)`). Ebből következik, hogy sikertelen C-hívásnál, ha az API nem ír a pointerbe, a `p` már üres, és a régi erőforrás elveszett. Ha a régi ownert meg kell őrizni, tranzakciós mintát kell használni: nyers ideiglenes pointerbe hívjuk az API-t, siker esetén `p.reset(tmp)`, hiba esetén `p` érintetlen. Az `inout_ptr` más szerződés: a kezdeti pointert átadja a C API-nak (a régi erőforrás felszabadítása az API dolga, és a smart pointer nem szabadítja fel előtte), így a `realloc`-szerű, replace-ra tervezett API-hoz való. Az adaptert az API **tulajdonlási szerződése** dönti el: kizárólag új erőforrást létrehozó (out-only), siker esetén felülíró C API-hoz `inout_ptr`-t tenni szivárgást okozhat, ha a régi erőforrást senki nem zárja le; ilyenkor `out_ptr` vagy a tranzakciós minta való. Három példa: out-only (`out_ptr`), replace/inout (`inout_ptr`), failure-preserving (nyers temp + `reset` siker esetén).

Példa `std::unique_ptr`-ral:

```
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <memory>

struct Header {
    std::uint32_t magic;
    std::uint32_t length;
};

extern "C" int get_buffer(unsigned char** data, std::size_t* size) {
    auto* p = static_cast<unsigned char*>(std::malloc(64));
    if (!p) return -1;
    std::memset(p, 0, 64);
    const Header init{0x47414944u, 16u};   // natív reprezentáció: minden
    std::memcpy(p, &init, sizeof init);    // később olvasott bit definiált
    *data = p;
    *size = 64;
    return 0;
}

int main() {
    std::unique_ptr<unsigned char, decltype(&std::free)> buf(nullptr,
     ↪  &std::free);
    std::size_t n = 0;
    if (get_buffer(std::out_ptr(buf), &n) != 0) return 1;
    if (n >= sizeof(Header)) {
        Header* h = std::start_lifetime_as<Header>(buf.get());
        std::printf("%u %u\n", h->magic, h->length);
    }
}
```

A buildparancs: `g++ -std=c++23 -Wall -Wextra -pedantic -o outptr outptr.cpp`. A `std::start_lifetime_as` egyetlen dolgot tesz: a megadott tárolón elindítja egy `Header` objektum élettartamát, a tárolóban már meglévő bájtok értelmezésével. **Nem** inicializál: ha a bájtok határozatlanok (például egy frissen `malloc`-olt pufferben), a mezők olvasása határozatlan értéket ad. Ezért a `get_buffer` a példában minden később olvasott bájtot beállít. A bájtsorrendet

sem kezeli: a bájtokat a gép natív reprezentációjaként értelmezi. A protokollt sem validálja: a `length` értékének ellenőrzése a hívó feladata.

Hálózatról vagy fájlból érkező, rögzített bájtsorrendű adatnál ezért a mezőnkénti dekódolás a helyes út, amely C++20-ban is működik:

```
// decode_header.cpp - C++20: mezőnkénti dekódolás little-endian wire formátumból
#include <cstddef>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <memory>

struct Header {
    std::uint32_t magic;
    std::uint32_t length;
};

// A C oldal 8 bájtos fejlécet ad: magic és length, mindkettő little-endian.
extern "C" int get_buffer(unsigned char** data, std::size_t* size) {
    auto* p = static_cast<unsigned char*>(std::malloc(64));
    if (!p) return -1;
    std::memset(p, 0, 64);                                  // minden bájt definiált
    const unsigned char wire[8] = {'D', 'I', 'A', 'G', 16, 0, 0, 0};
    std::memcpy(p, wire, sizeof wire);
    *data = p;
    *size = 64;
    return 0;
}

std::uint32_t load_le32(const unsigned char* p) noexcept {
    return std::uint32_t{p[0]} | std::uint32_t{p[1]} << 8 |
           std::uint32_t{p[2]} << 16 | std::uint32_t{p[3]} << 24;
}

int main() {
    unsigned char* raw = nullptr;
    std::size_t n = 0;
    if (get_buffer(&raw, &n) != 0) return 1;
    std::unique_ptr<unsigned char, decltype(&std::free)> buf(raw, &std::free);
    if (n < 8) return 1;                                    // méretvalidálás
    const Header h{load_le32(buf.get()), load_le32(buf.get() + 4)};
    if (h.length > n - 8) return 1;                         // protokollvalidálás
    std::printf("magic=0x%08x length=%u\n", static_cast<unsigned>(h.magic),
                static_cast<unsigned>(h.length));
}
```

Buildjegyzőkönyv (`lab-gcc` profil). A `decode_header.cpp` `g++ -std=c++20 -Wall -Wextra -Wpedantic` beállítással figyelmeztetés nélkül fordult, és `magic=0x47414944 length=16` kimenettel futott, ASan/UBSan alatt is. A `std::out_ptr` és a `std::start_lifetime_as` a GCC

13.3 libstdc++-ában nem érhető el, ezért az `outptr.cpp`-t a `lab-gcc` profilban nem fordítottuk le; az a `lab-next` profilhoz tartozik, és ott ellenőrizendő. A C oldali fejlécben a `get_buffer` `extern "C"` blokkban deklarálandó; a kétnyelvű header részleteit extern C és két nyelvű header tárgyalta. A C függvény szerződése: siker esetén (0) új, `malloc`-olt puffert ír a `*data`-ba, hiba esetén (−1, `malloc`-kudarc) a `*data`-t nem írja. Az `out_ptr` viselkedése ehhez: a proxy **már a létrehozásakor** reseteli az okos pointert (a korábbi erőforrást felszabadítja), és a hívás után csak akkor veszi át a beírt pointert, ha az nem null. Sikernél tehát a régi erőforrás felszabadul és az új kerül a helyére; hibánál a régi szintén felszabadult, és az okos pointer üres marad — szivárgás és dupla lezárás nincs, de a régi erőforrás elveszett. Ha a hibaágban a régi erőforrásnak meg kell maradnia, `inout_ptr` kell: az a meglévő pointert adja át, és ha a C függvény hibánál érintetlenül hagyja, az okos pointer visszakapja; vagy manuális `raw` változó és csak sikernél `reset`. **Buildjegyzőkönyv:** a Clang 22 / libc++ `std::out_ptr`/`std::inout_ptr`-ével egy élő-erőforrás-számlálós próba nem üres `unique_ptr`-rel: `out_ptr` sikernél új erőforrás (élő: 1), hibánál üres pointer (élő: 0); `inout_ptr` sikernél új erőforrás, hibánál a régi megmaradt (élő: 1); a végén minden erőforrás felszabadult (élő: 0). A `Header*` érvényessége a `buf` élettartamához kötött: a `buf` felszabadítása után a pointer dangling.

43.4.4 Allokáció és string puffer

Az `allocate_at_least` (P0401) C++23-ban az allokátorok olyan művelete, amely legalább a kért elemszámot allokálja, és visszaadja a ténylegesen allokált elemszámot is. Két felületen érhető el: tagfüggvényként a `std::allocator<T>::allocate_at_least(n)`, általános allokátorra pedig a `std::allocator_traits<A>::allocate_at_least(a, n)`; `std::allocate_at_least` szabad függvény nincs. Az eredmény egy `std::allocation_result<T*>`, `ptr` és `count` taggal, ahol `count >= n`. A felszabadítás szerződése (C++23 [allocator.members]): ha a `p` egy `allocate_at_least(req)` hívásból származik, a `deallocate(p, n)` hívásban az `n` bármely érték lehet, amelyre `req <= n <= ret.count`. Kérés 10, `count` 16 esetén tehát a 10 és a 16 közötti bármely méret elfogadható. A `count` megőrzése és használata ajánlott, mert így a tényleges kapacitást a teljes élettartam alatt ismerjük, de a kért mérettel való felszabadítás nem hiba. Példa:

```
#include <cstdio>
#include <memory>

int main() {
    std::allocator<int> a;
    auto r = a.allocate_at_least(10);
    std::printf("count = %zu\n", r.count);
    a.deallocate(r.ptr, r.count);
}
```

A buildparancs: `g++ -std=c++23 -Wall -Wextra -pedantic -o alloc alloc.cpp`. A `std::vector` C++23-as implementációja használhatja ezt a növekedésnél, de ez implementációfüggő, nem garantált. A `lab-gcc` profil GCC 13.3-as libstdc++-ában az `allocate_at_least` nem érhető el (nincs `__cpp_lib_allocate_at_least`), ezért a példa a `lab-next` profilhoz tartozik, és ott ellenőrizendő.

A `std::string::resize_and_overwrite` (P1072) C++23-as tagfüggvény lehetővé teszi, hogy a string pufferebe közvetlenül írjunk, elkerülve a kétszeres inicializálást. Alakja:

```
template<class Operation>
constexpr void resize_and_overwrite(size_type n, Operation op);
```

Az `op` hívása `(char* p, size_type n)` szignatúrával történik; visszatérési értéke a string új mérete, amely nem lehet nagyobb `n`-nél. Az `op` csak a puffer első `n` karakterébe írhat, a string objektumot nem módosíthatja. Példa POSIX `read`-dal:

```
// read_up_to.cpp - C++23: legfeljebb max bájt, a limit túllépése külön állapot
#include <cerrno>
#include <cstddef>
#include <string>
#include <unistd.h>

enum class read_status { complete, limit_exceeded, error };

// Legfeljebb max bájtot olvas az fd-ből az out-ba. complete: EOF-ig minden
// adat belefért; limit_exceeded: max bájt után még volt adat (az out csak az
// első max bájt, és a forrásból egy további bájt elfogyott); error: olvasási
// hiba, az out a hiba előtti részadat, a kód err-ben.
read_status read_up_to(int fd, std::string& out, std::size_t max, int& err) {
    err = 0;
    out.resize_and_overwrite(max, [&](char* p, std::size_t n) -> std::size_t {
        std::size_t total = 0;
        while (total < n) {
            ssize_t r = ::read(fd, p + total, n - total);
            if (r < 0) {
                if (errno == EINTR) continue;
                err = errno;
                return total;
            }
            if (r == 0) break;
            total += static_cast<std::size_t>(r);
        }
        return total;
    });
    if (err != 0) return read_status::error;
    if (out.size() < max) return read_status::complete;   // EOF a limit előtt
    for (;;) {                                            // pontosan max: van
     ↪  még adat?
        char extra;
        ssize_t r = ::read(fd, &extra, 1);
        if (r < 0 && errno == EINTR) continue;
        if (r < 0) { err = errno; return read_status::error; }
        return r == 0 ? read_status::complete : read_status::limit_exceeded;
    }
}
```

A függvény neve szándékosan `read_up_to`, nem `read_all`: a siker csak a legfeljebb `max` bájtos prefixre szól, és a limit túllépése külön állapot (`limit_exceeded`), nem észrevétlen teljes siker. A túllépés észleléséhez a függvény pontosan `max` bájt után még egy bájtot megpróbál olvasni; ez a bájt a forrásból elfogy, ami a hívó szerződésének része. A hibaágban a string a hiba előtti részadat, a kód `err`-ben. Ha a lambda visszatérési értéke nagyobb `n`-nél, az nem definiált viselkedés.

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++23 -Wall -Wextra -Wpedantic -fsanitize=address,undefined -DREAD_UP_TO_TEST` mellett egy pipe-os tesztprogrammal: `max = 8` mellett 7 bájt → `complete` (7), 8 bájt → `complete` (8), 9 bájt → `limit_exceeded` (8); `max`

`= 0` mellett üres bemenet → `complete`, egy bájt → `limit_exceeded`; érvénytelen fd → `error, EBADF`. Kimenet: `read_up_to: OK`.

A `std::string::contains` és `std::string_view::contains` C++23-as bővítés: `bool contains(basic_string_view sv) const` és `bool contains(char c) const`. Példa:

```
#include <cassert>
#include <string>

int main() {
    std::string s = "GET / HTTP/1.1";
    assert(s.contains("HTTP/1.1"));
    assert(s.contains('/'));
}
```

A C++20-as fallback: `find(...) != std::string::npos`.

A következő táblázat a feature-test makrókat és a C++20-as fallbackokat foglalja össze. A makrók pontos értékét és elérhetőségét a választott fordító fejléce és support-táblázata alapján kell ellenőrizni; a táblázat nem állítja, hogy bármelyik implementáció támogatja őket.

Eszköz	Feature-test makró	Fallback
`std::byteswap`	`__cpp_lib_byteswap`	(1)
`std::to_underlying`	`__cpp_lib_to_underlying`	(2)
`std::is_scoped_enum`	`__cpp_lib_is_scoped_enum`	(3)
`std::start_lifetime_as`	`__cpp_lib_start_lifetime_as`	(4)
`std::out_ptr`	`__cpp_lib_out_ptr`	(5)
`allocate_at_least`	`__cpp_lib_allocate_at_least`	(6)
`resize_and_overwrite`	`__cpp_lib_string_resize_and_overwrite`	(7)
`string::contains`	`__cpp_lib_string_contains`	(8)

A C++20-as fallbackek:

1. Saját bájtcsere előjel nélküli típusra (shift és maszk).
2. `static_cast<std::underlying_type_t<E>>(e)`.
3. `std::is_enum_v<T> && !std::is_convertible_v<T, int>`.
4. `std::memcpy` egy lokális `T` objektumba — ez másol, nem ugyanaz a szemantika.
5. Manuális `T* raw = nullptr; c_api(&raw); if (raw) p.reset(raw);`.
6. `allocate(n)`, a kapacitás `count = n`.
7. `resize(n)`, írás, majd `resize(r)`.
8. `find(...) != npos`.

Ellenőrzés (`-std=c++23 -dM -E`, `<version>`): a táblázat makrói közül

- a GCC 13.3 libstdc++-ából hiányzik a `start_lifetime_as`, az `out_ptr` és az `allocate_at_least` makrója;
- a Clang 22 libc++-ából csak a `start_lifetime_as` makrója hiányzik.

A labor első lépése lehet egy compile probe, amely hiányzó képesség esetén szándékosan fordítási hibát ad:

```cpp
// interop_probe.cpp - C++23: képességleltár (<version>) + valódi minimálhívások
#include <version>      // a feature-test makrók forrása, minden más előtt
#include <bit>          // byteswap
#include <cstdint>
#include <memory>       // start_lifetime_as, out_ptr,
 ↪  allocator::allocate_at_least
#include <string>       // resize_and_overwrite, contains
#include <type_traits>  // is_scoped_enum
#include <utility>      // to_underlying

#if !defined(__cpp_lib_byteswap) || !defined(__cpp_lib_to_underlying) || \
    !defined(__cpp_lib_is_scoped_enum) || !defined(__cpp_lib_start_lifetime_as)
 ↪  || \
    !defined(__cpp_lib_out_ptr) || !defined(__cpp_lib_allocate_at_least) || \
    !defined(__cpp_lib_string_resize_and_overwrite) || \
    !defined(__cpp_lib_string_contains)
#error "A laborprofil C++23 interop-kepessegei hianyosak"
#endif

enum class E : std::uint8_t { a = 1 };
static_assert(std::byteswap(std::uint16_t{0x0102}) == 0x0201);
static_assert(std::to_underlying(E::a) == 1);
static_assert(std::is_scoped_enum_v<E>);

int main() {
    alignas(int) unsigned char buf[sizeof(int)] = {};
    int* p = std::start_lifetime_as<int>(buf);
    std::unique_ptr<int> up;
    [](int** out) { *out = nullptr; }(std::out_ptr(up));
    std::allocator<int> al;
    auto r = al.allocate_at_least(4);
    al.deallocate(r.ptr, r.count);
    std::string s;
    s.resize_and_overwrite(3, [](char* q, std::size_t) { q[0] = 'a'; return
 ↪  std::size_t{1}; });
    return (s.contains('a') && p != nullptr) ? 0 : 1;
}
```

A képességleltár a `<version>`-ből jön, minden makrót a saját fejléce mellett nézünk, és a makrók mellett minden képességet egy valódi minimálhívással is lefordítunk; egyetlen képességhez külön probe is írható, amely csak a saját fejléceit húzza be:

```cpp
// probe_to_underlying.cpp - egyetlen képesség, csak a saját fejléceivel
#include <version>
#include <utility>
#if !defined(__cpp_lib_to_underlying) || __cpp_lib_to_underlying < 202102L
#error "std::to_underlying nem erheto el"
#endif
enum class Color { red, green = 2 };
static_assert(std::to_underlying(Color::green) == 2);
int main() {}
```

Buildjegyzőkönyv. A teljes probe a GCC 13.3 és a Clang 22 / libc++ alatt is a saját `#error`

üzenetével állt meg (mindkettőből hiányzik a `start_lifetime_as`); a `probe_to_underlying.cpp` mindkettővel lefordult. A `start_lifetime_as` nélküli változat minimálhívásai a Clang 22 / libc++ alatt lefordultak és 0 kóddal futottak.

43.4.5 Tipikus hibák

- A `std::byteswap` önmagában nem endian-konverzió; a `std::endian::native` ellenőrzése nélkül használva rossz protokollsorrendet ad.
- A `std::start_lifetime_as` nem implicit-lifetime típusra (például `std::string`-re) fordítási hiba (Mandates), nem futásidejű UB.
- A `std::start_lifetime_as` rosszul igazított vagy túl kicsi tárolóra: nem definiált viselkedés; az `alignas` és a méret előzetes rögzítése a védelem.
- A `std::out_ptr` a konstruktorában resetel, ezért hibaágban elveszítheti a régi erőforrást, ha a C API nem ír a pointerbe; megőrzéshez tranzakciós minta (nyers ideiglenes pointer, siker esetén `reset`) kell. Az `inout_ptr` nem általános javítás: csak olyan API-hoz való, amely a régi erőforrást maga kezeli (replace); out-only API-hoz szivárgást okozhat.
- A `resize_and_overwrite` visszatérési értéke nem lehet nagyobb a kért `n`-nél; ha mégis, nem definiált viselkedés.
- Az `allocate_at_least` eredményét olyan mérettel deallokálni, amely kisebb a kértnél vagy nagyobb a visszaadott `count`-nál. A `req <= n <= count` tartományon belül bármely méret megengedett; a `count` megőrzése a legegyszerűbb, egyértelmű szabály.
- A `string::contains` C++23-as; C++20-ban a `find` és a `npos` összehasonlítása szükséges.

43.4.6 Feladatok

1. Írj egy `to_network_order(std::uint32_t)` függvényt, amely a hoszt byte orderből hálózati byte orderbe konvertál. Használd a `std::endian` és `std::byteswap` eszközöket. Add meg a buildparancsot, és magyarázd el, miért nem elég a `std::byteswap` önmagában. Ellenőrizd az eredményt little-endian és big-endian esetre.

2. Adott a következő C API:

```
extern "C" int get_buffer(unsigned char** data, std::size_t* size);
```

amely egy `malloc`-olt puffert ad vissza. Készíts egy `std::unique_ptr<unsigned char, decltype(&std::free)>`-t, és használd `std::out_ptr`-t. Ezután értelmezd a puffer első `sizeof(Header)` bájtját `std::start_lifetime_as<Header>`-rel. Sorolj fel minden szükséges előfeltételt, és mondd meg, meddig érvényes a kapott `Header*`.

3. Írj tesztprogramot a fenti `read_up_to` függvényhez: pipe-on át adj neki `max - 1`, `max` és `max + 1` hosszú bemenetet, valamint `max == 0` mellett üres és nem üres bemenetet, és ellenőrizd az állapotot és a string méretét. Magyarázd el, miért kell a pontosan `max` hosszú esetet a limit túllépésétől megkülönböztetni.

43.4.7 Megoldások

43.4.7.1 1. feladat megoldása

```
#include <bit>
#include <cstdint>
#include <cstdio>
```

```
constexpr std::uint32_t to_network_order(std::uint32_t v) noexcept {
    if constexpr (std::endian::native == std::endian::little) {
        return std::byteswap(v);
    } else {
        return v;
    }
}

int main() {
    std::printf("%08x\n", to_network_order(0x01020304u));
}
```

A `std::byteswap` csak bájtokat cserél; a hálózati sorrendet a protokoll rögzíti, ezért kell a `std::endian::native` feltétel. Little-endian gépen a várt kimenet `04030201`, big-endian gépen `01020304`. Ellenőrzési feltétel: a kimenet a gépi endiannek megfelelően a hálózati sorrendet tükrözze, és a program `-Wall -Wextra -pedantic` mellett figyelmeztetés nélkül forduljon.

43.4.7.2 2. feladat megoldása

```
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <memory>

struct Header {
    std::uint32_t magic;
    std::uint32_t length;
};

extern "C" int get_buffer(unsigned char** data, std::size_t* size) {
    auto* p = static_cast<unsigned char*>(std::malloc(64));
    if (!p) return -1;
    std::memset(p, 0, 64);
    const Header init{0x47414944u, 16u};   // natív reprezentáció: minden
    std::memcpy(p, &init, sizeof init);    // később olvasott bit definiált
    *data = p;
    *size = 64;
    return 0;
}

int main() {
    std::unique_ptr<unsigned char, decltype(&std::free)> buf(nullptr,
     ↪  &std::free);
    std::size_t n = 0;
    if (get_buffer(std::out_ptr(buf), &n) != 0) return 1;
    if (n < sizeof(Header)) return 1;
    Header* h = std::start_lifetime_as<Header>(buf.get());
    std::printf("%u %u\n", h->magic, h->length);
}
```

Előfeltételek: a `get_buffer` sikeres hívás esetén ír a `data` pointerbe, és a puffer első `sizeof(Header)` bájtját definiált értékkel tölti fel (a `start_lifetime_as` nem inicializál); a puffer mérete legalább `sizeof(Header)`; a `malloc` igazítása legalább `alignof(Header)`; a `Header` implicit-lifetime típus; a pufferben nincs élő `Header` objektum. A `Header*` a `buf` élettartamáig érvényes; a `buf` felszabadítása után dangling. Ellenőrzési feltétel: a program csak akkor hívja meg a `start_lifetime_as`-t, ha a méret megfelelő, és a `Header*`-ot nem használja a `buf` felszabadítása után.

43.4.7.3 3. feladat megoldása

```
// read_up_to.cpp (folytatás) - tesztmain, csak -DREAD_UP_TO_TEST mellett
#ifdef READ_UP_TO_TEST
#include <cassert>
#include <cstdio>
static read_status run(const std::string& data, std::size_t max, std::string&
 ↪  out) {
    int fds[2];
    if (::pipe(fds) != 0) return read_status::error;
    (void)!::write(fds[1], data.data(), data.size());
    ::close(fds[1]);
    int err = 0;
    read_status s = read_up_to(fds[0], out, max, err);
    ::close(fds[0]);
    return s;
}
int main() {
    const std::size_t max = 8;
    std::string out;
    assert(run(std::string(max - 1, 'a'), max, out) == read_status::complete &&
     ↪  out.size() == max - 1);
    assert(run(std::string(max, 'a'), max, out) == read_status::complete &&
     ↪  out.size() == max);
    assert(run(std::string(max + 1, 'a'), max, out) ==
     ↪  read_status::limit_exceeded && out.size() == max);
    assert(run("", 0, out) == read_status::complete && out.empty());
    assert(run("x", 0, out) == read_status::limit_exceeded && out.empty());
    int err = 0;
    assert(read_up_to(-1, out, max, err) == read_status::error && err == EBADF);
    std::puts("read_up_to: OK");
}
#endif
```

A tesztet a fenti `read_up_to.cpp` végéhez fűzzük; az `#ifdef READ_UP_TO_TEST` blokk miatt a main csak tesztbuildben készül el, és `-DREAD_UP_TO_TEST`-tel fordítjuk. A pontosan `max` hosszú bemenet és a `max + 1` hosszú bemenet első `max` bájtja azonos; a különbséget csak a további olvasási kísérlet (EOF vagy újabb bájt) dönti el. E nélkül a csonkolt olvasás teljes sikerként jelenne meg. Ellenőrzési feltétel: a fenti `assert`-ek teljesülnek, és a program `read_up_to: OK`-t ír ki.

43.4.8 Összefoglaló

A C++23 alacsony szintű eszközei akkor biztonságosak, ha a mögöttes objektum-élettartam- és C API-szerződést pontosan betartjuk. A `std::byteswap` csak bájtcserét végez, az endian-döntést a `std::endian` és a protokoll adja. A `std::start_lifetime_as` nyers tárolóban indít implicit-lifetime objektumot, feltételei megsértése nem definiált viselkedés. A `std::out_ptr` és `std::inout_ptr` az okos pointer és a C API közötti tulajdonjog-átadást teszi kifejezővé, de a hibaág szerződését ellenőrizni kell. Az `allocate_at_least`, a `resize_and_overwrite` és a `contains` a teljesítmény és a kényelem irányába bővítik a standard libraryt; használatukhoz feature-test makró és a tényleges implementáció ellenőrzése szükséges.

44 Modulok, generátorok és migráció C++23-ra

44.1 import std és import std.compat

A C++20 nyelvi moduljai önmagukban még nem oldották meg azt a problémát, ami miatt a modulokat egyáltalán kitalálták: a szabványos könyvtár továbbra is headerekből állt, és minden fordítási egység újra és újra beolvasta, előfeldolgozta és értelmezte ugyanazokat a szöveges fejléceket. A C++23 egyik leginkább „üzemi” jellegű újdonsága, hogy a könyvtár nagy része elérhetővé vált modulként is: `import std;` és `import std.compat;`. Ez a rész azt tanítja meg, hogyan építsünk ilyen projektet buildrendszerrel együtt, milyen feltételek mellett érdemes, és – ami a gyakorlatban legalább ilyen fontos – mikor marad jobb választás a `#include`.

A tárgyalás az alapprofil szerint ISO C++23 (`-std=c++23`), a C++20-as laborprofil kiterjesztéseként. Ahol konkrét fordító-, könyvtár- vagy CMake-verzió szerepel, az a megnevezett profilra vonatkozó ellenőrzendő hivatkozás; igazoltnak csak az tekinthető, amit a szöveg futtatott eredményként jelöl. A C++20-as modulmechanika alapjait itt nem tanítjuk újra; ezekre a C++20 modules és header unit (33.4) részben rögzítettek szerint építünk, a target-alapú buildet a Modern CMake target-alapú projekt (38.2), a laborprofil és feature-test szemléletet a C++23 laborprofil, szabványstátusz és feature-test (40.1) rész adja.

44.1.1 A szabványos könyvtár mint modul: definíció és feltételek

Az `import std;` egy import-deklaráció, amely a `std` nevű szabványos könyvtári modult importálja. A `std` modul tartalma definíció szerint a szabványos C++ headerek által biztosított deklarációk halmaza, a `std` névtérben – vagyis ugyanazok az entitások, amelyeket `#include <vector>` és társai deklarálnak, csak nem szövegesen újra beolvasva, hanem egy lefordított modul-interfész egységen (module interface unit) keresztül elérve.

Ez a megfogalmazás két dolgot rögzít. Egyrészt az `import std;` nem „varázslat”: nem ad hozzá semmit a nyelvhez, ugyanazokat a deklarációkat teszi láthatóvá, mint a megfelelő headerek. Másrészt viszont a *hozzáférés módja* más: a fogyasztó fordítási egysége nem a fejlécszöveget dolgozza fel, hanem a könyvtár által előre legyártott modul bináris interfészt (BMI, Binary Module Interface) olvassa be. A P2465 jelű WG21-javaslat („Standard Library Modules std and std.compat”) ezt a szerződést rögzíti.

A használat feltételei hármasak, és mindháromnak egyszerre kell teljesülnie. A fordítónak támogatnia kell a modulokat és konkrétan a `std` modult; a használt könyvtár-implementációnak (libstdc++ vagy libc++) szállítania kell a modul interfészét; végül a buildrendszernek kezelnie kell a BMI-t, mint köztes terméket, a megfelelő sorrendezéssel és függőségkövetéssel. Ha bármelyik hiányzik, az `import std;` nem „lassabb lesz”, hanem nem fordul le. Ezért ezt a döntést nem a forráskódban, hanem a buildkonfigurációban kell meghozni, és a laborprofilban rögzíteni.

44.1.2 import std és import std.compat

A két modul viszonya egyszerű, de könnyű félreérteni. Az `import std;` a `std` névtérbe exportálja a szabványos könyvtár deklarációit. Az `import std.compat;` ezen felül a C fejlécek globális névtérbeli deklarációit is elérhetővé teszi – mintha a `<stdio.h>`, `<stdlib.h>`, `<string.h>` stb. is be lennének húzva, nem csak a `std::`-be exportált `<cstdio>`, `<cstdlib>`, `<cstring>` változatok. Az `std.compat` a `std` modult is exportálja, tehát nem kell mindkettőt importálni; a `std.compat` önmagában tartalmazza a `std`-et.

A különbség lényegét egy táblázat rögzíti. A táblázat állításai a P2465 szerződésén alapulnak, de a konkrét könyvtárban elérhető névhalmazt a használt implementáció dokumentációjával kell igazolni.

Név	`import std;`	`import std.compat;`
`std::vector`, `std::cout`, `std::string`	elérhető	elérhető
`std::printf`, `std::size_t`, `std::FILE`	elérhető	elérhető
`::printf`, `::size_t`, `::FILE`, `::memcpy`	nem elérhető	elérhető
`assert`, `errno`, `offsetof`, `NULL` (makró)	nem elérhető	nem elérhető
tartalmazza-e a `std` tartalmát	–	igen

A gyakorlati döntés: új kódban az `import std;` a természetes választás, mert a modern C++ kód úgyis `std::`-minősítést használ. Az `import std.compat;` átmeneti eszköz: akkor indokolt, ha egy régi, C-stílusú kódbázist (vagy harmadik féltől származó fejlécet) kell egy fordítási egységben továbbvinni anélkül, hogy a `printf`-hívásokat tömegesen `std::printf`-re írnánk át. A `std.compat` tehát migrációs híd, nem hosszú távú stílus.

44.1.3 Amit a modul nem ad: makrók és feature-test

A legtöbb meglepetés itt éri a fejlesztőt. A modulok nem exportálnak makrókat, és ez nem hiányosság, hanem a fordítási modell következménye: a preprocesszor a modulok feloldása előtt, a fordítási fázisok korábbi szakaszában fut, a makróknak pedig nincs névtere és nincs modul-hozzáférésük. Ezért az `assert`, az `errno`, az `offsetof`, a `NULL`, a `setjmp`, a `va_arg` és a `__cpp_lib_*` feature-test makrók továbbra is csak headerekből érhetők el. Az `NDEBUG` más természetű: nem könyvtári header szolgáltatja, hanem a build vagy a forrás definiálja (`-DNDEBUG`, `#define`), és a `<cassert>` `assert`-jének viselkedését vezérli; modulból tehát sem importálni, sem include-ból kapni nem lehet.

Ez a következmény a gyakorlatban azt jelenti, hogy az `import std;` mellé a makrót igénylő fejléceket külön include-olni kell:

```
import std;          // vector, string, iostream, ...
#include <cassert>   // assert makró - modulból nem jön
#include <cerrno>    // errno makró
#include <version>   // __cpp_lib_* feature-test makrók
```

A feature-test itt külön figyelmet érdemel, mert a modulok elérhetőségének vizsgálata nem történhet az `import std;` „kipróbálásával” a forrásban. A szabvány a szabványos könyvtári modulokhoz a `__cpp_lib_modules` feature-test makrót rendeli, amelyet a `<version>` headerből lehet kiolvasni. A jelenléte azt jelzi, hogy a könyvtár szállítja a modulokat; a konkrét értéket és a tényleges fordíthatóságot azonban a használt könyvtár dokumentációjával és compile probe-bal kell igazolni, mert a makró jelenléte és a modul tényleges használhatósága nem feltétlenül esik egybe minden kapcsolóbeállítás mellett.

Egy tipikus feltételes szerkezet, amely ugyanazt a forrást fordítja modullal és headerrel:

```
#include <version>

#ifdef DEMO_USE_STD_MODULE
import std;
#else
#  include <iostream>
#  include <vector>
#endif

int main()
{
    std::vector<int> v{1, 2, 3};
    std::cout << v.size() << '\n';
}
```

A minta előnye, hogy a buildkonfiguráció dönti el a módot, a forrás pedig mindkét úton értelmes marad. Hátránya, hogy a `#ifdef`-ágak eltérő szemantikájú kódot rejthetnek: a header-ágban a deklarációk a globális modul-szegmensben, az `import` ágban a modulból érkeznek, és a két ág csak akkor ekvivalens, ha valóban ugyanazokat az entitásokat használja.

44.1.4 Build: CMake, generátor és BMI

A build oldalán három fogalmat kell együtt kezelni. A CMake-ben a szabványos könyvtári modul használatát a `CMAKE_CXX_MODULE_STD` változó, illetve a `CXX_MODULE_STD` target property vezérli; ezek a 3.30 körüli kiadásban jelentek meg, és a laborprofilban a pontos minimumverziót a használt CMake dokumentációja alapján kell rögzíteni. A modulok általános kezelése ennél korábbi, a `CXX_SCAN_FOR_MODULES` property és a hozzá tartozó scan-lépés révén.

A második fogalom a generátor. A modulok fordításához olyan buildgenerátor kell, amely dinamikusan fel tudja venni a BMI-k közötti függőségeket, és a fogyasztót a BMI elkészülte után indítja. A gyakorlatban ez a Ninja; a Makefile generátor ezt a függőségi mintát nem kezeli megfelelően. A pontos Ninja-minimumverziót szintén a CMake dokumentációja rögzíti, és ellenőrizni kell.

A harmadik fogalom a BMI természete. A BMI nem hordozható artefaktum: fordítóhoz, annak verziójához és a fordítási kapcsolókhoz kötött. Ha a `-DNDEBUG`, a `-O2`, a `-std=c++23` vagy a standard library választása változik, a BMI újragenerálandó. Ez DevOps szempontból két következménnyel jár. Egyrészt a konténeres buildben érdemes a `std` modult a toolchain image részeként előre legyártani, különben minden tiszta build újra előállítja. Másrészt a gyorsítótár (ccache, sccache) csak akkor hasznos, ha a BMI-t is kezeli, és a kulcs tartalmazza a fordítót, a verziót és a releváns kapcsolókat.

Egy minimális, kétmódú CMake-projekt:

```
cmake_minimum_required(VERSION 3.30)
# CMake 3.30: az `import std` támogatása KÍSÉRLETI. A CMAKE_CXX_MODULE_STD csak
 ↪  akkor értelmes,
# ha ez a gate a project() / enable_language(CXX) ELŐTT be van állítva (a UUID
 ↪  verziónként
# változik: a 3.30-ashoz tartozó értéket a cmake-cxxmodules(7) dokumentációja
 ↪  adja).
set(CMAKE_EXPERIMENTAL_CXX_IMPORT_STD "0e5b6991-d74f-4b3d-a41c-cf096e0b2508")
project(stdmod_demo LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 23)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

option(DEMO_USE_STD_MODULE "Use import std; instead of headers" ON)

if(DEMO_USE_STD_MODULE)
  set(CMAKE_CXX_MODULE_STD ON)
endif()

add_executable(demo main.cpp)
target_compile_definitions(demo PRIVATE
  $<$<BOOL:${DEMO_USE_STD_MODULE}>:DEMO_USE_STD_MODULE>)
target_compile_options(demo PRIVATE -Wall -Wextra)
```

Támogatott környezet (3.30). Generátor: csak Ninja (1.11) vagy Visual Studio; Makefile generátor nem. Fordító+könyvtár (a CMake 3.30-as recept minimuma, nem a `lab-next` profil, amelynek Clang 22-je újabb): Clang 18.1.2 vagy újabb libc++ 18-cal (`-stdlib=libc++`, a `libc++.modules.json` megtalálható), vagy MSVC 19.36+; a GCC-s libstdc++ `import std`-jét a 3.30 még nem kezeli. A deklarált minimumverzión (3.30), tiszta build könyvtárból a modulág csak ezzel a gate-tel és ezzel a toolchain-párossal épül fel; ezt a receptet ebben a munkamenetben nem futtattuk, a kiadás előtti ellenőrzése a profilmátrix feladata. Újabb CMake-kiadásban a gate neve és állapota változhat — mindig a használt kiadás dokumentációját kell követni.

A hozzá tartozó buildparancsok, amelyek a Ninja-generátort és a két módot rögzítik:

```
cmake -S . -B build-import -G Ninja -DDEMO_USE_STD_MODULE=ON
cmake --build build-import

cmake -S . -B build-include -G Ninja -DDEMO_USE_STD_MODULE=OFF
cmake --build build-include
```

A fordítóoldali kapcsolókról csak annyit érdemes rögzíteni, amennyi a laborprofilhoz kell: GCC esetén a modulok kísérleti támogatását hagyományosan a `-fmodules-ts` jelzi, Clang esetén a `std` modult a libc++ szállítja. A konkrét kapcsolónevet, a kísérleti státuszt és a támogatott standard library párosítást minden esetben a választott fordító adott kiadásának dokumentációja rögzíti; ezek verziófüggő részletek, amelyeket nem szabad ellenőrzöttnek tekinteni.

44.1.5 #include és import keverése

A szabvány szerint a `std` modul deklarációi ugyanazokat az entitásokat írják le, mint a megfelelő headerek, így elvileg egy fordítási egységen belül a `#include <vector>` és az `import std;` együtt is konzisztens lehet. A gyakorlat azonban ettől eltér: a két út eltérő deklarációs környezetet hoz

létre, a fordítók és a build-láncok viselkedése eltérő lehet, és a hibák nehezen olvashatók (duplikált deklaráció, ODR-hez közeli tünetek, modul-feloldási hibák).

A mérnöki szabály ezért az, hogy egy fordítási egységen belül lehetőleg egy utat használjunk. Ha a keverés elkerülhetetlen – például egy harmadik féltől származó fejlécet kell behúzni –, akkor azt a globális modul-szegmensben, az `import std;` előtt tegyük meg, és dokumentáljuk, hogy ez a határ. A harmadik, tisztább lehetőség a header unit (`import <vector>;`), amely a fejlécet modulként dolgozza fel, de ez a C++20-as rész C++20 modules és header unit (33.4) anyagához tartozik, itt csak a lehetőség nevesítése a cél.

44.1.6 Fordítási idő mérése

A modulok egyik ígérete a fordítási idő csökkenése: a fejlécek újraelemzésének elmaradása miatt, különösen sok fordítási egység esetén. Ez azonban nem automatikus, és nem is mérhető egyetlen futással. A BMI betöltése, a modul feloldása és a függőségi gráf kezelése is költség; kis projektekben a modul akár lassabb is lehet, mert a BMI előállítása egyszeri többletköltség.

A mérés módszertana ezért a következő. Rögzíteni kell a gépet, a fordítót és verzióját, a könyvtárat, a teljes kapcsolósort és a buildgenerátort. Külön kell mérni a hideg (üres build könyvtár, nincs BMI) és a meleg (BMI megvan, csak a fogyasztó fordul) esetet. A fordítási időt több futás mediánjaként kell megadni, mert az első futás torzít. A méréshez használható `/usr/bin/time -v` a falidőhöz és a csúcs-memóriához, GCC esetén a `-ftime-report`, Clang esetén a `-ftime-trace` a fázisok bontásához. A saját mérőszámunk a build teljes falideje legyen, ne a fordító belső számlálója.

A mérési mátrix, amelyet a labornak ki kell töltenie – itt szándékosan üres cellákkal, mert ezt a szöveget nem futtattuk:

Változat	Konfiguráció	Hideg futás (s)	Medián, meleg, n=5 (s)	Csúcs RSS (MB)
header	`DEMO_USE_STD_MODULE=OFF`			
modul	`DEMO_USE_STD_MODULE=ON`			

Az értelmezésnél három dolgot kell kimondani. A modul előnye a fordítási egységek számával nő, mert a megtakarítás egységenként jelentkezik. A BMI előállítása egyszeri költség, amely a toolchain image-be építve „ingyenessé” tehető. És a mérés csak akkor érvényes, ha a két változat ugyanazt a forráslogikát fordítja – ezért hasznos a fenti `#ifdef`-es szerkezet.

44.1.7 Gyakori hibák

A leggyakoribb hiba, hogy a fejlesztő az `import std;` után `assert`-et vagy `errno`-t használ, és a fordítás érthetetlen módon elszáll, mert a makró nem érhető el. A javítás mindig ugyanaz: a makrót adó fejlécet külön include-olni kell, és ezt a fejlécet a modul-import elé vagy mellé tenni.

A második hiba a `std` és a `std.compat` összekeverése: a fejlesztő mindkettőt importálja, holott a `std.compat` tartalmazza a `std`-et, majd a globális `::printf` és a `std::printf` közötti különbségen akad el. A `std.compat` migrációs híd; új kódban nem indokolt.

A harmadik hiba a build oldalán jelentkezik: a `CMAKE_CXX_MODULE_STD` bekapcsolása Makefile generátorral, vagy a modul támogatásának elvárása olyan fordítóverziótól, amelynél a laborprofil nem igazolta. A tünet gyakran nem az, hogy „nem fordul”, hanem hogy a BMI nem generálódik, és a fogyasztó egy régi vagy hiányzó artefaktumot lát.

A negyedik hiba a mérés torzítása: egyetlen futás, meleg gyorsítótárral, összehasonlíthatatlan kapcsolókkal. Az így kapott szám nem mérés, hanem illusztráció, és nem alapozhat meg döntést.

Az ötödik hiba a BMI-t érinti: a fejlesztő a build cache-t változatlanul hagyja a kapcsolók módosításakor, és a cache egy nem kompatibilis BMI-t ad vissza. A javítás a cache-kulcs kiterjesztése a fordítóra, verzióra és a releváns kapcsolókra.

44.1.8 Feladatok

1. feladat (alap). Adott a fenti `main.cpp`, amely `import std;`-vel dolgozik. Egészítsd ki úgy, hogy `assert`-et használjon a vektor méretének ellenőrzésére, és magyarázd el, miért kell külön fejlécet behúzni. Add meg a pontos include-ot és a helyét.

2. feladat (középszintű). Egészítsd ki a CMake-projektet configure-time compile probe-bal, amely eldönti, hogy a `std` modul használható-e, és ennek megfelelően állítja a `CMAKE_CXX_MODULE_STD` értékét. A próbának a `__cpp_lib_modules` makrót és egy tényleges `import std;`-et is tartalmazó forrást kell fordítania. Fogalmazd meg, milyen esetben ad a próba hamis negatívot, és hogyan ellenőrizhető a döntés.

3. feladat (haladó). Tervezz mérési kísérletet a header- és a modulváltozat fordítási idejének összehasonlítására. Add meg a kontrollváltozókat, a hideg/meleg eset kezelését, a futásszámot, a statisztikát, a használt eszközöket, és azt, hogy milyen következtetés vonható le a kapott adatokból. Készítsd el a mérési jegyzőkönyv sablonját.

44.1.9 Megoldások

1. megoldás. Az `assert` makró, ezért nem érkezik a `std` modulból; a `<cassert>` fejlécet kell include-olni. A helyes forrás:

```
import std;
#include <cassert>

int main()
{
    std::vector<int> v{1, 2, 3};
    assert(v.size() == 3);
}
```

Az include helye azért az `import` után is lehet, mert a makró a preprocesszoré, nem a modulé; a lényeg, hogy az `assert` felhasználása előtt definiálva legyen. Indoklás: a preprocesszor a modul-feloldás előtt fut, a makróknak nincs modul-hozzáférésük. Ellenőrzési feltétel: a forrás `-DNDEBUG` nélkül és `-DNDEBUG` mellett is forduljon, és az utóbbi esetben az állítás ne szólaljon meg.

2. megoldás. A CMake configure-time próba vázlata:

```
# A gate (CMAKE_EXPERIMENTAL_CXX_IMPORT_STD) a project() előtt már be van
 ↪  állítva (lásd fent).
# A döntés a CMake dokumentált támogatási információján alapul, nem egy
 ↪  forráspróbán:
# a CMAKE_CXX_COMPILER_IMPORT_STD a project() után a támogatott
 ↪  szabványváltozatokat listázza.
if("23" IN_LIST CMAKE_CXX_COMPILER_IMPORT_STD)
  set(CMAKE_CXX_MODULE_STD ON)
```

```
else()
  message(STATUS "import std nem támogatott ezen a toolchain-en: header-alapú
   ↪  build")
endif()
# Ha mégis fordítási próba kell: try_compile() projekt-módban, a modult kezelő
# (CXX_MODULE_STD ON, Ninja-generátoros) tesztprojekttel, majd végrehajtott
 ↪  buildpróbával.
```

Az indoklás: a `check_cxx_source_compiles` az `import std;` működését egy olyan próbában akarná eldönteni, amelynek a modulfüggőségeit (BMI-előállítás, scan) nem a modulkezelő build-út állítja be, ezért **hamis negatívot** adhat akkor is, ha az éles build működne — napló-olvasás ezen nem segít. Helyes javítás: a CMake saját, dokumentált támogatási információját (`CMAKE_CXX_COMPILER_IMPORT_STD`) használjuk, vagy modulkezelésre konfigurált külön `try_compile` projektet, és mindkét esetben végrehajtott buildpróbát. Ismert támogatott (Clang 18 + libc++ 18, Ninja) és ismert nem támogatott (GCC 13 libstdc++) toolchainen a konfiguráció így külön kézi javítás nélkül helyesen dönt (a GCC 13-on a `"23"` nincs a listában). Ellenőrzési feltétel: a configure naplója mutassa a próba forrását és annak fordítási eredményét, és a beállított `CMAKE_CXX_MODULE_STD` érték egyezzen a tényleges build sikerességével.

3. megoldás. A kísérlet kontrollváltozói: ugyanaz a gép, ugyanaz a fordító és verzió, ugyanaz a könyvtár, ugyanaz a teljes kapcsolósor (a módot kivéve), ugyanaz a buildgenerátor és verzió, ugyanaz a forráslogika. Két esetet kell mérni: hideg (üres build könyvtár, nincs BMI) és meleg (BMI megvan). A **meleg** mérés ne legyen no-op build: ellenőrizd, hogy a mért futás ténylegesen fordít (a Ninja `-d explain`/a buildnapló mutatja a lefutott lépéseket, vagy a mért célforrás `touch`-olása után számolunk), és rögzítsd a mért lépések számát; egy „ninja: no work to do” futás nem fordítási idő. Futásszám: legalább öt, a mediánt és a szórást jelenteni kell; az első futást a hideg esetbe kell számítani, nem a melegbe. Eszközök: `/usr/bin/time -v` a falidőhöz és a csúcs-RSS-hez, GCC-nél `-ftime-report`, Clang-nál `-ftime-trace` a fázisbontáshoz. A mérőszám a build teljes falideje, nem a belső számláló. Következtetés csak a mért tartományra vonatkozóan vonható le; a modul előnye a fordítási egységek számával nő, a BMI előállítása egyszeri költség, és kis projektekben a modul lehet lassabb is. A jegyzőkönyv sablonja a fenti táblázat, kiegészítve a fordító, a könyvtár, a CMake, a Ninja és a kapcsolók rögzítésével, valamint a medián és a szórás kiszámításának módjával. Ellenőrzési feltétel: a jegyzőkönyvből egyértelműen kiderüljön, melyik szám hideg és melyik meleg futásból származik, és a két változat kapcsolósora csak a módban térjen el.

44.1.10 Összefoglaló

Az `import std;` a szabványos könyvtár deklarációit teszi elérhetővé a `std` névtérben, modul-interfészen keresztül, a P2465 szerződése szerint. Az `import std.compat;` ugyanezt adja, kiegészítve a C fejlécek globális névtérbeli deklarációival; ez migrációs eszköz, nem új kód stílusa. A modulok nem exportálnak makrókat, ezért az `assert`, az `errno`, az `offsetof` és a feature-test makrók továbbra is headerekből jönnek; a `__cpp_lib_modules` a `<version>`-ből olvasható, de a tényleges támogatást compile probe-bal kell igazolni. A build oldalán a `CMAKE_CXX_MODULE_STD` és a `CXX_MODULE_STD` property, valamint a Ninja-generátor a feltétel; a BMI fordító-, verzió- és kapcsolófüggő, ezért a toolchain image és a gyorsítótár tudatos kezelést igényel. A kevert `#include`/`import` használat kerülendő, a fordítási idő pedig csak kontrollált, hideg/meleg esetet megkülönböztető méréssel értékelhető.

44.2 std::generator mélyebben: rekurzió, allokáció és ranges

A Generator és streaming feldolgozás (35.4) részben a `std::generator` alapvető használatát már rögzítettük: a coroutine felfüggeszthető, a fogyasztó pedig egy input range-en keresztül kéri le az elemeket. Itt a teljesebb típus- és élettartam-modellt vesszük elő: hogyan függ a `generator<Ref, V, Allocator>` viselkedése a sablonparaméterektől, mikor biztonságos a referencia szerinti yieldelés, hogyan delegálunk rekurzívan `ranges::elements_of`-fal, és milyen korlátokat jelent a ranges pipeline-okban, hogy a generátor input range.

A tárgyalás feltételezi a promise_type, awaiter és coroutine_handle (35.2) és a Kombináló view-k: zip, adjacent, enumerate, cartesian_product (42.2) ismeretét. A C++20-as coroutine-alapokat nem tanítjuk újra; a hangsúly a C++23-as `std::generator` szerződésén és a gyakorlati hibamódokon van. A példák C++23-at igényelnek; a konkrét fordító- és könyvtártámogatást nem tekintjük automatikusan adottnak, ezért feature-test makrót és compile probe-ot használunk.

44.2.1 A generator típusparaméterei és szerződése

A fejlécben deklarált típus alakja:

```
template<class Ref, class V = void, class Allocator = void>
class generator;
```

A paraméterek szerepe:

Paraméter	Szerep	Tipikus választás
`Ref`	ebből képződik a `reference` típus (lásd lent)	`const T&`, `T&`, `T`
`V`	a generátor `value_type`-ja; ha `void`, a `Ref`-ből `remove_cvref_t`-tel képződik	legtöbbször elhagyható
`Allocator`	a coroutine frame allokátora; `void` esetén a könyvtár alapértelmezett allokátora	speciális allokációs politika esetén

A `std::generator` move-only, `view` és `input_range`. Az `iterator` input iterator, a lezáró pedig `default_sentinel_t`; a bejárás single-pass, azaz a sorozatot nem lehet visszatekerés nélkül, párhuzamosan vagy több iterátorral újraolvasni. A `begin()` indítja a coroutine-t, az `end()` pedig azt jelzi, hogy a coroutine `co_return`-nal vagy a függvény végére érve befejeződött. A `generator` nem tárolja el az összes elemet: a coroutine frame-ben felfüggesztett állapot termeli őket igény szerint.

A `Ref` megválasztása nem esztétikai kérdés, hanem tulajdonlási döntés. A három származtatott típus képlete (C++23 [coro.generator.class]); a két utolsó oszlop a `generator<std::string>` és a `generator<const std::string&>` esete (a `V` mindkettőnél `void`):

Típus	Képlet	`Ref` = `std::string`	`Ref` = `const std::string&`
`value`	`V` void esetén `remove_cvref_t<Ref>`, különben `V`	`std::string`	`std::string`

Típus	Képlet	`Ref = std::string`	`Ref = const std::string&`
`reference`	`V` void esetén `Ref&&`, különben `Ref`	`std::string&&`	`const std::string&`
`yielded`	ha `reference` referencia, akkor az, különben `const reference&`	`std::string&&`	`const std::string&`

Az iterátor `operator*`-a `reference`-t ad ([coro.generator.iterator]): a coroutine frame-ben a felfüggesztés idejére megtartott objektumra mutató hivatkozást. Ez `generator<T>` esetén is így van: a `*it` eredménye `T&&`, **nem** másolat. A `V` akkor kap szerepet, ha a `value_type` nem vezethető le egyszerűen a `Ref`-ből; a legtöbb gyakorlati esetben elég a `Ref`-et megadni.

A C++23-as elérhetőséget a `<generator>` include és a `__cpp_lib_generator` feature-test makró jelzi. A következő fordítószelet ezt ellenőrzi; a makró pontos értékét a használt standard könyvtár dokumentációja rögzíti, ezért azt ne kezeld örökre rögzített konstansként:

```
#include <version>
#if !defined(__cpp_lib_generator)
#error "A std::generator nem erheto el ebben a standard konyvtarban"
#endif
```

44.2.2 Yieldelt referencia és érték élettartama

A legfontosabb szabály: sem a `co_yield`, sem a dereferálás nem másol automatikusan; az, hogy a fogyasztónál másolat keletkezik-e, a fogyasztó változójának deklarációján múlik. A `for (std::string x : g)` minden elemből saját `x` objektumot konstruál: `generator<std::string>`-nél a `std::string&&`-ből mozgatással, `generator<const std::string&>`-nél másolással. Ez az `x` a továbblépés után is érvényes. A `for (auto&& x : g)` és a `const auto& r = *it` viszont csak hivatkozik az aktuálisan yieldelt objektumra, és ez a hivatkozás a következő `++it` (a coroutine folytatása) után nem használható, mindkét generátorfajtánál.

A coroutine frame élettartama segít megérteni a látszólag meglepő eseteket. A coroutine lokális változói nem a natív stacken, hanem a frame-ben élnek, és a frame addig marad életben, amíg a generátor le nem fut vagy meg nem semmisül. Ezért az alábbi kód első ránézésre ijesztő, de a generátor élettartamán belül helyes lehet:

```
std::generator<const std::string&> local_owner() {
    std::string s = "hello";
    co_yield s;          // a frame-ben élő s-re hivatkozik
}
```

A hivatkozás addig érvényes, amíg a coroutine fel nem ébred és be nem fejeződik. Ha a fogyasztó továbblépteti az iterátort, a coroutine eléri a függvény végét, a frame lokalisa, köztük `s`, megsemmisül; a korábban eltárolt referencia ekkor dangling lesz. Ezért a `generator<const T&>` tipikus használata nem önmagában a „lokális változó”, hanem az, hogy a hivatkozott objektum tulajdonosa egyértelműen túlél.

A `generator<T>` és a `generator<const T&>` különbsége tehát nem az, hogy az egyik másolatot ad a dereferáláskor, hanem az, hogy ki tartja életben a yieldelt objektumot. A `generator<T>`-nél a yieldelt érték a coroutine frame-ben él a felfüggesztés alatt, így a forrásobjektumtól független. A `generator<const T&>`-nél a generátor csak a hivatkozást adja tovább, a hivatkozott objektum

élettartamáért a yieldelő kód felel. Mindkettőnél igaz: a dereferálással kapott hivatkozás csak az aktuális felfüggesztésig érvényes, aki tovább akarja tartani, annak saját értéket kell deklarálnia. A `std::string_view` például önmagában olcsó, de a mögötte lévő karaktertömb élettartamát nem hosszabbítja meg.

Egy temporális közvetlen yieldelése `generator<const T&>`-be önmagában **nem** hiba:

```
std::generator<const std::string&> temp() {
    co_yield std::string{"temporary"};   // a temporális a co_yield teljes
}                                        // kifejezésének végéig él

void consume() {
    auto g = temp();
    auto it = g.begin();
    std::string copy = *it;              // helyes: a felfüggesztés alatt
     ↪  olvasunk
    const std::string& kept = *it;       // csak a következő ++it-ig érvényes
    ++it;                                // a coroutine folytatódik: a teljes
                                         // kifejezés véget ér, a temporális
                                          ↪  megszűnik
    // kept itt már dangling; copy továbbra is érvényes
}
```

A temporális élettartama a `co_yield`-et tartalmazó teljes kifejezéshez kötődik. A coroutine ennek a kifejezésnek a kiértékelése közben függeszti fel magát, így a fogyasztó a felfüggesztés alatt biztonságosan olvashatja a temporálist. A teljes kifejezés a folytatáskor, a következő `++it` hatására ér véget. A valódi veszély tehát az, ha a fogyasztó a hivatkozást a továbblépés után is megtartja, vagy ha a yieldelt hivatkozás egy a coroutine-nál rövidebb élettartamú külső objektumra mutat. Ilyenkor a fogyasztónak saját értéket kell deklarálnia, a generátornak pedig `generator<std::string>`-re vagy stabil tulajdonosra kell váltania. Hasonló a helyzet akkor, ha egy vektorra hivatkozó generátort készítünk, majd a vektort módosítjuk: a `push_back` vagy `reserve` érvényteleníthet i az elemekre adott referenciákat, és a generátor minden további dereferálása hibás lehet.

Ellenőrzési megjegyzés. A `lab-gcc` profil GCC 13.3-as libstdc++-ában nincs `<generator>` (`__cpp_lib_generator` nincs definiálva), ezért a fejezet `std::generator`-os példáit ott nem fordítottuk le; a `lab-next` profilban ellenőrizendők. A fenti típusképletek és az élettartam-szabályok forrása az N4950 [coro.generator.class] és [coro.generator.iterator] szakasza.

44.2.3 Rekurzív generátor ranges::elements_of-fal

A `std::ranges::elements_of` egy C++23-as delegáló eszköz. A

```
co_yield std::ranges::elements_of(range);
```

utasítás azt jelenti, hogy a generátor nem egyetlen elemet, hanem a megadott range összes elemét yieldeli, mégpedig a range bejárásával. Ez különösen rekurzív generátoroknál hasznos: a szülő generátor egy gyermekgenerátorra delegálhat anélkül, hogy kézzel írnánk egy belső `for` ciklust.

Tekintsünk egy fájlstruktúrát leíró fát. A preorder bejárás rekurzív generátorral így néz ki:

```
#include <version>
#if !defined(__cpp_lib_generator)
#error "A std::generator nem erheto el ebben a standard konyvtarban"
#endif
```

```cpp
#include <generator>
#include <ranges>
#include <string>
#include <vector>
#include <iostream>

struct Node {
    std::string name;
    std::vector<Node> children;
};

std::generator<const Node&> walk(const Node& n) {
    co_yield n;
    for (const auto& child : n.children) {
        co_yield std::ranges::elements_of(walk(child));
    }
}

int main() {
    Node root{"root", {
        {"etc", {{"nginx.conf", {}}}},
        {"var", {{"log", {{"app.log", {}}}}}}
    }};

    auto names = walk(root)
        | std::views::filter([](const Node& n) { return !n.name.empty(); })
        | std::views::transform([](const Node& n) -> const std::string& {
              return n.name;
          });

    for (const std::string& name : names) {
        std::cout << name << '\n';
    }
}
```

A példa preorder sorrendben adja a csomópontokat. A `walk(child)` egy ideiglenes `generator<const Node&>`, amelyet az `elements_of` átvesz, és a szülő generátor a gyermek elemeit delegálva yieldeli. A `Ref = const Node&` választás azért megfelelő, mert a fa a bejárás teljes ideje alatt él; a generátor nem másolja a csomópontokat.

A rekurzió mélysége nem feltétlenül a natív hívási verem mélysége. A szülő coroutine a delegálás alatt felfüggesztve marad, a gyermek frame-je pedig a heapen lévő coroutine-lánc része. A fogyasztó léptetése a láncot járja be; a frame-ek élettartama a generátor-lánc élettartamához kötődik. Ez fontos különbség a hagyományos rekurzív függvényhíváshoz képest, ahol a mélység a stacket fogyasztja. A pontos garanciát az `elements_of` szabványos szemantikája és a könyvtár implementációja adja; a stack- és allokációs viselkedést a laborban külön kell mérni.

Bizonyításvázlat (vázlat). Tegyük fel, hogy a fogyasztó egy elem kérésével léptet. A szülő generátor `co_yield elements_of(walk(child))` pontján felfüggesztett, a gyermek pedig a delegált range-et szolgálja ki. A fogyasztó a gyermek aktuális elemét kapja, a szülő frame nem aktiválódik újra, amíg a gyermek ki nem merül. Így a bejárás mélysége a felfüggesztett frame-ek láncában,

nem a hívási veremben jelenik meg. Ez a vázlat a delegálás szerkezetére támaszkodik; a tényleges memóriahasználatot és a mély rekurzió korlátait mérni kell.

A `std::ranges::elements_of` opcionális allokátorargumentumot is elfogadhat, amely a delegált generátor frame-jének allokációs politikájához kapcsolódhat. Ennek pontos támogatása és típusillesztése könyvtárfüggő; csak akkor használd, ha a fogadó generátor promise-típusa ezt kifejezetten támogatja, és a laborprofilban ellenőrizted.

44.2.4 Allokáció, frame és költségek

A `generator<Ref, V, Allocator>` harmadik paramétere a coroutine frame allokátora. A coroutine frame tartalmazza a promise-t, a paramétereket, a lokális változókat és a yieldelt érték tárolását. A frame a coroutine első hívásakor jön létre, és a generátor megsemmisülésekor szabadul fel. A `generator` tehát nem allokál külön tárolót az elemek számára; a „tároló" a coroutine állapota, amelynek mérete a coroutine lokálisaitól és a yieldelt érték típusától függ.

Az `Allocator = void` alapértelmezés a könyvtár alapallokátorát használja. Egyedi allokátorral például arena- vagy pool-alapú coroutine-frame kezelés valósítható meg, de az allokátor típusa a generátor típusának része lesz, ami a pipeline-okban és a függvényszignatúrákban megjelenik. A gyakorlatban először azt érdemes eldönteni, hogy a generátor rövid életű-e, és a frame-allokáció mérhető költséget jelent-e; csak utána érdemes egyedi allokátorhoz nyúlni.

A rekurzív `elements_of` használatakor minden delegált generátor saját frame-mel rendelkezhet. Ez kényelmes, de mély fáknál sok kicsi allokációt jelenthet. A `generator<T>` érték szerinti yieldelése további másolási költséget adhat; a `generator<const T&>` referenciával dolgozik, de szigorúbb élettartamszerződést követel. A választás tehát nem csak nyelvi, hanem költség- és biztonsági döntés.

44.2.5 Ranges pipeline és input range korlátok

A `std::generator` `view`, ezért ranges pipeline-ba illeszthető. A szűrő, az átalakító, a levágó és az összefűző adapter (`views::filter`, `views::transform`, `views::take`, `views::join`) jellemzően működik vele, mert csak input range-et igényelnek. A `views::reverse`, a `ranges::sort` vagy bármely random access / bidirectional művelet viszont nem, mert a generátor nem teszi lehetővé a visszafelé lépkedést és a többszöri bejárást.

```
auto first_three = walk(root)
    | std::views::filter([](const Node& n) { return n.name.starts_with("a"); })
    | std::views::take(3);
```

A `take(3)` korai leállást eredményez: a generátor nem feltétlenül fut a végéig. Amikor a range-for vagy a `take` abbahagyja a fogyasztást, a generátor megsemmisül, a coroutine frame felszabadul, és a frame-ben élő lokális objektumok destruktorai lefutnak. Ez a korai leállás nem külön „cancel" hívás, hanem a generátor élettartamának vége.

Ha a teljes sorozatra többször vagy véletlen hozzáféréssel van szükség, materializálni kell. C++23-ban erre a `std::ranges::to` szolgálhat, de a könyvtártámogatást feature-testtel kell ellenőrizni; fallbackként egy `std::vector` feltöltése is megfelel:

```
std::vector<const Node*> nodes;
for (const Node& n : walk(root)) {
    nodes.push_back(&n);
}
std::ranges::reverse(nodes);
```

Itt a `nodes` a csomópontokra mutató pointereket tárolja; a fa élettartama továbbra is a hívó felelőssége. A pointerek érvényessége akkor szűnik meg, ha a fa módosul és a csomópontok megsemmisülnek.

44.2.6 Kivétel és korai leállás

Ha a coroutine teste kivételt dob, a generátor azt a fogyasztó felé továbbítja a következő léptetés során. A kivétel nem tűnik el csendben, és nem lesz belőle „csak egy hiányzó elem”. A fogyasztónak a bejárást `try/catch` blokkban kell végeznie, ha a generátor teste dobhat:

```
std::generator<int> risky() {
    co_yield 1;
    throw std::runtime_error("boom");
}

try {
    for (int x : risky()) {
        std::cout << x << '\n';
    }
} catch (const std::runtime_error& e) {
    std::cerr << "generator hiba: " << e.what() << '\n';
}
```

A kivétel általában a léptetéskor, az `operator++` hívásakor jelenik meg, mert a coroutine ekkor folytatódik. A generátor destruktora ekkor is felszabadítja a frame-et. Ha a fogyasztó `break`-kel áll le, a coroutine nem kap külön értesítést; a frame destruktora takarít. Ezért a generátor belsejében használt RAII-erőforrások (fájlleíró, lock, allokált buffer) akkor is felszabadulnak, ha a fogyasztó korán kilép, feltéve, hogy a coroutine frame destruktora lefut.

Gyakori hibák:

- `generator<const T&>`-be lokális vagy temporális objektumot yieldelni, majd a generátor léptetése után használni a referenciát.
- A generátort többször, párhuzamosan bejárni, vagy `views::reverse`-vel, `ranges::sort`-tal használni.
- A `generator<T>` érték szerinti yieldelését „ingyenesnek” tekinteni nagy objektumoknál.
- Az `elements_of`-nak átadott range élettartamát nem ellenőrizni; a delegált range-nek a delegálás idejéig érvényesnek kell lennie.
- A coroutine frame allokációs hibáját figyelmen kívül hagyni egyedi allokátor használatakor.

44.2.7 Feladatok

44.2.7.1 1. feladat: Referencia vagy érték?

Adott egy `std::string_view input`-ot feldolgozó függvény, amely a bemenet szeleteit szeretné yieldelni. Válaszd ki a helyes generátortípust: `std::generator<std::string_view>` vagy `std::generator<std::string_view&>`? Indokold a választ, és nevezd meg, kinek kell életben tartania a bemeneti karaktereket.

44.2.7.2 2. feladat: Rekurzív fa bejárása

Írj `std::generator<const Node&> walk(const Node&)` függvényt, amely preorder sorrendben adja a csomópontokat, és a gyermekekre `std::ranges::elements_of`-fal delegál. Egészítsd ki egy pipeline-nal, amely csak a `log` szót tartalmazó neveket adja vissza, majd `take(2)`-vel korlátozd.

44.2.7.3 3. feladat: Élettartam- és pipeline-diagnosztika

Az alábbi kódban két problémát kell találnod: egy élettartamhibát és egy ranges-korlátot.

```
std::generator<const std::string&> bad() {
    std::string s = "hello";
    co_yield s;
}

auto g = bad();
auto it = g.begin();
const std::string& r = *it;
++it;
std::cout << r << '\n'; // ?

auto reversed = bad() | std::views::reverse; // ?
```

Magyarázd el, miért hibás a `r` használata a léptetés után, és miért nem fordul a `views::reverse`. Adj javítást mindkettőre.

44.2.8 Megoldások

44.2.8.1 1. megoldás

A helyes választás a `std::generator<std::string_view>`. A `std::string_view` kis, olcsón másolható nézet; érték szerinti yieldelése nem okoz jelentős másolási költséget, és elkerüli, hogy egy lokális `string_view` objektumra mutató referenciát adjunk ki. A `std::generator<std::string_view&>` esetében a generátor egy `string_view` objektumra hivatkozó referenciát adna; ha ez a `string_view` a coroutine frame lokálisa vagy egy temporális, a fogyasztó a felfüggesztés után dangling referenciát kaphat.

A mögöttes karaktertömb tulajdonosa továbbra is a hívó: a `string_view` nem hosszabbítja meg a bemenet élettartamát. A generátort csak addig szabad fogyasztani, amíg az `input` érvényes. Ellenőrzési feltétel: a generátor típusa `std::generator<std::string_view>`, a bemenet élettartama a bejárás teljes idejét lefedi, és a kód nem tárol a `string_view`-nál hosszabb életű referenciát a bemenetre.

44.2.8.2 2. megoldás

```
std::generator<const Node&> walk(const Node& n) {
    co_yield n;
    for (const auto& child : n.children) {
        co_yield std::ranges::elements_of(walk(child));
    }
}
```

```
auto log_names = walk(root)
    | std::views::filter([](const Node& n) {
          return n.name.find("log") != std::string::npos;
      })
    | std::views::take(2);
```

A `walk` preorder sorrendben adja a csomópontokat. A `elements_of` a gyermekgenerátor elemeit delegálja, így nem kell kézi belső ciklust írni. A `filter` és a `take` input range-en működik; a `take(2)` korai leállást okoz, ami a generátor destruktorán keresztül takarít. Ellenőrzési feltétel: a kimenet preorder sorrendű, a `log` szűrés helyes, és a `take(2)` után a generátor nem fut tovább. A `root` élettartama a bejárás teljes idejét fedje le.

44.2.8.3 3. megoldás

A `bad()` coroutine frame-jében élő `s` addig érvényes, amíg a coroutine fel nem ébred. Az `it` léptetésekor a coroutine eléri a függvény végét, `s` megsemmisül; a korábban eltárolt `r` ekkor dangling referenciává válik. Az `std::cout << r` viselkedése nem definiált. A javítás vagy `std::generator<std::string>` érték szerinti yieldelés, vagy olyan tulajdonos biztosítása, amely a referencia használatáig él.

A `views::reverse` azért nem fordul, mert a `std::generator` csak input range: nincs bidirectional iterátora, nem lehet visszafelé lépkedni. A javítás az, hogy előbb materializáljuk a sorozatot egy `std::vector`-ba, és azon végezzük a fordítást:

```
std::vector<std::string> values;
for (const std::string& s : bad()) {
    values.push_back(s);
}
std::ranges::reverse(values);
```

Ellenőrzési feltétel: a módosított generátor nem ad ki a frame lokalisa után használható referenciát; a reverse-t csak materializált, random access tárolón használjuk. A fordító által jelzett „no matching function” hiba a reverse-nél várható, és nem javítható `views::reverse`-vel közvetlenül a generátoron.

44.2.9 Összefoglaló

A `std::generator<Ref, V, Allocator>` legfontosabb döntése a `Ref`: érték szerinti yieldelés biztonságosabb, referencia szerinti yieldelés olcsóbb, de a hivatkozott objektum élettartamáért a hívó felel. A coroutine frame a generátor élettartamához kötött; a `co_yield` utáni léptetés vagy a generátor megsemmisülése lefuttatja a frame lokális destruktorait. A `ranges::elements_of` rekurzív delegálást tesz lehetővé, és a felfüggesztett frame-ek láncát használja a natív rekurzió helyett. A generátor input range: pipeline-ban `filter`, `transform`, `take` használható, de reverse, sort vagy többszöri bejárás nem. Kivétel esetén a hiba a fogyasztó felé továbbítódik; korai `break` esetén a generátor destruktora takarít.

44.3 Elavult, eltávolított és viselkedést váltó elemek C++23-ban

Az alábbi rész abban segít, hogy a C++20-ról C++23-ra váltás ne látens regresszióként, hanem ellenőrzött migrációként történjen. A C++23 háromféle módon érinti a meglévő kódot: egyes elemeket **eltávolít**, másokat **elavultnak jelöl**, és van, ahol a forrás változatlan marad, de a **szemantika módosul**. Az első kettőt a fordító többnyire kiszűri, a harmadikat nem: azt a build- és tesztmátrixnak kell kimutatnia.

A rész a C++23 laborprofilra épül: a laborprofil rögzíti a fordítót, a libstdc++/libc++ kiadást és a `-std=c++23` kapcsolót. Az implicit move és a range-for élettartam részletes nyelvi levezetését itt nem ismétlem meg, csak a migrációs következményeit tárgyalom; a szemantikát a korábbi rész tartalmazza. Minden itt szereplő fordítási parancs a tanuló saját laborjában futtatandó; a szöveg nem állítja, hogy bármely eredményt mértem.

44.3.1 A változás három kategóriája és a velük járó kockázat

Eltávolított (removed) elem: a C++23 szövege már nem tartalmazza a deklarációt, tehát a rá épülő kód C++23-ban **már nem szabványos** — megszűnik a hordozhatósági garancia. Az nem garantált, hogy minden fordító elutasítja: egy implementáció kompatibilitási kiterjesztésként tovább biztosíthatja a régi felületet, és ez nem sérti a szabványt. A konkrét diagnosztika a fordító és a könyvtár kiadásán múlik, ezért compile-fail tesztet csak ellenőrzött profilhoz kössünk, a migrációs audit pedig a forrásban is keresse az eltávolított nevek használatát. **Mérés** (`std::declare_reachable`, a C++23-ban eltávolított szemétgyűjtő-támogatás része): a GCC 13.3 `-std=c++20` alatt elfogadta, `-std=c++23` alatt `'declare_reachable' is not a member of 'std'` hibát adott; a Clang 22 / libc++ `-std=c++23` alatt szintén elutasította. Ha a fordító elutasítja, ez a legkevésbé kockázatos eset, mert nem lehet figyelmen kívül hagyni. **Elavult (deprecated) elem**: továbbra is a szabvány része, de a jövőbeli eltávolítás jelöltje. Fontos részlet: az elavulás a *szabvány* nyelvén fogalmazódik meg, nem feltétlenül jár `[[deprecated]]` attribútummal, így a könyvtári implementáció dönti el, ad-e diagnosztikát, és ezt makróval ki is kapcsolhatja. Ezért az elavult elemekre nem lehet vakon `-Werror`-t építeni. **Viselkedésváltozás (behavior change)**: a kód fordul mindkét módban, de mást jelent – más overloadot választ a túlterhelés-feloldás, máskor pusztul el egy temporary, más diagnosztikát ad a preprocesszor. Ez az a kategória, ahol a csendes regresszió él, és ahol a kétmódú összehasonlító teszt a legjobb eszköz.

Végül külön kell kezelni a **DR-t (defect report)**: a visszamenőleg is alkalmazható javítást a fordító a korábbi nyelvi módban is implementálhatja, így a „C++20-as” viselkedés nem garantált. Ezért a migrációs mérés mindig a rögzített laborprofilban, dokumentált fordítóverzióval érvényes.

44.3.2 Eltávolított elemek: a garbage collection támogatás (P2186)

A C++ korábbi kiadásai tartalmaztak egy szemétgyűjtés-támogató felületet a `<memory>` fejlécben: `std::declare_reachable`, `std::undeclare_reachable`, `std::declare_no_pointers`, `std::undeclare_no_pointers`, valamint a `std::pointer_safety` típus és a `std::get_pointer_safety()` függvény. A P2186 alapján ezek kikerültek a szabványból. A felület gyakorlati problémája az volt, hogy nem lehetett hordozhatóan implementálni, és a valós programok nem használták: a `declare_reachable` lényegében egy „a gyűjtő ne bántsa ezt a pointert” jelzés volt, ami csak együttműködő, úgynevezett *conservative* gyűjtő mellett értelmezhető.

A tipikus törő minta, amelyet tanítási célból külön jelölök – **ez a példa szándékosan nem fordítható C++23 módban**:

```
// tanítási példa: C++23-ban nem fordul
#include <memory>
void f() {
    int* p = new int(42);
    std::declare_reachable(p);   // C++23: a név nem létezik
    // ... munka a pointerrel ...
    std::undeclare_reachable(p);
    delete p;
}
```

A javítás nem átírás, hanem törlés: a hívások elhagyhatók, a tulajdonlás pedig marad explicit `new`/`delete`, vagy – helyesebben – `std::unique_ptr`. A keresés CI-ban egyszerű szövegkereséssel is elvégezhető a `declare_reachable|declare_no_pointers|pointer_safety` mintára, de a döntő kapu a C++23-as build maga.

44.3.3 Elavult elemek: `aligned_storage`, `aligned_union` és `has_denorm`

A P1413 a `std::aligned_storage` és a `std::aligned_union` típusokat jelöli elavultnak. Ezek soha nem voltak teljes értékű eszközök: egy nyers, igazított byte-tömböt adtak, amelybe placement new-val kellett objektumot tenni, és a típus-punning körüli szabályok (effective type, laundering) miatt könnyű volt velük nem definiált viselkedést előállítani. A javasolt irány az `alignas` közvetlen használata, illetve a `std::optional<T>`, ha a cél egy „esetleg létező T”.

A `std::numeric_limits<T>::has_denorm` elavulása ugyanebbe a családba tartozik: olyan tulajdonságot kérdezett, amely a gyakorlatban vagy mindig jelen volt, vagy a lekérdezés nem adott cselekvésre váltható információt, és a válasz erősen implementációfüggő. Aki mégis a denormális viselkedésre akar építeni, az a `denorm_min()` és a `min()` viszonyát vizsgálja, illetve a célplatform dokumentált lebegőpontos modelljére támaszkodik; ezt a laborprofilban kell rögzíteni, mert a szabványos lebegőpontos szemantika nem írja elő a denormálisok hardveres támogatását.

Az `aligned_storage` kiváltására egy önállóan fordítható példa:

```
// build: g++ -std=c++23 -Wall -Wextra -Wdeprecated -pedantic -o storage
 ↪  storage.cpp
#include <cstddef>
#include <cstdio>
#include <new>
#include <type_traits>
#include <utility>

template <class T>
class InlineStorage {
public:
    // A sablon ne legyen jelölt egyetlen saját típusú argumentumra: különben
    // egy nem const lvalue-ból a törölt másoló helyett ez nyerne.
    template <class... Args>
        requires (!(sizeof...(Args) == 1 &&
                    (std::is_same_v<std::remove_cvref_t<Args>, InlineStorage> &&
                     ↪  ...)))
              && std::is_constructible_v<T, Args...>
    explicit InlineStorage(Args&&... args) {
        ::new (static_cast<void*>(buf_)) T(std::forward<Args>(args)...);
    }
```

```
    ~InlineStorage() { get()->~T(); }

    // Az implicit másolás/mozgatás a buf_ BÁJTJAIT másolná, nem a T objektumot:
    // a két példány ugyanazt az erőforrást birtokolná (std::string esetén
    // kétszeres felszabadítás). A minimális owner ezért nem másolható.
    InlineStorage(const InlineStorage&) = delete;
    InlineStorage& operator=(const InlineStorage&) = delete;
    T* get() { return std::launder(reinterpret_cast<T*>(buf_)); }
private:
    alignas(T) std::byte buf_[sizeof(T)];
};

struct Point { int x, y; };

int main() {
    InlineStorage<Point> s(1, 2);
    std::printf("%d %d\n", s.get()->x, s.get()->y);
}
```

Az owner itt az `InlineStorage` objektum, a tároló a `buf_` tomb, a `get()` nem tulajdonló hozzáférés, az invalidáló művelet pedig a destruktor (utána a pointer dangling). A `std::launder` azért kell, mert a `reinterpret_cast` által adott pointer eredetileg `std::byte`-ra mutatott. Ha nem kell kézi élettartam-kezelés, a `std::optional<Point>` rövidebb és biztonságosabb.

A sablon szerződése nem csak a triviális `Point`-ra kell, hogy helyes legyen. Az implicit másoló és mozgató művelet a `buf_` tömböt bájtonként másolná, nem a benne élő `T` objektumot konstruálná. `InlineStorage<std::string>` esetén a két példány ugyanazt a heap-puffert birtokolná, és a második destruktor kétszeres felszabadítást okozna. A minimális owner ezért nem másolható és nem mozgatható: a másoló műveletek törölve vannak, a mozgatók pedig így nincsenek deklarálva, és a mozgatás is a törölt másolóra esik. (A másik lehetőség a `T` valódi másoló/mozgató konstrukciója a célpufferben, a megfelelő kivételgaranciával; ez már egy `std::optional`-szerű típus újraírása.) A variadikus konstruktor-sablont korlátozni kell, különben egy nem `const` lvalue-ból történő direkt inicializálásnál (`InlineStorage<std::string> b(a);`) jobb jelölt lenne a törölt másolónál.

Buildjegyzőkönyv (`lab-gcc` profil, `g++ -std=c++23 -Wall -Wextra -Wdeprecated -pedantic`). A program figyelmeztetés nélkül fordult, kimenete `1 2`. Az `InlineStorage<std::string> b = a;` és a `b(a)` alak, `std::any` elemtípussal is, `use of deleted function` hibát ad. Egy `std::string`-et tároló példány és egy élő objektumokat számláló típus ASan/UBSan alatt hibátlanul fut, a számláló a végén 0. A javítás előtti változatban az `InlineStorage<std::string>` másolata ASan alatt `attempting double-free` hibát adott.

44.3.4 Viselkedést váltó elemek: implicit move és range-for

Az implicit move C++23-as egyszerűsítése (P2266, a Érték, élettartam és kifejezések: auto(x), implicit move és range-for (40.5) részben tárgyalt módon) migrációs szempontból elsősorban **fordíthatósági** változás: a `return` utasításban megnevezett implicit mozgatható entitás xvalue lesz, ezért az `int&& fwd(int&& x) { return x; }` C++23-ban fordul, a lokálisra lvalue-referenciát visszaadó kód pedig nem. Az érték szerinti *paraméter* visszaadása viszont nem változik: az implicit move C++11 óta kiterjed rá. Ezt is érdemes kétmódú teszttel rögzíteni, mert a „C++23-ban más konstruktor fut” feltételezés gyakori tévedés:

```
// passthrough.cpp - build: g++ -std=c++20 ... ; majd ugyanez -std=c++23 ...
#include <cstdio>
```

```
struct S {
    int copies = 0, moves = 0;
    S() = default;
    S(const S& o) : copies(o.copies + 1), moves(o.moves) {}
    S(S&& o) : copies(o.copies), moves(o.moves + 1) {}
};
S passthrough(S s) { return s; }   // érték szerinti paraméter: implicit move
 ↪ (C++11 óta)
int main() {
    S b = passthrough(S{});        // az argumentum inicializálása elided
    std::printf("copies=%d moves=%d\n", b.copies, b.moves);
}
```

Buildjegyzőkönyv (`lab-gcc` profil). A kimenet `-std=c++11`, `-std=c++17`, `-std=c++20` és `-std=c++23` módban egyaránt `copies=0 moves=1`: a paraméter mindegyik módban mozog, másolás egyikben sincs. A mérési eljárás általános: azonos forrás, két `-std` mód; ha a kimenet eltér, a különbséget dokumentáljuk. A valódi C++23-as törést (fordul / nem fordul) a Érték, élettartam és kifejezések: auto(x), implicit move és range-for (40.5) mutatja be.

A range-for élettartam-szigorítása (C++23) a `for (auto e : kif().tag())` alakot érinti: korábban csak a legfelső temporary élettartama hosszabbodott meg, a köztes temporary a deklaráció végén elpusztult, és a ciklusmag dangling referencián futott – azaz nem definiált viselkedés. C++23-ban a range-init minden temporaryje a ciklus végéig él — a függvényparaméter-objektumok azonban nem temporaryk, ezért egy paraméterre visszaadott referencia továbbra is lógó (lásd 40.5). Csendes változás: a korábban UB-t produkáló kód most definiáltan működik, és a destruktorok sorrendje is megváltozik. Migrációs szabály: a korábbi „véletlenül működött” kódot nem szabad megtartani mintaértékűnek; a helyes kód tulajdonost és élettartamot nevez meg, és nem épít a temporary pusztulásának pillanatára.

44.3.5 Vegyes szélességű string literal összefűzés

A C++23 szigorítása miatt a különböző típusú string literalek összefűzése – például `L"id: " "42"` – **ill-formed** lett; korábban feltételesen támogatott, implementációfüggő jelentésű konstrukció volt. Ez jó hír: fordítási hiba, nem csendes viselkedésváltozás. A tipikus örökség a `_T()`/`TCHAR` makróvilág és a kézzel vegyített `char`/`wchar_t` üzenetek. A javítás: egységes literalprefix (a forrás és a feldolgozó lánc egy kódolásán), vagy explicit konverziós függvény a határon; a `u8` prefixet használó kódnál figyelni kell, hogy a `u8` literal típusa `char8_t` tömb, nem `char`.

44.3.6 Migrációs ellenőrzőlista

Ellenőrzés	Keresési minta	Eszköz	Következmény, ha talál
GC-felület	`declare_reachable`, `declare_no_pointers`, `pointer_safety`	C++23 build, grep	hard error; hívások törlése
Igazított tároló	`aligned_storage`, `aligned_union`	C++23 build + `-Wdeprecated`	elavulási diagnosztika; `alignas`/`optional`
Denormális lekérdezés	`has_denorm`, `denorm_style`	grep + `-Wdeprecated`	elavulás; `denorm_min()`/platformdok.

Ellenőrzés	Keresési minta	Eszköz	Következmény, ha talál
Visszaadott paraméter	`return` + by-value paraméter	kétmódú teszt	másolás→mozgatás; dokumentált eltérés
Range-init temporary	`for (... : kif().tag())`	kétmódú teszt, TSan	UB megszűnése, destruktorsorrend
Literál-összefűzés	`L"/u"/u8" + "`	C++23 build	hard error; egységes prefix

A CI-átállítás fokozatokban történjen. **1. fokozat:** minden target fordul `-std=c++20` és `-std=c++23` módban is, de a C++20-as build marad a kötelező kapu, a C++23-as csak jelző (warning, nem hiba). **2. fokozat:** az új kódra és az új targetekre `-Werror=deprecated-declarations`; az örökölt kód külön targeten, target-szintű `target_compile_options`-szal kapja a szigorítást, hogy a figyelmeztetés ne globális `-Wno-...`-tal tűnjön el. **3. fokozat:** a kétmódú tesztek kimenet-összehasonlítása, eltérés esetén dokumentált döntés; majd a C++23 lesz az alapértelmezett, a C++20 pedig ideiglenes visszaállási út. Az elavulási figyelmeztetést soha ne kapcsold ki projekt-szinten: a lokális, indokolt elnyomás (egy blokkra szűkített diagnosztika) megőrzi a jelzést a következő migrációs hullámra.

44.3.7 Tipikus hibák

- **Elavulásra `-Werror`-t tenni globálisan.** A könyvtári implementáció nem köteles diagnosztizálni, és a diagnosztika makróval kikapcsolható; a globális `-Werror` a könyvtárfejlesztés miatt is elhasal.
- **A C++23-as buildet csak release-ben futtatni.** Az implicit move és a range-for osztályok NDEBUG-tól függetlenül változnak, de a diagnosztikák gyakran csak debug módban jelennek meg.
- **A korábbi UB-t „működő” referenciaként kezelni.** A range-init temporary esetén a C++20-as futás eredménye nem viszonyítási alap.
- **DR-t figyelmen kívül hagyni.** Ha a fordító visszamenőleg alkalmazza a javítást, a C++20-as build is mutathatja az új viselkedést.

44.3.8 Feladatok

1. **Osztályozó feladat.** Az alábbi négy állítás mindegyikéről döntsd el, hogy eltávolított, elavult vagy viselkedésváltozó kategória-e, és indokold a CI-következményt: (a) `std::declare_reachable`, (b) `std::aligned_storage`, (c) `return param;` egy by-value paraméternél, (d) `L"a" "b"`.
2. **Kódolási feladat.** Írj egy `Slot<T>` osztályt, amely `aligned_storage` nélkül, `alignas(T) std::byte` pufferen tárol egy opcionálisan létező `T`-t. Készíts `emplace`, `has_value`, `reset` műveletet, és dokumentáld az owner–tároló–nézet viszonyt, valamint az invalidáló műveleteket.
3. **Diagnosztikai feladat.** Készíts kétmódú tesztet a `passthrough(S{})` példára, és egészítsd ki a CI-t úgy, hogy az eltérés ne bukás, hanem dokumentált riport legyen. Add meg az ellenőrzési feltételt is.

44.3.9 Megoldások

1. megoldás. (a) eltávolított – a C++23-as build hard errorral jelzi, ezért a legelső fokozatban javítandó; (b) elavult – a kód továbbra is fordul, de a diagnosztika implementációfüggő, ezért feature-/compile-probe és `-Wdeprecated` kell, nem elég a build; (c) viselkedésváltozó – mindkét módban fordul, csak a választott konstruktor más, ezért kétmódú futásidejű összehasonlítás szükséges; (d) ill-formed lett – azaz az eltávolításhoz hasonlóan hard error, de nem könyvtári, hanem nyelvi szigorítás; a keresés a vegyes literalprefixekre irányul. Indoklás: a kategória határozza meg a kapu *típusát* (build, diagnosztika, futásidejű diff), és ezt a CI-fokozatokhoz kell rendelni.

2. megoldás. A váz:

```
// build: g++ -std=c++23 -Wall -Wextra -pedantic -o slot slot.cpp
#include <cstddef>
#include <new>
#include <utility>

template <class T>
class Slot {
public:
    Slot() = default;
    ~Slot() { reset(); }
    Slot(const Slot&) = delete;
    Slot& operator=(const Slot&) = delete;

    template <class... Args>
    T& emplace(Args&&... args) {
        reset();
        ::new (static_cast<void*>(buf_)) T(std::forward<Args>(args)...);
        live_ = true;
        return *get();
    }
    bool has_value() const noexcept { return live_; }
    void reset() noexcept {
        if (live_) { get()->~T(); live_ = false; }
    }
    T* get() { return std::launder(reinterpret_cast<T*>(buf_)); }
private:
    alignas(T) std::byte buf_[sizeof(T)];
    bool live_ = false;
};
```

Owner a `Slot`, tároló a `buf_` és a `live_` jelző, nézet a `get()` pointer. Invalidál: `reset()` és a destruktor; `emplace()` előbb resetel, ezért a korábbi pointer érvénytelenné válik. Ellenőrzési feltétel: nem hívunk destruktort nem létező objektumra, és `get()` csak `has_value()` mellett dereferálható. Megjegyzés: a `T` legyen legalább move-konstruálható, és a `Slot` maga nem másolható, mert a belső pointer nem vihető át triviálisan.

3. megoldás. A teszt a fenti `passthrough` program két buildje; a CI-ben két CTest-tesz, `probe_cxx20` és `probe_cxx23`, mindkettő saját elvárt kimenettel, és egy harmadik, `probe_diff`, amely a kettő eltérését riportfájlba írja, de nem bukik el. Ellenőrzési feltétel: a riport tartalmazza a fordítóverziót, a `-std` kapcsolót és a két kimenetet; a kapu csak akkor szigorodik hibára, ha a

laborprofilban dokumentáltuk, hogy a vizsgált elemre nincs visszamenőleges DR-alkalmazás. Így a migráció akkor is nyomon követhető, ha a fordító frissítése megváltoztatja a viselkedést.

44.3.10 Összefoglaló

A C++23 migráció három különböző kaput igényel: a hard errorokat a C++23-as build adja, az elavulásokat csak compile probe és implementációfüggő diagnosztika fogja meg, a viselkedésváltozásokat pedig kizárólag kétmódú, összehasonlító teszt. A garbage collection felület eltűnt, az `aligned_storage`/`aligned_union` és a `has_denorm` elavult, az implicit move és a range-for temporary élettartama csendesen változik, a vegyes literal-összefűzés pedig fordítási hibává vált. A fokozatos CI-átállítás kulcsa, hogy a figyelmeztetés ne tűnjön el globálisan, és hogy minden eltérés dokumentált, verzióhoz kötött megfigyelés legyen – ne feltevés.

44.4 Migrációs labor: a C++ záróprojekt átállítása C++23-ra

A C++23-ra való migráció ebben a laborban nem a záróprojekt teljes újraírását jelenti, hanem a C++ core: parser, erőforrások és értéktípusok (39.2) és CMake, tesztek és mérési csomag (39.4) során felépített projekt fokozatos átállítását mérhető, visszagörgethető lépésekben. A cél, hogy a projekt kritikus komponenseit C++23-as szabványos eszközökkel váltsuk ki, miközben a C++20-as build továbbra is működik, és a viselkedés nem változik. A migráció során minden új képesség elérhetőségét feature-test makróval és compile probe-bal ellenőrizzük, és minden új eszközhöz fallbacket adunk. A labor eredménye egy kétprofilos build, egy CI-mátrix, egy migrációs jelentés és egy release notes. A fejezet az Elavult, eltávolított és viselkedést váltó elemek C++23-ban (44.3) során rögzített elavulási és eltávolítási listára épül, de nem ismétli meg annak teljes tartalmát.

44.4.1 A kiinduló állapot és a viselkedési szerződés rögzítése

A migráció első lépése a kiinduló állapot (baseline) és a viselkedési szerződés (behavioral contract) rögzítése. A viselkedési szerződés a projekt azon megfigyelhető tulajdonságainak összessége, amelyekre a felhasználók és a tesztek támaszkodnak: a CLI kimenete, a hibakódok, a protokoll üzenetek, a szolgáltatás életciklusa és a timing. A szerződés nem tartalmazza a belső implementáció részleteit, például azt, hogy egy result típus hogyan tárolja a hibát, vagy hogy egy tároló milyen iterátorokat használ. A migráció során a szerződés nem változhat, kivéve, ha a változás dokumentált, tesztelt és a release notes-ben jelölt.

A baseline rögzítése a következő lépésekből áll. Először a C++20-as buildet le kell fordítani a CMake, tesztek és mérési csomag (39.4) szerinti CMake-profillal, és le kell futtatni a teljes CTest-készletet, beleértve a CTest, negatív tesztek és sanitizer CI (38.4) szerinti negatív teszteket és sanitizeres futtatásokat. Az eredményt rögzíteni kell: hány teszt fut, mennyi a fordítási idő, mekkora a bináris mérete. Másodszor a CLI kimenetét golden fájlokban kell rögzíteni a tipikus bemenetekre, beleértve a hibaeseteket is. Harmadszor a hibakódokat és a hibaüzeneteket kell dokumentálni, mert a felhasználók ezekre támaszkodhatnak. Negyedszer a hálózati protokoll üzeneteit kell rögzíteni, ha a projekt ilyet használ. Ötödször a mérési pontokat kell kijelölni: a parser futásideje, a szolgáltatás válaszideje, a memóriahasználat és a bináris méret. A méréseket a CMake, tesztek és mérési csomag (39.4) mérési csomagjával kell elvégezni, és az eredményeket verziókezelt fájlban kell tárolni.

A baseline rögzítése után a migráció minden lépését ugyanezekkel a tesztekkel és mérésekkel kell ellenőrizni. Ha egy lépés megváltoztatja a viselkedést, vagy nem hoz mérhető javulást, vissza kell

vonni. A visszagörgethetőség érdekében minden lépés külön commit legyen, és a build profilok külön könyvtárban legyenek.

44.4.2 Kétprofilos build és CI-mátrix

A kétprofilos build azt jelenti, hogy ugyanaz a forráskód két nyelvi módban fordul: C++20 és C++23. A CMake-ben ezt egy opcióval vezéreljük. A következő példa a projekt gyökerében lévő `CMakeLists.txt` releváns részét mutatja. A példa illusztratív, a CMake és a compiler verzióját a laborprofilban rögzíteni kell.

```
cmake_minimum_required(VERSION 3.28)
project(diag LANGUAGES CXX)

option(DIAG_USE_CXX23 "Build with C++23" OFF)

add_executable(diag
  src/main.cpp
  src/parser.cpp
  src/service.cpp
)

if(DIAG_USE_CXX23)
  target_compile_features(diag PRIVATE cxx_std_23)
else()
  target_compile_features(diag PRIVATE cxx_std_20)
endif()

target_compile_options(diag PRIVATE -Wall -Wextra -Wpedantic)
```

A build parancsok a két profilra:

```
cmake -S . -B build-cxx20 -G Ninja -DDIAG_USE_CXX23=OFF
 ↪  -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake --build build-cxx20
ctest --test-dir build-cxx20 --output-on-failure

cmake -S . -B build-cxx23 -G Ninja -DDIAG_USE_CXX23=ON
 ↪  -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake --build build-cxx23
ctest --test-dir build-cxx23 --output-on-failure
```

A CI-mátrix két dimenziója a fordító (GCC, Clang) és a nyelvi mód (C++20, C++23). Minden kombinációhoz rögzíteni kell a konkrét compiler- és standard library-verziót, a build parancsot és a teszt parancsot. A mátrix minden elemén futnak a CTest-tesztek, a sanitizeres futtatások és a golden fájlok ellenőrzése. A C++23-as buildben a feature-test makrók alapján kapcsolódnak be a komponensek; a C++20-as buildben a fallbacknek kell fordulnia. A CI-mátrix akkor tekinthető teljesnek, ha a C++20-as és a C++23-as build ugyanazokat a viselkedési teszteket teljesíti.

44.4.3 Komponensenkénti csere méréssel és fallbackkal

A migrációt komponensenként végezzük. Minden komponensnél meg kell határozni a motivációt, a pontos cserét, a fallbacket és a mérési szempontokat. A javasolt sorrend: `std::expected`,

`std::print`, `ranges::to`, `std::flat_map`. A sorrend a kockázat és a fallback költsége szerint növekszik.

44.4.3.1 Saját result típus helyett std::expected

A C++ core: parser, erőforrások és értéktípusok (39.2) során a projekt hibakezelése egy saját result típusra épült. A C++23-as `std::expected<T, E>` szabványos, monadikus műveleteket (`and_then`, `or_else`, `transform`) kínál, és jobban illeszkedik a standard library többi részéhez. A csere nem jelentheti a hívási helyek teljes újraírását: vezessünk be egy `DiagResult` aliast, amely a C++23-as buildben a `std::expected`-et, a C++20-as buildben a meglévő típust használja.

```
// feature.hpp
#pragma once

#if __has_include(<version>)
#  include <version>
#endif

#if defined(__cpp_lib_expected) && __cpp_lib_expected >= 202202L
#  include <expected>
#  define DIAG_HAVE_EXPECTED 1
#else
#  define DIAG_HAVE_EXPECTED 0
#endif
```

```
// result.hpp
#include "feature.hpp"
#include <string>

#if DIAG_HAVE_EXPECTED
template<class T, class E = std::string>
using DiagResult = std::expected<T, E>;
#else
template<class T, class E = std::string>
using DiagResult = diag::legacy_result<T, E>;
#endif
```

A hívási helyeken a `DiagResult`-ot kell használni. A monadikus láncokat csak ott érdemes átírni, ahol a `std::expected` szemantikája egyértelmű; a régi result típus eltérő hibakezelése miatt a viselkedést tesztekkel kell ellenőrizni. A tipikus hiba, hogy a `std::expected` nem dob kivételt, a régi result típus viszont igen; a hibakezelést a hívási helyeken egységesíteni kell. A mérési szempontok: a fordítási idő, a bináris méret és a hibaút futásideje.

44.4.3.2 Naplózás: printf/iostream helyett std::print

A projekt naplózása jelenleg `printf` vagy `iostream` alapon működik. A C++23-as `std::print` típusbiztos, formázott kimenetet ad, és a formázási szintaxisa a `std::format`-tal azonos. A fallback a meglévő `printf` vagy `std::format` + `std::cout` marad. A naplózó függvényt érdemes egyetlen interfész mögé rejteni, hogy a hívók ne lássák a különbséget.

```
// log.hpp
#include "feature.hpp"
#include <string_view>

namespace diag {
void log(std::string_view msg);
}

// log.cpp
#include "log.hpp"

#if DIAG_HAVE_PRINT
#  include <print>
void diag::log(std::string_view msg) {
  std::print("{}\n", msg);
}
#else
#  include <cstdio>
void diag::log(std::string_view msg) {
  std::printf("%.*s\n", static_cast<int>(msg.size()), msg.data());
}
#endif
```

A tipikus hiba, hogy a `std::print` nem flush-el automatikusan minden esetben; a naplózás végén szükség lehet `std::flush`-re. A formázási sztringeket egységesen `{}` helyőrzőkkel kell írni, és a fallbackben át kell alakítani `%`-re, vagy a fallbackben is `std::format`-et kell használni, ha elérhető. A mérési szempontok: a naplózási throughput, a bináris méret és a fordítási idő.

44.4.3.3 Materializálás: kézi gyűjtés helyett ranges::to

A projekt több helyen kézi ciklussal gyűjt egy tartományból vektort. A C++23-as `std::ranges::to<std::vector>()` tömörebb és kifejezőbb, és a standard range-ekkel működik. A fallback egy egyszerű ciklus.

```
// collect.hpp
#include "feature.hpp"
#include <ranges>
#include <vector>

template<std::ranges::range R>
auto collect(R&& r) {
#if DIAG_HAVE_RANGES_TO
  return std::forward<R>(r) | std::ranges::to<std::vector>();
#else
  std::vector<std::ranges::range_value_t<R>> out;
  for (auto&& e : r) out.push_back(std::forward<decltype(e)>(e));
  return out;
#endif
}
```

A tipikus hiba, hogy a bemeneti tartomány nem forward range, és a `ranges::to` többször is bejárhatja; ilyenkor a tartományt előbb materializálni kell, vagy a fallbacket kell használni. A mérési szempontok: az allokációk száma, a futásidejű teljesítmény és a bináris méret.

44.4.3.4 Rendezett tároló: std::map helyett std::flat_map

A projekt rendezett asszociatív tárolóként `std::map`-et használ. A C++23-as `std::flat_map` folytonos tárolásban tartja a kulcs-érték párokat, ami jobb locality-t és kisebb memóriahasználatot ad, de a beszúrás és a törlés drágább, és az iterátorok a beszúrás után érvénytelenné válnak. A fallback a `std::map`.

Tulajdonság	`std::map`	`std::flat_map`
Tárolás	csomópontos	folytonos
Keresés	$O(\log n)$	$O(\log n)$
Beszúrás	$O(\log n)$	$O(n)$
Iterátor stabilitás	beszúrás után is érvényes	beszúrás után érvénytelen
Memória	nagyobb	kisebb
Locality	gyengébb	jobb

```
// map.hpp
#include "feature.hpp"
#include <map>

#if DIAG_HAVE_FLAT_MAP
template<class K, class V>
using DiagMap = std::flat_map<K, V>;
#else
template<class K, class V>
using DiagMap = std::map<K, V>;
#endif
```

A tipikus hiba, hogy a kód iterátort tárol a tárolóban, és a beszúrás után azt használja; ez `std::flat_map` esetén dangling iterátorhoz vezet. A cserét csak olyan helyen szabad elvégezni, ahol a kód nem támaszkodik az iterátorok stabilitására. A mérési szempontok: a keresési idő, a beszúrási idő, a memóriahasználat és a bináris méret.

44.4.4 Feature-test, compile probe és fallback stratégia

A feature-test makrók a fordító és a szabványos könyvtár által definiált makrók, amelyek egy adott nyelvi vagy könyvtári képesség elérhetőségét jelzik. A `__cpp_lib_*` makrók a könyvtári képességeket jelzik, és a `<version>` headerben érhetők el. A laborban használt makrók: `__cpp_lib_expected`, `__cpp_lib_print`, `__cpp_lib_ranges_to_container`, `__cpp_lib_flat_map`. A pontos értékek verziófüggők; a kódban a szabványos minimális értékre kell összehasonlítani, de az elérhetőséget a compiler és a standard library dokumentációja szerint kell ellenőrizni.

A compile probe egy olyan minimális forrás, amelynek sikeres fordítása igazolja egy képesség elérhetőségét. A feature-test makrók mellett akkor hasznos, ha a makró nem megbízható, például részleges implementáció esetén. Például:

```
// probe_expected.cpp
#include <expected>
int main() {
  std::expected<int, int> e = 42;
  return e.value();
}
```

A fallback stratégia lényege, hogy a hívók csak a projekt saját absztrakcióját használják, és a feature-test makrók alapján a build válassza ki az implementációt. A fallbacknek ugyanazt a viselkedést kell biztosítania, mint az új implementációnak. A release notes-ben dokumentálni kell, hogy melyik komponens milyen feature-test makrót igényel, mi a fallback, és milyen viselkedésbeli eltérés van. A release notes a migráció visszagörgethetőségének is az alapja.

44.4.5 Teljesítmény és kódméret összehasonlítása

A migráció mérhető, ezért minden lépés előtt és után mérni kell. A metrikák: a fordítási idő, a bináris méret, a futásidejű benchmark, a memóriahasználat és a tesztek száma. A méréshez használható eszközök: `time` a fordítási időhöz, `size` vagy `bloaty` a bináris mérethez, `/usr/bin/time -v` a memóriahasználathoz, és egy saját benchmark vagy `hyperfine` a futásidejű teljesítményhez. A méréseket azonos környezetben, azonos bemenettel és többszöri futtatással kell végezni; az eredmények mediánját érdemes rögzíteni.

A mérési terv része a döntési feltétel is. Ha egy komponens cseréje nem hoz mérhető javulást, vagy rontja a viselkedést, vissza kell vonni. Például a `std::flat_map` bevezetését csak akkor érdemes megtartani, ha a keresési idő javul, a beszúrási idő nem romlik jelentősen, és a kód nem támaszkodik az iterátorok stabilitására. A mérések eredményét a migrációs jelentésben kell dokumentálni. A labor során a tanuló végzi a méréseket; a fejezet nem állítja, hogy a példák mért eredményeket tartalmaznak.

44.4.6 Tipikus hibák és javításuk

Az első hiba, hogy a feature-test makrót a header include előtt ellenőrizzük. A helyes sorrend: `#include <version>`, majd a makró ellenőrzése, majd a szükséges header includolása. A második hiba, hogy a `std::print` nem flush-el, és a napló késleltetve jelenik meg. A javítás: a naplózó függvény végén `std::flush` vagy a kimenet lezárása. A harmadik hiba, hogy a `std::flat_map` iterátorait a beszúrás után is használjuk; a javítás: az iterátorokat a beszúrás után újra kell szerezni, vagy `std::map`-et kell használni. A negyedik hiba, hogy a `ranges::to` többször bejárja a bemeneti tartományt; a javítás: a tartományt előbb materializálni kell, vagy a fallbacket kell használni. Az ötödik hiba, hogy a C++20-as buildben a fallback hiányzik, és a build nem fordul; a javítás: minden új képességhez kötelező a fallback. A hatodik hiba, hogy a CMake verziója nem ismeri a `cxx_std_23` feature-t; a javítás: a laborprofilban rögzített CMake-verziót kell használni, vagy a `CXX_STANDARD` tulajdonságot kell beállítani, ha az támogatott.

44.4.7 Feladatok

1. Készíts döntési táblázatot a négy komponens (`std::expected`, `std::print`, `ranges::to`, `std::flat_map`) migrációs sorrendjéről. Szempontok: kockázat, mérési lehetőség, fallback költsége és függőségek. Indokold a sorrendet.

2. Írj egy feature detection headert és egy `DiagResult` aliast, amely a C++23-as `std::expected`-et használja, ha elérhető, különben a meglévő `diag::legacy_result`-ot. Add meg a build parancsot mindkét profilra.

3. A C++20-as build a következő hibát adja: `error: 'expected' is not a member of 'std'`. Mi a hiba oka, és hogyan javítod? Írd le a javítás lépéseit.

4. Alakítsd át a naplózást `std::print`-re, és biztosíts fallbacket. A naplózó függvény szignatúrája `void log(std::string_view)`. Add meg a kódot és a build parancsot.

5. Tervezz egy mérési tervet a `std::flat_map` bevezetéséhez. Milyen metrikákat mérsz, milyen eszközökkel, milyen környezetben, és hogyan döntesz a bevezetésről?

44.4.8 Megoldások

44.4.8.1 Az első feladat megoldása

A javasolt sorrend: `std::expected`, `std::print`, `ranges::to`, `std::flat_map`. Az `std::expected` kockázata a legkisebb, mert egy alias mögé rejthető, és a fallback a meglévő típus. A `std::print` szintén jól elkülöníthető, a fallback egy függvény. A `ranges::to` lokális csere, a fallback egy ciklus. A `std::flat_map` kockázata a legnagyobb, mert az iterátorok invalidációja és az API eltérése miatt a kód több helyen is módosítást igényelhet. Az ellenőrzési feltétel: minden lépés után a C++20-as és a C++23-as build is fordul, és a viselkedési tesztek zöldek.

44.4.8.2 A második feladat megoldása

A feature detection header és az alias a fenti `feature.hpp` és `result.hpp` példa szerint épül fel. A build parancsok:

```
cmake -S . -B build-cxx20 -G Ninja -DDIAG_USE_CXX23=OFF
cmake --build build-cxx20
cmake -S . -B build-cxx23 -G Ninja -DDIAG_USE_CXX23=ON
cmake --build build-cxx23
```

Az ellenőrzési feltétel: a C++20-as buildben a `DiagResult` a `diag::legacy_result`, a C++23-as buildben a `std::expected` típusra oldódik fel, és a tesztek mindkét profilban zöldek.

44.4.8.3 A harmadik feladat megoldása

A hiba oka, hogy a C++20-as buildben a `std::expected` nem elérhető, mert a `__cpp_lib_expected` makró nem definiált, vagy a `<expected>` header nem elérhető. A javítás lépései: először a feature detection headerben ellenőrizni kell a `__cpp_lib_expected` makrót; másodszor a `DiagResult` aliast a `diag::legacy_result`-ra kell irányítani; harmadszor a hívási helyeken nem szabad közvetlenül `std::expected`-et használni, csak a `DiagResult`-ot. Az ellenőrzési feltétel: a C++20-as build fordul, és a hibakezelési tesztek zöldek.

44.4.8.4 A negyedik feladat megoldása

A naplózó függvény a fenti `log.hpp` és `log.cpp` példa szerint épül fel. A build parancsok a második feladatnál megadottak. Az ellenőrzési feltétel: a napló kimenete mindkét profilban ugyanaz, és a `std::print` esetében a kimenet flush-elődik. A fallback a `printf`-et használja, amely a stringet és az új sort kiírja.

44.4.8.5 Az ötödik feladat megoldása

A mérési terv: metrikák a keresési idő, a beszúrási idő, az iterációs idő, a memóriahasználat és a bináris méret. Eszközök: saját benchmark a műveletekhez, `/usr/bin/time -v` a memóriahasználathoz, `size` a bináris mérethez. A környezet: rögzített CPU, memória, compiler-verzió és optimalizálási szint. A döntés: ha a keresési idő javul, a beszúrási idő nem romlik jelentősen, és a kód nem támaszkodik az iterátorok stabilitására, a `std::flat_map` bevezethető; különben a `std::map` marad. A döntést a release notes-ben dokumentálni kell.

44.4.9 Összefoglaló

A migrációs labor a záróprojekt C++23-ra állítását mérhető, visszagörgethető lépésekben végzi. A kiinduló állapot és a viselkedési szerződés rögzítése után a kétprofilos build és a CI-mátrix biztosítja, hogy a C++20-as és a C++23-as build ugyanazt a viselkedést nyújtsa. A komponensenkénti csere során a saját result típust `std::expected`-re, a naplózást `std::print`-re, a kézi materializálást `ranges::to`-ra, a rendezett tárolót `std::flat_map`-re cseréljük, minden esetben feature-test makróval, compile probe-bal és fallbackkal. A teljesítmény- és kódméret-összehasonlítás alapján döntünk a megtartásról vagy a visszavonásról. A tipikus hibák elkerülése és a release notes vezetése a migráció sikerének feltétele.

45 Rövid kitekintés: C++26

45.1 C++26 nyelvi kitekintés: reflexió, contracts és kisebb újdonságok

Ez a fejezet szándékosan kitekintés, nem tananyagmélységű tárgyalás. A C++26 nyelvi képességeit azért érdemes korán megismerni, mert több közülük olyan mintákat vált ki, amelyeket a könyv korábbi részeiben C-vel és C++20/23-mal kézzel építettünk fel: a makrós „reflexiót”, a kézzel írt elő- és utófeltétel-ellenőrzéseket, a bináris erőforrások beágyazását és a `static_assert` üzeneteinek kézi összefűzését. A cél tehát nem az, hogy most rögtön termelési kódot írjunk C++26-ban, hanem hogy felismerjük: melyik korábbi szerződésünket fogja a nyelv vagy a könyvtár átvenni, és mi marad továbbra is a mérnök feladata.

A státuszállítások forrása a forrásjegyzék C++26 tétele, 2026-10-25-i ellenőrzéssel: a C++26 technikai munkája 2026. március 28-án lezárult, a szabvány DIS-szavazáson van, ISO-kiadását az ellenőrzés napján nem találtuk igazolva. A fejezetben tárgyalt képességek (statikus reflexió, P2996R13; contracts, P2900R14; `#embed`, P1967R14; pack indexing, P2662R3; a `_` helyőrző, P2169R4; strukturált kötés csomagja, P1061R10; `= delete("ok")`, P2573R2; számított `static_assert`-üzenet, P2741R3; erroneous behaviour, P2795R5) a lezárt C++26 részei; a papírlistát a cppreference C++26-oldalának ugyanazon napi állapotából vettük (másodlagos forrás). Az implementációs támogatást a könyv csak a saját profiljain méri: a `lab-gcc` profil GCC 13.3.0 fordítója egyiket sem támogatja, `-std=c++2c` módja sincs. A fejezet kódpéldáit ezért nem fordítottuk le; ahol egy állítás verziófüggő, ott ezt jelölöm.

45.1.1 A státuszolvasás: elfogadott, kiadott, implementált

A C++ szabványosításban három, egymástól élesen elkülönítendő állapot van, és a gyakorlati hibák többsége ennek a háromnak az összemosásából származik.

Elfogadott (adopted). Egy papírt a WG21 munkacsoport a munkatervezetbe (working draft) fogadott. Ettől a képesség „bent van” a következő szabvány tervezetében, de ez nem jelenti azt, hogy bármelyik fordító ismeri.

Kiadott (published). Az ISO megjelentette a szabvány végleges szövegét. A fejezetterv kiinduló adata szerint a C++26 nyelvi jellemzői 2025-ben befagytak (feature complete), a kiadás pedig ezt követően várható. Ez a dátum és állapot **ellenőrizendő** a hivatalos ISO/WG21 közleményekből; a fejezet nem állítja, hogy a kiadás megtörtént.

Implementált (implemented). Egy adott fordító- és standard library-kiadás ténylegesen fordítja és helyesen viselkedik. Ez a szint a legfontosabb a mindennapi munkában, és ez a leglassabb. Egy képességhez ezért három dolgot kell megnézni: a papír státuszát, a `__cpp_...` feature-test makró jelenlétét az adott implementációban, és a konkrét compiler/library verziót.

A fejezet képességeinek státusza ebben a könyvben egységesen: **kísérleti, nem része a laborprofilnak**. A laborok továbbra is C++20-at (`-std=c++20`) használnak, a C++23-as részek pedig külön jelölt profillal futnak.

Képesség	Papír	Állapot ebben a fejezetben
statikus reflexió	P2996	kísérleti, csak szemléltetés
contracts	P2900	kísérleti, csak szemléltetés
pack indexing	P2662	kitekintés
_ placeholder	P2169	kitekintés
`#embed`	P1967	kitekintés (C23-ban is létezik)
erroneous behaviour	P2795	szemantikai kitekintés

45.1.2 Statikus reflexió: `^^`, `std::meta` és a splice

A statikus reflexió (static reflection) lényege, hogy a program **fordítási időben** tudjon kérdezni a saját típusairól, tagjairól, enumerátorairól, és ezekre hivatkozni tudjon. A P2996 mechanizmusa két fogalomra épül.

A `^^` operátor (reflection operator) egy `std::meta::info` típusú értéket állít elő: ez egy átlátszatlan, fordítási idejű fogantyú egy entitásra (típusra, névtérre, tagra). A lekérdezések a `std::meta` névtér `consteval` függvényei: például `nonstatic_data_members_of`, `identifier_of`, `type_of`, `parent_of`. A **splice** a másik irány: a `[: ... :]` szintaxis egy `std::meta::info` alapján *visszahelyettesít* egy entitást a forrásba — típusnév, tag, kifejezés vagy sablonargumentum helyére.

```
// Illusztráció a P2996 papír szerinti alakból; kísérleti fordítóval sem
 ↪  ellenőriztük.
#include <meta>
#include <string_view>

struct listener {
    int port;
    std::string_view bind_address;
};

consteval std::size_t field_count(std::meta::info type) {
    return std::meta::nonstatic_data_members_of(
                type, std::meta::access_context::current())
        .size();
}

static_assert(field_count(^^listener) == 2);

template<std::meta::info Member>
constexpr auto& field(auto& obj) {
    return obj.[:Member:];   // splice: a tag neve helyére
}
```

A kód buildparancsa a *stabil* részhez (a `field_count` és a `static_assert` nélkül) a szokásos profil; a teljes példa fordítása kísérleti fordítóágat igényel, amelynek kapcsolóit a kiadott dokumentációból kell ellenőrizni:

```
# A kísérleti profil kiinduló parancsa; a flagek verziófüggők, ellenőrizendők.
# g++ --version && g++ --help=common | grep -i reflect
g++ -std=c++26 -fsyntax-only reflection_demo.cpp
```

Miért fontos ez a könyv szempontjából? A Makrók, feltételes fordítás és veszélyes helyettesítés részben (2.3, „X-macro: egyetlen taglista több helyen”) az X-macro technikát használtuk arra, hogy egy `struct` tagjait egyetlen listából deklaráljuk, megszámoljuk és név szerint kiírjuk. Ha a `struct` és a metaadat ugyanabból az egyetlen X-macro listából készül, nincs két kézzel szinkronban tartandó lista — ez az X-macro erőssége. A valódi különbség máshol van: az X-macro a deklaráció *formáját* egy generátorsémához köti (minden tagnak a makró által ismert alakban kell szerepelnie; bázisosztály, attribútum, bonyolult típus vagy alapérték csak a séma bővítésével írható le, és a típus nem olvasható „rendes” deklarációként), és csak az általa generált típusokra működik. A reflexió ezzel szemben egy *már létező*, szabad formában írt deklarációból kérdez — bármely típusról, utólag is, a makró-sémához való igazítás nélkül. Ugyanez igaz a Layout- és protokolllabor diagnosztikájára: az offsetek és nevek generálhatók, nem kézzel írandók.

45.1.3 Contracts: `pre`, `post` és `contract_assert`

A P2900 contracts a függvény **szerződését** viszi be a szintaxisba. A `pre(cond)` előfeltétel a paraméterlista után, a `post(cond)` utófeltétel a törzs előtt áll; az utófeltételben névvel hivatkozhatunk a visszatérési értékre. A `contract_assert(cond)` egy utasítás, amely a törzsön belüli belső invariánst ellenőrzi.

```
// Illusztráció a P2900 papír szerinti alakból; kísérleti, nem tesztelt.
#include <cstddef>

int clamp_index(int i, std::size_t n)
    pre(i >= 0)
    pre(static_cast<std::size_t>(i) < n)
    post(r: r >= 0 && static_cast<std::size_t>(r) < n)
{
    contract_assert(n > 0);
    return i;
}
```

A lényeg a **kiértékelési szemantika**, mert ez különbözteti meg az `assert`-nél megszokott mindent-vagy-semmit modelltől. A papír négy üzemmódot ír le: `ignore` (a feltétel nem értékelődik ki), `observe` (kiértékelődik, megsértés esetén a violation handler lefut, de a program folytatódik), `enforce` (kiértékelődik, megsértés esetén a handler lefut, majd a program leáll), `quick-enforce` (kiértékelődik, handler nélkül azonnal leáll). A módot fordításidejű beállítás választja, tehát ugyanaz a forrás kerül a fejlesztői és a termelési buildbe — ez a contracts legfontosabb ígérete.

Ez a korábbi anyagban két helyet érint. Egyrészt az Unit test, property test és fuzzing hibafelfedező rétegét: az előfeltételek most a fejlesztői buildben automatikusan aktívak, a release buildből pedig eltávolíthatók, anélkül hogy a forrás kettéválna. Másrészt a Hibakód, errno és hibaadatok élettartama hibamodelljét: a contracts **programozói hibát** jelez, nem futásidejű, várható hibát. Egy hiányzó fájl, egy megszakadt kapcsolat nem szerződéssértés, hanem normál hibaút, amelyre továbbra is `std::expected` vagy hibakód a válasz (std::expected mélyebben).

45.1.4 Kisebb nyelvi újdonságok

Pack indexing (P2662). Sablon-paramétercsomag eddig csak expandálható volt; most közvetlenül indexelhető: `Ts...[0]`, `Ts...[I]`. Ez kiváltja a rekurzív „vesd el az elsőt” segédsablonokat, és tipikusan olyan helyen hasznos, ahol eddig `std::tuple_element`-hez kellett menekülni.

```cpp
template<class... Ts>
using first_t = Ts...[0];

template<std::size_t I, class... Ts>
using nth_t = Ts...[I];
```

A `_` placeholder (P2169). A `_` újra deklarálható helyőrző név, amely szándékosan „nem nevez meg” semmit: nem ad rá figyelmeztetést a `-Wunused`, és ugyanabban a hatókörben többször is szerepelhet. A `for (auto _ : xs)` és a strukturált kötés tipikus felhasználási esete.

Strukturált kötés bővítései. A P1061 lehetővé teszi, hogy a strukturált kötés csomagot vezessen be (a papír szintaxisa szerint `auto [head, ...tail] = t;` alakban), ami a rekurzív tuple-feldolgozást váltja ki. Emellett a konstans kiértékelés oldalán is bővült a nyelv: a `constexpr` strukturált kötések, a `void*`-ból végzett `constexpr` cast és a `constexpr` placement new a C++26 papírjai között szerepelnek — a teljes lista ebben a kitekintésben nem teljes.

`= delete("ok")` és a `static_assert` üzenete. A törölt függvény indoklása most szövegesen is megadható: `fd(int) = delete("fd nem építhető nyers int-ből");` — a diagnosztika a felhasználó szövegét is kiírja. A `static_assert` üzenete pedig lehet konstans kiértékelésű kifejezés eredménye, így a diagnosztika számított adatot (méretet, típusnevet) tartalmazhat. Ez közvetlenül a Konstans kiértékelés C++23-ban és a constexpr, consteval és constant evaluation tudására épül: az üzenetnek fordítási időben kiszámolhatónak kell lennie.

`#embed` (P1967). A preprocessor bináris fájl tartalmát ágyazza be, vesszővel elválasztott egész literálok listájaként (egy `ABC` tartalmú fájlból `65, 66, 67` lesz). A paraméterek: a `limit(n)` legfeljebb `n` bájtot olvas; a `prefix(...)` és a `suffix(...)` tokensorozata **egyszer**, a teljes lista elé, illetve mögé kerül, és csak akkor, ha a beágyazott tartalom nem üres; az `if_empty(...)` üres tartalom esetén lép a lista helyére. A `prefix` tehát nem elemenkénti formázás. A `prefix(0x)` a `0x` tokent egyszer, az első szám elé teszi, és ebből egyetlen érvénytelen tokenszerű sorozat (`0x65`-höz hasonló) keletkezne; a fordító ezt elutasítja.

```c
/* embed_demo.c - C23: #embed és paraméterei */
#include <stdio.h>

/* Egyszerű bájttömb: a #embed vesszővel elválasztott egész literálokat ad. */
static const unsigned char icon[] = {
#embed "three.bin" limit(64)
};

/* prefix/suffix: egyszer, a teljes lista elé/mögé kerül, ha a fájl nem üres;
   if_empty: üres fájl esetén ez kerül a helyére. */
static const unsigned char framed[] = {
#embed "one.bin" prefix(0xFF, ) suffix(, 0x00) if_empty(0xEE)
};

static const unsigned char none[] = {
#embed "empty.bin" prefix(0xFF, ) suffix(, 0x00) if_empty(0xEE)
};

int main(void) {
    printf("icon: %zu bájt, első=%d\n", sizeof icon, icon[0]);
    printf("framed:");
    for (size_t i = 0; i < sizeof framed; ++i) printf(" %d", framed[i]);
    printf("\nnone:");
```

```
    for (size_t i = 0; i < sizeof none; ++i) printf(" %d", none[i]);
    printf("\n");
    return 0;
}
```

Az előfeldolgozott initializerek: `three.bin` (`ABC`) esetén `{65, 66, 67}`; az egybájtos `one.bin` (`A`) prefixszel és suffixszel `{0xFF, 65, 0x00}`; az üres `empty.bin` esetén az `if_empty` miatt `{0xEE}`.

Buildjegyzőkönyv. A `lab-gcc` profil GCC 13.3-a nem támogatja az `#embed`-et. Ellenőrzésképpen a `ziglang` 0.17.0 pip-csomagban szállított Clang 22.1.8-cal fordítottuk, `-std=c23 -Wall -Wextra -Wpedantic` mellett (ez a `lab-next` profil része). A program figyelmeztetés nélkül fordult; kimenete `icon: 3 bájt, első=65`, `framed: 255 65 0`, `none: 238`, a `-E` kimenet a fenti initializereket mutatta. Az eredeti `prefix(0x) suffix(,)` alak `invalid suffix 'x' on integer constant` hibát adott.

Figyeljük meg a kereszthivatkozást: a `#embed` a C23-ban is megjelent (C23 könyvtár és preprocessor), tehát itt nem új C++-saját ötletről van szó, hanem a két szabvány közös preprocessor-képességéről. A C++ oldalon a `std::embed`-féle könyvtári változat nem része ennek a kitekintésnek.

Erroneous behaviour (P2795R5, a C++26 munkatervezetében, [basic.indet]). A szabály hatóköre szűk: csak az **automatikus tárolású** objektumokra (lokális változókra és temporarykra) vonatkozik, amelyeknek nincs inicializálójuk. Ezek kezdőértéke a szabály szerint *erroneous value* (egy implementáció által választott, a forrástól független érték), és ennek olvasása *erroneous behaviour*: hibás, de nem UB — az implementáció diagnosztizálhatja, és nem kell „bármi megtörténhet” alapon optimalizálnia. Ami **nem** változik: (1) a `[[indeterminate]]` attribútummal jelölt automatikus változó értéke továbbra is *indeterminate*, és olvasása UB; (2) a dinamikus tárolású objektumok (`new int`, `malloc`) kezdőértéke továbbra is indeterminate, olvasásuk UB; (3) az `unsigned char` és a `std::byte` különszabályai megmaradnak: indeterminate értékük bizonyos másoló műveletekkel UB nélkül továbbvihető, a többi felhasználás UB. Az erroneous behaviour tehát nem teszi helyessé a kódot, és nem minden inicializálatlan olvasás diagnosztizált. Ez pontosan a Warningok, optimalizálási profilok és link-driver és az Optimalizáló, as-if és UB következményei részben tárgyalt UB-fogalom finomítása — a helyes válasz változatlanul az inicializálás.

45.1.5 Tipikus hibák

A leggyakoribb hiba a **státusz-összemosás**: valaki a munkatervezetben látott szintaxist beírja a termelési kódba, majd a CI egy régebbi fordítón elhasal. A második: a **contracts és a futásidejű hibakezelés** összetévesztése — szerződéssértésre `catch`-csel reagálni értelmetlen, mert `enforce` módban a program leáll. A harmadik: a reflexiót futásidejű mechanizmusnak hinni; a `std::meta` lekérdezések `consteval` kontextusban élnek, futásidőben nem hívhatók. A negyedik: az `#embed` és a `#include` összekeverése — a `#embed` nem fejlécet, hanem bájttömb-literalokat illeszt be. Az ötödik: a `prefix`/`suffix` elemenkénti formázásnak hinni; mindkettő egyszer, a teljes lista körül jelenik meg.

45.1.6 Feladatok

1. feladat (státusz). Sorold be az alábbi állításokat „elfogadott / kiadott / implementált” kategóriába, és indokold: (a) „A statikus reflexió benne van a C++26 munkatervezetben.” (b) „A `-std=c++26` kapcsolóval a GCC lefordítja a `^^T` kifejezést.” (c) „Az ISO megjelentette a C++26-ot.”

2. feladat (kódolás). Írj egy `consteval` függvényt, amely egy `struct` nem statikus adattagjainak számát adja vissza, és ellenőrizd `static_assert`-tel. Készítsd el ugyanennek a C++20-as megfelelőjét X-macro-val, és fogalmazd meg, melyik változat milyen karbantartási hibát zár ki.

3. feladat (diagnosztika). Adott a `clamp_index` függvény. Írd le, mi történik `observe`, illetve `enforce` módban, ha `n == 0` és `i == 0` mellett hívjuk. Hol lép fel a hiba: a hívásnál, a belépésnél vagy a visszatérésnél?

4. feladat (tervezés). Egy bináris protokollverzió-tömböt szeretnél beágyazni. Hasonlítsd össze a `#embed`, a generált fejléc és a futásidejű fájlbeolvasás megoldását: melyik milyen build- és hordozhatósági szerződést igényel?

45.1.7 Megoldások

1. (a) elfogadott — a munkatervezetbe fogadás ténye, függetlenül a kiadástól és az implementációktól. (b) implementált — ez konkrét fordító és konkrét verzió viselkedése, amelyet feature-test makróval és a `--version` kimenetével kell rögzíteni. (c) kiadott — *jelentés:* az ISO hivatalosan közzétette a szabványt; *ellenőrzés módja:* hivatalos ISO-közlemény vagy katalógusbejegyzés, dátummal; *ellenőrzött státusz (2026-10-25):* nem igazolt — a C++26 technikailag lezárt és DIS-szavazáson van, ISO-kiadását hiteles forrásból nem tudtuk igazolni. Ez nem azt jelenti, hogy az állítás hamis, hanem hogy bizonyíték nélkül nem állítható; igaznak vagy hamisnak csak dátummal és hiteles forrással minősíthető. *Ellenőrzési feltétel:* mindhárom állításhoz nevezz meg egy-egy elsődleges forrástípust (papír, compiler-dokumentáció, ISO-közlemény), amely eldönti.

2. A `consteval` függvényre vonatkozó minősítő: változót nem lehet `consteval`-lal deklarálni (`a variable cannot be declared 'consteval'`). A helyes szerkezet egy `consteval` függvény, amelynek eredményét `constexpr` változóba vagy `static_assert`-be tesszük. A reflexiós változat:

```
template <class T>
consteval std::size_t field_count() {
    return std::meta::nonstatic_data_members_of(
               ^^T, std::meta::access_context::current()).size();
}

constexpr std::size_t n = field_count<Config>();
static_assert(n == 2);
```

```
// fields_xmacro.cpp - C++20: egyetlen FIELDS lista -> struct, darabszám és
 ↪  névkiírás
#include <cstddef>
#include <cstdio>

#define FIELDS(X) X(int, port) X(const char*, bind_address)

struct Config {
#define X(type, name) type name;
    FIELDS(X)
#undef X
};

// A consteval FÜGGVÉNYRE vonatkozik: minden hívása konstans kiértékelés.
consteval std::size_t field_count() {
```

```
    std::size_t n = 0;
#define X(type, name) ++n;
    FIELDS(X)
#undef X
    return n;
}

constexpr std::size_t n = field_count();   // eredmény constexpr változóban
static_assert(n == 2);

int main() {
#define X(type, name) std::puts(#name);
    FIELDS(X)                                  // a mezőnevek ugyanabból a listából
#undef X
    return static_cast<int>(n) - 2;
}
```

Karbantartási feladat: vegyél fel egy harmadik mezőt (`X(bool, verbose)`) **kizárólag** a `FIELDS` listába; a `struct`, a `field_count()` és a névkiírás ugyanebből a listából automatikusan változik (a `static_assert(n == 2)` ekkor jelzi, hogy a várt darabszámot frissíteni kell). Ellenőrizve GCC 13.3-mal (`-std=c++20 -Wall -Wextra`): kimenet `port`, `bind_address`, kilépési kód 0; a harmadik mező felvétele után a `static_assert n == 3` mellett fordul.

Az X-macro változatban a taglista egy `#define FIELDS(X) X(int, port) X(std::string_view, bind_address)` típusú makróban él, és a `struct` ugyanebből a makróból épül. Mindkét változat egyetlen forrásból dolgozik, ezért egyik sem csúszik el új tag felvételekor. A különbség: a reflexiós változat bármely, szabadon megírt `struct`-ra működik, de kísérleti fordítót igényel; az X-macro minden C++20-as fordítón működik, viszont a `struct`-ot a makrósémához kell igazítani, és csak az így generált típusokra alkalmazható. *Ellenőrzési feltétel:* a reflexió nélküli minimálpélda a `lab-gcc` profilban figyelmeztetés nélkül fordul és 0 kóddal fut, a `consteval` változó pedig fordítási hiba; mindkettőt ellenőriztük. A reflexiós példát nem fordítottuk le: ehhez P2996-ot támogató fordító kell, amely a könyv profiljai között nem ellenőrzött.

3. `n == 0` mellett a második előfeltétel (`i < n`) hamis. `observe` módban a feltétel kiértékelődik, a violation handler lefut, majd a vezérlés a függvény törzsére lép, ahol a `contract_assert(n > 0)` szintén megsérül — a viselkedés tehát továbbra is hibás, csak nem áll le a program azonnal. `enforce` módban a handler után a program leáll, még a törzs előtt; a hiba tehát **belépéskor**, nem a visszatérésnél jelentkezik. *Ellenőrzési feltétel:* a válasznak meg kell neveznie, hogy a hiba a hívó oldal szerződésszegése, nem a hívott függvény belső hibája.

4. A `#embed` a legegyszerűbb, de a fájlnak a fordításkor elérhetőnek kell lennie, és a méret a binárisba fordítási időben bekerül; a build ezáltal a forrásfájl mellett a binárisra is függ. A generált fejléc ugyanezt a függést hozza létre, csak egy generátor-lépéssel, viszont hordozható minden C++20-as fordítón. A futásidejű beolvasás a legerősebb hordozhatóságot adja, de új hibautat (hiányzó fájl, részleges olvasás) és indítási költséget visz a programba. *Ellenőrzési feltétel:* a döntéshez mindhárom esetben add meg, hogy mi történik, ha a bináris tartalma megváltozik, és hogyan derül ki a CI-ban.

45.1.8 Összefoglaló

A C++26 nyelvi kitekintésében négy dolgot érdemes megjegyezni. Először: a státuszt mindig három szinten kell olvasni (elfogadott, kiadott, implementált), és a kísérleti képességek nem kerülnek be a laborprofilba. Másodszor: a statikus reflexió (`^^`, `std::meta`, splice) a makrós,

kézzel szinkronban tartott mintákat váltja ki — ez a legnagyobb hatású változás a rendszerszintű kódra. Harmadszor: a contracts a szerződésszegést (programozói hibát) választja el a futásidejű hibakezeléstől, és a kiértékelési módot a build választja meg, nem a forrás. Negyedszer: a kisebb újdonságok (`pack indexing`, `_`, strukturált kötés csomagja, `= delete("ok")`, számított `static_assert`-üzenet, `#embed`, erroneous behaviour) egyenként aprók, együtt viszont érezhetően csökkentik a kézi és a törékeny kódot. A C++26 utáni irányokat és a részletes státusztáblát a C++26 és későbbi irányok ellenőrzött státusszal rész adja; a könyvtári és konkurencia-oldalt a C++26 könyvtári és konkurencia-kitekintés tárgyalja.

45.2 C++26 könyvtári és konkurencia-kitekintés

Ez a rész szándékosan kitekintés, nem laboranyag. A könyv szerződése szerint a laborprofil alapja az ISO C++20, a C++23-at külön jelöltük, a C++26 pedig nem része a fordítási és tesztelési mátrixnak. A cél itt kettős: egyrészt fel kell ismerni, hogy a korábbi fejezetekben kézzel megépített megoldások melyik szabványos elem előképének tekinthetők, másrészt meg kell tanulni óvatosan olvasni a „C++26-ban lesz” típusú állításokat. Ez utóbbi nem formai kitétel: a szabvány státusza és a fordítók támogatása két külön dolog, és a generálás pillanatában ebben a szövegben nem történt forrásellenőrzés, így az alábbiakban minden konkrét papírszám és képesség mellett jelezzük, hogy az adott állítás ellenőrzése a kiválasztott forrásból még hátra van. A papírszámok a fejezettervből származó azonosítók; önmagukban nem bizonyítják sem az elfogadást, sem az implementáltságot.

45.2.1 Hogyan olvassuk a státuszjelöléseket

Négy dolgot érdemes külön tartani. A **papír** azonosítója (például P2300) a javaslatot nevezi meg, nem annak állapotát. Az **elfogadás** azt jelenti, hogy a bizottság beemelte a következő szabványtervezetbe; erről a WG21 jegyzőkönyvei és a nyilvános munkatervezet ad felvilágosítást. Az **implementáltság** azt jelenti, hogy egy konkrét compiler- és library-kiadásban a képesség elérhető; ezt feature-test makróval vagy compile-probe-bal lehet ellenőrizni, és verzióhoz kötött. A **stabilitás** pedig azt, hogy a könyvtár viselkedése nem változik-e visszamenőleg. A négy státusz nem esik egybe: egy elfogadott papír lehet, hogy egyik általunk használt fordítóban sem elérhető, és egy kísérleti implementáció létezhet már az elfogadás előtt is. Ezért az alábbi táblázat a két szintet külön oszlopban, dátummal adja meg.

A bizottsági oszlop forrása: a papírok a megadott revízióval; a C++26-ba kerülésüket a cppreference C++26-oldalának 2026-10-25-i állapota alapján rögzítettük (másodlagos forrás), a C++26 lezárásának tényét pedig a forrásjegyzék C++26 tétele. Az implementációs oszlop helyi mérés: a `lab-gcc` profil GCC 13.3.0 fordítója nem ismeri a `-std=c++2c` módot, és `-std=c++2b` mellett a megfelelő feature-test makrók (`__cpp_lib_senders`, `__cpp_lib_function_ref`, `__cpp_lib_copyable_function`, `__cpp_lib_hive`, `__cpp_lib_inplace_vector`, `__cpp_lib_hazard_pointer` `__cpp_lib_rcu`) nincsenek definiálva. Más fordítók támogatását a saját profiljukban, feature-test makróval kell mérni.

Korábbi labormegoldás	C++26 irány	Papír	Bizottsági státusz (2026-10-25)	`lab-gcc`
coroutine-alapú task (Aszinkron I/O adapter és coroutine-labor (35.5))	`std::execution` sender/receiver	P2300R10	C++26 része; technikailag lezárt, ISO-kiadás nem igazolt	nem támogatott
`move_only_functio` (Callable és forwarding eszközök: move_only_function, forward_like, bind_back (43.3))	`function_ref`, `copyable_function`	P0792R14, P2548R6	C++26 része	nem támogatott
`flat_map`/`flat_set` (flat_map és flat_set: rendezett tárolás folytonos memóriában (43.1))	`hive`, `inplace_vector`	P0447R28, P0843R14	C++26 része	nem támogatott
`shared_ptr` és visszanyerés (shared ownership, reclamation és konkurens review (34.6))	hazard pointer, RCU	P2530R3, P2545R4	C++26 része	nem támogatott
kézi bounds-ellenőrzés	hardened standard library	P3471R4	C++26 része	nem támogatott (nincs C++26 mód)

45.2.2 A konkurencia: sender/receiver és a coroutine-task viszonya

A korábbi coroutine-laborban (Aszinkron I/O adapter és coroutine-labor (35.5)) egy `task<T>`-t építettünk, amely `co_await`-tel felfüggeszti a hívót, a `promise_type` tárolja az eredményt vagy a kivételt, és a `coroutine_handle` élettartama a hívó felelőssége. Ez a modell akkor kényelmes, ha a futtatás egy szálhoz és egy végrehajtóhoz kötött, és a felfüggesztési pontokat kézzel jelöljük ki.

A `std::execution` (P2300) egy ennél általánosabb, deklaratív modellt javasol. A **sender** egy value-szemantikájú műveletleíró (a koncepció a mozgathatóságot követeli meg; az egyszerű sender típusok másolhatóak is lehetnek, a move-only csak egy megvalósítási lehetőség), amely deklarálja, milyen befejezési csatornái vannak (sikerérték, hiba, leállítás). A **receiver** fogadja ezeket a befejezéseket. A kettő összekapcsolásából jön létre az **operation state**, amely egyetlen, egyszer elindítható aszinkron műveletet reprezentál, és amelynek élettartama a befejezésig tart. A szerződés kulcselemei: a sender, a receiver és az operation state **külön követelményeket** kap (a sender és a receiver mozgatható, nem feltétlenül csak mozgatható; az operation state, amelyet nem mozgatunk az elindítása után, élettartam-kötött), a befejezés pontosan egyszer történik meg, és

a befejezés után a receiver nem hívható újra. A `co_await` ebben a modellben nem az egyetlen kompozíciós eszköz: a sender-ek `then`, `when_all`, `let_value` jellegű adapterekkel gráffá fűzhetők, és a coroutine-task egy ilyen gráf csomópontjaként is megjelenhet.

A gyakorlati következtetés a mi szintünkön az, hogy a kézi `task` nem elavult, hanem egy szűkebb szerződés: egy szálon, egy végrehajtón, explicit felfüggesztési pontokkal. Ha a C++26-os modell elérhetővé válik a laborprofilban, a `task` belső implementációja és a hívási felület is változhat, de a tanult élettartamkérdések — ki birtokolja a `coroutine_handle`-t, meddig él a frame, mi történik a megszakításkor — változatlanul relevánsak maradnak.

45.2.3 Tárolók és callable-ek: a korábbi saját megoldások helyettesítése

A `flat_map`/`flat_set` (flat_map és flat_set: rendezett tárolás folytonos memóriában (43.1)) folytonos memóriában tartja a kulcs-érték párokat, és a beszúrás átrendezéssel jár. A `hive` egy másik kompromisszum: bucketekbe szervezett, rendezetlen tároló, amelynek fő ígérete, hogy a beszúrás nem invalidálja a többi elemre mutató pointert és referenciát, a törlés pedig csak a törölt elemre hat. Ez pont az a tulajdonság, amiért a `std::vector`-alapú megoldásnál `stable` indexeket kellett kézzel fenntartanunk. Az `inplace_vector<T, N>` fix kapacitású, de dinamikus méretű tároló, amelynek elemei magában az objektumban foglalnak helyet; nem allokál, így a „van-e elég hely” kérdés a beszúrás előfeltételévé válik, és a túlcsordulás kezelése az API választásától függ — ezt a részletet a konkrét könyvtár dokumentációjából kell ellenőrizni.

A callable-eknél a `move_only_function` (Callable és forwarding eszközök: move_only_function, forward_like, bind_back (43.3)) után a `function_ref` a nem tulajdonló, olcsón másolható nézetet jelöli: nem tartja életben a hívott objektumot, ezért a hívott callable élettartamának a használat teljes idejét fednie kell. A `copyable_function` ezzel szemben értékszemantikájú, másolható típus-törlést ad. A korábbi laborban mindkettőt kézzel modelleztük; a C++26-os elemek ezeket a szerződéseket emelik standard szintre, de a lifetime-szabály nem enyhül: a `function_ref` ugyanúgy danglingolhat, mint a saját nézetünk.

45.2.4 Adatpárhuzamosság: simd, linalg, submdspan

A `std::simd` data-parallel típusokat javasol: egy érték több lane-t hordoz, a műveletek elemenként hatnak, és a lane-szám implementációfüggő. Ez nem helyettesíti a szálakat, hanem a szálon belüli adatszintű párhuzamosságot teszi hordozhatóvá — pont azt a kézi, compiler-intrinsic alapú vektorizálást, amit a teljesítménylaborban (Optimalizálási labor correctness kapukkal (37.6)) csak méréssel mertünk volna állítani. A `std::linalg` BLAS-szerződésű lineáris algebrai műveleteket ad, a `submdspan` pedig a C++23-as `mdspan` (mdspan mélyebben: layoutok, accessorok és C-interop (43.2)) szeletelését egészíti ki. Mindhárom esetben a numerikus pontosság, a lane-szám és a mögöttes hardver nem része a hordozhatósági ígéretnek; ezeket targetprofilhoz kell kötni, ahogy eddig is tettük.

45.2.5 Visszanyerés: hazard pointer és RCU

A `shared_ptr` konkurens használata (shared ownership, reclamation és konkurens review (34.6)) atomikus referenciaszámlálással működik, ami olvasási oldalon is írási forgalmat generál a közös kontrol blokkra. A **hazard pointer** (P2530) ezt úgy kerüli el, hogy minden olvasó egy thread-local listában publikálja, mely csomópontot használja éppen; a felszabadító csak akkor szabadítja fel a csomópontot, ha egyik publikált hazard pointer sem mutat rá. Az **RCU** (P2545) ennél is tovább megy: az olvasók lock nélkül dolgoznak egy verzióval, az írók új verziót publikálnak, a régi felszabadítása pedig egy grace period után történik, amikor már biztos, hogy nincs olvasó azon a

verzión. Mindkettő lock-free struktúrák visszanyerési mintája; a helyesség feltétele a publikálás és a visszanyerés közötti szinkronizációs rend, amelyet a korábbi happens-before fejezet (Data race, happens-before és publikalás (15.2)) szerint kell indokolni. A standard library ezekből várhatóan eszközöket ad, nem kész adatszerkezeteket.

45.2.6 Hardened library és kisebb kiegészítések

A **hardened standard library** (P3471) irányelve, hogy a standard könyvtár előfeltétel-sértéseit — például egy `vector::operator[]` tartományon kívüli indexelését — diagnosztizálhatóvá tegye, ahelyett hogy csendben nem definiált viselkedésbe futnánk. Ez nem azonos a fordító `-D_GLIBCXX_ASSERTIONS` kapcsolójával, és nem garantálja, hogy minden implementáció ugyanazt a viselkedést választja; a lényeg, hogy a könyvtár saját szerződése explicit legyen. A **telítő aritmetika** a túlcsordulást a típus szélső értékén megállító műveleteket ad — hasznos jelfeldolgozásban és bizonytalan bemenetű összegzésnél, de nem helyettesíti a C-ben tanult overflow-ellenőrzést (Promóciók, signedness és túlcsordulás (4.2)). A `std::text_encoding` kódlap-azonosítót ad (IANA-név és MIB-szám), amellyel a korábbi locale-kezelést (Szövegkódolás, locale és UTF-8 C-ben (7.5), Szövegkódolás, locale és random (32.5)) lehet pontosítani. A `<debugging>` header `breakpoint()` és `is_debugger_present()` jellegű eszközöket javasol, amelyek a debugger-labor (Debugger, core dump és optimalizált program (16.1)) kézi trükkjeit standardizálnák.

45.2.7 Kipróbálás a gyakorlatban

A kipróbálás első lépése nem az, hogy C++26-os kódot írunk, hanem hogy megmérjük, mit tud a rendelkezésre álló eszköz. A fordítási mód támogatása önmagában nem bizonyítja a könyvtár támogatását:

```
for std in c++20 c++23 c++26 c++2c; do
  printf '%-6s ' "$std"
  if g++ -std=$std -x c++ -fsyntax-only /dev/null 2>/dev/null; then
    echo "mód elfogadva"
  else
    echo "mód nem támogatott"
  fi
done
```

A könyvtári elemeket feature-test makróval vagy közvetlen include-próbával érdemes ellenőrizni. Az alábbi példa szándékosan nem állít semmit a támogatásról; csak a mérés mintáját mutatja, és a makrónév a választott könyvtár dokumentációjából ellenőrizendő:

```
// probe_inplace_vector.cpp - a fordítás sikeressége a mérés, nem a program
 ↪  futása
#include <version>

#if defined(__cpp_lib_inplace_vector)
#  include <inplace_vector>
int main() { std::inplace_vector<int, 4> v; v.push_back(1); return 0; }
#else
#  error "std::inplace_vector nem elerheto: a kepesseg hianyzik ebben a
 ↪  fordito/stdlib kombinacioban"
#endif
```

```
g++ -std=c++20 -O2 -Wall -Wextra -pedantic -o demo_fixed_queue
 ↪  demo_fixed_queue.cpp
# a támogatás verziófüggő: hiányzó szolgáltatásnál a próba #error-ral BUKIK (nem
 ↪  lehet képességbizonyíték)
clang++ -std=c++26 -stdlib=libc++ -fsyntax-only probe_inplace_vector.cpp
```

A lenti C++20-as példa azt a két mintát közelíti, amelyeket a C++26 standardizálna: egy fix kapacitású, nem allokáló sor és egy nem tulajdonló callable-nézet. A kódot ebben a szövegben nem fordítottuk le és nem futtattuk; a várt viselkedés leírás, nem mért eredmény.

```
// demo_fixed_queue.cpp - C++20, saját közelítés; nem tesztelt
#include <array>
#include <cstddef>
#include <cstdio>
#include <optional>
#include <utility>

// Követelmény T-re: default-konstruálható és move-olható (a queue saját
 ↪  tárolója
// std::array<T, N>; hogy egy T elem konstruálása/mozgatása allokál-e, T
 ↪  műveletein múlik).
template <class T, std::size_t N>
class fixed_queue {
public:
    static constexpr std::size_t capacity = N;

    bool push(T v) {
        if (size_ == N) return false;
        data_[tail_] = std::move(v);
        tail_ = (tail_ + 1) % N;
        ++size_;
        return true;
    }

    std::optional<T> pop() {
        if (size_ == 0) return std::nullopt;
        T v = std::move(data_[head_]);   // a kivett elem helye moved-from
 ↪  állapotú marad
        head_ = (head_ + 1) % N;
        --size_;
        return v;
    }

    std::size_t size() const { return size_; }

private:
    std::array<T, N> data_{};
    std::size_t head_ = 0, tail_ = 0, size_ = 0;
};

template <class F>
int apply_twice(F& f, int x) { return f(f(x)); }  // nem tulajdonol: F
 ↪  élettartama a hívóé
```

```
int main() {
    fixed_queue<int, 4> q;
    std::printf("capacity=%zu\n", fixed_queue<int, 4>::capacity);
    for (int i = 0; i < 6; ++i) {
        std::printf("push(%d) -> %d\n", i, q.push(i) ? 1 : 0);
    }
    while (auto v = q.pop()) std::printf("pop %d\n", *v);

    int base = 10;
    auto add_base = [base](int v) { return v + base; };
    std::printf("apply_twice=%d\n", apply_twice(add_base, 1));
}
```

Az `apply_twice` szándékosan referenciát vesz át: ez a `function_ref` szerződésének lényege, hogy a hívott objektum élettartama a hívó felelőssége. Két esetet kell szétválasztani. A **szinkron felhasználás** szabályos akkor is, ha a callable temporary: a temporary a teljes kifejezés végéig él, a hívott függvény pedig ezen belül használja fel. A **tárolt nézet** viszont dangling: ha a temporaryből képzett nézetet a teljes kifejezésen túl megtartjuk, a hívás már egy megszűnt objektumot ér el. Mivel a `std::function_ref` a `lab-gcc` profilban nem érhető el, a két esetet egy minimális, kézzel írt nézettel mutatjuk meg:

```
// fn_ref.cpp - C++20: function_ref-szerű, nem tulajdonló callable-nézet
#include <cstdio>
#include <memory>
#include <type_traits>
#include <utility>

template <class Sig> class fn_ref;
template <class R, class... A>
class fn_ref<R(A...)> {
public:
    template <class F>
        requires (!std::is_same_v<std::remove_cvref_t<F>, fn_ref>)
    fn_ref(F&& f) noexcept                                  // nem másol: csak
 ↪  címet tárol
        : obj_(std::addressof(f)),
          call_([](void* o, A... a) -> R {
              return
               ↪  (*static_cast<std::remove_reference_t<F>*>(o))(std::forward<A>(a)...);
          }) {}
    R operator()(A... a) const { return call_(obj_, std::forward<A>(a)...); }
private:
    void* obj_;
    R (*call_)(void*, A...);
};

int apply_twice(fn_ref<int(int)> f, int x) { return f(f(x)); }   // csak a hívás
 ↪  idejére

int main(int argc, char**) {
    int base = 10;
```

```
    // Szabályos: a temporary lambda a teljes kifejezés végéig él, az
     ↪  apply_twice
    // közben, szinkron módon használja fel.
    std::printf("apply_twice=%d\n", apply_twice([base](int v) { return v + base;
     ↪  }, 1));
    if (argc > 1) {
        // HIBA: a nézet túléli a teljes kifejezést; a temporary lambda már
        // megszűnt, amikor a stored-ot hívjuk - dangling.
        fn_ref<int(int)> stored = [base](int v) { return v + base; };
        std::printf("stored=%d\n", stored(1));
    }
}
```

Buildjegyzőkönyv (`lab-gcc` profil). `g++ -std=c++20 -Wall -Wextra -Wpedantic -O1 -g -fsanitize=address,undefined fn_ref.cpp` figyelmeztetés nélkül fordult; argumentum nélkül `apply_twice=21`, 0-s kód; a `neg` argumentummal a tárolt nézet hívására az ASan `stack-use-after-scope` hibát jelzett. A `std::function_ref` (`__cpp_lib_function_ref`) sem a GCC 13.3-ban, sem a Clang 22 / libc++-ban nem volt elérhető.

45.2.8 Tipikus hibák

A leggyakoribb hiba, hogy valaki a fejezettervben vagy egy hírportálon látott papírszámból arra következtet, hogy a képesség „már a C++26-ban van”, és éles kódba teszi. Ez két okból hibás: az elfogadás és az implementáltság nem ugyanaz, és a laborprofilunk C++20. A második hiba a `-std=c++26` és a `-std=c++2c` összekeverése: a fordítók a kiadás előtt gyakran a munkanevet fogadják el, és a két kapcsoló viselkedése kiadásonként változhat. A harmadik, hogy a `function_ref`-et tulajdonló típusnak használják, és egy temporary callable-ből képzett nézetet a teljes kifejezésen túl eltárolnak (a hívás idejére átadott temporary önmagában szabályos). A negyedik, hogy a `hive` pointerstabilitását kihasználva egy pointert a tároló megszűnése után is használnak — a stabilitás a tároló élettartamán belül értendő. Végül tipikus hiba a hardened módot biztonsági garanciának tekinteni: az előfeltétel-sértés diagnosztizálása nem azt jelenti, hogy a program helyes.

45.2.9 Feladatok

1. feladat (magyarázó). Rendeld hozzá a négy előfeltétel-részben (shared ownership, reclamation és konkurens review (34.6), Aszinkron I/O adapter és coroutine-labor (35.5), flat_map és flat_set: rendezett tárolás folytonos memóriában (43.1), Callable és forwarding eszközök: move_only_function, forward_like, bind_back (43.3)) épített megoldásokat a fejezetben szereplő C++26-os irányokhoz, és mindegyiknél indokold, mi a nyereség és mi a megmaradó kockázat.

2. feladat (kódolási). Egészítsd ki a `fixed_queue` példát egy `pop_all` művelettel, amely egy nem tulajdonló callable-n keresztül dolgozza fel az elemeket. Fogalmazd meg a callable élettartamára vonatkozó előfeltételt, és mutasd meg, hogy a tároló nem allokál.

3. feladat (diagnosztikai). Készíts olyan mérési jegyzőkönyvet, amely legalább három C++26-os könyvtári elem elérhetőségét vizsgálja a rendelkezésre álló fordítóval, és rögzíti a compiler- és library-verziót.

45.2.10 Megoldások

1. megoldás. A coroutine-task (Aszinkron I/O adapter és coroutine-labor (35.5)) a `std::execution` sender/receiverhez kapcsolódik: a nyereség a kompozíció és a befejezési csatornák deklaratív kezelése, a kockázat a hosszabb élettartam-lánc (operation state) és a megszakítás szemantikájának újbóli megtanulása. A `move_only_function` (Callable és forwarding eszközök: move_only_function, forward_like, bind_back (43.3)) a `function_ref`/`copyable_function` irányhoz: a nyereség a nem tulajdonló hívás olcsó átadása, a kockázat a dangling, amelyet a típus nem akadályoz meg. A `flat_map`/`flat_set` (flat_map és flat_set: rendezett tárolás folytonos memóriában (43.1)) a `hive`/`inplace_vector` irányhoz: a nyereség a pointerstabilitás, illetve az allokációmentesség, a kockázat a fix kapacitás miatti előfeltétel és a rendezetlenség. A `shared_ptr` (shared ownership, reclamation és konkurens review (34.6)) a hazard pointer/RCU irányhoz: a nyereség az olvasási oldal írásmentessége, a kockázat a grace period és a publikálás helyes indoklása. Mindegyiknél közös, hogy a státuszt a konkrét implementációból kell ellenőrizni.

2. megoldás. A `pop_all` sablonparaméterként referenciát vegyen át, ne értéket:

```
template <class F>
void pop_all(F& sink) {
    while (auto v = pop()) sink(*v);
}
```

Az előfeltétel: a `sink` élettartama fedje a `pop_all` teljes futását, és a `sink` ne tegye tönkre a `fixed_queue`-t (például ne hívjon `push`-t rekurzívan). Az állítás hatóköre: a `fixed_queue` **saját tárolója** nem allokál (tagjai csak `std::array<T, N>` és skalárok), és az `int` példányra a `push`/`pop`/`pop_all` nem hív allokáló függvényt. Általános `T`-re ez nem következik: a `T` alapértelmezett konstrukciója, mozgatása és a `sink` allokálhat; a példa move-only típusokkal is működik a `std::move` használata miatt, a kivett elem helye moved-from állapotú marad, és a `T`-nek default-konstruálhatónak kell lennie. ezt kódolvasással és — a laborban — allokációfigyelő eszközzel lehet igazolni, nem e szöveg állításaként.

3. megoldás. A jegyzőkönyv mintája:

```
g++ --version | head -n1
clang++ --version | head -n1
for std in c++20 c++23 c++26 c++2c; do
  g++ -std=$std -x c++ -fsyntax-only /dev/null 2>/dev/null \
    && echo "$std: mód elfogadva" || echo "$std: mód nem támogatott"
done
```

Ezután elemenként külön próbafájlt kell fordítani, és rögzíteni kell a feature-test makró jelenlétét vagy hiányát. Az értékelés szabálya: a „mód elfogadva” csak a nyelvi módra vonatkozik, a könyvtári elemre nem; a hiányzó makró nem bizonyítja, hogy a képesség nincs a szabványban, és a jelenlévő makró nem bizonyítja, hogy a viselkedés stabil. A jegyzőkönyvbe a dátumot és a pontos verziókat is fel kell venni, különben a mérés nem reprodukálható.

45.2.11 Összefoglaló

A C++26 könyvtári irányai jól olvashatók a könyv korábbi laborjainak folytatásaként: a `std::execution` a coroutine-task általánosítása, a `function_ref`/`copyable_function` a kézi callable-nézet standardizálása, a `hive`/`inplace_vector` a folytonos és stabil tárolás más kompromisszumai, a `simd`/`linalg`/`submdspan` az adatpárhuzamos és többdimenziós számítás eszközei, a hazard pointer és az RCU a visszanyerés lock-free mintái, a hardened library pedig a

könyvtári előfeltételek diagnosztizálásának iránya. Egyik sem része a laborprofilnak, és egyik állítás sem tekinthető ellenőrzöttnek addig, amíg a konkrét papír státuszát, a compiler- és library-verziót, valamint a feature-test makrót nem ellenőriztük. A helyes munkamód az, amit a könyv eddig is követett: először mérni, aztán állítani — és a mérést dátummal, verzióval és a korlátok kimondásával rögzíteni.

IV. rész: Kitekintés és referencia

46 Haladó irányok, új szabványok és referencia-függelék

46.1 C++23 összefoglaló és migrációs táblázat

A kurzus alapja a C++20, a C++23-as elemeket pedig végig külön jelöléssel, esetenként fallbackkel használtuk. Ez az alfejezet abból a szempontból zárja le a C++ részt, hogy a szétszórt jelöléseket egyetlen migrációs leltárba rendezi: milyen képesség, melyik a minimumstandard, mit kell a laborprofilban ellenőrizni, és mi a visszaút, ha a kiválasztott fordító vagy standard library nem tudja. Ez nem újratanítás: a szemantikát a Záróreview és C/C++ összehasonlítás már rögzítette, a mérési és feature-detection technikát pedig a Feature-test, standard library és build capability írta le. Itt ezekre építünk.

46.1.1 A migráció alapszabálya: képesség nem egyenlő elérhetőséggel

A migráció legfontosabb fogalmi különbsége, hogy a **szabványos képesség** (capability) és a **támogatottság** (availability) két külön dolog. A szabvány azt mondja meg, hogy egy kifejezés, típus vagy fejléc milyen szemantikával létezik; az implementáció azt mondja meg, hogy a te konkrét fordító-, illetve libstdc++/libc++ kiadásodban ez tényleg ott van-e. A kettő közé nem lehet egyenlőségjelet tenni, és nem lehet egyikből a másikra következtetni.

Egy C++23 képesség elérhetőségét képességenként, a következő döntési fa szerint vizsgáljuk:

1. **Nyelvi képesség** (például `if consteval`, deducing this): a fordító előre definiált **nyelvi** makrója dönt (`__cpp_if_consteval`, `__cpp_explicit_this_parameter`), a várt vagy annál nagyobb értékkel. Könyvtári makrót ilyenkor nem keresünk — nincs is.
2. **Könyvtári képesség** (például `std::byteswap`, `std::expected`): a `<version>`-ból (vagy a fejlécből) olvasható **könyvtári** makró dönt (`__cpp_lib_byteswap`, `__cpp_lib_expected`). Külön nyelvi makrót nem követelünk meg; a `-std=c++23` mód természetesen előfeltétel.
3. **Kombinált képesség** (például az `expected` monadikus műveletei, amelyek a library makró mellett nyelvi funkciót, pl. `constexpr`-t használnak): mindkét makrót ellenőrizzük, a megfelelő értékkel; a sémát képességenként a táblázat „Teszt” oszlopa rögzíti.
4. **Minden esetben** egy fordítási (és ahol a szerződés futásidejű, futtatási) próba zárja a döntést: a kód lefordul, és a viselkedés megfelel a szerződésnek — nem csak a deklaráció látszik.

Csak olyan képességnél kell egyszerre nyelvi és könyvtári makró, amely mindkét rétegre támaszkodik; ez a táblázat elemeinél nem fordul elő.

Ha a rá vonatkozó feltétel hiányzik, akkor a képesség a te profilodban nem használható, függetlenül attól, hogy a szabvány szerint létezik. A `-std=c++23` kapcsoló önmagában csak a nyelvi módot kéri; nem garantálja, hogy a library is követte. A fordítók egy része a módot régebbi nevén (`-std=c++2b`) is elfogadja, ezt a laborprofilban rögzíteni kell, mert két különböző build ugyanazt a forrást másképp láthatja.

A táblázatokban a küszöb a C++23 szabványszövegben (N4950) szereplő érték; a feltétel `makró >= küszöb`. Az értékeket a GCC 13.3 (libstdc++) és a Clang 22 (libc++) `-std=c++23 -dM -E` kimenetével vetettük össze: ahol egy könyvtár definiálja a makrót, az érték egyezett; a hiányzó makró a képesség hiányát jelzi (például a GCC 13.3-ban a `__cpp_lib_print`, `__cpp_lib_mdspan`, `__cpp_lib_generator`). Minden képesség minimumstandardja C++23.

46.1.2 Capability tábla a kurzus C++23 példáihoz

Nyelvi képességek (nyelvi makró):

Képesség	Makró	Küszöb	Fallback C++20-hoz
deducing this	`__cpp_explicit_this_parameter`	202110L	ref-minősített overloadok
if consteval	`__cpp_if_consteval`	202106L	lásd (1)
constexpr bővítések	`__cpp_constexpr`	202211L	runtime-ra korlátozás

Könyvtári képességek (`<version>`-makró):

Képesség	Makró	Küszöb	Fallback C++20-hoz
expected	`__cpp_lib_expected`	202211L	lásd (2)
print, println	`__cpp_lib_print`	202207L	format + kiírás
mdspan	`__cpp_lib_mdspan`	202207L	lásd (3)
generator	`__cpp_lib_generator`	202207L	callback-iteráció
move-only function	`__cpp_lib_move_only_function`	202110L	function, ha másolható
zip nézet	`__cpp_lib_ranges_zip`	202110L	kézi indexelés
chunk nézet	`__cpp_lib_ranges_chunk`	202202L	kézi indexelés
slide nézet	`__cpp_lib_ranges_slide`	202202L	kézi indexelés
join with nézet	`__cpp_lib_ranges_join_with`	202202L	transform + copy
stride nézet	`__cpp_lib_ranges_stride`	202207L	kézi indexelés
Descartes-szorzat nézet	`__cpp_lib_ranges_cartesian_product`	202207L	egymásba ágyazott ciklus
ranges to	`__cpp_lib_ranges_to_container`	202202L	lásd (4)
from range konstruktorok	`__cpp_lib_containers_ranges`	202202L	lásd (4)
byteswap	`__cpp_lib_byteswap`	202110L	kézi shift/maszk
to underlying	`__cpp_lib_to_underlying`	202102L	lásd (5)
unreachable	`__cpp_lib_unreachable`	202202L	assert + bővítmény
stacktrace	`__cpp_lib_stacktrace`	202011L	`backtrace()`

A képességek teljes neve sorrendben: `std::expected`, `std::print`/`std::println`, `std::mdspan`, `std::generator`, `std::move_only_function`, `views::zip`, `views::chunk`, `views::slide`, `views::join_with`, `views::stride`, `views::cartesian_product`, `ranges::to`, a `from_range` konstruktorok, `std::byteswap`, `std::to_underlying`, `std::unreachable`, `std::stacktrace`.

A hivatkozott fallbackek:

1. `std::is_constant_evaluated()` (C++20).
2. `std::variant<T, E>` alapú saját `Result<T, E>` típus, `has_value()`/`value()`/`error()` taggal.
3. Saját, kiterjedést és lépésközt tároló view struct, kézi indexszámítással.
4. `std::vector<T> v(r.begin(), r.end());` — common range esetén.
5. `static_cast<std::underlying_type_t<E>>(e)`.

A táblázat nem teljes körű C++23-leltár, hanem a könyvben ténylegesen előforduló elemek inventoryja. A hiányzó C++23-as képességek (például a formázási bővítések egy része) migrációs szempontból ugyanezt a mintát követik: makró, fejléc, fallback.

46.1.3 Nyelvi képességek

A **deducing this** (explicit object parameter) a tagfüggvény implicit `this` paraméterét teszi láthatóvá és template-paraméterré. A feltétel, hogy a paraméterlista első eleme `this` előtagú legyen; ekkor a függvény nem kap külön implicit objektumparamétert. Az eredmény az, hogy ugyanaz a függvénytörzs szolgálja ki az lvalue és rvalue hívást, a value category a `Self`-en keresztül megőrizve:

```
struct Builder {
    std::string name;
    template <class Self>
    auto&& with_name(this Self&& self, std::string n) {
        self.name = std::move(n);
        return std::forward<Self>(self);   // lvalue -> Builder&, rvalue ->
         ↪ Builder&&
    }
};
```

A C++20-as fallback két külön overload: `Builder& with_name(std::string) &` és `Builder&& with_name(std::string) &&`, azonos törzzsel. A hívási oldal mindkét esetben `Builder{}.with_name("x")`, illetve `b.with_name("x")`.

Az `if consteval` ágat nyit aszerint, hogy a kiértékelés constant evaluation alatt történik-e; a hamis ág csak futásidőben értékelődik. Szintaktikailag nem kizárólag `constexpr` függvényben megengedett (egy közönséges függvényben szabályos, de ott értelmes konstans kiértékelési használata nincs: az `if consteval` igaz ága csak konstans kiértékelésből érhető el). Az igaz ág *immediate function context*, így ott `consteval` függvény is hívható. A C++20-as megfelelője az `if (std::is_constant_evaluated())`, amely **nem ekvivalens**: nem hoz létre immediate function contextet, ezért `consteval` függvényt nem lehet benne hívni, és a nem-constexpr műveleteket kézzel kell elkerülni; immediate functiont hívó kódhoz C++20-ban más tervezés kell (például külön `consteval` változat). A `constexpr` bővítések (például `static` és `thread_local` változók constexpr függvényben) ugyanebbe a családba tartoznak: a makró értékét ellenőrizni kell, mert a bővítések nem egyetlen kapcsolóhoz kötődnek.

A `std::unreachable()` azt fejezi ki, hogy az adott pontot a program soha nem érheti el. Ez nem egy „ez úgyis ritka” jelölés, hanem szerződés: ha mégis elérhető, a viselkedés nem definiált, és az optimalizáló erre építve tetszőleges kódot generálhat. Csak bizonyított bemeneti invariáns mellett ajánlott. Egy fix alaptípusú `enum class E : int` tárolhat nem nevesített, reprezentálható értéket is (`static_cast<E>(99)`), ezért egy default nélküli, minden felsorolt enumerátort kezelő `switch` még átengedhet ilyen értéket az `unreachable()`-ig: a lefedettség önmagában nem bizonyítja az elérhetetlenséget. Külső vagy ellenőrizetlen enumértékhez hibakezelő `default` kell. Fallbackként `assert(false)` használható, ami legalább a debug buildben megáll.

46.1.4 Könyvtári képességek

A `std::expected<T,E>` egy value-or-error típus: vagy `T`-t, vagy `E`-t tartalmaz, soha nem mindkettőt, és nem allokál. Feltétele, hogy `T` és `E` destruktálható (és a megfelelő objektumtípus) legyen. A `T` és `E` **lehet ugyanaz a típus**: `expected<int,int>` és `expected<std::error_code,std::error_code>` érvényes; a siker- és hibakonstrukciót az `unexpected`/`unexpect` címkézés választja el (`expected<int,int>{5}` érték, `expected<int,int>{std::unexpec` hiba, eltérő `has_value()` eredménnyel). C++23-ban monadikus műveletei is vannak (`.and_then`, `.transform`, `.or_else`). A `std::print`/`std::println` a `<print>` fejlécben él, és a `std::format` szemantikáját használja a streams állapotgép helyett. A `std::mdspan` többdimenziós, nem tulajdonló nézet kontinuus tárolóra, saját extents és layout policy szerződéssel; a C++23-as többargumentumos `operator[]` a kulcs. A `std::generator` coroutine-alapú, lusta sorozat, amelyet `co_yield`-tel táplálunk. A `std::move_only_function` type-erasure wrapper, amely maga csak mozgatható (a *wrapper* nem másolható); a tárolt *target* ettől függetlenül lehet másolható is (egy másolható lambda is eltárolható benne), de lehet move-only is — ott kell, ahol a `std::function` másolási követelménye miatt nem lehetne `unique_ptr`-t capture-elni.

A **Ranges bővítések** (`views::zip`, `chunk`, `slide`, `join_with`, `stride`, `cartesian_product`) a pipeline-ba illeszkedő, lusta nézetek; a `ranges::to<C>` és a `from_range_t` konténer-konstruktorok a range-ből konténerbe konvertálás hivatalos útját adják. A `std::byteswap` bájtsorrend-megfordítást végez (nem önmagában endian-konverzió: host/network konverzióhoz a `std::endian::native` ellenőrzésével együtt kell alkalmazni), a `std::to_underlying` enumról a mögöttes egész típusra konvertál, a `std::stacktrace` pedig szimbolizált hívási láncot ad — utóbbi erősen implementációfüggő, és a szimbolizációhoz gyakran külső library kell.

46.1.5 Kidolgozott példa: capability probe fallbackkel

Az alábbi példa a migráció tipikus alakját mutatja: egy vékony aliastípus, amely a makrótól függően a szabványos vagy a fallback implementációra képződik. A konstrukciót szándékosan egységesre vettük, hogy a hívási oldal ne változzon.

```
// probe_result.cpp
#include <version>
#include <charconv>
#include <string>
#include <string_view>
#include <variant>

#if defined(__cpp_lib_expected) && __cpp_lib_expected >= 202202L
#include <expected>
template <class T, class E> using Result = std::expected<T, E>;
inline constexpr const char* kBackend = "std::expected";
template <class T, class E> Result<T, E> fail(E e) { return
 ↪  std::unexpected(std::move(e)); }
#else
template <class T, class E>
struct Result {
    std::variant<T, E> v;
    bool has_value() const { return v.index() == 0; }
    T& value() { return std::get<0>(v); }
    E& error() { return std::get<1>(v); }
};
```

```
inline constexpr const char* kBackend = "fallback/variant";
template <class T, class E> Result<T, E> fail(E e) {
    return Result<T, E>{std::variant<T, E>{std::in_place_index<1>,
     ↪  std::move(e)}};
}
#endif

Result<int, std::string> parse_int(std::string_view s) {
    int value{};
    auto [ptr, ec] = std::from_chars(s.data(), s.data() + s.size(), value);
    if (ec == std::errc{} && ptr == s.data() + s.size())
        return Result<int, std::string>{value};            // value ág
    return fail<int, std::string>(std::string{"nem egesz: "} + std::string{s});
     ↪  // error ág:
    // std::expected-nél a hibaágat csak std::unexpected jelöli; a puszta E
     ↪  érték
    // nem konvertálódik (azonos T és E esetén értékként értelmeződne)
}

// Próbahívó: jó és rossz bemenettel (a buildparancsok teljes programot
 ↪  linkelnek).
#include <cstdio>
int main() {
    auto good = parse_int("42");
    auto bad  = parse_int("x1");
    std::printf("%s good=%d bad=%d\n", kBackend, good.has_value(),
     ↪  bad.has_value());
    return (good.has_value() && !bad.has_value() && good.value() == 42) ? 0 : 1;
}

# C++23 mód, a szabványos ág remélt útja:
g++ -std=c++23 -Wall -Wextra -Wpedantic -O2 probe_result.cpp -o probe_result

# Ugyanaz a forrás C++20 módban: itt a fallback ágnak kell fordulnia.
g++ -std=c++20 -Wall -Wextra -Wpedantic -O2 probe_result.cpp -o probe_result20
```

A példa korlátját érdemes kimondani: ez a shim nem teljes `expected`-helyettesítő, monadikus műveletek és `void`-specializáció nélkül. A célja az, hogy a migrációt egyetlen fordítási ponton tartsa. A fordítást a saját laborprofilodban kell elvégezni; a szöveg nem állít mért eredményt, és a `kBackend` értéke a te implementációdtól függ.

46.1.6 Tipikus hibák

A leggyakoribb hiba, hogy a build `-std=c++20`-szal zöld marad, mert a fallback ág fordul — a migráció csendben nem történt meg. Ez ellen build-time őr kell: `static_assert(__cpp_lib_expected)`, ha a szabványos ág kötelező. A második, hogy a nyelvi és a library makrót összekeverik: a fordító ismerheti a szintaxist, miközben a régi libstdc++ még nem adja a fejlécet. A harmadik a `std::unreachable()` visszaélése olyan értékre, amely előfordulhat — ez nem definiált viselkedés, nem „gyorsítás”. A negyedik, hogy a `<version>`-t nem include-olják, és a makró „hiányzik”; a library makrókat a megfelelő fejlécből vagy a `<version>`-ból kell olvasni, de a kettő nem mindig ekvivalens.

46.1.7 Összefoglaló

A C++23-migráció nem egyetlen kapcsoló, hanem képességenkénti döntés. Minden képességhez három dolgot kell rögzíteni: a minimumstandardot, a tényleges laborprofilbeli elérhetőséget (nyelvi és library makróval igazolva) és a fallbacket. A táblázat ezt a leltárt adja; a `Result<T,E>` példa pedig azt mutatja meg, hogyan tartható a hívási oldal változatlanul. A C++26 irányai külön fejezetben, külön ellenőrzéssel szerepelnek — itt szándékosan nem garantálunk semmit arról, ami még nem stabil.

46.1.8 Feladatok

1. **Rendszerezés.** Válogasd szét a fenti táblázat sorait nyelvi és könyvtári képességekre, és indokold, melyiknél nagyobb a migrációs kockázat, ha csak a fordítót frissíted, de a standard libraryt nem.
2. **Kódolás.** Egészítsd ki a `Result<T,E>` példát egy `unwrap_or(Result<T,E>, T fallback)` szabad függvénnyel mindkét ágon (az `expected` ág `using` alias, tagfüggvényt nem kaphat), és add meg a buildparancsot. Készíts olyan őrt, amely C++20 módban fordítási hibát ad, ha a fallback ág véletlenül a szabványos ágra képződne.
3. **Diagnosztika.** Egy CI-ban ugyanaz a forrás C++20-ban zöld, C++23-ban piros. Sorolj fel három lehetséges okot, és add meg, milyen makrókkal és parancsokkal szűkítenéd a hibát.
4. **Élettartam.** A `std::mdspan` és a `std::string_view` közös tulajdonsága, hogy nem tulajdonló nézetek. Fogalmazz meg egy szabályt arra, mikor biztonságos egy `mdspan`-t függvényből visszaadni.

46.1.9 Megoldások

1. Nyelvi képesség a deducing this, az `if consteval`, a constexpr bővítések és a `std::unreachable` szemantikai magja; könyvtári az `expected`, `print`, `mdspan`, `generator`, `move_only_function`, a Ranges bővítések, a `byteswap`, `to_underlying` és a `stacktrace`. A kockázat a könyvtári oldalon nagyobb: a nyelvi makró a fordítóval együtt jön, a library makró viszont attól függ, hogy a rendszeren melyik libstdc++/libc++ van linkelve — és konténerben ez gyakran nem az, amit a fordítóhoz vártunk. Ellenőrzés: a két build `-v` kimenete és a `<version>` tartalma alapján a profil egyezzen.

2. Az `expected` ágon a `Result` csak `using` alias, ezért **nem egészíthető ki tagfüggvénnyel**; mindkét ágra egy közös szabad függvény kell: `template <class T, class E> T unwrap_or(Result<T, E> r, T fb) { return r.has_value() ? std::move(r.value()) : std::move(fb); }` (az értéket `value()` útján vesszük ki; a `*this` az `expected` objektumot jelölné, nem a benne lévő `T`-t). A közös hívási oldal így `unwrap_or(parse_int(s), -1)` mindkét backenden. A backendet saját `constexpr` jelző (`kBackend`/`kStdExpected`) mutatja, nem az `NDEBUG`. Az őr a backendválasztáshoz kötött, nem az `NDEBUG`-hoz: az `expected` ágban (`#if defined(__cpp_lib_expected) …`) `static_assert(std::is_same_v<Result<int, std::string>, std::expected<int, std::string>>);` áll, a `#else` (fallback) ágban pedig egy `static_assert(!std::is_same_v<Result<int, std::string>, void>)`-szerű jelző helyett a `kBackend` értékét ellenőrzi a teszt. Ellenőrzési feltétel: a C++20-as buildben a `static_assert` nem fut le, a C++23-asban a szabványos ág aktív.

4. Szabály: egy nem tulajdonló nézetet (`mdspan`, `string_view`) csak akkor biztonságos visszaadni, ha a mögöttes tároló élettartama a hívó oldalán biztosított (a függvény paramétere által átadott, a hívónál tovább élő tároló), és a visszaadás után nem történik a tárolón élettartam-vagy újraallokációs invalidáció. *Helyes:* `std::span<const int> head(const std::vector<int>&`

`v) { return {v.data(), 2}; }` — a nézet `v`-re hivatkozik, amely a hívónál él. *Dangling:* `std::string_view bad() { std::string s = "abc"; return s; }` vagy `std::mdspan<int, ext2> bad(){ std::vector<int> v(6); return {v.data(), 2, 3}; }` — a helyi tároló a visszatéréskor megszűnik. Invalidáció: a nézett vektor `push_back` utáni újraallokációja (a kapacitás túllépése) a nézetet érvényteleníthetí; ilyenkor a nézetet újra kell képezni.

3. (a) A `-std=c++23` mellett a fordító új szintaxist fogad el, de a régi libstdc++ nem adja a fejlécet → hiányzó include. (b) A kód olyan konstrukciót használ, amely C++20-ban mást jelentett (például megváltozott overload-feloldás), és a régi build ezt nem fedte fel. (c) Egy harmadik féltől származó fejléc a saját `expected`-jét definiálja, ami ütközik. Szűkítés: `-E -dM` a makrókhoz, `-H` az include-lánchoz, `-fdiagnostics-show-template-tree` a template-hibákhoz, és két külön build könyvtár, hogy a régi object fájlok ne maszkolják a hibát.

46.2 C++26 és későbbi irányok ellenőrzött státusszal

Ez a szakasz tudatosan nem ismétli meg a fő kurzus anyagát (lásd C++23 összefoglaló és migrációs táblázat (46.1)), és nem állítja, hogy bármelyik alábbi képesség elérhető. Célja egy **státusz-ellenőrzési keretrendszer** és egy elsődleges forrásokhoz vezető térkép adása, amelyet a generálás, illetve az olvasás napján aktuális WG21- és compiler-/library-forrásokból kell kitölteni. A C++26 egészének státuszát a forrásjegyzék C++26 tétele rögzíti (2026-10-25: technikailag lezárt, DIS-szavazáson, ISO-kiadás nem igazolt), az egyes könyvtári képességekét pedig a C++26 könyvtári és konkurencia-kitekintés táblázata. Az itt szereplő, C++26 utáni irányok konkrét státusza viszont **nem ellenőrzött, kitöltendő mező**, nem pedig tényállítás; a lenti sablon ennek kitöltésére szolgál.

46.2.1 A „státusz” négy szintje

A C++ ökoszisztémában egy új képesség élete több, egymástól szigorúan megkülönböztetendő állapoton halad át, és a gyakorlati döntéseknél a négy szintet nem szabad összemosni.

1. **Javaslat (proposal)** – P-számú WG21-dolgozat. Lehet elvetett, aktív revízión eső, vagy plenáris szavazással elfogadott egy következő szabványba. Az elfogadás ténye a working draftba kerüléshez vezet, de önmagában nem tesz egy képességet „szabványossá”.
2. **Working draft** – az N-számú munkapiszkozat (például az N4950 a C++23 esetében). Ez a szabvány előkészítő szövege; a nyilvános draft hivatkozási alap, de nem az ISO által kiadott, jogilag közzétett szabvány. A draft és a megjelent ISO szöveg között eltérések lehetnek (szavazási javítások, szerkesztői pontosítások).
3. **Közzétett ISO szabvány** – a nemzeti testületek szavazásával jóváhagyott, az ISO által kiadott szöveg. A hivatalos jelölés és a végleges tartalom ehhez a szinthez kötődik; a fizetős ISO kiadvány és a working draft nem azonos.
4. **Implementációs támogatás** – az, amit a választott compiler- és standard library-verzió ténylegesen fordítani és futtatni képes. Ezt feature-test makrók (`__cpp_lib_*`, `__cpp_*`) és compiler-státuszoldalak alapján kell ellenőrizni, nem a javaslat szövege alapján.

Ezért a helyes kérdés sosem az, hogy „benne van-e C++26-ban?”, hanem az, hogy **melyik szinten** áll, **melyik revízión**, **melyik konkrét compiler/library párosítással**, és **mikor** ellenőriztük.

46.2.2 A forrás és a dátum rögzítése

A státuszt csak forrással és dátummal szabad megjelölni. A szakaszhoz használható táblázatsablon a következő; az első három oszlopot a tanulónak kell kitöltenie elsődleges forrásból, a negyedik az, amit a szöveg megenged.

Képesség (terület)	Szint: proposal / draft / ISO / implementált	Elsődleges forrás azonosítója (P-/N-szám, oldal, szakasz)	Ellenőrzés dátuma	Megjegyzés (korlát)
Reflection / compile-time generálás	kitöltendő	kitöltendő	kitöltendő	Nem helyettesíti a fordító támogatását
Contracts	kitöltendő	kitöltendő	kitöltendő	Runtime/diagnosztika szemantika a draft szerint
`std::execution` sender/receiver	kitöltendő	kitöltendő	kitöltendő	Library-támogatás külön ellenőrizendő
Hazard pointer / RCU reclamation	kitöltendő	kitöltendő	kitöltendő	A `shared_ptr` alternatívája, nem cseréje
SIMD / linear algebra	kitöltendő	kitöltendő	kitöltendő	Header- és standard library-függés

A „szint” oszlopban csak a fenti négy kategória egyike állhat; a forrásokhoz elsődleges anyagot (WG21 papers, working draft, compiler-/stdlib-dokumentáció, ABI-specifikáció) kell használni, nem blogbejegyzést vagy összefoglalót. A tarisznyába kerülő `source_id`-k helyére itt a konkrét P-/N-jelölést és a megtalálás dátumát kell írni.

46.2.3 Reflection és compile-time code generation

A reflection (tükrözés) célja, hogy a program a saját típusait és tagjait compile time enumerálhatóvá tegye, és erre építve generikus szerializációt, ORM-et, teszteket lehessen írni kézi felsorolás nélkül. A C++20-as alap ezt makrókkal és sablonmanipulációval utánozza.

```
// serialize_cxx20.cpp - C++20 baseline: kézi felsorolás
#include <cstdio>

struct Point { int x; int y; };

// A mezőket kézzel kell megismételni: ez az a terület,
// amit a reflection a jövőben kiválthat.
void dump(const Point& p) {
    std::printf("{x=%d, y=%d}\n", p.x, p.y);
}

int main() {
```

```
    Point p{1, 2};
    dump(p);
    return 0;
}

# Fordítási parancs (a tanuló laborjában ellenőrizendő):
g++ -std=c++20 -O2 -Wall -Wextra serialize_cxx20.cpp -o serialize_cxx20
```

Ez a kód a C++20 alapszabályok szerint állítja össze a kimenetet; a reflection-javaslat megjelenési formája (API-nevek, metaobjektum-típus, iteráció módja) revíziónként változhat, ezért nem tanácsos előre rögzített szintaxist feltételezni. Ha a szakaszhoz később reflection-példát illesztünk, a `-std=c++26` kapcsoló és a hozzá tartozó compiler-státusz ellenőrzése csak a working draft szintjén értelmezhető; ezt tilos verziófüggetlen garanciaként olvasni.

46.2.4 Contracts

A contracts célja: előfeltétel, utófeltétel és invariáns deklaratív rögzítése úgy, hogy a diagnosztikai és runtime-szemantikát a szabvány definiálja, ne az adott `assert`-implementáció. A C++20-as alap: `assert`, `[[likely]]`, `std::abort`, illetve saját invariant-ellenőrző makrók. A contracts szövegében a *semantics* (mikor értékelődik a predikátum, mi történik megsértés esetén, hogyan viszonyul a `noexcept`-hoz) a kritikus pont, mert ez határozza meg, hogy a kód viselkedése hogyan módosul debug- és release-profilban. A pontos megfogalmazás a working draftból ellenőrizendő, és az implementációs támogatás compilerenként eltér.

46.2.5 `std::execution` sender/receiver

A sender/receiver modell az asszinkron végrehajtást egy kompozicionálható pipeline-nyá teszi: a munka leírása (sender) elkülönül a futtatástól (scheduler), és a hibakezelés, megszakítás, több erőforrás-típus egyetlen típusrendszerbe kerül. A C++20-as alap a `std::future`/`std::async` és kézi szálkezelés, amelyek a pipeline-szemantikát csak egyedi megoldásokkal közelítik. A szakaszhoz ezért a C++20-as mintát mutatjuk meg, a sender/receiver formát csak a working draft alapján, jelöléssel.

```
// baseline_async.cpp - C++20: std::future, mint a sender/receiver előtti alap
#include <future>
#include <cstdio>

int compute() { return 42; }

int main() {
    std::future<int> f = std::async(std::launch::async, compute);
    std::printf("%d\n", f.get());
    return 0;
}

g++ -std=c++20 -O2 -Wall -Wextra -pthread baseline_async.cpp -o baseline_async
```

A példa a meglévő alapot mutatja. A sender/receiver bevezetése ezt a felületet **nem helyettesíti azonnal**; a library implementáció és a standard library-ben elérhető komponensek köre külön ellenőrizendő. A konkurens élettartam és reclamation kérdései a shared ownership, reclamation és konkurens review (34.6) szakaszra épülnek, ahol az ownership-szerződés pontosabb.

46.2.6 Hazard pointer / RCU és reclamation library

A lock-free adatszerkezetekben a legnehezebb kérdés nem a mutáció, hanem a **biztonságos visszanyerés (reclamation)**: mikor szabad egy csomópontot felszabadítani, ha egy olvasó még mutat rá. A C++20-ban erre a `std::shared_ptr` az egyetlen szabványos, de atomi referenciaszámítás miatt költséges válasz. A hazard pointer és az RCU ennél olcsóbb alternatívák; a hozzájuk tartozó library-eszközök státusza a working draftban és az implementációkban külön ellenőrizendő. A választás nem helyettesíthető: a `shared_ptr` továbbra is a helyes alap, ha az ownership megosztott, és a reclamation-költség nem kritikus.

46.2.7 SIMD, linear algebra és könyvtári térkép

A SIMD (single instruction, multiple data) típusok célja, hogy a vektoros hardveres párhuzamosságot hordozhatóan, típusosan lehessen kihasználni, a kézi intrinsics és az `__m128`-család helyett. A linear algebra (mátrix-, vektor-műveletek) célja, hogy a numerikus kód standard könyvtári alapot kapjon, és a BLAS/LAPACK-függést hordozható réteg váltsa fel. A két terület státusza eltér: a SIMD típusok nyelvi magas szintű leírása és a linear algebra library-felülete külön P-/N-anyagokban követhető, és mindkettő implementációs támogatása külön ellenőrizendő. A jelen szakasz csak a térképet adja: a részletes könyvtári választást a Standard könyvtári lefedettség és ökoszisztéma-térkép (46.4) tárgyalja.

46.2.8 Papers és working draft nem közzétett ISO szabvány

A legfontosabb félreértés, hogy egy P-számú dolgozat vagy egy N-számú working draft azonos a kiadott szabványos szöveggel. A különbség nem formális: az ISO-szavazás szerkesztheti a szöveget, a nemzeti adaptáció eltérhet, és a working draft egy adott pillanat **pillanatképe**, amelybe későbbi javaslatok már bekerülhettek, illetve korábbiak kikerülhettek. Ezért a „C++26-ban van” állítás csak a keletkezés dátumával és a hivatkozott konkrét N-/ISO-számmal együtt érvényes. Ha a kurzus bármely fejezetében egy képességre hivatkozunk, a hivatkozást a keletkezés dátumához és a konkrét draft-kiadáshoz kell kötni.

46.2.9 Tipikus hibák

- **Szint-összemosás**: elfogadott javaslatot „szabványosnak” nevezni, vagy a working draftot az ISO-kiadással azonosítani.
- **Támogatás igazolása a javaslat szövegével**: a P-dolgozat nem bizonyítja, hogy a fordító támogatja; ehhez feature-test makró és compiler-státuszoldal kell.
- **Dátum nélküli hivatkozás**: a C++26 státusz gyorsan változik; dátum nélkül a hivatkozás nem reprodukálható.
- **`shared_ptr` „kiváltása”**: a hazard pointer/RCU a megosztott tulajdonlás alternatívája csak ott, ahol a reclamation-költség kritikus és az ownership-szerződés ezt megengedi.
- **Előre rögzített szintaxis**: a reflection- és sender/receiver-API revíziónként változhat; a szakasz példáiban ezért a C++20-as alapot és a jelölést kell követni.

46.2.10 Összefoglaló

A C++26 és későbbi irányok tárgyalásakor a kérdés nem az, hogy „benne van-e a szabványban”, hanem az, hogy **melyik szinten** (proposal / working draft / ISO / implementált), **melyik revízión**, **melyik forrásból** és **melyik dátummal** ellenőriztük. A fejezet öt területe (reflection,

contracts, sender/receiver, hazard pointer/RCU, SIMD és linear algebra) mindegyike a working draft és a compiler-/library-státusz együttes vizsgálatát igényli; egyik sem helyettesíti a fő kurzus C++20-as alapját, és egyik sem garantálható verziófüggetlenül. A státuszkategóriák és a táblázatsablon biztosítják, hogy a későbbi átdolgozások a konkrét forráshoz és dátumhoz köthetők legyenek.

46.2.11 Feladatok

46.2.11.1 1. feladat (magyarázó)

Egy kollégád azt állítja: „a contracts támogatott C++26-ban, tehát a `g++ -std=c++26` lefordítja”. Fogalmazd meg, milyen szint- és bizonyítékhiányt tartalmaz ez az állítás, és sorolj fel legalább három ellenőrzési lépést.

46.2.11.2 2. feladat (kódolási)

Egészítsd ki a `serialize_cxx20.cpp` példát úgy, hogy a `dump` függvény egy `Point` tömböt is kezeljen, és a buildparancsot add meg. A kód a C++20 alapszabályok szerint forduljon.

46.2.11.3 3. feladat (diagnosztikai)

Válaszd ki a fejezet öt területe közül a `std::execution`-t, és készíts egy táblázatsort a „forrás és dátum” sablon szerint. Add meg, hogy mely feature-test makrót (`__cpp_lib_*`) fogsz ellenőrizni, és miért nem elegendő a P-dolgozat hivatkozása.

46.2.12 Megoldások

46.2.12.1 1. megoldás

Az állítás két szintet mos össze: az „elfogadva C++26-ra” a **working draft** szintjét érinti, nem a **közzétett ISO szabvány** szintjét, és nem bizonyítja az **implementációs támogatást**. A `-std=c++26` kapcsoló léte sem garancia arra, hogy a contracts szemantikája a választott compilerben megfelel a draftnak. Ellenőrzési lépések: (a) keresd meg a contracts-javaslat pontos P-számát és azt, hogy melyik working draft-kiadásba került be, dátum megjelölésével; (b) ellenőrizd, hogy a választott compiler **státuszoldala** a képességet „implemented”-nek jelöli-e, és melyik verziótól; (c) ellenőrizd a feature-test makrót a szabványkönyvtárban, és hogy a dokumentált szemantika (runtime/diagnosztika) egyezik-e a working draft szövegével. Csak a három lépés együttes eredménye után állítható bármi a gyakorlati használhatóságról.

46.2.12.2 2. megoldás

```
// serialize_cxx20_tomb.cpp - kiegészített C++20 baseline
#include <cstdio>
#include <cstddef>

struct Point { int x; int y; };

void put_point(const Point& p) {                    // csak az elem formázása,
 ↪  sorlezárás nélkül
```

```
    std::printf("{x=%d, y=%d}", p.x, p.y);
}

void dump(const Point& p) {                        // önálló pont: sorlezárással
    put_point(p);
    std::printf("\n");
}

void dump_array(const Point* a, std::size_t n) { // az elemformázás és a
 ↪  sorlezárás külön
    std::printf("[");
    for (std::size_t i = 0; i < n; ++i) {
        if (i) std::printf(", ");
        put_point(a[i]);
    }
    std::printf("]\n");
}

int main() {
    Point ps[3] = {{1,2}, {3,4}, {5,6}};
    dump_array(ps, 3);
    return 0;
}
```

```
g++ -std=c++20 -O2 -Wall -Wextra serialize_cxx20_tomb.cpp -o
 ↪  serialize_cxx20_tomb
```

Ellenőrzési feltétel: a fordítás `-Wall -Wextra` mellett figyelmeztetés nélkül menjen át; a kimenet `[{x=1, y=2}, {x=3, y=4}, {x=5, y=6}]` legyen. A példa a C++20 alapszabályok szerint fordítható; a program GCC 13.3-mal, `-std=c++20 -Wall -Wextra` mellett figyelmeztetés nélkül fordult és pontosan ezt az egy sort írta ki (lefuttatva).

46.2.12.3 3. megoldás

A táblázatsor:

Dátumos mintamegoldás (ellenőrzés: 2026-10-25; elsődleges forrás: a megnevezett WG21-papír és a C++26 munkatervezet, a bizottsági státusz másodlagos forrása a cppreference C++26-oldala — a dinamikusan változó státusz **frissítési határa**: az ISO-kiadás megjelenése, illetve az implementációk release note-jai; addig ez a dátum az érvényes, utána újraellenőrzendő):

Képesség	Szint	Forrás (P-/N-szám, szakasz)	Dátum	Megjegyzés
Reflection / compile-time generálás	C++26 munkatervezet része (technikailag lezárt; ISO-kiadás nem igazolt)	P2996R13	2026-10-25	`lab-gcc`-ben nem támogatott; nem helyettesíti a fordító támogatását

Képesség	Szint	Forrás (P-/N-szám, szakasz)	Dátum	Megjegyzés
Contracts	C++26 munkatervezet része	P2900R14	2026-10-25	A runtime/diagnosztika szemantika a draft szerint; fordítóverzió-függő
`std::execution` sender/receiver	C++26 munkatervezet része	P2300R10 ([exec] szakasz)	2026-10-25	Library-támogatás külön ellenőrizendő; `lab-gcc`-ben nem támogatott
Hazard pointer / RCU	C++26 munkatervezet része	P2530R3, P2545R4	2026-10-25	A `shared_ptr` alternatívája, nem cseréje
SIMD / linear algebra	C++26 munkatervezet része	P1928R15 (`std::simd`), P1673R13 (lineáris algebra)	2026-10-25	Header- és standard library-függés; `lab-gcc`-ben nem támogatott

A `std::execution` kiválasztott sora tehát: szint = „C++26 munkatervezet része (nem implementált a laborprofilban)”, forrás = P2300R10, dátum = 2026-10-25.

A feature-test makrót (`__cpp_lib_execution` vagy a sender/receiver-hez tartozó, a választott standard library dokumentációjában megnevezett makró) a fordítóval kell kiértékeltetni. A P-dolgozat hivatkozása azért nem elegendő, mert az a javaslat szövegét rögzíti, nem a standard library tényleges implementációját; a makró és a library release note együtt adja a gyakorlati támogatás bizonyítékát. A `shared_ptr`-alapú összehasonlításhoz lásd shared ownership, reclamation és konkurens review (34.6).

46.3 Freestanding, embedded, kernel és GPU irányok

Ez a kitekintő alfejezet a fő hosted Linux user-space profilon túli területekhez ad orientációt; célja nem az, hogy bármelyiket a fő kurzus mélységében tárgyalja, hanem hogy a saját célodhoz illeszkedő továbbtanulási irányt választhass. A C hordozhatósági audit és freestanding kitekintés már rögzítette a nyelvi minimumot, a C++26 és későbbi irányok ellenőrzött státusszal fejezet pedig megtanított arra, hogy külső verzió- és támogatási állításokat forrás nélkül ne tekintsünk biztosnak; a volatile, compiler barrier és hardver barrier a memóriahozzáférés-hardver kapcsolat legalapvetőbb eszközeit tisztázta. A jelen forrásjegyzet ehhez a részhez nem tartalmazott elsődleges hivatkozást, ezért a leírás orientáló, és a konkrét compiler-, target- vagy library-támogatást mindig a saját laborprofilodban ellenőrizd.

46.3.1 Freestanding C/C++ és a runtime határ

Az ISO C17 két implementációtípust különböztet meg. A hosted (gazdált) implementáció a teljes standard könyvtárat és `main`-központú indítást biztosítja. A freestanding (szabadon álló) imple-

mentáció ennél szűkebb: jellemzően csak a `<float.h>`, `<iso646.h>`, `<limits.h>`, `<stdalign.h>`, `<stdarg.h>`, `<stdbool.h>`, `<stddef.h>`, `<stdint.h>` fejléceket kötelező, a program belépési pontját és futásidejű környezetét pedig nem a nyelv, hanem a célplatform határozza meg. A C++ megfelelő része a freestanding subset, amely a C++23-ban jelentősen kibővült; hogy egy adott fejléc freestanding-e, a szabvány jelölésén túl a tényleges library és target támogatástól is függ.

A gyakorlatban a különbség a runtime hiányában mutatkozik meg. Freestanding környezetben nincs automatikus heap, nincs `stdout`, gyakran `malloc` és `printf` sem, nincs statikus konstruktorokat lefuttató startkód, hacsak a linkelési folyamat nem pótolja. C++-ban a globális objektumok, az exception, az RTTI és a `thread_local` külön döntés tárgya: `-fno-exceptions`, `-fno-rtti`, `-fno-threadsafe-statics`, `-fno-use-cxa-atexit`, és hasonlók, mert a hozzájuk tartozó runtime-kódot a toolchain nem biztosítja ingyen. A “freestanding” jelölés tehát nem azonos a “minimál” és “kicsi” jelzőkkel, hanem szerződés a nyelv és az implementáció között arról, mi garantált.

46.3.2 Kernel és embedded: startup, linker script, interrupt és MMIO

Kernel- vagy bare-metal embedded környezetben a megszokott operációs rendszer-absztrakció (folyamat, virtuális memória, `stdin`/`stdout`, fájlok) hiányzik vagy máshogy néz ki. A belépési pont nem `main`, hanem a platform által definiált start: reset vektor, `_start`, `start_kernel`, `module_init`, illetve egy adott bootprotokoll lépései. A linker script (`.ld`) rögzíti a szakaszok (`.text`, `.rodata`, `.data`, `.bss`, `.heap`, `.stack`) címét, sorrendjét és a memóriarégiók hozzáférési jogait; nélküle a linker nem tudja eldönteni, hova tegye őket, és nem lehet garantálni sem a vektorok helyét, sem a `.data` inicializálásához szükséges forrást.

Az eszközregisztereket MMIO-n (memory-mapped I/O) keresztül érjük el: adott fizikai címekre mutató `volatile` pointerekkel, hogy a fordító ne optimalizálja el és ne rendezze át a hozzáféréseket. A `volatile` azonban nem hardveres barrier és nem atomi művelet, amint azt a volatile, compiler barrier és hardver barrier rész részletesen tisztázta. A megszakításkezelő hívási konvencióját, regisztermentését, az engedélyezés/tiltás szabályait és a prioritási sémát az adott ABI és a gyártói kézikönyv rögzíti, nem a C szabvány.

46.3.3 Real-time: allocation, exception és latency policy

Real-time kontextusban a helyesség része a válaszidő korlátja, nem csak a funkcionális eredmény. Két tipikus szabály: a hot path ne allokáljon dinamikusan, mert a `malloc`/`new` worst-case futásideje és a lock-viselkedése nem garantált; és az exception unwind ne essen kritikus szakaszba, mert a stack frame méretétől és az unwind implementációjától (table-alapú, gyakran nem lineáris) függően a latencia nőhet. A szokásos megoldás `noexcept` interfész, statikus vagy előre lefoglalt pufferek, memóriapool és arena. A latency budgetet mérni kell; a becslés nem elég, és a mérési profilt (CPU, frekvencia, kernel, cache) a tanuló laborjában rögzíteni kell.

46.3.4 SIMD, intrinsics és target capability

A hordozható magas szintű kód mellett a teljesítménykritikus részek gyakran target-specifikus intrinsics-re épülnek (SSE/AVX x86-on, NEON/SVE AArch64-en), amelyeket a fordító `-march`/`-mcpu`/`-mavx2` kapcsolóval és a gyártói headerekkel tesz elérhetővé. Ezek nem hordozhatók, és futásidejű diszpécseléshez CPU capability-t kell ellenőrizni (`__builtin_cpu_supports` GCC/Clang alatt). C++ oldalon a szabványos `std::simd` támogatása verziófüggő, ezért a státuszát a C++26 és későbbi irányok ellenőrzött státusszal szellemében kezeld. A hordozható, fordító által autovektorizált kód általában jó kiindulópont; az intrinsics ott indokolt, ahol a mérés igazolja a nyereséget.

46.3.5 OpenMP, CUDA, HIP, SYCL: külön specifikáció és toolchain

Ezek a programozási modellek nem a C/C++ szabvány részei, hanem külön specifikációk saját memóriamodellel, fordítással és runtime-mal. Az OpenMP pragma-vezérelt megosztott memóriás párhuzamosságot ír le; a CUDA és a HIP saját nyelvi kiterjesztéssel különíti el a host- és az eszközkódot, és a launch-, szinkronizációs és memóriahierarchia-szabályok a saját specifikáció részei; a SYCL single-source C++ modellt kínál. A C/C++ alap csak alapot ad; az eszközmemória, a szinkronizáció és a hibakezelés szemantikája a saját specifikáció szerint értendő, nem az ISO nyelven.

46.3.6 eBPF és a natív userspace program határa

Az eBPF nem általános C futtatókörnyezet. A forrást speciális fordítóval (tipikusan `clang -target bpf`) fordítjuk, a bájtkódot verifier ellenőrzi (korlátos ciklusok, stack-méret, pointer-aritmetikai szabályok, helper-hívások), és a futtatás a kernel egy erre kijelölt pontján történik. Nincs libc, nincs dinamikus linker, nincs standard `main`, és a memória-hozzáférés csak bizonyítottan biztonságos formában engedett. A natív userspace program és az eBPF közötti interakció map-eken és helpereken keresztül történik, nem `dlopen`-szerűen, és a két oldal közötti adatszerkezet-szerződést mindkét oldalon ellenőrizni kell.

46.3.7 Kidolgozott példa: minimál freestanding belépési pont

Az alábbi szemléltető példa egy minimál freestanding x86-64 Linux user-space program. Nem futtattuk, csak a runtime határát mutatja meg; a tényleges viselkedés a linkertől és a kerneltől is függ.

```
/* start.c - minimál freestanding belépési pont, Linux x86-64, GCC */
void _start(void) {
    for (;;) {
        __asm__ volatile ("pause");   /* nincs libc, nincs main, nincs
 ↪  visszatérés */
    }
}
```

```
gcc -std=c17 -ffreestanding -fno-stack-protector -nostdlib \
    -nostartfiles -Wl,-e,_start -o start start.c
```

A `-ffreestanding` jelzi a fordítónak, hogy ne feltételezze a `main` jelenlétét és a teljes standard könyvtár elérhetőségét; a `-fno-stack-protector` elkerüli a `__stack_chk_fail` runtime-függőséget; a `-nostdlib` és `-nostartfiles` elhagyja a standard startkódot és a libc linkelését; a `-Wl,-e,_start` a belépési szimbólumot állítja be. Valós embedded vagy kernelkörnyezetben ehhez linker script, startup assembly és eszközleírás is tartozik, a `_start` törzse pedig target-specifikus inicializálást végez.

Az MMIO-oldali minta ugyanebből a környezetből:

```
#include <stdint.h>
#define UART_TX ((volatile uint8_t *)0x10000000u)

static void uart_putc(char c) {
    *UART_TX = (uint8_t)c;   /* volatile írás: a fordító nem ejti el */
}
```

A `volatile` itt csak a compiler-oldali mellékhatást védi: nem biztosít sorrendet más szálak vagy hardveres események felé, és nem atomi. A cím illusztráció; valós perifériacímet a lapka kézikönyve ad.

46.3.8 Tipikus hibák

A leggyakoribb freestanding hiba, hogy a kód `printf`-et vagy `malloc`-ot hív, és ezt a fordító nem, csak a linker jelzi hibaként. Gyakori tévedés a `volatile`-t hardveres barriernek vagy atomi védelemnek hinni, holott a CPU és a memóriarendszer sorrendezését az MMIO esetén külön architektúra-specifikus kerítések biztosítják. Real-time kódban gyakran marad bent kivétel egy `noexcept` felületen, vagy allokál a hot path; ezt a mérés hozza elő, nem a kódolvasás. GPU-portoláskor sokan feltételezik, hogy a host memóriamodell és szinkronizáció változatlanul érvényes. eBPF-nél tipikus hiba libc-függvény vagy végtelen ciklus használata, amit a verifier elutasít; ilyenkor a hibaüzenet nem fordítóhiba, hanem a betöltő verifier diagnosztikája.

46.3.9 Feladatok

1. **(magyarázó)** Mi a hosted és freestanding implementáció nyelvi alapja, és miért nem garantálja a freestanding jelölés a `std::vector` elérhetőségét?
2. **(kódolási)** Írj freestanding C programot, amely egy kitalált MMIO-címen “szívverést” ír, és mutass két különböző `volatile`-hozzáférési mintát. Add meg a buildparancsot, és indokold, miért nem biztos, hogy a program bármit is csinál.
3. **(diagnosztikai)** Az alábbi kódrészletben nevezd meg a real-time latency-kockázatokat, és javasolj javítást:

```
void handle_packet(const Packet& p) {
    std::vector<Byte> buf(p.begin(), p.end());  // allokál
    parse(buf);                                  // kivételt dobhat
    log("packet processed");                     // I/O
}
```

46.3.10 Megoldások

1. A hosted implementációtól a szabvány teljes standard könyvtárat és `main`-alapú indítást követel meg; a freestanding csak a minimális fejléc-halmazt és egy platform-specifikus belépési módot. A freestanding jelölés nem mondja ki, hogy `<vector>` elérhető; erről a C++ freestanding subset listája és a tényleges library/compiler támogatás együtt dönt, verziófüggően. *Ellenőrzés:* a válasz nevezze meg a két kategóriát, példálódzzon a minimális fejlécekkel, és különítse el a szabvány követelményét a tényleges implementációtól.

2. Szemléltetés (nem tesztelt):

```
#include <stdint.h>
#define HEARTBEAT ((volatile uint32_t *)0xDEAD0000u)

void heartbeat_once(uint32_t seq) {
    *HEARTBEAT = seq;          /* egyszerű tárolás */
    (void)*HEARTBEAT;          /* kötelező olvasás: volatile miatt nem esik ki
     ↪  */
}
```

Build: `gcc -std=c17 -ffreestanding -nostdlib -nostartfiles -Wl,-e,_start -o hb hb.c`, ahol a `_start` külön forrásban van. A `0xDEAD0000` cím nincs feltétlenül mapelve, a processzus nem szabályos (`exit` és libc-init nélkül), így szignál vagy azonnali leállás is lehet a végeredmény — a `volatile` csak azt zárja ki, hogy a két hozzáférés eltűnjön vagy egymáshoz képest újrarendeződjön. *Ellenőrzés:* a válasz nevezze meg a `volatile` célját, a buildparancsot és a bizonytalanság okát.

3. Kockázatok: (a) `std::vector` allokál a hot pathon, worst-case latencia nem garantált; (b) `parse` kivételt dobhat, az unwind költsége szabályozatlan, és a hívási lánc minden tagjának exception-safe-nek kellene lennie; (c) `log` I/O-t végez, blokkolhat, lockolhat, allokálhat. Javítás: előre lefoglalt fix puffer vagy arena; `parse noexcept`, hibakód-alapú; naplózás nem-blokkoló gyűrűn. *Ellenőrzés:* mindhárom kockázat megnevezve, a javaslat különítse el az allokációt, a hibakezelést és az I/O-t.

46.3.11 Összefoglaló

A fő user-space profilon túl több eltérő runtime- és erőforrásmodell létezik. Freestanding környezetben a nyelv minimális garanciát ad, a startkódot és a runtime-ot a célplatform biztosítja. Kernel és embedded környezetben a linker script, a startup, a megszakítási konvenció és az MMIO `volatile`-kezelése platform-specifikus. Real-time rendszerben az allokáció, a kivétel és az I/O latency-politika kérdés, nem stílus. SIMD és GPU modellek saját specifikációval és toolchainnel rendelkeznek, az eBPF pedig ellenőrzött, korlátozott bájtkód-környezet, nem általános C futtatókörnyezet. A továbbtanulási irányt a konkrét cél (beágyazott, kernel, HPC, hálózati packet processing) és az ahhoz tartozó elsődleges dokumentáció alapján érdemes választani; ez a fejezet ehhez ad térképet, a részletes forrásolvasást nem helyettesíti.

46.4 Standard könyvtári lefedettség és ökoszisztéma-térkép

Ez a fejezet nem új nyelvi elemet tanít, hanem **lefedettségi térképet** ad: melyik standard fejléc melyik kurzusrészhez visz vissza, és hol húzódik a határ a szabványos könyvtár és a harmadik féltől származó ökoszisztéma között. A „standard fejléc” itt pontos definíció szerint az ISO C vagy ISO C++ szabvány által meghatározott header, amelynek deklarációit a kiválasztott implementáció hosted vagy freestanding módban biztosítja. Elérhetősége nem azonos a teljes funkcionális támogatással: a nyelvi mód, a könyvtár verziója, a hosted/freestanding választás és az implementáció dokumentációja együtt dönt. Ezért a fejléc jelenléte helyett lehetőleg feature-test makrót vagy fordítási próbát használj. A freestanding korlátokat Freestanding, embedded, kernel és GPU irányok (46.3) tárgyalta; a szövegkódolás és random részletei Szövegkódolás, locale és random (32.5), a szinkronizációs primitívek semaphore, latch, barrier és szinkron stream (34.5) alatt találhatók.

46.4.1 C header inventory: C17 és C23

A C17 hosted könyvtára 29 szabványos fejlécből áll. Az alábbi csoportosítás index, nem helyettesíti a fejezetek szerződéseit.

Terület	C17 fejlécek
Diagnosztika	`<assert.h>`, `<errno.h>`
Karakter és szöveg	`<ctype.h>`, `<string.h>`, `<wchar.h>`, `<wctype.h>`, `<uchar.h>`, `<locale.h>`

Terület	C17 fejlécek
Matematika	`<complex.h>`, `<fenv.h>`, `<float.h>`, `<math.h>`, `<tgmath.h>`, `<limits.h>`
I/O és általános eszközök	`<stdio.h>`, `<stdlib.h>`, `<stddef.h>`, `<stdint.h>`, `<inttypes.h>`, `<stdarg.h>`, `<stdbool.h>`, `<stdalign.h>`, `<stdnoreturn.h>`, `<iso646.h>`
Vezérlés, jel, idő	`<setjmp.h>`, `<signal.h>`, `<time.h>`
Konkurencia	`<stdatomic.h>`, `<threads.h>`

A `<threads.h>` és a `<stdatomic.h>` támogatása implementációfüggő; a C11 threads különösen gyakran opcionális. A C23 két új fejlécet hoz: `<stdbit.h>` a bitkezelő makrókhoz, `<stdckdint.h>` az ellenőrzött egész aritmetikához. A C23 egyetlen C17-es fejlécet sem szüntet meg, de háromnak a tartalma változik, mert a `bool`, `true`, `false`, `alignas`, `alignof` kulcsszó lett: a `<stdbool.h>` már csak az obsolescent `__bool_true_false_are_defined` makrót adja, a `<stdalign.h>` üres, a `<stdnoreturn.h>` és a `noreturn` makró pedig obsolescent (helyette `[[noreturn]]`). A headerenkénti C17/C23 táblát a C hordozhatósági audit és freestanding kitekintés (18.4) freestanding szakasza adja; a tényleges támogatást a választott fordító és libc dokumentációja, illetve fordítási próba dönti el, nem a fejléc neve. C17-es fordítással a `<stdckdint.h>` és `<stdbit.h>` jellemzően nem része a szabványos környezetnek.

46.4.2 C++ header index kategóriánként

A C++20 alapkönyvtár jóval nagyobb; az alábbi tábla a kurzus szempontjából fontos fejléceket rendeli a korábbi részekhez. A `<version>` fejléc a feature-test makrók elsődleges forrása, nem maga a funkcionalitás.

Terület	Fontos C++20 fejlécek	Kurzus
Utility, memória, típusok	`<utility>`, `<tuple>`, `<optional>`, `<variant>`, `<any>`, `<compare>`, `<type_traits>`, `<functional>`, `<memory>`, `<memory_resource>`, `<version>`, `<source_location>`	optional, variant és visit (31.1), tuple, pair, any és type erasure (31.2), unique_ptr és egyedi tulajdonlás (21.3), Allocator model és allocator_traits (37.1)
Tárolók	`<array>`, `<vector>`, `<deque>`, `<list>`, `<forward_list>`, `<map>`, `<set>`, `<unordered_map>`, `<unordered_set>`, `<stack>`, `<queue>`, `<span>`, `<bitset>`	std::array, std::vector és folytonos tárolás (28.1)–Tárolóválasztási és invalidációs labor (28.6), span, byte és bounds-szerződések (31.3)
Iteratorok, algoritmusok, ranges	`<iterator>`, `<algorithm>`, `<numeric>`, `<execution>`, `<ranges>`	Iterator-kategóriák és fél-nyílt tartomány (29.1)–Borrowed range, dangling és iterator lifetime (30.5)

Terület	Fontos C++20 fejlécek	Kurzus
Numerika	`<cmath>`, `<complex>`, `<valarray>`, `<random>`, `<ratio>`, `<numbers>`, `<bit>`, `<limits>`, `<cfenv>`	Numerikus algoritmusok és kiértékelési sorrend (29.4), Bitműveletek és objektumreprezentáció C++-ban (31.4), Szövegkódolás, locale és random (32.5)
Szöveg, I/O, fájl, idő	`<string>`, `<string_view>`, `<charconv>`, `<format>`, `<regex>`, `<locale>`, `<iostream>`, `<fstream>`, `<sstream>`, `<filesystem>`, `<chrono>`	iostream, stream state és pufferelés (32.1)–Szövegkódolás, locale és random (32.5)
Konkurencia	`<thread>`, `<stop_token>`, `<mutex>`, `<shared_mutex>`, `<condition_variable>`, `<future>`, `<atomic>`, `<barrier>`, `<latch>`, `<semaphore>`, `<syncstream>`, `<coroutine>`	thread, jthread és kooperatív cancellation (34.1)–Aszinkron I/O adapter és coroutine-labor (35.5)

A C++23-cal bővülő fejlécek, például az `<expected>`, `<print>`, `<mdspan>`, `<flat_map>`, `<flat_set>`, `<generator>`, `<stacktrace>` és `<stdfloat>`, külön migrációs táblát igényelnek; elérhetőségüket `__cpp_lib_*` makróval és a könyvtár verziójával kell ellenőrizni. A `<format>` C++20-as fejléc, de a tényleges támogatását szintén feature-test mutatja; ha hiányzik, `<charconv>`-alapú fallback használható.

46.4.3 Ökoszisztéma: mi standard és mi nem

A standard könyvtár és a third-party csomag megkülönböztetése nem formai kérdés. A Boost/Asio, a Catch2, a GoogleTest, a pybind11, a SWIG, a Conan vagy a vcpkg saját verzióval, saját API- és ABI-szerződéssel, külön dokumentációval rendelkezik. Ezek **nem ISO részek**, és nem kerülnek be a `<version>` feature-test makrói közé. Egy Boost.Asio-s hálózati rétegnél a Boost verziója, a rendszer ABI-ja, a linkelési mód és a fordítási kapcsolók együtt számítanak; a standard `<thread>` vagy `<atomic>` jelenléte nem bizonyítja az Asio kompatibilitását. A továbbtanulási forrásokat mindig a projekt saját, verziózott dokumentációjában kell keresni; ez a könyv nem állítja, hogy bármelyik csomag adott kiadása elérhető vagy tesztelt.

46.4.4 Kidolgozott példa: naplófeldolgozó header-térképe

Egy szolgáltatás fájlból olvas, egész számokat parsol, időbélyeget formáz, munkaszálakat indít, majd gracefully leáll. C++20-ban a minimális térkép: `<fstream>` az olvasáshoz, `<string>`/`<string_view>` a szöveghez, `<charconv>` a szöveg–szám konverzióhoz (számfeldolgozáshoz), `<chrono>` az időhöz, `<thread>` és `<stop_token>` a szálakhoz, `<system_error>` a hibákhoz. Ha `<format>` valóban elérhető, a formázás ahhoz kerül; különben marad a `<charconv>` és a stream-alapú fallback. C17-ben ugyanez `<stdio.h>`, `<stdlib.h>`, `<string.h>`, `<time.h>`, `<errno.h>`, valamint opcionálisan `<threads.h>` és `<stdatomic.h>`; a hálózat POSIX-rétegen keresztül jön be.

Az alábbi C++20 próba nem a teljes funkcionalitást, hanem a feature-test jelenlétét ellenőrzi. Illusztratív kimenet; futási eredményt nem állítok mért tényként.

```
#include <iostream>
#include <version>

int main() {
#ifdef __cpp_lib_ranges
    std::cout << "ranges: " << __cpp_lib_ranges << '\n';
#else
    std::cout << "ranges: nincs __cpp_lib_ranges\n";
#endif
#ifdef __cpp_lib_format
    std::cout << "format: " << __cpp_lib_format << '\n';
#else
    std::cout << "format: nincs __cpp_lib_format\n";
#endif
#ifdef __cpp_lib_expected
    std::cout << "expected: " << __cpp_lib_expected << '\n';
#else
    std::cout << "expected: nincs __cpp_lib_expected\n";
#endif
#ifdef __cpp_lib_print
    std::cout << "print: " << __cpp_lib_print << '\n';
#else
    std::cout << "print: nincs __cpp_lib_print\n";
#endif
}
```

```
c++ -std=c++20 -pedantic-errors -Wall -Wextra probe.cpp -o probe
```

C oldalon a `__STDC_VERSION__` és a `__has_include(<stdckdint.h>)` hasonló próbát ad; a `cc -std=c17` és `cc -std=c23` külön buildprofil. A fordító tényleges verzióját rögzíteni kell, mert a `cc`/`c++` mögötti implementáció nem univerzális.

46.4.5 Tipikus hibák

- A fejléc létezéséből teljes funkcionalitásra következtetni; feature-test vagy dokumentált próba kell.
- C++20 módban C++23-as `<print>`/`<expected>` használatát feltételezni.
- Boostot, Catch2-t vagy pybind11-et standard fejlécként kezelni, és elhagyni a csomagverzió ellenőrzését.
- Tranzitív include-okra hagyatkozni; a „include what you use” elv itt is érvényes.
- C és C++ C-kompatibilitási fejléceit összekeverni: például `<atomic>` és `<stdatomic.h>` nem ugyanaz a szerződés.
- A `<codecvt>`-et általános kódolási megoldásként használni; a szövegkódolás részletei Szövegkódolás, locale és random (32.5) alatt vannak.

46.4.6 Feladatok

1. **Alap:** Sorolja be a `<stdckdint.h>`, `<threads.h>`, `<complex.h>`, `<expected>`, `<ranges>` és `<filesystem>` fejléceket nyelv, szabványverzió és fő szerep szerint.

2. **Közepes:** Egy C++20 program fájlt olvas, `int` értékeket parsol, időt mér, két szálat indít, és hibát jelent. Adja meg a minimális fejlécfelsorolást, és jelölje, melyik elemhez kell feature-test.
3. **Haladó:** Egy build Boost.Asio-t és Catch2-t használ, valamint `<format>`-et. Készítsen audit-táblát arról, mi standard, mi third-party, és milyen ellenőrzés szükséges a verzió, az ABI és a funkcionalitás igazolására.

46.4.7 Megoldások

46.4.7.1 1. feladat megoldása

`<stdckdint.h>` C23, ellenőrzött egész aritmetika; `<threads.h>` C11/C17 opcionális szálkezelés; `<complex.h>` C17 komplex számok; `<expected>` C++23 hibavisszaadási típus; `<ranges>` C++20 ranges; `<filesystem>` C++17 fájlrendszer. Ellenőrzés: a nyelvi mód (`-std=c17`, `-std=c23`, `-std=c++20`) és a feature-test makró együtt dönt, nem a fejléc neve önmagában.

46.4.7.2 2. feladat megoldása

Minimális C++20 felsorolás: `<fstream>`, `<string>`, `<charconv>`, `<chrono>`, `<thread>`, `<stop_token>`, `<system_error>`, `<vector>`. A `<format>` csak akkor kerül be, ha `__cpp_lib_format` elérhető; különben `<charconv>` és stream-alapú kiírás a fallback. Ellenőrzés: `-std=c++20` fordítás, a `__cpp_lib_format` és a szálkönyvtár tényleges linkelése; a puszta include siker nem elég.

46.4.7.3 3. feladat megoldása

Standard: `<format>` és a C++20 alapfejlécek; third-party: Boost.Asio és Catch2. Az audit táblázatban külön sor szerepel a Boost és a Catch2 verziójára, a fordítási ABI-ra, a linkelési rendre és a package manager által rögzített csomagazonosítóra. A `<version>` csak a standard library feature-testjeit mutatja, a Boost/Catch2 verzióját nem. Ellenőrzés: reprodukálható buildprofil, a third-party fejlécek include-útja nem keveredik a standard include-okkal, és a `<format>` hiányára dokumentált fallback van.

46.5 Hibaminták, döntési táblák és önellenőrző térkép

Ez a fejezet nem új mechanizmust vezet be, hanem a korábbi részek újraelérésének eszközeit rendezi össze: hibából szabályhoz, döntési helyzetből szerződéshez, tanulási célból ellenőrizhető bizonyítékhoz vezető útvonalakat ad. A cél az, hogy a kurzus anyaga ne lineáris olvasmányként, hanem kereshető referenciaként is használható legyen. Ezért itt nem ismételjük meg a definíciókat és a levezetéseket; minden sor egy korábbi részre mutat, ahol a feltételek és az indoklás megtalálhatók. A táblázatok használatának szabálya: előbb a megfigyelt tünetet vagy a meghozandó döntést azonosítod, aztán kikeresed a sort, végül visszalépve elolvasod az adott rész előfeltételeit. Egy táblázatsor önmagában nem bizonyíték, csak mutató.

46.5.1 A döntési táblák használata

46.5.1.1 Owner, borrower, move, copy és API paraméter

A visszatérő hibaforrás nem a szintaxis, hanem hogy a felek nem ugyanazt értik tulajdonláson. Az alábbi tábla a kérdést fordítja döntéssé; a részletes szerződéseket a Tulajdonlási szerződés és cleanup ágak, az Owner, view és erőforrásszerződés C++ API-ban, a Rule of zero, three és five és a Move constructor, moved-from állapot és std::move tartalmazza.

Helyzet	Választás	Következmény	Tipikus hiba
A függvény új erőforrást ad	visszatérési érték owner (`unique_ptr`, dokumentált `free`)	a hívó felel a lezárásért	nyers pointer ownerként, dokumentálatlanul
A függvény csak olvas	`const T*`, `std::string_view`, `std::span<const T>`	nincs másolás, nincs tulajdonátadás; elem sem módosítható	a view túléli az ownert; `const std::span<T>` (csak a nézet const)
A függvény átveszi a tulajdont	érték szerinti owner paraméter (a tulajdont automatikusan átveszi); `unique_ptr&&` csak az átadás *lehetőségét* fejezi ki	érték szerint: a hívó pointere a híváskor kiürül; `&&`-nél csak a törzsben végzett tényleges move viszi át (üres törzsű `f(unique_ptr<T>&&)` után a hívó pointere továbbra is owner)	`const&` átvétel, majd belső `free`; azt hinni, hogy a `&&` kötés már átmozgatott
Az erőforrás lehet „nincs”	`unique_ptr`, `std::optional` owner	a hiány típusban látszik	nullable nyers pointer ownerként
A paramétert eltároljuk	tulajdonló, mélyen másoló reprezentáció (`std::string`, `std::vector`)	csak ilyen értéktároláskor független az élettartam; view (`string_view`, `span`), nyers pointer vagy referenciatag másolata továbbra is külső tárolótól függ	`string_view` tagként tárolva — külön külső lifetime-szerződés kell hozzá
Nagy objektumot adunk át	`const&` vagy `span`	nincs másolás	temporary `const&`, amely a kifejezés végén elhal

A `const` helye számít: a `std::span<const T>` elemei nem módosíthatók, a `const std::span<T>` viszont csak a nézetobjektumot teszi konstanssá — rajta keresztül a hívó elemei továbbra is írhatók:

```
// span_const.cpp - C++20: const nézetobjektum vs const elemtípus
#include <span>

void reads_only(std::span<const int> s) {   // elem nem módosítható
    // s[0] = 1;                            // hiba: const int
    (void)s;
```

```
}

void looks_read_only(const std::span<int> s) {  // a NÉZET const, az elem nem
    s[0] = 42;                                  // fordul: a hívó adata
 ↪  változik!
}

int main() {
    int a[3] = {1, 2, 3};
    reads_only(a);
    looks_read_only(a);
    return a[0] == 42 ? 0 : 1;
}
```

Buildjegyzőkönyv (`lab-gcc` profil). A program figyelmeztetés nélkül fordult, és 0 kóddal futott: a `looks_read_only` a `const std::span<int>` ellenére átírta `a[0]`-t. A `reads_only` kikommentezett sorát visszakapcsolva a fordítás `assignment of read-only location` hibával leáll.

A döntés lényege mindig ugyanaz a kérdés: ki zárja le az erőforrást, és meddig él a hozzáférés? Ha a válasz nem fér bele egy mondatba, az API szerződése hiányos.

A következő C példa a „tulajdonos visszatérési érték” sort mutatja; a fordításhoz szükséges include-ok és a parancs is szerepelnek. A futtatás és a viselkedés ellenőrzése a saját laborprofilodban történik, ezt a fejezet nem állítja mérésnek.

```
/* c17: gcc -std=c17 -Wall -Wextra -Wpedantic -O2 -o dup dup.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* Szerződés: a visszatérési érték owner, a hívó free()-eli.
   Bemenet: borrowed, csak a hívás idejére kell érvényesnek lennie. */
char *dup_upper(const char *borrowed)
{
    if (borrowed == NULL) return NULL;
    size_t n = strlen(borrowed);
    char *owner = malloc(n + 1);
    if (owner == NULL) return NULL;
    for (size_t i = 0; i <= n; ++i) {
        char c = borrowed[i];
        owner[i] = (c >= 'a' && c <= 'z') ? (char)(c - 'a' + 'A') : c;
    }
    return owner;
}

int main(void)
{
    char *s = dup_upper("devops");
    if (s == NULL) { fputs("alloc failure\n", stderr); return 1; }
    puts(s);
    free(s);   /* owner lezárása */
    s = NULL;  /* a dangling újrahasználat megelőzése */
```

```
    return 0;
}
```

A C++ változat ugyanezt a döntést mutatja be a nem tulajdonló `string_view` és a tulajdonló `std::string` párral; a lényeg, hogy a view-t nem tároljuk a hívás után.

```
// c++20: g++ -std=c++20 -Wall -Wextra -Wpedantic -fsanitize=address -o count
 ↪  count.cpp
#include <cstdio>
#include <string>
#include <string_view>

// Szerződés: s csak a hívás idejére kell érvényesnek lennie (borrowed).
std::size_t count_upper(std::string_view s)
{
    std::size_t n = 0;
    for (char c : s)
        if (c >= 'A' && c <= 'Z') ++n;
    return n;
}

int main()
{
    std::string owner{"DevOps"};              // owner: élettartama main vége
    std::printf("%zu\n", count_upper(owner)); // a view az ownerre mutat
    return 0;
}
```

A tiltott minta ugyanennek a tükörképe: `std::string_view v = std::string{"tmp"};` után a temporary elhal, a view dangling lesz. A helyes változat vagy owner stringet tart, vagy a view élettartamát a hívásra korlátozza; erről a Temporary, value category és dangling és a Borrowed range, dangling és iterator lifetime szól.

46.5.1.2 Pointer, tömb, span, string és view invalidációs index

Az alábbi index nem típuslexikon, hanem invalidációs térkép: azt mutatja, melyik művelet teszi érvénytelenné a hozzáférést. A részletek a Pointeraritmetika, objektumhatár és provenance, a Tárolóválasztási és invalidációs labor és a span, byte és bounds-szerződések részekben találhatók.

Hozzáférés	Biztosan érvénytelen	Lehet érvénytelen	Érvényes marad
`T*` heapen	`free`; sikeres `realloc` (régi pointer)	—	sikertelen `realloc` után a régi blokk
`T[N]` automatikus	a hatókör vége	—	a hatókörön belül végig
`T[N]` statikus	—	—	a program végéig
`std::span<T>`	a tároló megszűnése	a tároló újraallokálása vagy átméretezése (ha a span a tároló puffert nézi)	kapacitáson belüli bővítés, ha a nézett tartomány nem tolódik

Hozzáférés	Biztosan érvénytelen	Lehet érvénytelen	Érvényes marad
`std::string_view`	az owner megszűnése	az owner bármely módosítása, amely újraallokál vagy a karaktereket mozgatja (SSO!)	az owner olyan módosítása, amely nem érinti a nézett karaktereket és nem allokál újra (feltételes)
`vector::data()`, elempointer	újraallokálás	—	`push_back` kapacitáson belül; `reserve(n)`, ha `n <= capacity()`
`vector` iterátor	újraallokálás; `insert`/`erase` pontja után	—	a beszúrási pont előttiek kapacitáson belül
`map`/`set` iterátor és referencia	a törölt elemre	—	minden más elemre insert/erase alatt
`unordered_*` iterátor	rehash; a törölt elemre	beszúrás (rehasht okozhat)	—
`unordered_*` referencia/pointer	a törölt elemre	—	rehash alatt is

Feltételek a gyorsindexhez: a `push_back`/`insert`/`resize`/`reserve` csak **újraallokációnál** (kapacitás-túllépés vagy `reserve(n)` `n > capacity()` esetén) érvénytelenít mindent; `insert`/`erase` **pozíciófüggő** (a ponttól kezdve, plusz az `end()`); sikertelen `realloc` a régi blokkot érintetlenül hagyja; a `string` módosítása csak akkor érvénytelenít pointert, ha újraallokál vagy a nézett karakterek elmozdulnak. A pointer, a referencia, az iterátor és a logikai nézet (index) különbözik: az index a művelet után is használható lehet, ha az elem logikai helye nem változott. A részletes szerződés a 28. fejezetben van; a gyorsindex ezzel nem mondhat ellent.

A *tartalomváltozás* külön kérdés: egy érvényes pointer is mutathat új értékre (például `insert` után a beszúrási pont mögötti elemek eltolódnak), ezért az érvényesség nem jelenti az érték változatlanságát.

Buildjegyzőkönyv (`lab-gcc` profil). Egy ASan/UBSan alatt futtatott próba igazolta a szélső eseteket: `reserve(8)` után a kapacitáson belüli `push_back` és a `reserve(4)` a `data()`-t nem változtatta; `rehash(1024)` után az előtte vett `unordered_map`-referencián át írt érték megmaradt; a teljesíthetetlen `realloc` `NULL`-t adott, és az eredeti blokk utána szabályosan felszabadítható volt.

Formálisan: ha egy hozzáférés az owner $[b, b+n)$ tartományára mutat, akkor érvényes mindaddig, amíg az owner objektum élettartama tart és a tartomány nem invalidálódott. Az `end` és az one-past érték nem dereferálható; ez a fél-nyílt tartomány szerződése. Az invalidáció nem mindig jár együtt memóriahibával: a `string` kis méretű optimalizálása miatt a buffer helyben is átíródhat, ezért a régi `data()` pointer csendesen más tartalmat mutathat.

46.5.1.3 C és C++: inicializálás, élettartam, aliasing

Ez a táblázat kizárólag a két nyelv eltérésének gyors felidézésére való; a C oldalt a Scope, linkage, storage duration és lifetime és az Effective type, aliasing és objektumreprezentáció, a C++ oldalt az Az inicializálás különböző alakjai és a Tároló újrafelhasználása és C++ objektumszabályok tárgyalja.

Kérdés	C (C17/C23)	C++ (C++20)
Élettartam kezdete	tárolási idő szerint: statikus a program indulásakor; **nem VLA automatikus** objektumnál a blokkba lépéskor (a hatókör nem azonos az élettartammal); **VLA**-nál a deklaráció elérésekor (blokk közepén deklarált VLA élettartama a deklarációnál kezdődik); allokált a foglaláskor	konstrukció befejezésekor; implicit-lifetime típusnál implicit létrehozással is
Nyers tárolóban objektum	nincs „objektum-létrehozás”; az allokált tároló effective type-ját az első nem karaktertípusú írás vagy a `memcpy` forrása adja	implicit-lifetime típusnál a `malloc`, `operator new`, `std::allocator::allocate`, `memcpy`/`memmove`, `unsigned char`/`std::byte` tömb implicit létrehozza az objektumot; más típushoz konstrukció kell (placement new, `construct_at`)
Alias-szabály	effective type és strict aliasing; karaktertípuson át minden olvasható	dinamikus típus és strict aliasing ([basic.lval]); `char`, `unsigned char`, `std::byte` kivétel; átlátszó helyettesítés híján `std::launder`
Nem inicializált érték	indeterminate; olvasása UB, ha csapdareprezentáció lehet, vagy ha a címét soha nem vett automatikus változót olvassuk; `unsigned char`-ként olvasható	default inicializált automatikus változó értéke indeterminate; olvasása UB, kivéve az `unsigned char`/`std::byte` egyes másolási eseteit (C++26: erroneous behaviour)
Kijelölt inicializálás	C99 óta: tetszőleges sorrend, beágyazott (`.a.b`) és tömbdesignator (`[3] =`), keverhető pozícionálissal, felülírás megengedett	C++20 óta: csak aggregate-nél, csak közvetlen tagra, deklarációs sorrendben, nem keverhető pozícionálissal, nincs tömb- és beágyazott designator
Lista-inicializálás	C17: nem üres `{...}` aggregate-re és skalárra (`int x = {1};`), az üres `{}` C17-ben nem általánosan szabályos; **C23-ban** az üres inicializáló (`int x = {};`) érvényes, nullára inicializál; szűkítés megengedett	nem üres és üres `{}` is különbözik: üres `{}` értékinicializál (default konstruktor); `{...}` szűkítés tiltott; `std::initializer_list` konstruktor elsőbbsége; a típus és konstruktorok követelményei továbbra is számítanak (nem minden típusra szabályos)

A gyakorlati következtetés: a C++ kódban a „nyers memória + cast” minta nem átvihető

gondolkodás nélkül C-ből, mert ott az objektum élettartama nem a byte-ok létezéséhez, hanem a konstrukcióhoz kötődik — kivéve az implicit-lifetime típusokat, amelyeket a `malloc`/`memcpy` jellegű műveletek C++20-ban implicit módon létrehozhatnak (lásd Tároló újrafelhasználása és C++ objektumszabályok (20.6)). Erről bővebben a C és C++: azonos szintaxis, eltérő szemantika szól.

46.5.1.4 Jelölésjegyzék: milyen bizonyítékot fogadunk el

A könyv következetesen megkülönbözteti az állítás forrásának típusát. A jelölés a szövegben az állítás mellett áll, nem helyettesíti a hivatkozást, és nem garantálja, hogy az adott verzió a te laborodban elérhető.

Címke	Mit jelent	Hol ellenőrizd
ISO-C	kiadott C szabvány nyelvi szabálya	a rögzített szabványverzió szövege
ISO-C++	kiadott C++ szabvány nyelvi szabálya	a rögzített szabványverzió szövege
DRAFT	nyilvános munkatervezet, nem kiadott szabvány	a tervezet státusza és dátuma
POSIX	POSIX specifikációs követelmény	a vonatkozó specifikációs fejezet
LINUX	Linux-specifikus viselkedés	kernel/libc dokumentáció
ABI	bináris felület, layout, hívási konvenció	a cél ABI specifikációja
COMPILER	fordító vagy könyvtár dokumentált viselkedése	a rögzített kiadás dokumentációja
GUIDELINE	ajánlás, nem normatív szabály	az ajánlás szövege
MEASUREMENT	konkrét mérés eredménye	a mérés leírása és környezete
ILLUSTRATION	szemléltető érték, például cím	nem állítás, nem bizonyíték

A címkék a szabvány állapotát és a támogatottságot külön kezelik. A C++23 kiadott szabvány (ISO/IEC 14882:2024): a normatív szabályai `ISO-C++` címkét kapnak; ha az állítás a nyilvános N4950 tervezet szövegéből idéz, az `DRAFT`; hogy egy fordító mit tud belőle, az `COMPILER`. A C++26 a 2026-10-25-i állapot szerint még nem kiadott szabvány, ezért rá `DRAFT` (illetve a támogatásra `COMPILER`) címke vonatkozik, dátummal. A támogatás mindkét esetben verziófüggő; erről a C++23 összefoglaló és migrációs táblázat és a C++26 és későbbi irányok ellenőrzött státusszal ad eligazítást. Ez a fejezet nem garantál semmilyen jövőbeli támogatást.

46.5.1.5 Build-, link- és futásidejű diagnosztikai útvonal

A hiba helyét a tünet alapján érdemes szűkíteni, mert a rossz szinten keresett hiba időt rabol. Az útvonal sorrendje nem esztétikai kérdés: a korábbi fázis hibája elfedi a későbbit.

1. Preprocesszor: hiányzó include, makróhelyettesítés, feltételes fordítás. Lásd Headerek, deklarációk és include-higiénia és Makrók, feltételes fordítás és veszélyes helyettesítés.
2. Fordítás: típus, deklaráció, konverzió. Lásd Deklarációk, typedef és típusminősítők.
3. Linkelés: `undefined reference` esetén deklaráció/definíció, `extern "C"`, szimbólumláthatóság. Lásd Szimbólumok, linkage és relocation és extern C és két nyelvű header.
4. Betöltés: nem található megosztott könyvtár, eltérő SONAME, rpath. Lásd ELF-betöltés, PLT/GOT és dinamikus szimbólumok.

5. Indulás és leállás: statikus inicializálás sorrendje, `atexit`, nem lokális vezérlés. Lásd Statikus és thread-local objektumok életciklusa.
6. Futásidejű helyesség: UB, adatverseny, élettartam-hiba. Lásd ASan, UBSan, MSan, TSan és Valgrind és Konkurens élettartam és nehéz hibák.
7. Erőforrás- és teljesítményprofil: szivárgás, allokációs minta, költség. Lásd Allokációs hibainjektálás és leak diagnosztika és Profilozás, benchmark és cost model.

Az útvonal ellenőrzési szabálya: minden lépésnél írd le, mit vársz és mit figyeltél meg; ha a kettő eltér, az eltérést vizsgáld meg, és ne ugord át a diagnosztikai lépést. A „buildel, de összeomlik” tünet például nem a 3. lépésnél kezdődik, hanem a 6.-nál, viszont a 3. lépés figyelmeztetései gyakran a 6. okát jelzik előre.

46.5.1.6 Cél–feladat–bizonyíték lefedettségi mátrix

A mátrix nem haladási napló, hanem annak ellenőrzése, hogy egy témacsoportot valóban lefedtél-e: van-e hozzá megoldott feladat és olyan bizonyíték, amelyet más is reprodukálhat. A lefedettség aránya:

$$\text{lefedettség} = \frac{\#\{\text{igazolt cél}\}}{\#\{\text{cél}\}}.$$

Az arány önmagában félrevezető: a nevezőt és a bizonyíték típusát is rögzíteni kell, különben a mutató tartalom nélkül telítődik.

Témacsoport	Cél	Feladat típusa	Bizonyíték
Tulajdonlás és élettartam	owner/borrower API-t indokolni	kódolási	szerződéslista + sanitizer futás naplója
Invalidáció	view érvényességét megőrizni	diagnosztikai	reprodukált hiba és javított változat
C/C++ szemantika	eltérést megnevezni	magyarázó	kétnyelvű példa és indoklás
Build és link	hibát fázishoz kötni	diagnosztikai	parancs, hibaüzenet, ok
Teljesítmény	költséget mérni	mérési	mérési környezet és nyers adat

A mátrix használatakor a bizonyíték oszlopba ne „megértettem” kerüljön, hanem visszakereshető nyom: parancs, kimenet, hibaüzenet vagy indoklás. A Reprodukálható fordítási és diagnosztikai labor és a Release, debug symbol és reprodukálhatóság adja az ehhez szükséges környezeti fegyelmet.

46.5.2 Feladatok

1. **Diagnosztikai útvonal.** Egy C program a build után azonnal `Segmentation fault`-tal áll le, a `-Wall -Wextra` nem jelez semmit, a linker hibátlan. Írd le, melyik diagnosztikai lépésekből melyiket választod, milyen sorrendben, és melyik megfigyelés zárná ki az adott szintet. Add meg a használt kapcsolókat is.
2. **API-szerződés javítása.** Adott egy C API, amely `const char *name` paramétert kap és a mutatót egy hosszú élettartamú struktúrában eltárolja. Írd át a szerződést és a kódot úgy, hogy az owner/borrower döntési tábla szerinti legyen, és adj ellenőrzési tervet arra az esetre, amikor a hívó ideiglenes buffert ad át.

3. **Lefedettségi terv.** Válassz egy témacsoportot a mátrixból, és készíts hozzá egy cél–feladat–bizonyíték sort úgy, hogy a bizonyíték más által is reprodukálható legyen. Jelöld meg, melyik állításod `ISO-C`, `ISO-C++`, `ABI` vagy `MEASUREMENT` címkét kapna.

46.5.3 Megoldások

1. megoldás. A tünet alapján a futásidejű szint a valószínű, de a sorrendet nem szabad megfordítani. Először ellenőrizd, hogy a lefordított program a várt forrásból származik-e (friss build, nem elavult bináris), majd futtasd címzett diagnosztikával: `-fsanitize=address,undefined` és debug információ. Ha az ASan pontos helyet ad, a hiba a 6. szintre tartozik; ha nem szólal meg, de a program összeomlik, gyanús a nem definiált viselkedés olyan formája, amelyet a sanitizer nem fog el, vagy a linkelt könyvtár eltérő. Az indoklás: az „nincs figyelmeztetés” nem bizonyíték a helyességre, csak annyit jelent, hogy a fordító nem talált diagnosztizálható mintát. Ellenőrzési feltétel: a hibaüzenet és a javított futás összevethető, és a javítás okát egy mondatban meg tudod nevezni.

2. megoldás. A hiba nem maga a pointer eltárolása — egy hosszabb élettartamú kölcsönzés is szabályos lehet, ha a szerződés kimondja, hogy a hívónak a struktúra élettartamáig életben kell tartania a nevet —, hanem az, hogy az élettartam nincs igazolva: az API szerződése nem mondja meg, meddig kell a `name`-nek élnie, így egy ideiglenes buffer dangling pointert hagy. A javítás két helyes irány egyike: vagy a függvény másolatot készít (`strdup`-szerű, dokumentált `free` kötelezettséggel), vagy a hívó adja át a tulajdont egyértelmű típusban. C-ben az első a szokásos; a szerződést a headerben kell rögzíteni, és a hibakimenetet is meg kell nevezni. Ha a hívó ideiglenes buffert ad, a másolat elkészítése után a buffer felszabadítható; ezért az ellenőrzési terv: hozz létre egy lokális buffert, add át, majd azonnal írd felül, és igazold, hogy a tárolt érték változatlan. Ez a teszt a tulajdonlást ellenőrzi, nem a memóriát. Egy teljes, kis megoldás owner-másolattal, OOM esetén megmaradó állapottal és a korábbi név cseréjével:

```
/* reg_name.c - C17: a név tulajdonlása a regiszterben (owner-másolat) */
#include <errno.h>
#include <stdlib.h>
#include <string.h>

struct reg {
    char *name;                 /* owner: a reg_destroy szabadítja fel */
};

/* Teszteléshez cserélhető allokátor (alapból malloc). */
static void *(*reg_alloc)(size_t) = malloc;

void reg_init(struct reg *r) { r->name = NULL; }

/* Szerződés: a hívó name-je csak a hívás idejére kölcsönzött; a regiszter
   saját másolatot tart. Siker: 0. Hiba (EINVAL, ENOMEM): a korábbi név
   változatlanul megmarad. */
int reg_set_name(struct reg *r, const char *name)
{
    if (r == NULL || name == NULL) return EINVAL;
    size_t n = strlen(name) + 1;
    char *copy = reg_alloc(n);
    if (copy == NULL) return ENOMEM;        /* a régi név érintetlen */
    memcpy(copy, name, n);
```

```
    free(r->name);                          /* a régi owner-másolat cseréje */
    r->name = copy;
    return 0;
}

void reg_destroy(struct reg *r)
{
    if (r == NULL) return;
    free(r->name);
    r->name = NULL;
}

#ifdef REG_NAME_TEST
#include <assert.h>
#include <stdio.h>
static void *fail_alloc(size_t n) { (void)n; return NULL; }
int main(void)
{
    struct reg r;
    reg_init(&r);
    char tmp[16];
    strcpy(tmp, "alpha");
    assert(reg_set_name(&r, tmp) == 0);
    strcpy(tmp, "XXXXX");                   /* a hívó felülírja az ideiglenes
 ↪  inputot */
    assert(strcmp(r.name, "alpha") == 0);   /* a másolat független */
    assert(reg_set_name(&r, "beta") == 0);  /* csere: a régi másolat felszabadul
 ↪  */
    reg_alloc = fail_alloc;                 /* OOM injektálás */
    assert(reg_set_name(&r, "gamma") == ENOMEM);
    assert(strcmp(r.name, "beta") == 0);    /* hiba esetén megmaradó állapot */
    reg_alloc = malloc;
    assert(reg_set_name(NULL, "x") == EINVAL && reg_set_name(&r, NULL) ==
 ↪  EINVAL);
    reg_destroy(&r);
    reg_destroy(&r);                        /* kétszeri destroy biztonságos */
    puts("reg_name: OK");
    return 0;
}
#endif
```

Buildjegyzőkönyv (`lab-gcc` profil). `gcc -std=c17 -Wall -Wextra -Wpedantic -fsanitize=address,undefined -DREG_NAME_TEST reg_name.c` figyelmeztetés nélkül fordult, és `reg_name: OK`-val futott: az ideiglenes input felülírása után a tárolt név változatlan, az injektált allokációs hibánál `ENOMEM` jön vissza és a korábbi `beta` megmarad, a csere és a kétszeri `reg_destroy` szivárgás és dupla felszabadítás nélküli. Ha a szerződés a hosszabb kölcsönzést választaná, a `reg_set_name` dokumentációjának kellene kimondania, hogy a `name` a `reg_destroy`-ig él; ezt a teszt nem tudja automatikusan igazolni. Kapcsolódó részek: Opaque handle és stabil C könyvtári interfész, C API-szerződések ellenőrzése.

3. megoldás. Például az invalidáció témacsoportnál a cél: „egy view érvényességi határát meg tudom nevezni”. A feladat: készíts olyan programot, amelyben egy `string_view` túléli az owner módosítását, figyeld meg a következményt, majd javítsd a kódot owner-másolattal vagy a view

élettartamának szűkítésével. A bizonyíték: a hiba előtti és utáni kimenet, a sanitizer jelentése és a javítás indoklása; a mérési környezet rögzítése. A címkék: a `string_view` invalidációs szabálya `ISO-C++`, a sanitizer viselkedése `COMPILER`, a konkrét kimenet `MEASUREMENT`. Az ellenőrzési feltétel, hogy a bizonyítékból kiderüljön, melyik állítás melyik forrásra támaszkodik, és hogy a javított változat nem függ a régi buffer tartalmától.

46.5.4 Összefoglaló

Ez a fejezet a kurzus anyagát keresőfelületté alakítja: a döntési táblák a tulajdonlás és az API-szerződés kérdéseit fordítják választássá, az invalidációs index a hozzáférések érvényességi határait rögzíti, a C/C++ táblázat a két nyelv eltérését idézi fel, a jelölésjegyzék pedig azt szabályozza, milyen bizonyítékot fogadunk el egy állítás mellett. A diagnosztikai útvonal a tünettől a fázisig vezet, a lefedettségi mátrix pedig a tanulást teszi ellenőrizhetővé: nem elég egy témát elolvasni, kell hozzá megoldott feladat és reprodukálható bizonyíték. A táblázatok nem helyettesítik a korábbi részek feltételeit; minden sor mögött ott van a hivatkozott rész, amelyet döntés előtt érdemes újraolvasni.

Függelékek

Fogalomtár (magyar–angol)

Ez a függelék a könyv kulcsfogalmait rögzíti egységes magyar és angol névvel, alfabetikus sorrendben. A számítási szabályokat és jelöléseket a Képlet- és jelölésgyűjtemény függelék tartalmazza.

Magyar	English	Jelentés / megkülönböztetés
ABI	application binary interface	a bináris felület szabályai: regiszterek, layout, névdekoráció
absztrakt gép	abstract machine	a C/C++ szabvány által definiált végrehajtási modell
alignment	alignment / igazítás	egy típus címének kötelező oszthatósága
allocator	allocator	C++ memóriafoglalási stratégia és típus
atomikus művelet	atomic operation	oszthatatlan, adatverseny nélküli művelet
auto	type deduction	C++ típuslevezetés kulcsszava
bitmező	bit-field	egy struct adott szélességű bitcsoportja
C++20	C++20	a könyv egyik fő nyelvi referenciaszintje
C++23	C++23	a III. rész nyelvi szintje
C++26	C++26	kitekintésként tárgyalt következő szabvány
C bájt	C byte	a `sizeof` egysége; bitszáma `CHAR_BIT`
constraint	constraint	concept által megfogalmazott template-feltétel
coroutine	coroutine	felfüggeszthető és folytatható C++ függvény
destruktor	destructor	az objektum élettartamának végén futó függvény
dinamikus memória	dynamic memory	futásidőben foglalt, explicit kezelésű tár
dinamikus típusazonosítás	dynamic type identification	`dynamic_cast` és RTTI
egység	translation unit	egy forrásfájl a beemelt fejlécekkel
élő objektum	live object	létező, élettartamában lévő objektum
exception safety	exception safety	kivétel esetén is érvényes állapot garanciája

Magyar	English	Jelentés / megkülönböztetés
extern “C”	C linkage	C ABI-hoz illeszkedő névkezelés
függvénypointer	function pointer	függvény címét tároló pointer
header	header	deklarációkat tartalmazó beillesztett fájl
inicializálás	initialization	objektum kezdőértékének beállítása
iterator	iterator	tároló elemein való bejárás absztrakciója
lambda	lambda expression	névtelen, helyben definiált callable
linker	linker	objektumok és könyvtárak összefűzése
memóriamodell	memory model	objektumok, élettartamok és láthatóság szabályai
move szemantika	move semantics	erőforrás átadása másolás helyett
mutex	mutual exclusion	kölcsönös kizárást biztosító zár
namespace	namespace	névteret adó nyelvi eszköz
ODR	one definition rule	egy definíció szabálya
overload	overload	azonos nevű, eltérő paraméterezésű függvény
ownership	ownership	erőforrás feletti felelősség és felszabadítás
pointer	pointer	címet tároló objektum
RAII	resource acquisition is initialization	erőforrás élettartamhoz kötése
range	range	bejárható elemsorozat (C++20 ranges)
referencia	reference	létező objektumra kötött alias
Rule of Five	rule of five	destruktor + másoló/mozgató konstruktor és értékadó szabálya
Rule of Zero	rule of zero	erőforrást nem kezelő osztály ne írjon speciális tagfüggvényt
sanitizer	sanitizer	futásidejű memória- és UB-ellenőrző eszköz
smart pointer	smart pointer	tulajdonlást kifejező RAII-pointer
static_cast	static cast	fordítási idejű, ellenőrzött típuskonverzió
std::string_view	string view	nem tulajdonló karakterlánc-nézet
szál	thread	párhuzamos végrehajtási egység
template	template	típus- és értékparaméteres generikus sablon
type trait	type trait	típus tulajdonságait lekérdező metafüggvény

Magyar	English	Jelentés / megkülönböztetés
undefined behavior	undefined behavior (UB)	a szabvány által nem definiált viselkedés
value category	value category	kifejezés értékkategóriája (lvalue, prvalue, xvalue)
virtuális függvény	virtual function	dinamikusan feloldott tagfüggvény
weak_ptr	weak pointer	nem tulajdonló, ciklust nem tartó smart pointer
zero initialization	zero initialization	statikus/nullázott kezdőérték-adás

Képlet- és jelölésgyűjtemény

Ez a függelék a könyv fontosabb számítási szabályait, méretezési képleteit és jelöléseit gyűjti egy helyre. A pontos nyelvi feltételeket és a megengedett implementációfüggő eltéréseket mindig az adott fejezet rögzíti.

1. Egész számok és bitműveletek

n bites előjel nélküli egész tartománya:

$$0 \le x \le 2^{n} - 1.$$

n bites kettes komplemenses előjeles egész tartománya:

$$-2^{n-1} \le x \le 2^{n-1} - 1.$$

A negáció bitenkénti tagadással és eggyel való növeléssel:

$$-x = \sim x + 1.$$

Bitmező kivágása s eltolással és w szélességgel:

$$y = (x \gg s) \;\&\; ((1 \ll w) - 1).$$

Előjel-kiterjesztés w bites mezőből n bites szóba:

$$y = (x \ll (n - w)) \gg (n - w),$$

ahol a jobbra tolás aritmetikai. A jobbra tolás negatív előjeles értéken implementációfüggő, ezért hordozható kódban óvatosan használjuk.

2. Méret, igazítás és memórialayout

Egy T típus `sizeof(T)` mérete C bájtban, `alignof(T)` igazítása. Egy o offsetet a igazításra kerekítve:

$$\operatorname{alignup}(o, a) = \left\lceil \frac{o}{a} \right\rceil a.$$

Egy struct mezőjének offsetje a megelőző mezők méretétől és az igazítási szabályoktól függ; a konkrét layouthoz mindig meg kell adni a target ABI-t és a fordítási profilt. A struct mérete az utolsó mező után is igazított:

$$\operatorname{sizeof}(S) \equiv 0 \pmod{\operatorname{alignof}(S)}.$$

A tömb i-edik elemének címe:

$$\operatorname{addr}(a[i]) = \operatorname{addr}(a) + i \cdot \operatorname{sizeof}(a[0]).$$

A pointer-aritmetika csak ugyanazon tömb (vagy one-past) elemeire definiált.

3. Fordítás, linker és ABI

A fordítási folyamat:

```
forrás -> preprocesszor -> fordító -> assembler -> objektum -> linker ->
↪ futtatható
```

A statikus és dinamikus könyvtár, a szimbólumfeloldás és a relokáció a linker feladata. Az `extern "C"` a C linkage-t biztosítja; a C++ névdekoráció és az ABI eltérő lehet. A `sizeof` és az alignment értéke nem hordozható univerzálisan: a cél-ABI és a fordító profilja határozza meg.

4. Komplexitás és költség

Egy algoritmus aszimptotikus futásideje $O(f(n))$ jelöléssel:

$$T(n) = O(f(n)) \quad \Longleftrightarrow \quad \exists c > 0,\ n_0 :\ T(n) \le c f(n) \text{ ha } n \ge n_0.$$

A Big-O mellett a gyakorlati döntéshez meg kell adni a művelet előfeltételét, a memóriahozzáférést, az allokációt és az invalidációs szabályokat. Az amortizált és a legrosszabb eset külön értelmezendő.

5. Memóriamodell és élettartam

- **Tárolási idő:** automatic, static, thread, dynamic.
- **Élettartam:** az objektum létezésének intervalluma; a pointer léte nem jelenti, hogy a mutatott objektum él.
- **Adatverseny:** két szál ugyanazt a memóriát éri el, legalább az egyik ír, és nincs köztük happens-before kapcsolat.
- **Sorrend:** sequenced-before, synchronizes-with, happens-before.

A C++ memóriamodellben az atomikus műveletek és a megfelelő szinkronizáció biztosítják a láthatóságot; a `volatile` nem szinkronizációs eszköz.

6. Lebegőpont és numerika

Az IEEE 754 bináris lebegőpontos érték:

$$(-1)^s \cdot (1.m)_2 \cdot 2^{e-bias}.$$

A lebegőpontos műveletek nem asszociatívak; a sorrend és a kerekítés megváltoztathatja az eredményt. A NaN nem egyenlő önmagával, és a `-ffast-math` jellegű kapcsolók megváltoztathatják a nyelvi garanciákat.

7. Egyidejűség és teljesítmény

A párhuzamos gyorsulás Amdahl-törvénye:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}},$$

ahol p a párhuzamosítható hányad. A mért gyorsulás ettől elmaradhat a szinkronizáció, a cache-hatás és a terhelés miatt. A cache line mérete és a mátrix elrendezése (sor- vagy oszlopfolytonos) jelentősen befolyásolja a hozzáférést; a konkrét érték hardverfüggő.

8. Build- és kiadási szerződés

A reprodukálható buildhez rögzíteni kell a fordítót és verziót, a cél-ABI-t, a kapcsolókat, a könyvtárverziókat és a tesztkörnyezetet. A `-O0`, `-O2`, `-O3`, `-fsanitize=...` és a `-std=c17`/`-std=c++20` kapcsolók eltérő viselkedést adhatnak; a mérési eredményt csak a rögzített profillal szabad összehasonlítani.

Kompetenciatérkép és tanulási útvonal

A könyv célprofilja: DevOps/SRE/platformmérnök, aki magas szintű nyelvi rutinnal rendelkezik, de a C/C++ rendszerprogramozást, memóriamodellt és modern C++-t rendszerszemlélettel akarja elsajátítani. A térkép tíz kompetenciát és tíz ellenőrző pontot rögzít; a pontok a hozzájuk tartozó blokk végén egy önálló, mérhető feladattal zárulnak.

Belépési és kilépési feltételek

Belépés. Általános programozási rutin egy magas szintű nyelven, folyamatok, szálak, virtuális memória, fájlrendszer és TCP/IP alapismeretek, Linux-parancssor és build/CI-alapok.

Kilépés. A tanuló önállóan megír egy memória- és élettartam-helyes C modult, követi a forrástól a binárisig tartó fordítási láncot, RAII-alapú modern C++ kódot tervez, és reprodukálható buildet, tesztmátrixot valamint C/C++ FFI-szerződést készít.

A tíz kompetencia és ellenőrző pontjai

#	Kompetencia	Mit tud a tanuló?	Ellenőrző pont	Bizonyíték
1	C mentális modell, fordítás, linker	Megkülönbözteti az absztrakt gépet a fizikai memóriától; követi a preprocesszor–linker láncot	`s015`	saját fordítási napló, ELF- és linker-elemzés
2	C típusok, memória, pointerek	Helyesen kezeli a típusokat, konverziókat, tárolási időt, pointereket és dinamikus memóriát	`s040`	memória- és pointerfeladat, sanitizer-eredmény
3	Adatszerkezetek, rendszerprogramozás, szálak	Parsert, POSIX eseménykezelőt és szálbiztos kódot ír	`s077`	működő parser/CLI, ThreadSanitizer-ellenőrzés
4	Hibakeresés, biztonság, C záróprojekt	Debuggerrel, sanitizerrel és teszttel vizsgál; biztonságos C API-t tervez	`s090`	diagnosztikai szolgáltatás, hibajegyzőkönyv
5	Modern C++ alapok, RAII, értékszemantika	RAII-t, smart pointert, copy/move-ot és élettartamot helyesen használ	`s116`	erőforrás-kezelő osztály, mozgatási teszt
6	OOP, kivételek, template/concepts	Indokolja az öröklést, a virtuális felületet, a kivételgaranciákat és a concepteket	`s137`	osztályhierarchia, exception safety teszt

#	Kompetencia	Mit tud a tanuló?	Ellenőrző pont	Bizonyíték
7	STL, iteratorok, lambdák, utility	Tárolót, algoritmust, range-et és nézetet választ érvényességi és költség szerint	`s164`	STL-feladat, iterator-invalidációs teszt
8	Compile-time, konkurencia, coroutine, ABI	Modult, coroutine-t és szálbiztos kódot készít; érti a C ABI-határt	`s185`	coroutine-labor, FFI-teszt
9	Teljesítmény, build, CI, C++ záróprojekt	Profiloz, allocatort és layoutot választ, reprodukálható buildet és tesztmátrixot állít össze	`s202`	mért teljesítményjegyzőkönyv, CI-konfiguráció
10	C++23/C++26 és referencia	Használja a C++23 újdonságait, és elhelyezi a C++26 irányait	`s231`	C++23 migrációs feladat, záróellenőrzés

Ajánlott útvonal

1. **C alapok (1–4. kompetencia):** `ch01`–`ch18`. A fordítási lánc, a memóriamodell és a biztonságos C API nélkül a modern C++ nem építhető.
2. **Modern C++ (5–7. kompetencia):** `ch19`–`ch32`. A RAII, az értékszemantika, az OOP és az STL a mindennapi C++ alapja.
3. **Haladó C++ (8–9. kompetencia):** `ch33`–`ch39`. A compile-time, a konkurencia, az ABI és a teljesítmény a rendszereszközök szintje.
4. **C++23 és kitekintés (10. kompetencia):** `ch41`–`ch46`, majd `ch40`. A záró blokk a legújabb nyelvi eszközöket és a továbbtanulási irányokat adja.

Útmutató a haladáshoz

- Minden memória- és pointerfeladatnál először rajzold fel az objektumok élettartamát és a tulajdonviszonyt; csak utána írj kódot.
- A mért teljesítményhez mindig rögzítsd a fordítót, a kapcsolókat, a cél-ABI-t és a hardvert; a mért és a levezetett értéket különítsd el.
- A C/C++ határon a névdekoráció, az alignment, az allocator és a hibakezelés külön szerződés; ezeket teszteld, ne feltételezd.
- Ha egy előfeltétel hiányzik, előbb azt pótold; a könyv jelzi az előfeltételeket.

Bibliográfia és forráskulcsok

Az itt szereplő URL-ek kiindulópontok, nem ellenőrzött bizonyítékok. A ténylegesen olvasott forrásjegyzet külön forrásjegyzetbe kerüljön.

Közös kiindulópontok

- https://www.open-std.org/jtc1/sc22/wg14/
- https://www.open-std.org/jtc1/sc22/wg21/
- https://gcc.gnu.org/onlinedocs/gcc/Standards.html
- https://clang.llvm.org/docs/
- https://man7.org/linux/man-pages/
- https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf
- https://gcc.gnu.org/onlinedocs/gcc/
- https://sourceware.org/binutils/docs/
- https://www.gnu.org/software/libc/manual/html_node/index.html
- https://gitlab.com/x86-psABIs/x86-64-ABI
- https://sourceware.org/gdb/current/onlinedocs/gdb/
- https://clang.llvm.org/docs/AddressSanitizer.html
- https://github.com/ARM-software/abi-aa
- https://pubs.opengroup.org/onlinepubs/9799919799/
- https://llvm.org/docs/LibFuzzer.html
- https://clang.llvm.org/docs/ThreadSanitizer.html
- https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
- https://itanium-cxx-abi.github.io/cxx-abi/abi.html
- https://cmake.org/cmake/help/latest/
- https://gcc.gnu.org/projects/cxx-status.html

ch01: A C és C++ mentális modellje és a labor környezete

Az OS- és hálózati tudásra építve vezesse be a natív programok megfigyelését, a szabvány és implementáció különbségét, valamint a reprodukálható laborokat.

Kutatási fókusz: Nyelvi absztrakt gép, GCC/Clang fordítás, hosted környezet; alapértelmezett zászlók és verziók hivatalos ellenőrzése.

- https://www.open-std.org/jtc1/sc22/wg14/
- https://www.open-std.org/jtc1/sc22/wg21/
- https://gcc.gnu.org/onlinedocs/gcc/Standards.html
- https://clang.llvm.org/docs/
- https://man7.org/linux/man-pages/

ch02: A fordítás és a preprocessor C-ben

Építse fel a fordítási folyamat pontos modelljét; a makrók és headerek hibáit laborokban tegye láthatóvá.

Kutatási fókusz: Translation phases, preprocessing tokenek, include, makrók, diagnosztika és külön fordítás.

- https://www.open-std.org/jtc1/sc22/wg14/
- https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf
- https://gcc.gnu.org/onlinedocs/gcc/
- https://clang.llvm.org/docs/
- https://sourceware.org/binutils/docs/

ch03: Linkelés, ELF, programindulás és leállás

Kapcsolja össze a C nyelvi programindulást az ELF, a dinamikus linker és a folyamat erőforrásainak ismert világával.

Kutatási fókusz: ELF szimbólumok, relocation, statikus/dinamikus könyvtárak, runtime startup és main.

- https://www.open-std.org/jtc1/sc22/wg14/
- https://gcc.gnu.org/onlinedocs/gcc/
- https://sourceware.org/binutils/docs/
- https://www.gnu.org/software/libc/manual/html_node/index.html
- https://man7.org/linux/man-pages/
- https://gitlab.com/x86-psABIs/x86-64-ABI

ch04: C adattípusok, konverziók és kifejezések

A típusrendszert reprezentáció, tartomány és konverziók felől tanítsa; minden méretállítást platformhoz vagy szabványhoz kössön.

Kutatási fókusz: C17 típusok, integer promotion, floating point, bitműveletek és sequencing; C23 eltérések csak megjelölve.

- https://www.open-std.org/jtc1/sc22/wg14/
- https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf
- https://gcc.gnu.org/onlinedocs/gcc/Standards.html
- https://www.gnu.org/software/libc/manual/html_node/index.html

ch05: C objektumok, tárolási idő és memóriamodell

A C nyelvi élettartamot külön kezelje a virtuális memória gyakori stack/heap felosztásától.

Kutatási fókusz: Scope, linkage, storage duration, object lifetime, effective type és indeterminate értékek C17 alapján.

- https://www.open-std.org/jtc1/sc22/wg14/
- https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf
- https://man7.org/linux/man-pages/
- https://sourceware.org/gdb/current/onlinedocs/gdb/

ch06: Pointerek és összetett deklarációk C-ben

A pointert szerződéssel rendelkező értékként tanítsa: referált típus, érvényesség, tulajdonlás, tartomány és indirection szint.

Kutatási fókusz: C pointerek, pointer arithmetic, provenance óvatos szabványkezelése, const és function pointerek.

- https://www.open-std.org/jtc1/sc22/wg14/
- https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf
- https://gcc.gnu.org/onlinedocs/gcc/
- https://man7.org/linux/man-pages/

ch07: Tömbök, sztringek és nem tulajdonló adatszeletek C-ben

A tömbök és pointerek különbségét, a méretet és a határokat az egész fejezetben explicit szerződéssel kövesse.

Kutatási fókusz: Array-to-pointer conversion, VLA, multidimenziós tömb, C string, byte buffer és UTF-8.

- https://www.open-std.org/jtc1/sc22/wg14/
- https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf
- https://www.gnu.org/software/libc/manual/html_node/index.html
- https://man7.org/linux/man-pages/

ch08: Dinamikus memória és explicit tulajdonlás C-ben

A malloc családot lifetime-, overflow- és hibakezelési szerződésekkel tanítsa; allocator belső részletek csak megjelölten.

Kutatási fókusz: malloc, calloc, realloc, free, alignment, ownership és erőforrásbiztos C API.

- https://www.open-std.org/jtc1/sc22/wg14/
- https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf
- https://www.gnu.org/software/libc/manual/html_node/index.html
- https://man7.org/linux/man-pages/
- https://clang.llvm.org/docs/AddressSanitizer.html

ch09: struct, union, alignment és bináris layout

A memóriaelrendezésből vezesse le az ABI és szerializálás következményeit, különösen hálózati adatoknál.

Kutatási fókusz: Padding, sizeof, offsetof, alignment, tagged union, bit-field és explicit wire encoding.

- https://www.open-std.org/jtc1/sc22/wg14/
- https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf
- https://gitlab.com/x86-psABIs/x86-64-ABI
- https://github.com/ARM-software/abi-aa
- https://gcc.gnu.org/onlinedocs/gcc/

ch10: Függvények, modulok és C API-tervezés

A C interfészek típus- és erőforrásszerződését tegye explicitté, elválasztva a hívási ABI részletes tárgyalásától.

Kutatási fókusz: Pass-by-value, pointerparaméter, variadic API, _Generic, opaque handle és kompatibilis interfész.

- https://www.open-std.org/jtc1/sc22/wg14/
- https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf
- https://www.gnu.org/software/libc/manual/html_node/index.html
- https://gcc.gnu.org/onlinedocs/gcc/

ch11: A C standard könyvtár és hibakezelés

A gyakori libc API-kat pontos visszatérési szerződéssel mutassa be, és adjon térképet a ritkábban használt fejlécekhez.

Kutatási fókusz: stdio buffering, parsing, errno, time, math és standard header inventory; ISO/POSIX külön jelölés.

- https://www.open-std.org/jtc1/sc22/wg14/
- https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf
- https://www.gnu.org/software/libc/manual/html_node/index.html
- https://man7.org/linux/man-pages/

ch12: C mint Linux/POSIX rendszerprogramozási nyelv

Az ismert operációs rendszer funkciókat C API-n keresztül tanítsa, hangsúlyozva az életciklust, rövid műveletet és hibakezelést.

Kutatási fókusz: FD ownership, partial IO, fork/exec, signal safety, socket API, epoll és mmap; platformrétegek jelölése.

- https://pubs.opengroup.org/onlinepubs/9799919799/
- https://man7.org/linux/man-pages/
- https://www.gnu.org/software/libc/manual/html_node/index.html

ch13: Adatszerkezetek és parser C-ben

Az algoritmikai alapokat ismertnek véve mutassa be a tároló, pointerstabilitás és invariáns explicit kezelését.

Kutatási fókusz: Dinamikus tömb, listák, hash table, arena, parser és amortizált költség; ownership minden csomóponton.

- https://www.open-std.org/jtc1/sc22/wg14/
- https://www.gnu.org/software/libc/manual/html_node/index.html
- https://llvm.org/docs/LibFuzzer.html

ch14: Assembly, hívási ABI és hardverkapcsolatok

A C forrást gépi működéssel kapcsolja össze, de egyetlen assemblymegfigyelést se általánosítson nyelvi garanciává.

Kutatási fókusz: x86-64 System V alap, AArch64 összehasonlítás, register/stack ABI, optimizer és hardvermemória.

- https://gitlab.com/x86-psABIs/x86-64-ABI
- https://github.com/ARM-software/abi-aa
- https://gcc.gnu.org/onlinedocs/gcc/
- https://clang.llvm.org/docs/
- https://sourceware.org/binutils/docs/
- https://sourceware.org/gdb/current/onlinedocs/gdb/

ch15: Szálak és atomikus műveletek C-ben

A C standard memóriamodellt a meglévő OS-tudásra építse; előbb mutex alapú helyesség, utána atomics.

Kutatási fókusz: pthreads és C11 threads elkülönítése, data race, happens-before, mutex/CV és C atomics.

- https://www.open-std.org/jtc1/sc22/wg14/
- https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf
- https://pubs.opengroup.org/onlinepubs/9799919799/
- https://man7.org/linux/man-pages/
- https://clang.llvm.org/docs/ThreadSanitizer.html

ch16: C hibakeresés, tesztelés és biztonságos kód

A memória- és erőforrásszabályokat gyakorlati diagnosztikával ellenőrizze; egyetlen eszköz hibamentes futása se legyen helyességigazolás.

Kutatási fókusz: Debugging, sanitizer profilok, fuzzing, boundary tests, compiler warnings és defensive parsing.

- https://sourceware.org/gdb/current/onlinedocs/gdb/
- https://clang.llvm.org/docs/AddressSanitizer.html
- https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html
- https://clang.llvm.org/docs/ThreadSanitizer.html
- https://llvm.org/docs/LibFuzzer.html
- https://clang.llvm.org/docs/
- https://gcc.gnu.org/onlinedocs/gcc/

ch17: C záróprojekt: stream-alapú diagnosztikai szolgáltatás

Egy kis capstone kössön össze CLI-t, parserkönyvtárat és korlátozott nonblocking TCP végpontot; a hálózati funkció oktatási körű.

Kutatási fókusz: Végigvezetett C projekt build, ownership map, IO state machine, tests és dokumentált release ellenőrzéssel.

- https://pubs.opengroup.org/onlinepubs/9799919799/
- https://man7.org/linux/man-pages/
- https://gcc.gnu.org/onlinedocs/gcc/
- https://llvm.org/docs/LibFuzzer.html
- https://clang.llvm.org/docs/AddressSanitizer.html

ch18: Modern C, C23 és a C/C++ határ

A C17 alaphoz képest adjon külön verziózott C23 áttekintést; minden kódhoz nyelvi és libc feature supportot ellenőrizzen.

Kutatási fókusz: C23 language/library delta, feature detection és C/C++ semantic differences. N3096 nyilvános munkatervezet, nem az ISO kiadvány.

- https://www.open-std.org/jtc1/sc22/wg14/
- https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf
- https://gcc.gnu.org/onlinedocs/gcc/Standards.html
- https://clang.llvm.org/docs/
- https://www.open-std.org/jtc1/sc22/wg21/

ch19: Átmenet C-ről modern C++-ra

A C++20 alapot C tudásra építve tanítsa, hangsúlyozva az értékszemantikát és a biztonságos erőforrás-absztrakciót.

Kutatási fókusz: Namespaces, references, type deduction, overload, initialization és value categories előképe.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
- https://gcc.gnu.org/onlinedocs/gcc/Standards.html
- https://clang.llvm.org/docs/

ch20: C++ objektumok, inicializálás és élettartam

Részletesen tárgyalja az initialization és destruction sorrendet, a subobjecteket és a temporary lifetime csapdákat.

Kutatási fókusz: Default/value/direct/list initialization, ctor/dtor sorrend, temporaries és object lifetime C++20/23 szerint.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
- https://clang.llvm.org/docs/AddressSanitizer.html

ch21: RAII, erőforráskezelés és smart pointerek

A C cleanup szerződéseiből vezesse le a modern C++ determinisztikus erőforrás-kezelését; owner és view minden példában külön.

Kutatási fókusz: new/delete, RAII, unique_ptr/shared_ptr/weak_ptr, custom deleter és nem memóriajellegű erőforrások.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

ch22: Értékszemantika, másolás és mozgatás

A C shallow copy problémájából vezesse le a C++ special member functions és value semantics szabályait.

Kutatási fókusz: Rule of zero/three/five, move, copy elision, noexcept és forwarding előkép; teljes példák invariánsokkal.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

ch23: Osztálytervezés, láthatóság és operátorok

Az OOP-t invariánsok és publikus szerződés felől tanítsa; a típusok és operátorok jelentése legyen következetes.

Kutatási fókusz: Access control, const member, friend, operators, comparisons és API value types.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

ch24: Öröklés, virtuális függvények és OOP

A dinamikus polimorfizmus szerződését és költségét tanítsa, majd hasonlítsa össze a kompozícióval és statikus alternatívákkal.

Kutatási fókusz: Inheritance access, virtual override, destructor, slicing, RTTI, multiple inheritance és ABI implementáció.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
- https://itanium-cxx-abi.github.io/cxx-abi/abi.html

ch25: Kivételek és C++ hibakezelési stratégiák

A hibaátadást az erőforrás- és invariánsgaranciákhoz kösse; kivétel, error_code és expected szerepe legyen indokolt.

Kutatási fókusz: Stack unwinding, exception safety, noexcept, system_error és expected C++23 külön jelöléssel.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
- https://itanium-cxx-abi.github.io/cxx-abi/abi.html

ch26: Template-ek és generikus programozás alapjai

A generikusságot típusbiztos algoritmusok és reusable value types felől tanítsa; instantiation és lookup szabályok legyenek láthatók.

Kutatási fókusz: Function/class templates, deduction, specialization, two-phase lookup és ADL C++20 alapokon.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
- https://gcc.gnu.org/onlinedocs/gcc/
- https://clang.llvm.org/docs/

ch27: Concepts, type traits és compile-time generikusság

A modern constraint-eket részesítse előnyben; a SFINAE-t örökölt kód olvasásához is mutassa meg.

Kutatási fókusz: C++20 concepts/requires, traits, constexpr branching, variadic templates, SFINAE és customization.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

ch28: STL tárolók és érvényességi szerződések

Tárolóválasztást locality, műveleti költség, ownership és invalidáció szerint tanítson, nem pusztán API-listából.

Kutatási fókusz: array/vector/string/deque/list/map/set/unordered/adapters, invalidation, complexity és allocator előkép.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

ch29: Iteratorok és standard algoritmusok

A tárolótól független algoritmust range-szerződéssel és költséggel tanítsa, világos előfeltételekkel.

Kutatási fókusz: Iterator categories, sentinel alap, half-open range, mutation, sorting/searching, numeric és execution policy.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

ch30: Lambdák, callable-ek és ranges

A callback és non-owning buffer tudást építse tovább; lazy pipeline minden capture- és lifetime-feltétele legyen explicit.

Kutatási fókusz: Lambda captures, std::function/invoke, ranges algorithms, view ownership, borrowed_range és dangling.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

ch31: Standard utility típusok és memória-view-k

A modellválasztást ownership, aktív változat, hibaszerződés és lifetime szerint tanítsa; a C tagged unionnal hasonlítsa össze.

Kutatási fókusz: optional, variant, any, tuple, span, byte, bit utilities és mdspan C++23 capability checkkel.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

ch32: C++ szöveg, fájlok, idő és könyvtári I/O

A standard absztrakciók szerződését a libc/POSIX szemantikához viszonyítva tanítsa, exact version jelöléssel.

Kutatási fókusz: iostream, charconv, format/print, filesystem, chrono, random és locale/encoding.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
- https://man7.org/linux/man-pages/

ch33: Compile-time C++, ODR és modulok

A fordítási időt önálló erőforrásként kezelje; a constexpr és modules korlátait egyértelműen tárgyalja.

Kutatási fókusz: constexpr/consteval/constinit, ODR, modules/header units, feature detection és compile-time cost.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
- https://gcc.gnu.org/onlinedocs/gcc/
- https://clang.llvm.org/docs/
- https://cmake.org/cmake/help/latest/

ch34: Modern C++ konkurencia és szinkronizáció

A C memóriamodellre építve C++ ownership és RAII API-kon tanítsa a szálak és shared state életciklusát.

Kutatási fókusz: thread/jthread/stop_token, mutex/CV, atomics, future/async és C++20 synchronization primitives.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
- https://clang.llvm.org/docs/ThreadSanitizer.html

ch35: Coroutine-ok és aszinkron erőforrás-életciklus

A coroutine-t nyelvi állapotgépnek tanítsa; scheduler, I/O és thread policy külön implementációs szerződés.

Kutatási fókusz: C++20 coroutine mechanics, promise/handle/frame ownership, cancellation és async IO lifetime; generator C++23.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://clang.llvm.org/docs/
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

ch36: C++ ABI, C interfész és FFI

A könyvtári és nyelvi határt erőforrás-, exception- és ABI-szerződésként tanítsa, konkrét toolchain-profilhoz kötve.

Kutatási fókusz: Name mangling, object/vtable ABI, extern C, stable handles, shared library boundaries és binding ownership.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://itanium-cxx-abi.github.io/cxx-abi/abi.html
- https://gitlab.com/x86-psABIs/x86-64-ABI

- https://github.com/ARM-software/abi-aa
- https://gcc.gnu.org/onlinedocs/gcc/
- https://man7.org/linux/man-pages/

ch37: Teljesítmény, allocatorok és memóriaelrendezés C++-ban

A helyességi szerződés után mérésből vezesse le az optimalizálást, világos cost modellel és reprodukcióval.

Kutatási fókusz: Allocators/pmr, small object layout, locality, profiling, code size és runtime/static dispatch costs.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
- https://gcc.gnu.org/onlinedocs/gcc/
- https://clang.llvm.org/docs/
- https://gitlab.com/x86-psABIs/x86-64-ABI

ch38: Natív build, tesztek, CI és disztribúció

DevOps környezetbe illő, moduláris C/C++ projektet mutasson be; a már ismert CI fogalmak helyett a natív toolchain problémák legyenek fókuszban.

Kutatási fókusz: Target-based CMake, dependency/link transitivity, test matrix, cross compilation, packaging és reproducible build.

- https://cmake.org/cmake/help/latest/
- https://gcc.gnu.org/onlinedocs/gcc/
- https://clang.llvm.org/docs/
- https://sourceware.org/binutils/docs/
- https://sourceware.org/gdb/current/onlinedocs/gdb/

ch39: C++ záróprojekt: a C rendszer átalakítása

A C capstone funkcionális szerződését megtartva modern C++20 implementációt készítsen, külön C++23 és async kísérleti ággal.

Kutatási fókusz: C++ port RAII, value types, span, errors, container/ranges, stable C API, build/tests és opcionális coroutine adapter.

- https://www.open-std.org/jtc1/sc22/wg21/
- https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
- https://cmake.org/cmake/help/latest/
- https://man7.org/linux/man-pages/
- https://clang.llvm.org/docs/AddressSanitizer.html
- https://clang.llvm.org/docs/ThreadSanitizer.html

ch40: Haladó irányok, új szabványok és referencia-függelék

Kiegészítő rész: adjon széles körű, elsődleges forrásokhoz vezető térképet a fő tananyag mélységétől elkülönítve.

Kutatási fókusz: C++23/C++26 státusz generáláskori ellenőrzése; library header inventory, platform és specializációs továbbtanulási térkép.

- https://www.open-std.org/jtc1/sc22/wg14/
- https://www.open-std.org/jtc1/sc22/wg21/
- https://gcc.gnu.org/onlinedocs/gcc/Standards.html
- https://gcc.gnu.org/projects/cxx-status.html
- https://clang.llvm.org/docs/
- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines